The Python Library Reference

Release 3.14.0rc3

Guido van Rossum and the Python development team

outubro 01, 2025

Python Software Foundation
Email: docs@python.org

Sumário

1 Introdução **3**
- 1.1 Observações sobre disponibilidade . . . 4
  - 1.1.1 Plataformas WebAssembly . . . 4
  - 1.1.2 Plataformas móveis . . . 4

2 Funções embutidas **7**

3 Constantes embutidas **37**
- 3.1 Constantes adicionadas pelo módulo `site` . . . 38

4 Tipos embutidos **39**
- 4.1 Teste do valor verdade . . . 39
- 4.2 Operações booleanas — `and`, `or`, `not` . . . 39
- 4.3 Comparações . . . 40
- 4.4 Tipos numéricos — `int`, `float`, `complex` . . . 40
  - 4.4.1 Operações de bits em tipos inteiros . . . 42
  - 4.4.2 Métodos adicionais em tipos inteiros . . . 43
  - 4.4.3 Métodos adicionais em ponto flutuante . . . 45
  - 4.4.4 Métodos adicionais em números complexos . . . 46
  - 4.4.5 Hashing de tipos numéricos . . . 46
- 4.5 Tipo booleano - `bool` . . . 48
- 4.6 Tipos iteradores . . . 48
  - 4.6.1 Tipos geradores . . . 49
- 4.7 Tipos sequências — `list`, `tuple`, `range` . . . 49
  - 4.7.1 Operações comuns de sequências . . . 49
  - 4.7.2 Tipos sequência imutáveis . . . 51
  - 4.7.3 Tipos sequências mutáveis . . . 51
  - 4.7.4 Listas . . . 52
  - 4.7.5 Tuplas . . . 53
  - 4.7.6 Intervalos . . . 54
- 4.8 Resumo dos métodos de tipo de sequência binária e de texto . . . 55
- 4.9 Tipo sequência de texto — `str` . . . 56
  - 4.9.1 Métodos de string . . . 57
  - 4.9.2 Strings literais formatadas (f-strings) . . . 67
  - 4.9.3 Formatação de string no estilo `printf` . . . 69
- 4.10 Tipos de sequência binária — `bytes`, `bytearray`, `memoryview` . . . 71
  - 4.10.1 Objetos bytes . . . 71
  - 4.10.2 Objetos Bytearray . . . 72
  - 4.10.3 Operações com Bytes e Bytearray . . . 74
  - 4.10.4 Estilo de Formatação de bytes `printf`-style . . . 85
  - 4.10.5 Memory Views . . . 88

4.11 Tipo conjuntos — set, frozenset . . . 95
4.12 Tipo mapeamento — dict . . . 98
4.12.1 Objetos de visão de dicionário . . . 101
4.13 Tipos de Gerenciador de Contexto . . . 103
4.14 Tipos de anotação de tipo — Apelido genérico, União . . . 104
4.14.1 Tipo Generic Alias . . . 104
4.14.2 Tipo União . . . 108
4.15 Outros tipos embutidos . . . 110
4.15.1 Módulos . . . 110
4.15.2 Classes e Instâncias de Classes . . . 110
4.15.3 Funções . . . 110
4.15.4 Métodos . . . 110
4.15.5 Objetos código . . . 111
4.15.6 Objetos tipo . . . 111
4.15.7 O objeto nulo . . . 111
4.15.8 O Objeto Ellipsis . . . 111
4.15.9 O Objeto NotImplemented . . . 112
4.15.10 Objetos Internos . . . 112
4.16 Atributos Especiais . . . 112
4.17 Limitação de comprimento de string na conversão para inteiro . . . 112
4.17.1 APIs afetadas . . . 113
4.17.2 Configurando o limiter . . . 114
4.17.3 Configuração recomendada . . . 114

5 Exceções embutidas 115
5.1 Contexto da exceção . . . 115
5.2 Herdando de exceções embutidas . . . 116
5.3 Classes base . . . 116
5.4 Exceções concretas . . . 117
5.4.1 Exceções de sistema operacional . . . 123
5.5 Avisos . . . 124
5.6 Grupos de exceções . . . 125
5.7 Hierarquia das exceções . . . 127

6 Serviços de Processamento de Texto 129
6.1 string — Operações comuns de strings . . . 129
6.1.1 Constantes de strings . . . 129
6.1.2 Formatação personalizada de strings . . . 130
6.1.3 Sintaxe das strings de formato . . . 131
6.1.4 Template strings ($-strings) . . . 139
6.1.5 Funções auxiliares . . . 141
6.2 string.templatelib — Suporte para literais de string template . . . 142
6.2.1 Strings template . . . 142
6.2.2 Tipos . . . 142
6.2.3 Funções auxiliares . . . 146
6.3 re — Operações com expressões regulares . . . 147
6.3.1 Sintaxe de expressão regular . . . 147
6.3.2 Conteúdo do módulo . . . 154
6.3.3 Objetos expressão regular . . . 161
6.3.4 Objetos correspondência . . . 162
6.3.5 Exemplos de expressão regular . . . 165
6.4 difflib — Helpers for computing deltas . . . 171
6.4.1 SequenceMatcher Objects . . . 175
6.4.2 SequenceMatcher Examples . . . 178
6.4.3 Differ Objects . . . 178
6.4.4 Differ Example . . . 179
6.4.5 A command-line interface to difflib . . . 180
6.4.6 ndiff example . . . 181

6.5 textwrap — Quebra automática e preenchimento de texto . . . 183
6.6 unicodedata — Banco de dados de Unicode . . . 187
6.7 stringprep — Preparação de string para internet . . . 189
6.8 readline — Interface para o GNU readline . . . 191
6.8.1 Arquivo init . . . 192
6.8.2 Buffer de linha . . . 192
6.8.3 Arquivo de histórico . . . 192
6.8.4 Lista de histórico . . . 193
6.8.5 Ganchos de inicialização . . . 193
6.8.6 Autocomplemento . . . 194
6.8.7 Exemplo . . . 194
6.9 rlcompleter — Função de autocomplemento para GNU readline . . . 196

7 Serviços de Dados Binários **197**
7.1 struct — Interpreta bytes como dados binários empacotados . . . 197
7.1.1 Funções e exceções . . . 198
7.1.2 Format Strings . . . 198
7.1.3 Applications . . . 202
7.1.4 Classes . . . 204
7.2 codecs — Codec registry and base classes . . . 204
7.2.1 Codec Base Classes . . . 208
7.2.2 Encodings and Unicode . . . 214
7.2.3 Standard Encodings . . . 215
7.2.4 Python Specific Encodings . . . 218
7.2.5 encodings — Encodings package . . . 220
7.2.6 encodings.idna — Internationalized Domain Names in Applications . . . 221
7.2.7 encodings.mbcs — Windows ANSI codepage . . . 222
7.2.8 encodings.utf_8_sig — UTF-8 codec with BOM signature . . . 222

8 Tipos de Dados **223**
8.1 datetime — Tipos básicos de data e hora . . . 223
8.1.1 Objetos Conscientes e Ingênuos . . . 224
8.1.2 Constantes . . . 224
8.1.3 Tipos disponíveis . . . 224
8.1.4 Objetos timedelta . . . 225
8.1.5 Objetos date . . . 229
8.1.6 Objetos datetime . . . 235
8.1.7 Objetos time . . . 246
8.1.8 Objetos tzinfo . . . 250
8.1.9 Objetos timezone . . . 257
8.1.10 Comportamento de strftime() e strptime() . . . 258
8.2 zoneinfo — Suporte a fuso horário da IANA . . . 262
8.2.1 Usando ZoneInfo . . . 262
8.2.2 Fontes de dados . . . 263
8.2.3 A classe ZoneInfo . . . 264
8.2.4 Funções . . . 266
8.2.5 Globals . . . 267
8.2.6 Exceptions and warnings . . . 267
8.3 calendar — Funções gerais relacionadas ao calendário . . . 267
8.3.1 Uso na linha de comando . . . 274
8.4 collections — Tipos de dados de contêineres . . . 275
8.4.1 Objetos ChainMap . . . 276
8.4.2 Objetos Counter . . . 278
8.4.3 Objetos deque . . . 282
8.4.4 Objetos defaultdict . . . 285
8.4.5 Função de fábrica para tuplas com campos nomeados namedtuple() . . . 287
8.4.6 Objetos OrderedDict . . . 290
8.4.7 Objetos UserDict . . . 293

8.4.8 Objetos UserList . . . 293
8.4.9 Objetos UserString . . . 293
8.5 collections.abc — Classes Base Abstratas para Contêineres . . . 294
8.5.1 Classes Base Abstratas de Coleções . . . 295
8.5.2 Classes Base Abstrata de Coleções – Descrições Detalhadas . . . 297
8.5.3 Exemplos e receitas . . . 299
8.6 heapq — Algoritmo de fila heap . . . 300
8.6.1 Exemplos básicos . . . 302
8.6.2 Notas de implementação da fila de prioridade . . . 303
8.6.3 Teoria . . . 304
8.7 bisect — Algoritmo de bisseção de vetor . . . 305
8.7.1 Observações sobre desempenho . . . 306
8.7.2 Buscando em listas ordenadas . . . 307
8.7.3 Exemplos . . . 307
8.8 array— Vetores eficientes de valores numéricos . . . 308
8.9 weakref — Referências fracas . . . 311
8.9.1 Objetos referência fraca . . . 316
8.9.2 Exemplo . . . 317
8.9.3 Objetos finalizadores . . . 317
8.9.4 Comparando finalizadores com métodos __del__() . . . 318
8.10 types — Criação de tipos dinâmicos e nomes para tipos embutidos . . . 319
8.10.1 Criação de tipos dinâmicos . . . 320
8.10.2 Tipos padrão do interpretador . . . 321
8.10.3 Classes e funções de utilidades adicionais . . . 325
8.10.4 Funções de utilidade de corrotina . . . 325
8.11 copy — Operações de cópia profunda e cópia rasa . . . 326
8.12 pprint — Impressão bonita de dados . . . 327
8.12.1 Funções . . . 327
8.12.2 Objetos PrettyPrinter . . . 329
8.12.3 Exemplo . . . 330
8.13 reprlib — Implementação alternativa à repr() . . . 333
8.13.1 Objetos Repr . . . 334
8.13.2 Estendendo objetos Repr . . . 336
8.14 enum — Suporte a enumerações . . . 336
8.14.1 Conteúdo do módulo . . . 337
8.14.2 Tipos de Dados . . . 338
8.14.3 Utilities and Decorators . . . 350
8.14.4 Notas . . . 352
8.15 graphlib — Funcionalidade para operar com estruturas do tipo grafo . . . 352
8.15.1 Exceções . . . 355

9 Módulos Matemáticos e Numéricos 357
9.1 numbers — Classes base abstratas numéricas . . . 357
9.1.1 A torre numérica . . . 357
9.1.2 Notas para implementadores de tipos . . . 358
9.2 math — Funções matemáticas . . . 360
9.2.1 Funções de teoria dos números . . . 362
9.2.2 Arimética de ponto flutuante . . . 363
9.2.3 Funções de manipulação de ponto flutuante . . . 364
9.2.4 Funções de potências, exponenciais e logarítmicas . . . 365
9.2.5 Funções de soma e produto . . . 366
9.2.6 Conversão angular . . . 367
9.2.7 Funções trigonométricas . . . 367
9.2.8 Funções hiperbólicas . . . 368
9.2.9 Funções especiais . . . 368
9.2.10 Constantes . . . 369
9.3 cmath — Funções matemáticas para números complexos . . . 370
9.3.1 Conversões de e para coordenadas polares . . . 371

9.3.2 Funções de potência e logarítmicas . . . 372
9.3.3 Funções trigonométricas . . . 372
9.3.4 Funções hiperbólicas . . . 372
9.3.5 Funções de classificação . . . 373
9.3.6 Constantes . . . 373
9.4 decimal — Aritmética de ponto fixo decimal e ponto flutuante . . . 374
9.4.1 Tutorial de início rápido . . . 375
9.4.2 Objetos de Decimal . . . 378
9.4.3 Objetos de contexto . . . 386
9.4.4 Constantes . . . 393
9.4.5 Modos de arredondamento . . . 393
9.4.6 Sinais . . . 394
9.4.7 Observações sobre ponto flutuante . . . 396
9.4.8 Trabalhando com threads . . . 397
9.4.9 Receitas . . . 398
9.4.10 FAQ sobre Decimal . . . 400
9.5 fractions — Números racionais . . . 404
9.6 random — Gera números pseudoaleatórios . . . 407
9.6.1 Funções de contabilidade . . . 408
9.6.2 Funções para bytes . . . 409
9.6.3 Funções para inteiros . . . 409
9.6.4 Funções para sequências . . . 409
9.6.5 Distribuições discretas . . . 410
9.6.6 Distribuições com valor real . . . 411
9.6.7 Gerador alternativo . . . 412
9.6.8 Notas sobre reprodutibilidade . . . 413
9.6.9 Exemplos . . . 413
9.6.10 Receitas . . . 415
9.6.11 Uso na linha de comando . . . 416
9.6.12 Exemplos na linha de comando . . . 417
9.7 statistics — Funções estatísticas . . . 418
9.7.1 Médias e medidas de valor central . . . 418
9.7.2 Medidas de espalhamento . . . 419
9.7.3 Estatísticas para relações entre duas entradas . . . 419
9.7.4 Detalhes das funções . . . 419
9.7.5 Exceções . . . 429
9.7.6 Objetos NormalDist . . . 429
9.7.7 Exemplos e receitas . . . 431

10 Módulos de Programação Funcional **435**
10.1 itertools — Funções que criam iteradores para laços eficientes . . . 435
10.1.1 Funções do intertools . . . 436
10.1.2 Receitas com itertools . . . 447
10.2 functools — Higher-order functions and operations on callable objects . . . 452
10.2.1 Objetos partial . . . 463
10.3 operator — Operadores padrões como funções . . . 463
10.3.1 Mapeando os operadores para suas respectivas funções . . . 468
10.3.2 Operadores in-place . . . 469

11 Acesso a arquivos e diretórios **471**
11.1 pathlib — Caminhos do sistema de arquivos orientados a objetos . . . 471
11.1.1 Uso básico . . . 472
11.1.2 Exceções . . . 473
11.1.3 Caminhos puros . . . 473
11.1.4 Caminhos concretos . . . 483
11.1.5 Linguagem de padrões . . . 496
11.1.6 Comparação com o módulo glob . . . 497
11.1.7 Comparação com os módulos os e os.path . . . 497

11.1.8 Protocolos . . . 498
11.2 os.path — Manipulações comuns de nomes de caminhos . . . 499
11.3 stat — Interpretando resultados de stat() . . . 506
11.4 filecmp — Comparações de arquivos e diretórios . . . 512
11.4.1 A classe dircmp . . . 513
11.5 tempfile — Generate temporary files and directories . . . 514
11.5.1 Exemplos . . . 518
11.5.2 Deprecated functions and variables . . . 519
11.6 glob — Expansão de padrão de nome de arquivo no estilo Unix . . . 520
11.6.1 Exemplos . . . 522
11.7 fnmatch — Correspondência de padrões de nome de arquivo Unix . . . 522
11.8 linecache — Acesso aleatório a linhas de texto . . . 524
11.9 shutil — Operações de arquivo de alto nível . . . 524
11.9.1 Operações de diretório e arquivos . . . 525
11.9.2 Operações de arquivamento . . . 531
11.9.3 Consultando o tamanho do terminal de saída . . . 535

12 Persistência de Dados **537**
12.1 pickle — Serialização de objetos Python . . . 537
12.1.1 Relacionamento com outros módulos Python . . . 538
12.1.2 Formato de fluxo de dados . . . 538
12.1.3 Interface do módulo . . . 539
12.1.4 O que pode ser serializado e desserializado com pickle? . . . 543
12.1.5 Serializando com pickle instâncias de classes . . . 544
12.1.6 Redução personalizada para tipos, funções e outros objetos . . . 550
12.1.7 Buffers fora da banda . . . 551
12.1.8 Restringindo globais . . . 552
12.1.9 Desempenho . . . 553
12.1.10 Exemplos . . . 553
12.1.11 Interface de linha de comando . . . 554
12.2 copyreg — Registra funções de suporte pickle . . . 554
12.2.1 Exemplo . . . 555
12.3 shelve — Python object persistence . . . 555
12.3.1 Restrições . . . 556
12.3.2 Exemplo . . . 557
12.4 marshal — Serialização interna de objetos Python . . . 558
12.5 dbm — Interfaces para “banco de dados” Unix . . . 560
12.5.1 dbm.sqlite3 — Backend de SQLite para dbm . . . 562
12.5.2 dbm.gnu — Gerenciador de banco de dados do GNU . . . 562
12.5.3 dbm.ndbm — New Database Manager . . . 564
12.5.4 dbm.dumb — Implementação portátil do DBM . . . 565
12.6 sqlite3 —- interface DB-API 2.0 interface para bancos de dados SQLite . . . 566
12.6.1 Tutorial . . . 566
12.6.2 Referência . . . 568
12.6.3 Guias de como fazer . . . 589
12.6.4 Explicação . . . 596

13 Compressão de Dados e Arquivamento **599**
13.1 O pacote compression . . . 599
13.2 compression.zstd — Compactação compatível com o formato Zstandard . . . 599
13.2.1 Exceções . . . 600
13.2.2 Lendo e escrevendo arquivos comprimidos . . . 600
13.2.3 Comprimindo e descomprimindo dados na memória . . . 601
13.2.4 Dicionários de Zstandard . . . 604
13.2.5 Advanced parameter control . . . 605
13.2.6 Diversos . . . 609
13.2.7 Exemplos . . . 610
13.3 zlib — Compactação compatível com **gzip** . . . 611

13.4 gzip — Suporte para arquivos **gzip** . . . 615
13.4.1 Exemplos de uso . . . 617
13.4.2 Interface de linha de comando . . . 618
13.5 bz2 — Suporte para compressão **bzip2** . . . 618
13.5.1 (Des)compressão de arquivos . . . 619
13.5.2 (Des)compressão incremental . . . 621
13.5.3 (De)compressão de uma só vez (one-shot) . . . 622
13.5.4 Exemplos de uso . . . 622
13.6 lzma — Compressão usando o algoritmo LZMA . . . 623
13.6.1 Lendo e escrevendo arquivos comprimidos . . . 623
13.6.2 Comprimindo e descomprimindo dados na memória . . . 625
13.6.3 Diversos . . . 627
13.6.4 Especificando cadeias de filtros personalizadas . . . 627
13.6.5 Exemplos . . . 628
13.7 zipfile — Trabalha com arquivos ZIP . . . 629
13.7.1 Objetos ZipFile . . . 631
13.7.2 Objetos Path . . . 635
13.7.3 Objetos PyZipFile . . . 637
13.7.4 Objetos ZipInfo . . . 638
13.7.5 Interface de Linha de Comando . . . 640
13.7.6 Armadilhas de descompressão . . . 640
13.8 tarfile — Ler e gravar arquivos do tipo tar . . . 641
13.8.1 TarFile Objects . . . 645
13.8.2 Objetos TarInfo . . . 649
13.8.3 Filtros de extração . . . 651
13.8.4 Interface de Linha de Comando . . . 654
13.8.5 Exemplos . . . 655
13.8.6 Formatos tar suportados . . . 657
13.8.7 Problemas de Unicode . . . 657

14 Formatos de Arquivo **659**
14.1 csv — Leitura e escrita de arquivos CSV . . . 659
14.1.1 Conteúdo do módulo . . . 659
14.1.2 Dialetos e parâmetros de formatação . . . 664
14.1.3 Objetos Reader . . . 665
14.1.4 Objetos Writer . . . 665
14.1.5 Exemplos . . . 666
14.2 configparser — Analisador sintático de arquivo de configuração . . . 667
14.2.1 Início rápido . . . 667
14.2.2 Tipos de dados suportados . . . 669
14.2.3 Valores reservas . . . 669
14.2.4 Estrutura dos arquivos INI . . . 670
14.2.5 Seções não nomeadas . . . 671
14.2.6 Interpolação de valores . . . 672
14.2.7 Acesso através do protocolo de mapeamento . . . 673
14.2.8 Personalizando o comportamento do analisador sintático . . . 674
14.2.9 Exemplos de APIs legadas . . . 678
14.2.10 Objetos ConfigParser . . . 680
14.2.11 Objetos RawConfigParser . . . 684
14.2.12 Exceções . . . 685
14.3 tomllib — Analisa arquivos TOML . . . 686
14.3.1 Exemplos . . . 687
14.3.2 Tabela de conversão . . . 688
14.4 netrc — Arquivo de processamento netrc . . . 688
14.4.1 Objetos netrc . . . 689
14.5 plistlib — Gera e analisa arquivos Apple .plist . . . 689
14.5.1 Exemplos . . . 691

15 Serviços Criptográficos **693**
15.1 hashlib — resumos de mensagens e hashes seguros . . . 693
15.1.1 Algoritmos de hash . . . 693
15.1.2 Uso . . . 694
15.1.3 Construtores . . . 694
15.1.4 Atributos . . . 695
15.1.5 Objetos hash . . . 695
15.1.6 Resumos de comprimento variável de SHAKE . . . 696
15.1.7 Hash de arquivo . . . 696
15.1.8 Derivação de chave . . . 697
15.1.9 BLAKE2 . . . 698
15.2 hmac — Código de autenticação de mensagem com chave hash . . . 705
15.3 secrets — Gera números aleatórios seguros para gerenciar segredos . . . 706
15.3.1 Números aleatórios . . . 707
15.3.2 Gerando tokens . . . 707
15.3.3 Outras funções . . . 708
15.3.4 Receitas e melhores práticas . . . 708

16 Serviços Genéricos do Sistema Operacional **711**
16.1 os — Diversas interfaces de sistema operacional . . . 711
16.1.1 Nomes de arquivos, argumentos de linha de comando e variáveis de ambiente . . . 712
16.1.2 Modo UTF-8 do Python . . . 712
16.1.3 Parâmetros de processo . . . 713
16.1.4 Criação de objetos arquivos . . . 721
16.1.5 Operações dos descritores de arquivos . . . 722
16.1.6 Arquivos e diretórios . . . 735
16.1.7 Gerenciamento de processo . . . 762
16.1.8 Interface do agendador . . . 776
16.1.9 Diversas informações de sistema . . . 777
16.1.10 Números aleatórios . . . 779
16.2 io — Ferramentas essenciais para trabalhar com fluxos . . . 780
16.2.1 Visão Geral . . . 781
16.2.2 Codificação do texto . . . 782
16.2.3 High-level Module Interface . . . 782
16.2.4 hierarquia de classe . . . 783
16.2.5 Static Typing . . . 794
16.2.6 Desempenho . . . 794
16.3 time — Acesso ao horário e conversões . . . 795
16.3.1 Funções . . . 796
16.3.2 Constantes de ID de Relógio . . . 805
16.3.3 Constantes de Fuso Horário . . . 807
16.4 logging — Recursos de registro de eventos para Python . . . 808
16.4.1 Objetos Logger . . . 809
16.4.2 Níveis de Logging . . . 814
16.4.3 Objetos Handler . . . 814
16.4.4 Formatter Objects . . . 816
16.4.5 Filter Objects . . . 817
16.4.6 LogRecord Objects . . . 818
16.4.7 Atributos LogRecord . . . 819
16.4.8 LoggerAdapter Objects . . . 821
16.4.9 Thread Safety . . . 821
16.4.10 Funções de Nível de Módulo . . . 821
16.4.11 Module-Level Attributes . . . 825
16.4.12 Integration with the warnings module . . . 826
16.5 logging.config — Configuração do módulo logging . . . 826
16.5.1 Funções de configuração . . . 827
16.5.2 Considerações de segurança . . . 829
16.5.3 Configuration dictionary schema . . . 829

16.5.4 Formato do arquivo de configuração . . . 836
16.6 logging.handlers — Tratadores de registro de eventos . . . 838
16.6.1 StreamHandler . . . 839
16.6.2 FileHandler . . . 839
16.6.3 NullHandler . . . 840
16.6.4 WatchedFileHandler . . . 840
16.6.5 BaseRotatingHandler . . . 841
16.6.6 RotatingFileHandler . . . 842
16.6.7 TimedRotatingFileHandler . . . 842
16.6.8 SocketHandler . . . 844
16.6.9 DatagramHandler . . . 845
16.6.10 SysLogHandler . . . 845
16.6.11 NTEventLogHandler . . . 847
16.6.12 SMTPHandler . . . 848
16.6.13 MemoryHandler . . . 848
16.6.14 HTTPHandler . . . 849
16.6.15 QueueHandler . . . 850
16.6.16 QueueListener . . . 851
16.7 platform — Acesso aos dados de identificação da plataforma subjacente . . . 852
16.7.1 Multiplataforma . . . 853
16.7.2 Plataforma Java . . . 855
16.7.3 Plataforma Windows . . . 855
16.7.4 Plataforma macOS . . . 855
16.7.5 Plataforma iOS . . . 855
16.7.6 Plataformas Unix . . . 856
16.7.7 Plataformas Linux . . . 856
16.7.8 Plataforma Android . . . 856
16.7.9 Uso na linha de comando . . . 857
16.8 errno — Símbolos padrão do sistema errno . . . 857
16.9 ctypes — Uma biblioteca de funções externas para Python . . . 865
16.9.1 Tutorial ctypes . . . 865
16.9.2 Referência ctypes . . . 884

17 Bibliotecas de interface de linha de comando **905**
17.1 argparse — Analisador sintático para opções de linha de comando, argumentos e subcomandos . 905
17.1.1 Objetos ArgumentParser . . . 906
17.1.2 O método add_argument() . . . 914
17.1.3 O método parse_args() . . . 925
17.1.4 Outros utilitários . . . 928
17.1.5 Exceções . . . 937
17.2 optparse — Analisador sintático para opções de linha de comando . . . 951
17.2.1 Escolhendo uma biblioteca de análise de linha de comando . . . 951
17.2.2 Introdução . . . 953
17.2.3 Contexto . . . 954
17.2.4 Tutorial . . . 956
17.2.5 Guia de referência . . . 963
17.2.6 Funções de retorno de opção . . . 973
17.2.7 Estendendo optparse . . . 977
17.2.8 Exceções . . . 979
17.3 getpass — Entrada de senha portátil . . . 980
17.4 fileinput — Itera sobre linhas de múltiplos fluxos de entrada . . . 981
17.5 curses — Gerenciador de terminal para visualizadores de células de caracteres. . . . 983
17.5.1 Funções . . . 984
17.5.2 Window Objects . . . 991
17.5.3 Constantes . . . 997
17.6 curses.textpad — Text input widget for curses programs . . . 1009
17.6.1 Textbox objects . . . 1009
17.7 curses.ascii — Utilitários para caracteres ASCII . . . 1010

17.8 curses.panel — Uma extensão de pilha de painéis para o curses . . . 1014
17.8.1 Funções . . . 1014
17.8.2 Objetos painel . . . 1014
17.9 cmd — Suporte para interpretadores de comando orientado a linhas . . . 1015
17.9.1 Objetos Cmd . . . 1016
17.9.2 Exemplo do Cmd . . . 1018

18 Execução Concorrente **1021**
18.1 threading — Paralelismo baseado em threads . . . 1021
18.1.1 Introdução . . . 1021
18.1.2 GIL and performance considerations . . . 1022
18.1.3 Referência . . . 1023
18.1.4 Using locks, conditions, and semaphores in the with statement . . . 1038
18.2 multiprocessing — Paralelismo baseado em processo . . . 1038
18.2.1 Introdução . . . 1038
18.2.2 Referência . . . 1046
18.2.3 Programming guidelines . . . 1075
18.2.4 Exemplos . . . 1079
18.3 multiprocessing.shared_memory — Memória compartilhada para acesso direto entre processos 1084
18.4 O pacote concurrent . . . 1091
18.5 concurrent.futures — Launching parallel tasks . . . 1091
18.5.1 Executor Objects . . . 1091
18.5.2 ThreadPoolExecutor . . . 1092
18.5.3 InterpreterPoolExecutor . . . 1094
18.5.4 `ProcessPoolExecutor` . . . 1095
18.5.5 Future Objects . . . 1097
18.5.6 Module Functions . . . 1098
18.5.7 Exception classes . . . 1099
18.6 concurrent.interpreters — Múltiplos interpretadores no mesmo processo . . . 1099
18.6.1 Detalhes-chave . . . 1100
18.6.2 Introdução . . . 1100
18.6.3 Referência . . . 1102
18.6.4 Uso básico . . . 1103
18.7 subprocess — Gerenciamento de subprocessos . . . 1104
18.7.1 Usando o módulo subprocess . . . 1104
18.7.2 Considerações de Segurança . . . 1113
18.7.3 Objetos Popen . . . 1113
18.7.4 Windows Popen Helpers . . . 1115
18.7.5 API de alto nível mais antiga . . . 1118
18.7.6 Replacing Older Functions with the subprocess Module . . . 1119
18.7.7 Legacy Shell Invocation Functions . . . 1121
18.7.8 Notas . . . 1122
18.8 sched — Agendador de eventos . . . 1123
18.8.1 Objetos Scheduler . . . 1124
18.9 queue — A synchronized queue class . . . 1125
18.9.1 Objetos Queue . . . 1126
18.9.2 Objetos SimpleQueue . . . 1128
18.10 contextvars — Variáveis de contexto . . . 1128
18.10.1 Variáveis de contexto . . . 1129
18.10.2 Gerenciamento de contexto manual . . . 1130
18.10.3 Suporte a asyncio . . . 1132
18.11 _thread— API de segmentação de baixo nível . . . 1133

19 Comunicação em Rede e Interprocesso **1137**
19.1 asyncio — E/S assíncrona . . . 1137
19.1.1 Runners . . . 1138
19.1.2 Corrotinas e Tarefas . . . 1141
19.1.3 Streams . . . 1161

19.1.4 Synchronization Primitives . . . 1169
19.1.5 Subprocessos . . . 1174
19.1.6 Filas . . . 1179
19.1.7 Exceções . . . 1182
19.1.8 Call Graph Introspection . . . 1183
19.1.9 Laço de Eventos . . . 1184
19.1.10 Futuros . . . 1210
19.1.11 Transports and Protocols . . . 1213
19.1.12 Políticas . . . 1226
19.1.13 Suporte a plataformas . . . 1228
19.1.14 Extensão . . . 1229
19.1.15 Índice da API de alto nível . . . 1231
19.1.16 Índice de APIs de baixo nível . . . 1233
19.1.17 Desenvolvendo com asyncio . . . 1238
19.2 `socket` — Interface de rede de baixo nível . . . 1241
19.2.1 Famílias de soquete . . . 1242
19.2.2 Conteúdo do módulo . . . 1245
19.2.3 Socket Objects . . . 1259
19.2.4 Notas sobre tempo limite de soquete . . . 1266
19.2.5 Exemplo . . . 1267
19.3 `ssl` — Invólucro de TLS/SSL para objetos socket . . . 1270
19.3.1 Funções, constantes e exceções. . . . 1271
19.3.2 SSL Sockets . . . 1283
19.3.3 SSL Contexts . . . 1287
19.3.4 Certificados . . . 1296
19.3.5 Exemplos . . . 1298
19.3.6 Notes on non-blocking sockets . . . 1301
19.3.7 Підтримка BIO пам'яті . . . 1301
19.3.8 SSL session . . . 1303
19.3.9 Considerações de segurança . . . 1304
19.3.10 TLS 1.3 . . . 1305
19.4 `select` — Waiting for I/O completion . . . 1306
19.4.1 `/dev/poll` Polling Objects . . . 1308
19.4.2 Edge and Level Trigger Polling (epoll) Objects . . . 1309
19.4.3 Polling Objects . . . 1310
19.4.4 Kqueue Objects . . . 1310
19.4.5 Kevent Objects . . . 1311
19.5 `selectors` — High-level I/O multiplexing . . . 1312
19.5.1 Introdução . . . 1313
19.5.2 Classes . . . 1313
19.5.3 Exemplos . . . 1315
19.6 `signal` — Define manipuladores para eventos assíncronos . . . 1316
19.6.1 Regras gerais . . . 1316
19.6.2 Conteúdo do módulo . . . 1317
19.6.3 Exemplos . . . 1324
19.6.4 Nota sobre SIGPIPE . . . 1324
19.6.5 Nota sobre manipuladores de sinais e exceções . . . 1325
19.7 `mmap` — Suporte a arquivos mapeados na memória . . . 1326
19.7.1 Constantes MADV_* . . . 1330
19.7.2 Constantes MAP_* . . . 1330

20 Manuseio de Dados na Internet **1333**
20.1 `email` — Um e-mail e um pacote MIME manipulável . . . 1333
20.1.1 `email.message`: Representing an email message . . . 1334
20.1.2 `email.parser`: Parsing email messages . . . 1342
20.1.3 `email.generator`: Geração de documentos MIME . . . 1345
20.1.4 `email.policy`: Policy Objects . . . 1348
20.1.5 `email.errors`: Classes de Exceção e Defeito. . . . 1355

20.1.6 email.headerregistry: Custom Header Objects . . . 1356
20.1.7 email.contentmanager: Gerenciando conteúdo MIME . . . 1362
20.1.8 email: Exemplos . . . 1364
20.1.9 email.message.Message: Representing an email message using the compat32 API . . . 1371
20.1.10 email.mimeemail.mime: Criando e-mail e objetos MIME do zero . . . 1379
20.1.11 email.header: Internationalized headers . . . 1382
20.1.12 email.charset: Representando conjuntos de caracteres . . . 1384
20.1.13 email.encoders: Codificadores . . . 1386
20.1.14 email.utils: Utilitários diversos . . . 1387
20.1.15 email.iterators: Iteradores . . . 1390
20.2 json — Codificador e decodificador JSON . . . 1391
20.2.1 Uso básico . . . 1393
20.2.2 Codificadores e decodificadores . . . 1396
20.2.3 Exceções . . . 1398
20.2.4 Conformidade e interoperabilidade entre padrões . . . 1398
20.2.5 Interface de linha de comando . . . 1400
20.3 mailbox — Manipulate mailboxes in various formats . . . 1401
20.3.1 Mailbox objects . . . 1401
20.3.2 Message objects . . . 1411
20.3.3 Exceções . . . 1419
20.3.4 Exemplos . . . 1419
20.4 mimetypes — Mapeia nomes de arquivos para tipos MIME . . . 1420
20.4.1 MimeTypes objects . . . 1422
20.4.2 Uso na linha de comando . . . 1424
20.4.3 Exemplos na linha de comando . . . 1424
20.5 base64 — Base16, Base32, Base64, Base85 Data Encodings . . . 1425
20.5.1 RFC 4648 Encodings . . . 1425
20.5.2 Base85 Encodings . . . 1427
20.5.3 Legacy Interface . . . 1428
20.5.4 Considerações de Segurança . . . 1428
20.6 binascii — Converte entre binário e ASCII . . . 1429
20.7 quopri — Codifica e decodifica dados MIME imprimidos entre aspas . . . 1431

21 Ferramentas de Processamento de Markup Estruturado 1433
21.1 html — Suporte HTML (HyperText Markup Language) . . . 1433
21.2 html.parser — Simple HTML and XHTML parser . . . 1434
21.2.1 Example HTML Parser Application . . . 1434
21.2.2 HTMLParser Methods . . . 1435
21.2.3 Exemplos . . . 1436
21.3 html.entities — Definições de entidades gerais de HTML . . . 1438
21.4 Módulos de Processamento de XML . . . 1439
21.4.1 Segurança no XML . . . 1439
21.5 xml.etree.ElementTree — A API XML ElementTree . . . 1440
21.5.1 Tutorial . . . 1440
21.5.2 Suporte a XPath . . . 1445
21.5.3 Referência . . . 1447
21.5.4 Suporte a XInclude . . . 1450
21.5.5 Referência . . . 1451
21.6 xml.dom — The Document Object Model API . . . 1458
21.6.1 Conteúdo do módulo . . . 1459
21.6.2 Objects in the DOM . . . 1460
21.6.3 Conformance . . . 1467
21.7 xml.dom.minidom — Minimal DOM implementation . . . 1468
21.7.1 Objetos DOM . . . 1470
21.7.2 DOM Example . . . 1471
21.7.3 minidom e o padrão DOM . . . 1472
21.8 xml.dom.pulldom — Suporte para construir árvores parciais de DOM . . . 1472
21.8.1 Objetos DOMEventStream . . . 1474

21.9 xml.sax — Support for SAX2 parsers . . . 1474
21.9.1 SAXException Objects . . . 1476
21.10 xml.sax.handler — Base classes for SAX handlers . . . 1476
21.10.1 ContentHandler Objects . . . 1478
21.10.2 DTDHandler Objects . . . 1480
21.10.3 EntityResolver Objects . . . 1480
21.10.4 ErrorHandler Objects . . . 1480
21.10.5 LexicalHandler Objects . . . 1481
21.11 xml.sax.saxutils — Utilitários de SAX . . . 1481
21.12 xml.sax.xmlreader — Interface for XML parsers . . . 1482
21.12.1 XMLReader Objects . . . 1483
21.12.2 IncrementalParser Objects . . . 1484
21.12.3 Locator Objects . . . 1484
21.12.4 InputSource Objects . . . 1485
21.12.5 The Attributes Interface . . . 1486
21.12.6 The AttributesNS Interface . . . 1486
21.13 xml.parsers.expat — Fast XML parsing using Expat . . . 1486
21.13.1 Objetos XMLParser . . . 1488
21.13.2 Exceções ExpatError . . . 1492
21.13.3 Exemplo . . . 1492
21.13.4 Content Model Descriptions . . . 1493
21.13.5 Constantes de erro de expatriados . . . 1493

22 Protocolos de Internet e Suporte 1497
22.1 webbrowser — Controlador de navegador web conveniente . . . 1497
22.1.1 Objetos controladores de navegador . . . 1500
22.2 wsgiref — Implementação de referência e utilitários WSGI . . . 1500
22.2.1 wsgiref.util – Utilidades do ambiente WSGI . . . 1500
22.2.2 wsgiref.headers – Ferramentas de cabeçalho de resposta WSGI . . . 1502
22.2.3 wsgiref.simple_server – um servidor HTTP WSGI simples . . . 1503
22.2.4 wsgiref.validate — Verificador de conformidade WSGI . . . 1504
22.2.5 wsgiref.handlers – classes base de servidor/gateway . . . 1505
22.2.6 wsgiref.types – WSGI types for static type checking . . . 1509
22.2.7 Exemplos . . . 1509
22.3 urllib — Módulos de manipulação de URL . . . 1510
22.4 urllib.request — Biblioteca extensível para abrir URLs . . . 1511
22.4.1 Objeto Request . . . 1516
22.4.2 OpenerDirector Objects . . . 1518
22.4.3 BaseHandler Objects . . . 1519
22.4.4 HTTPRedirectHandler Objects . . . 1520
22.4.5 HTTPCookieProcessor Objects . . . 1521
22.4.6 ProxyHandler Objects . . . 1521
22.4.7 HTTPPasswordMgr Objects . . . 1521
22.4.8 HTTPPasswordMgrWithPriorAuth Objects . . . 1521
22.4.9 AbstractBasicAuthHandler Objects . . . 1522
22.4.10 HTTPBasicAuthHandler Objects . . . 1522
22.4.11 ProxyBasicAuthHandler Objects . . . 1522
22.4.12 AbstractDigestAuthHandler Objects . . . 1522
22.4.13 HTTPDigestAuthHandler Objects . . . 1522
22.4.14 ProxyDigestAuthHandler Objects . . . 1522
22.4.15 HTTPHandler Objects . . . 1522
22.4.16 Objetos HTTPSHandler . . . 1522
22.4.17 FileHandler Objects . . . 1523
22.4.18 DataHandler Objects . . . 1523
22.4.19 FTPHandler Objects . . . 1523
22.4.20 CacheFTPHandler Objects . . . 1523
22.4.21 Objetos UnknownHandler . . . 1523
22.4.22 HTTPErrorProcessor Objects . . . 1523

22.4.23 Exemplos . . . 1523
22.4.24 Legacy interface . . . 1526
22.4.25 urllib.request Restrictions . . . 1527
22.5 urllib.response — Response classes used by urllib . . . 1527
22.6 urllib.parse — Analisa URLs para componentes . . . 1528
22.6.1 Análise de URL . . . 1528
22.6.2 Segurança ao analisar URLs . . . 1533
22.6.3 Analisando bytes codificados em ASCII . . . 1534
22.6.4 Structured Parse Results . . . 1534
22.6.5 URL Quoting . . . 1535
22.7 urllib.error — Classes de exceção levantadas por urllib.request . . . 1537
22.8 urllib.robotparser — Analisador sintático de robots.txt . . . 1538
22.9 http — HTTP modules . . . 1539
22.9.1 códigos de status HTTP . . . 1540
22.9.2 Categorias de HTTP status . . . 1541
22.9.3 Métodos HTTP . . . 1542
22.10 http.client — HTTP protocol client . . . 1543
22.10.1 Objetos de HTTPConnection . . . 1545
22.10.2 Objetos HTTPResponse . . . 1548
22.10.3 Exemplos . . . 1549
22.10.4 HTTPMessage Objects . . . 1550
22.11 ftplib — FTP protocol client . . . 1550
22.11.1 Referência . . . 1551
22.12 poplib — Cliente de protocolo POP3 . . . 1557
22.12.1 Objetos POP3 . . . 1558
22.12.2 Exemplo POP3 . . . 1560
22.13 imaplib — IMAP4 protocol client . . . 1560
22.13.1 Objetos IMAP4 . . . 1562
22.13.2 Exemplo IMAP4 . . . 1567
22.14 smtplib — SMTP protocol client . . . 1568
22.14.1 Objetos SMTP . . . 1570
22.14.2 Exemplo SMTP . . . 1574
22.15 uuid — Objetos UUID conforme **RFC 9562** . . . 1574
22.15.1 Uso na linha de comando . . . 1578
22.15.2 Exemplo . . . 1579
22.15.3 Exemplo de linha de comando . . . 1580
22.16 socketserver — Um framework para servidores de rede . . . 1580
22.16.1 Notas sobre criação do servidor . . . 1581
22.16.2 Objetos Server . . . 1583
22.16.3 Objetos manipulador de requisições . . . 1585
22.16.4 Exemplos . . . 1586
22.17 http.server — HTTP servers . . . 1589
22.17.1 Interface de linha de comando . . . 1595
22.17.2 Considerações de segurança . . . 1596
22.18 http.cookies — HTTP state management . . . 1597
22.18.1 Objetos Cookie . . . 1597
22.18.2 Objetos Morsel . . . 1598
22.18.3 Exemplo . . . 1599
22.19 http.cookiejar — Cookie handling for HTTP clients . . . 1600
22.19.1 CookieJar and FileCookieJar Objects . . . 1602
22.19.2 FileCookieJar subclasses and co-operation with web browsers . . . 1604
22.19.3 Objeto CookiePolicy . . . 1604
22.19.4 DefaultCookiePolicy Objects . . . 1605
22.19.5 Objetos Cookie . . . 1607
22.19.6 Exemplos . . . 1608
22.20 xmlrpc — Módulos de servidor e cliente XMLRPC . . . 1609
22.21 xmlrpc.client — Cliente XML-RPC . . . 1609
22.21.1 Objetos ServerProxy . . . 1611

22.21.2 Objetos DateTime . . . 1612
22.21.3 Objetos Binários . . . 1612
22.21.4 Objetos Fault . . . 1613
22.21.5 Objetos ProtocolError . . . 1614
22.21.6 Objetos MultiCall . . . 1614
22.21.7 Convenience Functions . . . 1615
22.21.8 Exemplo de uso do cliente . . . 1616
22.21.9 Example of Client and Server Usage . . . 1616
22.22 xmlrpc.server — Servidores XML-RPC básicos . . . 1617
22.22.1 Objetos de SimpleXMLRPCServer . . . 1617
22.22.2 CGIXMLRPCRequestHandler . . . 1621
22.22.3 Documentando servidor XMLRPC . . . 1622
22.22.4 Objetos de DocXMLRPCServer . . . 1622
22.22.5 DocCGIXMLRPCRequestHandler . . . 1622
22.23 ipaddress — Biblioteca de manipulação de IPv4/IPv6 . . . 1623
22.23.1 Funções de fábrica de conveniência . . . 1623
22.23.2 Endereços IP . . . 1623
22.23.3 Definições de rede IP . . . 1628
22.23.4 Objetos de interface . . . 1634
22.23.5 Outras funções de nível de módulo . . . 1636
22.23.6 Exceções personalizadas . . . 1636

23 Serviços Multimídia **1639**
23.1 wave — Read and write WAV files . . . 1639
23.1.1 Objetos Wave_read . . . 1640
23.1.2 Objetos Wave_write . . . 1641
23.2 colorsys — Conversões entre sistemas de cores . . . 1642

24 Internacionalização **1643**
24.1 gettext — Serviços de internacionalização multilíngues . . . 1643
24.1.1 API do GNU **gettext** . . . 1643
24.1.2 API baseada em classe . . . 1644
24.1.3 Internacionalizando seus programas e módulos . . . 1648
24.1.4 Reconhecimentos . . . 1651
24.2 locale — Serviços de internacionalização . . . 1651
24.2.1 Histórico, detalhes, dicas, dicas e advertências . . . 1658
24.2.2 Nomes de localidades . . . 1658
24.2.3 Para escritores de extensão e programas que incorporam Python . . . 1659
24.2.4 Acesso a catálogos de mensagens . . . 1659

25 Interfaces gráficas de usuário com Tk **1661**
25.1 tkinter — Interface Python para Tcl/Tk . . . 1661
25.1.1 Arquitetura . . . 1662
25.1.2 Módulos Tkinter . . . 1663
25.1.3 Preservador de vida Tkinter . . . 1665
25.1.4 Modelo de threading . . . 1667
25.1.5 Referência Útil . . . 1668
25.1.6 Tratadores de arquivos . . . 1674
25.2 tkinter.colorchooser — Diálogo de escolha de cor . . . 1674
25.3 tkinter.font — Invólucro de fontes Tkinter . . . 1675
25.4 Diálogos Tkinter . . . 1676
25.4.1 tkinter.simpledialog — Diálogos de entrada padrão do Tkinter . . . 1676
25.4.2 tkinter.filedialog — Caixas de diálogo de seleção de arquivo . . . 1676
25.4.3 tkinter.commondialog — Modelos de janela de diálogo . . . 1679
25.5 tkinter.messagebox — Prompts de mensagem do Tkinter . . . 1679
25.6 tkinter.scrolledtext — Widget de texto de rolado . . . 1681
25.7 tkinter.dnd — Suporte para arrastar e soltar . . . 1682
25.8 tkinter.ttk — Widgets temáticos do Tk . . . 1682
25.8.1 Usando Ttk . . . 1683

25.8.2 Widgets Ttk . . . 1683
25.8.3 Widget . . . 1684
25.8.4 Combobox . . . 1686
25.8.5 Spinbox . . . 1687
25.8.6 Notebook . . . 1688
25.8.7 Progressbar . . . 1690
25.8.8 Separator . . . 1690
25.8.9 Sizegrip . . . 1691
25.8.10 Treeview . . . 1691
25.8.11 Ttk Styling . . . 1696
25.9 IDLE — editor e console Python . . . 1701
25.9.1 Menus . . . 1701
25.9.2 Editing and Navigation . . . 1705
25.9.3 Startup and Code Execution . . . 1708
25.9.4 Help and Preferences . . . 1711
25.9.5 idlelib — implementation of IDLE application . . . 1712
25.10 `turtle` — Gráficos tartaruga . . . 1712
25.10.1 Introdução . . . 1712
25.10.2 Começando . . . 1712
25.10.3 Tutorial . . . 1713
25.10.4 Como fazer… . . . 1715
25.10.5 Referência Gráficos de Tartaruga . . . 1717
25.10.6 Métodos de RawTurtle/Turtle e funções correspondentes . . . 1720
25.10.7 métodos do TurtleScreen/Screen e as funções correspondentes . . . 1737
25.10.8 Classes Públicas . . . 1745
25.10.9 Explicação . . . 1746
25.10.10 Ajuda e Configuração . . . 1747
25.10.11 `turtledemo` — Scripts de Demonstração . . . 1749
25.10.12 Modificações desde a versão do Python 2.6 . . . 1750
25.10.13 Modificações desde a versão do Python 3.0 . . . 1751

26 Ferramentas de Desenvolvimento **1753**
26.1 `typing` — Suporte para dicas de tipo . . . 1753
26.1.1 Especificação para o sistema de tipos do Python . . . 1754
26.1.2 Apelidos de tipo . . . 1754
26.1.3 NewType . . . 1755
26.1.4 Anotações de objetos chamáveis . . . 1756
26.1.5 Genéricos . . . 1757
26.1.6 Anotando tuplas . . . 1758
26.1.7 O tipo de objetos de classe . . . 1759
26.1.8 Anotando geradores e corrotinas . . . 1759
26.1.9 Tipos genéricos definidos pelo usuário . . . 1761
26.1.10 O tipo `Any` . . . 1763
26.1.11 Subtipagem nominal vs estrutural . . . 1765
26.1.12 Conteúdo do módulo . . . 1765
26.1.13 Cronograma de descontinuação dos principais recursos . . . 1809
26.2 `pydoc` — Gerador de documentação e sistema de ajuda online . . . 1809
26.3 Modo de Desenvolvimento do Python . . . 1811
26.3.1 Efeitos do Modo de Desenvolvimento do Python . . . 1811
26.3.2 Exemplo de ResourceWarning . . . 1812
26.3.3 Exemplo de erro de descritor de arquivo inválido . . . 1813
26.4 `doctest` — Teste exemplos interativos de Python . . . 1814
26.4.1 Uso simples: verificando exemplos em Docstrings . . . 1816
26.4.2 Utilização comum: Verificando exemplos em um arquivo texto . . . 1816
26.4.3 Uso na linha de comando . . . 1817
26.4.4 Como ele funciona . . . 1818
26.4.5 Basic API . . . 1825
26.4.6 API do Unittest . . . 1826

26.4.7 Advanced API . . . 1829
26.4.8 Depuração . . . 1833
26.4.9 Soapbox . . . 1836
26.5 unittest — Unit testing framework . . . 1837
26.5.1 Exemplo Básico . . . 1838
26.5.2 Interface de Linha de Comando . . . 1839
26.5.3 Test Discovery . . . 1841
26.5.4 Organizando código teste . . . 1842
26.5.5 Reutilizando códigos de teste antigos . . . 1844
26.5.6 Ignorando testes e falhas esperadas . . . 1844
26.5.7 Distinguindo iterações de teste utilizando subtestes . . . 1846
26.5.8 Classes e funções . . . 1847
26.5.9 Classes e Módulos de Definição de Contexto . . . 1866
26.5.10 Tratamento de sinal . . . 1868
26.6 unittest.mock — mock object library . . . 1869
26.6.1 Guia Rápido . . . 1869
26.6.2 A classe Mock . . . 1871
26.6.3 Os criadores de patches . . . 1888
26.6.4 MagicMock and magic method support . . . 1897
26.6.5 Ajudantes . . . 1901
26.6.6 Order of precedence of side_effect, return_value and *wraps* . . . 1908
26.7 unittest.mock — primeiros passos . . . 1910
26.7.1 Usando Mock . . . 1910
26.7.2 Patch Decorators . . . 1916
26.7.3 Further Examples . . . 1918
26.8 test — Pacote de Testes de Regressão do Python . . . 1930
26.8.1 Escrever testes unitários para o pacote test . . . 1931
26.8.2 Executando testes usando a interface de linha de comando . . . 1932
26.9 test.support — Utilitários para o conjunto de teste do Python . . . 1933
26.10 test.support.socket_helper — Utilities for socket tests . . . 1942
26.11 test.support.script_helper — Utilities for the Python execution tests . . . 1943
26.12 test.support.bytecode_helper — Ferramentas de suporte para testar a geração correta de bytecode . . . 1944
26.13 test.support.threading_helper — Utilities for threading tests . . . 1945
26.14 test.support.os_helper — Utilities for os tests . . . 1946
26.15 test.support.import_helper — Utilities for import tests . . . 1947
26.16 test.support.warnings_helper — Utilities for warnings tests . . . 1949

27 Depuração e perfilamento 1951
27.1 Tabela de eventos de auditoria . . . 1951
27.2 bdb — Framework do depurador . . . 1955
27.3 faulthandler — Dump the Python traceback . . . 1961
27.3.1 Dumping the traceback . . . 1961
27.3.2 Dumping the C stack . . . 1962
27.3.3 Fault handler state . . . 1962
27.3.4 Dumping the tracebacks after a timeout . . . 1963
27.3.5 Dumping the traceback on a user signal . . . 1963
27.3.6 Issue with file descriptors . . . 1963
27.3.7 Exemplo . . . 1963
27.4 pdb — O depurador do Python . . . 1964
27.4.1 Comandos de depuração . . . 1968
27.5 Os Profilers do Python . . . 1975
27.5.1 Introdução aos profilers . . . 1975
27.5.2 Instant User's Manual . . . 1976
27.5.3 profile and cProfile Module Reference . . . 1978
27.5.4 A classe Stats . . . 1979
27.5.5 What Is Deterministic Profiling? . . . 1982
27.5.6 Limitações . . . 1982

27.5.7 Calibration . . . 1982
27.5.8 Using a custom timer . . . 1983
27.6 timeit — Mede o tempo de execução de pequenos trechos de código . . . 1983
27.6.1 Exemplos básicos . . . 1984
27.6.2 Interface em Python . . . 1984
27.6.3 Interface de Linha de Comando . . . 1986
27.6.4 Exemplos . . . 1987
27.7 trace — Rastreia ou acompanha a execução de instruções Python . . . 1988
27.7.1 Uso na linha de comando . . . 1989
27.7.2 Interface programática . . . 1990
27.8 tracemalloc — Trace memory allocations . . . 1991
27.8.1 Exemplos . . . 1992
27.8.2 API . . . 1996

28 Empacotamento e Distribuição de Software 2003
28.1 ensurepip — Inicialização do instalador do pip . . . 2003
28.1.1 Interface de linha de comando . . . 2004
28.1.2 API do módulo . . . 2004
28.2 venv — Criação de ambientes virtuais . . . 2005
28.2.1 Criando ambientes virtuais . . . 2006
28.2.2 Como funcionam os venvs . . . 2007
28.2.3 API . . . 2008
28.2.4 Um exemplo de extensão de EnvBuilder . . . 2011
28.3 zipapp — Gerencia arquivos zip executáveis do Python . . . 2015
28.3.1 Exemplo básico . . . 2015
28.3.2 Interface de linha de comando . . . 2015
28.3.3 API do Python . . . 2016
28.3.4 Exemplos . . . 2017
28.3.5 Especificando o interpretador . . . 2018
28.3.6 Criando aplicações autônomas com zipapp . . . 2018
28.3.7 O formato de arquivo da aplicação zip Python . . . 2019

29 Serviços de Tempo de Execução Python 2021
29.1 sys — Parâmetros e funções específicas do sistema . . . 2021
29.2 sys.monitoring — Monitoramento de eventos de execução . . . 2049
29.2.1 Identificadores de ferramenta . . . 2050
29.2.2 Eventos . . . 2050
29.2.3 Ativação e desativação de eventos . . . 2052
29.2.4 Registro de funções de retorno de chamada . . . 2053
29.3 sysconfig — Fornece acesso às informações de configuração do Python . . . 2054
29.3.1 Variáveis de configuração . . . 2055
29.3.2 Caminhos de instalação . . . 2055
29.3.3 Esquema de usuário . . . 2056
29.3.4 Esquema de home . . . 2057
29.3.5 Esquema de prefixo . . . 2057
29.3.6 Funções de caminho de instalação . . . 2058
29.3.7 Outras funções . . . 2059
29.3.8 Uso na linha de comando . . . 2060
29.4 builtins — Objetos embutidos . . . 2061
29.5 __main__ — Ambiente de código principal . . . 2061
29.5.1 __name__ == '__main__' . . . 2062
29.5.2 __main__.py em pacotes Python . . . 2064
29.5.3 import __main__ . . . 2065
29.6 warnings — Controle de avisos . . . 2066
29.6.1 Categorias de avisos . . . 2067
29.6.2 O filtro de avisos . . . 2068
29.6.3 Temporarily Suppressing Warnings . . . 2070
29.6.4 Testing Warnings . . . 2071

29.6.5 Updating Code For New Versions of Dependencies . . . 2072
29.6.6 Funções disponíveis . . . 2072
29.6.7 Gerenciadores de contexto disponíveis . . . 2074
29.6.8 Concurrent safety of Context Managers . . . 2074
29.7 dataclasses — Data Classes . . . 2075
29.7.1 Conteúdo do módulo . . . 2076
29.7.2 Processamento pós-inicialização . . . 2082
29.7.3 Variáveis de classe . . . 2083
29.7.4 Variáveis de inicialização apenas . . . 2083
29.7.5 Frozen instances . . . 2083
29.7.6 Herança . . . 2083
29.7.7 Re-ordering of keyword-only parameters in __init__() . . . 2084
29.7.8 Funções padrão de fábrica . . . 2084
29.7.9 Valores padrão mutáveis . . . 2084
29.7.10 Descriptor-typed fields . . . 2085
29.8 contextlib — Utilitários para contextos da instrução with . . . 2086
29.8.1 Utilitários . . . 2086
29.8.2 Exemplos e receitas . . . 2095
29.8.3 Single use, reusable and reentrant context managers . . . 2099
29.9 abc — Classes base abstratas . . . 2101
29.10 atexit — Manipuladores de saída . . . 2106
29.10.1 Exemplo do atexit . . . 2107
29.11 traceback — Imprime ou recupera um traceback da pilha . . . 2107
29.11.1 Funções de nível de módulo . . . 2108
29.11.2 TracebackException Objects . . . 2110
29.11.3 StackSummary Objects . . . 2112
29.11.4 FrameSummary Objects . . . 2113
29.11.5 Examples of Using the Module-Level Functions . . . 2113
29.11.6 Examples of Using TracebackException . . . 2116
29.12 __future__ — Definições de instruções future . . . 2117
29.12.1 Conteúdo do módulo . . . 2118
29.13 gc — Interface para o coletor de lixo . . . 2119
29.14 inspect — Inspecione objetos vivos . . . 2123
29.14.1 Tipos e membros . . . 2124
29.14.2 Retrieving source code . . . 2129
29.14.3 Introspecting callables with the Signature object . . . 2129
29.14.4 Classes e funções . . . 2134
29.14.5 A pilha to interpretador . . . 2136
29.14.6 Fetching attributes statically . . . 2139
29.14.7 Current State of Generators, Coroutines, and Asynchronous Generators . . . 2139
29.14.8 Code Objects Bit Flags . . . 2141
29.14.9 Buffer flags . . . 2142
29.14.10Interface de linha de comando . . . 2142
29.15 annotationlib — Funcionalidade para introspecção de anotações . . . 2143
29.15.1 Semânticas de anotação . . . 2143
29.15.2 Classes . . . 2144
29.15.3 Funções . . . 2145
29.15.4 Receitas . . . 2148
29.15.5 Limitações do formato STRING . . . 2149
29.15.6 Limitações do formato FORWARDREF . . . 2151
29.15.7 Security implications of introspecting annotations . . . 2151
29.16 site — Gancho de configuração específico do site . . . 2151
29.16.1 sitecustomize . . . 2153
29.16.2 usercustomize . . . 2153
29.16.3 Configuração Readline . . . 2153
29.16.4 Conteúdo do módulo . . . 2153
29.16.5 Interface de linha de comando . . . 2154

30 Interpretadores Python Personalizados **2155**
30.1 code — Classes bases do interpretador . . . 2155
30.1.1 Objetos de interpretador interativo . . . 2156
30.1.2 Objetos de console Interativo . . . 2157
30.2 codeop — Compila código Python . . . 2157

31 Importando módulos **2159**
31.1 zipimport - Importa módulos de arquivos Zip . . . 2159
31.1.1 Objetos zipimporter . . . 2160
31.1.2 Exemplos . . . 2161
31.2 pkgutil — Utilitário de extensão de pacote . . . 2161
31.3 modulefinder — Procura módulos usados por um script . . . 2164
31.3.1 Exemplo de uso de ModuleFinder . . . 2164
31.4 runpy — Localizando e executando módulos Python . . . 2165
31.5 importlib — A implementação de import . . . 2168
31.5.1 Introdução . . . 2168
31.5.2 Funções . . . 2169
31.5.3 importlib.abc – classes base abstratas relacionadas a importação . . . 2170
31.5.4 importlib.machinery – Importadores e ganchos de caminho . . . 2177
31.5.5 importlib.util – Utility code for importers . . . 2183
31.5.6 Exemplos . . . 2185
31.6 importlib.resources – Leitura, abertura e acesso a recursos de pacotes . . . 2188
31.6.1 API funcional . . . 2189
31.7 importlib.resources.abc – Classes base abstratas para recursos . . . 2191
31.8 importlib.metadata – Acessando metadados do pacote . . . 2193
31.8.1 Visão Geral . . . 2194
31.8.2 API funcional . . . 2194
31.8.3 Distribuições . . . 2198
31.8.4 Descoberta de distribuição . . . 2199
31.8.5 Implementando provedores personalizados . . . 2199
31.9 A inicialização do caminho de pesquisa de módulos sys.path . . . 2201
31.9.1 Ambientes virtuais . . . 2202
31.9.2 Arquivos _pth . . . 2202
31.9.3 Python embarcado . . . 2202

32 Serviços da Linguagem Python **2203**
32.1 ast — Árvores de sintaxe abstrata . . . 2203
32.1.1 Gramática abstrata . . . 2203
32.1.2 Classes de nós . . . 2206
32.1.3 Auxiliares de ast . . . 2237
32.1.4 Sinalizadores do compilador . . . 2241
32.1.5 Uso na linha de comando . . . 2242
32.2 symtable — Acesso a tabela de símbolos do compilador . . . 2243
32.2.1 Generating Symbol Tables . . . 2243
32.2.2 Examining Symbol Tables . . . 2243
32.2.3 Uso na linha de comando . . . 2247
32.3 token — Constantes usadas com árvores de análises do Python . . . 2247
32.4 keyword — Testando palavras reservadas do Python . . . 2252
32.5 tokenize — Tokenizador para código-fonte Python . . . 2253
32.5.1 Tokenizando entradas . . . 2253
32.5.2 Uso na linha de comando . . . 2254
32.5.3 Exemplos . . . 2255
32.6 tabnanny — Detecção de indentação ambígua . . . 2257
32.7 pyclbr — Suporte a navegador de módulos do Python . . . 2257
32.7.1 Objetos Function . . . 2258
32.7.2 Objetos Class . . . 2259
32.8 py_compile — Compila arquivos fonte do Python . . . 2259
32.8.1 Interface de Linha de Comando . . . 2261

32.9 compileall — Compilar bibliotecas do Python para bytecode . . . 2261
32.9.1 Uso na linha de comando . . . 2261
32.9.2 Funções públicas . . . 2263
32.10 dis — Disassembler de bytecode do Python . . . 2265
32.10.1 Interface de linha de comando . . . 2266
32.10.2 Análise de bytecode . . . 2267
32.10.3 Funções de análise . . . 2268
32.10.4 Instruções em bytecode do Python . . . 2270
32.10.5 Opcode collections . . . 2288
32.11 pickletools — Ferramentas para desenvolvedores pickle . . . 2288
32.11.1 Uso na linha de comando . . . 2289
32.11.2 Interface programática . . . 2289

33 Serviços Específicos do MS Windows 2291
33.1 msvcrt — Rotinas úteis do runtime MS VC++ . . . 2291
33.1.1 Operações com arquivos . . . 2291
33.1.2 E/S de console . . . 2292
33.1.3 Outras funções . . . 2292
33.2 winreg — Acesso aos registros do Windows . . . 2294
33.2.1 Funções . . . 2294
33.2.2 Constantes . . . 2300
33.2.3 Objetos identificador de registro . . . 2302
33.3 winsound — Interface de reprodução de som para o Windows . . . 2303

34 Serviços específicos do Unix 2307
34.1 shlex — Simple lexical analysis . . . 2307
34.1.1 shlex Objects . . . 2309
34.1.2 Regras de análise . . . 2311
34.1.3 Improved Compatibility with Shells . . . 2311
34.2 posix — As chamadas de sistema mais comuns do POSIX . . . 2312
34.2.1 Suporte a arquivos grandes . . . 2313
34.2.2 Conteúdo notável do módulo . . . 2313
34.3 pwd — A senha do banco de dados . . . 2313
34.4 grp — O banco de dados de grupos . . . 2314
34.5 termios — Controle de tty no estilo POSIX . . . 2315
34.5.1 Exemplo . . . 2316
34.6 tty — Funções de controle de terminal . . . 2317
34.7 pty — Utilitários de pseudoterminal . . . 2317
34.7.1 Exemplo . . . 2318
34.8 fcntl — as chamadas de sistema fcntl e ioctl . . . 2319
34.9 resource — Informação de uso de recursos . . . 2322
34.9.1 Limites de recursos . . . 2322
34.9.2 Uso de recursos . . . 2325
34.10 syslog — Rotinas da biblioteca syslog do Unix . . . 2326
34.10.1 Exemplos . . . 2329

35 Interface de linha de comando (CLI) de módulos 2331

36 Módulos substituídos 2333
36.1 getopt — Analisador sintático no estilo C para opções de linha de comando . . . 2333

37 Módulos removidos 2337
37.1 aifc — Lê e escreve arquivos AIFF e AIFC . . . 2337
37.2 asynchat — Manipulador de comando/resposta de socket assíncrono . . . 2337
37.3 asyncore — Manipulador de socket assíncrono . . . 2337
37.4 audioop — Manipulando dados de áudio original . . . 2338
37.5 cgi — Suporte a Common Gateway Interface . . . 2338
37.6 cgitb — Gerenciador de traceback para scripts de CGI . . . 2338
37.7 chunk — Lê dados em blocos IFF . . . 2338

37.8 crypt — Função para verificar senhas Unix . . . 2338
37.9 distutils — Criação e instalação de módulos do Python . . . 2338
37.10 imghdr — Determina o tipo de uma imagem . . . 2339
37.11 imp — Acessa os componentes internos de importação . . . 2339
37.12 mailcap — Manipulação de arquivo Mailcap . . . 2339
37.13 msilib — Lê e escreve arquivos do Microsoft Installer . . . 2339
37.14 nis — Interface para NIS da Sun (Yellow Pages) . . . 2339
37.15 nntplib — Cliente de protocolo NNTP . . . 2339
37.16 ossaudiodev — Acesso a dispositivos de áudio compatíveis com OSS . . . 2340
37.17 pipes — Interface para pipelines do shell . . . 2340
37.18 smtpd — Serviços SMTP . . . 2340
37.19 sndhdr — Determina o tipo de arquivos de som . . . 2340
37.20 spwd — O banco de dados de senhas shadow . . . 2340
37.21 sunau — Lê e escreve arquivos AU da Sun . . . 2341
37.22 telnetlib — cliente Telnet . . . 2341
37.23 uu — Codifica e decodifica arquivos uuencode . . . 2341
37.24 xdrlib — Codifica e decodifica dados XDR . . . 2341

38 Considerações de segurança **2343**

A Glossário **2345**

B Sobre esta documentação **2365**
B.1 Contribuidores da documentação do Python . . . 2365

C História e Licença **2367**
C.1 História do software . . . 2367
C.2 Termos e condições para acessar ou usar Python . . . 2368
C.2.1 PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2 . . . 2368
C.2.2 ACORDO DE LICENCIAMENTO DA BEOPEN.COM PARA PYTHON 2.0 . . . 2369
C.2.3 CONTRATO DE LICENÇA DA CNRI PARA O PYTHON 1.6.1 . . . 2370
C.2.4 ACORDO DE LICENÇA DA CWI PARA PYTHON 0.9.0 A 1.2 . . . 2371
C.2.5 ZERO-CLAUSE BSD LICENSE FOR CODE IN THE PYTHON DOCUMENTATION . . . 2371
C.3 Licenças e Reconhecimentos para Software Incorporado . . . 2371
C.3.1 Mersenne Twister . . . 2371
C.3.2 Soquetes . . . 2372
C.3.3 Serviços de soquete assíncrono . . . 2373
C.3.4 Gerenciamento de cookies . . . 2373
C.3.5 Rastreamento de execução . . . 2374
C.3.6 Funções UUencode e UUdecode . . . 2374
C.3.7 Chamadas de procedimento remoto XML . . . 2375
C.3.8 test_epoll . . . 2375
C.3.9 kqueue de seleção . . . 2376
C.3.10 SipHash24 . . . 2376
C.3.11 strtod e dtoa . . . 2377
C.3.12 OpenSSL . . . 2377
C.3.13 expat . . . 2381
C.3.14 libffi . . . 2381
C.3.15 zlib . . . 2382
C.3.16 cfuhash . . . 2382
C.3.17 libmpdec . . . 2383
C.3.18 Conjunto de testes C14N do W3C . . . 2383
C.3.19 mimalloc . . . 2384
C.3.20 asyncio . . . 2384
C.3.21 Global Unbounded Sequences (GUS) . . . 2385
C.3.22 Ligações Zstandard . . . 2385

D Direitos autorais **2387**

Referências Bibliográficas **2389**

Índice de Módulos Python **2391**

Índice **2395**

Enquanto reference-index descreve a sintaxe e a semântica exatas da linguagem Python, este manual de referência de bibliotecas descreve a biblioteca padrão que é distribuída com o Python. Ele também descreve alguns dos componentes opcionais que são comumente incluídos nas distribuições do Python.

A biblioteca padrão do Python é muito extensa, oferecendo uma ampla gama de recursos, conforme indicado pelo longo índice listado abaixo. A biblioteca contém módulos embutidos (escritos em C) que fornecem acesso à funcionalidade do sistema, como E/S de arquivos que de outra forma seriam inacessíveis para programadores Python, bem como módulos escritos em Python que fornecem soluções padronizadas para muitos problemas que ocorrem em programação cotidiana. Alguns desses módulos são explicitamente projetados para incentivar e aprimorar a portabilidade de programas em Python, abstraindo os detalhes da plataforma em APIs neutras em plataforma.

Os instaladores do Python para a plataforma Windows geralmente incluem toda a biblioteca padrão e muitas vezes também incluem muitos componentes adicionais. Para sistemas operacionais semelhantes a Unix, o Python é normalmente fornecido como uma coleção de pacotes, portanto, pode ser necessário usar as ferramentas de empacotamento fornecidas com o sistema operacional para obter alguns ou todos os componentes opcionais.

Além da biblioteca padrão, há uma coleção ativa de centenas de milhares de componentes (de programas e módulos individuais a pacotes e frameworks de desenvolvimento de aplicativos inteiros), disponíveis no Python Package Index.

CAPÍTULO 1

Introdução

A “biblioteca Python” contém vários tipos diferentes de componentes.

Ela contém tipos de dados que seriam normalmente considerados como parte “central” de uma linguagem, tais como números e listas. Para esses tipos, o núcleo da linguagem Python define a forma de literais e coloca algumas restrições em suas semânticas, mas não define completamente as semânticas. (Por outro lado, o núcleo da linguagem define propriedades sintáticas como a ortografia e a prioridade de operadores.)

A biblioteca também contém exceções e funções embutidas — objetos que podem ser usados por todo o código Python sem a necessidade de uma instrução `import`. Alguns desses são definidos pelo núcleo da linguagem, mas muitos não são essenciais para as semânticas principais e são apenas descritos aqui.

A maior parte da biblioteca, entretanto, consiste em uma coleção de módulos. Há muitas formas de dissecar essa coleção. Alguns módulos são escritos em C e colocados no interpretador do Python; outros são escritos em Python e importados na forma de código. Alguns módulos fornecem interfaces que são muito específicas do Python, como imprimir um stack trace (situação da pilha de execução); alguns fornecem interfaces que são específicas para um sistema operacional em particular, tais como acessar hardware específico; outros fornecem interfaces que são específicas de um domínio de aplicação em particular, como a World Wide Web. Alguns módulos estão disponíveis em todas as versões do Python; outros estão apenas disponíveis quando o sistema subjacente suporta ou necessita deles; e ainda outros estão disponíveis apenas quando uma opção de configuração em particular foi escolhida no momento em que o Python foi compilado e instalado.

Este manual está organizado “de dentro para fora”: ele primeiro descreve as funções embutidas, tipos de dados e exceções, e finalmente os módulos, agrupados em capítulos de módulos relacionados.

Isto significa que, se você começar a ler este manual do início, e pular para o próximo capítulo quando estiver entediado, você terá uma visão geral razoável dos módulos disponíveis e áreas de aplicação que são suportadas pela biblioteca Python. É claro, você não *tem* que ler como se fosse um romance — você também pode navegar pela tabela de conteúdos (no início do manual), ou procurar por uma função, módulo ou termo específicos no índice (na parte final). E finalmente, se você gostar de aprender sobre assuntos diversos, você pode escolher um número de página aleatório (veja o módulo `random`) e leia uma seção ou duas. Independente da ordem na qual você leia as seções deste manual, ajuda iniciar pelo capítulo *Funções embutidas*, já que o resto do manual requer familiaridade com este material.

E que o show comece!

1.1 Observações sobre disponibilidade

- Uma observação "Disponibilidade: Unix" significa que essa função é comumente encontrada em sistemas Unix. Não faz nenhuma reivindicação sobre sua existência em um sistema operacional específico.
- Caso não seja definido separadamente, todas as funções que se definem como "Disponibilidade: Unix" são suportadas no macOS, IOS e Android, todas construídas em um Unix core.
- Se uma nota de disponibilidade contiver uma versão mínima do Kernel e uma versão mínima da libc, ambas as condições deverão ser atendidas. Por exemplo, um recurso com a observação *Disponibilidade: Linux >= 3.17 com glibc >= 2.27* requer Linux 3.17 ou mais recente e glibc 2.27 ou mais recente.

1.1.1 Plataformas WebAssembly

As plataformas WebAssembly `wasm32-emscripten` (Emscripten) e `wasm32-wasi` (WASI) fornecem um subconjunto de APIs POSIX. Os tempos de execução e navegadores do WebAssembly são colocados em área restrita e têm acesso limitado ao host e aos recursos externos. Qualquer módulo de biblioteca padrão do Python que usa processos, encadeamento, rede, sinais ou outras formas de comunicação entre processos (IPC) não está disponível ou pode não funcionar como em outros sistemas semelhantes ao Unix. E/S de arquivo, sistema de arquivos e funções relacionadas à permissão do Unix também são restritas. Emscripten não permite bloqueio de E/S. Outras operações de bloqueio como *`sleep()`* bloqueiam o laço de eventos do navegador.

As propriedades e o comportamento do Python em plataformas WebAssembly dependem da versão Emscripten-SDK ou WASI-SDK, tempos de execução WASM (navegador, NodeJS, wasmtime) e sinalizadores de tempo de construção do Python. WebAssembly, Emscripten e WASI são padrões em evolução; alguns recursos como rede podem ser suportados no futuro.

Para Python no navegador, os usuários devem considerar Pyodide ou PyScript. O PyScript é construído sobre o Pyodide, que por sua vez é construído sobre o CPython e o Emscripten. O Pyodide fornece acesso às APIs JavaScript e DOM dos navegadores, bem como recursos de rede limitados com as APIs `XMLHttpRequest` e `Fetch` do JavaScript.

- As APIs relacionadas a processo não estão disponíveis ou sempre falham com um erro. Isso inclui APIs que geram novos processos (*`fork()`*, *`execve()`*), aguardam processos (*`waitpid()`*), enviam sinais (*`kill()`*), ou interagir com processos. O *`subprocess`* é importável, mas não funciona.
- O módulo *`socket`* está disponível, mas é limitado e se comporta de maneira diferente de outras plataformas. No Emscripten, os soquetes são sempre não bloqueantes e requerem código JavaScript adicional e auxiliares no servidor para intermediar TCP por meio de WebSockets; veja Emscripten Networking para mais informações. A primeira snapshot de preview do WASI permite apenas soquetes de um descritor de arquivo existente.
- Algumas funções são esboço que não fazem nada e sempre retornam valores definidos no código.
- As funções relacionadas a descritores de arquivo, permissões de arquivo, propriedade de arquivo e links são limitadas e não suportam algumas operações. Por exemplo, WASI não permite links simbólicos com nomes de arquivo absolutos.

1.1.2 Plataformas móveis

O Android e o IOS são em linhas gerais sistemas operacionais POSIX. File I/O, socket handling, e threading se comportam como em qualquer sistema operacional POSIX. Entretanto, existem diversas diferenças:

- Plataformas móveis podem usar o Python apenas no modo "incorporado". Não existe um Python REPL, e não existe possibilidade para usar executáveis em separado como **`python`** ou **`pip`**. Para adicionar código Python para seu aplicativo móvel, você deve usar a API de incorporação do Python. Para maiores detalhes veja using-android e using-ios.
- Subprocessos:
  - No Android, criar subprocessos é possível mas oficialmente sem suporte. Em particular, Android não oferece suporte para nenhuma parte da API System V IPC, assim *`multiprocessing`* não está disponível.

  - Uma aplicação de iOS não pode usar qualquer forma de subprocessamento, multiprocessamento ou comunicação entre processos. Se uma aplicação de iOS tentar criar um subprocesso, o processo que cria o subprocesso será travado ou encerrado abruptamente. Uma aplicação de iOS não tem visibilidade de outras aplicações em execução, nem capacidade de comunicação com outras aplicações em execução, fora das APIs específicas do iOS que existem para essa finalidade.

- Aplicativos móveis possuem acesso limitado a recursos do sistema (como o relógio do sistema). Esses recursos serão em sua maioria apenas *leitura*, e tentativas de modificar esses recusos geralmente falhará.
- Entrada e saída do console:
  - No Android, o `stdout` e o `stderr` nativos não estão conectados a nada, então o Python instala seus próprios fluxos que redirecionam mensagens para o log do sistema. Esses podem ser vistos sob a tags `python.stdout` e `python.stderr`, respectivamente.
  - As aplicações de iOS têm um conceito limitado de saída do console. `stdout` e `stderr` *existem*, e o conteúdo escrito em `stdout` e `stderr` será visível nos registros (logs) quando executado no Xcode, mas este conteúdo *não* será gravado no registro do sistema. Se um usuário que instalou sua aplicação fornecer os registros da aplicação como ajuda de diagnóstico, eles não incluirão nenhum detalhe escrito em `stdout` ou `stderr`.
  - As aplicações para dispositivos móveis não têm nenhum conceito de `stdin`. Embora essas aplicações possam exibir um teclado na tele, este é um recurso de software, não algo anexado ao `stdin`.

    Como consequência, módulos Python que envolvem manipulação de console (como *curses* e *readline*) não estão disponíveis em plataformas móveis.

CAPÍTULO 2

Funções embutidas

O interpretador do Python possui várias funções e tipos embutidos que sempre estão disponíveis. A seguir listamos todas as funções em ordem alfabética.

Funções embutidas

A
abs()
aiter()
all()
anext()
any()
ascii()

B
bin()
bool()
breakpoint()
bytearray()
bytes()

C
callable()
chr()
classmethod()
compile()
complex()

D
delattr()
dict()
dir()
divmod()

E
enumerate()
eval()
exec()

F
filter()
float()
format()
frozenset()

G
getattr()
globals()

H
hasattr()
hash()
help()
hex()

I
id()
input()
int()
isinstance()
issubclass()
iter()

L
len()
list()
locals()

M
map()
max()
memoryview()
min()

N
next()

O
object()
oct()
open()
ord()

P
pow()
print()
property()

R
range()
repr()
reversed()
round()

S
set()
setattr()
slice()
sorted()
staticmethod()
str()
sum()
super()

T
tuple()
type()

V
vars()

Z
zip()

_
__import__()

abs(*number*, */ (Separador de parâmetros somente-posicionais (PEP 570))*)

Retorna o valor absoluto de um número. O argumento pode ser um inteiro, um número de ponto flutuante ou um objeto implementando `__abs__()`. Se o argumento é um número complexo, sua magnitude é retornada.

aiter(*async_iterable*, */*)

Retorna um *iterador assíncrono* para um *iterável assíncrono*. Equivalente a chamar `x.__aiter__()`.

Nota: Ao contrário de `iter()`, `aiter()` não tem uma variante de 2 argumentos.

Adicionado na versão 3.10.

all(*iterable*, */*)

Retorna `True` se todos os elementos de *iterable* são verdadeiros (ou se *iterable* estiver vazio). Equivalente a:

```
def all(iterable):
    for elemento in iterable:
        if not elemento:
            return False
    return True
```

awaitable anext(*async_iterator*, */*)

awaitable anext(*async_iterator*, *default*, /)

Quando aguardado, retorna o próximo item do *iterador assíncrono* fornecido, ou *default* se fornecido e o iterador for esgotado.

Esta é a variante assíncrona do `next()` embutido, e se comporta de forma semelhante.

Isso chama o método `__anext__()` de *async_iterator*, retornando um *aguardável*. Ao aguardar isso, retorna o próximo valor do iterador. Se *default* for fornecido, ele será retornado se o iterador for esgotado, caso contrário, a exceção `StopAsyncIteration` será levantada.

Adicionado na versão 3.10.

any(*iterable*, /)

Retorna `True` se algum elemento de *iterable* for verdadeiro. Se *iterable* estiver vazio, retorna `False`. Equivalente a:

```
def any(iterable):
    for elemento in iterable:
        if elemento:
            return True
    return False
```

ascii(*object*, /)

Como `repr()`, retorna uma string contendo uma representação imprimível de um objeto, mas faz escape de caracteres não-ASCII na string retornada por `repr()` usando sequências de escapes `\x`, `\u` ou `\U`. Isto gera uma string similar ao que é retornado por `repr()` no Python 2.

bin(*integer*, /)

Converte um número inteiro para uma string de binários prefixada com "0b". O resultado é uma expressão Python válida. Se *integer* não é um objeto Python `int`, ele tem que definir um método `__index__()` que devolve um inteiro. Alguns exemplos:

```
>>> bin(3)
'0b11'
>>> bin(-10)
'-0b1010'
```

Se o prefixo "0b" é desejado ou não, você pode usar uma das seguintes maneiras.

```
>>> format(14, '#b'), format(14, 'b')
('0b1110', '1110')
>>> f'{14:#b}', f'{14:b}'
('0b1110', '1110')
```

Veja também `format()` para mais informações.

class bool(*object=False*, /)

Retorna um valor Booleano, isto é, `True` ou `False`. O argumento é convertido usando o *procedimento de teste de verdade* padrão. Se o argumento for falso ou foi omitido, isso retorna `False`; senão `True`. A classe `bool` é uma subclasse de `int` (veja *Tipos numéricos — int, float, complex*). Ela não pode ser usada para criar outra subclasse. Suas únicas instâncias são `False` e `True` (veja *Tipo booleano - bool*).

Alterado na versão 3.7: O parâmetro agora é somente-posicional

breakpoint(**args*, ***kws*)

Esta função coloca você no depurador no local da chamada. Especificamente, ela chama `sys.breakpointhook()`, passando `args` e `kws` diretamente. Por padrão, `sys.breakpointhook()` chama `pdb.set_trace()` não esperando nenhum argumento. Neste caso, isso é puramente uma função de conveniência para você não precisar importar `pdb` explicitamente ou digitar mais código para entrar no depurador. Contudo, `sys.breakpointhook()` pode ser configurado para alguma outra função e `breakpoint()` irá automaticamente chamá-la, permitindo você ir para o depurador de sua escolha. Se `sys.breakpointhook()` não estiver acessível, esta função vai levantar `RuntimeError`.

Por padrão, o comportamento de *breakpoint()* pode ser alterado com a variável de ambiente PYTHONBREAKPOINT. Veja *sys.breakpointhook()* para detalhes de uso.

Observe que isso não é garantido se *sys.breakpointhook()* tiver sido substituído.

Levanta um *evento de auditoria* builtins.breakpoint com o argumento breakpointhook.

Adicionado na versão 3.7.

class bytearray(*source=b''*)

class bytearray(*source*, *encoding*, *errors='strict'*)

Retorna um novo vetor de bytes. A classe *bytearray* é uma sequência mutável de inteiros no intervalo 0 <= x < 256. Ela tem a maior parte dos métodos mais comuns de sequências mutáveis, descritas em *Tipos sequências mutáveis*, assim como a maior parte dos métodos que o tipo *bytes* tem, veja *Operações com Bytes e Bytearray*.

O parâmetro opcional *source* pode ser usado para inicializar o vetor de algumas maneiras diferentes:

- Se é uma *string*, você deve informar o parâmetro *encoding* (e opcionalmente, *errors*); *bytearray()* então converte a string para bytes usando *str.encode()*.
- Se é um *inteiro*, o vetor terá esse tamanho e será inicializado com bytes nulos.
- Se é um objeto em conformidade com a interface de buffer, um buffer somente leitura do objeto será usado para inicializar o vetor de bytes.
- Se é um *iterável*, deve ser um iterável de inteiros no intervalo `0 <= x < 256`, que serão usados como o conteúdo inicial do vetor.

Sem nenhum argumento, um vetor de tamanho 0 é criado.

Veja também *Tipos de sequência binária — bytes, bytearray, memoryview* e *Objetos Bytearray*.

class bytes(*source=b''*)

class bytes(*source*, *encoding*, *errors='strict'*)

Retorna um novo objeto "bytes" que é uma sequência imutável de inteiros no intervalo `0 <= x < 256`. *bytes* é uma versão imutável de *bytearray* – tem os mesmos métodos de objetos imutáveis e o mesmo comportamento de índices e fatiamento.

Consequentemente, argumentos do construtor são interpretados como os de *bytearray()*.

Objetos bytes também podem ser criados com literais, veja strings.

Veja também *Tipos de sequência binária — bytes, bytearray, memoryview*, *Objetos bytes*, e *Operações com Bytes e Bytearray*.

callable(*object*, */*)

Retorna *True* se o argumento *object* parece ser chamável, *False* caso contrário. Se retorna True, ainda é possível que a chamada falhe, mas se é False, chamar *object* nunca será bem sucedido. Note que classes são chamáveis (chamar uma classe devolve uma nova instância); instâncias são chamáveis se suas classes possuem um método __call__().

Adicionado na versão 3.2: Esta função foi removida na versão 3.0, mas retornou no Python 3.2.

chr(*codepoint*, */*)

Retorna a string representando um caractere com o ponto de código Unicode especificado. Por exemplo, chr(97) retorna a string 'a', enquanto chr(8364) retorna a string '€'. É o inverso de *ord()*.

O intervalo válido para o argumento vai de 0 até 1.114.111 (0x10FFFF na base 16). Será lançada uma exceção *ValueError* se ele estiver fora desse intervalo.

@classmethod

Transforma um método em um método de classe.

Um método de classe recebe a classe como um primeiro argumento implícito, exatamente como um método de instância recebe a instância. Para declarar um método de classe, faça dessa forma:

```
class C:
    @classmethod
    def f(cls, arg1, arg2): ...
```

O termo `@classmethod` é uma função *decoradora* – veja function para detalhes.

Um método de classe pode ser chamado tanto da classe (como em `C.f()`) quanto da instância (como em `C().f()`). A instância é ignorada, exceto por sua classe. Se um método de classe é chamado por uma classe derivada, o objeto da classe derivada é passado como primeiro argumento implícito.

Métodos de classe são diferentes de métodos estáticos em C++ ou Java. Se você quer saber desses, veja *staticmethod()* nesta seção. Para mais informações sobre métodos de classe, consulte types.

Alterado na versão 3.9: Métodos de classe agora podem envolver outros *descritores* tal como *property()*.

Alterado na versão 3.10: Métodos de classe agora herdam os atributos do método (`__module__`, `__name__`, `__qualname__`, `__doc__` e `__annotations__`) e têm um novo atributo `__wrapped__`.

Deprecated since version 3.11, removed in version 3.13: Métodos de classe não podem mais envolver outros *descritores* tal como *property()*.

compile(*source*, *filename*, *mode*, *flags=0*, *dont_inherit=False*, *optimize=-1*)

Compila o argumento *source* em código ou objeto AST. Objetos código podem ser executados por *exec()* ou *eval()*. *source* pode ser uma string normal, uma string byte, ou um objeto AST. Consulte a documentação do módulo *ast* para saber como trabalhar com objetos AST.

O argumento *filename* deve ser o arquivo de onde o código será lido; passe algum valor reconhecível se isso não foi lido de um arquivo (`'<string>'` é comumente usado).

O argumento *mode* especifica qual o tipo de código deve ser compilado; pode ser `'exec'` se *source* consiste em uma sequência de instruções, `'eval'` se consiste de uma única expressão, ou `'single'` se consiste de uma única instrução interativa (neste último caso, instruções que são avaliadas para alguma coisa diferente de `None` serão exibidas).

Os argumentos opcionais *flags* e *dont_inherit* controlam quais *opções do compilador* devem ser ativadas e quais recursos futuros devem ser permitidos. Se nenhum estiver presente (ou ambos forem zero), o código é compilado com os mesmos sinalizadores que afetam o código que está chamando *compile()*. Se o argumento *flags* for fornecido e *dont_inherit* não for (ou for zero), as opções do compilador e as instruções futuras especificadas pelo argumento *flags* são usadas além daquelas que seriam usadas de qualquer maneira. Se *dont_inherit* for um inteiro diferente de zero, então o argumento *flags* é – os sinalizadores (recursos futuros e opções do compilador) no código circundante são ignorados.

Opções de compilador e instruções futuras são especificadas por bits, assim pode ocorrer uma operação *OU* bit a bit para especificar múltiplas instruções. O sinalizador necessário para especificar um dado recurso futuro pode ser encontrado no atributo *compiler_flag* na instância *_Feature* do módulo *__future__*. *Sinalizadores de compilador* podem ser encontrados no módulo *ast*, com o prefixo `PyCF_`.

O argumento *optimize* especifica o nível de otimização do compilador; o valor padrão de `-1` seleciona o nível de otimização do interpretador dado pela opção `-O`. Níveis explícitos são `0` (nenhuma otimização; `__debug__` é verdadeiro), `1` (instruções `assert` são removidas, `__debug__` é falso) ou `2` (strings de documentação também são removidas).

Essa função levanta *SyntaxError* se o código para compilar é inválido, e *ValueError* se o código contém bytes nulos.

Se você quer analisar código Python em sua representação AST, veja *ast.parse()*.

Levanta um *evento de auditoria* `compile` com os argumentos `source`, `filename`. Esse evento pode também ser levantando por uma compilação implícita.

Nota

Quando compilando uma string com código multi-linhas em modo `'single'` ou `'eval'`, entrada deve ser terminada por ao menos um caractere de nova linhas. Isso é para facilitar a detecção de instruções completas e incompletas no módulo *code*.

Aviso

É possível quebrar o interpretador Python com uma string suficientemente grande/complexa ao compilar para um objeto AST, devido limitações do tamanho da pilha no compilador AST do Python.

Alterado na versão 3.2: Permitido o uso de marcadores de novas linhas no estilo Windows e Mac. Além disso, em modo `'exec'` a entrada não precisa mais terminar com uma nova linha. Também foi adicionado o parâmetro *optimize*.

Alterado na versão 3.5: Anteriormente, *TypeError* era levantada quando havia bytes nulos em *source*.

Adicionado na versão 3.8: `ast.PyCF_ALLOW_TOP_LEVEL_AWAIT` agora pode ser passado em *flags* para habilitar o suporte em nível superior a `await`, `async for`, e `async with`.

class complex(*number=0*, */*)

class complex(*string*, */*)

class complex(*real=0*, *imag=0*)

Converte uma única string ou número para um número complexo, ou cria um número complexo a partir de partes real e imaginária.

Exemplos:

```
>>> complex('+1.23')
(1.23+0j)
>>> complex('-4.5j')
-4.5j
>>> complex('-1.23+4.5j')
(-1.23+4.5j)
>>> complex('\t( -1.23+4.5J )\n')
(-1.23+4.5j)
>>> complex('-Infinity+NaNj')
(-inf+nanj)
>>> complex(1.23)
(1.23+0j)
>>> complex(imag=-4.5)
-4.5j
>>> complex(-1.23, 4.5)
(-1.23+4.5j)
```

Se o argumento for uma string, ele deve conter ou a parte real (com o mesmo formato usado em *float()*) ou uma parte imaginária (com o mesmo formato, mas com um sufixo `'j'` ou `'J'`), ou então ambas as partes real e imaginária (caso no qual o sinal da parte imaginária é obrigatório). A string pode opcionalmente ser cercada por espaços em branco e parênteses `'('` e `')'`, que são ignorados. A string não deve conter espaços em branco entre os símbolos `'+'`, `'-'`, o sufixo `'j'` ou `'J'`, e o número decimal. Por exemplo, `complex('1+2j')` é ok, mas `complex('1 + 2j')` levanta *ValueError*. Mais precisamente, após descartar os parênteses e os espaços em branco do início e do final, a entrada deve ser conforme a regra de produção *complexvalue* da gramática a seguir:

```
complexvalue: floatvalue |
              floatvalue ("j" | "J") |
              floatvalue sign absfloatvalue ("j" | "J")
```

Se o argumento for um número, o construtor serve como uma conversão numérica tal qual *int* e *float*. Para um objeto Python `x` qualquer, `complex(x)` delega para `x.__complex__()`. Se `__complex__()` não

está definido então a chamada é repassada para `__float__()`. Se `__float__()` não está definido então a chamada é, novamente, repassada para `__index__()`.

Se dois argumentos forem passados ou argumentos nomeados forem usados, cada argumento pode ser de qualquer tipo numérico (incluindo complexo). Se ambos argumentos forem números reais, é retornado um número complexo com *real* como parte real e *imag* como parte imaginária. Se ambos os argumentos forem números complexos, é retornado um número complexo com parte real `real.real-imag.imag` e parte imaginária `real.imag+imag.real`. Se um dos argumentos for um número real, somente a sua parte real é usada nas expressões anteriores.

Veja também *complex.from_number()* que aceita apenas um único argumento numérico.

Se todos os argumentos forem omitidos, retorna `0j`.

O tipo complexo está descrito em *Tipos numéricos — int, float, complex*.

Alterado na versão 3.6: Agrupar dígitos com sublinhados como em literais de código é permitido.

Alterado na versão 3.8: Chamadas para `__index__()` se `__complex__()` e `__float__()` não estão definidas.

Descontinuado desde a versão 3.14: Passar um número complexo como argumento *real* ou *imag* agora está descontinuado; deve ser passado apenas como um único argumento posicional.

delattr(*object*, *name*, /)

Essa função está relacionada com *setattr()*. Os argumentos são um objeto e uma string. A string deve ser o nome de um dos atributos do objeto. A função remove o atributo indicado, desde que o objeto permita. Por exemplo, `delattr(x, 'foobar')` é equivalente a `del x.foobar`. *name* não precisa ser um identificador Python (veja *setattr()*).

class dict(***kwargs*)

class dict(*mapping*, /, ***kwargs*)

class dict(*iterable*, /, ***kwargs*)

Cria um novo dicionário. O objeto *dict* é a classe do dicionário. Veja *dict* e *Tipo mapeamento — dict* para documentação sobre esta classe.

Para outros contêineres, consulte as classes embutidas *list*, *set* e *tuple*, bem como o módulo *collections*.

dir()

dir(*object*, /)

Sem argumentos, devolve a lista de nomes no escopo local atual. Com um argumento, tentará devolver uma lista de atributos válidos para esse objeto.

Se o objeto tiver um método chamado `__dir__()`, esse método será chamado e deve devolver a lista de atributos. Isso permite que objetos que implementam uma função personalizada `__getattr__()` ou `__getattribute__()` personalizem a maneira como *dir()* relata seus atributos.

Se o objeto não fornecer `__dir__()`, a função tentará o melhor possível para coletar informações do atributo `__dict__` do objeto, se definido, e do seu objeto de tipo. A lista resultante não está necessariamente completa e pode ser imprecisa quando o objeto possui um `__getattr__()` personalizado.

O mecanismo padrão *dir()* se comporta de maneira diferente com diferentes tipos de objetos, pois tenta produzir as informações mais relevantes e não completas:

- Se o objeto for um objeto de módulo, a lista conterá os nomes dos atributos do módulo.
- Se o objeto for um objeto de tipo ou classe, a lista conterá os nomes de seus atributos e recursivamente os atributos de suas bases.
- Caso contrário, a lista conterá os nomes dos atributos do objeto, os nomes dos atributos da classe e recursivamente os atributos das classes base da classe.

A lista resultante é alfabeticamente ordenada. Por exemplo:

```
>>> import struct
>>> dir()   # show the names in the module namespace
['__builtins__', '__name__', 'struct']
>>> dir(struct)   # show the names in the struct module
['Struct', '__all__', '__builtins__', '__cached__', '__doc__', '__file__',
 '__initializing__', '__loader__', '__name__', '__package__',
 '_clearcache', 'calcsize', 'error', 'pack', 'pack_into',
 'unpack', 'unpack_from']
>>> class Shape:
...     def __dir__(self):
...         return ['area', 'perimeter', 'location']
...
>>> s = Shape()
>>> dir(s)
['area', 'location', 'perimeter']
```

Nota

Como *dir()* é fornecido principalmente como uma conveniência para uso em um prompt interativo, ele tenta fornecer um conjunto interessante de nomes mais do que tenta fornecer um conjunto de nomes definido de forma rigorosa ou consistente, e seu comportamento detalhado pode mudar nos lançamentos. Por exemplo, os atributos de metaclasse não estão na lista de resultados quando o argumento é uma classe.

divmod(*a*, *b*, */*)

Recebe dois números (não complexos) como argumentos e retorna um par de números que consiste em seu quociente e resto ao usar a divisão inteira. Com tipos de operandos mistos, as regras para operadores aritméticos binários se aplicam. Para números inteiros, o resultado é o mesmo que `(a // b, a % b)`. Para números de ponto flutuante, o resultado é `(q, a % b)`, onde *q* geralmente é `math.floor(a / b)`, mas pode ser 1 a menos que isso. Em qualquer caso, `q * b + a % b` está muito próximo de *a*, se `a % b` é diferente de zero, tem o mesmo sinal que *b* e `0 <= abs(a % b) < abs(b)`.

enumerate(*iterable*, *start=0*)

Devolve um objeto enumerado. *iterable* deve ser uma sequência, um *iterador* ou algum outro objeto que suporte a iteração. O método *__next__()* do iterador retornado por *enumerate()* devolve uma tupla contendo uma contagem (a partir de *start*, cujo padrão é 0) e os valores obtidos na iteração sobre *iterable*.

```
>>> seasons = ['Spring', 'Summer', 'Fall', 'Winter']
>>> list(enumerate(seasons))
[(0, 'Spring'), (1, 'Summer'), (2, 'Fall'), (3, 'Winter')]
>>> list(enumerate(seasons, start=1))
[(1, 'Spring'), (2, 'Summer'), (3, 'Fall'), (4, 'Winter')]
```

Equivalente a:

```
def enumerate(iterable, start=0):
    n = start
    for elem in iterable:
        yield n, elem
        n += 1
```

eval(*source*, */*, *globals=None*, *locals=None*)

Parâmetros

- **source** (*str* | code object) – Uma expressão Python.
- **globals** (*dict* | `None`) – O espaço de nomes global (por padrão, `None`).

- **locals** (*mapping* | None) – O espaço de nomes local (por padrão, None).

Retorna
O resultado da expressão avaliada.

levanta
Erros de sintaxe são reportados como exceções.

> **Aviso**
>
> Esta função executa código arbitrário. Chamá-la com entrada fornecida pelo usuário pode levar a vulnerabilidades de segurança.

O argumento *expression* é analisado e avaliado como uma expressão Python (tecnicamente falando, uma lista de condições) usando os mapeamentos *globals* e *locals* como espaços de nomes globais e locais. Se o dicionário *globals* estiver presente e não contiver um valor para a chave __builtins__, uma referência ao dicionário do módulo embutido *builtins* será inserida sob essa chave antes de *expression* ser analisado. Dessa forma, você pode controlar quais funções embutidas estão disponíveis para o código executado inserindo seu próprio dicionário __builtins__ em *globals* antes de passá-lo para *eval()*. Se o mapeamento *locals* for omitido, o padrão será o dicionário *globals*. Se os dois mapeamentos forem omitidos, a expressão será executada com os *globals* e *locals* no ambiente em que *eval()* é chamado. Observe que *eval()* terá acesso a *escopos aninhados* (não locais) no ambiente anexo somente se eles já forem referenciados pelo escopo que está chamando *eval()* (por exemplo, via uma instrução nonlocal).

Exemplo:

```
>>> x = 1
>>> eval('x+1')
2
```

Esta função também pode ser usada para executar objetos código arbitrários (como os criados por *compile()*). Nesse caso, passe um objeto código em vez de uma string. Se o objeto código foi compilado com 'exec' como o argumento *mode*, o valor de retorno de *eval()* será None.

Dicas: a execução dinâmica de instruções é suportada pela função *exec()*. As funções *globals()* e *locals()* retornam o dicionário global e local atual, respectivamente, o que pode ser útil para ser usado por *eval()* ou *exec()*.

Se a fonte fornecida for uma string, os espaços e tabulações à esquerda ou à direita serão removidos.

Veja *ast.literal_eval()* para uma função que pode avaliar com segurança strings com expressões contendo apenas literais.

Levanta um *evento de auditoria* exec com o código objeto como argumento. Eventos de compilação de código também podem ser levantados.

Alterado na versão 3.13: Os argumentos *globals* e *locals* podem agora ser passados como argumentos nomeados.

Alterado na versão 3.13: A forma de uso do espaço de nomes *locals* padrão foi ajustado conforme descrito na função embutida *locals()*.

exec(*source*, */*, *globals=None*, *locals=None*, ** (Separador de parâmetros somente-nomeados (PEP 3102))*, *closure=None*)

> **Aviso**
>
> Esta função executa código arbitrário. Chamá-la com entrada fornecida pelo usuário pode levar a vulnerabilidades de segurança.

Esta função suporta a execução dinâmica de código Python. O parâmetro *source* deve ser ou uma string ou um objeto código. Se for uma string, a mesma é analisada como um conjunto de instruções Python, o qual é então executado (exceto caso um erro de sintaxe ocorra).[1] Se for um objeto código, ele é simplesmente executado. Em todos os casos, espera-se que o código a ser executado seja válido como um arquivo de entrada (veja a seção file-input no Manual de Referência). Tenha cuidado, pois as instruções `nonlocal`, `yield` e `return` não podem ser usadas fora das definições de funções mesmo dentro do contexto do código passado para a função *exec()* . O valor de retorno é sempre `None`.

Em todos os casos, se os parâmetros opcionais são omitidos, o código é executado no escopo atual. Se somente *globals* é fornecido, deve ser um dicionário (e não uma subclasse de dicionário), que será usado tanto para as variáveis globais quanto para locais. Se *globals* e *locals* são fornecidos, eles são usados para as variáveis globais e locais, respectivamente. Se fornecido, *locals* pode ser qualquer objeto de mapeamento. Lembre que no nível de módulo, globais e locais são o mesmo dicionário.

Nota

Quando or `exec` recebe dois objetos separados como *globals* and *locals*, o código será executado como se estivesse embutido em uma definição de classe. Isso significa que funções e classes definidas no código executado não poderão acessar variáveis que sofreram atribuições no escopo mais externo (pois, em uma definição de classe, tais variáveis seriam tratadas como variáveis de classe).

Se o dicionário *globals* não contém um valor para a chave `__builtins__`, a referência para o dicionário do módulo embutido *builtins* é inserido com essa chave. A maneira que você pode controlar quais embutidos estão disponíveis para o código executado é inserindo seu próprio `__builtins__` dicionário em *globals* antes de passar para *exec()*.

O argumento *closure* especifica uma clausura – uma tupla de cellvars. Só é válido quando o *objeto* é um objeto código contendo *variáveis livres (de clausura)*. O comprimento da tupla deve corresponder exatamente ao comprimento do atributo `co_freevars` do objeto código.

Levanta um *evento de auditoria* `exec` com o código objeto como argumento. Eventos de compilação de código também podem ser levantados.

Nota

As funções embutidas *globals()* e *locals()* devolvem o espaço de nomes global e local, respectivamente, o que pode ser útil para passar adiante e usar como segundo ou terceiro argumento para *exec()*.

Nota

locals padrão atua como descrito pela função *locals()* abaixo. Se você precisa ver efeitos do código em *locals* depois da função *exec()* retornar passe um dicionário *locals* explícito.

Alterado na versão 3.11: Adicionado o parâmetro *closure*.

Alterado na versão 3.13: Os argumentos *globals* e *locals* podem agora ser passados como argumentos nomeados.

Alterado na versão 3.13: A forma de uso do espaço de nomes *locals* padrão foi ajustado conforme descrito na função embutida *locals()*.

filter(*function*, *iterable*, /)

Constrói um iterador a partir dos elementos de *iterable* para os quais *function* é verdadeiro. *iterable* pode ser uma sequência, um contêiner que com suporte a iteração, ou um iterador. Se *function* for `None`, a função identidade será usada, isto é, todos os elementos de *iterable* que são falsos são removidos.

[1] Observe que o analisador sintático aceita apenas a convenção de fim de linha no estilo Unix. Se você estiver lendo o código de um arquivo, use o modo de conversão de nova linha para converter novas linhas no estilo Windows ou Mac.

Note que `filter(function, iterable)` é equivalente a expressão geradora `(item for item in iterable if function(item))` se *function* não for `None` e `(item for item in iterable if item)` se *function* for `None`.

Veja *itertools.filterfalse()* para a função complementar que devolve elementos de *iterable* para os quais *function* é falso.

class float(*number=0.0*, */*)

class float(*string*, */*)

Retorna um número de ponto flutuante construído a partir de um número ou string.

Exemplos:

```
>>> float('+1.23')
1.23
>>> float('   -12345\n')
-12345.0
>>> float('1e-003')
0.001
>>> float('+1E6')
1000000.0
>>> float('-Infinity')
-inf
```

Se o argumento for uma string, ele deve conter um número decimal, opcionalmente precedido por um sinal e opcionalmente embutido em um espaço em branco. O sinal opcional pode ser `'+'` ou `'-'`; um sinal `'+'` não tem efeito no valor produzido. O argumento também pode ser uma string representando um NaN (não um número) ou infinito positivo ou negativo. Mais precisamente, a entrada deve estar de acordo com a regra de produção *floatvalue* na seguinte gramática, depois que os espaços em branco iniciais e finais forem removidos:

```
sign:           "+" | "-"
infinity:       "Infinity" | "inf"
nan:            "nan"
digit:          <a Unicode decimal digit, i.e. characters in Unicode general category Nd>
digitpart:      digit (["_"] digit)*
number:         [digitpart] "." digitpart | digitpart ["."]
exponent:       ("e" | "E") [sign] digitpart
floatnumber:    number [exponent]
absfloatvalue: floatnumber | infinity | nan
floatvalue:     [sign] absfloatvalue
```

O caso não é significativo, então, por exemplo, “inf”, “Inf”, “INFINITY” e “iNfINity” são todas grafias aceitáveis para o infinito positivo.

Caso contrário, se o argumento é um inteiro ou um número de ponto flutuante, um número de ponto flutuante com o mesmo valor (com a precisão de ponto flutuante de Python) é retornado. Se o argumento está fora do intervalo de um ponto flutuante Python, uma exceção *OverflowError* será lançada.

Para um objeto Python genérico `x`, `float(x)` delega para o método `x.__float__()`. Se `__float__()` não estiver definido, então ele delega para o método `__index__()`.

Veja também *float.from_number()* que aceita apenas um argumento numérico.

Se nenhum argumento for fornecido, será retornado `0.0`.

O tipo float é descrito em *Tipos numéricos — int, float, complex*.

Alterado na versão 3.6: Agrupar dígitos com sublinhados como em literais de código é permitido.

Alterado na versão 3.7: O parâmetro agora é somente-posicional

Alterado na versão 3.8: Chamada para `__index__()` se `__float__()` não está definido.

format(*value*, *format_spec*='', /)

Converte um valor *value* em uma representação "formatada", controlado por *format_spec*. A interpretação de *format_spec* dependerá do tipo do argumento *value*; no entanto há uma sintaxe de formatação padrão usada pela maioria dos tipos embutidos: *Minilinguagem de especificação de formato*.

O *format_spec* padrão é uma string vazia que geralmente produz o mesmo efeito que chamar `str(value)`.

Uma chamada de `format(value, format_spec)` é convertida em `type(value).__format__(value, format_spec)`, que ignora o dicionário da instância ao pesquisar o método `__format__()` de `value`. Uma exceção `TypeError` é levantada se a pesquisa do método atingir `object` e o *format_spec* não estiver vazio, ou se o *format_spec* ou o valor de retorno não forem strings.

Alterado na versão 3.4: `object().__format__(format_spec)` levanta um `TypeError` se *format_spec* não for uma string vazia.

class frozenset(*iterable=()*, /)

Devolve um novo objeto `frozenset`, opcionalmente com elementos obtidos de *iterable*. `frozenset` é uma classe embutida. Veja `frozenset` e *Tipo conjuntos — set, frozenset* para documentação sobre essas classes.

Para outros contêineres veja as classes embutidas `set`, `list`, `tuple`, e `dict`, assim como o módulo `collections`.

getattr(*object*, *name*, /)

getattr(*object*, *name*, *default*, /)

Devolve o valor do atributo *name* de *object*. *name* deve ser uma string. Se a string é o nome de um dos atributos do objeto, o resultado é o valor de tal atributo. Por exemplo, `getattr(x, 'foobar')` é equivalente a `x.foobar`. Se o atributo não existir, *default* é devolvido se tiver sido fornecido, caso contrário a exceção `AttributeError` é levantada. *name* não precisa ser um identificador Python (veja `setattr()`).

> **Nota**
>
> Uma vez que desfiguração de nome privado acontece em tempo de compilação, deve-se manualmente mutilar o nome de um atributo privado (atributos com dois sublinhados à esquerda) para recuperá-lo com `getattr()`.

globals()

Retorna o dicionário implementando o espaço de nomes do módulo atual. Para código dentro de funções, isso é definido quando a função é definida e permanece o mesmo, independentemente de onde a função é chamada.

hasattr(*object*, *name*, /)

Os argumentos são um objeto e uma string. O resultado é `True` se a string é o nome de um dos atributos do objeto, e `False` se ela não for. (Isto é implementado chamando `getattr(object, name)` e vendo se levanta um `AttributeError` ou não.)

hash(*object*, /)

Retorna o valor hash de um objeto (se houver um). Valores hash são números inteiros. Eles são usados para rapidamente comparar chaves de dicionários durante uma pesquisa em um dicionário. Valores numéricos que ao serem comparados são iguais, possuem o mesmo valor hash (mesmo que eles sejam de tipos diferentes, como é o caso de 1 e 1.0).

> **Nota**
>
> Para objetos com métodos `__hash__()` personalizados, fique atento que `hash()` trunca o valor devolvido baseado no comprimento de bits da máquina hospedeira.

help()

help(*request*)

Invoca o sistema de ajuda embutido. (Esta função é destinada para uso interativo.) Se nenhum argumento é passado, o sistema interativo de ajuda inicia no interpretador do console. Se o argumento é uma string, então a string é pesquisada como o nome de um módulo, função, classe, método, palavra-chave, ou tópico de documentação, e a página de ajuda é exibida no console. Se o argumento é qualquer outro tipo de objeto, uma página de ajuda para o objeto é gerada.

Note que se uma barra(/) aparecer na lista de parâmetros de uma função, quando invocando *help()*, significa que os parâmetros anteriores a barra são apenas posicionais. Para mais informações, veja a entrada no FAQ sobre parâmetros somente-posicionais.

Esta função é adicionada ao espaço de nomes embutido pelo módulo *site*.

Alterado na versão 3.4: Mudanças em *pydoc* e *inspect* significam que as assinaturas reportadas para chamáveis agora são mais compreensíveis e consistentes.

hex(*integer*, /)

Converte um número inteiro para uma string hexadecimal em letras minúsculas prefixada com "0x". Se *integer* não é um objeto *int* do Python, ele tem que definir um método `__index__()` que retorne um inteiro. Alguns exemplos:

```
>>> hex(255)
'0xff'
>>> hex(-42)
'-0x2a'
```

Se você quer converter um número inteiro para uma string hexadecimal em letras maiúsculas ou minúsculas, com prefixo ou sem, você pode usar qualquer uma das seguintes maneiras:

```
>>> '%#x' % 255, '%x' % 255, '%X' % 255
('0xff', 'ff', 'FF')
>>> format(255, '#x'), format(255, 'x'), format(255, 'X')
('0xff', 'ff', 'FF')
>>> f'{255:#x}', f'{255:x}', f'{255:X}'
('0xff', 'ff', 'FF')
```

Veja também *format()* para mais informações.

Veja também *int()* para converter uma string hexadecimal para um inteiro usando a base 16.

> **Nota**
>
> Para obter uma string hexadecimal de um ponto flutuante, use o método *float.hex()*.

id(*object*, /)

Devolve a "identidade" de um objeto. Ele é um inteiro, o qual é garantido que será único e constante para este objeto durante todo o seu ciclo de vida. Dois objetos com ciclos de vida não sobrepostos podem ter o mesmo valor para *id()*.

Este é o endereço do objeto na memória.

Levanta um *evento de auditoria* `builtins.id` com o argumento `id`.

input()

input(*prompt*, /)

Se o argumento *prompt* estiver presente, escreve na saída padrão sem uma nova linha ao final. A função então lê uma linha da fonte de entrada, converte a mesma para uma string (removendo o caractere de nova linha ao final), e devolve isso. Quando o final do arquivo (EOF / end-of-file) é encontrado, um erro *EOFError* é levantado. Exemplo:

```
>>> s = input('--> ')
--> Monty Python's Flying Circus
>>> s
"Monty Python's Flying Circus"
```

Se o módulo *readline* foi carregado, então *input()* usará ele para prover edição de linhas elaboradas e funcionalidades de histórico.

Levanta um *evento de auditoria* `builtins.input` com argumento `prompt` antes de ler a entrada.

Levanta um *evento de auditoria* `builtins.input/result` com o resultado depois de ler a entrada com sucesso.

class int(*number=0*, */*)

class int(*string*, */*, *base=10*)

Retorna um objeto do tipo inteiro construído a partir de um número ou string, ou retorna `0` caso nenhum argumento seja passado.

Exemplos:

```
>>> int(123.45)
123
>>> int('123')
123
>>> int('   -12_345\n')
-12345
>>> int('FACE', 16)
64206
>>> int('0xface', 0)
64206
>>> int('01110011', base=2)
115
```

Se o argumento definir um método `__int__()`, então `int(x)` retorna `x.__int__()`. Se *x* definir um método `__index__()`, então ele retorna `x.__index__()`. Para números de ponto flutuante, isto trunca o número na direção do zero.

Se o argumento não for um número ou se *base* for fornecido, então o argumento deve ser uma instância de string, *bytes* ou *bytearray* representando um inteiro na base *base*. Opcionalmente, a string pode ser precedida por `+` ou `-` (sem espaço entre eles), ter zeros à esquerda, estar entre espaços em branco e ter sublinhados simples intercalados entre os dígitos.

Uma string de inteiro de base n contém dígitos, cada um representando um valor de 0 a n-1. Os valores 0–9 podem ser representados por qualquer dígito decimal Unicode. Os valores 10–35 podem ser representados por `a` a `z` (ou `A` a `Z`). A *base* padrão é 10. As bases permitidas são 0 e 2–36. As strings base-2, -8 e -16 podem ser opcionalmente prefixadas com `0b`/`0B`, `0o`/`0O` ou `0x`/`0X`, como acontece com literais inteiros no código. Para a base 0, a string é interpretada de maneira semelhante a um literal inteiro no código, em que a base real é 2, 8, 10 ou 16 conforme determinado pelo prefixo. A base 0 também não permite zeros à esquerda: `int('010', 0)` não é válido, enquanto `int('010')` e `int('010', 8)` são.

O tipo inteiro está descrito em *Tipos numéricos — int, float, complex*.

Alterado na versão 3.4: Se *base* não é uma instância de *int* e o objeto *base* tem um método `base.__index__`, então esse método é chamado para obter um inteiro para a base. Versões anteriores usavam `base.__int__` ao invés de `base.__index__`.

Alterado na versão 3.6: Agrupar dígitos com sublinhados como em literais de código é permitido.

Alterado na versão 3.7: O primeiro parâmetro agora é somente-posicional.

Alterado na versão 3.8: Chamada para `__index__()` se `__int__()` não está definido.

Alterado na versão 3.11: Entradas de strings para *int* e representações em strings podem ser limitadas para ajudar a evitar ataques de negação de serviço. Uma exceção *ValueError* é levantada quando o limite é excedido durante a conversão de uma string em um *int* ou quando a conversão de um *int* em uma string excede o limite. Consulte a documentação sobre *limitação de comprimento de conversão de string em inteiro*.

Alterado na versão 3.14: *int()* não delega mais ao método `__trunc__()`.

isinstance(*object*, *classinfo*, /)

Retorna `True` se o argumento *object* é uma instância do argumento *classinfo*, ou de uma subclasse dele (direta, indireta ou *virtual*). Se *object* não é um objeto do tipo dado, a função sempre devolve `False`. Se *classinfo* é uma tupla de tipos de objetos (ou recursivamente, como outras tuplas) ou um *Tipo União* de vários tipos, retorna `True` se *object* é uma instância de qualquer um dos tipos. Se *classinfo* não é um tipo ou tupla de tipos ou outras tuplas, é levantada uma exceção *TypeError*. *TypeError* pode não ser levantada por um tipo inválido se uma verificação anterior for bem-sucedida.

Alterado na versão 3.10: *classinfo* pode ser um *Tipo União*.

issubclass(*class*, *classinfo*, /)

Retorna `True` se *class* for uma subclasse (direta, indireta ou *virtual*) de *classinfo*. Uma classe é considerada uma subclasse de si mesma. *classinfo* pode ser uma tupla de objetos de classe (ou recursivamente, outras tuplas) ou um *Tipo União*, caso em que retorna `True` se *class* for uma subclasse de qualquer entrada em *classinfo*. Em qualquer outro caso, é levantada uma exceção *TypeError*.

Alterado na versão 3.10: *classinfo* pode ser um *Tipo União*.

iter(*iterable*, /)

iter(*callable*, *sentinel*, /)

Retorna um objeto *iterador*. O primeiro argumento é interpretado muito diferentemente dependendo da presença do segundo argumento. Sem um segundo argumento, o primeiro argumento deve ser uma coleção de objetos com suporte ao protocolo *iterável* (o método `__iter__()`), ou ele deve ter suporte ao protocolo de sequência (o método `__getitem__()` com argumentos inteiros começando em `0`). Se ele não tem suporte nenhum desses protocolos, uma *TypeError* é levantada. Se o segundo argumento, *sentinel*, é fornecido, então o primeiro argumento deve ser um objeto chamável. O iterador criado neste caso irá chamar *callable* sem nenhum argumento para cada chamada para o seu método *__next__()*; se o valor devolvido é igual a *sentinel*, então *StopIteration* será levantado, caso contrário o valor será devolvido.

Veja também *Tipos iteradores*.

Uma aplicação útil da segunda forma de *iter()* é para construir um bloco de leitura. Por exemplo, ler blocos de comprimento fixo de um arquivo binário de banco de dados até que o final do arquivo seja atingido:

```
from functools import partial
with open('meusdados.db', 'rb') as f:
    for bloco in iter(partial(f.read, 64), b''):
        process_block(bloco)
```

len(*object*, /)

Devolve o comprimento (o número de itens) de um objeto. O argumento pode ser uma sequência (tal como uma string, bytes, tupla, lista, ou um intervalo) ou uma coleção (tal como um dicionário, conjunto, ou conjunto imutável).

`len` levanta *OverflowError* em tamanhos maiores que *sys.maxsize*, tal como *range(2 ** 100)*.

class list(*iterable=()*, /)

Ao invés de ser uma função, *list* é na verdade um tipo de sequência mutável, conforme documentado em *Listas* e *Tipos sequências — list, tuple, range*.

locals()

Retorna um objeto de mapeamento representando a tabela de símbolos atual, com nomes de variáveis como as chaves, e as referências às quais cada variável está atrelada no momento como os valores.

Em um escopo de módulo, assim como quando usando *exec()* ou *eval()* com um único espaço de nomes, esta função retorna o mesmo espaço de nomes que *globals()*.

Em um escopo de classe, ela retorna o espaço de nomes que será passado para o construtor da metaclasse.

Quando usando `exec()` ou `eval()` com argumentos de espaço local e global distintos, ela retorna o espaço de nomes local passado na chamada.

Em todos os casos acima, cada chamada a `locals()` em um dado quadro de execução vai retornar o *mesmo* objeto de mapeamento. Mudanças feitas através do objeto de mapeamento retornado por `locals()` serão visíveis tal qual variáveis locais atribuídas, reatribuídas ou deletadas, e atribuir, reatribuir ou deletar variáveis locais afetará imediatamente o conteúdo do objeto de mapeamento retornado.

Em contraste aos casos acima, em um *escopo otimizado* (incluindo funções, geradores e corrotinas), cada chamada a `locals()` retorna um dicionário novo contendo as ligações atuais das variáveis locais da função e de quaisquer células com referências não-locais. Nesse caso, mudanças em ligações de nomes feitas através do dicionário retornado *não* são escritas de volta nas variáveis locais correspondentes nem nas células com referências não-locais, e atribuir, reatribuir ou deletar variáveis locais e células com referências não-locais *não* afeta o conteúdo dos dicionários que já foram retornados.

Chamar a função `locals()` como parte de uma compreensão em uma função, gerador ou corrotina é equivalente a chamá-la no escopo que contém a compreensão, exceto que as variáveis de iteração inicializadas da compreensão serão incluídas. Em outros escopos, essa função se comporta como se a compreensão estivesse executando como uma função aninhada.

Chamar a função `locals()` como parte de uma expressão geradora é equivalente a chamá-la em uma função geradora aninhada.

Alterado na versão 3.12: O comportamento de `locals()` em uma compreensão foi atualizado conforme descrito na **PEP 709**.

Alterado na versão 3.13: Como parte da **PEP 667**, o que acontece após a mutação de objetos de mapeamento retornados desta função está agora definido. O comportamento em *escopos otimizados* agora é o descrito acima. Além de estar definido, o comportamento em outros escopos não foi modificado em relação aos comportamentos em versões passadas.

map(*function*, *iterable*, */*, **iterables*, *strict=False*)

Devolve um iterador que aplica *function* para cada item de *iterable*, gerando os resultados. Se argumentos *iterables* adicionais são passados, *function* deve ter a mesma quantidade de argumentos e ela é aplicada aos itens de todos os iteráveis em paralelo. Com múltiplos iteráveis, o iterador para quando o iterador mais curto é encontrado. Se *strict* for `True` e um dos iteráveis for esgotado antes dos outros, um *`ValueError`* será levanta. Para casos onde os parâmetros de entrada da função já estão organizados em tuplas, veja *`itertools.starmap()`*.

Alterado na versão 3.14: Adicionado o parâmetro *track*.

max(*iterable*, ***, *key=None*)

max(*iterable*, ***, *default*, *key=None*)

max(*arg1*, *arg2*, **args*, *key=None*)

Devolve o maior item em um iterável ou o maior de dois ou mais argumentos.

Se um argumento posicional é fornecido, ele deve ser um *iterável*. O maior item no iterável é retornado. Se dois ou mais argumentos posicionais são fornecidos, o maior dos argumentos posicionais é devolvido.

Existem dois parâmetros somente-nomeados opcionais. O parâmetro *key* especifica uma função de ordenamento de um argumento, como aquelas usadas por *`list.sort()`*. O parâmetro *default* especifica um objeto a ser devolvido se o iterável fornecido estiver vazio. Se o iterável estiver vazio, e *default* não foi fornecido, uma exceção *`ValueError`* é levantada.

Se múltiplos itens são máximos, a função devolve o primeiro encontrado. Isto é consistente com outras ferramentas de ordenamento que preservam a estabilidade, tais como `sorted(iterable, key=keyfunc, reverse=True)[0]` e `heapq.nlargest(1, iterable, key=keyfunc)`.

Alterado na versão 3.4: Adicionado o parâmetro *default* somente-nomeado.

Alterado na versão 3.8: O valor de *key* pode ser `None`.

class memoryview(*object*)

Devolve um objeto de "visão da memória" criado a partir do argumento fornecido. Veja *Memory Views* para mais informações.

min(*iterable*, *, *key=None*)

min(*iterable*, *, *default*, *key=None*)

min(*arg1*, *arg2*, **args*, *key=None*)

Devolve o menor item de um iterável ou o menor de dois ou mais argumentos.

Se um argumento posicional é fornecido, ele deve ser um *iterável*. O menor item no iterável é devolvido. Se dois ou mais argumentos posicionais são fornecidos, o menor dos argumentos posicionais é devolvido.

Existem dois parâmetros somente-nomeados opcionais. O parâmetro *key* especifica uma função de ordenamento de um argumento, como aquelas usadas por `list.sort()`. O parâmetro *default* especifica um objeto a ser devolvido se o iterável fornecido estiver vazio. Se o iterável estiver vazio, e *default* não foi fornecido, uma exceção `ValueError` é levantada.

Se múltiplos itens são mínimos, a função devolve o primeiro encontrado. Isto é consistente com outras ferramentas de ordenamento que preservam a estabilidade, tais como `sorted(iterable, key=keyfunc)[0]` e `heapq.nsmallest(1, iterable, key=keyfunc)`.

Alterado na versão 3.4: Adicionado o parâmetro *default* somente-nomeado.

Alterado na versão 3.8: O valor de *key* pode ser `None`.

next(*iterator*, /)

next(*iterator*, *default*, /)

Recupera o próximo item do *iterador* chamando o seu método `__next__()`. Se *default* foi fornecido, ele é devolvido caso o iterável tenha sido percorrido por completo, caso contrário `StopIteration` é levantada.

class object

Esta é a classe base definitiva de todas as outras classes. Ela tem métodos que são comuns a todas as instâncias de classes Python. Quando o construtor é chamado, ele retorna um novo objeto sem características. O construtor não aceita nenhum argumento.

> **Nota**
>
> Instâncias de `object` *não* têm atributos `__dict__`, então você não pode atribuir atributos arbitrários a uma instância de `object`.

oct(*integer*, /)

Converte um número inteiro para uma string em base octal, prefixada com "0o". O resultado é uma expressão Python válida. Se *integer* não for um objeto `int` Python, ele tem que definir um método `__index__()` que devolve um inteiro. Por exemplo:

```
>>> oct(8)
'0o10'
>>> oct(-56)
'-0o70'
```

Se você quiser converter um número inteiro para uma string octal, com o prefixo "0o" ou não, você pode usar qualquer uma das formas a seguir.

```
>>> '%#o' % 10, '%o' % 10
('0o12', '12')
>>> format(10, '#o'), format(10, 'o')
('0o12', '12')
>>> f'{10:#o}', f'{10:o}'
('0o12', '12')
```

Veja também *format()* para mais informações.

open(*file*, *mode='r'*, *buffering=-1*, *encoding=None*, *errors=None*, *newline=None*, *closefd=True*, *opener=None*)

Abre *file* e retorna um *objeto arquivo* correspondente. Se o arquivo não puder ser aberto, uma *OSError* é levantada. Veja tut-files para mais exemplos de como usar esta função.

file é um *objeto caminho ou similar* fornecendo o caminho (absoluto ou relativo ao diretório de trabalho atual) do arquivo que será aberto, ou de um inteiro descritor de arquivo a ser manipulado (Se um descritor de arquivo é fornecido, ele é fechado quando o objeto de I/O retornado é fechado, a não ser que *closefd* esteja marcado como `False`).

mode é uma string opcional que especifica o modo no qual o arquivo é aberto. O valor padrão é `'r'`, o qual significa abrir para leitura em modo texto. Outros valores comuns são `'w'` para escrever (truncando o arquivo se ele já existe), `'x'` para criação exclusiva e `'a'` para anexar (o qual em *alguns* sistemas Unix, significa que *todas* as escritas anexam ao final do arquivo independentemente da posição de busca atual). No modo texto, se *encoding* não for especificada, a codificação usada depende da plataforma: *locale.getencoding()* é chamada para obter a codificação da localidade atual (Para ler e escrever bytes diretamente, use o modo binário e não especifique *encoding*). Os modos disponíveis são:

Caractere	Significado
`'r'`	abre para leitura (padrão)
`'w'`	abre para escrita, truncando o arquivo primeiro (removendo tudo o que estiver contido no mesmo)
`'x'`	abre para criação exclusiva, falhando caso o arquivo exista
`'a'`	abre para escrita, anexando ao final do arquivo caso o mesmo exista
`'b'`	modo binário
`'t'`	modo texto (padrão)
`'+'`	aberto para atualização (leitura e escrita)

O modo padrão é `'r'` (abre para leitura de texto, um sinônimo de `'rt'`). Modos `'w+'` e `'w+b'` abrem e truncam o arquivo. Modos `'r+'` e `'r+b'` abrem o arquivo sem truncar o mesmo.

Conforme mencionado em *Visão Geral*, Python diferencia entre entrada/saída binária e de texto. Arquivos abertos em modo binário (incluindo `'b'` no parâmetro *mode*) retornam o conteúdo como objetos *bytes* sem usar nenhuma decodificação. No modo texto (o padrão, ou quando `'t'` é incluído no parâmetro *mode*), o conteúdo do arquivo é retornado como *str*, sendo os bytes primeiramente decodificados usando uma codificação dependente da plataforma, ou usando a codificação definida em *encoding* se fornecida.

> **Nota**
>
> Python não depende da noção básica do sistema operacional sobre arquivos de texto; todo processamento é feito pelo próprio Python, e é então independente de plataforma.

buffering é um número inteiro opcional usado para definir a política de buffering. Passe 0 para desativar o buffer (permitido apenas no modo binário), 1 para selecionar o buffer de linha (usável apenas ao gravar no modo de texto) e um inteiro > 1 para indicar o tamanho em bytes de um buffer de bloco de tamanho fixo. Observe que especificar um tamanho de buffer dessa maneira se aplica a E/S com buffer binário, mas `TextIOWrapper` (ou seja, arquivos abertos com `mode='r+'`) teriam outro buffer. Para desabilitar o buffer em `TextIOWrapper`, considere usar o sinalizador `write_through` para *io.TextIOWrapper.reconfigure()*. Quando nenhum argumento *buffering* é fornecido, a política de buffering padrão funciona da seguinte forma:

- Arquivos binários são armazenados em buffer em blocos de tamanho fixo; o tamanho do buffer é `max(min(blocksize, 8 MiB), DEFAULT_BUFFER_SIZE)` quando o tamanho do bloco do dispositivo está disponível. Na maioria dos sistemas, o buffer normalmente tem 128 quilobytes de comprimento.
- Arquivos de texto "interativos" (arquivos para os quais *isatty()* retornam `True`) usam buffering de linha. Outros arquivos de texto usam a política descrita acima para arquivos binários.

encoding é o nome da codificação usada para codificar ou decodificar o arquivo. Isto deve ser usado apenas no modo texto. A codificação padrão depende da plataforma (seja qual valor *locale.getencoding()* retornar), mas qualquer *codificador de texto* suportado pelo Python pode ser usada. Veja o módulo *codecs* para a lista de codificações suportadas.

errors é uma string opcional que especifica como erros de codificação e de decodificação devem ser tratados — isso não pode ser utilizado no modo binário. Uma variedade de tratadores de erro padrão estão disponíveis (listados em *Error Handlers*), apesar que qualquer nome para tratamento de erro registrado com *codecs.register_error()* também é válido. Os nomes padrões incluem:

- `'strict'` para levantar uma exceção *ValueError* se existir um erro de codificação. O valor padrão `None` tem o mesmo efeito.
- `'ignore'` ignora erros. Note que ignorar erros de código pode levar à perda de dados.
- `'replace'` faz um marcador de substituição (tal como `'?'`) ser inserido onde existem dados malformados.
- representará quaisquer bytes incorretos como unidades de código substituto baixo variando de U + DC80 a U + DCFF. Essas unidades de código substituto serão então transformadas de volta nos mesmos bytes quando o manipulador de erros for usado ao gravar dados. Isso é útil para processar arquivos em uma codificação desconhecida.
- `'xmlcharrefreplace'` é suportado apenas ao gravar em um arquivo. Os caracteres não suportados pela codificação são substituídos pela referência de caracteres XML apropriada `&#nnn;`.
- `'backslashreplace'` substitui dados malformados pela sequência de escape utilizando contrabarra do Python.
- `'namereplace'` (também é suportado somente quando estiver escrevendo) substitui caractere não suportados com sequências de escape `\N{...}`.

newline determina como analisar caracteres de nova linha do fluxo. Ele pode ser `None`, `''`, `'\n'`, `'\r'` e `'\r\n'`. Ele funciona da seguinte forma:

- Ao ler a entrada do fluxo, se *newline* for `None`, o modo universal de novas linhas será ativado. As linhas na entrada podem terminar em `'\n'`, `'\r'` ou `'\r\n'`, e são traduzidas para `'\n'` antes de retornar ao chamador. Se for `''`, o modo de novas linhas universais será ativado, mas as terminações de linha serão retornadas ao chamador sem tradução. Se houver algum dos outros valores legais, as linhas de entrada são finalizadas apenas pela string especificada e a finalização da linha é retornada ao chamador sem tradução.
- Ao gravar a saída no fluxo, se *newline* for `None`, quaisquer caracteres `'\n'` gravados serão traduzidos para o separador de linhas padrão do sistema, *os.linesep*. Se *newline* for `''` ou `'\n'`, nenhuma tradução ocorrerá. Se *newline* for um dos outros valores legais, qualquer caractere `'\n'` escrito será traduzido para a string especificada.

Se *closefd* for `False` e um descritor de arquivo em vez de um nome de arquivo for fornecido, o descritor de arquivo subjacente será mantido aberto quando o arquivo for fechado. Se um nome de arquivo for fornecido *closefd* deve ser `True` (o padrão), caso contrário, um erro será levantado.

Um abridor personalizado pode ser usado passando um chamável como *opener*. O descritor de arquivo subjacente para o objeto arquivo é obtido chamando *opener* com (*file*, *flags*). *opener* deve retornar um descritor de arquivo aberto (passando *os.open* como *opener* resulta em funcionalidade semelhante à passagem de `None`).

O arquivo recém-criado é *não-herdável*.

O exemplo a seguir usa o parâmetro *dir_fd* da função *os.open()* para abrir um arquivo relativo a um determinado diretório:

```
>>> import os
>>> dir_fd = os.open('algum_dir', os.O_RDONLY)
>>> def opener(path, flags):
...     return os.open(path, flags, dir_fd=dir_fd)
...
```

(continua na próxima página)

(continuação da página anterior)

```
>>> with open('spamspam.txt', 'w', opener=opener) as f:
...     print('Isso será escrito para algum_dir/spamspam.txt', file=f)
...
>>> os.close(dir_fd)  # não deixe vazar um descritor de arquivo
```

O tipo de *objeto arquivo* retornado pela função *open()* depende do modo. Quando *open()* é usado para abrir um arquivo no modo texto (`'w'`, `'r'`, `'wt'`, `'rt'`, etc.), retorna uma subclasse de *io.TextIOBase* (especificamente *io.TextIOWrapper*). Quando usada para abrir um arquivo em modo binário com buffer, a classe retornada é uma subclasse de *io.BufferedIOBase*. A classe exata varia: no modo binário de leitura, ele retorna uma *io.BufferedReader*; nos modos binário de gravação e binário anexado, ele retorna um *io.BufferedWriter* e, no modo leitura/gravação, retorna um *io.BufferedRandom*. Quando o buffer está desativado, o fluxo bruto, uma subclasse de *io.RawIOBase*, *io.FileIO*, é retornado.

Veja também os módulos de para lidar com arquivos, tais como *fileinput*, *io* (onde *open()* é declarado), *os*, *os.path*, *tempfile* e *shutil*.

Levanta um *evento de auditoria* `open` com os argumentos `path`, `mode`, `flags`.

Os argumentos `mode` e `flags` podem ter sido modificados ou inferidos a partir da chamada original.

Alterado na versão 3.3:

- O parâmetro *opener* foi adicionado.
- O modo `'x'` foi adicionado.
- *IOError* costumava ser levantado, agora ele é um codinome para *OSError*.
- *FileExistsError* agora é levantado se o arquivo aberto no modo de criação exclusivo (`'x'`) já existir.

Alterado na versão 3.4:

- O arquivo agora é não herdável.

Alterado na versão 3.5:

- Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção *InterruptedError* (consulte **PEP 475** para entender a justificativa).
- O tratador de erros `'namereplace'` foi adicionado.

Alterado na versão 3.6:

- Suporte adicionado para aceitar objetos implementados *os.PathLike*.
- No Windows, a abertura de um buffer do console pode retornar uma subclasse de *io.RawIOBase* que não seja *io.FileIO*.

Alterado na versão 3.11: O modo `'U'` foi removido.

ord(*character*, */*)

Retorna o valor ordinal de um caractere.

Se o argumento for uma string de um caractere, retorna o ponto de código Unicode desse caractere.. Por exemplo, `ord('a')` retorna o número inteiro `97` e `ord('€')` (sinal do Euro) retorna `8364`. Este é o inverso de *chr()*.

Se o argumento for um objeto *bytes* ou *bytearray* de comprimento 1, retorna seu valor de byte único. Por exemplo, `ord(b'a')` retorna o inteiro `97`.

pow(*base*, *exp*, *mod=None*)

Retorna *base* à potência de *exp*; se *mod* estiver presente, retorna *base* à potência *exp*, módulo *mod* (calculado com mais eficiência do que `pow(base, exp) % mod`). A forma de dois argumentos `pow(base, exp)` é equivalente a usar o operador de potência: `base**exp`.

Os argumentos devem ter tipos numéricos. Com tipos de operandos mistos, aplicam-se as regras de coerção para operadores aritméticos binários. Para operandos `int`, o resultado tem o mesmo tipo que os operandos (após coerção), a menos que o segundo argumento seja negativo; nesse caso, todos os argumentos são convertidos em ponto flutuante e um resultado ponto flutuante é entregue. Por exemplo, `pow(10, 2)` retorna `100`, mas `pow(10, -2)` retorna `0.01`. Para uma base negativa do tipo `int` ou `float` e um expoente não integral, um resultado complexo é entregue. Por exemplo, `pow(-9, 0.5)` retorna um valor próximo a `3j`. Enquanto isso, para uma base negativa do tipo `int` ou `float` com um expoente integral, um resultado de ponto flutuante é retornado. Por exemplo, `pow(-9, 2.0)` retorna `81.0`.

Para operandos `int` *base* e *exp*, se *mod* estiver presente, *mod* também deve ser do tipo inteiro e *mod* deve ser diferente de zero. Se *mod* estiver presente e *exp* for negativo, *base* deve ser relativamente primo para *mod*. Nesse caso, `pow(inv_base, -exp, mod)` é retornado, onde *inv_base* é um inverso ao *base* módulo *mod*.

Aqui está um exemplo de computação de um inverso para `38` módulo `97`:

```
>>> pow(38, -1, mod=97)
23
>>> 23 * 38 % 97 == 1
True
```

Alterado na versão 3.8: Para operandos `int`, a forma de três argumentos de `pow` agora permite que o segundo argumento seja negativo, permitindo o cálculo de inversos modulares.

Alterado na versão 3.8: Permite argumentos de palavra reservada. Anteriormente, apenas argumentos posicionais eram suportados.

print(**objects*, *sep=' '*, *end='\n'*, *file=None*, *flush=False*)

Exibe *objects* no fluxo de texto *arquivo*, separado por *sep* e seguido por *end*. *sep*, *end*, *file* e *flush*, se houver, devem ser fornecidos como argumentos nomeados.

Todos os argumentos que não são nomeados são convertidos em strings como `str()` faz e gravados no fluxo, separados por *sep* e seguidos por *end*. *sep* e *end* devem ser strings; eles também podem ser `None`, o que significa usar os valores padrão. Se nenhum *object* for fornecido, `print()` escreverá apenas *end*.

O argumento *file* deve ser um objeto com um método `write(string)`; se ele não estiver presente ou `None`, então `sys.stdout` será usado. Como argumentos exibidos no console são convertidos para strings de texto, `print()` não pode ser usado com objetos arquivo em modo binário. Para esses casos, use `file.write(...)` ao invés.

O buffer de saída geralmente é determinado por *arquivo*. No entanto, se *flush* for verdadeiro, o fluxo será descarregado à força.

Alterado na versão 3.3: Adicionado o argumento nomeado *flush*.

class property(*fget=None*, *fset=None*, *fdel=None*, *doc=None*)

Retorna um atributo de propriedade.

fget é uma função para obter o valor de um atributo. *fset* é uma função para definir um valor para um atributo. *fdel* é uma função para deletar um valor de um atributo. E *doc* cria um docstring para um atributo.

Um uso comum é para definir um atributo gerenciável `x`:

```
class C:
    def __init__(self):
        self._x = None

    def getx(self):
        return self._x

    def setx(self, valor):
        self._x = valor

    def delx(self):
```

(continua na próxima página)

(continuação da página anterior)

```
        del self._x

    x = property(getx, setx, delx, "Eu sou a propriedade de 'x'.")
```

Se *c* é uma instância de *C*, `c.x` irá invocar o método getter, `c.x = valor` irá invocar o método setter, e `del c.x` o método deleter.

Se fornecido, *doc* será a docstring do atributo definido por property. Caso contrário, a property copiará a docstring de *fget* (se ela existir). Isso torna possível criar facilmente propriedades apenas para leitura usando *`property()`* como um *decorador*:

```
class Parrot:
    def __init__(self):
        self._voltagem = 100000

    @property
    def voltagem(self):
        """Obtém a voltagem atual."""
        return self._voltagem
```

O decorador `@property` transforma o método `voltage()` em um "getter" para um atributo somente leitura com o mesmo nome, e define a docstring de *voltage* para "Get the current voltage."

`@getter`

`@setter`

`@deleter`

Um objeto property possui métodos `getter`, `setter` e `deleter` usáveis como decoradores, que criam uma cópia da property com o assessor correspondente a função definida para a função com decorador. Isso é explicado melhor com um exemplo:

```
class C:
    def __init__(self):
        self._x = None

    @property
    def x(self):
        """Sou a propriedade de 'x'."""
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter
    def x(self):
        del self._x
```

Esse código é exatamente equivalente ao primeiro exemplo. Tenha certeza de nas funções adicionais usar o mesmo nome que a property original (`x` neste caso).

O objeto property retornado também tem os atributos `fget`, `fset`, e `fdel` correspondendo aos argumentos do construtor.

Alterado na versão 3.5: Agora é possível escrever nas docstrings de objetos property.

`__name__`

Atributo que contém o nome da propriedade. O nome da propriedade pode ser alterado em tempo de execução.

Adicionado na versão 3.13.

class range(*stop*, /)

class range(*start*, *stop*, *step=1*, /)

Em vez de ser uma função, *range* é realmente um tipo de sequência imutável, conforme documentado em *Intervalos* e *Tipos sequências — list, tuple, range*.

repr(*object*, /)

Retorna uma string contendo uma representação imprimível de um objeto. Para muitos tipos, essa função tenta retornar uma string que produziria um objeto com o mesmo valor quando passado para *eval()*, caso contrário, a representação é uma string entre colchetes angulares que contém o nome do tipo do objeto juntamente com informações adicionais, geralmente incluindo o nome e o endereço do objeto. Uma classe pode controlar o que essa função retorna para suas instâncias, definindo um método `__repr__()`. Se *sys.displayhook()* não estiver acessível, esta função vai levantar *RuntimeError*.

Esta classe possui uma representação personalizada que pode ser executada:

```
class Pessoa:
   def __init__(self, nome, idade):
      self.name = nome
      self.age = idade

   def __repr__(self):
      return f"Pessoa('{self.nome}', {self.idade})"
```

reversed(*object*, /)

Retorna um *iterador* reverso. *seq* deve ser um objeto que possui o método `__reversed__()` ou suporta o protocolo de sequência (o método `__len__()` e o método `__getitem__()` com argumentos inteiros começando em `0`).

round(*number*, *ndigits=None*)

Retorna *number* arredondado para *ndigits* precisão após o ponto decimal. Se *ndigits* for omitido ou for `None`, ele retornará o número inteiro mais próximo de sua entrada.

Para os tipos embutidos com suporte a *round()*, os valores são arredondados para o múltiplo mais próximo de 10 para a potência de menos *ndigit*; se dois múltiplos são igualmente próximos, o arredondamento é feito para a opção par (por exemplo, `round(0.5)` e `round(-0.5)` são `0` e `round(1.5)` é `2`). Qualquer valor inteiro é válido para *ndigits* (positivo, zero ou negativo). O valor de retorno é um número inteiro se *ndigits* for omitido ou `None`. Caso contrário, o valor de retorno tem o mesmo tipo que *number*.

Para um objeto Python geral `number`, `round` delega para `number.__round__`.

> **Nota**
>
> O comportamento de *round()* para pontos flutuantes pode ser surpreendente: por exemplo, `round(2.675, 2)` fornece `2.67` em vez do esperado `2.68`. Isso não é um bug: é resultado do fato de que a maioria das frações decimais não pode ser representada exatamente como um ponto flutuante. Veja tut-fp-issues para mais informações.

class set(*iterable=()*, /)

Retorna um novo objeto *set*, opcionalmente com elementos retirados de *iterable*. `set` é uma classe embutida. Veja *set* e *Tipo conjuntos — set, frozenset* para documentação sobre esta classe.

Para outros contêineres, consulte as classes embutidas *frozenset*, *list*, *tuple* e *dict*, bem como o módulo *collections*.

setattr(*object*, *name*, *value*, /)

Esta é a contrapartida de *getattr()*. Os argumentos são um objeto, uma string e um valor arbitrário. A string pode nomear um atributo existente ou um novo atributo. A função atribui o valor ao atributo, desde que o objeto permita. Por exemplo, `setattr(x, 'foobar', 123)` é equivalente a `x.foobar = 123`.

name não precisa ser um identificador do Python conforme definido em identifiers a menos que o objeto opte por impor isso, por exemplo, em um `__getattribute__()` personalizado ou via `__slots__`. Um atributo cujo nome não é um identificador não será acessível usando a notação de ponto, mas pode ser acessado através de *`getattr()`* etc.

> **Nota**
>
> Uma vez que desfiguração de nome privado acontece em tempo de compilação, deve-se manualmente mutilar o nome de um atributo privado (atributos com dois sublinhados à esquerda) para defini-lo com *`setattr()`*.

class slice(*stop*, /)

class slice(*start*, *stop*, *step=None*, /)

Retorna um objeto *fatia* representando o conjunto de índices especificados por `range(start, stop, step)`. Os argumentos *start* e *step* têm o padrão `None`.

Objetos fatia têm atributos de dados somente leitura `start`, `stop` e `step` que simplesmente retornam os valores dos argumentos (ou seus padrões). Eles não possuem outra funcionalidade explícita; no entanto, eles são usados pelo NumPy e outros pacotes de terceiros.

start

stop

step

Objetos fatia também são gerados quando a sintaxe de indexação estendida é usada. Por exemplo: `a[start:stop:step]` ou `a[start:stop, i]`. Veja *`itertools.islice()`* para uma versão alternativa que retorna um *iterador*.

Alterado na versão 3.12: Os objetos slice agora são *hasheáveis* (desde que *`start`*, *`stop`* e *`step`* sejam hasheáveis).

sorted(*iterable*, /, *, *key=None*, *reverse=False*)

Retorna uma nova lista classificada dos itens em *iterable*.

Possui dois argumentos opcionais que devem ser especificados como argumentos nomeados.

key especifica a função de um argumento usado para extrair uma chave de comparação de cada elemento em *iterable* (por exemplo, `key=str.lower`). O valor padrão é `None` (compara os elementos diretamente).

reverse é um valor booleano. Se definido igual a `True`, então os elementos da lista são classificados como se cada comparação estivesse invertida.

Usa *`functools.cmp_to_key()`* para converter a função das antigas *cmp* para uma função *key*.

A função embutida *`sorted()`* é garantida como estável. Uma ordenação é estável se garantir não alterar a ordem relativa dos elementos que se comparam da mesma forma — isso é útil para ordenar em várias passagens (por exemplo, ordenar por departamento e depois por nível de salário).

O algoritmo de classificação usa apenas comparações < entre itens. Embora definir um método `__lt__()` seja suficiente para ordenação, **PEP 8** recomenda que todas as seis comparações ricas sejam implementadas. Isso ajudará a evitar erros ao usar os mesmos dados com outras ferramentas de ordenação, como *`max()`*, que dependem de um método subjacente diferente. Implementar todas as seis comparações também ajuda a evitar confusão para comparações de tipo misto que podem chamar refletido o método `__gt__()`.

Para exemplos de classificação e um breve tutorial de classificação, veja sortinghowto.

@staticmethod

Transforma um método em método estático.

Um método estático não recebe um primeiro argumento implícito. Para declarar um método estático, use este idioma:

```
class C:
    @staticmethod
    def f(arg1, arg2, argN): ...
```

A forma `@staticmethod` é uma função de *decorador* – veja function para detalhes.

Um método estático pode ser chamado na classe (tal como `C.f()`) ou em uma instância (tal como `C().f()`). Além disso, o *descritor* de método estático também é um chamável, então ele pode ser usado na definição de classe (como `f()`).

Métodos estáticos em Python são similares àqueles encontrados em Java ou C++. Veja também *classmethod()* para uma variante útil na criação de construtores de classe alternativos.

Como todos os decoradores, também é possível chamar `staticmethod` como uma função regular e fazer algo com seu resultado. Isso é necessário em alguns casos em que você precisa de uma referência para uma função de um corpo de classe e deseja evitar a transformação automática em método de instância. Para esses casos, use este idioma:

```
def função_comum():
    ...

class C:
    método = staticmethod(função_comum)
```

Para mais informações sobre métodos estáticos, consulte types.

Alterado na versão 3.10: Métodos estáticos agora herdam os atributos do método (`__module__`, `__name__`, `__qualname__`, `__doc__` e `__annotations__`), têm um novo atributo `__wrapped__` e são agora chamáveis como funções regulares.

class str(*, *encoding='utf-8'*, *errors='strict'*)

class str(*object*)

class str(*object*, *encoding*, *errors='strict'*)

class str(*object*, *, *errors*)

Retorna uma versão *str* de *object*. Consulte *str()* para detalhes.

`str` é uma *classe* de string embutida. Para informações gerais sobre strings, consulte *Tipo sequência de texto — str*.

sum(*iterable*, */*, *start=0*)

Soma *start* e os itens de um *iterable* da esquerda para a direita e retornam o total. Os itens do *iterable* são normalmente números e o valor inicial não pode ser uma string.

Para alguns casos de uso, existem boas alternativas para *sum()*. A maneira rápida e preferida de concatenar uma sequência de strings é chamando `''.join(sequence)`. Para adicionar valores de ponto flutuante com precisão estendida, consulte *math.fsum()*. Para concatenar uma série de iteráveis, considere usar *itertools.chain()*.

Alterado na versão 3.8: O parâmetro *start* pode ser especificado como um argumento nomeado.

Alterado na versão 3.12: A soma dos pontos flutuantes foi alterada para um algoritmo que oferece maior precisão e melhor comutatividade na maioria das compilações.

Alterado na versão 3.14: Adicionada especialização para soma de complexos, usando o mesmo algoritmo para soma de flutuantes.

class super

class super(*type*, *object_or_type=None*, */*)

Retorna um objeto proxy que delega chamadas de método a uma classe pai ou irmão do *type*. Isso é útil para acessar métodos herdados que foram substituídos em uma classe.

O *object_or_type* determina a *ordem de resolução de métodos* a ser pesquisada. A pesquisa inicia a partir da classe logo após o *type*.

Por exemplo, se `__mro__` de *object_or_type* é `D -> B -> C -> A -> object` e o valor de *type* é `B`, então *super()* procura por `C -> A -> object`.

O atributo `__mro__` da classe correspondente ao *object_or_type* lista a ordem de pesquisa de resolução de método usada por *getattr()* e *super()*. O atributo é dinâmico e pode mudar sempre que a hierarquia da herança é atualizada.

Se o segundo argumento for omitido, o objeto super retornado é desacoplado. Se o segundo argumento é um objeto, `isinstance(obj, type)` deve ser verdadeiro. Se o segundo argumento é um tipo, `issubclass(type2, type)` deve ser verdadeiro (isto é útil para classmethods).

Quando chamado diretamente de dentro de um método comum de uma classe, pode-se omitir ambos os argumentos ("`super()` de zero argumentos"). Neste caso, *type* será a classe em questão, e *obj* será o primeiro argumento da função que imediatamente chamou `super()` (geralmente o `self`). (Isso significa que o `super()` de zero argumentos não vai funcionar como esperado em funções aninhadas, incluindo expressões geradoras, que criam funções aninhadas implicitamente.)

Existem dois casos de uso típicos para *super*. Em uma hierarquia de classes com herança única, *super* pode ser usado para se referir a classes base sem nomeá-las explicitamente, tornando o código mais sustentável. Esse uso é paralelo ao uso de *super* em outras linguagens de programação.

O segundo caso de uso é oferecer suporte à herança múltipla cooperativa em um ambiente de execução dinâmica. Esse caso de uso é exclusivo do Python e não é encontrado em idiomas ou linguagens compiladas estaticamente que suportam apenas herança única. Isso torna possível implementar "diagramas em losango", onde várias classes base implementam o mesmo método. Um bom design exige que tais implementações tenham a mesma assinatura de chamada em todos os casos (porque a ordem das chamadas é determinada em tempo de execução, porque essa ordem se adapta às alterações na hierarquia de classes e porque essa ordem pode incluir classes de irmãos desconhecidas antes do tempo de execução).

Nos dois casos de uso, uma chamada típica de superclasse se parece com isso:

```
class C(B):
    def método(self, arg):
        super().método(arg)     # Isso faz o mesmo que:
                                # super(C, self).método(arg)
```

Além das pesquisas de método, *super()* também funciona para pesquisas de atributo. Um possível caso de uso para isso é chamar *descritores* em uma classe pai ou irmã.

Observe que *super()* é implementada como parte do processo de vinculação para procura explícita de atributos com ponto, tal como `super().__getitem__(nome)`. Ela faz isso implementando seu próprio método `__getattribute__()` para pesquisar classes em uma ordem predizível que possui suporte a herança múltipla cooperativa. Logo, *super()* não é definida para procuras implícitas usando instruções ou operadores como `super()[name]`.

Observe também que, além da forma de argumento zero, *super()* não se limita ao uso de métodos internos. O formulário de dois argumentos especifica exatamente os argumentos e faz as referências apropriadas. O formulário de argumento zero funciona apenas dentro de uma definição de classe, pois o compilador preenche os detalhes necessários para recuperar corretamente a classe que está sendo definida, além de acessar a instância atual para métodos comuns.

Para sugestões práticas sobre como projetar classes cooperativas usando *super()*, consulte o guia para uso de super().

Alterado na versão 3.14: os objetos *super* agora são *serializáveis* e *copiáveis*.

class tuple(*iterable=()*, */*)

Ao invés de ser uma função, *tuple* é na verdade um tipo de sequência imutável, conforme documentado em *Tuplas* e *Tipos sequências — list, tuple, range*.

class type(*object*, */*)

class type(*name*, *bases*, *dict*, */*, ***kwargs*)

Com um argumento, retorna o tipo de um *object*. O valor de retorno é um tipo de objeto e geralmente o mesmo objeto retornado por `object.__class__`.

A função embutida *isinstance()* é recomendada para testar o tipo de um objeto, porque ela leva sub-classes em consideração.

Com três argumentos, retorna um novo objeto type. Esta é essencialmente a forma dinâmica da instrução `class`. A string *name* é o nome da classe e se torna o atributo `__name__`. A tupla *bases* contém as classes bases e se torna o atributo `__bases__`; se vazio, *object*, a base final de todas as classes é adicionada. O dicionário *dict* contém definições de atributo e método para o corpo da classe; ele pode ser copiado ou envolto antes de se tornar o atributo `__dict__`. As duas instruções a seguir criam objetos `type` idênticos:

```
>>> class X:
...     a = 1
...
>>> X = type('X', (), dict(a=1))
```

Veja também:

- Documentação sobre atributos e métodos em classes.
- *Objetos tipo*

Argumentos nomeados fornecidos para a forma de três argumentos são passados para a máquina metaclasse apropriada (geralmente `__init_subclass__()`) da mesma forma que palavras-chave em uma definição de classe (além de *metaclasse*) fariam.

Veja também class-customization.

Alterado na versão 3.6: Subclasses de `type` que não substituem `type.__new__` não podem mais usar a forma com apenas um argumento para obter o tipo de um objeto.

vars()

vars(*object*, /)

Retorna o atributo `__dict__` para um módulo, classe, instância, or qualquer outro objeto com um atributo `__dict__`.

Objetos como modelos e instâncias têm um atributo atualizável `__dict__`; porém, outros projetos podem ter restrições de escrita em seus atributos `__dict__` (por exemplo, classes usam um *types.MappingProxyType* para prevenir atualizações diretas a dicionário).

Sem nenhum argumento, *vars()* age como *locals()*.

Uma exceção *TypeError* é levantada se um objeto é especificado, mas ela não tem um atributo `__dict__` (por exemplo, se sua classe define o atributo `__slots__`).

Alterado na versão 3.13: O resultado de chamar esta função sem um argumento foi atualizado conforme descrito na embutida *locals()*.

zip(**iterables*, *strict=False*)

Itera sobre vários iteráveis em paralelo, produzindo tuplas com um item de cada um.

Exemplo:

```
>>> for item in zip([1, 2, 3], ['açúcar', 'tempero', 'tudo que há de bom']):
...     print(item)
...
(1, 'açúcar')
(2, 'tempero')
(3, 'tudo que há de bom')
```

Mais formalmente: *zip()* retorna um iterador de tuplas, onde a *i*-ésima tupla contém o *i*-ésimo elemento de cada um dos iteráveis do argumento.

Outra maneira de pensar em *zip()* é que ela transforma linhas em colunas e colunas em linhas. Isso é semelhante a transpor uma matriz.

zip() é preguiçoso: Os elementos não serão processados até que o iterável seja iterado. Por exemplo, por um loop `for` ou por um *list*.

Uma coisa a considerar é que os iteráveis passados para *zip()* podem ter comprimentos diferentes; às vezes por design e às vezes por causa de um bug no código que preparou esses iteráveis. Python oferece três abordagens diferentes para lidar com esse problema:

- Por padrão, *zip()* para quando o iterável mais curto se esgota. Ele irá ignorar os itens restantes nos iteráveis mais longos, cortando o resultado para o comprimento do iterável mais curto:

```
>>> list(zip(range(3), ['fi', 'fa', 'fo', 'fum']))
[(0, 'fi'), (1, 'fa'), (2, 'fo')]
```

- *zip()* é frequentemente usado em casos onde os iteráveis são considerados de tamanho igual. Nesses casos, é recomendado usar a opção `strict=True`. Sua saída é a mesma do *zip()*:: normal

```
>>> list(zip(('a', 'b', 'c'), (1, 2, 3), strict=True))
[('a', 1), ('b', 2), ('c', 3)]
```

  Ao contrário do comportamento padrão, ele levanta uma exceção *ValueError* se um iterável for esgotado antes dos outros:

```
>>> for item in zip(range(3), ['fee', 'fi', 'fo', 'fum'], strict=True):
...     print(item)
...
(0, 'fee')
(1, 'fi')
(2, 'fo')
Traceback (most recent call last):
  ...
ValueError: zip() argument 2 is longer than argument 1
```

  Sem o argumento `strict=True`, qualquer bug que resulte em iteráveis de diferentes comprimentos será silenciado, possivelmente se manifestando como um bug difícil de encontrar em outra parte do programa.

- Iteráveis mais curtos podem ser preenchidos com um valor constante para fazer com que todos os iteráveis tenham o mesmo comprimento. Isso é feito por *itertools.zip_longest()*.

Casos extremos: Com um único argumento iterável, *zip()* retorna um iterador de tuplas de um elemento. Sem argumentos, retorna um iterador vazio.

Dicas e truques:

- A ordem de avaliação da esquerda para a direita dos iteráveis é garantida. Isso torna possível um idioma para agrupar uma série de dados em grupos de comprimento n usando `zip(*[iter(s)]*n, strict=True)`. Isso repete o *mesmo* iterador `n` vezes para que cada tupla de saída tenha o resultado de chamadas `n` para o iterador. Isso tem o efeito de dividir a entrada em pedaços de n comprimentos.

- *zip()* em conjunto com o operador `*` pode ser usado para descompactar uma lista:

```
>>> x = [1, 2, 3]
>>> y = [4, 5, 6]
>>> list(zip(x, y))
[(1, 4), (2, 5), (3, 6)]
>>> x2, y2 = zip(*zip(x, y))
>>> x == list(x2) and y == list(y2)
True
```

Alterado na versão 3.10: Adicionado o argumento `strict`.

__import__(*name*, *globals=None*, *locals=None*, *fromlist=()*, *level=0*)

> **Nota**
>
> Esta é uma função avançada que não é necessária na programação diária do Python, ao contrário de *importlib.import_module()*.

Esta função é chamada pela instrução `import`. Ela pode ser substituída (importando o módulo *builtins* e atribuindo a `builtins.__import__`) para alterar a semântica da instrução `import`, mas isso é **fortemente** desencorajado, pois geralmente é mais simples usar ganchos de importação (consulte **PEP 302**) para atingir os mesmos objetivos e não causa problemas com o código que pressupõe que a implementação de importação padrão esteja em uso. O uso direto de *__import__()* também é desencorajado em favor de *importlib.import_module()*.

A função importa o módulo *name*, potencialmente usando os dados *globals* e *locals* para determinar como interpretar o nome em um contexto de pacote. O *fromlist* fornece os nomes de objetos ou submódulos que devem ser importados do módulo, fornecidos por *name*. A implementação padrão não usa seu argumento *locals* e usa seus *globals* apenas para determinar o contexto do pacote da instrução `import`.

level especifica se é necessário usar importações absolutas ou relativas. `0` (o padrão) significa apenas realizar importações absolutas. Valores positivos para *level* indicam o número de diretórios pai a serem pesquisados em relação ao diretório do módulo que chama *__import__()* (consulte **PEP 328** para obter detalhes).

Quando a variável *name* está no formato `pacote.módulo`, normalmente, o pacote de nível superior (o nome até o primeiro ponto) é retornado, *não* o módulo nomeado por *name*. No entanto, quando um argumento *fromlist* não vazio é fornecido, o módulo nomeado por *name* é retornado.

Por exemplo, a instrução `import spam` resulta em bytecode semelhante ao seguinte código:

```
spam = __import__('spam', globals(), locals(), [], 0)
```

A instrução `import spam.presunto` resulta nesta chamada:

```
spam = __import__('spam.presunto', globals(), locals(), [], 0)
```

Observe como *__import__()* retorna o módulo de nível superior aqui, porque este é o objeto vinculado a um nome pela instrução `import`.

Por outro lado, a instrução `from spam.presunto import ovos, salsicha as sals` resulta em

```
_temp = __import__('spam.presunto', globals(), locals(), ['ovos', 'salsicha'],␣
↪0)
ovos = _temp.ovos
sals = _temp.salsicha
```

Aqui, o módulo `spam.ham` é retornado de *__import__()*. A partir desse objeto, os nomes a serem importados são recuperados e atribuídos aos seus respectivos nomes.

Se você simplesmente deseja importar um módulo (potencialmente dentro de um pacote) pelo nome, use *importlib.import_module()*.

Alterado na versão 3.3: Valores negativos para *level* não são mais suportados (o que também altera o valor padrão para 0).

Alterado na versão 3.9: Quando as opções de linha de comando `-E` ou `-I` estão sendo usadas, a variável de ambiente `PYTHONCASEOK` é agora ignorada.

CAPÍTULO 3

Constantes embutidas

Um pequeno número de constantes são definidas no espaço de nomes embutido da linguagem. São elas:

`False`

O valor falso do tipo *`bool`*. As atribuições a `False` são ilegais e levantam *`SyntaxError`*.

`True`

O valor verdadeiro do tipo *`bool`*. As atribuições a `True` são ilegais e levantam *`SyntaxError`*.

`None`

Um objeto frequentemente usado para representar a ausência de um valor, como quando os argumentos padrões não são passados para uma função. As atribuições a `None` são ilegais e levantam *`SyntaxError`*. `None` é a única instância do tipo *`NoneType`*.

`NotImplemented`

Um valor especial que deve ser retornado pelos métodos binários especiais (por exemplo: `__eq__()`, `__lt__()`, `__add__()`, `__rsub__()`, etc.) não é implementado em relação ao outro tipo; pode ser retornado pelos métodos especiais binários no local (por exemplo: `__imul__()`, `__iand__()`, etc.) para o mesmo propósito. Ele não deve ser avaliado em um contexto booleano. `NotImplemented` é a única instância do tipo *`types.NotImplementedType`*.

> **Nota**
>
> Quando um método binário (ou local) retorna `NotImplemented`, o interpretador tentará a operação refletida no outro tipo (ou algum outro fallback, dependendo do operador). Se todas as tentativas retornarem `NotImplemented`, o interpretador levantará uma exceção apropriada. Retornar incorretamente `NotImplemented` resultará em uma mensagem de erro enganosa ou no valor `NotImplemented` sendo retornado ao código Python.
>
> Consulte *Implementando as operações aritméticas* para ver exemplos.

> **Cuidado**
>
> `NotImplemented` e `NotImplementedError` não são intercambiáveis. Esta constante deve ser usada somente conforme descrito acima; veja *`NotImplementedError`* para detalhes sobre o uso correto da exceção.

Alterado na versão 3.9: A avaliação de `NotImplemented` em um contexto booleano foi descontinuada.

Alterado na versão 3.14: A avaliação de `NotImplemented` em um contexto booleano agora levanta um *`TypeError`*. Anteriormente, ele era avaliado como *`True`* e emitia um *`DeprecationWarning`* desde o Python 3.9.

`Ellipsis`

O mesmo que o literal de reticências "`...`", um objeto frequentemente usado para indicar que algo foi omitido. A atribuição a `Ellipsis` é possível, mas a atribuição a `...` levanta um *`SyntaxError`*. `Ellipsis` é a única instância do tipo *`types.EllipsisType`*.

`__debug__`

Esta constante é verdadeira se o Python não foi iniciado com uma opção `-O`. Veja também a instrução `assert`.

Nota

Os nomes *`None`*, *`False`*, *`True`* e *`__debug__`* não podem ser reatribuídos (atribuições a eles, mesmo como um nome de atributo, levantam *`SyntaxError`*), para que possam ser consideradas "verdadeiras" constantes.

3.1 Constantes adicionadas pelo módulo `site`

O módulo *`site`* (que é importado automaticamente durante a inicialização, exceto se a opção de linha de comando `-S` for fornecida) adiciona várias constantes ao espaço de nomes embutido. Eles são úteis para o console do interpretador interativo e não devem ser usados em programas.

`quit`(*code=None*)

`exit`(*code=None*)

Objetos que, quando exibidos, exibem uma mensagem como "Use quit() or Ctrl-D (i.e. EOF) to exit" e, quando acessados diretamente no interpretador interativo ou chamados como funções, levantam *`SystemExit`* com o código de saída especificado.

`help`

Objeto que, quando exibido, exibe a mensagem "Type help() for interactive help, or help(object) for help about object." e, quando acessado diretamente no interpretador interativo, invoca o sistema de ajuda embutido (veja *`help()`*).

`copyright`

`credits`

Objetos que ao serem impressos ou chamados, imprimem o texto dos direitos autorais ou créditos, respectivamente.

`license`

Objeto que, quando impresso, imprime a mensagem "Type license() to see the full license text" e, quando chamado, exibe o texto completo da licença de maneira semelhante a um paginador (uma tela por vez).

CAPÍTULO 4

Tipos embutidos

As seções a seguir descrevem os tipos padrão que são embutidos ao interpretador.

Os principais tipos embutidos são numéricos, sequências, mapeamentos, classes, instâncias e exceções.

Algumas classes de coleção são mutáveis. Os métodos que adicionam, subtraem ou reorganizam seus membros no lugar, e não retornam um item específico, nunca retornam a instância da coleção propriamente dita, mas um `None`.

Algumas operações são suportadas por vários tipos de objetos; em particular, praticamente todos os objetos podem ser comparados em termos de igualdade, testados quanto ao valor verdade e convertidos em uma string (com a função *`repr()`* ou a função ligeiramente diferente *`str()`*). A última função é implicitamente usada quando um objeto é escrito pela função *`print()`*.

4.1 Teste do valor verdade

Qualquer objeto pode ser testado quanto ao valor verdade, para uso em uma condição `if` ou `while` ou como operando das operações booleanas abaixo.

Por padrão, um objeto é considerado verdadeiro, a menos que sua a classe defina um método `__bool__()` que retorne `False` ou um método `__len__()` que retorna zero, quando chamado com o objeto.[1] Aqui estão a maioria dos objetos embutidos considerados falsos:

- constantes definidas para serem falsas: `None` e `False`
- zero de qualquer tipo numérico: `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0, 1)`
- sequências e coleções vazias: `''`, `()`, `[]`, `{}`, `set()`, `range(0)`

Operações e funções embutidas que têm um resultado Booleano retornam `0` ou `False` para falso e `1` ou `True` para verdadeiro, salvo indicações ao contrário. (Exceção importante: as operações Booleanas `or` e `and` sempre retornam um de seus operandos.)

4.2 Operações booleanas — `and`, `or`, `not`

Esses são as operações booleanas, ordenados por prioridade ascendente:

[1] Informações adicionais sobre esses métodos especiais podem ser encontradas no Manual de Referência do Python (Customização básica).

Operação	Resultado	Notas
`x or y`	se *x* é true, então *x*, do contrário *y*	(1)
`x and y`	se *x* é falso, então *x*, do contrário *y*	(2)
`not x`	se *x* é falso, então `True`, caso contrário `False`	(3)

Notas:

(1) Esse é um operador de curto-circuito, por isso só avalia o segundo argumento se o primeiro é falso.

(2) Este é um operador de curto-circuito, por isso só avalia o segundo argumento se o primeiro é verdadeiro.

(3) `not` tem uma prioridade mais baixa do que operadores não booleanos, então `not a == b` é interpretado como `not (a == b)` e `a == not b` é um erro de sintaxe.

4.3 Comparações

Há oito operadores comparativos no Python. Todos eles possuem a mesma prioridade (que é maior do que aquela das operações Booleanas). Comparações podem ser encadeadas arbitrariamente; por exemplo, `x < y <= z` é equivalente a `x < y and y <= z`, exceto que *y* é avaliado apenas uma vez (porém em ambos os casos *z* não é avaliado de todo quando `x < y` é sabido ser falso).

Esta tabela resume as operações de comparação:

Operação	Significado
`<`	estritamente menor que
`<=`	menor que ou igual
`>`	estritamente maior que
`>=`	maior que ou igual
`==`	igual
`!=`	não é igual
`is`	identidade do objeto
`is not`	identidade de objeto negada

Objetos de tipos diferentes, exceto tipos numéricos diferentes, nunca comparam iguais. O operador `==` é sempre definido, mas para alguns tipos de objetos (por exemplo, objetos de classe) é equivalente a `is`. Os operadores `<`, `<=`, `>` e `>=` são definidos apenas onde fazem sentido; por exemplo, eles levantam uma exceção *`TypeError`* quando um dos argumentos é um número complexo.

Instâncias não idênticas de uma classe normalmente comparam-se como desiguais ao menos que a classe defina o método `__eq__()`.

Instâncias de uma classe não podem ser ordenadas com respeito a outras instâncias da mesma classe, ou outros tipos de objeto, ao menos que a classe defina o suficiente dos métodos `__lt__()`, `__le__()`, `__gt__()` e `__ge__()` (no geral, `__lt__()` e `__eq__()` são suficientes, se você quiser o significado convencional dos operadores de comparação).

O comportamento dos operadores `is` e `is not` não pode ser personalizado; além disso eles podem ser aplicados a quaisquer dois objetos e nunca levantam uma exceção.

Mais duas operações com a mesma prioridade sintática, `in` e `not in`, são suportadas por tipos que são *iteráveis* ou implementam o método `__contains__()`.

4.4 Tipos numéricos — `int`, `float`, `complex`

Existem três tipos numéricos distintos: *inteiros*, *números de ponto flutuante* e *números complexos*. Além disso, os booleanos são um subtipo de números inteiros. Inteiros têm precisão ilimitada. Números de ponto flutuante são geralmente implementados usando `double` em C; informações sobre a precisão e representação interna dos números de

ponto flutuante para a máquina na qual seu programa está sendo executado estão disponíveis em *sys.float_info*. Números complexos têm uma parte real e imaginária, cada um com um número de ponto flutuante. Para extrair essas partes de um número complexo *z*, use `z.real` e `z.imag`. (A biblioteca padrão inclui os tipos numéricos adicionais *fractions.Fraction*, para racionais, e *decimal.Decimal*, para números de ponto flutuante com precisão definida pelo usuário.)

Números são criados por literais numéricos ou como resultado de operadores e funções embutidas. Integrais literais planos (incluindo números hexadecimais, octais e binários) culminam em integrais. Literais numéricos contendo um ponto decimal ou um sinal exponencial resultam em números de ponto flutuante. Anexando `'j'` ou `'J'` para um literal numérico resulta em um número imaginário (um número complexo com uma parte real zero) com a qual você pode adicionar a um integral ou flutuante para receber um número complexo com partes reais e imaginárias.

Os construtores *int()*, *float()*, e *complex()* podem ser usados para produzir números de um tipo específico.

Python oferece suporte completo a aritmética mista: quando um operador de aritmética binária tem operandos de tipos numéricos diferentes, o operando com o tipo "mais estreito" é ampliado para o tipo do outro operando, onde um inteiro é mais estreito do que um ponto flutuante. A aritmética com operandos complexos e reais é definida pela fórmula matemática usual, por exemplo:

```
x + complex(u, v) = complex(x + u, v)
x * complex(u, v) = complex(x * u, x * v)
```

Uma comparação entre números de tipos diferentes se comporta como se os valores exatos desses números estivessem sendo comparados.[2]

Todos os tipos numéricos (exceto complexos) suportam as seguintes operações (para prioridades das operações, consulte operator-summary):

Operação	Resultado	Notas	Documentação completa
`x + y`	soma de *x* e *y*		
`x - y`	diferença de *x* e *y*		
`x * y`	produto de *x* e *y*		
`x / y`	quociente de *x* e *y*		
`x // y`	piso do quociente de *x* e *y*	(1)(2)	
`x % y`	restante de `x / y`	(2)	
`-x`	*x* negado		
`+x`	*x* inalterado		
`abs(x)`	valor absoluto ou magnitude de *x*		*abs()*
`int(x)`	*x* convertido em inteiro	(3)(6)	*int()*
`float(x)`	*x* convertido em ponto flutuante	(4)(6)	*float()*
`complex(re, im)`	um número complexo com parte real *re*, parte imaginária *im*. *im* tem como padrão zero.	(6)	*complex()*
`c.conjugate()`	conjugado do número complexo *c*		
`divmod(x, y)`	o par `(x // y, x % y)`	(2)	*divmod()*
`pow(x, y)`	*x* elevado a *y*	(5)	*pow()*
`x ** y`	*x* elevado a *y*	(5)	

Notas:

(1) Também referido como uma divisão inteira. Para operandos do tipo *int*, o resultado tem o tipo *int*; Para operandos do tipo *float*, o resultado tem o tipo *float*. Em geral o resultado é inteiro como todo, apesar do tipo do resultado não necessariamente ser *int*. O resultado é sempre arredondado para menos infinito: `1//2` é `0`, `(-1)//2` é `-1`, `1//(-2)` é `-1` e `(-1)//(-2)` é `0`.

(2) Não para números complexos. Ao invés disso, converte para pontos flutuantes usando *abs()* se for apropriado.

[2] Como uma consequência, a lista `[1, 2]` é considerada igual a `[1.0, 2.0]`, e similarmente para tuplas.

(3) Conversão de *float* para *int* trunca, descartando a parte fracionária. Veja as funções *math.floor()* e *math.ceil()* para conversões alternativas.

(4) ponto flutuante também aceita a string "nan" e "inf" com um prefixo opcional "+" ou "-" a Não é um Número (NaN) e infinidade positiva ou negativa.

(5) Python define `pow(0, 0)` e `0 ** 0` sendo `1`, como é comum para linguagens de programação.

(6) Os literais numéricos aceitos incluem os dígitos de `0` a `9` ou qualquer equivalente Unicode (pontos de código com a propriedade `Nd`).

Veja o Unicode Standard para obter uma lista completa de pontos de código com a propriedade `Nd`.

Todos os tipos *numbers.Real* (*int* e *float*) também incluem as seguintes operações.

Operação	Resultado
math.trunc(x)	*x* truncado para *Integral*
round(x[, n])	*x* arredondado para *n* dígitos, arredondando metade para igualar. Se *n* é omitido, ele toma o padrão de 0.
math.floor(x)	o maior *Integral* <= *x*
math.ceil(x)	pelo menos *Integral* >= *x*

Para operações numéricas adicionais, consulte os módulos *math* e *cmath*.

4.4.1 Operações de bits em tipos inteiros

Operações bit a bit só fazem sentido para números inteiros. O resultado de operações bit a bit é calculado como se fosse realizado no complemento de dois com um número infinito de bits de sinal.

As prioridades das operações bit a bit binárias são todas menores do que as operações numéricas e maiores que as comparações; a operação unária `~` tem a mesma prioridade que as outras operações numéricas unárias (`+` e `-`).

Esta tabela lista as operações de bits classificadas em prioridade ascendente:

Operação	Resultado	Notas
`x | y`	*ou* bit a bit de *x* e *y*	(4)
`x ^ y`	*ou exclusivo* bit a bit de *x* e *y*	(4)
`x & y`	*e* bit a bit de *x* e *y*	(4)
`x << n`	*x* deslocado para a esquerda por *n* bits	(1)(2)
`x >> n`	*x* deslocado para a direita por *n* bits	(1)(3)
`~x`	os bits de *x* invertidos	

Notas:

(1) Contagens de deslocamento negativo são ilegais e levantam uma *ValueError*.

(2) Um deslocamento à esquerda por *n* bits é equivalente à multiplicação por `pow(2, n)`.

(3) Um deslocamento à direita por *n* bits é equivalente à divisão pelo piso por `pow(2, n)`.

(4) Executar esses cálculos com pelo menos um bit extra de extensão de sinal na representação de complemento de dois finitos (uma largura de bit de trabalho `1 + max(x.bit_length(), y.bit_length())` ou mais) é suficiente para obter o mesmo resultado como se houvesse um número infinito de bits de sinal.

4.4.2 Métodos adicionais em tipos inteiros

O tipo int implementa a *classe base abstrata* `numbers.Integral`. Além disso, ele provê mais alguns métodos:

`int.bit_length()`

Retorna o número de bits necessários para representar um inteiro em binário, excluindo o sinal e entrelinha zeros:

```
>>> n = -37
>>> bin(n)
'-0b100101'
>>> n.bit_length()
6
```

Mais precisamente, se `x` for diferente de zero, então `x.bit_length()` é o único positivo inteiro `k` tal que `2**(k-1) <= abs(x) < 2**k`. Equivalentemente, quando `abs(x)` for pequeno o suficiente para ter um logaritmo corretamente arrendondado, então `k = 1 + int(log(abs(x), 2))`. Se `x` é zero, então `x.bit_length()` retorna `0`.

Equivalente a:

```
def bit_length(self):
    s = bin(self)       # representação binária:  bin(-37) --> '-0b100101'
    s = s.lstrip('-0b') # remove zeros no início e sinal negativo
    return len(s)       # len('100101') --> 6
```

Adicionado na versão 3.1.

`int.bit_count()`

Retorna o número de unidades na representação binária do valor absoluto do inteiro. Isso também é conhecido como contagem da população. Exemplo:

```
>>> n = 19
>>> bin(n)
'0b10011'
>>> n.bit_count()
3
>>> (-n).bit_count()
3
```

Equivalente a:

```
def bit_count(self):
    return bin(self).count("1")
```

Adicionado na versão 3.10.

`int.to_bytes`(*length=1*, *byteorder='big'*, ***, *signed=False*)

Retorna um vetor de bytes representando um inteiro.

```
>>> (1024).to_bytes(2, byteorder='big')
b'\x04\x00'
>>> (1024).to_bytes(10, byteorder='big')
b'\x00\x00\x00\x00\x00\x00\x00\x00\x04\x00'
>>> (-1024).to_bytes(10, byteorder='big', signed=True)
b'\xff\xff\xff\xff\xff\xff\xff\xff\xfc\x00'
>>> x = 1000
>>> x.to_bytes((x.bit_length() + 7) // 8, byteorder='little')
b'\xe8\x03'
```

O inteiro é representado usando *length* bytes, e o padrão é 1. Uma *OverflowError* é levantada se um inteiro não é representável com o dado número de bytes.

O argumento *byteorder* determina a ordem de bytes usada para representar um inteiro, e o padrão é `"big"`. Se o *byteorder* é `"big"`, o byte mais significativo está no início do vetor de byte. Se *byteorder* é `"little"`, o byte mais significativo está no final do vetor de byte.

O argumento *signed* determina aonde o modo de complemento de dois é usado para representar o inteiro. Se *signed* é `False` e um inteiro negativo é dado, uma *OverflowError* é levantada. O valor padrão para *signed* é `False`.

Os valores padrão podem ser usados para transformar convenientemente um inteiro em um objeto de byte único:

```
>>> (65).to_bytes()
b'A'
```

No entanto, ao usar os argumentos padrão, não tente converter um valor maior que 255 ou você obterá um *OverflowError*.

Equivalente a:

```
def to_bytes(n, length=1, byteorder='big', signed=False):
    if byteorder == 'little':
        order = range(length)
    elif byteorder == 'big':
        order = reversed(range(length))
    else:
        raise ValueError("byteorder must be either 'little' or 'big'")

    return bytes((n >> i*8) & 0xff for i in order)
```

Adicionado na versão 3.2.

Alterado na versão 3.11: Adicionados valores de argumento padrão para `length` e `byteorder`.

classmethod `int.`**from_bytes**(*bytes*, *byteorder='big'*, ***, *signed=False*)

Retorna o inteiro representado pelo vetor de bytes dado.

```
>>> int.from_bytes(b'\x00\x10', byteorder='big')
16
>>> int.from_bytes(b'\x00\x10', byteorder='little')
4096
>>> int.from_bytes(b'\xfc\x00', byteorder='big', signed=True)
-1024
>>> int.from_bytes(b'\xfc\x00', byteorder='big', signed=False)
64512
>>> int.from_bytes([255, 0, 0], byteorder='big')
16711680
```

O argumento *bytes* precisa ou ser um *objeto bytes ou similar* ou um iterador produzindo bytes.

O argumento *byteorder* determina a ordem de bytes usada para representar um inteiro, e o padrão é `"big"`. Se o *byteorder* é `"big"`, o byte mais significativo está no início do vetor de byte. Se *byteorder* é `"little"`, o byte mais significativo está no final do vetor de byte. Para requisitar a ordem nativa de byte do sistema hospedeiro, use *sys.byteorder* como o valor da ordem de byte.

O argumento *signed* indica quando o complemento de dois é usado para representar o inteiro.

Equivalente a:

```
def from_bytes(bytes, byteorder='big', signed=False):
    if byteorder == 'little':
        little_ordered = list(bytes)
    elif byteorder == 'big':
        little_ordered = list(reversed(bytes))
    else:
        raise ValueError("byteorder must be either 'little' or 'big'")

    n = sum(b << i*8 for i, b in enumerate(little_ordered))
    if signed and little_ordered and (little_ordered[-1] & 0x80):
        n -= 1 << 8*len(little_ordered)

    return n
```

Adicionado na versão 3.2.

Alterado na versão 3.11: Adicionado valor de argumento padrão para `byteorder`.

`int.`**`as_integer_ratio`**`()`

Retorna um par de números inteiros cuja razão é igual ao número inteiro original e tem um denominador positivo. A proporção inteira de números inteiros (números inteiros) é sempre o número inteiro como numerador e `1` como denominador.

Adicionado na versão 3.8.

`int.`**`is_integer`**`()`

Retorna `True`. Existe para compatibilidade com tipo pato com *`float.is_integer()`*.

Adicionado na versão 3.12.

4.4.3 Métodos adicionais em ponto flutuante

O tipo float implementa a *classe base abstrata* *`numbers.Real`*. float também possui os seguintes métodos adicionais.

`classmethod` `float.`**`from_number`**`(`*x*`)`

Método de classe para retornar um número de ponto flutuante construído a partir de um número *x*.

Se o argumento é um inteiro ou um número de ponto flutuante, um número de ponto flutuante com o mesmo valor (com a precisão de ponto flutuante de Python) é retornado. Se o argumento está fora do intervalo de um ponto flutuante Python, uma exceção *`OverflowError`* será levantada.

Para um objeto Python genérico `x`, `float.from_number(x)` delega para o método `x.__float__()`. Se `__float__()` não estiver definido, então ele delega para o método `__index__()`.

Adicionado na versão 3.14.

`float.`**`as_integer_ratio`**`()`

Retorna um par de inteiros dos quais a proporção é exatamente igual ao float original. A proporção está em seus termos mais baixos e tem um denominador positivo. Levanta um *`OverflowError`* em infinidades e um *`ValueError`* em NaNs.

`float.`**`is_integer`**`()`

Retorna `True` se a instância do float for finita com o valor integral e `False`, caso contrário:

```
>>> (-2.0).is_integer()
True
>>> (3.2).is_integer()
False
```

Dois métodos suportam conversão para e de strings hexadecimais. Uma vez que os pontos flutuantes do Python são armazenados internamente como números binários, a conversão de um ponto flutuante para ou de uma string

decimal geralmente envolve um pequeno erro de arredondamento. Em contraste, as strings hexadecimais permitem a representação exata e a especificação de números de ponto flutuante. Isso pode ser útil na depuração e no trabalho numérico.

`float.`**`hex`**`()`

Retorna a representação de um número de ponto flutuante como uma string hexadecimal. Para números de ponto flutuante finitos, essa representação vai sempre incluir um `0x` inicial e um `p` final e expoente.

classmethod `float.`**`fromhex`**`(`*s*`)`

Método de classe para retornar um ponto flutuante representado por uma string hexadecimal *s*. A string *s* pode ter espaços em branco iniciais e finais.

Note que *`float.hex()`* é um método de instância, enquanto *`float.fromhex()`* é um método de classe.

Uma string hexadecimal toma a forma:

```
[sign] ['0x'] integer ['.' fraction] ['p' exponent]
```

onde o sinal opcional `sign` pode ser tanto `+` or `-`, `integer` e `fraction` são strings de dígitos hexadecimais, e `exponent` é um inteiro decimal com um sinal opcional no início. Maiúsculo ou minúsculo não é significativo, e deve haver ao menos um dígito hexadecimal tanto no inteiro ou na fração. Essa sintaxe é similar à sintaxe especificada na seção 6.4.4.2 do padrão C99, e também do da sintaxe usada no Java 1.5 em diante. Em particular, a saída de *`float.hex()`* é usável como um literal hexadecimal de ponto flutuante em código C ou Java, e strings hexadecimais produzidas pelo formato do caractere `%a` do C ou `Double.toHexString` do Java são aceitos pelo *`float.fromhex()`*.

Note que o expoente é escrito em decimal ao invés de hexadecimal, e que ele dá a potência de 2 pela qual se multiplica o coeficiente. Por exemplo, a string hexadecimal `0x3.a7p10` representa o número de ponto flutuante `(3 + 10./16 + 7./16**2) * 2.0**10` ou `3740.0`:

```
>>> float.fromhex('0x3.a7p10')
3740.0
```

Aplicando a conversão inversa a `3740.0` retorna uma string hexadecimal diferente representada pelo mesmo número:

```
>>> float.hex(3740.0)
'0x1.d380000000000p+11'
```

4.4.4 Métodos adicionais em números complexos

O tipo `complex` implementa a *classe base abstrata* *`numbers.Complex`*. `complex` também possui os seguintes métodos adicionais.

classmethod `complex.`**`from_number`**`(`*x*`)`

Método de classe para converter um número para um número complexo.

Para um objeto Python `x` qualquer, `complex.from_number(x)` delega para `x.__complex__()`. Se `__complex__()` não está definido então a chamada é repassada para `__float__()`. Se `__float__()` não está definido então a chamada é, novamente, repassada para `__index__()`.

Adicionado na versão 3.14.

4.4.5 Hashing de tipos numéricos

Para números `x` e `y`, possivelmente de diferentes tipos, é um requisito que `hash(x) == hash(y)` sempre que `x == y` (veja a documentação do método `__hash__()` para mais detalhes). Para facilitar a implementação e eficiência através de uma variedade de tipos numéricos (incluindo *`int`*, *`float`*, *`decimal.Decimal`* e *`fractions.Fraction`*), o hash do Python para tipos numéricos é baseado em uma única função matemática que é definida para qualquer número racional e, portanto, se aplica para todas as instâncias de *`int`* e *`fractions.Fraction`*, e todas as instâncias finitas das classes *`float`* e *`decimal.Decimal`*. Essencialmente, essa função é dada pelo módulo de redução `P` para um primo fixado `P`. O valor de `P` é disponibilizado ao Python como um atributo *`modulus`* do *`sys.hash_info`*.

Atualmente, o primo usado é `P = 2**31 - 1` em máquinas com longos de 32 bits do C e `P = 2**61 - 1` em máquinas com longos de 64 bits do C.

Aqui estão as regras em detalhe:

- Se `x = m / n` é um número racional não negativo e `n` não é divisível por `P`, define `hash(x)` como `m * invmod(n, P) % P`, onde `invmod(n, P)` retorna o inverso do módulo de `n` com `P`.
- Se `x = m / n` é um número racional não negativo e `n` é divisível por `P` (porém `m` não é) então `n` não possui modulo `P` inverso e a regra acima não se aplica; nesse caso defina `hash(x)` para ser o valor constante `sys.hash_info.inf`.
- Se `x = m / n` é um número racional negativo, defina `hash(x)` como `-hash(-x)`. Se a hash resultante é `-1`, a substituo com `-2`.
- Os valores particulares `sys.hash_info.inf` e `-sys.hash_info.inf` são usados como valores de hash para infinidade positiva ou infinidade negativa (respectivamente).
- Para número *complex* `z`, o valor da hash do número real partes imaginárias são combinados calculando `hash(z.real) + sys.hash_info.imag * hash(z.imag)`, módulo reduzido `2**sys.hash_info.width` de modo que isto permaneça em `range(-2**(sys.hash_info.width - 1), 2**(sys.hash_info.width - 1))`. Novamente, se o resultado é `-1`, ele é substituído por `-2`.

Para esclarecer as regras acima, aqui estão alguns exemplos de código em Python, equivalentes ao hash embutido, para calcular o hash de números racionais, *float* ou *complex*:

```
import sys, math

def hash_fraction(m, n):
    """Calcula o hash de um número racional m / n.

    Presume que m e n são inteiros, com n positivo.
    Equivalente a hash(fractions.Fraction(m, n)).

    """
    P = sys.hash_info.modulus
    # Remove fatores comuns de P.  (Desnecessário se m e n são primos entre si.)
    while m % P == n % P == 0:
        m, n = m // P, n // P

    if n % P == 0:
        hash_value = sys.hash_info.inf
    else:
        # Pequeno teorema de Fermat: pow(n, P-1, P) é 1, então
        # pow(n, P-2, P) dá o inverso de n módulo P.
        hash_value = (abs(m) % P) * pow(n, P - 2, P) % P
    if m < 0:
        hash_value = -hash_value
    if hash_value == -1:
        hash_value = -2
    return hash_value

def hash_float(x):
    """Calcula o hash de um ponto flutuante x."""

    if math.isnan(x):
        return object.__hash__(x)
    elif math.isinf(x):
        return sys.hash_info.inf if x > 0 else -sys.hash_info.inf
    else:
        return hash_fraction(*x.as_integer_ratio())
```

(continua na próxima página)

(continuação da página anterior)

```
def hash_complex(z):
    """Calcula o hash de um número complexo z."""

    hash_value = hash_float(z.real) + sys.hash_info.imag * hash_float(z.imag)
    # faz um módulo para redução de sinal 2**sys.hash_info.width
    M = 2**(sys.hash_info.width - 1)
    hash_value = (hash_value & (M - 1)) - (hash_value & M)
    if hash_value == -1:
        hash_value = -2
    return hash_value
```

4.5 Tipo booleano - `bool`

Booleanos representam valores verdade. O tipo *bool* tem exatamente duas instâncias constantes: `True` e `False`.

A função embutida *bool()* converte qualquer valor em um booleano, se o valor puder ser interpretado como um valor verdade (consultar seção *Teste do valor verdade* acima).

Para operações lógicas, use *operadores booleanos* `and`, `or` e `not`. Ao aplicar operadores bit a bit `&`, `|`, `^` a dois booleanos, o resultado é um booleano equivalente às operações lógicas "e", "ou", "xor". Entretanto, é preferível usar os operadores lógicos `and`, `or` e `!=` em relação aos `&`, `|` e `^`.

Descontinuado desde a versão 3.12: O uso do operador de inversão bit a bit `~` foi descontinuado e levantará um erro no Python 3.16.

bool é uma subclasse *int* (veja *Tipos numéricos — int, float, complex*). Em diversos contextos numéricos, `False` e `True` se comportam como 0 e 1, respectivamente. Entretanto, basear-se somente nisso, é desencorajado; ao invés disso, a conversão deve ser feita explicitamente usando *int()*.

4.6 Tipos iteradores

Python suporta o conceito de iteração por contêineres. Isso é implementado usando dois métodos distintos; estes são usados para permitir classes definidas pelo usuário suportem iteração. Sequências, descritas abaixo em mais detalhes, sempre suportam os métodos de iteração.

Um método necessita ser definido para objetos contêineres afim destes proverem suporte a *iterável*:

`container.__iter__()`

> Retorna um objeto *iterador*. O objeto deve suportar o protocolo iterador descrito abaixo. Se um contêiner suporta diferentes tipos de iterador, métodos adicionais podem ser providenciados para requisitar especificamente iteradores para aqueles tipos de iterações. (Um exemplo de um objeto suportando múltiplas formas de iteração seria uma estrutura em árvore a qual suporta ambas travessias de travessia em largura e em profundidade.) Esse método corresponde ao slot `tp_iter` da estrutura de tipos para objetos Python na API Python/C.

Os próprios objetos iteradores são obrigados a suportarem os dois métodos a seguir, que juntos formam o *protocolo iterador*:

`iterator.__iter__()`

> Retorna o próprio objeto *iterador* em si. Isso é necessário para permitir que ambos os contêineres e iteradores sejam usados com as instruções `for` e `in`. Este método corresponde ao slot `tp_iter` da estrutura de tipos para objetos Python na API Python/C.

`iterator.__next__()`

> Retorna o próximo item do *iterador*. Se não houver itens além, a exceção *StopIteration* é levantada. Esse método corresponde ao slot `tp_iternext` da estrutura de tipos para objetos Python na API Python/C.

Python define diversos objetos iteradores para suportar iterações sobre tipos de sequências gerais e específicas, dicionários, e outras formas mais especializadas. Os tipos específicos não são importantes além de sua implementação do protocolo iterador.

Uma vez que o método *__next__()* do iterador levantou *StopIteration*, ele deve continuar fazendo isso em chamadas subsequentes. Implementações que não obedecem essa propriedade são consideradas quebradas.

4.6.1 Tipos geradores

Os *geradores* do Python proveem uma maneira conveniente para implementar o protocolo iterador. Se o método `__iter__()` de um objeto contêiner é implementado como um gerador, ele irá automaticamente retornar um objeto iterador (tecnicamente, um objeto gerador) fornecendo os métodos *__iter__()* e `__next__()`. Mais informações sobre geradores podem ser encontradas na documentação para a expressão yield.

4.7 Tipos sequências — `list`, `tuple`, `range`

Existem três tipos básicos de sequência: listas, tuplas e objetos range. Tipos de sequência adicionais adaptados para o processamento de *dados binários* e *strings de texto* são descritos em seções dedicadas.

4.7.1 Operações comuns de sequências

As operações nas seguintes tabelas são suportadas pela maioria dos tipos sequências, ambos mutáveis e imutáveis. A classe ABC *collections.abc.Sequence* é fornecida para tornar fácil a correta implementação desses operadores em tipos sequências personalizados.

Essa tabela lista as operações de sequência ordenadas em prioridade ascendente. Na tabela, *s* e *t* são sequências do mesmo tipo, *n*, *i*, *j* e *k* são inteiros e *x* é um objeto arbitrário que atende a qualquer restrição de valor e tipo imposta por *s*.

As operações `in` e `not in` têm as mesmas prioridades que as operações de comparação. As operações `+` (concatenação) e `*` (repetição) têm a mesma prioridade que as operações numéricas correspondentes.[3]

Operação	Resultado	Notas
`x in s`	`True` caso um item de *s* seja igual a *x*, caso contrário `False`	(1)
`x not in s`	`False` caso um item de *s* for igual a *x*, caso contrário `True`	(1)
`s + t`	a concatenação de *s* e *t*	(6)(7)
`s * n` ou `n * s`	equivalente a adicionar *s* a si mesmo *n* vezes	(2)(7)
`s[i]`	*i*-ésimo item de *s*, origem 0	(3)(8)
`s[i:j]`	fatia de *s* de *i* até *j*	(3)(4)
`s[i:j:k]`	fatia de *s* de *i* até *j* com passo *k*	(3)(5)
`len(s)`	comprimento de *s*	
`min(s)`	menor item de *s*	
`max(s)`	maior item de *s*	

Sequências do mesmo tipo também suportam comparações. Em particular, tuplas e listas são comparadas lexicograficamente pela comparação de elementos correspondentes. Isso significa que para comparar igualmente, cada elemento deve comparar igual e as duas sequências devem ser do mesmo tipo e possuírem o mesmo comprimento. (Para detalhes completos, veja comparisons na referência da linguagem.)

Iteradores para frente e reversos sobre sequências mutáveis acessam valores usando um índice. Esse índice continuará avançando (ou retrocedendo) mesmo que a sequência subjacente seja alterada. O iterador termina somente quando um *IndexError* ou um *StopIteration* é encontrado (ou quando o índice cai abaixo de zero).

Notas:

(1) Enquanto as operações `in` e `not in` são usadas somente para testes de contenção simples em modo geral, algumas sequências especializadas (tais como *str*, *bytes* e *bytearray*) também usam-nas para testes de subsequências:

[3] Eles precisam ter, já que o analisador sintático não consegue dizer o tipo dos operandos.

```
>>> "gg" in "eggs"
True
```

(2) Os valores de *n* menos `0` são tratados como `0` (o que produz uma sequência vazia do mesmo tipo que *s*). Observe que os itens na sequência *s* não são copiados; eles são referenciados várias vezes. Isso frequentemente assombra novos programadores Python; considere então que:

```
>>> lists = [[]] * 3
>>> lists
[[], [], []]
>>> lists[0].append(3)
>>> lists
[[3], [3], [3]]
```

O que aconteceu é que `[[]]` é uma lista de um elemento contendo uma lista vazia, então todos os três elementos de `[[]] * 3` são referências a esta única lista vazia. Modificar qualquer um dos elementos de `lists` modifica a lista vazia. Podemos criar uma lista de listas diferentes dessa maneira:

```
>>> lists = [[] for i in range(3)]
>>> lists[0].append(3)
>>> lists[1].append(5)
>>> lists[2].append(7)
>>> lists
[[3], [5], [7]]
```

Mais explicação está disponível no FAQ faq-multidimensional-list.

(3) Se *i* ou *j* for negativo, o índice será relativo ao fim da sequência *s*: `len(s) + i` ou `len(s) + j` será substituído. Mas note que `-0` ainda será `0`.

(4) A fatia *s* de *i* até *j* é definida como a sequência de itens com índice *k* tal que `i <= k < j`. Se *i* ou *j* foi maior do que `len(s)`, usa `len(s)`. Se *i* for omitido ou `None`, use `0`. Se *j* for omitido ou `None`, usa `len(s)`. Se *i* for maior ou igual a *j*, a fatia é vazia.

(5) A fatia *s* de *i* até *j* com passo *k* é definida como sendo a sequência de itens com índice `x = i + n*k` tal que `0 <= n < (j-i)/k`. Em outras palavras, os índices são `i`, `i+k`, `i+2*k`, `i+3*k` e assim por diante, parando quando *j* for atingido (mas nunca incluindo *j*). Quando *k* for positivo, *i* e *j* serão reduzidos a `len(s)` se forem maiores. Quando *k* for negativo, *i* e *j* são reduzidos para `len(s) - 1` se forem maiores. Se *i* ou *j* forem omitidos ou `None`, eles se tornam valores "finais" (cujo final depende de *k*). Nota: *k* não pode ser zero. Se *k* for `None`, o mesmo será tratado como sendo igual a `1`.

(6) Concatenar sequências imutáveis sempre resulta em um novo objeto. Isso significa que a criação de uma sequência por concatenação repetida terá um custo quadrático de tempo de execução no comprimento total da sequência. Para obter um custo de tempo de execução linear, devemos alternar para uma das alternativas abaixo:

- Se estiver concatenando objetos *str*, você pode criar uma lista e usar *str.join()* no final ou então escrever numa instância de *io.StringIO* e recuperar o seu valor ao final
- Se estiver concatenando objetos *bytes*, você também pode usar o método *bytes.join()* ou *io.BytesIO*, ou você pode fazer concatenação interna com um objeto *bytearray*. A classe *bytearray* são objetos mutáveis e possuem um eficiente mecanismo de superalocação
- Se estiver concatenando objetos *tuple*, estenda a classe *list* em vez disso
- para outros tipos, busque na documentação relevante da classe

(7) Alguns tipos sequências (como *range*) apenas suportam sequências de itens que seguem padrões específicos e, portanto, não suportam concatenação ou repetição de sequência.

(8) Uma *IndexError* é levantada se *i* estiver fora do intervalo da sequência.

Métodos de sequência

Tipos de sequência também oferecem suporte aos seguintes métodos:

`sequence.`**`count`**`(`*value*`, /)`

Retorna o número total de ocorrências de *value* em *sequence*.

`sequence.`**`index`**`(`*value*`[,` *start*`[,` *stop*`]])`

Retorna o índice da primeira ocorrência de *value* em *sequence*.

Levanta `ValueError` se *value* não for encontrado em *sequence*.

Os argumentos *start* ou *stop* permitem uma busca eficiente de subseções da sequência, começando em *start* e terminando em *stop*. Isso é aproximadamente equivalente a `start + sequence[start:stop].index(value)`, só que sem copiar nenhum dado.

> **Cuidado**
>
> Nem todos os tipos de sequência oferecem suporte a passagem dos argumentos *start* e *stop*.

4.7.2 Tipos sequência imutáveis

A única operação que os tipos sequências imutáveis geralmente implementam que também não é implementada pelos tipos sequências mutáveis é suporte para a função embutida `hash()`.

Esse suporte permite sequências imutáveis, tais como instâncias da classe `tuple`, serem usadas como chaves de dicionários `dict` e armazenados em instâncias de `set` e de `frozenset`.

A tentativa de obter um hash de uma sequência imutável que contém valores desnecessários resultará em um erro `TypeError`.

4.7.3 Tipos sequências mutáveis

As operações na tabela a seguir são definidas em tipos sequência mutáveis. A ABC `collections.abc.MutableSequence` é fornecida para tornar mais fácil a implementação correta dessas operações em tipos sequências personalizados.

Na tabela *s* é uma instância de um tipo de sequência mutável, *t* é qualquer objeto iterável e *x* é um objeto arbitrário que atende a qualquer restrição de tipo e valor imposto por *s* (por exemplo `bytearray` só aceita inteiros que atendam a restrição de valor `0 <= x <= 255`).

Operação	Resultado	Notas
`s[i] = x`	item *i* de *s* é substituído por *x*	
`del s[i]`	remove o item *i* de *s*	
`s[i:j] = t`	fatias de *s* de *i* até *j* são substituídas pelo conteúdo do iterável *t*	
`del s[i:j]`	remove os elementos de `s[i:j]` da lista (mesmo que `s[i:j] = []`)	
`s[i:j:k] = t`	os elementos de `s[i:j:k]` são substituídos por aqueles de *t*	(1)
`del s[i:j:k]`	remove os elementos de `s[i:j:k]` desde a listas	
`s += t`	estende *s* com o conteúdo de *t* (na maior parte do mesmo `s[len(s):len(s)] = t`)	
`s *= n`	atualiza *s* com o seu conteúdo por *n* vezes	(2)

Notas:

(1) se *k* não for igual a `1`, *t* deve ter o mesmo comprimento que a fatia a qual ele está substituindo.

(2) O valor *n* é um inteiro, ou um objeto implementando `__index__()`. Valores zero e negativos de *n* limparão a sequência. Os itens na sequência não são copiados; eles são referenciados várias vezes, como explicado para `s * n` em *Operações comuns de sequências*.

Métodos de sequências mutáveis

Tipos de sequências mutáveis também oferecem suporte aos seguintes métodos:

`sequence.`**`append`**`(`*value*`, /)`

Adiciona *value* ao final da sequência. Isso equivale a escrever `seq[len(seq):len(seq)] = [value]`.

`sequence.`**`clear`**`()`

Adicionado na versão 3.3.

Remove todos os itens de *sequence*. Isso equivale a escrever `del sequence[:]`.

`sequence.`**`copy`**`()`

Adicionado na versão 3.3.

Cria uma cópia rasa de *sequence*. Isso equivale a escrever `sequence[:]`.

> **Dica**
>
> O método `copy()` não faz parte de *`MutableSequence` ABC*, mas a maioria dos tipos de sequências mutáveis concretas o fornecem.

`sequence.`**`extend`**`(`*iterable*`, /)`

Estende *sequence* com o conteúdo de *iterable*. Em geral, isso equivale a escrever `seq[len(seq):len(seq)] = iterable`.

`sequence.`**`insert`**`(`*index*`, `*value*`, /)`

Insire *value* em *sequence* no *index* fornecido. Isso equivale a escrever `sequence[index:index] = [value]`.

`sequence.`**`pop`**`(`*index=-1*`, /)`

Recupera o item em *index* e também o remove de *sequence*. Por padrão, o último item em *sequence* é removido e retornado.

`sequence.`**`remove`**`(`*value*`, /)`

Remove o primeiro item de *sequence* onde `sequence[i] == value`.

Levanta *`ValueError`* se *value* não for encontrado em *sequence*.

`sequence.`**`reverse`**`()`

Inverte os itens da *sequence* no lugar. Este método mantém a economia de espaço ao inverter uma sequência grande. Para lembrar aos usuários que ele opera por efeito colateral, ele retorna `None`.

4.7.4 Listas

As listas são sequências mutáveis, normalmente usadas para armazenar coleções de itens homogêneos (onde o grau preciso de similaridade variará de acordo com a aplicação).

`class list``(`*iterable=()*`, /)`

As listas podem ser construídas de várias maneiras:

- Usando um par de colchetes para denotar uma lista vazia: `[]`
- Usando colchetes, separando itens por vírgulas: `[a]`, `[a, b, c]`
- Usando uma compreensão de lista: `[x for x in iterable]`
- Usando o construtor de tipo: `list()` ou `list(iterable)`

O construtor produz uma lista cujos itens são iguais e na mesma ordem que os itens de *iterable*. *iterable* pode ser uma sequência, um contêiner que suporte iteração ou um objeto iterador. Se *iterable* já for uma lista, uma cópia será feita e retornada, semelhante a `iterable[:]`. Por exemplo, `list('abc')` retorna `['a', 'b', 'c']` e `list( (1, 2, 3) )` retorna `[1, 2, 3]`. Se nenhum argumento for dado, o construtor criará uma nova lista vazia `[]`.

Muitas outras operações também produzem listas, incluindo a função embutida *sorted()*.

Listas implementam todas as operações de sequências *comuns* e *mutáveis*. As listas também fornecem o seguinte método adicional:

sort(*, *key=None*, *reverse=False*)

Esse método classifica a lista in-place, usando apenas comparações < entre itens. As exceções não são suprimidas – se qualquer operação de comparação falhar, toda a operação de ordenação falhará (e a lista provavelmente será deixada em um estado parcialmente modificado).

sort() aceita 2 argumentos que só podem ser passados como *argumentos somente-nomeados*:

key especifica uma função de um argumento que é usado para extrair uma chave de comparação de cada elemento da lista (por exemplo, `key=str.lower`). A chave correspondente a cada item na lista é calculada uma vez e depois usada para todo o processo de classificação. O valor padrão `None` significa que os itens da lista são classificados diretamente sem calcular um valor de chave separado.

A função utilitária *functools.cmp_to_key()* está disponível para converter a função *cmp* no estilo 2.x para uma função *key*.

reverse é um valor booleano. Se definido igual a `True`, então os elementos da lista são classificados como se cada comparação estivesse invertida.

Este método modifica a sequência in-place para economizar espaço ao classificar uma sequência grande. Para lembrar aos usuários que os mesmos operam por efeito colateral, ele não retorna a sequência ordenada (utilize a função *sorted()* para solicitar explicitamente uma nova instância da lista ordenada).

O método *sort()* é garantido como sendo estável. Uma classificação é estável se ela garantir não alterar a ordem relativa de elementos que comparam igual – isso é útil para classificar em várias passagens (por exemplo, classificar por departamento, depois por nota salarial).

Para exemplos de classificação e um breve tutorial de classificação, veja sortinghowto.

No momento em que uma lista está sendo ordenada, o efeito de tentar alterar, ou mesmo inspecionar, a lista é indefinida. A implementação C do Python faz com que a lista apareça vazia durante o tempo de processamento, e levanta a exceção *ValueError* se puder detectar que a lista foi alterada durante uma ordenação.

4.7.5 Tuplas

Tuplas são sequências imutáveis, tipicamente usadas para armazenar coleções de dados heterogêneos (como as tuplas de 2 elementos produzidas pelo função embutida *enumerate()*). Tuplas também são usadas para casos em que seja necessária uma sequência imutável de dados homogêneos (como permitir o armazenamento em uma instância *set* ou *dict*).

class tuple(*iterable=()*, */*)

As tuplas podem ser construídas de várias maneiras:

- Usando um par de parênteses para denotar a tupla vazia: `()`
- Usando uma vírgula à direita para uma tupla singleton: `a,` ou `(a,)`
- Separando os itens com vírgulas: `a, b, c` ou `(a, b, c)`
- Usando a função embutida *tuple()*: `tuple()` ou `tuple(iterable)`

O construtor constrói uma tupla cujos itens são iguais e na mesma ordem dos itens de *iterable*. *iterable* pode ser uma sequência, um contêiner que suporta iteração ou um objeto iterador. Se *iterable* já for uma tupla, este será retornado inalterado. Por exemplo, `tuple('abc')` retorna `('a', 'b', 'c')` e `tuple( [1, 2, 3] )` retorna `(1, 2, 3)`. Se nenhum argumento for dado, o construtor criará uma tupla vazia, `()`.

Observe que, na verdade, é a vírgula que faz uma tupla, e não os parênteses. Os parênteses são opcionais, exceto no caso de tupla vazia, ou quando são necessários para evitar ambiguidades sintáticas. Por exemplo, `f(a, b, c)` é uma chamada da função com três argumentos, enquanto que `f((a, b, c))` é uma chamada de função com uma tupla de 3 elementos com um único argumento.

As tuplas implementam todas as operações *comuns* de sequência.

Para coleções heterogêneas de dados onde o acesso pelo nome é mais claro do que o acesso pelo índice, *`collections.namedtuple()`* pode ser uma escolha mais apropriada do que um objeto tupla simples.

4.7.6 Intervalos

O tipo *`range`* representa uma sequência imutável de números e é comumente usado para percorrer um número determinado de vezes em um laço `for`.

class range(*stop*, */*)

class range(*start*, *stop*, *step=1*, */*)

Os argumentos para o construtor de intervalo devem ser inteiros (*`int`* embutido ou qualquer objeto que implemente o método especial `__index__()`). Se o argumento *step* for omitido, será usado o padrão `1`. Se o argumento *start* for omitido, será usado o padrão `0`. Se *step* for zero, uma exceção *`ValueError`* será levantada.

Para um *step* positivo, o conteúdo de um intervalo `r` será determinado pela fórmula `r[i] = start + step*i` onde `i >= 0` e `r[i] < stop`.

Para um *step* negativo, o conteúdo do intervalo ainda será determinado pela fórmula `r[i] = start + step*i`, mas as restrições serão `i >= 0` e `r[i] > stop`.

Um objeto intervalo estará vazio se `r[0]` não atender à restrição de valor. Intervalos suportam índices negativos, mas estes são interpretados como indexadores partindo do final da sequência determinada pelos índices positivos.

Intervalos contendo valores absolutos maiores que *`sys.maxsize`* são permitidos, mas alguns recursos (como *`len()`*) podem levantar *`OverflowError`*.

Exemplos de intervalos:

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list(range(0, 30, 5))
[0, 5, 10, 15, 20, 25]
>>> list(range(0, 10, 3))
[0, 3, 6, 9]
>>> list(range(0, -10, -1))
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
>>> list(range(0))
[]
>>> list(range(1, 0))
[]
```

Ranges implementam todas as operações *comuns* de sequência, exceto a concatenação e a repetição (devido ao fato de que os objetos intervalos só podem representar sequências que seguem um padrão rígido. e a repetição e a concatenação geralmente vão violar esse padrão).

start

O valor do parâmetro *start* (ou `0` se o parâmetro não for fornecido)

stop

O valor do parâmetro *stop*

step

O valor do parâmetro *step* (ou `1` se o parâmetro não for fornecido)

A vantagem do tipo *range* sobre um *list* ou *tuple* regular é que um objeto *range* sempre terá a mesma quantidade (pequena) de memória, não importa o tamanho do intervalo o mesmo esteja representando (como ele apenas armazena os valores `start`, `stop` e `step`, calculando itens individuais e subintervalos conforme necessário).

Objetos intervalos implementam a ABC *collections.abc.Sequence*, e fornecem recursos como testes de contenção, pesquisa de índice de elemento, fatiamento e suporte a índices negativos (veja *Tipos sequências — list, tuple, range*):

```
>>> r = range(0, 20, 2)
>>> r
range(0, 20, 2)
>>> 11 in r
False
>>> 10 in r
True
>>> r.index(10)
5
>>> r[5]
10
>>> r[:5]
range(0, 10, 2)
>>> r[-1]
18
```

Testar objetos intervalos por igualdade com `==` e `!=` os compara como sequências. Ou seja, dois objetos intervalos são considerados iguais se representarem a mesma sequência de valores. (Observe que dois objetos intervalos considerados iguais podem ter diferentes atributos *start*, *stop* e *step*, por exemplo `range(0) == range(2, 1, 3)` ou `range(0, 3, 2) == range(0, 4, 2)`.)

Alterado na versão 3.2: Implementa a ABC Sequence. Suporte a fatiamento e a índices negativos. Testa objetos *int* para associação em tempo constante em vez de iterar através de todos os itens.

Alterado na versão 3.3: Define '==' e '!=' para comparar objetos intervalos com base na sequência de valores que eles definem (em vez de comparar com base na identidade do objeto).

Adicionados os atributos *start*, *stop* e *step*.

Ver também

- A receita de linspace mostra como implementar uma versão preguiçosa de um intervalo adequado para aplicações de ponto flutuante.

4.8 Resumo dos métodos de tipo de sequência binária e de texto

A tabela a seguir resume os métodos de tipos de sequência binária e de texto por categoria.

Categoria	Métodos *str*	
Formatação	*str.format()*	
	str.format_map()	
	f-strings	
	Formatação de string no estilo printf	
Pesquisa e substituição	*str.find()*	*str.rfind()*
	str.index()	*str.rindex()*

Categoria	Métodos *str*		
	str.startswith()		
	str.endswith()		
	str.count()		
	str.replace()		
Divisão e união	*str.split()*		*str.rsplit()*
	str.splitlines()		
	str.partition()		
	str.rpartition()		
	str.join()		
String Classification	*str.isalpha()*		
	str.isdecimal()		
	str.isdigit()		
	str.isnumeric()		
	str.isalnum()		
	str.isidentifier()		
	str.islower()		
	str.isupper()		
	str.istitle()		
	str.isspace()		
	str.isprintable()		
Case Manipulation	*str.lower()*		
	str.upper()		
	str.casefold()		
	str.capitalize()		
	str.title()		
	str.swapcase()		
Preenchimento e remoção	*str.ljust()*		*str.rjust()*
	str.center()		
	str.expandtabs()		
	str.strip()		
	str.lstrip()		
Tradução e codificação	*str.translate()*		
	str.maketrans()		
	str.encode()		

4.9 Tipo sequência de texto — `str`

Os dados textuais em Python são tratados com objetos *str*, ou *strings*. Strings são *sequências* imutáveis de pontos de código Unicode. As strings literais são escritas de diversas maneiras:

- Aspas simples: `'permitem aspas "duplas" internas'`
- Aspas duplas: `"permitem aspas 'simples' internas"`
- Aspas triplas: `'''Três aspas simples'''`, `"""Três aspas duplas"""`

Strings de aspas triplas podem expandir por várias linhas – todos os espaços em branco associados serão incluídos em literal string.

Os literais strings que fazem parte de uma única expressão e têm apenas espaços em branco entre eles serão implicitamente convertidos em um único literal string. Isso é, `("spam " "eggs") == "spam eggs"`.

Veja strings para mais informações sobre as várias formas de literal de string, incluindo o suporte a sequências de escape, e o prefixo `r` ("raw") que desabilita a maioria dos processos de sequência de escape.

As strings também podem ser criadas a partir de outros objetos usando o construtor *str*.

Uma vez que não há nenhum tipo de "caractere" separador, indexar uma string produz strings de comprimento 1. Ou seja, para uma string não vazia *s*, `s[0] == s[0:1]`.

Também não existe um tipo string mutável, mas o método *str.join()* ou a classe *io.StringIO* podem ser usados para construir strings de forma eficiente a partir de vários partes distintas.

Alterado na versão 3.3: Para a compatibilidade com versões anteriores do Python 2, o prefixo `u` foi mais uma vez permitido em literais strings. Não possui quaisquer efeito sobre o significado de literais strings e não pode ser combinado com o prefixo `r`.

class str(*, *encoding='utf-8'*, *errors='strict'*)

class str(*object*)

class str(*object*, *encoding*, *errors='strict'*)

class str(*object*, *, *errors*)

Retorna uma versão *string* de *object*. Se *object* não é fornecido, retorna uma string vazia. Caso contrário, o comportamento de `str()` dependerá se o *encoding* ou *errors* são fornecidos, da seguinte forma.

Se nem *encoding* nem *errors* for fornecido, `str(object)` retornará `type(object).__str__(object)`, que é a representação da string "informal" ou que pode ser facilmente imprimível de *object*. Para objetos string, esta é a própria string. Se *object* não tiver um método:`__str__()`, então a função *str()* retornará *repr(object)*.

Se pelo menos um de *encoding* ou *errors* for fornecido, *object* deve ser um *objeto bytes ou similar* (por exemplo, *bytes* ou *bytearray*). Nesse caso, se *object* for um objeto *bytes* (ou *bytearray*), então `str(bytes, encoding, errors)` equivale a *bytes.decode(encoding, errors)*. Caso contrário, o objeto bytes subjacente ao objeto buffer é obtido antes de chamar *bytes.decode()*. Veja *Tipos de sequência binária — bytes, bytearray, memoryview* e bufferobjects para obter informações sobre objetos buffer.

Passando um objeto *bytes* para *str()* sem os argumentos *encoding* ou *errors* se enquadra no primeiro caso de retornar a representação informal de strings (consulte também a opção de linha de comando `-b` para Python). Por exemplo:

```
>>> str(b'Zoot!')
"b'Zoot!'"
```

Para mais informações sobre a classe `str` e seus métodos, veja *Tipo sequência de texto — str* e a seção *Métodos de string* abaixo. Para gerar strings formatadas, veja as seções f-strings e *Sintaxe das strings de formato*. Além disso, veja a seção *Serviços de Processamento de Texto*.

4.9.1 Métodos de string

Strings implementam todas as operações *comuns* de sequências, juntamente com os métodos adicionais descritos abaixo.

Strings também possuem suporte para duas formas de formatação de string, uma fornecendo uma ampla gama de flexibilidade e customização (veja *str.format()*, *Sintaxe das strings de formato* e *Formatação personalizada de strings*) e a outra baseada no estilo de formatação `printf` da linguagem C, que lida com uma gama menor de tipos e é levemente mais difícil de utilizar corretamente, mas é frequentemente mais rápida para os casos na qual ela consegue manipular (*Formatação de string no estilo printf*).

A seção *Serviços de Processamento de Texto* da biblioteca padrão cobre um número de diversos outros módulos que fornecem vários utilitários relacionados a texto (incluindo suporte a expressões regulares no módulo *re*).

`str.`**capitalize**()

Retorna uma cópia da string com o seu primeiro caractere em maiúsculo e o restante em minúsculo.

Alterado na versão 3.8: O primeiro caractere agora é colocado em *titlecase* ao invés de letras maiúsculas. Isso significa que caracteres como dígrafos apenas terão sua primeira letra alterada para maiúscula, ao invés de todos os caracteres.

`str.`**casefold**()

Retorna uma cópia da string em *casefolded*. Strings em *casefold* podem ser usadas para corresponder letras sem importar se são minúsculas ou maiúsculas.

Casefolding é similar a mudar para letras minúsculas, mas mais agressivo porque destina-se a remover todas as diferenças maiúsculas/minúsculas em uma string. Por exemplo, a letra minúscula alemã `'ß'` é equivalente a `"ss"`. Como ela já é uma minúscula, o método *lower()* não irá fazer nada para `'ß'`; já o método *casefold()* converte a letra para `"ss"`.

O algorítimo *casefolding* está descrito na seção 3.13 'Diretriz de Caixa Alta e Caixa Baixa' do Padrão Unicode.

Adicionado na versão 3.3.

`str.`**`center`**(*width*, *fillchar*=' ', /)

Retorna um texto centralizado em uma string de comprimento *width*. Preenchimento é feito usando o parâmetro *fillchar* especificado (padrão é o caractere de espaço ASCII). A string original é retornada se *width* é menor ou igual que `len(s)`. Por exemplo:

```
>>> 'Python'.center(10)
'  Python  '
>>> 'Python'.center(10, '-')
'--Python--'
>>> 'Python'.center(4)
'Python'
```

`str.`**`count`**(*sub*[, *start*[, *end*]])

Retorna o número de ocorrências da sub-string *sub* que não se sobrepõem no intervalo [*start*, *end*]. Argumentos opcionais *start* e *end* são interpretados como na notação de fatias.

Se *sub* estiver vazio, retorna o número de strings vazias entre os caracteres, que é o comprimento da string mais um. Por exemplo:

```
>>> 'spam, spam, spam'.count('spam')
3
>>> 'spam, spam, spam'.count('spam', 5)
2
>>> 'spam, spam, spam'.count('spam', 5, 10)
1
>>> 'spam, spam, spam'.count('eggs')
0
>>> 'spam, spam, spam'.count('')
17
```

`str.`**`encode`**(*encoding*='utf-8', *errors*='strict')

Retorna a string codificada para *bytes*.

encoding tem como padrão `'utf-8'`; veja *Standard Encodings* para valores possíveis.

errors controla como os erros de codificação são tratados. Se `'strict'` (o padrão), uma exceção *UnicodeError* é levantada. Outros valores possíveis são `'ignore'`, `'replace'`, `'xmlcharrefreplace'`, `'backslashreplace'` e qualquer outro nome registrado via *codecs.register_error()*. Veja *Error Handlers* para detalhes.

Por motivos de desempenho, o valor de *errors* não é verificado quanto à validade, a menos que um erro de codificação realmente ocorra, *Modo de Desenvolvimento do Python* esteja ativado ou uma construção de depuração seja usada. Por exemplo:

```
>>> encoded_str_to_bytes = 'Python'.encode()
>>> type(encoded_str_to_bytes)
<class 'bytes'>
>>> encoded_str_to_bytes
b'Python'
```

Alterado na versão 3.1: Adicionado suporte para argumentos nomeados.

Alterado na versão 3.9: O valor do argumento *errors* agora é verificado em *Modo de Desenvolvimento do Python* e no modo de depuração.

`str.`**`endswith`**(*suffix*[, *start*[, *end*]])

Retorna `True` se a string terminar com o *suffix* especificado, caso contrário retorna `False`. *suffix* também pode ser uma tupla de sufixos para procurar. Com o parâmetro opcional *start*, começamos a testar a partir daquela posição. Com o parâmetro opcional *end*, devemos parar de comparar na posição especificada. Usar *start* e *end* equivale a `str[start:end].endswith(suffix)`. Por exemplo:

```
>>> 'Python'.endswith('on')
True
>>> 'a tuple of suffixes'.endswith(('at', 'in'))
False
>>> 'a tuple of suffixes'.endswith(('at', 'es'))
True
>>> 'Python is amazing'.endswith('is', 0, 9)
True
```

Veja também *`startswith()`* e *`removesuffix()`*.

`str.`**`expandtabs`**(*tabsize=8*)

Retorna uma cópia da string onde todos os caracteres de tabulação são substituídos por um ou mais espaços, dependendo da coluna atual e do tamanho fornecido para a tabulação. Posições de tabulação ocorrem a cada *tabsize* caracteres (o padrão é 8, dada as posições de tabulação nas colunas 0, 8, 16 e assim por diante). Para expandir a string, a coluna atual é definida como zero e a string é examinada caractere por caractere. Se o caractere é uma tabulação (`\t`), um ou mais caracteres de espaço são inseridos no resultado até que a coluna atual seja igual a próxima posição de tabulação. (O caractere de tabulação em si não é copiado.) Se o caractere é um caractere de nova linha (`\n`) ou de retorno (`\r`), ele é copiado e a coluna atual é redefinida para zero. Qualquer outro caractere é copiado sem ser modificado e a coluna atual é incrementada em uma unidade independentemente de como o caractere é representado quando é impresso. Por exemplo:

```
>>> '01\t012\t0123\t01234'.expandtabs()
'01      012     0123    01234'
>>> '01\t012\t0123\t01234'.expandtabs(4)
'01  012 0123    01234'
>>> print('01\t012\n0123\t01234'.expandtabs(4))
01  012
0123    01234
```

`str.`**`find`**(*sub*[, *start*[, *end*]])

Retorna o índice mais baixo na string onde a substring *sub* é encontrado dentro da fatia `s[start:end]`. Argumentos opcionais como *start* e *end* são interpretados como na notação de fatiamento. Retorna `-1` se *sub* não for localizado. Por exemplo:

```
>>> 'spam, spam, spam'.find('sp')
0
>>> 'spam, spam, spam'.find('sp', 5)
6
```

Veja também *`rfind()`* e *`index()`*.

Nota

O método *`find()`* deve ser usado apenas se precisarmos conhecer a posição de *sub*. Para verificar se *sub* é ou não uma substring, use o operador `in`:

```
>>> 'Py' in 'Python'
True
```

str.**format**(**args*, ***kwargs*)

Executa uma operação de formatação de string. A string na qual este método é chamado pode conter texto literal ou campos para substituição delimitados por chaves `{}`. Cada campo de substituição contém um índice numérico de um argumento posicional ou o nome de um argumento nomeado. Retorna a cópia da string onde cada campo para substituição é substituído com o valor da string do argumento correspondente.

```
>>> "The sum of 1 + 2 is {0}".format(1+2)
'The sum of 1 + 2 is 3'
```

Veja *Sintaxe das strings de formato* para uma descrição das várias opções de formatação que podem ser especificadas em uma strings de formato.

Nota

Ao formatar um número (*int*, *float*, *complex*, *decimal.Decimal* e sub-classes) com o tipo `n` (ex: `'{:n}'.format(1234)`), a função define temporariamente a localidade `LC_CTYPE` para a localidade `LC_NUMERIC` a fim de decodificar campos `decimal_point` e `thousands_sep` de `localeconv()` se eles são caracteres não-ASCII ou maiores que 1 byte, e a localidade `LC_NUMERIC` é diferente da localidade `LC_CTYPE`. Esta mudança temporária afeta outras threads.

Alterado na versão 3.7: Ao formatar um número com o tipo `n`, a função define temporariamente a localidade `LC_CTYPE` para a localidade `LC_NUMERIC` em alguns casos.

str.**format_map**(*mapping*, */*)

Semelhante a `str.format(**mapping)`, exceto pelo fato de que `mapping` é usado diretamente e não copiado para uma classe *dict*. Isso é útil se, por exemplo, `mapping` é uma subclasse de dict:

```
>>> class Default(dict):
...     def __missing__(self, key):
...         return key
...
>>> '{name} was born in {country}'.format_map(Default(name='Guido'))
'Guido was born in country'
```

Adicionado na versão 3.2.

str.**index**(*sub*[, *start*[, *end*]])

Senelhante a *find()*, mas levanta *ValueError* quando a substring não é encontrada.

str.**isalnum**()

Retorna `True` se todos os caracteres na string são alfanuméricos e existe pelo menos um caractere, ou `False` caso contrário. Um caractere `c` é alfanumérico se um dos seguintes retorna `True`: `c.isalpha()`, `c.isdecimal()`, `c.isdigit()`, ou `c.isnumeric()`.

str.**isalpha**()

Retorna `True` se todos os caracteres na string são alfabéticos e existe pelo menos um caractere, `False` caso contrário. Caracteres alfabéticos são aqueles caracteres definidos na base de dados de caracteres Unicode como "Letra", isto é, aqueles cuja propriedade na categoria geral é um destes: "Lm", "Lt", "Lu", "Ll" ou "Lo". Perceba que isso é diferente da propriedade Alfabética definida na seção 4.10 "Letters, Alphabetic, and Ideographic" do Padrão Unicode.

str.**isascii**()

Retorna `True` se a string é vazia ou se todos os caracteres na string são ASCII, `False` caso contrário. Caracteres ASCII têm pontos de código no intervalo U+0000-U+007F.

Adicionado na versão 3.7.

str.**isdecimal**()

Retorna `True` se todos os caracteres na string são caracteres decimais e existe pelo menos um caractere,

`False` caso contrário. Caracteres decimais são aqueles que podem ser usados para formar números na base 10, exemplo U+0660, ou dígito zero para arábico-índico. Formalmente, um caractere decimal é um caractere em Unicode cuja categoria geral é "Nd".

`str.`**`isdigit`**`()`

Retorna `True` se todos os caracteres na string são dígitos e existe pelo menos um caractere, `False` caso contrário. Dígitos incluem caracteres decimais e dígitos que precisam de tratamento especial, tal como a compatibilidade com dígitos sobre-escritos. Isso inclui dígitos que não podem ser usados para formar números na base 10, como por exemplo os números de Kharosthi. Formalmente, um dígito é um caractere que tem a propriedade com valor Numeric_Type=Digit ou Numeric_Type=Decimal.

`str.`**`isidentifier`**`()`

Retorna `True` se a string é um identificador válido conforme a definição da linguagem, seção identifiers.

`keyword.iskeyword()` pode ser usada para testar se a string `s` é uma palavra reservada, tal como `def` e `class`.

Exemplo:

```
>>> from keyword import iskeyword

>>> 'olá.isidentifier(), iskeyword('olá')
(True, False)
>>> 'def'.isidentifier(), iskeyword('def')
(True, True)
```

`str.`**`islower`**`()`

Retorna `True` se todos os caracteres em caixa (que possuem maiúsculo e minúsculo)[4] na string são minúsculos e existe pelo menos um caractere em caixa, `False` caso contrário.

`str.`**`isnumeric`**`()`

Retorna `True` se todos os caracteres na string são caracteres numéricos, e existe pelo menos um caractere, `False` caso contrário. Caracteres numéricos incluem dígitos, e todos os caracteres que têm a propriedade com valor numérico Unicode, isto é: U+2155, um quinto de fração vulgar. Formalmente, caracteres numéricos são aqueles que possuem propriedades com valor Numeric_Type=Digit, Numeric_Type=Decimal ou Numeric_Type=Numeric.

`str.`**`isprintable`**`()`

Retorna `True` se todos os caracteres na string forem imprimíveis, `False` se contiver pelo menos um caractere não imprimível.

Aqui, "imprimíveis" significa que o caractere é adequado para *`repr()`* usar em sua saída; "não imprimível" significa que *`repr()`* em tipos embutidos fará o escape hexadecimal do caractere. Não tem relação com o tratamento de strings escritas em *`sys.stdout`* ou *`sys.stderr`*.

Os caracteres imprimíveis são aqueles que no banco de dados de caracteres Unicode (veja *`unicodedata`*) têm uma categoria geral no grupo Letra, Marca, Número, Pontuação ou Símbolo (L, M, N, P ou S); mais o espaço ASCII 0x20. Caracteres não imprimíveis são aqueles no grupo Separador ou Outro (Z ou C), exceto o espaço ASCII.

`str.`**`isspace`**`()`

Retorna `True` se existem apenas caracteres de espaço em branco na string e existe pelo menos um caractere, `False` caso contrário.

Um caractere é *espaço em branco* se no banco de dados de caracteres Unicode (veja *`unicodedata`*), ou pertence a categoria geral `Zs` ("Separador, espaço"), ou sua classe bidirecional é `WS`, `B` ou `S`.

[4] Caracteres que possuem maiúsculo e minúsculo são aqueles com a propriedade de categoria geral igual a "Lu" (Letra, maiúscula), "Ll" (Letra, minúscula), ou "Lt" (Letra, em formato de título).

str.**istitle**()

Retorna `True` se a string é titlecased e existe pelo menos um caractere, por exemplo caracteres maiúsculos somente podem proceder caracteres que não diferenciam maiúsculas/minúsculas, e caracteres minúsculos somente podem proceder caracteres que permitem ambos. Retorna `False` caso contrário.

str.**isupper**()

Retorna `True` se todos os caracteres que permitem maiúsculas ou minúsculas[Página 61, 4] na string estão com letras maiúsculas, e existe pelo menos um caractere maiúsculo, `False` caso contrário.

```
>>> 'BANANA'.isupper()
True
>>> 'banana'.isupper()
False
>>> 'baNana'.isupper()
False
>>> ' '.isupper()
False
```

str.**join**(*iterable*, /)

Retorna a string que é a concatenação das strings no *iterável*. Uma *TypeError* será levantada se existirem quaisquer valores que não sejam strings no *iterável*, incluindo objetos *bytes*. O separador entre elementos é a string que está fornecendo este método.

str.**ljust**(*width*, *fillchar=' '*, /)

Retorna a string alinhada a esquerda em uma string de comprimento *width*. Preenchimento é feito usando *fillchar* que for especificado (o padrão é o caractere ASCII de espaço). A string original é retornada se *width* é menor ou igual que `len(s)`.

str.**lower**()

Retorna uma cópia da string com todos os caracteres que permitem maiúsculo E minúsculo[Página 61, 4] convertidos para letras minúsculas.

O algoritmo de letras minúsculas está descrito na seção 3.13 'Default Case Folding' do Padrão Unicode.

str.**lstrip**(*chars=None*, /)

Retorna uma cópia da string com caracteres iniciais removidos. O argumento *chars* é uma string que especifica o conjunto de caracteres a serem removidos. Se for omitido ou se for `None`, o argumento *chars* será considerado como espaço em branco por padrão para a remoção. O argumento *chars* não é um prefixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> '   spacious   '.lstrip()
'spacious   '
>>> 'www.example.com'.lstrip('cmowz.')
'example.com'
```

Consulte *str.removeprefix()* para um método que removerá uma única string de prefixo em vez de todo um conjunto de caracteres. Por exemplo:

```
>>> 'Arthur: three!'.lstrip('Arthur: ')
'ee!'
>>> 'Arthur: three!'.removeprefix('Arthur: ')
'three!'
```

static str.**maketrans**(*dict*, /)

static str.**maketrans**(*from*, *to*, *remove=''*, /)

Este método estático retorna uma tabela de tradução usável para *str.translate()*.

Se existe apenas um argumento, ele deve ser um dicionário mapeando números Unicode (inteiros) ou caracteres (strings de comprimento 1) para números Unicode, strings (de comprimento arbitrário) ou `None`. Caracteres chave serão então convertidos para números ordinais.

Se existirem dois argumentos, eles devem ser strings de igual comprimento, e no dicionário resultante, cada caractere em *from* será mapeado para o caractere na mesma posição em *to*. Se existir um terceiro argumento, ele deve ser uma string, cujos caracteres serão mapeados para `None` no resultado.

`str.`**`partition`**(*sep*, /)

Quebra a string na primeira ocorrência de *sep*, e retorna uma tupla com 3 elementos contendo a parte antes do separador, o próprio separador, e a parte após o separador. Se o separador não for encontrado, retorna uma tupla com 3 elementos contendo a string, seguido de duas strings vazias.

`str.`**`removeprefix`**(*prefix*, /)

Se a string começar com a string *prefix*, retorna `string[len(prefix):]`. Caso contrário, retorna uma cópia da string original:

```
>>> 'TestHook'.removeprefix('Test')
'Hook'
>>> 'BaseTestCase'.removeprefix('Test')
'BaseTestCase'
```

Adicionado na versão 3.9.

`str.`**`removesuffix`**(*suffix*, /)

Se a string terminar com a string *suffix* e a *suffix* não estiver vazia, retorna `string[:-len(suffix)]`. Caso contrário, retorna uma cópia da string original:

```
>>> 'MiscTests'.removesuffix('Tests')
'Misc'
>>> 'TmpDirMixin'.removesuffix('Tests')
'TmpDirMixin'
```

Adicionado na versão 3.9.

`str.`**`replace`**(*old*, *new*, /, *count=-1*)

Retorna uma cópia da string com todas as ocorrências da substring *old* substituídas por *new*. Se o argumento opcional *count* é fornecido, apenas as primeiras *count* ocorrências são substituídas. Se *count* não é especificado ou é igual a `-1`, então todas as ocorrências serão substituídas.

Alterado na versão 3.13: *count* agora é suportado como um argumento nomeado.

`str.`**`rfind`**(*sub*[, *start*[, *end*]])

Retorna o maior índice onde a substring *sub* foi encontrada dentro da string, onde *sub* está contida dentro do intervalo `s[start:end]`. Argumentos opcionais *start* e *end* são interpretados usando a notação slice. Retorna `-1` em caso de falha.

`str.`**`rindex`**(*sub*[, *start*[, *end*]])

Similar a *`rfind()`* mas levanta um *`ValueError`* quando a substring *sub* não é encontrada.

`str.`**`rjust`**(*width*, *fillchar=' '*, /)

Retorna a string alinhada à direita em uma string de comprimento *width*. Preenchimento é feito usando o caractere *fillchar* especificado (o padrão é um caractere de espaço ASCII). A string original é retornada se *width* é menor que ou igual a `len(s)`.

`str.`**`rpartition`**(*sep*, /)

Quebra a string na última ocorrência de *sep*, e retorna uma tupla com 3 elementos contendo a parte antes do separador, o próprio separador, e a parte após o separador. Se o separador não for encontrado, retorna uma tupla com 3 elementos contendo duas strings vazias, seguido da própria string original.

`str.`**`rsplit`**(*sep=None*, *maxsplit=-1*)

Retorna uma lista de palavras na string, usando *sep* como a string delimitadora. Se *maxsplit* é fornecido, no máximo *maxsplit* cortes são feitos, sendo estes mais *à direita*. Se *sep* não foi especificado ou `None` foi informado, qualquer string de espaço em branco é um separador. Exceto pelo fato de separar pela direita, *`rsplit()`* se comporta como *`split()`*, o qual é descrito em detalhes abaixo.

str.**rstrip**(*chars=None*, /)

Retorna uma cópia da string com caracteres no final removidos. O argumento *chars* é uma string que especifica o conjunto de caracteres para serem removidos. Se omitidos ou tiver o valor `None`, o argumento *chars* considera como padrão a remoção dos espaços em branco. O argumento *chars* não é um sufixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> '   spacious   '.rstrip()
'   spacious'
>>> 'mississippi'.rstrip('ipz')
'mississ'
```

Consulte *`str.removesuffix()`* para um método que removerá uma única string de sufixo em vez de todo um conjunto de caracteres. Por exemplo:

```
>>> 'Monty Python'.rstrip(' Python')
'M'
>>> 'Monty Python'.removesuffix(' Python')
'Monty'
```

str.**split**(*sep=None*, *maxsplit=-1*)

Retorna uma lista de palavras na string, usando *sep* como a string delimitadora. Se *maxsplit* é fornecido, no máximo *maxsplit* cortes são feitos (portando, a lista terá no máximo `maxsplit+1` elementos). Se *maxsplit* não foi especificado ou `-1` foi informado, então não existe limite no número de cortes (todos os cortes possíveis são realizados).

Se *sep* é fornecido, delimitadores consecutivos não são agrupados juntos e eles são destinados para delimitar strings vazias (por exemplo, `'1,,2'.split(',')` retorna `['1', '', '2']`). O argumento *sep* pode consistir em vários caracteres como um único delimitador (para separar com vários delimitadores, use *`re.split()`*). Separar uma string vazia com um separador especificado retorna `['']`.

Por exemplo:

```
>>> '1,2,3'.split(',')
['1', '2', '3']
>>> '1,2,3'.split(',', maxsplit=1)
['1', '2,3']
>>> '1,2,,3,'.split(',')
['1', '2', '', '3', '']
>>> '1<>2<>3<4'.split('<>')
['1', '2', '3<4']
```

Se *sep* não for especificado ou for `None`, um algoritmo diferente de separação é aplicado: ocorrências consecutivas de espaços em branco são consideradas como um separador único, e o resultado não conterá strings vazias no início ou no final, se a string tiver espaços em branco no início ou no final. Consequentemente, separar uma string vazia ou uma string que consiste apenas de espaços em branco com o separador `None`, retorna `[]`.

Por exemplo:

```
>>> '1 2 3'.split()
['1', '2', '3']
>>> '1 2 3'.split(maxsplit=1)
['1', '2 3']
>>> '   1   2   3   '.split()
['1', '2', '3']
```

Se *sep* não for especificado ou for `None` e *maxsplit* for `0`, somente as sequências iniciais de espaços em branco consecutivos serão consideradas.

Por exemplo:

```
>>> "".split(None, 0)
[]
>>> "   ".split(None, 0)
[]
>>> "   foo   ".split(maxsplit=0)
['foo   ']
```

`str.`**`splitlines`**(*keepends=False*)

Retorna uma lista das linhas na string, quebrando a mesma nas fronteiras de cada linha. Quebras de linhas não são incluídas na lista resultante a não ser que *keepends* seja fornecido e seja verdadeiro.

Este método divide nos seguintes limites das linhas. Em particular, os limites são um superconjunto de *novas linhas universais*.

Representação	Descrição
`\n`	Feed de linha
`\r`	Retorno de Carro
`\r\n`	Retorno do Carro + Feed da Linha
`\v` ou `\x0b`	Tabulação de Linha
`\f` ou `\x0c`	Formulário de Feed
`\x1c`	Separador de Arquivos
`\x1d`	Separador de Grupo
`\x1e`	Separador de Registro
`\x85`	Próxima Linha (C1 Control Code)
`\u2028`	Separador de Linha
`\u2029`	Separador de Parágrafo

Alterado na versão 3.2: `\v` e `\f` adicionado à lista de limites de linha.

Por exemplo:

```
>>> 'ab c\n\nde fg\rkl\r\n'.splitlines()
['ab c', '', 'de fg', 'kl']
>>> 'ab c\n\nde fg\rkl\r\n'.splitlines(keepends=True)
['ab c\n', '\n', 'de fg\r', 'kl\r\n']
```

Ao contrário do método *`split()`* quando um delimitador de String *sep* é fornecido, este método retorna uma lista vazia para a uma String vazia e uma quebra de linha de terminal não resulta numa linha extra:

```
>>> "".splitlines()
[]
>>> "Uma linha\n".splitlines()
['Uma linha']
```

Para comparação, temos `split('\n')`:

```
>>> ''.split('\n')
['']
>>> 'Duas linhas\n'.split('\n')
['Duas linhas', '']
```

`str.`**`startswith`**(*prefix*[, *start*[, *end*]])

Retorne `True` se a String começar com o *prefixo*, caso contrário, retorna `False`. *prefixo* também pode ser uma tupla de prefixos a serem procurados. Com *start* opcional, a String de teste começa nessa posição. Com *fim* opcional, interrompe a comparação de String nessa posição.

str.**strip**(*chars=None*, */*)

Retorna uma cópia da string com caracteres no início e no final removidos. O argumento *chars* é uma string que especifica o conjunto de caracteres a serem removidos. Se for omitido ou for `None`, o argumento *chars* irá remover por padrão os caracteres em branco. O argumento *chars* não é um prefixo, nem um sufixo; ao contrário disso, todas as combinações dos seus seus valores são removidas:

```
>>> '   spacious   '.strip()
'spacious'
>>> 'www.example.com'.strip('cmowz.')
'example'
```

Os valores do argumento *chars* são removidos dos extremos inicial e final da string. Caracteres são removidos do extremo inicial até atingir um caractere da string que não está contido no conjunto de caracteres em *chars*. Uma ação similar acontece no extremo final da string. Por exemplo:

```
>>> comment_string = '#....... Section 3.2.1 Issue #32 .......'
>>> comment_string.strip('.#! ')
'Section 3.2.1 Issue #32'
```

str.**swapcase**()

Retorna uma cópia da string com caracteres maiúsculos convertidos para minúsculos e vice-versa. Perceba que não é necessariamente verdade que `s.swapcase().swapcase() == s`.

str.**title**()

Retorna uma versão titlecased da string, onde palavras iniciam com um caractere com letra maiúscula e os caracteres restantes são em letras minúsculas.

Por exemplo:

```
>>> 'Olá mundo'.title()
'Olá mundo'
```

O algoritmo usa uma definição simples independente de idioma para uma palavra, como grupos de letras consecutivas. A definição funciona em muitos contextos, mas isso significa que apóstrofes em contrações e possessivos formam limites de palavras, os quais podem não ser o resultado desejado:

```
>>> "a caixa d'água do joe's bar está vazia".title()
"A Caixa D'Água Do Joe'S Bar Está Vazia"
```

A função *string.capwords()* não tem esse problema, pois ela divide palavras apenas em espaços.

Alternativamente, uma solução alternativa para os apóstrofes pode ser construída usando expressões regulares:

```
>>> import re
>>> def titlecase(s):
...     return re.sub(r"[A-Za-z]+('[A-Za-z]+)?",
...                   lambda mo: mo.group(0).capitalize(),
...                   s)
...
>>> titlecase("o joe's bar está aberto")
"O Joe's Bar Está Aberto"
```

str.**translate**(*table*, */*)

Retorna uma cópia da string na qual cada caractere foi mapeado através da tabela de tradução fornecida. A tabela deve ser um objeto que implementa indexação através de `__getitem__()`, tipicamente um *mapeamento* ou uma *sequência*. Quando indexada por um ordinal unicode (um inteiro), o objeto tabela pode fazer qualquer uma das seguintes ações: retornar um ordinal unicode ou uma string, para mapear o caractere para um ou mais caracteres; retornar `None`, para deletar o caractere da string de retorno; ou levantar uma exceção *LookupError*, para mapear o caractere para si mesmo.

Você pode usar *str.maketrans()* para criar um mapa de tradução com mapeamentos caractere para caractere em diferentes formatos.

Veja também o módulo *codecs* para uma abordagem mais flexível para mapeamento de caractere customizado.

`str.`**`upper`**`()`

Retorna uma cópia da string com todos os caracteres que permitem maiúsculo e minúsculo[Página 61, 4] convertidos para letras maiúsculas. Perceba que `s.upper().isupper()` pode ser `False` se `s` contiver caracteres que não possuem maiúsculas e minúsculas, ou se a categoria Unicode do(s) caractere(s) resultante(s) não for "Lu" (Letra maiúscula), mas por ex "Lt" (Letra em titlecase).

O algoritmo de letras maiúsculas está descrito na seção 3.13 'Default Case Folding' do Padrão Unicode.

`str.`**`zfill`**`(`*width*`, /)`

Retorna uma cópia da String deixada preenchida com dígitos ASCII `'0'` para fazer uma string de comprimento *width*. Um prefixo sinalizador principal (`'+'`/`'-'`) será tratado inserindo o preenchimento *após* o caractere de sinal em vez de antes. A String original será retornada se o *width* for menor ou igual a `len(s)`.

Por exemplo:

```
>>> "42".zfill(5)
'00042'
>>> "-42".zfill(5)
'-0042'
```

4.9.2 Strings literais formatadas (f-strings)

Adicionado na versão 3.6.

Alterado na versão 3.7: `await` e `async for` podem ser usados em expressões dentro de f-strings.

Alterado na versão 3.8: Adicionado o operador de depuração (=)

Alterado na versão 3.12: Muitas restrições a expressões dentro de f-strings foram removidas. Notavelmente, strings aninhadas, comentários e contrabarras agora são permitidos.

Uma *f-string* (formalmente uma *literal de string formatada*) é uma literal de string prefixada com `f` ou `F`. Esse tipo de literal de string permite incorporar expressões Python arbitrárias dentro de *campos de substituição*, delimitados por chaves (`{}`). Essas expressões são avaliadas em tempo de execução, de forma semelhante a *str.format()*, e convertidas em objetos *str* regulares. Por exemplo:

```
>>> who = 'Ninguém'
>>> nationality = 'Espanhola'
>>> f'{who.title()} está à espera da Inquisição {nationality}!'
'Ninguém está à espera da Inquisição Espanhola!'
```

Também é possível usar uma f-string de várias linhas:

```
>>> f'''Esta é uma string
... em duas linhas'''
'Esta é uma string\nem duas linhas'
```

Uma única chave de abertura, `'{'`, marca um *campo de substituição* que pode conter qualquer expressão Python:

```
>>> nationality = 'Espanhola'
>>> f'A Inquisição {nationality}!'
'A Inquisição Espanhola!'
```

Para incluir um literal `{` ou `}`, use chaves duplas:

```
>>> x = 42
>>> f'{{x}} is {x}'
'{x} is 42'
```

Funções também podem ser usadas, como *especificadores de formato*:

```
>>> from math import sqrt
>>> f'√2 \N{ALMOST EQUAL TO} {sqrt(2):.5f}'
'√2 ≈ 1.41421'
```

Qualquer expressão que não seja string é convertida usando *str()*, por padrão:

```
>>> from fractions import Fraction
>>> f'{Fraction(1, 3)}'
'1/3'
```

Para usar uma conversão explícita, use o operador `!` (ponto de exclamação), seguido por qualquer um dos formatos válidos, que são:

Conversão	Significado
`!a`	*ascii()*
`!r`	*repr()*
`!s`	*str()*

Por exemplo:

```
>>> from fractions import Fraction
>>> f'{Fraction(1, 3)!s}'
'1/3'
>>> f'{Fraction(1, 3)!r}'
'Fraction(1, 3)'
>>> question = '¿Dónde está el Presidente?'
>>> print(f'{question!a}')
'\xbfD\xf3nde est\xe1 el Presidente?'
```

Durante a depuração, pode ser útil visualizar tanto a expressão quanto seu valor, usando o sinal de igual (=) após a expressão. Isso preserva os espaços entre chaves e pode ser usado com um conversor. Por padrão, o operador de depuração usa a conversão *repr()* (`!r`). Por exemplo:

```
>>> from fractions import Fraction
>>> calculation = Fraction(1, 3)
>>> f'{calculation=}'
'calculation=Fraction(1, 3)'
>>> f'{calculation = }'
'calculation = Fraction(1, 3)'
>>> f'{calculation = !s}'
'calculation = 1/3'
```

Após a avaliação da saída, ela pode ser formatada usando um *especificador de formato* seguido de dois pontos (`':'`). Após a avaliação da expressão e sua possível conversão para uma string, o método `__format__()` do resultado é chamado com o especificador de formato, ou a string vazia, caso nenhum especificador de formato seja fornecido. O resultado formatado é então usado como o valor final para o campo de substituição. Por exemplo:

```
>>> from fractions import Fraction
>>> f'{Fraction(1, 7):.6f}'
'0.142857'
```

(continua na próxima página)

(continuação da página anterior)

```
>>> f'{Fraction(1, 7):_^+10}'
'___+1/7___'
```

4.9.3 Formatação de string no estilo `printf`

Nota

As operações de formatação descritas aqui exibem uma variedade de peculiaridades que levam a uma série de erros comuns (como a falha na exibição de Tuplas e Dicionários corretamente).

O uso de literais de strings formatadas, a interface *`str.format()`* ou *`string.Template`* pode ajudar a evitar esses erros. Cada uma dessas alternativas oferece suas próprias vantagens e desvantagens de simplicidade, flexibilidade e/ou extensibilidade.

Os objetos string possuem um único operador embutido: o operador `%` (módulo). O mesmo também é conhecido como o operador de *formatação* ou *interpolação*. Dado `format % values` (onde *format* é uma string), as especificações de conversão `%` em *format* são substituídas por zero ou mais elementos de *values*. O efeito é semelhante ao uso da função `sprintf()` na linguagem C. Por exemplo:

```
>>> print('%s tem %d tipos de aspas.' % ('Python', 2))
Python tem 2 tipos de aspas.
```

Se *format* precisar de um único operador, *valores* podem ser objetos simples ou que não sejam uma tupla.[5] Caso contrário, *valores* precisarão ser uma tupla com exatamente o número de itens especificados pela string de formatação, ou um único mapa de objetos (por exemplo, um dicionário).

Um especificador de conversão contém dois ou mais caracteres e tem os seguintes componentes, que devem aparecer nesta ordem:

1. O caractere `'%'`, que determina o início do especificador.
2. Mapeamento de chaves (opcional), consistindo de uma sequência entre parênteses de caracteres (por exemplo, `(algumnome)`).
3. Flags de conversão (opcional), que afetam o resultado de alguns tipos de conversão.
4. Largura mínima do campo (opcional). Se for especificado como um `'*'` (asterisco), a largura real será lida a partir do próximo elemento da tupla em *values* e o objeto a converter virá após a largura mínima do campo e a precisão que é opcional.
5. Precisão (opcional), fornecido como uma `'.'` (ponto) seguido pela precisão. Se determinado como um `'*'` (um asterisco), a precisão real será lida a partir do próximo elemento da tupla em *values*, e o valor a converter virá após a precisão.
6. Modificador de comprimento (opcional).
7. Tipo de conversão.

Quando o argumento certo for um dicionário (ou outro tipo de mapeamento), então os formatos na string *deverão* incluir uma chave de mapeamento entre parênteses nesse dicionário inserido imediatamente após o caractere `'%'`. A key de mapeamento seleciona o valor a ser formatado a partir do mapeamento. Por exemplo:

```
>>> print('%(language)s has %(number)03d quote types.' %
...       {'language': "Python", "number": 2})
Python has 002 quote types.
```

Nesse caso, nenhum especificador `*` poderá ocorrer num formato (uma vez que eles exigem uma lista de parâmetros sequenciais).

Os caracteres flags de conversão são:

[5] Para formatar apenas uma tupla, você deve portanto fornecer uma tupla com apenas um elemento, que é a tupla a ser formatada.

Sinalizador	Significado
`'#'`	A conversão de valor usará o "formulário alternativo" (conforme definido abaixo).
`'0'`	A conversão será preenchida por zeros para valores numéricos.
`'-'`	O valor convertido será ajustado à esquerda (substitui a conversão `'0'` se ambos forem fornecidos).
`' '`	(um espaço) Um espaço em branco deverá ser deixado antes de um número positivo (ou uma string vazia) produzido por uma conversão assinada.
`'+'`	Um sinal de caractere (`'+'` ou `'-'`) precederá a conversão (substituindo o sinalizador de "espaço").

Um modificador de comprimento (`h`, `l`, ou `L`) pode estar presente, mas será ignorado, pois o mesmo não é necessário para o Python – então, por exemplo, `%ld` é idêntico a `%d`.

Os tipos de conversão são:

Conversão	Significado	Notas
`'d'`	Número decimal inteiro sinalizador.	
`'i'`	Número decimal inteiro sinalizador.	
`'o'`	Valor octal sinalizador.	(1)
`'u'`	Tipo obsoleto – é idêntico a `'d'`.	(6)
`'x'`	Hexadecimal com sinal (minúsculo).	(2)
`'X'`	Hexadecimal com sinal (maiúsculo).	(2)
`'e'`	Formato exponencial de ponto flutuante (minúsculo).	(3)
`'E'`	Formato exponencial de ponto flutuante (maiúsculo).	(3)
`'f'`	Formato decimal de ponto flutuante.	(3)
`'F'`	Formato decimal de ponto flutuante.	(3)
`'g'`	O formato de ponto flutuante. Usa o formato exponencial em minúsculo se o expoente for inferior a -4 ou não inferior a precisão, formato decimal, caso contrário.	(4)
`'G'`	Formato de ponto flutuante. Usa o formato exponencial em maiúsculo se o expoente for inferior a -4 ou não inferior que a precisão, formato decimal, caso contrário.	(4)
`'c'`	Caractere único (aceita inteiro ou um único caractere string).	
`'r'`	String (converte qualquer objeto Python usando a função *repr()*).	(5)
`'s'`	String (converte qualquer objeto Python usando a função *str()*).	(5)
`'a'`	String (converte qualquer objeto Python usando a função *ascii()*).	(5)
`'%'`	Nenhum argumento é convertido, resultando em um caractere `'%'` no resultado.	

Notas:

(1) A forma alternativa faz com que um especificador octal principal (`'0o'`) seja inserido antes do primeiro dígito.

(2) O formato alternativo produz um `'0x'` ou `'0X'` (dependendo se o formato `'x'` or `'X'` foi usado) para ser inserido antes do primeiro dígito.

(3) A forma alternativa faz com que o resultado sempre contenha um ponto decimal, mesmo que nenhum dígito o siga.

A precisão determina o número de dígitos após o ponto decimal e o padrão é 6.

(4) A forma alternativa faz com que o resultado sempre contenha um ponto decimal e os zeros à direita não sejam removidos, como de outra forma seriam.

A precisão determina o número de dígitos significativos antes e depois do ponto decimal e o padrão seja 6.

(5) Se a precisão for `N`, a saída será truncada em `N` caracteres.

(6) Veja **PEP 237**.

Como as strings no Python possuem comprimento explícito, conversões de `%s` não presumem que `'\0'` seja o fim da string.

Alterado na versão 3.1: `%f` as conversões para números cujo valor absoluto é superior a 1e50 não são mais substituídas pela conversão `%g`.

4.10 Tipos de sequência binária — `bytes`, `bytearray`, `memoryview`

Os principais tipos embutidos para manipular dados binários são *bytes* e *bytearray*. Eles são suportados por *memoryview* a qual usa o protocolo buffer para acessar a memória de outros objetos binários sem precisar fazer uma cópia.

O módulo *array* provê suporte a um armazenamento eficiente de tipos de dados básicos como inteiros de 32 bits e valores de ponto flutuante com precisão dupla IEEE754.

4.10.1 Objetos bytes

Objetos bytes são sequências imutáveis de bytes simples. Como muitos protocolos binários importantes são baseados em codificação ASCII de texto, objetos bytes oferecem diversos métodos que são válidos apenas quando trabalhamos com dados compatíveis com ASCII, e são proximamente relacionados com objetos string em uma variedade de outros sentidos.

class bytes(*source=b''*)

class bytes(*source*, *encoding*, *errors='strict'*)

Em primeiro lugar, a sintaxe para literais de bytes é em grande parte a mesma para literais de string, exceto que um prefixo `b` é adicionado:

- Aspas simples: `b'ainda permite aspas "duplas" dentro'`
- Aspas duplas: `b"ainda permite aspas 'simples' dentro"`
- Aspas triplas: `b'''3 aspas simples'''`, `b"""3 aspas duplas"""`

Apenas caracteres ASCII são permitidos em literais de bytes (independentemente da codificação declarada no código-fonte). Qualquer valor binário superior a 127 deverá ser inserido em literais de bytes usando a sequência de escape apropriada.

Assim como string literais, bytes literais também podem usar um prefixo `r` para desabilitar o processamento de sequências de escape. Veja strings para mais sobre as várias formas de bytes literais, incluindo suporte a sequências de escape.

Enquanto bytes literais e representações são baseados em texto ASCII, objetos bytes na verdade se comportam como sequências imutáveis de inteiros, com cada valor na sequência restrito aos limites `0 <= x < 256` (tentativas de violar essa restrição irão disparar *ValueError*). Isto é feito deliberadamente para enfatizar que enquanto muitos formatos binários incluem elementos baseados em ASCII e podem ser utilmente manipulados com alguns algoritmos orientados a texto, esse não é geralmente o caso para dados binários arbitrários (aplicar algoritmos de processamento de texto cegamente em formatos de dados binários que não são compatíveis com ASCII irá usualmente resultar em dados corrompidos).

Além das formas literais, os objetos bytes podem ser criados de várias outras maneiras:

- Um bytes preenchido com objetos zero com um comprimento especificado: `bytes(10)`
- De um iterável de inteiros: `bytes(range(20))`
- Copiando dados binários existentes através do protocolo de Buffer: `bytes(obj)`

Veja também os embutidos *bytes*.

Como 2 dígitos hexadecimais correspondem precisamente a apenas um byte, números hexadecimais são um formato comumente usado para descrever dados binários. Portanto, o tipo bytes tem um método de classe adicional para ler dados nesse formato:

classmethod fromhex(*string*, */*)

Este método de classe da classe *bytes* retorna um objeto bytes, decodificando o objeto string fornecido. A string deve conter dois dígitos hexadecimais por byte, com espaço em branco ASCII sendo ignorado.

```
>>> bytes.fromhex('2Ef0 F1f2  ')
b'.\xf0\xf1\xf2'
```

Alterado na versão 3.7: *bytes.fromhex()* agora ignora todos os espaços em branco em ASCII na string, não apenas espaços.

Alterado na versão 3.14: *bytes.fromhex()* agora aceita objetos *bytes* e *similares* ASCII como entrada.

Uma função de conversão reversa existe para transformar um objeto bytes na sua representação hexadecimal.

hex(*, *bytes_per_sep=1*)

hex(*sep*, *bytes_per_sep=1*)

Retorna um objeto string contendo dois dígitos hexadecimais para cada byte na instância.

```
>>> b'\xf0\xf1\xf2'.hex()
'f0f1f2'
```

Se você quiser tornar a string hexadecimal mais fácil de ler, você pode especificar um caractere separador através do parâmetro *sep*, que será incluído no resultado. Por padrão, este separador será incluído entre cada byte. Um segundo parâmetro opcional *bytes_per_sep* controla o espaçamento. Valores positivos calculam a posição do separador a partir da direita, valores negativos calculam a partir da esquerda.

```
>>> value = b'\xf0\xf1\xf2'
>>> value.hex('-')
'f0-f1-f2'
>>> value.hex('_', 2)
'f0_f1f2'
>>> b'UUDDLRLRAB'.hex(' ', -4)
'55554444 4c524c52 4142'
```

Adicionado na versão 3.5.

Alterado na versão 3.8: *bytes.hex()* agora suporta os parâmetros opcionais *sep* e *bytes_per_sep* para inserir separadores entre bytes na saída hexadecimal.

Como objetos bytes são sequências de inteiros (certo parentesco a uma tupla), para um objeto bytes *b*, `b[0]` será um inteiro, enquanto `b[0:1]` será um objeto bytes de comprimento 1. (Isso contrasta com strings de texto, onde tanto o uso por índice quanto por fatiamento irão produzir uma string de comprimento 1)

A representação de objetos bytes utiliza o formato literal (`b'...'`), uma vez que ela é frequentemente mais útil do que, por exemplo, `bytes([46, 46, 46])`. Você sempre pode converter um objeto bytes em uma lista de inteiros usando `list(b)`.

4.10.2 Objetos Bytearray

Objetos *bytearray* são mutáveis, em contrapartida a objetos *bytes*.

class bytearray(*source=b''*)

class bytearray(*source*, *encoding*, *errors='strict'*)

Não existe sintaxe literal dedicada para objetos de bytearray, ao invés disso eles sempre são criados através da chamada do construtor:

- Criando uma instância vazia: `bytearray()`
- Criando uma instância cheia de zero com um determinado comprimento: `bytearray(10)`
- A partir de inteiros iteráveis: `bytearray(range(20))`
- Copiando dados binários existentes através do protocolo de buffer: `bytearray(b'Hi!')`

Como os objetos bytearray são mutáveis, os mesmos suportam as operações de sequência *mutable* além das operações comuns de bytes e bytearray descritas em *Operações com Bytes e Bytearray*.

Veja também o tipo embutido *bytearray*.

Uma vez que 2 dígitos hexadecimais correspondem precisamente a um único byte, os números hexadecimais são um formato comumente usado para descrever dados binários. Consequentemente, o tipo de bytearray tem um método de classe adicional para ler dados nesse formato:

classmethod fromhex(*string*, */*)

Este método de classe da classe *bytearray* retorna um objeto bytearray, decodificando o objeto string fornecido. A string deve conter dois dígitos hexadecimais por byte, com espaços em branco ASCII sendo ignorados.

```
>>> bytearray.fromhex('2Ef0 F1f2  ')
bytearray(b'.\xf0\xf1\xf2')
```

Alterado na versão 3.7: *bytearray.fromhex()* agora ignora todos os espaços em branco em ASCII na string, não apenas espaços.

Alterado na versão 3.14: *bytearray.fromhex()* agora aceita objetos *bytes* e *similares* ASCII como entrada.

Existe uma função de conversão reversa para transformar um objeto bytearray em sua representação hexadecimal.

hex(*, *bytes_per_sep=1*)

hex(*sep*, *bytes_per_sep=1*)

Retorna um objeto string contendo dois dígitos hexadecimais para cada byte na instância.

```
>>> bytearray(b'\xf0\xf1\xf2').hex()
'f0f1f2'
```

Adicionado na versão 3.5.

Alterado na versão 3.8: Similar a *bytes.hex()*, *bytearray.hex()* agora suporta os parâmetros opcionais *sep* e *bytes_per_sep* para inserir separadores entre bytes na saída hexadecimal.

resize(*size*, */*)

Redimensiona *bytearray* para conter *size* bytes. *size* deve ser maior ou igual a 0.

Se *bytearray* precisar ser reduzido, os bytes além de *size* serão truncados.

Se *bytearray* precisar crescer, todos os novos bytes, aqueles além de *size*, serão definidos como bytes nulos.

Isso equivale a:

```
>>> def resize(ba, size):
...     if len(ba) > size:
...         del ba[size:]
...     else:
...         ba += b'\0' * (size - len(ba))
```

Exemplos:

```
>>> shrink = bytearray(b'abc')
>>> shrink.resize(1)
>>> (shrink, len(shrink))
(bytearray(b'a'), 1)
>>> grow = bytearray(b'abc')
>>> grow.resize(5)
>>> (grow, len(grow))
(bytearray(b'abc\x00\x00'), 5)
```

Adicionado na versão 3.14.

Como os objetos bytearray são sequências de inteiros (semelhante a uma lista), para um objeto bytearray *b*, `b[0]` será um inteiro, enquanto que `b[0:1]` será um objeto bytearray de comprimento 1. (Isso contrasta com as Strings de texto, onde tanto a indexação como o fatiamento produzirão sequências de comprimento 1)

A representação de objetos bytearray utiliza o formato literal bytes (`bytearray(b'...')`), uma vez que muitas vezes é mais útil do que, por exemplo, `bytearray([46, 46, 46])`. Você sempre pode converter um objeto bytearray em uma lista de inteiros usando `list(b)`.

4.10.3 Operações com Bytes e Bytearray

Ambos os tipos, bytes e os objetos bytearray suportam as operações de sequências *comuns*. Os mesmo interagem não apenas com operandos do mesmo tipo, mas com qualquer *objeto bytes ou similar*. Devido a esta flexibilidade, os mesmos podem ser misturados livremente em operações sem causar erros. No entanto, o tipo de retorno do resultado pode depender da ordem dos operandos.

Nota

Os métodos em bytes e objetos Bytearray não aceitam Strings como argumentos, assim como os métodos de Strings não aceitam bytes como argumentos. Por exemplo, devemos escrever:

```
a = "abc"
b = a.replace("a", "f")
```

e:

```
a = b"abc"
b = a.replace(b"a", b"f")
```

Alguns operações com bytes e bytearray presumem o uso de formatos binários compatíveis com ASCII e, portanto, devem ser evitados ao trabalhar com dados binários arbitrários. Essas restrições são abordadas a seguir.

Nota

O uso dessas operações baseadas em ASCII para manipular dados binários que não são armazenados num formato baseado em ASCII poderá resultar na corrupção de dados.

Os métodos a seguir em Bytes e Bytearray podem ser usados com dados binários arbitrários.

`bytes.`**`count`**(*sub*[, *start*[, *end*]])

`bytearray.`**`count`**(*sub*[, *start*[, *end*]])

Retorna o número de ocorrências não sobrepostas de subsequência *sub* no intervalo [*start*, *end*]. Os argumentos opcionais *start* e *end* são interpretados como na notação de fatiamento.

A subsequência a ser procurada poderá ser qualquer *objeto bytes ou similar* ou um inteiro no intervalo de 0 a 255.

Se *sub* estiver vazio, retorna o número de fatias vazias entre os caracteres, que é o comprimento do objeto bytes mais um.

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

`bytes.`**`removeprefix`**(*prefix*, /)

`bytearray.`**`removeprefix`**(*prefix*, /)

Se os dados binários começarem com a string *prefix*, retorna `bytes[len(prefix):]`. Caso contrário, retorna uma cópia dos dados binários originais:

```
>>> b'TestHook'.removeprefix(b'Test')
b'Hook'
```

(continua na próxima página)

(continuação da página anterior)

```
>>> b'BaseTestCase'.removeprefix(b'Test')
b'BaseTestCase'
```

O *prefix* pode ser qualquer *objeto bytes ou similar*.

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

Adicionado na versão 3.9.

`bytes.`**`removesuffix`**(*suffix*, /)

`bytearray.`**`removesuffix`**(*suffix*, /)

Se os dados binários terminarem com a string *suffix* e a *suffix* não estiver vazia, retorna `bytes[:-len(suffix)]`. Caso contrário, retorna uma cópia dos dados binários originais:

```
>>> b'MiscTests'.removesuffix(b'Tests')
b'Misc'
>>> b'TmpDirMixin'.removesuffix(b'Tests')
b'TmpDirMixin'
```

O *suffix* pode ser qualquer *objeto bytes ou similar*.

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

Adicionado na versão 3.9.

`bytes.`**`decode`**(*encoding='utf-8'*, *errors='strict'*)

`bytearray.`**`decode`**(*encoding='utf-8'*, *errors='strict'*)

Retorna os bytes decodificados para uma *str*.

encoding tem como padrão `'utf-8'`; veja *Standard Encodings* para valores possíveis.

errors controla como os erros de decodificação são tratados. Se `'strict'` (o padrão), uma exceção *UnicodeError* é levantada. Outros valores possíveis são `'ignore'`, `'replace'` e qualquer outro nome registrado via *codecs.register_error()*. Veja *Error Handlers* para detalhes.

Por motivos de desempenho, o valor de *errors* não é verificado quanto à validade, a menos que um erro de decodificação realmente ocorra, *Modo de Desenvolvimento do Python* esteja ativado ou uma construção de depuração seja usada.

> **Nota**
>
> Passar o argumento *encoding* para *str* permite decodificar qualquer objeto *objeto bytes ou similar* diretamente, sem a necessidade de criar um objeto `bytes` ou `bytearray` temporário.

Alterado na versão 3.1: Adicionado suporte para argumentos nomeados.

Alterado na versão 3.9: O valor do argumento *errors* agora é verificado em *Modo de Desenvolvimento do Python* e no modo de depuração.

`bytes.`**`endswith`**(*suffix*[, *start*[, *end*]])

bytearray.**endswith**(*suffix*[, *start*[, *end*]])

Retorna `True` se os dados binários encerra com o parâmetro *suffix* especificado, caso contrário retorna `False`. *suffix* também pode ser uma tupla de sufixos para averiguar. Com o parâmetro opcional *start*, a procura começa naquela posição. Com o parâmetro opcional *end*, o método encerra a comparação na posição fornecida.

O sufixo(es) para buscas pode ser qualquer termos *objeto bytes ou similar*.

bytes.**find**(*sub*[, *start*[, *end*]])

bytearray.**find**(*sub*[, *start*[, *end*]])

Retorna o índice mais baixo nos dados onde a sub-sequência *sub* é encontrada, tal que *sub* está contida na fatia `s[start:end]`. Argumentos opcionais *start* e *end* são interpretados como na notação de fatiamento. Retorna `-1` se *sub* não for localizada.

A subsequência a ser procurada poderá ser qualquer *objeto bytes ou similar* ou um inteiro no intervalo de 0 a 255.

Nota

O método `find()` deve ser usado apenas se você precisa saber a posição de *sub*. Para verificar se *sub* é uma substring ou não, use o operador `in`:

```
>>> b'Py' in b'Python'
True
```

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.**index**(*sub*[, *start*[, *end*]])

bytearray.**index**(*sub*[, *start*[, *end*]])

Como `find()`, mas levanta a exceção `ValueError` quando a subsequência não for encontrada.

A subsequência a ser procurada poderá ser qualquer *objeto bytes ou similar* ou um inteiro no intervalo de 0 a 255.

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.**join**(*iterable*, /)

bytearray.**join**(*iterable*, /)

Retorna um objeto bytes ou bytearray que é a concatenação das sequências de dados binários no iterável *iterable*. Uma exceção `TypeError` será levantada se existirem quaisquer valores que não sejam *objeto bytes ou similar*, no *iterable*, incluindo objetos `str`. O separador entre elementos é o conteúdo do objeto bytes ou bytearray que está fornecendo este método.

static bytes.**maketrans**(*from*, *to*, /)

static bytearray.**maketrans**(*from*, *to*, /)

Este método estático retorna uma tabela de tradução usável para `bytes.translate()`, que irá mapear cada caractere em *from* no caractere na mesma posição em *to*; *from* e *to* devem ambos ser *objetos bytes ou similares* e ter o mesmo comprimento.

Adicionado na versão 3.1.

bytes.**partition**(*sep*, /)

bytearray.**partition**(*sep*, /)

Quebra a sequência na primeira ocorrência de *sep*, e retorna uma tupla com 3 elementos contendo a parte antes do separador, o próprio separador, e a parte após o separador. Se o separador não for encontrado, retorna uma tupla com 3 elementos contendo uma cópia da sequência original, seguido de dois bytes ou objetos bytearray vazios.

O separador para buscar pode ser qualquer *objeto bytes ou similar*.

bytes.**replace**(*old*, *new*, *count=-1*, /)

bytearray.**replace**(*old*, *new*, *count=-1*, /)

Retornar uma cópia da sequência com todas as ocorrências de subsequências *antigas* substituídas por *novo*. Se o argumento opcional *count* for fornecido, apenas as primeiras ocorrências de *count* serão substituídas.

A subsequência para buscar e substituição pode ser qualquer termos *objeto bytes ou similar*.

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.**rfind**(*sub*[, *start*[, *end*]])

bytearray.**rfind**(*sub*[, *start*[, *end*]])

Retorna o índice mais alto na sequência onde a subsequência *sub* foi encontrada, de modo que *sub* esteja contido dentro de `s[start:end]`. Os argumentos opcionais *start* e *end* são interpretados como na notação de fatiamento. Caso ocorra algum problema será retornado `-1`.

A subsequência a ser procurada poderá ser qualquer *objeto bytes ou similar* ou um inteiro no intervalo de 0 a 255.

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.**rindex**(*sub*[, *start*[, *end*]])

bytearray.**rindex**(*sub*[, *start*[, *end*]])

Semelhante a *`rfind()`* mas levanta *`ValueError`* quando a subsequência *sub* não é encontrada.

A subsequência a ser procurada poderá ser qualquer *objeto bytes ou similar* ou um inteiro no intervalo de 0 a 255.

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.**rpartition**(*sep*, /)

bytearray.**rpartition**(*sep*, /)

Quebra a sequência na primeira ocorrência de *sep*, e retorna uma tupla com 3 elementos contendo a parte antes do separador, o próprio separador, e a parte após o separador. Se o separador não for encontrado, retorna uma tupla com 3 elementos contendo dois bytes ou objetos bytearray vazios, seguido por uma cópia da sequência original.

O separador para buscar pode ser qualquer *objeto bytes ou similar*.

bytes.**startswith**(*prefix*[, *start*[, *end*]])

bytearray.**startswith**(*prefix*[, *start*[, *end*]])

Retorna `True` se os dados binários começam com o *prefix* especificado, caso contrário retorna `False`. *prefix* também pode ser uma tupla de prefixos para procurar. Com o parâmetro opcional *start*, começa a procura na posição indicada. Com o parâmetro opcional *end*, encerra a procura na posição indicada.

Os prefixos para pesquisar podem ser qualquer *objeto bytes ou similar*.

bytes.**translate**(*table*, /, *delete=b''*)

bytearray.**translate**(*table*, /, *delete=b''*)

Retorna uma cópia dos bytes ou objeto bytearray, onde todas as ocorrências de bytes do argumento opcional *delete* são removidas, e os bytes restantes foram mapeados através da tabela de tradução fornecida, a qual deve ser um objeto bytes de comprimento 256.

Você pode usar o método *`bytes.maketrans()`* para criar uma tabela de tradução.

Define o argumento *table* como `None` para traduções que excluem apenas caracteres:

```
>>> b'leia este curto texto'.translate(None, b'aeiou')
b'l st crt txt'
```

Alterado na versão 3.6: *delete* agora é suportado como um argumento nomeado.

Os seguintes métodos de objetos bytes e bytearray tem comportamentos padrões que presumem o uso de formatos binários compatíveis com ASCII, mas ainda podem ser usados com dados binários arbitrários através da passagem argumentos apropriados. Perceba que todos os métodos de bytearray nesta seção *não* alteram os argumentos, e ao invés disso produzem novos objetos.

`bytes.`**`center`**`(`*width*, *fillbyte=b' '*, `/)`

`bytearray.`**`center`**`(`*width*, *fillbyte=b' '*, `/)`

Retorna uma cópia do objeto centralizado em uma sequência de comprimento *width*. Preenchimento é feito usando o *fillbyte* especificado (o padrão é um espaço ASCII). Para objetos *bytes*, a sequência original é retornada se *width* é menor que ou igual a `len(s)`.

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

`bytes.`**`ljust`**`(`*width*, *fillbyte=b' '*, `/)`

`bytearray.`**`ljust`**`(`*width*, *fillbyte=b' '*, `/)`

Retorna uma cópia do objeto alinhado a esquerda em uma sequência de comprimento *width*. Preenchimento é feito usando o *fillbyte* especificado (o padrão é um espaço ASCII). Para objetos *bytes*, a sequência original é retornada se *width* é menor que ou igual a `len(s)`.

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

`bytes.`**`lstrip`**`(`*bytes=None*, `/)`

`bytearray.`**`lstrip`**`(`*bytes=None*, `/)`

Retorna uma cópia da sequência com bytes iniciais especificados removidos. O argumento *bytes* é uma sequência binária que especifica o conjunto de valores de bytes a serem removidos. Se for omitido ou se for `None`, o argumento *bytes* será considerado como espaço em branco ASCII por padrão para a remoção. O argumento *bytes* não é um prefixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> b'   spacious   '.lstrip()
b'spacious   '
>>> b'www.example.com'.lstrip(b'cmowz.')
b'example.com'
```

A sequência binária de valores de bytes a serem removidos pode ser *objeto bytes ou similar*. Consulte *`removeprefix()`* para um método que removerá uma única string de prefixo em vez de todo um conjunto de caracteres. Por exemplo:

```
>>> b'Arthur: three!'.lstrip(b'Arthur: ')
b'ee!'
>>> b'Arthur: three!'.removeprefix(b'Arthur: ')
b'three!'
```

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.**rjust**(*width*, *fillbyte=b' '*, */*)

bytearray.**rjust**(*width*, *fillbyte=b' '*, */*)

Retorna uma cópia do objeto alinhado a direita em uma sequência de comprimento *width*. Preenchimento é feito usando o *fillbyte* especificado (o padrão é um espaço ASCII). Para objetos *bytes*, a sequência original é retornada se *width* é menor que ou igual a `len(s)`.

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.**rsplit**(*sep=None*, *maxsplit=-1*)

bytearray.**rsplit**(*sep=None*, *maxsplit=-1*)

Divide a sequência binária em subsequência do mesmo tipo, usando *sep* como um delimitador de string. Se *maxsplit* é fornecido, no máximo *maxsplit* divisões são feitas, aquelas que estão *mais à direita*. Se *sep* não é especificado ou é `None`, qualquer subsequência consistindo unicamente de espaço em branco ASCII é um separador. Exceto por dividir pela direita, *rsplit()* comporta-se como *split()*, o qual é descrito em detalhes abaixo.

bytes.**rstrip**(*bytes=None*, */*)

bytearray.**rstrip**(*bytes=None*, */*)

Retorna uma cópia da sequência com bytes finais especificados removidos. O argumento *bytes* é uma sequência binária que especifica o conjunto de valores de bytes a serem removidos. Se for omitido ou se for `None`, o argumento *bytes* será considerado como espaço em branco ASCII por padrão para a remoção. O argumento *bytes* não é um sufixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> b'   spacious   '.rstrip()
b'   spacious'
>>> b'mississippi'.rstrip(b'ipz')
b'mississ'
```

A sequência binária de valores de bytes a serem removidos pode ser *objeto bytes ou similar*. Consulte *removesuffix()* para um método que removerá uma única string de sufixo em vez de todo um conjunto de caracteres. Por exemplo:

```
>>> b'Monty Python'.rstrip(b' Python')
b'M'
>>> b'Monty Python'.removesuffix(b' Python')
b'Monty'
```

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.**split**(*sep=None*, *maxsplit=-1*)

bytearray.**split**(*sep=None*, *maxsplit=-1*)

Divide a sequência binária em subsequências do mesmo tipo, usando *sep* como o delimitador de string. Se *maxsplit* é fornecido e não-negativo, no máximo *maxsplit* divisões são feitas (logo, a lista terá no máximo `maxsplit+1` elementos). Se *maxsplit* não é especificado ou é `-1`, então não existe limite no número de divisões (todas as divisões possíveis são feitas).

Se *sep* é fornecido, delimitadores consecutivos não são agrupados juntos e eles são destinados para delimitar strings vazias (por exemplo, `b'1,,2'.split(b',')` retorna `[b'1', b'', b'2']`). O argumento *sep* pode consistir de uma sequência de múltiplos bytes como um único delimitador. Separar uma sequência vazia

com um separador especificado retorna `[b'']` ou `[bytearray(b'')]` dependendo do tipo do objeto que está sendo separado. O argumento *sep* pode ser qualquer *objeto bytes ou similar*.

Por exemplo:

```
>>> b'1,2,3'.split(b',')
[b'1', b'2', b'3']
>>> b'1,2,3'.split(b',', maxsplit=1)
[b'1', b'2,3']
>>> b'1,2,,3,'.split(b',')
[b'1', b'2', b'', b'3', b'']
>>> b'1<>2<>3<4'.split(b'<>')
[b'1', b'2', b'3<4']
```

Se *sep* não for especificado ou for `None`, um algoritmo diferente de separação é aplicado: ocorrências consecutivas de espaços em branco ASCII são consideradas como um separador único, e o resultado não conterá strings vazias no início ou no final, se a sequência tiver espaços em branco no início ou no final. Consequentemente, separar uma sequência vazia ou uma sequência que consiste apenas de espaços em branco ASCII sem um separador especificado, retorna `[]`.

Por exemplo:

```
>>> b'1 2 3'.split()
[b'1', b'2', b'3']
>>> b'1 2 3'.split(maxsplit=1)
[b'1', b'2 3']
>>> b'   1   2   3   '.split()
[b'1', b'2', b'3']
```

`bytes.`**`strip`**(*bytes=None*, */*)

`bytearray.`**`strip`**(*bytes=None*, */*)

Retorna uma cópia da sequência com bytes iniciais e finais especificados removidos. O argumento *bytes* é uma sequência binária que especifica o conjunto de valores de bytes a serem removidos. Se for omitido ou se for `None`, o argumento *bytes* será considerado como espaço em branco ASCII por padrão para a remoção. O argumento *bytes* não é um prefixo ou sufixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> b'   spacious   '.strip()
b'spacious'
>>> b'www.example.com'.strip(b'cmowz.')
b'example'
```

A sequência binária de valores de bytes a serem removidos pode ser qualquer *objeto bytes ou similar*.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

O seguintes métodos de bytes e objetos bytearray presumem o uso de formatos binários compatíveis com ASCII e não devem ser aplicados a dados binários arbitrários. Perceba que todos os métodos de bytearray nesta seção *não* alteram os argumentos, e ao invés disso produzem novos objetos.

`bytes.`**`capitalize`**()

`bytearray.`**`capitalize`**()

Retorna uma cópia da sequência com cada byte interpretado como um caractere ASCII, e o primeiro byte em letra maiúscula e o resto em letras minúsculas. Valores de bytes que não são ASCII são passados adiante sem mudanças.

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.**expandtabs**(*tabsize=8*)

bytearray.**expandtabs**(*tabsize=8*)

Retorna uma cópia da sequência onde todos os caracteres de tabulação ASCII são substituídos por um ou mais espaços ASCII, dependendo da coluna atual e do tamanho fornecido para a tabulação. Posições de tabulação ocorrem a cada *tabsize* bytes (o padrão é 8, dada as posições de tabulação nas colunas 0, 8, 16 e assim por diante). Para expandir a sequência, a coluna atual é definida como zero e a sequência é examinada byte por byte. Se o byte é um caractere ASCII de tabulação (`b'\t'`), um ou mais caracteres de espaço são inseridos no resultado até que a coluna atual seja igual a próxima posição de tabulação. (O caractere de tabulação em si não é copiado.) Se o byte atual é um caractere ASCII de nova linha (`b'\n'`) ou de retorno (`b'\r'`), ele é copiado e a coluna atual é redefinida para zero. Qualquer outro byte é copiado sem ser modificado e a coluna atual é incrementada em uma unidade independentemente de como o byte é representado quanto é impresso:

```
>>> b'01\t012\t0123\t01234'.expandtabs()
b'01      012     0123    01234'
>>> b'01\t012\t0123\t01234'.expandtabs(4)
b'01  012 0123    01234'
```

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.**isalnum**()

bytearray.**isalnum**()

Retorna `True` se todos os bytes na sequência são caracteres alfabéticos ASCII ou dígitos decimais ASCII e a sequência não é vazia, `False` caso contrário. Caracteres alfabéticos ASCII são aqueles valores de byte na sequência `b'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'`. Dígitos decimais ASCII são aqueles valores de byte na sequência `b'0123456789'`.

Por exemplo:

```
>>> b'ABCabc1'.isalnum()
True
>>> b'ABC abc1'.isalnum()
False
```

bytes.**isalpha**()

bytearray.**isalpha**()

Retorna `True` se todos os bytes na sequência são caracteres alfabéticos ASCII e a sequência não é vazia, `False` caso contrário. Caracteres alfabéticos ASCII são aqueles cujo valor dos bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

Por exemplo:

```
>>> b'ABCabc'.isalpha()
True
>>> b'ABCabc1'.isalpha()
False
```

bytes.**isascii**()

bytearray.**isascii**()

Retorna `True` se a sequência é vazia ou todos os bytes na sequência são ASCII, `False` caso contrário. Bytes ASCII estão no intervalo 0-0x7F.

Adicionado na versão 3.7.

bytes.**isdigit**()

bytearray.**isdigit**()

Retorna `True` se todos os bytes na sequência são dígitos decimais ASCII e a sequência não é vazia, `False` caso contrário. Dígitos decimais ASCII são aqueles cujos valores dos bytes estão na sequência `b'0123456789'`.

Por exemplo:

```
>>> b'1234'.isdigit()
True
>>> b'1.23'.isdigit()
False
```

bytes.**islower**()

bytearray.**islower**()

Retorna `True` se existe pelo menos um caractere minúsculo ASCII na sequência e nenhum caractere maiúsculo ASCII, `False` caso contrário.

Por exemplo:

```
>>> b'hello world'.islower()
True
>>> b'Hello world'.islower()
False
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

bytes.**isspace**()

bytearray.**isspace**()

Retorna `True` se todos os bytes na sequência são espaço em branco ASCII e a sequência não é vazia, `False` caso contrário. Caracteres de espaço em branco ASCII são aqueles cujos valores de bytes estão na sequência `b' \t\n\r\x0b\f'` (espaço, tabulação, nova linha, retorno do cursor, tabulação vertical, formulário de entrada).

bytes.**istitle**()

bytearray.**istitle**()

Retorna `True` se a sequência é titlecased ASCII e a sequência não é vazia, `False` caso contrário. Veja *bytes.title()* para mais detalhes sobre a definição de "titlecased".

Por exemplo:

```
>>> b'Hello World'.istitle()
True
>>> b'Hello world'.istitle()
False
```

bytes.**isupper**()

bytearray.**isupper**()

Retorna `True` se existe pelo menos um caractere maiúsculo alfabético ASCII na sequência e nenhum caractere minúsculo ASCII, `False` caso contrário.

Por exemplo:

```
>>> b'HELLO WORLD'.isupper()
True
>>> b'Hello world'.isupper()
False
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

`bytes.`**`lower`**`()`

`bytearray.`**`lower`**`()`

Retorna uma cópia da sequência com todos os caracteres maiúsculos ASCII convertidos para os seus correspondentes caracteres minúsculos.

Por exemplo:

```
>>> b'Hello World'.lower()
b'hello world'
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

`bytes.`**`splitlines`**`(`*keepends=False*`)`

`bytearray.`**`splitlines`**`(`*keepends=False*`)`

Retorna uma lista das linhas na sequência binária, quebrando nos limites ASCII das linhas. Este método usa a abordagem de *novas linhas universais* para separar as linhas. Quebras de linhas não são incluídas na lista resultante a não ser que *keepends* seja fornecido e verdadeiro.

Por exemplo:

```
>>> b'ab c\n\nde fg\rkl\r\n'.splitlines()
[b'ab c', b'', b'de fg', b'kl']
>>> b'ab c\n\nde fg\rkl\r\n'.splitlines(keepends=True)
[b'ab c\n', b'\n', b'de fg\r', b'kl\r\n']
```

Ao contrário de *`split()`*, quando uma string delimitadora *sep* é fornecida, este método retorna uma lista vazia para a string vazia, e uma quebra de linha terminal não resulta em uma linha extra:

```
>>> b"".split(b'\n'), b"Two lines\n".split(b'\n')
([b''], [b'Two lines', b''])
>>> b"".splitlines(), b"One line\n".splitlines()
([], [b'One line'])
```

`bytes.`**`swapcase`**`()`

`bytearray.`**`swapcase`**`()`

Retorna uma cópia da sequência com todos os caracteres minúsculos ASCII convertidos para caracteres maiúsculos correspondentes, e vice-versa.

Por exemplo:

```
>>> b'Hello World'.swapcase()
b'hELLO wORLD'
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

Ao contrário de *str.swapcase()*, é sempre fato que `bin.swapcase().swapcase() == bin` para as versões binárias. Conversões maiúsculas/minúsculas são simétricas em ASCII, apesar que isso não é geralmente verdade para pontos de código arbitrários Unicode.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

`bytes.`**`title`**`()`

`bytearray.`**`title`**`()`

Retorna uma versão titlecased da sequência binária, onde palavras iniciam com um caractere ASCII com letra maiúscula e os caracteres restantes são em letras minúsculas. Bytes quem não possuem diferença entre maiúscula/minúscula não são alterados.

Por exemplo:

```
>>> b'Hello world'.title()
b'Hello World'
```

Caracteres minúsculos ASCII são aqueles cujos valores de byte estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de byte estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. Todos os outros valores de bytes ignoram maiúsculas/minúsculas.

O algoritmo usa uma definição simples independente de idioma para uma palavra, como grupos de letras consecutivas. A definição funciona em muitos contextos, mas isso significa que apóstrofes em contrações e possessivos formam limites de palavras, os quais podem não ser o resultado desejado:

```
>>> b"they're bill's friends from the UK".title()
b"They're Bill's Friends From The Uk"
```

Uma solução alternativa para os apóstrofes pode ser construída usando expressões regulares:

```
>>> import re
>>> def titlecase(s):
...     return re.sub(rb"[A-Za-z]+('[A-Za-z]+)?",
...                   lambda mo: mo.group(0)[0:1].upper() +
...                              mo.group(0)[1:].lower(),
...                   s)
...
>>> titlecase(b"they're bill's friends.")
b"They're Bill's Friends."
```

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

`bytes.`**`upper`**`()`

`bytearray.`**`upper`**`()`

Retorna uma cópia da sequência com todos os caracteres minúsculos ASCII convertidos para sua contraparte maiúscula correspondente.

Por exemplo:

```
>>> b'Hello World'.upper()
b'HELLO WORLD'
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

`bytes.`**`zfill`**`(`*width*`, /)`

`bytearray.`**`zfill`**`(`*width*`, /)`

Retorna uma cópia da sequência preenchida a esquerda com dígitos `b'0'` ASCII para produzir uma sequência se comprimento *width*. Um sinal de prefixo inicial (`b'+'`/`b'-'`) é controlado através da inserção de preenchimento *depois* do caractere de sinal, ao invés de antes. Para objetos *bytes*, a sequência original é retornada se *width* é menor que ou igual a `len(seq)`.

Por exemplo:

```
>>> b"42".zfill(5)
b'00042'
>>> b"-42".zfill(5)
b'-0042'
```

> **Nota**
>
> A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

4.10.4 Estilo de Formatação de bytes `printf`-style

> **Nota**
>
> As operações de formatação descritas aqui exibem uma variedade de peculiaridades que levam a uma série de erros comuns (como a falha na exibição de Tuplas e Dicionários corretamente). Se o valor que está sendo impresso puder ser uma tupla ou dicionário, envolva-o numa tupla.

Objetos Bytes (`bytes`/`bytearray`) possuem uma operação integrada exclusiva: o operador `%` (modulo). Isso também é conhecido como o bytes de *formatação* ou o operador de *interpolação*. Dado `format % values` (onde *format* é um objeto bytes), as especificações de conversão `%` em *format* são substituídas por zero ou mais elementos de *values*. O efeito é semelhante ao uso da função `sprintf()` na linguagem C.

Se *format* requer um único argumento, *values* poderá ser um único objeto não-tuple.[Página 69, 5] Caso contrário, *values* deverá ser uma tupla com exatamente o número de itens especificados pelo objeto de formatação de Bytes, ou um único objeto de mapeamento (por exemplo, um dicionário).

Um especificador de conversão contém dois ou mais caracteres e tem os seguintes componentes, que devem aparecer nesta ordem:

1. O caractere `'%'`, que determina o início do especificador.
2. Mapeamento de chaves (opcional), consistindo de uma sequência entre parênteses de caracteres (por exemplo, `(algumnome)`).

3. Flags de conversão (opcional), que afetam o resultado de alguns tipos de conversão.
4. Largura mínima do campo (opcional). Se for especificado como um `'*'` (asterisco), a largura real será lida a partir do próximo elemento da tupla em *values* e o objeto a converter virá após a largura mínima do campo e a precisão que é opcional.
5. Precisão (opcional), fornecido como uma `'.'` (ponto) seguido pela precisão. Se determinado como um `'*'` (um asterisco), a precisão real será lida a partir do próximo elemento da tupla em *values*, e o valor a converter virá após a precisão.
6. Modificador de comprimento (opcional).
7. Tipo de conversão.

Quando o argumento a direita é um dicionário (ou outro tipo de mapeamento), então o formato no objeto bytes *deve* incluir um mapeamento de chaves entre parêntesis neste dicionário inserido imediatamente após o caractere `'%'`. O mapeamento de chaves seleciona o valor a ser formatado a partir do mapeamento. Por exemplo:

```
>>> print(b'%(language)s has %(number)03d quote types.' %
...       {b'language': b"Python", b"number": 2})
b'Python has 002 quote types.'
```

Nesse caso, nenhum especificador `*` poderá ocorrer num formato (uma vez que eles exigem uma lista de parâmetros sequenciais).

Os caracteres flags de conversão são:

Sinalizador	Significado
`'#'`	A conversão de valor usará o "formulário alternativo" (conforme definido abaixo).
`'0'`	A conversão será preenchida por zeros para valores numéricos.
`'-'`	O valor convertido será ajustado à esquerda (substitui a conversão `'0'` se ambos forem fornecidos).
`' '`	(um espaço) Um espaço em branco deverá ser deixado antes de um número positivo (ou uma string vazia) produzido por uma conversão assinada.
`'+'`	Um sinal de caractere (`'+'` ou `'-'`) precederá a conversão (substituindo o sinalizador de "espaço").

Um modificador de comprimento (`h`, `l`, ou `L`) pode estar presente, mas será ignorado, pois o mesmo não é necessário para o Python – então, por exemplo, `%ld` é idêntico a `%d`.

Os tipos de conversão são:

Conversão	Significado	Notas
`'d'`	Número decimal inteiro sinalizador.	
`'i'`	Número decimal inteiro sinalizador.	
`'o'`	Valor octal sinalizador.	(1)
`'u'`	Tipo obsoleto – é idêntico a `'d'`.	(8)
`'x'`	Hexadecimal com sinal (minúsculo).	(2)
`'X'`	Hexadecimal com sinal (maiúsculo).	(2)
`'e'`	Formato exponencial de ponto flutuante (minúsculo).	(3)
`'E'`	Formato exponencial de ponto flutuante (maiúsculo).	(3)
`'f'`	Formato decimal de ponto flutuante.	(3)
`'F'`	Formato decimal de ponto flutuante.	(3)
`'g'`	O formato de ponto flutuante. Usa o formato exponencial em minúsculo se o expoente for inferior a -4 ou não inferior a precisão, formato decimal, caso contrário.	(4)
`'G'`	Formato de ponto flutuante. Usa o formato exponencial em maiúsculo se o expoente for inferior a -4 ou não inferior que a precisão, formato decimal, caso contrário.	(4)
`'c'`	Byte simples (aceita objetos inteiros ou de byte único).	
`'b'`	Bytes (qualquer objeto que segue o buffer protocol o que possui `__bytes__()`).	(5)
`'s'`	`'s'` é um alias para `'b'` e só deve ser usado para bases de código Python2/3.	(6)
`'a'`	Bytes (converte qualquer objeto Python usando `repr(obj).encode('ascii', 'backslashreplace')`).	(5)
`'r'`	`'r'` é um alias para `'a'` e só deve ser usado para bases de código Python2/3.	(7)
`'%'`	Nenhum argumento é convertido, resultando em um caractere `'%'` no resultado.	

Notas:

(1) A forma alternativa faz com que um especificador octal principal (`'0o'`) seja inserido antes do primeiro dígito.

(2) O formato alternativo produz um `'0x'` ou `'0X'` (dependendo se o formato `'x'` or `'X'` foi usado) para ser inserido antes do primeiro dígito.

(3) A forma alternativa faz com que o resultado sempre contenha um ponto decimal, mesmo que nenhum dígito o siga.

A precisão determina o número de dígitos após o ponto decimal e o padrão é 6.

(4) A forma alternativa faz com que o resultado sempre contenha um ponto decimal e os zeros à direita não sejam removidos, como de outra forma seriam.

A precisão determina o número de dígitos significativos antes e depois do ponto decimal e o padrão seja 6.

(5) Se a precisão for `N`, a saída será truncada em `N` caracteres.

(6) `b'%s'` está descontinuado, mas não será removido durante a versão 3.x.

(7) `b'%r'` entrou em desuso, mas não serão removidos na versão 3.x.

(8) Veja **PEP 237**.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

Ver também

PEP 461 - Adicionar formatação % para to bytes e bytearray

Adicionado na versão 3.5.

4.10.5 Memory Views

O objeto *memoryview* permite que o código Python acesse os dados internos de um objeto que suporte o buffer protocol sem copiá-lo.

class memoryview(*object*)

Cria uma *memoryview* que referencia *object*. *object* deve ter suporte ao protocolo de buffer. Objetos embutidos que suportam o protocolo de buffer incluem *bytes* e *bytearray*.

Uma *memoryview* tem a noção de um *elemento*, o qual é a unidade de memória atômica manipulada pelo objeto de origem *object*. Para muitos tipos simples tais como *bytes* e *bytearray*, um elemento é um byte único, mas outros tipos tais como *array.array* podem ter elementos maiores.

`len(view)` é igual ao comprimento de *tolist*, que é a representação de lista aninhada da view. Se `view.ndim = 1`, isso é igual ao número de elementos na visualização.

Alterado na versão 3.12: Se `view.ndim == 0`, `len(view)` agora levanta *TypeError* em vez de retornar 1.

O atributo *itemsize* vai lhe dar um número de bytes em um único elemento.

Um *memoryview* suporta fatiamento e indexação para expor seus dados. Fatiamento unidimensional irá resultar em uma subview:

```
>>> v = memoryview(b'abcefg')
>>> v[1]
98
>>> v[-1]
103
>>> v[1:4]
<memory at 0x7f3ddc9f4350>
>>> bytes(v[1:4])
b'bce'
```

Se *format* é um dos especificadores de formatação nativos do módulo *struct*, indexar com um inteiro ou uma tupla de inteiros também é suportado, e retorna um *element* único com o tipo correto. Memoryviews unidimensionais podem ser indexadas com um inteiro ou uma tupla contendo um inteiro. Memoryviews multi-dimensionais podem ser indexadas com tuplas de exatamente *ndim* inteiros, onde *ndim* é o número de dimensões. Memoryviews zero-dimensionais podem ser indexadas com uma tupla vazia.

Aqui temos um exemplo usando um formato não-byte:

```
>>> import array
>>> a = array.array('l', [-11111111, 22222222, -33333333, 44444444])
>>> m = memoryview(a)
>>> m[0]
-11111111
>>> m[-1]
44444444
>>> m[::2].tolist()
[-11111111, -33333333]
```

Se o objeto subjacente é gravável, a memoryview suporta atribuição de fatias unidimensionais. Redimensionamento não é permitido:

```
>>> data = bytearray(b'abcefg')
>>> v = memoryview(data)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> v.readonly
False
>>> v[0] = ord(b'z')
>>> data
bytearray(b'zbcefg')
>>> v[1:4] = b'123'
>>> data
bytearray(b'z123fg')
>>> v[2:3] = b'spam'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: memoryview assignment: lvalue and rvalue have different␣
↪structures
>>> v[2:6] = b'spam'
>>> data
bytearray(b'z1spam')
```

Memoryviews unidimensionais de tipos *hasheáveis* (somente leitura) com formatos 'B', 'b' ou 'c' também são hasheáveis. O hash é definido como `hash(m) == hash(m.tobytes())`:

```
>>> v = memoryview(b'abcefg')
>>> hash(v) == hash(b'abcefg')
True
>>> hash(v[2:4]) == hash(b'ce')
True
>>> hash(v[::-2]) == hash(b'abcefg'[::-2])
True
```

Alterado na versão 3.3: Memoryviews unidimensionais agora podem ser fatiadas. Memoryviews unidimensionais com formatos 'B', 'b' ou 'c' agora são *hasheáveis*.

Alterado na versão 3.4: O memoryview agora é registrado automaticamente como uma classe *collections.abc.Sequence*

Alterado na versão 3.5: Atualmente, os memoryvies podem ser indexadas com uma tupla de números inteiros.

Alterado na versão 3.14: memoryview é agora um *tipo genérico*.

memoryview possui vários métodos:

__eq__(*exporter*)

Uma memoryview e um exportador **PEP 3118** são iguais se as suas formas são equivalentes e se todos os valores correspondentes são iguais quando os códigos de formatação dos respectivos operadores são interpretados usando a sintaxe *struct*.

Para o subconjunto *struct* a formatação de Strings atualmente suportadas por *tolist()*, `v` e `w` são iguais se `v.tolist() == w.tolist()`:

```
>>> import array
>>> a = array.array('I', [1, 2, 3, 4, 5])
>>> b = array.array('d', [1.0, 2.0, 3.0, 4.0, 5.0])
>>> c = array.array('b', [5, 3, 1])
>>> x = memoryview(a)
>>> y = memoryview(b)
>>> x == a == y == b
True
>>> x.tolist() == a.tolist() == y.tolist() == b.tolist()
True
```

(continua na próxima página)

(continuação da página anterior)

```
>>> z = y[::-2]
>>> z == c
True
>>> z.tolist() == c.tolist()
True
```

Se qualquer string de formatação não for suportada pelo módulo *struct*, então os objetos irão sempre comparar como diferentes (mesmo se as strings de formatação e o conteúdo do buffer são idênticos):

```
>>> from ctypes import BigEndianStructure, c_long
>>> class BEPoint(BigEndianStructure):
...     _fields_ = [("x", c_long), ("y", c_long)]
...
>>> point = BEPoint(100, 200)
>>> a = memoryview(point)
>>> b = memoryview(point)
>>> a == point
False
>>> a == b
False
```

Perceba que, assim como com números de ponto flutuante, `v is w` *não* implica em `v == w` para objetos memoryview.

Alterado na versão 3.3: Versões anteriores comparavam a memória bruta desconsiderando o formato do item e estrutura lógica do array.

tobytes(*order='C'*)

Retorna os dados no buffer como um bytestring. Isso é equivalente a chamar o construtor de *bytes* na memoryview.

```
>>> m = memoryview(b"abc")
>>> m.tobytes()
b'abc'
>>> bytes(m)
b'abc'
```

Para arrays não contíguos, o resultado é igual a representação de lista achatada com todos os elementos convertidos para bytes. *tobytes()* suporta todos os formatos de strings, incluindo aqueles que não estão na sintaxe do módulo *struct*.

Adicionado na versão 3.8: *order* pode ser {'C', 'F', 'A'}. Quando *order* é 'C' ou 'F', os dados do array original são convertidos para a ordem de C our Fortran. Para views contígua, 'A' retorna uma cópia exata da memória física. Em particular, ordem de Fortran em memória é preservada. Para views não contígua, os dados são convertidos primeiro para C. *order=None* é o mesmo que *order='C'*.

hex(**, bytes_per_sep=1*)

hex(*sep, bytes_per_sep=1*)

Retorna um objeto string contendo dois dígitos hexadecimais para cada byte no buffer.

```
>>> m = memoryview(b"abc")
>>> m.hex()
'616263'
```

Adicionado na versão 3.5.

Alterado na versão 3.8: Similar a *bytes.hex()*, *memoryview.hex()* agora suporta os parâmetros opcionais *sep* e *bytes_per_sep* para inserir separadores entre bytes na saída hexadecimal.

tolist()

Retorna os dados no buffer como uma lista de elementos.

```
>>> memoryview(b'abc').tolist()
[97, 98, 99]
>>> import array
>>> a = array.array('d', [1.1, 2.2, 3.3])
>>> m = memoryview(a)
>>> m.tolist()
[1.1, 2.2, 3.3]
```

Alterado na versão 3.3: *tolist()* agora suporta todos os formatos nativos de caracteres únicos na sintaxe do módulo *struct*, assim como representações multi-dimensionais.

toreadonly()

Retorna uma versão somente leitura do objeto memoryview. O objeto memoryview original não é alterado.

```
>>> m = memoryview(bytearray(b'abc'))
>>> mm = m.toreadonly()
>>> mm.tolist()
[97, 98, 99]
>>> mm[0] = 42
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot modify read-only memory
>>> m[0] = 43
>>> mm.tolist()
[43, 98, 99]
```

Adicionado na versão 3.8.

release()

Libera o buffer subjacente exposto pelo objeto memoryview. Muitos objetos aceitam ações especiais quando a view é mantida com eles (por exemplo, um *bytearray* iria temporariamente proibir o redimensionamento); portanto, chamar release() é útil para remover essas restrições (e liberar quaisquer recursos pendurados) o mais breve possível.

Depois que este método foi chamado, qualquer operação posterior na visão levanta um *ValueError* (exceto *release()*, o qual pode ser chamado múltiplas vezes):

```
>>> m = memoryview(b'abc')
>>> m.release()
>>> m[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: operation forbidden on released memoryview object
```

O protocolo de gerenciamento de contexto pode ser usado para efeitos similares, usando a instrução `with`:

```
>>> with memoryview(b'abc') as m:
...     m[0]
...
97
```

(continua na próxima página)

(continuação da página anterior)

```
>>> m[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: operation forbidden on released memoryview object
```

Adicionado na versão 3.2.

cast(*format*, /)

cast(*format*, *shape*, /)

Converte uma memoryview para um novo formato ou forma. *shape* por padrão é `[byte_length//new_itemsize]`, o que significa que a visão resultante será unidimensional. O valor de retorno é uma nova memoryview, mas o buffer por si mesmo não é copiado. Conversões suportadas são 1D -> C-*contíguo* r C-contíguo -> 1D.

O formato de destino é restrito a um elemento em formato nativo na sintaxe `struct`. Um dos formatos deve ser um formato de byte ('B', 'b' ou 'c'). O comprimento de bytes do resultado deve ser o mesmo que o comprimento original. Observe que todos os comprimentos de bytes podem depender do sistema operacional.

Converte de 1D/long para 1D/unsigned bytes:

```
>>> import array
>>> a = array.array('l', [1,2,3])
>>> x = memoryview(a)
>>> x.format
'l'
>>> x.itemsize
8
>>> len(x)
3
>>> x.nbytes
24
>>> y = x.cast('B')
>>> y.format
'B'
>>> y.itemsize
1
>>> len(y)
24
>>> y.nbytes
24
```

Converte de 1D/unsigned bytes para 1D/char:

```
>>> b = bytearray(b'zyz')
>>> x = memoryview(b)
>>> x[0] = b'a'
Traceback (most recent call last):
  ...
TypeError: memoryview: invalid type for format 'B'
>>> y = x.cast('c')
>>> y[0] = b'a'
>>> b
bytearray(b'ayz')
```

Converte de 1D/bytes para 3D/ints para 1D/signed char:

```
>>> import struct
>>> buf = struct.pack("i"*12, *list(range(12)))
>>> x = memoryview(buf)
>>> y = x.cast('i', shape=[2,2,3])
>>> y.tolist()
[[[0, 1, 2], [3, 4, 5]], [[6, 7, 8], [9, 10, 11]]]
>>> y.format
'i'
>>> y.itemsize
4
>>> len(y)
2
>>> y.nbytes
48
>>> z = y.cast('b')
>>> z.format
'b'
>>> z.itemsize
1
>>> len(z)
48
>>> z.nbytes
48
```

Converte 1D/unsigned long para 2D/unsigned long:

```
>>> buf = struct.pack("L"*6, *list(range(6)))
>>> x = memoryview(buf)
>>> y = x.cast('L', shape=[2,3])
>>> len(y)
2
>>> y.nbytes
48
>>> y.tolist()
[[0, 1, 2], [3, 4, 5]]
```

Adicionado na versão 3.3.

Alterado na versão 3.5: O formato fonte não é mais restrito ao converter para uma visão de byte.

count(*value*, /)

Conta o número de ocorrências de *value*.

Adicionado na versão 3.14.

index(*value*, *start=0*, *stop=sys.maxsize*, /)

Retorna o índice da primeira ocorrência de *value* (no índice *index* ou depois dele e antes do índice *stop*).

Levanta uma exceção *ValueError* se *value* não puder ser encontrado.

Adicionado na versão 3.14.

Existem também diversos atributos somente leitura disponíveis:

obj

O objeto subjacente da memoryview:

```
>>> b  = bytearray(b'xyz')
>>> m = memoryview(b)
>>> m.obj is b
True
```

Adicionado na versão 3.3.

nbytes

`nbytes == product(shape) * itemsize == len(m.tobytes())`. Este é a quantidade de espaço em bytes que o array deve usar em uma representação contígua. Ela não é necessariamente igual a `len(m)`:

```
>>> import array
>>> a = array.array('i', [1,2,3,4,5])
>>> m = memoryview(a)
>>> len(m)
5
>>> m.nbytes
20
>>> y = m[::2]
>>> len(y)
3
>>> y.nbytes
12
>>> len(y.tobytes())
12
```

Arrays Multi-dimensional:

```
>>> import struct
>>> buf = struct.pack("d"*12, *[1.5*x for x in range(12)])
>>> x = memoryview(buf)
>>> y = x.cast('d', shape=[3,4])
>>> y.tolist()
[[0.0, 1.5, 3.0, 4.5], [6.0, 7.5, 9.0, 10.5], [12.0, 13.5, 15.0, 16.5]]
>>> len(y)
3
>>> y.nbytes
96
```

Adicionado na versão 3.3.

readonly

Um bool que indica se a memória é somente leitura.

format

Uma string contendo o formato (no estilo do módulo *struct*) para cada elemento na visão. Uma memoryview pode ser criada a partir de exportadores com strings de formato arbitrário, mas alguns métodos (ex: *tolist()*) são restritos a formatos de elementos nativos.

Alterado na versão 3.3: formato `'B'` agora é tratado de acordo com a sintaxe do módulo struct. Isso significa que `memoryview(b'abc')[0] == b'abc'[0] == 97`.

itemsize

O tamanho em Bytes de cada elemento do memoryview:

```
>>> import array, struct
>>> m = memoryview(array.array('H', [32000, 32001, 32002]))
>>> m.itemsize
```

(continua na próxima página)

(continuação da página anterior)

```
2
>>> m[0]
32000
>>> struct.calcsize('H') == m.itemsize
True
```

ndim

Um número inteiro que indica quantas dimensões de uma matriz multidimensional a memória representa.

shape

Uma tupla de inteiros de comprimento *ndim* dando a forma da memória como uma matriz N-dimensional.

Alterado na versão 3.3: Uma tupla vazia ao invés de `None` quando ndim = 0.

strides

Uma tupla de inteiros de comprimento de *ndim* dando o tamanho em bytes para acessar cada elemento de cada dimensão da matriz.

Alterado na versão 3.3: Uma tupla vazia ao invés de `None` quando ndim = 0.

suboffsets

Usado internamente para estilos de Arrays PIL. O valor é apenas informativo.

c_contiguous

Um bool que indica se a memória é *contígua* C.

Adicionado na versão 3.3.

f_contiguous

Um bool que indica se a memória é *contígua* Fortran.

Adicionado na versão 3.3.

contiguous

Um bool que indica se a memória é *contígua*.

Adicionado na versão 3.3.

4.11 Tipo conjuntos — `set`, `frozenset`

Um objeto *conjunto* é uma coleção não ordenada de objetos *hasheáveis* distintos. Usos comuns incluem testes de associação, remover duplicatas de uma sequência e computar operações matemáticas tais como interseção, união, diferença e diferença simétrica. (Para outros tipos de contêineres veja as classes embutidas *dict*, *list* e *tuple*, e o módulo *collections*.)

Assim como outras coleções, conjuntos suportam `x in set`, `len(set)` e `for x in set`. Sendo uma coleção não ordenada, conjuntos não armazenam posição de elementos ou ordem de inserção. Portanto, conjuntos não suportam indexação, fatiamento ou outros comportamentos de sequências ou similares.

Existem atualmente dois tipos de conjuntos embutidos, *set* e *frozenset*. O tipo *set* é mutável – o conteúdo pode ser alterado usando métodos como `add()` e `remove()`. Como ele é mutável, ele não tem valor hash e não pode ser usado como chave de dicionário ou um elemento de um outro conjunto. O tipo *frozenset* é imutável e *hasheável* – seu conteúdo não pode ser alterado depois de ter sido criado; ele pode então ser usado como chave de dicionário ou como um elemento de outro conjunto.

Conjuntos não vazios (que não sejam frozensets) podem ser criados posicionando uma lista de elementos separados por vírgula dentro de chaves, por exemplo: `{'jack', 'sjoerd'}`, além do construtor *set*.

Os construtores de ambas as classes funcionam da mesma forma:

class set(*iterable=()*, */*)

class frozenset(*iterable=()*, */*)

Retorna um novo objeto set ou frozenset, cujos elementos são obtidos a partir de um *iterable*. Os elementos de um conjunto devem ser *hasheáveis*. Para representar conjuntos de sets, os sets internos devem ser objetos `frozenset`. Se *iterable* não for especificado, um novo conjunto vazio é retornado.

Conjuntos podem ser criados de várias formas:

- Usar uma lista de elementos separados por vírgulas entre chaves: `{'jack', 'sjoerd'}`
- Usar uma compreensão de conjunto: `{c for c in 'abracadabra' if c not in 'abc'}`
- Usar o construtor da classe: `set()`, `set('foobar')`, `set(['a', 'b', 'foo'])`

Instâncias de `set` e `frozenset` fornecem as seguintes operações:

len(s)

Retorna o número de elementos no set *s* (cardinalidade de *s*).

x in s

Testa se *x* pertence a *s*.

x not in s

Testa se *x* não pertence a *s*.

isdisjoint(*other*, */*)

Retorna `True` se o conjunto não tem elementos em comum com *other*. Conjuntos são disjuntos se e somente se a sua interseção é o conjunto vazio.

issubset(*other*, */*)

set <= other

Testa se cada elemento do conjunto está contido em *other*.

set < other

Testa se o conjunto é um subconjunto próprio de *other*, isto é, `set <= other and set != other`.

issuperset(*other*, */*)

set >= other

Testa se cada elemento em *other* está contido no conjunto.

set > other

Testa se o conjunto é um superconjunto próprio de *other*, isto é, `set >= other and set != other`.

union(**others*)

set | other | ...

Retorna um novo conjunto com elementos do conjunto e de todos os outros.

intersection(**others*)

set & other & ...

Retorna um novo conjunto com elementos comuns do conjunto e de todos os outros.

difference(**others*)

set - other - ...

Retorna um novo conjunto com elementos no conjunto que não estão nos outros.

symmetric_difference(*other*, */*)

set ^ other

Retorna um novo conjunto com elementos estejam ou no conjunto ou em *other*, mas não em ambos.

copy()

Retorna uma cópia rasa do conjunto.

Observe que, as versões não-operador dos métodos *union()*, *intersection()*, *difference()*, *symmetric_difference()*, *issubset()*, e *issuperset()* irão aceitar qualquer iterável como um argumento. Em contraste, suas contrapartes baseadas em operadores exigem que seus argumentos sejam conjuntos. Isso impede construções suscetíveis a erros como `set('abc') & 'cbs'` e favorece a forma mais legível `set('abc').intersection('cbs')`.

Tanto *set* quanto *frozenset* suportam comparar um conjunto com outro. Dois conjuntos são iguais se, e somente se, cada elemento de cada conjunto está contido no outro conjunto (cada um é um subconjunto do outro). Um conjunto é menor que outro se, e somente se, o primeiro conjunto é um subconjunto adequado do segundo (é um subconjunto, mas não é igual). Um conjunto é maior que outro conjunto se, e somente se, o primeiro conjunto é um superconjunto próprio do segundo conjunto (é um superconjunto, mas não é igual).

Instâncias de *set* são comparadas a instâncias de *frozenset* baseados nos seus membros. Por exemplo, `set('abc') == frozenset('abc')` retorna `True` e assim como `set('abc') in set([frozenset('abc')])`.

O subconjunto e comparações de igualdade não generalizam para a função de ordenamento total. Por exemplo, quaisquer dois conjuntos deslocados não vazios, não são iguais e não são subconjuntos um do outro, então *todos* os seguintes retornam `False`: `a<b`, `a==b` ou `a>b`.

Como conjuntos apenas definem ordenamento parcial (subconjunto de relacionamentos), a saída do método *list.sort()* é indefinida para listas e conjuntos.

Elementos de conjuntos, assim como chaves de dicionário, devem ser *hasheáveis*.

Operações binárias que misturam instâncias de *set* com *frozenset* retornam o tipo do primeiro operando. Por exemplo: `frozenset('ab') | set('bc')` retorna uma instância de *frozenset*.

A seguinte tabela lista operações disponíveis para *set* que não se aplicam para instâncias imutáveis de *frozenset*:

update(**others*)

set |= other | ...

Atualiza o conjunto, adicionando elementos dos outros.

intersection_update(**others*)

set &= other & ...

Atualiza o conjunto, mantendo somente elementos encontrados nele e em outros.

difference_update(**others*)

set -= other | ...

Atualiza o conjunto, removendo elementos encontrados em outros.

symmetric_difference_update(*other*, /)

set ^= other

Atualiza o conjunto, mantendo somente elementos encontrados em qualquer conjunto, mas não em ambos.

add(*elem*, /)

Adiciona o elemento *elem* ao conjunto.

remove(*elem*, /)

Remove o elemento *elem* do conjunto. Levanta *KeyError* se *elem* não estiver contido no conjunto.

discard(*elem*, /)

Remove o elemento *elem* do conjunto se ele estiver presente.

pop()

Remove e retorna um elemento arbitrário do conjunto. Levanta *KeyError* se o conjunto estiver vazio.

clear()

Remove todos os elementos do conjunto.

Perceba, as versões sem operador dos métodos *update()*, *intersection_update()*, *difference_update()* e *symmetric_difference_update()* irão aceitar qualquer iterável como um argumento.

Perceba, o argumento *elem* para os métodos __contains__(), *remove()* e *discard()* pode ser um conjunto. Para suportar pesquisas por um frozenset equivalente, um frozenset temporário é criado a partir de *elem*.

4.12 Tipo mapeamento — dict

Um objeto *mapeamento* mapeia valores *hasheáveis* para objetos arbitrários. Mapeamentos são objetos mutáveis. Existe no momento apenas um tipo de mapeamento padrão, o *dicionário*. (Para outros contêineres, veja as classes embutidas *list*, *set* e *tuple*, e o módulo *collections*.)

As chaves de um dicionário são *quase* valores arbitrários. Valores que não são *hasheáveis*, ou seja, valores contendo listas, dicionários ou outros tipos mutáveis (que são comparados por valor e não por identidade de objeto) não podem ser usados como chaves. Valores que comparam iguais (como `1`, `1.0` e `True`) podem ser usados alternadamente para indexar a mesma entrada do dicionário.

class dict(***kwargs*)

class dict(*mapping*, */*, ***kwargs*)

class dict(*iterable*, */*, ***kwargs*)

Retorna um novo dicionário inicializado a partir de um argumento posicional opcional, e um conjunto de argumentos nomeados possivelmente vazio.

Os dicionários podem ser criados de várias formas:

- Usar uma lista de pares `key: value` separados por vírgula com chaves: `{'jack': 4098, 'sjoerd': 4127}` ou `{4098: 'jack', 4127: 'sjoerd'}`
- Usar uma compreensão de dicionário: `{}`, `{x: x ** 2 for x in range(10)}`
- Usar o construtor de tipo: `dict()`, `dict([('foo', 100), ('bar', 200)])`, `dict(foo=100, bar=200)`

Se nenhum argumento posicional é fornecido, um dicionário vazio é criado. Se um argumento posicional é fornecido e ele define um método `keys()`, um dicionário é criado chamando __getitem__() no argumento com cada chave retornada pelo método. Caso contrário, o argumento posicional deve ser um objeto *iterável*. Cada item no iterável deve ser por si mesmo um iterável com exatamente dois elementos. O primeiro elemento de cada item torna-se a chave no novo dicionário, e o segundo elemento, o valor correspondente. Se a chave ocorrer mais do que uma vez, o último valor para aquela chave torna-se o valor correspondente no novo dicionário.

Se argumentos nomeados são fornecidos, os argumentos nomeados e seus valores são adicionados ao dicionário criado a partir do argumento posicional. Se uma chave sendo adicionada já está presente, o valor do argumento nomeado substitui o valor do argumento posicional.

Fornecer argumentos nomeados conforme no primeiro exemplo somente funciona para chaves que são identificadores válidos no Python. Caso contrário, quaisquer chaves válidas podem ser usadas.

Dicionários são iguais se e somente se eles os mesmos pares `(key, value)` (independente de ordem). Comparações de ordem ('<', '<=', '>=', '>') levantam *TypeError*. Para ilustrar a criação e igualdade de dicionários, os exemplos a seguir retornam um dicionário igual a `{"one": 1, "two": 2, "three": 3}`:

```
>>> a = dict(one=1, two=2, three=3)
>>> b = {'one': 1, 'two': 2, 'three': 3}
>>> c = dict(zip(['one', 'two', 'three'], [1, 2, 3]))
>>> d = dict([('two', 2), ('one', 1), ('three', 3)])
>>> e = dict({'three': 3, 'one': 1, 'two': 2})
>>> f = dict({'one': 1, 'three': 3}, two=2)
>>> a == b == c == d == e == f
True
```

Fornecer argumentos nomeados conforme no primeiro exemplo somente funciona para chaves que são identificadores válidos no Python. Caso contrário, quaisquer chaves válidas podem ser usadas.

Dicionários preservam a ordem de inserção. Perceba que atualizar a chave não afeta a ordem. Chaves adicionadas após a deleção são inseridas no final.

```
>>> d = {"um": 1, "dois": 2, "três": 3, "quatro": 4}
>>> d
{'um': 1, 'dois': 2, 'três': 3, 'quatro': 4}
>>> list(d)
['um', 'dois', 'três', 'quatro']
>>> list(d.values())
[1, 2, 3, 4]
>>> d["um"] = 42
>>> d
{'um': 42, 'dois': 2, 'três': 3, 'quatro': 4}
>>> del d["dois"]
>>> d["dois"] = None
>>> d
{'um': 42, 'três': 3, 'quatro': 4, 'dois': None}
```

Alterado na versão 3.7: Ordem do dicionário é garantida conforme a ordem de inserção. Este comportamento era um detalhe de implementação do CPython a partir da versão 3.6.

Estas são as operações que dicionários suportam (e portanto, tipos de mapeamento personalizados devem suportar também):

`list(d)`

Retorna uma lista de todas as chaves usadas no dicionário *d*.

`len(d)`

Retorna o número de itens no dicionário *d*.

`d[key]`

Retorna o item de *d* com a chave *key*. Levanta um *KeyError* se *key* não estiver no mapeamento.

Se uma subclasse de um dict define um método `__missing__()` e *key* não estiver presente, a operação `d[key]` chama aquele método com a chave *key* como argumento. A operação `d[key]` então retorna ou levanta o que é retornado ou levantado pela chamada de `__missing__(key)`. Nenhuma operação ou métodos invocam `__missing__()`. Se `__missing__()` não for definido, então *KeyError* é levantada. `__missing__()` deve ser um método; ele não pode ser uma variável de instância:

```
>>> class Counter(dict):
...     def __missing__(self, key):
...         return 0
...
>>> c = Counter()
>>> c['red']
0
>>> c['red'] += 1
>>> c['red']
1
```

O exemplo acima mostra parte da implementação de *collections.Counter*. Um método `__missing__()` diferente é usado para *collections.defaultdict*.

`d[key] = value`

Define `d[key]` para *value*.

`del d[key]`

Remove `d[key]` do *d*. Levanta uma exceção *KeyError* se *key* não estiver no mapeamento.

key in d

Retorna `True` se *d* tiver uma chave *key*, caso contrário `False`.

key not in d

Equivalente a `not key in d`.

iter(d)

Retorna um iterador para as chaves do dicionário. Isso é um atalho para `iter(d.keys())`.

clear()

Remove todos os itens do dicionário.

copy()

Retorna uma cópia superficial do dicionário.

classmethod fromkeys(*iterable*, *value=None*, /)

Cria um novo dicionário com chaves provenientes de *iterable* e valores definidos como *value*.

fromkeys() é um método de classe que retorna um novo dicionário. *value* tem como valor padrão `None`. Todos os valores referem-se a apenas uma única instância, então geralmente não faz sentido que *value* seja um objeto mutável tal como uma lista vazia. Para obter valores distintos, use uma compreensão de dicionário ao invés.

get(*key*, *default=None*, /)

Retorna o valor para *key* se *key* está no dicionário, caso contrário *default*. Se *default* não é fornecido, será usado o valor padrão `None`, de tal forma que este método nunca levanta um *KeyError*.

items()

Retorna uma nova visão dos itens do dicionário (pares de `(key, value)`). Veja a *documentação de objetos de visão de dicionário*.

keys()

Retorna uma nova visão das chaves do dicionário. Veja a *documentação de objetos de visão de dicionário*.

pop(*key*, /)

pop(*key*, *default*, /)

Se *key* está no dicionário, remove a mesma e retorna o seu valor, caso contrário retorna *default*. Se *default* não foi fornecido e *key* não está no dicionário, um *KeyError* é levantado.

popitem()

Remove e retorna um par `(key, value)` do dicionário. Pares são retornados como uma pilha, ou seja em ordem LIFO (last-in, first-out).

popitem() é útil para destrutivamente iterar sobre um dicionário, algo comumente usado em algoritmos de conjunto. Se o dicionário estiver vazio, chamar *popitem()* levanta um *KeyError*.

Alterado na versão 3.7: Ordem LIFO agora é garantida. Em versões anteriores, *popitem()* iria retornar um par chave/valor arbitrário.

reversed(d)

Retorna um iterador revertido sobre as chaves do dicionário. Isso é um atalho para `reversed(d.keys())`.

Adicionado na versão 3.8.

setdefault(*key*, *default=None*, /)

Se *key* está no dicionário, retorna o seu valor. Se não, insere *key* com o valor *default* e retorna *default*. *default* por padrão usa o valor `None`.

update(***kwargs*)

update(*mapping*, /, ***kwargs*)

update(*iterable*, /, ***kwargs*)

Atualiza o dicionário com os pares chave/valor de *mapping* ou *iterable* e *kwargs*, sobrescrevendo chaves existentes. Retorna `None`.

update() aceita ou outro objeto com um método `keys()` (em cada caso `__getitem__()` é chamado com cada chave retornada pelo método) ou um iterável de pares de chave/valor (como tuplas ou outros iteráveis de comprimento dois). Se argumentos nomeados são especificados, o dicionário é então atualizado com esses pares de chave/valor: `d.update(red=1, blue=2)`.

values()

Retorna uma nova visão dos valores do dicionário. Veja a *documentação de objetos de visão de dicionário*.

Uma comparação de igualdade entre uma visão de `dict.values()` e outra, sempre irá retornar `False`. Isso também se aplica ao comparar `dict.values()` entre si:

```
>>> d = {'a': 1}
>>> d.values() == d.values()
False
```

d | other

Cria um novo dicionário com as chaves e os valores mesclados de *d* e *other*, que devem ser dicionários. Os valores de *other* têm prioridade quando *d* e *other* compartilham chaves.

Adicionado na versão 3.9.

d |= other

Atualiza o dicionário *d* com chaves e valores de *other*, que podem ser a *mapeamento* ou um *iterável* dos pares chave/valor. Os valores de *other* têm prioridade quando *d* e *other* compartilham chaves.

Adicionado na versão 3.9.

Dicionários e visões de dicionários são reversíveis.

```
>>> d = {"um": 1, "dois": 2, "três": 3, "quatro": 4}
>>> d
{'um': 1, 'dois': 2, 'três': 3, 'quatro': 4}
>>> list(reversed(d))
['quatro', 'três', 'dois', 'um']
>>> list(reversed(d.values()))
[4, 3, 2, 1]
>>> list(reversed(d.items()))
[('quatro', 4), ('três', 3), ('dois', 2), ('um', 1)]
```

Alterado na versão 3.8: Dicionários agora são reversíveis.

Ver também

types.MappingProxyType podem ser usados para criar uma visão somente leitura de um *dict*.

4.12.1 Objetos de visão de dicionário

Os objetos retornados por *dict.keys()*, *dict.values()* e *dict.items()* são *objetos de visão*. Eles fornecem uma visão dinâmica das entradas do dicionário, o que significa que quando o dicionário muda, a visão reflete essas mudanças.

Visões de dicionários podem ser iteradas para apresentar seus respectivos dados, e suportar testes de existência:

len(dictview)

Retorna o número de entradas no dicionário.

iter(dictview)

Retorna um iterador sobre as chaves, valores ou itens (representados como tuplas de `(key, value)`) no dicionário.

Chaves e valores são iterados em ordem de inserção. Isso permite a criação de pares `(value, key)` usando *zip()*: `pairs = zip(d.values(), d.keys())`. Outra maneira de criar a mesma lista é `pairs = [(v, k) for (k, v) in d.items()]`.

Iterar sobre visões enquanto adiciona ou deleta entradas no dicionário pode levantar um *RuntimeError* ou falhar a iteração sobre todas as entradas.

Alterado na versão 3.7: Ordem do dicionário é garantida como a ordem de inserção.

x in dictview

Retorna `True` se *x* está nas chaves, valores ou itens do dicionário subjacente (no último caso, *x* deve ser uma tupla de `(key, value)`).

reversed(dictview)

Retorna um iterador reverso sobre as chaves, valores ou itens do dicionário. A visão será iterada na ordem reversa de inserção.

Alterado na versão 3.8: Visões de dicionário agora são reversíveis.

dictview.mapping

Retorna um *types.MappingProxyType* que envolve o dicionário original ao qual a visão se refere.

Adicionado na versão 3.10.

Visões chave são similar a conjunto, como suas entradas são únicas e *hasheáveis*. As visões de itens também têm operações semelhantes a conjuntos, uma vez que os pares (chave, valor) são únicos e as chaves são hasheáveis. Se todos os valores em uma visão de itens também forem hasheáveis, a visão de itens poderá interoperar com outros conjuntos. (Visões de valores não são tratadas de como similar a conjunto, pois as entradas geralmente não são únicas.) Para visões similares a conjunto, todas as operações definidas para a classe base abstrata *collections.abc.Set* estão disponíveis (por exemplo, ==, < ou ^). Ao usar operadores de conjunto, visões de conjunto ou similares aceitam qualquer iterável como outro operando, ao contrário de conjuntos que aceitam apenas conjuntos como entrada.

Um exemplo da utilização da visualização de dicionário:

```
>>> dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
>>> keys = dishes.keys()
>>> values = dishes.values()

>>> # iteration
>>> n = 0
>>> for val in values:
...     n += val
...
>>> print(n)
504

>>> # a iteração sobre chaves e valores é feita na mesma ordem (ordem de inserção)
>>> list(keys)
['eggs', 'sausage', 'bacon', 'spam']
>>> list(values)
[2, 1, 1, 500]

>>> # objetos de visualização são dinâmicos e refletem mudanças no dicionário
>>> del dishes['eggs']
>>> del dishes['sausage']
>>> list(keys)
['bacon', 'spam']
```

(continua na próxima página)

(continuação da página anterior)

```
>>> # operações de conjunto
>>> keys & {'eggs', 'bacon', 'salad'}
{'bacon'}
>>> keys ^ {'sausage', 'juice'} == {'juice', 'sausage', 'bacon', 'spam'}
True
>>> keys | ['juice', 'juice', 'juice'] == {'bacon', 'spam', 'juice'}
True

>>> # recupera um proxy somente de leitura para o dicionário original
>>> values.mapping
mappingproxy({'bacon': 1, 'spam': 500})
>>> values.mapping['spam']
500
```

4.13 Tipos de Gerenciador de Contexto

A instrução `with` do Python suporta o conceito de um contexto em tempo de execução definido por um gerenciador de contexto. Isto é implementado usando um par de métodos, que permitem que classes definidas pelo usuário especifiquem um contexto em tempo de execução, o qual é inicializado antes da execução das instruções, e encerrado quando as instruções terminam:

`contextmanager.__enter__()`

Entra no contexto em tempo de execução e retorna este objeto ou outro objeto relacionado ao contexto em tempo de execução. O valor retornado por este método é ligado ao identificador na cláusula `as` das instruções `with` usando este gerenciador de contexto.

Um exemplo de gerenciador de contexto que retorna a si mesmo é um *objeto arquivo*. Objeto arquivos retornam a si mesmos do método __enter__() para permitir que *open()* seja usado como a expressão de contexto em uma instrução `with`.

Um exemplo de gerenciador de contexto que retorna um objeto relacionado é aquele retornado por *decimal.localcontext()*. Esses gerenciadores definem o contexto decimal ativo para uma cópia do contexto decimal original, e então retornam a cópia. Isso permite que mudanças sejam feitas no contexto decimal atual, no corpo contido na instrução `with`, sem afetar o código fora da instrução `with`.

`contextmanager.__exit__`(*exc_type*, *exc_val*, *exc_tb*)

Sai do contexto em tempo de execução e retorna um sinalizador booleano indicando se qualquer exceção que ocorreu deve ser suprimida. Se uma exceção ocorreu enquanto era executado o corpo da instrução `with`, os argumentos contêm o tipo da exceção, valor e informação da traceback (situação da pilha de execução). Caso contrário, os três argumentos são `None`.

Retornar um valor verdadeiro deste método fará com que a instrução `with` suprima a exceção e continue a execução com a instrução imediatamente após a instrução `with`. Caso contrário a exceção continuará propagando após este método ter encerrado sua execução. Exceções que ocorrerem durante a execução deste método irão substituir qualquer exceção que tenha ocorrido dentro do corpo da instrução `with`.

A exceção passada nunca deve ser re-levantada explicitamente - ao invés disso, este método deve retornar um valor falso para indicar que o método completou sua execução com sucesso, e não quer suprimir a exceção levantada. Isso permite ao código do gerenciador de contexto facilmente detectar se um método `__exit__()` realmente falhou ou não.

Python define diversos gerenciadores de contexto para suportar facilmente sincronização de threads, solicita o fechamento de arquivos ou outros objetos, e manipula de forma simples o contexto ativo de aritmética decimal. Os tipos especificados não são tratados de forma especial além da sua implementação e do protocolo do gerenciador de contexto. Veja o módulo *contextlib* para alguns exemplos.

Os *gerador*es do Python e o decorador *contextlib.contextmanager* fornecem uma maneira conveniente de implementar esses protocolos. Se uma função geradora for decorada com o decorador *contextlib.*

contextmanager, ela retornará um gerenciador de contexto implementando os métodos necessários *__enter__()* e *__exit__()*, em vez de o iterador produzido por uma função geradora não decorada.

Observe que não existe nenhum slot específico para qualquer um desses métodos na estrutura de tipos para objetos Python na API Python/C. Tipos de extensão que desejam definir estes métodos devem fornecê-los como um método acessível normal do Python. Comparado com a sobrecarga de configurar o contexto em tempo de execução, a sobrecarga na pesquisa de dicionário em uma única classe é negligenciável.

4.14 Tipos de anotação de tipo — Apelido genérico, União

Os principais tipos embutidos para *anotações de tipo* são *Apelido genérico* e *União*.

4.14.1 Tipo Generic Alias

Objetos `GenericAlias` são geralmente criados subscrevendo uma classe. Eles são mais usados com classes contêineres, como *list* ou *dict*. Por exemplo, `list[int]` é um objeto `GenericAlias` criado pela subscrição da classe `list` com o argumento *int*. Objetos `GenericAlias` são destinados principalmente para uso com *anotações de tipo*.

> **Nota**
>
> Geralmente só é possível subscrever uma classe se a classe implementar o método especial `__class_getitem__()`.

Um objeto `GenericAlias` atua como um proxy para um *tipo genérico*, implementando *genéricos parametrizados*.

Para uma classe contêiner, o(s) argumento(s) fornecido(s) para uma subscrição da classe pode indicar o(s) tipo(s) dos elementos que um objeto contém. Por exemplo, `set[bytes]` pode ser usado em anotações de tipo para significar um *set* em que todos os elementos são do tipo *bytes*.

Para uma classe que define `__class_getitem__()`, mas não é um contêiner, o(s) argumento(s) fornecido(s) para uma subscrição da classe geralmente indicará o(s) tipo(s) de retorno de um ou mais métodos definidos em um objeto. Por exemplo, *expressões regulares* podem ser usadas tanto no tipo de dados *str* quanto no tipo de dados *bytes*:

- Se `x = re.search('foo', 'foo')`, `x` será um objeto *re.Match* onde os valores de retorno de `x.group(0)` e `x[0]` serão ambos do tipo *str*. Podemos representar este tipo de objeto em anotações de tipo com o `GenericAlias` `re.Match[str]`.
- Se `y = re.search(b'bar', b'bar')` (observe o `b` para *bytes*), `y` também será uma instância de `re.Match`, mas os valores de retorno de `y.group(0)` e `y[0]` serão ambos do tipo *bytes*. Em anotações de tipo, representaríamos esta variedade de objetos *re.Match* com `re.Match[bytes]`.

Objetos `GenericAlias` são instâncias da classe *types.GenericAlias*, que também podem ser usadas para criar objetos `GenericAlias` diretamente.

`T[X, Y, ...]`

Cria um `GenericAlias` representando um tipo `T` parametrizado por tipos *X*, *Y* e mais, dependendo do `T` usados. Por exemplo, uma função esperando uma *list* contendo elementos *float*:

```
def average(values: list[float]) -> float:
    return sum(values) / len(values)
```

Outro exemplo para *mapeamento* de objetos, usando um *dict*, o qual é um tipo genérico esperando 2 tipos de parâmetros representando o tipo da chave e o tipo do valor. Neste exemplo, a função espera um `dict` com chaves do tipo *str* e valores do tipo *int*:

```
def send_post_request(url: str, body: dict[str, int]) -> None:
    ...
```

As funções embutidas *isinstance()* e *issubclass()* não aceitam tipos GenericAlias para o seu segundo argumento:

```
>>> isinstance([1, 2], list[str])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: isinstance() argument 2 cannot be a parameterized generic
```

O runtime Python não obriga *anotações de tipo*. Isso se aplica a tipos genéricos e seus parâmetros tipados. Ao criar um objeto contêiner a partir de um GenericAlias, os elementos no contêiner não são verificados pelo seu tipo. Por exemplo, o seguinte código é desencorajado, mas irá executar sem erros:

```
>>> t = list[str]
>>> t([1, 2, 3])
[1, 2, 3]
```

Além disso, genéricos parametrizados removem parâmetros tipados durante a criação do objeto:

```
>>> t = list[str]
>>> type(t)
<class 'types.GenericAlias'>

>>> l = t()
>>> type(l)
<class 'list'>
```

Chamar *repr()* ou *str()* sobre um genérico mostra o tipo parametrizado:

```
>>> repr(list[int])
'list[int]'

>>> str(list[int])
'list[int]'
```

O método __getitem__() de contêineres genéricos irá levantar uma exceção para não permitir erros como dict[str][str]:

```
>>> dict[str][str]
Traceback (most recent call last):
  ...
TypeError: dict[str] is not a generic class
```

Entretanto, tais expressões são válidas quando *type variáveis* são usadas. O índice deve ter tantos elementos quantos forem os itens de variável de tipo no objeto GenericAlias *__args__*.

```
>>> from typing import TypeVar
>>> Y = TypeVar('Y')
>>> dict[str, Y][int]
dict[str, int]
```

Classes genéricas padrão

As seguintes classes de biblioteca padrão oferecem suporte a genéricos parametrizados. Esta lista não é exaustiva.

- *tuple*
- *list*
- *dict*
- *set*

- *frozenset*
- *type*
- *asyncio.Future*
- *asyncio.Task*
- *collections.deque*
- *collections.defaultdict*
- *collections.OrderedDict*
- *collections.Counter*
- *collections.ChainMap*
- *collections.abc.Awaitable*
- *collections.abc.Coroutine*
- *collections.abc.AsyncIterable*
- *collections.abc.AsyncIterator*
- *collections.abc.AsyncGenerator*
- *collections.abc.Iterable*
- *collections.abc.Iterator*
- *collections.abc.Generator*
- *collections.abc.Reversible*
- *collections.abc.Container*
- *collections.abc.Collection*
- *collections.abc.Callable*
- *collections.abc.Set*
- *collections.abc.MutableSet*
- *collections.abc.Mapping*
- *collections.abc.MutableMapping*
- *collections.abc.Sequence*
- *collections.abc.MutableSequence*
- *collections.abc.ByteString*
- *collections.abc.MappingView*
- *collections.abc.KeysView*
- *collections.abc.ItemsView*
- *collections.abc.ValuesView*
- *contextlib.AbstractContextManager*
- *contextlib.AbstractAsyncContextManager*
- *dataclasses.Field*
- *functools.cached_property*
- *functools.partialmethod*
- *os.PathLike*
- *queue.LifoQueue*

- *queue.Queue*
- *queue.PriorityQueue*
- *queue.SimpleQueue*
- *re.Pattern*
- *re.Match*
- *shelve.BsdDbShelf*
- *shelve.DbfilenameShelf*
- *shelve.Shelf*
- *types.MappingProxyType*
- *weakref.WeakKeyDictionary*
- *weakref.WeakMethod*
- *weakref.WeakSet*
- *weakref.WeakValueDictionary*

Atributos especiais de objetos `GenericAlias`

Todos os genéricos parametrizados implementam atributos especiais somente leitura.

`genericalias.__origin__`

Este atributo aponta para a classe genérica não parametrizada:

```
>>> list[int].__origin__
<class 'list'>
```

`genericalias.__args__`

Este atributo é uma *tuple* (possivelmente de comprimento 1) de tipos genéricos passado para o método `__class_getitem__()` original da classe genérica:

```
>>> dict[str, list[int]].__args__
(<class 'str'>, list[int])
```

`genericalias.__parameters__`

O atributo é uma tupla computada preguiçosamente (possivelmente vazia) de variáveis de tipo único encontradas em `__args__`:

```
>>> from typing import TypeVar

>>> T = TypeVar('T')
>>> list[T].__parameters__
(~T,)
```

Nota

Um objeto `GenericAlias` com os parâmetros *typing.ParamSpec* pode não ter `__parameters__` correto após a substituição porque *typing.ParamSpec* se destina principalmente à verificação de tipo estático.

`genericalias.__unpacked__`

Um booleano que é true se o apelido foi desempacotado usando o operador `*` (consulte *TypeVarTuple*).

Adicionado na versão 3.11.

Ver também

PEP 484 - Dicas de tipo
Apresentando a estrutura do Python para anotações de tipo.

PEP 585 - Sugestão de tipo para Genéricos em coleções padrão
Apresentando a capacidade de parametrizar nativamente as classes da biblioteca padrão, desde que implementem o método de classe especial `__class_getitem__()`.

***Genéricos*, *genéricos definidos pelo usuário* e `typing.Generic`**
Documentação sobre como implementar classes genéricas que podem ser parametrizadas em tempo de execução e compreendidas por verificadores de tipo estático.

Adicionado na versão 3.9.

4.14.2 Tipo União

Um objeto união contém o valor da operação `|` (bit a bit ou) em vários *objetos tipo*. Esses tipos são destinados principalmente para *anotações de tipos*. A expressão de tipo de união habilita a sintaxe de sugestão de tipo mais limpo em comparação com subscrever `typing.Union`.

`X | Y | ...`

Define um objeto união que contém os tipos *X*, *Y* e assim por diante. `X | Y` significa X ou Y. É equivalente a `typing.Union[X, Y]`. Por exemplo, a seguinte função espera um argumento do tipo `int` ou `float`:

```
def square(number: int | float) -> int | float:
    return number ** 2
```

Nota

O operando `|` não pode ser usado em tempo de execução para definir uniões onde um ou mais membros são uma referência direta. Por exemplo, `int | "Foo"`, onde `"Foo"` é uma referência a uma classe ainda não definida, falhará em tempo de execução. Para uniões que incluem referências futuras, apresente a expressão inteira como uma string, por exemplo. `"int | Foo"`.

`union_object == other`

Os objetos união podem ser testados quanto à igualdade com outros objetos união. Detalhes:

- Uniões de uniões são achatadas:

```
(int | str) | float == int | str | float
```

- Tipos redundantes são removidos:

```
int | str | int == int | str
```

- Ao comparar uniões, a ordem é ignorada:

```
int | str == str | int
```

- Cria instâncias de `typing.Union`:

```
int | str == typing.Union[int, str]
type(int | str) is typing.Union
```

- Tipos opcionais podem ser escritos como uma união com `None`:

```
str | None == typing.Optional[str]
```

isinstance(obj, union_object)

issubclass(obj, union_object)

Chamadas para *isinstance()* e *issubclass()* também são suportados com um objeto união:

```
>>> isinstance("", int | str)
True
```

No entanto, *genéricos parametrizados* em objetos de união não podem ser verificados:

```
>>> isinstance(1, int | list[int])  # cálculo em curto-circuito
True
>>> isinstance([1], int | list[int])
Traceback (most recent call last):
  ...
TypeError: isinstance() argument 2 cannot be a parameterized generic
```

O tipo exposto ao usuário para o objeto união pode ser acessado a partir de *typing.Union* e usado por verificações de *isinstance()*:

```
>>> import typing
>>> isinstance(int | str, typing.Union)
True
>>> typing.Union()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot create 'typing.Union' instances
```

Nota

O método `__or__()` para objetos de tipo foi adicionado para suportar a sintaxe `X | Y`. Se uma metaclasse implementa `__or__()`, a União pode substituí-la:

```
>>> class M(type):
...     def __or__(self, other):
...         return "Hello"
...
>>> class C(metaclass=M):
...     pass
...
>>> C | int
'Hello'
>>> int | C
int | C
```

Ver também

PEP 604 – PEP propondo a sintaxe `X | Y` e o tipo União.

Adicionado na versão 3.10.

Alterado na versão 3.14: Objetos Union agora são instâncias de *typing.Union*. Anteriormente, eram instâncias de *types.UnionType*, que continua sendo um apelido para *typing.Union*.

4.15 Outros tipos embutidos

O interpretador suporta diversos outros tipos de objetos. Maior parte desses, suporta apenas uma ou duas operações.

4.15.1 Módulos

A única operação especial em um módulo é o acesso a um atributo: `m.name`, onde *m* é um módulo e *name* acessa o nome defino na tabela de símbolos de *m*. Atributos de módulo podem receber atribuição. (Perceba que a instrução `import` não é, estritamente falando, uma operação em um objeto do módulo; `import foo` não requer que um objeto do módulo chamado *foo* exista, ao invés disso requer uma *definição* (externa) de um módulo chamado *foo* em algum lugar.)

Um atributo especial de cada módulo é `__dict__`. Este é o dicionário contendo a tabela de símbolos do modulo. Modificar este dicionário vai na verdade modificar a tabela de símbolos do módulo, mas atribuição direta para o atributo `__dict__` não é possível (você pode escrever `m.__dict__['a'] = 1`, o qual define `m.a` para ser `1`, mas você não consegue escrever `m.__dict__ = {}`). Modificar `__dict__` diretamente não é recomendado.

Módulos embutidos no interpretador são escritos desta forma: `<module 'sys' (built-in)>`. Se carregados a partir de um arquivo, eles são escritos como `<module 'os' from '/usr/local/lib/pythonX.Y/os.pyc'>`.

4.15.2 Classes e Instâncias de Classes

Veja objects e class para estes.

4.15.3 Funções

Objetos função são criados através da definição de funções. A única operação que pode ser feita em um objeto função é chamá-la: `func(lista-de-argumentos)`.

Existem na verdade duas possibilidades de objetos função: funções embutidas e funções definidas pelo usuário. Ambas suportam a mesma operação (chamar a função), mas a implementação é diferente, portanto os diferentes tipos de objetos.

Veja a função function para mais informações.

4.15.4 Métodos

Métodos são funções que são chamadas usando a notação de atributo. Existem duas opções: métodos embutidos (tal como *append()* em listas) e métodos de instância de classe. Métodos embutidos são descritos com os tipos que suportam eles.

Se você acessar um método (uma função definida em um espaço de nomes de uma classe) através de uma instância, você obtém um objeto especial: um objeto com *método vinculado* (também chamado de método de instância). Quando chamado, ele irá adicionar o argumento `self` para a lista de argumentos. Métodos vinculados tem dois atributos somente leitura especiais: `m.__self__` é o objeto no qual o método opera, e `m.__func__` é a função que implementa o método. Chamar `m(arg-1, arg-2, ..., arg-n)` é completamente equivalente a chamar `m.__func__(m.__self__, arg-1, arg-2, ..., arg-n)`.

Assim como objetos função, objetos de métodos vinculados tem suporte para receber atributos arbitrários. Entretanto, como atributos de método na verdade são armazenados no objeto função subjacente (`method.__func__`), definir atributos de método em métodos vinculados não é permitido. Tentar definir um atributo em um método resulta em um *AttributeError* sendo levantado. A fim de definir um atributo de método, você precisa definir explicitamente ele no objeto função subjacente:

```
>>> class C:
...     def method(self):
...         pass
...
>>> c = C()
>>> c.method.whoami = 'my name is method'  # proibido atribuir diretamente no␣
↪método
```

(continua na próxima página)

(continuação da página anterior)

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'method' object has no attribute 'whoami'
>>> c.method.__func__.whoami = 'my name is method'
>>> c.method.whoami
'my name is method'
```

Veja instance-methods para mais informações.

4.15.5 Objetos código

Objetos código são usados pela implementação para representar código Python executável "pseudo-compilado", tal como corpo de uma função. Eles são diferentes de objetos função porque eles não contém uma referência para os seus ambientes de execução global. Objetos código são retornados pela função embutida *compile()* e podem ser extraídos de objetos função através do seu atributo `__code__`. Veja também o módulo *code*.

Acessar `__code__` levanta um *evento de auditoria* `object.__getattr__` com os argumentos `obj` e `"__code__"`.

Um objeto de código pode ser executado ou avaliado passando-o (ao invés da string fonte) para as funções embutidas *exec()* ou *eval()*.

Veja o tipo types para maiores informações.

4.15.6 Objetos tipo

Objetos de tipos representam os vários tipos de objetos. Um tipo de um objeto é acessado pela função embutida *type()*. Não existem operações especiais sobre tipos. O módulo padrão *types* define nomes para todos os tipos padrão embutidos.

Tipos são escritos como isto: `<class 'int'>`.

4.15.7 O objeto nulo

Este objeto é retornado por funções que não retornam um valor explicitamente. Ele não suporta operações especiais. Existe exatamente um objeto nulo, chamado de `None` (um nome embutido). `type(None)()` produz o mesmo singleton.

Ele é escrito como `None`.

4.15.8 O Objeto Ellipsis

Este objeto é comumente usado para indicar que algo foi omitido. Ela não suporta operações especiais. Existe exatamente um objeto ellipsis, nomeado *Ellipsis* (um nome embutido). `type(Ellipsis)()` produz o singleton *Ellipsis*.

Está escrito com `Ellipsis` ou `...`.

No uso típico, `...` como o objeto `Ellipsis` aparece em alguns lugares diferentes, por exemplo:

- Em anotações de tipo, como *argumentos chamáveis* ou *elementos de tupla*.
- Como o corpo de uma função em vez de uma instrução pass.
- Em bibliotecas de terceiros, como slicing e striding do Numpy.

Python também usa três pontos de maneiras que não são objetos `Ellipsis`, por exemplo:

- *ELLIPSIS* do doctest, como um padrão para conteúdo ausente.
- O prompt Python padrão do console *interativo* quando a entrada parcial está incompleta.

Por fim, a documentação do Python geralmente usa três pontos no uso convencional do inglês para indicar conteúdo omitido, mesmo em exemplos de código que também os usam como `Ellipsis`.

4.15.9 O Objeto NotImplemented

Este objeto é retornado a partir de comparações e operações binárias quando elas são solicitadas para operar em tipos nos quais eles não suportam. Veja comparisons para mais informações. Existe exatamente um objeto *NotImplemented*. `type(NotImplemented)()` produz o mesmo valor.

Está escrito como `NotImplemented`.

4.15.10 Objetos Internos

Veja types para esta informação. Descreve objetos de quadro de pilha, objetos traceback (situação da pilha de execução), e objetos fatia.

4.16 Atributos Especiais

A implementação adiciona alguns atributos especiais somente leitura para diversos tipos de objetos, onde eles são relevantes. Alguns desses não são reportados pela função embutida *dir()*.

`definition.`**`__name__`**

O nome da classe, função, método, descritor, ou instância geradora.

`definition.`**`__qualname__`**

O *nome qualificado* da classe, função, método, descritor, ou instância geradora.

Adicionado na versão 3.3.

`definition.`**`__module__`**

O nome do módulo no qual uma classe ou função foi definida.

`definition.`**`__doc__`**

A string de documentação de uma classe ou função, ou `None` se não definida.

`definition.`**`__type_params__`**

Os parâmetros de tipo de classes genéricas, funções e *apelidos de tipo*. Para classes e funções que não são genéricas, isso será uma tupla vazia.

Adicionado na versão 3.12.

4.17 Limitação de comprimento de string na conversão para inteiro

CPython tem um limite global para conversão entre *int* e *str* para mitigar ataques de negação de serviço. Esse limite *somente* se aplica a bases numéricas decimais ou outras que não sejam potência de dois. As conversões hexadecimais, octais e binárias são ilimitadas. O limite pode ser configurado.

O tipo *int* no CPython é um número de comprimento arbitrário armazenado em formato binário (comumente conhecido como "bignum"). Não existe nenhum algoritmo que possa converter uma string em um inteiro binário ou um inteiro binário em uma string em tempo linear, *a menos que* a base seja uma potência de 2. Mesmo os algoritmos mais conhecidos para a base 10 têm complexidade subquadrática. Converter um valor grande como `int('1' * 500_000)` pode levar mais de um segundo em uma CPU rápida.

Limitar o tamanho da conversão oferece uma maneira prática de evitar **CVE 2020-10735**.

O limite é aplicado ao número de caracteres de dígitos na string de entrada ou saída quando um algoritmo de conversão não linear estiver envolvido. Sublinhados e o sinal não são contados para o limite.

Quando uma operação excede o limite, uma exceção *ValueError* é levantada:

```
>>> import sys
>>> sys.set_int_max_str_digits(4300)  # Illustrativo, este é o padrão.
>>> _ = int('2' * 5432)
Traceback (most recent call last):
```

(continua na próxima página)

(continuação da página anterior)

```
...
ValueError: Exceeds the limit (4300 digits) for integer string conversion: value␣
↪has 5432 digits; use sys.set_int_max_str_digits() to increase the limit
>>> i = int('2' * 4300)
>>> len(str(i))
4300
>>> i_squared = i*i
>>> len(str(i_squared))
Traceback (most recent call last):
...
ValueError: Exceeds the limit (4300 digits) for integer string conversion; use sys.
↪set_int_max_str_digits() to increase the limit
>>> len(hex(i_squared))
7144
>>> assert int(hex(i_squared), base=16) == i*i  # Conversão envolvendo␣
↪hexadecimais é ilimitada.
```

O limite padrão é de 4300 dígitos conforme fornecido em *sys.int_info.default_max_str_digits*. O limite mínimo que pode ser configurado é de 640 dígitos conforme fornecido em *sys.int_info.str_digits_check_threshold*.

Verificação:

```
>>> import sys
>>> assert sys.int_info.default_max_str_digits == 4300, sys.int_info
>>> assert sys.int_info.str_digits_check_threshold == 640, sys.int_info
>>> msg = int('578966293710682886880994035146873798396722250538762761564'
...           '9252925514383915483333812743580549779436104706260696366600'
...           '571186405732').to_bytes(53, 'big')
...
```

Adicionado na versão 3.11.

4.17.1 APIs afetadas

A limitação só se aplica a conversões potencialmente lentas entre *int* e *str* ou *bytes*:

- `int(string)` com padrão sendo base 10.
- `int(string, base)` para todas as bases que não são uma potência de 2.
- `str(integer)`.
- `repr(integer)`.
- qualquer outra conversão de string para base 10 como, por exemplo, `f"{integer}"`, `"{}".format(integer)` ou `b"%d" % integer`.

As limitações não se aplicam a funções com um algoritmo linear:

- `int(string, base)` com base 2, 4, 8, 16 ou 32.
- *int.from_bytes()* e *int.to_bytes()*.
- *hex()*, *oct()*, *bin()*.
- *Minilinguagem de especificação de formato* para números hexa, octal e binários
- *str* para *float*.
- *str* para *decimal.Decimal*.

4.17.2 Configurando o limiter

Antes de iniciar o Python, você pode usar uma variável de ambiente ou um sinalizador de linha de comando do interpretador para configurar o limite:

- `PYTHONINTMAXSTRDIGITS`, por exemplo `PYTHONINTMAXSTRDIGITS=640 python3` para definir o limite para 640 ou `PYTHONINTMAXSTRDIGITS=0 python3` para desabilitar a limitação.
- `-X int_max_str_digits`, por exemplo `python3 -X int_max_str_digits=640`
- *sys.flags.int_max_str_digits* contém o valor de `PYTHONINTMAXSTRDIGITS` ou `-X int_max_str_digits`. Se a variável env e a opção `-X` estiverem definidas, a opção `-X` terá precedência. Um valor de *-1* indica que ambos não foram definidos, portanto, um valor de *sys.int_info.default_max_str_digits* foi usado durante a inicialização.

A partir do código, você pode inspecionar o limite atual e definir um novo usando estas APIs *sys*:

- *sys.get_int_max_str_digits()* e *sys.set_int_max_str_digits()* são um getter e um setter para o limite de todo o interpretador. Os subinterpretadores têm seu próprio limite.

Informações sobre o padrão e o mínimo podem ser encontradas em *sys.int_info*:

- *sys.int_info.default_max_str_digits* é o limite padrão compilado.
- *sys.int_info.str_digits_check_threshold* é o menor valor aceito para o limite (diferente de 0 que o desabilita).

Adicionado na versão 3.11.

Cuidado

Definir um limite baixo *pode* levar a problemas. Embora raro, existe um código que contém constantes inteiras em decimal em sua origem que excedem o limite mínimo. Uma consequência de definir o limite é que o código-fonte do Python contendo literais inteiros decimais maiores que o limite encontrará um erro durante a análise, geralmente no momento da inicialização ou no momento da importação ou até mesmo no momento da instalação – sempre que um `.pyc` atualizado ainda não existe para o código. Uma solução alternativa para source que contém tais constantes grandes é convertê-las para a forma hexadecimal `0x`, pois não há limite.

Teste sua aplicação completamente se você usar um limite baixo. Certifique-se de que seus testes sejam executados com o limite definido anteriormente por meio do ambiente ou sinalizador para que ele seja aplicado durante a inicialização e até mesmo durante qualquer etapa de instalação que possa invocar o Python para pré-compilar fontes `.py` para arquivos `.pyc`.

4.17.3 Configuração recomendada

Espera-se que o padrão *sys.int_info.default_max_str_digits* seja razoável para a maioria das aplicações. Se sua aplicação exigir um limite diferente, defina-o em seu ponto de entrada principal usando código agnóstico de versão Python, pois essas APIs foram adicionadas em lançamentos de patch de segurança em versões anteriores a 3.12.

Exemplo:

```
>>> import sys
>>> if hasattr(sys, "set_int_max_str_digits"):
...     upper_bound = 68000
...     lower_bound = 4004
...     current_limit = sys.get_int_max_str_digits()
...     if current_limit == 0 or current_limit > upper_bound:
...         sys.set_int_max_str_digits(upper_bound)
...     elif current_limit < lower_bound:
...         sys.set_int_max_str_digits(lower_bound)
```

Se você precisar desativá-lo totalmente, defina-o como `0`.

CAPÍTULO 5

Exceções embutidas

No Python, todas as exceções devem ser instâncias de uma classe derivada de *BaseException*. Em uma instrução `try` com uma cláusula `except` que menciona uma classe específica, essa cláusula também lida com qualquer classe de exceção derivada dessa classe (mas não com as classes de exceção a partir das quais *ela* é derivada). Duas classes de exceção que não são relacionadas por subclasse nunca são equivalentes, mesmo que tenham o mesmo nome.

As exceções embutidas listadas neste capítulo podem ser geradas pelo interpretador ou pelas funções embutidas. Exceto onde mencionado, eles têm um "valor associado" indicando a causa detalhada do erro. Pode ser uma string ou uma tupla de vários itens de informação (por exemplo, um código de erro e uma string que explica o código). O valor associado geralmente é passado como argumentos para o construtor da classe de exceção.

O código do usuário pode gerar exceções embutidas. Isso pode ser usado para testar um manipulador de exceções ou para relatar uma condição de erro "exatamente como" a situação na qual o interpretador gera a mesma exceção; mas lembre-se de que nada impede o código do usuário de gerar um erro inadequado.

As classes de exceções embutidas podem ser usadas como subclasses para definir novas exceções; Os programadores são incentivados a derivar novas exceções da classe *Exception* ou de uma de suas subclasses, e não de *BaseException*. Mais informações sobre a definição de exceções estão disponíveis no Tutorial do Python em tut-userexceptions.

5.1 Contexto da exceção

Três atributos em objetos exceções fornecem informações sobre o contexto em que a exceção foi levantada.

`BaseException.__context__`

`BaseException.__cause__`

`BaseException.__suppress_context__`

Ao levantar uma nova exceção enquanto outra exceção já está sendo tratada, o atributo `__context__` da nova exceção é automaticamente definido para a exceção tratada. Uma exceção pode ser tratada quando uma cláusula `except` ou `finally`, ou uma instrução `with`, é usada.

Esse contexto implícito da exceção pode ser complementado com uma causa explícita usando `from` com `raise`:

```
raise new_exc from original_exc
```

A expressão a seguir `from` deve ser uma exceção ou `None`. Ela será definida como `__cause__` na exceção levantada. A definição de `__cause__` também define implicitamente o atributo `__suppress_context__` como `True`, de modo que o uso de `raise new_exc from None` substitui efetivamente a exceção antiga

pela nova para fins de exibição (por exemplo, convertendo *KeyError* para *AttributeError*), deixando a exceção antiga disponível em __context__ para introspecção durante a depuração.

O código de exibição padrão do traceback mostra essas exceções encadeadas, além do traceback da própria exceção. Uma exceção explicitamente encadeada em __cause__ sempre é mostrada quando presente. Uma exceção implicitamente encadeada em __context__ é mostrada apenas se __cause__ for *None* e __suppress_context__ for falso.

Em qualquer um dos casos, a exceção em si sempre é mostrada após todas as exceções encadeadas, de modo que a linha final do traceback sempre mostre a última exceção que foi levantada.

5.2 Herdando de exceções embutidas

O código do usuário pode criar subclasses que herdam de um tipo de exceção. É recomendado criar subclasse de apenas um tipo de exceção por vez para evitar possíveis conflitos entre como as bases tratam o atributo `args`, bem como devido a possíveis incompatibilidades de layout de memória.

A maioria das exceções embutidas são implementadas em C para eficiência, veja: Objects/exceptions.c. Algumas têm layouts de memória personalizados, o que impossibilita a criação de uma subclasse que herda de vários tipos de exceção. O layout de memória de um tipo é um detalhe de implementação e pode mudar entre as versões do Python, levando a novos conflitos no futuro. Portanto, é recomendável evitar criar subclasses de vários tipos de exceção.

5.3 Classes base

As seguintes exceções são usadas principalmente como classes base para outras exceções.

exception BaseException

A classe base para todas as exceções embutidas. Não é para ser herdada diretamente por classes definidas pelo usuário (para isso, use *Exception*). Se *str()* for chamado em uma instância desta classe, a representação do(s) argumento(s) para a instância será retornada ou a string vazia quando não houver argumentos.

args

A tupla de argumentos fornecidos ao construtor de exceções. Algumas exceções embutidas (como *OSError*) esperam um certo número de argumentos e atribuem um significado especial aos elementos dessa tupla, enquanto outras são normalmente chamadas apenas com uma única string que fornece uma mensagem de erro.

with_traceback(*tb*)

Este método define *tb* como o novo traceback (situação da pilha de execução) para a exceção e retorna o objeto exceção. Era mais comumente usado antes que os recursos de encadeamento de exceções de **PEP 3134** se tornassem disponíveis. O exemplo a seguir mostra como podemos converter uma instância de `SomeException` em uma instância de `OtherException` enquanto preservamos o traceback. Uma vez gerado, o quadro atual é empurrado para o traceback de `OtherException`, como teria acontecido com o traceback de `SomeException` original se tivéssemos permitido que ele se propagasse para o chamador.

```
try:
    ...
except SomeException:
    tb = sys.exception().__traceback__
    raise OtherException(...).with_traceback(tb)
```

__traceback__

Um campo gravável que contém o objeto traceback associado a esta exceção. Veja também: raise.

add_note(*note*)

Adiciona a string `note` às notas da exceção que aparecem no traceback padrão após a string de exceção. Uma exceção *TypeError* é levantada se `note` não for uma string.

Adicionado na versão 3.11.

__notes__

Uma lista das notas desta exceção, que foram adicionadas com *add_note()*. Este atributo é criado quando *add_note()* é chamado.

Adicionado na versão 3.11.

exception Exception

Todas as exceções embutidas que não saem para o sistema são derivadas dessa classe. Todas as exceções definidas pelo usuário também devem ser derivadas dessa classe.

exception ArithmeticError

A classe base para as exceções embutidas levantadas para vários erros aritméticos: *OverflowError*, *ZeroDivisionError*, *FloatingPointError*.

exception BufferError

Levantado quando uma operação relacionada a buffer não puder ser realizada.

exception LookupError

A classe base para as exceções levantadas quando uma chave ou índice usado em um mapeamento ou sequência é inválido: *IndexError*, *KeyError*. Isso pode ser levantado diretamente por *codecs.lookup()*.

5.4 Exceções concretas

As seguintes exceções são as que geralmente são levantados.

exception AssertionError

Levantado quando uma instrução `assert` falha.

exception AttributeError

Levantado quando uma referência de atributo (consulte attribute-references) ou atribuição falha. (Quando um objeto não oferece suporte a referências ou atribuições de atributos, *TypeError* é levantado.)

O argumentos somente-nomeados opcionais *name* e o *obj* definem o atributos correspondente:

name

O nome do atributo que tentou-se fazer o acesso.

obj

O objeto que foi acessado para o atributo nomeado.

Alterado na versão 3.10: Adicionado os atributos *name* e *obj*.

exception EOFError

Levantado quando a função *input()* atinge uma condição de fim de arquivo (EOF) sem ler nenhum dado. (Note: os métodos `io.IOBase.read()` e *io.IOBase.readline()* retornam uma string vazia quando pressionam o EOF.)

exception FloatingPointError

Não usado atualmente.

exception GeneratorExit

Levantado quando um *gerador* ou uma *corrotina* está fechado(a); veja `generator.close()` e `coroutine.close()`. Herda diretamente de *BaseException* em vez de *Exception*, já que tecnicamente não é um erro.

exception ImportError

Levantada quando a instrução `import` tem problemas ao tentar carregar um módulo. Também é gerado quando o "from list" em `from ... import` tem um nome que não pode ser encontrado.

O argumentos somente-nomeados opcionais *name* e o *path* definem o atributos correspondente:

name

O nome do módulo que tentou-se fazer a importação.

path

O caminho para qualquer arquivo que acionou a exceção.

Alterado na versão 3.3: Adicionados os atributos *name* e *path*.

exception ModuleNotFoundError

Uma subclasse de *ImportError* que é levantada por `import` quando um módulo não pôde ser localizado. Também é levantada quando `None` é encontrado em *sys.modules*.

Adicionado na versão 3.6.

exception IndexError

Levantada quando um índice de alguma sequência está fora do intervalo. (Índices de fatia são truncados silenciosamente para cair num intervalo permitido; se um índice não for um inteiro, *TypeError* é levantada.)

exception KeyError

Levantada quando uma chave de mapeamento (dicionário) não é encontrada no conjunto de chaves existentes.

exception KeyboardInterrupt

Levantada quando um usuário aperta a tecla de interrupção (normalmente `Control-C` ou `Delete`). Durante a execução, uma checagem de interrupção é feita regularmente. A exceção herda de *BaseException* para que não seja capturada acidentalmente por códigos que tratam *Exception* e assim evita que o interpretador saia.

> **Nota**
>
> Capturar uma *KeyboardInterrupt* requer consideração especial. Como pode ser levantada em pontos imprevisíveis, pode, em algumas circunstâncias, deixar o programa em execução em um estado inconsistente. Geralmente é melhor permitir que *KeyboardInterrupt* termine o programa o mais rápido possível ou evitar levantá-la de todo. (Veja *Nota sobre manipuladores de sinais e exceções*.)

exception MemoryError

Levantada quando uma operação fica sem memória mas a situação ainda pode ser recuperada (excluindo alguns objetos). O valor associado é uma string que indica o tipo de operação (interna) que ficou sem memória. Observe que, por causa da arquitetura de gerenciamento de memória subjacente (função `malloc()` do C), o interpretador pode não ser capaz de se recuperar completamente da situação; no entanto, levanta uma exceção para que um traceback possa ser impresso, no caso de um outro programa ser a causa.

exception NameError

Levantada quando um nome local ou global não é encontrado. Isso se aplica apenas a nomes não qualificados. O valor associado é uma mensagem de erro que inclui o nome que não pode ser encontrado.

O argumento somente-nomeado opcional *name* define o atributo:

name

O nome do variável que tentou-se fazer o acesso.

Alterado na versão 3.10: Adicionado o atributo *name*.

exception NotImplementedError

Essa exceção é derivada da *RuntimeError*. Em classes base, definidas pelo usuário, os métodos abstratos devem gerar essa exceção quando requerem que classes derivadas substituam o método, ou enquanto a classe está sendo desenvolvida, para indicar que a implementação real ainda precisa ser adicionada.

Nota

Não deve ser usada para indicar que um operador ou método não será mais suportado – nesse caso deixe o operador / método indefinido ou, se é uma subclasse, defina-o como `None`.

Cuidado

`NotImplementedError` e `NotImplemented` não são intercambiáveis. Esta exceção deve ser usada somente conforme descrito acima; veja `NotImplemented` para detalhes sobre o uso correto da constante embutida.

exception OSError(*[arg]*)

exception OSError(*errno*, *strerror*[, *filename*[, *winerror*[, *filename2*]]])

Esta exceção é levantada quando uma função do sistema retorna um erro relacionado ao sistema, incluindo falhas do tipo E/S como "file not found" ou "disk full" (não para tipos de argumentos não permitidos ou outro erro acessório).

A segunda forma do construtor definir os atributos correspondentes, descritos abaixo. Os atributos usarão o valor padrão `None` se não forem especificados. Por compatibilidade com versões anteriores, se três argumentos são passados, o atributo `args` contêm somente uma tupla de 2 elementos, os dois primeiros argumentos do construtor.

O construtor geralmente retorna uma subclasse de `OSError`, como descrito abaixo em *OS exceptions* . A subclasse particular depende do valor final de `errno`. Este comportamento ocorre apenas durante a construção direta ou por meio de um apelido de `OSError`, e não é herdado na criação de subclasses.

errno

Um código de erro numérico da variável C `errno`.

winerror

No Windows, isso fornece o código de erro nativo do Windows. O atributo `errno` é então uma tradução aproximada, em termos POSIX, desse código de erro nativo.

No Windows, se o argumento de construtor *winerror* for um inteiro, o atributo `errno` é determinado a partir do código de erro do Windows e o argumento *errno* é ignorado. Em outras plataformas, o argumento *winerror* é ignorado e o atributo `winerror` não existe.

strerror

A mensagem de erro correspondente, conforme fornecida pelo sistema operacional. É formatada pelas funções C `perror()` no POSIX e `FormatMessage()` no Windows.

filename

filename2

Para exceções que envolvem um caminho do sistema de arquivos (como `open()` ou `os.unlink()`), `filename` é o nome do arquivo passado para a função. Para funções que envolvem dois caminhos de sistema de arquivos (como `os.rename()`), `filename2` corresponde ao segundo nome de arquivo passado para a função.

Alterado na versão 3.3: `EnvironmentError`, `IOError`, `WindowsError`, `socket.error`, `select.error` e `mmap.error` foram fundidos em `OSError`, e o construtor pode retornar uma subclasse.

Alterado na versão 3.4: O atributo `filename` agora é o nome do arquivo original passado para a função, ao invés do nome codificado ou decodificado da *tratador de erros e codificação do sistema de arquivos*. Além disso, o argumento e o atributo de construtor *filename2* foi adicionado.

exception OverflowError

Levantada quando o resultado de uma operação aritmética é muito grande para ser representada. Isso não pode ocorrer para inteiros (que prefere levantar `MemoryError` a desistir). No entanto, por motivos históricos,

OverflowError às vezes é levantada para inteiros que estão fora de um intervalo obrigatório. Devido à falta de padronização do tratamento de exceção de ponto flutuante em C, a maioria das operações de ponto flutuante não são verificadas.

exception PythonFinalizationError

Este exceção é derivada de *RuntimeError*. É levantada quando uma operação é bloqueada durante o encerramento do interpretador e também conhecido como *finalização do Python*.

Exemplos de operações que podem ser bloqueadas com um *PythonFinalizationError* durante a finalização do Python:

- Criação de uma nova thread no Python.
- *União* de uma thread de daemon em execução.
- *os.fork()*.

Veja também a função *sys.is_finalizing()*.

Adicionado na versão 3.13: Anteriormente, uma *RuntimeError* simples era levantada.

Alterado na versão 3.14: *threading.Thread.join()* pode agora levantar essa exceção.

exception RecursionError

Esta exceção é derivada de *RuntimeError*. É levantada quando o interpretador detecta que a profundidade máxima de recursão (veja *sys.getrecursionlimit()*) foi excedida.

Adicionado na versão 3.5: Anteriormente, uma *RuntimeError* simples era levantada.

exception ReferenceError

Esta exceção é levantada quando um intermediário de referência fraca, criado pela função *weakref.proxy()*, é usado para acessar um atributo do referente após ter sido coletado como lixo. Para mais informações sobre referências fracas, veja o módulo *weakref*.

exception RuntimeError

Levantada quando um erro é detectado e não se encaixa em nenhuma das outras categorias. O valor associado é uma string indicando o que precisamente deu errado.

exception StopIteration

Levantada pela função embutida *next()* e o método *__next__()* de um *iterador* para sinalizar que não há mais itens produzidos pelo iterador.

value

O objeto exceção tem um único atributo `value`, que é fornecido como um argumento ao construir a exceção, e o padrão é *None*.

Quando uma função *geradora* ou *corrotina* retorna, uma nova instância *StopIteration* é levantada, e o valor retornado pela função é usado como o parâmetro *value* para o construtor da exceção.

Se um código gerador direta ou indiretamente levantar *StopIteration*, ele é convertido em uma *RuntimeError* (mantendo o *StopIteration* como a nova causa da exceção).

Alterado na versão 3.3: Adicionado o atributo `value` e a capacidade das funções geradoras de usá-lo para retornar um valor.

Alterado na versão 3.5: Introduzida a transformação RuntimeError via `from __future__ import generator_stop`, consulte **PEP 479**.

Alterado na versão 3.7: Habilita **PEP 479** para todo o código por padrão: um erro *StopIteration* levantado em um gerador é transformado em uma *RuntimeError*.

exception StopAsyncIteration

Deve ser levantada pelo método `__anext__()` de um objeto *iterador assíncrono* para parar a iteração.

Adicionado na versão 3.5.

exception SyntaxError(*message*, *details*)

Levantada quando o analisador encontra um erro de sintaxe. Isso pode ocorrer em uma instrução `import`, em uma chamada às funções embutidas *compile()*, *exec()* ou *eval()*, ou ao ler o script inicial ou entrada padrão (também interativamente).

A função *str()* da instância de exceção retorna apenas a mensagem de erro. Detalhes é uma tupla cujos membros também estão disponíveis como atributos separados.

filename

O nome do arquivo em que ocorreu o erro de sintaxe.

lineno

Em qual número de linha no arquivo o erro ocorreu. Este é indexado em 1: a primeira linha no arquivo tem um `lineno` de 1.

offset

A coluna da linha em que ocorreu o erro. Este é indexado em 1: o primeiro caractere na linha tem um `offset` de 1.

text

O texto do código-fonte envolvido no erro.

end_lineno

Em qual número de linha no arquivo o erro ocorrido termina. Este é indexado em 1: a primeira linha no arquivo tem um `lineno` de 1.

end_offset

A coluna da linha final em que erro ocorrido finaliza Este é indexado em 1: o primeiro caractere na linha tem um `offset` de 1.

Para erros em campos de f-string, a mensagem é prefixada por "f-string: " e os "offsets" são deslocamentos em um texto construído a partir da expressão de substituição. Por exemplo, compilar o campo f'Bad {a b}' resulta neste atributo de argumentos: ('f-string: …', (", 1, 2, '(a b)n', 1, 5)).

Alterado na versão 3.10: Adicionado os atributos *end_lineno* e *end_offset*.

exception IndentationError

Classe base para erros de sintaxe relacionados a indentação incorreta. Esta é uma subclasse de *SyntaxError*.

exception TabError

Levantada quando o indentação contém um uso inconsistente de tabulações e espaços. Esta é uma subclasse de *IndentationError*.

exception SystemError

Levantada quando o interpretador encontra um erro interno, mas a situação não parece tão grave para fazer com que perca todas as esperanças. O valor associado é uma string que indica o que deu errado (em termos de baixo nível). Em *CPython*, isso pode ser causado pelo uso incorreto da API C do Python, como retornar um valor `NULL` sem um conjunto de exceções.

Se você tem certeza de que essa exceção não foi culpa sua ou de um pacote que você está usando, você deve relatar isso ao autor ou mantenedor do seu interpretador Python. Certifique-se de relatar a versão do interpretador Python (`sys.version`; também é impresso no início de uma sessão Python interativa), a mensagem de erro exata (o valor associado da exceção) e se possível a fonte do programa que acionou o erro.

exception SystemExit

Esta exceção é levantada pela função *sys.exit()*. Ele herda de *BaseException* em vez de *Exception* para que não seja acidentalmente capturado pelo código que captura *Exception*. Isso permite que a exceção se propague corretamente e faça com que o interpretador saia. Quando não é tratado, o interpretador Python sai; nenhum traceback (situação da pilha de execução) é impresso. O construtor aceita o mesmo argumento opcional passado para *sys.exit()*. Se o valor for um inteiro, ele especifica o status de saída do sistema (passado para a função C `exit()`); se for `None`, o status de saída é zero; se tiver outro tipo (como uma string), o valor do objeto é exibido e o status de saída é um.

Uma chamada para *sys.exit()* é traduzida em uma exceção para que os tratadores de limpeza (cláusulas `finally` das instruções `try`) possam ser executados, e para que um depurador possa executar um script sem correr o risco de perder o controle. A função *os._exit()* pode ser usada se for absolutamente necessário sair imediatamente (por exemplo, no processo filho após uma chamada para *os.fork()*).

code

O status de saída ou mensagem de erro transmitida ao construtor. (O padrão é `None`.)

exception TypeError

Levantada quando uma operação ou função é aplicada a um objeto de tipo inadequado. O valor associado é uma string que fornece detalhes sobre a incompatibilidade de tipo.

Essa exceção pode ser levantada pelo código do usuário para indicar que uma tentativa de operação em um objeto não é suportada e não deveria ser. Se um objeto deve ter suporte a uma dada operação, mas ainda não forneceu uma implementação, *NotImplementedError* é a exceção apropriada a ser levantada.

Passar argumentos do tipo errado (por exemplo, passar uma *list* quando um *int* é esperado) deve resultar em uma *TypeError*, mas passar argumentos com o valor errado (por exemplo, um número fora limites esperados) deve resultar em uma *ValueError*.

exception UnboundLocalError

Levantada quando uma referência é feita a uma variável local em uma função ou método, mas nenhum valor foi vinculado a essa variável. Esta é uma subclasse de *NameError*.

exception UnicodeError

Levantada quando ocorre um erro de codificação ou decodificação relacionado ao Unicode. É uma subclasse de *ValueError*.

UnicodeError possui atributos que descrevem o erro de codificação ou decodificação. Por exemplo, `err.object[err.start:err.end]` fornece a entrada inválida específica na qual o codec falhou.

encoding

O nome da codificação que levantou o erro.

reason

Uma string que descreve o erro de codec específico.

object

O objeto que o codec estava tentando codificar ou decodificar.

start

O primeiro índice de dados inválidos em *object*.

Este valor não deve ser negativo, pois é interpretado como um deslocamento absoluto, mas essa restrição não é aplicada em tempo de execução.

end

O índice após os últimos dados inválidos em *object*.

Este valor não deve ser negativo, pois é interpretado como um deslocamento absoluto, mas essa restrição não é aplicada em tempo de execução.

exception UnicodeEncodeError

Levantada quando ocorre um erro relacionado ao Unicode durante a codificação. É uma subclasse de *UnicodeError*.

exception UnicodeDecodeError

Levantada quando ocorre um erro relacionado ao Unicode durante a decodificação. É uma subclasse de *UnicodeError*.

exception UnicodeTranslateError

Levantada quando ocorre um erro relacionado ao Unicode durante a tradução. É uma subclasse de *UnicodeError*.

exception ValueError

Levantada quando uma operação ou função recebe um argumento que tem o tipo certo, mas um valor inadequado, e a situação não é descrita por uma exceção mais precisa, como *IndexError*.

exception ZeroDivisionError

Levantada quando o segundo argumento de uma divisão ou operação de módulo é zero. O valor associado é uma string que indica o tipo dos operandos e a operação.

As seguintes exceções são mantidas para compatibilidade com versões anteriores; a partir do Python 3.3, eles são apelidos de *OSError*.

exception EnvironmentError

exception IOError

exception WindowsError

Disponível apenas no Windows.

5.4.1 Exceções de sistema operacional

As seguintes exceções são subclasses de *OSError*, elas são levantadas dependendo do código de erro do sistema.

exception BlockingIOError

Levantada quando uma operação bloquearia em um objeto (por exemplo, soquete) definido para operação sem bloqueio. Corresponde a `errno` *EAGAIN*, *EALREADY*, *EWOULDBLOCK* e *EINPROGRESS*.

Além daquelas de *OSError*, *BlockingIOError* pode ter mais um atributo:

characters_written

Um inteiro contendo o número de caracteres gravados no fluxo antes de ser bloqueado. Este atributo está disponível ao usar as classes de E/S em buffer do módulo *io*.

exception ChildProcessError

Levantada quando uma operação em um processo filho falha. Corresponde a `errno` *ECHILD*.

exception ConnectionError

Uma classe base para problemas relacionados à conexão.

Suas subclasses são *BrokenPipeError*, *ConnectionAbortedError*, *ConnectionRefusedError* e *ConnectionResetError*.

exception BrokenPipeError

Uma subclasse de *ConnectionError*, levantada ao tentar escrever em um encadeamento, enquanto a outra extremidade foi fechada, ou ao tentar escrever em um soquete que foi desligado para escrita. Corresponde a `errno` *EPIPE* e *ESHUTDOWN*.

exception ConnectionAbortedError

Uma subclasse de *ConnectionError*, levantada quando uma tentativa de conexão é cancelada pelo par. Corresponde a `errno` *ECONNABORTED*.

exception ConnectionRefusedError

Uma subclasse de *ConnectionError*, levantada quando uma tentativa de conexão é recusada pelo par. Corresponde a `errno` *ECONNREFUSED*.

exception ConnectionResetError

Uma subclasse de *ConnectionError*, levantada quando uma conexão é redefinida pelo par. Corresponde a `errno` *ECONNRESET*.

exception FileExistsError

Levantada ao tentar criar um arquivo ou diretório que já existe. Corresponde a `errno` *EEXIST*.

exception FileNotFoundError

Levantada quando um arquivo ou diretório é solicitado, mas não existe. Corresponde a `errno` *ENOENT*.

exception InterruptedError

Levantada quando uma chamada do sistema é interrompida por um sinal de entrada. Corresponde a errno *EINTR*.

Alterado na versão 3.5: Python agora tenta novamente chamadas de sistema quando uma *syscall* é interrompida por um sinal, exceto se o tratador de sinal levanta uma exceção (veja **PEP 475** para a justificativa), em vez de levantar *InterruptedError*.

exception IsADirectoryError

Levantada quando uma operação de arquivo (como *os.remove()*) é solicitada em um diretório. Corresponde a errno *EISDIR*.

exception NotADirectoryError

Levantada quando uma operação de diretório (como *os.listdir()*) é solicitada em algo que não é um diretório. Na maioria das plataformas POSIX, ela também pode ser levantada se uma operação tentar abrir ou percorrer um arquivo não pertencente ao diretório como se fosse um diretório. Corresponde a errno *ENOTDIR*.

exception PermissionError

Levantada ao tentar executar uma operação sem os direitos de acesso adequados - por exemplo, permissões do sistema de arquivos. Corresponde a errno *EACCES*, *EPERM*, e *ENOTCAPABLE*.

Alterado na versão 3.11.1: *ENOTCAPABLE* do WASI agora é mapeado para *PermissionError*.

exception ProcessLookupError

Levantada quando um determinado processo não existe. Corresponde a errno *ESRCH*.

exception TimeoutError

Levantada quando uma função do sistema expirou no nível do sistema. Corresponde a errno *ETIMEDOUT*.

Adicionado na versão 3.3: Todas as subclasses de *OSError* acima foram adicionadas.

Ver também

PEP 3151 - Reworking the OS and IO exception hierarchy

5.5 Avisos

As seguintes exceções são usadas como categorias de aviso; veja a documentação de *Categorias de avisos* para mais detalhes.

exception Warning

Classe base para categorias de aviso.

exception UserWarning

Classe base para avisos gerados pelo código do usuário.

exception DeprecationWarning

Classe base para avisos sobre recursos descontinuados quando esses avisos se destinam a outros desenvolvedores Python.

Ignorado pelos filtros de aviso padrão, exceto no módulo __main__ (**PEP 565**). Habilitar o *Modo de Desenvolvimento do Python* mostra este aviso.

A política de descontinuação está descrita na **PEP 387**.

exception PendingDeprecationWarning

Classe base para avisos sobre recursos que foram descontinuados e devem ser descontinuados no futuro, mas não foram descontinuados ainda.

Esta classe raramente é usada para emitir um aviso sobre uma possível descontinuação futura, é incomum, e *DeprecationWarning* é preferível para descontinuações já ativas.

Ignorado pelos filtros de aviso padrão. Habilitar o *Modo de Desenvolvimento do Python* mostra este aviso.

A política de descontinuação está descrita na **PEP 387**.

exception SyntaxWarning

Classe base para avisos sobre sintaxe duvidosa.

Este aviso normalmente é emitido ao compilar código-fonte Python e normalmente não será relatado ao executar código já compilado.

exception RuntimeWarning

Classe base para avisos sobre comportamento duvidoso de tempo de execução.

exception FutureWarning

Classe base para avisos sobre recursos descontinuados quando esses avisos se destinam a usuários finais de aplicações escritas em Python.

exception ImportWarning

Classe base para avisos sobre prováveis erros na importação de módulos.

Ignorado pelos filtros de aviso padrão. Habilitar o *Modo de Desenvolvimento do Python* mostra este aviso.

exception UnicodeWarning

Classe base para avisos relacionados a Unicode.

exception EncodingWarning

Classe base para avisos relacionados a codificações.

Veja *Ativar EncodingWarning* para detalhes.

Adicionado na versão 3.10.

exception BytesWarning

Classe base para avisos relacionados a `bytes` e `bytearray`.

exception ResourceWarning

Classe base para avisos relacionados a uso de recursos.

Ignorado pelos filtros de aviso padrão. Habilitar o *Modo de Desenvolvimento do Python* mostra este aviso.

Adicionado na versão 3.2.

5.6 Grupos de exceções

Os itens a seguir são usados quando é necessário levantar várias exceções não relacionadas. Eles fazem parte da hierarquia de exceção, portanto, podem ser tratados com `except` como todas as outras exceções. Além disso, eles são reconhecidos por `except*`, que corresponde a seus subgrupos com base nos tipos de exceções contidas.

exception ExceptionGroup(*msg*, *excs*)

exception BaseExceptionGroup(*msg*, *excs*)

Ambos os tipos de exceção agrupam as exceções na sequência `excs`. O parâmetro `msg` deve ser uma string. A diferença entre as duas classes é que `BaseExceptionGroup` estende `BaseException` e pode envolver qualquer exceção, enquanto `ExceptionGroup` estende `Exception` e só pode agrupar subclasses de `Exception`. Este design é para que `except Exception` capture um `ExceptionGroup` mas não `BaseExceptionGroup`.

O construtor de `BaseExceptionGroup` retorna uma `ExceptionGroup` ao invés de uma `BaseExceptionGroup` se todas as exceções contidas forem instâncias `Exception`, então ela pode ser usada para tornar a seleção automática. O construtor de `ExceptionGroup`, por outro lado, levanta `TypeError` se qualquer exceção contida não for uma subclasse de `Exception`.

message

O argumento `msg` para o construtor. Este é um atributo somente leitura.

exceptions

Uma tupla de exceções na sequência `excs` dada ao construtor. Este é um atributo somente leitura.

subgroup(*condition*)

Retorna um grupo de exceções que contém apenas as exceções do grupo atual que correspondem à condição *condition* ou `None` se o resultado estiver vazio.

A condição pode ser um tipo exceção ou tupla de tipos de exceção. Nesse caso, cada exceção é verificada quanto a um match usando a mesma verificação que é usada em uma cláusula de `except`. A condição também pode ser um chamável (diferente de um objeto de tipo) que aceita uma exceção como seu único argumento e retorna verdadeiro para a exceção que deve estar no subgrupo.

A estrutura de aninhamento da exceção atual é preservada no resultado, assim como os valores de seus campos *message*, *__traceback__*, *__cause__*, *__context__* e *__notes__*. Grupos aninhados vazios são omitidos do resultado.

A condição é verificada para todas as exceções no grupo de exceções aninhadas, incluindo o nível superior e quaisquer grupos de exceções aninhadas. Se a condição for verdadeira para tal grupo de exceções, ela será incluída no resultado por completo.

Adicionado na versão 3.13: `condition` pode ser qualquer chamável que não seja um objeto de tipo.

split(*condition*)

Como *subgroup()*, mas retorna o par `(match, rest)` onde `match` é `subgroup(condition)` e `rest` é a parte restante não correspondente.

derive(*excs*)

Retorna um grupo de exceções com o mesmo *message*, mas que agrupa as exceções em `excs`.

Este método é usado por *subgroup()* e *split()*, que são usados em vários contextos para dividir um grupo de exceções. Uma subclasse precisa substituí-la para fazer com que *subgroup()* e *split()* retorne instâncias da subclasse em vez de *ExceptionGroup*.

subgroup() e *split()* copiam os campos *__traceback__*, *__cause__*, *__context__* e *__notes__* do grupo de exceções original para o retornado por *derive()*, então esses campos não precisam ser atualizados por *derive()*.

```
>>> class MyGroup(ExceptionGroup):
...     def derive(self, excs):
...         return MyGroup(self.message, excs)
...
>>> e = MyGroup("eg", [ValueError(1), TypeError(2)])
>>> e.add_note("a note")
>>> e.__context__ = Exception("context")
>>> e.__cause__ = Exception("cause")
>>> try:
...     raise e
... except Exception as e:
...     exc = e
...
>>> match, rest = exc.split(ValueError)
>>> exc, exc.__context__, exc.__cause__, exc.__notes__
(MyGroup('eg', [ValueError(1), TypeError(2)]), Exception('context'),␣
↪Exception('cause'), ['a note'])
>>> match, match.__context__, match.__cause__, match.__notes__
(MyGroup('eg', [ValueError(1)]), Exception('context'), Exception('cause'),␣
↪['a note'])
>>> rest, rest.__context__, rest.__cause__, rest.__notes__
(MyGroup('eg', [TypeError(2)]), Exception('context'), Exception('cause'), [
↪'a note'])
>>> exc.__traceback__ is match.__traceback__ is rest.__traceback__
True
```

Observe que *BaseExceptionGroup* define `__new__()`, então subclasses que precisam de uma assinatura de construtor diferente precisam substituir isso ao invés de `__init__()`. Por exemplo, o seguinte define uma subclasse de grupo de exceções que aceita um exit_code e constrói a mensagem do grupo a partir dele.

```
class Errors(ExceptionGroup):
   def __new__(cls, errors, exit_code):
      self = super().__new__(Errors, f"exit code: {exit_code}", errors)
      self.exit_code = exit_code
      return self

   def derive(self, excs):
      return Errors(excs, self.exit_code)
```

Como *ExceptionGroup*, qualquer subclasse de *BaseExceptionGroup* que também é uma subclasse de *Exception* só pode agrupar instâncias de *Exception*.

Adicionado na versão 3.11.

5.7 Hierarquia das exceções

A hierarquia de classes para exceções embutidas é:

```
BaseException
 ├── BaseExceptionGroup
 ├── GeneratorExit
 ├── KeyboardInterrupt
 ├── SystemExit
 └── Exception
      ├── ArithmeticError
      │    ├── FloatingPointError
      │    ├── OverflowError
      │    └── ZeroDivisionError
      ├── AssertionError
      ├── AttributeError
      ├── BufferError
      ├── EOFError
      ├── ExceptionGroup [BaseExceptionGroup]
      ├── ImportError
      │    └── ModuleNotFoundError
      ├── LookupError
      │    ├── IndexError
      │    └── KeyError
      ├── MemoryError
      ├── NameError
      │    └── UnboundLocalError
      ├── OSError
      │    ├── BlockingIOError
      │    ├── ChildProcessError
      │    ├── ConnectionError
      │    │    ├── BrokenPipeError
      │    │    ├── ConnectionAbortedError
      │    │    ├── ConnectionRefusedError
      │    │    └── ConnectionResetError
      │    ├── FileExistsError
      │    ├── FileNotFoundError
      │    ├── InterruptedError
      │    ├── IsADirectoryError
      │    ├── NotADirectoryError
```

(continua na próxima página)

(continuação da página anterior)

```
      │    ├── PermissionError
      │    ├── ProcessLookupError
      │    └── TimeoutError
      ├── ReferenceError
      ├── RuntimeError
      │    ├── NotImplementedError
      │    ├── PythonFinalizationError
      │    └── RecursionError
      ├── StopAsyncIteration
      ├── StopIteration
      ├── SyntaxError
      │    └── IndentationError
      │         └── TabError
      ├── SystemError
      ├── TypeError
      ├── ValueError
      │    └── UnicodeError
      │         ├── UnicodeDecodeError
      │         ├── UnicodeEncodeError
      │         └── UnicodeTranslateError
      └── Warning
           ├── BytesWarning
           ├── DeprecationWarning
           ├── EncodingWarning
           ├── FutureWarning
           ├── ImportWarning
           ├── PendingDeprecationWarning
           ├── ResourceWarning
           ├── RuntimeWarning
           ├── SyntaxWarning
           ├── UnicodeWarning
           └── UserWarning
```

CAPÍTULO 6

Serviços de Processamento de Texto

Os módulos descritos neste capítulo fornecem uma ampla variedade de operações de manipulação de string e outros serviços de processamento de texto.

O módulo `codecs` descrito em *Serviços de Dados Binários* também é altamente relevante para o processamento de texto. Além disso, consulte a documentação do tipo string do Python em *Tipo sequência de texto — str*.

6.1 `string` — Operações comuns de strings

Código-fonte: Lib/string.py

Ver também

Tipo sequência de texto — str

Métodos de string

6.1.1 Constantes de strings

As constantes definidas neste módulo são:

`string.`**`ascii_letters`**

A concatenação das constantes `ascii_lowercase` e `ascii_uppercase` descritas abaixo. Este valor não depende da localidade.

`string.`**`ascii_lowercase`**

As letras minúsculas `'abcdefghijklmnopqrstuvwxyz'`. Este valor não depende da localidade e não mudará.

`string.`**`ascii_uppercase`**

As letras maiúsculas `'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. Este valor não depende da localidade e não mudará.

`string.`**`digits`**

A string `'0123456789'`.

string.**hexdigits**

A string `'0123456789abcdefABCDEF'`.

string.**octdigits**

A string `'01234567'`.

string.**punctuation**

String de caracteres ASCII que são considerados caracteres de pontuação na localidade `C`: `!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~`.

string.**printable**

String de caracteres ASCII que são considerados imprimíveis pelo Python. Esta é uma combinação de *digits*, *ascii_letters*, *punctuation* e *whitespace*.

> **Nota**
>
> Por design, *string.printable.isprintable()* retorna *False*. Em particular, `string.printable` não é imprimível no sentido POSIX (veja *LC_CTYPE*).

string.**whitespace**

Uma string contendo todos os caracteres ASCII que são considerados espaços em branco. Isso inclui espaço de caracteres, tabulação, avanço de linha, retorno, avanço de formulário e tabulação vertical.

6.1.2 Formatação personalizada de strings

A classe embutida de string fornece a capacidade de fazer substituições de variáveis complexas e formatação de valor por meio do método *format()* descrito na **PEP 3101**. A classe *Formatter* no módulo *string* permite que você crie e personalize seus próprios comportamentos de formatação de strings usando a mesma implementação que o método embutido *format()*.

class string.**Formatter**

A classe *Formatter* tem os seguintes métodos públicos:

format(*format_string*, /, **args*, ***kwargs*)

O método principal da API. Ele aceita uma string de formato e um conjunto arbitrário de argumentos posicionais e nomeados. É apenas um invólucro que chama *vformat()*.

Alterado na versão 3.7: Um argumento de string de formato é agora *somente-posicional*.

vformat(*format_string*, *args*, *kwargs*)

Esta função realiza o trabalho real de formatação. Ela é exposta como uma função separada para casos onde você deseja passar um dicionário predefinido de argumentos, ao invés de desempacotar e empacotar novamente o dicionário como argumentos individuais usando a sintaxe `*args` e `**kwargs`. *vformat()* faz o trabalho de quebrar a string de formato em dados de caracteres e campos de substituição. Ela chama os vários métodos descritos abaixo.

Além disso, o *Formatter* define uma série de métodos que devem ser substituídos por subclasses:

parse(*format_string*)

Percorre format_string e retorna um iterável de tuplas (*literal_text*, *field_name*, *format_spec*, *conversion*). Isso é usado por *vformat()* para quebrar a string em texto literal ou campos de substituição.

Os valores na tupla representam conceitualmente um intervalo de texto literal seguido por um único campo de substituição. Se não houver texto literal (o que pode acontecer se dois campos de substituição ocorrerem consecutivamente), então *literal_text* será uma string de comprimento zero. Se não houver campo de substituição, então os valores de *field_name*, *format_spec* e *conversion* serão `None`. O valor de *field_name* não é modificado e a numeração automática de campos posicionais não numerados é feita por *vformat()*.

get_field(*field_name*, *args*, *kwargs*)

Dado *field_name*, converte-o em um objeto a ser formatado. A autonumeração de *field_name* retornada de `parse()` é feira por `vformat()` antes de chamar este método. Retorna uma tupla (obj, used_key). A versão padrão aceita strings no formato definido na **PEP 3101**, como "0[name]" ou "label.title". *args* e *kwargs* são como passados para `vformat()`. O valor de retorno *used_key* tem o mesmo significado que o parâmetro *key* para `get_value()`.

get_value(*key*, *args*, *kwargs*)

Obtém um determinado valor de campo. O argumento *key* será um inteiro ou uma string. Se for um inteiro, ele representa o índice do argumento posicional em *args*; se for uma string, então representa um argumento nomeado em *kwargs*.

O parâmetro *args* é definido para a lista de argumentos posicionais para `vformat()`, e o parâmetro *kwargs* é definido para o dicionário de argumentos nomeados.

Para nomes de campos compostos, essas funções são chamadas apenas para o primeiro componente do nome do campo; os componentes subsequentes são tratados por meio de operações normais de atributo e indexação.

Então, por exemplo, a expressão de campo '0.name' faria com que `get_value()` fosse chamado com um argumento *key* de 0. O atributo `name` será pesquisado após `get_value()` retorna chamando a função embutida `getattr()`.

Se o índice ou palavra-chave se referir a um item que não existe, um `IndexError` ou `KeyError` deve ser levantada.

check_unused_args(*used_args*, *args*, *kwargs*)

Implementa a verificação de argumentos não usados, se desejar. Os argumentos para esta função são o conjunto de todas as chaves de argumento que foram realmente referidas na string de formato (inteiros para argumentos posicionais e strings para argumentos nomeados) e uma referência a *args* e *kwargs* que foi passada para vformat. O conjunto de argumentos não utilizados pode ser calculado a partir desses parâmetros. Presume-se que `check_unused_args()` levata uma exceção se a verificação falhar.

format_field(*value*, *format_spec*)

`format_field()` simplesmente chama o global embutido `format()`. O método é fornecido para que as subclasses possam substituí-lo.

convert_field(*value*, *conversion*)

Converte o valor (retornado por `get_field()`) dado um tipo de conversão (como na tupla retornada pelo método `parse()`). A versão padrão entende os tipos de conversão "s" (str), "r" (repr) e "a" (ascii).

6.1.3 Sintaxe das strings de formato

The `str.format()` method and the `Formatter` class share the same syntax for format strings (although in the case of `Formatter`, subclasses can define their own format string syntax). The syntax is related to that of formatted string literals and template string literals, but it is less sophisticated and, in particular, does not support arbitrary expressions in interpolations.

As strings de formato contêm "campos de substituição" entre chaves `{}`. Tudo o que não estiver entre chaves é considerado texto literal, que é copiado inalterado para a saída. Se você precisar incluir um caractere de chave no texto literal, ele pode ser escapado duplicando: `{{` e `}}`.

A gramática para um campo de substituição é a seguinte:

```
replacement_field: "{" [field_name] ["!" conversion] [":" format_spec] "}"
field_name:        arg_name ("." attribute_name | "[" element_index "]")*
arg_name:          [identifier | digit+]
attribute_name:    identifier
element_index:     digit+ | index_string
index_string:      <any source character except "]"> +
conversion:        "r" | "s" | "a"
format_spec:       format-spec:format_spec
```

Em termos menos formais, o campo de substituição pode começar com um *field_name* que especifica o objeto cujo valor deve ser formatado e inserido na saída em vez do campo de substituição. O *field_name* é opcionalmente seguido por um campo *conversion*, que é precedido por um ponto de exclamação `'!'`, e um *format_spec*, que é precedido por dois pontos `':'`. Eles especificam um formato não padrão para o valor de substituição.

Veja também a seção *Minilinguagem de especificação de formato*.

O próprio *field_name* começa com um *arg_name* que é um número ou uma palavra-chave. Se for um número, ele se refere a um argumento posicional, e se for uma palavra-chave, ele se refere a um argumento nomeado. Um *arg_name* é tratado como um número se uma chamada para `str.isdecimal()` na string retorna verdadeira. Se os arg_names numéricos em uma string de formato forem 0, 1, 2, … em sequência, eles podem ser todos omitidos (não apenas alguns) e os números 0, 1, 2, … serão inseridos automaticamente nessa ordem. Como *arg_name* não é delimitado por aspas, não é possível especificar chaves de dicionário arbitrárias (por exemplo, as strings `'10'` ou `':-]'`) dentro de uma string de formato. O *arg_name* pode ser seguido por qualquer número de expressões de índice ou atributo. Uma expressão da forma `'.name'` seleciona o atributo nomeado usando `getattr()`, enquanto uma expressão da forma `'[index]'` faz uma pesquisa de índice usando `__getitem__()`.

Alterado na versão 3.1: Os especificadores de argumento posicional podem ser omitidos para `str.format()`, de forma que `'{} {}'.format(a, b)` é equivalente a `'{0} {1}'.format(a, b)`.

Alterado na versão 3.4: Os especificadores de argumento posicional podem ser omitidos para `Formatter`.

Alguns exemplos simples de string de formato:

```
"Primeiro, tu deverás contar até {0}"  # Referencia o primeiro argumento posicional
"Traga-me um {}"                       # Referencia implicitamente o primeiro␣
↪argumento posicional
"De {} para {}"                        # Mesmo que "De {0} para {1}"
"Minha missão é {nome}"                # Referencia argumento nomeado 'nome'
"Peso em toneladas {0.peso}"           # O atributo 'peso' do primeiro argumento␣
↪posicional
"Unidades destruídas: {jogadores[0]}"  # Primeiro elemento do argumento nomeado
↪'jogadores'.
```

O campo *conversion* causa uma coerção de tipo antes da formatação. Normalmente, o trabalho de formatação de um valor é feito pelo método `__format__()` do próprio valor. No entanto, em alguns casos, é desejável forçar um tipo a ser formatado como uma string, substituindo sua própria definição de formatação. Ao converter o valor em uma string antes de chamar `__format__()`, a lógica de formatação normal é contornada.

Três sinalizadores de conversão são atualmente suportados: `'!s'`, que chama `str()` no valor; `'!r'`, que chama `repr()`; e `'!a'`, que chama `ascii()`.

Alguns exemplos:

```
"Harold's a clever {0!s}"        # Chama str() no primeiro argumento
"Bring out the holy {name!r}"    # Chama repr() no primeiro argumento
"More {!a}"                      # Chama ascii() no primeiro argumento
```

O campo *format_spec* contém uma especificação de como o valor deve ser apresentado, incluindo detalhes como largura do campo, alinhamento, preenchimento, precisão decimal e assim por diante. Cada tipo de valor pode definir sua própria "minilinguagem de formatação" ou interpretação de *format_spec*.

A maioria dos tipos embutidos oferece suporte a uma minilinguagem de formatação comum, que é descrita na próxima seção.

Um campo *format_spec* também pode incluir campos de substituição aninhados dentro dele. Esses campos de substituição aninhados podem conter um nome de campo, sinalizador de conversão e especificação de formato, mas um aninhamento mais profundo não é permitido. Os campos de substituição em format_spec são substituídos antes que a string *format_spec* seja interpretada. Isso permite que a formatação de um valor seja especificada dinamicamente.

Veja a seção *Exemplos de formato* para alguns exemplos.

Minilinguagem de especificação de formato

"Format specifications" are used within replacement fields contained within a format string to define how individual values are presented (see *Sintaxe das strings de formato*, f-strings, and t-strings). They can also be passed directly to the built-in `format()` function. Each formattable type may define how the format specification is to be interpreted.

A maioria dos tipos embutidos implementa as seguintes opções para especificações de formato, embora algumas das opções de formatação sejam suportadas apenas pelos tipos numéricos.

Uma convenção geral é que uma especificação de formato vazia produz o mesmo resultado como se você tivesse chamado `str()` no valor. Uma especificação de formato não vazio normalmente modifica o resultado.

A forma geral de um *especificador de formato padrão* é:

```
format_spec:              [options][width_and_precision][type]
options:                  [[fill]align][sign]["z"]["#"]["0"]
fill:                     <any character>
align:                    "<" | ">" | "=" | "^"
sign:                     "+" | "-" | " "
width_and_precision:      [width_with_grouping][precision_with_grouping]
width_with_grouping:      [width][grouping]
precision_with_grouping: "." [precision][grouping] | "." grouping
width:                    digit+
precision:                digit+
grouping:                 "," | "_"
type:                     "b" | "c" | "d" | "e" | "E" | "f" | "F" | "g"
                          | "G" | "n" | "o" | "s" | "x" | "X" | "%"
```

Se um valor *align* válido for especificado, ele pode ser precedido por um caractere de preenchimento *fill* que pode ser qualquer caractere e o padrão é um espaço se omitido. Não é possível usar uma chave literal ("`{`" ou "`}`") como o caractere *fill* em uma string formatada literal ou ao usar o método `str.format()`. No entanto, é possível inserir uma chave com um campo de substituição aninhado. Esta limitação não afeta a função `format()`.

O significado das várias opções de alinhamento é o seguinte:

Opção	Significado
`'<'`	Força o alinhamento à esquerda do campo dentro do espaço disponível (este é o padrão para a maioria dos objetos).
`'>'`	Força o alinhamento à direita do campo dentro do espaço disponível (este é o padrão para números).
`'='`	Força o preenchimento a ser colocado após o sinal (se houver), mas antes dos dígitos. É usado para imprimir campos na forma "+000000120". Esta opção de alinhamento só é válida para tipos numéricos, excluindo `complex`. Torna-se o padrão para números quando "0" precede imediatamente a largura do campo.
`'^'`	Força a centralização do campo no espaço disponível.

Observe que, a menos que uma largura de campo mínima seja definida, a largura do campo sempre será do mesmo tamanho que os dados para preenchê-lo, de modo que a opção de alinhamento não tem significado neste caso.

A opção *sign* só é válida para tipos numéricos e pode ser um dos seguintes:

Opção	Significado
`'+'`	indica que um sinal deve ser usado para números positivos e negativos.
`'-'`	indica que um sinal deve ser usado apenas para números negativos (este é o comportamento padrão).
espaço	indica que um espaço inicial deve ser usado em números positivos e um sinal de menos em números negativos.

A opção `'z'` força valores de ponto flutuante de zero negativo para zero positivo após o arredondamento para a precisão do formato. Esta opção só é válida para tipos de apresentação de ponto flutuante.

Alterado na versão 3.11: Adicionada a opção `'z'` (veja também **PEP 682**).

A opção `'#'` faz com que a "forma alternativa" seja usada para a conversão. A forma alternativa é definida de forma diferente para diferentes tipos. Esta opção é válida apenas para tipos inteiros, pontos flutuantes e complexos. Para inteiros, quando a saída binária, octal ou hexadecimal é usada, esta opção adiciona o prefixo respectivo `'0b'`, `'0o'`, `'0x'` ou `'0X'` ao valor de saída. Para pontos flutuante e complexo, a forma alternativa faz com que o resultado da conversão sempre contenha um caractere de ponto decimal, mesmo se nenhum dígito o seguir. Normalmente, um caractere de ponto decimal aparece no resultado dessas conversões apenas se um dígito o seguir. Além disso, para conversões `'g'` e `'G'`, os zeros finais não são removidos do resultado.

O *width* é um número inteiro decimal que define a largura total mínima do campo, incluindo quaisquer prefixos, separadores e outros caracteres de formatação. Se não for especificado, a largura do campo será determinada pelo conteúdo.

Quando nenhum alinhamento explícito é fornecido, preceder o campo *width* com um caractere zero (`'0'`) habilita o preenchimento por zero com reconhecimento de sinal para tipos numéricos excluindo *complex*. Isso é equivalente a um caractere de *fill* de valor `'0'` com um tipo de *alignment* de `'='`.

Alterado na versão 3.10: Precedendo o campo *width* com `'0'` não afeta mais o alinhamento padrão para strings.

precision é um número decimal que indica quantos dígitos devem ser exibidos depois do ponto decimal para um valor de ponto flutuante formatado com `'f'` e `'F'`, ou antes e depois do ponto decimal para um valor de ponto flutuante formatado com `'g'` ou `'G'`. Para tipos não numéricos, o campo indica o tamanho máximo do campo – em outras palavras, quantos caracteres serão usados do conteúdo do campo. *precision* não é permitido para valores inteiros.

A opção *grouping* após os campos *width* e *precision* especifica um separador de grupo de dígitos para as partes inteira e fracionária de um número, respectivamente. Pode ser um dos seguintes:

Opção	Significado
`','`	Insere uma vírgula a cada 3 dígitos para tipos de apresentação inteiros `'d'` e de ponto flutuante, excluindo `'n'`. Para outros tipos de apresentação, esta opção não é suportada.
`'_'`	Insere um sublinhado a cada 3 dígitos para tipos de apresentação inteiros `'d'` e tipos de apresentação de ponto flutuante, excluindo `'n'`. Para tipos de apresentação inteiros `'b'`, `'o'`, `'x'` e `'X'`, os sublinhados são inseridos a cada 4 dígitos. Para outros tipos de apresentação, esta opção não é suportada.

Para um separador com reconhecimento de localidade, usa o tipo de apresentação `'n'`.

Alterado na versão 3.1: Adicionada a opção `','` (veja também **PEP 378**).

Alterado na versão 3.6: Adicionada a opção `'_'` (veja também **PEP 515**).

Alterado na versão 3.14: Suporte a opção *grouping* para a parte fracionária.

Finalmente, o *type* determina como os dados devem ser apresentados.

Os tipos de apresentação de string disponíveis são:

Tipo	Significado
`'s'`	Formato de string. Este é o tipo padrão para strings e pode ser omitido.
None	O mesmo que `'s'`.

Os tipos de apresentação inteira disponíveis são:

Tipo	Significado
`'b'`	Formato binário. Exibe o número na base 2.
`'c'`	Caractere. Converte o inteiro no caractere Unicode correspondente antes de imprimir.
`'d'`	Inteiro decimal. Exibe o número na base 10.
`'o'`	Formato octal. Exibe o número na base 8.
`'x'`	Formato hexadecimal. Produz o número na base 16, usando letras minúsculas para os dígitos acima de 9.
`'X'`	Formato hexadecimal. Produz o número na base 16, usando letras maiúsculas para os dígitos acima de 9. No caso de `'#'` ser especificado, o prefixo `'0x'` será maiúsculo para `'0X'` também.
`'n'`	Número. É o mesmo que `'d'`, exceto que usa a configuração de localidade atual para inserir os separadores de grupo de dígitos apropriados.
None	O mesmo que `'d'`.

Além dos tipos de apresentação acima, os inteiros podem ser formatados com os tipos de apresentação de ponto flutuante listados abaixo (exceto `'n'` e `None`). Ao fazer isso, *`float()`* é usado para converter o inteiro em um número de ponto flutuante antes da formatação.

Os tipos de apresentação disponíveis para *`float`* e *`Decimal`* valores são:

Tipo	Significado
`'e'`	Notação científica. Para uma dada precisão `p`, formata o número em notação científica com a letra "e" separando o coeficiente do expoente. O coeficiente tem um dígito antes e `p` dígitos depois do ponto decimal, para um total de `p + 1` dígitos significativos. Sem precisão fornecida, usa uma precisão de `6` dígitos após o ponto decimal para *float*, e mostra todos os dígitos de coeficiente para *Decimal*. Se `p=0`, o ponto decimal é omitido a menos que a opção `#` seja usada.
`'E'`	Notação científica. O mesmo que `'e'`, exceto que usa um 'E' maiúsculo como caractere separador.
`'f'`	Notação de ponto fixo. Para uma dada precisão `p`, formata o número como um número decimal com exatamente os `p` dígitos após o ponto decimal. Sem precisão fornecida, usa uma precisão de `6` dígitos após o ponto decimal para *float*, e usa uma precisão grande o suficiente para mostrar todos os dígitos de coeficiente para *Decimal*. Se `p=0`, o ponto decimal é omitido a menos que a opção `#` seja usada.
`'F'`	Notação de ponto fixo. O mesmo que `'f'`, mas converte `nan` para `NAN` e `inf` para `INF`.
`'g'`	Formato geral. Para uma determinada precisão `p >= 1`, isso arredonda o número para `p` dígitos significativos e então formata o resultado em formato de ponto fixo ou em notação científica, dependendo de sua magnitude. Uma precisão de `0` é tratada como equivalente a uma precisão de `1`. As regras precisas são as seguintes: suponha que o resultado formatado com tipo de apresentação `'e'` e precisão `p-1` teria o expoente `exp`. Então, se `m <= exp < p`, onde `m` é -4 para pontos flutuantes e -6 para *Decimals*, o número é formatado com o tipo de apresentação `'f'` e precisão `p-1-exp`. Caso contrário, o número é formatado com tipo de apresentação `'e'` e precisão `p-1`. Em ambos os casos, zeros à direita insignificantes são removidos do significando, e o ponto decimal também é removido se não houver dígitos restantes após ele, a menos que a opção `'#'` seja usada. Sem precisão fornecida, usa uma precisão de `6` dígitos significativos para *float*. Para *Decimal*, o coeficiente do resultado é formado a partir dos dígitos do coeficiente do valor; a notação científica é usada para valores menores que `1e-6` em valor absoluto e valores onde o valor posicional do dígito menos significativo é maior que 1, e a notação de ponto fixo é usada de outra forma. Infinito positivo e negativo, zero positivo e negativo e nans, são formatados como `inf`, `-inf`, `0`, `-0` e `nan`, respectivamente, independentemente da precisão.
`'G'`	Formato geral. O mesmo que `'g'`, exceto muda para `'E'` se o número ficar muito grande. As representações de infinito e NaN também são maiúsculas.
`'n'`	Número. É o mesmo que `'g'`, exceto que usa a configuração de localidade atual para inserir os separadores de grupo de dígitos apropriados para a parte inteira de um número.
`'%'`	Porcentagem. Multiplica o número por 100 e exibe no formato fixo (`'f'`), seguido por um sinal de porcentagem.
None	Para *float*, é o como o tipo `'g'`, exceto que quando a notação de ponto fixo é usada para formatar o resultado, ela sempre inclui pelo menos um dígito após a ponto decimal, e troca para a notação científica quando `exp >= p - 1`. Quando a precisão não é especificada, este último será tão grande quanto necessário para representar o valor fornecido fielmente. Para *Decimal*, é o mesmo que `'g'` ou `'G'` dependendo do valor de `context.capitals` para o contexto decimal atual. O efeito geral é combinar a saída de *str()* conforme alterada pelos outros modificadores de formato.

O resultado deve ser arredondado corretamente para uma precisão dada `p` de dígitos após o ponto decimal. O modo de arredondamento para *float* corresponde ao do builtin *round()*. Para *Decimal*, o modo de arredondamento do *contexto* atual será usado.

Os tipos de apresentação disponíveis para *complex* são os mesmos que para *float* (`'%'` não é permitido). Os componentes real e imaginário de um número complexo são formatados como números de ponto flutuante, de acordo com o tipo de apresentação especificado. Eles são separados pelo sinal obrigatório da parte imaginária, sendo esta última terminada por um sufixo `j`. Se o tipo de apresentação estiver faltando, o resultado corresponderá à saída de *str()* (números complexos com uma parte real diferente de zero também são cercados por parênteses), possivelmente

alterados por outros modificadores de formato.

Exemplos de formato

Esta seção contém exemplos da sintaxe de *str.format()* e comparação com a antiga formatação `%`.

Na maioria dos casos a sintaxe é semelhante à antiga formatação de `%`, com a adição de `{}` e com `:` usado em vez de `%`. Por exemplo, `'%03.2f'` pode ser traduzido para `'{:03.2f}'`.

A nova sintaxe de formato também oferece suporte a opções novas e diferentes, mostradas nos exemplos a seguir.

Acessando os argumentos por posição:

```
>>> '{0}, {1}, {2}'.format('a', 'b', 'c')
'a, b, c'
>>> '{}, {}, {}'.format('a', 'b', 'c')  # 3.1+ only
'a, b, c'
>>> '{2}, {1}, {0}'.format('a', 'b', 'c')
'c, b, a'
>>> '{2}, {1}, {0}'.format(*'abc')      # desempacotando sequência de argumentos
'c, b, a'
>>> '{0}{1}{0}'.format('abra', 'cad')   # índices dos argumentos podem ser␣
↪repetidos
'abracadabra'
```

Acessando os argumentos por nome:

```
>>> 'Coordinates: {latitude}, {longitude}'.format(latitude='37.24N', longitude='-
↪115.81W')
'Coordinates: 37.24N, -115.81W'
>>> coord = {'latitude': '37.24N', 'longitude': '-115.81W'}
>>> 'Coordinates: {latitude}, {longitude}'.format(**coord)
'Coordinates: 37.24N, -115.81W'
```

Acessando os atributos dos argumentos:

```
>>> c = 3-5j
>>> ('The complex number {0} is formed from the real part {0.real} '
...  'and the imaginary part {0.imag}.').format(c)
'The complex number (3-5j) is formed from the real part 3.0 and the imaginary part␣
↪-5.0.'
>>> class Point:
...     def __init__(self, x, y):
...         self.x, self.y = x, y
...     def __str__(self):
...         return 'Point({self.x}, {self.y})'.format(self=self)
...
>>> str(Point(4, 2))
'Point(4, 2)'
```

Acessando os itens dos argumentos:

```
>>> coord = (3, 5)
>>> 'X: {0[0]};  Y: {0[1]}'.format(coord)
'X: 3;  Y: 5'
```

Substituindo `%s` e `%r`:

```
>>> "repr() mostra aspas: {!r}; str() não mostra: {!s}".format('teste1', 'teste2')
"repr() mostra aspas: 'teste1'; str() não mostra: teste2"
```

Alinhando o texto e especificando uma largura:

```
>>> '{:<30}'.format('alinhado à esquerda')
'alinhado à esquerda           '
>>> '{:>30}'.format('alinhado à direita')
'            alinhado à direita'
>>> '{:^30}'.format('centralizado')
'         centralizado         '
>>> '{:*^30}'.format('centralizado')  # usa '*' como preenchimento
'*********centralizado*********'
```

Substituindo `%+f`, `%-f` e `% f` e especificando um sinal:

```
>>> '{:+f}; {:+f}'.format(3.14, -3.14)  # mostra sempre
'+3.140000; -3.140000'
>>> '{: f}; {: f}'.format(3.14, -3.14)  # mostra um espaço para números positivos
' 3.140000; -3.140000'
>>> '{:-f}; {:-f}'.format(3.14, -3.14)  # mostra apenas o sinal de menos -- mesmo␣
↪que '{:f}; {:f}'
'3.140000; -3.140000'
```

Substituindo `%x` e `%o` e convertendo o valor para bases diferentes:

```
>>> # format também tem suporte a números binários
>>> "int: {0:d};  hex: {0:x};  oct: {0:o};  bin: {0:b}".format(42)
'int: 42;  hex: 2a;  oct: 52;  bin: 101010'
>>> # com 0x, 0o ou 0b como prefixo:
>>> "int: {0:d};  hex: {0:#x};  oct: {0:#o};  bin: {0:#b}".format(42)
'int: 42;  hex: 0x2a;  oct: 0o52;  bin: 0b101010'
```

Usando a vírgula ou o sublinhado como separador de grupos de dígitos:

```
>>> '{:,}'.format(1234567890)
'1,234,567,890'
>>> '{:_}'.format(1234567890)
'1_234_567_890'
>>> '{:_b}'.format(1234567890)
'100_1001_1001_0110_0000_0010_1101_0010'
>>> '{:_x}'.format(1234567890)
'4996_02d2'
>>> '{:_}'.format(123456789.123456789)
'123_456_789.12345679'
>>> '{:.,}'.format(123456789.123456789)
'123456789.123,456,79'
>>> '{:,._}'.format(123456789.123456789)
'123,456,789.123_456_79'
```

Expressando uma porcentagem:

```
>>> points = 19
>>> total = 22
>>> 'Correct answers: {:.2%}'.format(points/total)
'Correct answers: 86.36%'
```

Usando formatação específica do tipo:

```
>>> import datetime
>>> d = datetime.datetime(2010, 7, 4, 12, 15, 58)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> '{:%Y-%m-%d %H:%M:%S}'.format(d)
'2010-07-04 12:15:58'
```

Argumentos de aninhamento e exemplos mais complexos:

```
>>> for align, text in zip('<^>', ['left', 'center', 'right']):
...     '{0:{fill}{align}16}'.format(text, fill=align, align=align)
...
'left<<<<<<<<<<<<'
'^^^^^center^^^^^'
'>>>>>>>>>>>right'
>>>
>>> octets = [192, 168, 0, 1]
>>> '{:02X}{:02X}{:02X}{:02X}'.format(*octets)
'C0A80001'
>>> int(_, 16)
3232235521
>>>
>>> width = 5
>>> for num in range(5,12):
...     for base in 'dXob':
...         print('{0:{width}{base}}'.format(num, base=base, width=width), end=' ')
...     print()
...
    5     5     5   101
    6     6     6   110
    7     7     7   111
    8     8    10  1000
    9     9    11  1001
   10     A    12  1010
   11     B    13  1011
```

6.1.4 Template strings ($-strings)

Nota

The feature described here was introduced in Python 2.4; a simple templating method based upon regular expressions. It predates *`str.format()`*, formatted string literals, and *template string literals*.

It is unrelated to template string literals (t-strings), which were introduced in Python 3.14. These evaluate to *`string.templatelib.Template`* objects, found in the *`string.templatelib`* module.

Strings template fornecem substituições de string mais simples, conforme descrito em **PEP 292**. Um caso de uso primário para strings template é para internacionalização (i18n), uma vez que, nesse contexto, a sintaxe e a funcionalidade mais simples tornam mais fácil traduzir do que outros recursos embutidos de formatação de strings no Python. Como um exemplo de biblioteca construída sobre strings template para i18n, veja o pacote flufl.i18n.

Strings template oferecem suporte a substituições baseadas em `$`, usando as seguintes regras:

- `$$` é um escape; é substituído por um único `$`.
- `$identifier` nomeia um espaço reservado de substituição correspondendo a uma chave de mapeamento de `"identifier"`. Por padrão, `"identifier"` é restrito a qualquer string ASCII alfanumérica que não faz distinção entre maiúsculas e minúsculas (incluindo sublinhados) que começa com um sublinhado ou letra ASCII. O primeiro caractere não identificador após o caractere `$` termina esta especificação de espaço reservado.
- `${identifier}` é equivalente a `$identifier`. É necessário quando caracteres identificadores válidos seguem o marcador de posição, mas não fazem parte do marcador, como `"${noun}ification"`.

Qualquer outra ocorrência de `$` na string resultará em uma *`ValueError`* sendo levantada.

O módulo *`string`* fornece uma classe *`Template`* que implementa essas regras. Os métodos de *`Template`* são:

class `string.`**Template**(*template*)

O construtor recebe um único argumento que é a string template

substitute(*mapping={}*, */*, ***kwds*)

Executa a substituição do modelo, retornando uma nova string. *mapping* é qualquer objeto dicionário ou similar com chaves que correspondem aos marcadores de posição no modelo. Como alternativa, você pode fornecer argumentos nomeados, os quais são espaços reservados. Quando *mapping* e *kwds* são fornecidos e há duplicatas, os marcadores de *kwds* têm precedência.

safe_substitute(*mapping={}*, */*, ***kwds*)

Como *`substitute()`*, exceto que se os espaços reservados estiverem faltando em *mapping* e *kwds*, em vez de levantar uma exceção *`KeyError`*, o espaço reservado original aparecerá na string resultante intacta. Além disso, ao contrário de *`substitute()`*, qualquer outra ocorrência de `$` simplesmente retornará `$` em vez de levantar *`ValueError`*.

Embora outras exceções ainda possam ocorrer, esse método é chamado de "seguro" porque sempre tenta retornar uma string utilizável em vez de levantar uma exceção. Em outro sentido, *`safe_substitute()`* pode ser qualquer coisa diferente de seguro, uma vez que irá ignorar silenciosamente modelos malformados contendo delimitadores pendentes, chaves não correspondidas ou espaços reservados que não são identificadores Python válidos.

is_valid()

Retorna `False` se o modelo tiver espaços reservados inválidos que farão com que *`substitute()`* levante *`ValueError`*.

Adicionado na versão 3.11.

get_identifiers()

Retorna uma lista dos identificadores válidos no modelo, na ordem em que aparecem pela primeira vez, ignorando quaisquer identificadores inválidos.

Adicionado na versão 3.11.

Instâncias de *`Template`* também fornecem um atributo de dados públicos:

template

Este é o objeto passado para o argumento *template* do construtor. Em geral, você não deve alterá-lo, mas o acesso somente leitura não é obrigatório.

Aqui está um exemplo de como usar uma instância de Template:

```
>>> from string import Template
>>> s = Template('$who likes $what')
>>> s.substitute(who='tim', what='kung pao')
'tim likes kung pao'
>>> d = dict(who='tim')
>>> Template('Give $who $100').substitute(d)
Traceback (most recent call last):
...
ValueError: Invalid placeholder in string: line 1, col 11
>>> Template('$who likes $what').substitute(d)
Traceback (most recent call last):
...
KeyError: 'what'
>>> Template('$who likes $what').safe_substitute(d)
'tim likes $what'
```

Uso avançado: você pode derivar subclasses de *Template* para personalizar a sintaxe do espaço reservado, caractere delimitador ou toda a expressão regular usada para analisar strings template. Para fazer isso, você pode substituir estes atributos de classe:

- *delimiter* – Este é a string literal que descreve um delimitador de introdução do espaço reservado. O valor padrão é `$`. Note que esta *não* deve ser uma expressão regular, já que a implementação irá chamar *re.escape()* nesta string conforme necessário. Observe também que você não pode alterar o delimitador após a criação da classe (ou seja, um delimitador diferente deve ser definido no espaço de nomes da classe da subclasse).
- *idpattern* – Esta é a expressão regular que descreve o padrão para espaço reservado sem envolto em chaves. O valor padrão é a expressão regular `(?a:[_a-z][_a-z0-9]*)`. Se for fornecido e *braceidpattern* for `None`, esse padrão também se aplicará o espaço reservado com chaves.

  Nota

  Uma vez que *flags* padrão é `re.IGNORECASE`, o padrão `[a-z]` pode corresponder a alguns caracteres não ASCII. É por isso que usamos o sinalizador local `a` aqui.

  Alterado na versão 3.7: *braceidpattern* pode ser usado para definir padrões separados usados dentro e fora das chaves.
- *braceidpattern* – É como *idpattern*, mas descreve o padrão para espaços reservados com chaves. O padrão é `None`, o que significa recorrer a *idpattern* (ou seja, o mesmo padrão é usado dentro e fora das chaves). Se fornecido, permite definir padrões diferentes para espaço reservado com e sem chaves.

  Adicionado na versão 3.7.
- *flags* – Os sinalizadores de expressão regular que serão aplicados ao compilar a expressão regular usada para reconhecer substituições. O valor padrão é `re.IGNORECASE`. Note que `re.VERBOSE` sempre será adicionado aos sinalizadores, então *idpattern*s personalizados devem seguir as convenções para expressões regulares verbosas.

  Adicionado na versão 3.2.

Como alternativa, você pode fornecer todo o padrão de expressão regular substituindo o atributo *pattern* de classe. Se você fizer isso, o valor deve ser um objeto de expressão regular com quatro grupos de captura nomeados. Os grupos de captura correspondem às regras fornecidas acima, junto com a regra inválida do espaço reservado:

- *escaped* – Este grupo corresponde à sequência de escape, por exemplo `$$`, no padrão.
- *named* – Este grupo corresponde ao nome do espaço reservado sem chaves; não deve incluir o delimitador no grupo de captura.
- *braced* – Este grupo corresponde ao nome do espaço reservado entre chaves; ele não deve incluir o delimitador ou chaves no grupo de captura.
- *invalid* – Esse grupo corresponde a qualquer outro padrão de delimitador (geralmente um único delimitador) e deve aparecer por último na expressão regular.

Os métodos nesta classe irão levantar *ValueError* se o padrão corresponder ao modelo sem que um desses grupos nomeados corresponda.

6.1.5 Funções auxiliares

`string.`**`capwords`**(*s*, *sep=None*)

Divide o argumento em palavras usando *str.split()*, coloca cada palavra em maiúscula usando *str.capitalize()*, e junte as palavras em maiúsculas usando *str.join()*. Se o segundo argumento opcional *sep* estiver ausente ou `None`, os caracteres de espaço em branco são substituídos por um único espaço e os espaços em branco à esquerda e à direita são removidos, caso contrário *sep* é usado para dividir e unir as palavras.

6.2 `string.templatelib` — Suporte para literais de string template

Código-fonte: Lib/string/templatelib.py

Ver também

- Strings de formatação
- Sintaxe de literal de string template (t-string)
- **PEP 750**

6.2.1 Strings template

Adicionado na versão 3.14.

Strings template são um mecanismo para processamento personalizado de strings. Elas têm toda a flexibilidade das f-strings do Python, mas retornam uma instância *`Template`* que dá acesso às partes estáticas e interpoladas (entre chaves) de uma string *antes* de serem combinadas.

Para escrever uma t-string, use um prefixo `'t'` em vez de `'f'`, assim:

```
>>> pi = 3.14
>>> t't-strings são novas no Python {pi!s}!'
Template(
   strings=('t-strings são novas no Python ', '!'),
   interpolations=(Interpolation(3.14, 'pi', 's', ''),)
)
```

6.2.2 Tipos

class string.templatelib.**Template**

A classe `Template` descreve o conteúdo de uma string template. Ela é imutável, o que significa que os atributos de um template não podem ser reatribuídos.

A maneira mais comum de criar uma instância de `Template` é usar a sintaxe de literal de string template. Essa sintaxe é idêntica à de f-strings, exceto pelo fato de usar um prefixo `t` no lugar de um `f`:

```
>>> cheese = 'Red Leicester'
>>> template = t"We're fresh out of {cheese}, sir."
>>> type(template)
<class 'string.templatelib.Template'>
```

Os templates são armazenados como sequências de literais *`strings`* e dinâmicos *`interpolations`*. Um atributo *`values`* contém os valores das interpolações:

```
>>> cheese = 'Camembert'
>>> template = t'Ah! We do have {cheese}.'
>>> template.strings
('Ah! We do have ', '.')
>>> template.interpolations
(Interpolation('Camembert', ...),)
>>> template.values
('Camembert',)
```

A tupla `strings` tem um elemento a mais que `interpolations` e `values`; as interpolações “pertencem” às strings. Isso pode ser mais fácil de entender quando as tuplas estão alinhadas

```
template.strings:  ('Ah! We do have ',              '.')
template.values:   (                  'Camembert',    )
```

Atributos

strings: *tuple*[*str*, ...]

Uma *tuple* das strings estáticas no template.

```
>>> cheese = 'Camembert'
>>> template = t'Ah! We do have {cheese}.'
>>> template.strings
('Ah! We do have ', '.')
```

Strings vazias *estão* incluídas na tupla:

```
>>> response = 'We do have '
>>> cheese = 'Camembert'
>>> template = t'Ah! {response}{cheese}.'
>>> template.strings
('Ah! ', '', '.')
```

A tupla `strings` nunca está vazia e sempre contém uma string a mais que as tuplas `interpolations` e `values`:

```
>>> t''.strings
('',)
>>> t''.values
()
>>> t'{'cheese'}'.strings
('', '')
>>> t'{'cheese'}'.values
('cheese',)
```

interpolations: *tuple*[*Interpolation*, ...]

Uma *tuple* das interpolações no template.

```
>>> cheese = 'Camembert'
>>> template = t'Ah! We do have {cheese}.'
>>> template.interpolations
(Interpolation('Camembert', 'cheese', None, ''),)
```

A tupla `interpolações` pode estar vazia e sempre contém um valor a menos que a tupla `strings`:

```
>>> t'Red Leicester'.interpolations
()
```

values: *tuple*[*object*, ...]

Uma tupla de todos os valores interpolados no template.

```
>>> cheese = 'Camembert'
>>> template = t'Ah! We do have {cheese}.'
>>> template.values
('Camembert',)
```

A tupla `values` sempre tem o mesmo comprimento que a tupla `interpolations`. Isso sempre equivale a `tuple(i.value for i in template.interpolations)`.

Métodos

__new__(**args:* str | Interpolation)

Embora a sintaxe de literal seja a maneira mais comum de criar um `Template`, também é possível criá-las diretamente usando o construtor:

```
>>> from string.templatelib import Interpolation, Template
>>> cheese = 'Camembert'
>>> template = Template(
...     'Ah! We do have ', Interpolation(cheese, 'cheese'), '.'
... )
>>> list(template)
['Ah! We do have ', Interpolation('Camembert', 'cheese', None, ''), '.']
```

Se várias strings forem passadas consecutivamente, elas serão concatenadas em um único valor no atributo *`strings`*. Por exemplo, o código a seguir cria uma *`Template`* com uma única string final:

```
>>> from string.templatelib import Template
>>> template = Template('Ah! We do have ', 'Camembert', '.')
>>> template.strings
('Ah! We do have Camembert.',)
```

Se várias interpolações forem passadas consecutivamente, elas serão tratadas como interpolações separadas e uma string vazia será inserida entre elas. Por exemplo, o código a seguir cria um modelo com espaços reservados vazios no atributo *`strings`*:

```
>>> from string.templatelib import Interpolation, Template
>>> template = Template(
...     Interpolation('Camembert', 'cheese'),
...     Interpolation('.', 'punctuation'),
... )
>>> template.strings
('', '', '')
```

iter(template)

Itera sobre o template, produzindo cada string não-vazia e *`Interpolation`* em ordem correta.

```
>>> cheese = 'Camembert'
>>> list(t'Ah! We do have {cheese}.')
['Ah! We do have ', Interpolation('Camembert', 'cheese', None, ''), '.']
```

Cuidado

Strings vazias *não* são incluídas na iteração:

```
>>> response = 'We do have '
>>> cheese = 'Camembert'
>>> list(t'Ah! {response}{cheese}.')
['Ah! ',
 Interpolation('We do have ', 'response', None, ''),
 Interpolation('Camembert', 'cheese', None, ''),
 '.']
```

template + other

template += other

Concatena este template com outro, retornando uma nova instância `Template`:

```
>>> cheese = 'Camembert'
>>> list(t'Ah! ' + t'We do have {cheese}.')
['Ah! We do have ', Interpolation('Camembert', 'cheese', None, ''), '.']
```

A concatenação de uma `Template` e uma `str` **não** é suportada. Isso ocorre porque não está claro se a string deve ser tratada como uma string estática ou uma interpolação. Se você quiser concatenar uma `Template` com uma string, você deve encapsular a string diretamente em uma `Template` (para tratá-la como uma string estática) ou usar uma `Interpolation` (para tratá-la como dinâmica):

```
>>> from string.templatelib import Interpolation, Template
>>> template = t'Ah! '
>>> # Treat 'We do have ' as a static string
>>> template += Template('We do have ')
>>> # Treat cheese as an interpolation
>>> cheese = 'Camembert'
>>> template += Template(Interpolation(cheese, 'cheese'))
>>> list(template)
['Ah! We do have ', Interpolation('Camembert', 'cheese', None, '')]
```

class string.templatelib.**Interpolation**

O tipo `Interpolation` representa uma expressão dentro de uma string template. É imutável, o que significa que os atributos de uma interpolação não podem ser reatribuídos.

As interpolações oferecem suporte à correspondência de padrões, permitindo que você compare seus atributos com a instrução match:

```
>>> from string.templatelib import Interpolation
>>> interpolation = t'{1. + 2.:.2f}'.interpolations[0]
>>> interpolation
Interpolation(3.0, '1. + 2.', None, '.2f')
>>> match interpolation:
...     case Interpolation(value, expression, conversion, format_spec):
...         print(value, expression, conversion, format_spec, sep=' | ')
...
3.0 | 1. + 2. | None | .2f
```

Atributos

value: *object*

O valor avaliado da interpolação.

```
>>> t'{1 + 2}'.interpolations[0].value
3
```

expression: *str*

O texto de uma expressão Python válida ou uma string vazia.

expression é o texto original da expressão Python da interpolação, se a interpolação tiver sido criada a partir de um literal de t-string. Desenvolvedores que criam interpolações manualmente devem definir isso como uma string vazia ou escolher uma expressão Python válida e adequada.

```
>>> t'{1 + 2}'.interpolations[0].expression
'1 + 2'
```

conversion: *Literal*['a', 'r', 's'] | *None*

A conversão a ser aplicada ao valor, ou `None`.

O `conversion` é a conversão opcional a ser aplicada ao valor:

```
>>> t'{1 + 2!a}'.interpolations[0].conversion
'a'
```

Nota

Ao contrário das f-strings, onde as conversões são aplicadas automaticamente, o comportamento esperado com t-strings é que o código que *processa* o `Template` decidirá como interpretar e se aplicará o `conversion`. Por conveniência, a função *`convert()`* pode ser usada para imitar a semântica de conversão de f-strings.

`format_spec`: ***str***

A especificação de formato a ser aplicada ao valor.

O `format_spec` é uma string arbitrária opcional usada como especificação de formato para apresentar o valor:

```
>>> t'{1 + 2:.2f}'.interpolations[0].format_spec
'.2f'
```

Nota

Ao contrário das f-strings, onde as especificações de formato são aplicadas automaticamente por meio do protocolo *`format()`*, o comportamento esperado com t-strings é que o código que *processa* a interpolação decida como interpretar e se a especificação de formato deve ser aplicada. Como resultado, os valores `format_spec` em interpolações podem ser strings arbitrárias, incluindo aquelas que não estão em conformidade com o protocolo de *`format()`*.

Métodos

`__new__`(*value:* object, *expression:* str, *conversion:* Literal*['a', 'r', 's']* | None = *None*, *format_spec:* str = *''*)

Cria um novo objeto `Interpolation` a partir de partes do componente.

Parâmetros

- **`value`** – O resultado avaliado e dentro do escopo da interpolação.
- **`expression`** – O texto de uma expressão Python válida ou uma string vazia.
- **`conversion`** – A *conversão* a ser usada, uma entre `None`, `'a'`, `'r'` ou `'s'`.
- **`format_spec`** – Uma string arbitrária opcional usada como *especificação de formato* para apresentar o valor.

6.2.3 Funções auxiliares

`string.templatelib.`**`convert`**(*obj*, */*, *conversion*)

Aplica a semântica de *conversão* de literal de string formatada ao objeto *obj* fornecido. Isso é frequentemente útil para lógica de processamento de string template personalizada.

Atualmente, há três sinalizadores de conversão suportados:

- `'s'` que chama *`str()`* no valor (como `!s`),
- `'r'` que chama *`repr()`* (como `!r`) e
- `'a'` que chama *`ascii()`* (como `!a`).

Se o sinalizador de conversão é `None`, *obj* é retornado sem alterações.

6.3 `re` — Operações com expressões regulares

Código-fonte: Lib/re/

Este módulo fornece operações para correspondência de expressões regulares semelhantes às encontradas em Perl. O nome do módulo vem das iniciais do termo em inglês *regular expressions*, também frequentemente chamadas de *regex*.

Tanto os padrões quanto as strings a serem pesquisados podem ser strings Unicode (`str`) assim como strings de 8 bits (`bytes`). No entanto, strings Unicode e strings de 8 bits não podem ser misturadas: ou seja, você não pode corresponder uma string Unicode com um padrão de bytes ou vice-versa; da mesma forma, ao solicitar uma substituição, a string de substituição deve ser do mesmo tipo que o padrão e a string de pesquisa.

Expressões regulares usam o caractere de contrabarra (`'\'`) para indicar formas especiais ou para permitir que caracteres especiais sejam usados sem invocar seu significado especial. Isso colide com o uso em Python do mesmo caractere para o mesmo propósito em literais de string; por exemplo, para corresponder a uma contrabarra literal, pode-se ter que escrever `'\\\\'` como a string do padrão, porque a expressão regular deve ser `\\`, e cada contrabarra deve ser expressa como `\\` dentro de um literal de string Python regular. Além disso, observe que quaisquer sequências de escape inválidas no uso do Python da contrabarra em literais de string agora levantam uma exceção `SyntaxWarning` e no futuro isso se tornará um `SyntaxError`. Esse comportamento acontecerá mesmo se for uma sequência de escape válida para uma expressão regular.

A solução é usar a notação de string bruta do Python para padrões de expressão regular; as contrabarras não são tratadas de nenhuma maneira especial em uma string literal com o prefixo `'r'`. Portanto, `r"\n"` é uma string de dois caracteres contendo `'\'` e `'n'`, enquanto `"\n"` é uma string de um caractere contendo uma nova linha. Normalmente, os padrões serão expressos em código Python usando esta notação de string bruta.

É importante notar que a maioria das operações de expressão regular estão disponíveis como funções e métodos em nível de módulo em *expressões regulares compiladas*. As funções são atalhos que não exigem que você compile um objeto de expressão regular primeiro, mas perdem-se alguns parâmetros de ajuste fino.

Ver também

O módulo de terceiros regex possui uma API compatível com o módulo da biblioteca padrão `re`, mas oferece funcionalidades adicionais e um suporte mais completo a Unicode.

6.3.1 Sintaxe de expressão regular

Uma expressão regular (ou ER) especifica um conjunto de strings que corresponde a ela; as funções neste módulo permitem que você verifique se uma determinada string corresponde a uma determinada expressão regular (ou se uma determinada expressão regular corresponde a uma determinada string, o que resulta na mesma coisa).

As expressões regulares podem ser concatenadas para formar novas expressões regulares; se *A* e *B* forem expressões regulares, então *AB* também será uma expressão regular. Em geral, se uma string *p* corresponder a *A* e outra string *q* corresponder a *B*, a string *pq* corresponderá a AB. Isso é válido, a menos que *A* ou *B* contenham operações de baixa precedência; condições de contorno entre *A* e *B*; ou ter referências de grupo numeradas. Assim, expressões complexas podem ser facilmente construídas a partir de expressões primitivas mais simples, como as descritas aqui. Para obter detalhes sobre a teoria e implementação de expressões regulares, consulte o livro de Friedl [Frie09], ou quase qualquer livro sobre construção de compiladores.

Segue uma breve explicação do formato das expressões regulares. Para mais informações e uma apresentação mais suave, consulte o tutorial de regex-howto.

As expressões regulares podem conter caracteres especiais e comuns. A maioria dos caracteres comuns, como `'A'`, `'a'` ou `'0'`, são as expressões regulares mais simples; eles simplesmente se correspondem. Você pode concatenar caracteres comuns, de forma que `último` corresponda à string `'último'`. (No restante desta seção, escreveremos ERs `neste estilo especial`, geralmente sem aspas, e strings para serem correspondidas `'entre aspas simples'`.)

Alguns caracteres, como `'|'` ou `'('`, são especiais. Os caracteres especiais representam classes de caracteres comuns ou afetam como as expressões regulares em torno deles são interpretadas.

Operadores de repetição ou quantificadores (`*`, `+`, `?`, `{m,n}` etc) não podem ser aninhados diretamente. Isso evita ambiguidade com o sufixo modificador não guloso `?`, e com outros modificadores em outras implementações. Para aplicar uma segunda repetição a uma repetição interna, podem ser usados parênteses. Por exemplo, a expressão `(?:a{6})*` corresponde a qualquer múltiplo de seis caracteres `'a'`.

Os caracteres especiais são:

.

(Ponto.) No modo padrão, corresponde a qualquer caractere, exceto uma nova linha. Se o sinalizador *DOTALL* foi especificado, ele corresponde a qualquer caractere, incluindo uma nova linha. `(?s:.)` corresponde a qualquer caractere independente de sinalizadores.

^

(Sinal de circunflexo.) Corresponde ao início da string, e no modo *MULTILINE* também corresponde imediatamente após cada nova linha.

$

Corresponde ao final da string ou logo antes da nova linha no final da string, e no modo *MULTILINE* também corresponde antes de uma nova linha. `foo` corresponde a 'foo' e 'foobar', enquanto a expressão regular `foo$` corresponde apenas a 'foo'. Mais interessante, pesquisar por `foo.$` em `'foo1\nfoo2\n'` corresponde a 'foo2' normalmente, mas 'foo1' no modo *MULTILINE*; procurando por um único `$` em `'foo\n'` encontrará duas correspondências (vazias): uma logo antes da nova linha e uma no final da string.

*****

Faz a ER resultante corresponder a 0 ou mais repetições da ER anterior, tantas repetições quantas forem possíveis. `ab*` corresponderá a 'a', 'ab' ou 'a' seguido por uma quantidade qualquer de 'b's.

+

Faz com que a ER resultante corresponda a 1 ou mais repetições da ER anterior. `ab+` irá corresponder a 'a' seguido por qualquer número diferente de zero de 'b's; não corresponderá apenas a 'a'.

?

Faz a ER resultante corresponder a 0 ou 1 repetição da ER anterior. `ab?` irá corresponder a 'a' ou 'ab'.

***?, +?, ??**

Os quantificadores `'*'`, `'+'` e `'?'` são todos *gulosos*; eles correspondem ao máximo de texto possível. Às vezes, esse comportamento não é desejado; se a ER `<.*>` for correspondida com `'<a> b <c>'`, ela irá corresponder a toda a string, e não apenas `'<a>'`. Adicionar `?` após o quantificador faz com que ele execute a correspondência da maneira *não gulosa* ou *minimalista*; tão *poucos* caracteres quanto possível serão correspondidos. Usando a ER `<.*?>` irá corresponder apenas `'<a>'`.

***+, ++, ?+**

Como os quantificadores `'*'`, `'+'` e `'?'`, aqueles em que `'+'` é anexado também correspondem tantas vezes quanto possível. No entanto, ao contrário dos quantificadores realmente gulosos, eles não permitem retrocesso quando a expressão que o segue não corresponde. Estes são conhecidos como quantificadores *possessivos*. Por exemplo, `a*a` corresponderá a `'aaaa'` porque o `a*` corresponderá a todos os 4 `'a'`s, mas, quando o final `'a'` for encontrado, a expressão é retrocedida para que no final o `a*` acabe correspondendo a 3 `'a'`s no total, e o quarto `'a'` seja correspondido por o final `'a'`. No entanto, quando `a*+a` é usado para corresponder a `'aaaa'`, o `a*+` corresponderá a todos os 4 `'a'`, mas quando o `'a'` final não encontrar mais caracteres para corresponder, a expressão não pode ser retrocedida e, portanto, não corresponderá. `x*+`, `x++` e `x?+` são equivalentes a `(?>x*)`, `(?>x+)` e `(?>x?)` respectivamente.

Adicionado na versão 3.11.

{m}

Especifica que exatamente *m* cópias da ER anterior devem ser correspondidas; menos correspondências fazem com que toda a ER não seja correspondida. Por exemplo, `a{6}` irá corresponder exatamente a seis caracteres `'a'`, mas não a cinco.

{m,n}

Faz a ER resultante corresponder de *m* a *n* repetições da ER anterior, tentando corresponder ao máximo de

repetições possível. Por exemplo, `a{3,5}` irá corresponder de 3 a 5 caracteres `'a'`. A omissão de *m* especifica um limite inferior de zero e a omissão de *n* especifica um limite superior infinito. Como exemplo, `a{4,}b` irá corresponder a `'aaaab'` ou mil caracteres `'a'` seguidos por um `'b'`, mas não `'aaab'`. A vírgula não pode ser omitida ou o modificador será confundido com a forma descrita anteriormente.

`{m,n}?`
Faz a ER resultante corresponder de *m* a *n* repetições da ER anterior, tentando corresponder o *mínimo* de repetições possível. Esta é a versão não gulosa do quantificador anterior. Por exemplo, na string de 6 caracteres `'aaaaaa'`, `a{3,5}` irá corresponder a 5 caracteres `'a'`, enquanto `a{3,5}?` corresponderá apenas a 3 caracteres.

`{m,n}+`
Faz a ER resultante corresponder de *m* a *n* repetições da ER anterior, tentando corresponder o maior número possível de repetições *sem* estabelecer nenhum ponto de retrocesso. Esta é a versão possessiva do quantificador acima. Por exemplo, na string de 6 caracteres `'aaaaaa'`, `a{3,5}+aa` tenta corresponder 5 caracteres `'a'`, então, requerer mais 2 `'a'`s, precisará de mais caracteres do que os disponíveis e, portanto, falhará, enquanto `a{3,5}aa` corresponderá a `a{3,5}` capturando 5, depois 4 `'a'`s por retrocesso e então os 2 `'a'`s finais são combinados com o `aa` final no padrão. `x{m,n}+` é equivalente a `(?>x{m,n})`.

Adicionado na versão 3.11.

`\`
Ou anula o efeito de caracteres especiais (permitindo a você corresponder caracteres como `'*'`, `'?'` e assim por diante), ou sinaliza uma sequência especial; sequências especiais são discutidas abaixo.

Se você não estiver usando uma string bruta para expressar o padrão, lembre-se de que o Python também usa a contrabarra como uma sequência de escape em literais de string; se a sequência de escape não for reconhecida pelo analisador sintático do Python, a contrabarra e o caractere subsequente serão incluídos na string resultante. No entanto, se Python reconhecer a sequência resultante, a contrabarra deve ser repetida duas vezes. Isso é complicado e difícil de entender, portanto, é altamente recomendável que você use strings brutas para todas as expressões, exceto as mais simples.

`[]`
Usado para indicar um conjunto de caracteres. Em um conjunto:

- Caracteres podem ser listados individualmente, por exemplo, `[amk]` vai corresponder a `'a'`, `'m'` ou `'k'`.
- Intervalos de caracteres podem ser indicados fornecendo dois caracteres e separando-os por `'-'`, por exemplo `[a-z]` irá corresponder a qualquer letra ASCII minúscula, `[0-5][0-9]` irá corresponder a todos os números de dois dígitos de `00` a `59`, e `[0-9A-Fa-f]` irá corresponder a qualquer dígito hexadecimal. Se `-` for precedido de uma contrabarra (por exemplo, `[a\-z]`) ou se for colocado como o primeiro ou último caractere (por exemplo, `[-a]` ou `[a-]`), ele corresponderá a um literal `'-'`.
- Os caracteres especiais, exceto contrabarra, perdem seu significado especial dentro dos conjuntos. Por exemplo, `[(+*)]` vai corresponder a qualquer um dos caracteres literais `'('`, `'+'`, `'*'` ou `')'`.
- A contrabarra escapa caracteres que têm significado especial em um conjunto, como `'-'`, `']'`, `'^'` e `'\\'`, ou sinaliza uma sequência especial que representa um único caractere, como `\xa0` ou `\n`, ou uma classe de caracteres como `\w` ou `\S` (definida abaixo). Observe que `\b` representa um único caractere de "backspace", não um limite de palavra como fora de um conjunto, e escapes numéricos como `\1` são sempre escapes octais, não referências de grupo. Sequências especiais que não correspondem a um único caractere, como `\A` e `\z`, não são permitidas.
- Os caracteres que não estão dentro de um intervalo podem ser correspondidos *complementando* o conjunto. Se o primeiro caractere do conjunto for `'^'`, todos os caracteres que *não* estiverem no conjunto serão correspondidos. Por exemplo, `[^5]` irá corresponder a qualquer caractere exceto `'5'`, e `[^^]` irá corresponder a qualquer caractere exceto `'^'`. `^` não tem nenhum significado especial se não for o primeiro caractere do conjunto.
- Para corresponder a um `']'` literal dentro de um conjunto, preceda-o com uma contrabarra ou coloque-o no início do conjunto. Por exemplo, `[()[\]{}]` e `[]()[{}]` vai corresponder ao colchete à direita, assim como o colchete, chave ou parêntese à esquerda.

- Suporte para conjuntos aninhados e operações de conjunto como no Padrão Técnico do Unicode #18 podem ser adicionados no futuro. Isso mudaria a sintaxe, então para facilitar essa mudança uma *FutureWarning* será levantada em casos ambíguos por enquanto. Isso inclui conjuntos que começam com um `'['` literal ou contendo sequências de caracteres literais `'--'`, `'&&'`, `'~~'` e `'||'`. Para evitar um aviso, use uma contrabarra antes deles.

Alterado na versão 3.7: *FutureWarning* é levantada se um conjunto de caracteres contém construções que mudarão semanticamente no futuro.

|

`A|B`, onde *A* e *B* podem ser ERs arbitrárias, cria uma expressão regular que corresponderá a *A* ou *B*. Um número arbitrário de ERs pode ser separado por `'|'` desta forma. Isso também pode ser usado dentro de grupos (veja abaixo). Conforme a string alvo é percorrida, ERs separadas por `'|'` são avaliadas da esquerda para a direita. Quando um padrão corresponde completamente, essa ramificação é aceita. Isso significa que, assim que *A* corresponder, *B* não será avaliado posteriormente, mesmo que produza uma correspondência geral mais longa. Em outras palavras, o operador `'|'` nunca é guloso. Para corresponder a um `'|'` literal, use `\|`, ou coloque-o dentro de uma classe de caractere, como em `[|]`.

(...)

Corresponde a qualquer expressão regular que esteja entre parênteses e indica o início e o fim de um grupo; o conteúdo de um grupo pode ser recuperado após uma correspondência ter sido realizada e pode ser correspondido posteriormente na string com a sequência especial `\número`, descrita abaixo. Para corresponder aos literais `'('` ou `')'`, use `\(` ou `\)`, ou coloque-os dentro de uma classe de caracteres: `[(]`, `[)]`.

(?...)

Esta é uma notação de extensão (um `'?'` após um `'('` não é significativo de outra forma). O primeiro caractere após o `'?'` determina qual o significado e sintaxe posterior da construção. As extensões normalmente não criam um novo grupo; `(?P<nome>...)` é a única exceção a esta regra. A seguir estão as extensões atualmente implementadas.

(?aiLmsux)

(Uma ou mais letras do conjunto `'a'`, `'i'`, `'L'`, `'m'`, `'s'`, `'u'`, `'x'`.) O grupo corresponde à string vazia; as letras definem os sinalizadores correspondentes para toda a expressão regular:

- *re.A* (correspondência somente ASCII)
- *re.I* (ignorar maiúsculas/minúsculas)
- *re.L* (dependente da localidade)
- *re.M* (multi-linha)
- *re.S* (ponto corresponde a tudo)
- *re.U* (correspondência Unicode)
- *re.X* (verboso)

(Os sinalizadores são descritos em *Conteúdo do módulo*.) Isso é útil se você deseja incluir os sinalizadores como parte da expressão regular, em vez de passar um argumento *flag* para a função *re.compile()*. Os sinalizadores devem ser usados primeiro na string de expressão.

Alterado na versão 3.11: Esta construção só pode ser usada no início da expressão.

(?:...)

Uma versão de não-captura de parênteses regulares. Corresponde a qualquer expressão regular que esteja entre parênteses, mas a substring correspondida pelo grupo *não pode* ser recuperada após realizar uma correspondência ou referenciada posteriormente no padrão.

(?aiLmsux-imsx:...)

(Zero ou mais letras do conjunto `'a'`, `'i'`, `'L'`, `'m'`, `'s'`, `'u'`, `'x'`, opcionalmente seguidas por `'-'` seguido por uma ou mais letras do conjunto `'i'`, `'m'`, `'s'`, `'x'`.) O conjunto de letras define ou remove os sinalizadores correspondentes para a parte da expressão:

- *re.A* (correspondência somente ASCII)
- *re.I* (ignorar maiúsculas/minúsculas)

- *re.L* (dependente da localidade)
- *re.M* (multi-linha)
- *re.S* (ponto corresponde a tudo)
- *re.U* (correspondência Unicode)
- *re.X* (verboso)

(Os sinalizadores são descritos em *Conteúdo do módulo*.)

As letras `'a'`, `'L'` e `'u'` são mutuamente exclusivas quando usadas como sinalizadores em linha, portanto, não podem ser combinadas ou seguir `'-'`. Em vez disso, quando um deles aparece em um grupo embutido, ele substitui o modo de correspondência no grupo anexo. Em padrões Unicode `(?a:...)` muda para correspondência somente ASCII, e `(?u:...)` muda para correspondência Unicode (padrão). Em padrões de bytes `(?L:...)` muda para a correspondência dependente da localidade, e `(?a:...)` muda para correspondência apenas ASCII (padrão). Esta substituição só tem efeito para o grupo estreito em linha e o modo de correspondência original é restaurado fora do grupo.

Adicionado na versão 3.6.

Alterado na versão 3.7: As letras `'a'`, `'L'` e `'u'` também podem ser usadas em um grupo.

(?>...)
Tenta corresponder `...` como se fosse uma expressão regular separada e, se for bem-sucedida, continua correspondendo ao restante do padrão que a segue. Se o padrão subsequente não corresponder, a pilha só pode ser desenrolada até um ponto *antes* do `(?>...)` porque, uma vez encerrada, a expressão, conhecida como *grupo atômico*, jogou fora todos os pontos de pilha dentro de si. Assim, `(?>.*).` nunca corresponderia a nada porque primeiro o `.*` corresponderia a todos os caracteres possíveis, então, não tendo mais nada para corresponder, o `.` final falharia em corresponder. Como não há pontos de pilha salvos no Grupo Atômico e não há ponto de pilha antes dele, a expressão inteira não corresponderia.

Adicionado na versão 3.11.

(?P<nome>...)
Semelhante aos parênteses regulares, mas a substring correspondida pelo grupo é acessível por meio do nome de grupo simbólico *nome*. Os nomes de grupo devem ser identificadores Python válidos e em padrões *bytes* eles só podem conter bytes no intervalo ASCII. Cada nome de grupo deve ser definido apenas uma vez em uma expressão regular. Um grupo simbólico também é um grupo numerado, como se o grupo não tivesse um nome.

Grupos nomeados podem ser referenciados em três contextos. Se o padrão for `(?P<citação>['"]).*?(?P=citação)` (ou seja, corresponder a uma string entre aspas simples ou duplas):

Contexto de referência ao grupo "citação"	Formas de referenciá-lo
no mesmo padrão	• `(?P=citação)` (como mostrado) • `\1`
ao processar a correspondência do objeto *m*	• `m.group('citação')` • `m.end('citação')` (etc.)
em uma string passada para o argumento *repl* de `re.sub()`	• `\g<citação>` • `\g<1>` • `\1`

Alterado na versão 3.12: Nos padrões *bytes*, o grupo *nome* só pode conter bytes no intervalo ASCII (`b'\x00'-b'\x7f'`).

(?P=nome)
Uma referência anterior a um grupo nomeado; corresponde a qualquer texto que corresponda ao grupo anterior denominado *nome*.

(?#...)
Um comentário; o conteúdo dos parênteses é simplesmente ignorado.

(?=...)
Corresponde se `...` corresponder a próxima, mas não consome nada da string. Isso é chamado de *asserção preditiva*. Por exemplo, `Isaac (?=Asimov)` corresponderá a `'Isaac '` apenas se for seguido por `'Asimov'`.

(?!...)
Corresponde se `...` não corresponder a próxima. Isso é uma *asserção preditiva negativa*. Por exemplo, `Isaac (?!Asimov)` corresponderá a `'Isaac '` apenas se *não* for seguido por `'Asimov'`.

(?<=...)
Corresponde se a posição atual na string for precedida por uma correspondência para `...` que termina na posição atual. Isso é chamado de *asserção retroativa positiva*. `(?<=abc)def` irá encontrar uma correspondência em `'abcdef'`, uma vez que a expressão regular vai voltar 3 caracteres e verificar se o padrão contido corresponde. O padrão contido deve corresponder apenas a strings de algum comprimento fixo, o que significa que `abc` ou `a|b` são permitidos, mas `a*` e `a{3,4}` não são. Observe que os padrões que começam com asserções retroativas positivas não corresponderão ao início da string que está sendo pesquisada; você provavelmente desejará usar a função *search()* em vez da função *match()*:

```
>>> import re
>>> m = re.search('(?<=abc)def', 'abcdef')
>>> m.group(0)
'def'
```

Este exemplo procura por uma palavra logo após um hífen:

```
>>> m = re.search(r'(?<=-)\w+', 'spam-egg')
>>> m.group(0)
'egg'
```

Alterado na versão 3.5: Adicionado suporte para referências de grupo de comprimento fixo.

(?<!...)
Corresponde se a posição atual na string não for precedida por uma correspondência para `...`. Isso é chamado de *asserção retroativa negativa*. Semelhante às asserções retroativas positivas, o padrão contido deve corresponder apenas a strings de algum comprimento fixo. Os padrões que começam com asserções retroativas negativas podem corresponder ao início da string que está sendo pesquisada.

(?(id/nome)padrão-sim|padrão-não)
Tentará corresponder com `padrão-sim` se o grupo com determinado *id* ou *nome* existir, e com `padrão-não` se não existir. `padrão-não` é opcional e pode ser omitido. Por exemplo, `(<)?(\w+@\w+(?:\.\w+)+)(?(1)>|$)` é um padrão ruim de correspondência de e-mail, que corresponderá com `'<usuario@host.com>'` bem como `'usuario@host.com'`, mas não com `'<usuario@host.com>'` nem `'usuario@host.com>'`.

Alterado na versão 3.12: O grupo *id* só podem conter dígitos ASCII. Nos padrões *bytes*, o grupo *nome* só pode conter bytes no intervalo ASCII (`b'\x00'-b'\x7f'`).

As sequências especiais consistem em `'\'` e um caractere da lista abaixo. Se o caractere comum não for um dígito ASCII ou uma letra ASCII, a ER resultante corresponderá ao segundo caractere. Por exemplo, `\$` corresponde ao caractere `'$'`.

\número
Corresponde ao conteúdo do grupo de mesmo número. Os grupos são numerados a partir de 1. Por exemplo, `(.+) \1` corresponde a `'de de'` ou `'55 55'`, mas não `'dede'` (note o espaço após o grupo). Esta sequência especial só pode ser usada para corresponder a um dos primeiros 99 grupos. Se o primeiro dígito de *número* for 0, ou *número* tiver 3 dígitos octais de comprimento, ele não será interpretado como uma correspondência

de grupo, mas como o caractere com *número* de valor octal. Dentro de `'['` e `']'` de uma classe de caracteres, todos os escapes numéricos são tratados como caracteres.

\A

Corresponde apenas ao início da string.

\b

Corresponde à string vazia, mas apenas no início ou no final de uma palavra. Uma palavra é definida como uma sequência de caracteres de palavras. Observe que, formalmente, `\b` é definido como a fronteira entre um caractere `\w` e um `\W` (ou vice-versa), ou entre `\w` e o início/fim da string. Isso significa que `r'\bat\b'` corresponde a `'at'`, `'at.'`, `'(at)'`, `'as at ay'`, mas não a `'attempt'` ou `'atlas'`.

Os caracteres de palavras predefinidos em padrões Unicode (str) são alfanuméricos Unicode e o sublinhado, mas isso pode ser alterado usando o sinalizador *ASCII*. Os limites das palavras são determinados pela localidade atual se o sinalizador *LOCALE* for usado.

> **Nota**
>
> Dentro de um intervalo de caracteres, `\b` representa o caractere backspace, para compatibilidade com strings literais do Python.

\B

Corresponde à string vazia, mas apenas quando *não* estiver no início ou no final de uma palavra. Isso significa que `r'at\B'` corresponde a `'athens'`, `'atom'`, `'attorney'`, mas não `'at'`, `'at.'` ou `'at!'`. `\B` é exatamente o oposto de `\b`, então caracteres de palavras em padrões Unicode são alfanuméricos Unicode ou o sublinhado, embora isso possa ser alterado usando o sinalizador *ASCII*. Os limites das palavras são determinados pela localidade atual se o sinalizador *LOCALE* for usado.

Alterado na versão 3.14: `\B` agora corresponde string de entrada vazia.

\d

Para padrões Unicode (str):
Corresponde a qualquer dígito decimal Unicode (ou seja, qualquer caractere na categoria de caractere Unicode [Nd]). Isso inclui `[0-9]`, e também muitos outros caracteres de dígitos.

Se o sinalizador *ASCII* for usado, apenas `[0-9]` será correspondido.

Para padrões de 8 bits (isto é, bytes):
Corresponde a qualquer dígito decimal no conjunto de caracteres ASCII; isso é equivalente a `[0-9]`.

\D

Corresponde a qualquer caractere que não seja um dígito decimal. Isso é o oposto de `\d`.

Se o sinalizador *ASCII* for usado, apenas `[^0-9]` será correspondido.

\s

Para padrões Unicode (str):
Corresponde a caracteres de espaço em branco Unicode (como definido por *str.isspace()*) . Isso inclui `[ \t\n\r\f\v]`, e também muitos outros caracteres, como, por exemplo, os espaços não separáveis exigidos pelas regras de tipografia em muitos idiomas.

Se o sinalizador *ASCII* for usado, apenas `[ \t\n\r\f\v]` é correspondido.

Para padrões de 8 bits (isto é, bytes):
Corresponde a caracteres considerados espaços em branco no conjunto de caracteres ASCII; isso é equivalente a `[ \t\n\r\f\v]`.

\S

Corresponde a qualquer caractere que não seja um espaço em branco. Isso é o oposto de `\s`.

Se o sinalizador *ASCII* for usado, apenas `[^ \t\n\r\f\v]` é correspondido.

\w

Para padrões Unicode (str):
Corresponde a caracteres de palavras Unicode; isso inclui todos os caracteres alfanuméricos Unicode (conforme definido por *str.isalnum()*), bem como o sublinhado (_).

Se o sinalizador *ASCII* for usado, apenas `[a-zA-Z0-9_]` será correspondido.

Para padrões de 8 bits (isto é, bytes):
Corresponde a caracteres considerados alfanuméricos no conjunto de caracteres ASCII; isso é equivalente a `[a-zA-Z0-9_]`. Se o sinalizador *LOCALE* for usado, corresponde aos caracteres considerados alfanuméricos na localidade atual e o sublinhado.

\W
Corresponde a qualquer caractere que não seja um caractere de palavra. Isso é o oposto de `\w`. Por padrão, corresponde a caracteres que não são sublinhados (_) para os quais *str.isalnum()* retorna `False`.

Se o sinalizador *ASCII* for usado, apenas `[^a-zA-Z0-9_]` será correspondido.

Se o sinalizador *LOCALE* for usado, corresponde a caracteres que não são alfanuméricos na localidade atual nem ao sublinhado.

\z
Corresponde apenas ao final da string.

Adicionado na versão 3.14.

\Z
O mesmo que `\z`. Para compatibilidade com versões mais antigas de Python.

A maioria das sequências de escape com suporte das literais de string Python também são aceitos pelo analisador sintático de expressão regular:

```
\a      \b      \f      \n
\N      \r      \t      \u
\U      \v      \x      \\
```

(Observe que `\b` é usado para representar limites de palavras e significa fazer "backspace" apenas dentro das classes de caracteres.)

As sequências de escape `'\u'`, `'\U'` e `'\N'` são reconhecidas apenas em padrões Unicode (str). Em padrões de bytes, eles são erros. Escapes desconhecidos de letras ASCII são reservados para uso futuro e tratados como erros.

Os escapes octais são incluídos em um formulário limitado. Se o primeiro dígito for 0, ou se houver três dígitos octais, é considerado um escape octal. Caso contrário, é uma referência de grupo. Quanto aos literais de string, os escapes octais têm sempre no máximo três dígitos.

Alterado na versão 3.3: As sequências de escape `'\u'` e `'\U'` foram adicionadas.

Alterado na versão 3.6: Escapes desconhecidos consistindo em `'\'` e uma letra ASCII agora são erros.

Alterado na versão 3.8: A sequência de escape `'\N{name}'` foi adicionada. Como em literais de string, ela se expande para o caractere Unicode nomeado (por exemplo, `'\N{EM DASH}'`).

6.3.2 Conteúdo do módulo

O módulo define várias funções, constantes e uma exceção. Algumas das funções são versões simplificadas dos métodos completos para expressões regulares compiladas. A maioria das aplicações não triviais sempre usa a forma compilada.

Sinalizadores

Alterado na versão 3.6: Constantes de sinalizadores agora são instâncias de *RegexFlag*, que é uma subclasse de *enum.IntFlag*.

class re.**RegexFlag**

Uma subclasse de *enum.IntFlag* contendo as opções de expressão regular listadas abaixo.

Adicionado na versão 3.11: - added to __all__

re.**A**

re.**ASCII**

Faz com que \w, \W, \b, \B, \d, \D, \s e \S executem a correspondência somente ASCII em vez da correspondência Unicode completa. Isso é significativo apenas para padrões Unicode (str) e é ignorado para padrões de bytes.

Corresponde ao sinalizador em linha (?a).

> **Nota**
>
> O sinalizador *U* ainda existe para compatibilidade com versões anteriores, mas é redundante no Python 3, pois as correspondências são feitas em Unicode por padrão para padrões de str e correspondência Unicode não é permitida para padrões de "bytes". *UNICODE* e o sinalizador em linha (?u) são igualmente redundantes.

re.**DEBUG**

Exibe informações de depuração sobre a expressão compilada.

Nenhum sinalizador em linha correspondente.

re.**I**

re.**IGNORECASE**

Faz uma correspondência que não diferencia maiúsculas de minúsculas; expressões como [A-Z] também corresponderão a letras minúsculas. A correspondência Unicode completa (como Ü correspondendo a ü) também funciona, a menos que o sinalizador *ASCII* seja usado para desabilitar correspondências não ASCII. A localidade atual não muda o efeito deste sinalizador a menos que o sinalizador *LOCALE* também seja usado.

Corresponde ao sinalizador em linha (?i).

Observe que quando os padrões Unicode [a-z] ou [A-Z] são usados em combinação com o sinalizador *IGNORECASE*, eles corresponderão às 52 letras ASCII e 4 letras não-ASCII adicionais: 'İ' (U+0130, letra latina I maiúscula com ponto em cima), 'ı' (U+0131, letra latina i minúscula sem ponto), 'ſ' (U+017F, letra latina s minúscula longa) e 'K' (U+212A, sinal de Kelvin). Se o sinalizador *ASCII* for usado, apenas as letras 'a' a 'z' e 'A' a 'Z' serão correspondidas.

re.**L**

re.**LOCALE**

Faz com que \w, \W, \b, \B e a correspondência sem diferenciação de maiúsculas e minúsculas dependam da localidade atual. Esse sinalizador só pode ser usado com padrões de bytes.

Corresponde ao sinalizador em linha (?L).

> **Aviso**
>
> Este sinalizador é desencorajado; considere usar correspondência Unicode em vez disso. O mecanismo de localidade é muito pouco confiável, pois só manipula uma "cultura" por vez e só funciona com localidades de 8 bits. A correspondência Unicode é habilitada por padrão para padrões Unicode (str) e é capaz de manipular diferentes localidades e idiomas.

Alterado na versão 3.6: *LOCALE* pode ser usado apenas com padrões de bytes e não é compatível com *ASCII*.

Alterado na versão 3.7: Objetos de expressão regular compilados com o sinalizador *LOCALE* não dependem mais da localidade em tempo de compilação. Apenas a localidade no momento da correspondência afeta o resultado da correspondência.

re.**M**

re.**MULTILINE**

Quando especificado, o caractere padrão `'^'` corresponde ao início da string e ao início de cada linha (imediatamente após cada nova linha); e o caractere padrão `'$'` corresponde ao final da string e ao final de cada linha (imediatamente antes de cada nova linha). Por padrão, `'^'` corresponde apenas no início da string, e `'$'` apenas no final da string e imediatamente antes da nova linha (se houver) no final da string.

Corresponde ao sinalizador em linha `(?m)`.

re.**NOFLAG**

Indica que nenhum sinalizador está sendo aplicado, o valor é `0`. Este sinalizador pode ser usado como um valor padrão para um argumento nomeado de função ou como um valor base que será condicionalmente OU com outros sinalizadores. Exemplo de uso como valor padrão:

```
def minha_funcao(texto, sinalizador=re.NOFLAG):
    return re.match(texto, sinalizador)
```

Adicionado na versão 3.11.

re.**S**

re.**DOTALL**

Faz o caractere especial `'.'` corresponder com qualquer caractere que seja, incluindo uma nova linha; sem este sinalizador, `'.'` irá corresponder a qualquer coisa, *exceto* uma nova linha.

Corresponde ao sinalizador em linha `(?s)`.

re.**U**

re.**UNICODE**

Em Python 3, os caracteres Unicode são correspondidos por padrão para padrões `str`. Esse sinalizador é, portanto, redundante e **sem efeito**, sendo mantido apenas para compatibilidade com versões anteriores.

Veja *`ASCII`* para restringir a correspondência apenas a caracteres ASCII.

re.**X**

re.**VERBOSE**

Este sinalizador permite que você escreva expressões regulares que parecem mais agradáveis e são mais legíveis, permitindo que você separe visualmente seções lógicas do padrão e adicione comentários. O espaço em branco dentro do padrão é ignorado, exceto quando em uma classe de caractere, ou quando precedido por uma contrabarra sem escape, ou dentro de tokens como `*?`, `(?:` ou `(?P<...>`. Por exemplo, `(? :` e `* ?` não são permitidos. Quando uma linha contém um `#` que não está em uma classe de caractere e não é precedido por uma contrabarra sem escape, todos os caracteres a partir do `#` mais à esquerda até o final da linha são ignorados.

Isso significa que os dois seguintes objetos expressão regular que correspondem a um número decimal são funcionalmente iguais:

```
a = re.compile(r"""\d +  # a parte inteira
                   \.    # o ponto decimal
                   \d *  # alguns digitos fracionários""", re.X)
b = re.compile(r"\d+\.\d*")
```

Corresponde ao sinalizador em linha `(?x)`.

Funções

re.**compile**(*pattern*, *flags=0*)

Compila um padrão de expressão regular em um *objeto expressão regular*, que pode ser usado para correspondência usando seu *`match()`*, *`search()`* e outros métodos, descritos abaixo.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

A sequência

```
prog = re.compile(pattern)
result = prog.match(string)
```

é equivalente a

```
result = re.match(pattern, string)
```

mas usar *re.compile()* e salvar o objeto expressão regular resultante para reutilização é mais eficiente quando a expressão será usada várias vezes em um único programa.

> **Nota**
>
> As versões compiladas dos padrões mais recentes passados para *re.compile()* e as funções de correspondência em nível de módulo são armazenadas em cache, de modo que programas que usam apenas algumas expressões regulares por vez não precisam se preocupar em compilar expressões regulares.

`re.`**`search`**(*pattern*, *string*, *flags=0*)

Percorre a *string* procurando o primeiro local onde o padrão *pattern* de expressão regular produz uma correspondência e retorna um *Match* correspondente. Retorna `None` se nenhuma posição na string corresponder ao padrão; observe que isso é diferente de encontrar uma correspondência de comprimento zero em algum ponto da string.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

`re.`**`match`**(*pattern*, *string*, *flags=0*)

Se zero ou mais caracteres no início da *string* corresponderem ao padrão *pattern* da expressão regular, retorna um *Match* correspondente. Retorna `None` se a string não corresponder ao padrão; observe que isso é diferente de uma correspondência de comprimento zero.

Observe que mesmo no modo *MULTILINE*, *re.match()* irá corresponder apenas no início da string e não no início de cada linha.

Se você quiser localizar uma correspondência em qualquer lugar em *string*, use *search()* (veja também *search() vs. match()*).

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

`re.`**`fullmatch`**(*pattern*, *string*, *flags=0*)

Se toda a *string* corresponder ao padrão *pattern* da expressão regular, retorna um *Match* correspondente. Retorna `None` se a string não corresponder ao padrão; observe que isso é diferente de uma correspondência de comprimento zero.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

Adicionado na versão 3.4.

`re.`**`split`**(*pattern*, *string*, *maxsplit=0*, *flags=0*)

Divide a *string* pelas ocorrências do padrão *pattern*. Se parênteses de captura forem usados em *pattern*, o texto de todos os grupos no padrão também será retornado como parte da lista resultante. Se *maxsplit* for diferente de zero, no máximo *maxsplit* divisões ocorrerão e o restante da string será retornado como o elemento final da lista.

```
>>> re.split(r'\W+', 'Palavras, palavras, palavras.')
['Palavras', 'palavras', 'palavras', '']
>>> re.split(r'(\W+)', 'Palavras, palavras, palavras.')
```

(continua na próxima página)

(continuação da página anterior)

```
['Palavras', ', ', 'palavras', ', ', 'palavras', '.', '']
>>> re.split(r'\W+', 'Palavras, palavras, palavras.', maxsplit=1)
['Palavras', 'palavras, palavras.']
>>> re.split('[a-f]+', '0a3B9', flags=re.IGNORECASE)
['0', '3', '9']
```

Se houver grupos de captura no separador e ele corresponder ao início da string, o resultado começará com uma string vazia. O mesmo vale para o final da string:

```
>>> re.split(r'(\W+)', '...palavras, palavras...')
['', '...', 'palavras', ', ', 'palavras', '...', '']
```

Dessa forma, os componentes do separador são sempre encontrados nos mesmos índices relativos na lista de resultados.

Correspondências vazias adjacentes não são possíveis, mas uma correspondência vazia pode ocorrer imediatamente após uma correspondência não-vazia.

```
>>> re.split(r'\b', 'Palavras, palavras, palavras.')
['', 'Palavras', ', ', 'palavras', ', ', 'palavras', '.']
>>> re.split(r'\W*', '...palavras...')
['', '', 'p', 'a', 'l', 'a', 'v', 'r', 'a', 's', '', '']
>>> re.split(r'(\W*)', '...palavras...')
['', '...', '', '', 'p', '', 'a', '', 'l', '', 'a', '', 'v', '', 'r', '', 'a',
↪'', 's', '...', '', '', '']
```

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

Alterado na versão 3.1: Adicionado o argumento de sinalizadores opcionais.

Alterado na versão 3.7: Adicionado suporte de divisão em um padrão que pode corresponder a uma string vazia.

Descontinuado desde a versão 3.13: Passar *maxsplit* e *flags* como argumentos posicionais foi descontinuado. Nas versões futuras do Python eles serão *parâmetros somente-nomeados*.

`re.`**`findall`**(*pattern*, *string*, *flags=0*)

Retorna todas as correspondências não sobrepostas do padrão *pattern* em *string*, como uma lista de strings ou tuplas. A *string* é verificada da esquerda para a direita e as correspondências são retornadas na ordem encontrada. Correspondências vazias são incluídas no resultado.

O resultado depende do número de grupos de captura no padrão. Se não houver grupos, retorna uma lista de strings que correspondem a todo o padrão. Se houver exatamente um grupo, retorna uma lista de strings correspondentes a esse grupo. Se vários grupos estiverem presentes, retorna uma lista de tuplas de strings correspondentes aos grupos. Grupos de não-captura não afetam a forma do resultado.

```
>>> re.findall(r'\bf[a-z]*', 'which foot or hand fell fastest')
['foot', 'fell', 'fastest']
>>> re.findall(r'(\w+)=(\d+)', 'set width=20 and height=10')
[('width', '20'), ('height', '10')]
```

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

Alterado na versão 3.7: Correspondências não vazias agora podem começar logo após uma correspondência vazia anterior.

`re.`**`finditer`**(*pattern*, *string*, *flags=0*)

Retorna um *iterador* produzindo objetos `Match` sobre todas as correspondências não sobrepostas para o padrão

pattern em *string*. A *string* é percorrida da esquerda para a direita e as correspondências são retornadas na ordem encontrada. Correspondências vazias são incluídas no resultado.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

Alterado na versão 3.7: Correspondências não vazias agora podem começar logo após uma correspondência vazia anterior.

`re.`**`sub`**(*pattern*, *repl*, *string*, *count=0*, *flags=0*)

Retorna a string obtida substituindo as ocorrências não sobrepostas da extremidade esquerda do padrão *pattern* na *string* por *repl*. Se o padrão não for encontrado, *string* será retornado inalterado. *repl* pode ser uma string ou uma função; se for uma string, qualquer escape de contrabarra será processado. Ou seja, `\n` é convertido em um único caractere de nova linha, `\r` é convertido em um retorno de carro e assim por diante. Escapes desconhecidos de letras ASCII são reservados para uso futuro e tratados como erros. Outros escapes desconhecidos como `\&` são deixados como estão. Referências anteriores, como `\6`, são substituídos pela substring correspondida pelo grupo 6 no padrão. Por exemplo:

```
>>> re.sub(r'def\s+([a-zA-Z_][a-zA-Z_0-9]*)\s*\(\s*\):',
...        r'static PyObject*\npy_\1(void)\n{',
...        'def myfunc():')
'static PyObject*\npy_myfunc(void)\n{'
```

Se *repl* for uma função, ela será chamada para cada ocorrência não sobreposta do padrão *pattern*. A função recebe um único argumento *Match* e retorna a string de substituição. Por exemplo:

```
>>> def dashrepl(matchobj):
...     if matchobj.group(0) == '-': return ' '
...     else: return '-'
...
>>> re.sub('-{1,2}', dashrepl, 'arquivos-de-pro----grama')
'arquivos de pro--grama'
>>> re.sub(r'\sE\s', ' & ', 'Feijões queimados e spam', flags=re.IGNORECASE)
'Feijões queimados & spam'
```

O padrão pode ser uma string ou um *Pattern*.

O argumento opcional `count` é o número máximo de ocorrências do padrão a ser substituído; `count` deve ser um número inteiro não negativo. Se omitido ou igual a zero, todas as ocorrências serão substituídas.

Correspondências vazias adjacentes não são possíveis, mas uma correspondência vazia pode ocorrer imediatamente após uma correspondência não-vazia. Como resultado, `sub('x*', '-', 'abxd')` retorna `'-a-b--d-'` invés de `'-a-b-d-'`.

Em argumentos *repl* do tipo string, além dos escapes de caractere e referências anteriores descritas acima, `\g<nome>` usará a substring correspondida pelo grupo denominado `nome`, conforme definido pela sintaxe `(?P<nome>...)`. `\g<número>` usa o número do grupo correspondente; `\g<2>` é portanto equivalente a `\2`, mas não é ambíguo em uma substituição como `\g<2>0`. `\20` seria interpretado como uma referência ao grupo 20, não uma referência ao grupo 2 seguida pelo caractere literal `'0'`. A referência anterior `\g6` substitui em toda a substring correspondida pela ER.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

Alterado na versão 3.1: Adicionado o argumento de sinalizadores opcionais.

Alterado na versão 3.5: Grupos sem correspondência são substituídos por uma string vazia.

Alterado na versão 3.6: Escapes desconhecidos no padrão *pattern* consistindo em `'\'` e uma letra ASCII agora são erros.

Alterado na versão 3.7: Escapes desconhecidos em *repl* consistindo em `'\'` e uma letra ASCII agora são erros. Uma correspondência vazia pode ocorrer imediatamente após uma correspondência não-vazia.

Alterado na versão 3.12: O grupo *id* só pode conter dígitos ASCII. Nas strings de substituição *bytes*, o grupo *name* só pode conter bytes no intervalo ASCII (`b'\x00'-b'\x7f'`).

Descontinuado desde a versão 3.13: Passar *count* e *flags* como argumentos posicionais foi descontinuado. Nas versões futuras do Python eles serão *parâmetros somente-nomeados*.

`re.`**`subn`**(*pattern*, *repl*, *string*, *count=0*, *flags=0*)

Executa a mesma operação que *sub()*, mas retorna uma tupla `(nova_string, número_de_substituições_feitas)`.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos *sinalizadores*, combinados usando OU bit a bit (o operador `|`).

`re.`**`escape`**(*pattern*)

Escapa caracteres especiais no padrão *pattern*. Isso é útil se você deseja corresponder uma string literal arbitrária que pode conter metacaracteres de expressão regular. Por exemplo:

```
>>> print(re.escape('https://www.python.org'))
https://www\.python\.org

>>> legal_chars = string.ascii_lowercase + string.digits + "!#$%&'*+-.^_`|~:"
>>> print('[%s]+' % re.escape(legal_chars))
[abcdefghijklmnopqrstuvwxyz0123456789!\#\$\%\&\'\*\+\-\.\^_`\|\~:]+

>>> operators = ['+', '-', '*', '/', '**']
>>> print('|'.join(map(re.escape, sorted(operators, reverse=True))))
/|\-|\+|\*\*|\*
```

Esta função não deve ser usada para a string de substituição em *sub()* e *subn()*, apenas contrabarras devem ser escapadas. Por exemplo:

```
>>> digits_re = r'\d+'
>>> sample = '/usr/sbin/sendmail - 0 errors, 12 warnings'
>>> print(re.sub(digits_re, digits_re.replace('\\', r'\\'), sample))
/usr/sbin/sendmail - \d+ errors, \d+ warnings
```

Alterado na versão 3.3: O caractere `'_'` não é mais escapado.

Alterado na versão 3.7: Somente caracteres que podem ter um significado especial em uma expressão regular são escapados. Como resultado, `'!'`, `'"'`, `'%'`, `"'"`, `','`, `'/'`, `':'`, `';'`, `'<'`, `'='`, `'>'`, `'@'`, e `"`" não são mais escapados.

`re.`**`purge`**()

Limpa o cache da expressão regular.

Exceções

exception `re.`**`PatternError`**(*msg*, *pattern=None*, *pos=None*)

Exceção levantada quando uma string passada para uma das funções desde módulo não é uma expressão regular válida (por exemplo, ela pode conter parênteses não correspondentes) ou quando algum outro erro ocorre durante a compilação ou correspondência. Nunca é um erro se uma string não contém correspondência para um padrão. A instância de `PatternError` possui os seguintes atributos adicionais:

msg

A mensagem de erro não formatada.

pattern

O padrão da expressão regular.

pos

O índice no padrão *pattern* no qual a compilação falhou (pode ser `None`).

lineno

A linha correspondente a *pos* (pode ser `None`).

colno

A coluna correspondente a *pos* (pode ser `None`).

Alterado na versão 3.5: Adicionados os atributos adicionais.

Alterado na versão 3.13: `PatternError` era originalmente chamado de `error`; o último é mantido como um apelido para compatibilidade com versões anteriores.

6.3.3 Objetos expressão regular

class re.**Pattern**

Objeto expressão regular compilado retornado por *re.compile()*.

Alterado na versão 3.9: *re.Pattern* tem suporte a `[]` para indicar um padrão Unicode (str) ou bytes. Veja *Tipo Generic Alias*.

Pattern.**search**(*string*[, *pos*[, *endpos*]])

Percorre a *string* procurando o primeiro local onde esta expressão regular produz uma correspondência e retorna um *Match* correspondente. Retorna `None` se nenhuma posição na string corresponder ao padrão; observe que isso é diferente de encontrar uma correspondência de comprimento zero em algum ponto da string.

O segundo parâmetro opcional *pos* fornece um índice na string onde a pesquisa deve começar; o padrão é `0`. Isso não é totalmente equivalente a fatiar a string; o caractere padrão `'^'` corresponde no início real da string e nas posições logo após uma nova linha, mas não necessariamente no índice onde a pesquisa deve começar.

O parâmetro opcional *endpos* limita o quão longe a string será pesquisada; será como se a string tivesse *endpos* caracteres, então apenas os caracteres de *pos* a `endpos - 1` serão procurados por uma correspondência. Se *endpos* for menor que *pos*, nenhuma correspondência será encontrada; caso contrário, se *rx* é um objeto de expressão regular compilado, `rx.search(string, 0, 50)` é equivalente a `rx.search(string[:50], 0)`.

```
>>> pattern = re.compile("d")
>>> pattern.search("dog")     # Correspondeu no índice 0
<re.Match object; span=(0, 1), match='d'>
>>> pattern.search("dog", 1)  # Não correspondeu; a busca não inclui "d"
```

Pattern.**match**(*string*[, *pos*[, *endpos*]])

Se zero ou mais caracteres no início da *string* corresponderem a esta expressão regular, retorna um *Match* correspondente. Retorna `None` se a string não corresponder ao padrão; observe que isso é diferente de uma correspondência de comprimento zero.

Os parâmetros opcionais *pos* e *endpos* têm o mesmo significado que para o método *search()*.

```
>>> pattern = re.compile("o")
>>> pattern.match("dog")      # Não correspondeu, pois "o" não é o início de
↪"dog".
>>> pattern.match("dog", 1)   # Correspondeu, pois "o" é o 2º caractere de "dog
↪".
<re.Match object; span=(1, 2), match='o'>
```

Se você quiser localizar uma correspondência em qualquer lugar em *string*, use *search()* ao invés (veja também *search() vs. match()*).

Pattern.**fullmatch**(*string*[, *pos*[, *endpos*]])

Se toda a *string* corresponder a esta expressão regular, retorna um *Match* correspondente. Retorna `None` se a string não corresponder ao padrão; observe que isso é diferente de uma correspondência de comprimento zero.

Os parâmetros opcionais *pos* e *endpos* têm o mesmo significado que para o método *search()*.

```
>>> pattern = re.compile("o[gh]")
>>> pattern.fullmatch("dog")      # Não correspondeu, pois "o" não é o início␣
↪de "dog".
>>> pattern.fullmatch("ogre")     # Não correspondeu, pois não é uma␣
↪correspondência da string inteira.
>>> pattern.fullmatch("doggie", 1, 3)   # Corresponde dentro dos limites dados.
<re.Match object; span=(1, 3), match='og'>
```

Adicionado na versão 3.4.

`Pattern.`**`split`**(*string*, *maxsplit=0*)

Idêntico à função `split()`, usando o padrão compilado.

`Pattern.`**`findall`**(*string*[, *pos*[, *endpos*]])

Semelhante à função `findall()`, usando o padrão compilado, mas também aceita os parâmetros *pos* e *endpos* opcionais que limitam a região de pesquisa como para `search()`.

`Pattern.`**`finditer`**(*string*[, *pos*[, *endpos*]])

Semelhante à função `finditer()`, usando o padrão compilado, mas também aceita os parâmetros *pos* e *endpos* opcionais que limitam a região de pesquisa como para `search()`.

`Pattern.`**`sub`**(*repl*, *string*, *count=0*)

Idêntico à função `sub()`, usando o padrão compilado.

`Pattern.`**`subn`**(*repl*, *string*, *count=0*)

Idêntico à função `subn()`, usando o padrão compilado.

`Pattern.`**`flags`**

Os sinalizadores de correspondência de expressões regulares. Esta é uma combinação dos sinalizadores fornecidos para `compile()`, qualquer sinalizador em linha `(?...)` no padrão e sinalizadores implícitos como `UNICODE` se o padrão for uma string Unicode.

`Pattern.`**`groups`**

O número de grupos de captura no padrão.

`Pattern.`**`groupindex`**

Um dicionário que mapeia qualquer nome de grupo simbólico definido por `(?P<id>)` para números de grupo. O dicionário estará vazio se nenhum grupo simbólico for usado no padrão.

`Pattern.`**`pattern`**

A string de padrão da qual o objeto de padrão foi compilado.

Alterado na versão 3.7: Adicionado suporte de `copy.copy()` e `copy.deepcopy()`. Os objetos expressão regular compilados são considerados atômicos.

6.3.4 Objetos correspondência

Objetos correspondência sempre têm um valor booleano de `True`. Como `match()` e `search()` retornam `None` quando não há correspondência, você pode testar se houve uma correspondência com uma simples instrução `if`:

```
match = re.search(pattern, string)
if match:
    process(match)
```

`class` `re.`**`Match`**

Objeto correspondência retornado por `match`s e `search`s bem sucedidos.

Alterado na versão 3.9: `re.Match` tem suporte a `[]` para indicar uma correspondência Unicode (str) ou bytes. Veja *Tipo Generic Alias*.

Match.**expand**(*template*)

Retorna a string obtida fazendo a substituição da contrabarra na string template *template*, como feito pelo método *sub()*. Escapes como \n são convertidos para os caracteres apropriados, e referências anteriores numéricas (\1, \2) e referências anteriores nomeadas (\g<1>, \g<nome>) são substituídas pelo conteúdo do grupo correspondente. A referência anterior \g<0> será substituída pela correspondência completa.

Alterado na versão 3.5: Grupos sem correspondência são substituídos por uma string vazia.

Match.**group**([*group1*, ...])

Retorna um ou mais subgrupos da correspondência. Se houver um único argumento, o resultado será uma única string; se houver vários argumentos, o resultado é uma tupla com um item por argumento. Sem argumentos, *group1* padroniza para zero (toda a correspondência é retornada). Se um argumento *groupN* for zero, o valor de retorno correspondente será toda a string correspondente; se estiver no intervalo inclusivo [1..99], é a string que corresponde ao grupo entre parênteses correspondente. Se um número de grupo for negativo ou maior do que o número de grupos definidos no padrão, uma exceção *IndexError* é levantada. Se um grupo estiver contido em uma parte do padrão que não correspondeu, o resultado correspondente será None. Se um grupo estiver contido em uma parte do padrão que correspondeu várias vezes, a última correspondência será retornada.

```
>>> m = re.match(r"(\w+) (\w+)", "Isaac Newton, físico")
>>> m.group(0)       # A correspondência inteira
'Isaac Newton'
>>> m.group(1)       # O primeiro subgrupo entre parênteses.
'Isaac'
>>> m.group(2)       # O segundo subgrupo entre parênteses.
'Newton'
>>> m.group(1, 2)    # Múltiplos argumentos retornam uma tupla.
('Isaac', 'Newton')
```

Se a expressão regular usa a sintaxe (?P<nome>...), os argumentos *groupN* também podem ser strings que identificam grupos por seus nomes de grupo. Se um argumento string não for usado como um nome de grupo no padrão, uma exceção *IndexError* é levantada.

Um exemplo moderadamente complicado:

```
>>> m = re.match(r"(?P<primeiro_nome>\w+) (?P<sobrenome>\w+)", "Malcolm␣
↪Reynolds")
>>> m.group('primeiro_nome')
'Malcolm'
>>> m.group('sobrenome')
'Reynolds'
```

Grupos nomeados também podem ser referidos por seu índice:

```
>>> m.group(1)
'Malcolm'
>>> m.group(2)
'Reynolds'
```

Se um grupo corresponder várias vezes, apenas a última correspondência estará acessível:

```
>>> m = re.match(r"(..)+", "a1b2c3")  # Corresponde 3 vezes.
>>> m.group(1)                        # Retorna somente a última␣
↪correspondência.
'c3'
```

Match.**__getitem__**(*g*)

Isso é idêntico a m.group(g). Isso permite acesso mais fácil a um grupo individual de uma correspondência:

```
>>> m = re.match(r"(\w+) (\w+)", "Isaac Newton, físico")
>>> m[0]       # A correspondência inteira
'Isaac Newton'
>>> m[1]       # O primeiro subgrupo entre parênteses.
'Isaac'
>>> m[2]       # O segundo subgrupo entre parênteses.
'Newton'
```

Também há suporte para grupos nomeados:

```
>>> m = re.match(r"(?P<primeiro_nome>\w+) (?P<sobrenome>\w+)", "Isaac Newton")
>>> m['primeiro_nome']
'Isaac'
>>> m['sobrenome']
'Newton'
```

Adicionado na versão 3.6.

`Match.`**`groups`**`(`*default=None*`)`

Retorna uma tupla contendo todos os subgrupos da correspondência, de 1 até quantos grupos estiverem no padrão. O argumento *default* é usado para grupos que não participaram da correspondência; o padrão é `None`.

Por exemplo:

```
>>> m = re.match(r"(\d+)\.(\d+)", "24.1632")
>>> m.groups()
('24', '1632')
```

Se colocarmos a casa decimal e tudo depois dela opcional, nem todos os grupos podem participar da correspondência. Esses grupos serão padronizados como `None`, a menos que o argumento *default* seja fornecido:

```
>>> m = re.match(r"(\d+)\.?(\d+)?", "24")
>>> m.groups()      # Segundo grupo padronizado para None.
('24', None)
>>> m.groups('0')   # Agora, o segundo grupo é padronizado para '0'.
('24', '0')
```

`Match.`**`groupdict`**`(`*default=None*`)`

Retorna um dicionário contendo todos os subgrupos *nomeados* da correspondência, tendo como chave o nome do subgrupo. O argumento *default* usado para grupos que não participaram da correspondência; o padrão é `None`. Por exemplo:

```
>>> m = re.match(r"(?P<primeiro_nome>\w+) (?P<sobrenome>\w+)", "Malcolm␣
↪Reynolds")
>>> m.groupdict()
{'primeiro_nome': 'Malcolm', 'sobrenome': 'Reynolds'}
```

`Match.`**`start`**`(`[*group*]`)`

`Match.`**`end`**`(`[*group*]`)`

Retorna os índices de início e fim da substring correspondida pelo grupo *group*; *group* tem como padrão zero (o que significa que toda a substring é correspondida). Retorna `-1` se *group* existe, mas não contribuiu para a correspondência. Para um objeto correspondência *m* e um grupo *g* que contribuiu para a correspondência, a substring correspondida pelo grupo *g* (equivalente a `m.group(g)`) é

```
m.string[m.start(g):m.end(g)]
```

Observe que `m.start(group)` será igual a `m.end(group)` se *group* correspondeu a uma string nula. Por exemplo, após `m = re.search('b(c?)', 'cba')`, `m.start(0)` é 1, `m.end(0)` é 2, `m.start(1)` e `m.end(1)` são 2, e `m.start(2)` levanta uma exceção *IndexError*.

Um exemplo que removerá *remova_isto* dos endereços de e-mail:

```
>>> email = "tony@tiremova_istoger.net"
>>> m = re.search("remova_isto", email)
>>> email[:m.start()] + email[m.end():]
'tony@tiger.net'
```

`Match.`**`span`**`(`[*group*]`)`

Para uma correspondência *m*, retorna a tupla com dois elementos `(m.start(group), m.end(group))`. Observe que se *group* não contribuiu para a correspondência, isso é `(-1, -1)`. *group* tem como padrão zero, a correspondência inteira.

`Match.`**`pos`**

O valor de *pos* que foi passado para o método *search()* ou *match()* de um *objeto expressão regular*. Este é o índice da string na qual o mecanismo de ER começou a procurar uma correspondência.

`Match.`**`endpos`**

O valor de *endpos* que foi passado para o método *search()* ou *match()* de um *objeto expressão regular*. Este é o índice da string após o qual o mecanismo de ER não vai chegar.

`Match.`**`lastindex`**

O índice em número inteiro do último grupo de captura correspondido, ou `None` se nenhum grupo foi correspondido. Por exemplo, as expressões `(a)b`, `((a)(b))` e `((ab))` terão `lastindex == 1` se aplicadas à string `'ab'`, enquanto a expressão `(a)(b)` terá `lastindex == 2`, se aplicada à mesma string.

`Match.`**`lastgroup`**

O nome do último grupo de captura correspondido, ou `None` se o grupo não tinha um nome, ou se nenhum grupo foi correspondido.

`Match.`**`re`**

O *objeto expressão regular* cujo método *match()* ou *search()* produziu esta instância de correspondência.

`Match.`**`string`**

A string passada para *match()* ou *search()*.

Alterado na versão 3.7: Adicionado suporte para *copy.copy()* e *copy.deepcopy()*. Objetos correspondência são considerados atômicos.

6.3.5 Exemplos de expressão regular

Verificando por um par

Neste exemplo, usaremos a seguinte função auxiliar para exibir objetos correspondência com um pouco mais de elegância:

```
def displaymatch(match):
    if match is None:
        return None
    return '<Match: %r, groups=%r>' % (match.group(), match.groups())
```

Suponha que você esteja escrevendo um programa de pôquer onde a mão de um jogador é representada como uma string de 5 caracteres com cada caractere representando uma carta, "a" para ás, "k" para rei, "q" para dama, "j" para valete, "t" para 10 e "2" a "9" representando a carta com esse valor.

Para ver se uma determinada string é uma mão válida, pode-se fazer o seguinte:

```
>>> valid = re.compile(r"^[a2-9tjqk]{5}$")
>>> displaymatch(valid.match("akt5q"))  # Válida.
"<Match: 'akt5q', groups=()>"
>>> displaymatch(valid.match("akt5e"))  # Inválida.
>>> displaymatch(valid.match("akt"))    # Inválida.
```

(continua na próxima página)

(continuação da página anterior)

```
>>> displaymatch(valid.match("727ak"))  # Válida.
"<Match: '727ak', groups=()>"
```

Essa última mão, `"727ak"`, continha um par, ou duas cartas com o mesmo valor. Para combinar isso com uma expressão regular, pode-se usar referências anteriores como:

```
>>> pair = re.compile(r".*(.).*\1")
>>> displaymatch(pair.match("717ak"))     # Par de 7s.
"<Match: '717', groups=('7',)>"
>>> displaymatch(pair.match("718ak"))     # Sem pares.
>>> displaymatch(pair.match("354aa"))     # Par de áses.
"<Match: '354aa', groups=('a',)>"
```

Para descobrir em que carta o par consiste, pode-se usar o método *`group()`* do objeto correspondência da seguinte maneira:

```
>>> pair = re.compile(r".*(.).*\1")
>>> pair.match("717ak").group(1)
'7'

# Erro porque re.match() retorna None, que não tem um método group()::
>>> pair.match("718ak").group(1)
Traceback (most recent call last):
  File "<pyshell#23>", line 1, in <module>
    re.match(r".*(.).*\1", "718ak").group(1)
AttributeError: 'NoneType' object has no attribute 'group'

>>> pair.match("354aa").group(1)
'a'
```

Simulando scanf()

Atualmente, Python não possui um equivalente a `scanf()`. Expressões regulares são geralmente mais poderosas, embora também mais prolixas, do que strings de formato `scanf()`. A tabela abaixo oferece alguns mapeamentos mais ou menos equivalentes entre strings de formato `scanf()` e expressões regulares.

Símbolo do `scanf()`	Expressão regular
`%c`	`.`
`%5c`	`.{5}`
`%d`	`[-+]?\d+`
`%e, %E, %f, %g`	`[-+]?(\d+(\.\d*)?|\.\d+)([eE][-+]?\d+)?`
`%i`	`[-+]?(0[xX][\dA-Fa-f]+|0[0-7]*|\d+)`
`%o`	`[-+]?[0-7]+`
`%s`	`\S+`
`%u`	`\d+`
`%x, %X`	`[-+]?(0[xX])?[\dA-Fa-f]+`

Para extrair um nome de arquivo e números de uma string como

```
/usr/sbin/sendmail - 0 errors, 4 warnings
```

você usaria um formato `scanf()` como

```
%s - %d errors, %d warnings
```

A expressão regular equivalente seria

```
(\S+) - (\d+) errors, (\d+) warnings
```

search() vs. match()

Python oferece diferentes operações primitivas baseadas em expressões regulares:

- *re.match()* verifica uma correspondência apenas no início da string
- *re.search()* verifica uma correspondência em qualquer lugar na string (isso é o que o Perl faz por padrão)
- *re.fullmatch()* verifica toda a string para ser uma correspondência

Por exemplo:

```
>>> re.match("c", "abcdef")    # Sem correspondência
>>> re.search("c", "abcdef")   # Correspondência
<re.Match object; span=(2, 3), match='c'>
>>> re.fullmatch("p.*n", "python") # Correspondência
<re.Match object; span=(0, 6), match='python'>
>>> re.fullmatch("r.*n", "python") # Sem correspondência
```

Expressões regulares começando com `'^'` podem ser usadas com *search()* para restringir a correspondência no início da string:

```
>>> re.match("c", "abcdef")    # Sem correspondência
>>> re.search("^c", "abcdef")  # Sem correspondência
>>> re.search("^a", "abcdef")  # Correspondência
<re.Match object; span=(0, 1), match='a'>
```

Observe, entretanto, que no modo *MULTILINE* *match()* apenas corresponde ao início da string, enquanto que usar *search()* com uma expressão regular começando com `'^'` irá corresponder no início de cada linha.

```
>>> re.match("X", "A\nB\nX", re.MULTILINE)  # Sem correspondência
>>> re.search("^X", "A\nB\nX", re.MULTILINE)  # Correspondência
<re.Match object; span=(4, 5), match='X'>
```

Criando uma lista telefônica

split() divide uma string em uma lista delimitada pelo padrão passado. O método é inestimável para converter dados textuais em estruturas de dados que podem ser facilmente lidas e modificadas pelo Python, conforme demonstrado no exemplo a seguir que cria uma lista telefônica.

Primeiro, aqui está a entrada. Normalmente pode vir de um arquivo, aqui estamos usando a sintaxe de string entre aspas triplas.

```
>>> texto = """Ross McFluff: 834.345.1254 155 Elm Street
...
... Ronald Heathmore: 892.345.3428 436 Finley Avenue
... Frank Burger: 925.541.7625 662 South Dogwood Way
...
...
... Heather Albrecht: 548.326.4584 919 Park Place"""
```

Os registros são separados por uma ou mais novas linhas. Agora, convertemos a string em uma lista com cada linha não vazia tendo seu próprio registro:

```
>>> registros = re.split("\n+", texto)
>>> registros
['Ross McFluff: 834.345.1254 155 Elm Street',
'Ronald Heathmore: 892.345.3428 436 Finley Avenue',
```

(continua na próxima página)

(continuação da página anterior)

```
'Frank Burger: 925.541.7625 662 South Dogwood Way',
'Heather Albrecht: 548.326.4584 919 Park Place']
```

Finalmente, divida cada registro em uma lista com nome, sobrenome, número de telefone e endereço. Usamos o parâmetro `maxsplit` de *`split()`* porque o endereço contém espaços, nosso padrão de divisão:

```
>>> [re.split(":? ", registro, maxsplit=3) for registro in registros]
[['Ross', 'McFluff', '834.345.1254', '155 Elm Street'],
['Ronald', 'Heathmore', '892.345.3428', '436 Finley Avenue'],
['Frank', 'Burger', '925.541.7625', '662 South Dogwood Way'],
['Heather', 'Albrecht', '548.326.4584', '919 Park Place']]
```

O padrão `:?` corresponde ao caractere de dois pontos após o sobrenome, de modo que não ocorre na lista de resultados. Com um `maxsplit` de `4`, podemos separar o número da casa do nome da rua:

```
>>> [re.split(":? ", registro, maxsplit=4) for registro in registros]
[['Ross', 'McFluff', '834.345.1254', '155', 'Elm Street'],
['Ronald', 'Heathmore', '892.345.3428', '436', 'Finley Avenue'],
['Frank', 'Burger', '925.541.7625', '662', 'South Dogwood Way'],
['Heather', 'Albrecht', '548.326.4584', '919', 'Park Place']]
```

Mastigando texto

`sub()` substitui cada ocorrência de um padrão por uma string ou o resultado de uma função. Este exemplo demonstra o uso de *`sub()`* com uma função para "mastigar" o texto ou aleatorizar a ordem de todos os caracteres em cada palavra de uma frase, exceto o primeiro e o último caracteres:

```
>>> def repl(m):
...     inner_word = list(m.group(2))
...     random.shuffle(inner_word)
...     return m.group(1) + "".join(inner_word) + m.group(3)
...
>>> text = "Professor Masanori, favor comunicar suas ausências imediatamente."
>>> re.sub(r"(\w)(\w+)(\w)", repl, text)
'Psesforor Mranoasi, favor cmounacir suas acaêiusns idaaeemttnmie.'
>>> re.sub(r"(\w)(\w+)(\w)", repl, text)
'Prfseosor Monsaari, fovar camonucir saus acuêsains ientdaammetie.'
```

Encontrando todos os advérbios

`findall()` corresponde *todas* as ocorrências de um padrão, não apenas a primeira como *`search()`* faz. Por exemplo, se uma pessoa deseja encontrar todos os advérbios em algum texto, ela pode usar *`findall()`* da seguinte maneira:

```
>>> text = "Ele estava cuidadosamente disfarçado, mas foi rapidamente capturado␣
↪pela polícia."
>>> re.findall(r"\w+mente\b", text)
['cuidadosamente', 'rapidamente']
```

Encontrando todos os advérbios e suas posições

Se alguém deseja obter mais informações sobre todas as correspondências de um padrão além do texto correspondido, *`finditer()`* é útil, pois fornece objetos *`Match`* em vez de strings. Continuando com o exemplo anterior, se um escritor quisesse encontrar todos os advérbios *e suas posições* em algum texto, ele usaria *`finditer()`* da seguinte maneira:

```
>>> text = "Ele estava cuidadosamente disfarçado, mas foi rapidamente capturado␣
↪pela polícia."
>>> for m in re.finditer(r"\w+mente\b", text):
... print('%02d-%02d: %s' % (m.start(), m.end(), m.group(0)))
11-25: cuidadosamente
46-57: rapidamente
```

Notação de string bruta

A notação de string bruta (`r"texto"`) mantém as expressões regulares sãs. Sem ela, cada contrabarra (`'\'`) em uma expressão regular teria que ser prefixada com outra para ter seu efeito de caractere especial anulado. Por exemplo, as duas linhas de código a seguir são funcionalmente idênticas:

```
>>> re.match(r"\W(.)\1\W", " ff ")
<re.Match object; span=(0, 4), match=' ff '>
>>> re.match("\\W(.)\\1\\W", " ff ")
<re.Match object; span=(0, 4), match=' ff '>
```

Quando se deseja corresponder a uma contrabarra literal, ela deve ser prefixada com outra contrabarra na expressão regular. Com a notação de string bruta, isso significa `r"\\"`. Sem a notação de string bruta, deve-se usar `"\\\\"`, tornando as seguintes linhas de código funcionalmente idênticas:

```
>>> re.match(r"\\", r"\\")
<re.Match object; span=(0, 1), match='\\'>
>>> re.match("\\\\", r"\\")
<re.Match object; span=(0, 1), match='\\'>
```

Escrevendo um tokenizador

Um tokenizador ou scanner analisa uma string para categorizar grupos de caracteres. Este é um primeiro passo útil para escrever um compilador ou interpretador.

As categorias de texto são especificadas com expressões regulares. A técnica é combiná-las em uma única expressão regular mestre e fazer um laço em correspondências sucessivas:

```
from typing import NamedTuple
import re

class Token(NamedTuple):
    type: str
    value: str
    line: int
    column: int

def tokenize(code):
    keywords = {'IF', 'THEN', 'ENDIF', 'FOR', 'NEXT', 'GOSUB', 'RETURN'}
    token_specification = [
        ('NUMBER',   r'\d+(\.\d*)?'),  # Número inteiro ou decimal
        ('ASSIGN',   r':='),           # Operador de atribuição
        ('END',      r';'),            # Fiinalizador de instrução
        ('ID',       r'[A-Za-z]+'),    # Identificadores
        ('OP',       r'[+\-*/]'),      # Operadores aritméticos
        ('NEWLINE',  r'\n'),           # Finais de linha
        ('SKIP',     r'[ \t]+'),       # Pula espaços e tabulações
        ('MISMATCH', r'.'),            # Qualquer outro caracter
    ]
    tok_regex = '|'.join('(?P<%s>%s)' % pair for pair in token_specification)
```

(continua na próxima página)

(continuação da página anterior)

```
    line_num = 1
    line_start = 0
    for mo in re.finditer(tok_regex, code):
        kind = mo.lastgroup
        value = mo.group()
        column = mo.start() - line_start
        if kind == 'NUMBER':
            value = float(value) if '.' in value else int(value)
        elif kind == 'ID' and value in keywords:
            kind = value
        elif kind == 'NEWLINE':
            line_start = mo.end()
            line_num += 1
            continue
        elif kind == 'SKIP':
            continue
        elif kind == 'MISMATCH':
            raise RuntimeError(f'{value!r} unexpected on line {line_num}')
        yield Token(kind, value, line_num, column)

statements = '''
    IF quantity THEN
        total := total + price * quantity;
        tax := price * 0.05;
    ENDIF;
'''

for token in tokenize(statements):
    print(token)
```

O tokenizador produz a seguinte saída:

```
Token(type='IF', value='IF', line=2, column=4)
Token(type='ID', value='quantity', line=2, column=7)
Token(type='THEN', value='THEN', line=2, column=16)
Token(type='ID', value='total', line=3, column=8)
Token(type='ASSIGN', value=':=', line=3, column=14)
Token(type='ID', value='total', line=3, column=17)
Token(type='OP', value='+', line=3, column=23)
Token(type='ID', value='price', line=3, column=25)
Token(type='OP', value='*', line=3, column=31)
Token(type='ID', value='quantity', line=3, column=33)
Token(type='END', value=';', line=3, column=41)
Token(type='ID', value='tax', line=4, column=8)
Token(type='ASSIGN', value=':=', line=4, column=12)
Token(type='ID', value='price', line=4, column=15)
Token(type='OP', value='*', line=4, column=21)
Token(type='NUMBER', value=0.05, line=4, column=23)
Token(type='END', value=';', line=4, column=27)
Token(type='ENDIF', value='ENDIF', line=5, column=4)
Token(type='END', value=';', line=5, column=9)
```

6.4 difflib — Helpers for computing deltas

Código-fonte: Lib/difflib.py

This module provides classes and functions for comparing sequences. It can be used for example, for comparing files, and can produce information about file differences in various formats, including HTML and context and unified diffs. For comparing directories and files, see also, the *filecmp* module.

class difflib.**SequenceMatcher**

This is a flexible class for comparing pairs of sequences of any type, so long as the sequence elements are *hashable*. The basic algorithm predates, and is a little fancier than, an algorithm published in the late 1980's by Ratcliff and Obershelp under the hyperbolic name "gestalt pattern matching." The idea is to find the longest contiguous matching subsequence that contains no "junk" elements; these "junk" elements are ones that are uninteresting in some sense, such as blank lines or whitespace. (Handling junk is an extension to the Ratcliff and Obershelp algorithm.) The same idea is then applied recursively to the pieces of the sequences to the left and to the right of the matching subsequence. This does not yield minimal edit sequences, but does tend to yield matches that "look right" to people.

Timing: The basic Ratcliff-Obershelp algorithm is cubic time in the worst case and quadratic time in the expected case. *SequenceMatcher* is quadratic time for the worst case and has expected-case behavior dependent in a complicated way on how many elements the sequences have in common; best case time is linear.

Automatic junk heuristic: *SequenceMatcher* supports a heuristic that automatically treats certain sequence items as junk. The heuristic counts how many times each individual item appears in the sequence. If an item's duplicates (after the first one) account for more than 1% of the sequence and the sequence is at least 200 items long, this item is marked as "popular" and is treated as junk for the purpose of sequence matching. This heuristic can be turned off by setting the `autojunk` argument to `False` when creating the *SequenceMatcher*.

Alterado na versão 3.2: Added the *autojunk* parameter.

class difflib.**Differ**

This is a class for comparing sequences of lines of text, and producing human-readable differences or deltas. Differ uses *SequenceMatcher* both to compare sequences of lines, and to compare sequences of characters within similar (near-matching) lines.

Each line of a *Differ* delta begins with a two-letter code:

Código	Significado
`'- '`	line unique to sequence 1
`'+ '`	line unique to sequence 2
`'  '`	line common to both sequences
`'? '`	line not present in either input sequence

Lines beginning with '`?`' attempt to guide the eye to intraline differences, and were not present in either input sequence. These lines can be confusing if the sequences contain whitespace characters, such as spaces, tabs or line breaks.

class difflib.**HtmlDiff**

This class can be used to create an HTML table (or a complete HTML file containing the table) showing a side by side, line by line comparison of text with inter-line and intra-line change highlights. The table can be generated in either full or contextual difference mode.

The constructor for this class is:

__init__(*tabsize=8*, *wrapcolumn=None*, *linejunk=None*, *charjunk=IS_CHARACTER_JUNK*)

Initializes instance of *HtmlDiff*.

tabsize is an optional keyword argument to specify tab stop spacing and defaults to `8`.

wrapcolumn is an optional keyword to specify column number where lines are broken and wrapped, defaults to `None` where lines are not wrapped.

linejunk and *charjunk* are optional keyword arguments passed into *ndiff()* (used by *HtmlDiff* to generate the side by side HTML differences). See *ndiff()* documentation for argument default values and descriptions.

The following methods are public:

make_file(*fromlines*, *tolines*, *fromdesc=''*, *todesc=''*, *context=False*, *numlines=5*, ***, *charset='utf-8'*)

Compares *fromlines* and *tolines* (lists of strings) and returns a string which is a complete HTML file containing a table showing line by line differences with inter-line and intra-line changes highlighted.

fromdesc and *todesc* are optional keyword arguments to specify from/to file column header strings (both default to an empty string).

context and *numlines* are both optional keyword arguments. Set *context* to `True` when contextual differences are to be shown, else the default is `False` to show the full files. *numlines* defaults to `5`. When *context* is `True` *numlines* controls the number of context lines which surround the difference highlights. When *context* is `False` *numlines* controls the number of lines which are shown before a difference highlight when using the "next" hyperlinks (setting to zero would cause the "next" hyperlinks to place the next difference highlight at the top of the browser without any leading context).

> **Nota**
>
> *fromdesc* and *todesc* are interpreted as unescaped HTML and should be properly escaped while receiving input from untrusted sources.

Alterado na versão 3.5: *charset* keyword-only argument was added. The default charset of HTML document changed from `'ISO-8859-1'` to `'utf-8'`.

make_table(*fromlines*, *tolines*, *fromdesc=''*, *todesc=''*, *context=False*, *numlines=5*)

Compares *fromlines* and *tolines* (lists of strings) and returns a string which is a complete HTML table showing line by line differences with inter-line and intra-line changes highlighted.

The arguments for this method are the same as those for the *make_file()* method.

difflib.**context_diff**(*a*, *b*, *fromfile=''*, *tofile=''*, *fromfiledate=''*, *tofiledate=''*, *n=3*, *lineterm='\n'*)

Compare *a* and *b* (lists of strings); return a delta (a *generator* generating the delta lines) in context diff format.

Context diffs are a compact way of showing just the lines that have changed plus a few lines of context. The changes are shown in a before/after style. The number of context lines is set by *n* which defaults to three.

By default, the diff control lines (those with `***` or `---`) are created with a trailing newline. This is helpful so that inputs created from *io.IOBase.readlines()* result in diffs that are suitable for use with *io.IOBase.writelines()* since both the inputs and outputs have trailing newlines.

For inputs that do not have trailing newlines, set the *lineterm* argument to `""` so that the output will be uniformly newline free.

The context diff format normally has a header for filenames and modification times. Any or all of these may be specified using strings for *fromfile*, *tofile*, *fromfiledate*, and *tofiledate*. The modification times are normally expressed in the ISO 8601 format. If not specified, the strings default to blanks.

```
>>> import sys
>>> from difflib import *
>>> s1 = ['bacon\n', 'eggs\n', 'ham\n', 'guido\n']
>>> s2 = ['python\n', 'eggy\n', 'hamster\n', 'guido\n']
>>> sys.stdout.writelines(context_diff(s1, s2, fromfile='before.py',
...                       tofile='after.py'))
*** before.py
--- after.py
```

(continua na próxima página)

(continuação da página anterior)

```
***************
*** 1,4 ****
! bacon
! eggs
! ham
  guido
--- 1,4 ----
! python
! eggy
! hamster
  guido
```

See *A command-line interface to difflib* for a more detailed example.

`difflib.`**`get_close_matches`**(*word*, *possibilities*, *n=3*, *cutoff=0.6*)

Return a list of the best "good enough" matches. *word* is a sequence for which close matches are desired (typically a string), and *possibilities* is a list of sequences against which to match *word* (typically a list of strings).

Optional argument *n* (default `3`) is the maximum number of close matches to return; *n* must be greater than `0`.

Optional argument *cutoff* (default `0.6`) is a float in the range [0, 1]. Possibilities that don't score at least that similar to *word* are ignored.

The best (no more than *n*) matches among the possibilities are returned in a list, sorted by similarity score, most similar first.

```
>>> get_close_matches('appel', ['ape', 'apple', 'peach', 'puppy'])
['apple', 'ape']
>>> import keyword
>>> get_close_matches('wheel', keyword.kwlist)
['while']
>>> get_close_matches('pineapple', keyword.kwlist)
[]
>>> get_close_matches('accept', keyword.kwlist)
['except']
```

`difflib.`**`ndiff`**(*a*, *b*, *linejunk=None*, *charjunk=IS_CHARACTER_JUNK*)

Compare *a* and *b* (lists of strings); return a *`Differ`*-style delta (a *generator* generating the delta lines).

Optional keyword parameters *linejunk* and *charjunk* are filtering functions (or `None`):

linejunk: A function that accepts a single string argument, and returns true if the string is junk, or false if not. The default is `None`. There is also a module-level function *`IS_LINE_JUNK()`*, which filters out lines without visible characters, except for at most one pound character (`'#'`) – however the underlying *`SequenceMatcher`* class does a dynamic analysis of which lines are so frequent as to constitute noise, and this usually works better than using this function.

charjunk: A function that accepts a character (a string of length 1), and returns if the character is junk, or false if not. The default is module-level function *`IS_CHARACTER_JUNK()`*, which filters out whitespace characters (a blank or tab; it's a bad idea to include newline in this!).

```
>>> diff = ndiff('one\ntwo\nthree\n'.splitlines(keepends=True),
...              'ore\ntree\nemu\n'.splitlines(keepends=True))
>>> print(''.join(diff), end="")
- one
?  ^
+ ore
?  ^
- two
```

(continua na próxima página)

(continuação da página anterior)

```
- three
?  -
+ tree
+ emu
```

difflib.**restore**(*sequence*, *which*)

Return one of the two sequences that generated a delta.

Given a *sequence* produced by *Differ.compare()* or *ndiff()*, extract lines originating from file 1 or 2 (parameter *which*), stripping off line prefixes.

Exemplo:

```
>>> diff = ndiff('one\ntwo\nthree\n'.splitlines(keepends=True),
...              'ore\ntree\nemu\n'.splitlines(keepends=True))
>>> diff = list(diff) # materialize the generated delta into a list
>>> print(''.join(restore(diff, 1)), end="")
one
two
three
>>> print(''.join(restore(diff, 2)), end="")
ore
tree
emu
```

difflib.**unified_diff**(*a*, *b*, *fromfile=''*, *tofile=''*, *fromfiledate=''*, *tofiledate=''*, *n=3*, *lineterm='\n'*)

Compare *a* and *b* (lists of strings); return a delta (a *generator* generating the delta lines) in unified diff format.

Unified diffs are a compact way of showing just the lines that have changed plus a few lines of context. The changes are shown in an inline style (instead of separate before/after blocks). The number of context lines is set by *n* which defaults to three.

By default, the diff control lines (those with `---`, `+++`, or `@@`) are created with a trailing newline. This is helpful so that inputs created from *io.IOBase.readlines()* result in diffs that are suitable for use with *io.IOBase.writelines()* since both the inputs and outputs have trailing newlines.

For inputs that do not have trailing newlines, set the *lineterm* argument to `""` so that the output will be uniformly newline free.

The unified diff format normally has a header for filenames and modification times. Any or all of these may be specified using strings for *fromfile*, *tofile*, *fromfiledate*, and *tofiledate*. The modification times are normally expressed in the ISO 8601 format. If not specified, the strings default to blanks.

```
>>> s1 = ['bacon\n', 'eggs\n', 'ham\n', 'guido\n']
>>> s2 = ['python\n', 'eggy\n', 'hamster\n', 'guido\n']
>>> sys.stdout.writelines(unified_diff(s1, s2, fromfile='before.py', tofile=
↪'after.py'))
--- before.py
+++ after.py
@@ -1,4 +1,4 @@
-bacon
-eggs
-ham
+python
+eggy
+hamster
 guido
```

See *A command-line interface to difflib* for a more detailed example.

difflib.**diff_bytes**(*dfunc*, *a*, *b*, *fromfile=b''*, *tofile=b''*, *fromfiledate=b''*, *tofiledate=b''*, *n=3*, *lineterm=b'\n'*)

Compare *a* and *b* (lists of bytes objects) using *dfunc*; yield a sequence of delta lines (also bytes) in the format returned by *dfunc*. *dfunc* must be a callable, typically either *unified_diff()* or *context_diff()*.

Allows you to compare data with unknown or inconsistent encoding. All inputs except *n* must be bytes objects, not str. Works by losslessly converting all inputs (except *n*) to str, and calling `dfunc(a, b, fromfile, tofile, fromfiledate, tofiledate, n, lineterm)`. The output of *dfunc* is then converted back to bytes, so the delta lines that you receive have the same unknown/inconsistent encodings as *a* and *b*.

Adicionado na versão 3.5.

difflib.**IS_LINE_JUNK**(*line*)

Return `True` for ignorable lines. The line *line* is ignorable if *line* is blank or contains a single `'#'`, otherwise it is not ignorable. Used as a default for parameter *linejunk* in *ndiff()* in older versions.

difflib.**IS_CHARACTER_JUNK**(*ch*)

Return `True` for ignorable characters. The character *ch* is ignorable if *ch* is a space or tab, otherwise it is not ignorable. Used as a default for parameter *charjunk* in *ndiff()*.

Ver também

Pattern Matching: The Gestalt Approach
Discussion of a similar algorithm by John W. Ratcliff and D. E. Metzener. This was published in Dr. Dobb's Journal in July, 1988.

6.4.1 SequenceMatcher Objects

The *SequenceMatcher* class has this constructor:

class difflib.**SequenceMatcher**(*isjunk=None*, *a=''*, *b=''*, *autojunk=True*)

Optional argument *isjunk* must be `None` (the default) or a one-argument function that takes a sequence element and returns true if and only if the element is "junk" and should be ignored. Passing `None` for *isjunk* is equivalent to passing `lambda x: False`; in other words, no elements are ignored. For example, pass:

```
lambda x: x in " \t"
```

if you're comparing lines as sequences of characters, and don't want to synch up on blanks or hard tabs.

The optional arguments *a* and *b* are sequences to be compared; both default to empty strings. The elements of both sequences must be *hashable*.

The optional argument *autojunk* can be used to disable the automatic junk heuristic.

Alterado na versão 3.2: Added the *autojunk* parameter.

SequenceMatcher objects get three data attributes: *bjunk* is the set of elements of *b* for which *isjunk* is `True`; *bpopular* is the set of non-junk elements considered popular by the heuristic (if it is not disabled); *b2j* is a dict mapping the remaining elements of *b* to a list of positions where they occur. All three are reset whenever *b* is reset with *set_seqs()* or *set_seq2()*.

Adicionado na versão 3.2: The *bjunk* and *bpopular* attributes.

SequenceMatcher objects have the following methods:

set_seqs(*a*, *b*)

Set the two sequences to be compared.

SequenceMatcher computes and caches detailed information about the second sequence, so if you want to compare one sequence against many sequences, use *set_seq2()* to set the commonly used sequence once and call *set_seq1()* repeatedly, once for each of the other sequences.

set_seq1(*a*)

Set the first sequence to be compared. The second sequence to be compared is not changed.

set_seq2(*b*)

Set the second sequence to be compared. The first sequence to be compared is not changed.

find_longest_match(*alo=0*, *ahi=None*, *blo=0*, *bhi=None*)

Find longest matching block in `a[alo:ahi]` and `b[blo:bhi]`.

If *isjunk* was omitted or `None`, *find_longest_match()* returns `(i, j, k)` such that `a[i:i+k]` is equal to `b[j:j+k]`, where `alo <= i <= i+k <= ahi` and `blo <= j <= j+k <= bhi`. For all `(i', j', k')` meeting those conditions, the additional conditions `k >= k'`, `i <= i'`, and if `i == i'`, `j <= j'` are also met. In other words, of all maximal matching blocks, return one that starts earliest in *a*, and of all those maximal matching blocks that start earliest in *a*, return the one that starts earliest in *b*.

```
>>> s = SequenceMatcher(None, " abcd", "abcd abcd")
>>> s.find_longest_match(0, 5, 0, 9)
Match(a=0, b=4, size=5)
```

If *isjunk* was provided, first the longest matching block is determined as above, but with the additional restriction that no junk element appears in the block. Then that block is extended as far as possible by matching (only) junk elements on both sides. So the resulting block never matches on junk except as identical junk happens to be adjacent to an interesting match.

Here's the same example as before, but considering blanks to be junk. That prevents `' abcd'` from matching the `' abcd'` at the tail end of the second sequence directly. Instead only the `'abcd'` can match, and matches the leftmost `'abcd'` in the second sequence:

```
>>> s = SequenceMatcher(lambda x: x==" ", " abcd", "abcd abcd")
>>> s.find_longest_match(0, 5, 0, 9)
Match(a=1, b=0, size=4)
```

If no blocks match, this returns `(alo, blo, 0)`.

This method returns a *named tuple* `Match(a, b, size)`.

Alterado na versão 3.9: Added default arguments.

get_matching_blocks()

Return list of triples describing non-overlapping matching subsequences. Each triple is of the form `(i, j, n)`, and means that `a[i:i+n] == b[j:j+n]`. The triples are monotonically increasing in *i* and *j*.

The last triple is a dummy, and has the value `(len(a), len(b), 0)`. It is the only triple with `n == 0`. If `(i, j, n)` and `(i', j', n')` are adjacent triples in the list, and the second is not the last triple in the list, then `i+n < i'` or `j+n < j'`; in other words, adjacent triples always describe non-adjacent equal blocks.

```
>>> s = SequenceMatcher(None, "abxcd", "abcd")
>>> s.get_matching_blocks()
[Match(a=0, b=0, size=2), Match(a=3, b=2, size=2), Match(a=5, b=4, size=0)]
```

get_opcodes()

Return list of 5-tuples describing how to turn *a* into *b*. Each tuple is of the form `(tag, i1, i2, j1, j2)`. The first tuple has `i1 == j1 == 0`, and remaining tuples have *i1* equal to the *i2* from the preceding tuple, and, likewise, *j1* equal to the previous *j2*.

The *tag* values are strings, with these meanings:

Valor	Significado
`'replace'`	`a[i1:i2]` should be replaced by `b[j1:j2]`.
`'delete'`	`a[i1:i2]` should be deleted. Note that `j1 == j2` in this case.
`'insert'`	`b[j1:j2]` should be inserted at `a[i1:i1]`. Note that `i1 == i2` in this case.
`'equal'`	`a[i1:i2] == b[j1:j2]` (the sub-sequences are equal).

Por exemplo:

```
>>> a = "qabxcd"
>>> b = "abycdf"
>>> s = SequenceMatcher(None, a, b)
>>> for tag, i1, i2, j1, j2 in s.get_opcodes():
...     print('{:7}   a[{}:{}] --> b[{}:{}] {!r:>8} --> {!r}'.format(
...         tag, i1, i2, j1, j2, a[i1:i2], b[j1:j2]))
delete    a[0:1] --> b[0:0]      'q' --> ''
equal     a[1:3] --> b[0:2]     'ab' --> 'ab'
replace   a[3:4] --> b[2:3]      'x' --> 'y'
equal     a[4:6] --> b[3:5]     'cd' --> 'cd'
insert    a[6:6] --> b[5:6]       '' --> 'f'
```

get_grouped_opcodes(*n=3*)

Return a *generator* of groups with up to *n* lines of context.

Starting with the groups returned by *get_opcodes()*, this method splits out smaller change clusters and eliminates intervening ranges which have no changes.

The groups are returned in the same format as *get_opcodes()*.

ratio()

Return a measure of the sequences' similarity as a float in the range [0, 1].

Where T is the total number of elements in both sequences, and M is the number of matches, this is 2.0*M / T. Note that this is `1.0` if the sequences are identical, and `0.0` if they have nothing in common.

This is expensive to compute if *get_matching_blocks()* or *get_opcodes()* hasn't already been called, in which case you may want to try *quick_ratio()* or *real_quick_ratio()* first to get an upper bound.

Nota

Caution: The result of a *ratio()* call may depend on the order of the arguments. For instance:

```
>>> SequenceMatcher(None, 'tide', 'diet').ratio()
0.25
>>> SequenceMatcher(None, 'diet', 'tide').ratio()
0.5
```

quick_ratio()

Return an upper bound on *ratio()* relatively quickly.

real_quick_ratio()

Return an upper bound on *ratio()* very quickly.

The three methods that return the ratio of matching to total characters can give different results due to differing levels of approximation, although *quick_ratio()* and *real_quick_ratio()* are always at least as large as *ratio()*:

```
>>> s = SequenceMatcher(None, "abcd", "bcde")
>>> s.ratio()
0.75
>>> s.quick_ratio()
0.75
>>> s.real_quick_ratio()
1.0
```

6.4.2 SequenceMatcher Examples

This example compares two strings, considering blanks to be "junk":

```
>>> s = SequenceMatcher(lambda x: x == " ",
...                     "private Thread currentThread;",
...                     "private volatile Thread currentThread;")
```

ratio() returns a float in [0, 1], measuring the similarity of the sequences. As a rule of thumb, a *ratio()* value over 0.6 means the sequences are close matches:

```
>>> print(round(s.ratio(), 3))
0.866
```

If you're only interested in where the sequences match, *get_matching_blocks()* is handy:

```
>>> for block in s.get_matching_blocks():
...     print("a[%d] and b[%d] match for %d elements" % block)
a[0] and b[0] match for 8 elements
a[8] and b[17] match for 21 elements
a[29] and b[38] match for 0 elements
```

Note that the last tuple returned by *get_matching_blocks()* is always a dummy, `(len(a), len(b), 0)`, and this is the only case in which the last tuple element (number of elements matched) is `0`.

If you want to know how to change the first sequence into the second, use *get_opcodes()*:

```
>>> for opcode in s.get_opcodes():
...     print("%6s a[%d:%d] b[%d:%d]" % opcode)
 equal a[0:8] b[0:8]
insert a[8:8] b[8:17]
 equal a[8:29] b[17:38]
```

Ver também

- The *get_close_matches()* function in this module which shows how simple code building on *SequenceMatcher* can be used to do useful work.
- Simple version control recipe for a small application built with *SequenceMatcher*.

6.4.3 Differ Objects

Note that *Differ*-generated deltas make no claim to be **minimal** diffs. To the contrary, minimal diffs are often counter-intuitive, because they synch up anywhere possible, sometimes accidental matches 100 pages apart. Restricting synch points to contiguous matches preserves some notion of locality, at the occasional cost of producing a longer diff.

The *Differ* class has this constructor:

class difflib.**Differ**(*linejunk=None*, *charjunk=None*)

Optional keyword parameters *linejunk* and *charjunk* are for filter functions (or `None`):

linejunk: A function that accepts a single string argument, and returns true if the string is junk. The default is `None`, meaning that no line is considered junk.

charjunk: A function that accepts a single character argument (a string of length 1), and returns true if the character is junk. The default is `None`, meaning that no character is considered junk.

These junk-filtering functions speed up matching to find differences and do not cause any differing lines or characters to be ignored. Read the description of the *find_longest_match()* method's *isjunk* parameter for an explanation.

Differ objects are used (deltas generated) via a single method:

compare(*a*, *b*)

Compare two sequences of lines, and generate the delta (a sequence of lines).

Each sequence must contain individual single-line strings ending with newlines. Such sequences can be obtained from the *readlines()* method of file-like objects. The delta generated also consists of newline-terminated strings, ready to be printed as-is via the *writelines()* method of a file-like object.

6.4.4 Differ Example

This example compares two texts. First we set up the texts, sequences of individual single-line strings ending with newlines (such sequences can also be obtained from the *readlines()* method of file-like objects):

```
>>> text1 = '''  1. Beautiful is better than ugly.
...   2. Explicit is better than implicit.
...   3. Simple is better than complex.
...   4. Complex is better than complicated.
... '''.splitlines(keepends=True)
>>> len(text1)
4
>>> text1[0][-1]
'\n'
>>> text2 = '''  1. Beautiful is better than ugly.
...   3.   Simple is better than complex.
...   4. Complicated is better than complex.
...   5. Flat is better than nested.
... '''.splitlines(keepends=True)
```

Next we instantiate a Differ object:

```
>>> d = Differ()
```

Note that when instantiating a *Differ* object we may pass functions to filter out line and character "junk." See the *Differ()* constructor for details.

Finally, we compare the two:

```
>>> result = list(d.compare(text1, text2))
```

`result` is a list of strings, so let's pretty-print it:

```
>>> from pprint import pprint
>>> pprint(result)
['    1. Beautiful is better than ugly.\n',
 '-   2. Explicit is better than implicit.\n',
 '-   3. Simple is better than complex.\n',
 '+   3.   Simple is better than complex.\n',
```

(continua na próxima página)

(continuação da página anterior)

```
 '?     ++\n',
 '-   4. Complex is better than complicated.\n',
 '?            ^                     ---- ^\n',
 '+   4. Complicated is better than complex.\n',
 '?           ++++ ^                      ^\n',
 '+   5. Flat is better than nested.\n']
```

As a single multi-line string it looks like this:

```
>>> import sys
>>> sys.stdout.writelines(result)
    1. Beautiful is better than ugly.
-   2. Explicit is better than implicit.
-   3. Simple is better than complex.
+   3.   Simple is better than complex.
?     ++
-   4. Complex is better than complicated.
?            ^                     ---- ^
+   4. Complicated is better than complex.
?           ++++ ^                      ^
+   5. Flat is better than nested.
```

6.4.5 A command-line interface to difflib

This example shows how to use difflib to create a `diff`-like utility.

```
""" Command line interface to difflib.py providing diffs in four formats:

* ndiff:    lists every line and highlights interline changes.
* context:  highlights clusters of changes in a before/after format.
* unified:  highlights clusters of changes in an inline format.
* html:     generates side by side comparison with change highlights.

"""

import sys, os, difflib, argparse
from datetime import datetime, timezone

def file_mtime(path):
    t = datetime.fromtimestamp(os.stat(path).st_mtime,
                               timezone.utc)
    return t.astimezone().isoformat()

def main():

    parser = argparse.ArgumentParser()
    parser.add_argument('-c', action='store_true', default=False,
                        help='Produce a context format diff (default)')
    parser.add_argument('-u', action='store_true', default=False,
                        help='Produce a unified format diff')
    parser.add_argument('-m', action='store_true', default=False,
                        help='Produce HTML side by side diff '
                             '(can use -c and -l in conjunction)')
    parser.add_argument('-n', action='store_true', default=False,
                        help='Produce a ndiff format diff')
    parser.add_argument('-l', '--lines', type=int, default=3,
```

(continua na próxima página)

(continuação da página anterior)

```
                        help='Set number of context lines (default 3)')
    parser.add_argument('fromfile')
    parser.add_argument('tofile')
    options = parser.parse_args()

    n = options.lines
    fromfile = options.fromfile
    tofile = options.tofile

    fromdate = file_mtime(fromfile)
    todate = file_mtime(tofile)
    with open(fromfile) as ff:
        fromlines = ff.readlines()
    with open(tofile) as tf:
        tolines = tf.readlines()

    if options.u:
        diff = difflib.unified_diff(fromlines, tolines, fromfile, tofile, fromdate,
↪ todate, n=n)
    elif options.n:
        diff = difflib.ndiff(fromlines, tolines)
    elif options.m:
        diff = difflib.HtmlDiff().make_file(fromlines,tolines,fromfile,tofile,
↪context=options.c,numlines=n)
    else:
        diff = difflib.context_diff(fromlines, tolines, fromfile, tofile, fromdate,
↪ todate, n=n)

    sys.stdout.writelines(diff)

if __name__ == '__main__':
    main()
```

6.4.6 ndiff example

This example shows how to use *difflib.ndiff()*.

```
"""ndiff [-q] file1 file2
    or
ndiff (-r1 | -r2) < ndiff_output > file1_or_file2

Print a human-friendly file difference report to stdout.  Both inter-
and intra-line differences are noted.  In the second form, recreate file1
(-r1) or file2 (-r2) on stdout, from an ndiff report on stdin.

In the first form, if -q ("quiet") is not specified, the first two lines
of output are

-: file1
+: file2

Each remaining line begins with a two-letter code:

    "- "    line unique to file1
    "+ "    line unique to file2
    "  "    line common to both files
```

(continua na próxima página)

(continuação da página anterior)

```
    "? "    line not present in either input file

Lines beginning with "? " attempt to guide the eye to intraline
differences, and were not present in either input file.  These lines can be
confusing if the source files contain tab characters.

The first file can be recovered by retaining only lines that begin with
"  " or "- ", and deleting those 2-character prefixes; use ndiff with -r1.

The second file can be recovered similarly, but by retaining only "  " and
"+ " lines; use ndiff with -r2; or, on Unix, the second file can be
recovered by piping the output through

    sed -n '/^[+ ] /s/^..//p'
"""

__version__ = 1, 7, 0

import difflib, sys

def fail(msg):
    out = sys.stderr.write
    out(msg + "\n\n")
    out(__doc__)
    return 0

# open a file & return the file object; gripe and return 0 if it
# couldn't be opened
def fopen(fname):
    try:
        return open(fname)
    except IOError as detail:
        return fail("couldn't open " + fname + ": " + str(detail))

# open two files & spray the diff to stdout; return false iff a problem
def fcompare(f1name, f2name):
    f1 = fopen(f1name)
    f2 = fopen(f2name)
    if not f1 or not f2:
        return 0

    a = f1.readlines(); f1.close()
    b = f2.readlines(); f2.close()
    for line in difflib.ndiff(a, b):
        print(line, end=' ')

    return 1

# crack args (sys.argv[1:] is normal) & compare;
# return false iff a problem

def main(args):
    import getopt
    try:
        opts, args = getopt.getopt(args, "qr:")
    except getopt.error as detail:
```

(continua na próxima página)

(continuação da página anterior)

```
        return fail(str(detail))
    noisy = 1
    qseen = rseen = 0
    for opt, val in opts:
        if opt == "-q":
            qseen = 1
            noisy = 0
        elif opt == "-r":
            rseen = 1
            whichfile = val
    if qseen and rseen:
        return fail("can't specify both -q and -r")
    if rseen:
        if args:
            return fail("no args allowed with -r option")
        if whichfile in ("1", "2"):
            restore(whichfile)
            return 1
        return fail("-r value must be 1 or 2")
    if len(args) != 2:
        return fail("need 2 filename args")
    f1name, f2name = args
    if noisy:
        print('-:', f1name)
        print('+:', f2name)
    return fcompare(f1name, f2name)

# read ndiff output from stdin, and print file1 (which=='1') or
# file2 (which=='2') to stdout

def restore(which):
    restored = difflib.restore(sys.stdin.readlines(), which)
    sys.stdout.writelines(restored)

if __name__ == '__main__':
    main(sys.argv[1:])
```

6.5 `textwrap` — Quebra automática e preenchimento de texto

Código-fonte: Lib/textwrap.py

O módulo *`textwrap`* fornece algumas funções convenientes, assim como *`TextWrapper`*, a classe que faz todo o trabalho. Se você estiver apenas preenchendo ou fazendo quebra de linha de uma ou duas strings, as funções de conveniência deverão ser boas o suficiente; caso contrário, você deve usar uma instância de *`TextWrapper`* para eficiência.

textwrap.**wrap**(*text*, *width=70*, ***, *initial_indent=''*, *subsequent_indent=''*, *expand_tabs=True*, *replace_whitespace=True*, *fix_sentence_endings=False*, *break_long_words=True*, *drop_whitespace=True*, *break_on_hyphens=True*, *tabsize=8*, *max_lines=None*, *placeholder=' [...]'*)

Quebra do parágrafo único em *text* (uma string) para que cada linha tenha no máximo *width* caracteres. Retorna uma lista de linhas de saída, sem novas linhas ao final.

Argumentos nomeados opcionais correspondem aos atributos de instância de *`TextWrapper`*, documentados abaixo.

Veja o método `TextWrapper.wrap()` para detalhes adicionais sobre como `wrap()` se comporta.

`textwrap.`**`fill`**(*text*, *width=70*, ***, *initial_indent=''*, *subsequent_indent=''*, *expand_tabs=True*, *replace_whitespace=True*, *fix_sentence_endings=False*, *break_long_words=True*, *drop_whitespace=True*, *break_on_hyphens=True*, *tabsize=8*, *max_lines=None*, *placeholder=' [...]'*)

Quebra o parágrafo único em *text* e retorna uma única string contendo o parágrafo quebrado. `fill()` é uma abreviação de

```
"\n".join(wrap(text, ...))
```

Em particular, `fill()` aceita exatamente os mesmos argumentos nomeados que `wrap()`.

`textwrap.`**`shorten`**(*text*, *width*, ***, *fix_sentence_endings=False*, *break_long_words=True*, *break_on_hyphens=True*, *placeholder=' [...]'*)

Recolhe e trunca o texto *text* fornecido para caber na largura *width* fornecida.

Primeiro, o espaço em branco em *text* é recolhido (todos os espaços em branco são substituídos por espaços simples). Se o resultado couber em *width*, ele será retornado. Caso contrário, palavras suficientes serão eliminadas do final para que as palavras restantes mais o *placeholder* caibam em *largura*:

```
>>> textwrap.shorten("Olá  mundo!", width=12)
'Olá mundo!'
>>> textwrap.shorten("Olá  mundo!", width=11)
'Olá [...]'
>>> textwrap.shorten("Olá mundo", width=10, placeholder="...")
'Olá...'
```

Argumentos nomeados opcionais correspondem aos atributos de instância de `TextWrapper`, documentados abaixo. Observe que o espaço em branco é recolhido antes do texto ser passado para a função `TextWrapper fill()`, alterando assim o valor de `tabsize`, `expand_tabs`, `drop_whitespace` e `replace_whitespace` não terão efeito.

Adicionado na versão 3.4.

`textwrap.`**`dedent`**(*text*)

Remove qualquer espaço em branco inicial em comum de toda linha em *text*.

Isso pode ser usado para alinhar strings entre aspas triplas com a borda esquerda da tela, enquanto ainda as apresenta no código-fonte em formato com indentação.

Observe que tabulações e espaços são tratados como espaços em branco, mas não são iguais: as linhas `" hello"` e `"\thello"` são consideradas como não tendo espaços em branco iniciais comuns.

Linhas contendo apenas espaços em branco são ignoradas na entrada e normalizadas para um único caractere de nova linha na saída.

Por exemplo:

```
def test():
    # fim da primeira linha com \ para evitar uma linha vazia!
    s = '''\
    olá
      mundo
    '''
    print(repr(s))          # exibe '    olá\n      mundo\n    '
    print(repr(dedent(s)))  # exibe 'olá\n  mundo\n'
```

`textwrap.`**`indent`**(*text*, *prefix*, *predicate=None*)

Adiciona *prefix* ao início das linhas selecionadas em *text*.

As linhas são separadas chamando `text.splitlines(True)`.

Por padrão, *prefix* é adicionado a todas as linhas que não consistem apenas em espaços em branco (incluindo quaisquer finais de linha).

Por exemplo:

```
>>> s = 'olá\n\n \nmundo'
>>> indent(s, '  ')
'  olá\n\n \n  mundo'
```

O argumento opcional *predicate* pode ser usado para controlar quais linhas têm indentação. Por exemplo, é fácil adicionar *prefix* até mesmo em linhas vazias e apenas com espaços em branco:

```
>>> print(indent(s, '+ ', lambda line: True))
+ olá
+
+
+ mundo
```

Adicionado na versão 3.3.

wrap(), *fill()* e *shorten()* funcionam criando uma instância *TextWrapper* e chamando um único método nela. Essa instância não é reutilizada, então para aplicações que processam muitas strings de texto usando *wrap()* e/ou *fill()*, pode ser mais eficiente criar seu próprio objeto *TextWrapper*.

O texto é preferencialmente colocado em espaços em branco e logo após os hífens nas palavras hifenizadas; somente então palavras longas serão quebradas se necessário, a menos que *TextWrapper.break_long_words* seja definido como falso.

class textwrap.**TextWrapper**(***kwargs*)

O construtor *TextWrapper* aceita vários argumentos nomeados opcionais. Cada argumento nomeado corresponde a um atributo de instância, por exemplo:

```
wrapper = TextWrapper(initial_indent="* ")
```

é o mesmo que

```
wrapper = TextWrapper()
wrapper.initial_indent = "* "
```

Você pode reutilizar o mesmo objeto *TextWrapper* muitas vezes e pode alterar qualquer uma de suas opções através de atribuição direta a atributos de instância entre os usos.

Os atributos de instância *TextWrapper* (e argumentos nomeados para o construtor) são os seguintes:

width

(padrão: `70`) O comprimento máximo das linhas quebradas. Contanto que não existam palavras individuais no texto de entrada maiores que *width*, *TextWrapper* garante que nenhuma linha de saída será maior que *width* caracteres.

expand_tabs

(padrão: `True`) Se verdadeiro, então todos os caracteres de tabulação em *text* serão expandidos para espaços usando o método *expandtabs()* de *text*.

tabsize

(padrão: `8`) Se *expand_tabs* for verdadeiro, então todos os caracteres de tabulação em *text* serão expandidos para zero ou mais espaços, dependendo da coluna atual e do tamanho da tabulação fornecido.

Adicionado na versão 3.3.

replace_whitespace

(padrão: `True`) Se verdadeiro, após a expansão da tabulação, mas antes da quebra, o método *wrap()* substituirá cada caractere de espaço em branco por um único espaço. Os caracteres de espaço em branco

substituídos são os seguintes: tabulação, nova linha, tabulação vertical, feed de formulário e retorno de carro (`'\t\n\v\f\r'`).

> **Nota**
>
> Se *expand_tabs* for falso e *replace_whitespace* for verdadeiro, cada caractere de tabulação será substituído por um único espaço, que *não* é o mesmo que expansão de tabulação.

> **Nota**
>
> Se *replace_whitespace* for falso, novas linhas podem aparecer no meio de uma linha e causar uma saída estranha. Por esta razão, o texto deve ser dividido em parágrafos (usando *str.splitlines()* ou similar) que são agrupados separadamente.

drop_whitespace

(padrão: `True`) Se verdadeiro, os espaços em branco no início e no final de cada linha (após quebra automática, mas antes do recuo) são eliminados. O espaço em branco no início do parágrafo, entretanto, não será eliminado se não houver espaço em branco após ele. Se o espaço em branco eliminado ocupar uma linha inteira, a linha inteira será eliminada.

initial_indent

(padrão: `''`) String que será anexada à primeira linha da saída com quebra de linha. Conta para o comprimento da primeira linha. A string vazia não recebe indentação.

subsequent_indent

(padrão: `''`) String que será anexada a todas as linhas da saída com quebra de linha, exceto a primeira. Conta para o comprimento de cada linha, exceto a primeira.

fix_sentence_endings

(padrão: `False`) Se verdadeiro, *TextWrapper* tenta detectar finais de sentenças e garantir que as sentenças sejam sempre separadas por exatamente dois espaços. Isso geralmente é desejado para texto em fonte monoespaçada. No entanto, o algoritmo de detecção de frase é imperfeito: ele presume que o final de uma frase consiste em uma letra minúscula seguida por uma de `'.'`, `'!'` ou `'?'`, possivelmente seguido por um de `'"'` ou `"'"`, seguido por um espaço. Um problema com este algoritmo é que ele é incapaz de detectar a diferença entre "Dr." em

```
[...] Dr. Frankenstein's monster [...]
```

e "Spot." em

```
[...] See Spot. See Spot run [...]
```

fix_sentence_endings é falso por padrão

Como o algoritmo de detecção de frase depende de `string.lowercase` para a definição de "letra minúscula" e de uma convenção de usar dois espaços após um ponto final para separar frases na mesma linha, ele é específico para textos em inglês.

break_long_words

(padrão: `True`) Se verdadeiro, palavras maiores que *width* serão quebradas para garantir que nenhuma linha seja maior que *width*. Se for falso, palavras longas não serão quebradas e algumas linhas poderão ser maiores que *width*. (Palavras longas serão colocadas sozinhas em uma linha, a fim de minimizar o valor pelo qual *width* é excedido.)

break_on_hyphens

(padrão: `True`) Se verdadeiro, a quebra ocorrerá preferencialmente em espaços em branco e logo após hífens em palavras compostas, como é habitual em inglês. Se for falso, apenas os espaços em branco serão considerados locais potencialmente bons para quebras de linha, mas você precisa definir

break_long_words como falso se quiser palavras verdadeiramente inseparáveis. O comportamento padrão nas versões anteriores era sempre permitir a quebra de palavras hifenizadas.

max_lines

(padrão: `None`) Se não for `None`, então a saída conterá no máximo *max_lines* linhas, com *placeholder* aparecendo no final da saída.

Adicionado na versão 3.4.

placeholder

(padrão: `' [...]'`) String que aparecerá no final do texto de saída se ele tiver sido truncado.

Adicionado na versão 3.4.

TextWrapper também fornece alguns métodos públicos, análogos às funções de conveniência em nível de módulo:

wrap(*text*)

Quebra o parágrafo único em *text* (uma string) para que cada linha tenha no máximo *width* caracteres. Todas as opções de quebra de linha são obtidas dos atributos da instância *TextWrapper*. Retorna uma lista de linhas de saída, sem novas linhas finais. Se a saída agrupada não tiver conteúdo, a lista retornada estará vazia.

fill(*text*)

Quebra o parágrafo único em *text* e retorna uma única string contendo o parágrafo quebrado.

6.6 unicodedata — Banco de dados de Unicode

Este módulo fornece acesso ao Banco de Dados de Caracteres Unicode (UCD), que define as propriedades de todos os caracteres Unicode. Os dados contidos neste banco de dados são compilados a partir do UCD versão 16.0.0.

O módulo utiliza os mesmos nomes e símbolos definidos no Anexo 44 do Padrão Unicode, "Unicode Character Database". Ele define as seguintes funções:

Ver também

O unicode-howto para mais informações sobre Unicode e como usar este módulo.

unicodedata.**lookup**(*name*)

Procurar personagem pelo nome. Se um personagem com o nome fornecido for encontrado, retornar o personagem correspondente. Se não for encontrado, *KeyError* é levantada. Por exemplo:

```
>>> unicodedata.lookup('LEFT CURLY BRACKET')
'{'
```

Os caracteres retornados por esta função são os mesmos produzidos pela sequência de escape `\N` em literais de string. Por exemplo:

```
>>> unicodedata.lookup('MIDDLE DOT') == '\N{MIDDLE DOT}'
True
```

Alterado na versão 3.3: Foi adicionado suporte para apelidos de nomes[1] e sequências nomeadas[2].

[1] https://www.unicode.org/Public/16.0.0/ucd/NameAliases.txt
[2] https://www.unicode.org/Public/16.0.0/ucd/NamedSequences.txt

unicodedata.**name**(*chr*, *default=None*, */*)

Retorna o nome atribuído ao caractere *chr* como uma string. Se nenhum nome for definido, *default* será retornado ou, se não for fornecido, `ValueError` será levantada. Por exemplo:

```
>>> unicodedata.name('½')
'VULGAR FRACTION ONE HALF'
>>> unicodedata.name('\uFFFF', 'fallback')
'fallback'
```

unicodedata.**decimal**(*chr*, *default=None*, */*)

Retorna o valor decimal atribuído ao caractere *chr* como inteiro. Se nenhum valor for definido, *default* será retornado ou, se não for fornecido, `ValueError` será levantada. Por exemplo:

```
>>> unicodedata.decimal('\N{ARABIC-INDIC DIGIT NINE}')
9
>>> unicodedata.decimal('\N{SUPERSCRIPT NINE}', -1)
-1
```

unicodedata.**digit**(*chr*, *default=None*, */*)

Retorna o valor de dígito atribuído ao caractere *chr* como inteiro. Se nenhum valor for definido, *default* será retornado ou, se não for fornecido, `ValueError` será levantada.

```
>>> unicodedata.digit('\N{SUPERSCRIPT NINE}')
9
```

unicodedata.**numeric**(*chr*, *default=None*, */*)

Retorna o valor numérico atribuído ao caractere *chr* como float. Se nenhum valor for definido, *default* será retornado ou, se não for fornecido, `ValueError` será levantada.

```
>>> unicodedata.numeric('½')
0.5
```

unicodedata.**category**(*chr*)

Returns the general category assigned to the character *chr* as string. General category names consist of two letters. See the General Category Values section of the Unicode Character Database documentation for a list of category codes. For example:

```
>>> unicodedata.category('A')  # 'L'etter, 'u'ppercase
'Lu'
```

unicodedata.**bidirectional**(*chr*)

Returns the bidirectional class assigned to the character *chr* as string. If no such value is defined, an empty string is returned. See the Bidirectional Class Values section of the Unicode Character Database documentation for a list of bidirectional codes. For example:

```
>>> unicodedata.bidirectional('\N{ARABIC-INDIC DIGIT SEVEN}') # 'A'rabic, 'N
↪'umber
'AN'
```

unicodedata.**combining**(*chr*)

Returns the canonical combining class assigned to the character *chr* as integer. Returns `0` if no combining class is defined. See the Canonical Combining Class Values section of the Unicode Character Database for more information.

unicodedata.**east_asian_width**(*chr*)

Returns the east asian width assigned to the character *chr* as string. For a list of widths and or more information, see the Unicode Standard Annex #11.

unicodedata.**mirrored**(*chr*)

Returns the mirrored property assigned to the character *chr* as integer. Returns `1` if the character has been identified as a "mirrored" character in bidirectional text, `0` otherwise. For example:

```
>>> unicodedata.mirrored('>')
1
```

unicodedata.**decomposition**(*chr*)

Returns the character decomposition mapping assigned to the character *chr* as string. An empty string is returned in case no such mapping is defined. For example:

```
>>> unicodedata.decomposition('Ã')
'0041 0303'
```

unicodedata.**normalize**(*form*, *unistr*)

Return the normal form *form* for the Unicode string *unistr*. Valid values for *form* are 'NFC', 'NFKC', 'NFD', and 'NFKD'.

The Unicode standard defines various normalization forms of a Unicode string, based on the definition of canonical equivalence and compatibility equivalence. In Unicode, several characters can be expressed in various way. For example, the character U+00C7 (LATIN CAPITAL LETTER C WITH CEDILLA) can also be expressed as the sequence U+0043 (LATIN CAPITAL LETTER C) U+0327 (COMBINING CEDILLA).

For each character, there are two normal forms: normal form C and normal form D. Normal form D (NFD) is also known as canonical decomposition, and translates each character into its decomposed form. Normal form C (NFC) first applies a canonical decomposition, then composes pre-combined characters again.

In addition to these two forms, there are two additional normal forms based on compatibility equivalence. In Unicode, certain characters are supported which normally would be unified with other characters. For example, U+2160 (ROMAN NUMERAL ONE) is really the same thing as U+0049 (LATIN CAPITAL LETTER I). However, it is supported in Unicode for compatibility with existing character sets (for example, gb2312).

The normal form KD (NFKD) will apply the compatibility decomposition, that is, replace all compatibility characters with their equivalents. The normal form KC (NFKC) first applies the compatibility decomposition, followed by the canonical composition.

Even if two unicode strings are normalized and look the same to a human reader, if one has combining characters and the other doesn't, they may not compare equal.

unicodedata.**is_normalized**(*form*, *unistr*)

Return whether the Unicode string *unistr* is in the normal form *form*. Valid values for *form* are 'NFC', 'NFKC', 'NFD', and 'NFKD'.

Adicionado na versão 3.8.

In addition, the module exposes the following constant:

unicodedata.**unidata_version**

The version of the Unicode database used in this module.

unicodedata.**ucd_3_2_0**

This is an object that has the same methods as the entire module, but uses the Unicode database version 3.2 instead, for applications that require this specific version of the Unicode database (such as IDNA).

6.7 `stringprep` — Preparação de string para internet

Código-fonte: Lib/stringprep.py

Ao identificar coisas (como nomes de host) na internet, muitas vezes é necessário comparar tais identificações para "igualdade". Exatamente como essa comparação é executada pode depender do domínio da aplicação, por exemplo, se deve ser insensível a maiúsculas e minúsculas ou não. Também pode ser necessário restringir as possíveis identificações, para permitir apenas identificações consistindo de caracteres "imprimíveis".

RFC 3454 define um procedimento para "preparar" strings Unicode em protocolos de internet. Antes de passar strings para o fio, elas são processadas com o procedimento de preparação, após o qual elas têm uma certa forma normalizada. O RFC define um conjunto de tabelas, que podem ser combinadas em perfis. Cada perfil deve definir quais tabelas ele usa, e quais outras partes opcionais do procedimento `stringprep` são parte do perfil. Um exemplo de um perfil `stringprep` é `nameprep`, que é usado para nomes de domínio internacionalizados.

O módulo *`stringprep`* expõe apenas as tabelas do **RFC 3454**. Como essas tabelas seriam muito grandes para representar como dicionários ou listas, o módulo usa o banco de dados de caracteres Unicode internamente. O código-fonte do módulo em si foi gerado usando o utilitário `mkstringprep.py`.

Como resultado, essas tabelas são expostas como funções, não como estruturas de dados. Existem dois tipos de tabelas no RFC: conjuntos e mapeamentos. Para um conjunto, *`stringprep`* fornece a "função característica", ou seja, uma função que retorna `True` se o parâmetro fizer parte do conjunto. Para mapeamentos, ele fornece a função de mapeamento: dada a chave, ele retorna o valor associado. Abaixo está uma lista de todas as funções disponíveis no módulo.

`stringprep.`**`in_table_a1`**(*code*)

Determina se *code* está na tabela A.1 (pontos de código não atribuídos no Unicode 3.2).

`stringprep.`**`in_table_b1`**(*code*)

Determina se *code* está na tabela B.1 (comumente mapeado para nada).

`stringprep.`**`map_table_b2`**(*code*)

Retorna o valor mapeado para *code* de acordo com a tabela B.2 (mapeamento para dobramento de casos usado com NFKC).

`stringprep.`**`map_table_b3`**(*code*)

Retorna o valor mapeado para *code* de acordo com a tabela B.3 (mapeamento para dobramento de casos usado com nenhuma normalização).

`stringprep.`**`in_table_c11`**(*code*)

Determina se *code* está na tabela C.1.1 (caracteres de espaço ASCII).

`stringprep.`**`in_table_c12`**(*code*)

Determina se *code* está na tabela C.1.2 (caracteres de espaço não-ASCII).

`stringprep.`**`in_table_c11_c12`**(*code*)

Determina se *code* está na tabela C.1 (caracteres de espaço, união de C.1.1 e C.1.2).

`stringprep.`**`in_table_c21`**(*code*)

Determina se *code* está na tabela C.2.1 (caracteres de controle ASCII).

`stringprep.`**`in_table_c22`**(*code*)

Determina se *code* está na tabela C.2.2 (caracteres de controle não-ASCII).

`stringprep.`**`in_table_c21_c22`**(*code*)

Determina se *code* está na tabela C.2 (caracteres de controle, união de C.2.1 e C.2.2).

`stringprep.`**`in_table_c3`**(*code*)

Determina se *code* está na tabela C.3 (uso privado).

`stringprep.`**`in_table_c4`**(*code*)

Determina se *code* está na tabela C.4 (pontos de código não-caracteres).

`stringprep.`**`in_table_c5`**(*code*)

Determina se *code* está na tabela C.5 (códigos substitutos).

`stringprep.`**`in_table_c6`**(*code*)

Determina se *code* está na tabela C.6 (inapropriado para texto simples).

`stringprep.`**`in_table_c7`**(*code*)

Determina se *code* está na tabela C.7 (inapropriado para para representação canônica).

`stringprep.`**`in_table_c8`**(*code*)

Determina se *code* está na tabela C.8 (alteram propriedades de exibição ou estão descontinuadas).

`stringprep.`**`in_table_c9`**(*code*)

Determina se *code* está na tabela C.9 (caracteres de tag).

`stringprep.`**`in_table_d1`**(*code*)

Determina se *code* está na tabela D.1 (Caracteres com propriedade bidirecional "R" ou "AL").

`stringprep.`**`in_table_d2`**(*code*)

Determina se *code* está na tabela D.2 (Caracteres com propriedade bidirecional "L").

6.8 `readline` — Interface para o GNU readline

O módulo *`readline`* define uma série de funções para facilitar o autocomplemento e leitura/gravação de arquivos históricos do interpretador Python. Este módulo pode ser usado diretamente ou através do módulo *`rlcompleter`*, que provê o autocomplemento de identificadores Python no prompt interativo. As configurações feitas usando este módulo afetam o comportamento do prompt interativo do interpretador e dos prompts oferecidos pela função embutida *`input()`*.

As combinações de teclas do Readline podem ser configuradas através de um arquivo de inicialização, normalmente `.inputrc` em seu diretório inicial. Consulte Readline Init File no manual do GNU Readline para obter informações sobre o formato e construções permitidas desse arquivo e os recursos da biblioteca do Readline em geral.

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em *plataformas móveis* ou *plataformas WebAssembly*.

Nota

A API da biblioteca subjacente do Readline pode ser implementada pela biblioteca `editline` (`libedit`) em vez do GNU readline. No macOS, o módulo *`readline`* detecta qual biblioteca está sendo usada em tempo de execução.

O arquivo de configuração do `editline` é diferente daquele do GNU readline. Se você carregar strings de configuração programaticamente, você pode usar *`backend`* para determinar qual biblioteca está sendo usada.

Se você usar a emulação de readline do `editline`/`libedit` no macOS, o arquivo de inicialização localizado em seu diretório inicial será denominado `.editrc`. Por exemplo, o seguinte conteúdo em `~/.editrc` ativará os atalhos de teclado *vi* e o autocomplemento de TAB:

```
python:bind -v
python:bind ^I rl_complete
```

Observe também que bibliotecas diferentes podem usar formatos de arquivo de histórico diferentes. Ao alternar a biblioteca subjacente, os arquivos de histórico existentes podem se tornar inutilizáveis.

`readline.`**`backend`**

O nome da biblioteca Readline subjacente que está sendo usada, seja `"readline"` ou `"editline"`.

Adicionado na versão 3.13.

6.8.1 Arquivo init

As seguintes funções estão relacionadas ao arquivo init e à configuração do usuário:

readline.**parse_and_bind**(*string*)

Executa a linha de init fornecida no argumento *string*. Isso chama `rl_parse_and_bind()` na biblioteca subjacente.

readline.**read_init_file**([*filename*])

Executa um arquivo de inicialização readline. O nome de arquivo padrão é o último usado. Isso chama `rl_read_init_file()` na biblioteca subjacente. Ele levanta um *evento de auditoria* `open` com o nome do arquivo, se fornecido, e `"<readline_init_file>"` caso contrário, independentemente de qual arquivo a biblioteca resolve.

Alterado na versão 3.14: O evento de auditoria foi adicionado.

6.8.2 Buffer de linha

As seguintes funções operam no buffer de linha:

readline.**get_line_buffer**()

Retorna o conteúdo atual do buffer de linha (`rl_line_buffer` na biblioteca subjacente).

readline.**insert_text**(*string*)

Insere texto no buffer de linha na posição do cursor. Isso chama `rl_insert_text()` na biblioteca subjacente, mas ignora o valor de retorno.

readline.**redisplay**()

Altera o que é exibido na tela para refletir o conteúdo atual do buffer de linha. Isso chama `rl_redisplay()` na biblioteca subjacente.

6.8.3 Arquivo de histórico

As seguintes funções operam em um arquivo histórico:

readline.**read_history_file**([*filename*])

Carrega um arquivo de histórico do readline e anexa-o à lista de histórico. O nome do arquivo padrão é `~/.history`. Isso chama `read_history()` na biblioteca subjacente e levanta um *evento de auditoria* `open` com o nome de arquivo se dado e, do contrário, `"~/.history"`.

Alterado na versão 3.14: O evento de auditoria foi adicionado.

readline.**write_history_file**([*filename*])

Salva a lista de histórico para um arquivo de histórico do readline, sobrescrevendo qualquer arquivo existente. O nome do arquivo padrão é `~/.history`. Isso chama `write_history()` na biblioteca subjacente e levanta um *evento de auditoria* `open` com o nome de arquivo se dado e, do contrário, `"~/.history"`.

Alterado na versão 3.14: O evento de auditoria foi adicionado.

readline.**append_history_file**(*nelements*[, *filename*])

Anexa os últimos *nelements* itens do histórico a um arquivo. O nome do arquivo padrão é `~/.history`. O arquivo já deve existir. Isso chama `append_history()` na biblioteca subjacente. Esta função só existe se o Python foi compilado para uma versão da biblioteca que a suporta. Ela levanta um *evento de auditoria* `open` com o nome de arquivo se dado e, do contrário, `"~/.history"`.

Adicionado na versão 3.5.

Alterado na versão 3.14: O evento de auditoria foi adicionado.

readline.**get_history_length**()

readline.**set_history_length**(*length*)

Define ou retorna o número desejado de linhas para salvar no arquivo de histórico. A função *write_history_file()* usa este valor para truncar o arquivo de histórico, chamando history_truncate_file() na biblioteca subjacente. Valores negativos implicam tamanho ilimitado do arquivo de histórico.

6.8.4 Lista de histórico

As seguintes funções operam em uma lista de histórico global:

readline.**clear_history**()

Limpa o histórico atual. Isso chama clear_history() na biblioteca subjacente. A função Python só existe se o Python foi compilado para uma versão da biblioteca que a suporta.

readline.**get_current_history_length**()

Retorna o número de itens atualmente no histórico. (Isso é diferente de *get_history_length()*, que retorna o número máximo de linhas que serão gravadas em um arquivo de histórico.)

readline.**get_history_item**(*index*)

Retorna o conteúdo atual do item do histórico em *index*. O índice do item é baseado em um. Isso chama history_get() na biblioteca subjacente.

readline.**remove_history_item**(*pos*)

Remove o item de histórico especificado por sua posição do histórico. A posição conta a partir de zero. Isso chama remove_history() na biblioteca subjacente.

readline.**replace_history_item**(*pos*, *line*)

Substitui o item de histórico especificado pela sua posição por *linha*. A posição conta a partir do zero. Isso chama replace_history_entry() na biblioteca subjacente.

readline.**add_history**(*line*)

Acrescenta *line* ao buffer do histórico, como se fosse a última linha digitada. Isso chama add_history() na biblioteca subjacente.

readline.**set_auto_history**(*enabled*)

Habilita ou desabilita chamadas automáticas para add_history() ao ler a entrada via readline. O argumento *enabled* deve ser um valor booleano que, quando verdadeiro, ativa o histórico automático e, quando falso, desativa o histórico automático.

Adicionado na versão 3.6.

O histórico automático está ativado por padrão e as alterações não persistem em várias sessões.

6.8.5 Ganchos de inicialização

readline.**set_startup_hook**([*function*])

Define ou remove a função invocada pelo retorno de chamada rl_startup_hook da biblioteca subjacente. Se *function* for especificada, ela será usada como a nova função de gancho; se omitido ou None, qualquer função já instalada será removida. O gancho é chamado sem argumentos antes de readline imprimir o primeiro prompt.

readline.**set_pre_input_hook**([*function*])

Define ou remove a função invocada pelo retorno de chamada rl_pre_input_hook da biblioteca subjacente. Se *function* for especificada, ela será usada como a nova função de gancho; se omitida ou None, qualquer função já instalada será removida. O gancho é chamado sem argumentos após a impressão do primeiro prompt e pouco antes de readline começar a ler os caracteres de entrada. Esta função só existe se o Python foi compilado para uma versão da biblioteca que a suporta.

6.8.6 Autocomplemento

As funções a seguir estão relacionadas à implementação de uma função personalizada de autocomplemento ou *completion*, em inglês, de palavras. Isso normalmente é operado pela tecla Tab e pode sugerir e completar automaticamente uma palavra que está sendo digitada. Por padrão, Readline está configurado para ser usado por `rlcompleter` para completar identificadores Python para o interpretador interativo. Se o módulo `readline` for usado com um autocomplemento personalizado, um conjunto diferente de delimitadores de palavras deverá ser definido.

`readline.`**`set_completer`**`(`[*function*]`)`

Define ou remove a função de autocomplemento. Se *function* for especificada, ela será usada como a nova função de autocomplemento; se omitido ou `None`, qualquer função de autocomplemento já instalada será removida. A função de autocomplemento é chamada como `function(text, state)`, para *state* em `0`, `1`, `2`, …, até retornar um valor que não seja string. Deve retornar o próximo autocomplemento possível começando com *text*.

A função de autocomplemento instalada é invocada pelo retorno de chamada *entry_func* passado para `rl_completion_matches()` na biblioteca subjacente. A string *text* vem do primeiro parâmetro para o retorno de chamada `rl_attempted_completion_function` da biblioteca subjacente.

`readline.`**`get_completer`**`()`

Obtém a função de autocomplemento ou `None` se nenhuma função de autocomplemento tiver sido definida.

`readline.`**`get_completion_type`**`()`

Obtém o tipo de autocomplemento que está sendo tentado. Isso retorna a variável `rl_completion_type` na biblioteca subjacente como um número inteiro.

`readline.`**`get_begidx`**`()`

`readline.`**`get_endidx`**`()`

Obtém o índice inicial ou final do escopo de autocomplemento. Esses índices são os argumentos *start* e *end* passados para o retorno de chamada `rl_attempted_completion_function` da biblioteca subjacente. Os valores podem ser diferentes no mesmo cenário de edição de entrada com base na implementação de C readline subjacente. Por exemplo, sabe-se que o libedit se comporta de maneira diferente do libreadline.

`readline.`**`set_completer_delims`**`(`*string*`)`

`readline.`**`get_completer_delims`**`()`

Define ou obtém os delimitadores de palavras para autocomplemento. Estes determinam o início da palavra a ser considerada para autocomplemento (o escopo de autocomplemento). Essas funções acessam a variável `rl_completer_word_break_characters` na biblioteca subjacente.

`readline.`**`set_completion_display_matches_hook`**`(`[*function*]`)`

Define ou remove a função de exibição de autocomplemento. Se *function* for especificada, ela será usada como a nova função de exibição de autocomplemento; se omitida ou `None`, qualquer função de exibição de autocomplemento já instalada será removida. Isso define ou limpa o retorno de chamada `rl_completion_display_matches_hook` na biblioteca subjacente. A função de exibição de autocomplemento é chamada como `function(substitution, [matches], longest_match_length)` uma vez que cada correspondência precisa ser exibida.

6.8.7 Exemplo

O exemplo a seguir demonstra como usar as funções de leitura e gravação de histórico do módulo `readline` para carregar e salvar automaticamente um arquivo de histórico chamado `.python_history` do diretório inicial do usuário. O código abaixo normalmente seria executado automaticamente durante sessões interativas do arquivo `PYTHONSTARTUP` do usuário.

```
import atexit
import os
import readline

histfile = os.path.join(os.path.expanduser("~"), ".python_history")
try:
```

(continua na próxima página)

(continuação da página anterior)

```
    readline.read_history_file(histfile)
    # comprimento padrão do histórico é -1 (infinito), que pode crescer sem␣
↪controle
    readline.set_history_length(1000)
except FileNotFoundError:
    pass

atexit.register(readline.write_history_file, histfile)
```

Na verdade, este código é executado automaticamente quando o Python é executado no modo interativo (veja *Configuração Readline*).

O exemplo a seguir atinge o mesmo objetivo, mas oferece suporte a sessões interativas simultâneas, anexando apenas o novo histórico.

```
import atexit
import os
import readline
histfile = os.path.join(os.path.expanduser("~"), ".python_history")

try:
    readline.read_history_file(histfile)
    h_len = readline.get_current_history_length()
except FileNotFoundError:
    open(histfile, 'wb').close()
    h_len = 0

def save(prev_h_len, histfile):
    new_h_len = readline.get_current_history_length()
    readline.set_history_length(1000)
    readline.append_history_file(new_h_len - prev_h_len, histfile)
atexit.register(save, h_len, histfile)
```

O exemplo a seguir estende a classe `code.InteractiveConsole` para prover salvamento/restauração do histórico.

```
import atexit
import code
import os
import readline

class HistoryConsole(code.InteractiveConsole):
    def __init__(self, locals=None, filename="<console>",
                 histfile=os.path.expanduser("~/.console-history")):
        code.InteractiveConsole.__init__(self, locals, filename)
        self.init_history(histfile)

    def init_history(self, histfile):
        readline.parse_and_bind("tab: complete")
        if hasattr(readline, "read_history_file"):
            try:
                readline.read_history_file(histfile)
            except FileNotFoundError:
                pass
            atexit.register(self.save_history, histfile)

    def save_history(self, histfile):
        readline.set_history_length(1000)
```

(continua na próxima página)

(continuação da página anterior)

```
        readline.write_history_file(histfile)
```

6.9 `rlcompleter` — Função de autocomplemento para GNU readline

Código-fonte: Lib/rlcompleter.py

O módulo `rlcompleter` define uma função de autocompletamento adequada para ser passada para *`set_completer()`* no módulo *`readline`*.

Quando este módulo é importado em uma plataforma Unix com o módulo *`readline`* disponível, uma instância da classe *`Completer`* é criada automaticamente e seu método *`complete()`* é definido como o *autocompletamento do readline*. O método fornece o autocompletamento de identificadores e palavras-chaves válidas do Python.

Exemplo:

```
>>> import rlcompleter
>>> import readline
>>> readline.parse_and_bind("tab: complete")
>>> readline. <TAB PRESSED>
readline.__doc__          readline.get_line_buffer(  readline.read_init_file(
readline.__file__         readline.insert_text(      readline.set_completer(
readline.__name__         readline.parse_and_bind(
>>> readline.
```

O módulo `rlcompleter` é projetado para uso com o modo interativo do Python. A menos que Python seja executado com a opção `-S`, o módulo é automaticamente importado e configurado (veja *Configuração Readline*).

Em plataformas sem *`readline`*, a classe *`Completer`* definida por este módulo ainda pode ser usada para propósitos personalizados.

class rlcompleter.**Completer**

Os objetos Completer têm o seguinte método:

complete(*text*, *state*)

Retorna o próximo completamento possível para *text*.

Quando chamado pelo módulo *`readline`*, este método é chamado sucessivamente com `state == 0, 1, 2, ...` até que o método retorne `None`.

Se chamado para *text* que não inclui um caractere de ponto (`'.'`), ele será completado a partir dos nomes atualmente definidos em *`__main__`*, *`builtins`* e palavras reservadas (conforme definido pelo módulo *`keyword`*).

Se chamado por um nome pontilhado, vai tentar avaliar qualquer coisa sem efeitos colaterais óbvios (as funções não serão avaliadas, mas pode levantar chamadas para `__getattr__()`) até a última parte e encontrar correspondências para o resto por meio da função *`dir()`*. Qualquer exceção levantada durante a avaliação da expressão é capturada, silenciada e *`None`* é retornado.

CAPÍTULO 7

Serviços de Dados Binários

Os módulos descritos neste capítulo fornecem algumas operações de serviços básicos para manipulação de dados binários. Outras operações sobre dados binários, especificamente em relação a formatos de arquivo e protocolos de rede, são descritas nas seções relevantes.

Algumas bibliotecas descritas em *Serviços de Processamento de Texto* também funcionam com formatos binários compatíveis com ASCII (por exemplo `re`) ou com todos os dados binários (por exemplo `difflib`).

Além disso, consulte a documentação dos tipos de dados binários embutidos do Python em *Tipos de sequência binária — bytes, bytearray, memoryview*.

7.1 `struct` — Interpreta bytes como dados binários empacotados

Código-fonte: Lib/struct.py

Este módulo converte entre valores Python e structs C representados como objetos `bytes` do Python. *Strings de formato* compactas descrevem as conversões pretendidas de/para valores Python. As funções e objetos do módulo podem ser usados para duas aplicações amplamente distintas, troca de dados com fontes externas (arquivos ou conexões de rede) ou transferência de dados entre a aplicação Python e a camada C.

> **Nota**
>
> Quando nenhum caractere de prefixo é fornecido, o modo nativo é o padrão. Ele empacota ou descompacta dados com base na plataforma e no compilador nos quais o interpretador Python foi construído. O resultado do empacotamento de uma dada estrutura C inclui bytes de preenchimento que mantêm o alinhamento adequado para os tipos C envolvidos; similarmente, o alinhamento é levado em conta ao desempacotar. Em contraste, ao comunicar dados entre fontes externas, o programador é responsável por definir a ordenação de bytes e o preenchimento entre os elementos. Veja *Byte Order, Size, and Alignment* para detalhes.

Várias funções `struct` (e métodos de `Struct`) recebem um argumento *buffer*. Isso se refere a objetos que implementam bufferobjects e fornecem um buffer legível ou de leitura-gravação. Os tipos mais comuns usados para esse propósito são `bytes` e `bytearray`, mas muitos outros tipos que podem ser vistos como uma matriz de bytes implementam o protocolo de buffer, para que possam ser lidos/preenchidos sem cópia adicional de um objeto `bytes`.

7.1.1 Funções e exceções

O módulo define as seguintes exceções e funções:

exception `struct.`**error**

Exceção levantada em várias ocasiões; argumento é uma string que descreve o que está errado.

`struct.`**pack**(*format*, *v1*, *v2*, ...)

Return a bytes object containing the values *v1*, *v2*, … packed according to the format string *format*. The arguments must match the values required by the format exactly.

`struct.`**pack_into**(*format*, *buffer*, *offset*, *v1*, *v2*, ...)

Pack the values *v1*, *v2*, … according to the format string *format* and write the packed bytes into the writable buffer *buffer* starting at position *offset*. Note that *offset* is a required argument.

`struct.`**unpack**(*format*, *buffer*)

Unpack from the buffer *buffer* (presumably packed by `pack(format, ...)`) according to the format string *format*. The result is a tuple even if it contains exactly one item. The buffer's size in bytes must match the size required by the format, as reflected by *calcsize()*.

`struct.`**unpack_from**(*format*, */*, *buffer*, *offset=0*)

Unpack from *buffer* starting at position *offset*, according to the format string *format*. The result is a tuple even if it contains exactly one item. The buffer's size in bytes, starting at position *offset*, must be at least the size required by the format, as reflected by *calcsize()*.

`struct.`**iter_unpack**(*format*, *buffer*)

Iteratively unpack from the buffer *buffer* according to the format string *format*. This function returns an iterator which will read equally sized chunks from the buffer until all its contents have been consumed. The buffer's size in bytes must be a multiple of the size required by the format, as reflected by *calcsize()*.

Each iteration yields a tuple as specified by the format string.

Adicionado na versão 3.4.

`struct.`**calcsize**(*format*)

Return the size of the struct (and hence of the bytes object produced by `pack(format, ...)`) corresponding to the format string *format*.

7.1.2 Format Strings

Format strings describe the data layout when packing and unpacking data. They are built up from *format characters*, which specify the type of data being packed/unpacked. In addition, special characters control the *byte order, size and alignment*. Each format string consists of an optional prefix character which describes the overall properties of the data and one or more format characters which describe the actual data values and padding.

Byte Order, Size, and Alignment

By default, C types are represented in the machine's native format and byte order, and properly aligned by skipping pad bytes if necessary (according to the rules used by the C compiler). This behavior is chosen so that the bytes of a packed struct correspond exactly to the memory layout of the corresponding C struct. Whether to use native byte ordering and padding or standard formats depends on the application.

Alternatively, the first character of the format string can be used to indicate the byte order, size and alignment of the packed data, according to the following table:

Caractere	Ordem de bytes	Tamanho	Alinhamento
`@`	nativo	nativo	nativo
`=`	nativo	padrão	nenhum
`<`	little-endian	padrão	nenhum
`>`	big-endian	padrão	nenhum
`!`	rede (= big-endian)	padrão	nenhum

Se o primeiro caractere não for um destes, `'@'` é presumido.

Nota

The number 1023 (`0x3ff` in hexadecimal) has the following byte representations:

- `03 ff` in big-endian (>)
- `ff 03` in little-endian (<)

Python example:

```
>>> import struct
>>> struct.pack('>h', 1023)
b'\x03\xff'
>>> struct.pack('<h', 1023)
b'\xff\x03'
```

Native byte order is big-endian or little-endian, depending on the host system. For example, Intel x86, AMD64 (x86-64), and Apple M1 are little-endian; IBM z and many legacy architectures are big-endian. Use *`sys.byteorder`* to check the endianness of your system.

Native size and alignment are determined using the C compiler's `sizeof` expression. This is always combined with native byte order.

Standard size depends only on the format character; see the table in the *Caracteres Formatados* section.

Note the difference between `'@'` and `'='`: both use native byte order, but the size and alignment of the latter is standardized.

The form `'!'` represents the network byte order which is always big-endian as defined in IETF RFC 1700.

There is no way to indicate non-native byte order (force byte-swapping); use the appropriate choice of `'<'` or `'>'`.

Notas:

(1) Padding is only automatically added between successive structure members. No padding is added at the beginning or the end of the encoded struct.

(2) No padding is added when using non-native size and alignment, e.g. with '<', '>', '=', and '!'.

(3) To align the end of a structure to the alignment requirement of a particular type, end the format with the code for that type with a repeat count of zero. See *Exemplos*.

Caracteres Formatados

Format characters have the following meaning; the conversion between C and Python values should be obvious given their types. The 'Standard size' column refers to the size of the packed value in bytes when using standard size; that is, when the format string starts with one of `'<'`, `'>'`, `'!'` or `'='`. When using native size, the size of the packed value is platform-dependent.

Formatação	Tipo em C	Tipo em Python	Standard size	Notas
x	pad byte	no value		(7)
c	char	bytes of length 1	1	
b	signed char	inteiro	1	(1), (2)
B	unsigned char	inteiro	1	(2)
?	_Bool	bool	1	(1)
h	short	inteiro	2	(2)
H	unsigned short	inteiro	2	(2)
i	int	inteiro	4	(2)
I	unsigned int	inteiro	4	(2)
l	long	inteiro	4	(2)
L	unsigned long	inteiro	4	(2)
q	long long	inteiro	8	(2)
Q	unsigned long long	inteiro	8	(2)
n	ssize_t	inteiro		(3)
N	size_t	inteiro		(3)
e	(6)	float	2	(4)
f	float	float	4	(4)
d	double	float	8	(4)
F	float complex	complexo	8	(10)
D	double complex	complexo	16	(10)
s	char[]	bytes		(9)
p	char[]	bytes		(8)
P	void*	inteiro		(5)

Alterado na versão 3.3: Added support for the `'n'` and `'N'` formats.

Alterado na versão 3.6: Added support for the `'e'` format.

Alterado na versão 3.14: Added support for the `'F'` and `'D'` formats.

Notas:

(1) The `'?'` conversion code corresponds to the `_Bool` type defined by C standards since C99. In standard mode, it is represented by one byte.

(2) When attempting to pack a non-integer using any of the integer conversion codes, if the non-integer has a `__index__()` method then that method is called to convert the argument to an integer before packing.

Alterado na versão 3.2: Added use of the `__index__()` method for non-integers.

(3) The `'n'` and `'N'` conversion codes are only available for the native size (selected as the default or with the `'@'` byte order character). For the standard size, you can use whichever of the other integer formats fits your application.

(4) For the `'f'`, `'d'` and `'e'` conversion codes, the packed representation uses the IEEE 754 binary32, binary64 or binary16 format (for `'f'`, `'d'` or `'e'` respectively), regardless of the floating-point format used by the platform.

(5) The `'P'` format character is only available for the native byte ordering (selected as the default or with the `'@'` byte order character). The byte order character `'='` chooses to use little- or big-endian ordering based on the host system. The struct module does not interpret this as native ordering, so the `'P'` format is not available.

(6) The IEEE 754 binary16 "half precision" type was introduced in the 2008 revision of the IEEE 754 standard. It has a sign bit, a 5-bit exponent and 11-bit precision (with 10 bits explicitly stored), and can represent numbers between approximately `6.1e-05` and `6.5e+04` at full precision. This type is not widely supported by C compilers: on a typical machine, an unsigned short can be used for storage, but not for math operations. See the Wikipedia page on the half-precision floating-point format for more information.

(7) When packing, `'x'` inserts one NUL byte.

(8) The `'p'` format character encodes a "Pascal string", meaning a short variable-length string stored in a *fixed number of bytes*, given by the count. The first byte stored is the length of the string, or 255, whichever is smaller. The bytes of the string follow. If the string passed in to *pack()* is too long (longer than the count minus 1), only the leading `count-1` bytes of the string are stored. If the string is shorter than `count-1`, it is padded with null bytes so that exactly count bytes in all are used. Note that for *unpack()*, the `'p'` format character consumes `count` bytes, but that the string returned can never contain more than 255 bytes.

(9) For the `'s'` format character, the count is interpreted as the length of the bytes, not a repeat count like for the other format characters; for example, `'10s'` means a single 10-byte string mapping to or from a single Python byte string, while `'10c'` means 10 separate one byte character elements (e.g., `cccccccccc`) mapping to or from ten different Python byte objects. (See *Exemplos* for a concrete demonstration of the difference.) If a count is not given, it defaults to 1. For packing, the string is truncated or padded with null bytes as appropriate to make it fit. For unpacking, the resulting bytes object always has exactly the specified number of bytes. As a special case, `'0s'` means a single, empty string (while `'0c'` means 0 characters).

(10) For the `'F'` and `'D'` format characters, the packed representation uses the IEEE 754 binary32 and binary64 format for components of the complex number, regardless of the floating-point format used by the platform. Note that complex types (`F` and `D`) are available unconditionally, despite complex types being an optional feature in C. As specified in the C11 standard, each complex type is represented by a two-element C array containing, respectively, the real and imaginary parts.

A format character may be preceded by an integral repeat count. For example, the format string `'4h'` means exactly the same as `'hhhh'`.

Whitespace characters between formats are ignored; a count and its format must not contain whitespace though.

When packing a value `x` using one of the integer formats (`'b'`, `'B'`, `'h'`, `'H'`, `'i'`, `'I'`, `'l'`, `'L'`, `'q'`, `'Q'`), if `x` is outside the valid range for that format then *struct.error* is raised.

Alterado na versão 3.1: Previously, some of the integer formats wrapped out-of-range values and raised *DeprecationWarning* instead of *struct.error*.

For the `'?'` format character, the return value is either *True* or *False*. When packing, the truth value of the argument object is used. Either 0 or 1 in the native or standard bool representation will be packed, and any non-zero value will be `True` when unpacking.

Exemplos

Nota

Native byte order examples (designated by the `'@'` format prefix or lack of any prefix character) may not match what the reader's machine produces as that depends on the platform and compiler.

Pack and unpack integers of three different sizes, using big endian ordering:

```
>>> from struct import *
>>> pack(">bhl", 1, 2, 3)
b'\x01\x00\x02\x00\x00\x00\x03'
>>> unpack('>bhl', b'\x01\x00\x02\x00\x00\x00\x03')
(1, 2, 3)
>>> calcsize('>bhl')
7
```

Attempt to pack an integer which is too large for the defined field:

```
>>> pack(">h", 99999)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
struct.error: 'h' format requires -32768 <= number <= 32767
```

Demonstrate the difference between `'s'` and `'c'` format characters:

```
>>> pack("@ccc", b'1', b'2', b'3')
b'123'
>>> pack("@3s", b'123')
b'123'
```

Unpacked fields can be named by assigning them to variables or by wrapping the result in a named tuple:

```
>>> record = b'raymond   \x32\x12\x08\x01\x08'
>>> name, serialnum, school, gradelevel = unpack('<10sHHb', record)

>>> from collections import namedtuple
>>> Student = namedtuple('Student', 'name serialnum school gradelevel')
>>> Student._make(unpack('<10sHHb', record))
Student(name=b'raymond   ', serialnum=4658, school=264, gradelevel=8)
```

The ordering of format characters may have an impact on size in native mode since padding is implicit. In standard mode, the user is responsible for inserting any desired padding. Note in the first `pack` call below that three NUL bytes were added after the packed `'#'` to align the following integer on a four-byte boundary. In this example, the output was produced on a little endian machine:

```
>>> pack('@ci', b'#', 0x12131415)
b'#\x00\x00\x00\x15\x14\x13\x12'
>>> pack('@ic', 0x12131415, b'#')
b'\x15\x14\x13\x12#'
>>> calcsize('@ci')
8
>>> calcsize('@ic')
5
```

The following format `'llh0l'` results in two pad bytes being added at the end, assuming the platform's longs are aligned on 4-byte boundaries:

```
>>> pack('@llh0l', 1, 2, 3)
b'\x00\x00\x00\x01\x00\x00\x00\x02\x00\x03\x00\x00'
```

Ver também

Module *array*
Packed binary storage of homogeneous data.

Módulo *json*
JSON encoder and decoder.

Módulo *pickle*
Python object serialization.

7.1.3 Applications

Two main applications for the *struct* module exist, data interchange between Python and C code within an application or another application compiled using the same compiler (*native formats*), and data interchange between applications using agreed upon data layout (*standard formats*). Generally speaking, the format strings constructed for these two domains are distinct.

Native Formats

When constructing format strings which mimic native layouts, the compiler and machine architecture determine byte ordering and padding. In such cases, the `@` format character should be used to specify native byte ordering and data sizes. Internal pad bytes are normally inserted automatically. It is possible that a zero-repeat format code will be needed at the end of a format string to round up to the correct byte boundary for proper alignment of consecutive chunks of data.

Consider these two simple examples (on a 64-bit, little-endian machine):

```
>>> calcsize('@lhl')
24
>>> calcsize('@llh')
18
```

Data is not padded to an 8-byte boundary at the end of the second format string without the use of extra padding. A zero-repeat format code solves that problem:

```
>>> calcsize('@llh0l')
24
```

The `'x'` format code can be used to specify the repeat, but for native formats it is better to use a zero-repeat format like `'0l'`.

By default, native byte ordering and alignment is used, but it is better to be explicit and use the `'@'` prefix character.

Standard Formats

When exchanging data beyond your process such as networking or storage, be precise. Specify the exact byte order, size, and alignment. Do not assume they match the native order of a particular machine. For example, network byte order is big-endian, while many popular CPUs are little-endian. By defining this explicitly, the user need not care about the specifics of the platform their code is running on. The first character should typically be `<` or `>` (or `!`). Padding is the responsibility of the programmer. The zero-repeat format character won't work. Instead, the user must explicitly add `'x'` pad bytes where needed. Revisiting the examples from the previous section, we have:

```
>>> calcsize('<qh6xq')
24
>>> pack('<qh6xq', 1, 2, 3) == pack('@lhl', 1, 2, 3)
True
>>> calcsize('@llh')
18
>>> pack('@llh', 1, 2, 3) == pack('<qqh', 1, 2, 3)
True
>>> calcsize('<qqh6x')
24
>>> calcsize('@llh0l')
24
>>> pack('@llh0l', 1, 2, 3) == pack('<qqh6x', 1, 2, 3)
True
```

The above results (executed on a 64-bit machine) aren't guaranteed to match when executed on different machines. For example, the examples below were executed on a 32-bit machine:

```
>>> calcsize('<qqh6x')
24
>>> calcsize('@llh0l')
12
>>> pack('@llh0l', 1, 2, 3) == pack('<qqh6x', 1, 2, 3)
False
```

7.1.4 Classes

The `struct` module also defines the following type:

class struct.**Struct**(*format*)

Return a new Struct object which writes and reads binary data according to the format string *format*. Creating a `Struct` object once and calling its methods is more efficient than calling module-level functions with the same format since the format string is only compiled once.

> **Nota**
>
> The compiled versions of the most recent format strings passed to the module-level functions are cached, so programs that use only a few format strings needn't worry about reusing a single `Struct` instance.

Compiled Struct objects support the following methods and attributes:

pack(*v1*, *v2*, ...)

Identical to the `pack()` function, using the compiled format. (`len(result)` will equal `size`.)

pack_into(*buffer*, *offset*, *v1*, *v2*, ...)

Identical to the `pack_into()` function, using the compiled format.

unpack(*buffer*)

Identical to the `unpack()` function, using the compiled format. The buffer's size in bytes must equal `size`.

unpack_from(*buffer*, *offset=0*)

Identical to the `unpack_from()` function, using the compiled format. The buffer's size in bytes, starting at position *offset*, must be at least `size`.

iter_unpack(*buffer*)

Identical to the `iter_unpack()` function, using the compiled format. The buffer's size in bytes must be a multiple of `size`.

Adicionado na versão 3.4.

format

The format string used to construct this Struct object.

Alterado na versão 3.7: The format string type is now `str` instead of `bytes`.

size

The calculated size of the struct (and hence of the bytes object produced by the `pack()` method) corresponding to `format`.

Alterado na versão 3.13: The *repr()* of structs has changed. It is now:

```
>>> Struct('i')
Struct('i')
```

7.2 codecs — Codec registry and base classes

Código-fonte: Lib/codecs.py

This module defines base classes for standard Python codecs (encoders and decoders) and provides access to the internal Python codec registry, which manages the codec and error handling lookup process. Most standard codecs are *text encodings*, which encode text to bytes (and decode bytes to text), but there are also codecs provided that encode text to text, and bytes to bytes. Custom codecs may encode and decode between arbitrary types, but some module features are restricted to be used specifically with *text encodings* or with codecs that encode to `bytes`.

The module defines the following functions for encoding and decoding with any codec:

codecs.**encode**(*obj*, *encoding='utf-8'*, *errors='strict'*)

Encodes *obj* using the codec registered for *encoding*.

Errors may be given to set the desired error handling scheme. The default error handler is `'strict'` meaning that encoding errors raise `ValueError` (or a more codec specific subclass, such as `UnicodeEncodeError`). Refer to *Codec Base Classes* for more information on codec error handling.

codecs.**decode**(*obj*, *encoding='utf-8'*, *errors='strict'*)

Decodes *obj* using the codec registered for *encoding*.

Errors may be given to set the desired error handling scheme. The default error handler is `'strict'` meaning that decoding errors raise `ValueError` (or a more codec specific subclass, such as `UnicodeDecodeError`). Refer to *Codec Base Classes* for more information on codec error handling.

codecs.**charmap_build**(*string*)

Return a mapping suitable for encoding with a custom single-byte encoding. Given a `str` *string* of up to 256 characters representing a decoding table, returns either a compact internal mapping object `EncodingMap` or a `dictionary` mapping character ordinals to byte values. Raises a `TypeError` on invalid input.

The full details for each codec can also be looked up directly:

codecs.**lookup**(*encoding*, */*)

Looks up the codec info in the Python codec registry and returns a `CodecInfo` object as defined below.

Encodings are first looked up in the registry's cache. If not found, the list of registered search functions is scanned. If no `CodecInfo` object is found, a `LookupError` is raised. Otherwise, the `CodecInfo` object is stored in the cache and returned to the caller.

class codecs.**CodecInfo**(*encode*, *decode*, *streamreader=None*, *streamwriter=None*, *incrementalencoder=None*, *incrementaldecoder=None*, *name=None*)

Codec details when looking up the codec registry. The constructor arguments are stored in attributes of the same name:

name

The name of the encoding.

encode

decode

The stateless encoding and decoding functions. These must be functions or methods which have the same interface as the `encode()` and `decode()` methods of Codec instances (see *Codec Interface*). The functions or methods are expected to work in a stateless mode.

incrementalencoder

incrementaldecoder

Incremental encoder and decoder classes or factory functions. These have to provide the interface defined by the base classes `IncrementalEncoder` and `IncrementalDecoder`, respectively. Incremental codecs can maintain state.

streamwriter

streamreader

Stream writer and reader classes or factory functions. These have to provide the interface defined by the base classes `StreamWriter` and `StreamReader`, respectively. Stream codecs can maintain state.

To simplify access to the various codec components, the module provides these additional functions which use `lookup()` for the codec lookup:

codecs.**getencoder**(*encoding*)

Look up the codec for the given encoding and return its encoder function.

Raises a `LookupError` in case the encoding cannot be found.

codecs.**getdecoder**(*encoding*)

Look up the codec for the given encoding and return its decoder function.

Raises a *LookupError* in case the encoding cannot be found.

codecs.**getincrementalencoder**(*encoding*)

Look up the codec for the given encoding and return its incremental encoder class or factory function.

Raises a *LookupError* in case the encoding cannot be found or the codec doesn't support an incremental encoder.

codecs.**getincrementaldecoder**(*encoding*)

Look up the codec for the given encoding and return its incremental decoder class or factory function.

Raises a *LookupError* in case the encoding cannot be found or the codec doesn't support an incremental decoder.

codecs.**getreader**(*encoding*)

Look up the codec for the given encoding and return its *StreamReader* class or factory function.

Raises a *LookupError* in case the encoding cannot be found.

codecs.**getwriter**(*encoding*)

Look up the codec for the given encoding and return its *StreamWriter* class or factory function.

Raises a *LookupError* in case the encoding cannot be found.

Custom codecs are made available by registering a suitable codec search function:

codecs.**register**(*search_function*, */*)

Register a codec search function. Search functions are expected to take one argument, being the encoding name in all lower case letters with hyphens and spaces converted to underscores, and return a *CodecInfo* object. In case a search function cannot find a given encoding, it should return `None`.

Alterado na versão 3.9: Hyphens and spaces are converted to underscore.

codecs.**unregister**(*search_function*, */*)

Unregister a codec search function and clear the registry's cache. If the search function is not registered, do nothing.

Adicionado na versão 3.10.

While the builtin *open()* and the associated *io* module are the recommended approach for working with encoded text files, this module provides additional utility functions and classes that allow the use of a wider range of codecs when working with binary files:

codecs.**open**(*filename*, *mode='r'*, *encoding=None*, *errors='strict'*, *buffering=-1*)

Open an encoded file using the given *mode* and return an instance of *StreamReaderWriter*, providing transparent encoding/decoding. The default file mode is `'r'`, meaning to open the file in read mode.

> **Nota**
>
> If *encoding* is not `None`, then the underlying encoded files are always opened in binary mode. No automatic conversion of `'\n'` is done on reading and writing. The *mode* argument may be any binary mode acceptable to the built-in *open()* function; the `'b'` is automatically added.

encoding specifies the encoding which is to be used for the file. Any encoding that encodes to and decodes from bytes is allowed, and the data types supported by the file methods depend on the codec used.

errors may be given to define the error handling. It defaults to `'strict'` which causes a *ValueError* to be raised in case an encoding error occurs.

buffering has the same meaning as for the built-in *open()* function. It defaults to -1 which means that the default buffer size will be used.

Alterado na versão 3.11: O modo `'U'` foi removido.

Descontinuado desde a versão 3.14: *codecs.open()* has been superseded by *open()*.

codecs.**EncodedFile**(*file*, *data_encoding*, *file_encoding=None*, *errors='strict'*)

Return a *StreamRecoder* instance, a wrapped version of *file* which provides transparent transcoding. The original file is closed when the wrapped version is closed.

Data written to the wrapped file is decoded according to the given *data_encoding* and then written to the original file as bytes using *file_encoding*. Bytes read from the original file are decoded according to *file_encoding*, and the result is encoded using *data_encoding*.

If *file_encoding* is not given, it defaults to *data_encoding*.

errors may be given to define the error handling. It defaults to `'strict'`, which causes *ValueError* to be raised in case an encoding error occurs.

codecs.**iterencode**(*iterator*, *encoding*, *errors='strict'*, ***kwargs*)

Uses an incremental encoder to iteratively encode the input provided by *iterator*. *iterator* must yield *str* objects. This function is a *generator*. The *errors* argument (as well as any other keyword argument) is passed through to the incremental encoder.

This function requires that the codec accept text *str* objects to encode. Therefore it does not support bytes--to-bytes encoders such as `base64_codec`.

codecs.**iterdecode**(*iterator*, *encoding*, *errors='strict'*, ***kwargs*)

Uses an incremental decoder to iteratively decode the input provided by *iterator*. *iterator* must yield *bytes* objects. This function is a *generator*. The *errors* argument (as well as any other keyword argument) is passed through to the incremental decoder.

This function requires that the codec accept *bytes* objects to decode. Therefore it does not support text-to-text encoders such as `rot_13`, although `rot_13` may be used equivalently with *iterencode()*.

codecs.**readbuffer_encode**(*buffer*, *errors=None*, */*)

Return a *tuple* containing the raw bytes of *buffer*, a buffer-compatible object or *str* (encoded to UTF-8 before processing), and their length in bytes.

The *errors* argument is ignored.

```
>>> codecs.readbuffer_encode(b"Zito")
(b'Zito', 4)
```

The module also provides the following constants which are useful for reading and writing to platform dependent files:

codecs.**BOM**

codecs.**BOM_BE**

codecs.**BOM_LE**

codecs.**BOM_UTF8**

codecs.**BOM_UTF16**

codecs.**BOM_UTF16_BE**

codecs.**BOM_UTF16_LE**

codecs.**BOM_UTF32**

codecs.**BOM_UTF32_BE**

codecs.**BOM_UTF32_LE**

These constants define various byte sequences, being Unicode byte order marks (BOMs) for several encodings. They are used in UTF-16 and UTF-32 data streams to indicate the byte order used, and in UTF-8 as a Unicode signature. *BOM_UTF16* is either *BOM_UTF16_BE* or *BOM_UTF16_LE* depending on the platform's native byte order, *BOM* is an alias for *BOM_UTF16*, *BOM_LE* for *BOM_UTF16_LE* and *BOM_BE* for *BOM_UTF16_BE*. The others represent the BOM in UTF-8 and UTF-32 encodings.

7.2.1 Codec Base Classes

The *codecs* module defines a set of base classes which define the interfaces for working with codec objects, and can also be used as the basis for custom codec implementations.

Each codec has to define four interfaces to make it usable as codec in Python: stateless encoder, stateless decoder, stream reader and stream writer. The stream reader and writers typically reuse the stateless encoder/decoder to implement the file protocols. Codec authors also need to define how the codec will handle encoding and decoding errors.

Error Handlers

To simplify and standardize error handling, codecs may implement different error handling schemes by accepting the *errors* string argument:

```
>>> 'German ß, ♬'.encode(encoding='ascii', errors='backslashreplace')
b'German \\xdf, \\u266c'
>>> 'German ß, ♬'.encode(encoding='ascii', errors='xmlcharrefreplace')
b'German &#223;, &#9836;'
```

The following error handlers can be used with all Python *Standard Encodings* codecs:

Valor	Significado
`'strict'`	Raise *UnicodeError* (or a subclass), this is the default. Implemented in *strict_errors()*.
`'ignore'`	Ignore the malformed data and continue without further notice. Implemented in *ignore_errors()*.
`'replace'`	Replace with a replacement marker. On encoding, use `?` (ASCII character). On decoding, use `&#65533;` (U+FFFD, the official REPLACEMENT CHARACTER). Implemented in *replace_errors()*.
`'backslashreplace'`	Replace with backslashed escape sequences. On encoding, use hexadecimal form of Unicode code point with formats `\xhh` `\uxxxx` `\Uxxxxxxxx`. On decoding, use hexadecimal form of byte value with format `\xhh`. Implemented in *backslashreplace_errors()*.
`'surrogateescape'`	On decoding, replace byte with individual surrogate code ranging from `U+DC80` to `U+DCFF`. This code will then be turned back into the same byte when the `'surrogateescape'` error handler is used when encoding the data. (See **PEP 383** for more.)

The following error handlers are only applicable to encoding (within *text encodings*):

Valor	Significado
`'xmlcharref`	Replace with XML/HTML numeric character reference, which is a decimal form of Unicode code point with format `&#num;`. Implemented in *xmlcharrefreplace_errors()*.
`'namereplac`	Replace with `\N{...}` escape sequences, what appears in the braces is the Name property from Unicode Character Database. Implemented in *namereplace_errors()*.

In addition, the following error handler is specific to the given codecs:

Valor	Codecs	Significado
`'surrog`	utf-8, utf-16, utf-32, utf--16-be, utf-16-le, utf-32--be, utf-32-le	Allow encoding and decoding surrogate code point (`U+D800` - `U+DFFF`) as normal code point. Otherwise these codecs treat the presence of surrogate code point in *str* as an error.

Adicionado na versão 3.1: The `'surrogateescape'` and `'surrogatepass'` error handlers.

Alterado na versão 3.4: The `'surrogatepass'` error handler now works with utf-16* and utf-32* codecs.

Adicionado na versão 3.5: The `'namereplace'` error handler.

Alterado na versão 3.5: The `'backslashreplace'` error handler now works with decoding and translating.

The set of allowed values can be extended by registering a new named error handler:

`codecs.`**`register_error`**(*name*, *error_handler*, `/`)

Register the error handling function *error_handler* under the name *name*. The *error_handler* argument will be called during encoding and decoding in case of an error, when *name* is specified as the errors parameter.

For encoding, *error_handler* will be called with a *UnicodeEncodeError* instance, which contains information about the location of the error. The error handler must either raise this or a different exception, or return a tuple with a replacement for the unencodable part of the input and a position where encoding should continue. The replacement may be either *str* or *bytes*. If the replacement is bytes, the encoder will simply copy them into the output buffer. If the replacement is a string, the encoder will encode the replacement. Encoding continues on original input at the specified position. Negative position values will be treated as being relative to the end of the input string. If the resulting position is out of bound an *IndexError* will be raised.

Decoding and translating works similarly, except *UnicodeDecodeError* or *UnicodeTranslateError* will be passed to the handler and that the replacement from the error handler will be put into the output directly.

Previously registered error handlers (including the standard error handlers) can be looked up by name:

`codecs.`**`lookup_error`**(*name*, `/`)

Return the error handler previously registered under the name *name*.

Raises a *LookupError* in case the handler cannot be found.

The following standard error handlers are also made available as module level functions:

`codecs.`**`strict_errors`**(*exception*)

Implementa a tratativa de erro `'strict'`.

Each encoding or decoding error raises a *UnicodeError*.

`codecs.`**`ignore_errors`**(*exception*)

Implementa a tratativa de erro `'ignore'`.

Malformed data is ignored; encoding or decoding is continued without further notice.

`codecs.`**`replace_errors`**(*exception*)

Implementa a tratativa de erro `'replace'`.

Substitutes `?` (ASCII character) for encoding errors or `&#65533;` (U+FFFD, the official REPLACEMENT CHARACTER) for decoding errors.

`codecs.`**`backslashreplace_errors`**(*exception*)

Implementa a tratativa de erro `'backslashreplace'`.

Malformed data is replaced by a backslashed escape sequence. On encoding, use the hexadecimal form of Unicode code point with formats `\x`*hh* `\u`*xxxx* `\U`*xxxxxxxx*. On decoding, use the hexadecimal form of byte value with format `\x`*hh*.

Alterado na versão 3.5: Works with decoding and translating.

`codecs.`**`xmlcharrefreplace_errors`**(*exception*)

Implements the `'xmlcharrefreplace'` error handling (for encoding within *text encoding* only).

The unencodable character is replaced by an appropriate XML/HTML numeric character reference, which is a decimal form of Unicode code point with format `&#`*num*`;` .

`codecs.`**`namereplace_errors`**(*exception*)

Implements the `'namereplace'` error handling (for encoding within *text encoding* only).

The unencodable character is replaced by a `\N{...}` escape sequence. The set of characters that appear in the braces is the Name property from Unicode Character Database. For example, the German lowercase letter `'ß'` will be converted to byte sequence `\N{LATIN SMALL LETTER SHARP S}`.

Adicionado na versão 3.5.

Stateless Encoding and Decoding

The base *Codec* class defines these methods which also define the function interfaces of the stateless encoder and decoder:

class codecs.**Codec**

encode(*input*, *errors='strict'*)

Encodes the object *input* and returns a tuple (output object, length consumed). For instance, *text encoding* converts a string object to a bytes object using a particular character set encoding (e.g., `cp1252` or `iso-8859-1`).

The *errors* argument defines the error handling to apply. It defaults to `'strict'` handling.

The method may not store state in the *Codec* instance. Use *StreamWriter* for codecs which have to keep state in order to make encoding efficient.

The encoder must be able to handle zero length input and return an empty object of the output object type in this situation.

decode(*input*, *errors='strict'*)

Decodes the object *input* and returns a tuple (output object, length consumed). For instance, for a *text encoding*, decoding converts a bytes object encoded using a particular character set encoding to a string object.

For text encodings and bytes-to-bytes codecs, *input* must be a bytes object or one which provides the read-only buffer interface – for example, buffer objects and memory mapped files.

The *errors* argument defines the error handling to apply. It defaults to `'strict'` handling.

The method may not store state in the *Codec* instance. Use *StreamReader* for codecs which have to keep state in order to make decoding efficient.

The decoder must be able to handle zero length input and return an empty object of the output object type in this situation.

Incremental Encoding and Decoding

The *IncrementalEncoder* and *IncrementalDecoder* classes provide the basic interface for incremental encoding and decoding. Encoding/decoding the input isn't done with one call to the stateless encoder/decoder function, but with multiple calls to the *encode()*/*decode()* method of the incremental encoder/decoder. The incremental encoder/decoder keeps track of the encoding/decoding process during method calls.

The joined output of calls to the *encode()*/*decode()* method is the same as if all the single inputs were joined into one, and this input was encoded/decoded with the stateless encoder/decoder.

IncrementalEncoder Objects

The *IncrementalEncoder* class is used for encoding an input in multiple steps. It defines the following methods which every incremental encoder must define in order to be compatible with the Python codec registry.

class codecs.**IncrementalEncoder**(*errors='strict'*)

Constructor for an *IncrementalEncoder* instance.

All incremental encoders must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.

The *IncrementalEncoder* may implement different error handling schemes by providing the *errors* keyword argument. See *Error Handlers* for possible values.

The *errors* argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch between different error handling strategies during the lifetime of the `IncrementalEncoder` object.

encode(*object*, *final=False*)

Encodes *object* (taking the current state of the encoder into account) and returns the resulting encoded object. If this is the last call to `encode()` *final* must be true (the default is false).

reset()

Reset the encoder to the initial state. The output is discarded: call `.encode(object, final=True)`, passing an empty byte or text string if necessary, to reset the encoder and to get the output.

getstate()

Return the current state of the encoder which must be an integer. The implementation should make sure that `0` is the most common state. (States that are more complicated than integers can be converted into an integer by marshaling/pickling the state and encoding the bytes of the resulting string into an integer.)

setstate(*state*)

Set the state of the encoder to *state*. *state* must be an encoder state returned by `getstate()`.

IncrementalDecoder Objects

The `IncrementalDecoder` class is used for decoding an input in multiple steps. It defines the following methods which every incremental decoder must define in order to be compatible with the Python codec registry.

class codecs.**IncrementalDecoder**(*errors='strict'*)

Constructor for an `IncrementalDecoder` instance.

All incremental decoders must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.

The `IncrementalDecoder` may implement different error handling schemes by providing the *errors* keyword argument. See *Error Handlers* for possible values.

The *errors* argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch between different error handling strategies during the lifetime of the `IncrementalDecoder` object.

decode(*object*, *final=False*)

Decodes *object* (taking the current state of the decoder into account) and returns the resulting decoded object. If this is the last call to `decode()` *final* must be true (the default is false). If *final* is true the decoder must decode the input completely and must flush all buffers. If this isn't possible (e.g. because of incomplete byte sequences at the end of the input) it must initiate error handling just like in the stateless case (which might raise an exception).

reset()

Reset the decoder to the initial state.

getstate()

Return the current state of the decoder. This must be a tuple with two items, the first must be the buffer containing the still undecoded input. The second must be an integer and can be additional state info. (The implementation should make sure that `0` is the most common additional state info.) If this additional state info is `0` it must be possible to set the decoder to the state which has no input buffered and `0` as the additional state info, so that feeding the previously buffered input to the decoder returns it to the previous state without producing any output. (Additional state info that is more complicated than integers can be converted into an integer by marshaling/pickling the info and encoding the bytes of the resulting string into an integer.)

setstate(*state*)

Set the state of the decoder to *state*. *state* must be a decoder state returned by `getstate()`.

Stream Encoding and Decoding

The `StreamWriter` and `StreamReader` classes provide generic working interfaces which can be used to implement new encoding submodules very easily. See `encodings.utf_8` for an example of how this is done.

StreamWriter Objects

The `StreamWriter` class is a subclass of `Codec` and defines the following methods which every stream writer must define in order to be compatible with the Python codec registry.

class codecs.**StreamWriter**(*stream*, *errors='strict'*)

Constructor for a `StreamWriter` instance.

All stream writers must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.

The *stream* argument must be a file-like object open for writing text or binary data, as appropriate for the specific codec.

The `StreamWriter` may implement different error handling schemes by providing the *errors* keyword argument. See *Error Handlers* for the standard error handlers the underlying stream codec may support.

The *errors* argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch between different error handling strategies during the lifetime of the `StreamWriter` object.

write(*object*)

Writes the object's contents encoded to the stream.

writelines(*list*)

Writes the concatenated iterable of strings to the stream (possibly by reusing the `write()` method). Infinite or very large iterables are not supported. The standard bytes-to-bytes codecs do not support this method.

reset()

Resets the codec buffers used for keeping internal state.

Chamar este método deve garantir que os dados na saída estejam num estado limpo, que permite anexar novos dados sem ter que verificar novamente todo o fluxo para recuperar o estado.

In addition to the above methods, the `StreamWriter` must also inherit all other methods and attributes from the underlying stream.

StreamReader Objects

The `StreamReader` class is a subclass of `Codec` and defines the following methods which every stream reader must define in order to be compatible with the Python codec registry.

class codecs.**StreamReader**(*stream*, *errors='strict'*)

Constructor for a `StreamReader` instance.

All stream readers must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.

The *stream* argument must be a file-like object open for reading text or binary data, as appropriate for the specific codec.

The `StreamReader` may implement different error handling schemes by providing the *errors* keyword argument. See *Error Handlers* for the standard error handlers the underlying stream codec may support.

The *errors* argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch between different error handling strategies during the lifetime of the `StreamReader` object.

The set of allowed values for the *errors* argument can be extended with `register_error()`.

read(*size=-1*, *chars=-1*, *firstline=False*)

Decodes data from the stream and returns the resulting object.

The *chars* argument indicates the number of decoded code points or bytes to return. The `read()` method will never return more data than requested, but it might return less, if there is not enough available.

The *size* argument indicates the approximate maximum number of encoded bytes or code points to read for decoding. The decoder can modify this setting as appropriate. The default value -1 indicates to read and decode as much as possible. This parameter is intended to prevent having to decode huge files in one step.

The *firstline* flag indicates that it would be sufficient to only return the first line, if there are decoding errors on later lines.

The method should use a greedy read strategy meaning that it should read as much data as is allowed within the definition of the encoding and the given size, e.g. if optional encoding endings or state markers are available on the stream, these should be read too.

readline(*size=None*, *keepends=True*)

Read one line from the input stream and return the decoded data.

size, if given, is passed as size argument to the stream's `read()` method.

If *keepends* is false line-endings will be stripped from the lines returned.

readlines(*sizehint=None*, *keepends=True*)

Read all lines available on the input stream and return them as a list of lines.

Line-endings are implemented using the codec's `decode()` method and are included in the list entries if *keepends* is true.

sizehint, if given, is passed as the *size* argument to the stream's `read()` method.

reset()

Resets the codec buffers used for keeping internal state.

Note that no stream repositioning should take place. This method is primarily intended to be able to recover from decoding errors.

In addition to the above methods, the `StreamReader` must also inherit all other methods and attributes from the underlying stream.

StreamReaderWriter Objects

The `StreamReaderWriter` is a convenience class that allows wrapping streams which work in both read and write modes.

The design is such that one can use the factory functions returned by the `lookup()` function to construct the instance.

class codecs.**StreamReaderWriter**(*stream*, *Reader*, *Writer*, *errors='strict'*)

Creates a `StreamReaderWriter` instance. *stream* must be a file-like object. *Reader* and *Writer* must be factory functions or classes providing the `StreamReader` and `StreamWriter` interface resp. Error handling is done in the same way as defined for the stream readers and writers.

`StreamReaderWriter` instances define the combined interfaces of `StreamReader` and `StreamWriter` classes. They inherit all other methods and attributes from the underlying stream.

StreamRecoder Objects

The `StreamRecoder` translates data from one encoding to another, which is sometimes useful when dealing with different encoding environments.

The design is such that one can use the factory functions returned by the `lookup()` function to construct the instance.

class codecs.**StreamRecoder**(*stream*, *encode*, *decode*, *Reader*, *Writer*, *errors='strict'*)

Creates a `StreamRecoder` instance which implements a two-way conversion: *encode* and *decode* work on the frontend — the data visible to code calling `read()` and `write()`, while *Reader* and *Writer* work on the backend — the data in *stream*.

You can use these objects to do transparent transcodings, e.g., from Latin-1 to UTF-8 and back.

The *stream* argument must be a file-like object.

The *encode* and *decode* arguments must adhere to the `Codec` interface. *Reader* and *Writer* must be factory functions or classes providing objects of the `StreamReader` and `StreamWriter` interface respectively.

Error handling is done in the same way as defined for the stream readers and writers.

`StreamRecoder` instances define the combined interfaces of `StreamReader` and `StreamWriter` classes. They inherit all other methods and attributes from the underlying stream.

7.2.2 Encodings and Unicode

Strings are stored internally as sequences of code points in range `U+0000–U+10FFFF`. (See **PEP 393** for more details about the implementation.) Once a string object is used outside of CPU and memory, endianness and how these arrays are stored as bytes become an issue. As with other codecs, serialising a string into a sequence of bytes is known as *encoding*, and recreating the string from the sequence of bytes is known as *decoding*. There are a variety of different text serialisation codecs, which are collectivity referred to as *text encodings*.

The simplest text encoding (called `'latin-1'` or `'iso-8859-1'`) maps the code points 0–255 to the bytes `0x0–0xff`, which means that a string object that contains code points above `U+00FF` can't be encoded with this codec. Doing so will raise a `UnicodeEncodeError` that looks like the following (although the details of the error message may differ): `UnicodeEncodeError: 'latin-1' codec can't encode character '\u1234' in position 3: ordinal not in range(256).`

There's another group of encodings (the so called charmap encodings) that choose a different subset of all Unicode code points and how these code points are mapped to the bytes `0x0–0xff`. To see how this is done simply open e.g. `encodings/cp1252.py` (which is an encoding that is used primarily on Windows). There's a string constant with 256 characters that shows you which character is mapped to which byte value.

All of these encodings can only encode 256 of the 1114112 code points defined in Unicode. A simple and straightforward way that can store each Unicode code point, is to store each code point as four consecutive bytes. There are two possibilities: store the bytes in big endian or in little endian order. These two encodings are called `UTF-32-BE` and `UTF-32-LE` respectively. Their disadvantage is that if e.g. you use `UTF-32-BE` on a little endian machine you will always have to swap bytes on encoding and decoding. `UTF-32` avoids this problem: bytes will always be in natural endianness. When these bytes are read by a CPU with a different endianness, then bytes have to be swapped though. To be able to detect the endianness of a `UTF-16` or `UTF-32` byte sequence, there's the so called BOM ("Byte Order Mark"). This is the Unicode character `U+FEFF`. This character can be prepended to every `UTF-16` or `UTF-32` byte sequence. The byte swapped version of this character (`0xFFFE`) is an illegal character that may not appear in a Unicode text. So when the first character in a `UTF-16` or `UTF-32` byte sequence appears to be a `U+FFFE` the bytes have to be swapped on decoding. Unfortunately the character `U+FEFF` had a second purpose as a `ZERO WIDTH NO-BREAK SPACE`: a character that has no width and doesn't allow a word to be split. It can e.g. be used to give hints to a ligature algorithm. With Unicode 4.0 using `U+FEFF` as a `ZERO WIDTH NO-BREAK SPACE` has been deprecated (with `U+2060` (`WORD JOINER`) assuming this role). Nevertheless Unicode software still must be able to handle `U+FEFF` in both roles: as a BOM it's a device to determine the storage layout of the encoded bytes, and vanishes once the byte sequence has been decoded into a string; as a `ZERO WIDTH NO-BREAK SPACE` it's a normal character that will be decoded like any other.

There's another encoding that is able to encode the full range of Unicode characters: UTF-8. UTF-8 is an 8-bit encoding, which means there are no issues with byte order in UTF-8. Each byte in a UTF-8 byte sequence consists of two parts: marker bits (the most significant bits) and payload bits. The marker bits are a sequence of zero to four `1` bits followed by a `0` bit. Unicode characters are encoded like this (with x being payload bits, which when concatenated give the Unicode character):

Range	Encoding
U-00000000 ... U-0000007F	0xxxxxxx
U-00000080 ... U-000007FF	110xxxxx 10xxxxxx
U-00000800 ... U-0000FFFF	1110xxxx 10xxxxxx 10xxxxxx
U-00010000 ... U-0010FFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

The least significant bit of the Unicode character is the rightmost x bit.

As UTF-8 is an 8-bit encoding no BOM is required and any `U+FEFF` character in the decoded string (even if it's the first character) is treated as a `ZERO WIDTH NO-BREAK SPACE`.

Without external information it's impossible to reliably determine which encoding was used for encoding a string. Each charmap encoding can decode any random byte sequence. However that's not possible with UTF-8, as UTF-8 byte sequences have a structure that doesn't allow arbitrary byte sequences. To increase the reliability with which a UTF-8 encoding can be detected, Microsoft invented a variant of UTF-8 (that Python calls `"utf-8-sig"`) for its Notepad program: Before any of the Unicode characters is written to the file, a UTF-8 encoded BOM (which looks like this as a byte sequence: `0xef`, `0xbb`, `0xbf`) is written. As it's rather improbable that any charmap encoded file starts with these byte values (which would e.g. map to

> LATIN SMALL LETTER I WITH DIAERESIS
> RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
> INVERTED QUESTION MARK

in iso-8859-1), this increases the probability that a `utf-8-sig` encoding can be correctly guessed from the byte sequence. So here the BOM is not used to be able to determine the byte order used for generating the byte sequence, but as a signature that helps in guessing the encoding. On encoding the utf-8-sig codec will write `0xef`, `0xbb`, `0xbf` as the first three bytes to the file. On decoding `utf-8-sig` will skip those three bytes if they appear as the first three bytes in the file. In UTF-8, the use of the BOM is discouraged and should generally be avoided.

7.2.3 Standard Encodings

Python comes with a number of codecs built-in, either implemented as C functions or with dictionaries as mapping tables. The following table lists the codecs by name, together with a few common aliases, and the languages for which the encoding is likely used. Neither the list of aliases nor the list of languages is meant to be exhaustive. Notice that spelling alternatives that only differ in case or use a hyphen instead of an underscore are also valid aliases because they are equivalent when normalized by *`normalize_encoding()`*. For example, `'utf-8'` is a valid alias for the `'utf_8'` codec.

> **Nota**
>
> The below table lists the most common aliases, for a complete list refer to the source aliases.py file.

On Windows, `cpXXX` codecs are available for all code pages. But only codecs listed in the following table are guarantead to exist on other platforms.

Detalhes da implementação do CPython: Some common encodings can bypass the codecs lookup machinery to improve performance. These optimization opportunities are only recognized by CPython for a limited set of (case insensitive) aliases: utf-8, utf8, latin-1, latin1, iso-8859-1, iso8859-1, mbcs (Windows only), ascii, us-ascii, utf-16, utf16, utf-32, utf32, and the same using underscores instead of dashes. Using alternative aliases for these encodings may result in slower execution.

Alterado na versão 3.6: Optimization opportunity recognized for us-ascii.

Many of the character sets support the same languages. They vary in individual characters (e.g. whether the EURO SIGN is supported or not), and in the assignment of characters to code positions. For the European languages in particular, the following variants typically exist:

- an ISO 8859 codeset

- a Microsoft Windows code page, which is typically derived from an 8859 codeset, but replaces control characters with additional graphic characters
- an IBM EBCDIC code page
- an IBM PC code page, which is ASCII compatible

Codec	Aliases	Idiomas
ascii	646, us-ascii	Inglês
big5	big5-tw, csbig5	Traditional Chinese
big5hkscs	big5-hkscs, hkscs	Traditional Chinese
cp037	IBM037, IBM039	Inglês
cp273	273, IBM273, csIBM273	Alemão Adicionado na versão 3.4.
cp424	EBCDIC-CP-HE, IBM424	Hebraico
cp437	437, IBM437	Inglês
cp500	EBCDIC-CP-BE, EBCDIC-CP--CH, IBM500	Western Europe
cp720		Árabe
cp737		Grego
cp775	IBM775	Baltic languages
cp850	850, IBM850	Western Europe
cp852	852, IBM852	Central and Eastern Europe
cp855	855, IBM855	Belarusian, Bulgarian, Macedonian, Russian, Serbian
cp856		Hebraico
cp857	857, IBM857	Turco
cp858	858, IBM858	Western Europe
cp860	860, IBM860	Português
cp861	861, CP-IS, IBM861	Islandês
cp862	862, IBM862	Hebraico
cp863	863, IBM863	Canadense
cp864	IBM864	Árabe
cp865	865, IBM865	Danish, Norwegian
cp866	866, IBM866	Russo
cp869	869, CP-GR, IBM869	Grego
cp874		Tailandês
cp875		Grego
cp932	932, ms932, mskanji, ms-kanji, windows-31j	Japonês
cp949	949, ms949, uhc	Coreano
cp950	950, ms950	Traditional Chinese
cp1006		Urdu
cp1026	ibm1026	Turco
cp1125	1125, ibm1125, cp866u, ruscii	Ucraniano Adicionado na versão 3.4.
cp1140	ibm1140	Western Europe
cp1250	windows-1250	Central and Eastern Europe
cp1251	windows-1251	Belarusian, Bulgarian, Macedonian, Russian, Serbian
cp1252	windows-1252	Western Europe
cp1253	windows-1253	Grego
cp1254	windows-1254	Turco
cp1255	windows-1255	Hebraico
cp1256	windows-1256	Árabe
cp1257	windows-1257	Baltic languages
cp1258	windows-1258	Vietnamita

continua na próxima página

Tabela 1 – continuação da página anterior

Codec	Aliases	Idiomas
euc_jp	eucjp, ujis, u-jis	Japonês
euc_jis_2004	jisx0213, eucjis2004	Japonês
euc_jisx0213	eucjisx0213	Japonês
euc_kr	euckr, korean, ksc5601, ks_c-5601, ks_c-5601-1987, ksx1001, ks_x-1001	Coreano
gb2312	chinese, csiso58gb231280, euc-cn, euccn, eucgb2312-cn, gb2312-1980, gb2312-80, iso-ir-58	Simplified Chinese
gbk	936, cp936, ms936	Unified Chinese
gb18030	gb18030-2000	Unified Chinese
hz	hzgb, hz-gb, hz-gb-2312	Simplified Chinese
iso2022_jp	csiso2022jp, iso2022jp, iso-2022-jp	Japonês
iso2022_jp_1	iso2022jp-1, iso-2022-jp-1	Japonês
iso2022_jp_2	iso2022jp-2, iso-2022-jp-2	Japanese, Korean, Simplified Chinese, Western Europe, Greek
iso2022_jp_2004	iso2022jp-2004, iso-2022-jp-2004	Japonês
iso2022_jp_3	iso2022jp-3, iso-2022-jp-3	Japonês
iso2022_jp_ext	iso2022jp-ext, iso-2022-jp-ext	Japonês
iso2022_kr	csiso2022kr, iso2022kr, iso-2022-kr	Coreano
latin_1	iso-8859-1, iso8859-1, 8859, cp819, latin, latin1, L1	Western Europe
iso8859_2	iso-8859-2, latin2, L2	Central and Eastern Europe
iso8859_3	iso-8859-3, latin3, L3	Esperanto, Maltese
iso8859_4	iso-8859-4, latin4, L4	Baltic languages
iso8859_5	iso-8859-5, cyrillic	Belarusian, Bulgarian, Macedonian, Russian, Serbian
iso8859_6	iso-8859-6, arabic	Árabe
iso8859_7	iso-8859-7, greek, greek8	Grego
iso8859_8	iso-8859-8, hebrew	Hebraico
iso8859_9	iso-8859-9, latin5, L5	Turco
iso8859_10	iso-8859-10, latin6, L6	Nordic languages
iso8859_11	iso-8859-11, thai	Thai languages
iso8859_13	iso-8859-13, latin7, L7	Baltic languages
iso8859_14	iso-8859-14, latin8, L8	Celtic languages
iso8859_15	iso-8859-15, latin9, L9	Western Europe
iso8859_16	iso-8859-16, latin10, L10	South-Eastern Europe
johab	cp1361, ms1361	Coreano
koi8_r		Russo
koi8_t		Tajik Adicionado na versão 3.5.
koi8_u		Ucraniano
kz1048	kz_1048, strk1048_2002, rk1048	Cazaque Adicionado na versão 3.5.
mac_cyrillic	maccyrillic	Belarusian, Bulgarian, Macedonian, Russian, Serbian
mac_greek	macgreek	Grego
mac_iceland	maciceland	Islandês
mac_latin2	maclatin2, maccentraleurope, mac_centeuro	Central and Eastern Europe
mac_roman	macroman, macintosh	Western Europe

continua na próxima página

Tabela 1 – continuação da página anterior

Codec	Aliases	Idiomas
mac_turkish	macturkish	Turco
ptcp154	csptcp154, pt154, cp154, cyrillic--asian	Cazaque
shift_jis	csshiftjis, shiftjis, sjis, s_jis	Japonês
shift_jis_2004	shiftjis2004, sjis_2004, sjis2004	Japonês
shift_jisx0213	shiftjisx0213, sjisx0213, s_jisx0213	Japonês
utf_32	U32, utf32	todas linguagens
utf_32_be	UTF-32BE	todas linguagens
utf_32_le	UTF-32LE	todas linguagens
utf_16	U16, utf16	todas linguagens
utf_16_be	UTF-16BE	todas linguagens
utf_16_le	UTF-16LE	todas linguagens
utf_7	U7, unicode-1-1-utf-7	todas linguagens
utf_8	U8, UTF, utf8, cp65001	todas linguagens
utf_8_sig		todas linguagens

Alterado na versão 3.4: The utf-16* and utf-32* encoders no longer allow surrogate code points (`U+D800`–`U+DFFF`) to be encoded. The utf-32* decoders no longer decode byte sequences that correspond to surrogate code points.

Alterado na versão 3.8: `cp65001` is now an alias to `utf_8`.

Alterado na versão 3.14: On Windows, `cpXXX` codecs are now available for all code pages.

7.2.4 Python Specific Encodings

A number of predefined codecs are specific to Python, so their codec names have no meaning outside Python. These are listed in the tables below based on the expected input and output types (note that while text encodings are the most common use case for codecs, the underlying codec infrastructure supports arbitrary data transforms rather than just text encodings). For asymmetric codecs, the stated meaning describes the encoding direction.

Text Encodings

The following codecs provide `str` to `bytes` encoding and *bytes-like object* to `str` decoding, similar to the Unicode text encodings.

Codec	Aliases	Significado
idna		Implement **RFC 3490**, see also *encodings.idna*. Only `errors='strict'` is supported.
mbcs	ansi, dbcs	Windows only: Encode the operand according to the ANSI codepage (CP_ACP).
oem		Windows only: Encode the operand according to the OEM codepage (CP_OEMCP). Adicionado na versão 3.6.
palmos		Encoding of PalmOS 3.5.
punycode		Implement **RFC 3492**. Stateful codecs are not supported.
raw_unicode_escape		Latin-1 encoding with `\uXXXX` and `\UXXXXXXXX` for other code points. Existing backslashes are not escaped in any way. It is used in the Python pickle protocol.
undefined		This Codec should only be used for testing purposes. Raise an exception for all conversions, even empty strings. The error handler is ignored.
unicode_escape		Encoding suitable as the contents of a Unicode literal in ASCII-encoded Python source code, except that quotes are not escaped. Decode from Latin-1 source code. Beware that Python source code actually uses UTF-8 by default.

Alterado na versão 3.8: "unicode_internal" codec is removed.

Binary Transforms

The following codecs provide binary transforms: *bytes-like object* to `bytes` mappings. They are not supported by `bytes.decode()` (which only produces `str` output).

Codec	Aliases	Significado	Encoder / decoder
base64_codec[1]	base64, base_64	Convert the operand to multiline MIME base64 (the result always includes a trailing `'\n'`). Alterado na versão 3.4: accepts any *bytes-like object* as input for encoding and decoding	*base64.encodebytes()* / *base64.decodebytes()*
bz2_codec	bz2	Compress the operand using bz2.	*bz2.compress()* / *bz2.decompress()*
hex_codec	hex	Convert the operand to hexadecimal representation, with two digits per byte.	*binascii.b2a_hex()* / *binascii.a2b_hex()*
quopri_codec	quopri, quotedprintable, quoted_printable	Convert the operand to MIME quoted printable.	*quopri.encode()* with `quotetabs=True` / *quopri.decode()*
uu_codec	uu	Convert the operand using uuencode.	
zlib_codec	zip, zlib	Compress the operand using gzip.	*zlib.compress()* / *zlib.decompress()*

Adicionado na versão 3.2: Restoration of the binary transforms.

Alterado na versão 3.4: Restoration of the aliases for the binary transforms.

Text Transforms

The following codec provides a text transform: a *str* to *str* mapping. It is not supported by *str.encode()* (which only produces *bytes* output).

Codec	Aliases	Significado
rot_13	rot13	Return the Caesar-cypher encryption of the operand.

Adicionado na versão 3.2: Restoration of the `rot_13` text transform.

Alterado na versão 3.4: Restoration of the `rot13` alias.

7.2.5 `encodings` — Encodings package

This module implements the following functions:

`encodings.`**`normalize_encoding`**`(`*encoding*`)`

Normalize encoding name *encoding*.

Normalization works as follows: all non-alphanumeric characters except the dot used for Python package names are collapsed and replaced with a single underscore, leading and trailing underscores are removed. For example, `' -;#'` becomes `'_'`.

Note that *encoding* should be ASCII only.

Nota

[1] In addition to *bytes-like objects*, `'base64_codec'` also accepts ASCII-only instances of *str* for decoding

The following functions should not be used directly, except for testing purposes; *codecs.lookup()* should be used instead.

encodings.**search_function**(*encoding*)

Search for the codec module corresponding to the given encoding name *encoding*.

This function first normalizes the *encoding* using *normalize_encoding()*, then looks for a corresponding alias. It attempts to import a codec module from the encodings package using either the alias or the normalized name. If the module is found and defines a valid `getregentry()` function that returns a *codecs.CodecInfo* object, the codec is cached and returned.

If the codec module defines a `getaliases()` function any returned aliases are registered for future use.

encodings.**win32_code_page_search_function**(*encoding*)

Search for a Windows code page encoding *encoding* of the form `cpXXXX`.

If the code page is valid and supported, return a *codecs.CodecInfo* object for it.

Disponibilidade: Windows.

Adicionado na versão 3.14.

This module implements the following exception:

exception encodings.**CodecRegistryError**

Raised when a codec is invalid or incompatible.

7.2.6 encodings.idna — Internationalized Domain Names in Applications

This module implements **RFC 3490** (Internationalized Domain Names in Applications) and **RFC 3492** (Nameprep: A Stringprep Profile for Internationalized Domain Names (IDN)). It builds upon the `punycode` encoding and *stringprep*.

If you need the IDNA 2008 standard from **RFC 5891** and **RFC 5895**, use the third-party idna module.

These RFCs together define a protocol to support non-ASCII characters in domain names. A domain name containing non-ASCII characters (such as `www.Alliancefrançaise.nu`) is converted into an ASCII-compatible encoding (ACE, such as `www.xn--alliancefranaise-npb.nu`). The ACE form of the domain name is then used in all places where arbitrary characters are not allowed by the protocol, such as DNS queries, HTTP *Host* fields, and so on. This conversion is carried out in the application; if possible invisible to the user: The application should transparently convert Unicode domain labels to IDNA on the wire, and convert back ACE labels to Unicode before presenting them to the user.

Python supports this conversion in several ways: the `idna` codec performs conversion between Unicode and ACE, separating an input string into labels based on the separator characters defined in **section 3.1 of RFC 3490** and converting each label to ACE as required, and conversely separating an input byte string into labels based on the `.` separator and converting any ACE labels found into unicode. Furthermore, the *socket* module transparently converts Unicode host names to ACE, so that applications need not be concerned about converting host names themselves when they pass them to the socket module. On top of that, modules that have host names as function parameters, such as *http.client* and *ftplib*, accept Unicode host names (*http.client* then also transparently sends an IDNA hostname in the *Host* field if it sends that field at all).

When receiving host names from the wire (such as in reverse name lookup), no automatic conversion to Unicode is performed: applications wishing to present such host names to the user should decode them to Unicode.

The module *encodings.idna* also implements the nameprep procedure, which performs certain normalizations on host names, to achieve case-insensitivity of international domain names, and to unify similar characters. The nameprep functions can be used directly if desired.

encodings.idna.**nameprep**(*label*)

Return the nameprepped version of *label*. The implementation currently assumes query strings, so `AllowUnassigned` is true.

encodings.idna.**ToASCII**(*label*)

Convert a label to ASCII, as specified in **RFC 3490**. `UseSTD3ASCIIRules` is assumed to be false.

encodings.idna.**ToUnicode**(*label*)

Convert a label to Unicode, as specified in **RFC 3490**.

7.2.7 `encodings.mbcs` — Windows ANSI codepage

This module implements the ANSI codepage (CP_ACP).

Disponibilidade: Windows.

Alterado na versão 3.2: Before 3.2, the *errors* argument was ignored; `'replace'` was always used to encode, and `'ignore'` to decode.

Alterado na versão 3.3: Support any error handler.

7.2.8 `encodings.utf_8_sig` — UTF-8 codec with BOM signature

This module implements a variant of the UTF-8 codec. On encoding, a UTF-8 encoded BOM will be prepended to the UTF-8 encoded bytes. For the stateful encoder this is only done once (on the first write to the byte stream). On decoding, an optional UTF-8 encoded BOM at the start of the data will be skipped.

CAPÍTULO 8

Tipos de Dados

Os módulos descritos neste capítulo fornecem uma variedade de tipos de dados especializados, como datas e horas, vetores de tipo fixo, filas de heap, filas de extremidade dupla e enumerações.

O Python também fornece alguns tipos de dados embutidos, em especial `dict`, `list`, `set` e `frozenset` e `tuple`. A classe `str` é usada para armazenar strings Unicode, e as classes `bytes` e `bytearray` são usadas para armazenar dados binários.

Os seguintes módulos estão documentados neste capítulo:

8.1 `datetime` — Tipos básicos de data e hora

Código-fonte: Lib/datetime.py

O módulo `datetime` fornece as classes para manipulação de datas e horas.

Ainda que a aritmética de data e hora seja suportada, o foco da implementação é na extração eficiente do atributo para formatação da saída e manipulação.

Dica

Pular para *os códigos de formatação*.

Ver também

Módulo `calendar`
Funções gerais relacionadas ao calendário.

Módulo `time`
Acesso de hora e conversões.

Módulo `zoneinfo`
Fusos horários concretos representando o banco de dados de fusos horários IANA.

Pacote dateutil
Biblioteca de terceiros com fuso horário expandido e suporte à análise.

Pacote DateType
Biblioteca de terceiros que apresenta tipos estáticos distintos para, por exemplo, permitir que *verificadores de tipo estático* diferenciem datas ingênuas e conscientes.

8.1.1 Objetos Conscientes e Ingênuos

Objetos de data e hora podem ser categorizados como "consciente" ou "ingênuo" dependendo se eles incluem ou não informação sobre fuso horário.

Com conhecimento suficiente dos ajustes de tempo algorítmicos e políticos aplicáveis, como informações de fuso horário e horário de verão, um objeto **consciente** pode se localizar em relação a outros objetos conscientes. Um objeto consciente representa um momento específico no tempo que não está aberto à interpretação.[1]

Um objeto **ingênuo** não contém informações suficientes para se localizar inequivocamente em relação a outros objetos de data/hora. Se um objeto ingênuo representa o Coordinated Universal Time (UTC), a hora local, ou a hora em algum outro fuso horário, isso depende exclusivamente do programa, assim como é tarefa do programa decidir se um número específico representa metros, milhas ou massa. Objetos ingênuos são fáceis de entender e trabalhar, com o custo de ignorar alguns aspectos da realidade.

Para aplicativos que requerem objetos conscientes, os objetos *datetime* e *time* possuem um atributo opcional de informações do fuso horário, `tzinfo`, que pode ser definido como uma instância de uma subclasse da classe abstrata *tzinfo*. Esses objetos *tzinfo* capturam informações sobre a diferença da hora UTC, o nome do fuso horário e se o horário de verão está em vigor.

Somente uma classe concreta *tzinfo*, a classe *timezone*, é fornecida pelo módulo `datetime`. A classe `timezone` pode representar fusos horários simples com diferenças fixas do UTC, como o próprio UTC, ou os fusos horários norte-americanos EST e EDT. O suporte a fusos horários em níveis mais detalhados depende da aplicação. As regras para ajuste de tempo em todo o mundo são mais políticas do que racionais, mudam com frequência e não há um padrão adequado para todas as aplicações além da UTC.

8.1.2 Constantes

O módulo `datetime` exporta as seguintes constantes:

`datetime.`**`MINYEAR`**
O menor número de ano permitido em um objeto *date* ou *datetime*. *MINYEAR* é 1.

`datetime.`**`MAXYEAR`**
O maior número de ano permitido no objeto *date* ou *datetime*. *MAXYEAR* é 9999.

`datetime.`**`UTC`**
Apelido para o singleton de fuso horário UTC *datetime.timezone.utc*.

Adicionado na versão 3.11.

8.1.3 Tipos disponíveis

`class` `datetime.`**`date`**
Uma data ingênua idealizada, presumindo que o atual calendário Gregoriano sempre foi, e sempre estará em vigor. Atributos: *year*, *month* e *day*.

`class` `datetime.`**`time`**
Um horário ideal, independente de qualquer dia em particular, presumindo que todos os dias tenham exatamente 24*60*60 segundos. (Não há noção de "segundos bissextos" aqui.) Atributos: *hour*, *minute*, *second*, *microsecond* e *tzinfo*.

`class` `datetime.`**`datetime`**
Uma combinação de uma data e uma hora. Atributos: *year*, *month*, *day*, *hour*, *minute*, *second*, *microsecond* e *tzinfo*.

[1] Se, isto é, nós ignoramos os efeitos da Relatividade

class datetime.**timedelta**

Uma duração que expressa a diferença entre duas instâncias *datetime* ou *date* para resolução de microssegundos.

class datetime.**tzinfo**

Uma classe base abstrata para objetos de informações de fuso horário. Eles são usados pelas classes *datetime* e *time* para fornecer uma noção personalizável de ajuste de horário (por exemplo, para considerar o fuso horário e/ou o horário de verão).

class datetime.**timezone**

Uma classe que implementa a classe base abstrata *tzinfo* como uma diferença fixa do UTC.

Adicionado na versão 3.2.

Objetos desse tipo são imutáveis.

Relacionamentos de subclasse:

```
object
    timedelta
    tzinfo
        timezone
    time
    date
        datetime
```

Propriedades Comuns

Os tipos *date*, *datetime*, *time* e *timezone* compartilham esses recursos comuns:

- Objetos desse tipo são imutáveis.
- Objetos desses tipos são *hasheáveis*, o que significa que podem ser usados como chaves de dicionário.
- Objetos desse tipo suportam decapagem eficiente através do módulo *pickle*.

Determinando se um Objeto é Consciente ou Ingênuo

Objetos do tipo *date* são sempre ingênuos.

Um objeto do tipo *time* ou *datetime* pode ser consciente ou ingênuo.

O objeto *datetime* `d` é consciente se ambos os seguintes itens forem verdadeiros:

1. `d.tzinfo` não é `None`
2. `d.tzinfo.utcoffset(d)` não retorna `None`

Caso contrário, `d` é ingênuo.

O objeto *time* `t` é consciente, se os seguintes itens são verdadeiros:

1. `t.tzinfo` não é `None`
2. `t.tzinfo.utcoffset(None)` não retorna `None`.

Caso contrário, `t` é ingênuo.

A distinção entre consciente e ingênuo não se aplica a objetos *timedelta*.

8.1.4 Objetos `timedelta`

O objeto *timedelta* representa uma duração, a diferença entre duas instâncias *datetime* ou *date*.

class datetime.**timedelta**(*days=0*, *seconds=0*, *microseconds=0*, *milliseconds=0*, *minutes=0*, *hours=0*, *weeks=0*)

Todos os argumentos são opcionais e o padrão é 0. Os argumentos podem ser números inteiros ou ponto flutuantes, e podem ser positivos ou negativos.

Apenas *days*, *seconds* e *microseconds* são armazenados internamente. Os argumentos são convertidos para essas unidades:

- Um milissegundo é convertido em 1000 microssegundos.
- Um minuto é convertido em 60 segundos.
- Uma hora é convertida em 3600 segundos.
- Uma semana é convertida para 7 dias.

e dias, segundos e microssegundos são normalizados para que a representação seja única, com

- `0 <= microseconds < 1000000`
- `0 <= seconds < 3600*24` (o número de segundos em um dia)
- `-999999999 <= days <= 999999999`

O exemplo a seguir ilustra como quaisquer argumentos além de *days*, *seconds* e *microseconds* são "mesclados" e normalizados nos três atributos resultantes:

```
>>> from datetime import timedelta
>>> delta = timedelta(
...     days=50,
...     seconds=27,
...     microseconds=10,
...     milliseconds=29000,
...     minutes=5,
...     hours=8,
...     weeks=2
... )
>>> # só restam dias, segundos e microssegundos
>>> delta
datetime.timedelta(days=64, seconds=29156, microseconds=10)
```

Se qualquer argumento for um ponto flutuante e houver microssegundos fracionários, os microssegundos fracionários restantes de todos os argumentos serão combinados e sua soma será arredondada para o microssegundo mais próximo usando o desempatador de metade da metade para o par. Se nenhum argumento é ponto flutuante, os processos de conversão e normalização são exatos (nenhuma informação é perdida).

Se o valor normalizado de dias estiver fora do intervalo indicado, a exceção *OverflowError* é levantada.

Observe que a normalização de valores negativos pode ser surpreendente a princípio. Por exemplo:

```
>>> from datetime import timedelta
>>> d = timedelta(microseconds=-1)
>>> (d.days, d.seconds, d.microseconds)
(-1, 86399, 999999)
```

Como a representação de string dos objetos `timedelta` pode ser confusa, use a seguinte receita para produzir um formato mais legível:

```
>>> def pretty_timedelta(td):
...     if td.days >= 0:
...         return str(td)
...     return f'-({-td!s})'
...
>>> d = timedelta(hours=-1)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> str(d)  # not human-friendly
'-1 day, 23:00:00'
>>> pretty_timedelta(d)
'-(1:00:00)'
```

Atributos de classe:

`timedelta.`**`min`**

O mais negativo objeto *`timedelta`*, `timedelta(-999999999)`.

`timedelta.`**`max`**

O mais positivo objeto *`timedelta`*, `timedelta(days=999999999, hours=23, minutes=59, seconds=59, microseconds=999999)`.

`timedelta.`**`resolution`**

A menor diferença possível entre objetos não iguais *`timedelta`*, `timedelta(microseconds=1)`.

Observe que, devido à normalização, `timedelta.max` é maior que `-timedelta.min`. `-timedelta.max` não é representável como um objeto *`timedelta`*.

Atributos de instância (somente leitura):

`timedelta.`**`days`**

Entre -999.999.999 e 999.999.999 inclusive

`timedelta.`**`seconds`**

Entre 0 e 86.399 inclusive.

Cuidado

É um bug relativamente comum utilizar este atributo de forma não intencional quando, na verdade, o objetivo é obter o valor de *`total_seconds()`*:

```
>>> from datetime import timedelta
>>> duration = timedelta(seconds=11235813)
>>> duration.days, duration.seconds
(130, 3813)
>>> duration.total_seconds()
11235813.0
```

`timedelta.`**`microseconds`**

Entre 0 e 999.999 inclusive.

Operações suportadas:

Operação	Resultado
`t1 = t2 + t3`	Soma de `t2` e `t3`. Depois `t1 - t2 == t3` e `t1 - t3 == t2` são verdadeiros. (1)
`t1 = t2 - t3`	Diferença de `t2` e `t3`. Depois `t1 == t2 - t3` e `t2 == t1 + t3` são verdadeiros (1)(6)
`t1 = t2 * i or t1 = i * t2`	Delta multiplicado por um número inteiro. Depois `t1 // i == t2` é verdadeiro, desde que `i != 0`.
	Em geral, `t1 * i == t1 * (i-1) + t1` é verdadeiro. (1)
`t1 = t2 * f or t1 = f * t2`	Delta multiplicado por um float, ponto flutuante. O resultado é arredondado para o múltiplo mais próximo de timedelta.resolution usando a metade da metade para o par.
`f = t2 / t3`	Divisão (3) da duração total `t2` por unidade de intervalo `t3`. Retorna um objeto *float*.
`t1 = t2 / f or t1 = t2 / i`	Delta dividido por um float ou um int. O resultado é arredondado para o múltiplo mais próximo de timedelta.resolution usando a metade da metade para o par.
`t1 = t2 // i` ou `t1 = t2 // t3`	O piso é calculado e o restante (se houver) é jogado fora. No segundo caso, um número inteiro é retornado. (3)
`t1 = t2 % t3`	O restante é calculado como um objeto *timedelta*. (3)
`q, r = divmod(t1, t2)`	Calcula o quociente e o resto: `q = t1 // t2` (3) e `r = t1 % t2`. `q` é um inteiro e `r` é um objeto *timedelta*.
`+t1`	Retorna um objeto *timedelta* com o mesmo valor. (2)
`-t1`	Equivalente a `timedelta(-t1.days, -t1.seconds, -t1.microseconds)` e a `t1 * -1`. (1)(4)
`abs(t)`	Equivalente a `+t` quando `t.days >= 0` e a `-t` quando `t.days < 0`. (2)
`str(t)`	Retorna uma string no formato `[D day[s], ][H]H:MM:SS[.UUUUUU]`, onde D é negativo para `t` negativo. (5)
`repr(t)`	Retorna uma representação em string do objeto *timedelta* como uma chamada do construtor com valores de atributos canônicos.

Notas:

(1) Isso é exato, mas pode transbordar.

(2) Isso é exato e não pode transbordar.

(3) A divisão por zero levanta *ZeroDivisionError*.

(4) `-timedelta.max` não é representável como um objeto *timedelta*.

(5) As representações de string de objetos *timedelta* são normalizadas de maneira semelhante à sua representação interna. Isso leva a resultados um tanto incomuns para timedeltas negativos. Por exemplo:

```
>>> timedelta(hours=-5)
datetime.timedelta(days=-1, seconds=68400)
>>> print(_)
-1 day, 19:00:00
```

(6) A expressão `t2 - t3` sempre será igual à expressão `t2 + (-t3)` exceto quando t3 for igual a `timedelta.max`; nesse caso, o primeiro produzirá um resultado enquanto o último transbordará.

Além das operações listadas acima, os objetos *timedelta* suportam certas adições e subtrações com os objetos *date* e *datetime* (veja abaixo).

Alterado na versão 3.2: A divisão pelo piso e a divisão verdadeira de um objeto *timedelta* por outro objeto *timedelta* agora são suportadas, assim como as operações restantes e a função *divmod()*. A divisão verdadeira e multiplicação de um objeto *timedelta* por um objeto *float* agora são suportadas.

Objetos *timedelta* dão suporte a comparações de igualdade e ordem.

Em contexto booleano, um objeto *timedelta* é considerado verdadeiro se, e somente se, não for igual a `timedelta(0)`.

Métodos de instância:

timedelta.**total_seconds**()

Retorna o número total de segundos contidos na duração. Equivalente a `td / timedelta(seconds=1)`. Para unidades de intervalo diferentes de segundos, use a forma de divisão diretamente (por exemplo `td / timedelta(microseconds=1)`).

Observe que, em intervalos de tempo muito grandes (mais de 270 anos na maioria das plataformas), esse método perde a precisão de microssegundos.

Adicionado na versão 3.2.

Exemplos de uso: `timedelta`

Um exemplo adicional de normalização:

```
>>> # Os componentes de outro_ano somam exatamente 365 dias
>>> from datetime import timedelta
>>> year = timedelta(days=365)
>>> outro_ano = timedelta(weeks=40, days=84, hours=23,
...                       minutes=50, seconds=600)
>>> year == outro_ano
True
>>> year.total_seconds()
31536000.0
```

Exemplos de aritmética com *timedelta*:

```
>>> from datetime import timedelta
>>> ano = timedelta(days=365)
>>> dez_anos = 10 * ano
>>> dez_anos
datetime.timedelta(days=3650)
>>> dez_anos.days // 365
10
>>> nove_anos = dez_anos - ano
>>> nove_anos
datetime.timedelta(days=3285)
>>> três_anos = nove_anos // 3
>>> três_anos, três_anos.days // 365
(datetime.timedelta(days=1095), 3)
```

8.1.5 Objetos `date`

O objeto *date* representa uma data (ano, mês e dia) em um calendário idealizado, o atual calendário Gregoriano estendido indefinidamente em ambas as direções.

1º de janeiro do ano 1 é chamado de dia número 1, 2º de janeiro do ano 1 é chamado de dia número 2, e assim por diante.[2]

class datetime.**date**(*year*, *month*, *day*)

Todos os argumentos são obrigatórios. Os argumentos devem ser números inteiros, nos seguintes intervalos:

- `MINYEAR <= year <= MAXYEAR`
- `1 <= month <= 12`
- `1 <= day <= número de dias no mês e ano fornecidos`

Se um argumento fora desses intervalos for fornecido, a exceção *ValueError* é levantada.

[2] Isso combina com a definição do calendário "proléptico Gregoriano" no livro *Cálculos Calendáricos* de Dershowitz e Reingold, onde ele é o calendário base para todas as computações. Veja o livro para algoritmos para conversão entre ordinais proléptico Gregoriano e muitos outros sistemas de calendário.

Outros construtores, todos os métodos de classe.

classmethod date.**today**()

Retorna a data local atual.

Isso é equivalente a `date.fromtimestamp(time.time())`.

classmethod date.**fromtimestamp**(*timestamp*)

Retorna a data local correspondente ao registro de data e hora do POSIX, como é retornado por *time.time()*.

Isso pode levantar *OverflowError*, se o registro de data e hora estiver fora do intervalo de valores suportados pela função C `localtime()` da plataforma, e em caso de falha *OSError* em `localtime()`. É comum que isso seja restrito a anos de 1970 a 2038. Observe que, em sistemas não POSIX que incluem segundos bissextos na sua notação de registro de data e hora, os segundos bissextos são ignorados por *fromtimestamp()*.

Alterado na versão 3.3: Levanta *OverflowError* ao invés de *ValueError* se o registro de data e hora estiver fora do intervalo de valores suportados pela plataforma C `localtime()` função. Levanta *OSError* ao invés de *ValueError* em falha de `localtime()` .

classmethod date.**fromordinal**(*ordinal*)

Retorna a data correspondente ao ordinal proléptico gregoriano, considerando que 1º de janeiro do ano 1 tem o ordinal 1.

ValueError é levantado a menos que `1 <= ordinal <= date.max.toordinal()`. Para qualquer data `d`, `date.fromordinal(d.toordinal()) == d`.

classmethod date.**fromisoformat**(*date_string*)

Retorna um *date* correspondendo a *date_string* dada em qualquer formato válido de ISO 8601, com as seguintes exceções:

1. Datas de precisão reduzida não são atualmente suportadas (`YYYY-MM`, `YYYY`).
2. Representações de data estendidas não são atualmente suportadas (`±YYYYYY-MM-DD`).
3. Atualmente, as datas ordinais não são suportadas (`YYYY-OOO`).

Exemplos:

```
>>> from datetime import date
>>> date.fromisoformat('2019-12-04')
datetime.date(2019, 12, 4)
>>> date.fromisoformat('20191204')
datetime.date(2019, 12, 4)
>>> date.fromisoformat('2021-W01-1')
datetime.date(2021, 1, 4)
```

Adicionado na versão 3.7.

Alterado na versão 3.11: Anteriormente, este método tinha suporte apenas ao formato `YYYY-MM-DD`.

classmethod date.**fromisocalendar**(*year*, *week*, *day*)

Retorna um objeto *date* correspondendo a data em calendário ISO especificada por year, week e day. Esta é o inverso da função *date.isocalendar()*.

Adicionado na versão 3.8.

classmethod date.**strptime**(*date_string*, *format*)

Retorna um *date* correspondente ao *date_string*, analisado de acordo com *format*. Isso equivale a:

```
date(*(time.strptime(date_string, format)[0:3]))
```

ValueError é levantado se o date_string e o format não puderem ser interpretados por *time.strptime()* ou se ele retorna um valor o qual não é uma tupla temporal. Veja também *Comportamento de strftime() e strptime()* e *date.fromisoformat()*.

Nota

Se *format* especificar um dia do mês sem um ano, um *DeprecationWarning* é emitido. Isso é para evitar um bug de ano bissexto quadrienal no código que busca analisar apenas um mês e dia, pois o ano padrão usado na ausência de um no formato não é um ano bissexto. Esses valores de *format* podem gerar um erro a partir do Python 3.15. A solução alternativa é sempre incluir um ano em seu *format*. Se estiver analisando valores de *date_string* que não têm um ano, adicione explicitamente um ano que seja um ano bissexto antes da análise:

```
>>> from datetime import date
>>> date_string = "02/29"
>>> when = date.strptime(f"{date_string};1984", "%m/%d;%Y")  # Evita bug do␣
↪ano  bissexto.
>>> when.strftime("%B %d")
'February 29'
```

Adicionado na versão 3.14.

Atributos de classe:

`date.`**`min`**

A data representável mais antiga, `date(MINYEAR, 1, 1)`.

`date.`**`max`**

A data representável mais tardia, `date(MAXYEAR, 12, 31)`.

`date.`**`resolution`**

A menor diferença possível entre objetos date não iguais, `timedelta(days=1)`.

Atributos de instância (somente leitura):

`date.`**`year`**

Entre *MINYEAR* e *MAXYEAR* incluindo extremos.

`date.`**`month`**

Entre 1 e 12 incluindo extremos.

`date.`**`day`**

Entre 1 e o número de dias no mês especificado do ano especificado.

Operações suportadas:

Operação	Resultado
`date2 = date1 + timedelta`	`date2` terá `timedelta.days` dias após `date1`. (1)
`date2 = date1 - timedelta`	Calcula `date2` de modo que `date2 + timedelta == date1`. (2)
`timedelta = date1 - date2`	(3)
`date1 == date2` `date1 != date2`	Comparação de igualdade. (4)

`date1 < date2`
`date1 > date2`
`date1 <= date2`
`date1 >= date2`
	Comparação de ordem. (5)

Notas:

(1) *date2* é movida para frente no tempo se `timedelta.days > 0`, ou para trás se `timedelta.days < 0`. Posteriormente `date2 - date1 == timedelta.days`. `timedelta.seconds` e `timedelta.microseconds` são ignorados. A exceção *OverflowError* é levantada se `date2.year` for menor que *MINYEAR* ou maior que *MAXYEAR*.

(2) `timedelta.seconds` e `timedelta.microseconds` são ignoradas.

(3) Isso é exato e não pode estourar. `timedelta.seconds` e `timedelta.microseconds` são 0, e `date2 + timedelta == date1` depois.

(4) Objetos *date* são iguais se representam a mesma data.

Objetos `date` que não são também instâncias *datetime* nunca são iguais a objetos `datetime`, mesmo que representem a mesma data.

(5) *date1* é considerado menor que *date2* quando *date1* preceder *date2* no tempo. Em outras palavras, `date1 < date2` se e somente se `date1.toordinal() < date2.toordinal()`.

A comparação de ordem entre um objeto `date` que não é também uma instância *datetime* e um objeto `datetime` levanta *TypeError*.

Alterado na versão 3.13: A comparação entre o objeto *datetime* e uma instância da subclasse *date* que não é uma subclasse `datetime` não converte mais a última para `date`, ignorando o parte horária e o fuso horário. O comportamento padrão pode ser alterado substituindo os métodos especiais de comparação nas subclasses.

Em contextos booleanos, todo objeto *date* é considerado verdadeiro.

Métodos de instância:

`date.`**`replace`**`(`*year=self.year*, *month=self.month*, *day=self.day*`)`

Retorna um novo objeto *date* com os mesmos valores, mas com os parâmetros especificados atualizados.

Exemplo:

```
>>> from datetime import date
>>> d = date(2002, 12, 31)
>>> d.replace(day=26)
datetime.date(2002, 12, 26)
```

A função genérica *copy.replace()* também oferece suporte a objetos *date*.

`date.`**`timetuple`**`()`

Retorna uma *time.struct_time* tal como retornado por *time.localtime()*.

As horas, minutos e segundos são 0, e o sinalizador de horário de verão é -1.

`d.timetuple()` é equivalente a:

```
time.struct_time((d.year, d.month, d.day, 0, 0, 0, d.weekday(), yday, -1))
```

sendo `yday = d.toordinal() - date(d.year, 1, 1).toordinal() + 1` o número do dia no ano atual, começando com 1 para 1º de janeiro.

`date.`**`toordinal`**`()`

Retorna o ordinal proléptico gregoriano da data, considerando que 1º de janeiro do ano 1 tem o ordinal 1. Para qualquer objeto *date* `d`, `date.fromordinal(d.toordinal()) == d`.

`date.`**`weekday`**`()`

Retorna o dia da semana como um inteiro, onde Segunda é 0 e Domingo é 6. Por exemplo, `date(2002, 12, 4).weekday() == 2`, uma Quarta-feira. Veja também *isoweekday()*.

`date.`**`isoweekday`**`()`

Retorna o dia da semana como um inteiro, onde Segunda é 1 e Domingo é 7. Por exemplo, `date(2002, 12, 4).isoweekday() == 3`, uma Quarta-feira. Veja também *weekday()*, *isocalendar()*.

date.**isocalendar**()

Retorna um objeto *tupla nomeada* com três componentes: `year`, `week` e `weekday`.

O calendário ISO é uma variação amplamente usada do calendário gregoriano.[3]

O ano ISO consiste de 52 ou 53 semanas completas, e onde uma semana começa na segunda-feira e termina no domingo. A primeira semana de um ano ISO é a primeira semana no calendário (Gregoriano) de um ano contendo uma quinta-feira. Isso é chamado de semana número 1, e o ano ISO dessa quinta-feira é o mesmo que o seu ano Gregoriano.

Por exemplo, 2004 começa em uma quinta-feira, então a primeira semana do ano ISO 2004 começa na segunda-feira, 29 de dezembro de 2003, e termina no domingo, 4 de janeiro de 2004:

```
>>> from datetime import date
>>> date(2003, 12, 29).isocalendar()
datetime.IsoCalendarDate(year=2004, week=1, weekday=1)
>>> date(2004, 1, 4).isocalendar()
datetime.IsoCalendarDate(year=2004, week=1, weekday=7)
```

Alterado na versão 3.9: Resultado alterado de uma tupla para uma *tupla nomeada*.

date.**isoformat**()

Retorna uma string representando a data no formato ISO 8601, `YYYY-MM-DD`:

```
>>> from datetime import date
>>> date(2002, 12, 4).isoformat()
'2002-12-04'
```

date.**__str__**()

Para um data `d`, `str(d)` é equivalente a `d.isoformat()`.

date.**ctime**()

Retorna uma string representando a data:

```
>>> from datetime import date
>>> date(2002, 12, 4).ctime()
'Wed Dec  4 00:00:00 2002'
```

`d.ctime()` é equivalente a:

```
time.ctime(time.mktime(d.timetuple()))
```

em plataformas em que a função C nativa `ctime()` (que é invocada pela função *`time.ctime()`*, mas não pelo método *`date.ctime()`*) se conforma com o padrão C.

date.**strftime**(*format*)

Retorna uma string representando a data, controlado por uma string explícita de formatação. Códigos de formatação referenciando horas, minutos ou segundos irão ver valores 0. Veja também *Comportamento de strftime() e strptime()* e *`date.isoformat()`*.

date.**__format__**(*format*)

O mesmo que *`date.strftime()`*. Isso torna possível especificar uma string de formatação para o objeto *`date`* em literais de string formatados e ao usar *`str.format()`*. Veja também *Comportamento de strftime() e strptime()* e *`date.isoformat()`*.

[3] Veja O guia para a matemática de calendário ISO 8601 de R. H. van Gent para uma boa explicação.

Exemplos de uso: `date`

Exemplo de contagem de dias para um evento:

```
>>> import time
>>> from datetime import date
>>> hoje = date.today()
>>> hoje
datetime.date(2007, 12, 5)
>>> hoje == date.fromtimestamp(time.time())
True
>>> meu_aniversário = date(hoje.year, 6, 24)
>>> if meu_aniversário < hoje:
...     meu_aniversário = meu_aniversário.replace(year=hoje.year + 1)
...
>>> meu_aniversário
datetime.date(2008, 6, 24)
>>> tempo_até_aniversário = abs(meu_aniversário - hoje)
>>> tempo_até_aniversário.days
202
```

Mais exemplos de uso da classe *date*:

```
>>> from datetime import date
>>> d = date.fromordinal(730920) # 730920th day after 1. 1. 0001
>>> d
datetime.date(2002, 3, 11)

>>> # Methods related to formatting string output
>>> d.isoformat()
'2002-03-11'
>>> d.strftime("%d/%m/%y")
'11/03/02'
>>> d.strftime("%A %d. %B %Y")
'Monday 11. March 2002'
>>> d.ctime()
'Mon Mar 11 00:00:00 2002'
>>> 'The {1} is {0:%d}, the {2} is {0:%B}.'.format(d, "day", "month")
'The day is 11, the month is March.'

>>> # Methods for to extracting 'components' under different calendars
>>> t = d.timetuple()
>>> for i in t:
...     print(i)
2002                # year
3                   # month
11                  # day
0
0
0
0                   # weekday (0 = Monday)
70                  # 70th day in the year
-1
>>> ic = d.isocalendar()
>>> for i in ic:
...     print(i)
2002                # ISO year
11                  # ISO week number
```

(continua na próxima página)

(continuação da página anterior)

```
1                   # ISO day number ( 1 = Monday )

>>> # A date object is immutable; all operations produce a new object
>>> d.replace(year=2005)
datetime.date(2005, 3, 11)
```

8.1.6 Objetos `datetime`

Um objeto *datetime* é um único objeto contendo todas as informações de um objeto *date* e um objeto *time*.

Assim como um objeto *date*, *datetime* presume o atual calendário Gregoriano estendido em ambas as direções; assim como um objeto *time*, *datetime* presume que existem exatamente 3600*24 segundos em cada dia.

Construtor:

class datetime.**datetime**(*year*, *month*, *day*, *hour=0*, *minute=0*, *second=0*, *microsecond=0*, *tzinfo=None*, *, *fold=0*)

Os argumentos *year*, *month* e *day* são obrigatórios. *tzinfo* pode ser `None`, ou uma instância de subclasse de *tzinfo*. Os argumentos remanescentes devem ser inteiros nos seguintes intervalos:

- `MINYEAR <= year <= MAXYEAR`,
- `1 <= month <= 12`,
- `1 <= day <= número de dias no mês e ano fornecidos`,
- `0 <= hour < 24`,
- `0 <= minute < 60`,
- `0 <= second < 60`,
- `0 <= microsecond < 1000000`,
- `fold in [0, 1]`.

Se um argumento fora desses intervalos for fornecido, a exceção *ValueError* é levantada.

Alterado na versão 3.6: Adicionado o parâmetro *fold*.

Outros construtores, todos os métodos de classe.

classmethod datetime.**today**()

Retorna a data e hora local atual, com o atributo *tzinfo* definido para `None`.

Equivalente a:

```
datetime.fromtimestamp(time.time())
```

Veja também *now()*, *fromtimestamp()*.

Este método é funcionalmente equivalente a *now()*, mas sem um parâmetro `tz`.

classmethod datetime.**now**(*tz=None*)

Retorna a data e hora local atual.

Se o argumento opcional *tz* é `None` ou não especificado, isto é o mesmo que *today()*, mas, se possível, fornece mais precisão do que pode ser obtido indo por um registro de data e hora da função *time.time()* (por exemplo, isto pode ser possível em plataformas que fornecem a função C `gettimeofday()`).

Se *tz* não for `None`, ele deve ser uma instância de uma subclasse de *tzinfo*, e a data e hora local atual são convertidas para o fuso horário de *tz*.

Esta função é preferida ao invés de *today()* e *utcnow()*.

> **Nota**
>
> Chamadas subsequentes para `datetime.now()` podem retornar o mesmo instante, dependendo da precisão do relógio subjacente.

classmethod datetime.**utcnow**()

Retorna a data e hora atual em UTC, com *tzinfo* como `None`.

Este é similar a *now()*, mas retorna a data e hora atual em UTC, como um objeto *datetime* ingênuo. Um datetime UTC consciente pode ser obtido chamando `datetime.now(timezone.utc)`. Veja também *now()*.

> **Aviso**
>
> Devido ao fato de objetos `datetime` ingênuos serem tratados por muitos métodos `datetime` como hora local, é preferível usar datetimes conscientes para representar horas em UTC. De tal forma, a maneira recomendada para criar um objeto representando a hora local em UTC é chamando `datetime.now(timezone.utc)`.

Descontinuado desde a versão 3.12: Use *datetime.now()* com *UTC*.

classmethod datetime.**fromtimestamp**(*timestamp*, *tz=None*)

Retorna a data e hora local correspondente ao registro de data e hora POSIX, como é retornado por *time.time()*. Se o argumento opcional *tz* é `None` ou não especificado, o registro de data e hora é convertido para a data e hora local da plataforma, e o objeto *datetime* retornado é ingênuo.

Se *tz* não for `None`, ela deve ser uma instância de uma subclasse de *tzinfo*, e o registro de data e hora é convertido para o fuso horário de *tz*.

fromtimestamp() pode levantar *OverflowError*, se o registro de data e hora estiver fora do intervalo de valores suportados pelas funções em C `localtime()` ou `gmtime()` da plataforma, e ocorrer uma falha de *OSError* em `localtime()` ou `gmtime()`. É comum para isso ser restrito aos anos 1970 até 2038. Perceba que em sistemas não-POSIX que incluem segundos bissextos na sua notação de registro de data e hora, segundos bissextos são ignorados por *fromtimestamp()*, e então é possível ter dois registros de data e hora com diferença de um segundo que apresentam objetos *datetime* idênticos. Este método é preferido sobre *utcfromtimestamp()*.

Alterado na versão 3.3: Levanta um *OverflowError* ao invés de *ValueError* se o registro de data e hora estiver fora do intervalo dos valores suportados pelas funções C `localtime()` ou `gmtime()` da plataforma. Levanta *OSError* ao invés de *ValueError* em falhas de `localtime()` ou `gmtime()`.

Alterado na versão 3.6: *fromtimestamp()* pode retornar instâncias com *fold* igual a 1.

classmethod datetime.**utcfromtimestamp**(*timestamp*)

Retorna o *datetime* UTC correspondente ao registro de data e hora POSIX, com *tzinfo* setado para `None`. (O objeto resultante é ingênuo.)

Isso pode levantar *OverflowError*, se o registro de data e hora estiver fora do intervalo de valores suportados pela função C `gmtime()` da plataforma, e em caso de falha *OSError* em `gmtime()`. É comum que isso seja restrito a anos de 1970 a 2038.

Para conseguir um objeto *datetime* consciente, chame *fromtimestamp()*:

```
datetime.fromtimestamp(timestamp, timezone.utc)
```

Nas plataformas compatíveis com POSIX, é equivalente à seguinte expressão:

```
datetime(1970, 1, 1, tzinfo=timezone.utc) + timedelta(seconds=timestamp)
```

com a exceção de que a última fórmula sempre dá suporte ao intervalo completo de anos: entre *MINYEAR* e *MAXYEAR* inclusive.

> **Aviso**
>
> Devido ao fato de objetos `datetime` ingênuos serem tratados por muitos métodos `datetime` como hora local, é preferível usar datetimes conscientes para representar horas em UTC. De tal forma, a maneira recomendada para criar um objeto representando um registro de data e hora específico em UTC é chamando `datetime.fromtimestamp(timestamp, tz=timezone.utc)`.

Alterado na versão 3.3: Levanta *OverflowError* ao invés de *ValueError* se o registro de data e hora estiver fora do intervalo de valores suportados pela função C `gmtime()` da plataforma. Levanta *OSError* ao invés de *ValueError* em caso de falha `gmtime()`.

Descontinuado desde a versão 3.12: Use *datetime.fromtimestamp()* com *UTC*.

classmethod datetime.**fromordinal**(*ordinal*)

Retorna um *datetime* correspondente ao ordinal proléptico Gregoriano, onde 1º de janeiro do ano 1 tem ordinal 1. *ValueError* é levantado a não ser que `1 <= ordinal <= datetime.max.toordinal()`. As horas, minutos, segundos e micro segundos do resultado são todos 0, e *tzinfo* é `None`.

classmethod datetime.**combine**(*date*, *time*, *tzinfo=time.tzinfo*)

Retorna um novo objeto *datetime* no qual os componentes de data são iguais ao objeto *date* fornecido, e nos quais os componentes de hora são iguais ao do objeto *time* fornecido. Se o argumento *tzinfo* é fornecido, seu valor é usado para definir o atributo *tzinfo* do resultado, caso contrário o atributo *tzinfo* do argumento *time* é usado. Se o argumento *date* é um objeto *datetime*, seus componentes de hora e atributos *tzinfo* são ignorados.

Para qualquer objeto *datetime* `d`, `d == datetime.combine(d.date(), d.time(), d.tzinfo)`.

Alterado na versão 3.6: Adicionado o argumento *tzinfo*.

classmethod datetime.**fromisoformat**(*date_string*)

Retorna um *datetime* correspondendo a *date_string* em qualquer formato válido de ISO 8601, com as seguintes exceções:

1. Os deslocamentos de fuso horário podem ter segundos fracionários.
2. O separador `T` pode ser substituído por qualquer caractere unicode único.
3. Horas e minutos fracionários não são suportados.
4. Datas de precisão reduzida não são atualmente suportadas (`YYYY-MM`, `YYYY`).
5. Representações de data estendidas não são atualmente suportadas (`±YYYYYY-MM-DD`).
6. Atualmente, as datas ordinais não são suportadas (`YYYY-OOO`).

Exemplos:

```
>>> from datetime import datetime
>>> datetime.fromisoformat('2011-11-04')
datetime.datetime(2011, 11, 4, 0, 0)
>>> datetime.fromisoformat('20111104')
datetime.datetime(2011, 11, 4, 0, 0)
>>> datetime.fromisoformat('2011-11-04T00:05:23')
datetime.datetime(2011, 11, 4, 0, 5, 23)
>>> datetime.fromisoformat('2011-11-04T00:05:23Z')
datetime.datetime(2011, 11, 4, 0, 5, 23, tzinfo=datetime.timezone.utc)
>>> datetime.fromisoformat('20111104T000523')
datetime.datetime(2011, 11, 4, 0, 5, 23)
>>> datetime.fromisoformat('2011-W01-2T00:05:23.283')
datetime.datetime(2011, 1, 4, 0, 5, 23, 283000)
>>> datetime.fromisoformat('2011-11-04 00:05:23.283')
datetime.datetime(2011, 11, 4, 0, 5, 23, 283000)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> datetime.fromisoformat('2011-11-04 00:05:23.283+00:00')
datetime.datetime(2011, 11, 4, 0, 5, 23, 283000, tzinfo=datetime.timezone.utc)
>>> datetime.fromisoformat('2011-11-04T00:05:23+04:00')
datetime.datetime(2011, 11, 4, 0, 5, 23,
    tzinfo=datetime.timezone(datetime.timedelta(seconds=14400)))
```

Adicionado na versão 3.7.

Alterado na versão 3.11: Anteriormente, este método suportava apenas formatos que podiam ser emitidos por *date.isoformat()* ou *datetime.isoformat()*.

classmethod datetime.**fromisocalendar**(*year*, *week*, *day*)

Retorna um *datetime* correspondente à data no calendário ISO especificada por *year*, *week* e *day*. Os componentes não-data do datetime são preenchidos normalmente com seus valores padrões. Isso é o inverso da função *datetime.isocalendar()*.

Adicionado na versão 3.8.

classmethod datetime.**strptime**(*date_string*, *format*)

Retorna um *datetime* correspondente ao *date_string*, analisado de acordo com *format*.

Se *format* não contiver microssegundos ou informações de fuso horário, isso é equivalente a:

```
datetime(*(time.strptime(date_string, format)[0:6]))
```

ValueError é levantado se o date_string e o format não puderem ser interpretados por *time.strptime()* ou se ele retorna um valor o qual não é uma tupla temporal. Veja também *Comportamento de strftime() e strptime()* e *datetime.fromisoformat()*.

Alterado na versão 3.13: Se *format* especificar um dia do mês sem um ano, um *DeprecationWarning* agora é emitido. Isso é para evitar um bug de ano bissexto quadrienal no código que busca analisar apenas um mês e dia, pois o ano padrão usado na ausência de um no formato não é um ano bissexto. Esses valores de *format* podem gerar um erro a partir do Python 3.15. A solução alternativa é sempre incluir um ano em seu *format*. Se estiver analisando valores de *date_string* que não têm um ano, adicione explicitamente um ano que seja um ano bissexto antes da análise:

```
>>> from datetime import datetime
>>> date_string = "02/29"
>>> when = datetime.strptime(f"{date_string};1984", "%m/%d;%Y")  # Avoids leap␣
↪year bug.
>>> when.strftime("%B %d")
'February 29'
```

Atributos de classe:

datetime.**min**

O primeiro *datetime* representável, datetime(MINYEAR, 1, 1, tzinfo=None).

datetime.**max**

O último *datetime* representável, datetime(MAXYEAR, 12, 31, 23, 59, 59, 999999, tzinfo=None).

datetime.**resolution**

A menor diferença possível entre objetos *datetime* diferentes, timedelta(microseconds=1).

Atributos de instância (somente leitura):

datetime.**year**

Entre *MINYEAR* e *MAXYEAR* incluindo extremos.

`datetime.`**`month`**

Entre 1 e 12 incluindo extremos.

`datetime.`**`day`**

Entre 1 e o número de dias no mês especificado do ano especificado.

`datetime.`**`hour`**

No intervalo `range(24)`.

`datetime.`**`minute`**

No intervalo `range(60)`.

`datetime.`**`second`**

No intervalo `range(60)`.

`datetime.`**`microsecond`**

No intervalo `range(1000000)`.

`datetime.`**`tzinfo`**

O objeto passado como o argumento *tzinfo* do construtor *datetime*, ou `None` se nada foi passado.

`datetime.`**`fold`**

Entre `[0, 1]`. Usado para desambiguar tempos reais durante um intervalo repetido. (Um intervalo repetido ocorre quando relógios são atrasados ao final do horário de verão ou quando a diferença UTC para o fuso horário atual é reduzida por razões políticas.) Os valores 0 e 1 representam, respectivamente, o primeiro e o segundo dos dois momentos com a mesma representação de tempo real.

Adicionado na versão 3.6.

Operações suportadas:

Operação	Resultado
`datetime2 = datetime1 + timedelta`	(1)
`datetime2 = datetime1 - timedelta`	(2)
`timedelta = datetime1 - datetime2`	(3)
`datetime1 == datetime2` `datetime1 != datetime2`	Comparação de igualdade. (4)

`datetime1 < datetime2`
`datetime1 > datetime2`
`datetime1 <= datetime2`
`datetime1 >= datetime2`
	Comparação de ordem. (5)

(1) `datetime2` representa a duração de `timedelta` removido de `datetime1`, movendo o tempo para frente se `timedelta.days > 0`, ou para trás se `timedelta.days < 0`. O resultado tem o mesmo atributo *tzinfo* que o datetime de entrada, e `datetime2 - datetime1 == timedelta` após. *OverflowError* é levantada se `datetime2.year` fosse menor que *MINYEAR* ou maior que *MAXYEAR*. Perceba que nenhum ajuste no fuso horário é feito mesmo se a entrada é um objeto consciente disso.

(2) Computa o `datetime2` tal que `datetime2 + timedelta == datetime1`. Assim como para adição, o resultado tem o mesmo atributo *tzinfo* que datetime de entrada, e nenhum ajuste de fuso horário é feito mesmo que a entrada seja consciente disso.

(3) Subtração de um *datetime* de outro *datetime* é definido apenas se ambos os operandos são ingênuos, ou se ambos são conscientes. Se um deles é consciente e o outro é ingênuo, *TypeError* é levantado.

Se ambos são ingênuos, ou ambos são conscientes e tiverem o mesmo atributo *tzinfo*, os atributos *tzinfo* são ignorados, e o resultado é um objeto `t` do tipo *timedelta*, tal que `datetime2 + t == datetime1`. Nenhum ajuste de fuso horário é feito neste caso.

Se ambas são conscientes e têm atributos *tzinfo* diferentes, `a-b` age como se `a` e `b` tivessem sido primeiro convertidos para datetimes ingênuas em UTC. O resultado é `(a.replace(tzinfo=None) - a.utcoffset()) - (b.replace(tzinfo=None) - b.utcoffset())` exceto que a implementação nunca estoura.

(4) Objetos *datetime* são iguais se representarem a mesma data e hora, levando em consideração o fuso horário.

Objetos `datetime` ingênuos e conscientes nunca são iguais.

Se ambos os comparandos forem conscientes e tiverem o mesmo atributo `tzinfo`, os atributos `tzinfo` e *fold* serão ignorados e os datetimes base serão comparados. Se ambos os comparandos forem conscientes e tiverem atributos *tzinfo* diferentes, a comparação atua como se os comparandos fossem primeiro convertidos para datetimes UTC, exceto que a implementação nunca estoura. Instâncias `datetime` em um intervalo repetido nunca são iguais a instâncias `datetime` em outro fuso horário.

(5) *datetime1* é considerado menor que *datetime2* quando *datetime1* precede *datetime2* no tempo. levando em consideração o fuso horário.

A comparação de ordens entre objetos *datetime* ingênuos e conscientes levanta *TypeError*.

Se ambos os comparandos forem conscientes e tiverem o mesmo atributo `tzinfo`, os atributos `tzinfo` e *fold* serão ignorados e os datetimes base serão comparados. Se ambos os comparandos forem conscientes e tiverem atributos *tzinfo* diferentes, a comparação atua como se os comparandos fossem primeiro convertidos para datetimes UTC, exceto que a implementação nunca estoura.

Alterado na versão 3.3: Comparações de igualdade entre instâncias de *datetime* conscientes e nativas não levantam *TypeError*.

Alterado na versão 3.13: A comparação entre o objeto *datetime* e uma instância da subclasse *date* que não é uma subclasse `datetime` não converte mais a última para `date`, ignorando o parte horária e o fuso horário. O comportamento padrão pode ser alterado substituindo os métodos especiais de comparação nas subclasses.

Métodos de instância:

`datetime.`**`date`**`()`

Retorna um objeto *date* com o mesmo ano, mês e dia.

`datetime.`**`time`**`()`

Retorna um objeto *time* com a mesma hora, minuto, segundo, microssegundo e fold. O atributo *tzinfo* é `None`. Veja também o método *timetz()*.

Alterado na versão 3.6: O valor fold é copiado para o objeto *time* retornado.

`datetime.`**`timetz`**`()`

Retorna um objeto *time* com os mesmos atributos de hora, minuto, segundo, microssegundo, fold e tzinfo. Veja também o método *time()*.

Alterado na versão 3.6: O valor fold é copiado para o objeto *time* retornado.

`datetime.`**`replace`**`(`*year=self.year*, *month=self.month*, *day=self.day*, *hour=self.hour*, *minute=self.minute*, *second=self.second*, *microsecond=self.microsecond*, *tzinfo=self.tzinfo*, ***, *fold=0*`)`

Retorna um novo objeto *datetime* com os mesmos atributos, mas com os parâmetros especificados atualizados. Perceba que `tzinfo=None` pode ser especificado para criar um datetime ingênuo a partir de um datetime consciente, sem conversão de dados da data ou hora.

Objetos *datetime* também são suportados pela função genérica *copy.replace()*.

Alterado na versão 3.6: Adicionado o parâmetro *fold*.

`datetime.`**`astimezone`**`(`*tz=None*`)`

Retorna um objeto *datetime* com novo atributo *tzinfo* definido por *tz*, ajustando a data e hora de forma que o resultado seja o mesmo horário UTC que *self*, mas na hora local de *tz*.

Se fornecido, *tz* deve ser uma instância de uma subclasse *tzinfo*, e seus métodos *utcoffset()* e *dst()* não devem retornar None. Se *self* for ingênuo, é presumido que ele representa o tempo no fuso horário do sistema.

Se for chamado sem argumentos (ou com tz=None) o fuso horário do sistema local é presumido como o fuso horário desejado. O atributo .tzinfo da instância datetime convertida será definido para uma instância de *timezone* com o nome da zona e um deslocamento obtido a partir do sistema operacional.

Se self.tzinfo for *tz*, self.astimezone(tz) é igual a *self*: nenhum ajuste nos dados de data ou hora é realizado. Caso contrário o resultado é a hora local no fuso horário *tz*, representando a mesma hora UTC que *self*: depois astz = dt.astimezone(tz), astz - astz.utcoffset() terá os mesmos dados de data e hora que dt - dt.utcoffset().

Se você quer meramente anexar um objeto *timezone* *tz* a um datetime *dt* sem ajustes de dados de data e hora, use dt.replace(tzinfo=tz). Se você meramente quer remover o objeto timezone de um datetime consciente *dt* sem conversão de dados de data e hora, use dt.replace(tzinfo=None).

Perceba que o método padrão *tzinfo.fromutc()* pode ser substituído em uma subclasse *tzinfo* para afetar o resultado retornado por *astimezone()*. Ignorando erros de letras maiúsculas/minúsculas, *astimezone()* funciona como:

```
def astimezone(self, tz):
    if self.tzinfo is tz:
        return self
    # Converte self para UTC e anexa o novo objeto timezone.
    utc = (self - self.utcoffset()).replace(tzinfo=tz)
    # Converte o horário local de UTC para tz.
    return tz.fromutc(utc)
```

Alterado na versão 3.3: *tz* agora pode ser omitido.

Alterado na versão 3.6: O método *astimezone()* agora pode ser chamado em instâncias ingênuas que presumidamente representam a hora local do sistema.

datetime.**utcoffset**()

Se *tzinfo* for None, retorna None, caso contrário retorna self.tzinfo.utcoffset(self), e levanta uma exceção se o segundo não retornar None ou um objeto *timedelta* com magnitude menor que um dia.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

datetime.**dst**()

Se *tzinfo* for None, retorna None, caso contrário retorna self.tzinfo.dst(self), e levanta uma exceção se o segundo não retornar None ou um objeto *timedelta* com magnitude menor que um dia.

Alterado na versão 3.7: A diferença de horário de verão não é restrita a um número completo de minutos.

datetime.**tzname**()

Se *tzinfo* for None, retorna None, caso contrário retorna self.tzinfo.tzname(self), levanta uma exceção se o segundo não retornar None ou um objeto string.

datetime.**timetuple**()

Retorna uma *time.struct_time* tal como retornado por *time.localtime()*.

d.timetuple() é equivalente a:

```
time.struct_time((d.year, d.month, d.day,
                  d.hour, d.minute, d.second,
                  d.weekday(), yday, dst))
```

onde yday = d.toordinal() - date(d.year, 1, 1).toordinal() + 1 é o número de dias dentro do ano atual, começando com 1 para 1º de janeiro. O sinalizador *tm_isdst* do resultado é definido conforme o método *dst()*: *tzinfo* é None ou *dst()* retorna None, tm_isdst é definido para -1; caso contrário se *dst()* retornar um valor diferente de zero, tm_isdst é definido para 1; caso contrário tm_isdst é definido para 0.

datetime.**utctimetuple**()

Se a instância *datetime* `d` é ingênua, isto é o mesmo que `d.timetuple()`, exceto que *tm_isdst* é forçado para 0, independentemente do que `d.dst()` retornar. O horário de verão nunca é afetado por um horário UTC.

Se `d` é consciente, `d` é normalizado para horário UTC, subtraindo `d.utcoffset()`, e um *time.struct_time* para o horário normalizado é retornado. `tm_isdst` é forçado para 0. Observe que uma exceção *OverflowError* pode ser levantada se `d.year` for `MINYEAR` ou `MAXYEAR` e os ajustes UTC ultrapassar o limite de um ano.

Aviso

Por causa que objetos `datetime` ingênuos são tratados por muitos métodos `datetime` como hora local, é preferido usar datetimes conscientes para representar horários em UTC; como resultado, usar *datetime.utctimetuple()* pode dar resultados enganosos. Se você tem um `datetime` ingênuo representando UTC, use `datetime.replace(tzinfo=timezone.utc)` para torná-lo consciente, ponto no qual você pode usar *datetime.timetuple()*.

datetime.**toordinal**()

Retorna o ordinal proléptico gregoriano da data. o mesmo que `self.date().toordinal()`.

datetime.**timestamp**()

Retorna o registro de data e hora POSIX correspondente a instância *datetime*. O valor de retorno é um *float* similar aquele retornado por *time.time()*.

Presume-se que instâncias *datetime* ingênuas representam a hora local e este método depende da função C `mktime()` da plataforma para realizar a conversão. Como *datetime* suporta um intervalo maior de valores que `mktime()` em muitas plataformas, este método pode levantar *OverflowError* ou *OSError* para horários longe no passado ou longe no futuro.

Para instâncias conscientes de *datetime*, o valor retornado é computado como:

```
(dt - datetime(1970, 1, 1, tzinfo=timezone.utc)).total_seconds()
```

Adicionado na versão 3.3.

Alterado na versão 3.6: O método *timestamp()* usa o atributo *fold* para desambiguar os tempos durante um intervalo repetido.

Nota

Não existe método para obter o timestamp POSIX diretamente de uma instância *datetime* ingênua representando tempo em UTC. Se a sua aplicação usa esta convenção e o fuso horário do seu sistema não está setado para UTC, você pode obter o registro de data e hora POSIX fornecendo `tzinfo=timezone.utc`:

```
timestamp = dt.replace(tzinfo=timezone.utc).timestamp()
```

ou calculando o registro de data e hora diretamente:

```
timestamp = (dt - datetime(1970, 1, 1)) / timedelta(seconds=1)
```

datetime.**weekday**()

Retorna o dia da semana como um inteiro, em que segunda-feira é 0 e domingo é 6. O mesmo que `self.date().weekday()`. Veja também *isoweekday()*.

datetime.**isoweekday**()

Retorna o dia da semana como um inteiro, em que segunda-feira é 1 e domingo é 7. O mesmo que `self.date().isoweekday()`. Veja também *weekday()*, *isocalendar()*.

datetime.**isocalendar**()

Retorna uma *tupla nomeada* com três componentes: `year`, `week` e `weekday`. O mesmo que `self.date().isocalendar()`.

datetime.**isoformat**(*sep='T'*, *timespec='auto'*)

Retorna uma string representando a data e o tempo no formato ISO 8601:

- `YYYY-MM-DDTHH:MM:SS.ffffff`, se *microsecond* não é 0
- `YYYY-MM-DDTHH:MM:SS`, se *microsecond* é 0

Se *utcoffset()* não retorna `None`, uma string é adicionada com a diferença UTC:

- `YYYY-MM-DDTHH:MM:SS.ffffff+HH:MM[:SS[.ffffff]]`, se *microsecond* não é 0
- `YYYY-MM-DDTHH:MM:SS+HH:MM[:SS[.ffffff]]`, se *microsecond* é 0

Exemplos:

```
>>> from datetime import datetime, timezone
>>> datetime(2019, 5, 18, 15, 17, 8, 132263).isoformat()
'2019-05-18T15:17:08.132263'
>>> datetime(2019, 5, 18, 15, 17, tzinfo=timezone.utc).isoformat()
'2019-05-18T15:17:00+00:00'
```

O argumento opcional *sep* (por padrão, `'T'`) é um separador de caractere único, colocado entre as porções de data e tempo do resultado. Por exemplo:

```
>>> from datetime import tzinfo, timedelta, datetime
>>> class TZ(tzinfo):
...     """Um fuso horário com uma posição arbitrária constante de -06:39."""
...     def utcoffset(self, dt):
...         return timedelta(hours=-6, minutes=-39)
...
>>> datetime(2002, 12, 25, tzinfo=TZ()).isoformat(' ')
'2002-12-25 00:00:00-06:39'
>>> datetime(2009, 11, 27, microsecond=100, tzinfo=TZ()).isoformat()
'2009-11-27T00:00:00.000100-06:39'
```

O argumento opcional *timespec* especifica o número de componentes adicionais do tempo a incluir (o padrão é `'auto'`). Pode ser uma das seguintes strings:

- `'auto'`: O mesmo que `'seconds'` se *microsecond* é 0, o mesmo que `'microseconds'` caso contrário.
- `'hours'`: Inclui o atributo *hour* no formato de dois dígitos `HH`.
- `'minutes'`: Inclui os atributos *hour* e *minute* no formato `HH:MM`.
- `'seconds'`: Inclui os atributos *hour*, *minute* e *second* no formato `HH:MM:SS`.
- `'milliseconds'`: Inclui o tempo completo, mas trunca a parte fracional dos segundos em milissegundos. Formato `HH:MM:SS.sss`.
- `'microseconds'`: Inclui o tempo completo no formato `HH:MM:SS.ffffff`.

Nota

Componentes do tempo excluídos são truncados, não arredondados.

A exceção *ValueError* vai ser levantada no caso de um argumento *timespec* inválido:

```
>>> from datetime import datetime
>>> datetime.now().isoformat(timespec='minutes')
'2002-12-25T00:00'
>>> dt = datetime(2015, 1, 1, 12, 30, 59, 0)
>>> dt.isoformat(timespec='microseconds')
'2015-01-01T12:30:59.000000'
```

Alterado na versão 3.6: Adicionado o parâmetro *timespec*.

`datetime.__str__()`

Para uma instância *datetime* `d`, `str(d)` é equivalente a `d.isoformat(' ')`.

`datetime.ctime()`

Retorna uma string representando a data e hora:

```
>>> from datetime import datetime
>>> datetime(2002, 12, 4, 20, 30, 40).ctime()
'Wed Dec  4 20:30:40 2002'
```

A string de saída *não* irá incluir informações de fuso horário, independente de a entrada ser consciente ou ingênua.

`d.ctime()` é equivalente a:

```
time.ctime(time.mktime(d.timetuple()))
```

em plataformas onde a função nativa em C `ctime()` (a qual *time.ctime()* invoca, mas a qual *datetime.ctime()* não invoca) conforma com o padrão C.

`datetime.strftime(`*format*`)`

Retorna uma string representando a data e hora, controladas por uma string com formato explícito. Veja também *Comportamento de strftime() e strptime()* e *datetime.isoformat()*.

`datetime.__format__(`*format*`)`

O mesmo que *datetime.strftime()*. Isso torna possível especificar uma string de formatação para o objeto *datetime* em literais de string formatados e ao usar *str.format()*. Veja também *Comportamento de strftime() e strptime()* e *datetime.isoformat()*.

Exemplos de uso: `datetime`

Exemplos para trabalhar com objetos *datetime*:

```
>>> from datetime import datetime, date, time, timezone

>>> # Usando datetime.combine()
>>> d = date(2005, 7, 14)
>>> t = time(12, 30)
>>> datetime.combine(d, t)
datetime.datetime(2005, 7, 14, 12, 30)

>>> # Usando datetime.now()
>>> datetime.now()
datetime.datetime(2007, 12, 6, 16, 29, 43, 79043)   # GMT +1
>>> datetime.now(timezone.utc)
datetime.datetime(2007, 12, 6, 15, 29, 43, 79060, tzinfo=datetime.timezone.utc)

>>> # Usando datetime.strptime()
>>> dt = datetime.strptime("21/11/06 16:30", "%d/%m/%y %H:%M")
>>> dt
```

(continua na próxima página)

(continuação da página anterior)

```
datetime.datetime(2006, 11, 21, 16, 30)

>>> # Usando datetime.timetuple() para obter uma tupla de todos os atributos
>>> tt = dt.timetuple()
>>> for it in tt:
...     print(it)
...
2006    # ano
11      # mês
21      # dia
16      # hora
30      # minuto
0       # segundo
1       # dia da semana (0 = Segunda-feira)
325     # número de dias desde 1º de janeiro
-1      # dst - método tzinfo.dst() retornou None

>>> # Data em formato ISO
>>> ic = dt.isocalendar()
>>> for it in ic:
...     print(it)
...
2006    # ano ISO
47      # semana ISO
2       # dia da semana ISO

>>> # Formatando um datetime
>>> dt.strftime("%A, %d. %B %Y %I:%M%p")
'Tuesday, 21. November 2006 04:30PM'
>>> 'O {1} é {0:%d}, o {2} é {0:%B}, a {3} é {0:%I:%M%p}.'.format(dt, "dia", "mês",
↪ "hora")
'O dia é 21, o mês é November, a hora é 04:30PM.'
```

O exemplo abaixo define uma subclasse *tzinfo* capturando informações de fuso horário para Kabul, Afeganistão, o qual usou +4 UTC até 1945 e depois +4:30 UTC após esse período:

```
from datetime import timedelta, datetime, tzinfo, timezone

class KabulTz(tzinfo):
    # Kabul usou +4 até 1945, quando eles moveram para +4:30
    UTC_MOVE_DATE = datetime(1944, 12, 31, 20, tzinfo=timezone.utc)

    def utcoffset(self, dt):
        if dt.year < 1945:
            return timedelta(hours=4)
        elif (1945, 1, 1, 0, 0) <= dt.timetuple()[:5] < (1945, 1, 1, 0, 30):
            # Um intervalo de meia hora ambíguo ("imaginário") representando
            # um 'fold' no tempo por causa da troca de +4 para +4:30.
            # Se dt cair no intervalo imaginário, usa a fold para decidir como
            # para resolver. Veja PEP495.
            return timedelta(hours=4, minutes=(30 if dt.fold else 0))
        else:
            return timedelta(hours=4, minutes=30)

    def fromutc(self, dt):
        # Segue as mesmas validações como em datetime.tzinfo
```

(continua na próxima página)

(continuação da página anterior)

```
        if not isinstance(dt, datetime):
            raise TypeError("fromutc() requer um argumento datetime")
        if dt.tzinfo is not self:
            raise ValueError("dt.tzinfo não é self")

        # Uma implementação personalizada é necessária para fromutc
        # como a entrada para esta função é um datetime com valores utc
        # mas com um tzinfo definido para self.
        # Veja datetime.astimezone ou fromtimestamp.
        if dt.replace(tzinfo=timezone.utc) >= self.UTC_MOVE_DATE:
            return dt + timedelta(hours=4, minutes=30)
        else:
            return dt + timedelta(hours=4)

    def dst(self, dt):
        # Kabul não tem horário de verão.
        return timedelta(0)

    def tzname(self, dt):
        if dt >= self.UTC_MOVE_DATE:
            return "+04:30"
        return "+04"
```

Uso de `KabulTz` mostrado acima:

```
>>> tz1 = KabulTz()

>>> # Datetime antes da alteração
>>> dt1 = datetime(1900, 11, 21, 16, 30, tzinfo=tz1)
>>> print(dt1.utcoffset())
4:00:00

>>> # Datetime após a alteração
>>> dt2 = datetime(2006, 6, 14, 13, 0, tzinfo=tz1)
>>> print(dt2.utcoffset())
4:30:00

>>> # Converte datetime para outro fuso horário
>>> dt3 = dt2.astimezone(timezone.utc)
>>> dt3
datetime.datetime(2006, 6, 14, 8, 30, tzinfo=datetime.timezone.utc)
>>> dt2
datetime.datetime(2006, 6, 14, 13, 0, tzinfo=KabulTz())
>>> dt2 == dt3
True
```

8.1.7 Objetos `time`

Um objeto *time* representa a hora (local) do dia, independente de qualquer dia em particular, e sujeito a ajustes através de um objeto *tzinfo*.

class datetime.**time**(*hour=0*, *minute=0*, *second=0*, *microsecond=0*, *tzinfo=None*, ***, *fold=0*)

Todos os argumentos são opcionais. *tzinfo* pode ser `None`, ou uma instância de uma subclasse de *tzinfo*. Os argumentos remanescentes devem ser inteiros nos seguintes intervalos:

- `0 <= hour < 24`,
- `0 <= minute < 60`,

- `0 <= second < 60`,
- `0 <= microsecond < 1000000`,
- `fold in [0, 1]`.

Se um argumento fora desses intervalos é fornecido, *ValueError* é levantada. Todos tem como padrão o valor 0 exceto *tzinfo*, o qual tem o valor padrão `None`.

Atributos de classe:

`time.`**`min`**

O *time* mais cedo que pode ser representado, `time(0, 0, 0, 0)`.

`time.`**`max`**

O *time* mais tardio que pode ser representado, `time(23, 59, 59, 999999)`.

`time.`**`resolution`**

A menor diferença possível entre objetos *time* diferentes, `timedelta(microseconds=1)`, embora perceba que aritmética sobre objetos *time* não é suportada.

Atributos de instância (somente leitura):

`time.`**`hour`**

No intervalo `range(24)`.

`time.`**`minute`**

No intervalo `range(60)`.

`time.`**`second`**

No intervalo `range(60)`.

`time.`**`microsecond`**

No intervalo `range(1000000)`.

`time.`**`tzinfo`**

O objeto passado como argumento tzinfo para o construtor da classe *time*, ou `None` se nada foi passado.

`time.`**`fold`**

Entre `[0, 1]`. Usado para desambiguar tempos reais durante um intervalo repetido. (Um intervalo repetido ocorre quando relógios são atrasados ao final do horário de verão ou quando a diferença UTC para o fuso horário atual é reduzida por razões políticas.) Os valores 0 e 1 representam, respectivamente, o primeiro e o segundo dos dois momentos com a mesma representação de tempo real.

Adicionado na versão 3.6.

Objetos *time* oferecem suporte para comparações de igualdade e ordem, onde `a` é considerado menor que `b` quando `a` precede `b` no tempo.

Objetos `time` ingênuos e conscientes nunca são iguais. A comparação de ordem entre objetos `time` ingênuos e conscientes levanta *TypeError*.

Se ambos os comparandos são conscientes, e tem o mesmo atributo *tzinfo*, os atributos `tzinfo` e `fold` são ignorados e os tempos base são comparados. Se ambos os comparandos são conscientes e tem atributos `tzinfo` diferentes, os comparandos são primeiro ajustados subtraindo sua diferença em UTC (obtida através de `self.utcoffset()`).

Alterado na versão 3.3: Comparações de igualdade entre instâncias de *time* conscientes e nativas não levantam *TypeError*.

Em contextos Booleanos, um objeto *time* é sempre considerado como verdadeiro.

Alterado na versão 3.5: Antes do Python 3.5, um objeto *time* era considerado falso se ele representava meia-noite em UTC. Este comportamento era considerado obscuro e suscetível a erros, e foi removido no Python 3.5. Veja bpo-13936 para todos os detalhes.

Outros construtores:

classmethod time.**fromisoformat**(*time_string*)

Retorna um *time* correspondendo a *time_string* em qualquer formato válido de ISO 8601, com as seguintes exceções:

1. Os deslocamentos de fuso horário podem ter segundos fracionários.
2. O `T` inicial, normalmente exigido nos casos em que pode haver ambiguidade entre uma data e uma hora, não é necessário.
3. Segundos fracionários podem ter qualquer número de dígitos (algo além de 6 será truncado).
4. Horas e minutos fracionários não são suportados.

Exemplos:

```
>>> from datetime import time
>>> time.fromisoformat('04:23:01')
datetime.time(4, 23, 1)
>>> time.fromisoformat('T04:23:01')
datetime.time(4, 23, 1)
>>> time.fromisoformat('T042301')
datetime.time(4, 23, 1)
>>> time.fromisoformat('04:23:01.000384')
datetime.time(4, 23, 1, 384)
>>> time.fromisoformat('04:23:01,000384')
datetime.time(4, 23, 1, 384)
>>> time.fromisoformat('04:23:01+04:00')
datetime.time(4, 23, 1, tzinfo=datetime.timezone(datetime.
↪timedelta(seconds=14400)))
>>> time.fromisoformat('04:23:01Z')
datetime.time(4, 23, 1, tzinfo=datetime.timezone.utc)
>>> time.fromisoformat('04:23:01+00:00')
datetime.time(4, 23, 1, tzinfo=datetime.timezone.utc)
```

Adicionado na versão 3.7.

Alterado na versão 3.11: Anteriormente, este método suportava apenas formatos que podiam ser emitidos por *time.isoformat()*.

classmethod time.**strptime**(*date_string*, *format*)

Retorna um *time* correspondente ao *date_string*, analisado de acordo com *format*.

Se *format* não contiver microssegundos ou informações de fuso horário, isso é equivalente a:

```
time(*(time.strptime(date_string, format)[3:6]))
```

ValueError é levantado se o *date_string* e o *format* não puderem ser interpretados por *time.strptime()* ou se ele retorna um valor o qual não é uma tupla temporal. Veja também *Comportamento de strftime() e strptime()* e *time.fromisoformat()*.

Adicionado na versão 3.14.

Métodos de instância:

time.**replace**(*hour=self.hour*, *minute=self.minute*, *second=self.second*, *microsecond=self.microsecond*, *tzinfo=self.tzinfo*, ***, *fold=0*)

Retorna um novo *time* com os mesmos valores, mas com os parâmetros especificados atualizados. Perceba que `tzinfo=None` pode ser especificado para criar um *time* ingênuo a partir de um *time* consciente, sem conversão de dados do tempo.

Objetos *time* também são suportados pela função genérica *copy.replace()*.

Alterado na versão 3.6: Adicionado o parâmetro *fold*.

time.**isoformat**(*timespec='auto'*)

Retorna uma string representando a hora em formato ISO 8601, um destes:

- `HH:MM:SS.ffffff`, se *microsecond* não é 0
- `HH:MM:SS`, se *microsecond* é 0
- `HH:MM:SS.ffffff+HH:MM[:SS[.ffffff]]`, se *utcoffset()* não retorna `None`
- `HH:MM:SS+HH:MM[:SS[.ffffff]]`, se *microsecond* é 0 e *utcoffset()* não retorna `None`

O argumento opcional *timespec* especifica o número de componentes adicionais do tempo a incluir (o padrão é `'auto'`). Pode ser uma das seguintes strings:

- `'auto'`: O mesmo que `'seconds'` se *microsecond* é 0, o mesmo que `'microseconds'` caso contrário.
- `'hours'`: Inclui o atributo *hour* no formato de dois dígitos `HH`.
- `'minutes'`: Inclui os atributos *hour* e *minute* no formato `HH:MM`.
- `'seconds'`: Inclui os atributos *hour*, *minute* e *second* no formato `HH:MM:SS`.
- `'milliseconds'`: Inclui o tempo completo, mas trunca a parte fracional dos segundos em milissegundos. Formato `HH:MM:SS.sss`.
- `'microseconds'`: Inclui o tempo completo no formato `HH:MM:SS.ffffff`.

> **Nota**
>
> Componentes do tempo excluídos são truncados, não arredondados.

ValueError será levantado com um argumento *timespec* inválido.

Exemplo:

```
>>> from datetime import time
>>> time(hour=12, minute=34, second=56, microsecond=123456).isoformat(timespec=
↪'minutes')
'12:34'
>>> dt = time(hour=12, minute=34, second=56, microsecond=0)
>>> dt.isoformat(timespec='microseconds')
'12:34:56.000000'
>>> dt.isoformat(timespec='auto')
'12:34:56'
```

Alterado na versão 3.6: Adicionado o parâmetro *timespec*.

time.**__str__**()

Para um tempo `t`, `str(t)` é equivalente a `t.isoformat()`.

time.**strftime**(*format*)

Retorna uma string representando a hora, controladas por uma string com formato explícito. Veja também *Comportamento de strftime() e strptime()* e *time.isoformat()*.

time.**__format__**(*format*)

O mesmo que *time.strftime()*. Isso torna possível especificar uma string de formatação para o objeto *time* em literais de string formatados e ao usar *str.format()*. Veja também *Comportamento de strftime() e strptime()* e *time.isoformat()*.

time.**utcoffset**()

Se *tzinfo* for `None`, retorna `None`, caso contrário retorna `self.tzinfo.utcoffset(None)`, e levanta uma exceção se o segundo não retornar `None` ou um objeto *timedelta* com magnitude menor que um dia.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

time.**dst**()

Se *tzinfo* for None, retorna None, caso contrário retorna self.tzinfo.dst(None), e levanta uma exceção se o segundo não retornar None, ou um objeto *timedelta* com magnitude menor que um dia.

Alterado na versão 3.7: A diferença de horário de verão não é restrita a um número completo de minutos.

time.**tzname**()

Se *tzinfo* for None, retorna None, caso contrário retorna self.tzinfo.tzname(None), ou levanta uma exceção se o último caso não retornar None ou um objeto string.

Exemplos de uso: time

Exemplos para trabalhar com um objeto *time*:

```
>>> from datetime import time, tzinfo, timedelta
>>> class TZ1(tzinfo):
...     def utcoffset(self, dt):
...         return timedelta(hours=1)
...     def dst(self, dt):
...         return timedelta(0)
...     def tzname(self,dt):
...         return "+01:00"
...     def  __repr__(self):
...         return f"{self.__class__.__name__}()"
...
>>> t = time(12, 10, 30, tzinfo=TZ1())
>>> t
datetime.time(12, 10, 30, tzinfo=TZ1())
>>> t.isoformat()
'12:10:30+01:00'
>>> t.dst()
datetime.timedelta(0)
>>> t.tzname()
'+01:00'
>>> t.strftime("%H:%M:%S %Z")
'12:10:30 +01:00'
>>> 'A {} é {:%H:%M}.'.format("hora", t)
'A hora é 12:10.'
```

8.1.8 Objetos tzinfo

class datetime.**tzinfo**

Esta é uma classe base abstrata, o que significa que esta classe não deve ser instanciada diretamente. Defina uma subclasse de *tzinfo* para capturar informações sobre um fuso horário em particular.

Uma instância de (uma subclasse concreta de) *tzinfo* pode ser passada para os construtores de objetos *datetime* e *time*. Os objetos time veem seus atributos como se estivessem em horário local, e o objeto *tzinfo* suporta métodos revelando a diferença da hora local a partir de UTC, o nome do fuso horário, e diferença de horário em horário de verão, todos relativos ao objeto date ou time passado para eles.

Você precisa derivar uma subclasse concreta, e (pelo menos) fornecer implementações dos métodos padrões de *tzinfo* necessários pelos métodos *datetime* que você usa. O módulo datetime fornece *timezone*, uma subclasse concreta simples de *tzinfo*, que pode representar fusos horários com diferença fixa a partir de UTC, tais como o próprio UTC, ou EST (Eastern Standard Time ou Horário padrão oriental) e EDT (Eastern Daylight Time ou Horário de verão oriental) na América do Norte.

Requisito especial para preservação: uma subclasse *tzinfo* deve ter um método __init__() que pode ser chamado sem nenhum argumento, caso contrário ele pode ser conservado, mas não alterado novamente. Isso é um requisito técnico que pode ser relaxado no futuro.

Uma subclasse concreta de *tzinfo* pode precisar implementar os seguintes métodos. Exatamente quais métodos são necessários depende do uso feito de objetos `datetime` conscientes. Se estiver em dúvida, simplesmente implemente todos eles.

`tzinfo.`**`utcoffset`**`(`*dt*`)`

Retorna a diferença da hora local a partir do UTC, como um objeto *timedelta*, que é positivo a leste do UTC. Se a hora local está a oeste do UTC, isto deve ser negativo.

Isto representa a diferença *total* a partir de UTC; por exemplo, se um objeto *tzinfo* representa fuso horário e ajustes de horário de verão, *utcoffset()* deve retornar a soma deles. Se a diferença UTC não é conhecida, retorna `None`. Caso contrário o valor retornado deve ser um objeto *timedelta* estritamente contido entre `-timedelta(hours=24)` e `timedelta(hours=24)` (a magnitude da diferença deve ser menor que um dia). A maior parte das implementações de *utcoffset()* irá provavelmente parecer com um destes dois:

```
return CONSTANT                 # classe de posição fixa
return CONSTANT + self.dst(dt)  # classe consciente de horário de verão
```

Se *utcoffset()* não retorna `None`, *dst()* também não deve retornar `None`.

A implementação padrão de *utcoffset()* levanta *NotImplementedError*.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

`tzinfo.`**`dst`**`(`*dt*`)`

Retorna o ajuste para o horário de verão (DST - daylight saving time), como um objeto *timedelta* ou `None` se informação para o horário de verão é desconhecida.

Retorna `timedelta(0)` se o horário de verão não estiver ativo. Se o horário de verão estiver ativo, retorna a diferença como um objeto *timedelta* (veja *utcoffset()* para detalhes). Perceba que a diferença do horário de verão, se aplicável, já foi adicionada a diferença UTC retornada por *utcoffset()*, então não existe necessidade de consultar *dst()* a não ser que você esteja interessado em obter a informação de horário de verão separadamente. Por exemplo, *datetime.timetuple()* chama o método *dst()* do seu atributo *tzinfo* para determinar como o flag *tm_isdst* deve ser definido, e *tzinfo.fromutc()* chama *dst()* para contabilizar as mudanças de horário de verão quando ocorrem mudanças de fuso horário.

Uma instância *tz* de uma subclasse *tzinfo* que modela tanto horário padrão quanto horário de verão deve ser consistente neste sentido:

`tz.utcoffset(dt) - tz.dst(dt)`

deve retornar o mesmo resultado para cada *datetime* *dt* com `dt.tzinfo == tz` para subclasses *tzinfo* sãs, esta expressão produz a "diferença padrão" do fuso horário, a qual não deve depender de data ou hora, mas apenas de localização geográfica. A implementação de *datetime.astimezone()* depende disso, mas não pode detectar violações; é responsabilidade do programador garantir isso. Se uma subclasse *tzinfo* não pode garantir isso, ele pode ser capaz de substituir a implementação padrão de *tzinfo.fromutc()* para funcionar corretamente com *astimezone()* independente disso.

Maior parte das implementações de *dst()* provavelmente irá parecer com um destes dois:

```
def dst(self, dt):
    # uma classe de posição fixa: não levam em conta DST
    return timedelta(0)
```

ou:

```
def dst(self, dt):
    # Código para definir dston e dstoff para os horários
    # de transição do horário de versão com base
    # no dt.year de entrada e expresso no horário local padrão.

    if dston <= dt.replace(tzinfo=None) < dstoff:
        return timedelta(hours=1)
```

(continua na próxima página)

(continuação da página anterior)

```
    else:
        return timedelta(0)
```

A implementação padrão de *dst()* levanta *NotImplementedError*.

Alterado na versão 3.7: A diferença de horário de verão não é restrita a um número completo de minutos.

tzinfo.**tzname**(*dt*)

Retorna o nome do fuso horário correspondente ao objeto *datetime* *dt*, como uma string. Nada sobre nomes de strings é definido pelo módulo `datetime`, e não há nenhuma exigência de que signifique algo em particular. Por exemplo, `"GMT"`, `"UTC"`, `"-500"`, `"-5:00"`, `"EDT"`, `"US/Eastern"`, `"America/New York"` são todas respostas válidas. Retorna `None` se um nome de string não for conhecido. Observe que este é um método em vez de uma string fixa, principalmente porque algumas subclasses *tzinfo* desejarão retornar nomes diferentes dependendo do valor específico de *dt* passado, especialmente se a classe *tzinfo* estiver contabilizando o horário de verão.

A implementação padrão de *tzname()* levanta *NotImplementedError*.

Estes métodos são chamados por um objeto *datetime* ou *time*, em resposta aos seus métodos de mesmo nome. Um objeto *datetime* passa a si mesmo como argumento, e um objeto *time* passa `None` como o argumento. Os métodos de uma subclasse *tzinfo* devem portanto estar preparados para aceitar um argumento *dt* com valor `None`, ou uma classe *datetime*.

Quando `None` é passado, cabe ao projetista da classe decidir a melhor resposta. Por exemplo, retornar `None` é apropriado se a classe deseja dizer que objetos time não participam nos protocolos da classe *tzinfo*. Pode ser mais útil para `utcoffset(None)` retornar a diferença UTC padrão, como não existe outra convenção para descobrir a diferença padrão.

Quando um objeto *datetime* é passado em resposta a um método *datetime*, `dt.tzinfo` é o mesmo objeto que *self*. Os métodos *tzinfo* podem confiar nisso, a menos que o código do usuário chame métodos *tzinfo* diretamente. A intenção é que os métodos *tzinfo* interpretem *dt* como estando no horário local, e não precisem se preocupar com objetos em outros fusos horários.

Existe mais um método *tzinfo* que uma subclasse pode desejar substituição:

tzinfo.**fromutc**(*dt*)

Isto é chamado a partir do padrão implementação de *datetime.astimezone()*. Quando chamado a partir dele, `dt.tzinfo` é *self*, e os dados de data e hora de *dt* devem ser vistos como expressando um Horário Universal Coordenado (UTC). O propósito de *fromutc()* é ajustar os dados de data e hora, retornando um *datetime* equivalente na hora local de *self*.

A maioria das subclasses de *tzinfo* deve ser capaz de herdar a implementação padrão de *fromutc()* sem problemas. Ela é robusta o suficiente para lidar com fusos horários de deslocamento fixo, e fusos horários que contabilizam tanto o horário padrão quanto o horário de verão, sendo que este último é tratado mesmo que os horários de transição do horário de verão sejam diferentes em anos distintos. Um exemplo de fuso horário que a implementação padrão de *fromutc()* pode não lidar corretamente em todos os casos é aquele onde o deslocamento padrão (de UTC) depende da data e hora específicas passadas, o que pode acontecer por razões políticas. As implementações padrão de *astimezone()* e *fromutc()* podem não produzir o resultado desejado se o resultado estiver entre as horas que abrangem o momento em que o deslocamento padrão muda.

Ignorando o código para casos de erros, a implementação padrão *fromutc()* funciona como:

```
def fromutc(self, dt):
    # levanta o erro ValueError se dt.tzinfo não é o self
    dtoff = dt.utcoffset()
    dtdst = dt.dst()
    # levanta ValueError se dtoff é None ou dtdst é None
    delta = dtoff - dtdst  # este é o deslocamento padrão do self
    if delta:
        dt += delta   # converter para hora local padrão
```

(continua na próxima página)

(continuação da página anterior)

```
        dtdst = dt.dst()
        # levanta ValueError se dtdst é None
    if dtdst:
        return dt + dtdst
    else:
        return dt
```

No seguinte arquivo `tzinfo_examples.py` existem alguns exemplos de classes *tzinfo*:

```
from datetime import tzinfo, timedelta, datetime

ZERO = timedelta(0)
HOUR = timedelta(hours=1)
SECOND = timedelta(seconds=1)

# Uma classe que captura a ideia da plataforma sobre o horário local.
# (Pode resultar em valores errados em horários históricos em fusos horários
#  onde o deslocamento UTC e/ou as regras de horário de verão
#  foram alteradas no passado.)
import time as _time

STDOFFSET = timedelta(seconds = -_time.timezone)
if _time.daylight:
    DSTOFFSET = timedelta(seconds = -_time.altzone)
else:
    DSTOFFSET = STDOFFSET

DSTDIFF = DSTOFFSET - STDOFFSET

class LocalTimezone(tzinfo):

    def fromutc(self, dt):
        assert dt.tzinfo is self
        stamp = (dt - datetime(1970, 1, 1, tzinfo=self)) // SECOND
        args = _time.localtime(stamp)[:6]
        dst_diff = DSTDIFF // SECOND
        # Detect fold
        fold = (args == _time.localtime(stamp - dst_diff))
        return datetime(*args, microsecond=dt.microsecond,
                        tzinfo=self, fold=fold)

    def utcoffset(self, dt):
        if self._isdst(dt):
            return DSTOFFSET
        else:
            return STDOFFSET

    def dst(self, dt):
        if self._isdst(dt):
            return DSTDIFF
        else:
            return ZERO

    def tzname(self, dt):
        return _time.tzname[self._isdst(dt)]
```

(continua na próxima página)

(continuação da página anterior)

```
    def _isdst(self, dt):
        tt = (dt.year, dt.month, dt.day,
              dt.hour, dt.minute, dt.second,
              dt.weekday(), 0, 0)
        stamp = _time.mktime(tt)
        tt = _time.localtime(stamp)
        return tt.tm_isdst > 0

Local = LocalTimezone()


# Uma implementação completa das regras atuais do horário de verão para os␣
↪principais fusos horários dos EUA.

def first_sunday_on_or_after(dt):
    days_to_go = 6 - dt.weekday()
    if days_to_go:
        dt += timedelta(days_to_go)
    return dt


# Regras do horário de verão dos EUA
#
# Este é um conjunto simplificado (ou seja, incorreto em alguns casos)
# de regras para os horários de início e término do horário de verão dos EUA.
# Para um conjunto completo e atualizado de regras de horário de verão
# e definições de fuso horário, visite o Olson Database (ou tente pytz):
# http://www.twinsun.com/tz/tz-link.htm
# https://sourceforge.net/projects/pytz/ (pode não estar atualizado)
#
# Nos EUA, desde 2007, o horário de verão começa às 2h (horário padrão) no
# segundo domingo de março, que é o primeiro domingo em ou após 8 de março.
DSTSTART_2007 = datetime(1, 3, 8, 2)
# e termina às 2h (horário de verão) no primeiro domingo de novembro.
DSTEND_2007 = datetime(1, 11, 1, 2)
# De 1987 a 2006, o horário de verão costumava começar às 2h
# (horário padrão) no primeiro domingo de abril e terminar às 2h
# (horário de verão) no último Domingo de outubro, que é o primeiro
# domingo em ou após 25 de outubro.
DSTSTART_1987_2006 = datetime(1, 4, 1, 2)
DSTEND_1987_2006 = datetime(1, 10, 25, 2)
# De 1967 a 1986, o horário de verão costumava começar às 2h
# (horário padrão) no último domingo de abril (aquele em ou após
# 24 de abril) e terminar às 2h (horário de verão) no último domingo
# de outubro, que é o primeiro domingo em ou após 25 de outubro.
DSTSTART_1967_1986 = datetime(1, 4, 24, 2)
DSTEND_1967_1986 = DSTEND_1987_2006

def us_dst_range(year):
    # Encontra os horários de início e fim do horário de verão dos EUA.
    # Para anos anteriores a 1967, retorne start = end para nenhum horário
    # de verão.
    if 2006 < year:
        dststart, dstend = DSTSTART_2007, DSTEND_2007
    elif 1986 < year < 2007:
        dststart, dstend = DSTSTART_1987_2006, DSTEND_1987_2006
```

(continua na próxima página)

(continuação da página anterior)

```
    elif 1966 < year < 1987:
        dststart, dstend = DSTSTART_1967_1986, DSTEND_1967_1986
    else:
        return (datetime(year, 1, 1), ) * 2

    start = first_sunday_on_or_after(dststart.replace(year=year))
    end = first_sunday_on_or_after(dstend.replace(year=year))
    return start, end


class USTimeZone(tzinfo):

    def __init__(self, hours, reprname, stdname, dstname):
        self.stdoffset = timedelta(hours=hours)
        self.reprname = reprname
        self.stdname = stdname
        self.dstname = dstname

    def __repr__(self):
        return self.reprname

    def tzname(self, dt):
        if self.dst(dt):
            return self.dstname
        else:
            return self.stdname

    def utcoffset(self, dt):
        return self.stdoffset + self.dst(dt)

    def dst(self, dt):
        if dt is None or dt.tzinfo is None:
            # Uma exceção pode ser sensata aqui, em um ou ambos os casos.
            # Depende de como você quer tratá-las. A implementação padrão
            # fromutc() (chamada pela implementação padrão astimezone()) passa
            # um datetime com dt.tzinfo é self.
            return ZERO
        assert dt.tzinfo is self
        start, end = us_dst_range(dt.year)
        # Não é possível comparar objetos ingênuos com objetos conscientes,
        # então retira o fuso horário de dt primeiro.
        dt = dt.replace(tzinfo=None)
        if start + HOUR <= dt < end - HOUR:
            # Horário de verão está em vigor.
            return HOUR
        if end - HOUR <= dt < end:
            # Fold (uma hora ambígua): usa dt.fold para desambiguar.
            return ZERO if dt.fold else HOUR
        if start <= dt < start + HOUR:
            # Intervalo (uma hora inexistente): inverta a regra da dobra.
            return HOUR if dt.fold else ZERO
        # DST is off.
        return ZERO

    def fromutc(self, dt):
        assert dt.tzinfo is self
```

(continua na próxima página)

(continuação da página anterior)

```
        start, end = us_dst_range(dt.year)
        start = start.replace(tzinfo=self)
        end = end.replace(tzinfo=self)
        std_time = dt + self.stdoffset
        dst_time = std_time + HOUR
        if end <= dst_time < end + HOUR:
            # Hora repetida
            return std_time.replace(fold=1)
        if std_time < start or dst_time >= end:
            # Horário padrão
            return std_time
        if start <= std_time < end - HOUR:
            # Horário de verão
            return dst_time


Eastern  = USTimeZone(-5, "Eastern",  "EST", "EDT")
Central  = USTimeZone(-6, "Central",  "CST", "CDT")
Mountain = USTimeZone(-7, "Mountain", "MST", "MDT")
Pacific  = USTimeZone(-8, "Pacific",  "PST", "PDT")
```

Perceba que existem sutilezas inevitáveis duas vezes por ano em uma subclasse *tzinfo* contabilizando tanto hora normal e horário de verão, nos pontos de transição do horário de verão. Para concretude, considere a costa leste dos EUA (UTC -0500), onde o horário de verão EDT começa no minuto após 1:59 (EST, hora padrão) no segundo domingo de Março, e termina no minuto posterior a 1:59 (EDT, horário de verão) no primeiro domingo de Novembro:

```
  UTC   3:MM  4:MM  5:MM  6:MM  7:MM  8:MM
  EST  22:MM 23:MM  0:MM  1:MM  2:MM  3:MM
  EDT  23:MM  0:MM  1:MM  2:MM  3:MM  4:MM

start  22:MM 23:MM  0:MM  1:MM  3:MM  4:MM

  end  23:MM  0:MM  1:MM  1:MM  2:MM  3:MM
```

Quando o horário de verão inicia (a linha de "início"), o relógio de parede local salta de 1:59 para 3:00. Um horário de parede da forma 2:MM realmente não faz sentido nesse dia, então `astimezone(Eastern)` não irá entregar um resultado com `hour == 2` no dia que o horário de verão começar. Por exemplo, na primavera de transição para frente em 2016, nós tivemos:

```
>>> from datetime import datetime, timezone
>>> from tzinfo_examples import HOUR, Eastern
>>> u0 = datetime(2016, 3, 13, 5, tzinfo=timezone.utc)
>>> for i in range(4):
...     u = u0 + i*HOUR
...     t = u.astimezone(Eastern)
...     print(u.time(), 'UTC =', t.time(), t.tzname())
...
05:00:00 UTC = 00:00:00 EST
06:00:00 UTC = 01:00:00 EST
07:00:00 UTC = 03:00:00 EDT
08:00:00 UTC = 04:00:00 EDT
```

Quando o horário de verão termina (a linha "end"), há um problema potencialmente pior: existe uma hora que não pode ser expressa de forma inequívoca no horário local: a última hora do horário de verão. No fuso Eastern, isso corresponde a horários no formato 5:MM UTC no dia em que o horário de verão termina. O relógio de parede local salta de 1:59 (horário de verão) para 1:00 (horário padrão) novamente. Horários locais no formato 1:MM são ambíguos. *astimezone()* imita o comportamento do relógio local mapeando então duas horas UTC adjacentes para

a mesma hora local. No exemplo do fuso Eastern, horários UTC no formato 5:MM e 6:MM ambos se transformam em 1:MM quando convertidos para o fuso Eastern, mas os horários anteriores têm o atributo *fold* definido como 0 e os horários posteriores têm o atributo definido como 1. Por exemplo, na transição de retorno de 2016, obtemos:

```
>>> u0 = datetime(2016, 11, 6, 4, tzinfo=timezone.utc)
>>> for i in range(4):
...     u = u0 + i*HOUR
...     t = u.astimezone(Eastern)
...     print(u.time(), 'UTC =', t.time(), t.tzname(), t.fold)
...
04:00:00 UTC = 00:00:00 EDT 0
05:00:00 UTC = 01:00:00 EDT 0
06:00:00 UTC = 01:00:00 EST 1
07:00:00 UTC = 02:00:00 EST 0
```

Observe que as instâncias de *datetime* que diferem apenas pelo valor do atributo *fold* são consideradas iguais nas comparações.

Aplicações que não suportam ambiguidades de tempo devem verificar explicitamente o valor do atributo *fold* ou evitar utilizar subclasses híbridas *tzinfo*; não há ambiguidades ao usar *timezone* ou qualquer outra subclasse *tzinfo* de deslocamento fixo (como uma classe que representa apenas EST (deslocamento fixo de -5 horas) ou apenas EDT (deslocamento fixo de -4 horas)).

Ver também

zoneinfo
O módulo `datetime` possui uma classe básica *timezone* (para lidar com deslocamentos fixos arbitrários do UTC) e seu atributo *timezone.utc* (uma instância UTC `timezone`).

`zoneinfo` traz o *banco de dados de fuso horário IANA* (também conhecido como banco de dados Olson) para Python, e seu uso é recomendado.

Banco de dados de fuso horário da IANA
O banco de dados de fuso horário (comumente chamado de tz, tzdata ou zoneinfo) contém código e dados que representam o histórico de hora local para muitas localizações representativas ao redor do globo. Ele é atualizado periodicamente para refletir mudanças feitas por corpos políticos para limites de fuso horário, diferenças UTC, e regras de horário de verão.

8.1.9 Objetos `timezone`

A classe *timezone* é uma subclasse de *tzinfo*, cada instância da qual representa um fuso horário definido por um deslocamento fixo do UTC.

Objetos desta classe não podem ser usados para representar informações de fuso horário em locais onde diferentes deslocamentos são usados em diferentes dias do ano ou onde mudanças históricas foram feitas no horário civil.

class `datetime.`**timezone**(*offset*, *name=None*)

O argumento *offset* deve ser especificado como um objeto *timedelta* representando a diferença entre o tempo local e o UTC. Ele deve estar estritamente entre `-timedelta(hours=24)` e `timedelta(hous=24)`, caso contrário a exceção *ValueError* será provocada.

O argumento *name* é opcional. Se especificado, deve ser uma string que será usada como o valor retornado pelo método *datetime.tzname()*.

Adicionado na versão 3.2.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

`timezone.`**utcoffset**(*dt*)

Retorna o valor fixo especificado quando a instância *timezone* é construída.

O argumento *dt* é ignorado. O valor de retorno é uma instância *timedelta* equivalente à diferença entre o tempo local e o UTC.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

timezone.**tzname**(*dt*)

Retorna o valor fixo especificado quando a instância *timezone* é construída.

Se *name* não é passado ao construtor, o nome retornado por `tzname(dt)` é gerado a partir do valor de `offset` do seguinte modo: Se *offset* é `timedelta(0)`, o nome é "UTC", caso contrário é uma string no formato `UTC±HH:MM`, na qual ± é o sinal do `offset`, HH e MM são dois dígitos de `offset.hours` e `offset.minutes` respectivamente.

Alterado na versão 3.6: Nome gerado de `offset=timedelta(0)` é agora simplesmente `'UTC'`, não `'UTC+00:00'`.

timezone.**dst**(*dt*)

Sempre retorna `None`.

timezone.**fromutc**(*dt*)

Retorna `dt + offset`. O argumento *dt* deve ser uma instância *datetime* consciente, com `tzinfo` definida para `self`.

Atributos de classe:

timezone.**utc**

O fuso horário UTC, `timezone(timedelta(0))`.

8.1.10 Comportamento de `strftime()` e `strptime()`

Todos os objetos *date*, *datetime* e *time* dão suporte ao método `strftime(format)`, para criar uma string representando o tempo sob o controle de uma string de formatação explícita.

Em recíproca, os métodos de classe *date.strptime()*, *datetime.strptime()* e *time.strptime()* criam um objeto a partir de uma string representando o tempo e uma string de formatação correspondente.

A tabela abaixo fornece uma comparação de alto nível entre *strftime()* e *strptime()*:

	`strftime`	`strptime`
Uso	Converte objeto para uma string conforme um formato fornecido	Interpreta uma string como um objeto dado um formato correspondente
Tipo de método	Método de instância	Método de classe
Assinatura	`strftime(format)`	`strptime(date_string, format)`

Códigos de formato `strftime()` e `strptime()`

Esses métodos aceitam códigos de formato que podem ser usados para analisar e formatar datas:

```
>>> datetime.strptime('31/01/22 23:59:59.999999',
...                   '%d/%m/%y %H:%M:%S.%f')
datetime.datetime(2022, 1, 31, 23, 59, 59, 999999)
>>> _.strftime('%a %d %b %Y, %I:%M%p')
'Mon 31 Jan 2022, 11:59PM'
```

A seguir é exibida uma lista de todos os códigos de formatação que o padrão C de 1989 requer, e eles funcionam em todas as plataformas com implementação padrão C.

Diretiva	Significado	Exemplo	Notas
%a	Dias da semana como nomes abreviados da localidade.	Sun, Mon, …, Sat (en_US); So, Mo, …, Sa (de_DE)	(1)
%A	Dia da semana como nome completo da localidade.	Sunday, Monday, …, Saturday (en_US); Sonntag, Montag, …, Samstag (de_DE)	(1)
%w	Dia da semana como um número decimal, onde 0 é domingo e 6 é sábado.	0, 1, …, 6	
%d	Dia do mês como um número decimal com zeros a esquerda.	01, 02, …, 31	(9)
%b	Mês como nome da localidade abreviado.	Jan, Feb, …, Dec (en_US); Jan, Feb, …, Dez (de_DE)	(1)
%B	Mês como nome completo da localidade.	January, February, …, December (en_US); janeiro, fevereiro, …, dezembro (pt_BR)	(1)
%m	Mês como um número decimal com zeros a esquerda.	01, 02, …, 12	(9)
%y	Ano sem século como um número decimal com zeros a esquerda.	00, 01, …, 99	(9)
%Y	Ano com século como um número decimal.	0001, 0002, …, 2013, 2014, …, 9998, 9999	(2)
%H	Hora (relógio de 24 horas) como um número decimal com zeros a esquerda.	00, 01, …, 23	(9)
%I	Hora (relógio de 12 horas) como um número decimal com zeros a esquerda.	01, 02, …, 12	(9)
%p	Equivalente da localidade a AM ou PM.	AM, PM (en_US); am, pm (de_DE)	(1), (3)
%M	Minutos como um número decimal, com zeros a esquerda.	00, 01, …, 59	(9)
%S	Segundos como um número decimal, com zeros a esquerda.	00, 01, …, 59	(4), (9)
%f	Microssegundos como um número decimal, com zeros à esquerda até completar 6 dígitos.	000000, 000001, …, 999999	(5)
%z	Diferença UTC no	(vazio), +0000, -0400,	(6)

Diversas diretivas adicionais não necessárias pelo padrão C89 são incluídas para conveniência. Estes parâmetros todos correspondem a valores de datas na ISO 8601.

Diretiva	Significado	Exemplo	Notas
`%G`	Ano ISO 8601 com o século representando o ano que a maior parte da semana ISO (`%V`).	0001, 0002, …, 2013, 2014, …, 9998, 9999	(8)
`%u`	Dia de semana ISO 8601 como um número decimal onde 1 é segunda-feira.	1, 2, …, 7	
`%V`	Semana ISO 8601 como um número decimal, com segunda-feira como o primeiro dia da semana. A semana 01 é a semana contendo o dia 4 de Janeiro.	01, 02, …, 53	(8), (9)
`%:z`	Diferença UTC no formato `±HH:MM[:SS[.ffffff]]` (string vazia se o objeto é ingênuo).	(vazio), +00:00, -04:00, +10:30, +06:34:15, -03:07:12.345216	(6)

Eles podem não estar disponíveis em todas as plataformas quando usados com o método *`strftime()`*. As diretivas de ano e semana da ISO 8601 e ISO 8601 não são intercambiáveis com as diretivas ano e número da semana acima. Chamar *`strptime()`* com diretivas ISO 8601 incompletas ou ambíguas levantará um *`ValueError`*.

O conjunto completo de códigos de formato suportados varia entre as plataformas, porque o Python chama a função `strftime()` da biblioteca C da plataforma, e variações de plataforma são comuns. Para ver o conjunto completo de códigos de formato suportados em sua plataforma, consulte a documentação *`strftime(3)`*. Também existem diferenças entre plataformas no tratamento de especificadores de formato não suportados.

Adicionado na versão 3.6: `%G`, `%u` e `%V` foram adicionados.

Adicionado na versão 3.12: `%:z` foi adicionado.

Detalhes técnicos

Em termos gerais, `d.strftime(fmt)` age como `time.strftime(fmt, d.timetuple())` do módulo *`time`*, embora nem todos os objetos suportem um método *`timetuple()`*.

Para o método de classe *`datetime.strptime()`*, o valor padrão é `1900-01-01T00:00:00.000`: quaisquer componentes não especificados na string de formato serão extraídos do valor padrão.[4]

Usar `datetime.strptime(date_string, format)` é equivalente a:

```
datetime(*(time.strptime(date_string, format)[0:6]))
```

exceto quando o formato inclui componentes de sub-segundos ou informações de deslocamento de fuso horário, que são suportados em `datetime.strptime` mas são descartados por `time.strptime`.

Para objetos *`time`*, os códigos de formato para ano, mês e dia não devem ser usados, pois objetos `time` não têm tais valores. Se forem usados de qualquer forma, o ano é substituído por 1900, e o mês e dia por 1.

Para objetos *`date`*, os códigos de formato para horas, minutos, segundos e microssegundos não devem ser usados, pois objetos *`date`* não têm tais valores. Se forem usados de qualquer forma, são substituídos por 0.

Pela mesma razão, o tratamento de formato de strings contendo pontos de código Unicode que não podem ser representados no conjunto de caracteres da localidade atual também é dependente da plataforma. Em algumas plataformas, tais pontos de código são preservados intactos na saída, enquanto em outros `strftime` pode levantar *`UnicodeError`* ou retornar uma string vazia ao invés.

Notas:

(1) Como o formato depende da localidade atual, deve-se tomar cuidado ao fazer suposições sobre o valor de saída. A ordenação dos campos irá variar (por exemplo, “mês/dia/ano” versus “dia/mês/ano”) e a saída pode conter caracteres não ASCII.

[4] Passar `datetime.strptime('Feb 29', '%b %d')` irá falhar porque 1900 não é um ano bissexto.

(2) O método *strptime()* pode analisar anos no intervalo completo de [1, 9999], mas anos < 1000 devem ser preenchidos com zeros até uma largura de 4 dígitos.

Alterado na versão 3.2: Em versões anteriores, o método *strftime()* era limitado a anos >= 1900.

Alterado na versão 3.3: Na versão 3.2, o método *strftime()* era limitado a anos >= 1000.

(3) Quando usado com o método *strptime()*, a diretiva `%p` afeta somente o campo de hora de saída se a diretiva `%I` for usada para analisar a hora.

(4) Ao contrário do módulo *time*, o módulo `datetime` não oferece suporte para segundos intercalares.

(5) Quando usado com o método *strptime()*, a diretiva `%f` aceita de um a seis dígitos e zeros à direita. `%f` é uma extensão do conjunto de caracteres de formato no padrão do C (mas implementado separadamente em objetos datetime e, portanto, sempre disponível).

(6) Para um objeto ingênuo, os códigos de formatação `%z`, `%:z` e `%Z` são substituídos por strings vazias.

Para um objeto consciente:

%z
: *utcoffset()* é transformado em uma string do formato `±HHMM[SS[.ffffff]]`, onde `HH` é uma string de 2 dígitos que fornece o número de horas de deslocamento UTC, `MM` é uma string de 2 dígitos que fornece o número de minutos de deslocamento UTC, `SS` é uma string de 2 dígitos que fornece o número de segundos de deslocamento UTC e `ffffff` é uma string de 6 dígitos que fornece o número de microssegundos de deslocamento UTC. A parte `ffffff` é omitida quando o deslocamento é um número inteiro de segundos e tanto a parte `ffffff` quanto a parte `SS` são omitidas quando o deslocamento é um número inteiro de minutos. Por exemplo, se *utcoffset()* retornar `timedelta(hours=-3, minutes=-30)`, `%z` será substituído pela string `'-0330'`.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

Alterado na versão 3.7: Quando a diretiva `%z` é fornecida ao método *strptime()*, os deslocamentos UTC podem ter dois pontos como separador entre horas, minutos e segundos. Por exemplo, `'+01:00:00'` será analisado como um deslocamento de uma hora. Além disso, fornecer `'Z'` é idêntico a `'+00:00'`.

%:z
: Comporta-se exatamente como `%z`, mas possui um separador de dois pontos adicionado entre horas, minutos e segundos.

%Z
: Em *strftime()*, `%Z` é substituído por uma string vazia se *tzname()* retornar `None`; caso contrário, `%Z` é substituído pelo valor retornado, que deve ser uma string.

strptime() aceita apenas certos valores para `%Z`:

1. qualquer valor em `time.tzname` para a localidade da sua máquina
2. os valores codificados `UTC` e `GMT`

Então alguém vivendo no Japão pode ter `JST`, `UTC`, e `GMT` como valores válidos, mas provavelmente não `EST`. Isso levantará `ValueError` para valores inválidos.

Alterado na versão 3.2: Quando a diretiva `%z` é fornecida ao método *strptime()*, um objeto *datetime* consciente será produzido. O `tzinfo` do resultado será definido para uma instância *timezone*.

(7) Quando utilizados com o método *strptime()*, `%U` e `%W` são usados somente em cálculos quando o dia da semana e o ano (`%Y`) são especificados.

(8) Semelhante a `%U` e `%W`, `%V` é usado apenas em cálculos quando o dia da semana e o ano ISO (`%G`) são especificados em um formato de string *strptime()*. Observe também que `%G` e `%Y` não são intercambiáveis.

(9) Quando usado com o método *strptime()*, o zero à esquerda é opcional para os formatos `%d`, `%m`, `%H`, `%I`, `%M`, `%S`, `%j`, `%U`, `%W` e `%V`. O formato `%y` requer um zero à esquerda.

(10) Ao analisar um mês e dia usando *strptime()*, sempre inclua um ano no formato. Se o valor que você precisa analisar não possui um ano, adicione um ano bissexto fictício de forma explícita. Caso contrário, seu código levantará uma exceção ao encontrar o dia bissexto, porque o ano padrão usado pelo analisador sintático não é um ano bissexto. Os usuários encontram esse bug a cada quatro anos…

```
>>> month_day = "02/29"
>>> datetime.strptime(f"{month_day};1984", "%m/%d;%Y")  # Sem bug de ano␣
↪bissexto.
datetime.datetime(1984, 2, 29, 0, 0)
```

Deprecated since version 3.13, will be removed in version 3.15: Chamadas ao método *strptime()* usando uma string de formato contendo um dia do mês sem um ano agora emitem um *DeprecationWarning*. Na versão 3.15 ou posterior, podemos mudar isso para um erro ou mudar o ano padrão para um ano bissexto. Consulte gh-70647.

8.2 zoneinfo — Suporte a fuso horário da IANA

Adicionado na versão 3.9.

Código-fonte: Lib/zoneinfo

O módulo *zoneinfo* fornece uma implementação concreta de fuso horário para oferecer suporte ao banco de dados de fuso horário da IANA, conforme especificado originalmente na **PEP 615**. Por padrão, *zoneinfo* usa os dados de fuso horário do sistema, se disponíveis; se nenhum dado de fuso horário do sistema estiver disponível, a biblioteca voltará a usar o pacote original tzdata disponível no PyPI.

Ver também

Módulo: *datetime*
: Fornece os tipos *time* e *datetime* com os quais a classe *ZoneInfo* foi projetada para ser usada.

Pacote tzdata
: Pacote próprio mantido pelos desenvolvedores principais do CPython para fornecer dados de fuso horário via PyPI.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

8.2.1 Usando ZoneInfo

ZoneInfo é uma implementação concreta da classe base abstrata *datetime.tzinfo* e deve ser anexada a `tzinfo`, por meio do construtor, do método *datetime.replace* ou *datetime.astimezone*:

```
>>> from zoneinfo import ZoneInfo
>>> from datetime import datetime, timedelta

>>> dt = datetime(2020, 10, 31, 12, tzinfo=ZoneInfo("America/Los_Angeles"))
>>> print(dt)
2020-10-31 12:00:00-07:00

>>> dt.tzname()
'PDT'
```

Datas e horas construídas dessa maneira são compatíveis com a aritmética de data e hora e lidam com transições de horário de verão sem intervenção adicional:

```
>>> dt_add = dt + timedelta(days=1)

>>> print(dt_add)
```

(continua na próxima página)

(continuação da página anterior)

```
2020-11-01 12:00:00-08:00

>>> dt_add.tzname()
'PST'
```

Esses fusos horários também oferecem suporte ao atributo *fold* introduzido em **PEP 495**. Durante transições de deslocamento que induzem horários ambíguos (como uma transição do horário de verão para o horário padrão), o deslocamento *anterior* à transição é usado quando `fold=0`, e o deslocamento *posterior* à transição é usado quando `fold=1`, por exemplo:

```
>>> dt = datetime(2020, 11, 1, 1, tzinfo=ZoneInfo("America/Los_Angeles"))
>>> print(dt)
2020-11-01 01:00:00-07:00

>>> print(dt.replace(fold=1))
2020-11-01 01:00:00-08:00
```

Ao converter de outro fuso horário, a dobra será definida para o valor correto:

```
>>> from datetime import timezone
>>> LOS_ANGELES = ZoneInfo("America/Los_Angeles")
>>> dt_utc = datetime(2020, 11, 1, 8, tzinfo=timezone.utc)

>>> # Antes da transição PDT -> PST
>>> print(dt_utc.astimezone(LOS_ANGELES))
2020-11-01 01:00:00-07:00

>>> # Após a transição PDT -> PST
>>> print((dt_utc + timedelta(hours=1)).astimezone(LOS_ANGELES))
2020-11-01 01:00:00-08:00
```

8.2.2 Fontes de dados

O módulo `zoneinfo` não fornece dados de fuso horário diretamente, mas, em vez disso, extrai informações de fuso horário do banco de dados de fuso horário do sistema ou do pacote PyPI original tzdata, se disponível. Alguns sistemas, incluindo principalmente os Windows, não possuem um banco de dados IANA disponível e, portanto, para projetos que buscam compatibilidade multiplataforma e exigem dados de fuso horário, recomenda-se declarar uma dependência em tzdata. Se nem os dados do sistema nem o tzdata estiverem disponíveis, todas as chamadas para *ZoneInfo* levantarão *ZoneInfoNotFoundError*.

Configurando as fontes de dados

Quando `ZoneInfo(key)` é chamado, o construtor primeiro pesquisa nos diretórios especificados em *TZPATH* por um arquivo que corresponda a `key` e, em caso de falha, procura uma correspondência no pacote tzdata. Esse comportamento pode ser configurado de três maneiras:

1. O padrão *TZPATH*, quando não especificado de outra forma, pode ser configurado em *tempo de compilação*.
2. *TZPATH* pode ser configurado usando *uma variável de ambiente*.
3. Em *tempo de execução*, o caminho de pesquisa pode ser manipulado usando a função *reset_tzpath()*.

Configuração em tempo de compilação

O *TZPATH* padrão inclui vários locais de implantação comuns para o banco de dados de fuso horário (exceto no Windows, onde não há locais "conhecidos" para dados de fuso horário). Em sistemas POSIX, distribuidores downstream e aqueles que constroem o Python a partir do código-fonte e que sabem onde os dados de fuso horário do

sistema estão implantados podem alterar o caminho do fuso horário padrão especificando a opção de tempo de compilação `TZPATH` (ou, mais provavelmente, o `sinalizador de configuração --with-tzpath`), que deve ser uma string delimitada por *os.pathsep*.

Em todas as plataformas, o valor configurado está disponível como a chave `TZPATH` em *sysconfig.get_config_var()*.

Configuração do ambiente

Ao inicializar *TZPATH* (no momento da importação ou sempre que *reset_tzpath()* for chamado sem argumentos), o módulo `zoneinfo` usará a variável de ambiente `PYTHONTZPATH`, se existir, para definir o caminho de pesquisa.

PYTHONTZPATH

Esta é uma string separada por *os.pathsep* contendo o caminho de pesquisa de fuso horário a ser usado. Ela deve consistir apenas em caminhos absolutos, e não relativos. Componentes relativos especificados em `PYTHONTZPATH` não serão usados, mas, caso contrário, o comportamento quando um caminho relativo é especificado é definido pela implementação; o CPython levantará *InvalidTZPathWarning*, mas outras implementações podem ignorar silenciosamente o componente incorreto ou levantar uma exceção.

Para configurar o sistema para ignorar os dados do sistema e usar o pacote tzdata, defina `PYTHONTZPATH=""`.

Configuração em tempo de execução

O caminho de busca TZ também pode ser configurado em tempo de execução usando a função *reset_tzpath()*. Geralmente, essa não é uma operação aconselhável, embora seja razoável usá-la em funções de teste que exigem o uso de um caminho de fuso horário específico (ou que exijam a desativação do acesso aos fusos horários do sistema).

8.2.3 A classe `ZoneInfo`

class zoneinfo.**ZoneInfo**(*key*)

Uma subclasse concreta de *datetime.tzinfo* que representa um fuso horário IANA especificado pela string `key`. Chamadas ao construtor primário sempre retornarão objetos que sejam idênticos em comparação; em outras palavras, exceto pela invalidação de cache via *ZoneInfo.clear_cache()*, para todos os valores de `key`, a seguinte asserção será sempre verdadeira:

```
a = ZoneInfo(key)
b = ZoneInfo(key)
assert a is b
```

`key` deve estar no formato de um caminho POSIX relativo e normalizado, sem referências de nível superior. O construtor levantará *ValueError* se uma chave não conforme for passada.

Se nenhum arquivo correspondente a `key` for encontrado, o construtor levantará *ZoneInfoNotFoundError*.

A classe `ZoneInfo` tem dois construtores alternativos:

classmethod ZoneInfo.**from_file**(*file_obj*, */*, *key=None*)

Constructs a `ZoneInfo` object from a file-like object returning bytes (e.g. a file opened in binary mode or an *io.BytesIO* object). Unlike the primary constructor, this always constructs a new object.

The `key` parameter sets the name of the zone for the purposes of `__str__()` and `__repr__()`.

Objects created via this constructor cannot be pickled (see *pickling*).

classmethod ZoneInfo.**no_cache**(*key*)

An alternate constructor that bypasses the constructor's cache. It is identical to the primary constructor, but returns a new object on each call. This is most likely to be useful for testing or demonstration purposes, but it can also be used to create a system with a different cache invalidation strategy.

Objects created via this constructor will also bypass the cache of a deserializing process when unpickled.

> **Cuidado**
>
> Using this constructor may change the semantics of your datetimes in surprising ways, only use it if you know that you need to.

The following class methods are also available:

classmethod ZoneInfo.**clear_cache**(*, *only_keys=None*)

A method for invalidating the cache on the `ZoneInfo` class. If no arguments are passed, all caches are invalidated and the next call to the primary constructor for each key will return a new instance.

If an iterable of key names is passed to the `only_keys` parameter, only the specified keys will be removed from the cache. Keys passed to `only_keys` but not found in the cache are ignored.

> **Aviso**
>
> Invoking this function may change the semantics of datetimes using `ZoneInfo` in surprising ways; this modifies module state and thus may have wide-ranging effects. Only use it if you know that you need to.

A classe possui um atributo:

ZoneInfo.**key**

This is a read-only *attribute* that returns the value of `key` passed to the constructor, which should be a lookup key in the IANA time zone database (e.g. `America/New_York`, `Europe/Paris` or `Asia/Tokyo`).

For zones constructed from file without specifying a `key` parameter, this will be set to `None`.

> **Nota**
>
> Although it is a somewhat common practice to expose these to end users, these values are designed to be primary keys for representing the relevant zones and not necessarily user-facing elements. Projects like CLDR (the Unicode Common Locale Data Repository) can be used to get more user-friendly strings from these keys.

String representations

The string representation returned when calling *str* on a *ZoneInfo* object defaults to using the *ZoneInfo.key* attribute (see the note on usage in the attribute documentation):

```
>>> zone = ZoneInfo("Pacific/Kwajalein")
>>> str(zone)
'Pacific/Kwajalein'

>>> dt = datetime(2020, 4, 1, 3, 15, tzinfo=zone)
>>> f"{dt.isoformat()} [{dt.tzinfo}]"
'2020-04-01T03:15:00+12:00 [Pacific/Kwajalein]'
```

For objects constructed from a file without specifying a `key` parameter, `str` falls back to calling *repr()*. `ZoneInfo`'s `repr` is implementation-defined and not necessarily stable between versions, but it is guaranteed not to be a valid `ZoneInfo` key.

Pickle serialization

Rather than serializing all transition data, `ZoneInfo` objects are serialized by key, and `ZoneInfo` objects constructed from files (even those with a value for `key` specified) cannot be pickled.

The behavior of a `ZoneInfo` file depends on how it was constructed:

1. `ZoneInfo(key)`: When constructed with the primary constructor, a `ZoneInfo` object is serialized by key, and when deserialized, the deserializing process uses the primary and thus it is expected that these are expected to be the same object as other references to the same time zone. For example, if `europe_berlin_pkl` is a string containing a pickle constructed from `ZoneInfo("Europe/Berlin")`, one would expect the following behavior:

```
>>> a = ZoneInfo("Europe/Berlin")
>>> b = pickle.loads(europe_berlin_pkl)
>>> a is b
True
```

2. `ZoneInfo.no_cache(key)`: When constructed from the cache-bypassing constructor, the `ZoneInfo` object is also serialized by key, but when deserialized, the deserializing process uses the cache bypassing constructor. If `europe_berlin_pkl_nc` is a string containing a pickle constructed from `ZoneInfo.no_cache("Europe/Berlin")`, one would expect the following behavior:

```
>>> a = ZoneInfo("Europe/Berlin")
>>> b = pickle.loads(europe_berlin_pkl_nc)
>>> a is b
False
```

3. `ZoneInfo.from_file(file_obj, /, key=None)`: When constructed from a file, the `ZoneInfo` object raises an exception on pickling. If an end user wants to pickle a `ZoneInfo` constructed from a file, it is recommended that they use a wrapper type or a custom serialization function: either serializing by key or storing the contents of the file object and serializing that.

This method of serialization requires that the time zone data for the required key be available on both the serializing and deserializing side, similar to the way that references to classes and functions are expected to exist in both the serializing and deserializing environments. It also means that no guarantees are made about the consistency of results when unpickling a `ZoneInfo` pickled in an environment with a different version of the time zone data.

8.2.4 Funções

`zoneinfo.`**`available_timezones`**`()`

Get a set containing all the valid keys for IANA time zones available anywhere on the time zone path. This is recalculated on every call to the function.

This function only includes canonical zone names and does not include "special" zones such as those under the `posix/` and `right/` directories, or the `posixrules` zone.

Cuidado

This function may open a large number of files, as the best way to determine if a file on the time zone path is a valid time zone is to read the "magic string" at the beginning.

Nota

These values are not designed to be exposed to end-users; for user facing elements, applications should use something like CLDR (the Unicode Common Locale Data Repository) to get more user-friendly strings. See also the cautionary note on *`ZoneInfo.key`*.

`zoneinfo.`**`reset_tzpath`**`(`*to=None*`)`

Sets or resets the time zone search path (*`TZPATH`*) for the module. When called with no arguments, *`TZPATH`* is set to the default value.

Calling `reset_tzpath` will not invalidate the *`ZoneInfo`* cache, and so calls to the primary `ZoneInfo` constructor will only use the new `TZPATH` in the case of a cache miss.

The `to` parameter must be a *sequence* of strings or `os.PathLike` and not a string, all of which must be absolute paths. `ValueError` will be raised if something other than an absolute path is passed.

8.2.5 Globals

`zoneinfo.`**`TZPATH`**

A read-only sequence representing the time zone search path – when constructing a `ZoneInfo` from a key, the key is joined to each entry in the `TZPATH`, and the first file found is used.

`TZPATH` may contain only absolute paths, never relative paths, regardless of how it is configured.

The object that `zoneinfo.TZPATH` points to may change in response to a call to `reset_tzpath()`, so it is recommended to use `zoneinfo.TZPATH` rather than importing `TZPATH` from `zoneinfo` or assigning a long-lived variable to `zoneinfo.TZPATH`.

For more information on configuring the time zone search path, see *Configurando as fontes de dados*.

8.2.6 Exceptions and warnings

`exception` `zoneinfo.`**`ZoneInfoNotFoundError`**

Raised when construction of a `ZoneInfo` object fails because the specified key could not be found on the system. This is a subclass of `KeyError`.

`exception` `zoneinfo.`**`InvalidTZPathWarning`**

Raised when `PYTHONTZPATH` contains an invalid component that will be filtered out, such as a relative path.

8.3 `calendar` — Funções gerais relacionadas ao calendário

Código-fonte: Lib/calendar.py

Este módulo permite que você exiba calendários como o programa Unix **`cal`**, e fornece funções adicionais úteis relacionadas ao calendário. Por padrão, esses calendários têm a segunda-feira como o primeiro dia da semana, e domingo como o último (a convenção europeia). Use `setfirstweekday()` para colocar o primeiro dia da semana como domingo (6) ou para qualquer outro dia da semana. Parâmetros que especificam datas são dados como inteiros. Para funcionalidade relacionada, veja também os módulos `datetime` e `time`.

As funções e classes definidas neste módulo usam um calendário idealizado, o calendário Gregoriano atual estendido indefinidamente nas duas direções. Isso corresponde à definição do calendário "proleptic Gregorian" (gregoriano proléptico) no livro "Calendrical Calculations" de Dershowitz e Reingold, onde está o calendário base para todas os cálculos. Anos com zero ou negativos são interpretados e prescritos pelo padrão ISO 8601. Ano 0 é 1 A.C., ano -1 é 2 A.C, e de assim em diante.

`class` `calendar.`**`Calendar`**(*firstweekday=0*)

Cria um objeto `Calendar`. *firstweekday* é um inteiro que especifica o primeiro dia da semana. `MONDAY` é `0` (o padrão), `SUNDAY` é `6`.

Um objeto `Calendar` fornece vários métodos que podem ser usados para preparar os dados do calendário para formatação. Esta classe não realiza nenhuma formatação por si mesma. Esse é o trabalho das subclasses.

Instâncias de `Calendar` têm os seguintes métodos e atributos:

`firstweekday`

O primeiro dia da semana como um inteiro (0–6).

Esta propriedade também pode ser definida e lida usando `setfirstweekday()` e `getfirstweekday()` respectivamente.

`getfirstweekday`()

Retorna um `int` para o primeiro dia da semana atual (0-6).

Idêntico à leitura da propriedade `firstweekday`.

setfirstweekday(*firstweekday*)

Define o primeiro dia da semana como *firstweekday*, passado como `int` (0–6)

Idêntico à definição da propriedade `firstweekday`.

iterweekdays()

Retorna um iterador para os números dos dias da semana que serão usados em uma semana. O primeiro valor do iterador será o mesmo que o valor da propriedade `firstweekday`.

itermonthdates(*year*, *month*)

Retorna um iterador para o mês *month* (1–12) no ano *year*. Este iterador retornará todos os dias (como objetos `datetime.date`) para o mês e todos os dias antes do início do mês ou após o final do mês que são necessários para obter uma semana completa.

itermonthdays(*year*, *month*)

Retorna um iterador para o mês *month* no ano *year* semelhante a `itermonthdates()`, mas não restrito pelo intervalo de `datetime.date`. Os dias retornados serão simplesmente os números dos dias do mês. Para os dias fora do mês especificado, o número do dia será `0`.

itermonthdays2(*year*, *month*)

Retorna um iterador para o mês *month* no ano *year* semelhante a `itermonthdates()`, mas não restrito pelo intervalo de `datetime.date`. Os dias retornados serão tuplas consistindo de um número de dia do mês e um número de dia da semana.

itermonthdays3(*year*, *month*)

Retorna um iterador para o mês *month* no ano *year* semelhante a `itermonthdates()`, mas não restrito pelo intervalo de `datetime.date`. Os dias retornados serão tuplas consistindo de números de um ano, um mês e um dia do mês.

Adicionado na versão 3.7.

itermonthdays4(*year*, *month*)

Retorna um iterador para o mês *month* no ano *year* semelhante a `itermonthdates()`, mas não restrito pelo intervalo de `datetime.date`. Os dias retornados serão tuplas consistindo de números de um ano, um mês, um dia do mês e um dia da semana.

Adicionado na versão 3.7.

monthdatescalendar(*year*, *month*)

Retorna uma lista das semanas do mês *month* do *year* como semanas completas. As semanas são listas de sete objetos `datetime.date`.

monthdays2calendar(*year*, *month*)

Retorna uma lista das semanas do mês *month* do ano *year* como semanas completas. As semanas são listas de sete tuplas de números dias e de dias de semanas.

monthdayscalendar(*year*, *month*)

Retorna uma lista das semanas do mês *month* do ano *year* como semanas completas. As semanas são listas de números de sete dias.

yeardatescalendar(*year*, *width=3*)

Retorna os dados para o ano especificado prontos para formatação. O valor de retorno é uma lista de linhas de meses. Cada linha de mês contém até *width* meses (padrão é 3). Cada mês contém entre 4 e 6 semanas, e cada semana contém 1–7 dias. Os dias são objetos `datetime.date`.

yeardays2calendar(*year*, *width=3*)

Retorna os dados para o ano especificado prontos para formatação (semelhante a `yeardatescalendar()`). Entradas nas listas semanais são tuplas de números de dias e números de dias de semana. Números de dias fora deste mês são zero.

yeardayscalendar(*year*, *width=3*)

Retorna a data para o ano especificado prontos para formatação (semelhante a `yeardatescalendar()`). Entradas nas listas de semanas são números de dias. Números de dias fora deste mês são zero.

class calendar.**TextCalendar**(*firstweekday=0*)

Esta classe pode ser usada para gerar texto plano para calendários.

Instâncias de `TextCalendar` têm os seguintes métodos:

formatday(*theday*, *weekday*, *width*)

Retorna uma string representando um único dia formatado com a largura fornecida em *width*. Se *theday* for `0`, retorna uma string de espaços da largura especificada, representando um dia vazio. O parâmetro *weekday* não é usado.

formatweek(*theweek*, *w=0*)

Retorna uma única semana em uma string sem nova linha. Se *w* for providenciado, isto especifica a largura das colunas de data, que são centrais. Dependendo do primeiro dia da semana conforme especificado no construtor ou configurado pelo método `setfirstweekday()`.

formatweekday(*weekday*, *width*)

Retorna uma string representando o nome de um único dia da semana formatado para a largura especificada em *width*. O parâmetro *weekday* é um inteiro representando o dia da semana, onde `0` é segunda-feira e `6` é domingo.

formatweekheader(*width*)

Retorna uma string contendo a linha de cabeçalho dos nomes dos dias da semana, formatada com a largura fornecida por *width* para cada coluna. Os nomes dependem das configurações de localidade e são preenchidos até a largura especificada.

formatmonth(*theyear*, *themonth*, *w=0*, *l=0*)

Retorna o calendário do mês em uma string multilinha. Se *w* for providenciado, isto especifica a largura das colunas de data, que são centrais. Se *l* for dado, este especifica o número de linhas que cada semana vai usar. Dependendo do primeiro dia da semana conforme especificado no construtor ou configurado pelo método `setfirstweekday()`.

formatmonthname(*theyear*, *themonth*, *width=0*, *withyear=True*)

Retorna uma string representando o nome do mês centralizado dentro da largura especificada em *width*. Se *withyear* for `True`, inclui o ano na saída. Os parâmetros *theyear* e *themonth* especificam o ano e o mês para o nome a ser formatado, respectivamente.

prmonth(*theyear*, *themonth*, *w=0*, *l=0*)

Imprime um calendário do mês conforme retornado pelo `formatmonth()`.

formatyear(*theyear*, *w=2*, *l=1*, *c=6*, *m=3*)

Retorna um calendário com *m* colunas para um ano inteiro conforme uma string multilinha. Parâmetros opcionais *w*, *l* e *c* definem a largura da coluna data, linhas por semana e números de espaços entre as colunas dos meses, respectivamente. Depende do primeiro dia da semana conforme especificado no construtor ou definido pelo método `setfirstweekday()`. O ano mais novo para o qual o calendário pode ser gerado depende da plataforma.

pryear(*theyear*, *w=2*, *l=1*, *c=6*, *m=3*)

Imprime o calendário para um ano inteiro conforme retornado por `formatyear()`.

class calendar.**HTMLCalendar**(*firstweekday=0*)

Esta classse pode ser usada para gerar calendários HTML.

Instâncias de `HTMLCalendar` têm os seguintes métodos:

formatmonth(*theyear*, *themonth*, *withyear=True*)

Retorna um calendário do mês como uma tabela HTML. Se *withyear* for verdadeiro, o ano será incluído no cabeçalho, senão apenas o nome do mês será utilizado.

formatyear(*theyear*, *width=3*)

Retorna um calendário do ano como uma tabela HTML. *width* (padronizada para 3) especifica o número de meses por linha.

formatyearpage(*theyear*, *width=3*, *css='calendar.css'*, *encoding=None*)

Retorna o calendário de um ano como uma página HTML completa. *width* (padrão 3) especifica o número de meses por linha. *css* é o nome da folha de estilo em cascata a ser usada. `None` pode ser passado se nenhuma folha de estilo deve ser usada. *encoding* especifica a codificação a ser usada para a saída (padrão para a codificação padrão do sistema).

formatmonthname(*theyear*, *themonth*, *withyear=True*)

Retorna um nome de mês como uma linha de tabela HTML. Se *withyear* for verdadeiro, o ano será incluído na linha, senão apenas o nome do mês será utilizado.

`HTMLCalendar` tem os seguintes atributos que você pode substituir para personalizar as classes CSS usadas pelo calendário:

cssclasses

Uma lista de classes CSS usadas para cada dia da semana. A lista de classes padrão é:

```
cssclasses = ["mon", "tue", "wed", "thu", "fri", "sat", "sun"]
```

mais estilos podem ser adicionados para cada dia:

```
cssclasses = ["mon text-bold", "tue", "wed", "thu", "fri", "sat", "sun red
↪"]
```

Observe que o comprimento desta lista deve ser de sete itens.

cssclass_noday

A classe CSS para um dia da semana que ocorre no mês anterior ou seguinte.

Adicionado na versão 3.7.

cssclasses_weekday_head

Uma lista de classes CSS usadas para nomes de dias da semana na linha de cabeçalho. O padrão é o mesmo que *cssclasses*.

Adicionado na versão 3.7.

cssclass_month_head

A classe CSS principal do mês (usada por *formatmonthname()*). O valor padrão é `"month"`.

Adicionado na versão 3.7.

cssclass_month

A classe CSS para a tabela do mês inteiro (usada por *formatmonth()*). O valor padrão é `"month"`.

Adicionado na versão 3.7.

cssclass_year

A classe CSS para a tabela de tabelas do ano inteiro (usada por *formatyear()*). O valor padrão é `"year"`.

Adicionado na versão 3.7.

cssclass_year_head

A classe CSS para o cabeçalho da tabela para o ano inteiro (usado por *formatyear()*). O valor padrão é `"year"`.

Adicionado na versão 3.7.

Observe que, embora a nomenclatura dos atributos de classe descritos acima seja singular (por exemplo, `cssclass_month` `cssclass_noday`), é possível substituir a única classe CSS por uma lista de classes CSS separadas por espaços, por exemplo:

```
"text-bold text-red"
```

Aqui está um exemplo de como `HTMLCalendar` pode ser personalizado:

```
class CustomHTMLCal(calendar.HTMLCalendar):
    cssclasses = [style + " text-nowrap" for style in
                  calendar.HTMLCalendar.cssclasses]
    cssclass_month_head = "text-center month-head"
    cssclass_month = "text-center month"
    cssclass_year = "text-italic lead"
```

class calendar.**LocaleTextCalendar**(*firstweekday=0*, *locale=None*)

Esta subclasse de *TextCalendar* pode receber um nome de localidade no construtor e retornará nomes de meses e dias da semana na localidade especificada.

class calendar.**LocaleHTMLCalendar**(*firstweekday=0*, *locale=None*)

Esta subclasse de *HTMLCalendar* pode receber um nome de localidade no construtor e retornará nomes de meses e dias da semana na localidade especificada.

Nota

Os métodos construtores `formatweekday()` e `formatmonthname()` dessas duas classes alteram temporariamente a localidade `LC_TIME` para o *locale* fornecido. Como a localidade atual é uma configuração de todo o processo, eles não são seguros para threads.

Para calendários de texto simples, este módulo fornece as seguintes funções.

calendar.**setfirstweekday**(*weekday*)

Define o dia da semana (`0` é segunda-feira, `6` é domingo) para começar cada semana. Os valores *MONDAY*, *TUESDAY*, *WEDNESDAY*, *THURSDAY*, *FRIDAY*, *SATURDAY* e *SUNDAY* são fornecidos para conveniência. Por exemplo, para definir o primeiro dia da semana como domingo:

```
import calendar
calendar.setfirstweekday(calendar.SUNDAY)
```

calendar.**firstweekday**()

Retorna a configuração atual para o dia da semana que inicia cada semana.

calendar.**isleap**(*year*)

Retorna *True* se *year* for um ano bissexto, caso contrário, *False*.

calendar.**leapdays**(*y1*, *y2*)

Retorna o número de anos bissextos no intervalo de *y1* a *y2* (exclusivo), onde *y1* e *y2* são anos.

Esta função funciona para intervalos que abrangem uma mudança de século.

calendar.**weekday**(*year*, *month*, *day*)

Retorna o dia da semana (`0` é segunda-feira) para *year* (`1970`–…), *month* (`1`–`12`), *day* (`1`–`31`).

calendar.**weekheader**(*n*)

Retorna um cabeçalho contendo nomes abreviados de dias da semana. *n* especifica a largura em caracteres para um dia da semana.

calendar.**monthrange**(*year*, *month*)

Retorna o dia da semana do primeiro dia do mês e o número de dias do mês, para o *year* e *month* especificados.

calendar.**monthcalendar**(*year*, *month*)

Retorna uma matriz representando o calendário de um mês. Cada linha representa uma semana; dias fora do mês são representados por zeros. Cada semana começa com segunda-feira, a menos que seja definida por *setfirstweekday()*.

calendar.**prmonth**(*theyear*, *themonth*, *w=0*, *l=0*)

Imprime um calendário do mês conforme retornado pelo *month()*.

calendar.**month**(*theyear*, *themonth*, *w=0*, *l=0*)

Retorna o calendário de um mês em uma string de várias linhas usando *formatmonth()* da classe *TextCalendar*.

calendar.**prcal**(*year*, *w=0*, *l=0*, *c=6*, *m=3*)

Imprime o calendário para um ano inteiro conforme retornado por *calendar()*.

calendar.**calendar**(*year*, *w=2*, *l=1*, *c=6*, *m=3*)

Retorna um calendário de 3 colunas para um ano inteiro como uma string de várias linhas usando *formatyear()* da classe *TextCalendar*.

calendar.**timegm**(*tuple*)

Uma função não relacionada, mas útil, que pega uma tupla de tempo, como a retornada pela função *gmtime()* no módulo *time*, e retorna o valor de registro de data e hora Unix correspondente, presumindo uma época de 1970 e a codificação POSIX. Na verdade, *time.gmtime()* e *timegm()* são inversos um do outro.

O módulo *calendar* exporta os seguintes atributos de dados:

calendar.**day_name**

Uma sequência que representa os dias da semana no local atual, onde Monday é o dia número 0.

```
>>> import calendar
>>> list(calendar.day_name)
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
```

calendar.**day_abbr**

Uma sequência que representa os dias abreviados da semana no local atual, onde Mon é o dia número 0.

```
>>> import calendar
>>> list(calendar.day_abbr)
['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
```

calendar.**MONDAY**

calendar.**TUESDAY**

calendar.**WEDNESDAY**

calendar.**THURSDAY**

calendar.**FRIDAY**

calendar.**SATURDAY**

calendar.**SUNDAY**

Apelidos para os dias da semana, onde `MONDAY` é `0` e `SUNDAY` é `6`.

Adicionado na versão 3.12.

class calendar.**Day**

Enumeração que define os dias da semana como constantes inteiras. Os membros dessa enumeração são exportados para o escopo do módulo como *MONDAY* até *SUNDAY*.

Adicionado na versão 3.12.

calendar.**month_name**

Uma sequência que representa os meses do ano na localidade atual. Isso segue a convenção normal de janeiro sendo o mês número 1, então tem um comprimento de 13 e `month_name[0]` é a string vazia.

```
>>> import calendar
>>> list(calendar.month_name)
['', 'January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
↪'September', 'October', 'November', 'December']
```

calendar.**month_abbr**

Uma sequência que representa os meses abreviados do ano na localidade atual. Isso segue a convenção normal de janeiro sendo o mês número 1, então tem um comprimento de 13 e `month_abbr[0]` é a string vazia.

```
>>> import calendar
>>> list(calendar.month_abbr)
['', 'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov
↪', 'Dec']
```

calendar.**JANUARY**

calendar.**FEBRUARY**

calendar.**MARCH**

calendar.**APRIL**

calendar.**MAY**

calendar.**JUNE**

calendar.**JULY**

calendar.**AUGUST**

calendar.**SEPTEMBER**

calendar.**OCTOBER**

calendar.**NOVEMBER**

calendar.**DECEMBER**

Apelidos para os meses do ano, onde `JANUARY` é `1` e `DECEMBER` é `12`.

Adicionado na versão 3.12.

class calendar.**Month**

Enumeração que define os meses do ano como constantes inteiras. Os membros dessa enumeração são exportados para o escopo do módulo como *JANUARY* até *DECEMBER*.

Adicionado na versão 3.12.

O módulo *calendar* define as seguintes exceções:

exception calendar.**IllegalMonthError**(*month*)

Uma subclasse de *ValueError*, levantada quando o número do mês fornecido está fora do intervalo de 1 a 12 (inclusive).

month

O número de meses inválidos.

exception calendar.**IllegalWeekdayError**(*weekday*)

Uma subclasse de *ValueError*, levantada quando o número do dia da semana fornecido está fora do intervalo de 0 a 6 (inclusive).

weekday

O número de dias da semana inválidos.

Ver também

Módulo *datetime*
Interface orientada a objetos para datas e horas com funcionalidade semelhante ao módulo *time*.

Módulo *time*
Funções de baixo nível relacionadas ao tempo.

8.3.1 Uso na linha de comando

Adicionado na versão 2.5.

O módulo *calendar* pode ser executado como um script na linha de comando para exibir interativamente um calendário.

```
python -m calendar [-h] [-L LOCALE] [-e ENCODING] [-t {text,html}]
                   [-w WIDTH] [-l LINES] [-s SPACING] [-m MONTHS] [-c CSS]
                   [-f FIRST_WEEKDAY] [year] [month]
```

Por exemplo, para exibir um calendário para o ano 2000:

```
$ python -m calendar 2000
                                  2000

      January                   February                  March
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                1  2          1  2  3  4  5  6            1  2  3  4  5
 3  4  5  6  7  8  9       7  8  9 10 11 12 13       6  7  8  9 10 11 12
10 11 12 13 14 15 16      14 15 16 17 18 19 20      13 14 15 16 17 18 19
17 18 19 20 21 22 23      21 22 23 24 25 26 27      20 21 22 23 24 25 26
24 25 26 27 28 29 30      28 29                     27 28 29 30 31
31

       April                     May                       June
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                1  2       1  2  3  4  5  6  7               1  2  3  4
 3  4  5  6  7  8  9       8  9 10 11 12 13 14       5  6  7  8  9 10 11
10 11 12 13 14 15 16      15 16 17 18 19 20 21      12 13 14 15 16 17 18
17 18 19 20 21 22 23      22 23 24 25 26 27 28      19 20 21 22 23 24 25
24 25 26 27 28 29 30      29 30 31                  26 27 28 29 30

        July                    August                  September
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                1  2          1  2  3  4  5  6                  1  2  3
 3  4  5  6  7  8  9       7  8  9 10 11 12 13       4  5  6  7  8  9 10
10 11 12 13 14 15 16      14 15 16 17 18 19 20      11 12 13 14 15 16 17
17 18 19 20 21 22 23      21 22 23 24 25 26 27      18 19 20 21 22 23 24
24 25 26 27 28 29 30      28 29 30 31               25 26 27 28 29 30
31

      October                   November                  December
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                   1             1  2  3  4  5                  1  2  3
 2  3  4  5  6  7  8       6  7  8  9 10 11 12       4  5  6  7  8  9 10
 9 10 11 12 13 14 15      13 14 15 16 17 18 19      11 12 13 14 15 16 17
16 17 18 19 20 21 22      20 21 22 23 24 25 26      18 19 20 21 22 23 24
23 24 25 26 27 28 29      27 28 29 30               25 26 27 28 29 30 31
30 31
```

As seguintes opções são aceitas:

--help, -h
: Mostra a mensagem de ajuda e sai.

--locale LOCALE, **-L** LOCALE
: A localidade a ser usada para nomes de meses e dias da semana. O padrão é inglês.

--encoding ENCODING, **-e** ENCODING
: A codificação a ser usada para saída. *--encoding* é necessário se *--locale* estiver definido.

--type {text,html}, **-t** {text,html}

Exibe o calendário no terminal como texto ou como um documento HTML.

--first-weekday FIRST_WEEKDAY, **-f** FIRST_WEEKDAY

O dia da semana para começar cada semana. Deve ser um número entre 0 (segunda-feira) e 6 (domingo). O padrão é 0.

Adicionado na versão 3.13.

year

O ano para exibir o calendário. O padrão é o ano atual.

month

O mês do *year* especificado para exibir o calendário. Deve ser um número entre 1 e 12 e pode ser usado somente no modo texto. O padrão é exibir um calendário para o ano inteiro.

Opções de modo texto:

--width WIDTH, **-w** WIDTH

A largura da coluna de data em colunas terminais. A data é exibida centralizada na coluna. Qualquer valor menor que 2 é ignorado. O padrão é 2.

--lines LINES, **-l** LINES

O número de linhas para cada semana em linhas terminais. A data é exibida alinhada no topo. Qualquer valor menor que 1 é ignorado. O padrão é 1.

--spacing SPACING, **-s** SPACING

O espaço entre os meses em colunas. Qualquer valor menor que 2 é ignorado. O padrão é 6.

--months MONTHS, **-m** MONTHS

O número de meses exibidos por linha. O padrão é 3.

Alterado na versão 3.14: Por padrão, a data de hoje é destacada em cor e pode ser controlada usando variáveis de ambiente.

Opções de modo HTML:

--css CSS, **-c** CSS

O caminho de uma folha de estilo CSS a ser usada para o calendário. Isso deve ser relativo ao HTML gerado ou um HTTP absoluto ou URL `file:///`.

8.4 `collections` — Tipos de dados de contêineres

Código-fonte: Lib/collections/__init__.py

Este módulo implementa tipos de dados de contêineres especializados que fornecem alternativas aos contêineres embutidos do Python, *dict*, *list*, *set* e *tuple*.

namedtuple()	função fábrica (*factory function*) para criar subclasses de tuplas com campos nomeados
deque	contêiner lista ou similar com acréscimos e retiradas rápidas nas duas extremidades
ChainMap	classe dict ou similar para criar uma visão única de vários mapeamentos
Counter	subclasse de dict para contar objetos *hasheáveis*
OrderedDict	subclasse de dict que lembra a ordem que as entradas foram adicionadas
defaultdict	subclasse de dict que chama uma função fábrica para fornecer valores não encontrados
UserDict	invólucro em torno de objetos dicionário para facilitar fazer subclasse de dict
UserList	invólucro em torno de objetos lista para facilitar criação de subclasse de lista
UserString	invólucro em torno de objetos string para uma facilitar criação de subclasse de string

8.4.1 Objetos `ChainMap`

Adicionado na versão 3.3.

Uma classe *ChainMap* é fornecida para ligar rapidamente uma série de mapeamentos de forma que possam ser tratados como um só. O que é frequentemente mais rápido do que criar um novo dicionário e executar múltiplas chamadas de *update()*.

A classe pode ser usada para simular escopos aninhados e é útil em modelos.

class collections.**ChainMap**(**maps*)

ChainMap agrupa múltiplos dicts ou outros mapeamentos para criar uma única vista atualizável. Se nenhum *maps* for especificado, um dicionário vazio será fornecido para que uma nova cadeia tenha sempre pelo menos um mapeamento.

Os mapeamentos subjacentes são armazenados em uma lista. Essa lista é pública e pode ser acessada ou atualizada usando o atributo *maps*. Não existe outro estado.

Faz uma busca nos mapeamentos subjacentes sucessivamente até que uma chave seja encontrada. Em contraste, escrita, atualizações e remoções operam apenas no primeiro mapeamento.

Uma *ChainMap* incorpora os mapeamentos subjacentes por referência. Então, se um dos mapeamentos subjacentes for atualizado, essas alterações serão refletidas na *ChainMap*.

Todos os métodos usuais do dicionário são suportados. Além disso, existe um atributo *maps*, um método para criar novos subcontextos e uma propriedade para acessar todos, exceto o primeiro mapeamento:

maps

Uma lista de mapeamentos atualizáveis pelo usuário. A lista é ordenada desde o primeiro pesquisado até a última pesquisado. É o único estado armazenado e pode ser modificado para alterar quais mapeamentos são pesquisados. A lista deve sempre conter pelo menos um mapeamento.

new_child(*m=None*, ***kwargs*)

Retorna uma nova *ChainMap* contendo um novo mapa seguido de todos os mapas na instância atual. Se `m` for especificado, torna-se o novo mapa na frente da lista de mapeamentos; Se não especificado, é usado um dicionário vazio, de modo que chamar `d.new_child()` é equivalente a: `ChainMap({}, *d.maps)`. Se algum argumento nomeado for especificado, ele atualizará o mapa passado ou o novo dicionário vazio. Esse método é usado para criar subcontextos que podem ser atualizados sem alterar valores em nenhum dos mapeamentos pai.

Alterado na versão 3.4: O parâmetro opcional `m` foi adicionado.

Alterado na versão 3.10: Suporte a argumentos nomeados foi adicionado.

parents

Propriedade que retorna um novo *ChainMap* contendo todos os mapas da instância atual, exceto o primeiro. Isso é útil para pular o primeiro mapa da pesquisa. Os casos de uso são semelhantes aos do argumento nomeado `nonlocal` usada em *escopos aninhados*. Os casos de uso também são paralelos aos da função embutida *super()*. Uma referência a `d.parents` é equivalente a: `ChainMap(*d.maps[1:])`.

Observe, a ordem de iteração de um *ChainMap* é determinada pela varredura dos mapeamentos do último ao primeiro:

```
>>> baseline = {'music': 'bach', 'art': 'rembrandt'}
>>> adjustments = {'art': 'van gogh', 'opera': 'carmen'}
>>> list(ChainMap(adjustments, baseline))
['music', 'art', 'opera']
```

Isso dá a mesma ordem de uma série de chamadas de *dict.update()* começando com o último mapeamento:

```
>>> combined = baseline.copy()
>>> combined.update(adjustments)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> list(combined)
['music', 'art', 'opera']
```

Alterado na versão 3.9: Adicionado suporte para os operadores `|` e `|=`, especificados na **PEP 584**.

Ver também

- A classe MultiContext no pacote CodeTools de Enthought tem opções para oferecer suporte à escrita em qualquer mapeamento na cadeia.
- A classe Context do Django para modelos é uma cadeia de mapeamentos somente leitura. Ela também oferece o recurso de push e pop (inserir e retirar) contextos semelhantes ao método `new_child()` e a propriedade `parents`.
- A receita de Contextos Aninhados possui opções para controlar se escritas e outras mutações se aplicam a apenas o primeiro mapeamento ou para qualquer mapeamento na cadeia.
- Uma versão muito simplificada somente leitura do Chainmap.

Exemplos e Receitas de `ChainMap`

Esta seção mostra várias abordagens para trabalhar com mapas encadeados.

Exemplo de simulação da cadeia de busca interna do Python:

```
import builtins
pylookup = ChainMap(locals(), globals(), vars(builtins))
```

Exemplo de como permitir que os argumentos de linha de comando especificados pelo usuário tenham precedência sobre as variáveis de ambiente que, por sua vez, têm precedência sobre os valores padrão:

```
import os, argparse

defaults = {'color': 'red', 'user': 'guest'}

parser = argparse.ArgumentParser()
parser.add_argument('-u', '--user')
parser.add_argument('-c', '--color')
namespace = parser.parse_args()
command_line_args = {k: v for k, v in vars(namespace).items() if v is not None}

combined = ChainMap(command_line_args, os.environ, defaults)
print(combined['color'])
print(combined['user'])
```

Padrões de exemplo para utilização da classe *ChainMap* para simular contextos aninhados:

```
c = ChainMap()          # Cria o contexto raiz
d = c.new_child()       # Cria um contexto filho aninhado
e = c.new_child()       # Filho de c, independente de d
e.maps[0]               # Dicionário do contexto atual -- como locals() do Python
e.maps[-1]              # Contexto raiz -- como globals() do Python
e.parents               # Envolvendo cadeia de contexto -- como os nonlocals do␣
↪Python

d['x'] = 1              # Define valor no contexto atual
d['x']                  # Obtém a primeira chave na cadeia de contextos
del d['x']              # Exclui do contexto atual
```

(continua na próxima página)

(continuação da página anterior)

```
list(d)                  # Todos os valores aninhados
k in d                   # Verifica todos os valores aninhados
len(d)                   # Número de valores aninhados
d.items()                # Todos os itens aninhados
dict(d)                  # Achata em um dicionário comum
```

A classe *ChainMap* só faz atualizações (escritas e remoções) no primeiro mapeamento na cadeia, enquanto as pesquisas irão buscar em toda a cadeia. Contudo, se há o desejo de escritas e remoções profundas, é fácil fazer uma subclasse que atualiza chaves encontradas mais a fundo na cadeia:

```
class DeepChainMap(ChainMap):
    'Variant of ChainMap that allows direct updates to inner scopes'

    def __setitem__(self, key, value):
        for mapping in self.maps:
            if key in mapping:
                mapping[key] = value
                return
        self.maps[0][key] = value

    def __delitem__(self, key):
        for mapping in self.maps:
            if key in mapping:
                del mapping[key]
                return
        raise KeyError(key)

>>> d = DeepChainMap({'zebra': 'black'}, {'elephant': 'blue'}, {'lion': 'yellow'})
>>> d['lion'] = 'orange'         # atualiza uma chave existente dois níveis abaixo
>>> d['snake'] = 'red'           # novas chaves são adicionadas ao dict mais acima
>>> del d['elephant']            # remove uma chave existente um nível abaixo
>>> d                            # exibe o resultado
DeepChainMap({'zebra': 'black', 'snake': 'red'}, {}, {'lion': 'orange'})
```

8.4.2 Objetos `Counter`

Uma ferramenta de contagem é fornecida para apoiar contas rápidas e convenientes. Por exemplo:

```
>>> # Conta ocorrências de palavras em uma lista
>>> cnt = Counter()
>>> for word in ['red', 'blue', 'red', 'green', 'blue', 'blue']:
...     cnt[word] += 1
...
>>> cnt
Counter({'blue': 3, 'red': 2, 'green': 1})

>>> # Encontra as dez palavras mais comuns em Hamlet
>>> import re
>>> words = re.findall(r'\w+', open('hamlet.txt').read().lower())
>>> Counter(words).most_common(10)
[('the', 1143), ('and', 966), ('to', 762), ('of', 669), ('i', 631),
 ('you', 554),  ('a', 546), ('my', 514), ('hamlet', 471), ('in', 451)]
```

class collections.**Counter**([*iterable-or-mapping*])

Um *Counter* é uma subclasse de *dict* subclass para contagem de objetos *hasheáveis*. É uma coleção na qual elementos são armazenados como chaves de dicionário e suas contagens são armazenadas como valores

de dicionário. Contagens podem ser qualquer valor inteiro incluindo zero e contagens negativas. A classe *Counter* é similar a sacos ou multiconjuntos em outras linguagens.

Os elementos são contados a partir de um iterável *iterable* ou inicializado a partir de um outro mapeamento *mapping* (ou contador):

```
>>> c = Counter()                           # a new, empty counter
>>> c = Counter('gallahad')                 # a new counter from an iterable
>>> c = Counter({'red': 4, 'blue': 2})      # a new counter from a mapping
>>> c = Counter(cats=4, dogs=8)             # a new counter from keyword args
```

Objetos Counter tem uma interface de dicionário, com a diferença que devolvem uma contagem zero para itens que não estão presentes em vez de levantar a exceção *KeyError*:

```
>>> c = Counter(['eggs', 'ham'])
>>> c['bacon']                              # count of a missing element is␣
↪zero
0
```

Definir uma contagem como zero não remove um elemento do contador. Use `del` para o remover completamente.

```
>>> c['sausage'] = 0                        # counter entry with a zero count
>>> del c['sausage']                        # del actually removes the entry
```

Adicionado na versão 3.1.

Alterado na versão 3.7: Como uma subclasse de *dict*, *Counter* herda a capacidade de lembrar a ordem de inserção. Operações matemáticas em objetos *Counter* também preservam ordem. Os resultados são ordenados de acordo com o momento que um elemento é encontrado pela primeira vez no operando da esquerda e, em seguida, pela ordem encontrada no operando da direita.

Objetos Counter têm suporte a métodos adicionais além daqueles que já estão disponíveis para todos os dicionários:

elements()

Retorna um iterador sobre os elementos, repetindo cada um tantas vezes quanto sua contagem. Os elementos são retornados na ordem em que foram encontrados pela primeira vez. Se a contagem de um elemento é menor que um, ele será ignorado por *elements()* .

```
>>> c = Counter(a=4, b=2, c=0, d=-2)
>>> sorted(c.elements())
['a', 'a', 'a', 'a', 'b', 'b']
```

most_common([*n*])

Retorna uma lista dos *n* elementos mais comuns e suas contagens, do mais comum para o menos comum. Se *n* for omitido ou igual a `None`, *most_common()* retorna *todos* os elementos no contador. Elementos com contagens iguais são ordenados na ordem em que foram encontrados pela primeira vez:

```
>>> Counter('abracadabra').most_common(3)
[('a', 5), ('b', 2), ('r', 2)]
```

subtract([*iterable-or-mapping*])

Os elementos são subtraídos de um iterável *iterable* ou de outro mapeamento *mapping* (ou contador). Funciona como *dict.update()*, mas subtraindo contagens ao invés de substituí-las. Tanto as entradas quanto as saídas podem ser zero ou negativas.

```
>>> c = Counter(a=4, b=2, c=0, d=-2)
>>> d = Counter(a=1, b=2, c=3, d=4)
>>> c.subtract(d)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> c
Counter({'a': 3, 'b': 0, 'c': -3, 'd': -6})
```

Adicionado na versão 3.2.

total()

Calcula a soma das contagens.

```
>>> c = Counter(a=10, b=5, c=0)
>>> c.total()
15
```

Adicionado na versão 3.10.

Os métodos usuais de dicionário estão disponíveis para objetos *Counter*, exceto por dois que funcionam de forma diferente para contadores.

fromkeys(*iterable*)

Este método de classe não está implementado para objetos *Counter*.

update(*[iterable-or-mapping]*)

Elementos são contados a partir de um iterável *iterable* ou adicionados de outro mapeamento *mapping* (ou contador). Funciona como *dict.update()*, mas adiciona contagens em vez de substituí-las. Além disso, é esperado que o *iterable* seja uma sequência de elementos, e não uma sequência de pares `(key, value)`.

Os contadores oferecem suporte a operadores de comparação rica para relacionamentos de igualdade, subconjunto e superconjunto: `==`, `!=`, `<`, `<=`, `>`, `>=`. Todos esses testes tratam os elementos ausentes como tendo contagens zero para que `Counter(a=1) == Counter(a=1, b=0)` retorne verdadeiro.

Alterado na versão 3.10: Foram adicionadas operações de rica comparação.

Alterado na versão 3.10: Em testes de igualdade, os elementos ausentes são tratados como tendo contagens zero. Anteriormente, `Counter(a=3)` e `Counter(a=3, b=0)` eram considerados distintos.

Padrões comuns para trabalhar com objetos *Counter*:

```
c.total()                       # total de todas as contagens
c.clear()                       # redefine todas as contagens
list(c)                         # lista elementos únicos
set(c)                          # converte para um conjunto
dict(c)                         # converte para um dicionário comum
c.items()                       # acessa os pares (elem, cnt)
Counter(dict(list_of_pairs))    # converte de uma lista de pares (elem, cnt)
c.most_common()[:-n-1:-1]       # n últimos elementos comuns
+c                              # remove zero e contagens negativas
```

Diversas operações matemáticas são fornecidas combinando objetos do tipo *Counter* afim de se produzir multiconjuntos (Counters que possuem contagens maiores que 0). Adição e subtração combinam contadores adicionando ou subtraindo o contagem do elemento correspondente. Intersecção ou união retorna o mínimo e máximo da contagem correspondente. Igualdade e inclusão compara as contagens correspondentes. Cada operação aceita entradas com contagens assinadas, mas a saída vai excluir resultados com contagens de zero ou menos.

```
>>> c = Counter(a=3, b=1)
>>> d = Counter(a=1, b=2)
>>> c + d                       # adiciona dois contadores juntos:  c[x] + d[x]
Counter({'a': 4, 'b': 3})
>>> c - d                       # subtrai (mantendo apenas contagens positivas)
Counter({'a': 2})
>>> c & d                       # interseção:  min(c[x], d[x])
```

(continua na próxima página)

(continuação da página anterior)

```
Counter({'a': 1, 'b': 1})
>>> c | d                       # união:  max(c[x], d[x])
Counter({'a': 3, 'b': 2})
>>> c == d                      # igualdade:  c[x] == d[x]
False
>>> c <= d                      # inclusão:  c[x] <= d[x]
False
```

A adição e subtração unárias são atalhos para adicionar um contador vazio ou subtrair de um contador vazio.

```
>>> c = Counter(a=2, b=-4)
>>> +c
Counter({'a': 2})
>>> -c
Counter({'b': 4})
```

Adicionado na versão 3.3: Adicionado suporte para operador unário mais, unário menos e operações *in-place* em multiconjuntos.

Nota

Os contadores foram projetados principalmente para funcionar com números inteiros positivos para representar contagens contínuas; no entanto, foi tomado cuidado para não impedir desnecessariamente os casos de uso que precisassem de outros tipos ou valores negativos. Para ajudar nesses casos de uso, esta seção documenta o intervalo mínimo e as restrições de tipo.

- A própria classe `Counter` é uma subclasse de dicionário sem restrições em suas chaves e valores. Os valores pretendem ser números que representam contagens, mas você *pode* armazenar qualquer coisa no campo de valor.
- O método `most_common()` requer apenas que os valores sejam ordenáveis.
- Para operações in-place, como `c[key] += 1`, o tipo de valor precisa oferecer suporte a apenas adição e subtração. Portanto, frações, números de ponto flutuante e decimais funcionariam e os valores negativos são suportados. O mesmo também é verdadeiro para `update()` e `subtract()` que permitem valores negativos e zero para entradas e saídas.
- Os métodos de multiconjuntos são projetados apenas para casos de uso com valores positivos. As entradas podem ser negativas ou zero, mas apenas saídas com valores positivos são criadas. Não há restrições de tipo, mas o tipo de valor precisa suportar adição, subtração e comparação.
- O método `elements()` requer contagens de inteiros. Ele ignora contagens zero e negativas.

Ver também

- Classe Bag do Smalltalk.
- Entrada da Wikipédia para Multiconjuntos.
- Tutorial com exemplos de multiconjuntos no C++.
- Para operações matemáticas em multiconjuntos e seus casos de uso, consulte *Knuth, Donald. The Art of Computer Programming Volume II, Seção 4.6.3, Exercício 19*.
- Para enumerar todos os multiconjuntos distintos de um determinado tamanho em um determinado conjunto de elementos, consulte `itertools.combinations_with_replacement()`:

```
map(Counter, combinations_with_replacement('ABC', 2)) # --> AA AB AC BB BC␣
↪CC
```

8.4.3 Objetos deque

class collections.**deque**([*iterable*[, *maxlen*]])

Retorna um novo objeto deque inicializado da esquerda para a direita (usando *append()*) com dados do iterável *iterable*. Se *iterable* não for especificado, o novo deque estará vazio.

Deques são uma generalização de pilhas e filas (o nome é pronunciado "deck" e é abreviação de "double--ended queue", e conhecida como "fila duplamente terminada" em português). O Deques oferece suporte para acréscimos e retiradas seguros para thread e eficientes em uso memória de ambos os lados do deque com aproximadamente o mesmo desempenho $O(1)$ em qualquer direção.

Embora os objetos *list* ofereçam suporte a operações semelhantes, eles são otimizados para operações rápidas de comprimento fixo e sujeitam em custos de movimentação de memória $O(n)$ para as operações `pop(0)` e `insert(0, v)` que alteram o tamanho e a posição da representação de dados subjacente.

Se *maxlen* não for especificado ou for `None`, deques podem crescer para um comprimento arbitrário. Caso contrário, o deque é limitado ao comprimento máximo especificado. Quando um deque de comprimento limitado está cheio, quando novos itens são adicionados, um número correspondente de itens é descartado da extremidade oposta. Deques de comprimento limitado fornecem funcionalidade semelhante ao filtro `tail` no Unix. Eles também são úteis para rastrear transações e outras pools de dados onde apenas a atividade mais recente é de interesse.

Os objetos Deque oferecem suporte aos seguintes métodos:

append(*x*)

Adiciona *x* ao lado direito do deque.

appendleft(*x*)

Adiciona *x* ao lado esquerdo do deque

clear()

Remove todos os elementos do deque deixando-o com comprimento 0.

copy()

Cria uma cópia rasa do deque.

Adicionado na versão 3.5.

count(*x*)

Conta o número de elementos deque igual a *x*.

Adicionado na versão 3.2.

extend(*iterable*)

Estende o lado direito do deque anexando elementos do argumento iterável.

extendleft(*iterable*)

Estende o lado esquerdo do deque anexando elementos de *iterable*. Observe que a série de acréscimos à esquerda resulta na reversão da ordem dos elementos no argumento iterável.

index(*x*[, *start*[, *stop*]])

Retorna a posição de *x* no deque (no ou após o índice *start* e antes do índice *stop*). Retorna a primeira correspondência ou levanta *ValueError* se não for encontrado.

Adicionado na versão 3.5.

insert(*i*, *x*)

Insere *x* no deque na posição *i*.

Se a inserção fizer com que um deque limitado cresça além de *maxlen*, uma *IndexError* é levantada.

Adicionado na versão 3.5.

pop()

Remove e devolve um elemento do lado direito do deque. Se nenhum elemento estiver presente, levanta um *IndexError*.

popleft()

Remove e devolve um elemento do lado esquerdo do deque. Se nenhum elemento estiver presente, levanta um *IndexError*.

remove(*value*)

Remove a primeira ocorrência de *value*. Se não for encontrado, levanta um *ValueError*.

reverse()

Inverte os elementos do deque no local e, em seguida, retorna `None`.

Adicionado na versão 3.2.

rotate(*n=1*)

Gira o deque *n* passos para a direita. Se *n* for negativo, gire para a esquerda.

Quando o deque não está vazio, girar um passo para a direita é equivalente a `d.appendleft(d.pop())` e girar um passo para a esquerda é equivalente a `d.append(d.popleft())`.

Os objetos Deque também fornecem um atributo somente leitura:

maxlen

Tamanho máximo de um deque ou `None` se ilimitado.

Adicionado na versão 3.1.

Além do acima, deques oferece suporte a iteração, serialização com pickle, `len(d)`, `reversed(d)`, `copy.copy(d)`, `copy.deepcopy(d)` e teste de associação com o operador `in` e referências subscritas, como `d[0]` para acessar o primeiro elemento. O acesso indexado é *O*(1) em ambas as extremidades, mas diminui para *O*(*n*) no meio. Para acesso aleatório rápido, use listas.

A partir da versão 3.5, deques oferecem suporte a `__add__()`,"__mul__()" e `__imul__()`.

Exemplo:

```
>>> from collections import deque
>>> d = deque('ghi')                 # torna um novo deque com três itens
>>> for elem in d:                   # itera os elementos do deque
...     print(elem.upper())
G
H
I

>>> d.append('j')                    # adiciona uma nova entrada ao lado direito
>>> d.appendleft('f')                # adiciona uma nova entrada ao lado esquerdo
>>> d                                # mostra a representação do deque
deque(['f', 'g', 'h', 'i', 'j'])

>>> d.pop()                          # retorna e remove o item mais à direita
'j'
>>> d.popleft()                      # retorna e remove o item mais à esquerda
'f'
>>> list(d)                          # lista o conteúdo do deque
['g', 'h', 'i']
>>> d[0]                             # exibe o item mais à esquerda
'g'
>>> d[-1]                            # exibe o item mais à direita
'i'
```

(continua na próxima página)

(continuação da página anterior)

```
>>> list(reversed(d))               # lista o conteúdo de um deque ao inverso
['i', 'h', 'g']
>>> 'h' in d                        # pesquisa no deque
True
>>> d.extend('jkl')                 # adiciona vários elementos de uma só vez
>>> d
deque(['g', 'h', 'i', 'j', 'k', 'l'])
>>> d.rotate(1)                     # rotação para direita
>>> d
deque(['l', 'g', 'h', 'i', 'j', 'k'])
>>> d.rotate(-1)                    # rotação para esquerda
>>> d
deque(['g', 'h', 'i', 'j', 'k', 'l'])

>>> deque(reversed(d))              # cria um novo deque na ordem inversa
deque(['l', 'k', 'j', 'i', 'h', 'g'])
>>> d.clear()                       # esvazia o deque
>>> d.pop()                         # não é possível retirar um item de um deque␣
↪vazio
Traceback (most recent call last):
    File "<pyshell#6>", line 1, in -toplevel-
        d.pop()
IndexError: pop from an empty deque

>>> d.extendleft('abc')             # extendleft() inverte a ordem da entrada
>>> d
deque(['c', 'b', 'a'])
```

Receitas de deque

Esta seção mostra várias abordagens para trabalhar com deques.

Deques de comprimento limitado fornecem funcionalidade semelhante ao filtro `tail` em Unix:

```
def tail(filename, n=10):
    'Return the last n lines of a file'
    with open(filename) as f:
        return deque(f, n)
```

Outra abordagem para usar deques é manter uma sequência de elementos adicionados recentemente, acrescentando à direita e clicando à esquerda:

```
def moving_average(iterable, n=3):
    # moving_average([40, 30, 50, 46, 39, 44]) --> 40.0 42.0 45.0 43.0
    # https://en.wikipedia.org/wiki/Moving_average
    it = iter(iterable)
    d = deque(itertools.islice(it, n-1))
    d.appendleft(0)
    s = sum(d)
    for elem in it:
        s += elem - d.popleft()
        d.append(elem)
        yield s / n
```

Um escalonador round robin pode ser implementado com iteradores de entrada armazenados em um *deque*. Os valores são produzidos a partir do iterador ativo na posição zero. Se esse iterador estiver esgotado, ele pode ser removido com *popleft()*; caso contrário, ele pode voltar ao fim com o método *rotate()*:

```
def roundrobin(*iterables):
    "roundrobin('ABC', 'D', 'EF') --> A D E B F C"
    iterators = deque(map(iter, iterables))
    while iterators:
        try:
            while True:
                yield next(iterators[0])
                iterators.rotate(-1)
        except StopIteration:
            # Remove um iterador esgotado.
            iterators.popleft()
```

O método *rotate()* fornece uma maneira de implementar o fatiamento e exclusão *deque*. Por exemplo, uma implementação Python pura de `del d[n]` depende do método `rotate()` para posicionar os elementos a serem retirados:

```
def delete_nth(d, n):
    d.rotate(-n)
    d.popleft()
    d.rotate(n)
```

Para implementar o fatiamento de *deque*, use uma abordagem semelhante aplicando *rotate()* para trazer um elemento alvo para o lado esquerdo do deque. Remova as entradas antigas com *popleft()*, adicione novas entradas com *extend()*, e, então, inverta a rotação. Com pequenas variações dessa abordagem, é fácil implementar manipulações de pilha de estilo Forth, como `dup`, `drop`, `swap`, `over`, `pick`, `rot` e `roll`.

8.4.4 Objetos `defaultdict`

class collections.**defaultdict**(*default_factory=None*, /[, ...])

Retorna um novo objeto dicionário ou similar. *defaultdict* é uma subclasse da classe embutida *dict*. Ele substitui um método e adiciona uma variável de instância gravável. A funcionalidade restante é a mesma da classe *dict* e não está documentada aqui.

O primeiro argumento fornece o valor inicial para o atributo *default_factory*; o padrão é `None`. Todos os argumentos restantes são tratados da mesma forma como se fossem passados para o construtor *dict*, incluindo argumentos nomeados.

Os objetos *defaultdict* oferecem suporta ao seguinte método além das operações padrão *dict*:

__missing__(*key*)

Se o atributo *default_factory* for `None`, isso levanta uma exceção *KeyError* com *key* como argumento.

Se *default_factory* não for `None`, ele é chamado sem argumentos para fornecer um valor padrão para a chave *key* fornecida, este valor é inserido no dicionário para *key* e retornado.

Se chamar *default_factory* levanta uma exceção, esta exceção é propagada inalterada.

Este método é chamado pelo método `__getitem__()` da classe *dict* quando a chave solicitada não é encontrada; tudo o que ele retorna ou levanta é então retornado ou levantado por `__getitem__()`.

Observe que *__missing__() não* é chamado para nenhuma operação além de `__getitem__()`. Isso significa que *get()* irá, como dicionários normais, retornar `None` como padrão ao invés de usar *default_factory*.

Objetos *defaultdict* permitem a seguinte variável de instância:

default_factory

Este atributo é usado pelo método *__missing__()*; ele é inicializado a partir do primeiro argumento para o construtor, se presente, ou para `None`, se ausente.

Alterado na versão 3.9: Adicionado operadores de mesclagem (|) e de atualização (|=), especificados na **PEP 584**.

Exemplos de `defaultdict`

Usando *list* como *default_factory*, se for fácil agrupar a sequencia dos pares chave-valores num dicionário de listas

```
>>> s = [('yellow', 1), ('blue', 2), ('yellow', 3), ('blue', 4), ('red', 1)]
>>> d = defaultdict(list)
>>> for k, v in s:
...     d[k].append(v)
...
>>> sorted(d.items())
[('blue', [2, 4]), ('red', [1]), ('yellow', [1, 3])]
```

Quando cada chave é encontrada pela primeira vez, ela ainda não está no mapeamento; então uma entrada é criada automaticamente usando a função *default_factory* que retorna uma *list* vazia. A operação *list.append()* então anexa o valor à nova lista. Quando as chaves são encontradas novamente, a pesquisa prossegue normalmente (retornando a lista daquela chave) e a operação *list.append()* adiciona outro valor à lista. Esta técnica é mais simples e rápida que uma técnica equivalente usando *dict.setdefault()*:

```
>>> d = {}
>>> for k, v in s:
...     d.setdefault(k, []).append(v)
...
>>> sorted(d.items())
[('blue', [2, 4]), ('red', [1]), ('yellow', [1, 3])]
```

Definir *default_factory* como *int* torna *defaultdict* útil para contagem (como um multiconjunto em outras linguagens):

```
>>> s = 'mississippi'
>>> d = defaultdict(int)
>>> for k in s:
...     d[k] += 1
...
>>> sorted(d.items())
[('i', 4), ('m', 1), ('p', 2), ('s', 4)]
```

Quando uma letra é encontrada pela primeira vez, ela está ausente no mapeamento, então a função *default_factory* chama *int()* para fornecer uma contagem padrão de zero. A operação de incremento então cria a contagem para cada letra.

A função *int()* que sempre retorna zero é apenas um caso especial de funções constantes. Uma maneira mais rápida e flexível de criar funções constantes é usar uma função lambda que pode fornecer qualquer valor constante (não apenas zero):

```
>>> def constant_factory(value):
...     return lambda: value
...
>>> d = defaultdict(constant_factory('<missing>'))
>>> d.update(name='John', action='ran')
>>> '%(name)s %(action)s to %(object)s' % d
'John ran to <missing>'
```

Definir *default_factory* como *set* torna *defaultdict* útil para construir um dicionário de conjuntos:

```
>>> s = [('red', 1), ('blue', 2), ('red', 3), ('blue', 4), ('red', 1), ('blue', 4)]
>>> d = defaultdict(set)
>>> for k, v in s:
...     d[k].add(v)
...
>>> sorted(d.items())
[('blue', {2, 4}), ('red', {1, 3})]
```

8.4.5 Função de fábrica para tuplas com campos nomeados `namedtuple()`

Tuplas nomeadas determinam o significado de cada posição numa tupla e permitem um código mais legível e autodocumentado. Podem ser usadas sempre que tuplas regulares forem utilizadas, e adicionam a possibilidade de acessar campos pelo nome ao invés da posição do índice.

`collections.`**`namedtuple`**(*typename*, *field_names*, *, *rename=False*, *defaults=None*, *module=None*)

Retorna uma nova subclasse de tupla chamada *typename*. A nova subclasse é usada para criar objetos tupla ou similares que possuem campos acessíveis por pesquisa de atributos, além de serem indexáveis e iteráveis. As instâncias da subclasse também possuem uma docstring útil (com *typename* e *field_names*) e um método útil `__repr__()` que lista o conteúdo da tupla em um formato `nome=valor`.

field_names são uma sequência de strings como `['x', 'y']`. Alternativamente, *field_names* pode ser uma única string com cada nome de campo separado por espaços em branco e/ou vírgulas como, por exemplo, `'x y'` ou `'x, y'`.

Qualquer identificador Python válido pode ser usado para um nome de campo, exceto para nomes que começam com um sublinhado. Identificadores válidos consistem em letras, dígitos e sublinhados, mas não começam com um dígito ou sublinhado e não podem ser uma *keyword* como *class*, *for*, *return*, *global*, *pass* ou *raise*.

Se *rename* for verdadeiro, nomes de campos inválidos serão automaticamente substituídos por nomes posicionais. Por exemplo, `['abc', 'def', 'ghi', 'abc']` é convertido para `['abc', '_1', 'ghi', '_3']`, eliminando a palavra reservada `def` e o nome de campo duplicado `abc`.

defaults pode ser `None` ou um *iterável* de valores padrão. Como os campos com valor padrão devem vir depois de qualquer campo sem padrão, os *padrões* são aplicados aos parâmetros mais à direita. Por exemplo, se os nomes dos campos forem `['x', 'y', 'z']` e os padrões forem `(1, 2)`, então `x` será um argumento obrigatório, `y` será o padrão `1`, e `z` será o padrão `2`.

Se *module* for definido, o atributo `__module__` da tupla nomeada será definido com esse valor.

As instâncias de tuplas nomeadas não possuem dicionários por instância, portanto são leves e não requerem mais memória do que as tuplas normais.

Para prover suporte para a serialização com pickle, a classe de tupla nomeada deve ser atribuída a uma variável que corresponda a *typename*.

Alterado na versão 3.1: Adicionado suporte a *rename*.

Alterado na versão 3.6: Os parâmetros *verbose* e *rename* tornaram-se *argumentos somente-nomeados*.

Alterado na versão 3.6: Adicionado o parâmetro *module*.

Alterado na versão 3.7: Removido o parâmetro *verbose* e o atributo `_source`.

Alterado na versão 3.7: Adicionado o parâmetro *defaults* e o atributo `_field_defaults`.

```
>>> # Exemplo básico
>>> Point = namedtuple('Point', ['x', 'y'])
>>> p = Point(11, y=22)     # instancia com argumentos nomeados ou posicionais
>>> p[0] + p[1]             # indexável como a tupla plana (11, 22)
33
>>> x, y = p                # desempacota como uma tupla normal
>>> x, y
(11, 22)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> p.x + p.y               # campos também acessíveis pelo nome
33
>>> p                       # __repr__ legível com o estilo nome=valor
Point(x=11, y=22)
```

Tuplas nomeadas são especialmente úteis para atribuir nomes de campos a tuplas de resultados retornadas pelos módulos *csv* ou *sqlite3*:

```
EmployeeRecord = namedtuple('EmployeeRecord', 'name, age, title, department,␣
↪paygrade')

import csv
for emp in map(EmployeeRecord._make, csv.reader(open("employees.csv", "rb"))):
    print(emp.name, emp.title)

import sqlite3
conn = sqlite3.connect('/companydata')
cursor = conn.cursor()
cursor.execute('SELECT name, age, title, department, paygrade FROM employees')
for emp in map(EmployeeRecord._make, cursor.fetchall()):
    print(emp.name, emp.title)
```

Além dos métodos herdados das tuplas, as tuplas nomeadas oferecem suporte a três métodos adicionais e dois atributos. Para evitar conflitos com nomes de campos, os nomes de métodos e atributos começam com um sublinhado.

classmethod somenamedtuple.**_make**(*iterable*)

Método de classe que cria uma nova instância a partir de uma sequência existente ou iterável.

```
>>> t = [11, 22]
>>> Point._make(t)
Point(x=11, y=22)
```

somenamedtuple.**_asdict**()

Retorna um novo *dict* que mapeia nomes de campo para seus respectivos valores:

```
>>> p = Point(x=11, y=22)
>>> p._asdict()
{'x': 11, 'y': 22}
```

Alterado na versão 3.1: Retorna um *OrderedDict* em vez de um *dict* normal.

Alterado na versão 3.8: Retorna um *dict* regular em vez de um *OrderedDict*. A partir do Python 3.7, é garantido que os dicionários regulares sejam ordenados. Se os recursos extras de *OrderedDict* forem necessários, a correção sugerida é converter o resultado para o tipo desejado: `OrderedDict(nt._asdict())`.

somenamedtuple.**_replace**(***kwargs*)

Retorna uma nova instância da tupla nomeada substituindo os campos especificados por novos valores:

```
>>> p = Point(x=11, y=22)
>>> p._replace(x=33)
Point(x=33, y=22)

>>> for partnum, record in inventory.items():
...     inventory[partnum] = record._replace(price=newprices[partnum],␣
↪timestamp=time.now())
```

Tuplas nomeadas também são suportados pela função genérica *copy.replace()*.

Alterado na versão 3.13: Levanta *TypeError* em vez de *ValueError* para argumentos nomeados inválidos.

somenamedtuple.**_fields**

Tupla de strings listando os nomes dos campos. Útil para introspecção e para criar novos tipos de tuplas nomeadas a partir de tuplas nomeadas existentes.

```
>>> p._fields            # exibe os nomes de campos
('x', 'y')

>>> Color = namedtuple('Color', 'red green blue')
>>> Pixel = namedtuple('Pixel', Point._fields + Color._fields)
>>> Pixel(11, 22, 128, 255, 0)
Pixel(x=11, y=22, red=128, green=255, blue=0)
```

somenamedtuple.**_field_defaults**

Dicionário mapeando nomes de campos para valores padrão.

```
>>> Account = namedtuple('Account', ['type', 'balance'], defaults=[0])
>>> Account._field_defaults
{'balance': 0}
>>> Account('premium')
Account(type='premium', balance=0)
```

Para recuperar um campo cujo nome está armazenado em uma string, use a função *getattr()*:

```
>>> getattr(p, 'x')
11
```

Para converter um dicionário em uma tupla nomeada, use o operador estrela dupla (conforme descrito em tut-unpacking-arguments):

```
>>> d = {'x': 11, 'y': 22}
>>> Point(**d)
Point(x=11, y=22)
```

Como uma tupla nomeada é uma classe regular do Python, é fácil adicionar ou alterar funcionalidades com uma subclasse. Veja como adicionar um campo calculado e um formato de impressão de largura fixa:

```
>>> class Point(namedtuple('Point', ['x', 'y'])):
...     __slots__ = ()
...     @property
...     def hypot(self):
...         return (self.x ** 2 + self.y ** 2) ** 0.5
...     def __str__(self):
...         return 'Point: x=%6.3f  y=%6.3f  hypot=%6.3f' % (self.x, self.y, self.
↪hypot)

>>> for p in Point(3, 4), Point(14, 5/7):
...     print(p)
Point: x= 3.000  y= 4.000  hypot= 5.000
Point: x=14.000  y= 0.714  hypot=14.018
```

A subclasse mostrada acima define `__slots__` como uma tupla vazia. Isso ajuda a manter baixos os requisitos de memória, evitando a criação de dicionários de instância.

A criação de subclasse não é útil para adicionar novos campos armazenados. Em vez disso, simplesmente crie um novo tipo de tupla nomeado a partir do atributo *_fields*:

```
>>> Point3D = namedtuple('Point3D', Point._fields + ('z',))
```

Docstrings podem ser personalizados fazendo atribuições diretas aos campos __doc__:

```
>>> Book = namedtuple('Book', ['id', 'title', 'authors'])
>>> Book.__doc__ += ': Hardcover book in active collection'
>>> Book.id.__doc__ = '13-digit ISBN'
>>> Book.title.__doc__ = 'Title of first printing'
>>> Book.authors.__doc__ = 'List of authors sorted by last name'
```

Alterado na versão 3.5: Os docstrings de propriedade tornaram-se graváveis.

Ver também

- Veja *typing.NamedTuple* para uma maneira de adicionar dicas de tipo para tuplas nomeadas. Ele também fornece uma notação elegante usando a palavra reservada `class`:

```
class Component(NamedTuple):
    part_number: int
    weight: float
    description: Optional[str] = None
```

- Veja *types.SimpleNamespace()* para um espaço de nomes mutável baseado em um dicionário subjacente em vez de uma tupla.
- O módulo *dataclasses* fornece um decorador e funções para adicionar automaticamente métodos especiais gerados a classes definidas pelo usuário.

8.4.6 Objetos `OrderedDict`

Os dicionários ordenados são como os dicionários normais, mas possuem alguns recursos extras relacionados às operações de pedido. Eles se tornaram menos importantes agora que a classe embutida *dict* ganhou a capacidade de lembrar a ordem de inserção (esse novo comportamento foi garantido no Python 3.7).

Algumas diferenças de *dict* ainda permanecem:

- O *dict* regular foi projetado para ser muito bom em operações de mapeamento. O rastreamento do pedido de inserção era secundário.
- O *OrderedDict* foi projetado para ser bom em operações de reordenação. A eficiência de espaço, a velocidade de iteração e o desempenho das operações de atualização eram secundários.
- O algoritmo *OrderedDict* pode lidar com operações de reordenação frequentes melhor do que *dict*. Conforme mostrado nas receitas abaixo, isso o torna adequado para implementar vários tipos de caches LRU.
- A operação de igualdade para *OrderedDict* verifica a ordem correspondente.

  Um *dict* regular pode emular o teste de igualdade sensível à ordem com `p == q and all(k1 == k2 for k1, k2 in zip(p, q))`.
- O método *popitem()* de *OrderedDict* tem uma assinatura diferente. Ele aceita um argumento opcional para especificar qual item será exibido.

  Um *dict* normal pode emular o `od.popitem(last=True)` do OrderedDict com `d.popitem()` que é garantido para exibir o (último) item mais à direita.

  Um *dict* normal pode emular o `od.popitem(last=False)` do OrderedDict com `(k := next(iter(d)), d.pop(k))` que retornará e remova o item mais à esquerda (primeiro), se existir.
- *OrderedDict* possui um método *move_to_end()* para reposicionar eficientemente um elemento em um endpoint.

  Um *dict* normal pode emular o `od.move_to_end(k, last=True)` do OrderedDict com `d[k] = d.pop(k)` que moverá a chave e seu valor associado para a posição mais à direita (última).

  Um *dict* regular não tem um equivalente eficiente para o `od.move_to_end(k, last=False)` do OrderedDict, que move a chave e seu valor associado para a posição mais à esquerda (primeira).

- Até o Python 3.8, *dict* não tinha um método `__reversed__()`.

class collections.**OrderedDict**([*items*])

Retorna uma instância de uma subclasse *dict* que possui métodos especializados para reorganizar a ordem do dicionário.

Adicionado na versão 3.1.

popitem(*last=True*)

O método *popitem()* para dicionários ordenados retorna e remove um par (chave, valor). Os pares são retornados na ordem LIFO se *last* for verdadeiro ou na ordem FIFO (primeiro a entrar, primeiro a sair) se for falso.

move_to_end(*key*, *last=True*)

Move uma chave *key* existente para qualquer extremidade de um dicionário ordenado. O item é movido para a extremidade direita se *last* for verdadeiro (o padrão) ou para o início se *último* for falso. Levanta *KeyError* se a *key* não existir:

```
>>> d = OrderedDict.fromkeys('abcde')
>>> d.move_to_end('b')
>>> ''.join(d)
'acdeb'
>>> d.move_to_end('b', last=False)
>>> ''.join(d)
'bacde'
```

Adicionado na versão 3.2.

Além dos métodos usuais de mapeamento, dicionários ordenados também oferecem suporte a iteração reversa usando a função *reversed()*.

Testes de igualdade entre objetos *OrderedDict* são sensíveis à ordem e são aproximadamente equivalentes a `list(od1.items())==list(od2.items())`.

Testes de igualdade entre objetos *OrderedDict* e outros objetos *Mapping* não diferenciam a ordem como dicionários regulares. Isso permite que objetos *OrderedDict* sejam substituídos em qualquer lugar que um dicionário regular seja usado.

Alterado na versão 3.5: Os itens, chaves e valores de *visões* de *OrderedDict* agora oferecem suporte a iteração reversa usando *reversed()*.

Alterado na versão 3.6: Com a aceitação da **PEP 468**, a ordem é mantida para argumentos nomeados passados para o construtor *OrderedDict* e seu método *update()*.

Alterado na versão 3.9: Adicionado operadores de mesclagem (`|`) e de atualização (`|=`), especificados na **PEP 584**.

Exemplos e receitas de `OrderedDict`

É simples criar uma variante de dicionário ordenado que lembre a ordem em que as chaves foram inseridas pela *última* vez. Se uma nova entrada substituir uma entrada existente, a posição de inserção original será alterada e movida para o final:

```
class LastUpdatedOrderedDict(OrderedDict):
    'Store items in the order the keys were last added'

    def __setitem__(self, key, value):
        super().__setitem__(key, value)
        self.move_to_end(key)
```

Um *OrderedDict* também seria útil para implementar variantes de *functools.lru_cache()*:

```
from collections import OrderedDict
from time import time

class TimeBoundedLRU:
    "LRU Cache that invalidates and refreshes old entries."

    def __init__(self, func, maxsize=128, maxage=30):
        self.cache = OrderedDict()      # { args : (timestamp, result)}
        self.func = func
        self.maxsize = maxsize
        self.maxage = maxage

    def __call__(self, *args):
        if args in self.cache:
            self.cache.move_to_end(args)
            timestamp, result = self.cache[args]
            if time() - timestamp <= self.maxage:
                return result
        result = self.func(*args)
        self.cache[args] = time(), result
        if len(self.cache) > self.maxsize:
            self.cache.popitem(last=False)
        return result
```

```
class MultiHitLRUCache:
    """ LRU cache that defers caching a result until
        it has been requested multiple times.

        To avoid flushing the LRU cache with one-time requests,
        we don't cache until a request has been made more than once.

    """

    def __init__(self, func, maxsize=128, maxrequests=4096, cache_after=1):
        self.requests = OrderedDict()   # { uncached_key : request_count }
        self.cache = OrderedDict()      # { cached_key : function_result }
        self.func = func
        self.maxrequests = maxrequests  # max number of uncached requests
        self.maxsize = maxsize          # max number of stored return values
        self.cache_after = cache_after

    def __call__(self, *args):
        if args in self.cache:
            self.cache.move_to_end(args)
            return self.cache[args]
        result = self.func(*args)
        self.requests[args] = self.requests.get(args, 0) + 1
        if self.requests[args] <= self.cache_after:
            self.requests.move_to_end(args)
            if len(self.requests) > self.maxrequests:
                self.requests.popitem(last=False)
        else:
            self.requests.pop(args, None)
            self.cache[args] = result
            if len(self.cache) > self.maxsize:
                self.cache.popitem(last=False)
        return result
```

8.4.7 Objetos `UserDict`

A classe *`UserDict`* atua como um invólucro em torno de objetos dicionário. A necessidade desta classe foi parcialmente suplantada pela capacidade de criar subclasses diretamente de *`dict`*; entretanto, essa classe pode ser mais fácil de trabalhar porque o dicionário subjacente é acessível como um atributo.

class collections.**UserDict**([*initialdata*])

Classe que simula um dicionário. O conteúdo da instância é mantido em um dicionário regular, que é acessível através do atributo *`data`* das instâncias *`UserDict`*. Se *initialdata* for fornecido, *`data`* é inicializado com seu conteúdo; observe que a referência a *initialdata* não será mantida, permitindo sua utilização para outros fins.

Além de prover suporte aos métodos e operações de mapeamentos, as instâncias *`UserDict`* fornecem o seguinte atributo:

data

Um dicionário real usado para armazenar o conteúdo da classe *`UserDict`*.

8.4.8 Objetos `UserList`

Esta classe atua como um invólucro em torno de objetos de lista. É uma classe base útil para suas próprias classes semelhantes a listas, que podem herdar delas e substituir métodos existentes ou adicionar novos. Desta forma, é possível adicionar novos comportamentos às listas.

A necessidade desta classe foi parcialmente suplantada pela capacidade de criar subclasses diretamente de *`list`*; no entanto, pode ser mais fácil trabalhar com essa classe porque a lista subjacente pode ser acessada como um atributo.

class collections.**UserList**([*list*])

Classe que simula uma lista. O conteúdo da instância é mantido em uma lista regular, que é acessível através do atributo *`data`* das instâncias *`UserList`*. O conteúdo da instância é inicialmente definido como uma cópia de *list*, padronizando a lista vazia `[]`. *list* pode ser qualquer iterável, por exemplo, uma lista Python real ou um objeto *`UserList`*.

Além de prover suporte aos métodos e operações de sequências mutáveis, as instâncias *`UserList`* fornecem o seguinte atributo:

data

Um objeto *`list`* real usado para armazenar o conteúdo da classe *`UserList`*.

Requisitos para criar subclasse: Espera-se que as subclasses de *`UserList`* ofereçam um construtor que pode ser chamado sem argumentos ou com um argumento. Listar operações que retornam uma nova sequência tenta criar uma instância da classe de implementação real. Para isso, presume que o construtor pode ser chamado com um único parâmetro, que é um objeto de sequência usado como fonte de dados.

Se uma classe derivada não desejar atender a este requisito, todos os métodos especiais suportados por esta classe precisarão ser substituídos; consulte as fontes para obter informações sobre os métodos que precisam ser fornecidos nesse caso.

8.4.9 Objetos `UserString`

A classe *`UserString`* atua como um invólucro em torno de objetos string. A necessidade desta classe foi parcialmente suplantada pela capacidade de criar subclasses diretamente de *`str`*; entretanto, essa classe pode ser mais fácil de trabalhar porque a string subjacente é acessível como um atributo.

class collections.**UserString**(*seq*)

Classe que simula um objeto string. O conteúdo da instância é mantido em um objeto string regular, que é acessível através do atributo *`data`* das instâncias *`UserString`*. O conteúdo da instância é inicialmente definido como uma cópia de *seq*. O argumento *seq* pode ser qualquer objeto que possa ser convertido em uma string usando a função embutida *`str()`*.

Além de prover suporte aos métodos e operações de strings, as instâncias *`UserString`* fornecem o seguinte atributo:

data

Um objeto *str* real usado para armazenar o conteúdo da classe *UserString*.

Alterado na versão 3.5: Novos métodos `__getnewargs__`, `__rmod__`, `casefold`, `format_map`, `isprintable` e `maketrans`.

8.5 `collections.abc` — Classes Base Abstratas para Contêineres

Adicionado na versão 3.3: Anteriormente, esse módulo fazia parte do módulo *collections*.

Código-fonte: Lib/_collections_abc.py

Esse módulo fornece *classes base abstratas* que podem ser usadas para testar se uma classe fornece uma interface específica; por exemplo, se é *hasheável* ou se é um *mapeamento*.

Um teste *issubclass()* ou *isinstance()* para uma interface funciona em uma das três formas.

1) Uma classe recém-escrita pode herdar diretamente de uma das classes base abstratas. A classe deve fornecer os métodos abstratos necessários. Os métodos mixin restantes vêm da herança e podem ser substituídos se desejado. Outros métodos podem ser adicionados conforme necessário:

```
class C(Sequence):                          # Herança direta
    def __init__(self): ...                 # Método extra não exigido pela ABC
    def __getitem__(self, index):  ...      # Método abstrato exigido
    def __len__(self):  ...                 # Método abstrato exigido
    def count(self, value): ...             # Opcionalmente substitui um método␣
↪mixin
```

```
>>> issubclass(C, Sequence)
True
>>> isinstance(C(), Sequence)
True
```

2) Classes existentes e classes embutidas podem ser registradas como "subclasses virtuais" dos ABCs. Essas classes devem definir a API completa, incluindo todos os métodos abstratos e todos os métodos mixin. Isso permite que os usuários confiem nos testes *issubclass()* ou *isinstance()* para determinar se a interface completa é suportada. A exceção a essa regra é para métodos que são automaticamente inferidos do restante da API:

```
class D:                                    # Sem herança
    def __init__(self): ...                 # Método extra exigido pela ABC
    def __getitem__(self, index):  ...      # Método abstrato
    def __len__(self):  ...                 # Método abstrato
    def count(self, value): ...             # Método mixin
    def index(self, value): ...             # Método mixin

Sequence.register(D)                        # Registra ao invés de herdar
```

```
>>> issubclass(D, Sequence)
True
>>> isinstance(D(), Sequence)
True
```

Neste exemplo, a classe `D` não precisa definir `__contains__`, `__iter__` e `__reversed__` porque o operador in, a lógica *iteration* e a função *reversed()* retornam automaticamente para o uso de `__getitem__` e `__len__`.

3) Algumas interfaces simples são diretamente reconhecíveis pela presença dos métodos necessários (a menos que esses métodos tenham sido definidos como *None*):

```
class E:
    def __iter__(self): ...
    def __next__(self): ...
```

```
>>> issubclass(E, Iterable)
True
>>> isinstance(E(), Iterable)
True
```

Interfaces complexas não oferecem suporte a esta última técnica porque uma interface é mais do que apenas a presença de nomes de métodos. Interfaces especificam semântica e relacionamentos entre métodos que não podem ser inferidos somente da presença de nomes de métodos específicos. Por exemplo, saber que uma classe fornece `__getitem__`, `__len__` e `__iter__` é insuficiente para distinguir uma *`Sequence`* de uma *`Mapping`*.

Adicionado na versão 3.9: Essas classes abstratas agora oferecem suporte a `[]`. Veja *Tipo Generic Alias* e **PEP 585**.

8.5.1 Classes Base Abstratas de Coleções

O módulo de coleções oferece o seguinte *ABCs*:

ABC	Herda de	Métodos abstratos	Métodos mixin
Container[1]		`__contains__`	
Hashable[1]		`__hash__`	
Iterable[1][2]		`__iter__`	
Iterator[1]	*Iterable*	`__next__`	`__iter__`
Reversible[1]	*Iterable*	`__reversed__`	
Generator[1]	*Iterator*	`send`, `throw`	`close`, `__iter__`, `__next__`
Sized[1]		`__len__`	
Callable[1]		`__call__`	
Collection[1]	*Sized*, *Iterable*, *Containe*	`__contains__`, `__iter__`, `__len__`	
Sequence	*Reversib* *Collecti*	`__getitem__`, `__len__`	`__contains__`, `__iter__`, `__reversed__`, `index`, and `count`
MutableSequence	*Sequence*	`__getitem__`, `__setitem__`, `__delitem__`, `__len__`, `insert`	Herdados métodos de *Sequence* e `append`, `clear`, `reverse`, `extend`, `pop`, `remove` e `__iadd__`
ByteString	*Sequence*	`__getitem__`, `__len__`	Métodos herdados de *Sequence*
Set	*Collecti*	`__contains__`, `__iter__`, `__len__`	`__le__`, `__lt__`, `__eq__`, `__ne__`, `__gt__`, `__ge__`, `__and__`, `__or__`, `__sub__`, `__rsub__`, `__xor__`, `__rxor__` e `isdisjoint`
MutableSet	*Set*	`__contains__`, `__iter__`, `__len__`, `add`, `discard`	Herdado *Set* métodos e `clear`, `pop`, `remove`, `__ior__`, `__iand__`, `__ixor__`, e `__isub__`
Mapping	*Collecti*	`__getitem__`, `__iter__`, `__len__`	`__contains__`, `keys`, `items`, `values`, `get`, `__eq__`, e `__ne__`
MutableMapping	*Mapping*	`__getitem__`, `__setitem__`, `__delitem__`, `__iter__`, `__len__`	Herdado *Mapping* métodos e `pop`, `popitem`, `clear`, `update`, e `setdefault`
MappingView	*Sized*		`__init__`, `__len__` e `__repr__`
ItemsView	*MappingV* *Set*		`__contains__`, `__iter__`
KeysView	*MappingV* *Set*		`__contains__`, `__iter__`
ValuesView	*MappingV* *Collecti*		`__contains__`, `__iter__`
Awaitable[1]		`__await__`	
Coroutine[1]	*Awaitable*	`send`, `throw`	`close`
AsyncIterable[1]		`__aiter__`	
AsyncIterator[1]	*AsyncIte*	`__anext__`	`__aiter__`
AsyncGenerator[1]	*AsyncIte*	`asend`, `athrow`	`aclose`, `__aiter__`, `__anext__`
Buffer[1]		`__buffer__`	

[1] Essas ABCs substituem *__subclasshook__()* para dar suporte ao teste de uma interface verificando se os métodos necessários estão presentes e não foram definidos como *None*. Isso só funciona para interfaces simples. Interfaces mais complexas exigem registro ou subclassificação direta.

[2] A verificação `isinstance(obj, Iterable)` detecta classes que são registradas como *Iterable* ou que possuem um método *__iter__()*, mas que não detecta classes que iteram com o método `__getitem__()`. A única maneira confiável de determinar se um objeto é *iterável* é chamar `iter(obj)`.

8.5.2 Classes Base Abstrata de Coleções – Descrições Detalhadas

class collections.abc.**Container**

ABC para classes que fornecem o método `__contains__()`.

class collections.abc.**Hashable**

ABC para classes que fornecem o método `__hash__()`.

class collections.abc.**Sized**

ABC para classes que fornecem o método `__len__()`.

class collections.abc.**Callable**

ABC para classes que fornecem o método `__call__()`.

Veja *Anotações de objetos chamáveis* para detalhes sobre como usar `Callable` em anotações de tipos.

class collections.abc.**Iterable**

ABC para classes que fornecem o método *`__iter__()`*.

A verificação `isinstance(obj, Iterable)` detecta classes que são registradas como *`Iterable`* ou que possuem um método *`__iter__()`*, mas que não detecta classes que iteram com o método `__getitem__()`. A única maneira confiável de determinar se um objeto é *iterável* é chamar `iter(obj)`.

class collections.abc.**Collection**

ABC para classes de contêiner iterável de tamanho.

Adicionado na versão 3.6.

class collections.abc.**Iterator**

ABC para classes que fornecem os métodos *`__iter__()`* e métodos *`__next__()`*. Veja também a definição de *iterator*.

class collections.abc.**Reversible**

ABC para classes iteráveis que também fornecem o método `__reversed__()`.

Adicionado na versão 3.6.

class collections.abc.**Generator**

ABC para classes *geradores* que implementam o protocolo definido em **PEP 342** que estende os *iteradores* com os métodos `send()`, `throw()` e `close()`.

Consulte *Anotando geradores e corrotinas* para obter detalhes sobre como usar `Generator` em anotações de tipos.

Adicionado na versão 3.5.

class collections.abc.**Sequence**

class collections.abc.**MutableSequence**

class collections.abc.**ByteString**

ABCs para *sequências* somente de leitura e mutáveis.

Nota de implementação: Alguns métodos mixin, como *`__iter__()`*, `__reversed__()` e *`index()`*, fazem chamadas repetidas ao método subjacente `__getitem__()`. Consequentemente, se `__getitem__()` for implementado com velocidade de acesso constante, os métodos mixin terão desempenho linear; no entanto, se o método subjacente for linear (como seria com uma lista encadeada), os mixins terão desempenho quadrático e provavelmente precisarão ser substituídos.

index(*value*, *start=0*, *stop=None*)

Retorna o primeiro índice de *valor*.

Levanta *`ValueError`* se o valor não estiver presente.

Suporte aos argumentos *start* e *stop* é opcional, mas recomendado.

Alterado na versão 3.5: O método *`index()`* ganhou suporte para os argumentos *stop* e *start*.

Deprecated since version 3.12, will be removed in version 3.17: A ABC *ByteString* foi descontinuada.

Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use *Buffer* ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

`ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de *bytes* e *bytearray*. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de `ByteString` nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como *memoryview*, também nunca foram entendidos como subtipos de `ByteString` (seja em tempo de execução ou por verificadores de tipo estáticos).

Consulte **PEP** para mais detalhes.

class collections.abc.**Set**

class collections.abc.**MutableSet**

ABCs para *conjuntos* somente leitura e mutáveis.

class collections.abc.**Mapping**

class collections.abc.**MutableMapping**

ABCs para somente leitura e mutável *mappings*.

class collections.abc.**MappingView**

class collections.abc.**ItemsView**

class collections.abc.**KeysView**

class collections.abc.**ValuesView**

ABCs para mapeamento, itens, chaves e valores *views*.

class collections.abc.**Awaitable**

ABC para objetos *aguardáveis*, que podem ser usados em expressões de `await`. Implementações personalizadas devem fornecer o método `__await__()`.

Objetos e instâncias de *corrotina* da ABC *Coroutine* são todas instâncias dessa ABC.

> **Nota**
>
> No CPython, as corrotinas baseados em gerador (*geradores* decorados com *@types.coroutine*) são *aguardáveis*, embora não possuam o método `__await__()`. Usar `isinstance(gencoro, Awaitable)` para eles retornará `False`. Use *inspect.isawaitable()* para detectá-los.

Adicionado na versão 3.5.

class collections.abc.**Coroutine**

ABC para classes compatíveis com *corrotina*. Eles implementam os seguintes métodos, definidos em coroutine-objects: `send()`, `throw()`, e `close()`. Implementações personalizadas também devem implementar `__await__()`. Todas as instâncias *Coroutine* também são instâncias de *Awaitable*.

> **Nota**
>
> No CPython, as corrotinas baseados em gerador (*geradores* decorados com *@types.coroutine*) são *aguardáveis*, embora não possuam o método `__await__()`. Usar `isinstance(gencoro, Coroutine)` para eles retornará `False`. Use *inspect.isawaitable()* para detectá-los.

Consulte *Anotando geradores e corrotinas* para obter detalhes sobre como usar `Coroutine` em anotações de tipos. A variância e a ordem dos parâmetros de tipo correspondem às de *Generator*.

Adicionado na versão 3.5.

class collections.abc.**AsyncIterable**

ABC para classes que fornecem um método `__aiter__`. Veja também a definição de *iterável assíncrono*.

Adicionado na versão 3.5.

class collections.abc.**AsyncIterator**

ABC para classes que fornecem os métodos `__aiter__` e `__anext__`. Veja também a definição de *iterador assíncrono*.

Adicionado na versão 3.5.

class collections.abc.**AsyncGenerator**

ABC para classes de *gerador assíncrono* que implementam o protocolo definido em **PEP 525** e **PEP 492**.

Consulte *Anotando geradores e corrotinas* para obter detalhes sobre como usar `AsyncGenerator` em anotações de tipos.

Adicionado na versão 3.6.

class collections.abc.**Buffer**

ABC para classes que fornecem o método `__buffer__()`, implementando o protocolo buffer. Veja **PEP 688**.

Adicionado na versão 3.12.

8.5.3 Exemplos e receitas

ABCs nos permitem perguntar a classes ou instâncias se elas fornecem funcionalidades específicas, por exemplo:

```
size = None
if isinstance(myvar, collections.abc.Sized):
    size = len(myvar)
```

Vários ABCs também são também úteis como mixins que facilitam o desenvolvimento de classes que suportam APIs de contêiner. Por exemplo, para escrever uma classe que suporte toda a API *Set* , é necessário fornecer apenas os três métodos abstratos subjacentes: `__contains__()`, *__iter__()*, e `__len__()`. O ABC fornece os métodos restantes, como `__and__()` e *isdisjoint()*:

```
class ListBasedSet(collections.abc.Set):
    ''' Alternate set implementation favoring space over speed
        and not requiring the set elements to be hashable. '''
    def __init__(self, iterable):
        self.elements = lst = []
        for value in iterable:
            if value not in lst:
                lst.append(value)

    def __iter__(self):
        return iter(self.elements)

    def __contains__(self, value):
        return value in self.elements

    def __len__(self):
        return len(self.elements)

s1 = ListBasedSet('abcdef')
s2 = ListBasedSet('defghi')
overlap = s1 & s2            # O método __and__() é automaticamente suportado
```

Notas sobre o uso de *Set* e *MutableSet* como um mixin:

(1) Como algumas operações de conjunto criam novos conjuntos, os métodos de mixin padrão precisam de uma maneira de criar novas instâncias a partir de um *iterável*. Supõe-se que a classe construtor tenha uma assinatura no formato `ClassName(iterable)`. Essa suposição é fatorada em um `classmethod` interno chamado `_from_iterable()` que chama `cls(iterable)` para produzir um novo conjunto. Se o mixin `Set` estiver sendo usado em uma classe com uma assinatura de construtor diferente, você precisará substituir `_from_iterable()` por um método de classe ou um método regular que possa construir novas instâncias a partir de um argumento iterável.

(2) Para substituir as comparações (presumivelmente para velocidade, já que a semântica é fixa), redefina `__le__()` e `__ge__()`, então as outras operações seguirão o exemplo automaticamente.

(3) O mixin `Set` fornece um método `_hash()` para calcular um valor de hash para o conjunto; no entanto, `__hash__()` não é definido porque nem todos os conjuntos são *hasheáveis* ou imutáveis. Para adicionar hasheabilidade em conjuntos usando mixin, herde de ambos `Set()` e `Hashable()`, e então defina `__hash__ = Set._hash`.

Ver também

- OrderedSet receita para um exemplo baseado em `MutableSet`.
- Para mais informações sobre ABCs, consulte o módulo `abc` e **PEP 3119**.

8.6 `heapq` — Algoritmo de fila heap

Código-fonte: Lib/heapq.py

Este módulo fornece uma implementação do algoritmo de fila heap, também conhecido como algoritmo de fila de prioridade.

Min-heaps are binary trees for which every parent node has a value less than or equal to any of its children. We refer to this condition as the heap invariant.

For min-heaps, this implementation uses lists for which `heap[k] <= heap[2*k+1]` and `heap[k] <= heap[2*k+2]` for all *k* for which the compared elements exist. Elements are counted from zero. The interesting property of a min-heap is that its smallest element is always the root, `heap[0]`.

Max-heaps satisfy the reverse invariant: every parent node has a value *greater* than any of its children. These are implemented as lists for which `maxheap[2*k+1] <= maxheap[k]` and `maxheap[2*k+2] <= maxheap[k]` for all *k* for which the compared elements exist. The root, `maxheap[0]`, contains the *largest* element; `heap.sort(reverse=True)` maintains the max-heap invariant.

The `heapq` API differs from textbook heap algorithms in two aspects: (a) We use zero-based indexing. This makes the relationship between the index for a node and the indexes for its children slightly less obvious, but is more suitable since Python uses zero-based indexing. (b) Textbooks often focus on max-heaps, due to their suitability for in-place sorting. Our implementation favors min-heaps as they better correspond to Python `lists`.

These two aspects make it possible to view the heap as a regular Python list without surprises: `heap[0]` is the smallest item, and `heap.sort()` maintains the heap invariant!

Like `list.sort()`, this implementation uses only the < operator for comparisons, for both min-heaps and max-heaps.

In the API below, and in this documentation, the unqualified term *heap* generally refers to a min-heap. The API for max-heaps is named using a `_max` suffix.

To create a heap, use a list initialized as `[]`, or transform an existing list into a min-heap or max-heap using the `heapify()` or `heapify_max()` functions, respectively.

The following functions are provided for min-heaps:

heapq.**heappush**(*heap*, *item*)

Push the value *item* onto the *heap*, maintaining the min-heap invariant.

heapq.**heappop**(*heap*)

Pop and return the smallest item from the *heap*, maintaining the min-heap invariant. If the heap is empty, `IndexError` is raised. To access the smallest item without popping it, use `heap[0]`.

heapq.**heappushpop**(*heap*, *item*)

Coloca *item* no heap, depois retira e retorna o menor item do *heap*. A ação combinada é executada com mais eficiência do que `heappush()` seguida por uma chamada separada para `heappop()`.

heapq.**heapify**(*x*)

Transform list *x* into a min-heap, in-place, in linear time.

heapq.**heapreplace**(*heap*, *item*)

Abre e retorna o menor item da *heap* e também coloca o novo *item*. O tamanho do heap não muda. Se o heap estiver vazio, a exceção `IndexError` será levantada.

Esta operação de uma etapa é mais eficiente que `heappop()` seguida por `heappush()` e pode ser mais apropriada ao usar um heap de tamanho fixo. A combinação de retirar/colocar sempre retorna um elemento do heap e o substitui por *item*.

O valor retornado pode ser maior que o *item* adicionado. Se isso não for desejado, considere usar `heappushpop()`. Sua combinação de retirar/colocar retorna o menor dos dois valores, deixando o valor maior no heap.

For max-heaps, the following functions are provided:

heapq.**heapify_max**(*x*)

Transform list *x* into a max-heap, in-place, in linear time.

Adicionado na versão 3.14.

heapq.**heappush_max**(*heap*, *item*)

Push the value *item* onto the max-heap *heap*, maintaining the max-heap invariant.

Adicionado na versão 3.14.

heapq.**heappop_max**(*heap*)

Pop and return the largest item from the max-heap *heap*, maintaining the max-heap invariant. If the max-heap is empty, `IndexError` is raised. To access the largest item without popping it, use `maxheap[0]`.

Adicionado na versão 3.14.

heapq.**heappushpop_max**(*heap*, *item*)

Push *item* on the max-heap *heap*, then pop and return the largest item from *heap*. The combined action runs more efficiently than `heappush_max()` followed by a separate call to `heappop_max()`.

Adicionado na versão 3.14.

heapq.**heapreplace_max**(*heap*, *item*)

Pop and return the largest item from the max-heap *heap* and also push the new *item*. The max-heap size doesn't change. If the max-heap is empty, `IndexError` is raised.

The value returned may be smaller than the *item* added. Refer to the analogous function `heapreplace()` for detailed usage notes.

Adicionado na versão 3.14.

O módulo também oferece três funções de propósito geral baseadas em heaps.

heapq.**merge**(**iterables*, *key=None*, *reverse=False*)

Mescla diversas entradas classificadas em uma única saída classificada (por exemplo, mescla entradas com registro de data e hora de vários arquivos de log). Retorna um *iterador* sobre os valores classificados.

Semelhante a `sorted(itertools.chain(*iterables))` mas retorna um iterável, não puxa os dados para a memória todos de uma vez e presume que cada um dos fluxos de entrada já está classificado (do menor para o maior).

Possui dois argumentos opcionais que devem ser especificados como argumentos nomeados.

key especifica uma *função chave* de um argumento que é usado para extrair uma chave de comparação de cada elemento de entrada. O valor padrão é `None` (compare os elementos diretamente).

reverse é um valor booleano. Se definido como `True`, então os elementos de entrada serão mesclados como se cada comparação fosse invertida. Para obter um comportamento semelhante a `sorted(itertools.chain(*iterables), reverse=True)`, todos os iteráveis devem ser classificados do maior para o menor.

Alterado na versão 3.5: Adicionados os parâmetros opcionais *key* e *reverse*.

`heapq.`**`nlargest`**`(`*n*, *iterable*, *key=None*`)`

Retorna uma lista com os *n* maiores elementos do conjunto de dados definido por *iterable*. *key*, se fornecido, especifica uma função de um argumento que é usado para extrair uma chave de comparação de cada elemento em *iterable* (por exemplo, `key=str.lower`). Equivalente a: `sorted(iterable, key=key, reverse=True)[:n]`.

`heapq.`**`nsmallest`**`(`*n*, *iterable*, *key=None*`)`

Retorna uma lista com os *n* menores elementos do conjunto de dados definido por *iterable*. *key*, se fornecido, especifica uma função de um argumento que é usado para extrair uma chave de comparação de cada elemento em *iterable* (por exemplo, `key=str.lower`). Equivalente a: `sorted(iterable, key=key)[:n]`.

As duas últimas funções têm melhor desempenho para valores menores de *n*. Para valores maiores, é mais eficiente usar a função *`sorted()`*. Além disso, quando `n==1`, é mais eficiente usar as funções embutidas *`min()`* e *`max()`*. Se for necessário o uso repetido dessas funções, considere transformar o iterável em um heap real.

8.6.1 Exemplos básicos

Um heapsort pode ser implementado colocando todos os valores em um heap e, em seguida, retirando os menores valores, um de cada vez:

```
>>> def heapsort(iterable):
...     h = []
...     for value in iterable:
...         heappush(h, value)
...     return [heappop(h) for i in range(len(h))]
...
>>> heapsort([1, 3, 5, 7, 9, 2, 4, 6, 8, 0])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Isto é semelhante a `sorted(iterable)`, mas diferente de *`sorted()`*, esta implementação não é estável.

Os elementos de heap podem ser tuplas. Isto é útil para atribuir valores de comparação (como prioridades de tarefas) juntamente com o registro principal que está sendo rastreado:

```
>>> h = []
>>> heappush(h, (5, 'write code'))
>>> heappush(h, (7, 'release product'))
>>> heappush(h, (1, 'write spec'))
>>> heappush(h, (3, 'create tests'))
>>> heappop(h)
(1, 'write spec')
```

8.6.2 Notas de implementação da fila de prioridade

Uma fila de prioridade é de uso comum para um heap e apresenta vários desafios de implementação:

- Estabilidade de classificação: como fazer com que duas tarefas com prioridades iguais sejam retornadas na ordem em que foram adicionadas originalmente?
- A comparação de tuplas quebra para pares (prioridade, tarefa) se as prioridades forem iguais e as tarefas não tiverem uma ordem de comparação padrão.
- Se a prioridade de uma tarefa mudar, como movê-la para uma nova posição no heap?
- Ou se uma tarefa pendente precisar ser excluída, como encontrá-la e removê-la da fila?

Uma solução para os dois primeiros desafios é armazenar as entradas como uma lista de 3 elementos, incluindo a prioridade, uma contagem de entradas e a tarefa. A contagem de entradas serve de desempate para que duas tarefas com a mesma prioridade sejam retornadas na ordem em que foram adicionadas. E como não há duas contagens de entradas iguais, a comparação de tuplas nunca tentará comparar diretamente duas tarefas.

Outra solução para o problema de tarefas não comparáveis é criar uma classe wrapper que ignore o item da tarefa e compare apenas o campo de prioridade:

```
from dataclasses import dataclass, field
from typing import Any

@dataclass(order=True)
class PrioritizedItem:
    priority: int
    item: Any=field(compare=False)
```

Os desafios restantes giram em torno de encontrar uma tarefa pendente e fazer alterações em sua prioridade ou removê-la totalmente. Encontrar uma tarefa pode ser feito com um dicionário apontando para uma entrada na fila.

Remover a entrada ou alterar sua prioridade é mais difícil porque quebraria os invariantes da estrutura de heap. Assim, uma possível solução é marcar a entrada como removida e adicionar uma nova entrada com a prioridade revisada:

```
pq = []                         # lista de entradas organizadas em uma heap
entry_finder = {}               # mapeamentos de tarefas para entradas
REMOVED = '<removed-task>'      # espaço reservado para uma tarefa removida
counter = itertools.count()     # contagem de sequência única

def add_task(task, priority=0):
    'Adiciona uma nova tarefa ou atualiza a prioridade de uma tarefa existente'
    if task in entry_finder:
        remove_task(task)
    count = next(counter)
    entry = [priority, count, task]
    entry_finder[task] = entry
    heappush(pq, entry)

def remove_task(task):
    'Marca uma tarefa existente com REMOVED (removida). Levanta KeyError se não␣
↪encontrada.'
    entry = entry_finder.pop(task)
    entry[-1] = REMOVED

def pop_task():
    'Remove e retorna a tarefa de mais baixa prioridade. Levanta KeyError se vazio.
↪'
    while pq:
        priority, count, task = heappop(pq)
```

(continua na próxima página)

(continuação da página anterior)

```
        if task is not REMOVED:
            del entry_finder[task]
            return task
    raise KeyError('pop from an empty priority queue')
```

8.6.3 Teoria

Heaps são arrays para os quais `a[k] <= a[2*k+1]` e `a[k] <= a[2*k+2]` para todos *k*, contando elementos de 0. Para fins de comparação, os elementos inexistentes são considerados infinitos. A propriedade interessante de um heap é que `a[0]` é sempre seu menor elemento.

O estranho invariante acima pretende ser uma representação de memória eficiente para um torneio. Os números abaixo são *k*, não `a[k]`:

```
                                0

             1                                  2

      3              4                5               6

  7       8       9       10      11      12      13      14

15 16   17 18   19 20   21 22   23 24   25 26   27 28   29 30
```

Na árvore acima, cada célula *k* está no topo de `2*k+1` e `2*k+2`. Num torneio binário normal que vemos nos desportos, cada célula é a vencedora das duas células que está no topo, e podemos rastrear o vencedor na árvore para ver todos os adversários que teve. Contudo, em muitas aplicações informáticas de tais torneios, não precisamos de traçar a história de um vencedor. Para sermos mais eficientes em termos de memória, quando um vencedor é promovido, tentamos substituí-lo por algo de nível inferior, e a regra passa a ser que uma célula e as duas células que ela cobre contêm três itens diferentes, mas a célula de cima "ganha" sobre as duas células superiores.

Se esse invariante de heap estiver protegido o tempo todo, o índice 0 é claramente o vencedor geral. A maneira algorítmica mais simples de removê-lo e encontrar o "próximo" vencedor é mover algum perdedor (digamos a célula 30 no diagrama acima) para a posição 0 e, em seguida, filtrar esse novo 0 pela árvore, trocando valores, até que o invariante é restabelecido. Isto é claramente logarítmico do número total de itens na árvore. Ao iterar todos os itens, você obtém uma classificação $O(n \log n)$.

Um recurso interessante desse tipo é que você pode inserir novos itens com eficiência enquanto a classificação está em andamento, desde que os itens inseridos não sejam "melhores" que o último 0º elemento extraído. Isto é especialmente útil em contextos de simulação, onde a árvore contém todos os eventos recebidos e a condição "vitória" significa o menor tempo programado. Quando um evento agenda outros eventos para execução, eles são agendados para o futuro, para que possam entrar facilmente no heap. Portanto, um heap é uma boa estrutura para implementar escalonadores (foi isso que usei no meu sequenciador MIDI :-).

Várias estruturas para implementação de escalonadores foram extensivamente estudadas, e os heaps são bons para isso, pois são razoavelmente rápidos, a velocidade é quase constante e o pior caso não é muito diferente do caso médio. No entanto, existem outras representações que são globalmente mais eficientes, embora os piores casos possam ser terríveis.

Heaps também são muito úteis em classificações de discos grandes. Provavelmente todos vocês sabem que uma classificação grande implica a produção de "execuções" (que são sequências pré-classificadas, cujo tamanho geralmente está relacionado à quantidade de memória da CPU), seguidas de passagens de fusão para essas execuções, cuja fusão geralmente é muito inteligente. organizado[1]. É muito importante que a classificação inicial produza as execuções mais longas possíveis. Os torneios são uma boa maneira de conseguir isso. Se, usando toda a memória disponível

[1] Os algoritmos de balanceamento de disco atuais, hoje em dia, são mais incômodos do que inteligentes, e isso é consequência da capacidade de busca dos discos. Em dispositivos que não são capazes de buscar, como grandes drives de fita, a história era bem diferente, e era preciso ser muito inteligente para garantir (com muita antecedência) que cada movimento da fita seria o mais eficaz possível (isto é, participaria melhor na "progressão" da fusão). Algumas fitas podiam até ser lidas de trás para frente, o que também era usado para evitar o tempo de rebobinar. Acredite em mim, fitas realmente boas eram espetaculares de assistir! Desde sempre, ordenar sempre foi uma Grande Arte! :-)

para realizar um torneio, você substituir e filtrar itens que se encaixem na corrida atual, você produzirá corridas que têm o dobro do tamanho da memória para entradas aleatórias e muito melhores para entradas ordenadas de maneira imprecisa.

Além disso, se você gerar o 0º item no disco e obter uma entrada que pode não caber no torneio atual (porque o valor "ganha" sobre o último valor de saída), ele não poderá caber no heap, então o tamanho do heap diminui. A memória liberada poderia ser reutilizada de maneira inteligente e imediata para a construção progressiva de um segundo heap, que cresce exatamente na mesma proporção que o primeiro heap está derretendo. Quando o primeiro heap desaparece completamente, você troca os heaps e inicia uma nova execução. Inteligente e bastante eficaz!

Em uma palavra, heaps são estruturas de memória úteis para conhecer. Eu os uso em alguns aplicativos e acho bom manter um módulo "heap" por perto. :-)

8.7 `bisect` — Algoritmo de bisseção de vetor

Código-fonte: Lib/bisect.py

Este módulo fornece suporte para manter uma lista em ordem de classificação sem ter que classificar a lista após cada inserção. Para longas listas de itens com operações de comparação caras, isso pode ser uma melhoria em relação a pesquisas lineares ou recorrência frequente.

O módulo é chamado de *bisect* porque usa um algoritmo básico de bisseção para fazer seu trabalho. Ao contrário de outras ferramentas de bisseção que buscam um valor específico, as funções neste módulo são projetadas para localizar um ponto de inserção. Da mesma forma, as funções nunca chamam um método `__eq__()` para determinar se um valor foi encontrado. Em vez disso, as funções apenas chamam o método `__lt__()` e retornarão um ponto de inserção entre valores em um vetor.

Nota

As funções neste módulo não são seguras para thread. Se várias threads usarem funções de *bisect* simultaneamente na mesma sequência, isso poderá resultar em um comportamento indefinido. Da mesma forma, se a sequência fornecida for mutada por uma thread diferente enquanto uma função de *bisect* estiver operando nela, o resultado será indefinido. Por exemplo, usar *insort_left()* na mesma lista de várias threads pode fazer com que a lista fique desordenada.

As seguintes funções são fornecidas:

`bisect.`**`bisect_left`**(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

Localiza o ponto de inserção de *x* em *a* para manter a ordem de classificação. Os parâmetros *lo* e *hi* podem ser usados para especificar um subconjunto da lista que deve ser considerado; por padrão, toda a lista é usada. Se *x* já estiver presente em *a*, o ponto de inserção estará antes (à esquerda) de qualquer entrada existente. O valor de retorno é adequado para uso como o primeiro parâmetro para `list.insert()` supondo que *a* já esteja ordenado.

O ponto de inserção retornado *ip* particiona o vetor *a* em duas fatias de forma que `all(elem < x for elem in a[lo : ip])` seja verdadeiro para a fatia esquerda e `all(elem >= x for elem in a[ip : hi])` é verdadeiro para a fatia certa.

key especifica uma *função chave* de um argumento que é usado para extrair uma chave de comparação de cada elemento no vetor. Para oferecer suporte à pesquisa de registros complexos, a função chave não é aplicada ao valor *x*.

Se *key* for `None`, os elementos serão comparados diretamente e nenhuma função chave será chamada.

Alterado na versão 3.10: Adicionado o parâmetro *key*.

`bisect.`**`bisect_right`**(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

bisect.**bisect**(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

Semelhante a `bisect_left()`, mas retorna um ponto de inserção que vem depois (à direita de) qualquer entrada existente de *x* em *a*.

O ponto de inserção retornado *ip* particiona o vetor *a* em duas fatias de forma que `all(elem <= x for elem in a[lo : ip])` seja verdadeiro para a fatia esquerda e `all(elem > x for elem in a[ip : hi])` é verdadeiro para a fatia certa.

Alterado na versão 3.10: Adicionado o parâmetro *key*.

bisect.**insort_left**(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

Insere *x* em *a* na ordem de classificação.

Esta função primeiro executa `bisect_left()` para localizar um ponto de inserção. Em seguida, executa o método `insert()` em *a* para inserir *x* na posição apropriada para manter a ordem de classificação.

Para oferecer suporte à inserção de registros em uma tabela, a função *key* (se houver) é aplicada a *x* para a etapa de pesquisa, mas não para a etapa de inserção.

Tenha em mente que a busca *O*(log *n*) é dominada pelo etapa de inserção lenta O(n).

Alterado na versão 3.10: Adicionado o parâmetro *key*.

bisect.**insort_right**(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

bisect.**insort**(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

Semelhante a `insort_left()`, mas inserindo *x* em *a* após qualquer entrada existente de *x*.

Esta função primeiro executa `bisect_right()` para localizar um ponto de inserção. Em seguida, executa o método `insert()` em *a* para inserir *x* na posição apropriada para manter a ordem de classificação.

Para oferecer suporte à inserção de registros em uma tabela, a função *key* (se houver) é aplicada a *x* para a etapa de pesquisa, mas não para a etapa de inserção.

Tenha em mente que a busca *O*(log *n*) é dominada pelo etapa de inserção lenta O(n).

Alterado na versão 3.10: Adicionado o parâmetro *key*.

8.7.1 Observações sobre desempenho

Ao escrever um código sensível ao tempo usando *bisect()* e *insort()*, lembre-se do seguinte:

- A bisseção é eficaz para pesquisar intervalos de valores. Para localizar valores específicos, os dicionários são mais eficientes.
- As funções *insort()* são *O*(*n*) porque a etapa de busca logarítmica é dominada pela etapa de inserção de tempo linear.
- As funções de busca são stateless e descartam os resultados da função chave depois que são usadas. Consequentemente, se as funções de busca forem usadas em um laço, a função chave pode ser chamada repetidamente nos mesmos elementos do vetor. Se a função chave não for rápida, considere envolvê-la com `functools.cache()` para evitar cálculos duplicados. Como alternativa, considere buscar um vetor de chaves pré-calculadas para localizar o ponto de inserção (conforme mostrado na seção de exemplos abaixo).

Ver também

- Sorted Collections é um módulo de alto desempenho que usa bisseção para gerenciar coleções de dados classificadas.
- A receita de SortedCollection usa bisseção para construir uma classe de coleção completa com métodos de busca diretos e suporte para uma função chave. As chaves são pré-calculadas para economizar em chamadas desnecessárias para a função chave durante as buscas.

8.7.2 Buscando em listas ordenadas

As *funções de bisseção* acima são úteis para encontrar pontos de inserção, mas podem ser complicadas ou difíceis de usar para tarefas comuns de busca. As cinco funções a seguir mostram como transformá-las nas buscas padrão para listas ordenadas:

```
def index(a, x):
    'Encontra o valor mais à esquerda exatamente igual a x'
    i = bisect_left(a, x)
    if i != len(a) and a[i] == x:
        return i
    raise ValueError

def find_lt(a, x):
    'Encontra o valor mais à direita menor que x'
    i = bisect_left(a, x)
    if i:
        return a[i-1]
    raise ValueError

def find_le(a, x):
    'Encontra o valor mais à direita menor ou igual a x'
    i = bisect_right(a, x)
    if i:
        return a[i-1]
    raise ValueError

def find_gt(a, x):
    'Encontra o valor mais à esquerda maior que x'
    i = bisect_right(a, x)
    if i != len(a):
        return a[i]
    raise ValueError

def find_ge(a, x):
    'Encontra o item mais à esquerda maior ou igual a x'
    i = bisect_left(a, x)
    if i != len(a):
        return a[i]
    raise ValueError
```

8.7.3 Exemplos

A função *bisect()* pode ser útil para buscas em tabelas numéricas. Este exemplo usa *bisect()* para buscar uma nota em letra para uma pontuação de exame (digamos) com base em um conjunto de pontos de interrupção numéricos ordenados: 90 e acima é um "A", 80 a 89 é um "B" e por aí vai:

```
>>> def grade(score, breakpoints=[60, 70, 80, 90], grades='FDCBA'):
...     i = bisect(breakpoints, score)
...     return grades[i]
...
>>> [grade(score) for score in [33, 99, 77, 70, 89, 90, 100]]
['F', 'A', 'C', 'C', 'B', 'A', 'A']
```

As funções *bisect()* e *insort()* também funcionam com listas de tuplas. O argumento *key* pode servir para extrair o campo usado para ordenar registros em uma tabela:

```
>>> from collections import namedtuple
>>> from operator import attrgetter
>>> from bisect import bisect, insort
>>> from pprint import pprint

>>> Movie = namedtuple('Movie', ('name', 'released', 'director'))

>>> movies = [
...     Movie('Jaws', 1975, 'Spielberg'),
...     Movie('Titanic', 1997, 'Cameron'),
...     Movie('The Birds', 1963, 'Hitchcock'),
...     Movie('Aliens', 1986, 'Cameron')
... ]

>>> # Encontra o primeiro filme lançado após 1960
>>> by_year = attrgetter('released')
>>> movies.sort(key=by_year)
>>> movies[bisect(movies, 1960, key=by_year)]
Movie(name='The Birds', released=1963, director='Hitchcock')

>>> # Insere um filme enquanto mantém a ordem de classificação
>>> romance = Movie('Love Story', 1970, 'Hiller')
>>> insort(movies, romance, key=by_year)
>>> pprint(movies)
[Movie(name='The Birds', released=1963, director='Hitchcock'),
 Movie(name='Love Story', released=1970, director='Hiller'),
 Movie(name='Jaws', released=1975, director='Spielberg'),
 Movie(name='Aliens', released=1986, director='Cameron'),
 Movie(name='Titanic', released=1997, director='Cameron')]
```

Se a função chave for custosa, é possível evitar chamadas de função repetidas buscando uma lista de chaves pré-calculadas para encontrar o índice de um registro:

```
>>> data = [('red', 5), ('blue', 1), ('yellow', 8), ('black', 0)]
>>> data.sort(key=lambda r: r[1])       # Ou use operator.itemgetter(1).
>>> keys = [r[1] for r in data]         # Pré-calcula uma lista de chaves.
>>> data[bisect_left(keys, 0)]
('black', 0)
>>> data[bisect_left(keys, 1)]
('blue', 1)
>>> data[bisect_left(keys, 5)]
('red', 5)
>>> data[bisect_left(keys, 8)]
('yellow', 8)
```

8.8 `array`— Vetores eficientes de valores numéricos

Esse módulo define um tipo de objeto que pode representar compactamente um vetor de valores básicos: caracteres, inteiros, números de ponto flutuante. Vetores são tipos de sequência e funcionam bem parecidamente com listas, porém o tipo dos objetos armazenados é restringido. O tipo é especificado na criação do objeto usando um *código de tipo*, que é um único caractere. São definidos os seguintes códigos de tipo:

Código de tipo	Tipo em C	Tipo em Python	Tamanho mínimo em bytes	Notas
`'b'`	signed char	int	1	
`'B'`	unsigned char	int	1	
`'u'`	wchar_t	Caractere unicode	2	(1)
`'w'`	Py_UCS4	Caractere unicode	4	(2)
`'h'`	signed short	int	2	
`'H'`	unsigned short	int	2	
`'i'`	signed int	int	2	
`'I'`	unsigned int	int	2	
`'l'`	signed long	int	4	
`'L'`	unsigned long	int	4	
`'q'`	signed long long	int	8	
`'Q'`	unsigned long long	int	8	
`'f'`	ponto flutuante	ponto flutuante	4	
`'d'`	double	ponto flutuante	8	

Notas:

(1) Pode ser de 16 ou 32 bits dependendo da plataforma.

Alterado na versão 3.9: `array('u')` agora usa `wchar_t` como tipo C no lugar do descontinuado `Py_UNICODE`. Essa mudança não afeta o comportamento pois `Py_UNICODE` é um apelido para `wchar_t` desde Python 3.3.

Deprecated since version 3.3, will be removed in version 3.16: Por favor, migre para o código de tipo `'w'`.

(2) Adicionado na versão 3.13.

A representação dos valores é definida pela arquitetura da máquina, mais especificamente da implementação do C. O tamanho real pode ser acessado pelo atributo *`array.itemsize`*.

O módulo define o seguinte item:

`array.`**`typecodes`**

String com todos os códigos de tipo disponíveis.

O módulo define o seguinte tipo:

`class` `array.`**`array`**(*typecode*[, *initializer*])

Um novo vetor cujos itens são restritos por *typecode* e inicializados a partir do valor opcional *initializer*, que deve ser um objeto *`bytes`* ou *`bytearray`*, uma string Unicode ou iterável sobre elementos do tipo apropriado.

Se for fornecido um objeto *`bytes`* ou *`bytearray`*, o inicializador é passado para o método *`frombytes()`* do novo vetor; se for fornecida uma string Unicode, o inicializador é passado para o método *`fromunicode()`*; caso contrário, o iterador do inicializador é passado para o método *`extend()`* para adicionar itens iniciais ao vetor.

Objetos vetor tem suporte para as operações de sequência comuns: indexação, fatiamento, concatenação e multiplicação. Quando usando a atribuição de fatias, o valor associado deve ser um objeto vetor com o mesmo código de tipo; caso contrário, *`TypeError`* é levantada. Objetos vetor também implementam a interface buffer, e também podem ser usados em qualquer lugar onde *objetos bytes ou similares* é permitido.

Levanta um *evento de auditoria* `array.__new__` com os argumentos `typecode`, `initializer`.

`typecode`

O caractere de código de tipo usado para criar o vetor.

`itemsize`

O tamanho em bytes de um item do vetor em representação interna.

`append`(*x*)

Adiciona um novo item com valor *x* ao final do vetor.

buffer_info()

Retorna uma tupla `(address, length)` com o endereço corrente da memória e o tamanho em elementos do buffer usado para armazenar conteúdos do vetor. O tamanho do buffer da memória em bytes pode ser computado como `array.buffer_info()[1] * array.itemsize`. Isso é ocasionalmente útil quando se está trabalhando com interfaces I/O de baixo nível (inerentemente inseguras) que precisam de endereços de memória, como algumas operações `ioctl()`. Os números retornados são válidos enquanto o vetor existir e nenhuma operação de alteração de tamanho for aplicada a ele.

> **Nota**
>
> Quando se está usando vetores de código escrito em C ou C++ (o único jeito efetivo de usar essa informação), faz mais sentido usar a interface do buffer suportada pelos vetores. Esse método é mantido para retrocompatibilidade e deve ser evitado em código novo. A interface de buffers está documentada em bufferobjects.

byteswap()

"Byteswap" todos os itens do vetor. Isso é somente suportado para valores de 1, 2, 4 ou 8 bytes de tamanho; para outros tipos de valores é levantada *RuntimeError*. Isso é útil quando estamos lendo dados de um arquivo para serem escritos em um arquivo de outra máquina de ordem de bytes diferente.

count(*x*)

Retorna a quantidade de ocorrências de *x* no vetor.

extend(*iterable*)

Acrescenta os itens de *iterable* ao final do vetor. Se *iterable* for outro vetor, ele deve ter *exatamente* o mesmo código de tipo; senão, ocorrerá uma *TypeError*. Se *iterable* não for um vetor, ele deve ser iterável e seus elementos devem ser do tipo correto para ser acrescentado ao vetor.

frombytes(*buffer*)

Adiciona itens do *objeto bytes ou similar*, interpretando seu conteúdo como um vetor de valores de máquina (como se tivesse sido lido de um arquivo usando o método *fromfile()*).

Adicionado na versão 3.2: `fromstring()` foi renomeado para *frombytes()* para maior clareza.

fromfile(*f*, *n*)

Lê *n* itens (como valores de máquinas) do *objeto arquivo* *f* e adiciona-os ao fim do vetor. Se estão disponíveis menos de *n* itens, *EOFError* é levantada, mas os itens disponíveis ainda são inseridos ao final do vetor.

fromlist(*list*)

Adiciona itens de *list*. Isso é equivalente a `for x in list: a.append(x)` exceto que se ocorrer um erro de tipo, o vetor não é alterado.

fromunicode(*s*)

Estende este vetor com dados da string Unicode fornecida. O vetor deve ter o código de tipo `'u'` ou `'w'`; caso contrário, uma *ValueError* é levantada. Use `array.frombytes(unicodestring.encode(enc))` para anexar dados Unicode a um vetor de algum outro tipo.

index(*x*[, *start*[, *stop*]])

Retorna o menor *i* tal que *i* seja o índice da primeira ocorrência de *x* no vetor. Os argumentos opcionais *start* e *stop* podem ser especificados para procurar por *x* dentro de uma subseção do vetor. Levanta *ValueError* se *x* não for encontrado.

Alterado na versão 3.10: Adicionados os parâmetros opcionais *start e *stop*.

insert(*i*, *x*)

Insere um novo item com o valor *x* no vetor antes da posição *i*. Valores negativos são tratados como sendo em relação ao fim do vetor.

pop([*i*])

Remove o item com o índice *i* do vetor e retorna este item. O valor padrão do argumento opcional é `-1`, assim por padrão o último item é removido e retornado.

remove(*x*)

Remove a primeira ocorrência de *x* do vetor.

clear()

Remove todos os elementos do vetor.

Adicionado na versão 3.13.

reverse()

Inverte a ordem dos itens no vetor.

tobytes()

Devolve os itens do vetor como um vetor de valores de máquina com a representação em bytes (a mesma sequência de bytes que seria escrita pelo método *tofile()*.)

Adicionado na versão 3.2: `tostring()` foi nomeado para *tobytes()* para maior clareza.

tofile(*f*)

Escreve todos os itens (como valores de máquinas) para o *objeto arquivo* *f*.

tolist()

Devolve os itens do vetor como uma lista comum.

tounicode()

Devolve os itens do vetor como uma string Unicode. O vetor deve ser do tipo `'u'` ou `'w'`; caso contrário *ValueError* é levantada. Use `array.tobytes().decode(enc)` para obter uma string Unicode de um vetor de outros tipos.

A representação em string de objetos vetor tem o formato `array(typecode, initializer)`. O *initializer* é omitido se o vetor estiver vazio; caso contrário, é uma string Unicode se o *typecode* for `'u'` ou `'w'`; caso contrário, é uma lista de números. A representação em string pode ser convertida novamente para um array com o mesmo tipo e valor usando *eval()*, desde que a classe *array* tenha sido importada usando `from array import array`. As variáveis `inf` e `nan` também devem ser definidas se contiverem valores de ponto flutuante correspondentes. Exemplos:

```
array('l')
array('w', 'hello \u2641')
array('l', [1, 2, 3, 4, 5])
array('d', [1.0, 2.0, 3.14, -inf, nan])
```

Ver também

Módulo *struct*
Empacotamento e desempacotamento de dados binários heterogêneos.

NumPy
O pacote NumPy define outro tipo de vetor.

8.9 `weakref` — Referências fracas

Código-fonte: Lib/weakref.py

O módulo *weakref* permite ao programador Python criar *referências fracas* para objetos.

A seguir, o termo *referente* significa o objeto ao qual é referido por uma referência fraca.

Uma referência fraca a um objeto não é suficiente para mantê-lo vivo: quando as únicas referências restantes a um referente são referências fracas, a *coleta de lixo* está livre para destruir o referente e reutilizar sua memória para outra coisa. Entretanto, até que o objeto seja realmente destruído, a referência fraca poderá retornar o objeto mesmo que não haja referências fortes a ele.

Um uso principal para referências fracas é implementar caches ou mapeamentos contendo objetos grandes, onde é desejado que um objeto grande não seja mantido ativo apenas porque aparece em um cache ou mapeamento.

Por exemplo, se você tiver vários objetos de imagem binária grandes, poderá associar um nome a cada um. Se você usasse um dicionário Python para mapear nomes para imagens, ou imagens para nomes, os objetos de imagem permaneceriam vivos apenas porque apareceriam como valores ou chaves nos dicionários. As classes `WeakKeyDictionary` e `WeakValueDictionary` fornecidas pelo módulo `weakref` são uma alternativa, usando referências fracas para construir mapeamentos que não mantêm objetos vivos apenas porque aparecem nos objetos de mapeamento. Se, por exemplo, um objeto de imagem for um valor em um `WeakValueDictionary`, então quando as últimas referências restantes a esse objeto de imagem forem as referências fracas mantidas por mapeamentos fracos, a coleta de lixo poderá recuperar o objeto e suas entradas correspondentes em mapeamentos fracos são simplesmente excluídos.

`WeakKeyDictionary` e `WeakValueDictionary` usam referências fracas em sua implementação, configurando funções de retorno de chamada nas referências fracas que notificam os dicionários fracos quando uma chave ou valor foi recuperado pela coleta de lixo. `WeakSet` implementa a interface `set`, mas mantém referências fracas aos seus elementos, assim como `WeakKeyDictionary` faz.

`finalize` fornece uma maneira direta de registrar uma função de limpeza a ser chamada quando um objeto é coletado como lixo. Isso é mais simples de usar do que configurar uma função de retorno de chamada em uma referência fraca não tratada, pois o módulo garante automaticamente que o finalizador permaneça ativo até que o objeto seja coletado.

A maioria dos programas deve descobrir que usar um desses tipos de contêineres fracos ou `finalize` é tudo que eles precisam – geralmente não é necessário criar suas próprias referências fracas diretamente. O maquinário de baixo nível é exposto pelo módulo `weakref` para benefício de usos avançados.

Nem todos os objetos podem ser referenciados de maneira fraca. Objetos que oferecem suporte a referências fracas incluem instâncias de classe, funções escritas em Python (mas não em C), métodos de instância, conjuntos, frozensets, alguns *objetos arquivos*, *geradores*, objetos tipo, sockets, arrays, deques, objetos padrão de expressão regular e objetos código.

Alterado na versão 3.2: Adicionado suporte para thread.lock, threading.Lock e objetos código.

Vários tipos embutidos como `list` e `dict` não oferecem suporta diretamente a referências fracas, mas podem adicionar suporte através de subclasses:

```
class Dict(dict):
    pass

obj = Dict(red=1, green=2, blue=3)   # este objeto é potencialmente fracamente␣
↪referenciável
```

Outros tipos embutidos como `tuple` e `int` não oferecem suporte a referências fracas mesmo em subclasses.

Os tipos de extensão podem ser facilmente criados para oferecer suporte a referências fracas; veja weakref-support.

Quando `__slots__` são definidos para um determinado tipo, o suporte a referência fraca é desativado a menos que uma string `'__weakref__'` também esteja presente na sequência de strings na declaração de `__slots__`. Veja a documentação de __slots__ para detalhes.

class weakref.**ref**(*object*[, *callback*])

Retorna uma referência fraca para *object*. O objeto original pode ser recuperado chamando o objeto referência se o referente ainda estiver ativo; se o referente não estiver mais ativo, chamar o objeto referência fará com que `None` seja retornado. Se *callback* for fornecido e não for `None`, e o objeto referência fraca retornado ainda estiver ativo, o função de retorno será chamada quando o objeto estiver prestes a ser finalizado; o objeto referência fraca será passado como único parâmetro para a função de retorno; o referente não estará mais disponível.

É permitido que muitas referências fracas sejam construídas para o mesmo objeto. As funções de retorno registradas para cada referência fraca serão chamadas da função de retorno registrada mais recentemente para a função de retorno registrada mais antiga.

As exceções levantadas pela função de retorno serão anotadas na saída de erro padrão, mas não poderão ser propagadas; elas são tratadas exatamente da mesma maneira que as exceções levantadas pelo método `__del__()` de um objeto.

Referências fracas são *hasheáveis* se o *object* for hasheável. Elas manterão seu valor de hash mesmo depois que *object* for excluído. Se `hash()` for chamada pela primeira vez somente após o *object* ter sido excluído, a chamada vai levantar `TypeError`.

Referências fracas oferecem suporte a testes de igualdade, mas não de ordenação. Se os referentes ainda estiverem vivos, duas referências terão a mesma relação de igualdade que seus referentes (independentemente do *callback*). Se um dos referentes tiver sido excluído, as referências serão iguais somente se os objetos referência forem o mesmo objeto.

Este é um tipo do qual pode ser feita subclasse em vez de uma função de fábrica.

__callback__

Este atributo somente leitura retorna a função de retorno atualmente associada à referência fraca. Se não houver função de retorno ou se o referente da referência fraca não estiver mais ativo, então este atributo terá o valor `None`.

Alterado na versão 3.4: Adicionado o atributo `__callback__`.

`weakref.`**`proxy`**(*object*[, *callback*])

Retorna um intermediário (proxy, em inglês) para *object* que usa uma referência fraca. Isto provê suporte ao uso do intermediário na maioria dos contextos, em vez de exigir a desreferenciação explícita usada com objetos de referência fraca. O objeto retornado terá um tipo `ProxyType` ou `CallableProxyType`, dependendo se *object* é chamável. Objetos intermediários não são *hasheáveis* independentemente do referente; isso evita uma série de problemas relacionados à sua natureza fundamentalmente mutável e impede seu uso como chaves de dicionário. *callback* é igual ao parâmetro de mesmo nome da função `ref()`.

Acessar um atributo do objeto intermediário depois que o referente é coletado como lixo levanta `ReferenceError`.

Alterado na versão 3.8: Estendeu o suporte ao operador em objetos intermediários para incluir os operadores `@` e `@=` para multiplicação de matrizes.

`weakref.`**`getweakrefcount`**(*object*)

Retorna o número de referências fracas e intermediários que fazem referência a *object*.

`weakref.`**`getweakrefs`**(*object*)

Retorna uma lista de todos os objetos intermediários e de referência fraca que fazem referência a *object*.

`class` `weakref.`**`WeakKeyDictionary`**([*dict*])

Classe de mapeamento que faz referência fraca a chaves. Entradas no dicionário serão descartadas quando não houver mais uma referência forte à chave. Isso pode ser usado para associar dados adicionais a um objeto de propriedade de outras partes de uma aplicação sem adicionar atributos a esses objetos. Isso pode ser especialmente útil com objetos que substituem acessos de atributos.

Note que quando uma chave com valor igual a uma chave existente (mas não identidade igual) é inserida no dicionário, ela substitui o valor, mas não substitui a chave existente. Devido a isso, quando a referência à chave original é excluída, ela também exclui a entrada no dicionário:

```
>>> class T(str): pass
...
>>> k1, k2 = T(), T()
>>> d = weakref.WeakKeyDictionary()
>>> d[k1] = 1   # d = {k1: 1}
>>> d[k2] = 2   # d = {k1: 2}
>>> del k1      # d = {}
```

Uma solução alternativa seria remover a chave antes da reatribuição:

```
>>> class T(str): pass
...
>>> k1, k2 = T(), T()
>>> d = weakref.WeakKeyDictionary()
>>> d[k1] = 1   # d = {k1: 1}
>>> del d[k1]
>>> d[k2] = 2   # d = {k2: 2}
>>> del k1      # d = {k2: 2}
```

Alterado na versão 3.9: Adicionado suporte para operadores | e |=, conforme especificado na **PEP 584**.

Objetos *WeakKeyDictionary* têm um método adicional que expõe as referências internas diretamente. Não há garantia de que as referências estejam "ativas" no momento em que são usadas, então o resultado da chamada das referências precisa ser verificado antes de ser usado. Isso pode ser usado para evitar a criação de referências que farão com que o coletor de lixo mantenha as chaves por mais tempo do que o necessário.

WeakKeyDictionary.**keyrefs**()

Retorna um iterável das referências fracas às chaves.

class weakref.**WeakValueDictionary**([*dict*])

Classe de mapeamento que faz referência fraca a valores. Entradas no dicionário serão descartadas quando nenhuma referência forte ao valor existir mais.

Alterado na versão 3.9: Adicionado suporte para operadores | e |=, conforme especificado na **PEP 584**.

Os objetos *WeakValueDictionary* têm um método adicional que apresenta os mesmos problemas que o método *WeakKeyDictionary.keyrefs()*.

WeakValueDictionary.**valuerefs**()

Retorna um iterável das referências fracas aos valores.

class weakref.**WeakSet**([*elements*])

Define a classe que mantém referências fracas para seus elementos. Um elemento será descartado quando nenhuma referência forte a ele existir mais.

class weakref.**WeakMethod**(*method*[, *callback*])

Uma subclasse personalizada de *ref* que simula uma referência fraca a um método vinculado (ou seja, um método definido em uma classe e pesquisado em uma instância). Como um método vinculado é efêmero, uma referência fraca padrão não pode mantê-lo. *WeakMethod* tem um código especial para recriar o método vinculado até que o objeto ou a função original morra:

```
>>> class C:
...     def method(self):
...         print("method called!")
...
>>> c = C()
>>> r = weakref.ref(c.method)
>>> r()
>>> r = weakref.WeakMethod(c.method)
>>> r()
<bound method C.method of <__main__.C object at 0x7fc859830220>>
>>> r()()
method called!
>>> del c
>>> gc.collect()
0
>>> r()
>>>
```

callback é o mesmo que o parâmetro de mesmo nome da função `ref()`.

Adicionado na versão 3.4.

class weakref.**finalize**(*obj*, *func*, */*, **args*, ***kwargs*)

Retorna um objeto finalizador chamável que será chamado quando *obj* for coletado como lixo. Diferentemente de uma referência fraca comum, um finalizador sempre sobreviverá até que o objeto referência seja coletado, simplificando muito o gerenciamento do ciclo de vida.

Um finalizador é considerado *alive* (vivo) até ser chamado (explicitamente ou na coleta de lixo), e depois disso ele é *dead* (morto). Chamar um finalizador vivo retorna o resultado da avaliação de `func(*arg, **kwargs)`, enquanto chamar um finalizador morto retorna `None`.

Exceções levantadas por funções de retorno do finalizador durante a coleta de lixo serão mostradas na saída de erro padrão, mas não podem ser propagadas. Elas são tratadas da mesma forma que exceções levantadas de um método `__del__()` de um objeto ou de uma função de retorno da referência fraca.

Quando o programa é encerrado, cada finalizador vivo restante é chamado, a menos que seu atributo `atexit` tenha sido definido como falso. Eles são chamados na ordem reversa da criação.

Um finalizador nunca vai invocar sua função de retorno durante a parte posterior do *desligamento do interpretador* quando os globais do módulo podem ter sido substituídos por `None`.

__call__()

Se *self* estiver vivo, marca-o como morto e retorna o resultado da chamada `func(*args, **kwargs)`. Se *self* estiver morto, retorna `None`.

detach()

Se *self* estiver vivo, marca-o como morto e retorna a tupla `(obj, func, args, kwargs)`. Se *self* estiver morto, retorna `None`.

peek()

Se *self* estiver vivo, retorna a tupla `(obj, func, args, kwargs)`. Se *self* estiver morto, retorna `None`.

alive

Propriedade que é verdadeiro se o finalizador estiver ativo, falsa caso contrário.

atexit

Uma propriedade booleana gravável que por padrão é verdadeiro. Quando o programa sai, ele chama todos os finalizadores vivos restantes para os quais `atexit` é verdadeiro. Eles são chamados na ordem reversa da criação.

Nota

É importante garantir que *func*, *args* e *kwargs* não possuam nenhuma referência a *obj*, direta ou indiretamente, pois, caso contrário, *obj* nunca será coletado como lixo. Em particular, *func* não deve ser um método vinculado de *obj*.

Adicionado na versão 3.4.

weakref.**ReferenceType**

O objeto tipo para objetos referências fracas.

weakref.**ProxyType**

O tipo de objeto para intermediários de objetos que não são chamáveis.

weakref.**CallableProxyType**

O objeto de tipo para intermediários de objetos chamáveis.

weakref.**ProxyTypes**

Sequência contendo todos os objetos de tipo para intermediários. Isso pode tornar mais simples testar se um objeto é um intermediário sem depender da nomeação de ambos os tipos de intermediário.

Ver também

PEP 205 - Referências fracas
A proposta e a justificativa para esse recurso, incluindo links para implementações anteriores e informações sobre recursos semelhantes em outras linguagens.

8.9.1 Objetos referência fraca

Objetos referência fraca não têm métodos nem atributos além de *ref.__callback__*. Um objeto referência fraca permite que o referente seja obtido, se ele ainda existir, chamando-o:

```
>>> import weakref
>>> class Object:
...     pass
...
>>> o = Object()
>>> r = weakref.ref(o)
>>> o2 = r()
>>> o is o2
True
```

Se o referente não existir mais, chamar o objeto referência retornará *None*:

```
>>> del o, o2
>>> print(r())
None
```

O teste de se um objeto referência fraca ainda está vivo deve ser feito usando a expressão `ref() is not None`. Normalmente, o código da aplicação que precisa usar um objeto de referência deve seguir este padrão:

```
# r é um objeto referência fraca
o = r()
if o is None:
    # referente foi coletado como lixo
    print("O objeto foi desalocado; não é possível mexer com ele.")
else:
    print("O objeto ainda está vivo!")
    o.faz_algo_útil()
```

Usar um teste separado para "vivacidade" cria condições de corrida em aplicações que usam mais de uma thread; uma outra thread pode fazer com que uma referência fraca seja invalidada antes que a referência fraca seja chamada; o idioma mostrado acima é seguro em aplicações que usam mais de uma thread, bem como em aplicações de thread única.

Versões especializadas de objetos *ref* podem ser criadas por meio de subclasse. Isso é usado na implementação do *WeakValueDictionary* para reduzir a sobrecarga de memória para cada entrada no mapeamento. Isso pode ser mais útil para associar informações adicionais a uma referência, mas também pode ser usado para inserir processamento adicional em chamadas para recuperar o referente.

Este exemplo mostra como uma subclasse de *ref* pode ser usada para armazenar informações adicionais sobre um objeto e afetar o valor retornado quando o referente é acessado:

```
import weakref
```

(continua na próxima página)

(continuação da página anterior)

```
class ExtendedRef(weakref.ref):
    def __init__(self, ob, callback=None, /, **annotations):
        super().__init__(ob, callback)
        self.__counter = 0
        for k, v in annotations.items():
            setattr(self, k, v)

    def __call__(self):
        """Retorna um par contendo o referente e o número de
        vezes que a referência foi chamada.
        """
        ob = super().__call__()
        if ob is not None:
            self.__counter += 1
            ob = (ob, self.__counter)
        return ob
```

8.9.2 Exemplo

Este exemplo simples mostra como uma aplicação pode usar IDs de objeto para recuperar objetos que ele viu antes. Os IDs dos objetos podem então ser usados em outras estruturas de dados sem forçar os objetos a permanecerem vivos, mas os objetos ainda podem ser recuperados por ID se o fizerem.

```
import weakref

_id2obj_dict = weakref.WeakValueDictionary()

def remember(obj):
    oid = id(obj)
    _id2obj_dict[oid] = obj
    return oid

def id2obj(oid):
    return _id2obj_dict[oid]
```

8.9.3 Objetos finalizadores

O principal benefício de usar *finalize* é que ele simplifica o registro de um retorno de chamada sem precisar preservar o objeto finalizador retornado. Por exemplo

```
>>> import weakref
>>> class Object:
...     pass
...
>>> kenny = Object()
>>> weakref.finalize(kenny, print, "You killed Kenny!")
<finalize object at ...; for 'Object' at ...>
>>> del kenny
You killed Kenny!
```

O finalizador pode ser chamado diretamente também. No entanto, o finalizador vai invocar a função de retorno no máximo uma vez.

```
>>> def callback(x, y, z):
...     print("CALLBACK")
...     return x + y + z
```

(continua na próxima página)

(continuação da página anterior)

```
...
>>> obj = Object()
>>> f = weakref.finalize(obj, callback, 1, 2, z=3)
>>> assert f.alive
>>> assert f() == 6
CALLBACK
>>> assert not f.alive
>>> f()                     # callback not called because finalizer dead
>>> del obj                 # callback not called because finalizer dead
```

Você pode desfazer o registro de um finalizador usando seu método *detach()*. Isso elimina o finalizador e retorna os argumentos passados ao construtor quando ele foi criado.

```
>>> obj = Object()
>>> f = weakref.finalize(obj, callback, 1, 2, z=3)
>>> f.detach()
(<...Object object ...>, <function callback ...>, (1, 2), {'z': 3})
>>> newobj, func, args, kwargs = _
>>> assert not f.alive
>>> assert newobj is obj
>>> assert func(*args, **kwargs) == 6
CALLBACK
```

A menos que você defina o atributo *atexit* como *False*, um finalizador será chamado quando o programa sair se ele ainda estiver vivo. Por exemplo

```
>>> obj = Object()
>>> weakref.finalize(obj, print, "obj dead or exiting")
<finalize object at ...; for 'Object' at ...>
>>> exit()
obj dead or exiting
```

8.9.4 Comparando finalizadores com métodos `__del__()`

Suponha que queremos criar uma classe cujas instâncias representam diretórios temporários. Os diretórios devem ser excluídos com seus conteúdos quando o primeiro dos seguintes eventos ocorrer:

- o objeto é um lixo coletado,
- o método `remove()` do objeto é chamado, ou
- o programa finaliza.

Podemos tentar implementar a classe usando um método `__del__()` da seguinte maneira:

```
class TempDir:
    def __init__(self):
        self.name = tempfile.mkdtemp()

    def remove(self):
        if self.name is not None:
            shutil.rmtree(self.name)
            self.name = None

    @property
    def removed(self):
        return self.name is None
```

(continua na próxima página)

(continuação da página anterior)

```
    def __del__(self):
        self.remove()
```

A partir do Python 3.4, os métodos `__del__()` não impedem mais que os ciclos de referência sejam coletados como lixo, e os módulos globais não são mais forçados a `None` durante *desligamento do interpretador*. Então, esse código deve funcionar sem problemas no CPython.

Entretanto, o tratamento dos métodos `__del__()` é notoriamente específico da implementação, pois depende de detalhes internos da implementação do coletor de lixo do interpretador.

Uma alternativa mais robusta pode ser definir um finalizador que faça referência apenas às funções e objetos específicos de que necessita, em vez de ter acesso ao estado completo do objeto:

```
class TempDir:
    def __init__(self):
        self.name = tempfile.mkdtemp()
        self._finalizer = weakref.finalize(self, shutil.rmtree, self.name)

    def remove(self):
        self._finalizer()

    @property
    def removed(self):
        return not self._finalizer.alive
```

Definido assim, nosso finalizador recebe apenas uma referência aos detalhes que ele precisa para limpar o diretório apropriadamente. Se o objeto nunca for coletado como lixo, o finalizador ainda será chamado na saída.

A outra vantagem dos finalizadores baseados em referências fracas é que eles podem ser usados para registrar finalizadores para classes onde a definição é controlada por terceiros, como executar código quando um módulo é descarregado:

```
import weakref, sys
def unloading_module():
    # referência implícita aos globais do módulo do corpo da função
weakref.finalize(sys.modules[__name__], unloading_module)
```

Nota

Se você criar um objeto finalizador em uma thread em um daemon assim que o programa sair, então há a possibilidade de que o finalizador não seja chamado na saída. No entanto, em um thread em um daemon *`atexit.register()`*, `try: ... finally: ...` e `with: ...` não garantem que a limpeza ocorra também.

8.10 `types` — Criação de tipos dinâmicos e nomes para tipos embutidos

Código-fonte: Lib/types.py

Este módulo define funções utilitárias para auxiliar na criação dinâmica de novos tipos.

Também define nomes para alguns tipos de objetos usados pelo interpretador Python padrão, mas não expostos como componentes embutidos como *`int`* ou *`str`* são.

Por fim, fornece algumas classes e funções adicionais relacionadas ao tipo que não são fundamentais o suficiente para serem incorporadas.

8.10.1 Criação de tipos dinâmicos

types.**new_class**(*name*, *bases=()*, *kwds=None*, *exec_body=None*)

Cria um objeto de classe dinamicamente usando a metaclasse apropriada.

Os três primeiros argumentos são os componentes que compõem um cabeçalho de definição de classe: o nome da classe, as classes base (em ordem), os argumentos nomeados (como `metaclass`).

O argumento *exec_body* é um retorno de chamada usado para preencher o espaço para nome da classe recém-criado. Ele deve aceitar o espaço para nome da classe como seu único argumento e atualizar o espaço para nome diretamente com o conteúdo da classe. Se nenhum retorno de chamada for fornecido, ele terá o mesmo efeito que passar em `lambda ns: None`.

Adicionado na versão 3.3.

types.**prepare_class**(*name*, *bases=()*, *kwds=None*)

Calcula a metaclasse apropriada e cria o espaço de nomes da classe.

Os argumentos são os componentes que compõem um cabeçalho de definição de classe: o nome da classe, as classes base (em ordem) e os argumentos nomeados (como `metaclass`).

O valor de retorno é uma tupla de 3: `metaclass, namespace, kwds`

metaclass é a metaclasse apropriada, *namespace* é o espaço de nomes da classe preparada e *kwds* é uma cópia atualizada do argumento passado no *kwds* com qualquer entrada `'metaclass'` removida. Se nenhum argumento *kwds* for passado, este será um ditado vazio.

Adicionado na versão 3.3.

Alterado na versão 3.6: O valor padrão para o elemento `namespace` da tupla retornada foi alterado. Agora, um mapeamento preservando-ordem-inserção é usado quando a metaclasse não possui um método `__prepare__`.

Ver também

metaclasses
Detalhes completos do processo de criação de classe suportado por essas funções

PEP 3115 - Metaclasses no Python 3000
Introduzido o gancho de espaço de nomes `__prepare__`

types.**resolve_bases**(*bases*)

Resolve entradas MRO dinamicamente, conforme especificado pela **PEP 560**.

Esta função procura por itens em *bases* que não sejam instâncias de *type* e retorna uma tupla onde cada objeto que possui um método `__mro_entries__()` é substituído por um resultado desempacotado da chamada desse método. Se um item *bases* é uma instância de *type*, ou não possui o método `__mro_entries__()`, ele é incluído na tupla de retorno inalterada.

Adicionado na versão 3.7.

types.**get_original_bases**(*cls*, */*)

Retorna a tupla de objetos originalmente dados como as bases de *cls* antes do método `__mro_entries__()` ter sido chamado em qualquer base (seguindo os mecanismos apresentados na **PEP 560**). Isso é útil para introspecção de *Generics*.

Para classes que possuem um atributo `__orig_bases__`, esta função retorna o valor de `cls.__orig_bases__`. Para classes sem o atributo `__orig_bases__`, `cls.__bases__` é retornado.

Exemplos:

```
from typing import TypeVar, Generic, NamedTuple, TypedDict

T = TypeVar("T")
```

(continua na próxima página)

(continuação da página anterior)

```
class Foo(Generic[T]): ...
class Bar(Foo[int], float): ...
class Baz(list[str]): ...
Eggs = NamedTuple("Eggs", [("a", int), ("b", str)])
Spam = TypedDict("Spam", {"a": int, "b": str})

assert Bar.__bases__ == (Foo, float)
assert get_original_bases(Bar) == (Foo[int], float)

assert Baz.__bases__ == (list,)
assert get_original_bases(Baz) == (list[str],)

assert Eggs.__bases__ == (tuple,)
assert get_original_bases(Eggs) == (NamedTuple,)

assert Spam.__bases__ == (dict,)
assert get_original_bases(Spam) == (TypedDict,)

assert int.__bases__ == (object,)
assert get_original_bases(int) == (object,)
```

Adicionado na versão 3.12.

Ver também

PEP 560 - Suporte básico para módulo typing e tipos genéricos

8.10.2 Tipos padrão do interpretador

Este módulo fornece nomes para muitos dos tipos necessários para implementar um interpretador Python. Evita deliberadamente incluir alguns dos tipos que surgem apenas incidentalmente durante o processamento, como o tipo `listiterator`.

O uso típico desses nomes é para verificações *isinstance()* ou *issubclass()*.

Se você instanciar algum desses tipos, observe que as assinaturas podem variar entre as versões do Python.

Os nomes padrão são definidos para os seguintes tipos:

`types.`**`NoneType`**

O tipo de *None*.

Adicionado na versão 3.10.

`types.`**`FunctionType`**

`types.`**`LambdaType`**

O tipo de funções definidas pelo usuário e funções criadas por expressões `lambda`.

Levanta um *evento de auditoria* `function.__new__` com o argumento `code`.

O evento de auditoria ocorre apenas para instanciação direta de objetos de função e não é levantado para compilação normal.

`types.`**`GeneratorType`**

O tipo de objetos de iterador *gerador*, criados pelas funções de gerador.

`types.`**`CoroutineType`**

O tipo de objetos de *corrotina*, criado por funções de `async def`.

Adicionado na versão 3.5.

types.**AsyncGeneratorType**

O tipo de objetos de iterador *gerador assíncrono*, criados pelas funções do gerador assíncrono.

Adicionado na versão 3.6.

class types.**CodeType**(***kwargs*)

O tipo de objetos código retornados por `compile()`.

Levanta um `code.__new__` de *evento de auditoria* com os argumentos `code`, `filename`, `name`, `argcount`, `posonlyargcount`, `kwonlyargcount`, `nlocals`, `stacksize`, `flags`.

Observe que os argumentos auditados podem não corresponder aos nomes ou posições exigidos pelo inicializador. O evento de auditoria ocorre apenas para instanciação direta de objetos de código e não é levantado para compilação normal.

types.**CellType**

O tipo para objetos de célula: tais objetos são usados como contêineres para as *variáveis de clausura* de uma função.

Adicionado na versão 3.8.

types.**MethodType**

O tipo de método de instâncias de classe definidas pelo usuário.

types.**BuiltinFunctionType**

types.**BuiltinMethodType**

O tipo de funções embutidas como `len()` ou `sys.exit()`, e métodos de classes embutidas. (Aqui, o termo "embutidas" significa "escrito em C".)

types.**WrapperDescriptorType**

O tipo de método de alguns tipos de dados embutidos e classes base, como `object.__init__()` ou `object.__lt__()`.

Adicionado na versão 3.7.

types.**MethodWrapperType**

O tipo de métodos *vinculados* de alguns tipos de dados embutidos e classes base. Por exemplo, é o tipo de `object().__str__`.

Adicionado na versão 3.7.

types.**NotImplementedType**

O tipo de `NotImplemented`.

Adicionado na versão 3.10.

types.**MethodDescriptorType**

O tipo de método de alguns tipos de dados embutidos, como `str.join()`.

Adicionado na versão 3.7.

types.**ClassMethodDescriptorType**

O tipo de métodos de classe *não vinculados* de alguns tipos de dados embutidos, como `dict.__dict__['fromkeys']`.

Adicionado na versão 3.7.

class types.**ModuleType**(*name*, *doc=None*)

O tipo de *módulos*. O construtor aceita o nome do módulo a ser criado e, opcionalmente, seu *docstring*.

Ver também

Documentação em objetos de módulo
Fornece detalhes sobre os atributos especiais que podem ser encontrados em instâncias de `ModuleType`.

importlib.util.module_from_spec()
Módulos criados usando o construtor `ModuleType` são criados com muitos de seus atributos especiais não definidos ou definidos com valores padrão. `module_from_spec()` fornece uma maneira mais robusta de criar instâncias `ModuleType`, o que garante que os vários atributos sejam definidos adequadamente.

`types.`**`EllipsisType`**
O tipo de *`Ellipsis`*.

Adicionado na versão 3.10.

`class` `types.`**`GenericAlias`**(*t_origin*, *t_args*)
O tipo dos *genéricos parametrizados* como `list[int]`.

`t_origin` deve ser uma classe genérica não parametrizada, como `list`, `tuple` ou `dict`. `t_args` deve ser uma *`tuple`* (possivelmente com comprimento 1) de tipos que parametrizam `t_origin`:

```
>>> from types import GenericAlias

>>> list[int] == GenericAlias(list, (int,))
True
>>> dict[str, int] == GenericAlias(dict, (str, int))
True
```

Adicionado na versão 3.9.

Alterado na versão 3.9.2: Este tipo pode agora ter uma subclasse.

Ver também

Tipos Generic Alias
Documentação detalhada sobre instâncias de `types.GenericAlias`

PEP 585 - Sugestão de tipo para Genéricos em coleções padrão
Apresentação da classe `types.GenericAlias`

`class` `types.`**`UnionType`**
The type of *union type expressions*.

Adicionado na versão 3.10.

Alterado na versão 3.14: Isso é agora um apelido para *`typing.Union`*.

`class` `types.`**`TracebackType`**(*tb_next*, *tb_frame*, *tb_lasti*, *tb_lineno*)
O tipo de objetos traceback, como encontrados em `sys.exception().__traceback__`.

Veja a referência de linguagem para detalhes dos atributos e operações disponíveis, e orientação sobre como criar tracebacks dinamicamente.

`types.`**`FrameType`**
O tipo de objetos de quadro como encontrado em `tb.tb_frame` se `tb` é um objeto traceback.

`types.`**`GetSetDescriptorType`**
O tipo de objetos definidos em módulos de extensão com `PyGetSetDef`, como `FrameType.f_locals` ou `array.array.typecode`. Este tipo é usado como descritor para atributos de objeto; tem o mesmo propósito que o tipo *`property`*, mas para classes definidas em módulos de extensão.

types.**MemberDescriptorType**

O tipo de objetos definidos em módulos de extensão com `PyMemberDef`, como `datetime.timedelta.days`. Este tipo é usado como descritor para membros de dados C simples que usam funções de conversão padrão; tem o mesmo propósito que o tipo *property*, mas para classes definidas em módulos de extensão.

Além disso, quando uma classe é definida com um atributo `__slots__`, então para cada atributo, uma instância de `MemberDescriptorType` será adicionada como um atributo na classe. Isso permite que o atributo apareça no `__dict__` da classe.

Em outras implementações de Python, este tipo pode ser idêntico a `GetSetDescriptorType`.

class types.**MappingProxyType**(*mapping*)

Proxy somente leitura de um mapeamento. Ele fornece uma visão dinâmica das entradas do mapeamento, o que significa que quando o mapeamento muda, a visão reflete essas mudanças.

Adicionado na versão 3.3.

Alterado na versão 3.9: Atualizado para ter suporte ao novo operador de união (`|`) da **PEP 584**, que simplesmente delega para o mapeamento subjacente.

key in proxy

Retorna `True` se o mapeamento subjacente tiver uma chave *key*, senão `False`.

proxy[key]

Retorna e o item do mapeamento subjacente com a chave *key*. Levanta um *KeyError* se *key* não estiver no mapeamento subjacente.

iter(proxy)

Retorna um iterador sobre as chaves do mapeamento subjacente. Este é um atalho para `iter(proxy.keys())`.

len(proxy)

Retorna o número de itens no mapeamento subjacente.

copy()

Retorna uma cópia rasa do mapeamento subjacente.

get(*key*[, *default*])

Retorna o valor para *key* se *key* estiver no mapeamento subjacente, caso contrário, *default*. Se *default* não for fornecido, o padrão é `None`, de forma que este método nunca levante uma *KeyError*.

items()

Retorna uma nova visão dos itens do mapeamento subjacente (pares `(chave, valor)`).

keys()

Retorna uma nova visão das chaves do mapeamento subjacente.

values()

Retorna uma nova visão dos valores do mapeamento subjacente.

reversed(proxy)

Retorna um iterador reverso sobre as chaves do mapeamento subjacente.

Adicionado na versão 3.9.

hash(proxy)

Retorna um hash do mapeamento subjacente.

Adicionado na versão 3.12.

class types.**CapsuleType**

O tipo de objetos cápsula.

Adicionado na versão 3.13.

8.10.3 Classes e funções de utilidades adicionais

class types.**SimpleNamespace**

Uma subclasse *object* simples que fornece acesso de atributo ao seu espaço de nomes, bem como um repr significativo.

Ao contrário de *object*, com SimpleNamespace você pode adicionar e remover atributos.

Objetos *SimpleNamespace* podem ser inicializados da mesma forma que *dict*: ou com argumentos nomeados, com um único argumento posicional, ou com ambos. Quando inicializados com argumentos nomeados, eles são adicionados diretamente ao espaço de nomes subjacente. Alternativamente, quando inicializado com um argumento posicional, o espaço de nomes subjacente será atualizado com pares de valores-chave desse argumento (um objeto mapeamento ou um objeto *iterável* produzindo pares de valores-chave). Todas essas chaves devem ser strings.

O tipo é aproximadamente equivalente ao seguinte código:

```
class SimpleNamespace:
    def __init__(self, mapping_or_iterable=(), /, **kwargs):
        self.__dict__.update(mapping_or_iterable)
        self.__dict__.update(kwargs)

    def __repr__(self):
        items = (f"{k}={v!r}" for k, v in self.__dict__.items())
        return "{}({})".format(type(self).__name__, ", ".join(items))

    def __eq__(self, other):
        if isinstance(self, SimpleNamespace) and isinstance(other,␣
↪SimpleNamespace):
            return self.__dict__ == other.__dict__
        return NotImplemented
```

SimpleNamespace pode ser útil como um substituto para `class NS: pass`. No entanto, para um tipo de registro estruturado, use *namedtuple()*.

Objetos SimpleNamespace são suportados por *copy.replace()*.

Adicionado na versão 3.3.

Alterado na versão 3.9: A ordem dos atributos no repr mudou de alfabética para inserção (como no dict).

Alterado na versão 3.13: Adicionado suporte para um argumento posicional opcional.

types.**DynamicClassAttribute**(*fget=None*, *fset=None*, *fdel=None*, *doc=None*)

Roteia o acesso ao atributo em uma classe para __getattr__.

Este é um descritor, usado para definir atributos que atuam de forma diferente quando acessados por meio de uma instância e por meio de uma classe. O acesso à instância permanece normal, mas o acesso a um atributo por meio de uma classe será roteado para o método __getattr__ da classe; isso é feito levantando AttributeError.

Isso permite ter propriedades ativas em uma instância, e ter atributos virtuais na classe com o mesmo nome (veja *enum.Enum* para um exemplo).

Adicionado na versão 3.4.

8.10.4 Funções de utilidade de corrotina

types.**coroutine**(*gen_func*)

Esta função transforma uma função *geradora* em uma *função de corrotina* que retorna uma corrotina baseada em gerador. A corrotina baseada em gerador ainda é um *iterador gerador*, mas também é considerada um objeto *corrotina* e é *aguardável*. No entanto, pode não necessariamente implementar o método `__await__()`.

Se *gen_func* for uma função geradora, ela será modificada no local.

Se *gen_func* não for uma função geradora, ela será envolta. Se ele retornar uma instância de `Collections.abc.Generator`, a instância será envolvida em um objeto proxy *aguardável*. Todos os outros tipos de objetos serão retornados como estão.

Adicionado na versão 3.5.

8.11 `copy` — Operações de cópia profunda e cópia rasa

Código-fonte: Lib/copy.py

As instruções de atribuição no Python não copiam objetos, elas criam ligações entre um destino e um objeto. Para coleções que são mutáveis ou contêm itens mutáveis, às vezes é necessária uma cópia para que seja possível alterar uma cópia sem alterar a outra. Este módulo fornece operações genéricas de cópia profunda e rasa (explicadas abaixo).

Resumo da interface:

`copy.`**`copy`**(*obj*)

Retorna uma cópia rasa de *obj*.

`copy.`**`deepcopy`**(*obj*[, *memo*])

Retorna uma cópia profunda de *obj*.

`copy.`**`replace`**(*obj*, /, ***changes*)

Cria um novo objeto do mesmo tipo que *obj*, substituindo campos por valores de *changes*.

Adicionado na versão 3.13.

exception `copy.`**`Error`**

Levantada para erros específicos do módulo.

A diferença entre cópia profunda e rasa é relevante apenas para objetos compostos (objetos que contêm outros objetos, como listas ou instâncias de classe):

- Uma *cópia rasa* constrói um novo objeto composto e então (na medida do possível) insere nele *referências* aos objetos encontrados no original.
- Uma *cópia profunda* constrói um novo objeto composto e então, recursivamente, insere nele *cópias* dos objetos encontrados no original.

Frequentemente, existem dois problemas com operações de cópia profunda que não existem com operações de cópia rasa:

- Objetos recursivos (objetos compostos que, direta ou indiretamente, contêm uma referência a si mesmos) podem causar um laço recursivo.
- Como a cópia profunda copia tudo, ela pode copiar muito, como dados que devem ser compartilhados entre as cópias.

A função *`deepcopy()`* evita esses problemas:

- mantendo um dicionário `memo` de objetos já copiados durante a passagem de cópia atual; e
- permitindo que as classes definidas pelo usuário substituam a operação de cópia ou o conjunto de componentes copiados.

Este módulo não copia tipos como módulo, método, stack trace (situação da pilha de execução), quadro de empilhamento, arquivo, soquete, janela ou outros tipos semelhantes. Ele "copia" funções e classes (rasas e profundamente), devolvendo o objeto original inalterado; isso é compatível com a maneira que estes itens são tratados pelo módulo *`pickle`*.

Cópias rasas de dicionários podem ser feitas usando *`dict.copy()`*, e de listas atribuindo uma fatia de toda a lista, por exemplo, `lista_copiada = lista_original[:]`.

As classes podem usar as mesmas interfaces para controlar a cópia que usam para controlar a serialização com pickle. Veja a descrição do módulo *pickle* para informações sobre esses métodos. Na verdade, o módulo *copy* usa as funções pickle registradas do módulo *copyreg*.

Para que uma classe defina sua própria implementação de cópia, ela pode definir métodos especiais *__copy__()* e *__deepcopy__()*.

object.**__copy__**(*self*)

Chamado para implementar a operação de cópia rasa; nenhum argumento adicional é passado.

object.**__deepcopy__**(*self*, *memo*)

Chamado para implementar a operação de cópia profunda; é passado um argumento, o dicionário *memo*. Se a implementação __deepcopy__ precisar fazer uma cópia profunda de um componente, ela deve chamar a função *deepcopy()* com o componente como primeiro argumento e o dicionário *memo* como segundo argumento. O dicionário *memo* deve ser tratado como um objeto opaco.

A função `copy.replace()` é mais limitada que *copy()* e *deepcopy()*, e tem suporte a apenas tuplas nomeadas criadas por *namedtuple()*, *dataclasses* e outras classes que definem o método *__replace__()*.

object.**__replace__**(*self*, */*, ***changes*)

Este método deve criar um novo objeto do mesmo tipo, substituindo campos por valores de *changes*.

Adicionado na versão 3.13.

Ver também

Módulo *pickle*
Discussão dos métodos especiais usados para dar suporte à recuperação e restauração do estado do objeto.

8.12 `pprint` — Impressão bonita de dados

Código-fonte: Lib/pprint.py

O módulo *pprint* fornece a capacidade de "imprimir de forma bonita" estruturas de dados Python arbitrárias em um formato que pode ser usado como entrada para o interpretador. Se as estruturas formatadas incluírem objetos que não sejam tipos fundamentais do Python, a representação poderá não ser carregável. Este pode ser o caso se objetos como arquivos, soquetes ou classes forem incluídos, bem como muitos outros objetos que não são representáveis como literais do Python.

A representação formatada mantém os objetos em uma única linha, se possível, e os divide em múltiplas linhas se eles não couberem na largura permitida, ajustável pelo parâmetro *width* padrão para 80 caracteres.

Os dicionários são classificados por chave antes que a exibição seja calculada.

Alterado na versão 3.9: Adicionado suporte para impressão bonita de *types.SimpleNamespace*.

Alterado na versão 3.10: Adicionado suporte para impressão bonita de *dataclasses.dataclass*.

8.12.1 Funções

pprint.**pp**(*object*, *stream=None*, *indent=1*, *width=80*, *depth=None*, *, *compact=False*, *sort_dicts=False*, *underscore_numbers=False*)

Imprime a representação formatada de *object*, seguida por uma nova linha. Esta função pode ser usado no interpretador interativo em vez da função *print()* para inspecionar valores. Dica: você pode reatribuir `print = pprint.pp` para uso dentro de um escopo.

Parâmetros

- **`object`** – O objeto a ser impresso.

- **stream** (*file-like object* | None) – Um objeto arquivo ou similar no qual a saída será gravada chamando seu método `write()`. Se `None` (o padrão), *sys.stdout* será usado.
- **indent** (`int`) – A quantidade de indentação adicionado para cada nível de aninhamento.
- **width** (`int`) – O número máximo desejado de caracteres por linha na saída. Se uma estrutura não puder ser formatada dentro da restrição de largura, será feito o melhor esforço.
- **depth** (`int` | *None*) – O número de níveis de aninhamento que podem ser impressos. Se a estrutura de dados que está sendo impressa for muito profunda, o próximo nível contido será substituído por `....` Se `None` (o padrão), não há restrição na profundidade dos objetos que estão sendo formatados.
- **compact** (`bool`) – Controla a forma como *sequências* são formatadas. Se `False` (o padrão), cada item de uma sequência será formatado em uma linha separada, caso contrário, tantos itens quantos couberem na *largura* serão formatados em cada linha de saída.
- **sort_dicts** (`bool`) – Se `True`, os dicionários serão formatados com suas chaves classificadas, caso contrário, serão exibidos na ordem de inserção (o padrão).
- **underscore_numbers** (`bool`) – Se `True`, os números inteiros serão formatados com o caractere _ para um separador de milhares, caso contrário, os sublinhados não serão exibidos (o padrão).

```
>>> import pprint
>>> stuff = ['spam', 'eggs', 'lumberjack', 'knights', 'ni']
>>> stuff.insert(0, stuff)
>>> pprint.pp(stuff)
[<Recursion on list with id=...>,
 'spam',
 'eggs',
 'lumberjack',
 'knights',
 'ni']
```

Adicionado na versão 3.8.

pprint.**pprint**(*object*, *stream=None*, *indent=1*, *width=80*, *depth=None*, ***, *compact=False*, *sort_dicts=True*, *underscore_numbers=False*)

Apelido para *pp()* com *sort_dicts* definido como `True` por padrão, o que classificaria automaticamente as chaves dos dicionários, você pode querer usar *pp()* em vez disso, onde é `False` por padrão.

pprint.**pformat**(*object*, *indent=1*, *width=80*, *depth=None*, ***, *compact=False*, *sort_dicts=True*, *underscore_numbers=False*)

Retorna a representação formatada de *object* como uma string. *indent*, *width*, *depth*, *compact*, *sort_dicts* e *underscore_numbers* são passados para o construtor de *PrettyPrinter* como parâmetros de formatação e seus significados são descritos na documentação acima.

pprint.**isreadable**(*object*)

Determina se a representação formatada de *object* é "legível" ou pode ser usada para reconstruir o valor usando *eval()*. Isso sempre retorna `False` para objetos recursivos.

```
>>> pprint.isreadable(stuff)
False
```

pprint.**isrecursive**(*object*)

Determina se *object* requer uma representação recursiva. Esta função está sujeita às mesmas limitações mencionadas em *saferepr()* abaixo e pode levantar uma exceção *RecursionError* se falhar em detectar um objeto recursivo.

pprint.**saferepr**(*object*)

Retorna uma representação de string de *object*, protegida contra recursão em algumas estruturas de dados

comuns, nomeadamente instâncias de *dict*, *list* e *tuple* ou subclasses cujo __repr__ não foi substituído. Se a representação do objeto expõe uma entrada recursiva, a referência recursiva será representada como <Recursion on typename with id=number>. A representação não é formatada de outra forma.

```
>>> pprint.saferepr(stuff)
"[<Recursion on list with id=...>, 'spam', 'eggs', 'lumberjack', 'knights', 'ni
↪']"
```

8.12.2 Objetos PrettyPrinter

class pprint.**PrettyPrinter**(*indent=1*, *width=80*, *depth=None*, *stream=None*, *, *compact=False*, *sort_dicts=True*, *underscore_numbers=False*)

Constrói uma instância *PrettyPrinter*.

Os argumentos têm o mesmo significado que para *pp()*. Observe que eles estão em uma ordem diferente e que o padrão *sort_dicts* é True.

```
>>> import pprint
>>> stuff = ['spam', 'eggs', 'lumberjack', 'knights', 'ni']
>>> stuff.insert(0, stuff[:])
>>> pp = pprint.PrettyPrinter(indent=4)
>>> pp.pprint(stuff)
[   ['spam', 'eggs', 'lumberjack', 'knights', 'ni'],
    'spam',
    'eggs',
    'lumberjack',
    'knights',
    'ni']
>>> pp = pprint.PrettyPrinter(width=41, compact=True)
>>> pp.pprint(stuff)
[['spam', 'eggs', 'lumberjack',
  'knights', 'ni'],
 'spam', 'eggs', 'lumberjack', 'knights',
 'ni']
>>> tup = ('spam', ('eggs', ('lumberjack', ('knights', ('ni', ('dead',
... ('parrot', ('fresh fruit',)))))))
>>> pp = pprint.PrettyPrinter(depth=6)
>>> pp.pprint(tup)
('spam', ('eggs', ('lumberjack', ('knights', ('ni', ('dead', (...)))))))
```

Alterado na versão 3.4: Adicionado o parâmetro *compact*.

Alterado na versão 3.8: Adicionado o parâmetro *sort_dicts*.

Alterado na versão 3.10: Adicionado o parâmetro *underscore_numbers*.

Alterado na versão 3.11: Não tenta mais escrever em sys.stdout se for None.

Instâncias *PrettyPrinter* contêm os seguintes métodos:

PrettyPrinter.**pformat**(*object*)

Retorna a representação formatada de *object*. Isso leva em consideração as opções passadas para o construtor *PrettyPrinter*.

PrettyPrinter.**pprint**(*object*)

Exibe a representação formatada de *object* no fluxo configurado, seguida por uma nova linha.

Os métodos a seguir fornecem as implementações para as funções correspondentes com os mesmos nomes. Usar esses métodos em uma instância é um pouco mais eficiente já que novos objetos *PrettyPrinter* não precisam ser criados.

PrettyPrinter.**isreadable**(*object*)

Determina se a representação formatada do objeto é "legível" ou pode ser usada para reconstruir o valor usando *eval()*. Observe que isso retorna False para objetos recursivos. Se o parâmetro *depth* de *PrettyPrinter* estiver definido e o objeto for mais profundo que o permitido, isso retornará False.

PrettyPrinter.**isrecursive**(*object*)

Determina se o objeto requer uma representação recursiva.

Este método é fornecido como um gancho para permitir que as subclasses modifiquem a maneira como os objetos são convertidos em strings. A implementação padrão usa os componentes internos da implementação de *saferepr()*.

PrettyPrinter.**format**(*object*, *context*, *maxlevels*, *level*)

Retorna três valores: a versão formatada de *object* como uma string, um sinalizador indicando se o resultado é legível e um sinalizador indicando se a recursão foi detectada. O primeiro argumento é o objeto a ser apresentado. O segundo é um dicionário que contém o *id()* de objetos que fazem parte do contexto de apresentação atual (contêineres diretos e indiretos para *objects* que estão afetando a apresentação) como chaves; se for necessário apresentar um objeto que já esteja representado em *context*, o terceiro valor de retorno deverá ser True. Chamadas recursivas ao método *format()* devem adicionar entradas adicionais para contêineres neste dicionário. O terceiro argumento, *maxlevels*, fornece o limite solicitado para recursão; será 0 se não houver limite solicitado. Este argumento deve ser passado sem modificações para chamadas recursivas. O quarto argumento, *level*, fornece o nível atual; chamadas recursivas devem receber um valor menor que o da chamada atual.

8.12.3 Exemplo

Para demonstrar vários usos da função *pp()* e seus parâmetros, vamos buscar informações sobre um projeto no PyPI:

```
>>> import json
>>> import pprint
>>> from urllib.request import urlopen
>>> with urlopen('https://pypi.org/pypi/sampleproject/1.2.0/json') as resp:
...     project_info = json.load(resp)['info']
```

Em sua forma básica, *pp()* mostra o objeto inteiro:

```
>>> pprint.pp(project_info)
{'author': 'The Python Packaging Authority',
 'author_email': 'pypa-dev@googlegroups.com',
 'bugtrack_url': None,
 'classifiers': ['Development Status :: 3 - Alpha',
                 'Intended Audience :: Developers',
                 'License :: OSI Approved :: MIT License',
                 'Programming Language :: Python :: 2',
                 'Programming Language :: Python :: 2.6',
                 'Programming Language :: Python :: 2.7',
                 'Programming Language :: Python :: 3',
                 'Programming Language :: Python :: 3.2',
                 'Programming Language :: Python :: 3.3',
                 'Programming Language :: Python :: 3.4',
                 'Topic :: Software Development :: Build Tools'],
 'description': 'A sample Python project\n'
                '=======================\n'
                '\n'
                'This is the description file for the project.\n'
                '\n'
                'The file should use UTF-8 encoding and be written using '
                'ReStructured Text. It\n'
                'will be used to generate the project webpage on PyPI, and '
                'should be written for\n'
```

(continua na próxima página)

(continuação da página anterior)

```
                'that purpose.\n'
                '\n'
                'Typical contents for this file would include an overview of '
                'the project, basic\n'
                'usage examples, etc. Generally, including the project '
                'changelog in here is not\n'
                'a good idea, although a simple "What\'s New" section for the '
                'most recent version\n'
                'may be appropriate.',
 'description_content_type': None,
 'docs_url': None,
 'download_url': 'UNKNOWN',
 'downloads': {'last_day': -1, 'last_month': -1, 'last_week': -1},
 'home_page': 'https://github.com/pypa/sampleproject',
 'keywords': 'sample setuptools development',
 'license': 'MIT',
 'maintainer': None,
 'maintainer_email': None,
 'name': 'sampleproject',
 'package_url': 'https://pypi.org/project/sampleproject/',
 'platform': 'UNKNOWN',
 'project_url': 'https://pypi.org/project/sampleproject/',
 'project_urls': {'Download': 'UNKNOWN',
                  'Homepage': 'https://github.com/pypa/sampleproject'},
 'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
 'requires_dist': None,
 'requires_python': None,
 'summary': 'A sample Python project',
 'version': '1.2.0'}
```

O resultado pode ser limitado a uma certa *depth* (reticências são usadas para conteúdos mais profundos):

```
>>> pprint.pp(project_info, depth=1)
{'author': 'The Python Packaging Authority',
 'author_email': 'pypa-dev@googlegroups.com',
 'bugtrack_url': None,
 'classifiers': [...],
 'description': 'A sample Python project\n'
                '=======================\n'
                '\n'
                'This is the description file for the project.\n'
                '\n'
                'The file should use UTF-8 encoding and be written using '
                'ReStructured Text. It\n'
                'will be used to generate the project webpage on PyPI, and '
                'should be written for\n'
                'that purpose.\n'
                '\n'
                'Typical contents for this file would include an overview of '
                'the project, basic\n'
                'usage examples, etc. Generally, including the project '
                'changelog in here is not\n'
                'a good idea, although a simple "What\'s New" section for the '
                'most recent version\n'
                'may be appropriate.',
 'description_content_type': None,
```

(continua na próxima página)

(continuação da página anterior)

```
 'docs_url': None,
 'download_url': 'UNKNOWN',
 'downloads': {...},
 'home_page': 'https://github.com/pypa/sampleproject',
 'keywords': 'sample setuptools development',
 'license': 'MIT',
 'maintainer': None,
 'maintainer_email': None,
 'name': 'sampleproject',
 'package_url': 'https://pypi.org/project/sampleproject/',
 'platform': 'UNKNOWN',
 'project_url': 'https://pypi.org/project/sampleproject/',
 'project_urls': {...},
 'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
 'requires_dist': None,
 'requires_python': None,
 'summary': 'A sample Python project',
 'version': '1.2.0'}
```

Além disso, a largura *width* máxima do caractere pode ser sugerida. Se um objeto longo não puder ser dividido, a largura especificada será excedida:

```
>>> pprint.pp(project_info, depth=1, width=60)
{'author': 'The Python Packaging Authority',
 'author_email': 'pypa-dev@googlegroups.com',
 'bugtrack_url': None,
 'classifiers': [...],
 'description': 'A sample Python project\n'
                '=======================\n'
                '\n'
                'This is the description file for the '
                'project.\n'
                '\n'
                'The file should use UTF-8 encoding and be '
                'written using ReStructured Text. It\n'
                'will be used to generate the project '
                'webpage on PyPI, and should be written '
                'for\n'
                'that purpose.\n'
                '\n'
                'Typical contents for this file would '
                'include an overview of the project, '
                'basic\n'
                'usage examples, etc. Generally, including '
                'the project changelog in here is not\n'
                'a good idea, although a simple "What\'s '
                'New" section for the most recent version\n'
                'may be appropriate.',
 'description_content_type': None,
 'docs_url': None,
 'download_url': 'UNKNOWN',
 'downloads': {...},
 'home_page': 'https://github.com/pypa/sampleproject',
 'keywords': 'sample setuptools development',
 'license': 'MIT',
 'maintainer': None,
```

(continua na próxima página)

(continuação da página anterior)

```
'maintainer_email': None,
'name': 'sampleproject',
'package_url': 'https://pypi.org/project/sampleproject/',
'platform': 'UNKNOWN',
'project_url': 'https://pypi.org/project/sampleproject/',
'project_urls': {...},
'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
'requires_dist': None,
'requires_python': None,
'summary': 'A sample Python project',
'version': '1.2.0'}
```

8.13 `reprlib` — Implementação alternativa à `repr()`

Código-fonte: Lib/reprlib.py

O módulo `reprlib` fornece um meio de produzir representações de objetos com limites no tamanho das strings resultantes. Isso é usado no depurador Python e pode ser útil em outros contextos também.

Este módulo fornece uma classe, uma instância e uma função.

class reprlib.**Repr**(*, *maxlevel=6*, *maxtuple=6*, *maxlist=6*, *maxarray=5*, *maxdict=4*, *maxset=6*, *maxfrozenset=6*, *maxdeque=6*, *maxstring=30*, *maxlong=40*, *maxother=30*, *fillvalue='...'*, *indent=None*)

Classe que fornece serviços de formatação úteis na implementação de funções semelhantes à embutida *repr()*; limites de tamanho para diferentes tipos de objetos são adicionados para evitar a geração de representações excessivamente longas.

Os argumentos nomeados do construtor podem ser usados como um atalho para definir os atributos da instância de *Repr*. O que significa que a inicialização a seguir:

```
aRepr = reprlib.Repr(maxlevel=3)
```

É equivalente a:

```
aRepr = reprlib.Repr()
aRepr.maxlevel = 3
```

Consulte a seção *Objetos Repr* para obter mais informações sobre atributos *Repr*.

Alterado na versão 3.12: Permite que atributos sejam definidos por meio de argumentos nomeados.

reprlib.**aRepr**

Esta é uma instância de *Repr* que é usada para fornecer a função *repr()* descrita abaixo. Alterar os atributos deste objeto afetará os limites de tamanho usados por *repr()* e pelo depurador Python.

reprlib.**repr**(*obj*)

Este é o método *repr()* de `aRepr`. Ele retorna uma string semelhante à retornada pela função embutida de mesmo nome, mas com limites na maioria dos tamanhos.

Além das ferramentas de limitação de tamanho, o módulo também fornece um decorador para detectar chamadas recursivas para `__repr__()` e substituir uma string de espaço reservado.

@reprlib.**recursive_repr**(*fillvalue='...'*)

Decorador para métodos `__repr__()` para detectar chamadas recursivas dentro da mesma thread. Se uma chamada recursiva for feita, o *fillvalue* é retornado; caso contrário, a chamada usual `__repr__()` é feita. Por exemplo:

```
>>> from reprlib import recursive_repr
>>> class MyList(list):
...     @recursive_repr()
...     def __repr__(self):
...         return '<' + '|'.join(map(repr, self)) + '>'
...
>>> m = MyList('abc')
>>> m.append(m)
>>> m.append('x')
>>> print(m)
<'a'|'b'|'c'|...|'x'>
```

Adicionado na versão 3.2.

8.13.1 Objetos Repr

Instâncias *Repr* fornecem vários atributos que podem ser usados para fornecer limites de tamanho para as representações de diferentes tipos de objetos e métodos que formatam tipos de objetos específicos.

Repr.**fillvalue**

Esta string é exibida para referências recursivas. O padrão é `...`.

Adicionado na versão 3.11.

Repr.**maxlevel**

Limite de profundidade na criação de representações recursivas. O padrão é `6`.

Repr.**maxdict**

Repr.**maxlist**

Repr.**maxtuple**

Repr.**maxset**

Repr.**maxfrozenset**

Repr.**maxdeque**

Repr.**maxarray**

Limites no número de entradas representadas para o tipo de objeto nomeado. O padrão é `4` para *maxdict*, `5` para *maxarray* e `6` para os demais.

Repr.**maxlong**

Número máximo de caracteres na representação para um inteiro. Os dígitos são omitidos a partir do meio. O padrão é `40`.

Repr.**maxstring**

Limite no número de caracteres na representação da string. Observe que a representação "normal" da string é usada como fonte de caracteres: se sequências de escape forem necessárias na representação, elas poderão ser distorcidas quando a representação for encurtada. O padrão é `30`.

Repr.**maxother**

Este limite é usado para controlar o tamanho de tipos de objetos para os quais nenhum método de formatação específico está disponível no objeto *Repr*. Ele é aplicado de maneira semelhante a *maxstring*. O padrão é `20`.

Repr.**indent**

Se este atributo for definido como `None` (o padrão), a saída será formatada sem quebras de linha ou recuo, como o padrão *repr()*. Por exemplo:

```
>>> example = [
...     1, 'spam', {'a': 2, 'b': 'spam eggs', 'c': {3: 4.5, 6: []}}, 'ham']
>>> import reprlib
```

(continua na próxima página)

(continuação da página anterior)

```
>>> aRepr = reprlib.Repr()
>>> print(aRepr.repr(example))
[1, 'spam', {'a': 2, 'b': 'spam eggs', 'c': {3: 4.5, 6: []}}, 'ham']
```

Se *indent* for definido como uma string, cada nível de recursão será colocado em sua própria linha, recuado por essa string:

```
>>> aRepr.indent = '-->'
>>> print(aRepr.repr(example))
[
-->1,
-->'spam',
-->{
-->-->'a': 2,
-->-->'b': 'spam eggs',
-->-->'c': {
-->-->-->3: 4.5,
-->-->-->6: [],
-->-->},
-->},
-->'ham',
]
```

Definir *indent* como um valor inteiro positivo se comporta como se fosse definido como uma string com esse número de espaços:

```
>>> aRepr.indent = 4
>>> print(aRepr.repr(example))
[
    1,
    'spam',
    {
        'a': 2,
        'b': 'spam eggs',
        'c': {
            3: 4.5,
            6: [],
        },
    },
    'ham',
]
```

Adicionado na versão 3.12.

`Repr.`**`repr`**(*obj*)

O equivalente ao *repr()* embutido que usa a formatação imposta pela instância.

`Repr.`**`repr1`**(*obj*, *level*)

Implementação recursiva usada por *repr()*. Esta usa o tipo de *obj* para determinar qual método de formatação chamar, passando *obj* e *level*. Os métodos específicos de tipo devem chamar *repr1()* para realizar a formatação recursiva, com `level - 1` para o valor de *level* na chamada recursiva.

`Repr.`**`repr_TYPE`**(*obj*, *level*)

Métodos de formatação para tipos específicos são implementados como métodos com um nome baseado no nome do tipo. No nome do método, **TYPE** é substituído por `'_'.join(type(obj).__name__.split())`. O despacho para esses métodos é tratado por *repr1()*. Métodos específicos de tipo que precisam formatar um valor recursivamente devem chamar `self.repr1(subobj, level - 1)`.

8.13.2 Estendendo objetos Repr

O uso de despacho dinâmico por *Repr.repr1()* permite que subclasses de *Repr* adicionem suporte para tipos de objetos embutidos adicionais ou modifiquem o tratamento de tipos já suportados. Este exemplo mostra como suporte especial para objetos arquivo pode ser adicionado:

```
import reprlib
import sys

class MyRepr(reprlib.Repr):

    def repr_TextIOWrapper(self, obj, level):
        if obj.name in {'<stdin>', '<stdout>', '<stderr>'}:
            return obj.name
        return repr(obj)

aRepr = MyRepr()
print(aRepr.repr(sys.stdin))          # exibe '<stdin>'
```

```
<stdin>
```

8.14 `enum` — Suporte a enumerações

Adicionado na versão 3.4.

Código-fonte: Lib/enum.py

Important

Esta página contêm informação de referência da API. Para informação tutorial e discussão de tópicos mais avançados, consulte

- Tutorial básico
- Tutorial avançado
- Livro de receitas de enum

Uma enumeração:

- é um conjunto de nomes simbólicos (membros) vinculados a valores únicos
- pode ser iterado para retornar seus membros canônicos (ou seja, não incluindo apelidos) na ordem de definição
- usa a sintaxe *call* para retornar membros por valor
- usa a sintaxe *index* para retornar membros por nome

As enumerações são criadas usando a sintaxe de `class` ou usando a sintaxe de chamada de função:

```
>>> from enum import Enum

>>> # sintaxe de classe
>>> class Color(Enum):
...     RED = 1
...     GREEN = 2
...     BLUE = 3

>>> # sintaxe funcional
>>> Color = Enum('Color', [('RED', 1), ('GREEN', 2), ('BLUE', 3)])
```

Embora possamos usar a sintaxe `class` para criar Enums, Enums não são classes Python normais. Veja Como Enums são diferentes? para mais detalhes.

Nota

Nomenclatura

- A classe `Color` é uma *enumeração* (ou *enum*)
- Os atributos `Color.RED`, `Color.GREEN`, etc., são *membros de enumeração* (ou *membros*) e são constantes funcionalmente.
- Os membros enum têm *nomes* e *valores* (o nome de `Color.RED` é `RED`, o valor de `Color.BLUE` é `3`, etc.)

8.14.1 Conteúdo do módulo

EnumType

O `type` para Enum e suas subclasses.

Enum

Classe base para criação de constantes enumeradas.

IntEnum

Classe base para criar constantes enumeradas que também são subclasses de *int*. (*Notas*)

StrEnum

Classe base para criar constantes enumeradas que também são subclasses de *str*. (*Notas*)

Flag

Classe base para criar constantes enumeradas que podem ser combinadas usando operações bit a bit sem perder sua associação *Flag*.

IntFlag

Classe base para criar constantes enumeradas que podem ser combinadas usando operadores bit a bit sem perder sua associação *IntFlag*. Membros de *IntFlag* também são subclasses de *int*. (*Notas*)

ReprEnum

Usado por *IntEnum*, *StrEnum* e *IntFlag* para manter o *str()* do tipo misto.

EnumCheck

Uma enumeração com os valores `CONTINUOUS`, `NAMED_FLAGS` e `UNIQUE`, para uso com *verify()* para garantir que várias restrições sejam atendidas por uma determinada enumeração.

FlagBoundary

Uma enumeração com os valores `STRICT`, `CONFORM`, `EJECT` e `KEEP` que permite um controle mais refinado sobre como valores inválidos são tratados em uma enumeração.

EnumDict

Uma subclasse de *dict* para uso ao criar subclasse de *EnumType*.

auto

As instâncias são substituídas por um valor apropriado para membros Enum. *StrEnum* assume como padrão a versão em minúsculas do nome do membro, enquanto outros Enums assumem como padrão 1 e aumentam a partir daí.

property()

Permite que membros *Enum* tenham atributos sem conflitar com nomes de membros. Os atributos `value` e `name` são implementados dessa forma.

unique()

Decorador de classe Enum que garante que apenas um nome seja vinculado a cada valor.

verify()

Decorador de classe Enum que verifica restrições selecionáveis pelo usuário em uma enumeração.

member()

Torna `obj` um membro. Pode ser usado como um decorador.

nonmember()

Não torna `obj` um membro. Pode ser usado como um decorador.

global_enum()

Modifica *str()* e *repr()* de uma enumeração para mostrar seus membros como pertencentes ao módulo em vez de sua classe, e exporta os membros da enumeração para o espaço de nomes global.

show_flag_values()

Retorna uma lista de todos os inteiros de potência de dois contidos em um sinalizador.

Adicionado na versão 3.6: `Flag`, `IntFlag`, `auto`

Adicionado na versão 3.11: `StrEnum`, `EnumCheck`, `ReprEnum`, `FlagBoundary`, `property`, `member`, `nonmember`, `global_enum`, `show_flag_values`

Adicionado na versão 3.13: `EnumDict`

8.14.2 Tipos de Dados

class enum.**EnumType**

EnumType é a *metaclasse* para enumerações *enum*. É possível criar subclasse de *EnumType* – veja Criando subclasses de EnumType para detalhes.

`EnumType` é responsável por definir os métodos `__repr__()`, `__str__()`, `__format__()` e `__reduce__()` corretos no *enum* final, bem como criar os membros do enum, manipular adequadamente as duplicatas, fornecer iteração sobre a classe do enum, etc.

Adicionado na versão 3.11: Before 3.11 `EnumType` was called `EnumMeta`, which is still available as an alias.

__call__(*cls*, *value*, *names=None*, ***, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

This method is called in two different ways:

- to look up an existing member:

  cls
  The enum class being called.

  value
  The value to lookup.

- to use the `cls` enum to create a new enum (only if the existing enum does not have any members):

  cls
  The enum class being called.

value
The name of the new Enum to create.

nomes
The names/values of the members for the new Enum.

módulo
The name of the module the new Enum is created in.

qualname
The actual location in the module where this Enum can be found.

tipo
A mix-in type for the new Enum.

start
The first integer value for the Enum (used by *auto*).

boundary
How to handle out-of-range values from bit operations (*Flag* only).

__contains__(*cls*, *member*)

Returns `True` if member belongs to the `cls`:

```
>>> some_var = Color.RED
>>> some_var in Color
True
>>> Color.RED.value in Color
True
```

Alterado na versão 3.12: Before Python 3.12, a `TypeError` is raised if a non-Enum-member is used in a containment check.

__dir__(*cls*)

Returns `['__class__', '__doc__', '__members__', '__module__']` and the names of the members in *cls*:

```
>>> dir(Color)
['BLUE', 'GREEN', 'RED', '__class__', '__contains__', '__doc__', '__
↪getitem__', '__init_subclass__', '__iter__', '__len__', '__members__', '_
↪_module__', '__name__', '__qualname__']
```

__getitem__(*cls*, *name*)

Returns the Enum member in *cls* matching *name*, or raises a *KeyError*:

```
>>> Color['BLUE']
<Color.BLUE: 3>
```

__iter__(*cls*)

Returns each member in *cls* in definition order:

```
>>> list(Color)
[<Color.RED: 1>, <Color.GREEN: 2>, <Color.BLUE: 3>]
```

__len__(*cls*)

Returns the number of member in *cls*:

```
>>> len(Color)
3
```

__members__

Returns a mapping of every enum name to its member, including aliases

__reversed__(*cls*)

Returns each member in *cls* in reverse definition order:

```
>>> list(reversed(Color))
[<Color.BLUE: 3>, <Color.GREEN: 2>, <Color.RED: 1>]
```

class enum.**Enum**

Enum is the base class for all *enum* enumerations.

name

The name used to define the `Enum` member:

```
>>> Color.BLUE.name
'BLUE'
```

value

The value given to the `Enum` member:

```
>>> Color.RED.value
1
```

Value of the member, can be set in *__new__()*.

Nota

Enum member values

Member values can be anything: *int*, *str*, etc. If the exact value is unimportant you may use *auto* instances and an appropriate value will be chosen for you. See *auto* for the details.

While mutable/unhashable values, such as *dict*, *list* or a mutable *dataclass*, can be used, they will have a quadratic performance impact during creation relative to the total number of mutable/unhashable values in the enum.

name

Name of the member.

value

Value of the member, can be set in *__new__()*.

order

No longer used, kept for backward compatibility. (class attribute, removed during class creation).

ignore

`_ignore_` is only used during creation and is removed from the enumeration once creation is complete.

`_ignore_` is a list of names that will not become members, and whose names will also be removed from the completed enumeration. See TimePeriod for an example.

__dir__(*self*)

Returns `['__class__', '__doc__', '__module__', 'name', 'value']` and any public methods defined on *self.__class__*:

```
>>> from datetime import date
>>> class Weekday(Enum):
...     MONDAY = 1
```

(continua na próxima página)

(continuação da página anterior)

```
...     TUESDAY = 2
...     WEDNESDAY = 3
...     THURSDAY = 4
...     FRIDAY = 5
...     SATURDAY = 6
...     SUNDAY = 7
...     @classmethod
...     def today(cls):
...         print('today is %s' % cls(date.today().isoweekday()).name)
...
>>> dir(Weekday.SATURDAY)
['__class__', '__doc__', '__eq__', '__hash__', '__module__', 'name', 'today
↪', 'value']
```

_generate_next_value_(*name*, *start*, *count*, *last_values*)

nome
The name of the member being defined (e.g. 'RED').

start
The start value for the Enum; the default is 1.

count
The number of members currently defined, not including this one.

last_values
A list of the previous values.

A *staticmethod* that is used to determine the next value returned by `auto`:

```
>>> from enum import auto
>>> class PowersOfThree(Enum):
...     @staticmethod
...     def _generate_next_value_(name, start, count, last_values):
...         return 3 ** (count + 1)
...     FIRST = auto()
...     SECOND = auto()
...
>>> PowersOfThree.SECOND.value
9
```

__init__(*self*, **args*, ***kwds*)

By default, does nothing. If multiple values are given in the member assignment, those values become separate arguments to `__init__`; e.g.

```
>>> from enum import Enum
>>> class Weekday(Enum):
...     MONDAY = 1, 'Mon'
```

`Weekday.__init__()` would be called as `Weekday.__init__(self, 1, 'Mon')`

__init_subclass__(*cls*, ***kwds*)

A *classmethod* that is used to further configure subsequent subclasses. By default, does nothing.

missing(*cls*, *value*)

A *classmethod* for looking up values not found in *cls*. By default it does nothing, but can be overridden to implement custom search behavior:

```
>>> from enum import StrEnum
>>> class Build(StrEnum):
...     DEBUG = auto()
...     OPTIMIZED = auto()
...     @classmethod
...     def _missing_(cls, value):
...         value = value.lower()
...         for member in cls:
...             if member.value == value:
...                 return member
...         return None
...
>>> Build.DEBUG.value
'debug'
>>> Build('deBUG')
<Build.DEBUG: 'debug'>
```

__new__(*cls*, **args*, ***kwds*)

By default, doesn't exist. If specified, either in the enum class definition or in a mixin class (such as `int`), all values given in the member assignment will be passed; e.g.

```
>>> from enum import Enum
>>> class MyIntEnum(int, Enum):
...     TWENTYSIX = '1a', 16
```

results in the call `int('1a', 16)` and a value of `26` for the member.

Nota

When writing a custom `__new__`, do not use `super().__new__` – call the appropriate `__new__` instead.

__repr__(*self*)

Returns the string used for *repr()* calls. By default, returns the *Enum* name, member name, and value, but can be overridden:

```
>>> class OtherStyle(Enum):
...     ALTERNATE = auto()
...     OTHER = auto()
...     SOMETHING_ELSE = auto()
...     def __repr__(self):
...         cls_name = self.__class__.__name__
...         return f'{cls_name}.{self.name}'
...
>>> OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f"{OtherStyle.
↪ALTERNATE}"
(OtherStyle.ALTERNATE, 'OtherStyle.ALTERNATE', 'OtherStyle.ALTERNATE')
```

__str__(*self*)

Returns the string used for *str()* calls. By default, returns the *Enum* name and member name, but can be overridden:

```
>>> class OtherStyle(Enum):
...     ALTERNATE = auto()
...     OTHER = auto()
...     SOMETHING_ELSE = auto()
```

(continua na próxima página)

(continuação da página anterior)

```
...     def __str__(self):
...         return f'{self.name}'
...
>>> OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f"{OtherStyle.
↪ALTERNATE}"
(<OtherStyle.ALTERNATE: 1>, 'ALTERNATE', 'ALTERNATE')
```

__format__(*self*)

Returns the string used for *format()* and *f-string* calls. By default, returns `__str__()` return value, but can be overridden:

```
>>> class OtherStyle(Enum):
...     ALTERNATE = auto()
...     OTHER = auto()
...     SOMETHING_ELSE = auto()
...     def __format__(self, spec):
...         return f'{self.name}'
...
>>> OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f"{OtherStyle.
↪ALTERNATE}"
(<OtherStyle.ALTERNATE: 1>, 'OtherStyle.ALTERNATE', 'ALTERNATE')
```

Nota

Using `auto` with `Enum` results in integers of increasing value, starting with `1`.

Alterado na versão 3.12: Added enum-dataclass-support

_add_alias_()

Adds a new name as an alias to an existing member:

```
>>> Color.RED._add_alias_("ERROR")
>>> Color.ERROR
<Color.RED: 1>
```

Raises a `NameError` if the name is already assigned to a different member.

Adicionado na versão 3.13.

_add_value_alias_()

Adds a new value as an alias to an existing member:

```
>>> Color.RED._add_value_alias_(42)
>>> Color(42)
<Color.RED: 1>
```

Raises a `ValueError` if the value is already linked with a different member.

Adicionado na versão 3.13.

class enum.**IntEnum**

IntEnum is the same as `Enum`, but its members are also integers and can be used anywhere that an integer can be used. If any integer operation is performed with an *IntEnum* member, the resulting value loses its enumeration status.

```
>>> from enum import IntEnum
>>> class Number(IntEnum):
```

(continua na próxima página)

(continuação da página anterior)

```
...     ONE = 1
...     TWO = 2
...     THREE = 3
...
>>> Number.THREE
<Number.THREE: 3>
>>> Number.ONE + Number.TWO
3
>>> Number.THREE + 5
8
>>> Number.THREE == 3
True
```

Nota

Using *auto* with *IntEnum* results in integers of increasing value, starting with 1.

Alterado na versão 3.11: `__str__()` is now `int.__str__()` to better support the *replacement of existing constants* use-case. `__format__()` was already `int.__format__()` for that same reason.

class enum.**StrEnum**

StrEnum is the same as *Enum*, but its members are also strings and can be used in most of the same places that a string can be used. The result of any string operation performed on or with a *StrEnum* member is not part of the enumeration.

```
>>> from enum import StrEnum, auto
>>> class Color(StrEnum):
...     RED = 'r'
...     GREEN = 'g'
...     BLUE = 'b'
...     UNKNOWN = auto()
...
>>> Color.RED
<Color.RED: 'r'>
>>> Color.UNKNOWN
<Color.UNKNOWN: 'unknown'>
>>> str(Color.UNKNOWN)
'unknown'
```

Nota

There are places in the stdlib that check for an exact *str* instead of a *str* subclass (i.e. `type(unknown) == str` instead of `isinstance(unknown, str)`), and in those locations you will need to use `str(MyStrEnum.MY_MEMBER)`.

Nota

Using *auto* with *StrEnum* results in the lower-cased member name as the value.

Nota

`__str__()` is `str.__str__()` to better support the *replacement of existing constants* use-case. `__format__()` is likewise `str.__format__()` for that same reason.

Adicionado na versão 3.11.

class enum.**Flag**

`Flag` is the same as *Enum*, but its members support the bitwise operators `&` (*AND*), `|` (*OR*), `^` (*XOR*), and `~` (*INVERT*); the results of those operations are (aliases of) members of the enumeration.

__contains__(*self*, *value*)

Returns *True* if value is in self:

```
>>> from enum import Flag, auto
>>> class Color(Flag):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> purple = Color.RED | Color.BLUE
>>> white = Color.RED | Color.GREEN | Color.BLUE
>>> Color.GREEN in purple
False
>>> Color.GREEN in white
True
>>> purple in white
True
>>> white in purple
False
```

__iter__(self):

Returns all contained non-alias members:

```
>>> list(Color.RED)
[<Color.RED: 1>]
>>> list(purple)
[<Color.RED: 1>, <Color.BLUE: 4>]
```

Adicionado na versão 3.11.

__len__(self):

Returns number of members in flag:

```
>>> len(Color.GREEN)
1
>>> len(white)
3
```

Adicionado na versão 3.11.

__bool__(self):

Returns *True* if any members in flag, *False* otherwise:

```
>>> bool(Color.GREEN)
True
>>> bool(white)
True
>>> black = Color(0)
>>> bool(black)
False
```

__or__ (*self*, *other*)

Returns current flag binary or'ed with other:

```
>>> Color.RED | Color.GREEN
<Color.RED|GREEN: 3>
```

__and__ (*self*, *other*)

Returns current flag binary and'ed with other:

```
>>> purple & white
<Color.RED|BLUE: 5>
>>> purple & Color.GREEN
<Color: 0>
```

__xor__ (*self*, *other*)

Returns current flag binary xor'ed with other:

```
>>> purple ^ white
<Color.GREEN: 2>
>>> purple ^ Color.GREEN
<Color.RED|GREEN|BLUE: 7>
```

__invert__(self):

Returns all the flags in *type(self)* that are not in *self*:

```
>>> ~white
<Color: 0>
>>> ~purple
<Color.GREEN: 2>
>>> ~Color.RED
<Color.GREEN|BLUE: 6>
```

_numeric_repr_()

Function used to format any remaining unnamed numeric values. Default is the value's repr; common choices are *hex()* and *oct()*.

> **Nota**
>
> Using *auto* with *Flag* results in integers that are powers of two, starting with `1`.

Alterado na versão 3.11: The *repr()* of zero-valued flags has changed. It is now:

```
>>> Color(0)
<Color: 0>
```

class enum.**IntFlag**

`IntFlag` is the same as *Flag*, but its members are also integers and can be used anywhere that an integer can be used.

```
>>> from enum import IntFlag, auto
>>> class Color(IntFlag):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> Color.RED & 2
```

(continua na próxima página)

(continuação da página anterior)

```
<Color: 0>
>>> Color.RED | 2
<Color.RED|GREEN: 3>
```

If any integer operation is performed with an *IntFlag* member, the result is not an *IntFlag*:

```
>>> Color.RED + 2
3
```

If a *Flag* operation is performed with an *IntFlag* member and:

- the result is a valid *IntFlag*: an *IntFlag* is returned
- the result is not a valid *IntFlag*: the result depends on the *FlagBoundary* setting

The *repr()* of unnamed zero-valued flags has changed. It is now:

```
>>> Color(0)
<Color: 0>
```

Nota

Using *auto* with *IntFlag* results in integers that are powers of two, starting with `1`.

Alterado na versão 3.11: `__str__()` is now `int.__str__()` to better support the *replacement of existing constants* use-case. `__format__()` was already `int.__format__()` for that same reason.

Inversion of an `IntFlag` now returns a positive value that is the union of all flags not in the given flag, rather than a negative value. This matches the existing *Flag* behavior.

class enum.**ReprEnum**

`ReprEnum` uses the *repr()* of *Enum*, but the *str()* of the mixed-in data type:

- `int.__str__()` for *IntEnum* and *IntFlag*
- `str.__str__()` for *StrEnum*

Inherit from `ReprEnum` to keep the *str()* / *format()* of the mixed-in data type instead of using the *Enum*-default *str()*.

Adicionado na versão 3.11.

class enum.**EnumCheck**

EnumCheck contains the options used by the *verify()* decorator to ensure various constraints; failed constraints result in a *ValueError*.

UNIQUE

Ensure that each value has only one name:

```
>>> from enum import Enum, verify, UNIQUE
>>> @verify(UNIQUE)
... class Color(Enum):
...     RED = 1
...     GREEN = 2
...     BLUE = 3
...     CRIMSON = 1
Traceback (most recent call last):
...
ValueError: aliases found in <enum 'Color'>: CRIMSON -> RED
```

CONTINUOUS

Ensure that there are no missing values between the lowest-valued member and the highest-valued member:

```
>>> from enum import Enum, verify, CONTINUOUS
>>> @verify(CONTINUOUS)
... class Color(Enum):
...     RED = 1
...     GREEN = 2
...     BLUE = 5
Traceback (most recent call last):
...
ValueError: invalid enum 'Color': missing values 3, 4
```

NAMED_FLAGS

Ensure that any flag groups/masks contain only named flags – useful when values are specified instead of being generated by *auto()*:

```
>>> from enum import Flag, verify, NAMED_FLAGS
>>> @verify(NAMED_FLAGS)
... class Color(Flag):
...     RED = 1
...     GREEN = 2
...     BLUE = 4
...     WHITE = 15
...     NEON = 31
Traceback (most recent call last):
...
ValueError: invalid Flag 'Color': aliases WHITE and NEON are missing␣
↪combined values of 0x18 [use enum.show_flag_values(value) for details]
```

Nota

CONTINUOUS and NAMED_FLAGS are designed to work with integer-valued members.

Adicionado na versão 3.11.

class enum.**FlagBoundary**

`FlagBoundary` controls how out-of-range values are handled in *Flag* and its subclasses.

STRICT

Out-of-range values cause a *ValueError* to be raised. This is the default for *Flag*:

```
>>> from enum import Flag, STRICT, auto
>>> class StrictFlag(Flag, boundary=STRICT):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> StrictFlag(2**2 + 2**4)
Traceback (most recent call last):
...
ValueError: <flag 'StrictFlag'> invalid value 20
    given 0b0 10100
  allowed 0b0 00111
```

CONFORM

Out-of-range values have invalid values removed, leaving a valid *Flag* value:

```
>>> from enum import Flag, CONFORM, auto
>>> class ConformFlag(Flag, boundary=CONFORM):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> ConformFlag(2**2 + 2**4)
<ConformFlag.BLUE: 4>
```

EJECT

Out-of-range values lose their *Flag* membership and revert to *int*.

```
>>> from enum import Flag, EJECT, auto
>>> class EjectFlag(Flag, boundary=EJECT):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> EjectFlag(2**2 + 2**4)
20
```

KEEP

Out-of-range values are kept, and the *Flag* membership is kept. This is the default for *IntFlag*:

```
>>> from enum import Flag, KEEP, auto
>>> class KeepFlag(Flag, boundary=KEEP):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> KeepFlag(2**2 + 2**4)
<KeepFlag.BLUE|16: 20>
```

Adicionado na versão 3.11.

class enum.**EnumDict**

EnumDict is a subclass of *dict* that is used as the namespace for defining enum classes (see prepare). It is exposed to allow subclasses of *EnumType* with advanced behavior like having multiple values per member. It should be called with the name of the enum class being created, otherwise private names and internal classes will not be handled correctly.

Note that only the *MutableMapping* interface (`__setitem__()` and *update()*) is overridden. It may be possible to bypass the checks using other `dict` operations like `|=`.

member_names

A list of member names.

Adicionado na versão 3.13.

Nomes `__dunder__` suportados

__members__ is a read-only ordered mapping of `member_name:member` items. It is only available on the class.

__new__(), if specified, must create and return the enum members; it is also a very good idea to set the member's `_value_` appropriately. Once all the members are created it is no longer used.

Nomes _sunder_ suportados

- *_name_* – name of the member
- *_value_* – value of the member; can be set in __new__
- *_missing_()* – a lookup function used when a value is not found; may be overridden
- *_ignore_* – a list of names, either as a *list* or a *str*, that will not be transformed into members, and will be removed from the final class
- *_order_* – no longer used, kept for backward compatibility (class attribute, removed during class creation)
- *_generate_next_value_()* – used to get an appropriate value for an enum member; may be overridden

> **Nota**
>
> For standard *Enum* classes the next value chosen is the highest value seen incremented by one.
>
> For *Flag* classes the next value chosen will be the next highest power-of-two.

- *_add_alias_()* – adds a new name as an alias to an existing member.
- *_add_value_alias_()* – adds a new value as an alias to an existing member.
- While _sunder_ names are generally reserved for the further development of the *Enum* class and can not be used, some are explicitly allowed:
  - _repr_* (e.g. _repr_html_), as used in IPython's rich display

Adicionado na versão 3.6: _missing_, _order_, _generate_next_value_

Adicionado na versão 3.7: _ignore_

Adicionado na versão 3.13: _add_alias_, _add_value_alias_, _repr_*

8.14.3 Utilities and Decorators

class enum.**auto**

auto can be used in place of a value. If used, the *Enum* machinery will call an *Enum*'s *_generate_next_value_()* to get an appropriate value. For *Enum* and *IntEnum* that appropriate value will be the last value plus one; for *Flag* and *IntFlag* it will be the first power-of-two greater than the highest value; for *StrEnum* it will be the lower-cased version of the member's name. Care must be taken if mixing *auto()* with manually specified values.

auto instances are only resolved when at the top level of an assignment:

- FIRST = auto() will work (auto() is replaced with 1);
- SECOND = auto(), -2 will work (auto is replaced with 2, so 2, -2 is used to create the SECOND enum member;
- THREE = [auto(), -3] will *not* work (<auto instance>, -3 is used to create the THREE enum member)

Alterado na versão 3.11.1: In prior versions, auto() had to be the only thing on the assignment line to work properly.

_generate_next_value_ can be overridden to customize the values used by *auto*.

> **Nota**
>
> in 3.13 the default _generate_next_value_ will always return the highest member value incremented by 1, and will fail if any member is an incompatible type.

`@enum.`**`property`**

A decorator similar to the built-in *property*, but specifically for enumerations. It allows member attributes to have the same names as members themselves.

> **Nota**
>
> the *property* and the member must be defined in separate classes; for example, the *value* and *name* attributes are defined in the *Enum* class, and *Enum* subclasses can define members with the names `value` and `name`.

Adicionado na versão 3.11.

`@enum.`**`unique`**

A `class` decorator specifically for enumerations. It searches an enumeration's *__members__*, gathering any aliases it finds; if any are found *ValueError* is raised with the details:

```
>>> from enum import Enum, unique
>>> @unique
... class Errado(Enum):
...     UM = 1
...     DOIS = 2
...     TRES = 3
...     QUATRO = 3
...
Traceback (most recent call last):
...
ValueError: duplicate values found in <enum 'Errado'>: QUATRO -> TRES
```

`@enum.`**`verify`**

A `class` decorator specifically for enumerations. Members from *EnumCheck* are used to specify which constraints should be checked on the decorated enumeration.

Adicionado na versão 3.11.

`@enum.`**`member`**

A decorator for use in enums: its target will become a member.

Adicionado na versão 3.11.

`@enum.`**`nonmember`**

A decorator for use in enums: its target will not become a member.

Adicionado na versão 3.11.

`@enum.`**`global_enum`**

A decorator to change the *str()* and *repr()* of an enum to show its members as belonging to the module instead of its class. Should only be used when the enum members are exported to the module global namespace (see *re.RegexFlag* for an example).

Adicionado na versão 3.11.

`enum.`**`show_flag_values`**(*value*)

Return a list of all power-of-two integers contained in a flag *value*.

Adicionado na versão 3.11.

8.14.4 Notas

IntEnum, *StrEnum*, and *IntFlag*

These three enum types are designed to be drop-in replacements for existing integer- and string-based values; as such, they have extra limitations:

- `__str__` uses the value and not the name of the enum member
- `__format__`, because it uses `__str__`, will also use the value of the enum member instead of its name

If you do not need/want those limitations, you can either create your own base class by mixing in the `int` or `str` type yourself:

```
>>> from enum import Enum
>>> class MyIntEnum(int, Enum):
...     pass
```

or you can reassign the appropriate *str()*, etc., in your enum:

```
>>> from enum import Enum, IntEnum
>>> class MyIntEnum(IntEnum):
...     __str__ = Enum.__str__
```

8.15 `graphlib` — Funcionalidade para operar com estruturas do tipo grafo

Código-fonte: Lib/graphlib.py

---

class graphlib.**TopologicalSorter**(*graph=None*)

Fornece funcionalidade para classificar topologicamente um grafo de nós *hasheáveis*.

Uma ordem topológica é uma ordenação linear dos vértices em um grafo, de modo que para cada aresta direcionada u -> v do vértice u ao vértice v, o vértice u vem antes do vértice v na ordenação. Por exemplo, os vértices do grafo podem representar tarefas a serem executadas e as arestas podem representar restrições de que uma tarefa deve ser executada antes de outra; neste exemplo, uma ordem topológica é apenas uma sequência válida para as tarefas. Uma ordenação topológica completa é possível se e somente se o grafo não tiver ciclos direcionados, ou seja, se for um grafo acíclico direcionado.

Se o argumento opcional *graph* for fornecido, ele deverá ser um dicionário que represente um grafo acíclico direcionado no qual as chaves sejam nós e os valores sejam iteráveis de todos os predecessores desse nó no grafo (os nós que possuem bordas que apontam para o valor na chave). Nós adicionais podem ser adicionados ao grafo usando o método *add()*.

No caso geral, as etapas necessárias para executar a classificação de um determinado grafo são as seguintes:

- Cria uma instância da classe *TopologicalSorter* com um grafo inicial opcional.
- Adiciona nós adicionais ao grafo.
- Chama o método *prepare()* no grafo.
- Enquanto *is_active()* é `True`, itera pelos nós retornados por *get_ready()* e os processa. Chama *done()* em cada nó na medida em que finaliza o processamento.

Caso apenas uma classificação imediata dos nós no grafo seja necessária e nenhum paralelismo esteja envolvido, o método de conveniência *TopologicalSorter.static_order()* pode ser usado diretamente:

```
>>> graph = {"D": {"B", "C"}, "C": {"A"}, "B": {"A"}}
>>> ts = TopologicalSorter(graph)
>>> tuple(ts.static_order())
('A', 'C', 'B', 'D')
```

A classe foi projetada para suportar facilmente o processamento paralelo dos nós à medida que eles se tornam prontos. Por exemplo:

```
topological_sorter = TopologicalSorter()

# Adiciona nodos para 'topological_sorter'...

topological_sorter.prepare()
while topological_sorter.is_active():
    for node in topological_sorter.get_ready():
        # Processos ou threads de workers obtém nós para
        # trabalhar na fila 'task_queue'.
        task_queue.put(node)

    # Quando o trabalho para um nó é feito, os workers colocam o
    # nó em 'finalized_tasks_queue' para que possamos obter mais
    # nós para trabalhar. A definição de 'is_active()' garante que,
    # neste ponto, pelo menos um nó foi colocado em 'task_queue'
    # que ainda não foi passado para 'done()', então este 'get()' de
    # bloqueio deve (eventualmente) ter sucesso. Depois de chamar
    # 'done()', fazemos um laço de volta para chamar 'get_ready()'
    # novamente, então colocamos nós recém-liberados em
    # 'task_queue' o mais rápido possível.
    node = finalized_tasks_queue.get()
    topological_sorter.done(node)
```

add(*node*, **predecessors*)

Adiciona um novo nó e seus predecessores ao grafo. O *node* e todos os elementos em *predecessors* devem ser *hasheáveis*.

Se chamado várias vezes com o mesmo argumento do nó, o conjunto de dependências será a união de todas as dependências transmitidas.

É possível adicionar um nó sem dependências (*predecessors* não são fornecidos) ou fornecer uma dependência duas vezes. Se um nó que não foi fornecido anteriormente for incluído entre os *predecessors*, ele será automaticamente adicionado ao grafo sem predecessores próprios.

Levanta *ValueError* se chamado após *prepare()*.

prepare()

Marca o grafo como concluído e verifica os ciclos no grafo. Se qualquer ciclo for detectado, *CycleError* será gerado, mas *get_ready()* ainda poderá ser usado para obter o maior número possível de nós até que os ciclos bloqueiem mais progressos. Após uma chamada para esta função, o grafo não pode ser modificado e, portanto, nenhum nó pode ser adicionado usando *add()*.

Uma *ValueError* será levantada se a classificação tiver sido iniciada por *static_order()* ou *get_ready()*.

Alterado na versão 3.14: `prepare()` agora pode ser chamado mais de uma vez, desde que a classificação não tenha sido iniciada. Anteriormente, isso levantava *ValueError*.

is_active()

Retorna `True` se mais progresso puder ser feito e `False` caso contrário. É possível progredir se os ciclos não bloquearem a resolução e ainda houver nós prontos que ainda não foram retornados por *TopologicalSorter.get_ready()* ou o número de nós marcados *TopologicalSorter.done()* é menor que o número retornado por *TopologicalSorter.get_ready()*.

O método `__bool__()` desta classe adia para essa função, então, em vez de:

```
if ts.is_active():
    ...
```

é possível simplesmente fazer:

```
if ts:
    ...
```

Levanta *ValueError* se chamado sem chamar *prepare()* anteriormente.

done(**nodes*)

Marca um conjunto de nós retornados por *TopologicalSorter.get_ready()* como processado, desbloqueando qualquer sucessor de cada nó em *nodes* para ser retornado no futuro por uma chamada para *TopologicalSorter.get_ready()*.

Levanta *ValueError* se algum nó em *nodes* já foi marcado como processado por uma chamada anterior a este método ou se um nó não foi adicionado ao grafo usando *TopologicalSorter.add()*, se chamado sem chamar `prepare()` ou se o nó ainda não foi retornado por *get_ready()*.

get_ready()

Retorna uma `tupla` com todos os nós que estão prontos. Inicialmente, ele retorna todos os nós sem predecessores e, uma vez marcados como processados, chamando *TopologicalSorter.done()*, novas chamadas retornarão todos os novos nós que já tenham seus antecessores já processados. Quando não for possível fazer mais progresso, as tuplas vazias serão retornadas.

Levanta *ValueError* se chamado sem chamar *prepare()* anteriormente.

static_order()

Retorna um objeto iterador que irá iterar sobre os nós em uma ordem topológica. Ao usar este método, *prepare()* e *done()* não devem ser chamados. Este método é equivalente a:

```
def static_order(self):
    self.prepare()
    while self.is_active():
        node_group = self.get_ready()
        yield from node_group
        self.done(*node_group)
```

A ordem específica retornada pode depender da ordem específica em que os itens foram inseridos no grafo. Por exemplo:

```
>>> ts = TopologicalSorter()
>>> ts.add(3, 2, 1)
>>> ts.add(1, 0)
>>> print([*ts.static_order()])
[2, 0, 1, 3]

>>> ts2 = TopologicalSorter()
>>> ts2.add(1, 0)
>>> ts2.add(3, 2, 1)
>>> print([*ts2.static_order()])
[0, 2, 1, 3]
```

Isso se deve ao fato de que “0” e “2” estão no mesmo nível no grafo (eles teriam sido retornados na mesma chamada para *get_ready()*) e a ordem entre eles é determinada pela ordem de inserção.

Se qualquer ciclo for detectado, *CycleError* será levantada.

Adicionado na versão 3.9.

8.15.1 Exceções

O módulo *graphlib* define as seguintes classes de exceção:

exception graphlib.**CycleError**

Subclasse de *ValueError* levantada por *TopologicalSorter.prepare()* se houver ciclos no grafo de trabalho. Se existirem vários ciclos, apenas uma opção indefinida entre eles será relatada e incluída na exceção.

O ciclo detectado pode ser acessado através do segundo elemento no atributo *args* da instância de exceção e consiste em uma lista de nós, de modo que cada nó seja, no grafo, um predecessor imediato do próximo nó na lista. Na lista relatada, o primeiro e o último nó serão os mesmos, para deixar claro que é cíclico.

CAPÍTULO 9

Módulos Matemáticos e Numéricos

Os módulos descritos neste capítulo fornecem funções e tipos de dados relacionados com o numérico e matemática. O módulo *numbers* define uma hierarquia abstrata de tipos numéricos. Os módulos *math* e *cmath* contêm várias funções matemáticas para números de ponto flutuante e números complexos. O módulo *decimal* suporta representações exatas de números decimais, usando aritmética de precisão arbitrária.

Os seguintes módulos estão documentados neste capítulo:

9.1 numbers — Classes base abstratas numéricas

Código-fonte: Lib/numbers.py

O módulo numbers (**PEP 3141**) define uma hierarquia de *classes base abstratas* que progressivamente definem mais operações. Nenhum dos tipos definidos neste módulo se destina a ser instanciado.

class numbers.**Number**

A raiz da hierarquia numérica. Se você quiser apenas verificar se um argumento *x* é um número, sem se importar com o tipo, use `isinstance(x, Number)`.

9.1.1 A torre numérica

class numbers.**Complex**

As subclasses deste tipo descrevem números complexos e incluem as operações que funcionam no tipo embutido *complex*. Elas são: conversões para *complex* e *bool*, *real*, *imag*, `+`, `-`, `*`, `/`, `**`, *abs()*, *conjugate()*, `==` e `!=`. Todos exceto `-` e `!=` são abstratos.

real

Abstrata. Obtém o componente real deste número.

imag

Abstrata. Obtém o componente imaginário deste número.

abstractmethod conjugate()

Abstrata. Retorna o conjugado complexo. Por exemplo, `(1+3j).conjugate() == (1-3j)`.

class numbers.**Real**

Para *Complex*, Real adiciona as operações que funcionam em números reais.

Em suma, são: uma conversão para *float*, *math.trunc()*, *round()*, *math.floor()*, *math.ceil()*, *divmod()*, //, %, <, <=, > e >=.

Real também fornece padrão para *complex()*, *real*, *imag* e *conjugate()*.

class numbers.**Rational**

Estende *Real* e adiciona as propriedades *numerator* e *denominator*. Ele fornece um padrão para *float()*.

Os valores *numerator* e *denominator* devem ser instâncias de *Integral* e devem estar nos termos mais baixos com *denominator* positivo.

numerator

Abstrata.

denominator

Abstrata.

class numbers.**Integral**

Estende *Rational* e adiciona uma conversão para *int*. Fornece padrões para *float()*, *numerator* e *denominator*. Adiciona métodos abstratos para *pow()* com operações de módulo e de string de bits: <<, >>, &, ^, |, ~.

9.1.2 Notas para implementadores de tipos

Os implementadores devem ter o cuidado de tornar iguais números iguais e fazer hash deles com os mesmos valores. Isso pode ser sutil se houver duas extensões diferentes dos números reais. Por exemplo, *fractions.Fraction* implementa *hash()* desta forma:

```
def __hash__(self):
    if self.denominator == 1:
        # Obtém inteiros corretamente.
        return hash(self.numerator)
    # Verificação custosa, mas certamente correta.
    if self == float(self):
        return hash(float(self))
    else:
        # Use o hash da tupla para evitar uma alta taxa de colisão
        # em frações simples.
        return hash((self.numerator, self.denominator))
```

Adicionando mais ABCs numéricas

Existem, é claro, mais ABCs possíveis para números, e isso seria uma hierarquia pobre se excluísse a possibilidade de adicioná-los. Você pode adicionar MyFoo entre *Complex* e *Real* com:

```
class MyFoo(Complex): ...
MyFoo.register(Real)
```

Implementando as operações aritméticas

Queremos implementar as operações aritméticas de forma que as operações de modo misto chamem uma implementação cujo autor conhecia os tipos de ambos os argumentos ou convertam ambos para o tipo embutido mais próximo e façam a operação lá. Para subtipos de *Integral*, isso significa que __add__() e __radd__() devem ser definidos com:

```
class MyIntegral(Integral):

    def __add__(self, other):
        if isinstance(other, MyIntegral):
            return do_my_adding_stuff(self, other)
        elif isinstance(other, OtherTypeIKnowAbout):
            return do_my_other_adding_stuff(self, other)
        else:
            return NotImplemented

    def __radd__(self, other):
        if isinstance(other, MyIntegral):
            return do_my_adding_stuff(other, self)
        elif isinstance(other, OtherTypeIKnowAbout):
            return do_my_other_adding_stuff(other, self)
        elif isinstance(other, Integral):
            return int(other) + int(self)
        elif isinstance(other, Real):
            return float(other) + float(self)
        elif isinstance(other, Complex):
            return complex(other) + complex(self)
        else:
            return NotImplemented
```

Existem 5 casos diferentes para uma operação de tipo misto em subclasses de *Complex*. Vou me referir a todo o código acima que não se refere a `MyIntegral` e `OtherTypeIKnowAbout` com um "modelo". `a` será uma instância de `A`, que é um subtipo de *Complex* (`a : A <: Complex`) e `b : B <: Complex`. Vou considerar `a + b`:

1. Se `A` define um `__add__()`, que aceita `b`, está tudo bem.
2. Se `A` voltar ao código padrão e tivesse que retornar um valor de `__add__()`, perderíamos a possibilidade de que `B` definisse um `__radd__()` mais inteligente, então o código padrão deve retornar *NotImplemented* de `__add__()`. (Ou `A` pode não implementar `__add__()`.)
3. Então, `__radd__()` do `B` consegue uma chance. Se ele aceitar `a`, está tudo bem.
4. Se ele recorrer ao padrão, não há mais métodos possíveis para tentar, então é aqui que a implementação padrão deve residir.
5. Se `B <: A`, Python tenta `B.__radd__` antes de `A.__add__`. Isso está ok, porque foi implementado com conhecimento de `A`, então ele pode lidar com essas instâncias antes de delegar para *Complex*.

Se `A <: Complex` e `B <: Real` sem compartilhar nenhum outro conhecimento, então a operação compartilhada apropriada é aquela envolvendo a *complex* embutida, e `__radd__()` de ambos chegam lá, de forma que `a+b == b+a`.

Como a maioria das operações em qualquer tipo será muito semelhante, pode ser útil definir uma função auxiliar que gera as instâncias de avanço e reversão de qualquer operador. Por exemplo, *fractions.Fraction* usa:

```
def _operator_fallbacks(monomorphic_operator, fallback_operator):
    def forward(a, b):
        if isinstance(b, (int, Fraction)):
            return monomorphic_operator(a, b)
        elif isinstance(b, float):
            return fallback_operator(float(a), b)
        elif isinstance(b, complex):
            return fallback_operator(complex(a), b)
        else:
            return NotImplemented
    forward.__name__ = '__' + fallback_operator.__name__ + '__'
```

(continua na próxima página)

(continuação da página anterior)

```
    forward.__doc__ = monomorphic_operator.__doc__

    def reverse(b, a):
        if isinstance(a, Rational):
            # Inclui ints.
            return monomorphic_operator(a, b)
        elif isinstance(a, Real):
            return fallback_operator(float(a), float(b))
        elif isinstance(a, Complex):
            return fallback_operator(complex(a), complex(b))
        else:
            return NotImplemented
    reverse.__name__ = '__r' + fallback_operator.__name__ + '__'
    reverse.__doc__ = monomorphic_operator.__doc__

    return forward, reverse

def _add(a, b):
    """a + b"""
    return Fraction(a.numerator * b.denominator +
                    b.numerator * a.denominator,
                    a.denominator * b.denominator)

__add__, __radd__ = _operator_fallbacks(_add, operator.add)

# ...
```

9.2 `math` — Funções matemáticas

Este módulo fornece acesso à constantes e funções matemáticas comuns, incluindo aquelas definidas pelo padrão C.

Essas funções não podem ser usadas com números complexos; use as funções de mesmo nome do módulo *cmath* se você precisar de suporte para números complexos. A distinção entre funções que suportam números complexos e aquelas que não suportam é feita uma vez que a maioria dos usuários não quer aprender a matemática necessária para entender números complexos. Receber uma exceção em vez de um resultado complexo permite a detecção antecipada do número complexo inesperado usado como parâmetro, para que o programador possa determinar como e por que ele foi gerado em primeiro lugar.

As funções a seguir são fornecidas por este módulo. Exceto quando explicitamente indicado de outra forma, todos os valores de retorno são pontos flutuantes.

Funções de teoria dos números	
comb(n, k)	Número de maneiras de escolher *k* itens a partir de *n* itens sem repetição e
factorial(n)	Fatorial de *n*
*gcd(*integers)*	Máximo divisor comum dos argumentos inteiros
isqrt(n)	Raiz quadrada inteira de um inteiro não negativo *n*
*lcm(*integers)*	Mínimo múltiplo comum dos argumentos inteiros
perm(n, k)	Número de maneiras de escolher *k* itens de *n* itens sem repetição e com or
Arimética de ponto flutuante	
ceil(x)	Teto de *x*, o menor número inteiro maior ou igual a *x*
fabs(x)	Valor absoluto de *x*
floor(x)	Piso de *x*, o maior inteiro menor ou igual a *x*
fma(x, y, z)	Operação de multiplicação e adição combinadas: `(x * y) + z`

continua na próxi

Tabela 1 – continuação da página anterior

`fmod(x, y)`	Resto da divisão `x / y`
`modf(x)`	Partes fracionárias e inteiras de *x*
`remainder(x, y)`	Resto de *x* em relação a *y*
`trunc(x)`	Parte inteira de *x*
Funções de manipulação de ponto flutuante	
`copysign(x, y)`	Magnitude (valor absoluto) de *x* com o sinal de *y*
`frexp(x)`	Mantissa e expoente de *x*
`isclose(a, b, rel_tol, abs_tol)`	Verifica se os valores *a* e *b* estão próximos um do outro
`isfinite(x)`	Verifica se *x* não é infinito e nem um NaN
`isinf(x)`	Verifica se *x* é um infinito positivo ou negativo
`isnan(x)`	Verifica se *x* é um NaN (não um número)
`ldexp(x, i)`	`x * (2**i)`, inverso da função `frexp()`
`nextafter(x, y, steps)`	Valor de ponto flutuante de *steps* passos depois de *x* em direção a *y*
`ulp(x)`	Valor do bit menos significativo de *x*
Funções de potências, exponenciais e logarítmicas	
`cbrt(x)`	Raiz cúbica de *x*
`exp(x)`	*e* elevado à potência *x*
`exp2(x)`	*2* elevado a potência de *x*
`expm1(x)`	*e* elevado a potência de *x*, menos 1
`log(x, base)`	Logaritmo de *x* para a base fornecida (*e* por padrão)
`log1p(x)`	Logaritmo natural de *1+x* (base *e*)
`log2(x)`	Logaritmo de base 2 de *x*
`log10(x)`	Logaritmo de base 10 de *x*
`pow(x, y)`	*x* elevado à potência *y*
`sqrt(x)`	Raiz quadrada de *x*
Funções de soma e produto	
`dist(p, q)`	Distância euclidiana entre dois pontos *p* e *q* dada como um iterável de coor
`fsum(iterable)`	Soma dos valores na entrada *iterable*
`hypot(*coordinates)`	Norma euclidiana de um iterável de coordenadas
`prod(iterable, start)`	Produto de elementos da entrada *iterable* com um valor *start*
`sumprod(p, q)`	Soma dos produtos de dois iteráveis *p* e *q*
Conversão angular	
`degrees(x)`	Converte o ângulo *x* de radianos para graus
`radians(x)`	Converte o ângulo *x* de graus para radianos
Funções trigonométricas	
`acos(x)`	Arco cosseno de *x*
`asin(x)`	Arco seno de *x*/
`atan(x)`	Arco tangente de *x*
`atan2(y, x)`	`atan(y / x)`
`cos(x)`	Cosseno de *x*
`sin(x)`	Seno de *x*
`tan(x)`	Tangente de *x*
Funções hiperbólicas	
`acosh(x)`	Cosseno hiperbólico inverso de *x*
`asinh(x)`	Seno hiperbólico inverso de *x*
`atanh(x)`	Tangente hiperbólico inverso de *x*
`cosh(x)`	Cosseno hiperbólico de *x*
`sinh(x)`	Seno hiperbólico de *x*
`tanh(x)`	Tangente hiperbólico de *x*
Funções especiais	
`erf(x)`	Função erro em *x*
`erfc(x)`	Função erro complementar em *x*
`gamma(x)`	Função gama em *x*
`lgamma(x)`	Logaritmo natural do valor absoluto da função gama em *x*
Constantes	
`pi`	π = 3.141592…

continua na próxi

Tabela 1 – continuação da página anterior

e	$e = 2.718281\ldots$
tau	$\tau = 2\pi = 6.283185\ldots$
inf	Infinito positivo
nan	"Not a number" (NaN)

9.2.1 Funções de teoria dos números

math.**comb**(*n*, *k*)

Retorna o número de maneiras de escolher *k* itens de *n* itens sem repetição e sem ordem.

Avalia para `n! / (k! * (n - k)!)` quando `k <= n` e avalia para zero quando `k > n`.

Também conhecido como coeficiente binomial pois equivale ao coeficiente do k-ésimo termo na expansão polinomial `(1 + x)**n`

Levanta *TypeError* se algum dos argumentos não for inteiro. Levanta *ValueError* se algum dos argumentos for negativo.

Adicionado na versão 3.8.

math.**factorial**(*n*)

Retorna fatorial do inteiro não negativo *n*.

Alterado na versão 3.10: Números de ponto flutuate com valor integral (como `5.0`) não são mais aceitos.

math.**gcd**(**integers*)

Retorna o maior divisor comum dos argumentos inteiros especificados. Se algum dos argumentos for diferente de zero, o valor retornado será o maior inteiro positivo que é um divisor de todos os argumentos. Se todos os argumentos forem zero, o valor retornado será `0`. `gcd()` sem argumentos retorna `0`.

Adicionado na versão 3.5.

Alterado na versão 3.9: Adicionado suporte para um número arbitrário de argumentos. Anteriormente, apenas dois argumentos eram suportados.

math.**isqrt**(*n*)

Retorna a raiz quadrada inteira do inteiro não negativo *n*. Este é o piso da raiz quadrada exata de *n*, ou equivalentemente o maior inteiro *a* tal que $a^2 \le n$.

Para algumas aplicações, pode ser mais conveniente ter o menor número inteiro *a* tal que $n \le a^2$ ou, em outras palavras, o teto da raiz quadrada exata de *n*. Para *n* positivo, isso pode ser calculado usando `a = 1 + isqrt(n - 1)`.

Adicionado na versão 3.8.

math.**lcm**(**integers*)

Retorna o mínimo múltiplo comum dos argumentos inteiros especificados. Se todos os argumentos forem diferentes de zero, o valor retornado será o menor inteiro positivo que é um múltiplo de todos os argumentos. Se algum dos argumentos for zero, o valor retornado será `0`. `lcm()` sem argumentos retorna `1`.

Adicionado na versão 3.9.

math.**perm**(*n*, *k=None*)

Retorna o número de maneiras de escolher *k* itens de *n* itens sem repetição e com ordem.

Avalia para `n! / (n - k)!` quando `k <= n` e avalia para zero quando `k > n`.

Se *k* não for especificado ou for `None`, *k* usará o padrão *n* e a função retornará `n!`.

Levanta *TypeError* se algum dos argumentos não for inteiro. Levanta *ValueError* se algum dos argumentos for negativo.

Adicionado na versão 3.8.

9.2.2 Aritmética de ponto flutuante

math.**ceil**(*x*)

Retorna o teto de *x*, o menor inteiro maior ou igual que *x*. Se *x* não é um float, delega para `x.__ceil__`, que deve retornar um valor do tipo *Integral*.

math.**fabs**(*x*)

Retorna o valor absoluto de *x*.

math.**floor**(*x*)

Retorna o chão de *x*, o maior inteiro menor ou igual a *x*. Se *x* não é um ponto flutuante, delega para `x.__floor__`, que deve retornar um valor do tipo *Integral*

math.**fma**(*x*, *y*, *z*)

Operação de multiplicação-adição combinada. Retorna `(x * y) + z`, calculado de forma a obter o mesmo resultado que seria obtido se o cálculo desfrutasse de precisão e limites infinitos, seguido de um único arredondamento para o formato `float`. Esta operação frequentemente oferece acurácia melhor do que a expressão direta `(x * y) + z` ofereceria.

Esta função segue a especificação da operação fusedMultiplyAdd descrita no padrão IEEE 754. O padrão deixa um caso a ser definido pela implementação, que são os resultados de `fma(0, inf, nan)` e `fma(inf, 0, nan)`. Nestes casos, `math.fma` retorna NaN, e não levanta nenhuma exceção.

Adicionado na versão 3.13.

math.**fmod**(*x*, *y*)

Retorna o resto ponto flutuante de `x / y`, conforme definido pela função `fmod(x, y)` da biblioteca C da plataforma. Observe que a expressão Python `x % y` pode não retornar o mesmo resultado. A intenção do padrão C é que `fmod(x, y)` seja exatamente (matematicamente; com precisão infinita) igual a `x - n*y` para algum inteiro *n* de modo que o resultado tenha o mesmo sinal que *x* e magnitude menor que `abs(y)`. O `x % y` do Python retorna um resultado com o sinal de *y*, e pode não ser exatamente computável para argumentos de ponto flutuante. Por exemplo, `fmod(-1e-100, 1e100)` é `-1e-100`, mas o resultado de `-1e-100 % 1e100` do Python é `1e100-1e-100`, que não pode ser representado exatamente como um ponto flutuante, e é arredondado para o surpreendente `1e100`. Por esta razão, a função *fmod()* é geralmente preferida ao trabalhar com pontos flutuantes, enquanto o `x % y` do Python é preferido ao trabalhar com inteiros.

math.**modf**(*x*)

Retorna as partes fracionárias e inteiras de *x*. Ambos os resultados carregam o sinal de *x* e são pontos flutuantes.

Observe que *modf()* tem um padrão de chamada/retorno diferente de seu equivalente C: elas pegam um único argumento e retornam um par de valores, ao invés de retornar seu segundo valor de retorno por meio de um "parâmetro de saída" (não existe tal coisa em Python).

math.**remainder**(*x*, *y*)

Retorna o resto no estilo IEEE 754 de *x* em relação a *y*. Para o finito *x* e o finito diferente de zero *y*, esta é a diferença `x - n*y`, onde `n` é o número inteiro mais próximo do valor exato do quociente `x / y`. Se `x / y` está exatamente no meio do caminho entre dois inteiros consecutivos, o inteiro *par* mais próximo é usado para `n`. O resto `r = remainder(x, y)` assim sempre satisfaz `abs(r) <= 0.5 * abs(y)`.

Casos especiais seguem IEEE 754: em particular, `remainder(x, math.inf)` é *x* para qualquer *x* finito, e `remainder(x, 0)` e `remainder(math.inf, x)` levantam *ValueError* para qualquer *x* não NaN. Se o resultado da operação `remainder` for zero, esse zero terá o mesmo sinal de *x*.

Em plataformas que usam ponto flutuante binário do IEEE 754, o resultado dessa operação é sempre exatamente representável: nenhum erro de arredondamento é introduzido.

Adicionado na versão 3.7.

math.**trunc**(*x*)

Retorna *x* com a parte fracionária removida, deixando a parte inteira. Isso arredonda para 0: `trunc()` é equivalente a *floor()* para *x* positivos, e equivalentes a *ceil()* para *x* negativos. Se *x* não é um ponto flutuante, então delega para `x.__trunc__`, cujo qual deve retornar um valor do tipo *Integral*.

Para as funções *ceil()*, *floor()* e *modf()*, observe que *todos* os números de ponto flutuante de magnitude suficientemente grande são inteiros exatos. Os pontos flutuantes do Python normalmente não carregam mais do que 53 bits de precisão (o mesmo que o tipo duplo da plataforma C), caso em que qualquer ponto flutuante *x* com `abs(x) >= 2**52` necessariamente não tem bits fracionários.

9.2.3 Funções de manipulação de ponto flutuante

`math.`**`copysign`**(*x*, *y*)

Retorna um ponto flutuante com a magnitude (valor absoluto) de *x*, mas o sinal de *y*. Em plataformas que suportam zeros com sinal, `copysign(1.0, -0.0)` retorna *-1.0*.

`math.`**`frexp`**(*x*)

Retorna a mantissa e o expoente de *x* como o par `(m, e)`. *m* é um ponto flutuante e *e* é um inteiro tal que `x == m * 2**e` exatamente. Se *x* for zero, retorna `(0.0, 0)`, caso contrário, `0.5 <= abs(m) < 1`. Isso é usado para "separar" a representação interna de um ponto flutuante de forma portátil.

Observe que *frexp()* tem um padrão de chamada/retorno diferente de seu equivalente C: elas pegam um único argumento e retornam um par de valores, ao invés de retornar seu segundo valor de retorno por meio de um "parâmetro de saída" (não existe tal coisa em Python).

`math.`**`isclose`**(*a*, *b*, ***, *rel_tol=1e-09*, *abs_tol=0.0*)

Retorna `True` se os valores *a* e *b* estiverem próximos e `False` caso contrário.

Se dois valores são considerados próximos ou não é determinado de acordo com tolerâncias absolutas e relativas fornecidas. Se nenhum erro ocorrer, o resultado será: `abs(a-b) <= max(rel_tol * max(abs(a), abs(b)), abs_tol)`.

rel_tol é a tolerância relativa – é a diferença máxima permitida entre *a* e *b*, em relação ao maior valor absoluto de *a* ou *b*. Por exemplo, para definir uma tolerância de 5%, passe `rel_tol=0.05`. A tolerância padrão é `1e-09`, o que garante que os dois valores sejam iguais em cerca de 9 dígitos decimais. *rel_tol* deve ser não negativo e menor que `1.0`.

abs_tol é a tolerância absoluta; o padrão é `0.0` e deve ser não negativo. Ao comparar `x` com `0.0`, `isclose(x, 0)` é computado como `abs(x) <= rel_tol * abs(x)`, que é `False` para qualquer `x` não zero e *rel_tol* menor que `1.0`. Então adicione um argumento *abs_tol* positivo apropriado à chamada.

Os valores especiais do IEEE 754 de `NaN`, `inf` e `-inf` serão tratados de acordo com as regras do IEEE. Especificamente, `NaN` não é considerado próximo a qualquer outro valor, incluindo `NaN`. `inf` e `-inf` são considerados apenas próximos a si mesmos.

Adicionado na versão 3.5.

> **Ver também**
>
> **PEP 485** – Uma função para testar igualdade aproximada

`math.`**`isfinite`**(*x*)

Retorna `True` se *x* não for um infinito nem um NaN, e `False` caso contrário. (Observe que `0.0` *é* considerado finito.)

Adicionado na versão 3.2.

`math.`**`isinf`**(*x*)

Retorna `True` se *x* for um infinito positivo ou negativo, e `False` caso contrário.

`math.`**`isnan`**(*x*)

Retorna `True` se *x* for um NaN (não um número), e `False` caso contrário.

`math.`**`ldexp`**(*x*, *i*)

Retorna `x * (2**i)`. Este é essencialmente o inverso da função *frexp()*.

math.**nextafter**(*x*, *y*, *steps=1*)

Retorna o valor de ponto flutuante com *steps* passos após *x* em direção a *y*.

Se *x* for igual a *y*, retorna *y*, a menos que *steps* seja zero.

Exemplos:

- `math.nextafter(x, math.inf)` sobe: em direção ao infinito positivo.
- `math.nextafter(x, -math.inf)` desce: em direção ao menos infinito.
- `math.nextafter(x, 0.0)` vai em direção a zero.
- `math.nextafter(x, math.copysign(math.inf, x))` se afasta do zero.

Veja também `math.ulp()`

Adicionado na versão 3.9.

Alterado na versão 3.12: Adicionado o argumento *steps*.

math.**ulp**(*x*)

Retorna o valor do bit menos significativo do ponto flutuante *x*:

- Se *x* for um NaN (não um número), retorna *x*.
- Se *x* for negativo, retorna `ulp(-x)`.
- Se *x* for um infinito positivo, retorna *x*.
- Se *x* for igual a zero, retorna o menor valor flutuante positivo *desnormalizado* representável (menor que o ponto flutuante de valor mínimo positivo *normalizado*, *sys.float_info.min*).
- Se *x* for igual ao maior ponto flutuante positivo representável, retorna o valor do bit menos significativo de *x*, tal que o primeiro ponto flutuante menor que *x* seja `x - ulp(x)`.
- Caso contrário (*x* é um número finito positivo), retorna o valor do bit menos significativo de *x*, de modo que o primeiro ponto flutuante maior que *x* seja `x + ulp(x)`.

ULP significa "Unit in the Last Place" ou, em português, unidade na última posição.

Veja também *math.nextafter()* e *sys.float_info.epsilon*.

Adicionado na versão 3.9.

9.2.4 Funções de potências, exponenciais e logarítmicas

math.**cbrt**(*x*)

Retorna a raiz cúbica de *x*.

Adicionado na versão 3.11.

math.**exp**(*x*)

Retorna *e* elevado à potência *x*, onde *e* = 2.718281… é a base dos logaritmos naturais. Isso geralmente é mais preciso do que `math.e ** x` ou `pow(math.e, x)`.

math.**exp2**(*x*)

Retorna *2* elevado a *x*

Adicionado na versão 3.11.

math.**expm1**(*x*)

Retorna *e* elevado à potência *x*, menos 1. Aqui *e* é a base dos logaritmos naturais. Para pequenos pontos flutuantes *x*, a subtração em `exp(x) - 1` pode resultar em uma perda significativa de precisão; a função *expm1()* fornece uma maneira de calcular essa quantidade com precisão total:

```
>>> from math import exp, expm1
>>> exp(1e-5) - 1  # gives result accurate to 11 places
1.0000050000069649e-05
>>> expm1(1e-5)    # result accurate to full precision
1.0000050000166668e-05
```

Adicionado na versão 3.2.

math.**log**(*x*[, *base*])

Com um argumento, retorna o logaritmo natural de *x* (para base *e*).

Com dois argumentos, retorna o logaritmo de *x* para a *base* fornecida, calculada como `log(x)/log(base)`.

math.**log1p**(*x*)

Retorna o logaritmo natural de *1+x* (base *e*). O resultado é calculado de forma precisa para *x* próximo a zero.

math.**log2**(*x*)

Retorna o logaritmo de base 2 de *x*. Isso geralmente é mais preciso do que `log(x, 2)`.

Adicionado na versão 3.3.

Ver também

`int.bit_length()` retorna o número de bits necessários para representar um inteiro em binário, excluindo o sinal e os zeros à esquerda.

math.**log10**(*x*)

Retorna o logaritmo de base 10 de *x*. Isso geralmente é mais preciso do que `log(x, 10)`.

math.**pow**(*x*, *y*)

Retorna *x* elevado a potência de *y*. Exceções seguem o padrão IEEE 754 o máximo possível. `pow(1.0, x)` e `pow(x, 0.0)` em particular sempre retornam `1.0`, mesmo quando *x* é ZERO ou NaN. Se ambos *x* e *y* são números finitos, *x* é negativo, e *y* não é um inteiro então `pow(x, y)` é indefinido e levanta *`ValueError`*.

Ao contrário do operador embutido `**`, *`math.pow()`* converte ambos os seus argumentos para o tipo *`float`*. Use `**` ou a função embutida *`pow()`* para calcular potências inteiras exatas.

Alterado na versão 3.11: Os casos especiais `pow(0.0, -inf)` e `pow(-0.0, -inf)` foram alterados para retornar `inf` ao invés de levantar *`ValueError`*, para ter consistência com a IEEE 754.

math.**sqrt**(*x*)

Retorna a raiz quadrada de *x*.

9.2.5 Funções de soma e produto

math.**dist**(*p*, *q*)

Retorna a distância euclidiana entre dois pontos *p* e *q*, cada um dado como uma sequência (ou iterável) de coordenadas. Os dois pontos devem ter a mesma dimensão.

Aproximadamente equivalente a:

```
sqrt(sum((px - qx) ** 2.0 for px, qx in zip(p, q)))
```

Adicionado na versão 3.8.

math.**fsum**(*iterable*)

Retorna uma soma precisa dos valores de ponto flutuante no iterável. Evita perda de precisão rastreando várias somas parciais intermediárias.

A precisão do algoritmo depende das garantias aritméticas do IEEE-754 e do caso típico em que o modo de arredondamento é meio par. Em algumas compilações que não são do Windows, a biblioteca C subjacente usa

adição de precisão estendida e pode ocasionalmente arredondar uma soma intermediária fazendo com que ela introduza um erro no bit menos significativo.

Para uma discussão mais aprofundada e duas abordagens alternativas, consulte o ASPN cookbook recipes for accurate floating point summation.

`math.`**`hypot`**`(`*`*coordinates`*`)`

Retorna a norma euclidiana, `sqrt(sum(x**2 for x in coordinates))`. Este é o comprimento do vetor da origem até o ponto dado pelas coordenadas.

Para um ponto bidimensional `(x, y)`, isso é equivalente a calcular a hipotenusa de um triângulo retângulo usando o teorema de Pitágoras, `sqrt(x*x + y*y)`.

Alterado na versão 3.8: Adicionado suporte para pontos n-dimensionais. Anteriormente, apenas o caso bidimensional era suportado.

Alterado na versão 3.10: Melhorou a precisão do algoritmo para que o erro máximo seja inferior a 1 ulp (unidade no último lugar). Mais tipicamente, o resultado é quase sempre arredondado corretamente para 1/2 ulp.

`math.`**`prod`**`(`*iterable*, *, *start=1*`)`

Calcula o produto de todos os elementos na entrada *iterable*. O valor *start* padrão para o produto é `1`.

Quando o iterável estiver vazio, retorna o valor de start. Esta função deve ser usada especificamente com valores numéricos e pode rejeitar tipos não numéricos.

Adicionado na versão 3.8.

`math.`**`sumprod`**`(`*p*, *q*`)`

Retorna a soma dos produtos dos valores de dois iteráveis *p* e *q*.

Levanta *`ValueError`* se as entradas não tiverem o mesmo comprimento.

Aproximadamente equivalente a:

```
sum(map(operator.mul, p, q, strict=True))
```

Para entradas float e mistas int/float, os produtos intermediários e as somas são calculados com precisão estendida.

Adicionado na versão 3.12.

9.2.6 Conversão angular

`math.`**`degrees`**`(`*x*`)`

Converte o ângulo *x* de radianos para graus.

`math.`**`radians`**`(`*x*`)`

Converte o ângulo *x* de graus para radianos.

9.2.7 Funções trigonométricas

`math.`**`acos`**`(`*x*`)`

Retorna o arco cosseno de *x*, em radianos. O resultado está entre `0` e `pi`.

`math.`**`asin`**`(`*x*`)`

Retorna o arco seno de *x*, em radianos. O resultado está entre `-pi/2` e `pi/2`.

`math.`**`atan`**`(`*x*`)`

Retorna o arco tangente de *x*, em radianos. O resultado está entre `-pi/2` e `pi/2`.

math.**atan2**(*y*, *x*)

Retorna `atan(y / x)`, em radianos. O resultado está entre `-pi` e `pi`. O vetor no plano da origem ao ponto `(x, y)` faz este ângulo com o eixo X positivo. O ponto de *atan2()* é que os sinais de ambas as entradas são conhecidos por ele, então ele pode calcular o quadrante correto para o ângulo. Por exemplo, `atan(1)` e `atan2(1, 1)` são ambos `pi/4`, mas `atan2(-1, -1)` é `-3*pi/4`.

math.**cos**(*x*)

Retorna o cosseno de *x* radianos.

math.**sin**(*x*)

Retorna o seno de *x* radianos.

math.**tan**(*x*)

Retorna o tangente de *x* radianos.

9.2.8 Funções hiperbólicas

Funções hiperbólicas são análogas às funções trigonométricas baseadas em hipérboles em vez de círculos.

math.**acosh**(*x*)

Retorna o cosseno hiperbólico inverso de *x*.

math.**asinh**(*x*)

Retorna o seno hiperbólico inverso de *x*.

math.**atanh**(*x*)

Retorna a tangente hiperbólica inversa de *x*.

math.**cosh**(*x*)

Retorna o cosseno hiperbólico de *x*.

math.**sinh**(*x*)

Retorna o seno hiperbólico de *x*.

math.**tanh**(*x*)

Retorna a tangente hiperbólica de *x*.

9.2.9 Funções especiais

math.**erf**(*x*)

Retorna a função erro em *x*.

A função *erf()* pode ser usada para calcular funções estatísticas tradicionais, como a distribuição normal padrão cumulativa:

```
def phi(x):
    'Função de distribuição cumulativa para a distribuição normal padrão.'
    return (1.0 + erf(x / sqrt(2.0))) / 2.0
```

Adicionado na versão 3.2.

math.**erfc**(*x*)

Retorna a função erro complementar em *x*. A função erro complementar é definida como `1.0 - erf(x)`. É usado para grandes valores de *x* onde uma subtração de um causaria uma perda de significância.

Adicionado na versão 3.2.

math.**gamma**(*x*)

Retorna a função gama em *x*.

Adicionado na versão 3.2.

math.**lgamma**(*x*)

Retorna o logaritmo natural do valor absoluto da função gama em *x*.

Adicionado na versão 3.2.

9.2.10 Constantes

math.**pi**

A constante matemática $\pi = 3.141592\ldots$, para a precisão disponível.

math.**e**

A constante matemática $e = 2.718281\ldots$, para a precisão disponível.

math.**tau**

A constante matemática $\tau = 6.283185\ldots$, para a precisão disponível. Tau é uma constante de círculo igual a 2π, a razão entre a circunferência de um círculo e seu raio. Para saber mais sobre Tau, confira o vídeo Pi is (still) Wrong de Vi Hart, e comece a comemorar o dia do Tau comendo duas vezes mais torta!

Adicionado na versão 3.6.

math.**inf**

Um infinito positivo de ponto flutuante. (Para infinito negativo, use `-math.inf`.) Equivalente à saída de `float('inf')`.

Adicionado na versão 3.5.

math.**nan**

Um valor de ponto flutuante "não é um número" (NaN). Equivalente à saída de `float('nan')`. Devido aos requisitos do padrão IEEE-754, `math.nan` e `float('nan')` não são considerados para ser igual a qualquer outro valor numérico, incluindo eles próprios. Para verificar se um número é NaN, use a função *`isnan()`* para testar NaNs em vez de `is` ou `==`. Exemplo:

```
>>> import math
>>> math.nan == math.nan
False
>>> float('nan') == float('nan')
False
>>> math.isnan(math.nan)
True
>>> math.isnan(float('nan'))
True
```

Adicionado na versão 3.5.

Alterado na versão 3.11: Agora está sempre disponível

O módulo *`math`* consiste principalmente em invólucros finos em torno das funções da biblioteca matemática C da plataforma. O comportamento em casos excepcionais segue o Anexo F da norma C99 quando apropriado. A implementação atual levantará *`ValueError`* para operações inválidas como `sqrt(-1.0)` ou `log(0.0)` (onde C99 Anexo F recomenda sinalizar operação inválida ou divisão por zero), e *`OverflowError`* para resultados que estouram (por exemplo, `exp(1000.0)`). Um NaN não será retornado de nenhuma das funções acima, a menos que um ou mais dos argumentos de entrada sejam um NaN; nesse caso, a maioria das funções retornará um NaN, mas (novamente seguindo C99 Anexo F) há algumas exceções a esta regra, por exemplo, `pow(float('nan'), 0.0)` ou `hypot(float('nan'), float('inf'))`.

Observe que o Python não faz nenhum esforço para distinguir NaNs de sinalização de NaNs silenciosos, e o comportamento para NaNs de sinalização permanece não especificado. O comportamento típico é tratar todos os NaNs como se estivessem quietos.

Ver também

Módulo *cmath*
Versões de números complexos de muitas dessas funções.

9.3 cmath — Funções matemáticas para números complexos

Este módulo fornece acesso a funções matemáticas para números complexos. As funções neste módulo aceitam inteiros, números de ponto flutuante ou números complexos como argumentos. Eles também aceitarão qualquer objeto Python que tenha um método `__complex__()` ou `__float__()`: esses métodos são usados para converter o objeto em um número complexo ou de ponto flutuante, respectivamente, e a função é então aplicada ao resultado da conversão.

Nota

Para funções que envolvem cortes de ramificação, temos o problema de decidir como definir essas funções no próprio corte. Seguindo o artigo de Kahan intitulado "Branch cuts for complex elementary functions" (em tradução livre, "Cortes de ramificação para funções complexas elementares"), bem como o Anexo G do C99 e padrões C posteriores, usamos o sinal de zero para distinguir um lado do outro no corte de ramificação: para um corte de ramificação ao longo (de uma porção) do eixo real olhamos para o sinal da parte imaginária, enquanto para um corte de ramificação ao longo do eixo imaginário olhamos para o sinal da parte real.

Por exemplo, a função *cmath.sqrt()* tem um corte de ramificação ao longo do eixo real negativo. Um argumento de `-2-0j` é tratado como se estivesse *abaixo* do corte de ramificação, e assim dá um resultado no eixo imaginário negativo:

```
>>> cmath.sqrt(-2-0j)
-1.4142135623730951j
```

Mas um argumento de `-2+0j` é tratado como se estivesse acima do corte de ramificação:

```
>>> cmath.sqrt(-2+0j)
1.4142135623730951j
```

Conversões de e para coordenadas polares	
phase(z)	Retorna a fase de z
polar(z)	Retorna a representação de z em coordenadas polares
rect(r, phi)	Retorna o número complexo z com coordenadas polares r e *phi*
Funções de potência e logarítmicas	
exp(z)	Retorna e elevado à potência z
log(z[, base])	Retorna o logaritmo de z para a *base* fornecida (e por padrão)
log10(z)	Retorna o logaritmo de base 10 de z
sqrt(z)	Retorna a raiz quadrada de z
Funções trigonométricas	
acos(z)	Retorna o arco cosseno de z
asin(z)	Retorna o arco seno de z
atan(z)	Retorna o arco tangente de z
cos(z)	Retorna o cosseno de z
sin(z)	Retorna o seno de z
tan(z)	Retorna a tangente de z
Funções hiperbólicas	
acosh(z)	Retorna o cosseno hiperbólico inverso de z
asinh(z)	Retorna o seno hiperbólico inverso de z
atanh(z)	Retorna a tangente hiperbólica inversa de z

continua na próxima página

Tabela 2 – continuação da página anterior

`cosh(z)`	Retorna o cosseno hiperbólico de z
`sinh(z)`	Retorna o seno hiperbólico de z
`tanh(z)`	Retorna a tangente hiperbólica de z
Funções de classificação	
`isfinite(z)`	Verifica se todos os componentes de z são finitos
`isinf(z)`	Verifica se algum componente de z é infinito
`isnan(z)`	Verifica se algum componente de z é NaN
`isclose(a, b, *, rel_tol, abs_tol)`	Verifica se os valores a e b estão próximos um do outro
Constantes	
`pi`	$\pi = 3.141592\ldots$
`e`	$e = 2.718281\ldots$
`tau`	$\tau = 2\pi = 6.283185\ldots$
`inf`	Infinito positivo
`infj`	Infinito imaginário puro
`nan`	"Not a number" (NaN)
`nanj`	NaN imaginário puro

9.3.1 Conversões de e para coordenadas polares

Um número complexo Python `z` é armazenado internamente usando coordenadas *retangulares* ou *cartesianas*. É completamente determinado por sua *parte real* `z.real` e sua *parte imaginária* `z.imag`.

Coordenadas polares fornecem uma forma alternativa de representar um número complexo. Em coordenadas polares, um número complexo z é definido pelo módulo r e pelo ângulo de fase *phi*. O módulo r é a distância de z à origem, enquanto a fase *phi* é o ângulo anti-horário, medido em radianos, do eixo x positivo ao segmento de reta que une a origem a z.

As funções a seguir podem ser usadas para converter coordenadas retangulares nativas em coordenadas polares e vice-versa.

`cmath.`**`phase`**`(z)`

Retorna a fase de z (também conhecido como *argumento* de z), como um ponto flutuante. `phase(z)` equivale a `math.atan2(z.imag, z.real)`. O resultado está no intervalo $[-\pi, \pi]$, e o corte de ramificação para esta operação está ao longo do eixo real negativo. O sinal do resultado é igual ao sinal de `z.imag`, mesmo quando `z.imag` é zero:

```
>>> phase(-1+0j)
3.141592653589793
>>> phase(-1-0j)
-3.141592653589793
```

Nota

O módulo (valor absoluto) de um número complexo z pode ser calculado usando a função embutida `abs()`. Não há função do módulo `cmath` separada para esta operação.

`cmath.`**`polar`**`(z)`

Retorna a representação de z em coordenadas polares. Retorna um par `(r, phi)` onde r é o módulo de z e phi é a fase de z. `polar(z)` equivale a `(abs(z), phase(z))`.

`cmath.`**`rect`**`(r, phi)`

Retorna o número complexo z com coordenadas polares r e *phi*. Equivale a `complex(r * math.cos(phi), r * math.sin(phi))`.

9.3.2 Funções de potência e logarítmicas

cmath.**exp**(*z*)

Retorna *e* elevado à potência *z*, onde *e* é a base de logaritmos naturais.

cmath.**log**(*z*[, *base*])

Retorna o logaritmo de *z* para a *base* fornecida. Se a *base* não for especificada, retorna o logaritmo natural de *z*. Há um corte de ramificação, de 0 ao longo do eixo real negativo até -∞.

cmath.**log10**(*z*)

Retorna o logaritmo de *z* na base 10. Este tem o mesmo corte de ramificação que *log()*.

cmath.**sqrt**(*z*)

Retorna a raiz quadrada de *z*. Este tem o mesmo corte de ramificação que *log()*.

9.3.3 Funções trigonométricas

cmath.**acos**(*z*)

Retorna o arco cosseno de *z*. Existem dois cortes de ramificação: um se estende desde 1 ao longo do eixo real até ∞. O outro se estende para a esquerda de -1 ao longo do eixo real até -∞.

cmath.**asin**(*z*)

Retorna o arco seno de *z*. Tem os mesmos cortes de ramificação que *acos()*.

cmath.**atan**(*z*)

Retorna o arco tangente de *z*. Existem dois cortes de ramificação: Um se estende de `1j` ao longo do eixo imaginário até ∞`j`. O outro se estende de `-1j` ao longo do eixo imaginário até −∞`j`.

cmath.**cos**(*z*)

Retorna o cosseno de *z*.

cmath.**sin**(*z*)

Retorna o seno de *z*.

cmath.**tan**(*z*)

Retorna a tangente de *z*.

9.3.4 Funções hiperbólicas

cmath.**acosh**(*z*)

Retorna o cosseno hiperbólico inverso de *z*. Há um corte de ramificação, estendendo-se para a esquerda de 1 ao longo do eixo real até -∞.

cmath.**asinh**(*z*)

Retorna o seno hiperbólico inverso de *z*. Existem dois cortes de ramificação: Um se estende de `1j` ao longo do eixo imaginário até ∞`j`. O outro se estende de `-1j` ao longo do eixo imaginário até −∞`j`.

cmath.**atanh**(*z*)

Retorna a tangente hiperbólica inversa de *z*. Existem dois cortes de ramificação: Um se estende de `1` ao longo do eixo real até ∞. O outro se estende de `-1` ao longo do eixo real até −∞.

cmath.**cosh**(*z*)

Retorna o cosseno hiperbólico de *z*.

cmath.**sinh**(*z*)

Retorna o seno hiperbólico de *z*.

cmath.**tanh**(*z*)

Retorna a tangente hiperbólica de *z*.

9.3.5 Funções de classificação

cmath.**isfinite**(*z*)

Retorna `True` se ambas as partes real e imaginária de *z* forem finitas, e `False` caso contrário.

Adicionado na versão 3.2.

cmath.**isinf**(*z*)

Retorna `True` se ou a parte real ou a imaginária de *z* for infinita, e `False` caso contrário.

cmath.**isnan**(*z*)

Retorna `True` se ou a parte real ou a imaginária de *z* for NaN, e `False` caso contrário.

cmath.**isclose**(*a*, *b*, ***, *rel_tol=1e-09*, *abs_tol=0.0*)

Retorna `True` se os valores *a* e *b* estiverem próximos e `False` caso contrário.

Se dois valores são considerados próximos ou não é determinado de acordo com tolerâncias absolutas e relativas fornecidas. Se nenhum erro ocorrer, o resultado será: `abs(a-b) <= max(rel_tol * max(abs(a), abs(b)), abs_tol)`.

rel_tol é a tolerância relativa – é a diferença máxima permitida entre *a* e *b*, em relação ao maior valor absoluto de *a* ou *b*. Por exemplo, para definir uma tolerância de 5%, passe `rel_tol=0.05`. A tolerância padrão é `1e-09`, o que garante que os dois valores sejam iguais em cerca de 9 dígitos decimais. *rel_tol* deve ser não negativo e menor que `1.0`.

abs_tol é a tolerância absoluta; o padrão é `0.0` e deve ser não negativo. Ao comparar `x` com `0.0`, `isclose(x, 0)` é computado como `abs(x) <= rel_tol * abs(x)`, que é `False` para qualquer `x` e rel_tol menor que `1.0`. Então adicione um argumento abs_tol positivo apropriado à chamada.

Os valores especiais do IEEE 754 de `NaN`, `inf` e `-inf` serão tratados de acordo com as regras do IEEE. Especificamente, `NaN` não é considerado próximo a qualquer outro valor, incluindo `NaN`. `inf` e `-inf` são considerados apenas próximos a si mesmos.

Adicionado na versão 3.5.

> **Ver também**
>
> **PEP 485** – Uma função para testar igualdade aproximada

9.3.6 Constantes

cmath.**pi**

A constante matemática π, como um ponto flutuante.

cmath.**e**

A constante matemática *e*, como um ponto flutuante.

cmath.**tau**

A constante matemática τ, como um ponto flutuante.

Adicionado na versão 3.6.

cmath.**inf**

Infinito positivo de ponto flutuante. Equivale a `float('inf')`.

Adicionado na versão 3.6.

cmath.**infj**

Número complexo com parte real zero e parte imaginária infinita positiva. Equivale a `complex(0.0, float('inf'))`.

Adicionado na versão 3.6.

cmath.**nan**

Um valor de ponto flutuante "não um número" (NaN). Equivale a `float('nan')`.

Adicionado na versão 3.6.

cmath.**nanj**

Número complexo com parte real zero e parte imaginária NaN. Equivale a `complex(0.0, float('nan'))`.

Adicionado na versão 3.6.

Observe que a seleção de funções é semelhante, mas não idêntica, àquela no módulo *math*. A razão para ter dois módulos é que alguns usuários não estão interessados em números complexos e talvez nem saibam o que são. Eles preferem que `math.sqrt(-1)` gere uma exceção do que retorne um número complexo. Observe também que as funções definidas em *cmath* sempre retornam um número complexo, mesmo que a resposta possa ser expressa como um número real (nesse caso o número complexo tem uma parte imaginária de zero).

Uma nota sobre cortes de ramificação: são curvas ao longo das quais a função dada não é contínua. Eles são um recurso necessário de muitas funções complexas. Presume-se que se você precisar calcular com funções complexas, você entenderá sobre cortes de ramificação. Consulte quase qualquer livro (não muito elementar) sobre variáveis complexas para obter esclarecimento. Para informações sobre a escolha adequada dos cortes de ramificação para fins numéricos, uma boa referência deve ser a seguinte:

Ver também

Kahan, W: Branch cuts for complex elementary functions; or, Much ado about nothing's sign bit. Em Iserles, A. e Powell, M. (eds.), The state of the art in numerical analysis. Clarendon Press (1987) pp165–211.

9.4 `decimal` — Aritmética de ponto fixo decimal e ponto flutuante

Código-fonte: Lib/decimal.py

O módulo *decimal* fornece suporte a aritmética rápida de ponto flutuante decimal corretamente arredondado. Oferece várias vantagens sobre o tipo de dados *float*:

- Decimal "é baseado em um modelo de ponto flutuante que foi projetado com as pessoas em mente e necessariamente tem um princípio orientador primordial – os computadores devem fornecer uma aritmética que funcione da mesma maneira que a aritmética que as pessoas aprendem na escola". – trecho da especificação aritmética decimal.
- Os números decimais podem ser representados exatamente. Por outro lado, números como `1.1` e `2.2` não possuem representações exatas em ponto flutuante binário. Os usuários finais normalmente não esperam que `1.1 + 2.2` sejam exibidos como `3.3000000000000003`, como acontece com o ponto flutuante binário.
- A exatidão transita para a aritmética. No ponto flutuante decimal, `0.1 + 0.1 + 0.1 - 0.3` é exatamente igual a zero. No ponto flutuante binário, o resultado é `5.5511151231257827e-017`. Embora próximas de zero, as diferenças impedem o teste de igualdade confiável e as diferenças podem se acumular. Por esse motivo, o decimal é preferido em aplicações de contabilidade que possuem invariáveis estritos de igualdade.
- O módulo decimal incorpora uma noção de casas significativas para que `1.30 + 1.20` seja `2.50`. O zero à direita é mantido para indicar significância. Esta é a apresentação habitual para aplicações monetárias. Para multiplicação, a abordagem "livro escolar" usa todas as figuras nos multiplicandos. Por exemplo, `1.3 * 1.2` é igual a `1.56` enquanto `1.30 * 1.20` é igual a `1.5600`.
- Diferentemente do ponto flutuante binário baseado em hardware, o módulo decimal possui uma precisão alterável pelo usuário (padrão de 28 casas), que pode ser tão grande quanto necessário para um determinado problema:

```
>>> from decimal import *
>>> getcontext().prec = 6
>>> Decimal(1) / Decimal(7)
Decimal('0.142857')
>>> getcontext().prec = 28
>>> Decimal(1) / Decimal(7)
Decimal('0.1428571428571428571428571429')
```

- O ponto flutuante binário e decimal é implementado em termos de padrões publicados. Enquanto o tipo ponto flutuante embutido expõe apenas uma parte modesta de seus recursos, o módulo decimal expõe todas as partes necessárias do padrão. Quando necessário, o programador tem controle total sobre o arredondamento e o manuseio do sinal. Isso inclui uma opção para impor aritmética exata usando exceções para bloquear quaisquer operações inexatas.
- O módulo decimal foi projetado para dar suporte, "sem prejuízo, a aritmética decimal não arredondada exata (às vezes chamada aritmética de ponto fixo) e aritmética arredondada de ponto flutuante". – trecho da especificação aritmética decimal.

O design do módulo é centrado em torno de três conceitos: o número decimal, o contexto da aritmética e os sinais.

Um número decimal é imutável. Possui um sinal, dígitos de coeficiente e um expoente. Para preservar a significância, os dígitos do coeficiente não truncam zeros à direita. Os decimais também incluem valores especiais, tais como `Infinity`, `-Infinity` e `NaN`. O padrão também diferencia `-0` de `+0`.

O contexto da aritmética é um ambiente que especifica precisão, regras de arredondamento, limites de expoentes, sinalizadores indicando os resultados das operações e ativadores de interceptação que determinam se os sinais são tratados como exceções. As opções de arredondamento incluem *`ROUND_CEILING`*, *`ROUND_DOWN`*, *`ROUND_FLOOR`*, *`ROUND_HALF_DOWN`*, *`ROUND_HALF_EVEN`*, *`ROUND_HALF_UP`*, *`ROUND_UP`* e *`ROUND_05UP`*.

Sinais são grupos de condições excepcionais que surgem durante o curso da computação. Dependendo das necessidades da aplicação, os sinais podem ser ignorados, considerados informativos ou tratados como exceções. Os sinais no módulo decimal são: *`Clamped`*, *`InvalidOperation`*, *`DivisionByZero`*, *`Inexact`*, *`Rounded`*, *`Subnormal`*, *`Overflow`*, *`Underflow`* e *`FloatOperation`*.

Para cada sinal, há um sinalizador e um ativador de interceptação. Quando um sinal é encontrado, seu sinalizador é definido como um e, se o ativador de interceptação estiver definido como um, uma exceção será gerada. Os sinalizadores são fixos; portanto, o usuário precisa redefini-los antes de monitorar um cálculo.

Ver também

- A especificação geral aritmética decimal da IBM, The General Decimal Arithmetic Specification.

9.4.1 Tutorial de início rápido

O início usual do uso de decimais é importar o módulo, exibir o contexto atual com *`getcontext()`* e, se necessário, definir novos valores para precisão, arredondamento ou armadilhas ativados:

```
>>> from decimal import *
>>> getcontext()
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
        capitals=1, clamp=0, flags=[], traps=[Overflow, DivisionByZero,
        InvalidOperation])

>>> getcontext().prec = 7       # Define uma nova precisão
```

Instâncias decimais podem ser construídas a partir de números inteiros, strings, pontos flutuantes ou tuplas. A construção de um número inteiro ou de um ponto flutuante realiza uma conversão exata do valor desse número inteiro ou ponto flutuante. Os números decimais incluem valores especiais como `NaN`, que significa "Não é um número", `Infinity` positivo e negativo e `-0`:

```
>>> getcontext().prec = 28
>>> Decimal(10)
Decimal('10')
>>> Decimal('3.14')
Decimal('3.14')
>>> Decimal(3.14)
Decimal('3.140000000000000124344978758017532527446746826171875')
>>> Decimal((0, (3, 1, 4), -2))
Decimal('3.14')
>>> Decimal(str(2.0 ** 0.5))
Decimal('1.4142135623730951')
>>> Decimal(2) ** Decimal('0.5')
Decimal('1.4142135623730950488016887242')
>>> Decimal('NaN')
Decimal('NaN')
>>> Decimal('-Infinity')
Decimal('-Infinity')
```

Se o sinal *FloatOperation* for capturado na armadilha, a mistura acidental de decimais e pontos flutuantes em construtores ou comparações de ordenação levanta uma exceção:

```
>>> c = getcontext()
>>> c.traps[FloatOperation] = True
>>> Decimal(3.14)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.FloatOperation: [<class 'decimal.FloatOperation'>]
>>> Decimal('3.5') < 3.7
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.FloatOperation: [<class 'decimal.FloatOperation'>]
>>> Decimal('3.5') == 3.5
True
```

Adicionado na versão 3.3.

O significado de um novo decimal é determinado apenas pelo número de dígitos inseridos. A precisão e o arredondamento do contexto só entram em jogo durante operações aritméticas.

```
>>> getcontext().prec = 6
>>> Decimal('3.0')
Decimal('3.0')
>>> Decimal('3.1415926535')
Decimal('3.1415926535')
>>> Decimal('3.1415926535') + Decimal('2.7182818285')
Decimal('5.85987')
>>> getcontext().rounding = ROUND_UP
>>> Decimal('3.1415926535') + Decimal('2.7182818285')
Decimal('5.85988')
```

Se os limites internos da versão C forem excedidos, a construção de um decimal levanta *InvalidOperation*:

```
>>> Decimal("1e9999999999999999999")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.InvalidOperation: [<class 'decimal.InvalidOperation'>]
```

Alterado na versão 3.3.

Os decimais interagem bem com grande parte do resto do Python. Aqui está um pequeno circo voador de ponto flutuante decimal:

```
>>> data = list(map(Decimal, '1.34 1.87 3.45 2.35 1.00 0.03 9.25'.split()))
>>> max(data)
Decimal('9.25')
>>> min(data)
Decimal('0.03')
>>> sorted(data)
[Decimal('0.03'), Decimal('1.00'), Decimal('1.34'), Decimal('1.87'),
 Decimal('2.35'), Decimal('3.45'), Decimal('9.25')]
>>> sum(data)
Decimal('19.29')
>>> a,b,c = data[:3]
>>> str(a)
'1.34'
>>> float(a)
1.34
>>> round(a, 1)
Decimal('1.3')
>>> int(a)
1
>>> a * 5
Decimal('6.70')
>>> a * b
Decimal('2.5058')
>>> c % a
Decimal('0.77')
```

E algumas funções matemáticas também estão disponíveis no Decimal:

```
>>> getcontext().prec = 28
>>> Decimal(2).sqrt()
Decimal('1.4142135623730950488016887242')
>>> Decimal(1).exp()
Decimal('2.7182818284590452353602874713')
>>> Decimal('10').ln()
Decimal('2.3025850929940456840179914547')
>>> Decimal('10').log10()
Decimal('1')
```

O método *quantize()* arredonda um número para um expoente fixo. Esse método é útil para aplicações monetárias que geralmente arredondam os resultados para um número fixo de locais:

```
>>> Decimal('7.325').quantize(Decimal('.01'), rounding=ROUND_DOWN)
Decimal('7.32')
>>> Decimal('7.325').quantize(Decimal('1.'), rounding=ROUND_UP)
Decimal('8')
```

Como mostrado acima, a função *getcontext()* acessa o contexto atual e permite que as configurações sejam alteradas. Essa abordagem atende às necessidades da maioria das aplicações.

Para trabalhos mais avançados, pode ser útil criar contextos alternativos usando o construtor Context(). Para ativar uma alternativa, use a função *setcontext()*.

De acordo com o padrão, o módulo *decimal* fornece dois contextos padrão prontos para uso, *BasicContext* e *ExtendedContext*. O primeiro é especialmente útil para depuração porque muitas das armadilhas estão ativadas:

```
>>> myothercontext = Context(prec=60, rounding=ROUND_HALF_DOWN)
>>> setcontext(myothercontext)
>>> Decimal(1) / Decimal(7)
Decimal('0.142857142857142857142857142857142857142857142857142857142857')

>>> ExtendedContext
Context(prec=9, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
        capitals=1, clamp=0, flags=[], traps=[])
>>> setcontext(ExtendedContext)
>>> Decimal(1) / Decimal(7)
Decimal('0.142857143')
>>> Decimal(42) / Decimal(0)
Decimal('Infinity')

>>> setcontext(BasicContext)
>>> Decimal(42) / Decimal(0)
Traceback (most recent call last):
  File "<pyshell#143>", line 1, in -toplevel-
    Decimal(42) / Decimal(0)
DivisionByZero: x / 0
```

Os contextos também possuem sinalizadores para monitorar condições excepcionais encontradas durante os cálculos. Os sinalizadores permanecem definidos até que sejam explicitamente limpos, portanto, é melhor limpar os sinalizadores antes de cada conjunto de cálculos monitorados usando o método *clear_flags()*.

```
>>> setcontext(ExtendedContext)
>>> getcontext().clear_flags()
>>> Decimal(355) / Decimal(113)
Decimal('3.14159292')
>>> getcontext()
Context(prec=9, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
        capitals=1, clamp=0, flags=[Inexact, Rounded], traps=[])
```

A entrada *flags* mostra que a aproximação racional de pi foi arredondada (dígitos além da precisão do contexto foram descartados) e que o resultado é inexato (alguns dos dígitos descartados eram diferentes de zero).

As armadilhas individuais são definidas usando o dicionário no atributo *traps* de um contexto:

```
>>> setcontext(ExtendedContext)
>>> Decimal(1) / Decimal(0)
Decimal('Infinity')
>>> getcontext().traps[DivisionByZero] = 1
>>> Decimal(1) / Decimal(0)
Traceback (most recent call last):
  File "<pyshell#112>", line 1, in -toplevel-
    Decimal(1) / Decimal(0)
DivisionByZero: x / 0
```

A maioria dos programas ajusta o contexto atual apenas uma vez, no início do programa. E, em muitas aplicações, os dados são convertidos para *Decimal* com uma única conversão dentro de um loop. Com o conjunto de contextos e decimais criados, a maior parte do programa manipula os dados de maneira diferente do que com outros tipos numéricos do Python.

9.4.2 Objetos de Decimal

class decimal.**Decimal**(*value='0'*, *context=None*)

Constrói um novo objeto de *Decimal* com base em *value*.

value pode ser um inteiro, string, tupla, *float* ou outro objeto de *Decimal*. Se nenhum *value* for fornecido, retornará `Decimal('0')`. Se *value* for uma string, ele deverá estar em conformidade com a sintaxe da string numérica decimal após caracteres de espaço em branco à esquerda e à direita, bem como sublinhados em toda parte, serem removidos:

```
sign           ::=  '+' | '-'
digit          ::=  '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
indicator      ::=  'e' | 'E'
digits         ::=  digit [digit]...
decimal-part   ::=  digits '.' [digits] | ['.'] digits
exponent-part  ::=  indicator [sign] digits
infinity       ::=  'Infinity' | 'Inf'
nan            ::=  'NaN' [digits] | 'sNaN' [digits]
numeric-value  ::=  decimal-part [exponent-part] | infinity
numeric-string ::=  [sign] numeric-value | [sign] nan
```

Outros dígitos decimais Unicode também são permitidos onde `digit` aparece acima. Isso inclui dígitos decimais de vários outros alfabetos (por exemplo, dígitos em árabes-índicos e devanágaris), além dos dígitos de largura total `'\uff10'` a `'\uff19'`. A capitalização não é significativa, então, por exemplo, `inf`, `Inf`, `INFINITY` e `iNfINity` são todas grafias aceitáveis para infinito positivo.

Se *value* for um *tuple*, ele deverá ter três componentes, um sinal (`0` para positivo ou `1` para negativo), um *tuple* de dígitos e um expoente inteiro. Por exemplo, `Decimal((0, (1, 4, 1, 4), -3))` returna `Decimal('1.414')`.

Se *value* é um *float*, o valor do ponto flutuante binário é convertido sem perdas no seu equivalente decimal exato. Essa conversão geralmente requer 53 ou mais dígitos de precisão. Por exemplo, `Decimal(float('1.1'))` converte para `Decimal('1.100000000000000088817841970012523233890533447265625')`.

A precisão *context* não afeta quantos dígitos estão armazenados. Isso é determinado exclusivamente pelo número de dígitos em *value*. Por exemplo, `Decimal('3.00000')` registra todos os cinco zeros, mesmo que a precisão do contexto seja apenas três.

O objetivo do argumento *context* é determinar o que fazer se *value* for uma string malformada. Se o contexto capturar *InvalidOperation*, uma exceção será levantada; caso contrário, o construtor retornará um novo decimal com o valor de `NaN`.

Uma vez construídos, objetos de *Decimal* são imutáveis.

Alterado na versão 3.2: O argumento para o construtor agora pode ser uma instância de *float*.

Alterado na versão 3.3: Os argumentos de *float* levantam uma exceção se a armadilha *FloatOperation* estiver definida. Por padrão, a armadilha está desativada.

Alterado na versão 3.6: Sublinhados são permitidos para agrupamento, como nos literais de ponto flutuante e integral no código.

Objetos decimais de ponto flutuante compartilham muitas propriedades com outros tipos numéricos embutidos, como *float* e *int*. Todas as operações matemáticas usuais e métodos especiais se aplicam. Da mesma forma, objetos decimais podem ser copiados, separados, impressos, usados como chaves de dicionário, usados como elementos de conjunto, comparados, classificados e convertidos a outro tipo (como *float* ou *int*).

Existem algumas pequenas diferenças entre aritmética em objetos decimais e aritmética em números inteiros e flutuantes. Quando o operador de resto `%` é aplicado a objetos decimais, o sinal do resultado é o sinal do *dividend* em vez do sinal do divisor:

```
>>> (-7) % 4
1
>>> Decimal(-7) % Decimal(4)
Decimal('-3')
```

O operador de divisão inteira `//` se comporta de maneira análoga, retornando a parte inteira do quociente verdadeiro (truncando em direção a zero) em vez de seu resto, de modo a preservar a identidade usual `x == (x // y) * y + x % y`:

```
>>> -7 // 4
-2
>>> Decimal(-7) // Decimal(4)
Decimal('-1')
```

Os operadores `%` e `//` implementam as operações de `remainder` e `divide-integer` (respectivamente) como descrito na especificação.

Objetos decimais geralmente não podem ser combinados com pontos flutuantes ou instâncias de *`fractions.Fraction`* em operações aritméticas: uma tentativa de adicionar um *`Decimal`* a um *`float`*, por exemplo, vai levantar um *`TypeError`*. No entanto, é possível usar os operadores de comparação do Python para comparar uma instância de *`Decimal`* `x` com outro número `y`. Isso evita resultados confusos ao fazer comparações de igualdade entre números de tipos diferentes.

Alterado na versão 3.2: As comparações de tipos mistos entre instâncias de *`Decimal`* e outros tipos numéricos agora são totalmente suportadas.

Além das propriedades numéricas padrões, os objetos de ponto flutuante decimal também possuem vários métodos especializados:

`adjusted``()`

Retorna o expoente ajustado depois de deslocar os dígitos mais à direita do coeficiente até restar apenas o dígito principal: `Decimal('321e+5').adjusted()` retorna sete. Usado para determinar a posição do dígito mais significativo em relação ao ponto decimal.

`as_integer_ratio``()`

Retorna um par `(n, d)` de números inteiros que representam a instância dada *`Decimal`* como uma fração, nos termos mais baixos e com um denominador positivo:

```
>>> Decimal('-3.14').as_integer_ratio()
(-157, 50)
```

A conversão é exata. Levanta OverflowError em infinitos e ValueError em NaNs.

Adicionado na versão 3.6.

`as_tuple``()`

Retorna uma representação de *tupla nomeada* do número: `DecimalTuple(sign, digits, exponent)`.

`canonical``()`

Retorna a codificação canônica do argumento. Atualmente, a codificação de uma instância de *`Decimal`* é sempre canônica, portanto, esta operação retorna seu argumento inalterado.

`compare`(*other*, *context=None*)

Compara os valores de duas instâncias decimais. *`compare()`* retorna uma instância decimal, e se qualquer operando for um NaN, o resultado será um NaN:

```
a or b is a NaN  ==> Decimal('NaN')
a < b            ==> Decimal('-1')
a == b           ==> Decimal('0')
a > b            ==> Decimal('1')
```

`compare_signal`(*other*, *context=None*)

Esta operação é idêntica ao método *`compare()`*, exceto que todos os NaNs sinalizam. Ou seja, se nenhum operando for um NaN sinalizador, qualquer operando NaN silencioso será tratado como se fosse um NaN sinalizador.

`compare_total`(*other*, *context=None*)

Compara dois operandos usando sua representação abstrata em vez de seu valor numérico. Semelhante ao método *`compare()`*, mas o resultado fornece uma ordem total nas instâncias de *`Decimal`*. Duas

instâncias de *Decimal* com o mesmo valor numérico, mas diferentes representações, se comparam desiguais nesta ordem:

```
>>> Decimal('12.0').compare_total(Decimal('12'))
Decimal('-1')
```

Os NaNs silenciosos e sinalizadores também estão incluídos no pedido total. O resultado dessa função é `Decimal('0')` se os dois operandos tiverem a mesma representação, `Decimal('-1')` se o primeiro operando for menor na ordem total que o segundo e `Decimal('1')` se o primeiro operando for maior na ordem total que o segundo operando. Veja a especificação para detalhes da ordem total.

Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado. Como uma exceção, a versão C pode levantar InvalidOperation se o segundo operando não puder ser convertido exatamente.

compare_total_mag(*other*, *context=None*)

Compara dois operandos usando sua representação abstrata em vez de seu valor, como em *compare_total()*, mas ignorando o sinal de cada operando. `x.compare_total_mag(y)` é equivalente a `x.copy_abs().compare_total(y.copy_abs())`.

Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado. Como uma exceção, a versão C pode levantar InvalidOperation se o segundo operando não puder ser convertido exatamente.

conjugate()

Apenas retorna a si próprio, sendo esse método apenas para atender à Especificação de Decimal.

copy_abs()

Retorna o valor absoluto do argumento. Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado.

copy_negate()

Retorna a negação do argumento. Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado.

copy_sign(*other*, *context=None*)

Retorna uma cópia do primeiro operando com o sinal definido para ser o mesmo que o sinal do segundo operando. Por exemplo:

```
>>> Decimal('2.3').copy_sign(Decimal('-1.5'))
Decimal('-2.3')
```

Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado. Como uma exceção, a versão C pode levantar InvalidOperation se o segundo operando não puder ser convertido exatamente.

exp(*context=None*)

Retorna o valor da função exponencial (natural) `e**x` no número especificado. O resultado é arredondado corretamente usando o modo de arredondamento *ROUND_HALF_EVEN*.

```
>>> Decimal(1).exp()
Decimal('2.718281828459045235360287471')
>>> Decimal(321).exp()
Decimal('2.561702493119680037517373933E+139')
```

classmethod from_float(*f*, */*)

Construtor alternativo que aceita apenas instâncias de *float* ou *int*.

Observe que `Decimal.from_float(0.1)` não é o mesmo que `Decimal('0.1')`. Como 0.1 não é exatamente representável no ponto flutuante binário, o valor é armazenado como o valor representável mais próximo que é `0x1.999999999999ap-4`.; Esse valor equivalente em decimal é `0.1000000000000000055511151231257827021181583404541015625`.

> **Nota**
>
> A partir do Python 3.2 em diante, uma instância de `Decimal` também pode ser construída diretamente a partir de um `float`.

```
>>> Decimal.from_float(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015625')
>>> Decimal.from_float(float('nan'))
Decimal('NaN')
>>> Decimal.from_float(float('inf'))
Decimal('Infinity')
>>> Decimal.from_float(float('-inf'))
Decimal('-Infinity')
```

Adicionado na versão 3.1.

classmethod from_number(*number*, */*)

Construtor alternativo que aceita apenas instâncias de `float`, `int` ou `Decimal`, mas não strings ou tuplas.

```
>>> Decimal.from_number(314)
Decimal('314')
>>> Decimal.from_number(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015625')
>>> Decimal.from_number(Decimal('3.14'))
Decimal('3.14')
```

Adicionado na versão 3.14.

fma(*other*, *third*, *context=None*)

Multiplicação e adição fundidos. Retorna self*other+third sem arredondamento do produto intermediário self*other.

```
>>> Decimal(2).fma(3, 5)
Decimal('11')
```

is_canonical()

Retorna `True` se o argumento for canônico e `False` caso contrário. Atualmente, uma instância de `Decimal` é sempre canônica, portanto, esta operação sempre retorna `True`.

is_finite()

Retorna `True` se o argumento for um número finito e `False` se o argumento for um infinito ou um NaN.

is_infinite()

Retorna `True` se o argumento for infinito positivo ou negativo e `False` caso contrário.

is_nan()

Retorna `True` se o argumento for NaN (silencioso ou sinalizador) e `False` caso contrário.

is_normal(*context=None*)

Retorna `True` se o argumento for um número finito *normal*. Retorna `False` se o argumento for zero, subnormal, infinito ou NaN.

is_qnan()

Retorna `True` se o argumento for um NaN silencioso, e `False` caso contrário.

is_signed()

Retorna `True` se o argumento tiver um sinal negativo e `False` caso contrário. Observe que zeros e NaNs podem carregar sinais.

is_snan()

Retorna *True* se o argumento for um sinal NaN e *False* caso contrário.

is_subnormal(*context=None*)

Retorna *True* se o argumento for subnormal e *False* caso contrário.

is_zero()

Retorna *True* se o argumento for um zero (positivo ou negativo) e *False* caso contrário.

ln(*context=None*)

Retorna o logaritmo (base e) natural do operando. O resultado é arredondado corretamente usando o modo de arredondamento *ROUND_HALF_EVEN*.

log10(*context=None*)

Retorna o logaritmo da base dez do operando. O resultado é arredondado corretamente usando o modo de arredondamento *ROUND_HALF_EVEN*.

logb(*context=None*)

Para um número diferente de zero, retorna o expoente ajustado de seu operando como uma instância de *Decimal*. Se o operando é zero, `Decimal('-Infinity')` é retornado e o sinalizador *DivisionByZero* é levantado. Se o operando for um infinito, `Decimal('Infinity')` será retornado.

logical_and(*other*, *context=None*)

logical_and() é uma operação lógica que leva dois *operandos lógicos* (consulte *Operandos lógicos*). O resultado é o `and` dígito a dígito dos dois operandos.

logical_invert(*context=None*)

logical_invert() é uma operação lógica. O resultado é a inversão dígito a dígito do operando.

logical_or(*other*, *context=None*)

logical_or() é uma operação lógica que leva dois *operandos lógicos* (consulte *Operandos lógicos*). O resultado é o `or` dígito a dígito dos dois operandos.

logical_xor(*other*, *context=None*)

logical_xor() é uma operação lógica que leva dois *operandos lógicos* (consulte *Operandos lógicos*). O resultado é o "ou exclusivo" dígito a dígito ou dos dois operandos.

max(*other*, *context=None*)

Como `max(self, other)`, exceto que a regra de arredondamento de contexto é aplicada antes de retornar e que os valores `NaN` são sinalizados ou ignorados (dependendo do contexto e se estão sinalizando ou silenciosos).

max_mag(*other*, *context=None*)

Semelhante ao método *max()*, mas a comparação é feita usando os valores absolutos dos operandos.

min(*other*, *context=None*)

Como `min(self, other)`, exceto que a regra de arredondamento de contexto é aplicada antes de retornar e que os valores `NaN` são sinalizados ou ignorados (dependendo do contexto e se estão sinalizando ou silenciosos).

min_mag(*other*, *context=None*)

Semelhante ao método *min()*, mas a comparação é feita usando os valores absolutos dos operandos.

next_minus(*context=None*)

Retorna o maior número representável no contexto fornecido (ou no contexto atual da thread, se nenhum contexto for fornecido) que seja menor que o operando especificado.

next_plus(*context=None*)

Retorna o menor número representável no contexto fornecido (ou no contexto atual da thread, se nenhum contexto for fornecido) que seja maior que o operando fornecido.

next_toward(*other*, *context=None*)

Se os dois operandos forem desiguais, retorna o número mais próximo ao primeiro operando na direção do segundo operando. Se os dois operandos forem numericamente iguais, retorna uma cópia do primeiro operando com o sinal configurado para ser o mesmo que o sinal do segundo operando.

normalize(*context=None*)

Usado para produzir valores canônicos de uma classe de equivalência no contexto atual ou no contexto especificado.

Esta tem a mesma semântica que a operação unária mais, exceto que se o resultado final for finito, ele será reduzido à sua forma mais simples, com todos os zeros à direita removidos e seu sinal preservado. Ou seja, enquanto o coeficiente for diferente de zero e múltiplo de dez, o coeficiente é dividido por dez e o expoente é incrementado em 1. Caso contrário (o coeficiente é zero), o expoente é definido como 0. Em todos os casos, o sinal permanece inalterado. .

Por exemplo, `Decimal('32.100')` e `Decimal('0.321000e+2')` ambos normalizam para o valor equivalente `Decimal('32.1')`.

Observe que o arredondamento é aplicado *antes* de reduzir para a forma mais simples.

Nas versões mais recentes da especificação, esta operação também é conhecida como `reduce`.

number_class(*context=None*)

Retorna uma string descrevendo a *classe* do operando. O valor retornado é uma das dez sequências a seguir.

- `"-Infinity"`, indicando que o operando é infinito negativo.
- `"-Normal"`, indicando que o operando é um número normal negativo.
- `"-Subnormal"`, indicando que o operando é negativo e subnormal.
- `"-Zero"`, indicando que o operando é um zero negativo.
- `"+Zero"`, indicando que o operando é um zero positivo.
- `"+Subnormal"`, indicando que o operando é positivo e subnormal.
- `"+Normal"`, indicando que o operando é um número normal positivo.
- `"+Infinity"`, indicando que o operando é infinito positivo.
- `"NaN"`, indicando que o operando é um NaN (“Not a Number”) silencioso.
- `"sNaN"`, indicando que o operando é um NaN sinalizador.

quantize(*exp*, *rounding=None*, *context=None*)

Retorna um valor igual ao primeiro operando após o arredondamento e com o expoente do segundo operando.

```
>>> Decimal('1.41421356').quantize(Decimal('1.000'))
Decimal('1.414')
```

Diferentemente de outras operações, se o comprimento do coeficiente após a operação de quantização for maior que a precisão, então *InvalidOperation* é sinalizado. Isso garante que, a menos que haja uma condição de erro, o expoente quantizado é sempre igual ao do operando do lado direito.

Também, diferentemente de outras operações, a quantização nunca sinaliza Underflow, mesmo que o resultado seja subnormal e inexato.

Se o expoente do segundo operando for maior que o do primeiro, o arredondamento poderá ser necessário. Nesse caso, o modo de arredondamento é determinado pelo argumento `rounding`, se fornecido, ou pelo argumento `context` fornecido; se nenhum argumento for fornecido, o modo de arredondamento do contexto da thread atual será usado.

Um erro é retornado sempre que o expoente resultante for maior que *Emax* ou menor que *Etiny()*.

radix()

Retorna `Decimal(10)`, a raiz (base) na qual a classe *Decimal* faz toda a sua aritmética. Incluído para compatibilidade com a especificação.

remainder_near(*other*, *context=None*)

Retorna o resto da divisão de *self* por *other*. Isso é diferente de `self % other`, pois o sinal do resto é escolhido para minimizar seu valor absoluto. Mais precisamente, o valor de retorno é `self - n * other`, onde `n` é o número inteiro mais próximo do valor exato de `self / other`, e se dois números inteiros estiverem igualmente próximos, o par será é escolhido.

Se o resultado for zero, seu sinal será o sinal de *self*.

```
>>> Decimal(18).remainder_near(Decimal(10))
Decimal('-2')
>>> Decimal(25).remainder_near(Decimal(10))
Decimal('5')
>>> Decimal(35).remainder_near(Decimal(10))
Decimal('-5')
```

rotate(*other*, *context=None*)

Retorna o resultado da rotação dos dígitos do primeiro operando em uma quantidade especificada pelo segundo operando. O segundo operando deve ser um número inteiro no intervalo - precisão através da precisão. O valor absoluto do segundo operando fornece o número de locais a serem rotacionados. Se o segundo operando for positivo, a rotação será para a esquerda; caso contrário, a rotação será para a direita. O coeficiente do primeiro operando é preenchido à esquerda com zeros na precisão do comprimento, se necessário. O sinal e o expoente do primeiro operando não são alterados.

same_quantum(*other*, *context=None*)

Testa se "self" e "other" têm o mesmo expoente ou se ambos são `NaN`.

Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado. Como uma exceção, a versão C pode levantar InvalidOperation se o segundo operando não puder ser convertido exatamente.

scaleb(*other*, *context=None*)

Retorna o primeiro operando com o expoente ajustado pelo segundo. Da mesma forma, retorna o primeiro operando multiplicado por `10**other`. O segundo operando deve ser um número inteiro.

shift(*other*, *context=None*)

Retorna o resultado da troca dos dígitos do primeiro operando em uma quantidade especificada pelo segundo operando. O segundo operando deve ser um número inteiro no intervalo - precisão através da precisão. O valor absoluto do segundo operando fornece o número de locais a serem deslocados. Se o segundo operando for positivo, o deslocamento será para a esquerda; caso contrário, a mudança é para a direita. Os dígitos deslocados para o coeficiente são zeros. O sinal e o expoente do primeiro operando não são alterados.

sqrt(*context=None*)

Retorna a raiz quadrada do argumento para a precisão total.

to_eng_string(*context=None*)

Converte em uma string, usando notação de engenharia, se for necessário um expoente.

A notação de engenharia possui um expoente que é múltiplo de 3. Isso pode deixar até 3 dígitos à esquerda da casa decimal e pode exigir a adição de um ou dois zeros à direita.

Por exemplo, isso converte `Decimal('123E+1')` para `Decimal('1.23E+3')`.

to_integral(*rounding=None*, *context=None*)

Idêntico ao método *to_integral_value()*. O nome `to_integral` foi mantido para compatibilidade com versões mais antigas.

to_integral_exact(*rounding=None*, *context=None*)

Arredonda para o número inteiro mais próximo, sinalizando *Inexact* ou *Rounded*, conforme apropriado, se o arredondamento ocorrer. O modo de arredondamento é determinado pelo parâmetro `rounding`, se fornecido, ou pelo `context` especificado. Se nenhum parâmetro for fornecido, o modo de arredondamento do contexto atual será usado.

to_integral_value(*rounding=None*, *context=None*)

Arredonda para o número inteiro mais próximo sem sinalizar *Inexact* ou `Rounding`. Se fornecido, aplica *rounding*; caso contrário, usa o método de arredondamento no *context* especificado ou no contexto atual.

Os números decimais podem ser arredondados usando a função *round()*:

round(number)

round(number, ndigits)

Se *ndigits* não for fornecido ou `None`, retorna o *int* de *number* mais próximo, arredondando até chegar em par, e ignorando o modo de arredondamento do contexto *Decimal*. Levanta *OverflowError* se *number* for um infinito ou *ValueError* se for um NaN (silencioso ou de sinalização).

Se *ndigits* for um *int*, o modo de arredondamento do contexto é respeitado e um *Decimal* representando *número* arredondado para o múltiplo mais próximo de `Decimal('1E-ndigits')` é retornado; neste caso, `round(number, ndigits)` é equivalente a `self.quantize(Decimal('1E-ndigits'))`. Retorna `Decimal('NaN')` se *number* for um NaN silencioso. Levanta *InvalidOperation* se *number* for um infinito, uma sinalização NaN, ou se o comprimento do coeficiente após a operação de quantização for maior que a precisão do contexto atual. Em outras palavras, para os situações comuns:

- se *ndigits* for positivo, retorna *number* arredondado para *ndigits* casas decimais;
- se *ndigits* for zero, retorna *number* arredondado para o número inteiro mais próximo;
- se *ndigits* for negativo, retorna *number* arredondado para o múltiplo mais próximo de `10**abs(ndigits)`.

Por exemplo:

```
>>> from decimal import Decimal, getcontext, ROUND_DOWN
>>> getcontext().rounding = ROUND_DOWN
>>> round(Decimal('3.75'))      # arredondamento de contexto ignorado
4
>>> round(Decimal('3.5'))       # arredondamento para par mais próximo
4
>>> round(Decimal('3.75'), 0)   # usa o arredondamento de contexto
Decimal('3')
>>> round(Decimal('3.75'), 1)
Decimal('3.7')
>>> round(Decimal('3.75'), -1)
Decimal('0E+1')
```

Operandos lógicos

Os métodos *logical_and()*, *logical_invert()*, *logical_or()* e *logical_xor()* esperam que seus argumentos sejam *operandos lógicos*. Um *operando lógico* é uma instância de *Decimal* cujo expoente e sinal são zero e cujos dígitos são todos `0` ou `1`.

9.4.3 Objetos de contexto

Contextos são ambientes para operações aritméticas. Eles governam a precisão, estabelecem regras para arredondamento, determinam quais sinais são tratados como exceções e limitam o intervalo dos expoentes.

Cada thread possui seu próprio contexto atual que é acessado ou alterado usando as funções *getcontext()* e *setcontext()*:

decimal.**getcontext**()

Retorna o contexto atual para a thread ativa.

decimal.**setcontext**(*c*, /)

Define o contexto atual para a thread ativa como *C*.

Você também pode usar a instrução `with` e a função *localcontext()* para alterar temporariamente o contexto ativo.

decimal.**localcontext**(*ctx=None*, ***kwargs*)

Retorna um gerenciador de contexto que vai definir o contexto atual da thread ativa para uma cópia de *ctx* na entrada da instrução "with" e restaurar o contexto anterior ao sair da instrução "with". Se nenhum contexto for especificado, uma cópia do contexto atual será usada. O argumento *kwargs* é usado para definir os atributos do novo contexto.

Por exemplo, o código a seguir define a precisão decimal atual para 42 casas, executa um cálculo e restaura automaticamente o contexto anterior:

```
from decimal import localcontext

with localcontext() as ctx:
    ctx.prec = 42   # Efetua um cálculo de alta precisão
    s = calculate_something()
s = +s  # Arredonda o resultado final de volta para a precisão padrão
```

Usando argumentos nomeados, o código seria o seguinte:

```
from decimal import localcontext

with localcontext(prec=42) as ctx:
    s = calculate_something()
s = +s
```

Levanta *TypeError* se *kwargs* fornecer um atributo que *Context* não oferecer suporte. Levanta *TypeError* ou *ValueError* se *kwargs* fornecer um valor inválido para um atributo.

Alterado na versão 3.11: *localcontext()* agora tem suporte à configuração de atributos de contexto através do uso de argumentos nomeados.

decimal.**IEEEContext**(*bits*)

Retorna um objeto de contexto inicializado com os valores apropriados para um dos formatos de intercâmbio IEEE. O argumento deve ser um múltiplo de 32 e menor que *IEEE_CONTEXT_MAX_BITS*.

Adicionado na versão 3.14.

Novos contextos também podem ser criados usando o construtor *Context* descrito abaixo. Além disso, o módulo fornece três contextos pré-criados:

decimal.**BasicContext**

Este é um contexto padrão definido pela Especificação Aritmética Decimal Geral. A precisão está definida como nove. O arredondamento está definido como *ROUND_HALF_UP*. Todos os sinalizadores estão limpos. Todos as armadilhas estão ativadas (tratadas como exceções), exceto por *Inexact*, *Rounded* e *Subnormal*.

Como muitas das armadilhas estão ativadas, esse contexto é útil para depuração.

decimal.**ExtendedContext**

Este é um contexto padrão definido pela Especificação Aritmética Decimal Geral. A precisão está definida como nove. O arredondamento está definido como *ROUND_HALF_EVEN*. Todos os sinalizadores estão limpos. Nenhuma armadilha está ativada (de forma que exceções não são levantadas durante os cálculos).

Como as armadilhas estão desativadas, esse contexto é útil para aplicativos que preferem ter o valor de resultado de `NaN` ou `Infinity` em vez de levantar exceções. Isso permite que uma aplicação conclua uma execução na presença de condições que interromperiam o programa.

decimal.**DefaultContext**

Este contexto é usado pelo construtor *Context* como um protótipo para novos contextos. Alterar um campo (tal como precisão) tem o efeito de alterar o padrão para novos contextos criados pelo construtor *Context*.

Esse contexto é mais útil em ambientes multithread. A alteração de um dos campos antes do início das threads tem o efeito de definir os padrões para todo o sistema. Não é recomendável alterar os campos após o início das threads, pois exigiria sincronização de threads para evitar condições de corrida.

Em ambientes de thread única, é preferível não usar esse contexto. Em vez disso, basta criar contextos explicitamente, conforme descrito abaixo.

Os valores padrão são *Context.prec*=28, *Context.rounding*=*ROUND_HALF_EVEN* e armadilhas ativadas para *Overflow*, *InvalidOperation* e *DivisionByZero*.

Além dos três contextos fornecidos, novos contextos podem ser criados com o construtor *Context*.

class decimal.**Context**(*prec=None*, *rounding=None*, *Emin=None*, *Emax=None*, *capitals=None*, *clamp=None*, *flags=None*, *traps=None*)

Cria um novo contexto. Se um campo não for especificado ou for *None*, os valores padrão serão copiados de *DefaultContext*. Se o campo *flags* não for especificado ou for *None*, todos os sinalizadores serão limpados.

prec

Um inteiro no intervalo [1, *MAX_PREC*] que define a precisão para operações aritméticas no contexto.

rounding

Uma das constantes listadas na seção *Modos de arredondamento*.

traps

flags

Listas de todos os sinais a serem configurados. Geralmente, novos contextos devem apenas definir armadilhas e deixar os sinalizadores limpos.

Emin

Emax

Números inteiros que especificam os limites externos permitidos para expoentes. *Emin* deve estar no intervalo [*MIN_EMIN*, 0], *Emax* no intervalo [0, *MAX_EMAX*].

capitals

Assume 0 ou 1 (o padrão). Se definido como 1, os expoentes serão impressos com um E maiúsculo; caso contrário, um e minúsculo é usado: `Decimal('6.02e+23')`.

clamp

Assume 0 (o padrão) ou 1. Se definido como 1, o expoente e de uma instância de *Decimal* representável nesse contexto é estritamente limitado ao intervalo `Emin - prec + 1 <= e <= Emax - prec + 1`. Se *clamp* for 0, uma condição mais fraca será mantida: o expoente ajustado da instância de *Decimal* é no máximo *Emax*. Quando *clamp* é 1, um grande número normal terá, sempre que possível, seu expoente reduzido e um número correspondente de zeros adicionado ao seu coeficiente, para ajustar as restrições do expoente; isso preserva o valor do número, mas perde informações sobre zeros à direita significativos. Por exemplo:

```
>>> Context(prec=6, Emax=999, clamp=1).create_decimal('1.23e999')
Decimal('1.23000E+999')
```

Um valor de *clamp* de 1 permite compatibilidade com os formatos de intercâmbio decimal de largura fixa especificados na IEEE 754.

A classe *Context* define vários métodos de uso geral, bem como um grande número de métodos para fazer aritmética diretamente em um determinado contexto. Além disso, para cada um dos métodos de *Decimal* descritos acima (com exceção dos métodos *adjusted()* e *as_tuple()*) existe um método correspondente em *Context*. Por exemplo, para uma instância C de *Context* e uma instância x de *Decimal*, `C.exp(x)`

é equivalente a `x.exp(context=C)`. Cada método de *Context* aceita um número inteiro do Python (uma instância de *int*) em qualquer lugar em que uma instância de Decimal seja aceita.

clear_flags()

Redefine todos os sinalizadores para `0`.

clear_traps()

Redefine todas as armadilhas para `0`.

Adicionado na versão 3.3.

copy()

Retorna uma duplicata do contexto.

copy_decimal(*num*, /)

Retorna uma cópia da instância de Decimal *num*.

create_decimal(*num='0'*, /)

Cria uma nova instância decimal a partir de *num*, mas usando *self* como contexto. Diferentemente do construtor de *Decimal*, a precisão do contexto, o método de arredondamento, os sinalizadores e as armadilhas são aplicadas à conversão.

Isso é útil porque as constantes geralmente são fornecidas com uma precisão maior do que a necessária pela aplicação. Outro benefício é que o arredondamento elimina imediatamente os efeitos indesejados dos dígitos além da precisão atual. No exemplo a seguir, o uso de entradas não arredondadas significa que adicionar zero a uma soma pode alterar o resultado:

```
>>> getcontext().prec = 3
>>> Decimal('3.4445') + Decimal('1.0023')
Decimal('4.45')
>>> Decimal('3.4445') + Decimal(0) + Decimal('1.0023')
Decimal('4.44')
```

Este método implementa a operação "to-number" da especificação IBM. Se o argumento for uma string, nenhum espaço em branco à esquerda ou à direita ou sublinhado serão permitidos.

create_decimal_from_float(*f*, /)

Cria uma nova instância de Decimal a partir de um ponto flutuante *f*, mas arredondando usando *self* como contexto. Diferentemente do método da classe *Decimal.from_float()*, a precisão do contexto, o método de arredondamento, os sinalizadores e as armadilhas são aplicados à conversão.

```
>>> context = Context(prec=5, rounding=ROUND_DOWN)
>>> context.create_decimal_from_float(math.pi)
Decimal('3.1415')
>>> context = Context(prec=5, traps=[Inexact])
>>> context.create_decimal_from_float(math.pi)
Traceback (most recent call last):
    ...
decimal.Inexact: None
```

Adicionado na versão 3.1.

Etiny()

Retorna um valor igual a `Emin - prec + 1`, que é o valor mínimo do expoente para resultados subnormais. Quando ocorre o estouro negativo, o expoente é definido como *Etiny*.

Etop()

Retorna um valor igual a `Emax - prec + 1`.

A abordagem usual para trabalhar com decimais é criar instâncias de *Decimal* e depois aplicar operações aritméticas que ocorrem no contexto atual da thread ativa. Uma abordagem alternativa é usar métodos de contexto para calcular dentro de um contexto específico. Os métodos são semelhantes aos da classe *Decimal* e são contados apenas brevemente aqui.

abs(*x*, /)

Retorna o valor absoluto de *x*.

add(*x*, *y*, /)

Retorna a soma de *x* e *y*.

canonical(*x*, /)

Retorna o mesmo objeto de Decimal *x*.

compare(*x*, *y*, /)

Compara *x* e *y* numericamente.

compare_signal(*x*, *y*, /)

Compara os valores dos dois operandos numericamente.

compare_total(*x*, *y*, /)

Compara dois operandos usando sua representação abstrata.

compare_total_mag(*x*, *y*, /)

Compara dois operandos usando sua representação abstrata, ignorando o sinal.

copy_abs(*x*, /)

Retorna uma cópia de *x* com o sinal definido para 0.

copy_negate(*x*, /)

Retorna uma cópia de *x* com o sinal invertido.

copy_sign(*x*, *y*, /)

Copia o sinal de *y* para *x*.

divide(*x*, *y*, /)

Retorna *x* dividido por *y*.

divide_int(*x*, *y*, /)

Retorna *x* dividido por *y*, truncado para um inteiro.

divmod(*x*, *y*, /)

Divide dois números e retorna a parte inteira do resultado.

exp(*x*, /)

Retorna `e ** x`.

fma(*x*, *y*, *z*, /)

Retorna *x* multiplicado por *y*, mais *z*.

is_canonical(*x*, /)

Retorna `True` se *x* for canonical; caso contrário, retorna `False`.

is_finite(*x*, /)

Retorna `True` se *x* for finito; caso contrário, retorna `False`.

is_infinite(*x*, /)

Retorna `True` se *x* for infinito; caso contrário, retorna `False`.

is_nan(*x*, /)

Retorna `True` se *x* for qNaN ou sNaN; caso contrário, retorna `False`.

is_normal(*x*, /)

Retorna `True` se *x* for um número normal; caso contrário, retorna `False`.

is_qnan(*x*, /)

Retorna `True` se *x* for um NaN silencioso; caso contrário, retorna `False`.

is_signed(*x*, /)

Retorna `True` se *x* for negativo; caso contrário, retorna `False`.

is_snan(*x*, /)

Retorna `True` se *x* for um NaN sinalizador; caso contrário, retorna `False`.

is_subnormal(*x*, /)

Retorna `True` se *x* for subnormal; caso contrário, retorna `False`.

is_zero(*x*, /)

Retorna `True` se *x* for zero; caso contrário, retorna `False`.

ln(*x*, /)

Retorna o logaritmo natural (base e) de *x*.

log10(*x*, /)

Retorna o logaritmo de base 10 de *x*.

logb(*x*, /)

Retorna o expoente da magnitude do MSD do operando.

logical_and(*x*, *y*, /)

Aplica a operação lógica *e* entre cada dígito do operando.

logical_invert(*x*, /)

Inverte todos os dígitos em *x*.

logical_or(*x*, *y*, /)

Aplica a operação lógica *ou* entre cada dígito do operando.

logical_xor(*x*, *y*, /)

Aplica a operação lógica *ou exclusivo* entre cada dígito do operando.

max(*x*, *y*, /)

Compara dois valores numericamente e retorna o máximo.

max_mag(*x*, *y*, /)

Compara dois valores numericamente com seu sinal ignorado.

min(*x*, *y*, /)

Compara dois valores numericamente e retorna o mínimo.

min_mag(*x*, *y*, /)

Compara dois valores numericamente com seu sinal ignorado.

minus(*x*, /)

Minus corresponde ao operador de subtração de prefixo unário no Python.

multiply(*x*, *y*, /)

Retorna o produto de *x* e *y*.

next_minus(*x*, /)

Retorna o maior número representável menor que *x*.

next_plus(*x*, /)

Retorna o menor número representável maior que *x*.

next_toward(*x*, *y*, /)

Retorna o número mais próximo a *x*, em direção a *y*.

normalize(*x*, /)

Reduz *x* para sua forma mais simples.

number_class(*x*, */*)

Retorna uma indicação da classe de *x*.

plus(*x*, */*)

Plus corresponde ao operador de soma de prefixo unário no Python. Esta operação aplica a precisão e o arredondamento do contexto, portanto *não* é uma operação de identidade.

power(*x*, *y*, *modulo=None*)

Retorna `x` à potência de `y`, com a redução de módulo `modulo` se fornecido.

Com dois argumentos, calcula `x**y`. Se `x` for negativo, `y` deve ser inteiro. O resultado será inexato, a menos que `y` seja inteiro e o resultado seja finito e possa ser expresso exatamente em "precisão" dígitos. O modo de arredondamento do contexto é usado. Os resultados são sempre arredondados corretamente na versão Python.

`Decimal(0) ** Decimal(0)` resulta em `InvalidOperation`, e se `InvalidOperation` não for capturado, resulta em `Decimal('NaN')`.

Alterado na versão 3.3: O módulo C calcula *power()* em termos das funções corretamente arredondadas *exp()* e *ln()*. O resultado é bem definido, mas apenas "quase sempre corretamente arredondado".

Com três argumentos, calcula `(x**y) % modulo`. Para o formulário de três argumentos, as seguintes restrições nos argumentos são válidas:

- todos os três argumentos devem ser inteiros
- `y` não pode ser negativo
- pelo menos um de `x` ou `y` não pode ser negativo
- `modulo` não pode ser zero e deve ter pelo menos "precisão" dígitos

O valor resultante de `Context.power(x, y, modulo)` é igual ao valor que seria obtido ao computar `(x**y) % modulo` com precisão ilimitada, mas é calculado com mais eficiência . O expoente do resultado é zero, independentemente dos expoentes de `x`, `y` e `modulo`. O resultado é sempre exato.

quantize(*x*, *y*, */*)

Retorna um valor igual a *x* (arredondado), com o expoente de *y*.

radix()

Só retorna 10, já que isso é Decimal, :)

remainder(*x*, *y*, */*)

Retorna o resto da divisão inteira.

O sinal do resultado, se diferente de zero, é o mesmo que o do dividendo original.

remainder_near(*x*, *y*, */*)

Retorna `x - y * n`, onde *n* é o número inteiro mais próximo do valor exato de `x / y` (se o resultado for 0, seu sinal será o sinal de *x*).

rotate(*x*, *y*, */*)

Retorna uma cópia re de *x*, *y* vezes.

same_quantum(*x*, *y*, */*)

Retorna `True` se os dois operandos tiverem o mesmo expoente.

scaleb(*x*, *y*, */*)

Retorna o primeiro operando após adicionar o segundo valor seu exp.

shift(*x*, *y*, */*)

Retorna uma cópia deslocada de *x*, *y* vezes.

sqrt(*x*, */*)

Raiz quadrada de um número não negativo para precisão do contexto.

subtract(*x*, *y*, /)

Retorna a diferença entre *x* e *y*.

to_eng_string(*x*, /)

Converte em uma string, usando notação de engenharia, se for necessário um expoente.

A notação de engenharia possui um expoente que é múltiplo de 3. Isso pode deixar até 3 dígitos à esquerda da casa decimal e pode exigir a adição de um ou dois zeros à direita.

to_integral_exact(*x*, /)

Arredonda para um número inteiro.

to_sci_string(*x*, /)

Converte um número em uma string usando notação científica.

9.4.4 Constantes

As constantes nesta seção são relevantes apenas para o módulo C. Eles também estão incluídos na versão pura do Python para compatibilidade.

	32 bits	64 bits
decimal.**MAX_PREC**	425000000	999999999999999999
decimal.**MAX_EMAX**	425000000	999999999999999999
decimal.**MIN_EMIN**	-425000000	-999999999999999999
decimal.**MIN_ETINY**	-849999999	-1999999999999999997
decimal.**IEEE_CONTEXT_MAX_BITS**	256	512

decimal.**HAVE_THREADS**

O valor é `True`. Descontinuado porque o Python agora sempre tem threads.

Descontinuado desde a versão 3.9.

decimal.**HAVE_CONTEXTVAR**

O valor padrão é `True`. Se o Python for `configurado usando a opção --without-decimal-contextvar`, a versão C usará um contexto local de thread em vez de local de corrotina e o valor será `False`. Isso é um pouco mais rápido em alguns cenários de contexto aninhados.

Adicionado na versão 3.8.3.

9.4.5 Modos de arredondamento

decimal.**ROUND_CEILING**

Arredonda para `Infinity`.

decimal.**ROUND_DOWN**

Arredonda para zero.

decimal.**ROUND_FLOOR**

Arredonda para `-Infinity`.

decimal.**ROUND_HALF_DOWN**

Arrendonda para o mais próximo com empates tendendo a zero.

decimal.**ROUND_HALF_EVEN**

Arredonda para o mais próximo com empates indo para o mais próximo inteiro par.

decimal.**ROUND_HALF_UP**

Arrendonda para o mais próximo com empates se afastando de zero.

decimal.**ROUND_UP**

Arredonda se afastando de zero.

decimal.**ROUND_05UP**

Arredonda se afastando de zero se o último dígito após o arredondamento para zero fosse 0 ou 5; caso contrário, arredonda para zero.

9.4.6 Sinais

Sinais representam condições que surgem durante o cálculo. Cada um corresponde a um sinalizador de contexto e um ativador de armadilha de contexto.

O sinalizador de contexto é definido sempre que a condição é encontrada. Após o cálculo, os sinalizadores podem ser verificados para fins informativos (por exemplo, para determinar se um cálculo era exato). Depois de verificar os sinalizadores, certifique-se de limpar todos os sinalizadores antes de iniciar o próximo cálculo.

Se o ativador de armadilha de contexto estiver definido para o sinal, a condição fará com que uma exceção Python seja levantada. Por exemplo, se a armadilha *DivisionByZero* for configurada, uma exceção *DivisionByZero* será levantada ao encontrar a condição.

class decimal.**Clamped**

Altera um expoente para ajustar as restrições de representação.

Normalmente, *clamping* ocorre quando um expoente fica fora dos limites do contexto *Emin* e *Emax*. Se possível, o expoente é reduzido para caber adicionando zeros ao coeficiente.

class decimal.**DecimalException**

Classe base para outros sinais e uma subclasse de *ArithmeticError*.

class decimal.**DivisionByZero**

Sinaliza a divisão de um número não infinito por zero.

Pode ocorrer com divisão, divisão de módulo ou ao elevar um número a uma potência negativa. Se este sinal não for capturado, retornará `Infinity` ou `-Infinity` com o sinal determinado pelas entradas do cálculo.

class decimal.**Inexact**

Indica que o arredondamento ocorreu e o resultado não é exato.

Sinaliza quando dígitos diferentes de zero foram descartados durante o arredondamento. O resultado arredondado é retornado. O sinalizador ou armadilha de sinal é usado para detectar quando os resultados são inexatos.

class decimal.**InvalidOperation**

Uma operação inválida foi realizada.

Indica que uma operação foi solicitada que não faz sentido. Se não for capturado, retorna `NaN`. As possíveis causas incluem:

```
Infinity - Infinity
0 * Infinity
Infinity / Infinity
```

(continua na próxima página)

(continuação da página anterior)

```
x % 0
Infinity % x
sqrt(-x) and x > 0
0 ** 0
x ** (non-integer)
x ** Infinity
```

class decimal.**Overflow**

Estouro numérico.

Indica que o expoente é maior que *Context.Emax* após o arredondamento ocorrer. Se não for capturado, o resultado depende do modo de arredondamento, puxando para dentro para o maior número finito representável ou arredondando para fora para `Infinity`. Nos dois casos, *Inexact* e *Rounded* também são sinalizados.

class decimal.**Rounded**

O arredondamento ocorreu, embora possivelmente nenhuma informação tenha sido perdida.

Sinalizado sempre que o arredondamento descarta dígitos; mesmo que esses dígitos sejam zero (como arredondamento `5.00` a `5.0`). Se não for capturado, retorna o resultado inalterado. Este sinal é usado para detectar a perda de dígitos significativos.

class decimal.**Subnormal**

O expoente foi menor que *Emin* antes do arredondamento.

Ocorre quando um resultado da operação é subnormal (o expoente é muito pequeno). Se não for capturado, retorna o resultado inalterado.

class decimal.**Underflow**

Estouro negativo numérico com resultado arredondado para zero.

Ocorre quando um resultado subnormal é empurrado para zero arredondando. *Inexact* e *Subnormal* também são sinalizados.

class decimal.**FloatOperation**

Ativa semânticas mais rigorosas para misturar objetos de float com de Decimal.

Se o sinal não for capturado (padrão), a mistura de tipos float e Decimal será permitida no construtor *Decimal*, *create_decimal()* e em todos os operadores de comparação. Tanto a conversão quanto as comparações são exatas. Qualquer ocorrência de uma operação mista é registrada silenciosamente pela configuração *FloatOperation* nos sinalizadores de contexto. Conversões explícitas com *from_float()* ou *create_decimal_from_float()* não definem o sinalizador.

Caso contrário (o sinal é capturado), apenas comparações de igualdade e conversões explícitas são silenciosas. Todas as outras operações mistas levantam *FloatOperation*.

A tabela a seguir resume a hierarquia de sinais:

```
exceptions.ArithmeticError(exceptions.Exception)
    DecimalException
        Clamped
        DivisionByZero(DecimalException, exceptions.ZeroDivisionError)
        Inexact
            Overflow(Inexact, Rounded)
            Underflow(Inexact, Rounded, Subnormal)
        InvalidOperation
        Rounded
        Subnormal
        FloatOperation(DecimalException, exceptions.TypeError)
```

9.4.7 Observações sobre ponto flutuante

Atenuando o erro de arredondamento com maior precisão

O uso do ponto flutuante decimal elimina o erro de representação decimal (possibilitando representar `0.1` de forma exata); no entanto, algumas operações ainda podem sofrer erros de arredondamento quando dígitos diferentes de zero excederem a precisão fixa.

Os efeitos do erro de arredondamento podem ser amplificados pela adição ou subtração de quantidades quase compensadoras, resultando em perda de significância. Knuth fornece dois exemplos instrutivos em que a aritmética de ponto flutuante arredondado com precisão insuficiente causa a quebra das propriedades associativas e distributivas da adição:

```
# exemplos de Seminumerical Algorithms, Section 4.2.2.
>>> from decimal import Decimal, getcontext
>>> getcontext().prec = 8

>>> u, v, w = Decimal(11111113), Decimal(-11111111), Decimal('7.51111111')
>>> (u + v) + w
Decimal('9.5111111')
>>> u + (v + w)
Decimal('10')

>>> u, v, w = Decimal(20000), Decimal(-6), Decimal('6.0000003')
>>> (u*v) + (u*w)
Decimal('0.01')
>>> u * (v+w)
Decimal('0.0060000')
```

O módulo *decimal* permite restaurar as identidades expandindo a precisão o suficiente para evitar perda de significância:

```
>>> getcontext().prec = 20
>>> u, v, w = Decimal(11111113), Decimal(-11111111), Decimal('7.51111111')
>>> (u + v) + w
Decimal('9.51111111')
>>> u + (v + w)
Decimal('9.51111111')
>>>
>>> u, v, w = Decimal(20000), Decimal(-6), Decimal('6.0000003')
>>> (u*v) + (u*w)
Decimal('0.0060000')
>>> u * (v+w)
Decimal('0.0060000')
```

Valores especiais

O sistema numérico para o módulo *decimal* fornece valores especiais, incluindo `NaN`, `sNaN`, `-Infinity`, `Infinity`, e dois zeros, `+0` e `-0`.

Os infinitos podem ser construídos diretamente com: `Decimal('Infinity')`. Além disso, eles podem resultar da divisão por zero quando o sinal *DivisionByZero* não é capturado. Da mesma forma, quando o sinal *Overflow* não é capturado, o infinito pode resultar do arredondamento além dos limites do maior número representável.

Os infinitos contêm sinais (afins) e podem ser usados em operações aritméticas, onde são tratados como números muito grandes e indeterminados. Por exemplo, adicionar uma constante ao infinito fornece outro resultado infinito.

Algumas operações são indeterminadas e retornam `NaN` ou, se o sinal *InvalidOperation* for capturado, levanta uma exceção. Por exemplo, `0/0` retorna `NaN`, que significa "não é um número" em inglês. Esta variação de `NaN` é silenciosa e, uma vez criada, fluirá através de outros cálculos sempre resultando em outra `NaN`. Esse comportamento

pode ser útil para uma série de cálculos que ocasionalmente têm entradas ausentes — ele permite que o cálculo continue enquanto sinaliza resultados específicos como inválidos.

Uma variante é `sNaN`, que sinaliza em vez de permanecer em silêncio após cada operação. Esse é um valor de retorno útil quando um resultado inválido precisa interromper um cálculo para tratamento especial.

O comportamento dos operadores de comparação do Python pode ser um pouco surpreendente onde um `NaN` está envolvido. Um teste de igualdade em que um dos operandos é um `NaN` silencioso ou sinalizador sempre retorna *`False`* (mesmo ao fazer `Decimal('NaN')==Decimal('NaN')`), enquanto um teste de desigualdade sempre retorna *`True`*. Uma tentativa de comparar dois decimais usando qualquer um dos operadores <, <=, > ou >= levantará o sinal *`InvalidOperation`* se um dos operandos for um `NaN` e retorna *`False`* se esse sinal não for capturado. Observe que a especificação aritmética decimal geral não especifica o comportamento das comparações diretas; estas regras para comparações envolvendo a `NaN` foram retiradas do padrão IEEE 854 (consulte a Tabela 3 na seção 5.7). Para garantir uma rígida conformidade com os padrões, use os métodos *`compare()`* e *`compare_signal()`*.

Os zeros com sinais podem resultar de cálculos insuficientes. Eles mantêm o sinal que teria resultado se o cálculo tivesse sido realizado com maior precisão. Como sua magnitude é zero, os zeros positivos e negativos são tratados como iguais e seu sinal é informacional.

Além dos dois zeros com sinais que são distintos e iguais, existem várias representações de zero com diferentes precisões e ainda com valor equivalente. Isso leva um pouco de tempo para se acostumar. Para um olho acostumado a representações de ponto flutuante normalizadas, não é imediatamente óbvio que o seguinte cálculo retorne um valor igual a zero:

```
>>> 1 / Decimal('Infinity')
Decimal('0E-1000026')
```

9.4.8 Trabalhando com threads

A função *`getcontext()`* acessa um objeto *`Context`* diferente para cada thread. Ter contextos de threads separadas significa que as threads podem fazer alterações (como `getcontext().prec=10`) sem interferir em outras threads.

Da mesma forma, a função *`setcontext()`* atribui automaticamente seu alvo à thread atual.

Se *`setcontext()`* não tiver sido chamado antes de *`getcontext()`*, então *`getcontext()`* criará automaticamente um novo contexto para uso na thread atual. Novos objetos de contexto têm valores padrão definidos a partir do objeto *`decimal.DefaultContext`*.

O sinalizador *`sys.flags.thread_inherit_context`* afeta o contexto de novas threads. Se o sinalizador for falso, as novas threads começarão com um contexto vazio. Nesse caso, *`getcontext()`* criará um novo objeto de contexto quando chamado e usará os valores padrão de *DefaultContext*. Se o sinalizador for verdadeiro, as novas threads começarão com uma cópia do contexto do chamador de *`threading.Thread.start()`*.

Para controlar os padrões para que cada thread, use os mesmos valores em toda a aplicação, modifique diretamente o objeto *DefaultContext*. Isso deve ser feito *antes* de qualquer thread ser iniciada, para que não haja uma condição de corrida entre as threads chamando *`getcontext()`*. Por exemplo:

```
# Define padrões para todo a aplicação para todas as threads prestes a serem␣
↪iniciadas
DefaultContext.prec = 12
DefaultContext.rounding = ROUND_DOWN
DefaultContext.traps = ExtendedContext.traps.copy()
DefaultContext.traps[InvalidOperation] = 1
setcontext(DefaultContext)

# Depois, as threads podem ser iniciadas
t1.start()
t2.start()
t3.start()
 . . .
```

9.4.9 Receitas

Aqui estão algumas receitas que servem como funções utilitárias e que demonstram maneiras de trabalhar com a classe *Decimal*:

```
def moneyfmt(value, places=2, curr='', sep=',', dp='.',
             pos='', neg='-', trailneg=''):
    """Converte Decimal para uma string formatada de moeda.

    places:  número obrigatório de casas após o ponto decimal
    curr:    símbolo de moeda opcional antes do sinal (pode estar em branco)
    sep:     separador de agrupamento opcional (vírgula, ponto, espaço ou espaço␣
↪em branco)
    dp:      indicador de ponto decimal (vírgula ou ponto)
             especificar como espaço em branco somente quando places for zero
    pos:     sinal opcional para números positivos: '+', espaço ou espaço em branco
    neg:     sinal opcional para números negativos: '-', '(', espaço ou espaço em␣
↪branco
    trailneg:indicador de menos final opcional: '-', ')', espaço ou espaço em␣
↪branco

    >>> d = Decimal('-1234567.8901')
    >>> moneyfmt(d, curr='$')
    '-$1,234,567.89'
    >>> moneyfmt(d, places=0, sep='.', dp='', neg='', trailneg='-')
    '1.234.568-'
    >>> moneyfmt(d, curr='$', neg='(', trailneg=')')
    '($1,234,567.89)'
    >>> moneyfmt(Decimal(123456789), sep=' ')
    '123 456 789.00'
    >>> moneyfmt(Decimal('-0.02'), neg='<', trailneg='>')
    '<0.02>'

    """
    q = Decimal(10) ** -places      # 2 casas --> '0.01'
    sign, digits, exp = value.quantize(q).as_tuple()
    result = []
    digits = list(map(str, digits))
    build, next = result.append, digits.pop
    if sign:
        build(trailneg)
    for i in range(places):
        build(next() if digits else '0')
    if places:
        build(dp)
    if not digits:
        build('0')
    i = 0
    while digits:
        build(next())
        i += 1
        if i == 3 and digits:
            i = 0
            build(sep)
    build(curr)
    build(neg if sign else pos)
    return ''.join(reversed(result))
```

(continua na próxima página)

(continuação da página anterior)

```
def pi():
    """Calcula Pi com a precisão atual.

    >>> print(pi())
    3.141592653589793238462643383

    """
    getcontext().prec += 2  # dígitos extras para casas intermediárias
    three = Decimal(3)      # substitui "three=3.0" para pontos flutuantes␣
↪regulares
    lasts, t, s, n, na, d, da = 0, three, 3, 1, 0, 0, 24
    while s != lasts:
        lasts = s
        n, na = n+na, na+8
        d, da = d+da, da+32
        t = (t * n) / d
        s += t
    getcontext().prec -= 2
    return +s               # unário com mais aplica a nova precisão

def exp(x):
    """Retorna e elevado à potência de x. O tipo de resultado corresponde ao tipo␣
↪de entrada.

    >>> print(exp(Decimal(1)))
    2.718281828459045235360287471
    >>> print(exp(Decimal(2)))
    7.389056098930650227230427461
    >>> print(exp(2.0))
    7.38905609893
    >>> print(exp(2+0j))
    (7.38905609893+0j)

    """
    getcontext().prec += 2
    i, lasts, s, fact, num = 0, 0, 1, 1, 1
    while s != lasts:
        lasts = s
        i += 1
        fact *= i
        num *= x
        s += num / fact
    getcontext().prec -= 2
    return +s

def cos(x):
    """Retorna o cosseno de x medido em radianos.

    A aproximação da série de Taylor funciona melhor para um valor pequeno de x.
    Para valores maiores, primeiro calcule x = x % (2 * pi).

    >>> print(cos(Decimal('0.5')))
    0.8775825618903727161162815826
    >>> print(cos(0.5))
    0.87758256189
    >>> print(cos(0.5+0j))
```

(continua na próxima página)

(continuação da página anterior)

```
    (0.87758256189+0j)

    """
    getcontext().prec += 2
    i, lasts, s, fact, num, sign = 0, 0, 1, 1, 1, 1
    while s != lasts:
        lasts = s
        i += 2
        fact *= i * (i-1)
        num *= x * x
        sign *= -1
        s += num / fact * sign
    getcontext().prec -= 2
    return +s

def sin(x):
    """Retorna o seno de x medido em radianos.

    A aproximação da série de Taylor funciona melhor para um valor pequeno de x.
    Para valores maiores, primeiro calcule x = x % (2 * pi).

    >>> print(sin(Decimal('0.5')))
    0.47942553860420300027328793521
    >>> print(sin(0.5))
    0.479425538604
    >>> print(sin(0.5+0j))
    (0.479425538604+0j)

    """
    getcontext().prec += 2
    i, lasts, s, fact, num, sign = 1, 0, x, 1, x, 1
    while s != lasts:
        lasts = s
        i += 2
        fact *= i * (i-1)
        num *= x * x
        sign *= -1
        s += num / fact * sign
    getcontext().prec -= 2
    return +s
```

9.4.10 FAQ sobre Decimal

P. É complicado digitar `decimal.Decimal('1234.5')`. Existe uma maneira de minimizar a digitação ao usar o interpretador interativo?

R. Alguns usuários abreviam o construtor para apenas uma única letra:

```
>>> D = decimal.Decimal
>>> D('1.23') + D('3.45')
Decimal('4.68')
```

P. Em uma aplicação de ponto fixo com duas casas decimais, algumas entradas têm muitas casas e precisam ser arredondadas. Outros não devem ter dígitos em excesso e precisam ser validados. Quais métodos devem ser usados?

R. O método *quantize()* arredonda para um número fixo de casas decimais. Se a armadilha *Inexact* estiver configurada, também será útil para validação:

```
>>> TWOPLACES = Decimal(10) ** -2       # same as Decimal('0.01')
```

```
>>> # Round to two places
>>> Decimal('3.214').quantize(TWOPLACES)
Decimal('3.21')
```

```
>>> # Validate that a number does not exceed two places
>>> Decimal('3.21').quantize(TWOPLACES, context=Context(traps=[Inexact]))
Decimal('3.21')
```

```
>>> Decimal('3.214').quantize(TWOPLACES, context=Context(traps=[Inexact]))
Traceback (most recent call last):
   ...
Inexact: None
```

P. Assim que eu tiver entradas de duas casas válidas, como mantenho essa invariante em uma aplicação?

R. Algumas operações como adição, subtração e multiplicação por um número inteiro preservam automaticamente o ponto fixo. Outras operações, como divisão e multiplicação não inteira, alteram o número de casas decimais e precisam ser seguidas com uma etapa *quantize()*:

```
>>> a = Decimal('102.72')           # Initial fixed-point values
>>> b = Decimal('3.17')
>>> a + b                           # Addition preserves fixed-point
Decimal('105.89')
>>> a - b
Decimal('99.55')
>>> a * 42                          # So does integer multiplication
Decimal('4314.24')
>>> (a * b).quantize(TWOPLACES)     # Must quantize non-integer multiplication
Decimal('325.62')
>>> (b / a).quantize(TWOPLACES)     # And quantize division
Decimal('0.03')
```

No desenvolvimento de aplicações de ponto fixo, é conveniente definir funções para manipular a etapa *quantize()*:

```
>>> def mul(x, y, fp=TWOPLACES):
...     return (x * y).quantize(fp)
...
>>> def div(x, y, fp=TWOPLACES):
...     return (x / y).quantize(fp)
```

```
>>> mul(a, b)                       # Automatically preserve fixed-point
Decimal('325.62')
>>> div(b, a)
Decimal('0.03')
```

P. Existem várias maneiras de expressar o mesmo valor. Os números `200`, `200.000`, `2E2` e `.02E+4` têm todos o mesmo valor em várias precisões. Existe uma maneira de transformá-los em um único valor canônico reconhecível?

R. O método *normalize()* mapeia todos os valores equivalentes para um único representativo:

```
>>> values = map(Decimal, '200 200.000 2E2 .02E+4'.split())
>>> [v.normalize() for v in values]
[Decimal('2E+2'), Decimal('2E+2'), Decimal('2E+2'), Decimal('2E+2')]
```

P. Quando ocorre o arredondamento em um cálculo?

R. Ocorre *após* o cálculo. A filosofia da especificação decimal é que os números são considerados exatos e criados independentemente do contexto atual. Eles podem até ter maior precisão do que o contexto atual. O processo de cálculo com essas entradas exatas e, em seguida, o arredondamento (ou outras operações de contexto) é aplicado ao *resultado* do cálculo:

```
>>> getcontext().prec = 5
>>> pi = Decimal('3.1415926535')    # Mais de 5 dígitos
>>> pi                              # Todos os dígitos são retidos
Decimal('3.1415926535')
>>> pi + 0                          # Arredondado após uma adição
Decimal('3.1416')
>>> pi - Decimal('0.00005')         # Subtrai números não arredondados e depois␣
↪arredonda
Decimal('3.1415')
>>> pi + 0 - Decimal('0.00005').    # Os valores intermediários são arredondados
Decimal('3.1416')
```

P. Alguns valores decimais sempre são exibidas com notação exponencial. Existe uma maneira de obter uma representação não exponencial?

R. Para alguns valores, a notação exponencial é a única maneira de expressar o número de casas significativas no coeficiente. Por exemplo, expressar `5.0E+3` como `5000` mantém o valor constante, mas não pode mostrar a significância de duas casa do original.

Se uma aplicação não se importa com o rastreamento da significância, é fácil remover o expoente e os zeros à direita, perdendo a significância, mas mantendo o valor inalterado:

```
>>> def remove_exponent(d):
...     return d.quantize(Decimal(1)) if d == d.to_integral() else d.normalize()
```

```
>>> remove_exponent(Decimal('5E+3'))
Decimal('5000')
```

P. Existe uma maneira de converter um float comum em um *Decimal*?

R. Sim, qualquer número de ponto flutuante binário pode ser expresso exatamente como um Decimal, embora uma conversão exata possa exigir mais precisão do que a intuição sugere:

```
>>> Decimal(math.pi)
Decimal('3.141592653589793115997963468544185161590576171875')
```

P. Em um cálculo complexo, como posso ter certeza de que não obtive um resultado falso devido à precisão insuficiente ou a anomalias de arredondamento.

R. O módulo decimal facilita o teste de resultados. Uma prática recomendada é executar novamente os cálculos usando maior precisão e com vários modos de arredondamento. Resultados amplamente diferentes indicam precisão insuficiente, problemas no modo de arredondamento, entradas mal condicionadas ou um algoritmo numericamente instável.

P. Notei que a precisão do contexto é aplicada aos resultados das operações, mas não às entradas. Há algo a observar ao misturar valores de diferentes precisões?

R. Sim. O princípio é que todos os valores são considerados exatos, assim como a aritmética desses valores. Somente os resultados são arredondados. A vantagem das entradas é que "o que você vê é o que você obtém". Uma desvantagem é que os resultados podem parecer estranhos se você esquecer que as entradas não foram arredondadas:

```
>>> getcontext().prec = 3
>>> Decimal('3.104') + Decimal('2.104')
Decimal('5.21')
>>> Decimal('3.104') + Decimal('0.000') + Decimal('2.104')
Decimal('5.20')
```

A solução é aumentar a precisão ou forçar o arredondamento das entradas usando a operação unária de mais:

```
>>> getcontext().prec = 3
>>> +Decimal('1.23456789')      # unary plus triggers rounding
Decimal('1.23')
```

Como alternativa, as entradas podem ser arredondadas na criação usando o método *Context.create_decimal()*:

```
>>> Context(prec=5, rounding=ROUND_DOWN).create_decimal('1.2345678')
Decimal('1.2345')
```

P. A implementação do CPython é rápida para números grandes?

A. Sim. Nas implementações CPython e PyPy3, as versões C/CFFI do módulo decimal integram a biblioteca de alta velocidade libmpdec para precisão arbitrária de aritmética de ponto flutuante decimal corretamente arredondado[1]. `libmpdec` usa a multiplicação de Karatsuba para números com tamanho médio e a Transformada Numérica de Fourier para números muito grandes.

O contexto deve ser adaptado para uma aritmética exata de precisão arbitrária. *Emin* e *Emax* devem sempre ser configurados com os valores máximos, *clamp* deve sempre ser 0 (o padrão). A configuração de *prec* requer alguns cuidados.

A abordagem mais fácil para testar a aritmética do bignum é usar o valor máximo para *prec* também[2]:

```
>>> setcontext(Context(prec=MAX_PREC, Emax=MAX_EMAX, Emin=MIN_EMIN))
>>> x = Decimal(2) ** 256
>>> x / 128
Decimal(
↪'904625697166532776746648320380374280103671755200316906558262375061821325312')
```

Para resultados inexatos, *MAX_PREC* é muito grande em plataformas de 64 bits e a memória disponível será insuficiente:

```
>>> Decimal(1) / 3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

Em sistemas com alocação excessiva (por exemplo, Linux), uma abordagem mais sofisticada é ajustar *prec* à quantidade de RAM disponível. Suponha que você tenha 8 GB de RAM e espere 10 operandos simultâneos usando no máximo 500 MB cada:

```
>>> import sys
>>>
>>> # Número máximo de dígitos para um único operando usando 500 MB
>>> # em palavras de 8 bytes com 19 dígitos por palavra (4 bytes e 9 dígitos
>>> # para a construção de 32 bits):
>>> maxdigits = 19 * ((500 * 1024**2) // 8)
>>>
>>> # Confere que isso funciona:
>>> c = Context(prec=maxdigits, Emax=MAX_EMAX, Emin=MIN_EMIN)
>>> c.traps[Inexact] = True
>>> setcontext(c)
>>>
>>> # Preenche a precisão disponível com noves:
```

(continua na próxima página)

[1] Adicionado na versão 3.3.

[2] Alterado na versão 3.9: Esta abordagem agora funciona para todos os resultados exatos, exceto para potências de números que não sejam inteiros.

(continuação da página anterior)

```
>>> x = Decimal(0).logical_invert() * 9
>>> sys.getsizeof(x)
524288112
>>> x + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  decimal.Inexact: [<class 'decimal.Inexact'>]
```

Em geral (e especialmente em sistemas sem alocação excessiva), recomenda-se estimar limites ainda mais apertados e definir a armadilha *Inexact* se for esperado que todos os cálculos sejam mais precisos.

9.5 `fractions` — Números racionais

Código-fonte: Lib/fractions.py

O módulo *fractions* fornece suporte para aritmética de números racionais.

Uma instância de Fraction pode ser construída a partir de um par de números inteiros, de outro número racional ou de uma string.

class fractions.**Fraction**(*numerator=0*, *denominator=1*)

class fractions.**Fraction**(*number*)

class fractions.**Fraction**(*string*)

A primeira versão requer que *numerator* e *denominator* sejam instâncias de *numbers.Rational* e retorna uma nova instância de *Fraction* com o valor `numerator/denominator`. Se *denominador* for `0`, ele levanta uma *ZeroDivisionError*.

A segunda versão requer que *number* seja uma instância de *numbers.Rational* ou tenha o método `as_integer_ratio()` (isso inclui *float* e *decimal.Decimal*). Ela retorna uma instância de *Fraction* com exatamente o mesmo valor. É presumido que o método `as_integer_ratio()` retorne um par de inteiros primos entre si e que o último seja positivo. Observe que, devido aos problemas comuns com pontos binários (consulte tut-fp-issues), o argumento para `Fraction(1.1)` não é exatamente igual a 11/10 e, portanto, `Fraction(1.1)` *não* retorna `Fraction(11, 10)` como seria de se esperar. (Mas consulte a documentação do método *limit_denominator()* abaixo.)

A última versão do construtor espera uma string. O formato usual para esta instância é:

```
[sinal] numerador ['/' denominador]
```

onde o `sign` opcional pode ser '+' ou '-' e `numerator` e `denominator` (se presente) são strings de dígitos decimais (sublinhados podem ser usados para delimitar dígitos como com literais integrais no código). Além disso, qualquer string que represente um valor finito e seja aceita pelo construtor *float* também é aceita pelo construtor *Fraction*. Em qualquer forma, a string de entrada também pode ter espaços em branco à esquerda e/ou à direita. Aqui estão alguns exemplos:

```
>>> from fractions import Fraction
>>> Fraction(16, -10)
Fraction(-8, 5)
>>> Fraction(123)
Fraction(123, 1)
>>> Fraction()
Fraction(0, 1)
>>> Fraction('3/7')
Fraction(3, 7)
>>> Fraction(' -3/7 ')
Fraction(-3, 7)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> Fraction('1.414213 \t\n')
Fraction(1414213, 1000000)
>>> Fraction('-.125')
Fraction(-1, 8)
>>> Fraction('7e-6')
Fraction(7, 1000000)
>>> Fraction(2.25)
Fraction(9, 4)
>>> Fraction(1.1)
Fraction(2476979795053773, 2251799813685248)
>>> from decimal import Decimal
>>> Fraction(Decimal('1.1'))
Fraction(11, 10)
```

A classe *Fraction* herda da classe base abstrata *numbers.Rational* e implementa todos os métodos e operações dessa classe. As instâncias de *Fraction* são *hasheável* e devem ser tratadas como imutáveis. Além disso, *Fraction* tem as seguintes propriedades e métodos:

Alterado na versão 3.2: O construtor *Fraction* agora aceita instâncias *float* e *decimal.Decimal*.

Alterado na versão 3.9: A função *math.gcd()* agora é usada para normalizar o *numerator* e o *denominator*. *math.gcd()* sempre retorna um tipo *int*. Anteriormente, o tipo GCD dependia do *numerator* e do *denominator*.

Alterado na versão 3.11: Sublinhados agora são permitidos ao criar uma instância *Fraction* a partir de uma string, seguindo as regras **PEP 515**.

Alterado na versão 3.11: *Fraction* implementa `__int__` agora para satisfazer verificações de instância `typing.SupportsInt`.

Alterado na versão 3.12: É permitido espaço ao redor da barra para entradas de string: `Fraction('2 / 3')`.

Alterado na versão 3.12: Instâncias de *Fraction* agora oferecem suporte à formatação no estilo float, com tipos de apresentação `"e"`, `"E"`, `"f"`, `"F"`, `"g"`, `"G"` e `"%"`.

Alterado na versão 3.13: A formatação de instâncias *Fraction* sem um tipo de apresentação agora oferece suporte a preenchimento, alinhamento, tratamento de sinais, largura mínima e agrupamento.

Alterado na versão 3.14: O construtor *Fraction* agora aceita qualquer objeto com o método `as_integer_ratio()`.

numerator

Numerador de Fraction no menor termo.

denominator

Denominador de Fraction no menor termo.

as_integer_ratio()

Retorna uma tupla de dois inteiros, cuja razão é igual a Fraction original. A razão está em termos mais baixos e tem um denominador positivo.

Adicionado na versão 3.8.

is_integer()

Retorna `True` se a Fraction for um inteiro.

Adicionado na versão 3.12.

classmethod from_float(*f*)

Construtor alternativo que aceita apenas instâncias de *float* ou *numbers.Integral*. Esteja ciente de que `Fraction.from_float(0.3)` não é o mesmo valor que `Fraction(3, 10)`.

> **Nota**
>
> A partir do Python 3.2, você também pode construir uma instância *Fraction* diretamente de um *float*.

classmethod from_decimal(*dec*)

Construtor alternativo que aceita somente instâncias de *decimal.Decimal* ou *numbers.Integral*.

> **Nota**
>
> A partir do Python 3.2, você também pode construir uma instância *Fraction* diretamente de uma instância de *decimal.Decimal*.

classmethod from_number(*number*)

Construtor alternativo que aceita apenas instâncias de *numbers.Integral*, *numbers.Rational*, *float* ou *decimal.Decimal* e objetos com o método `as_integer_ratio()`, mas não strings.

Adicionado na versão 3.14.

limit_denominator(*max_denominator=1000000*)

Encontra e retorna o *Fraction* mais próximo de `self` que tem denominador no máximo max_denominator. Este método é útil para encontrar aproximações racionais para um dado número de ponto flutuante:

```
>>> from fractions import Fraction
>>> Fraction('3.1415926535897932').limit_denominator(1000)
Fraction(355, 113)
```

ou para recuperar um número racional que é representado como um ponto flutuante:

```
>>> from math import pi, cos
>>> Fraction(cos(pi/3))
Fraction(4503599627370497, 9007199254740992)
>>> Fraction(cos(pi/3)).limit_denominator()
Fraction(1, 2)
>>> Fraction(1.1).limit_denominator()
Fraction(11, 10)
```

__floor__()

Retorna o maior *int* `<= self`. Este método também pode ser acessado por meio da função *math.floor()*:

```
>>> from math import floor
>>> floor(Fraction(355, 113))
3
```

__ceil__()

Retorna o menor *int* `>= self`. Este método também pode ser acessado por meio da função *math.ceil()*.

__round__()

__round__(*ndigits*)

A primeira versão retorna o *int* mais próximo de `self`, arredondando a metade para par. A segunda versão arredonda `self` para o múltiplo mais próximo de `Fraction(1, 10**ndigits)` (logicamente, se `ndigits` for negativo), novamente arredondando a metade para par. Este método também pode ser acessado por meio da função *round()*.

__format__ (*format_spec*, /)

Fornece suporte para formatação de instâncias de *Fraction* por meio do método *str.format()*, da função embutida *format()* ou literais de strings formatadas.

Se a string de especificação de formato `format_spec` não terminar com um dos tipos de apresentação `'e'`, `'E'`, `'f'`, `'F'`, `'g'`, `'G'` ou `'%'`, então a formatação segue as regras gerais para preenchimento, alinhamento, tratamento de sinais, largura mínima e agrupamento, conforme descrito na *minilinguagem de especificação de formato*. O sinalizador de "formato alternativo" `'#'` é suportado: se presente, ele força a string de saída a sempre incluir um denominador explícito, mesmo quando o valor que está sendo formatado é um inteiro exato. O sinalizador de preenchimento de zeros `'0'` não é suportado.

Se a string de especificação de formato `format_spec` terminar com um dos tipos de apresentação `'e'`, `'E'`, `'f'`, `'F'`, `'g'`, `'G'` ou `'%'`, então a formatação segue as regras descritas para o tipo *float* na seção *Minilinguagem de especificação de formato*.

Veja alguns exemplos:

```
>>> from fractions import Fraction
>>> format(Fraction(103993, 33102), '_')
'103_993/33_102'
>>> format(Fraction(1, 7), '.^+10')
'...+1/7...'
>>> format(Fraction(3, 1), '')
'3'
>>> format(Fraction(3, 1), '#')
'3/1'
>>> format(Fraction(1, 7), '.40g')
'0.1428571428571428571428571428571428571429'
>>> format(Fraction('1234567.855'), '_.2f')
'1_234_567.86'
>>> f"{Fraction(355, 113):*>20.6e}"
'********3.141593e+00'
>>> old_price, new_price = 499, 672
>>> "{:.2%} price increase".format(Fraction(new_price, old_price) - 1)
'34.67% price increase'
```

Ver também

Módulo *numbers*
As classes base abstratas que compõem a torre numérica.

9.6 `random` — Gera números pseudoaleatórios

Código-fonte: Lib/random.py

Este módulo implementa geradores de números pseudoaleatórios para várias distribuições.

Para números inteiros, há uma seleção uniforme de um intervalo. Para sequências, há uma seleção uniforme de um elemento aleatório, uma função para gerar uma permutação aleatória de uma lista localmente e uma função para amostragem aleatória sem substituição.

Na linha real, existem funções para calcular distribuições uniforme, normal (gaussiana), log-normal, exponencial negativa, gama e beta. Para gerar distribuições de ângulos, a distribuição de von Mises está disponível.

Quase todas as funções do módulo dependem da função básica *random()*, que gera um ponto flutuante aleatório uniformemente no intervalo semiaberto `0.0 <= X < 1.0`. Python usa o Mersenne Twister como gerador de núcleo. Produz pontos flutuantes de precisão de 53 bits e possui um período de 2**19937-1. A implementação subjacente em

C é rápida e segura para threads. O Mersenne Twister é um dos geradores de números aleatórios mais amplamente testados existentes. No entanto, sendo completamente determinístico, não é adequado para todos os fins e é totalmente inadequado para fins criptográficos.

As funções fornecidas por este módulo são, na verdade, métodos vinculados de uma instância oculta da classe `random.Random`. Você pode instanciar suas próprias instâncias de `Random` para obter geradores que não compartilham estado.

A classe `Random` também pode ser subclassificada se você quiser usar um gerador básico diferente de sua própria criação: veja a documentação dessa classe para mais detalhes.

O módulo `random` também fornece a classe `SystemRandom` que usa a função do sistema `os.urandom()` para gerar números aleatórios a partir de fontes fornecidas pelo sistema operacional.

Aviso

Os geradores pseudoaleatórios deste módulo não devem ser usados para fins de segurança. Para segurança ou uso criptográfico, veja o módulo `secrets`.

Ver também

M. Matsumoto and T. Nishimura, "Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator", ACM Transactions on Modeling and Computer Simulation Vol. 8, No. 1, January pp.3–30 1998.

Receita de Complementary-Multiply-with-Carry para um gerador de números aleatórios alternativo compatível com um longo período e operações de atualização comparativamente simples.

Nota

O gerador global de números aleatórios e as instâncias de `Random` são seguros para thread. No entanto, na construção com threads livres, chamadas simultâneas ao gerador global ou à mesma instância de `Random` podem encontrar contenção e baixo desempenho. Considere usar instâncias separadas de `Random` por thread.

9.6.1 Funções de contabilidade

`random.seed(a=None, version=2)`

Inicializa o gerador de números aleatórios.

Se *a* for omitido ou `None`, a hora atual do sistema será usada. Se fontes de aleatoriedade são fornecidas pelo sistema operacional, elas são usadas no lugar da hora do sistema (consulte a função `os.urandom()` para detalhes sobre disponibilidade).

Se *a* é um int, ele é usado diretamente.

Com a versão 2 (o padrão), o objeto a `str`, `bytes` ou `bytearray` é convertido em um objeto `int` e todos os seus bits são usados.

Com a versão 1 (fornecida para reproduzir sequências aleatórias de versões mais antigas do Python), o algoritmo para `str` e `bytes` gera um intervalo mais restrito de sementes.

Alterado na versão 3.2: Movido para o esquema da versão 2, que usa todos os bits em uma semente de strings.

Alterado na versão 3.11: *seed* deve ser um dos seguintes tipos: `None`, `int`, `float`, `str`, `bytes` ou `bytearray`.

`random.getstate()`

Retorna um objeto capturando o estado interno atual do gerador. Este objeto pode ser passado para `setstate()` para restaurar o estado.

random.**setstate**(*state*)

state deveria ter sido obtido de uma chamada anterior para *getstate()*, e *setstate()* restaura o estado interno do gerador para o que era no momento que *getstate()* foi chamado.

9.6.2 Funções para bytes

random.**randbytes**(*n*)

Gera *n* bytes aleatórios.

Este método não deve ser usado para gerar tokens de segurança. Use *secrets.token_bytes()*.

Adicionado na versão 3.9.

9.6.3 Funções para inteiros

random.**randrange**(*stop*)

random.**randrange**(*start*, *stop*[, *step*])

Retorna um elemento selecionado aleatoriamente de `range(start, stop, step)`.

Isso é aproximadamente equivalente a `choice(range(start, stop, step))` mas provê suporte a intervalos arbitrariamente grandes e é otimizado para casos comuns.

O padrão de argumento posicional corresponde à função *range()*.

Argumentos nomeados não devem ser usados porque podem ser interpretados de maneiras inesperadas. Por exemplo `randrange(start=100)` é interpretado como `randrange(0, 100, 1)`.

Alterado na versão 3.2: *randrange()* é mais sofisticado em produzir valores igualmente distribuídos. Anteriormente, usava um estilo como `int(random()*n)`, que poderia produzir distribuições ligeiramente desiguais.

Alterado na versão 3.12: A conversão automática de tipos não inteiros não é mais suportada. Chamadas como `randrange(10.0)` e `randrange(Fraction(10, 1))` agora levantam uma *TypeError*.

random.**randint**(*a*, *b*)

Retorna um inteiro aleatório *N* de forma que `a <= N <= b`. Apelido para `randrange(a, b+1)`.

random.**getrandbits**(*k*)

Retorna um número inteiro Python não-negativo com bits aleatórios *k*. Este método é fornecido com o gerador Mersenne Twister e alguns outros geradores também podem fornecê-lo como uma parte opcional da API. Quando disponível, *getrandbits()* permite que *randrange()* manipule intervalos arbitrariamente grandes.

Alterado na versão 3.9: Este método agora aceita zero em *k*.

9.6.4 Funções para sequências

random.**choice**(*seq*)

Retorna um elemento aleatório da sequência não vazia *seq*. Se *seq* estiver vazio, levanta *IndexError*.

random.**choices**(*population*, *weights=None*, ***, *cum_weights=None*, *k=1*)

Retorna uma lista do tamanho de *k* de elementos escolhidos da *population* com substituição. Se a *population* estiver vazio, levanta *IndexError*.

Se uma sequência *weights* for especificada, as seleções serão feitas de acordo com os pesos relativos. Alternativamente, se uma sequência *cum_weights* for fornecida, as seleções serão feitas de acordo com os pesos cumulativos (talvez calculados usando *itertools.accumulate()*). Por exemplo, os pesos relativos `[10, 5, 30, 5]` são equivalentes aos pesos cumulativos `[10, 15, 45, 50]`. Internamente, os pesos relativos são convertidos em pesos acumulados antes de fazer seleções, portanto, fornecer pesos cumulativos economiza trabalho.

Se nem *weights* nem *cum_weights* forem especificados, as seleções serão feitas com igual probabilidade. Se uma sequência de pesos for fornecida, ela deverá ter o mesmo comprimento que a sequência *population*. É um `TypeError` para especificar ambos os *weights* e *cum_weights*.

weights ou *cum_weights* pode usar qualquer tipo numérico que interopera com os valores `float` retornados por `random()` (que inclui inteiros, pontos flutuantes, e frações mas exclui decimais). Presume-se que pesos serão não-negativos e finitos. Uma `ValueError` é levantada se todos os pesos forem zero.

Para uma dada semente, a função `choices()` com igual peso normalmente produz uma sequência diferente das chamadas repetidas para `choice()`. O algoritmo usado por `choice()` usa aritmética de ponto flutuante para consistência e velocidade internas. O algoritmo usado por `choice()` assume como padrão aritmética inteira com seleções repetidas para evitar pequenos vieses de erro de arredondamento.

Adicionado na versão 3.6.

Alterado na versão 3.9: Levanta uma `ValueError` se todos os pesos forem zero.

`random.`**`shuffle`**(*x*)

Embaralha a sequência *x* internamente.

Para embaralhar uma sequência imutável e retornar uma nova lista embaralhada, use `sample(x, k=len(x))`.

Observe que, mesmo para pequenos `len(x)`, o número total de permutações de *x* pode crescer rapidamente maior que o período da maioria dos geradores de números aleatórios. Isso implica que a maioria das permutações de uma longa sequência nunca pode ser gerada. Por exemplo, uma sequência de comprimento 2080 é a maior que pode caber no período do gerador de números aleatórios Mersenne Twister.

Alterado na versão 3.11: Removido o parâmetro opcional *random*.

`random.`**`sample`**(*population*, *k*, *, *counts=None*)

Retorna uma lista de comprimento *k* de elementos únicos escolhidos na sequência populacional. Usado para amostragem aleatória sem reposição.

Retorna uma nova lista contendo elementos da população, mantendo a população original inalterada. A lista resultante está na ordem de seleção, para que todas as subfatias também sejam amostras aleatórias válidas. Isso permite que os vencedores do sorteio (a amostra) sejam divididos em grandes prêmios e vencedores de segundo lugar (as subfatias).

Os membros da população não precisam ser *hasheáveis* ou únicos. Se a população contiver repetições, cada ocorrência é uma seleção possível na amostra.

Elementos repetidos podem ser especificados um de cada vez ou com o parâmetro somente-nomeado opcional *count*. Por exemplo, `sample(['red', 'blue'], counts=[4, 2], k=5)` é equivalente a `sample(['red', 'red', 'red', 'red', 'blue', 'blue'], k=5)`.

Para escolher uma amostra de um intervalo de números inteiros, use um objeto `range()` como argumento. Isso é especialmente rápido e com eficiência de espaço para amostragem de uma grande população: `sample(range(10000000), k=60)`.

Se o tamanho da amostra for maior que o tamanho da população, uma `ValueError` é levantada.

Alterado na versão 3.9: Adicionado o parâmetro *counts*.

Alterado na versão 3.11: *population* deve ser uma sequência. A conversão automática de conjuntos em listas não é mais suportada.

9.6.5 Distribuições discretas

A função a seguir gera uma distribuição discreta.

`random.`**`binomialvariate`**(*n=1*, *p=0.5*)

Distribuição binomial. Retorna o número de sucessos para *n* tentativas independentes com a probabilidade de sucesso em cada tentativa sendo *p*:

Matematicamente equivalente a:

```
sum(random() < p for i in range(n))
```

O número de tentativas *n* deve ser um número inteiro não negativo. A probabilidade de sucesso *p* deve estar entre `0.0 <= p <= 1.0`. O resultado é um número inteiro no intervalo `0 <= X <= n`.

Adicionado na versão 3.12.

9.6.6 Distribuições com valor real

As funções a seguir geram distribuições específicas com valor real. Os parâmetros de função são nomeados após as variáveis correspondentes na equação da distribuição, conforme usadas na prática matemática comum; a maioria dessas equações pode ser encontrada em qualquer texto estatístico.

`random.`**`random`**`()`

Retorna o próximo número aleatório de ponto flutuante no intervalo `0.0 <= X < 1.0`.

`random.`**`uniform`**`(`*a*, *b*`)`

Retorna um número de ponto flutuante aleatório *N* de forma que `a <= N <= b` para `a <= b` e `b <= N <= a` para `b < a`.

O valor final `b` pode ou não ser incluído no intervalo dependendo do arredondamento de ponto flutuante na expressão `a + (b-a) * random()`.

`random.`**`triangular`**`(`*low*, *high*, *mode*`)`

Retorna um número de ponto flutuante aleatório *N* de forma que `low <= N <= high` e com o *mode* especificado entre esses limites. Os limites *low* e *high* são padronizados como zero e um. O argumento *mode* assume como padrão o ponto médio entre os limites, fornecendo uma distribuição simétrica.

`random.`**`betavariate`**`(`*alpha*, *beta*`)`

Distribuição beta. As condições nos parâmetros são `alpha > 0` e `beta > 0`. Os valores retornados variam entre 0 e 1.

`random.`**`expovariate`**`(`*lambd=1.0*`)`

Distribuição exponencial. *lambd* é 1.0 dividido pela média desejada. Deve ser diferente de zero. (O parâmetro seria chamado "lambda", mas é uma palavra reservada em Python.) Os valores retornados variam de 0 a infinito positivo se *lambd* for positivo e de infinito negativo a 0 se *lambd* for negativo.

Alterado na versão 3.12: Adicionado o valor padrão para `lambd`.

`random.`**`gammavariate`**`(`*alpha*, *beta*`)`

Distribuição gama. (*Não* a função gama!) Os parâmetros de forma e escala, *alfa* e *beta*, devem ter valores positivos. (As convenções de chamada variam e algumas fontes definem 'beta' como o inverso da escala).

A função de distribuição de probabilidade é:

```
            x ** (alpha - 1) * math.exp(-x / beta)
pdf(x) =  --------------------------------------
            math.gamma(alpha) * beta ** alpha
```

`random.`**`gauss`**`(`*mu=0.0*, *sigma=1.0*`)`

Distribuição normal, também chamada de distribuição gaussiana. *mu* é a média e *sigma* é o desvio padrão. Isto é um pouco mais rápido que a função *`normalvariate()`* definida abaixo.

Nota sobre multithreading: quando duas threads chamam esta função simultaneamente, é possível que recebam o mesmo valor de retorno. Isso pode ser evitado de três maneiras. 1) Fazer com que cada thread use uma instância diferente do gerador de números aleatórios. 2) Colocar travas em todas as chamadas. 3) Usar a função mais lenta, mas segura para thread *`normalvariate()`*.

Alterado na versão 3.11: *mu* e *sigma* agora têm argumentos padrão.

random.**lognormvariate**(*mu*, *sigma*)

Distribuição log normal. Se você usar o logaritmo natural dessa distribuição, obterá uma distribuição normal com média *mu* e desvio padrão *sigma*. *mu* pode ter qualquer valor e *sigma* deve ser maior que zero.

random.**normalvariate**(*mu=0.0*, *sigma=1.0*)

Distribuição normal. *mu* é a média e *sigma* é o desvio padrão.

Alterado na versão 3.11: *mu* e *sigma* agora têm argumentos padrão.

random.**vonmisesvariate**(*mu*, *kappa*)

mu é o ângulo médio, expresso em radianos entre 0 e 2**pi*, e *kappa* é o parâmetro de concentração, que deve ser maior ou igual a zero. Se *kappa* for igual a zero, essa distribuição será reduzida para um ângulo aleatório uniforme no intervalo de 0 a 2**pi*.

random.**paretovariate**(*alpha*)

Distribuição de Pareto. *alpha* é o parâmetro de forma.

random.**weibullvariate**(*alpha*, *beta*)

Distribuição Weibull. *alpha* é o parâmetro de escala e *beta* é o parâmetro de forma.

9.6.7 Gerador alternativo

class random.**Random**([*seed*])

Classe que implementa o gerador de números pseudoaleatórios padrão usado pelo módulo *random*.

Alterado na versão 3.11: Anteriormente, o *seed* poderia ser qualquer objeto hasheável. Agora está limitado a: None, *int*, *float*, *str*, *bytes* ou *bytearray*.

Subclasses de Random devem substituir os seguintes métodos se desejarem fazer uso de um gerador básico diferente:

seed(*a=None*, *version=2*)

Substitua este método nas subclasses para personalizar o comportamento de *seed()* das instâncias de Random.

getstate()

Substitua este método nas subclasses para personalizar o comportamento de *getstate()* das instâncias de Random.

setstate(*state*)

Substitua este método nas subclasses para personalizar o comportamento de *setstate()* das instâncias de Random.

random()

Substitua este método nas subclasses para personalizar o comportamento de *random()* das instâncias de Random.

Opcionalmente, uma subclasse geradora personalizada também pode fornecer o seguinte método:

getrandbits(*k*)

Substitua este método nas subclasses para personalizar o comportamento de *getrandbits()* das instâncias de Random.

randbytes(*n*)

Substitua este método nas subclasses para personalizar o comportamento de *randbytes()* das instâncias de Random.

class random.**SystemRandom**([*seed*])

Classe que usa a função *os.urandom()* para gerar números aleatórios a partir de fontes fornecidas pelo sistema operacional. Não disponível em todos os sistemas. Não depende do estado do software e as sequências não são reproduzíveis. Assim, o método *seed()* não tem efeito e é ignorado. Os métodos *getstate()* e *setstate()* levantam *NotImplementedError* se chamados.

9.6.8 Notas sobre reprodutibilidade

Às vezes, é útil poder reproduzir as sequências fornecidas por um gerador de números pseudoaleatórios. Reutilizando um valor inicial, a mesma sequência deve ser reproduzível de execução para execução, desde que mais de uma thread não estejam em execução.

A maioria dos algoritmos e funções de propagação do módulo aleatório está sujeita a alterações nas versões de Python, mas dois aspectos são garantidos para não serem alterados:

- Se um novo método de semeadura for adicionado, será oferecida um semeador compatível com versões anteriores.
- O método *random()* do gerador continuará produzindo a mesma sequência quando o semeador compatível receber a mesma semente.

9.6.9 Exemplos

Exemplos básicos:

```
>>> random()                            # Ponto flutuante aleatório:  0.0 <= x < 1.0
0.37444887175646646

>>> uniform(2.5, 10.0)                  # Ponto flutuante aleatório:  2.5 <= x <= 10.
↪0
3.1800146073117523

>>> expovariate(1 / 5)                  # Intervalo entre chegadas com médias de 5␣
↪segundos
5.148957571865031

>>> randrange(10)                       # Inteiro de 0 até 9 inclusivo
7

>>> randrange(0, 101, 2)                # Inteiro par entre 0 até 100 inclusivo
26

>>> choice(['win', 'lose', 'draw'])     # Um único elemento aleatório de uma␣
↪sequência
'draw'

>>> deck = 'ace two three four'.split()
>>> shuffle(deck)                       # Embaralha uma lista
>>> deck
['four', 'two', 'ace', 'three']

>>> sample([10, 20, 30, 40, 50], k=4) # Quatro amostras sem substitutos
[40, 10, 50, 30]
```

Simulações:

```
>>> # Seis giros da roleta (amostragem ponderada com substituição)
>>> choices(['red', 'black', 'green'], [18, 18, 2], k=6)
['red', 'green', 'black', 'black', 'red', 'black']

>>> # Distribui 20 cartas sem reposição de um baralho de
>>> # 52 cartas e determine a proporção de cartas com
>>> # valor dez: dez, valete, rainha ou rei.
>>> deal = sample(['tens', 'low cards'], counts=[16, 36], k=20)
>>> deal.count('tens') / 20
0.15
```

(continua na próxima página)

(continuação da página anterior)

```
>>> # Estima a probabilidade de obter 5 ou mais caras em 7 giros
>>> # de uma moeda viciada que cai em cara 60% das vezes.
>>> sum(binomialvariate(n=7, p=0.6) >= 5 for i in range(10_000)) / 10_000
0.4169

>>> # Probabilidade da mediana de 5 amostras estar nos dois quartis do meio
>>> def trial():
...     return 2_500 <= sorted(choices(range(10_000), k=5))[2] < 7_500
...
>>> sum(trial() for i in range(10_000)) / 10_000
0.7958
```

Exemplo de bootstrapping estatístico usando reamostragem com substituição para estimar um intervalo de confiança para a média de uma amostra:

```
# https://www.thoughtco.com/example-of-bootstrapping-3126155
from statistics import fmean as mean
from random import choices

data = [41, 50, 29, 37, 81, 30, 73, 63, 20, 35, 68, 22, 60, 31, 95]
means = sorted(mean(choices(data, k=len(data))) for i in range(100))
print(f'The sample mean of {mean(data):.1f} has a 90% confidence '
      f'interval from {means[5]:.1f} to {means[94]:.1f}')
```

Exemplo de um teste de permutação de reamostragem para determinar a significância estatística ou valor-p de uma diferença observada entre os efeitos de uma droga em comparação com um placebo:

```
# Exemplo de "Statistics is Easy" de Dennis Shasha e Manda Wilson
from statistics import fmean as mean
from random import shuffle

drug = [54, 73, 53, 70, 73, 68, 52, 65, 65]
placebo = [54, 51, 58, 44, 55, 52, 42, 47, 58, 46]
observed_diff = mean(drug) - mean(placebo)

n = 10_000
count = 0
combined = drug + placebo
for i in range(n):
    shuffle(combined)
    new_diff = mean(combined[:len(drug)]) - mean(combined[len(drug):])
    count += (new_diff >= observed_diff)

print(f'{n} label reshufflings produced only {count} instances with a difference')
print(f'at least as extreme as the observed difference of {observed_diff:.1f}.')
print(f'The one-sided p-value of {count / n:.4f} leads us to reject the null')
print(f'hypothesis that there is no difference between the drug and the placebo.')
```

Simulação de tempos de chegada e entregas de serviços para uma fila multisservidor:

```
from heapq import heapify, heapreplace
from random import expovariate, gauss
from statistics import mean, quantiles

average_arrival_interval = 5.6
average_service_time = 15.0
```

(continua na próxima página)

(continuação da página anterior)

```
stdev_service_time = 3.5
num_servers = 3

waits = []
arrival_time = 0.0
servers = [0.0] * num_servers  # hora em que cada servidor fica disponível
heapify(servers)
for i in range(1_000_000):
    arrival_time += expovariate(1.0 / average_arrival_interval)
    next_server_available = servers[0]
    wait = max(0.0, next_server_available - arrival_time)
    waits.append(wait)
    service_duration = max(0.0, gauss(average_service_time, stdev_service_time))
    service_completed = arrival_time + wait + service_duration
    heapreplace(servers, service_completed)

print(f'Mean wait: {mean(waits):.1f}  Max wait: {max(waits):.1f}')
print('Quartiles:', [round(q, 1) for q in quantiles(waits)])
```

Ver também

Statistics for Hackers um tutorial em vídeo por Jake Vanderplas sobre análise estatística usando apenas alguns conceitos fundamentais, incluindo simulação, amostragem, embaralhamento e validação cruzada.

Economics Simulation uma simulação de um mercado por Peter Norvig que mostra o uso eficaz de muitas das ferramentas e distribuições fornecidas por este módulo (gauss, uniform, sample, betavariate, choice, triangular e randrange).

A Concrete Introduction to Probability (using Python) um tutorial de Peter Norvig cobrindo os fundamentos da teoria das probabilidades, como escrever simulações e como realizar análises de dados usando Python.

9.6.10 Receitas

Estas receitas mostram como fazer seleções aleatórias de forma eficiente a partir dos iteradores combinatórios no módulo *itertools*:

```
def random_product(*args, repeat=1):
    "Random selection from itertools.product(*args, **kwds)"
    pools = [tuple(pool) for pool in args] * repeat
    return tuple(map(random.choice, pools))

def random_permutation(iterable, r=None):
    "Random selection from itertools.permutations(iterable, r)"
    pool = tuple(iterable)
    r = len(pool) if r is None else r
    return tuple(random.sample(pool, r))

def random_combination(iterable, r):
    "Random selection from itertools.combinations(iterable, r)"
    pool = tuple(iterable)
    n = len(pool)
    indices = sorted(random.sample(range(n), r))
    return tuple(pool[i] for i in indices)

def random_combination_with_replacement(iterable, r):
    "Choose r elements with replacement.  Order the result to match the iterable."
```

(continua na próxima página)

(continuação da página anterior)

```
    # Result will be in set(itertools.combinations_with_replacement(iterable, r)).
    pool = tuple(iterable)
    n = len(pool)
    indices = sorted(random.choices(range(n), k=r))
    return tuple(pool[i] for i in indices)
```

O padrão de `random()` é retornar múltiplos de 2^{-53} no intervalo $0.0 \le x < 1.0$. Todos esses números são espaçados uniformemente e são representáveis exatamente como pontos flutuantes Python. No entanto, muitos outros pontos flutuantes representados nesse intervalo não são seleções possíveis. Por exemplo, `0.05954861408025609` não é um múltiplo inteiro de 2^{-53}.

A receita a seguir tem uma abordagem diferente. Todos os pontos flutuantes no intervalo são seleções possíveis. A mantissa vem de uma distribuição uniforme de inteiros no intervalo $2^{52} \le mantissa < 2^{53}$. O expoente vem de uma distribuição geométrica onde expoentes menores que *-53* ocorrem com metade da frequência do próximo expoente maior.

```
from random import Random
from math import ldexp

class FullRandom(Random):

    def random(self):
        mantissa = 0x10_0000_0000_0000 | self.getrandbits(52)
        exponent = -53
        x = 0
        while not x:
            x = self.getrandbits(32)
            exponent += x.bit_length() - 32
        return ldexp(mantissa, exponent)
```

Todas as *distribuições reais valoradas* na classe usarão o novo método:

```
>>> fr = FullRandom()
>>> fr.random()
0.05954861408025609
>>> fr.expovariate(0.25)
8.87925541791544
```

A receita é conceitualmente equivalente a um algoritmo que escolhe entre todos os múltiplos de 2^{-1074} no intervalo $0.0 \le x < 1.0$. Todos esses números são espaçados uniformemente, mas a maioria deve ser arredondada para o ponto flutuante Python representável mais próximo. (O valor 2^{-1074} é o menor ponto flutuante positivo não normalizado e é igual a `math.ulp(0.0)`.)

Ver também

Generating Pseudo-random Floating-Point Values um artigo de Allen B. Downey descrevendo formas de gerar pontos flutuantes mais refinados do que normalmente gerados por `random()`.

9.6.11 Uso na linha de comando

Adicionado na versão 3.13.

O módulo `random` pode ser invocado como um script na linha de comando:

```
python -m random [-h] [-c ESCOLHA [ESCOLHA ...] | -i N | -f N] [entrada ...]
```

As seguintes opções são aceitas:

-h, --help
Mostra a mensagem de ajuda e sai.

-c CHOICE [CHOICE ...]
--choice CHOICE [CHOICE ...]
Imprime uma escolha aleatória, usando *choice()*.

-i <N>
--integer <N>
Imprime um inteiro aleatório entre 1 e N inclusive, usando *randint()*.

-f <N>
--float <N>
Imprime um número de ponto flutuante aleatório entre 0 e N inclusive, usando *uniform()*.

Se nenhuma opção for fornecida, a saída dependerá da entrada:

- String ou vários: mesmo que *--choice*.
- Inteiro mesmo que *--integer*.
- Ponto flutuante: mesmo que *--float*.

9.6.12 Exemplos na linha de comando

Aqui estão alguns exemplos da interface de linha de comando `random`:

```
$ # Escolhe um aleatório
$ python -m random egg bacon sausage spam "Lobster Thermidor aux crevettes with a␣
↪Mornay sauce"
Lobster Thermidor aux crevettes with a Mornay sauce

$ # Inteiro aleatório
$ python -m random 6
6

$ # Número de ponto flutuante aleatório
$ python -m random 1.8
1.7080016272295635

$ # Com argumentos explícitos
$ python  -m random --choice egg bacon sausage spam "Lobster Thermidor aux␣
↪crevettes with a Mornay sauce"
egg

$ python -m random --integer 6
3

$ python -m random --float 1.8
1.5666339105010318

$ python -m random --integer 6
5

$ python -m random --float 6
3.1942323316565915
```

9.7 `statistics` — Funções estatísticas

Adicionado na versão 3.4.

Código-fonte: Lib/statistics.py

Esse módulo fornece funções para o cálculo de estatísticas matemáticas de dados numéricos (para valores do tipo *Real*).

O módulo não tem a intenção de competir com bibliotecas de terceiros como NumPy, SciPy, ou pacotes proprietários de estatísticas com todos os recursos destinados a estatísticos profissionais como Minitab, SAS e Matlab. Ela destina-se ao nível de calculadoras gráficas e científicas.

A menos que seja explicitamente indicado, essas funções suportam *int*, *float*, *Decimal* e *Fraction*. O uso com outros tipos (sejam numéricos ou não) não é atualmente suportado. Coleções com uma mistura de tipos também são indefinidas e dependentes da implementação. Se os seus dados de entrada consistem de tipos misturados, você pode usar *map()* para garantir um resultado consistente, por exemplo `map(float, dado_entrada)`.

Alguns conjuntos de dados usam valores `NaN` (não um número) para representar dados ausentes. Como os NaNs possuem semântica de comparação incomum, eles causam comportamentos surpreendentes ou indefinidos nas funções estatísticas que classificam dados ou contam ocorrências. As funções afetadas são `median()`, `median_low()`, `median_high()`, `median_grouped()`, `mode()`, `multimode()` e `quantiles()`. Os valores `NaN` devem ser removidos antes de chamar estas funções:

```
>>> from statistics import median
>>> from math import isnan
>>> from itertools import filterfalse

>>> data = [20.7, float('NaN'),19.2, 18.3, float('NaN'), 14.4]
>>> sorted(data)  # Isso tem um comportamento surpreendente
[20.7, nan, 14.4, 18.3, 19.2, nan]
>>> median(data)  # Este resultado é inesperado
16.35

>>> sum(map(isnan, data))     # Número de valores em falta
2
>>> clean = list(filterfalse(isnan, data))  # Remove valores NaN
>>> clean
[20.7, 19.2, 18.3, 14.4]
>>> sorted(clean)  # A classificação agora funciona como esperado
[14.4, 18.3, 19.2, 20.7]
>>> median(clean)       # Este resultado é agora bem definido
18.75
```

9.7.1 Médias e medidas de valor central

Essas funções calculam a média ou o valor típico de uma população ou amostra.

mean()	Média aritmética dos dados.
fmean()	Média aritmética rápida de ponto flutuante, com ponderação opcional.
geometric_mean()	Média geométrica dos dados.
harmonic_mean()	Média harmônica dos dados.
kde()	Estima a distribuição de densidade de probabilidade dos dados.
kde_random()	Amostragem aleatória do PDF gerado por kde().
median()	Mediana (valor do meio) dos dados.
median_low()	Mediana inferior dos dados.
median_high()	Mediana superior dos dados.
median_grouped()	Mediana (50º percentil) dos dados agrupados.
mode()	Moda (valor mais comum) de dados discretos ou nominais.
multimode()	Lista de modas (valores mais comuns) de dados discretos ou nominais.
quantiles()	Divide os dados em intervalos com probabilidade igual.

9.7.2 Medidas de espalhamento

Essas funções calculam o quanto a população ou amostra tendem a desviar dos valores típicos ou médios.

pstdev()	Desvio padrão populacional dos dados.
pvariance()	Variância populacional dos dados.
stdev()	Desvio padrão amostral dos dados.
variance()	Variância amostral dos dados.

9.7.3 Estatísticas para relações entre duas entradas

Essas funções calculam estatísticas sobre relações entre duas entradas.

covariance()	Covariância amostral para duas variáveis.
correlation()	Coeficientes de correlação de Pearson e Spearman.
linear_regression()	Inclinação e intersecção para regressão linear simples.

9.7.4 Detalhes das funções

Nota: as funções não exigem que os dados estejam ordenados. No entanto, para conveniência do leitor, a maioria dos exemplos mostrará sequências ordenadas.

`statistics.`**`mean`**`(`*data*`)`

Retorna a média aritmética amostral de *data* que pode ser uma sequência ou iterável.

A média aritmética é a soma dos dados dividida pela quantidade de dados. É comumente chamada apenas de "média", apesar de ser uma das diversas médias matemáticas. Ela representa uma medida da localização central dos dados.

Se *data* for vazio, uma exceção do tipo *StatisticsError* será levantada.

Alguns exemplos de uso:

```
>>> mean([1, 2, 3, 4, 4])
2.8
>>> mean([-1.0, 2.5, 3.25, 5.75])
2.625

>>> from fractions import Fraction as F
>>> mean([F(3, 7), F(1, 21), F(5, 3), F(1, 3)])
Fraction(13, 21)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> from decimal import Decimal as D
>>> mean([D("0.5"), D("0.75"), D("0.625"), D("0.375")])
Decimal('0.5625')
```

Nota

A média é fortemente afetada por outliers (valor atípico) e não é necessariamente um exemplo típico dos pontos de dados. Para uma medida mais robusta, embora menos eficiente, de tendência central, veja *median()*.

A média amostral fornece uma estimativa não enviesada da média populacional verdadeira, ou seja, quando a média é obtida para todas as possíveis amostras, `mean(sample)` converge para a média verdadeira de toda população. Se *data* representa toda população ao invés de uma amostra, então `mean(data)` é equivalente a calcular a verdadeira média populacional μ.

`statistics.`**`fmean`**(*data*, *weights=None*)

Converte valores em *data* para ponto flutuante e calcula a média aritmética.

Essa função executa mais rapidamente do que a função *mean()* e sempre retorna um *float*. *data* pode ser uma sequência ou iterável. Se o conjunto de dados de entrada estiver vazio, levanta uma exceção *StatisticsError*.

```
>>> fmean([3.5, 4.0, 5.25])
4.25
```

A ponderação opcional é suportada. Por exemplo, um professor atribui uma nota para um curso ponderando questionários em 20%, trabalhos de casa em 20%, um exame de meio de período em 30% e um exame final em 30%:

```
>>> grades = [85, 92, 83, 91]
>>> weights = [0.20, 0.20, 0.30, 0.30]
>>> fmean(grades, weights)
87.6
```

Se *weights* for fornecido, deve ter o mesmo comprimento que *data* ou uma *ValueError* será levantada.

Adicionado na versão 3.8.

Alterado na versão 3.11: Adicionado suporte a *weights*.

`statistics.`**`geometric_mean`**(*data*)

Converte valores em *data* para ponto flutuante e calcula a média geométrica.

A média geométrica indica a tendência central ou valor típico de *data* usando o produto dos valores (em oposição à média aritmética que usa a soma deles).

Levanta uma exceção *StatisticsError* se a entrada do conjunto de dados for vazia, contiver um zero ou um valor negativo. *data* pode ser uma sequência ou iterável.

Nenhum esforço especial é feito para alcançar resultados exatos. (Mas, isso pode mudar no futuro).

```
>>> round(geometric_mean([54, 24, 36]), 1)
36.0
```

Adicionado na versão 3.8.

`statistics.`**`harmonic_mean`**(*data*, *weights=None*)

Retorna a média harmônica de *data*, uma sequência ou iterável de números de valor real. Se *weights* for omitido ou `None`, então a ponderação igual é presumida.

A média harmônica é a recíproca da média arimética calculada pela função *mean()* dos recíprocos dos dados. Por exemplo, a média harmônica de três valores *a*, *b* e *c* será equivalente a `3/(1/a + 1/b + 1/c)`. Se um dos valores for zero, o resultado também será zero.

A média harmônica é um tipo de média, uma medida de localização central dos dados. Ela é geralmente apropriada quando se está calculando a média de razões e taxas; por exemplo, velocidades.

Suponha que um carro viaje 10 km a 40 km/h, e em seguida viaje mais 10 km a 60 km/h. Qual é a velocidade média?

```
>>> harmonic_mean([40, 60])
48.0
```

Suponha que um carro viaja a 40 km/h por 5 km e, quando o trânsito melhora, acelera para 60 km/h pelos 30 km restantes da viagem. Qual é a velocidade média?

```
>>> harmonic_mean([40, 60], weights=[5, 30])
56.0
```

StatisticsError é levantada se *data* for vazio, qualquer elemento for menor que zero, ou se a soma ponderada não for positiva.

O algoritmo atual tem uma saída antecipada quando encontra um zero na entrada. Isso significa que as entradas subsequentes não tem a validade testada. (Esse comportamento pode mudar no futuro.)

Adicionado na versão 3.6.

Alterado na versão 3.10: Adicionado suporte a *weights*.

`statistics.`**`kde`**(*data*, *h*, *kernel='normal'*, ***, *cumulative=False*)

Estimativa de densidade por Kernel (KDE): Cria uma função de densidade de probabilidade contínua ou uma função de distribuição cumulativa a partir de amostras discretas.

A ideia básica é suavizar os dados usando uma função kernel. para ajudar a tirar inferências sobre uma população a partir de uma amostra.

O grau de suavização é controlado pelo parâmetro de escala *h*, que é chamado de largura de banda. Valores menores enfatizam características locais, enquanto valores maiores dão resultados mais suaves.

O *kernel* determina os pesos relativos dos pontos de dados da amostra. Geralmente, a escolha do formato de kernel não importa tanto quanto o parâmetro de suavização de largura de banda mais influente.

Os kernels que dão algum peso a cada ponto de amostra incluem *normal* (*gauss*), *logistic* e *sigmoid*.

Os kernels que apenas dão peso aos pontos de amostra dentro da largura de banda incluem *rectangular* (*uniform*), *triangular*, *parabolic* (*epanechnikov*), *quartic* (*biweight*), *triweight* e *cosine*.

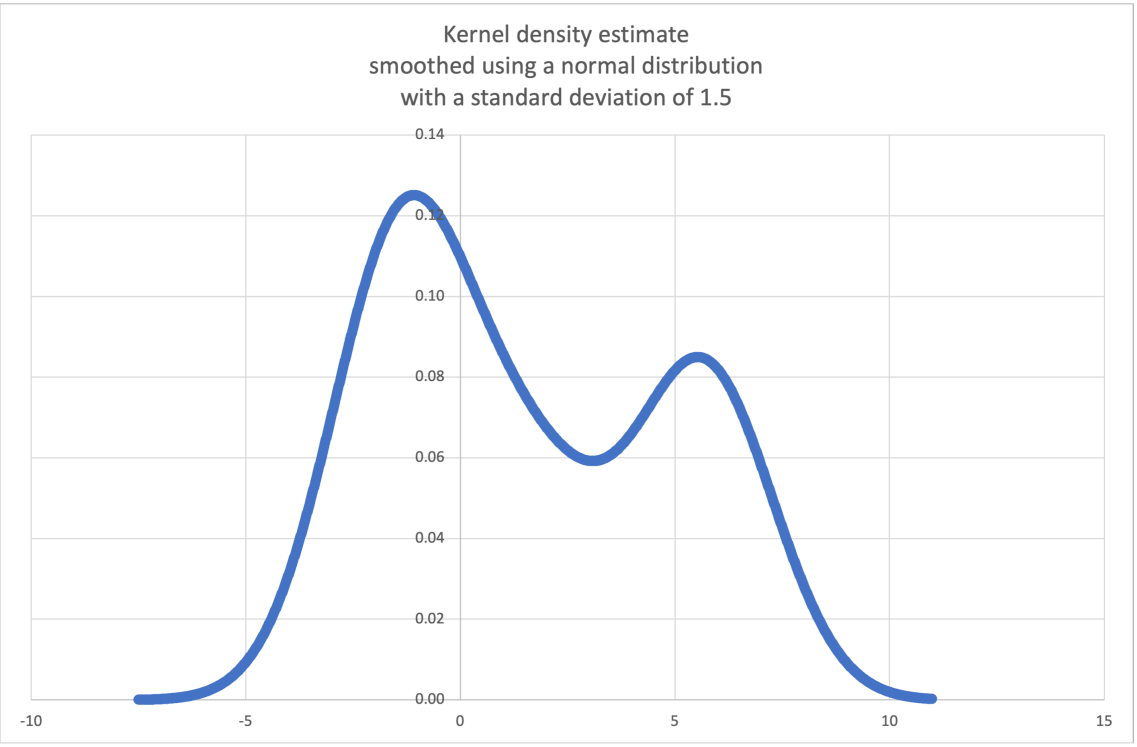
Se *cumulative* for verdadeiro, retornará uma função de distribuição cumulativa.

Uma *StatisticsError* será levantada se a sequência de dados em *data* estiver vazia.

A Wikipédia tem um exemplo onde podemos usar *kde()* para gerar e plotar uma função de densidade de probabilidade estimada a partir de uma pequena amostra:

```
>>> sample = [-2.1, -1.3, -0.4, 1.9, 5.1, 6.2]
>>> f_hat = kde(sample, h=1.5)
>>> xarr = [i/100 for i in range(-750, 1100)]
>>> yarr = [f_hat(x) for x in xarr]
```

Os pontos em `xarr` e `yarr` podem ser usados para fazer um gráfico PDF:



Adicionado na versão 3.13.

`statistics.`**`kde_random`**(*data*, *h*, *kernel='normal'*, *, *seed=None*)

Retorna uma função que faz uma seleção aleatória da função de densidade de probabilidade estimada produzida por `kde(data, h, kernel)`.

Fornecer uma *seed* permite seleções reproduzíveis. No futuro, os valores podem mudar um pouco conforme estimativas de funções de distribuição acumulada (CDF) inversas de kernel mais precisas forem implementadas. *seed* pode ser um inteiro, float, str ou bytes.

Uma *`StatisticsError`* será levantada se a sequência de dados em *data* estiver vazia.

Continuando o exemplo de *`kde()`*, podemos usar *`kde_random()`* para gerar novas seleções aleatórias a partir de uma função de densidade de probabilidade estimada:

```
>>> data = [-2.1, -1.3, -0.4, 1.9, 5.1, 6.2]
>>> rand = kde_random(data, h=1.5, seed=8675309)
>>> new_selections = [rand() for i in range(10)]
>>> [round(x, 1) for x in new_selections]
[0.7, 6.2, 1.2, 6.9, 7.0, 1.8, 2.5, -0.5, -1.8, 5.6]
```

Adicionado na versão 3.13.

`statistics.`**`median`**(*data*)

Retorna a mediana (o valor do meio) de dados numéricos, usando o método comum de "média entre os dois do meio". Se *data* for vazio, é levantada uma exceção *`StatisticsError`*. *data* pode ser uma sequência ou um iterável.

A mediana é uma medida robusta de localização central e é menos afetada por valores discrepantes. Quando a quantidade de pontos de dados for ímpar, o valor de meio é retornado:

```
>>> median([1, 3, 5])
3
```

Quando o número de elementos for par, a mediana é calculada tomando-se a média entre os dois valores no meio:

```
>>> median([1, 3, 5, 7])
4.0
```

Isso serve quando seus dados forem discretos e você não se importa que a média possa não ser um valor que de fato ocorre nos seus dados.

Caso os dados sejam ordinais (oferecem suporte para operações de ordenação) mas não são numéricos (não oferecem suporte para adição), considere usar a função *median_low()* ou *median_high()* no lugar.

statistics.**median_low**(*data*)

Retorna a mediana inferior de dados numéricos. Se *data* for vazio, a exceção *StatisticsError* é levantada. *data* pode ser uma sequência ou um iterável.

A mediana inferior sempre é um membro do conjunto de dados. Quando o número de elementos for ímpar, o valor intermediário é retornado. Se houver um número par de elementos, o menor entre os dois valores centrais é retornado.

```
>>> median_low([1, 3, 5])
3
>>> median_low([1, 3, 5, 7])
3
```

Use a mediana inferior caso seus dados forem discretos e você prefira que a mediana seja um valor que de fato existe nos seus dados ao invés de um valor interpolado.

statistics.**median_high**(*data*)

Retorna a mediana superior de dados numéricos. Se *data* for vazio, a exceção *StatisticsError* é levantada. *data* pode ser uma sequência ou um iterável.

A mediana superior sempre é um membro do conjunto de dados. Quando o número de elementos for ímpar, o valor intermediário é retornado. Se houver um número par de elementos, o maior entre os dois valores centrais é retornado.

```
>>> median_high([1, 3, 5])
3
>>> median_high([1, 3, 5, 7])
5
```

Use a mediana superior caso seus dados forem discretos e você prefira que a mediana seja um valor que de fato existe nos seus dados ao invés de um valor interpolado.

statistics.**median_grouped**(*data*, *interval=1.0*)

Estima a mediana para dados numéricos que foram agrupados ou "binned" em torno dos pontos médios de intervalos consecutivos de largura fixa.

data pode ser qualquer iterável de dados numéricos com cada valor sendo exatamente o ponto médio de um bin. Pelo menos um valor deve estar presente.

interval é a largura de cada compartimento.

Por exemplo, as informações demográficas podem ter sido resumidas em grupos etários consecutivos de dez anos, com cada grupo sendo representado pelos pontos médios de cinco anos dos intervalos:

```
>>> from collections import Counter
>>> demographics = Counter({
...    25: 172,   # 20 to 30 years old
...    35: 484,   # 30 to 40 years old
...    45: 387,   # 40 to 50 years old
...    55:  22,   # 50 to 60 years old
...    65:   6,   # 60 to 70 years old
... })
...
```

O 50º percentil (mediana) é a 536ª pessoa de 1071 membros. Essa pessoa está na faixa etária de 30 a 40 anos.

A função regular *median()* iria presumir que todos na faixa etária tricenariana tinham exatamente 35 anos. Uma suposição mais sustentável é que os 484 membros dessa faixa etária estão distribuídos uniformemente entre 30 e 40. Para isso, usamos *median_grouped()*:

```
>>> data = list(demographics.elements())
>>> median(data)
35
>>> round(median_grouped(data, interval=10), 1)
37.5
```

O chamador é responsável por garantir que os pontos de dados sejam separados por múltiplos exatos de *intervalo*. Isso é essencial para obter um resultado correto. A função não verifica essa pré-condição.

As entradas podem ser qualquer tipo numérico que possa ser convertido para um ponto flutuante durante a etapa de interpolação.

statistics.**mode**(*data*)

Retorna o valor mais comum dos dados discretos ou nominais em *data*. A moda (quando existe) é o valor mais típico e serve como uma medida de localização central.

Se existirem múltiplas modas com a mesma frequência, retorna a primeira encontrada em *data*. Se ao invés disso se desejar a menor ou a maior dentre elas, use `min(multimode(data))` ou `max(multimode(data))`. Se a entrada *data* é vazia, a exceção *StatisticsError* é levantada.

`mode` presume que os dados são discretos e retorna um único valor. Esse é o tratamento padrão do conceito de moda normalmente ensinado nas escolas:

```
>>> mode([1, 1, 2, 3, 3, 3, 3, 4])
3
```

A moda é única no sentido que é a única medida estatística nesse módulo que também se aplica a dados nominais (não-numéricos):

```
>>> mode(["red", "blue", "blue", "red", "green", "red", "red"])
'red'
```

Somente entradas hasheáveis são suportadas. Para manipular o tipo *set*, considere fazer a conversão para *frozenset*. Para manipular o tipo *list*, considere fazer a conversão para *tuple*. Para entradas mistas ou aninhadas, considere usar este algoritmo quadrático mais lento que depende apenas de testes de igualdade: `max(data, key=data.count)`.

Alterado na versão 3.8: Agora lida com conjunto de dados multimodais retornando a primeira moda encontrada. Anteriormente, ela levantava a exceção *StatisticsError* quando mais do que uma moda era encontrada.

statistics.**multimode**(*data*)

Retorna uma lista dos valores mais frequentes na ordem em que eles foram encontrados em *data*. Irá retornar mais do que um resultado se houver múltiplas modas ou uma lista vazia se *data* for vazio.

```
>>> multimode('aabbbbccddddeeffffgg')
['b', 'd', 'f']
>>> multimode('')
[]
```

Adicionado na versão 3.8.

statistics.**pstdev**(*data*, *mu=None*)

Retorna o desvio padrão populacional (a raiz quadrada da variância populacional). Veja os argumentos e outros detalhes em *pvariance()*.

```
>>> pstdev([1.5, 2.5, 2.5, 2.75, 3.25, 4.75])
0.986893273527251
```

statistics.**pvariance**(*data*, *mu=None*)

Retorna a variância populacional de *data*, que deve ser uma sequência ou iterável não-vazio de números reais. A variância, o segundo momento estatístico a partir da média, é uma medida da variabilidade (espalhamento ou dispersão) dos dados. Uma variância grande indica que os dados são espalhados; uma variância menor indica que os dados estão agrupado em volta da média.

Se o segundo argumento opcional *mu* for fornecido, ele deve ser a média da *população* dos dados em *data*. Ele também pode ser usado para calcular o segundo momento em torno de um ponto que não é a média. Se estiver faltando ou for `None` (o padrão), a média aritmética é calculada automaticamente.

Use essa função para calcular a variância de toda a população. Para estimar a variância de uma amostra, a função *variance()* costuma ser uma escolha melhor.

Levanta *StatisticsError* se *data* for vazio.

Exemplos:

```
>>> data = [0.0, 0.25, 0.25, 1.25, 1.5, 1.75, 2.75, 3.25]
>>> pvariance(data)
1.25
```

Se você já calculou a média dos seus dados, você pode passar o valor no segundo argumento opcional *mu* para evitar recálculos:

```
>>> mu = mean(data)
>>> pvariance(data, mu)
1.25
```

Decimais e frações são suportadas:

```
>>> from decimal import Decimal as D
>>> pvariance([D("27.5"), D("30.25"), D("30.25"), D("34.5"), D("41.75")])
Decimal('24.815')

>>> from fractions import Fraction as F
>>> pvariance([F(1, 4), F(5, 4), F(1, 2)])
Fraction(13, 72)
```

Nota

Quando os dados de entrada representarem toda a população, ele retorna a variância populacional σ^2. Se em vez disso, amostras forem usadas, então a variância amostral enviesada s^2, também conhecida como variância com N graus de liberdade é retornada.

Se de alguma forma você souber a verdadeira média populacional μ, você pode usar essa função para calcular a variância de uma amostra, fornecendo a média populacional conhecida como segundo argumento. Caso seja fornecido um conjunto de amostras aleatórias da população, o resultado será um estimador não enviesado da variância populacional.

statistics.**stdev**(*data*, *xbar=None*)

Retorna o desvio padrão amostral (a raiz quadrada da variância amostral). Veja *variance()* para argumentos e outros detalhes.

```
>>> stdev([1.5, 2.5, 2.5, 2.75, 3.25, 4.75])
1.0810874155219827
```

statistics.**variance**(*data*, *xbar=None*)

Retorna a variância amostral de *data*, que deve ser um iterável com pelo menos dois números reais. Variância, ou o segundo momento estatístico a partir da média, é uma medida de variabilidade (espalhamento ou dispersão) dos dados. Uma variância grande indica que os dados são espalhados, uma variância menor indica que os dados estão agrupados em volta da média.

Se o segundo argumento opcional *xbar* for dado, ele deve a média *sample* de *data*. Se ele não estiver presente ou for `None` (valor padrão), a média é automaticamente calculada.

Use essa função quando seus dados representarem uma amostra da população. Para calcular a variância de toda população veja *pvariance()*.

Levanta a exceção *StatisticsError* se *data* tiver menos do que dois valores.

Exemplos:

```
>>> data = [2.75, 1.75, 1.25, 0.25, 0.5, 1.25, 3.5]
>>> variance(data)
1.3720238095238095
```

Se você já calculou a média amostral dos seus dados, você pode passar o valor no segundo argumento opcional *xbar* para evitar recálculos:

```
>>> m = mean(data)
>>> variance(data, m)
1.3720238095238095
```

Essa função não verifica se você passou a média verdadeira como *xbar*. Usar valores arbitrários para *xbar* pode levar a resultados inválidos ou impossíveis.

Decimais e frações são suportados.

```
>>> from decimal import Decimal as D
>>> variance([D("27.5"), D("30.25"), D("30.25"), D("34.5"), D("41.75")])
Decimal('31.01875')

>>> from fractions import Fraction as F
>>> variance([F(1, 6), F(1, 2), F(5, 3)])
Fraction(67, 108)
```

Nota

Essa é a variância amostral s^2 com a correção de Bessel, também conhecida como variância com N-1 graus de liberdade. Desde que os pontos de dados sejam representativos (por exemplo, independentes e distribuídos de forma idêntica), o resultado deve ser uma estimativa não enviesada da verdadeira variação populacional.

Caso você de alguma forma saiba a verdadeira média populacional μ você deveria passar para a função *pvariance()* como o parâmetro *mu* para obter a variância da amostra.

statistics.**quantiles**(*data*, ***, *n=4*, *method='exclusive'*)

Divide *data* em *n* intervalos contínuos com igual probabilidade. Retorna uma lista de `n - 1` pontos de corte separando os intervalos.

Defina *n* como 4 para quartis (o padrão). Defina *n* como 10 para decis. Defina *n* como 100 para percentis, o que fornece os 99 pontos de corte que separam *data* em 100 grupos de tamanhos iguais. Levanta a exceção *StatisticsError* se *n* não for pelo menos 1.

data pode ser qualquer iterável contendo dados amostrais. Para resultados significativos, a quantidade de dados em *data* deve ser maior do que *n*. Levanta a exceção *StatisticsError* se não houver pelo menos um ponto de dados.

Os pontos de corte são linearmente interpolados a partir dos dois pontos mais próximos. Por exemplo, se um ponto de corte cair em um terço da distância entre dois valores, `100` e `112`, o ponto de corte será avaliado como `104`.

O *method* para computar quantis pode variar dependendo se *data* incluir ou excluir os menores e maiores valores possíveis da população.

O valor padrão do parâmetro *method* é "exclusive" e é usado para dados amostrados de uma população que pode ter valores mais extremos do que os encontrados nas amostras. A porção da população que fica abaixo do *i-ésimo* item de *m* pontos ordenados é calculada como `i / (m + 1)`. Dados nove valores, o método os ordena e atribui a eles os seguintes percentis: 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%.

Definimos o parâmetro *method* para "inclusive" para descrever dados da população ou para amostras que são conhecidas por incluir os valores mais extremos da população. O mínimo valor em *data* é tratado como o percentil 0 e o máximo valor é tratado como percentil 100. A porção da população que fica abaixo do *i-ésimo* item de *m* pontos ordenados é calculada como `(i - 1) / (m - 1)`. Dados 11 valores, o método os ordena e atribui a eles os seguintes percentis: 0%, 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%, 100%.

```
# Pontos de corte de decil para dados amostrados empiricamente
>>> data = [105, 129, 87, 86, 111, 111, 89, 81, 108, 92, 110,
...         100, 75, 105, 103, 109, 76, 119, 99, 91, 103, 129,
...         106, 101, 84, 111, 74, 87, 86, 103, 103, 106, 86,
...         111, 75, 87, 102, 121, 111, 88, 89, 101, 106, 95,
...         103, 107, 101, 81, 109, 104]
>>> [round(q, 1) for q in quantiles(data, n=10)]
[81.0, 86.2, 89.0, 99.4, 102.5, 103.6, 106.0, 109.8, 111.0]
```

Adicionado na versão 3.8.

Alterado na versão 3.13: Não levanta mais uma exceção para uma entrada com apenas um único ponto de dados. Isso permite que estimativas de quantil sejam construídas um ponto de amostra por vez, tornando-se gradualmente mais refinadas com cada novo ponto de dados.

`statistics.`**`covariance`**(*x*, *y*, */*)

Retorna a covariância da amostra de duas entradas *x* e *y*. Covariância é uma medida da variabilidade conjunta de duas entradas.

Ambas as entradas devem ter o mesmo comprimento (não menos que dois), caso contrário, *`StatisticsError`* será levantada.

Exemplos:

```
>>> x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> y = [1, 2, 3, 1, 2, 3, 1, 2, 3]
>>> covariance(x, y)
0.75
>>> z = [9, 8, 7, 6, 5, 4, 3, 2, 1]
>>> covariance(x, z)
-7.5
>>> covariance(z, x)
-7.5
```

Adicionado na versão 3.10.

`statistics.`**`correlation`**(*x*, *y*, */*, *, *method='linear'*)

Retorna o coeficiente de correlação de Pearson para duas entradas. O coeficiente de correlação de Pearson *r* assume valores entre -1 e +1. Ele mede a força e a direção de uma relação linear.

Se *method* for "ranked", calcula coeficiente de correlação de classificação de Spearman para duas entradas. Os dados são substituídos por classificações. Os empates são calculados em média para que valores iguais recebam a mesma classificação. O coeficiente resultante mede a força de um relacionamento monotônico.

O coeficiente de correlação de Spearman é apropriado para dados ordinais ou para dados contínuos que não atendem ao requisito de proporção linear para o coeficiente de correlação de Pearson.

Ambas as entradas devem ter o mesmo comprimento (não menos que dois) e não precisam ser constantes, caso contrário, *StatisticsError* será levantada.

Exemplo com leis de Kepler do movimento planetário:

```
>>> # Mercúrio, Venus, Terra, Marte, Jupiter, Saturno, Urano e Netuno
>>> orbital_period = [88, 225, 365, 687, 4331, 10_756, 30_687, 60_190]    #␣
↪dias
>>> dist_from_sun = [58, 108, 150, 228, 778, 1_400, 2_900, 4_500] # milhões de␣
↪km

>>> # Mostra que existe uma perfeita relação monótona
>>> correlation(orbital_period, dist_from_sun, method='ranked')
1.0

>>> # Observe que uma relação linear é imperfeita
>>> round(correlation(orbital_period, dist_from_sun), 4)
0.9882

>>> # Demonstra a terceira lei de Kepler: Há uma correlação linear
>>> # entre o quadrado do período orbital e o cubo da
>>> # distância a partir do sol.
>>> period_squared = [p * p for p in orbital_period]
>>> dist_cubed = [d * d * d for d in dist_from_sun]
>>> round(correlation(period_squared, dist_cubed), 4)
1.0
```

Adicionado na versão 3.10.

Alterado na versão 3.12: Adicionado suporte para o coeficiente de correlação de classificação de Spearman.

`statistics.`**`linear_regression`**(*x*, *y*, */*, ***, *proportional=False*)

Retorna a inclinação e a interceptação dos parâmetros de regressão linear simples estimados usando mínimos quadrados ordinários. A regressão linear simples descreve a relação entre uma variável independente *x* e uma variável dependente *y* em termos desta função linear:

*y = slope * x + intercept + noise*

onde `slope` e `intercept` são os parâmetros de regressão que são estimados, e `noise` representa a variabilidade dos dados que não foi explicada pela regressão linear (é igual à diferença entre os valores previstos e reais da variável dependente).

Ambas as entradas devem ter o mesmo comprimento (não menos que dois) e a variável independente *x* não pode ser constante; caso contrário, um *StatisticsError* será levantada.

Por exemplo, podemos usar as datas de lançamento dos filmes do Monty Python para prever o número cumulativo de filmes do Monty Python que teriam sido produzidos até 2019, supondo que eles tivessem mantido o ritmo.

```
>>> year = [1971, 1975, 1979, 1982, 1983]
>>> films_total = [1, 2, 3, 4, 5]
>>> slope, intercept = linear_regression(year, films_total)
>>> round(slope * 2019 + intercept)
16
```

Se *proportional* for verdadeiro, a variável independente *x* e a variável dependente *y* são assumidas como diretamente proporcionais. Os dados são ajustados a uma linha que passa pela origem. Como o *intercept* sempre será 0,0, a função linear subjacente simplifica para:

*y = slope * x + noise*

Continuando o exemplo de *correlation()*, vamos ver o quão bem um modelo baseado em planetas principais pode prever as distâncias orbitais de planetas anões:

```
>>> model = linear_regression(period_squared, dist_cubed, proportional=True)
>>> slope = model.slope

>>> # Planetas anões:   Pluto,  Eris,    Makemake, Haumea, Ceres
>>> orbital_periods = [90_560, 204_199, 111_845, 103_410, 1_680]  # dias
>>> predicted_dist = [math.cbrt(slope * (p * p)) for p in orbital_periods]
>>> list(map(round, predicted_dist))
[5912, 10166, 6806, 6459, 414]

>>> [5_906, 10_152, 6_796, 6_450, 414]  # distância real em milhões de km
[5906, 10152, 6796, 6450, 414]
```

Adicionado na versão 3.10.

Alterado na versão 3.11: Adicionado suporte a *proportional*.

9.7.5 Exceções

Uma única exceção é definida:

exception statistics.**StatisticsError**

Subclasse de *ValueError* para exceções relacionadas a estatísticas.

9.7.6 Objetos `NormalDist`

NormalDist é uma ferramenta para criar e manipular distribuições normais de uma variável aleatória. É uma classe que trata a média e o desvio padrão das medições de dados como uma entidade única.

Distribuições normais surgem do Teorema Central do Limite e possuem uma gama de aplicações em estatísticas.

class statistics.**NormalDist**(*mu=0.0*, *sigma=1.0*)

Retorna um novo objeto *NormalDist* onde *mu* representa a média aritmética e *sigma* representa o desvio padrão.

Se *sigma* for negativo, levanta a exceção *StatisticsError*.

mean

Uma propriedade somente leitura para a média aritmética de uma distribuição normal.

median

Uma propriedade somente leitura para a mediana de uma distribuição normal.

mode

Uma propriedade somente leitura para a moda de uma distribuição normal.

stdev

Uma propriedade somente leitura para o desvio padrão de uma distribuição normal.

variance

Uma propriedade somente leitura para a variância de uma distribuição normal. Igual ao quadrado do desvio padrão.

classmethod **from_samples**(*data*)

Faz uma instância da distribuição normal com os parâmetros *mu* e *sigma* estimados a partir de *data* usando *fmean()* e *stdev()*.

data pode ser qualquer *iterável* e deve consistir de valores que pode ser convertidos para o tipo *float*. Se *data* não contém pelo menos dois elementos, levanta a exceção *StatisticsError* porque é preciso pelo menos um ponto para estimar um valor central e pelo menos dois pontos para estimar a dispersão.

samples(*n*, *, *seed=None*)

Gera *n* amostras aleatórias para uma dada média e desvio padrão. Retorna uma *list* de valores *float*.

Se o parâmetro *seed* for fornecido, cria uma nova instância do gerador de número aleatório subjacente. Isso é útil para criar resultados reproduzíveis, mesmo em um contexto multithreading.

Alterado na versão 3.13.

Alterado para um algoritmo mais rápido. Para reproduzir amostras de versões anteriores, use *random.seed()* e *random.gauss()*.

pdf(*x*)

Usando uma função densidade de probabilidade (fdp), calcula a probabilidade relativa que uma variável aleatória *X* estará perto do valor dado *x*. Matematicamente, é o limite da razão `P(x <= X < x+dx) / dx` quando *dx* se aproxima de zero.

A probabilidade relativa é calculada como a probabilidade de uma amostra ocorrer em um intervalo estreito dividida pela largura do intervalo (daí a palavra "densidade"). Como a probabilidade é relativa a outros pontos, seu valor pode ser maior que `1.0`.

cdf(*x*)

Usando uma função distribuição acumulada (fda), calcula a probabilidade de que uma variável aleatória *X* seja menor ou igual a *x*. Matematicamente, é representada da seguinte maneira: `P(X <= x)`.

inv_cdf(*p*)

Calcula a função distribuição acumulada inversa, também conhecida como função quantil ou o função ponto percentual. Matematicamente, é representada como `x : P(X <= x) = p`.

Encontra o valor *x* da variável aleatória *X* de tal forma que a probabilidade da variável ser menor ou igual a esse valor seja igual à probabilidade dada *p*.

overlap(*other*)

Mede a concordância entre duas distribuições de probabilidade normais. Retorna um valor entre 0,0 e 1,0 fornecendo a área de sobreposição para as duas funções de densidade de probabilidade.

quantiles(*n=4*)

Divide a distribuição normal em *n* intervalos contínuos com probabilidade igual. Retorna uma lista de (n - 1) pontos de corte separando os intervalos.

Defina *n* como 4 para quartis (o padrão). Defina *n* como 10 para decis. Defina *n* como 100 para percentis, o que dá os 99 pontos de corte que separam a distribuição normal em 100 grupos de tamanhos iguais.

zscore(*x*)

Calcula a Pontuação Padrão (z-score) descrevendo *x* em termos do número de desvios padrão acima ou abaixo da média da distribuição normal: `(x - mean) / stdev`.

Adicionado na versão 3.9.

Instâncias de *NormalDist* suportam adição, subtração, multiplicação e divisão por uma constante. Essas operações são usadas para translação e dimensionamento. Por exemplo:

```
>>> temperature_february = NormalDist(5, 2.5)             # Celsius
>>> temperature_february * (9/5) + 32                     # Fahrenheit
NormalDist(mu=41.0, sigma=4.5)
```

A divisão de uma constante por uma instância de *NormalDist* não é suportada porque o resultado não seria distribuído normalmente.

Uma vez que distribuições normais surgem de efeitos aditivos de variáveis independentes, é possível adicionar e subtrair duas variáveis aleatórias independentes normalmente distribuídas representadas como instâncias de *NormalDist*. Por exemplo:

```
>>> birth_weights = NormalDist.from_samples([2.5, 3.1, 2.1, 2.4, 2.7, 3.5])
>>> drug_effects = NormalDist(0.4, 0.15)
>>> combined = birth_weights + drug_effects
>>> round(combined.mean, 1)
3.1
>>> round(combined.stdev, 1)
0.5
```

Adicionado na versão 3.8.

9.7.7 Exemplos e receitas

Problemas clássicos de probabilidade

NormalDist facilmente resolve problemas de probabilidade clássicos.

Por exemplo, considerando os dados históricos para exames SAT mostrando que as pontuações são normalmente distribuídas com média de 1060 e desvio padrão de 195, determine o percentual de alunos com pontuações de teste entre 1100 e 1200, após arredondar para o número inteiro mais próximo:

```
>>> sat = NormalDist(1060, 195)
>>> fraction = sat.cdf(1200 + 0.5) - sat.cdf(1100 - 0.5)
>>> round(fraction * 100.0, 1)
18.4
```

Encontrar os quartis e decis para as pontuações SAT:

```
>>> list(map(round, sat.quantiles()))
[928, 1060, 1192]
>>> list(map(round, sat.quantiles(n=10)))
[810, 896, 958, 1011, 1060, 1109, 1162, 1224, 1310]
```

Entradas de Monte Carlo para simulações

Para estimar a distribuição de um modelo que não seja fácil de resolver analiticamente, *NormalDist* pode gerar amostras de entrada para uma simulação Monte Carlo:

```
>>> def model(x, y, z):
...     return (3*x + 7*x*y - 5*y) / (11 * z)
...
>>> n = 100_000
>>> X = NormalDist(10, 2.5).samples(n, seed=3652260728)
>>> Y = NormalDist(15, 1.75).samples(n, seed=4582495471)
>>> Z = NormalDist(50, 1.25).samples(n, seed=6582483453)
>>> quantiles(map(model, X, Y, Z))
[1.4591308524824727, 1.8035946855390597, 2.175091447274739]
```

Aproximação de distribuições binomiais

As distribuições normais podem ser usadas para aproximar distribuições binomiais quando o tamanho da amostra é grande e quando a probabilidade de um teste bem-sucedido é próximo a 50%.

Por exemplo, uma conferência de código aberto tem 750 participantes e duas salas com capacidade para 500 pessoas. Há uma palestra sobre Python e outra sobre Ruby. Em conferências anteriores, 65% dos participantes preferiram ouvir palestras sobre Python. Supondo que as preferências da população não tenham mudado, qual é a probabilidade da sala de Python permanecer dentro de seus limites de capacidade?

```
>>> n = 750             # Tamanho da amostra
>>> p = 0.65            # Preferência para Python
>>> q = 1.0 - p         # Preferência para Ruby
>>> k = 500             # Capacidade

>>> # Aproximação usando a distribuição normal cumulativa
>>> from math import sqrt
>>> round(NormalDist(mu=n*p, sigma=sqrt(n*p*q)).cdf(k + 0.5), 4)
0.8402

>>> # Solução exata usando a distribuição binomial cumulativa
>>> from math import comb, fsum
>>> round(fsum(comb(n, r) * p**r * q**(n-r) for r in range(k+1)), 4)
0.8402

>>> # Aproximação usando uma simulação
>>> from random import seed, binomialvariate
>>> seed(8675309)
>>> mean(binomialvariate(n, p) <= k for i in range(10_000))
0.8406
```

Classificador bayesiano ingênuo

Distribuições normais geralmente surgem em problemas de aprendizado de máquina.

A Wikipedia tem um bom exemplo de um Classificador Bayesiano Ingênuo. O desafio é prever o sexo de uma pessoa a partir de medidas de características normalmente distribuídas, incluindo altura, peso e tamanho do pé.

Recebemos um conjunto de dados de treinamento com medições para oito pessoas. As medidas são consideradas normalmente distribuídas, então resumimos os dados com *NormalDist*:

```
>>> height_male = NormalDist.from_samples([6, 5.92, 5.58, 5.92])
>>> height_female = NormalDist.from_samples([5, 5.5, 5.42, 5.75])
>>> weight_male = NormalDist.from_samples([180, 190, 170, 165])
>>> weight_female = NormalDist.from_samples([100, 150, 130, 150])
>>> foot_size_male = NormalDist.from_samples([12, 11, 12, 10])
>>> foot_size_female = NormalDist.from_samples([6, 8, 7, 9])
```

Em seguida, encontramos uma nova pessoa cujas características de medidas são conhecidas, mas cujo gênero é desconhecido:

```
>>> ht = 6.0        # altura
>>> wt = 130        # peso
>>> fs = 8          # tamanho do pé
```

Começando com uma probabilidade a priori de 50% de ser homem ou mulher, calculamos a posteriori como a priori vezes o produto das probabilidade para as características de medidas dado o gênero:

```
>>> prior_male = 0.5
>>> prior_female = 0.5
>>> posterior_male = (prior_male * height_male.pdf(ht) *
...                   weight_male.pdf(wt) * foot_size_male.pdf(fs))

>>> posterior_female = (prior_female * height_female.pdf(ht) *
...                     weight_female.pdf(wt) * foot_size_female.pdf(fs))
```

A previsão final vai para a probabilidade posterior maior. Isso é conhecido como máximo a posteriori ou MAP:

```
>>> 'male' if posterior_male > posterior_female else 'female'
'female'
```

CAPÍTULO 10

Módulos de Programação Funcional

Os módulos descritos neste capítulo fornecem funções e classes que suportam um estilo de programação funcional e operações gerais em chamáveis.

Os seguintes módulos estão documentados neste capítulo:

10.1 `itertools` — Funções que criam iteradores para laços eficientes

Esse módulo implementa diversos blocos de instruções com *iteradores*, inspirados por construções de APL, Haskell, e SML. Cada uma foi adequadamente reformulada para Python.

Esse módulo padroniza um conjunto central de ferramentas rápidas e de uso eficiente da memória, que podem ser utilizadas sozinhas ou combinadas. Juntas, eles formam uma "álgebra de iteradores" tornando possível construir ferramentas sucintas e eficientes em Python puro.

Por exemplo, SML fornece uma ferramenta para tabulação: `tabulate(f)` que produz uma sequência `f(0), f(1), ...`. O mesmo efeito pode ser obtido em Python combinando *`map()`* e *`count()`* para formar `map(f, count())`.

Iteradores infinitos:

Iterador	Argumentos	Resultado	Exemplo
`count()`	[start[, step]]	start, start+step, start+2*step, …	`count(10) → 10 11 12 13 14 ...`
`cycle()`	p	p0, p1, … ultimo elemento de p, p0, p1, …	`cycle('ABCD') → A B C D A B C D ...`
`repeat()`	elem [,n]	elem, elem, elem, … repete infinitamente ou até n vezes	`repeat(10, 3) → 10 10 10`

Iteradores terminando na sequencia de entrada mais curta:

Iterador	Argumentos	Resultado	Exemplo
accumulate()	p [,func]	p0, p0+p1, p0+p1+p2, …	`accumulate([1,2,3,4,5]) → 1 3 6 10 15`
batched()	p, n	(p0, p1, …, p_n-1), …	`batched('ABCDEFG', n=2) → AB CD EF G`
chain()	p, q, …	p0, p1, … último elemento de p, q0, q1, …	`chain('ABC', 'DEF') → A B C D E F`
chain.from_iterable	iterável	p0, p1, … último elemento de p, q0, q1, …	`chain.from_iterable(['ABC', 'DEF']) → A B C D E F`
compress()	data, selectors	(d[0] if s[0]), (d[1] if s[1]), …	`compress('ABCDEF', [1,0,1,0,1,1]) → A C E F`
dropwhile()	predicate, seq	seq[n], seq[n+1], começando quando o predicate falha	`dropwhile(lambda x: x<5, [1,4,6,3,8]) → 6 3 8`
filterfalse()	predicate, seq	elementos de seq onde predicate(elem) falha	`filterfalse(lambda x: x<5, [1,4,6,3,8]) → 6 8`
groupby()	iterable[, key]	sub-iteradores agrupados pelo valor de key(v)	`groupby(['A','B','DEF'], len) → (1, A B) (3, DEF)`
islice()	seq, [start,] stop [, step]	elementos de seq[start:stop:step]	`islice('ABCDEFG', 2, None) → C D E F G`
pairwise()	iterável	(p[0], p[1]), (p[1], p[2])	`pairwise('ABCDEFG') → AB BC CD DE EF FG`
starmap()	func, seq	func(*seq[0]), func(*seq[1]), …	`starmap(pow, [(2,5), (3,2), (10,3)]) → 32 9 1000`
takewhile()	predicate, seq	seq[0], seq[1], até predicate falhar	`takewhile(lambda x: x<5, [1,4,6,3,8]) → 1 4`
tee()	it, n	n iteradores it independentes	`tee('ABC', 2) → A B C, A B C`
zip_longest()	p, q, …	(p[0], q[0]), (p[1], q[1]), …	`zip_longest('ABCD', 'xy', fillvalue='-') → Ax By C- D-`

Iteradores combinatórios:

Iterador	Argumentos	Resultado
product()	p, q, … [repeat=1]	produto cartesiano, equivalente a laços `for` aninhados
permutations()	p[, r]	tuplas de tamanho r, com todas ordenações possíveis, sem elementos repetidos
combinations()	p, r	tuplas de tamanho r, ordenadas, sem elementos repetidos
combinations_with_replace	p, r	tuplas de tamanho r, ordenadas, com elementos repetidos

Exemplos	Resultado
`product('ABCD', repeat=2)`	`AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD`
`permutations('ABCD', 2)`	`AB AC AD BA BC BD CA CB CD DA DB DC`
`combinations('ABCD', 2)`	`AB AC AD BC BD CD`
`combinations_with_replacement('ABCD', 2)`	`AA AB AC AD BB BC BD CC CD DD`

10.1.1 Funções do intertools

Todas as funções a seguir constroem e retornam iteradores. Algumas fornecem fluxos de comprimento infinito, então elas devem ser acessadas somente por funções ou laços que truncam o fluxo.

itertools.**accumulate**(*iterable*[, *function*, *, *initial=None*])

Constrói um iterador que retorna somas acumuladas, ou resultados acumulados de outras funções binárias.

A função *function* assume como padrão a adição. *function* deve aceitar dois argumentos, um total acumulado e um valor de *iterable*.

Se um valor inicial *initial* for fornecido, o acúmulo começará com esse valor e a saída terá um elemento a mais que o iterável de entrada.

Aproximadamente equivalente a:

```
def accumulate(iterable, function=operator.add, *, initial=None):
    'Retorna totais atuais'
    # accumulate([1,2,3,4,5]) → 1 3 6 10 15
    # accumulate([1,2,3,4,5], initial=100) → 100 101 103 106 110 115
    # accumulate([1,2,3,4,5], operator.mul) → 1 2 6 24 120

    iterator = iter(iterable)
    total = initial
    if initial is None:
        try:
            total = next(iterator)
        except StopIteration:
            return

    yield total
    for element in iterator:
        total = function(total, element)
        yield total
```

Para calcular um mínimo atual, defina *function* como `min()`. Para um máximo atual, defina *function* como `max()`. Ou para um produto atual, defina *function* como `operator.mul()`. Para construir uma tabela de amortização, acumular os juros e aplicar os pagamentos:

```
>>> data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]
>>> list(accumulate(data, max))                # máximo atual
[3, 4, 6, 6, 6, 9, 9, 9, 9, 9]
>>> list(accumulate(data, operator.mul))       # produto atual
[3, 12, 72, 144, 144, 1296, 0, 0, 0, 0]

# Amortiza um empréstimo de 5% de 1000 com 10 pagamentos anuais de 90
>>> update = lambda balance, payment: round(balance * 1.05) - payment
>>> list(accumulate(repeat(90, 10), update, initial=1_000))
[1000, 960, 918, 874, 828, 779, 728, 674, 618, 559, 497]
```

Veja `functools.reduce()` para uma função similar que devolve apenas o valor acumulado final.

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionado o parâmetro opcional *function*.

Alterado na versão 3.8: Adicionado o parâmetro opcional *initial*.

itertools.**batched**(*iterable*, *n*, *, *strict=False*)

Dados de lote do *iterable* em tuplas de comprimento *n*. O último lote pode ser menor que *n*.

Se *strict* for verdadeiro, será levantada uma `ValueError` se o lote final for menor que *n*.

Faz um laço sobre o iterável de entrada e acumula dados em tuplas de até o tamanho *n*. A entrada é consumida preguiçosamente, apenas o suficiente para preencher um lote. O resultado é produzido assim que o lote estiver cheio ou quando o iterável de entrada estiver esgotado:

```
>>> flattened_data = ['roses', 'red', 'violets', 'blue', 'sugar', 'sweet']
>>> unflattened = list(batched(flattened_data, 2))
>>> unflattened
[('roses', 'red'), ('violets', 'blue'), ('sugar', 'sweet')]
```

Aproximadamente equivalente a:

```
def batched(iterable, n, *, strict=False):
    # batched('ABCDEFG', 2) → AB CD EF G
    if n < 1:
        raise ValueError('n must be at least one')
    iterator = iter(iterable)
    while batch := tuple(islice(iterator, n)):
        if strict and len(batch) != n:
            raise ValueError('batched(): incomplete batch')
        yield batch
```

Adicionado na versão 3.12.

Alterado na versão 3.13: Adicionada a opção *strict*.

`itertools.`**`chain`**`(`*iterables*`)`

Cria um iterador que devolve elementos do primeiro iterável até o esgotamento, então prossiga para o próximo iterável, até que todos os iteráveis sejam esgotados Isso combina várias fontes de dados em um único iterador. Aproximadamente equivalente a:

```
def chain(*iterables):
    # chain('ABC', 'DEF') → A B C D E F
    for iterable in iterables:
        yield from iterable
```

`classmethod` `chain.`**`from_iterable`**`(`*iterable*`)`

Construtor alternativo para `chain()`. Obtém entradas encadeadas a partir de um único argumento iterável que avaliado preguiçosamente. Aproximadamente equivalente a:

```
def from_iterable(iterables):
    # chain.from_iterable(['ABC', 'DEF']) → A B C D E F
    for iterable in iterables:
        yield from iterable
```

`itertools.`**`combinations`**`(`*iterable*, *r*`)`

Devolve subsequências de elementos com comprimento *r* a partir da entrada *iterável*

A saída é uma subsequência de `product()` mantendo apenas entradas que são subsequências do *iterable*. O comprimento da saída é dado por `math.comb()` que calcula `n! / r! / (n - r)!` quando `0 ≤ r ≤ n` ou zero quando `r > n`.

As tuplas das combinações são emitidas em ordem lexicográfica de acordo com a ordem do iterável de entrada. Se o iterável estiver ordenado, as tuplas de combinação serão produzidas em sequência ordenada.

Os elementos são tratados como únicos baseado em suas posições, não em seus valores. Se os elementos de entrada são únicos, não haverá repetição de valores nas sucessivas combinações.

Aproximadamente equivalente a:

```
def combinations(iterable, r):
    # combinations('ABCD', 2) → AB AC AD BC BD CD
    # combinations(range(4), 3) → 012 013 023 123

    pool = tuple(iterable)
```

(continua na próxima página)

(continuação da página anterior)

```
    n = len(pool)
    if r > n:
        return
    indices = list(range(r))

    yield tuple(pool[i] for i in indices)
    while True:
        for i in reversed(range(r)):
            if indices[i] != i + n - r:
                break
        else:
            return
        indices[i] += 1
        for j in range(i+1, r):
            indices[j] = indices[j-1] + 1
        yield tuple(pool[i] for i in indices)
```

itertools.**combinations_with_replacement**(*iterable*, *r*)

Devolve subsequências de comprimento *r* de elementos do *iterável* de entrada permitindo que elementos individuais sejam repetidos mais de uma vez.

A saída é uma subsequência de *product()* que mantém apenas entradas que são subsequências (com possíveis elementos repetidos) do *iterable*. O número de subsequências retornadas é `(n + r - 1)! / r! / (n - 1)!` quando `n > 0`.

As tuplas das combinações são emitidas em ordem lexicográfica de acordo com a ordem do iterável de entrada. Se o iterável estiver ordenado, as tuplas de combinação serão produzidas em sequência ordenada.

Os elementos são tratados como únicos baseado em suas posições, não em seus valores. Se os elementos de entrada forem únicos, não haverá repetição de valores nas combinações geradas.

Aproximadamente equivalente a:

```
def combinations_with_replacement(iterable, r):
    # combinations_with_replacement('ABC', 2) → AA AB AC BB BC CC

    pool = tuple(iterable)
    n = len(pool)
    if not n and r:
        return
    indices = [0] * r

    yield tuple(pool[i] for i in indices)
    while True:
        for i in reversed(range(r)):
            if indices[i] != n - 1:
                break
        else:
            return
        indices[i:] = [indices[i] + 1] * (r - i)
        yield tuple(pool[i] for i in indices)
```

Adicionado na versão 3.1.

itertools.**compress**(*data*, *selectors*)

Cria um iterador que retorne elementos de *data* onde o elemento correspondente em *selectors* é verdadeiro. Para quando os iteráveis *data* ou *selectors* forem esgotados. Aproximadamente equivalente a:

```
def compress(data, selectors):
    # compress('ABCDEF', [1,0,1,0,1,1]) → A C E F
    return (datum for datum, selector in zip(data, selectors) if selector)
```

Adicionado na versão 3.1.

itertools.**count**(*start=0*, *step=1*)

Cria um iterador que retorne valores uniformemente espaçados começando com *start*. Pode ser usado com *map()* para gerar pontos de dados consecutivos ou com *zip()* para adicionar números de sequência. Aproximadamente equivalente a:

```
def count(start=0, step=1):
    # count(10) → 10 11 12 13 14 ...
    # count(2.5, 0.5) → 2.5 3.0 3.5 ...
    n = start
    while True:
        yield n
        n += step
```

Quando é feita uma contagem usando números de ponto flutuante, é possível ter melhor precisão substituindo código multiplicativo como `(start + step * i for i in count())`.

Alterado na versão 3.1: Adicionou argumento *step* e permitiu argumentos não-inteiros.

itertools.**cycle**(*iterable*)

Crie um iterador que devolve elementos do *iterable* assim como salva uma cópia de cada um. Quando o iterável é esgotado, devolve elementos da cópia salva. Repete indefinidamente. Aproximadamente equivalente a:

```
def cycle(iterable):
    # cycle('ABCD') → A B C D A B C D A B C D ...

    saved = []
    for element in iterable:
        yield element
        saved.append(element)

    while saved:
        for element in saved:
            yield element
```

Esta itertool pode exigir armazenamento auxiliar significativo (dependendo do comprimento do iterável).

itertools.**dropwhile**(*predicate*, *iterable*)

Cria um iterador que remove elementos do *iterable* enquanto o *predicate* for verdadeiro e, em seguida, retorna todos os elementos. Equivalente a:

```
def dropwhile(predicate, iterable):
    # dropwhile(lambda x: x<5, [1,4,6,3,8]) → 6 3 8

    iterator = iter(iterable)
    for x in iterator:
        if not predicate(x):
            yield x
            break

    for x in iterator:
        yield x
```

Observe que isso não produz *nenhuma* saída até que o predicado se torne falso, portanto, esta itertool pode ter um longo tempo de inicialização.

itertools.**filterfalse**(*predicate*, *iterable*)

Cria um iterador que filtre elementos do *iterable*, retornando apenas aqueles para os quais o *predicate* retorna um valor falso. Se o *predicate* for `None`, retorna os itens que são falsos. Equivalente aproximadamente a:

```
def filterfalse(predicate, iterable):
    # filterfalse(lambda x: x<5, [1,4,6,3,8]) → 6 8

    if predicate is None:
        predicate = bool

    for x in iterable:
        if not predicate(x):
            yield x
```

itertools.**groupby**(*iterable*, *key=None*)

Cria um iterador que retorna chaves e grupos consecutivos do *iterable*. A *key* é uma função que calcula um valor-chave para cada elemento. Se não for especificado ou for `None`, *key* assume como padrão uma função de identidade e retorna o elemento inalterado. Geralmente, o iterável precisa já estar ordenado na mesma função chave.

O funcionamento de *groupby()* é semelhante ao filtro `uniq` no Unix. Ele gera uma quebra ou um novo grupo sempre que o valor da função da tecla muda (razão pela qual geralmente é necessário ordenar os dados usando a mesma função da tecla). Esse comportamento difere do GROUP BY do SQL, que agrega elementos comuns independentemente da ordem de entrada.

O grupo retornado é, ele próprio, um iterador que compartilha o iterável subjacente com *groupby()*. Como a origem é compartilhada, quando o objeto *groupby()* avança, o grupo anterior não fica mais visível. Portanto, se esses dados forem necessários posteriormente, eles devem ser armazenados como uma lista:

```
groups = []
uniquekeys = []
data = sorted(data, key=keyfunc)
for k, g in groupby(data, keyfunc):
    groups.append(list(g))      # armazena um iterador de grupo como uma lista
    uniquekeys.append(k)
```

groupby() é aproximadamente equivalente a:

```
def groupby(iterable, key=None):
    # [k for k, g in groupby('AAAABBBCCDAABBB')] → A B C D A B
    # [list(g) for k, g in groupby('AAAABBBCCD')] → AAAA BBB CC D

    keyfunc = (lambda x: x) if key is None else key
    iterator = iter(iterable)
    exhausted = False

    def _grouper(target_key):
        nonlocal curr_value, curr_key, exhausted
        yield curr_value
        for curr_value in iterator:
            curr_key = keyfunc(curr_value)
            if curr_key != target_key:
                return
            yield curr_value
        exhausted = True

    try:
        curr_value = next(iterator)
    except StopIteration:
```

(continua na próxima página)

(continuação da página anterior)

```
        return
    curr_key = keyfunc(curr_value)

    while not exhausted:
        target_key = curr_key
        curr_group = _grouper(target_key)
        yield curr_key, curr_group
        if curr_key == target_key:
            for _ in curr_group:
                pass
```

itertools.**islice**(*iterable*, *stop*)

itertools.**islice**(*iterable*, *start*, *stop*[, *step*])

Cria um iterador que retorna os elementos selecionados do iterável. Funciona como o fatiamento de sequência, mas não oferece suporte a valores negativos para *start*, *stop* ou *step*.

Se *start* for zero ou `None`, a iteração começa em zero. Caso contrário, os elementos do iterável são pulados até que *start* seja alcançado.

Se *stop* for `None`, a iteração continua até que a entrada se esgote, se for o caso. Caso contrário, ela para na posição especificada.

Se *step* for `None`, o padrão do passo é 1. Os elementos são retornados consecutivamente, a menos que *step* seja definido como maior que um, o que resulta na omissão de itens.

Aproximadamente equivalente a:

```
def islice(iterable, *args):
    # islice('ABCDEFG', 2) → A B
    # islice('ABCDEFG', 2, 4) → C D
    # islice('ABCDEFG', 2, None) → C D E F G
    # islice('ABCDEFG', 0, None, 2) → A C E G

    s = slice(*args)
    start = 0 if s.start is None else s.start
    stop = s.stop
    step = 1 if s.step is None else s.step
    if start < 0 or (stop is not None and stop < 0) or step <= 0:
        raise ValueError

    indices = count() if stop is None else range(max(start, stop))
    next_i = start
    for i, element in zip(indices, iterable):
        if i == next_i:
            yield element
            next_i += step
```

Se a entrada for um iterador, o consumo total do *islice* avança o iterador de entrada em `max(start, stop)` passos, independentemente do valor do *step*.

itertools.**pairwise**(*iterable*)

Return successive overlapping pairs taken from the input *iterable*.

The number of 2-tuples in the output iterator will be one fewer than the number of inputs. It will be empty if the input iterable has fewer than two values.

Aproximadamente equivalente a:

```
def pairwise(iterable):
    # pairwise('ABCDEFG') → AB BC CD DE EF FG

    iterator = iter(iterable)
    a = next(iterator, None)

    for b in iterator:
        yield a, b
        a = b
```

Adicionado na versão 3.10.

itertools.**permutations**(*iterable*, *r=None*)

Return successive *r* length permutations of elements from the *iterable*.

If *r* is not specified or is `None`, then *r* defaults to the length of the *iterable* and all possible full-length permutations are generated.

The output is a subsequence of *`product()`* where entries with repeated elements have been filtered out. The length of the output is given by *`math.perm()`* which computes `n! / (n - r)!` when `0 ≤ r ≤ n` or zero when `r > n`.

The permutation tuples are emitted in lexicographic order according to the order of the input *iterable*. If the input *iterable* is sorted, the output tuples will be produced in sorted order.

Elements are treated as unique based on their position, not on their value. If the input elements are unique, there will be no repeated values within a permutation.

Aproximadamente equivalente a:

```
def permutations(iterable, r=None):
    # permutations('ABCD', 2) → AB AC AD BA BC BD CA CB CD DA DB DC
    # permutations(range(3)) → 012 021 102 120 201 210

    pool = tuple(iterable)
    n = len(pool)
    r = n if r is None else r
    if r > n:
        return

    indices = list(range(n))
    cycles = list(range(n, n-r, -1))
    yield tuple(pool[i] for i in indices[:r])

    while n:
        for i in reversed(range(r)):
            cycles[i] -= 1
            if cycles[i] == 0:
                indices[i:] = indices[i+1:] + indices[i:i+1]
                cycles[i] = n - i
            else:
                j = cycles[i]
                indices[i], indices[-j] = indices[-j], indices[i]
                yield tuple(pool[i] for i in indices[:r])
                break
        else:
            return
```

itertools.**product**(**iterables*, *repeat=1*)

Cartesian product of the input iterables.

Aproximadamente equivalente a laços for aninhados em uma expressão geradora. Por exemplo, `product(A, B)` devolve o mesmo que `((x,y) for x in A for y in B)`.

Os laços aninhados circulam como um hodômetro com o elemento mais à direita avançando a cada iteração. Este padrão cria uma ordenação lexicográfica de maneira que se os iteráveis de entrada estiverem ordenados, as tuplas produzidas são emitidas de maneira ordenada.

To compute the product of an iterable with itself, specify the number of repetitions with the optional *repeat* keyword argument. For example, `product(A, repeat=4)` means the same as `product(A, A, A, A)`.

This function is roughly equivalent to the following code, except that the actual implementation does not build up intermediate results in memory:

```
def product(*iterables, repeat=1):
    # product('ABCD', 'xy') → Ax Ay Bx By Cx Cy Dx Dy
    # product(range(2), repeat=3) → 000 001 010 011 100 101 110 111

    if repeat < 0:
        raise ValueError('repeat argument cannot be negative')
    pools = [tuple(pool) for pool in iterables] * repeat

    result = [[]]
    for pool in pools:
        result = [x+[y] for x in result for y in pool]

    for prod in result:
        yield tuple(prod)
```

Before *product()* runs, it completely consumes the input iterables, keeping pools of values in memory to generate the products. Accordingly, it is only useful with finite inputs.

`itertools.`**`repeat`**(*object*[, *times*])

Make an iterator that returns *object* over and over again. Runs indefinitely unless the *times* argument is specified.

Aproximadamente equivalente a:

```
def repeat(object, times=None):
    # repeat(10, 3) → 10 10 10
    if times is None:
        while True:
            yield object
    else:
        for i in range(times):
            yield object
```

Um uso comum de *repeat* é para fornecer um fluxo de valores constantes para *map* ou *zip*.

```
>>> list(map(pow, range(10), repeat(2)))
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

`itertools.`**`starmap`**(*function*, *iterable*)

Make an iterator that computes the *function* using arguments obtained from the *iterable*. Used instead of *map()* when argument parameters have already been "pre-zipped" into tuples.

The difference between *map()* and *starmap()* parallels the distinction between `function(a,b)` and `function(*c)`. Roughly equivalent to:

```
def starmap(function, iterable):
    # starmap(pow, [(2,5), (3,2), (10,3)]) → 32 9 1000
    for args in iterable:
        yield function(*args)
```

itertools.**takewhile**(*predicate*, *iterable*)

Make an iterator that returns elements from the *iterable* as long as the *predicate* is true. Roughly equivalent to:

```
def takewhile(predicate, iterable):
    # takewhile(lambda x: x<5, [1,4,6,3,8]) → 1 4
    for x in iterable:
        if not predicate(x):
            break
        yield x
```

Note, the element that first fails the predicate condition is consumed from the input iterator and there is no way to access it. This could be an issue if an application wants to further consume the input iterator after *takewhile* has been run to exhaustion. To work around this problem, consider using more-itertools before_and_after() instead.

itertools.**tee**(*iterable*, *n=2*)

Return *n* independent iterators from a single iterable.

Aproximadamente equivalente a:

```
def tee(iterable, n=2):
    if n < 0:
        raise ValueError
    if n == 0:
        return ()
    iterator = _tee(iterable)
    result = [iterator]
    for _ in range(n - 1):
        result.append(_tee(iterator))
    return tuple(result)

class _tee:

    def __init__(self, iterable):
        it = iter(iterable)
        if isinstance(it, _tee):
            self.iterator = it.iterator
            self.link = it.link
        else:
            self.iterator = it
            self.link = [None, None]

    def __iter__(self):
        return self

    def __next__(self):
        link = self.link
        if link[1] is None:
            link[0] = next(self.iterator)
            link[1] = [None, None]
        value, self.link = link
        return value
```

When the input *iterable* is already a tee iterator object, all members of the return tuple are constructed as if they had been produced by the upstream `tee()` call. This "flattening step" allows nested `tee()` calls to share the same underlying data chain and to have a single update step rather than a chain of calls.

The flattening property makes tee iterators efficiently peekable:

```
def lookahead(tee_iterator):
     "Return the next value without moving the input forward"
     [forked_iterator] = tee(tee_iterator, 1)
     return next(forked_iterator)
```

```
>>> iterator = iter('abcdef')
>>> [iterator] = tee(iterator, 1)    # Make the input peekable
>>> next(iterator)                   # Move the iterator forward
'a'
>>> lookahead(iterator)              # Check next value
'b'
>>> next(iterator)                   # Continue moving forward
'b'
```

`tee` iterators are not threadsafe. A *RuntimeError* may be raised when simultaneously using iterators returned by the same *tee()* call, even if the original *iterable* is threadsafe.

This itertool may require significant auxiliary storage (depending on how much temporary data needs to be stored). In general, if one iterator uses most or all of the data before another iterator starts, it is faster to use *list()* instead of *tee()*.

itertools.**zip_longest**(**iterables*, *fillvalue=None*)

Make an iterator that aggregates elements from each of the *iterables*.

If the iterables are of uneven length, missing values are filled-in with *fillvalue*. If not specified, *fillvalue* defaults to `None`.

Iteration continues until the longest iterable is exhausted.

Aproximadamente equivalente a:

```
def zip_longest(*iterables, fillvalue=None):
    # zip_longest('ABCD', 'xy', fillvalue='-') → Ax By C- D-

    iterators = list(map(iter, iterables))
    num_active = len(iterators)
    if not num_active:
        return

    while True:
        values = []
        for i, iterator in enumerate(iterators):
            try:
                value = next(iterator)
            except StopIteration:
                num_active -= 1
                if not num_active:
                    return
                iterators[i] = repeat(fillvalue)
                value = fillvalue
            values.append(value)
        yield tuple(values)
```

If one of the iterables is potentially infinite, then the *zip_longest()* function should be wrapped with something that limits the number of calls (for example *islice()* or *takewhile()*).

10.1.2 Receitas com itertools

Esta seção mostra receitas para criação de um ferramental ampliado usando as ferramentas existentes de itertools como elementos construtivos.

The primary purpose of the itertools recipes is educational. The recipes show various ways of thinking about individual tools — for example, that `chain.from_iterable` is related to the concept of flattening. The recipes also give ideas about ways that the tools can be combined — for example, how `starmap()` and `repeat()` can work together. The recipes also show patterns for using itertools with the *`operator`* and *`collections`* modules as well as with the built-in itertools such as `map()`, `filter()`, `reversed()`, and `enumerate()`.

A secondary purpose of the recipes is to serve as an incubator. The `accumulate()`, `compress()`, and `pairwise()` itertools started out as recipes. Currently, the `sliding_window()`, `iter_index()`, and `sieve()` recipes are being tested to see whether they prove their worth.

Substantially all of these recipes and many, many others can be installed from the more-itertools project found on the Python Package Index:

```
python -m pip install more-itertools
```

Many of the recipes offer the same high performance as the underlying toolset. Superior memory performance is kept by processing elements one at a time rather than bringing the whole iterable into memory all at once. Code volume is kept small by linking the tools together in a functional style. High speed is retained by preferring "vectorized" building blocks over the use of for-loops and *generators* which incur interpreter overhead.

```
from collections import Counter, deque
from contextlib import suppress
from functools import reduce
from math import comb, prod, sumprod, isqrt
from operator import itemgetter, getitem, mul, neg

def take(n, iterable):
    "Return first n items of the iterable as a list."
    return list(islice(iterable, n))

def prepend(value, iterable):
    "Prepend a single value in front of an iterable."
    # prepend(1, [2, 3, 4]) → 1 2 3 4
    return chain([value], iterable)

def tabulate(function, start=0):
    "Return function(0), function(1), ..."
    return map(function, count(start))

def repeatfunc(function, times=None, *args):
    "Repeat calls to a function with specified arguments."
    if times is None:
        return starmap(function, repeat(args))
    return starmap(function, repeat(args, times))

def flatten(list_of_lists):
    "Flatten one level of nesting."
    return chain.from_iterable(list_of_lists)

def ncycles(iterable, n):
    "Returns the sequence elements n times."
    return chain.from_iterable(repeat(tuple(iterable), n))

def loops(n):
    "Loop n times. Like range(n) but without creating integers."
```

(continua na próxima página)

(continuação da página anterior)

```
    # for _ in loops(100): ...
    return repeat(None, n)

def tail(n, iterable):
    "Return an iterator over the last n items."
    # tail(3, 'ABCDEFG') → E F G
    return iter(deque(iterable, maxlen=n))

def consume(iterator, n=None):
    "Advance the iterator n-steps ahead. If n is None, consume entirely."
    # Use functions that consume iterators at C speed.
    if n is None:
        deque(iterator, maxlen=0)
    else:
        next(islice(iterator, n, n), None)

def nth(iterable, n, default=None):
    "Returns the nth item or a default value."
    return next(islice(iterable, n, None), default)

def quantify(iterable, predicate=bool):
    "Given a predicate that returns True or False, count the True results."
    return sum(map(predicate, iterable))

def first_true(iterable, default=False, predicate=None):
    "Returns the first true value or the *default* if there is no true value."
    # first_true([a,b,c], x) → a or b or c or x
    # first_true([a,b], x, f) → a if f(a) else b if f(b) else x
    return next(filter(predicate, iterable), default)

def all_equal(iterable, key=None):
    "Returns True if all the elements are equal to each other."
    # all_equal('4□□□□', key=int) → True
    return len(take(2, groupby(iterable, key))) <= 1

def unique_justseen(iterable, key=None):
    "Yield unique elements, preserving order. Remember only the element just seen."
    # unique_justseen('AAAABBBCCDAABBB') → A B C D A B
    # unique_justseen('ABBcCAD', str.casefold) → A B c A D
    if key is None:
        return map(itemgetter(0), groupby(iterable))
    return map(next, map(itemgetter(1), groupby(iterable, key)))

def unique_everseen(iterable, key=None):
    "Yield unique elements, preserving order. Remember all elements ever seen."
    # unique_everseen('AAAABBBCCDAABBB') → A B C D
    # unique_everseen('ABBcCAD', str.casefold) → A B c D
    seen = set()
    if key is None:
        for element in filterfalse(seen.__contains__, iterable):
            seen.add(element)
            yield element
    else:
        for element in iterable:
            k = key(element)
            if k not in seen:
```

(continua na próxima página)

(continuação da página anterior)

```
                seen.add(k)
                yield element

def unique(iterable, key=None, reverse=False):
   "Yield unique elements in sorted order. Supports unhashable inputs."
   # unique([[1, 2], [3, 4], [1, 2]]) → [1, 2] [3, 4]
   sequenced = sorted(iterable, key=key, reverse=reverse)
   return unique_justseen(sequenced, key=key)

def sliding_window(iterable, n):
    "Collect data into overlapping fixed-length chunks or blocks."
    # sliding_window('ABCDEFG', 4) → ABCD BCDE CDEF DEFG
    iterator = iter(iterable)
    window = deque(islice(iterator, n - 1), maxlen=n)
    for x in iterator:
        window.append(x)
        yield tuple(window)

def grouper(iterable, n, *, incomplete='fill', fillvalue=None):
    "Collect data into non-overlapping fixed-length chunks or blocks."
    # grouper('ABCDEFG', 3, fillvalue='x') → ABC DEF Gxx
    # grouper('ABCDEFG', 3, incomplete='strict') → ABC DEF ValueError
    # grouper('ABCDEFG', 3, incomplete='ignore') → ABC DEF
    iterators = [iter(iterable)] * n
    match incomplete:
        case 'fill':
            return zip_longest(*iterators, fillvalue=fillvalue)
        case 'strict':
            return zip(*iterators, strict=True)
        case 'ignore':
            return zip(*iterators)
        case _:
            raise ValueError('Expected fill, strict, or ignore')

def roundrobin(*iterables):
    "Visit input iterables in a cycle until each is exhausted."
    # roundrobin('ABC', 'D', 'EF') → A D E B F C
    # Algorithm credited to George Sakkis
    iterators = map(iter, iterables)
    for num_active in range(len(iterables), 0, -1):
        iterators = cycle(islice(iterators, num_active))
        yield from map(next, iterators)

def subslices(seq):
    "Return all contiguous non-empty subslices of a sequence."
    # subslices('ABCD') → A AB ABC ABCD B BC BCD C CD D
    slices = starmap(slice, combinations(range(len(seq) + 1), 2))
    return map(getitem, repeat(seq), slices)

def iter_index(iterable, value, start=0, stop=None):
    "Return indices where a value occurs in a sequence or iterable."
    # iter_index('AABCADEAF', 'A') → 0 1 4 7
    seq_index = getattr(iterable, 'index', None)
    if seq_index is None:
        iterator = islice(iterable, start, stop)
        for i, element in enumerate(iterator, start):
```

(continua na próxima página)

(continuação da página anterior)

```
            if element is value or element == value:
                yield i
    else:
        stop = len(iterable) if stop is None else stop
        i = start
        with suppress(ValueError):
            while True:
                yield (i := seq_index(value, i, stop))
                i += 1

def iter_except(function, exception, first=None):
    "Convert a call-until-exception interface to an iterator interface."
    # iter_except(d.popitem, KeyError) → non-blocking dictionary iterator
    with suppress(exception):
        if first is not None:
            yield first()
        while True:
            yield function()
```

The following recipes have a more mathematical flavor:

```
def multinomial(*counts):
    "Number of distinct arrangements of a multiset."
    # Counter('abracadabra').values() → 5 2 2 1 1
    # multinomial(5, 2, 2, 1, 1) → 83160
    return prod(map(comb, accumulate(counts), counts))

def powerset(iterable):
    "Subsequences of the iterable from shortest to longest."
    # powerset([1,2,3]) → () (1,) (2,) (3,) (1,2) (1,3) (2,3) (1,2,3)
    s = list(iterable)
    return chain.from_iterable(combinations(s, r) for r in range(len(s)+1))

def sum_of_squares(iterable):
    "Add up the squares of the input values."
    # sum_of_squares([10, 20, 30]) → 1400
    return sumprod(*tee(iterable))

def reshape(matrix, columns):
    "Reshape a 2-D matrix to have a given number of columns."
    # reshape([(0, 1), (2, 3), (4, 5)], 3) →  (0, 1, 2), (3, 4, 5)
    return batched(chain.from_iterable(matrix), columns, strict=True)

def transpose(matrix):
    "Swap the rows and columns of a 2-D matrix."
    # transpose([(1, 2, 3), (11, 22, 33)]) → (1, 11) (2, 22) (3, 33)
    return zip(*matrix, strict=True)

def matmul(m1, m2):
    "Multiply two matrices."
    # matmul([(7, 5), (3, 5)], [(2, 5), (7, 9)]) → (49, 80), (41, 60)
    n = len(m2[0])
    return batched(starmap(sumprod, product(m1, transpose(m2))), n)

def convolve(signal, kernel):
    """Discrete linear convolution of two iterables.
```

(continua na próxima página)

(continuação da página anterior)

```
    Equivalent to polynomial multiplication.

    Convolutions are mathematically commutative; however, the inputs are
    evaluated differently.  The signal is consumed lazily and can be
    infinite. The kernel is fully consumed before the calculations begin.

    Article:  https://betterexplained.com/articles/intuitive-convolution/
    Video:    https://www.youtube.com/watch?v=KuXjwB4LzSA
    """
    # convolve([1, -1, -20], [1, -3]) → 1 -4 -17 60
    # convolve(data, [0.25, 0.25, 0.25, 0.25]) → Moving average (blur)
    # convolve(data, [1/2, 0, -1/2]) → 1st derivative estimate
    # convolve(data, [1, -2, 1]) → 2nd derivative estimate
    kernel = tuple(kernel)[::-1]
    n = len(kernel)
    padded_signal = chain(repeat(0, n-1), signal, repeat(0, n-1))
    windowed_signal = sliding_window(padded_signal, n)
    return map(sumprod, repeat(kernel), windowed_signal)

def polynomial_from_roots(roots):
    """Compute a polynomial's coefficients from its roots.

       (x - 5) (x + 4) (x - 3)  expands to:   x³ -4x² -17x + 60
    """
    # polynomial_from_roots([5, -4, 3]) → [1, -4, -17, 60]
    factors = zip(repeat(1), map(neg, roots))
    return list(reduce(convolve, factors, [1]))

def polynomial_eval(coefficients, x):
    """Evaluate a polynomial at a specific value.

    Computes with better numeric stability than Horner's method.
    """
    # Evaluate x³ -4x² -17x + 60 at x = 5
    # polynomial_eval([1, -4, -17, 60], x=5) → 0
    n = len(coefficients)
    if not n:
        return type(x)(0)
    powers = map(pow, repeat(x), reversed(range(n)))
    return sumprod(coefficients, powers)

def polynomial_derivative(coefficients):
    """Compute the first derivative of a polynomial.

       f(x)  =  x³ -4x² -17x + 60
       f'(x) = 3x² -8x  -17
    """
    # polynomial_derivative([1, -4, -17, 60]) → [3, -8, -17]
    n = len(coefficients)
    powers = reversed(range(1, n))
    return list(map(mul, coefficients, powers))

def sieve(n):
    "Primes less than n."
    # sieve(30) → 2 3 5 7 11 13 17 19 23 29
    if n > 2:
```

(continua na próxima página)

(continuação da página anterior)

```
        yield 2
    data = bytearray((0, 1)) * (n // 2)
    for p in iter_index(data, 1, start=3, stop=isqrt(n) + 1):
        data[p*p : n : p+p] = bytes(len(range(p*p, n, p+p)))
    yield from iter_index(data, 1, start=3)

def factor(n):
    "Prime factors of n."
    # factor(99) → 3 3 11
    # factor(1_000_000_000_000_007) → 47 59 360620266859
    # factor(1_000_000_000_000_403) → 1000000000000403
    for prime in sieve(isqrt(n) + 1):
        while not n % prime:
            yield prime
            n //= prime
            if n == 1:
                return
    if n > 1:
        yield n

def is_prime(n):
    "Return True if n is prime."
    # is_prime(1_000_000_000_000_403) → True
    return n > 1 and next(factor(n)) == n

def totient(n):
    "Count of natural numbers up to n that are coprime to n."
    # https://mathworld.wolfram.com/TotientFunction.html
    # totient(12) → 4 because len([1, 5, 7, 11]) == 4
    for prime in set(factor(n)):
        n -= n // prime
    return n
```

10.2 `functools` — Higher-order functions and operations on callable objects

Código-fonte: Lib/functools.py

O módulo *`functools`* é para funções de ordem superior: funções que atuam ou retornam outras funções. Em geral, qualquer objeto chamável pode ser tratado como uma função para os propósitos deste módulo.

O módulo *`functools`* define as seguintes funções:

`@functools.cache`(*user_function*)

Cache simples e leve de funções sem vínculo. Às vezes chamado de "memoizar".

Returns the same as `lru_cache(maxsize=None)`, creating a thin wrapper around a dictionary lookup for the function arguments. Because it never needs to evict old values, this is smaller and faster than *`lru_cache()`* with a size limit.

Por exemplo:

```
@cache
def factorial(n):
    return n * factorial(n-1) if n else 1
```

(continua na próxima página)

(continuação da página anterior)

```
>>> factorial(10)      # no previously cached result, makes 11 recursive calls
3628800
>>> factorial(5)       # just looks up cached value result
120
>>> factorial(12)      # makes two new recursive calls, the other 10 are cached
479001600
```

The cache is threadsafe so that the wrapped function can be used in multiple threads. This means that the underlying data structure will remain coherent during concurrent updates.

It is possible for the wrapped function to be called more than once if another thread makes an additional call before the initial call has been completed and cached.

Adicionado na versão 3.9.

`@functools.`**cached_property**(*func*)

Transforma um método de uma classe em uma propriedade cujo valor é calculado uma vez e, em seguida, armazenado em cache como um atributo normal para a vida útil da instância. Semelhante a *property()*, com a adição de armazenamento em cache. Útil para propriedades computadas caras de instâncias que são efetivamente imutáveis.

Exemplo:

```
class DataSet:

    def __init__(self, sequence_of_numbers):
        self._data = tuple(sequence_of_numbers)

    @cached_property
    def stdev(self):
        return statistics.stdev(self._data)
```

The mechanics of *cached_property()* are somewhat different from *property()*. A regular property blocks attribute writes unless a setter is defined. In contrast, a *cached_property* allows writes.

The *cached_property* decorator only runs on lookups and only when an attribute of the same name doesn't exist. When it does run, the *cached_property* writes to the attribute with the same name. Subsequent attribute reads and writes take precedence over the *cached_property* method and it works like a normal attribute.

The cached value can be cleared by deleting the attribute. This allows the *cached_property* method to run again.

The *cached_property* does not prevent a possible race condition in multi-threaded usage. The getter function could run more than once on the same instance, with the latest run setting the cached value. If the cached property is idempotent or otherwise not harmful to run more than once on an instance, this is fine. If synchronization is needed, implement the necessary locking inside the decorated getter function or around the cached property access.

Note, this decorator interferes with the operation of **PEP 412** key-sharing dictionaries. This means that instance dictionaries can take more space than usual.

Also, this decorator requires that the `__dict__` attribute on each instance be a mutable mapping. This means it will not work with some types, such as metaclasses (since the `__dict__` attributes on type instances are read-only proxies for the class namespace), and those that specify `__slots__` without including `__dict__` as one of the defined slots (as such classes don't provide a `__dict__` attribute at all).

If a mutable mapping is not available or if space-efficient key sharing is desired, an effect similar to *cached_property()* can also be achieved by stacking *property()* on top of *lru_cache()*. See faq-cache-method-calls for more details on how this differs from *cached_property()*.

Adicionado na versão 3.8.

Alterado na versão 3.12: Prior to Python 3.12, `cached_property` included an undocumented lock to ensure that in multi-threaded usage the getter function was guaranteed to run only once per instance. However, the lock was per-property, not per-instance, which could result in unacceptably high lock contention. In Python 3.12+ this locking is removed.

`functools.`**`cmp_to_key`**(*func*)

Transforma uma função de comparação de estilo antigo para um *função chave*. Usado com ferramentas que aceitam funções chave (como *`sorted()`*, *`min()`*, *`max()`*, *`heapq.nlargest()`*, *`heapq.nsmallest()`*, *`itertools.groupby()`*). Esta função é usada principalmente como uma ferramenta de transição para programas que estão sendo convertidos a partir do Python 2, que suportou o uso de funções de comparação.

A comparison function is any callable that accepts two arguments, compares them, and returns a negative number for less-than, zero for equality, or a positive number for greater-than. A key function is a callable that accepts one argument and returns another value to be used as the sort key.

Exemplo:

```
sorted(iterable, key=cmp_to_key(locale.strcoll))  # locale-aware sort order
```

Para exemplos de classificação e um breve tutorial de classificação, veja sortinghowto.

Adicionado na versão 3.2.

`@functools.`**`lru_cache`**(*user_function*)

`@functools.`**`lru_cache`**(*maxsize=128*, *typed=False*)

Decorador para embrulhar uma função com um chamável memoizável que economiza até as chamadas mais recentes *maxsize*. Pode economizar tempo quando uma função cara ou E/S é periodicamente chamada com os mesmos argumentos.

The cache is threadsafe so that the wrapped function can be used in multiple threads. This means that the underlying data structure will remain coherent during concurrent updates.

It is possible for the wrapped function to be called more than once if another thread makes an additional call before the initial call has been completed and cached.

Since a dictionary is used to cache results, the positional and keyword arguments to the function must be *hashable*.

Distinct argument patterns may be considered to be distinct calls with separate cache entries. For example, `f(a=1, b=2)` and `f(b=2, a=1)` differ in their keyword argument order and may have two separate cache entries.

Se *user_function* especificado, deve ser um chamável. Isso permite que o decorador *lru_cache* seja aplicado diretamente a uma função do usuário, deixando *maxsize* em seu valor padrão de 128:

```
@lru_cache
def count_vowels(sentence):
    return sum(sentence.count(vowel) for vowel in 'AEIOUaeiou')
```

Se *maxsize* for definido como `None`, o recurso LRU é desabilitado e o cache pode crescer sem limites.

If *typed* is set to true, function arguments of different types will be cached separately. If *typed* is false, the implementation will usually regard them as equivalent calls and only cache a single result. (Some types such as *str* and *int* may be cached separately even when *typed* is false.)

Note, type specificity applies only to the function's immediate arguments rather than their contents. The scalar arguments, `Decimal(42)` and `Fraction(42)` are be treated as distinct calls with distinct results. In contrast, the tuple arguments `('answer', Decimal(42))` and `('answer', Fraction(42))` are treated as equivalent.

The wrapped function is instrumented with a `cache_parameters()` function that returns a new *`dict`* showing the values for *maxsize* and *typed*. This is for information purposes only. Mutating the values has no effect. To help measure the effectiveness of the cache and tune the *maxsize* parameter, the wrapped function is instrumented with a `cache_info()` function that returns a *named tuple* showing *hits*, *misses*, *maxsize* and *currsize*. The decorator also provides a `cache_clear()` function for clearing or invalidating the cache.

A função subjacente original é acessível através do atributo `__wrapped__`. Isso é útil para introspecção, para ignorar o cache, ou para reinstalar a função com um cache diferente.

The cache keeps references to the arguments and return values until they age out of the cache or until the cache is cleared.

If a method is cached, the `self` instance argument is included in the cache. See faq-cache-method-calls

An LRU (least recently used) cache works best when the most recent calls are the best predictors of upcoming calls (for example, the most popular articles on a news server tend to change each day). The cache's size limit assures that the cache does not grow without bound on long-running processes such as web servers.

In general, the LRU cache should only be used when you want to reuse previously computed values. Accordingly, it doesn't make sense to cache functions with side-effects, functions that need to create distinct mutable objects on each call (such as generators and async functions), or impure functions such as time() or random().

Exemplo de um cache LRU para conteúdo web estático:

```
@lru_cache(maxsize=32)
def get_pep(num):
    'Retrieve text of a Python Enhancement Proposal'
    resource = f'https://peps.python.org/pep-{num:04d}'
    try:
        with urllib.request.urlopen(resource) as s:
            return s.read()
    except urllib.error.HTTPError:
        return 'Not Found'

>>> for n in 8, 290, 308, 320, 8, 218, 320, 279, 289, 320, 9991:
...     pep = get_pep(n)
...     print(n, len(pep))

>>> get_pep.cache_info()
CacheInfo(hits=3, misses=8, maxsize=32, currsize=8)
```

Exemplo de computação eficiente dos números Fibonacci usando um cache para implementar uma programação dinâmica técnica:

```
@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)

>>> [fib(n) for n in range(16)]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610]

>>> fib.cache_info()
CacheInfo(hits=28, misses=16, maxsize=None, currsize=16)
```

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionada a opção *typed*.

Alterado na versão 3.8: Adicionada a opção *user_function*.

Alterado na versão 3.9: Added the function `cache_parameters()`

`@functools.total_ordering`

Given a class defining one or more rich comparison ordering methods, this class decorator supplies the rest. This simplifies the effort involved in specifying all of the possible rich comparison operations:

The class must define one of `__lt__()`, `__le__()`, `__gt__()`, or `__ge__()`. In addition, the class should supply an `__eq__()` method.

Por exemplo:

```
@total_ordering
class Student:
    def _is_valid_operand(self, other):
        return (hasattr(other, "lastname") and
                hasattr(other, "firstname"))
    def __eq__(self, other):
        if not self._is_valid_operand(other):
            return NotImplemented
        return ((self.lastname.lower(), self.firstname.lower()) ==
                (other.lastname.lower(), other.firstname.lower()))
    def __lt__(self, other):
        if not self._is_valid_operand(other):
            return NotImplemented
        return ((self.lastname.lower(), self.firstname.lower()) <
                (other.lastname.lower(), other.firstname.lower()))
```

Nota

While this decorator makes it easy to create well behaved totally ordered types, it *does* come at the cost of slower execution and more complex stack traces for the derived comparison methods. If performance benchmarking indicates this is a bottleneck for a given application, implementing all six rich comparison methods instead is likely to provide an easy speed boost.

Nota

This decorator makes no attempt to override methods that have been declared in the class *or its superclasses*. Meaning that if a superclass defines a comparison operator, *total_ordering* will not implement it again, even if the original method is abstract.

Adicionado na versão 3.2.

Alterado na versão 3.4: Returning `NotImplemented` from the underlying comparison function for unrecognised types is now supported.

`functools.`**`Placeholder`**

A singleton object used as a sentinel to reserve a place for positional arguments when calling *`partial()`* and *`partialmethod()`*.

Adicionado na versão 3.14.

`functools.`**`partial`**(*func*, */*, **args*, ***keywords*)

Return a new *partial object* which when called will behave like *func* called with the positional arguments *args* and keyword arguments *keywords*. If more arguments are supplied to the call, they are appended to *args*. If additional keyword arguments are supplied, they extend and override *keywords*. Roughly equivalent to:

```
def partial(func, /, *args, **keywords):
    def newfunc(*more_args, **more_keywords):
        return func(*args, *more_args, **(keywords | more_keywords))
    newfunc.func = func
    newfunc.args = args
    newfunc.keywords = keywords
    return newfunc
```

The `partial()` function is used for partial function application which "freezes" some portion of a function's arguments and/or keywords resulting in a new object with a simplified signature. For example, *partial()* can be used to create a callable that behaves like the *int()* function where the *base* argument defaults to `2`:

```
>>> basetwo = partial(int, base=2)
>>> basetwo.__doc__ = 'Convert base 2 string to an int.'
>>> basetwo('10010')
18
```

If *Placeholder* sentinels are present in *args*, they will be filled first when `partial()` is called. This makes it possible to pre-fill any positional argument with a call to `partial()`; without `Placeholder`, only the chosen number of leading positional arguments can be pre-filled.

If any `Placeholder` sentinels are present, all must be filled at call time:

```
>>> say_to_world = partial(print, Placeholder, Placeholder, "world!")
>>> say_to_world('Hello', 'dear')
Hello dear world!
```

Calling `say_to_world('Hello')` raises a *TypeError*, because only one positional argument is provided, but there are two placeholders that must be filled in.

If `partial()` is applied to an existing `partial()` object, `Placeholder` sentinels of the input object are filled in with new positional arguments. A placeholder can be retained by inserting a new `Placeholder` sentinel to the place held by a previous `Placeholder`:

```
>>> from functools import partial, Placeholder as _
>>> remove = partial(str.replace, _, _, '')
>>> message = 'Hello, dear dear world!'
>>> remove(message, ' dear')
'Hello, world!'
>>> remove_dear = partial(remove, _, ' dear')
>>> remove_dear(message)
'Hello, world!'
>>> remove_first_dear = partial(remove_dear, _, 1)
>>> remove_first_dear(message)
'Hello, dear world!'
```

`Placeholder` cannot be passed to `partial()` as a keyword argument.

Alterado na versão 3.14: Added support for *Placeholder* in positional arguments.

class functools.**partialmethod**(*func*, */*, **args*, ***keywords*)

Return a new *partialmethod* descriptor which behaves like *partial* except that it is designed to be used as a method definition rather than being directly callable.

func must be a *descriptor* or a callable (objects which are both, like normal functions, are handled as descriptors).

When *func* is a descriptor (such as a normal Python function, *classmethod()*, *staticmethod()*, *abstractmethod()* or another instance of *partialmethod*), calls to `__get__` are delegated to the underlying descriptor, and an appropriate *partial object* returned as the result.

When *func* is a non-descriptor callable, an appropriate bound method is created dynamically. This behaves like a normal Python function when used as a method: the *self* argument will be inserted as the first positional argument, even before the *args* and *keywords* supplied to the *partialmethod* constructor.

Exemplo:

```
>>> class Cell:
...     def __init__(self):
...         self._alive = False
```

(continua na próxima página)

(continuação da página anterior)

```
...     @property
...     def alive(self):
...         return self._alive
...     def set_state(self, state):
...         self._alive = bool(state)
...     set_alive = partialmethod(set_state, True)
...     set_dead = partialmethod(set_state, False)
...
>>> c = Cell()
>>> c.alive
False
>>> c.set_alive()
>>> c.alive
True
```

Adicionado na versão 3.4.

`functools.`**`reduce`**(*function*, *iterable*, /[, *initial*])

Apply *function* of two arguments cumulatively to the items of *iterable*, from left to right, so as to reduce the iterable to a single value. For example, `reduce(lambda x, y: x+y, [1, 2, 3, 4, 5])` calculates `((((1+2)+3)+4)+5)`. The left argument, *x*, is the accumulated value and the right argument, *y*, is the update value from the *iterable*. If the optional *initial* is present, it is placed before the items of the iterable in the calculation, and serves as a default when the iterable is empty. If *initial* is not given and *iterable* contains only one item, the first item is returned.

Aproximadamente equivalente a:

```
initial_missing = object()

def reduce(function, iterable, /, initial=initial_missing):
    it = iter(iterable)
    if initial is initial_missing:
        value = next(it)
    else:
        value = initial
    for element in it:
        value = function(value, element)
    return value
```

See *`itertools.accumulate()`* for an iterator that yields all intermediate values.

Alterado na versão 3.14: *initial* is now supported as a keyword argument.

`@functools.`**`singledispatch`**

Transform a function into a *single-dispatch generic function*.

To define a generic function, decorate it with the `@singledispatch` decorator. When defining a function using `@singledispatch`, note that the dispatch happens on the type of the first argument:

```
>>> from functools import singledispatch
>>> @singledispatch
... def fun(arg, verbose=False):
...     if verbose:
...         print("Let me just say,", end=" ")
...     print(arg)
```

To add overloaded implementations to the function, use the `register()` attribute of the generic function, which can be used as a decorator. For functions annotated with types, the decorator will infer the type of the first argument automatically:

```
>>> @fun.register
... def _(arg: int, verbose=False):
...     if verbose:
...         print("Strength in numbers, eh?", end=" ")
...     print(arg)
...
>>> @fun.register
... def _(arg: list, verbose=False):
...     if verbose:
...         print("Enumerate this:")
...     for i, elem in enumerate(arg):
...         print(i, elem)
```

typing.Union can also be used:

```
>>> @fun.register
... def _(arg: int | float, verbose=False):
...     if verbose:
...         print("Strength in numbers, eh?", end=" ")
...     print(arg)
...
>>> from typing import Union
>>> @fun.register
... def _(arg: Union[list, set], verbose=False):
...     if verbose:
...         print("Enumerate this:")
...     for i, elem in enumerate(arg):
...         print(i, elem)
...
```

For code which doesn't use type annotations, the appropriate type argument can be passed explicitly to the decorator itself:

```
>>> @fun.register(complex)
... def _(arg, verbose=False):
...     if verbose:
...         print("Better than complicated.", end=" ")
...     print(arg.real, arg.imag)
...
```

For code that dispatches on a collections type (e.g., `list`), but wants to typehint the items of the collection (e.g., `list[int]`), the dispatch type should be passed explicitly to the decorator itself with the typehint going into the function definition:

```
>>> @fun.register(list)
... def _(arg: list[int], verbose=False):
...     if verbose:
...         print("Enumerate this:")
...     for i, elem in enumerate(arg):
...         print(i, elem)
```

Nota

At runtime the function will dispatch on an instance of a list regardless of the type contained within the list i.e. `[1,2,3]` will be dispatched the same as `["foo", "bar", "baz"]`. The annotation provided in this example is for static type checkers only and has no runtime impact.

To enable registering *lambdas* and pre-existing functions, the *register()* attribute can also be used in a functional form:

```
>>> def nothing(arg, verbose=False):
...     print("Nothing.")
...
>>> fun.register(type(None), nothing)
```

The *register()* attribute returns the undecorated function. This enables decorator stacking, *pickling*, and the creation of unit tests for each variant independently:

```
>>> @fun.register(float)
... @fun.register(Decimal)
... def fun_num(arg, verbose=False):
...     if verbose:
...         print("Half of your number:", end=" ")
...     print(arg / 2)
...
>>> fun_num is fun
False
```

When called, the generic function dispatches on the type of the first argument:

```
>>> fun("Hello, world.")
Hello, world.
>>> fun("test.", verbose=True)
Let me just say, test.
>>> fun(42, verbose=True)
Strength in numbers, eh? 42
>>> fun(['spam', 'spam', 'eggs', 'spam'], verbose=True)
Enumerate this:
0 spam
1 spam
2 eggs
3 spam
>>> fun(None)
Nothing.
>>> fun(1.23)
0.615
```

Where there is no registered implementation for a specific type, its method resolution order is used to find a more generic implementation. The original function decorated with `@singledispatch` is registered for the base *object* type, which means it is used if no better implementation is found.

If an implementation is registered to an *abstract base class*, virtual subclasses of the base class will be dispatched to that implementation:

```
>>> from collections.abc import Mapping
>>> @fun.register
... def _(arg: Mapping, verbose=False):
...     if verbose:
...         print("Keys & Values")
...     for key, value in arg.items():
...         print(key, "=>", value)
...
>>> fun({"a": "b"})
a => b
```

To check which implementation the generic function will choose for a given type, use the `dispatch()` attribute:

```
>>> fun.dispatch(float)
<function fun_num at 0x1035a2840>
>>> fun.dispatch(dict)    # note: default implementation
<function fun at 0x103fe0000>
```

To access all registered implementations, use the read-only `registry` attribute:

```
>>> fun.registry.keys()
dict_keys([<class 'NoneType'>, <class 'int'>, <class 'object'>,
           <class 'decimal.Decimal'>, <class 'list'>,
           <class 'float'>])
>>> fun.registry[float]
<function fun_num at 0x1035a2840>
>>> fun.registry[object]
<function fun at 0x103fe0000>
```

Adicionado na versão 3.4.

Alterado na versão 3.7: The *`register()`* attribute now supports using type annotations.

Alterado na versão 3.11: The *`register()`* attribute now supports *`typing.Union`* as a type annotation.

class functools.**singledispatchmethod**(*func*)

Transform a method into a *single-dispatch generic function*.

To define a generic method, decorate it with the `@singledispatchmethod` decorator. When defining a function using `@singledispatchmethod`, note that the dispatch happens on the type of the first non-*self* or non-*cls* argument:

```
class Negator:
    @singledispatchmethod
    def neg(self, arg):
        raise NotImplementedError("Cannot negate a")

    @neg.register
    def _(self, arg: int):
        return -arg

    @neg.register
    def _(self, arg: bool):
        return not arg
```

`@singledispatchmethod` supports nesting with other decorators such as *`@classmethod`*. Note that to allow for `dispatcher.register`, `singledispatchmethod` must be the *outer most* decorator. Here is the `Negator` class with the `neg` methods bound to the class, rather than an instance of the class:

```
class Negator:
    @singledispatchmethod
    @classmethod
    def neg(cls, arg):
        raise NotImplementedError("Cannot negate a")

    @neg.register
    @classmethod
    def _(cls, arg: int):
        return -arg

    @neg.register
    @classmethod
```

(continua na próxima página)

(continuação da página anterior)

```
    def _(cls, arg: bool):
        return not arg
```

The same pattern can be used for other similar decorators: `@staticmethod`, `@abstractmethod`, and others.

Adicionado na versão 3.8.

`functools.`**`update_wrapper`**(*wrapper*, *wrapped*, *assigned=WRAPPER_ASSIGNMENTS*, *updated=WRAPPER_UPDATES*)

Update a *wrapper* function to look like the *wrapped* function. The optional arguments are tuples to specify which attributes of the original function are assigned directly to the matching attributes on the wrapper function and which attributes of the wrapper function are updated with the corresponding attributes from the original function. The default values for these arguments are the module level constants `WRAPPER_ASSIGNMENTS` (which assigns to the wrapper function's `__module__`, `__name__`, `__qualname__`, `__annotations__`, `__type_params__`, and `__doc__`, the documentation string) and `WRAPPER_UPDATES` (which updates the wrapper function's `__dict__`, i.e. the instance dictionary).

To allow access to the original function for introspection and other purposes (e.g. bypassing a caching decorator such as `lru_cache()`), this function automatically adds a `__wrapped__` attribute to the wrapper that refers to the function being wrapped.

The main intended use for this function is in *decorator* functions which wrap the decorated function and return the wrapper. If the wrapper function is not updated, the metadata of the returned function will reflect the wrapper definition rather than the original function definition, which is typically less than helpful.

`update_wrapper()` may be used with callables other than functions. Any attributes named in *assigned* or *updated* that are missing from the object being wrapped are ignored (i.e. this function will not attempt to set them on the wrapper function). `AttributeError` is still raised if the wrapper function itself is missing any attributes named in *updated*.

Alterado na versão 3.2: The `__wrapped__` attribute is now automatically added. The `__annotations__` attribute is now copied by default. Missing attributes no longer trigger an `AttributeError`.

Alterado na versão 3.4: The `__wrapped__` attribute now always refers to the wrapped function, even if that function defined a `__wrapped__` attribute. (see bpo-17482)

Alterado na versão 3.12: The `__type_params__` attribute is now copied by default.

`@functools.`**`wraps`**(*wrapped*, *assigned=WRAPPER_ASSIGNMENTS*, *updated=WRAPPER_UPDATES*)

This is a convenience function for invoking `update_wrapper()` as a function decorator when defining a wrapper function. It is equivalent to `partial(update_wrapper, wrapped=wrapped, assigned=assigned, updated=updated)`. For example:

```
>>> from functools import wraps
>>> def my_decorator(f):
...     @wraps(f)
...     def wrapper(*args, **kwds):
...         print('Calling decorated function')
...         return f(*args, **kwds)
...     return wrapper
...
>>> @my_decorator
... def example():
...     """Docstring"""
...     print('Called example function')
...
>>> example()
Calling decorated function
Called example function
>>> example.__name__
```

(continua na próxima página)

(continuação da página anterior)

```
'example'
>>> example.__doc__
'Docstring'
```

Without the use of this decorator factory, the name of the example function would have been `'wrapper'`, and the docstring of the original `example()` would have been lost.

10.2.1 Objetos `partial`

partial objects are callable objects created by *partial()*. They have three read-only attributes:

`partial.`**`func`**

A callable object or function. Calls to the *partial* object will be forwarded to *func* with new arguments and keywords.

`partial.`**`args`**

The leftmost positional arguments that will be prepended to the positional arguments provided to a *partial* object call.

`partial.`**`keywords`**

The keyword arguments that will be supplied when the *partial* object is called.

partial objects are like function objects in that they are callable, weak referenceable, and can have attributes. There are some important differences. For instance, the *__name__* and *__doc__* attributes are not created automatically.

10.3 `operator` — Operadores padrões como funções

Código-fonte: Lib/operator.py

O módulo *operator* exporta um conjunto de funções eficientes correspondentes aos operadores intrínsecos do Python. Por exemplo, `operator.add(x,y)` é equivalente à expressão `x+y`. Muitos nomes de função são aqueles usados para métodos especiais, sem os sublinhados duplos. Para compatibilidade com versões anteriores, muitos deles têm uma variante com os sublinhados duplos mantidos. As variantes sem os sublinhados duplos são preferenciais para maior clareza.

As funções se enquadram em categorias que realizam comparações de objetos, operações lógicas, operações matemáticas e operações de sequência.

As funções de comparação de objetos são úteis para todos os objetos e são nomeadas conforme os operadores de comparação que os mesmos suportam:

`operator.`**`lt`**(*a*, *b*)

`operator.`**`le`**(*a*, *b*)

`operator.`**`eq`**(*a*, *b*)

`operator.`**`ne`**(*a*, *b*)

`operator.`**`ge`**(*a*, *b*)

`operator.`**`gt`**(*a*, *b*)

`operator.`**`__lt__`**(*a*, *b*)

`operator.`**`__le__`**(*a*, *b*)

`operator.`**`__eq__`**(*a*, *b*)

`operator.`**`__ne__`**(*a*, *b*)

`operator.`**`__ge__`**(*a*, *b*)

operator.**__gt__**(*a*, *b*)

Executam "comparações ricas" entre *a* e *b*. Especialmente, `lt(a, b)` é equivalente a `a < b`, `le(a, b)` é equivalente a `a <= b`, `eq(a, b)` é equivalente a `a == b`, `ne(a, b)` é equivalente a `a != b`, `gt(a, b)` é equivalente a `a > b` e `ge(a, b)` é equivalente a `a >= b`. Observe que essas funções podem retornar qualquer valor, que pode ou não ser interpretável como um valor booleano. Consulte comparisons para obter mais informações sobre comparações ricas.

As operações lógicas também são geralmente aplicáveis a todos os objetos e tem suporte para testes de verdade, testes de identidade e operações booleanas:

operator.**not_**(*obj*)

operator.**__not__**(*obj*)

Retorna o resultado de `not` *obj*. (Veja que não existe nenhum método `__not__()` para instâncias de objetos; apenas o núcleo do interpretador definirá esta operação. O resultado será afetado pelos métodos `__bool__()` e `__len__()`.)

operator.**truth**(*obj*)

Retorna *True* se o *obj* for verdadeiro, e *False* caso contrário. Isso é equivalente a utilizar a construção *bool*.

operator.**is_**(*a*, *b*)

Retorna `a is b`. Testa a identidade do objeto.

operator.**is_not**(*a*, *b*)

Retorna `a is not b`. Testa a identidade do objeto.

operator.**is_none**(*a*)

Retorna `a is None`. Testa a identidade do objeto.

Adicionado na versão 3.14.

operator.**is_not_none**(*a*)

Retorna `a is not None`. Testa a identidade do objeto.

Adicionado na versão 3.14.

As operações matemáticas bit a bit são as mais numerosas:

operator.**abs**(*obj*)

operator.**__abs__**(*obj*)

Retorna o valor absoluto de *obj*.

operator.**add**(*a*, *b*)

operator.**__add__**(*a*, *b*)

Retorna `a + b`, onde *a* e *b* são números.

operator.**and_**(*a*, *b*)

operator.**__and__**(*a*, *b*)

Retorna bit a bit de *a* e *b*.

operator.**floordiv**(*a*, *b*)

operator.**__floordiv__**(*a*, *b*)

Retorna `a // b`.

operator.**index**(*a*)

operator.**__index__**(*a*)

Retorna *a* convertendo para um inteiro. Equivalente a `a.__index__()`.

Alterado na versão 3.10: O resultado sempre tem o tipo exato *int*. Anteriormente, o resultado poderia ter sido uma instância de uma subclasse de `int`.

operator.**inv**(*obj*)

operator.**invert**(*obj*)

operator.**__inv__**(*obj*)

operator.**__invert__**(*obj*)

Retorna o inverso bit a bit do número *obj*. Isso equivale a `~obj`.

operator.**lshift**(*a*, *b*)

operator.**__lshift__**(*a*, *b*)

Retorna *a* deslocado para a esquerda por *b*.

operator.**mod**(*a*, *b*)

operator.**__mod__**(*a*, *b*)

Retorna `a % b`.

operator.**mul**(*a*, *b*)

operator.**__mul__**(*a*, *b*)

Retorna `a * b`, onde *a* e *b* são números.

operator.**matmul**(*a*, *b*)

operator.**__matmul__**(*a*, *b*)

Retorna `a @ b`.

Adicionado na versão 3.5.

operator.**neg**(*obj*)

operator.**__neg__**(*obj*)

Retorna *obj* negado (`-obj`).

operator.**or_**(*a*, *b*)

operator.**__or__**(*a*, *b*)

Retorna bit a bit de *a* e *b*.

operator.**pos**(*obj*)

operator.**__pos__**(*obj*)

Retorna *obj* positivo (`+obj`).

operator.**pow**(*a*, *b*)

operator.**__pow__**(*a*, *b*)

Retorna `a ** b`, onde *a* e *b* são números.

operator.**rshift**(*a*, *b*)

operator.**__rshift__**(*a*, *b*)

Retorna *a* deslocado para a direita por *b*.

operator.**sub**(*a*, *b*)

operator.**__sub__**(*a*, *b*)

Retorna `a - b`.

operator.**truediv**(*a*, *b*)

operator.**__truediv__**(*a*, *b*)

Retorna `a / b` onde 2/3 é .66 em vez de 0. Isso também é conhecido como divisão “verdadeira”.

operator.**xor**(*a*, *b*)

operator.**__xor__**(*a*, *b*)

Retorna o OU exclusivo bit a bit de *a* e *b*.

Operações que funcionam com sequências (algumas delas com mapas também) incluem:

operator.**concat**(*a*, *b*)

operator.**__concat__**(*a*, *b*)

Retorna `a + b` para as sequências *a* e *b*.

operator.**contains**(*a*, *b*)

operator.**__contains__**(*a*, *b*)

Retorna o resultado do teste `b in a`. Observe os operandos invertidos.

operator.**countOf**(*a*, *b*)

Retorna o número de ocorrências de *b* em *a*.

operator.**delitem**(*a*, *b*)

operator.**__delitem__**(*a*, *b*)

Remove de *a* o valor no índice *b*.

operator.**getitem**(*a*, *b*)

operator.**__getitem__**(*a*, *b*)

Retorna de *a* o valor no índice *b*.

operator.**indexOf**(*a*, *b*)

Retorna o índice da primeira ocorrência de *b* em *a*.

operator.**setitem**(*a*, *b*, *c*)

operator.**__setitem__**(*a*, *b*, *c*)

Define em *a* o valor no índice *b* para *c*.

operator.**length_hint**(*obj*, *default=0*)

Retorna um comprimento estimado para o objeto *obj*. Primeiro tenta retornar o seu comprimento real, em seguida, uma estimativa utilizando `object.__length_hint__()`, e finalmente retorna o valor padrão.

Adicionado na versão 3.4.

Os operadores seguintes funcionam com chamáveis:

operator.**call**(*obj*, */*, **args*, ***kwargs*)

operator.**__call__**(*obj*, */*, **args*, ***kwargs*)

Retorna `obj(*args, **kwargs)`.

Adicionado na versão 3.11.

O módulo *operator* também define ferramentas para procura de itens e atributos generalizados. Estes são úteis para fazer extração de campo rapidamente como argumentos para as funções *map()*, *sorted()*, *itertools.groupby()*, ou outra função que espera uma função como argumento.

operator.**attrgetter**(*attr*)

operator.**attrgetter**(**attrs*)

Retorna um objeto chamável que pode buscar o *attr* do seu operando. Caso seja solicitado mais de um atributo, retorna uma tupla de atributos. Os nomes dos atributos também podem conter pontos. Por exemplo:

- Depois de `f = attrgetter('name')`, a chamada a `f(b)` retorna `b.name`.
- Depois de `f = attrgetter('name', 'date')`, a chamada a `f(b)` retorna `(b.name, b.date)`.
- Depois de `f = attrgetter('name.first', 'name.last')`, a chamada a `f(b)` retorna `(b.name.first, b.name.last)`.

Equivalente a:

```
def attrgetter(*items):
    if any(not isinstance(item, str) for item in items):
        raise TypeError('attribute name must be a string')
    if len(items) == 1:
        attr = items[0]
```

(continua na próxima página)

(continuação da página anterior)

```
        def g(obj):
            return resolve_attr(obj, attr)
    else:
        def g(obj):
            return tuple(resolve_attr(obj, attr) for attr in items)
    return g

def resolve_attr(obj, attr):
    for name in attr.split("."):
        obj = getattr(obj, name)
    return obj
```

operator.**itemgetter**(*item*)

operator.**itemgetter**(**items*)

Retorna um objeto chamável que busca *item* de seu operando usando o método `__getitem__()` do operando. Se vários itens forem especificados, retorna um tupla de valores de pesquisa. Por exemplo:

- Depois de `f = itemgetter(2)`, a chamada a `f(r)` retorna `r[2]`.
- Depois de `g = itemgetter(2, 5, 3)`, a chamada a `g(r)` retorna `(r[2], r[5], r[3])`.

Equivalente a:

```
def itemgetter(*items):
    if len(items) == 1:
        item = items[0]
        def g(obj):
            return obj[item]
    else:
        def g(obj):
            return tuple(obj[item] for item in items)
    return g
```

Os itens podem ser de qualquer tipo aceito pelo método `__getitem__()` do operando. Dicionários aceitam qualquer valor *hasheável* . Listas, tuplas e strings aceitam um índice ou um fatiamento:

```
>>> itemgetter(1)('ABCDEFG')
'B'
>>> itemgetter(1, 3, 5)('ABCDEFG')
('B', 'D', 'F')
>>> itemgetter(slice(2, None))('ABCDEFG')
'CDEFG'
>>> soldier = dict(rank='captain', name='dotterbart')
>>> itemgetter('rank')(soldier)
'captain'
```

Exemplo de uso *`itemgetter()`* para recuperar campos específicos de um registro de tupla:

```
>>> inventory = [('apple', 3), ('banana', 2), ('pear', 5), ('orange', 1)]
>>> getcount = itemgetter(1)
>>> list(map(getcount, inventory))
[3, 2, 5, 1]
>>> sorted(inventory, key=getcount)
[('orange', 1), ('banana', 2), ('apple', 3), ('pear', 5)]
```

operator.**methodcaller**(*name*, */*, **args*, ***kwargs*)

Retorna um objeto chamável que invoca o método *name* em seu operando. Se argumentos adicionais e/ou argumentos nomeados forem fornecidos, os mesmos serão passados para o método. Por exemplo:

- Depois de `f = methodcaller('name')`, a chamada a `f(b)` retorna `b.name()`.
- Depois de `f = methodcaller('name', 'foo', bar=1)`, a chamada `f(b)` retorna `b.name('foo', bar=1)`.

Equivalente a:

```
def methodcaller(name, /, *args, **kwargs):
    def caller(obj):
        return getattr(obj, name)(*args, **kwargs)
    return caller
```

10.3.1 Mapeando os operadores para suas respectivas funções

Esta tabela mostra como as operações abstratas correspondem aos símbolos do operador na sintaxe Python e às funções no módulo *operator*.

Operação	Sintaxe	Função
Adição	`a + b`	`add(a, b)`
Concatenação	`seq1 + seq2`	`concat(seq1, seq2)`
Teste de pertinência	`obj in seq`	`contains(seq, obj)`
Divisão	`a / b`	`truediv(a, b)`
Divisão	`a // b`	`floordiv(a, b)`
E bit a bit	`a & b`	`and_(a, b)`
Ou exclusivo bit a bit	`a ^ b`	`xor(a, b)`
Inversão bit a bit	`~ a`	`invert(a)`
Ou bit a bit	`a | b`	`or_(a, b)`
Exponenciação	`a ** b`	`pow(a, b)`
Identidade	`a is b`	`is_(a, b)`
Identidade	`a is not b`	`is_not(a, b)`
Identidade	`a is None`	`is_none(a)`
Identidade	`a is not None`	`is_not_none(a)`
Atribuição Indexada	`obj[k] = v`	`setitem(obj, k, v)`
Eliminação indexada	`del obj[k]`	`delitem(obj, k)`
Indexação	`obj[k]`	`getitem(obj, k)`
Deslocamento à esquerda	`a << b`	`lshift(a, b)`
Módulo	`a % b`	`mod(a, b)`
Multiplicação	`a * b`	`mul(a, b)`
Multiplicação de matrizes	`a @ b`	`matmul(a, b)`
Negação (aritmética)	`- a`	`neg(a)`
Negação (lógica)	`not a`	`not_(a)`
Positivo	`+ a`	`pos(a)`
Deslocamento à direita	`a >> b`	`rshift(a, b)`
Atribuição de fatia	`seq[i:j] = values`	`setitem(seq, slice(i, j), values)`
Remoção de fatia	`del seq[i:j]`	`delitem(seq, slice(i, j))`
Fatiamento	`seq[i:j]`	`getitem(seq, slice(i, j))`
Formatação de strings	`s % obj`	`mod(s, obj)`
Subtração	`a - b`	`sub(a, b)`
Teste verdadeiro	`obj`	`truth(obj)`
Ordenação	`a < b`	`lt(a, b)`
Ordenação	`a <= b`	`le(a, b)`
Igualdade	`a == b`	`eq(a, b)`
Diferença	`a != b`	`ne(a, b)`
Ordenação	`a >= b`	`ge(a, b)`
Ordenação	`a > b`	`gt(a, b)`

10.3.2 Operadores in-place

Muitas operações possuem uma versão "in-place". Listadas abaixo, as funções fornecem um acesso mais primitivo aos operadores locais do que a sintaxe usual; por exemplo, a *instrução* `x += y` é equivalente a `x = operator.iadd(x, y)`. Outra maneira de colocá-lo é dizendo que `z = operator.iadd(x, y)` é equivalente à instrução composta `z = x; z += y`.

Nesses exemplos, note que, quando um método in-place é invocado, a computação e a atribuição são realizadas em duas etapas separadas. As funções in-place listadas abaixo apenas fazem o primeiro passo, invocando o método in-place. O segundo passo, a atribuição, não é tratado.

Para os casos imutáveis, como as strings, números e tuplas, o valor atualizado será calculado, mas não será atribuído de volta à variável de entrada:

```
>>> a = 'hello'
>>> iadd(a, ' world')
'hello world'
>>> a
'hello'
```

Para alvos mutáveis tal como listas e dicionários, o método in-place vai realizar a atualização, então nenhuma atribuição subsequente é necessária:

```
>>> s = ['h', 'e', 'l', 'l', 'o']
>>> iadd(s, [' ', 'w', 'o', 'r', 'l', 'd'])
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
>>> s
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
```

operator.**iadd**(*a*, *b*)

operator.**__iadd__**(*a*, *b*)

`a = iadd(a, b)` é equivalente a `a += b`.

operator.**iand**(*a*, *b*)

operator.**__iand__**(*a*, *b*)

`a = iand(a, b)` é equivalente a `a &= b`.

operator.**iconcat**(*a*, *b*)

operator.**__iconcat__**(*a*, *b*)

`a = iconcat(a, b)` é equivalente a `a += b` onde *a* e *b* são sequências.

operator.**ifloordiv**(*a*, *b*)

operator.**__ifloordiv__**(*a*, *b*)

`a = ifloordiv(a, b)` é equivalente a `a //= b`.

operator.**ilshift**(*a*, *b*)

operator.**__ilshift__**(*a*, *b*)

`a = ilshift(a, b)` é equivalente a `a <<= b`.

operator.**imod**(*a*, *b*)

operator.**__imod__**(*a*, *b*)

`a = imod(a, b)` é equivalente a `a %= b`.

operator.**imul**(*a*, *b*)

operator.**__imul__**(*a*, *b*)

`a = imul(a, b)` é equivalente a `a *= b`.

operator.**imatmul**(*a*, *b*)

operator.**__imatmul__**(*a*, *b*)

a = imatmul(a, b) é equivalente a a @= b.

Adicionado na versão 3.5.

operator.**ior**(*a*, *b*)

operator.**__ior__**(*a*, *b*)

a = ior(a, b) é equivalente a a |= b.

operator.**ipow**(*a*, *b*)

operator.**__ipow__**(*a*, *b*)

a = ipow(a, b) é equivalente a a **= b.

operator.**irshift**(*a*, *b*)

operator.**__irshift__**(*a*, *b*)

a = irshift(a, b) é equivalente a a >>= b.

operator.**isub**(*a*, *b*)

operator.**__isub__**(*a*, *b*)

a = isub(a, b) é equivalente a a -= b.

operator.**itruediv**(*a*, *b*)

operator.**__itruediv__**(*a*, *b*)

a = itruediv(a, b) é equivalente a a /= b.

operator.**ixor**(*a*, *b*)

operator.**__ixor__**(*a*, *b*)

a = ixor(a, b) é equivalente a a ^= b.

CAPÍTULO 11

Acesso a arquivos e diretórios

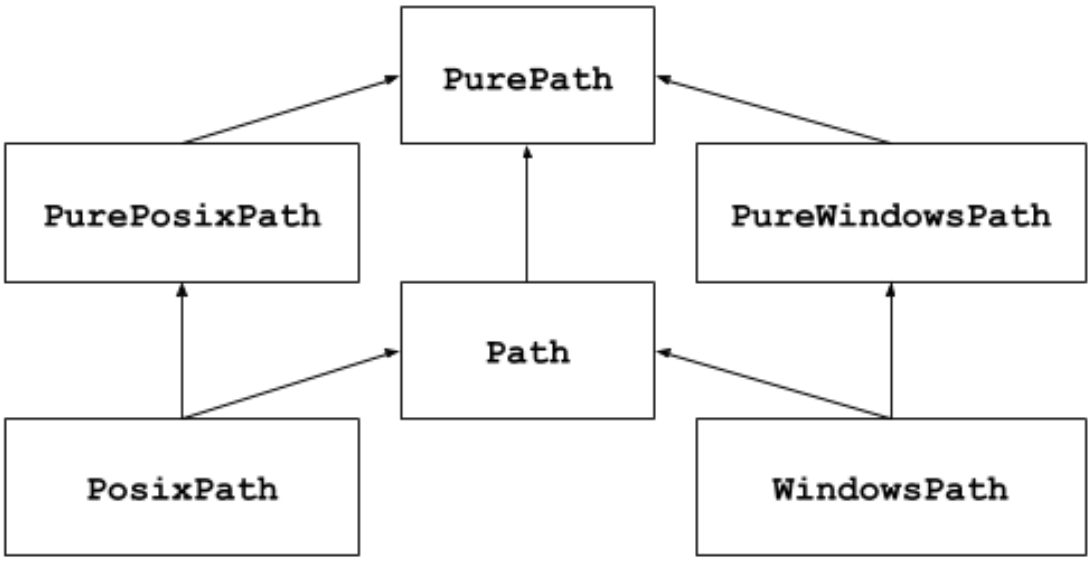
Os módulos descritos neste capítulo dizem respeito aos arquivos e diretórios no disco. Por exemplo, existem módulos para ler as propriedades dos arquivos, manipular o caminhos de forma multiplataforma e para criar arquivos temporários. A lista completa de módulos neste capítulo é:

11.1 `pathlib` — Caminhos do sistema de arquivos orientados a objetos

Adicionado na versão 3.4.

Código-fonte: Lib/pathlib/

Este módulo oferece classes que representam caminhos de sistema de arquivos com semântica apropriada para diferentes sistemas operacionais. As classes de caminho são divididas entre *caminhos puros*, que fornecem operações puramente computacionais sem E/S, e *caminhos concretos*, que herdam de caminhos puros, mas também fornecem operações de E/S.



Se você nunca usou este módulo antes ou apenas não tem certeza de qual classe é a certa para sua tarefa, provavelmente `Path` é o que você precisa. Esta classe instancia um *caminho concreto* para a plataforma em que o código está sendo executado.

Caminhos puros são úteis em alguns casos especiais. Por exemplo:

1. Se você deseja manipular os caminhos do Windows em uma máquina Unix (ou vice-versa). Você não pode instanciar `WindowsPath` quando está no Unix, mas você pode instanciar `PureWindowsPath`.
2. Você quer ter certeza de que seu código apenas manipula caminhos, sem realmente acessar o sistema operacional. Nesse caso, instanciar uma das classes puras pode ser útil, pois elas simplesmente não têm nenhuma operação de acesso ao sistema operacional.

Ver também

PEP 428: O módulo pathlib – caminhos de sistema de arquivos orientados a objetos.

Ver também

Para manipulação de caminho de baixo nível em strings, você também pode usar o módulo `os.path`.

11.1.1 Uso básico

Importando a classe principal:

```
>>> from pathlib import Path
```

Listando os subdiretórios:

```
>>> p = Path('.')
>>> [x for x in p.iterdir() if x.is_dir()]
[PosixPath('.hg'), PosixPath('docs'), PosixPath('dist'),
 PosixPath('__pycache__'), PosixPath('build')]
```

Listando os arquivos fontes do Python e sua árvore de diretórios:

```
>>> list(p.glob('**/*.py'))
[PosixPath('test_pathlib.py'), PosixPath('setup.py'),
 PosixPath('pathlib.py'), PosixPath('docs/conf.py'),
 PosixPath('build/lib/pathlib.py')]
```

Navegando dentro da árvore de diretórios:

```
>>> p = Path('/etc')
>>> q = p / 'init.d' / 'reboot'
>>> q
PosixPath('/etc/init.d/reboot')
>>> q.resolve()
PosixPath('/etc/rc.d/init.d/halt')
```

Consultando as propriedades do path:

```
>>> q.exists()
True
>>> q.is_dir()
False
```

Abrindo um arquivo:

```
>>> with q.open() as f: f.readline()
...
'#!/bin/bash\n'
```

11.1.2 Exceções

exception pathlib.**UnsupportedOperation**

Uma exceção herdada de *NotImplementedError* que é levantada quando uma operação não suportada é chamada em um objeto caminho.

Adicionado na versão 3.13.

11.1.3 Caminhos puros

Objetos de caminho puro fornecem operações de manipulação de caminho que, na verdade, não acessam um sistema de arquivos. Existem três maneiras de acessar essas classes, que também chamamos de *sabores*:

class pathlib.**PurePath**(**pathsegments*)

Uma classe genérica que representa o tipo de caminho do sistema (ao instanciá-la, cria-se uma *PurePosixPath* ou uma *PureWindowsPath*):

```
>>> PurePath('setup.py')      # Executando em uma máquina Unix
PurePosixPath('setup.py')
```

Cada elemento de *pathsegments* pode ser uma string representando um segmento de caminho, um objeto que implementa a interface que retorna uma string ou um objeto implementando a interface *os.PathLike*, onde o método *__fspath__()* retorna uma string, como outro objeto caminho:

```
>>> PurePath('foo', 'algum/caminho', 'bar')
PurePosixPath('foo/algum/caminho/bar')
>>> PurePath(Path('foo'), Path('bar'))
PurePosixPath('foo/bar')
```

Quando *pathsegments* está vazio, o diretório atual é presumido:

```
>>> PurePath()
PurePosixPath('.')
```

Se um segmento for um caminho absoluto, todos os segmentos anteriores serão ignorados (como *os.path.join()*):

```
>>> PurePath('/etc', '/usr', 'lib64')
PurePosixPath('/usr/lib64')
>>> PureWindowsPath('c:/Windows', 'd:bar')
PureWindowsPath('d:bar')
```

No Windows, a unidade não é redefinida quando um segmento de caminho relativo enraizado (por exemplo, `r'\foo'`) é encontrado:

```
>>> PureWindowsPath('c:/Windows', '/Program Files')
PureWindowsPath('c:/Program Files')
```

Barras espúrias e pontos simples são recolhidos, mas pontos duplos (`'..'`) e barras duplas iniciais (`'//'`) não são, pois isso mudaria o significado de um caminho por vários motivos (por exemplo, links simbólicos, caminhos UNC):

```
>>> PurePath('foo//bar')
PurePosixPath('foo/bar')
>>> PurePath('//foo/bar')
PurePosixPath('//foo/bar')
>>> PurePath('foo/./bar')
PurePosixPath('foo/bar')
>>> PurePath('foo/../bar')
PurePosixPath('foo/../bar')
```

(uma abordagem ingênua seria criar `PurePosixPath('foo/../bar')` equivalente a `PurePosixPath('bar')`, o que é errado se `foo` for um link simbólico para outro diretório)

Objetos caminho puro implementam a interface *os.PathLike*, permitindo que sejam usados em qualquer lugar em que a interface seja aceita.

Alterado na versão 3.6: Adicionado suporte para a interface *os.PathLike*.

class pathlib.**PurePosixPath**(**pathsegments*)

Uma subclasse de *PurePath*, este tipo de caminho representa caminhos de sistema de arquivos não Windows:

```
>>> PurePosixPath('/etc/hosts')
PurePosixPath('/etc/hosts')
```

pathsegments é especificado de forma similar para *PurePath*.

class pathlib.**PureWindowsPath**(**pathsegments*)

Uma subclasse de *PurePath*, este tipo de caminho representa os caminhos do sistema de arquivos do Windows, incluindo Caminhos UNC:

```
>>> PureWindowsPath('c:/', 'Users', 'Ximénez')
PureWindowsPath('c:/Users/Ximénez')
>>> PureWindowsPath('//servidor/compartilhamento/arquivo')
PureWindowsPath('//servidor/compartilhamento/arquivo')
```

pathsegments é especificado de forma similar para *PurePath*.

Independentemente do sistema em que você está usando, você pode instanciar todas essas classes, uma vez que elas não fornecem nenhuma operação que faça chamadas de sistema.

Propriedades gerais

Os caminhos são imutáveis e *hasheáveis*. Os caminhos do mesmo sabor são comparáveis e ordenáveis. Essas propriedades respeitam a semântica de caixa alta e baixa do sabor:

```
>>> PurePosixPath('foo') == PurePosixPath('FOO')
False
>>> PureWindowsPath('foo') == PureWindowsPath('FOO')
True
>>> PureWindowsPath('FOO') in { PureWindowsPath('foo') }
True
>>> PureWindowsPath('C:') < PureWindowsPath('d:')
True
```

Caminhos de um sabor diferente são comparados de forma desigual e não podem ser ordenados:

```
>>> PureWindowsPath('foo') == PurePosixPath('foo')
False
>>> PureWindowsPath('foo') < PurePosixPath('foo')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: '<' not supported between instances of 'PureWindowsPath' and
↪'PurePosixPath'
```

Operadores

O operador barra ajuda a criar caminhos filhos, como *os.path.join()*. Se o argumento for um caminho absoluto, o caminho anterior será ignorado. No Windows, a unidade não é redefinida quando o argumento é um caminho relativo enraizado (por exemplo, `r'\foo'`):

```
>>> p = PurePath('/etc')
>>> p
PurePosixPath('/etc')
>>> p / 'init.d' / 'apache2'
PurePosixPath('/etc/init.d/apache2')
>>> q = PurePath('bin')
>>> '/usr' / q
PurePosixPath('/usr/bin')
>>> p / '/um_caminho_absoluto'
PurePosixPath('/um_caminho_absoluto')
>>> PureWindowsPath('c:/Windows', '/Program Files')
PureWindowsPath('c:/Program Files')
```

Um objeto de caminho pode ser usado em qualquer lugar em que um objeto implementando *os.PathLike* seja aceito:

```
>>> import os
>>> p = PurePath('/etc')
>>> os.fspath(p)
'/etc'
```

A representação de string de um caminho é o próprio caminho do sistema de arquivos bruto (na forma nativa, por exemplo, com contrabarras no Windows), que você pode passar para qualquer função usando um caminho de arquivo como uma string:

```
>>> p = PurePath('/etc')
>>> str(p)
'/etc'
>>> p = PureWindowsPath('c:/Program Files')
```

(continua na próxima página)

(continuação da página anterior)

```
>>> str(p)
'c:\\Program Files'
```

Da mesma forma, chamar *bytes* em um caminho fornece o caminho do sistema de arquivos bruto como um objeto bytes, codificado por *os.fsencode()*:

```
>>> bytes(p)
b'/etc'
```

Nota

A chamada de *bytes* só é recomendada no Unix. No Windows, a forma Unicode é a representação canônica dos caminhos do sistema de arquivos.

Acessando partes individuais

Para acessar as "partes" individuais (componentes) de um caminho, use a seguinte propriedade:

`PurePath.`**`parts`**

Uma tupla que dá acesso aos vários componentes do caminho:

```
>>> p = PurePath('/usr/bin/python3')
>>> p.parts
('/', 'usr', 'bin', 'python3')

>>> p = PureWindowsPath('c:/Program Files/PSF')
>>> p.parts
('c:\\', 'Program Files', 'PSF')
```

(observe como a unidade e a raiz local são reagrupadas em uma única parte)

Métodos e propriedades

Caminhos puros fornecem os seguintes métodos e propriedades:

`PurePath.`**`parser`**

A implementação do módulo *os.path* usado para análise e junção de caminhos de baixo nível: `posixpath` ou `ntpath`.

Adicionado na versão 3.13.

`PurePath.`**`drive`**

Uma string que representa a letra ou nome da unidade, se houver:

```
>>> PureWindowsPath('c:/Program Files/').drive
'c:'
>>> PureWindowsPath('/Program Files/').drive
''
>>> PurePosixPath('/etc').drive
''
```

Os compartilhamentos UNC também são considerados unidades:

```
>>> PureWindowsPath('//host/compartilhamento/foo.txt').drive
'\\\\host\\compartilhamento'
```

PurePath.**root**

Uma string que representa a raiz (local ou global), se houver:

```
>>> PureWindowsPath('c:/Program Files/').root
'\\'
>>> PureWindowsPath('c:Program Files/').root
''
>>> PurePosixPath('/etc').root
'/'
```

Os compartilhamentos UNC sempre têm uma raiz:

```
>>> PureWindowsPath('//host/compartilhamento').root
'\\'
```

Se o caminho começa com mais de duas barras sucessivas, *PurePosixPath* as recolhe:

```
>>> PurePosixPath('//etc').root
'//'
>>> PurePosixPath('///etc').root
'/'
>>> PurePosixPath('////etc').root
'/'
```

Nota

Este comportamento está em conformidade com o parágrafo 4.11 Pathname Resolution do *The Open Group Base Specifications Issue 6*:

"Um nome de caminho que começa com duas barras sucessivas pode ser interpretado de maneira definida pela implementação, embora mais de duas barras iniciais devam ser tratadas como uma única barra."

PurePath.**anchor**

A concatenação da unidade e da raiz:

```
>>> PureWindowsPath('c:/Program Files/').anchor
'c:\\'
>>> PureWindowsPath('c:Program Files/').anchor
'c:'
>>> PurePosixPath('/etc').anchor
'/'
>>> PureWindowsPath('//host/compartilhamento').anchor
'\\\\host\\compartilhamento\\'
```

PurePath.**parents**

Uma sequência imutável que fornece acesso aos ancestrais lógicos do caminho:

```
>>> p = PureWindowsPath('c:/foo/bar/setup.py')
>>> p.parents[0]
PureWindowsPath('c:/foo/bar')
>>> p.parents[1]
PureWindowsPath('c:/foo')
>>> p.parents[2]
PureWindowsPath('c:/')
```

Alterado na versão 3.10: A sequência pai agora tem suporte *fatias* e valores de índice negativos.

PurePath.**parent**

O pai lógico do caminho:

```
>>> p = PurePosixPath('/a/b/c/d')
>>> p.parent
PurePosixPath('/a/b/c')
```

Você não pode passar por uma âncora ou caminho vazio:

```
>>> p = PurePosixPath('/')
>>> p.parent
PurePosixPath('/')
>>> p = PurePosixPath('.')
>>> p.parent
PurePosixPath('.')
```

Nota

Esta é uma operação puramente lexical, daí o seguinte comportamento:

```
>>> p = PurePosixPath('foo/..')
>>> p.parent
PurePosixPath('foo')
```

Se você deseja percorrer um caminho arbitrário do sistema de arquivos para cima, é recomendável primeiro chamar *Path.resolve()* para resolver os links simbólicos e eliminar os componentes `".."`.

PurePath.**name**

Uma string que representa o componente do caminho final, excluindo a unidade e a raiz, se houver:

```
>>> PurePosixPath('minha/biblioteca/setup.py').name
'setup.py'
```

Nomes de unidades UNC não são considerados::

```
>>> PureWindowsPath('//algum/compartilhamento/setup.py').name
'setup.py'
>>> PureWindowsPath('//algum/compartilhamento').name
''
```

PurePath.**suffix**

A última parte separada por pontos do componente final, se houver:

```
>>> PurePosixPath('minha/biblioteca/setup.py').suffix
'.py'
>>> PurePosixPath('minha/biblioteca.tar.gz').suffix
'.gz'
>>> PurePosixPath('minha/biblioteca').suffix
''
```

Isso é comumente chamado de extensão de arquivo.

Alterado na versão 3.14: Um único ponto (”`.`”) é considerado um sufixo válido.

PurePath.**suffixes**

Uma lista dos sufixos do caminho, com frequência chamados de extensões de arquivo:

```
>>> PurePosixPath('minha/biblioteca.tar.gar').suffixes
['.tar', '.gar']
>>> PurePosixPath('minha/biblioteca.tar.gz').suffixes
['.tar', '.gz']
>>> PurePosixPath('minha/biblioteca').suffixes
[]
```

Alterado na versão 3.14: Um único ponto (".") é considerado um sufixo válido.

PurePath.**stem**

O componente final do caminho, sem seu sufixo:

```
>>> PurePosixPath('minha/biblioteca.tar.gz').stem
'library.tar'
>>> PurePosixPath('minha/biblioteca.tar').stem
'library'
>>> PurePosixPath('minha/biblioteca').stem
'library'
```

PurePath.**as_posix**()

Retorna uma representação de string do caminho com barras (/):

```
>>> p = PureWindowsPath('c:\\windows')
>>> str(p)
'c:\\windows'
>>> p.as_posix()
'c:/windows'
```

PurePath.**is_absolute**()

Retorna se o caminho é absoluto ou não. Um caminho é considerado absoluto se tiver uma raiz e (se o tipo permitir) uma unidade:

```
>>> PurePosixPath('/a/b').is_absolute()
True
>>> PurePosixPath('a/b').is_absolute()
False

>>> PureWindowsPath('c:/a/b').is_absolute()
True
>>> PureWindowsPath('/a/b').is_absolute()
False
>>> PureWindowsPath('c:').is_absolute()
False
>>> PureWindowsPath('//some/share').is_absolute()
True
```

PurePath.**is_relative_to**(*other*)

Retorna se este caminho é ou não relativo a outro caminho, representado por *other*.

```
>>> p = PurePath('/etc/passwd')
>>> p.is_relative_to('/etc')
True
>>> p.is_relative_to('/usr')
False
```

Este método é baseado em string; ele não acessa o sistema de arquivos nem trata segmentos "..." especialmente. O código a seguir é equivalente:

```
>>> u = PurePath('/usr')
>>> u == p or u in p.parents
False
```

Adicionado na versão 3.9.

Deprecated since version 3.12, removed in version 3.14: A passagem de argumentos adicionais está descontinuada; se fornecidos, eles são unidos com *other*.

`PurePath.`**`is_reserved`**`()`

Com *PureWindowsPath*, retorna `True` se o caminho é considerado reservado no Windows, `False` caso contrário. Com *PurePosixPath*, `False` é sempre retornado.

Alterado na versão 3.13: Nomes de caminho do Windows que contêm dois pontos, ou terminam com um ponto ou um espaço, são considerados reservados. Caminhos UNC podem ser reservados.

Deprecated since version 3.13, will be removed in version 3.15: Este método está descontinuado; use *os.path.isreserved()* para detectar caminhos reservados no Windows.

`PurePath.`**`joinpath`**`(`*pathsegments*`)`

Chamar este método é equivalente a combinar o caminho com cada um dos *pathsegments* por vez:

```
>>> PurePosixPath('/etc').joinpath('passwd')
PurePosixPath('/etc/passwd')
>>> PurePosixPath('/etc').joinpath(PurePosixPath('passwd'))
PurePosixPath('/etc/passwd')
>>> PurePosixPath('/etc').joinpath('init.d', 'apache2')
PurePosixPath('/etc/init.d/apache2')
>>> PureWindowsPath('c:').joinpath('/Program Files')
PureWindowsPath('c:/Program Files')
```

`PurePath.`**`full_match`**`(`*pattern*, *, *case_sensitive=None*`)`

Compara esse caminho com o padrão de estilo glob fornecido. Retorna `True` se a correspondência for bem-sucedida, `False` caso contrário. Por exemplo:

```
>>> PurePath('a/b.py').full_match('a/*.py')
True
>>> PurePath('a/b.py').full_match('*.py')
False
>>> PurePath('/a/b/c.py').full_match('/a/**')
True
>>> PurePath('/a/b/c.py').full_match('**/*.py')
True
```

Ver também

Documentação de *Linguagem de padrões*.

Tal como acontece com outros métodos, a distinção entre maiúsculas e minúsculas segue os padrões da plataforma:

```
>>> PurePosixPath('b.py').full_match('*.PY')
False
>>> PureWindowsPath('b.py').full_match('*.PY')
True
```

Defina *case_sensitive* como `True` ou `False` para substituir esse comportamento.

Adicionado na versão 3.13.

PurePath.**match**(*pattern*, *, *case_sensitive=None*)

Compara esse caminho com o padrão no recursivo de estilo glob fornecido. Retorna `True` se a correspondência for bem-sucedida, `False` caso contrário.

Este método é semelhante a *full_match()*, mas padrões vazios não são permitidos (*ValueError* é levantada), o curinga recursivo "`**`" não é suportado (ele age como o "`*`" não recursivo) e, se um padrão relativo for fornecido, a correspondência será feita da direita:

```
>>> PurePath('a/b.py').match('*.py')
True
>>> PurePath('/a/b/c.py').match('b/*.py')
True
>>> PurePath('/a/b/c.py').match('a/*.py')
False
```

Alterado na versão 3.12: O parâmetro *pattern* aceita um *objeto caminho ou similar*.

Alterado na versão 3.12: O parâmetro *case_sensitive* foi adicionado.

PurePath.**relative_to**(*other*, *walk_up=False*)

Calcula uma versão deste caminho em relação ao caminho representado por *other*. Se for impossível, *ValueError* é levantada:

```
>>> p = PurePosixPath('/etc/passwd')
>>> p.relative_to('/')
PurePosixPath('etc/passwd')
>>> p.relative_to('/etc')
PurePosixPath('passwd')
>>> p.relative_to('/usr')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "pathlib.py", line 941, in relative_to
    raise ValueError(error_message.format(str(self), str(formatted)))
ValueError: '/etc/passwd' is not in the subpath of '/usr' OR one path is␣
↪relative and the other is absolute.
```

Quando *walk_up* é falso (o padrão), o caminho deve começar com *other*. Quando o argumento é verdadeiro, entradas `..` podem ser adicionadas para formar o caminho relativo. Em todos os outros casos, como os caminhos que fazem referência a unidades diferentes, *ValueError* é levantada.:

```
>>> p.relative_to('/usr', walk_up=True)
PurePosixPath('../etc/passwd')
>>> p.relative_to('foo', walk_up=True)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "pathlib.py", line 941, in relative_to
    raise ValueError(error_message.format(str(self), str(formatted)))
ValueError: '/etc/passwd' is not on the same drive as 'foo' OR one path is␣
↪relative and the other is absolute.
```

Aviso

Esta função faz parte de *PurePath* e funciona com strings. Ela não verifica ou acessa a estrutura do arquivo subjacente. Isso pode impactar a opção *walk_up*, pois ela presume que não há links simbólicos presentes no caminho; chame *resolve()* primeiro se necessário para resolver links simbólicos.

Alterado na versão 3.12: O parâmetro *walk_up* foi adicionado (o comportamento antigo é o mesmo de `walk_up=False`).

Deprecated since version 3.12, removed in version 3.14: A passagem de argumentos posicionais adicionais está descontinuada; se fornecidos, eles são unidos com *other*.

PurePath.**with_name**(*name*)

Retorna um novo caminho com o *name* alterado. Se o caminho original não tiver um nome, ValueError é levantada:

```
>>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz')
>>> p.with_name('setup.py')
PureWindowsPath('c:/Downloads/setup.py')
>>> p = PureWindowsPath('c:/')
>>> p.with_name('setup.py')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/antoine/cpython/default/Lib/pathlib.py", line 751, in with_name
    raise ValueError("%r has an empty name" % (self,))
ValueError: PureWindowsPath('c:/') has an empty name
```

PurePath.**with_stem**(*stem*)

Retorna um novo caminho com o *stem* alterado. Se o caminho original não tiver um nome, ValueError é levantada:

```
>>> p = PureWindowsPath('c:/Downloads/draft.txt')
>>> p.with_stem('final')
PureWindowsPath('c:/Downloads/final.txt')
>>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz')
>>> p.with_stem('lib')
PureWindowsPath('c:/Downloads/lib.gz')
>>> p = PureWindowsPath('c:/')
>>> p.with_stem('')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/antoine/cpython/default/Lib/pathlib.py", line 861, in with_stem
    return self.with_name(stem + self.suffix)
  File "/home/antoine/cpython/default/Lib/pathlib.py", line 851, in with_name
    raise ValueError("%r has an empty name" % (self,))
ValueError: PureWindowsPath('c:/') has an empty name
```

Adicionado na versão 3.9.

PurePath.**with_suffix**(*suffix*)

Retorna um novo caminho com o *suffix* alterado. Se o caminho original não tiver um sufixo, o novo *suffixo* será anexado. Se o *suffix* for uma string vazia, o sufixo original será removido:

```
>>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz')
>>> p.with_suffix('.bz2')
PureWindowsPath('c:/Downloads/pathlib.tar.bz2')
>>> p = PureWindowsPath('README')
>>> p.with_suffix('.txt')
PureWindowsPath('README.txt')
>>> p = PureWindowsPath('README.txt')
>>> p.with_suffix('')
PureWindowsPath('README')
```

Alterado na versão 3.14: Um único ponto (".") é considerado um sufixo válido. Em versões anteriores, *ValueError* era levantada se um único ponto fosse fornecido.

PurePath.**with_segments**(**pathsegments*)

Cria um novo objeto caminho do mesmo tipo combinando os *pathsegments* fornecidos. Este método é chamado

sempre que um caminho derivado é criado, como de *parent* e *relative_to()*. Subclasses podem substituir este método para passar informações para caminhos derivados, por exemplo:

```
from pathlib import PurePosixPath

class MyPath(PurePosixPath):
    def __init__(self, *pathsegments, session_id):
        super().__init__(*pathsegments)
        self.session_id = session_id

    def with_segments(self, *pathsegments):
        return type(self)(*pathsegments, session_id=self.session_id)

etc = MyPath('/etc', session_id=42)
hosts = etc / 'hosts'
print(hosts.session_id)  # 42
```

Adicionado na versão 3.12.

11.1.4 Caminhos concretos

Caminhos concretos são subclasses das classes de caminho puro. Além das operações fornecidas por este último, eles também fornecem métodos para fazer chamadas de sistema em objetos de caminho. Existem três maneiras de instanciar caminhos concretos:

class pathlib.**Path**(**pathsegments*)

Uma subclasse de *PurePath*, esta classe representa caminhos concretos do tipo de caminho do sistema (instanciando-o cria uma *PosixPath* ou uma *WindowsPath*):

```
>>> Path('setup.py')
PosixPath('setup.py')
```

pathsegments é especificado de forma similar para *PurePath*.

class pathlib.**PosixPath**(**pathsegments*)

Uma subclasse de *Path* e *PurePosixPath*, esta classe representa caminhos concretos de sistemas de arquivos não-Windows:

```
>>> PosixPath('/etc/hosts')
PosixPath('/etc/hosts')
```

pathsegments é especificado de forma similar para *PurePath*.

Alterado na versão 3.13: Levanta *UnsupportedOperation* no Windows. Em versões anteriores, *NotImplementedError* era levantada em vez disso.

class pathlib.**WindowsPath**(**pathsegments*)

Uma subclasse de *Path* e *PureWindowsPath*, esta classe representa caminhos concretos de sistemas de arquivos do Windows:

```
>>> WindowsPath('c:/', 'Users', 'Ximénez')
WindowsPath('c:/Users/Ximénez')
```

pathsegments é especificado de forma similar para *PurePath*.

Alterado na versão 3.13: Levanta *UnsupportedOperation* em plataformas não-Windows. Em versões anteriores, *NotImplementedError* era levantada em vez disso.

Você só pode instanciar o tipo de classe que corresponde ao seu sistema (permitir chamadas de sistema em tipos de caminho não compatíveis pode levar a bugs ou falhas em sua aplicação):

```
>>> import os
>>> os.name
'posix'
>>> Path('setup.py')
PosixPath('setup.py')
>>> PosixPath('setup.py')
PosixPath('setup.py')
>>> WindowsPath('setup.py')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "pathlib.py", line 798, in __new__
    % (cls.__name__,))
UnsupportedOperation: cannot instantiate 'WindowsPath' on your system
```

Alguns métodos de caminho concretos podem levantar um *OSError* se uma chamada de sistema falhar (por exemplo, porque o caminho não existe).

Analisando e gerando URIs

Objetos caminho concretos podem ser criados e representados como URIs de 'file' em conformidade com **RFC 8089**.

Nota

URIs de arquivos não são portáteis entre máquinas com diferentes *codificações de sistema de arquivos*.

classmethod Path.**from_uri**(*uri*)

Retorna um novo objeto de caminho a partir da análise sintática de um URI 'file'. Por exemplo:

```
>>> p = Path.from_uri('file:///etc/hosts')
PosixPath('/etc/hosts')
```

No Windows, os caminhos do dispositivo DOS e UNC podem ser analisados a partir de URIs:

```
>>> p = Path.from_uri('file:///c:/windows')
WindowsPath('c:/windows')
>>> p = Path.from_uri('file://servidor/compartilhamento')
WindowsPath('//servidor/compartilhamento')
```

Várias formas variantes são suportadas:

```
>>> p = Path.from_uri('file:////servidor/compartilhamento')
WindowsPath('//servidor/compartilhamento')
>>> p = Path.from_uri('file://///servidor/compartilhamento')
WindowsPath('//servidor/compartilhamento')
>>> p = Path.from_uri('file:c:/windows')
WindowsPath('c:/windows')
>>> p = Path.from_uri('file:/c|/windows')
WindowsPath('c:/windows')
```

ValueError é levantada se o URI não começar com `file:` ou se o caminho analisado não for absoluto.

Adicionado na versão 3.13.

Alterado na versão 3.14: A autoridade de URL é descartada se corresponder ao nome do host local. Caso contrário, se a autoridade não estiver vazia ou for `localhost`, no Windows um caminho UNC será retornado (como antes) e, em outras plataformas, um *ValueError* será levantada.

Path.**as_uri**()

Representa o caminho como um URI 'file'. *ValueError* é levantada se o caminho não for absoluto.

```
>>> p = PosixPath('/etc/passwd')
>>> p.as_uri()
'file:///etc/passwd'
>>> p = WindowsPath('c:/Windows')
>>> p.as_uri()
'file:///c:/Windows'
```

Deprecated since version 3.14, will be removed in version 3.19: Chamar este método de *PurePath* em vez de *Path* é possível, mas está descontinuado. O uso de *os.fsencode()* pelo método o torna estritamente impuro.

Expandindo e resolvendo caminhos

classmethod Path.**home**()

Retorna um novo objeto caminho representando o diretório pessoal do usuário (conforme retornado por *os.path.expanduser()* com a construção ~). Se o diretório pessoal não puder ser resolvido, *RuntimeError* é levantada.

```
>>> Path.home()
PosixPath('/home/antoine')
```

Adicionado na versão 3.5.

Path.**expanduser**()

Retorna um novo caminho com as construções `~` e `~user` expandidas, como retornado por *os.path.expanduser()*. Se um diretório pessoal não puder ser resolvido, *RuntimeError* é levantada.

```
>>> p = PosixPath('~/filmes/Monty Python')
>>> p.expanduser()
PosixPath('/home/eric/filmes/Monty Python')
```

Adicionado na versão 3.5.

classmethod Path.**cwd**()

Retorna um novo objeto caminho que representa o diretório atual (conforme retornado por *os.getcwd()*):

```
>>> Path.cwd()
PosixPath('/home/antoine/pathlib')
```

Path.**absolute**()

Torna o caminho absoluto, sem normalização ou resolução de links simbólicos. Retorna um novo objeto de caminho:

```
>>> p = Path('tests')
>>> p
PosixPath('tests')
>>> p.absolute()
PosixPath('/home/antoine/pathlib/tests')
```

Path.**resolve**(*strict=False*)

Torna o caminho absoluto, resolvendo quaisquer links simbólicos. Um novo objeto de caminho é retornado:

```
>>> p = Path()
>>> p
PosixPath('.')
```

(continua na próxima página)

(continuação da página anterior)

```
>>> p.resolve()
PosixPath('/home/antoine/pathlib')
```

Componentes "." também são eliminados (este é o único método para fazer isso):

```
>>> p = Path('docs/../setup.py')
>>> p.resolve()
PosixPath('/home/antoine/pathlib/setup.py')
```

Se um caminho não existe ou um laço de link simbólico é encontrado, e *strict* é `True`, *OSError* é levantada. Se *strict* for `False`, o caminho será resolvido tanto quanto possível e qualquer resto é anexado sem verificar se existe.

Alterado na versão 3.6: O parâmetro *strict* foi adicionado (comportamento pré-3.6 é estrito).

Alterado na versão 3.13: Os laços em link simbólico são tratados como outros erros: *OSError* é levantada no modo estrito, e nenhuma exceção é levantada no modo não estrito. Em versões anteriores, *RuntimeError* é levantada não importando o valor de *strict*.

`Path.`**`readlink`**`()`

Retorna o caminho para o qual o link simbólico aponta (conforme retornado por *os.readlink()*):

```
>>> p = Path('meulink')
>>> p.symlink_to('setup.py')
>>> p.readlink()
PosixPath('setup.py')
```

Adicionado na versão 3.9.

Alterado na versão 3.13: Levanta *UnsupportedOperation* se *os.readlink()* não estiver disponível. Em versões anteriores, *NotImplementedError* era levantada.

Consultando tipo de arquivo e status

Alterado na versão 3.8: *exists()*, *is_dir()*, *is_file()*, *is_mount()*, *is_symlink()*, *is_block_device()*, *is_char_device()*, *is_fifo()*, *is_socket()* agora retornam `False` em vez de levantar uma exceção para caminhos que contêm caracteres não representáveis no nível do sistema operacional.

Alterado na versão 3.14: Os métodos fornecidos acima agora retornam `False` em vez de levantar qualquer exceção *OSError* do sistema operacional. Em versões anteriores, alguns tipos de exceção *OSError* eram levantados e outros suprimidos. O novo comportamento é consistente com *os.path.exists()*, *os.path.isdir()*, etc. Use *stat()* para recuperar o status do arquivo sem suprimir exceções.

`Path.`**`stat`**`(*, `*follow_symlinks=True*`)`

Retorna um objeto *os.stat_result* contendo informações sobre este caminho, como *os.stat()*. O resultado é consultado em cada chamada para este método.

Este método normalmente segue links simbólicos; para obter estado de um link simbólico, adicione o argumento `follow_symlinks=False`, ou use *lstat()*.

```
>>> p = Path('setup.py')
>>> p.stat().st_size
956
>>> p.stat().st_mtime
1327883547.852554
```

Alterado na versão 3.10: O parâmetro *follow_symlinks* foi adicionado.

`Path.`**`lstat`**`()`

Como *Path.stat()*, mas, se o caminho apontar para um link simbólico, retorna as informações do link simbólico ao invés de seu alvo.

Path.**exists**(*, *follow_symlinks=True*)

Retorna `True` se o caminho apontar para um arquivo ou diretório existente. `False` será retornado se o caminho for inválido, inacessível ou ausente. Use *`Path.stat()`* para distinguir entre esses casos.

Este método normalmente segue links simbólicos; para verificar se um link simbólico existe, adicione o argumento `follow_symlinks=False`.

```
>>> Path('.').exists()
True
>>> Path('setup.py').exists()
True
>>> Path('/etc').exists()
True
>>> Path('arquivoienxistente').exists()
False
```

Alterado na versão 3.12: O parâmetro *follow_symlinks* foi adicionado.

Path.**is_file**(*, *follow_symlinks=True*)

Retorna `True` se o caminho apontar para um arquivo comum. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um arquivo comum. Use *`Path.stat()`* para distinguir entre esses casos.

Este método normalmente segue links simbólicos; para excluir links simbólicos, adicione o argumento `follow_symlinks=False`.

Alterado na versão 3.13: O parâmetro *follow_symlinks* foi adicionado.

Path.**is_dir**(*, *follow_symlinks=True*)

Retorna `True` se o caminho apontar para um diretório. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um diretório. Use *`Path.stat()`* para distinguir entre esses casos.

Este método normalmente segue links simbólicos; para excluir links simbólicos para diretórios, adicione o argumento `follow_symlinks=False`.

Alterado na versão 3.13: O parâmetro *follow_symlinks* foi adicionado.

Path.**is_symlink**()

Retorna `True` se o caminho apontar para um link simbólico, ainda que esteja quebrado. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um arquivo comum. Use *`Path.stat()`* para distinguir entre esses casos.

Path.**is_junction**()

Retorna `True` se o caminho apontar para uma junção, e `False` para qualquer outro tipo de arquivo. Atualmente, apenas o Windows dá suporte a junções.

Adicionado na versão 3.12.

Path.**is_mount**()

Retorna `True` se o caminho for um *ponto de montagem*: um ponto em um sistema de arquivos onde um sistema de arquivos diferente foi montado. No POSIX, a função verifica se o pai do *path*, `path/..`, está em um dispositivo diferente de *path*, ou se `path/..` e *path* apontam para o mesmo nó-i no mesmo dispositivo – isso deve detectar pontos de montagem para todas as variantes Unix e POSIX. No Windows, um ponto de montagem é considerado uma raiz de letra de unidade (por exemplo, `c:\`), um compartilhamento UNC (por exemplo, `\\server\share`) ou um diretório de sistema de arquivos montado.

Adicionado na versão 3.7.

Alterado na versão 3.12: Suporte ao Windows foi adicionado.

Path.**is_socket**()

Retorna `True` se o caminho apontar para um socket Unix. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um socket Unix. Use *Path.stat()* para distinguir entre esses casos.

Path.**is_fifo**()

Retorna `True` se o caminho apontar para um FIFO. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um FIFO. Use *Path.stat()* para distinguir entre esses casos.

Path.**is_block_device**()

Retorna `True` se o caminho apontar para um dispositivo de bloco. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um de bloco. Use *Path.stat()* para distinguir entre esses casos.

Path.**is_char_device**()

Retorna `True` se o caminho apontar para um dispositivo de caractere. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um dispositivo de caractere. Use *Path.stat()* para distinguir entre esses casos.

Path.**samefile**(*other_path*)

Retorna se este path apontar para o mesmo arquivo como *other_path*, que pode ser um objeto PATH ou uma String. A semântica é semelhante a função *os.path.samefile()* e a função *os.path.samestat()*.

Um *OSError* poderá ser levantado caso algum arquivo não puder ser acessado por alguma razão.

```
>>> p = Path('spam')
>>> q = Path('eggs')
>>> p.samefile(q)
False
>>> p.samefile('spam')
True
```

Adicionado na versão 3.5.

Path.**info**

Um objeto *PathInfo* que oferece suporte a consultas de informações sobre o tipo de arquivo. O objeto expõe métodos que armazenam seus resultados em cache, o que pode ajudar a reduzir o número de chamadas de sistema necessárias ao alternar entre tipos de arquivo. Por exemplo:

```
>>> p = Path('src')
>>> if p.info.is_symlink():
...     print('link simbólico')
... elif p.info.is_dir():
...     print('diretório')
... elif p.info.exists():
...     print('alguma coisa mais')
... else:
...     print('não encontrado')
...
diretório
```

Se o caminho foi gerado a partir de *Path.iterdir()*, este atributo é inicializado com algumas informações sobre o tipo de arquivo obtidas a partir da varredura do diretório pai. O simples acesso a *Path.info* não realiza nenhuma consulta ao sistema de arquivos.

Para obter informações atualizadas, é melhor chamar *Path.is_dir()*, *is_file()* e *is_symlink()* em vez dos métodos deste atributo. Não há como redefinir o cache; em vez disso, você pode criar um novo objeto de caminho com um cache de informações vazio via `p = Path(p)`.

Adicionado na versão 3.14.

Lendo e escrevendo arquivos

`Path.`**`open`**(*mode='r'*, *buffering=-1*, *encoding=None*, *errors=None*, *newline=None*)

Abre o arquivo apontado pelo caminho, como a função embutida *open()* faz:

```
>>> p = Path('setup.py')
>>> with p.open() as f:
...     f.readline()
...
'#!/usr/bin/env python3\n'
```

`Path.`**`read_text`**(*encoding=None*, *errors=None*, *newline=None*)

Retorna o conteúdo decodificado do arquivo apontado como uma string:

```
>>> p = Path('meu_arquivo_texto')
>>> p.write_text('Conteúdo de arquivo texto')
18
>>> p.read_text()
'Conteúdo de arquivo texto'
```

O arquivo é aberto e, então, fechado. Os parâmetros opcionais têm o mesmo significado que em *open()*.

Adicionado na versão 3.5.

Alterado na versão 3.13: O parâmetro *newline* foi adicionado.

`Path.`**`read_bytes`**()

Retorna o conteúdo binário do arquivo apontado como um objeto bytes:

```
>>> p = Path('meu_arquivo_binário')
>>> p.write_bytes(b'meu_arquivo_binário')
20
>>> p.read_bytes()
b'Binary file contents'
```

Adicionado na versão 3.5.

`Path.`**`write_text`**(*data*, *encoding=None*, *errors=None*, *newline=None*)

Abre o arquivo apontado no modo de texto, escreve *data* e fecha o arquivo:

```
>>> p = Path('meu_arquivo_texto')
>>> p.write_text('Conteúdo de arquivo texto')
18
>>> p.read_text()
'Conteúdo de arquivo texto'
```

Um arquivo existente com o mesmo nome é sobrescrito. Os parâmetros opcionais têm o mesmo significado que em *open()*.

Adicionado na versão 3.5.

Alterado na versão 3.10: O parâmetro *newline* foi adicionado.

`Path.`**`write_bytes`**(*data*)

Abre o arquivo apontado no modo bytes, escreve *data* e fecha o arquivo:

```
>>> p = Path('meu_arquivo_binário')
>>> p.write_bytes(b'meu_arquivo_binário')
20
>>> p.read_bytes()
b'Binary file contents'
```

Um arquivo existente de mesmo nome será substituído.

Adicionado na versão 3.5.

Lendo diretórios

`Path.`**`iterdir`**`()`

Quando o caminho aponta para um diretório, produz objetos caminho do conteúdo do diretório:

```
>>> p = Path('docs')
>>> for child in p.iterdir(): child
...
PosixPath('docs/conf.py')
PosixPath('docs/_templates')
PosixPath('docs/make.bat')
PosixPath('docs/index.rst')
PosixPath('docs/_build')
PosixPath('docs/_static')
PosixPath('docs/Makefile')
```

Os filhos são gerados em ordem arbitrária e as entradas especiais `'.'` e `'..'` não são incluídas. Se um arquivo for removido ou adicionado ao diretório após a criação do iterador, não está especificado se um objeto caminho para esse arquivo é incluído.

Se o caminho não for um diretório ou estiver inacessível, *`OSError`* será levantada.

`Path.`**`glob`**`(`*pattern*, *, *case_sensitive=None*, *recurse_symlinks=False*`)`

Faz o glob do *pattern* relativo fornecido no diretório representado por este caminho, produzindo todos os arquivos correspondentes (de qualquer tipo):

```
>>> sorted(Path('.').glob('*.py'))
[PosixPath('pathlib.py'), PosixPath('setup.py'), PosixPath('test_pathlib.py')]
>>> sorted(Path('.').glob('*/*.py'))
[PosixPath('docs/conf.py')]
>>> sorted(Path('.').glob('**/*.py'))
[PosixPath('build/lib/pathlib.py'),
 PosixPath('docs/conf.py'),
 PosixPath('pathlib.py'),
 PosixPath('setup.py'),
 PosixPath('test_pathlib.py')]
```

Ver também

Documentação de *Linguagem de padrões*.

Por padrão, ou quando o argumento somente-nomeado *case_sensitive* é definido como `None`, esse método corresponde a caminhos usando regras de capitalização específicas da plataforma: normalmente, diferencia maiúsculas de minúsculas no POSIX e não diferencia maiúsculas de minúsculas no Windows. Defina *case_sensitive* como `True` ou `False` para substituir esse comportamento.

Por padrão, ou quando o argumento somente-nomeado *recurse_symlinks* é definido como `False`, este método segue links simbólicos, exceto ao expandir curingas "`**`". Defina *recurse_symlinks* como `True` para sempre seguir links simbólicos.

Levanta um *evento de auditoria* `pathlib.Path.glob` com os argumentos `self`, `pattern`.

Alterado na versão 3.12: O parâmetro *case_sensitive* foi adicionado.

Alterado na versão 3.13: O parâmetro *recurse_symlinks* foi adicionado.

Alterado na versão 3.13: O parâmetro *pattern* aceita um *objeto caminho ou similar*.

Alterado na versão 3.13: Quaisquer exceções `OSError` levantadas pela varredura do sistema de arquivos são suprimidas. Em versões anteriores, tais exceções são suprimidas em muitos casos, mas não em todos.

`Path.`**`rglob`**(*pattern*, *, *case_sensitive=None*, *recurse_symlinks=False*)

Faz um glob no *pattern* relativo dado recursivamente. Isso é como chamar `Path.glob()` com "`**/`" adicionado na frente do *pattern*.

Ver também

Documentação de *Linguagem de padrões* e `Path.glob()`.

Levanta um *evento de auditoria* `pathlib.Path.rglob` com argumentos `self`, `pattern`.

Alterado na versão 3.12: O parâmetro *case_sensitive* foi adicionado.

Alterado na versão 3.13: O parâmetro *recurse_symlinks* foi adicionado.

Alterado na versão 3.13: O parâmetro *pattern* aceita um *objeto caminho ou similar*.

`Path.`**`walk`**(*top_down=True*, *on_error=None*, *follow_symlinks=False*)

Gera os nomes de arquivos em uma árvore de diretórios percorrendo a árvore de cima para baixo ou de baixo para cima.

Para cada diretório na árvore de diretórios com raiz em *self* (incluindo *self*, mas excluindo '.' e '..'), o método produz uma tupla de 3 elementos de `(dirpath, dirnames, filenames)`.

dirpath é um `Path` para o diretório que está sendo percorrido no momento, *dirnames* é uma lista de strings para os nomes dos subdiretórios em *dirpath* (excluindo `'.'` e `'..'`), e *filenames* é uma lista de strings para os nomes dos arquivos não diretório em *dirpath*. Para obter um caminho completo (que começa com *self*) para um arquivo ou diretório em *dirpath*, faça `dirpath / name`. Se as listas são ou não classificadas depende do sistema de arquivos.

Se o argumento opcional *top_down* for verdadeiro (que é o padrão), o trio para um diretório é gerado antes dos trios para qualquer um de seus subdiretórios (os diretórios são percorridos de cima para baixo). Se *top_down* for falso, o trio para um diretório é gerado após os trios para todos os seus subdiretórios (os diretórios são percorridos de baixo para cima). Não importa o valor de *top_down*, a lista de subdiretórios é recuperada antes que os trios para o diretório e seus subdiretórios sejam percorridos.

Quando *top_down* é verdadeiro, o chamador pode modificar a lista *dirnames* no local (por exemplo, usando `del` ou atribuição de fatias), e `Path.walk()` só recursivamente nos subdiretórios cujos nomes permanecem em *dirnames*. Isso pode ser usado para podar a pesquisa, ou para impor uma ordem específica de visita, ou mesmo para informar `Path.walk()` sobre diretórios que o chamador cria ou renomeia antes de retomar `Path.walk()` novamente. Modificar *dirnames* quando *top_down* é false não tem efeito no comportamento de `Path.walk()`, pois os diretórios em *dirnames* já foram gerados no momento em que *dirnames* é cedido ao chamador.

Por padrão, os erros de `os.scandir()` são ignorados. Se o argumento opcional *on_error* for especificado, deve ser um chamável; ela será chamado com um argumento, uma instância da exceção `OSError`. O chamável pode tratar do erro para continuar a caminhada ou levantá-la novamente para parar a caminhada. Observe que o nome do arquivo está disponível como o atributo `filename` do objeto de exceção.

Por padrão, `Path.walk()` não segue links simbólicos e, em vez disso, os adiciona à lista *filenames*. Defina *follow_symlinks* como true para resolver links simbólicos e colocá-los em *dirnames* e *filenames* conforme apropriado para seus alvos e, consequentemente, visitar diretórios apontados por links simbólicos (quando suportado).

Nota

Esteja ciente de que definir *follow_symlinks* como true pode levar a uma recursão infinita se um link apontar para um diretório pai dele mesmo. `Path.walk()` não rastreia os diretórios que já visitou.

Nota

`Path.walk()` presume que os diretórios que percorre não sejam modificados durante a execução. Por exemplo, se um diretório de *dirnames* tiver sido substituído por um link simbólico e *follow_symlinks* for falso, `Path.walk()` ainda tentará descer até ele. Para evitar esse comportamento, remova os diretórios de *dirnames* conforme apropriado.

Nota

Diferentemente de `os.walk()`, `Path.walk()` lista links simbólicos para diretórios em *filenames* se *follow_symlinks* for falso.

Esse exemplo mostra o número de bytes usados por todos os arquivos em cada diretório, ignorando diretórios `__pycache__`:

```
from pathlib import Path
for root, dirs, files in Path("cpython/Lib/concurrent").walk(on_error=print):
  print(
      root,
      "consumes",
      sum((root / file).stat().st_size for file in files),
      "bytes in",
      len(files),
      "non-directory files"
  )
  if '__pycache__' in dirs:
        dirs.remove('__pycache__')
```

O próximo exemplo é uma implementação simples de `shutil.rmtree()`. Percorrer a árvore de baixo para cima é essencial, pois `rmdir()` não permite excluir um diretório antes que ele esteja vazio.

```
# Exclui tudo o que estiver acessível no diretório "top".
# CUIDADO: Isso é perigoso! Por exemplo, se top == Path('/'),
# todos os seus arquivos poderão ser excluídos.
for root, dirs, files in top.walk(top_down=False):
    for name in files:
        (root / name).unlink()
    for name in dirs:
        (root / name).rmdir()
```

Adicionado na versão 3.12.

Criando arquivos e diretórios

`Path.`**`touch`**(*mode=0o666*, *exist_ok=True*)

Cria um arquivo neste caminho especifico. Caso o *modo* for dado, ele será combinado com o valor do processo `umask` para determinar o modo de arquivo e as flags de acesso. Se o arquivo já existir, a função será bem-sucedida quando *exist_ok* for verdadeiro (e o tempo de modificação for atualizado para a hora atual), caso contrário a exceção `FileExistsError` será levantada.

Ver também

Os métodos `open()`, `write_text()` e `write_bytes()` são frequentemente usados para criar arquivos.

Path.**mkdir**(*mode=0o777*, *parents=False*, *exist_ok=False*)

Cria um novo diretório neste caminho fornecido. Se *mode* for fornecido, ele é combinado com o valor `umask` do processo para determinar o modo do arquivo e os sinalizadores de acesso. Se o caminho já existe, `FileExistsError` é levantada.

Se *parents* for verdadeiro, quaisquer pais ausentes neste caminho serão criados conforme necessário; eles são criados com as permissões padrão sem levar o *mode* em consideração (imitando o comando POSIX `mkdir -p`).

Se *parents* for falso (o padrão), um pai ausente levanta `FileNotFoundError`.

Se *exist_ok* for falso (o padrão), `FileExistsError` será levantada se o diretório alvo já existir.

Se *exist_ok* for verdadeiro, a exceção `FileExistsError` não será levantada, a menos que o caminho fornecido já exista no sistema de arquivos e não seja um diretório (mesmo comportamento do comando POSIX `mkdir -p`).

Alterado na versão 3.5: O parâmetro *exist_ok* foi adicionado.

Path.**symlink_to**(*target*, *target_is_directory=False*)

Faça desse path um symbolic link apontando para um *target*

No Windows, um link simbólico representa um arquivo ou um diretório e não se transforma no destino dinamicamente. Se o alvo estiver presente, será criado um link simbólico de mesmo tipo. Caso contrário, o link simbólico será criado como um diretório se *target_is_directory* for verdadeiro ou um link simbólico de arquivo (o padrão) caso contrário. Em plataformas não Windows, *target_is_directory* é ignorado.

```
>>> p = Path('meulink')
>>> p.symlink_to('setup.py')
>>> p.resolve()
PosixPath('/home/antoine/pathlib/setup.py')
>>> p.stat().st_size
956
>>> p.lstat().st_size
8
```

Nota

A ordem dos argumentos (link, target) é o inverso da função `os.symlink()`'s.

Alterado na versão 3.13: Levanta `UnsupportedOperation` se `os.symlink()` não estiver disponível. Em versões anteriores, `NotImplementedError` era levantada.

Path.**hardlink_to**(*target*)

Faz deste caminho um link físico para o mesmo arquivo que *target*.

Nota

A ordem dos argumentos (link, target) é o inverso da função `os.link()`'s.

Adicionado na versão 3.10.

Alterado na versão 3.13: Levanta `UnsupportedOperation` se `os.link()` não estiver disponível. Em versões anteriores, `NotImplementedError` era levantada.

Copiando, movendo e excluindo

`Path.`**`copy`**(*target*, *, *follow_symlinks=True*, *preserve_metadata=False*)

Copia este arquivo ou árvore de diretório para o *target* fornecido e retorna uma nova instância `Path` apontando para *target*.

Se a origem for um arquivo, o destino será substituído, caso seja um arquivo existente. Se a origem for um link simbólico e *follow_symlinks* for verdadeiro (o padrão), o destino do link simbólico será copiado. Caso contrário, o link simbólico será recriado no destino.

Se *preserve_metadata* for falso (o padrão), apenas estruturas de diretório e dados de arquivo terão a garantia de serem copiados. Defina *preserve_metadata* como verdadeiro para garantir que permissões de arquivo e diretório, sinalizadores, horários do último acesso e modificação e atributos estendidos sejam copiados onde suportados. Este argumento não tem efeito ao copiar arquivos no Windows (onde os metadados são sempre preservados).

> **Nota**
>
> Quando suportado pelo sistema operacional e pelo sistema de arquivos, este método realiza uma cópia leve, na qual os blocos de dados são copiados somente quando modificados. Isso é conhecido como cópia na gravação ou, em inglês, copy-on-write.

Adicionado na versão 3.14.

`Path.`**`copy_into`**(*target_dir*, *, *follow_symlinks=True*, *preserve_metadata=False*)

Copia este arquivo ou diretório para o *target_dir* fornecido, que deve ser um diretório existente. Os demais argumentos são tratados de forma idêntica a *`Path.copy()`*. Retorna uma nova instância de `Path` apontando para a cópia.

Adicionado na versão 3.14.

`Path.`**`rename`**(*target*)

Renomeia este arquivo ou diretório para o *target* fornecido e retorna uma nova instância de `Path` apontando para *target*. No Unix, se *target* existir e for um arquivo, ele será substituído silenciosamente se o usuário tiver permissão. No Windows, se *target* existir, a exceção *`FileExistsError`* será levantada. *target* pode ser uma string ou outro objeto caminho:

```
>>> p = Path('foo')
>>> p.open('w').write('algum texto')
9
>>> target = Path('bar')
>>> p.rename(target)
PosixPath('bar')
>>> target.open().read()
'algum texto'
```

O caminho de destino pode ser absoluto ou relativo. Caminhos relativos são interpretados em relação ao diretório de trabalho atual, *não* ao diretório do objeto `Path`.

Está implementada em termos de *`os.rename()`* e fornece as mesmas garantias.

Alterado na versão 3.8: Adicionado valor de retorno, retorna a nova instância de `Path`.

`Path.`**`replace`**(*target*)

Renomeia este arquivo ou diretório para o *target* fornecido e retorna uma nova instância de `Path` apontando para *target*. Se *target* apontar para um arquivo ou diretório vazio, ele será substituído incondicionalmente.

O caminho de destino pode ser absoluto ou relativo. Caminhos relativos são interpretados em relação ao diretório de trabalho atual, *não* ao diretório do objeto `Path`.

Alterado na versão 3.8: Adicionado valor de retorno, retorna a nova instância de `Path`.

Path.**move**(*target*)

Move este arquivo ou árvore de diretório para o *target* fornecido e retorna uma nova instância `Path` apontando para *target*.

Se o *destino* não existir, ele será criado. Se tanto este caminho quanto o *destino* forem arquivos existentes, o destino será sobrescrito. Se ambos os caminhos apontarem para o mesmo arquivo ou diretório, ou se o *destino* for um diretório não vazio, então *OSError* será levantada.

Se ambos os caminhos estiverem no mesmo sistema de arquivos, a movimentação será realizada com *os.replace()*. Caso contrário, este caminho será copiado (preservando metadados e links simbólicos) e, em seguida, excluído.

Adicionado na versão 3.14.

Path.**move_into**(*target_dir*)

Move este arquivo ou diretório para o *target_dir* fornecido, que deve ser um diretório existente. Retorna uma nova instância de `Path` apontando para o caminho movido.

Adicionado na versão 3.14.

Path.**unlink**(*missing_ok=False*)

Remova esse arquivo ou link simbólico. Caso o caminho aponte para um diretório, use a função *Path.rmdir()* em vez disso.

Se *missing_ok* for falso (o padrão), *FileNotFoundError* é levantada se o caminho não existir.

Se *missing_ok* for verdadeiro, exceções de *FileNotFoundError* serão ignoradas (mesmo comportamento que o comando POSIX `rm -f`).

Alterado na versão 3.8: O parâmetro *missing_ok* foi adicionado.

Path.**rmdir**()

Remove este diretório. O diretório deve estar vazio.

Permissões e propriedade

Path.**owner**(*, *follow_symlinks=True*)

Retorna o nome do usuário que possui o arquivo. *KeyError* é levantada caso o identificador de usuário (UID) não for encontrado no banco de dados do sistema.

Esse método normalmente segue links simbólicos; para retornar o dono do link simbólico, adicione o argumento `follow_symlinks=False`.

Alterado na versão 3.13: Levanta *UnsupportedOperation* se o módulo *pwd* não estiver disponível. Em versões anteriores, *NotImplementedError* era levantada.

Alterado na versão 3.13: O parâmetro *follow_symlinks* foi adicionado.

Path.**group**(*, *follow_symlinks=True*)

Retorna o nome do usuário que possui o grupo. *KeyError* é levantada caso o identificador de grupo (GID) não for encontrado no banco de dados do sistema.

Esse método normalmente segue links simbólicos; para retornar o grupo do link simbólico, adicione o argumento `follow_symlinks=False`.

Alterado na versão 3.13: Levanta *UnsupportedOperation* se o módulo *grp* não estiver disponível. Em versões anteriores, *NotImplementedError* era levantada.

Alterado na versão 3.13: O parâmetro *follow_symlinks* foi adicionado.

Path.**chmod**(*mode*, *, *follow_symlinks=True*)

Altera o modo de arquivo e as permissões, como *os.chmod()*.

This method normally follows symlinks. Some Unix flavours support changing permissions on the symlink itself; on these platforms you may add the argument `follow_symlinks=False`, or use *lchmod()*.

```
>>> p = Path('setup.py')
>>> p.stat().st_mode
33277
>>> p.chmod(0o444)
>>> p.stat().st_mode
33060
```

Alterado na versão 3.10: O parâmetro *follow_symlinks* foi adicionado.

`Path.`**`lchmod`**`(`*mode*`)`

Como *Path.chmod()*, mas, se o caminho apontar para um link simbólico, o modo do link simbólico é alterado ao invés de seu alvo.

11.1.5 Linguagem de padrões

Os seguintes curingas são suportados em padrões para *full_match()*, *glob()* e *rglob()*:

**** (todo o segmento)**
Corresponde a qualquer número de segmentos de arquivo ou diretório, incluindo zero.

*** (todo o segmento)**
Corresponde a um segmento de arquivo ou diretório.

*** (parte de um segmento)**
Corresponde a qualquer número de caracteres não separadores, incluindo zero.

?
Corresponde a um caractere não separador.

[seq]
Corresponde a um caractere em *seq*, onde *seq* é uma sequência de caracteres. Expressões de intervalo são suportadas; por exemplo, `[a-z]` corresponde a qualquer letra ASCII minúscula. Vários intervalos podem ser combinados: `[a-zA-Z0-9_]` corresponde a qualquer letra, dígito ou sublinhado ASCII.

[!seq]
Corresponde a um caractere que não está em *seq*, onde *seq* segue as mesmas regras acima.

Para uma correspondência literal, coloque os metacaracteres entre colchetes. Por exemplo, `"[?]"` corresponde ao caractere `"?"`.

O curinga "`**`" habilita fazer glob recursivo. Alguns exemplos:

Padrão	Significado
"`**/*`"	Qualquer caminho com pelo menos um segmento.
"`**/*.py`"	Qualquer caminho com um segmento final terminando em "`.py`".
"`assets/**`"	Qualquer caminho que comece com "`assets/`".
"`assets/**/*`"	Qualquer caminho que comece com "`assets/`", excluindo o próprio "`assets/`".

Nota

Fazer globs com o curinga "`**`" visita todos os diretórios da árvore. Árvores de diretórios grandes podem levar muito tempo para serem pesquisadas.

Alterado na versão 3.13: Fazer glob com um padrão que termina com "`**`" retorna arquivos e diretórios. Em versões anteriores, apenas diretórios eram retornados.

Em *Path.glob()* e *rglob()*, uma barra final pode ser adicionada ao padrão para corresponder apenas aos diretórios.

Alterado na versão 3.11: Fazer glob com um padrão que termina com um separador de componentes de caminho (*sep* ou *altsep*) retorna apenas diretórios.

11.1.6 Comparação com o módulo `glob`

Os padrões aceitos e os resultados gerados por *Path.glob()* e *Path.rglob()* diferem ligeiramente daqueles pelo módulo *glob*:

1. Arquivos que começam com um ponto não são especiais em pathlib. Isso é como passar `include_hidden=True` para *glob.glob()*.
2. Os componentes do padrão "`**`" são sempre recursivos em pathlib. Isso é como passar `recursive=True` para *glob.glob()*.
3. Os componentes do padrão "`**`" não seguem links simbólicos por padrão em pathlib. Esse comportamento não tem equivalente em *glob.glob()*, mas você pode passar `recurse_symlinks=True` para *Path.glob()* para obter um comportamento compatível.
4. Como todos os objetos *PurePath* e *Path*, os valores retornados de *Path.glob()* e *Path.rglob()* não incluem barras finais.
5. Os valores retornados de `path.glob()` e `path.rglob()` do pathlib incluem o *path* como prefixo, diferente dos resultados de `glob.glob(root_dir=path)`.
6. Os valores retornados de `path.glob()` e `path.rglob()` do pathlib podem incluir o próprio *path*, por exemplo, ao globar "`**`", enquanto os resultados de `glob.glob(root_dir=path)` nunca incluem uma string vazia que corresponderia a *path*.

11.1.7 Comparação com os módulos `os` e `os.path`

pathlib implementa operações de caminho usando os objetos *PurePath* e *Path*, e por isso é considerado *orientado a objetos*. Por outro lado, os módulos *os* e *os.path* fornecem funções que funcionam com objetos de baixo nível `str` e `bytes`, o que é uma abordagem mais *procedimental*. Alguns usuários consideram o estilo orientado a objetos mais legível.

Muitas funções em *os* e *os.path* oferecem suporte a caminhos `bytes` e *caminhos relativos aos descritores de diretório*. Esses recursos não estão disponíveis em pathlib.

Os tipos `str` e `bytes` do Python e partes dos módulos *os* e *os.path* são escritos em C e são muito rápidos. pathlib é escrito em Python puro e geralmente é mais lento, mas raramente lento o suficiente para fazer diferença.

A normalização de caminho do pathlib é um pouco mais opinativa e consistente do que a do *os.path*. Por exemplo, enquanto *os.path.abspath()* elimina segmentos "`..`" de um caminho, o que pode alterar seu significado se links simbólicos estiverem envolvidos, *Path.absolute()* preserva esses segmentos para maior segurança.

A normalização de caminho do pathlib pode torná-lo inadequado para algumas aplicações:

1. pathlib normaliza `Path("minha_pasta/")` para `Path("minha_pasta")`, o que altera o significado de um caminho quando fornecido a diversas APIs de sistemas operacionais e utilitários de linha de comando. Especificamente, a ausência de um separador final pode permitir que o caminho seja resolvido como um arquivo ou diretório, em vez de apenas um diretório.
2. pathlib normaliza `Path("./meu_programa")` para `Path("meu_programa")`, o que altera o significado de um caminho quando usado como um caminho de pesquisa executável, como em um shell ou ao gerar um processo filho. Especificamente, a ausência de um separador no caminho pode forçar a busca em `PATH` em vez do diretório atual.

Como consequência dessas diferenças, pathlib não é um substituto imediato para *os.path*.

Ferramentas correspondentes

Abaixo está uma tabela mapeando várias funções *os* a sua *PurePath*/*Path* equivalente.

os e os.path	pathlib
os.path.dirname()	PurePath.parent
os.path.basename()	PurePath.name
os.path.splitext()	PurePath.stem, PurePath.suffix
os.path.join()	PurePath.joinpath()
os.path.isabs()	PurePath.is_absolute()
os.path.relpath()	PurePath.relative_to()[1]
os.path.expanduser()	Path.expanduser()[2]
os.path.realpath()	Path.resolve()
os.path.abspath()	Path.absolute()[3]
os.path.exists()	Path.exists()
os.path.isfile()	Path.is_file()
os.path.isdir()	Path.is_dir()
os.path.islink()	Path.is_symlink()
os.path.isjunction()	Path.is_junction()
os.path.ismount()	Path.is_mount()
os.path.samefile()	Path.samefile()
os.getcwd()	Path.cwd()
os.stat()	Path.stat()
os.lstat()	Path.lstat()
os.listdir()	Path.iterdir()
os.walk()	Path.walk()[4]
os.mkdir(), os.makedirs()	Path.mkdir()
os.link()	Path.hardlink_to()
os.symlink()	Path.symlink_to()
os.readlink()	Path.readlink()
os.rename()	Path.rename()
os.replace()	Path.replace()
os.remove(), os.unlink()	Path.unlink()
os.rmdir()	Path.rmdir()
os.chmod()	Path.chmod()
os.lchmod()	Path.lchmod()

11.1.8 Protocolos

O módulo *pathlib.types* fornece tipos para verificação de tipo estático.

Adicionado na versão 3.14.

class pathlib.types.**PathInfo**

Um *typing.Protocol* descrevendo o atributo *Path.info*. Implementações podem retornar resultados em cache de seus métodos.

exists(*, *follow_symlinks=True*)

Retorna `True` se o caminho for um arquivo ou diretório existente, ou qualquer outro tipo de arquivo; retorna `False` se o caminho não existir.

Se *follow_symlinks* for `False`, retorna `True` para links simbólicos sem verificar se seus alvos existem.

[1] *os.path.relpath()* chama *abspath()* para tornar os caminhos absolutos e remover partes "..", enquanto *PurePath.relative_to()* é uma operação lexical que levanta *ValueError* quando as âncoras de suas entradas são diferentes (por exemplo, se um caminho for absoluto e o outro relativo).

[2] *os.path.expanduser()* retorna o caminho inalterado se o diretório inicial não puder ser resolvido, enquanto *Path.expanduser()* levanta *RuntimeError*.

[3] *os.path.abspath()* remove componentes ".." sem resolver links simbólicos, o que pode alterar o significado do caminho, enquanto *Path.absolute()* deixa quaisquer componentes ".." no caminho.

[4] *os.walk()* sempre segue links simbólicos ao categorizar caminhos em *dirnames* e *filenames*, enquanto *Path.walk()* categoriza todos os links simbólicos em *filenames* quando *follow_symlinks* é falso (o padrão).

is_dir(*, *follow_symlinks=True*)

Retorna `True` se o caminho for um diretório ou um link simbólico apontando para um diretório; retorna `False` se o caminho for (ou apontar para) qualquer outro tipo de arquivo ou se ele não existir.

Se *follow_symlinks* for `False`, retorna `True` apenas se este caminho for um diretório (sem seguir os links simbólicos); retorna `False` se o caminho for qualquer outro tipo de arquivo ou se ele não existe.

is_file(*, *follow_symlinks=True*)

Retorna `True` se este caminho entrada for um arquivo ou um link simbólico apontando para um arquivo; retorna `False` se a entrada é ou aponta para um diretório ou outro caminho que não seja de arquivo, ou se ela não existe.

Se *follow_symlinks* for `False`, retorna `True` apenas se este caminho for um arquivo (sem seguir os links simbólicos); retorna `False` se o caminho for um diretório ou outro que não seja um arquivo, ou se ela não existir.

is_symlink()

Retorna `True` se este caminho for um link simbólico (mesmo se quebrado); retorna `False` se o caminho é um diretório ou qualquer tipo de arquivo, ou se ele não existir.

11.2 `os.path` — Manipulações comuns de nomes de caminhos

Código-fonte: Lib/genericpath.py, Lib/posixpath.py (para POSIX) e Lib/ntpath.py (para Windows).

Este módulo implementa algumas funções úteis em nomes de caminho. Para ler ou escrever arquivos veja *open()*, e para acessar o sistema de arquivos veja o módulo *os*. Os parâmetros de caminho podem ser passados como strings, ou bytes, ou qualquer objeto que implemente o protocolo *os.PathLike*.

Ao contrário de um shell Unix, Python não faz nenhuma expansão *automática* de caminho. Funções como *expanduser()* e *expandvars()* podem ser invocadas explicitamente quando uma aplicação deseja uma expansão de caminho no estilo do shell. (Veja também o módulo *glob*.)

Ver também

O módulo *pathlib* oferece objetos de caminho de alto nível.

Nota

Todas essas funções aceitam apenas bytes ou apenas objetos de string como seus parâmetros. O resultado é um objeto do mesmo tipo, se um caminho ou nome de arquivo for retornado.

Nota

Uma vez que diferentes sistemas operacionais têm diferentes convenções de nome de caminho, existem várias versões deste módulo na biblioteca padrão. O módulo *os.path* é sempre o módulo de caminho adequado para o sistema operacional em que o Python está sendo executado e, portanto, pode ser usado para caminhos locais. No entanto, você também pode importar e usar os módulos individuais se quiser manipular um caminho que esteja *sempre* em um dos diferentes formatos. Todos eles têm a mesma interface:

- `posixpath` para caminhos no estilo UNIX
- `ntpath` para caminhos do Windows

Alterado na versão 3.8: *exists()*, *lexists()*, *isdir()*, *isfile()*, *islink()* e *ismount()* agora retornam `False` em vez de levantar uma exceção para caminhos que contêm caracteres ou bytes não representáveis no nível de sistema de operacional.

os.path.**abspath**(*path*)

Retorna uma versão normalizada e absolutizada do nome de caminho *path*. Na maioria das plataformas, isso é equivalente a chamar a função *normpath()* da seguinte forma: `normpath(join(os.getcwd(), path))`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**basename**(*path*, */*)

Retorna o nome base do caminho *path*. Este é o segundo elemento do par retornado pela passagem de *path* para a função *split()*. Observe que o resultado desta função é diferente do programa Unix **basename**; onde **basename** para `'/foo/bar/'` retorna `'bar'`, a função *basename()* retorna uma string vazia (`''`).

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**commonpath**(*paths*)

Retorna o subcaminho comum mais longo de cada nome de caminho no iterável *paths*. Levanta *ValueError* se *paths* contiverem nomes de caminho absolutos e relativos, os *paths* estiverem em unidades diferentes ou se *paths* estiverem vazios. Ao contrário de *commonprefix()*, retorna um caminho válido.

Adicionado na versão 3.5.

Alterado na versão 3.6: Aceita uma sequência de *objetos caminho ou similares*.

Alterado na versão 3.13: Qualquer iterável agora pode ser passado, em vez de apenas sequências.

os.path.**commonprefix**(*list*, */*)

Retorna o prefixo de caminho mais longo (obtido caractere por caractere) que é um prefixo de todos os caminhos em *list*. Se *list* estiver vazia, retorna a string vazia (`''`).

Nota

Esta função pode retornar caminhos inválidos porque funciona um caractere por vez. Para obter um caminho válido, consulte *commonpath()*.

```
>>> os.path.commonprefix(['/usr/lib', '/usr/local/lib'])
'/usr/l'

>>> os.path.commonpath(['/usr/lib', '/usr/local/lib'])
'/usr'
```

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**dirname**(*path*, */*)

Retorna o nome do diretório do nome de caminho *path*. Este é o primeiro elemento do par retornado passando *path* para a função *split()*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**exists**(*path*)

Retorna `True` se *path* se referir a um caminho existente ou um descritor de arquivo aberto. Retorna `False` para links simbólicos quebrados. Em algumas plataformas, esta função pode retornar `False` se a permissão não for concedida para executar *os.stat()* no arquivo solicitado, mesmo se o *path* existir fisicamente.

Alterado na versão 3.3: *path* agora pode ser um inteiro: `True` é retornado se for um descritor de arquivo aberto, `False` caso contrário.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**lexists**(*path*)

Retorna `True` se *path* se referir a um caminho existente, incluindo links simbólicos quebrados. Equivalente a *exists()* em plataformas sem *os.lstat()*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**expanduser**(*path*)

No Unix e no Windows, retorna o argumento com um componente inicial de ~ ou ~user substituído pelo diretório inicial daquele usuário *user*.

No Unix, um ~ no início é substituído pela variável de ambiente HOME se estiver definida; caso contrário, o diretório pessoal do usuário atual é procurado no diretório de senha através do módulo embutido *pwd*. Um ~user no início é procurado diretamente no diretório de senhas.

No Windows, USERPROFILE será usada se definida; caso contrário, uma combinação de HOMEPATH e HOMEDRIVE será usada. Um ~user inicial é tratado verificando se o último componente do diretório home do usuário atual corresponde a USERNAME, e substituindo-o se for o caso.

Se a expansão falhar ou se o caminho não começar com um til, o caminho será retornado inalterado.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.8: Não usa mais HOME no Windows.

os.path.**expandvars**(*path*)

Retorna o argumento com as variáveis de ambiente expandidas. Substrings da forma $name ou ${name} são substituídas pelo valor da variável de ambiente *name*. Nomes de variáveis malformados e referências a variáveis não existentes permanecem inalterados.

No Windows, expansões %name% são suportadas juntamente a $name e ${name}.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**getatime**(*path*, /)

Retorna a hora do último acesso de *path*. O valor de retorno é um número de ponto flutuante dando o número de segundos desde a Era Unix (veja o módulo *time*). Levanta *OSError* se o arquivo não existe ou está inacessível.

os.path.**getmtime**(*path*, /)

Retorna a hora da última modificação de *path*. O valor de retorno é um número de ponto flutuante dando o número de segundos desde a Era Unix (veja o módulo *time*). Levanta *OSError* se o arquivo não existe ou está inacessível.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**getctime**(*path*, /)

Retorna o ctime do sistema que, em alguns sistemas (como Unix) é a hora da última alteração de metadados, e, em outros (como Windows), é a hora de criação de *path*. O valor de retorno é um número que fornece o número de segundos desde a Era Unix (veja o módulo *time*). Levanta *OSError* se o arquivo não existe ou está inacessível.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**getsize**(*path*, /)

Retorna o tamanho, em bytes, de *path*. Levanta *OSError* se o arquivo não existe ou está inacessível.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**isabs**(*path*, /)

Retorna True se *path* for um nome de caminho absoluto. No Unix, isso significa que começa com uma barra, no Windows começa com duas (contra)barras ou uma letra de unidade, caractere de dois pontos e (contra)barra juntos.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.13: No Windows, retorna False se o caminho fornecido começar com exatamente uma (contra)barra.

os.path.**isfile**(*path*)

Retorna True se *path* for um arquivo regular *existente*. Isso segue links simbólicos, então *islink()* e *isfile()* podem ser verdadeiros para o mesmo caminho.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**isdir**(*path*, */*)

Retorna `True` se *path* for um diretório *existente*. Isso segue links simbólicos, então *islink()* e *isdir()* podem ser verdadeiros para o mesmo caminho.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**isjunction**(*path*)

Retorna `True` se *path* se referir a uma entrada de diretório *existente* que é uma junção. Sempre retorna `False` se junções não forem suportados na plataforma atual.

Adicionado na versão 3.12.

os.path.**islink**(*path*)

Retorna `True` se *path* se referir a uma entrada de diretório *existente* que é um link simbólico. Sempre `False` se links simbólicos não forem suportados pelo tempo de execução Python.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**ismount**(*path*)

Retorna `True` se o nome de caminho *path* for um *ponto de montagem*: um ponto em um sistema de arquivos onde um sistema de arquivos diferente foi montado. No POSIX, a função verifica se o pai de *path*, `path/..`, está em um dispositivo diferente de *path*, ou se `path/..` e *path* apontam para o mesmo nó-i no mesmo dispositivo – isso deve detectar pontos de montagem para todas as variantes Unix e POSIX. Não é capaz de detectar confiavelmente montagens bind no mesmo sistema de arquivos. Em sistemas Linux, sempre vai retornar `True` para subvolumes btrfs, mesmo se eles não forem pontos de montagem. No Windows, uma raiz de letra de unidade e um UNC de compartilhamento são sempre pontos de montagem e, para qualquer outro caminho, `GetVolumePathName` é chamado para ver se é diferente do caminho de entrada.

Alterado na versão 3.4: Adicionado suporte para detecção de pontos de montagem não raiz no Windows.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**isdevdrive**(*path*)

Retorna `True` se o nome do caminho *path* estiver localizado em um Windows Dev Drive. Um Dev Drive é otimizado para cenários de desenvolvedor e oferece desempenho mais rápido para leitura e gravação de arquivos. É recomendado para uso em código-fonte, diretórios de construção temporários, caches de pacotes e outras operações com uso intensivo de ES.

Pode levantar um erro para um caminho inválido, por exemplo, um sem uma unidade reconhecível, mas retorna `False` em plataformas que não provêm suporte a Dev Drives. Consulte a documentação do Windows para obter informações sobre como habilitar e criar Dev Drives.

Adicionado na versão 3.12.

Alterado na versão 3.13: A função já está disponível em todas as plataformas, e sempre retornará `False` naquelas que não possuem suporte para Dev Drives

os.path.**isreserved**(*path*)

Retorna `True` se *path* for um caminho reservado no sistema atual.

No Windows, os nomes de arquivos reservados incluem aqueles que terminam com espaço ou ponto; aqueles que contêm dois pontos (ou seja, fluxos de arquivos como "nome:fluxo"), caracteres curinga (ou seja, `'*?"<>'`), encadeamento (pipe) ou caracteres de controle ASCII; bem como nomes de dispositivos DOS, como "NUL", "CON", "CONIN$", "CONOUT$", "AUX", "PRN", "COM1" e "LPT1".

Nota

Esta função aproxima regras para caminhos reservados na maioria dos sistemas Windows. Essas regras mudam com o tempo em diversas versões do Windows. Esta função pode ser atualizada em versões futuras do Python à medida que as alterações nas regras se tornarem amplamente disponíveis.

Disponibilidade: Windows.

Adicionado na versão 3.13.

os.path.**join**(*path*, */*, **paths*)

Junta um ou mais segmentos do caminho de forma inteligente. O valor de retorno é a concatenação de *path* e todos os membros de **paths* com exatamente um separador de diretório seguindo cada parte não vazia exceto a última. Significa que o resultado só terminará em um separador se a última parte estiver vazia ou terminar em um separador. Se um segmento for um caminho absoluto (que no Windows requer a unidade/drive e uma raiz), todos os segmentos anteriores serão ignorados e a união continuará a partir do segmento do caminho absoluto.

No Windows, a unidade não é redefinida quando um segmento de caminho raiz (por exemplo, `r'\foo'`) é encontrado. Se um segmento contiver uma unidade diferente ou um caminho absoluto, todos os segmentos anteriores serão ignorados e a unidade será redefinida. Observe que, como há um diretório atual para cada unidade, `os.path.join("c:", "foo")` representa um caminho relativo ao diretório atual na unidade `C:` (`c:foo`), e não `c:\foo`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *path* e *paths*.

os.path.**normcase**(*path*, */*)

Normaliza o estado de letras maiúsculas/minúsculas de um nome de caminho. No Windows, converte todos os caracteres do nome do caminho em minúsculas e também converte barras normais em barras invertidas. Em outros sistemas operacionais, retorna o caminho inalterado.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**normpath**(*path*)

Normaliza um nome de caminho retirando separadores redundantes e referências de nível superior para que `A//B`, `A/B/`, `A/./B` e `A/foo/../B` todos se tornem `A/B`. Essa manipulação de string pode mudar o significado de um caminho que contém links simbólicos. No Windows, ele converte barras normais em barras invertidas. Para normalizar o estado de letras maiúsculas/minúsculas, use *`normcase()`*.

> **Nota**
>
> Em sistemas POSIX, de acordo com IEEE Std 1003.1 2013 Edition; 4.13 Pathname Resolution, se um nome de caminho começa com exatamente duas barras, o primeiro componente após os caracteres iniciais pode ser interpretado em um forma definida pela implementação, embora mais de dois caracteres iniciais devam ser tratados como um único caractere.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**realpath**(*path*, */*, ***, *strict=False*)

Retorna o caminho canônico do nome de arquivo especificado, eliminando quaisquer links simbólicos encontrados no caminho (se forem suportados pelo sistema operacional). No Windows, esta função também resolverá nomes de estilo do MS-DOS (também chamado de 8.3), como `C:\\PROGRA~1` para `C:\\Program Files`.

Por padrão, o caminho é avaliado até o primeiro componente que não existe, é um laço de link simbólico ou cuja avaliação levanta *`OSError`*. Todos esses componentes são anexados inalterados à parte existente do caminho.

Alguns erros tratados dessa forma incluem "acesso negado", "não é um diretório" ou "argumento inválido para função interna". Assim, o caminho resultante pode estar ausente ou inacessível, ainda pode conter links ou laços e pode passar por diretórios não especificados.

Esse comportamento pode ser modificado por argumentos nomeados:

Se *strict* for `True`, o primeiro erro encontrado ao avaliar o caminho será levantado novamente. Em particular, *`FileNotFoundError`* é levantada se *path* não existir, ou outro *`OSError`* se ele estiver inacessível.

Se *strict* for *`os.path.ALLOW_MISSING`*, erros diferentes de *`FileNotFoundError`* serão levantados novamente (como em `strict=True`). Portanto, o caminho retornado não conterá nenhum link simbólico, mas o arquivo nomeado e alguns de seus diretórios pais podem estar ausentes.

> **Nota**
>
> Esta função emula o procedimento do sistema operacional para tornar um caminho canônico, que difere ligeiramente entre o Windows e o UNIX em relação a como links e componentes de caminho subsequentes interagem.
>
> As APIs do sistema operacional tornam os caminhos canônicos conforme necessário, portanto, normalmente não é necessário chamar esta função.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.8: Links simbólicos e junções agora são resolvidos no Windows.

Alterado na versão 3.10: O parâmetro *strict* foi adicionado.

Alterado na versão 3.14: O valor `ALLOW_MISSING` para o parâmetro *strict* foi adicionado.

`os.path.`**`ALLOW_MISSING`**

Valor especial usado para o argumento *strict* em `realpath()`.

Adicionado na versão 3.14.

`os.path.`**`relpath`**(*path*, *start=os.curdir*)

Retorna um caminho de arquivo relativo a *caminho* do diretório atual ou de um diretório *start* opcional. Este é um cálculo de caminho: o sistema de arquivos não é acessado para confirmar a existência ou natureza de *path* ou *start*. No Windows, `ValueError` é levantada quando *path* e *start* estão em unidades diferentes.

start tem como padrão `os.curdir`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.path.`**`samefile`**(*path1*, *path2*, */*)

Retorna `True` se ambos os argumentos de nome de caminho se referem ao mesmo arquivo ou diretório. Isso é determinado pelo número do dispositivo e número do nó-i e levanta uma exceção se uma chamada `os.stat()` em qualquer um dos caminhos falhar.

Alterado na versão 3.2: Adicionado suporte ao Windows.

Alterado na versão 3.4: O Windows agora usa a mesma implementação que todas as outras plataformas.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.path.`**`sameopenfile`**(*fp1*, *fp2*)

Retorna `True` se os descritores de arquivo *fp1* e *fp2* fazem referência ao mesmo arquivo.

Alterado na versão 3.2: Adicionado suporte ao Windows.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.path.`**`samestat`**(*stat1*, *stat2*, */*)

Retorna `True` se as tuplas de estatísticas *stat1* e *stat2* fazem referência ao mesmo arquivo. Essas estruturas podem ter sido retornadas por `os.fstat()`, `os.lstat()` ou `os.stat()`. Esta função implementa a comparação subjacente usada por `samefile()` e `sameopenfile()`.

Alterado na versão 3.4: Adicionado suporte ao Windows.

`os.path.`**`split`**(*path*, */*)

Divide o caminho *path* em um par, `(cabeça, rabo)` onde *rabo* é o último componente do nome do caminho e *cabeça* é tudo o que leva a isso. A parte *rabo* nunca conterá uma barra; se *path* terminar com uma barra, *tail* ficará vazio. Se não houver uma barra no *path*, o *head* ficará vazio. Se *path* estiver vazio, *cabeça* e *rabo* estarão vazios. As barras finais são retiradas da *cabeça*, a menos que seja a raiz (uma ou mais barras apenas). Em todos os casos, `join(cabeça, rabo)` retorna um caminho para o mesmo local que *path* (mas as strings podem ser diferentes). Veja também as funções `dirname()` e `basename()`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**splitdrive**(*path*, /)

Divide o nome do caminho *path* em um par `(unidade, rabo)` onde *unidade* é um ponto de montagem ou uma string vazia. Em sistemas que não usam especificações de unidade, *unidade* sempre será a string vazia. Em todos os casos, `unidade + rabo` será o mesmo que *path*.

No Windows, divide um nome de caminho em unidade/ponto de compartilhamento UNC e caminho relativo.

Se o caminho contiver uma letra de unidade, a unidade conterá tudo, incluindo os dois pontos:

```
>>> splitdrive("c:/dir")
("c:", "/dir")
```

Se o caminho contiver um caminho UNC, a unidade conterá o nome do host e o compartilhamento:

```
>>> splitdrive("//host/computador/diretório")
("//host/computador", "/diretório")
```

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**splitroot**(*path*, /)

Divide o nome de caminho *path* em uma tupla de 3 itens `(drive, root, tail)` onde *drive* é um nome de dispositivo ou ponto de montagem, *root* é uma string de separadores após a unidade e *tail* é tudo depois da raiz. Qualquer um desses itens pode ser uma string vazia. Em todos os casos, `drive + root + tail` será igual a *path*.

Em sistemas POSIX, *drive* está sempre vazio. A *root* pode estar vazia (se *path* for relativo), uma única barra (se *path* for absoluto) ou duas barras (definidas pela implementação de acordo com IEEE Std 1003.1-2017; 4.13 Pathname Resolution.) Por exemplo:

```
>>> splitroot('/home/sam')
('', '/', 'home/sam')
>>> splitroot('//home/sam')
('', '//', 'home/sam')
>>> splitroot('///home/sam')
('', '/', '//home/sam')
```

No Windows, *drive* pode estar vazio, ser um nome de letra de unidade, um compartilhamento UNC ou um nome de dispositivo. *root* pode estar vazio, ser uma barra ou uma barra invertida. Por exemplo:

```
>>> splitroot('C:/Users/Sam')
('C:', '/', 'Users/Sam')
>>> splitroot('//Server/Share/Users/Sam')
('//Server/Share', '/', 'Users/Sam')
```

Adicionado na versão 3.12.

os.path.**splitext**(*path*, /)

Divida o nome do caminho *path* em um par `(root, ext)` de modo que `root + ext == path`, e a extensão, *ext*, esteja vazia ou comece com um ponto e contenha no máximo um período.

Se o caminho não contiver extensão, *ext* será `''`:

```
>>> splitext('bar')
('bar', '')
```

Se o caminho contiver uma extensão, *ext* será definido para esta extensão, incluindo o ponto inicial. Observe que os períodos anteriores serão ignorados:

```
>>> splitext('foo.bar.exe')
('foo.bar', '.exe')
```

(continua na próxima página)

(continuação da página anterior)

```
>>> splitext('/foo/bar.exe')
('/foo/bar', '.exe')
```

Os períodos iniciais do último componente do caminho são considerados parte da raiz:

```
>>> splitext('.cshrc')
('.cshrc', '')
>>> splitext('/foo/....jpg')
('/foo/....jpg', '')
```

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.path.**supports_unicode_filenames**

`True` se strings Unicode arbitrárias podem ser usadas como nomes de arquivo (dentro das limitações impostas pelo sistema de arquivos).

11.3 `stat` — Interpretando resultados de `stat()`

Código-fonte: Lib/stat.py

O módulo *`stat`* define constantes e funções para interpretar os resultados de *`os.stat()`*, *`os.fstat()`* e *`os.lstat()`* (se existirem). Para detalhes completos sobre chamadas `stat()`, `fstat()` e `lstat()`, consulte a documentação do seu sistema.

Alterado na versão 3.4: O módulo stat é apoiado por uma implementação C.

O módulo *`stat`* define as seguintes funções para testar tipos de arquivos específicos:

stat.**S_ISDIR**(*mode*)

Retorna diferente de zero se o modo for de um diretório.

stat.**S_ISCHR**(*mode*)

Retorna diferente de zero se o modo for de um arquivo de dispositivo especial de caractere.

stat.**S_ISBLK**(*mode*)

Retorna diferente de zero se o modo for de um arquivo de dispositivo especial de bloco.

stat.**S_ISREG**(*mode*)

Retorna diferente de zero se o modo for de um arquivo regular.

stat.**S_ISFIFO**(*mode*)

Retorna diferente de zero se o modo for de um FIFO (encadeamento nomeado).

stat.**S_ISLNK**(*mode*)

Retorna diferente de zero se o modo for de um link simbólico.

stat.**S_ISSOCK**(*mode*)

Retorna diferente de zero se o modo for de um soquete.

stat.**S_ISDOOR**(*mode*)

Retorna diferente de zero se o modo for de uma porta.

Adicionado na versão 3.4.

stat.**S_ISPORT**(*mode*)

Retorna diferente de zero se o modo for de uma porta de eventos.

Adicionado na versão 3.4.

stat.**S_ISWHT**(*mode*)

Retorna diferente de zero se o modo for de um apagamento.

Adicionado na versão 3.4.

Duas funções adicionais são definidas para manipulação mais geral do modo do arquivo:

stat.**S_IMODE**(*mode*)

Retorna a parte do modo do arquivo que pode ser definido por *os.chmod()* — ou seja, os bits de permissão do arquivo, mais os bits sticky bit, set-group-id e set-user-id (em sistemas que têm suporte a eles).

stat.**S_IFMT**(*mode*)

Retorna a parte do modo do arquivo que descreve o tipo de arquivo (usado pelas funções `S_IS*()` acima).

Normalmente, você usaria as funções `os.path.is*()` para testar o tipo de um arquivo; as funções aqui são úteis quando você está fazendo vários testes do mesmo arquivo e deseja evitar a sobrecarga da chamada de sistema `stat()` para cada teste. Eles também são úteis ao verificar informações sobre um arquivo que não é manipulado por *os.path*, como os testes para dispositivos de blocos e caracteres.

Exemplo:

```
import os, sys
from stat import *

def walktree(top, callback):
    '''desce recursivamente a árvore de diretórios com raiz no topo,
     chamando a função de retorno de chamada para cada arquivo regular'''

    for f in os.listdir(top):
        pathname = os.path.join(top, f)
        mode = os.lstat(pathname).st_mode
        if S_ISDIR(mode):
            # É um diretório, percorre recursivamente nele
            walktree(pathname, callback)
        elif S_ISREG(mode):
            # É um arquivo, chama a função de retorno
            callback(pathname)
        else:
            # Tipo de arquivo desconhecido, imprime uma mensagem
            print('Pulando %s' % pathname)

def visitfile(file):
    print('visitando, file)

if __name__ == '__main__':
    walktree(sys.argv[1], visitfile)
```

Uma função utilitária adicional é fornecida para converter o modo de um arquivo em uma string legível por humanos:

stat.**filemode**(*mode*)

Converte o modo de um arquivo em uma string no formato '-rwxrwxrwx'.

Adicionado na versão 3.3.

Alterado na versão 3.4: A função tem suporte a *S_IFDOOR*, *S_IFPORT* e *S_IFWHT*.

Todas as variáveis abaixo são simplesmente índices simbólicos nas 10 tuplas retornadas por *os.stat()*, *os.fstat()* ou *os.lstat()*.

stat.**ST_MODE**

Modo de proteção de nó-i.

stat.**ST_INO**
: Número de nó-i.

stat.**ST_DEV**
: Nó-i em que o dispositivo reside.

stat.**ST_NLINK**
: Número de links para o nó-i.

stat.**ST_UID**
: O ID de usuário para o proprietário.

stat.**ST_GID**
: O ID de grupo para o proprietário.

stat.**ST_SIZE**
: Tamanho em bytes de um arquivo simples; quantidade de dados aguardando em alguns arquivos especiais.

stat.**ST_ATIME**
: Hora do último acesso.

stat.**ST_MTIME**
: Hora da última modificação.

stat.**ST_CTIME**
: O "ctime" conforme relatado pelo sistema operacional. Em alguns sistemas (como Unix) é o horário da última alteração de metadados e, em outros (como Windows), é o horário de criação (consulte a documentação da plataforma para obter detalhes).

A interpretação do "tamanho do arquivo" muda de acordo com o tipo de arquivo. Para arquivos simples, este é o tamanho do arquivo em bytes. Para FIFOs e soquetes na maioria dos tipos de Unix (incluindo Linux em particular), o "tamanho" é o número de bytes aguardando para serem lidos no momento da chamada para *os.stat()*, *os.fstat()* ou *os.lstat()*; isso às vezes pode ser útil, especialmente para pesquisar um desses arquivos especiais após uma abertura sem bloqueio. O significado do campo de tamanho para outros dispositivos de caracteres e blocos varia mais, dependendo da implementação da chamada de sistema subjacente.

As variáveis abaixo definem os sinalizadores utilizadas no campo *ST_MODE*.

O uso das funções acima é mais portátil do que o uso do primeiro conjunto de sinalizadores:

stat.**S_IFSOCK**
: Soquete.

stat.**S_IFLNK**
: Link simbólico.

stat.**S_IFREG**
: Arquivo regular.

stat.**S_IFBLK**
: Dispositivo de bloco.

stat.**S_IFDIR**
: Diretório.

stat.**S_IFCHR**
: Dispositivo de caracteres.

stat.**S_IFIFO**
: FIFO.

stat.**S_IFDOOR**

Porta.

Adicionado na versão 3.4.

stat.**S_IFPORT**

Porta de eventos.

Adicionado na versão 3.4.

stat.**S_IFWHT**

Apagamento.

Adicionado na versão 3.4.

Nota

S_IFDOOR, *S_IFPORT* ou *S_IFWHT* são definidos como 0 quando a plataforma não possui suporte para os tipos de arquivo.

Os seguintes sinalizadores também podem ser usados no argumento *mode* de *os.chmod()*:

stat.**S_ISUID**

Define o bit de UID.

stat.**S_ISGID**

Bit de set-group-ID. Este bit tem vários usos especiais. Para um diretório, indica que a semântica BSD deve ser usada para esse diretório: os arquivos criados lá herdam seu ID de grupo do diretório, não do ID de grupo efetivo do processo de criação, e os diretórios criados lá também receberão o conjunto de bits *S_ISGID*. Para um arquivo que não possui o bit de execução de grupo (*S_IXGRP*) definido, o bit de set-group-ID indica trava obrigatória de arquivo/registro (veja também *S_ENFMT*).

stat.**S_ISVTX**

Sticky bit. Quando este bit é definido em um diretório, significa que um arquivo nesse diretório pode ser renomeado ou excluído apenas pelo proprietário do arquivo, pelo proprietário do diretório ou por um processo privilegiado.

stat.**S_IRWXU**

Máscara para permissões de proprietário de arquivo.

stat.**S_IRUSR**

Proprietário tem permissão de leitura.

stat.**S_IWUSR**

Proprietário tem permissão de escrita.

stat.**S_IXUSR**

Proprietário tem permissão de execução.

stat.**S_IRWXG**

Máscara para permissões de grupo.

stat.**S_IRGRP**

Grupo tem permissão de leitura.

stat.**S_IWGRP**

Grupo tem permissão de escrita.

stat.**S_IXGRP**

Grupo tem permissão de execução.

stat.**S_IRWXO**

Máscara para permissões para outros (não no grupo).

stat.**S_IROTH**

Outros têm permissão de leitura.

stat.**S_IWOTH**

Outros têm permissão de escrita.

stat.**S_IXOTH**

Outros têm permissão de execução.

stat.**S_ENFMT**

Aplicação de trava de arquivo do System V. Este sinalizador é compartilhada com *S_ISGID*: a trava de arquivo/registro é aplicada em arquivos que não possuem o bit de execução de grupo (*S_IXGRP*) definido.

stat.**S_IREAD**

Sinônimo Unix V7 para *S_IRUSR*.

stat.**S_IWRITE**

Sinônimo Unix V7 para *S_IWUSR*.

stat.**S_IEXEC**

Sinônimo Unix V7 para *S_IXUSR*.

Os seguintes sinalizadores podem ser usados no argumento *flags* de *os.chflags()*:

stat.**UF_SETTABLE**

Todos os sinalizadores definíveis pelo usuário

Adicionado na versão 3.13.

stat.**UF_NODUMP**

Não despeja o arquivo.

stat.**UF_IMMUTABLE**

O arquivo não pode ser alterado.

stat.**UF_APPEND**

O arquivo só pode sofrer acréscimos.

stat.**UF_OPAQUE**

O diretório é opaco quando visualizado por meio de uma pilha de união.

stat.**UF_NOUNLINK**

O arquivo não pode ser renomeado ou excluído.

stat.**UF_COMPRESSED**

O arquivo é armazenado compactado (macOS 10.6+).

stat.**UF_TRACKED**

Usado para manipular IDs de documentos (macOS)

Adicionado na versão 3.13.

stat.**UF_DATAVAULT**

O arquivo precisa de direito para leitura ou gravação (macOS 10.13+)

Adicionado na versão 3.13.

stat.**UF_HIDDEN**

O arquivo não deve ser exibido em uma GUI (macOS 10.5+).

stat.**SF_SETTABLE**

Todos os sinalizadores mutáveis de superusuário

Adicionado na versão 3.13.

stat.**SF_SUPPORTED**

Todos os sinalizadores válidos de superusuário

Disponibilidade: macOS

Adicionado na versão 3.13.

stat.**SF_SYNTHETIC**

Todos os sinalizadores sintéticos de somente leitura de superusuário

Disponibilidade: macOS

Adicionado na versão 3.13.

stat.**SF_ARCHIVED**

O arquivo não pode ser arquivado.

stat.**SF_IMMUTABLE**

O arquivo não pode ser alterado.

stat.**SF_APPEND**

O arquivo só pode sofrer acréscimos.

stat.**SF_RESTRICTED**

O arquivo precisa de um direito para ser escrito (macOS 10.13+)

Adicionado na versão 3.13.

stat.**SF_NOUNLINK**

O arquivo não pode ser renomeado ou excluído.

stat.**SF_SNAPSHOT**

O arquivo é um arquivo de captura (snapshot).

stat.**SF_FIRMLINK**

O arquivo é um firmlink (macOS 10.15+)

Adicionado na versão 3.13.

stat.**SF_DATALESS**

O arquivo é um objeto sem dados (macOS 10.15+)

Adicionado na versão 3.13.

Consulte a página man dos sistemas *BSD ou macOS *chflags(2)* para obter mais informações.

No Windows, as seguintes constantes de atributos de arquivo estão disponíveis para uso ao testar bits no membro `st_file_attributes` retornado por *os.stat()*. Consulte a documentação da API do Windows para obter mais detalhes sobre o significado dessas constantes.

stat.**FILE_ATTRIBUTE_ARCHIVE**

stat.**FILE_ATTRIBUTE_COMPRESSED**

stat.**FILE_ATTRIBUTE_DEVICE**

stat.**FILE_ATTRIBUTE_DIRECTORY**

stat.**FILE_ATTRIBUTE_ENCRYPTED**

stat.**FILE_ATTRIBUTE_HIDDEN**

stat.**FILE_ATTRIBUTE_INTEGRITY_STREAM**

stat.**FILE_ATTRIBUTE_NORMAL**

stat.**FILE_ATTRIBUTE_NOT_CONTENT_INDEXED**

stat.**FILE_ATTRIBUTE_NO_SCRUB_DATA**

stat.**FILE_ATTRIBUTE_OFFLINE**

stat.**FILE_ATTRIBUTE_READONLY**

stat.**FILE_ATTRIBUTE_REPARSE_POINT**

stat.**FILE_ATTRIBUTE_SPARSE_FILE**

stat.**FILE_ATTRIBUTE_SYSTEM**

stat.**FILE_ATTRIBUTE_TEMPORARY**

stat.**FILE_ATTRIBUTE_VIRTUAL**

Adicionado na versão 3.5.

No Windows, as seguintes constantes estão disponíveis para comparação com o membro `st_reparse_tag` retornado por *os.lstat()*. Estas são constantes bem conhecidas, mas não são uma lista exaustiva.

stat.**IO_REPARSE_TAG_SYMLINK**

stat.**IO_REPARSE_TAG_MOUNT_POINT**

stat.**IO_REPARSE_TAG_APPEXECLINK**

Adicionado na versão 3.8.

11.4 `filecmp` — Comparações de arquivos e diretórios

Código-fonte: Lib/filecmp.py

O módulo *filecmp* define funções para comparar arquivos e diretórios, com várias compensações opcionais de tempo/correção. Para comparar arquivos, veja também o módulo *difflib*.

O módulo *filecmp* define as seguintes funções:

filecmp.**cmp**(*f1*, *f2*, *shallow=True*)

Compara os arquivos chamados *f1* e *f2*, retornando `True` se eles parecerem iguais, `False` caso contrário.

Se *shallow* for verdadeiro e as assinaturas de *os.stat()* (tipo de arquivo, tamanho e hora de modificação) de ambos os arquivos forem idênticas, os arquivos serão considerados iguais.

Caso contrário, os arquivos serão tratados como diferentes se seus tamanhos ou conteúdos forem diferentes.

Observe que nenhum programa externo é chamado a partir desta função, o que lhe confere portabilidade e eficiência.

Esta função usa um cache para comparações passadas e os resultados, com entradas de cache invalidadas se as informações de *os.stat()* para o arquivo mudarem. O cache inteiro pode ser limpo usando *clear_cache()*.

filecmp.**cmpfiles**(*dir1*, *dir2*, *common*, *shallow=True*)

Compara os arquivos nos dois diretórios *dir1* e *dir2* cujos nomes são dados por *common*.

Retorna três listas de nomes de arquivos: *match*, *mismatch*, *errors*. *match* contém a lista de arquivos que correspondem, *mismatch* contém os nomes daqueles que não correspondem e *errors* lista os nomes dos arquivos que não puderam ser comparados. Os arquivos são listados em *errors* se não existirem em um dos diretórios, o usuário não tiver permissão para lê-los ou se a comparação não puder ser feita por algum outro motivo.

O parâmetro *shallow* tem o mesmo significado e valor padrão que *filecmp.cmp()*.

Por exemplo, `cmpfiles('a', 'b', ['c', 'd/e'])` comparará `a/c` com `b/c` e `a/d/e` com `b/d/e`. `'c'` e `'d/e'` estarão cada um em uma das três listas retornadas.

filecmp.**clear_cache**()

Limpa o cache do filecmp. Isso pode ser útil se um arquivo for comparado tão rapidamente após ser modificado que ele esteja dentro da resolução de mtime do sistema de arquivos subjacente.

Adicionado na versão 3.4.

11.4.1 A classe `dircmp`

class filecmp.**dircmp**(*a*, *b*, *ignore=None*, *hide=None*, *, *shallow=True*)

Constrói um novo objeto de comparação de diretórios para comparar os diretórios *a* e *b*. *ignore* é uma lista de nomes a serem ignorados e o padrão é *filecmp.DEFAULT_IGNORES*. *hide* é uma lista de nomes a serem ocultados e o padrão é `[os.curdir, os.pardir]`.

A classe *dircmp* compara arquivos fazendo comparações rasas, isto é, *shallow* conforme descrito para *filecmp.cmp()* por padrão usando o parâmetro *shallow*.

Alterado na versão 3.13: Adicionado o parâmetro *shallow*.

A classe *dircmp* fornece os seguintes métodos:

report()

Exibe (para *sys.stdout*) uma comparação entre *a* e *b*.

report_partial_closure()

Exibe uma comparação entre *a* e *b* e subdiretórios imediatos comuns.

report_full_closure()

Exibe uma comparação entre *a* e *b* e subdiretórios comuns (recursivamente).

A classe *dircmp* oferece uma série de atributos interessantes que podem ser usados para obter várias informações sobre as árvores de diretórios que estão sendo comparadas.

Observe que, por meio dos ganchos `__getattr__()`, todos os atributos são computados preguiçosamente, portanto, não há perda de velocidade se apenas os atributos que são leves para computar forem usados.

left

O diretório *a*.

right

O diretório *b*.

left_list

Arquivos e subdiretórios em *a*, filtrados por *hide* e *ignore*.

right_list

Arquivos e subdiretórios em *b*, filtrados por *hide* e *ignore*.

common

Arquivos e subdiretórios em *a* e *b*.

left_only

Arquivos e subdiretórios em apenas *a*.

right_only

Arquivos e subdiretórios em apenas *b*.

common_dirs

Subdiretórios em *a* e *b*.

common_files

Arquivos em *a* e *b*.

common_funny

Nomes em *a* e *b*, de modo que o tipo difere entre os diretórios, ou nomes para os quais *os.stat()* relata um erro.

same_files

Arquivos que são idênticos em *a* e *b*, usando o operador de comparação de arquivos da classe.

diff_files

Arquivos que estão em *a* e *b*, cujos conteúdos diferem de acordo com o operador de comparação de arquivos da classe.

funny_files

Arquivos que estão em *a* e *b*, mas não puderam ser comparados.

subdirs

Um dicionário que mapeia nomes em *common_dirs* para instâncias *dircmp* (ou instâncias MyDirCmp se esta instância for do tipo MyDirCmp, uma subclasse de *dircmp*).

Alterado na versão 3.10: Anteriormente, as entradas eram sempre instâncias de *dircmp*. Agora, as entradas são do mesmo tipo que *self*, se *self* for uma subclasse de *dircmp*.

filecmp.**DEFAULT_IGNORES**

Adicionado na versão 3.4.

Lista de diretórios ignorados por *dircmp* por padrão.

Aqui está um exemplo simplificado do uso do atributo `subdirs` para pesquisar recursivamente em dois diretórios para mostrar arquivos diferentes em comum:

```
>>> from filecmp import dircmp
>>> def print_diff_files(dcmp):
...     for name in dcmp.diff_files:
...         print("diff_file %s found in %s and %s" % (name, dcmp.left,
...               dcmp.right))
...     for sub_dcmp in dcmp.subdirs.values():
...         print_diff_files(sub_dcmp)
...
>>> dcmp = dircmp('dir1', 'dir2')
>>> print_diff_files(dcmp)
```

11.5 `tempfile` — Generate temporary files and directories

Código-fonte: Lib/tempfile.py

This module creates temporary files and directories. It works on all supported platforms. *TemporaryFile*, *NamedTemporaryFile*, *TemporaryDirectory*, and *SpooledTemporaryFile* are high-level interfaces which provide automatic cleanup and can be used as *context managers*. *mkstemp()* and *mkdtemp()* are lower-level functions which require manual cleanup.

All the user-callable functions and constructors take additional arguments which allow direct control over the location and name of temporary files and directories. Files names used by this module include a string of random characters which allows those files to be securely created in shared temporary directories. To maintain backward compatibility, the argument order is somewhat odd; it is recommended to use keyword arguments for clarity.

The module defines the following user-callable items:

tempfile.**TemporaryFile**(*mode='w+b'*, *buffering=-1*, *encoding=None*, *newline=None*, *suffix=None*, *prefix=None*, *dir=None*, ***, *errors=None*)

Return a *file-like object* that can be used as a temporary storage area. The file is created securely, using the same rules as *mkstemp()*. It will be destroyed as soon as it is closed (including an implicit close when the object is garbage collected). Under Unix, the directory entry for the file is either not created at all or is removed immediately after the file is created. Other platforms do not support this; your code should not rely on a temporary file created using this function having or not having a visible name in the file system.

The resulting object can be used as a *context manager* (see *Exemplos*). On completion of the context or destruction of the file object the temporary file will be removed from the filesystem.

The *mode* parameter defaults to `'w+b'` so that the file created can be read and written without being closed. Binary mode is used so that it behaves consistently on all platforms without regard for the data that is stored. *buffering*, *encoding*, *errors* and *newline* are interpreted as for `open()`.

The *dir*, *prefix* and *suffix* parameters have the same meaning and defaults as with `mkstemp()`.

The returned object is a true file object on POSIX platforms. On other platforms, it is a file-like object whose `file` attribute is the underlying true file object.

The `os.O_TMPFILE` flag is used if it is available and works (Linux-specific, requires Linux kernel 3.11 or later).

On platforms that are neither Posix nor Cygwin, TemporaryFile is an alias for NamedTemporaryFile.

Levanta um *evento de auditoria* `tempfile.mkstemp` com o argumento `fullpath`.

Alterado na versão 3.5: The `os.O_TMPFILE` flag is now used if available.

Alterado na versão 3.8: Parâmetro *errors* foi adicionado.

`tempfile.`**`NamedTemporaryFile`**(*mode='w+b'*, *buffering=-1*, *encoding=None*, *newline=None*, *suffix=None*, *prefix=None*, *dir=None*, *delete=True*, ***, *errors=None*, *delete_on_close=True*)

This function operates exactly as `TemporaryFile()` does, except the following differences:

- This function returns a file that is guaranteed to have a visible name in the file system.
- To manage the named file, it extends the parameters of `TemporaryFile()` with *delete* and *delete_on_close* parameters that determine whether and how the named file should be automatically deleted.

The returned object is always a *file-like object* whose `file` attribute is the underlying true file object. This file-like object can be used in a `with` statement, just like a normal file. The name of the temporary file can be retrieved from the `name` attribute of the returned file-like object. On Unix, unlike with the `TemporaryFile()`, the directory entry does not get unlinked immediately after the file creation.

If *delete* is true (the default) and *delete_on_close* is true (the default), the file is deleted as soon as it is closed. If *delete* is true and *delete_on_close* is false, the file is deleted on context manager exit only, or else when the *file-like object* is finalized. Deletion is not always guaranteed in this case (see `object.__del__()`). If *delete* is false, the value of *delete_on_close* is ignored.

Therefore to use the name of the temporary file to reopen the file after closing it, either make sure not to delete the file upon closure (set the *delete* parameter to be false) or, in case the temporary file is created in a `with` statement, set the *delete_on_close* parameter to be false. The latter approach is recommended as it provides assistance in automatic cleaning of the temporary file upon the context manager exit.

Opening the temporary file again by its name while it is still open works as follows:

- On POSIX the file can always be opened again.
- On Windows, make sure that at least one of the following conditions are fulfilled:
  - *delete* is false
  - additional open shares delete access (e.g. by calling `os.open()` with the flag `O_TEMPORARY`)
  - *delete* is true but *delete_on_close* is false. Note, that in this case the additional opens that do not share delete access (e.g. created via builtin `open()`) must be closed before exiting the context manager, else the `os.unlink()` call on context manager exit will fail with a `PermissionError`.

On Windows, if *delete_on_close* is false, and the file is created in a directory for which the user lacks delete access, then the `os.unlink()` call on exit of the context manager will fail with a `PermissionError`. This cannot happen when *delete_on_close* is true because delete access is requested by the open, which fails immediately if the requested access is not granted.

On POSIX (only), a process that is terminated abruptly with SIGKILL cannot automatically delete any NamedTemporaryFiles it created.

Levanta um *evento de auditoria* `tempfile.mkstemp` com o argumento `fullpath`.

Alterado na versão 3.8: Parâmetro *errors* foi adicionado.

Alterado na versão 3.12: Added *delete_on_close* parameter.

class tempfile.**SpooledTemporaryFile**(*max_size=0*, *mode='w+b'*, *buffering=-1*, *encoding=None*, *newline=None*, *suffix=None*, *prefix=None*, *dir=None*, ***, *errors=None*)

This class operates exactly as *TemporaryFile()* does, except that data is spooled in memory until the file size exceeds *max_size*, or until the file's *fileno()* method is called, at which point the contents are written to disk and operation proceeds as with *TemporaryFile()*.

rollover()

The resulting file has one additional method, `rollover()`, which causes the file to roll over to an on-disk file regardless of its size.

The returned object is a file-like object whose `_file` attribute is either an *io.BytesIO* or *io.TextIOWrapper* object (depending on whether binary or text *mode* was specified) or a true file object, depending on whether *rollover()* has been called. This file-like object can be used in a `with` statement, just like a normal file.

Alterado na versão 3.3: the truncate method now accepts a *size* argument.

Alterado na versão 3.8: Parâmetro *errors* foi adicionado.

Alterado na versão 3.11: Fully implements the *io.BufferedIOBase* and *io.TextIOBase* abstract base classes (depending on whether binary or text *mode* was specified).

class tempfile.**TemporaryDirectory**(*suffix=None*, *prefix=None*, *dir=None*, *ignore_cleanup_errors=False*, ***, *delete=True*)

This class securely creates a temporary directory using the same rules as *mkdtemp()*. The resulting object can be used as a *context manager* (see *Exemplos*). On completion of the context or destruction of the temporary directory object, the newly created temporary directory and all its contents are removed from the filesystem.

name

The directory name can be retrieved from the `name` attribute of the returned object. When the returned object is used as a *context manager*, the `name` will be assigned to the target of the `as` clause in the `with` statement, if there is one.

cleanup()

The directory can be explicitly cleaned up by calling the `cleanup()` method. If *ignore_cleanup_errors* is true, any unhandled exceptions during explicit or implicit cleanup (such as a *PermissionError* removing open files on Windows) will be ignored, and the remaining removable items deleted on a "best-effort" basis. Otherwise, errors will be raised in whatever context cleanup occurs (the `cleanup()` call, exiting the context manager, when the object is garbage-collected or during interpreter shutdown).

The *delete* parameter can be used to disable cleanup of the directory tree upon exiting the context. While it may seem unusual for a context manager to disable the action taken when exiting the context, it can be useful during debugging or when you need your cleanup behavior to be conditional based on other logic.

Levanta um *evento de auditoria* `tempfile.mkdtemp` com o argumento `fullpath`.

Adicionado na versão 3.2.

Alterado na versão 3.10: Added *ignore_cleanup_errors* parameter.

Alterado na versão 3.12: Added the *delete* parameter.

tempfile.**mkstemp**(*suffix=None*, *prefix=None*, *dir=None*, *text=False*)

Creates a temporary file in the most secure manner possible. There are no race conditions in the file's creation, assuming that the platform properly implements the *os.O_EXCL* flag for *os.open()*. The file is readable and writable only by the creating user ID. If the platform uses permission bits to indicate whether a file is executable, the file is executable by no one. The file descriptor is not inherited by child processes.

Unlike *TemporaryFile()*, the user of *mkstemp()* is responsible for deleting the temporary file when done with it.

If *suffix* is not `None`, the file name will end with that suffix, otherwise there will be no suffix. *mkstemp()* does not put a dot between the file name and the suffix; if you need one, put it at the beginning of *suffix*.

If *prefix* is not `None`, the file name will begin with that prefix; otherwise, a default prefix is used. The default is the return value of *gettempprefix()* or *gettempprefixb()*, as appropriate.

If *dir* is not `None`, the file will be created in that directory; otherwise, a default directory is used. The default directory is chosen from a platform-dependent list, but the user of the application can control the directory location by setting the *TMPDIR*, *TEMP* or *TMP* environment variables. There is thus no guarantee that the generated filename will have any nice properties, such as not requiring quoting when passed to external commands via `os.popen()`.

If any of *suffix*, *prefix*, and *dir* are not `None`, they must be the same type. If they are bytes, the returned name will be bytes instead of str. If you want to force a bytes return value with otherwise default behavior, pass `suffix=b''`.

If *text* is specified and true, the file is opened in text mode. Otherwise, (the default) the file is opened in binary mode.

mkstemp() returns a tuple containing an OS-level handle to an open file (as would be returned by *os.open()*) and the absolute pathname of that file, in that order.

Levanta um *evento de auditoria* `tempfile.mkstemp` com o argumento `fullpath`.

Alterado na versão 3.5: *suffix*, *prefix*, and *dir* may now be supplied in bytes in order to obtain a bytes return value. Prior to this, only str was allowed. *suffix* and *prefix* now accept and default to `None` to cause an appropriate default value to be used.

Alterado na versão 3.6: The *dir* parameter now accepts a *path-like object*.

`tempfile.`**`mkdtemp`**(*suffix=None*, *prefix=None*, *dir=None*)

Creates a temporary directory in the most secure manner possible. There are no race conditions in the directory's creation. The directory is readable, writable, and searchable only by the creating user ID.

The user of *mkdtemp()* is responsible for deleting the temporary directory and its contents when done with it.

The *prefix*, *suffix*, and *dir* arguments are the same as for *mkstemp()*.

mkdtemp() returns the absolute pathname of the new directory.

Levanta um *evento de auditoria* `tempfile.mkdtemp` com o argumento `fullpath`.

Alterado na versão 3.5: *suffix*, *prefix*, and *dir* may now be supplied in bytes in order to obtain a bytes return value. Prior to this, only str was allowed. *suffix* and *prefix* now accept and default to `None` to cause an appropriate default value to be used.

Alterado na versão 3.6: The *dir* parameter now accepts a *path-like object*.

Alterado na versão 3.12: *mkdtemp()* now always returns an absolute path, even if *dir* is relative.

`tempfile.`**`gettempdir`**()

Return the name of the directory used for temporary files. This defines the default value for the *dir* argument to all functions in this module.

Python searches a standard list of directories to find one which the calling user can create files in. The list is:

1. The directory named by the `TMPDIR` environment variable.
2. The directory named by the `TEMP` environment variable.
3. The directory named by the `TMP` environment variable.
4. Uma localização específica por plataforma:
   - On Windows, the directories `C:\TEMP`, `C:\TMP`, `\TEMP`, and `\TMP`, in that order.
   - On all other platforms, the directories `/tmp`, `/var/tmp`, and `/usr/tmp`, in that order.
5. As a last resort, the current working directory.

The result of this search is cached, see the description of *tempdir* below.

Alterado na versão 3.10: Always returns a str. Previously it would return any *tempdir* value regardless of type so long as it was not None.

tempfile.**gettempdirb**()

Same as *gettempdir()* but the return value is in bytes.

Adicionado na versão 3.5.

tempfile.**gettempprefix**()

Return the filename prefix used to create temporary files. This does not contain the directory component.

tempfile.**gettempprefixb**()

Same as *gettempprefix()* but the return value is in bytes.

Adicionado na versão 3.5.

The module uses a global variable to store the name of the directory used for temporary files returned by *gettempdir()*. It can be set directly to override the selection process, but this is discouraged. All functions in this module take a *dir* argument which can be used to specify the directory. This is the recommended approach that does not surprise other unsuspecting code by changing global API behavior.

tempfile.**tempdir**

When set to a value other than None, this variable defines the default value for the *dir* argument to the functions defined in this module, including its type, bytes or str. It cannot be a *path-like object*.

If tempdir is None (the default) at any call to any of the above functions except *gettempprefix()* it is initialized following the algorithm described in *gettempdir()*.

Nota

Beware that if you set tempdir to a bytes value, there is a nasty side effect: The global default return type of *mkstemp()* and *mkdtemp()* changes to bytes when no explicit prefix, suffix, or dir arguments of type str are supplied. Please do not write code expecting or depending on this. This awkward behavior is maintained for compatibility with the historical implementation.

11.5.1 Exemplos

Here are some examples of typical usage of the *tempfile* module:

```
>>> import tempfile

# create a temporary file and write some data to it
>>> fp = tempfile.TemporaryFile()
>>> fp.write(b'Hello world!')
# read data from file
>>> fp.seek(0)
>>> fp.read()
b'Hello world!'
# close the file, it will be removed
>>> fp.close()

# create a temporary file using a context manager
>>> with tempfile.TemporaryFile() as fp:
...     fp.write(b'Hello world!')
...     fp.seek(0)
...     fp.read()
b'Hello world!'
>>>
```

(continua na próxima página)

(continuação da página anterior)

```
# file is now closed and removed

# create a temporary file using a context manager
# close the file, use the name to open the file again
>>> with tempfile.NamedTemporaryFile(delete_on_close=False) as fp:
...     fp.write(b'Hello world!')
...     fp.close()
... # the file is closed, but not removed
... # open the file again by using its name
...     with open(fp.name, mode='rb') as f:
...         f.read()
b'Hello world!'
>>>
# file is now removed

# create a temporary directory using the context manager
>>> with tempfile.TemporaryDirectory() as tmpdirname:
...     print('created temporary directory', tmpdirname)
>>>
# directory and contents have been removed
```

11.5.2 Deprecated functions and variables

A historical way to create temporary files was to first generate a file name with the *mktemp()* function and then create a file using this name. Unfortunately this is not secure, because a different process may create a file with this name in the time between the call to *mktemp()* and the subsequent attempt to create the file by the first process. The solution is to combine the two steps and create the file immediately. This approach is used by *mkstemp()* and the other functions described above.

tempfile.**mktemp**(*suffix=''*, *prefix='tmp'*, *dir=None*)

Descontinuado desde a versão 2.3: Use *mkstemp()*.

Return an absolute pathname of a file that did not exist at the time the call is made. The *prefix*, *suffix*, and *dir* arguments are similar to those of *mkstemp()*, except that bytes file names, `suffix=None` and `prefix=None` are not supported.

Aviso

Use of this function may introduce a security hole in your program. By the time you get around to doing anything with the file name it returns, someone else may have beaten you to the punch. *mktemp()* usage can be replaced easily with *NamedTemporaryFile()*, passing it the `delete=False` parameter:

```
>>> f = NamedTemporaryFile(delete=False)
>>> f.name
'/tmp/tmptjujjt'
>>> f.write(b"Hello World!\n")
13
>>> f.close()
>>> os.unlink(f.name)
>>> os.path.exists(f.name)
False
```

11.6 `glob` — Expansão de padrão de nome de arquivo no estilo Unix

Código-fonte: Lib/glob.py

O módulo *glob* encontra todos os nomes de caminho que correspondem a um padrão especificado de acordo com as regras usadas pelo shell Unix, embora os resultados sejam retornados em ordem arbitrária. Nenhuma expansão de til é feita, mas `*`, `?` e os intervalos de caracteres expressos com `[]` serão correspondidos corretamente. Isso é feito usando as funções *os.scandir()* e *fnmatch.fnmatch()* em conjunto, e não invocando realmente um subshell.

Observe que arquivos iniciados com um ponto (`.`) só podem ser correspondidos com padrões que também iniciam com um ponto, ao contrário de *fnmatch.fnmatch()* ou *pathlib.Path.glob()*. (Para expansão de til e variável de shell, use *os.path.expanduser()* e *os.path.expandvars()*.)

Para uma correspondência literal, coloque os metacaracteres entre colchetes. Por exemplo, `'[?]'` corresponde ao caractere `'?'`.

O módulo *glob* define as seguintes funções:

`glob.`**`glob`**(*pathname*, *, *root_dir=None*, *dir_fd=None*, *recursive=False*, *include_hidden=False*)

Retorna uma lista possivelmente vazia de nomes de caminho que correspondem a *pathname*, que deve ser uma string contendo uma especificação de caminho. *pathname* pode ser absoluto (como `/usr/src/Python-1.5/Makefile`) ou relativo (como `../../Tools/*/*.gif`) e pode conter curingas no estilo shell. Links simbólicos quebrados são incluídos nos resultados (como no shell). Se os resultados são classificados ou não depende do sistema de arquivos. Se um arquivo que satisfaz as condições for removido ou adicionado durante a chamada desta função, não é especificado se um nome de caminho para esse arquivo será incluído.

Se *root_dir* não for `None`, deve ser um *objeto caminho ou similar* especificando o diretório raiz para pesquisa. Tem o mesmo efeito em *glob()* que alterar o diretório atual antes de chamá-lo. Se *pathname* for relativo, o resultado conterá caminhos relativos a *root_dir*.

Esta função oferece suporte para *caminhos relativos aos descritores de diretório* com o parâmetro *dir_fd*.

Se *recursive* for verdadeiro, o padrão "`**`" corresponderá a qualquer arquivo e zero ou mais diretórios, subdiretórios e links simbólicos para diretórios. Se o padrão for seguido por um *os.sep* ou *os.altsep*, então os arquivos não irão corresponder.

Se *include_hidden* for verdadeiro, o padrão "`**`" corresponderá aos diretórios ocultos.

Levanta um *evento de auditoria* `glob.glob` com os argumentos `pathname`, `recursive`.

Levanta um *evento de auditoria* `glob.glob/2` com os argumentos `pathname`, `recursive`, `root_dir`, `dir_fd`.

> **Nota**
>
> Usar o padrão "`**`" em grandes árvores de diretório pode consumir uma quantidade excessiva de tempo.

> **Nota**
>
> Esta função pode retornar nomes de caminhos duplicados se *pathname* contiver vários padrões "`**`" e *recursive* for verdadeiro.

Alterado na versão 3.5: Suporte a globs recursivos usando "`**`".

Alterado na versão 3.10: Adicionados os parâmetros *root_dir* e *dir_fd*.

Alterado na versão 3.11: Adicionado o parâmetro *include_hidden*.

glob.**iglob**(*pathname*, *, *root_dir=None*, *dir_fd=None*, *recursive=False*, *include_hidden=False*)

Retorna um *iterador* que produz os mesmos valores que `glob()` sem realmente armazená-los todos simultaneamente.

Levanta um *evento de auditoria* `glob.glob` com os argumentos `pathname`, `recursive`.

Levanta um *evento de auditoria* `glob.glob/2` com os argumentos `pathname`, `recursive`, `root_dir`, `dir_fd`.

> **Nota**
>
> Esta função pode retornar nomes de caminhos duplicados se *pathname* contiver vários padrões "`**`" e *recursive* for verdadeiro.

Alterado na versão 3.5: Suporte a globs recursivos usando "`**`".

Alterado na versão 3.10: Adicionados os parâmetros *root_dir* e *dir_fd*.

Alterado na versão 3.11: Adicionado o parâmetro *include_hidden*.

glob.**escape**(*pathname*)

Escapa todos os caracteres especiais (`'?'`, `'*'` e `'['`). Isso é útil se você deseja corresponder a uma string literal arbitrária que pode conter caracteres especiais. Os caracteres especiais nos pontos de compartilhamento de unidade/UNC não têm escape, por exemplo, no Windows `escape('//?/c:/Quo vadis?.txt')` retorna `'//?/c:/Quo vadis[?].txt'`.

Adicionado na versão 3.4.

glob.**translate**(*pathname*, *, *recursive=False*, *include_hidden=False*, *seps=None*)

Converte a especificação de caminho dada para uma expressão regular para uso com `re.match()`. A especificação de caminho pode conter curingas no estilo shell.

Por exemplo:

```
>>> import glob, re
>>>
>>> regex = glob.translate('**/*.txt', recursive=True, include_hidden=True)
>>> regex
'(?s:(?:.+/)?[^/]*\\.txt)\\z'
>>> reobj = re.compile(regex)
>>> reobj.match('foo/bar/baz.txt')
<re.Match object; span=(0, 15), match='foo/bar/baz.txt'>
```

Separadores de caminho e segmentos são significativos para esta função, diferentemente de `fnmatch.translate()`. Por padrão, curingas não correspondem a separadores de caminho, e segmentos de padrão `*` correspondem precisamente a um segmento de caminho.

Se *recursive* for verdadeiro, o segmento de padrão "`**`" corresponderá a qualquer número de segmentos de caminho.

Se *include_hidden* for verdadeiro, os curingas poderão corresponder a segmentos de caminho que começam com um ponto (`.`).

Uma sequência de separadores de caminho pode ser fornecida ao argumento *seps*. Se não for fornecido, `os.sep` e `altsep` (se disponível) são usados.

> **Ver também**
>
> Os métodos `pathlib.PurePath.full_match()` e `pathlib.Path.glob()`, que chamam esta função para implementar correspondência de padrões e globbing.

Adicionado na versão 3.13.

11.6.1 Exemplos

Considere um diretório contendo os seguintes arquivos: `1.gif`, `2.txt`, `card.gif` e um subdiretório `sub` que contém apenas o arquivo `3.txt`. *glob()* produzirá os seguintes resultados. Observe como todos os componentes principais do caminho são preservados.

```
>>> import glob
>>> glob.glob('./[0-9].*')
['./1.gif', './2.txt']
>>> glob.glob('*.gif')
['1.gif', 'card.gif']
>>> glob.glob('?.gif')
['1.gif']
>>> glob.glob('**/*.txt', recursive=True)
['2.txt', 'sub/3.txt']
>>> glob.glob('./**/', recursive=True)
['./', './sub/']
```

Se o diretório contém arquivos começando com `.` eles não serão correspondidos por padrão. Por exemplo, considere um diretório contendo `card.gif` e `.card.gif`

```
>>> import glob
>>> glob.glob('*.gif')
['card.gif']
>>> glob.glob('.c*')
['.card.gif']
```

Ver também

O módulo *fnmatch* oferece expansão de nome de arquivo (não caminho) no estilo shell.

Ver também

O módulo *pathlib* oferece objetos de caminho de alto nível.

11.7 `fnmatch` — Correspondência de padrões de nome de arquivo Unix

Código-fonte: Lib/fnmatch.py

Este módulo fornece suporte para curingas no estilo shell do Unix, que *não* são iguais às expressões regulares (documentadas no módulo *re*). Os caracteres especiais usados nos curingas no estilo de shell são:

Padrão	Significado
`*`	corresponde a tudo
`?`	Corresponde a qualquer caractere único
`[seq]`	corresponde a qualquer caractere em *seq*
`[!seq]`	corresponde a qualquer caractere ausente em *seq*

Para uma correspondência literal, coloque os metacaracteres entre colchetes. Por exemplo, `'[?]'` corresponde ao caractere `'?'`.

Note que o separador de nome de arquivo (`'/'` no Unix) *não* é especial para este módulo. Veja o módulo *glob* para expansão do nome do caminho (*glob* usa *filter()* para corresponder aos segmentos do nome do caminho). Da mesma forma, os nomes de arquivos que começam com um ponto final não são especiais para este módulo e são correspondidos pelos padrões `*` e `?`.

A menos que indicado de outra forma, "string de nome de arquivo" e "string de padrão" referem-se a objetos *str* ou *bytes* codificados em `ISO-8859-1`. Observe que as funções documentadas abaixo não permitem misturar um padrão `bytes` com um nome de arquivo `str` e vice-versa.

Por fim, observe que *functools.lru_cache()* com um *maxsize* de 32768 é usado para armazenar em cache os padrões regex compilados (tipados) nas seguintes funções: *fnmatch()*, *fnmatchcase()*, *filter()*, *filterfalse()*.

`fnmatch.`**`fnmatch`**(*name*, *pat*)

Testa se a string do nome de arquivo *name* corresponde à string de padrão *pat*, retornando `True` ou `False`. Ambos os parâmetros são normalizados em maiúsculas e minúsculas usando *os.path.normcase()*. *fnmatchcase()* pode ser usado para realizar uma comparação com distinção entre maiúsculas e minúsculas, independentemente de ser padrão para o sistema operacional.

Este exemplo vai exibir todos os nomes de arquivos no diretório atual com a extensão `.txt`:

```
import fnmatch
import os

for arquivo in os.listdir('.'):
    if fnmatch.fnmatch(arquivo, '*.txt'):
        print(arquivo)
```

`fnmatch.`**`fnmatchcase`**(*name*, *pat*)

Testa se a string do nome de arquivo *name* corresponde à string de padrão *pat*, retornando `True` ou `False`; a comparação diferencia maiúsculas de minúsculas e não se aplica a *os.path.normcase()*.

`fnmatch.`**`filter`**(*names*, *pat*)

Constrói uma lista a partir daqueles elementos do *iterável* de strings de nome de arquivo *names* que correspondem à string de padrão *pat*. É o mesmo que `[n for n in names if fnmatch(n, pat)]`, mas implementado com mais eficiência.

`fnmatch.`**`filterfalse`**(*names*, *pat*)

Constrói uma lista a partir daqueles elementos do *iterável* de strings de nome de arquivo *names* que não correspondem à string de padrão *pat*. É o mesmo que `[n for n in names if not fnmatch(n, pat)]`, mas implementado com mais eficiência.

Adicionado na versão 3.14.

`fnmatch.`**`translate`**(*pat*)

Retorna o padrão *pat* no estilo shell convertido em uma expressão regular para usar com *re.match()*. O padrão é esperado ser um *str*.

Exemplo:

```
>>> import fnmatch, re
>>>
>>> regex = fnmatch.translate('*.txt')
>>> regex
'(?s:.*\\.txt)\\z'
>>> reobj = re.compile(regex)
>>> reobj.match('foobar.txt')
<re.Match object; span=(0, 10), match='foobar.txt'>
```

Ver também

Módulo *glob*
Expansão de caminho no estilo shell do Unix.

11.8 linecache — Acesso aleatório a linhas de texto

Código-fonte: Lib/linecache.py

O módulo *linecache* permite obter qualquer linha de um arquivo fonte Python, enquanto tenta otimizar internamente, usando um cache, o caso comum em que muitas linhas são lidas em um único arquivo. Isso é usado pelo módulo *traceback* para recuperar as linhas de origem para inclusão no traceback (situação da pilha de execução) formatado.

A função *tokenize.open()* é usada para abrir arquivos. Esta função usa *tokenize.detect_encoding()* para obter a codificação do arquivo; na ausência de um token de codificação, o padrão de codificação do arquivo é UTF-8.

O módulo *linecache* define as seguintes funções:

linecache.**getline**(*filename*, *lineno*, *module_globals=None*)

Obtém a linha *lineno* do arquivo chamado *filename*. Essa função nunca levanta uma exceção — ela retornará `''` em erros (o caractere de nova linha final será incluído para as linhas encontradas).

Se *filename* indicar um módulo congelado (começando com `'<frozen '`), a função tentará obter o nome real do arquivo de `module_globals['__file__']` se *module_globals* não for `None`.

Se um arquivo chamado *filename* não for encontrado, a função primeiro verifica por um `__loader__` da **PEP 302** em *module_globals*. Se existe um carregador e ele define um método `get_source`, isso determina as linhas fonte (se `get_source()` retornar `None`, então `''` será retornado). Por fim, se *filename* for um nome de arquivo relativo, ele será procurado em relação às entradas no caminho de pesquisa do módulo, `sys.path`.

Alterado na versão 3.14: Suporte a *filename* de módulos congelados.

linecache.**clearcache**()

Limpa o cache. Use esta função se você não precisar mais de linhas de arquivos lidos anteriormente usando *getline()*.

linecache.**checkcache**(*filename=None*)

Verifica a validade do cache. Use esta função se os arquivos no cache tiverem sido alterados no disco e você precisar da versão atualizada. Se *filename* for omitido, ele verificará todas as entradas no cache.

linecache.**lazycache**(*filename*, *module_globals*)

Captura detalhes suficientes sobre um módulo não baseado em arquivo para permitir obter suas linhas posteriormente via *getline()* mesmo se *module_globals* for `None` na chamada posterior. Isso evita a execução de E/S até que uma linha seja realmente necessária, sem ter que carregar o módulo global indefinidamente.

Adicionado na versão 3.5.

Exemplo:

```
>>> import linecache
>>> linecache.getline(linecache.__file__, 8)
'import sys\n'
```

11.9 shutil — Operações de arquivo de alto nível

Código-fonte: Lib/shutil.py

O módulo *shutil* oferece várias operações de alto nível em arquivos e coleções de arquivos. Em particular, são fornecidas funções que possuem suporte a cópia e remoção de arquivos. Para operações em arquivos individuais, veja também o módulo *os*.

Aviso

Mesmo as funções de cópia de arquivos de nível mais alto (*shutil.copy()*, *shutil.copy2()*) não podem copiar todos os metadados do arquivo.

Nas plataformas POSIX, isso significa que o proprietário e o grupo do arquivo são perdidos, bem como as ACLs. No Mac OS, a bifurcação de recursos e outros metadados não são usados. Isso significa que os recursos serão perdidos e o tipo de arquivo e os códigos do criador não estarão corretos. No Windows, os proprietários de arquivos, ACLs e fluxos de dados alternativos não são copiados.

11.9.1 Operações de diretório e arquivos

shutil.**copyfileobj**(*fsrc*, *fdst*[, *length*])

Copia o conteúdo do *objeto arquivo ou similar* *fsrc* para o objeto arquivo ou similar *fdst*. O número inteiro *length*, se fornecido, é o tamanho do buffer. Em particular, um valor negativo *length* significa copiar os dados sem repetir os dados de origem em pedaços; por padrão, os dados são lidos em pedaços para evitar o consumo descontrolado de memória. Observe que, se a posição atual do arquivo do objeto *fsrc* não for 0, apenas o conteúdo da posição atual do arquivo até o final do arquivo será copiado.

copyfileobj() *não* garante que o fluxo de destino tenha sido liberado após a conclusão da cópia. Se você quiser ler do destino após a conclusão da operação de cópia (por exemplo, ler o conteúdo de um arquivo temporário copiado de um fluxo HTTP), certifique-se de ter chamado *flush()* ou *close()* no objeto arquivo ou similar antes de tentar ler o arquivo de destino.

shutil.**copyfile**(*src*, *dst*, *, *follow_symlinks=True*)

Copia o conteúdo (sem metadados) do arquivo chamado *src* para um arquivo chamado *dst* e retorna *dst* da maneira mais eficiente possível. *src* e *dst* são *objetos caminho ou similares* ou nomes de caminhos dados como strings.

dst deve ser o nome completo do arquivo de destino; veja *copy()* para uma cópia que aceita um caminho de diretório de destino. Se *src* e *dst* especificarem o mesmo arquivo, *SameFileError* será levantada.

O local de destino deve ser gravável; caso contrário, uma exceção *OSError* será levantada. Se o *dst* já existir, ele será substituído. Arquivos especiais como dispositivos de caractere ou bloco e encadeamentos (pipe) não podem ser copiados com esta função.

Se *follow_symlinks* for falso e *src* for um link simbólico, um novo link simbólico será criado em vez de copiar o arquivo *src* para o qual o arquivo aponta.

Levanta um *evento de auditoria* `shutil.copyfile` com os argumentos `src`, `dst`.

Alterado na versão 3.3: *IOError* costumava ser levantada em vez de *OSError*. Adicionado argumento *follow_symlinks*. Agora retorna *dst*.

Alterado na versão 3.4: Levanta *SameFileError* em vez de *Error*. Como a primeira é uma subclasse da última, essa alteração é compatível com versões anteriores.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção *Operações de cópia eficientes dependentes da plataforma*.

exception shutil.**SpecialFileError**

Esta exceção é levantada quando *copyfile()* ou *copytree()* tenta copiar um pipe nomeado.

Adicionado na versão 2.7.

exception shutil.**SameFileError**

Essa exceção é levantada se a origem e o destino em *copyfile()* forem o mesmo arquivo.

Adicionado na versão 3.4.

shutil.**copymode**(*src*, *dst*, *, *follow_symlinks=True*)

Copia os bits de permissão de *src* para *dst*. O conteúdo do arquivo, o proprietário e o grupo não são afetados. *src* e *dst* são *objetos caminho ou similares* ou nomes de caminhos dados como strings. Se *follow_symlinks* for falso e *src* e *dst* forem links simbólicos, `copymode()` tentará modificar o modo do *dst* (e não o arquivo para o qual ele aponta). Essa funcionalidade não está disponível em todas as plataformas; veja `copystat()` para mais informações. Se `copymode()` não puder modificar links simbólicos na plataforma local, e for solicitado a fazê-lo, não fará nada e retornará.

Levanta um *evento de auditoria* `shutil.copymode` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionado argumento *follow_symlinks*.

shutil.**copystat**(*src*, *dst*, *, *follow_symlinks=True*)

Copia os bits de permissão, hora do último acesso, hora da última modificação e sinalizadores de *src* para *dst*. No Linux, `copystat()` também copia os "atributos estendidos" sempre que possível. O conteúdo do arquivo, o proprietário e o grupo não são afetados. *src* e *dst* são *objetos caminho ou similares* ou nomes de caminhos dados como strings.

Se *follow_symlinks* for falso e *src* e *dst* se referirem a links simbólicos, `copystat()` operará nos próprios links simbólicos, e não nos arquivos aos quais os links simbólicos se referem - lendo as informações do link simbólico *src* e gravando as informações no link simbólico *dst*.

Nota

Nem todas as plataformas oferecem a capacidade de examinar e modificar links simbólicos. O próprio Python pode dizer qual funcionalidade está disponível localmente.

- Se `os.chmod in os.supports_follow_symlinks` for `True`, `copystat()` pode modificar os bits de permissão de um link simbólico.
- Se `os.utime in os.supports_follow_symlinks` for `True`, `copystat()` pode modificar as horas da última modificação e do último acesso de um link simbólico.
- Se `os.chflags in os.supports_follow_symlinks` for `True`, `copystat()` pode modificar os sinalizadores de um link simbólico. (`os.chflags` não está disponível em todas as plataformas.)

Nas plataformas em que algumas ou todas essas funcionalidades não estão disponíveis, quando solicitado a modificar um link simbólico, `copystat()` copiará tudo o que puder. `copystat()` nunca retorna falha.

Por favor, veja `os.supports_follow_symlinks` para mais informações.

Levanta um *evento de auditoria* `shutil.copystat` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionado argumento *follow_symlinks* e suporte a atributos estendidos do Linux.

shutil.**copy**(*src*, *dst*, *, *follow_symlinks=True*)

Copia o arquivo *src* para o arquivo ou diretório *dst*. *src* e *dst* devem ser *objetos arquivo ou similares* ou strings. Se *dst* especificar um diretório, o arquivo será copiado para *dst* usando o nome de arquivo base de *src*. Se *dst* especifica um arquivo que já existe, ele será substituído. Retorna o caminho para o arquivo recém-criado.

Se *follow_symlinks* for falso e *src* for um link simbólico, *dst* será criado como um link simbólico. Se *follow_symlinks* for verdadeiro e *src* for um link simbólico, *dst* será uma cópia do arquivo ao qual *src* se refere.

`copy()` copia os dados do arquivo e o modo de permissão do arquivo (consulte `os.chmod()`). Outros metadados, como os tempos de criação e modificação do arquivo, não são preservados. Para preservar todos os metadados do arquivo do original, use `copy2()`.

Levanta um *evento de auditoria* `shutil.copyfile` com os argumentos `src`, `dst`.

Levanta um *evento de auditoria* `shutil.copymode` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionado argumento *follow_symlinks*. Agora retorna o caminho para o arquivo recém-criado.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção *Operações de cópia eficientes dependentes da plataforma*.

shutil.**copy2**(*src*, *dst*, *, *follow_symlinks=True*)

Idêntico a `copy()`, exceto que `copy2()` também tenta preservar os metadados do arquivo.

Quando *follow_symlinks* for falso e *src* for um link simbólico, `copy2()` tenta copiar todos os metadados do link simbólico *src* para o link simbólico *dst* recém-criado. No entanto, essa funcionalidade não está disponível em todas as plataformas. Nas plataformas em que algumas ou todas essas funcionalidades não estão disponíveis, `copy2()` preservará todos os metadados que puder; `copy2()` nunca levanta uma exceção porque não pode preservar os metadados do arquivo.

`copy2()` usa `copystat()` para copiar os metadados do arquivo. Por favor, veja `copystat()` para obter mais informações sobre o suporte da plataforma para modificar os metadados do link simbólico.

Levanta um *evento de auditoria* `shutil.copyfile` com os argumentos `src`, `dst`.

Levanta um *evento de auditoria* `shutil.copystat` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionado argumento *follow_symlinks*, tenta copiar também atributos estendidos do sistema de arquivos (atualmente apenas no Linux). Agora retorna o caminho para o arquivo recém-criado.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção *Operações de cópia eficientes dependentes da plataforma*.

shutil.**ignore_patterns**(**patterns*)

Esta função de fábrica cria uma função que pode ser usada como um chamável para o argumento *ignore* de `copytree()`, ignorando arquivos e diretórios que correspondem a um dos padrões *patterns* de estilo glob fornecidos. Veja o exemplo abaixo.

shutil.**copytree**(*src*, *dst*, *symlinks=False*, *ignore=None*, *copy_function=copy2*, *ignore_dangling_symlinks=False*, *dirs_exist_ok=False*)

Copia recursivamente uma árvore de diretórios inteira com raiz em *src* para um diretório chamado *dst* e retorna o diretório de destino. Todos os diretórios intermediários necessários para conter *dst* também serão criados por padrão.

Permissões e horas dos diretórios são copiados com `copystat()`, arquivos individuais são copiados usando `copy2()`.

Se *symlinks* for verdadeiro, os links simbólicos na árvore de origem são representados como links simbólicos na nova árvore e os metadados dos links originais serão copiados na medida do permitido pela plataforma; se falso ou omitido, o conteúdo e os metadados dos arquivos vinculados são copiados para a nova árvore.

Quando *symlinks* for falso, se o arquivo apontado pelo link simbólico não existir, uma exceção será adicionada na lista de erros levantada em uma exceção `Error` no final do processo de cópia. Você pode definir o sinalizador opcional *ignore_dangling_symlinks* como true se desejar silenciar esta exceção. Observe que esta opção não tem efeito em plataformas que não possuem suporte a `os.symlink()`.

Se *ignore* for fornecido, deve ser um chamável que receberá como argumento o diretório que está sendo visitado por `copytree()`, e uma lista de seu conteúdo, retornada por `os.listdir()`. Como `copytree()` é chamada recursivamente, o chamável *ignore* será chamado uma vez para cada diretório que é copiado. O chamável deve retornar uma sequência de nomes de diretório e arquivo em relação ao diretório atual (ou seja, um subconjunto dos itens em seu segundo argumento); esses nomes serão ignorados no processo de cópia. `ignore_patterns()` pode ser usado para criar um chamável que ignore nomes com base em padrões de estilo glob.

Se uma ou mais exceções ocorrerem, uma `Error` é levantada com uma lista dos motivos.

Se *copy_function* for fornecida, deverá ser um chamável que será usado para copiar cada arquivo. Ele será chamado com o caminho de origem e o caminho de destino como argumentos. Por padrão, `copy2()` é usada, mas qualquer função que possua suporte à mesma assinatura (como `copy()`) pode ser usada.

Se *dirs_exist_ok* for false (o padrão) e *dst* já existir, uma `FileExistsError` será levantada. Se *dirs_exist_ok* for true, a operação de cópia continuará se encontrar diretórios existentes, e os arquivos dentro da árvore *dst* serão sobrescritos pelos arquivos correspondentes da árvore *src*.

Levanta um *evento de auditoria* `shutil.copytree` com os argumentos `src`, `dst`.

Alterado na versão 3.2: Adicionado o argumento *copy_function* para poder fornecer uma função de cópia personalizada. Adicionado o argumento *ignore_dangling_symlinks* para silenciar erros de links simbólicos quando *symlinks* for falso.

Alterado na versão 3.3: Copia metadados quando *symlinks* for falso. Agora, retorna *dst*.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção *Operações de cópia eficientes dependentes da plataforma*.

Alterado na versão 3.8: Adicionado o parâmetro *dirs_exist_ok*.

`shutil.`**`rmtree`**(*path*, *ignore_errors=False*, *onerror=None*, *, *onexc=None*, *dir_fd=None*)

Exclui uma árvore de diretório inteira; *path* deve apontar para um diretório (mas não um link simbólico para um diretório). Se *ignore_errors* for verdadeiro, os erros resultantes de remoções com falha serão ignorados; se falso ou omitido, tais erros são tratados chamando um manipulador especificado por *onexc* ou *onerror* ou, se ambos forem omitidos, exceções são propagadas para o chamador.

Esta função tem suporte a *caminhos relativos para descritores de diretório*.

Nota

Em plataformas que suportam as funções baseadas em descritores de arquivo necessárias, uma versão resistente a ataques de links simbólicos de `rmtree()` é usada por padrão. Em outras plataformas, a implementação `rmtree()` é suscetível a um ataque de link simbólico: dados o tempo e as circunstâncias apropriados, os invasores podem manipular links simbólicos no sistema de arquivos para excluir arquivos que eles não seriam capazes de acessar de outra forma. Os aplicativos podem usar o atributo de função `rmtree.avoids_symlink_attacks` para determinar qual caso se aplica.

Se *onexc* for fornecido, deve ser um chamável que aceite três parâmetros: *function*, *path*, e *excinfo*.

O primeiro parâmetro, *function*, é a função que levantou a exceção; depende da plataforma e da implementação. O segundo parâmetro, *path*, será o nome do caminho passado para a *função*. O terceiro parâmetro, *excinfo*, é a exceção que foi levantada. Exceções levantadas por *onexc* não serão capturadas.

O descontinuado *onerror* é semelhante ao *onexc*, exceto que o terceiro parâmetro que ele recebe é a tupla retornada de `sys.exc_info()`.

Ver também

exemplo rmtree para um exemplo de tratamento da remoção de uma árvore de diretórios que contém arquivos somente leitura.

Levanta um *evento de auditoria* `shutil.rmtree` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.3: Adicionada uma versão resistente a ataques de link simbólico que é usada automaticamente se a plataforma suportar funções baseadas em descritor de arquivo.

Alterado na versão 3.8: No Windows, não excluirá mais o conteúdo de uma junção de diretório antes de remover a junção.

Alterado na versão 3.11: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.12: Adicionado o parâmetro *onexc*, descontinuado *onerror*.

Alterado na versão 3.13: `rmtree()` agora ignora exceções *FileNotFoundError* para todos, exceto o caminho de nível superior. Exceções diferentes de *OSError* e subclasses de `OSError` agora são sempre propagadas para o chamador.

`rmtree.`**`avoids_symlink_attacks`**

Indica se a plataforma e implementação atuais fornecem uma versão resistente a ataques de link simbólico de *rmtree()*. Atualmente, isso só é verdade para plataformas que suportam funções de acesso ao diretório baseadas em descritor de arquivo.

Adicionado na versão 3.3.

`shutil.`**`move`**(*src*, *dst*, *copy_function=copy2*)

Move recursivamente um arquivo ou diretório (*src*) para outro local e retorna ao destino.

Se *dst* for um diretório existente ou um symlink para um diretório, então *src* é movido para dentro desse diretório. O caminho de destino nesse diretório não deve existir ainda.

Se *dst* já existir, mas não for um diretório, ele poderá ser substituído dependendo da semântica de *os.rename()*.

Se o destino está no sistema de arquivos atual, então *os.rename()* é usado. Caso contrário, *src* é copiado para o destino usando *copy_function* e depois removido. No caso de links simbólicos, um novo link simbólico apontando para o destino de *src* será criado como o destino e *src* serão removidos.

Se *copy_function* for fornecido, deve ser um chamável que leva dois argumentos *src* e a destinação, e será usado para copiar *src* para a destinação se *os.rename()* não puder ser usado. Se a fonte for um diretório, *copytree()* é chamado, passando *copy_function*. A *copy_function* padrão é *copy2()*. Usar *copy()* como *copy_function* permite que a movimentação seja bem-sucedida quando não é possível copiar também os metadados, às custas de não copiar nenhum dos metadados.

Levanta um *evento de auditoria* `shutil.move` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionada manipulação de links simbólicos explícitos para sistemas de arquivos externos, adaptando-os ao comportamento do GNU **`mv`**. Agora retorna *dst*.

Alterado na versão 3.5: Adicionado o argumento nomeado *copy_function*.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção *Operações de cópia eficientes dependentes da plataforma*.

Alterado na versão 3.9: Aceita um *objeto caminho ou similar* para *src* e *dst*.

`shutil.`**`disk_usage`**(*path*)

Retorna estatísticas de uso de disco sobre o caminho fornecido como *tupla nomeada* com os atributos *total*, *used* e *free*, que são a quantidade de espaço total, usado e livre, em bytes. *path* pode ser um arquivo ou diretório.

> **Nota**
>
> Em sistemas de arquivos Unix, *path* deve apontar para um caminho dentro de uma partição de sistema de arquivos **montada**. Nessas plataformas, o CPython não tenta recuperar informações de uso de disco de sistemas de arquivos não montados.

Adicionado na versão 3.3.

Alterado na versão 3.8: No Windows, *path* pode agora ser um arquivo ou diretório.

Disponibilidade: Unix, Windows.

`shutil.`**`chown`**(*path*, *user=None*, *group=None*, ***, *dir_fd=None*, *follow_symlinks=True*)

Altera o proprietário *usuário* e/ou *group* do *path* fornecido.

user pode ser um nome de usuário do sistema ou um uid; o mesmo se aplica ao *group*. É necessário pelo menos um argumento.

Veja também *os.chown()*, a função subjacente.

Levanta um *evento de auditoria* `shutil.chown` com os argumentos `path`, `user`, `group`.

Disponibilidade: Unix.

Adicionado na versão 3.3.

Alterado na versão 3.13: Adicionados os parâmetros *dir_fd* e *follow_symlinks*.

`shutil.`**`which`**(*cmd*, *mode=os.F_OK | os.X_OK*, *path=None*)

Retorna o caminho para um executável que seria executado se o *cmd* fornecido fosse chamado. Se nenhum *cmd* for chamado, retorna `None`.

mode é uma máscara de permissão passada para *os.access()*, por padrão determinando se o arquivo existe e é executável.

path é uma "string `PATH`" que especifica os diretórios a serem procurados, delimitados por *os.pathsep*. Quando nenhum *path* é especificado, a variável de ambiente `PATH` é lida de *os.environ*, retornando para *os.defpath* se não estiver definida.

Se *cmd* contiver um componente de diretório, `which()` verificará apenas o caminho especificado diretamente e não pesquisará os diretórios listados em *path* ou na variável de ambiente `PATH` do sistema.

No Windows, o diretório atual é prefixado ao *path* se o *mode* não incluir `os.X_OK`. Quando o *mode* inclui `os.X_OK`, a API do Windows `NeedCurrentDirectoryForExePathW` será consultada para determinar se o diretório atual deve ser prefixado ao *path*. Para evitar consultar o diretório de trabalho atual para executáveis: defina a variável de ambiente `NoDefaultCurrentDirectoryInExePath`.

Também no Windows, a variável de ambiente `PATHEXT` é usada para resolver comandos que podem não incluir uma extensão. Por exemplo, se você chamar `shutil.which("python")`, *which()* pesquisará `PATHEXT` para saber que deve procurar por `python.exe` dentro dos diretórios *path*. Por exemplo, no Windows:

```
>>> shutil.which("python")
'C:\\Python33\\python.EXE'
```

Isso também se aplica quando *cmd* é um caminho que contém um componente de diretório:

```
>>> shutil.which("C:\\Python33\\python")
'C:\\Python33\\python.EXE'
```

Adicionado na versão 3.3.

Alterado na versão 3.8: O tipo *bytes* é agora aceitado. Se o tipo de *cmd* é *bytes*, o tipo resultante também é *bytes*.

Alterado na versão 3.12: No Windows, o diretório atual não é mais adicionado ao caminho de pesquisa se *mode* incluir `os.X_OK` e WinAPI `NeedCurrentDirectoryForExePathW(cmd)` for falso, caso contrário, o diretório atual será adicionado mesmo se já estiver no caminho de pesquisa; `PATHEXT` agora é usado mesmo quando *cmd* inclui um componente de diretório ou termina com uma extensão que está em `PATHEXT`; e nomes de arquivos que não têm extensão agora podem ser encontrados.

exception `shutil.`**`Error`**

Esta exceção coleta exceções que são levantadas durante uma operação de vários arquivos. Para *copytree()*, o argumento de exceção é uma lista de tuplas de 3 elementos (*srcname*, *dstname*, *exception*).

Operações de cópia eficientes dependentes da plataforma

A partir do Python 3.8, todas as funções envolvendo uma cópia de arquivo (*copyfile()*, *copy()*, *copy2()*, *copytree()* e *move()*) podem usar chamadas do sistema de "cópia rápida" específicas da plataforma para copiar o arquivo de forma mais eficiente (veja bpo-33671). "cópia rápida" significa que a operação de cópia ocorre dentro do kernel, evitando o uso de buffers de espaço de usuário em Python como em "`outfd.write(infd.read())`".

No macOS, fcopyfile é usado para copiar o conteúdo do arquivo (não metadados).

No Linux, *os.copy_file_range()* ou *os.sendfile()* é usado.

No Solaris, *os.sendfile()* é usado.

No Windows, *shutil.copyfile()* usa um tamanho de buffer padrão maior (1 MiB ao invés de 64 KiB) e uma variante de *shutil.copyfileobj()* baseada em *memoryview()* é usada.

Se a operação de cópia rápida falhar e nenhum dado foi escrito no arquivo de destino, o shutil irá silenciosamente voltar a usar a função menos eficiente *copyfileobj()* internamente.

Alterado na versão 3.8.

Alterado na versão 3.14: Solaris agora usa *os.sendfile()*.

Alterado na versão 3.14: A cópia na gravação (copy-on-write) ou cópia do lado do servidor podem ser usadas internamente via *os.copy_file_range()* em sistemas de arquivos Linux suportados.

Exemplo de copytree

Um exemplo que usa o auxiliar *ignore_patterns()*:

```
from shutil import copytree, ignore_patterns

copytree(source, destination, ignore=ignore_patterns('*.pyc', 'tmp*'))
```

Isso irá copiar tudo, exceto os arquivos `.pyc` e arquivos ou diretórios cujo nome começa com `tmp`.

Outro exemplo que usa o argumento *ignore* para adicionar uma chamada de registro:

```
from shutil import copytree
import logging

def _logpath(path, names):
    logging.info('Working in %s', path)
    return []   # nada será ignorado

copytree(source, destination, ignore=_logpath)
```

exemplo rmtree

Este exemplo mostra como remover uma árvore de diretório no Windows onde alguns dos arquivos têm seu conjunto de bits somente leitura. Ele usa a função de retorno onexc para limpar o bit somente leitura e tentar remover novamente. Qualquer falha subsequente se propagará.

```
import os, stat
import shutil

def remove_readonly(func, path, _):
    "Clear the readonly bit and reattempt the removal"
    os.chmod(path, stat.S_IWRITE)
    func(path)

shutil.rmtree(directory, onexc=remove_readonly)
```

11.9.2 Operações de arquivamento

Adicionado na versão 3.2.

Alterado na versão 3.5: Adicionado suporte ao formato *xztar*.

Utilitários de alto nível para criar e ler arquivos compactados e arquivados também são fornecidos. Eles contam com os módulos *zipfile* e *tarfile*.

shutil.**make_archive**(*base_name*, *format*[, *root_dir*[, *base_dir*[, *verbose*[, *dry_run*[, *owner*[, *group*[, *logger*]]]]]])

Cria um arquivo compactado (como zip ou tar) e retorna seu nome.

base_name é o nome do arquivo a ser criado, incluindo o caminho, menos qualquer extensão específica do formato.

format é o formato do arquivo: um de "zip" (se o módulo *zlib* estiver disponível), "tar", "gztar" (se o módulo *zlib* estiver disponível), "bztar" (se o módulo *bz2* estiver disponível), "xztar" (se o módulo *lzma* estiver disponível) ou "zstdtar" (se o módulo *compression.zstd* estiver disponível).

root_dir é um diretório que será o diretório raiz do arquivo, todos os caminhos no arquivo serão relativos a ele; por exemplo, normalmente chdir em *root_dir* antes de criar o arquivo.

base_dir é o diretório de onde iniciamos o arquivamento; ou seja, *base_dir* será o prefixo comum de todos os arquivos e diretórios no arquivo. *base_dir* deve ser fornecido em relação a *root_dir*. Veja *Exemplo de arquivamento com base_dir* para como usar *base_dir* e *root_dir* juntos.

root_dir e *base_dir* têm com padrão o diretório atual.

Se *dry_run* for verdadeiro, nenhum arquivo é criado, mas as operações que seriam executadas são registradas no *logger*.

owner e *group* são usados ao criar um arquivo tar. Por padrão, usa o proprietário e grupo atuais.

logger deve ser um objeto compatível com a **PEP 282**, geralmente uma instância de *logging.Logger*.

O argumento *verbose* não é usado e foi descontinuado.

Levanta um *evento de auditoria* `shutil.make_archive` com os argumentos `base_name`, `format`, `root_dir`, `base_dir`.

> **Nota**
>
> Esta função não é segura para thread quando arquivadores personalizados registrados com *register_archive_format()* não suportam o argumento *root_dir*. Neste caso, ele altera temporariamente o diretório de trabalho atual do processo para *root_dir* para executar o arquivamento.

Alterado na versão 3.8: O formato pax moderno (POSIX.1-2001) agora é usado em vez do formato GNU legado para arquivos criados com `format="tar"`.

Alterado na versão 3.10.6: Esta função agora é segura para thread durante a criação de arquivos `.zip` e tar padrão.

shutil.**get_archive_formats**()

Retorna uma lista de formatos suportados para arquivamento. Cada elemento da sequência retornada é uma tupla `(nome, descrição)`.

Por padrão, *shutil* fornece estes formatos:

- *zip*: arquivo ZIP (se o módulo *zlib* estiver disponível).
- *tar*: Arquivo tar não compactado. Usa o formato POSIX.1-2001 pax para novos arquivos.
- *gztar*: arquivo tar compactado com gzip (se o módulo *zlib* estiver disponível).
- *bztar*: arquivo tar compactado com bzip2 (se o módulo *bz2* estiver disponível).
- *xztar*: Arquivo tar compactado com xz (se o módulo *lzma* estiver disponível).
- *zstdtar*: arquivo tar compactado com Zstandard (se o módulo *compression.zstd* estiver disponível).

Você pode registrar novos formatos ou fornecer seu próprio arquivador para quaisquer formatos existentes, usando *register_archive_format()*.

shutil.**register_archive_format**(*name*, *function*[, *extra_args*[, *description*]])

Registra um arquivador para o formato *name*.

function é o chamável que será usado para desempacotar arquivos. O chamável receberá o *base_name* do arquivo a ser criado, seguido pelo *base_dir* (cujo padrão é `os.curdir`) para iniciar o arquivamento. Outros argumentos são passados como argumentos nomeados *owner*, *group*, *dry_run* e *logger* (como passado em `make_archive()`).

Se *function* tiver o atributo personalizado `function.supports_root_dir` definido como `True`, o argumento *root_dir* será passado como um argumento nomeado. Caso contrário, o diretório de trabalho atual do processo será temporariamente alterado para *root_dir* antes de chamar *function*. Nesse caso, `make_archive()` não é segura para thread.

Se fornecido, *extra_args* é uma sequência de pares `(nome, valor)` que serão usados como argumentos nomeados extras quando o arquivador chamável for usado.

description é usado por `get_archive_formats()` que retorna a lista de arquivadores. O padrão é uma string vazia.

Alterado na versão 3.12: Adicionado suporte para funções que suportam o argumento *root_dir*.

shutil.**unregister_archive_format**(*name*)

Remove o formato de arquivo *name* da lista de formatos suportados.

shutil.**unpack_archive**(*filename*[, *extract_dir*[, *format*[, *filter*]]])

Desempacota um arquivo. *filename* é o caminho completo do arquivo.

extract_dir é o nome do diretório de destino onde o arquivo é desempacotado. Se não for fornecido, o diretório de trabalho atual será usado.

format é o formato do arquivo: um de "zip", "tar", "gztar", "bztar", "xztar" e "zstdtar". Ou qualquer outro formato registrado com `register_unpack_format()`. Se não for fornecido, `unpack_archive()` irá usar a extensão do nome do arquivo e ver se um desempacotador foi registrado para essa extensão. Caso nenhum seja encontrado, uma `ValueError` é levantada.

O argumento somente-nomeado *filter* é passado para a função de desempacotamento subjacente. Para arquivos zip, *filter* não é aceito. Para arquivos tar, é recomendado usar `'data'` (padrão desde o Python 3.14), a menos que esteja usando recursos específicos para tar e sistemas de arquivos do tipo UNIX. (Veja *Filtros de extração* para detalhes.)

Levanta um *evento de auditoria* `shutil.unpack_archive` com os argumentos `filename`, `extract_dir`, `format`.

> **Aviso**
>
> Nunca extrai arquivos de fontes não confiáveis sem inspeção prévia. É possível que arquivos sejam criados fora do caminho especificado no argumento *extract_dir*, por exemplo, membros que têm nomes de arquivo absolutos começando com "/" ou nomes de arquivo com dois pontos "..".
>
> Desde o Python 3.14, os padrões para ambos os formatos embutidos (arquivos zip e tar) evitarão os problemas de segurança mais perigosos, mas não impedirão *todos* os comportamentos indesejados. Leia a seção *Dicas para verificação adicional* para detalhes específicos do tar.

Alterado na versão 3.7: Aceita um *objeto caminho ou similar* para *filename* e *extract_dir*.

Alterado na versão 3.12: Adicionado o argumento *filter*.

shutil.**register_unpack_format**(*name*, *extensions*, *function*[, *extra_args*[, *description*]])

Registra um formato de desempacotamento. *name* é o nome do formato e *extensions* é uma lista de extensões correspondentes ao formato, como `.zip` para arquivos Zip.

function é o chamável que será usado para desempacotar arquivos. O chamável receberá:

- o caminho do arquivo, como argumento posicional;

- o diretório para o qual o arquivo deve ser extraído, como um argumento posicional;
- possivelmente um argumento nomeado *filter*, se foi dado a `unpack_archive()`;
- argumentos nomeados adicionais, especificados por *extra_args* como uma sequência de tuplas `(nome, valor)`.

description pode ser fornecido para descrever o formato e será devolvido pela função `get_unpack_formats()`.

`shutil.`**`unregister_unpack_format`**`(`*name*`)`

Cancela o registro de um formato de desempacotamento. *name* é o nome do formato.

`shutil.`**`get_unpack_formats`**`()`

Retorna uma lista de todos os formatos registrados para desempacotamento. Cada elemento da sequência retornada é uma tupla `(name, extensions, description)`.

Por padrão, `shutil` fornece estes formatos:

- *zip*: arquivo ZIP (desempacotar arquivos compactados funciona apenas se o módulo correspondente estiver disponível).
- *tar*: arquivo tar não comprimido.
- *gztar*: arquivo tar compactado com gzip (se o módulo `zlib` estiver disponível).
- *bztar*: arquivo tar compactado com bzip2 (se o módulo `bz2` estiver disponível).
- *xztar*: Arquivo tar compactado com xz (se o módulo `lzma` estiver disponível).
- *zstdtar*: arquivo tar compactado com Zstandard (se o módulo `compression.zstd` estiver disponível).

Você pode registrar novos formatos ou fornecer seu próprio desempacotador para quaisquer formatos existentes, usando `register_unpack_format()`.

Exemplo de arquivo

Neste exemplo, criamos um arquivo tar compactado com gzip contendo todos os arquivos encontrados no diretório `.ssh` do usuário:

```
>>> from shutil import make_archive
>>> import os
>>> archive_name = os.path.expanduser(os.path.join('~', 'meuarquivo'))
>>> root_dir = os.path.expanduser(os.path.join('~', '.ssh'))
>>> make_archive(archive_name, 'gztar', root_dir)
'/Users/tarek/meuarquivo.tar.gz'
```

O arquivo resultante contém:

```
$ tar -tzvf /Users/tarek/meuarquivo.tar.gz
drwx------ tarek/staff       0 2010-02-01 16:23:40 ./
-rw-r--r-- tarek/staff     609 2008-06-09 13:26:54 ./authorized_keys
-rwxr-xr-x tarek/staff      65 2008-06-09 13:26:54 ./config
-rwx------ tarek/staff     668 2008-06-09 13:26:54 ./id_dsa
-rwxr-xr-x tarek/staff     609 2008-06-09 13:26:54 ./id_dsa.pub
-rw------- tarek/staff    1675 2008-06-09 13:26:54 ./id_rsa
-rw-r--r-- tarek/staff     397 2008-06-09 13:26:54 ./id_rsa.pub
-rw-r--r-- tarek/staff   37192 2010-02-06 18:23:10 ./known_hosts
```

Exemplo de arquivamento com *base_dir*

Neste exemplo, semelhante ao *acima*, mostramos como usar `make_archive()`, mas desta vez com o uso de *base_dir*. Agora temos a seguinte estrutura de diretório:

```
$ tree tmp
tmp
└── root
    └── estrutura
        ├── conteudo
        │   └── por_favor_adicionar.txt
        └── nao_adicionar.txt
```

No arquivo final, `please_add.txt` deve ser incluído, mas `do_not_add.txt` não deve. Portanto, usamos o seguinte:

```
>>> from shutil import make_archive
>>> import os
>>> archive_name = os.path.expanduser(os.path.join('~', 'meuarquivo'))
>>> make_archive(
...     archive_name,
...     'tar',
...     root_dir='tmp/root',
...     base_dir='estrutura/conteudo',
... )
'/Users/tarek/meu_arquivo.tar'
```

Listar os arquivos no arquivo resultante nos dá:

```
$ python -m tarfile -l /Users/tarek/meuarquivo.tar
estrutura/conteudo/
estrutura/conteudo/por_favor_adicionar.txt
```

11.9.3 Consultando o tamanho do terminal de saída

`shutil.`**`get_terminal_size`**(*fallback=(columns, lines)*)

Obtém o tamanho da janela do terminal.

Para cada uma das duas dimensões, a variável de ambiente, `COLUMNS` e `LINES` respectivamente, é verificada. Se a variável estiver definida e o valor for um número inteiro positivo, ela será usada.

Quando `COLUMNS` ou `LINES` não está definida, que é o caso comum, o terminal conectado a *`sys.__stdout__`* é consultado invocando *`os.get_terminal_size()`*.

Se o tamanho do terminal não pode ser consultado com sucesso, ou porque o sistema não tem suporte a consultas, ou porque não estamos conectados a um terminal, o valor dado no parâmetro `fallback` é usado. O padrão de `fallback` é `(80, 24)`, que é o tamanho padrão usado por muitos emuladores de terminal.

O valor retornado é uma tupla nomeada do tipo *`os.terminal_size`*.

Veja também: The Single UNIX Specification, Versão 2, Other Environment Variables.

Adicionado na versão 3.3.

Alterado na versão 3.11: Os valores `fallback` também são usados se *`os.get_terminal_size()`* retornar zeros.

Ver também

Módulo *`os`*
Interfaces do sistema operacional, incluindo funções para trabalhar com arquivos num nível inferior a *objetos arquivos* do Python.

Módulo *`io`*
A biblioteca embutida de E/S do Python, incluindo as classes abstratas e algumas classes concretas, como

E/S de arquivos.

Função embutida `open()`
A maneira padrão de abrir arquivos para ler e escrever em Python.

CAPÍTULO 12

Persistência de Dados

Os módulos descritos neste capítulo possuem suporte ao armazenamento de dados do Python em um formato persistente no disco. Os módulos `pickle` e `marshal` podem transformar muitos tipos de dados do Python em um fluxo de bytes e então recriar os objetos a partir dos bytes. Os vários módulos relacionados ao DBM possuem suporte a uma família de formatos de arquivo baseados em hash que armazenam um mapeamento de strings para outras strings.

A lista de módulos descritos neste capítulo é:

12.1 `pickle` — Serialização de objetos Python

Código-fonte: Lib/pickle.py

O módulo `pickle` implementa protocolos binários para serializar e desserializar uma estrutura de objeto Python. *"Pickling"* é o processo pelo qual uma hierarquia de objetos Python é convertida em um fluxo de bytes, e *"unpickling"* é a operação inversa, em que um fluxo de bytes (de um *arquivo binário* ou *objeto bytes ou similar*) é convertido de volta em uma hierarquia de objetos. Pickling (e unpickling) com pickle é alternativamente conhecido como "serialização", "marshalling"[1] ou "flattening"; no entanto, para evitar confusão, usa-se is termos "pickling" e "unpickling". Nesta documentação traduzida, usaremos "serialização com pickle" e "desserialização com pickle", respectivamente.

Aviso

O módulo `pickle` **não é seguro**. Desserialize com pickle apenas os dados em que você confia.

É possível construir dados maliciosos em pickle que irão **executar código arbitrário durante o processo de desserialização com pickle**. Nunca desserialize com pickle os dados que possam vir de uma fonte não confiável ou que possam ter sido adulterados.

Considere assinar dados com `hmac` se você precisar garantir que eles não foram adulterados.

Formatos de serialização mais seguros como `json` podem ser mais apropriados se você estiver processando dados não confiáveis. Vejo *Comparação com json*.

[1] Não confunda isso com o módulo `marshal`

12.1.1 Relacionamento com outros módulos Python

Comparação com `marshal`

Python tem um módulo de serialização mais primitivo chamado *marshal*, mas em geral *pickle* deve ser sempre a forma preferida de serializar objetos Python. *marshal* existe principalmente para oferecer suporte a arquivos `.pyc` do Python.

O módulo *pickle* difere do *marshal* de várias maneiras significativas:

- O módulo *pickle* mantém o controle dos objetos que já serializou, para que referências posteriores ao mesmo objeto não sejam serializadas novamente. *marshal* não faz isso.

  Isso tem implicações tanto para objetos recursivos quanto para compartilhamento de objetos. Objetos recursivos são objetos que contêm referências a si mesmos. Eles não são tratados pelo marshal e, de fato, tentar usar marshal em objetos recursivos irá travar seu interpretador Python. O compartilhamento de objetos ocorre quando há várias referências ao mesmo objeto em locais diferentes na hierarquia de objetos sendo serializados. *pickle* armazena tais objetos apenas uma vez, e garante que todas as outras referências apontem para a cópia mestre. Os objetos compartilhados permanecem compartilhados, o que pode ser muito importante para objetos mutáveis.

- *marshal* não pode ser usado para serializar classes definidas pelo usuário e suas instâncias. *pickle* pode salvar e restaurar instâncias de classe de forma transparente, no entanto, a definição de classe deve ser importável e viver no mesmo módulo de quando o objeto foi armazenado.

- O formato de serialização do *marshal* não tem garantia de portabilidade entre as versões do Python. Como sua principal tarefa em vida é oferecer suporte a arquivos `.pyc`, os implementadores do Python se reservam o direito de alterar o formato de serialização de maneiras não compatíveis com versões anteriores, caso haja necessidade. O formato de serialização do *pickle* tem a garantia de ser compatível com versões anteriores em todas as versões do Python, desde que um protocolo pickle compatível seja escolhido e o código de serialização e desserialização com pickle lide com diferenças de tipo Python 2 a Python 3 se seus dados estiverem cruzando aquele limite de mudança de linguagem exclusivo.

Comparação com `json`

Existem diferenças fundamentais entre os protocolos pickle e JSON (JavaScript Object Notation):

- JSON é um formato de serialização de texto (ele produz texto unicode, embora na maioria das vezes seja codificado para `utf-8`), enquanto pickle é um formato de serialização binário;

- JSON é legível por humanos, enquanto pickle não é;

- JSON é interoperável e amplamente usado fora do ecossistema Python, enquanto pickle é específico para Python;

- JSON, por padrão, só pode representar um subconjunto dos tipos embutidos do Python, e nenhuma classe personalizada; pickle pode representar um número extremamente grande de tipos Python (muitos deles automaticamente, pelo uso inteligente dos recursos de introspecção do Python; casos complexos podem ser resolvidos implementando *APIs de objetos específicos*);

- Ao contrário do pickle, a desserialização não confiável do JSON não cria, por si só, uma vulnerabilidade de execução de código arbitrário.

Ver também

O módulo *json*: um módulo de biblioteca padrão que permite a serialização e desserialização JSON.

12.1.2 Formato de fluxo de dados

O formato de dados usado pelo *pickle* é específico do Python. Isso tem a vantagem de não haver restrições impostas por padrões externos, como JSON (que não pode representar o compartilhamento de ponteiros); no entanto, isso significa que programas não Python podem não ser capazes de reconstruir objetos Python serializados com pickle.

Por padrão, o formato de dados do *pickle* usa uma representação binária relativamente compacta. Se você precisa de características de tamanho ideal, pode com eficiência *comprimir* dados processados com pickle.

O módulo *pickletools* contém ferramentas para analisar fluxos de dados gerados por *pickle*. O código-fonte do *pickletools* tem extensos comentários sobre códigos de operações usados por protocolos de pickle.

Existem atualmente 6 protocolos diferentes que podem ser usados para a serialização com pickle. Quanto mais alto o protocolo usado, mais recente é a versão do Python necessária para ler o pickle produzido.

- A versão 0 do protocolo é o protocolo original "legível por humanos" e é compatível com versões anteriores do Python.
- A versão 1 do protocolo é um formato binário antigo que também é compatível com versões anteriores do Python.
- A versão 2 do protocolo foi introduzida no Python 2.3. Ela fornece uma serialização com pickle muito mais eficiente de *classes estilo novo*. Consulte **PEP 307** para obter informações sobre as melhorias trazidas pelo protocolo 2.
- A versão 3 do protocolo foi adicionada ao Python 3.0. Ela tem suporte explícito a objetos *bytes* e não é possível desserializar com pickle a partir do Python 2.x. Este era o protocolo padrão no Python 3.0–3.7.
- A versão 4 do protocolo foi adicionada ao Python 3.4. Ela adiciona suporte para objetos muito grandes, serialização com pickle de mais tipos de objetos e algumas otimizações de formato de dados. Isso é o protocolo padrão no Python 3.8–3.13. Consulte **PEP 3154** para obter informações sobre as melhorias trazidas pelo protocolo 4.
- A versão 5 do protocolo foi adicionada ao Python 3.8. Ela adiciona suporte a dados fora da banda e aumento de velocidade para dados dentro da banda. É o protocolo padrão a partir do Python 3.14. Consulte **PEP 574** para obter informações sobre as melhorias trazidas pelo protocolo 5.

Nota

A serialização é uma noção mais primitiva do que a persistência; embora o *pickle* leia e escreva objetos arquivo, ele não lida com a questão de nomear objetos persistentes, nem a questão (ainda mais complicada) de acesso simultâneo a objetos persistentes. O módulo *pickle* pode transformar um objeto complexo em um fluxo de bytes e pode transformar o fluxo de bytes em um objeto com a mesma estrutura interna. Talvez a coisa mais óbvia a fazer com esses fluxos de bytes seja escrevê-los em um arquivo, mas também é concebível enviá-los através de uma rede ou armazená-los em um banco de dados. O módulo *shelve* fornece uma interface simples para serializar e desserializar com pickle os objetos em arquivos de banco de dados no estilo DBM.

12.1.3 Interface do módulo

Para serializar uma hierarquia de objeto, você simplesmente chama a função *dumps()*. Da mesma forma, para desserializar um fluxo de dados, você chama a função *loads()*. No entanto, se você quiser mais controle sobre a serialização e desserialização, pode criar um objeto *Pickler* ou *Unpickler*, respectivamente.

O módulo *pickle* fornece as seguintes constantes:

pickle.**HIGHEST_PROTOCOL**

Um inteiro, a mais alta *versão de protocolo* disponível. Este valor pode ser passado como um valor de *protocol* para as funções *dump()* e *dumps()*, bem como o construtor de *Pickler*.

pickle.**DEFAULT_PROTOCOL**

Um inteiro, a *versão de protocolo* padrão usada para a serialização com pickle. Pode ser menor que *HIGHEST_PROTOCOL*. Atualmente, o protocolo padrão é 5, introduzido pela primeira vez no Python 3.8 e incompatível com as versões anteriores. Esta versão introduz suporte para buffers fora da banda, nos quais dados compatíveis com **PEP 3118** podem ser transmitidos separadamente do fluxo principal do pickle.

Alterado na versão 3.0: O protocolo padrão é 3.

Alterado na versão 3.8: O protocolo padrão é 4.

Alterado na versão 3.14: O protocolo padrão é o 5.

O módulo *pickle* fornece as seguintes funções para tornar o processo de serialização com pickle mais conveniente:

pickle.**dump**(*obj*, *file*, *protocol=None*, ***, *fix_imports=True*, *buffer_callback=None*)

Escreve a representação após a serialização com pickle do objeto *obj* no *objeto arquivo* aberto *file*. Isso é equivalente a `Pickler(file, protocol).dump(obj)`.

Os argumentos *file*, *protocol*, *fix_imports* e *buffer_callback* têm o mesmo sentido que no construtor de *Pickler*.

Alterado na versão 3.8: O argumento *buffer_callback* foi adicionado.

pickle.**dumps**(*obj*, *protocol=None*, ***, *fix_imports=True*, *buffer_callback=None*)

Retorna a representação em após a serialização com pickle do objeto *obj* como um objeto *bytes*, ao invés de escrevê-lo em um arquivo.

Os argumentos *protocol*, *fix_imports* e *buffer_callback* têm o mesmo sentido que no construtor de *Pickler*.

Alterado na versão 3.8: O argumento *buffer_callback* foi adicionado.

pickle.**load**(*file*, ***, *fix_imports=True*, *encoding='ASCII'*, *errors='strict'*, *buffers=None*)

Lê a representação serializada com pickle de um objeto a partir de *objeto arquivo* aberto *file* e retorna a hierarquia de objeto reconstituído especificada nele. Isso é equivalente a `Unpickler(file).load()`.

A versão do protocolo pickle é detectada automaticamente, portanto, nenhum argumento de protocolo é necessário. Bytes após a representação serializada com pickle do objeto são ignorados.

Os argumentos *file*, *fix_imports*, *encoding*, *errors*, *strict* e *buffers* têm o mesmo significado que no construtor construtor *Unpickler*.

Alterado na versão 3.8: O argumento *buffers* foi adicionado.

pickle.**loads**(*data*, */*, ***, *fix_imports=True*, *encoding='ASCII'*, *errors='strict'*, *buffers=None*)

Retorna a hierarquia de objeto reconstituído da representação serializada com pickle *data* de um objeto. *data* deve ser um *objeto bytes ou similar*.

A versão do protocolo pickle é detectada automaticamente, portanto, nenhum argumento de protocolo é necessário. Bytes após a representação serializada com pickle do objeto são ignorados.

Os argumentos *fix_imports*, *encoding*, *errors*, *strict* e *buffers* têm o mesmo significado que no construtor construtor *Unpickler*.

Alterado na versão 3.8: O argumento *buffers* foi adicionado.

O módulo *pickle* define três exceções:

exception pickle.**PickleError**

Classe base comum para as outras exceções de serialização com pickle. Herda de *Exception*.

exception pickle.**PicklingError**

Erro levantado quando um objeto não serializável com pickle é encontrado por *Pickler*. Herda de *PickleError*.

Consulte *O que pode ser serializado e desserializado com pickle?* para saber quais tipos de objetos podem ser serializados com pickle.

exception pickle.**UnpicklingError**

Erro levantado quando há um problema ao desserializar com pickle um objeto, como dados corrompidos ou violação de segurança. Herda de *PickleError*.

Observe que outras exceções também podem ser levantadas durante a desserialização com pickle, incluindo (mas não necessariamente limitado a) AttributeError, EOFError, ImportError e IndexError.

O módulo *pickle* exporta três classes, *Pickler*, *Unpickler* e *PickleBuffer*:

class pickle.**Pickler**(*file*, *protocol=None*, *, *fix_imports=True*, *buffer_callback=None*)

Isso leva um arquivo binário a escrever um fluxo de dados pickle.

O argumento opcional *protocol*, um inteiro, diz ao pickler para usar o protocolo fornecido; os protocolos suportados são de 0 a *HIGHEST_PROTOCOL*. Se não for especificado, o padrão é *DEFAULT_PROTOCOL*. Se um número negativo for especificado, *HIGHEST_PROTOCOL* é selecionado.

O argumento *file* deve ter um método write() que aceite um argumento de um único byte. Portanto, pode ser um arquivo em disco aberto para escrita binária, uma instância *io.BytesIO* ou qualquer outro objeto personalizado que atenda a esta interface.

Se *fix_imports* for verdadeiro e *protocolo* for menor que 3, pickle tentará mapear os novos nomes do Python 3 para os nomes dos módulos antigos usados no Python 2, de modo que o fluxo de dados pickle seja legível com o Python 2.

Se *buffer_callback* for `None` (o padrão), as visualizações de buffer são serializadas em *file* como parte do fluxo pickle.

Se *buffer_callback* não for `None`, ele pode ser chamado qualquer número de vezes com uma visualização de buffer. Se essa chamada retornar um valor falso (tal como `None`), o buffer fornecido é *fora da banda*; caso contrário, o buffer é serializado dentro da banda, ou seja, dentro do fluxo pickle.

É um erro se *buffer_callback* não for `None` e *protocol* for `None` ou menor que 5.

Alterado na versão 3.8: O argumento *buffer_callback* foi adicionado.

dump(*obj*)

Escreve a representação serializada em pickle de *obj* no objeto arquivo aberto fornecido no construtor.

persistent_id(*obj*)

Não faz nada por padrão. Isso existe para que uma subclasse possa substituí-lo.

Se *persistent_id()* retornar `None`, *obj* é serializado com pickle como de costume. Qualquer outro valor faz com que *Pickler* emita o valor retornado como um ID persistente para *obj*. O significado deste ID persistente deve ser definido por *Unpickler.persistent_load()*. Observe que o valor retornado por *persistent_id()* não pode ter um ID persistente.

Consulte *Persistência de objetos externos* para detalhes e exemplos de usos.

Alterado na versão 3.13: Foi adicionada a implementação padrão desse método na implementação em C da `Pickler`.

dispatch_table

A tabela de despacho de um objeto pickler é um registro de *funções de redução* do tipo que pode ser declarado usando *copyreg.pickle()*. É um mapeamento cujas chaves são classes e cujos valores são funções de redução. Uma função de redução leva um único argumento da classe associada e deve estar de acordo com a mesma interface de um método *__reduce__()*.

Por padrão, um objeto pickler não terá um atributo *dispatch_table*, e em vez disso usará a tabela de despacho global gerenciada pelo módulo *copyreg*. No entanto, para personalizar a serialização com pickle de um objeto pickler específico, pode-se definir o atributo *dispatch_table* para um objeto do tipo dict. Alternativamente, se uma subclasse de *Pickler* tem um atributo *dispatch_table* então ele será usado como a tabela de despacho padrão para instâncias daquela classe.

Consulte *Tabelas de despacho* para exemplos de uso.

Adicionado na versão 3.3.

reducer_override(*obj*)

Redutor especial que pode ser definido em subclasses de *Pickler*. Este método tem prioridade sobre qualquer redutor em *dispatch_table*. Ele deve estar de acordo com a mesma interface que um método *__reduce__()* e pode opcionalmente retornar *NotImplemented* como alternativa em redutores registrados em *dispatch_table* para serializar com pickle `obj`.

Para exemplo detalhado, consulte *Redução personalizada para tipos, funções e outros objetos*.

Adicionado na versão 3.8.

fast

Descontinuado. Ative o modo rápido se definido como um valor verdadeiro. O modo rápido desabilita o uso de memo, portanto, agilizando o processo de serialização com pickle por não gerar códigos de operação PUT supérfluos. Ele não deve ser usado com objetos autorreferenciais, fazer o contrário fará com que `Pickler` recorra infinitamente.

Use `pickletools.optimize()` se você precisar de serializações com pickle mais compactas.

clear_memo()

Limpa o "memo" do pickler

O memo é a estrutura de dados que lembra quais objetos o pickler já viu, de modo que objetos compartilhados ou recursivos sejam serializados por pickle por referência e não por valor. Este método é útil ao reutilizar picklers.

class pickle.**Unpickler**(*file*, *, *fix_imports=True*, *encoding='ASCII'*, *errors='strict'*, *buffers=None*)

Recebe um arquivo binário para ler um fluxo de dados pickle.

A versão do protocolo do pickle é detectada automaticamente, portanto, nenhum argumento de protocolo é necessário.

O argumento *file* deve ter três métodos: um método read() que recebe um argumento inteiro, um método readinto() que recebe um argumento buffer e um método readline() que não requer argumentos, como na interface `io.BufferedIOBase`. Assim, *file* pode ser um arquivo em disco aberto para leitura binária, um objeto `io.BytesIO` ou qualquer outro objeto personalizado que atenda a esta interface.

Os argumentos opcionais *fix_imports*, *encoding* e *errors* são usados para controlar o suporte de compatibilidade ao fluxo pickle gerado pelo Python 2. Se *fix_imports* for verdadeiro, pickle tentará mapear os nomes antigos do Python 2 para os novos nomes usados no Python 3. Os *encoding* e *erros* dizem ao pickle como decodificar instâncias de string de 8 bits capturadas pelo Python 2; o padrão é 'ASCII' e 'strict', respectivamente. O argumento *encoding* pode ser 'bytes' para ler essas instâncias de string de 8 bits como objetos de bytes. Usar `encoding='latin1'` é necessário para a desserialização com pickle de vetores NumPy e instâncias de `datetime`, `date` e `time` serializadas com pickle pelo Python 2.

Se *buffers* for `None` (o padrão), todos os dados necessários para desserialização devem estar contidos no fluxo pickle. Isso significa que o argumento *buffer_callback* era `None` quando um `Pickler` foi instanciado (ou quando `dump()` ou `dumps()` foi chamado).

Se *buffers* for `None`, deve ser um iterável de objetos habilitados para buffer que é consumido cada vez que o fluxo de serialização com pickle faz referência a uma visualização de buffer *fora da banda*. Esses buffers foram fornecidos em ordem para o *buffer_callback* de um objeto Pickler.

Alterado na versão 3.8: O argumento *buffers* foi adicionado.

load()

Lê a representação serializada com pickle de um objeto a partir do objeto arquivo aberto fornecido no construtor e retorna a hierarquia de objeto reconstituído especificada nele. Os bytes após a representação serializada com pickle do objeto são ignorados.

persistent_load(*pid*)

Levanta um `UnpicklingError` por padrão.

Se definido, `persistent_load()` deve retornar o objeto especificado pelo ID persistente *pid*. Se um ID persistente inválido for encontrado, uma `UnpicklingError` deve ser levantada.

Consulte *Persistência de objetos externos* para detalhes e exemplos de usos.

Alterado na versão 3.13: Foi adicionada a implementação padrão desse método na implementação em C da `Unpickler`.

find_class(*module*, *name*)

Importa *module* se necessário e retorna o objeto chamado *name* dele, onde os argumentos *module* e *name* são objetos `str`. Observe, ao contrário do que seu nome sugere, `find_class()` também é usado para encontrar funções.

As subclasses podem substituir isso para obter controle sobre quais tipos de objetos e como eles podem ser carregados, reduzindo potencialmente os riscos de segurança. Confira *Restringindo globais* para detalhes.

Levanta um *evento de auditoria* `pickle.find_class` com os argumentos `module`, `name`.

class pickle.**PickleBuffer**(*buffer*)

Um invólucro para um buffer que representa dados serializáveis com pickle. *buffer* deve ser um objeto provedor de buffer, como um *objeto bytes ou similar* ou um vetor N-dimensional.

`PickleBuffer` é ele próprio um provedor de buffer, de forma que é possível passá-lo para outras APIs que esperam um objeto provedor de buffer, como `memoryview`.

Objetos `PickleBuffer` só podem ser serializados usando o protocolo pickle 5 ou superior. Eles são elegíveis para *serialização fora de banda*.

Adicionado na versão 3.8.

raw()

Retorna um `memoryview` da área de memória subjacente a este buffer. O objeto retornado é um memoryview unidimensional, contíguo C com formato `B` (bytes não assinados). `BufferError` é levantada se o buffer não for contíguo C nem Fortran.

release()

Libera o buffer subjacente exposto pelo objeto PickleBuffer.

12.1.4 O que pode ser serializado e desserializado com pickle?

Os seguintes tipos podem ser serializados com pickle:

- constantes embutidas (`None`, `True`, `False`, `Ellipsis` e `NotImplemented`);
- inteiros, números de ponto flutuante, números complexos;
- strings, bytes, bytearrays;
- tuplas, listas, conjuntos e dicionários contendo apenas objetos serializáveis com pickle;
- funções (embutidas ou definidas pelo usuário) acessíveis no nível superior de um módulo (usando `def`, não `lambda`);
- classes acessíveis no nível superior de um módulo;
- instâncias de classes cujo o resultado da chamada de `__getstate__()` seja serializável com pickle (veja a seção *Serializando com pickle instâncias de classes* para detalhes).

As tentativas de serializar objetos não serializáveis com pickle vão levantar a exceção `PicklingError`; quando isso acontece, um número não especificado de bytes pode já ter sido escrito no arquivo subjacente. Tentar serializar com pickle uma estrutura de dados altamente recursiva pode exceder a profundidade máxima de recursão, a `RecursionError` será levantada neste caso. Você pode aumentar este limite cuidadosamente com `sys.setrecursionlimit()`.

Observe que as funções (embutidas e definidas pelo usuário) são serializadas com pickle pelo *nome qualificado*, não pelo valor.[2] Isso significa que apenas o nome da função é serializado com pickle, junto com o nome do módulo e das classes contidos. Nem o código da função, nem qualquer um de seus atributos de função são serializados com pickle. Assim, o módulo de definição deve ser importável no ambiente de desserialização com pickle, e o módulo deve conter o objeto nomeado, caso contrário, uma exceção será levantada.[3]

Da mesma forma, as classes são serializadas com pickle pelo nome qualificado, portanto, aplicam-se as mesmas restrições no ambiente de desserialização com pickle. Observe que nenhum código ou dado da classe é coletado, portanto, no exemplo a seguir, o atributo de classe `attr` não é restaurado no ambiente de desserialização com pickle:

[2] É por isso que funções `lambda` não podem ser serializadas com pickle: todas as funções `lambda` compartilham o mesmo nome: `<lambda>`.
[3] A exceção levantada provavelmente será uma `ImportError` ou uma `AttributeError`, mas poderia ser outra coisa.

```
class Foo:
    attr = 'A class attribute'

picklestring = pickle.dumps(Foo)
```

Essas restrições são a razão pela qual as funções e classes serializáveis com pickle devem ser definidas no nível superior de um módulo.

Da mesma forma, quando as instâncias da classe são serializadas com pickle, o código e os dados de sua classe não são serializados junto com elas. Apenas os dados da instância são serializados com pickle. Isso é feito de propósito para que você possa corrigir bugs em uma classe ou adicionar métodos à classe e ainda carregar objetos que foram criados com uma versão anterior da classe. Se você planeja ter objetos de longa duração que verão muitas versões de uma classe, pode valer a pena colocar um número de versão nos objetos para que as conversões adequadas possam ser feitas pelo método *__setstate__()* da classe.

12.1.5 Serializando com pickle instâncias de classes

Nesta seção, descrevemos os mecanismos gerais disponíveis para você definir, personalizar e controlar como as instâncias de classe são serializadas e desserializadas com pickle.

Na maioria dos casos, nenhum código adicional é necessário para tornar as instâncias serializáveis com pickle. Por padrão, o pickle recuperará a classe e os atributos de uma instância por meio de introspecção. Quando uma instância de classe não está serializada com pickle, seu método `__init__()` geralmente *não* é invocado. O comportamento padrão primeiro cria uma instância não inicializada e, em seguida, restaura os atributos salvos. O código a seguir mostra uma implementação desse comportamento:

```
def save(obj):
    return (obj.__class__, obj.__dict__)

def restore(cls, attributes):
    obj = cls.__new__(cls)
    obj.__dict__.update(attributes)
    return obj
```

As classes podem alterar o comportamento padrão, fornecendo um ou vários métodos especiais:

`object.`**`__getnewargs_ex__`**`()`

Nos protocolos 2 e mais recentes, as classes que implementam o método *__getnewargs_ex__()* podem ditar os valores passados para o método `__new__()` após a desserialização com pickle. O método deve retornar um par `(args, kwargs)` onde *args* é uma tupla de argumentos posicionais e *kwargs* um dicionário de argumentos nomeados para construir o objeto. Esses serão passados para o método `__new__()` após a desserialização com pickle.

Você deve implementar este método se o método `__new__()` de sua classe requer argumentos somente-nomeados. Caso contrário, é recomendado para compatibilidade implementar *__getnewargs__()*.

Alterado na versão 3.6: *__getnewargs_ex__()* é agora usado em protocolos 2 e 3.

`object.`**`__getnewargs__`**`()`

Este método serve a um propósito semelhante ao de *__getnewargs_ex__()*, mas tem suporte apenas a argumentos posicionais. Ele deve retornar uma tupla de argumentos `args` que serão passados para o método `__new__()` após a desserialização com pickle.

__getnewargs__() não será chamado se *__getnewargs_ex__()* estiver definido.

Alterado na versão 3.6: Antes do Python 3.6, *__getnewargs__()* era chamado em vez de *__getnewargs_ex__()* nos protocolos 2 e 3.

`object.`**`__getstate__`**`()`

Classes podem influenciar ainda mais como suas instâncias são serializadas com pickle, substituindo o método *__getstate__()*. Ele é chamado e o objeto retornado é serializado com pickle como o conteúdo da instância, em vez de um estado padrão. Existem vários casos:

- Para uma classe que não possui instância `__dict__` e não possui `__slots__`, o estado padrão é `None`.
- Para uma classe que não uma possui instância `__dict__` nem `__slots__`, o estado padrão é `self.__dict__`.
- Para uma classe que possui uma instância `__dict__` e `__slots__`, o estado padrão é uma tupla consistindo de dois dicionários: `self.__dict__`, e um dicionário de mapeamento de nomes de slot para valores de slot. Apenas os slots que possuem um valor são incluídos neste último.
- Para uma classe que possui `__slots__` e nenhuma `__dict__` de instância, o estado padrão é uma tupla cujo primeiro item é `None` e cujo segundo item é um dicionário mapeando os nomes dos slots para os valores dos slots descritos no tópico anterior.

Alterado na versão 3.11: Adicionada a implementação padrão do método `__getstate__()` na classe `objeto`.

`object.`**`__setstate__`**(*state*)

Ao desserializar com pickle, se a classe define *`__setstate__()`*, ela é chamada com o estado não desserializado. Nesse caso, não há nenhum requisito para que o objeto de estado seja um dicionário. Caso contrário, o estado serializado com pickle deve ser um dicionário e seus itens são atribuídos ao dicionário da nova instância.

> **Nota**
>
> Se *`__reduce__()`* retorna um estado com valor `None` na serialização com pickle, o método *`__setstate__()`* não será chamado quando da desserialização com pickle.

Confira a seção *Manipulação de objetos com estado* para mais informações sobre como usar os métodos *`__getstate__()`* e *`__setstate__()`*.

> **Nota**
>
> Quando da desserialização com pickle, alguns métodos, como `__getattr__()`, `__getattribute__()` ou `__setattr__()`, podem ser chamados na instância. No caso desses métodos dependerem de alguma invariante interna ser verdadeira, o tipo deve ser implementado `__new__()` para estabelecer tal invariante, pois `__init__()` não é chamada quando da desserialização com pickle em uma instância.

Como veremos, o pickle não usa diretamente os métodos descritos acima. Na verdade, esses métodos são parte do protocolo de cópia que implementa o método especial *`__reduce__()`*. O protocolo de cópia fornece uma interface unificada para recuperar os dados necessários para serialização com pickle e cópia de objetos.[4]

Apesar de poderoso, implementar *`__reduce__()`* diretamente em sua classe é algo propenso a erro. Por este motivo, designers de classe devem usar a interface de alto nível (ou seja, *`__getnewargs_ex__()`*, *`__getstate__()`* e *`__setstate__()`*) sempre que possível. Vamos mostrar, porém, casos em que o uso de `__reduce__()` é a única opção ou leva a uma serialização com pickle mais eficiente, ou as ambas.

`object.`**`__reduce__`**()

A interface está atualmente definida da seguinte maneira. O método *`__reduce__()`* não aceita nenhum argumento e deve retornar uma string ou preferencialmente uma tupla (o objeto retornado é frequentemente referido como o "valor de redução").

Se uma string é retornada, ela deve ser interpretada como o nome de uma variável global. Deve ser o nome local do objeto relativo ao seu módulo; o módulo pickle pesquisa o espaço de nomes do módulo para determinar o módulo do objeto. Esse comportamento é normalmente útil para singletons.

Quando uma tupla é retornada, ela deve ter entre dois e seis itens. Os itens opcionais podem ser omitidos ou `None` pode ser fornecido como seu valor. A semântica de cada item está em ordem:

- Um objeto chamável que será chamado para criar a versão inicial do objeto.

[4] O módulo *`copy`* usa este protocolo para operações de cópia rasa e cópia profunda.

- Uma tupla de argumentos para o objeto chamável. Uma tupla vazia deve ser fornecida se o chamável não aceitar nenhum argumento.
- Opcionalmente, o estado do objeto, que será passado para o método *__setstate__()* do objeto conforme descrito anteriormente. Se o objeto não tiver tal método, o valor deve ser um dicionário e será adicionado ao atributo `__dict__` do objeto.
- Opcionalmente, um iterador (e não uma sequência) produzindo itens sucessivos. Esses itens serão anexados ao objeto usando `obj.append(item)` ou, em lote, usando `obj.extend(list_of_items)`. Isso é usado principalmente para subclasses de lista, mas pode ser usado por outras classes, desde que tenham os métodos *append()* e *extend()* com a assinatura apropriada. (Se `append()` ou `extend()` é usado depende de qual versão do protocolo pickle é usada, bem como o número de itens a anexar, então ambos devem ser suportados.)
- Opcionalmente, um iterador (não uma sequência) produzindo pares de valor-chave sucessivos. Esses itens serão armazenados no objeto usando `obj[chave]=valor`. Isso é usado principalmente para subclasses de dicionário, mas pode ser usado por outras classes, desde que implementem `__setitem__()`.
- Opcionalmente, um chamável com uma assinatura `(obj, estado)`. Este chamável permite ao usuário controlar programaticamente o comportamento de atualização de estado de um objeto específico, ao invés de usar o método estático *__setstate__()* de `obj`. Se não for `None`, este chamável terá prioridade sobre o *__setstate__()* de `obj`.

  Adicionado na versão 3.8: O sexto item opcional de tupla, `(obj, estado)`, foi adicionado.

`object.`**`__reduce_ex__`**(*protocol*)

Alternativamente, um método *__reduce_ex__()* pode ser definido. A única diferença é que este método deve ter um único argumento inteiro, a versão do protocolo. Quando definido, pickle irá preferir isso ao método *__reduce__()*. Além disso, *__reduce__()* automaticamente se torna um sinônimo para a versão estendida. O principal uso desse método é fornecer valores de redução com compatibilidade reversa para versões mais antigas do Python.

Persistência de objetos externos

Para o benefício da persistência do objeto, o módulo *pickle* tem suporte à noção de uma referência a um objeto fora do fluxo de dados serializados com pickle. Esses objetos são referenciados por um ID persistente, que deve ser uma string de caracteres alfanuméricos (para o protocolo 0)[5] ou apenas um objeto arbitrário (para qualquer protocolo mais recente).

A resolução de tais IDs persistentes não é definida pelo módulo *pickle*; ele vai delegar esta resolução aos métodos definidos pelo usuário no selecionador e no separador, *persistent_id()* e *persistent_load()* respectivamente.

Para serializar com pickle objetos que têm um ID externo persistente, o pickler deve ter um método *persistent_id()* personalizado que recebe um objeto como um argumento e retorna `None` ou o ID persistente para esse objeto. Quando `None` é retornado, o pickler simplesmente serializa o objeto normalmente. Quando uma string de ID persistente é retornada, o pickler serializa aquele objeto, junto com um marcador para que o unpickler o reconheça como um ID persistente.

Para desserializar com pickle objetos externos, o unpickler deve ter um método *persistent_load()* personalizado que recebe um objeto de ID persistente e retorna o objeto referenciado.

Aqui está um exemplo abrangente que apresenta como o ID persistente pode ser usado para serializar com pickle objetos externos por referência.

```
# Exemplo simples que mostra como o ID persistente pode ser usado
# serializar com pickle objetos externos por referência.

import pickle
```

(continua na próxima página)

[5] A limitação de caracteres alfanuméricos se deve ao fato de que os IDs persistentes, no protocolo 0, serem delimitados pelo caractere de nova linha. Portanto, se qualquer tipo de caractere de nova linha ocorrer em IDs persistentes, os dados resultantes da serialização com pickle se tornarão ilegíveis.

(continuação da página anterior)

```
import sqlite3
from collections import namedtuple

# Classe simples que representa um registro em nosso banco de dados.
MemoRecord = namedtuple("MemoRecord", "key, task")

class DBPickler(pickle.Pickler):

    def persistent_id(self, obj):
        # Em vez de conservar MemoRecord como uma instância de classe regular,
        # emitimos um ID persistente.
        if isinstance(obj, MemoRecord):
            # Aqui, nosso ID persistente é simplesmente uma tupla, contendo
            # uma tag e uma chave, que se refere a um registro específico
            # no banco de dados.
            return ("MemoRecord", obj.key)
        else:
            # Se obj não tiver um ID persistente, retorna None. Significa que
            # obj precisa ser conservado como de costume.
            return None


class DBUnpickler(pickle.Unpickler):

    def __init__(self, file, connection):
        super().__init__(file)
        self.connection = connection

    def persistent_load(self, pid):
        # Este método é invocado sempre que um ID persistente é encontrado.
        # Aqui, pid é a tupla retornada por DBPickler.
        cursor = self.connection.cursor()
        type_tag, key_id = pid
        if type_tag == "MemoRecord":
            # Busca o registro referenciado no banco de dados e retorna-o.
            cursor.execute("SELECT * FROM memos WHERE key=?", (str(key_id),))
            key, task = cursor.fetchone()
            return MemoRecord(key, task)
        else:
            # Sempre levanta um erro se você puder retornar o objeto correto.
            # Caso contrário, a desserialização com pickle considerará que
            # None é o objeto referenciado pelo ID persistente.
            raise pickle.UnpicklingError("unsupported persistent object")


def main():
    import io
    import pprint

    # Inicializa e popula nosso banco de dados.
    conn = sqlite3.connect(":memory:")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE memos(key INTEGER PRIMARY KEY, task TEXT)")
    tasks = (
        'give food to fish',
        'prepare group meeting',
```

(continua na próxima página)

(continuação da página anterior)

```
        'fight with a zebra',
        )
    for task in tasks:
        cursor.execute("INSERT INTO memos VALUES(NULL, ?)", (task,))

    # Busca os registros a serem serializados com pickle.
    cursor.execute("SELECT * FROM memos")
    memos = [MemoRecord(key, task) for key, task in cursor]
    # Salva os registros usando nosso DBPickler personalizado.
    file = io.BytesIO()
    DBPickler(file).dump(memos)

    print("Pickled records:")
    pprint.pprint(memos)

    # Atualiza um registro, só por precaução.
    cursor.execute("UPDATE memos SET task='learn italian' WHERE key=1")

    # Carrega os registros do fluxo de dados pickle.
    file.seek(0)
    memos = DBUnpickler(file, conn).load()

    print("Unpickled records:")
    pprint.pprint(memos)


if __name__ == '__main__':
    main()
```

Tabelas de despacho

Se alguém quiser personalizar a serialização com pickle de algumas classes sem perturbar nenhum outro código que dependa da serialização, pode-se criar um pickler com uma tabela de despacho privada.

A tabela de despacho global gerenciada pelo módulo *copyreg* está disponível como `copyreg.dispatch_table`. Portanto, pode-se escolher usar uma cópia modificada de `copyreg.dispatch_table` como uma tabela de despacho privada.

Por exemplo

```
f = io.BytesIO()
p = pickle.Pickler(f)
p.dispatch_table = copyreg.dispatch_table.copy()
p.dispatch_table[SomeClass] = reduce_SomeClass
```

cria uma instância de *pickle.Pickler* com uma tabela de despacho privada que trata a classe `SomeClass` especialmente. Alternativamente, o código

```
class MyPickler(pickle.Pickler):
    dispatch_table = copyreg.dispatch_table.copy()
    dispatch_table[SomeClass] = reduce_SomeClass
f = io.BytesIO()
p = MyPickler(f)
```

faz o mesmo, mas todas as instâncias de `MyPickler` compartilharão por padrão a tabela de despacho privada. Por outro lado, o código

```
copyreg.pickle(SomeClass, reduce_SomeClass)
f = io.BytesIO()
p = pickle.Pickler(f)
```

modifica a tabela de despacho global compartilhada por todos os usuários do módulo *copyreg*.

Manipulação de objetos com estado

Aqui está um exemplo que mostra como modificar o comportamento de serialização com pickle de uma classe. A classe `TextReader` abaixo abre um arquivo texto e retorna o número da linha e o conteúdo da linha cada vez que seu método `readline()` é chamado. Se uma instância de `TextReader` for selecionada, todos os atributos *exceto* o membro do objeto arquivo são salvos. Quando a instância é removida, o arquivo é reaberto e a leitura continua a partir do último local. Os métodos `__setstate__()` e `__getstate__()` são usados para implementar este comportamento.

```
class TextReader:
    """Exibe e numera linhas em um arquivo texto."""

    def __init__(self, filename):
        self.filename = filename
        self.file = open(filename)
        self.lineno = 0

    def readline(self):
        self.lineno += 1
        line = self.file.readline()
        if not line:
            return None
        if line.endswith('\n'):
            line = line[:-1]
        return "%i: %s" % (self.lineno, line)

    def __getstate__(self):
        # Copia o estado do objeto de self.__dict__, que
        # contém todos os nossos atributos de instância.
        # Sempre use o método dict.copy() para evitar
        # modificar o estado original.
        state = self.__dict__.copy()
        # Remove as entradas não serializáveis com pickle.
        del state['file']
        return state

    def __setstate__(self, state):
        # Restaura os atributos da instância (isto é, filename e lineno).
        self.__dict__.update(state)
        # Restaura o estado do arquivo aberto anteriormente.
        # Para fazer isso, precisamos reabri-lo e ler dele até
        # a contagem de linha ser restaurada.
        file = open(self.filename)
        for _ in range(self.lineno):
            file.readline()
        # Ao final, salva o arquivo.
        self.file = file
```

Um exemplo de uso pode ser algo assim:

```
>>> reader = TextReader("hello.txt")
```

(continua na próxima página)

(continuação da página anterior)

```
>>> reader.readline()
'1: Hello world!'
>>> reader.readline()
'2: I am line number two.'
>>> new_reader = pickle.loads(pickle.dumps(reader))
>>> new_reader.readline()
'3: Goodbye!'
```

12.1.6 Redução personalizada para tipos, funções e outros objetos

Adicionado na versão 3.8.

Às vezes, *dispatch_table* pode não ser flexível o suficiente. Em particular, podemos querer personalizar a serialização com pickle com base em outro critério que não o tipo do objeto, ou podemos personalizar a serialização com pickle de funções e classes.

Para esses casos, é possível criar uma subclasse da classe *Pickler* e implementar um método *reducer_override()*. Este método pode retornar uma tupla de redução arbitrária (veja *__reduce__()*). Ele pode, alternativamente, retornar *NotImplemented* para retornar ao comportamento tradicional.

Se *dispatch_table* e *reducer_override()* forem definidos, o método *reducer_override()* tem prioridade.

Nota

Por motivos de desempenho, *reducer_override()* não pode ser chamado para os seguintes objetos: `None`, `True`, `False`, e as instâncias exatas de *int*, *float*, *bytes*, *str*, *dict*, *set*, *frozenset*, *list* e *tuple*.

Aqui está um exemplo simples onde permitimos serialização com pickle e reconstrução de uma determinada classe:

```
import io
import pickle

class MyClass:
    my_attribute = 1

class MyPickler(pickle.Pickler):
    def reducer_override(self, obj):
        """Redutor personalizado para MyClass."""
        if getattr(obj, "__name__", None) == "MyClass":
            return type, (obj.__name__, obj.__bases__,
                          {'my_attribute': obj.my_attribute})
        else:
            # Para qualquer outro objeto, recorre à redução de costume
            return NotImplemented

f = io.BytesIO()
p = MyPickler(f)
p.dump(MyClass)

del MyClass

unpickled_class = pickle.loads(f.getvalue())

assert isinstance(unpickled_class, type)
assert unpickled_class.__name__ == "MyClass"
assert unpickled_class.my_attribute == 1
```

12.1.7 Buffers fora da banda

Adicionado na versão 3.8.

Em alguns contextos, o módulo `pickle` é usado para transferir grandes quantidades de dados. Portanto, pode ser importante minimizar o número de cópias de memória para preservar o desempenho e o consumo de recursos. No entanto, a operação normal do módulo `pickle`, à medida que transforma uma estrutura semelhante a um gráfico de objetos em um fluxo sequencial de bytes, envolve intrinsecamente a cópia de dados de e para o fluxo pickle.

Esta restrição pode ser evitada se tanto o *fornecedor* (a implementação dos tipos de objetos a serem transferidos) e o *consumidor* (a implementação do sistema de comunicações) tiverem suporte aos recursos de transferência fora de banda fornecidos pelo protocolo pickle 5 e superior.

API de provedor

Os grandes objetos de dados a serem serializados com pickle devem implementar um método `__reduce_ex__()` especializado para o protocolo 5 e superior, que retorna uma instância `PickleBuffer` (em vez de, por exemplo, um objeto `bytes`) para quaisquer dados grandes.

Um objeto `PickleBuffer` *sinaliza* que o buffer subjacente é elegível para transferência de dados fora de banda. Esses objetos permanecem compatíveis com o uso normal do módulo `pickle`. No entanto, os consumidores também podem optar por dizer ao `pickle` que eles irão lidar com esses buffers por conta própria.

API de consumidor

Um sistema de comunicação pode permitir o manuseio personalizado dos objetos `PickleBuffer` gerados ao serializar um grafo de objeto.

No lado emissor, é necessário passar um argumento *buffer_callback* para `Pickler` (ou para a função `dump()` ou `dumps()`), que será chamada com cada `PickleBuffer` gerado durante a serialização com pickle do grafo do objeto. Os buffers acumulados pelo *buffer_callback* não verão seus dados copiados no fluxo pickle, apenas um marcador barato será inserido.

No lado receptor, é necessário passar um argumento *buffers* para `Unpickler` (ou para a função `load()` ou `load()`), que é um iterável dos buffers que foram passado para *buffer_callback*. Esse iterável deve produzir buffers na mesma ordem em que foram passados para *buffer_callback*. Esses buffers fornecerão os dados esperados pelos reconstrutores dos objetos cuja serialização com pickle produziu os objetos `PickleBuffer` originais.

Entre o lado emissor e o lado receptor, o sistema de comunicações está livre para implementar seu próprio mecanismo de transferência para buffers fora de banda. As otimizações potenciais incluem o uso de memória compartilhada ou compactação dependente do tipo de dados.

Exemplo

Aqui está um exemplo trivial onde implementamos uma subclasse de `bytearray` capaz de participar de serialização com pickle de buffer fora de banda:

```
class ZeroCopyByteArray(bytearray):

    def __reduce_ex__(self, protocol):
        if protocol >= 5:
            return type(self)._reconstruct, (PickleBuffer(self),), None
        else:
            # PickleBuffer é proibido no pickle nos protocolos <= 4.
            return type(self)._reconstruct, (bytearray(self),)

    @classmethod
    def _reconstruct(cls, obj):
        with memoryview(obj) as m:
            # Obtém um controle sobre o objeto buffer original
            obj = m.obj
            if type(obj) is cls:
```

(continua na próxima página)

(continuação da página anterior)

```
                # O objeto do buffer original é um ZeroCopyByteArray,
                # retorna-o como está.
                return obj
            else:
                return cls(obj)
```

O reconstrutor (o método de classe `_reconstruct`) retorna o objeto de fornecimento do buffer se ele tiver o tipo correto. Esta é uma maneira fácil de simular o comportamento de cópia zero neste exemplo de brinquedo.

Do lado consumidor, podemos serializar com pickle esses objetos da maneira usual, que quando não serializados nos dará uma cópia do objeto original:

```
b = ZeroCopyByteArray(b"abc")
data = pickle.dumps(b, protocol=5)
new_b = pickle.loads(data)
print(b == new_b)  # True
print(b is new_b)  # False: uma cópia foi feita
```

Mas se passarmos um *buffer_callback* e, em seguida, retornarmos os buffers acumulados ao desserializar, seremos capazes de recuperar o objeto original:

```
b = ZeroCopyByteArray(b"abc")
buffers = []
data = pickle.dumps(b, protocol=5, buffer_callback=buffers.append)
new_b = pickle.loads(data, buffers=buffers)
print(b == new_b)  # True
print(b is new_b)  # True: nenhuma cópia foi feita
```

Este exemplo é limitado pelo fato de que *bytearray* aloca sua própria memória: você não pode criar uma instância de *bytearray* que é apoiada pela memória de outro objeto. No entanto, tipos de dados de terceiros, como arrays de NumPy, não têm essa limitação e permitem o uso de serialização com pickle de cópia zero (ou fazer o mínimo de cópias possível) ao transferir entre processos ou sistemas distintos.

Ver também

PEP 574 – Protocolo de Pickle 5 com buffers de dados fora da banda

12.1.8 Restringindo globais

Por padrão, a desserialização com pickle importará qualquer classe ou função que encontrar nos dados pickle. Para muitos aplicativos, esse comportamento é inaceitável, pois permite que o unpickler importe e invoque código arbitrário. Basta considerar o que este fluxo de dados pickle feito à mão faz quando carregado:

```
>>> import pickle
>>> pickle.loads(b"cos\nsystem\n(S'echo hello world'\ntR.")
hello world
0
```

Neste exemplo, o unpickler importa a função *os.system()* e então aplica o argumento string "echo hello world". Embora este exemplo seja inofensivo, não é difícil imaginar um que possa danificar seu sistema.

Por esta razão, você pode querer controlar o que é desserializado com pickle personalizando *Unpickler.find_class()*. Ao contrário do que seu nome sugere, *Unpickler.find_class()* é chamado sempre que um global (ou seja, uma classe ou uma função) é solicitado. Assim, é possível proibir completamente os globais ou restringi-los a um subconjunto seguro.

Aqui está um exemplo de um unpickler que permite que apenas algumas classes seguras do módulo *builtins* sejam carregadas:

```
import builtins
import io
import pickle

safe_builtins = {
    'range',
    'complex',
    'set',
    'frozenset',
    'slice',
}

class RestrictedUnpickler(pickle.Unpickler):

    def find_class(self, module, name):
        # Só permite classes seguras de bultins.
        if module == "builtins" and name in safe_builtins:
            return getattr(builtins, name)
        # Proíbe todo o resto.
        raise pickle.UnpicklingError("global '%s.%s' is forbidden" %
                                     (module, name))

def restricted_loads(s):
    """Função auxiliar análoga a pickle.loads()."""
    return RestrictedUnpickler(io.BytesIO(s)).load()
```

Um exemplo de uso do nosso unpickler funcionando como esperado:

```
>>> restricted_loads(pickle.dumps([1, 2, range(15)]))
[1, 2, range(0, 15)]
>>> restricted_loads(b"cos\nsystem\n(S'echo hello world'\ntR.")
Traceback (most recent call last):
  ...
pickle.UnpicklingError: global 'os.system' is forbidden
>>> restricted_loads(b'cbuiltins\neval\n'
...                  b'(S\'getattr(__import__("os"), "system")'
...                  b'("echo hello world")\'\ntR.')
Traceback (most recent call last):
  ...
pickle.UnpicklingError: global 'builtins.eval' is forbidden
```

Como nossos exemplos mostram, você deve ter cuidado com o que permite que seja desserializado com pickle. Portanto, se a segurança é uma preocupação, você pode querer considerar alternativas como a API de marshalling em *xmlrpc.client* ou soluções de terceiros.

12.1.9 Desempenho

Versões recentes do protocolo pickle (do protocolo 2 em diante) apresentam codificações binárias eficientes para vários recursos comuns e tipos embutidos. Além disso, o módulo *pickle* tem um otimizador transparente escrito em C.

12.1.10 Exemplos

Para código mais simples, use as funções *dump()* e *load()*.

```
import pickle
```

(continua na próxima página)

(continuação da página anterior)

```
# Uma coleção arbitrária de objetos aceitos por pickle.
data = {
    'a': [1, 2.0, 3+4j],
    'b': ("character string", b"byte string"),
    'c': {None, True, False}
}

with open('data.pickle', 'wb') as f:
    # Serializa com pickle o dicionário 'data' usando
    # o protocolo mais alto disponível.
    pickle.dump(data, f, pickle.HIGHEST_PROTOCOL)
```

O exemplo a seguir lê os dados resultantes em serializados com pickle.

```
import pickle

with open('data.pickle', 'rb') as f:
    # A versão do protocolo utilizada é detectada automaticamente,
    # portanto não precisamos especificá-la.
    data = pickle.load(f)
```

12.1.11 Interface de linha de comando

O módulo *pickle* pode ser invocado como um script a partir da linha de comando e exibirá o conteúdo dos arquivos de pickle. No entanto, quando o arquivo de pickle que você deseja examinar vem de uma fonte não confiável, `-m pickletools` é uma opção mais segura, pois não executa o bytecode do pickle. Consulte o *uso da CLI do pickletools*.

```
python -m pickle aquivo_pickle [aquivo_pickle ...]
```

A seguinte opção é aceita:

pickle_file
: Um arquivo pickle para ler, ou – para indicar leitura da entrada padrão.

Ver também

Módulo *copyreg*
: Registro de construtor de interface Pickle para tipos de extensão.

Módulo *pickletools*
: Ferramentas para trabalhar e analisar dados serializados com pickle.

Módulo *shelve*
: Banco de dados indexado de objetos; usa *pickle*.

Módulo *copy*
: Cópia rasa e cópia profunda de objeto.

Módulo *marshal*
: Serialização de alto desempenho de tipos embutidos.

12.2 copyreg — Registra funções de suporte pickle

Código-fonte: Lib/copyreg.py

O módulo *copyreg* oferece uma maneira de definir as funções usadas durante a remoção de objetos específicos. Os módulos *pickle* e *copy* usam essas funções ao selecionar/copiar esses objetos. O módulo fornece informações de

configuração sobre construtores de objetos que não são classes. Esses construtores podem ser funções de fábrica ou instâncias de classes.

copyreg.**constructor**(*object*)

Declara *object* para ser um construtor válido. Se *object* não for chamável (e, portanto, não for válido como um construtor), levanta *TypeError*.

copyreg.**pickle**(*type*, *function*, *constructor_ob=None*)

Declara que a *function* deve ser usada como uma função de "redução" para objetos do tipo *type*. *function* deve retornar uma string ou uma tupla contendo entre dois e seis elementos. Veja *dispatch_table* para mais detalhes sobre a interface da *function*.

O parâmetro *constructor_ob* é um recurso herdado e agora é ignorado, mas se passado deve ser UM chamável.

Note que o atributo *dispatch_table* de um objeto pickler ou subclasse de *pickle.Pickler* também podem ser usados para declarar funções de redução.

12.2.1 Exemplo

O exemplo abaixo gostaria de mostrar como registrar uma função de pickle e como ela será usada:

```
>>> import copyreg, copy, pickle
>>> class C:
...     def __init__(self, a):
...         self.a = a
...
>>> def pickle_c(c):
...     print("pickling a C instance...")
...     return C, (c.a,)
...
>>> copyreg.pickle(C, pickle_c)
>>> c = C(1)
>>> d = copy.copy(c)
pickling a C instance...
>>> p = pickle.dumps(c)
pickling a C instance...
```

12.3 `shelve` — Python object persistence

Código-fonte: Lib/shelve.py

A "shelf" is a persistent, dictionary-like object. The difference with "dbm" databases is that the values (not the keys!) in a shelf can be essentially arbitrary Python objects — anything that the *pickle* module can handle. This includes most class instances, recursive data types, and objects containing lots of shared sub-objects. The keys are ordinary strings.

shelve.**open**(*filename*, *flag='c'*, *protocol=None*, *writeback=False*)

Open a persistent dictionary. The filename specified is the base filename for the underlying database. As a side-effect, an extension may be added to the filename and more than one file may be created. By default, the underlying database file is opened for reading and writing. The optional *flag* parameter has the same interpretation as the *flag* parameter of *dbm.open()*.

By default, pickles created with *pickle.DEFAULT_PROTOCOL* are used to serialize values. The version of the pickle protocol can be specified with the *protocol* parameter.

Because of Python semantics, a shelf cannot know when a mutable persistent-dictionary entry is modified. By default modified objects are written *only* when assigned to the shelf (see *Exemplo*). If the optional *writeback* parameter is set to `True`, all entries accessed are also cached in memory, and written back on *sync()* and *close()*; this can make it handier to mutate mutable entries in the persistent dictionary, but, if many entries

are accessed, it can consume vast amounts of memory for the cache, and it can make the close operation very slow since all accessed entries are written back (there is no way to determine which accessed entries are mutable, nor which ones were actually mutated).

Alterado na versão 3.10: *pickle.DEFAULT_PROTOCOL* is now used as the default pickle protocol.

Alterado na versão 3.11: Aceita um *objeto caminho ou similar* como nome de arquivo.

Nota

Do not rely on the shelf being closed automatically; always call *close()* explicitly when you don't need it any more, or use *shelve.open()* as a context manager:

```
with shelve.open('spam') as db:
    db['eggs'] = 'eggs'
```

Aviso

Because the *shelve* module is backed by *pickle*, it is insecure to load a shelf from an untrusted source. Like with pickle, loading a shelf can execute arbitrary code.

Shelf objects support most of methods and operations supported by dictionaries (except copying, constructors and operators `|` and `|=`). This eases the transition from dictionary based scripts to those requiring persistent storage.

Two additional methods are supported:

`Shelf.`**`sync`**`()`

Write back all entries in the cache if the shelf was opened with *writeback* set to *True*. Also empty the cache and synchronize the persistent dictionary on disk, if feasible. This is called automatically when the shelf is closed with *close()*.

`Shelf.`**`close`**`()`

Synchronize and close the persistent *dict* object. Operations on a closed shelf will fail with a *ValueError*.

Ver também

Persistent dictionary recipe with widely supported storage formats and having the speed of native dictionaries.

12.3.1 Restrições

- The choice of which database package will be used (such as *dbm.ndbm* or *dbm.gnu*) depends on which interface is available. Therefore it is not safe to open the database directly using *dbm*. The database is also (unfortunately) subject to the limitations of *dbm*, if it is used — this means that (the pickled representation of) the objects stored in the database should be fairly small, and in rare cases key collisions may cause the database to refuse updates.
- The *shelve* module does not support *concurrent* read/write access to shelved objects. (Multiple simultaneous read accesses are safe.) When a program has a shelf open for writing, no other program should have it open for reading or writing. Unix file locking can be used to solve this, but this differs across Unix versions and requires knowledge about the database implementation used.
- On macOS *dbm.ndbm* can silently corrupt the database file on updates, which can cause hard crashes when trying to read from the database.

`class` `shelve.`**`Shelf`**`(`*dict*, *protocol=None*, *writeback=False*, *keyencoding='utf-8'*`)`

A subclass of *collections.abc.MutableMapping* which stores pickled values in the *dict* object.

By default, pickles created with *pickle.DEFAULT_PROTOCOL* are used to serialize values. The version of the pickle protocol can be specified with the *protocol* parameter. See the *pickle* documentation for a discussion of the pickle protocols.

If the *writeback* parameter is `True`, the object will hold a cache of all entries accessed and write them back to the *dict* at sync and close times. This allows natural operations on mutable entries, but can consume much more memory and make sync and close take a long time.

The *keyencoding* parameter is the encoding used to encode keys before they are used with the underlying dict.

A *Shelf* object can also be used as a context manager, in which case it will be automatically closed when the `with` block ends.

Alterado na versão 3.2: Added the *keyencoding* parameter; previously, keys were always encoded in UTF-8.

Alterado na versão 3.4: Adicionado suporte a gerenciador de contexto.

Alterado na versão 3.10: *pickle.DEFAULT_PROTOCOL* is now used as the default pickle protocol.

class shelve.**BsdDbShelf**(*dict*, *protocol=None*, *writeback=False*, *keyencoding='utf-8'*)

A subclass of *Shelf* which exposes `first()`, `next()`, `previous()`, `last()` and `set_location()` methods. These are available in the third-party `bsddb` module from pybsddb but not in other database modules. The *dict* object passed to the constructor must support those methods. This is generally accomplished by calling one of `bsddb.hashopen()`, `bsddb.btopen()` or `bsddb.rnopen()`. The optional *protocol*, *writeback*, and *keyencoding* parameters have the same interpretation as for the *Shelf* class.

class shelve.**DbfilenameShelf**(*filename*, *flag='c'*, *protocol=None*, *writeback=False*)

A subclass of *Shelf* which accepts a *filename* instead of a dict-like object. The underlying file will be opened using *dbm.open()*. By default, the file will be created and opened for both read and write. The optional *flag* parameter has the same interpretation as for the *open()* function. The optional *protocol* and *writeback* parameters have the same interpretation as for the *Shelf* class.

12.3.2 Exemplo

To summarize the interface (`key` is a string, `data` is an arbitrary object):

```
import shelve

d = shelve.open(filename)  # open -- file may get suffix added by low-level
                           # library

d[key] = data              # store data at key (overwrites old data if
                           # using an existing key)
data = d[key]              # retrieve a COPY of data at key (raise KeyError
                           # if no such key)
del d[key]                 # delete data stored at key (raises KeyError
                           # if no such key)

flag = key in d            # true if the key exists
klist = list(d.keys())     # a list of all existing keys (slow!)

# as d was opened WITHOUT writeback=True, beware:
d['xx'] = [0, 1, 2]        # this works as expected, but...
d['xx'].append(3)          # *this doesn't!* -- d['xx'] is STILL [0, 1, 2]!

# having opened d without writeback=True, you need to code carefully:
temp = d['xx']             # extracts the copy
temp.append(5)             # mutates the copy
d['xx'] = temp             # stores the copy right back, to persist it

# or, d=shelve.open(filename,writeback=True) would let you just code
```

(continua na próxima página)

(continuação da página anterior)

```
# d['xx'].append(5) and have it work as expected, BUT it would also
# consume more memory and make the d.close() operation slower.

d.close()                   # close it
```

Ver também

Módulo *dbm*
Generic interface to `dbm`-style databases.

Módulo *pickle*
Object serialization used by *shelve*.

12.4 `marshal` — Serialização interna de objetos Python

Este módulo contém funções que podem ler e gravar valores Python em formato binário. O formato é específico para Python, mas independente dos problemas de arquitetura da máquina (por exemplo, você pode gravar um valor Python em um arquivo em um PC, transportar o arquivo para um Mac e lê-lo de volta lá). Os detalhes do formato não são documentados propositalmente; ele pode mudar entre as versões do Python (embora raramente mude).[1]

Este não é um módulo de "persistência" geral. Para persistência geral e transferência de objetos Python através de chamadas RPC, veja os módulos *pickle* e *shelve*. O módulo *marshal* existe principalmente para ter suporte à leitura e escrita do código "pseudocompilado" para módulos Python de arquivos `.pyc`. Portanto, os mantenedores do Python se reservam o direito de modificar o formato do marshal de maneiras incompatíveis com versões anteriores, caso seja necessário. O formato dos objetos código não é compatível entre as versões do Python, mesmo que a versão do formato seja a mesma. Desserializar um objeto código na versão incorreta do Python tem um comportamento indefinido. Se você estiver serializando e desserializando objetos Python, use o módulo *pickle* – o desempenho é comparável, a independência de versão é garantida e pickle tem suporte a uma gama substancialmente maior de objetos do que marshal.

Aviso

O módulo *marshal* não se destina a ser seguro contra dados errôneos ou construídos de forma maliciosa. Nunca faça o unmarshalling de dados recebidos de uma fonte não confiável ou não autenticada.

Existem funções que leem/gravam arquivos, bem como funções que operam em objetos bytes ou similares.

Not all Python object types are supported; in general, only objects whose value is independent from a particular invocation of Python can be written and read by this module. The following types are supported:

- Numeric types: *int*, *bool*, *float*, *complex*.
- Strings (*str*) and *bytes*. *Bytes-like objects* like *bytearray* are marshalled as `bytes`.
- Containers: *tuple*, *list*, *set*, *frozenset*, and (since *version* 5), *slice*. It should be understood that these are supported only if the values contained therein are themselves supported. Recursive containers are supported since *version* 3.
- The singletons *None*, *Ellipsis* and *StopIteration*.
- *code* objects, if *allow_code* is true. See note above about version dependence.

Alterado na versão 3.4:

[1] O nome deste módulo deriva de um pouco da terminologia usada pelos designers do Modula-3 (entre outros), que usam o termo "marshalling" para enviar dados em um formato independente. Estritamente falando, "to marshal" significa converter alguns dados da forma interna para a externa (em um buffer RPC, por exemplo) e "unmarshalling" para o processo reverso.

- Added format version 3, which supports marshalling recursive lists, sets and dictionaries.
- Added format version 4, which supports efficient representations of short strings.

Alterado na versão 3.14: Added format version 5, which allows marshalling slices.

O módulo define estas funções:

`marshal.`**`dump`**(*value*, *file*, *version=version*, */*, ***, *allow_code=True*)

Grava o valor no arquivo aberto. O valor deve ser um tipo compatível. O arquivo deve ser *arquivo binário* gravável.

Se o valor tem (ou contém um objeto que tem) um tipo não suportado, uma exceção *ValueError* é levantada – mas dados de lixo também serão gravados no arquivo. O objeto não será lido corretamente por *load()*. Há suporte a objetos código somente se *allow_code* for verdadeiro.

O argumento *version* indica o formato de dados que o `dump` deve usar (veja abaixo).

Levanta um *evento de auditoria* `marshal.dumps` com os argumentos `value`, `version`.

Alterado na versão 3.13: Adicionado o parâmetro *allow_code*.

`marshal.`**`load`**(*file*, */*, ***, *allow_code=True*)

Lê um valor do arquivo aberto e retorna-o. Se nenhum valor válido for lido (por exemplo, porque os dados têm um formato de empacotamento incompatível com uma versão diferente do Python), levanta *EOFError*, *ValueError* ou *TypeError*. Há suporte a objetos código somente se *allow_code* for verdadeiro. O arquivo deve ser um *arquivo binário* legível.

Levanta um *evento de auditoria* `marshal.load` sem argumentos.

> **Nota**
>
> Se um objeto contendo um tipo não suportado foi empacotado com *dump()*, *load()* irá substituir `None` pelo tipo não empacotável.

Alterado na versão 3.10: Esta chamada costumava levantar um evento de auditoria `code.__new__` para cada objeto código. Agora, ele levanta um único evento `marshal.load` para toda a operação de carregamento.

Alterado na versão 3.13: Adicionado o parâmetro *allow_code*.

`marshal.`**`dumps`**(*value*, *version=version*, */*, ***, *allow_code=True*)

Retorna o objeto bytes que seria escrito em um arquivo por `dump(value, file)`. O valor deve ser um tipo compatível. Levanta uma exceção *ValueError* se o valor tem (ou contém um objeto que tem) um tipo não suportado. Há suporte a objetos código somente se *allow_code* for verdadeiro.

O argumento *version* indica o formato de dados que `dumps` deve usar (veja abaixo).

Levanta um *evento de auditoria* `marshal.dumps` com os argumentos `value`, `version`.

Alterado na versão 3.13: Adicionado o parâmetro *allow_code*.

`marshal.`**`loads`**(*bytes*, */*, ***, *allow_code=True*)

Converte o *objeto bytes ou similar* em um valor. Se nenhum valor válido for encontrado, levanta *EOFError*, *ValueError* ou *TypeError*. Há suporte a objetos código somente se *allow_code* for verdadeiro. Bytes extras na entrada são ignorados.

Levanta um *evento de auditoria* `marshal.loads` com o argumento `bytes`.

Alterado na versão 3.10: Esta chamada costumava levantar um evento de auditoria `code.__new__` para cada objeto código. Agora, ele levanta um único evento `marshal.loads` para toda a operação de carregamento.

Alterado na versão 3.13: Adicionado o parâmetro *allow_code*.

Além disso, as seguintes constantes são definidas:

marshal.**version**

Indicates the format that the module uses. Version 0 is the historical first version; subsequent versions add new features. Generally, a new version becomes the default when it is introduced.

Versão	Available since	Novas funcionalidades
1	Python 2.4	Sharing interned strings
2	Python 2.5	Binary representation of floats
3	Python 3.4	Support for object instancing and recursion
4	Python 3.4	Efficient representation of short strings
5	Python 3.14	Support for *slice* objects

12.5 dbm — Interfaces para "banco de dados" Unix

Código-fonte: Lib/dbm/__init__.py

---

dbm é uma interface genérica para variantes do banco de dados DBM:

- *dbm.sqlite3*
- *dbm.gnu*
- *dbm.ndbm*

Se nenhum desses módulos estiverem instalados, a implementação lenta, mas simples, no módulo *dbm.dumb* será usada. Há uma interface de terceiros para o Oracle Berkeley DB.

exception dbm.**error**

Uma tupla contendo as exceções que podem ser levantadas por cada um dos módulos suportados, com a única exceção também chamada *dbm.error* como o primeiro item — o último é usado quando *dbm.error* é levantada.

dbm.**whichdb**(*filename*)

Esta função tenta adivinhar qual dos vários módulos de banco de dados simples disponíveis — *dbm.sqlite3*, *dbm.gnu*, *dbm.ndbm* ou *dbm.dumb* — deve ser usado para abrir o arquivo fornecido.

Retorna um dos seguintes valores:

- `None` se o arquivo não puder ser aberto porque está ilegível ou não existe
- a string vazia (`''`) se o formato do arquivo não puder ser adivinhado
- uma string contendo o nome do módulo necessário, tal como `'dbm.ndbm'` ou `'dbm.gnu'`

Alterado na versão 3.11: *filename* aceita um *objeto caminho ou similar*.

dbm.**open**(*file*, *flag='r'*, *mode=0o666*)

Abre um banco de dados e retorna o objeto banco de dados correspondendo.

Parâmetros

- **file** (*path-like object*) – O arquivo de banco de dados para abrir. Se o arquivo de banco de dados já existe, a função *whichdb()* é usada para determinar seu tipo e o módulo apropriado é usado; se ele não existir, o primeiro submódulo listado acima que pode ser importado é usado.
- **flag** (*str*) –
  - `'r'` (padrão): Open existing database for reading only.
  - `'w'`: Open existing database for reading and writing.
  - `'c'`: Open database for reading and writing, creating it if it doesn't exist.

  - `'n'`: Always create a new, empty database, open for reading and writing.

- **mode** (`int`) – The Unix file access mode of the file (default: octal `0o666`), used only when the database has to be created.

Alterado na versão 3.11: *file* aceita um *objeto caminho ou similar*.

O objeto retornado por *open()* oferece suporte à mesma funcionalidade básica que um *dict*; chaves e seus valores correspondentes podem ser armazenados, obtidos e excluídos, e o operador `in` e o método `keys()` estão disponíveis, bem como métodos `get()` e `setdefault()`.

Chaves e valores são sempre armazenados como *bytes*. Isso significa que quando strings são usadas, elas são implicitamente convertidas para a codificação padrão antes de serem armazenadas.

Estes objetos também oferecem suporte a serem usados em uma instrução `with`, que vai fechá-los automaticamente quando estiver concluída.

Alterado na versão 3.2: O métodos `get()` e `setdefault()` estão agora disponíveis para todos os backends do *dbm*.

Alterado na versão 3.4: Adicionado suporte nativo para o protocolo de gerenciamento de contexto para os objetos retornados por *open()*.

Alterado na versão 3.8: Excluir uma chave de um banco de dados somente leitura levanta uma exceção de banco de dados específica do módulo em vez de *KeyError*.

Os seguintes exemplos registram alguns hostnames e um título correspondente, e então exibe o conteúdo do banco de dados:

```
import dbm

# Abre o banco de dados, criando-o se necessário.
with dbm.open('cache', 'c') as db:

    # Registra alguns valores
    db[b'hello'] = b'there'
    db['www.python.org'] = 'Python Website'
    db['www.cnn.com'] = 'Cable News Network'

    # Note que as chaves são consideradas bytes agora.
    assert db[b'www.python.org'] == b'Python Website'
    # Observe como o valor é agora em bytes.
    assert db['www.cnn.com'] == b'Cable News Network'

    # Métodos geralmente usados da interface dict funcionam também.
    print(db.get('python.org', b'not present'))

    # Armazenar uma chave não-string ou valor  vai levantar uma exceção
    # (provavelmente uma TypeError).
    db['www.yahoo.com'] = 4

# db é automaticamente fechado a sair da instrução with.
```

Ver também

Módulo *shelve*
Módulo persistente que armazena dados não-string.

Os submódulos individuais são descritos nas seções a seguir.

12.5.1 dbm.sqlite3 — Backend de SQLite para dbm

Adicionado na versão 3.13.

Código-fonte: Lib/dbm/sqlite3.py

Este módulo usa o módulo da biblioteca padrão *sqlite3* para fornecer um backend de SQLite para o módulo *dbm*. Os arquivos criados por *dbm.sqlite3* podem então ser abertos por *sqlite3* ou quaisquer outros navegadores de SQLite, incluindo CLI do SQLite.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

dbm.sqlite3.**open**(*filename*, */*, *flag='r'*, *mode=0o666*)

Abre um banco de dados de SQLite. O objeto retornado se comporta como um *mapeamento*, implementa um método `close()` e oferece suporte um gerenciador de contexto de "fechamento" via a palavra reservada `with`.

Parâmetros

- **filename** (*path-like object*) – O caminho para o banco de dados a ser aberta.
- **flag** (`str`) –
  - `'r'` (padrão): Open existing database for reading only.
  - `'w'`: Open existing database for reading and writing.
  - `'c'`: Open database for reading and writing, creating it if it doesn't exist.
  - `'n'`: Always create a new, empty database, open for reading and writing.
- **mode** – O modo de acesso a arquivos do Unix do arquivo (padrão: octal `0o666`), usado apenas quando o banco de dados tem que ser criado.

12.5.2 dbm.gnu — Gerenciador de banco de dados do GNU

Código-fonte: Lib/dbm/gnu.py

O módulo *dbm.gnu* fornece uma interface para a biblioteca GDBM (GNU dbm), semelhante ao módulo *dbm.ndbm*, mas com funcionalidades adicionais, como tolerância a falhas.

> **Nota**
>
> Os formatos de arquivo criados por *dbm.gnu* e *dbm.ndbm* são incompatíveis e não podem ser usados de forma intercambiável.

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em *plataformas móveis* ou *plataformas WebAssembly*.

exception dbm.gnu.**error**

Levantada em erros específicos do *dbm.gnu*, como erros de E/S. *KeyError* é levantada para erros gerais de mapeamento, como especificar uma chave incorreta.

dbm.gnu.**open**(*filename*, *flag='r'*, *mode=0o666*, */*)

Abre um banco de dados GDBM e retorna um objeto `gdbm`.

Parâmetros

- **filename** (*path-like object*) – O arquivo de banco de dados para abrir.

- **flag** (str) –
  - 'r' (padrão): Open existing database for reading only.
  - 'w': Open existing database for reading and writing.
  - 'c': Open database for reading and writing, creating it if it doesn't exist.
  - 'n': Always create a new, empty database, open for reading and writing.

  Os seguintes caracteres adicionais podem ser acrescentados para controlar como o banco de dados é aberto:
  - 'f': Abre o banco de dados em modo rápido. As gravações no banco de dados não serão sincronizadas.
  - 's': Modo sincronizado. Alterações no banco de dados serão gravadas imediatamente no arquivo.
  - 'u': Não trava o banco de dados.

  Nem todos os sinalizadores são válidos para todas as versões do GDBM. Veja o membro *open_flags* para uma lista de caracteres de sinalizadores suportados.
- **mode** (int) – The Unix file access mode of the file (default: octal 0o666), used only when the database has to be created.

Levanta

error – Se um argumento *flag* inválido for passado.

Alterado na versão 3.11: *filename* aceita um *objeto caminho ou similar*.

dbm.gnu.**open_flags**

Uma sequência de caracteres que o parâmetro *flag* de *open()* suporta.

Os objetos gdbm se comportam de forma semelhante a *mapeamentos*, mas os métodos items() e values() não são suportados. Os seguintes métodos também são fornecidos:

gdbm.**firstkey**()

É possível fazer um laço em cada chave no banco de dados usando este método e o método *nextkey()*. A travessia é ordenada pelos valores de hash internos do GDBM e não será classificada pelos valores de chave. Este método retorna a chave inicial.

gdbm.**nextkey**(*key*)

Retorna a chave que segue *key* na travessia. O código a seguir imprime cada chave no banco de dados db, sem precisar criar uma lista na memória que contenha todas elas:

```
k = db.firstkey()
while k is not None:
    print(k)
    k = db.nextkey(k)
```

gdbm.**reorganize**()

Se você realizou muitas exclusões e gostaria de reduzir o espaço usado pelo arquivo GDBM, esta rotina reorganizará o banco de dados. Objetos gdbm não reduzirão o tamanho de um arquivo de banco de dados, exceto usando esta reorganização; caso contrário, o espaço de arquivo excluído será mantido e reutilizado à medida que novos pares (chave, valor) forem adicionados.

gdbm.**sync**()

Quando o banco de dados é aberto no modo rápido, esse método força a gravação de todos os dados não gravados no disco.

gdbm.**close**()

Fecha o banco de dados GDBM.

gdbm.**clear**()

Remove todos os itens do banco de dados GDBM.

Adicionado na versão 3.13.

12.5.3 dbm.ndbm — New Database Manager

Código-fonte: Lib/dbm/ndbm.py

O módulo *dbm.ndbm* fornece uma interface para a biblioteca NDBM (New Database Manager). Este módulo pode ser usado com a interface NDBM "clássica" ou a interface de compatibilidade GDBM.

Nota

Os formatos de arquivo criados por *dbm.gnu* e *dbm.ndbm* são incompatíveis e não podem ser usados de forma intercambiável.

Aviso

A biblioteca NDBM enviada como parte do macOS tem uma limitação não documentada no tamanho dos valores, o que pode resultar em arquivos de banco de dados corrompidos ao armazenar valores maiores que esse limite. Ler esses arquivos corrompidos pode resultar em uma falha grave (falha de segmentação).

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em *plataformas móveis* ou *plataformas WebAssembly*.

exception dbm.ndbm.**error**

Levantada em erros específicos de *dbm.ndbm*, como erros de E/S. *KeyError* é levantada para erros gerais de mapeamento, como especificar uma chave incorreta.

dbm.ndbm.**library**

Nome da biblioteca de implementação NDBM usada.

dbm.ndbm.**open**(*filename*, *flag='r'*, *mode=0o666*, */*)

Abre um banco de dados NDBM e retorna um objeto ndbm.

Parâmetros

- **filename** (*path-like object*) – O nome base do arquivo de banco de dados (sem as extensões .dir ou .pag).
- **flag** (str) –
  - 'r' (padrão): Open existing database for reading only.
  - 'w': Open existing database for reading and writing.
  - 'c': Open database for reading and writing, creating it if it doesn't exist.
  - 'n': Always create a new, empty database, open for reading and writing.
- **mode** (int) – The Unix file access mode of the file (default: octal 0o666), used only when the database has to be created.

Os objetos ndbm se comportam de forma semelhante a *mapeamentos*, mas os métodos items() e values() não são suportados. Os seguintes métodos também são fornecidos:

Alterado na versão 3.11: Aceita um *objeto caminho ou similar* como nome de arquivo.

ndbm.**close**()

Fecha o banco de dados NDBM.

ndbm.**clear**()

Remove todos os itens do banco de dados NDBM.

Adicionado na versão 3.13.

12.5.4 dbm.dumb — Implementação portátil do DBM

Código-fonte: Lib/dbm/dumb.py

> **Nota**
>
> O módulo *dbm.dumb* é pensado como um último recurso alternativo para o módulo *dbm* quando um módulo mais robusto não está disponível. O módulo *dbm.dumb* não é escrito para velocidade e não é tão usado quanto os outros módulos de banco de dados.

O módulo *dbm.dumb* fornece uma interface persistente do tipo *dict* que é escrita inteiramente em Python. Ao contrário de outros backends *dbm*, como *dbm.gnu*, nenhuma biblioteca externa é necessária.

O módulo dbm.dumb define o seguinte:

exception dbm.dumb.**error**

Levantada em erros específicos de *dbm.dumb*, como erros de E/S. *KeyError* é levantada para erros gerais de mapeamento, como especificar uma chave incorreta.

dbm.dumb.**open**(*filename*, *flag='c'*, *mode=0o666*)

Abre um banco de dados dbm.dumb. O objeto banco de dados retornado se comporta de forma semelhante a um *mapeamento*, além de fornecer os métodos *sync()* e *close()*.

Parâmetros

- **filename** – O nome base do arquivo do banco de dados (sem extensões). Um novo banco de dados cria os seguintes arquivos: - *filename*.dat - *filename*.dir
- **flag** (str) –
  - 'r': Open existing database for reading only.
  - 'w': Open existing database for reading and writing.
  - 'c' (padrão): Open database for reading and writing, creating it if it doesn't exist.
  - 'n': Always create a new, empty database, open for reading and writing.
- **mode** (int) – The Unix file access mode of the file (default: octal 0o666), used only when the database has to be created.

> **Aviso**
>
> É possível travar o interpretador Python ao carregar um banco de dados com uma entrada suficientemente grande/complexa devido a limitações de profundidade de pilha no compilador AST do Python.

Alterado na versão 3.5: *open()* sempre cria um novo banco de dados quando *flag* é 'n'.

Alterado na versão 3.8: Um banco de dados aberto somente leitura se *flag* for 'r'. Um banco de dados não é criado se não existir se *flag* for 'r' ou 'w'.

Alterado na versão 3.11: *filename* aceita um *objeto caminho ou similar*.

Além dos métodos fornecidos pela classe *collections.abc.MutableMapping*, os seguintes métodos são fornecidos:

dumbdbm.**sync**()

Sincroniza o diretório no disco e os arquivos de dados. Este método é chamado pelo método *shelve.Shelf.sync()*.

dumbdbm.**close**()

Fecha o banco de dados.

12.6 sqlite3 —- interface DB-API 2.0 interface para bancos de dados SQLite

Código-fonte: Lib/sqlite3/ SQLite é uma biblioteca C que fornece um banco de dados leve baseado em disco que não requer um processo de servidor separado e permite acessar o banco de dados usando uma variante não padrão da linguagem de consulta SQL. Algumas aplicações podem usar SQLite para armazenamento interno de dados. Também é possível prototipar uma aplicação usando SQLite e, em seguida, portar o código para um banco de dados maior, como PostgreSQL ou Oracle.

O módulo `sqlite3` foi escrito por Gerhard Häring. Ele oferece uma interface SQL compatível com a especificação DB-API 2.0 descrita pela **PEP 249** e requer o SQLite 3.15.2 ou o mais recente.

Esse documento inclui quatro seções principais:

- *Tutorial* ensina como usar o módulo `sqlite3`.
- *Referência* descreve as classes e funções que este módulo define.
- *Guias de como fazer* detalha como lidar com tarefas específicas.
- *Explicação* fornece informações detalhadas sobre controle de transações.

Ver também

https://www.sqlite.org
A página web do SQLite; a documentação descreve a sintaxe e os tipos de dados disponíveis para o dialeto SQL suportado.

https://www.w3schools.com/sql/
Tutoriais, referências e exemplos para aprender a sintaxe SQL.

PEP 249 - Especificação 2.0 da API de banco de dados
PEP escrita por Marc-André Lemburg.

12.6.1 Tutorial

Neste tutorial, você criará um banco de dados de filmes do Monty Python usando a funcionalidade básica `sqlite3`. Ele pressupõe uma compreensão fundamental dos conceitos de banco de dados, incluindo cursores e transações.

Primeiro, precisamos criar um novo banco de dados e abrir uma conexão com o banco de dados para permitir que `sqlite3` funcione com ele. Chame *sqlite3.connect()* para criar uma conexão com o banco de dados `tutorial.db` no diretório de trabalho atual, criando-o implicitamente se ele não existir:

```
import sqlite3
con = sqlite3.connect("tutorial.db")
```

O objeto *Connection* `con` retornado representa a conexão com o banco de dados em disco.

Para executar instruções SQL e buscar resultados de consultas SQL, precisaremos usar um cursor de banco de dados. Chame *con.cursor()* para criar o *Cursor*:

```
cur = con.cursor()
```

Agora que temos uma conexão com o banco de dados e um cursor, podemos criar uma tabela de banco de dados `movie` com colunas para título, ano de lançamento e pontuação da revisão. Para simplificar, podemos apenas usar nomes de colunas na declaração da tabela - graças ao recurso tipagem flexível do SQLite, especificar os tipos de dados é opcional. Execute a instrução `CREATE TABLE` chamando *cur.execute(...)*:

```
cur.execute("CREATE TABLE movie(title, year, score)")
```

Podemos verificar se a nova tabela foi criada consultando a tabela embutida `sqlite_master` no SQLite, que agora deve conter uma entrada para a definição da tabela `movie` (veja The Schema Table para detalhes). Execute essa consulta chamando *cur.execute(...)*, atribua o resultado a `res` e chame *res.fetchone()* para buscar a linha resultante:

```
>>> res = cur.execute("SELECT name FROM sqlite_master")
>>> res.fetchone()
('movie',)
```

Podemos ver que a tabela foi criada, pois a consulta retorna *tuple* contendo o nome da tabela. Se fizermos uma consulta `sqlite_master` para uma tabela inexistente `spam`, `res.fetchone()` ela retornará `None`:

```
>>> res = cur.execute("SELECT name FROM sqlite_master WHERE name='spam'")
>>> res.fetchone() is None
True
```

Agora, adicione duas linhas de dados fornecidos como literais SQL executando uma instrução `INSERT`, mais uma vez chamando *cur.execute(...)*:

```
cur.execute("""
    INSERT INTO movie VALUES
        ('Monty Python and the Holy Grail', 1975, 8.2),
        ('And Now for Something Completely Different', 1971, 7.5)
""")
```

A instrução `INSERT` abre implicitamente uma transação, que precisa ser confirmada antes que as alterações sejam salvas no banco de dados (veja *Controle de transações* para detalhes). Chame *con.commit()* no objeto de conexão para confirmar a transação:

```
con.commit()
```

Podemos verificar que os dados foram inseridos corretamente executando uma consulta `SELECT`. Use o já conhecido *cur.execute(...)* para atribuir o resultado a `res` e chame *res.fetchall()* para retornar todas as linhas resultantes.

```
>>> res = cur.execute("SELECT score FROM movie")
>>> res.fetchall()
[(8.2,), (7.5,)]
```

O resultado é uma *list* de duas `tuple`s, uma por linha, cada uma contendo o valor `score` dessa linha.

Agora, insira mais três linhas chamando *cur.executemany(...)*:

```
data = [
    ("Monty Python Live at the Hollywood Bowl", 1982, 7.9),
    ("Monty Python's The Meaning of Life", 1983, 7.5),
    ("Monty Python's Life of Brian", 1979, 8.0),
]
cur.executemany("INSERT INTO movie VALUES(?, ?, ?)", data)
con.commit()  # Lembre-sede executar a transação após o INSERT.
```

Observe que espaços reservados `?` são usados para vincular `data` à consulta. Sempre use espaços reservados em vez de formatação de string para vincular valores Python a instruções SQL, para evitar ataques de injeção de SQL (consulte *How to use placeholders to bind values in SQL queries* para mais detalhes).

Podemos verificar que as novas linhas foram inseridas executando uma consulta `SELECT`, desta vez iterando sobre os resultados da consulta.

```
>>> for row in cur.execute("SELECT year, title FROM movie ORDER BY year"):
...     print(row)
(1971, 'And Now for Something Completely Different')
(1975, 'Monty Python and the Holy Grail')
(1979, "Monty Python's Life of Brian")
(1982, 'Monty Python Live at the Hollywood Bowl')
(1983, "Monty Python's The Meaning of Life")
```

Cada linha é uma *tuple* de dois itens `(year, title)`, correspondendo às colunas selecionadas na consulta.

Finalmente, verifique se o banco de dados foi gravado no disco chamando *con.close()* para fechar a conexão existente, abrir uma nova, criar um novo cursor e, em seguida, consultar o banco de dados.

```
>>> con.close()
>>> new_con = sqlite3.connect("tutorial.db")
>>> new_cur = new_con.cursor()
>>> res = new_cur.execute("SELECT title, year FROM movie ORDER BY score DESC")
>>> title, year = res.fetchone()
>>> print(f'The highest scoring Monty Python movie is {title!r}, released in {year}
↪')
The highest scoring Monty Python movie is 'Monty Python and the Holy Grail',␣
↪released in 1975
>>> new_con.close()
```

Você agora criou um banco de dados SQLite usando o módulo `sqlite3`, inseriu dados e recuperou valores dele de várias maneiras.

Ver também

- *Guias de como fazer* para leitura adicional:
  - *How to use placeholders to bind values in SQL queries*
  - *How to adapt custom Python types to SQLite values*
  - *How to convert SQLite values to custom Python types*
  - *How to use the connection context manager*
  - *How to create and use row factories*
- *Explicação* para obter informações detalhadas sobre o controle de transações.

12.6.2 Referência

Funções do módulo

`sqlite3.`**`connect`**(*database*, *timeout=5.0*, *detect_types=0*, *isolation_level='DEFERRED'*, *check_same_thread=True*, *factory=sqlite3.Connection*, *cached_statements=128*, *uri=False*, ***, *autocommit=sqlite3.LEGACY_TRANSACTION_CONTROL*)

Abra uma conexão com um banco de dados SQLite.

Parâmetros

- **`database`** (*path-like object*) – O caminho para o arquivo do banco de dados a ser aberto.

Você pode passar `":memory:"` para criar um banco de dados SQLite que existirá apenas na memória, e abrir uma conexão com ele.

- **timeout** (`float`) – Quantos segundos a conexão deve aguardar antes de levantar uma exceção *`OperationalError`* quando uma tabela estiver bloqueada. Se outra conexão abrir uma transação para modificar uma tabela, essa tabela permanecerá bloqueada até que a transação seja confirmada. O padrão é cinco segundos.
- **detect_types** (`int`) – Control whether and how data types not *natively supported by SQLite* are looked up to be converted to Python types, using the converters registered with *`register_converter()`*. Set it to any combination (using `|`, bitwise or) of *`PARSE_DECLTYPES`* and *`PARSE_COLNAMES`* to enable this. Column names takes precedence over declared types if both flags are set. By default (`0`), type detection is disabled.
- **isolation_level** (`str | None`) – Control legacy transaction handling behaviour. See *`Connection.isolation_level`* and *Controle de transação através do atributo isolation_level* for more information. Can be `"DEFERRED"` (default), `"EXCLUSIVE"` or `"IMMEDIATE"`; or `None` to disable opening transactions implicitly. Has no effect unless *`Connection.autocommit`* is set to *`LEGACY_TRANSACTION_CONTROL`* (the default).
- **check_same_thread** (`bool`) – If `True` (default), *`ProgrammingError`* will be raised if the database connection is used by a thread other than the one that created it. If `False`, the connection may be accessed in multiple threads; write operations may need to be serialized by the user to avoid data corruption. See *`threadsafety`* for more information.
- **factory** (`Connection`) – A custom subclass of *`Connection`* to create the connection with, if not the default *`Connection`* class.
- **cached_statements** (`int`) – The number of statements that `sqlite3` should internally cache for this connection, to avoid parsing overhead. By default, 128 statements.
- **uri** (`bool`) – If set to `True`, *database* is interpreted as a URI (Uniform Resource Identifier) with a file path and an optional query string. The scheme part *must* be `"file:"`, and the path can be relative or absolute. The query string allows passing parameters to SQLite, enabling various *How to work with SQLite URIs*.
- **autocommit** (`bool`) – Control **PEP 249** transaction handling behaviour. See *`Connection.autocommit`* and *Controle de transações através do atributo autocommit* for more information. *autocommit* currently defaults to *`LEGACY_TRANSACTION_CONTROL`*. The default will change to `False` in a future Python release.

Tipo de retorno
Connection

Levanta um *evento de auditoria* `sqlite3.connect` com o argumento `database`.

Levanta um *evento de auditoria* `sqlite3.connect/handle` com o argumento `connection_handle`.

Alterado na versão 3.4: Adicionado o parâmetro *uri*.

Alterado na versão 3.7: *database* can now also be a *path-like object*, not only a string.

Alterado na versão 3.10: Added the `sqlite3.connect/handle` auditing event.

Alterado na versão 3.12: Added the *autocommit* parameter.

Alterado na versão 3.13: Positional use of the parameters *timeout*, *detect_types*, *isolation_level*, *check_same_thread*, *factory*, *cached_statements*, and *uri* is deprecated. They will become keyword-only parameters in Python 3.15.

`sqlite3.`**complete_statement**(*statement*)

Return `True` if the string *statement* appears to contain one or more complete SQL statements. No syntactic verification or parsing of any kind is performed, other than checking that there are no unclosed string literals and the statement is terminated by a semicolon.

Por exemplo:

```
>>> sqlite3.complete_statement("SELECT foo FROM bar;")
True
>>> sqlite3.complete_statement("SELECT foo")
False
```

This function may be useful during command-line input to determine if the entered text seems to form a complete SQL statement, or if additional input is needed before calling *execute()*.

See `runsource()` in Lib/sqlite3/__main__.py for real-world use.

`sqlite3.`**`enable_callback_tracebacks`**(*flag*, `/`)

Enable or disable callback tracebacks. By default you will not get any tracebacks in user-defined functions, aggregates, converters, authorizer callbacks etc. If you want to debug them, you can call this function with *flag* set to `True`. Afterwards, you will get tracebacks from callbacks on *sys.stderr*. Use `False` to disable the feature again.

> **Nota**
>
> Errors in user-defined function callbacks are logged as unraisable exceptions. Use an *unraisable hook handler* for introspection of the failed callback.

`sqlite3.`**`register_adapter`**(*type*, *adapter*, `/`)

Register an *adapter* *callable* to adapt the Python type *type* into an SQLite type. The adapter is called with a Python object of type *type* as its sole argument, and must return a value of a *type that SQLite natively understands*.

`sqlite3.`**`register_converter`**(*typename*, *converter*, `/`)

Register the *converter* *callable* to convert SQLite objects of type *typename* into a Python object of a specific type. The converter is invoked for all SQLite values of type *typename*; it is passed a *bytes* object and should return an object of the desired Python type. Consult the parameter *detect_types* of *connect()* for information regarding how type detection works.

Note: *typename* and the name of the type in your query are matched case-insensitively.

Constantes do módulo

`sqlite3.`**`LEGACY_TRANSACTION_CONTROL`**

Set *autocommit* to this constant to select old style (pre-Python 3.12) transaction control behaviour. See *Controle de transação através do atributo isolation_level* for more information.

`sqlite3.`**`PARSE_DECLTYPES`**

Pass this flag value to the *detect_types* parameter of *connect()* to look up a converter function using the declared types for each column. The types are declared when the database table is created. `sqlite3` will look up a converter function using the first word of the declared type as the converter dictionary key. For example:

```
CREATE TABLE test(
   i integer primary key,   ! will look up a converter named "integer"
   p point,                 ! will look up a converter named "point"
   n number(10)             ! will look up a converter named "number"
 )
```

This flag may be combined with *PARSE_COLNAMES* using the `|` (bitwise or) operator.

> **Nota**
>
> Generated fields (for example `MAX(p)`) are returned as *str*. Use `PARSE_COLNAMES` to enforce types for such queries.

sqlite3.**PARSE_COLNAMES**

Pass this flag value to the *detect_types* parameter of *connect()* to look up a converter function by using the type name, parsed from the query column name, as the converter dictionary key. The query column name must be wrapped in double quotes (") and the type name must be wrapped in square brackets ([]).

```
SELECT MAX(p) as "p [point]" FROM test;  ! will look up converter "point"
```

This flag may be combined with *PARSE_DECLTYPES* using the | (bitwise or) operator.

sqlite3.**SQLITE_OK**

sqlite3.**SQLITE_DENY**

sqlite3.**SQLITE_IGNORE**

Flags that should be returned by the *authorizer_callback* *callable* passed to *Connection.set_authorizer()*, to indicate whether:

- Access is allowed (SQLITE_OK),
- The SQL statement should be aborted with an error (SQLITE_DENY)
- The column should be treated as a NULL value (SQLITE_IGNORE)

sqlite3.**apilevel**

String constant stating the supported DB-API level. Required by the DB-API. Hard-coded to "2.0".

sqlite3.**paramstyle**

String constant stating the type of parameter marker formatting expected by the sqlite3 module. Required by the DB-API. Hard-coded to "qmark".

> **Nota**
>
> The named DB-API parameter style is also supported.

sqlite3.**sqlite_version**

Version number of the runtime SQLite library as a *string*.

sqlite3.**sqlite_version_info**

Version number of the runtime SQLite library as a *tuple* of *integers*.

sqlite3.**threadsafety**

Integer constant required by the DB-API 2.0, stating the level of thread safety the sqlite3 module supports. This attribute is set based on the default threading mode the underlying SQLite library is compiled with. The SQLite threading modes are:

1. **Single-thread**: In this mode, all mutexes are disabled and SQLite is unsafe to use in more than a single thread at once.
2. **Multi-thread**: In this mode, SQLite can be safely used by multiple threads provided that no single database connection is used simultaneously in two or more threads.
3. **Serialized**: In serialized mode, SQLite can be safely used by multiple threads with no restriction.

The mappings from SQLite threading modes to DB-API 2.0 threadsafety levels are as follows:

SQLite threading mode	threadsafety	SQLITE_THREADSAFE	DB-API 2.0 meaning
single-thread	0	0	Threads may not share the module
multi-thread	1	2	Threads may share the module, but not connections
serialized	3	1	Threads may share the module, connections and cursors

Alterado na versão 3.11: Set *threadsafety* dynamically instead of hard-coding it to `1`.

sqlite3.**SQLITE_DBCONFIG_DEFENSIVE**

sqlite3.**SQLITE_DBCONFIG_DQS_DDL**

sqlite3.**SQLITE_DBCONFIG_DQS_DML**

sqlite3.**SQLITE_DBCONFIG_ENABLE_FKEY**

sqlite3.**SQLITE_DBCONFIG_ENABLE_FTS3_TOKENIZER**

sqlite3.**SQLITE_DBCONFIG_ENABLE_LOAD_EXTENSION**

sqlite3.**SQLITE_DBCONFIG_ENABLE_QPSG**

sqlite3.**SQLITE_DBCONFIG_ENABLE_TRIGGER**

sqlite3.**SQLITE_DBCONFIG_ENABLE_VIEW**

sqlite3.**SQLITE_DBCONFIG_LEGACY_ALTER_TABLE**

sqlite3.**SQLITE_DBCONFIG_LEGACY_FILE_FORMAT**

sqlite3.**SQLITE_DBCONFIG_NO_CKPT_ON_CLOSE**

sqlite3.**SQLITE_DBCONFIG_RESET_DATABASE**

sqlite3.**SQLITE_DBCONFIG_TRIGGER_EQP**

sqlite3.**SQLITE_DBCONFIG_TRUSTED_SCHEMA**

sqlite3.**SQLITE_DBCONFIG_WRITABLE_SCHEMA**

These constants are used for the *Connection.setconfig()* and *getconfig()* methods.

The availability of these constants varies depending on the version of SQLite Python was compiled with.

Adicionado na versão 3.12.

Ver também

https://www.sqlite.org/c3ref/c_dbconfig_defensive.html
SQLite docs: Database Connection Configuration Options

Deprecated since version 3.12, removed in version 3.14: The `version` and `version_info` constants.

Connection objects

class sqlite3.**Connection**

Each open SQLite database is represented by a `Connection` object, which is created using *sqlite3.connect()*. Their main purpose is creating *Cursor* objects, and *Controle de transações*.

Ver também

- *How to use connection shortcut methods*
- *How to use the connection context manager*

Alterado na versão 3.13: A *ResourceWarning* is emitted if *close()* is not called before a `Connection` object is deleted.

An SQLite database connection has the following attributes and methods:

cursor(*factory=Cursor*)

Create and return a *Cursor* object. The cursor method accepts a single optional parameter *factory*. If supplied, this must be a *callable* returning an instance of *Cursor* or its subclasses.

blobopen(*table*, *column*, *row*, */*, ***, *readonly=False*, *name='main'*)

Open a *Blob* handle to an existing BLOB (Binary Large OBject).

Parâmetros

- **table** (str) – The name of the table where the blob is located.
- **column** (str) – The name of the column where the blob is located.
- **row** (str) – The name of the row where the blob is located.
- **readonly** (bool) – Set to `True` if the blob should be opened without write permissions. Defaults to `False`.
- **name** (str) – The name of the database where the blob is located. Defaults to `"main"`.

Levanta

OperationalError – When trying to open a blob in a `WITHOUT ROWID` table.

Tipo de retorno

Blob

> **Nota**
>
> The blob size cannot be changed using the *Blob* class. Use the SQL function `zeroblob` to create a blob with a fixed size.

Adicionado na versão 3.11.

commit()

Commit any pending transaction to the database. If *autocommit* is `True`, or there is no open transaction, this method does nothing. If `autocommit` is `False`, a new transaction is implicitly opened if a pending transaction was committed by this method.

rollback()

Roll back to the start of any pending transaction. If *autocommit* is `True`, or there is no open transaction, this method does nothing. If `autocommit` is `False`, a new transaction is implicitly opened if a pending transaction was rolled back by this method.

close()

Close the database connection. If *autocommit* is `False`, any pending transaction is implicitly rolled back. If `autocommit` is `True` or *LEGACY_TRANSACTION_CONTROL*, no implicit transaction control is executed. Make sure to *commit()* before closing to avoid losing pending changes.

execute(*sql*, *parameters=()*, */*)

Create a new *Cursor* object and call *execute()* on it with the given *sql* and *parameters*. Return the new cursor object.

executemany(*sql*, *parameters*, */*)

Create a new *Cursor* object and call *executemany()* on it with the given *sql* and *parameters*. Return the new cursor object.

executescript(*sql_script*, */*)

Create a new *Cursor* object and call *executescript()* on it with the given *sql_script*. Return the new cursor object.

create_function(*name*, *narg*, *func*, ***, *deterministic=False*)

Create or remove a user-defined SQL function.

Parâmetros

- **name** (str) – O nome da função SQL.
- **narg** (int) – The number of arguments the SQL function can accept. If `-1`, it may take any number of arguments.

- **func** (*callback* | None) – A *callable* that is called when the SQL function is invoked. The callable must return *a type natively supported by SQLite*. Set to `None` to remove an existing SQL function.
- **deterministic** (`bool`) – If `True`, the created SQL function is marked as deterministic, which allows SQLite to perform additional optimizations.

Alterado na versão 3.8: Added the *deterministic* parameter.

Exemplo:

```
>>> import hashlib
>>> def md5sum(t):
...     return hashlib.md5(t).hexdigest()
>>> con = sqlite3.connect(":memory:")
>>> con.create_function("md5", 1, md5sum)
>>> for row in con.execute("SELECT md5(?)", (b"foo",)):
...     print(row)
('acbd18db4cc2f85cedef654fccc4a4d8',)
>>> con.close()
```

Alterado na versão 3.13: Passing *name*, *narg*, and *func* as keyword arguments is deprecated. These parameters will become positional-only in Python 3.15.

create_aggregate(*name*, *n_arg*, *aggregate_class*)

Create or remove a user-defined SQL aggregate function.

Parâmetros

- **name** (`str`) – The name of the SQL aggregate function.
- **n_arg** (`int`) – The number of arguments the SQL aggregate function can accept. If `-1`, it may take any number of arguments.
- **aggregate_class** (*class* | None) – Uma classe deve implementar os seguintes métodos:
  - `step()`: Add a row to the aggregate.
  - `finalize()`: Return the final result of the aggregate as *a type natively supported by SQLite*.

  The number of arguments that the `step()` method must accept is controlled by *n_arg*.

  Set to `None` to remove an existing SQL aggregate function.

Exemplo:

```
class MySum:
    def __init__(self):
        self.count = 0

    def step(self, value):
        self.count += value

    def finalize(self):
        return self.count

con = sqlite3.connect(":memory:")
con.create_aggregate("mysum", 1, MySum)
cur = con.execute("CREATE TABLE test(i)")
cur.execute("INSERT INTO test(i) VALUES(1)")
cur.execute("INSERT INTO test(i) VALUES(2)")
cur.execute("SELECT mysum(i) FROM test")
```

(continua na próxima página)

(continuação da página anterior)

```
print(cur.fetchone()[0])

con.close()
```

Alterado na versão 3.13: Passing *name*, *n_arg*, and *aggregate_class* as keyword arguments is deprecated. These parameters will become positional-only in Python 3.15.

create_window_function(*name*, *num_params*, *aggregate_class*, */*)

Create or remove a user-defined aggregate window function.

Parâmetros

- **name** (str) – The name of the SQL aggregate window function to create or remove.
- **num_params** (int) – The number of arguments the SQL aggregate window function can accept. If `-1`, it may take any number of arguments.
- **aggregate_class** (*class* | None) – Uma classe que deve implementar os seguintes métodos:
  - `step()`: Add a row to the current window.
  - `value()`: Return the current value of the aggregate.
  - `inverse()`: Remove a row from the current window.
  - `finalize()`: Return the final result of the aggregate as *a type natively supported by SQLite*.

  The number of arguments that the `step()` and `value()` methods must accept is controlled by *num_params*.

  Set to `None` to remove an existing SQL aggregate window function.

Levanta

NotSupportedError – If used with a version of SQLite older than 3.25.0, which does not support aggregate window functions.

Adicionado na versão 3.11.

Exemplo:

```
# Example taken from https://www.sqlite.org/windowfunctions.html#udfwinfunc
class WindowSumInt:
    def __init__(self):
        self.count = 0

    def step(self, value):
        """Add a row to the current window."""
        self.count += value

    def value(self):
        """Return the current value of the aggregate."""
        return self.count

    def inverse(self, value):
        """Remove a row from the current window."""
        self.count -= value

    def finalize(self):
        """Return the final value of the aggregate.

        Any clean-up actions should be placed here.
```

(continua na próxima página)

(continuação da página anterior)

```
        """
        return self.count


con = sqlite3.connect(":memory:")
cur = con.execute("CREATE TABLE test(x, y)")
values = [
    ("a", 4),
    ("b", 5),
    ("c", 3),
    ("d", 8),
    ("e", 1),
]
cur.executemany("INSERT INTO test VALUES(?, ?)", values)
con.create_window_function("sumint", 1, WindowSumInt)
cur.execute("""
    SELECT x, sumint(y) OVER (
        ORDER BY x ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) AS sum_y
    FROM test ORDER BY x
""")
print(cur.fetchall())
con.close()
```

create_collation(*name*, *callable*, */*)

Create a collation named *name* using the collating function *callable*. *callable* is passed two *string* arguments, and it should return an *integer*:

- `1` if the first is ordered higher than the second
- `-1` if the first is ordered lower than the second
- `0` if they are ordered equal

The following example shows a reverse sorting collation:

```
def collate_reverse(string1, string2):
    if string1 == string2:
        return 0
    elif string1 < string2:
        return 1
    else:
        return -1

con = sqlite3.connect(":memory:")
con.create_collation("reverse", collate_reverse)

cur = con.execute("CREATE TABLE test(x)")
cur.executemany("INSERT INTO test(x) VALUES(?)", [("a",), ("b",)])
cur.execute("SELECT x FROM test ORDER BY x COLLATE reverse")
for row in cur:
    print(row)
con.close()
```

Remove a collation function by setting *callable* to `None`.

Alterado na versão 3.11: The collation name can contain any Unicode character. Earlier, only ASCII characters were allowed.

interrupt()

Call this method from a different thread to abort any queries that might be executing on the connection. Aborted queries will raise an *OperationalError*.

set_authorizer(*authorizer_callback*)

Register *callable* *authorizer_callback* to be invoked for each attempt to access a column of a table in the database. The callback should return one of *SQLITE_OK*, *SQLITE_DENY*, or *SQLITE_IGNORE* to signal how access to the column should be handled by the underlying SQLite library.

The first argument to the callback signifies what kind of operation is to be authorized. The second and third argument will be arguments or `None` depending on the first argument. The 4th argument is the name of the database ("main", "temp", etc.) if applicable. The 5th argument is the name of the inner-most trigger or view that is responsible for the access attempt or `None` if this access attempt is directly from input SQL code.

Please consult the SQLite documentation about the possible values for the first argument and the meaning of the second and third argument depending on the first one. All necessary constants are available in the `sqlite3` module.

Passing `None` as *authorizer_callback* will disable the authorizer.

Alterado na versão 3.11: Added support for disabling the authorizer using `None`.

Alterado na versão 3.13: Passing *authorizer_callback* as a keyword argument is deprecated. The parameter will become positional-only in Python 3.15.

set_progress_handler(*progress_handler*, *n*)

Register *callable* *progress_handler* to be invoked for every *n* instructions of the SQLite virtual machine. This is useful if you want to get called from SQLite during long-running operations, for example to update a GUI.

If you want to clear any previously installed progress handler, call the method with `None` for *progress_handler*.

Returning a non-zero value from the handler function will terminate the currently executing query and cause it to raise a *DatabaseError* exception.

Alterado na versão 3.13: Passing *progress_handler* as a keyword argument is deprecated. The parameter will become positional-only in Python 3.15.

set_trace_callback(*trace_callback*)

Register *callable* *trace_callback* to be invoked for each SQL statement that is actually executed by the SQLite backend.

The only argument passed to the callback is the statement (as *str*) that is being executed. The return value of the callback is ignored. Note that the backend does not only run statements passed to the *Cursor.execute()* methods. Other sources include the *transaction management* of the `sqlite3` module and the execution of triggers defined in the current database.

Passing `None` as *trace_callback* will disable the trace callback.

Nota

Exceptions raised in the trace callback are not propagated. As a development and debugging aid, use *enable_callback_tracebacks()* to enable printing tracebacks from exceptions raised in the trace callback.

Adicionado na versão 3.3.

Alterado na versão 3.13: Passing *trace_callback* as a keyword argument is deprecated. The parameter will become positional-only in Python 3.15.

enable_load_extension(*enabled*, */*)

Enable the SQLite engine to load SQLite extensions from shared libraries if *enabled* is `True`; else, disallow loading SQLite extensions. SQLite extensions can define new functions, aggregates or whole new virtual table implementations. One well-known extension is the fulltext-search extension distributed with SQLite.

> **Nota**
>
> The `sqlite3` module is not built with loadable extension support by default, because some platforms (notably macOS) have SQLite libraries which are compiled without this feature. To get loadable extension support, you must pass the `--enable-loadable-sqlite-extensions` option to **configure**.

Levanta um *evento de auditoria* `sqlite3.enable_load_extension` com os argumentos `connection`, `enabled`.

Adicionado na versão 3.2.

Alterado na versão 3.10: Added the `sqlite3.enable_load_extension` auditing event.

```
con.enable_load_extension(True)

# Load the fulltext search extension
con.execute("select load_extension('./fts3.so')")

# alternatively you can load the extension using an API call:
# con.load_extension("./fts3.so")

# disable extension loading again
con.enable_load_extension(False)

# example from SQLite wiki
con.execute("CREATE VIRTUAL TABLE recipe USING fts3(name, ingredients)")
con.executescript("""
    INSERT INTO recipe (name, ingredients) VALUES('broccoli stew',
↪'broccoli peppers cheese tomatoes');
    INSERT INTO recipe (name, ingredients) VALUES('pumpkin stew', 'pumpkin␣
↪onions garlic celery');
    INSERT INTO recipe (name, ingredients) VALUES('broccoli pie',
↪'broccoli cheese onions flour');
    INSERT INTO recipe (name, ingredients) VALUES('pumpkin pie', 'pumpkin␣
↪sugar flour butter');
    """)
for row in con.execute("SELECT rowid, name, ingredients FROM recipe WHERE␣
↪name MATCH 'pie'"):
    print(row)
```

load_extension(*path*, */*, ***, *entrypoint=None*)

Load an SQLite extension from a shared library. Enable extension loading with *enable_load_extension()* before calling this method.

Parâmetros

- **path** (`str`) – The path to the SQLite extension.
- **entrypoint** (`str | None`) – Entry point name. If `None` (the default), SQLite will come up with an entry point name of its own; see the SQLite docs Loading an Extension for details.

Levanta um *evento de auditoria* `sqlite3.load_extension` com os argumentos `connection`, `path`.

Adicionado na versão 3.2.

Alterado na versão 3.10: Added the `sqlite3.load_extension` auditing event.

Alterado na versão 3.12: Added the *entrypoint* parameter.

iterdump(*, *filter=None*)

Return an *iterator* to dump the database as SQL source code. Useful when saving an in-memory database for later restoration. Similar to the `.dump` command in the **sqlite3** shell.

Parâmetros

filter (*str | None*) – An optional `LIKE` pattern for database objects to dump, e.g. `prefix_%`. If `None` (the default), all database objects will be included.

Exemplo:

```
# Convert file example.db to SQL dump file dump.sql
con = sqlite3.connect('example.db')
with open('dump.sql', 'w') as f:
    for line in con.iterdump():
        f.write('%s\n' % line)
con.close()
```

Ver também

How to handle non-UTF-8 text encodings

Alterado na versão 3.13: Adicionado o parâmetro *filter*.

backup(*target*, *, *pages=-1*, *progress=None*, *name='main'*, *sleep=0.250*)

Create a backup of an SQLite database.

Works even if the database is being accessed by other clients or concurrently by the same connection.

Parâmetros

- **target** (`Connection`) – The database connection to save the backup to.
- **pages** (`int`) – The number of pages to copy at a time. If equal to or less than `0`, the entire database is copied in a single step. Defaults to `-1`.
- **progress** (*callback* | None) – If set to a *callable*, it is invoked with three integer arguments for every backup iteration: the *status* of the last iteration, the *remaining* number of pages still to be copied, and the *total* number of pages. Defaults to `None`.
- **name** (`str`) – The name of the database to back up. Either `"main"` (the default) for the main database, `"temp"` for the temporary database, or the name of a custom database as attached using the `ATTACH DATABASE` SQL statement.
- **sleep** (`float`) – The number of seconds to sleep between successive attempts to back up remaining pages.

Example 1, copy an existing database into another:

```
def progress(status, remaining, total):
    print(f'Copied {total-remaining} of {total} pages...')

src = sqlite3.connect('example.db')
dst = sqlite3.connect('backup.db')
with dst:
    src.backup(dst, pages=1, progress=progress)
dst.close()
src.close()
```

Example 2, copy an existing database into a transient copy:

```
src = sqlite3.connect('example.db')
dst = sqlite3.connect(':memory:')
src.backup(dst)
dst.close()
src.close()
```

Adicionado na versão 3.7.

Ver também

How to handle non-UTF-8 text encodings

getlimit(*category*, */*)

Get a connection runtime limit.

Parâmetros

category (int) – The SQLite limit category to be queried.

Tipo de retorno

int

Levanta

ProgrammingError – If *category* is not recognised by the underlying SQLite library.

Example, query the maximum length of an SQL statement for *Connection* con (the default is 1000000000):

```
>>> con.getlimit(sqlite3.SQLITE_LIMIT_SQL_LENGTH)
1000000000
```

Adicionado na versão 3.11.

setlimit(*category*, *limit*, */*)

Set a connection runtime limit. Attempts to increase a limit above its hard upper bound are silently truncated to the hard upper bound. Regardless of whether or not the limit was changed, the prior value of the limit is returned.

Parâmetros

- **category** (int) – The SQLite limit category to be set.
- **limit** (int) – The value of the new limit. If negative, the current limit is unchanged.

Tipo de retorno

int

Levanta

ProgrammingError – If *category* is not recognised by the underlying SQLite library.

Example, limit the number of attached databases to 1 for *Connection* con (the default limit is 10):

```
>>> con.setlimit(sqlite3.SQLITE_LIMIT_ATTACHED, 1)
10
>>> con.getlimit(sqlite3.SQLITE_LIMIT_ATTACHED)
1
```

Adicionado na versão 3.11.

getconfig(*op*, */*)

Query a boolean connection configuration option.

Parâmetros
op (int) – A *SQLITE_DBCONFIG code*.

Tipo de retorno
bool

Adicionado na versão 3.12.

setconfig(*op*, *enable=True*, */*)

Set a boolean connection configuration option.

Parâmetros

- **op** (int) – A *SQLITE_DBCONFIG code*.
- **enable** (bool) – `True` if the configuration option should be enabled (default); `False` if it should be disabled.

Adicionado na versão 3.12.

serialize(***, *name='main'*)

Serialize a database into a *bytes* object. For an ordinary on-disk database file, the serialization is just a copy of the disk file. For an in-memory database or a "temp" database, the serialization is the same sequence of bytes which would be written to disk if that database were backed up to disk.

Parâmetros
name (str) – The database name to be serialized. Defaults to `"main"`.

Tipo de retorno
bytes

> **Nota**
>
> This method is only available if the underlying SQLite library has the serialize API.

Adicionado na versão 3.11.

deserialize(*data*, */*, ***, *name='main'*)

Deserialize a *serialized* database into a *Connection*. This method causes the database connection to disconnect from database *name*, and reopen *name* as an in-memory database based on the serialization contained in *data*.

Parâmetros

- **data** (bytes) – A serialized database.
- **name** (str) – The database name to deserialize into. Defaults to `"main"`.

Levanta

- ***OperationalError*** – If the database connection is currently involved in a read transaction or a backup operation.
- ***DatabaseError*** – If *data* does not contain a valid SQLite database.
- ***OverflowError*** – If *len(data)* is larger than `2**63 - 1`.

> **Nota**
>
> This method is only available if the underlying SQLite library has the deserialize API.

Adicionado na versão 3.11.

autocommit

This attribute controls **PEP 249**-compliant transaction behaviour. `autocommit` has three allowed values:

- `False`: Select **PEP 249**-compliant transaction behaviour, implying that `sqlite3` ensures a transaction is always open. Use *`commit()`* and *`rollback()`* to close transactions.

  This is the recommended value of `autocommit`.

- `True`: Use SQLite's autocommit mode. *`commit()`* and *`rollback()`* have no effect in this mode.

- *`LEGACY_TRANSACTION_CONTROL`*: Pre-Python 3.12 (non-**PEP 249**-compliant) transaction control. See *`isolation_level`* for more details.

  This is currently the default value of `autocommit`.

Changing `autocommit` to `False` will open a new transaction, and changing it to `True` will commit any pending transaction.

See *Controle de transações através do atributo autocommit* for more details.

> **Nota**
>
> The *`isolation_level`* attribute has no effect unless *`autocommit`* is *`LEGACY_TRANSACTION_CONTROL`*.

Adicionado na versão 3.12.

in_transaction

This read-only attribute corresponds to the low-level SQLite autocommit mode.

`True` if a transaction is active (there are uncommitted changes), `False` otherwise.

Adicionado na versão 3.2.

isolation_level

Controls the *legacy transaction handling mode* of `sqlite3`. If set to `None`, transactions are never implicitly opened. If set to one of `"DEFERRED"`, `"IMMEDIATE"`, or `"EXCLUSIVE"`, corresponding to the underlying SQLite transaction behaviour, *implicit transaction management* is performed.

If not overridden by the *isolation_level* parameter of *`connect()`*, the default is `""`, which is an alias for `"DEFERRED"`.

> **Nota**
>
> Using *`autocommit`* to control transaction handling is recommended over using `isolation_level`. `isolation_level` has no effect unless *`autocommit`* is set to *`LEGACY_TRANSACTION_CONTROL`* (the default).

row_factory

The initial *`row_factory`* for *`Cursor`* objects created from this connection. Assigning to this attribute does not affect the `row_factory` of existing cursors belonging to this connection, only new ones. Is `None` by default, meaning each row is returned as a *`tuple`*.

Consulte *How to create and use row factories* para mais detalhes.

text_factory

A *callable* that accepts a *`bytes`* parameter and returns a text representation of it. The callable is invoked for SQLite values with the `TEXT` data type. By default, this attribute is set to *`str`*.

Consulte *How to handle non-UTF-8 text encodings* para mais detalhes.

total_changes

Return the total number of database rows that have been modified, inserted, or deleted since the database connection was opened.

Cursor objects

A `Cursor` object represents a database cursor which is used to execute SQL statements, and manage the context of a fetch operation. Cursors are created using `Connection.cursor()`, or by using any of the *connection shortcut methods*.

Cursor objects are *iterators*, meaning that if you `execute()` a `SELECT` query, you can simply iterate over the cursor to fetch the resulting rows:

```
for row in cur.execute("SELECT t FROM data"):
    print(row)
```

class sqlite3.**Cursor**

A `Cursor` instance has the following attributes and methods.

execute(*sql*, *parameters=()*, */*)

Execute a single SQL statement, optionally binding Python values using *placeholders*.

Parâmetros

- **sql** (`str`) – A single SQL statement.
- **parameters** (*dict* | *sequence*) – Python values to bind to placeholders in *sql*. A `dict` if named placeholders are used. A sequence if unnamed placeholders are used. See *How to use placeholders to bind values in SQL queries*.

Levanta

ProgrammingError – When *sql* contains more than one SQL statement. When *named placeholders* are used and *parameters* is a sequence instead of a `dict`.

If `autocommit` is `LEGACY_TRANSACTION_CONTROL`, `isolation_level` is not `None`, *sql* is an `INSERT`, `UPDATE`, `DELETE`, or `REPLACE` statement, and there is no open transaction, a transaction is implicitly opened before executing *sql*.

Alterado na versão 3.14: `ProgrammingError` is emitted if *named placeholders* are used and *parameters* is a sequence instead of a `dict`.

Use `executescript()` to execute multiple SQL statements.

executemany(*sql*, *parameters*, */*)

For every item in *parameters*, repeatedly execute the *parameterized* DML (Data Manipulation Language) SQL statement *sql*.

Uses the same implicit transaction handling as `execute()`.

Parâmetros

- **sql** (`str`) – A single SQL DML statement.
- **parameters** (*iterable*) – An iterable of parameters to bind with the placeholders in *sql*. See *How to use placeholders to bind values in SQL queries*.

Levanta

ProgrammingError – When *sql* contains more than one SQL statement or is not a DML statement, When *named placeholders* are used and the items in *parameters* are sequences instead of `dict`s.

Exemplo:

```
rows = [
    ("row1",),
    ("row2",),
]
# cur is an sqlite3.Cursor object
cur.executemany("INSERT INTO data VALUES(?)", rows)
```

Nota

Any resulting rows are discarded, including DML statements with RETURNING clauses.

Alterado na versão 3.14: *ProgrammingError* is emitted if *named placeholders* are used and the items in *parameters* are sequences instead of *dict*s.

executescript(*sql_script*, /)

Execute the SQL statements in *sql_script*. If the *autocommit* is *LEGACY_TRANSACTION_CONTROL* and there is a pending transaction, an implicit `COMMIT` statement is executed first. No other implicit transaction control is performed; any transaction control must be added to *sql_script*.

sql_script must be a *string*.

Exemplo:

```
# cur is an sqlite3.Cursor object
cur.executescript("""
    BEGIN;
    CREATE TABLE person(firstname, lastname, age);
    CREATE TABLE book(title, author, published);
    CREATE TABLE publisher(name, address);
    COMMIT;
""")
```

fetchone()

If *row_factory* is `None`, return the next row query result set as a *tuple*. Else, pass it to the row factory and return its result. Return `None` if no more data is available.

fetchmany(*size=cursor.arraysize*)

Return the next set of rows of a query result as a *list*. Return an empty list if no more rows are available.

The number of rows to fetch per call is specified by the *size* parameter. If *size* is not given, *arraysize* determines the number of rows to be fetched. If fewer than *size* rows are available, as many rows as are available are returned.

Note there are performance considerations involved with the *size* parameter. For optimal performance, it is usually best to use the arraysize attribute. If the *size* parameter is used, then it is best for it to retain the same value from one *fetchmany()* call to the next.

fetchall()

Return all (remaining) rows of a query result as a *list*. Return an empty list if no rows are available. Note that the *arraysize* attribute can affect the performance of this operation.

close()

Close the cursor now (rather than whenever `__del__` is called).

The cursor will be unusable from this point forward; a *ProgrammingError* exception will be raised if any operation is attempted with the cursor.

setinputsizes(*sizes*, /)

Required by the DB-API. Does nothing in `sqlite3`.

setoutputsize(*size*, *column=None*, */*)

Required by the DB-API. Does nothing in `sqlite3`.

arraysize

Read/write attribute that controls the number of rows returned by *fetchmany()*. The default value is 1 which means a single row would be fetched per call.

connection

Read-only attribute that provides the SQLite database *Connection* belonging to the cursor. A *Cursor* object created by calling *con.cursor()* will have a *connection* attribute that refers to *con*:

```
>>> con = sqlite3.connect(":memory:")
>>> cur = con.cursor()
>>> cur.connection == con
True
>>> con.close()
```

description

Read-only attribute that provides the column names of the last query. To remain compatible with the Python DB API, it returns a 7-tuple for each column where the last six items of each tuple are `None`.

It is set for `SELECT` statements without any matching rows as well.

lastrowid

Read-only attribute that provides the row id of the last inserted row. It is only updated after successful `INSERT` or `REPLACE` statements using the *execute()* method. For other statements, after *executemany()* or *executescript()*, or if the insertion failed, the value of `lastrowid` is left unchanged. The initial value of `lastrowid` is `None`.

> **Nota**
>
> Inserts into `WITHOUT ROWID` tables are not recorded.

Alterado na versão 3.6: Added support for the `REPLACE` statement.

rowcount

Read-only attribute that provides the number of modified rows for `INSERT`, `UPDATE`, `DELETE`, and `REPLACE` statements; is `-1` for other statements, including CTE (Common Table Expression) queries. It is only updated by the *execute()* and *executemany()* methods, after the statement has run to completion. This means that any resulting rows must be fetched in order for `rowcount` to be updated.

row_factory

Control how a row fetched from this `Cursor` is represented. If `None`, a row is represented as a *tuple*. Can be set to the included *sqlite3.Row*; or a *callable* that accepts two arguments, a *Cursor* object and the `tuple` of row values, and returns a custom object representing an SQLite row.

Defaults to what *Connection.row_factory* was set to when the `Cursor` was created. Assigning to this attribute does not affect *Connection.row_factory* of the parent connection.

Consulte *How to create and use row factories* para mais detalhes.

Row objects

class sqlite3.**Row**

A `Row` instance serves as a highly optimized *row_factory* for *Connection* objects. It supports iteration, equality testing, *len()*, and *mapping* access by column name and index.

Two `Row` objects compare equal if they have identical column names and values.

Consulte *How to create and use row factories* para mais detalhes.

keys()

Return a *list* of column names as *strings*. Immediately after a query, it is the first member of each tuple in *Cursor.description*.

Alterado na versão 3.5: Added support of slicing.

Blob objects

class sqlite3.**Blob**

Adicionado na versão 3.11.

A *Blob* instance is a *file-like object* that can read and write data in an SQLite BLOB. Call *len(blob)* to get the size (number of bytes) of the blob. Use indices and *slices* for direct access to the blob data.

Use the *Blob* as a *context manager* to ensure that the blob handle is closed after use.

```
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE test(blob_col blob)")
con.execute("INSERT INTO test(blob_col) VALUES(zeroblob(13))")

# Write to our blob, using two write operations:
with con.blobopen("test", "blob_col", 1) as blob:
    blob.write(b"hello, ")
    blob.write(b"world.")
    # Modify the first and last bytes of our blob
    blob[0] = ord("H")
    blob[-1] = ord("!")

# Read the contents of our blob
with con.blobopen("test", "blob_col", 1) as blob:
    greeting = blob.read()

print(greeting)  # outputs "b'Hello, world!'"
con.close()
```

close()

Close the blob.

The blob will be unusable from this point onward. An *Error* (or subclass) exception will be raised if any further operation is attempted with the blob.

read(*length=-1*, */*)

Read *length* bytes of data from the blob at the current offset position. If the end of the blob is reached, the data up to EOF (End of File) will be returned. When *length* is not specified, or is negative, *read()* will read until the end of the blob.

write(*data*, */*)

Write *data* to the blob at the current offset. This function cannot change the blob length. Writing beyond the end of the blob will raise *ValueError*.

tell()

Return the current access position of the blob.

seek(*offset*, *origin=os.SEEK_SET*, */*)

Set the current access position of the blob to *offset*. The *origin* argument defaults to *os.SEEK_SET* (absolute blob positioning). Other values for *origin* are *os.SEEK_CUR* (seek relative to the current position) and *os.SEEK_END* (seek relative to the blob's end).

PrepareProtocol objects

class sqlite3.**PrepareProtocol**

The PrepareProtocol type's single purpose is to act as a **PEP 246** style adaption protocol for objects that can *adapt themselves* to *native SQLite types*.

Exceções

The exception hierarchy is defined by the DB-API 2.0 (**PEP 249**).

exception sqlite3.**Warning**

This exception is not currently raised by the `sqlite3` module, but may be raised by applications using `sqlite3`, for example if a user-defined function truncates data while inserting. `Warning` is a subclass of *`Exception`*.

exception sqlite3.**Error**

The base class of the other exceptions in this module. Use this to catch all errors with one single `except` statement. `Error` is a subclass of *`Exception`*.

If the exception originated from within the SQLite library, the following two attributes are added to the exception:

sqlite_errorcode

The numeric error code from the SQLite API

Adicionado na versão 3.11.

sqlite_errorname

The symbolic name of the numeric error code from the SQLite API

Adicionado na versão 3.11.

exception sqlite3.**InterfaceError**

Exception raised for misuse of the low-level SQLite C API. In other words, if this exception is raised, it probably indicates a bug in the `sqlite3` module. `InterfaceError` is a subclass of *`Error`*.

exception sqlite3.**DatabaseError**

Exception raised for errors that are related to the database. This serves as the base exception for several types of database errors. It is only raised implicitly through the specialised subclasses. `DatabaseError` is a subclass of *`Error`*.

exception sqlite3.**DataError**

Exception raised for errors caused by problems with the processed data, like numeric values out of range, and strings which are too long. `DataError` is a subclass of *`DatabaseError`*.

exception sqlite3.**OperationalError**

Exception raised for errors that are related to the database's operation, and not necessarily under the control of the programmer. For example, the database path is not found, or a transaction could not be processed. `OperationalError` is a subclass of *`DatabaseError`*.

exception sqlite3.**IntegrityError**

Exception raised when the relational integrity of the database is affected, e.g. a foreign key check fails. It is a subclass of *`DatabaseError`*.

exception sqlite3.**InternalError**

Exception raised when SQLite encounters an internal error. If this is raised, it may indicate that there is a problem with the runtime SQLite library. `InternalError` is a subclass of *`DatabaseError`*.

exception sqlite3.**ProgrammingError**

Exception raised for `sqlite3` API programming errors, for example supplying the wrong number of bindings to a query, or trying to operate on a closed *`Connection`*. `ProgrammingError` is a subclass of *`DatabaseError`*.

exception sqlite3.**NotSupportedError**

Exception raised in case a method or database API is not supported by the underlying SQLite library. For example, setting *deterministic* to `True` in *create_function()*, if the underlying SQLite library does not support deterministic functions. `NotSupportedError` is a subclass of *DatabaseError*.

SQLite and Python types

SQLite natively supports the following types: `NULL`, `INTEGER`, `REAL`, `TEXT`, `BLOB`.

The following Python types can thus be sent to SQLite without any problem:

Tipo em Python	Tipo em SQLite
`None`	`NULL`
int	`INTEGER`
float	`REAL`
str	`TEXT`
bytes	`BLOB`

This is how SQLite types are converted to Python types by default:

Tipo em SQLite	Tipo em Python
`NULL`	`None`
`INTEGER`	*int*
`REAL`	*float*
`TEXT`	depends on *text_factory*, *str* by default
`BLOB`	*bytes*

The type system of the `sqlite3` module is extensible in two ways: you can store additional Python types in an SQLite database via *object adapters*, and you can let the `sqlite3` module convert SQLite types to Python types via *converters*.

Default adapters and converters (deprecated)

Nota

The default adapters and converters are deprecated as of Python 3.12. Instead, use the *Adapter and converter recipes* and tailor them to your needs.

The deprecated default adapters and converters consist of:

- An adapter for *datetime.date* objects to *strings* in ISO 8601 format.
- An adapter for *datetime.datetime* objects to strings in ISO 8601 format.
- A converter for *declared* "date" types to *datetime.date* objects.
- A converter for declared "timestamp" types to *datetime.datetime* objects. Fractional parts will be truncated to 6 digits (microsecond precision).

Nota

The default "timestamp" converter ignores UTC offsets in the database and always returns a naive *datetime.datetime* object. To preserve UTC offsets in timestamps, either leave converters disabled, or register an offset-aware converter with *register_converter()*.

Descontinuado desde a versão 3.12.

Interface de linha de comando

The `sqlite3` module can be invoked as a script, using the interpreter's `-m` switch, in order to provide a simple SQLite shell. The argument signature is as follows:

```
python -m sqlite3 [-h] [-v] [filename] [sql]
```

Type `.quit` or CTRL-D to exit the shell.

`-h, --help`
Print CLI help.

`-v, --version`
Print underlying SQLite library version.

Adicionado na versão 3.12.

12.6.3 Guias de como fazer

How to use placeholders to bind values in SQL queries

SQL operations usually need to use values from Python variables. However, beware of using Python's string operations to assemble queries, as they are vulnerable to SQL injection attacks. For example, an attacker can simply close the single quote and inject `OR TRUE` to select all rows:

```
>>> # Never do this -- insecure!
>>> symbol = input()
' OR TRUE; --
>>> sql = "SELECT * FROM stocks WHERE symbol = '%s'" % symbol
>>> print(sql)
SELECT * FROM stocks WHERE symbol = '' OR TRUE; --'
>>> cur.execute(sql)
```

Instead, use the DB-API's parameter substitution. To insert a variable into a query string, use a placeholder in the string, and substitute the actual values into the query by providing them as a *`tuple`* of values to the second argument of the cursor's *`execute()`* method.

An SQL statement may use one of two kinds of placeholders: question marks (qmark style) or named placeholders (named style). For the qmark style, *parameters* must be a *sequence* whose length must match the number of placeholders, or a *`ProgrammingError`* is raised. For the named style, *parameters* must be an instance of a *`dict`* (or a subclass), which must contain keys for all named parameters; any extra items are ignored. Here's an example of both styles:

```
con = sqlite3.connect(":memory:")
cur = con.execute("CREATE TABLE lang(name, first_appeared)")

# This is the named style used with executemany():
data = (
    {"name": "C", "year": 1972},
    {"name": "Fortran", "year": 1957},
    {"name": "Python", "year": 1991},
    {"name": "Go", "year": 2009},
)
cur.executemany("INSERT INTO lang VALUES(:name, :year)", data)

# This is the qmark style used in a SELECT query:
params = (1972,)
cur.execute("SELECT * FROM lang WHERE first_appeared = ?", params)
print(cur.fetchall())
con.close()
```

Nota

PEP 249 numeric placeholders are *not* supported. If used, they will be interpreted as named placeholders.

How to adapt custom Python types to SQLite values

SQLite supports only a limited set of data types natively. To store custom Python types in SQLite databases, *adapt* them to one of the *Python types SQLite natively understands*.

There are two ways to adapt Python objects to SQLite types: letting your object adapt itself, or using an *adapter callable*. The latter will take precedence above the former. For a library that exports a custom type, it may make sense to enable that type to adapt itself. As an application developer, it may make more sense to take direct control by registering custom adapter functions.

How to write adaptable objects

Suppose we have a `Point` class that represents a pair of coordinates, `x` and `y`, in a Cartesian coordinate system. The coordinate pair will be stored as a text string in the database, using a semicolon to separate the coordinates. This can be implemented by adding a `__conform__(self, protocol)` method which returns the adapted value. The object passed to *protocol* will be of type *`PrepareProtocol`*.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __conform__(self, protocol):
        if protocol is sqlite3.PrepareProtocol:
            return f"{self.x};{self.y}"

con = sqlite3.connect(":memory:")
cur = con.cursor()

cur.execute("SELECT ?", (Point(4.0, -3.2),))
print(cur.fetchone()[0])
con.close()
```

How to register adapter callables

The other possibility is to create a function that converts the Python object to an SQLite-compatible type. This function can then be registered using *`register_adapter()`*.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

def adapt_point(point):
    return f"{point.x};{point.y}"

sqlite3.register_adapter(Point, adapt_point)

con = sqlite3.connect(":memory:")
cur = con.cursor()

cur.execute("SELECT ?", (Point(1.0, 2.5),))
print(cur.fetchone()[0])
con.close()
```

How to convert SQLite values to custom Python types

Writing an adapter lets you convert *from* custom Python types *to* SQLite values. To be able to convert *from* SQLite values *to* custom Python types, we use *converters*.

Let's go back to the `Point` class. We stored the x and y coordinates separated via semicolons as strings in SQLite.

First, we'll define a converter function that accepts the string as a parameter and constructs a `Point` object from it.

> **Nota**
>
> Converter functions are **always** passed a *bytes* object, no matter the underlying SQLite data type.

```
def convert_point(s):
    x, y = map(float, s.split(b";"))
    return Point(x, y)
```

We now need to tell `sqlite3` when it should convert a given SQLite value. This is done when connecting to a database, using the *detect_types* parameter of *connect()*. There are three options:

- Implicit: set *detect_types* to *PARSE_DECLTYPES*
- Explicit: set *detect_types* to *PARSE_COLNAMES*
- Both: set *detect_types* to `sqlite3.PARSE_DECLTYPES | sqlite3.PARSE_COLNAMES`. Column names take precedence over declared types.

The following example illustrates the implicit and explicit approaches:

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

def adapt_point(point):
    return f"{point.x};{point.y}"

def convert_point(s):
    x, y = list(map(float, s.split(b";")))
    return Point(x, y)

# Register the adapter and converter
sqlite3.register_adapter(Point, adapt_point)
sqlite3.register_converter("point", convert_point)

# 1) Parse using declared types
p = Point(4.0, -3.2)
con = sqlite3.connect(":memory:", detect_types=sqlite3.PARSE_DECLTYPES)
cur = con.execute("CREATE TABLE test(p point)")

cur.execute("INSERT INTO test(p) VALUES(?)", (p,))
cur.execute("SELECT p FROM test")
print("with declared types:", cur.fetchone()[0])
cur.close()
con.close()

# 2) Parse using column names
con = sqlite3.connect(":memory:", detect_types=sqlite3.PARSE_COLNAMES)
```

(continua na próxima página)

(continuação da página anterior)

```
cur = con.execute("CREATE TABLE test(p)")

cur.execute("INSERT INTO test(p) VALUES(?)", (p,))
cur.execute('SELECT p AS "p [point]" FROM test')
print("with column names:", cur.fetchone()[0])
cur.close()
con.close()
```

Adapter and converter recipes

This section shows recipes for common adapters and converters.

```
import datetime
import sqlite3

def adapt_date_iso(val):
    """Adapt datetime.date to ISO 8601 date."""
    return val.isoformat()

def adapt_datetime_iso(val):
    """Adapt datetime.datetime to timezone-naive ISO 8601 date."""
    return val.replace(tzinfo=None).isoformat()

def adapt_datetime_epoch(val):
    """Adapt datetime.datetime to Unix timestamp."""
    return int(val.timestamp())

sqlite3.register_adapter(datetime.date, adapt_date_iso)
sqlite3.register_adapter(datetime.datetime, adapt_datetime_iso)
sqlite3.register_adapter(datetime.datetime, adapt_datetime_epoch)

def convert_date(val):
    """Convert ISO 8601 date to datetime.date object."""
    return datetime.date.fromisoformat(val.decode())

def convert_datetime(val):
    """Convert ISO 8601 datetime to datetime.datetime object."""
    return datetime.datetime.fromisoformat(val.decode())

def convert_timestamp(val):
    """Convert Unix epoch timestamp to datetime.datetime object."""
    return datetime.datetime.fromtimestamp(int(val))

sqlite3.register_converter("date", convert_date)
sqlite3.register_converter("datetime", convert_datetime)
sqlite3.register_converter("timestamp", convert_timestamp)
```

How to use connection shortcut methods

Using the *execute()*, *executemany()*, and *executescript()* methods of the *Connection* class, your code can be written more concisely because you don't have to create the (often superfluous) *Cursor* objects explicitly. Instead, the *Cursor* objects are created implicitly and these shortcut methods return the cursor objects. This way, you can execute a SELECT statement and iterate over it directly using only a single call on the *Connection* object.

```
# Create and fill the table.
con = sqlite3.connect(":memory:")
```

(continua na próxima página)

(continuação da página anterior)

```
con.execute("CREATE TABLE lang(name, first_appeared)")
data = [
    ("C++", 1985),
    ("Objective-C", 1984),
]
con.executemany("INSERT INTO lang(name, first_appeared) VALUES(?, ?)", data)

# Print the table contents
for row in con.execute("SELECT name, first_appeared FROM lang"):
    print(row)

print("I just deleted", con.execute("DELETE FROM lang").rowcount, "rows")

# close() is not a shortcut method and it's not called automatically;
# the connection object should be closed manually
con.close()
```

How to use the connection context manager

A *Connection* object can be used as a context manager that automatically commits or rolls back open transactions when leaving the body of the context manager. If the body of the `with` statement finishes without exceptions, the transaction is committed. If this commit fails, or if the body of the `with` statement raises an uncaught exception, the transaction is rolled back. If *autocommit* is `False`, a new transaction is implicitly opened after committing or rolling back.

If there is no open transaction upon leaving the body of the `with` statement, or if *autocommit* is `True`, the context manager does nothing.

Nota

The context manager neither implicitly opens a new transaction nor closes the connection. If you need a closing context manager, consider using *contextlib.closing()*.

```
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE lang(id INTEGER PRIMARY KEY, name VARCHAR UNIQUE)")

# Successful, con.commit() is called automatically afterwards
with con:
    con.execute("INSERT INTO lang(name) VALUES(?)", ("Python",))

# con.rollback() is called after the with block finishes with an exception,
# the exception is still raised and must be caught
try:
    with con:
        con.execute("INSERT INTO lang(name) VALUES(?)", ("Python",))
except sqlite3.IntegrityError:
    print("couldn't add Python twice")

# Connection object used as context manager only commits or rollbacks transactions,
# so the connection object should be closed manually
con.close()
```

How to work with SQLite URIs

Some useful URI tricks include:

- Open a database in read-only mode:

```
>>> con = sqlite3.connect("file:tutorial.db?mode=ro", uri=True)
>>> con.execute("CREATE TABLE readonly(data)")
Traceback (most recent call last):
OperationalError: attempt to write a readonly database
>>> con.close()
```

- Do not implicitly create a new database file if it does not already exist; will raise *OperationalError* if unable to create a new file:

```
>>> con = sqlite3.connect("file:nosuchdb.db?mode=rw", uri=True)
Traceback (most recent call last):
OperationalError: unable to open database file
```

- Create a shared named in-memory database:

```
db = "file:mem1?mode=memory&cache=shared"
con1 = sqlite3.connect(db, uri=True)
con2 = sqlite3.connect(db, uri=True)
with con1:
    con1.execute("CREATE TABLE shared(data)")
    con1.execute("INSERT INTO shared VALUES(28)")
res = con2.execute("SELECT data FROM shared")
assert res.fetchone() == (28,)

con1.close()
con2.close()
```

More information about this feature, including a list of parameters, can be found in the SQLite URI documentation.

How to create and use row factories

By default, `sqlite3` represents each row as a *tuple*. If a `tuple` does not suit your needs, you can use the *sqlite3.Row* class or a custom *row_factory*.

While `row_factory` exists as an attribute both on the *Cursor* and the *Connection*, it is recommended to set *Connection.row_factory*, so all cursors created from the connection will use the same row factory.

`Row` provides indexed and case-insensitive named access to columns, with minimal memory overhead and performance impact over a `tuple`. To use `Row` as a row factory, assign it to the `row_factory` attribute:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = sqlite3.Row
```

Queries now return `Row` objects:

```
>>> res = con.execute("SELECT 'Earth' AS name, 6378 AS radius")
>>> row = res.fetchone()
>>> row.keys()
['name', 'radius']
>>> row[0]          # Access by index.
'Earth'
>>> row["name"]     # Access by name.
'Earth'
>>> row["RADIUS"]   # Column names are case-insensitive.
```

(continua na próxima página)

(continuação da página anterior)

```
6378
>>> con.close()
```

Nota

The `FROM` clause can be omitted in the `SELECT` statement, as in the above example. In such cases, SQLite returns a single row with columns defined by expressions, e.g. literals, with the given aliases `expr AS alias`.

You can create a custom *`row_factory`* that returns each row as a *`dict`*, with column names mapped to values:

```
def dict_factory(cursor, row):
    fields = [column[0] for column in cursor.description]
    return {key: value for key, value in zip(fields, row)}
```

Using it, queries now return a `dict` instead of a `tuple`:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = dict_factory
>>> for row in con.execute("SELECT 1 AS a, 2 AS b"):
...     print(row)
{'a': 1, 'b': 2}
>>> con.close()
```

The following row factory returns a *named tuple*:

```
from collections import namedtuple

def namedtuple_factory(cursor, row):
    fields = [column[0] for column in cursor.description]
    cls = namedtuple("Row", fields)
    return cls._make(row)
```

`namedtuple_factory()` can be used as follows:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = namedtuple_factory
>>> cur = con.execute("SELECT 1 AS a, 2 AS b")
>>> row = cur.fetchone()
>>> row
Row(a=1, b=2)
>>> row[0]  # Indexed access.
1
>>> row.b   # Attribute access.
2
>>> con.close()
```

With some adjustments, the above recipe can be adapted to use a *`dataclass`*, or any other custom class, instead of a *`namedtuple`*.

How to handle non-UTF-8 text encodings

By default, `sqlite3` uses *`str`* to adapt SQLite values with the `TEXT` data type. This works well for UTF-8 encoded text, but it might fail for other encodings and invalid UTF-8. You can use a custom *`text_factory`* to handle such cases.

Because of SQLite's flexible typing, it is not uncommon to encounter table columns with the `TEXT` data type containing non-UTF-8 encodings, or even arbitrary data. To demonstrate, let's assume we have a database with

ISO-8859-2 (Latin-2) encoded text, for example a table of Czech-English dictionary entries. Assuming we now have a *Connection* instance `con` connected to this database, we can decode the Latin-2 encoded text using this *text_factory*:

```
con.text_factory = lambda data: str(data, encoding="latin2")
```

For invalid UTF-8 or arbitrary data in stored in `TEXT` table columns, you can use the following technique, borrowed from the unicode-howto:

```
con.text_factory = lambda data: str(data, errors="surrogateescape")
```

Nota

The `sqlite3` module API does not support strings containing surrogates.

Ver também

unicode-howto

12.6.4 Explicação

Controle de transações

`sqlite3` oferece vários métodos para controlar se, quando e como as transações do banco de dados são abertas e fechadas. *Controle de transações através do atributo autocommit* é recomendado, enquanto *Controle de transação através do atributo isolation_level* mantém o comportamento pré-Python 3.12.

Controle de transações através do atributo `autocommit`

A forma recomendada de controlar o comportamento da transação é através do atributo *Connection.autocommit*, que deve ser preferencialmente definido usando o parâmetro *autocommit* de *connect()*.

É sugerido definir *autocommit* como `False`, o que implica controle de transação compatível com a **PEP 249**. Isso significa:

- `sqlite3` garante que uma transação esteja sempre aberta, então *connect()*, *Connection.commit()* e *Connection.rollback()* abrirão implicitamente uma nova transação (imediatamente após fechando a pendência, para as duas últimas). `sqlite3` usa instruções `BEGIN DEFERRED` ao abrir transações.
- Transações devem ser executadas explicitamente usando `commit()`.
- Transações devem ser revertidas explicitamente usando `rollback()`.
- Uma reversão implícita é executada se o banco de dados estiver em estado *close()* com alterações pendentes.

Defina *autocommit* como `True` para ativar o modo autocommit do SQLite. Neste modo, *Connection.commit()* e *Connection.rollback()* não têm efeito. Observe que o modo autocommit do SQLite é distinto do atributo *Connection.autocommit* compatível com **PEP 249**; use *Connection.in_transaction* para consultar o modo de confirmação automática do SQLite de baixo nível.

Defina *autocommit* como *LEGACY_TRANSACTION_CONTROL* para deixar o comportamento de controle de transação para o atributo *Connection.isolation_level*. Veja *Controle de transação através do atributo isolation_level* para mais informações.

Controle de transação através do atributo `isolation_level`

Nota

A forma recomendada de controlar transações é através do atributo *autocommit*. Veja *Controle de transações através do atributo autocommit*.

Se *Connection.autocommit* estiver definido como *LEGACY_TRANSACTION_CONTROL* (o padrão), o comportamento da transação é controlado usando o atributo *Connection.isolation_level*. Caso contrário, isolation_level não tem efeito.

Se o atributo de conexão *isolation_level* não for None, novas transações são abertas implicitamente antes de *execute()* e *executemany()* executa instruções INSERT, UPDATE, DELETE ou REPLACE; para outras instruções, nenhuma manipulação de transação implícita é executada. Use os métodos *commit()* e *rollback()* para fazer commit e reverter respectivamente transações pendentes. Você pode escolher o comportamento subjacente de transação do SQLite – isto é, se e que tipo de instruções BEGIN do sqlite3 são executadas implicitamente – através do atributo *isolation_level*.

Se *isolation_level* estiver definido como None, nenhuma transação será aberta implicitamente. Isso deixa a biblioteca SQLite subjacente no modo autocommit, mas também permite que o usuário execute sua própria manipulação de transações usando instruções SQL explícitas. O modo de autocommit da biblioteca SQLite subjacente pode ser consultado usando o atributo *in_transaction*.

O método *executescript()* compromete implicitamente qualquer transação pendente antes da execução do script SQL fornecido, independentemente do valor de *isolation_level*.

Alterado na versão 3.6: sqlite3 costumava fazer commit de forma implícita de uma transação aberta antes das instruções DDL. Este não é mais o caso.

Alterado na versão 3.12: A forma recomendada de controlar transações agora é através do atributo *autocommit*.

CAPÍTULO 13

Compressão de Dados e Arquivamento

Os módulos descritos neste capítulo oferecem suporte à compressão de dados com os algoritmos zlib, gzip, bzip2, lzma e zstd, e a criação de arquivos no formato ZIP e tar. Veja também *Operações de arquivamento* fornecido pelo módulo *shutil*.

13.1 O pacote compression

Adicionado na versão 3.14.

O pacote compression contém os módulos de compressão canônicos, que contêm interfaces para diversos algoritmos de compressão. Alguns desses módulos historicamente estiveram disponíveis como módulos separados; estes continuarão disponíveis com seus nomes originais por motivos de compatibilidade e não serão removidos sem um ciclo de descontinuação. O uso de módulos em compression é incentivado sempre que possível.

- compression.bz2 – Re-exporta *bz2*
- compression.gzip – Re-exporta *gzip*
- compression.lzma – Re-exporta *lzma*
- compression.zlib – Re-exporta *zlib*
- *compression.zstd* – Invólucro para a biblioteca de compressão Zstandard

13.2 compression.zstd — Compactação compatível com o formato Zstandard

Adicionado na versão 3.14.

Código-fonte: Lib/compression/zstd/__init__.py

Este módulo fornece classes e funções para compactar e descompactar dados usando o algoritmo de compactação Zstandard (ou *zstd*). O manual do zstd descreve o Zstandard como "um algoritmo de compactação rápido e sem perdas, voltado para cenários de compactação em tempo real no nível zlib e melhores taxas de compactação". Também está incluída uma interface de arquivo que oferece suporte à leitura e à gravação do conteúdo de arquivos .zst criados pelo utilitário **zstd**, bem como fluxos compactados zstd brutos.

O módulo compression.zstd contém:

- A função *open()* e a classe *ZstdFile* para leitura e escrita de arquivos compactados.
- As classes *ZstdCompressor* e *ZstdDecompressor* para (des)compactação incremental.
- As funções *compress()* e *decompress()* para (des)compressão de uma só vez.
- As funções *train_dict()* e *finalize_dict()* e a classe *ZstdDict* para treinar e gerenciar dicionários Zstandard.
- As classes *CompressionParameter*, *DecompressionParameter* e *Strategy* para definir parâmetros avançados de (des)compactação.

13.2.1 Exceções

exception compression.zstd.**ZstdError**

Essa exceção é levantada quando ocorre um erro durante a compactação ou descompactação ou durante a inicialização do estado (des)compactador.

13.2.2 Lendo e escrevendo arquivos comprimidos

compression.zstd.**open**(*file*, */*, *mode='rb'*, ***, *level=None*, *options=None*, *zstd_dict=None*, *encoding=None*, *errors=None*, *newline=None*)

Abre um arquivo compactado com Zstandard em modo binário ou texto, retornando *objeto arquivo*.

O argumento *file* pode ser um nome de arquivo (dado como um objeto *str*, *bytes* ou *caminho ou similar*), neste caso o arquivo nomeado é aberto , ou pode ser um objeto arquivo existente para leitura ou escrita.

O argumento mode pode ser `'rb'` para leitura (padrão), `'wb'` para sobrescrever, `'ab'` para anexar ou `'xb'` para criação exclusiva. Estes podem ser fornecidos como `'r'`, `'w'`, `'a'` e `'x'`, respectivamente. Você também pode abrir em modo texto com `'rt'`, `'wt'`, `'at'` e `'xt'`, respectivamente.

Durante a leitura, o argumento *options* pode ser um dicionário que fornece parâmetros avançados de descompactação; consulte *DecompressionParameter* para obter informações detalhadas sobre os parâmetros suportados. O argumento *zstd_dict* é uma instância de *ZstdDict* a ser usada durante a descompactação. Durante a leitura, se o argumento *level* não for None, uma exceção `TypeError` será levantada.

Ao escrever, o argumento *options* pode ser um dicionário que fornece parâmetros avançados de descompactação; consulte *CompressionParameter* para obter informações detalhadas sobre os parâmetros suportados. O argumento *level* é o nível de compactação a ser usado ao escrever dados compactados. Apenas *level* ou *options* pode ser diferente de None. O argumento *zstd_dict* é uma instância de *ZstdDict* a ser usada durante a compactação.

Em modo binário, esta função equivale ao construtor *ZstdFile*: `ZstdFile(file, mode, ...)`. Nesse caso, os parâmetros *encoding*, *errors* e *newline* não devem ser fornecidos.

Em modo texto, um objeto *ZstdFile* é criado e encapsulado em uma instância *io.TextIOWrapper* com a codificação especificada, comportamento de tratamento de erros e fins de linha.

class compression.zstd.**ZstdFile**(*file*, */*, *mode='rb'*, ***, *level=None*, *options=None*, *zstd_dict=None*)

Abre um arquivo compactado com Zstandard no modo binário.

Um *ZstdFile* pode envolver um *objeto arquivo* já aberto, ou operar diretamente em um arquivo nomeado. O argumento *file* especifica o objeto arquivo a ser envolvido ou o nome do arquivo a ser aberto (como um objeto *str*, *bytes* ou *objeto caminho ou similar*). Se estiver envolvendo um objeto arquivo existente, o arquivo envolvido não será fechado quando o *ZstdFile* for fechado.

O argumento *mode* pode ser `'rb'` para leitura (padrão), `'wb'` para substituição, `'xb'` para criação exclusiva ou `'ab'` para anexar. Estes podem ser equivalentemente dados como `'r'`, `'w'`, `'x'` e `'a'` respectivamente.

Se *file* for um objeto arquivo (ao invés de um nome de arquivo real), um modo de `'w'` não truncará o arquivo e será equivalente a `'a'`.

Durante a leitura, o argumento *options* pode ser um dicionário que fornece parâmetros avançados de descompactação; consulte *DecompressionParameter* para obter informações detalhadas sobre os parâmetros

suportados. O argumento *zstd_dict* é uma instância de `ZstdDict` a ser usada durante a descompactação. Durante a leitura, se o argumento *level* não for None, uma exceção `TypeError` será levantada.

Ao escrever, o argumento *options* pode ser um dicionário que fornece parâmetros avançados de descompactação; consulte `CompressionParameter` para obter informações detalhadas sobre os parâmetros suportados. O argumento *level* é o nível de compactação a ser usado ao escrever dados compactados. Apenas *level* ou *options* pode ser passado. O argumento *zstd_dict* é uma instância de `ZstdDict` a ser usada durante a compactação.

`ZstdFile` oferece suporte a todos os membros especificados por `io.BufferedIOBase`, exceto `detach()` e `truncate()`. Iteração e a instrução `with` são suportadas.

O método e os atributos a seguir também são fornecidos:

peek(*size=-1*)

Retorna dados armazenados em buffer sem avançar a posição do arquivo. Pelo menos um byte de dados será retornado, a menos que o EOF tenha sido atingido. O número exato de bytes retornados não é especificado (o argumento *size* é ignorado).

> **Nota**
>
> Enquanto chamar `peek()` não altera a posição do arquivo de `ZstdFile`, pode alterar a posição do objeto arquivo subjacente (por exemplo, se o `ZstdFile` foi construído passando um objeto arquivo para *file*).

mode

`'rb'` para leitura e `'wb'` para escrita.

name

O nome do arquivo Zstandard. Equivalente ao atributo `name` do *objeto arquivo* subjacente.

13.2.3 Comprimindo e descomprimindo dados na memória

`compression.zstd.`**compress**(*data*, *level=None*, *options=None*, *zstd_dict=None*)

Compacta *data* (um *objeto bytes ou similar*), retornando os dados compactados como um objeto `bytes`.

O argumento *level* é um inteiro que controla o nível de compactação. *level* é uma alternativa à configuração `CompressionParameter.compression_level` em *options*. Use `bounds()` em `compression_level` para obter os valores que podem ser passados para *level*. Se opções avançadas de compactação forem necessárias, o argumento *level* deve ser omitido e, no dicionário *options*, o parâmetro `CompressionParameter.compression_level` deve ser definido.

O argumento *options* é um dicionário Python que contém parâmetros de compactação avançados. As chaves e valores válidos para parâmetros de compactação estão documentados na documentação `CompressionParameter`.

O argumento *zstd_dict* é uma instância de `ZstdDict` contendo dados treinados para melhorar a eficiência da compactação. A função `train_dict()` pode ser usada para gerar um dicionário Zstandard.

`compression.zstd.`**decompress**(*data*, *zstd_dict=None*, *options=None*)

Descompacta *data* (um *objeto bytes ou similar*), retornando os dados descompactados como um objeto `bytes`.

O argumento *options* é um dicionário Python que contém parâmetros de descompactação avançados. As chaves e valores válidos para parâmetros de compactação estão documentados na documentação `DecompressionParameter`.

O argumento *zstd_dict* é uma instância de `ZstdDict` contendo dados treinados usados durante a compactação. Este deve ser o mesmo dicionário Zstandard usado durante a compactação.

Se *data* forem a concatenação de vários quadros comprimidos distintos, descomprime todos esses quadros e retorna a concatenação dos resultados.

class compression.zstd.**ZstdCompressor**(*level=None*, *options=None*, *zstd_dict=None*)

Cria um objeto compressão, que pode ser usado para comprimir dados de forma incremental.

Para uma maneira mais conveniente de compactar um único bloco de dados, consulte a função a nível de módulo *compress()*.

O argumento *level* é um inteiro que controla o nível de compactação. *level* é uma alternativa à configuração *CompressionParameter.compression_level* em *options*. Use *bounds()* em *compression_level* para obter os valores que podem ser passados para *level*. Se opções avançadas de compactação forem necessárias, o argumento *level* deve ser omitido e, no dicionário *options*, o parâmetro `CompressionParameter.compression_level` deve ser definido.

O argumento *options* é um dicionário Python que contém parâmetros de compactação avançados. As chaves e valores válidos para parâmetros de compactação estão documentados na documentação *CompressionParameter*.

O argumento *zstd_dict* é uma instância opcional de *ZstdDict* contendo dados treinados para melhorar a eficiência da compactação. A função *train_dict()* pode ser usada para gerar um dicionário Zstandard.

compress(*data*, *mode=ZstdCompressor.CONTINUE*)

Compacte *data* (um *objeto bytes ou similar*), retornando um objeto *bytes* com dados compactados, se possível, ou, caso contrário, um objeto `bytes` vazio. Alguns dos *data* podem ser armazenados internamente em buffer, para uso em chamadas posteriores a `compress()` e *flush()*. Os dados retornados devem ser concatenados com a saída de quaisquer chamadas anteriores a *compress()*.

O argumento *mode* é um atributo *ZstdCompressor*, podendo ser definido com *CONTINUE*, *FLUSH_BLOCK* ou *FLUSH_FRAME*.

Quando todos os dados forem fornecidos ao compactador, chama o método *flush()* para finalizar o processo de compactação. Se *compress()* for chamado com *mode* definido como *FLUSH_FRAME*, *flush()* não deve ser chamado, pois gravaria um novo quadro vazio.

flush(*mode=ZstdCompressor.FLUSH_FRAME*)

Conclui o processo de compressão, retornando um objeto *bytes* contendo todos os dados armazenados nos buffers internos do compressor.

O argumento *mode* é um atributo *ZstdCompressor*, podendo ser definido com *FLUSH_BLOCK* ou *FLUSH_FRAME*.

set_pledged_input_size(*size*)

Especifica a quantidade de *size* dados descompactados que será fornecida para o próximo quadro. *size* será gravado no cabeçalho do quadro do próximo quadro, a menos que *CompressionParameter.content_size_flag* seja `False` ou `0`. Um tamanho `0` significa que o quadro está vazio. Se *size* for `None`, o cabeçalho do quadro omitirá o tamanho do quadro. Quadros que incluem o tamanho dos dados descompactados requerem menos memória para descompactação, especialmente em níveis de compactação mais altos.

Se *last_mode* não for *FLUSH_FRAME*, uma *ValueError* será levantada, pois o compressor não está no início de um quadro. Se o tamanho prometido não corresponder ao tamanho real dos dados fornecidos para *compress()*, chamadas futuras para `compress()` ou *flush()* poderão levantar *ZstdError* e o último bloco de dados poderá ser perdido.

Depois que *flush()* ou *compress()* são chamados com o modo *FLUSH_FRAME*, o próximo quadro não incluirá o tamanho do quadro no cabeçalho, a menos que `set_pledged_input_size()` seja chamado novamente.

CONTINUE

Coleta mais dados para compactação, o que pode ou não gerar saída imediata. Este modo otimiza a taxa de compactação maximizando a quantidade de dados por bloco e quadro.

FLUSH_BLOCK

Conclui e grava um bloco no fluxo de dados. Os dados retornados até o momento podem ser descompactados imediatamente. Dados anteriores ainda podem ser referenciados em blocos futuros gerados por chamadas a *compress()*, melhorando a compactação.

FLUSH_FRAME

Conclui e escreve um quadro. Dados futuros fornecidos a *compress()* serão gravados em um novo quadro e *não poderão* fazer referência a dados passados.

last_mode

O último modo passado para *compress()* ou *flush()*. O valor pode ser *CONTINUE*, *FLUSH_BLOCK* ou *FLUSH_FRAME*. O valor inicial é *FLUSH_FRAME*, significando que o compactador está no início de um novo quadro.

class compression.zstd.**ZstdDecompressor**(*zstd_dict=None*, *options=None*)

Cria um objeto descompressor, que pode ser usado para descomprimir dados de forma incremental.

Para uma maneira mais conveniente de descompactar todo um fluxo compactado de uma só vez, consulte a função a nível de módulo *decompress()*.

O argumento *options* é um dicionário Python que contém parâmetros de descompactação avançados. As chaves e valores válidos para parâmetros de compactação estão documentados na documentação *DecompressionParameter*.

O argumento *zstd_dict* é uma instância de *ZstdDict* contendo dados treinados usados durante a compactação. Este deve ser o mesmo dicionário Zstandard usado durante a compactação.

> **Nota**
>
> Esta classe não lidam de forma transparente entradas contendo múltiplos quadros compactados, ao contrário da função *decompress()* e da classe *ZstdFile*. Para descompactar uma entrada com múltiplos quadros, você deve usar *decompress()*, *ZstdFile* se estiver trabalhando com um *objeto arquivo* ou múltiplas instâncias de `ZstdDecompressor`.

decompress(*data*, *max_length=-1*)

Descompacta dados *data* (um *objeto bytes ou similar*), retornando dados não compactados como bytes. Alguns dos *data* podem ser armazenados em buffer internamente, para uso em chamadas posteriores para `decompress()`. Os dados retornados devem ser concatenados com a saída de qualquer chamada anterior para `decompress()`.

Se *max_length* for não negativo, o método retornará no máximo *max_length* bytes de dados descomprimidos. Se este limite for atingido e mais saída puder ser produzida, o atributo *needs_input* será definido como `False`. Neste caso, a próxima chamada para *decompress()* pode fornecer *data* como `b''` para obter mais saída.

Se todos os dados de entrada foram descomprimidos e retornados (seja porque era menor que *max_length* bytes, ou porque *max_length* era negativo), o atributo *needs_input* será definido como `True` .

Tentar descompactar dados após o final de um quadro levantará um erro *ZstdError*. Quaisquer dados encontrados após o final do quadro serão ignorados e salvos no atributo *unused_data*.

eof

`True` se o marcador de fim de fluxo foi atingido.

unused_data

Dados encontrados após o término do fluxo comprimido.

Antes do final do fluxo ser alcançado, isso será `b''`.

needs_input

`False` se o método *decompress()* puder fornecer mais dados descompactados antes de exigir uma nova entrada compactada.

13.2.4 Dicionários de Zstandard

compression.zstd.**train_dict**(*samples*, *dict_size*)

Treina um dicionário Zstandard, retornando uma instância *ZstdDict*. Os dicionários Zstandard permitem uma compactação mais eficiente de dados menores, que tradicionalmente são difíceis de comprimir devido à menor repetição. Se você estiver compactando vários grupos de dados semelhantes (como arquivos semelhantes), os dicionários Zstandard podem melhorar significativamente as taxas de compactação e a velocidade.

O argumento *samples* (um iterável de objetos *bytes*) é a população de amostras usada para treinar o dicionário Zstandard.

O argumento *dict_size*, um inteiro, é o tamanho máximo (em bytes) que o dicionário Zstandard deve ter. A documentação do Zstandard sugere um máximo absoluto de no máximo 100 KB, mas o máximo pode ser menor dependendo dos dados. Dicionários maiores geralmente reduzem a compactação, mas melhoram as taxas de compactação. Dicionários menores levam a uma compactação mais rápida, mas reduzem a taxa de compactação.

compression.zstd.**finalize_dict**(*zstd_dict*, /, *samples*, *dict_size*, *level*)

Uma função avançada para converter um dicionário Zstandard de "conteúdo bruto" em um dicionário Zstandard comum. Dicionários de "conteúdo bruto" são uma sequência de bytes que não precisa seguir a estrutura de um dicionário Zstandard comum.

O argumento *zstd_dict* é uma instância *ZstdDict* com *dict_content* contendo o conteúdo bruto do dicionário.

O argumento *samples* (um iterável de objetos *bytes*) contém dados de amostra para gerar o dicionário Zstandard.

O argumento *dict_size*, um inteiro, é o tamanho máximo (em bytes) que o dicionário Zstandard deve ter. Consulte *train_dict()* para sugestões sobre o tamanho máximo do dicionário.

O argumento *level* (um inteiro) é o nível de compactação que se espera que seja passado aos compactadores usando este dicionário. As informações do dicionário variam para cada nível de compactação, portanto, ajustar o nível de compactação adequado pode tornar a compactação mais eficiente.

class compression.zstd.**ZstdDict**(*dict_content*, /, *, *is_raw=False*)

Um invólucro para dicionários Zstandard. Dicionários podem ser usados para melhorar a compactação de muitos pequenos blocos de dados. Use *train_dict()* se precisar treinar um novo dicionário a partir de dados de amostra.

O argumento *dict_content* (um *objeto bytes ou similar*) é a informação do dicionário já treinada.

O argumento *is_raw*, um booleano, é um parâmetro avançado que controla o significado de *dict_content*. `True` significa que *dict_content* é um dicionário de "conteúdo bruto", sem nenhuma restrição de formato. `False` significa que *dict_content* é um dicionário Zstandard comum, criado a partir de funções Zstandard, por exemplo, *train_dict()* ou a CLI externa **zstd**.

Ao passar um `ZstdDict` para uma função, os atributos `as_digested_dict` e `as_undigested_dict` podem controlar como o dicionário é carregado, passando-os como o argumento `zstd_dict`, por exemplo, `compress(data, zstd_dict=zd.as_digested_dict)`. A digestão de um dicionário é uma operação custosa que ocorre ao carregar um dicionário Zstandard. Ao fazer várias chamadas para compactação ou descompactação, passar um dicionário digerido reduzirá a sobrecarga de carregamento do dicionário.

Tabela1: Diferença para compactação

	Dicionário resumido	Dicionário não resumido
Parâmetros avançados do compressor que podem ser substituídos pelos parâmetros do dicionário	`window_log`, `hash_log`, `chain_log`, `search_log`, `min_match`, `target_length`, `strategy`, `enable_long_distance_matchin` `ldm_hash_log`, `ldm_min_match`, `ldm_bucket_size_log`, `ldm_hash_rate_log` e alguns parâmetros não-públicos.	None
`ZstdDict` armazena em cache internamente o dicionário	Sim. É mais rápido carregar um dicionário resumido novamente com o mesmo nível de compactação.	Não. Se você deseja carregar um dicionário não digerido várias vezes, considere reutilizar um objeto compressor.

Se passar um `ZstdDict` sem nenhum atributo, um dicionário não digerido será passado por padrão ao compactar e um dicionário digerido será gerado, se necessário, e passado por padrão ao descompactar.

dict_content

The content of the Zstandard dictionary, a `bytes` object. It's the same as the *dict_content* argument in the `__init__` method. It can be used with other programs, such as the `zstd` CLI program.

dict_id

Identifier of the Zstandard dictionary, a non-negative int value.

Non-zero means the dictionary is ordinary, created by Zstandard functions and following the Zstandard format.

`0` means a "raw content" dictionary, free of any format restriction, used for advanced users.

> **Nota**
>
> The meaning of `0` for `ZstdDict.dict_id` is different from the `dictionary_id` attribute to the *`get_frame_info()`* function.

as_digested_dict

Load as a digested dictionary.

as_undigested_dict

Load as an undigested dictionary.

13.2.5 Advanced parameter control

class `compression.zstd.`**CompressionParameter**

Uma *`IntEnum`* contendo as chaves de parâmetros de compactação avançada que podem ser usadas ao compactar dados.

The *`bounds()`* method can be used on any attribute to get the valid values for that parameter.

Parameters are optional; any omitted parameter will have it's value selected automatically.

Exemplo de obtenção da delimitação inferior e superior de *`compression_level`*:

```
lower, upper = CompressionParameter.compression_level.bounds()
```

Example setting the *window_log* to the maximum size:

```
_lower, upper = CompressionParameter.window_log.bounds()
options = {CompressionParameter.window_log: upper}
compress(b'venezuelan beaver cheese', options=options)
```

bounds()

Retorna a tupla de limites int, `(lower, upper)`, de um parâmetro de compressão. Este método deve ser chamado no atributo cujos limites você deseja recuperar. Por exemplo, para obter os valores válidos para *compression_level*, pode-se verificar o resultado de `CompressionParameter.compression_level.bounds()`.

Both the lower and upper bounds are inclusive.

compression_level

Um meio de alto nível para definir outros parâmetros de compactação que afetam a velocidade e a taxa de compactação de dados.

Os níveis de compactação regulares são maiores que `0`. Valores maiores que `20` são considerados compactação "ultra" e requerem mais memória do que outros níveis. Valores negativos podem ser usados para compensar uma compactação mais rápida por taxas de compactação piores.

Define o nível como zero usa *COMPRESSION_LEVEL_DEFAULT*.

window_log

Distância máxima de referência reversa permitida que o compressor pode usar ao compactar dados, expressa como potência de dois, `1 << window_log` bytes. Este parâmetro influencia bastante o uso de memória da compactação. Valores mais altos requerem mais memória, mas geram melhores valores de compactação.

A value of zero causes the value to be selected automatically.

hash_log

Tamanho da tabela de sondagem inicial, como uma potência de dois. O uso de memória resultante é `1 << (hash_log+2)` bytes. Tabelas maiores melhoram a taxa de compressão de estratégias <= *dfast* e melhoram a velocidade de compactação de estratégias > *dfast*.

A value of zero causes the value to be selected automatically.

chain_log

Tamanho da tabela de busca multiprobe, como uma potência de dois. O uso de memória resultante é `1 << (chain_log+2)` bytes. Tabelas maiores resultam em uma compactação melhor e mais lenta. Este parâmetro não tem efeito para a estratégia *fast*. Ele ainda é útil ao usar a estratégia *dfast*, caso em que define uma tabela de sondagem secundária.

A value of zero causes the value to be selected automatically.

search_log

Número de tentativas de busca, como uma potência de dois. Mais tentativas resultam em uma compressão melhor e mais lenta. Este parâmetro é inútil para as estratégias *fast* e *dfast*.

A value of zero causes the value to be selected automatically.

min_match

Tamanho mínimo das correspondências pesquisadas. Valores maiores aumentam a velocidade de compactação e descompactação, mas diminuem a taxa. Observe que o Zstandard ainda consegue encontrar correspondências de tamanho menor; ele apenas ajusta seu algoritmo de busca para procurar por esse tamanho e por tamanhos maiores. Para todas as estratégias < *btopt*, o mínimo efetivo é `4`; para todas as estratégias > *fast*, o máximo efetivo é `6`.

A value of zero causes the value to be selected automatically.

target_length

The impact of this field depends on the selected *Strategy*.

Para as estratégias *btopt*, *btultra* e *btultra2*, o valor é o comprimento de uma correspondência considerada "boa o suficiente" para interromper a busca. Valores maiores melhoram as taxas de compactação, mas a compactação é mais lenta.

Para a estratégia *fast*, é a distância entre as amostragens de correspondência. Valores maiores tornam a compactação mais rápida, mas com uma taxa de compactação pior.

A value of zero causes the value to be selected automatically.

strategy

Quanto maior o valor da estratégia selecionada, mais complexa será a técnica de compactação usada pelo zstd, resultando em taxas de compactação mais altas, mas em compactação mais lenta.

Ver também

Strategy

enable_long_distance_matching

A correspondência de longa distância pode ser usada para melhorar a compactação de entradas grandes, encontrando correspondências maiores em distâncias maiores. Ela aumenta o uso de memória e o tamanho da janela.

`True` or `1` enable long distance matching while `False` or `0` disable it.

Habilitar este parâmetro aumenta o tamanho padrão de *window_log* para 128 MiB, exceto quando expressamente definido com um valor diferente. Esta configuração é habilitada por padrão se `window_log` >= 128 MiB e a estratégia de compactação >= *btopt* (nível de compactação 16+).

ldm_hash_log

Tamanho da tabela para correspondência de longa distância, como uma potência de dois. Valores maiores aumentam o uso de memória e a taxa de compactação, mas diminuem a velocidade de compactação.

A value of zero causes the value to be selected automatically.

ldm_min_match

Tamanho mínimo de correspondência para correspondência de longa distância. Valores maiores ou muito pequenos podem frequentemente diminuir a taxa de compactação.

A value of zero causes the value to be selected automatically.

ldm_bucket_size_log

Tamanho do log de cada bucket na tabela hash do comparador de longa distância para resolução de colisões. Valores maiores melhoram a resolução de colisões, mas diminuem a velocidade de compactação.

A value of zero causes the value to be selected automatically.

ldm_hash_rate_log

Frequência de inserção/pesquisa de entradas na tabela hash do comparador de longa distância. Valores maiores melhoram a velocidade de compactação. Desvios muito grandes em relação ao valor padrão provavelmente resultarão em uma redução na taxa de compactação.

A value of zero causes the value to be selected automatically.

content_size_flag

Escreve o tamanho dos dados a serem compactados no cabeçalho do quadro Zstandard quando conhecido antes da compactação.

This flag only takes effect under the following scenarios:

- Chamando *compress()* para compactação única

- Fornecendo todos os dados a serem compactados no quadro em uma única chamada *ZstdCompressor.compress()*, com o modo *ZstdCompressor.FLUSH_FRAME*.
- Chamando *ZstdCompressor.set_pledged_input_size()* com a quantidade exata de dados que serão fornecidos ao compressor antes de qualquer chamada para *ZstdCompressor.compress()* para o quadro atual. `ZstdCompressor.set_pledged_input_size()` deve ser chamado para cada novo quadro.

Todas as outras chamadas de compressão podem não escrever as informações de tamanho no cabeçalho do quadro.

`True` or `1` enable the content size flag while `False` or `0` disable it.

checksum_flag

Uma soma de verificação de quatro bytes usando XXHash64 do conteúdo descompactado é estrita ao final de cada quadro. O código de descompactação do Zstandard verifica a soma de verificação. Se houver uma incompatibilidade, uma exceção *ZstdError* é levantada.

`True` or `1` enable checksum generation while `False` or `0` disable it.

dict_id_flag

Ao compactar com um *ZstdDict*, o ID do dicionário é escrito no cabeçalho do quadro.

`True` or `1` enable storing the dictionary ID while `False` or `0` disable it.

nb_workers

Seleciona quantas threads serão geradas para compactação em paralelo. Quando `nb_workers` > 0, habilita a compactação multithread, um valor `1` significa "modo multithread de uma thread". Mais workers melhoram a velocidade, mas também aumentam o uso de memória e reduzem ligeiramente a taxa de compactação.

A value of zero disables multi-threading.

job_size

Tamanho de um trabalho de compactação, em bytes. Este valor é aplicado somente quando *nb_workers* >= 1. Cada trabalho de compactação é concluído em paralelo, portanto, este valor pode impactar indiretamente o número de threads ativas.

A value of zero causes the value to be selected automatically.

overlap_log

Define a quantidade de dados recarregados de trabalhos anteriores (threads) para novos trabalhos a serem usados pela janela "Look Behind" durante a compactação. Este valor só é usado quando *nb_workers* >= 1. Os valores aceitáveis variam de 0 a 9.

- 0 significa definir dinamicamente a quantidade de sobreposição
- 1 significa nenhuma sobreposição
- 9 significa usar o tamanho total da janela do trabalho anterior

Cada incremento reduz pela metade/dobra o tamanho da sobreposição. "8" significa uma sobreposição de `window_size/2`, "7" significa uma sobreposição de `window_size/4`, etc.

class compression.zstd.**DecompressionParameter**

An *IntEnum* containing the advanced decompression parameter keys that can be used when decompressing data. Parameters are optional; any omitted parameter will have it's value selected automatically.

The *bounds()* method can be used on any attribute to get the valid values for that parameter.

Example setting the *window_log_max* to the maximum size:

```
data = compress(b'Some very long buffer of bytes...')

_lower, upper = DecompressionParameter.window_log_max.bounds()
```

(continua na próxima página)

(continuação da página anterior)

```
options = {DecompressionParameter.window_log_max: upper}
decompress(data, options=options)
```

bounds()

Return the tuple of int bounds, `(lower, upper)`, of a decompression parameter. This method should be called on the attribute you wish to retrieve the bounds of.

Both the lower and upper bounds are inclusive.

window_log_max

The base-two logarithm of the maximum size of the window used during decompression. This can be useful to limit the amount of memory used when decompressing data. A larger maximum window size leads to faster decompression.

A value of zero causes the value to be selected automatically.

class compression.zstd.**Strategy**

An *IntEnum* containing strategies for compression. Higher-numbered strategies correspond to more complex and slower compression.

> **Nota**
>
> The values of attributes of `Strategy` are not necessarily stable across zstd versions. Only the ordering of the attributes may be relied upon. The attributes are listed below in order.

The following strategies are available:

fast

dfast

greedy

lazy

lazy2

btlazy2

btopt

btultra

btultra2

13.2.6 Diversos

compression.zstd.**get_frame_info**(*frame_buffer*)

Retrieve a *FrameInfo* object containing metadata about a Zstandard frame. Frames contain metadata related to the compressed data they hold.

class compression.zstd.**FrameInfo**

Metadata related to a Zstandard frame.

decompressed_size

The size of the decompressed contents of the frame.

dictionary_id

An integer representing the Zstandard dictionary ID needed for decompressing the frame. `0` means the dictionary ID was not recorded in the frame header. This may mean that a Zstandard dictionary is not needed, or that the ID of a required dictionary was not recorded.

`compression.zstd.`**COMPRESSION_LEVEL_DEFAULT**

The default compression level for Zstandard: `3`.

`compression.zstd.`**zstd_version_info**

Version number of the runtime zstd library as a tuple of integers (major, minor, release).

13.2.7 Exemplos

Reading in a compressed file:

```
from compression import zstd

with zstd.open("file.zst") as f:
    file_content = f.read()
```

Creating a compressed file:

```
from compression import zstd

data = b"Insert Data Here"
with zstd.open("file.zst", "w") as f:
    f.write(data)
```

Compressing data in memory:

```
from compression import zstd

data_in = b"Insert Data Here"
data_out = zstd.compress(data_in)
```

Incremental compression:

```
from compression import zstd

comp = zstd.ZstdCompressor()
out1 = comp.compress(b"Some data\n")
out2 = comp.compress(b"Another piece of data\n")
out3 = comp.compress(b"Even more data\n")
out4 = comp.flush()
# Concatenate all the partial results:
result = b"".join([out1, out2, out3, out4])
```

Writing compressed data to an already-open file:

```
from compression import zstd

with open("myfile", "wb") as f:
    f.write(b"This data will not be compressed\n")
    with zstd.open(f, "w") as zstf:
        zstf.write(b"This *will* be compressed\n")
    f.write(b"Not compressed\n")
```

Creating a compressed file using compression parameters:

```
from compression import zstd

options = {
   zstd.CompressionParameter.checksum_flag: 1
}
with zstd.open("file.zst", "w", options=options) as f:
    f.write(b"Mind if I squeeze in?")
```

13.3 zlib — Compactação compatível com gzip

Para aplicações que exigem compactação de dados, as funções deste módulo permitem a compactação e a descompactação, usando a biblioteca zlib. A biblioteca zlib tem sua própria página em https://www.zlib.net. Existem algumas incompatibilidades conhecidas entre o Python módulo e as versões da biblioteca zlib anteriores à 1.1.3; a 1.1.3 tem uma vulnerabilidade de segurança, portanto, recomendamos o uso da 1.1.4 ou posterior.

As funções do zlib têm muitas opções e geralmente precisam ser usadas em uma ordem específica. Esta documentação não tenta cobrir todas as permutações; consulte o manual do zlib em http://www.zlib.net/manual.html para obter informações oficiais.

Para leitura e escrita de arquivos `.gz`, consulte o módulo *gzip*.

A exceção e as funções disponíveis neste módulo são:

exception zlib.**error**

Exceção levantada em erros de compactação e descompactação.

zlib.**adler32**(*data*[, *value*])

Calcula uma soma de verificação Adler-32 de *data*. (Uma soma de verificação Adler-32 é quase tão confiável quanto uma CRC32, mas pode ser calculada muito mais rapidamente.) O resultado é um número inteiro sem sinal de 32 bits. Se *value* estiver presente, ele será usado como o valor inicial da soma de verificação; caso contrário, um valor padrão de 1 é usado. A passagem de *value* permite calcular uma soma de verificação em execução através da concatenação de várias entradas. O algoritmo não é criptograficamente forte e não deve ser usado para autenticação ou assinaturas digitais. Como o algoritmo foi projetado para uso como um algoritmo de soma de verificação, não é adequado para uso como um algoritmo de hash geral.

Alterado na versão 3.0: O resultado é sempre sem sinal.

zlib.**compress**(*data*, */*, *level=-1*, *wbits=MAX_WBITS*)

Comprime os bytes em *data*, retornando um objeto de bytes que contém dados comprimidos. *level* é um inteiro de `0` a `9` ou `-1` que controla o nível de compressão; `1` (Z_BEST_SPEED) é o mais rápido e produz a menor compressão, `9` (Z_BEST_COMPRESSION) é o mais lento e produz a maior. `0` (Z_NO_COMPRESSION) não produz compactação. O valor padrão é `-1` (Z_DEFAULT_COMPRESSION). Z_DEFAULT_COMPRESSION representa um meio termo entre velocidade e compressão (atualmente equivale ao nível 6).

O argumento *wbits* controla o tamanho do buffer do histórico (ou o "tamanho da janela") usado ao compactar dados e se um cabeçalho e um trailer estão incluídos na saída. Pode levar vários intervalos de valores, padronizando para `15` (MAX_WBITS):

- +9 a +15: o logaritmo de base dois do tamanho da janela, que varia entre 512 e 32768. Valores maiores produzem melhor compactação às custas de maior uso de memória. A saída resultante incluirá um cabeçalho e uma sequência específicos para zlib.
- -9 a -15: Usa o valor absoluto de *wbits* como o logaritmo do tamanho da janela, enquanto produz um fluxo de saída bruto sem cabeçalho ou soma de verificação à direita.
- +25 a +31 = 16 + (9 a 15): Usa os 4 bits baixos do valor como logaritmo do tamanho da janela, incluindo um cabeçalho básico **gzip** e a soma de verificação à direita na saída.

Levanta uma exceção do tipo *error*, se ocorrer algum erro.

Alterado na versão 3.6: *level* pode agora ser usado como um palavra reservada nomeada.

Alterado na versão 3.11: O parâmetro *wbits* agora está disponível para definir janelas de bits e tipo de compactação.

zlib.**compressobj**(*level=-1*, *method=DEFLATED*, *wbits=MAX_WBITS*, *memLevel=DEF_MEM_LEVEL*, *strategy=Z_DEFAULT_STRATEGY*[, *zdict*])

Retorna um objeto de compactação, a ser usado para compactar fluxos de dados que não cabem na memória de uma só vez.

level é o nível de compactação – um número inteiro de `0` a `9` ou `-1`. Um valor de `1` (Z_BEST_SPEED) é mais rápido e produz a menor compactação, enquanto um valor de `9` (Z_BEST_COMPRESSION) é mais lento e produz o máximo. `0` (Z_NO_COMPRESSION) é nenhuma compactação. O valor padrão é `-1` (Z_DEFAULT_COMPRESSION). Z_DEFAULT_COMPRESSION representa um meio termo padrão entre velocidade e compactação (atualmente equivalente ao nível 6).

method é o algoritmo de compactação. Atualmente, o único valor suportado é `DEFLATED`.

O parâmetro *wbits* controla o tamanho do histórico buffer (ou o "tamanho da janela do buffer") e qual o formato do cabeçalho e do trailer serão usados. Ele tem o mesmo significado que o *descrito para compress()*.

O argumento *memLevel* controla a quantidade de memória usada para o estado de compactação interno. Os valores válidos variam de `1` a `9`. Valores mais altos usam mais memória, mas são mais rápidos e produzem uma saída menor.

strategy é usado para ajustar o algoritmo de compactação. Os valores possíveis são `Z_DEFAULT_STRATEGY`, `Z_FILTERED`, `Z_HUFFMAN_ONLY`, `Z_RLE` (zlib 1.2.0.1) e `Z_FIXED` (zlib 1.2.2.2).

zdict é um dicionário de compactação predefinido. Esta é uma sequência de bytes (como um objeto *bytes*) que contém subsequências que se espera que ocorram com frequência nos dados a serem compactados. As subsequências que se espera serem mais comuns devem aparecer no final do dicionário.

Alterado na versão 3.3: Adicionado o suporte ao parâmetro e argumento nomeado *zdict*.

zlib.**crc32**(*data*[, *value*])

Calcula uma soma de verificação CRC (Cyclic Redundancy Check) de *data*. O resultado é um número inteiro sem sinal de 32 bits. Se *value* estiver presente, ele será usado como o valor inicial da soma de verificação; caso contrário, um valor padrão de 1 é usado. A passagem de *value* permite calcular uma soma de verificação em execução através da concatenação de várias entradas. O algoritmo não é criptograficamente forte e não deve ser usado para autenticação ou assinaturas digitais. Como o algoritmo foi projetado para uso como um algoritmo de soma de verificação, não é adequado para uso como um algoritmo de hash geral.

Alterado na versão 3.0: O resultado é sempre sem sinal.

zlib.**decompress**(*data*, */*, *wbits=MAX_WBITS*, *bufsize=DEF_BUF_SIZE*)

Descompacta os bytes em *data*, retornando um objeto de bytes que contém os dados não compactados. O parâmetro *wbits* depende do formato de *data* e é discutido mais abaixo. Se *bufsize* for fornecido, ele será usado como o tamanho inicial do buffer de saída. Levanta a exceção *error* se ocorrer algum erro.

O parâmetro *wbits* controla o tamanho do buffer do histórico (ou "tamanho da janela") e qual formato de cabeçalho e sequência é esperado. É semelhante ao parâmetro para *compressobj()*, mas aceita mais intervalos de valores:

- +8 a +15: O logaritmo de base dois do tamanho da janela. A entrada deve incluir um cabeçalho e uma sequência de zlib.
- 0: Determina automaticamente o tamanho da janela no cabeçalho zlib. Suportado apenas desde o zlib 1.2.3.5.
- −8 a −15: Usa o valor absoluto de *wbits* como o logaritmo do tamanho da janela. A entrada deve ser um fluxo bruto sem cabeçalho ou sequência.
- +24 a +31 = 16 + (8 a 15): Usa os 4 bits baixos do valor como logaritmo do tamanho da janela. A entrada deve incluir um cabeçalho e sequência de gzip.

- +40 a +47 = 32 + (8 a 15): Usa os 4 bits baixos do valor como logaritmo do tamanho da janela e aceita automaticamente o formato zlib ou gzip.

Ao descompactar um fluxo, o tamanho da janela não deve ser menor que o tamanho originalmente usado para compactar o fluxo; o uso de um valor muito pequeno pode resultar em uma exceção `error`. O valor padrão *wbits* corresponde ao maior tamanho da janela e requer que um cabeçalho e uma sequência de zlib sejam incluídos.

bufsize é o tamanho inicial do buffer usado para armazenar dados descompactados. Se for necessário mais espaço, o tamanho do buffer será aumentado conforme necessário, para que você não precise obter esse valor exatamente correto; sintonizando, apenas algumas chamadas serão salvas em `malloc()`.

Alterado na versão 3.6: *wbits* e *bufsize* podem ser usados como argumentos nomeados.

`zlib.`**`decompressobj`**(*wbits=MAX_WBITS*[, *zdict*])

Retorna um objeto descompactado, a ser usado para descompactar fluxos de dados que não cabem na memória de uma só vez.

O parâmetro *wbits* controla o tamanho do histórico do buffer (ou o "tamanho da janela do buffer") e qual formato do cabeçalho e trailer são esperados. Ele tem o mesmo significado que o *descrito para decompress()*.

O parâmetro *zdict* especifica uma dicionário pre-definido de compressão. Se fornecido, deve ser o mesmo dicionário usado pelo compressor que produziu os dados a serem descompactados.

> **Nota**
>
> Se *zdict* for um objeto mutável (como um `bytearray`), você não deve modificar seu conteúdo entre a chamada de `decompressobj()` e a primeira chamada para o método de descompatação `decompress()`.

Alterado na versão 3.3: Adicionado o parâmetro *zdict*.

Um objeto do tipo Compress oferece suporte aos seguintes métodos:

`Compress.`**`compress`**(*data*)

Comprime *data*, retornando um objeto de bytes que contém dados compactados para pelo menos parte dos dados em *data*. Esses dados devem ser concatenados à saída produzida por quaisquer chamadas anteriores ao método `compress()`. Algumas entradas podem ser mantidas em buffers internos para processamento posterior.

`Compress.`**`flush`**([*mode*])

Toda a entrada pendente é processada e um objeto de bytes contendo a saída compactada restante é retornado. O *mode* pode ser selecionado entre constantes `Z_NO_FLUSH` , `Z_PARTIAL_FLUSH`, `Z_SYNC_FLUSH`, `Z_FULL_FLUSH`, `Z_BLOCK` (zlib 1.2.3.4) ou `Z_FINISH`, com o padrão sendo `Z_FINISH`. Exceto `Z_FINISH`, todas as demais constantes permitem a compactação de mais bytestrings de dados, enquanto `Z_FINISH` finaliza o fluxo compactado e impede a compactação de mais dados. Depois de chamar `flush()` com *mode* definido como `Z_FINISH`, o método `compress()` não pode ser chamado novamente; a única ação restante possível é excluir o objeto.

`Compress.`**`copy`**()

Retorna uma cópia do objeto de compactação. Isso pode ser usado para compactar com eficiência um conjunto de dados que compartilham um prefixo inicial comum.

Alterado na versão 3.8: As funções `copy.copy()` e `copy.deepcopy()` foram adicionadas como suporte para a compressão de objetos.

Um objeto do tipo Decompress oferece suporte aos seguintes métodos:

`Decompress.`**`unused_data`**

Um objeto de bytes que contém todos os bytes após o final dos dados compactados. Ou seja, ele permanece `b""` até que o último byte que contém dados compactados esteja disponível. Se todo o bytestring contiver dados compactados, este será `b""`, um objeto de bytes vazio.

Decompress.**unconsumed_tail**

Um objeto de bytes que contém todos os dados que não foram consumidos pela última chamada *decompress()* porque excederam o limite dos dados não compactados no buffer. Esses dados ainda não foram vistos pela zlib, portanto, você deve alimentá-los (possivelmente com outros dados concatenados a eles) em uma chamada subsequente para o método *decompress()* e, com isso, obter a saída correta.

Decompress.**eof**

Um booleano indicando se o fim do fluxo de dados compactados foi alcançado.

Isso permite distinguir entre um fluxo compactado formado corretamente e um fluxo incompleto ou truncado.

Adicionado na versão 3.3.

Decompress.**decompress**(*data*, *max_length=0*)

Descompacta *data*, retornando um objeto de bytes que contém os dados não compactados correspondentes a pelo menos uma parte dos dados em *string*. Esses dados devem ser concatenados com a saída produzida por quaisquer chamadas anteriores ao método *decompress()* . Alguns dos dados de entrada podem ser preservados em buffers internos para processamento posterior.

Se o parâmetro opcional *max_length* for diferente de zero, o valor retornado não será maior que *max_length*. Isso pode significar que nem toda a entrada compactada poderá ser processada, e os dados não consumidos serão armazenados no atributo *unconsumed_tail* . Esse bytestring deve ser passado para uma chamada subsequente a *decompress()* se a descompressão tiver que continuar. Se *max_length* for zero, toda a entrada será descompactada e *unconsumed_tail* ficará vazio.

Alterado na versão 3.6: *max_length* pode ser usado como argumento nomeado.

Decompress.**flush**(*[length]*)

Toda a entrada que estiver pendente é processada e um objeto de bytes contendo a saída descompactada restante é retornado. Depois de chamar *flush()*, o método *decompress()* não pode ser chamado novamente; a única ação possível é excluir o objeto.

O parâmetro opcional *comprimento* define o tamanho inicial da saída do buffer.

Decompress.**copy**()

Retorna uma cópia do objeto de descompressão. Isso pode ser usado para salvar o estado do descompressor no meio do fluxo de dados, a fim de acelerar as buscas aleatórias no fluxo em um ponto futuro.

Alterado na versão 3.8: As funções *copy.copy()* e *copy.deepcopy()* foram adicionadas como suporte para a descompressão de objetos.

As informações sobre o versão da biblioteca zlib em uso estão disponíveis no seguinte constantes:

zlib.**ZLIB_VERSION**

Uma string com a versão da biblioteca zlib que foi usada para construir o módulo. Isso pode ser diferente da biblioteca zlib realmente usada no tempo de execução, que está disponível como *ZLIB_RUNTIME_VERSION*.

zlib.**ZLIB_RUNTIME_VERSION**

Uma string com a versão da biblioteca zlib atualmente utilizada pelo interpretador.

Adicionado na versão 3.3.

zlib.**ZLIBNG_VERSION**

A string de versão da biblioteca zlib-ng que foi usada para construir o módulo, caso o zlib-ng tenha sido usado. Quando presentes, as constantes *ZLIB_VERSION* e *ZLIB_RUNTIME_VERSION* refletem a versão da API zlib fornecida pelo zlib-ng.

Se zlib-ng não foi usado para construir o módulo, esta constante estará ausente.

Adicionado na versão 3.14.

Ver também

módulo *gzip*
Leia e escreva arquivos no formato **gzip**

http://www.zlib.net
A página inicial da biblioteca zlib.

http://www.zlib.net/manual.html
O manual da zlib explica a semântica e uso de diversas funções desta biblioteca.

Caso a (des)compressão do gzip seja um gargalo, o pacote python-isal acelera a (des)compressão com uma API quase sempre compatível.

13.4 `gzip` — Suporte para arquivos `gzip`

Código-fonte: Lib/gzip.py

Esse módulo disponibiliza uma interface simples para comprimir e extrair arquivos como os programas GNU **gzip** e **gunzip** fazem.

A compressão de dados é realizada pelo módulo *zlib*.

O módulo *gzip* fornece a classe *GzipFile*, bem como as funções de conveniência *open()*, *compress()* e *decompress()*. A classe *GzipFile* lê e grava arquivos no formato **gzip**, compactando ou descompactando automaticamente os dados para que se pareçam com um *objeto arquivo* comum.

Observe que formatos de arquivo adicionais que podem ser descompactados pelos programas **gzip** e **gunzip**, como aqueles produzidos por **compress** e **pack**, não são suportados por este módulo.

Este módulo define os seguintes itens:

`gzip.`**`open`**(*filename*, *mode='rb'*, *compresslevel=9*, *encoding=None*, *errors=None*, *newline=None*)

Abre um arquivo comprimido com gzip em modo binário ou texto, retornando *objeto arquivo*.

O argumento *filename* pode ser um nome de arquivo real (um objeto *str* ou *bytes*), ou um objeto arquivo existente para ler ou gravar.

O argumento *mode* pode ser qualquer um de `'r'`, `'rb'`, `'a'`, `'ab'`, `'w'`, `'wb'`, `'x'` ou `'xb'` para modo binário, ou `'rt'`, `'at'`, `'wt'` ou `'xt'` para modo texto. O padrão é `'rb'`.

O argumento *compresslevel* é um inteiro de 0 a 9, como para o construtor de *GzipFile*.

Para o modo binário, esta função equivale ao construtor de *GzipFile*: `GzipFile(filename, mode, compresslevel)`. Neste caso, os argumentos *encoding*, *errors* e *newline* não devem ser fornecidos.

Para o modo texto, um objeto *GzipFile* é criado e encapsulado em uma instância *io.TextIOWrapper* com a codificação especificada, comportamento de tratamento de erros e final(is) de linha.

Alterado na versão 3.3: Adicionado suporte para *filename* ser um objeto arquivo, suporte para modo texto e os argumentos *encoding*, *errors* e *newline*.

Alterado na versão 3.4: Adicionado suporte para os modos `'x'`, `'xb'` e `'xt'`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

exception `gzip.`**`BadGzipFile`**

Uma exceção levantada para arquivos gzip inválidos. Herda de *OSError*. *EOFError* e *zlib.error* também podem ser levantadas para arquivos gzip inválidos.

Adicionado na versão 3.8.

class gzip.**GzipFile**(*filename=None*, *mode=None*, *compresslevel=9*, *fileobj=None*, *mtime=None*)

Construtor para a classe *GzipFile*, que simula a maioria dos métodos de um *objeto arquivo*, com exceção do método *truncate()*. Pelo menos um dos métodos *fileobj* e *filename* deve receber um valor não trivial.

A nova instância de classe é baseada em *fileobj*, que pode ser um arquivo comum, um objeto *io.BytesIO* ou qualquer outro objeto que simule um arquivo. O padrão é None, caso em que *filename* é aberto para fornecer um objeto arquivo.

Quando *fileobj* não é None, o argumento *filename* é usado apenas para ser incluído no cabeçalho do arquivo **gzip**, que pode incluir o nome original do arquivo descompactado. O padrão é o nome de arquivo *fileobj*, se discernível; caso contrário, o padrão é a string vazia e, neste caso, o nome original do arquivo não é incluído no cabeçalho.

O argumento *mode* pode ser qualquer um dos seguintes: 'r', 'rb', 'a', 'ab', 'w', 'wb', 'x' ou 'xb', dependendo se o arquivo será lido ou escrito. O padrão é o modo de *fileobj*, se discernível; caso contrário, o padrão é 'rb'. Em versões futuras do Python, o modo de *fileobj* não será usado. É melhor sempre especificar *mode* para escrita.

Observe que o arquivo é sempre aberto em modo binário. Para abrir um arquivo compactado em modo texto, use *open()* (ou envolva seu *GzipFile* com um *io.TextIOWrapper*).

O argumento *compresslevel* é um inteiro de 0 a 9 que controla o nível de compressão; 1 é o mais rápido e produz a menor compressão, e 9 é o mais lento e produz a maior compressão. 0 significa sem compressão. O padrão é 9.

O argumento opcional *mtime* é o registro de data e hora solicitado pelo gzip. O horário está no formato Unix, ou seja, segundos desde 00:00:00 UTC, 1º de janeiro de 1970. Se *mtime* for omitido ou definido como None, o horário atual será usado. Use *mtime* = 0 para gerar um fluxo compactado que não depende do horário de criação.

Veja abaixo o atributo *mtime* que é definido durante a descompactação.

Chamar o método close() de um objeto *GzipFile* não fecha *fileobj*, pois você pode querer adicionar mais material após os dados compactados. Isso também permite que você passe um objeto *io.BytesIO* aberto para escrita como *fileobj* e recupere o buffer de memória resultante usando o método *getvalue()* do objeto *io.BytesIO*.

GzipFile suporta a interface *io.BufferedIOBase*, incluindo iteração e a instrução with. Apenas o método *truncate()* não é implementado.

GzipFile também disponibiliza os seguintes métodos e atributos:

peek(*n*)

Lê *n* bytes não compactados sem avançar a posição do arquivo. O número de bytes retornados pode ser maior ou menor que o solicitado.

> **Nota**
>
> Embora chamar *peek()* não altere a posição do arquivo *GzipFile*, ele pode alterar a posição do objeto arquivo subjacente (por exemplo, se o *GzipFile* foi construído com o parâmetro *fileobj*).

Adicionado na versão 3.2.

mode

'rb' para leitura e 'wb' para escrita.

Alterado na versão 3.13: Em versões anteriores era um inteiro 1 ou 2.

mtime

Ao descompactar, este atributo é definido como o último registro de data e hora no cabeçalho lido mais recentemente. É um inteiro que contém o número de segundos desde a era Unix (00:00:00 UTC, 1º de janeiro de 1970). O valor inicial antes da leitura de qualquer cabeçalho é None.

name

O caminho para o arquivo gzip no disco, como *str* ou *bytes*. Equivalente à saída de *os.fspath()* no caminho de entrada original, sem nenhuma outra normalização, resolução ou expansão.

Alterado na versão 3.1: Foi adicionado suporte para a instrução `with`, juntamente com o argumento do construtor *mtime* e o atributo *mtime*.

Alterado na versão 3.2: Foi adicionado suporte para arquivos preenchidos com zeros e não pesquisáveis.

Alterado na versão 3.3: O método *io.BufferedIOBase.read1()* agora está implementado.

Alterado na versão 3.4: Adicionado suporte para os modos `'x'` e `'xb'`.

Alterado na versão 3.5: Adicionado suporte para escrever *objetos bytes ou similares* arbitrários. O método *read()* agora aceita o argumento `None`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Descontinuado desde a versão 3.9: Abrir *GzipFile* para escrita sem especificar o argumento *mode* está descontinuado.

Alterado na versão 3.12: Remove o atributo `filename` e usa o atributo *name*.

`gzip.`**`compress`**(*data*, *compresslevel=9*, ***, *mtime=0*)

Compacta *data*, retornando um objeto *bytes* contendo os dados compactados. *compresslevel* e *mtime* têm o mesmo significado que no construtor de *GzipFile* acima, mas *mtime* assume o valor padrão 0 para uma saída reprodutível.

Adicionado na versão 3.2.

Alterado na versão 3.8: Adicionado o parâmetro *mtime* para saída reproduzível.

Alterado na versão 3.11: A velocidade é melhorada pela compactação de todos os dados de uma só vez, em vez de em fluxo contínuo. Chamadas com *mtime* definido como `0` são delegadas a *zlib.compress()* para maior velocidade. Nessa situação, a saída pode conter um valor de byte "OS" no cabeçalho gzip diferente de 255 "unknown", conforme fornecido pela implementação subjacente do zlib.

Alterado na versão 3.13: O byte do cabeçalho gzip do sistema operacional tem a garantia de ser definido como 255 quando esta função é usada, como era o caso na versão 3.10 e anteriores.

Alterado na versão 3.14: O parâmetro *mtime* agora assume o valor padrão 0 para uma saída reprodutível. Para o comportamento anterior de usar a hora atual, passe `None` para *mtime*.

`gzip.`**`decompress`**(*data*)

Descompacta *data*, retornando um objeto *bytes* contendo os dados descompactados. Esta função é capaz de descompactar dados de gzip de vários membros (vários blocos gzip concatenados). Quando é certo que os dados contêm apenas um membro, a função *zlib.decompress()* com *wbits* definido como 31 é mais rápida.

Adicionado na versão 3.2.

Alterado na versão 3.11: A velocidade é melhorada ao descompactar os membros de uma só vez na memória, em vez de fazê-lo de forma contínua.

13.4.1 Exemplos de uso

Exemplo de como ler um arquivo comprimido:

```
import gzip
with gzip.open('/home/joe/arquivo.txt.gz', 'rb') as f:
    file_content = f.read()
```

Exemplo de como criar um arquivo comprimido com GZIP:

```
import gzip
content = b"Muito conteúdo aqui"
with gzip.open('/home/joe/arquivo.txt.gz', 'wb') as f:
    f.write(content)
```

Exemplo de como comprimir um arquivo existente com GZIP:

```
import gzip
import shutil
with open('/home/joe/arquivo.txt', 'rb') as f_in:
    with gzip.open('/home/joe/arquivo.txt.gz', 'wb') as f_out:
        shutil.copyfileobj(f_in, f_out)
```

Exemplo de como comprimir uma string binária com compressão GZIP:

```
import gzip
s_in = b"Muito conteúdo aqui"
s_out = gzip.compress(s_in)
```

Ver também

Módulo *zlib*
O módulo básico de compactação de dados necessário para dar suporte ao formato de arquivo do **gzip**.

Caso a (des)compressão do gzip seja um gargalo, o pacote python-isal acelera a (des)compressão com uma API quase sempre compatível.

13.4.2 Interface de linha de comando

O módulo *gzip* fornece uma interface de linha de comando simples para compactar ou descompactar arquivos.

Uma vez executado, o módulo *gzip* mantém o(s) arquivo(s) de entrada.

Alterado na versão 3.8: Adiciona uma nova interface de linha de comando com um mensagem de uso. Por padrão, ao executar a CLI, o nível de compactação padrão é 6.

Opções da linha de comando

file
Se o *arquivo* não for especificado, ler de *sys.stdin*

--fast
Indica o método mais rápido de compressão (menor compressão)

--best
Indica o método mais lento de compressão (melhor compressão).

-d, --decompress
Descompacta o arquivo dado.

-h, --help
Exibe a mensagem de ajuda.

13.5 bz2 — Suporte para compressão bzip2

Código-fonte: Lib/bz2.py

Este módulo fornece uma interface abrangente para compactar e descompactar dados usando o algoritmo de compactação bzip2.

O módulo *bz2* contém:

- A função *open()* e a classe *BZ2File* para leitura e escrita de arquivos compactados.
- As classes *BZ2Compressor* e *BZ2Decompressor* para (des)compressão incremental.
- As funções *compress()* e *decompress()* para (des)compressão de uma só vez.

13.5.1 (Des)compressão de arquivos

bz2.**open**(*filename*, *mode='rb'*, *compresslevel=9*, *encoding=None*, *errors=None*, *newline=None*)

Abre um arquivo compactado com bzip2 no modo binário ou texto, retornando um *objeto arquivo*.

Assim como no construtor para *BZ2File*, o argumento *filename* pode ser um nome de arquivo real (um objeto *str* ou *bytes*), ou um objeto arquivo existente para ler ou gravar.

O argumento *mode* pode ser qualquer um de `'r'`, `'rb'`, `'w'`, `'wb'`, `'x'`, `'xb'`, `'a'` ou `'ab'` para modo binário, ou `'rt'`, `'wt'`, `'xt'` ou `'at'` para modo texto. O padrão é `'rb'`.

O argumento *compresslevel* é um inteiro de 1 a 9, como para o construtor *BZ2File*.

Para o modo binário, esta função é equivalente ao construtor de *BZ2File*: `BZ2File(filename, mode, compresslevel=compresslevel)`. Neste caso, os argumentos *encoding*, *errors* e *newline* não devem ser fornecidos.

Para o modo texto, um objeto *BZ2File* é criado e envolto em uma instância *io.TextIOWrapper* com a codificação especificada, comportamento de tratamento de erros e final(is) de linha.

Adicionado na versão 3.3.

Alterado na versão 3.4: O modo `'x'` (criação exclusiva) foi adicionado.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

class bz2.**BZ2File**(*filename*, *mode='r'*, ***, *compresslevel=9*)

Abre um arquivo compactado com bzip2 no modo binário.

Se *filename* for um objeto *str* ou *bytes*, abra o arquivo nomeado diretamente. Caso contrário, *filename* deve ser um *objeto arquivo*, que será usado para ler ou gravar os dados compactados.

O argumento *mode* pode ser `'r'` para leitura (padrão), `'w'` para substituição, `'x'` para criação exclusiva ou `'a'` para anexar. Estes podem ser equivalentemente dados como `'rb'`, `'wb'`, `'xb'` e `'ab'` respectivamente.

Se *filename* for um objeto arquivo (ao invés de um nome de arquivo real), um modo de `'w'` não truncará o arquivo e será equivalente a `'a'`.

Se *mode* for `'w'` ou `'a'`, *compresslevel* pode ser um inteiro entre `1` e `9` especificando o nível de compressão: `1` produz a menor compressão e `9` (padrão) produz a maior compactação.

Se *mode* for `'r'`, o arquivo de entrada pode ser a concatenação de vários fluxos compactados.

BZ2File fornece todos os membros especificados pelo *io.BufferedIOBase*, exceto *detach()* e *truncate()*. Iteração e a instrução `with` são suportadas.

BZ2File também fornece os seguintes métodos e atributos:

peek([*n*])

Retorna dados armazenados em buffer sem avançar a posição do arquivo. Pelo menos um byte de dados será retornado (a menos que em EOF). O número exato de bytes retornados não é especificado.

> **Nota**
>
> Enquanto chamar `peek()` não altera a posição do arquivo de `BZ2File`, pode alterar a posição do objeto arquivo subjacente (por exemplo, se o `BZ2File` foi construído passando um objeto arquivo para *filename*).

Adicionado na versão 3.3.

fileno()

Retorna o endereço descritor de arquivo do arquivo subjacente.

Adicionado na versão 3.3.

readable()

Retorna se o arquivo foi aberto para leitura.

Adicionado na versão 3.3.

seekable()

Retorna se o arquivo suporta a busca.

Adicionado na versão 3.3.

writable()

Retorna se o arquivo foi aberto para gravação.

Adicionado na versão 3.3.

read1(*size=-1*)

Lê até o tamanho *size* de bytes não compactados, tentando evitar várias leituras do fluxo subjacente. Lê até um valor buffer de dados se o tamanho for negativo.

Retorna `b''` se o arquivo tiver atingido EOF, ou seja, o fim do arquivo.

Adicionado na versão 3.3.

readinto(*b*)

Lê bytes para *b*.

Retorna o número de bytes lidos (0 para EOF).

Adicionado na versão 3.3.

mode

`'rb'` para leitura e `'wb'` para escrita.

Adicionado na versão 3.13.

name

O nome do arquivo bzip2. Equivalente ao atributo `name` do *objeto arquivo* subjacente.

Adicionado na versão 3.13.

Alterado na versão 3.1: Suporte para a instrução `with` foi adicionado.

Alterado na versão 3.3: Foi adicionado suporte para *filename* ser um *objeto arquivo* em vez de um nome de arquivo real.

O modo `'a'` (anexar) foi adicionado, juntamente com suporte para leitura de arquivos multifluxo.

Alterado na versão 3.4: O modo `'x'` (criação exclusiva) foi adicionado.

Alterado na versão 3.5: O método `read()` agora aceita um argumento de `None`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.9: O parâmetro *buffering* foi removido. Foi ignorado e descontinuado desde o Python 3.0. Passe um objeto arquivo aberto para controlar como o arquivo é aberto.

O parâmetro *compresslevel* tornou-se somente-nomeado.

Alterado na versão 3.10: Esta classe não é segura para threads diante de vários leitores ou escritores simultâneos, assim como suas classes equivalentes em `gzip` e `lzma` sempre foram.

13.5.2 (Des)compressão incremental

class bz2.**BZ2Compressor**(*compresslevel=9*)

Cria um novo objeto compressor. Este objeto pode ser usado para compactar dados de forma incremental. Para compactação única, use a função `compress()`.

compresslevel, se fornecido, deve ser um inteiro entre `1` e `9`. O padrão é `9`.

compress(*data*)

Fornece dados para o objeto compressor. Retorna um pedaço de dados compactados, se possível, ou uma string de bytes vazia, caso contrário.

Quando você terminar de fornecer dados ao compactador, chame o método `flush()` para finalizar o processo de compressão.

flush()

Finaliza o processo de compactação. Retorna os dados compactados deixados em buffers internos.

O objeto compactador não pode ser usado após a chamada deste método.

class bz2.**BZ2Decompressor**

Cria um novo objeto descompactador. Este objeto pode ser usado para descompactar dados de forma incremental. Para compactação única, use a função `decompress()`.

> **Nota**
>
> Esta classe não trata de forma transparente entradas contendo múltiplos fluxos compactados, ao contrário de `decompress()` e `BZ2File`. Se você precisar descompactar uma entrada multifluxo com `BZ2Decompressor`, você deve usar um novo descompactador para cada fluxo.

decompress(*data*, *max_length=-1*)

Descompacta dados *data* (um *objeto bytes ou similar*), retornando dados não compactados como bytes. Alguns dos *data* podem ser armazenados em buffer internamente, para uso em chamadas posteriores para `decompress()`. Os dados retornados devem ser concatenados com a saída de qualquer chamada anterior para `decompress()`.

Se *max_length* for não negativo, retornará no máximo *max_length* bytes de dados descompactados. Se este limite for atingido e mais saída puder ser produzida, o atributo `needs_input` será definido como `False`. Neste caso, a próxima chamada para `decompress()` pode fornecer *data* como `b''` para obter mais saída.

Se todos os dados de entrada foram descompactados e retornados (seja porque era menor que *max_length* bytes, ou porque *max_length* era negativo), o atributo `needs_input` será definido como `True` .

A tentativa de descompactar os dados após o final do fluxo ser atingido gera um `EOFError`. Quaisquer dados encontrados após o final do fluxo são ignorados e salvos no atributo `unused_data`.

Alterado na versão 3.5: Adicionado o parâmetro *max_length*.

eof

`True` se o marcador de fim de fluxo foi atingido.

Adicionado na versão 3.3.

unused_data

Dados encontrados após o término do fluxo compactado.

Se este atributo for acessado antes do final do fluxo ser alcançado, seu valor será `b''`.

needs_input

`False` se o método *decompress()* puder fornecer mais dados descompactados antes de exigir uma nova entrada descompactada.

Adicionado na versão 3.5.

13.5.3 (De)compressão de uma só vez (one-shot)

`bz2.`**compress**(*data*, *compresslevel=9*)

Compacta *data*, um *objeto bytes ou similar*.

compresslevel, se fornecido, deve ser um inteiro entre `1` e `9`. O padrão é `9`.

Para compressão incremental, use um *BZ2Compressor*.

`bz2.`**decompress**(*data*)

Descompacta *data*, um *objeto bytes ou similar*.

Se *data* for a concatenação de vários fluxos compactados, descompacta todos os fluxos.

Para descompressão incremental, use um *BZ2Decompressor*.

Alterado na versão 3.3: Suporte para entradas multifluxo foi adicionado.

13.5.4 Exemplos de uso

Abaixo estão alguns exemplos de uso típico do módulo *bz2*.

Usando *compress()* e *decompress()* para demonstrar a compactação de ida e volta:

```
>>> import bz2
>>> data = b"""\
... Donec rhoncus quis sapien sit amet molestie. Fusce scelerisque vel augue
... nec ullamcorper. Nam rutrum pretium placerat. Aliquam vel tristique lorem,
... sit amet cursus ante. In interdum laoreet mi, sit amet ultrices purus
... pulvinar a. Nam gravida euismod magna, non varius justo tincidunt feugiat.
... Aliquam pharetra lacus non risus vehicula rutrum. Maecenas aliquam leo
... felis. Pellentesque semper nunc sit amet nibh ullamcorper, ac elementum
... dolor luctus. Curabitur lacinia mi ornare consectetur vestibulum."""
>>> c = bz2.compress(data)
>>> len(data) / len(c)  # Data compression ratio
1.513595166163142
>>> d = bz2.decompress(c)
>>> data == d  # Check equality to original object after round-trip
True
```

Usando *BZ2Compressor* para compressão incremental:

```
>>> import bz2
>>> def gen_data(chunks=10, chunksize=1000):
...     """Yield incremental blocks of chunksize bytes."""
...     for _ in range(chunks):
...         yield b"z" * chunksize
...
>>> comp = bz2.BZ2Compressor()
>>> out = b""
>>> for chunk in gen_data():
```

(continua na próxima página)

(continuação da página anterior)

```
...     # Provide data to the compressor object
...     out = out + comp.compress(chunk)
...
>>> # Finish the compression process.  Call this once you have
>>> # finished providing data to the compressor.
>>> out = out + comp.flush()
```

O exemplo acima usa um fluxo de dados muito "não aleatório" (um fluxo de partes `b"z"`). Dados aleatórios tendem a compactar mal, enquanto dados ordenados e repetitivos geralmente produzem uma alta taxa de compactação.

Escrevendo e lendo um arquivo compactado com bzip2 no modo binário:

```
>>> import bz2
>>> data = b"""\
... Donec rhoncus quis sapien sit amet molestie. Fusce scelerisque vel augue
... nec ullamcorper. Nam rutrum pretium placerat. Aliquam vel tristique lorem,
... sit amet cursus ante. In interdum laoreet mi, sit amet ultrices purus
... pulvinar a. Nam gravida euismod magna, non varius justo tincidunt feugiat.
... Aliquam pharetra lacus non risus vehicula rutrum. Maecenas aliquam leo
... felis. Pellentesque semper nunc sit amet nibh ullamcorper, ac elementum
... dolor luctus. Curabitur lacinia mi ornare consectetur vestibulum."""
>>> with bz2.open("myfile.bz2", "wb") as f:
...     # Write compressed data to file
...     unused = f.write(data)
...
>>> with bz2.open("myfile.bz2", "rb") as f:
...     # Decompress data from file
...     content = f.read()
...
>>> content == data  # Check equality to original object after round-trip
True
```

13.6 `lzma` — Compressão usando o algoritmo LZMA

Adicionado na versão 3.3.

Código-fonte: Lib/lzma.py

Este módulo fornece classes e funções de conveniência para comprimir e descomprimir dados usando o algoritmo de compressão LZMA. Também está incluída uma interface de arquivo que oferece suporte aos formatos de arquivo `.xz` e legado `.lzma` usados pelo utilitário **xz**, bem como fluxos brutos comprimidos.

A interface fornecida por este módulo é muito semelhante à do módulo *bz2*. Observe que *LZMAFile* e *bz2.BZ2File* *não* são seguro para thread, portanto, se você precisar usar uma única instância *LZMAFile* de vários threads, é necessário protegê-la com uma trava.

exception lzma.**LZMAError**

Essa exceção é levantada quando ocorre um erro durante a compressão ou descompressão ou durante a inicialização do estado compressor/descompressor.

13.6.1 Lendo e escrevendo arquivos comprimidos

lzma.**open**(*filename*, *mode='rb'*, ***, *format=None*, *check=-1*, *preset=None*, *filters=None*, *encoding=None*, *errors=None*, *newline=None*)

Abre um arquivo comprimido com LZMA no modo binário ou texto, retornando um *objeto arquivo*.

O argumento *filename* pode ser um nome de arquivo real (dado como um objeto `str`, `bytes` ou *caminho ou similar*), neste caso o arquivo nomeado é aberto , ou pode ser um objeto arquivo existente para leitura ou escrita.

O argumento *mode* pode ser qualquer um de `"r"`, `"rb"`, `"w"`, `"wb"`, `"x"`, `"xb"`, `"a"` ou `"ab"` para modo binário, ou `"rt"`, `"wt"`, `"xt"` , ou `"at"` para o modo de texto. O padrão é `"rb"`.

Ao abrir um arquivo para leitura, os argumentos *format* e *filters* têm os mesmos significados que em `LZMADecompressor`. Neste caso, os argumentos *check* e *preset* não devem ser usados.

Ao abrir um arquivo para escrita, os argumentos *format*, *check*, *preset* e *filters* têm os mesmos significados que em `LZMACompressor`.

Para o modo binário, esta função é equivalente ao construtor `LZMAFile`: `LZMAFile(filename, mode, ...)`. Nesse caso, os argumentos *encoding*, *errors* e *newline* não devem ser fornecidos.

Para o modo texto, um objeto `LZMAFile` é criado e encapsulado em uma instância `io.TextIOWrapper` com a codificação especificada, comportamento de tratamento de erros e final(is) de linha.

Alterado na versão 3.4: Adicionado suporte para os modos `"x"`, `"xb"` e `"xt"`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

class lzma.**LZMAFile**(*filename=None*, *mode='r'*, ***, *format=None*, *check=-1*, *preset=None*, *filters=None*)

Abre um arquivo comprimido com LZMA no modo binário.

Um `LZMAFile` pode envolver um *objeto arquivo* já aberto, ou operar diretamente em um arquivo nomeado. O argumento *filename* especifica o objeto arquivo a ser encapsulado ou o nome do arquivo a ser aberto (como um objeto `str`, `bytes` ou *objeto caminho ou similar*). Ao agrupar um objeto arquivo existente, o arquivo agrupado não será fechado quando o `LZMAFile` for fechado.

O argumento *mode* pode ser `"r"` para leitura (padrão), `"w"` para substituição, `"x"` para criação exclusiva ou `"a"` para anexar. Estes podem ser equivalentemente dados como `"rb"`, `"wb"`, `"xb"` e `"ab"` respectivamente.

Se *filename* for um objeto arquivo (em vez de um nome de arquivo real), um modo de `"w"` não truncará o arquivo e será equivalente a `"a"`.

Ao abrir um arquivo para leitura, o arquivo de entrada pode ser a concatenação de vários fluxos comprimidos separados. Estes são decodificados de forma transparente como um único fluxo lógico.

Ao abrir um arquivo para leitura, os argumentos *format* e *filters* têm os mesmos significados que em `LZMADecompressor`. Neste caso, os argumentos *check* e *preset* não devem ser usados.

Ao abrir um arquivo para escrita, os argumentos *format*, *check*, *preset* e *filters* têm os mesmos significados que em `LZMACompressor`.

`LZMAFile` suporta todos os membros especificados por `io.BufferedIOBase`, exceto `detach()` e `truncate()`. Iteração e a instrução `with` são suportadas.

O método e os atributos a seguir também são fornecidos:

peek(*size=-1*)

Retorna dados armazenados em buffer sem avançar a posição do arquivo. Pelo menos um byte de dados será retornado, a menos que o EOF tenha sido atingido. O número exato de bytes retornados não é especificado (o argumento *size* é ignorado).

> **Nota**
>
> Enquanto chamar `peek()` não altera a posição do arquivo de `LZMAFile`, pode alterar a posição do objeto arquivo subjacente (por exemplo, se o `LZMAFile` foi construído passando um objeto arquivo para *nome do arquivo*).

mode

`'rb'` para leitura e `'wb'` para escrita.

Adicionado na versão 3.13.

name

O nome do arquivo lzma. Equivalente ao atributo `name` do *objeto arquivo* subjacente.

Adicionado na versão 3.13.

Alterado na versão 3.4: Adicionado suporte para os modos `"x"` e `"xb"`.

Alterado na versão 3.5: O método `read()` agora aceita um argumento de `None`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

13.6.2 Comprimindo e descomprimindo dados na memória

class lzma.**LZMACompressor**(*format=FORMAT_XZ*, *check=-1*, *preset=None*, *filters=None*)

Cria um objeto compressão, que pode ser usado para comprimir dados de forma incremental.

Para uma maneira mais conveniente de comprimir um único bloco de dados, consulte `compress()`.

O argumento *format* especifica qual formato de contêiner deve ser usado. Os valores possíveis são:

- **FORMAT_XZ: O formato de contêiner `.xz`.**
  Este é o formato padrão.
- **FORMAT_ALONE: O formato de contêiner legado `.lzma`.**
  Este formato é mais limitado que `.xz` – ele não oferece suporte a verificações de integridade ou filtros múltiplos.
- **FORMAT_RAW: Um fluxo de dados brutos, que não usa nenhum formato de contêiner.**
  Esse especificador de formato não oferece suporte a verificações de integridade e exige que você sempre especifique uma cadeia de filtros personalizada (para compressão e descompressão). Além disso, dados comprimidos dessa maneira não podem ser descomprimidos usando `FORMAT_AUTO` (veja `LZMADecompressor`).

O argumento *check* especifica o tipo de verificação de integridade a ser incluída nos dados comprimidos. Essa verificação é usada ao descomprimir, para garantir que os dados não foram corrompidos. Os valores possíveis são:

- `CHECK_NONE`: Sem verificação de integridade. Este é o padrão (e o único valor aceitável) para `FORMAT_ALONE` e `FORMAT_RAW`.
- `CHECK_CRC32`: Verificação de redundância cíclica de 32 bits.
- `CHECK_CRC64`: Verificação de redundância cíclica de 64 bits. Este é o padrão para `FORMAT_XZ`.
- `CHECK_SHA256`: Algoritmo de hash seguro de 256 bits.

Se a verificação especificada não for suportada, uma exceção `LZMAError` será levantada.

As configurações de compressão podem ser especificadas como um nível de compressão predefinido (com o argumento *preset*) ou em detalhes como uma cadeia de filtros personalizada (com o argumento *filters*).

O argumento *preset* (se fornecido) deve ser um inteiro entre `0` e `9` (inclusive), opcionalmente com OR com a constante `PRESET_EXTREME`. Se nem *preset* nem *filters* forem fornecidos, o comportamento padrão é usar `PRESET_DEFAULT` (nível predefinido `6`). Predefinições mais altas produzem uma saída menor, mas tornam o processo de compressão mais lento.

> **Nota**
>
> Além de consumir mais CPU, a compressão com predefinições mais altas também requer muito mais memória (e produz uma saída que precisa de mais memória para descompressão). Com a predefinição `9` por exemplo, a sobrecarga para um objeto `LZMACompressor` pode chegar a 800 MiB. Por esse motivo, geralmente é melhor ficar com a predefinição padrão.

O argumento *filters* (se fornecido) deve ser um especificador de cadeia de filtros. Veja *Especificando cadeias de filtros personalizadas* para detalhes.

compress(*data*)

Comprime *data* (um objeto *bytes*), retornando um objeto *bytes* contendo dados comprimidos para pelo menos parte da entrada. Alguns dos *data* podem ser armazenados internamente, para uso em chamadas posteriores para *compress()* e *flush()*. Os dados retornados devem ser concatenados com a saída de quaisquer chamadas anteriores para *compress()*.

flush()

Conclui o processo de compressão, retornando um objeto *bytes* contendo todos os dados armazenados nos buffers internos do compressor.

O compressor não pode ser usado após a chamada deste método.

class lzma.**LZMADecompressor**(*format=FORMAT_AUTO*, *memlimit=None*, *filters=None*)

Cria um objeto descompressor, que pode ser usado para descomprimir dados de forma incremental.

Para uma maneira mais conveniente de descomprimir um fluxo comprimido inteiro de uma só vez, consulte *decompress()*.

O argumento *format* especifica o formato do contêiner que deve ser usado. O padrão é FORMAT_AUTO, que pode descomprimir os arquivos .xz e .lzma. Outros valores possíveis são FORMAT_XZ, FORMAT_ALONE e FORMAT_RAW.

O argumento *memlimit* especifica um limite (em bytes) na quantidade de memória que o descompressor pode usar. Quando esse argumento é usado, a descompressão falhará com um *LZMAError* se não for possível descomprimir a entrada dentro do limite de memória fornecido.

O argumento *filters* especifica a cadeia de filtros que foi usada para criar o fluxo que está sendo descomprimido. Este argumento é necessário se *format* for FORMAT_RAW, mas não deve ser usado para outros formatos. Veja *Especificando cadeias de filtros personalizadas* para mais informações sobre cadeias de filtros.

> **Nota**
>
> Esta classe não manipula de forma transparente entradas contendo múltiplos fluxos comprimidos, diferentemente de *decompress()* e *LZMAFile*. Para descomprimir uma entrada multi-fluxo com *LZMADecompressor*, você deve criar um novo descompactador para cada fluxo.

decompress(*data*, *max_length=-1*)

Descomprime dados *data* (um *objeto bytes ou similar*), retornando dados não comprimidos como bytes. Alguns dos *data* podem ser armazenados em buffer internamente, para uso em chamadas posteriores para *decompress()*. Os dados retornados devem ser concatenados com a saída de qualquer chamada anterior para *decompress()*.

Se *max_length* for não negativo, retornará no máximo *max_length* bytes de dados descomprimidos. Se este limite for atingido e mais saída puder ser produzida, o atributo *needs_input* será definido como False. Neste caso, a próxima chamada para *decompress()* pode fornecer *data* como b'' para obter mais saída.

Se todos os dados de entrada foram descomprimidos e retornados (seja porque era menor que *max_length* bytes, ou porque *max_length* era negativo), o atributo *needs_input* será definido como True .

A tentativa de descomprimir os dados após o final do fluxo ser atingido gera um *EOFError*. Quaisquer dados encontrados após o final do fluxo são ignorados e salvos no atributo *unused_data*.

Alterado na versão 3.5: Adicionado o parâmetro *max_length*.

check

O ID da verificação de integridade usada pelo fluxo de entrada. Pode ser CHECK_UNKNOWN até que o suficiente da entrada tenha sido decodificado para determinar qual verificação de integridade ela usa.

eof

True se o marcador de fim de fluxo foi atingido.

unused_data

Dados encontrados após o término do fluxo comprimido.

Antes do final do fluxo ser alcançado, isso será `b""`.

needs_input

`False` se o método *decompress()* puder fornecer mais dados descomprimidos antes de exigir uma nova entrada não comprimida.

Adicionado na versão 3.5.

`lzma.`**compress**(*data*, *format=FORMAT_XZ*, *check=-1*, *preset=None*, *filters=None*)

Comprime *data* (um objeto *bytes*), retornando os dados comprimidos como um objeto *bytes*.

Veja *LZMACompressor* acima para uma descrição dos argumentos *format*, *check*, *preset* e *filters*.

`lzma.`**decompress**(*data*, *format=FORMAT_AUTO*, *memlimit=None*, *filters=None*)

Descomprime *data* (um objeto *bytes*), retornando os dados não comprimidos como um objeto *bytes*.

Se *data* forem a concatenação de vários fluxos comprimidos distintos, descomprime todos esses fluxos e retorna a concatenação dos resultados.

Veja *LZMADecompressor* acima para uma descrição dos argumentos *format*, *memlimit* e *filters*.

13.6.3 Diversos

`lzma.`**is_check_supported**(*check*)

Retorna `True` se a verificação de integridade fornecida for suportada neste sistema.

`CHECK_NONE` e `CHECK_CRC32` são sempre suportados. `CHECK_CRC64` e `CHECK_SHA256` podem não estar disponíveis se você estiver usando uma versão do **liblzma** que foi compilada com um conjunto de recursos limitado.

13.6.4 Especificando cadeias de filtros personalizadas

Um especificador de cadeia de filtros é uma sequência de dicionários, onde cada dicionário contém o ID e as opções para um único filtro. Cada dicionário deve conter a chave `"id"` e pode conter chaves adicionais para especificar opções dependentes do filtro. Os IDs de filtro válidos são os seguintes:

- Filtro Compression:
  - `FILTER_LZMA1` (para ser usado com `FORMAT_ALONE`)
  - `FILTER_LZMA2` (para ser utilizado com `FORMAT_XZ` and `FORMAT_RAW`)
- Filtro delta:
  - `FILTER_DELTA`
- Filtros Branch-Call-Jump (BCJ):
  - `FILTER_X86`
  - `FILTER_IA64`
  - `FILTER_ARM`
  - `FILTER_ARMTHUMB`
  - `FILTER_POWERPC`
  - `FILTER_SPARC`

Uma cadeia de filtros pode consistir em até 4 filtros e não pode estar vazia. O último filtro na cadeia deve ser um filtro de compressão e quaisquer outros filtros devem ser filtros delta ou BCJ.

Os filtros de compressão suportam as seguintes opções (especificadas como entradas adicionais no dicionário que representa o filtro):

- `preset`: Uma predefinição de compressão para usar como fonte de valores padrão para opções que não são especificadas explicitamente.
- `dict_size`: Tamanho do dicionário em bytes. Deve estar entre 4 KiB e 1,5 GiB (inclusive).
- `lc`: Número de bits de contexto literal.
- `lp`: Número de bits de posição literal. A soma `lc + lp` deve ser no máximo 4.
- `pb`: Número de bits de posição; deve ser no máximo 4.
- `mode`: `MODE_FAST` ou `MODE_NORMAL`.
- `nice_len`: O que deve ser considerado um "tamanho agradável" para uma correspondência. Deve ser 273 ou menos.
- `mf`: Qual localizador de correspondências usar – `MF_HC3`, `MF_HC4`, `MF_BT2`, `MF_BT3` ou `MF_BT4`.
- `depth`: Profundidade máxima de pesquisa usada pelo localizador de correspondências. 0 (padrão) significa selecionar automaticamente com base em outras opções de filtro.

O filtro delta armazena as diferenças entre bytes, produzindo uma entrada mais repetitiva para o compressor em certas circunstâncias. Ele suporta uma opção, `dist`. Isso indica a distância entre bytes a ser subtraída. O padrão é 1, ou seja, pega as diferenças entre bytes adjacentes.

Os filtros BCJ são destinados a serem aplicados ao código de máquina. Eles convertem ramificações relativas, chamadas e saltos no código para usar endereçamento absoluto, com o objetivo de aumentar a redundância que pode ser explorada pelo compressor. Esses filtros suportam uma opção, `start_offset`. Isso especifica o endereço que deve ser mapeado para o início dos dados de entrada. O padrão é 0.

13.6.5 Exemplos

Lendo em um arquivo comprimido:

```
import lzma
with lzma.open("file.xz") as f:
    file_content = f.read()
```

Criando um arquivo comprimido:

```
import lzma
data = b"Insira dados aqui"
with lzma.open("file.xz", "w") as f:
    f.write(data)
```

Comprimindo dados na memória:

```
import lzma
data_in = b"Insert Data Here"
data_out = lzma.compress(data_in)
```

Compressão incremental:

```
import lzma
lzc = lzma.LZMACompressor()
out1 = lzc.compress(b"Alguns dados\n")
out2 = lzc.compress(b"Outro pedaço de dados\n")
out3 = lzc.compress(b"Ainda mais dados\n")
out4 = lzc.flush()
# Concatena todos os resultados parciais:
result = b"".join([out1, out2, out3, out4])
```

Escrevendo dados comprimidos em um arquivo já aberto:

```
import lzma
with open("file.xz", "wb") as f:
    f.write(b"Este dados serão comprimidos\n")
    with lzma.open(f, "w") as lzf:
        lzf.write(b"Isso *vai* ser comprimidos\n")
    f.write(b"Não comprimidos\n")
```

Criando um arquivo comprimido usando uma cadeia de filtros personalizada:

```
import lzma
my_filters = [
    {"id": lzma.FILTER_DELTA, "dist": 5},
    {"id": lzma.FILTER_LZMA2, "preset": 7 | lzma.PRESET_EXTREME},
]
with lzma.open("file.xz", "w", filters=my_filters) as f:
    f.write(b"blá blá blá")
```

13.7 `zipfile` — Trabalha com arquivos ZIP

Código-fonte: Lib/zipfile/

O formato de arquivo ZIP é um padrão de compactação e arquivamento. Este módulo fornece ferramentas para criar, ler, escrever, adicionar, e listar um arquivo ZIP. Qualquer uso avançado deste módulo vai exigir um entendimento do formato, como definido nas Notas de Aplicação do PKZIP.

Esse módulo atualmente não suporta arquivos ZIP multi-disco. Ele pode manipular arquivos ZIP que usam as extensões ZIP64 (ou seja arquivos ZIP com tamanho maior do que 4 Gb). Ele suporta descriptografia de arquivos criptografados dentro do ZIP, mas atualmente não pode criar um arquivo criptografado. A descriptografia é extremamente lenta pois é implementada em Python nativo ao invés de C.

Este módulo define os seguintes itens:

exception zipfile.**BadZipFile**

Este erro é levantado para arquivos ZIP corrompidos.

Adicionado na versão 3.2.

exception zipfile.**BadZipfile**

Alias para *BadZipFile*, para compatibilidade com versões mais antigas de Python.

Descontinuado desde a versão 3.2.

exception zipfile.**LargeZipFile**

Este erro é levantado quando um arquivo ZIP precisa da funcionalidade ZIP64 que não está habilitada.

class zipfile.**ZipFile**

A classe para ler e escrever arquivos ZIP. Veja a seção *Objetos ZipFile* para detalhes do construtor.

class zipfile.**Path**

Classe que implementa um subconjunto da interface fornecida por *pathlib.Path*, incluindo a interface completa *importlib.resources.abc.Traversable*.

Adicionado na versão 3.8.

class zipfile.**PyZipFile**

Classe para criar arquivos ZIP contendo bibliotecas Python.

class zipfile.**ZipInfo**(*filename='NoName'*, *date_time=(1980, 1, 1, 0, 0, 0)*)

Classe usada para representar informação sobre um membro de um archive. Instâncias desta classe são retornadas pelos métodos *getinfo()* e *infolist()* de objetos da classe *ZipFile*. A maioria dos usuários do módulo *zipfile* não vai precisar criar, mas apenas usar objetos criados pelo módulo. *filename* deveria ser o caminho completo do membro do arquivo, e *date_time* deveria ser uma tupla contendo seis campos que descrevem o momento da última modificação no arquivo; os campos são descritos na seção *Objetos ZipInfo*.

Alterado na versão 3.13: Um atributo público `compress_level` foi adicionado para expor o `_compresslevel` anteriormente protegido. O nome protegido mais antigo continua a funcionar como uma propriedade para compatibilidade com versões anteriores.

_for_archive(*archive*)

Resolve os atributos date_time, compression e external para padrões adequados, conforme usado por *ZipFile.writestr()*.

Retorna self para encadeamento.

Adicionado na versão 3.14.

zipfile.**is_zipfile**(*filename*)

Retorna `True` se *filename* é um arquivo ZIP válido baseado no seu "magic number", caso contrário retorna `False`. *filename* pode ser um arquivo ou um objeto arquivo ou similar também.

Alterado na versão 3.1: Suporte para arquivo e objetos arquivo ou similares.

zipfile.**ZIP_STORED**

Código numérico para um membro de um arquivo descompactado

zipfile.**ZIP_DEFLATED**

Código numérico para o método de compactação usual. Requer o módulo *zlib* .

zipfile.**ZIP_BZIP2**

Código numérico para o método de compactação BZIP2. Requer o módulo *bz2*.

Adicionado na versão 3.3.

zipfile.**ZIP_LZMA**

Código numérico para o método de compactação LZMA. Requer o módulo *lzma*.

Adicionado na versão 3.3.

zipfile.**ZIP_ZSTANDARD**

Código numérico para a compactação com Zstandard. Requer o módulo *compression.zstd*.

> **Nota**
>
> No APPNOTE 6.3.7, o ID do método `20` foi atribuído à compactação Zstandard. No APPNOTE 6.3.8, esse ID foi alterado para o ID do método `93` para evitar conflitos, com o ID do método `20` sendo descontinuado. Para compatibilidade, o módulo `zipfile` lê ambos os IDs de método, mas gravará apenas dados com o ID do método `93`.

Adicionado na versão 3.14.

> **Nota**
>
> A especificação do formato ZIP incluiu suporte para compactação bzip2 desde 2001, para compactação LZMA desde 2006 e para compactação Zstandard desde 2020. Porém, algumas ferramentas (incluindo versões mais antigas de Python) não suportam esses métodos de compactação, e podem recusar processar o arquivo ZIP como um todo, ou falhar em extrair arquivos individuais.

Ver também

Notas da Aplicação do PKZIP
Documentação do formato de arquivo ZIP feita por Phil Katz, criador do formato e dos algoritmos usados.

Site do Info-ZIP
Informações sobre o programas de arquivamento e desenvolvimento de bibliotecas do projeto Info-ZIP.

13.7.1 Objetos ZipFile

class zipfile.**ZipFile**(*file*, *mode='r'*, *compression=ZIP_STORED*, *allowZip64=True*, *compresslevel=None*, ***, *strict_timestamps=True*, *metadata_encoding=None*)

Abre um arquivo ZIP, onde *file* pode ser um caminho para um arquivo (uma string), um objeto arquivo ou similar, ou um *objeto caminho ou similar*.

O parâmetro *mode* deve ser `'r'` para ler um arquivo existente, `'w'` para truncar e gravar um novo arquivo, `'a'` para adicionar a um arquivo existente, ou `'x'` exclusivamente para criar e gravar um novo arquivo. Se o *mode* é `'x'` e *file* se refere a um arquivo existente, um *FileExistsError* vai ser levantado. Se o *mode* é `'a'` e *file* se refere a um arquivo ZIP existente, então arquivos adicionais são adicionados ao mesmo. Se *file* não se refere a um arquivo ZIP, então um novo arquivo ZIP é adicionado ao arquivo. Isso diz respeito a adicionar um arquivo ZIP a um outro arquivo (como por exemplo `python.exe`). Se o *mode* é `'a'` e o arquivo não existe, ele será criado. Se o *mode* é `'r'` ou `'a'`, o arquivo deve ser percorrível.

compression é o método de compactação ZIP para usar ao escrever o arquivo, e deve ser *ZIP_STORED*, *ZIP_DEFLATED*, *ZIP_BZIP2*, *ZIP_LZMA* e *ZIP_ZSTANDARD*; valores desconhecidos devem causar *NotImplementedError* ser levantada. Se *ZIP_DEFLATED*, *ZIP_BZIP2*, *ZIP_LZMA* ou *ZIP_ZSTANDARD* for especificado mas o módulo correspondente (*zlib*, *bz2*, *lzma* e *compression.zstd*) não estiver disponível, é levantada uma exceção *RuntimeError*. O valor padrão é *ZIP_STORED*.

Se *allowZip64* é `True` (valor padrão), então zipfile vai criar arquivos ZIP que usem as extensões ZIP64 quando o arquivo ZIP é maior do que 4 GiB. Se é `false`, *zipfile* levanta uma exceção quando o arquivo ZIP precisaria das extensões ZIP64.

O parâmetro *compresslevel* controla o nível de compactação para usar ao gravar no arquivo ZIP. Quando usado *ZIP_STORED* ou *ZIP_LZMA* não tem efeito. Quando usado *ZIP_DEFLATED* inteiros de `0` a `9` são aceitos (veja *zlib* para mais informações). Quando usado *ZIP_BZIP2* inteiros de `1` a `9` são aceitos (veja *bz2* para mais informações). Ao usar *ZIP_ZSTANDARD*, os inteiros `-131072` a `22` são comumente aceitos (consulte *CompressionParameter.compression_level* para obter mais informações sobre como recuperar valores válidos e seus significados).

O argumento *strict_timestamps*, quando definido como `False`, permite compactar arquivos anteriores a 1980-01-01 com o custo de definir o carimbo de data/hora para 1980-01-01. Comportamento semelhante ocorre com arquivos mais recentes que 2107-12-31, o carimbo de data/hora também é definido como o limite.

Quando o modo é `'r'`, *metadata_encoding* pode ser definido como o nome de um codec, que será usado para decodificar metadados, como os nomes dos membros e comentários ZIP.

Se o arquivo é criado com modo `'w'`, `'x'` ou `'a'` e então `closed()` sem adicionar nada ao arquivo, a estrutura própria para um arquivo vazio será escrita no arquivo.

ZipFile também é um gerenciador de contexto e portanto suporta a instrução `with`. Neste exemplo, *myzip* é fechado ao final da execução da instrução `with` – mesmo que ocorra uma exceção:

```
with ZipFile('spam.zip', 'w') as myzip:
    myzip.write('eggs.txt')
```

Nota

metadata_encoding é uma configuração em toda a instância para o ZipFile. Atualmente, não é possível definir isso em uma base por membro.

Este atributo é uma solução alternativa para implementações legadas que produzem arquivos com nomes na codificação da localidade atual ou página de código (principalmente no Windows). De acordo com o padrão .ZIP, a codificação dos metadados pode ser especificada como página de código IBM (padrão) ou UTF-8 por meio de um sinalizador no cabeçalho do arquivo. Esse sinalizador tem precedência sobre *metadata_encoding*, que é uma extensão específica do Python.

Alterado na versão 3.2: Adicionado o uso de `ZipFile` como um gerenciador de contexto.

Alterado na versão 3.3: Adicionado suporte para compactação `bzip2` e `lzma`.

Alterado na versão 3.4: Extensões ZIP64 são habilitadas por padrão.

Alterado na versão 3.5: Adicionado suporte para escrever em streams não percorríveis. Adicionado suporte ao modo `'x'`.

Alterado na versão 3.6: Anteriormente, um simples `RuntimeError` era levantado para valores de compactação desconhecidos.

Alterado na versão 3.6.2: O parâmetro *file* aceita um *objeto caminho ou similar*.

Alterado na versão 3.7: Adicionado o parâmetro *compresslevel*.

Alterado na versão 3.8: O parâmetro somente-nomeado *strict_timestamps*.

Alterado na versão 3.11: Adicionado suporte para especificar a codificação do nome do membro ao ler metadados no diretório e cabeçalhos de arquivo do arquivo zip.

`ZipFile.`**`close`**`()`

Fecha o arquivo. Você deve chamar `close()` antes de sair do seu programa ou registros essenciais não serão gravados.

`ZipFile.`**`getinfo`**`(`*name*`)`

Retorna um objeto `ZipInfo` com informações sobre o *name* do membro do arquivo. Chamar `getinfo()` para um nome não encontrado no arquivo levanta um `KeyError`.

`ZipFile.`**`infolist`**`()`

Retorna uma lista contendo um objeto `ZipInfo` para cada membro do arquivo. Os objetos estão na mesma ordem das entradas no arquivo ZIP em disco se um arquivo existente foi aberto.

`ZipFile.`**`namelist`**`()`

Retorna uma lista de membros do arquivo por nome.

`ZipFile.`**`open`**`(`*name*, *mode='r'*, *pwd=None*, ***, *force_zip64=False*`)`

Acessa um membro do arquivo como um objeto binário arquivo ou similar. *name* pode ser o nome de um arquivo membro ou um objeto `ZipInfo`. O parâmetro *mode*, se informado, deve ser `'r'` (valor padrão) or `'w'`. *pwd* é a senha usada para descriptografar arquivos ZIP criptografados como um objeto `bytes`.

`open()` também é um gerenciador de contexto e, portanto, suporta a instrução `with`:

```
with ZipFile('spam.zip') as myzip:
    with myzip.open('eggs.txt') as myfile:
        print(myfile.read())
```

Com *mode* `'r'` o objeto arquivo ou similar (`ZipExtFile`) é somente leitura e fornece os seguintes métodos: `read()`, `readline()`, `readlines()`, `seek()`, `tell()`, `__iter__()`, `__next__()`. Esses objetos podem operar independentemente do ZipFile.

Com `mode='w'`, é retornado um manipulador de arquivo, que suporta o método `write()`. Quando um manipulador de arquivo modificável é aberto, tentativas de ler ou gravar outros arquivos no arquivo ZIP levanta um `ValueError`.

Em ambos os casos o objeto arquivo ou similar também possui atributos `name`, que é equivalente ao nome de um arquivo dentro do arquivo, e `mode`, que é `'rb'` ou `'wb'` dependendo do modo de entrada.

Ao gravar um arquivo, se o tamanho do arquivo não é conhecido mas pode exceder 2 GiB, passe `force_zip64=True` para assegurar que o formato do header é capaz de suportar arquivos grandes. Se o tamanho do arquivo é conhecido, construa um objeto *ZipInfo* com *file_size* informado, então use-o como parâmetro *name*.

> **Nota**
>
> Os métodos *open()*, *read()* e *extract()* podem receber um nome de arquivo ou um objeto *ZipInfo*. Você vai gostar disso quando tentar ler um arquivo ZIP que contém membros com nomes duplicados.

Alterado na versão 3.6: Removido suporte ao `mode='U'`. Uso de *io.TextIOWrapper* para leitura de arquivos texto compactados em modo de *novas linhas universais*.

Alterado na versão 3.6: *ZipFile.open()* agora pode ser usado para escrever arquivos no aquivo compactado com a opção `mode='w'`.

Alterado na versão 3.6: Chama *open()* em um ZipFile fechado levanta um *ValueError*. Anteriormente, um *RuntimeError* era levantado.

Alterado na versão 3.13: Adicionados atributos `name` e `mode` para o objeto arquivo ou similar gravável. O valor do atributo `mode` para o objeto arquivo ou similar legível foi alterado de `'r'` para `'rb'`.

`ZipFile.`**`extract`**(*member*, *path=None*, *pwd=None*)

Extrai um membro do arquivo para o diretório atual; *member* deve ser o nome completo ou um objeto *ZipInfo*. A informação do arquivo é extraída com maior precisão possível. *path* especifica um outro diretório em que deve ser gravado. *member* pode ser um nome de arquivo ou um objeto *ZipInfo*. *pwd* é a senha usada para criptografar arquivos como um objeto *bytes*.

Retorna o caminho normalizado criado (um diretório ou novo arquivo).

> **Nota**
>
> Se um nome de arquivo membro é um caminho absoluto, o drive/UNC e (contra)barras no início serão removidos, por exemplo: `///foo/bar` se torna `foo/bar` no Unix, e `C:\foo\bar` vira `foo\bar` no Windows. E todos os componentes `".."` no nome de um arquivo membro serão removidos, por exemplo: `../../foo../../ba..r` vira `foo../ba..r`. No Windows caracteres ilegais (`:`, `<`, `>`, `|`, `"`, `?`, and `*`) são substituídos por underscore (_).

Alterado na versão 3.6: Chama *extract()* em um ZipFile fechado levanto um *ValueError*. Anteriormente, um *RuntimeError* era levantado.

Alterado na versão 3.6.2: O parâmetro *path* aceita um *objeto caminho ou similar*.

`ZipFile.`**`extractall`**(*path=None*, *members=None*, *pwd=None*)

Extrai todos os membros de um arquivo para o diretório atual. *path* especifica um diretório diferente para gravar os arquivos extraídos. *members* é opcional e deve ser um sub-conjunto da lista retornada por *namelist()*. *pwd* é uma senha usada para criptografar arquivos como um objeto *bytes*.

> **Aviso**
>
> Nunca extrai arquivos de fontes não confiáveis sem inspeção prévia. É possível que os arquivos sejam criados fora do *path*, por exemplo membros que tem nomes absolutos de arquivos começando com `"/"` ou nomes com dois pontos `".."`. Este módulo tenta prevenir isto. Veja nota em *extract()*.

Alterado na versão 3.6: Chama *extractall()* em um ZipFile fechado levanta um *ValueError*. Anteriormente, um *RuntimeError* era levantado.

Alterado na versão 3.6.2: O parâmetro *path* aceita um *objeto caminho ou similar*.

ZipFile.**printdir**()

Imprime a tabela de conteúdos de um arquivo para `sys.stdout`.

ZipFile.**setpassword**(*pwd*)

Define *pwd* (um objeto *bytes*) como senha padrão para extrair arquivos criptografados.

ZipFile.**read**(*name*, *pwd=None*)

Retorna os bytes do arquivo *name* no arquivo compactado. *name* é o nome do arquivo no arquivo compactado, ou um objeto *ZipInfo*. O arquivo compactado deve estar aberto para leitura ou acréscimo. *pwd* é a senha usada para arquivos criptografados como um objeto *bytes* e, se especificada, substitui a senha padrão configurada com *setpassword()*. Chamar *read()* em um ZipFile que use um método de compactação diferente de *ZIP_STORED*, *ZIP_DEFLATED*, *ZIP_BZIP2*, *ZIP_LZMA* e *ZIP_ZSTANDARD* levanta um *NotImplementedError*. Um erro também é levantado se o módulo de compactação correspondente não está disponível.

Alterado na versão 3.6: Chama *read()* em um ZipFile fechado levanta um *ValueError*. Anteriormente, um *RuntimeError* era levantado.

ZipFile.**testzip**()

Lê todos os arquivos no arquivo compactado e verifica seus CRC's e cabeçalhos de arquivo. Retorna o nome do primeiro arquivo corrompido, or então retorna `None`.

Alterado na versão 3.6: Chama *testzip()* em um ZipFile fechado levanta um *ValueError*. Anteriormente, um *RuntimeError* era levantado.

ZipFile.**write**(*filename*, *arcname=None*, *compress_type=None*, *compresslevel=None*)

Grava o arquivo chamado *filename* no arquivo compactado, dando ao arquivo compactado o nome *arcname* (por padrão, este é o mesmo de *filename*, mas sem a letra do drive e com separadores removidos do início do nome). Se informado, *compress_type* sobrescreve o valor dado ao parâmetro *compression* do construtor para a nova entrada. Da mesma forma, *compresslevel* vai sobrescrever o construtor se informado. O arquivo compactado deve ser aberto em modo `'w'`, `'x'` ou `'a'`.

Nota

O padrão de arquivo ZIP historicamente não especificava uma codificação de metadados, mas recomendava fortemente o CP437 (a codificação original do PC da IBM) para interoperabilidade. Versões recentes permitem o uso de (somente) UTF-8. Neste módulo, o UTF-8 será automaticamente usado para escrever os nomes dos membros se eles contiverem caracteres não ASCII. Não é possível escrever nomes de membros em qualquer codificação que não seja ASCII ou UTF-8.

Nota

Nomes de arquivo compactado devem ser relativos a raiz do mesmo, isto é, não devem começar com um separador de caminho.

Nota

Se `arcname` (ou `filename`, se `arcname` não for informado) contém um byte nulo, o nome do arquivo no arquivo compactado será truncado no byte nulo.

Nota

Uma barra inicial no nome do arquivo pode fazer com que o arquivo seja impossível de abrir em alguns programas zip em sistemas Windows.

Alterado na versão 3.6: Chama *write()* em um ZipFile criado com modo `'r'` ou em um ZipFile fechado levanta um *ValueError*. Anteriormente, um *RuntimeError* era levantado

ZipFile.**writestr**(*zinfo_or_arcname*, *data*, *compress_type=None*, *compresslevel=None*)

Grava um arquivo no arquivo compactado. O conteúdo é *data*, que pode ser uma instância de *str* ou de *bytes*; Se é uma *str*, ela é encodada como UTF-8 primeiro. *zinfo_or_arcname* é o nome que será dado ao arquivo no arquivo compactado, ou uma instância de *ZipInfo*. Se é uma instância, pelo menos o nome do arquivo, a data, e a hora devem ser informados. Se é um nome, a data e hora recebem a data e hora atual. O arquivo compactado deve ser aberto em modo `'w'`, `'x'` ou `'a'`.

Se informado, *compress_type* sobrescreve o valor do parâmetro *compression* do construtor para a nova entrada, ou no *zinfo_or_arcname* (se é uma instância de *ZipInfo*). Da mesma forma, *compresslevel* vai sobrescrever o construtor se informado.

> **Nota**
>
> Quando é passada uma instância de *ZipInfo* ou o parâmetro *zinfo_or_arcname*, o método de compactação usado será aquele especificado no *compress_type* da instância de *ZipInfo*. Por padrão, o construtor da classe *ZipInfo* seta este membro para *ZIP_STORED*.

Alterado na versão 3.2: O argumento *compress_type*.

Alterado na versão 3.6: Chama *writestr()* em um ZipFile criado com modo `'r'` ou em um ZipFile fechado levanta um *ValueError*. Anteriormente, um *RuntimeError* era levantado.

ZipFile.**mkdir**(*zinfo_or_directory*, *mode=511*)

Cria um diretório dentro do arquivo. Se *zinfo_or_directory* for uma string, um diretório é criado dentro do arquivo com o modo especificado no argumento *mode*. No entanto, se *zinfo_or_directory* for uma instância *ZipInfo*, o argumento *mode* é ignorado.

O arquivo deve ser aberto com o modo `'w'`, `'x'` ou `'a'`.

Adicionado na versão 3.11.

Os seguintes atributos de dados também estão disponíveis:

ZipFile.**filename**

Nome do arquivo ZIP.

ZipFile.**debug**

O nível de saída de debug para usar. Pode ser setado de `0` (valor padrão, sem nenhuma saída) a `3` (com mais saída). A informação de debug é escrita em `sys.stdout`.

ZipFile.**comment**

O comentário associado ao arquivo ZIP como um objeto *bytes*. Se atribuir um comentário a uma instância *ZipFile* criada com o modo `'w'`, `'x'` ou `'a'`, não deve ser maior que 65535 bytes. Comentários mais longos do que isso serão truncados.

13.7.2 Objetos Path

class zipfile.**Path**(*root*, *at=''*)

Construir um objeto Path a partir de um arquivo zip `root` (que pode ser uma instância *ZipFile* ou `file` adequado para passar para o construtor *ZipFile*).

`at` especifica a localização deste caminho dentro do arquivo zip, por exemplo, "dir/arquivo.txt", "dir/" ou "". O padrão é a string vazia, indicando a raiz.

> **Nota**

> A classe *Path* não sanitiza nomes de arquivos dentro do arquivo ZIP. Ao contrário dos métodos *ZipFile.extract()* e *ZipFile.extractall()*, é responsabilidade do chamador validar ou sanitizar nomes de arquivos para evitar vulnerabilidades de travessia de caminho (por exemplo, nomes de arquivos contendo ".." ou caminhos absolutos). Ao lidar com arquivos não confiáveis, considere resolver nomes de arquivos usando *os.path.abspath()* e verificar o diretório de destino com *os.path.commonpath()*.

Objetos Path expõem os seguintes recursos de objetos *pathlib.Path*:

Objetos Path podem ser percorridos usando o operador `/` ou `joinpath`.

`Path.`**`name`**

O componente final do caminho.

`Path.`**`open`**(*mode='r'*, ***, *pwd*, ****)

Invoca *ZipFile.open()* no caminho atual. Permite a abertura para leitura ou escrita, texto ou binário através dos modos suportados: "r", "w", "rb", "wb". Argumentos posicionais e argumentos nomeados são passados para *io.TextIOWrapper* quando abertos como texto e ignorados caso contrário. `pwd` é o parâmetro `pwd` para *ZipFile.open()*.

Alterado na versão 3.9: Adicionado suporte para modos de texto e binários para aberto. O modo padrão agora é texto.

Alterado na versão 3.11.2: O parâmetro `encoding` pode ser fornecido como um argumento posicional sem causar um *TypeError*. Como poderia em 3.9. O código que precisa ser compatível com as versões 3.10 e 3.11 não corrigidas deve passar todos os argumentos de *io.TextIOWrapper*, incluindo `encoding`, como palavras reservadas.

`Path.`**`iterdir`**()

Enumera os filhos do diretório atual.

`Path.`**`is_dir`**()

Retorna `True` se o contexto atual fizer referência a um diretório.

`Path.`**`is_file`**()

Retorna `True` se o contexto atual fizer referência a um arquivo.

`Path.`**`is_symlink`**()

Retorna `True` se o contexto atual fizer referência a um link simbólico.

Adicionado na versão 3.12.

Alterado na versão 3.13: Anteriormente, `is_symlink` retornava `False` incondicionalmente.

`Path.`**`exists`**()

Retorna `True` se o contexto atual fizer referência a um arquivo ou diretório no arquivo zip.

`Path.`**`suffix`**

A última parte separada por pontos do componente final, se houver. Isso é comumente chamado de extensão de arquivo.

Adicionado na versão 3.11: Adicionada a propriedade *Path.suffix*.

`Path.`**`stem`**

O componente final do caminho, sem seu sufixo.

Adicionado na versão 3.11: Adicionada a propriedade *Path.stem*.

`Path.`**`suffixes`**

Uma lista dos sufixos do caminho, comumente chamados de extensões de arquivo.

Adicionado na versão 3.11: Adicionada a propriedade *Path.suffixes*.

Path.**read_text**(*, **)

Leia o arquivo atual como texto Unicode. Argumentos posicionais e argumentos nomeados são passados para *io.TextIOWrapper* (exceto `buffer`, que está implícito no contexto).

Alterado na versão 3.11.2: O parâmetro `encoding` pode ser fornecido como um argumento posicional sem causar um *TypeError*. Como poderia em 3.9. O código que precisa ser compatível com as versões 3.10 e 3.11 não corrigidas deve passar todos os argumentos de *io.TextIOWrapper*, incluindo `encoding`, como palavras reservadas.

Path.**read_bytes**()

Lê o arquivo atual como bytes.

Path.**joinpath**(*other*)

Retorna um novo objeto Path com cada um dos *outros* argumentos unidos. Os seguintes são equivalentes:

```
>>> Path(...).joinpath('child').joinpath('grandchild')
>>> Path(...).joinpath('child', 'grandchild')
>>> Path(...) / 'child' / 'grandchild'
```

Alterado na versão 3.10: Antes de 3.10, `joinpath` não estava documentado e aceitava exatamente um parâmetro.

O projeto zipp fornece backports da funcionalidade mais recente de objeto caminho para versões mais antigas do Pythons. Use `zipp.Path` internamente de `zipfile.Path` para acesso antecipado às alterações.

13.7.3 Objetos PyZipFile

O construtor *PyZipFile* usa os mesmos parâmetros que o construtor *ZipFile*, e um parâmetro adicional, *otimize*.

class zipfile.**PyZipFile**(*file*, *mode='r'*, *compression=ZIP_STORED*, *allowZip64=True*, *optimize=-1*)

Alterado na versão 3.2: Adicionado o parâmetro *optimize*.

Alterado na versão 3.4: Extensões ZIP64 são habilitadas por padrão.

As instâncias têm um método além daqueles dos objetos *ZipFile*:

writepy(*pathname*, *basename=''*, *filterfunc=None*)

Pesquisa por arquivos `*.py` e adiciona o arquivo correspondente ao arquivo.

Se o parâmetro *optimize* para *PyZipFile* não foi fornecido ou `-1`, o arquivo correspondente é um arquivo `*.pyc`, compilando se necessário.

Se o parâmetro *Optimize* para *PyZipFile* era `0`, `1` ou `2`, apenas arquivos com esse nível de otimização (ver *compile()*) são adicionados ao o arquivo, compilando se necessário.

Se *pathname* for um arquivo, o nome do arquivo deverá terminar com `.py`, e apenas o arquivo (`*.pyc` correspondente) será adicionado no nível superior (sem informações do caminho). Se *pathname* for um arquivo que não termine com `.py`, um *RuntimeError* será levantado. Se for um diretório, e o diretório não for um diretório de pacotes, todos os arquivos `*.pyc` serão adicionados no nível superior. Se o diretório for um diretório de pacotes, todos `*.pyc` serão adicionados sob o nome do pacote como um caminho de arquivo e, se algum subdiretório for um diretório de pacotes, todos serão adicionados recursivamente na ordem de classificação.

basename destina-se apenas a uso interno.

filterfunc, se fornecido, deve ser uma função que recebe um único argumento de string. Cada caminho será passado (incluindo cada caminho de arquivo completo individual) antes de ser adicionado ao arquivo. Se *filterfunc* retornar um valor falso, o caminho não será adicionado e, se for um diretório, seu conteúdo será ignorado. Por exemplo, se nossos arquivos de teste estão todos nos diretórios `test` ou começam com a string `test_`, podemos usar um *filterfunc* para excluí-los:

```
>>> zf = PyZipFile('myprog.zip')
>>> def notests(s):
...     fn = os.path.basename(s)
...     return (not (fn == 'test' or fn.startswith('test_')))
...
>>> zf.writepy('myprog', filterfunc=notests)
```

O método *writepy()* faz arquivos com nomes de arquivo como este:

```
string.pyc                  # Nome de nível superior
test/__init__.pyc           # Diretório do pacote
test/testall.pyc            # Módulo test.testall
test/bogus/__init__.pyc     # Diretório do subpacote
test/bogus/myfile.pyc       # Submódulo test.bogus.myfile
```

Alterado na versão 3.4: Adicionado o parâmetro *filterfunc*.

Alterado na versão 3.6.2: O parâmetro *pathname* aceita um *objeto caminho ou similar*.

Alterado na versão 3.7: A recursão classifica as entradas de diretório.

13.7.4 Objetos ZipInfo

Instâncias da classe *ZipInfo* são retornadas pelos métodos *getinfo()* e *infolist()* dos objetos *ZipFile*. Cada objeto armazena informações sobre um único membro do arquivo ZIP.

Existe um método de classe para fazer uma instância *ZipInfo* para um arquivo de sistema de arquivos:

classmethod ZipInfo.**from_file**(*filename*, *arcname=None*, *, *strict_timestamps=True*)

Constrói uma instância *ZipInfo* para um arquivo no sistema de arquivos, em preparação para adicioná-lo a um arquivo zip.

filename deve ser o caminho para um arquivo ou diretório no sistema de arquivos.

Se *arcname* for especificado, ele será usado como o nome dentro do arquivo. Se *arcname* não for especificado, o nome será igual a *filename*, mas com qualquer letra de unidade e separadores de caminho removidos.

O argumento *strict_timestamps*, quando definido como `False`, permite compactar arquivos anteriores a 1980--01-01 com o custo de definir o carimbo de data/hora para 1980-01-01. Comportamento semelhante ocorre com arquivos mais recentes que 2107-12-31, o carimbo de data/hora também é definido como o limite.

Adicionado na versão 3.6.

Alterado na versão 3.6.2: O parâmetro *filename* aceita um *objeto caminho ou similar*.

Alterado na versão 3.8: Adicionado o parâmetro somente-nomeado *strict_timestamps*.

As instâncias têm os seguintes métodos e atributos:

ZipInfo.**is_dir**()

Retorna `True` se este membro do arquivo for um diretório.

Isso usa o nome da entrada: os diretórios devem sempre terminar com `/`.

Adicionado na versão 3.6.

ZipInfo.**filename**

Nome do arquivo no pacote.

ZipInfo.**date_time**

A hora e a data da última modificação no membro do arquivo. Esta é uma tupla de seis valores que representam os campos "last [modified] file time" (última hora [de modificado] do arquivo) e "last [modified] file date" (última data [de modificação] do arquivo) do diretório central do arquivo ZIP.

A tupla contém:

Índice	Valor
0	Ano (>= 1980)
1	Mês (iniciado em 1)
2	Dia do mês (iniciado em 1)
3	Horas (iniciado em 0)
4	Minutos (base zero)
5	Segundos (iniciado em 0)

Nota

O formato ZIP oferece suporte a vários campos de registro de data e hora em diferentes locais (diretório central, campos extras para sistemas NTFS/UNIX, etc.). Este atributo retorna especificamente o registro de data e hora do diretório central. O formato de registro de data e hora do diretório central em arquivos ZIP não oferece suporte a registros de data e hora anteriores a 1980. Embora alguns formatos de campos extras (como registros de data e hora do UNIX) possam representar datas anteriores, este atributo retorna apenas o registro de data e hora do diretório central.

O registro de data e hora do diretório central é interpretado como representando a hora local, em vez da hora UTC, para corresponder ao comportamento de outras ferramentas zip.

`ZipInfo.`**`compress_type`**

Tipo de compressão do membro do pacote.

`ZipInfo.`**`comment`**

Comentário para o membro individual do pacote como um objeto *`bytes`*.

`ZipInfo.`**`extra`**

Dados do campo de expansão. O PKZIP Application Note contém alguns comentários sobre a estrutura interna dos dados contidos neste objeto *`bytes`*.

`ZipInfo.`**`create_system`**

O sistema que criou o pacote ZIP.

`ZipInfo.`**`create_version`**

A versão do PKZIP que criou o pacote ZIP.

`ZipInfo.`**`extract_version`**

A versão do PKZIP necessária para extrair o pacote.

`ZipInfo.`**`reserved`**

Deve ser zero

`ZipInfo.`**`flag_bits`**

Bits de sinalizador do ZIP.

`ZipInfo.`**`volume`**

Número de volume do cabeçalho do arquivo.

`ZipInfo.`**`internal_attr`**

Atributos internos.

`ZipInfo.`**`external_attr`**

Atributos de arquivo externo.

`ZipInfo.`**`header_offset`**

Deslocamento de byte para o cabeçalho do arquivo.

ZipInfo.**CRC**
: CRC-32 do arquivo não comprimido.

ZipInfo.**compress_size**
: Tamanho dos dados comprimidos.

ZipInfo.**file_size**
: Tamanho do arquivo não comprimido.

13.7.5 Interface de Linha de Comando

O módulo *zipfile* fornece uma interface de linha de comando simples para interagir com arquivos ZIP.

Se você deseja criar um novo arquivo ZIP, especifique seu nome após a opção *-c* e, em seguida, liste os nomes dos arquivos que devem ser incluídos:

```
$ python -m zipfile -c monty.zip spam.txt eggs.txt
```

Passar um diretório também é aceitável:

```
$ python -m zipfile -c monty.zip life-of-brian_1979/
```

Se você deseja extrair um arquivo ZIP para o diretório especificado, use a opção *-e*:

```
$ python -m zipfile -e monty.zip target-dir/
```

Para obter uma lista dos arquivos em um arquivo ZIP, use a opção *-l*:

```
$ python -m zipfile -l monty.zip
```

Opções de linha de comando

-l <zipfile>

--list <zipfile>
: Lista arquivos em um arquivo zip.

-c <zipfile> <source1> ... <sourceN>

--create <zipfile> <source1> ... <sourceN>
: Cria um arquivo zip a partir dos arquivos fonte.

-e <zipfile> <output_dir>

--extract <zipfile> <output_dir>
: Extrai um arquivo zip para um diretório de destino.

-t <zipfile>

--test <zipfile>
: Testa se o arquivo zip é válido ou não.

--metadata-encoding <encoding>
: Especifica a codificação dos nomes dos membros para *-l*, *-e* e *-t*.

  Adicionado na versão 3.11.

13.7.6 Armadilhas de descompressão

A extração no módulo zipfile pode falhar devido a algumas armadilhas listadas abaixo.

Do próprio arquivo

A descompactação pode falhar devido a senha / soma de verificação CRC / formato ZIP incorretos ou método de compactação / descriptografia não compatível.

Limitações do sistema de arquivos

Exceder as limitações em sistemas de arquivos diferentes pode causar falha na descompactação. Como caracteres permitidos nas entradas do diretório, comprimento do nome do arquivo, comprimento do caminho, tamanho de um único arquivo e número de arquivos, etc.

Limitações de recursos

A falta de memória ou volume de disco levaria a uma falha de descompactação. Por exemplo, bombas de descompressão (também conhecidas como ZIP bomb) aplicam-se à biblioteca de arquivos zip que podem causar o esgotamento do volume do disco.

Interrupção

A interrupção durante a descompressão, como pressionar Control-C ou interromper o processo de descompressão pode resultar na descompressão incompleta do arquivo.

Comportamentos padrão da extração

Não saber os comportamentos de extração padrão pode causar resultados de descompressão inesperados. Por exemplo, ao extrair o mesmo arquivo duas vezes, ele sobrescreve os arquivos sem perguntar.

13.8 `tarfile` — Ler e gravar arquivos do tipo tar

Código-fonte: Lib/tarfile.py

O módulo `tarfile` possibilita a leitura e gravação de arquivos tar, incluindo aqueles que utilizam compactação gzip, bz2 e lzma. Use o módulo `zipfile` para ler ou gravar arquivos `.zip` ou as funções de nível mais alto em *shutil*.

Alguns fatos e números:

- lê e grava arquivos compactados com `gzip`, `bz2`, `compression.zstd` e `lzma` se os respectivos módulos estiverem disponíveis.
- suporte de leitura/gravação para o formato POSIX.1-1988 (ustar).
- suporte de leitura/gravação para o formato GNU tar, incluindo extensões *longname* e *longlink*, suporte somente leitura para todas as variantes da extensão *sparse*, incluindo restauração de arquivos esparsos.
- suporte de leitura/gravação para o formato POSIX.1-2001 (pax).
- manipula diretórios, arquivos regulares, links físicos, links simbólicos, fifos, dispositivos de caracteres e dispositivos de bloco e é capaz de adquirir e restaurar informações de arquivo, como registro de data e hora, permissões de acesso e proprietário.

Alterado na versão 3.3: Adicionado suporte para compactação `lzma`.

Alterado na versão 3.12: Os arquivos são extraídos usando um *filtro*, o que torna possível limitar recursos surpreendentes/perigosos ou reconhecer que eles são esperados e que o arquivo é totalmente confiável.

Alterado na versão 3.14: Define o filtro de extração padrão como `data`, o que desabilita alguns recursos perigosos, como links para caminhos absolutos ou caminhos fora do destino. Anteriormente, a estratégia de filtro era equivalente a `fully_trusted`.

Alterado na versão 3.14: Adicionado suporte para a compactação com Zstandard usando `compression.zstd`.

tarfile.**open**(*name=None*, *mode='r'*, *fileobj=None*, *bufsize=10240*, ***kwargs*)

Retorna um objeto *TarFile* para o caminho *name*. Para obter informações detalhadas sobre objetos *TarFile* e os argumentos nomeados permitidos, consulte *TarFile Objects*.

mode deve ser uma string no formato `'filemode[:compression]'`, o padrão é `'r'`. Aqui está uma lista completa de combinações de modos:

modo	ação
`'r'` ou `'r:*'`	Abre para leitura com compactação transparente (recomendado).
`'r:'`	Abre para leitura exclusivamente sem compactação.
`'r:gz'`	Abre para leitura com compactação gzip.
`'r:bz2'`	Abre para leitura com compactação bzip2.
`'r:xz'`	Abre para leitura com compactação lzma.
`'r:zst'`	Abre para leitura com compactação Zstandard.
`'x'` ou `'x:'`	Cria um arquivo tar exclusivamente sem compactação. Levanta uma exceção *FileExistsError* se ele já existir.
`'x:gz'`	Cria um arquivo tar com compactação gzip. Levanta uma exceção *FileExistsError* se ele já existir.
`'x:bz2'`	Cria um arquivo tar com compactação bzip2. Levanta uma exceção *FileExistsError* se ele já existir.
`'x:xz'`	Cria um arquivo tar com compactação lzma. Levanta uma exceção *FileExistsError* se ele já existir.
`'x:zst'`	Cria um arquivo tar com compactação Zstandard. Levanta uma exceção *FileExistsError* se ele já existir.
`'a'` ou `'a:'`	Aberto para acrescentar conteúdo sem compactação. O arquivo é criado se não existir.
`'w'` ou `'w:'`	Abre para gravação descompactada.
`'w:gz'`	Abre para gravação compactada com gzip.
`'w:bz2'`	Abre para gravação compactada com bzip2.
`'w:xz'`	Abre para gravação compactada com lzma.
`'w:zst'`	Abre para gravação compactada com Zstandard.

Observe que `'a:gz'`, `'a:bz2'` ou `'a:xz'` não são possíveis. Se o *mode* não for adequado para abrir um determinado arquivo (compactado) para leitura, *ReadError* será levantada. Use o *mode* `'r'` para evitar isso. Se um método de compactação não for suportado, *CompressionError* será levantada.

Se *fileobj* for especificado, ele será usado como alternativa a um *objeto arquivo* aberto em modo binário para *name*. Ele deve estar na posição 0.

Para os modos `'w:gz'`, `'x:gz'`, `'w|gz'`, `'w:bz2'`, `'x:bz2'`, `'w|bz2'`, *tarfile.open()* aceita o argumento nomeado *compresslevel* (padrão `9`) para especificar o nível de compactação do arquivo.

Para os modos `'w:xz'`, `'x:xz'` e `'w|xz'`, *tarfile.open()* aceita o argumento nomeado *preset* para especificar o nível de compactação do arquivo.

Para os modos `'w:zst'`, `'x:zst'` e `'w|zst'`, *tarfile.open()* aceita o argumento nomeado *level* para especificar o nível de compactação do arquivo. O argumento nomeado *options* também pode ser passado, fornecendo parâmetros avançados de compactação Zstandard descritos por *CompressionParameter*. O argumento nomeado *zstd_dict* pode ser passado para fornecer um *ZstdDict*, um dicionário Zstandard usado para melhorar a compactação de pequenas quantidades de dados.

Para fins especiais, existe um segundo formato para *mode*: `'filemode|[compression]'`. *tarfile.open()* retornará um objeto *TarFile* que processa seus dados como um fluxo de blocos. Nenhuma busca aleatória será realizada no arquivo. Se fornecido, *fileobj* pode ser qualquer objeto que tenha um método *read()* ou *write()* (dependendo do *mode*) que funcione com bytes. *bufsize* especifica o tamanho do bloco e o padrão é `20 * 512` bytes. Use esta variante em combinação com, por exemplo, `sys.stdin.buffer`, um *objeto arquivo* de soquete ou um dispositivo de fita. No entanto, tal objeto *TarFile* é limitado, pois não permite acesso aleatório; consulte *Exemplos*. Os modos atualmente possíveis:

Modo	Ação
`'r|*'`	Abre um *stream* de blocos tar para leitura com compactação transparente.
`'r|'`	Abre um *stream* de blocos tar não compactados para leitura.
`'r|gz'`	Abre um *stream* compactado com gzip para leitura.
`'r|bz2'`	Abre um *stream* compactado com bzip2 para leitura.
`'r|xz'`	Abre um *stream* compactado com lzma para leitura.
`'r|zst'`	Abre um *stream* compactado com Zstandard para leitura.
`'w|'`	Abre um *stream* descompactado para gravação.
`'w|gz'`	Abre um *stream* compactado com gzip para gravação.
`'w|bz2'`	Abre um *stream* compactado com bzip2 para gravação.
`'w|xz'`	Abre um *stream* compactado com lzma para gravação.
`'w|zst'`	Abre um *stream* compactado com Zstandard para gravação.

Alterado na versão 3.5: O modo `'x'` (criação exclusiva) foi adicionado.

Alterado na versão 3.6: O parâmetro *name* aceita um *objeto caminho ou similar*.

Alterado na versão 3.12: O argumento nomeado *compresslevel* também funciona para fluxos.

Alterado na versão 3.14: O argumento nomeado *preset* também funciona para fluxos.

class tarfile.**TarFile**

Classe para ler e escrever arquivos tar. Não use esta classe diretamente: use `tarfile.open()` em vez disso. Veja *TarFile Objects*.

tarfile.**is_tarfile**(*name*)

Retorna `True` se *name* for um arquivo tar que o módulo `tarfile` possa ler. *name* pode ser uma `str`, objeto arquivo ou similar.

Alterado na versão 3.9: Suporte para arquivo e objetos arquivo ou similares.

O módulo `tarfile` define as seguintes exceções:

exception tarfile.**TarError**

Classe base para todas as exceções de `tarfile`.

exception tarfile.**ReadError**

É levantada quando um arquivo tar é aberto e não pode ser manipulado pelo módulo `tarfile` ou é inválido de alguma forma.

exception tarfile.**CompressionError**

É levantada quando um método de compactação não é suportado ou quando os dados não podem ser decodificados corretamente.

exception tarfile.**StreamError**

É levantada para as limitações típicas de objetos `TarFile` do tipo fluxo.

exception tarfile.**ExtractError**

É levantada para erros *não fatais* ao usar `TarFile.extract()`, mas somente se `TarFile.errorlevel==2`.

exception tarfile.**HeaderError**

É levantada por `TarInfo.frombuf()` se o buffer obtido for inválido.

exception tarfile.**FilterError**

Classe base para membros *recusados* por filtros.

tarinfo

Informações sobre o membro que o filtro se recusou a extrair, como *TarInfo*.

exception tarfile.**AbsolutePathError**

Levantada para recusar a extração de um membro com um caminho absoluto.

exception tarfile.**OutsideDestinationError**

Levantada para recusar a extração de um membro fora do diretório de destino.

exception tarfile.**SpecialFileError**

Levantada para recusar a extração de um arquivo especial (por exemplo, um dispositivo ou encadeamento).

exception tarfile.**AbsoluteLinkError**

Levantada para recusar a extração de um link simbólico com um caminho absoluto.

exception tarfile.**LinkOutsideDestinationError**

Levantada para recusar a extração de um link simbólico apontando para fora do diretório de destino.

exception tarfile.**LinkFallbackError**

Levantada para recusar a emulação de um link (físico ou simbólico) extraindo outro membro do arquivo, quando esse membro seria rejeitado pelo local do filtro. A exceção levantada para rejeitar o membro substituto está disponível como `BaseException.__context__`.

Adicionado na versão 3.14.

As seguintes constantes estão disponíveis a nível de módulo:

tarfile.**ENCODING**

A codificação de caracteres padrão: `'utf-8'` no Windows, o valor retornado por *sys.getfilesystemencoding()*, caso contrário.

tarfile.**REGTYPE**

tarfile.**AREGTYPE**

Um arquivo normal *type*.

tarfile.**LNKTYPE**

Um link (dentro do arquivo tar) *type*.

tarfile.**SYMTYPE**

Um link simbólico *type*.

tarfile.**CHRTYPE**

Um dispositivo especial de caracteres *type*.

tarfile.**BLKTYPE**

Um dispositivo especial de bloco *type*.

tarfile.**DIRTYPE**

Um diretório *type*.

tarfile.**FIFOTYPE**

Um dispositivo especial de FIFO *type*.

tarfile.**CONTTYPE**

Um arquivo contíguo *type*.

tarfile.**GNUTYPE_LONGNAME**

A GNU tar longname *type*.

tarfile.**GNUTYPE_LONGLINK**

A GNU tar longlink *type*.

tarfile.**GNUTYPE_SPARSE**

A GNU tar sparse file *type*.

Cada uma das constantes a seguir define um formato de arquivo tar que o módulo *tarfile* é capaz de criar. Consulte a seção *Formatos tar suportados* para obter detalhes.

tarfile.**USTAR_FORMAT**
: formato POSIX.1-1988 (ustar).

tarfile.**GNU_FORMAT**
: Formato tar GNU

tarfile.**PAX_FORMAT**
: formato POSIX.1-2001 (pax).

tarfile.**DEFAULT_FORMAT**
: O formato padrão para criação de arquivos. Atualmente, é *PAX_FORMAT*.

  Alterado na versão 3.8: O formato padrão para novos arquivos foi alterado de *GNU_FORMAT* para *PAX_FORMAT*.

Ver também

Módulo *zipfile*
: Documentação do módulo padrão *zipfile*.

Operações de arquivamento
: Documentation of the higher-level archiving facilities provided by the standard *shutil* module.

GNU tar manual, Basic Tar Format
: Documentation for tar archive files, including GNU tar extensions.

13.8.1 TarFile Objects

The *TarFile* object provides an interface to a tar archive. A tar archive is a sequence of blocks. An archive member (a stored file) is made up of a header block followed by data blocks. It is possible to store a file in a tar archive several times. Each archive member is represented by a *TarInfo* object, see *Objetos TarInfo* for details.

A *TarFile* object can be used as a context manager in a `with` statement. It will automatically be closed when the block is completed. Please note that in the event of an exception an archive opened for writing will not be finalized; only the internally used file object will be closed. See the *Exemplos* section for a use case.

Adicionado na versão 3.2: Adicionado suporte para o protocolo de gerenciamento de contexto.

class tarfile.**TarFile**(*name=None*, *mode='r'*, *fileobj=None*, *format=DEFAULT_FORMAT*, *tarinfo=TarInfo*, *dereference=False*, *ignore_zeros=False*, *encoding=ENCODING*, *errors='surrogateescape'*, *pax_headers=None*, *debug=0*, *errorlevel=1*, *stream=False*)

All following arguments are optional and can be accessed as instance attributes as well.

name is the pathname of the archive. *name* may be a *path-like object*. It can be omitted if *fileobj* is given. In this case, the file object's `name` attribute is used if it exists.

mode is either `'r'` to read from an existing archive, `'a'` to append data to an existing file, `'w'` to create a new file overwriting an existing one, or `'x'` to create a new file only if it does not already exist.

If *fileobj* is given, it is used for reading or writing data. If it can be determined, *mode* is overridden by *fileobj*'s mode. *fileobj* will be used from position 0.

Nota

fileobj is not closed, when *TarFile* is closed.

format controls the archive format for writing. It must be one of the constants *USTAR_FORMAT*, *GNU_FORMAT* or *PAX_FORMAT* that are defined at module level. When reading, format will be automatically detected, even if different formats are present in a single archive.

The *tarinfo* argument can be used to replace the default *TarInfo* class with a different one.

If *dereference* is `False`, add symbolic and hard links to the archive. If it is `True`, add the content of the target files to the archive. This has no effect on systems that do not support symbolic links.

If *ignore_zeros* is `False`, treat an empty block as the end of the archive. If it is `True`, skip empty (and invalid) blocks and try to get as many members as possible. This is only useful for reading concatenated or damaged archives.

debug can be set from `0` (no debug messages) up to `3` (all debug messages). The messages are written to `sys.stderr`.

errorlevel controls how extraction errors are handled, see *the corresponding attribute*.

The *encoding* and *errors* arguments define the character encoding to be used for reading or writing the archive and how conversion errors are going to be handled. The default settings will work for most users. See section *Problemas de Unicode* for in-depth information.

The *pax_headers* argument is an optional dictionary of strings which will be added as a pax global header if *format* is `PAX_FORMAT`.

If *stream* is set to `True` then while reading the archive info about files in the archive are not cached, saving memory.

Alterado na versão 3.2: Use `'surrogateescape'` as the default for the *errors* argument.

Alterado na versão 3.5: O modo `'x'` (criação exclusiva) foi adicionado.

Alterado na versão 3.6: O parâmetro *name* aceita um *objeto caminho ou similar*.

Alterado na versão 3.13: Add the *stream* parameter.

classmethod TarFile.**open**(...)

Alternative constructor. The *tarfile.open()* function is actually a shortcut to this classmethod.

TarFile.**getmember**(*name*)

Return a *TarInfo* object for member *name*. If *name* can not be found in the archive, *KeyError* is raised.

> **Nota**
>
> If a member occurs more than once in the archive, its last occurrence is assumed to be the most up-to-date version.

TarFile.**getmembers**()

Retorna os membros do arquivo como uma lista de objetos *TarInfo*. A lista tem a mesma ordem que os membros no arquivo.

TarFile.**getnames**()

Return the members as a list of their names. It has the same order as the list returned by *getmembers()*.

TarFile.**list**(*verbose=True*, *, *members=None*)

Print a table of contents to `sys.stdout`. If *verbose* is `False`, only the names of the members are printed. If it is `True`, output similar to that of **`ls -l`** is produced. If optional *members* is given, it must be a subset of the list returned by *getmembers()*.

Alterado na versão 3.5: Added the *members* parameter.

TarFile.**next**()

Return the next member of the archive as a *TarInfo* object, when *TarFile* is opened for reading. Return *None* if there is no more available.

TarFile.**extractall**(*path='.'*, *members=None*, *, *numeric_owner=False*, *filter=None*)

Extract all members from the archive to the current working directory or directory *path*. If optional *members* is given, it must be a subset of the list returned by *getmembers()*. Directory information like owner, modification time and permissions are set after all members have been extracted. This is done to work around

two problems: A directory's modification time is reset each time a file is created in it. And, if a directory's permissions do not allow writing, extracting files to it will fail.

If *numeric_owner* is `True`, the uid and gid numbers from the tarfile are used to set the owner/group for the extracted files. Otherwise, the named values from the tarfile are used.

The *filter* argument specifies how `members` are modified or rejected before extraction. See *Filtros de extração* for details. It is recommended to set this explicitly only if specific *tar* features are required, or as `filter='data'` to support Python versions with a less secure default (3.13 and lower).

Aviso

Never extract archives from untrusted sources without prior inspection.

Since Python 3.14, the default (`data`) will prevent the most dangerous security issues. However, it will not prevent *all* unintended or insecure behavior. Read the *Filtros de extração* section for details.

Alterado na versão 3.5: Adicionado o parâmetro *numeric_owner*.

Alterado na versão 3.6: O parâmetro *path* aceita um *objeto caminho ou similar*.

Alterado na versão 3.12: Adicionado o parâmetro *filter*.

Alterado na versão 3.14: The *filter* parameter now defaults to `'data'`.

`TarFile.`**`extract`**(*member*, *path=''*, *set_attrs=True*, ***, *numeric_owner=False*, *filter=None*)

Extract a member from the archive to the current working directory, using its full name. Its file information is extracted as accurately as possible. *member* may be a filename or a `TarInfo` object. You can specify a different directory using *path*. *path* may be a *path-like object*. File attributes (owner, mtime, mode) are set unless *set_attrs* is false.

The *numeric_owner* and *filter* arguments are the same as for `extractall()`.

Nota

The `extract()` method does not take care of several extraction issues. In most cases you should consider using the `extractall()` method.

Aviso

Never extract archives from untrusted sources without prior inspection. See the warning for `extractall()` for details.

Alterado na versão 3.2: Added the *set_attrs* parameter.

Alterado na versão 3.5: Adicionado o parâmetro *numeric_owner*.

Alterado na versão 3.6: O parâmetro *path* aceita um *objeto caminho ou similar*.

Alterado na versão 3.12: Adicionado o parâmetro *filter*.

`TarFile.`**`extractfile`**(*member*)

Extract a member from the archive as a file object. *member* may be a filename or a `TarInfo` object. If *member* is a regular file or a link, an `io.BufferedReader` object is returned. For all other existing members, `None` is returned. If *member* does not appear in the archive, `KeyError` is raised.

Alterado na versão 3.3: Return an `io.BufferedReader` object.

Alterado na versão 3.13: The returned `io.BufferedReader` object has the `mode` attribute which is always equal to `'rb'`.

TarFile.**errorlevel**: *int*

Se *errorlevel* for `0`, os erros serão ignorados ao usar `TarFile.extract()` e `TarFile.extractall()`. No entanto, eles aparecem como mensagens de erro na saída de depuração quando *debug* for maior que 0. Se `1` (o padrão), todos os erros *fatais* serão levantados como exceções `OSError` ou `FilterError`. Se `2`, todos os erros *não fatais* também serão levantados como exceções `TarError`.

Some exceptions, e.g. ones caused by wrong argument types or data corruption, are always raised.

Custom *extraction filters* should raise `FilterError` for *fatal* errors and `ExtractError` for *non-fatal* ones.

Note that when an exception is raised, the archive may be partially extracted. It is the user's responsibility to clean up.

TarFile.**extraction_filter**

Adicionado na versão 3.12.

O *filtro de extração* usado como um padrão para o argumento *filter* de `extract()` e `extractall()`.

The attribute may be `None` or a callable. String names are not allowed for this attribute, unlike the *filter* argument to `extract()`.

If `extraction_filter` is `None` (the default), extraction methods will use the `data` filter by default.

The attribute may be set on instances or overridden in subclasses. It also is possible to set it on the `TarFile` class itself to set a global default, although, since it affects all uses of *tarfile*, it is best practice to only do so in top-level applications or `site configuration`. To set a global default this way, a filter function needs to be wrapped in `staticmethod()` to prevent injection of a `self` argument.

Alterado na versão 3.14: The default filter is set to `data`, which disallows some dangerous features such as links to absolute paths or paths outside of the destination. Previously, the default was equivalent to `fully_trusted`.

TarFile.**add**(*name*, *arcname=None*, *recursive=True*, ***, *filter=None*)

Add the file *name* to the archive. *name* may be any type of file (directory, fifo, symbolic link, etc.). If given, *arcname* specifies an alternative name for the file in the archive. Directories are added recursively by default. This can be avoided by setting *recursive* to `False`. Recursion adds entries in sorted order. If *filter* is given, it should be a function that takes a `TarInfo` object argument and returns the changed `TarInfo` object. If it instead returns `None` the `TarInfo` object will be excluded from the archive. See *Exemplos* for an example.

Alterado na versão 3.2: Adicionado o parâmetro *filter*.

Alterado na versão 3.7: Recursion adds entries in sorted order.

TarFile.**addfile**(*tarinfo*, *fileobj=None*)

Add the `TarInfo` object *tarinfo* to the archive. If *tarinfo* represents a non zero-size regular file, the *fileobj* argument should be a *binary file*, and `tarinfo.size` bytes are read from it and added to the archive. You can create `TarInfo` objects directly, or by using `gettarinfo()`.

Alterado na versão 3.13: *fileobj* must be given for non-zero-sized regular files.

TarFile.**gettarinfo**(*name=None*, *arcname=None*, *fileobj=None*)

Create a `TarInfo` object from the result of `os.stat()` or equivalent on an existing file. The file is either named by *name*, or specified as a *file object* *fileobj* with a file descriptor. *name* may be a *path-like object*. If given, *arcname* specifies an alternative name for the file in the archive, otherwise, the name is taken from *fileobj*'s `name` attribute, or the *name* argument. The name should be a text string.

You can modify some of the `TarInfo`'s attributes before you add it using `addfile()`. If the file object is not an ordinary file object positioned at the beginning of the file, attributes such as `size` may need modifying. This is the case for objects such as `GzipFile`. The `name` may also be modified, in which case *arcname* could be a dummy string.

Alterado na versão 3.6: O parâmetro *name* aceita um *objeto caminho ou similar*.

TarFile.**close**()

Close the `TarFile`. In write mode, two finishing zero blocks are appended to the archive.

TarFile.**pax_headers**: *dict*

Um dicionário contendo pares de chave-valor de cabeçalhos globais pax.

13.8.2 Objetos TarInfo

A *TarInfo* object represents one member in a *TarFile*. Aside from storing all required attributes of a file (like file type, size, time, permissions, owner etc.), it provides some useful methods to determine its type. It does *not* contain the file's data itself.

TarInfo objects are returned by *TarFile*'s methods *getmember()*, *getmembers()* and *gettarinfo()*.

Modifying the objects returned by *getmember()* or *getmembers()* will affect all subsequent operations on the archive. For cases where this is unwanted, you can use *copy.copy()* or call the *replace()* method to create a modified copy in one step.

Several attributes can be set to `None` to indicate that a piece of metadata is unused or unknown. Different *TarInfo* methods handle `None` differently:

- The *extract()* or *extractall()* methods will ignore the corresponding metadata, leaving it set to a default.
- *addfile()* falhará.
- *list()* imprimirá uma string como um espaço reservado.

class tarfile.**TarInfo**(*name=''*)

Cria um objeto *TarInfo*.

classmethod TarInfo.**frombuf**(*buf*, *encoding*, *errors*)

Cria e retorna um objeto *TarInfo* a partir do buffer de string do parâmetro *buf*.

Levanta *HeaderError* se o buffer for inválido.

classmethod TarInfo.**fromtarfile**(*tarfile*)

Read the next member from the *TarFile* object *tarfile* and return it as a *TarInfo* object.

TarInfo.**tobuf**(*format=DEFAULT_FORMAT*, *encoding=ENCODING*, *errors='surrogateescape'*)

Create a string buffer from a *TarInfo* object. For information on the arguments see the constructor of the *TarFile* class.

Alterado na versão 3.2: Use `'surrogateescape'` as the default for the *errors* argument.

A `TarInfo` object has the following public data attributes:

TarInfo.**name**: *str*

Nome do arquivo.

TarInfo.**size**: *int*

Tamanho em bytes.

TarInfo.**mtime**: *int* | *float*

Time of last modification in seconds since the *epoch*, as in *os.stat_result.st_mtime*.

Alterado na versão 3.12: Can be set to `None` for *extract()* and *extractall()*, causing extraction to skip applying this attribute.

TarInfo.**mode**: *int*

Permission bits, as for *os.chmod()*.

Alterado na versão 3.12: Can be set to `None` for *extract()* and *extractall()*, causing extraction to skip applying this attribute.

TarInfo.**type**

File type. *type* is usually one of these constants: *REGTYPE*, *AREGTYPE*, *LNKTYPE*, *SYMTYPE*, *DIRTYPE*, *FIFOTYPE*, *CONTTYPE*, *CHRTYPE*, *BLKTYPE*, *GNUTYPE_SPARSE*. To determine the type of a *TarInfo* object more conveniently, use the `is*()` methods below.

TarInfo.**linkname**: *str*

Name of the target file name, which is only present in *TarInfo* objects of type *LNKTYPE* and *SYMTYPE*.

For symbolic links (SYMTYPE), the *linkname* is relative to the directory that contains the link. For hard links (LNKTYPE), the *linkname* is relative to the root of the archive.

TarInfo.**uid**: *int*

ID do usuário do usuário que originalmente armazenou este membro.

Alterado na versão 3.12: Can be set to None for *extract()* and *extractall()*, causing extraction to skip applying this attribute.

TarInfo.**gid**: *int*

ID do grupo do usuário que originalmente armazenou este membro.

Alterado na versão 3.12: Can be set to None for *extract()* and *extractall()*, causing extraction to skip applying this attribute.

TarInfo.**uname**: *str*

Nome do usuário.

Alterado na versão 3.12: Can be set to None for *extract()* and *extractall()*, causing extraction to skip applying this attribute.

TarInfo.**gname**: *str*

Nome do grupo.

Alterado na versão 3.12: Can be set to None for *extract()* and *extractall()*, causing extraction to skip applying this attribute.

TarInfo.**chksum**: *int*

Cabeçalho de soma de verificação.

TarInfo.**devmajor**: *int*

Número principal do dispositivo.

TarInfo.**devminor**: *int*

Número menor do dispositivo.

TarInfo.**offset**: *int*

O cabeçalho do tar começa aqui.

TarInfo.**offset_data**: *int*

Os dados do arquivo começam aqui.

TarInfo.**sparse**

Sparse member information.

TarInfo.**pax_headers**: *dict*

A dictionary containing key-value pairs of an associated pax extended header.

TarInfo.**replace**(*name=...*, *mtime=...*, *mode=...*, *linkname=...*, *uid=...*, *gid=...*, *uname=...*, *gname=...*, *deep=True*)

Adicionado na versão 3.12.

Return a *new* copy of the TarInfo object with the given attributes changed. For example, to return a TarInfo with the group name set to 'staff', use:

```
new_tarinfo = old_tarinfo.replace(gname='staff')
```

By default, a deep copy is made. If *deep* is false, the copy is shallow, i.e. pax_headers and any custom attributes are shared with the original TarInfo object.

A *TarInfo* object also provides some convenient query methods:

`TarInfo.isfile()`
: Return `True` if the `TarInfo` object is a regular file.

`TarInfo.isreg()`
: Igual a `isfile()`.

`TarInfo.isdir()`
: Retorna `True` se for um diretório.

`TarInfo.issym()`
: Retorna `True` se for um link simbólico.

`TarInfo.islnk()`
: Retorna `True` se for um link físico.

`TarInfo.ischr()`
: Retorna `True` se for um caractere.

`TarInfo.isblk()`
: Return `True` if it is a block device.

`TarInfo.isfifo()`
: Retorna `True` se for um FIFO.

`TarInfo.isdev()`
: Return `True` if it is one of character device, block device or FIFO.

13.8.3 Filtros de extração

Adicionado na versão 3.12.

The *tar* format is designed to capture all details of a UNIX-like filesystem, which makes it very powerful. Unfortunately, the features make it easy to create tar files that have unintended – and possibly malicious – effects when extracted. For example, extracting a tar file can overwrite arbitrary files in various ways (e.g. by using absolute paths, `..` path components, or symlinks that affect later members).

In most cases, the full functionality is not needed. Therefore, *tarfile* supports extraction filters: a mechanism to limit functionality, and thus mitigate some of the security issues.

Aviso

None of the available filters blocks *all* dangerous archive features. Never extract archives from untrusted sources without prior inspection. See also *Dicas para verificação adicional*.

Ver também

PEP 706
: Contém a motivação e lógica escolhida para este design.

The *filter* argument to `TarFile.extract()` or `extractall()` can be:

- the string `'fully_trusted'`: Honor all metadata as specified in the archive. Should be used if the user trusts the archive completely, or implements their own complex verification.
- the string `'tar'`: Honor most *tar*-specific features (i.e. features of UNIX-like filesystems), but block features that are very likely to be surprising or malicious. See `tar_filter()` for details.
- the string `'data'`: Ignore or block most features specific to UNIX-like filesystems. Intended for extracting cross-platform data archives. See `data_filter()` for details.

- `None` (default): Use *`TarFile.extraction_filter`*.

  If that is also `None` (the default), the `'data'` filter will be used.

  > Alterado na versão 3.14: The default filter is set to *`data`*. Previously, the default was equivalent to *`fully_trusted`*.

- A callable which will be called for each extracted member with a *TarInfo* describing the member and the destination path to where the archive is extracted (i.e. the same path is used for all members):

  ```
  filter(member: TarInfo, path: str, /) -> TarInfo | None
  ```

  The callable is called just before each member is extracted, so it can take the current state of the disk into account. It can:

  - return a *`TarInfo`* object which will be used instead of the metadata in the archive, or
  - return `None`, in which case the member will be skipped, or
  - raise an exception to abort the operation or skip the member, depending on *`errorlevel`*. Note that when extraction is aborted, *`extractall()`* may leave the archive partially extracted. It does not attempt to clean up.

Default named filters

The pre-defined, named filters are available as functions, so they can be reused in custom filters:

`tarfile.`**`fully_trusted_filter`**(*member*, *path*)

Retorna *member* inalterado.

This implements the `'fully_trusted'` filter.

`tarfile.`**`tar_filter`**(*member*, *path*)

Implements the `'tar'` filter.

- Strip leading slashes (`/` and *`os.sep`*) from filenames.
- *Refuse* to extract files with absolute paths (in case the name is absolute even after stripping slashes, e.g. `C:/foo` on Windows). This raises *`AbsolutePathError`*.
- *Refuse* to extract files whose absolute path (after following symlinks) would end up outside the destination. This raises *`OutsideDestinationError`*.
- Clear high mode bits (setuid, setgid, sticky) and group/other write bits (*`S_IWGRP`* | *`S_IWOTH`*).

Return the modified `TarInfo` member.

`tarfile.`**`data_filter`**(*member*, *path*)

Implements the `'data'` filter. In addition to what `tar_filter` does:

- Normalize link targets (*`TarInfo.linkname`*) using *`os.path.normpath()`*. Note that this removes internal `..` components, which may change the meaning of the link if the path in `TarInfo.linkname` traverses symbolic links.
- *Refuse* to extract links (hard or soft) that link to absolute paths, or ones that link outside the destination.

  This raises *`AbsoluteLinkError`* or *`LinkOutsideDestinationError`*.

  Note that such files are refused even on platforms that do not support symbolic links.
- *Refuse* to extract device files (including pipes). This raises *`SpecialFileError`*.
- Para arquivos comuns, incluindo links físicos:
  - Set the owner read and write permissions (*`S_IRUSR`* | *`S_IWUSR`*).
  - Remove the group & other executable permission (*`S_IXGRP`* | *`S_IXOTH`*) if the owner doesn't have it (*`S_IXUSR`*).
- For other files (directories), set `mode` to `None`, so that extraction methods skip applying permission bits.

- Set user and group info (`uid`, `gid`, `uname`, `gname`) to `None`, so that extraction methods skip setting it.

Return the modified `TarInfo` member.

Note that this filter does not block *all* dangerous archive features. See *Dicas para verificação adicional* for details.

Alterado na versão 3.14: Link targets are now normalized.

Filter errors

When a filter refuses to extract a file, it will raise an appropriate exception, a subclass of *`FilterError`*. This will abort the extraction if *`TarFile.errorlevel`* is 1 or more. With `errorlevel=0` the error will be logged and the member will be skipped, but extraction will continue.

Dicas para verificação adicional

Even with `filter='data'`, *tarfile* is not suited for extracting untrusted files without prior inspection. Among other issues, the pre-defined filters do not prevent denial-of-service attacks. Users should do additional checks.

Aqui está uma lista incompleta de itens a serem considerados:

- Extract to a *`new temporary directory`* to prevent e.g. exploiting pre-existing links, and to make it easier to clean up after a failed extraction.
- Disallow symbolic links if you do not need the functionality.
- When working with untrusted data, use external (e.g. OS-level) limits on disk, memory and CPU usage.
- Check filenames against an allow-list of characters (to filter out control characters, confusables, foreign path separators, and so on).
- Check that filenames have expected extensions (discouraging files that execute when you "click on them", or extension-less files like Windows special device names).
- Limit the number of extracted files, total size of extracted data, filename length (including symlink length), and size of individual files.
- Check for files that would be shadowed on case-insensitive filesystems.

Observe também que:

- Tar files may contain multiple versions of the same file. Later ones are expected to overwrite any earlier ones. This feature is crucial to allow updating tape archives, but can be abused maliciously.
- *tarfile* does not protect against issues with "live" data, e.g. an attacker tinkering with the destination (or source) directory while extraction (or archiving) is in progress.

Suporte a versões mais antigas do Python

Extraction filters were added to Python 3.12, but may be backported to older versions as security updates. To check whether the feature is available, use e.g. `hasattr(tarfile, 'data_filter')` rather than checking the Python version.

The following examples show how to support Python versions with and without the feature. Note that setting `extraction_filter` will affect any subsequent operations.

- Fully trusted archive:

```
my_tarfile.extraction_filter = (lambda member, path: member)
my_tarfile.extractall()
```

- Use the `'data'` filter if available, but revert to Python 3.11 behavior (`'fully_trusted'`) if this feature is not available:

```
my_tarfile.extraction_filter = getattr(tarfile, 'data_filter',
                                       (lambda member, path: member))
my_tarfile.extractall()
```

- Use o filtro `'data'`; *fail* se ele não estiver disponível:

```
my_tarfile.extractall(filter=tarfile.data_filter)
```

ou:

```
my_tarfile.extraction_filter = tarfile.data_filter
my_tarfile.extractall()
```

- Use o filtro `'data'`; *warn* se ele não estiver disponível:

```
if hasattr(tarfile, 'data_filter'):
    my_tarfile.extractall(filter='data')
else:
    # remove this when no longer needed
    warn_the_user('Extracting may be unsafe; consider updating Python')
    my_tarfile.extractall()
```

Stateful extraction filter example

While *tarfile*'s extraction methods take a simple *filter* callable, custom filters may be more complex objects with an internal state. It may be useful to write these as context managers, to be used like this:

```
with StatefulFilter() as filter_func:
    tar.extractall(path, filter=filter_func)
```

Such a filter can be written as, for example:

```
class StatefulFilter:
    def __init__(self):
        self.file_count = 0

    def __enter__(self):
        return self

    def __call__(self, member, path):
        self.file_count += 1
        return member

    def __exit__(self, *exc_info):
        print(f'{self.file_count} files extracted')
```

13.8.4 Interface de Linha de Comando

Adicionado na versão 3.4.

The `tarfile` module provides a simple command-line interface to interact with tar archives.

If you want to create a new tar archive, specify its name after the `-c` option and then list the filename(s) that should be included:

```
$ python -m tarfile -c monty.tar  spam.txt eggs.txt
```

Passar um diretório também é aceito:

```
$ python -m tarfile -c monty.tar life-of-brian_1979/
```

If you want to extract a tar archive into the current directory, use the `-e` option:

```
$ python -m tarfile -e monty.tar
```

You can also extract a tar archive into a different directory by passing the directory's name:

```
$ python -m tarfile -e monty.tar  other-dir/
```

For a list of the files in a tar archive, use the `-l` option:

```
$ python -m tarfile -l monty.tar
```

Opções de linha de comando

-l <tarfile>

--list <tarfile>

Lista os arquivos em um arquivo tar.

-c <tarfile> <source1> ... <sourceN>

--create <tarfile> <source1> ... <sourceN>

Cria um arquivo tar a partir dos arquivos de origem.

-e <tarfile> [<output_dir>]

--extract <tarfile> [<output_dir>]

Extract tarfile into the current directory if *output_dir* is not specified.

-t <tarfile>

--test <tarfile>

Test whether the tarfile is valid or not.

-v, --verbose

Saída detalhada.

--filter <filtername>

Specifies the *filter* for `--extract`. See *Filtros de extração* for details. Only string names are accepted (that is, `fully_trusted`, `tar`, and `data`).

13.8.5 Exemplos

Reading examples

How to extract an entire tar archive to the current working directory:

```
import tarfile
tar = tarfile.open("sample.tar.gz")
tar.extractall(filter='data')
tar.close()
```

How to extract a subset of a tar archive with `TarFile.extractall()` using a generator function instead of a list:

```
import os
import tarfile

def py_files(members):
    for tarinfo in members:
        if os.path.splitext(tarinfo.name)[1] == ".py":
```

(continua na próxima página)

(continuação da página anterior)

```
            yield tarinfo

tar = tarfile.open("sample.tar.gz")
tar.extractall(members=py_files(tar))
tar.close()
```

How to read a gzip compressed tar archive and display some member information:

```
import tarfile
tar = tarfile.open("sample.tar.gz", "r:gz")
for tarinfo in tar:
    print(tarinfo.name, "is", tarinfo.size, "bytes in size and is ", end="")
    if tarinfo.isreg():
        print("a regular file.")
    elif tarinfo.isdir():
        print("a directory.")
    else:
        print("something else.")
tar.close()
```

Writing examples

How to create an uncompressed tar archive from a list of filenames:

```
import tarfile
tar = tarfile.open("sample.tar", "w")
for name in ["foo", "bar", "quux"]:
    tar.add(name)
tar.close()
```

The same example using the `with` statement:

```
import tarfile
with tarfile.open("sample.tar", "w") as tar:
    for name in ["foo", "bar", "quux"]:
        tar.add(name)
```

How to create and write an archive to stdout using *sys.stdout.buffer* in the *fileobj* parameter in *TarFile.add()*:

```
import sys
import tarfile
with tarfile.open("sample.tar.gz", "w|gz", fileobj=sys.stdout.buffer) as tar:
    for name in ["foo", "bar", "quux"]:
        tar.add(name)
```

How to create an archive and reset the user information using the *filter* parameter in *TarFile.add()*:

```
import tarfile
def reset(tarinfo):
    tarinfo.uid = tarinfo.gid = 0
    tarinfo.uname = tarinfo.gname = "root"
    return tarinfo
tar = tarfile.open("sample.tar.gz", "w:gz")
tar.add("foo", filter=reset)
tar.close()
```

13.8.6 Formatos tar suportados

There are three tar formats that can be created with the `tarfile` module:

- The POSIX.1-1988 ustar format (`USTAR_FORMAT`). It supports filenames up to a length of at best 256 characters and linknames up to 100 characters. The maximum file size is 8 GiB. This is an old and limited but widely supported format.
- The GNU tar format (`GNU_FORMAT`). It supports long filenames and linknames, files bigger than 8 GiB and sparse files. It is the de facto standard on GNU/Linux systems. `tarfile` fully supports the GNU tar extensions for long names, sparse file support is read-only.
- The POSIX.1-2001 pax format (`PAX_FORMAT`). It is the most flexible format with virtually no limits. It supports long filenames and linknames, large files and stores pathnames in a portable way. Modern tar implementations, including GNU tar, bsdtar/libarchive and star, fully support extended *pax* features; some old or unmaintained libraries may not, but should treat *pax* archives as if they were in the universally supported *ustar* format. It is the current default format for new archives.

  It extends the existing *ustar* format with extra headers for information that cannot be stored otherwise. There are two flavours of pax headers: Extended headers only affect the subsequent file header, global headers are valid for the complete archive and affect all following files. All the data in a pax header is encoded in *UTF-8* for portability reasons.

There are some more variants of the tar format which can be read, but not created:

- The ancient V7 format. This is the first tar format from Unix Seventh Edition, storing only regular files and directories. Names must not be longer than 100 characters, there is no user/group name information. Some archives have miscalculated header checksums in case of fields with non-ASCII characters.
- The SunOS tar extended format. This format is a variant of the POSIX.1-2001 pax format, but is not compatible.

13.8.7 Problemas de Unicode

The tar format was originally conceived to make backups on tape drives with the main focus on preserving file system information. Nowadays tar archives are commonly used for file distribution and exchanging archives over networks. One problem of the original format (which is the basis of all other formats) is that there is no concept of supporting different character encodings. For example, an ordinary tar archive created on a *UTF-8* system cannot be read correctly on a *Latin-1* system if it contains non-*ASCII* characters. Textual metadata (like filenames, linknames, user/group names) will appear damaged. Unfortunately, there is no way to autodetect the encoding of an archive. The pax format was designed to solve this problem. It stores non-ASCII metadata using the universal character encoding *UTF-8*.

The details of character conversion in `tarfile` are controlled by the *encoding* and *errors* keyword arguments of the `TarFile` class.

encoding defines the character encoding to use for the metadata in the archive. The default value is `sys.getfilesystemencoding()` or `'ascii'` as a fallback. Depending on whether the archive is read or written, the metadata must be either decoded or encoded. If *encoding* is not set appropriately, this conversion may fail.

The *errors* argument defines how characters are treated that cannot be converted. Possible values are listed in section *Error Handlers*. The default scheme is `'surrogateescape'` which Python also uses for its file system calls, see *Nomes de arquivos, argumentos de linha de comando e variáveis de ambiente*.

For `PAX_FORMAT` archives (the default), *encoding* is generally not needed because all the metadata is stored using *UTF-8*. *encoding* is only used in the rare cases when binary pax headers are decoded or when strings with surrogate characters are stored.

CAPÍTULO 14

Formatos de Arquivo

Os módulos descritos neste capítulo analisam vários formatos de arquivo diversos que não são linguagens de marcação e não estão relacionados ao e-mail.

14.1 `csv` — Leitura e escrita de arquivos CSV

Código-fonte: Lib/csv.py

O chamado formato CSV (Comma Separated Values) é o formato mais comum de importação e exportação de planilhas e bancos de dados. O formato CSV foi usado por muitos anos antes das tentativas de descrever o formato de maneira padronizada em **RFC 4180**. A falta de um padrão bem definido significa que existem diferenças sutis nos dados produzidos e consumidos por diferentes aplicativos. Essas diferenças podem tornar irritante o processamento de arquivos CSV de várias fontes. Ainda assim, enquanto os delimitadores e os caracteres de citação variam, o formato geral é semelhante o suficiente para que seja possível escrever um único módulo que possa manipular eficientemente esses dados, ocultando os detalhes da leitura e gravação dos dados do programador.

O módulo `csv` implementa classes para ler e gravar dados tabulares no formato CSV. Ele permite que os programadores digam "escreva esses dados no formato preferido pelo Excel" ou "leia os dados desse arquivo gerado pelo Excel", sem conhecer os detalhes precisos do formato CSV usado pelo Excel. Os programadores também podem descrever os formatos CSV entendidos por outros aplicativos ou definir seus próprios formatos CSV para fins especiais.

Os objetos de `reader` e `writer` do módulo `csv` leem e escrevem sequências. Os programadores também podem ler e gravar dados no formato de dicionário usando as classes `DictReader` e `DictWriter`.

Ver também

PEP 305 - API de arquivo CSV
A proposta de melhoria do Python que propôs essa adição ao Python.

14.1.1 Conteúdo do módulo

O módulo `csv` define as seguintes funções:

csv.**reader**(*csvfile*, */*, *dialect='excel'*, ***fmtparams*)

Retorna um *objeto leitor* que irá iterar sobre as linhas no *csvfile* fornecido. Um csvfile deve ser um iterável de strings, cada uma no formato csv definido pelo leitor. Um csvfile é muito comumente um objeto similar a arquivo ou uma lista. Se *csvfile* for um objeto arquivo, ele deverá ser aberto com `newline=''`.[1] Pode ser fornecido um parâmetro opcional *dialect*, usado para definir um conjunto de parâmetros específicos para um dialeto CSV específico. Pode ser uma instância de uma subclasse da classe `Dialect` ou uma das strings retornadas pela função `list_dialects()`. Os outros argumentos nomeados opcionais *fmtparams* podem ser dados para substituir parâmetros de formatação individuais no dialeto atual. Para detalhes completos sobre os parâmetros de dialeto e formatação, consulte a seção *Dialetos e parâmetros de formatação*.

Cada linha lida no arquivo csv é retornada como uma lista de strings. Nenhuma conversão automática de tipo de dados é executada, a menos que a opção de formato `QUOTE_NONNUMERIC` seja especificada (nesse caso, os campos não citados são transformados em pontos flutuantes).

Um pequeno exemplo de uso:

```
>>> import csv
>>> with open('eggs.csv', newline='') as csvfile:
...     spamreader = csv.reader(csvfile, delimiter=' ', quotechar='|')
...     for row in spamreader:
...         print(', '.join(row))
Spam, Spam, Spam, Spam, Spam, Baked Beans
Spam, Lovely Spam, Wonderful Spam
```

csv.**writer**(*csvfile*, */*, *dialect='excel'*, ***fmtparams*)

Retorna um objeto de escrita responsável por converter os dados de usuário em strings delimitadas no objeto arquivo ou similar. *csvfile* pode ser qualquer objeto com um método `write()`. Se *csvfile* for um objeto arquivo, ele deverá ser aberto com `newline=''`[1]. Pode ser fornecido um parâmetro opcional *dialect*, usado para definir um conjunto de parâmetros específicos para um dialeto CSV específico. Pode ser uma instância de uma subclasse da classe `Dialect` ou uma das strings retornadas pela função `list_dialects()`. Os outros argumentos nomeados opcionais *fmtparams* podem ser dados para substituir parâmetros de formatação individuais no dialeto atual. Para detalhes completos sobre os parâmetros de dialeto e formatação, consulte a seção *Dialetos e parâmetros de formatação*. Para tornar o mais fácil possível a interface com os módulos que implementam a API do DB, o valor `None` é escrito como uma string vazia. Embora isso não seja uma transformação reversível, torna mais fácil despejar valores de dados SQL NULL em arquivos CSV sem preprocessar os dados retornados de uma chamada `cursor.fetch*`. Todos os outros dados que não são de strings são codificados com `str()` antes de serem escritos.

Um pequeno exemplo de uso:

```
import csv
with open('eggs.csv', 'w', newline='') as csvfile:
    spamwriter = csv.writer(csvfile, delimiter=' ',
                            quotechar='|', quoting=csv.QUOTE_MINIMAL)
    spamwriter.writerow(['Spam'] * 5 + ['Baked Beans'])
    spamwriter.writerow(['Spam', 'Lovely Spam', 'Wonderful Spam'])
```

csv.**register_dialect**(*name*[, *dialect*[, ***fmtparams*]])

Associa *dialect* a *name*. *name* deve ser uma string. O dialeto pode ser especificado passando uma subclasse de `Dialect` ou por *fmtparams* argumentos nomeados, ou ambos, com argumentos nomeados substituindo os parâmetros do dialeto. Para detalhes completos sobre os parâmetros de dialeto e formatação, consulte a seção *Dialetos e parâmetros de formatação*.

csv.**unregister_dialect**(*name*)

Exclui o dialeto associado ao *name* do registro do dialeto. Um `Error` é levantado se *name* não for um nome de dialeto registrado.

[1] Se `newline=''` não for especificado, quebras de linha inseridas em campos entre aspas não serão interpretadas corretamente e, em plataformas que usam terminações de linha `\r\n` na gravação, um `\r` extra será adicionado. É sempre seguro especificar `newline=''`, já que o módulo csv faz seu próprio tratamento de quebra de linha (*universal*).

csv.**get_dialect**(*name*)

Retorna o dialeto associado a *name*. Um *Error* é levantado se *name* não for um nome de dialeto registrado. Esta função retorna uma classe *Dialect* imutável.

csv.**list_dialects**()

Retorna os nomes de todos os dialetos registrados

csv.**field_size_limit**([*new_limit*])

Retorna o tamanho máximo atual do campo permitido pelo analisador sintático. Se *new_limit* for fornecido, este se tornará o novo limite.

O módulo *csv* define as seguintes classes:

class csv.**DictReader**(*f*, *fieldnames=None*, *restkey=None*, *restval=None*, *dialect='excel'*, **args*, ***kwds*)

Cria um objeto que funcione como um leitor comum, mas mapeia as informações em cada linha para *dict* cujas chaves são fornecidas pelo parâmetro opcional *fieldnames*.

O parâmetro *fieldnames* é uma *sequência*. Se *fieldnames* for omitido, os valores na primeira linha do arquivo *f* serão usados como nomes de campo e serão omitidos dos resultados. Se *fieldnames* for fornecido, eles serão usados e a primeira linha será incluída nos resultados. Independentemente de como os nomes de campo são determinados, o dicionário preserva sua ordem original.

Se uma linha tiver mais campos que nomes de campo, os dados restantes serão colocados em uma lista e armazenados com o nome do campo especificado por *restkey* (o padrão é `None`). Se uma linha que não estiver em branco tiver menos campos que nomes de campo, os valores ausentes serão preenchidos com o valor *restval* (o padrão é `None`).

Todos os outros argumentos nomeados ou opcionais são passados para a instância subjacente de *reader*.

Se o argumento passado para *fieldnames* for um iterador, ele será convertido para uma *list*.

Alterado na versão 3.6: Linhas retornadas agora são do tipo `OrderedDict`.

Alterado na versão 3.8: As linhas retornadas agora são do tipo *dict*.

Um pequeno exemplo de uso:

```
>>> import csv
>>> with open('names.csv', newline='') as csvfile:
...     reader = csv.DictReader(csvfile)
...     for row in reader:
...         print(row['first_name'], row['last_name'])
...
Eric Idle
John Cleese

>>> print(row)
{'first_name': 'John', 'last_name': 'Cleese'}
```

class csv.**DictWriter**(*f*, *fieldnames*, *restval=''*, *extrasaction='raise'*, *dialect='excel'*, **args*, ***kwds*)

Cria um objeto que funcione como um de escrita comum, mas mapeia dicionários nas linhas de saída. O parâmetro *fieldnames* é uma *sequência* de chaves que identificam a ordem na qual os valores no dicionário transmitidos para o método *writerow()* são escritos no arquivo *f*. O parâmetro opcional *restval* especifica o valor a ser escrito se o dicionário estiver com falta de uma chave em *fieldnames*. Se o dicionário transmitido para o método *writerow()* contiver uma chave não encontrada em *fieldnames*, o parâmetro opcional *extrasaction* indica qual ação executar. Se estiver definido como `'raise'`, o valor padrão, a *ValueError* é levantada. Se estiver definido como `'ignore'`, valores extras no dicionário serão ignorados. Quaisquer outros argumentos nomeados ou opcionais são passados para a instância subjacente de *writer*.

Observe que, diferentemente da classe *DictReader*, o parâmetro *fieldnames* da classe *DictWriter* não é opcional.

Se o argumento passado para *fieldnames* for um iterador, ele será convertido para uma *list*.

Um pequeno exemplo de uso:

```
import csv

with open('names.csv', 'w', newline='') as csvfile:
    fieldnames = ['first_name', 'last_name']
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

    writer.writeheader()
    writer.writerow({'first_name': 'Baked', 'last_name': 'Beans'})
    writer.writerow({'first_name': 'Lovely', 'last_name': 'Spam'})
    writer.writerow({'first_name': 'Wonderful', 'last_name': 'Spam'})
```

class csv.**Dialect**

A classe *Dialect* é uma classe de contêiner cujos atributos contêm informações sobre como lidar com aspas duplas, espaços em branco, delimitadores, etc. Devido à falta de uma especificação CSV estrita, diferentes aplicativos produzem dados CSV sutilmente diferentes. As instâncias de *Dialect* definem como as instâncias *reader* e *writer* se comportam.

Todos os nomes de *Dialect* disponíveis são retornados por *list_dialects()*, e eles podem ser registrados com classes *reader* e *writer* específicas através de suas funções inicializadoras (`__init__`) como esta:

```
import csv

with open('students.csv', 'w', newline='') as csvfile:
    writer = csv.writer(csvfile, dialect='unix')
```

class csv.**excel**

A classe *excel* define as propriedades usuais de um arquivo CSV gerado pelo Excel. Ele é registrado com o nome do dialeto `'excel'`.

class csv.**excel_tab**

A classe *excel_tab* define as propriedades usuais de um arquivo delimitado por TAB gerado pelo Excel. Ela é registrado com o nome do dialeto `'excel-tab'`.

class csv.**unix_dialect**

A classe *unix_dialect* define as propriedades usuais de um arquivo CSV gerado em sistemas UNIX, ou seja, usando `'\n'` como terminador de linha e citando todos os campos. É registrado com o nome do dialeto `'unix'`.

Adicionado na versão 3.2.

class csv.**Sniffer**

A classe *Sniffer* é usada para deduzir o formato de um arquivo CSV.

A classe *Sniffer* fornece dois métodos:

sniff(*sample*, *delimiters=None*)

Analisa a *sample* fornecida e retorna uma subclasse *Dialect*, refletindo os parâmetros encontrados. Se o parâmetro opcional *delimiters* for fornecido, ele será interpretado como uma string contendo possíveis caracteres válidos de delimitador.

has_header(*sample*)

Analisa o texto de amostra (presume-se que esteja no formato CSV) e retorne *True* se a primeira linha parecer uma série de cabeçalhos de coluna. Inspecionando cada coluna, um dos dois critérios principais será considerado para estimar se a amostra contém um cabeçalho:

- da segunda até a enésima linha contém valores numéricos
- da segunda até a enésima linha contém strings em que pelo menos o comprimento de um valor difere daquele do cabeçalho putativo dessa coluna.

Vinte linhas após a primeira linha são amostradas; se mais da metade das colunas + linhas atenderem aos critérios, *True* será retornado.

> **Nota**
>
> Este método é uma heurística aproximada e pode produzir falsos positivos e negativos.

Um exemplo para uso de *Sniffer*:

```
with open('example.csv', newline='') as csvfile:
    dialect = csv.Sniffer().sniff(csvfile.read(1024))
    csvfile.seek(0)
    reader = csv.reader(csvfile, dialect)
    # ... processa conteúdo do arquivo CSV aqui ...
```

O módulo *csv* define as seguintes constantes:

csv.**QUOTE_ALL**

Instrui objetos *writer* a colocar aspas em todos os campos.

csv.**QUOTE_MINIMAL**

Instrui objetos *writer* a colocar aspas apenas nos campos que contêm caracteres especiais como *delimiters*, *quotechar*, `'\r'`, `'\n'` ou qualquer um dos caracteres em *lineterminator*.

csv.**QUOTE_NONNUMERIC**

Instrui objetos *writer* a colocar aspas em todos os campos não numéricos.

Instrui objetos *reader* a converter todos os campos não envoltos por aspas no tipo *float*.

> **Nota**
>
> Alguns tipos numéricos, como *bool*, *Fraction* ou *IntEnum*, têm uma representação de string que não pode ser convertida para *float*. Eles não podem ser lidos nos modos *QUOTE_NONNUMERIC* e *QUOTE_STRINGS*.

csv.**QUOTE_NONE**

Instrui objetos *writer* a nunca colocar aspas nos campos. Quando o *delimiter*, *quotechar*, *escapechar*, `'\r'`, `'\n'` ou qualquer um dos caracteres em *lineterminator* ocorre nos dados de saída, é precedido pelo caractere *escapechar* atual. Se *escapeechar* não estiver definido, o escritor levantará *Error* se algum caractere que exija escape for encontrado. Defina *quotechar* para `None` para evitar seu escape.

Instrui objetos *reader* a não executar nenhum processamento especial de caracteres de aspas.

csv.**QUOTE_NOTNULL**

Instrui objetos *writer* a coloca entre aspas os campos que não são `None`. Isso é semelhante a *QUOTE_ALL*, exceto que se o valor de um campo for `None`, uma string vazia (sem aspas) é escrita.

Instrui objetos *reader* a interpretar um campo vazio (sem aspas) como `None` e a se comportar de outra forma como *QUOTE_ALL*.

Adicionado na versão 3.12.

csv.**QUOTE_STRINGS**

Instrui objetos *writer* a sempre coloca aspas em volta dos campos que são strings. Isso é semelhante a *QUOTE_NONNUMERIC*, exceto que se o valor de um campo for `None`, uma string vazia (sem aspas) é escrita.

Instrui objetos *reader* a interpretar uma string vazia (sem aspas) como `None` e a se comportar de outra forma como *QUOTE_NONNUMERIC*.

Adicionado na versão 3.12.

O módulo *csv* define a seguinte exceção:

exception csv.**Error**

Levantada por qualquer uma das funções quando um erro é detectado.

14.1.2 Dialetos e parâmetros de formatação

Para facilitar a especificação do formato dos registros de entrada e saída, parâmetros específicos de formatação são agrupados em dialetos. Um dialeto é uma subclasse da classe *Dialect* contendo vários atributos descrevendo o formato do arquivo CSV. Ao criar objetos *reader* ou *writer*, o programador pode especificar uma string ou uma subclasse da classe *Dialect* como parâmetro de dialeto. Além do parâmetro *dialect*, ou em vez do parâmetro *dialect*, o programador também pode especificar parâmetros de formatação individuais, com os mesmos nomes dos atributos definidos abaixo para a classe *Dialect*.

Os dialetos possuem suporte aos seguintes atributos:

Dialect.**delimiter**

Uma string de um caractere usada para separar campos. O padrão é `','`.

Dialect.**doublequote**

Controla como as instâncias de *quotechar* que aparecem dentro de um campo devem estar entre aspas. Quando *True*, o caractere é dobrado. Quando *False*, o *escapechar* é usado como um prefixo para o *quotechar*. O padrão é *True*.

Na saída, se *doublequote* for *False* e não *escapeechar* for definido, *Error* é levantada se um *quotechar* é encontrado em um campo.

Dialect.**escapechar**

Uma string de caracteres de um caractere usada pelo escritor para escapar caracteres que exigem escape:

- o *delimiter*, o *quotechar*, `'\r'`, `'\n'` e qualquer um dos caracteres em *lineterminator* são escapados se *quoting* estiver definido como *QUOTE_NONE*;
- o *quotechar* é escapado se *doublequote* for *False*;
- o *escapechar* em si.

Na leitura, o *escapechar* remove qualquer significado especial do caractere seguinte. O padrão é *None*, o que desabilita o escape.

Alterado na versão 3.11: Um *escapechar* vazio não é permitido.

Dialect.**lineterminator**

A string usada para terminar as linhas produzidas pelo *writer*. O padrão é `'\r\n'`.

> **Nota**
>
> A *reader* é codificada para reconhecer `'\r'` ou `'\n'` como fim de linha e ignora *lineterminator*. Esse comportamento pode mudar no futuro.

Dialect.**quotechar**

Uma string de um caractere usada para colocar entre aspas campos que contêm caracteres especiais, como o *delimiter* ou o *quotechar*, ou que contêm caracteres de nova linha (`'\r'`, `'\n'` ou qualquer um dos caracteres em *lineterminator*). O padrão é `'"'`. Pode ser definido como `None` para evitar o escape de `'"'` se *aspas* estiver definido como *QUOTE_NONE*.

Alterado na versão 3.11: Um *quotechar* vazio não é permitido.

Dialect.**quoting**

Controla quando as aspas devem ser geradas pelo escritor e reconhecidas pelo leitor. Ele pode assumir qualquer uma das constantes *QUOTE_* constants* e o padrão é *QUOTE_MINIMAL*. se *quotechar* não for `None` e, do contrário, *QUOTE_NONE*.

Dialect.**skipinitialspace**

Quando *True*, os espaços imediatamente após o *delimitador* são ignorados. O padrão é *False*. Ao combinar `delimiter=' '` com `skipinitialspace=True`, campos vazios sem aspas não são permitidos.

Dialect.**strict**

Quando `True`, levanta a exceção *Error* em uma entrada CSV ruim. O padrão é `False`.

14.1.3 Objetos Reader

Os objetos Reader (instâncias *DictReader* e objetos retornados pela função *reader()*) têm os seguintes métodos públicos:

csvreader.**__next__**()

Retorna a próxima linha do objeto iterável do leitor como uma lista (se o objeto foi retornado de `leitor()`) ou um dict (se for uma instância de *DictReader*), analisado de acordo com a *Dialect* atual. Normalmente, você deve chamar isso de `next(reader)`.

Os objetos Reader possuem os seguintes atributos públicos:

csvreader.**dialect**

Uma descrição somente leitura do dialeto em uso pelo analisador sintático.

csvreader.**line_num**

O número de linhas lidas no iterador de origem. Não é o mesmo que o número de registros retornados, pois os registros podem abranger várias linhas.

Os objetos DictReader têm o seguinte atributo público:

DictReader.**fieldnames**

Se não for passado como parâmetro ao criar o objeto, esse atributo será inicializado no primeiro acesso ou quando o primeiro registro for lido no arquivo.

14.1.4 Objetos Writer

Objetos *writer* (instâncias e objetos *DictWriter* retornados pela função *writer()*) possuem os seguintes métodos públicos. Uma *row* deve ser iterável de strings ou números para objetos *writer* e um dicionário mapeando nomes de campos para strings ou números (passando-os por *str()* primeiro) para *DictWriter*. Observe que números complexos são escritos cercados por parênteses. Isso pode causar alguns problemas para outros programas que leem arquivos CSV (supondo que eles aceitem números complexos).

csvwriter.**writerow**(*row*)

Escreve o parâmetro *row* no objeto arquivo do escritor, formatado de acordo com a *Dialect* atual. Retorna o valor de retorno da chamada ao método *write* do objeto arquivo subjacente.

Alterado na versão 3.5: Adicionado suporte a iteráveis arbitrários.

csvwriter.**writerows**(*rows*)

Escreve todos os elementos em *rows* (um iterável de objetos *row*, conforme descrito acima) no objeto arquivo do escritor, formatado de acordo com o dialeto atual.

Os objetos Writer têm o seguinte atributo público:

csvwriter.**dialect**

Uma descrição somente leitura do dialeto em uso pelo escritor.

Os objetos DictWriter têm o seguinte método público:

DictWriter.**writeheader**()

Escreve uma linha com os nomes dos campos (conforme especificado no construtor) no objeto arquivo do escritor, formatado de acordo com o dialeto atual. Retorna o valor de retorno da chamada *csvwriter.writerow()* usada internamente.

Adicionado na versão 3.2.

Alterado na versão 3.8: *writeheader()* agora também retorna o valor retornado pelo método *csvwriter.writerow()* que ele usa internamente.

14.1.5 Exemplos

O exemplo mais simples de leitura de um arquivo CSV:

```
import csv
with open('some.csv', newline='') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Lendo um arquivo com um formato alternativo:

```
import csv
with open('passwd', newline='') as f:
    reader = csv.reader(f, delimiter=':', quoting=csv.QUOTE_NONE)
    for row in reader:
        print(row)
```

O exemplo de escrita possível mais simples possível é:

```
import csv
with open('some.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(someiterable)
```

Como *open()* é usado para abrir um arquivo CSV para leitura, o arquivo será decodificado por padrão em unicode usando a codificação padrão do sistema (consulte *locale.getencoding()*). Para decodificar um arquivo usando uma codificação diferente, use o argumento `encoding` do open:

```
import csv
with open('some.csv', newline='', encoding='utf-8') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

O mesmo se aplica à escrita em algo diferente da codificação padrão do sistema: especifique o argumento de codificação ao abrir o arquivo de saída.

Registrando um novo dialeto:

```
import csv
csv.register_dialect('unixpwd', delimiter=':', quoting=csv.QUOTE_NONE)
with open('passwd', newline='') as f:
    reader = csv.reader(f, 'unixpwd')
```

Um uso um pouco mais avançado do leitor — capturando e relatando erros:

```
import csv, sys
filename = 'some.csv'
with open(filename, newline='') as f:
    reader = csv.reader(f)
    try:
        for row in reader:
            print(row)
    except csv.Error as e:
        sys.exit(f'file {filename}, line {reader.line_num}: {e}')
```

E embora o módulo não tenha suporte diretamente à análise sintática de strings, isso pode ser feito facilmente:

```
import csv
for row in csv.reader(['one,two,three']):
    print(row)
```

14.2 `configparser` — Analisador sintático de arquivo de configuração

Código-fonte: Lib/configparser.py

Este módulo fornece a classe *`ConfigParser`* que implementa uma linguagem de configuração básica que fornece uma estrutura semelhante à encontrada nos arquivos INI do Microsoft Windows. Você pode usar isso para escrever programas Python que podem ser facilmente personalizados pelos usuários finais.

Nota

Esta biblioteca *não* interpreta nem escreve os prefixos de tipo de valor usados na versão estendida do Registro do Windows da sintaxe INI.

Ver também

Módulo *`tomllib`*
TOML é um formato bem especificado para arquivos de configuração de aplicações. Ele foi projetado especificamente para ser uma versão melhorada do INI.

Módulo *`shlex`*
Suporte para criação de minilinguagens semelhantes a shell Unix que também podem ser usadas para arquivos de configuração de aplicações.

Módulo *`json`*
O módulo `json` implementa um subconjunto de sintaxe JavaScript que às vezes é usado para configuração, mas não suporta comentários.

14.2.1 Início rápido

Vamos pegar um arquivo de configuração bem básico parecido com este:

```
[DEFAULT]
ServerAliveInterval = 45
Compression = yes
CompressionLevel = 9
ForwardX11 = yes

[forge.example]
User = hg

[topsecret.server.example]
Port = 50022
ForwardX11 = no
```

A estrutura dos arquivos INI é descrita *na seção seguinte*. Essencialmente, o arquivo consiste em seções, cada uma contendo chaves com valores. As classes *`configparser`* podem ler e escrever tais arquivos. Vamos começar criando o arquivo de configuração acima programaticamente.

```
>>> import configparser
>>> config = configparser.ConfigParser()
>>> config['DEFAULT'] = {'ServerAliveInterval': '45',
...                      'Compression': 'yes',
...                      'CompressionLevel': '9'}
>>> config['forge.example'] = {}
>>> config['forge.example']['User'] = 'hg'
>>> config['topsecret.server.example'] = {}
>>> topsecret = config['topsecret.server.example']
>>> topsecret['Port'] = '50022'     # mutates the parser
>>> topsecret['ForwardX11'] = 'no'  # same here
>>> config['DEFAULT']['ForwardX11'] = 'yes'
>>> with open('example.ini', 'w') as configfile:
...   config.write(configfile)
...
```

Como você pode ver, podemos tratar um analisador sintático de configuração como um dicionário. Existem diferenças, *descritas posteriormente*, mas o comportamento é muito próximo do que você esperaria de um dicionário.

Agora que criamos e salvamos um arquivo de configuração, vamos lê-lo e explorar os dados que ele contém.

```
>>> config = configparser.ConfigParser()
>>> config.sections()
[]
>>> config.read('example.ini')
['example.ini']
>>> config.sections()
['forge.example', 'topsecret.server.example']
>>> 'forge.example' in config
True
>>> 'python.org' in config
False
>>> config['forge.example']['User']
'hg'
>>> config['DEFAULT']['Compression']
'yes'
>>> topsecret = config['topsecret.server.example']
>>> topsecret['ForwardX11']
'no'
>>> topsecret['Port']
'50022'
>>> for key in config['forge.example']:
...     print(key)
user
compressionlevel
serveraliveinterval
compression
forwardx11
>>> config['forge.example']['ForwardX11']
'yes'
```

Como podemos ver acima, a API é bastante simples. A única mágica envolve a seção `DEFAULT` que fornece valores padrão para todas as outras seções[1]. Observe também que as chaves nas seções não diferenciam maiúsculas de minúsculas e são armazenadas em letras minúsculas[1].

É possível ler diversas configurações para um único *`ConfigParser`*, onde a configuração adicionada mais recentemente terá a maior prioridade. Para quaisquer chaves repetidas serão usados os valores da configuração mais

[1] Os analisadores sintáticos de configuração permitem muita personalização. Caso você tenha interesse em alterar o comportamento descrito na referência da nota de rodapé, consulte a seção *Personalizando o comportamento do analisador sintático*.

recente, enquanto os valores das chaves anteriores serão ignorados. O exemplo a seguir lê um arquivo chamado `override.ini`, que irá substituir chaves repetidas do arquivo `example.ini`.

```
[DEFAULT]
ServerAliveInterval = -1
```

```
>>> config_override = configparser.ConfigParser()
>>> config_override['DEFAULT'] = {'ServerAliveInterval': '-1'}
>>> with open('override.ini', 'w') as configfile:
...     config_override.write(configfile)
...
>>> config_override = configparser.ConfigParser()
>>> config_override.read(['example.ini', 'override.ini'])
['example.ini', 'override.ini']
>>> print(config_override.get('DEFAULT', 'ServerAliveInterval'))
-1
```

Este comportamento é equivalente a uma chamada *ConfigParser.read()* com vários arquivos passados para o parâmetro *filenames*.

14.2.2 Tipos de dados suportados

Os analisadores sintáticos de configuração não adivinham os tipos de dados dos valores nos arquivos de configuração, sempre os armazenando internamente como strings. Isso significa que se você precisar de outros tipos de dados, deverá converter por conta própria:

```
>>> int(topsecret['Port'])
50022
>>> float(topsecret['CompressionLevel'])
9.0
```

Como essa tarefa é tão comum, os analisadores sintáticos de configuração fornecem uma variedade de métodos getter úteis para manipular com números inteiros, pontos flutuantes e booleanos. O último é o mais interessante porque simplesmente passar o valor para `bool()` não adiantaria nada já que `bool('False')` ainda é `True`. É por isso que os analisadores sintáticos de configuração também fornecem *getboolean()*. Este método não diferencia maiúsculas de minúsculas e reconhece valores booleanos de `'yes'`/`'no'`, `'on'`/`'off'`, `'true'`/`'false'` e `'1'`/`'0'`[Página 668, 1]. Por exemplo:

```
>>> topsecret.getboolean('ForwardX11')
False
>>> config['forge.example'].getboolean('ForwardX11')
True
>>> config.getboolean('forge.example', 'Compression')
True
```

Além de *getboolean()*, os analisadores sintáticos de configuração também fornecem métodos *getint()* e *getfloat()* equivalentes. Você pode registrar seus próprios conversores e personalizar os fornecidos.[Página 668, 1]

14.2.3 Valores reservas

Assim como acontece com um dicionário, você pode usar o método *get()* de uma seção para fornecer valores reservas (fallback):

```
>>> topsecret.get('Port')
'50022'
>>> topsecret.get('CompressionLevel')
'9'
>>> topsecret.get('Cipher')
```

(continua na próxima página)

(continuação da página anterior)

```
>>> topsecret.get('Cipher', '3des-cbc')
'3des-cbc'
```

Observe que os valores padrão têm precedência sobre os valores substitutos. Por exemplo, em nosso exemplo a chave `'CompressionLevel'` foi especificada apenas na seção `'DEFAULT'`. Se tentarmos obtê-lo na seção `'topsecret.server.example'`, sempre obteremos o padrão, mesmo se especificarmos um substituto:

```
>>> topsecret.get('CompressionLevel', '3')
'9'
```

Mais uma coisa a ter em conta é que o método *get()* no nível do analisador fornece uma interface personalizada e mais complexa, mantida para compatibilidade com versões anteriores. Ao usar este método, um valor substituto pode ser fornecido através do argumento somente-nomeado `fallback`:

```
>>> config.get('forge.example', 'monster',
...            fallback='No such things as monsters')
'No such things as monsters'
```

O mesmo argumento `fallback` pode ser usado com os métodos *getint()*, *getfloat()* e *getboolean()*, por exemplo:

```
>>> 'BatchMode' in topsecret
False
>>> topsecret.getboolean('BatchMode', fallback=True)
True
>>> config['DEFAULT']['BatchMode'] = 'no'
>>> topsecret.getboolean('BatchMode', fallback=True)
False
```

14.2.4 Estrutura dos arquivos INI

Um arquivo de configuração consiste em seções, cada uma liderada por um cabeçalho `[section]`, seguido por entradas de chave/valor separadas por uma string específica (= ou : por padrão[Página 668, 1]) . Por padrão, os nomes das seções diferenciam maiúsculas de minúsculas, mas as chaves não[Página 668, 1]. Os espaços em branco à esquerda e à direita são removidos das chaves e dos valores. Os valores podem ser omitidos se o analisador sintático estiver configurado para permitir[Página 668, 1], caso em que o delimitador chave/valor também pode ser omitido. Os valores também podem abranger várias linhas, desde que sejam indentados de forma mais profunda que a primeira linha do valor. Dependendo do modo do analisador sintático, as linhas em branco podem ser tratadas como partes de valores multilinhas ou ignoradas.

Por padrão, um nome de seção válido pode ser qualquer string que não contenha '\n'. Para alterar isso, consulte *ConfigParser.SECTCRE*.

O nome da primeira seção pode ser omitido se o analisador sintático estiver configurado para permitir uma seção de nível superior sem nome com `allow_unnamed_section=True`. Neste caso, as chaves/valores podem ser recuperadas por *UNNAMED_SECTION* como em `config[UNNAMED_SECTION]`.

Os arquivos de configuração podem incluir comentários, prefixados por caracteres específicos (`#` e `;` por padrão[Página 668, 1]). Os comentários podem aparecer sozinhos em uma linha vazia, possivelmente indentados.[Página 668, 1]

Por exemplo:

```
[Valores Simples]
chave=valor
espaços nas chaves=permitidos
espaços nos valores=permitidos também
espaços em volta do delimitador = obviamente
você também pode usar : para delimitar chaves de valores
```

(continua na próxima página)

(continuação da página anterior)

```
[Todos Valores São Strings]
valores como este: 1000000
ou este: 3.14159265359
são tratados como números? : não
inteiros, pontos flutuantes e booleanos são considerados: strings
pode usar a API para obter valores convertidos diretamente: verdadeiro

[Valores multilinha]
chorus: Eu sou um lenhador e estou bem
    Durmo a noite toda e trabalho o dia inteiro

[Sem valores]
chave_sem_valor
valor string vazia aqui =

[Você pode usar comentários]
# como este
; ou este

# Por padrão, apenas em uma linha vazia.
# Comentários em linha podem ser prejudiciais porque eles
# impedem os usuários de usar os caracteres delimitadores
# como partes dos valores.
# Isto posto, isso pode ser personalizado.

    [Seções Podem Ser Indentadas]
        podem_ser_valores_também = True
        isso_significa_algo_especial = False
        propósito = formatação para legibilidade
        valores_multilinha = são
            tratados sem problemas desde
            que eles estejam indentados
            mais profundamente do que a
            primeira linha de um valor
        # Mencionamos que também podemos indentar comentários?
```

14.2.5 Seções não nomeadas

O nome da primeira seção (ou único) pode ser omitido e os valores recuperados pelo atributo *UNNAMED_SECTION*.

```
>>> config = """
... option = value
...
... [  Section 2  ]
... another = val
... """
>>> unnamed = configparser.ConfigParser(allow_unnamed_section=True)
>>> unnamed.read_string(config)
>>> unnamed.get(configparser.UNNAMED_SECTION, 'option')
'value'
```

14.2.6 Interpolação de valores

Além da funcionalidade principal, *ConfigParser* oferece suporte a interpolação. Isso significa que os valores podem ser pré-processados antes de retorná-los das chamadas `get()`.

class configparser.**BasicInterpolation**

A implementação padrão usada por *ConfigParser*. Ele permite que os valores contenham strings de formato que se referem a outros valores na mesma seção ou valores na seção padrão especial[Página 668, 1]. Valores padrão adicionais podem ser fornecidos na inicialização.

Por exemplo:

```
[Paths]
home_dir: /Users
my_dir: %(home_dir)s/lumberjack
my_pictures: %(my_dir)s/Pictures

[Escape]
# use um %% para escapar o sinal de % (% é o único caractere que precisa ser␣
↪escapado):
gain: 80%%
```

No exemplo acima, *ConfigParser* com *interpolation* definido como `BasicInterpolation()` resolveria `%(home_dir)s` para o valor de `home_dir` (`/Users` neste caso). `%(my_dir)s` na verdade resolveria para `/Users/lumberjack`. Todas as interpolações são feitas sob demanda, portanto as chaves usadas na cadeia de referências não precisam ser especificadas em nenhuma ordem específica no arquivo de configuração.

Com `interpolation` definido como `None`, o analisador sintático simplesmente retornaria `%(my_dir)s/Pictures` como o valor de `my_pictures` e `%(home_dir)s/lumberjack` como o valor de `my_dir`.

class configparser.**ExtendedInterpolation**

Um manipulador alternativo para interpolação que implementa uma sintaxe mais avançada, usada, por exemplo, em `zc.buildout`. A interpolação estendida usa `${section:option}` para denotar um valor de uma seção estrangeira. A interpolação pode abranger vários níveis. Por conveniência, se a parte `section:` for omitida, a interpolação será padronizada para a seção atual (e possivelmente para os valores padrão da seção especial).

Por exemplo, a configuração especificada acima com interpolação básica ficaria assim com interpolação estendida:

```
[Paths]
home_dir: /Users
my_dir: ${home_dir}/lumberjack
my_pictures: ${my_dir}/Pictures

[Escape]
# use a $$ para escapar o sinal de $ ($ é o único caractere que precisa ser␣
↪escapado):
cost: $$80
```

Valores de outras seções também podem ser obtidos:

```
[Common]
home_dir: /Users
library_dir: /Library
system_dir: /System
macports_dir: /opt/local

[Frameworks]
Python: 3.2
```

(continua na próxima página)

(continuação da página anterior)

```
path: ${Common:system_dir}/Library/Frameworks/

[Arthur]
nickname: Two Sheds
last_name: Jackson
my_dir: ${Common:home_dir}/twosheds
my_pictures: ${my_dir}/Pictures
python_dir: ${Frameworks:path}/Python/Versions/${Frameworks:Python}
```

14.2.7 Acesso através do protocolo de mapeamento

Adicionado na versão 3.2.

Acesso através do protocolo de mapeamento é um nome genérico para funcionalidade que permite usar objetos personalizados como se fossem dicionários. No caso de *configparser*, a implementação da interface de mapeamento está usando a notação `parser['section']['option']`.

`parser['section']` em particular retorna um intermediário para os dados da seção no analisador sintático. Isso significa que os valores não são copiados, mas obtidos do analisador sintático original sob demanda. O que é ainda mais importante é que quando os valores são alterados em um intermediário de seção, eles são, na verdade, modificados no analisador sintático original.

Os objetos *configparser* se comportam o mais próximo possível dos dicionários reais. A interface de mapeamento é completa e segue a ABC *MutableMapping*. No entanto, existem algumas diferenças que devem ser levadas em consideração:

- Por padrão, todas as chaves nas seções são acessíveis sem distinção entre maiúsculas e minúsculas[Página 668, 1]. Por exemplo. `for option in parser["section"]` produz apenas nomes de chaves de opção `optionxform`ada. Isso significa chaves em letras minúsculas por padrão. Ao mesmo tempo, para uma seção que contém a chave `'a'`, ambas as expressões retornam `True`:

  ```
  "a" in parser["section"]
  "A" in parser["section"]
  ```

- Todas as seções também incluem valores `DEFAULTSECT`, o que significa que `.clear()` em uma seção não pode deixá-la visivelmente vazia. Isso ocorre porque os valores padrão não podem ser excluídos da seção (porque tecnicamente eles não estão lá). Se eles forem substituídos na seção, a exclusão fará com que o valor padrão fique visível novamente. Tentar excluir um valor padrão causa um *KeyError*.

- `DEFAULTSECT` não pode ser removido do analisador sintático:

  - tentar excluí-lo levanta *ValueError*,
  - `parser.clear()` deixa-o intacto,
  - `parser.popitem()` nunca o retorna.

- `parser.get(section, option, **kwargs)` - o segundo argumento **não** é um valor substituto. Observe, entretanto, que os métodos `get()` em nível de seção são compatíveis tanto com o protocolo de mapeamento quanto com a API clássica do configparser.

- `parser.items()` é compatível com o protocolo de mapeamento (retorna uma lista de pares *section_name*, *section_proxy* incluindo o DEFAULTSECT). Entretanto, este método também pode ser invocado com argumentos: `parser.items(section, raw, vars)`. A última chamada retorna uma lista de pares *option*, *value* para uma `section` especificada, com todas as interpolações expandidas (a menos que `raw=True` seja fornecido).

O protocolo de mapeamento é implementado sobre a API legada existente para que as subclasses que substituem a interface original ainda tenham mapeamentos funcionando conforme o esperado.

14.2.8 Personalizando o comportamento do analisador sintático

Existem quase tantas variantes de formato INI quanto aplicações que o utilizam. *configparser* percorre um longo caminho para fornecer suporte para o maior conjunto sensato de estilos INI disponíveis. A funcionalidade padrão é determinada principalmente pelo histórico e é muito provável que você queira personalizar alguns dos recursos.

A maneira mais comum de alterar a forma como um analisador sintático de configuração específico funciona é usar as opções `__init__()`:

- *defaults*, valor padrão: `None`

  Esta opção aceita um dicionário de pares chave-valor que será inicialmente colocado na seção `DEFAULT`. Isso é uma maneira elegante de oferecer suporte a arquivos de configuração concisos que não especificam valores iguais ao padrão documentado.

  Dica: se você deseja especificar valores padrão para uma seção específica, use *read_dict()* antes de ler o arquivo real.

- *dict_type*, valor padrão: *dict*

  Esta opção tem um grande impacto no comportamento do protocolo de mapeamento e na aparência dos arquivos de configuração gravados. Com o dicionário padrão, cada seção é armazenada na ordem em que foram adicionadas ao analisador sintático. O mesmo vale para opções dentro das seções.

  Um tipo de dicionário alternativo pode ser usado, por exemplo, para ordenar as seções e opções ao fazer *write-back*.

  Observação: existem maneiras de adicionar um conjunto de pares de valores-chave em uma única operação. Quando você usa um dicionário regular nessas operações, a classificação das chaves será ordenada. Por exemplo:

```
>>> parser = configparser.ConfigParser()
>>> parser.read_dict({'seção1': {'chave1': 'valor1',
...                              'chave2': 'valor2',
...                              'chave3': 'valor3'},
...                   'seção2': {'chaveA': 'valorA',
...                              'chaveB': 'valorB',
...                              'chaveC': 'valorC'},
...                   'seção3': {'foo': 'x',
...                              'bar': 'y',
...                              'baz': 'z'}
... })
>>> parser.sections()
['seção1', 'seção2', 'seção3']
>>> [option for option in parser['seção3']]
['foo', 'bar', 'baz']
```

- *allow_no_value*, valor padrão: `False`

  Alguns arquivos de configuração são conhecidos por incluir configurações sem valores, mas que de outra forma estão em conformidade com a sintaxe suportada por *configparser*. O parâmetro *allow_no_value* para o construtor pode ser usado para indicar que tais valores devem ser aceitos:

```
>>> import configparser

>>> sample_config = """
... [mysqld]
...   user = mysql
...   pid-file = /var/run/mysqld/mysqld.pid
...   skip-external-locking
...   old_passwords = 1
...   skip-bdb
...   # não precisamos de ACID hoje
```

(continua na próxima página)

(continuação da página anterior)

```
...   skip-innodb
... """
>>> config = configparser.ConfigParser(allow_no_value=True)
>>> config.read_string(sample_config)

>>> # Configurações com valores são tratados como antes:
>>> config["mysqld"]["user"]
'mysql'

>>> # Configurações sem valores fornecem None:
>>> config["mysqld"]["skip-bdb"]

>>> # Configurações que não são especificadas ainda levantam um erro:
>>> config["mysqld"]["não-existe"]
Traceback (most recent call last):
  ...
KeyError: 'não-existe'
```

- *delimiters*, valor padrão: `('=', ':')`

  Delimitadores são substrings que delimitam chaves de valores dentro de uma seção. A primeira ocorrência de uma substring delimitadora em uma linha é considerada um delimitador. Isso significa que os valores (mas não as chaves) podem conter os delimitadores.

  Veja também o argumento *space_around_delimiters* para *`ConfigParser.write()`*.

- *comment_prefixes*, valor padrão: `('#', ';')`

- *inline_comment_prefixes*, valor padrão: `None`

  Prefixos de comentários são strings que indicam o início de um comentário válido em um arquivo de configuração. *comment_prefixes* são usados apenas em linhas vazias (opcionalmente indentadas), enquanto *inline_comment_prefixes* pode ser usado após cada valor válido (por exemplo, nomes de seções, opções e linhas vazias também). Por padrão, os comentários embutidos estão desabilitados e `'#'` e `';'` são usados como prefixos para comentários de linha inteira.

  Alterado na versão 3.2: Nas versões anteriores do *`configparser`*, o comportamento correspondia a `comment_prefixes=('#',';')` e `inline_comment_prefixes=(';',)`.

  Observe que os analisadores sintáticos de configuração não oferecem suporte a escape de prefixos de comentários, portanto, usar *inline_comment_prefixes* pode impedir que os usuários especifiquem valores de opção com caracteres usados como prefixos de comentários. Em caso de dúvida, evite definir *inline_comment_prefixes*. Em qualquer circunstância, a única maneira de armazenar caracteres de prefixo de comentário no início de uma linha em valores multilinha é interpolar o prefixo, por exemplo:

```
>>> from configparser import ConfigParser, ExtendedInterpolation
>>> parser = ConfigParser(interpolation=ExtendedInterpolation())
>>> # the default BasicInterpolation could be used as well
>>> parser.read_string("""
... [DEFAULT]
... hash = #
...
... [hashes]
... shebang =
...   ${hash}!/usr/bin/env python
...   ${hash} -*- coding: utf-8 -*-
...
... extensions =
...   enabled_extension
...   another_extension
```

(continua na próxima página)

(continuação da página anterior)

```
...   #disabled_by_comment
...   yet_another_extension
...
... interpolation not necessary = if # is not at line start
... even in multiline values = line #1
...   line #2
...   line #3
... """)
>>> print(parser['hashes']['shebang'])

#!/usr/bin/env python
# -*- coding: utf-8 -*-
>>> print(parser['hashes']['extensions'])

enabled_extension
another_extension
yet_another_extension
>>> print(parser['hashes']['interpolation not necessary'])
if # is not at line start
>>> print(parser['hashes']['even in multiline values'])
line #1
line #2
line #3
```

- *strict*, valor padrão: `True`

  Ao definir como `True`, o analisador sintático não permitirá nenhuma seção ou opção duplicada durante a leitura de uma única fonte (usando *read_file()*, *read_string()* ou *read_dict()*). Recomenda-se usar analisadores sintáticos estritos em novas aplicações.

  Alterado na versão 3.2: Nas versões anteriores do *configparser*, o comportamento correspondia a `strict=False`.

- *empty_lines_in_values*, valor padrão: `True`

  Em analisadores sintáticos de configuração, os valores podem abranger várias linhas, desde que sejam mais indentados do que a chave que os contém. Por padrão, os analisadores sintáticos também permitem que linhas vazias façam parte de valores. Ao mesmo tempo, as chaves podem ser indentados arbitrariamente para melhorar a legibilidade. Consequentemente, quando os arquivos de configuração ficam grandes e complexos, é fácil para o usuário perder o controle da estrutura do arquivo. Tomemos por exemplo:

  ```
  [Seção]
  chave = multilinha
    valor com uma pegadinha

   esta = ainda é parte do valor multilinha de 'chave'
  ```

  Isso pode ser especialmente problemático para o usuário ver se está usando uma fonte proporcional para editar o arquivo. É por isso que quando sua aplicação não precisa de valores com linhas vazias, você deve considerar proibi-los. Isso fará com que as linhas vazias dividam as chaves sempre. No exemplo acima, seriam produzidas duas chaves, `key` e `this`.

- *default_section*, valor padrão: `configparser.DEFAULTSECT` (isto é: `"DEFAULT"`)

  A convenção de permitir uma seção especial de valores padrão para outras seções ou fins de interpolação é um conceito poderoso desta biblioteca, permitindo aos usuários criar configurações declarativas complexas. Esta seção normalmente é chamada de `"DEFAULT"`, mas pode ser personalizada para apontar para qualquer outro nome de seção válido. Alguns valores típicos incluem: `"general"` ou `"common"`. O nome fornecido é usado para reconhecer seções padrão ao ler de qualquer fonte e é usado ao gravar a configuração em um arquivo.

Seu valor atual pode ser recuperado usando o atributo `parser_instance.default_section` e pode ser modificado em tempo de execução (ou seja, para converter arquivos de um formato para outro).

- *interpolation*, valor padrão: `configparser.BasicInterpolation`

  O comportamento de interpolação pode ser personalizado fornecendo um manipulador personalizado por meio do argumento *interpolation*. `None` pode ser usado para desligar completamente a interpolação, `ExtendedInterpolation()` fornece uma variante mais avançada inspirada em `zc.buildout`. Mais sobre o assunto na *seção de documentação dedicada*. *`RawConfigParser`* tem um valor padrão de `None`.

- *converters*, valor padrão: não definido

  Os analisadores sintáticos de configuração fornecem getters de valor de opção que realizam conversão de tipo. Por padrão, *`getint()`*, *`getfloat()`* e *`getboolean()`* são implementados. Caso outros getters sejam desejáveis, os usuários podem defini-los em uma subclasse ou passar um dicionário onde cada chave é um nome do conversor e cada valor é um chamável que implementa a referida conversão. Por exemplo, passar `{'decimal': decimal.Decimal}` adicionaria `getdecimal()` no objeto analisador sintático e em todos os proxies de seção. Em outras palavras, será possível escrever `parser_instance.getdecimal('section', 'key', fallback=0)` e `parser_instance['section'].getdecimal('key', 0)`.

  Se o conversor precisar acessar o estado do analisador sintático, ele poderá ser implementado como um método em uma subclasse do analisador sintático de configuração. Se o nome deste método começar com `get`, ele estará disponível em todos os intermediários de seção, na forma compatível com dict (veja o exemplo `getdecimal()` acima).

Uma personalização mais avançada pode ser obtida substituindo os valores padrão desses atributos do analisador sintático. Os padrões são definidos nas classes, portanto podem ser substituídos por subclasses ou por atribuição de atributos.

`ConfigParser.`**`BOOLEAN_STATES`**

Por padrão, ao usar *`getboolean()`*, os analisadores sintáticos de configuração consideram os seguintes valores `True`: `'1'`, `'yes'`, `'true'`, `'on'` e os seguintes valores `False`: `'0'`, `'no'`, `'false'`, `'off'`. Você pode substituir isso especificando um dicionário personalizado de strings e seus resultados booleanos. Por exemplo:

```
>>> custom = configparser.ConfigParser()
>>> custom['seção1'] = {'funky': 'nope'}
>>> custom['seção1'].getboolean('funky')
Traceback (most recent call last):
...
ValueError: Not a boolean: nope
>>> custom.BOOLEAN_STATES = {'sure': True, 'nope': False}
>>> custom['seção1'].getboolean('funky')
False
```

Outros pares booleanos típicos incluem `accept`/`reject` ou `enabled`/`disabled`.

`ConfigParser.`**`optionxform`**(*option*)

Este método transforma nomes de opções em cada operação de leitura, obtenção ou definição. O padrão converte o nome em letras minúsculas. Isso também significa que quando um arquivo de configuração for gravado, todas as chaves estarão em letras minúsculas. Substitua esse método se for inadequado. Por exemplo:

```
>>> config = """
... [Seção1]
... Chave = Valor
...
... [Seção2]
... OutraChave = Valor
... """
>>> typical = configparser.ConfigParser()
>>> typical.read_string(config)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> list(typical['Seção1'].keys())
['chave']
>>> list(typical['Seção2'].keys())
['outrachave']
>>> custom = configparser.RawConfigParser()
>>> custom.optionxform = lambda option: option
>>> custom.read_string(config)
>>> list(custom['Seção1'].keys())
['Chave']
>>> list(custom['Seção2'].keys())
['OutraChave']
```

Nota

A função optionxform transforma nomes de opções em um formato canônico. Esta deve ser uma função idempotente: se o nome já estiver na forma canônica, deverá ser retornado inalterado.

`ConfigParser.`**`SECTCRE`**

Uma expressão regular compilada usada para analisar cabeçalhos de seção. O padrão corresponde a `[section]` para o nome `"section"`. O espaço em branco é considerado parte do nome da seção, portanto `[ larch ]` será lido como uma seção de nome `" larch "`. Substitua esse atributo se for inadequado. Por exemplo:

```
>>> import re
>>> config = """
... [Seção 1]
... opção = valor
...
... [  Seção 2  ]
... outro = valor
... """
>>> typical = configparser.ConfigParser()
>>> typical.read_string(config)
>>> typical.sections()
['Seção 1', '  Seção 2  ']
>>> custom = configparser.ConfigParser()
>>> custom.SECTCRE = re.compile(r"\[ *(?P<header>[^]]+?) *\]")
>>> custom.read_string(config)
>>> custom.sections()
['Seção 1', 'Seção 2']
```

Nota

Embora os objetos ConfigParser também usem um atributo `OPTCRE` para reconhecer linhas de opção, não é recomendado substituí-lo porque isso interferiria nas opções do construtor *allow_no_value* e *delimiters*.

14.2.9 Exemplos de APIs legadas

Principalmente por questões de compatibilidade com versões anteriores, *configparser* fornece também uma API legada com métodos `get`/`set` explícitos. Embora existam casos de uso válidos para os métodos descritos abaixo, o acesso ao protocolo de mapeamento é preferido para novos projetos. A API legada é às vezes mais avançada, de baixo nível e totalmente contraintuitiva.

Um exemplo de escrita em um arquivo de configuração:

```
import configparser

config = configparser.RawConfigParser()

# Observe que usando as funções set do RawConfigParser, você pode atribuir
# valores não string a chaves internamente, mas receberá um erro ao tentar
# gravar em um arquivo ou quando obtê-lo em modo não bruta. Definir valores
# usando o protocolo de mapeamento ou set() do ConfigParser não permite que
# tais atribuições ocorram.
config.add_section('Seção1')
config.set('Seção1', 'an_int', '15')
config.set('Seção1', 'a_bool', 'true')
config.set('Seção1', 'a_float', '3.1415')
config.set('Seção1', 'baz', 'divertido')
config.set('Seção1', 'bar', 'Python')
config.set('Seção1', 'foo', '%(bar)s é %(baz)s!')

# Escrevendo nosso arquivo de configuração em 'example.cfg'
with open('example.cfg', 'w') as configfile:
    config.write(configfile)
```

Um exemplo de leitura do arquivo de configuração novamente:

```
import configparser

config = configparser.RawConfigParser()
config.read('example.cfg')

# getfloat() levanta uma exceção se o valor não for um ponto flutuante
# getint() e getboolean() também fazem isso para seus respectivos tipos
a_float = config.getfloat('Seção1', 'a_float')
an_int = config.getint('Seção1', 'an_int')
print(a_float + an_int)

# Note que a próxima saída não interpola '%(bar)s' ou '%(baz)s'.
# Isso ocorre porque estamos usando RawConfigParser().
if config.getboolean('Seção1', 'a_bool'):
    print(config.get('Seção1', 'foo'))
```

Para obter interpolação, use *ConfigParser*:

```
import configparser

cfg = configparser.ConfigParser()
cfg.read('example.cfg')

# Defina o argumento *raw* opcional de get() para True se você quiser desabilitar
# interpolação em uma única operação de get.
print(cfg.get('Seção1', 'foo', raw=False))  # -> "Python é divertido!"
print(cfg.get('Seção1', 'foo', raw=True))   # -> "%(bar)s é %(baz)s!"

# O argumento opcional *vars* é um dicionário com membros que vão aceitar
# precedência na interpolação.
print(cfg.get('Seção1', 'foo', vars={'bar': 'Documentação',
                                     'baz': 'maligna'}))

# The optional *fallback* argument can be used to provide a fallback value
```

(continua na próxima página)

(continuação da página anterior)

```
print(cfg.get('Seção1', 'foo'))
      # -> "Python é divertido!"

print(cfg.get('Seção1', 'foo', fallback='Monty não é.'))
      # -> "Python é divertido!"

print(cfg.get('Seção1', 'monstro', fallback='Não existem monstros.'))
      # -> "Não existem monstros."

# Um print(cfg.get('Seção1', 'monster')) levantaria NoOptionError,
# mas nós também podemos usar:

print(cfg.get('Seção1', 'mosntro', fallback=None))
      # -> None
```

Os valores padrão estão disponíveis em ambos os tipos de ConfigParsers. Eles são usados em interpolação se uma opção usada não estiver definida em outro lugar.

```
import configparser

# Nova instância com 'bar' e 'baz' padrão para 'A vida' e 'difícil' cada
config = configparser.ConfigParser({'bar': 'A vida', 'baz': 'difícil'})
config.read('example.cfg')

print(config.get('Seção1', 'foo'))      # -> "Python é divertido!"
config.remove_option('Seção1', 'bar')
config.remove_option('Seção1', 'baz')
print(config.get('Seção1', 'foo'))      # -> "A vida é difícil!"
```

14.2.10 Objetos ConfigParser

class configparser.**ConfigParser**(*defaults=None*, *dict_type=dict*, *allow_no_value=False*, ***, *delimiters=('=', ':')*, *comment_prefixes=('#', ';')*, *inline_comment_prefixes=None*, *strict=True*, *empty_lines_in_values=True*, *default_section=configparser.DEFAULTSECT*, *interpolation=BasicInterpolation()*, *converters={}*, *allow_unnamed_section=False*)

O principal analisador sintático de configuração. Quando *defaults* é fornecido, ele é inicializado no dicionário de padrões intrínsecos. Quando *dict_type* for fornecido, ele será usado para criar os objetos dicionário para a lista de seções, para as opções dentro de uma seção e para os valores padrão.

Quando *delimiters* são fornecidos, eles são usados como o conjunto de substrings que dividem chaves de valores. Quando *comment_prefixes* for fornecido, ele será usado como o conjunto de substrings que prefixam comentários em linhas vazias. Os comentários podem ser indentados. Quando *inline_comment_prefixes* for fornecido, ele será usado como o conjunto de substrings que prefixam comentários em linhas não vazias.

Quando *strict* for `True` (o padrão), o analisador sintático não permitirá nenhuma seção ou opção duplicada durante a leitura de uma única fonte (arquivo, string ou dicionário), levantando *DuplicateSectionError* ou *DuplicateOptionError*. Quando *empty_lines_in_values* é `False` (padrão: `True`), cada linha vazia marca o fim de uma opção. Caso contrário, as linhas vazias internas de uma opção multilinha serão mantidas como parte do valor. Quando *allow_no_value* for `True` (padrão: `False`), opções sem valores serão aceitas; o valor mantido para estes é `None` e eles são serializados sem o delimitador final.

Quando *default_section* é fornecido, ele especifica o nome da seção especial que contém valores padrão para outras seções e propósitos de interpolação (normalmente chamada de `"DEFAULT"`). Este valor pode ser recuperado e alterado em tempo de execução usando o atributo de instância `default_section`. Isso não

reavaliará um arquivo de configuração já analisado, mas será usado ao escrever configurações analisadas em um novo arquivo de configuração.

O comportamento de interpolação pode ser personalizado fornecendo um manipulador personalizado por meio do argumento *interpolation*. `None` pode ser usado para desligar completamente a interpolação, `ExtendedInterpolation()` fornece uma variante mais avançada inspirada em `zc.buildout`. Mais sobre o assunto na *seção de documentação dedicada*.

Todos os nomes de opções usados na interpolação serão passados através do método *optionxform()* assim como qualquer outra referência de nome de opção. Por exemplo, usando a implementação padrão de *optionxform()* (que converte nomes de opções para letras minúsculas), os valores `foo %(bar)s` e `foo %(BAR)s` são equivalentes.

Quando *converters* é fornecido, deve ser um dicionário onde cada chave representa o nome de um conversor de tipo e cada valor é um chamável implementando a conversão de string para o tipo de dados desejado. Cada conversor obtém seu próprio método `get*()` correspondente no objeto analisador sintático e nos intermediários de seção.

Quando *allow_unnamed_section* é `True` (padrão: `False`), o primeiro nome da seção pode ser omitido. Veja a seção *"Seções não nomeadas"*.

É possível ler diversas configurações para um único *ConfigParser*, onde a configuração adicionada mais recentemente terá a maior prioridade. Para quaisquer chaves repetidas serão usados os valores da configuração mais recente, enquanto os valores das chaves anteriores serão ignorados. O exemplo a seguir lê um arquivo chamado `override.ini`, que irá substituir chaves repetidas do arquivo `example.ini`.

```
[DEFAULT]
ServerAliveInterval = -1
```

```
>>> config_override = configparser.ConfigParser()
>>> config_override['DEFAULT'] = {'ServerAliveInterval': '-1'}
>>> with open('override.ini', 'w') as configfile:
...     config_override.write(configfile)
...
>>> config_override = configparser.ConfigParser()
>>> config_override.read(['example.ini', 'override.ini'])
['example.ini', 'override.ini']
>>> print(config_override.get('DEFAULT', 'ServerAliveInterval'))
-1
```

Alterado na versão 3.1: O padrão *dict_type* é *collections.OrderedDict*.

Alterado na versão 3.2: *allow_no_value*, *delimiters*, *comment_prefixes*, *strict*, *empty_lines_in_values*, *default_section* e *interpolation* foram adicionados.

Alterado na versão 3.5: O argumento *converters* foi adicionado.

Alterado na versão 3.7: O argumento *defaults* é lido com *read_dict()*, fornecendo um comportamento consistente em todo o analisador: chaves e valores que não são de string são convertidos implicitamente em strings.

Alterado na versão 3.8: O *dict_type* padrão é *dict*, pois agora preserva a ordem de inserção.

Alterado na versão 3.13: Levanta *MultilineContinuationError* quando *allow_no_value* for `True`, e uma chave sem valor continua com uma linha indentada.

Alterado na versão 3.13: O argumento *allow_unnamed_section* foi adicionado.

defaults()

Retorna um dicionário contendo os padrões de toda a instância.

sections()

Retorna uma lista das seções disponíveis; a *seção padrão* não está incluída na lista.

add_section(*section*)

Adiciona uma seção de nome *section* à instância. Se já existir uma seção com o nome fornecido, `DuplicateSectionError` será levantada. Se o nome da *seção padrão* for passado, `ValueError` será levantada. O nome da seção deve ser uma string; caso contrário, `TypeError` será levantada.

Alterado na versão 3.2: Nomes de seções sem string levantam `TypeError`.

has_section(*section*)

Indica se a *section* nomeada está presente na configuração. A *seção padrão* não é reconhecida.

options(*section*)

Retorna uma lista de opções disponíveis na *section* especificada.

has_option(*section*, *option*)

Se a *section* fornecida existir e contiver a *option* fornecida, retorna `True`; caso contrário, retorna `False`. Se a *section* especificada for `None` ou uma string vazia, DEFAULT será presumido.

read(*filenames*, *encoding=None*)

Tenta ler e analisar um iterável de nomes de arquivos, retornando uma lista de nomes de arquivos que foram analisados com sucesso.

Se *filenames* for uma string, um objeto `bytes` ou um *objeto caminho ou similar*, este parâmetro será tratado como um único nome de arquivo. Se um arquivo nomeado em *filenames* não puder ser aberto, esse arquivo será ignorado. Isso foi projetado para que você possa especificar um iterável de possíveis locais de arquivo de configuração (por exemplo, o diretório atual, o diretório inicial do usuário e algum diretório de todo o sistema) e todos os arquivos de configuração existentes no iterável serão lidos.

Se nenhum dos arquivos nomeados existir, a instância `ConfigParser` conterá um conjunto de dados vazio. Uma aplicação que requer que valores iniciais sejam carregados de um arquivo deve carregar o arquivo ou arquivos necessários usando `read_file()` antes de chamar `read()` para quaisquer arquivos opcionais:

```
import configparser, os

config = configparser.ConfigParser()
config.read_file(open('defaults.cfg'))
config.read(['site.cfg', os.path.expanduser('~/.myapp.cfg')],
            encoding='cp1250')
```

Alterado na versão 3.2: Adicionado o parâmetro *encoding*. Anteriormente, todos os arquivos eram lidos usando a codificação padrão para `open()`.

Alterado na versão 3.6.1: O parâmetro *filenames* aceita um *objeto caminho ou similar*.

Alterado na versão 3.7: O parâmetro *filenames* aceita um objeto `bytes`.

read_file(*f*, *source=None*)

Lê e analisa dados de configuração de *f* que devem ser iteráveis, produzindo strings Unicode (por exemplo, arquivos abertos em modo texto).

O argumento opcional *source* especifica o nome do arquivo que está sendo lido. Se não for fornecido e *f* tiver um atributo `name`, que é usado para *source*; o padrão é `'<???>'`.

Adicionado na versão 3.2: Substitui `readfp()`.

read_string(*string*, *source='<string>'*)

Analisa dados de configuração de uma string.

O argumento opcional *source* especifica um nome específico do contexto da string passada. Se não for fornecido, `'<string>'` será usado. Geralmente deve ser um caminho do sistema de arquivos ou uma URL.

Adicionado na versão 3.2.

read_dict(*dictionary*, *source='<dict>'*)

Carrega a configuração de qualquer objeto que forneça um método `items()` dict ou similar. Chaves são nomes de seções, valores são dicionários com chaves e valores que devem estar presentes na seção. Se o tipo de dicionário usado preservar a ordem, as seções e suas chaves serão adicionadas em ordem. Os valores são convertidos automaticamente em strings.

O argumento opcional *source* especifica um nome específico do contexto do dicionário passado. Se não for fornecido, `<dict>` será usado.

Este método pode ser usado para copiar o estado entre analisadores sintáticos.

Adicionado na versão 3.2.

get(*section*, *option*, *, *raw=False*, *vars=None*[, *fallback*])

Obtém um valor de *option* para a *section* nomeada. Se *vars* for fornecido, deverá ser um dicionário. A *option* é pesquisada em *vars* (se fornecido), *section* e em *DEFAULTSECT* nesta ordem. Se a chave não for encontrada e *fallback* for fornecido, ele será usado como um valor alternativo. `None` pode ser fornecido como um valor *fallback*.

Todas as interpolações `'%'` são expandidas nos valores de retorno, a menos que o argumento *raw* seja verdadeiro. Os valores das chaves de interpolação são consultados da mesma maneira que a opção.

Alterado na versão 3.2: Os argumentos *raw*, *vars* e *fallback* são palavras somente-nomeadas para proteger os usuários de tentarem usar o terceiro argumento como substituto *fallback* (especialmente ao usar o protocolo de mapeamento).

getint(*section*, *option*, *, *raw=False*, *vars=None*[, *fallback*])

Um método de conveniência que força a *option* na *section* especificada ser um número inteiro. Veja *get()* para explicação de *raw*, *vars* e *fallback*.

getfloat(*section*, *option*, *, *raw=False*, *vars=None*[, *fallback*])

Um método de conveniência que força a *option* na *section* especificada ser um número de ponto flutuante. Veja *get()* para explicação de *raw*, *vars* e *fallback*.

getboolean(*section*, *option*, *, *raw=False*, *vars=None*[, *fallback*])

Um método de conveniência que força a *option* na *section* especificada a um valor booleano. Observe que os valores aceitos para a opção são `'1'`, `'yes'`, `'true'`, e `'on'`, o que fazem com que este método retorne `True`, e `'0'`, `'no'`, `'false'`, e `'off'`, o que fazem com que ele retorne `False`. Esses valores de string são verificados sem distinção entre maiúsculas e minúsculas. Qualquer outro valor fará com que ele levante *ValueError*. Veja *get()* para explicação de *raw*, *vars* e *fallback*.

items(*raw=False*, *vars=None*)

items(*section*, *raw=False*, *vars=None*)

Quando *section* não é fornecido, retorna uma lista de pares *section_name*, *section_proxy*, incluindo DEFAULTSECT.

Caso contrário, retorna uma lista de pares *name*, *value* para as opções na *section* fornecida. Argumentos opcionais têm o mesmo significado do método *get()*.

Alterado na versão 3.8: Os itens presentes em *vars* não aparecem mais no resultado. O comportamento anterior misturava opções reais do analisador sintático com variáveis fornecidas para interpolação.

set(*section*, *option*, *value*)

Se a seção fornecida existir, defina a opção fornecida com o valor especificado; caso contrário, levanta *NoSectionError*. *option* e *value* devem ser strings; caso contrário, *TypeError* será levantada.

write(*fileobject*, *space_around_delimiters=True*)

Escreve uma representação da configuração no *objeto arquivo* especificado, que deve ser aberto em modo texto (aceitando strings). Esta representação pode ser analisada por uma futura chamada *read()*. Se *space_around_delimiters* for verdadeiro, os delimitadores entre chaves e valores serão envoltos por espaços.

Alterado na versão 3.14: Levanta InvalidWriteError se isso gravar uma representação que não pode ser analisada com precisão por uma futura chamada *read()* deste analisador sintático.

> **Nota**
>
> Os comentários no arquivo de configuração original não são preservados ao escrever a configuração. O que é considerado um comentário depende dos valores fornecidos para *comment_prefix* e *inline_comment_prefix*.

remove_option(*section*, *option*)

Remove a *option* especificada da *section* especificada. Se a seção não existir, levanta `NoSectionError`. Se existisse a opção de ser removida, retorna `True`; caso contrário, retorna `False`.

remove_section(*section*)

Remove a *section* especificada da configuração. Se a seção de fato existiu, retorna `True`. Caso contrário, retorna `False`.

optionxform(*option*)

Transforma o nome da opção *option* conforme encontrado em um arquivo de entrada ou conforme passado pelo código do cliente no formato que deve ser usado nas estruturas internas. A implementação padrão retorna uma versão em minúsculas de *option*; subclasses podem substituir isso ou o código do cliente pode definir um atributo com esse nome nas instâncias para afetar esse comportamento.

Você não precisa criar uma subclasse do analisador sintático para usar esse método; você também pode configurá-lo em uma instância, para uma função que recebe um argumento de string e retorna uma string. Definir como `str`, por exemplo, faria com que sejam diferenciadas as letras maiúsculas das minúsculas nos nomes das opções:

```
cfgparser = ConfigParser()
cfgparser.optionxform = str
```

Observe que ao ler arquivos de configuração, os espaços em branco ao redor dos nomes das opções são removidos antes de `optionxform()` ser chamado.

configparser.**UNNAMED_SECTION**

Um objeto especial que representa um nome de seção usado para referenciar a seção sem nome (veja *Seções não nomeadas*).

configparser.**MAX_INTERPOLATION_DEPTH**

A profundidade máxima para interpolação recursiva para `get()` quando o parâmetro *raw* é falso. Isso é relevante apenas quando a *interpolation* padrão é usada.

14.2.11 Objetos RawConfigParser

class configparser.**RawConfigParser**(*defaults=None*, *dict_type=dict*, *allow_no_value=False*, ***, *delimiters=('=', ':')*, *comment_prefixes=('#', ';')*, *inline_comment_prefixes=None*, *strict=True*, *empty_lines_in_values=True*, *default_section=configparser.DEFAULTSECT*, *interpolation=BasicInterpolation()*, *converters={}*, *allow_unnamed_section=False*)

Variante legada do `ConfigParser`. Ela tem a interpolação desabilitada por padrão e permite nomes de seções não-string, nomes de opções e valores através de seus métodos inseguros `add_section` e `set`, bem como o tratamento de argumentos nomeados legados `defaults=` .

Alterado na versão 3.2: *allow_no_value*, *delimiters*, *comment_prefixes*, *strict*, *empty_lines_in_values*, *default_section* e *interpolation* foram adicionados.

Alterado na versão 3.5: O argumento *converters* foi adicionado.

Alterado na versão 3.8: O *dict_type* padrão é `dict`, pois agora preserva a ordem de inserção.

Alterado na versão 3.13: O argumento *allow_unnamed_section* foi adicionado.

> **Nota**
>
> Considere usar *ConfigParser*, que verifica os tipos de valores a serem armazenados internamente. Se você não quiser interpolação, você pode usar `ConfigParser(interpolation=None)`.

add_section(*section*)

Adiciona uma seção chamada *section* ou *UNNAMED_SECTION* à instância.

Se a seção com o nome fornecido já existir, *DuplicateSectionError* é levantada. Se o nome da *seção padrão* for passado, *ValueError* é levantada. Se *UNNAMED_SECTION* for passado e o suporte for desabilitado *UnnamedSectionDisabledError* é levantada.

O tipo da *section* não está marcado, o que permite aos usuários criar seções nomeadas sem string. Este comportamento não é compatível e pode causar erros internos.

Alterado na versão 3.14: Adicionado suporte a *UNNAMED_SECTION*.

set(*section*, *option*, *value*)

Se a seção fornecida existir, defina a opção fornecida com o valor especificado; caso contrário, levanta *NoSectionError*. Embora seja possível usar *RawConfigParser* (ou *ConfigParser* com parâmetros *raw* definidos como verdadeiro) para armazenamento *interno* de valores não-string, funcionalidade completa (incluindo interpolação e saída para arquivos) só pode ser alcançado usando valores de string.

Este método permite que os usuários atribuam valores não-string às chaves internamente. Este comportamento não é suportado e causará erros ao tentar escrever em um arquivo ou obtê-lo no modo não bruto. **Use a API do protocolo de mapeamento** que não permite que tais atribuições ocorram.

14.2.12 Exceções

exception configparser.**Error**

Classe base para todas as outras exceções do *configparser*.

exception configparser.**NoSectionError**

Exceção levantada quando uma seção especificada não é encontrada.

exception configparser.**DuplicateSectionError**

Exceção levantada se *add_section()* for chamado com o nome de uma seção que já está presente ou em analisadores sintáticos estritos quando uma seção for encontrada mais de uma vez em um único arquivo de entrada, string ou dicionário.

Alterado na versão 3.2: Adicionados os atributos e parâmetros opcionais *source* e *lineno* a `__init__()`.

exception configparser.**DuplicateOptionError**

Exceção levantada por analisadores sintáticos estritos se uma única opção aparecer duas vezes durante a leitura de um único arquivo, string ou dicionário. Isso detecta erros ortográficos e erros relacionados a diferenciação de letras maiúsculas e minúsculas como, p. ex., um dicionário pode ter duas chaves representando a mesma chave de configuração que não diferencia maiúsculas de minúsculas.

exception configparser.**NoOptionError**

Exceção levantada quando uma opção especificada não é encontrada na seção especificada.

exception configparser.**InterpolationError**

Classe base para exceções levantadas quando ocorrem problemas ao executar a interpolação de strings.

exception configparser.**InterpolationDepthError**

Exceção levantada quando a interpolação de string não pode ser concluída porque o número de iterações excede *MAX_INTERPOLATION_DEPTH*. Subclasse de *InterpolationError*.

exception configparser.**InterpolationMissingOptionError**

Exceção levantada quando uma opção referenciada a partir de um valor não existe. Subclasse de *InterpolationError*.

exception configparser.**InterpolationSyntaxError**

Exceção levantada quando o texto fonte no qual são feitas as substituições não está em conformidade com a sintaxe exigida. Subclasse de *InterpolationError*.

exception configparser.**MissingSectionHeaderError**

Exceção levantada ao tentar analisar um arquivo que não possui cabeçalhos de seção.

exception configparser.**ParsingError**

Exceção levantada quando ocorrem erros ao tentar analisar um arquivo.

Alterado na versão 3.12: O atributo `filename` e o argumento do construtor `__init__()` foram removidos. Eles estiveram disponíveis usando o nome `source` desde 3.2.

exception configparser.**MultilineContinuationError**

Exceção levantada quando uma chave sem valor correspondente continua com uma linha indentada.

Adicionado na versão 3.13.

exception configparser.**UnnamedSectionDisabledError**

Exceção levantada ao tentar usar *UNNAMED_SECTION* sem habilitá-lo.

Adicionado na versão 3.14.

exception configparser.**InvalidWriteError**

Exceção levantada quando uma tentativa de *ConfigParser.write()* não seria analisada com precisão com uma futura chamada de *ConfigParser.read()*.

Ex: Escrever uma chave que comece com o padrão *ConfigParser.SECTCRE* seria interpretado como um cabeçalho de seção ao ser lido. Tentar escrever isso levantará esta exceção.

Adicionado na versão 3.14.

14.3 tomllib — Analisa arquivos TOML

Adicionado na versão 3.11.

Código-fonte: Lib/tomllib

Este módulo fornece uma interface para analisar TOML 1.0.0 (Tom's Obvious Minimal Language, https://toml.io). Este módulo não oferece suporte para escrever TOML.

Ver também

O pacote Tomli-W é um editor de TOML que pode ser usado em conjunto com este módulo, fornecendo uma API de escrita familiar aos usuários da biblioteca padrão: módulos *marshal* e *pickle*.

Ver também

O pacote TOML Kit é uma biblioteca TOML de preservação de estilo com capacidade de leitura e escrita. É uma substituição recomendada para este módulo para edição de arquivos TOML já existentes.

Este módulo define as seguintes funções:

tomllib.**load**(*fp*, /, *, *parse_float=float*)

Lê um arquivo TOML. O primeiro argumento deve ser um objeto arquivo binário e legível. Retorna um *dict*. Converte tipos TOML para Python usando esta *tabela de conversão*.

parse_float será chamado com a string de cada ponto flutuante (float) do TOML a ser decodificado. Por padrão, isso é equivalente a `float(num_str)`. Isso pode ser usado para usar outro tipo de dados ou analisador

sintático para pontos flutuantes do TOML (por exemplo, `decimal.Decimal`). O chamável não deve retornar um `dict` ou um `list`, senão uma exceção `ValueError` é levantada.

Uma exceção `TOMLDecodeError` será levantada no caso de um documento TOML inválido.

tomllib.**loads**(*s*, /, *, *parse_float=float*)

Carrega TOML de um objeto `str`. Retorna um `dict`. Converte tipos TOML para Python usando esta *tabela de conversão*. O argumento *parse_float* tem o mesmo significado que em `load()`.

Uma exceção `TOMLDecodeError` será levantada no caso de um documento TOML inválido.

As seguintes exceções estão disponíveis:

exception tomllib.**TOMLDecodeError**(*msg*, *doc*, *pos*)

Subclasse de `ValueError` com os seguintes atributos adicionais:

msg

A mensagem de erro não formatada.

doc

O documento TOML sendo analisado.

pos

O índice de *doc* onde a análise falhou.

lineno

A linha correspondente a *pos*.

colno

A coluna correspondente a *pos*.

Alterado na versão 3.14: Adicionados os parâmetros *msg*, *doc* e *pos*. Adicionados os atributos `msg`, `doc`, `pos`, `lineno` e `colno`.

Descontinuado desde a versão 3.14: A passagem de argumentos posicionais de forma livre está descontinuada.

14.3.1 Exemplos

Analisando um arquivo TOML:

```
import tomllib

with open("pyproject.toml", "rb") as f:
    data = tomllib.load(f)
```

Analisando uma string TOML:

```
import tomllib

toml_str = """
python-version = "3.11.0"
python-implementation = "CPython"
"""

data = tomllib.loads(toml_str)
```

14.3.2 Tabela de conversão

TOML	Python
documento TOML	dict
string	str
inteiro	int
ponto flutuante	ponto flutuante (configurável com *parse_float*)
booleano	bool
deslocamento de data-hora	datetime.datetime (atributo de `tzinfo` definido com uma instância de `datetime.timezone`)
data-hora local	datetime.datetime (atributo de `tzinfo` definido com `None`)
data local	datetime.date
hora local	datetime.time
array	lista
tabela	dict
tabela inline	dict
array de tabelas	lista de dicionários

14.4 `netrc` — Arquivo de processamento netrc

Código-fonte: Lib/netrc.py

A classe *netrc* analisa e encapsula o formato do arquivo netrc usado pelo programa Unix **ftp** e outros clientes FTP.

class netrc.**netrc**([*file*])

Uma instância ou instância de subclasse de *netrc* encapsula dados de um arquivo netrc. O argumento de inicialização, se presente, especifica o arquivo a ser analisado. Se nenhum argumento for fornecido, o arquivo `.netrc` no diretório inicial do usuário – conforme determinado por *os.path.expanduser()* – será lido. Caso contrário, uma exceção *FileNotFoundError* será levantada. Os erros de análise levantam *NetrcParseError* com informações de diagnóstico, incluindo o nome do arquivo, o número da linha e o token final.

Se nenhum argumento for especificado em um sistema POSIX, a presença de senhas no arquivo `.netrc` levantará um *NetrcParseError* se a propriedade ou as permissões do arquivo forem inseguras (pertencentes a um usuário que não seja o usuário executando o processo ou acessível para leitura ou gravação por qualquer outro usuário). Isso implementa um comportamento de segurança equivalente ao do ftp e de outros programas que usam `.netrc`. Essas verificações de segurança não estão disponíveis em plataformas que não oferecem suporte a *os.getuid()*.

Alterado na versão 3.4: Adicionada a verificação de permissão POSIX.

Alterado na versão 3.7: *os.path.expanduser()* é usado para encontrar a localização do arquivo `.netrc` quando *file* não é passado como argumento.

Alterado na versão 3.10: *netrc* tenta a codificação UTF-8 antes de usar a codificação específica da localidade. A entrada no arquivo netrc não precisa mais conter todos os tokens. O valor padrão dos tokens ausentes é uma string vazia. Todos os tokens e seus valores agora podem conter caracteres arbitrários, como espaços em branco e caracteres não ASCII. Se o nome de login for anônimo, ele não acionará a verificação de segurança.

exception netrc.**NetrcParseError**

Exceção levantada pela classe *netrc* quando erros sintáticos são encontrados no texto fonte. As instâncias desta exceção fornecem três atributos interessantes:

msg

Explicação textual do erro.

filename

O nome do arquivo de origem.

lineno

O número da linha em que o erro foi encontrado.

14.4.1 Objetos netrc

Uma instância da classe *netrc* tem os seguintes métodos:

netrc.**authenticators**(*host*)

Retorna uma tupla de 3 elementos `(login, conta, senha)` dos autenticadores do *host*. Se o arquivo netrc não contém uma entrada para o host dado, retorna a tupla associada com a entrada padrão. Se não houver nenhum host correspondente nem uma entrada padrão estiver disponível, retorna `None`.

netrc.**__repr__**()

Despeja os dados da classe como uma string no formato de um arquivo netrc. (Isso descarta os comentários e pode reordenar as entradas.)

Instâncias de *netrc* possuem variáveis de instância públicas:

netrc.**hosts**

Dicionário mapeando nomes de host para tuplas `(login, conta, senha)`. A entrada *default*, se houver, é representada como um pseudo-host por esse nome.

netrc.**macros**

Dicionário mapeando nomes de macros para listas de strings.

14.5 `plistlib` — Gera e analisa arquivos Apple `.plist`

Código-fonte: Lib/plistlib.py

Este módulo fornece uma interface para ler e escrever os arquivos "property listas (lista de propriedades) usados pela Apple, principalmente no macOS e iOS. Este módulo tem suporte a arquivos plist binários e XML.

O formato de arquivo de lista de propriedades (`.plist`) é uma serialização simples que oferece suporte a tipos básicos de objetos, como dicionários, listas, números e strings. Normalmente, o objeto de nível superior é um dicionário.

Para escrever e analisar um arquivo plist, use as funções *dump()* e *load()*.

Para trabalhar com dados plist em objetos bytes ou string, use *dumps()* e *loads()*.

Os valores podem ser strings, inteiros, floats, booleanos, tuplas, listas, dicionários (mas somente com chaves de string), objetos *bytes*, *bytearray* ou *datetime.datetime*.

Alterado na versão 3.4: Nova API, API antiga descontinuada. Suporte para plists de formato binário adicionado.

Alterado na versão 3.8: Adicionado suporte para leitura e escrita de tokens *UID* em plists binários, conforme usado por NSKeyedArchiver e NSKeyedUnarchiver.

Alterado na versão 3.9: API antiga removida.

Ver também

Pagina de manual de PList
Documentação da Apple sobre o formato de arquivo.

Este módulo define as seguintes funções:

plistlib.**load**(*fp*, *, *fmt=None*, *dict_type=dict*, *aware_datetime=False*)

Lê um arquivo plist. *fp* deve ser um objeto arquivo binário e legível. Retorna o objeto raiz desempacotado (que geralmente é um dicionário).

fmt é o formato do arquivo e os seguintes valores são válidos:

- *None*: Detecta automaticamente o formato do arquivo
- *FMT_XML*: Formato de arquivo XML
- *FMT_BINARY*: Formato binário plist

O *dict_type* é o tipo usado para dicionários que são lidos do arquivo plist.

Quando *aware_datetime* for verdadeiro, campos com o tipo `datetime.datetime` serão criados como *objeto conscientes*, com `tzinfo` como *datetime.UTC*.

Dados XML para o formato *FMT_XML* são analisados usando o analisador sintático Expat de *xml.parsers.expat* – veja sua documentação para possíveis exceções em XML malformado. Elementos desconhecidos serão simplesmente ignorados pelo analisador de plist.

O analisador sintático levanta *InvalidFileException* quando o arquivo não pode ser analisado.

Adicionado na versão 3.4.

Alterado na versão 3.13: O parâmetro somente-nomeado *aware_datetime* foi adicionado.

plistlib.**loads**(*data*, *, *fmt=None*, *dict_type=dict*, *aware_datetime=False*)

Carrega um plist de um objeto bytes ou string. Veja *load()* para uma explicação dos argumentos nomeados.

Adicionado na versão 3.4.

Alterado na versão 3.13: *data* pode ser uma string quando *fmt* é igual a *FMT_XML*.

plistlib.**dump**(*value*, *fp*, *, *fmt=FMT_XML*, *sort_keys=True*, *skipkeys=False*, *aware_datetime=False*)

Escreve *value* em um arquivo plist. *fp* deve ser um objeto arquivo binário gravável.

O argumento *fmt* especifica o formato do arquivo plist e pode ser um dos seguintes valores:

- *FMT_XML*: Arquivo plist formatado em XML
- *FMT_BINARY*: Arquivo plist formatado em binário

Quando *sort_keys* é verdadeiro (o padrão), as chaves dos dicionários serão escritas ordenadas no plist, caso contrário, elas serão escritas na ordem de iteração do dicionário.

Quando *skipkeys* é falso (o padrão), a função levanta *TypeError* quando uma chave de um dicionário não é uma string, caso contrário, tais chaves são ignoradas.

Quando *aware_datetime* é verdadeiro e qualquer campo com o tipo `datetime.datetime` é definido como um *objeto consciente*, ele será convertido para o fuso horário UTC antes de escrevê-lo.

Uma *TypeError* será levantada se o objeto for de um tipo não suportado ou um contêiner que contém objetos de tipos não suportados.

Uma *OverflowError* será levantada para valores inteiros que não podem ser representados em arquivos plist (binários).

Adicionado na versão 3.4.

Alterado na versão 3.13: O parâmetro somente-nomeado *aware_datetime* foi adicionado.

plistlib.**dumps**(*value*, *, *fmt=FMT_XML*, *sort_keys=True*, *skipkeys=False*, *aware_datetime=False*)

Retorna *value* como um objeto bytes formatado em plist. Veja a documentação para *dump()* para uma explicação dos argumentos nomeados desta função.

Adicionado na versão 3.4.

As seguintes classes estão disponíveis:

class plistlib.**UID**(*data*)

Envolve um *int*. Isso é usado ao ler ou escrever dados codificados NSKeyedArchiver, que contém UID (veja o manual de PList).

data

Valor inteiro do UID. Deve estar no intervalo `0 <= data < 2**64`.

Adicionado na versão 3.8.

As seguintes constantes estão disponíveis:

plistlib.**FMT_XML**

O formato XML para arquivos plist.

Adicionado na versão 3.4.

plistlib.**FMT_BINARY**

O formato binário para arquivos plist

Adicionado na versão 3.4.

O módulo define as seguintes exceções:

exception plistlib.**InvalidFileException**

Levantada quando um arquivo não puder ser analisado.

Adicionado na versão 3.4.

14.5.1 Exemplos

Gerando um plist:

```
import datetime
import plistlib

pl = dict(
    aString = "Doodah",
    aList = ["A", "B", 12, 32.1, [1, 2, 3]],
    aFloat = 0.1,
    anInt = 728,
    aDict = dict(
        anotherString = "<hello & hi there!>",
        aThirdString = "M\xe4ssig, Ma\xdf",
        aTrueValue = True,
        aFalseValue = False,
    ),
    someData = b"<binary gunk>",
    someMoreData = b"<lots of binary gunk>" * 10,
    aDate = datetime.datetime.now()
)
print(plistlib.dumps(pl).decode())
```

Analisando um plist:

```
import plistlib

plist = b"""<plist version="1.0">
<dict>
    <key>foo</key>
    <string>bar</string>
</dict>
</plist>"""
```

(continua na próxima página)

(continuação da página anterior)

```
pl = plistlib.loads(plist)
print(pl["foo"])
```

CAPÍTULO 15

Serviços Criptográficos

Os módulos descritos neste capítulo implementam vários algoritmos de uma natureza criptográfica. Eles estão disponíveis a critério da instalação. Aqui está uma visão geral:

15.1 `hashlib` — resumos de mensagens e hashes seguros

Código-fonte: Lib/hashlib.py

Este módulo implementa uma interface comum para diversos algoritmos de hash. Estão incluídos os algoritmos de hash seguros FIPS SHA224, SHA256, SHA384 e SHA512 (definidos no padrão FIPS 180-4), a série SHA-3 (definida no padrão FIPS 202), bem como os algoritmos legados SHA1 (anteriormente parte do FIPS) e o algoritmo MD5 (definido em internet **RFC 1321**).

> **Nota**
>
> Se você quiser as funções de hash adler32 ou crc32, elas estão disponíveis no módulo `zlib`.

15.1.1 Algoritmos de hash

Há um método construtor nomeado para cada tipo de *hash*. Todos retornam um objeto hash com a mesma interface simples. Por exemplo: use `sha256()` para criar um objeto hash SHA-256. Agora você pode alimentar este objeto com *objetos bytes ou similares* (normalmente `bytes`) usando o método `update`. A qualquer momento, você pode solicitar o *digest* da concatenação dos dados alimentados até o momento usando os métodos `digest()` ou `hexdigest()`.

Para permitir multithreading, a *GIL* do Python é liberada ao calcular um hash fornecido com mais de 2047 bytes de dados de uma só vez em seu construtor ou método `.update`.

Os construtores para algoritmos de hash sempre presentes neste módulo são `sha1()`, `sha224()`, `sha256()`, `sha384()`, `sha512()`, `sha3_224()`, `sha3_256()`, `sha3_384()`, `sha3_512()`, `shake_128()`, `shake_256()`, `blake2b()` e `blake2s()`. `md5()` normalmente também está disponível, embora possa estar ausente ou bloqueado se você estiver usando uma construção rara do Python "compatível com FIPS". Eles correspondem a `algorithms_guaranteed`.

Algoritmos adicionais também podem estar disponíveis se o *hashlib* da sua distribuição Python tiver sido vinculado a uma construção do OpenSSL que forneça outros algoritmos. Outros *não têm garantia de disponibilidade* em todas as instalações e só serão acessíveis pelo nome via *new()*. Consulte *algorithms_available*.

Aviso

Alguns algoritmos apresentam vulnerabilidades conhecidas em colisões de hash (incluindo MD5 e SHA1). Consulte Ataques a algoritmos de hash criptográficos e a seção *hashlib-seealso* no final deste documento.

Adicionado na versão 3.6: Os construtores SHA3 (Keccak) e SHAKE *sha3_224()*, *sha3_256()*, *sha3_384()*, *sha3_512()*, *shake_128()*, *shake_256()* foram adicionados. *blake2b()* e *blake2s()* foram adicionados. Alterado na versão 3.9: Todos os construtores de hashlib aceitam um argumento somente-nomeado *usedforsecurity* com o valor padrão `True`. Um valor falso permite o uso de algoritmos de hash inseguros e bloqueados em ambientes restritos. `False` indica que o algoritmo de hash não é usado em um contexto de segurança, por exemplo, como uma função de compressão unidirecional não criptográfica.

Alterado na versão 3.9: O hashlib agora usa SHA3 e SHAKE do OpenSSL, se ele os fornecer.

Alterado na versão 3.12: Para qualquer um dos algoritmos MD5, SHA1, SHA2 ou SHA3 que o OpenSSL vinculado não fornece, recorremos a uma implementação verificada do projeto HACL*.

15.1.2 Uso

Para obter o resumo da string de bytes `b"Nobody inspects the spammish repetition"`:

```
>>> import hashlib
>>> m = hashlib.sha256()
>>> m.update(b"Nobody inspects")
>>> m.update(b" the spammish repetition")
>>> m.digest()
b'\x03\x1e\xdd}Ae\x15\x93\xc5\xfe\\\x00o\xa5u+7\xfd\xdf\xf7\xbcN\x84:\xa6\xaf\x0c\
↪x95\x0fK\x94\x06'
>>> m.hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
```

Mais condensado:

```
>>> hashlib.sha256(b"Nobody inspects the spammish repetition").hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
```

15.1.3 Construtores

hashlib.**new**(*name*, [*data*,]*, *usedforsecurity=True*)

É um construtor genérico que recebe a string *name* do algoritmo desejado como seu primeiro parâmetro. Ele também existe para permitir acesso aos hashes listados acima, bem como a quaisquer outros algoritmos que sua biblioteca OpenSSL possa oferecer.

Usando *new()* com um nome de algoritmo:

```
>>> h = hashlib.new('sha256')
>>> h.update(b"Nobody inspects the spammish repetition")
>>> h.hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
```

hashlib.**md5**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha1**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha224**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha256**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha384**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha512**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha3_224**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha3_256**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha3_384**([*data*,]*, *usedforsecurity=True*)

hashlib.**sha3_512**([*data*,]*, *usedforsecurity=True*)

Construtores nomeados como esses são mais rápidos do que passar um nome de algoritmo para *new()*.

15.1.4 Atributos

O hashlib fornece os seguintes atributos de módulo constantes:

hashlib.**algorithms_guaranteed**

Um conjunto contendo os nomes dos algoritmos de hash com suporte garantido por este módulo em todas as plataformas. Observe que "md5" está nesta lista, apesar de alguns fornecedores originais oferecerem uma estranha construção Python "compatível com FIPS" que o exclui.

Adicionado na versão 3.2.

hashlib.**algorithms_available**

Um conjunto contendo os nomes dos algoritmos de hash disponíveis no interpretador Python em execução. Esses nomes serão reconhecidos quando passados para *new()*. *algorithms_guaranteed* sempre será um subconjunto. O mesmo algoritmo pode aparecer várias vezes neste conjunto com nomes diferentes (graças ao OpenSSL).

Adicionado na versão 3.2.

15.1.5 Objetos hash

Os seguintes valores são fornecidos como atributos constantes dos objetos hash retornados pelos construtores:

hash.**digest_size**

O tamanho do hash resultante em bytes.

hash.**block_size**

O tamanho do bloco interno do algoritmo de hash em bytes.

Um objeto hash tem os seguintes atributos:

hash.**name**

O nome canônico deste hash, sempre em minúsculas e sempre adequado como parâmetro para *new()* para criar outro hash deste tipo.

Alterado na versão 3.4: O atributo name está presente no CPython desde o seu início, mas até o Python 3.4 não era especificado formalmente, então pode não existir em algumas plataformas.

Um objeto hash tem os seguintes métodos:

hash.**update**(*data*)

Atualiza o objeto hash com o *objeto bytes ou similar*. Chamadas repetidas são equivalentes a uma única chamada com a concatenação de todos os argumentos: `m.update(a); m.update(b)` é equivalente a `m.update(a+b)`.

hash.**digest**()

Retorna o resumo dos dados passados ao método *update()* até o momento. Este é um objeto bytes de tamanho *digest_size* que pode conter bytes em todo o intervalo de 0 a 255.

hash.**hexdigest**()

Similar a *digest()*, exceto que o resumo é retornado como um objeto string de comprimento duplo, contendo apenas dígitos hexadecimais. Isso pode ser usado para trocar o valor com segurança em e-mails ou outros ambientes não binários.

hash.**copy**()

Retorna uma cópia ("clone") do objeto hash. Isso pode ser usado para calcular com eficiência os resumos de dados que compartilham uma substring inicial comum.

15.1.6 Resumos de comprimento variável de SHAKE

hashlib.**shake_128**([*data*,]*, *usedforsecurity=True*)

hashlib.**shake_256**([*data*,]*, *usedforsecurity=True*)

Os algoritmos *shake_128()* e *shake_256()* fornecem resumos de comprimento variável com length_in_bits//2 de até 128 ou 256 bits de segurança. Portanto, seus métodos de resumo exigem um comprimento. O comprimento máximo não é limitado pelo algoritmo SHAKE.

shake.**digest**(*length*)

Retorna o resumo dos dados passados ao método *update()* até o momento. Este é um objeto bytes de tamanho *length* que pode conter bytes em todo o intervalo de 0 a 255.

shake.**hexdigest**(*length*)

Similar a *digest()*, exceto que o resumo é retornado como um objeto string de comprimento duplo, contendo apenas dígitos hexadecimais. Isso pode ser usado para trocar o valor em e-mails ou outros ambientes não binários.

Exemplo de uso:

```
>>> h = hashlib.shake_256(b'Nobody inspects the spammish repetition')
>>> h.hexdigest(20)
'44709d6fcb83d92a76dcb0b668c98e1b1d3dafe7'
```

15.1.7 Hash de arquivo

O módulo hashlib fornece uma função auxiliar para hash eficiente de um objeto arquivo ou similar.

hashlib.**file_digest**(*fileobj*, *digest*, */*)

Retorna um objeto resumo que foi atualizado com o conteúdo do objeto arquivo.

fileobj deve ser um objeto arquivo ou similar aberto para leitura em modo binário. Ele aceita objetos arquivo da *open()* embutida, instâncias de *BytesIO*, objetos SocketIO de *socket.socket.makefile()* e similares. *fileobj* deve ser aberto em modo bloqueante, caso contrário, uma exceção *BlockingIOError* pode ser levantada.

A função pode ignorar a E/S do Python e usar o descritor de arquivo de *fileno()* diretamente. *fileobj* deve ser presumido como sendo um estado desconhecido após o retorno ou o levantamento desta função. Cabe a quem a chamou fechar *fileobj*.

digest deve ser um nome de algoritmo de hash como *str*, um construtor de hash ou um chamável que retorna um objeto de hash.

Exemplo:

```
>>> import io, hashlib, hmac
>>> with open("library/hashlib.rst", "rb") as f:
...     digest = hashlib.file_digest(f, "sha256")
...
>>> digest.hexdigest()
'...'
```

```
>>> buf = io.BytesIO(b"somedata")
>>> mac1 = hmac.HMAC(b"key", digestmod=hashlib.sha512)
>>> digest = hashlib.file_digest(buf, lambda: mac1)
```

```
>>> digest is mac1
True
>>> mac2 = hmac.HMAC(b"key", b"somedata", digestmod=hashlib.sha512)
>>> mac1.digest() == mac2.digest()
True
```

Adicionado na versão 3.11.

Alterado na versão 3.14: Agora levanta uma *BlockingIOError* se o arquivo for aberto em modo bloqueante. Anteriormente, bytes nulos espúrios eram adicionados ao resumo.

15.1.8 Derivação de chave

Algoritmos de alongamento de chave e derivação de chave são projetados para criar hashes de senhas seguros. Algoritmos ingênuos como `sha1(password)` não são resistentes a ataques de força bruta. Uma boa função de hashing de senhas deve ser ajustável, lenta e incluir um salt.

`hashlib.`**`pbkdf2_hmac`**(*hash_name*, *password*, *salt*, *iterations*, *dklen=None*)

A função fornece a função 2 de derivação de chave baseada em senha PKCS#5. Ela usa HMAC como função pseudoaleatória.

A string *hash_name* é o nome desejado do algoritmo de resumo de hash para HMAC, por exemplo, 'sha1' ou 'sha256'. *password* e *salt* são interpretados como buffers de bytes. Aplicações e bibliotecas devem limitar *password* a um comprimento razoável (por exemplo, 1024). *salt* deve ter cerca de 16 bytes ou mais de uma fonte adequada, por exemplo, *os.urandom()*.

O número de *iterations* deve ser escolhido com base no algoritmo de hash e no poder computacional. A partir de 2022, centenas de milhares de iterações do SHA-256 são sugeridas. Para entender por que e como escolher o que é melhor para sua aplicação, leia o *Appendix A.2.2* do NIST-SP-800-132. As respostas à pergunta sobre iterações de pbkdf2 no StackExchange explicam em detalhes.

dklen é o comprimento da chave derivada em bytes. Se *dklen* for `None`, o tamanho do resumo do algoritmo de hash *hash_name* será usado, por exemplo, 64 para SHA-512.

```
>>> from hashlib import pbkdf2_hmac
>>> our_app_iters = 500_000  # Application specific, read above.
>>> dk = pbkdf2_hmac('sha256', b'password', b'bad salt' * 2, our_app_iters)
>>> dk.hex()
'15530bba69924174860db778f2c6f8104d3aaf9d26241840c8c4a641c8d000a9'
```

Função disponível somente quando Python é compilado com OpenSSL.

Adicionado na versão 3.4.

Alterado na versão 3.12: Função agora disponível apenas quando o Python é criado com OpenSSL. A implementação lenta do Python puro foi removida.

`hashlib.`**`scrypt`**(*password*, ***, *salt*, *n*, *r*, *p*, *maxmem=0*, *dklen=64*)

A função fornece a função de derivação de chave baseada em senha scrypt, conforme definido em **RFC 7914**.

password e *salt* devem ser *objetos bytes ou similares*. Aplicações e bibliotecas devem limitar *password* a um tamanho razoável (por exemplo, 1024). *salt* deve ter cerca de 16 bytes ou mais de uma fonte adequada, por exemplo, `os.urandom()`.

n é o fator de custo de CPU/memória, *r* o tamanho do bloco, *p* o fator de paralelismo e *maxmem* limita a memória (o padrão do OpenSSL 1.1.0 é 32 MiB). *dklen* é o comprimento da chave derivada em bytes.

Adicionado na versão 3.6.

15.1.9 BLAKE2

BLAKE2 é uma função hash criptográfica definida em **RFC 7693** que vem em dois sabores:

- **BLAKE2b**, otimizado para plataformas de 64 bits e produz resumos de qualquer tamanho entre 1 e 64 bytes,
- **BLAKE2s**, otimizado para plataformas de 8 a 32 bits e produz resumos de qualquer tamanho entre 1 e 32 bytes.

BLAKE2 supports **keyed mode** (a faster and simpler replacement for HMAC), **salted hashing**, **personalization**, and **tree hashing**.

Hash objects from this module follow the API of standard library's `hashlib` objects.

Creating hash objects

New hash objects are created by calling constructor functions:

`hashlib.`**`blake2b`**(*data=b''*, ***, *digest_size=64*, *key=b''*, *salt=b''*, *person=b''*, *fanout=1*, *depth=1*, *leaf_size=0*, *node_offset=0*, *node_depth=0*, *inner_size=0*, *last_node=False*, *usedforsecurity=True*)

`hashlib.`**`blake2s`**(*data=b''*, ***, *digest_size=32*, *key=b''*, *salt=b''*, *person=b''*, *fanout=1*, *depth=1*, *leaf_size=0*, *node_offset=0*, *node_depth=0*, *inner_size=0*, *last_node=False*, *usedforsecurity=True*)

These functions return the corresponding hash objects for calculating BLAKE2b or BLAKE2s. They optionally take these general parameters:

- *data*: initial chunk of data to hash, which must be *bytes-like object*. It can be passed only as positional argument.
- *digest_size*: size of output digest in bytes.
- *key*: key for keyed hashing (up to 64 bytes for BLAKE2b, up to 32 bytes for BLAKE2s).
- *salt*: salt for randomized hashing (up to 16 bytes for BLAKE2b, up to 8 bytes for BLAKE2s).
- *person*: personalization string (up to 16 bytes for BLAKE2b, up to 8 bytes for BLAKE2s).

The following table shows limits for general parameters (in bytes):

Hash	digest_size	len(key)	len(salt)	len(person)
BLAKE2b	64	64	16	16
BLAKE2s	32	32	8	8

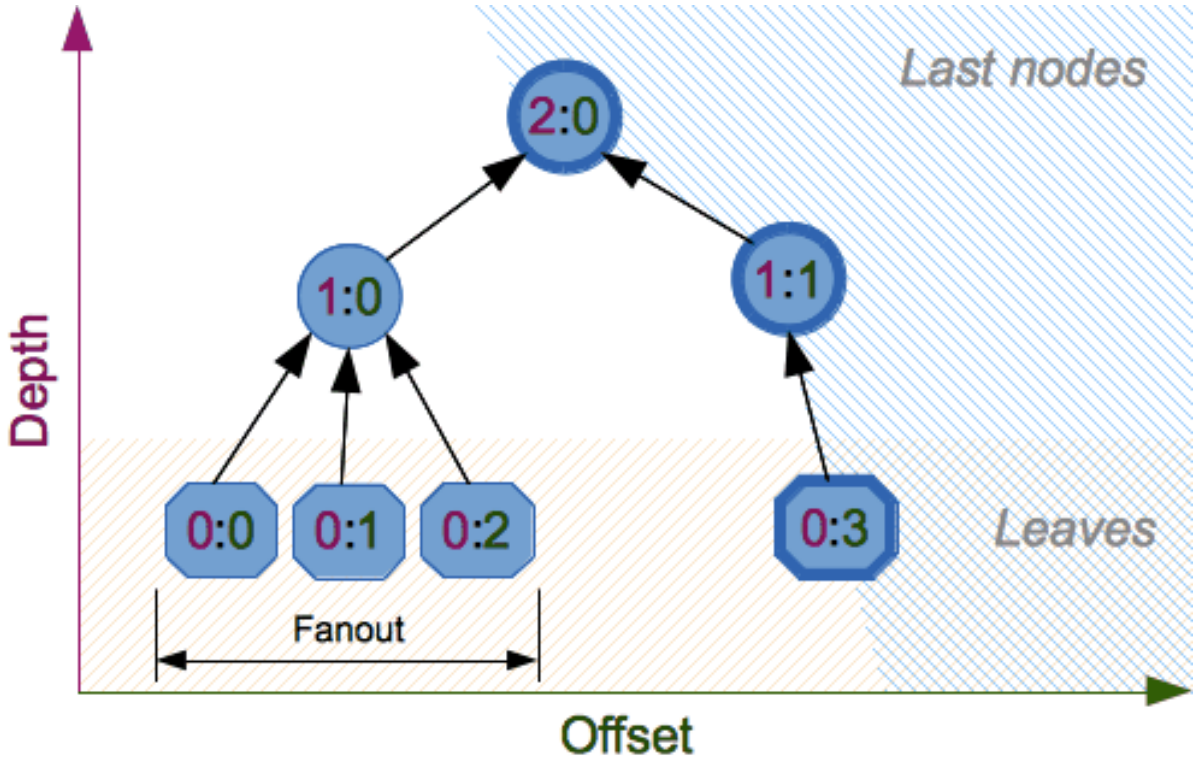
Nota

BLAKE2 specification defines constant lengths for salt and personalization parameters, however, for convenience, this implementation accepts byte strings of any size up to the specified length. If the length of the parameter is less than specified, it is padded with zeros, thus, for example, `b'salt'` and `b'salt\x00'` is the same value. (This is not the case for *key*.)

These sizes are available as module *constants* described below.

Constructor functions also accept the following tree hashing parameters:

- *fanout*: fanout (0 to 255, 0 if unlimited, 1 in sequential mode).
- *depth*: maximal depth of tree (1 to 255, 255 if unlimited, 1 in sequential mode).
- *leaf_size*: maximal byte length of leaf (0 to `2**32-1`, 0 if unlimited or in sequential mode).
- *node_offset*: node offset (0 to `2**64-1` for BLAKE2b, 0 to `2**48-1` for BLAKE2s, 0 for the first, leftmost, leaf, or in sequential mode).
- *node_depth*: node depth (0 to 255, 0 for leaves, or in sequential mode).
- *inner_size*: inner digest size (0 to 64 for BLAKE2b, 0 to 32 for BLAKE2s, 0 in sequential mode).
- *last_node*: boolean indicating whether the processed node is the last one (`False` for sequential mode).



See section 2.10 in BLAKE2 specification for comprehensive review of tree hashing.

Constantes

`blake2b.`**`SALT_SIZE`**

`blake2s.`**`SALT_SIZE`**

Salt length (maximum length accepted by constructors).

`blake2b.`**`PERSON_SIZE`**

`blake2s.`**`PERSON_SIZE`**

Personalization string length (maximum length accepted by constructors).

`blake2b.`**`MAX_KEY_SIZE`**

`blake2s.`**`MAX_KEY_SIZE`**

Maximum key size.

`blake2b.`**`MAX_DIGEST_SIZE`**

blake2s.**MAX_DIGEST_SIZE**

Maximum digest size that the hash function can output.

Exemplos

Simple hashing

To calculate hash of some data, you should first construct a hash object by calling the appropriate constructor function (*blake2b()* or *blake2s()*), then update it with the data by calling *update()* on the object, and, finally, get the digest out of the object by calling *digest()* (or *hexdigest()* for hex-encoded string).

```
>>> from hashlib import blake2b
>>> h = blake2b()
>>> h.update(b'Hello world')
>>> h.hexdigest()

↪'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f33343a8c65551134ed1ae0f2b0dd2bb4
↪'
```

As a shortcut, you can pass the first chunk of data to update directly to the constructor as the positional argument:

```
>>> from hashlib import blake2b
>>> blake2b(b'Hello world').hexdigest()

↪'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f33343a8c65551134ed1ae0f2b0dd2bb4
↪'
```

You can call *hash.update()* as many times as you need to iteratively update the hash:

```
>>> from hashlib import blake2b
>>> items = [b'Hello', b' ', b'world']
>>> h = blake2b()
>>> for item in items:
...     h.update(item)
...
>>> h.hexdigest()

↪'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f33343a8c65551134ed1ae0f2b0dd2bb4
↪'
```

Using different digest sizes

BLAKE2 has configurable size of digests up to 64 bytes for BLAKE2b and up to 32 bytes for BLAKE2s. For example, to replace SHA-1 with BLAKE2b without changing the size of output, we can tell BLAKE2b to produce 20-byte digests:

```
>>> from hashlib import blake2b
>>> h = blake2b(digest_size=20)
>>> h.update(b'Replacing SHA1 with the more secure function')
>>> h.hexdigest()
'd24f26cf8de66472d58d4e1b1774b4c9158b1f4c'
>>> h.digest_size
20
>>> len(h.digest())
20
```

Hash objects with different digest sizes have completely different outputs (shorter hashes are *not* prefixes of longer hashes); BLAKE2b and BLAKE2s produce different outputs even if the output length is the same:

```
>>> from hashlib import blake2b, blake2s
>>> blake2b(digest_size=10).hexdigest()
'6fa1d8fcfd719046d762'
>>> blake2b(digest_size=11).hexdigest()
'eb6ec15daf9546254f0809'
>>> blake2s(digest_size=10).hexdigest()
'1bf21a98c78a1c376ae9'
>>> blake2s(digest_size=11).hexdigest()
'567004bf96e4a25773ebf4'
```

Keyed hashing

Keyed hashing can be used for authentication as a faster and simpler replacement for Hash-based message authentication code (HMAC). BLAKE2 can be securely used in prefix-MAC mode thanks to the indifferentiability property inherited from BLAKE.

This example shows how to get a (hex-encoded) 128-bit authentication code for message `b'message data'` with key `b'pseudorandom key'`:

```
>>> from hashlib import blake2b
>>> h = blake2b(key=b'pseudorandom key', digest_size=16)
>>> h.update(b'message data')
>>> h.hexdigest()
'3d363ff7401e02026f4a4687d4863ced'
```

As a practical example, a web application can symmetrically sign cookies sent to users and later verify them to make sure they weren't tampered with:

```
>>> from hashlib import blake2b
>>> from hmac import compare_digest
>>>
>>> SECRET_KEY = b'pseudorandomly generated server secret key'
>>> AUTH_SIZE = 16
>>>
>>> def sign(cookie):
...     h = blake2b(digest_size=AUTH_SIZE, key=SECRET_KEY)
...     h.update(cookie)
...     return h.hexdigest().encode('utf-8')
>>>
>>> def verify(cookie, sig):
...     good_sig = sign(cookie)
...     return compare_digest(good_sig, sig)
>>>
>>> cookie = b'user-alice'
>>> sig = sign(cookie)
>>> print("{0},{1}".format(cookie.decode('utf-8'), sig))
user-alice,b'43b3c982cf697e0c5ab22172d1ca7421'
>>> verify(cookie, sig)
True
>>> verify(b'user-bob', sig)
False
>>> verify(cookie, b'0102030405060708090a0b0c0d0e0f00')
False
```

Even though there's a native keyed hashing mode, BLAKE2 can, of course, be used in HMAC construction with *hmac* module:

```
>>> import hmac, hashlib
>>> m = hmac.new(b'secret key', digestmod=hashlib.blake2s)
>>> m.update(b'message')
>>> m.hexdigest()
'e3c8102868d28b5ff85fc35dda07329970d1a01e273c37481326fe0c861c8142'
```

Randomized hashing

By setting *salt* parameter users can introduce randomization to the hash function. Randomized hashing is useful for protecting against collision attacks on the hash function used in digital signatures.

> Randomized hashing is designed for situations where one party, the message preparer, generates all or part of a message to be signed by a second party, the message signer. If the message preparer is able to find cryptographic hash function collisions (i.e., two messages producing the same hash value), then they might prepare meaningful versions of the message that would produce the same hash value and digital signature, but with different results (e.g., transferring $1,000,000 to an account, rather than $10). Cryptographic hash functions have been designed with collision resistance as a major goal, but the current concentration on attacking cryptographic hash functions may result in a given cryptographic hash function providing less collision resistance than expected. Randomized hashing offers the signer additional protection by reducing the likelihood that a preparer can generate two or more messages that ultimately yield the same hash value during the digital signature generation process — even if it is practical to find collisions for the hash function. However, the use of randomized hashing may reduce the amount of security provided by a digital signature when all portions of the message are prepared by the signer.
>
> (NIST SP-800-106 “Randomized Hashing for Digital Signatures”)

In BLAKE2 the salt is processed as a one-time input to the hash function during initialization, rather than as an input to each compression function.

Aviso

Salted hashing (or just hashing) with BLAKE2 or any other general-purpose cryptographic hash function, such as SHA-256, is not suitable for hashing passwords. See BLAKE2 FAQ for more information.

```
>>> import os
>>> from hashlib import blake2b
>>> msg = b'some message'
>>> # Calculate the first hash with a random salt.
>>> salt1 = os.urandom(blake2b.SALT_SIZE)
>>> h1 = blake2b(salt=salt1)
>>> h1.update(msg)
>>> # Calculate the second hash with a different random salt.
>>> salt2 = os.urandom(blake2b.SALT_SIZE)
>>> h2 = blake2b(salt=salt2)
>>> h2.update(msg)
>>> # The digests are different.
>>> h1.digest() != h2.digest()
True
```

Personalization

Sometimes it is useful to force hash function to produce different digests for the same input for different purposes. Quoting the authors of the Skein hash function:

> We recommend that all application designers seriously consider doing this; we have seen many protocols where a hash that is computed in one part of the protocol can be used in an entirely different part because

> two hash computations were done on similar or related data, and the attacker can force the application to make the hash inputs the same. Personalizing each hash function used in the protocol summarily stops this type of attack.
>
> (The Skein Hash Function Family, p. 21)

BLAKE2 can be personalized by passing bytes to the *person* argument:

```
>>> from hashlib import blake2b
>>> FILES_HASH_PERSON = b'MyApp Files Hash'
>>> BLOCK_HASH_PERSON = b'MyApp Block Hash'
>>> h = blake2b(digest_size=32, person=FILES_HASH_PERSON)
>>> h.update(b'the same content')
>>> h.hexdigest()
'20d9cd024d4fb086aae819a1432dd2466de12947831b75c5a30cf2676095d3b4'
>>> h = blake2b(digest_size=32, person=BLOCK_HASH_PERSON)
>>> h.update(b'the same content')
>>> h.hexdigest()
'cf68fb5761b9c44e7878bfb2c4c9aea52264a80b75005e65619778de59f383a3'
```

Personalization together with the keyed mode can also be used to derive different keys from a single one.

```
>>> from hashlib import blake2s
>>> from base64 import b64decode, b64encode
>>> orig_key = b64decode(b'Rm5EPJai72qcK3RGBpW3vPNfZy5OZothY+kHY6h21KM=')
>>> enc_key = blake2s(key=orig_key, person=b'kEncrypt').digest()
>>> mac_key = blake2s(key=orig_key, person=b'kMAC').digest()
>>> print(b64encode(enc_key).decode('utf-8'))
rbPb15S/Z9t+agffno5wuhB77VbRi6F9Iv2qIxU7WHw=
>>> print(b64encode(mac_key).decode('utf-8'))
G9GtHFE1YluXY1zWPlYk1e/nWfu0WSEb0KRcjhDeP/o=
```

Modo árvore

Here's an example of hashing a minimal tree with two leaf nodes:

```
  10
 /  \
00  01
```

This example uses 64-byte internal digests, and returns the 32-byte final digest:

```
>>> from hashlib import blake2b
>>>
>>> FANOUT = 2
>>> DEPTH = 2
>>> LEAF_SIZE = 4096
>>> INNER_SIZE = 64
>>>
>>> buf = bytearray(6000)
>>>
>>> # Left leaf
... h00 = blake2b(buf[0:LEAF_SIZE], fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=0, node_depth=0, last_node=False)
>>> # Right leaf
... h01 = blake2b(buf[LEAF_SIZE:], fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=1, node_depth=0, last_node=True)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> # Root node
... h10 = blake2b(digest_size=32, fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=0, node_depth=1, last_node=True)
>>> h10.update(h00.digest())
>>> h10.update(h01.digest())
>>> h10.hexdigest()
'3ad2a9b37c6070e374c7a8c508fe20ca86b6ed54e286e93a0318e95e881db5aa'
```

Credits

BLAKE2 was designed by *Jean-Philippe Aumasson*, *Samuel Neves*, *Zooko Wilcox-O'Hearn*, and *Christian Winnerlein* based on SHA-3 finalist BLAKE created by *Jean-Philippe Aumasson*, *Luca Henzen*, *Willi Meier*, and *Raphael C.-W. Phan*.

It uses core algorithm from ChaCha cipher designed by *Daniel J. Bernstein*.

The stdlib implementation is based on pyblake2 module. It was written by *Dmitry Chestnykh* based on C implementation written by *Samuel Neves*. The documentation was copied from pyblake2 and written by *Dmitry Chestnykh*.

The C code was partly rewritten for Python by *Christian Heimes*.

The following public domain dedication applies for both C hash function implementation, extension code, and this documentation:

> To the extent possible under law, the author(s) have dedicated all copyright and related and neighboring rights to this software to the public domain worldwide. This software is distributed without any warranty.
>
> You should have received a copy of the CC0 Public Domain Dedication along with this software. If not, see https://creativecommons.org/publicdomain/zero/1.0/.

The following people have helped with development or contributed their changes to the project and the public domain according to the Creative Commons Public Domain Dedication 1.0 Universal:

- *Alexandr Sokolovskiy*

Ver também

Module ***hmac***
A module to generate message authentication codes using hashes.

Módulo ***base64***
Another way to encode binary hashes for non-binary environments.

https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.180-4.pdf
The FIPS 180-4 publication on Secure Hash Algorithms.

https://csrc.nist.gov/pubs/fips/202/final
The FIPS 202 publication on the SHA-3 Standard.

https://www.blake2.net/
Official BLAKE2 website.

https://en.wikipedia.org/wiki/Cryptographic_hash_function
Wikipedia article with information on which algorithms have known issues and what that means regarding their use.

https://www.ietf.org/rfc/rfc8018.txt
PKCS #5: Password-Based Cryptography Specification Version 2.1

https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-132.pdf
NIST Recommendation for Password-Based Key Derivation.

15.2 hmac — Código de autenticação de mensagem com chave hash

Código-fonte: Lib/hmac.py

Este módulo implementa o algoritmo HMAC conforme descrito por **RFC 2104**. A interface permite o uso de qualquer função hash com um tamanho de resumo *fixo*. Em particular, funções de saída extensíveis como SHAKE--128 ou SHAKE-256 não podem ser usadas com HMAC.

hmac.**new**(*key*, *msg=None*, *digestmod*)

Retorna um novo objeto hmac. *key* é um objeto bytes ou bytearray que fornece a chave secreta. Se *msg* estiver presente, a chamada de método `update(msg)` é feita. *digestmod* é o nome do resumo, construtor do resumo ou módulo para o objeto HMAC usar. Pode ser qualquer nome adequado para *hashlib.new()*. Apesar da posição do argumento, é obrigatório.

Alterado na versão 3.4: O parâmetro *key* pode ser um objeto bytes ou bytearray. O parâmetro *msg* pode ser de qualquer tipo suportado por *hashlib*. O parâmetro *digestmod* pode ser o nome de um algoritmo hash.

Alterado na versão 3.8: O argumento *digestmod* agora é obrigatório. Passe-o como um argumento nomeado para evitar constrangimentos quando você não tiver uma *msg* inicial.

hmac.**digest**(*key*, *msg*, *digest*)

Retorna o resumo de *msg* para o segredo *key* e *digest* fornecidos. A função equivale a `HMAC(key, msg, digest).digest()`, mas usa uma implementação C otimizada ou inline, que é mais rápida para mensagens que cabem na memória. Os parâmetros *key*, *msg* e *digest* têm o mesmo significado que em *new()*.

Detalhe da implementação do CPython: a implementação otimizada do C é usada somente quando *digest* é uma string e o nome de um algoritmo de resumo, que é suportado pelo OpenSSL.

Adicionado na versão 3.7.

class hmac.**HMAC**

Um objeto HMAC tem os seguintes métodos:

HMAC.**update**(*msg*)

Atualiza o objeto hmac com *msg*. Chamadas repetidas são equivalentes a uma única chamada com a concatenação de todos os argumentos: `m.update(a); m.update(b)` equivale a `m.update(a + b)`.

Alterado na versão 3.4: O parâmetro *msg* pode ser de qualquer tipo suportado por *hashlib*.

HMAC.**digest**()

Retorna o resumo dos bytes passados para o método *update()* até o momento. Este objeto bytes terá o mesmo comprimento que o *digest_size* do resumo dado ao construtor. Ele pode conter bytes não-ASCII, incluindo bytes NUL.

> **Aviso**
>
> Ao comparar a saída de *digest()* com um resumo fornecido externamente durante uma rotina de verificação, é recomendável usar a função *compare_digest()* em vez do operador == para reduzir a vulnerabilidade a ataques de temporização.

HMAC.**hexdigest**()

Como *digest()* exceto que o resumo é retornado como uma string com o dobro do comprimento contendo apenas dígitos hexadecimais. Isso pode ser usado para trocar o valor com segurança em e-mail ou outros ambientes não binários.

> **Aviso**
>
> Ao comparar a saída de *hexdigest()* com um resumo fornecido externamente durante uma rotina de verificação, é recomendável usar a função *compare_digest()* em vez do operador == para reduzir a vulnerabilidade a ataques de temporização.

HMAC.**copy**()

Retorna uma cópia ("clone") do objeto hmac. Isso pode ser usado para calcular eficientemente os resumos de strings que compartilham uma substring inicial comum.

Um objeto hash tem os seguintes atributos:

HMAC.**digest_size**

O tamanho do resumo HMAC resultante em bytes.

HMAC.**block_size**

O tamanho do bloco interna do algoritmo de hash em bytes.

Adicionado na versão 3.4.

HMAC.**name**

O nome canônico deste HMAC, sempre em letras minúsculas, por exemplo, `hmac-md5`.

Adicionado na versão 3.4.

Alterado na versão 3.10: Foram removidos os atributos não documentados `HMAC.digest_cons`, `HMAC.inner` e `HMAC.outer`.

Este módulo também fornece a seguinte função auxiliar:

hmac.**compare_digest**(*a*, *b*)

Retorna `a == b`. Esta função usa uma abordagem projetada para evitar análise de tempo evitando comportamento de curto-circuito baseado em conteúdo, tornando-a apropriada para criptografia. *a* e *b* devem ser ambos do mesmo tipo: *str* (somente ASCII, como, por exemplo, retornado por *HMAC.hexdigest()*), ou um *objeto bytes ou similar*.

> **Nota**
>
> Se *a* e *b* tiverem comprimentos diferentes, ou se ocorrer um erro, um ataque de temporização poderia teoricamente revelar informações sobre os tipos e comprimentos de *a* e *b*, mas não seus valores.

Adicionado na versão 3.3.

Alterado na versão 3.10: A função usa `CRYPTO_memcmp()` do OpenSSL internamente quando disponível.

> **Ver também**
>
> **Módulo *hashlib***
> O módulo Python que fornece funções hash seguras.

15.3 `secrets` — Gera números aleatórios seguros para gerenciar segredos

Adicionado na versão 3.6.

Código-fonte: Lib/secrets.py

O módulo *secrets* é usado para gerar números aleatórios criptograficamente fortes, adequados para o gerenciamento de dados, como senhas, autenticação de conta, tokens de segurança e segredos relacionados.

Em particular, *secrets* deve ser usado em preferência ao gerador de números pseudoaleatórios padrão no módulo *random*, que é projetado para modelagem e simulação, não segurança ou criptografia.

Ver também

PEP 506

15.3.1 Números aleatórios

O módulo *secrets* fornece acesso à fonte mais segura de aleatoriedade que seu sistema operacional fornece.

class secrets.**SystemRandom**

Uma classe para gerar números aleatórios usando as fontes da mais alta qualidade fornecidas pelo sistema operacional. Veja *random.SystemRandom* para detalhes adicionais.

secrets.**choice**(*seq*)

Retorna um elemento escolhido aleatoriamente de uma sequência não vazia.

secrets.**randbelow**(*exclusive_upper_bound*)

Retorna um int aleatório no intervalo [0, *exclusive_upper_bound*).

secrets.**randbits**(*k*)

Retorna um int não negativo com *k* bits aleatórios.

15.3.2 Gerando tokens

O módulo *secrets* fornece funções para gerar tokens seguros, adequados para aplicativos como redefinições de senha, URLs difíceis de adivinhar e semelhantes.

secrets.**token_bytes**([*nbytes=None*])

Retorna uma string de byte aleatória contendo *nbytes* número de bytes. Se *nbytes* for `None` ou não fornecido, um padrão razoável é usado.

```
>>> token_bytes(16)
b'\xebr\x17D*t\xae\xd4\xe3S\xb6\xe2\xebP1\x8b'
```

secrets.**token_hex**([*nbytes=None*])

Retorna uma string de texto aleatória, em hexadecimal. A string tem *nbytes* bytes aleatórios, cada byte convertido em dois dígitos hexadecimais. Se *nbytes* for `None` ou não fornecido, um padrão razoável é usado.

```
>>> token_hex(16)
'f9bf78b9a18ce6d46a0cd2b0b86df9da'
```

secrets.**token_urlsafe**([*nbytes=None*])

Retorna uma string de texto segura para URL aleatória, contendo *nbytes* bytes aleatórios. O texto é codificado em Base64, portanto, em média, cada byte resulta em aproximadamente 1,3 caracteres. Se *nbytes* for `None` ou não fornecido, um padrão razoável é usado.

```
>>> token_urlsafe(16)
'Drmhze6EPcv0fN_81Bj-nA'
```

Quantos bytes os tokens devem usar?

Para estar seguro contra ataques de força bruta, os tokens precisam ter aleatoriedade suficiente. Infelizmente, o que é considerado suficiente necessariamente aumentará à medida que os computadores ficarem mais poderosos e

capazes de fazer mais suposições em um período mais curto. A partir de 2015, acredita-se que 32 bytes (256 bits) de aleatoriedade são suficientes para o caso de uso típico esperado para o módulo *secrets*.

Para aqueles que desejam gerenciar seu próprio comprimento de token, você pode especificar explicitamente quanta aleatoriedade é usada para tokens, fornecendo um argumento *int* para as várias funções `token_*`. Esse argumento é considerado o número de bytes de aleatoriedade a serem usados.

Caso contrário, se nenhum argumento for fornecido, ou se o argumento for `None`, as funções `token_*` usarão um padrão razoável.

Nota

Esse padrão está sujeito a alterações a qualquer momento, inclusive durante as versões de manutenção.

15.3.3 Outras funções

`secrets.`**`compare_digest`**(*a*, *b*)

Retorna `True` se as strings ou *objetos tipo arquivo* *a* e *b* forem iguais, caso contrário, `False`, usando uma "comparação de tempo constante" para reduzir o risco de ataques de temporização. Veja *hmac.compare_digest()* para detalhes adicionais.

15.3.4 Receitas e melhores práticas

Esta seção mostra as receitas e melhores práticas para usar *secrets* para gerenciar um nível básico de segurança.

Gerar uma senha alfanumérica de oito caracteres:

```
import string
import secrets
alphabet = string.ascii_letters + string.digits
password = ''.join(secrets.choice(alphabet) for i in range(8))
```

Nota

As aplicações não devem **armazenar senhas em um formato recuperável**, seja em texto simples ou criptografado. Elas devem ser salgadas e transformadas em hash usando uma função hash de sentido único criptograficamente forte (irreversível).

Gerar uma senha alfanumérica de dez caracteres com pelo menos um caractere minúsculo, pelo menos um caractere maiúsculo e pelo menos três dígitos:

```
import string
import secrets
alphabet = string.ascii_letters + string.digits
while True:
    password = ''.join(secrets.choice(alphabet) for i in range(10))
    if (any(c.islower() for c in password)
            and any(c.isupper() for c in password)
            and sum(c.isdigit() for c in password) >= 3):
        break
```

Gerar uma senha longa do estilo XKCD:

```
import secrets
# Em sistemas Linux padrão, use um arquivo de dicionário conveniente.
# Outras plataformas podem precisar fornecer sua própria lista de palavras.
with open('/usr/share/dict/words') as f:
```

(continua na próxima página)

(continuação da página anterior)

```
    words = [word.strip() for word in f]
    password = ' '.join(secrets.choice(words) for i in range(4))
```

Gerar uma URL temporária difícil de adivinhação contendo um token de segurança adequado para aplicativos de recuperação de senha:

```
import secrets
url = 'https://example.com/reset=' + secrets.token_urlsafe()
```

CAPÍTULO 16

Serviços Genéricos do Sistema Operacional

Os módulos descritos neste capítulo fornecem interfaces aos recursos do sistema operacional e que estão disponíveis em (quase) todos os sistemas operacionais, como arquivos e um relógio. As interfaces geralmente são modeladas após as interfaces Unix ou C, mas elas também estão disponíveis na maioria dos outros sistemas. Aqui temos uma visão geral:

16.1 `os` — Diversas interfaces de sistema operacional

Código-fonte: Lib/os.py

Este módulo fornece uma maneira simples de usar funcionalidades que são dependentes de sistema operacional. Se você quiser ler ou escrever um arquivo, veja *open()*; se o que quer é manipular estruturas de diretórios, veja o módulo *os.path*; e se quiser ler todas as linhas, de todos os arquivos na linha de comando, veja o módulo *fileinput*. Para criar arquivos e diretórios temporários, veja o módulo *tempfile*; e, para manipulação em alto nível de arquivos e diretórios, veja o módulo *shutil*.

Notas sobre a disponibilidade dessas funções:

- O modelo dos módulos embutidos dependentes do sistema operacional no Python é tal que, desde que a mesma funcionalidade esteja disponível, a mesma interface é usada; por exemplo, a função `os.stat(path)` retorna informações estatísticas sobre *path* no mesmo formato (que é originado com a interface POSIX).
- Extensões específicas a um sistema operacional também estão disponíveis através do módulo *os*, mas usá-las é, naturalmente, uma ameaça à portabilidade.
- Todas as funções que aceitam nomes de caminhos ou arquivos, aceitam objetos bytes e string, e resultam em um objeto do mesmo tipo, se um caminho ou nome de arquivo for retornado.
- Em VxWorks, os.popen, os.fork, os.execv e os.spawn*p* não são suportados.
- Nas plataformas WebAssembly, Android e iOS, grandes partes do módulo *os* não estão disponíveis ou se comportam de maneira diferente. APIs relacionadas a processos (por exemplo, *fork()*, *execve()*) e recursos (por exemplo, *nice()*) não estão disponíveis. Outros como *getuid()* e *getpid()* são emulados ou esboços. As plataformas WebAssembly também não possuem suporte para sinais (por exemplo, *kill()*, *wait()*).

Nota

Todas as funções neste módulo levantam *OSError* (ou suas subclasses) no caso de nomes e caminhos de arquivos inválidos ou inacessíveis, ou outros argumentos que possuem o tipo correto, mas não são aceitos pelo sistema operacional.

exception os.**error**

Um apelido para a exceção embutida *OSError*.

os.**name**

O nome do módulo importado, dependente do sistema operacional. Atualmente, os seguintes nomes foram registrados: `'posix'`, `'nt'`, `'java'`.

Ver também

sys.platform tem uma granularidade mais fina. *os.uname()* fornece informações de versão específicas do sistema.

O módulo *platform* fornece verificações detalhadas sobre a identificação do sistema.

16.1.1 Nomes de arquivos, argumentos de linha de comando e variáveis de ambiente

No Python, nomes de arquivos, argumentos da linha de comando, e variáveis de ambiente são representadas usando o tipo string. Em alguns sistemas, decodificar essas strings de e para bytes é necessário antes de passá-las para o sistema operacional. Python usa o *tratador de erros e codificação do sistema de arquivos* para realizar essa conversão (veja *sys.getfilesystemencoding()*).

O *tratador de erros e codificação do sistema de arquivos* são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Alterado na versão 3.1: Em alguns sistemas a conversão, usando a codificação do sistema de arquivos, pode falhar. Neste caso, Python usa o *manipulador de erro de codificação surrogateescape*, o que significa que bytes não decodificados são substituídos por um caractere Unicode U+DC*xx* na decodificação, e estes são novamente traduzidos para o byte original na codificação.

A *codificação de sistema de arquivos* precisa garantir que todos os bytes abaixo de 128 serão decodificados com sucesso. Se a codificação do sistemas de arquivos falhar em garantir isso, funções de API podem levantar *UnicodeError*.

Veja também a *codificação da localidade*.

16.1.2 Modo UTF-8 do Python

Adicionado na versão 3.7: Veja **PEP 540** para mais detalhes.

O Modo UTF-8 do Python ignora a *codificação da localidade* e força o uso da codificação UTF-8:

- Usa UTF-8 como a *codificação do sistema de arquivos*.
- *sys.getfilesystemencoding()* retorna `'utf-8'`.
- *locale.getpreferredencoding()* retorna `'utf-8'` (o argumento *do_setlocale* não causa efeito).
- *sys.stdin*, *sys.stdout* e *sys.stderr* usam UTF-8 como codificação de texto, com o *tratador de erros* `surrogateescape` sendo habilitado para *sys.stdin* e *sys.stdout* (*sys.stderr* continua a usar `backslashreplace` como faz no modo padrão com reconhecimento de localidade)
- No Unix, *os.device_encoding()* retorna `'utf-8'` invés da codificação do dispositivo.

Observe que as configurações de fluxo padrão no modo UTF-8 podem ser substituídas por `PYTHONIOENCODING` (assim como podem estar no modo com reconhecimento de localidade padrão).

Como consequência das mudanças nessas APIs de baixo nível, outras APIs de alto nível também exibem comportamentos padrões diferentes:

- Argumentos de linha de comando, variáveis de ambiente e nomes de arquivos são decodificados em texto usando a codificação UTF-8.
- *`os.fsdecode()`* e *`os.fsencode()`* usam a codificação UTF-8.
- *`open()`*, *`io.open()`* e *`codecs.open()`* usam a codificação UTF-8 por padrão. No entanto, elas ainda usam o tratador de erros estrito por padrão, de modo que a tentativa de abrir um arquivo binário no modo de texto provavelmente levantará uma exceção em vez de produzir dados sem sentido.

O *Modo UTF-8 do Python* está habilitado se a localidade LC_CTYPE for `C` ou `POSIX` na inicialização do Python (veja a função `PyConfig_Read()`).

Ele pode ser habilitado ou desabilitado utilizando a opção `-X utf8` da linha de comando e a variável de ambiente `PYTHONUTF8`.

Se a variável de ambiente `PYTHONUTF8` não está com valor definido, então o interpretador usa por padrão a configuração de localidade atual, *a menos que* a localidade atual esteja identificada como uma localidade legada baseada em ASCII (como descrita para `PYTHONCOERCECLOCALE`), e a coerção de localidade estiver desabilitada ou falhar. Nessas localidades legadas, o interpretador irá habilitar por padrão o modo UTF-8 a menos que seja explicitamente instruído para não fazer isso.

O Modo UTF-8 do Python só pode ser ativado na inicialização do Python. Seu valor pode ser lido de *`sys.flags.utf8_mode`*.

Veja também o modo UTF-8 no Windows e o *tratador de erros e codificação do sistema de arquivos*.

Ver também

PEP 686
Python 3.15 utilizará *Modo UTF-8 do Python* por padrão.

16.1.3 Parâmetros de processo

Essas funções e itens de dados fornecem informações e operam no processo e usuário atuais.

`os.`**`ctermid`**`()`

Retorna o nome do arquivo correspondente ao terminal de controle do processo.

Disponibilidade: Unix, not WASI.

`os.`**`environ`**

Um objeto *mapeamento* onde as chaves e valores são strings que representam o ambiente do processo. Por exemplo, `environ['HOME']` é o nome do caminho do seu diretório pessoal (em algumas plataformas), e é equivalente a `getenv("HOME")` em C.

Este mapeamento é capturado na primeira vez que o módulo *`os`* é importado, normalmente durante a inicialização do Python, como parte do processamento do arquivo `site.py`. Mudanças no ambiente feitas após esse momento não são refletidas em *`os.environ`*, exceto pelas mudanças feitas modificando *`os.environ`* diretamente.

Este mapeamento pode ser usado para modificar o ambiente, além de consultá-lo. *`putenv()`* será chamado automaticamente quando o mapeamento for modificado.

No Unix, chaves e valores usam *`sys.getfilesystemencoding()`* e o tratador de erros `'surrogateescape'`. Use *`environb`* se quiser usar uma codificação diferente.

No Windows, as chaves são convertidas em maiúsculas. Isso também se aplica ao obter, definir ou excluir um item. Por exemplo, `environ['monty'] = 'python'` mapeia a chave `'MONTY'` para o valor `'python'`.

Nota

Chamar a função *putenv()* diretamente não muda *os.environ*, por isso é melhor modificar *os.environ*.

Nota

Em algumas plataformas, incluindo FreeBSD e Mac OS X, a modificação de `environ` pode causar vazamentos de memória. Consulte a documentação do sistema para `putenv()`.

Você pode excluir itens neste mapeamento para remover definição de variáveis de ambiente. *unsetenv()* será chamado automaticamente quando um item é excluído de *os.environ*, e quando um dos métodos `pop()` ou `clear()` é chamado.

Ver também

A função *os.reload_environ()*.

Alterado na versão 3.9: Atualizado para ter suporte aos operadores de mesclagem (`|`) e de atualização (`|=`) da **PEP 584**.

`os.`**`environb`**

Versão bytes de *environ*: um objeto de *mapeamento* onde as chaves e valores são objetos *bytes* representando o ambiente do processo. *environ* e *environb* são sincronizados (modificar *environb* atualiza *environ*, e vice versa).

environb está disponível somente se *supports_bytes_environ* for `True`.

Adicionado na versão 3.2.

Alterado na versão 3.9: Atualizado para ter suporte aos operadores de mesclagem (`|`) e de atualização (`|=`) da **PEP 584**.

`os.`**`reload_environ`**`()`

Os mapeamentos *os.environ* e *os.environb* são um cache de variáveis de ambiente no momento em que o Python foi iniciado. Dessa forma, alterações no ambiente do processo atual não são refletidas se feitas fora do Python, ou por *os.putenv()* ou *os.unsetenv()*. Use `os.reload_environ()` para atualizar *os.environ* e *os.environb* com quaisquer alterações no ambiente do processo atual.

Aviso

Esta função não é segura para thread. Chamá-la enquanto o ambiente está sendo modificado em outra thread é um comportamento indefinido. Ler de *os.environ* ou *os.environb*, ou chamar *os.getenv()* durante o recarregamento, pode retornar um resultado vazio.

Adicionado na versão 3.14.

`os.`**`chdir`**`(`*path*`)`

`os.`**`fchdir`**`(`*fd*`)`

`os.`**`getcwd`**`()`

Essas funções são descritas em: *Arquivos e diretórios*.

`os.`**`fsencode`**`(`*filename*`)`

Codifica o *filename* *caminho ou similar* para *tratador de erros e codificação do sistema de arquivos*; retorna *bytes* inalterados.

fsdecode() é a função inversa.

Adicionado na versão 3.2.

Alterado na versão 3.6: Adicionado suporte para aceitar objetos que implementam a interface *os.PathLike*.

os.**fsdecode**(*filename*)

Decodifica o *objeto caminho ou similar* *filename* do *tratador de erros e codificação do sistema de arquivos*; retorna *str* inalterada.

fsencode() é a função inversa.

Adicionado na versão 3.2.

Alterado na versão 3.6: Adicionado suporte para aceitar objetos que implementam a interface *os.PathLike*.

os.**fspath**(*path*)

Retorna a representação do sistema de arquivos do caminho (argumento *path*).

Se um objeto *str* ou *bytes* é passado, é retornado inalterado. Caso contrário, o método *__fspath__()* é chamado, e seu valor é retornado, desde que seja um objeto *str* ou *bytes*. Em todos os outros casos, *TypeError* é levantada.

Adicionado na versão 3.6.

class os.**PathLike**

Uma *classe base abstrata* para objetos representando um caminho do sistema de arquivos, como, por exemplo, *pathlib.PurePath*.

Adicionado na versão 3.6.

abstractmethod **__fspath__**()

Retorna a representação do caminho do sistema de arquivos do objeto.

O método deverá retornar somente objetos *str* ou *bytes*, preferencialmente *str*.

os.**getenv**(*key*, *default=None*)

Retorna o valor da variável de ambiente *key* como uma string se existir, ou *default* se não existir. *key* é uma string. Note que uma vez que *getenv()* utiliza *os.environ*, o mapeamento de *getenv()* é também similarmente capturado na importação, e a função pode não refletir futuras mudanças no ambiente.

No Unix, chaves e valores são decodificados com a função *sys.getfilesystemencoding()* e o o manipulador de erros `'surrogateescape'`. Use *os.getenvb()* se quiser usar uma codificação diferente.

Disponibilidade: Unix, Windows.

os.**getenvb**(*key*, *default=None*)

Retorna o valor da variável de ambiente *key* como bytes se existir, ou *default* se não existir. *key* deve ser bytes. Note que uma vez que *getenvb()* utiliza *os.environb*, o mapeamento de *getenvb()* é também similarmente capturado na importação, e a função pode não refletir futuras mudanças no ambiente.

getenvb() está disponível somente se *supports_bytes_environ* for `True`.

Disponibilidade: Unix.

Adicionado na versão 3.2.

os.**get_exec_path**(*env=None*)

Retorna a lista de diretórios que serão buscados por um executável nomeado, semelhante a um shell, ao iniciar um processo. *env*, quando especificado, deve ser um dicionário de variáveis de ambiente para procurar o PATH. Por padrão, quando *env* é `None`, *environ* é usado.

Adicionado na versão 3.2.

os.**getegid**()

Retorna o ID do grupo efetivo do processo atual. Isso corresponde ao bit "set id" no arquivo que está sendo executado no processo atual.

Disponibilidade: Unix, not WASI.

os.**geteuid**()

Retorna o ID de usuário efetivo do processo atual.

Disponibilidade: Unix, not WASI.

os.**getgid**()

Retorna o ID do grupo real do processo atual.

Disponibilidade: Unix.

A função é uma simulação em WASI, veja *Plataformas WebAssembly* para mais informações.

os.**getgrouplist**(*user*, *group*, */*)

Retorna a lista de IDs de grupo aos quais *user* pertence. Se *group* não estiver na lista, ele será incluído; normalmente, *group* é especificado como o campo de ID de grupo do registro de senha para *user*, porque esse ID de grupo será potencialmente omitido.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.**getgroups**()

Retorna a lista de IDs de grupos suplementares associados ao processo atual.

Disponibilidade: Unix, not WASI.

Nota

No Mac OS, o comportamento da função *getgroups()* difere um pouco de outras plataformas Unix. Se o interpretador Python foi compilado para distribuição na versão `10.5`, ou anterior, *getgroups()* retorna a lista de IDs de grupos efetivos, associados ao processo do usuário atual; esta lista é limitada a um número de entradas definido pelo sistema, tipicamente 16, e pode ser modificada por chamadas para *setgroups()* se tiver o privilégio adequado. Se foi compilado para distribuição na versão maior que `10.5`, *getgroups()* retorna a lista de acesso de grupo atual para o usuário associado ao ID de usuário efetivo do processo; a lista de acesso de grupo pode mudar durante a vida útil do processo, e ela não é afetada por chamadas para *setgroups()*, e seu comprimento não é limitado a 16. O valor da constante `MACOSX_DEPLOYMENT_TARGET`, pode ser obtido com *sysconfig.get_config_var()*.

os.**getlogin**()

Retorna o nome do usuário conectado no terminal de controle do processo. Para a maioria dos propósitos, é mais útil usar *getpass.getuser()*, já que esse último verifica as variáveis de ambiente `LOGNAME` ou `USERNAME` para descobrir quem é o usuário, e retorna para `pwd.getpwuid(os.getuid())[0]` para obter o nome de login do ID do usuário real atual.

Disponibilidade: Unix, Windows, not WASI.

os.**getpgid**(*pid*)

Retorna o ID do grupo de processo com *pid*. Se *pid* for 0, o ID do grupo do processos do processo atual é retornado.

Disponibilidade: Unix, not WASI.

os.**getpgrp**()

Retorna o ID do grupo do processo atual.

Disponibilidade: Unix, not WASI.

os.**getpid**()

Retorna o ID do processo atual.

A função é uma simulação em WASI, veja *Plataformas WebAssembly* para mais informações.

os.**getppid**()

Retorna o ID do processo pai. Quando o processo pai é encerrado, no Unix, o ID retornado é o do processo init (1); no Windows, ainda é o mesmo ID, que pode já estar sendo reutilizado por outro processo.

Disponibilidade: Unix, Windows, not WASI.

Alterado na versão 3.2: Adicionado suporte para Windows.

os.**getpriority**(*which*, *who*)

Obtém prioridade de agendamento de programa. O valor *which* é um entre *PRIO_PROCESS*, *PRIO_PGRP* ou *PRIO_USER*, e *who* é interpretado em relação a *which* (um identificador de processo para *PRIO_PROCESS*, identificador do grupo de processos para *PRIO_PGRP* e um ID de usuário para *PRIO_USER*). Um valor zero para *who* indica (respectivamente) o processo de chamada, o grupo de processos do processo de chamada ou o ID do usuário real do processo de chamada.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.**PRIO_PROCESS**

os.**PRIO_PGRP**

os.**PRIO_USER**

Parâmetros para as funções *getpriority()* e *setpriority()*.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.**PRIO_DARWIN_THREAD**

os.**PRIO_DARWIN_PROCESS**

os.**PRIO_DARWIN_BG**

os.**PRIO_DARWIN_NONUI**

Parâmetros para as funções *getpriority()* e *setpriority()*.

Disponibilidade: macOS

Adicionado na versão 3.12.

os.**getresuid**()

Retorna uma tupla (ruid, euid, suid) indicando os IDs de usuário real, efetivo e salvo do processo atual.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.2.

os.**getresgid**()

Retorna uma tupla (rgid, egid, sgid) indicando os IDs de grupos real, efetivo e salvo do processo atual.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.2.

os.**getuid**()

Retorna o ID de usuário real do processo atual.

Disponibilidade: Unix.

A função é uma simulação em WASI, veja *Plataformas WebAssembly* para mais informações.

os.**initgroups**(*username*, *gid*, */*)

Chama o initgroups() do sistema para inicializar a lista de acesso ao grupo com todos os grupos dos quais o nome de usuário especificado é membro, mais o ID do grupo especificado.

Disponibilidade: Unix, not WASI, not Android.

Adicionado na versão 3.2.

os.**putenv**(*key*, *value*, /)

Define a variável de ambiente denominada *key* como a string *value*. Tais mudanças no ambiente afetam os subprocessos iniciados com *os.system()*, *popen()* ou *fork()* e *execv()*.

Atribuições a itens em *os.environ* são automaticamente traduzidas em chamadas correspondentes para *putenv()*; entretanto, chamadas para *putenv()* não atualizam *os.environ*, então é preferível atribuir itens de *os.environ*. Isso também se aplica a *getenv()* e *getenvb()*, que usam respectivamente *os.environ* e *os.environb* em suas implementações.

Veja também a função *os.reload_environ()*.

Nota

Em algumas plataformas, incluindo FreeBSD e Mac OS X, a modificação de `environ` pode causar vazamentos de memória. Consulte a documentação do sistema para `putenv()`.

Levanta um *evento de auditoria* `os.putenv` com os argumentos `key`, `value`.

Alterado na versão 3.9: A função está agora sempre disponível.

os.**setegid**(*egid*, /)

Define o ID do grupo efetivo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.**seteuid**(*euid*, /)

Define o ID do usuário efetivo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.**setgid**(*gid*, /)

Define o ID do grupo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.**setgroups**(*groups*, /)

Define a lista de IDs de grupo suplementares associados ao processo atual como *groups*. *groups* deve ser uma sequência e cada elemento deve ser um número inteiro identificando um grupo. Esta operação normalmente está disponível apenas para o superusuário.

Disponibilidade: Unix, not WASI.

Nota

No Mac OS X, o comprimento de *groups* não pode exceder o número máximo definido pelo sistema de IDs de grupo efetivos, normalmente 16. Consulte a documentação de *getgroups()* para casos em que ele pode não retornar o mesmo grupo de listas definido chamando setgroups().

os.**setns**(*fd*, *nstype=0*)

Reassocia a thread atual com um espaço de nomes do Linux. Veja as páginas man *setns(2)* e *namespaces(7)* para mais detalhes.

Se *fd* se refere a um link `/proc/pid/ns/`, `setns()` reassocia a thread de chamada com o espaço de nomes associado a esse link, e *nstype* pode ser definido como uma das *constantes CLONE_NEW** para impor restrições na operação (`0` significa sem restrições).

Desde o Linux 5.8, *fd* pode se referir a um descritor de arquivo PID obtido de *pidfd_open()*. Neste caso, `setns()` reassocia a thread chamadora em um ou mais dos mesmos espaços de nomes que o thread referenciado por *fd*. Isso está sujeito a quaisquer restrições impostas por *nstype*, que é uma máscara de bits que combina uma ou mais das *constantes CLONE_NEW**, por exemplo `setns(fd, os.CLONE_NEWUTS |`

`os.CLONE_NEWPID`). As associações do chamador em espaços de nomes não especificados permanecem inalteradas.

fd pode ser qualquer objeto com um método *fileno()* ou um descritor de arquivo bruto.

Este exemplo reassocia a thread com o espaço de nomes de rede do processo `init`:

```
fd = os.open("/proc/1/ns/net", os.O_RDONLY)
os.setns(fd, os.CLONE_NEWNET)
os.close(fd)
```

Disponibilidade: Linux >= 3.0 with glibc >= 2.14.

Adicionado na versão 3.12.

Ver também

A função *unshare()*.

os.setpgrp()

Executa a chamada de sistema `setpgrp()` ou `setpgrp(0, 0)` dependendo da versão implementada (se houver). Veja o manual do Unix para a semântica.

Disponibilidade: Unix, not WASI.

os.setpgid(*pid*, *pgrp*, /)

Executa a chamada de sistema `setpgid()` para definir o ID do grupo do processo com *pid* para o grupo de processos com o id *pgrp*. Veja o manual do Unix para a semântica.

Disponibilidade: Unix, not WASI.

os.setpriority(*which*, *who*, *priority*)

Define a prioridade de agendamento de programa. O valor *which* é um entre *PRIO_PROCESS*, *PRIO_PGRP* ou *PRIO_USER*, e *who* é interpretado em relação a *which* (um identificador de processo para *PRIO_PROCESS*, identificador do grupo de processos para *PRIO_PGRP* e um ID de usuário para *PRIO_USER*). Um valor zero para *who* indica (respectivamente) o processo de chamada, o grupo de processos do processo de chamada ou o ID do usuário real do processo de chamada. *priority* é um valor na faixa de -20 a 19. A prioridade padrão é 0; prioridades menores resultam em um agendamento mais favorável.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.setregid(*rgid*, *egid*, /)

Define os IDs de grupo real e efetivo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.setresgid(*rgid*, *egid*, *sgid*, /)

Define os IDs de grupo real, efetivo e salvo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

Adicionado na versão 3.2.

os.setresuid(*ruid*, *euid*, *suid*, /)

Define os IDs de usuário real, efetivo e salvo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

Adicionado na versão 3.2.

os.**setreuid**(*ruid*, *euid*, /)

Define os IDs de usuário real e efetivo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.**getsid**(*pid*, /)

Executa a chamada de sistema `getsid()`. Veja o manual do Unix para semântica.

Disponibilidade: Unix, not WASI.

os.**setsid**()

Executa a chamada de sistema `setsid()`. Veja o manual do Unix para semântica.

Disponibilidade: Unix, not WASI.

os.**setuid**(*uid*, /)

Define o ID de usuário do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.**strerror**(*code*, /)

Retorna a mensagem de erro correspondente ao código de erro em *code*. Nas plataformas em que `strerror()` retorna `NULL` quando recebe um número de erro desconhecido, *`ValueError`* é levantada.

os.**supports_bytes_environ**

`True` se o tipo de sistema operacional nativo do ambiente estiver em bytes (por exemplo, `False` no Windows).

Adicionado na versão 3.2.

os.**umask**(*mask*, /)

Define o umask numérico atual e retorna o umask anterior.

A função é uma simulação em WASI, veja *Plataformas WebAssembly* para mais informações.

os.**uname**()

Retorna informações identificando o sistema operacional atual. O valor retornado é um objeto com cinco atributos:

- `sysname` - nome do sistema operacional
- `nodename` - nome da máquina na rede (definido pela implementação)
- `release` - lançamento do sistema operacional
- `version` - versão do sistema operacional
- `machine` - identificador de hardware

Para compatibilidade com versões anteriores, esse objeto também é iterável, comportando-se como uma tupla de 5 elementos contendo `sysname`, `nodename`, `release`, `version` e `machine` nessa ordem.

Alguns sistemas truncam `nodename` para 8 caracteres ou para o componente principal; uma maneira melhor de obter o nome do host é *`socket.gethostname()`* ou até mesmo `socket.gethostbyaddr(socket.gethostname())`.

No macOS, iOS e Android, retorna o nome e a versão do *kernel* (ou seja, `'Darwin'` no macOS e iOS; `'Linux'` no Android). *`platform.uname()`* pode ser usada para obter o nome e a versão do sistema operacional voltado para o usuário no iOS e Android.

Disponibilidade: Unix.

Alterado na versão 3.3: Tipo de retorno foi alterado de uma tupla para um objeto tupla ou similar com atributos nomeados.

os.**unsetenv**(*key*, */*)

Cancela (exclui) a variável de ambiente denominada *key*. Tais mudanças no ambiente afetam subprocessos iniciados com `os.system()`, `popen()` ou `fork()` e `execv()`.

A exclusão de itens em `os.environ` é automaticamente traduzida para uma chamada correspondente a `unsetenv()`; no entanto, chamadas a `unsetenv()` não atualizam `os.environ`, por isso, na verdade é preferível excluir itens de `os.environ`.

Veja também a função `os.reload_environ()`.

Levanta um *evento de auditoria* `os.unsetenv` com o argumento `key`.

Alterado na versão 3.9: A função está agora sempre disponível e também está disponível no Windows.

os.**unshare**(*flags*)

Desassocia partes do contexto de execução do processo e move-as para um espaço de nomes recém-criado. Veja a página man `unshare(2)` para mais detalhes. O argumento *flags* é uma máscara de bits, combinando zero ou mais *constantes CLONE_**, que especifica quais partes do contexto de execução devem ser descompartilhadas de suas associações existentes e movidas para um novo espaço de nomes. Se o argumento *flags* for `0`, nenhuma alteração será feita no contexto de execução do processo de chamada.

Disponibilidade: Linux >= 2.6.16.

Adicionado na versão 3.12.

Ver também

A função `setns()`.

Sinalizações para a função `unshare()`, se a implementação oferecer suporte. Veja `unshare(2)` no manual do Linux para seu exato efeito e disponibilidade.

os.**CLONE_FILES**

os.**CLONE_FS**

os.**CLONE_NEWCGROUP**

os.**CLONE_NEWIPC**

os.**CLONE_NEWNET**

os.**CLONE_NEWNS**

os.**CLONE_NEWPID**

os.**CLONE_NEWTIME**

os.**CLONE_NEWUSER**

os.**CLONE_NEWUTS**

os.**CLONE_SIGHAND**

os.**CLONE_SYSVSEM**

os.**CLONE_THREAD**

os.**CLONE_VM**

16.1.4 Criação de objetos arquivos

Estas funções criam novos *objetos arquivos*. (Veja também `open()` para abrir os descritores de arquivos.)

os.**fdopen**(*fd*, **args*, ***kwargs*)

Retorna um objeto arquivo aberto conectado ao descritor de arquivo *fd*. Este é um apelido para a função embutida `open()` e aceita os mesmos argumentos. A única diferença é que o primeiro argumento de `fdopen()` deve ser sempre um inteiro.

16.1.5 Operações dos descritores de arquivos

Estas funções operam em fluxos de E/S referenciados usando descritores de arquivos.

Descritores de arquivos são pequenos números inteiros correspondentes a um arquivo que foi aberto pelo processo atual. Por exemplo, a entrada padrão geralmente é o descritor de arquivo 0, a saída padrão 1 e erro padrão 2. Outros arquivos abertos por um processo serão então atribuídos 3, 4, 5, e assim por diante. O nome "descritor de arquivo" é um pouco enganoso; em plataformas UNIX, sockets e pipes também são referenciados como descritores de arquivos.

O método `fileno()` pode ser usado para obter o descritor de arquivo associado a um *objeto arquivo* quando solicitado. Note-se que usar o descritor de arquivo diretamente ignorará os métodos do objeto arquivo, ignorando aspectos como buffer interno de dados.

`os.close`(*fd*)

Fecha o descritor de arquivo *fd*.

> **Nota**
>
> Esta função destina-se a E/S de baixo nível e deve ser aplicada a um descritor de arquivo retornado por `os.open()` ou `pipe()`. Para fechar um "objeto arquivo" retornado pela função embutida `open()` ou por `popen()` ou `fdopen()`, use seu método `close()`.

`os.closerange`(*fd_low*, *fd_high*, */*)

Fecha todos os descritores de arquivos de *fd_low* (inclusivo) a *fd_high* (exclusivo), ignorando erros. Equivalente a (mas muito mais rápido do que):

```
for fd in range(fd_low, fd_high):
    try:
        os.close(fd)
    except OSError:
        pass
```

`os.copy_file_range`(*src*, *dst*, *count*, *offset_src=None*, *offset_dst=None*)

Copia *count* bytes do descritor de arquivo *src*, começando no deslocamento *offset_src*, para o descritor de arquivo *dst*, começando no deslocamento *offset_dst*. Se *offset_src* for `None`, *src* será lido a partir da posição atual; respectivamente para *offset_dst*.

No kernel Linux anterior a 5.3, os arquivos apontados por *src* e *dst* devem residir no mesmo sistema de arquivos, caso contrário, uma exceção `OSError` é levantada com `errno` definido como `errno.EXDEV`.

Essa cópia é feita sem o custo adicional de transferência de dados do kernel para o espaço do usuário e depois de volta para o kernel. Além disso, alguns sistemas de arquivos podem implementar otimizações extras, como o uso de reflinks (ou seja, dois ou mais nós-i que compartilham ponteiros para os mesmos blocos de disco copy-on-write; os sistemas de arquivos suportados incluem btrfs e XFS) e cópia do lado do servidor (no caso de NFS).

A função copia bytes entre dois descritores de arquivo. As opções de texto, como a codificação e o final da linha, são ignoradas.

O valor de retorno é a quantidade de bytes copiados. Ele pode ser inferior à quantidade solicitada.

> **Nota**
>
> No Linux, `os.copy_file_range()` não deve ser usada para copiar um intervalo de um pseudo arquivo de um sistema de arquivos especial como procfs e sysfs. Ele nunca copiará byte algum e retornará 0 como se o arquivo estivesse vazio devido a um problema conhecido do kernel do Linux.

Disponibilidade: Linux >= 4.5 with glibc >= 2.27.

Adicionado na versão 3.8.

os.**device_encoding**(*fd*)

Retorna uma string descrevendo a codificação do dispositivo associado a *fd* se estiver conectado a um terminal; senão retorna `None`.

No Unix, se o *Modo UTF-8 do Python* estiver habilitado, retorna `'UTF-8'` ao invés da codificação do dispositivo.

Alterado na versão 3.10: No Unix, a função agora implementa o Modo UTF-8 do Python.

os.**dup**(*fd*, */*)

Retorna uma cópia do descritor de arquivo *fd*. O novo descritor de arquivo é *não-herdável*.

No Windows, ao duplicar um fluxo padrão (0: stdin, 1: stdout, 2: stderr), o novo descritor de arquivo é *herdável*.

Disponibilidade: not WASI.

Alterado na versão 3.4: O novo descritor de arquivo agora é não-herdável.

os.**dup2**(*fd*, *fd2*, *inheritable=True*)

Duplica o descritor de arquivo *fd* como *fd2*, fechando o último antes disso, se necessário. Retorna *fd2*. O novo descritor de arquivo é *herdável* por padrão ou não-herdável se *inheritable* for `False`.

Disponibilidade: not WASI.

Alterado na versão 3.4: Adicionado o parâmetro opcional *inheritable*.

Alterado na versão 3.7: Retorna *fd2* em caso de sucesso. Anteriormente, retornava sempre `None`.

os.**fchmod**(*fd*, *mode*)

Altera o modo do arquivo dado por *fd* ao *mode* numérico. Veja a documentação de `chmod()` para valores possíveis de *mode*. A partir do Python 3.3, isto é equivalente a `os.chmod(fd, mode)`.

Levanta um *evento de auditoria* `os.chmod` com os argumentos `path`, `mode`, `dir_fd`.

Disponibilidade: Unix, Windows.

A função é limitada em WASI, veja *Plataformas WebAssembly* para mais informações.

Alterado na versão 3.13: Adicionado suporte no Windows.

os.**fchown**(*fd*, *uid*, *gid*)

Altera o ID do proprietário e do grupo do arquivo dado por *fd* para o *uid* e *gid* numérico. Para deixar um dos IDs inalteradas, defina-o como -1. Veja `chown()`. A partir do Python 3.3, isto é equivalente a `os.chown(fd, uid, gid)`.

Levanta um *evento de auditoria* `os.chown` com os argumentos `path`, `uid`, `gid`, `dir_fd`.

Disponibilidade: Unix.

A função é limitada em WASI, veja *Plataformas WebAssembly* para mais informações.

os.**fdatasync**(*fd*)

Força a gravação do arquivo com descritor de arquivo *fd* no disco. Não força a atualização de metadados.

Disponibilidade: Unix.

> **Nota**
>
> Esta função não está disponível no MacOS.

os.**fpathconf**(*fd*, *name*, */*)

Retorna informações de configuração de sistema relevantes para um arquivo aberto. *name* especifica o valor de configuração para recuperar; pode ser uma string que é o nome de um valor do sistema definido; estes nomes são especificados em uma série de padrões (POSIX.1, Unix 95, Unix 98 e outros). Algumas plataformas definem nomes adicionais também. Os nomes conhecidos do sistema operacional hospedeiro são dadas no dicionário

`pathconf_names`. Para variáveis de configuração não incluídas neste mapeamento, também é aceito passar um número inteiro para *name*.

Se *name* for uma string e não for conhecida, uma exceção *ValueError* é levantada. Se um valor específico para *name* não for compatível com o sistema hospedeiro, mesmo que seja incluído no `pathconf_names`, uma exceção *OSError* é levantada com *errno.EINVAL* como número do erro.

A partir do Python 3.3, é equivalente a `os.pathconf(fd, name)`.

Disponibilidade: Unix.

os.fstat(*fd*)

Captura o estado do descritor de arquivo *fd*. Retorna um objeto *stat_result*.

A partir do Python 3.3, é equivalente a `os.stat(fd)`.

Ver também

A função *stat()*.

os.fstatvfs(*fd*, /)

Retorna informações sobre o sistema de arquivos que contém o arquivo associado com descritor de arquivo *fd*, como *statvfs()*. A partir do Python 3.3, isso equivale a `os.statvfs(fd)`.

Disponibilidade: Unix.

os.fsync(*fd*)

Força a gravação no disco de arquivo com descritor de arquivo *fd*. No Unix, isto chama a função nativa `fsync()`; no Windows, a função de MS `_commit()`.

Se você estiver começando com um *objeto arquivo* *f* em buffer do Python, primeiro use `f.flush()`, e depois use `os.fsync(f.fileno())`, para garantir que todos os buffers internos associados com *f* sejam gravados no disco.

Disponibilidade: Unix, Windows.

os.ftruncate(*fd*, *length*, /)

Trunca o arquivo correspondente ao descritor de arquivo *fd*, de modo que tenha no máximo *length* bytes de tamanho. A partir do Python 3.3, isto é equivalente a `os.truncate(fd, length)`.

Levanta *evento de auditoria* `os.truncate` com os argumentos `fd`, `length`.

Disponibilidade: Unix, Windows.

Alterado na versão 3.5: Adicionado suporte para o Windows.

os.get_blocking(*fd*, /)

Obtém o modo de bloqueio do descritor de arquivo: `False` se o sinalizador *O_NONBLOCK* estiver marcado, `True` se o sinalizador estiver desmarcado.

Veja também *set_blocking()* e *socket.socket.setblocking()*.

Disponibilidade: Unix, Windows.

A função é limitada em WASI, veja *Plataformas WebAssembly* para mais informações.

No Windows, esta função é limitada a encadeamentos.

Adicionado na versão 3.5.

Alterado na versão 3.12: Adicionado suporte a encadeamentos no Windows.

os.grantpt(*fd*, /)

Concede acesso ao dispositivo pseudoterminal secundário associado ao dispositivo pseudoterminal principal ao qual o descritor de arquivo *fd* se refere. O descritor de arquivo *fd* não é fechado em caso de falha.

Chama a função `grantpt()` da biblioteca padrão C.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.13.

os.**isatty**(*fd*, /)

Retorna `True` se o descritor de arquivo *fd* estiver aberto e conectado a um dispositivo do tipo tty, senão `False`.

os.**lockf**(*fd*, *cmd*, *len*, /)

Aplica, testa ou remove uma trava POSIX em um descritor de arquivo aberto. *fd* é um descritor de arquivo aberto. *cmd* especifica o comando a ser usado - um dentre *F_LOCK*, *F_TLOCK*, *F_ULOCK* ou *F_TEST*. *len* especifica a seção do arquivo para travar.

Levanta um *evento de auditoria* `os.lockf` com os argumentos `fd`, `cmd`, `len`.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.**F_LOCK**

os.**F_TLOCK**

os.**F_ULOCK**

os.**F_TEST**

Sinalizadores que especificam qual ação *lockf()* vai executar.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.**login_tty**(*fd*, /)

Prepara o tty do qual fd é um descritor de arquivo para uma nova sessão de login. Faz do processo de chamada um líder de sessão; torna o tty o tty de controle, o stdin, o stdout e o stderr do processo de chamada; fecha o fd.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.11.

os.**lseek**(*fd*, *pos*, *whence*, /)

Define a posição atual do descritor de arquivo *fd* para a posição *pos*, modificada por *whence*, e retorna a nova posição em bytes relativa ao início do arquivo. Os valores válidos de *whence* são:

- *SEEK_SET* ou `0` – define *pos* em relação ao início do arquivo
- *SEEK_CUR* ou `1` – define *pos* em relação à posição atual do arquivo
- *SEEK_END* ou `2` – define *pos* em relação ao final do arquivo
- *SEEK_HOLE* – define *pos* para o próximo local de dados, em relação a *pos*
- *SEEK_DATA* – define *pos* para o próximo buraco de dados, em relação a *pos*

Alterado na versão 3.3: Adicionado suporte para `SEEK_HOLE` e `SEEK_DATA`.

os.**SEEK_SET**

os.**SEEK_CUR**

os.**SEEK_END**

Parâmetros para a função *lseek()* e o método *seek()* em *objetos arquivo ou similares*, para onde ajustar o indicador de posição do arquivo.

SEEK_SET
Ajusta a posição do arquivo em relação ao início do arquivo.

SEEK_CUR
Ajusta a posição do arquivo em relação a sua posição atual.

SEEK_END
Ajusta a posição do arquivo em relação ao fim do arquivo.

Seus valores são 0, 1 e 2, respectivamente.

os.**SEEK_HOLE**

os.**SEEK_DATA**

Parâmetros para a função *lseek()* e o método *seek()* em *objetos arquivo ou similares*, para para buscar dados de arquivos e buracos em arquivos alocados de forma esparsa.

SEEK_DATA
Ajusta o deslocamento do arquivo para o próximo local contendo dados, em relação à posição de busca.

SEEK_HOLE
Ajusta o deslocamento do arquivo para o próximo local contendo um buraco, em relação à posição de busca. Um buraco é definido como uma sequência de zeros.

> **Nota**
>
> Essas operações só fazem sentido para sistemas de arquivos que as suportam.

Disponibilidade: Linux >= 3.1, macOS, Unix

Adicionado na versão 3.3.

os.**open**(*path*, *flags*, *mode=0o777*, ***, *dir_fd=None*)

Abre o arquivo *path* e define vários sinalizadores de acordo com *flags* e, possivelmente, seu modo, de acordo com *mode*. Ao computar *mode*, o valor atual de umask é primeiro mascarado. Retorna o descritor de arquivo para o arquivo recém-aberto. O novo descritor de arquivo é *não-herdável*.

Para ler uma descrição dos valores de sinalizadores e modos, veja a documentação de C; constantes de sinalizador (como *O_RDONLY* e *O_WRONLY*) são definidas no módulo *os*. Em particular, no Windows é necessário adicionar *O_BINARY* para abrir arquivos em modo binário.

Esta função oferece suporte para *caminhos relativos aos descritores de diretório* com o parâmetro *dir_fd*.

Levanta um *evento de auditoria* `open` com os argumentos `path`, `mode`, `flags`.

Alterado na versão 3.4: O novo descritor de arquivo agora é não-herdável.

> **Nota**
>
> Esta função é destinada a E/S de baixo nível. Para uso normal, use a função embutida *open()*, que retorna um *objeto arquivo* com os métodos `read()` e `write()` (e muitos mais). Para envolver um descritor de arquivo em um objeto arquivo, use *fdopen()*.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.5: Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção *InterruptedError* (consulte **PEP 475** para entender a justificativa).

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

As seguintes constantes são opções para o parâmetro *flags* da função *open()*. Elas podem ser combinadas usando o operador bit a bit OU `|`. Algumas delas não estão disponíveis em todas as plataformas. Para obter descrições de sua disponibilidade e uso, consulte a página de manual *open(2)* para Unix ou o MSDN para Windows.

os.**O_RDONLY**

os.**O_WRONLY**

os.**O_RDWR**

os.**O_APPEND**

os.**O_CREAT**

os.**O_EXCL**

os.**O_TRUNC**

As constantes acima estão disponíveis no Unix e Windows.

os.**O_DSYNC**

os.**O_RSYNC**

os.**O_SYNC**

os.**O_NDELAY**

os.**O_NONBLOCK**

os.**O_NOCTTY**

os.**O_CLOEXEC**

As constantes acima estão disponíveis apenas no Unix.

Alterado na versão 3.3: Adicionada a constante *O_CLOEXEC*.

os.**O_BINARY**

os.**O_NOINHERIT**

os.**O_SHORT_LIVED**

os.**O_TEMPORARY**

os.**O_RANDOM**

os.**O_SEQUENTIAL**

os.**O_TEXT**

As constantes acima estão disponíveis apenas no Windows.

os.**O_EVTONLY**

os.**O_FSYNC**

os.**O_SYMLINK**

os.**O_NOFOLLOW_ANY**

As constantes acima estão disponíveis apenas no macOS.

Alterado na versão 3.10: Adicionadas as constantes *O_EVTONLY*, *O_FSYNC*, *O_SYMLINK* e *O_NOFOLLOW_ANY*.

os.**O_ASYNC**

os.**O_DIRECT**

os.**O_DIRECTORY**

os.**O_NOFOLLOW**

os.**O_NOATIME**

os.**O_PATH**

os.**O_TMPFILE**

os.**O_SHLOCK**

os.**O_EXLOCK**

As constantes acima são extensões e não estarão presentes, se não forem definidos pela biblioteca C.

Alterado na versão 3.4: Adicionada *O_PATH* em sistemas que oferecem suporte. Adicionada *O_TMPFILE*, somente disponível no kernel Linux 3.11 ou mais recente.

os.**openpty**()

Abre um novo par de pseudo-terminal. Retorna um par de descritores de arquivos `(master, slave)` para o pty e o tty, respectivamente. Os novos descritores de arquivos são *não-herdáveis*. Para uma abordagem (ligeiramente) mais portável, use o módulo *pty*.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.4: Os novos descritores de arquivos agora são não-herdáveis.

os.**pipe**()

Cria um encadeamento (pipe). Retorna um par de descritores de arquivos `(r, w)` usáveis para leitura e escrita, respectivamente. O novo descritor de arquivo é *não-herdável*.

Disponibilidade: Unix, Windows.

Alterado na versão 3.4: Os novos descritores de arquivos agora são não-herdáveis.

os.**pipe2**(*flags*, /)

Cria um encadeamento (pipe) com *flags* definidos atomicamente. *flags* podem ser construídos por aplicação de OU junto a um ou mais destes valores: *O_NONBLOCK*, *O_CLOEXEC*. Retorna um par de descritores de arquivos `(r, w)` utilizáveis para leitura e gravação, respectivamente.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.**posix_fallocate**(*fd*, *offset*, *len*, /)

Garante que um espaço em disco suficiente seja alocado para o arquivo especificado por *fd* iniciando em *offset* e continuando por *len* bytes.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.**posix_fadvise**(*fd*, *offset*, *len*, *advice*, /)

Anuncia a intenção de acessar dados em um padrão específico, permitindo assim que o kernel faça otimizações. O conteúdo em *advice* se aplica à região do arquivo especificado por *fd*, iniciando em *offset* e continuando por *len* bytes. *advice* é um entre *POSIX_FADV_NORMAL*, *POSIX_FADV_SEQUENTIAL*, *POSIX_FADV_RANDOM*, *POSIX_FADV_NOREUSE*, *POSIX_FADV_WILLNEED* ou *POSIX_FADV_DONTNEED*.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.**POSIX_FADV_NORMAL**

os.**POSIX_FADV_SEQUENTIAL**

os.**POSIX_FADV_RANDOM**

os.**POSIX_FADV_NOREUSE**

os.**POSIX_FADV_WILLNEED**

os.**POSIX_FADV_DONTNEED**

Sinalizadores que podem ser usados em *advice* em *posix_fadvise()* que especificam o padrão de acesso que provavelmente será usado.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.**pread**(*fd*, *n*, *offset*, /)

Lê no máximo *n* bytes do descritor de arquivo *fd* na posição *offset*, mantendo o deslocamento do arquivo inalterado.

Retorna uma bytestring contendo os bytes lidos. Se o final do arquivo referido por *fd* for atingido, um objeto de bytes vazio será retornado.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.**posix_openpt**(*oflag*, /)

Abre e retorna um descritor de arquivo para um dispositivo pseudoterminal mestre.

Chama a função `posix_openpt()` da biblioteca padrão C. O argumento *oflag* é usado para definir sinalizadores de status de arquivo e modos de acesso a arquivos conforme especificado na página de manual da `posix_openpt()` do seu sistema.

O descritor de arquivo retornado é *não herdável*. Se o valor `O_CLOEXEC` estiver disponível no sistema, ele é adicionado a *oflag*.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.13.

`os.`**`preadv`**(*fd*, *buffers*, *offset*, *flags=0*, */*)

Lê de um descritor de arquivo *fd* na posição de *offset* em *buffers* mutáveis de *objetos bytes ou similares*, deixando o deslocamento do arquivo inalterado. Transfere os dados para cada buffer até ficar cheio e depois passa para o próximo buffer na sequência para armazenar o restante dos dados.

O argumento *flags* contém um OR bit a bit de zero ou mais dos seguintes sinalizadores:

- `RWF_HIPRI`
- `RWF_NOWAIT`

Retorna o número total de bytes realmente lidos, que pode ser menor que a capacidade total de todos os objetos.

O sistema operacional pode definir um limite (`sysconf()` valor `'SC_IOV_MAX'`) no número de buffers que podem ser usados.

Combina a funcionalidade de `os.readv()` e `os.pread()`.

Disponibilidade: Linux >= 2.6.30, FreeBSD >= 6.0, OpenBSD >= 2.7, AIX >= 7.1.

O uso de sinalizadores requer Linux >= 4.6.

Adicionado na versão 3.7.

`os.`**`RWF_NOWAIT`**

Não aguarda por dados que não estão disponíveis imediatamente. Se esse sinalizador for especificado, a chamada do sistema retorna instantaneamente se for necessário ler dados do armazenamento de backup ou aguardar uma trava.

Se alguns dados foram lidos com sucesso, ele retorna o número de bytes lidos. Se nenhum bytes foi lido, ele retornará `-1` e definirá errno como `errno.EAGAIN`.

Disponibilidade: Linux >= 4.14.

Adicionado na versão 3.7.

`os.`**`RWF_HIPRI`**

Alta prioridade de leitura/gravação. Permite sistemas de arquivos baseados em blocos para usar a consulta do dispositivo, que fornece latência inferior, mas pode usar recursos adicionais.

Atualmente, no Linux, esse recurso é usável apenas em um descritor de arquivo aberto usando o sinalizador `O_DIRECT`.

Disponibilidade: Linux >= 4.6.

Adicionado na versão 3.7.

`os.`**`ptsname`**(*fd*, */*)

Retorna o nome do dispositivo pseudoterminal secundário associado ao dispositivo pseudoterminal principal ao qual o descritor de arquivo *fd* se refere. O descritor de arquivo *fd* não é fechado em caso de falha.

Chama a função reentrante `ptsname_r()` da biblioteca padrão C se estiver disponível; caso contrário, a função `ptsname()` da biblioteca padrão C, que não é garantida como sendo segura para thread, é chamada.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.13.

`os.`**`pwrite`**(*fd*, *str*, *offset*, */*)

Escreve a bytestring in *str* no descritor de arquivo *fd* na posição *offset*, mantendo o deslocamento do arquivo inalterado.

Retorna o número de bytes realmente escritos.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.**pwritev**(*fd*, *buffers*, *offset*, *flags=0*, /)

Escreve o conteúdo de *buffers* no descritor de arquivo *fd* com um deslocamento *offset*, deixando o deslocamento do arquivo inalterado. *buffers* deve ser uma sequência de *objetos bytes ou similares*. Os buffers são processados em ordem de vetor. Todo o conteúdo do primeiro buffer é gravado antes de prosseguir para o segundo, e assim por diante.

O argumento *flags* contém um OR bit a bit de zero ou mais dos seguintes sinalizadores:

- *RWF_DSYNC*
- *RWF_SYNC*
- *RWF_APPEND*

Retorna o número total de bytes realmente escritos.

O sistema operacional pode definir um limite (*sysconf()* valor `'SC_IOV_MAX'`) no número de buffers que podem ser usados.

Combina a funcionalidade de *os.writev()* e *os.pwrite()*.

Disponibilidade: Linux >= 2.6.30, FreeBSD >= 6.0, OpenBSD >= 2.7, AIX >= 7.1.

O uso de sinalizadores requer Linux >= 4.6.

Adicionado na versão 3.7.

os.**RWF_DSYNC**

Fornece um equivalente para gravação do sinalizador *O_DSYNC* da função *os.open()*. Este efeito de sinalizador se aplica apenas ao intervalo de dados escrito pela chamada de sistema.

Disponibilidade: Linux >= 4.7.

Adicionado na versão 3.7.

os.**RWF_SYNC**

Fornece um equivalente para gravação do sinalizador *O_SYNC* da função *os.open()*. Este efeito de sinalizador se aplica apenas ao intervalo de dados escrito pela chamada de sistema.

Disponibilidade: Linux >= 4.7.

Adicionado na versão 3.7.

os.**RWF_APPEND**

Fornece um equivalente para gravação do sinalizador *O_APPEND* da função *os.open()*. Esse sinalizador é significante apenas para *os.pwritev()*, e seu efeito se aplica apenas ao intervalo de dados escrito pela chamada de sistema. O argumento *offset* não afeta a operação de escrita; o dado é sempre adicionado ao fim do arquivo. Contudo, se o argumento *offset* for `-1`, o *offset* do arquivo atual é atualizado.

Disponibilidade: Linux >= 4.16.

Adicionado na versão 3.10.

os.**read**(*fd*, *n*, /)

Lê no máximo *n* bytes do descritor de arquivos *fd*.

Retorna uma bytestring contendo os bytes lidos. Se o final do arquivo referido por *fd* for atingido, um objeto de bytes vazio será retornado.

> **Nota**
>
> Esta função destina-se a E/S de baixo nível e deve ser aplicada a um descritor de arquivo retornado por *os.open()* ou *pipe()*. Para ler um "objeto arquivo" retornado pela função embutida *open()* ou por *popen()* ou *fdopen()*, ou *sys.stdin*, use seus métodos `read()` ou `readline()`.

Alterado na versão 3.5: Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção `InterruptedError` (consulte **PEP 475** para entender a justificativa).

os.**readinto**(*fd*, *buffer*, /)

Lê de um descritor de arquivo *fd* em um *buffer* de objeto buffer mutável.

O *buffer* deve ser mutável e *bytes ou similar*. Em caso de sucesso, retorna o número de bytes lidos. Menos bytes podem ser lidos do que o tamanho do buffer. A chamada de sistema subjacente será repetida quando interrompida por um sinal, a menos que o manipulador de sinais levante uma exceção. Outros erros não serão repetidos e um erro será levantado.

Retorna 0 se *fd* estiver no final do arquivo ou se o *buffer* fornecido tiver comprimento 0 (o que pode ser usado para verificar erros sem ler dados). Nunca retorna negativo.

> **Nota**
>
> Esta função destina-se a E/S de baixo nível e deve ser aplicada a um descritor de arquivo retornado por `os.open()` ou `os.pipe()`. Para ler um "objeto arquivo" retornado pela função embutida `open()` ou `sys.stdin`, use suas funções-membro, por exemplo `io.BufferedIOBase.readinto()`, `io.BufferedIOBase.read()` ou `io.TextIOBase.read()`.

Adicionado na versão 3.14.

os.**sendfile**(*out_fd*, *in_fd*, *offset*, *count*)

os.**sendfile**(*out_fd*, *in_fd*, *offset*, *count*, *headers=()*, *trailers=()*, *flags=0*)

Copia *count* bytes do descritor de arquivo *in_fd* para o descritor de arquivo *out_fd* começando em *offset*. Retorna o número de bytes enviados. Quando o EOF é alcançado, retorna `0`.

A primeira notação da função está disponível por todas as plataformas que definem `sendfile()`.

No Linux, se *offset* for fornecido como `None`, os bytes são lidos da posição atual de *in_fd* e a posição de *in_fd* é atualizada.

O segundo caso pode ser usado no macOS e FreeBSD onde *headers* e *trailers* são sequências arbitrárias de buffers que são escritos antes e depois dos dados de *in_fd* ser escrito. Retorna o mesmo que no primeiro caso.

No macOS e FreeBSD, um valor de `0` para *count* especifica enviar até o fim de *in_fd* ser alcançado.

Todas as plataformas tem suporte a soquetes como descritor de arquivo *out_fd*, e algumas plataformas permitem outros tipos (por exemplo, arquivo regular, encadeamento) também.

Aplicativos de plataforma cruzada não devem usar os argumentos *headers*, *trailers* e *flags*.

Disponibilidade: Unix, not WASI.

> **Nota**
>
> Para um invólucro de nível mais alto de `sendfile()`, consulte `socket.socket.sendfile()`.

Adicionado na versão 3.3.

Alterado na versão 3.9: Os parâmetros *out* e *in* foram renomeados para *out_fd* e *in_fd*.

os.**SF_NODISKIO**

os.**SF_MNOWAIT**

os.**SF_SYNC**

Parâmetros para a função `sendfile()`, se a implementação tiver suporte a eles.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.**SF_NOCACHE**

Parâmetro para a função *sendfile()*, se a implementação tiver suporte a isso. Os dados não serão armazenados em cache na memória virtual e serão liberados posteriormente.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.11.

os.**set_blocking**(*fd*, *blocking*, /)

Define o modo de bloqueio do descritor de arquivo especificado. Define o sinalizador *O_NONBLOCK* se blocking for `False`; do contrário, limpa o sinalizador.

Veja também *get_blocking()* e *socket.socket.setblocking()*.

Disponibilidade: Unix, Windows.

A função é limitada em WASI, veja *Plataformas WebAssembly* para mais informações.

No Windows, esta função é limitada a encadeamentos.

Adicionado na versão 3.5.

Alterado na versão 3.12: Adicionado suporte a encadeamentos no Windows.

os.**splice**(*src*, *dst*, *count*, *offset_src=None*, *offset_dst=None*, *flags=0*)

Transfere *count* bytes do descritor de arquivo *src*, começando no deslocamento *offset_src*, para o descritor de arquivo *dst*, começando no deslocamento *offset_dst*.

O comportamento da união via splice pode ser modificado especificando um valor para *flags*. Qualquer um dos seguintes valores pode ser usado, combinados usando OU (OR) bit a bit (o operador `|`):

- Se *SPLICE_F_MOVE* for especificado, o kernel será solicitado a mover páginas em vez de copiá-las, mas as páginas ainda poderão ser copiadas se o kernel não puder movê-las do encadeamento.
- Se *SPLICE_F_NONBLOCK* for especificado, o kernel será solicitado a não bloquear em E/S. Isso torna as operações de encadeamento de emenda não bloqueantes, mas a emenda pode, ainda assim, bloquear, pois os descritores de arquivo emendados podem bloquear.
- Se *SPLICE_F_MORE* for especificado, ele indica ao kernel que mais dados chegarão em uma emenda subsequente.

Pelo menos um dos descritores de arquivo deve se referir a um encadeamento. Se *offset_src* é `None`, então *src* é lido a partir da posição atual; o mesmo para *offset_dst*. O deslocamento associado a um descritor de arquivo que se refere a um encadeamento deve ser `None`. Os arquivos apontados por *src* e *dst* devem residir no mesmo sistema de arquivos, de outra forma uma *OSError* é levantada com *errno* definido para *errno.EXDEV*.

Essa cópia é feita sem o custo adicional de transferência de dados do kernel para o espaço do usuário e, em seguida, volta para o kernel. Além disso, alguns sistemas de arquivos poderiam implementar otimizações extras. A cópia é feita como se ambos os arquivos estivessem abertos como binários.

Após a conclusão bem-sucedida, retorna o número de bytes unidos ao encadeamento ou a partir dele. Um valor de retorno de 0 significa o fim da entrada. Se *src* se refere a um encadeamento, isso significa que não havia dados para transferir, e não faria sentido bloquear porque não há escritores conectados ao fim da escrita do encadeamento.

Ver também

A página man *splice(2)*.

Disponibilidade: Linux >= 2.6.17 with glibc >= 2.5

Adicionado na versão 3.10.

os.**SPLICE_F_MOVE**

os.**SPLICE_F_NONBLOCK**

os.**SPLICE_F_MORE**

Adicionado na versão 3.10.

os.**readv**(*fd*, *buffers*, /)

Lê de um descritor de arquivo *fd* em um número de *buffers* *objetos bytes ou similares* mutáveis. Transfere os dados para cada buffer até que esteja cheio e, a seguir, vai para o próximo buffer na sequência para armazenar o restante dos dados.

Retorna o número total de bytes realmente lidos, que pode ser menor que a capacidade total de todos os objetos.

O sistema operacional pode definir um limite (`sysconf()` valor `'SC_IOV_MAX'`) no número de buffers que podem ser usados.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.**tcgetpgrp**(*fd*, /)

Retorna o grupo de processos associado ao terminal fornecido por *fd* (um descritor de arquivo aberto retornado por `os.open()`).

Disponibilidade: Unix, not WASI.

os.**tcsetpgrp**(*fd*, *pg*, /)

Define o grupo de processos associado ao terminal fornecido por *fd* (um descritor de arquivo aberto retornado por `os.open()`) para *pg*.

Disponibilidade: Unix, not WASI.

os.**ttyname**(*fd*, /)

Retorna uma string que especifica o dispositivo de terminal associado ao descritor de arquivo *fd*. Se *fd* não estiver associado a um dispositivo de terminal, uma exceção é levantada.

Disponibilidade: Unix.

os.**unlockpt**(*fd*, /)

Destrava o dispositivo pseudoterminal secundário associado ao dispositivo pseudoterminal principal ao qual o descritor de arquivo *fd* se refere. O descritor de arquivo *fd* não é fechado em caso de falha.

Chama a função `unlockpt()` da biblioteca padrão C.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.13.

os.**write**(*fd*, *str*, /)

Escreve a string de bytes em *str* no descritor de arquivo *fd*.

Retorna o número de bytes realmente escritos.

> **Nota**
>
> Esta função destina-se a E/S de baixo nível e deve ser aplicada a um descritor de arquivo retornado por `os.open()` ou `pipe()`. Para escrever um "objeto arquivo" retornado pela função embutida `open()` ou por `popen()` ou `fdopen()`, ou `sys.stdout` ou `sys.stderr`, use seu método `write()`.

Alterado na versão 3.5: Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção `InterruptedError` (consulte **PEP 475** para entender a justificativa).

os.**writev**(*fd*, *buffers*, /)

Escreve o conteúdo de *buffers* no descritor de arquivo *fd*. *buffers* deve ser uma sequência de *objetos bytes ou similares*. Os buffers são processados na ordem em que estão. Todo o conteúdo do primeiro buffer é gravado antes de prosseguir para o segundo, e assim por diante.

Retorna o número total de bytes realmente escritos.

O sistema operacional pode definir um limite (*sysconf()* valor 'SC_IOV_MAX') no número de buffers que podem ser usados.

Disponibilidade: Unix.

Adicionado na versão 3.3.

Consultando o tamanho de um terminal

Adicionado na versão 3.3.

os.**get_terminal_size**(*fd=STDOUT_FILENO*, /)

Retorna o tamanho da janela do terminal como (columns, lines), tupla do tipo *terminal_size*.

O argumento opcional fd (padrão STDOUT_FILENO, ou saída padrão) especifica qual descritor de arquivo deve ser consultado.

Se o descritor de arquivo não estiver conectado a um terminal, uma exceção *OSError* é levantada.

shutil.get_terminal_size() é a função de alto nível que normalmente deve ser usada, os.get_terminal_size é a implementação de baixo nível.

Disponibilidade: Unix, Windows.

class os.**terminal_size**

Uma subclasse de tupla, contendo (columns, lines) do tamanho da janela do terminal.

columns

Largura da janela do terminal em caracteres.

lines

Altura da janela do terminal em caracteres.

Herança de descritores de arquivos

Adicionado na versão 3.4.

Um descritor de arquivo tem um sinalizador "herdável" que indica se o descritor de arquivo pode ser herdado por processos filho. A partir do Python 3.4, os descritores de arquivo criados pelo Python não são herdáveis por padrão.

No UNIX, os descritores de arquivo não herdáveis são fechados em processos filho na execução de um novo programa, outros descritores de arquivo são herdados.

No Windows, identificadores não herdáveis e descritores de arquivo são fechados em processos filho, exceto para fluxos padrão (descritores de arquivo 0, 1 e 2: stdin, stdout e stderr), que são sempre herdados. Usando as funções *spawn**, todos os identificadores herdáveis e todos os descritores de arquivo herdáveis são herdados. Usando o módulo *subprocess*, todos os descritores de arquivo, exceto fluxos padrão, são fechados, e os manipuladores herdáveis são herdados apenas se o parâmetro *close_fds* for False.

Nas plataformas WebAssembly, o descritor de arquivo não pode ser modificado.

os.**get_inheritable**(*fd*, /)

Obtém o sinalizador "herdável" do descritor de arquivo especificado (um booleano).

os.**set_inheritable**(*fd*, *inheritable*, /)

Define o sinalizador "herdável" do descritor de arquivo especificado.

os.**get_handle_inheritable**(*handle*, /)

Obtém o sinalizador "herdável" do manipulador especificado (um booleano).

Disponibilidade: Windows.

os.**set_handle_inheritable**(*handle*, *inheritable*, /)

Define o sinalizador "herdável" do manipulador especificado.

Disponibilidade: Windows.

16.1.6 Arquivos e diretórios

Em algumas plataformas Unix, muitas dessas funções oferecem suporte para um ou mais destes recursos:

- **especificar um descritor de arquivo:** normalmente o argumento *path* fornecido para funções no módulo `os` deve ser uma string especificando um caminho de arquivo. No entanto, algumas funções agora aceitam alternativamente um descritor de arquivo aberto para seu argumento *path*. A função então operará no arquivo referido pelo descritor. (Para sistemas POSIX, Python irá chamar a variante da função prefixada com `f` (por exemplo, chamar `fchdir` em vez de `chdir`).

  Você pode verificar se *path* pode ser especificado ou não como um descritor de arquivo para uma função particular em sua plataforma usando `os.supports_fd`. Se esta funcionalidade não estiver disponível, usá-la levantará uma `NotImplementedError`.

  Se a função também oferecer suporte para os argumentos *dir_fd* ou *follow_symlinks*, é um erro especificar um deles ao fornecer *path* como um descritor de arquivo.

- **caminhos relativos aos descritores de diretório:** se *dir_fd* não for `None`, deve ser um descritor de arquivo referindo-se a um diretório, e o caminho para operar deve ser relativo; o caminho será relativo a esse diretório. Se o caminho for absoluto, *dir_fd* será ignorado. Para sistemas POSIX, Python irá chamar a variante da função com um sufixo `at` e possivelmente prefixado com `f` (por exemplo, chamar `faccessat` ao invés de `access`).

  Você pode verificar se há ou não suporte para *dir_fd* em uma função particular em sua plataforma usando `os.supports_dir_fd`. Se não estiver disponível, usá-lo levantará uma `NotImplementedError`.

- **não seguir links simbólicos:** se *follow_symlinks* for `False`, e o último elemento do caminho para operar for um link simbólico, a função irá operar no próprio link simbólico ao invés do arquivo apontado pelo link. Para sistemas POSIX, Python irá chamar a variante `l...` da função.

  Você pode verificar se há ou não suporte para *follow_symlinks* em uma função particular em sua plataforma usando `os.supports_follow_symlinks`. Se não estiver disponível, usá-lo levantará uma `NotImplementedError`.

`os.`**`access`**(*path*, *mode*, *, *dir_fd=None*, *effective_ids=False*, *follow_symlinks=True*)

Usa o uid/gid real para testar o acesso ao *path*. Observe que a maioria das operações usará o uid/gid efetivo, portanto, essa rotina pode ser usada em um ambiente suid/sgid para testar se o usuário da chamada tem o acesso especificado ao *path*. *mode* deve ser `F_OK` para testar a existência de *path*, ou pode ser o OU inclusivo de um ou mais dos `R_OK`, `W_OK`, e `X_OK` para testar as permissões. Retorna `True` se o acesso for permitido, `False` se não for. Veja a página man do Unix `access(2)` para mais informações.

Esta função pode oferecer suporte a especificação de *caminhos relativos aos descritores de diretório* e *não seguir os links simbólicos*.

Se *effective_ids* for `True`, `access()` irá realizar suas verificações de acesso usando o uid/gid efetivo ao invés do uid/gid real. *effective_ids* pode não ser compatível com sua plataforma; você pode verificar se está ou não disponível usando `os.supports_effective_ids`. Se não estiver disponível, usá-lo levantará uma `NotImplementedError`.

> **Nota**
>
> Usar `access()` para verificar se um usuário está autorizado a, por exemplo, abrir um arquivo antes de realmente fazer isso usando `open()` cria uma brecha de segurança, porque o usuário pode explorar o curto intervalo de tempo entre a verificação e a abertura do arquivo para manipulá-lo. É preferível usar as técnicas *EAFP*. Por exemplo:
>
> ```
> if os.access("meuarquivo", os.R_OK):
>     with open("meuarquivo") as fp:
>         return fp.read()
> return "algum dado padrão"
> ```
>
> é melhor escrito como:

```
try:
    fp = open("meuarquivo")
except PermissionError:
    return "algum dado padrão"
else:
    with fp:
        return fp.read()
```

Nota

As operações de E/S podem falhar mesmo quando *access()* indica que elas teriam sucesso, particularmente para operações em sistemas de arquivos de rede que podem ter semântica de permissões além do modelo de bits de permissão POSIX usual.

Alterado na versão 3.3: Adicionados os parâmetros *dir_fd*, *effective_ids* e *follow_symlinks*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**F_OK**

os.**R_OK**

os.**W_OK**

os.**X_OK**

Valores a serem passados como o parâmetro *mode* de *access()* para testar a existência, legibilidade, capacidade de escrita e executabilidade de *path*, respectivamente.

os.**chdir**(*path*)

Altera o diretório de trabalho atual para *path*.

Esta função pode oferecer suporte a *especificar um descritor de arquivo*. O descritor deve fazer referência a um diretório aberto, não a um arquivo aberto.

Esta função pode levantar *OSError* e subclasses como *FileNotFoundError*, *PermissionError* e *NotADirectoryError*.

Levanta um *evento de auditoria* `os.chdir` com o argumento `path`.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo em algumas plataformas.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**chflags**(*path*, *flags*, ***, *follow_symlinks=True*)

Define os sinalizadores de *path* para os *flags* numéricos. *flags* podem assumir uma combinação (OU bit a bit) dos seguintes valores (conforme definido no módulo *stat*):

- *stat.UF_NODUMP*
- *stat.UF_IMMUTABLE*
- *stat.UF_APPEND*
- *stat.UF_OPAQUE*
- *stat.UF_NOUNLINK*
- *stat.UF_COMPRESSED*
- *stat.UF_HIDDEN*
- *stat.SF_ARCHIVED*
- *stat.SF_IMMUTABLE*

- *stat.SF_APPEND*
- *stat.SF_NOUNLINK*
- *stat.SF_SNAPSHOT*

Esta função pode oferecer suporte a *não seguir links simbólicos*.

Levanta um *evento de auditoria* `os.chflags` com os argumentos `path`, `flags`.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.3: Adicionado o parâmetro *follow_symlinks*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.`**`chmod`**(*path*, *mode*, ***, *dir_fd=None*, *follow_symlinks=True*)

Altera o modo de *path* para o *mode* numérico. *mode* pode assumir um dos seguintes valores (conforme definido no módulo *stat*) ou combinações de OU bit a bit deles:

- *stat.S_ISUID*
- *stat.S_ISGID*
- *stat.S_ENFMT*
- *stat.S_ISVTX*
- *stat.S_IREAD*
- *stat.S_IWRITE*
- *stat.S_IEXEC*
- *stat.S_IRWXU*
- *stat.S_IRUSR*
- *stat.S_IWUSR*
- *stat.S_IXUSR*
- *stat.S_IRWXG*
- *stat.S_IRGRP*
- *stat.S_IWGRP*
- *stat.S_IXGRP*
- *stat.S_IRWXO*
- *stat.S_IROTH*
- *stat.S_IWOTH*
- *stat.S_IXOTH*

Esta função pode oferecer suporte a *especificar um descritor de arquivo*, *caminhos relativos aos descritores de diretório* e *não seguir os links simbólicos*.

> **Nota**
>
> Embora o Windows ofereça suporte ao *chmod()*, você só pode definir o sinalizador de somente leitura do arquivo (através das constantes `stat.S_IWRITE` e `stat.S_IREAD` ou um valor inteiro correspondente). Todos os outros bits são ignorados. O valor padrão de *follow_symlinks* é `False` no Windows.
>
> A função é limitada em WASI, veja *Plataformas WebAssembly* para mais informações.

Levanta um *evento de auditoria* `os.chmod` com os argumentos `path`, `mode`, `dir_fd`.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto e os argumentos *dir_fd* e *follow_symlinks*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.13: Adicionado suporte para um descritor de arquivo e o argumento *follow_symlinks* no Windows.

`os.chown`(*path*, *uid*, *gid*, ***, *dir_fd=None*, *follow_symlinks=True*)

Altera o proprietário e o id de grupo de *path* para *uid* e *gid* numéricos. Para deixar um dos ids inalterado, defina-o como -1.

Esta função pode oferecer suporte a *especificar um descritor de arquivo*, *caminhos relativos aos descritores de diretório* e *não seguir os links simbólicos*.

Consulte `shutil.chown()` para uma função de alto nível que aceita nomes além de ids numéricos.

Levanta um *evento de auditoria* `os.chown` com os argumentos `path`, `uid`, `gid`, `dir_fd`.

Disponibilidade: Unix.

A função é limitada em WASI, veja *Plataformas WebAssembly* para mais informações.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto e os argumentos *dir_fd* e *follow_symlinks*.

Alterado na versão 3.6: Oferece suporte para um *objeto caminho ou similar*.

`os.chroot`(*path*)

Altera o diretório raiz do processo atual para *path*

Disponibilidade: Unix, not WASI, not Android.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.fchdir`(*fd*)

Altera o diretório de trabalho atual para o diretório representado pelo descritor de arquivo *fd*. O descritor deve se referir a um diretório aberto, não a um arquivo aberto. No Python 3.3, isso é equivalente a `os.chdir(fd)`.

Levanta um *evento de auditoria* `os.chdir` com o argumento `path`.

Disponibilidade: Unix.

`os.getcwd`()

Retorna uma string representando o diretório de trabalho atual.

`os.getcwdb`()

Retorna uma bytestring representando o diretório de trabalho atual.

Alterado na versão 3.8: A função agora usa a codificação UTF-8 no Windows, em vez da página de código ANSI: consulte a **PEP 529** para a justificativa. A função não está mais descontinuada no Windows.

`os.lchflags`(*path*, *flags*)

Define os sinalizadores de *path* para os *flags* numéricos, como `chflags()`, mas não segue links simbólicos. No Python 3.3, isso é equivalente a `os.chflags(path, flags, follow_symlinks=False)`.

Levanta um *evento de auditoria* `os.chflags` com os argumentos `path`, `flags`.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.lchmod`(*path*, *mode*)

Altera o modo de *path* para o *mode* numérico. Se o caminho for um link simbólico, isso afetará o link simbólico em vez do destino. Veja a documentação de `chmod()` para valores possíveis de *mode*. No Python 3.3, isso é equivalente a `os.chmod(path, mode, follow_symlinks=False)`.

`lchmod()` não faz parte do POSIX, mas as implementações Unix podem tê-lo se houver suporte para alterar o modo de links simbólicos.

Levanta um *evento de auditoria* `os.chmod` com os argumentos `path`, `mode`, `dir_fd`.

Disponibilidade: Unix, Windows, not Linux, FreeBSD >= 1.3, NetBSD >= 1.3, not OpenBSD

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.13: Adicionado suporte no Windows.

`os.lchown`(*path*, *uid*, *gid*)

Altera o proprietário e o id de grupo de *path* para *uid* e *gid* numéricos. Esta função não seguirá links simbólicos. No Python 3.3, isso é equivalente a `os.chown(path, uid, gid, follow_symlinks=False)`.

Levanta um *evento de auditoria* `os.chown` com os argumentos `path`, `uid`, `gid`, `dir_fd`.

Disponibilidade: Unix.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.link`(*src*, *dst*, ***, *src_dir_fd=None*, *dst_dir_fd=None*, *follow_symlinks=True*)

Cria um link físico apontando para *src* chamado *dst*.

Esta função oferece suporte à especificação de *src_dir_fd* e/ou *dst_dir_fd* para fornecer *caminhos relativos aos descritores de diretório* e *não seguir links simbólicos*. O valor padrão de *follow_symlinks* é `False` no Windows.

Levanta um *evento de auditoria* `os.link` com os argumentos `src`, `dst`, `src_dir_fd`, `dst_dir_fd`.

Disponibilidade: Unix, Windows.

Alterado na versão 3.2: Adicionado suporte ao Windows.

Alterado na versão 3.3: Adicionados os parâmetros *src_dir_fd*, *dst_dir_fd* e *follow_symlinks*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *src* e *dst*.

`os.listdir`(*path='.'*)

Retorna uma lista contendo os nomes das entradas no diretório fornecido por *path*. A lista está em ordem arbitrária e não inclui as entradas especiais `'.'` e `'..'` mesmo se estiverem presentes no diretório. Se um arquivo for removido ou adicionado ao diretório durante a chamada desta função, não é especificado se um nome para esse arquivo deve ser incluído.

path pode ser um *objeto caminho ou similar*. Se *path* for do tipo `bytes` (direta ou indiretamente por meio da interface `PathLike`), os nomes de arquivo retornados também serão do tipo `bytes`; em todas as outras circunstâncias, eles serão do tipo `str`.

Esta função também pode ter suporte a *especificar um descritor de arquivo*; o descritor de arquivo deve fazer referência a um diretório.

Levanta um *evento de auditoria* `os.listdir` com o argumento `path`.

Nota

Para codificar nomes de arquivos `str` para `bytes`, use `fsencode()`.

Ver também

A função `scandir()` retorna entradas de diretório junto com informações de atributo de arquivo, dando melhor desempenho para muitos casos de uso comuns.

Alterado na versão 3.2: O parâmetro *path* tornou-se opcional.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**listdrives**()

Retorna uma lista contendo os nomes das unidades em um sistema Windows.

Um nome de unidade geralmente se parece com `'C:\\'`. Nem todo nome de unidade estará associado a um volume e alguns podem estar inacessíveis por vários motivos, incluindo permissões, conectividade de rede ou mídia ausente. Esta função não testa o acesso.

Pode levantar *OSError* se ocorrer um erro ao coletar os nomes das unidades.

Levanta um *evento de auditoria* `os.listdrives` sem argumentos.

Disponibilidade: Windows

Adicionado na versão 3.12.

os.**listmounts**(*volume*)

Retorna uma lista contendo os pontos de montagem para um volume em um sistema Windows.

volume deve ser representado como um caminho GUID, como aqueles retornados por *os.listvolumes()*. Os volumes podem ser montados em vários locais ou não. No último caso, a lista estará vazia. Os pontos de montagem que não estão associados a um volume não serão retornados por esta função.

Os pontos de montagem retornados por esta função serão caminhos absolutos e podem ser mais longos que o nome da unidade.

Levanta *OSError* se o volume não for reconhecido ou se ocorrer um erro ao coletar os caminhos.

Levanta um *evento de auditoria* `os.listmounts` com o argumento `volume`.

Disponibilidade: Windows

Adicionado na versão 3.12.

os.**listvolumes**()

Retorna uma lista contendo os volumes no sistema.

Volumes são normalmente representados como um caminho GUID que se parece com `\\?\Volume{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}\`. Os arquivos geralmente podem ser acessados por meio de um caminho GUID, permitindo permissões. No entanto, os usuários geralmente não estão familiarizados com eles e, portanto, o uso recomendado desta função é recuperar pontos de montagem usando *os.listmounts()*.

Pode levantar *OSError* se ocorrer um erro ao coletar os volumes.

Levanta um *evento de auditoria* `os.listvolumes` sem argumentos.

Disponibilidade: Windows

Adicionado na versão 3.12.

os.**lstat**(*path*, ***, *dir_fd=None*)

Executa o equivalente a uma chamada de sistema `lstat()` no caminho fornecido. Semelhante a *stat()*, mas não segue links simbólicos. Retorna um objeto *stat_result*.

Em plataformas que não têm suporte a links simbólicos, este é um apelido para *stat()*.

No Python 3.3, isso é equivalente a `os.stat(path, dir_fd=dir_fd, follow_symlinks=False)`.

Esta função também pode ter suporte a *caminhos relativos a descritores de diretório*.

Ver também

A função *stat()*.

Alterado na versão 3.2: Adicionado suporte para links simbólicos do Windows 6.0 (Vista).

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.8: No Windows, agora abre pontos de nova análise que representam outro caminho (substitutos de nome), incluindo links simbólicos e junções de diretório. Outros tipos de pontos de nova análise são resolvidos pelo sistema operacional como para `stat()`.

`os.`**`mkdir`**(*path*, *mode=0o777*, ***, *dir_fd=None*)

Cria um diretório chamado *path* com o modo numérico *mode*.

Se o diretório já existe, uma exceção `FileExistsError` é levantada. Se um diretório pai no caminho não existir, uma exceção `FileNotFoundError` será levantada.

Em alguns sistemas, *mode* é ignorado. Onde ele é usado, o valor atual do umask é primeiro mascarado. Se bits diferentes dos últimos 9 (ou seja, os últimos 3 dígitos da representação octal do *mode*) são definidos, seu significado depende da plataforma. Em algumas plataformas, eles são ignorados e você deve chamar `chmod()` explicitamente para defini-los.

No Windows, um modo *mode* de `0o700` é tratado especificamente para aplicar controle de acesso ao novo diretório de forma que apenas o usuário atual e os administradores tenham acesso. Outros valores de *mode* são ignorados.

Esta função também pode ter suporte a *caminhos relativos a descritores de diretório*.

Também é possível criar diretórios temporários; veja a função `tempfile.mkdtemp()` do módulo `tempfile`.

Levanta um *evento de auditoria* `os.mkdir` com os argumentos `path`, `mode`, `dir_fd`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.13: O Windows agora lida com um *mode* de `0o700`.

`os.`**`makedirs`**(*name*, *mode=0o777*, *exist_ok=False*)

Função de criação recursiva de diretório. Como `mkdir()`, mas cria todos os diretórios de nível intermediário necessários para conter o diretório folha.

O parâmetro *mode* é passado para `mkdir()` para criar o diretório folha; veja *a descrição do mkdir()* para como é interpretado. Para definir os bits de permissão de arquivo de qualquer diretório pai recém-criado, você pode definir o umask antes de invocar `makedirs()`. Os bits de permissão de arquivo dos diretórios pais existentes não são alterados.

Se *exist_ok* for `False` (o padrão), uma `FileExistsError` é levantada se o diretório alvo já existir.

> **Nota**
>
> `makedirs()` ficará confuso se os elementos do caminho a serem criados incluírem `pardir` (por exemplo, ".." em sistemas UNIX).

Esta função trata os caminhos UNC corretamente.

Levanta um *evento de auditoria* `os.mkdir` com os argumentos `path`, `mode`, `dir_fd`.

Alterado na versão 3.2: Adicionado o parâmetro *exist_ok*.

Alterado na versão 3.4.1: Antes do Python 3.4.1, se *exist_ok* fosse `True` e o diretório existisse, `makedirs()` ainda levantaria um erro se *mode* não correspondesse ao modo do diretório existente. Como esse comportamento era impossível de implementar com segurança, ele foi removido no Python 3.4.1. Consulte bpo-21082.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.7: O argumento *mode* não afeta mais os bits de permissão de arquivo de diretórios de nível intermediário recém-criados.

os.**mkfifo**(*path*, *mode=0o666*, *, *dir_fd=None*)

Cria um FIFO (um encadeamento nomeado) chamado *path* com o modo numérico *mode*. O valor atual de umask é primeiro mascarado do modo.

Esta função também pode ter suporte a *caminhos relativos a descritores de diretório*.

FIFOs são canais que podem ser acessados como arquivos regulares. FIFOs existem até que sejam excluídos (por exemplo, com `os.unlink()`). Geralmente, os FIFOs são usados como ponto de encontro entre os processos do tipo "cliente" e "servidor": o servidor abre o FIFO para leitura e o cliente para escrita. Observe que `mkfifo()` não abre o FIFO – apenas cria o ponto de encontro.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**mknod**(*path*, *mode=0o600*, *device=0*, *, *dir_fd=None*)

Cria um nó de sistema de arquivos (arquivo, arquivo especial de dispositivo ou canal nomeado) chamado *path*. *mode* especifica as permissões de uso e o tipo de nó a ser criado, sendo combinado (OU bit a bit) com um de `stat.S_IFREG`, `stat.S_IFCHR`, `stat.S_IFBLK`, e `stat.S_IFIFO` (essas constantes estão disponíveis em `stat`). Para `stat.S_IFCHR` e `stat.S_IFBLK`, *device* define o arquivo especial do dispositivo recém-criado (provavelmente usando `os.makedev()`), caso contrário, ele será ignorado.

Esta função também pode ter suporte a *caminhos relativos a descritores de diretório*.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**major**(*device*, /)

Extrai o número principal de dispositivo de um número bruto de dispositivo (normalmente o campo `st_dev` ou `st_rdev` de `stat`).

os.**minor**(*device*, /)

Extrai o número menor de dispositivo de um número bruto de dispositivo (normalmente o campo `st_dev` ou `st_rdev` de `stat`).

os.**makedev**(*major*, *minor*, /)

Compõe um número de dispositivo bruto a partir dos números de dispositivo principais e secundários.

os.**pathconf**(*path*, *name*)

Retorna informações de configuração do sistema relevantes para um arquivo nomeado. *name* especifica o valor de configuração a ser recuperado; pode ser uma string que é o nome de um valor de sistema definido; esses nomes são especificados em vários padrões (POSIX.1, Unix 95, Unix 98 e outros). Algumas plataformas também definem nomes adicionais. Os nomes conhecidos do sistema operacional do host são fornecidos no dicionário `pathconf_names`. Para variáveis de configuração não incluídas nesse mapeamento, passar um número inteiro para *name* também é aceito.

Se *name* for uma string e não for conhecida, uma exceção `ValueError` é levantada. Se um valor específico para *name* não for compatível com o sistema hospedeiro, mesmo que seja incluído no `pathconf_names`, uma exceção `OSError` é levantada com `errno.EINVAL` como número do erro.

Esta função tem suporte a *especificar um descritor de arquivo*.

Disponibilidade: Unix.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**pathconf_names**

Nomes de mapeamento de dicionário aceitos por `pathconf()` e `fpathconf()` para os valores inteiros definidos para esses nomes pelo sistema operacional do host. Isso pode ser usado para determinar o conjunto de nomes conhecidos pelo sistema.

Disponibilidade: Unix.

os.**readlink**(*path*, *, *dir_fd=None*)

Retorna uma string representando o caminho para o qual o link simbólico aponta. O resultado pode ser um nome de caminho absoluto ou relativo; se for relativo, pode ser convertido para um caminho absoluto usando `os.path.join(os.path.dirname(path), result)`.

Se o *path* for um objeto string (direta ou indiretamente por meio de uma interface `PathLike`), o resultado também será um objeto string e a chamada pode levantar um UnicodeDecodeError. Se o *path* for um objeto de bytes (direto ou indireto), o resultado será um objeto de bytes.

Esta função também pode ter suporte a *caminhos relativos a descritores de diretório*.

Ao tentar resolver um caminho que pode conter links, use `realpath()` para lidar corretamente com a recursão e as diferenças de plataforma.

Disponibilidade: Unix, Windows.

Alterado na versão 3.2: Adicionado suporte para links simbólicos do Windows 6.0 (Vista).

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* no Unix.

Alterado na versão 3.8: Aceita um *objeto caminho ou similar* e um objeto bytes no Windows.

Adicionado suporte para junções de diretório e alterado para retornar o caminho de substituição (que normalmente inclui o prefixo `\\?\`) ao invés do campo opcional "print name" que era retornado anteriormente.

os.**remove**(*path*, *, *dir_fd=None*)

Remove (exclui) o arquivo *path*. Se *path* for um diretório, uma `OSError` é levantada. Use `rmdir()` para remover diretórios. Se o arquivo não existir, uma `FileNotFoundError` é levantada.

Esta função tem suporte a *caminhos relativos para descritores de diretório*.

No Windows, a tentativa de remover um arquivo que está em uso causa o surgimento de uma exceção; no Unix, a entrada do diretório é removida, mas o armazenamento alocado para o arquivo não é disponibilizado até que o arquivo original não esteja mais em uso.

Esta função é semanticamente idêntica a `unlink()`.

Levanta um *evento de auditoria* `os.remove` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**removedirs**(*name*)

Remove os diretórios recursivamente. Funciona como `rmdir()`, exceto que, se o diretório folha for removido com sucesso, `removedirs()` tenta remover sucessivamente todos os diretórios pai mencionados em *path* até que um erro seja levantado (que é ignorado, porque geralmente significa que um diretório pai não está vazio). Por exemplo, `os.removedirs('foo/bar/baz')` primeiro removerá o diretório `'foo/bar/baz'`, e então removerá `'foo/bar'` e `'foo'` se estiverem vazios. Levanta `OSError` se o diretório folha não pôde ser removido com sucesso.

Levanta um *evento de auditoria* `os.remove` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**rename**(*src*, *dst*, *, *src_dir_fd=None*, *dst_dir_fd=None*)

Renomeia o arquivo ou diretório *src* para *dst*. Se *dst* existir, a operação falhará com uma subclasse de `OSError` em vários casos:

No Windows, se *dst* existe então uma exceção `FileExistsError` será levantada. A operação pode falar se *src* e *dst* são de sistemas de arquivos diferentes. Use `shutil.move()` para mover arquivos entre sistemas de arquivos diferentes.

No Unix, se *src* é um arquivo e *dst* é um diretório ou vice-versa, uma `IsADirectoryError` ou uma `NotADirectoryError` será levantada respectivamente. Se ambos forem diretórios e *dst* estiver vazio, *dst* será substituído silenciosamente. Se *dst* for um diretório não vazio, uma `OSError` é levantada. Se ambos

forem arquivos, *dst* será substituído silenciosamente se o usuário tiver permissão. A operação pode falhar em alguns tipos de Unix se *src* e *dst* estiverem em sistemas de arquivos diferentes. Se for bem-sucedido, a renomeação será uma operação atômica (este é um requisito POSIX).

Esta função permite especificar *src_dir_fd* e/ou *dst_dir_fd* para fornecer *caminhos relativos a descritores de diretório*.

Se você quiser sobrescrita multiplataforma do destino, use `replace()`.

Levanta um *evento de auditoria* `os.rename` com os argumentos `src`, `dst`, `src_dir_fd`, `dst_dir_fd`.

Alterado na versão 3.3: Adicionados os parâmetros *src_dir_fd* e *dst_dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *src* e *dst*.

`os.`**`renames`**(*old*, *new*)

Função de renomeação de arquivo ou diretório recursiva. Funciona como `rename()`, exceto que a criação de qualquer diretório intermediário necessário para tornar o novo nome de caminho possível é tentada primeiro. Após a renomeação, os diretórios correspondentes aos segmentos de caminho mais à direita do nome antigo serão removidos usando `removedirs()`.

> **Nota**
>
> Esta função pode falhar com a nova estrutura de diretório criada se você não tiver as permissões necessárias para remover o arquivo ou diretório folha.

Levanta um *evento de auditoria* `os.rename` com os argumentos `src`, `dst`, `src_dir_fd`, `dst_dir_fd`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *old* e *new*.

`os.`**`replace`**(*src*, *dst*, ***, *src_dir_fd=None*, *dst_dir_fd=None*)

Renomeia o arquivo ou diretório *src* para *dst*. Se *dst* for um diretório não vazio, `OSError` será levantada. Se *dst* existir e for um arquivo, ele será substituído silenciosamente se o usuário tiver permissão. A operação pode falhar se *src* e *dst* estiverem em sistemas de arquivos diferentes. Se for bem-sucedido, a renomeação será uma operação atômica (este é um requisito POSIX).

Esta função permite especificar *src_dir_fd* e/ou *dst_dir_fd* para fornecer *caminhos relativos a descritores de diretório*.

Levanta um *evento de auditoria* `os.rename` com os argumentos `src`, `dst`, `src_dir_fd`, `dst_dir_fd`.

Adicionado na versão 3.3.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *src* e *dst*.

`os.`**`rmdir`**(*path*, ***, *dir_fd=None*)

Remove (exclui) o diretório *path*. Se o diretório não existe ou não está vazio, uma `FileNotFoundError` ou uma `OSError` é levantada respectivamente. Para remover árvores de diretório inteiras, pode ser usada `shutil.rmtree()`.

Esta função tem suporte a *caminhos relativos para descritores de diretório*.

Levanta um *evento de auditoria* `os.rmdir` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.`**`scandir`**(*path='.'*)

Retorna um iterador de objetos `os.DirEntry` correspondentes às entradas no diretório fornecido por *path*. As entradas são produzidas em ordem arbitrária, e as entradas especiais `'.'` e `'..'` não são incluídas. Se um arquivo for removido ou adicionado ao diretório após a criação do iterador, não é especificado se uma entrada para esse arquivo deve ser incluída.

Usar `scandir()` em vez de `listdir()` pode aumentar significativamente o desempenho do código que também precisa de tipo de arquivo ou informações de atributo de arquivo, porque os objetos `os.DirEntry`

expõem essas informações se o sistema operacional fornecer ao percorrer um diretório. Todos os métodos de *os.DirEntry* podem realizar uma chamada de sistema, mas *is_dir()* e *is_file()* normalmente requerem apenas uma chamada de sistema para links simbólicos; *os.DirEntry.stat()* sempre requer uma chamada de sistema no Unix, mas requer uma apenas para links simbólicos no Windows.

path pode ser um *objeto caminho ou similar*. Se *path* for do tipo `bytes` (direta ou indiretamente por meio da interface *PathLike*), o tipo dos atributos *name* e *path* de cada *os.DirEntry* serão `bytes`; em todas as outras circunstâncias, eles serão do tipo `str`.

Esta função também pode ter suporte a *especificar um descritor de arquivo*; o descritor de arquivo deve fazer referência a um diretório.

Levanta um *evento de auditoria* `os.scandir` com o argumento `path`.

O iterador *scandir()* implementa o protocolo *gerenciador de contexto* e tem o seguinte método:

`scandir.`**`close`**`()`

Fecha o iterador e libera os recursos alocados.

Isso é chamado automaticamente quando o iterador se esgota ou é coletado como lixo, ou quando ocorre um erro durante a iteração. No entanto, é aconselhável chamá-lo explicitamente ou usar a instrução `with`.

Adicionado na versão 3.6.

O exemplo a seguir mostra um uso simples de *scandir()* para exibir todos os arquivos (excluindo diretórios) no *path* fornecido que não começa com `'.'`. A chamada `entrada.is_file()` geralmente não fará uma chamada de sistema adicional:

```
with os.scandir(caminho) as coisa:
    for entrada in coisa:
        if not entrada.name.startswith('.') and entrada.is_file():
            print(entrada.name)
```

Nota

Em sistemas baseados no Unix, *scandir()* usa o sistema opendir() e as funções readdir(). No Windows, ela usa os funções FindFirstFileW e FindNextFileW do Win32.

Adicionado na versão 3.5.

Alterado na versão 3.6: Adicionado suporte para o protocolo de *gerenciador de contexto* e o método *close()*. Se um iterador *scandir()* não estiver esgotado nem explicitamente fechado, uma *ResourceWarning* será emitida em seu destrutor.

A função aceita um *objeto caminho ou similar*.

Alterado na versão 3.7: Adicionado suporte para *descritores de arquivo* no Unix.

`class` `os.`**`DirEntry`**

Objeto produzido por *scandir()* para expor o caminho do arquivo e outros atributos de arquivo de uma entrada de diretório.

scandir() fornecerá o máximo possível dessas informações sem fazer chamadas de sistema adicionais. Quando uma chamada de sistema `stat()` ou `lstat()` é feita, o objeto `os.DirEntry` irá armazenar o resultado em cache.

As instâncias de `os.DirEntry` não devem ser armazenadas em estruturas de dados de longa duração; se você sabe que os metadados do arquivo foram alterados ou se passou muito tempo desde a chamada de *scandir()*, chame `os.stat(entry.path)` para obter informações atualizadas.

Como os métodos `os.DirEntry` podem fazer chamadas ao sistema operacional, eles também podem levantar *OSError*. Se você precisa de um controle muito refinado sobre os erros, você pode pegar *OSError* ao chamar um dos métodos `os.DirEntry` e manipular conforme apropriado.

Para ser diretamente utilizável como um *objeto caminho ou similar*, `os.DirEntry` implementa a interface `PathLike`.

Atributos e métodos em uma instância de `os.DirEntry` são os seguintes:

name

O nome do arquivo base da entrada, relativo ao argumento *path* de `scandir()`.

O atributo `name` será `bytes` se o argumento *path* de `scandir()` for do tipo `bytes` e, caso contrário, `str`. Usa `fsdecode()` para decodificar nomes de arquivos de bytes.

path

O nome do caminho completo da entrada: equivalente a `os.path.join(scandir_path, entry.name)` onde *scandir_path* é o argumento *path* de `scandir()`. O caminho só é absoluto se o argumento *path* de `scandir()` for absoluto. Se o argumento *path* de `scandir()` era um *descritor de arquivo*, o atributo `path` é o mesmo que o atributo `name`.

O atributo `path` será `bytes` se o argumento *path* de `scandir()` for do tipo `bytes` e, caso contrário, `str`. Usa `fsdecode()` para decodificar nomes de arquivos de bytes.

inode()

Retorna o número de nó-i da entrada.

O resultado é armazenado em cache no objeto `os.DirEntry`. Use `os.stat(entry.path, follow_symlinks=False).st_ino` para obter informações atualizadas.

Na primeira chamada sem cache, uma chamada de sistema é necessária no Windows, mas não no Unix.

is_dir(*, *follow_symlinks=True*)

Retorna `True` se esta entrada for um diretório ou um link simbólico apontando para um diretório; retorna `False` se a entrada é ou aponta para qualquer outro tipo de arquivo, ou se ele não existe mais.

Se *follow_symlinks* for `False`, retorna `True` apenas se esta entrada for um diretório (sem seguir os links simbólicos); retorna `False` se a entrada for qualquer outro tipo de arquivo ou se ele não existe mais.

O resultado é armazenado em cache no objeto `os.DirEntry`, com um cache separado para *follow_symlinks* `True` e `False`. Chama `os.stat()` junto com `stat.S_ISDIR()` para buscar informações atualizadas.

Na primeira chamada sem cache, nenhuma chamada do sistema é necessária na maioria dos casos. Especificamente, para links não simbólicos, nem o Windows nem o Unix requerem uma chamada de sistema, exceto em certos sistemas de arquivos Unix, como sistemas de arquivos de rede, que retornam `dirent.d_type == DT_UNKNOWN`. Se a entrada for um link simbólico, uma chamada de sistema será necessária para seguir o link simbólico, a menos que *follow_symlinks* seja `False`.

Este método pode levantar `OSError`, tal como `PermissionError`, mas `FileNotFoundError` é capturada e não levantada.

is_file(*, *follow_symlinks=True*)

Retorna `True` se esta entrada for um arquivo ou um link simbólico apontando para um arquivo; retorna `False` se a entrada é ou aponta para um diretório ou outra entrada que não seja de arquivo, ou se ela não existe mais.

Se *follow_symlinks* for `False`, retorna `True` apenas se esta entrada for um arquivo (sem seguir os links simbólicos); retorna `False` se a entrada for um diretório ou outra entrada que não seja um arquivo, ou se ela não existir mais.

O resultado é armazenado em cache no objeto `os.DirEntry`. Chamadas de sistema em cache feitas e exceções levantadas são conforme `is_dir()`.

is_symlink()

Retorna `True` se esta entrada for um link simbólico (mesmo se quebrado); retorna `False` se a entrada apontar para um diretório ou qualquer tipo de arquivo, ou se ele não existir mais.

O resultado é armazenado em cache no objeto `os.DirEntry`. Chama `os.path.islink()` para buscar informações atualizadas.

Na primeira chamada sem cache, nenhuma chamada do sistema é necessária na maioria dos casos. Especificamente, nem o Windows nem o Unix exigem uma chamada de sistema, exceto em certos sistemas de arquivos Unix, como sistemas de arquivos de rede, que retornam `dirent.d_type == DT_UNKNOWN`.

Este método pode levantar *OSError*, tal como *PermissionError*, mas *FileNotFoundError* é capturada e não levantada.

is_junction()

Retorna `True` se esta entrada for uma junção (mesmo se quebrado); retorna `False` se a entrada apontar para um diretório regular, ou qualquer tipo de arquivo, um link simbólico ou se ele não existir mais.

O resultado é armazenado em cache no objeto `os.DirEntry`. Chama *os.path.isjunction()* para buscar informações atualizadas.

Adicionado na versão 3.12.

stat(*, *follow_symlinks=True*)

Retorna um objeto *stat_result* para esta entrada. Este método segue links simbólicos por padrão; para estabelecer um link simbólico, adicione o argumento `follow_symlinks=False`.

No Unix, esse método sempre requer uma chamada de sistema. No Windows, ele só requer uma chamada de sistema se *follow_symlinks* for `True` e a entrada for um ponto de nova análise (por exemplo, um link simbólico ou junção de diretório).

No Windows, os atributos `st_ino`, `st_dev` e `st_nlink` da *stat_result* são sempre definidos como zero. Chame *os.stat()* para obter esses atributos.

O resultado é armazenado em cache no objeto `os.DirEntry`, com um cache separado para *follow_symlinks* `True` e `False`. Chame *os.stat()* para buscar informações atualizadas.

Note que há uma boa correspondência entre vários atributos e métodos de `os.DirEntry` e de *pathlib.Path*. Em particular, o atributo `name` tem o mesmo significado, assim como os métodos `is_dir()`, `is_file()`, `is_symlink()`, `is_junction()` e `stat()`.

Adicionado na versão 3.5.

Alterado na versão 3.6: Adicionado suporte para a interface *PathLike*. Adicionado suporte para caminhos *bytes* no Windows.

Alterado na versão 3.12: O atributo `st_ctime` do resultado de stat foi descontinuado no Windows. A hora de criação do arquivo está devidamente disponível como `st_birthtime`, e no futuro `st_ctime` pode ser alterada para retornar zero ou a hora de alteração dos metadados, se disponível.

`os.`**stat**(*path*, *, *dir_fd=None*, *follow_symlinks=True*)

Obtém o status de um arquivo ou um descritor de arquivo. Executa o equivalente a uma chamada de sistema `stat()` no caminho fornecido. *path* pode ser especificado como uma string ou bytes – direta ou indiretamente através da interface *PathLike* – ou como um descritor de arquivo aberto. Retorna um objeto *stat_result*.

Esta função normalmente segue links simbólicos; para obter o status do link simbólico, adicione o argumento `follow_symlinks=False`, ou use *lstat()*.

Esta função tem suporte a *especificar um descritor de arquivo* e *não seguir links simbólicos*.

No Windows, ao definir `follow_symlinks=False` é desativado o recurso de seguir todos os pontos de nova análise, que incluem links simbólicos e junções de diretório. Outros tipos de pontos de nova análise que não se parecem com links ou que o sistema operacional não pode seguir serão abertos diretamente. Ao seguir uma cadeia de vários links, isso pode resultar no retorno do link original em vez do não-link que impediu o percurso completo. Para obter resultados de *stat* para o caminho final neste caso, use a função *os.path.realpath()* para resolver o nome do caminho tanto quanto possível e chame *lstat()* no resultado. Isso não se aplica a links simbólicos pendentes ou pontos de junção, que levantam as exceções usuais.

Exemplo:

```
>>> import os
>>> statinfo = os.stat('algumarquivo.txt')
```

(continua na próxima página)

(continuação da página anterior)

```
>>> statinfo
os.stat_result(st_mode=33188, st_ino=7876932, st_dev=234881026,
st_nlink=1, st_uid=501, st_gid=501, st_size=264, st_atime=1297230295,
st_mtime=1297230027, st_ctime=1297230027)
>>> statinfo.st_size
264
```

Ver também

As funções *fstat()* e *lstat()*.

Alterado na versão 3.3: Adicionados os parâmetros *dir_fd* e *follow_symlinks*, especificando um descritor de arquivo em vez de um caminho.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.8: No Windows, todos os pontos de nova análise que podem ser resolvidos pelo sistema operacional agora são seguidos, e passar `follow_symlinks=False` desativa seguir todos os pontos de nova análise substitutos de nome. Se o sistema operacional atingir um ponto de nova análise que não é capaz de seguir, *stat* agora retorna as informações do caminho original como se `follow_symlinks=False` tivesse sido especificado em vez de levantar um erro.

class os.**stat_result**

Objeto cujos atributos correspondem aproximadamente aos membros da estrutura `stat`. É usado para o resultado de *os.stat()*, *os.fstat()* e *os.lstat()*.

Atributos:

st_mode

Modo de arquivo: tipo de arquivo e bits de modo de arquivo (permissões).

st_ino

Dependente da plataforma, mas se diferente de zero, identifica exclusivamente o arquivo para um determinado valor de `st_dev`. Tipicamente:

- o número do nó-i no Unix,
- o índice de arquivo no Windows

st_dev

Identificador do dispositivo no qual este arquivo reside.

st_nlink

Número de links físicos.

st_uid

Identificador de usuário do proprietário do arquivo.

st_gid

Identificador de grupo do proprietário do arquivo.

st_size

Tamanho do arquivo em bytes, se for um arquivo normal ou um link simbólico. O tamanho de um link simbólico é o comprimento do nome do caminho que ele contém, sem um byte nulo final.

Registros de data e hora:

st_atime

Hora do acesso mais recente expressa em segundos.

st_mtime

Hora da modificação de conteúdo mais recente expressa em segundos.

st_ctime

Hora da alteração de metadados mais recente expressa em segundos.

Alterado na versão 3.12: `st_ctime` foi descontinuado no Windows. Use `st_birthtime` para o tempo de criação do arquivo. No futuro, `st_ctime` conterá a hora da alteração de metadados mais recente, como em outras plataformas.

st_atime_ns

Hora do acesso mais recente expresso em nanossegundos como um número inteiro.

Adicionado na versão 3.3.

st_mtime_ns

Hora da modificação de conteúdo mais recente expressa em nanossegundos como um número inteiro.

Adicionado na versão 3.3.

st_ctime_ns

Hora do alteração de metadados mais recente expresso em nanossegundos como um número inteiro.

Adicionado na versão 3.3.

Alterado na versão 3.12: `st_ctime_ns` foi descontinuado no Windows. Use `st_birthtime_ns` para o tempo de criação do arquivo. No futuro, `st_ctime` conterá a hora da alteração de metadados mais recente, como em outras plataformas.

st_birthtime

Hora de criação do arquivo expressa em segundos. Este atributo nem sempre está disponível e pode levantar *AttributeError*.

Alterado na versão 3.12: `st_birthtime` está agora disponível no Windows.

st_birthtime_ns

Hora de criação do arquivo expressa em nanossegundos em um inteiro. Este atributo nem sempre está disponível e pode levantar *AttributeError*.

Adicionado na versão 3.12.

Nota

O significado e resolução exatos dos atributos *st_atime*, *st_mtime*, *st_ctime* e *st_birthtime* dependem do sistema operacional e do sistema de arquivos. Por exemplo, em sistemas Windows que usam os sistemas de arquivos FAT32, *st_mtime* tem resolução de 2 segundos, e *st_atime* tem resolução de apenas 1 dia. Consulte a documentação do sistema operacional para obter detalhes.

Da mesma forma, embora *st_atime_ns*, *st_mtime_ns*, *st_ctime_ns* e *st_birthtime_ns* sejam sempre expressos em nanossegundos, muitos sistemas não fornecem precisão de nanossegundos. Em sistemas que fornecem precisão de nanossegundos, o objeto de ponto flutuante usado para armazenar *st_atime*, *st_mtime*, *st_ctime* e *st_birthtime* não pode preservar tudo, e como tal será ligeiramente inexato. Se você precisa dos carimbos de data/hora exatos, sempre deve usar *st_atime_ns*, *st_mtime_ns*, *st_ctime_ns* e *st_birthtime_ns*.

Em alguns sistemas Unix (como Linux), os seguintes atributos também podem estar disponíveis:

st_blocks

Número de blocos de 512 bytes alocados para o arquivo. Isso pode ser menor que *st_size*/512 quando o arquivo possuir lacunas.

st_blksize

Tamanho de bloco "preferido" para E/S eficiente do sistema de arquivos. Gravar em um arquivo em partes menores pode causar uma leitura-modificação-reescrita ineficiente.

st_rdev

Tipo de dispositivo, se for um dispositivo nó-i.

st_flags

Sinalizadores definidos pelo usuário para o arquivo.

Em outros sistemas Unix (como o FreeBSD), os seguintes atributos podem estar disponíveis (mas só podem ser preenchidos se o root tentar usá-los):

st_gen

Número de geração do arquivo.

No Solaris e derivados, os seguintes atributos também podem estar disponíveis:

st_fstype

String que identifica exclusivamente o tipo de sistema de arquivos que contém o arquivo.

Em sistemas macOS, os seguintes atributos também podem estar disponíveis:

st_rsize

Tamanho real do arquivo.

st_creator

Criador do arquivo.

st_type

Tipo de arquivo.

Em sistemas Windows, os seguintes atributos também estão disponíveis:

st_file_attributes

Atributos de arquivos no Windows: membro `dwFileAttributes` da estrutura `BY_HANDLE_FILE_INFORMATION` retornada por `GetFileInformationByHandle()`. Veja as constantes `FILE_ATTRIBUTE_* <stat.FILE_ATTRIBUTE_ARCHIVE>` no módulo *stat*.

Adicionado na versão 3.5.

st_reparse_tag

Quando *st_file_attributes* tem *FILE_ATTRIBUTE_REPARSE_POINT* definido, este campo contém uma tag identificando o tipo do ponto de nova análise. Veja as constantes *IO_REPARSE_TAG_** no módulo *stat*.

O módulo padrão *stat* define funções e constantes que são úteis para extrair informações de uma estrutura `stat`. (No Windows, alguns itens são preenchidos com valores fictícios.)

Para compatibilidade com versões anteriores, uma instância de *stat_result* também é acessível como uma tupla de pelo menos 10 inteiros, fornecendo os membros mais importantes (e portáveis) da estrutura `stat`, na ordem *st_mode*, *st_ino*, *st_dev*, *st_nlink*, *st_uid*, *st_gid*, *st_size*, *st_atime*, *st_mtime*, *st_ctime*. Mais itens podem ser adicionados no final por algumas implementações. Para compatibilidade com versões mais antigas do Python, acessar *stat_result* como uma tupla sempre retorna inteiros.

Alterado na versão 3.5: Windows agora retorna o arquivo de índice como *st_ino* quando disponível.

Alterado na versão 3.7: Adicionado o membro *st_fstype* ao Solaris/derivados.

Alterado na versão 3.8: Adicionado o membro *st_reparse_tag* no Windows.

Alterado na versão 3.8: No Windows, o membro *st_mode* agora identifica arquivos especiais como `S_IFCHR`, `S_IFIFO` ou `S_IFBLK` conforme apropriado.

Alterado na versão 3.12: No Windows, *st_ctime* foi descontinuado. Eventualmente, ele conterá a hora da última alteração de metadados, para fins de consistência com outras plataformas, mas por enquanto ainda contém a hora de criação. Use *st_birthtime* para a hora de criação.

No Windows, o *st_ino* agora pode ter até 128 bits, dependendo do sistema de arquivos. Anteriormente, não era possível ter mais de 64 bits, e identificadores de arquivo maiores seriam compactados arbitrariamente.

No Windows, *st_rdev* não retorna mais um valor. Anteriormente, ele continha o mesmo que *st_dev*, o que era incorreto.

Adicionado o membro *st_birthtime* no Windows.

os.**statvfs**(*path*)

Executa uma chamada de sistema `statvfs()` no caminho fornecido. O valor de retorno é um objeto cujos atributos descrevem o sistema de arquivos no caminho fornecido e correspondem aos membros da estrutura `statvfs`, a saber: `f_bsize`, `f_frsize`, `f_blocks`, `f_bfree`, `f_bavail`, `f_files`, `f_ffree`, `f_favail`, `f_flag`, `f_namemax`, `f_fsid`.

Duas constantes em nível de módulo são definidas para os sinalizadores de bit do atributo `f_flag`: se `ST_RDONLY` estiver definido, o sistema de arquivos é montado somente leitura, e se `ST_NOSUID` estiver definido, a semântica dos bits setuid/setgid é desabilitada ou não é implementada.

Constantes de nível de módulo adicionais são definidas para sistemas baseados em GNU/glibc. São elas `ST_NODEV` (impede o acesso aos arquivos especiais do dispositivo), `ST_NOEXEC` (não permite a execução do programa), `ST_SYNCHRONOUS` (as escritas são sincronizadas de uma vez), `ST_MANDLOCK` (permitir travas obrigatórias em um sistema de arquivos), `ST_WRITE` (escrever no arquivo/diretório/link simbólico), `ST_APPEND` (arquivo somente anexado), `ST_IMMUTABLE` (arquivo imutável), `ST_NOATIME` (não atualiza os tempos de acesso), `ST_NODIRATIME` (não atualiza os tempos de acesso ao diretório), `ST_RELATIME` (atualiza o atime relativo ao mtime/ctime).

Esta função tem suporte a *especificar um descritor de arquivo*.

Disponibilidade: Unix.

Alterado na versão 3.2: As constantes `ST_RDONLY` e `ST_NOSUID` foram adicionadas.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto.

Alterado na versão 3.4: As constantes `ST_NODEV`, `ST_NOEXEC`, `ST_SYNCHRONOUS`, `ST_MANDLOCK`, `ST_WRITE`, `ST_APPEND`, `ST_IMMUTABLE`, `ST_NOATIME`, `ST_NODIRATIME` e `ST_RELATIME` foram adicionadas.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.7: Adicionado o atributo `f_fsid`.

os.**supports_dir_fd**

Um objeto *set* indicando quais funções no módulo *os* aceitam um descritor de arquivo aberto para seu parâmetro *dir_fd*. Plataformas diferentes fornecem recursos diferentes, e a funcionalidade subjacente que o Python usa para implementar o parâmetro *dir_fd* não está disponível em todas as plataformas que o Python provê suporte. Para fins de consistência, as funções que podem implementar *dir_fd* sempre permitem especificar o parâmetro, mas lançarão uma exceção se a funcionalidade for usada quando não estiver disponível localmente. (Especificar `None` para *dir_fd* é sempre válido em todas as plataformas.)

Para verificar se uma função particular aceita um descritor de arquivo aberto para seu parâmetro *dir_fd*, use o operador `in` em `supports_dir_fd`. Como exemplo, esta expressão é avaliada como `True` se *os.stat()* aceita descritores de arquivo abertos para *dir_fd* na plataforma local:

```
os.stat in os.supports_dir_fd
```

Atualmente os parâmetros *dir_fd* funcionam apenas em plataformas Unix; nenhum deles funciona no Windows.

Adicionado na versão 3.3.

os.**supports_effective_ids**

Um objeto *set* que indica se *os.access()* permite especificar `True` para seu parâmetro *effective_ids* na plataforma local. (Especificar `False` para *effective_ids* é sempre válido em todas as plataformas.) Se a plataforma local oferecer suporte, a coleção conterá *os.access()*; caso contrário, ficará vazio.

Esta expressão é avaliada como `True` se *os.access()* oferecer suporte a `effective_ids=True` na plataforma local:

```
os.access in os.supports_effective_ids
```

Atualmente, *effective_ids* funciona apenas em plataformas Unix; não funciona no Windows.

Adicionado na versão 3.3.

os.supports_fd

Um objeto *set* que indica quais funções no módulo *os* permitem especificar seu parâmetro *path* como um descritor de arquivo aberto na plataforma local. Plataformas diferentes fornecem recursos diferentes, e a funcionalidade subjacente que o Python usa para aceitar descritores de arquivos abertos como argumentos *path* não está disponível em todas as plataformas que o Python provê suporte.

Para determinar se uma função particular permite especificar um descritor de arquivo aberto para seu parâmetro *path*, use o operador `in` em `supports_fd`. Como exemplo, esta expressão é avaliada como `True` se *os.chdir()* aceita descritores de arquivo abertos para *path* em sua plataforma local:

```
os.chdir in os.supports_fd
```

Adicionado na versão 3.3.

os.supports_follow_symlinks

Um objeto *set* que indica quais funções no módulo *os* aceitam `False` para seu parâmetro *follow_symlinks* na plataforma local. Plataformas diferentes fornecem recursos diferentes, e a funcionalidade subjacente que o Python usa para implementar *follow_symlinks* não está disponível em todas as plataformas às quais o Python tem suporte. Para fins de consistência, as funções que podem ter suporte a *follow_symlinks* sempre permitem especificar o parâmetro, mas irão lançar uma exceção se a funcionalidade for usada quando não estiver disponível localmente. (Especificar `True` para *follow_symlinks* é sempre aceito em todas as plataformas.)

Para verificar se uma função em particular aceita `False` para seu parâmetro *follow_symlinks*, use o operador `in` em `supports_follow_symlinks`. Como exemplo, esta expressão é avaliada como `True` se você pode especificar `follow_symlinks=False` ao chamar *os.stat()* na plataforma local:

```
os.stat in os.supports_follow_symlinks
```

Adicionado na versão 3.3.

os.symlink(*src*, *dst*, *target_is_directory=False*, ***, *dir_fd=None*)

Cria um link simbólico apontando para *src* chamado *dst*.

No Windows, um link simbólico representa um arquivo ou um diretório e não se transforma no destino dinamicamente. Se o alvo estiver presente, será criado um link simbólico de mesmo tipo. Caso contrário, o link simbólico será criado como um diretório se *target_is_directory* for `True` ou um link simbólico de arquivo (o padrão) caso contrário. Em plataformas não Windows, *target_is_directory* é ignorado.

Esta função tem suporte a *caminhos relativos para descritores de diretório*.

> **Nota**
>
> Em versões mais recentes do Windows 10, contas sem privilégios podem criar links simbólicos se o Modo de Desenvolvedor estiver habilitado. Quando o modo de desenvolvedor não está disponível/ativado, o privilégio *SeCreateSymbolicLinkPrivilege* é necessário ou o processo deve ser executado como um administrador.
>
> A exceção *OSError* é levantada quando a função é chamada por um usuário sem privilégios.

Levanta um *evento de auditoria* `os.symlink` com os argumentos `src`, `dst` e `dir_fd`.

Disponibilidade: Unix, Windows.

A função é limitada em WASI, veja *Plataformas WebAssembly* para mais informações.

Alterado na versão 3.2: Adicionado suporte para links simbólicos do Windows 6.0 (Vista).

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*, e agora permite *target_is_directory* em plataformas não Windows.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *src* e *dst*.

Alterado na versão 3.8: Adicionado suporte para links simbólicos não elevados no Windows com Modo de Desenvolvedor.

`os.sync()`

Força a escrita de tudo para o disco.

Disponibilidade: Unix.

Adicionado na versão 3.3.

`os.truncate(path, length)`

Trunca o arquivo correspondente ao *path*, de modo que tenha no máximo *length* bytes.

Esta função tem suporte a *especificar um descritor de arquivo*.

Levanta um *evento de auditoria* `os.truncate` com os argumentos `path` e `length`.

Disponibilidade: Unix, Windows.

Adicionado na versão 3.3.

Alterado na versão 3.5: Adicionado suporte para o Windows.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.unlink(path, *, dir_fd=None)`

Remove (exclui) o arquivo *path*. Esta função é semanticamente idêntica à `remove()`; o nome `unlink` é seu nome Unix tradicional. Por favor, veja a documentação de `remove()` para mais informações.

Levanta um *evento de auditoria* `os.remove` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.utime(path, times=None, *, [ns, ]dir_fd=None, follow_symlinks=True)`

Define os tempos de acesso e modificação do arquivo especificado por *path*.

`utime()` aceita dois parâmetros opcionais, *times* e *ns*. Eles especificam os horários definidos no *path* e são usados da seguinte forma:

- Se *ns* for especificado, deve ser uma tupla de 2 elementos na forma `(atime_ns, mtime_ns)` onde cada membro é um inteiro expressando nanossegundos.
- Se *times* não for `None`, deve ser uma tupla de 2 elementos na forma `(atime, mtime)` onde cada membro é um inteiro ou ponto flutuante expressando segundos.
- Se *times* for `None` e *ns* não for especificado, isso é equivalente a especificar `ns=(atime_ns, mtime_ns)` onde ambos os tempos são a hora atual.

É um erro especificar tuplas para ambos *times* e *ns*.

Observe que os tempos exatos que você definiu aqui podem não ser retornados por uma chamada subsequente de `stat()`, dependendo da resolução com a qual seu sistema operacional registra os tempos de acesso e modificação; veja `stat()`. A melhor maneira de preservar os tempos exatos é usar os campos *st_atime_ns* e *st_mtime_ns* do objeto de resultado `os.stat()` com o parâmetro *ns* para `utime()`.

Esta função pode oferecer suporte a *especificar um descritor de arquivo*, *caminhos relativos aos descritores de diretório* e *não seguir os links simbólicos*.

Levanta um *evento de auditoria* `os.utime` com os argumentos `path`, `times`, `ns` e `dir_fd`.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto e os parâmetros *dir_fd*, *follow_symlinks* e *ns*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.walk`(*top*, *topdown=True*, *onerror=None*, *followlinks=False*)

Gera os nomes dos arquivos em uma árvore de diretório percorrendo a árvore de cima para baixo ou de baixo para cima. Para cada diretório na árvore com raiz no diretório *top* (incluindo o próprio *top*), ele produz uma tupla de 3 elementos `(dirpath, dirnames, filenames)`.

dirpath é uma string, o caminho para o diretório. *dirnames* é uma lista dos nomes dos subdiretórios em *dirpath* (incluindo links simbólicos para diretórios e excluindo `'.'` e `'..'`). *filenames* é uma lista dos nomes dos arquivos não pertencentes ao diretório em *dirpath*. Observe que os nomes nas listas não contêm componentes de caminho. Para obter um caminho completo (que começa com *top*) para um arquivo ou diretório em *dirpath*, execute `os.path.join(dirpath, name)`. Se as listas são ou não ordenadas depende do sistema de arquivos. Se um arquivo for removido ou adicionado ao diretório *dirpath* durante a geração das listas, não é especificado se um nome para esse arquivo deve ser incluído.

Se o argumento opcional *topdown* for `True` ou não especificado, o triplo para um diretório é gerado antes dos triplos para qualquer um de seus subdiretórios (os diretórios são gerados de cima para baixo). Se *topdown* for `False`, o triplo para um diretório é gerado após os triplos para todos os seus subdiretórios (os diretórios são gerados de baixo para cima). Não importa o valor de *topdown*, a lista de subdiretórios é recuperada antes que as tuplas para o diretório e seus subdiretórios sejam geradas.

Quando *topdown* é `True`, o chamador pode modificar a lista de *dirnames* internamente (talvez usando `del` ou atribuição de fatia), e *walk()* só recursará nos subdiretórios cujos nomes permanecem em *dirnames*; isso pode ser usado para podar a busca, impor uma ordem específica de visita, ou mesmo informar à função *walk()* sobre os diretórios que o chamador cria ou renomeia antes de retomar *walk()* novamente. Modificar *dirnames* quando *topdown* for `False` não tem efeito no comportamento da caminhada, porque no modo de baixo para cima os diretórios em *dirnames* são gerados antes do próprio *dirpath* ser gerado.

Por padrão, os erros da chamada de *scandir()* são ignorados. Se o argumento opcional *onerror* for especificado, deve ser uma função; ela será chamado com um argumento, uma instância da exceção *OSError*. Ele pode relatar o erro para continuar com a caminhada ou levantar a exceção para abortar a caminhada. Observe que o nome do arquivo está disponível como o atributo `filename` do objeto de exceção.

Por padrão, a função *walk()* não vai seguir links simbólicos que resolvem para diretórios. Defina *followlinks* como `True` para visitar diretórios apontados por links simbólicos, em sistemas que oferecem suporte a eles.

Nota

Esteja ciente de que definir *followlinks* para `True` pode levar a recursão infinita se um link apontar para um diretório pai de si mesmo. *walk()* não mantém registro dos diretórios que já visitou.

Nota

Se você passar um nome de caminho relativo, não mude o diretório de trabalho atual entre as continuações de *walk()*. *walk()* nunca muda o diretório atual, e presume que seu chamador também não.

Este exemplo exibe o número total de bytes dos arquivos não-diretório em cada diretório no diretório inicial, exceto que ele não olha em nenhum subdiretório chamado `__pycache__`:

```
import os
from os.path import join, getsize
for root, dirs, files in os.walk('python/Lib/xml'):
    print(root, "consome", end=" ")
    print(sum(getsize(join(root, name)) for name in files), end=" ")
    print("bytes em", len(files), "arquivos que não são diretórios")
    if '__pycache__' in dirs:
        dirs.remove('__pycache__')  # não visita diretórios de __pycache__
```

No próximo exemplo (implementação simples de *shutil.rmtree()*), andar na árvore de baixo para cima é essencial, *rmdir()* não permite excluir um diretório antes que o diretório esteja vazio:

```
# Exclui tudo que for acessível do diretório nomeado em "top",
# presumindo que não há links simbólicos.
# CUIDADO: Isso é perigoso! Por exemplo, se top == '/',
# ele pode excluir todos os seus arquivos de disco.
import os
for root, dirs, files in os.walk(top, topdown=False):
    for name in files:
        os.remove(os.path.join(root, name))
    for name in dirs:
        os.rmdir(os.path.join(root, name))
os.rmdir(top)
```

Levanta um *evento de auditoria* `os.walk` com os argumentos `top`, `topdown`, `onerror` e `followlinks`.

Alterado na versão 3.5: Esta função agora chama *os.scandir()* em vez de *os.listdir()*, tornando-a mais rápida reduzindo o número de chamadas a *os.stat()*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os.`**`fwalk`**(*top='.'*, *topdown=True*, *onerror=None*, ***, *follow_symlinks=False*, *dir_fd=None*)

Se comporta exatamente como *walk()*, exceto que produz um tupla de 4 elementos `(dirpath, dirnames, filenames, dirfd)`, e tem suporte a `dir_fd`.

dirpath, *dirnames* e *filenames* são idênticos à saída de *walk()* e *dirfd* é um descritor de arquivo que faz referência ao diretório *dirpath*.

Esta função sempre tem suporte a *caminhos relativos aos descritores de diretório* e *não seguir links simbólicos*. Observe, entretanto, que, ao contrário de outras funções, o valor padrão *fwalk()* para *follow_symlinks* é `False`.

Nota

Uma vez que *fwalk()* produz descritores de arquivo, eles só são válidos até a próxima etapa de iteração, então você deve duplicá-los (por exemplo, com *dup()*) se quiser mantê-los por mais tempo.

Este exemplo exibe o número total de bytes dos arquivos não-diretório em cada diretório no diretório inicial, exceto que ele não olha em nenhum subdiretório chamado `__pycache__`:

```
import os
for root, dirs, files, rootfd in os.fwalk('python/Lib/xml'):
    print(root, "consome", end=" ")
    print(sum([os.stat(name, dir_fd=rootfd).st_size for name in files]),
          end=" ")
    print("bytes em", len(files), "arquivos que não são diretórios")
    if '__pycache__' in dirs:
        dirs.remove('__pycache__')  # não visita diretórios de __pycache__
```

No próximo exemplo, percorrer a árvore de baixo para cima é essencial: *rmdir()* não permite excluir um diretório antes que o diretório esteja vazio:

```
# Exclui tudo que for acessível do diretório nomeado em "top",
# presumindo que não há links simbólicos.
# CUIDADO: Isso é perigoso! Por exemplo, se top == '/',
# ele pode excluir todos os seus arquivos de disco.
import os
for root, dirs, files, rootfd in os.fwalk(top, topdown=False):
```

(continua na próxima página)

(continuação da página anterior)

```
    for name in files:
        os.unlink(name, dir_fd=rootfd)
    for name in dirs:
        os.rmdir(name, dir_fd=rootfd)
```

Levanta um *evento de auditoria* `os.fwalk` com os argumentos `top`, `topdown`, `onerror`, `follow_symlinks` e `dir_fd`.

Disponibilidade: Unix.

Adicionado na versão 3.3.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.7: Adicionado suporte para caminhos em `bytes`.

os.**memfd_create**(*name*[, *flags=os.MFD_CLOEXEC*])

Cria um arquivo anônimo e retorna um descritor de arquivo que faça referência a ele. *flags* deve ser uma das constantes `os.MFD_*` disponíveis no sistema (ou uma combinação de OU bit a bit delas). Por padrão, o novo descritor de arquivo é *não herdável*.

O nome fornecido em *name* é usado como um nome de arquivo e será exibido como o destino do link simbólico correspondente no diretório `/proc/self/fd/`. O nome exibido é sempre prefixado com `memfd:` e serve apenas para fins de depuração. Os nomes não afetam o comportamento do descritor de arquivo e, como tal, vários arquivos podem ter o mesmo nome sem quaisquer efeitos colaterais.

Disponibilidade: Linux >= 3.17 with glibc >= 2.27.

Adicionado na versão 3.8.

os.**MFD_CLOEXEC**

os.**MFD_ALLOW_SEALING**

os.**MFD_HUGETLB**

os.**MFD_HUGE_SHIFT**

os.**MFD_HUGE_MASK**

os.**MFD_HUGE_64KB**

os.**MFD_HUGE_512KB**

os.**MFD_HUGE_1MB**

os.**MFD_HUGE_2MB**

os.**MFD_HUGE_8MB**

os.**MFD_HUGE_16MB**

os.**MFD_HUGE_32MB**

os.**MFD_HUGE_256MB**

os.**MFD_HUGE_512MB**

os.**MFD_HUGE_1GB**

os.**MFD_HUGE_2GB**

os.**MFD_HUGE_16GB**

Esses sinalizadores podem ser passados para `memfd_create()`.

Disponibilidade: Linux >= 3.17 with glibc >= 2.27

Os sinalizadores `MFD_HUGE*` estão disponíveis somente a partir do Linux 4.14.

Adicionado na versão 3.8.

os.**eventfd**(*initval*[, *flags=os.EFD_CLOEXEC*])

Cria e retorna um descritor de arquivo de evento. Os descritores de arquivo implementam `read()` e `write()` brutos com um tamanho de buffer de 8, `select()`, `poll()` e similares. Veja a página man `eventfd(2)` para mais informações. Por padrão, o novo descritor de arquivo é *não herdável*.

initval é o valor inicial do contador de evento. O valor inicial deve ser um inteiro sem sinal de 32 bits. Observe que o valor inicial é limitado a um inteiro sem sinal de 32 bits ainda que o contador de evento seja um inteiro de 64 bits com um valor máximo de 2^{64}-2.

flags podem ser construídas a partir de *EFD_CLOEXEC*, *EFD_NONBLOCK* e *EFD_SEMAPHORE*.

Se *EFD_SEMAPHORE* for especificado e o contador de evento for diferente de zero, *eventfd_read()* retorna 1 e decrementa o contador em um.

Se *EFD_SEMAPHORE* não for especificado e o contador de evento for diferente de zero, *eventfd_read()* retorna o valor atual do contador de evento e zera o contador.

Se o contador de evento for zero e *EFD_NONBLOCK* não for especificado, *eventfd_read()* bloqueia.

eventfd_write() incrementa o contador de evento. A escrita bloqueia se a operação de escrita incrementar o contador para um valor maior que 2^{64}-2.

Exemplo:

```
import os

# semáforo com valor inicial '1'
fd = os.eventfd(1, os.EFD_SEMAPHORE | os.EFC_CLOEXEC)
try:
    # adquire o semário
    v = os.eventfd_read(fd)
    try:
        faz_o_trabalho()
    finally:
        # libera o semáforo
        os.eventfd_write(fd, v)
finally:
    os.close(fd)
```

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.10.

os.**eventfd_read**(*fd*)

Lê um valor de um descritor de arquivo *eventfd()* e retorna um inteiro sem sinal de 64 bits. A função não verifica se *fd* é um *eventfd()*.

Disponibilidade: Linux >= 2.6.27

Adicionado na versão 3.10.

os.**eventfd_write**(*fd*, *value*)

Adiciona *value* a um descritor de arquivo *eventfd()*. *value* deve ser um inteiro sem sinal de 64 bits. A função não verifica se *fd* é um *eventfd()*.

Disponibilidade: Linux >= 2.6.27

Adicionado na versão 3.10.

os.**EFD_CLOEXEC**

Define sinalizador close-on-exec para o novo descritor de arquivo *eventfd()*.

Disponibilidade: Linux >= 2.6.27

Adicionado na versão 3.10.

os.**EFD_NONBLOCK**

Define sinalizador de status *O_NONBLOCK* para um novo descritor de arquivo *eventfd()*.

Disponibilidade: Linux >= 2.6.27

Adicionado na versão 3.10.

os.**EFD_SEMAPHORE**

Fornece semântica de semáforo ou similar para leitura de um descritor de arquivo *eventfd()*. Na leitura o contador interno é decrementado em um.

Disponibilidade: Linux >= 2.6.30

Adicionado na versão 3.10.

Descritores de arquivo de temporizador

Adicionado na versão 3.13.

Essas funções fornecem suporte para a API de *descritores de arquivo de temporizador*.

os.**timerfd_create**(*clockid*, /, *, *flags=0*)

Cria e retorna um descritor de arquivo de temporizador (*timerfd*).

O descritor de arquivo retornado por *timerfd_create()* implementa:

- *read()*
- *select()*
- *poll()*

O método *read()* do descritor de arquivo pode ser chamado com um buffer de tamanho 8. Se o temporizador já expirou uma ou mais vezes, *read()* retorna o número de vezes que o descritor expirou com a ordem dos bytes no formato do hospedeiro, que pode ser convertido para um *int* com `int.from_bytes(x, byteorder=sys.byteorder)`.

select() e *poll()* pode ser usado para esperar até que o temporizador expire e o descritor de arquivo possa ser lido.

clockid deve ser um *ID de relógio* válido, conforme definido no módulo *time*:

- *time.CLOCK_REALTIME*
- *time.CLOCK_MONOTONIC*
- *time.CLOCK_BOOTTIME* (A partir de Linux 3.15 para timerfd_create)

Se *clockid* é *time.CLOCK_REALTIME*, é usado um relógio em tempo real definido em todo o sistema. Se o relógio do sistema muda, a configuração do temporizador precisa ser atualizado. Para cancelar o temporizador quando o relógio do sistema é alterado, veja *TFD_TIMER_CANCEL_ON_SET*.

Se *clockid* é *time.CLOCK_MONOTONIC*, é usado um relógio monotonicamente não estável. Mesmo se o relógio do sistema é alterado, a configuração do temporizador não será afetada.

Se *clockid* é *time.CLOCK_BOOTTIME*, é idêntica a *time.CLOCK_MONOTONIC* exceto por incluir qualquer tempo que o sistema está suspenso.

O comportamento do descritor de arquivo pode ser modificado especificando um valor para *flags*. Qualquer um dos seguintes valores pode ser usado, combinadas usando OU (OR) bit a bit (o operador `|`):

- *TFD_NONBLOCK*
- *TFD_CLOEXEC*

Se *TFD_NONBLOCK* não está definido como um sinalizador, *read()* bloqueia até o temporizador expirar. Se está definido como um sinalizador, *read()* não bloqueia, mas se não expirou desde a última chamada, *read()* levanta *OSError* com `errno` definido como *errno.EAGAIN*.

TFD_CLOEXEC é sempre definido pelo Python automaticamente.

O descritor de arquivo deve ser fechado com *os.close()* quando não é mais necessário, caso contrário o descritor de arquivo será vazado.

Ver também

A página man *timerfd_create(2)*.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.**timerfd_settime**(*fd*, /, *, *flags=flags*, *initial=0.0*, *interval=0.0*)

Altera um temporizador interno do descritor de arquivo de temporizador. Essa função opera no mesmo intervalo do temporizador definido com *timerfd_settime_ns()*.

fd deve ser um descritor de arquivo de temporizador válido.

O comportamento do temporizador pode ser modificado especificando um valor para *flags*. Qualquer um dos seguintes valores pode ser usado, combinados usando OU (OR) bit a bit (o operador `|`):

- *TFD_TIMER_ABSTIME*
- *TFD_TIMER_CANCEL_ON_SET*

O temporizador é desabilitado definindo *initial* para zero (`0`). Se *initial* é igual ou maior que zero, o temporizador é habilitado. Se *initial* é menor que zero, uma exceção *OSError* é levantada com `errno` definido como *errno.EINVAL*.

Por padrão, o temporizador vai disparar quando passarem *initial* segundos. (Se *initial* é zero, o temporizador dispara imediatamente.)

Contudo, se o sinalizador *TFD_TIMER_ABSTIME* está definido, o temporizador dispara quando o relógio do temporizador (definido por *clockid* em *timerfd_create()*) atingir *initial* segundos.

O intervalor do temporizador é definido pelo *float* *interval*. Se *interval* é zero, o temporizador irá disparar apenas uma vez, no disparo inicial. Se *interval* é maior que zero, o temporizador vai disparar a cada *interval* segundos a partir do disparo anterior. Se *interval* é menor que zero, uma exceção *OSError* será levantada com `errno` definido como *errno.EINVAL*.

Se o sinalizador *TFD_TIMER_CANCEL_ON_SET* é definido com *TFD_TIMER_ABSTIME* e o relógio para o temporizador é *time.CLOCK_REALTIME*, o temporizador é marcado como cancelável se o relógio em tempo real é alterado descontinuadamente. Leitura do descritor é abortada com o erro ECANCELED.

Linux gerencia relógio do sistema como UTC. A transição do horário de verão é feita apenas pela alteração da compensação de horário e não causa alteração descontínua do relógio do sistema.

A alteração descontínua do relógio do sistema será causada pelos seguintes eventos:

- `settimeofday`
- `clock_settime`
- definir hora e data do sistema com o comando `date`

Retorna uma tupla com dois elementos (`next_expiration`, `interval`) do estado anterior do temporizador, antes da execução dessa função.

Ver também

timerfd_create(2), *timerfd_settime(2)*, *settimeofday(2)*, *clock_settime(2)*, e *date(1)*.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.**timerfd_settime_ns**(*fd*, /, *, *flags=0*, *initial=0*, *interval=0*)

Similar a *timerfd_settime()*, mas usa nanossegundos. Essa função opera no mesmo intervalo do temporizador definido com *timerfd_settime()*.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.**timerfd_gettime**(*fd*, /)

Retorna uma tupla de ponto flutuante com dois elementos (`next_expiration`, `interval`).

`next_expiration` indica o tempo relativo até o próximo disparo do cronômetro, independentemente do sinalizador *TFD_TIMER_ABSTIME* estar definido.

`interval` indica o intervalo do cronômetro. Se for zero, o cronômetro será acionado apenas uma vez, após terem passados `next_expiration` segundos.

Ver também

timerfd_gettime(2)

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.**timerfd_gettime_ns**(*fd*, /)

Similar a *timerfd_gettime()*, mas retorna o tempo em nanossegundos.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.**TFD_NONBLOCK**

Um sinalizador para a função *timerfd_create()*, que define o estado do sinalizador *O_NONBLOCK* para o novo descritor de arquivo de temporizador. Se *TFD_NONBLOCK* não for definido como sinalizador, *read()* bloqueia.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.**TFD_CLOEXEC**

Um sinalizador para a função *timerfd_create()*. Se *TFD_CLOEXEC* for definido como um sinalizador, define o sinalizador close-on-exec para o novo descritor de arquivo.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.**TFD_TIMER_ABSTIME**

Um sinalizador para as funções *timerfd_settime()* e *timerfd_settime_ns()*. Se esse sinalizador for definido, *initial* será intepretador como um valor absoluto no relógio do temporizador (em segundos UTC ou nanossegundos da Era Unix).

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.**TFD_TIMER_CANCEL_ON_SET**

Um sinalizador para as funções *timerfd_settime()* e *timerfd_settime_ns()* juntamente com *TFD_TIMER_ABSTIME*. O cronômetro é cancelado quando a hora do relógio subjacente muda de forma descontínua.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

Atributos estendidos do Linux

Adicionado na versão 3.3.

Estas funções estão todas disponíveis apenas no Linux.

os.**getxattr**(*path*, *attribute*, ***, *follow_symlinks=True*)

Retorna o valor do atributo estendido do sistema de arquivos *attribute* para *path*. *attribute* pode ser bytes ou str (direta ou indiretamente por meio da interface *PathLike*). Se for str, ele é codificado com a codificação do sistema de arquivos.

Esta função tem suporte a *especificar um descritor de arquivo* e *não seguir links simbólicos*.

Levanta um *evento de auditoria* `os.getxattr` com os argumentos `path` e `attribute`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *path* e *attribute*.

os.**listxattr**(*path=None*, ***, *follow_symlinks=True*)

Retorna uma lista dos atributos estendidos do sistema de arquivos em *path*. Os atributos na lista são representados como strings decodificadas com a codificação do sistema de arquivos. Se *path* for `None`, *listxattr()* irá examinar o diretório atual.

Esta função tem suporte a *especificar um descritor de arquivo* e *não seguir links simbólicos*.

Levanta um *evento de auditoria* `os.listxattr` com o argumento `path`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

os.**removexattr**(*path*, *attribute*, ***, *follow_symlinks=True*)

Remove o atributo *attribute* do sistema de arquivos estendido do *path*. *attribute* deve ser bytes ou str (direta ou indiretamente através da interface *PathLike*). Se for uma string, é codificada com o *tratador de erros e codificação do sistema de arquivos*

Esta função tem suporte a *especificar um descritor de arquivo* e *não seguir links simbólicos*.

Levanta um *evento de auditoria* `os.removexattr` com os argumentos `path` e `attribute`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *path* e *attribute*.

os.**setxattr**(*path*, *attribute*, *value*, *flags=0*, ***, *follow_symlinks=True*)

Define o atributo de sistema de arquivos estendido *attribute* em *path* como *value*. *attribute* deve ser um bytes ou str sem NULs embutidos (direta ou indiretamente por meio da interface *PathLike*). Se for um str, ele é codificado com *tratador de erros e codificação do sistema de arquivos*. *flags* podem ser *XATTR_REPLACE* ou *XATTR_CREATE*. Se *XATTR_REPLACE* for fornecido e o atributo não existe, `ENODATA` será levantada. Se *XATTR_CREATE* for fornecido e o atributo já existir, o atributo não será criado e `EEXISTS` será levantada.

Esta função tem suporte a *especificar um descritor de arquivo* e *não seguir links simbólicos*.

> **Nota**
>
> Um bug nas versões do kernel Linux inferiores a 2.6.39 fez com que o argumento *flags* fosse ignorado em alguns sistemas de arquivos.

Levanta um *evento de auditoria* `os.setxattr` com os argumentos `path`, `attribute`, `value` e `flags`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar* para *path* e *attribute*.

os.**XATTR_SIZE_MAX**

O tamanho máximo que o valor de um atributo estendido pode ter. Atualmente, são 64 KiB no Linux.

os.**XATTR_CREATE**

Este é um valor possível para o argumento *flags* em *setxattr()*. Indica que a operação deve criar um atributo.

os.**XATTR_REPLACE**

Este é um valor possível para o argumento *flags* em *setxattr()*. Indica que a operação deve substituir um atributo existente.

16.1.7 Gerenciamento de processo

Estas funções podem ser usadas para criar e gerenciar processos.

As várias funções *exec** recebem uma lista de argumentos para o novo programa carregado no processo. Em cada caso, o primeiro desses argumentos é passado para o novo programa como seu próprio nome, e não como um argumento que um usuário pode ter digitado em uma linha de comando. Para o programador C, este é o `argv[0]` passado para um programa `main()`. Por exemplo, `os.execv('/bin/echo', ['foo', 'bar'])` exibirá apenas `bar` na saída padrão; `foo` parecerá ser ignorado.

os.**abort**()

Gera um sinal `SIGABRT` para o processo atual. No Unix, o comportamento padrão é produzir um despejo de memória; no Windows, o processo retorna imediatamente um código de saída `3`. Esteja ciente de que chamar esta função não chamará o manipulador de sinal Python registrado para `SIGABRT` com *signal.signal()*.

os.**add_dll_directory**(*path*)

Adiciona um caminho ao caminho de pesquisa de DLL.

Este caminho de pesquisa é usado ao resolver dependências para módulos de extensão importados (o próprio módulo é resolvido por meio de *sys.path*), e também por *ctypes*.

Remove o diretório chamando **close()** no objeto retornado ou usando-o em uma instrução `with`.

Consulte a documentação da Microsoft para obter mais informações sobre como as DLLs são carregadas.

Levanta um *evento de auditoria* `os.add_dll_directory` com o argumento `path`.

Disponibilidade: Windows.

Adicionado na versão 3.8: As versões anteriores do CPython resolveriam DLLs usando o comportamento padrão para o processo atual. Isso levou a inconsistências, como apenas às vezes pesquisar `PATH` ou o diretório de trabalho atual, e funções do sistema operacional como `AddDllDirectory` sem efeito.

No 3.8, as duas maneiras principais de carregar as DLLs agora substituem explicitamente o comportamento de todo o processo para garantir a consistência. Veja as notas de portabilidade para informações sobre atualização de bibliotecas.

os.**execl**(*path*, *arg0*, *arg1*, ...)

os.**execle**(*path*, *arg0*, *arg1*, ..., *env*)

os.**execlp**(*file*, *arg0*, *arg1*, ...)

os.**execlpe**(*file*, *arg0*, *arg1*, ..., *env*)

os.**execv**(*path*, *args*)

os.**execve**(*path*, *args*, *env*)

os.**execvp**(*file*, *args*)

os.**execvpe**(*file*, *args*, *env*)

Todas essas funções executam um novo programa, substituindo o processo atual; elas não retornam. No Unix, o novo executável é carregado no processo atual e terá a mesma identificação de processo do chamador. Os erros serão relatados como exceções de *OSError*.

O processo atual é substituído imediatamente. Objetos arquivo abertos e descritores não são descarregados, então se houver dados em buffer nesses arquivos abertos, você deve descarregá-los usando `sys.stdout.flush()` ou *os.fsync()* antes de chamar uma função *exec**.

As variantes "l" e "v" das funções *exec** diferem em como os argumentos da linha de comando são passados. As variantes "l" são talvez as mais fáceis de trabalhar se o número de parâmetros for fixo quando o código for escrito; os parâmetros individuais simplesmente se tornam parâmetros adicionais para as funções `execl*()`. As variantes "v" são boas quando o número de parâmetros é variável, com os argumentos sendo passados em

uma lista ou tupla como o parâmetro *args*. Em qualquer caso, os argumentos para o processo filho devem começar com o nome do comando que está sendo executado, mas isso não é obrigatório.

As variantes que incluem um "p" próximo ao final (`execlp()`, `execlpe()`, `execvp()` e `execvpe()`) usarão a variável de ambiente `PATH` para localizar o programa *file*. Quando o ambiente está sendo substituído (usando uma das variantes `exec*e`, discutidas no próximo parágrafo), o novo ambiente é usado como fonte da variável `PATH`. As outras variantes, `execl()`, `execle()`, `execv()` e `execve()`, não usarão a variável `PATH` para localizar o executável; *path* deve conter um caminho absoluto ou relativo apropriado. Caminhos relativos devem incluir pelo menos uma barra, mesmo no Windows, pois nomes simples não serão resolvidos.

Para `execle()`, `execlpe()`, `execve()` e `execvpe()` (observe que todos eles terminam em "e"), o parâmetro *env* deve ser um mapeamento que é usado para definir as variáveis de ambiente para o novo processo (elas são usadas no lugar do ambiente do processo atual); as funções `execl()`, `execlp()`, `execv()` e `execvp()` fazem com que o novo processo herde o ambiente do processo atual.

Para `execve()` em algumas plataformas, *path* também pode ser especificado como um descritor de arquivo aberto. Esta funcionalidade pode não ser compatível com sua plataforma; você pode verificar se está ou não disponível usando `os.supports_fd`. Se não estiver disponível, usá-lo vai levantar uma `NotImplementedError`.

Levanta um *evento de auditoria* `os.exec` com os argumentos `path`, `args` e `env`.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto para `execve()`.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`os._exit(n)`

Sai do processo com status *n*, sem chamar manipuladores de limpeza, liberando buffers de entrada e saída padrões etc.

> **Nota**
>
> A forma padrão de sair é `sys.exit(n)`. `_exit()` normalmente só deve ser usado no processo filho após uma função `fork()`.

Os seguintes códigos de saída são definidos e podem ser usados com `_exit()`, embora não sejam obrigatórios. Eles são normalmente usados para programas de sistema escritos em Python, como um programa de entrega de comando externo de servidor de e-mail.

> **Nota**
>
> Alguns deles podem não estar disponíveis em todas as plataformas Unix, pois há algumas variações. Essas constantes estão definidas onde elas são definidas pela plataforma subjacente.

`os.EX_OK`

Código de saída que significa que não ocorreu nenhum erro. Pode ser obtido do valor definido de `EXIT_SUCCESS` em algumas plataformas. Geralmente tem um valor de zero.

Disponibilidade: Unix, Windows.

`os.EX_USAGE`

Código de saída que significa que o comando foi usado incorretamente, como quando o número errado de argumentos é fornecido.

Disponibilidade: Unix, not WASI.

os.**EX_DATAERR**

Código de saída que significa que os dados inseridos estavam incorretos.

Disponibilidade: Unix, not WASI.

os.**EX_NOINPUT**

Código de saída que significa que um arquivo de entrada não existe ou não pôde ser lido.

Disponibilidade: Unix, not WASI.

os.**EX_NOUSER**

Código de saída que significa que um usuário especificado não existe.

Disponibilidade: Unix, not WASI.

os.**EX_NOHOST**

Código de saída que significa que um host especificado não existe.

Disponibilidade: Unix, not WASI.

os.**EX_UNAVAILABLE**

Código de saída que significa que um serviço necessário está indisponível.

Disponibilidade: Unix, not WASI.

os.**EX_SOFTWARE**

Código de saída que significa que um erro interno do software foi detectado.

Disponibilidade: Unix, not WASI.

os.**EX_OSERR**

Código de saída que significa que um erro do sistema operacional foi detectado, como a incapacidade de criar um fork ou um encadeamento.

Disponibilidade: Unix, not WASI.

os.**EX_OSFILE**

Código de saída que significa que algum arquivo do sistema não existia, não pôde ser aberto ou teve algum outro tipo de erro.

Disponibilidade: Unix, not WASI.

os.**EX_CANTCREAT**

Código de saída que significa que um arquivo de saída especificado pelo usuário não pôde ser criado.

Disponibilidade: Unix, not WASI.

os.**EX_IOERR**

Código de saída que significa que ocorreu um erro ao fazer E/S em algum arquivo.

Disponibilidade: Unix, not WASI.

os.**EX_TEMPFAIL**

Código de saída que significa que ocorreu uma falha temporária. Isso indica algo que pode não ser realmente um erro, como uma conexão de rede que não pôde ser feita durante uma operação de nova tentativa.

Disponibilidade: Unix, not WASI.

os.**EX_PROTOCOL**

Código de saída que significa que uma troca de protocolo foi ilegal, inválida ou não compreendida.

Disponibilidade: Unix, not WASI.

os.**EX_NOPERM**

Código de saída que significa que não havia permissões suficientes para executar a operação (mas sem intenção de causar problemas do sistema de arquivos).

Disponibilidade: Unix, not WASI.

os.**EX_CONFIG**

Código de saída que significa que ocorreu algum tipo de erro de configuração.

Disponibilidade: Unix, not WASI.

os.**EX_NOTFOUND**

Código de saída que significa algo como "uma entrada não foi encontrada".

Disponibilidade: Unix, not WASI.

os.**fork**()

Cria um processo filho. Retorna `0` no filho e o ID de processo do filho no pai. Se ocorrer um erro, uma exceção *OSError* é levantada.

Note que algumas plataformas incluindo FreeBSD <= 6.3 e Cygwin têm problemas conhecidos ao usar `fork()` a partir de um thread.

Levanta um *evento de auditoria* `os.fork` sem argumentos.

Aviso

Se você usa soquetes TLS em um aplicação chamando `fork()`, consulte alerta na documentação do módulo *ssl*.

Aviso

No macOS, o uso desta função é inseguro quando combinado com o uso de APIs de sistema de nível mais alto, e isso inclui o uso de *urllib.request*.

Alterado na versão 3.8: Chamar `fork()` em um subinterpretador não é mais possível (*RuntimeError* é levantada).

Alterado na versão 3.12: Se o Python for capaz de detectar que o seu processar tem várias threads, o *os.fork()* agora levanta uma exceção *DeprecationWarning*.

Optamos por apresentar isso como um alerta, quando detectável, para informar melhor aos desenvolvedores sobre um problema de design que a plataforma POSIX indica especificamente como sem suporte. Mesmo em códigos que *aparentem* funcionar, nunca foi seguro misturar threading com *os.fork()* em plataformas POSIX. O próprio ambiente de execução do CPython sempre fez chamadas à API que não são seguras para uso no processo filho quando existiam threads no pai (como `malloc` e `free`).

Os usuários do macOS ou usuários de implementações de libc ou malloc diferentes daquelas normalmente encontradas no glibc até o momento estão entre os que já têm maior probabilidade de sofrer com impasses (*deadlocks*) ao executar esse código.

Consulte esta discussão sobre fork ser incompatível com threads para obter detalhes técnicos sobre o motivo pelo qual estamos revelando aos desenvolvedores esse problema de longa data de compatibilidade da plataforma.

Disponibilidade: POSIX, not WASI, not Android, not iOS.

os.**forkpty**()

Cria um processo filho, usando um novo pseudoterminal como o terminal de controle do filho. Retorna um par de `(pid, fd)`, onde *pid* é `0` no filho, o novo ID de processo do filho no pai e *fd* é o descritor de arquivo do lado mestre do pseudoterminal. Para uma abordagem mais portátil, use o módulo *pty*. Se ocorrer um erro, *OSError* é levantada.

Levanta um *evento de auditoria* `os.forkpty` sem argumentos.

> **Aviso**
>
> No macOS, o uso desta função é inseguro quando combinado com o uso de APIs de sistema de nível mais alto, e isso inclui o uso de `urllib.request`.

Alterado na versão 3.8: Chamar `forkpty()` em um subinterpretador não é mais permitido (`RuntimeError` é levantada).

Alterado na versão 3.12: Se o Python for capaz de detectar que o seu processo tem vários threads, isso agora levanta uma exceção `DeprecationWarning`. Veja a explicação mais longa em `os.fork()`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

`os.`**`kill`**(*pid*, *sig*, **/**)

Envia o sinal *sig* para o processo *pid*. Constantes dos sinais específicos disponíveis na plataforma host são definidas no módulo `signal`.

Windows: os sinais `signal.CTRL_C_EVENT` e `signal.CTRL_BREAK_EVENT` são sinais especiais que só podem ser enviados para processos de console que compartilham uma janela de console comum, por exemplo, alguns subprocessos. Qualquer outro valor para *sig* fará com que o processo seja encerrado incondicionalmente pela API TerminateProcess e o código de saída será definido como *sig*.

Veja também `signal.pthread_kill()`.

Levanta um *evento de auditoria* `os.kill` com os argumentos `pid` e `sig`.

Disponibilidade: Unix, Windows, not WASI, not iOS.

Alterado na versão 3.2: Adicionado suporte ao Windows.

`os.`**`killpg`**(*pgid*, *sig*, **/**)

Envia o sinal *sig* para o grupo de processos *pgid*.

Levanta um *evento de auditoria* `os.killpg` com os argumentos `pgid` e `sig`.

Disponibilidade: Unix, not WASI, not iOS.

`os.`**`nice`**(*increment*, **/**)

Adiciona *increment* ao nível de “nice” do processo. Retorna um novo nível de “nice”.

Disponibilidade: Unix, not WASI.

`os.`**`pidfd_open`**(*pid*, *flags=0*)

Retorna um descritor de arquivo referente ao processo *pid* com as *flags* definidas. Este descritor pode ser usado para gerenciar o processo sem corridas e sinais.

Veja a página man `pidfd_open(2)` para mais detalhes.

Disponibilidade: Linux >= 5.3, Android >= `build-time` API level 31

Adicionado na versão 3.9.

`os.`**`PIDFD_NONBLOCK`**

Este sinalizador indica que o descritor de arquivo será não-bloqueante. Se o processo referenciado pelo descritor de arquivo ainda não tiver terminado, então uma tentativa de aguardar no descritor de arquivo usando `waitid(2)` retornará imediatamente o erro `EAGAIN` em vez de bloquear.

Disponibilidade: Linux >= 5.10

Adicionado na versão 3.12.

`os.`**`plock`**(*op*, **/**)

Trava os segmentos do programa na memória. O valor de *op* (definido em `<sys/lock.h>`) determina quais segmentos são travados.

Disponibilidade: Unix, not WASI, not iOS.

os.**popen**(*cmd*, *mode='r'*, *buffering=-1*)

Abre um encadeamento, também conhecido como "pipe", de ou para o comando *cmd*. O valor de retorno é um objeto arquivo aberto conectado ao encadeamento, que pode ser lido ou escrito dependendo se *mode* é `'r'` (padrão) ou `'w'`. O argumento *buffering* tem o mesmo significado que o argumento correspondente para a função embutida *open()*. O objeto arquivo retornado lê ou escreve strings de texto em vez de bytes.

O método `close` retorna *None* se o subprocesso foi encerrado com sucesso, ou o código de retorno do subprocesso se houve um erro. Em sistemas POSIX, se o código de retorno for positivo, ele representa o valor de retorno do processo deslocado para a esquerda em um byte. Se o código de retorno for negativo, o processo foi encerrado pelo sinal dado pelo valor negado do código de retorno. (Por exemplo, o valor de retorno pode ser `- signal.SIGKILL` se o subprocesso foi eliminado.) Em sistemas Windows, o valor de retorno contém o código de retorno inteiro com sinal do processo filho.

No Unix, *waitstatus_to_exitcode()* pode ser usado para converter o resultado do método `close` (status de saída) em um código de saída se não for `None`. No Windows, o resultado do método `close` é diretamente o código de saída (ou `None`).

Isso é implementado usando *subprocess.Popen*; consulte a documentação desta classe para maneiras mais poderosas de gerenciar e se comunicar com subprocessos.

Disponibilidade: not WASI, not Android, not iOS.

Nota

O *Modo UTF-8 do Python* afeta as codificações usadas para *cmd* e conteúdo de encadeamento.

popen() é um wrapper simples em torno de *subprocess.Popen*. Use *subprocess.Popen* ou *subprocess.run()* para controlar opções como codificações.

Descontinuado desde a versão 3.14: A função está *suavemente descontinuada* e não deve mais ser usada para escrever novos códigos. O módulo *subprocess* é recomendado em vez disso.

os.**posix_spawn**(*path*, *argv*, *env*, ***, *file_actions=None*, *setpgroup=None*, *resetids=False*, *setsid=False*, *setsigmask=()*, *setsigdef=()*, *scheduler=None*)

Envolve a API da biblioteca C `posix_spawn()` para uso em Python.

A maioria dos usuários deveria usar *subprocess.run()* em vez de *posix_spawn()*.

Os argumentos somente-posicional *path*, *args*, e *env* são similares a *execve()*. *env* pode ser `None`, caso em que o ambiente do processo atual é usado.

O parâmetro *path* é o caminho para o arquivo executável. O *path* deve conter um diretório. Use *posix_spawnp()* para passar um arquivo executável sem diretório.

O argumento *file_actions* pode ser uma sequência de tuplas descrevendo ações a serem tomadas em descritores de arquivo específicos no processo filho entre as etapas `fork()` e `exec()` de implementação da biblioteca C. O primeiro item em cada tupla deve ser um dos três indicadores de tipo listados abaixo, descrevendo os elementos restantes da tupla:

os.**POSIX_SPAWN_OPEN**

(`os.POSIX_SPAWN_OPEN`, *fd*, *path*, *flags*, *mode*)

Efetua `os.dup2(os.open(path, flags, mode), fd)`.

os.**POSIX_SPAWN_CLOSE**

(`os.POSIX_SPAWN_CLOSE`, *fd*)

Efetua `os.close(fd)`.

os.**POSIX_SPAWN_DUP2**

(`os.POSIX_SPAWN_DUP2`, *fd*, *new_fd*)

Efetua `os.dup2(fd, new_fd)`.

os.**POSIX_SPAWN_CLOSEFROM**

(os.POSIX_SPAWN_CLOSEFROM, *fd*)

Efetua os.closerange(fd, INF).

Estas tuplas correspondem a chamadas de API da biblioteca C de posix_spawn_file_actions_addopen(), posix_spawn_file_actions_addclose(), posix_spawn_file_actions_adddup2() e posix_spawn_file_actions_addclosefrom_np() usadas para preparar para a chamada de posix_spawn() em si.

O argumento *setpgroup* definirá o grupo de processos do filho para o valor especificado. Se o valor especificado for 0, o ID do grupo de processo filho será igual ao ID do processo. Se o valor de *setpgroup* não for definido, o filho herdará o ID do grupo de processos do pai. Este argumento corresponde ao sinalizador POSIX_SPAWN_SETPGROUP da biblioteca C.

Se o argumento *resetids* for True, ele irá reconfigurar o UID e GID efetivos do filho para o UID e GID reais do processo pai. Se o argumento for False, então o filho retém o UID e GID efetivos do pai. Em ambos os casos, se os bits de permissão set-user-ID e set-group-ID estiverem habilitados no arquivo executável, seu efeito vai substituir a configuração do UID e GID efetivos. Este argumento corresponde ao sinalizador POSIX_SPAWN_RESETIDS da biblioteca C.

Se o argumento *setsid* for True, ele criará um novo ID de sessão para posix_spawn. *setsid* requer POSIX_SPAWN_SETSID ou POSIX_SPAWN_SETSID_NP. Caso contrário, *NotImplementedError* é levantada.

O argumento *setsigmask* definirá a máscara de sinal para o conjunto de sinais especificado. Se o parâmetro não for usado, o filho herda a máscara de sinal do pai. Este argumento corresponde ao sinalizador POSIX_SPAWN_SETSIGMASK da biblioteca C.

O argumento *sigdef* redefinirá a disposição de todos os sinais no conjunto especificado. Este argumento corresponde ao sinalizador POSIX_SPAWN_SETSIGDEF da biblioteca C.

O argumento *scheduler* deve ser uma tupla contendo a política do agendador (opcional) e uma instância de *sched_param* com os parâmetros do agendador. Um valor None no lugar da política do agendador indica que a política não está sendo fornecido. Este argumento é uma combinação dos sinalizadores POSIX_SPAWN_SETSCHEDPARAM e POSIX_SPAWN_SETSCHEDULER da biblioteca C.

Levanta um *evento de auditoria* os.posix_spawn com os argumentos path, argv e env.

Adicionado na versão 3.8.

Alterado na versão 3.13: o parâmetro *env* pode ser None. os.POSIX_SPAWN_CLOSEFROM está disponível nas plataformas onde posix_spawn_file_actions_addclosefrom_np() existe.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**posix_spawnp**(*path*, *argv*, *env*, ***, *file_actions=None*, *setpgroup=None*, *resetids=False*, *setsid=False*, *setsigmask=()*, *setsigdef=()*, *scheduler=None*)

Envolve a API da biblioteca C posix_spawnp() para uso em Python.

Semelhante a *posix_spawn()* exceto que o sistema procura o arquivo *executable* na lista de diretórios especificados pela variável de ambiente PATH (da mesma forma que para execvp(3)).

Levanta um *evento de auditoria* os.posix_spawn com os argumentos path, argv e env.

Adicionado na versão 3.8.

Disponibilidade: POSIX, not WASI, not Android, not iOS.

Veja a documentação de *posix_spawn()*.

os.**register_at_fork**(***, *before=None*, *after_in_parent=None*, *after_in_child=None*)

Registra chamáveis a serem executados quando um novo processo filho é criado usando *os.fork()* ou APIs de clonagem de processos semelhantes. Os parâmetros são opcionais e somente-nomeados. Cada um especifica um ponto de chamada diferente.

- *before* é uma função chamada antes de criar um processo filho.

- *after_in_parent* é uma função chamada a partir do processo pai após criar um processo filho.
- *after_in_child* é uma função chamada a partir do processo filho.

Essas chamadas são feitas apenas se o controle deve retornar ao interpretador Python. Um lançamento típico de *subprocess* não irá acioná-los, pois o filho não entrará novamente no interpretador.

As funções registradas para execução antes de criar um processo filho são chamadas na ordem de registro reversa. As funções registradas para execução após o processo filho ser criado (no pai ou no filho) são chamadas na ordem de registro.

Note que chamadas a `fork()` feitas por código C de terceiros podem não chamar estas funções, a menos que ele explicitamente chamem `PyOS_BeforeFork()`, `PyOS_AfterFork_Parent()` e `PyOS_AfterFork_Child()`.

Não há uma forma de desfazer o registro de uma função.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.7.

os.**spawnl**(*mode*, *path*, *...*)

os.**spawnle**(*mode*, *path*, *...*, *env*)

os.**spawnlp**(*mode*, *file*, *...*)

os.**spawnlpe**(*mode*, *file*, *...*, *env*)

os.**spawnv**(*mode*, *path*, *args*)

os.**spawnve**(*mode*, *path*, *args*, *env*)

os.**spawnvp**(*mode*, *file*, *args*)

os.**spawnvpe**(*mode*, *file*, *args*, *env*)

Executa o programa *path* em um novo processo.

(Observe que o módulo *subprocess* fornece recursos mais poderosos para gerar novos processos e recuperar seus resultados; usar esse módulo é preferível a usar essas funções. Verifique especialmente a seção *Replacing Older Functions with the subprocess Module*.)

Se *mode* for *P_NOWAIT*, esta função retorna o id do processo do novo processo; se *mode* for *P_WAIT*, retorna o código de saída do processo se ele sair normalmente, ou `-signal`, onde *signal* é o sinal que matou o processo. No Windows, o id do processo será na verdade o manipulador do processo, portanto, pode ser usado com a função *waitpid()*.

Nota sobre VxWorks: esta função não retorna `-signal` se o novo processo é interrompido. Em vez disso, ele levanta a exceção OSError.

As variantes "l" e "v" das funções *spawn** diferem em como os argumentos de linha de comando são passados. As variantes "l" são talvez as mais fáceis de trabalhar se o número de parâmetros for fixo quando o código for escrito; os parâmetros individuais simplesmente se tornam parâmetros adicionais para as funções `spawnl*()`. As variantes "v" são boas quando o número de parâmetros é variável, com os argumentos sendo passados em uma lista ou tupla como o parâmetro *args*. Em ambos os casos, os argumentos para o processo filho devem começar com o nome do comando que está sendo executado.

As variantes que incluem um segundo "p" próximo ao final (*spawnlp()*, *spawnlpe()*, *spawnvp()* e *spawnvpe()*) usarão a variável de ambiente `PATH` para localizar o programa *file*. Quando o ambiente está sendo substituído (usando uma das variantes *spawn*e*, discutidas no próximo parágrafo), o novo ambiente é usado como fonte da variável `PATH`. As outras variantes, *spawnl()*, *spawnle()*, *spawnv()* e *spawnve()*, não usarão a variável `PATH` para localizar o executável; *path* deve conter um caminho absoluto ou relativo apropriado.

Para *spawnle()*, *spawnlpe()*, *spawnve()* e *spawnvpe()* (observe que todos eles terminam em "e"), o parâmetro *env* deve ser um mapeamento que é usado para definir as variáveis de ambiente para o novo processo (elas são usadas no lugar do ambiente do processo atual); as funções *spawnl()*, *spawnlp()*, *spawnv()* e *spawnvp()* fazem com que o novo processo herde o ambiente do processo atual. Note que as chaves e os valores no dicionário *env* devem ser strings; chaves ou valores inválidos farão com que a função falhe, com um valor de retorno de `127`.

Como exemplo, as seguintes chamadas a *spawnlp()* e *spawnvpe()* são equivalentes:

```
import os
os.spawnlp(os.P_WAIT, 'cp', 'cp', 'index.html', '/dev/null')

L = ['cp', 'index.html', '/dev/null']
os.spawnvpe(os.P_WAIT, 'cp', L, os.environ)
```

Levanta um *evento de auditoria* `os.spawn` com os argumentos `mode`, `path`, `args` e `env`.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

spawnlp(), *spawnlpe()*, *spawnvp()* e *spawnvpe()* não estão disponíveis no Windows. *spawnle()* e *spawnve()* não são seguros para thread no Windows; recomendamos que você use o módulo *subprocess*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Descontinuado desde a versão 3.14: Estas funções estão *suavemente descontinuadas* e não devem mais ser usadas para escrever novos códigos. O módulo *subprocess* é recomendado em vez disso.

`os.`**`P_NOWAIT`**

`os.`**`P_NOWAITO`**

Valores possíveis para o parâmetro *mode* para a família de funções *spawn**. Se qualquer um desses valores for fornecido, as funções *spawn** retornarão assim que o novo processo for criado, com o id do processo como o valor de retorno.

Disponibilidade: Unix, Windows.

`os.`**`P_WAIT`**

Valor possível para o parâmetro *mode* para a família de funções *spawn**. Se for fornecido como *mode*, as funções *spawn** não retornarão até que o novo processo seja executado até a conclusão e retornará o código de saída do processo em que a execução foi bem-sucedida, ou `-signal` se um sinal interromper o processo.

Disponibilidade: Unix, Windows.

`os.`**`P_DETACH`**

`os.`**`P_OVERLAY`**

Valores possíveis para o parâmetro *mode* para a família de funções *spawn**. Eles são menos portáveis do que os listados acima. *P_DETACH* é semelhante a *P_NOWAIT*, mas o novo processo é separado do console do processo de chamada. Se *P_OVERLAY* for usado, o processo atual será substituído; a função *spawn** não retornará.

Disponibilidade: Windows.

`os.`**`startfile`**(*path*[, *operation*][, *arguments*][, *cwd*][, *show_cmd*])

Inicia um arquivo com sua aplicação associada.

Quando *operation* não é especificado, isso atua como um clique duplo no arquivo no Windows Explorer, ou como fornecer o nome do arquivo como um argumento para o comando **`start`** do console interativo de comandos: o arquivo é aberto com qualquer aplicação (se houver) com a extensão associada.

Quando outra *operation* é fornecida, ela deve ser um "verbo de comando" que especifica o que deve ser feito com o arquivo. Verbos comuns documentados pela Microsoft são `'open'`, `'print'` e `'edit'` (para serem usados em arquivos), bem como `'explore'` e `'find'` (para serem usados em diretórios).

Ao iniciar uma aplicação, especifique *arguments* a serem passados como uma única string. Este argumento pode não ter efeito quando esta função é usada para iniciar um documento.

O diretório de trabalho padrão é herdado, mas pode ser substituído pelo argumento *cwd*. Este deve ser um caminho absoluto. Um *path* relativo será resolvido levando em consideração este argumento.

Use *show_cmd* para substituir o estilo de janela padrão. Se isso terá algum efeito irá depender da aplicação sendo iniciada. Os valores são inteiros conforme implementado pela função Win32 `ShellExecute()`.

startfile() retorna assim que a aplicação associada é iniciada. Não há opção de aguardar o fechamento da aplicação, e nenhuma maneira de recuperar o status de saída da aplicação. O parâmetro *path* é relativo ao

diretório atual ou ao *cwd*. Se você quiser usar um caminho absoluto, certifique-se que o primeiro caractere não seja uma barra (`'/'`). Use *pathlib* ou a função *os.path.normpath()* para garantir que os caminhos sejam codificados corretamente para Win32.

Para reduzir a sobrecarga de inicialização do interpretador, a função Win32 `ShellExecute()` não é resolvida até que esta função seja chamada pela primeira vez. Se a função não puder ser resolvida, uma exceção *NotImplementedError* será levantada.

Levanta um *evento de auditoria* `os.startfile` com os argumentos `path` e `operation`.

Levanta um *evento de auditoria* `os.startfile/2` com os argumentos `path`, `operation`, `arguments`, `cwd` e `show_cmd`.

Disponibilidade: Windows.

Alterado na versão 3.10: Adicionados os argumentos *arguments*, *cwd* e *show_cmd*, e o evento de auditoria `os.startfile/2`.

`os.`**`system`**`(`*command*`)`

Executa o comando (uma string) em um subshell. Isso é implementado chamando a função C padrão `system()`, e tem as mesmas limitações. Alterações em *sys.stdin* etc. não são refletidas no ambiente do comando executado. Se *command* gerar qualquer saída, ela será enviada ao fluxo de saída padrão do interpretador. O padrão C não especifica o significado do valor de retorno da função C, então o valor de retorno da função Python depende do sistema.

No Unix, o valor de retorno é o status de saída do processo codificado no formato especificado para *wait()*.

No Windows, o valor de retorno é aquele retornado pelo shell do sistema após a execução de *command*. O shell é fornecido pela variável de ambiente Windows `COMSPEC`: normalmente é **`cmd.exe`**, que retorna o status de saída da execução do comando; em sistemas que usam um shell não nativo, consulte a documentação do shell.

O módulo *subprocess* fornece recursos mais poderosos para gerar novos processos e recuperar seus resultados; usar esse módulo é recomendado a usar esta função. Veja a seção *Replacing Older Functions with the subprocess Module* na documentação do *subprocess* para algumas receitas úteis.

No Unix, *waitstatus_to_exitcode()* pode ser usada para converter o resultado (status de saída) em um código de saída. No Windows, o resultado é diretamente o código de saída.

Levanta um *evento de auditoria* `os.system` com o argumento `command`.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

`os.`**`times`**`()`

Retorna os tempos do processo global atual. O valor de retorno é um objeto com cinco atributos:

- `user` - tempo do usuário
- `system` - tempo do sistema
- `children_user` - tempo do usuário de todos os processos filhos
- `children_system` - tempo do sistema de todos os processos filhos
- `elapsed` - tempo real decorrido desde um ponto fixo no passado

Para compatibilidade com versões anteriores, esse objeto também se comporta como uma tupla de 5 elementos contendo `user`, `system`, `children_user`, `children_system` e `elapsed` nessa ordem.

Consulte as páginas de manual *times(2)* e times(3) no Unix ou o GetProcessTimes MSDN no Windows. No Windows, apenas `user` e `system` são conhecidos; os outros atributos são zero.

Disponibilidade: Unix, Windows.

Alterado na versão 3.3: Tipo de retorno foi alterado de uma tupla para um objeto tupla ou similar com atributos nomeados.

os.**wait**()

Aguarda a conclusão de um processo filho e retorna uma tupla contendo seu pid e indicação de status de saída: um número de 16 bits, cujo byte baixo é o número do sinal que interrompeu o processo e cujo byte alto é o status de saída (se o número do sinal é zero); o bit alto do byte baixo é definido se um arquivo principal foi produzido.

Se não houver filhos que possam ser aguardados, uma exceção *ChildProcessError* é levantada.

waitstatus_to_exitcode() pode ser usado para converter o status de saída em um código de saída.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Ver também

As outras funções `wait*()` documentadas abaixo podem ser usadas para aguardar a conclusão de um processo filho específico e ter mais opções. *waitpid()* é o único também disponível no Windows.

os.**waitid**(*idtype*, *id*, *options*, /)

Aguarda a conclusão de um processo filho.

idtype pode ser *P_PID*, *P_PGID*, *P_ALL* ou (no Linux) *P_PIDFD*. A interpretação de *id* depende disso; veja suas descrições individuais.

options é uma combinação OU de sinalizadores. Pelo menos um entre *WEXITED*, *WSTOPPED* ou *WCONTINUED* é obrigatório; *WNOHANG* e *WNOWAIT* são sinalizadores opcionais adicionais.

O valor de retorno é um objeto que representa os dados contidos na estrutura `siginfo_t` com os seguintes atributos:

- `si_pid` (ID do processo)
- `si_uid` (ID de usuário real do filho)
- `si_signo` (sempre *SIGCHLD*)
- `si_status` (o status de saída ou número do sinal, dependendo de `si_code`)
- `si_code` (veja *CLD_EXITED* para possíveis valores)

Se *WNOHANG* for especificado e não houver filhos correspondentes no estado solicitado, `None` será retornado. Caso contrário, se não houver filhos correspondentes que possam ser esperados, *ChildProcessError* será levantada.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.3.

Alterado na versão 3.13: Esta função agora também está disponível no MacOS.

os.**waitpid**(*pid*, *options*, /)

Os detalhes desta função diferem no Unix e no Windows.

No Unix: aguarda a conclusão de um processo filho dado pelo id de processo *pid* e retorna uma tupla contendo seu id de processo e indicação de status de saída (codificado como para *wait()*). A semântica da chamada é afetada pelo valor do inteiro *options*, que deve ser `0` para operação normal.

Se *pid* for maior que `0`, *waitpid()* solicita informações de status para aquele processo específico. Se *pid* for `0`, a solicitação é para o status de qualquer filho no grupo de processos do processo atual. Se *pid* for `-1`, a solicitação pertence a qualquer filho do processo atual. Se *pid* for menor que `-1`, o status é solicitado para qualquer processo no grupo de processos `-pid` (o valor absoluto de *pid*).

options é uma combinação OU de sinalizadores. Se contiver *WNOHANG* e não houver filhos correspondentes no estado solicitado, `(0, 0)` será retornado. Caso contrário, se não houver filhos correspondentes que possam ser esperados, *ChildProcessError* será levantada. Outras opções que podem ser usadas são *WUNTRACED* e *WCONTINUED*.

No Windows: aguarda a conclusão de um processo fornecido pelo identificador de processo *pid* e retorna uma tupla contendo *pid*, e seu status de saída deslocado 8 bits para a esquerda (o deslocamento torna o uso da função em várias plataformas mais fácil). Um *pid* menor ou igual a `0` não tem nenhum significado especial no Windows e levanta uma exceção. O valor de inteiros *options* não tem efeito. *pid* pode se referir a qualquer processo cujo id é conhecido, não necessariamente um processo filho. As funções *spawn** chamadas com *P_NOWAIT* retornam manipuladores de processo adequados.

waitstatus_to_exitcode() pode ser usado para converter o status de saída em um código de saída.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

Alterado na versão 3.5: Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção *InterruptedError* (consulte **PEP 475** para entender a justificativa).

os.**wait3**(*options*)

Semelhante a *waitpid()*, exceto que nenhum argumento de id de processo é fornecido e uma tupla de 3 elementos contendo o id do processo filho, indicação de status de saída e informações de uso de recursos é retornada. Consulte *resource.getrusage()* para obter detalhes sobre as informações de uso de recursos. O argumento *option* é o mesmo fornecido para *waitpid()* e *wait4()*.

waitstatus_to_exitcode() pode ser usado para converter o status de saída em um código de saída.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**wait4**(*pid*, *options*)

Semelhante a *waitpid()*, exceto uma tupla de 3 elementos, contendo o id do processo filho, indicação de status de saída e informações de uso de recursos é retornada. Consulte *resource.getrusage()* para obter detalhes sobre as informações de uso de recursos. Os argumentos para *wait4()* são os mesmos que aqueles fornecidos a *waitpid()*.

waitstatus_to_exitcode() pode ser usado para converter o status de saída em um código de saída.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**P_PID**

os.**P_PGID**

os.**P_ALL**

os.**P_PIDFD**

Estes são os valores possíveis para *idtype* em *waitid()*. Eles afetam como *id* é interpretado:

- `P_PID` - espera pelo filho cujo PID é *id*.
- `P_PGID` - espera por qualquer filho cujo ID do grupo de progresso seja *id*.
- `P_ALL` - espera por qualquer filho; *id* é ignorado.
- `P_PIDFD` - espera pelo filho identificado pelo descritor de arquivo *id* (um descritor de arquivo de processo criado com *pidfd_open()*).

Disponibilidade: Unix, not WASI, not Android, not iOS.

> **Nota**
>
> `P_PIDFD` só está disponível no Linux >= 5.4.

Adicionado na versão 3.3.

Adicionado na versão 3.9: A constante `P_PIDFD`.

os.**WCONTINUED**

Este sinalizador *options* para *waitpid()*, *wait3()*, *wait4()* e *waitid()* faz com que os processos filhos sejam reportados se eles tiverem continuado de uma parada de controle de trabalho desde que foram reportados pela última vez.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**WEXITED**

Este sinalizador *options* para *waitid()* faz com que os processos filhos que terminaram sejam reportados.

As outras funções `wait*` sempre reportam filhos que terminaram, então esta opção não está disponível para eles.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.3.

os.**WSTOPPED**

Este sinalizador *options* para *waitid()* faz com que os processos filhos que foram parados pela entrega de um sinal sejam reportados.

Esta opção não está disponível para as outras funções `wait*`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.3.

os.**WUNTRACED**

Este sinalizador de *options* para *waitpid()*, *wait3()* e *wait4()* faz com que os processos filho também sejam reportados se eles foram interrompidos, mas seu estado atual não foi reportado desde que foram parados.

Esta opção não está disponível para *waitid()*.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**WNOHANG**

Este sinalizador *options* faz com que *waitpid()*, *wait3()*, *wait4()* e *waitid()* retornem imediatamente se nenhum status de processo filho estiver disponível imediatamente.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**WNOWAIT**

Este sinalizador *options* faz com que *waitid()* deixe o filho em um estado de espera, para que uma chamada `wait*()` posterior possa ser usada para recuperar as informações de status do filho novamente.

Esta opção não está disponível para as outras funções `wait*`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**CLD_EXITED**

os.**CLD_KILLED**

os.**CLD_DUMPED**

os.**CLD_TRAPPED**

os.**CLD_STOPPED**

os.**CLD_CONTINUED**

Estes são os valores possíveis para `si_code` no resultado retornado por *waitid()*.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.3.

Alterado na versão 3.9: Adicionados os valores *CLD_KILLED* e *CLD_STOPPED*.

os.**waitstatus_to_exitcode**(*status*)

Converte um status de espera em um código de saída.

No Unix:

- Se o processo saiu normalmente (se `WIFEXITED(status)` for verdadeiro), retorna o status de saída do processo (retorna `WEXITSTATUS(status)`): resultado maior ou igual a 0.

- Se o processo foi encerrado por um sinal (se `WIFSIGNALED(status)` for verdadeiro), retorna `-signum` onde *signum* é o número do sinal que causou o encerramento do processo (retorna `-WTERMSIG(status)`): resultado menor que 0.
- Do contrário, levanta uma exceção *ValueError*.

No Windows, retorna *status* deslocado para a direita em 8 bits.

No Unix, se o processo está sendo rastreado ou se *waitpid()* foi chamado com a opção *WUNTRACED*, o chamador deve primeiro verificar se `WIFSTOPPED(status)` é verdadeiro. Esta função não deve ser chamada se `WIFSTOPPED(status)` for verdadeiro.

Ver também

As funções *WIFEXITED()*, *WEXITSTATUS()*, *WIFSIGNALED()*, *WTERMSIG()*, *WIFSTOPPED()*, *WSTOPSIG()*.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

Adicionado na versão 3.9.

As funções a seguir recebem um código de status do processo conforme retornado por *system()*, *wait()* ou *waitpid()* como parâmetro. Eles podem ser usados para determinar a disposição de um processo.

`os.`**`WCOREDUMP`**(*status*, `/`)

Retorna `True` se um despejo de memória *(core dump)* foi gerado para o processo; caso contrário, retorna `False`.

Esta função deve ser empregada apenas se *WIFSIGNALED()* for verdadeira.

Disponibilidade: Unix, not WASI, not Android, not iOS.

`os.`**`WIFCONTINUED`**(*status*)

Retorna `True` se um filho interrompido foi retomado pela entrega de *SIGCONT* (se o processo foi continuado de uma parada de controle de trabalho); caso contrário, retorna `False`.

Veja a opção *WCONTINUED*.

Disponibilidade: Unix, not WASI, not Android, not iOS.

`os.`**`WIFSTOPPED`**(*status*)

Retorna `True` se o processo foi interrompido pela entrega de um sinal; caso contrário, retorna `False`.

WIFSTOPPED() só retorna `True` se a chamada de *waitpid()* foi feita usando a opção *WUNTRACED* ou quando o processo está sendo rastreado (veja *ptrace(2)*).

Disponibilidade: Unix, not WASI, not Android, not iOS.

`os.`**`WIFSIGNALED`**(*status*)

Retorna `True` se o processo foi encerrado por um sinal; caso contrário, retorna `False`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

`os.`**`WIFEXITED`**(*status*)

Retorna `True` se o processo foi encerrado normalmente, isto é, chamando `exit()` ou `_exit()`, ou retornando de `main()`; caso contrário, retorna `False`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

`os.`**`WEXITSTATUS`**(*status*)

Retorna o status de saída do processo.

Esta função deve ser empregada apenas se *WIFEXITED()* for verdadeira.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**WSTOPSIG**(*status*)

Retorna o sinal que causou a interrupção do processo.

Esta função deve ser empregada apenas se *WIFSTOPPED()* for verdadeira.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.**WTERMSIG**(*status*)

Retorna o número do sinal que causou o encerramento do processo.

Esta função deve ser empregada apenas se *WIFSIGNALED()* for verdadeira.

Disponibilidade: Unix, not WASI, not Android, not iOS.

16.1.8 Interface do agendador

Essas funções controlam como o tempo de CPU de um processo é alocado pelo sistema operacional. Eles estão disponíveis apenas em algumas plataformas Unix. Para informações mais detalhadas, consulte suas páginas man do Unix.

Adicionado na versão 3.3.

As políticas de agendamento a seguir serão expostas se houver suporte pelo sistema operacional.

os.**SCHED_OTHER**

A política de agendamento padrão.

os.**SCHED_BATCH**

Política de agendamento para processos com uso intensivo de CPU que tenta preservar a interatividade no resto do computador.

os.**SCHED_DEADLINE**

Política de agendamento para tarefas com restrições de prazo.

Adicionado na versão 3.14.

os.**SCHED_IDLE**

Política de agendamento para tarefas em segundo plano de prioridade extremamente baixa.

os.**SCHED_NORMAL**

Apelido para *SCHED_OTHER*.

Adicionado na versão 3.14.

os.**SCHED_SPORADIC**

Política de agendamento para programas de servidor esporádicos.

os.**SCHED_FIFO**

Uma política de agendamento Primeiro a Entrar, Primeiro a Sair (First In First Out - FIFO).

os.**SCHED_RR**

Uma política de agendamento round-robin.

os.**SCHED_RESET_ON_FORK**

Este sinalizador pode ser operado em OU com qualquer outra política de agendamento. Quando um processo com este sinalizador definido bifurca, a política de agendamento e a prioridade de seu filho são redefinidas para o padrão.

class os.**sched_param**(*sched_priority*)

Esta classe representa parâmetros de agendamento ajustáveis usados em *sched_setparam()*, *sched_setscheduler()* e *sched_getparam()*. É imutável.

Neste momento, há somente um único parâmetro possível:

sched_priority

A prioridade de agendamento para uma política de agendamento.

os.**sched_get_priority_min**(*policy*)

Obtém o valor mínimo de prioridade para *policy*. *policy* é uma das constantes de política de agendamento acima.

os.**sched_get_priority_max**(*policy*)

Obtém o valor máximo de prioridade para *policy*. *policy* é uma das constantes de política de agendamento acima.

os.**sched_setscheduler**(*pid*, *policy*, *param*, /)

Define a política de agendamento para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada. *policy* é uma das constantes de política de agendamento acima. *param* é uma instância de *sched_param*.

os.**sched_getscheduler**(*pid*, /)

Retorna a política de agendamento para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada. O resultado é uma das constantes de política de agendamento acima.

os.**sched_setparam**(*pid*, *param*, /)

Define os parâmetros de agendamento para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada. *param* é uma instância de *sched_param*.

os.**sched_getparam**(*pid*, /)

Retorna os parâmetros de agendamento como uma instância de *sched_param* para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada.

os.**sched_rr_get_interval**(*pid*, /)

Retorna o quantum round-robin em segundos para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada.

os.**sched_yield**()

Abandona voluntariamente a CPU. Veja *sched_yield(2)* para detalhes.

os.**sched_setaffinity**(*pid*, *mask*, /)

Restringe o processo com PID *pid* (ou o processo atual se zero) para um conjunto de CPUs. *mask* é um iterável de inteiros que representam o conjunto de CPUs às quais o processo deve ser restrito.

os.**sched_getaffinity**(*pid*, /)

Retorna o conjunto de CPUs ao qual o processo com PID *pid* está restrito.

Se *pid* for zero, retorna o conjunto de CPUs às quais a thread chamadora do processo atual está restrita.

Veja também a função *process_cpu_count()*.

16.1.9 Diversas informações de sistema

os.**confstr**(*name*, /)

Retorna valores de configuração do sistema com valor de string. *name* especifica o valor de configuração a ser recuperado; pode ser uma string que é o nome de um valor de sistema definido; esses nomes são especificados em vários padrões (POSIX, Unix 95, Unix 98 e outros). Algumas plataformas também definem nomes adicionais. Os nomes conhecidos pelo sistema operacional são fornecidos como as chaves do dicionário `confstr_names`. Para variáveis de configuração não incluídas nesse mapeamento, passar um número inteiro para *name* também é aceito.

Se o valor de configuração especificado por *name* não for definido, retorna `None`.

Se *name* for uma string e não for conhecida, *ValueError* é levantada. Se um valor específico para *name* não for compatível com o sistema operacional do computador, mesmo que seja incluído em `confstr_names`, uma *OSError* é levantada com *errno.EINVAL* como número do erro.

Disponibilidade: Unix.

os.**confstr_names**

Nomes de mapeamento de dicionário aceitos por *confstr()* para os valores inteiros definidos para esses nomes pelo sistema operacional do computador. Isso pode ser usado para determinar o conjunto de nomes conhecidos pelo sistema.

Disponibilidade: Unix.

os.**cpu_count**()

Retorna o número de CPUs lógicas do **sistema**. Retorna None se não determinado.

A função *process_cpu_count()* pode ser usada para obter o número de CPUs lógicas utilizáveis pela thread de chamada do **processo atual**.

Adicionado na versão 3.4.

Alterado na versão 3.13: Se -X cpu_count for fornecido ou PYTHON_CPU_COUNT for definido, *cpu_count()* retornará o valor substituído *n*.

os.**getloadavg**()

Retorna o número de processos na fila de execução do sistema em média nos últimos 1, 5 e 15 minutos ou levanta *OSError* se a média de carga não foi obtida.

Disponibilidade: Unix.

os.**process_cpu_count**()

Obtém o número de CPUs lógicas utilizáveis pelo thread de chamada do **processo atual**. Retorna None se indeterminado. Pode ser menor que *cpu_count()* dependendo da afinidade da CPU.

A função *cpu_count()* pode ser usada para obter o número de CPUs lógicas no **sistema**.

Se -X cpu_count for fornecido ou PYTHON_CPU_COUNT for definido, *process_cpu_count()* retornará o valor substituído *n*.

Veja também a função *sched_getaffinity()*.

Adicionado na versão 3.13.

os.**sysconf**(*name*, /)

Retorna valores de configuração do sistema com valor inteiro. Se o valor de configuração especificado por *name* não estiver definido, -1 é retornado. Os comentários sobre o parâmetro *name* para *confstr()* se aplicam aqui também; o dicionário que fornece informações sobre os nomes conhecidos é fornecido por sysconf_names.

Disponibilidade: Unix.

os.**sysconf_names**

Nomes de mapeamento de dicionário aceitos por *sysconf()* para os valores inteiros definidos para esses nomes pelo sistema operacional. Isso pode ser usado para determinar o conjunto de nomes conhecidos pelo sistema.

Disponibilidade: Unix.

Alterado na versão 3.11: Adiciona o nome 'SC_MINSIGSTKSZ'.

Os dados a seguir são usados para operações de manipulação de caminhos de arquivos e diretórios. Estão definidos e disponíveis para todas as plataformas.

Operações de nível mais alto em nomes de caminho são definidos no módulo *os.path*.

os.**curdir**

A string constante usada pelo sistema operacional para se referir ao diretório atual. Isso é '.' para Windows e POSIX. Também disponível via *os.path*.

os.**pardir**

A string constante usada pelo sistema operacional para se referir ao diretório pai. Isso é '..' para Windows e POSIX. Também disponível via *os.path*.

os.**sep**

O caractere usado pelo sistema operacional para separar os componentes do nome do caminho. Este é `'/'` para POSIX e `'\\'` para Windows. Observe que saber disso não é suficiente para ser capaz de analisar ou concatenar nomes de caminho – use *os.path.split()* e *os.path.join()* – mas ocasionalmente é útil. Também disponível via *os.path*.

os.**altsep**

Um caractere alternativo usado pelo sistema operacional para separar os componentes do nome de caminho, ou `None` se apenas um caractere separador existir. Isso é definido como `'/'` em sistemas Windows onde `sep` é uma contrabarra. Também disponível via *os.path*.

os.**extsep**

O caractere que separa o nome do arquivo base da extensão; por exemplo, o `'.'` em `os.py`. Também disponível via *os.path*.

os.**pathsep**

O caractere convencionalmente usado pelo sistema operacional para separar os componentes do caminho de pesquisa (como em `PATH`), como `':'` para POSIX ou `';'` para Windows. Também disponível via *os.path*.

os.**defpath**

O caminho de pesquisa padrão usado por *exec*p** e *spawn*p** se o ambiente não tiver uma chave `'PATH'`. Também disponível via *os.path*.

os.**linesep**

A string usada para separar (ou melhor, encerrar) linhas na plataforma atual. Pode ser um único caractere, como `'\n'` para POSIX, ou múltiplos caracteres, por exemplo, `'\r\n'` para Windows. Não use *os.linesep* como terminador de linha ao escrever arquivos abertos em modo de texto (o padrão); use um único `'\n'` ao invés, em todas as plataformas.

os.**devnull**

O caminho do arquivo do dispositivo nulo. Por exemplo: `'/dev/null'` para POSIX, `'nul'` para Windows. Também disponível via *os.path*.

os.**RTLD_LAZY**

os.**RTLD_NOW**

os.**RTLD_GLOBAL**

os.**RTLD_LOCAL**

os.**RTLD_NODELETE**

os.**RTLD_NOLOAD**

os.**RTLD_DEEPBIND**

Sinalizadores para uso com as funções *setdlopenflags()* e *getdlopenflags()*. Veja a página man do Unix *dlopen(3)* para saber o que significam os diferentes sinalizadores.

Adicionado na versão 3.3.

16.1.10 Números aleatórios

os.**getrandom**(*size*, *flags=0*)

Obtém até *size* bytes aleatórios. Esta função pode retornar menos bytes que a quantia requisitada.

Esses bytes podem ser usados para propagar geradores de número aleatório no espaço do usuário ou para fins criptográficos.

`getrandom()` depende da entropia obtida de drivers de dispositivos e outras fontes de ruído ambiental. A leitura desnecessária de grandes quantidades de dados terá um impacto negativo sobre outros usuários dos dispositivos `/dev/random` e `/dev/urandom`.

O argumento *flags* é uma máscara de bits que pode conter zero ou mais dos seguintes valores operados com OU: *os.GRND_RANDOM* e *GRND_NONBLOCK*.

Veja também a página de manual do getrandom() do Linux.

Disponibilidade: Linux >= 3.17.

Adicionado na versão 3.6.

`os.`**`urandom`**(*size*, `/`)

Retorna uma sequência de bytes de tamanho *size* aleatórios próprios para uso criptográfico.

Esta função retorna bytes aleatórios de uma fonte de aleatoriedade específica do sistema operacional. Os dados retornados devem ser imprevisíveis o suficiente para aplicações criptográficos, embora sua qualidade exata dependa da implementação do sistema operacional.

No Linux, se a chamada de sistema `getrandom()` estiver disponível, ela é usada no modo bloqueante: bloqueia até que o pool de entropia urandom do sistema seja inicializado (128 bits de entropia são coletados pelo kernel). Veja a **PEP 524** para a justificativa. No Linux, a função *`getrandom()`* pode ser usada para obter bytes aleatórios no modo não bloqueante (usando a sinalização *`GRND_NONBLOCK`*) ou para pesquisar até que o pool de entropia urandom do sistema seja inicializado.

Em um sistema semelhante ao Unix, bytes aleatórios são lidos do dispositivo `/dev/urandom`. Se o dispositivo `/dev/urandom` não estiver disponível ou não for legível, a exceção *`NotImplementedError`* é levantada.

No Windows, ainda vai usar `BCryptGenRandom()`.

Ver também

O módulo *`secrets`* fornece funções de nível mais alto. Para uma interface fácil de usar para o gerador de números aleatórios fornecido por sua plataforma, consulte *`random.SystemRandom`*.

Alterado na versão 3.5: No Linux 3.17 e mais recente, a chamada de sistema `getrandom()` agora é usada quando disponível. No OpenBSD 5.6 e mais recentes, a função C `getentropy()` agora é usada. Essas funções evitam o uso de um descritor de arquivo interno.

Alterado na versão 3.5.2: No Linux, se a chamada de sistema `getrandom()` bloqueia (o pool de entropia urandom ainda não foi inicializado), recorre à leitura `/dev/urandom`.

Alterado na versão 3.6: No Linux, `getrandom()` é usado agora no modo de bloqueio para aumentar a segurança.

Alterado na versão 3.11: No Windows, `BCryptGenRandom()` é usado em vez de `CryptGenRandom()`, o qual foi descontinuado.

`os.`**`GRND_NONBLOCK`**

Por padrão, ao ler de `/dev/random`, *`getrandom()`* bloqueia se nenhum byte aleatório estiver disponível, e ao ler de `/dev/urandom`, ele bloqueia se o pool de entropia ainda não foi inicializado.

Se o sinalizador *`GRND_NONBLOCK`* estiver definido, então *`getrandom()`* não bloqueia nesses casos, mas, em vez disso, levanta *`BlockingIOError`* imediatamente.

Adicionado na versão 3.6.

`os.`**`GRND_RANDOM`**

Se este bit é definido os bytes aleatórios são sorteados a partir de `/dev/random` ao invés de `/dev/urandom`.

Adicionado na versão 3.6.

16.2 `io` — Ferramentas essenciais para trabalhar com fluxos

Código-fonte: Lib/io.py

16.2.1 Visão Geral

O módulo `io` fornece os principais recursos do Python para lidar com vários tipos de E/S. Existem três tipos principais de E/S: *E/S de texto*, *E/S binária* e *E/S bruta*. Essas são categorias genéricas, e vários repositórios de apoio podem ser usados para cada uma delas. Um objeto concreto pertencente a qualquer uma dessas categorias é chamado de *objeto arquivo*. Outros termos comuns são *fluxo* e *objeto arquivo ou similar*.

Independentemente de sua categoria, cada objeto de fluxo concreto também terá vários recursos: pode ser somente leitura, somente escrita ou leitura e escrita. Também pode permitir acesso aleatório arbitrário (buscando para frente ou para trás em qualquer local) ou apenas acesso sequencial (por exemplo, no caso de um soquete ou encadeamento).

Todos os fluxos são cuidadosos com o tipo de dados que você fornece a eles. Por exemplo, atribuir um objeto `str` ao método `write()` de um fluxo binário levantará uma `TypeError`. O mesmo ocorrerá com um objeto `bytes` ao método `write()` de um fluxo de texto.

Alterado na versão 3.3: Operações que costumavam levantar `IOError` agora levantam `OSError`, já que `IOError` agora é um apelido de `OSError`.

E/S de texto

A E/S de texto espera e produz objetos `str`. Isso significa que sempre que o armazenamento de apoio for composto nativamente por bytes (como no caso de um arquivo), a codificação e a decodificação dos dados são feitas de forma transparente, bem como a tradução opcional de caracteres de quebra de linha específicos da plataforma.

A maneira mais fácil de criar um fluxo de texto é com `open()`, especificando opcionalmente uma codificação:

```
f = open("myfile.txt", "r", encoding="utf-8")
```

Os fluxos de texto na memória também estão disponíveis como objetos `StringIO`:

```
f = io.StringIO("alguns dados em texto iniciais")
```

Nota

Ao trabalhar com um fluxo não bloqueante, esteja ciente de que operações de leitura em objetos de E/S de texto podem levantar uma `BlockingIOError` se o fluxo não puder executar a operação imediatamente.

A API de fluxo de texto é descrita em detalhes na documentação de `TextIOBase`.

E/S binária

E/S binária (também chamada de *E/S com buffer*) espera *objeto bytes ou similar* e produz objetos `bytes`. Nenhuma codificação, decodificação ou tradução de quebra de linha é realizada. Esta categoria de fluxos pode ser usada para todos os tipos de dados não textuais e também quando se deseja controle manual sobre o tratamento de dados textuais.

A maneira mais fácil de criar um fluxo binário é com `open()` com `'b'` na string de modo:

```
f = open("meuarquivo.jpg", "rb")
```

Os fluxos binários na memória também estão disponíveis como objetos `BytesIO`:

```
f = io.BytesIO(b"alguns dados bin\xe1rios iniciais: \x00\x01")
```

A API de fluxo binário é descrita em detalhes na documentação de `BufferedIOBase`.

Outros módulos de biblioteca podem fornecer maneiras adicionais de criar fluxos de texto ou binários. Veja `socket.socket.makefile()` como exemplo.

E/S bruta

A E/S bruta (também chamada de *E/S sem buffer*) é geralmente usada como um bloco de construção de baixo nível para fluxos binários e de texto; raramente é útil manipular diretamente um fluxo bruto a partir do código do usuário. No entanto, você pode criar um fluxo bruto abrindo um arquivo em modo binário com o buffer desabilitado:

```
f = open("meuarquivo.jpg", "rb", buffering=0)
```

A API de fluxo bruto é descrita em detalhes na documentação de *RawIOBase*.

16.2.2 Codificação do texto

A codificação padrão de *TextIOWrapper* e *open()* é específica da localidade (*locale.getencoding()*).

No entanto, muitos desenvolvedores esquecem de especificar a codificação ao abrir arquivos de texto codificados em UTF-8 (por exemplo, JSON, TOML, Markdown, etc.), já que a maioria das plataformas Unix usa a localidade UTF-8 por padrão. Isso causa bugs porque a codificação de localidade não é UTF-8 para a maioria dos usuários do Windows. Por exemplo:

```
# Pode não funcionar no Windows quando há caracteres não ASCII no arquivo.
with open("README.md") as f:
    long_description = f.read()
```

Portanto, é altamente recomendável que você especifique a codificação explicitamente ao abrir arquivos de texto. Se quiser usar UTF-8, passe `encoding="utf-8"`. Para usar a codificação da localidade atual, `encoding="locale"` é suportado desde o Python 3.10.

Ver também

Modo UTF-8 do Python
: O modo Python UTF-8 pode ser usado para alterar a codificação padrão para UTF-8 a partir da codificação específica de localidade.

PEP 686
: Python 3.15 utilizará *Modo UTF-8 do Python* por padrão.

Ativar EncodingWarning

Adicionado na versão 3.10: Veja a **PEP 597** para mais detalhes.

Para descobrir onde a codificação da localidade padrão é usada, você pode habilitar a opção de linha de comando `-X warn_default_encoding` ou definir a variável de ambiente `PYTHONWARNDEFAULTENCODING`, que emitirá um *EncodingWarning* quando a codificação padrão for usada.

Se você estiver fornecendo uma API que usa *open()* ou *TextIOWrapper* e passa `encoding=None` como parâmetro, você pode usar *text_encoding()* para que os chamadores da API emitam um *EncodingWarning* se não passarem um `encoding`. No entanto, considere usar UTF-8 por padrão (ou seja, `encoding="utf-8"`) para novas APIs.

16.2.3 High-level Module Interface

io.DEFAULT_BUFFER_SIZE
: An int containing the default buffer size used by the module's buffered I/O classes. *open()* uses the file's blksize (as obtained by *os.stat()*) if possible.

io.open(*file*, *mode='r'*, *buffering=-1*, *encoding=None*, *errors=None*, *newline=None*, *closefd=True*, *opener=None*)
: This is an alias for the builtin *open()* function.

  Esta função levanta um *evento de auditoria* `open` com os argumentos *path*, *mode* e *flags*. Os argumentos *mode* e *flags* podem ter sido modificados ou inferidos da chamada original.

io.**open_code**(*path*)

Opens the provided file with mode `'rb'`. This function should be used when the intent is to treat the contents as executable code.

path should be a *str* and an absolute path.

The behavior of this function may be overridden by an earlier call to the `PyFile_SetOpenCodeHook()`. However, assuming that *path* is a *str* and an absolute path, `open_code(path)` should always behave the same as `open(path, 'rb')`. Overriding the behavior is intended for additional validation or preprocessing of the file.

Adicionado na versão 3.8.

io.**text_encoding**(*encoding*, *stacklevel=2*, */*)

This is a helper function for callables that use *open()* or *TextIOWrapper* and have an `encoding=None` parameter.

This function returns *encoding* if it is not `None`. Otherwise, it returns `"locale"` or `"utf-8"` depending on *UTF-8 Mode*.

This function emits an *EncodingWarning* if *sys.flags.warn_default_encoding* is true and *encoding* is `None`. *stacklevel* specifies where the warning is emitted. For example:

```
def read_text(path, encoding=None):
    encoding = io.text_encoding(encoding)  # stacklevel=2
    with open(path, encoding) as f:
        return f.read()
```

In this example, an *EncodingWarning* is emitted for the caller of `read_text()`.

Veja *Codificação do texto* para mais informações.

Adicionado na versão 3.10.

Alterado na versão 3.11: *text_encoding()* returns "utf-8" when UTF-8 mode is enabled and *encoding* is `None`.

exception io.**BlockingIOError**

This is a compatibility alias for the builtin *BlockingIOError* exception.

exception io.**UnsupportedOperation**

An exception inheriting *OSError* and *ValueError* that is raised when an unsupported operation is called on a stream.

Ver também

sys
contains the standard IO streams: *sys.stdin*, *sys.stdout*, and *sys.stderr*.

16.2.4 hierarquia de classe

The implementation of I/O streams is organized as a hierarchy of classes. First *abstract base classes* (ABCs), which are used to specify the various categories of streams, then concrete classes providing the standard stream implementations.

Nota

The abstract base classes also provide default implementations of some methods in order to help implementation of concrete stream classes. For example, *BufferedIOBase* provides unoptimized implementations of `readinto()` and `readline()`.

At the top of the I/O hierarchy is the abstract base class *IOBase*. It defines the basic interface to a stream. Note, however, that there is no separation between reading and writing to streams; implementations are allowed to raise *UnsupportedOperation* if they do not support a given operation.

The *RawIOBase* ABC extends *IOBase*. It deals with the reading and writing of bytes to a stream. *FileIO* subclasses *RawIOBase* to provide an interface to files in the machine's file system.

The *BufferedIOBase* ABC extends *IOBase*. It deals with buffering on a raw binary stream (*RawIOBase*). Its subclasses, *BufferedWriter*, *BufferedReader*, and *BufferedRWPair* buffer raw binary streams that are writable, readable, and both readable and writable, respectively. *BufferedRandom* provides a buffered interface to seekable streams. Another *BufferedIOBase* subclass, *BytesIO*, is a stream of in-memory bytes.

The *TextIOBase* ABC extends *IOBase*. It deals with streams whose bytes represent text, and handles encoding and decoding to and from strings. *TextIOWrapper*, which extends *TextIOBase*, is a buffered text interface to a buffered raw stream (*BufferedIOBase*). Finally, *StringIO* is an in-memory stream for text.

Argument names are not part of the specification, and only the arguments of *open()* are intended to be used as keyword arguments.

The following table summarizes the ABCs provided by the *io* module:

ABC	Inherits	Stub Methods	Mixin Methods and Properties
IOBase		`fileno`, `seek`, and `truncate`	`close`, `closed`, `__enter__`, `__exit__`, `flush`, `isatty`, `__iter__`, `__next__`, `readable`, `readline`, `readlines`, `seekable`, `tell`, `writable`, and `writelines`
RawIOBase	*IOBase*	`readinto` and `write`	Inherited *IOBase* methods, `read`, and `readall`
BufferedIOBase	*IOBase*	`detach`, `read`, `read1`, and `write`	Inherited *IOBase* methods, `readinto`, and `readinto1`
TextIOBase	*IOBase*	`detach`, `read`, `readline`, and `write`	Inherited *IOBase* methods, `encoding`, `errors`, and `newlines`

I/O Base Classes

class io.**IOBase**

The abstract base class for all I/O classes.

This class provides empty abstract implementations for many methods that derived classes can override selectively; the default implementations represent a file that cannot be read, written or seeked.

Even though *IOBase* does not declare `read()` or `write()` because their signatures will vary, implementations and clients should consider those methods part of the interface. Also, implementations may raise a *ValueError* (or *UnsupportedOperation*) when operations they do not support are called.

The basic type used for binary data read from or written to a file is *bytes*. Other *bytes-like objects* are accepted as method arguments too. Text I/O classes work with *str* data.

Note that calling any method (even inquiries) on a closed stream is undefined. Implementations may raise *ValueError* in this case.

IOBase (and its subclasses) supports the iterator protocol, meaning that an *IOBase* object can be iterated over yielding the lines in a stream. Lines are defined slightly differently depending on whether the stream is a binary stream (yielding bytes), or a text stream (yielding character strings). See *readline()* below.

IOBase is also a context manager and therefore supports the `with` statement. In this example, *file* is closed after the `with` statement's suite is finished—even if an exception occurs:

```
with open('spam.txt', 'w') as file:
    file.write('Spam and eggs!')
```

IOBase provides these data attributes and methods:

close()

Flush and close this stream. This method has no effect if the file is already closed. Once the file is closed, any operation on the file (e.g. reading or writing) will raise a *ValueError*.

As a convenience, it is allowed to call this method more than once; only the first call, however, will have an effect.

closed

`True` if the stream is closed.

fileno()

Return the underlying file descriptor (an integer) of the stream if it exists. An *OSError* is raised if the IO object does not use a file descriptor.

flush()

Flush the write buffers of the stream if applicable. This does nothing for read-only and non-blocking streams.

isatty()

Return `True` if the stream is interactive (i.e., connected to a terminal/tty device).

readable()

Return `True` if the stream can be read from. If `False`, `read()` will raise *OSError*.

readline(*size=-1*, */*)

Read and return one line from the stream. If *size* is specified, at most *size* bytes will be read.

The line terminator is always `b'\n'` for binary files; for text files, the *newline* argument to *open()* can be used to select the line terminator(s) recognized.

readlines(*hint=-1*, */*)

Read and return a list of lines from the stream. *hint* can be specified to control the number of lines read: no more lines will be read if the total size (in bytes/characters) of all lines so far exceeds *hint*.

hint values of `0` or less, as well as `None`, are treated as no hint.

Note that it's already possible to iterate on file objects using `for line in file: ...` without calling `file.readlines()`.

seek(*offset*, *whence=os.SEEK_SET*, */*)

Change the stream position to the given byte *offset*, interpreted relative to the position indicated by *whence*, and return the new absolute position. Values for *whence* are:

- *os.SEEK_SET* or `0` – start of the stream (the default); *offset* should be zero or positive
- *os.SEEK_CUR* or `1` – current stream position; *offset* may be negative
- *os.SEEK_END* or `2` – end of the stream; *offset* is usually negative

Adicionado na versão 3.1: The `SEEK_*` constants.

Adicionado na versão 3.3: Some operating systems could support additional values, like *os.SEEK_HOLE* or *os.SEEK_DATA*. The valid values for a file could depend on it being open in text or binary mode.

seekable()

Return `True` if the stream supports random access. If `False`, *seek()*, *tell()* and *truncate()* will raise *OSError*.

tell()

Return the current stream position.

truncate(*size=None*, */*)

Resize the stream to the given *size* in bytes (or the current position if *size* is not specified). The current stream position isn't changed. This resizing can extend or reduce the current file size. In case of extension, the contents of the new file area depend on the platform (on most systems, additional bytes are zero-filled). The new file size is returned.

Alterado na versão 3.5: Windows will now zero-fill files when extending.

writable()

Return `True` if the stream supports writing. If `False`, `write()` and *truncate()* will raise *OSError*.

writelines(*lines*, */*)

Write a list of lines to the stream. Line separators are not added, so it is usual for each of the lines provided to have a line separator at the end.

__del__()

Prepare for object destruction. *IOBase* provides a default implementation of this method that calls the instance's *close()* method.

class io.**RawIOBase**

Base class for raw binary streams. It inherits from *IOBase*.

Raw binary streams typically provide low-level access to an underlying OS device or API, and do not try to encapsulate it in high-level primitives (this functionality is done at a higher-level in buffered binary streams and text streams, described later in this page).

RawIOBase provides these methods in addition to those from *IOBase*:

read(*size=-1*, */*)

Read up to *size* bytes from the object and return them. As a convenience, if *size* is unspecified or -1, all bytes until EOF are returned. Otherwise, only one system call is ever made. Fewer than *size* bytes may be returned if the operating system call returns fewer than *size* bytes.

If 0 bytes are returned, and *size* was not 0, this indicates end of file. If the object is in non-blocking mode and no bytes are available, `None` is returned.

The default implementation defers to *readall()* and *readinto()*.

readall()

Read and return all the bytes from the stream until EOF, using multiple calls to the stream if necessary.

readinto(*b*, */*)

Read bytes into a pre-allocated, writable *bytes-like object* *b*, and return the number of bytes read. For example, *b* might be a *bytearray*. If the object is in non-blocking mode and no bytes are available, `None` is returned.

write(*b*, */*)

Write the given *bytes-like object*, *b*, to the underlying raw stream, and return the number of bytes written. This can be less than the length of *b* in bytes, depending on specifics of the underlying raw stream, and especially if it is in non-blocking mode. `None` is returned if the raw stream is set not to block and no single byte could be readily written to it. The caller may release or mutate *b* after this method returns, so the implementation should only access *b* during the method call.

class io.**BufferedIOBase**

Base class for binary streams that support some kind of buffering. It inherits from *IOBase*.

The main difference with *RawIOBase* is that methods *read()*, *readinto()* and *write()* will try (respectively) to read as much input as requested or to emit all provided data.

In addition, if the underlying raw stream is in non-blocking mode, when the system returns would block *write()* will raise *BlockingIOError* with *BlockingIOError.characters_written* and *read()* will return data read so far or `None` if no data is available.

Besides, the *read()* method does not have a default implementation that defers to *readinto()*.

A typical *BufferedIOBase* implementation should not inherit from a *RawIOBase* implementation, but wrap one, like *BufferedWriter* and *BufferedReader* do.

BufferedIOBase provides or overrides these data attributes and methods in addition to those from *IOBase*:

raw

The underlying raw stream (a *RawIOBase* instance) that *BufferedIOBase* deals with. This is not part of the *BufferedIOBase* API and may not exist on some implementations.

detach()

Separate the underlying raw stream from the buffer and return it.

After the raw stream has been detached, the buffer is in an unusable state.

Some buffers, like *BytesIO*, do not have the concept of a single raw stream to return from this method. They raise *UnsupportedOperation*.

Adicionado na versão 3.1.

read(*size=-1*, */*)

Read and return up to *size* bytes. If the argument is omitted, `None`, or negative read as much as possible.

Fewer bytes may be returned than requested. An empty *bytes* object is returned if the stream is already at EOF. More than one read may be made and calls may be retried if specific errors are encountered, see *os.read()* and **PEP 475** for more details. Less than size bytes being returned does not imply that EOF is imminent.

When reading as much as possible the default implementation will use `raw.readall` if available (which should implement *RawIOBase.readall()*), otherwise will read in a loop until read returns `None`, an empty *bytes*, or a non-retryable error. For most streams this is to EOF, but for non-blocking streams more data may become available.

> **Nota**
>
> When the underlying raw stream is non-blocking, implementations may either raise *BlockingIOError* or return `None` if no data is available. *io* implementations return `None`.

read1(*size=-1*, */*)

Read and return up to *size* bytes, calling *readinto()* which may retry if *EINTR* is encountered per **PEP 475**. If *size* is `-1` or not provided, the implementation will choose an arbitrary value for *size*.

> **Nota**
>
> When the underlying raw stream is non-blocking, implementations may either raise *BlockingIOError* or return `None` if no data is available. *io* implementations return `None`.

readinto(*b*, */*)

Read bytes into a pre-allocated, writable *bytes-like object* *b* and return the number of bytes read. For example, *b* might be a *bytearray*.

Like *read()*, multiple reads may be issued to the underlying raw stream, unless the latter is interactive.

A *BlockingIOError* is raised if the underlying raw stream is in non blocking-mode, and has no data available at the moment.

readinto1(*b*, */*)

Read bytes into a pre-allocated, writable *bytes-like object* *b*, using at most one call to the underlying raw stream's *read()* (or *readinto()*) method. Return the number of bytes read.

A *BlockingIOError* is raised if the underlying raw stream is in non blocking-mode, and has no data available at the moment.

Adicionado na versão 3.5.

write(*b*, /)

Write the given *bytes-like object*, *b*, and return the number of bytes written (always equal to the length of *b* in bytes, since if the write fails an *OSError* will be raised). Depending on the actual implementation, these bytes may be readily written to the underlying stream, or held in a buffer for performance and latency reasons.

When in non-blocking mode, a *BlockingIOError* is raised if the data needed to be written to the raw stream but it couldn't accept all the data without blocking.

The caller may release or mutate *b* after this method returns, so the implementation should only access *b* during the method call.

Raw File I/O

class io.**FileIO**(*name*, *mode='r'*, *closefd=True*, *opener=None*)

A raw binary stream representing an OS-level file containing bytes data. It inherits from *RawIOBase*.

The *name* can be one of two things:

- a character string or *bytes* object representing the path to the file which will be opened. In this case closefd must be `True` (the default) otherwise an error will be raised.
- an integer representing the number of an existing OS-level file descriptor to which the resulting *FileIO* object will give access. When the FileIO object is closed this fd will be closed as well, unless *closefd* is set to `False`.

The *mode* can be `'r'`, `'w'`, `'x'` or `'a'` for reading (default), writing, exclusive creation or appending. The file will be created if it doesn't exist when opened for writing or appending; it will be truncated when opened for writing. *FileExistsError* will be raised if it already exists when opened for creating. Opening a file for creating implies writing, so this mode behaves in a similar way to `'w'`. Add a `'+'` to the mode to allow simultaneous reading and writing.

The *read()* (when called with a positive argument), *readinto()* and *write()* methods on this class will only make one system call.

A custom opener can be used by passing a callable as *opener*. The underlying file descriptor for the file object is then obtained by calling *opener* with (*name*, *flags*). *opener* must return an open file descriptor (passing *os.open* as *opener* results in functionality similar to passing `None`).

O arquivo recém-criado é *non-inheritable*.

See the *open()* built-in function for examples on using the *opener* parameter.

Alterado na versão 3.3: The *opener* parameter was added. The `'x'` mode was added.

Alterado na versão 3.4: O arquivo agora é não herdável.

FileIO provides these data attributes in addition to those from *RawIOBase* and *IOBase*:

mode

The mode as given in the constructor.

name

The file name. This is the file descriptor of the file when no name is given in the constructor.

Buffered Streams

Buffered I/O streams provide a higher-level interface to an I/O device than raw I/O does.

class io.**BytesIO**(*initial_bytes=b''*)

A binary stream using an in-memory bytes buffer. It inherits from *BufferedIOBase*. The buffer is discarded when the *close()* method is called.

The optional argument *initial_bytes* is a *bytes-like object* that contains initial data.

BytesIO provides or overrides these methods in addition to those from *BufferedIOBase* and *IOBase*:

getbuffer()

Return a readable and writable view over the contents of the buffer without copying them. Also, mutating the view will transparently update the contents of the buffer:

```
>>> b = io.BytesIO(b"abcdef")
>>> view = b.getbuffer()
>>> view[2:4] = b"56"
>>> b.getvalue()
b'ab56ef'
```

Nota

As long as the view exists, the *BytesIO* object cannot be resized or closed.

Adicionado na versão 3.2.

getvalue()

Return *bytes* containing the entire contents of the buffer.

read1(*size=-1*, */*)

In *BytesIO*, this is the same as *read()*.

Alterado na versão 3.7: The *size* argument is now optional.

readinto1(*b*, */*)

In *BytesIO*, this is the same as *readinto()*.

Adicionado na versão 3.5.

class io.**BufferedReader**(*raw*, *buffer_size=DEFAULT_BUFFER_SIZE*)

A buffered binary stream providing higher-level access to a readable, non seekable *RawIOBase* raw binary stream. It inherits from *BufferedIOBase*.

When reading data from this object, a larger amount of data may be requested from the underlying raw stream, and kept in an internal buffer. The buffered data can then be returned directly on subsequent reads.

The constructor creates a *BufferedReader* for the given readable *raw* stream and *buffer_size*. If *buffer_size* is omitted, *DEFAULT_BUFFER_SIZE* is used.

BufferedReader provides or overrides these methods in addition to those from *BufferedIOBase* and *IOBase*:

peek(*size=0*, */*)

Return bytes from the stream without advancing the position. The number of bytes returned may be less or more than requested. If the underlying raw stream is non-blocking and the operation would block, returns empty bytes.

read(*size=-1*, */*)

In *BufferedReader* this is the same as *io.BufferedIOBase.read()*

read1(*size=-1*, */*)

In *BufferedReader* this is the same as *io.BufferedIOBase.read1()*

Alterado na versão 3.7: The *size* argument is now optional.

class io.**BufferedWriter**(*raw*, *buffer_size=DEFAULT_BUFFER_SIZE*)

A buffered binary stream providing higher-level access to a writeable, non seekable *RawIOBase* raw binary stream. It inherits from *BufferedIOBase*.

When writing to this object, data is normally placed into an internal buffer. The buffer will be written out to the underlying *RawIOBase* object under various conditions, including:

- when the buffer gets too small for all pending data;
- when *flush()* is called;
- when a *seek()* is requested (for *BufferedRandom* objects);
- when the *BufferedWriter* object is closed or destroyed.

The constructor creates a *BufferedWriter* for the given writeable *raw* stream. If the *buffer_size* is not given, it defaults to *DEFAULT_BUFFER_SIZE*.

BufferedWriter provides or overrides these methods in addition to those from *BufferedIOBase* and *IOBase*:

flush()

Force bytes held in the buffer into the raw stream. A *BlockingIOError* should be raised if the raw stream blocks.

write(*b*, /)

Write the *bytes-like object*, *b*, and return the number of bytes written. When in non-blocking mode, a *BlockingIOError* with *BlockingIOError.characters_written* set is raised if the buffer needs to be written out but the raw stream blocks.

class io.**BufferedRandom**(*raw*, *buffer_size=DEFAULT_BUFFER_SIZE*)

A buffered binary stream providing higher-level access to a seekable *RawIOBase* raw binary stream. It inherits from *BufferedReader* and *BufferedWriter*.

The constructor creates a reader and writer for a seekable raw stream, given in the first argument. If the *buffer_size* is omitted it defaults to *DEFAULT_BUFFER_SIZE*.

BufferedRandom is capable of anything *BufferedReader* or *BufferedWriter* can do. In addition, *seek()* and *tell()* are guaranteed to be implemented.

class io.**BufferedRWPair**(*reader*, *writer*, *buffer_size=DEFAULT_BUFFER_SIZE*, /)

A buffered binary stream providing higher-level access to two non seekable *RawIOBase* raw binary streams—one readable, the other writeable. It inherits from *BufferedIOBase*.

reader and *writer* are *RawIOBase* objects that are readable and writeable respectively. If the *buffer_size* is omitted it defaults to *DEFAULT_BUFFER_SIZE*.

BufferedRWPair implements all of *BufferedIOBase*'s methods except for *detach()*, which raises *UnsupportedOperation*.

> **Aviso**
>
> *BufferedRWPair* does not attempt to synchronize accesses to its underlying raw streams. You should not pass it the same object as reader and writer; use *BufferedRandom* instead.

E/S de texto

class io.**TextIOBase**

Base class for text streams. This class provides a character and line based interface to stream I/O. It inherits from *IOBase*.

TextIOBase provides or overrides these data attributes and methods in addition to those from *IOBase*:

encoding

The name of the encoding used to decode the stream's bytes into strings, and to encode strings into bytes.

errors

The error setting of the decoder or encoder.

newlines

A string, a tuple of strings, or `None`, indicating the newlines translated so far. Depending on the implementation and the initial constructor flags, this may not be available.

buffer

The underlying binary buffer (a *BufferedIOBase* or *RawIOBase* instance) that *TextIOBase* deals with. This is not part of the *TextIOBase* API and may not exist in some implementations.

detach()

Separate the underlying binary buffer from the *TextIOBase* and return it.

After the underlying buffer has been detached, the *TextIOBase* is in an unusable state.

Some *TextIOBase* implementations, like *StringIO*, may not have the concept of an underlying buffer and calling this method will raise *UnsupportedOperation*.

Adicionado na versão 3.1.

read(*size=-1*, /)

Read and return at most *size* characters from the stream as a single *str*. If *size* is negative or `None`, reads until EOF.

readline(*size=-1*, /)

Read until newline or EOF and return a single *str*. If the stream is already at EOF, an empty string is returned.

If *size* is specified, at most *size* characters will be read.

seek(*offset*, *whence=SEEK_SET*, /)

Change the stream position to the given *offset*. Behaviour depends on the *whence* parameter. The default value for *whence* is `SEEK_SET`.

- `SEEK_SET` or `0`: seek from the start of the stream (the default); *offset* must either be a number returned by *TextIOBase.tell()*, or zero. Any other *offset* value produces undefined behaviour.
- `SEEK_CUR` or `1`: "seek" to the current position; *offset* must be zero, which is a no-operation (all other values are unsupported).
- `SEEK_END` or `2`: seek to the end of the stream; *offset* must be zero (all other values are unsupported).

Return the new absolute position as an opaque number.

Adicionado na versão 3.1: The `SEEK_*` constants.

tell()

Return the current stream position as an opaque number. The number does not usually represent a number of bytes in the underlying binary storage.

write(*s*, /)

Write the string *s* to the stream and return the number of characters written.

class io.**TextIOWrapper**(*buffer*, *encoding=None*, *errors=None*, *newline=None*, *line_buffering=False*, *write_through=False*)

A buffered text stream providing higher-level access to a *BufferedIOBase* buffered binary stream. It inherits from *TextIOBase*.

encoding gives the name of the encoding that the stream will be decoded or encoded with. In *UTF-8 Mode*, this defaults to UTF-8. Otherwise, it defaults to *locale.getencoding()*. `encoding="locale"` can be used to specify the current locale's encoding explicitly. See *Codificação do texto* for more information.

errors is an optional string that specifies how encoding and decoding errors are to be handled. Pass `'strict'` to raise a *ValueError* exception if there is an encoding error (the default of `None` has the same effect), or pass `'ignore'` to ignore errors. (Note that ignoring encoding errors can lead to data loss.) `'replace'` causes a replacement marker (such as `'?'`) to be inserted where there is malformed data. `'backslashreplace'` causes malformed data to be replaced by a backslashed escape sequence. When writing, `'xmlcharrefreplace'` (replace with the appropriate XML character reference) or `'namereplace'` (replace with `\N{...}` escape sequences) can be used. Any other error handling name that has been registered with *codecs.register_error()* is also valid.

newline controls how line endings are handled. It can be `None`, `''`, `'\n'`, `'\r'`, and `'\r\n'`. It works as follows:

- When reading input from the stream, if *newline* is `None`, *universal newlines* mode is enabled. Lines in the input can end in `'\n'`, `'\r'`, or `'\r\n'`, and these are translated into `'\n'` before being returned to the caller. If *newline* is `''`, universal newlines mode is enabled, but line endings are returned to the caller untranslated. If *newline* has any of the other legal values, input lines are only terminated by the given string, and the line ending is returned to the caller untranslated.
- Ao gravar a saída no fluxo, se *newline* for `None`, quaisquer caracteres `'\n'` gravados serão traduzidos para o separador de linhas padrão do sistema, *os.linesep*. Se *newline* for `''` ou `'\n'`, nenhuma tradução ocorrerá. Se *newline* for um dos outros valores legais, qualquer caractere `'\n'` escrito será traduzido para a string especificada.

If *line_buffering* is `True`, *flush()* is implied when a call to write contains a newline character or a carriage return.

If *write_through* is `True`, calls to *write()* are guaranteed not to be buffered: any data written on the *TextIOWrapper* object is immediately handled to its underlying binary *buffer*.

Alterado na versão 3.3: The *write_through* argument has been added.

Alterado na versão 3.3: The default *encoding* is now `locale.getpreferredencoding(False)` instead of `locale.getpreferredencoding()`. Don't change temporary the locale encoding using *locale.setlocale()*, use the current locale encoding instead of the user preferred encoding.

Alterado na versão 3.10: The *encoding* argument now supports the `"locale"` dummy encoding name.

Nota

When the underlying raw stream is non-blocking, a *BlockingIOError* may be raised if a read operation cannot be completed immediately.

TextIOWrapper provides these data attributes and methods in addition to those from *TextIOBase* and *IOBase*:

line_buffering

Whether line buffering is enabled.

write_through

Whether writes are passed immediately to the underlying binary buffer.

Adicionado na versão 3.7.

reconfigure(*, *encoding=None*, *errors=None*, *newline=None*, *line_buffering=None*, *write_through=None*)

Reconfigure this text stream using new settings for *encoding*, *errors*, *newline*, *line_buffering* and *write_through*.

Parameters not specified keep current settings, except `errors='strict'` is used when *encoding* is specified but *errors* is not specified.

It is not possible to change the encoding or newline if some data has already been read from the stream. On the other hand, changing encoding after write is possible.

This method does an implicit stream flush before setting the new parameters.

Adicionado na versão 3.7.

Alterado na versão 3.11: The method supports `encoding="locale"` option.

seek(*cookie*, *whence=os.SEEK_SET*, */*)

Set the stream position. Return the new stream position as an *int*.

Four operations are supported, given by the following argument combinations:

- `seek(0, SEEK_SET)`: Rewind to the start of the stream.
- `seek(cookie, SEEK_SET)`: Restore a previous position; *cookie* **must be** a number returned by *tell()*.
- `seek(0, SEEK_END)`: Fast-forward to the end of the stream.
- `seek(0, SEEK_CUR)`: Leave the current stream position unchanged.

Any other argument combinations are invalid, and may raise exceptions.

Ver também

os.SEEK_SET, *os.SEEK_CUR*, and *os.SEEK_END*.

tell()

Return the stream position as an opaque number. The return value of `tell()` can be given as input to *seek()*, to restore a previous stream position.

class io.**StringIO**(*initial_value=''*, *newline='\n'*)

A text stream using an in-memory text buffer. It inherits from *TextIOBase*.

The text buffer is discarded when the *close()* method is called.

The initial value of the buffer can be set by providing *initial_value*. If newline translation is enabled, newlines will be encoded as if by *write()*. The stream is positioned at the start of the buffer which emulates opening an existing file in a `w+` mode, making it ready for an immediate write from the beginning or for a write that would overwrite the initial value. To emulate opening a file in an `a+` mode ready for appending, use `f.seek(0, io.SEEK_END)` to reposition the stream at the end of the buffer.

The *newline* argument works like that of *TextIOWrapper*, except that when writing output to the stream, if *newline* is `None`, newlines are written as `\n` on all platforms.

StringIO provides this method in addition to those from *TextIOBase* and *IOBase*:

getvalue()

Return a *str* containing the entire contents of the buffer. Newlines are decoded as if by *read()*, although the stream position is not changed.

Exemplo de uso:

```
import io

output = io.StringIO()
output.write('First line.\n')
print('Second line.', file=output)

# Retrieve file contents -- this will be
# 'First line.\nSecond line.\n'
contents = output.getvalue()

# Close object and discard memory buffer --
# .getvalue() will now raise an exception.
output.close()
```

class io.**IncrementalNewlineDecoder**

A helper codec that decodes newlines for *universal newlines* mode. It inherits from *codecs.IncrementalDecoder*.

16.2.5 Static Typing

The following protocols can be used for annotating function and method arguments for simple stream reading or writing operations. They are decorated with *@typing.runtime_checkable*.

class io.**Reader**[*T*]()

Generic protocol for reading from a file or other input stream. `T` will usually be *str* or *bytes*, but can be any type that is read from the stream.

Adicionado na versão 3.14.

read()

read(*size*, /)

Read data from the input stream and return it. If *size* is specified, it should be an integer, and at most *size* items (bytes/characters) will be read.

Por exemplo:

```
def read_it(reader: Reader[str]):
    data = reader.read(11)
    assert isinstance(data, str)
```

class io.**Writer**[*T*]()

Generic protocol for writing to a file or other output stream. `T` will usually be *str* or *bytes*, but can be any type that can be written to the stream.

Adicionado na versão 3.14.

write(*data*, /)

Write *data* to the output stream and return the number of items (bytes/characters) written.

Por exemplo:

```
def write_binary(writer: Writer[bytes]):
    writer.write(b"Hello world!\n")
```

See *ABCs e protocolos para trabalhar com E/S* for other I/O related protocols and classes that can be used for static type checking.

16.2.6 Desempenho

This section discusses the performance of the provided concrete I/O implementations.

E/S binária

By reading and writing only large chunks of data even when the user asks for a single byte, buffered I/O hides any inefficiency in calling and executing the operating system's unbuffered I/O routines. The gain depends on the OS and the kind of I/O which is performed. For example, on some modern OSes such as Linux, unbuffered disk I/O can be as fast as buffered I/O. The bottom line, however, is that buffered I/O offers predictable performance regardless of the platform and the backing device. Therefore, it is almost always preferable to use buffered I/O rather than unbuffered I/O for binary data.

E/S de texto

Text I/O over a binary storage (such as a file) is significantly slower than binary I/O over the same storage, because it requires conversions between unicode and binary data using a character codec. This can become noticeable handling

huge amounts of text data like large log files. Also, *tell()* and *seek()* are both quite slow due to the reconstruction algorithm used.

StringIO, however, is a native in-memory unicode container and will exhibit similar speed to *BytesIO*.

Multi-threading

FileIO objects are thread-safe to the extent that the operating system calls (such as *read(2)* under Unix) they wrap are thread-safe too.

Binary buffered objects (instances of *BufferedReader*, *BufferedWriter*, *BufferedRandom* and *BufferedRWPair*) protect their internal structures using a lock; it is therefore safe to call them from multiple threads at once.

TextIOWrapper objects are not thread-safe.

Reentrancy

Binary buffered objects (instances of *BufferedReader*, *BufferedWriter*, *BufferedRandom* and *BufferedRWPair*) are not reentrant. While reentrant calls will not happen in normal situations, they can arise from doing I/O in a *signal* handler. If a thread tries to re-enter a buffered object which it is already accessing, a *RuntimeError* is raised. Note this doesn't prohibit a different thread from entering the buffered object.

The above implicitly extends to text files, since the *open()* function will wrap a buffered object inside a *TextIOWrapper*. This includes standard streams and therefore affects the built-in *print()* function as well.

16.3 `time` — Acesso ao horário e conversões

Esse módulo provê várias funções relacionadas à tempo. Para funcionalidades relacionadas veja também os módulos *datetime* e *calendar*

Apesar desse módulo sempre estar disponível, nem todas as suas funções estão disponíveis em todas as plataformas. A maioria das funções definidas nesse módulo chamam funções da biblioteca da plataforma de C com mesmo nome. Pode ser útil consultar a documentação da plataforma, pois da semântica dessas funções variam a depender da plataforma.

A seguir, uma explicação de algumas terminologias e convenções.

- O *era* (*epoch*) é o ponto onde o tempo começa, o valor de retorno de `time.gmtime(0)`. É 1º de janeiro de 1970, 00:00:00 (UTC) em todas as plataformas.
- O termo *segundos desde a era* refere-se ao número total de segundos decorrido desde a era, tipicamente excluindo-se os segundos bissextos. Segundos bissextos são excluídos desse total em todas as plataformas compatíveis com POSIX.
- As funções desse módulo podem não conseguir tratar de datas e horários antes do *epoch* ou muito distantes no futuro. O limite no futuro é determinado pela biblioteca C; para sistemas de 32 bit, geralmente é 2038.
- A função *strptime()* pode analisar anos de 2 dígitos quando é passado o código de formato `%y`. Quando anos de 2 dígitos são analisados, eles são convertidos de acordo com os padrões POSIX e ISO C: valores 69–99 são mapeados para 1969–1999, e valores 0–68 são mapeados para 2000–2068.
- UTC é Coordinated Universal Time (em português, Horário Universal Coordenado) e substituiu o GMT ou Greenwich Mean Time (em português, Horário Médio de Greenwich) como base da cronometragem internacional. A sigla UTC não é um erro, mas está em conformidade com um esquema de nomenclatura anterior, independente de idioma, para padrões de tempo como UT0, UT1 e UT2.
- DST é Daylight Saving Time (Horário de Verão), um ajuste de fuso horário por (normalmente) uma hora durante parte do ano. As regras de Horário de Verão são mágicas (determinadas por leis locais) e podem mudar de ano a ano. A biblioteca C possui uma tabela contendo as regras locais (normalmente lidas de um arquivo de sistema por flexibilidade) e nesse contexto é a única fonte de Conhecimento Verdadeiro.

- A precisão de várias funções em tempo real podem ser menores do que o que pode estar sugerido pelas unidades nas quais seu valor ou argumento estão expressos. Por exemplo, na maioria dos sistemas Unix, o relógio "conta" apenas 50 ou 100 vezes por segundo.
- Por outro lado, a precisão de *time()* e *sleep()* é melhor do que suas equivalentes Unix: tempos são expressos como números em ponto flutuante, *time()* retorna o tempo mais preciso disponível (utilizando `gettimeofday()` do Unix, quando disponível) e *sleep()* irá aceitar qualquer tempo como uma fração não zero (`select()` do Unix é utilizada para implementar isto, quando disponível).
- O valor de tempo conforme retornado pelas *gmtime()*, *localtime()*, e *strptime()*, e aceito pelas *asctime()*, *mktime()* e *strftime()*, é uma sequência de 9 inteiros. Os valores retornados das *gmtime()*, *localtime()*, e *strptime()* também oferecem nomes de atributo para campos individuais.

  Veja *struct_time* para a descrição desses objetos.

  Alterado na versão 3.3: O tipo *struct_time* foi estendido para prover os atributos *tm_gmtoff* e *tm_zone* quando a plataforma suporta os membros `struct tm` correspondentes

  Alterado na versão 3.6: Os atributos *tm_gmtoff* e *tm_zone* da classe *struct_time* estão disponíveis em todas as plataformas agora.
- Utilize as seguintes funções para converter entre representações de tempo:

De	Para	Utilize
segundos desde a era	*struct_time* em UTC	*gmtime()*
segundos desde a era	*struct_time* em tempo local	*localtime()*
struct_time em UTC	segundos desde a era	*calendar.timegm()*
struct_time em tempo local	segundos desde a era	*mktime()*

16.3.1 Funções

`time.`**`asctime`**`(`[*t*]`)`

Converte a tupla ou *struct_time* representando um tempo como retornado pela *gmtime()* ou *localtime()* para uma string com o seguinte formato: `'Sun Jun 20 23:21:05 1993'`. O campo dia contém dois caracteres e possui espaçamento se o dia é de apenas um dígito. Por exemplo, `'Wed Jun 9 04:26:40 1993'`.

Se *t* não é fornecido, o tempo atual como retornado por *localtime()* é utilizado. Informação de localidade não é utilizada por *asctime()*.

> **Nota**
>
> Diferentemente da função em C de mesmo nome, *asctime()* não adiciona uma nova linha em seguida.

`time.`**`pthread_getcpuclockid`**`(`*thread_id*`)`

Retorna o *clk_id* do relógio de tempo de CPU específico da thread para a *thread_id* especificada.

Utilize a *threading.get_ident()* ou o atributo *ident* dos objetos *threading.Thread* para obter um valor adequado para *thread_id*.

> **Aviso**
>
> Passando um *thread_id* inválido ou expirado pode resultar em um comportamento indefinido, como, por exemplo, falha de segmentação.

Disponibilidade: Unix

Veja a página man para *pthread_getcpuclockid(3)* para mais informações.

Adicionado na versão 3.7.

time.**clock_getres**(*clk_id*)

Retorna a resolução (precisão) do relógio *clk_id* especificado. Confira *Constantes de ID de Relógio* para uma lista de valores aceitos para *clk_id*

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.**clock_gettime**(*clk_id*) → *float*

Retorna o tempo to relógio *clk_id* especificado. Confira *Constantes de ID de Relógio* para uma lista de valores aceitos para *clk_id*.

Use `clock_gettime_ns()` para evitar perda de precisão causada pelo tipo `float`.

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.**clock_gettime_ns**(*clk_id*) → *int*

Semelhante à `clock_gettime()`, mas retorna o tempo em nanossegundos.

Disponibilidade: Unix.

Adicionado na versão 3.7.

time.**clock_settime**(*clk_id*, *time:* float)

Define o tempo do relógio *clk_id* especificado. Atualmente, `CLOCK_REALTIME` é o único valor aceito para *clk_id*.

Use `clock_settime_ns()` para evitar perda de precisão causada pelo tipo `float`.

Disponibilidade: Unix, not Android, not iOS.

Adicionado na versão 3.3.

time.**clock_settime_ns**(*clk_id*, *time:* int)

Semelhante à `clock_settime()`, mas define o tempo em nanossegundos.

Disponibilidade: Unix, not Android, not iOS.

Adicionado na versão 3.7.

time.**ctime**([*secs*])

Converte um tempo expresso em segundos desde o *epoch* para uma string de forma: `'Sun Jun 20 23:21:05 1993'` representando tempo local. O campo dia tem comprimento de dois caracteres e é deslocado por um espaço se o dia tem um único dígito. Por exemplo, `'Wed Jun 9 04:26:40 1993'`.

Se *secs* não é fornecido ou `None`, o tempo atual como retornado por `time()` é utilizado. `ctime(secs)` é equivalente a `asctime(localtime(secs))`. Informação de localidade não é utilizada por `ctime()`.

time.**get_clock_info**(*name*)

Obtém informação do relógio específico como um objeto espaço de nomes. Nomes de relógios suportados e as funções correspondentes para ler seus valores são:

- `'monotonic'`: `time.monotonic()`
- `'perf_counter'`: `time.perf_counter()`
- `'process_time'`: `time.process_time()`
- `'thread_time'`: `time.thread_time()`
- `'time'`: `time.time()`

O resultado tem os seguintes atributos:

- *adjustable*: `True` se o relógio pode ser alterado automaticamente (por exemplo, por um daemon NTP) ou manualmente por um administrador do sistema, `False` se contrário

- *implementation*: O nome da função C subjacente utilizada para obter o valor do relógio. Confira *Constantes de ID de Relógio* para valores possíveis.
- *monotonic*: `True` se o relógio não pode retornar a valores anteriores, backward, `False` contrário
- *resolution*: A resolução do relógio em segundos (*float*)

Adicionado na versão 3.3.

time.**gmtime**([*secs*])

Converte um tempo expresso em segundos desde o *epoch* para uma *struct_time* em UTC onde o sinalizador de horário de verão é sempre zero. Se *secs* não é fornecido ou *None*, o tempo atual como retornado por *time()* é utilizado. Frações de segundo são ignoradas. Veja acima para uma descrição do objeto *struct_time*. Veja *calendar.timegm()* para o inverso desta função.

time.**localtime**([*secs*])

Como *gmtime()*, mas converte para o tempo local. Se *secs* não é fornecido ou *None*, o tempo atual como retornado por *time()* é utilizado. O sinalizador de horário de verão é definido como `1` quando o Horário de verão for aplicável para o tempo fornecido.

localtime() pode levantar *OverflowError*, se o registro de data e hora estiver fora de valores suportados pelas funções `localtime()` or `gmtime()` da plataforma C, e *OSError* no caso de `localtime()` ou `gmtime()` falharem. É comum É comum que isso seja restrito a anos de 1970 a 2038.

time.**mktime**(*t*)

Esta é a função inversa de *localtime()*. Seu argumento é a *struct_time* ou uma tupla com 9 itens (sendo o sinalizador de horário de verão necessário; utilize `-1` como sinalizador de horário de verão quando este for desconhecido) que expressa o tempo em tempo *local*, não UTC. Retorna um número em ponto flutuante, para ter compatibilidade com *time()*. Se o valor de entrada não puder ser representado como um tempo válido, ou *OverflowError* ou *ValueError* serão levantadas (o que irá depender se o valor inválido é capturado pelo Python ou por bibliotecas C subjacentes). A data mais recente para qual um tempo pode ser gerado é dependente da plataforma.

time.**monotonic**() → *float*

Retorna o valor (em frações de segundos) de um relógio monotônico, i.e. um relógio que não pode voltar a valores anteriores. O relógio não é afetado por atualizações do relógio do sistema. O ponto de referência do valor retornado é indefinido, portanto apenas a diferença entre os resultados de duas chamadas é válida.

Relógio:

- No Windows, chama `QueryPerformanceCounter()` e `QueryPerformanceFrequency()`.
- No macOS, chama `mach_absolute_time()` e `mach_timebase_info()`.
- No HP-UX, chama `gethrtime()`.
- Chama `clock_gettime(CLOCK_HIGHRES)` se disponível.
- Do contrário, chama `clock_gettime(CLOCK_MONOTONIC)`.

Use *monotonic_ns()* para evitar perda de precisão causada pelo tipo *float*.

Adicionado na versão 3.3.

Alterado na versão 3.5: A função agora está sempre disponível e o relógio agora é o mesmo para todos os processos.

Alterado na versão 3.10: No macOS, o relógio agora é o mesmo para todos os processos.

time.**monotonic_ns**() → *int*

Semelhante à *monotonic()*, mas retorna tempo em nanossegundos.

Adicionado na versão 3.7.

time.**perf_counter**() → *float*

Retorna o valor (em frações de segundo) de um contador de desempenho, i.e. um relógio com a maior resolução disponível para medir uma duração curta. Inclui o tempo decorrido durante a pausa. O relógio é o mesmo para todos os processos. O ponto de referência é do valor retornado é indefinido, portanto apenas a diferença entre resultados de duas chamadas é válida.

No CPython, use o mesmo relógio que *time.monotonic()* e é um relógio monotônico, ou seja, um relógio que não pode retroceder.

Use *perf_counter_ns()* para evitar perda de precisão causada pelo tipo *float*.

Adicionado na versão 3.3.

Alterado na versão 3.10: No Windows, o relógio agora é o mesmo para todos os processos.

Alterado na versão 3.13: Usa o mesmo relógio que *time.monotonic()*.

time.**perf_counter_ns**() → *int*

Semelhante à *perf_counter()*, mas retorna o tempo em nanossegundos.

Adicionado na versão 3.7.

time.**process_time**() → *float*

Retorna o valor (em frações de segundo) da soma dos tempos do sistema e CPU de usuário do processo atual. Não inclui o tempo decorrido durante a pausa. É de todo o processo por definição. O ponto de referência do valor retornado é indefinido, então apenas a diferença dos resultados de duas chamadas é válida.

Use *process_time_ns()* para evitar perda de precisão causada pelo tipo *float*.

Adicionado na versão 3.3.

time.**process_time_ns**() → *int*

Semelhante à *process_time()*, mas retorna o tempo em nanossegundos.

Adicionado na versão 3.7.

time.**sleep**(*secs*)

Suspende a execução da thread de chamada por um determinado número de segundos. O argumento pode ser um número de ponto flutuante para indicar um tempo de pausa mais preciso.

Se a pausa for interrompida por um sinal e nenhuma exceção for levantada pelo manipulador de sinal, a pausa será reiniciada com um tempo limite recalculado.

O tempo de suspensão pode ser maior do que o solicitado por um valor arbitrário, devido ao agendamento de outras atividades no sistema.

Implementação no Windows

No Windows, se *secs* é zero, a thread renuncia ao restante de sua fatia de tempo para qualquer outra thread que esteja pronto para executar. Se nenhuma outra thread estiver pronta pra ser executada, a função retorna imediatamente, e a thread continua a execução. No Windows 8.1 e mais novos a implementação usa um temporizador de alta resolução que provê uma resolução de 100 nanossegundos. Se *secs* é zero, é usado `Sleep(0)`.

Implementação no Unix

- Usa `clock_nanosleep()` se disponível (resolução: 1 nanossegundo);
- Ou usa `nanosleep()` se disponível (resolução: 1 nanossegunda);
- Ou usa `select()` (resolução: 1 microssegundo).

Nota

Para emular um "no-op", use `pass` em vez de `time.sleep(0)`.

Para abrir mão voluntariamente da CPU, especifique uma *política de agendamento* em tempo real e use `os.sched_yield()` em seu lugar.

Levanta um *evento de auditoria* `time.sleep` com o argumento `secs`.

Alterado na versão 3.5: A função agora pausa por pelo menos *secs* mesmo se a pausa é interrompido por um sinal, exceto se o tratador de sinal levanta uma exceção (veja **PEP 475** para a explicação).

Alterado na versão 3.11: No Unix, as funções `clock_nanosleep()` e `nanosleep()` são agora usadas se estiverem disponíveis. No Windows, um temporizador aguardável é agora usado.

Alterado na versão 3.13: Levanta um evento de auditoria.

`time.`**`strftime`**(*format*[, *t*])

Converte a tupla ou `struct_time` representando um tempo como retornado por `gmtime()` ou `localtime()` para uma string como especificado pelo argumento *format*. Se *t* não é fornecido, o tempo atual como retornado pela `localtime()` é utilizado. *format* deve ser uma string. A exceção `ValueError` é levantada se qualquer campo em *t* está fora do intervalo permitido.

0 é um argumento legal para qualquer posição na tupla de tempo; se é normalmente ilegal, o valor é formado a um valor correto.

As diretivas a seguir podem ser incorporadas na string *format*. Elas estão mostradas sem os campos de comprimento e especificação de precisão opcionais, e estão substituídos pelos caracteres indicados no resultado da `strftime()`:

Diretiva	Significado	Notas
%a	Nome abreviado do dia da semana da localidade.	
%A	Nome completo do dia da semana da localidade.	
%b	Nome abreviado do mês da localidade.	
%B	Nome completo do mês da localidade.	
%c	Representação de data e hora apropriada da localidade.	
%d	Dia do mês como um número decimal [01,31].	
%f	**Microssegundos como um número decimal** [000000,999999].	(1)
%H	Hora (relógio 24 horas) como um número decimal [00,23].	
%I	Hora (relógio 12 horas) como um número decimal [01,12].	
%j	Dia do ano como um número decimal [001,366].	
%m	Mês como um número decimal [01,12].	
%M	Minuto como um número decimal [00,59].	
%p	Equivalente da localidade a AM ou PM.	(2)
%S	Segundo como um número decimal [00,61].	(3)
%U	Número da semana do ano (domingo como primeiro dia da semana) como um número decimal [00,53]. Todos os dias em um ano novo que precedem o primeiro domingo são considerados como estando na semana 0.	(4)
%u	Dia da semana (segunda-feira é 1; domingo é 7) como um número decimal [1, 7].	
%w	Dia da semana como um número decimal [0(Domingo),6]	
%W	Número da semana do ano (segunda-feira como o primeiro dia da semana) como um número decimal [00,53]. Todos os dias do ano que precedem o primeiro domingo serão considerados como estando na semana 0.	(4)
%x	Representação de data apropriada de localidade.	
%X	Representação de hora apropriada da localidade.	
%y	Ano sem século como um número decimal [00,99].	
%Y	Ano com século como um número decimal.	
%z	Deslocamento de fuso horário indicando uma diferença de tempo positiva ou negativa de UTC/GMT formatado como	

Notas:

(1) A diretiva de formatação `%f` é somente aplicada para *`strptime()`*, não para *`strftime()`*. Contudo, veja também *`datetime.datetime.strptime()`* e *`datetime.datetime.strftime()`* onde a diretiva de formatação `%f` *é aplicada para microssegundos*.

(2) Quando utilizado com a função *`strptime()`*, a diretiva `%p` apenas afeta a saída do campo se a diretiva `%I` é utilizada para analisar a hora.

(3) O intervalo é realmente `0` até `61`; o valor `60` é válido em registros de data e hora representando segundos bissextos e o valor `61` é suportado por razões históricas.

(4) Quando utilizado co a função *`strptime()`*, `%U` e `%W` são utilizados em cálculos apenas quando o dia da semana e ano são especificados.

Veja este exemplo, um formato para datas compatível com as especificações dos padrões de e-mail **RFC 2822**.[1]

```
>>> from time import gmtime, strftime
>>> strftime("%a, %d %b %Y %H:%M:%S +0000", gmtime())
'Thu, 28 Jun 2001 14:17:15 +0000'
```

Diretivas adicionais podem ser suportadas por algumas plataformas, mas apenas as listadas aqui possuem significado padronizado por ANSI C. Para ver a lista completa de códigos de formato suportados na sua plataforma, consulte a documentação *`strftime(3)`*.

Em algumas plataformas, um campo adicional de comprimento e especificação de precisão podem seguir imediatamente após `'%'` como uma diretiva da seguinte ordem; isto também não é portátil. O campo comprimento normalmente é 2 exceto para `%j` quando é 3.

`time.`**`strptime`**(*string*[, *format*])

Analisa a string representando um tempo de acordo com um formato. O valor retornado é um *`struct_time`* como retornado por *`gmtime()`* ou *`localtime()`*.

O parâmetro *format* utiliza as mesmas diretivas das utilizadas por *`strftime()`*; é definido por padrão para `"%a %b %d %H:%M:%S %Y"` que corresponde com a formatação retornada por *`ctime()`*. Se *string* não puder ser analisada de acordo com *format*, ou se possui excesso de dados após analisar, *`ValueError`* é levantado. Os valores padrão utilizados para preencher quaisquer dados faltantes quando valores mais precisos não puderem ser inferidos são `(1900, 1, 1, 0, 0, 0, 0, 1, -1)`. Ambos *string* e *format* devem ser strings.

Por exemplo:

```
>>> import time
>>> time.strptime("30 Nov 00", "%d %b %y")
time.struct_time(tm_year=2000, tm_mon=11, tm_mday=30, tm_hour=0, tm_min=0,
                 tm_sec=0, tm_wday=3, tm_yday=335, tm_isdst=-1)
```

Suporte para a diretiva `%Z` é baseado nos valores contidos em `tzname` e se `daylight` é verdade. Por causa disso, é específico de plataforma exceto por reconhecer UTC e GMT, que são sempre conhecidos (e considerados fuso horários sem horários de verão).

Apenas as diretivas especificadas na documentação são suportadas. Como `strftime()` é implementada por plataforma, esta pode apenas as vezes oferecer mais diretivas do que as listadas aqui. Mas `strptime()` é independente de quaisquer plataformas e portanto não necessariamente suporta todas as diretivas disponíveis que não estão documentadas como suportadas.

`class` `time.`**`struct_time`**

O tipo da sequência de valor de tempo retornado por *`gmtime()`*, *`localtime()`*, e *`strptime()`*. É um objeto com uma interface *named tuple*: os valores podem ser acessados por um índice e por um nome de atributo. Os seguintes valores estão presentes:

[1] A utilização de `%Z` está descontinuada, mas o escape `%z` que expande para um deslocamento hora/minuto preferido não é suportado por todas as bibliotecas ANSI C. Além disso, a leitura do padrão original **RFC 822** de 1982 mostra que este pede um ano com dois dígitos (`%y` em vez de `%Y`), mas a prática consolidou a utilização de anos com 4 dígitos mesmo antes dos anos 2000. Após isso, o **RFC 822** tornou-se obsoleto e o ano de 4 dígitos foi primeiro recomendado por **RFC 1123** e depois obrigatório por **RFC 2822**.

Índice	Atributo	Valores
0	**tm_year**	(por exemplo, 1993)
1	**tm_mon**	intervalo [1,12]
2	**tm_mday**	intervalo [1,31]
3	**tm_hour**	intervalo [0,23]
4	**tm_min**	intervalo [0,59]
5	**tm_sec**	intervalo [0, 61]; veja *Nota (2)* em `strftime()`
6	**tm_wday**	intervalo [0, 6]; segunda-feira é 0
7	**tm_yday**	intervalo [1, 366]
8	**tm_isdst**	0, 1 ou -1; veja abaixo
N/D	**tm_zone**	abreviação do nome do fuso horário
N/D	**tm_gmtoff**	deslocamento a leste de UTC em segundos

Note que diferentemente da estrutura C, o valor do mês é um intervalo [1,12] e não [0,11].

Em chamadas para `mktime()`, `tm_isdst` pode ser definido como 1 quando o horário de verão estiver em efeito, e 0 quando não. Um valor de -1 indica que esta informação não é conhecida, e geralmente resultará no preenchimento do estado correto.

Quando uma tupla com comprimento incorreto é passada para uma função que espera por um `struct_time`, ou por possuir elementos do tipo errado, um `TypeError` é levantado.

time.**time**() → *float*

Retorna o tempo em segundos desde o *epoch* como um número em ponto flutuante. A manipulação de segundos bissextos é dependente da plataforma. No Windows e na maioria dos sistemas Unix, o segundo bissexto não é contado para o tempo desde o *epoch*. Isso é comumente referido como epoch ou era Unix.

Note que mesmo o tempo sendo retornado sempre como um número em ponto flutuante, nem todos os sistemas fornecem o tempo com precisão melhor que 1 segundo. Enquanto esta função normalmente retorna valores não decrescentes, pode retornar valores menores do que os de uma chamada anterior se o relógio do sistema foi redefinido entre duas chamadas.

O número retornado por `time()` pode ser convertido a um formato de tempo mais comum (i.e. ano, mês, dia, hora etc…) em UTC por passá-lo para a função `gmtime()` ou em tempo local por passar para a função `localtime()`. Em ambos os casos, o objeto `struct_time` é retornado, por onde os componentes de data do calendário podem ser acessados ou atribuídos.

Relógio:

- No Windows, chama `GetSystemTimePreciseAsFileTime()`.
- Chama `clock_gettime(CLOCK_REALTIME)` se disponível.
- Caso contrário, chama `gettimeofday()`.

Use *time_ns()* para evitar perda de precisão causada pelo tipo *float*.

Alterado na versão 3.13: No Windows, chama `GetSystemTimePreciseAsFileTime()` em vez de `GetSystemTimeAsFileTime()`.

`time.`**`time_ns`**`()` → *int*

Semelhante à *time()*, mas retorna o tempo como um número inteiro de nanossegundos desde o *epoch*.

Adicionado na versão 3.7.

`time.`**`thread_time`**`()` → *float*

Retorna o valor (em fração de segundos) da soma dos tempos de sistema e CPU de usuário para a thread atual. Não inclui o tempo decorrido durante a pausa. É específico a thread por definição. O ponto de referência do valor retornado é indefinido, então apenas a diferença dos resultados de duas chamadas é válida.

Use *thread_time_ns()* para evitar perda de precisão causada pelo tipo *float*.

Disponibilidade: Linux, Unix, Windows.

Sistemas Unix possuem suporte para `CLOCK_THREAD_CPUTIME_ID`.

Adicionado na versão 3.7.

`time.`**`thread_time_ns`**`()` → *int*

Semelhante à *thread_time()*, mas retorna o tempo em nanossegundos.

Adicionado na versão 3.7.

`time.`**`tzset`**`()`

Redefine as regras de conversão utilizadas pelas rotinas da biblioteca. A variável de ambiente `TZ` especifica como isto é feito. Também irá redefinir as variáveis `tzname` (da variável de ambiente `TZ`), `timezone` (segundos sem horário de verão a oeste de UTC), `altzone` (segundos com horário de verão a oeste de UTC) e `daylight` (para 0 se este fuso horário não possui nenhuma regra de horário de verão, ou diferente de zero se há um tempo, no presente, passado ou futuro quando o horário de verão se aplica).

Disponibilidade: Unix.

Nota

Embora em vários casos, alterar a variável de sistema `TZ` pode afetar a saída de funções como *localtime()* sem chamar *tzset()*, este comportamento não deve ser confiado.

A variável de sistema `TZ` não deve conter espaços em branco.

O formato padrão da variável de sistema `TZ` é (espaços foram adicionados por motivos de clareza):

```
std offset [dst [offset [,start[/time], end[/time]]]]
```

Onde os componentes são:

`std` e `dst`
: Três ou mais alfanuméricos fornecendo a abreviação do fuso horário. Estes serão propagados para time.tzname

`offset`
: O deslocamento tem a forma: `± hh[:mm[:ss]]`. Isso indica que o valor adicionado adicionou o horário local para chegar a UTC. Se precedido por um '-', o fuso horário está a leste do Meridiano Primário; do contrário, está a oeste. Se nenhum deslocamento segue o horário de verão, o tempo no verão é presumido como estando uma hora a frente do horário padrão.

start[/time], end[/time]
Indica quando mudar e voltar do Horário de Verão. O formato das datas de início e fim é um dos seguintes:

Jn
I dia juliano *n* (1 <= *n* <= 365). Os dias bissextos não são contados, então, em todos os anos, 28 de fevereiro é o dia 59 e 1 de março é o dia 60.

n
O dia juliano baseado em zero (0 <= *n* <= 365). Dias bissextos são contados, e é possível fazer referência a 29 de fevereiro.

Mm.n.d
O *d*-ésimo dia (0 <= *d* <= 6) da semana *n* do mês *m* do ano (1 <= *n* <= 5, 1 <= *m* <= 12, onde semana 5 significa "o último dia *d* no mês *m*" que pode ocorrer tanto na quarta como quinta semana). Semana 1 é a primeira semana na qual o *d*-ésimo dia ocorre. Dia zero é o domingo.

`time` tem o mesmo formato que `offset`, exceto que nenhum sinal no início é permitido ('-' ou '+'). O padrão, se o tempo não é dado, é 02:00:00.

```
>>> os.environ['TZ'] = 'EST+05EDT,M4.1.0,M10.5.0'
>>> time.tzset()
>>> time.strftime('%X %x %Z')
'02:07:36 05/08/03 EDT'
>>> os.environ['TZ'] = 'AEST-10AEDT-11,M10.5.0,M3.5.0'
>>> time.tzset()
>>> time.strftime('%X %x %Z')
'16:08:12 05/08/03 AEST'
```

Em muitos sistemas Unix (incluindo *BSD, Linux, Solaris, e Darwin), é mais conveniente utilizar o banco de dados de informação de fuso do sistema (*tzfile(5)*) para especificar as regras de fuso horário. Para fazer isso, defina a variável de sistema `TZ` ao path do arquivo de dados requerido de fuso horários, relativo à raiz do banco de dados de fuso horário 'zoneinfo' do sistema, geralmente encontrado em `/usr/share/zoneinfo`. Por exemplo, `'US/Eastern'`, `'Australia/Melbourne'`, `'Egypt'` ou `'Europe/Amsterdam'`.

```
>>> os.environ['TZ'] = 'US/Eastern'
>>> time.tzset()
>>> time.tzname
('EST', 'EDT')
>>> os.environ['TZ'] = 'Egypt'
>>> time.tzset()
>>> time.tzname
('EET', 'EEST')
```

16.3.2 Constantes de ID de Relógio

Essas constantes são utilizadas como parâmetros para *clock_getres()* e *clock_gettime()*.

`time.`**CLOCK_BOOTTIME**
Idêntica a *CLOCK_MONOTONIC*, exceto por também incluir qualquer tempo que o sistema está suspenso.

Isto permite que aplicações recebam um relógio monotônico consciente suspenso sem precisar lidar com as complicações de *CLOCK_REALTIME*, que pode conter descontinuidades se o tempo é alterado utilizando `settimeofday()` ou algo semelhante.

Disponibilidade: Linux >= 2.6.39.

Adicionado na versão 3.7.

`time.`**CLOCK_HIGHRES**
O Solaris OS possui um timer `CLOCK_HIGHRES` que tenta utilizar recursos otimizados do hardware, e pode fornecer resolução perto de nanossegundos. `CLOCK_HIGHRES` é o relógio nanoajustável de alta resolução.

Disponibilidade: Solaris.

Adicionado na versão 3.3.

time.**CLOCK_MONOTONIC**

Relógio que não pode ser definido e representa um tempo monotônico desde um ponto de início não especificado.

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.**CLOCK_MONOTONIC_RAW**

Semelhante à *CLOCK_MONOTONIC*, mas fornece acesso a um tempo bruto baseado em hardware que não está sujeito a ajustes NTP.

Disponibilidade: Linux >= 2.6.28, macOS >= 10.12.

Adicionado na versão 3.3.

time.**CLOCK_MONOTONIC_RAW_APPROX**

Semelhante a *CLOCK_MONOTONIC_RAW*, mas lê um armazenado pelo sistema no gerenciador de contexto e, portanto, tem menor precisão.

Disponibilidade: macOS >= 10.12.

Adicionado na versão 3.13.

time.**CLOCK_PROCESS_CPUTIME_ID**

Timer de alta resolução por processo no CPU.

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.**CLOCK_PROF**

Timer de alta resolução por processo no CPU.

Disponibilidade: FreeBSD, NetBSD >= 7, OpenBSD.

Adicionado na versão 3.7.

time.**CLOCK_TAI**

Tempo Atômico Internacional

O sistema deve ter uma tabela de segundos bissextos para que possa fornecer a resposta correta. Softwares PTP ou NTP podem manter uma tabela de segundos bissextos.

Disponibilidade: Linux.

Adicionado na versão 3.9.

time.**CLOCK_THREAD_CPUTIME_ID**

Relógio de tempo de CPU específico a thread.

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.**CLOCK_UPTIME**

Tempo cujo valor absoluto é o tempo que o sistema está sendo executado e não suspenso, fornecendo medidas de tempo de atividade precisas, tanto em valor absoluto quanto intervalo.

Disponibilidade: FreeBSD, OpenBSD >= 5.5.

Adicionado na versão 3.7.

time.**CLOCK_UPTIME_RAW**

Relógio que incrementa de forma monotônica, contando o tempo desde um ponto arbitrário, não afetado pela frequência ou ajustes de tempo e não incrementado enquanto o sistema está dormindo.

Disponibilidade: macOS >= 10.12.

Adicionado na versão 3.8.

time.**CLOCK_UPTIME_RAW_APPROX**

Semelhante a *CLOCK_UPTIME_RAW*, mas o valor é armazenado pelo sistema no gerenciador de contexto e, portanto, tem menor precisão.

Disponibilidade: macOS >= 10.12.

Adicionado na versão 3.13.

A constante a seguir é o único parâmetro que pode ser enviado para *clock_settime()*.

time.**CLOCK_REALTIME**

Relógio em tempo real. Configurar este relógio requer privilégios apropriados. O relógio é o mesmo para todos os processos.

Disponibilidade: Unix.

Adicionado na versão 3.3.

16.3.3 Constantes de Fuso Horário

time.**altzone**

O deslocamento do fuso horário DST local, em segundos a oeste de UTC, se algum for fornecido. É negativo se o fuso horário DST local está a leste de UTC (como na Europa Ocidental, incluindo o Reino Unido). Somente utilize se `daylight` for diferente de zero. Veja a nota abaixo.

time.**daylight**

Diferente de zero se um fuso horário DST é definido. Veja nota abaixo.

time.**timezone**

O deslocamento para o fuso horário local (não DST), em segundos a oeste de UTC (negativo na maior parte da Europa Ocidental, positivo nos Estados Unidos e Brasil, zero no Reino Unido). Ver nota abaixo.

time.**tzname**

A tupla de duas strings: A primeira é o nome do fuso horário local não DST, a segunda é o nome do fuso horário local DST. Se nenhum fuso horário DST for definido, a segunda string é usada. Veja nota abaixo.

Nota

Para as constantes de Fuso Horário acima (*altzone*, *daylight*, *timezone*, e *tzname*), o valor é determinado pelas regras de fuso horário em efeito no módulo de carregamento de tempo ou a última vez que *tzset()* é chamada e pode estar incorreto para tempos no passado. É recomendado utilizar os resultados *tm_gmtoff* e *tm_zone* da *localtime()* para obter informação de fuso horário.

Ver também

Módulo *datetime*
Mais interfaces orientada a objetos para datas e tempos.

Módulo *locale*
Serviços de internacionalização. A configuração de localidade afeta a interpretação de muitos especificadores de formato em *strftime()* e *strptime()*.

Módulo *calendar*
Funções gerais relacionadas a calendários. `timegm()` é a função inversa de `gmtime()` deste módulo.

16.4 `logging` — Recursos de registro de eventos para Python

Código-fonte: Lib/logging/__init__.py

Important

Esta página contêm informação de referência da API. Para tutorial e discussão de tópicos mais avançados, consulte

- Tutorial básico
- Tutorial avançado
- Livro de receitas de logging

Este módulo define funções e classes que implementam um registro flexível de eventos de sistema para aplicações e bibliotecas.

O principal benefício de ter a API de registro de eventos a partir de um módulo da biblioteca padrão é que todos os módulos Python podem participar no registro de eventos, de forma que sua aplicação pode incluir suas próprias mensagens, integradas com mensagens de módulos de terceiros.

Aqui está um exemplo simples de uso idiomático:

```
# myapp.py
import logging
import mylib
logger = logging.getLogger(__name__)

def main():
    logging.basicConfig(filename='myapp.log', level=logging.INFO)
    logger.info('Started')
    mylib.do_something()
    logger.info('Finished')

if __name__ == '__main__':
    main()
```

```
# mylib.py
import logging
logger = logging.getLogger(__name__)

def do_something():
    logger.info('Doing something')
```

Se você executar *myapp.py*, deverá ver isso em *myapp.log*:

```
INFO:__main__:Started
INFO:mylib:Doing something
INFO:__main__:Finished
```

O ponto chave desse modo de uso idiomático é que a maior parte do código está simplesmente criando um registrador de nível de módulo com `getLogger(__name__)` e usando esse registrador para fazer qualquer registro necessário. Isso é conciso e, ao mesmo tempo, permite o controle refinado do código subjacente, se necessário. As mensagens

registradas pelo registrador de nível de módulo são encaminhadas para tratadores de registradores em nível de módulo mais alto, até o registrador de nível mais alto conhecido como registrador raiz; essa abordagem é conhecida como registro hierárquico.

Para que o registro de eventos seja útil, ele precisa ser configurado: definir os níveis e os destinos de cada registrador, possivelmente alterando a forma como módulos específico fazem seus registros, geralmente com base em argumentos de linha de comando ou configuração da aplicação. Na maioria dos casos, como o descrito acima, somente o registrador raiz precisa ser configurado dessa forma, já que todos os registradores de nível inferior no nível módulo acabam encaminhando suas mensagens para seus tratadores. *basicConfig()* oferece uma maneira rápida de configurar o registrador raiz que lida com muitos casos de uso.

O módulo provê várias funcionalidades e flexibilidade. Se você não está familiarizado com registro de eventos, a melhor maneira para se ter uma noção sobre é ver os tutoriais (**veja os links acima e à direita**).

As classes básicas definidas pelo módulo, juntamente com seus atributos e métodos, estão listadas nas seções abaixo.

- Loggers expõem a interface que o código da aplicação usa diretamente.
- Handlers enviam os registros do evento (criados por loggers) aos destinos apropriados.
- Filters fornecem uma facilidade granular para determinar quais registros de eventos enviar à saída.
- Formatters especificam o layout dos registros de eventos na saída final.

16.4.1 Objetos Logger

Registradores têm os atributos e métodos a seguir. Observem que registradores *NUNCA* devem ser instanciados diretamente, mas sempre através da função `logging.getLogger(name)`. Múltiplas chamadas à função *getLogger()* com o mesmo nome sempre retornará uma referência para o mesmo objeto Logger.

The `name` is potentially a period-separated hierarchical value, like `foo.bar.baz` (though it could also be just plain `foo`, for example). Loggers that are further down in the hierarchical list are children of loggers higher up in the list. For example, given a logger with a name of `foo`, loggers with names of `foo.bar`, `foo.bar.baz`, and `foo.bam` are all descendants of `foo`. In addition, all loggers are descendants of the root logger. The logger name hierarchy is analogous to the Python package hierarchy, and identical to it if you organise your loggers on a per-module basis using the recommended construction `logging.getLogger(__name__)`. That's because in a module, `__name__` is the module's name in the Python package namespace.

class `logging.`**Logger**

name

This is the logger's name, and is the value that was passed to *getLogger()* to obtain the logger.

> **Nota**
>
> Este atributo deve ser tratado como somente leitura.

level

The threshold of this logger, as set by the *setLevel()* method.

> **Nota**
>
> Do not set this attribute directly - always use *setLevel()*, which has checks for the level passed to it.

parent

The parent logger of this logger. It may change based on later instantiation of loggers which are higher up in the namespace hierarchy.

> **Nota**
>
> Este valor deve ser tratado como somente leitura.

propagate

Se este atributo for verdadeiro, os eventos registrados nesse registrador serão transmitidos aos tratadores dos registradores de nível mais alto (ancestrais), além de quaisquer tratadores anexados a esse registrador. As mensagens são passadas diretamente para os tratadores dos registradores ancestrais - nem o nível nem os filtros dos registradores ancestrais em questão são considerados.

Se o valor for falso, as mensagens de registro não serão passadas para os tratadores de registradores ancestrais.

Explicando com um exemplo: se o atributo propagate do registrador chamado `A.B.C` for avaliado como verdadeiro, qualquer evento registrado em `A.B.C` pela chamada de métodos como `logging.getLogger('A.B.C').error(...)` será [sujeito à passagem das configurações de filtro e nível do registrador] passado por sua vez para qualquer tratador ligado aos registradores nomeados `A.B`, `A` e ao registrador raiz, após ser passado para qualquer tratador ligado ao `A.B.C`. Se qualquer registrador na cadeia `A.B.C`, `A.B`, `A` tem o atributo `propagate` definido como falso, então esse será o último registrador cujos tratadores terão os eventos oferecidos para serem tratados e a propagação terminará naquele ponto.

O construtor atribui este valor como `True`.

> **Nota**
>
> Se você configurar um tratador para o registrador *e* um ou mais dos ancestrais, pode acontecer que a mesma mensagem seja registrada várias vezes. Em geral, você não precisa configurar saídas para mais que um registrador - se você configurar apenas para o registrador principal da hierarquia, então todos os eventos dos registradores descendentes serão visualizados ali, fornecido pela configuração de propagação, cujo padrão é `True`. Um cenário comum é configurar as saídas somente no registrador raiz, e deixar a propagação tomar conta do resto.

handlers

The list of handlers directly attached to this logger instance.

> **Nota**
>
> This attribute should be treated as read-only; it is normally changed via the *addHandler()* and *removeHandler()* methods, which use locks to ensure thread-safe operation.

disabled

This attribute disables handling of any events. It is set to `False` in the initializer, and only changed by logging configuration code.

> **Nota**
>
> Este atributo deve ser tratado como somente leitura.

setLevel(*level*)

Ajuste o limite para este logger para *level*. Mensagens de registro de eventos que forem menos severas que este *level* serão ignoradas; mensagens que tenham nível de severidade igual ou maior que *level* serão emitidas para os destinos de saída configurados para o logger, a menos que o nível da saída tenha sido configurado para uma severidade ainda maior.

Quando um logger é criado, o nível é definido como *NOTSET* (que faz com que todas as mensagens sejam processadas quando o logger for o logger raiz, ou delegação para o pai quando o logger não for um logger raiz). Observe que o logger raiz é criado com nível *WARNING*.

O termo "delegação ao pai" significa que o logger possui um nível de NOTSET, e a sua cadeia de loggers ancestrais será percorrida até que um ancestral com o nível diferente de NOTSET seja encontrado ou até a raiz ser alcançada.

Se um ancestral for achado com um nível diferente de NOTSET, então o nível daquele ancestral será tratado como o nível efetivo do logger que começou a busca por ancestrais, e é usado para determinar como um registro de evento será manipulado.

Se a raiz é alcançada e o seu nível é NOTSET, então todas as mensagens serão processadas. Caso contrário, o nível da raiz será usada como o nível efetivo.

Veja *Níveis de Logging* para uma lista de níveis.

Alterado na versão 3.2: O parâmetro *level* agora é aceito como uma string representando o nível tal qual 'INFO', como uma alternativa as constantes inteiras tal qual *INFO*. Note, entretanto, que os níveis são guardados internamente como inteiros e os métodos como, por exemplo, *getEffectiveLevel()* e *isEnabledFor()* retornarão/esperarão que sejam passados inteiros.

isEnabledFor(*level*)

Indica se a mensagem com gravidade *level* seria processada por esse logger. Esse método checa primeiro o nível à nível de módulo definido por `logging.disable(level)` e então o nível efetivo do logger como determinado por *getEffectiveLevel()*.

getEffectiveLevel()

Indica o nível efeito para esse logger. Se um valor diferente de *NOTSET* foi definido usando *setLevel()*, ele é retornado. Caso contrário, a hierarquia é percorrida em direção a raiz até um valor diferente de *NOTSET* ser encontrado e então esse valor é retornado. O valor retornado é um inteiro, tipicamento um entre *logging.DEBUG*, *logging.INFO* etc

getChild(*suffix*)

Retorna um logger que é descendente deste logger, como determinado pelo sufixo. Portanto, `logging.getLogger('abc').getChild('def.ghi')` retornaria o mesmo logger que seria retornado por `logging.getLogger('abc.def.ghi')`. Esse é um método de conveniência, útil quando o logger pai é nomeado usando, por exemplo `__name__` ao invés de uma string literal.

Adicionado na versão 3.2.

getChildren()

Returns a set of loggers which are immediate children of this logger. So for example `logging.getLogger().getChildren()` might return a set containing loggers named `foo` and `bar`, but a logger named `foo.bar` wouldn't be included in the set. Likewise, `logging.getLogger('foo').getChildren()` might return a set including a logger named `foo.bar`, but it wouldn't include one named `foo.bar.baz`.

Adicionado na versão 3.12.

debug(*msg*, **args*, ***kwargs*)

Logs a message with level *DEBUG* on this logger. The *msg* is the message format string, and the *args* are the arguments which are merged into *msg* using the string formatting operator. (Note that this means that you can use keywords in the format string, together with a single dictionary argument.) No % formatting operation is performed on *msg* when no *args* are supplied.

Existem quatro argumentos nomeados em *kwargs* que serão inspecionados: *exc_info*, *stack_info*, *stacklevel* e *extra*.

If *exc_info* does not evaluate as false, it causes exception information to be added to the logging message. If an exception tuple (in the format returned by *sys.exc_info()*) or an exception instance is provided, it is used; otherwise, *sys.exc_info()* is called to get the exception information.

The second optional keyword argument is *stack_info*, which defaults to `False`. If true, stack information is added to the logging message, including the actual logging call. Note that this is not the same stack

information as that displayed through specifying *exc_info*: The former is stack frames from the bottom of the stack up to the logging call in the current thread, whereas the latter is information about stack frames which have been unwound, following an exception, while searching for exception handlers.

You can specify *stack_info* independently of *exc_info*, e.g. to just show how you got to a certain point in your code, even when no exceptions were raised. The stack frames are printed following a header line which says:

```
Stack (most recent call last):
```

Isso imita o `Traceback (most recent call last):` que é usado ao exibir quadros de exceção.

The third optional keyword argument is *stacklevel*, which defaults to `1`. If greater than 1, the corresponding number of stack frames are skipped when computing the line number and function name set in the *LogRecord* created for the logging event. This can be used in logging helpers so that the function name, filename and line number recorded are not the information for the helper function/method, but rather its caller. The name of this parameter mirrors the equivalent one in the *warnings* module.

The fourth keyword argument is *extra* which can be used to pass a dictionary which is used to populate the `__dict__` of the *LogRecord* created for the logging event with user-defined attributes. These custom attributes can then be used as you like. For example, they could be incorporated into logged messages. For example:

```
FORMAT = '%(asctime)s %(clientip)-15s %(user)-8s %(message)s'
logging.basicConfig(format=FORMAT)
d = {'clientip': '192.168.0.1', 'user': 'fbloggs'}
logger = logging.getLogger('tcpserver')
logger.warning('Protocol problem: %s', 'connection reset', extra=d)
```

imprimiria algo como

```
2006-02-08 22:20:02,165 192.168.0.1 fbloggs  Protocol problem: connection␣
↪reset
```

The keys in the dictionary passed in *extra* should not clash with the keys used by the logging system. (See the section on *Atributos LogRecord* for more information on which keys are used by the logging system.)

If you choose to use these attributes in logged messages, you need to exercise some care. In the above example, for instance, the *Formatter* has been set up with a format string which expects 'clientip' and 'user' in the attribute dictionary of the *LogRecord*. If these are missing, the message will not be logged because a string formatting exception will occur. So in this case, you always need to pass the *extra* dictionary with these keys.

While this might be annoying, this feature is intended for use in specialized circumstances, such as multi-threaded servers where the same code executes in many contexts, and interesting conditions which arise are dependent on this context (such as remote client IP address and authenticated user name, in the above example). In such circumstances, it is likely that specialized *Formatter*s would be used with particular *Handler*s.

If no handler is attached to this logger (or any of its ancestors, taking into account the relevant *Logger.propagate* attributes), the message will be sent to the handler set on *lastResort*.

Alterado na versão 3.2: O parâmetro *stack_info* foi adicionado.

Alterado na versão 3.5: The *exc_info* parameter can now accept exception instances.

Alterado na versão 3.8: O parâmetro *stacklevel* foi adicionado.

info(*msg*, **args*, ***kwargs*)

Logs a message with level *INFO* on this logger. The arguments are interpreted as for *debug()*.

warning(*msg*, **args*, ***kwargs*)

Logs a message with level *WARNING* on this logger. The arguments are interpreted as for *debug()*.

> **Nota**
>
> There is an obsolete method `warn` which is functionally identical to `warning`. As `warn` is deprecated, please do not use it - use `warning` instead.

error(*msg*, **args*, ***kwargs*)

Logs a message with level `ERROR` on this logger. The arguments are interpreted as for `debug()`.

critical(*msg*, **args*, ***kwargs*)

Logs a message with level `CRITICAL` on this logger. The arguments are interpreted as for `debug()`.

log(*level*, *msg*, **args*, ***kwargs*)

Logs a message with integer level *level* on this logger. The other arguments are interpreted as for `debug()`.

exception(*msg*, **args*, ***kwargs*)

Logs a message with level `ERROR` on this logger. The arguments are interpreted as for `debug()`. Exception info is added to the logging message. This method should only be called from an exception handler.

addFilter(*filter*)

Adds the specified filter *filter* to this logger.

removeFilter(*filter*)

Removes the specified filter *filter* from this logger.

filter(*record*)

Apply this logger's filters to the record and return `True` if the record is to be processed. The filters are consulted in turn, until one of them returns a false value. If none of them return a false value, the record will be processed (passed to handlers). If one returns a false value, no further processing of the record occurs.

addHandler(*hdlr*)

Adiciona o tratador especificado por *hdlr* deste logger.

removeHandler(*hdlr*)

Remove o tratador especificado por *hdlr* deste logger.

findCaller(*stack_info=False*, *stacklevel=1*)

Finds the caller's source filename and line number. Returns the filename, line number, function name and stack information as a 4-element tuple. The stack information is returned as `None` unless *stack_info* is `True`.

The *stacklevel* parameter is passed from code calling the `debug()` and other APIs. If greater than 1, the excess is used to skip stack frames before determining the values to be returned. This will generally be useful when calling logging APIs from helper/wrapper code, so that the information in the event log refers not to the helper/wrapper code, but to the code that calls it.

handle(*record*)

Handles a record by passing it to all handlers associated with this logger and its ancestors (until a false value of *propagate* is found). This method is used for unpickled records received from a socket, as well as those created locally. Logger-level filtering is applied using `filter()`.

makeRecord(*name*, *level*, *fn*, *lno*, *msg*, *args*, *exc_info*, *func=None*, *extra=None*, *sinfo=None*)

This is a factory method which can be overridden in subclasses to create specialized `LogRecord` instances.

hasHandlers()

Checks to see if this logger has any handlers configured. This is done by looking for handlers in this logger and its parents in the logger hierarchy. Returns `True` if a handler was found, else `False`. The

method stops searching up the hierarchy whenever a logger with the 'propagate' attribute set to false is found - that will be the last logger which is checked for the existence of handlers.

Adicionado na versão 3.2.

Alterado na versão 3.7: Loggers can now be pickled and unpickled.

16.4.2 Níveis de Logging

Os valores numéricos dos níveis de logging estão listados na tabela abaixo. Eles são principalmente de interesse se você quiser definir seus próprios níveis, e precisa deles para definir seus valores específicos relativos aos níveis predefinidos. Se você define um nível com o mesmo valor numérico, ele sobrescreve o valor predefinido; o nome predefinido é perdido.

Nível	Valor numérico	What it means / When to use it
`logging.NOTSET`	0	When set on a logger, indicates that ancestor loggers are to be consulted to determine the effective level. If that still resolves to `NOTSET`, then all events are logged. When set on a handler, all events are handled.
`logging.DEBUG`	10	Detailed information, typically only of interest to a developer trying to diagnose a problem.
`logging.INFO`	20	Confirmação de que as coisas estão funcionando como esperado.
`logging.WARNING`	30	An indication that something unexpected happened, or that a problem might occur in the near future (e.g. 'disk space low'). The software is still working as expected.
`logging.ERROR`	40	Por conta de um problema mais grave, o software não conseguiu executar alguma função.
`logging.CRITICAL`	50	Um erro grave, indicando que o programa pode não conseguir continuar rodando.

16.4.3 Objetos Handler

Handlers have the following attributes and methods. Note that *Handler* is never instantiated directly; this class acts as a base for more useful subclasses. However, the `__init__()` method in subclasses needs to call *Handler.__init__()*.

class logging.**Handler**

__init__(*level=NOTSET*)

Initializes the *Handler* instance by setting its level, setting the list of filters to the empty list and creating a lock (using *createLock()*) for serializing access to an I/O mechanism.

createLock()

Initializes a thread lock which can be used to serialize access to underlying I/O functionality which may not be threadsafe.

acquire()

Acquires the thread lock created with *createLock()*.

release()

Releases the thread lock acquired with *acquire()*.

setLevel(*level*)

Sets the threshold for this handler to *level*. Logging messages which are less severe than *level* will be ignored. When a handler is created, the level is set to *NOTSET* (which causes all messages to be processed).

Veja *Níveis de Logging* para uma lista de níveis.

Alterado na versão 3.2: The *level* parameter now accepts a string representation of the level such as 'INFO' as an alternative to the integer constants such as *INFO*.

setFormatter(*fmt*)

Sets the formatter for this handler to *fmt*. The *fmt* argument must be a *Formatter* instance or `None`.

addFilter(*filter*)

Adds the specified filter *filter* to this handler.

removeFilter(*filter*)

Removes the specified filter *filter* from this handler.

filter(*record*)

Apply this handler's filters to the record and return `True` if the record is to be processed. The filters are consulted in turn, until one of them returns a false value. If none of them return a false value, the record will be emitted. If one returns a false value, the handler will not emit the record.

flush()

Ensure all logging output has been flushed. This version does nothing and is intended to be implemented by subclasses.

close()

Tidy up any resources used by the handler. This version does no output but removes the handler from an internal map of handlers, which is used for handler lookup by name.

Subclasses should ensure that this gets called from overridden *close()* methods.

handle(*record*)

Conditionally emits the specified logging record, depending on filters which may have been added to the handler. Wraps the actual emission of the record with acquisition/release of the I/O thread lock.

handleError(*record*)

This method should be called from handlers when an exception is encountered during an *emit()* call. If the module-level attribute *raiseExceptions* is `False`, exceptions get silently ignored. This is what is mostly wanted for a logging system - most users will not care about errors in the logging system, they are more interested in application errors. You could, however, replace this with a custom handler if you wish. The specified record is the one which was being processed when the exception occurred. (The default value of *raiseExceptions* is `True`, as that is more useful during development).

format(*record*)

Do formatting for a record - if a formatter is set, use it. Otherwise, use the default formatter for the module.

emit(*record*)

Do whatever it takes to actually log the specified logging record. This version is intended to be implemented by subclasses and so raises a *NotImplementedError*.

> **Aviso**
>
> This method is called after a handler-level lock is acquired, which is released after this method returns. When you override this method, note that you should be careful when calling anything that invokes

other parts of the logging API which might do locking, because that might result in a deadlock. Specifically:

- Logging configuration APIs acquire the module-level lock, and then individual handler-level locks as those handlers are configured.
- Many logging APIs lock the module-level lock. If such an API is called from this method, it could cause a deadlock if a configuration call is made on another thread, because that thread will try to acquire the module-level lock *before* the handler-level lock, whereas this thread tries to acquire the module-level lock *after* the handler-level lock (because in this method, the handler-level lock has already been acquired).

For a list of handlers included as standard, see *logging.handlers*.

16.4.4 Formatter Objects

class logging.**Formatter**(*fmt=None*, *datefmt=None*, *style='%'*, *validate=True*, ***, *defaults=None*)

Responsible for converting a *LogRecord* to an output string to be interpreted by a human or external system.

Parâmetros

- **fmt** (str) – A format string in the given *style* for the logged output as a whole. The possible mapping keys are drawn from the *LogRecord* object's *Atributos LogRecord*. If not specified, `'%(message)s'` is used, which is just the logged message.
- **datefmt** (str) – A format string for the date/time portion of the logged output. If not specified, the default described in *formatTime()* is used.
- **style** (str) – Can be one of `'%'`, `'{'` or `'$'` and determines how the format string will be merged with its data: using one of *Formatação de string no estilo printf* (`%`), *str.format()* (`{`) or *string.Template* (`$`). This only applies to *fmt* (e.g. `'%(message)s'` versus `'{message}'`), not to the actual log messages passed to the logging methods. However, there are other ways to use `{`- and `$`-formatting for log messages.
- **validate** (bool) – If `True` (the default), incorrect or mismatched *fmt* and *style* will raise a *ValueError*; for example, `logging.Formatter('%(asctime)s - %(message)s', style='{')`.
- **defaults** (dict[str, Any]) – A dictionary with default values to use in custom fields. For example, `logging.Formatter('%(ip)s %(message)s', defaults={"ip": None})`

Alterado na versão 3.2: Adicionado o parâmetro *style*.

Alterado na versão 3.8: Adicionado o parâmetro *validate*.

Alterado na versão 3.10: Adicionado o parâmetro *defaults*.

format(*record*)

The record's attribute dictionary is used as the operand to a string formatting operation. Returns the resulting string. Before formatting the dictionary, a couple of preparatory steps are carried out. The *message* attribute of the record is computed using *msg % args*. If the formatting string contains `'(asctime)'`, *formatTime()* is called to format the event time. If there is exception information, it is formatted using *formatException()* and appended to the message. Note that the formatted exception information is cached in attribute *exc_text*. This is useful because the exception information can be pickled and sent across the wire, but you should be careful if you have more than one *Formatter* subclass which customizes the formatting of exception information. In this case, you will have to clear the cached value (by setting the *exc_text* attribute to `None`) after a formatter has done its formatting, so that the next formatter to handle the event doesn't use the cached value, but recalculates it afresh.

If stack information is available, it's appended after the exception information, using *formatStack()* to transform it if necessary.

formatTime(*record*, *datefmt=None*)

This method should be called from *format()* by a formatter which wants to make use of a formatted time. This method can be overridden in formatters to provide for any specific requirement, but the basic behavior is as follows: if *datefmt* (a string) is specified, it is used with *time.strftime()* to format the creation time of the record. Otherwise, the format '%Y-%m-%d %H:%M:%S,uuu' is used, where the uuu part is a millisecond value and the other letters are as per the *time.strftime()* documentation. An example time in this format is `2003-01-23 00:29:50,411`. The resulting string is returned.

This function uses a user-configurable function to convert the creation time to a tuple. By default, *time.localtime()* is used; to change this for a particular formatter instance, set the `converter` attribute to a function with the same signature as *time.localtime()* or *time.gmtime()*. To change it for all formatters, for example if you want all logging times to be shown in GMT, set the `converter` attribute in the `Formatter` class.

Alterado na versão 3.3: Previously, the default format was hard-coded as in this example: `2010-09-06 22:38:15,292` where the part before the comma is handled by a strptime format string (`'%Y-%m-%d %H:%M:%S'`), and the part after the comma is a millisecond value. Because strptime does not have a format placeholder for milliseconds, the millisecond value is appended using another format string, `'%s,%03d'` — and both of these format strings have been hardcoded into this method. With the change, these strings are defined as class-level attributes which can be overridden at the instance level when desired. The names of the attributes are `default_time_format` (for the strptime format string) and `default_msec_format` (for appending the millisecond value).

Alterado na versão 3.9: The `default_msec_format` can be `None`.

formatException(*exc_info*)

Formats the specified exception information (a standard exception tuple as returned by *sys.exc_info()*) as a string. This default implementation just uses *traceback.print_exception()*. The resulting string is returned.

formatStack(*stack_info*)

Formats the specified stack information (a string as returned by *traceback.print_stack()*, but with the last newline removed) as a string. This default implementation just returns the input value.

class logging.**BufferingFormatter**(*linefmt=None*)

A base formatter class suitable for subclassing when you want to format a number of records. You can pass a *Formatter* instance which you want to use to format each line (that corresponds to a single record). If not specified, the default formatter (which just outputs the event message) is used as the line formatter.

formatHeader(*records*)

Return a header for a list of *records*. The base implementation just returns the empty string. You will need to override this method if you want specific behaviour, e.g. to show the count of records, a title or a separator line.

formatFooter(*records*)

Return a footer for a list of *records*. The base implementation just returns the empty string. You will need to override this method if you want specific behaviour, e.g. to show the count of records or a separator line.

format(*records*)

Return formatted text for a list of *records*. The base implementation just returns the empty string if there are no records; otherwise, it returns the concatenation of the header, each record formatted with the line formatter, and the footer.

16.4.5 Filter Objects

`Filters` can be used by `Handlers` and `Loggers` for more sophisticated filtering than is provided by levels. The base filter class only allows events which are below a certain point in the logger hierarchy. For example, a filter initialized with 'A.B' will allow events logged by loggers 'A.B', 'A.B.C', 'A.B.C.D', 'A.B.D' etc. but not 'A.BB', 'B.A.B' etc. If initialized with the empty string, all events are passed.

class logging.**Filter**(*name=''*)

Returns an instance of the *Filter* class. If *name* is specified, it names a logger which, together with its children, will have its events allowed through the filter. If *name* is the empty string, allows every event.

filter(*record*)

Is the specified record to be logged? Returns false for no, true for yes. Filters can either modify log records in-place or return a completely different record instance which will replace the original log record in any future processing of the event.

Note that filters attached to handlers are consulted before an event is emitted by the handler, whereas filters attached to loggers are consulted whenever an event is logged (using *debug()*, *info()*, etc.), before sending an event to handlers. This means that events which have been generated by descendant loggers will not be filtered by a logger's filter setting, unless the filter has also been applied to those descendant loggers.

You don't actually need to subclass `Filter`: you can pass any instance which has a `filter` method with the same semantics.

Alterado na versão 3.2: You don't need to create specialized `Filter` classes, or use other classes with a `filter` method: you can use a function (or other callable) as a filter. The filtering logic will check to see if the filter object has a `filter` attribute: if it does, it's assumed to be a `Filter` and its *filter()* method is called. Otherwise, it's assumed to be a callable and called with the record as the single parameter. The returned value should conform to that returned by *filter()*.

Alterado na versão 3.12: You can now return a *LogRecord* instance from filters to replace the log record rather than modifying it in place. This allows filters attached to a *Handler* to modify the log record before it is emitted, without having side effects on other handlers.

Although filters are used primarily to filter records based on more sophisticated criteria than levels, they get to see every record which is processed by the handler or logger they're attached to: this can be useful if you want to do things like counting how many records were processed by a particular logger or handler, or adding, changing or removing attributes in the *LogRecord* being processed. Obviously changing the LogRecord needs to be done with some care, but it does allow the injection of contextual information into logs (see filters-contextual).

16.4.6 LogRecord Objects

LogRecord instances are created automatically by the *Logger* every time something is logged, and can be created manually via *makeLogRecord()* (for example, from a pickled event received over the wire).

class logging.**LogRecord**(*name*, *level*, *pathname*, *lineno*, *msg*, *args*, *exc_info*, *func=None*, *sinfo=None*)

Contains all the information pertinent to the event being logged.

The primary information is passed in *msg* and *args*, which are combined using `msg % args` to create the `message` attribute of the record.

Parâmetros

- **name** (`str`) – The name of the logger used to log the event represented by this `LogRecord`. Note that the logger name in the `LogRecord` will always have this value, even though it may be emitted by a handler attached to a different (ancestor) logger.
- **level** (`int`) – The *numeric level* of the logging event (such as `10` for `DEBUG`, `20` for `INFO`, etc). Note that this is converted to *two* attributes of the LogRecord: `levelno` for the numeric value and `levelname` for the corresponding level name.
- **pathname** (`str`) – The full string path of the source file where the logging call was made.
- **lineno** (`int`) – The line number in the source file where the logging call was made.
- **msg** (`Any`) – The event description message, which can be a %-format string with placeholders for variable data, or an arbitrary object (see arbitrary-object-messages).
- **args** (`tuple | dict[str, Any]`) – Variable data to merge into the *msg* argument to obtain the event description.

- **exc_info** (*tuple[type[BaseException], BaseException, types.TracebackType] | None*) – An exception tuple with the current exception information, as returned by *sys.exc_info()*, or None if no exception information is available.
- **func** (*str | None*) – The name of the function or method from which the logging call was invoked.
- **sinfo** (*str | None*) – A text string representing stack information from the base of the stack in the current thread, up to the logging call.

getMessage()

Returns the message for this *LogRecord* instance after merging any user-supplied arguments with the message. If the user-supplied message argument to the logging call is not a string, *str()* is called on it to convert it to a string. This allows use of user-defined classes as messages, whose __str__ method can return the actual format string to be used.

Alterado na versão 3.2: The creation of a *LogRecord* has been made more configurable by providing a factory which is used to create the record. The factory can be set using *getLogRecordFactory()* and *setLogRecordFactory()* (see this for the factory's signature).

This functionality can be used to inject your own values into a *LogRecord* at creation time. You can use the following pattern:

```
old_factory = logging.getLogRecordFactory()

def record_factory(*args, **kwargs):
    record = old_factory(*args, **kwargs)
    record.custom_attribute = 0xdecafbad
    return record

logging.setLogRecordFactory(record_factory)
```

With this pattern, multiple factories could be chained, and as long as they don't overwrite each other's attributes or unintentionally overwrite the standard attributes listed above, there should be no surprises.

16.4.7 Atributos LogRecord

The LogRecord has a number of attributes, most of which are derived from the parameters to the constructor. (Note that the names do not always correspond exactly between the LogRecord constructor parameters and the LogRecord attributes.) These attributes can be used to merge data from the record into the format string. The following table lists (in alphabetical order) the attribute names, their meanings and the corresponding placeholder in a %-style format string.

If you are using {}-formatting (*str.format()*), you can use `{attrname}` as the placeholder in the format string. If you are using $-formatting (*string.Template*), use the form `${attrname}`. In both cases, of course, replace `attrname` with the actual attribute name you want to use.

In the case of {}-formatting, you can specify formatting flags by placing them after the attribute name, separated from it with a colon. For example: a placeholder of `{msecs:03.0f}` would format a millisecond value of `4` as `004`. Refer to the *str.format()* documentation for full details on the options available to you.

Attribute name	Formatação	Descrição
args	You shouldn't need to format this yourself.	The tuple of arguments merged into `msg` to produce `message`, or a dict whose values are used for the merge (when there is only one argument, and it is a dictionary).
asctime	`%(asctime)s`	Human-readable time when the *LogRecord* was created. By default this is of the form '2003-07-08 16:49:45,896' (the numbers after the comma are millisecond portion of the time).
created	`%(created)f`	Time when the *LogRecord* was created (as returned by *time.time_ns()* / 1e9).
exc_info	You shouldn't need to format this yourself.	Exception tuple (à la `sys.exc_info`) or, if no exception has occurred, `None`.
filename	`%(filename)s`	Filename portion of `pathname`.
funcName	`%(funcName)s`	Name of function containing the logging call.
levelname	`%(levelname)s`	Text logging level for the message (`'DEBUG'`, `'INFO'`, `'WARNING'`, `'ERROR'`, `'CRITICAL'`).
levelno	`%(levelno)s`	Numeric logging level for the message (*DEBUG*, *INFO*, *WARNING*, *ERROR*, *CRITICAL*).
lineno	`%(lineno)d`	Source line number where the logging call was issued (if available).
message	`%(message)s`	The logged message, computed as `msg % args`. This is set when *Formatter.format()* is invoked.
módulo	`%(module)s`	Module (name portion of `filename`).
msecs	`%(msecs)d`	Millisecond portion of the time when the *LogRecord* was created.
msg	You shouldn't need to format this yourself.	The format string passed in the original logging call. Merged with `args` to produce `message`, or an arbitrary object (see arbitrary-object-messages).
nome	`%(name)s`	Name of the logger used to log the call.
pathname	`%(pathname)s`	Full pathname of the source file where the logging call was issued (if available).
processo	`%(process)d`	Process ID (if available).
processName	`%(processName)`	Process name (if available).
relativeCreated	`%(relativeCrea`	Time in milliseconds when the LogRecord was created, relative to the time the logging module was loaded.
stack_info	You shouldn't need to format this yourself.	Stack frame information (where available) from the bottom of the stack in the current thread, up to and including the stack frame of the logging call which resulted in the creation of this record.
thread	`%(thread)d`	Thread ID (if available).
threadName	`%(threadName)s`	Thread name (if available).
taskName	`%(taskName)s`	*asyncio.Task* name (if available).

Alterado na versão 3.1: *processName* was added.

Alterado na versão 3.12: *taskName* was added.

16.4.8 LoggerAdapter Objects

LoggerAdapter instances are used to conveniently pass contextual information into logging calls. For a usage example, see the section on adding contextual information to your logging output.

class logging.**LoggerAdapter**(*logger*, *extra*, *merge_extra=False*)

Returns an instance of *LoggerAdapter* initialized with an underlying *Logger* instance, a dict-like object (*extra*), and a boolean (*merge_extra*) indicating whether or not the *extra* argument of individual log calls should be merged with the *LoggerAdapter* extra. The default behavior is to ignore the *extra* argument of individual log calls and only use the one of the *LoggerAdapter* instance

process(*msg*, *kwargs*)

Modifies the message and/or keyword arguments passed to a logging call in order to insert contextual information. This implementation takes the object passed as *extra* to the constructor and adds it to *kwargs* using key 'extra'. The return value is a (*msg*, *kwargs*) tuple which has the (possibly modified) versions of the arguments passed in.

manager

Delegates to the underlying `manager` on *logger*.

_log

Delegates to the underlying `_log()` method on *logger*.

In addition to the above, *LoggerAdapter* supports the following methods of *Logger*: *debug()*, *info()*, *warning()*, *error()*, *exception()*, *critical()*, *log()*, *isEnabledFor()*, *getEffectiveLevel()*, *setLevel()* and *hasHandlers()*. These methods have the same signatures as their counterparts in *Logger*, so you can use the two types of instances interchangeably.

Alterado na versão 3.2: The *isEnabledFor()*, *getEffectiveLevel()*, *setLevel()* and *hasHandlers()* methods were added to *LoggerAdapter*. These methods delegate to the underlying logger.

Alterado na versão 3.6: Attribute `manager` and method `_log()` were added, which delegate to the underlying logger and allow adapters to be nested.

Alterado na versão 3.13: The *merge_extra* argument was added.

16.4.9 Thread Safety

The logging module is intended to be thread-safe without any special work needing to be done by its clients. It achieves this through using threading locks; there is one lock to serialize access to the module's shared data, and each handler also creates a lock to serialize access to its underlying I/O.

If you are implementing asynchronous signal handlers using the *signal* module, you may not be able to use logging from within such handlers. This is because lock implementations in the *threading* module are not always re-entrant, and so cannot be invoked from such signal handlers.

16.4.10 Funções de Nível de Módulo

In addition to the classes described above, there are a number of module-level functions.

logging.**getLogger**(*name=None*)

Return a logger with the specified name or, if name is `None`, return the root logger of the hierarchy. If specified, the name is typically a dot-separated hierarchical name like *'a'*, *'a.b'* or *'a.b.c.d'*. Choice of these names is entirely up to the developer who is using logging, though it is recommended that `__name__` be used unless you have a specific reason for not doing that, as mentioned in *Objetos Logger*.

All calls to this function with a given name return the same logger instance. This means that logger instances never need to be passed between different parts of an application.

logging.**getLoggerClass**()

Return either the standard *Logger* class, or the last class passed to *setLoggerClass()*. This function may be called from within a new class definition, to ensure that installing a customized *Logger* class will not undo customizations already applied by other code. For example:

```
class MyLogger(logging.getLoggerClass()):
    # ... override behaviour here
```

logging.**getLogRecordFactory**()

Return a callable which is used to create a *LogRecord*.

Adicionado na versão 3.2: This function has been provided, along with *setLogRecordFactory()*, to allow developers more control over how the *LogRecord* representing a logging event is constructed.

See *setLogRecordFactory()* for more information about the how the factory is called.

logging.**debug**(*msg*, **args*, ***kwargs*)

This is a convenience function that calls *Logger.debug()*, on the root logger. The handling of the arguments is in every way identical to what is described in that method.

The only difference is that if the root logger has no handlers, then *basicConfig()* is called, prior to calling `debug` on the root logger.

For very short scripts or quick demonstrations of `logging` facilities, `debug` and the other module-level functions may be convenient. However, most programs will want to carefully and explicitly control the logging configuration, and should therefore prefer creating a module-level logger and calling *Logger.debug()* (or other level-specific methods) on it, as described at the beginning of this documentation.

logging.**info**(*msg*, **args*, ***kwargs*)

Logs a message with level *INFO* on the root logger. The arguments and behavior are otherwise the same as for *debug()*.

logging.**warning**(*msg*, **args*, ***kwargs*)

Logs a message with level *WARNING* on the root logger. The arguments and behavior are otherwise the same as for *debug()*.

> **Nota**
>
> There is an obsolete function `warn` which is functionally identical to `warning`. As `warn` is deprecated, please do not use it - use `warning` instead.

logging.**error**(*msg*, **args*, ***kwargs*)

Logs a message with level *ERROR* on the root logger. The arguments and behavior are otherwise the same as for *debug()*.

logging.**critical**(*msg*, **args*, ***kwargs*)

Logs a message with level *CRITICAL* on the root logger. The arguments and behavior are otherwise the same as for *debug()*.

logging.**exception**(*msg*, **args*, ***kwargs*)

Logs a message with level *ERROR* on the root logger. The arguments and behavior are otherwise the same as for *debug()*. Exception info is added to the logging message. This function should only be called from an exception handler.

logging.**log**(*level*, *msg*, **args*, ***kwargs*)

Logs a message with level *level* on the root logger. The arguments and behavior are otherwise the same as for *debug()*.

logging.**disable**(*level=CRITICAL*)

Provides an overriding level *level* for all loggers which takes precedence over the logger's own level. When the need arises to temporarily throttle logging output down across the whole application, this function can be useful. Its effect is to disable all logging calls of severity *level* and below, so that if you call it with a value of INFO, then all INFO and DEBUG events would be discarded, whereas those of severity WARNING and above would be processed according to the logger's effective level. If `logging.disable(logging.NOTSET)` is

called, it effectively removes this overriding level, so that logging output again depends on the effective levels of individual loggers.

Note that if you have defined any custom logging level higher than `CRITICAL` (this is not recommended), you won't be able to rely on the default value for the *level* parameter, but will have to explicitly supply a suitable value.

Alterado na versão 3.7: The *level* parameter was defaulted to level `CRITICAL`. See bpo-28524 for more information about this change.

`logging.`**`addLevelName`**(*level*, *levelName*)

Associates level *level* with text *levelName* in an internal dictionary, which is used to map numeric levels to a textual representation, for example when a *Formatter* formats a message. This function can also be used to define your own levels. The only constraints are that all levels used must be registered using this function, levels should be positive integers and they should increase in increasing order of severity.

> **Nota**
>
> If you are thinking of defining your own levels, please see the section on custom-levels.

`logging.`**`getLevelNamesMapping`**()

Returns a mapping from level names to their corresponding logging levels. For example, the string "CRITICAL" maps to *CRITICAL*. The returned mapping is copied from an internal mapping on each call to this function.

Adicionado na versão 3.11.

`logging.`**`getLevelName`**(*level*)

Returns the textual or numeric representation of logging level *level*.

If *level* is one of the predefined levels *CRITICAL*, *ERROR*, *WARNING*, *INFO* or *DEBUG* then you get the corresponding string. If you have associated levels with names using *addLevelName()* then the name you have associated with *level* is returned. If a numeric value corresponding to one of the defined levels is passed in, the corresponding string representation is returned.

The *level* parameter also accepts a string representation of the level such as 'INFO'. In such cases, this functions returns the corresponding numeric value of the level.

If no matching numeric or string value is passed in, the string 'Level %s' % level is returned.

> **Nota**
>
> Levels are internally integers (as they need to be compared in the logging logic). This function is used to convert between an integer level and the level name displayed in the formatted log output by means of the `%(levelname)s` format specifier (see *Atributos LogRecord*), and vice versa.

Alterado na versão 3.4: In Python versions earlier than 3.4, this function could also be passed a text level, and would return the corresponding numeric value of the level. This undocumented behaviour was considered a mistake, and was removed in Python 3.4, but reinstated in 3.4.2 due to retain backward compatibility.

`logging.`**`getHandlerByName`**(*name*)

Returns a handler with the specified *name*, or `None` if there is no handler with that name.

Adicionado na versão 3.12.

`logging.`**`getHandlerNames`**()

Returns an immutable set of all known handler names.

Adicionado na versão 3.12.

logging.**makeLogRecord**(*attrdict*)

Creates and returns a new *LogRecord* instance whose attributes are defined by *attrdict*. This function is useful for taking a pickled *LogRecord* attribute dictionary, sent over a socket, and reconstituting it as a *LogRecord* instance at the receiving end.

logging.**basicConfig**(***kwargs*)

Does basic configuration for the logging system by creating a *StreamHandler* with a default *Formatter* and adding it to the root logger. The functions *debug()*, *info()*, *warning()*, *error()* and *critical()* will call *basicConfig()* automatically if no handlers are defined for the root logger.

This function does nothing if the root logger already has handlers configured, unless the keyword argument *force* is set to `True`.

Nota

This function should be called from the main thread before other threads are started. In versions of Python prior to 2.7.1 and 3.2, if this function is called from multiple threads, it is possible (in rare circumstances) that a handler will be added to the root logger more than once, leading to unexpected results such as messages being duplicated in the log.

The following keyword arguments are supported.

Formatação	Descrição
filename	Specifies that a *FileHandler* be created, using the specified filename, rather than a *StreamHandler*.
filemode	If *filename* is specified, open the file in this *mode*. Defaults to `'a'`.
format	Use the specified format string for the handler. Defaults to attributes `levelname`, `name` and `message` separated by colons.
datefmt	Use the specified date/time format, as accepted by *time.strftime()*.
style	If *format* is specified, use this style for the format string. One of `'%'`, `'{'` or `'$'` for *printf-style*, *str.format()* or *string.Template* respectively. Defaults to `'%'`.
level	Set the root logger level to the specified *level*.
stream	Use the specified stream to initialize the *StreamHandler*. Note that this argument is incompatible with *filename* - if both are present, a `ValueError` is raised.
handlers	If specified, this should be an iterable of already created handlers to add to the root logger. Any handlers which don't already have a formatter set will be assigned the default formatter created in this function. Note that this argument is incompatible with *filename* or *stream* - if both are present, a `ValueError` is raised.
force	If this keyword argument is specified as true, any existing handlers attached to the root logger are removed and closed, before carrying out the configuration as specified by the other arguments.
encoding	If this keyword argument is specified along with *filename*, its value is used when the *FileHandler* is created, and thus used when opening the output file.
errors	If this keyword argument is specified along with *filename*, its value is used when the *FileHandler* is created, and thus used when opening the output file. If not specified, the value 'backslashreplace' is used. Note that if `None` is specified, it will be passed as such to *open()*, which means that it will be treated the same as passing 'errors'.

Alterado na versão 3.2: O argumento *style* foi adicionado.

Alterado na versão 3.3: The *handlers* argument was added. Additional checks were added to catch situations where incompatible arguments are specified (e.g. *handlers* together with *stream* or *filename*, or *stream* together with *filename*).

Alterado na versão 3.8: O argumento *force* foi adicionado.

Alterado na versão 3.9: Os argumentos *encoding* e *errors* foram adicionados.

logging.**shutdown**()

Informs the logging system to perform an orderly shutdown by flushing and closing all handlers. This should be called at application exit and no further use of the logging system should be made after this call.

When the logging module is imported, it registers this function as an exit handler (see *atexit*), so normally there's no need to do that manually.

logging.**setLoggerClass**(*klass*)

Tells the logging system to use the class *klass* when instantiating a logger. The class should define `__init__()` such that only a name argument is required, and the `__init__()` should call `Logger.__init__()`. This function is typically called before any loggers are instantiated by applications which need to use custom logger behavior. After this call, as at any other time, do not instantiate loggers directly using the subclass: continue to use the *logging.getLogger()* API to get your loggers.

logging.**setLogRecordFactory**(*factory*)

Set a callable which is used to create a *LogRecord*.

Parâmetros

factory – The factory callable to be used to instantiate a log record.

Adicionado na versão 3.2: This function has been provided, along with *getLogRecordFactory()*, to allow developers more control over how the *LogRecord* representing a logging event is constructed.

The factory has the following signature:

```
factory(name, level, fn, lno, msg, args, exc_info, func=None, sinfo=None,
**kwargs)
```

nome
The logger name.

level
The logging level (numeric).

fn
The full pathname of the file where the logging call was made.

lno
The line number in the file where the logging call was made.

msg
The logging message.

args
The arguments for the logging message.

exc_info
An exception tuple, or `None`.

func
The name of the function or method which invoked the logging call.

sinfo
A stack traceback such as is provided by *traceback.print_stack()*, showing the call hierarchy.

kwargs
Additional keyword arguments.

16.4.11 Module-Level Attributes

logging.**lastResort**

A "handler of last resort" is available through this attribute. This is a *StreamHandler* writing to `sys.stderr` with a level of `WARNING`, and is used to handle logging events in the absence of any logging configuration. The end result is to just print the message to `sys.stderr`. This replaces the earlier error message saying that "no

handlers could be found for logger XYZ". If you need the earlier behaviour for some reason, `lastResort` can be set to `None`.

Adicionado na versão 3.2.

`logging.`**`raiseExceptions`**

Used to see if exceptions during handling should be propagated.

Padrão: `True`.

If *`raiseExceptions`* is `False`, exceptions get silently ignored. This is what is mostly wanted for a logging system - most users will not care about errors in the logging system, they are more interested in application errors.

16.4.12 Integration with the warnings module

The *`captureWarnings()`* function can be used to integrate *`logging`* with the *`warnings`* module.

`logging.`**`captureWarnings`**(*capture*)

This function is used to turn the capture of warnings by logging on and off.

If *capture* is `True`, warnings issued by the *`warnings`* module will be redirected to the logging system. Specifically, a warning will be formatted using *`warnings.formatwarning()`* and the resulting string logged to a logger named `'py.warnings'` with a severity of *`WARNING`*.

If *capture* is `False`, the redirection of warnings to the logging system will stop, and warnings will be redirected to their original destinations (i.e. those in effect before `captureWarnings(True)` was called).

Ver também

Módulo *`logging.config`*
API de configuração para o módulo logging.

Módulo *`logging.handlers`*
Tratadores úteis incluídos no módulo logging.

PEP 282 - A Logging System
The proposal which described this feature for inclusion in the Python standard library.

Original Python logging package
This is the original source for the *`logging`* package. The version of the package available from this site is suitable for use with Python 1.5.2, 2.1.x and 2.2.x, which do not include the *`logging`* package in the standard library.

16.5 `logging.config` — Configuração do módulo logging

Código-fonte: Lib/logging/config.py

Important

Esta página contém apenas informações de referência. Para tutoriais, por favor consulte

- Tutorial básico
- Tutorial avançado
- Livro de receitas do logging

Esta seção descreve a API para configuração do módulo logging.

16.5.1 Funções de configuração

Os seguintes função configuram o módulo logging. Elas estão localizadas no módulo *logging.config*. Seu uso é opcional — você pode configurar o módulo logging usando essas funções ou fazendo chamadas para o API principal (definido no próprio *logging*) e definindo manipuladores que são declarados em *logging* ou *logging.handlers*.

logging.config.**dictConfig**(*config*)

Obtém a configuração de logging de um dicionário. O conteúdo desse dicionário está descrito em *Configuration dictionary schema* abaixo.

If an error is encountered during configuration, this function will raise a *ValueError*, *TypeError*, *AttributeError* or *ImportError* with a suitably descriptive message. The following is a (possibly incomplete) list of conditions which will raise an error:

- A `level` which is not a string or which is a string not corresponding to an actual logging level.
- A `propagate` value which is not a boolean.
- An id which does not have a corresponding destination.
- A non-existent handler id found during an incremental call.
- An invalid logger name.
- Inability to resolve to an internal or external object.

Parsing is performed by the `DictConfigurator` class, whose constructor is passed the dictionary used for configuration, and has a `configure()` method. The *logging.config* module has a callable attribute `dictConfigClass` which is initially set to `DictConfigurator`. You can replace the value of `dictConfigClass` with a suitable implementation of your own.

dictConfig() calls `dictConfigClass` passing the specified dictionary, and then calls the `configure()` method on the returned object to put the configuration into effect:

```
def dictConfig(config):
    dictConfigClass(config).configure()
```

For example, a subclass of `DictConfigurator` could call `DictConfigurator.__init__()` in its own `__init__()`, then set up custom prefixes which would be usable in the subsequent `configure()` call. `dictConfigClass` would be bound to this new subclass, and then *dictConfig()* could be called exactly as in the default, uncustomized state.

Adicionado na versão 3.2.

logging.config.**fileConfig**(*fname*, *defaults=None*, *disable_existing_loggers=True*, *encoding=None*)

Reads the logging configuration from a *configparser*-format file. The format of the file should be as described in *Formato do arquivo de configuração*. This function can be called several times from an application, allowing an end user to select from various pre-canned configurations (if the developer provides a mechanism to present the choices and load the chosen configuration).

It will raise *FileNotFoundError* if the file doesn't exist and *RuntimeError* if the file is invalid or empty.

Parâmetros

- **fname** – A filename, or a file-like object, or an instance derived from *RawConfigParser*. If a `RawConfigParser`-derived instance is passed, it is used as is. Otherwise, a *ConfigParser* is instantiated, and the configuration read by it from the object passed in `fname`. If that has a *readline()* method, it is assumed to be a file-like object and read using *read_file()*; otherwise, it is assumed to be a filename and passed to *read()*.
- **defaults** – Defaults to be passed to the `ConfigParser` can be specified in this argument.
- **disable_existing_loggers** – If specified as `False`, loggers which exist when this call is made are left enabled. The default is `True` because this enables old behaviour in

a backward-compatible way. This behaviour is to disable any existing non-root loggers unless they or their ancestors are explicitly named in the logging configuration.

- **encoding** – The encoding used to open file when *fname* is filename.

Alterado na versão 3.4: An instance of a subclass of *RawConfigParser* is now accepted as a value for `fname`. This facilitates:

- Use of a configuration file where logging configuration is just part of the overall application configuration.
- Use of a configuration read from a file, and then modified by the using application (e.g. based on command-line parameters or other aspects of the runtime environment) before being passed to `fileConfig`.

Alterado na versão 3.10: Added the *encoding* parameter.

Alterado na versão 3.12: An exception will be thrown if the provided file doesn't exist or is invalid or empty.

`logging.config.`**`listen`**(*port=DEFAULT_LOGGING_CONFIG_PORT*, *verify=None*)

Starts up a socket server on the specified port, and listens for new configurations. If no port is specified, the module's default `DEFAULT_LOGGING_CONFIG_PORT` is used. Logging configurations will be sent as a file suitable for processing by *dictConfig()* or *fileConfig()*. Returns a *Thread* instance on which you can call *start()* to start the server, and which you can *join()* when appropriate. To stop the server, call *stopListening()*.

The `verify` argument, if specified, should be a callable which should verify whether bytes received across the socket are valid and should be processed. This could be done by encrypting and/or signing what is sent across the socket, such that the `verify` callable can perform signature verification and/or decryption. The `verify` callable is called with a single argument - the bytes received across the socket - and should return the bytes to be processed, or `None` to indicate that the bytes should be discarded. The returned bytes could be the same as the passed in bytes (e.g. when only verification is done), or they could be completely different (perhaps if decryption were performed).

To send a configuration to the socket, read in the configuration file and send it to the socket as a sequence of bytes preceded by a four-byte length string packed in binary using `struct.pack('>L', n)`.

Nota

Because portions of the configuration are passed through *eval()*, use of this function may open its users to a security risk. While the function only binds to a socket on `localhost`, and so does not accept connections from remote machines, there are scenarios where untrusted code could be run under the account of the process which calls *listen()*. Specifically, if the process calling *listen()* runs on a multi-user machine where users cannot trust each other, then a malicious user could arrange to run essentially arbitrary code in a victim user's process, simply by connecting to the victim's *listen()* socket and sending a configuration which runs whatever code the attacker wants to have executed in the victim's process. This is especially easy to do if the default port is used, but not hard even if a different port is used. To avoid the risk of this happening, use the `verify` argument to *listen()* to prevent unrecognised configurations from being applied.

Alterado na versão 3.4: The `verify` argument was added.

Nota

If you want to send configurations to the listener which don't disable existing loggers, you will need to use a JSON format for the configuration, which will use *dictConfig()* for configuration. This method allows you to specify `disable_existing_loggers` as `False` in the configuration you send.

`logging.config.`**`stopListening`**()

Stops the listening server which was created with a call to *listen()*. This is typically called before calling `join()` on the return value from *listen()*.

16.5.2 Considerações de segurança

The logging configuration functionality tries to offer convenience, and in part this is done by offering the ability to convert text in configuration files into Python objects used in logging configuration - for example, as described in *User-defined objects*. However, these same mechanisms (importing callables from user-defined modules and calling them with parameters from the configuration) could be used to invoke any code you like, and for this reason you should treat configuration files from untrusted sources with *extreme caution* and satisfy yourself that nothing bad can happen if you load them, before actually loading them.

16.5.3 Configuration dictionary schema

Describing a logging configuration requires listing the various objects to create and the connections between them; for example, you may create a handler named 'console' and then say that the logger named 'startup' will send its messages to the 'console' handler. These objects aren't limited to those provided by the `logging` module because you might write your own formatter or handler class. The parameters to these classes may also need to include external objects such as `sys.stderr`. The syntax for describing these objects and connections is defined in *Object connections* below.

Dictionary Schema Details

The dictionary passed to `dictConfig()` must contain the following keys:

- *version* - to be set to an integer value representing the schema version. The only valid value at present is 1, but having this key allows the schema to evolve while still preserving backwards compatibility.

All other keys are optional, but if present they will be interpreted as described below. In all cases below where a 'configuring dict' is mentioned, it will be checked for the special `'()'` key to see if a custom instantiation is required. If so, the mechanism described in *User-defined objects* below is used to create an instance; otherwise, the context is used to determine what to instantiate.

- *formatters* - the corresponding value will be a dict in which each key is a formatter id and each value is a dict describing how to configure the corresponding `Formatter` instance.

  The configuring dict is searched for the following optional keys which correspond to the arguments passed to create a `Formatter` object:

  - `format`
  - `datefmt`
  - `style`
  - `validate` (since version >=3.8)
  - `defaults` (since version >=3.12)

  An optional `class` key indicates the name of the formatter's class (as a dotted module and class name). The instantiation arguments are as for `Formatter`, thus this key is most useful for instantiating a customised subclass of `Formatter`. For example, the alternative class might present exception tracebacks in an expanded or condensed format. If your formatter requires different or extra configuration keys, you should use *User-defined objects*.

- *filters* - the corresponding value will be a dict in which each key is a filter id and each value is a dict describing how to configure the corresponding Filter instance.

  The configuring dict is searched for the key `name` (defaulting to the empty string) and this is used to construct a `logging.Filter` instance.

- *handlers* - the corresponding value will be a dict in which each key is a handler id and each value is a dict describing how to configure the corresponding Handler instance.

  The configuring dict is searched for the following keys:

  - `class` (mandatory). This is the fully qualified name of the handler class.
  - `level` (optional). The level of the handler.
  - `formatter` (optional). The id of the formatter for this handler.

  - `filters` (optional). A list of ids of the filters for this handler.

    Alterado na versão 3.11: `filters` can take filter instances in addition to ids.

  All *other* keys are passed through as keyword arguments to the handler's constructor. For example, given the snippet:

```
handlers:
  console:
    class : logging.StreamHandler
    formatter: brief
    level   : INFO
    filters: [allow_foo]
    stream  : ext://sys.stdout
  file:
    class : logging.handlers.RotatingFileHandler
    formatter: precise
    filename: logconfig.log
    maxBytes: 1024
    backupCount: 3
```

  the handler with id `console` is instantiated as a *`logging.StreamHandler`*, using `sys.stdout` as the underlying stream. The handler with id `file` is instantiated as a *`logging.handlers.RotatingFileHandler`* with the keyword arguments `filename='logconfig.log', maxBytes=1024, backupCount=3`.

- *loggers* - the corresponding value will be a dict in which each key is a logger name and each value is a dict describing how to configure the corresponding Logger instance.

  The configuring dict is searched for the following keys:

  - `level` (optional). The level of the logger.
  - `propagate` (optional). The propagation setting of the logger.
  - `filters` (optional). A list of ids of the filters for this logger.

    Alterado na versão 3.11: `filters` can take filter instances in addition to ids.

  - `handlers` (optional). A list of ids of the handlers for this logger.

  The specified loggers will be configured according to the level, propagation, filters and handlers specified.

- *root* - this will be the configuration for the root logger. Processing of the configuration will be as for any logger, except that the `propagate` setting will not be applicable.

- *incremental* - whether the configuration is to be interpreted as incremental to the existing configuration. This value defaults to `False`, which means that the specified configuration replaces the existing configuration with the same semantics as used by the existing *`fileConfig()`* API.

  If the specified value is `True`, the configuration is processed as described in the section on *Incremental Configuration*.

- *disable_existing_loggers* - whether any existing non-root loggers are to be disabled. This setting mirrors the parameter of the same name in *`fileConfig()`*. If absent, this parameter defaults to `True`. This value is ignored if *incremental* is `True`.

Incremental Configuration

It is difficult to provide complete flexibility for incremental configuration. For example, because objects such as filters and formatters are anonymous, once a configuration is set up, it is not possible to refer to such anonymous objects when augmenting a configuration.

Furthermore, there is not a compelling case for arbitrarily altering the object graph of loggers, handlers, filters, formatters at run-time, once a configuration is set up; the verbosity of loggers and handlers can be controlled just by setting levels (and, in the case of loggers, propagation flags). Changing the object graph arbitrarily in a safe way is

problematic in a multi-threaded environment; while not impossible, the benefits are not worth the complexity it adds to the implementation.

Thus, when the `incremental` key of a configuration dict is present and is `True`, the system will completely ignore any `formatters` and `filters` entries, and process only the `level` settings in the `handlers` entries, and the `level` and `propagate` settings in the `loggers` and `root` entries.

Using a value in the configuration dict lets configurations to be sent over the wire as pickled dicts to a socket listener. Thus, the logging verbosity of a long-running application can be altered over time with no need to stop and restart the application.

Object connections

The schema describes a set of logging objects - loggers, handlers, formatters, filters - which are connected to each other in an object graph. Thus, the schema needs to represent connections between the objects. For example, say that, once configured, a particular logger has attached to it a particular handler. For the purposes of this discussion, we can say that the logger represents the source, and the handler the destination, of a connection between the two. Of course in the configured objects this is represented by the logger holding a reference to the handler. In the configuration dict, this is done by giving each destination object an id which identifies it unambiguously, and then using the id in the source object's configuration to indicate that a connection exists between the source and the destination object with that id.

So, for example, consider the following YAML snippet:

```
formatters:
  brief:
    # configuration for formatter with id 'brief' goes here
  precise:
    # configuration for formatter with id 'precise' goes here
handlers:
  h1: #This is an id
   # configuration of handler with id 'h1' goes here
   formatter: brief
  h2: #This is another id
   # configuration of handler with id 'h2' goes here
   formatter: precise
loggers:
  foo.bar.baz:
    # other configuration for logger 'foo.bar.baz'
    handlers: [h1, h2]
```

(Note: YAML used here because it's a little more readable than the equivalent Python source form for the dictionary.)

The ids for loggers are the logger names which would be used programmatically to obtain a reference to those loggers, e.g. `foo.bar.baz`. The ids for Formatters and Filters can be any string value (such as `brief`, `precise` above) and they are transient, in that they are only meaningful for processing the configuration dictionary and used to determine connections between objects, and are not persisted anywhere when the configuration call is complete.

The above snippet indicates that logger named `foo.bar.baz` should have two handlers attached to it, which are described by the handler ids `h1` and `h2`. The formatter for `h1` is that described by id `brief`, and the formatter for `h2` is that described by id `precise`.

User-defined objects

The schema supports user-defined objects for handlers, filters and formatters. (Loggers do not need to have different types for different instances, so there is no support in this configuration schema for user-defined logger classes.)

Objects to be configured are described by dictionaries which detail their configuration. In some places, the logging system will be able to infer from the context how an object is to be instantiated, but when a user-defined object is to be instantiated, the system will not know how to do this. In order to provide complete flexibility for user-defined object instantiation, the user needs to provide a 'factory' - a callable which is called with a configuration dictionary

and which returns the instantiated object. This is signalled by an absolute import path to the factory being made available under the special key `'()'`. Here's a concrete example:

```
formatters:
  brief:
    format: '%(message)s'
  default:
    format: '%(asctime)s %(levelname)-8s %(name)-15s %(message)s'
    datefmt: '%Y-%m-%d %H:%M:%S'
  custom:
      (): my.package.customFormatterFactory
      bar: baz
      spam: 99.9
      answer: 42
```

The above YAML snippet defines three formatters. The first, with id `brief`, is a standard *`logging.Formatter`* instance with the specified format string. The second, with id `default`, has a longer format and also defines the time format explicitly, and will result in a *`logging.Formatter`* initialized with those two format strings. Shown in Python source form, the `brief` and `default` formatters have configuration sub-dictionaries:

```
{
  'format' : '%(message)s'
}
```

e:

```
{
  'format' : '%(asctime)s %(levelname)-8s %(name)-15s %(message)s',
  'datefmt' : '%Y-%m-%d %H:%M:%S'
}
```

respectively, and as these dictionaries do not contain the special key `'()'`, the instantiation is inferred from the context: as a result, standard *`logging.Formatter`* instances are created. The configuration sub-dictionary for the third formatter, with id `custom`, is:

```
{
  '()' : 'my.package.customFormatterFactory',
  'bar' : 'baz',
  'spam' : 99.9,
  'answer' : 42
}
```

and this contains the special key `'()'`, which means that user-defined instantiation is wanted. In this case, the specified factory callable will be used. If it is an actual callable it will be used directly - otherwise, if you specify a string (as in the example) the actual callable will be located using normal import mechanisms. The callable will be called with the **remaining** items in the configuration sub-dictionary as keyword arguments. In the above example, the formatter with id `custom` will be assumed to be returned by the call:

```
my.package.customFormatterFactory(bar='baz', spam=99.9, answer=42)
```

Aviso

The values for keys such as `bar`, `spam` and `answer` in the above example should not be configuration dictionaries or references such as `cfg://foo` or `ext://bar`, because they will not be processed by the configuration machinery, but passed to the callable as-is.

The key `'()'` has been used as the special key because it is not a valid keyword parameter name, and so will not clash with the names of the keyword arguments used in the call. The `'()'` also serves as a mnemonic that the

corresponding value is a callable.

Alterado na versão 3.11: The `filters` member of `handlers` and `loggers` can take filter instances in addition to ids.

You can also specify a special key `'.'` whose value is a mapping of attribute names to values. If found, the specified attributes will be set on the user-defined object before it is returned. Thus, with the following configuration:

```
{
  '()' : 'my.package.customFormatterFactory',
  'bar' : 'baz',
  'spam' : 99.9,
  'answer' : 42,
  '.' {
    'foo': 'bar',
    'baz': 'bozz'
  }
}
```

the returned formatter will have attribute `foo` set to `'bar'` and attribute `baz` set to `'bozz'`.

Aviso

The values for attributes such as `foo` and `baz` in the above example should not be configuration dictionaries or references such as `cfg://foo` or `ext://bar`, because they will not be processed by the configuration machinery, but set as attribute values as-is.

Handler configuration order

Handlers are configured in alphabetical order of their keys, and a configured handler replaces the configuration dictionary in (a working copy of) the `handlers` dictionary in the schema. If you use a construct such as `cfg://handlers.foo`, then initially `handlers['foo']` points to the configuration dictionary for the handler named `foo`, and later (once that handler has been configured) it points to the configured handler instance. Thus, `cfg://handlers.foo` could resolve to either a dictionary or a handler instance. In general, it is wise to name handlers in a way such that dependent handlers are configured *after* any handlers they depend on; that allows something like `cfg://handlers.foo` to be used in configuring a handler that depends on handler `foo`. If that dependent handler were named `bar`, problems would result, because the configuration of `bar` would be attempted before that of `foo`, and `foo` would not yet have been configured. However, if the dependent handler were named `foobar`, it would be configured after `foo`, with the result that `cfg://handlers.foo` would resolve to configured handler `foo`, and not its configuration dictionary.

Access to external objects

There are times where a configuration needs to refer to objects external to the configuration, for example `sys.stderr`. If the configuration dict is constructed using Python code, this is straightforward, but a problem arises when the configuration is provided via a text file (e.g. JSON, YAML). In a text file, there is no standard way to distinguish `sys.stderr` from the literal string `'sys.stderr'`. To facilitate this distinction, the configuration system looks for certain special prefixes in string values and treat them specially. For example, if the literal string `'ext://sys.stderr'` is provided as a value in the configuration, then the `ext://` will be stripped off and the remainder of the value processed using normal import mechanisms.

The handling of such prefixes is done in a way analogous to protocol handling: there is a generic mechanism to look for prefixes which match the regular expression `^(?P<prefix>[a-z]+)://(?P<suffix>.*)$` whereby, if the `prefix` is recognised, the `suffix` is processed in a prefix-dependent manner and the result of the processing replaces the string value. If the prefix is not recognised, then the string value will be left as-is.

Access to internal objects

As well as external objects, there is sometimes also a need to refer to objects in the configuration. This will be done implicitly by the configuration system for things that it knows about. For example, the string value `'DEBUG'` for a `level` in a logger or handler will automatically be converted to the value `logging.DEBUG`, and the `handlers`, `filters` and `formatter` entries will take an object id and resolve to the appropriate destination object.

However, a more generic mechanism is needed for user-defined objects which are not known to the *logging* module. For example, consider *logging.handlers.MemoryHandler*, which takes a `target` argument which is another handler to delegate to. Since the system already knows about this class, then in the configuration, the given `target` just needs to be the object id of the relevant target handler, and the system will resolve to the handler from the id. If, however, a user defines a `my.package.MyHandler` which has an `alternate` handler, the configuration system would not know that the `alternate` referred to a handler. To cater for this, a generic resolution system allows the user to specify:

```
handlers:
  file:
    # configuration of file handler goes here

  custom:
    (): my.package.MyHandler
    alternate: cfg://handlers.file
```

The literal string `'cfg://handlers.file'` will be resolved in an analogous way to strings with the `ext://` prefix, but looking in the configuration itself rather than the import namespace. The mechanism allows access by dot or by index, in a similar way to that provided by `str.format`. Thus, given the following snippet:

```
handlers:
  email:
    class: logging.handlers.SMTPHandler
    mailhost: localhost
    fromaddr: my_app@domain.tld
    toaddrs:
      - support_team@domain.tld
      - dev_team@domain.tld
    subject: Houston, we have a problem.
```

in the configuration, the string `'cfg://handlers'` would resolve to the dict with key `handlers`, the string `'cfg://handlers.email` would resolve to the dict with key `email` in the `handlers` dict, and so on. The string `'cfg://handlers.email.toaddrs[1]` would resolve to `'dev_team@domain.tld'` and the string `'cfg://handlers.email.toaddrs[0]'` would resolve to the value `'support_team@domain.tld'`. The `subject` value could be accessed using either `'cfg://handlers.email.subject'` or, equivalently, `'cfg://handlers.email[subject]'`. The latter form only needs to be used if the key contains spaces or non-alphanumeric characters. Please note that the characters `[` and `]` are not allowed in the keys. If an index value consists only of decimal digits, access will be attempted using the corresponding integer value, falling back to the string value if needed.

Given a string `cfg://handlers.myhandler.mykey.123`, this will resolve to `config_dict['handlers']['myhandler']['mykey']['123']`. If the string is specified as `cfg://handlers.myhandler.mykey[123]`, the system will attempt to retrieve the value from `config_dict['handlers']['myhandler']['mykey'][123]`, and fall back to `config_dict['handlers']['myhandler']['mykey']['123']` if that fails.

Import resolution and custom importers

Import resolution, by default, uses the builtin *__import__()* function to do its importing. You may want to replace this with your own importing mechanism: if so, you can replace the `importer` attribute of the `DictConfigurator` or its superclass, the `BaseConfigurator` class. However, you need to be careful because of the way functions are accessed from classes via descriptors. If you are using a Python callable to do your imports, and you want to define it at class level rather than instance level, you need to wrap it with *staticmethod()*. For example:

```
from importlib import import_module
from logging.config import BaseConfigurator

BaseConfigurator.importer = staticmethod(import_module)
```

You don't need to wrap with *staticmethod()* if you're setting the import callable on a configurator *instance*.

Configuring QueueHandler and QueueListener

If you want to configure a *QueueHandler*, noting that this is normally used in conjunction with a *QueueListener*, you can configure both together. After the configuration, the QueueListener instance will be available as the *listener* attribute of the created handler, and that in turn will be available to you using *getHandlerByName()* and passing the name you have used for the QueueHandler in your configuration. The dictionary schema for configuring the pair is shown in the example YAML snippet below.

```
handlers:
  qhand:
    class: logging.handlers.QueueHandler
    queue: my.module.queue_factory
    listener: my.package.CustomListener
    handlers:
      - hand_name_1
      - hand_name_2
      ...
```

The queue and listener keys are optional.

If the queue key is present, the corresponding value can be one of the following:

- An object implementing the *Queue.put_nowait* and *Queue.get* public API. For instance, this may be an actual instance of *queue.Queue* or a subclass thereof, or a proxy obtained by *multiprocessing.managers.SyncManager.Queue()*.

  This is of course only possible if you are constructing or modifying the configuration dictionary in code.

- A string that resolves to a callable which, when called with no arguments, returns the queue instance to use. That callable could be a *queue.Queue* subclass or a function which returns a suitable queue instance, such as my.module.queue_factory().

- A dict with a '()' key which is constructed in the usual way as discussed in *User-defined objects*. The result of this construction should be a *queue.Queue* instance.

If the queue key is absent, a standard unbounded *queue.Queue* instance is created and used.

If the listener key is present, the corresponding value can be one of the following:

- A subclass of *logging.handlers.QueueListener*. This is of course only possible if you are constructing or modifying the configuration dictionary in code.

- A string which resolves to a class which is a subclass of QueueListener, such as 'my.package.CustomListener'.

- A dict with a '()' key which is constructed in the usual way as discussed in *User-defined objects*. The result of this construction should be a callable with the same signature as the QueueListener initializer.

If the listener key is absent, *logging.handlers.QueueListener* is used.

The values under the handlers key are the names of other handlers in the configuration (not shown in the above snippet) which will be passed to the queue listener.

Any custom queue handler and listener classes will need to be defined with the same initialization signatures as *QueueHandler* and *QueueListener*.

Adicionado na versão 3.12.

16.5.4 Formato do arquivo de configuração

The configuration file format understood by *fileConfig()* is based on *configparser* functionality. The file must contain sections called `[loggers]`, `[handlers]` and `[formatters]` which identify by name the entities of each type which are defined in the file. For each such entity, there is a separate section which identifies how that entity is configured. Thus, for a logger named `log01` in the `[loggers]` section, the relevant configuration details are held in a section `[logger_log01]`. Similarly, a handler called `hand01` in the `[handlers]` section will have its configuration held in a section called `[handler_hand01]`, while a formatter called `form01` in the `[formatters]` section will have its configuration specified in a section called `[formatter_form01]`. The root logger configuration must be specified in a section called `[logger_root]`.

Nota

The *fileConfig()* API is older than the *dictConfig()* API and does not provide functionality to cover certain aspects of logging. For example, you cannot configure *Filter* objects, which provide for filtering of messages beyond simple integer levels, using *fileConfig()*. If you need to have instances of *Filter* in your logging configuration, you will need to use *dictConfig()*. Note that future enhancements to configuration functionality will be added to *dictConfig()*, so it's worth considering transitioning to this newer API when it's convenient to do so.

Examples of these sections in the file are given below.

```
[loggers]
keys=root,log02,log03,log04,log05,log06,log07

[handlers]
keys=hand01,hand02,hand03,hand04,hand05,hand06,hand07,hand08,hand09

[formatters]
keys=form01,form02,form03,form04,form05,form06,form07,form08,form09
```

The root logger must specify a level and a list of handlers. An example of a root logger section is given below.

```
[logger_root]
level=NOTSET
handlers=hand01
```

The `level` entry can be one of `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL` or `NOTSET`. For the root logger only, `NOTSET` means that all messages will be logged. Level values are *evaluated* in the context of the `logging` package's namespace.

The `handlers` entry is a comma-separated list of handler names, which must appear in the `[handlers]` section. These names must appear in the `[handlers]` section and have corresponding sections in the configuration file.

For loggers other than the root logger, some additional information is required. This is illustrated by the following example.

```
[logger_parser]
level=DEBUG
handlers=hand01
propagate=1
qualname=compiler.parser
```

The `level` and `handlers` entries are interpreted as for the root logger, except that if a non-root logger's level is specified as `NOTSET`, the system consults loggers higher up the hierarchy to determine the effective level of the logger. The `propagate` entry is set to 1 to indicate that messages must propagate to handlers higher up the logger hierarchy from this logger, or 0 to indicate that messages are **not** propagated to handlers up the hierarchy. The `qualname` entry is the hierarchical channel name of the logger, that is to say the name used by the application to get the logger.

Sections which specify handler configuration are exemplified by the following.

```
[handler_hand01]
class=StreamHandler
level=NOTSET
formatter=form01
args=(sys.stdout,)
```

The `class` entry indicates the handler's class (as determined by *`eval()`* in the `logging` package's namespace). The `level` is interpreted as for loggers, and `NOTSET` is taken to mean 'log everything'.

The `formatter` entry indicates the key name of the formatter for this handler. If blank, a default formatter (`logging._defaultFormatter`) is used. If a name is specified, it must appear in the `[formatters]` section and have a corresponding section in the configuration file.

The `args` entry, when *evaluated* in the context of the `logging` package's namespace, is the list of arguments to the constructor for the handler class. Refer to the constructors for the relevant handlers, or to the examples below, to see how typical entries are constructed. If not provided, it defaults to `()`.

The optional `kwargs` entry, when *evaluated* in the context of the `logging` package's namespace, is the keyword argument dict to the constructor for the handler class. If not provided, it defaults to `{}`.

```
[handler_hand02]
class=FileHandler
level=DEBUG
formatter=form02
args=('python.log', 'w')

[handler_hand03]
class=handlers.SocketHandler
level=INFO
formatter=form03
args=('localhost', handlers.DEFAULT_TCP_LOGGING_PORT)

[handler_hand04]
class=handlers.DatagramHandler
level=WARN
formatter=form04
args=('localhost', handlers.DEFAULT_UDP_LOGGING_PORT)

[handler_hand05]
class=handlers.SysLogHandler
level=ERROR
formatter=form05
args=(('localhost', handlers.SYSLOG_UDP_PORT), handlers.SysLogHandler.LOG_USER)

[handler_hand06]
class=handlers.NTEventLogHandler
level=CRITICAL
formatter=form06
args=('Python Application', '', 'Application')

[handler_hand07]
class=handlers.SMTPHandler
level=WARN
formatter=form07
args=('localhost', 'from@abc', ['user1@abc', 'user2@xyz'], 'Logger Subject')
kwargs={'timeout': 10.0}

[handler_hand08]
```

(continua na próxima página)

(continuação da página anterior)

```
class=handlers.MemoryHandler
level=NOTSET
formatter=form08
target=
args=(10, ERROR)

[handler_hand09]
class=handlers.HTTPHandler
level=NOTSET
formatter=form09
args=('localhost:9022', '/log', 'GET')
kwargs={'secure': True}
```

Sections which specify formatter configuration are typified by the following.

```
[formatter_form01]
format=F1 %(asctime)s %(levelname)s %(message)s %(customfield)s
datefmt=
style=%
validate=True
defaults={'customfield': 'defaultvalue'}
class=logging.Formatter
```

The arguments for the formatter configuration are the same as the keys in the dictionary schema *formatters section*.

The `defaults` entry, when *evaluated* in the context of the `logging` package's namespace, is a dictionary of default values for custom formatting fields. If not provided, it defaults to `None`.

Nota

Due to the use of *`eval()`* as described above, there are potential security risks which result from using the *`listen()`* to send and receive configurations via sockets. The risks are limited to where multiple users with no mutual trust run code on the same machine; see the *`listen()`* documentation for more information.

Ver também

Módulo *`logging`*
Referência da API para o módulo de logging.

Módulo *`logging.handlers`*
Manipuladores úteis incluídos no módulo logging.

16.6 `logging.handlers` — Tratadores de registro de eventos

Código-fonte: Lib/logging/handlers.py

Important

Esta página contém apenas informações de referência. Para tutoriais, por favor consulte

- Tutorial básico
- Tutorial avançado
- Livro de receitas de logging

Os seguintes tratadores úteis são fornecidos neste pacote. Observe que três dos tratadores (*StreamHandler*, *FileHandler* e *NullHandler*) são, na verdade, definidos no próprio módulo *logging*, mas foram documentados aqui junto com os outros tratadores.

16.6.1 StreamHandler

The *StreamHandler* class, located in the core *logging* package, sends logging output to streams such as *sys.stdout*, *sys.stderr* or any file-like object (or, more precisely, any object which supports `write()` and `flush()` methods).

class logging.**StreamHandler**(*stream=None*)

Returns a new instance of the *StreamHandler* class. If *stream* is specified, the instance will use it for logging output; otherwise, *sys.stderr* will be used.

emit(*record*)

If a formatter is specified, it is used to format the record. The record is then written to the stream followed by *terminator*. If exception information is present, it is formatted using *traceback.print_exception()* and appended to the stream.

flush()

Flushes the stream by calling its *flush()* method. Note that the `close()` method is inherited from *Handler* and so does no output, so an explicit *flush()* call may be needed at times.

setStream(*stream*)

Sets the instance's stream to the specified value, if it is different. The old stream is flushed before the new stream is set.

Parâmetros
stream – The stream that the handler should use.

Retorna
the old stream, if the stream was changed, or `None` if it wasn't.

Adicionado na versão 3.7.

terminator

String used as the terminator when writing a formatted record to a stream. Default value is `'\n'`.

If you don't want a newline termination, you can set the handler instance's `terminator` attribute to the empty string.

In earlier versions, the terminator was hardcoded as `'\n'`.

Adicionado na versão 3.2.

16.6.2 FileHandler

The *FileHandler* class, located in the core *logging* package, sends logging output to a disk file. It inherits the output functionality from *StreamHandler*.

class logging.**FileHandler**(*filename*, *mode='a'*, *encoding=None*, *delay=False*, *errors=None*)

Returns a new instance of the *FileHandler* class. The specified file is opened and used as the stream for logging. If *mode* is not specified, `'a'` is used. If *encoding* is not `None`, it is used to open the file with that encoding. If *delay* is true, then file opening is deferred until the first call to *emit()*. By default, the file grows indefinitely. If *errors* is specified, it's used to determine how encoding errors are handled.

Alterado na versão 3.6: As well as string values, *Path* objects are also accepted for the *filename* argument.

Alterado na versão 3.9: The *errors* parameter was added.

close()

Closes the file.

emit(*record*)

Outputs the record to the file.

Note that if the file was closed due to logging shutdown at exit and the file mode is 'w', the record will not be emitted (see bpo-42378).

16.6.3 NullHandler

Adicionado na versão 3.1.

The *NullHandler* class, located in the core *logging* package, does not do any formatting or output. It is essentially a 'no-op' handler for use by library developers.

class logging.**NullHandler**

Returns a new instance of the *NullHandler* class.

emit(*record*)

This method does nothing.

handle(*record*)

This method does nothing.

createLock()

This method returns `None` for the lock, since there is no underlying I/O to which access needs to be serialized.

See library-config for more information on how to use *NullHandler*.

16.6.4 WatchedFileHandler

The *WatchedFileHandler* class, located in the *logging.handlers* module, is a `FileHandler` which watches the file it is logging to. If the file changes, it is closed and reopened using the file name.

A file change can happen because of usage of programs such as *newsyslog* and *logrotate* which perform log file rotation. This handler, intended for use under Unix/Linux, watches the file to see if it has changed since the last emit. (A file is deemed to have changed if its device or inode have changed.) If the file has changed, the old file stream is closed, and the file opened to get a new stream.

This handler is not appropriate for use under Windows, because under Windows open log files cannot be moved or renamed - logging opens the files with exclusive locks - and so there is no need for such a handler. Furthermore, *ST_INO* is not supported under Windows; *stat()* always returns zero for this value.

class logging.handlers.**WatchedFileHandler**(*filename*, *mode='a'*, *encoding=None*, *delay=False*, *errors=None*)

Returns a new instance of the *WatchedFileHandler* class. The specified file is opened and used as the stream for logging. If *mode* is not specified, `'a'` is used. If *encoding* is not `None`, it is used to open the file with that encoding. If *delay* is true, then file opening is deferred until the first call to *emit()*. By default, the file grows indefinitely. If *errors* is provided, it determines how encoding errors are handled.

Alterado na versão 3.6: As well as string values, *Path* objects are also accepted for the *filename* argument.

Alterado na versão 3.9: The *errors* parameter was added.

reopenIfNeeded()

Checks to see if the file has changed. If it has, the existing stream is flushed and closed and the file opened again, typically as a precursor to outputting the record to the file.

Adicionado na versão 3.6.

emit(*record*)

Outputs the record to the file, but first calls *reopenIfNeeded()* to reopen the file if it has changed.

16.6.5 BaseRotatingHandler

The *BaseRotatingHandler* class, located in the *logging.handlers* module, is the base class for the rotating file handlers, *RotatingFileHandler* and *TimedRotatingFileHandler*. You should not need to instantiate this class, but it has attributes and methods you may need to override.

class logging.handlers.**BaseRotatingHandler**(*filename*, *mode*, *encoding=None*, *delay=False*, *errors=None*)

The parameters are as for `FileHandler`. The attributes are:

namer

If this attribute is set to a callable, the *rotation_filename()* method delegates to this callable. The parameters passed to the callable are those passed to *rotation_filename()*.

> **Nota**
>
> The namer function is called quite a few times during rollover, so it should be as simple and as fast as possible. It should also return the same output every time for a given input, otherwise the rollover behaviour may not work as expected.
>
> It's also worth noting that care should be taken when using a namer to preserve certain attributes in the filename which are used during rotation. For example, *RotatingFileHandler* expects to have a set of log files whose names contain successive integers, so that rotation works as expected, and *TimedRotatingFileHandler* deletes old log files (based on the `backupCount` parameter passed to the handler's initializer) by determining the oldest files to delete. For this to happen, the filenames should be sortable using the date/time portion of the filename, and a namer needs to respect this. (If a namer is wanted that doesn't respect this scheme, it will need to be used in a subclass of *TimedRotatingFileHandler* which overrides the *getFilesToDelete()* method to fit in with the custom naming scheme.)

Adicionado na versão 3.3.

rotator

If this attribute is set to a callable, the *rotate()* method delegates to this callable. The parameters passed to the callable are those passed to *rotate()*.

Adicionado na versão 3.3.

rotation_filename(*default_name*)

Modify the filename of a log file when rotating.

This is provided so that a custom filename can be provided.

The default implementation calls the 'namer' attribute of the handler, if it's callable, passing the default name to it. If the attribute isn't callable (the default is `None`), the name is returned unchanged.

Parâmetros

default_name – The default name for the log file.

Adicionado na versão 3.3.

rotate(*source*, *dest*)

When rotating, rotate the current log.

The default implementation calls the 'rotator' attribute of the handler, if it's callable, passing the source and dest arguments to it. If the attribute isn't callable (the default is `None`), the source is simply renamed to the destination.

Parâmetros

- **source** – The source filename. This is normally the base filename, e.g. 'test.log'.
- **dest** – The destination filename. This is normally what the source is rotated to, e.g. 'test.log.1'.

Adicionado na versão 3.3.

The reason the attributes exist is to save you having to subclass - you can use the same callables for instances of *RotatingFileHandler* and *TimedRotatingFileHandler*. If either the namer or rotator callable raises an exception, this will be handled in the same way as any other exception during an `emit()` call, i.e. via the `handleError()` method of the handler.

If you need to make more significant changes to rotation processing, you can override the methods.

For an example, see cookbook-rotator-namer.

16.6.6 RotatingFileHandler

The *RotatingFileHandler* class, located in the *logging.handlers* module, supports rotation of disk log files.

class logging.handlers.**RotatingFileHandler**(*filename*, *mode='a'*, *maxBytes=0*, *backupCount=0*, *encoding=None*, *delay=False*, *errors=None*)

Returns a new instance of the *RotatingFileHandler* class. The specified file is opened and used as the stream for logging. If *mode* is not specified, `'a'` is used. If *encoding* is not `None`, it is used to open the file with that encoding. If *delay* is true, then file opening is deferred until the first call to *emit()*. By default, the file grows indefinitely. If *errors* is provided, it determines how encoding errors are handled.

You can use the *maxBytes* and *backupCount* values to allow the file to *rollover* at a predetermined size. When the size is about to be exceeded, the file is closed and a new file is silently opened for output. Rollover occurs whenever the current log file is nearly *maxBytes* in length; but if either of *maxBytes* or *backupCount* is zero, rollover never occurs, so you generally want to set *backupCount* to at least 1, and have a non-zero *maxBytes*. When *backupCount* is non-zero, the system will save old log files by appending the extensions '.1', '.2' etc., to the filename. For example, with a *backupCount* of 5 and a base file name of `app.log`, you would get `app.log`, `app.log.1`, `app.log.2`, up to `app.log.5`. The file being written to is always `app.log`. When this file is filled, it is closed and renamed to `app.log.1`, and if files `app.log.1`, `app.log.2`, etc. exist, then they are renamed to `app.log.2`, `app.log.3` etc. respectively.

Alterado na versão 3.6: As well as string values, *Path* objects are also accepted for the *filename* argument.

Alterado na versão 3.9: The *errors* parameter was added.

doRollover()

Does a rollover, as described above.

emit(*record*)

Outputs the record to the file, catering for rollover as described previously.

shouldRollover(*record*)

See if the supplied record would cause the file to exceed the configured size limit.

16.6.7 TimedRotatingFileHandler

The *TimedRotatingFileHandler* class, located in the *logging.handlers* module, supports rotation of disk log files at certain timed intervals.

class logging.handlers.**TimedRotatingFileHandler**(*filename*, *when='h'*, *interval=1*, *backupCount=0*, *encoding=None*, *delay=False*, *utc=False*, *atTime=None*, *errors=None*)

Returns a new instance of the *TimedRotatingFileHandler* class. The specified file is opened and used as the stream for logging. On rotating it also sets the filename suffix. Rotating happens based on the product of *when* and *interval*.

You can use the *when* to specify the type of *interval*. The list of possible values is below. Note that they are not case sensitive.

Valor	Type of interval	If/how *atTime* is used
`'S'`	Seconds	Ignored
`'M'`	Minutes	Ignored
`'H'`	Horas	Ignored
`'D'`	Dias	Ignored
`'W0'-'W6'`	Weekday (0=Monday)	Used to compute initial rollover time
`'midnight'`	Roll over at midnight, if *atTime* not specified, else at time *atTime*	Used to compute initial rollover time

When using weekday-based rotation, specify 'W0' for Monday, 'W1' for Tuesday, and so on up to 'W6' for Sunday. In this case, the value passed for *interval* isn't used.

The system will save old log files by appending extensions to the filename. The extensions are date-and-time based, using the strftime format `%Y-%m-%d_%H-%M-%S` or a leading portion thereof, depending on the rollover interval.

When computing the next rollover time for the first time (when the handler is created), the last modification time of an existing log file, or else the current time, is used to compute when the next rotation will occur.

If the *utc* argument is true, times in UTC will be used; otherwise local time is used.

If *backupCount* is nonzero, at most *backupCount* files will be kept, and if more would be created when rollover occurs, the oldest one is deleted. The deletion logic uses the interval to determine which files to delete, so changing the interval may leave old files lying around.

If *delay* is true, then file opening is deferred until the first call to `emit()`.

If *atTime* is not `None`, it must be a `datetime.time` instance which specifies the time of day when rollover occurs, for the cases where rollover is set to happen "at midnight" or "on a particular weekday". Note that in these cases, the *atTime* value is effectively used to compute the *initial* rollover, and subsequent rollovers would be calculated via the normal interval calculation.

If *errors* is specified, it's used to determine how encoding errors are handled.

Nota

Calculation of the initial rollover time is done when the handler is initialised. Calculation of subsequent rollover times is done only when rollover occurs, and rollover occurs only when emitting output. If this is not kept in mind, it might lead to some confusion. For example, if an interval of "every minute" is set, that does not mean you will always see log files with times (in the filename) separated by a minute; if, during application execution, logging output is generated more frequently than once a minute, *then* you can expect to see log files with times separated by a minute. If, on the other hand, logging messages are only output once every five minutes (say), then there will be gaps in the file times corresponding to the minutes where no output (and hence no rollover) occurred.

Alterado na versão 3.4: *atTime* parameter was added.

Alterado na versão 3.6: As well as string values, `Path` objects are also accepted for the *filename* argument.

Alterado na versão 3.9: The *errors* parameter was added.

doRollover()

Does a rollover, as described above.

emit(*record*)

Outputs the record to the file, catering for rollover as described above.

getFilesToDelete()

Returns a list of filenames which should be deleted as part of rollover. These

shouldRollover(*record*)

See if enough time has passed for a rollover to occur and if it has, compute the next rollover time.

16.6.8 SocketHandler

The *SocketHandler* class, located in the *logging.handlers* module, sends logging output to a network socket. The base class uses a TCP socket.

class logging.handlers.**SocketHandler**(*host*, *port*)

Returns a new instance of the *SocketHandler* class intended to communicate with a remote machine whose address is given by *host* and *port*.

Alterado na versão 3.4: If `port` is specified as `None`, a Unix domain socket is created using the value in `host` - otherwise, a TCP socket is created.

close()

Closes the socket.

emit()

Pickles the record's attribute dictionary and writes it to the socket in binary format. If there is an error with the socket, silently drops the packet. If the connection was previously lost, re-establishes the connection. To unpickle the record at the receiving end into a *LogRecord*, use the *makeLogRecord()* function.

handleError()

Handles an error which has occurred during *emit()*. The most likely cause is a lost connection. Closes the socket so that we can retry on the next event.

makeSocket()

This is a factory method which allows subclasses to define the precise type of socket they want. The default implementation creates a TCP socket (*socket.SOCK_STREAM*).

makePickle(*record*)

Pickles the record's attribute dictionary in binary format with a length prefix, and returns it ready for transmission across the socket. The details of this operation are equivalent to:

```
data = pickle.dumps(record_attr_dict, 1)
datalen = struct.pack('>L', len(data))
return datalen + data
```

Note that pickles aren't completely secure. If you are concerned about security, you may want to override this method to implement a more secure mechanism. For example, you can sign pickles using HMAC and then verify them on the receiving end, or alternatively you can disable unpickling of global objects on the receiving end.

send(*packet*)

Send a pickled byte-string *packet* to the socket. The format of the sent byte-string is as described in the documentation for *makePickle()*.

This function allows for partial sends, which can happen when the network is busy.

createSocket()

Tries to create a socket; on failure, uses an exponential back-off algorithm. On initial failure, the handler will drop the message it was trying to send. When subsequent messages are handled by the same instance, it will not try connecting until some time has passed. The default parameters are such that the initial delay is one second, and if after that delay the connection still can't be made, the handler will double the delay each time up to a maximum of 30 seconds.

This behaviour is controlled by the following handler attributes:

- `retryStart` (initial delay, defaulting to 1.0 seconds).
- `retryFactor` (multiplier, defaulting to 2.0).

- `retryMax` (maximum delay, defaulting to 30.0 seconds).

This means that if the remote listener starts up *after* the handler has been used, you could lose messages (since the handler won't even attempt a connection until the delay has elapsed, but just silently drop messages during the delay period).

16.6.9 DatagramHandler

The *DatagramHandler* class, located in the *logging.handlers* module, inherits from *SocketHandler* to support sending logging messages over UDP sockets.

class logging.handlers.**DatagramHandler**(*host*, *port*)

Returns a new instance of the *DatagramHandler* class intended to communicate with a remote machine whose address is given by *host* and *port*.

> **Nota**
>
> As UDP is not a streaming protocol, there is no persistent connection between an instance of this handler and *host*. For this reason, when using a network socket, a DNS lookup might have to be made each time an event is logged, which can introduce some latency into the system. If this affects you, you can do a lookup yourself and initialize this handler using the looked-up IP address rather than the hostname.

Alterado na versão 3.4: If `port` is specified as `None`, a Unix domain socket is created using the value in `host` - otherwise, a UDP socket is created.

emit()

Pickles the record's attribute dictionary and writes it to the socket in binary format. If there is an error with the socket, silently drops the packet. To unpickle the record at the receiving end into a *LogRecord*, use the *makeLogRecord()* function.

makeSocket()

The factory method of *SocketHandler* is here overridden to create a UDP socket (*socket.SOCK_DGRAM*).

send(*s*)

Send a pickled byte-string to a socket. The format of the sent byte-string is as described in the documentation for *SocketHandler.makePickle()*.

16.6.10 SysLogHandler

The *SysLogHandler* class, located in the *logging.handlers* module, supports sending logging messages to a remote or local Unix syslog.

class logging.handlers.**SysLogHandler**(*address=('localhost', SYSLOG_UDP_PORT)*, *facility=LOG_USER*, *socktype=socket.SOCK_DGRAM*, *timeout=None*)

Returns a new instance of the *SysLogHandler* class intended to communicate with a remote Unix machine whose address is given by *address* in the form of a `(host, port)` tuple. If *address* is not specified, `('localhost', 514)` is used. The address is used to open a socket. An alternative to providing a `(host, port)` tuple is providing an address as a string, for example '/dev/log'. In this case, a Unix domain socket is used to send the message to the syslog. If *facility* is not specified, `LOG_USER` is used. The type of socket opened depends on the *socktype* argument, which defaults to *socket.SOCK_DGRAM* and thus opens a UDP socket. To open a TCP socket (for use with the newer syslog daemons such as rsyslog), specify a value of *socket.SOCK_STREAM*. If *timeout* is specified, it sets a timeout (in seconds) for the socket operations. This can help prevent the program from hanging indefinitely if the syslog server is unreachable. By default, *timeout* is `None`, meaning no timeout is applied.

Note that if your server is not listening on UDP port 514, *SysLogHandler* may appear not to work. In that case, check what address you should be using for a domain socket - it's system dependent. For example, on

Linux it's usually '/dev/log' but on OS/X it's '/var/run/syslog'. You'll need to check your platform and use the appropriate address (you may need to do this check at runtime if your application needs to run on several platforms). On Windows, you pretty much have to use the UDP option.

Nota

On macOS 12.x (Monterey), Apple has changed the behaviour of their syslog daemon - it no longer listens on a domain socket. Therefore, you cannot expect *SysLogHandler* to work on this system.

See gh-91070 for more information.

Alterado na versão 3.2: *socktype* was added.

Alterado na versão 3.14: *timeout* foi adicionado.

close()

Closes the socket to the remote host.

createSocket()

Tries to create a socket and, if it's not a datagram socket, connect it to the other end. This method is called during handler initialization, but it's not regarded as an error if the other end isn't listening at this point - the method will be called again when emitting an event, if there is no socket at that point.

Adicionado na versão 3.11.

emit(*record*)

The record is formatted, and then sent to the syslog server. If exception information is present, it is *not* sent to the server.

Alterado na versão 3.2.1: (See: bpo-12168.) In earlier versions, the message sent to the syslog daemons was always terminated with a NUL byte, because early versions of these daemons expected a NUL terminated message - even though it's not in the relevant specification (**RFC 5424**). More recent versions of these daemons don't expect the NUL byte but strip it off if it's there, and even more recent daemons (which adhere more closely to RFC 5424) pass the NUL byte on as part of the message.

To enable easier handling of syslog messages in the face of all these differing daemon behaviours, the appending of the NUL byte has been made configurable, through the use of a class-level attribute, `append_nul`. This defaults to `True` (preserving the existing behaviour) but can be set to `False` on a `SysLogHandler` instance in order for that instance to *not* append the NUL terminator.

Alterado na versão 3.3: (See: bpo-12419.) In earlier versions, there was no facility for an "ident" or "tag" prefix to identify the source of the message. This can now be specified using a class-level attribute, defaulting to `""` to preserve existing behaviour, but which can be overridden on a `SysLogHandler` instance in order for that instance to prepend the ident to every message handled. Note that the provided ident must be text, not bytes, and is prepended to the message exactly as is.

encodePriority(*facility*, *priority*)

Encodes the facility and priority into an integer. You can pass in strings or integers - if strings are passed, internal mapping dictionaries are used to convert them to integers.

The symbolic `LOG_` values are defined in *SysLogHandler* and mirror the values defined in the `sys/syslog.h` header file.

Priorities

Name (string)	Symbolic value
`alert`	LOG_ALERT
`crit` or `critical`	LOG_CRIT
`debug`	LOG_DEBUG
`emerg` or `panic`	LOG_EMERG
`err` ou `error`	LOG_ERR
`info`	LOG_INFO
`notice`	LOG_NOTICE
`warn` or `warning`	LOG_WARNING

Facilities

Name (string)	Symbolic value
`auth`	LOG_AUTH
`authpriv`	LOG_AUTHPRIV
`cron`	LOG_CRON
`daemon`	LOG_DAEMON
`ftp`	LOG_FTP
`kern`	LOG_KERN
`lpr`	LOG_LPR
`mail`	LOG_MAIL
`news`	LOG_NEWS
`syslog`	LOG_SYSLOG
`user`	LOG_USER
`uucp`	LOG_UUCP
`local0`	LOG_LOCAL0
`local1`	LOG_LOCAL1
`local2`	LOG_LOCAL2
`local3`	LOG_LOCAL3
`local4`	LOG_LOCAL4
`local5`	LOG_LOCAL5
`local6`	LOG_LOCAL6
`local7`	LOG_LOCAL7

mapPriority(*levelname*)

Maps a logging level name to a syslog priority name. You may need to override this if you are using custom levels, or if the default algorithm is not suitable for your needs. The default algorithm maps `DEBUG`, `INFO`, `WARNING`, `ERROR` and `CRITICAL` to the equivalent syslog names, and all other level names to 'warning'.

16.6.11 NTEventLogHandler

The *`NTEventLogHandler`* class, located in the *`logging.handlers`* module, supports sending logging messages to a local Windows NT, Windows 2000 or Windows XP event log. Before you can use it, you need Mark Hammond's Win32 extensions for Python installed.

class `logging.handlers.`**NTEventLogHandler**(*appname*, *dllname=None*, *logtype='Application'*)

Returns a new instance of the *`NTEventLogHandler`* class. The *appname* is used to define the application name as it appears in the event log. An appropriate registry entry is created using this name. The *dllname* should give the fully qualified pathname of a .dll or .exe which contains message definitions to hold in the log (if not specified, `'win32service.pyd'` is used - this is installed with the Win32 extensions and contains some basic placeholder message definitions. Note that use of these placeholders will make your event logs big, as the entire message source is held in the log. If you want slimmer logs, you have to pass in the name of your own .dll or .exe which contains the message definitions you want to use in the event log). The *logtype* is one of `'Application'`, `'System'` or `'Security'`, and defaults to `'Application'`.

close()

At this point, you can remove the application name from the registry as a source of event log entries. However, if you do this, you will not be able to see the events as you intended in the Event Log Viewer - it needs to be able to access the registry to get the .dll name. The current version does not do this.

emit(*record*)

Determines the message ID, event category and event type, and then logs the message in the NT event log.

getEventCategory(*record*)

Returns the event category for the record. Override this if you want to specify your own categories. This version returns 0.

getEventType(*record*)

Returns the event type for the record. Override this if you want to specify your own types. This version does a mapping using the handler's typemap attribute, which is set up in `__init__()` to a dictionary which contains mappings for `DEBUG`, `INFO`, `WARNING`, `ERROR` and `CRITICAL`. If you are using your own levels, you will either need to override this method or place a suitable dictionary in the handler's *typemap* attribute.

getMessageID(*record*)

Returns the message ID for the record. If you are using your own messages, you could do this by having the *msg* passed to the logger being an ID rather than a format string. Then, in here, you could use a dictionary lookup to get the message ID. This version returns 1, which is the base message ID in `win32service.pyd`.

16.6.12 SMTPHandler

The *SMTPHandler* class, located in the *logging.handlers* module, supports sending logging messages to an email address via SMTP.

class logging.handlers.**SMTPHandler**(*mailhost*, *fromaddr*, *toaddrs*, *subject*, *credentials=None*, *secure=None*, *timeout=1.0*)

Returns a new instance of the *SMTPHandler* class. The instance is initialized with the from and to addresses and subject line of the email. The *toaddrs* should be a list of strings. To specify a non-standard SMTP port, use the (host, port) tuple format for the *mailhost* argument. If you use a string, the standard SMTP port is used. If your SMTP server requires authentication, you can specify a (username, password) tuple for the *credentials* argument.

To specify the use of a secure protocol (TLS), pass in a tuple to the *secure* argument. This will only be used when authentication credentials are supplied. The tuple should be either an empty tuple, or a single-value tuple with the name of a keyfile, or a 2-value tuple with the names of the keyfile and certificate file. (This tuple is passed to the *smtplib.SMTP.starttls()* method.)

A timeout can be specified for communication with the SMTP server using the *timeout* argument.

Alterado na versão 3.3: Added the *timeout* parameter.

emit(*record*)

Formats the record and sends it to the specified addressees.

getSubject(*record*)

If you want to specify a subject line which is record-dependent, override this method.

16.6.13 MemoryHandler

The *MemoryHandler* class, located in the *logging.handlers* module, supports buffering of logging records in memory, periodically flushing them to a *target* handler. Flushing occurs whenever the buffer is full, or when an event of a certain severity or greater is seen.

MemoryHandler is a subclass of the more general *BufferingHandler*, which is an abstract class. This buffers logging records in memory. Whenever each record is added to the buffer, a check is made by calling `shouldFlush()` to see if the buffer should be flushed. If it should, then `flush()` is expected to do the flushing.

class logging.handlers.**BufferingHandler**(*capacity*)

Initializes the handler with a buffer of the specified capacity. Here, *capacity* means the number of logging records buffered.

emit(*record*)

Append the record to the buffer. If *shouldFlush()* returns true, call *flush()* to process the buffer.

flush()

For a *BufferingHandler* instance, flushing means that it sets the buffer to an empty list. This method can be overwritten to implement more useful flushing behavior.

shouldFlush(*record*)

Return `True` if the buffer is up to capacity. This method can be overridden to implement custom flushing strategies.

class logging.handlers.**MemoryHandler**(*capacity*, *flushLevel=ERROR*, *target=None*, *flushOnClose=True*)

Returns a new instance of the *MemoryHandler* class. The instance is initialized with a buffer size of *capacity* (number of records buffered). If *flushLevel* is not specified, `ERROR` is used. If no *target* is specified, the target will need to be set using *setTarget()* before this handler does anything useful. If *flushOnClose* is specified as `False`, then the buffer is *not* flushed when the handler is closed. If not specified or specified as `True`, the previous behaviour of flushing the buffer will occur when the handler is closed.

Alterado na versão 3.6: The *flushOnClose* parameter was added.

close()

Calls *flush()*, sets the target to `None` and clears the buffer.

flush()

For a *MemoryHandler* instance, flushing means just sending the buffered records to the target, if there is one. The buffer is also cleared when buffered records are sent to the target. Override if you want different behavior.

setTarget(*target*)

Sets the target handler for this handler.

shouldFlush(*record*)

Checks for buffer full or a record at the *flushLevel* or higher.

16.6.14 HTTPHandler

The *HTTPHandler* class, located in the *logging.handlers* module, supports sending logging messages to a web server, using either `GET` or `POST` semantics.

class logging.handlers.**HTTPHandler**(*host*, *url*, *method='GET'*, *secure=False*, *credentials=None*, *context=None*)

Returns a new instance of the *HTTPHandler* class. The *host* can be of the form `host:port`, should you need to use a specific port number. If no *method* is specified, `GET` is used. If *secure* is true, a HTTPS connection will be used. The *context* parameter may be set to a *ssl.SSLContext* instance to configure the SSL settings used for the HTTPS connection. If *credentials* is specified, it should be a 2-tuple consisting of userid and password, which will be placed in a HTTP 'Authorization' header using Basic authentication. If you specify credentials, you should also specify secure=True so that your userid and password are not passed in cleartext across the wire.

Alterado na versão 3.5: The *context* parameter was added.

mapLogRecord(*record*)

Provides a dictionary, based on `record`, which is to be URL-encoded and sent to the web server. The default implementation just returns `record.__dict__`. This method can be overridden if e.g. only a subset of *LogRecord* is to be sent to the web server, or if more specific customization of what's sent to the server is required.

emit(*record*)

Sends the record to the web server as a URL-encoded dictionary. The *mapLogRecord()* method is used to convert the record to the dictionary to be sent.

> **Nota**
>
> Since preparing a record for sending it to a web server is not the same as a generic formatting operation, using *setFormatter()* to specify a *Formatter* for a *HTTPHandler* has no effect. Instead of calling *format()*, this handler calls *mapLogRecord()* and then *urllib.parse.urlencode()* to encode the dictionary in a form suitable for sending to a web server.

16.6.15 QueueHandler

Adicionado na versão 3.2.

The *QueueHandler* class, located in the *logging.handlers* module, supports sending logging messages to a queue, such as those implemented in the *queue* or *multiprocessing* modules.

Along with the *QueueListener* class, *QueueHandler* can be used to let handlers do their work on a separate thread from the one which does the logging. This is important in web applications and also other service applications where threads servicing clients need to respond as quickly as possible, while any potentially slow operations (such as sending an email via *SMTPHandler*) are done on a separate thread.

class logging.handlers.**QueueHandler**(*queue*)

Returns a new instance of the *QueueHandler* class. The instance is initialized with the queue to send messages to. The *queue* can be any queue-like object; it's used as-is by the *enqueue()* method, which needs to know how to send messages to it. The queue is not *required* to have the task tracking API, which means that you can use *SimpleQueue* instances for *queue*.

> **Nota**
>
> If you are using *multiprocessing*, you should avoid using *SimpleQueue* and instead use *multiprocessing.Queue*.

> **Aviso**
>
> The *multiprocessing* module uses an internal logger created and accessed via *get_logger()*. *multiprocessing.Queue* will log `DEBUG` level messages upon items being queued. If those log messages are processed by a *QueueHandler* using the same *multiprocessing.Queue* instance, it will cause a deadlock or infinite recursion.

emit(*record*)

Enqueues the result of preparing the LogRecord. Should an exception occur (e.g. because a bounded queue has filled up), the *handleError()* method is called to handle the error. This can result in the record silently being dropped (if *logging.raiseExceptions* is `False`) or a message printed to `sys.stderr` (if *logging.raiseExceptions* is `True`).

prepare(*record*)

Prepares a record for queuing. The object returned by this method is enqueued.

The base implementation formats the record to merge the message, arguments, exception and stack information, if present. It also removes unpickleable items from the record in-place. Specifically, it overwrites the record's `msg` and `message` attributes with the merged message (obtained by calling the handler's *`format()`* method), and sets the `args`, `exc_info` and `exc_text` attributes to `None`.

You might want to override this method if you want to convert the record to a dict or JSON string, or send a modified copy of the record while leaving the original intact.

> **Nota**
>
> The base implementation formats the message with arguments, sets the `message` and `msg` attributes to the formatted message and sets the `args` and `exc_text` attributes to `None` to allow pickling and to prevent further attempts at formatting. This means that a handler on the *`QueueListener`* side won't have the information to do custom formatting, e.g. of exceptions. You may wish to subclass `QueueHandler` and override this method to e.g. avoid setting `exc_text` to `None`. Note that the `message` / `msg` / `args` changes are related to ensuring the record is pickleable, and you might or might not be able to avoid doing that depending on whether your `args` are pickleable. (Note that you may have to consider not only your own code but also code in any libraries that you use.)

`enqueue`(*record*)

Enqueues the record on the queue using `put_nowait()`; you may want to override this if you want to use blocking behaviour, or a timeout, or a customized queue implementation.

`listener`

When created via configuration using *`dictConfig()`*, this attribute will contain a *`QueueListener`* instance for use with this handler. Otherwise, it will be `None`.

Adicionado na versão 3.12.

16.6.16 QueueListener

Adicionado na versão 3.2.

The *`QueueListener`* class, located in the *`logging.handlers`* module, supports receiving logging messages from a queue, such as those implemented in the *`queue`* or *`multiprocessing`* modules. The messages are received from a queue in an internal thread and passed, on the same thread, to one or more handlers for processing. While *`QueueListener`* is not itself a handler, it is documented here because it works hand-in-hand with *`QueueHandler`*.

Along with the *`QueueHandler`* class, *`QueueListener`* can be used to let handlers do their work on a separate thread from the one which does the logging. This is important in web applications and also other service applications where threads servicing clients need to respond as quickly as possible, while any potentially slow operations (such as sending an email via *`SMTPHandler`*) are done on a separate thread.

class `logging.handlers.`**`QueueListener`**(*queue*, **handlers*, *respect_handler_level=False*)

Returns a new instance of the *`QueueListener`* class. The instance is initialized with the queue to send messages to and a list of handlers which will handle entries placed on the queue. The queue can be any queue--like object; it's passed as-is to the *`dequeue()`* method, which needs to know how to get messages from it. The queue is not *required* to have the task tracking API (though it's used if available), which means that you can use *`SimpleQueue`* instances for *queue*.

> **Nota**
>
> If you are using *`multiprocessing`*, you should avoid using *`SimpleQueue`* and instead use *`multiprocessing.Queue`*.

If `respect_handler_level` is `True`, a handler's level is respected (compared with the level for the message) when deciding whether to pass messages to that handler; otherwise, the behaviour is as in previous Python versions - to always pass each message to each handler.

Alterado na versão 3.5: The `respect_handler_level` argument was added.

Alterado na versão 3.14: *`QueueListener`* can now be used as a context manager via `with`. When entering the context, the listener is started. When exiting the context, the listener is stopped. *`__enter__()`* returns the *`QueueListener`* object.

`dequeue`(*block*)

Dequeues a record and return it, optionally blocking.

The base implementation uses `get()`. You may want to override this method if you want to use timeouts or work with custom queue implementations.

`prepare`(*record*)

Prepare a record for handling.

This implementation just returns the passed-in record. You may want to override this method if you need to do any custom marshalling or manipulation of the record before passing it to the handlers.

`handle`(*record*)

Handle a record.

This just loops through the handlers offering them the record to handle. The actual object passed to the handlers is that which is returned from *`prepare()`*.

`start`()

Starts the listener.

This starts up a background thread to monitor the queue for LogRecords to process.

Alterado na versão 3.14: Raises *`RuntimeError`* if called and the listener is already running.

`stop`()

Stops the listener.

This asks the thread to terminate, and then waits for it to do so. Note that if you don't call this before your application exits, there may be some records still left on the queue, which won't be processed.

`enqueue_sentinel`()

Writes a sentinel to the queue to tell the listener to quit. This implementation uses `put_nowait()`. You may want to override this method if you want to use timeouts or work with custom queue implementations.

Adicionado na versão 3.3.

Ver também

Módulo *`logging`*
Referência da API para o módulo de logging.

Módulo *`logging.config`*
API de configuração para o módulo logging.

16.7 `platform` — Acesso aos dados de identificação da plataforma subjacente

Código-fonte: Lib/platform.py

Nota

Plataformas específicas listadas em ordem alfabética, com Linux incluído na seção Unix.

16.7.1 Multiplataforma

platform.**architecture**(*executable=sys.executable*, *bits=''*, *linkage=''*)

Consulta o executável fornecido (o padrão é o binário do interpretador Python) para obter várias informações de arquitetura.

Retorna uma tupla `(bits, linkage)` que contém informações sobre a arquitetura de bits e o formato de linkage usado para o executável. Ambos os valores são retornados como strings.

Valores que não podem ser determinados são retornados como fornecidos pelas predefinições de parâmetros. Se bits for fornecido como `''`, o `sizeof(pointer)` (ou `sizeof(long)` na versão Python < 1.5.2) é usado como indicador para o tamanho de ponteiro suportado.

A função depende do comando `file` do sistema para fazer o trabalho real. Isso está disponível na maioria, se não em todas as plataformas Unix e algumas plataformas não Unix e somente se o executável apontar para o interpretador Python. Padrões razoáveis são usados quando as necessidades acima não são atendidas.

> **Nota**
>
> No macOS (e talvez em outras plataformas), arquivos executáveis podem ser arquivos universais contendo diversas arquiteturas.
>
> Para obter a "característica de 64 bits" do interpretador atual, é mais confiável consultar o atributo *sys.maxsize*:
>
> ```
> is_64bits = sys.maxsize > 2**32
> ```

platform.**machine**()

Retorna o tipo de máquina, por exemplo, `'AMD64'`. Uma string vazia é retornada se o valor não puder ser determinado.

platform.**node**()

Retorna o nome de rede do computador (pode não ser totalmente qualificado!). Uma string vazia é retornada se o valor não puder ser determinado.

platform.**platform**(*aliased=False*, *terse=False*)

Retorna uma única string identificando a plataforma subjacente com o máximo de informações úteis possível.

A saída é pretendida para ser *legível por humanos* em vez de analisável por máquinas. Pode parecer diferente em diferentes plataformas e isso é intencional.

Se *aliased* for true, a função usará apelidos para várias plataformas que relatam nomes de sistemas que diferem de seus nomes comuns, por exemplo, SunOS será relatado como Solaris. A função *system_alias()* é usada para implementar isso.

Definir *terse* como true faz com que a função retorne apenas o mínimo absoluto de informações necessárias para identificar a plataforma.

Alterado na versão 3.8: No macOS, a função agora usa *mac_ver()*, se retornar uma string de versão não vazia, para obter a versão do macOS em vez da versão do darwin.

platform.**processor**()

Retorna o nome (real) do processador, por exemplo `'amdk6'`.

Uma string vazia é retornada se o valor não puder ser determinado. Note que muitas plataformas não fornecem essas informações ou simplesmente retornam o mesmo valor que para *machine()*. O NetBSD faz isso.

platform.**python_build**()

Retorna uma tupla `(buildno, builddate)` informando o número e a data da construção do Python como strings.

platform.**python_compiler**()

Retorna uma string identificando o compilador usado para compilar Python.

platform.**python_branch**()

Retorna uma string identificando a branch SCM da implementação do Python.

platform.**python_implementation**()

Retorna uma string identificando a implementação do Python. Os valores de retorno possíveis são: 'CPython', 'IronPython', 'Jython', 'PyPy'.

platform.**python_revision**()

Retorna uma string que identifica a revisão do SCM da implementação Python.

platform.**python_version**()

Retorna a versão do Python como string `'major.minor.patchlevel'`.

Observe que, diferentemente do `sys.version` do Python, o valor retornado sempre incluirá o patchlevel (o padrão é 0).

platform.**python_version_tuple**()

Retorna a versão do Python como tupla `(major, minor, patchlevel)` de strings.

Observe que, diferentemente do `sys.version` do Python, o valor retornado sempre incluirá o patchlevel (o padrão é `'0'`).

platform.**release**()

Retorna a versão do sistema, por exemplo, `'2.2.0'` ou `'NT'`. Uma string vazia é retornada se o valor não puder ser determinado.

platform.**system**()

Retorna o nome do sistema/SO, como `'Linux'`, `'Darwin'`, `'Java'`, `'Windows'`. Uma string vazia é retornada se o valor não puder ser determinado.

No iOS e Android, isso retorna o nome do SO voltado para o usuário (por exemplo, `'iOS'`, `'iPadOS'` ou `'Android'`). Para obter o nome do kernel (`'Darwin'` ou `'Linux'`), use *os.uname()*.

platform.**system_alias**(*system*, *release*, *version*)

Retorna `(sistema, release, versão)` com apelido para nomes comuns de marketing usados para alguns sistemas. Ele também faz alguma reordenação das informações em alguns casos em que, de outra forma, causaria confusão.

platform.**version**()

Retorna a versão de lançamento do sistema, por exemplo, `'#3 on degas'`. Uma string vazia é retornada se o valor não puder ser determinado.

No iOS e Android, esta é a versão do SO voltada para o usuário. Para obter a versão do kernel Darwin ou Linux, use *os.uname()*.

platform.**uname**()

Interface para uname razoavelmente portátil. Retorna um *namedtuple()* contendo seis atributos: *system*, *node*, *release*, *version*, *machine* e *processor*.

processor é resolvido tardiamente, sob demanda.

Nota: os dois primeiros nomes de atributos diferem dos nomes apresentados por *os.uname()*, onde são chamados `sysname` e `nodename`.

As entradas que não podem ser determinadas são definidas como `''`.

Alterado na versão 3.3: O resultado mudou de uma tupla para *namedtuple()*.

Alterado na versão 3.9: *processor* é resolvido tarde em vez de imediatamente.

platform.**invalidate_caches**()

Limpa o cache interno de informações, como *uname()*. Isso normalmente é útil quando o *node()* da plataforma é alterado por um processo externo e é necessário recuperar o valor atualizado.

Adicionado na versão 3.14.

16.7.2 Plataforma Java

`platform.`**`java_ver`**(*release=''*, *vendor=''*, *vminfo=('', '', '')*, *osinfo=('', '', '')*)

Interface de versão para Jython.

Retorna uma tupla `(release, vendor, vminfo, osinfo)` com *vminfo* sendo uma tupla `(vm_name, vm_release, vm_vendor)` e *osinfo* sendo uma tupla `(os_name, os_version, os_arch)`. Valores que não podem ser determinados são definidos para os padrões fornecidos como parâmetros (que são todos padrão para `''`).

Deprecated since version 3.13, will be removed in version 3.15: Ele não foi testado em grande parte, tinha uma API confusa e só era útil para suporte ao Jython.

16.7.3 Plataforma Windows

`platform.`**`win32_ver`**(*release=''*, *version=''*, *csd=''*, *ptype=''*)

Obtém informações adicionais sobre a versão do Registro do Windows e retorna uma tupla `(release, version, csd, ptype)` referente à versão do SO, número da versão, nível do CSD (service pack) e tipo de sistema operacional (processador múltiplo/único). Os valores que não podem ser determinados são definidos para os padrões fornecidos como parâmetros (que são todos padrão para uma string vazia).

Como dica: *ptype* é `'Uniprocessor Free'` em máquinas NT de processador único e `'Multiprocessor Free'` em máquinas de multiprocessador. O `'Free'` se refere à versão do sistema operacional que está livre de código de depuração. Ele também pode declarar `'Checked'`, o que significa que a versão do sistema operacional usa código de depuração, ou seja, código que verifica argumentos, intervalos, etc.

`platform.`**`win32_edition`**()

Retorna uma string representando a edição atual do Windows, ou `None` se o valor não puder ser determinado. Os valores possíveis incluem, mas não estão limitados a `'Enterprise'`, `'IoTUAP'`, `'ServerStandard'` e `'nanoserver'`.

Adicionado na versão 3.8.

`platform.`**`win32_is_iot`**()

Retorna `True` se a edição do Windows retornada por *win32_edition()* for reconhecida como uma edição IoT.

Adicionado na versão 3.8.

16.7.4 Plataforma macOS

`platform.`**`mac_ver`**(*release=''*, *versioninfo=('', '', '')*, *machine=''*)

Obtém informações sobre a versão do macOS e retorna-as como tupla `(release, versioninfo, machine)`, sendo *versioninfo* uma tupla `(version, dev_stage, non_release_version)`.

Entradas que não podem ser determinadas são definidas como `''`. Todas as entradas de tupla são strings.

16.7.5 Plataforma iOS

`platform.`**`ios_ver`**(*system=''*, *release=''*, *model=''*, *is_simulator=False*)

Obtém informações sobre a versão do iOS e retorna-as como *namedtuple()* com os seguintes atributos:

- `system` é o nome do sistema operacional; `'iOS'` ou `'iPadOS'`.
- `release` é o número da versão do iOS como uma string (por exemplo, `'17.2'`).
- `model` é o identificador do modelo do dispositivo; será uma string como `'iPhone13,2'` para um dispositivo físico ou `'iPhone'` em um simulador.
- `is_simulator` é um booleano que descreve se o aplicativo está sendo executado em um simulador ou em um dispositivo físico.

As entradas que não podem ser determinadas são definidas para os padrões fornecidos como parâmetros.

16.7.6 Plataformas Unix

platform.**libc_ver**(*executable=sys.executable*, *lib=''*, *version=''*, *chunksize=16384*)

Tenta determinar a versão libc contra a qual o executável do arquivo (padrão para o interpretador Python) está vinculado. Retorna uma tupla de strings `(lib, version)` que tem como padrão os parâmetros fornecidos caso a pesquisa falhe.

Observe que esta função tem conhecimento profundo de como diferentes versões da libc adicionam símbolos ao executável e provavelmente só pode ser usada para executáveis compilados usando **gcc**.

O arquivo é lido e escaneado em blocos de *chunksize* bytes.

16.7.7 Plataformas Linux

platform.**freedesktop_os_release**()

Obtém a identificação do sistema operacional do arquivo `os-release` e retorna-a como um dict. O arquivo `os-release` é um padrão freedesktop.org e está disponível na maioria das distribuições Linux. Uma exceção notável é o Android e as distribuições baseadas em Android.

Levanta *OSError* ou subclasse quando nem `/etc/os-release` nem `/usr/lib/os-release` podem ser lidos.

Em caso de sucesso, a função retorna um dicionário onde chaves e valores são strings. Valores têm seus caracteres especiais como `"` e `$` sem aspas. Os campos `NAME`, `ID` e `PRETTY_NAME` são sempre definidos de acordo com o padrão. Todos os outros campos são opcionais. Os fornecedores podem incluir campos adicionais.

Note que campos como `NAME`, `VERSION` e `VARIANT` são strings adequadas para apresentação aos usuários. Os programas devem usar campos como `ID`, `ID_LIKE`, `VERSION_ID` ou `VARIANT_ID` para identificar distribuições Linux.

Exemplo:

```
def get_like_distro():
    info = platform.freedesktop_os_release()
    ids = [info["ID"]]
    if "ID_LIKE" in info:
        # ids são separados por espaço e ordenado por precedência
        ids.extend(info["ID_LIKE"].split())
    return ids
```

Adicionado na versão 3.10.

16.7.8 Plataforma Android

platform.**android_ver**(*release=''*, *api_level=0*, *manufacturer=''*, *model=''*, *device=''*, *is_emulator=False*)

Obtém informações do dispositivo Android. Retorna um *namedtuple()* com os seguintes atributos. Valores que não podem ser determinados são definidos para os padrões fornecidos como parâmetros.

- `release` - Versão do Android, como uma string (por exemplo, `"14"`).
- `api_level` - Nível de API do dispositivo em execução, como um inteiro (por exemplo, `34` para Android 14). Para obter o nível de API contra o qual o Python foi criado, consulte *sys.getandroidapilevel()*.
- `manufacturer` - Nome do fabricante.
- `model` - Nome do modelo – normalmente o nome de marketing ou o número do modelo.
- `device` - Nome do dispositivo – normalmente o número do modelo ou um codinome.
- `is_emulator` - `True` se o dispositivo for um emulador; `False` se for um dispositivo físico.

O Google mantém uma lista de nomes de modelos e dispositivos conhecidos.

Adicionado na versão 3.13.

16.7.9 Uso na linha de comando

platform também pode ser invocado diretamente usando a opção `-m` do interpretador:

```
python -m platform [--terse] [--nonaliased] [{nonaliased,terse} ...]
```

As seguintes opções são aceitas:

--terse

Exibe informações concisas sobre a plataforma. Isso equivale a chamar *platform.platform()* com o argumento *terse* definido como `True`.

--nonaliased

Exibe informações da plataforma sem alias de nome do sistema/SO. Isso equivale a chamar *platform.platform()* com o argumento *aliased* definido como `True`.

Você também pode passar um ou mais argumentos posicionais (`terse`, `nonaliased`) para controlar explicitamente o formato de saída. Eles se comportam de forma semelhante às suas opções correspondentes.

16.8 errno — Símbolos padrão do sistema errno

Este módulo disponibiliza símbolos de sistema padrão `errno`. O valor de cada símbolo é o valor inteiro correspondente. Os nomes e descrições são emprestados de `linux/include/errno.h`, que deve ser tudo inclusivo.

errno.**errorcode**

Dicionário que fornece um mapeamento do valor errno para o nome da string no sistema subjacente. Por exemplo, `errno.errorcode[errno.EPERM]` mapeia para `'EPERM'`.

Para traduzir um código de erro numérico em uma mensagem de erro, use *os.strerror()*.

Da lista a seguir, símbolos que não são usados na plataforma atual não são definidos pelo módulo. A lista específica de símbolos definidos está disponível como `errno.errorcode.keys()`. Os símbolos disponíveis podem incluir:

errno.**EPERM**

Operação não permitida. Este erro é mapeado para a exceção *PermissionError*.

errno.**ENOENT**

Arquivo ou diretório inexistente. Este erro é mapeado para a exceção *FileNotFoundError*.

errno.**ESRCH**

Processo inexistente. Este erro é mapeado para a exceção *ProcessLookupError*.

errno.**EINTR**

Chamada de sistema interrompida. Este erro é mapeado para a exceção *InterruptedError*.

errno.**EIO**

Erro de E/S

errno.**ENXIO**

Endereço ou dispositivo inexistente

errno.**E2BIG**

Lista de argumentos muito longa

errno.**ENOEXEC**
: Erro no formato exec

errno.**EBADF**
: Descritor de arquivo inválido

errno.**ECHILD**
: Não há processos filhos. Este erro é mapeado para a exceção *ChildProcessError*.

errno.**EAGAIN**
: Tente novamente. Este erro é mapeado para a exceção *BlockingIOError*.

errno.**ENOMEM**
: Memória insuficiente

errno.**EACCES**
: Permissão negada. Este erro é mapeado para a exceção *PermissionError*.

errno.**EFAULT**
: Endereço inválido

errno.**ENOTBLK**
: Dispositivo de bloco requerido

errno.**EBUSY**
: Dispositivo ou recurso ocupado

errno.**EEXIST**
: Arquivo existe. Este erro é mapeado para a exceção *FileExistsError*.

errno.**EXDEV**
: Link entre dispositivos inválido

errno.**ENODEV**
: Dispositivo inexistente

errno.**ENOTDIR**
: Não é um diretório. Este erro é mapeado para a exceção *NotADirectoryError*.

errno.**EISDIR**
: É um diretório. Este erro é mapeado para a exceção *IsADirectoryError*.

errno.**EINVAL**
: Argumento inválido

errno.**ENFILE**
: Estouro de tabela de arquivos

errno.**EMFILE**
: Muitos arquivos abertos

errno.**ENOTTY**
: Not a typewriter

errno.**ETXTBSY**
: Arquivo texto ocupado

errno.**EFBIG**
: Arquivo muito grande

errno.**ENOSPC**
: Não há espaço disponível no dispositivo

errno.**ESPIPE**
: Procura ilegal

errno.**EROFS**
: Sistema de arquivos de somente leitura

errno.**EMLINK**
: Muitos links

errno.**EPIPE**
: Pipe quebrado. Este erro é mapeado para a exceção *BrokenPipeError*.

errno.**EDOM**
: Argumento matemático fora do domínio da função

errno.**ERANGE**
: Resultado matemático não representável

errno.**EDEADLK**
: Ocorreria um impasse (deadlock) de recursos

errno.**ENAMETOOLONG**
: Nome de arquivo muito longo

errno.**ENOLCK**
: Nenhuma trava de registro disponível

errno.**ENOSYS**
: Função não implementada

errno.**ENOTEMPTY**
: Diretório não vazio

errno.**ELOOP**
: Foram encontrados muitos links simbólicos

errno.**EWOULDBLOCK**
: Operation causaria bloqueio. Este erro é mapeado para a exceção *BlockingIOError*.

errno.**ENOMSG**
: Nenhuma mensagem do tipo desejado

errno.**EIDRM**
: Identificador removido

errno.**ECHRNG**
: Número do canal fora do intervalo

errno.**EL2NSYNC**
: Não sincronizado nível 2

errno.**EL3HLT**
: Parada de sistema nível 3

errno.**EL3RST**
: Reinicialização nível 3

errno.**ELNRNG**
: Número de link fora da faixa

errno.**EUNATCH**
: Driver de protocolo não anexado

errno.**ENOCSI**
: Não há estrutura CSI disponível

errno.**EL2HLT**
: Parada de sistema nível 2

errno.**EBADE**
: Troca inválida

errno.**EBADR**
: Descritor de requisição inválido

errno.**EXFULL**
: Troca completa

errno.**ENOANO**
: Sem anode

errno.**EBADRQC**
: Código de requisição inválido

errno.**EBADSLT**
: Slot inválido

errno.**EDEADLOCK**
: Erro de impasse em travamento de arquivo

errno.**EBFONT**
: Formato inválido do arquivo de fonte

errno.**ENOSTR**
: Dispositivo não é um stream

errno.**ENODATA**
: Não há dados disponíveis

errno.**ETIME**
: Temporizador expirado

errno.**ENOSR**
: Sem recursos de streams

errno.**ENONET**
: A máquina não está na rede

errno.**ENOPKG**
: Pacote não instalado

errno.**EREMOTE**
: O objeto é remoto

errno.**ENOLINK**
: A conexão foi rompida

errno.**EADV**
: Erro de anúncio

errno.**ESRMNT**
: Erro Srmount

errno.**ECOMM**
: Erro de comunicação ao enviar

errno.**EPROTO**
: Erro de Protocolo

errno.**EMULTIHOP**
: Tentativa de hops múltiplos

errno.**EDOTDOT**
: Erro específico de RFS

errno.**EBADMSG**
: Não é uma mensagem de dados

errno.**EOVERFLOW**
: Valor muito grande para o tipo de dados definido

errno.**ENOTUNIQ**
: O nome não é único na rede

errno.**EBADFD**
: Descritor de arquivo em mal estado

errno.**EREMCHG**
: Endereço remoto mudou

errno.**ELIBACC**
: Não é possível acessar uma biblioteca compartilhada necessária

errno.**ELIBBAD**
: Acessando uma biblioteca compartilhado corrompida

errno.**ELIBSCN**
: Seção .lib corrompida em a.out

errno.**ELIBMAX**
: Tentando ligar em muitas bibliotecas compartilhadas

errno.**ELIBEXEC**
: Não é possível executar uma biblioteca compartilhada diretamente

errno.**EILSEQ**
: Sequência de bytes ilegal

errno.**ERESTART**
: Chamada de sistema interrompida deve ser reiniciada

errno.**ESTRPIPE**
: Erro de fluxos de pipe

errno.**EUSERS**
: Muitos usuários

errno.**ENOTSOCK**
: Operação socket em um arquivo não-socket

errno.**EDESTADDRREQ**
: Endereço de destino necessário

errno.**EMSGSIZE**
: Mensagem muito longa

errno.**EPROTOTYPE**
: Tipo errado de protocolo para socket

errno.**ENOPROTOOPT**
Protocolo não disponível

errno.**EPROTONOSUPPORT**
Protocolo sem suporte

errno.**ESOCKTNOSUPPORT**
Tipo socket sem suporte

errno.**EOPNOTSUPP**
Operação sem suporte na extremidade do transporte

errno.**ENOTSUP**
Operação sem suporte

Adicionado na versão 3.2.

errno.**EPFNOSUPPORT**
Família de protocolo sem suporte

errno.**EAFNOSUPPORT**
Família de endereços sem suporte pelo protocolo

errno.**EADDRINUSE**
Endereço já em uso

errno.**EADDRNOTAVAIL**
Não é possível acessar o endereço requisitado

errno.**ENETDOWN**
A rede não responde

errno.**ENETUNREACH**
A rede está fora de alcance

errno.**ENETRESET**
A rede desconectou-se ao reiniciar

errno.**ECONNABORTED**
O software causou a interrupção da conexão. Este erro é mapeado para a exceção *ConnectionAbortedError*.

errno.**ECONNRESET**
Conexão fechada pela outra ponta. Este erro é mapeado para a exceção *ConnectionResetError*.

errno.**ENOBUFS**
Não há espaço de buffer disponível

errno.**EISCONN**
A extremidade do transporte já está conectada

errno.**ENOTCONN**
A extremidade do transporte não está conectada

errno.**ESHUTDOWN**
Cannot send after transport endpoint shutdown. This error is mapped to the exception *BrokenPipeError*.

errno.**ETOOMANYREFS**
Too many references: cannot splice

errno.**ETIMEDOUT**
Connection timed out. This error is mapped to the exception *TimeoutError*.

errno.**ECONNREFUSED**
Connection refused. This error is mapped to the exception *ConnectionRefusedError*.

errno.**EHOSTDOWN**
Host is down

errno.**EHOSTUNREACH**
No route to host

errno.**EHWPOISON**
Memory page has hardware error.

Adicionado na versão 3.14.

errno.**EALREADY**
Operation already in progress. This error is mapped to the exception *BlockingIOError*.

errno.**EINPROGRESS**
Operation now in progress. This error is mapped to the exception *BlockingIOError*.

errno.**ESTALE**
Stale NFS file handle

errno.**EUCLEAN**
Structure needs cleaning

errno.**ENOTNAM**
Not a XENIX named type file

errno.**ENAVAIL**
No XENIX semaphores available

errno.**EISNAM**
É um arquivo de tipo nomeado

errno.**EREMOTEIO**
Erro de E/S remoto

errno.**EDQUOT**
Quota exceeded

errno.**EQFULL**
Interface output queue is full

Adicionado na versão 3.11.

errno.**ENOMEDIUM**
No medium found

errno.**EMEDIUMTYPE**
Wrong medium type

errno.**ENOKEY**
Required key not available

errno.**EKEYEXPIRED**
Key has expired

errno.**EKEYREVOKED**
Key has been revoked

errno.**EKEYREJECTED**
Key was rejected by service

errno.**ERFKILL**
Operation not possible due to RF-kill

errno.**ELOCKUNMAPPED**
Locked lock was unmapped

errno.**ENOTACTIVE**
Facility is not active

errno.**EAUTH**
Authentication error

Adicionado na versão 3.2.

errno.**EBADARCH**
Bad CPU type in executable

Adicionado na versão 3.2.

errno.**EBADEXEC**
Bad executable (or shared library)

Adicionado na versão 3.2.

errno.**EBADMACHO**
Malformed Mach-o file

Adicionado na versão 3.2.

errno.**EDEVERR**
Device error

Adicionado na versão 3.2.

errno.**EFTYPE**
Inappropriate file type or format

Adicionado na versão 3.2.

errno.**ENEEDAUTH**
Need authenticator

Adicionado na versão 3.2.

errno.**ENOATTR**
Attribute not found

Adicionado na versão 3.2.

errno.**ENOPOLICY**
Policy not found

Adicionado na versão 3.2.

errno.**EPROCLIM**
Too many processes

Adicionado na versão 3.2.

errno.**EPROCUNAVAIL**
Bad procedure for program

Adicionado na versão 3.2.

errno.**EPROGMISMATCH**
Program version wrong

Adicionado na versão 3.2.

errno.**EPROGUNAVAIL**
RPC prog. not avail

Adicionado na versão 3.2.

errno.**EPWROFF**
Device power is off

Adicionado na versão 3.2.

errno.**EBADRPC**
RPC struct is bad

Adicionado na versão 3.2.

errno.**ERPCMISMATCH**
RPC version wrong

Adicionado na versão 3.2.

errno.**ESHLIBVERS**
Shared library version mismatch

Adicionado na versão 3.2.

errno.**ENOTCAPABLE**
Capabilities insufficient. This error is mapped to the exception *PermissionError*.

Disponibilidade: WASI, FreeBSD

Adicionado na versão 3.11.1.

errno.**ECANCELED**
Operation canceled

Adicionado na versão 3.2.

errno.**EOWNERDEAD**
Owner died

Adicionado na versão 3.2.

errno.**ENOTRECOVERABLE**
State not recoverable

Adicionado na versão 3.2.

16.9 ctypes — Uma biblioteca de funções externas para Python

Código-fonte: Lib/ctypes

---

ctypes é uma biblioteca de funções externas para Python. Ela fornece tipos de dados compatíveis com C e permite funções de chamada em DLLs ou bibliotecas compartilhadas. Ela pode ser usada para agrupar essas bibliotecas em Python puro.

16.9.1 Tutorial ctypes

Nota: Os exemplos de código neste tutorial usam *doctest* para garantir que eles realmente funcionem. Como algumas amostras de código se comportam de maneira diferente no Linux, Windows ou macOS, elas contêm diretrizes de doctest nos comentários.

Nota: Alguns exemplos de código fazem referência ao tipo ctypes *c_int*. Em plataformas onde `sizeof(long) == sizeof(int)` é um apelido para *c_long*. Então, você não deve ficar confuso se *c_long* for impresso se você esperaria *c_int* — eles são, na verdade, o mesmo tipo.

Carregando bibliotecas de links dinâmicos

ctypes exporta o *cdll* e, no Windows, os objetos *windll* e *oledll* para carregar bibliotecas de vínculo dinâmico.

Você carrega bibliotecas acessando-as como atributos desses objetos. *cdll* carrega bibliotecas que exportam funções usando a convenção de chamada padrão `cdecl`, enquanto bibliotecas *windll* chamam funções usando a convenção de chamada `stdcall`. *oledll* também usa a convenção de chamada `stdcall` e assume que as funções retornam um código de erro do Windows `HRESULT`. O código de erro é usado para levantar automaticamente uma exceção *OSError* quando a chamada da função falha.

Alterado na versão 3.3: Erros do Windows costumavam levantar *WindowsError*, que agora é um apelido de *OSError*.

Veja alguns exemplos para Windows. Note que `msvcrt` é a biblioteca C padrão da MS que contém a maioria das funções C padrão e usa a convenção de chamada `cdecl`:

```
>>> from ctypes import *
>>> print(windll.kernel32)
<WinDLL 'kernel32', handle ... at ...>
>>> print(cdll.msvcrt)
<CDLL 'msvcrt', handle ... at ...>
>>> libc = cdll.msvcrt
>>>
```

O Windows acrescenta automaticamente o sufixo de arquivo `.dll` usual.

Nota

Acessar a biblioteca padrão C por meio de `cdll.msvcrt` usará uma versão desatualizada da biblioteca que pode ser incompatível com a que está sendo usada pelo Python. Onde possível, use a funcionalidade nativa do Python ou então importe e use o módulo `msvcrt`.

No Linux, é necessário especificar o nome do arquivo *incluindo* a extensão para carregar uma biblioteca, então o acesso de atributo não pode ser usado para carregar bibliotecas. O método *LoadLibrary()* dos carregadores de dll deve ser usado, ou você deve carregar a biblioteca criando uma instância de CDLL chamando o construtor:

```
>>> cdll.LoadLibrary("libc.so.6")
<CDLL 'libc.so.6', handle ... at ...>
>>> libc = CDLL("libc.so.6")
>>> libc
<CDLL 'libc.so.6', handle ... at ...>
>>>
```

Acessando funções de dlls carregadas

Funções são acessadas como atributos de objetos dll:

```
>>> libc.printf
<_FuncPtr object at 0x...>
>>> print(windll.kernel32.GetModuleHandleA)
<_FuncPtr object at 0x...>
>>> print(windll.kernel32.MyOwnFunction)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "ctypes.py", line 239, in __getattr__
    func = _StdcallFuncPtr(name, self)
AttributeError: function 'MyOwnFunction' not found
>>>
```

Observe que DLLs de sistema do Win32, como `kernel32` e `user32`, frequentemente exportam versões ANSI e UNICODE de uma função. A versão UNICODE é exportada com um `W` anexado ao nome, enquanto a versão ANSI é exportada com um `A` anexado ao nome. A função `GetModuleHandle` do Win32, que retorna um *identificador de módulo* para um determinado nome de módulo, tem o seguinte protótipo em C, e uma macro é usada para expor um deles como `GetModuleHandle`, dependendo se UNICODE está definido ou não:

```
/* versão ANSI */
HMODULE GetModuleHandleA(LPCSTR lpModuleName);
/* versão UNICODE */
HMODULE GetModuleHandleW(LPCWSTR lpModuleName);
```

windll não tenta selecionar um deles magicamente, você deve acessar a versão necessária especificando `GetModuleHandleA` ou `GetModuleHandleW` explicitamente e então chamá-lo com objetos bytes ou string, respectivamente.

Às vezes, DLLs exportam funções com nomes que não são identificadores Python válidos, como `"??2@YAPAXI@Z"`. Nesse caso, você precisa usar *getattr()* para recuperar a função:

```
>>> getattr(cdll.msvcrt, "??2@YAPAXI@Z")
<_FuncPtr object at 0x...>
>>>
```

No Windows, algumas dlls exportam funções não por nome, mas por ordinal. Essas funções podem ser acessadas indexando o objeto dll com o número ordinal:

```
>>> cdll.kernel32[1]
<_FuncPtr object at 0x...>
>>> cdll.kernel32[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "ctypes.py", line 310, in __getitem__
    func = _StdcallFuncPtr(name, self)
AttributeError: function ordinal 0 not found
>>>
```

Chamando funções

Você pode chamar essas funções como qualquer outro chamável do Python. Este exemplo usa a função `rand()`, que não aceita argumentos e retorna um inteiro pseudoaleatório:

```
>>> print(libc.rand())
1804289383
```

No Windows, você pode chamar a função `GetModuleHandleA()`, que retorna um identificador de módulo win32 (passando `None` como único argumento para chamá-lo com um ponteiro `NULL`):

```
>>> print(hex(windll.kernel32.GetModuleHandleA(None)))
0x1d000000
>>>
```

ValueError é gerado quando você chama uma função `stdcall` com a convenção de chamada `cdecl`, ou vice-versa:

```
>>> cdll.kernel32.GetModuleHandleA(None)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Procedure probably called with not enough arguments (4 bytes missing)
>>>
```

(continua na próxima página)

(continuação da página anterior)

```
>>> windll.msvcrt.printf(b"spam")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Procedure probably called with too many arguments (4 bytes in excess)
>>>
```

Para descobrir a convenção de chamada correta, você precisa consultar o arquivo de cabeçalho C ou a documentação da função que deseja chamar.

No Windows, *ctypes* usa o tratamento de exceções estruturado do win32 para evitar travamentos devido a falhas gerais de proteção quando funções são chamadas com valores de argumentos inválidos:

```
>>> windll.kernel32.GetModuleHandleA(32)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OSError: exception: access violation reading 0x00000020
>>>
```

No entanto, existem maneiras suficientes de travar o Python com *ctypes*, então você deve ter cuidado de qualquer maneira. O módulo *faulthandler* pode ser útil na depuração de travamentos (por exemplo, de falhas de segmentação produzidas por chamadas errôneas da biblioteca C).

`None`, inteiros, objetos bytes e strings (unicode) são os únicos objetos nativos do Python que podem ser usados diretamente como parâmetros nessas chamadas de função. `None` é passado como um ponteiro `NULL` em C, objetos bytes e strings são passados como ponteiros para o bloco de memória que contém seus dados (`char*` ou `wchar_t*`). Inteiros em Python são passados como o tipo padrão da plataforma em C `int`, e seu valor é mascarado para se encaixar no tipo em C.

Antes de prosseguirmos com a chamada de funções com outros tipos de parâmetros, precisamos aprender mais sobre os tipos de dados *ctypes*.

Tipos de dados fundamentais

ctypes define uma série de tipos de dados primitivos compatíveis com C:

Tipo ctypes	Tipo em C	Tipo em Python
c_bool	`_Bool`	bool (1)
c_char	`char`	objeto bytes de 1 caractere
c_wchar	`wchar_t`	string de 1 caractere
c_byte	`char`	int
c_ubyte	`unsigned char`	int
c_short	`short`	int
c_ushort	`unsigned short`	int
c_int	`int`	int
c_int8	`int8_t`	int
c_int16	`int16_t`	int
c_int32	`int32_t`	int
c_int64	`int64_t`	int
c_uint	`unsigned int`	int
c_uint8	`uint8_t`	int
c_uint16	`uint16_t`	int
c_uint32	`uint32_t`	int
c_uint64	`uint64_t`	int
c_long	`long`	int
c_ulong	`unsigned long`	int
c_longlong	`__int64` ou `long long`	int
c_ulonglong	`unsigned __int64` ou `unsigned long long`	int

continua na próxima página

Tabela 1 – continuação da página anterior

Tipo ctypes	Tipo em C	Tipo em Python
c_size_t	size_t	int
c_ssize_t	ssize_t ou Py_ssize_t	int
c_time_t	time_t	int
c_float	float	float
c_double	double	float
c_longdouble	long double	float
c_char_p	char* (finalizado com NUL)	objeto bytes ou None
c_wchar_p	wchar_t* (finalizado com NUL)	String ou None
c_void_p	void*	int ou None

(1) O construtor aceita qualquer objeto com um valor verdade.

Além disso, se a aritmética complexa compatível com IEC 60559 (Anexo G) for suportada em C e `libffi`, os seguintes tipos complexos estarão disponíveis:

Tipo ctypes	Tipo em C	Tipo em Python
c_float_complex	float complex	complexo
c_double_complex	double complex	complexo
c_longdouble_complex	long double complex	complexo

Todos esses tipos podem ser criados chamando-os com um inicializador opcional do tipo e valor corretos:

```
>>> c_int()
c_long(0)
>>> c_wchar_p("Olá, mundo")
c_wchar_p(139878537078816)
>>> c_ushort(-3)
c_ushort(65533)
>>>
```

Como esses tipos são mutáveis, seus valores também podem ser alterados posteriormente:

```
>>> i = c_int(42)
>>> print(i)
c_long(42)
>>> print(i.value)
42
>>> i.value = -99
>>> print(i.value)
-99
>>>
```

Atribuir um novo valor a instâncias dos tipos de ponteiro *c_char_p*, *c_wchar_p* e *c_void_p* altera o *local de memória* para o qual eles apontam, *não o conteúdo* do bloco de memória (claro que não, porque objetos string do Python são imutáveis):

```
>>> s = "Olá, mundo"
>>> c_s = c_wchar_p(s)
>>> print(c_s)
c_wchar_p(139878542513936)
>>> print(c_s.value)
Olá, mundo
>>> c_s.value = "Opa, beleza?"
>>> print(c_s)               # o local da memória foi alterado
```

(continua na próxima página)

(continuação da página anterior)

```
c_wchar_p(139878536944240)
>>> print(c_s.value)
Opa, beleza?
>>> print(s)                # primeiro objeto está inalterado
Olá, mundo
>>>
```

No entanto, tome cuidado para não passá-los para funções que esperam ponteiros para memória mutável. Se precisar de blocos de memória mutáveis, o ctypes possui uma função *create_string_buffer()* que os cria de várias maneiras. O conteúdo do bloco de memória atual pode ser acessado (ou alterado) com a propriedade `raw`; se quiser acessá-lo como uma string terminada em NUL, use a propriedade `value`:

```
>>> from ctypes import *
>>> p = create_string_buffer(3)            # cria um buffer de 3 bytes,␣
↪inicializado para NUL bytes
>>> print(sizeof(p), repr(p.raw))
3 b'\x00\x00\x00'
>>> p = create_string_buffer(b"Opa")       # cria um buffer contendo uma string␣
↪terminando com NUL
>>> print(sizeof(p), repr(p.raw))
4 b'Opa\x00'
>>> print(repr(p.value))
b'Opa'
>>> p = create_string_buffer(b"Oi", 10)    # cria um buffer de 10 bytes
>>> print(sizeof(p), repr(p.raw))
10 b'Oi\x00\x00\x00\x00\x00\x00\x00\x00'
>>> p.value = b"Oi"
>>> print(sizeof(p), repr(p.raw))
10 b'Oi\x00\x00\x00\x00\x00\x00\x00\x00'
>>>
```

A função *create_string_buffer()* substitui a antiga função `c_buffer()` (que ainda está disponível como um apelido). Para criar um bloco de memória mutável contendo caracteres Unicode do tipo C `wchar_t`, use a função *create_unicode_buffer()*.

Chamando funções, continuação

Observe que printf imprime no canal de saída padrão real, *não* em *sys.stdout*, então esses exemplos só funcionarão no prompt do console, não de dentro do *IDLE* ou *PythonWin*:

```
>>> printf = libc.printf
>>> printf(b"Hello, %s\n", b"World!")
Hello, World!
14
>>> printf(b"Hello, %S\n", "World!")
Hello, World!
14
>>> printf(b"%d bottles of beer\n", 42)
42 bottles of beer
19
>>> printf(b"%f bottles of beer\n", 42.5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ctypes.ArgumentError: argument 2: TypeError: Don't know how to convert parameter 2
>>>
```

As has been mentioned before, all Python types except integers, strings, and bytes objects have to be wrapped in their corresponding *ctypes* type, so that they can be converted to the required C data type:

```
>>> printf(b"An int %d, a double %f\n", 1234, c_double(3.14))
An int 1234, a double 3.140000
31
>>>
```

Chamando funções variadas

On a lot of platforms calling variadic functions through ctypes is exactly the same as calling functions with a fixed number of parameters. On some platforms, and in particular ARM64 for Apple Platforms, the calling convention for variadic functions is different than that for regular functions.

On those platforms it is required to specify the *argtypes* attribute for the regular, non-variadic, function arguments:

```
libc.printf.argtypes = [ctypes.c_char_p]
```

Because specifying the attribute does not inhibit portability it is advised to always specify *argtypes* for all variadic functions.

Calling functions with your own custom data types

You can also customize *ctypes* argument conversion to allow instances of your own classes be used as function arguments. *ctypes* looks for an `_as_parameter_` attribute and uses this as the function argument. The attribute must be an integer, string, bytes, a *ctypes* instance, or an object with an `_as_parameter_` attribute:

```
>>> class Bottles:
...     def __init__(self, number):
...         self._as_parameter_ = number
...
>>> bottles = Bottles(42)
>>> printf(b"%d bottles of beer\n", bottles)
42 bottles of beer
19
>>>
```

If you don't want to store the instance's data in the `_as_parameter_` instance variable, you could define a *property* which makes the attribute available on request.

Specifying the required argument types (function prototypes)

It is possible to specify the required argument types of functions exported from DLLs by setting the *argtypes* attribute.

argtypes must be a sequence of C data types (the `printf()` function is probably not a good example here, because it takes a variable number and different types of parameters depending on the format string, on the other hand this is quite handy to experiment with this feature):

```
>>> printf.argtypes = [c_char_p, c_char_p, c_int, c_double]
>>> printf(b"String '%s', Int %d, Double %f\n", b"Hi", 10, 2.2)
String 'Hi', Int 10, Double 2.200000
37
>>>
```

Specifying a format protects against incompatible argument types (just as a prototype for a C function), and tries to convert the arguments to valid types:

```
>>> printf(b"%d %d %d", 1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ctypes.ArgumentError: argument 2: TypeError: 'int' object cannot be interpreted as␣
```

(continua na próxima página)

(continuação da página anterior)

```
↪ctypes.c_char_p
>>> printf(b"%s %d %f\n", b"X", 2, 3)
X 2 3.000000
13
>>>
```

If you have defined your own classes which you pass to function calls, you have to implement a *from_param()* class method for them to be able to use them in the *argtypes* sequence. The *from_param()* class method receives the Python object passed to the function call, it should do a typecheck or whatever is needed to make sure this object is acceptable, and then return the object itself, its `_as_parameter_` attribute, or whatever you want to pass as the C function argument in this case. Again, the result should be an integer, string, bytes, a *ctypes* instance, or an object with an `_as_parameter_` attribute.

Tipos de Retorno

By default functions are assumed to return the C `int` type. Other return types can be specified by setting the *restype* attribute of the function object.

The C prototype of `time()` is `time_t time(time_t *)`. Because `time_t` might be of a different type than the default return type `int`, you should specify the `restype` attribute:

```
>>> libc.time.restype = c_time_t
```

The argument types can be specified using *argtypes*:

```
>>> libc.time.argtypes = (POINTER(c_time_t),)
```

To call the function with a `NULL` pointer as first argument, use `None`:

```
>>> print(libc.time(None))
1150640792
```

Here is a more advanced example, it uses the `strchr()` function, which expects a string pointer and a char, and returns a pointer to a string:

```
>>> strchr = libc.strchr
>>> strchr(b"abcdef", ord("d"))
8059983
>>> strchr.restype = c_char_p    # c_char_p is a pointer to a string
>>> strchr(b"abcdef", ord("d"))
b'def'
>>> print(strchr(b"abcdef", ord("x")))
None
>>>
```

If you want to avoid the *ord("x")* calls above, you can set the *argtypes* attribute, and the second argument will be converted from a single character Python bytes object into a C char:

```
>>> strchr.restype = c_char_p
>>> strchr.argtypes = [c_char_p, c_char]
>>> strchr(b"abcdef", b"d")
b'def'
>>> strchr(b"abcdef", b"def")
Traceback (most recent call last):
ctypes.ArgumentError: argument 2: TypeError: one character bytes, bytearray or␣
↪integer expected
>>> print(strchr(b"abcdef", b"x"))
None
```

(continua na próxima página)

(continuação da página anterior)

```
>>> strchr(b"abcdef", b"d")
b'def'
>>>
```

You can also use a callable Python object (a function or a class for example) as the *restype* attribute, if the foreign function returns an integer. The callable will be called with the *integer* the C function returns, and the result of this call will be used as the result of your function call. This is useful to check for error return values and automatically raise an exception:

```
>>> GetModuleHandle = windll.kernel32.GetModuleHandleA
>>> def ValidHandle(value):
...     if value == 0:
...         raise WinError()
...     return value
...
>>>
>>> GetModuleHandle.restype = ValidHandle
>>> GetModuleHandle(None)
486539264
>>> GetModuleHandle("something silly")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in ValidHandle
OSError: [Errno 126] The specified module could not be found.
>>>
```

`WinError` is a function which will call Windows `FormatMessage()` api to get the string representation of an error code, and *returns* an exception. `WinError` takes an optional error code parameter, if no one is used, it calls *GetLastError()* to retrieve it.

Please note that a much more powerful error checking mechanism is available through the *errcheck* attribute; see the reference manual for details.

Passando ponteiros (ou: passando parâmetros por referência)

Sometimes a C api function expects a *pointer* to a data type as parameter, probably to write into the corresponding location, or if the data is too large to be passed by value. This is also known as *passing parameters by reference*.

ctypes exports the *byref()* function which is used to pass parameters by reference. The same effect can be achieved with the *pointer()* function, although *pointer()* does a lot more work since it constructs a real pointer object, so it is faster to use *byref()* if you don't need the pointer object in Python itself:

```
>>> i = c_int()
>>> f = c_float()
>>> s = create_string_buffer(b'\000' * 32)
>>> print(i.value, f.value, repr(s.value))
0 0.0 b''
>>> libc.sscanf(b"1 3.14 Hello", b"%d %f %s",
...             byref(i), byref(f), s)
3
>>> print(i.value, f.value, repr(s.value))
1 3.1400001049 b'Hello'
>>>
```

Estruturas e uniões

Structures and unions must derive from the *Structure* and *Union* base classes which are defined in the *ctypes* module. Each subclass must define a *_fields_* attribute. _fields_ must be a list of *2-tuples*, containing a *field name* and a *field type*.

The field type must be a *ctypes* type like *c_int*, or any other derived *ctypes* type: structure, union, array, pointer.

Here is a simple example of a POINT structure, which contains two integers named *x* and *y*, and also shows how to initialize a structure in the constructor:

```
>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = [("x", c_int),
...                 ("y", c_int)]
...
>>> point = POINT(10, 20)
>>> print(point.x, point.y)
10 20
>>> point = POINT(y=5)
>>> print(point.x, point.y)
0 5
>>> POINT(1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: too many initializers
>>>
```

You can, however, build much more complicated structures. A structure can itself contain other structures by using a structure as a field type.

Here is a RECT structure which contains two POINTs named *upperleft* and *lowerright*:

```
>>> class RECT(Structure):
...     _fields_ = [("upperleft", POINT),
...                 ("lowerright", POINT)]
...
>>> rc = RECT(point)
>>> print(rc.upperleft.x, rc.upperleft.y)
0 5
>>> print(rc.lowerright.x, rc.lowerright.y)
0 0
>>>
```

Nested structures can also be initialized in the constructor in several ways:

```
>>> r = RECT(POINT(1, 2), POINT(3, 4))
>>> r = RECT((1, 2), (3, 4))
```

Field *descriptor*s can be retrieved from the *class*, they are useful for debugging because they can provide useful information. See *CField*:

```
>>> POINT.x
<ctypes.CField 'x' type=c_int, ofs=0, size=4>
>>> POINT.y
<ctypes.CField 'y' type=c_int, ofs=4, size=4>
>>>
```

Aviso

ctypes does not support passing unions or structures with bit-fields to functions by value. While this may work on 32-bit x86, it's not guaranteed by the library to work in the general case. Unions and structures with bit-fields should always be passed to functions by pointer.

Structure/union layout, alignment and byte order

By default, Structure and Union fields are laid out in the same way the C compiler does it. It is possible to override this behavior entirely by specifying a *_layout_* class attribute in the subclass definition; see the attribute documentation for details.

It is possible to specify the maximum alignment for the fields by setting the *_pack_* class attribute to a positive integer. This matches what `#pragma pack(n)` does in MSVC.

It is also possible to set a minimum alignment for how the subclass itself is packed in the same way `#pragma align(n)` works in MSVC. This can be achieved by specifying a *_align_* class attribute in the subclass definition.

ctypes uses the native byte order for Structures and Unions. To build structures with non-native byte order, you can use one of the *BigEndianStructure*, *LittleEndianStructure*, *BigEndianUnion*, and *LittleEndianUnion* base classes. These classes cannot contain pointer fields.

Bit fields in structures and unions

It is possible to create structures and unions containing bit fields. Bit fields are only possible for integer fields, the bit width is specified as the third item in the *_fields_* tuples:

```
>>> class Int(Structure):
...     _fields_ = [("first_16", c_int, 16),
...                 ("second_16", c_int, 16)]
...
>>> print(Int.first_16)
<ctypes.CField 'first_16' type=c_int, ofs=0, bit_size=16, bit_offset=0>
>>> print(Int.second_16)
<ctypes.CField 'second_16' type=c_int, ofs=0, bit_size=16, bit_offset=16>
```

It is important to note that bit field allocation and layout in memory are not defined as a C standard; their implementation is compiler-specific. By default, Python will attempt to match the behavior of a "native" compiler for the current platform. See the *_layout_* attribute for details on the default behavior and how to change it.

Arrays

Arrays are sequences, containing a fixed number of instances of the same type.

The recommended way to create array types is by multiplying a data type with a positive integer:

```
TenPointsArrayType = POINT * 10
```

Here is an example of a somewhat artificial data type, a structure containing 4 POINTs among other stuff:

```
>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = ("x", c_int), ("y", c_int)
...
>>> class MyStruct(Structure):
...     _fields_ = [("a", c_int),
...                 ("b", c_float),
...                 ("point_array", POINT * 4)]
>>>
```

(continua na próxima página)

(continuação da página anterior)

```
>>> print(len(MyStruct().point_array))
4
>>>
```

Instances are created in the usual way, by calling the class:

```
arr = TenPointsArrayType()
for pt in arr:
    print(pt.x, pt.y)
```

The above code print a series of `0 0` lines, because the array contents is initialized to zeros.

Initializers of the correct type can also be specified:

```
>>> from ctypes import *
>>> TenIntegers = c_int * 10
>>> ii = TenIntegers(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
>>> print(ii)
<c_long_Array_10 object at 0x...>
>>> for i in ii: print(i, end=" ")
...
1 2 3 4 5 6 7 8 9 10
>>>
```

Ponteiros

Pointer instances are created by calling the *pointer()* function on a *ctypes* type:

```
>>> from ctypes import *
>>> i = c_int(42)
>>> pi = pointer(i)
>>>
```

Pointer instances have a *contents* attribute which returns the object to which the pointer points, the `i` object above:

```
>>> pi.contents
c_long(42)
>>>
```

Note that *ctypes* does not have OOR (original object return), it constructs a new, equivalent object each time you retrieve an attribute:

```
>>> pi.contents is i
False
>>> pi.contents is pi.contents
False
>>>
```

Assigning another *c_int* instance to the pointer's contents attribute would cause the pointer to point to the memory location where this is stored:

```
>>> i = c_int(99)
>>> pi.contents = i
>>> pi.contents
c_long(99)
>>>
```

Pointer instances can also be indexed with integers:

```
>>> pi[0]
99
>>>
```

Assigning to an integer index changes the pointed to value:

```
>>> print(i)
c_long(99)
>>> pi[0] = 22
>>> print(i)
c_long(22)
>>>
```

It is also possible to use indexes different from 0, but you must know what you're doing, just as in C: You can access or change arbitrary memory locations. Generally you only use this feature if you receive a pointer from a C function, and you *know* that the pointer actually points to an array instead of a single item.

Behind the scenes, the *pointer()* function does more than simply create pointer instances, it has to create pointer *types* first. This is done with the *POINTER()* function, which accepts any *ctypes* type, and returns a new type:

```
>>> PI = POINTER(c_int)
>>> PI
<class 'ctypes.LP_c_long'>
>>> PI(42)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: expected c_long instead of int
>>> PI(c_int(42))
<ctypes.LP_c_long object at 0x...>
>>>
```

Calling the pointer type without an argument creates a `NULL` pointer. `NULL` pointers have a `False` boolean value:

```
>>> null_ptr = POINTER(c_int)()
>>> print(bool(null_ptr))
False
>>>
```

ctypes checks for `NULL` when dereferencing pointers (but dereferencing invalid non-`NULL` pointers would crash Python):

```
>>> null_ptr[0]
Traceback (most recent call last):
    ....
ValueError: NULL pointer access
>>>

>>> null_ptr[0] = 1234
Traceback (most recent call last):
    ....
ValueError: NULL pointer access
>>>
```

Thread safety without the GIL

From Python 3.13 onward, the *GIL* can be disabled on *free threaded* builds. In ctypes, reads and writes to a single object concurrently is safe, but not across multiple objects:

```
>>> number = c_int(42)
>>> pointer_a = pointer(number)
>>> pointer_b = pointer(number)
```

In the above, it's only safe for one object to read and write to the address at once if the GIL is disabled. So, `pointer_a` can be shared and written to across multiple threads, but only if `pointer_b` is not also attempting to do the same. If this is an issue, consider using a *threading.Lock* to synchronize access to memory:

```
>>> import threading
>>> lock = threading.Lock()
>>> # Thread 1
>>> with lock:
...     pointer_a.contents = 24
>>> # Thread 2
>>> with lock:
...     pointer_b.contents = 42
```

Conversão de Tipos

Usually, ctypes does strict type checking. This means, if you have `POINTER(c_int)` in the *argtypes* list of a function or as the type of a member field in a structure definition, only instances of exactly the same type are accepted. There are some exceptions to this rule, where ctypes accepts other objects. For example, you can pass compatible array instances instead of pointer types. So, for `POINTER(c_int)`, ctypes accepts an array of c_int:

```
>>> class Bar(Structure):
...     _fields_ = [("count", c_int), ("values", POINTER(c_int))]
...
>>> bar = Bar()
>>> bar.values = (c_int * 3)(1, 2, 3)
>>> bar.count = 3
>>> for i in range(bar.count):
...     print(bar.values[i])
...
1
2
3
>>>
```

In addition, if a function argument is explicitly declared to be a pointer type (such as `POINTER(c_int)`) in *argtypes*, an object of the pointed type (`c_int` in this case) can be passed to the function. ctypes will apply the required *byref()* conversion in this case automatically.

To set a POINTER type field to `NULL`, you can assign `None`:

```
>>> bar.values = None
>>>
```

Sometimes you have instances of incompatible types. In C, you can cast one type into another type. *ctypes* provides a *cast()* function which can be used in the same way. The `Bar` structure defined above accepts `POINTER(c_int)` pointers or *c_int* arrays for its `values` field, but not instances of other types:

```
>>> bar.values = (c_byte * 4)()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: incompatible types, c_byte_Array_4 instance instead of LP_c_long␣
↪instance
>>>
```

For these cases, the *cast()* function is handy.

The *cast()* function can be used to cast a ctypes instance into a pointer to a different ctypes data type. *cast()* takes two parameters, a ctypes object that is or can be converted to a pointer of some kind, and a ctypes pointer type. It returns an instance of the second argument, which references the same memory block as the first argument:

```
>>> a = (c_byte * 4)()
>>> cast(a, POINTER(c_int))
<ctypes.LP_c_long object at ...>
>>>
```

So, *cast()* can be used to assign to the `values` field of `Bar` the structure:

```
>>> bar = Bar()
>>> bar.values = cast((c_byte * 4)(), POINTER(c_int))
>>> print(bar.values[0])
0
>>>
```

Tipos Incompletos

Incomplete Types are structures, unions or arrays whose members are not yet specified. In C, they are specified by forward declarations, which are defined later:

```
struct cell; /* forward declaration */

struct cell {
    char *name;
    struct cell *next;
};
```

The straightforward translation into ctypes code would be this, but it does not work:

```
>>> class cell(Structure):
...     _fields_ = [("name", c_char_p),
...                 ("next", POINTER(cell))]
...
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in cell
NameError: name 'cell' is not defined
>>>
```

because the new `class cell` is not available in the class statement itself. In *ctypes*, we can define the `cell` class and set the *_fields_* attribute later, after the class statement:

```
>>> from ctypes import *
>>> class cell(Structure):
...     pass
...
>>> cell._fields_ = [("name", c_char_p),
...                  ("next", POINTER(cell))]
>>>
```

Let's try it. We create two instances of `cell`, and let them point to each other, and finally follow the pointer chain a few times:

```
>>> c1 = cell()
>>> c1.name = b"foo"
>>> c2 = cell()
```

(continua na próxima página)

(continuação da página anterior)

```
>>> c2.name = b"bar"
>>> c1.next = pointer(c2)
>>> c2.next = pointer(c1)
>>> p = c1
>>> for i in range(8):
...     print(p.name, end=" ")
...     p = p.next[0]
...
foo bar foo bar foo bar foo bar
>>>
```

Funções Callbacks

ctypes allows creating C callable function pointers from Python callables. These are sometimes called *callback functions*.

First, you must create a class for the callback function. The class knows the calling convention, the return type, and the number and types of arguments this function will receive.

The *CFUNCTYPE()* factory function creates types for callback functions using the `cdecl` calling convention. On Windows, the *WINFUNCTYPE()* factory function creates types for callback functions using the `stdcall` calling convention.

Both of these factory functions are called with the result type as first argument, and the callback functions expected argument types as the remaining arguments.

I will present an example here which uses the standard C library's `qsort()` function, that is used to sort items with the help of a callback function. `qsort()` will be used to sort an array of integers:

```
>>> IntArray5 = c_int * 5
>>> ia = IntArray5(5, 1, 7, 33, 99)
>>> qsort = libc.qsort
>>> qsort.restype = None
>>>
```

`qsort()` must be called with a pointer to the data to sort, the number of items in the data array, the size of one item, and a pointer to the comparison function, the callback. The callback will then be called with two pointers to items, and it must return a negative integer if the first item is smaller than the second, a zero if they are equal, and a positive integer otherwise.

So our callback function receives pointers to integers, and must return an integer. First we create the `type` for the callback function:

```
>>> CMPFUNC = CFUNCTYPE(c_int, POINTER(c_int), POINTER(c_int))
>>>
```

To get started, here is a simple callback that shows the values it gets passed:

```
>>> def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return 0
...
>>> cmp_func = CMPFUNC(py_cmp_func)
>>>
```

O resultado:

```
>>> qsort(ia, len(ia), sizeof(c_int), cmp_func)
py_cmp_func 5 1
```

(continua na próxima página)

(continuação da página anterior)

```
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 5 7
py_cmp_func 1 7
>>>
```

Now we can actually compare the two items and return a useful result:

```
>>> def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return a[0] - b[0]
...
>>>
>>> qsort(ia, len(ia), sizeof(c_int), CMPFUNC(py_cmp_func))
py_cmp_func 5 1
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 1 7
py_cmp_func 5 7
>>>
```

As we can easily check, our array is sorted now:

```
>>> for i in ia: print(i, end=" ")
...
1 5 7 33 99
>>>
```

The function factories can be used as decorator factories, so we may as well write:

```
>>> @CFUNCTYPE(c_int, POINTER(c_int), POINTER(c_int))
... def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return a[0] - b[0]
...
>>> qsort(ia, len(ia), sizeof(c_int), py_cmp_func)
py_cmp_func 5 1
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 1 7
py_cmp_func 5 7
>>>
```

Nota

Make sure you keep references to *CFUNCTYPE()* objects as long as they are used from C code. *ctypes* doesn't, and if you don't, they may be garbage collected, crashing your program when a callback is made.

Also, note that if the callback function is called in a thread created outside of Python's control (e.g. by the foreign code that calls the callback), ctypes creates a new dummy Python thread on every invocation. This behavior is correct for most purposes, but it means that values stored with *threading.local* will *not* survive across different callbacks, even when those calls are made from the same C thread.

Accessing values exported from dlls

Some shared libraries not only export functions, they also export variables. An example in the Python library itself is the `Py_Version`, Python runtime version number encoded in a single constant integer.

ctypes can access values like this with the *in_dll()* class methods of the type. *pythonapi* is a predefined symbol giving access to the Python C api:

```
>>> version = ctypes.c_int.in_dll(ctypes.pythonapi, "Py_Version")
>>> print(hex(version.value))
0x30c00a0
```

An extended example which also demonstrates the use of pointers accesses the `PyImport_FrozenModules` pointer exported by Python.

Quoting the docs for that value:

> Este ponteiro é inicializado para apontar para um vetor de registros de `_frozen`, terminado por um cujos membros são todos `NULL` ou zero. Quando um módulo congelado é importado, ele é pesquisado nesta tabela. O código de terceiros pode fazer truques com isso para fornecer uma coleção criada dinamicamente de módulos congelados.

So manipulating this pointer could even prove useful. To restrict the example size, we show only how this table can be read with *ctypes*:

```
>>> from ctypes import *
>>>
>>> class struct_frozen(Structure):
...     _fields_ = [("name", c_char_p),
...                 ("code", POINTER(c_ubyte)),
...                 ("size", c_int),
...                 ("get_code", POINTER(c_ubyte)),  # Function pointer
...                ]
...
>>>
```

We have defined the `_frozen` data type, so we can get the pointer to the table:

```
>>> FrozenTable = POINTER(struct_frozen)
>>> table = FrozenTable.in_dll(pythonapi, "_PyImport_FrozenBootstrap")
>>>
```

Since `table` is a `pointer` to the array of `struct_frozen` records, we can iterate over it, but we just have to make sure that our loop terminates, because pointers have no size. Sooner or later it would probably crash with an access violation or whatever, so it's better to break out of the loop when we hit the `NULL` entry:

```
>>> for item in table:
...     if item.name is None:
...         break
...     print(item.name.decode("ascii"), item.size)
...
_frozen_importlib 31764
_frozen_importlib_external 41499
zipimport 12345
>>>
```

The fact that standard Python has a frozen module and a frozen package (indicated by the negative `size` member) is not well known, it is only used for testing. Try it out with `import __hello__` for example.

Surprises

There are some edges in *ctypes* where you might expect something other than what actually happens.

Consider the following example:

```
>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = ("x", c_int), ("y", c_int)
...
>>> class RECT(Structure):
...     _fields_ = ("a", POINT), ("b", POINT)
...
>>> p1 = POINT(1, 2)
>>> p2 = POINT(3, 4)
>>> rc = RECT(p1, p2)
>>> print(rc.a.x, rc.a.y, rc.b.x, rc.b.y)
1 2 3 4
>>> # now swap the two points
>>> rc.a, rc.b = rc.b, rc.a
>>> print(rc.a.x, rc.a.y, rc.b.x, rc.b.y)
3 4 3 4
>>>
```

Hm. We certainly expected the last statement to print `3 4 1 2`. What happened? Here are the steps of the `rc.a, rc.b = rc.b, rc.a` line above:

```
>>> temp0, temp1 = rc.b, rc.a
>>> rc.a = temp0
>>> rc.b = temp1
>>>
```

Note that `temp0` and `temp1` are objects still using the internal buffer of the `rc` object above. So executing `rc.a = temp0` copies the buffer contents of `temp0` into `rc` 's buffer. This, in turn, changes the contents of `temp1`. So, the last assignment `rc.b = temp1`, doesn't have the expected effect.

Keep in mind that retrieving sub-objects from Structure, Unions, and Arrays doesn't *copy* the sub-object, instead it retrieves a wrapper object accessing the root-object's underlying buffer.

Another example that may behave differently from what one would expect is this:

```
>>> s = c_char_p()
>>> s.value = b"abc def ghi"
>>> s.value
b'abc def ghi'
>>> s.value is s.value
False
>>>
```

Nota

Objects instantiated from *c_char_p* can only have their value set to bytes or integers.

Why is it printing `False`? ctypes instances are objects containing a memory block plus some *descriptor*s accessing the contents of the memory. Storing a Python object in the memory block does not store the object itself, instead the `contents` of the object is stored. Accessing the contents again constructs a new Python object each time!

Variable-sized data types

ctypes provides some support for variable-sized arrays and structures.

The *resize()* function can be used to resize the memory buffer of an existing ctypes object. The function takes the object as first argument, and the requested size in bytes as the second argument. The memory block cannot be made smaller than the natural memory block specified by the objects type, a *ValueError* is raised if this is tried:

```
>>> short_array = (c_short * 4)()
>>> print(sizeof(short_array))
8
>>> resize(short_array, 4)
Traceback (most recent call last):
    ...
ValueError: minimum size is 8
>>> resize(short_array, 32)
>>> sizeof(short_array)
32
>>> sizeof(type(short_array))
8
>>>
```

This is nice and fine, but how would one access the additional elements contained in this array? Since the type still only knows about 4 elements, we get errors accessing other elements:

```
>>> short_array[:]
[0, 0, 0, 0]
>>> short_array[7]
Traceback (most recent call last):
    ...
IndexError: invalid index
>>>
```

Another way to use variable-sized data types with *ctypes* is to use the dynamic nature of Python, and (re-)define the data type after the required size is already known, on a case by case basis.

16.9.2 Referência ctypes

Finding shared libraries

When programming in a compiled language, shared libraries are accessed when compiling/linking a program, and when the program is run.

The purpose of the *find_library()* function is to locate a library in a way similar to what the compiler or runtime loader does (on platforms with several versions of a shared library the most recent should be loaded), while the ctypes library loaders act like when a program is run, and call the runtime loader directly.

The `ctypes.util` module provides a function which can help to determine the library to load.

`ctypes.util.`**`find_library`**(*name*)

> Try to find a library and return a pathname. *name* is the library name without any prefix like *lib*, suffix like `.so`, `.dylib` or version number (this is the form used for the posix linker option `-l`). If no library can be found, returns `None`.

The exact functionality is system dependent.

On Linux, *find_library()* tries to run external programs (`/sbin/ldconfig`, `gcc`, `objdump` and `ld`) to find the library file. It returns the filename of the library file.

Alterado na versão 3.6: On Linux, the value of the environment variable `LD_LIBRARY_PATH` is used when searching for libraries, if a library cannot be found by any other means.

Veja alguns exemplos:

```
>>> from ctypes.util import find_library
>>> find_library("m")
'libm.so.6'
>>> find_library("c")
'libc.so.6'
>>> find_library("bz2")
'libbz2.so.1.0'
>>>
```

On macOS and Android, *find_library()* uses the system's standard naming schemes and paths to locate the library, and returns a full pathname if successful:

```
>>> from ctypes.util import find_library
>>> find_library("c")
'/usr/lib/libc.dylib'
>>> find_library("m")
'/usr/lib/libm.dylib'
>>> find_library("bz2")
'/usr/lib/libbz2.dylib'
>>> find_library("AGL")
'/System/Library/Frameworks/AGL.framework/AGL'
>>>
```

On Windows, *find_library()* searches along the system search path, and returns the full pathname, but since there is no predefined naming scheme a call like `find_library("c")` will fail and return `None`.

If wrapping a shared library with *ctypes*, it *may* be better to determine the shared library name at development time, and hardcode that into the wrapper module instead of using *find_library()* to locate the library at runtime.

Listing loaded shared libraries

When writing code that relies on code loaded from shared libraries, it can be useful to know which shared libraries have already been loaded into the current process.

The `ctypes.util` module provides the *dllist()* function, which calls the different APIs provided by the various platforms to help determine which shared libraries have already been loaded into the current process.

The exact output of this function will be system dependent. On most platforms, the first entry of this list represents the current process itself, which may be an empty string. For example, on glibc-based Linux, the return may look like:

```
>>> from ctypes.util import dllist
>>> dllist()
['', 'linux-vdso.so.1', '/lib/x86_64-linux-gnu/libm.so.6', '/lib/x86_64-linux-gnu/
↪libc.so.6', ... ]
```

Loading shared libraries

There are several ways to load shared libraries into the Python process. One way is to instantiate one of the following classes:

class ctypes.**CDLL**(*name*, *mode=DEFAULT_MODE*, *handle=None*, *use_errno=False*, *use_last_error=False*, *winmode=None*)

Instances of this class represent loaded shared libraries. Functions in these libraries use the standard C calling convention, and are assumed to return `int`.

On Windows creating a *CDLL* instance may fail even if the DLL name exists. When a dependent DLL of the loaded DLL is not found, a *OSError* error is raised with the message *"[WinError 126] The specified module could not be found"*. This error message does not contain the name of the missing DLL because the Windows API does not return this information making this error hard to diagnose. To resolve this error and determine

which DLL is not found, you need to find the list of dependent DLLs and determine which one is not found using Windows debugging and tracing tools.

Alterado na versão 3.12: The *name* parameter can now be a *path-like object*.

Ver também

Microsoft DUMPBIN tool – A tool to find DLL dependents.

class ctypes.**OleDLL**(*name*, *mode=DEFAULT_MODE*, *handle=None*, *use_errno=False*, *use_last_error=False*, *winmode=None*)

Instances of this class represent loaded shared libraries, functions in these libraries use the `stdcall` calling convention, and are assumed to return the windows specific *HRESULT* code. *HRESULT* values contain information specifying whether the function call failed or succeeded, together with additional error code. If the return value signals a failure, an *OSError* is automatically raised.

Disponibilidade: Windows

Alterado na versão 3.3: *WindowsError* used to be raised, which is now an alias of *OSError*.

Alterado na versão 3.12: The *name* parameter can now be a *path-like object*.

class ctypes.**WinDLL**(*name*, *mode=DEFAULT_MODE*, *handle=None*, *use_errno=False*, *use_last_error=False*, *winmode=None*)

Instances of this class represent loaded shared libraries, functions in these libraries use the `stdcall` calling convention, and are assumed to return `int` by default.

Disponibilidade: Windows

Alterado na versão 3.12: The *name* parameter can now be a *path-like object*.

The Python *global interpreter lock* is released before calling any function exported by these libraries, and reacquired afterwards.

class ctypes.**PyDLL**(*name*, *mode=DEFAULT_MODE*, *handle=None*)

Instances of this class behave like *CDLL* instances, except that the Python GIL is *not* released during the function call, and after the function execution the Python error flag is checked. If the error flag is set, a Python exception is raised.

Thus, this is only useful to call Python C api functions directly.

Alterado na versão 3.12: The *name* parameter can now be a *path-like object*.

All these classes can be instantiated by calling them with at least one argument, the pathname of the shared library. If you have an existing handle to an already loaded shared library, it can be passed as the `handle` named parameter, otherwise the underlying platform's `dlopen()` or `LoadLibrary()` function is used to load the library into the process, and to get a handle to it.

The *mode* parameter can be used to specify how the library is loaded. For details, consult the *dlopen(3)* manpage. On Windows, *mode* is ignored. On posix systems, RTLD_NOW is always added, and is not configurable.

The *use_errno* parameter, when set to true, enables a ctypes mechanism that allows accessing the system *errno* error number in a safe way. *ctypes* maintains a thread-local copy of the system's *errno* variable; if you call foreign functions created with `use_errno=True` then the *errno* value before the function call is swapped with the ctypes private copy, the same happens immediately after the function call.

The function *ctypes.get_errno()* returns the value of the ctypes private copy, and the function *ctypes.set_errno()* changes the ctypes private copy to a new value and returns the former value.

The *use_last_error* parameter, when set to true, enables the same mechanism for the Windows error code which is managed by the *GetLastError()* and `SetLastError()` Windows API functions; *ctypes.get_last_error()* and *ctypes.set_last_error()* are used to request and change the ctypes private copy of the windows error code.

The *winmode* parameter is used on Windows to specify how the library is loaded (since *mode* is ignored). It takes any value that is valid for the Win32 API `LoadLibraryEx` flags parameter. When omitted, the default is to use the flags that result in the most secure DLL load, which avoids issues such as DLL hijacking. Passing the full path to the DLL is the safest way to ensure the correct library and dependencies are loaded.

Alterado na versão 3.8: Added *winmode* parameter.

`ctypes.`**`RTLD_GLOBAL`**

Flag to use as *mode* parameter. On platforms where this flag is not available, it is defined as the integer zero.

`ctypes.`**`RTLD_LOCAL`**

Flag to use as *mode* parameter. On platforms where this is not available, it is the same as *RTLD_GLOBAL*.

`ctypes.`**`DEFAULT_MODE`**

The default mode which is used to load shared libraries. On OSX 10.3, this is *RTLD_GLOBAL*, otherwise it is the same as *RTLD_LOCAL*.

Instances of these classes have no public methods. Functions exported by the shared library can be accessed as attributes or by index. Please note that accessing the function through an attribute caches the result and therefore accessing it repeatedly returns the same object each time. On the other hand, accessing it through an index returns a new object each time:

```
>>> from ctypes import CDLL
>>> libc = CDLL("libc.so.6")  # On Linux
>>> libc.time == libc.time
True
>>> libc['time'] == libc['time']
False
```

The following public attributes are available, their name starts with an underscore to not clash with exported function names:

`PyDLL.`**`_handle`**

The system handle used to access the library.

`PyDLL.`**`_name`**

The name of the library passed in the constructor.

Shared libraries can also be loaded by using one of the prefabricated objects, which are instances of the *`LibraryLoader`* class, either by calling the *`LoadLibrary()`* method, or by retrieving the library as attribute of the loader instance.

`class` `ctypes.`**`LibraryLoader`**(*dlltype*)

Class which loads shared libraries. *dlltype* should be one of the *`CDLL`*, *`PyDLL`*, *`WinDLL`*, or *`OleDLL`* types.

`__getattr__()` has special behavior: It allows loading a shared library by accessing it as attribute of a library loader instance. The result is cached, so repeated attribute accesses return the same library each time.

`LoadLibrary`(*name*)

Load a shared library into the process and return it. This method always returns a new instance of the library.

These prefabricated library loaders are available:

`ctypes.`**`cdll`**

Creates *`CDLL`* instances.

`ctypes.`**`windll`**

Creates *`WinDLL`* instances.

Disponibilidade: Windows

ctypes.**oledll**

Creates *OleDLL* instances.

Disponibilidade: Windows

ctypes.**pydll**

Creates *PyDLL* instances.

For accessing the C Python api directly, a ready-to-use Python shared library object is available:

ctypes.**pythonapi**

An instance of *PyDLL* that exposes Python C API functions as attributes. Note that all these functions are assumed to return C `int`, which is of course not always the truth, so you have to assign the correct `restype` attribute to use these functions.

Carregar uma biblioteca através de qualquer um desses objetos levanta um *evento de auditoria* `ctypes.dlopen` com o argumento string `name`, o nome usado para carregar a biblioteca.

Accessing a function on a loaded library raises an auditing event `ctypes.dlsym` with arguments `library` (the library object) and `name` (the symbol's name as a string or integer).

In cases when only the library handle is available rather than the object, accessing a function raises an auditing event `ctypes.dlsym/handle` with arguments `handle` (the raw library handle) and `name`.

Foreign functions

As explained in the previous section, foreign functions can be accessed as attributes of loaded shared libraries. The function objects created in this way by default accept any number of arguments, accept any ctypes data instances as arguments, and return the default result type specified by the library loader.

They are instances of a private local class `_FuncPtr` (not exposed in `ctypes`) which inherits from the private *_CFuncPtr* class:

```
>>> import ctypes
>>> lib = ctypes.CDLL(None)
>>> issubclass(lib._FuncPtr, ctypes._CFuncPtr)
True
>>> lib._FuncPtr is ctypes._CFuncPtr
False
```

class ctypes.**_CFuncPtr**

Base class for C callable foreign functions.

Instances of foreign functions are also C compatible data types; they represent C function pointers.

This behavior can be customized by assigning to special attributes of the foreign function object.

restype

Assign a ctypes type to specify the result type of the foreign function. Use `None` for `void`, a function not returning anything.

It is possible to assign a callable Python object that is not a ctypes type, in this case the function is assumed to return a C `int`, and the callable will be called with this integer, allowing further processing or error checking. Using this is deprecated, for more flexible post processing or error checking use a ctypes data type as `restype` and assign a callable to the *errcheck* attribute.

argtypes

Assign a tuple of ctypes types to specify the argument types that the function accepts. Functions using the `stdcall` calling convention can only be called with the same number of arguments as the length of this tuple; functions using the C calling convention accept additional, unspecified arguments as well.

When a foreign function is called, each actual argument is passed to the *from_param()* class method of the items in the *argtypes* tuple, this method allows adapting the actual argument to an object that the foreign function accepts. For example, a *c_char_p* item in the *argtypes* tuple will convert a string passed as argument into a bytes object using ctypes conversion rules.

New: It is now possible to put items in argtypes which are not ctypes types, but each item must have a *from_param()* method which returns a value usable as argument (integer, string, ctypes instance). This allows defining adapters that can adapt custom objects as function parameters.

errcheck

Assign a Python function or another callable to this attribute. The callable will be called with three or more arguments:

callable(*result*, *func*, *arguments*)

result is what the foreign function returns, as specified by the `restype` attribute.

func is the foreign function object itself, this allows reusing the same callable object to check or post process the results of several functions.

arguments is a tuple containing the parameters originally passed to the function call, this allows specializing the behavior on the arguments used.

The object that this function returns will be returned from the foreign function call, but it can also check the result value and raise an exception if the foreign function call failed.

On Windows, when a foreign function call raises a system exception (for example, due to an access violation), it will be captured and replaced with a suitable Python exception. Further, an auditing event `ctypes.set_exception` with argument `code` will be raised, allowing an audit hook to replace the exception with its own.

Some ways to invoke foreign function calls as well as some of the functions in this module may raise an auditing event `ctypes.call_function` with arguments `function pointer` and `arguments`.

Function prototypes

Foreign functions can also be created by instantiating function prototypes. Function prototypes are similar to function prototypes in C; they describe a function (return type, argument types, calling convention) without defining an implementation. The factory functions must be called with the desired result type and the argument types of the function, and can be used as decorator factories, and as such, be applied to functions through the `@wrapper` syntax. See *Funções Callbacks* for examples.

ctypes.**CFUNCTYPE**(*restype*, **argtypes*, *use_errno=False*, *use_last_error=False*)

The returned function prototype creates functions that use the standard C calling convention. The function will release the GIL during the call. If *use_errno* is set to true, the ctypes private copy of the system *errno* variable is exchanged with the real *errno* value before and after the call; *use_last_error* does the same for the Windows error code.

ctypes.**WINFUNCTYPE**(*restype*, **argtypes*, *use_errno=False*, *use_last_error=False*)

The returned function prototype creates functions that use the `stdcall` calling convention. The function will release the GIL during the call. *use_errno* and *use_last_error* have the same meaning as above.

Disponibilidade: Windows

ctypes.**PYFUNCTYPE**(*restype*, **argtypes*)

The returned function prototype creates functions that use the Python calling convention. The function will *not* release the GIL during the call.

Function prototypes created by these factory functions can be instantiated in different ways, depending on the type and number of the parameters in the call:

prototype(*address*)

Returns a foreign function at the specified address which must be an integer.

prototype(*callable*)

Create a C callable function (a callback function) from a Python *callable*.

prototype(*func_spec*[, *paramflags*])

Returns a foreign function exported by a shared library. *func_spec* must be a 2-tuple `(name_or_ordinal, library)`. The first item is the name of the exported function as string, or the ordinal of the exported function as small integer. The second item is the shared library instance.

prototype(*vtbl_index*, *name*[, *paramflags*[, *iid*]])

Returns a foreign function that will call a COM method. *vtbl_index* is the index into the virtual function table, a small non-negative integer. *name* is name of the COM method. *iid* is an optional pointer to the interface identifier which is used in extended error reporting.

If *iid* is not specified, an *OSError* is raised if the COM method call fails. If *iid* is specified, a *COMError* is raised instead.

COM methods use a special calling convention: They require a pointer to the COM interface as first argument, in addition to those parameters that are specified in the `argtypes` tuple.

Disponibilidade: Windows

The optional *paramflags* parameter creates foreign function wrappers with much more functionality than the features described above.

paramflags must be a tuple of the same length as *argtypes*.

Each item in this tuple contains further information about a parameter, it must be a tuple containing one, two, or three items.

The first item is an integer containing a combination of direction flags for the parameter:

1
: Specifies an input parameter to the function.

2
: Output parameter. The foreign function fills in a value.

4
: Input parameter which defaults to the integer zero.

The optional second item is the parameter name as string. If this is specified, the foreign function can be called with named parameters.

The optional third item is the default value for this parameter.

The following example demonstrates how to wrap the Windows `MessageBoxW` function so that it supports default parameters and named arguments. The C declaration from the windows header file is this:

```
WINUSERAPI int WINAPI
MessageBoxW(
    HWND hWnd,
    LPCWSTR lpText,
    LPCWSTR lpCaption,
    UINT uType);
```

Here is the wrapping with *ctypes*:

```
>>> from ctypes import c_int, WINFUNCTYPE, windll
>>> from ctypes.wintypes import HWND, LPCWSTR, UINT
>>> prototype = WINFUNCTYPE(c_int, HWND, LPCWSTR, LPCWSTR, UINT)
>>> paramflags = (1, "hwnd", 0), (1, "text", "Hi"), (1, "caption", "Hello from␣
```

(continua na próxima página)

(continuação da página anterior)

```
↪ctypes"), (1, "flags", 0)
>>> MessageBox = prototype(("MessageBoxW", windll.user32), paramflags)
```

The `MessageBox` foreign function can now be called in these ways:

```
>>> MessageBox()
>>> MessageBox(text="Spam, spam, spam")
>>> MessageBox(flags=2, text="foo bar")
```

A second example demonstrates output parameters. The win32 `GetWindowRect` function retrieves the dimensions of a specified window by copying them into `RECT` structure that the caller has to supply. Here is the C declaration:

```
WINUSERAPI BOOL WINAPI
GetWindowRect(
     HWND hWnd,
     LPRECT lpRect);
```

Here is the wrapping with *ctypes*:

```
>>> from ctypes import POINTER, WINFUNCTYPE, windll, WinError
>>> from ctypes.wintypes import BOOL, HWND, RECT
>>> prototype = WINFUNCTYPE(BOOL, HWND, POINTER(RECT))
>>> paramflags = (1, "hwnd"), (2, "lprect")
>>> GetWindowRect = prototype(("GetWindowRect", windll.user32), paramflags)
>>>
```

Functions with output parameters will automatically return the output parameter value if there is a single one, or a tuple containing the output parameter values when there are more than one, so the GetWindowRect function now returns a RECT instance, when called.

Output parameters can be combined with the *errcheck* protocol to do further output processing and error checking. The win32 `GetWindowRect` api function returns a `BOOL` to signal success or failure, so this function could do the error checking, and raises an exception when the api call failed:

```
>>> def errcheck(result, func, args):
...     if not result:
...         raise WinError()
...     return args
...
>>> GetWindowRect.errcheck = errcheck
>>>
```

If the *errcheck* function returns the argument tuple it receives unchanged, *ctypes* continues the normal processing it does on the output parameters. If you want to return a tuple of window coordinates instead of a `RECT` instance, you can retrieve the fields in the function and return them instead, the normal processing will no longer take place:

```
>>> def errcheck(result, func, args):
...     if not result:
...         raise WinError()
...     rc = args[1]
...     return rc.left, rc.top, rc.bottom, rc.right
...
>>> GetWindowRect.errcheck = errcheck
>>>
```

Funções utilitárias

ctypes.**addressof**(*obj*)

Returns the address of the memory buffer as integer. *obj* must be an instance of a ctypes type.

Levanta um *evento de auditoria* `ctypes.addressof` com o argumento `obj`.

ctypes.**alignment**(*obj_or_type*)

Returns the alignment requirements of a ctypes type. *obj_or_type* must be a ctypes type or instance.

ctypes.**byref**(*obj*[, *offset*])

Returns a light-weight pointer to *obj*, which must be an instance of a ctypes type. *offset* defaults to zero, and must be an integer that will be added to the internal pointer value.

`byref(obj, offset)` corresponds to this C code:

```
(((char *)&obj) + offset)
```

The returned object can only be used as a foreign function call parameter. It behaves similar to `pointer(obj)`, but the construction is a lot faster.

ctypes.**CopyComPointer**(*src*, *dst*)

Copies a COM pointer from *src* to *dst* and returns the Windows specific `HRESULT` value.

If *src* is not `NULL`, its `AddRef` method is called, incrementing the reference count.

In contrast, the reference count of *dst* will not be decremented before assigning the new value. Unless *dst* is `NULL`, the caller is responsible for decrementing the reference count by calling its `Release` method when necessary.

Disponibilidade: Windows

Adicionado na versão 3.14.

ctypes.**cast**(*obj*, *type*)

This function is similar to the cast operator in C. It returns a new instance of *type* which points to the same memory block as *obj*. *type* must be a pointer type, and *obj* must be an object that can be interpreted as a pointer.

ctypes.**create_string_buffer**(*init*, *size=None*)

ctypes.**create_string_buffer**(*size*)

This function creates a mutable character buffer. The returned object is a ctypes array of *c_char*.

If *size* is given (and not `None`), it must be an *int*. It specifies the size of the returned array.

If the *init* argument is given, it must be *bytes*. It is used to initialize the array items. Bytes not initialized this way are set to zero (NUL).

If *size* is not given (or if it is `None`), the buffer is made one element larger than *init*, effectively adding a NUL terminator.

If both arguments are given, *size* must not be less than `len(init)`.

> **Aviso**
>
> If *size* is equal to `len(init)`, a NUL terminator is not added. Do not treat such a buffer as a C string.

Por exemplo:

```
>>> bytes(create_string_buffer(2))
b'\x00\x00'
>>> bytes(create_string_buffer(b'ab'))
b'ab\x00'
>>> bytes(create_string_buffer(b'ab', 2))
```

(continua na próxima página)

(continuação da página anterior)

```
b'ab'
>>> bytes(create_string_buffer(b'ab', 4))
b'ab\x00\x00'
>>> bytes(create_string_buffer(b'abcdef', 2))
Traceback (most recent call last):
   ...
ValueError: byte string too long
```

Levanta um *evento de auditoria* `ctypes.create_string_buffer` com os argumentos `init`, `size`.

`ctypes.`**`create_unicode_buffer`**(*init*, *size=None*)

`ctypes.`**`create_unicode_buffer`**(*size*)

This function creates a mutable unicode character buffer. The returned object is a ctypes array of *c_wchar*.

The function takes the same arguments as *create_string_buffer()* except *init* must be a string and *size* counts *c_wchar*.

Levanta um *evento de auditoria* `ctypes.create_unicode_buffer` com os argumentos `init`, `size`.

`ctypes.`**`DllCanUnloadNow`**()

This function is a hook which allows implementing in-process COM servers with ctypes. It is called from the DllCanUnloadNow function that the _ctypes extension dll exports.

Disponibilidade: Windows

`ctypes.`**`DllGetClassObject`**()

This function is a hook which allows implementing in-process COM servers with ctypes. It is called from the DllGetClassObject function that the `_ctypes` extension dll exports.

Disponibilidade: Windows

`ctypes.util.`**`find_library`**(*name*)

Try to find a library and return a pathname. *name* is the library name without any prefix like `lib`, suffix like `.so`, `.dylib` or version number (this is the form used for the posix linker option `-l`). If no library can be found, returns `None`.

The exact functionality is system dependent.

`ctypes.util.`**`find_msvcrt`**()

Returns the filename of the VC runtime library used by Python, and by the extension modules. If the name of the library cannot be determined, `None` is returned.

If you need to free memory, for example, allocated by an extension module with a call to the `free(void *)`, it is important that you use the function in the same library that allocated the memory.

Disponibilidade: Windows

`ctypes.util.`**`dllist`**()

Try to provide a list of paths of the shared libraries loaded into the current process. These paths are not normalized or processed in any way. The function can raise *OSError* if the underlying platform APIs fail. The exact functionality is system dependent.

On most platforms, the first element of the list represents the current executable file. It may be an empty string.

Disponibilidade: Windows, macOS, iOS, glibc, BSD libc, musl

Adicionado na versão 3.14.

`ctypes.`**`FormatError`**([*code*])

Returns a textual description of the error code *code*. If no error code is specified, the last error code is used by calling the Windows API function *GetLastError()*.

Disponibilidade: Windows

ctypes.**GetLastError**()

Returns the last error code set by Windows in the calling thread. This function calls the Windows `GetLastError()` function directly, it does not return the ctypes-private copy of the error code.

Disponibilidade: Windows

ctypes.**get_errno**()

Returns the current value of the ctypes-private copy of the system *errno* variable in the calling thread.

Levanta um *evento de auditoria* `ctypes.get_errno` sem argumentos.

ctypes.**get_last_error**()

Returns the current value of the ctypes-private copy of the system `LastError` variable in the calling thread.

Disponibilidade: Windows

Levanta um *evento de auditoria* `ctypes.get_last_error` sem argumentos.

ctypes.**memmove**(*dst*, *src*, *count*)

Same as the standard C memmove library function: copies *count* bytes from *src* to *dst*. *dst* and *src* must be integers or ctypes instances that can be converted to pointers.

ctypes.**memset**(*dst*, *c*, *count*)

Same as the standard C memset library function: fills the memory block at address *dst* with *count* bytes of value *c*. *dst* must be an integer specifying an address, or a ctypes instance.

ctypes.**POINTER**(*type*, /)

Create or return a ctypes pointer type. Pointer types are cached and reused internally, so calling this function repeatedly is cheap. *type* must be a ctypes type.

Detalhes da implementação do CPython: The resulting pointer type is cached in the `__pointer_type__` attribute of *type*. It is possible to set this attribute before the first call to `POINTER` in order to set a custom pointer type. However, doing this is discouraged: manually creating a suitable pointer type is difficult without relying on implementation details that may change in future Python versions.

ctypes.**pointer**(*obj*, /)

Create a new pointer instance, pointing to *obj*. The returned object is of the type `POINTER(type(obj))`.

Note: If you just want to pass a pointer to an object to a foreign function call, you should use `byref(obj)` which is much faster.

ctypes.**resize**(*obj*, *size*)

This function resizes the internal memory buffer of *obj*, which must be an instance of a ctypes type. It is not possible to make the buffer smaller than the native size of the objects type, as given by `sizeof(type(obj))`, but it is possible to enlarge the buffer.

ctypes.**set_errno**(*value*)

Set the current value of the ctypes-private copy of the system *errno* variable in the calling thread to *value* and return the previous value.

Levanta um *evento de auditoria* `ctypes.set_errno` com o argumento `errno`.

ctypes.**set_last_error**(*value*)

Sets the current value of the ctypes-private copy of the system `LastError` variable in the calling thread to *value* and return the previous value.

Disponibilidade: Windows

Levanta um *evento de auditoria* `ctypes.set_last_error` com o argumento `error`.

ctypes.**sizeof**(*obj_or_type*)

Returns the size in bytes of a ctypes type or instance memory buffer. Does the same as the C `sizeof` operator.

ctypes.**string_at**(*ptr*, *size=-1*)

Return the byte string at *void *ptr*. If *size* is specified, it is used as size, otherwise the string is assumed to be zero-terminated.

Levanta um *evento de auditoria* `ctypes.string_at` com os argumentos `ptr`, `size`.

ctypes.**WinError**(*code=None*, *descr=None*)

Creates an instance of *OSError*. If *code* is not specified, *GetLastError()* is called to determine the error code. If *descr* is not specified, *FormatError()* is called to get a textual description of the error.

Disponibilidade: Windows

Alterado na versão 3.3: An instance of *WindowsError* used to be created, which is now an alias of *OSError*.

ctypes.**wstring_at**(*ptr*, *size=-1*)

Return the wide-character string at *void *ptr*. If *size* is specified, it is used as the number of characters of the string, otherwise the string is assumed to be zero-terminated.

Levanta um *evento de auditoria* `ctypes.wstring_at` com os argumentos `ptr`, `size`.

ctypes.**memoryview_at**(*ptr*, *size*, *readonly=False*)

Return a *memoryview* object of length *size* that references memory starting at *void *ptr*.

If *readonly* is true, the returned `memoryview` object can not be used to modify the underlying memory. (Changes made by other means will still be reflected in the returned object.)

This function is similar to *string_at()* with the key difference of not making a copy of the specified memory. It is a semantically equivalent (but more efficient) alternative to `memoryview((c_byte * size).from_address(ptr))`. (While *from_address()* only takes integers, *ptr* can also be given as a *ctypes.POINTER* or a *byref()* object.)

Raises an *auditing event* `ctypes.memoryview_at` with arguments `address`, `size`, `readonly`.

Adicionado na versão 3.14.

Data types

class ctypes.**_CData**

This non-public class is the common base class of all ctypes data types. Among other things, all ctypes type instances contain a memory block that hold C compatible data; the address of the memory block is returned by the *addressof()* helper function. Another instance variable is exposed as *_objects*; this contains other Python objects that need to be kept alive in case the memory block contains pointers.

Common methods of ctypes data types, these are all class methods (to be exact, they are methods of the *metaclass*):

from_buffer(*source*[, *offset*])

This method returns a ctypes instance that shares the buffer of the *source* object. The *source* object must support the writeable buffer interface. The optional *offset* parameter specifies an offset into the source buffer in bytes; the default is zero. If the source buffer is not large enough a *ValueError* is raised.

Levanta um *evento de auditoria* `ctypes.cdata/buffer` com os argumentos `pointer`, `size`, `offset`.

from_buffer_copy(*source*[, *offset*])

This method creates a ctypes instance, copying the buffer from the *source* object buffer which must be readable. The optional *offset* parameter specifies an offset into the source buffer in bytes; the default is zero. If the source buffer is not large enough a *ValueError* is raised.

Levanta um *evento de auditoria* `ctypes.cdata/buffer` com os argumentos `pointer`, `size`, `offset`.

from_address(*address*)

This method returns a ctypes type instance using the memory specified by *address* which must be an integer.

Este método, e outros que indiretamente chamam este método, levantam um *evento de auditoria* `ctypes.cdata` com o argumento `address`.

from_param(*obj*)

This method adapts *obj* to a ctypes type. It is called with the actual object used in a foreign function call when the type is present in the foreign function's *argtypes* tuple; it must return an object that can be used as a function call parameter.

All ctypes data types have a default implementation of this classmethod that normally returns *obj* if that is an instance of the type. Some types accept other objects as well.

in_dll(*library*, *name*)

This method returns a ctypes type instance exported by a shared library. *name* is the name of the symbol that exports the data, *library* is the loaded shared library.

Common class variables of ctypes data types:

__pointer_type__

The pointer type that was created by calling *POINTER()* for corresponding ctypes data type. If a pointer type was not yet created, the attribute is missing.

Adicionado na versão 3.14.

Common instance variables of ctypes data types:

_b_base_

Sometimes ctypes data instances do not own the memory block they contain, instead they share part of the memory block of a base object. The *_b_base_* read-only member is the root ctypes object that owns the memory block.

_b_needsfree_

This read-only variable is true when the ctypes data instance has allocated the memory block itself, false otherwise.

_objects

This member is either `None` or a dictionary containing Python objects that need to be kept alive so that the memory block contents is kept valid. This object is only exposed for debugging; never modify the contents of this dictionary.

Tipos de dados fundamentais

class ctypes.**_SimpleCData**

This non-public class is the base class of all fundamental ctypes data types. It is mentioned here because it contains the common attributes of the fundamental ctypes data types. *_SimpleCData* is a subclass of *_CData*, so it inherits their methods and attributes. ctypes data types that are not and do not contain pointers can now be pickled.

Instances have a single attribute:

value

This attribute contains the actual value of the instance. For integer and pointer types, it is an integer, for character types, it is a single character bytes object or string, for character pointer types it is a Python bytes object or string.

When the `value` attribute is retrieved from a ctypes instance, usually a new object is returned each time. *ctypes* does *not* implement original object return, always a new object is constructed. The same is true for all other ctypes object instances.

Fundamental data types, when returned as foreign function call results, or, for example, by retrieving structure field members or array items, are transparently converted to native Python types. In other words, if a foreign function has a *restype* of *c_char_p*, you will always receive a Python bytes object, *not* a *c_char_p* instance.

Subclasses of fundamental data types do *not* inherit this behavior. So, if a foreign functions `restype` is a subclass of *c_void_p*, you will receive an instance of this subclass from the function call. Of course, you can get the value of the pointer by accessing the `value` attribute.

These are the fundamental ctypes data types:

class ctypes.**c_byte**

Represents the C `signed char` datatype, and interprets the value as small integer. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.**c_char**

Represents the C `char` datatype, and interprets the value as a single character. The constructor accepts an optional string initializer, the length of the string must be exactly one character.

class ctypes.**c_char_p**

Represents the C `char*` datatype when it points to a zero-terminated string. For a general character pointer that may also point to binary data, `POINTER(c_char)` must be used. The constructor accepts an integer address, or a bytes object.

class ctypes.**c_double**

Represents the C `double` datatype. The constructor accepts an optional float initializer.

class ctypes.**c_longdouble**

Represents the C `long double` datatype. The constructor accepts an optional float initializer. On platforms where `sizeof(long double) == sizeof(double)` it is an alias to *c_double*.

class ctypes.**c_float**

Represents the C `float` datatype. The constructor accepts an optional float initializer.

class ctypes.**c_double_complex**

Represents the C `double complex` datatype, if available. The constructor accepts an optional *complex* initializer.

Adicionado na versão 3.14.

class ctypes.**c_float_complex**

Represents the C `float complex` datatype, if available. The constructor accepts an optional *complex* initializer.

Adicionado na versão 3.14.

class ctypes.**c_longdouble_complex**

Represents the C `long double complex` datatype, if available. The constructor accepts an optional *complex* initializer.

Adicionado na versão 3.14.

class ctypes.**c_int**

Represents the C `signed int` datatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms where `sizeof(int) == sizeof(long)` it is an alias to *c_long*.

class ctypes.**c_int8**

Represents the C 8-bit `signed int` datatype. It is an alias for *c_byte*.

class ctypes.**c_int16**

Represents the C 16-bit `signed int` datatype. Usually an alias for *c_short*.

class ctypes.**c_int32**

Represents the C 32-bit `signed int` datatype. Usually an alias for *c_int*.

class ctypes.**c_int64**

Represents the C 64-bit `signed int` datatype. Usually an alias for *c_longlong*.

class ctypes.**c_long**

Represents the C `signed long` datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.**c_longlong**

Represents the C `signed long long` datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.**c_short**

Represents the C `signed short` datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.**c_size_t**

Represents the C `size_t` datatype.

class ctypes.**c_ssize_t**

Represents the C `ssize_t` datatype.

Adicionado na versão 3.2.

class ctypes.**c_time_t**

Represents the C `time_t` datatype.

Adicionado na versão 3.12.

class ctypes.**c_ubyte**

Represents the C `unsigned char` datatype, it interprets the value as small integer. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.**c_uint**

Represents the C `unsigned int` datatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms where `sizeof(int) == sizeof(long)` it is an alias for *c_ulong*.

class ctypes.**c_uint8**

Represents the C 8-bit `unsigned int` datatype. It is an alias for *c_ubyte*.

class ctypes.**c_uint16**

Represents the C 16-bit `unsigned int` datatype. Usually an alias for *c_ushort*.

class ctypes.**c_uint32**

Represents the C 32-bit `unsigned int` datatype. Usually an alias for *c_uint*.

class ctypes.**c_uint64**

Represents the C 64-bit `unsigned int` datatype. Usually an alias for *c_ulonglong*.

class ctypes.**c_ulong**

Represents the C `unsigned long` datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.**c_ulonglong**

Represents the C `unsigned long long` datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.**c_ushort**

Represents the C `unsigned short` datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.**c_void_p**

Represents the C `void*` type. The value is represented as integer. The constructor accepts an optional integer initializer.

class ctypes.**c_wchar**

Represents the C `wchar_t` datatype, and interprets the value as a single character unicode string. The constructor accepts an optional string initializer, the length of the string must be exactly one character.

class ctypes.**c_wchar_p**

Represents the C `wchar_t*` datatype, which must be a pointer to a zero-terminated wide character string. The constructor accepts an integer address, or a string.

class ctypes.**c_bool**

Represent the C `bool` datatype (more accurately, `_Bool` from C99). Its value can be `True` or `False`, and the constructor accepts any object that has a truth value.

class ctypes.**HRESULT**

Represents a `HRESULT` value, which contains success or error information for a function or method call.

Disponibilidade: Windows

class ctypes.**py_object**

Represents the C `PyObject*` datatype. Calling this without an argument creates a `NULL` `PyObject*` pointer.

Alterado na versão 3.14: `py_object` is now a *generic type*.

The `ctypes.wintypes` module provides quite some other Windows specific data types, for example `HWND`, `WPARAM`, or `DWORD`. Some useful structures like `MSG` or `RECT` are also defined.

Structured data types

class ctypes.**Union**(**args*, ***kw*)

Abstract base class for unions in native byte order.

Unions share common attributes and behavior with structures; see *`Structure`* documentation for details.

class ctypes.**BigEndianUnion**(**args*, ***kw*)

Abstract base class for unions in *big endian* byte order.

Adicionado na versão 3.11.

class ctypes.**LittleEndianUnion**(**args*, ***kw*)

Abstract base class for unions in *little endian* byte order.

Adicionado na versão 3.11.

class ctypes.**BigEndianStructure**(**args*, ***kw*)

Abstract base class for structures in *big endian* byte order.

class ctypes.**LittleEndianStructure**(**args*, ***kw*)

Abstract base class for structures in *little endian* byte order.

Structures and unions with non-native byte order cannot contain pointer type fields, or any other data types containing pointer type fields.

class ctypes.**Structure**(**args*, ***kw*)

Abstract base class for structures in *native* byte order.

Concrete structure and union types must be created by subclassing one of these types, and at least define a *`_fields_`* class variable. *`ctypes`* will create *descriptor*s which allow reading and writing the fields by direct attribute accesses. These are the

fields

A sequence defining the structure fields. The items must be 2-tuples or 3-tuples. The first item is the name of the field, the second item specifies the type of the field; it can be any ctypes data type.

For integer type fields like *`c_int`*, a third optional item can be given. It must be a small positive integer defining the bit width of the field.

Field names must be unique within one structure or union. This is not checked, only one field can be accessed when names are repeated.

It is possible to define the *`_fields_`* class variable *after* the class statement that defines the Structure subclass, this allows creating data types that directly or indirectly reference themselves:

```
class List(Structure):
    pass
List._fields_ = [("pnext", POINTER(List)),
                 ...
                ]
```

The _fields_ class variable can only be set once. Later assignments will raise an *AttributeError*.

Additionally, the _fields_ class variable must be defined before the structure or union type is first used: an instance or subclass is created, *sizeof()* is called on it, and so on. Later assignments to _fields_ will raise an *AttributeError*. If _fields_ has not been set before such use, the structure or union will have no own fields, as if _fields_ was empty.

Sub-subclasses of structure types inherit the fields of the base class plus the *_fields_* defined in the sub-subclass, if any.

pack

An optional small integer that allows overriding the alignment of structure fields in the instance. *_pack_* must already be defined when *_fields_* is assigned, otherwise it will have no effect. Setting this attribute to 0 is the same as not setting it at all.

This is only implemented for the MSVC-compatible memory layout.

Deprecated since version 3.14, will be removed in version 3.19: For historical reasons, if _pack_ is non-zero, the MSVC-compatible layout will be used by default. On non-Windows platforms, this default is deprecated and is slated to become an error in Python 3.19. If it is intended, set *_layout_* to `'ms'` explicitly.

align

An optional small integer that allows overriding the alignment of the structure when being packed or unpacked to/from memory. Setting this attribute to 0 is the same as not setting it at all.

Adicionado na versão 3.13.

layout

An optional string naming the struct/union layout. It can currently be set to:

- `"ms"`: the layout used by the Microsoft compiler (MSVC). On GCC and Clang, this layout can be selected with `__attribute__((ms_struct))`.
- `"gcc-sysv"`: the layout used by GCC with the System V or "SysV-like" data model, as used on Linux and macOS. With this layout, *_pack_* must be unset or zero.

If not set explicitly, `ctypes` will use a default that matches the platform conventions. This default may change in future Python releases (for example, when a new platform gains official support, or when a difference between similar platforms is found). Currently the default will be:

- On Windows: `"ms"`
- When *_pack_* is specified: `"ms"`. (This is deprecated; see *_pack_* documentation.)
- Otherwise: `"gcc-sysv"`

layout must already be defined when *_fields_* is assigned, otherwise it will have no effect.

Adicionado na versão 3.14.

anonymous

An optional sequence that lists the names of unnamed (anonymous) fields. *_anonymous_* must be already defined when *_fields_* is assigned, otherwise it will have no effect.

The fields listed in this variable must be structure or union type fields. *ctypes* will create descriptors in the structure type that allows accessing the nested fields directly, without the need to create the structure or union field.

Here is an example type (Windows):

```
class _U(Union):
    _fields_ = [("lptdesc", POINTER(TYPEDESC)),
                ("lpadesc", POINTER(ARRAYDESC)),
                ("hreftype", HREFTYPE)]

class TYPEDESC(Structure):
    _anonymous_ = ("u",)
    _fields_ = [("u", _U),
                ("vt", VARTYPE)]
```

The `TYPEDESC` structure describes a COM data type, the `vt` field specifies which one of the union fields is valid. Since the `u` field is defined as anonymous field, it is now possible to access the members directly off the TYPEDESC instance. `td.lptdesc` and `td.u.lptdesc` are equivalent, but the former is faster since it does not need to create a temporary union instance:

```
td = TYPEDESC()
td.vt = VT_PTR
td.lptdesc = POINTER(some_type)
td.u.lptdesc = POINTER(some_type)
```

It is possible to define sub-subclasses of structures, they inherit the fields of the base class. If the subclass definition has a separate *`_fields_`* variable, the fields specified in this are appended to the fields of the base class.

Structure and union constructors accept both positional and keyword arguments. Positional arguments are used to initialize member fields in the same order as they are appear in *`_fields_`*. Keyword arguments in the constructor are interpreted as attribute assignments, so they will initialize *`_fields_`* with the same name, or create new attributes for names not present in *`_fields_`*.

class ctypes.**CField**(**args*, ***kw*)

Descriptor for fields of a *`Structure`* and *`Union`*. For example:

```
>>> class Color(Structure):
...     _fields_ = (
...         ('red', c_uint8),
...         ('green', c_uint8),
...         ('blue', c_uint8),
...         ('intense', c_bool, 1),
...         ('blinking', c_bool, 1),
...     )
...
>>> Color.red
<ctypes.CField 'red' type=c_ubyte, ofs=0, size=1>
>>> Color.green.type
<class 'ctypes.c_ubyte'>
>>> Color.blue.byte_offset
2
>>> Color.intense
<ctypes.CField 'intense' type=c_bool, ofs=3, bit_size=1, bit_offset=0>
>>> Color.blinking.bit_offset
1
```

All attributes are read-only.

`CField` objects are created via *`_fields_`*; do not instantiate the class directly.

Adicionado na versão 3.14: Previously, descriptors only had `offset` and `size` attributes and a readable string representation; the `CField` class was not available directly.

name

Name of the field, as a string.

type

Type of the field, as a *ctypes class*.

offset

byte_offset

Offset of the field, in bytes.

For bitfields, this is the offset of the underlying byte-aligned *storage unit*; see `bit_offset`.

byte_size

Size of the field, in bytes.

For bitfields, this is the size of the underlying *storage unit*. Typically, it has the same size as the bitfield's type.

size

For non-bitfields, equivalent to `byte_size`.

For bitfields, this contains a backwards-compatible bit-packed value that combines `bit_size` and `bit_offset`. Prefer using the explicit attributes instead.

is_bitfield

True if this is a bitfield.

bit_offset

bit_size

The location of a bitfield within its *storage unit*, that is, within `byte_size` bytes of memory starting at `byte_offset`.

To get the field's value, read the storage unit as an integer, shift left by `bit_offset` and take the `bit_size` least significant bits.

For non-bitfields, `bit_offset` is zero and `bit_size` is equal to `byte_size * 8`.

is_anonymous

True if this field is anonymous, that is, it contains nested sub-fields that should be merged into a containing structure or union.

Arrays and pointers

class ctypes.**Array**(**args*)

Abstract base class for arrays.

The recommended way to create concrete array types is by multiplying any `ctypes` data type with a non-negative integer. Alternatively, you can subclass this type and define `_length_` and `_type_` class variables. Array elements can be read and written using standard subscript and slice accesses; for slice reads, the resulting object is *not* itself an `Array`.

length

A positive integer specifying the number of elements in the array. Out-of-range subscripts result in an `IndexError`. Will be returned by `len()`.

type

Specifies the type of each element in the array.

Array subclass constructors accept positional arguments, used to initialize the elements in order.

ctypes.**ARRAY**(*type*, *length*)

Create an array. Equivalent to `type * length`, where *type* is a `ctypes` data type and *length* an integer.

This function is *soft deprecated* in favor of multiplication. There are no plans to remove it.

class ctypes.**_Pointer**

Private, abstract base class for pointers.

Concrete pointer types are created by calling *POINTER()* with the type that will be pointed to; this is done automatically by *pointer()*.

If a pointer points to an array, its elements can be read and written using standard subscript and slice accesses. Pointer objects have no size, so *len()* will raise *TypeError*. Negative subscripts will read from the memory *before* the pointer (as in C), and out-of-range subscripts will probably crash with an access violation (if you're lucky).

type

Specifies the type pointed to.

contents

Returns the object to which to pointer points. Assigning to this attribute changes the pointer to point to the assigned object.

Exceções

exception ctypes.**ArgumentError**

This exception is raised when a foreign function call cannot convert one of the passed arguments.

exception ctypes.**COMError**(*hresult*, *text*, *details*)

This exception is raised when a COM method call failed.

hresult

The integer value representing the error code.

text

The error message.

details

The 5-tuple `(descr, source, helpfile, helpcontext, progid)`.

descr is the textual description. *source* is the language-dependent `ProgID` for the class or application that raised the error. *helpfile* is the path of the help file. *helpcontext* is the help context identifier. *progid* is the `ProgID` of the interface that defined the error.

Disponibilidade: Windows

Adicionado na versão 3.14.

CAPÍTULO 17

Bibliotecas de interface de linha de comando

Os módulos descritos neste capítulo ajudam a implementar as interfaces de linha de comando e de terminal para aplicações.

Veja aqui uma visão geral:

17.1 `argparse` — Analisador sintático para opções de linha de comando, argumentos e subcomandos

Adicionado na versão 3.2.

Código-fonte: Lib/argparse.py

Nota

Embora o `argparse` seja o módulo de biblioteca padrão recomendado para implementar aplicações básicas de linha de comando, autores com requisitos mais rigorosos sobre como exatamente suas aplicações de linha de comando se comportam podem descobrir que ele não fornece o nível de controle necessário. Consulte *Escolhendo uma biblioteca de análise de linha de comando* para alternativas a serem consideradas quando `argparse` não suporta comportamentos que a aplicação requer (como desabilitar completamente o suporte para opções intercaladas e argumentos posicionais, ou aceitar valores de parâmetros de opção que começam com – mesmo quando correspondem a outra opção definida).

Tutorial

Esta página contém informações da API de Referência. Para uma introdução mais prática para o parser de linha de comando Python, acesse o *tutorial do argparse*.

O módulo `argparse` torna fácil a escrita de interfaces de linha de comando amigáveis. O programa define quais argumentos são necessários e `argparse` descobrirá como analisá-lo e interpretá-los a partir do `sys.argv`. O módulo `argparse` também gera automaticamente o texto ajuda e mensagens de uso. O módulo também vai emitir erros quando o usuário prover argumentos inválidos para o programa.

O suporte do módulo `argparse` para interfaces de linha de comando é construído em torno de uma instância de *`argparse.ArgumentParser`*. É um contêiner para especificações de argumentos e possui opções que se aplicam ao analisador sintático como um todo:

```
parser = argparse.ArgumentParser(
                    prog='NomePrograma',
                    description='O que o programa faz',
                    epilog='Texto na parte inferior da memsagem de ajuda')
```

O método *`ArgumentParser.add_argument()`* anexa especificações de argumentos individuais ao analisador. Ele oferece suporte a argumentos posicionais, opções que aceitam valores e sinalizadores de ligar/desligar:

```
parser.add_argument('filename')           # argumento posicional
parser.add_argument('-c', '--count')      # a opção recebe um valor
parser.add_argument('-v', '--verbose',
                    action='store_true')  # sinalizador de ligar/desligar
```

O método *`ArgumentParser.parse_args()`* executa o analisador e coloca os dados extraídos em um objeto *`argparse.Namespace`*:

```
args = parser.parse_args()
print(args.filename, args.count, args.verbose)
```

Nota

Se você estiver procurando um guia sobre como atualizar um código *`optparse`* para `argparse`, veja *Atualizando código optparse*.

17.1.1 Objetos ArgumentParser

class argparse.**ArgumentParser**(*prog=None*, *usage=None*, *description=None*, *epilog=None*, *parents=[]*, *formatter_class=argparse.HelpFormatter*, *prefix_chars='-'*, *fromfile_prefix_chars=None*, *argument_default=None*, *conflict_handler='error'*, *add_help=True*, *allow_abbrev=True*, *exit_on_error=True*, ***, *suggest_on_error=False*, *color=True*)

Cria um novo objeto *`ArgumentParser`*. Todos os parâmetros devem ser passados como argumentos nomeados. Cada parâmetro tem sua própria descrição mais detalhada abaixo, mas em resumo eles são:

- *prog* - O nome do programa (padrão: gerado a partir de atributos de módulo `__main__` e de `sys.argv[0]`)
- *usage* - A string que descreve o uso do programa (padrão: gerado a partir de argumentos adicionados ao analisador sintático)
- *description* - Texto para exibir antes da ajuda dos argumentos (por padrão, nenhum texto)
- *epilog* - Texto para exibir após da ajuda dos argumentos (por padrão, nenhum texto)
- *parents* - Uma lista de objetos *`ArgumentParser`* cujos argumentos também devem ser incluídos
- *formatter_class* - Uma classe para personalizar a saída de ajuda
- *prefix_chars* - O conjunto de caracteres que prefixam argumentos opcionais (padrão: “-“)
- *fromfile_prefix_chars* - O conjunto de caracteres que prefixam os arquivos dos quais os argumentos adicionais devem ser lidos (padrão: `None`)
- *argument_default* - O valor padrão global para argumentos (padrão: `None`)
- *conflict_handler* - A estratégia para resolver opcionais conflitantes (geralmente desnecessário)
- *add_help* - Adiciona uma opção `-h/--help` para o analisador sintático (padrão: `True`)

- *allow_abbrev* - Permite que opções longas sejam abreviadas se a abreviação não for ambígua. (padrão: `True`)
- *exit_on_error* - Determina se `ArgumentParser` sai ou não com informações de erro quando ocorre um erro. (padrão: `True`)
- *suggest_on_error* - Habilita sugestões para escolhas de argumentos e nomes de subanalisadores digitados incorretamente (padrão: `False`)
- *color* - Permite saída colorida (padrão: `True`)

Alterado na versão 3.5: O parâmetro *allow_abbrev* foi adicionado.

Alterado na versão 3.8: Em versões anteriores, *allow_abbrev* também desabilitava o agrupamento de sinalizadores curtos, como `-vv` para significar `-v -v`.

Alterado na versão 3.9: O parâmetro *exit_on_error* foi adicionado.

Alterado na versão 3.14: Os parâmetros *suggest_on_error* e *color* foram adicionados.

As seções a seguir descrevem como cada um deles é usado.

prog

Por padrão, *ArgumentParser* calcula o nome do programa a ser exibido nas mensagens de ajuda dependendo da maneira como o interpretador Python foi executado:

- O *nome base* de `sys.argv[0]` se um arquivo foi passado como argumento.
- O nome do interpretador Python seguido por `sys.argv[0]` se um diretório ou um arquivo zip foi passado como argumento.
- O nome do interpretador Python seguido por `-m` seguido pelo nome do módulo ou pacote se a opção `-m` foi usada.

Este padrão é quase sempre desejável porque fará com que as mensagens de ajuda correspondam à string que foi usada para invocar o programa na linha de comando. No entanto, para alterar esse comportamento padrão, outro valor pode ser fornecido usando o argumento `prog=` para *ArgumentParser*:

```
>>> parser = argparse.ArgumentParser(prog='meuprograma')
>>> parser.print_help()
usage: meuprograma [-h]

options:
 -h, --help  show this help message and exit
```

Observe que o nome do programa, seja determinado a partir de `sys.argv[0]`, a partir de atributos de módulo `__main__` ou do argumento `prog=`, está disponível para mensagens de ajuda usando o especificador de formato `%(prog)s`.

```
>>> parser = argparse.ArgumentParser(prog='meuprograma')
>>> parser.add_argument('--foo', help='foo do programa %(prog)s')
>>> parser.print_help()
usage: meuprograma [-h] [--foo FOO]

options:
 -h, --help  show this help message and exit
 --foo FOO   foo do programa meuprograma
```

Alterado na versão 3.14: O valor padrão `prog` agora reflete como `__main__` foi realmente executado, em vez de ser sempre `os.path.basename(sys.argv[0])`.

usage

Por padrão, *ArgumentParser* calcula a mensagem de uso a partir dos argumentos que contém. A mensagem padrão pode ser substituída pelo argumento nomeado `usage=`:

```
>>> parser = argparse.ArgumentParser(prog='PROG', usage='%(prog)s [options]')
>>> parser.add_argument('--foo', nargs='?', help='foo help')
>>> parser.add_argument('bar', nargs='+', help='bar help')
>>> parser.print_help()
usage: PROG [options]

positional arguments:
 bar          bar help

options:
 -h, --help   show this help message and exit
 --foo [FOO]  foo help
```

O especificador de formato `%(prog)s` está disponível para preencher o nome do programa em suas mensagens de uso.

Quando uma mensagem de uso personalizada é especificada para o analisador sintático principal, você também pode considerar passar o argumento `prog` para *add_subparsers()* ou os argumentos `prog` e `usage` para `add_parser()`, para garantir prefixos de comando e informações de uso consistentes entre os subanalisadores.

description

A maioria das chamadas para o construtor *ArgumentParser* usará o argumento nomeado `description=`. Este argumento fornece uma breve descrição do que o programa faz e como funciona. Nas mensagens de ajuda, a descrição é exibida entre a string de uso da linha de comando e as mensagens de ajuda para os vários argumentos.

Por padrão, a descrição terá sua linha quebrada para que se encaixe no espaço fornecido. Para alterar esse comportamento, consulte o argumento *formatter_class*.

epilog

Alguns programas gostam de exibir uma descrição adicional do programa após a descrição dos argumentos. Esse texto pode ser especificado usando o argumento `epilog=` para *ArgumentParser*:

```
>>> parser = argparse.ArgumentParser(
...     description='Um foo que faz bars',
...     epilog="E é assim que você faria foo de bar")
>>> parser.print_help()
usage: argparse.py [-h]

Um foo que faz bars

options:
 -h, --help  show this help message and exit

E é assim que você faria foo de bar
```

Tal como acontece com o argumento *description*, o texto de `epilog=` tem sua quebra de linha habilitada por padrão, mas este comportamento pode ser ajustado com o argumento *formatter_class* para *ArgumentParser*.

parents

Às vezes, vários analisadores sintáticos compartilham um conjunto comum de argumentos. Ao invés de repetir as definições desses argumentos, um único analisador com todos os argumentos compartilhados e passado para o

argumento `parents=` para *ArgumentParser* pode ser usado. O argumento `parents=` pega uma lista de objetos *ArgumentParser*, coleta todas as ações posicionais e opcionais deles, e adiciona essas ações ao objeto *ArgumentParser* sendo construído:

```
>>> parent_parser = argparse.ArgumentParser(add_help=False)
>>> parent_parser.add_argument('--parent', type=int)

>>> foo_parser = argparse.ArgumentParser(parents=[parent_parser])
>>> foo_parser.add_argument('foo')
>>> foo_parser.parse_args(['--parent', '2', 'XXX'])
Namespace(foo='XXX', parent=2)

>>> bar_parser = argparse.ArgumentParser(parents=[parent_parser])
>>> bar_parser.add_argument('--bar')
>>> bar_parser.parse_args(['--bar', 'YYY'])
Namespace(bar='YYY', parent=None)
```

Observe que a maioria dos analisadores sintáticos pais especificará `add_help=False`. Caso contrário, o *ArgumentParser* verá duas opções `-h/--help` (uma no pai e outra no filho) e levantará um erro.

Nota

Você deve inicializar totalmente os analisadores sintáticos antes de passá-los via `parents=`. Se você alterar os analisadores pais após o analisador filho, essas mudanças não serão refletidas no filho.

formatter_class

Objetos *ArgumentParser* permitem que a formação do texto de ajuda seja personalizada por meio da especificação de uma classe de formatação alternativa. Atualmente, há quatro dessas classes:

class argparse.**RawDescriptionHelpFormatter**

class argparse.**RawTextHelpFormatter**

class argparse.**ArgumentDefaultsHelpFormatter**

class argparse.**MetavarTypeHelpFormatter**

RawDescriptionHelpFormatter e *RawTextHelpFormatter* dão mais controle sobre como as descrições textuais são exibidas. Por padrão, objetos *ArgumentParser* quebram em linha os textos *description* e *epilog* nas mensagens de ajuda da linha de comando:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     description='''esta descrição
...         foi indentada de forma estranha,
...             mas está tudo bem''',
...     epilog='''
...             da mesma forma para este epílogo, cujos espaços vão
...         ser apagados e cujas palavras serão quebradas
...         em algumas linhas''')
>>> parser.print_help()
usage: PROG [-h]

esta descrição foi indentada de forma estranha, mas está tudo bem

options:
 -h, --help  show this help message and exit
```

(continua na próxima página)

(continuação da página anterior)

```
da mesma forma para este epílogo, cujos espaços vão ser apagados e
cujas palavras serão quebradas em algumas linhas
```

Passar *RawDescriptionHelpFormatter* como `formatter_class=` indica que *description* e *epilog* já estão formatados corretamente e não devem ter suas linhas quebradas:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     formatter_class=argparse.RawDescriptionHelpFormatter,
...     description=textwrap.dedent('''\
...         Por favor, não bagunce este texto!
...         --------------------------------
...             Eu indentei o texto
...             exatamente na forma
...             que quero que fique
...         '''))
>>> parser.print_help()
usage: PROG [-h]

Por favor, não bagunce este texto!
----------------------------------
   Eu indentei o texto
   exatamente na forma
   que quero que fique

options:
 -h, --help  show this help message and exit
```

RawTextHelpFormatter mantém espaços em branco para todos os tipos de texto de ajuda, incluindo descrições de argumentos. No entanto, várias novas linhas são substituídas por uma. Se você deseja preservar várias linhas em branco, adicione espaços entre as novas linhas.

ArgumentDefaultsHelpFormatter adiciona automaticamente informações sobre os valores padrão para cada uma das mensagens de ajuda do argumento:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
>>> parser.add_argument('--foo', type=int, default=42, help='FOO!')
>>> parser.add_argument('bar', nargs='*', default=[1, 2, 3], help='BAR!')
>>> parser.print_help()
usage: PROG [-h] [--foo FOO] [bar ...]

positional arguments:
 bar         BAR! (default: [1, 2, 3])

options:
 -h, --help  show this help message and exit
 --foo FOO   FOO! (default: 42)
```

MetavarTypeHelpFormatter usa o nome de argumento *type* para cada argumento como o nome de exibição para seus valores (em vez de usar o *dest* como o formatador regular faz):

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     formatter_class=argparse.MetavarTypeHelpFormatter)
>>> parser.add_argument('--foo', type=int)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> parser.add_argument('bar', type=float)
>>> parser.print_help()
usage: PROG [-h] [--foo int] float

positional arguments:
  float

options:
  -h, --help  show this help message and exit
  --foo int
```

prefix_chars

A maioria das opções de linha de comando usará - como prefixo, por exemplo, `-f/--foo`. Analisadores sintáticos que precisam ter suporte a caracteres de prefixo diferentes ou adicionais, por exemplo, para opções como `+f` ou `/foo`, podem especificá-las usando o argumento `prefix_chars=` para o construtor *`ArgumentParser`*:

```
>>> parser = argparse.ArgumentParser(prog='PROG', prefix_chars='-+')
>>> parser.add_argument('+f')
>>> parser.add_argument('++bar')
>>> parser.parse_args('+f X ++bar Y'.split())
Namespace(bar='Y', f='X')
```

O argumento `prefix_chars=` é padronizado como `'-'`. Fornecer um conjunto de caracteres que não inclua - fará com que as opções `-f/--foo` não sejam permitidas.

fromfile_prefix_chars

Às vezes ao lidar com uma lista de argumentos particularmente longa, pode fazer sentido manter a lista de argumentos em um arquivo em vez de digitá-la na linha de comando. Se o argumento `fromfile_prefix_chars=` for dado ao construtor *`ArgumentParser`*, então os argumentos que começam com qualquer um dos caracteres especificados serão tratados como arquivos e serão substituídos pelos argumentos que eles contêm. Por exemplo:

```
>>> with open('args.txt', 'w', encoding=sys.getfilesystemencoding()) as fp:
...     fp.write('-f\nbar')
...
>>> parser = argparse.ArgumentParser(fromfile_prefix_chars='@')
>>> parser.add_argument('-f')
>>> parser.parse_args(['-f', 'foo', '@args.txt'])
Namespace(f='bar')
```

Por padrão, os argumentos lidos de um arquivo devem ser um por linha (mas veja também *`convert_arg_line_to_args()`*) e são tratados como se estivessem no mesmo lugar que o argumento original que faz referência ao arquivo na linha de comando. Portanto, no exemplo acima, a expressão `['-f', 'foo', '@args.txt']` é considerada equivalente à expressão `['-f', 'foo', '-f', 'bar']`.

Nota

Linhas vazias são tratadas como strings vazias (`''`), que são permitidas como valores, mas não como argumentos. Linhas vazias lidas como argumentos resultarão em um erro de "unrecognized arguments" ("argumentos não reconhecidos").

`ArgumentParser` usa *tratador de erros e codificação do sistema de arquivos* para ler o arquivo que contém argumentos.

O argumento `fromfile_prefix_chars=` é padronizado como `None`, significando que os argumentos nunca serão tratados como referências de arquivo.

Alterado na versão 3.12: *ArgumentParser* alterou a codificação e os erros para ler arquivos de argumentos do padrão (por exemplo, *locale.getpreferredencoding(False)* e "strict") para *tratador de erros e codificação do sistema de arquivos*. O arquivo de argumentos deve ser codificado em UTF-8 em vez de página de código ANSI no Windows.

argument_default

Geralmente, os padrões dos argumentos são especificados passando um padrão para *add_argument()* ou chamando os métodos *set_defaults()* com um conjunto específico de pares nome-valor. Às vezes, no entanto, pode ser útil especificar um único padrão para todo o analisador para argumentos. Isso pode ser feito passando o argumento nomeado argument_default= para *ArgumentParser*. Por exemplo, para suprimir globalmente a criação de atributos em chamadas *parse_args()*, fornecemos argument_default=SUPPRESS:

```
>>> parser = argparse.ArgumentParser(argument_default=argparse.SUPPRESS)
>>> parser.add_argument('--foo')
>>> parser.add_argument('bar', nargs='?')
>>> parser.parse_args(['--foo', '1', 'BAR'])
Namespace(bar='BAR', foo='1')
>>> parser.parse_args([])
Namespace()
```

allow_abbrev

Normalmente, quando você passa uma lista de argumentos para o método *parse_args()* de um *ArgumentParser*, ele *reconhece as abreviações* de opções longas.

Este recurso pode ser desabilitado configurando allow_abbrev para False:

```
>>> parser = argparse.ArgumentParser(prog='PROG', allow_abbrev=False)
>>> parser.add_argument('--foobar', action='store_true')
>>> parser.add_argument('--foonley', action='store_false')
>>> parser.parse_args(['--foon'])
usage: PROG [-h] [--foobar] [--foonley]
PROG: error: unrecognized arguments: --foon
```

Adicionado na versão 3.5.

conflict_handler

Objetos *ArgumentParser* não permitem duas ações com a mesma string de opções. Por padrão, objetos *ArgumentParser* levantam uma exceção se for feita uma tentativa de criar um argumento com uma string de opção que já esteja em uso:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-f', '--foo', help='ajuda antiga de foo')
>>> parser.add_argument('--foo', help='nova ajuda de foo')
Traceback (most recent call last):
 ..
ArgumentError: argument --foo: conflicting option string(s): --foo
```

Às vezes (por exemplo, ao usar os *parents*) pode ser útil simplesmente substituir quaisquer argumentos mais antigos com a mesma string de opções. Para obter este comportamento, o valor 'resolve' pode ser fornecido ao argumento conflict_handler= de *ArgumentParser*:

```
>>> parser = argparse.ArgumentParser(prog='PROG', conflict_handler='resolve')
>>> parser.add_argument('-f', '--foo', help='ajuda antiga de foo')
>>> parser.add_argument('--foo', help='nova ajuda de foo')
>>> parser.print_help()
usage: PROG [-h] [-f FOO] [--foo FOO]
```

(continua na próxima página)

(continuação da página anterior)

```
options:
 -h, --help  show this help message and exit
 -f FOO      ajuda antiga de foo
 --foo FOO   nova ajuda de foo
```

Observe que os objetos *ArgumentParser* só removem uma ação se todas as suas strings de opção forem substituídas. Assim, no exemplo acima, a antiga ação `-f/--foo` é mantida como a ação `-f`, porque apenas a string de opção `--foo` foi substituída.

add_help

Por padrão, os objetos *ArgumentParser* adicionam uma opção que simplesmente exibe a mensagem de ajuda do analisador sintático. Se `-h` ou `--help` for fornecido na linha de comando, a ajuda do `ArgumentParser` será exibida.

Às vezes, pode ser útil desabilitar o acréscimo desta opção de ajuda. Isto pode ser feito passando `False` como o argumento `add_help=` para a classe *ArgumentParser*:

```
>>> parser = argparse.ArgumentParser(prog='PROG', add_help=False)
>>> parser.add_argument('--foo', help='ajuda de foo')
>>> parser.print_help()
usage: PROG [--foo FOO]

options:
 --foo FOO  ajuda de foo
```

A opção de ajuda é normalmente `-h/--help`. A exceção a isso é se o `prefix_chars=` for especificado e não incluir `-`, neste caso `-h` e `--help` não são opções válidas. Neste caso, o primeiro caractere em `prefix_chars` é usado para prefixar as opções de ajuda:

```
>>> parser = argparse.ArgumentParser(prog='PROG', prefix_chars='+/')
>>> parser.print_help()
usage: PROG [+h]

options:
  +h, ++help  show this help message and exit
```

exit_on_error

Normalmente, quando você passa uma lista de argumentos inválidos para o método *parse_args()* de uma instância de *ArgumentParser*, ele vai exibir uma *message* para *sys.stderr* e vai sair com o código de status de 2.

Se o usuário quiser detectar erros manualmente, o recurso pode ser habilitado configurando `exit_on_error` para `False`:

```
>>> parser = argparse.ArgumentParser(exit_on_error=False)
>>> parser.add_argument('--integers', type=int)
_StoreAction(option_strings=['--integers'], dest='integers', nargs=None,␣
↪const=None, default=None, type=<class 'int'>, choices=None, help=None,␣
↪metavar=None)
>>> try:
...     parser.parse_args('--integers a'.split())
... except argparse.ArgumentError:
...     print('Capturando um argumentError')
...
Capturando um argumentError
```

Adicionado na versão 3.9.

suggest_on_error

Por padrão, quando um usuário passa uma escolha de argumento ou nome de subanalisador inválido, *ArgumentParser* será encerrado com informações de erro e listará as escolhas de argumento permitidas (se especificado) ou nomes de subanalisadores como parte da mensagem de erro.

Se o usuário desejar habilitar sugestões para escolhas de argumentos e nomes de subanalisadores com erros de digitação, o recurso pode ser habilitado definindo `suggest_on_error` como `True`. Observe que isso só se aplica a argumentos quando as opções especificadas são strings:

```
>>> parser = argparse.ArgumentParser(description='Process some integers.',
                                     suggest_on_error=True)
>>> parser.add_argument('--action', choices=['sum', 'max'])
>>> parser.add_argument('integers', metavar='N', type=int, nargs='+',
...                     help='an integer for the accumulator')
>>> parser.parse_args(['--action', 'sumn', 1, 2, 3])
tester.py: error: argument --action: invalid choice: 'sumn', maybe you meant 'sum'?
↪ (choose from 'sum', 'max')
```

Se você estiver escrevendo um código que precisa ser compatível com versões mais antigas do Python e quiser usar oportunisticamente `suggest_on_error` quando estiver disponível, você pode defini-lo como um atributo após inicializar o analisador sintático em vez de usar o argumento nomeado:

```
>>> parser = argparse.ArgumentParser(description='Process some integers.')
>>> parser.suggest_on_error = True
```

Adicionado na versão 3.14.

cor

Por padrão, a mensagem de ajuda é impressa em cores usando sequências de escape ANSI. Se desejar mensagens de ajuda em texto simples, você pode desabilitar esta no seu ambiente local ou no próprio analisador de argumentos, definindo `color` como `False`:

```
>>> parser = argparse.ArgumentParser(description='Process some integers.',
...                                  color=False)
>>> parser.add_argument('--action', choices=['sum', 'max'])
>>> parser.add_argument('integers', metavar='N', type=int, nargs='+',
...                     help='an integer for the accumulator')
>>> parser.parse_args(['--help'])
```

Adicionado na versão 3.14.

17.1.2 O método add_argument()

`ArgumentParser.`**`add_argument`**(*name or flags...*, *[, *action*][, *nargs*][, *const*][, *default*][, *type*][, *choices*][, *required*][, *help*][, *metavar*][, *dest*][, *deprecated*])

Define como um único argumento de linha de comando deve ser analisado. Cada parâmetro tem sua própria descrição mais detalhada abaixo, mas resumidamente são eles:

- *name ou flags* - Um nome ou uma lista de strings de opções, por exemplo. `'foo'` ou `'-f', '--foo'`.
- *action* - O tipo básico de ação a ser executada quando esse argumento é encontrado na linha de comando.
- *nargs* - O número de argumentos de linha de comando que devem ser consumidos.
- *const* - Um valor constante exigido por algumas seleções *action* e *nargs*.
- *default* - O valor produzido se o argumento estiver ausente da linha de comando e se estiver ausente do objeto espaço de nomes.
- *type* - O tipo para o qual o argumento de linha de comando deve ser convertido.

- *choices* - Uma sequência dos valores permitidos para o argumento.
- *required* - Se a opção de linha de comando pode ou não ser omitida (somente opcionais).
- *help* - Uma breve descrição do que o argumento faz.
- *metavar* - Um nome para o argumento nas mensagens de uso.
- *dest* - O nome do atributo a ser adicionado ao objeto retornado por `parse_args()`.
- *deprecated* - Se o uso do argumento foi descontinuado ou não.

As seções a seguir descrevem como cada um deles é usado.

name ou flags

O método `add_argument()` deve saber se um argumento opcional, como `-f` ou `--foo`, ou um argumento posicional, como uma lista de nomes de arquivos, é esperado. Os primeiros argumentos passados para `add_argument()` devem, portanto, ser uma série de sinalizadores ou um simples nome de argumento.

Por exemplo, um argumento opcional poderia ser criado como:

```
>>> parser.add_argument('-f', '--foo')
```

enquanto um argumento posicional pode ser criado como:

```
>>> parser.add_argument('bar')
```

Quando `parse_args()` é chamado, argumentos opcionais serão identificados pelo prefixo `-`, e os argumentos restantes serão considerados posicionais:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-f', '--foo')
>>> parser.add_argument('bar')
>>> parser.parse_args(['BAR'])
Namespace(bar='BAR', foo=None)
>>> parser.parse_args(['BAR', '--foo', 'FOO'])
Namespace(bar='BAR', foo='FOO')
>>> parser.parse_args(['--foo', 'FOO'])
usage: PROG [-h] [-f FOO] bar
PROG: error: the following arguments are required: bar
```

Por padrão, `argparse` lida automaticamente com a nomenclatura interna e os nomes de exibição dos argumentos, simplificando o processo sem exigir configuração adicional. Dessa forma, você não precisa especificar os parâmetros *dest* e *metavar*. O parâmetro *dest* assume por padrão o nome do argumento com sublinhados _ substituindo os hífens `-`. O parâmetro *metavar* assume por padrão o nome em letras maiúsculas. Por exemplo:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--foo-bar')
>>> parser.parse_args(['--foo-bar', 'FOO-BAR']
Namespace(foo_bar='FOO-BAR')
>>> parser.print_help()
usage:  [-h] [--foo-bar FOO-BAR]

optional arguments:
 -h, --help  show this help message and exit
 --foo-bar FOO-BAR
```

action

Objetos *ArgumentParser* associam argumentos de linha de comando com ações. Essas ações podem fazer praticamente qualquer coisa com os argumentos de linha de comando associados a elas, embora a maioria das ações simplesmente adicione um atributo ao objeto retornado por *parse_args()*. O argumento nomeado `action` especifica como os argumentos da linha de comando devem ser tratados. As ações fornecidas são:

- `'store'` - Isso apenas armazena o valor do argumento. Esta é a ação padrão.
- `'store_const'` - Isso armazena o valor especificado pelo argumento nomeado *const*; observe que o argumento nomeado *const* tem como padrão `None`. A ação `'store_const'` é mais comumente usada com argumentos opcionais que especificam algum tipo de sinalizador. Por exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='store_const', const=42)
>>> parser.parse_args(['--foo'])
Namespace(foo=42)
```

- `'store_true'` e `'store_false'` - Estes são casos especiais de `'store_const'` usados para armazenar os valores `True` e `False` respectivamente. Além disso, eles criam valores padrão de `False` e `True` respectivamente.

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='store_true')
>>> parser.add_argument('--bar', action='store_false')
>>> parser.add_argument('--baz', action='store_false')
>>> parser.parse_args('--foo --bar'.split())
Namespace(foo=True, bar=False, baz=True)
```

- `'append'` - Isso anexa cada valor de argumento a uma lista. É útil permitir para permitir que uma opção seja especificada várias vezes. Se o valor padrão for uma lista não-vazia, o valor analisado vai iniciar com os elementos padrão da lista e quaisquer valores da linha de comando serão anexados após os valores padrão. Exemplo de uso:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='append', default=['0'])
>>> parser.parse_args('--foo 1 --foo 2'.split())
Namespace(foo=['0', '1', '2'])
```

- `'append_const'` - Isso armazena uma lista e anexa o valor especificado pelo argumento nomeado *const* à lista; observe que o argumento nomeado *const* tem como padrão `None`. A ação `'append_const'` é normalmente útil quando vários argumentos precisam armazenar constantes na mesma lista. Por exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--str', dest='types', action='append_const',␣
↪const=str)
>>> parser.add_argument('--int', dest='types', action='append_const',␣
↪const=int)
>>> parser.parse_args('--str --int'.split())
Namespace(types=[<class 'str'>, <class 'int'>])
```

- `'extend'` - Isso armazena uma lista e anexa cada item da lista de argumentos multivalorada a ela. A ação `'extend'` é normalmente usada com o valor do argumento nomeado *nargs* `'+'` ou `'*'`. Observe que quando *nargs* é `None` (o padrão) ou `'?'`, cada caractere da string do argumento será anexado à lista. Exemplo de uso:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument("--foo", action="extend", nargs="+", type=str)
>>> parser.parse_args(["--foo", "f1", "--foo", "f2", "f3", "f4"])
Namespace(foo=['f1', 'f2', 'f3', 'f4'])
```

  Adicionado na versão 3.8.

- `'count'` - Isso conta o número de vezes que um argumento nomeado ocorre. Por exemplo, isso é útil para aumentar os níveis de verbosidade:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--verbose', '-v', action='count', default=0)
>>> parser.parse_args(['-vvv'])
Namespace(verbose=3)
```

  Observe que o *padrão* será `None`, a menos que seja explicitamente definido como *0*.

- `'help'` - Isso imprime uma mensagem de ajuda completa para todas as opções no analisador sintático atual e sai. Por padrão, uma ação de ajuda é adicionada automaticamente ao analisador sintático. Veja *ArgumentParser* para detalhes de como a saída é criada.

- `'version'` - Isso espera um argumento nomeado `version=` na chamada *add_argument()* e imprime informações de versão e sai quando invocado:

```
>>> import argparse
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--version', action='version', version='%(prog)s 2.0')
>>> parser.parse_args(['--version'])
PROG 2.0
```

Você também pode especificar uma ação arbitrária passando uma subclasse *Action* (por exemplo, *BooleanOptionalAction*) ou outro objeto que implemente a mesma interface. Somente ações que consomem argumentos de linha de comando (por exemplo, `'store'`, `'append'`, `'extend'` ou ações personalizadas com `nargs` diferentes de zero) podem ser usadas com argumentos posicionais.

A maneira recomendada de criar uma ação personalizada é estender *Action*, substituindo o método `__call__()` e opcionalmente os métodos `__init__()` e `format_usage()`. Você também pode registrar ações personalizadas usando o método *register()* e referenciá-las pelo nome registrado.

Um exemplo de uma ação personalizada:

```
>>> class FooAction(argparse.Action):
...     def __init__(self, option_strings, dest, nargs=None, **kwargs):
...         if nargs is not None:
...             raise ValueError("nargs not allowed")
...         super().__init__(option_strings, dest, **kwargs)
...     def __call__(self, parser, namespace, values, option_string=None):
...         print('%r %r %r' % (namespace, values, option_string))
...         setattr(namespace, self.dest, values)
...
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action=FooAction)
>>> parser.add_argument('bar', action=FooAction)
>>> args = parser.parse_args('1 --foo 2'.split())
Namespace(bar=None, foo=None) '1' None
Namespace(bar='1', foo=None) '2' '--foo'
>>> args
Namespace(bar='1', foo='2')
```

Para mais detalhes, veja *Action*.

nargs

Os objetos *ArgumentParser* geralmente associam um único argumento de linha de comando a uma única ação a ser executada. O argumento nomeado `nargs` associa um número diferente de argumentos de linha de comando com uma única ação. Veja também *Especificando argumentos ambíguos*. Os valores suportados são:

- `N` (um inteiro). Os argumentos `N` da linha de comando serão reunidos em uma lista. Por exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', nargs=2)
>>> parser.add_argument('bar', nargs=1)
>>> parser.parse_args('c --foo a b'.split())
Namespace(bar=['c'], foo=['a', 'b'])
```

Observe que `nargs=1` produz uma lista de um item. Isso é diferente do padrão, em que o item é produzido sozinho.

- `'?'`. Um argumento será consumido da linha de comando, se possível, e produzido como um único item. Se nenhum argumento de linha de comando estiver presente, o valor de *default* será produzido. Observe que, para argumentos opcionais, há um caso adicional - a string de opções está presente, mas não é seguida por um argumento de linha de comando. Neste caso o valor de *const* será produzido. Alguns exemplos para ilustrar isso:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', nargs='?', const='c', default='d')
>>> parser.add_argument('bar', nargs='?', default='d')
>>> parser.parse_args(['XX', '--foo', 'YY'])
Namespace(bar='XX', foo='YY')
>>> parser.parse_args(['XX', '--foo'])
Namespace(bar='XX', foo='c')
>>> parser.parse_args([])
Namespace(bar='d', foo='d')
```

Um dos usos mais comuns de `nargs='?'` é permitir arquivos de entrada e saída opcionais:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('infile', nargs='?')
>>> parser.add_argument('outfile', nargs='?')
>>> parser.parse_args(['input.txt', 'output.txt'])
Namespace(infile='input.txt', outfile='output.txt')
>>> parser.parse_args(['input.txt'])
Namespace(infile='input.txt', outfile=None)
>>> parser.parse_args([])
Namespace(infile=None, outfile=None)
```

- `'*'`. Todos os argumentos de linha de comando presentes são reunidos em uma lista. Note que geralmente não faz muito sentido ter mais de um argumento posicional com `nargs='*'`, mas vários argumentos opcionais com `nargs='*'` são possíveis. Por exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', nargs='*')
>>> parser.add_argument('--bar', nargs='*')
>>> parser.add_argument('baz', nargs='*')
>>> parser.parse_args('a b --foo x y --bar 1 2'.split())
Namespace(bar=['1', '2'], baz=['a', 'b'], foo=['x', 'y'])
```

- `'+'`. Assim como `'*'`, todos os argumentos de linha de comando presentes são reunidos em uma lista. Além disso, uma mensagem de erro será gerada se não houver pelo menos um argumento de linha de comando presente. Por exemplo:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('foo', nargs='+')
>>> parser.parse_args(['a', 'b'])
Namespace(foo=['a', 'b'])
>>> parser.parse_args([])
usage: PROG [-h] foo [foo ...]
```

(continua na próxima página)

(continuação da página anterior)

```
PROG: error: the following arguments are required: foo
```

Se o argumento nomeado `nargs` não for fornecido, o número de argumentos consumidos é determinado por *action*. Geralmente, isso significa que um único argumento de linha de comando será usado e um único item (não uma lista) será produzido. Ações que não fazem uso de argumentos da linha de comando (por exemplo, `'store_const'`) definem `nargs=0`.

const

O argumento `const` de *`add_argument()`* é usado para manter valores constantes que não são lidos da linha de comando, mas são necessários para as várias ações *`ArgumentParser`*. Os dois usos mais comuns são:

- Quando *`add_argument()`* é chamado com `action='store_const'` ou `action='append_const'`. Essas ações adicionam o valor `const` a um dos atributos do objeto retornado por *`parse_args()`*. Consulte a descrição da *action* para obter exemplos. Se `const` não for fornecido *`add_argument()`*, será recebido um valor padrão de `None`.
- Quando *`add_argument()`* é chamado com strings de opções (como `-f` ou `--foo`) e `nargs='?'`. Isso cria um argumento opcional que pode ser seguido por zero ou um argumento de linha de comando. Ao analisar a linha de comando, se a string de opções for encontrada sem nenhum argumento de linha de comando seguindo, o valor de `const` será usado. Veja a descrição de *nargs* para exemplos.

Alterado na versão 3.11: `const=None` por padrão, incluindo quando `action='append_const'` ou `action='store_const'`.

default

Todos os argumentos opcionais e alguns argumentos posicionais podem ser omitidos na linha de comando. O argumento nomeado `default` de *`add_argument()`*, cujo valor padrão é `None`, especifica qual valor deve ser usado se o argumento de linha de comando não estiver presente. Para argumentos opcionais, o valor `default` é usado quando a string de opção não estava presente na linha de comando:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=42)
>>> parser.parse_args(['--foo', '2'])
Namespace(foo='2')
>>> parser.parse_args([])
Namespace(foo=42)
```

Se o espaço de nomes de destino já tiver um atributo definido, a ação *default* não o substituirá:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=42)
>>> parser.parse_args([], namespace=argparse.Namespace(foo=101))
Namespace(foo=101)
```

Se o valor `default` for uma string, o analisador analisa o valor como se fosse um argumento de linha de comando. Em particular, o analisador aplica qualquer argumento de conversão *type*, se fornecido, antes de definir o atributo no valor de retorno *`Namespace`*. Caso contrário, o analisador usa o valor como está:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--length', default='10', type=int)
>>> parser.add_argument('--width', default=10.5, type=int)
>>> parser.parse_args()
Namespace(length=10, width=10.5)
```

Para argumentos posicionais com *nargs* igual a `?` ou `*`, o valor `default` é usado quando nenhum argumento de linha de comando estava presente:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('foo', nargs='?', default=42)
>>> parser.parse_args(['a'])
Namespace(foo='a')
>>> parser.parse_args([])
Namespace(foo=42)
```

Para argumentos obrigatórios, isto é, marcados com *required*, o valor `default` é ignorado. Por exemplo, isso se aplica a argumentos posicionais com valores de *nargs* diferentes de `?` ou `*`, ou argumentos opcionais marcados como `required=True`.

Fornecer `default=argparse.SUPPRESS` faz com que nenhum atributo seja adicionado se o argumento da linha de comando não estiver presente:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=argparse.SUPPRESS)
>>> parser.parse_args([])
Namespace()
>>> parser.parse_args(['--foo', '1'])
Namespace(foo='1')
```

type

Por padrão, o analisador sintático lê argumentos de linha de comando como strings simples. No entanto, muitas vezes a string da linha de comando deve ser interpretada como outro tipo, como *`float`* ou *`int`*. O argumento nomeado `type` para *`add_argument()`* permite que qualquer verificação de tipo e conversões de tipo necessárias sejam realizadas.

Se o argumento nomeado *type* for usado com *default*, o conversor de tipo só será aplicado se o padrão for uma string.

O argumento para `type` pode ser um chamável que aceite uma única string ou o nome de um tipo registrado (veja *`register()`*). Se a função levantar *`ArgumentTypeError`*, *`TypeError`* ou *`ValueError`*, a exceção será capturada e uma mensagem de erro bem formatada será exibida. Nenhum outro tipo de exceção é tratado.

Tipos e funções embutidas comuns podem ser usados como conversores de tipo:

```
import argparse
import pathlib

parser = argparse.ArgumentParser()
parser.add_argument('count', type=int)
parser.add_argument('distance', type=float)
parser.add_argument('street', type=ascii)
parser.add_argument('code_point', type=ord)
parser.add_argument('datapath', type=pathlib.Path)
```

Funções definidas pelo usuário também podem ser usadas:

```
>>> def hyphenated(string):
...     return '-'.join([word[:4] for word in string.casefold().split()])
...
>>> parser = argparse.ArgumentParser()
>>> _ = parser.add_argument('short_title', type=hyphenated)
>>> parser.parse_args(['"Um conto de duas cidades"'])
Namespace(short_title='"um-cont-de-duas-cida')
```

A função *`bool()`* não é recomendada como conversor de tipo. Tudo o que ele faz é converter strings vazias em `False` e strings não vazias em `True`. Geralmente não é isso que se deseja.

Em geral, o argumento nomeado `type` é uma conveniência que só deve ser usada para conversões simples que

só podem gerar uma das três exceções suportadas. Qualquer coisa com tratamento de erros ou gerenciamento de recursos mais interessante deve ser feita posteriormente, após a análise dos argumentos.

Por exemplo, conversões JSON ou YAML têm casos de erros complexos que exigem relatórios melhores do que os fornecidos pelo argumento nomeado `type`. Um *`JSONDecodeError`* não seria bem formatado e uma exceção *`FileNotFoundError`* não seria tratada.

Mesmo *`FileType`* tem suas limitações para uso com o argumento nomeado `type`. Se um argumento usar *`FileType`* e um argumento subsequente falhar, um erro será relatado, mas o arquivo não será fechado automaticamente. Neste caso, seria melhor esperar até que o analisador tenha sido executado e então usar a instrução `with` para gerenciar os arquivos.

Para verificadores de tipo que simplesmente verificam um conjunto fixo de valores, considere usar o argumento nomeado *choices*.

choices

Alguns argumentos de linha de comando devem ser selecionados em um conjunto restrito de valores. Eles podem ser tratados passando um objeto sequência como o argumento nomeado *choices* para *`add_argument()`*. Quando a linha de comando for analisada, os valores dos argumentos serão verificados e uma mensagem de erro será exibida se o argumento não for um dos valores aceitáveis:

```
>>> parser = argparse.ArgumentParser(prog='game.py')
>>> parser.add_argument('move', choices=['pedra', 'papel', 'tesoura'])
>>> parser.parse_args(['padra'])
Namespace(move='padr')
>>> parser.parse_args(['fogo'])
usage: game.py [-h] {pedra,papel,tesoura}
game.py: error: argument move: invalid choice: 'fogo' (choose from 'pedra',
'papel', 'tesoura')
```

Qualquer sequência pode ser passada como o valor *choices*, portanto objetos *`list`*, objetos *`tuple`* e sequências personalizadas são todos suportados.

O uso de *`enum.Enum`* não é recomendado porque é difícil controlar sua aparência no uso, na ajuda e nas mensagens de erro.

Observe que *choices* são verificadas após a realização de quaisquer conversões de *type*, portanto, os objetos em *choices* devem corresponder ao *type* especificado. Isso pode fazer com que *choices* pareçam estranhos no uso, na ajuda ou em mensagens de erro.

Para manter *choices* fácel de usar, considere um invólucro de tipo personalizado que converta e formate valores, ou omita *type* e lide com a conversão no código da sua aplicação.

As opções formatadas substituem o *metavar* padrão que normalmente é derivado de *dest*. Geralmente é isso que você deseja porque o usuário nunca vê o parâmetro *dest*. Se esta exibição não for desejável (talvez porque haja muitas opções), basta especificar um *metavar* explícito.

required

Em geral, o módulo `argparse` presume que sinalizadores como `-f` e `--bar` indicam argumentos *opcionais*, que sempre podem ser omitidos na linha de comando. Para tornar uma opção obrigatória, `True` pode ser especificado para o argumento nomeado `required=` para *`add_argument()`*:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', required=True)
>>> parser.parse_args(['--foo', 'BAR'])
Namespace(foo='BAR')
>>> parser.parse_args([])
usage: [-h] --foo FOO
: error: the following arguments are required: --foo
```

Como mostra o exemplo, se uma opção estiver marcada como `required`, *`parse_args()`* reportará um erro se essa opção não estiver presente na linha de comando.

> **Nota**
>
> As opções obrigatórias são geralmente consideradas inadequadas porque os usuários esperam que as *opções* sejam *opcionais* e, portanto, devem ser evitadas quando possível.

help

O valor `help` é uma string contendo uma breve descrição do argumento. Quando um usuário solicita ajuda (geralmente usando `-h` ou `--help` na linha de comando), estas descrições de `help` serão exibidas com cada argumento.

As strings `help` podem incluir vários especificadores de formato para evitar a repetição de coisas como o nome do programa ou o argumento *default*. Os especificadores disponíveis incluem o nome do programa, `%(prog)s` e a maioria dos argumentos nomeados para *`add_argument()`*, por exemplo. `%(default)s`, `%(type)s`, etc.:

```
>>> parser = argparse.ArgumentParser(prog='frobble')
>>> parser.add_argument('bar', nargs='?', type=int, default=42,
...                     help='the bar to %(prog)s (default: %(default)s)')
>>> parser.print_help()
usage: frobble [-h] [bar]

positional arguments:
 bar     the bar to frobble (default: 42)

options:
 -h, --help  show this help message and exit
```

Como a string de ajuda oferece suporte à formatação com %, se você quiser que um literal `%` apareça na string de ajuda, você deve escapá-lo como `%%`.

`argparse` oferece suporte a silenciar a entrada de ajuda para certas opções, definindo o valor `help` como `argparse.SUPPRESS`:

```
>>> parser = argparse.ArgumentParser(prog='frobble')
>>> parser.add_argument('--foo', help=argparse.SUPPRESS)
>>> parser.print_help()
usage: frobble [-h]

options:
  -h, --help  show this help message and exit
```

metavar

Quando *`ArgumentParser`* gera mensagens de ajuda, ele precisa de alguma forma de se referir a cada argumento esperado. Por padrão, os objetos `ArgumentParser` usam o valor *dest* como o "nome" de cada objeto. Por padrão, para ações de argumentos posicionais, o valor *dest* é usado diretamente, e para ações de argumentos opcionais, o valor *dest* é maiúsculo. Portanto, um único argumento posicional com `dest='bar'` será referido como `bar`. Um único argumento opcional `--foo` que deve ser seguido por um único argumento de linha de comando será referido como `FOO`. Um exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> parser.add_argument('bar')
>>> parser.parse_args('X --foo Y'.split())
Namespace(bar='X', foo='Y')
>>> parser.print_help()
```

(continua na próxima página)

(continuação da página anterior)

```
usage:  [-h] [--foo FOO] bar

positional arguments:
 bar

options:
 -h, --help  show this help message and exit
 --foo FOO
```

Um nome alternativo pode ser especificado com `metavar`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', metavar='YYY')
>>> parser.add_argument('bar', metavar='XXX')
>>> parser.parse_args('X --foo Y'.split())
Namespace(bar='X', foo='Y')
>>> parser.print_help()
usage:  [-h] [--foo YYY] XXX

positional arguments:
 XXX

options:
 -h, --help  show this help message and exit
 --foo YYY
```

Observe que `metavar` apenas altera o nome *exibido* - o nome do atributo no objeto *parse_args()* ainda é determinado pelo valor *dest*.

Valores diferentes de `nargs` podem fazer com que o metavar seja usado múltiplas vezes. Fornecer uma tupla para `metavar` especifica uma exibição diferente para cada um dos argumentos:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x', nargs=2)
>>> parser.add_argument('--foo', nargs=2, metavar=('bar', 'baz'))
>>> parser.print_help()
usage: PROG [-h] [-x X X] [--foo bar baz]

options:
 -h, --help     show this help message and exit
 -x X X
 --foo bar baz
```

dest

A maioria das ações *ArgumentParser* adiciona algum valor como um atributo do objeto retornado por *parse_args()*. O nome deste atributo é determinado pelo argumento nomeado `dest` de *add_argument()*. Para ações de argumento posicional, `dest` é normalmente fornecido como o primeiro argumento para *add_argument()*:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('bar')
>>> parser.parse_args(['XXX'])
Namespace(bar='XXX')
```

Para ações de argumentos opcionais, o valor de `dest` é normalmente inferido das strings de opções. *ArgumentParser* gera o valor de `dest` pegando a primeira string de opção longa e removendo a string inicial `--`. Se nenhuma string de opção longa for fornecida, `dest` será derivado da primeira string de opção curta removendo o caractere `-` inicial. Quaisquer caracteres `-` internos serão convertidos em caracteres `_` para garantir que a

string seja um nome de atributo válido. Os exemplos abaixo ilustram esse comportamento:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('-f', '--foo-bar', '--foo')
>>> parser.add_argument('-x', '-y')
>>> parser.parse_args('-f 1 -x 2'.split())
Namespace(foo_bar='1', x='2')
>>> parser.parse_args('--foo 1 -y 2'.split())
Namespace(foo_bar='1', x='2')
```

`dest` permite que um nome de atributo personalizado seja fornecido:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', dest='bar')
>>> parser.parse_args('--foo XXX'.split())
Namespace(bar='XXX')
```

deprecated

Durante a vida de um projeto, alguns argumentos podem precisar ser removidos da linha de comando. Antes de removê-los, você deve informar aos seus usuários que os argumentos estão descontinuados e serão removidos. O argumento nomeado `deprecated` de *`add_argument()`*, cujo padrão é `False`, especifica se o argumento está descontinuado e será removido no futuro. Para argumentos, se `deprecated` for `True`, então um aviso será enviado para *`sys.stderr`* (saída de erro) quando o argumento for usado:

```
>>> import argparse
>>> parser = argparse.ArgumentParser(prog='snake.py')
>>> parser.add_argument('--legs', default=0, type=int, deprecated=True)
>>> parser.parse_args([])
Namespace(legs=0)
>>> parser.parse_args(['--legs', '4'])
snake.py: warning: option '--legs' is deprecated
Namespace(legs=4)
```

Adicionado na versão 3.13.

Classes de ação

As classes `Action` implementam a API de Action, um chamável que retorna um chamável que processa argumentos da linha de comando. Qualquer objeto que siga esta API pode ser passado como parâmetro `action` para *`add_argument()`*.

class argparse.**Action**(*option_strings*, *dest*, *nargs=None*, *const=None*, *default=None*, *type=None*, *choices=None*, *required=False*, *help=None*, *metavar=None*)

Objetos `Action` são usados por um *`ArgumentParser`* para representar as informações necessárias para analisar um único argumento de uma ou mais strings da linha de comando. A classe `Action` deve aceitar os dois argumentos posicionais mais quaisquer argumentos nomeados passados para *`ArgumentParser.add_argument()`*, exceto o próprio `action`.

Instâncias de `Action` (ou valor de retorno de qualquer chamável para o parâmetro `action`) devem ter atributos `dest`, `option_strings`, `default`, `type`, `required`, `help`, etc. definidos. A maneira mais fácil de garantir que esses atributos sejam definidos é chamar `Action.__init__()`.

__call__(*parser*, *namespace*, *values*, *option_string=None*)

As instâncias de `Action` devem ser chamáveis, portanto, as subclasses devem substituir o método `__call__()`, que deve aceitar quatro parâmetros:

- *parser* - O objeto *`ArgumentParser`* que contém esta ação.
- *namespace* - O objeto *`Namespace`* que será retornado por *`parse_args()`*. A maioria das ações adicionam um atributo a este objeto usando *`setattr()`*.

- *values* - Os argumentos de linha de comando associados, com qualquer conversões de tipo aplicadas. Conversões de tipo são especificadas com o argumento nomeado *type* para `add_argument()`.
- *option_string* - A string da opção que foi usada para invocar esta ação. O argumento `option_string` é opcional e estará ausente se a ação estiver associada a um argumento posicional.

O método `__call__()` pode executar ações arbitrárias, mas normalmente definirá atributos em `namespace` com base em `dest` e `values`.

format_usage()

As subclasses de `Action` podem definir um método `format_usage()` que não recebe argumento e retorna uma string que será usada ao exibir a mensagem de uso do programa. Se tal método não for fornecido, um padrão sensato será usado.

class argparse.**BooleanOptionalAction**

Uma subclasse de `Action` para manipular sinalizadores booleanos com opções positivas e negativas. Adicionar um único argumento, como `--foo`, cria automaticamente as opções `--foo` e `--no-foo`, armazenando `True` e `False` respectivamente:

```
>>> import argparse
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action=argparse.BooleanOptionalAction)
>>> parser.parse_args(['--no-foo'])
Namespace(foo=False)
```

Adicionado na versão 3.9.

17.1.3 O método parse_args()

ArgumentParser.**parse_args**(*args=None*, *namespace=None*)

Converte strings de argumento em objetos e os atribui como atributos do espaço de nomes. Retorna o espaço de nomes preenchido.

Chamadas anteriores para `add_argument()` determinam exatamente quais objetos são criados e como eles são atribuídos. Veja a documentação de `add_argument()` para detalhes.

- *args* - Lista de strings para analisar. O padrão é obtido de `sys.argv`.
- *namespace* - Um objeto para receber os atributos. O padrão é um novo objeto `Namespace` vazio.

Sintaxe de valores da opção

O método `parse_args()` provê várias maneiras de especificar o valor de uma opção (se ele pegar uma). No caso mais simples, a opção e seu valor são passados como dois argumentos separados:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x')
>>> parser.add_argument('--foo')
>>> parser.parse_args(['-x', 'X'])
Namespace(foo=None, x='X')
>>> parser.parse_args(['--foo', 'FOO'])
Namespace(foo='FOO', x=None)
```

Para opções longas (opções com nomes maiores que um único caractere), a opção e o valor também podem ser passados como um único argumento de linha de comando, usando = para separá-los:

```
>>> parser.parse_args(['--foo=FOO'])
Namespace(foo='FOO', x=None)
```

Para opções curtas (opções com apenas um caractere), a opção e seu valor podem ser concatenados:

```
>>> parser.parse_args(['-xX'])
Namespace(foo=None, x='X')
```

Várias opções curtas podem ser unidas, usando apenas um único prefixo `-`, desde que apenas a última opção (ou nenhuma delas) exija um valor:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x', action='store_true')
>>> parser.add_argument('-y', action='store_true')
>>> parser.add_argument('-z')
>>> parser.parse_args(['-xyzZ'])
Namespace(x=True, y=True, z='Z')
```

Argumentos inválidos

Ao analisar a linha de comando, *parse_args()* verifica uma variedade de erros, incluindo opções ambíguas, tipos inválidos, opções inválidas, número incorreto de argumentos posicionais, etc. Quando encontra tal erro, ele sai e imprime o erro junto com uma mensagem de uso:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--foo', type=int)
>>> parser.add_argument('bar', nargs='?')

>>> # tipo inválido
>>> parser.parse_args(['--foo', 'spam'])
usage: PROG [-h] [--foo FOO] [bar]
PROG: error: argument --foo: invalid int value: 'spam'

>>> # opção inválida
>>> parser.parse_args(['--bar'])
usage: PROG [-h] [--foo FOO] [bar]
PROG: error: no such option: --bar

>>> # número errado de argumentos
>>> parser.parse_args(['spam', 'badger'])
usage: PROG [-h] [--foo FOO] [bar]
PROG: error: extra arguments found: badger
```

Argumentos contendo -

O método *parse_args()* tenta mostrar erros sempre que o usuário claramente cometeu um erro, mas algumas situações são inerentemente ambíguas. Por exemplo, o argumento de linha de comando `-1` pode ser uma tentativa de especificar uma opção ou uma tentativa de fornecer um argumento posicional. O método *parse_args()* é cauteloso aqui: argumentos posicionais só podem começar com `-` se eles se parecerem com números negativos e não houver opções no analisador sintático que se pareçam com números negativos:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x')
>>> parser.add_argument('foo', nargs='?')

>>> # nenhuma opção de número negativo, então -1 é um argumento posicional
>>> parser.parse_args(['-x', '-1'])
Namespace(foo=None, x='-1')

>>> # nenhuma opção de número negativo, então -1 e -5 são argumentos posicionais
>>> parser.parse_args(['-x', '-1', '-5'])
Namespace(foo='-5', x='-1')
```

(continua na próxima página)

(continuação da página anterior)

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-1', dest='one')
>>> parser.add_argument('foo', nargs='?')

>>> # opção de número negativo presente, então -1 é uma opção
>>> parser.parse_args(['-1', 'X'])
Namespace(foo=None, one='X')

>>> # opção de número negativo presente, então -2 é uma opção
>>> parser.parse_args(['-2'])
usage: PROG [-h] [-1 ONE] [foo]
PROG: error: no such option: -2

>>> # opções de número negativo presentes, então ambos -1s são opções
>>> parser.parse_args(['-1', '-1'])
usage: PROG [-h] [-1 ONE] [foo]
PROG: error: argument -1: expected one argument
```

Se você tiver argumentos posicionais que devem começar com - e não se parecem com números negativos, você pode inserir o pseudoargumento `'--'` que informa *`parse_args()`* que tudo depois disso é um argumento posicional:

```
>>> parser.parse_args(['--', '-f'])
Namespace(foo='-f', one=None)
```

Veja também *o tutorial do argparse sobre argumentos ambíguos* para mais detalhes.

Abreviações de argumento (correspondência de prefixo)

O método *`parse_args()` por padrão* permite que opções longas sejam abreviadas para um prefixo, se a abreviação não for ambígua (o prefixo corresponde a uma opção única):

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-bacon')
>>> parser.add_argument('-badger')
>>> parser.parse_args('-bac MMM'.split())
Namespace(bacon='MMM', badger=None)
>>> parser.parse_args('-bad WOOD'.split())
Namespace(bacon=None, badger='WOOD')
>>> parser.parse_args('-ba BA'.split())
usage: PROG [-h] [-bacon BACON] [-badger BADGER]
PROG: error: ambiguous option: -ba could match -badger, -bacon
```

Um erro é produzido para argumentos que podem produzir mais de uma opção. Este recurso pode ser desabilitado definindo *allow_abbrev* como `False`.

Além do `sys.argv`

Às vezes, pode ser útil ter uma instância de *`ArgumentParser`* analisando argumentos diferentes daqueles de *`sys.argv`*. Isso pode ser feito passando uma lista de strings para *`parse_args()`*. Isso é útil para testar no prompt interativo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument(
...     'integers', metavar='int', type=int, choices=range(10),
...     nargs='+', help='an integer in the range 0..9')
>>> parser.add_argument(
```

(continua na próxima página)

(continuação da página anterior)

```
...     '--sum', dest='accumulate', action='store_const', const=sum,
...     default=max, help='sum the integers (default: find the max)')
>>> parser.parse_args(['1', '2', '3', '4'])
Namespace(accumulate=<built-in function max>, integers=[1, 2, 3, 4])
>>> parser.parse_args(['1', '2', '3', '4', '--sum'])
Namespace(accumulate=<built-in function sum>, integers=[1, 2, 3, 4])
```

O objeto Namespace

class argparse.**Namespace**

Classe simples usada por padrão por *parse_args()* para criar um objeto contendo atributos e retorná-lo.

Esta classe é deliberadamente simples, apenas uma subclasse *object* com uma representação de string legível. Se você preferir ter uma visão dos atributos do tipo dict, você pode usar o idioma padrão do Python, *vars()*:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> args = parser.parse_args(['--foo', 'BAR'])
>>> vars(args)
{'foo': 'BAR'}
```

Também pode ser útil ter um *ArgumentParser* atribuindo atributos a um objeto já existente, em vez de um novo objeto *Namespace*. Isso pode ser obtido especificando o argumento nomeado `namespace=`:

```
>>> class C:
...     pass
...
>>> c = C()
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> parser.parse_args(args=['--foo', 'BAR'], namespace=c)
>>> c.foo
'BAR'
```

17.1.4 Outros utilitários

Subcomandos

ArgumentParser.**add_subparsers**(*[, *title*][, *description*][, *prog*][, *parser_class*][, *action*][, *dest*][, *required*][, *help*][, *metavar*])

Muitos programas dividem sua funcionalidade em vários subcomandos, por exemplo, o programa `svn` pode invocar subcomandos como `svn checkout`, `svn update` e `svn commit`. Dividir a funcionalidade dessa forma pode ser uma ideia particularmente boa quando um programa executa várias funções diferentes que exigem diferentes tipos de argumentos de linha de comando. *ArgumentParser* oferece suporte à criação de tais subcomandos com o método `add_subparsers()`. O método `add_subparsers()` é normalmente chamado sem argumentos e retorna um objeto de ação especial. Este objeto tem um único método, `add_parser()`, que recebe um nome de comando e quaisquer argumentos do construtor `ArgumentParser` e retorna um objeto `ArgumentParser` que pode ser modificado normalmente.

Descrição de parâmetros:

- *title* - título para o grupo de subanalisadores na saída de ajuda; por padrão "subcomandos" se a descrição for fornecida, caso contrário, usa o título para argumentos posicionais
- *description* - descrição para o grupo de subanalisadores na saída de ajuda, por padrão `None`
- *prog* - informações de uso que serão exibidas com a ajuda do subcomando, por padrão o nome do programa e quaisquer argumentos posicionais antes do argumento do subanalisador

- *parser_class* - classe que será usada para criar instâncias de subanalisadores, por padrão a classe do analisador atual (por exemplo, `ArgumentParser`)
- *action* - o tipo básico de ação a ser executada quando esse argumento é encontrado na linha de comando
- *dest* - nome do atributo sob o qual o nome do subcomando será armazenado; por padrão `None` e nenhum valor é armazenado
- *required* - Se um subcomando deve ou não ser fornecido, por padrão `False` (adicionado em 3.7)
- *help* - ajuda para o grupo de subanalisadores na saída de ajuda, por padrão `None`
- *metavar* - string que apresenta os subcomandos disponíveis na ajuda; por padrão é `None` e apresenta os subcomandos no formato {cmd1, cmd2, ..}

Alguns exemplos de uso:

```
>>> # cria o analisador de nível superior
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--foo', action='store_true', help='foo help')
>>> subparsers = parser.add_subparsers(help='subcommand help')
>>>
>>> # cria o analisador para o comando "a"
>>> parser_a = subparsers.add_parser('a', help='a help')
>>> parser_a.add_argument('bar', type=int, help='bar help')
>>>
>>> # cria o analisador para o comando "b"
>>> parser_b = subparsers.add_parser('b', help='b help')
>>> parser_b.add_argument('--baz', choices=('X', 'Y', 'Z'), help='baz help')
>>>
>>> # analisa algumas listas de argumentos
>>> parser.parse_args(['a', '12'])
Namespace(bar=12, foo=False)
>>> parser.parse_args(['--foo', 'b', '--baz', 'Z'])
Namespace(baz='Z', foo=True)
```

Note que o objeto retornado por `parse_args()` conterá apenas atributos para o analisador principal e o subanalisador que foi selecionado pela linha de comando (e não quaisquer outros subanalisadores). Então, no exemplo acima, quando o comando `a` é especificado, apenas os atributos `foo` e `bar` estão presentes, e quando o comando `b` é especificado, apenas os atributos `foo` e `baz` estão presentes.

Similarmente, quando uma mensagem de ajuda é solicitada de um subanalisador, somente a ajuda para aquele parser em particular será impressa. A mensagem de ajuda não incluirá mensagens do analisador sintático pai ou do analisador sintático irmão. (Uma mensagem de ajuda para cada comando do subanalisador, entretanto, pode ser dada fornecendo o argumento `help=` para `add_parser()` como acima.)

```
>>> parser.parse_args(['--help'])
usage: PROG [-h] [--foo] {a,b} ...

positional arguments:
  {a,b}   subcommand help
    a     a help
    b     b help

options:
  -h, --help  show this help message and exit
  --foo   foo help

>>> parser.parse_args(['a', '--help'])
usage: PROG a [-h] bar
```

(continua na próxima página)

(continuação da página anterior)

```
positional arguments:
  bar     bar help

options:
  -h, --help  show this help message and exit

>>> parser.parse_args(['b', '--help'])
usage: PROG b [-h] [--baz {X,Y,Z}]

options:
  -h, --help     show this help message and exit
  --baz {X,Y,Z}  baz help
```

O método *add_subparsers()* também oferece suporte aos argumentos nomeados `title` e `description`. Quando qualquer um deles estiver presente, os comandos do subanalisador aparecerão em seu próprio grupo na saída de ajuda. Por exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> subparsers = parser.add_subparsers(title='subcommands',
...                                    description='valid subcommands',
...                                    help='additional help')
>>> subparsers.add_parser('foo')
>>> subparsers.add_parser('bar')
>>> parser.parse_args(['-h'])
usage:  [-h] {foo,bar} ...

options:
  -h, --help  show this help message and exit

subcommands:
  valid subcommands

  {foo,bar}   additional help
```

Além disso, `add_parser()` oferece suporte a um argumento *aliases* adicional, que permite que múltiplas strings se refiram ao mesmo subanalisador. Este exemplo, como `svn`, alias `co` como uma abreviação para `checkout`:

```
>>> parser = argparse.ArgumentParser()
>>> subparsers = parser.add_subparsers()
>>> checkout = subparsers.add_parser('checkout', aliases=['co'])
>>> checkout.add_argument('foo')
>>> parser.parse_args(['co', 'bar'])
Namespace(foo='bar')
```

`add_parser()` oferece suporte também um argumento adicional *deprecated*, que permite descontinuar o subanalisador.

```
>>> import argparse
>>> parser = argparse.ArgumentParser(prog='chicken.py')
>>> subparsers = parser.add_subparsers()
>>> run = subparsers.add_parser('run')
>>> fly = subparsers.add_parser('fly', deprecated=True)
>>> parser.parse_args(['fly'])
chicken.py: warning: command 'fly' is deprecated
Namespace()
```

Adicionado na versão 3.13.

Uma maneira particularmente eficaz de lidar com subcomandos é combinar o uso do método *add_subparsers()* com chamadas para *set_defaults()* para que cada subanalisador saiba qual função Python ele deve executar. Por exemplo:

```
>>> # funções do subanalisador functions
>>> def foo(args):
...     print(args.x * args.y)
...
>>> def bar(args):
...     print('((%s))' % args.z)
...
>>> # cria o analisador de nível superior
>>> parser = argparse.ArgumentParser()
>>> subparsers = parser.add_subparsers(required=True)
>>>
>>> # cria o analisador do comando "foo"
>>> parser_foo = subparsers.add_parser('foo')
>>> parser_foo.add_argument('-x', type=int, default=1)
>>> parser_foo.add_argument('y', type=float)
>>> parser_foo.set_defaults(func=foo)
>>>
>>> # cria o analisador do comando "foo"
>>> parser_bar = subparsers.add_parser('bar')
>>> parser_bar.add_argument('z')
>>> parser_bar.set_defaults(func=bar)
>>>
>>> # analisa os argumentos e chama a função que for selecionada
>>> args = parser.parse_args('foo 1 -x 2'.split())
>>> args.func(args)
2.0
>>>
>>> # analisa os argumentos e chama a função que foi selecionada
>>> args = parser.parse_args('bar XYZYX'.split())
>>> args.func(args)
((XYZYX))
```

Dessa forma, você pode deixar *parse_args()* fazer o trabalho de chamar a função apropriada após a análise sintática do argumento ser concluída. Associar funções com ações como essa é normalmente a maneira mais fácil de lidar com as diferentes ações para cada um dos seus subanalisadores. No entanto, se for necessário verificar o nome do subanalisador que foi invocado, o argumento nomeado `dest` para a chamada *add_subparsers()* funcionará:

```
>>> parser = argparse.ArgumentParser()
>>> subparsers = parser.add_subparsers(dest='subparser_name')
>>> subparser1 = subparsers.add_parser('1')
>>> subparser1.add_argument('-x')
>>> subparser2 = subparsers.add_parser('2')
>>> subparser2.add_argument('y')
>>> parser.parse_args(['2', 'frobble'])
Namespace(subparser_name='2', y='frobble')
```

Alterado na versão 3.7: Novo parâmetro somente-nomeado *required*.

Alterado na versão 3.14: O *prog* do subanalisador não é mais afetado por uma mensagem de uso personalizada no analisador principal.

Objetos FileType

class argparse.**FileType**(*mode='r'*, *bufsize=-1*, *encoding=None*, *errors=None*)

A fábrica *FileType* cria objetos que podem ser passados para o argumento de tipo de *ArgumentParser.add_argument()*. Argumentos que têm objetos *FileType* como seu tipo abrirão argumentos de linha de comando como arquivos com os modos solicitados, tamanhos de buffer, codificações e tratamento de erros (veja a função *open()* para mais detalhes):

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--raw', type=argparse.FileType('wb', 0))
>>> parser.add_argument('out', type=argparse.FileType('w', encoding='UTF-8'))
>>> parser.parse_args(['--raw', 'raw.dat', 'file.txt'])
Namespace(out=<_io.TextIOWrapper name='file.txt' mode='w' encoding='UTF-8'>,␣
↪raw=<_io.FileIO name='raw.dat' mode='wb'>)
```

Objetos FileType entendem o pseudoargumento `'-'` e o convertem automaticamente em *sys.stdin* para objetos *FileType* legíveis e *sys.stdout* para objetos *FileType* graváveis:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('infile', type=argparse.FileType('r'))
>>> parser.parse_args(['-'])
Namespace(infile=<_io.TextIOWrapper name='<stdin>' encoding='UTF-8'>)
```

Nota

Se um argumento usar *FileType* e um argumento subsequente falhar, um erro será relatado, mas o arquivo não será fechado automaticamente. Isso também pode danificar os arquivos de saída. Neste caso, seria melhor esperar até que o analisador sintático tenha sido executado e então usar a instrução `with` para gerenciar os arquivos.

Alterado na versão 3.4: Adicionados os parâmetros *encodings* e *errors*.

Descontinuado desde a versão 3.14.

Grupos de argumentos

ArgumentParser.**add_argument_group**(*title=None*, *description=None*, ***[, *argument_default*][, *conflict_handler*])

Por padrão, *ArgumentParser* agrupa argumentos de linha de comando em "argumentos posicionais" e "opções" ao exibir mensagens de ajuda. Quando há um melhor agrupamento conceitual de argumentos do que este padrão, grupos apropriados podem ser criados usando o método `add_argument_group()`:

```
>>> parser = argparse.ArgumentParser(prog='PROG', add_help=False)
>>> group = parser.add_argument_group('grupo')
>>> group.add_argument('--foo', help='ajuda de foo')
>>> group.add_argument('bar', help='ajuda de bar')
>>> parser.print_help()
usage: PROG [--foo FOO] bar

grupo:
  bar    ajuda de bar
  --foo FOO  ajuda de foo
```

O método *add_argument_group()* retorna um objeto de grupo de argumentos que tem um método *add_argument()* como um *ArgumentParser* regular. Quando um argumento é adicionado ao grupo, o analisador o trata como um argumento normal, mas exibe o argumento em um grupo separado para mensagens de ajuda. O método `add_argument_group()` aceita argumentos *title* e *description* que podem ser usados para personalizar esta exibição:

```
>>> parser = argparse.ArgumentParser(prog='PROG', add_help=False)
>>> group1 = parser.add_argument_group('grupo1', 'descrição do grupo1')
>>> group1.add_argument('foo', help='ajuda de foo')
>>> group2 = parser.add_argument_group('grupo2', 'descrição do grupo2')
>>> group2.add_argument('--bar', help='ajuda de bar')
>>> parser.print_help()
usage: PROG [--bar BAR] foo

grupo1:
  descrição de grupo1

  foo    ajuda de foo

grupo2:
  descrição de grupo2

  --bar BAR  ajuda de bar
```

Os parâmetros opcionais somente-nomeados *argument_default* e *conflict_handler* permitem um controle mais refinado do comportamento do grupo de argumentos. Esses parâmetros têm o mesmo significado que no construtor `ArgumentParser`, mas se aplicam especificamente ao grupo de argumentos em vez de ao analisador sintático inteiro.

Observe que quaisquer argumentos que não estejam nos grupos definidos pelo usuário retornarão às seções usuais de "argumentos posicionais" e "argumentos opcionais".

Deprecated since version 3.11, removed in version 3.14: Chamar `add_argument_group()` em um grupo de argumentos agora levanta uma exceção. Esse aninhamento nunca foi suportado, frequentemente não funcionava corretamente e era exposto involuntariamente por meio de herança.

Descontinuado desde a versão 3.14: Passar *prefix_chars* para `add_argument_group()` agora está descontinuado.

Exclusão mútua

`ArgumentParser.`**`add_mutually_exclusive_group`**`(`*required=False*`)`

Cria um grupo mutuamente exclusivo. `argparse` garantirá que apenas um dos argumentos no grupo mutuamente exclusivo esteja presente na linha de comando:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> group = parser.add_mutually_exclusive_group()
>>> group.add_argument('--foo', action='store_true')
>>> group.add_argument('--bar', action='store_false')
>>> parser.parse_args(['--foo'])
Namespace(bar=True, foo=True)
>>> parser.parse_args(['--bar'])
Namespace(bar=False, foo=False)
>>> parser.parse_args(['--foo', '--bar'])
usage: PROG [-h] [--foo | --bar]
PROG: error: argument --bar: not allowed with argument --foo
```

O método `add_mutually_exclusive_group()` também aceita um argumento *obrigatório*, para indicar que pelo menos um dos argumentos mutuamente exclusivos é necessário:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> group = parser.add_mutually_exclusive_group(required=True)
>>> group.add_argument('--foo', action='store_true')
>>> group.add_argument('--bar', action='store_false')
>>> parser.parse_args([])
```

(continua na próxima página)

(continuação da página anterior)

```
usage: PROG [-h] (--foo | --bar)
PROG: error: one of the arguments --foo --bar is required
```

Observe que atualmente grupos de argumentos mutuamente exclusivos não oferecem suporte aos argumentos *title* e *description* de *add_argument_group()*. No entanto, um grupo mutuamente exclusivo pode ser adicionado a um grupo de argumentos que tenha um título e uma descrição. Por exemplo:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> group = parser.add_argument_group('Título do grupo', 'Descrição do grupo')
>>> exclusive_group = group.add_mutually_exclusive_group(required=True)
>>> exclusive_group.add_argument('--foo', help='ajuda de foo')
>>> exclusive_group.add_argument('--bar', help='ajuda de bar')
>>> parser.print_help()
usage: PROG [-h] (--foo FOO | --bar BAR)

options:
  -h, --help  show this help message and exit

Título do grupo:
  Descrição do grupo

  --foo FOO   foo help
  --bar BAR   bar help
```

Deprecated since version 3.11, removed in version 3.14: Chamar *add_argument_group()* ou *add_mutually_exclusive_group()* em um grupo mutuamente exclusivo agora levanta uma exceção. Esse aninhamento nunca foi suportado, frequentemente não funcionava corretamente e era exposto involuntariamente por meio de herança.

Padrões do analisador sintático

ArgumentParser.**set_defaults**(***kwargs*)

Na maioria das vezes, os atributos do objeto retornado por *parse_args()* serão totalmente determinados pela inspeção dos argumentos da linha de comando e das ações dos argumentos. *set_defaults()* permite que alguns atributos adicionais que são determinados sem qualquer inspeção da linha de comando sejam adicionados:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('foo', type=int)
>>> parser.set_defaults(bar=42, baz='badger')
>>> parser.parse_args(['736'])
Namespace(bar=42, baz='badger', foo=736)
```

Observe que os padrões no nível do analisador sempre substituem os padrões no nível do argumento:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default='bar')
>>> parser.set_defaults(foo='spam')
>>> parser.parse_args([])
Namespace(foo='spam')
```

Padrões de nível de analisador podem ser particularmente úteis ao trabalhar com vários analisadores. Veja o método *add_subparsers()* para um exemplo desse tipo.

ArgumentParser.**get_default**(*dest*)

Obtém o valor padrão para um atributo de espaço de nomes, conforme definido por *add_argument()* ou por *set_defaults()*:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default='badger')
>>> parser.get_default('foo')
'badger'
```

Imprimindo a ajuda

Na maioria das aplicações típicas, *parse_args()* cuidará da formatação e da impressão de quaisquer mensagens de uso ou erro. No entanto, vários métodos de formatação estão disponíveis:

ArgumentParser.**print_usage**(*file=None*)

Imprime uma breve descrição de como o *ArgumentParser* deve ser invocado na linha de comando. Se *file* for None, *sys.stdout* será presumido.

ArgumentParser.**print_help**(*file=None*)

Imprime uma mensagem de ajuda, incluindo o uso do programa e informações sobre os argumentos registrados com o *ArgumentParser*. Se *file* for None, *sys.stdout* será presumido.

Também há variantes desses métodos que simplesmente retornam uma string em vez de imprimi-la:

ArgumentParser.**format_usage**()

Retorna uma string contendo uma breve descrição de como o *ArgumentParser* deve ser invocado na linha de comando.

ArgumentParser.**format_help**()

Retorna uma string contendo uma mensagem de ajuda, incluindo o uso do programa e informações sobre os argumentos registrados com o *ArgumentParser*.

Análise parcial

ArgumentParser.**parse_known_args**(*args=None*, *namespace=None*)

Às vezes, um script precisa manipular apenas um conjunto específico de argumentos de linha de comando, deixando quaisquer argumentos não reconhecidos para outro script ou programa. Nesses casos, o método *parse_known_args()* pode ser útil.

Este método funciona de forma semelhante a *parse_args()*, mas não levanta erro para argumentos extras não reconhecidos. Em vez disso, ele analisa os argumentos conhecidos e retorna uma tupla de dois itens que contém o espaço de nomes preenchido e a lista de quaisquer argumentos não reconhecidos.

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='store_true')
>>> parser.add_argument('bar')
>>> parser.parse_known_args(['--foo', '--badger', 'BAR', 'spam'])
(Namespace(bar='BAR', foo=True), ['--badger', 'spam'])
```

Aviso

As regras de *correspondência de prefixo* se aplicam a *parse_known_args()*. O analisador pode usar uma opção mesmo que seja apenas um prefixo de uma de suas opções conhecidas, em vez de deixá-la na lista de argumentos restantes.

Personalizando a análise de arquivos

ArgumentParser.**convert_arg_line_to_args**(*arg_line*)

Argumentos lidos de um arquivo (veja o argumento nomeado *fromfile_prefix_chars* para o construtor *ArgumentParser*) são lidos com um argumento por linha. *convert_arg_line_to_args()* pode ser substituído para uma leitura mais sofisticada.

Este método recebe um único argumento *arg_line* que é uma string lida do arquivo de argumentos. Ele retorna uma lista de argumentos analisados dessa string. O método é chamado uma vez por linha lida do arquivo de argumentos, em ordem.

Uma substituição útil desse método é aquela que trata cada palavra separada por espaços como um argumento. O exemplo a seguir demonstra como fazer isso:

```
class MyArgumentParser(argparse.ArgumentParser):
    def convert_arg_line_to_args(self, arg_line):
        return arg_line.split()
```

Métodos de saída

ArgumentParser.**exit**(*status=0*, *message=None*)

Este método encerra o programa, saindo com o *status* especificado e, se fornecido, imprime uma *message* para *sys.stderr* antes disso. O usuário pode sobrescrever este método para lidar com essas etapas de forma diferente:

```
class ErrorCatchingArgumentParser(argparse.ArgumentParser):
    def exit(self, status=0, message=None):
        if status:
            raise Exception(f'Exiting because of an error: {message}')
        exit(status)
```

ArgumentParser.**error**(*message*)

Este método imprime uma mensagem de uso, incluindo a *message*, em *sys.stderr* e encerra o programa com um código de status 2.

Análise misturada

ArgumentParser.**parse_intermixed_args**(*args=None*, *namespace=None*)

ArgumentParser.**parse_known_intermixed_args**(*args=None*, *namespace=None*)

Vários comandos Unix permitem que o usuário misture argumentos opcionais com argumentos posicionais. Os métodos *parse_intermixed_args()* e *parse_known_intermixed_args()* oferecem suporte a esse estilo de análise.

Esses analisadores não oferecem suporte a todos os recursos do `argparse` e vão levantar exceções se recursos não suportados forem usados. Em particular, subanalisadores e grupos mutuamente exclusivos que incluem tanto opcionais quanto posicionais não são suportados.

O exemplo a seguir mostra a diferença entre *parse_known_args()* e *parse_intermixed_args()*: o primeiro retorna `['2', '3']` como argumentos não analisados, enquanto o último coleta todos os posicionais em `rest`.

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> parser.add_argument('cmd')
>>> parser.add_argument('rest', nargs='*', type=int)
>>> parser.parse_known_args('doit 1 --foo bar 2 3'.split())
(Namespace(cmd='doit', foo='bar', rest=[1]), ['2', '3'])
>>> parser.parse_intermixed_args('doit 1 --foo bar 2 3'.split())
Namespace(cmd='doit', foo='bar', rest=[1, 2, 3])
```

parse_known_intermixed_args() retorna uma tupla de dois itens contendo o espaço de nomes preenchido e a lista de strings de argumentos restantes. *parse_intermixed_args()* levanta um erro se houver strings de argumentos não analisadas restantes.

Adicionado na versão 3.7.

Registrando ações e tipos personalizados

`ArgumentParser.`**`register`**(*registry_name*, *value*, *object*)

Às vezes, é desejável usar uma string personalizada em mensagens de erro para fornecer uma saída mais amigável ao usuário. Nesses casos, `register()` pode ser usado para registrar ações ou tipos personalizados com um analisador sintático e permitir que você faça referência ao tipo pelo nome registrado em vez do nome do chamável.

O método `register()` aceita três argumentos - um *registry_name*, especificando o registro interno onde o objeto será armazenado (por exemplo, `action`, `type`), *value*, que é a chave sob a qual o objeto será registrado, e object, o chamável a ser registrado.

O exemplo a seguir mostra como registrar um tipo personalizado com um analisador:

```
>>> import argparse
>>> parser = argparse.ArgumentParser()
>>> parser.register('type', 'hexadecimal integer', lambda s: int(s, 16))
>>> parser.add_argument('--foo', type='hexadecimal integer')
_StoreAction(option_strings=['--foo'], dest='foo', nargs=None, const=None,␣
↪default=None, type='hexadecimal integer', choices=None, required=False,␣
↪help=None, metavar=None, deprecated=False)
>>> parser.parse_args(['--foo', '0xFA'])
Namespace(foo=250)
>>> parser.parse_args(['--foo', '1.2'])
usage: PROG [-h] [--foo FOO]
PROG: error: argument --foo: invalid 'hexadecimal integer' value: '1.2'
```

17.1.5 Exceções

exception `argparse.`**ArgumentError**

Um erro ao criar ou usar um argumento (opcional ou posicional).

O valor da string dessa exceção é a mensagem, complementada com informações sobre o argumento que a causou.

exception `argparse.`**ArgumentTypeError**

Levantada quando algo dá errado ao converter uma string de linha de comando em um tipo.

Guias e tutoriais

Tutorial de Argparse

autor
Tshepang Mbambo

Este tutorial pretende ser uma introdução gentil ao *`argparse`* — o módulo recomendado na biblioteca padrão do Python para fazer a análise de linha de comando.

Nota

A biblioteca padrão inclui duas outras bibliotecas diretamente relacionadas ao processamento de parâmetros de linha de comando: o módulo de nível inferior *`optparse`* (que pode exigir mais código para configurar uma determinada aplicação, mas também permite que uma aplicação solicite comportamentos que `argparse` não suporta), e o nível muito baixo *`getopt`* (que serve especificamente como um equivalente à família de funções `getopt()` disponível para programadores C). Embora nenhum desses módulos seja abordado diretamente neste guia, muitos dos conceitos principais em `argparse` se originaram primeiro em `optparse`, então alguns aspectos deste tutorial também serão relevantes para usuários de `optparse`.

Conceitos

Demonstraremos o tipo de funcionalidade que vamos explorar neste tutorial introdutório fazendo uso do comando **ls**:

```
$ ls
cpython  devguide  prog.py  pypy  rm-unused-function.patch
$ ls pypy
ctypes_configure  demo  dotviewer  include  lib_pypy  lib-python ...
$ ls -l
total 20
drwxr-xr-x 19 wena wena 4096 Feb 18 18:51 cpython
drwxr-xr-x  4 wena wena 4096 Feb  8 12:04 devguide
-rwxr-xr-x  1 wena wena  535 Feb 19 00:05 prog.py
drwxr-xr-x 14 wena wena 4096 Feb  7 00:59 pypy
-rw-r--r--  1 wena wena  741 Feb 18 01:01 rm-unused-function.patch
$ ls --help
Uso: ls [OPÇÃO]... [ARQUIVO]...
Lista informações sobre os ARQUIVOs (no diretório atual por padrão).
Lista as entradas em ordem alfabética se não for usada nenhuma opção -cftuvSUX
nem --sort..
...
```

Alguns conceitos que podemos aprender a partir destes quatro comandos:

- O comando **ls** é útil quando usado sem nenhuma opção. Por padrão, ele mostra o conteúdo do diretório atual.
- Se quisermos além do que ele fornece por padrão, contamos um pouco mais. Neste caso, queremos que ele exiba um diretório diferente, `pypy`. O que fizemos foi especificar o que é conhecido como argumento posicional. Ele é chamado assim porque o programa deve saber o que fazer com o valor, apenas com base em onde ele aparece na linha de comando. Este conceito é mais relevante para um comando como **cp**, cujo uso mais básico é `cp SRC DEST`. A primeira posição é *o que você deseja copiar* e a segunda posição é *para onde você deseja copiar*.
- Agora, digamos que queremos mudar o comportamento do programa. Em nosso exemplo, exibimos mais informações para cada arquivo em vez de apenas mostrar os nomes dos arquivos. O `-l` nesse caso é conhecido como um argumento opcional.
- Esse é um trecho do texto de ajuda. É muito útil que possas encontrar um programa que nunca usastes antes e poder descobrir como o mesmo funciona simplesmente lendo o seu texto de ajuda.

O básico

Comecemos com um exemplo muito simples que irá fazer (quase) nada:

```
import argparse
parser = argparse.ArgumentParser()
parser.parse_args()
```

A seguir, temos o resultado da execução do código:

```
$ python prog.py
$ python prog.py --help
usage: prog.py [-h]

options:
  -h, --help  show this help message and exit
$ python prog.py --verbose
usage: prog.py [-h]
prog.py: error: unrecognized arguments: --verbose
```

(continua na próxima página)

(continuação da página anterior)

```
$ python prog.py foo
usage: prog.py [-h]
prog.py: error: unrecognized arguments: foo
```

Eis aqui o que está acontecendo:

- Executar o script sem qualquer opção resultará que nada será exibido em stdout. Isso não é útil.
- O segundo começa a exibir as utilidades do módulo *argparse*. Não fizemos quase nada, mas já recebemos uma boa mensagem de ajuda.
- A opção `--help`, que também pode ser encurtada para `-h`, é a única opção que obtemos livremente (ou seja, não é necessário determiná-la). Determinar qualquer outra coisa resulta num erro. Mas mesmo assim, recebemos uma mensagem de uso bastante útil, também de graça.

Apresentando os argumentos posicionais

Um exemplo:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("echo")
args = parser.parse_args()
print(args.echo)
```

E executando o código:

```
$ python prog.py
usage: prog.py [-h] echo
prog.py: error: the following arguments are required: echo
$ python prog.py --help
usage: prog.py [-h] echo

positional arguments:
  echo

options:
  -h, --help  show this help message and exit
$ python prog.py foo
foo
```

Aqui está o que acontecerá:

- Nós adicionamos o método *add_argument()*, cujo o mesmo usamos para especificar quais opções de linha de comando o programa está disposto a aceitar. Neste caso, eu o nomeei `echo` para que ele esteja de acordo com sua função.
- Chamar o nosso programa neste momento, requer a especificação de uma opção.
- O método *parse_args()* realmente retorna alguns dados das opções especificadas, neste caso, `echo`.
- A variável é uma forma de "mágica" que *argparse* executa de brinde (ou seja, não é necessário especificar em qual variável esse valor é armazenado). Você também notará que seu nome corresponde ao argumento string dado ao método, `echo`.

Observe, no entanto, que, embora a tela de ajuda pareça boa e tudo, atualmente não é tão útil quanto poderia ser. Por exemplo, vemos que temos `echo` como um argumento posicional, mas não sabemos o que ele faz, além de adivinhar ou ler o código-fonte. Então, vamos torná-lo um pouco mais útil:

```
import argparse
parser = argparse.ArgumentParser()
```

(continua na próxima página)

(continuação da página anterior)

```
parser.add_argument("echo", help="echo the string you use here")
args = parser.parse_args()
print(args.echo)
```

E, iremos obter:

```
$ python prog.py -h
usage: prog.py [-h] echo

positional arguments:
  echo        echo the string you use here

options:
  -h, --help  show this help message and exit
```

Agora, que tal fazer algo ainda mais útil:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", help="display a square of a given number")
args = parser.parse_args()
print(args.square**2)
```

A seguir, temos o resultado da execução do código:

```
$ python prog.py 4
Traceback (most recent call last):
  File "prog.py", line 5, in <module>
    print(args.square**2)
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

Isso não correu tão bem. Isso porque *argparse* trata as opções que damos a ele como strings, a menos que digamos o contrário. Então, vamos dizer ao *argparse* para tratar essa entrada como um inteiro:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", help="display a square of a given number",
                    type=int)
args = parser.parse_args()
print(args.square**2)
```

A seguir, temos o resultado da execução do código:

```
$ python prog.py 4
16
$ python prog.py four
usage: prog.py [-h] square
prog.py: error: argument square: invalid int value: 'four'
```

Correu bem. O programa agora até fecha com ajuda de entrada ilegal ruim antes de prosseguir.

Apresentando os argumentos opcionais

Até agora, jogamos com argumentos posicionais. Vamos dar uma olhada em como adicionar opcionais:

```
import argparse
parser = argparse.ArgumentParser()
```

(continua na próxima página)

(continuação da página anterior)

```
parser.add_argument("--verbosity", help="increase output verbosity")
args = parser.parse_args()
if args.verbosity:
    print("verbosity turned on")
```

E a saída:

```
$ python prog.py --verbosity 1
verbosity turned on
$ python prog.py
$ python prog.py --help
usage: prog.py [-h] [--verbosity VERBOSITY]

options:
  -h, --help            show this help message and exit
  --verbosity VERBOSITY
                        increase output verbosity
$ python prog.py --verbosity
usage: prog.py [-h] [--verbosity VERBOSITY]
prog.py: error: argument --verbosity: expected one argument
```

Eis aqui o que está acontecendo:

- O programa é escrito de forma a exibir algo quando `--verbosity` é especificado e não exibir nada quando não for.
- Para mostrar que a opção é realmente opcional, não há erro ao executar o programa sem ela. Observe que, por padrão, se um argumento opcional não for usado, a variável relevante, neste caso `args.verbosity`, recebe `None` como valor, razão pela qual falha no teste de verdade da instrução `if`.
- A mensagem de ajuda é um pouco diferente.
- Ao usar a opção `--verbosity`, deve-se também especificar algum valor, qualquer valor.

O exemplo acima aceita valores inteiros arbitrários para `--verbosity`, mas para nosso programa simples, apenas dois valores são realmente úteis, `True` ou `False`. Vamos modificar o código de acordo:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--verbose", help="increase output verbosity",
                    action="store_true")
args = parser.parse_args()
if args.verbose:
    print("verbosity turned on")
```

E a saída:

```
$ python prog.py --verbose
verbosity turned on
$ python prog.py --verbose 1
usage: prog.py [-h] [--verbose]
prog.py: error: unrecognized arguments: 1
$ python prog.py --help
usage: prog.py [-h] [--verbose]

options:
  -h, --help  show this help message and exit
  --verbose   increase output verbosity
```

Eis aqui o que está acontecendo:

- A opção agora é mais um sinalizador do que algo que requer um valor. Até mudamos o nome da opção para corresponder a essa ideia. Observe que agora especificamos uma nova palavra reservada, `action`, e damos a ela o valor `"store_true"`. Isso significa que, se a opção for especificada, atribui o valor `True` para `args.verbose`. Não especificá-la implica em `False`.
- Ele reclama quando você especifica um valor, no verdadeiro espírito do que os sinalizadores realmente são.
- Observe o texto de ajuda diferente.

Opções curtas

Se você estiver familiarizado com o uso da linha de comando, notará que ainda não toquei no tópico das versões curtas das opções. É bem simples:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("-v", "--verbose", help="increase output verbosity",
                    action="store_true")
args = parser.parse_args()
if args.verbose:
    print("verbosity turned on")
```

E aqui vai:

```
$ python prog.py -v
verbosity turned on
$ python prog.py --help
usage: prog.py [-h] [-v]

options:
  -h, --help     show this help message and exit
  -v, --verbose  increase output verbosity
```

Observe que a nova habilidade também é refletida no texto de ajuda.

Combinando argumentos posicionais e opcionais

Nosso programa continua crescendo em complexidade:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbose", action="store_true",
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbose:
    print(f"the square of {args.square} equals {answer}")
else:
    print(answer)
```

E agora a saída:

```
$ python prog.py
usage: prog.py [-h] [-v] square
prog.py: error: the following arguments are required: square
$ python prog.py 4
16
```

(continua na próxima página)

(continuação da página anterior)

```
$ python prog.py 4 --verbose
the square of 4 equals 16
$ python prog.py --verbose 4
the square of 4 equals 16
```

- Trouxemos de volta um argumento posicional, daí a reclamação.
- Observe que a ordem não importa.

Que tal devolvermos a este nosso programa a capacidade de ter vários valores de verbosidade e realmente usá-los:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbosity", type=int,
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbosity == 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity == 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

E a saída:

```
$ python prog.py 4
16
$ python prog.py 4 -v
usage: prog.py [-h] [-v VERBOSITY] square
prog.py: error: argument -v/--verbosity: expected one argument
$ python prog.py 4 -v 1
4^2 == 16
$ python prog.py 4 -v 2
the square of 4 equals 16
$ python prog.py 4 -v 3
16
```

Todos eles parecem bons, exceto o último, que expõe um bug em nosso programa. Vamos corrigi-lo restringindo os valores que a opção `--verbosity` pode aceitar:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbosity", type=int, choices=[0, 1, 2],
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbosity == 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity == 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

E a saída:

```
$ python prog.py 4 -v 3
usage: prog.py [-h] [-v {0,1,2}] square
prog.py: error: argument -v/--verbosity: invalid choice: 3 (choose from 0, 1, 2)
$ python prog.py 4 -h
usage: prog.py [-h] [-v {0,1,2}] square

positional arguments:
  square                display a square of a given number

options:
  -h, --help            show this help message and exit
  -v, --verbosity {0,1,2}
                        increase output verbosity
```

Observe que a alteração também reflete tanto na mensagem de erro quanto na string de ajuda.

Agora, vamos usar uma abordagem diferente de brincar com a verbosidade, que é bastante comum. Ele também corresponde à maneira como o executável do CPython trata seu próprio argumento de verbosidade (verifique a saída de `python --help`):

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display the square of a given number")
parser.add_argument("-v", "--verbosity", action="count",
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbosity == 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity == 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

Introduzimos outra ação, “contar”, para contar o número de ocorrências de opções específicas.

```
$ python prog.py 4
16
$ python prog.py 4 -v
4^2 == 16
$ python prog.py 4 -vv
the square of 4 equals 16
$ python prog.py 4 --verbosity --verbosity
the square of 4 equals 16
$ python prog.py 4 -v 1
usage: prog.py [-h] [-v] square
prog.py: error: unrecognized arguments: 1
$ python prog.py 4 -h
usage: prog.py [-h] [-v] square

positional arguments:
  square           display a square of a given number

options:
  -h, --help       show this help message and exit
  -v, --verbosity  increase output verbosity
$ python prog.py 4 -vvv
```

(continua na próxima página)

(continuação da página anterior)

```
16
```

- Sim, agora é mais um sinalizador (semelhante a `action="store_true"`) na versão anterior do nosso script. Isso deve explicar a reclamação.
- Ele também se comporta de maneira semelhante à ação "store_true".
- Agora aqui está uma demonstração do que a ação "contar" oferece. Você provavelmente já viu esse tipo de uso antes.
- E se você não especificar o sinalizador `-v`, esse sinalizador será considerado como tendo valor `None`.
- Como deve ser esperado, especificando a forma longa do sinalizador, devemos obter a mesma saída.
- Infelizmente, nossa saída de ajuda não é muito informativa sobre a nova habilidade que nosso script adquiriu, mas isso sempre pode ser corrigido melhorando a documentação de nosso script (por exemplo, através do argumento nomeado `help`).
- Essa última saída expõe um bug em nosso programa.

Vamos corrigir:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbosity", action="count",
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2

# bugfix: replace == with >=
if args.verbosity >= 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity >= 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

E isso aqui é o mesmo retorna:

```
$ python prog.py 4 -vvv
the square of 4 equals 16
$ python prog.py 4 -vvvv
the square of 4 equals 16
$ python prog.py 4
Traceback (most recent call last):
  File "prog.py", line 11, in <module>
    if args.verbosity >= 2:
TypeError: '>=' not supported between instances of 'NoneType' and 'int'
```

- A primeira saída correu bem e corrige o bug que tínhamos antes. Ou seja, queremos que qualquer valor >= 2 seja o mais detalhado possível.
- A terceira saída não está tão boa.

Vamos corrigir esse bug:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
```

(continua na próxima página)

(continuação da página anterior)

```
                    help="display a square of a given number")
parser.add_argument("-v", "--verbosity", action="count", default=0,
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbosity >= 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity >= 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

Acabamos de introduzir outra palavra reservada, `default`. Nós o configuramos como `0` para torná-lo comparável aos outros valores int. Lembre-se que por padrão, se um argumento opcional não for especificado, ele obtém o valor `None`, e isso não pode ser comparado a um valor int (daí a exceção *`TypeError`*).

E:

```
$ python prog.py 4
16
```

Você pode ir muito longe apenas com o que aprendemos até agora, e nós apenas arranhamos a superfície. O módulo *`argparse`* é muito poderoso, e vamos explorar um pouco mais antes de terminar este tutorial.

Avançando um pouco mais

E se quiséssemos expandir nosso pequeno programa, ampliando seu potencial:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("x", type=int, help="the base")
parser.add_argument("y", type=int, help="the exponent")
parser.add_argument("-v", "--verbosity", action="count", default=0)
args = parser.parse_args()
answer = args.x**args.y
if args.verbosity >= 2:
    print(f"{args.x} to the power {args.y} equals {answer}")
elif args.verbosity >= 1:
    print(f"{args.x}^{args.y} == {answer}")
else:
    print(answer)
```

Saída:

```
$ python prog.py
usage: prog.py [-h] [-v] x y
prog.py: error: the following arguments are required: x, y
$ python prog.py -h
usage: prog.py [-h] [-v] x y

positional arguments:
  x                the base
  y                the exponent

options:
  -h, --help       show this help message and exit
  -v, --verbosity
```

(continua na próxima página)

(continuação da página anterior)

```
$ python prog.py 4 2 -v
4^2 == 16
```

Observe que até agora estamos usando o nível de verbosidade para *alterar* o texto que é exibido. O exemplo a seguir usa o nível de verbosidade para exibir *mais* texto:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("x", type=int, help="the base")
parser.add_argument("y", type=int, help="the exponent")
parser.add_argument("-v", "--verbosity", action="count", default=0)
args = parser.parse_args()
answer = args.x**args.y
if args.verbosity >= 2:
    print(f"Running '{__file__}'")
if args.verbosity >= 1:
    print(f"{args.x}^{args.y} == ", end="")
print(answer)
```

Saída:

```
$ python prog.py 4 2
16
$ python prog.py 4 2 -v
4^2 == 16
$ python prog.py 4 2 -vv
Running 'prog.py'
4^2 == 16
```

Especificando argumentos ambíguos

Quando há ambiguidade em decidir se um argumento é posicional ou para um argumento, `--` pode ser usado para dizer *parse_args()* que tudo depois disso é um argumento posicional:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-n', nargs='+')
>>> parser.add_argument('args', nargs='*')

>>> # ambiguous, so parse_args assumes it's an option
>>> parser.parse_args(['-f'])
usage: PROG [-h] [-n N [N ...]] [args ...]
PROG: error: unrecognized arguments: -f

>>> parser.parse_args(['--', '-f'])
Namespace(args=['-f'], n=None)

>>> # ambiguous, so the -n option greedily accepts arguments
>>> parser.parse_args(['-n', '1', '2', '3'])
Namespace(args=[], n=['1', '2', '3'])

>>> parser.parse_args(['-n', '1', '--', '2', '3'])
Namespace(args=['2', '3'], n=['1'])
```

Opções conflitantes

Até agora, trabalhamos com dois métodos de uma instância *argparse.ArgumentParser*. Vamos apresentar um terceiro, *add_mutually_exclusive_group()*. Ele nos permite especificar opções que entram em conflito umas com as outras. Vamos também alterar o resto do programa para que a nova funcionalidade faça mais sentido: vamos introduzir a opção `--quiet`, que será o oposto da opção `--verbose`:

```
import argparse

parser = argparse.ArgumentParser()
group = parser.add_mutually_exclusive_group()
group.add_argument("-v", "--verbose", action="store_true")
group.add_argument("-q", "--quiet", action="store_true")
parser.add_argument("x", type=int, help="the base")
parser.add_argument("y", type=int, help="the exponent")
args = parser.parse_args()
answer = args.x**args.y

if args.quiet:
    print(answer)
elif args.verbose:
    print(f"{args.x} to the power {args.y} equals {answer}")
else:
    print(f"{args.x}^{args.y} == {answer}")
```

Nosso programa agora está mais simples e perdemos algumas funcionalidades para demonstração. De qualquer forma, aqui está a saída:

```
$ python prog.py 4 2
4^2 == 16
$ python prog.py 4 2 -q
16
$ python prog.py 4 2 -v
4 to the power 2 equals 16
$ python prog.py 4 2 -vq
usage: prog.py [-h] [-v | -q] x y
prog.py: error: argument -q/--quiet: not allowed with argument -v/--verbose
$ python prog.py 4 2 -v --quiet
usage: prog.py [-h] [-v | -q] x y
prog.py: error: argument -q/--quiet: not allowed with argument -v/--verbose
```

Isso deve ser fácil de seguir. Eu adicionei essa última saída para que você possa ver o tipo de flexibilidade que obtém, ou seja, misturar opções de formato longo com formatos curtos.

Antes de concluirmos, você provavelmente quer dizer aos seus usuários o propósito principal do seu programa, caso eles não saibam:

```
import argparse

parser = argparse.ArgumentParser(description="calculate X to the power of Y")
group = parser.add_mutually_exclusive_group()
group.add_argument("-v", "--verbose", action="store_true")
group.add_argument("-q", "--quiet", action="store_true")
parser.add_argument("x", type=int, help="the base")
parser.add_argument("y", type=int, help="the exponent")
args = parser.parse_args()
answer = args.x**args.y

if args.quiet:
```

(continua na próxima página)

(continuação da página anterior)

```
    print(answer)
elif args.verbose:
    print(f"{args.x} to the power {args.y} equals {answer}")
else:
    print(f"{args.x}^{args.y} == {answer}")
```

Observe essa pequena diferença no texto de uso. Observe o `[-v | -q]`, que nos diz que podemos usar `-v` ou `-q`, mas não ambos ao mesmo tempo:

```
$ python prog.py --help
usage: prog.py [-h] [-v | -q] x y

calculate X to the power of Y

positional arguments:
  x              the base
  y              the exponent

options:
  -h, --help     show this help message and exit
  -v, --verbose
  -q, --quiet
```

Como traduzir a saída do argparse

A saída do módulo *argparse*, como seu texto de ajuda e mensagens de erro, são todos traduzidos usando o módulo *gettext*. Isso permite que os aplicativos localizem facilmente as mensagens produzidas por *argparse*. Veja também *Internacionalizando seus programas e módulos*.

Por exemplo, nesta saída *argparse*:

```
$ python prog.py --help
usage: prog.py [-h] [-v | -q] x y

calculate X to the power of Y

positional arguments:
  x              the base
  y              the exponent

options:
  -h, --help     show this help message and exit
  -v, --verbose
  -q, --quiet
```

As strings `usage:`, `positional arguments:`, `options:` e `show this help message and exit` são todos traduzíveis.

Para traduzir essas strings, elas devem primeiro ser extraídas em um arquivo `.po`. Por exemplo, usando Babel, execute este comando:

```
$ pybabel extract -o messages.po /usr/lib/python3.12/argparse.py
```

Este comando extrairá todas as strings traduzíveis do módulo *argparse* e as enviará para um arquivo chamado `messages.po`. Este comando presume que sua instalação do Python está em `/usr/lib`.

Você pode descobrir a localização do módulo *argparse* em seu sistema usando este script:

```
import argparse
print(argparse.__file__)
```

Uma vez que as mensagens no arquivo `.po` sejam traduzidas e as traduções instaladas usando *gettext*, *argparse* será capaz de exibir as mensagens traduzidas.

Para traduzir suas próprias strings na saída *argparse*, use *gettext*.

Conversores de tipo personalizados

O módulo *argparse* permite que você especifique conversores de tipo personalizados para seus argumentos de linha de comando. Isso permite modificar a entrada de usuário antes de armazená-la na *argparse.Namespace*. Isso pode ser útil quando você precisa pré-processar a entrada antes dela ser usada em seu programa.

Ao usar um conversor de tipo personalizado, você pode usar qualquer chamável que recebe uma única string como argumento (o valor do argumento) e retorna o valor convertido. Porém, se você precisa tratar de cenários mais complexos, você pode usar uma classe de ação personalizada com o parâmetro **action**.

Por exemplo, digamos que vcoê deseja tratar de argumentos com diferentes prefixos e processá-los adequadamente:

```
import argparse

parser = argparse.ArgumentParser(prefix_chars='-+')

parser.add_argument('-a', metavar='<value>', action='append',
                    type=lambda x: ('-', x))
parser.add_argument('+a', metavar='<value>', action='append',
                    type=lambda x: ('+', x))

args = parser.parse_args()
print(args)
```

Saída:

```
$ python prog.py -a valor1 +a valor2
Namespace(a=[('-', 'valor1'), ('+', 'valor2')])
```

Neste exemplos, nós:

- Criamos um analisador sintático com caracteres de prefixo personalizado usando parâmetro `prefix_chars`.
- Definimos dois argumentos, `-a` e `+a`, que usaram o parâmetro `type` para criar conversores de tipo personalizados para armazenar o valor em uma tupla com o prefixo.

Sem os conversores de tipo personalizados, os argumentos tratariam `-a` e `+a` como o mesmo argumento, o que não era o desejado. Ao usar conversores de tipo personalizados, conseguimos diferenciar entre dois argumentos.

Conclusão

O módulo *argparse* oferece muito mais do que o mostrado aqui. Sua documentação é bastante detalhada e completo, e cheia de exemplos. Tendo passado por este tutorial, você deve digeri-la facilmente sem se sentir sobrecarregado.

Migrando código `optparse` para `argparse`

O módulo *argparse* oferece vários recursos de nível mais alto não fornecidos nativamente pelo módulo *optparse*, incluindo:

- Tratando argumentos posicionais.
- Prover suporte a subcomandos.
- Permitir prefixos alternativos de opções como `+` e `/`.

- Manipular argumentos de estilo “zero ou mais” e “um ou mais”.
- Produzir mensagens de uso mais informativas.
- Fornecer uma interface muito mais simples para `type` e `action` personalizados.

Originalmente, o módulo *argparse* tentou manter a compatibilidade com *optparse*. No entanto, as diferenças fundamentais de design entre suportar o processamento de opções de linha de comando declarativas (enquanto deixa o processamento de argumentos posicionais para o código da aplicação) e suportar opções nomeadas e argumentos posicionais na interface declarativa significam que a API divergiu daquela de `optparse` ao longo do tempo.

Conforme descrito em *Escolhendo uma biblioteca de análise de linha de comando*, as aplicações que atualmente usam *optparse* e estão satisfeitos com a forma como ele funciona podem continuar usando `optparse`.

Os desenvolvedores de aplicações que estão pensando em migrar também devem revisar a lista de diferenças comportamentais intrínsecas descritas nessa seção antes de decidir se a migração é desejável ou não.

Para aplicações que optam por migrar de *optparse* para *argparse*, as seguintes sugestões devem ser úteis:

- Substituir todas as chamadas de *optparse.OptionParser.add_option()* por chamadas de *ArgumentParser.add_argument()*.
- Substituir `(options, args) = parser.parse_args()` por `args = parser.parse_args()` e adicionar chamadas adicionais a *ArgumentParser.add_argument()* para os argumentos posicionais. Tenha em mente que o que anteriormente era chamado de `options`, agora no contexto do *argparse* é chamado de `args`.
- Substituir *optparse.OptionParser.disable_interspersed_args()* usando *parse_intermixed_args()* em vez de *parse_args()*.
- Substituir ações de função de retorno e argumentos nomeados `callback_*` por argumentos `type` ou `action`.
- Substituir nomes de strings para argumentos nomeados `type` pelos objetos de tipo correspondentes (por exemplo, int, float, complex, etc).
- Substituir *optparse.Values* por *Namespace* e *optparse.OptionError* e *optparse.OptionValueError* por *ArgumentError*.
- Substituir strings com argumentos implícitos tal como `%default` ou `%prog` pela sintaxe padrão do Python para usar dicionários para formatar strings, ou seja, `%(default)s` e `%(prog)s`.
- Substituir o argumento `version` do construtor do OptionParser por uma chamada a `parser.add_argument('--version', action='version', version='<the version>')`.

17.2 `optparse` — Analisador sintático para opções de linha de comando

Código-fonte: Lib/optparse.py

17.2.1 Escolhendo uma biblioteca de análise de linha de comando

A biblioteca padrão inclui três bibliotecas de análise de argumentos:

- *getopt*: um módulo que espelha de perto a API procedural C `getopt`. Incluído na biblioteca padrão desde antes do lançamento inicial do Python 1.0.
- *optparse*: uma substituição declarativa para `getopt` que fornece funcionalidade equivalente sem exigir que cada aplicação implemente sua própria lógica de análise de opções procedural. Incluído na biblioteca padrão desde o lançamento do Python 2.3.
- *argparse*: uma alternativa mais opinativa a `optparse` que fornece mais funcionalidade por padrão, às custas de flexibilidade reduzida da aplicação em controlar exatamente como os argumentos são processados. Incluído na biblioteca padrão desde as versões Python 2.7 e Python 3.2.

Na ausência de restrições de design de análise de argumentos mais específicas, *argparse* é a escolha recomendada para implementar aplicações de linha de comando, pois oferece o mais alto nível de funcionalidade básica com o mínimo de código em nível de aplicação.

getopt é mantido quase inteiramente por razões de retrocompatibilidade. No entanto, ele também serve como um caso de uso de nicho como uma ferramenta para prototipagem e teste de manipulação de argumentos de linha de comando em aplicações C baseadas em `getopt`.

optparse deve ser considerado como uma alternativa ao *argparse* nos seguintes casos:

- uma aplicação já está usando *optparse* e não quer arriscar as sutis mudanças comportamentais que podem surgir ao migrar para *argparse*
- a aplicação requer controle adicional sobre a maneira como as opções e os parâmetros posicionais são intercalados na linha de comando (incluindo a capacidade de desabilitar completamente o recurso de intercalação)
- a aplicação requer controle adicional sobre a análise incremental de elementos de linha de comando (embora `argparse` suporte isso, a maneira exata como ele funciona na prática é indesejável para alguns casos de uso)
- a aplicação requer controle adicional sobre o tratamento de opções que aceitam valores de parâmetros que podem começar com `-` (como opções delegadas a serem passadas para subprocessos invocados)
- a aplicação requer algum outro comportamento de processamento de parâmetros de linha de comando que `argparse` não suporta, mas que pode ser implementado em termos da interface de nível inferior oferecida por `optparse`

Essas considerações também significam que *optparse* provavelmente fornecerá uma base melhor para autores de bibliotecas que escrevem bibliotecas de processamento de argumentos de linha de comando de terceiros.

Como exemplo concreto, considere as duas configurações de análise de argumentos de linha de comando a seguir, a primeira usando `optparse` e a segunda usando `argparse`:

```
import optparse

if __name__ == '__main__':
    parser = optparse.OptionParser()
    parser.add_option('-o', '--output')
    parser.add_option('-v', dest='verbose', action='store_true')
    opts, args = parser.parse_args()
    process(args, output=opts.output, verbose=opts.verbose)
```

```
import argparse

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('-o', '--output')
    parser.add_argument('-v', dest='verbose', action='store_true')
    parser.add_argument('rest', nargs='*')
    args = parser.parse_args()
    process(args.rest, output=args.output, verbose=args.verbose)
```

A diferença mais óbvia é que na versão `optparse`, os argumentos não opcionais são processados separadamente pelo aplicativo após o processamento da opção ser concluído. Na versão `argparse`, os argumentos posicionais são declarados e processados da mesma forma que as opções nomeadas.

No entanto, a versão `argparse` também manipulará algumas combinações de parâmetros de forma diferente da forma como a versão `optparse` as manipularia. Por exemplo (entre outras diferenças):

- fornecer `-o -v` retorna `output="-v"` e `verbose=False` ao usar `optparse`, mas um erro de uso com `argparse` (reclamando que nenhum valor foi fornecido para `-o/--output`, já que `-v` é interpretado como significando o sinalizador de verbosidade)
- da mesma forma, fornecer `-o --` retorna `output="--"` e `args=()` ao usar `optparse`, mas um erro de uso com `argparse` (também reclamando que nenhum valor foi fornecido para `-o/--output`, já que `--` é

interpretado como o término do processamento da opção e o tratamento de todos os valores restantes como argumentos posicionais)

- fornecer `-o=foo` retorna `output="=foo"` ao usar `optparse`, mas retorna `output="foo"` com `argparse` (já que = é um caso especial como um separador alternativo para valores de parâmetros de opção)

Se esses comportamentos diferentes na versão `argparse` são considerados desejáveis ou um problema dependerá do caso de uso específico da aplicação de linha de comando.

Ver também

click é uma biblioteca de processamento de argumentos de terceiros (originalmente baseada em `optparse`), que permite que aplicações de linha de comando sejam desenvolvidas como um conjunto de funções de implementação de comando decoradas.

Outras bibliotecas de terceiros, como typer ou msgspec-click, permitem que interfaces de linha de comando sejam especificadas de maneiras que se integrem de forma mais eficaz com a verificação estática de anotações de tipo Python.

17.2.2 Introdução

optparse é uma biblioteca mais conveniente, flexível e poderosa para analisar opções de linha de comando do que o minimalista módulo *getopt*. *optparse* usa um estilo mais declarativo de análise de linha de comando: você cria uma instância de *OptionParser*, preenche-a com opções e analisa a linha de comando. *optparse* permite que os usuários especifiquem opções na sintaxe convencional GNU/POSIX e, adicionalmente, gera mensagens de uso e ajuda para você.

Aqui está um exemplo de uso de *optparse* em um script simples:

```
from optparse import OptionParser
...
parser = OptionParser()
parser.add_option("-f", "--file", dest="filename",
                  help="write report to FILE", metavar="FILE")
parser.add_option("-q", "--quiet",
                  action="store_false", dest="verbose", default=True,
                  help="don't print status messages to stdout")

(options, args) = parser.parse_args()
```

Com essas poucas linhas de código, os usuários do seu script agora podem fazer "coisas comuns" na linha de comando, por exemplo:

```
<seuscript> --file=outfile -q
```

Ao analisar a linha de comando, *optparse* define atributos do objeto `options` retornado por *parse_args()* com base nos valores de linha de comando fornecidos pelo usuário. Quando *parse_args()* retorna da análise desta linha de comando, `options.filename` será `"outfile"` e `options.verbose` será `False`. *optparse* provê as opções longas e curtas, permite que opções curtas sejam mescladas e permite que opções sejam associadas a seus argumentos de várias maneiras. Portanto, as seguintes linhas de comando são todas equivalentes ao exemplo acima:

```
<seuscript> -f outfile --quiet
<seuscript> --quiet --file outfile
<seuscript> -q -foutfile
<seuscript> -qfoutfile
```

Além disso, os usuários podem executar um dos seguintes:

```
<seuscript> -h
<seuscript> --help
```

e *optparse* vai exibir um breve resumo das opções do seu script:

```
Usage: <seuscript> [options]

Options:
  -h, --help            show this help message and exit
  -f FILE, --file=FILE  write report to FILE
  -q, --quiet           don't print status messages to stdout
```

onde o valor de *seuscript* é determinado em tempo de execução (normalmente de `sys.argv[0]`).

17.2.3 Contexto

optparse foi explicitamente projetado para encorajar a criação de programas com interfaces de linha de comando diretas que seguem as convenções estabelecidas pela família `getopt()` de funções disponíveis para os desenvolvedores C. Para esse fim, ele provê apenas a sintaxe e a semântica de linha de comando mais comuns convencionalmente usadas no Unix. Se você não estiver familiarizado com essas convenções, ler esta seção vai permitir que você se familiarize com elas.

Terminologia

argumento
: uma string inserida na linha de comando e passada pelo shell para `execl()` ou `execv()`. Em Python, argumentos são elementos de `sys.argv[1:]` (`sys.argv[0]` é o nome do programa que está sendo executado). Shells Unix também usam o termo "word".

  Ocasionalmente, é desejável substituir uma lista de argumentos diferente de `sys.argv[1:]`, então você deve ler "argumento" como "um elemento de `sys.argv[1:]`, ou de alguma outra lista fornecida como um substituto para `sys.argv[1:]`".

opção
: um argumento usado para fornecer informações extras para guiar ou personalizar a execução de um programa. Existem muitas sintaxes diferentes para opções; a sintaxe tradicional do Unix é um hífen ("-") seguido por uma única letra, por exemplo, `-x` ou `-F`. Além disso, a sintaxe tradicional do Unix permite que várias opções sejam mescladas em um único argumento, por exemplo, `-x -F` é equivalente a `-xF`. O projeto GNU introduziu `--` seguido por uma série de palavras separadas por hífen, por exemplo, `--file` ou `--dry-run`. Essas são as únicas duas sintaxes de opção fornecidas por *optparse*.

  Algumas outras sintaxes de opções que o mundo viu incluem:

  - um hífen seguido por algumas letras, por exemplo `-pf` (isso *não* é o mesmo que várias opções mescladas em um único argumento)
  - um hífen seguido por uma palavra completa, por exemplo `-file` (isso é tecnicamente equivalente à sintaxe anterior, mas eles geralmente não são vistos no mesmo programa)
  - um sinal de mais seguido por uma única letra, ou algumas letras, ou uma palavra, por exemplo `+f`, `+rgb`
  - uma barra seguida de uma letra, ou algumas letras, ou uma palavra, por exemplo `/f`, `/file`

  Essas sintaxes de opção não são suportadas por *optparse*, e nunca serão. Isso é proposital: as três primeiras não são padrão em nenhum ambiente, e a última só faz sentido se você estiver mirando exclusivamente no Windows ou em certas plataformas legadas (por exemplo, VMS, MS-DOS).

argumento de opção
: um argumento que segue uma opção, está intimamente associado a essa opção e é consumido da lista de argumentos quando essa opção é. Com *optparse*, os argumentos de opção podem estar em um argumento separado de sua opção:

  ```
  -f foo
  --file foo
  ```

  ou incluídos no mesmo argumento:

```
-ffoo
--file=foo
```

Normalmente, uma opção dada ou aceita um argumento ou não. Muitas pessoas querem um recurso de "argumentos opcionais de opção", o que significa que algumas opções aceitarão um argumento se o virem, e não aceitarão se não o virem. Isso é um tanto controverso, porque torna a análise ambígua: se `-a` aceita um argumento opcional e `-b` é outra opção inteiramente diferente, como interpretamos `-ab`? Por causa dessa ambiguidade, *optparse* não provê esse recurso.

argumento posicional
algo que sobrou na lista de argumentos depois que as opções foram analisadas, ou seja, depois que as opções e seus argumentos foram analisados e removidos da lista de argumentos.

opção obrigatória
uma opção que deve ser fornecida na linha de comando; observe que a frase "opção obrigatória" é autocontraditória em inglês. *optparse* não impede que você implemente opções obrigatórias, mas também não ajuda muito nisso.

Por exemplo, considere esta linha de comando hipotética:

```
prog -v --report report.txt foo bar
```

`-v` e `--report` são ambas opções. Supondo que `--report` receba um argumento, `report.txt` é um argumento de opção. `foo` e `bar` são argumentos posicionais.

Para que servem as opções?

As opções são usadas para fornecer informações extras para ajustar ou personalizar a execução de um programa. Caso não tenha ficado claro, as opções geralmente são *opcionais*. Um programa deve ser capaz de executar muito bem sem nenhuma opção. (Escolha um programa aleatório dos conjuntos de ferramentas Unix ou GNU. Ele pode ser executado sem nenhuma opção e ainda fazer sentido? As principais exceções são `find`, `tar` e `dd` — todos eles são mutantes excêntricos que foram corretamente criticados por sua sintaxe não padrão e interfaces confusas.)

Muitas pessoas querem que seus programas tenham "opções obrigatórias". Pense nisso. Se é obrigatório, então *não é opcional*! Se há uma informação que seu programa absolutamente requer para ser executado com sucesso, é para isso que servem os argumentos posicionais.

Como um exemplo de bom design de interface de linha de comando, considere o humilde utilitário `cp`, para copiar arquivos. Não faz muito sentido tentar copiar arquivos sem fornecer um destino e pelo menos uma fonte. Portanto, `cp` falha se você executá-lo sem argumentos. No entanto, ele tem uma sintaxe flexível e útil que não requer nenhuma opção:

```
cp ORIGEM DESTINO
cp ORIGEM ... DIRETÓRIO
```

Você pode ir bem longe só com isso. A maioria das implementações de `cp` fornece um monte de opções para ajustar exatamente como os arquivos são copiados: você pode preservar o modo e o tempo de modificação, evitar seguir links simbólicos, perguntar antes de destruir arquivos existentes, etc. Mas nada disso distrai da missão principal de `cp`, que é copiar um arquivo para outro, ou vários arquivos para outro diretório.

Para que servem os argumentos posicionais?

Argumentos posicionais são para aquelas informações que seu programa absolutamente e positivamente requer para ser executado.

Uma boa interface de usuário deve ter o mínimo de requisitos absolutos possível. Se seu programa requer 17 informações distintas para ser executado com sucesso, não importa muito *como* você obtém essas informações do usuário — a maioria das pessoas desistirá e irá embora antes de executar o programa com sucesso. Isso se aplica se a interface de usuário for uma linha de comando, um arquivo de configuração ou uma GUI: se você fizer tantas exigências aos seus usuários, a maioria deles simplesmente desistirá.

Em resumo, tente minimizar a quantidade de informações que os usuários são absolutamente obrigados a fornecer — use padrões sensatos sempre que possível. Claro, você também quer tornar seus programas razoavelmente flexíveis. É para isso que servem as opções. Novamente, não importa se são entradas em um arquivo de configuração, widgets no diálogo "Preferências" de uma GUI ou opções de linha de comando — quanto mais opções você implementar, mais flexível será seu programa e mais complicada se tornará sua implementação. Muita flexibilidade também tem desvantagens, é claro; muitas opções podem sobrecarregar os usuários e tornar seu código muito mais difícil de manter.

17.2.4 Tutorial

Embora *optparse* seja bastante flexível e poderoso, ele também é simples de usar na maioria dos casos. Esta seção abrange os padrões de código que são comuns a qualquer programa baseado em *optparse*.

Primeiro, você precisa importar a classe OptionParser; então, no início do programa principal, crie uma instância de OptionParser:

```
from optparse import OptionParser
...
parser = OptionParser()
```

Então você pode começar a definir opções. A sintaxe básica é:

```
parser.add_option(opt_str, ...,
                  attr=value, ...)
```

Cada opção tem uma ou mais strings de opção, como `-f` ou `--file`, e vários atributos de opção que informam ao *optparse* o que esperar e o que fazer quando encontrar essa opção na linha de comando.

Normalmente, cada opção terá uma sequência de opções curta e uma sequência de opções longa, por exemplo:

```
parser.add_option("-f", "--file", ...)
```

Você tem a liberdade de definir quantas strings de opções curtas e longas quiser (incluindo zero), desde que haja pelo menos uma string de opção no geral.

As strings de opção passadas para *OptionParser.add_option()* são efetivamente rótulos para a opção definida por essa chamada. Para resumir, frequentemente nos referiremos a *encontrar uma opção* na linha de comando; na realidade, *optparse* encontra *strings de opção* e procura opções a partir delas.

Depois que todas as suas opções estiverem definidas, instrua *optparse* para analisar a linha de comando do seu programa:

```
(options, args) = parser.parse_args()
```

(Se desejar, você pode passar uma lista de argumentos personalizada para *parse_args()*, mas isso raramente é necessário: por padrão, ele usa `sys.argv[1:]`.)

parse_args() retorna dois valores:

- `options`, um objeto que contém valores para todas as suas opções — por exemplo, se `--file` receber um único argumento de string, então `options.file` será o nome do arquivo fornecido pelo usuário, ou `None` se o usuário não forneceu essa opção
- `args`, a lista de argumentos posicionais restantes após a análise de opções

Esta seção do tutorial abrange apenas os quatro atributos de opção mais importantes: *action*, *type*, *dest* (destino) e *help*. Destes, *action* é o mais fundamental.

Compreendendo as ações de opções

Ações dizem ao *optparse* o que fazer quando ele encontra uma opção na linha de comando. Há um conjunto fixo de ações codificadas em *optparse*; adicionar novas ações é um tópico avançado abordado na seção *Estendendo optparse*. A maioria das ações diz ao *optparse* para armazenar um valor em alguma variável — por exemplo, pegue uma string da linha de comando e armazene-a em um atributo de `options`.

Se você não especificar uma ação de opção, *optparse* assumirá como padrão `store`.

A ação store

A ação de opção mais comum é `store`, que diz ao *optparse* para pegar o próximo argumento (ou o restante do argumento atual), garantir que seja do tipo correto e armazená-lo no destino escolhido.

Por exemplo:

```
parser.add_option("-f", "--file",
                  action="store", type="string", dest="filename")
```

Agora vamos criar uma linha de comando falsa e pedir ao *optparse* para analisá-la:

```
args = ["-f", "foo.txt"]
(options, args) = parser.parse_args(args)
```

Quando *optparse* vê a string de opção `-f`, ele consome o próximo argumento, `foo.txt`, e o armazena em `options.filename`. Então, após essa chamada para *parse_args()*, `options.filename` é `"foo.txt"`.

Alguns outros tipos de opção suportados por *optparse* são `int` e `float`. Aqui está uma opção que espera um argumento inteiro:

```
parser.add_option("-n", type="int", dest="num")
```

Note que esta opção não tem uma string de opção longa, o que é perfeitamente aceitável. Além disso, não há nenhuma ação explícita, já que o padrão é `store`.

Vamos analisar outra linha de comando falsa. Desta vez, vamos colocar o argumento de opção bem perto da opção: já que `-n42` (um argumento) equivale a `-n 42` (dois argumentos), o código

```
(options, args) = parser.parse_args(["-n42"])
print(options.num)
```

vai exibir `42`.

Se você não especificar um tipo, *optparse* presume `string`. Combinado com o fato de que a ação padrão é `store`, isso significa que nosso primeiro exemplo pode ser muito mais curto:

```
parser.add_option("-f", "--file", dest="filename")
```

Se você não fornecer um destino, *optparse* descobre um padrão sensato a partir das strings de opção: se a primeira string de opção longa for `--foo-bar`, então o destino padrão é `foo_bar`. Se não houver strings de opção longas, *optparse* olha para a primeira string de opção curta: o destino padrão para `-f` é `f`.

optparse também inclui o tipo `complex` embutido. Adicionar tipos é abordado na seção *Estendendo optparse*.

Manipulando opções (sinalizadores) booleanas

Opções de sinalizador — define uma variável como true ou false quando uma opção específica é vista — são bem comuns. *optparse* suporta-as com duas ações separadas, `store_true` e `store_false`. Por exemplo, você pode ter um sinalizador `verbose` que é ativado com `-v` e desativado com `-q`:

```
parser.add_option("-v", action="store_true", dest="verbose")
parser.add_option("-q", action="store_false", dest="verbose")
```

Aqui temos duas opções diferentes com o mesmo destino, o que é perfeitamente OK. (Isso significa apenas que você tem que ter um pouco de cuidado ao definir valores padrão — veja abaixo.)

Quando *optparse* encontra `-v` na linha de comando, ele define `options.verbose` como `True`; quando encontra `-q`, `options.verbose` é definido como `False`.

Outras ações

Algumas outras ações suportadas por *optparse* são:

"store_const"
: armazena um valor constante, predefinido via *Option.const*

"append"
: anexa o argumento desta opção a uma lista

"count"
: incrementa um contador em um

"callback"
: chama uma função especificada

Elas são abordadas na seção *Guia de referência* e na seção *Funções de retorno de opção*.

Valores padrão

Todos os exemplos acima envolvem a configuração de alguma variável (o "destino") quando certas opções de linha de comando são vistas. O que acontece se essas opções nunca forem vistas? Como não fornecemos nenhum padrão, todas elas são definidas como `None`. Isso geralmente é bom, mas às vezes você quer mais controle. *optparse* permite que você forneça um valor padrão para cada destino, que é atribuído antes da análise da linha de comando.

Primeiro, considere o exemplo de verbose/quiet. Se quisermos que *optparse* defina `verbose` como `True` a menos que `-q` seja visto, então podemos fazer isso:

```
parser.add_option("-v", action="store_true", dest="verbose", default=True)
parser.add_option("-q", action="store_false", dest="verbose")
```

Como os valores padrão se aplicam ao *destino* e não a qualquer opção específica, e essas duas opções têm o mesmo destino, isso é exatamente equivalente:

```
parser.add_option("-v", action="store_true", dest="verbose")
parser.add_option("-q", action="store_false", dest="verbose", default=True)
```

Considere isso:

```
parser.add_option("-v", action="store_true", dest="verbose", default=False)
parser.add_option("-q", action="store_false", dest="verbose", default=True)
```

Novamente, o valor padrão para `verbose` será `True`: o último valor padrão fornecido para qualquer destino específico é o que conta.

Uma maneira mais clara de especificar valores padrão é o método `set_defaults()` do OptionParser, que você pode chamar a qualquer momento antes de chamar *parse_args()*:

```
parser.set_defaults(verbose=True)
parser.add_option(...)
(options, args) = parser.parse_args()
```

Como antes, o último valor especificado para um destino de opção dado é o que conta. Para maior clareza, tente usar um método ou outro de configuração de valores padrão, não ambos.

Gerando texto de ajuda

A capacidade do *optparse* de gerar texto de ajuda e uso automaticamente é útil para criar interfaces de linha de comando amigáveis ao usuário. Tudo o que você precisa fazer é fornecer um valor *help* para cada opção e, opcionalmente, uma curta mensagem de uso para todo o seu programa. Aqui está um OptionParser preenchido com opções amigáveis (documentadas) ao usuário:

```
usage = "usage: %prog [options] arg1 arg2"
parser = OptionParser(usage=usage)
parser.add_option("-v", "--verbose",
                  action="store_true", dest="verbose", default=True,
                  help="make lots of noise [default]")
parser.add_option("-q", "--quiet",
                  action="store_false", dest="verbose",
                  help="be vewwy quiet (I'm hunting wabbits)")
parser.add_option("-f", "--filename",
                  metavar="FILE", help="write output to FILE")
parser.add_option("-m", "--mode",
                  default="intermediate",
                  help="interaction mode: novice, intermediate, "
                       "or expert [default: %default]")
```

Se *optparse* encontrar `-h` ou `--help` na linha de comando, ou se você apenas chamar `parser.print_help()`, ele exibe o seguinte na saída padrão:

```
Usage: <yourscript> [options] arg1 arg2

Options:
  -h, --help            show this help message and exit
  -v, --verbose         make lots of noise [default]
  -q, --quiet           be vewwy quiet (I'm hunting wabbits)
  -f FILE, --filename=FILE
                        write output to FILE
  -m MODE, --mode=MODE  interaction mode: novice, intermediate, or
                        expert [default: intermediate]
```

(Se a saída de ajuda for acionada por uma opção de ajuda, *optparse* sai após imprimir o texto de ajuda.)

Há muita coisa acontecendo aqui para ajudar *optparse* a gerar a melhor mensagem de ajuda possível:

- o script define sua própria mensagem de uso:

  ```
  usage = "usage: %prog [options] arg1 arg2"
  ```

  optparse expande `%prog` na string de uso para o nome do programa atual, ou seja, `os.path.basename(sys.argv[0])`. A string expandida é então exibida antes da ajuda detalhada da opção.

  Se você não fornecer uma string de uso, *optparse* usa um padrão simples, mas sensato: `"Usage: %prog [options]"`, o que é bom se seu script não aceita nenhum argumento posicional.

- cada opção define uma string de ajuda e não se preocupa com quebra de linha — *optparse* cuida da quebra de linhas e faz com que a saída de ajuda tenha uma boa aparência.

- opções que assumem um valor indicam esse fato em sua mensagem de ajuda gerada automaticamente, por exemplo, para a opção “mode”:

  ```
  -m MODE, --mode=MODE
  ```

  Aqui, “MODE” é chamado de metavariável: ele representa o argumento que o usuário deve fornecer para `-m`/`--mode`. Por padrão, *optparse* converte o nome da variável de destino para maiúsculas e usa isso para a metavariável. Às vezes, não é isso que você quer — por exemplo, a opção `--filename` define explicitamente `metavar="FILE"`, resultando nesta descrição de opção gerada automaticamente:

```
-f FILE, --filename=FILE
```

Isso é importante para mais do que apenas economizar espaço: o texto de ajuda escrito manualmente usa a metavariável `FILE` para dar uma pista ao usuário de que há uma conexão entre a sintaxe semiformal `-f FILE` e a descrição semântica informal como, por exemplo, "write output to FILE". Essa é uma maneira simples, mas eficaz, de tornar seu texto de ajuda muito mais claro e útil para usuários finais.

- opções que têm um valor padrão podem incluir `%default` na string de ajuda — *optparse* vai substituí-lo por *str()* do valor padrão da opção. Se uma opção não tiver um valor padrão (ou o valor padrão for `None`), `%default` expande para `none`.

Agrupando opções

Ao lidar com muitas opções, é conveniente agrupar essas opções para melhor saída de ajuda. Um *OptionParser* pode conter vários grupos de opções, cada um dos quais pode conter várias opções.

Um grupo de opções é obtido usando a classe *OptionGroup*:

class optparse.**OptionGroup**(*parser*, *title*, *description=None*)

onde

- *parser* é a instância *OptionParser* na qual o grupo será inserido
- *title* é o título do grupo
- *description*, opcional, é uma descrição longa do grupo

OptionGroup herda de `OptionContainer` (como *OptionParser*) e, portanto, o método `add_option()` pode ser usado para adicionar uma opção ao grupo.

Depois que todas as opções são declaradas, usando o método *OptionParser* `add_option_group()` o grupo é adicionado ao analisador definido anteriormente.

Continuando com o analisador definido na seção anterior, adicionar um *OptionGroup* a um analisador é fácil:

```
group = OptionGroup(parser, "Dangerous Options",
                    "Caution: use these options at your own risk.  "
                    "It is believed that some of them bite.")
group.add_option("-g", action="store_true", help="Group option.")
parser.add_option_group(group)
```

Isso resultaria na seguinte saída de texto de ajuda:

```
Usage: <yourscript> [options] arg1 arg2

Options:
  -h, --help            show this help message and exit
  -v, --verbose         make lots of noise [default]
  -q, --quiet           be vewwy quiet (I'm hunting wabbits)
  -f FILE, --filename=FILE
                        write output to FILE
  -m MODE, --mode=MODE  interaction mode: novice, intermediate, or
                        expert [default: intermediate]

  Dangerous Options:
    Caution: use these options at your own risk.  It is believed that some
    of them bite.

    -g                  Group option.
```

Um exemplo um pouco mais completo pode envolver o uso de mais de um grupo: ainda estendendo o exemplo anterior:

```
group = OptionGroup(parser, "Dangerous Options",
                    "Caution: use these options at your own risk.  "
                    "It is believed that some of them bite.")
group.add_option("-g", action="store_true", help="Group option.")
parser.add_option_group(group)

group = OptionGroup(parser, "Debug Options")
group.add_option("-d", "--debug", action="store_true",
                 help="Print debug information")
group.add_option("-s", "--sql", action="store_true",
                 help="Print all SQL statements executed")
group.add_option("-e", action="store_true", help="Print every action done")
parser.add_option_group(group)
```

que resulta na seguinte saída:

```
Usage: <yourscript> [options] arg1 arg2

Options:
  -h, --help            show this help message and exit
  -v, --verbose         make lots of noise [default]
  -q, --quiet           be vewwy quiet (I'm hunting wabbits)
  -f FILE, --filename=FILE
                        write output to FILE
  -m MODE, --mode=MODE  interaction mode: novice, intermediate, or expert
                        [default: intermediate]

  Dangerous Options:
    Caution: use these options at your own risk.  It is believed that some
    of them bite.

    -g                  Group option.

  Debug Options:
    -d, --debug         Print debug information
    -s, --sql           Print all SQL statements executed
    -e                  Print every action done
```

Outro método interessante, especialmente ao trabalhar programaticamente com grupos de opções, é:

`OptionParser.`**`get_option_group`**(*opt_str*)

Retorna o objeto *OptionGroup* ao qual a string de opção curta ou longa *opt_str* (por exemplo, `'-o'` ou `'--option'`) pertence. Se não houver tal *OptionGroup*, retorna `None`.

Exibindo uma string de versão

Similar à string breve de uso, *optparse* também pode exibir uma string de versão para seu programa. Você tem que fornecer a string como o argumento `version` para OptionParser:

```
parser = OptionParser(usage="%prog [-f] [-q]", version="%prog 1.0")
```

`%prog` é expandido assim como em `usage`. Além disso, `version` pode conter qualquer coisa que você quiser. Quando você o fornece, *optparse* adiciona automaticamente uma opção `--version` ao seu analisador. Se ele encontrar essa opção na linha de comando, ele expande sua string `version` (substituindo `%prog`), exibindo-a no stdout e sai.

Por exemplo, se o seu script for chamado `/usr/bin/foo`:

```
$ /usr/bin/foo --version
foo 1.0
```

Os dois métodos a seguir podem ser usados para exibir e obter a string `version`:

`OptionParser.`**`print_version`**`(`*file=None*`)`

Exibe a mensagem de versão para o programa atual (`self.version`) em *file* (stdout padrão). Assim como em *`print_usage()`*, qualquer ocorrência de `%prog` em `self.version` é substituída pelo nome do programa atual. Não faz nada se `self.version` estiver vazio ou indefinido.

`OptionParser.`**`get_version`**`()`

O mesmo que *`print_version()`*, mas retorna a string da versão em vez de exibi-la.

Como `optparse` trata erros

Existem duas classes amplas de erros com os quais *`optparse`* precisa se preocupar: erros do programador e erros do usuário. Erros do programador geralmente são chamadas errôneas para *`OptionParser.add_option()`*, por exemplo, strings de opção inválidas, atributos de opção desconhecidos, atributos de opção ausentes, etc. Eles são tratados da maneira usual: levanta uma exceção (seja *`optparse.OptionError`* ou *`TypeError`*) e deixe o programa travar.

Lidar com erros do usuário é muito mais importante, pois eles certamente acontecerão não importa quão estável seu código seja. *`optparse`* pode detectar automaticamente alguns erros do usuário, como argumentos de opção ruins (passando `-n 4x` onde `-n` recebe um argumento inteiro), argumentos ausentes (`-n` no final da linha de comando, onde `-n` recebe um argumento de qualquer tipo). Além disso, você pode chamar `OptionParser.error()` para sinalizar uma condição de erro definida pelo aplicativo:

```
(options, args) = parser.parse_args()
...
if options.a and options.b:
    parser.error("options -a and -b are mutually exclusive")
```

Em ambos os casos, *`optparse`* lida com o erro da mesma maneira: ele imprime a mensagem de uso do programa e uma mensagem de erro no erro padrão e sai com o status de erro 2.

Considere o primeiro exemplo acima, onde o usuário passa `4x` para uma opção que aceita um inteiro:

```
$ /usr/bin/foo -n 4x
Usage: foo [options]

foo: error: option -n: invalid integer value: '4x'
```

Ou, quando o usuário não passa nenhum valor:

```
$ /usr/bin/foo -n
Usage: foo [options]

foo: error: -n option requires an argument
```

As mensagens de erro geradas pelo *`optparse`* tomam o cuidado de sempre mencionar a opção envolvida no erro; certifique-se de fazer o mesmo ao chamar `OptionParser.error()` do código do sua aplicação.

Se o comportamento padrão de tratamento de erros do *`optparse`* não atender às suas necessidades, você precisará criar uma subclasse de OptionParser e substituir seus métodos `exit()` e/ou `error()`.

Juntando tudo

Veja como os scripts baseados no *`optparse`* geralmente se parecem:

```
from optparse import OptionParser
...
def main():
    usage = "usage: %prog [options] arg"
    parser = OptionParser(usage)
    parser.add_option("-f", "--file", dest="filename",
                      help="read data from FILENAME")
    parser.add_option("-v", "--verbose",
                      action="store_true", dest="verbose")
    parser.add_option("-q", "--quiet",
                      action="store_false", dest="verbose")
    ...
    (options, args) = parser.parse_args()
    if len(args) != 1:
        parser.error("incorrect number of arguments")
    if options.verbose:
        print("reading %s..." % options.filename)
    ...

if __name__ == "__main__":
    main()
```

17.2.5 Guia de referência

Criando o analisador sintático

O primeiro passo para usar o *optparse* é criar uma instância de OptionParser.

class optparse.**OptionParser**(...)

O construtor do OptionParser não tem argumentos obrigatórios, mas vários argumentos nomeados opcionais. Você deve sempre passá-los como argumentos nomeados, ou seja, não confie na ordem em que os argumentos são declarados.

usage (padrão: **"%prog [options]"**)
O resumo de uso para exibir quando seu programa é executado incorretamente ou com uma opção de ajuda. Quando *optparse* exibe a string de uso, ele expande `%prog` para `os.path.basename(sys.argv[0])` (ou para `prog` se você passou esse argumento nomeado). Para suprimir uma mensagem de uso, passe o valor especial `optparse.SUPPRESS_USAGE`.

option_list (padrão: **[]**)
Uma lista de objetos Option para preencher o analisador sintático. As opções em `option_list` são adicionadas após quaisquer opções em `standard_option_list` (um atributo de classe que pode ser definido por subclasses de OptionParser), mas antes de quaisquer opções de versão ou ajuda. Descontinuado; use *add_option()* após criar o analisador sintático.

option_class (padrão: **optparse.Option**)
Classe para usar ao adicionar opções ao analisador sintático em *add_option()*.

version (padrão: **None**)
Uma string de versão para exibir quando o usuário fornece uma opção de versão. Se você fornecer um valor true para `version`, *optparse* adiciona automaticamente uma opção de versão com a string de opção única `--version`. A substring `%prog` é expandida da mesma forma que para `usage`.

conflict_handler (padrão: **"error"**)
Especifica o que fazer quando opções com strings de opções conflitantes são adicionadas ao analisador sintático; consulte a seção *Conflitos entre opções*.

description (padrão: **None**)
Um parágrafo de texto que fornece uma breve visão geral do seu programa. *optparse* reformata este parágrafo para ajustá-lo à largura atual do terminal e o exibe quando o usuário solicita ajuda (depois de `usage`, mas antes da lista de opções).

formatter (padrão: um novo `IndentedHelpFormatter`)
: Uma instância de optparse.HelpFormatter que será usada para exibir um texto de ajuda. *optparse* fornece duas classes concretas para essa finalidade: IndentedHelpFormatter e TitledHelpFormatter.

add_help_option (padrão: `True`)
: Se verdadeiro, *optparse* adicionará uma opção de ajuda (com strings de opção `-h` e `--help`) ao analisador sintático.

prog
: A string a ser usada ao expandir `%prog` em `usage` e `version` em vez de `os.path.basename(sys.argv[0])`.

epilog (padrão: `None`)
: Um parágrafo de texto de ajuda para exibir após a ajuda da opção.

Preenchendo o analisador sintático

Há várias maneiras de preencher o analisador sintático com opções. A maneira preferida é usando *`OptionParser.add_option()`*, como mostrado na seção *Tutorial*. `add_option()` pode ser chamado de uma das duas maneiras:

- passar uma instância de Option (conforme retornado por `make_option()`)
- passar qualquer combinação de argumentos nomeados e posicionais que sejam aceitáveis para `make_option()` (ou seja, para o construtor de Option), e ela criará a instância de Option para você

A outra alternativa é passar uma lista de instâncias de Option pré-construídas para o construtor de OptionParser, como em:

```
option_list = [
    make_option("-f", "--filename",
                action="store", type="string", dest="filename"),
    make_option("-q", "--quiet",
                action="store_false", dest="verbose"),
    ]
parser = OptionParser(option_list=option_list)
```

(`make_option()` é uma função de fábrica para criar instâncias de Option; atualmente é um apelido para o construtor de Option. Uma versão futura de *optparse* pode dividir Option em várias classes, e `make_option()` escolherá a classe certa para instanciar. Não instancie Option diretamente.)

Definindo as opções

Cada instância de Option representa um conjunto de strings de opções de linha de comando sinônimas, por exemplo, `-f` e `--file`. Você pode especificar qualquer número de strings de opções curtas ou longas, mas deve especificar pelo menos uma string de opção geral.

A maneira canônica de criar uma instância de *`Option`* é com o método `add_option()` de *`OptionParser`*.

`OptionParser.`**`add_option`**(*option*)

`OptionParser.`**`add_option`**(**opt_str*, *attr=value*, *...*)

Para definir uma opção com apenas uma sequência de opções curta:

```
parser.add_option("-f", attr=value, ...)
```

E para definir uma opção com apenas uma longa sequência de opções:

```
parser.add_option("--foo", attr=value, ...)
```

Os argumentos nomeados definem atributos do novo objeto Option. O atributo option mais importante é *`action`*, e ele determina amplamente quais outros atributos são relevantes ou obrigatórios. Se você passar atributos option irrelevantes, ou deixar de passar os obrigatórios, *`optparse`* levanta uma exceção *`OptionError`* explicando seu erro.

A *action* de uma opção determina o que *optparse* faz quando encontra essa opção na linha de comando. As ações de opção padrão codificadas no *optparse* são:

"store"
armazena o argumento desta opção (padrão)

"store_const"
armazena um valor constante, predefinido via *Option.const*

"store_true"
armazena `True`

"store_false"
armazena `False`

"append"
anexa o argumento desta opção a uma lista

"append_const"
anexar um valor constante a uma lista, predefinido via *Option.const*

"count"
incrementa um contador em um

"callback"
chama uma função especificada

"help"
exibe uma mensagem de uso incluindo todas as opções e a documentação para elas

(Se você não fornecer uma ação, o padrão é `"store"`. Para esta ação, você também pode fornecer os atributos de opção *type* e *dest*; consulte *Ações de opção padrão*.)

Como você pode ver, a maioria das ações envolve armazenar ou atualizar um valor em algum lugar. *optparse* sempre cria um objeto especial para isso, convencionalmente chamado de `options`, que é uma instância de *optparse.Values*.

class optparse.**Values**

Um objeto que contém nomes e valores de argumentos analisados como atributos. Normalmente criado chamando ao chamar *OptionParser.parse_args()*, e pode ser substituído por uma subclasse personalizada passada para o argumento *values* de *OptionParser.parse_args()* (conforme descrito em *Análise de argumentos*).

Argumentos de opção (e vários outros valores) são armazenados como atributos deste objeto, de acordo com o atributo de opção *dest* (destino).

Por exemplo, quando você chama

```
parser.parse_args()
```

uma das primeiras coisas que *optparse* faz é criar o objeto `options`:

```
options = Values()
```

Se uma das opções neste analisador sintático for definida com

```
parser.add_option("-f", "--file", action="store", type="string", dest="filename")
```

e a linha de comando que está sendo analisada inclui qualquer um dos seguintes:

```
-ffoo
-f foo
--file=foo
--file foo
```

então *optparse*, ao ver esta opção, fará o equivalente a

```
options.filename = "foo"
```

Os atributos de opção *type* e *dest* são quase tão importantes quanto *action*, mas *action* é o único que faz sentido para *todas* as opções.

Atributos da classe Option

class optparse.**Option**

Um único argumento de linha de comando, com vários atributos passados como nomeados para o construtor. Normalmente criado com *OptionParser.add_option()* em vez de diretamente, e pode ser substituído por uma classe personalizada por meio do argumento *option_class* para *OptionParser*.

Os seguintes atributos de opção podem ser passados como argumentos nomeados para *OptionParser.add_option()*. Se você passar um atributo de opção que não seja relevante para uma opção específica, ou deixar de passar um atributo de opção obrigatório, *optparse* levanta *OptionError*.

Option.**action**

(padrão: "store")

Determina o comportamento de *optparse* quando esta opção é vista na linha de comando; as opções disponíveis estão documentadas *aqui*.

Option.**type**

(padrão: "string")

O tipo de argumento esperado por esta opção (por exemplo, "string" ou "int"); os tipos de opções disponíveis estão documentados *aqui*.

Option.**dest**

(padrão: derivado de strings de opção)

Se a ação da opção implicar escrever ou modificar um valor em algum lugar, isso informa ao *optparse* onde escrevê-lo: *dest* nomeia um atributo do objeto options que o *optparse* constrói ao analisar a linha de comando.

Option.**default**

O valor para usar para o destino desta opção se a opção não for vista na linha de comando. Veja também *OptionParser.set_defaults()*.

Option.**nargs**

(padrão: 1)

Quantos argumentos do tipo *type* devem ser consumidos quando esta opção for vista. Se > 1, *optparse* armazenará uma tupla de valores em *dest*.

Option.**const**

Para ações que armazenam um valor constante, o valor constante a ser armazenado.

Option.**choices**

Para opções do tipo "choice", a lista de strings que o usuário pode escolher.

Option.**callback**

Para opções com ação "callback", o chamável para chamar quando esta opção for vista. Veja a seção *Funções de retorno de opção* para detalhes sobre os argumentos passados para o chamável.

Option.**callback_args**

Option.**callback_kwargs**

Argumentos nomeados e posicionais adicionais para passar para callback após os quatro argumentos de retorno de chamada padrão.

Option.**help**

Texto de ajuda para exibir para esta opção ao listar todas as opções disponíveis após o usuário fornecer uma opção *help* (como `--help`). Se nenhum texto de ajuda for fornecido, a opção será listada sem texto de ajuda. Para ocultar esta opção, use o valor especial `optparse.SUPPRESS_HELP`.

Option.**metavar**

(padrão: derivado de strings de opção)

Substituto para o(s) argumento(s) de opção a ser(em) usado(s) ao exibir texto de ajuda. Veja a seção *Tutorial* para um exemplo.

Ações de opção padrão

As várias ações de opção têm requisitos e efeitos ligeiramente diferentes. A maioria das ações tem vários atributos de opção relevantes que você pode especificar para guiar o comportamento de *optparse*; algumas têm atributos obrigatórios, que você deve especificar para qualquer opção que use essa ação.

- `"store"` [relevante: *type*, *dest*, *nargs*, *choices*]

  A opção deve ser seguida por um argumento, que é convertido em um valor de acordo com *type* e armazenado em *dest*. Se *nargs* > 1, vários argumentos serão consumidos da linha de comando; todos serão convertidos de acordo com *type* e armazenados em *dest* como uma tupla. Veja a seção *Tipos de opção padrão*.

  Se *choices* for fornecido (uma lista ou tupla de strings), o tipo padrão será `"choice"`.

  Se *type* não for fornecido, o padrão será `"string"`.

  Se *dest* não for fornecido, *optparse* deriva um destino da primeira string de opção longa (por exemplo, `--foo-bar` implica `foo_bar`). Se não houver strings de opção longas, *optparse* deriva um destino da primeira string de opção curta (por exemplo, `-f` implica `f`).

  Exemplo:

```
parser.add_option("-f")
parser.add_option("-p", type="float", nargs=3, dest="point")
```

  Como ele analisa a linha de comando

```
-f foo.txt -p 1 -3.5 4 -fbar.txt
```

  optparse vai definir

```
options.f = "foo.txt"
options.point = (1.0, -3.5, 4.0)
options.f = "bar.txt"
```

- `"store_const"` [obrigatório: *const*; relevante: *dest*]

  O valor de *const* é armazenado em *dest*.

  Exemplo:

```
parser.add_option("-q", "--quiet",
                  action="store_const", const=0, dest="verbose")
parser.add_option("-v", "--verbose",
                  action="store_const", const=1, dest="verbose")
parser.add_option("--noisy",
                  action="store_const", const=2, dest="verbose")
```

  Se `--noisy` é encontrado, *optparse* vai definir

```
options.verbose = 2
```

- `"store_true"` [relevante: *dest*]

  Um caso especial de `"store_const"` que armazena `True` em *dest*.

- `"store_false"` [relevante: *dest*]

  Como `"store_true"`, mas armazenad `False`.

  Exemplo:

```
parser.add_option("--clobber", action="store_true", dest="clobber")
parser.add_option("--no-clobber", action="store_false", dest="clobber")
```

- `"append"` [relevante: *type*, *dest*, *nargs*, *choices*]

  A opção deve ser seguida por um argumento, que é anexado à lista em *dest*. Se nenhum valor padrão para *dest* for fornecido, uma lista vazia será criada automaticamente quando *optparse* encontrar esta opção pela primeira vez na linha de comando. Se *nargs* > 1, vários argumentos serão consumidos e uma tupla de comprimento *nargs* será anexada a *dest*.

  Os padrões para *type* e *dest* são os mesmos da ação `"store"`.

  Exemplo:

```
parser.add_option("-t", "--tracks", action="append", type="int")
```

  Se `-t3` for encontrado na linha de comando, *optparse* faz o equivalente a:

```
options.tracks = []
options.tracks.append(int("3"))
```

  Se, um pouco mais tarde, `--tracks=4` for encontrado, ele faz:

```
options.tracks.append(int("4"))
```

  A ação `append` chama o método `append` no valor atual da opção. Isso significa que qualquer valor padrão especificado deve ter um método `append`. Isso também significa que se o valor padrão não for vazio, os elementos padrão estarão presentes no valor analisado para a opção, com quaisquer valores da linha de comando anexados após esses valores padrão:

```
>>> parser.add_option("--files", action="append", default=['~/.mypkg/defaults
↪'])
>>> opts, args = parser.parse_args(['--files', 'overrides.mypkg'])
>>> opts.files
['~/.mypkg/defaults', 'overrides.mypkg']
```

- `"append_const"` [obrigatório: *const*; relevante: *dest*]

  Como `"store_const"`, mas o valor de *const* é anexado a *dest*; assim como em `"append"`, *dest* assume como padrão `None`, e uma lista vazia é criada automaticamente na primeira vez que a opção é encontrada.

- `"count"` [relevante: *dest*]

  Incrementa o inteiro armazenado em *dest*. Se nenhum valor padrão for fornecido, *dest* é definido como zero antes de ser incrementado pela primeira vez.

  Exemplo:

```
parser.add_option("-v", action="count", dest="verbosity")
```

  A primeira vez que `-v` é encontrado na linha de comando, *optparse* faz o equivalente a:

```
options.verbosity = 0
options.verbosity += 1
```

Cada ocorrência subsequente de `-v` resulta em

```
options.verbosity += 1
```

- `"callback"` [obrigatório: *callback*; relevante: *type*, *nargs*, *callback_args*, *callback_kwargs*]

  Chama a função especificada por *callback*, que é chamada como

  ```
  func(option, opt_str, value, parser, *args, **kwargs)
  ```

  Veja a seção *Funções de retorno de opção* para mais detalhes.

- `"help"`

  Exibe uma mensagem de ajuda completa para todas as opções no analisador sintático de opções atual. A mensagem de ajuda é construída a partir da string `usage` passada para o construtor do OptionParser e da string *help* passada para cada opção.

  Se nenhuma string de *help* for fornecida para uma opção, ela ainda será listada na mensagem de ajuda. Para omitir uma opção completamente, use o valor especial `optparse.SUPPRESS_HELP`.

  optparse adiciona automaticamente uma opção *help* a todos as instâncias de OptionParsers, então normalmente você não precisa criar uma.

  Exemplo:

  ```
  from optparse import OptionParser, SUPPRESS_HELP

  # geralmente, uma opção de ajuda é adicionada
  # automaticamente, mas que podem ser suprimida
  # usando o argumento add_help_option
  parser = OptionParser(add_help_option=False)

  parser.add_option("-h", "--help", action="help")
  parser.add_option("-v", action="store_true", dest="verbose",
                    help="Be moderately verbose")
  parser.add_option("--file", dest="filename",
                    help="Input file to read data from")
  parser.add_option("--secret", help=SUPPRESS_HELP)
  ```

  Se *optparse* encontrar `-h` ou `--help` na linha de comando, ele vai exibir algo como a seguinte mensagem de ajuda no stdout (presumindo que `sys.argv[0]` é `"foo.py"`):

  ```
  Usage: foo.py [options]

  Options:
    -h, --help        Show this help message and exit
    -v                Be moderately verbose
    --file=FILENAME   Input file to read data from
  ```

  Após exibir a mensagem de ajuda, *optparse* encerra seu processo com `sys.exit(0)`.

- `"version"`

  Exibe o número da versão fornecido à instância de OptionParser para stdout e sai. O número da versão é realmente formatado e exibido pelo método `print_version()` do OptionParser. Geralmente, é relevante somente se o argumento `version` for fornecido ao construtor de OptionParser. Assim como com as opções de *help*, você raramente criará opções `version`, já que *optparse* as adiciona automaticamente quando necessário.

Tipos de opção padrão

optparse tem cinco tipos de opções embutidos: `"string"`, `"int"`, `"choice"`, `"float"` e `"complex"`. Se você precisar adicionar novos tipos de opções, veja a seção *Estendendo optparse*.

Os argumentos para opções de string não são verificados ou convertidos de forma alguma: o texto na linha de comando é armazenado no destino (ou passado para a função de retorno) como está.

Argumentos inteiros (tipo `"int"`) são analisados da seguinte forma:

- se o número começar com `0x`, ele será analisado como um número hexadecimal
- se o número começar com `0`, ele será analisado como um número octal
- se o número começar com `0b`, ele será analisado como um número binário
- caso contrário, o número é analisado como um número decimal

A conversão é feita chamando *int()* com a base apropriada (2, 8, 10 ou 16). Se isso falhar, *optparse* também falhará, embora com uma mensagem de erro mais útil.

Os argumentos de opção `"float"` e `"complex"` são convertidos diretamente com *float()* e *complex()*, com tratamento de erros semelhante.

As opções `"choice"` são um subtipo das opções `"string"`. O atributo da opção *choices* (uma sequência de strings) define o conjunto de argumentos de opção permitidos. `optparse.check_choice()` compara os argumentos de opção fornecidos pelo usuário com esta lista mestre e levanta *OptionValueError* se uma string inválida for fornecida.

Análise de argumentos

O objetivo de criar e preencher uma instância de OptionParser é chamar seu método *parse_args()*.

`OptionParser.`**`parse_args`**(*args=None*, *values=None*)

Analisa as opções de linha de comando encontradas em *args*.

Os parâmetros de entrada são

`args`
a lista de argumentos para processar (padrão: `sys.argv[1:]`)

`values`
um objeto *Values* para armazenar argumentos de opção (padrão: uma nova instância de *Values*) – se você fornecer um objeto existente, os padrões de opção não serão inicializados nele

e o valor de retorno é um par `(options, args)` sendo

`options`
o mesmo objeto que foi passado como *values*, ou a instância `optparse.Values` criada por *optparse*

`args`
os argumentos posicionais restantes depois que todas as opções foram processadas

O uso mais comum é não fornecer nenhum argumento nomeado. Se você fornecer `values`, ele será modificado com chamadas repetidas de *setattr()* (aproximadamente uma para cada argumento de opção armazenado em um destino de opção) e retornado por *parse_args()*.

Se *parse_args()* encontrar algum erro na lista de argumentos, ele chama o método `error()` da instância de OptionParser com uma mensagem de erro apropriada para o usuário final. Isso, em última análise, encerra seu processo com um status de saída 2 (o status de saída tradicional do Unix para erros de linha de comando).

Consultando e manipulando seu analisador sintático de opções

O comportamento padrão do analisador sintático de opções pode ser ligeiramente personalizado, e você também pode dar uma olhada no seu analisador de opções e ver o que há lá. A classe OptionParser fornece vários métodos para ajudar você:

OptionParser.**disable_interspersed_args**()

Define a análise para parar na primeira não opção. Por exemplo, se `-a` e `-b` forem ambas opções simples que não aceitam argumentos, *optparse* normalmente aceita esta sintaxe:

```
prog -a arg1 -b arg2
```

e trata-a como equivalente a

```
prog -a -b arg1 arg2
```

Para desabilitar esse recurso, chame *disable_interspersed_args()*. Isso restaura a sintaxe Unix tradicional, onde a análise de opções para com o primeiro argumento não-opcional.

Use isto se você tiver um processador de comando que executa outro comando que tem opções próprias e você quer ter certeza de que essas opções não fiquem confusas. Por exemplo, cada comando pode ter um conjunto diferente de opções.

OptionParser.**enable_interspersed_args**()

Define a análise para não parar na primeira não opção, permitindo intercalar alternadores com argumentos de comando. Esse é o comportamento padrão.

OptionParser.**get_option**(*opt_str*)

Retorna a instância de Option com a string de opção *opt_str*, ou `None` se nenhuma opção tiver essa string de opção.

OptionParser.**has_option**(*opt_str*)

Retorna `True` se o objeto OptionParser tiver uma opção com string de opção *opt_str* (por exemplo, `-q` ou `--verbose`).

OptionParser.**remove_option**(*opt_str*)

Se o *OptionParser* tiver uma opção correspondente a *opt_str*, essa opção será removida. Se essa opção fornecer quaisquer outras strings de opção, todas essas strings de opção se tornarão inválidas. Se *opt_str* não ocorrer em nenhuma opção pertencente a este *OptionParser*, levanta *ValueError*.

Conflitos entre opções

Se você não tomar cuidado, é fácil definir opções com strings de opções conflitantes:

```
parser.add_option("-n", "--dry-run", ...)
...
parser.add_option("-n", "--noisy", ...)
```

(Isto é particularmente verdadeiro se você definiu sua própria subclasse de OptionParser com algumas opções padrão.)

Toda vez que você adiciona uma opção, *optparse* verifica se há conflitos com opções existentes. Se encontrar algum, ele invoca o mecanismo de tratamento de conflitos atual. Você pode definir o mecanismo de tratamento de conflitos no construtor:

```
parser = OptionParser(..., conflict_handler=handler)
```

ou com uma chamada separada:

```
parser.set_conflict_handler(handler)
```

Os manipuladores de conflitos disponíveis são:

"error" (padrão)
presume que os conflitos de opções são um erro de programação e levanta *OptionConflictError*

"resolve"
resolve conflitos de opções de forma inteligente (veja abaixo)

Como exemplo, vamos definir um *OptionParser* que resolve conflitos de forma inteligente e adiciona opções conflitantes a ele:

```
parser = OptionParser(conflict_handler="resolve")
parser.add_option("-n", "--dry-run", ..., help="do no harm")
parser.add_option("-n", "--noisy", ..., help="be noisy")
```

Neste ponto, *optparse* detecta que uma opção adicionada anteriormente já está usando a string de opção `-n`. Como `conflict_handler` é `"resolve"`, ele resolve a situação removendo `-n` da lista de strings de opção da opção anterior. Agora `--dry-run` é a única maneira do usuário ativar essa opção. Se o usuário pedir ajuda, a mensagem de ajuda refletirá isso:

```
Options:
  --dry-run     do no harm
  ...
  -n, --noisy   be noisy
```

É possível reduzir as strings de opção para uma opção adicionada anteriormente até que não haja mais nenhuma, e o usuário não tenha como invocar essa opção da linha de comando. Nesse caso, *optparse* remove essa opção completamente, então ela não aparece no texto de ajuda ou em nenhum outro lugar. Continuando com nosso objeto OptionParser existente:

```
parser.add_option("--dry-run", ..., help="new dry-run option")
```

Neste ponto, a opção original `-n`/`--dry-run` não está mais acessível, então *optparse* a remove, deixando este texto de ajuda:

```
Options:
  ...
  -n, --noisy   be noisy
  --dry-run     new dry-run option
```

Limpeza

As instâncias de OptionParser têm várias referências cíclicas. Isso não deve ser um problema para o coletor de lixo do Python, mas você pode desejar quebrar as referências cíclicas explicitamente chamando `destroy()` no seu OptionParser quando terminar de usá-lo. Isso é particularmente útil em aplicações de longa execução, onde gráficos de objetos grandes podem ser acessados do seu OptionParser.

Outros métodos

OptionParser provê vários outros métodos públicos:

`OptionParser.`**`set_usage`**(*usage*)

Defina a string de uso de acordo com as regras descritas acima para o argumento nomeado do construtor `usage`. Passar `None` define a string de uso padrão; use `optparse.SUPPRESS_USAGE` para suprimir uma mensagem de uso.

`OptionParser.`**`print_usage`**(*file=None*)

Exibe a mensagem de uso para o programa atual (`self.usage`) em *file* (stdout padrão). Qualquer ocorrência da string `%prog` em `self.usage` é substituída pelo nome do programa atual. Não faz nada se `self.usage` estiver vazio ou não definido.

`OptionParser.`**`get_usage`**()

O mesmo que *print_usage()*, mas retorna a string de uso em vez de exibi-la.

`OptionParser.`**`set_defaults`**(*dest=value*, ...)

Define valores padrão para vários destinos de opção de uma vez. Usar *set_defaults()* é a maneira preferida de definir valores padrão para opções, já que várias opções podem compartilhar o mesmo destino. Por exemplo,

se várias opções de "modo" definirem o mesmo destino, qualquer uma delas pode definir o padrão, e a última vence:

```
parser.add_option("--advanced", action="store_const",
                  dest="mode", const="advanced",
                  default="novice")    # substituída abaixo
parser.add_option("--novice", action="store_const",
                  dest="mode", const="novice",
                  default="advanced")  # substitui a configuração acima
```

Para evitar essa confusão, use *set_defaults()*:

```
parser.set_defaults(mode="advanced")
parser.add_option("--advanced", action="store_const",
                  dest="mode", const="advanced")
parser.add_option("--novice", action="store_const",
                  dest="mode", const="novice")
```

17.2.6 Funções de retorno de opção

Quando as ações e tipos embutidos de *optparse* não são o bastante para suas necessidades, você tem duas escolhas: estender *optparse* ou definir uma opção de retorno de chamada. Estender *optparse* é mais geral, mas exagero para muitos casos simples. Muitas vezes, um retorno de chamada simples é tudo o que você precisa.

Há duas etapas para definir uma opção de retorno de chamada:

- definir a opção em si usando a ação `"callback"`
- escrever o função de retorno; esta é uma função (ou método) que recebe pelo menos quatro argumentos, conforme descrito abaixo

Definindo uma opção de retorno de chamada

Como sempre, a maneira mais fácil de definir uma opção de retorno de chamada é usando o método *OptionParser.add_option()*. Além de *action*, o único atributo de opção que você deve especificar é `callback`, a função a ser chamada:

```
parser.add_option("-c", action="callback", callback=my_callback)
```

`callback` é uma função (ou outro objeto chamável), então você já deve ter definido `my_callback()` quando criar esta opção de retorno de chamada. Neste caso simples, *optparse* nem sabe se `-c` aceita argumentos, o que geralmente significa que a opção não aceita argumentos — a mera presença de `-c` na linha de comando é tudo o que ela precisa saber. Em algumas circunstâncias, no entanto, você pode querer que seu retorno de chamada consuma um número arbitrário de argumentos de linha de comando. É aqui que escrever retornos de chamada fica complicado; isso é abordado mais adiante nesta seção.

optparse sempre passa quatro argumentos particulares para seu retorno de chamada, e ele só passará argumentos adicionais se você especificá-los via *callback_args* e *callback_kwargs*. Assim, a assinatura mínima da função de retorno de chamada é:

```
def my_callback(option, opt, value, parser):
```

Os quatro argumentos para um retorno de chamada são descritos abaixo.

Há vários outros atributos de opção que você pode fornecer ao definir uma opção de retorno de chamada:

type
: tem seu significado usual: como com as ações `"store"` ou `"append"`, ele instrui *optparse* a consumir um argumento e convertê-lo para *type*. Em vez de armazenar o(s) valor(es) convertido(s) em qualquer lugar, no entanto, *optparse* os passa para sua função de retorno de chamada.

nargs
: também tem seu significado usual: se for fornecido e > 1, *optparse* consumirá argumentos *nargs*, cada um dos quais deve ser conversível para *type*. Em seguida, ele passa uma tupla de valores convertidos para seu retorno de chamada.

callback_args
: uma tupla de argumentos posicionais extras para passar para a função de retorno

callback_kwargs
: um dicionário de argumentos nomeados extras para passar para a função de retorno

Como os retornos de chamada são chamados

Todos os retornos de chamada são chamados da seguinte forma:

```
func(option, opt_str, value, parser, *args, **kwargs)
```

onde

option
: é a instância de Option que está chamando o retorno de chamada

opt_str
: é a string de opção vista na linha de comando que está acionando o retorno de chamada. (Se uma opção longa abreviada foi usada, `opt_str` será a string de opção canônica completa — por exemplo, se o usuário colocar `--foo` na linha de comando como uma abreviação para `--foobar`, então `opt_str` será `"--foobar"`.)

value
: é o argumento para esta opção visto na linha de comando. *optparse* só esperará um argumento se *type* estiver definido; o tipo de `value` será o tipo implícito pelo tipo da opção. Se *type* para esta opção for `None` (nenhum argumento esperado), então `value` será `None`. Se *nargs* > 1, `value` será uma tupla de valores do tipo apropriado.

parser
: é a instância OptionParser que controla tudo, útil principalmente porque você pode acessar alguns outros dados interessantes por meio de seus atributos de instância:

  parser.largs
  : a lista atual de argumentos restantes, ou seja, argumentos que foram consumidos, mas não são opções nem argumentos de opção. Sinta-se à vontade para modificar `parser.largs`, por exemplo, adicionando mais argumentos a ele. (Esta lista se tornará `args`, o segundo valor de retorno de *parse_args()*.)

  parser.rargs
  : a lista atual de argumentos restantes, ou seja, com `opt_str` e `value` (se aplicável) removidos, e apenas os argumentos que os seguem ainda lá. Sinta-se à vontade para modificar `parser.rargs`, por exemplo, consumindo mais argumentos.

  parser.values
  : o objeto onde os valores de opção são armazenados por padrão (uma instância de optparse.OptionValues). Isso permite que as função de retorno usem o mesmo mecanismo que o resto de *optparse* para armazenar valores de opção; você não precisa mexer com globais ou closures. Você também pode acessar ou modificar o(s) valor(es) de quaisquer opções já encontradas na linha de comando.

args
: é uma tupla de argumentos posicionais arbitrários fornecidos por meio do atributo de opção *callback_args*.

kwargs
: é um dicionário de argumentos nomeados arbitrários fornecidos via *callback_kwargs*.

Levantando erros em uma função de retorno

A função de retorno de chamada deve levantar *OptionValueError* se houver algum problema com a opção ou seu(s) argumento(s). *optparse* captura isso e encerra o programa, exibindo a mensagem de erro que você fornece para stderr. Sua mensagem deve ser clara, concisa, precisa e mencionar a opção com defeito. Caso contrário, o usuário terá dificuldade em descobrir o que fez de errado.

Exemplo de função de retorno 1: retorno de chamada trivial

Aqui está um exemplo de uma opção de retorno de chamada que não aceita argumentos e simplesmente registra que a opção foi vista:

```
def record_foo_seen(option, opt_str, value, parser):
    parser.values.saw_foo = True

parser.add_option("--foo", action="callback", callback=record_foo_seen)
```

Claro, você pode fazer isso com a ação `"store_true"`.

Exemplo de função de retorno 2: verificar a ordem das opções

Aqui está um exemplo um pouco mais interessante: registra o fato de que `-a` é visto, mas explode se ele vier depois de `-b` na linha de comando.

```
def check_order(option, opt_str, value, parser):
    if parser.values.b:
        raise OptionValueError("can't use -a after -b")
    parser.values.a = 1
...
parser.add_option("-a", action="callback", callback=check_order)
parser.add_option("-b", action="store_true", dest="b")
```

Exemplo 3 de função de retorno: verificar a ordem das opções (generalizada)

Se você quiser reutilizar essa função de retorno para várias opções semelhantes (definir um sinalizador, mas explodir se `-b` já tiver sido visto), será preciso um pouco de trabalho: a mensagem de erro e o sinalizador que ele define devem ser generalizados.

```
def check_order(option, opt_str, value, parser):
    if parser.values.b:
        raise OptionValueError("can't use %s after -b" % opt_str)
    setattr(parser.values, option.dest, 1)
...
parser.add_option("-a", action="callback", callback=check_order, dest='a')
parser.add_option("-b", action="store_true", dest="b")
parser.add_option("-c", action="callback", callback=check_order, dest='c')
```

Exemplo de função de retorno 4: verificar uma condição arbitrária

Claro, você pode colocar qualquer condição ali—você não está limitado a verificar os valores de opções já definidas. Por exemplo, se você tem opções que não devem ser chamadas quando a lua está cheia, tudo o que você tem a fazer é isto:

```
def check_moon(option, opt_str, value, parser):
    if is_moon_full():
        raise OptionValueError("%s option invalid when moon is full"
                               % opt_str)
    setattr(parser.values, option.dest, 1)
...
```

(continua na próxima página)

(continuação da página anterior)

```
parser.add_option("--foo",
                  action="callback", callback=check_moon, dest="foo")
```

(A definição de `is_moon_full()` é deixada como um exercício para o leitor.)

Exemplo de função de retorno 5: argumentos fixos

As coisas ficam um pouco mais interessantes quando você define opções de retorno de chamada que aceitam um número fixo de argumentos. Especificar que uma opção de retorno de chamada aceita argumentos é semelhante a definir uma opção `"store"` ou `"append"`: se você definir *type*, então a opção aceita um argumento que deve ser conversível para esse tipo; se você definir ainda *nargs*, então a opção aceita argumentos *nargs*.

Aqui está um exemplo que apenas emula a ação padrão `"store"`:

```
def store_value(option, opt_str, value, parser):
    setattr(parser.values, option.dest, value)
...
parser.add_option("--foo",
                  action="callback", callback=store_value,
                  type="int", nargs=3, dest="foo")
```

Note que *optparse* cuida de consumir 3 argumentos e convertê-los em números inteiros para você; tudo o que você precisa fazer é armazená-los. (Ou o que for; obviamente você não precisa de uma função de retorno de chamada para este exemplo.)

Exemplo de função de retorno 6: argumentos variáveis

As coisas ficam complicadas quando você quer que uma opção receba um número variável de argumentos. Para esse caso, você deve escrever uma função de retorno, pois *optparse* não fornece nenhuma capacidade embutida para ele. E você tem que lidar com certas complexidades da análise sintática de linha de comando Unix convencional que *optparse* normalmente lida para você. Em particular, as funções de retorno devem implementar as regras convencionais para argumentos `--` e `-` simples:

- `--` ou `-` podem ser argumentos de opção
- `--` apenas (se não for o argumento para alguma opção): interrompe o processamento da linha de comando e descarta o `--`
- `-` apenas (se não for o argumento para alguma opção): interrompe o processamento da linha de comando, mas mantém o `-` (anexa-o a `parser.largs`)

Se você quiser uma opção que aceite um número variável de argumentos, há várias questões sutis e complicadas com as quais se preocupar. A implementação exata que você escolher será baseada em quais compensações você está disposto a fazer para sua aplicação (é por isso que *optparse* não oferece suporte a esse tipo de coisa diretamente).

No entanto, aqui vai uma tentativa de função de retorno para uma opção com argumentos variáveis:

```
def vararg_callback(option, opt_str, value, parser):
    assert value is None
    value = []

    def floatable(str):
        try:
            float(str)
            return True
        except ValueError:
            return False

    for arg in parser.rargs:
        # para em --foo como opções
```

(continua na próxima página)

(continuação da página anterior)

```
        if arg[:2] == "--" and len(arg) > 2:
            break
        # para em -a, mas não em -3 ou -3.0
        if arg[:1] == "-" and len(arg) > 1 and not floatable(arg):
            break
        value.append(arg)

    del parser.rargs[:len(value)]
    setattr(parser.values, option.dest, value)

...
parser.add_option("-c", "--callback", dest="vararg_attr",
                  action="callback", callback=vararg_callback)
```

17.2.7 Estendendo `optparse`

Como os dois principais fatores de controle sobre como *optparse* interpreta as opções de linha de comando são a ação e o tipo de cada opção, a direção mais provável de extensão é adicionar novas ações e novos tipos.

Adicionando novos tipos

Para adicionar novos tipos, você precisa definir sua própria subclasse da classe *Option* de *optparse*. Esta classe tem alguns atributos que definem os tipos de *optparse*: *TYPES* e *TYPE_CHECKER*.

`Option.`**`TYPES`**

Uma tupla de nomes de tipos; na sua subclasse, basta definir uma nova tupla *TYPES* que se baseia na tupla padrão.

`Option.`**`TYPE_CHECKER`**

Um dicionário que mapeia nomes de tipos para funções de verificação de tipos. Uma função de verificação de tipos tem a seguinte assinatura:

```
def check_mytype(option, opt, value)
```

sendo `option` uma instância *Option*, `opt` é uma string de opção (por exemplo, `-f`), e `value` é a string da linha de comando que deve ser verificada e convertida para o tipo desejado. `check_mytype()` deve retornar um objeto do tipo hipotético `mytype`. O valor retornado por uma função de verificação de tipo acabará na instância OptionValues retornada por *OptionParser.parse_args()*, ou será passado para uma função de retorno como o parâmetro `value`.

Sua função de verificação de tipo deve levantar *OptionValueError* se encontrar algum problema. *OptionValueError* recebe um único argumento de string, que é passado como está para o método `error()` do *OptionParser*, que por sua vez adiciona o nome do programa e a string `"error:"` e exibe tudo no stderr antes de encerrar o processo.

Aqui está um exemplo bobo que demonstra a adição de um tipo de opção `"complex"` para analisar números complexos no estilo Python na linha de comando. (Isso é ainda mais bobo do que costumava ser, porque *optparse* 1.3 adicionou suporte embutido para números complexos, mas não importa.)

Primeiro, as importações necessárias:

```
from copy import copy
from optparse import Option, OptionValueError
```

Você precisa definir seu verificador de tipo primeiro, pois ele será referenciado mais tarde (no atributo de classe *TYPE_CHECKER* da sua subclasse de Option):

```
def check_complex(option, opt, value):
    try:
        return complex(value)
    except ValueError:
        raise OptionValueError(
            "option %s: invalid complex value: %r" % (opt, value))
```

Finalmente, a subclasse de Option:

```
class MyOption (Option):
    TYPES = Option.TYPES + ("complex",)
    TYPE_CHECKER = copy(Option.TYPE_CHECKER)
    TYPE_CHECKER["complex"] = check_complex
```

(Se não fizéssemos uma *copy()* de *Option.TYPE_CHECKER*, acabaríamos modificando o atributo *TYPE_CHECKER* da classe Option de *optparse*. Sendo Python, nada impede você de fazer isso, exceto boas maneiras e bom senso.)

Pronto! Agora você pode escrever um script que usa o novo tipo de opção como qualquer outro script baseado em *optparse*, exceto que você tem que instruir seu OptionParser a usar MyOption em vez de Option:

```
parser = OptionParser(option_class=MyOption)
parser.add_option("-c", type="complex")
```

Como alternativa, você pode criar sua própria lista de opções e passá-la para o OptionParser; se você não usar `add_option()` da maneira acima, não precisa informar ao OptionParser qual classe de opção usar:

```
option_list = [MyOption("-c", action="store", type="complex", dest="c")]
parser = OptionParser(option_list=option_list)
```

Adicionando novas ações

A adição de novas ações é um pouco mais complicado, porque você precisa entender que *optparse* tem algumas classificações para ações:

Ações "store"
: ações que resultam em *optparse* armazenando um valor em um atributo da instância OptionValues atual; essas opções exigem que um atributo *dest* seja fornecido ao construtor Option.

Ações "typed"
: Ações que pegam um valor da linha de comando e esperam que ele seja de um certo tipo; ou melhor, uma string que pode ser convertida para um certo tipo. Essas opções requerem um atributo *type* para o construtor Option.

Esses são conjuntos sobrepostos: algumas ações "store" padrão são `"store"`, `"store_const"`, `"append"` e `"count"`, enquanto as ações "typed" padrão são `"store"`, `"append"` e `"callback"`.

Ao adicionar uma ação, você precisa categorizá-la listando-a em pelo menos um dos seguintes atributos de classe de Option (todos são listas de strings):

`Option.`**ACTIONS**
: Todas as ações devem ser listadas em ACTIONS.

`Option.`**STORE_ACTIONS**
: As ações "store" também são listadas aqui.

`Option.`**TYPED_ACTIONS**
: As ações "typed" também são listadas aqui.

`Option.`**ALWAYS_TYPED_ACTIONS**
: Ações que sempre assumem um tipo (ou seja, cujas opções sempre assumem um valor) são listadas aqui adicionalmente. O único efeito disso é que *optparse* atribui o tipo padrão, `"string"`, a opções sem tipo explícito cuja ação é listada em *ALWAYS_TYPED_ACTIONS*.

Para realmente implementar sua nova ação, você deve substituir o método `take_action()` de Option e adicionar um caso que reconheça sua ação.

Por exemplo, vamos adicionar uma ação `"extend"`. Isso é semelhante à ação padrão `"append"`, mas em vez de pegar um único valor da linha de comando e anexá-lo a uma lista existente, `"extend"` pegará vários valores em uma única string delimitada por vírgulas e estenderá uma lista existente com eles. Ou seja, se `--names` for uma opção `"extend"` do tipo `"string"`, a linha de comando

```
--names=foo,bar --names blah --names ding,dong
```

resultaria em uma lista

```
["foo", "bar", "blah", "ding", "dong"]
```

Novamente, nós definimos uma subclasse de Option:

```
class MyOption(Option):

    ACTIONS = Option.ACTIONS + ("extend",)
    STORE_ACTIONS = Option.STORE_ACTIONS + ("extend",)
    TYPED_ACTIONS = Option.TYPED_ACTIONS + ("extend",)
    ALWAYS_TYPED_ACTIONS = Option.ALWAYS_TYPED_ACTIONS + ("extend",)

    def take_action(self, action, dest, opt, value, values, parser):
        if action == "extend":
            lvalue = value.split(",")
            values.ensure_value(dest, []).extend(lvalue)
        else:
            Option.take_action(
                self, action, dest, opt, value, values, parser)
```

Características de nota:

- `"extend"` espera um valor na linha de comando e armazena esse valor em algum lugar, então ele vai em *STORE_ACTIONS* e *TYPED_ACTIONS*.
- para garantir que *optparse* atribua o tipo padrão de `"string"` às ações `"extend"`, colocamos a ação `"extend"` em *ALWAYS_TYPED_ACTIONS* também.
- `MyOption.take_action()` implementa apenas esta nova ação e passa o controle de volta para `Option.take_action()` para as ações padrão *optparse*.
- `values` é uma instância da classe optparse_parser.Values, que fornece o método muito útil `ensure_value()`. `ensure_value()` é essencialmente *getattr()* com uma válvula de segurança; é chamado como

  ```
  values.ensure_value(attr, value)
  ```

  Se o atributo `attr` de `values` não existir ou for `None`, então ensure_value() primeiro o define como `value` e então retorna `value`. Isso é muito útil para ações como `"extend"`, `"append"` e `"count"`, todas as quais acumulam dados em uma variável e esperam que essa variável seja de um certo tipo (uma lista para as duas primeiras, um inteiro para a última). Usar `ensure_value()` significa que os scripts que usam sua ação não precisam se preocupar em definir um valor padrão para os destinos de opção em questão; eles podem simplesmente deixar o padrão como `None` e `ensure_value()` cuidará de acertar quando for necessário.

17.2.8 Exceções

exception optparse.**OptionError**

Levantada se uma instância de *Option* for criada com argumentos inválidos ou inconsistentes.

exception optparse.**OptionConflictError**

Levantada se opções conflitantes forem adicionadas a um instância de *OptionParser*.

exception optparse.**OptionValueError**

Levantada se um valor de opção inválido for encontrado na linha de comando.

exception optparse.**BadOptionError**

Levantada se uma opção inválida for passada na linha de comando.

exception optparse.**AmbiguousOptionError**

Levantada se uma opção ambígua for passada na linha de comando.

17.3 getpass — Entrada de senha portátil

Código-fonte: Lib/getpass.py

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

O módulo *getpass* fornece duas funções:

getpass.**getpass**(*prompt='Password: '*, *stream=None*, ***, *echo_char=None*)

Solicita uma senha do usuário sem emiti-la. O usuário é solicitado usando a string *prompt*, cujo padrão é `'Password: '`. No Unix, o prompt é escrito no objeto arquivo ou similar *stream* usando o tratador de erros de substituição, se necessário. O *stream* padrão para o terminal de controle (`/dev/tty`) ou se não estiver disponível para `sys.stderr` (este argumento é ignorado no Windows).

O argumento *echo_char* controla como a entrada do usuário é exibida durante a digitação. Se *echo_char* for `None` (padrão), a entrada permanece oculta. Caso contrário, *echo_char* deve ser um único caractere ASCII imprimível e cada caractere digitado será substituído por ela. Por exemplo, `echo_char='*'` exibirá asteriscos em vez da entrada real.

Se uma entrada sem exibição em tela não estiver disponível, getpass() recorre a exibir uma mensagem de aviso para *stream* e lê de `sys.stdin` e levantar de um *GetPassWarning*.

> **Nota**
>
> Se você chamar getpass de dentro do IDLE, a entrada pode ser feita no terminal de onde você iniciou o IDLE, e não na própria janela ociosa.

> **Nota**
>
> Em sistemas Unix, quando *echo_char* é definido, o terminal será configurado para operar em *modo não canônico*. Em particular, isso significa que atalhos de edição de linha, como `Ctrl+U`, não funcionarão e podem inserir caracteres inesperados na entrada.

Alterado na versão 3.14: Adicionado o parâmetro *echo_char* para feedback do teclado.

exception getpass.**GetPassWarning**

A subclasse *UserWarning* é levantada quando a entrada de senha pode acabar sendo exibida na tela.

getpass.**getuser**()

Retorna o “nome de login” do usuário.

Esta função verifica as variáveis de ambiente `LOGNAME`, `USER`, `LNAME` e `USERNAME`, nesta ordem, e retorna o valor da primeiro que estiver definida como uma string não vazia. Se nenhuma estiver definida, o nome de login

do banco de dados de senhas é retornado em sistemas que oferecem suporte ao módulo *pwd*, caso contrário, uma exceção *OSError* é levantada.

Em geral, esta função deve ter preferência sobre *os.getlogin()*.

Alterado na versão 3.13: Anteriormente, várias exceções além de apenas *OSError* eram levantadas.

17.4 fileinput — Itera sobre linhas de múltiplos fluxos de entrada

Código-fonte: Lib/fileinput.py

Este módulo implementa uma classe auxiliar e funções para escrever rapidamente um laço sobre uma entrada padrão ou uma lista de arquivos. Se você quiser apenas ler ou escrever um arquivo veja *open()*.

O uso típico é:

```
import fileinput
for line in fileinput.input(encoding="utf-8"):
    process(line)
```

Isto itera sobre as linhas de todos os arquivos listados em `sys.argv[1:]`, padronizando `sys.stdin` se a lista estiver vazia. Se o nome de um arquivo for `'-'`, ele também será substituído por `sys.stdin` e os argumentos opcionais *mode* e *openhook* serão ignorados. Para especificar uma lista alternativa de nomes de arquivos, passe-a como primeiro argumento para *input()*. Um único nome de arquivo também é permitido.

Todos os arquivos são abertos em modo texto por padrão, mas você pode substituir isso especificando o parâmetro *mode* na chamada para *input()* ou *FileInput*. Se ocorrer um erro de E/S durante a abertura ou leitura de um arquivo, *OSError* será levantada.

Alterado na versão 3.3: *IOError* costumava ser levantada; agora é um apelido de *OSError*.

Se `sys.stdin` for usado mais de uma vez, o segundo e posterior uso não retornará nenhuma linha, exceto talvez para uso interativo, ou se tiver sido explicitamente redefinido (por exemplo, usando `sys.stdin.seek(0)`).

Arquivos vazios são abertos e fechados imediatamente; a única vez que sua presença na lista de nomes de arquivos é perceptível é quando o último arquivo aberto está vazio.

As linhas são retornadas com novas linhas intactas, o que significa que a última linha de um arquivo pode não ter nenhuma.

Você pode controlar como os arquivos são abertos fornecendo um gancho de abertura através do parâmetro *openhook* para *fileinput.input()* ou *FileInput()*. O gancho deve ser uma função que recebe dois argumentos, *filename* e *mode*, e retorna um objeto arquivo ou similar aberto de acordo. Se *encoding* e/ou *errors* forem especificados, eles serão passados para o gancho como argumentos nomeados adicionais. Este módulo fornece um *hook_compressed()* para oferece suporte a arquivos compactados.

A seguinte função é a interface principal deste módulo:

`fileinput.`**`input`**(*files=None*, *inplace=False*, *backup=''*, ***, *mode='r'*, *openhook=None*, *encoding=None*, *errors=None*)

Cria uma instância da classe *FileInput*. A instância será usada como estado global para as funções deste módulo e também será retornada para uso durante a iteração. Os parâmetros desta função serão passados para o construtor da classe *FileInput*.

A instância *FileInput* pode ser usada como um gerenciador de contexto na instrução `with`. Neste exemplo, *input* é fechado após a saída da instrução `with`, mesmo se ocorrer uma exceção:

```
with fileinput.input(files=('spam.txt', 'eggs.txt'), encoding="utf-8") as f:
    for line in f:
        process(line)
```

Alterado na versão 3.2: Pode ser usado como gerenciador de contexto.

Alterado na versão 3.8: Os parâmetros nomeados *mode* e *openhook* agora são somente-nomeados.

Alterado na versão 3.10: Os parâmetros somente-nomeados *encoding* e *errors* foram adicionados.

As funções a seguir usam o estado global criado por `fileinput.input()`; se não houver estado ativo, `RuntimeError` será levantada.

`fileinput.`**`filename`**`()`

Retorna o nome do arquivo que está sendo lido no momento. Antes da primeira linha ser lida, retorna `None`.

`fileinput.`**`fileno`**`()`

Retorna o número inteiro de "descritor de arquivo" para o arquivo atual. Quando nenhum arquivo é aberto (antes da primeira linha e entre arquivos), retorna `-1`.

`fileinput.`**`lineno`**`()`

Retorna o número cumulativo da linha que acabou de ser lida. Antes da primeira linha ser lida, retorna `0`. Após a leitura da última linha do último arquivo, retorna o número da linha dessa linha.

`fileinput.`**`filelineno`**`()`

Retorna o número da linha no arquivo atual. Antes da primeira linha ser lida, retorna `0`. Após a leitura da última linha do último arquivo, retorna o número da linha dessa linha no arquivo.

`fileinput.`**`isfirstline`**`()`

Retorna `True` se a linha que acabou de ler for a primeira linha do seu arquivo, caso contrário retorna `False`.

`fileinput.`**`isstdin`**`()`

Retorna `True` se a última linha foi lida em `sys.stdin`, caso contrário retorna `False`.

`fileinput.`**`nextfile`**`()`

Fecha o arquivo atual para que a próxima iteração leia a primeira linha do próximo arquivo (se houver); as linhas não lidas do arquivo não contarão para a contagem cumulativa de linhas. O nome do arquivo não é alterado até que a primeira linha do próximo arquivo seja lida. Antes da leitura da primeira linha, esta função não tem efeito; ele não pode ser usado para pular o primeiro arquivo. Após a leitura da última linha do último arquivo, esta função não terá efeito.

`fileinput.`**`close`**`()`

Fecha a sequência.

A classe que implementa o comportamento de sequência fornecido pelo módulo também está disponível para subclasses:

`class` `fileinput.`**`FileInput`**`(`*files=None*, *inplace=False*, *backup=''*, ***, *mode='r'*, *openhook=None*, *encoding=None*, *errors=None*`)`

A classe `FileInput` é a implementação; seus métodos `filename()`, `fileno()`, `lineno()`, `filelineno()`, `isfirstline()`, `isstdin()`, `nextfile()` e `close()` correspondem às funções de mesmo nome no módulo. Além disso, é *iterável* e possui um método `readline()` que retorna a próxima linha de entrada. A sequência deve ser acessada em ordem estritamente sequencial; acesso aleatório e `readline()` não podem ser misturados.

Com *mode* você pode especificar qual modo de arquivo será passado para `open()`. Deve ser um entre `'r'` e `'rb'`.

O *openhook*, quando fornecido, deve ser uma função que recebe dois argumentos, *filename* e *mode*, e retorna um objeto arquivo ou similar aberto de acordo. Você não pode usar *inplace* e *openhook* juntos.

Você pode especificar *encoding* e *errors* que são passados para `open()` ou *openhook*.

Uma instância `FileInput` pode ser usada como um gerenciador de contexto na instrução `with`. Neste exemplo, *input* é fechado após a saída da instrução `with`, mesmo se ocorrer uma exceção:

```
with FileInput(files=('spam.txt', 'eggs.txt')) as input:
    process(input)
```

Alterado na versão 3.2: Pode ser usado como gerenciador de contexto.

Alterado na versão 3.8: Os parâmetros nomeados *mode* e *openhook* agora são somente-nomeados.

Alterado na versão 3.10: Os parâmetros somente-nomeados *encoding* e *errors* foram adicionados.

Alterado na versão 3.11: Os modos `'rU'` e `'U'` e o método `__getitem__()` foram removidos.

Filtragem local opcional: se o argumento nomeado `inplace=True` for passado para *fileinput.input()* ou para o construtor *FileInput*, o arquivo é movido para um arquivo de backup e a saída padrão é direcionada para o arquivo de entrada (se já existir um arquivo com o mesmo nome do arquivo de backup, ele será substituído silenciosamente). Isso torna possível escrever um filtro que reescreva seu arquivo de entrada internamente. Se o parâmetro *backup* for fornecido (normalmente como `backup='.<some extension>'`), ele especifica a extensão do arquivo de backup, e o arquivo de backup permanece disponível; por padrão, a extensão é `'.bak'` e é excluída quando o arquivo de saída é fechado. A filtragem local é desativada quando a entrada padrão é lida.

Os dois ganchos de abertura a seguir são fornecidos por este módulo:

`fileinput.`**`hook_compressed`**(*filename*, *mode*, *, *encoding=None*, *errors=None*)

Abre de forma transparente arquivos compactados com gzip e bzip2 (reconhecidos pelas extensões `'.gz'` e `'.bz2'`) usando os módulos *gzip* e *bz2*. Se a extensão do nome do arquivo não for `'.gz'` ou `'.bz2'`, o arquivo é aberto normalmente (ou seja, usando *open()* sem qualquer descompactação).

Os valores *encoding* e *errors* são passados para *io.TextIOWrapper* para arquivos compactados e abertos para arquivos normais.

Exemplo de uso: `fi = fileinput.FileInput(openhook=fileinput.hook_compressed, encoding="utf-8")`

Alterado na versão 3.10: Os parâmetros somente-nomeados *encoding* e *errors* foram adicionados.

`fileinput.`**`hook_encoded`**(*encoding*, *errors=None*)

Retorna um gancho que abre cada arquivo com *open()*, usando a *encoding* e *errors* fornecidas para ler o arquivo.

Exemplo de uso: `fi = fileinput.FileInput(openhook=fileinput.hook_encoded("utf-8", "surrogateescape"))`

Alterado na versão 3.6: Adicionado o parâmetro opcional *errors*.

Descontinuado desde a versão 3.10: Esta função foi descontinuado já que *fileinput.input()* e *FileInput* agora possuem parâmetros *encoding* e *errors*.

17.5 `curses` — Gerenciador de terminal para visualizadores de células de caracteres.

Código-fonte: Lib/curses

O módulo *curses* provê uma interface para a livraria curses, o padrão de-facto para manuseio avançado de terminal portável.

While curses is most widely used in the Unix environment, versions are available for Windows, DOS, and possibly other systems as well. This extension module is designed to match the API of ncurses, an open-source curses library hosted on Linux and the BSD variants of Unix.

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em *plataformas móveis* ou *plataformas WebAssembly*.

Nota

Whenever the documentation mentions a *character* it can be specified as an integer, a one-character Unicode string or a one-byte byte string.

Whenever the documentation mentions a *character string* it can be specified as a Unicode string or a byte string.

Ver também

Módulo ***curses.ascii***
Utilidades para trabalhar com caracteres ASCII, independentemente de suas configurações locais.

Módulo ***curses.panel***
Uma extensão de painel stackeada que adiciona profundidade às janelas do curses.

Módulo ***curses.textpad***
Widget de texto editável para suporte ao curses **Emacs**-like bindings.

curses-howto
Tutorial material on using curses with Python, by Andrew Kuchling and Eric Raymond.

17.5.1 Funções

The module *curses* defines the following exception:

exception curses.**error**
Exception raised when a curses library function returns an error.

Nota

Whenever *x* or *y* arguments to a function or a method are optional, they default to the current cursor location. Whenever *attr* is optional, it defaults to *A_NORMAL*.

The module *curses* defines the following functions:

curses.**assume_default_colors**(*fg*, *bg*, */*)
Allow use of default values for colors on terminals supporting this feature. Use this to support transparency in your application.

- Assign terminal default foreground/background colors to color number `-1`. So `init_pair(x, COLOR_RED, -1)` will initialize pair *x* as red on default background and `init_pair(x, -1, COLOR_BLUE)` will initialize pair *x* as default foreground on blue.
- Change the definition of the color-pair `0` to `(fg, bg)`.

Adicionado na versão 3.14.

curses.**baudrate**()
Return the output speed of the terminal in bits per second. On software terminal emulators it will have a fixed high value. Included for historical reasons; in former times, it was used to write output loops for time delays and occasionally to change interfaces depending on the line speed.

curses.**beep**()
Emit a short attention sound.

curses.**can_change_color**()
Return `True` or `False`, depending on whether the programmer can change the colors displayed by the terminal.

curses.**cbreak**()

Enter cbreak mode. In cbreak mode (sometimes called "rare" mode) normal tty line buffering is turned off and characters are available to be read one by one. However, unlike raw mode, special characters (interrupt, quit, suspend, and flow control) retain their effects on the tty driver and calling program. Calling first *raw()* then *cbreak()* leaves the terminal in cbreak mode.

curses.**color_content**(*color_number*)

Return the intensity of the red, green, and blue (RGB) components in the color *color_number*, which must be between `0` and `COLORS - 1`. Return a 3-tuple, containing the R,G,B values for the given color, which will be between `0` (no component) and `1000` (maximum amount of component).

curses.**color_pair**(*pair_number*)

Return the attribute value for displaying text in the specified color pair. Only the first 256 color pairs are supported. This attribute value can be combined with *A_STANDOUT*, *A_REVERSE*, and the other `A_*` attributes. *pair_number()* is the counterpart to this function.

curses.**curs_set**(*visibility*)

Set the cursor state. *visibility* can be set to `0`, `1`, or `2`, for invisible, normal, or very visible. If the terminal supports the visibility requested, return the previous cursor state; otherwise raise an exception. On many terminals, the "visible" mode is an underline cursor and the "very visible" mode is a block cursor.

curses.**def_prog_mode**()

Save the current terminal mode as the "program" mode, the mode when the running program is using curses. (Its counterpart is the "shell" mode, for when the program is not in curses.) Subsequent calls to *reset_prog_mode()* will restore this mode.

curses.**def_shell_mode**()

Save the current terminal mode as the "shell" mode, the mode when the running program is not using curses. (Its counterpart is the "program" mode, when the program is using curses capabilities.) Subsequent calls to *reset_shell_mode()* will restore this mode.

curses.**delay_output**(*ms*)

Insert an *ms* millisecond pause in output.

curses.**doupdate**()

Atualiza a tela física. A biblioteca do curses mantém duas estruturas de dados, uma representando o conteúdo da tela física atual e uma tela virtual representando o próximo estado desejado. O *doupdate()* atualiza a tela física para corresponder à tela virtual.

The virtual screen may be updated by a *noutrefresh()* call after write operations such as *addstr()* have been performed on a window. The normal *refresh()* call is simply `noutrefresh()` followed by `doupdate()`; if you have to update multiple windows, you can speed performance and perhaps reduce screen flicker by issuing `noutrefresh()` calls on all windows, followed by a single `doupdate()`.

curses.**echo**()

Enter echo mode. In echo mode, each character input is echoed to the screen as it is entered.

curses.**endwin**()

De-initialize the library, and return terminal to normal status.

curses.**erasechar**()

Return the user's current erase character as a one-byte bytes object. Under Unix operating systems this is a property of the controlling tty of the curses program, and is not set by the curses library itself.

curses.**filter**()

The *filter()* routine, if used, must be called before *initscr()* is called. The effect is that, during those calls, `LINES` is set to `1`; the capabilities `clear`, `cup`, `cud`, `cud1`, `cuu1`, `cuu`, `vpa` are disabled; and the `home` string is set to the value of `cr`. The effect is that the cursor is confined to the current line, and so are screen updates. This may be used for enabling character-at-a-time line editing without touching the rest of the screen.

curses.**flash**()

Flash the screen. That is, change it to reverse-video and then change it back in a short interval. Some people prefer such as 'visible bell' to the audible attention signal produced by *beep()*.

curses.**flushinp**()

Flush all input buffers. This throws away any typeahead that has been typed by the user and has not yet been processed by the program.

curses.**getmouse**()

After *getch()* returns *KEY_MOUSE* to signal a mouse event, this method should be called to retrieve the queued mouse event, represented as a 5-tuple `(id, x, y, z, bstate)`. *id* is an ID value used to distinguish multiple devices, and *x*, *y*, *z* are the event's coordinates. (*z* is currently unused.) *bstate* is an integer value whose bits will be set to indicate the type of event, and will be the bitwise OR of one or more of the following constants, where *n* is the button number from 1 to 5: *BUTTONn_PRESSED*, *BUTTONn_RELEASED*, *BUTTONn_CLICKED*, *BUTTONn_DOUBLE_CLICKED*, *BUTTONn_TRIPLE_CLICKED*, *BUTTON_SHIFT*, *BUTTON_CTRL*, *BUTTON_ALT*.

Alterado na versão 3.10: The `BUTTON5_*` constants are now exposed if they are provided by the underlying curses library.

curses.**getsyx**()

Return the current coordinates of the virtual screen cursor as a tuple `(y, x)`. If *leaveok* is currently `True`, then return `(-1, -1)`.

curses.**getwin**(*file*)

Read window related data stored in the file by an earlier *window.putwin()* call. The routine then creates and initializes a new window using that data, returning the new window object.

curses.**has_colors**()

Return `True` if the terminal can display colors; otherwise, return `False`.

curses.**has_extended_color_support**()

Return `True` if the module supports extended colors; otherwise, return `False`. Extended color support allows more than 256 color pairs for terminals that support more than 16 colors (e.g. xterm-256color).

Extended color support requires ncurses version 6.1 or later.

Adicionado na versão 3.10.

curses.**has_ic**()

Return `True` if the terminal has insert- and delete-character capabilities. This function is included for historical reasons only, as all modern software terminal emulators have such capabilities.

curses.**has_il**()

Return `True` if the terminal has insert- and delete-line capabilities, or can simulate them using scrolling regions. This function is included for historical reasons only, as all modern software terminal emulators have such capabilities.

curses.**has_key**(*ch*)

Take a key value *ch*, and return `True` if the current terminal type recognizes a key with that value.

curses.**halfdelay**(*tenths*)

Used for half-delay mode, which is similar to cbreak mode in that characters typed by the user are immediately available to the program. However, after blocking for *tenths* tenths of seconds, raise an exception if nothing has been typed. The value of *tenths* must be a number between `1` and `255`. Use *nocbreak()* to leave half-delay mode.

curses.**init_color**(*color_number*, *r*, *g*, *b*)

Change the definition of a color, taking the number of the color to be changed followed by three RGB values (for the amounts of red, green, and blue components). The value of *color_number* must be between `0` and `COLORS - 1`. Each of *r*, *g*, *b*, must be a value between `0` and `1000`. When *init_color()* is used, all occurrences of that color on the screen immediately change to the new definition. This function is a no-op on most terminals; it is active only if *can_change_color()* returns `True`.

curses.**init_pair**(*pair_number*, *fg*, *bg*)

Change the definition of a color-pair. It takes three arguments: the number of the color-pair to be changed, the foreground color number, and the background color number. The value of *pair_number* must be between `1` and `COLOR_PAIRS - 1` (the `0` color pair can only be changed by *use_default_colors()* and *assume_default_colors()*). The value of *fg* and *bg* arguments must be between `0` and `COLORS - 1`, or, after calling `use_default_colors()` or `assume_default_colors()`, `-1`. If the color-pair was previously initialized, the screen is refreshed and all occurrences of that color-pair are changed to the new definition.

curses.**initscr**()

Initialize the library. Return a *window* object which represents the whole screen.

> **Nota**
>
> If there is an error opening the terminal, the underlying curses library may cause the interpreter to exit.

curses.**is_term_resized**(*nlines*, *ncols*)

Return `True` if *resize_term()* would modify the window structure, `False` otherwise.

curses.**isendwin**()

Return `True` if *endwin()* has been called (that is, the curses library has been deinitialized).

curses.**keyname**(*k*)

Return the name of the key numbered *k* as a bytes object. The name of a key generating printable ASCII character is the key's character. The name of a control-key combination is a two-byte bytes object consisting of a caret (`b'^'`) followed by the corresponding printable ASCII character. The name of an alt-key combination (128–255) is a bytes object consisting of the prefix `b'M-'` followed by the name of the corresponding ASCII character.

curses.**killchar**()

Return the user's current line kill character as a one-byte bytes object. Under Unix operating systems this is a property of the controlling tty of the curses program, and is not set by the curses library itself.

curses.**longname**()

Return a bytes object containing the terminfo long name field describing the current terminal. The maximum length of a verbose description is 128 characters. It is defined only after the call to *initscr()*.

curses.**meta**(*flag*)

If *flag* is `True`, allow 8-bit characters to be input. If *flag* is `False`, allow only 7-bit chars.

curses.**mouseinterval**(*interval*)

Set the maximum time in milliseconds that can elapse between press and release events in order for them to be recognized as a click, and return the previous interval value. The default value is 200 milliseconds, or one fifth of a second.

curses.**mousemask**(*mousemask*)

Set the mouse events to be reported, and return a tuple `(availmask, oldmask)`. *availmask* indicates which of the specified mouse events can be reported; on complete failure it returns `0`. *oldmask* is the previous value of the given window's mouse event mask. If this function is never called, no mouse events are ever reported.

curses.**napms**(*ms*)

Sleep for *ms* milliseconds.

curses.**newpad**(*nlines*, *ncols*)

Create and return a pointer to a new pad data structure with the given number of lines and columns. Return a pad as a window object.

A pad is like a window, except that it is not restricted by the screen size, and is not necessarily associated with a particular part of the screen. Pads can be used when a large window is needed, and only a part of the window will be on the screen at one time. Automatic refreshes of pads (such as from scrolling or echoing of input) do not occur. The *refresh()* and *noutrefresh()* methods of a pad require 6 arguments to specify the part of

the pad to be displayed and the location on the screen to be used for the display. The arguments are *pminrow*, *pmincol*, *sminrow*, *smincol*, *smaxrow*, *smaxcol*; the *p* arguments refer to the upper left corner of the pad region to be displayed and the *s* arguments define a clipping box on the screen within which the pad region is to be displayed.

curses.**newwin**(*nlines*, *ncols*)

curses.**newwin**(*nlines*, *ncols*, *begin_y*, *begin_x*)

Return a new *window*, whose left-upper corner is at `(begin_y, begin_x)`, and whose height/width is *nlines*/*ncols*.

By default, the window will extend from the specified position to the lower right corner of the screen.

curses.**nl**()

Enter newline mode. This mode translates the return key into newline on input, and translates newline into return and line-feed on output. Newline mode is initially on.

curses.**nocbreak**()

Leave cbreak mode. Return to normal "cooked" mode with line buffering.

curses.**noecho**()

Leave echo mode. Echoing of input characters is turned off.

curses.**nonl**()

Leave newline mode. Disable translation of return into newline on input, and disable low-level translation of newline into newline/return on output (but this does not change the behavior of `addch('\n')`, which always does the equivalent of return and line feed on the virtual screen). With translation off, curses can sometimes speed up vertical motion a little; also, it will be able to detect the return key on input.

curses.**noqiflush**()

When the `noqiflush()` routine is used, normal flush of input and output queues associated with the `INTR`, `QUIT` and `SUSP` characters will not be done. You may want to call `noqiflush()` in a signal handler if you want output to continue as though the interrupt had not occurred, after the handler exits.

curses.**noraw**()

Leave raw mode. Return to normal "cooked" mode with line buffering.

curses.**pair_content**(*pair_number*)

Return a tuple `(fg, bg)` containing the colors for the requested color pair. The value of *pair_number* must be between `0` and `COLOR_PAIRS - 1`.

curses.**pair_number**(*attr*)

Return the number of the color-pair set by the attribute value *attr*. *color_pair()* is the counterpart to this function.

curses.**putp**(*str*)

Equivalent to `tputs(str, 1, putchar)`; emit the value of a specified terminfo capability for the current terminal. Note that the output of *putp()* always goes to standard output.

curses.**qiflush**([*flag*])

If *flag* is `False`, the effect is the same as calling *noqiflush()*. If *flag* is `True`, or no argument is provided, the queues will be flushed when these control characters are read.

curses.**raw**()

Enter raw mode. In raw mode, normal line buffering and processing of interrupt, quit, suspend, and flow control keys are turned off; characters are presented to curses input functions one by one.

curses.**reset_prog_mode**()

Restore the terminal to "program" mode, as previously saved by *def_prog_mode()*.

curses.**reset_shell_mode**()

Restore the terminal to "shell" mode, as previously saved by *def_shell_mode()*.

curses.**resetty**()

Restore the state of the terminal modes to what it was at the last call to *savetty()*.

curses.**resize_term**(*nlines*, *ncols*)

Backend function used by *resizeterm()*, performing most of the work; when resizing the windows, *resize_term()* blank-fills the areas that are extended. The calling application should fill in these areas with appropriate data. The `resize_term()` function attempts to resize all windows. However, due to the calling convention of pads, it is not possible to resize these without additional interaction with the application.

curses.**resizeterm**(*nlines*, *ncols*)

Resize the standard and current windows to the specified dimensions, and adjusts other bookkeeping data used by the curses library that record the window dimensions (in particular the SIGWINCH handler).

curses.**savetty**()

Save the current state of the terminal modes in a buffer, usable by *resetty()*.

curses.**get_escdelay**()

Retrieves the value set by *set_escdelay()*.

Adicionado na versão 3.9.

curses.**set_escdelay**(*ms*)

Sets the number of milliseconds to wait after reading an escape character, to distinguish between an individual escape character entered on the keyboard from escape sequences sent by cursor and function keys.

Adicionado na versão 3.9.

curses.**get_tabsize**()

Retrieves the value set by *set_tabsize()*.

Adicionado na versão 3.9.

curses.**set_tabsize**(*size*)

Sets the number of columns used by the curses library when converting a tab character to spaces as it adds the tab to a window.

Adicionado na versão 3.9.

curses.**setsyx**(*y*, *x*)

Set the virtual screen cursor to *y*, *x*. If *y* and *x* are both `-1`, then *leaveok* is set `True`.

curses.**setupterm**(*term=None*, *fd=-1*)

Initialize the terminal. *term* is a string giving the terminal name, or `None`; if omitted or `None`, the value of the `TERM` environment variable will be used. *fd* is the file descriptor to which any initialization sequences will be sent; if not supplied or `-1`, the file descriptor for `sys.stdout` will be used.

curses.**start_color**()

Must be called if the programmer wants to use colors, and before any other color manipulation routine is called. It is good practice to call this routine right after *initscr()*.

start_color() initializes eight basic colors (black, red, green, yellow, blue, magenta, cyan, and white), and two global variables in the *curses* module, *COLORS* and *COLOR_PAIRS*, containing the maximum number of colors and color-pairs the terminal can support. It also restores the colors on the terminal to the values they had when the terminal was just turned on.

curses.**termattrs**()

Return a logical OR of all video attributes supported by the terminal. This information is useful when a curses program needs complete control over the appearance of the screen.

curses.**termname**()

Return the value of the environment variable `TERM`, as a bytes object, truncated to 14 characters.

curses.**tigetflag**(*capname*)

Return the value of the Boolean capability corresponding to the terminfo capability name *capname* as an integer. Return the value `-1` if *capname* is not a Boolean capability, or `0` if it is canceled or absent from the terminal description.

curses.**tigetnum**(*capname*)

Return the value of the numeric capability corresponding to the terminfo capability name *capname* as an integer. Return the value `-2` if *capname* is not a numeric capability, or `-1` if it is canceled or absent from the terminal description.

curses.**tigetstr**(*capname*)

Return the value of the string capability corresponding to the terminfo capability name *capname* as a bytes object. Return `None` if *capname* is not a terminfo "string capability", or is canceled or absent from the terminal description.

curses.**tparm**(*str*[, ...])

Instantiate the bytes object *str* with the supplied parameters, where *str* should be a parameterized string obtained from the terminfo database. E.g. `tparm(tigetstr("cup"), 5, 3)` could result in `b'\033[6;4H'`, the exact result depending on terminal type.

curses.**typeahead**(*fd*)

Specify that the file descriptor *fd* be used for typeahead checking. If *fd* is `-1`, then no typeahead checking is done.

The curses library does "line-breakout optimization" by looking for typeahead periodically while updating the screen. If input is found, and it is coming from a tty, the current update is postponed until refresh or doupdate is called again, allowing faster response to commands typed in advance. This function allows specifying a different file descriptor for typeahead checking.

curses.**unctrl**(*ch*)

Return a bytes object which is a printable representation of the character *ch*. Control characters are represented as a caret followed by the character, for example as `b'^C'`. Printing characters are left as they are.

curses.**ungetch**(*ch*)

Push *ch* so the next *getch()* will return it.

> **Nota**
>
> Only one *ch* can be pushed before `getch()` is called.

curses.**update_lines_cols**()

Update the *LINES* and *COLS* module variables. Useful for detecting manual screen resize.

Adicionado na versão 3.5.

curses.**unget_wch**(*ch*)

Push *ch* so the next *get_wch()* will return it.

> **Nota**
>
> Only one *ch* can be pushed before `get_wch()` is called.

Adicionado na versão 3.3.

curses.**ungetmouse**(*id*, *x*, *y*, *z*, *bstate*)

Push a *KEY_MOUSE* event onto the input queue, associating the given state data with it.

curses.**use_env**(*flag*)

If used, this function should be called before `initscr()` or newterm are called. When *flag* is `False`, the values of lines and columns specified in the terminfo database will be used, even if environment variables `LINES` and `COLUMNS` (used by default) are set, or if curses is running in a window (in which case default behavior would be to use the window size if `LINES` and `COLUMNS` are not set).

curses.**use_default_colors**()

Equivalent to `assume_default_colors(-1, -1)`.

curses.**wrapper**(*func*, /, **args*, ***kwargs*)

Initialize curses and call another callable object, *func*, which should be the rest of your curses-using application. If the application raises an exception, this function will restore the terminal to a sane state before re-raising the exception and generating a traceback. The callable object *func* is then passed the main window 'stdscr' as its first argument, followed by any other arguments passed to `wrapper()`. Before calling *func*, `wrapper()` turns on cbreak mode, turns off echo, enables the terminal keypad, and initializes colors if the terminal has color support. On exit (whether normally or by exception) it restores cooked mode, turns on echo, and disables the terminal keypad.

17.5.2 Window Objects

class curses.**window**

Window objects, as returned by `initscr()` and `newwin()` above, have the following methods and attributes:

window.**addch**(*ch*[, *attr*])

window.**addch**(*y*, *x*, *ch*[, *attr*])

Paint character *ch* at `(y, x)` with attributes *attr*, overwriting any character previously painted at that location. By default, the character position and attributes are the current settings for the window object.

> **Nota**
>
> Writing outside the window, subwindow, or pad raises a `curses.error`. Attempting to write to the lower right corner of a window, subwindow, or pad will cause an exception to be raised after the character is printed.

window.**addnstr**(*str*, *n*[, *attr*])

window.**addnstr**(*y*, *x*, *str*, *n*[, *attr*])

Paint at most *n* characters of the character string *str* at `(y, x)` with attributes *attr*, overwriting anything previously on the display.

window.**addstr**(*str*[, *attr*])

window.**addstr**(*y*, *x*, *str*[, *attr*])

Paint the character string *str* at `(y, x)` with attributes *attr*, overwriting anything previously on the display.

> **Nota**
>
> - Writing outside the window, subwindow, or pad raises `curses.error`. Attempting to write to the lower right corner of a window, subwindow, or pad will cause an exception to be raised after the string is printed.
> - A bug in ncurses, the backend for this Python module, can cause SegFaults when resizing windows. This is fixed in ncurses-6.1-20190511. If you are stuck with an earlier ncurses, you can avoid triggering this if you do not call `addstr()` with a *str* that has embedded newlines. Instead, call `addstr()` separately for each line.

window.**attroff**(*attr*)

Remove attribute *attr* from the "background" set applied to all writes to the current window.

window.**attron**(*attr*)

Add attribute *attr* from the "background" set applied to all writes to the current window.

window.**attrset**(*attr*)

Set the "background" set of attributes to *attr*. This set is initially `0` (no attributes).

window.**bkgd**(*ch*[, *attr*])

Set the background property of the window to the character *ch*, with attributes *attr*. The change is then applied to every character position in that window:

- The attribute of every character in the window is changed to the new background attribute.
- Wherever the former background character appears, it is changed to the new background character.

window.**bkgdset**(*ch*[, *attr*])

Set the window's background. A window's background consists of a character and any combination of attributes. The attribute part of the background is combined (OR'ed) with all non-blank characters that are written into the window. Both the character and attribute parts of the background are combined with the blank characters. The background becomes a property of the character and moves with the character through any scrolling and insert/delete line/character operations.

window.**border**([*ls*[, *rs*[, *ts*[, *bs*[, *tl*[, *tr*[, *bl*[, *br*]]]]]]]])

Draw a border around the edges of the window. Each parameter specifies the character to use for a specific part of the border; see the table below for more details.

> **Nota**
>
> A `0` value for any parameter will cause the default character to be used for that parameter. Keyword parameters can *not* be used. The defaults are listed in this table:

Parameter	Descrição	Valor padrão
ls	Left side	*ACS_VLINE*
rs	Right side	*ACS_VLINE*
ts	Top	*ACS_HLINE*
bs	Bottom	*ACS_HLINE*
tl	Upper-left corner	*ACS_ULCORNER*
tr	Upper-right corner	*ACS_URCORNER*
bl	Bottom-left corner	*ACS_LLCORNER*
br	Bottom-right corner	*ACS_LRCORNER*

window.**box**([*vertch*, *horch*])

Similar to *border()*, but both *ls* and *rs* are *vertch* and both *ts* and *bs* are *horch*. The default corner characters are always used by this function.

window.**chgat**(*attr*)

window.**chgat**(*num*, *attr*)

window.**chgat**(*y*, *x*, *attr*)

window.**chgat**(*y*, *x*, *num*, *attr*)

Set the attributes of *num* characters at the current cursor position, or at position `(y, x)` if supplied. If *num* is not given or is `-1`, the attribute will be set on all the characters to the end of the line. This function moves cursor to position `(y, x)` if supplied. The changed line will be touched using the *touchline()* method so that the contents will be redisplayed by the next window refresh.

window.**clear**()

Like *erase()*, but also cause the whole window to be repainted upon next call to *refresh()*.

window.**clearok**(*flag*)

If *flag* is `True`, the next call to *refresh()* will clear the window completely.

window.**clrtobot**()

Erase from cursor to the end of the window: all lines below the cursor are deleted, and then the equivalent of *clrtoeol()* is performed.

window.**clrtoeol**()

Erase from cursor to the end of the line.

window.**cursyncup**()

Update the current cursor position of all the ancestors of the window to reflect the current cursor position of the window.

window.**delch**([*y*, *x*])

Delete any character at `(y, x)`.

window.**deleteln**()

Delete the line under the cursor. All following lines are moved up by one line.

window.**derwin**(*begin_y*, *begin_x*)

window.**derwin**(*nlines*, *ncols*, *begin_y*, *begin_x*)

An abbreviation for "derive window", *derwin()* is the same as calling *subwin()*, except that *begin_y* and *begin_x* are relative to the origin of the window, rather than relative to the entire screen. Return a window object for the derived window.

window.**echochar**(*ch*[, *attr*])

Add character *ch* with attribute *attr*, and immediately call *refresh()* on the window.

window.**enclose**(*y*, *x*)

Test whether the given pair of screen-relative character-cell coordinates are enclosed by the given window, returning `True` or `False`. It is useful for determining what subset of the screen windows enclose the location of a mouse event.

Alterado na versão 3.10: Previously it returned `1` or `0` instead of `True` or `False`.

window.**encoding**

Encoding used to encode method arguments (Unicode strings and characters). The encoding attribute is inherited from the parent window when a subwindow is created, for example with *window.subwin()*. By default, current locale encoding is used (see *locale.getencoding()*).

Adicionado na versão 3.3.

window.**erase**()

Clear the window.

window.**getbegyx**()

Return a tuple `(y, x)` of coordinates of upper-left corner.

window.**getbkgd**()

Return the given window's current background character/attribute pair.

window.**getch**([*y*, *x*])

Get a character. Note that the integer returned does *not* have to be in ASCII range: function keys, keypad keys and so on are represented by numbers higher than 255. In no-delay mode, return `-1` if there is no input, otherwise wait until a key is pressed.

window.**get_wch**([*y*, *x*])

Get a wide character. Return a character for most keys, or an integer for function keys, keypad keys, and other special keys. In no-delay mode, raise an exception if there is no input.

Adicionado na versão 3.3.

window.**getkey**([*y*, *x*])

Get a character, returning a string instead of an integer, as *getch()* does. Function keys, keypad keys and other special keys return a multibyte string containing the key name. In no-delay mode, raise an exception if there is no input.

window.**getmaxyx**()

Return a tuple `(y, x)` of the height and width of the window.

window.**getparyx**()

Return the beginning coordinates of this window relative to its parent window as a tuple `(y, x)`. Return `(-1, -1)` if this window has no parent.

window.**getstr**()

window.**getstr**(*n*)

window.**getstr**(*y*, *x*)

window.**getstr**(*y*, *x*, *n*)

Read a bytes object from the user, with primitive line editing capacity. The maximum value for *n* is 2047.

Alterado na versão 3.14: The maximum value for *n* was increased from 1023 to 2047.

window.**getyx**()

Return a tuple `(y, x)` of current cursor position relative to the window's upper-left corner.

window.**hline**(*ch*, *n*)

window.**hline**(*y*, *x*, *ch*, *n*)

Display a horizontal line starting at `(y, x)` with length *n* consisting of the character *ch*.

window.**idcok**(*flag*)

If *flag* is `False`, curses no longer considers using the hardware insert/delete character feature of the terminal; if *flag* is `True`, use of character insertion and deletion is enabled. When curses is first initialized, use of character insert/delete is enabled by default.

window.**idlok**(*flag*)

If *flag* is `True`, *curses* will try and use hardware line editing facilities. Otherwise, line insertion/deletion are disabled.

window.**immedok**(*flag*)

If *flag* is `True`, any change in the window image automatically causes the window to be refreshed; you no longer have to call *refresh()* yourself. However, it may degrade performance considerably, due to repeated calls to wrefresh. This option is disabled by default.

window.**inch**([*y*, *x*])

Return the character at the given position in the window. The bottom 8 bits are the character proper, and upper bits are the attributes.

window.**insch**(*ch*[, *attr*])

window.**insch**(*y*, *x*, *ch*[, *attr*])

Paint character *ch* at `(y, x)` with attributes *attr*, moving the line from position *x* right by one character.

window.**insdelln**(*nlines*)

Insert *nlines* lines into the specified window above the current line. The *nlines* bottom lines are lost. For negative *nlines*, delete *nlines* lines starting with the one under the cursor, and move the remaining lines up. The bottom *nlines* lines are cleared. The current cursor position remains the same.

window.**insertln**()

Insert a blank line under the cursor. All following lines are moved down by one line.

window.**insnstr**(*str*, *n*[, *attr*])

window.**insnstr**(*y*, *x*, *str*, *n*[, *attr*])

Insert a character string (as many characters as will fit on the line) before the character under the cursor, up to *n* characters. If *n* is zero or negative, the entire string is inserted. All characters to the right of the cursor are shifted right, with the rightmost characters on the line being lost. The cursor position does not change (after moving to *y*, *x*, if specified).

window.**insstr**(*str*[, *attr*])

window.**insstr**(*y*, *x*, *str*[, *attr*])

Insert a character string (as many characters as will fit on the line) before the character under the cursor. All characters to the right of the cursor are shifted right, with the rightmost characters on the line being lost. The cursor position does not change (after moving to *y*, *x*, if specified).

window.**instr**([*n*])

window.**instr**(*y*, *x*[, *n*])

Return a bytes object of characters, extracted from the window starting at the current cursor position, or at *y*, *x* if specified. Attributes are stripped from the characters. If *n* is specified, *instr()* returns a string at most *n* characters long (exclusive of the trailing NUL). The maximum value for *n* is 2047.

Alterado na versão 3.14: The maximum value for *n* was increased from 1023 to 2047.

window.**is_linetouched**(*line*)

Return `True` if the specified line was modified since the last call to *refresh()*; otherwise return `False`. Raise a *curses.error* exception if *line* is not valid for the given window.

window.**is_wintouched**()

Return `True` if the specified window was modified since the last call to *refresh()*; otherwise return `False`.

window.**keypad**(*flag*)

If *flag* is `True`, escape sequences generated by some keys (keypad, function keys) will be interpreted by *curses*. If *flag* is `False`, escape sequences will be left as is in the input stream.

window.**leaveok**(*flag*)

If *flag* is `True`, cursor is left where it is on update, instead of being at "cursor position." This reduces cursor movement where possible. If possible the cursor will be made invisible.

If *flag* is `False`, cursor will always be at "cursor position" after an update.

window.**move**(*new_y*, *new_x*)

Move cursor to `(new_y, new_x)`.

window.**mvderwin**(*y*, *x*)

Move the window inside its parent window. The screen-relative parameters of the window are not changed. This routine is used to display different parts of the parent window at the same physical position on the screen.

window.**mvwin**(*new_y*, *new_x*)

Move the window so its upper-left corner is at `(new_y, new_x)`.

window.**nodelay**(*flag*)

If *flag* is `True`, *getch()* will be non-blocking.

window.**notimeout**(*flag*)

If *flag* is `True`, escape sequences will not be timed out.

If *flag* is `False`, after a few milliseconds, an escape sequence will not be interpreted, and will be left in the input stream as is.

window.**noutrefresh**()

Mark for refresh but wait. This function updates the data structure representing the desired state of the window, but does not force an update of the physical screen. To accomplish that, call `doupdate()`.

window.**overlay**(*destwin*[, *sminrow*, *smincol*, *dminrow*, *dmincol*, *dmaxrow*, *dmaxcol*])

Overlay the window on top of *destwin*. The windows need not be the same size, only the overlapping region is copied. This copy is non-destructive, which means that the current background character does not overwrite the old contents of *destwin*.

To get fine-grained control over the copied region, the second form of `overlay()` can be used. *sminrow* and *smincol* are the upper-left coordinates of the source window, and the other variables mark a rectangle in the destination window.

window.**overwrite**(*destwin*[, *sminrow*, *smincol*, *dminrow*, *dmincol*, *dmaxrow*, *dmaxcol*])

Overwrite the window on top of *destwin*. The windows need not be the same size, in which case only the overlapping region is copied. This copy is destructive, which means that the current background character overwrites the old contents of *destwin*.

To get fine-grained control over the copied region, the second form of `overwrite()` can be used. *sminrow* and *smincol* are the upper-left coordinates of the source window, the other variables mark a rectangle in the destination window.

window.**putwin**(*file*)

Write all data associated with the window into the provided file object. This information can be later retrieved using the `getwin()` function.

window.**redrawln**(*beg*, *num*)

Indicate that the *num* screen lines, starting at line *beg*, are corrupted and should be completely redrawn on the next `refresh()` call.

window.**redrawwin**()

Touch the entire window, causing it to be completely redrawn on the next `refresh()` call.

window.**refresh**([*pminrow*, *pmincol*, *sminrow*, *smincol*, *smaxrow*, *smaxcol*])

Update the display immediately (sync actual screen with previous drawing/deleting methods).

The 6 optional arguments can only be specified when the window is a pad created with `newpad()`. The additional parameters are needed to indicate what part of the pad and screen are involved. *pminrow* and *pmincol* specify the upper left-hand corner of the rectangle to be displayed in the pad. *sminrow*, *smincol*, *smaxrow*, and *smaxcol* specify the edges of the rectangle to be displayed on the screen. The lower right-hand corner of the rectangle to be displayed in the pad is calculated from the screen coordinates, since the rectangles must be the same size. Both rectangles must be entirely contained within their respective structures. Negative values of *pminrow*, *pmincol*, *sminrow*, or *smincol* are treated as if they were zero.

window.**resize**(*nlines*, *ncols*)

Reallocate storage for a curses window to adjust its dimensions to the specified values. If either dimension is larger than the current values, the window's data is filled with blanks that have the current background rendition (as set by `bkgdset()`) merged into them.

window.**scroll**([*lines=1*])

Scroll the screen or scrolling region upward by *lines* lines.

window.**scrollok**(*flag*)

Control what happens when the cursor of a window is moved off the edge of the window or scrolling region, either as a result of a newline action on the bottom line, or typing the last character of the last line. If *flag* is `False`, the cursor is left on the bottom line. If *flag* is `True`, the window is scrolled up one line. Note that in order to get the physical scrolling effect on the terminal, it is also necessary to call `idlok()`.

window.**setscrreg**(*top*, *bottom*)

Set the scrolling region from line *top* to line *bottom*. All scrolling actions will take place in this region.

window.**standend**()

Turn off the standout attribute. On some terminals this has the side effect of turning off all attributes.

window.**standout**()

Turn on attribute *A_STANDOUT*.

window.**subpad**(*begin_y*, *begin_x*)

window.**subpad**(*nlines*, *ncols*, *begin_y*, *begin_x*)

Return a sub-window, whose upper-left corner is at `(begin_y, begin_x)`, and whose width/height is *ncols*/*nlines*.

window.**subwin**(*begin_y*, *begin_x*)

window.**subwin**(*nlines*, *ncols*, *begin_y*, *begin_x*)

Return a sub-window, whose upper-left corner is at `(begin_y, begin_x)`, and whose width/height is *ncols*/*nlines*.

By default, the sub-window will extend from the specified position to the lower right corner of the window.

window.**syncdown**()

Touch each location in the window that has been touched in any of its ancestor windows. This routine is called by *refresh()*, so it should almost never be necessary to call it manually.

window.**syncok**(*flag*)

If *flag* is `True`, then *syncup()* is called automatically whenever there is a change in the window.

window.**syncup**()

Touch all locations in ancestors of the window that have been changed in the window.

window.**timeout**(*delay*)

Set blocking or non-blocking read behavior for the window. If *delay* is negative, blocking read is used (which will wait indefinitely for input). If *delay* is zero, then non-blocking read is used, and *getch()* will return `-1` if no input is waiting. If *delay* is positive, then *getch()* will block for *delay* milliseconds, and return `-1` if there is still no input at the end of that time.

window.**touchline**(*start*, *count*[, *changed*])

Pretend *count* lines have been changed, starting with line *start*. If *changed* is supplied, it specifies whether the affected lines are marked as having been changed (*changed*=`True`) or unchanged (*changed*=`False`).

window.**touchwin**()

Pretend the whole window has been changed, for purposes of drawing optimizations.

window.**untouchwin**()

Mark all lines in the window as unchanged since the last call to *refresh()*.

window.**vline**(*ch*, *n*[, *attr*])

window.**vline**(*y*, *x*, *ch*, *n*[, *attr*])

Display a vertical line starting at `(y, x)` with length *n* consisting of the character *ch* with attributes *attr*.

17.5.3 Constantes

The *curses* module defines the following data members:

curses.**ERR**

Some curses routines that return an integer, such as *getch()*, return *ERR* upon failure.

curses.**OK**

Some curses routines that return an integer, such as *napms()*, return *OK* upon success.

curses.**version**

curses.**__version__**

A bytes object representing the current version of the module.

curses.**ncurses_version**

A named tuple containing the three components of the ncurses library version: *major*, *minor*, and *patch*. All values are integers. The components can also be accessed by name, so `curses.ncurses_version[0]` is equivalent to `curses.ncurses_version.major` and so on.

Availability: if the ncurses library is used.

Adicionado na versão 3.8.

curses.**COLORS**

The maximum number of colors the terminal can support. It is defined only after the call to *start_color()*.

curses.**COLOR_PAIRS**

The maximum number of color pairs the terminal can support. It is defined only after the call to *start_color()*.

curses.**COLS**

The width of the screen, i.e., the number of columns. It is defined only after the call to *initscr()*. Updated by *update_lines_cols()*, *resizeterm()* and *resize_term()*.

curses.**LINES**

The height of the screen, i.e., the number of lines. It is defined only after the call to *initscr()*. Updated by *update_lines_cols()*, *resizeterm()* and *resize_term()*.

Some constants are available to specify character cell attributes. The exact constants available are system dependent.

Atributo	Significado
curses.**A_ALTCHARSET**	Alternate character set mode
curses.**A_BLINK**	Blink mode
curses.**A_BOLD**	Bold mode
curses.**A_DIM**	Dim mode
curses.**A_INVIS**	Invisible or blank mode
curses.**A_ITALIC**	Italic mode
curses.**A_NORMAL**	Normal attribute
curses.**A_PROTECT**	Protected mode
curses.**A_REVERSE**	Reverse background and foreground colors
curses.**A_STANDOUT**	Standout mode
curses.**A_UNDERLINE**	Underline mode
curses.**A_HORIZONTAL**	Horizontal highlight
curses.**A_LEFT**	Left highlight
curses.**A_LOW**	Low highlight
curses.**A_RIGHT**	Right highlight
curses.**A_TOP**	Top highlight
curses.**A_VERTICAL**	Vertical highlight

Adicionado na versão 3.7: A_ITALIC was added.

Several constants are available to extract corresponding attributes returned by some methods.

Bit-mask	Significado
curses.**A_ATTRIBUTES**	Bit-mask to extract attributes
curses.**A_CHARTEXT**	Bit-mask to extract a character
curses.**A_COLOR**	Bit-mask to extract color-pair field information

Keys are referred to by integer constants with names starting with KEY_. The exact keycaps available are system dependent.

Key constant	Chave
curses.**KEY_MIN**	Minimum key value
curses.**KEY_BREAK**	Break key (unreliable)
curses.**KEY_DOWN**	Down-arrow
curses.**KEY_UP**	Up-arrow
curses.**KEY_LEFT**	Left-arrow
curses.**KEY_RIGHT**	Right-arrow
curses.**KEY_HOME**	Home key (upward+left arrow)
curses.**KEY_BACKSPACE**	Backspace (unreliable)
curses.**KEY_F0**	Function keys. Up to 64 function keys are supported.
curses.**KEY_Fn**	Value of function key *n*
curses.**KEY_DL**	Delete line
curses.**KEY_IL**	Insert line
curses.**KEY_DC**	Delete character

continua na próxima página

Tabela 1 – continuação da página anterior

Key constant	Chave
curses.**KEY_IC**	Insert char or enter insert mode
curses.**KEY_EIC**	Exit insert char mode
curses.**KEY_CLEAR**	Clear screen
curses.**KEY_EOS**	Clear to end of screen
curses.**KEY_EOL**	Clear to end of line
curses.**KEY_SF**	Scroll 1 line forward
curses.**KEY_SR**	Scroll 1 line backward (reverse)
curses.**KEY_NPAGE**	Next page
curses.**KEY_PPAGE**	Previous page
curses.**KEY_STAB**	Set tab
curses.**KEY_CTAB**	Clear tab
curses.**KEY_CATAB**	Clear all tabs
curses.**KEY_ENTER**	Enter or send (unreliable)
curses.**KEY_SRESET**	Soft (partial) reset (unreliable)
curses.**KEY_RESET**	Reset or hard reset (unreliable)
curses.**KEY_PRINT**	Print
curses.**KEY_LL**	Home down or bottom (lower left)
curses.**KEY_A1**	Upper left of keypad

continua na próxima página

Tabela 1 – continuação da página anterior

Key constant	Chave
curses.**KEY_A3**	Upper right of keypad
curses.**KEY_B2**	Center of keypad
curses.**KEY_C1**	Lower left of keypad
curses.**KEY_C3**	Lower right of keypad
curses.**KEY_BTAB**	Back tab
curses.**KEY_BEG**	Beg (beginning)
curses.**KEY_CANCEL**	Cancelar
curses.**KEY_CLOSE**	Close
curses.**KEY_COMMAND**	Cmd (command)
curses.**KEY_COPY**	Copy
curses.**KEY_CREATE**	Create
curses.**KEY_END**	End
curses.**KEY_EXIT**	Exit
curses.**KEY_FIND**	Find
curses.**KEY_HELP**	Help
curses.**KEY_MARK**	Mark
curses.**KEY_MESSAGE**	Message
curses.**KEY_MOVE**	Move

continua na próxima página

Tabela 1 – continuação da página anterior

Key constant	Chave
curses.**KEY_NEXT**	Next
curses.**KEY_OPEN**	Open
curses.**KEY_OPTIONS**	Opções
curses.**KEY_PREVIOUS**	Prev (previous)
curses.**KEY_REDO**	Redo
curses.**KEY_REFERENCE**	Ref (reference)
curses.**KEY_REFRESH**	Refresh
curses.**KEY_REPLACE**	Replace
curses.**KEY_RESTART**	Restart
curses.**KEY_RESUME**	Resume
curses.**KEY_SAVE**	Salvar
curses.**KEY_SBEG**	Shifted Beg (beginning)
curses.**KEY_SCANCEL**	Shifted Cancel
curses.**KEY_SCOMMAND**	Shifted Command
curses.**KEY_SCOPY**	Shifted Copy
curses.**KEY_SCREATE**	Shifted Create
curses.**KEY_SDC**	Shifted Delete char
curses.**KEY_SDL**	Shifted Delete line

continua na próxima página

Tabela 1 – continuação da página anterior

Key constant	Chave
curses.**KEY_SELECT**	Select
curses.**KEY_SEND**	Shifted End
curses.**KEY_SEOL**	Shifted Clear line
curses.**KEY_SEXIT**	Shifted Exit
curses.**KEY_SFIND**	Shifted Find
curses.**KEY_SHELP**	Shifted Help
curses.**KEY_SHOME**	Shifted Home
curses.**KEY_SIC**	Shifted Input
curses.**KEY_SLEFT**	Shifted Left arrow
curses.**KEY_SMESSAGE**	Shifted Message
curses.**KEY_SMOVE**	Shifted Move
curses.**KEY_SNEXT**	Shifted Next
curses.**KEY_SOPTIONS**	Shifted Options
curses.**KEY_SPREVIOUS**	Shifted Prev
curses.**KEY_SPRINT**	Shifted Print
curses.**KEY_SREDO**	Shifted Redo
curses.**KEY_SREPLACE**	Shifted Replace
curses.**KEY_SRIGHT**	Shifted Right arrow

continua na próxima página

Tabela 1 – continuação da página anterior

Key constant	Chave
curses.**KEY_SRSUME**	Resumo alterado
curses.**KEY_SSAVE**	Shifted Save
curses.**KEY_SSUSPEND**	Shifted Suspend
curses.**KEY_SUNDO**	Shifted Undo
curses.**KEY_SUSPEND**	Suspend
curses.**KEY_UNDO**	Desfazer
curses.**KEY_MOUSE**	Mouse event has occurred
curses.**KEY_RESIZE**	Terminal resize event
curses.**KEY_MAX**	Maximum key value

On VT100s and their software emulations, such as X terminal emulators, there are normally at least four function keys (*KEY_F1*, *KEY_F2*, *KEY_F3*, *KEY_F4*) available, and the arrow keys mapped to *KEY_UP*, *KEY_DOWN*, *KEY_LEFT* and *KEY_RIGHT* in the obvious way. If your machine has a PC keyboard, it is safe to expect arrow keys and twelve function keys (older PC keyboards may have only ten function keys); also, the following keypad mappings are standard:

Keycap	Constante
Insert	KEY_IC
Delete	KEY_DC
Home	KEY_HOME
End	KEY_END
Page Up	KEY_PPAGE
Page Down	KEY_NPAGE

The following table lists characters from the alternate character set. These are inherited from the VT100 terminal, and will generally be available on software emulations such as X terminals. When there is no graphic available, curses falls back on a crude printable ASCII approximation.

Nota

These are available only after *initscr()* has been called.

Código ACS	Significado
curses.**ACS_BBSS**	alternate name for upper right corner
curses.**ACS_BLOCK**	solid square block
curses.**ACS_BOARD**	board of squares
curses.**ACS_BSBS**	alternate name for horizontal line
curses.**ACS_BSSB**	alternate name for upper left corner
curses.**ACS_BSSS**	alternate name for top tee
curses.**ACS_BTEE**	bottom tee
curses.**ACS_BULLET**	bullet
curses.**ACS_CKBOARD**	checker board (stipple)
curses.**ACS_DARROW**	arrow pointing down
curses.**ACS_DEGREE**	degree symbol
curses.**ACS_DIAMOND**	diamond
curses.**ACS_GEQUAL**	greater-than-or-equal-to
curses.**ACS_HLINE**	horizontal line
curses.**ACS_LANTERN**	lantern symbol
curses.**ACS_LARROW**	left arrow
curses.**ACS_LEQUAL**	less-than-or-equal-to
curses.**ACS_LLCORNER**	lower left-hand corner

continua na próxima página

Tabela 2 – continuação da página anterior

Código ACS	Significado
curses.**ACS_LRCORNER**	lower right-hand corner
curses.**ACS_LTEE**	left tee
curses.**ACS_NEQUAL**	not-equal sign
curses.**ACS_PI**	letter pi
curses.**ACS_PLMINUS**	plus-or-minus sign
curses.**ACS_PLUS**	big plus sign
curses.**ACS_RARROW**	right arrow
curses.**ACS_RTEE**	right tee
curses.**ACS_S1**	scan line 1
curses.**ACS_S3**	scan line 3
curses.**ACS_S7**	scan line 7
curses.**ACS_S9**	scan line 9
curses.**ACS_SBBS**	alternate name for lower right corner
curses.**ACS_SBSB**	alternate name for vertical line
curses.**ACS_SBSS**	alternate name for right tee
curses.**ACS_SSBB**	alternate name for lower left corner
curses.**ACS_SSBS**	alternate name for bottom tee
curses.**ACS_SSSB**	alternate name for left tee

continua na próxima página

Tabela 2 – continuação da página anterior

Código ACS	Significado
curses.**ACS_SSSS**	alternate name for crossover or big plus
curses.**ACS_STERLING**	pound sterling
curses.**ACS_TTEE**	top tee
curses.**ACS_UARROW**	up arrow
curses.**ACS_ULCORNER**	upper left corner
curses.**ACS_URCORNER**	upper right corner
curses.**ACS_VLINE**	vertical line

The following table lists mouse button constants used by *getmouse()*:

Mouse button constant	Significado
curses.**BUTTONn_PRESSED**	Mouse button *n* pressed
curses.**BUTTONn_RELEASED**	Mouse button *n* released
curses.**BUTTONn_CLICKED**	Mouse button *n* clicked
curses.**BUTTONn_DOUBLE_CLICKED**	Mouse button *n* double clicked
curses.**BUTTONn_TRIPLE_CLICKED**	Mouse button *n* triple clicked
curses.**BUTTON_SHIFT**	Shift was down during button state change
curses.**BUTTON_CTRL**	Control was down during button state change
curses.**BUTTON_ALT**	Control was down during button state change

Alterado na versão 3.10: The `BUTTON5_*` constants are now exposed if they are provided by the underlying curses library.

The following table lists the predefined colors:

Constante	Color
curses.**COLOR_BLACK**	Black
curses.**COLOR_BLUE**	Blue
curses.**COLOR_CYAN**	Cyan (light greenish blue)
curses.**COLOR_GREEN**	Green
curses.**COLOR_MAGENTA**	Magenta (purplish red)
curses.**COLOR_RED**	Red
curses.**COLOR_WHITE**	White
curses.**COLOR_YELLOW**	Yellow

17.6 curses.textpad — Text input widget for curses programs

The *curses.textpad* module provides a *Textbox* class that handles elementary text editing in a curses window, supporting a set of keybindings resembling those of Emacs (thus, also of Netscape Navigator, BBedit 6.x, FrameMaker, and many other programs). The module also provides a rectangle-drawing function useful for framing text boxes or for other purposes.

The module *curses.textpad* defines the following function:

curses.textpad.**rectangle**(*win*, *uly*, *ulx*, *lry*, *lrx*)

Draw a rectangle. The first argument must be a window object; the remaining arguments are coordinates relative to that window. The second and third arguments are the y and x coordinates of the upper left hand corner of the rectangle to be drawn; the fourth and fifth arguments are the y and x coordinates of the lower right hand corner. The rectangle will be drawn using VT100/IBM PC forms characters on terminals that make this possible (including xterm and most other software terminal emulators). Otherwise it will be drawn with ASCII dashes, vertical bars, and plus signs.

17.6.1 Textbox objects

You can instantiate a *Textbox* object as follows:

class curses.textpad.**Textbox**(*win*)

Return a textbox widget object. The *win* argument should be a curses *window* object in which the textbox is to be contained. The edit cursor of the textbox is initially located at the upper left hand corner of the containing window, with coordinates `(0, 0)`. The instance's *stripspaces* flag is initially on.

Textbox objects have the following methods:

edit([*validator*])

This is the entry point you will normally use. It accepts editing keystrokes until one of the termination keystrokes is entered. If *validator* is supplied, it must be a function. It will be called for each keystroke entered with the keystroke as a parameter; command dispatch is done on the result. This method

returns the window contents as a string; whether blanks in the window are included is affected by the *stripspaces* attribute.

do_command(*ch*)

Process a single command keystroke. Here are the supported special keystrokes:

Keystroke	Ação
Control-A	Go to left edge of window.
Control-B	Cursor left, wrapping to previous line if appropriate.
Control-D	Delete character under cursor.
Control-E	Go to right edge (stripspaces off) or end of line (stripspaces on).
Control-F	Cursor right, wrapping to next line when appropriate.
Control-G	Terminate, returning the window contents.
Control-H	Delete character backward.
Control-J	Terminate if the window is 1 line, otherwise insert newline.
Control-K	If line is blank, delete it, otherwise clear to end of line.
Control-L	Refresh screen.
Control-N	Cursor down; move down one line.
Control-O	Insert a blank line at cursor location.
Control-P	Cursor up; move up one line.

Move operations do nothing if the cursor is at an edge where the movement is not possible. The following synonyms are supported where possible:

Constante	Keystroke
KEY_LEFT	Control-B
KEY_RIGHT	Control-F
KEY_UP	Control-P
KEY_DOWN	Control-N
KEY_BACKSPACE	Control-h

All other keystrokes are treated as a command to insert the given character and move right (with line wrapping).

gather()

Return the window contents as a string; whether blanks in the window are included is affected by the *stripspaces* member.

stripspaces

This attribute is a flag which controls the interpretation of blanks in the window. When it is on, trailing blanks on each line are ignored; any cursor motion that would land the cursor on a trailing blank goes to the end of that line instead, and trailing blanks are stripped when the window contents are gathered.

17.7 curses.ascii — Utilitários para caracteres ASCII

Código-fonte: Lib/curses/ascii.py

O módulo *curses.ascii* fornece constantes de nome para caracteres ASCII e funções para testar associação em várias classes de caracteres ASCII. As constantes fornecidas são nomes para caracteres de controle, como detalhado a seguir:

Nome	Significado
curses.ascii.**NUL**	
curses.ascii.**SOH**	Início do cabeçalho, interrupção do console
curses.ascii.**STX**	Início de texto
curses.ascii.**ETX**	Fim de texto
curses.ascii.**EOT**	Fim de transmissão
curses.ascii.**ENQ**	Consulta, segue com controle de fluxo *ACK*
curses.ascii.**ACK**	Confirmação
curses.ascii.**BEL**	Campainha
curses.ascii.**BS**	Backspace
curses.ascii.**TAB**	Tabulação
curses.ascii.**HT**	Apelido para *TAB*: “Tabulação horizontal”
curses.ascii.**LF**	Alimentação de linha ou *line feed*
curses.ascii.**NL**	Apelido para *LF*: “Nova linha”
curses.ascii.**VT**	Tabulação vertical
curses.ascii.**FF**	Alimentação de formulário ou *form feed*
curses.ascii.**CR**	Retorno de carro ou *carriage return*
curses.ascii.**SO**	Deslocamento para fora ou *shift-out*, inicia um conjunto de caracteres alternativo
curses.ascii.**SI**	Deslocamento para dentro ou *shift-in*, retoma o conjunto de caracteres padrão

continua na próxima página

Tabela 3 – continuação da página anterior

Nome	Significado
curses.ascii.**DLE**	Escape de conexão ou *data-link escape*
curses.ascii.**DC1**	XON, para controle de fluxo
curses.ascii.**DC2**	Controle de dispositivo 2, control de fluxo em modo bloco
curses.ascii.**DC3**	XOFF, para controle de fluxo
curses.ascii.**DC4**	Controle de dispositivo 4
curses.ascii.**NAK**	Confirmação negativa
curses.ascii.**SYN**	Estado ocioso síncrono
curses.ascii.**ETB**	Bloco de fim de transmissão
curses.ascii.**CAN**	Cancelar
curses.ascii.**EM**	Fim de mídia
curses.ascii.**SUB**	Substituir
curses.ascii.**ESC**	Escapar
curses.ascii.**FS**	Separador de arquivos
curses.ascii.**GS**	Separador de grupos
curses.ascii.**RS**	Separador de registros, terminador de modo bloco
curses.ascii.**US**	Separador de unidades
curses.ascii.**SP**	Espaço
curses.ascii.**DEL**	Excluir

Note que muitos deles têm pouca significância prática no uso moderno. Os mnemônicos derivam de convenções de

teleimpressoras que antecedem os computadores digitais.

O módulo fornece as seguintes funções, baseadas nas da biblioteca C padrão:

curses.ascii.**isalnum**(*c*)

Verifica se há um caractere alfanumérico ASCII; equivale a `isalpha(c) or isdigit(c)`.

curses.ascii.**isalpha**(*c*)

Verifica se há um caractere alfabético ASCII; equivale a `isupper(c) or islower(c)`.

curses.ascii.**isascii**(*c*)

Verifica se há um valor de caractere que se encaixa no conjunto ASCII de 7 bits.

curses.ascii.**isblank**(*c*)

Verifica se há um caractere de espaço em branco ASCII; espaço ou tabulação horizontal.

curses.ascii.**iscntrl**(*c*)

Verifica se há um caractere de controle ASCII (no intervalo de 0x00 a 0x1f ou 0x7f).

curses.ascii.**isdigit**(*c*)

Verifica se há um dígito decimal ASCII, `'0'` a `'9'`. Isso equivale a `c in string.digits`.

curses.ascii.**isgraph**(*c*)

Verifica se há algum caractere ASCII imprimível, exceto espaço.

curses.ascii.**islower**(*c*)

Verifica se há um caractere ASCII minúsculo.

curses.ascii.**isprint**(*c*)

Verifica se há algum caractere ASCII imprimível, incluindo espaço.

curses.ascii.**ispunct**(*c*)

Verifica se há algum caractere ASCII imprimível que não seja um espaço ou um caractere alfanumérico.

curses.ascii.**isspace**(*c*)

Verifica caracteres de espaço em branco ASCII; espaço, quebra de linha, retorno de carro, quebra de página, tabulação horizontal, tabulação vertical.

curses.ascii.**isupper**(*c*)

Verifica se há uma letra maiúscula ASCII.

curses.ascii.**isxdigit**(*c*)

Verifica se há um dígito hexadecimal ASCII. Isso equivale a `c in string.hexdigits`.

curses.ascii.**isctrl**(*c*)

Verifica se há um caractere de controle ASCII (valores ordinais de 0 a 31).

curses.ascii.**ismeta**(*c*)

Verifica se há um caractere não ASCII (valores ordinais 0x80 e acima).

Essas funções aceitam números inteiros ou strings de caracteres únicos; quando o argumento é uma string, ele é primeiro convertido usando a função embutida *ord()*.

Observe que todas essas funções verificam valores de bits ordinais derivados do caractere da string que você passa; elas não sabem nada sobre a codificação de caracteres da máquina host.

As duas funções a seguir aceitam uma string de caractere único ou um valor de byte inteiro; elas retornam um valor do mesmo tipo.

curses.ascii.**ascii**(*c*)

Retorna o valor ASCII correspondente aos 7 bits mais baixos de *c*.

curses.ascii.**ctrl**(*c*)

Retorna o caractere de controle correspondente ao caractere fornecido (é feito um E bit a bit com 0x1f sobre o valor do bit do caractere).

curses.ascii.**alt**(*c*)

Retorna o caractere de 8 vits correspondente ao caractere ASCII fornecido (é feito um OU bit a bit com 0x80 sobre o valor do bit do caractere).

A função a seguir aceita uma string de um único caractere ou um valor inteiro; ela retorna uma string.

curses.ascii.**unctrl**(*c*)

Retorna uma representação de string do caractere ASCII *c*. Se *c* for imprimível, essa string é o próprio caractere. Se o caractere for um caractere de controle (0x00–0x1f), a string consiste em um acento circunflexo (`'^'`) seguido pela letra maiúscula correspondente. Se o caractere for de exclusão ASCII (0x7f), a string é `'^?'`. Se o caractere tiver seu metabit (0x80) definido, o metabit é removido, as regras precedentes são aplicadas e `'!'` é adicionado ao resultado.

curses.ascii.**controlnames**

Um vetor de strings de 33 elementos que contém os mnemônicos ASCII para os trinta e dois caracteres de controle ASCII de 0 (NUL) a 0x1f (US), em ordem, mais o mnemônico `SP` para o caractere de espaço.

17.8 `curses.panel` — Uma extensão de pilha de painéis para o curses

Painéis são janelas com o recurso adicional de profundidade, então eles podem ser empilhados uns sobre os outros, e somente as partes visíveis de cada janela serão exibidas. Painéis podem ser adicionados, movidos para cima ou para baixo na pilha, e removidos.

17.8.1 Funções

O módulo *curses.panel* define as seguintes funções:

curses.panel.**bottom_panel**()

Retorna o painel inferior da pilha de painéis.

curses.panel.**new_panel**(*win*)

Retorna um objeto painel, associando-o à janela fornecida *win*. Esteja ciente de que você precisa manter o objeto painel retornado referenciado explicitamente. Se não fizer isso, o objeto painel será coletado como lixo e removido da pilha de painéis.

curses.panel.**top_panel**()

Retorna o painel superior da pilha de painéis.

curses.panel.**update_panels**()

Atualiza a tela virtual após alterações na pilha de painéis. Isso não chama *curses.doupdate()*, então você terá que fazer isso sozinho.

17.8.2 Objetos painel

Objetos painel, como retornados por *new_panel()* acima, são janelas com uma ordem de empilhamento. Há sempre uma janela associada a um painel que determina o conteúdo, enquanto os métodos de painel são responsáveis pela profundidade da janela na pilha de painéis.

Objetos painel possuem os seguintes métodos:

Panel.**above**()

Retorna o painel acima do painel atual.

Panel.**below**()

Retorna o painel abaixo do painel atual.

Panel.**bottom**()

Insere o painel para o fundo da pilha.

Panel.**hidden**()

Retorna `True` se o painel estiver oculto (não visível), `False` caso contrário.

Panel.**hide**()

Oculta o painel. Isso não exclui o objeto, apenas torna a janela invisível na tela.

Panel.**move**(*y*, *x*)

Move o painel para as coordenadas `(y, x)` da tela.

Panel.**replace**(*win*)

Altera a janela associada ao painel para a janela *win*.

Panel.**set_userptr**(*obj*)

Define o ponteiro do usuário do painel para *obj*. Isso é usado para associar um pedaço arbitrário de dados ao painel, e pode ser qualquer objeto Python.

Panel.**show**()

Exibe o painel (que pode estar oculto).

Panel.**top**()

Insere o painel para o topo da pilha.

Panel.**userptr**()

Retorna o ponteiro do usuário para o painel. Pode ser qualquer objeto Python.

Panel.**window**()

Retorna o objeto de janela associado ao painel.

17.9 `cmd` — Suporte para interpretadores de comando orientado a linhas

Código-fonte: Lib/cmd.py

A classe *Cmd* fornece um framework simples para escrever interpretadores de comando orientados a linhas. Eles são frequentemente úteis para melhorar testes, ferramentas administrativas e protótipos que mais tarde serão encapsulados em uma interface mais sofisticada.

class cmd.**Cmd**(*completekey='tab'*, *stdin=None*, *stdout=None*)

Uma instância de *Cmd* ou instância de sua subclasse é um framework de interpretador orientado a linhas. Não há um bom motivo para instanciar *Cmd* em si; em vez disso, ele é útil como uma superclasse de uma classe de interpretador que você mesmo define para herdar os métodos de *Cmd* e encapsular métodos de ação.

O argumento opcional *completekey* é o nome *readline* de uma chave de conclusão; o padrão é `Tab`. Se *completekey* não for *None* e *readline* estiver disponível, a conclusão do comando será feita automaticamente.

O padrão, `'tab'`, é tratado especialmente, de modo que se refere à tecla `Tab` em cada *readline.backend*. Especificamente, se *readline.backend* for `editline`, `Cmd` usará `'^I'` em vez de `'tab'`. Observe que outros valores não são tratados dessa forma e podem funcionar apenas com um backend específico.

Os argumentos opcionais *stdin* e *stdout* especificam os objetos arquivo de entrada e saída que a instância Cmd ou instância de sua subclasse usará para entrada e saída. Se não forem especificados, eles serão padronizados para *sys.stdin* e *sys.stdout*.

Se você quiser que um determinado *stdin* seja usado, certifique-se de definir o atributo *use_rawinput* da instância como `False`, caso contrário, *stdin* será ignorado.

Alterado na versão 3.13: `completekey='tab'` é substituído por `'^I'` para `editline`.

17.9.1 Objetos Cmd

Uma instância *Cmd* tem os seguintes métodos:

`Cmd.`**`cmdloop`**(*intro=None*)

Emite um prompt repetidamente, aceite a entrada, analise um prefixo inicial da entrada recebida e despache para métodos de ação, passando a eles o restante da linha como argumento.

O argumento opcional é um banner ou uma string de introdução a ser emitida antes do primeiro prompt (isso substitui o atributo de classe *intro*).

Se o módulo *readline* for carregado, a entrada vai herdar automaticamente a edição de lista de histórico semelhante a **bash** (por exemplo, `Control-P` retorna ao último comando, `Control-N` avança para o próximo, `Control-F` move o cursor para a direita de forma não destrutiva, `Control-B` move o cursor para a esquerda de forma não destrutiva, etc.).

Um fim de arquivo na entrada é retornado como a string `'EOF'`.

Uma instância de interpretador reconhecerá um comando chamado `foo` se e somente se ele tiver um método `do_foo()`. Como um caso especial, uma linha começando com o caractere `'?'` é despachada para o método *do_help()*. Como outro caso especial, uma linha começando com o caractere `'!'` é despachada para o método `do_shell()` (se tal método for definido).

Este método retornará quando o método *postcmd()* retornar um valor true. O argumento *stop* para *postcmd()* é o valor de retorno do método `do_*()` correspondente do comando.

Se o autocompletamento estiver habilitado, os comandos serão completados automaticamente, e o autocompletamento dos argumentos dos comandos será feito chamando `complete_foo()` com os argumentos *text*, *line*, *begidx* e *endidx*. *text* é o prefixo da string que estamos tentando corresponder: todas as correspondências retornadas devem começar com ele. *line* é a linha de entrada atual com os espaços em branco iniciais removidos, *begidx* e *endidx* são os índices inicial e final do texto de prefixo, que podem ser usados para fornecer diferentes autocompletamentos dependendo da posição em que o argumento está.

`Cmd.`**`do_help`**(*arg*)

Todas as subclasses de *Cmd* herdam um `do_help()` predefinido. Este método, chamado com um argumento `'bar'`, invoca o método correspondente `help_bar()`, e se este não estiver presente, imprime a docstring de `do_bar()`, se disponível. Sem argumento, `do_help()` lista todos os tópicos de ajuda disponíveis (isto é, todos os comandos com métodos `help_*()` correspondentes ou comandos que têm docstrings), e também lista quaisquer comandos não documentados.

`Cmd.`**`onecmd`**(*str*)

Interpreta o argumento como se tivesse sido digitado em resposta ao prompt. Isso pode ser substituído, mas normalmente não precisa ser; veja os métodos *precmd()* e *postcmd()* para ganchos de execução úteis. O valor de retorno é um sinalizador que indica se a interpretação de comandos pelo interpretador deve parar. Se houver um método `do_*()` para o comando *str*, o valor de retorno desse método é retornado, caso contrário, o valor de retorno do método *default()* é retornado.

`Cmd.`**`emptyline`**()

Método chamado quando uma linha vazia é inserida em resposta ao prompt. Se esse método não for substituído, ele repete o último comando não vazio inserido.

`Cmd.`**`default`**(*line*)

Método chamado em uma linha de entrada quando o prefixo do comando não é reconhecido. Se esse método não for substituído, ele imprime uma mensagem de erro e retorna.

`Cmd.`**`completedefault`**(*text*, *line*, *begidx*, *endidx*)

Método chamado para completar uma linha de entrada quando nenhum método `complete_*()` especificado pelo comando está disponível. Por padrão, ele retorna uma lista vazia.

Cmd.**columnize**(*list*, *displaywidth=80*)

Método chamado para exibir uma lista de strings como um conjunto compacto de colunas. Cada coluna tem apenas a largura necessária. As colunas são separadas por dois espaços para legibilidade.

Cmd.**precmd**(*line*)

Método gancho executado logo antes da linha de comando *line* ser interpretada, mas depois que o prompt de entrada é gerado e emitido. Este método é um stub em *Cmd*; ele existe para ser substituído por subclasses. O valor de retorno é usado como o comando que será executado pelo método *onecmd()*; a implementação *precmd()* pode reescrever o comando ou simplesmente retornar *line* inalterado.

Cmd.**postcmd**(*stop*, *line*)

Método gancho executado logo após o término do despacho de um comando. Este método é um stub em *Cmd*; ele existe para ser substituído por subclasses. *line* é a linha de comando que foi executada, e *stop* é um sinalizador que indica se a execução será encerrada após a chamada para *postcmd()*; este será o valor de retorno do método *onecmd()*. O valor de retorno deste método será usado como o novo valor para o sinalizador interno que corresponde a *stop*; retornar false fará com que a interpretação continue.

Cmd.**preloop**()

Método gancho executado uma vez quando *cmdloop()* é chamado. Este método é um stub em *Cmd*; ele existe para ser substituído por subclasses.

Cmd.**postloop**()

Método gancho executado uma vez quando *cmdloop()* está para ser retornado. Este método é um stub em *Cmd*; ele existe para ser substituído por subclasses.

Instâncias das subclasses *Cmd* têm algumas variáveis de instância públicas:

Cmd.**prompt**

O prompt emitido para solicitar informações.

Cmd.**identchars**

A sequência de caracteres aceita para o prefixo do comando.

Cmd.**lastcmd**

O último prefixo de comando não vazio visto.

Cmd.**cmdqueue**

Uma lista de linhas de entrada enfileiradas. A lista cmdqueue é verificada em *cmdloop()* quando uma nova entrada é necessária; se não estiver vazia, seus elementos serão processados em ordem, como se tivessem sido inseridos no prompt.

Cmd.**intro**

Uma string para emitir como uma introdução ou banner. Pode ser substituída dando ao método *cmdloop()* um argumento.

Cmd.**doc_header**

O cabeçalho a ser emitido se a saída de ajuda tiver uma seção para comandos documentados.

Cmd.**misc_header**

O cabeçalho a ser emitido se a saída de ajuda tiver uma seção para tópicos de ajuda diversos (ou seja, há métodos `help_*()` sem métodos `do_*()` correspondentes).

Cmd.**undoc_header**

O cabeçalho a ser emitido se a saída de ajuda tiver uma seção para comandos não documentados (ou seja, houver métodos `do_*()` sem métodos `help_*()` correspondentes).

Cmd.**ruler**

O caractere usado para desenhar linhas separadoras sob os cabeçalhos de mensagem de ajuda. Se estiver vazio, nenhuma linha de régua será desenhada. O padrão é `'='`.

Cmd.**use_rawinput**

Um sinalizador, com padrão true. Se true, *cmdloop()* usa *input()* para exibir um prompt e ler o próximo comando; se false, *sys.stdout.write()* e *sys.stdin.readline()* são usados. (Isso significa que ao importar *readline*, em sistemas que o suportam, o interpretador suportará automaticamente edição de linha no estilo **Emacs** e pressionamentos de teclas de histórico de comando.)

17.9.2 Exemplo do Cmd

O módulo *cmd* é útil principalmente para criar shells personalizados que permitem ao usuário trabalhar com um programa interativamente.

Esta seção apresenta um exemplo simples de como construir um shell em torno de alguns dos comandos do módulo *turtle*.

Comandos básicos do turtle, como *forward()*, são adicionados a uma subclasse de *Cmd* com o método chamado `do_forward()`. O argumento é convertido em um número e despachado para o módulo turtle. A docstring é usada no utilitário de ajuda fornecido pelo shell.

O exemplo também inclui um recurso básico de gravação e reprodução implementado com o método *precmd()* que é responsável por converter a entrada para minúsculas e gravar os comandos em um arquivo. O método `do_playback()` lê o arquivo e adiciona os comandos gravados ao *cmdqueue* para reprodução imediata:

```
import cmd, sys
from turtle import *

class TurtleShell(cmd.Cmd):
    intro = 'Welcome to the turtle shell.   Type help or ? to list commands.\n'
    prompt = '(turtle) '
    file = None

    # ----- comandos básicos do turtle -----
    def do_forward(self, arg):
        'Move the turtle forward by the specified distance:  FORWARD 10'
        forward(*parse(arg))
    def do_right(self, arg):
        'Turn turtle right by given number of degrees:  RIGHT 20'
        right(*parse(arg))
    def do_left(self, arg):
        'Turn turtle left by given number of degrees:  LEFT 90'
        left(*parse(arg))
    def do_goto(self, arg):
        'Move turtle to an absolute position with changing orientation.  GOTO 100␣
↪200'
        goto(*parse(arg))
    def do_home(self, arg):
        'Return turtle to the home position:  HOME'
        home()
    def do_circle(self, arg):
        'Draw circle with given radius an options extent and steps:  CIRCLE 50'
        circle(*parse(arg))
    def do_position(self, arg):
        'Print the current turtle position:  POSITION'
        print('Current position is %d %d\n' % position())
    def do_heading(self, arg):
        'Print the current turtle heading in degrees:  HEADING'
        print('Current heading is %d\n' % (heading(),))
    def do_color(self, arg):
        'Set the color:  COLOR BLUE'
        color(arg.lower())
```

(continua na próxima página)

(continuação da página anterior)

```
    def do_undo(self, arg):
        'Undo (repeatedly) the last turtle action(s):  UNDO'
    def do_reset(self, arg):
        'Clear the screen and return turtle to center:  RESET'
        reset()
    def do_bye(self, arg):
        'Stop recording, close the turtle window, and exit:  BYE'
        print('Thank you for using Turtle')
        self.close()
        bye()
        return True

    # ----- registra e reproduz -----
    def do_record(self, arg):
        'Save future commands to filename:  RECORD rose.cmd'
        self.file = open(arg, 'w')
    def do_playback(self, arg):
        'Playback commands from a file:  PLAYBACK rose.cmd'
        self.close()
        with open(arg) as f:
            self.cmdqueue.extend(f.read().splitlines())
    def precmd(self, line):
        line = line.lower()
        if self.file and 'playback' not in line:
            print(line, file=self.file)
        return line
    def close(self):
        if self.file:
            self.file.close()
            self.file = None

def parse(arg):
    'Convert a series of zero or more numbers to an argument tuple'
    return tuple(map(int, arg.split()))

if __name__ == '__main__':
    TurtleShell().cmdloop()
```

Aqui está uma sessão de exemplo com o shell do turtle mostrando as funções de ajuda, usando linhas em branco para repetir comandos e o recurso simples de gravação e reprodução:

```
Welcome to the turtle shell.   Type help or ? to list commands.

(turtle) ?

Documented commands (type help <topic>):
========================================
bye     color    goto     home  playback  record  right
circle  forward  heading  left  position  reset   undo

(turtle) help forward
Move the turtle forward by the specified distance:  FORWARD 10
(turtle) record spiral.cmd
(turtle) position
Current position is 0 0
```

(continua na próxima página)

(continuação da página anterior)

```
(turtle) heading
Current heading is 0

(turtle) reset
(turtle) circle 20
(turtle) right 30
(turtle) circle 40
(turtle) right 30
(turtle) circle 60
(turtle) right 30
(turtle) circle 80
(turtle) right 30
(turtle) circle 100
(turtle) right 30
(turtle) circle 120
(turtle) right 30
(turtle) circle 120
(turtle) heading
Current heading is 180

(turtle) forward 100
(turtle)
(turtle) right 90
(turtle) forward 100
(turtle)
(turtle) right 90
(turtle) forward 400
(turtle) right 90
(turtle) forward 500
(turtle) right 90
(turtle) forward 400
(turtle) right 90
(turtle) forward 300
(turtle) playback spiral.cmd
Current position is 0 0

Current heading is 0

Current heading is 180

(turtle) bye
Thank you for using Turtle
```

CAPÍTULO 18

Execução Concorrente

Os módulos descritos neste capítulo fornecem suporte a execução simultânea de código. A escolha apropriada da ferramenta dependerá da tarefa a ser executada (CPU bound ou IO bound) e do estilo de desenvolvimento preferencial (multitarefa cooperativa orientada a eventos versus multitarefa preemptiva). Eis uma visão geral:

18.1 `threading` — Paralelismo baseado em threads

Código-fonte: Lib/threading.py

Este módulo constrói interfaces de alto nível para threading usando o módulo *`_thread`*, de mais baixo nível.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

18.1.1 Introdução

O módulo `threading` fornece uma maneira de executar múltiplas threads (unidades menores de um processo) simultaneamente dentro de um único processo. Ele permite a criação e o gerenciamento de threads, possibilitando a execução de tarefas em paralelo, compartilhando espaço de memória. Threads são particularmente úteis quando tarefas são limitadas por E/S, como operações de arquivo ou solicitações de rede, onde grande parte do tempo é gasto aguardando recursos externos.

Um caso de uso típico para `threading` inclui o gerenciamento de um conjunto de threads de trabalho que podem processar múltiplas tarefas simultaneamente. Aqui está um exemplo básico de criação e inicialização de threads usando *`Thread`*:

```
import threading
import time

def crawl(link, delay=3):
    print(f"crawl started for {link}")
    time.sleep(delay)  # Blocking I/O (simulating a network request)
    print(f"crawl ended for {link}")
```

(continua na próxima página)

(continuação da página anterior)

```
links = [
    "https://python.org",
    "https://docs.python.org",
    "https://peps.python.org",
]

# Start threads for each link
threads = []
for link in links:
    # Using `args` to pass positional arguments and `kwargs` for keyword arguments
    t = threading.Thread(target=crawl, args=(link,), kwargs={"delay": 2})
    threads.append(t)

# Start each thread
for t in threads:
    t.start()

# Wait for all threads to finish
for t in threads:
    t.join()
```

Alterado na versão 3.7: Este módulo costumava ser opcional, agora está sempre disponível.

Ver também

concurrent.futures.ThreadPoolExecutor offers a higher level interface to push tasks to a background thread without blocking execution of the calling thread, while still being able to retrieve their results when needed.

queue provides a thread-safe interface for exchanging data between running threads.

asyncio offers an alternative approach to achieving task level concurrency without requiring the use of multiple operating system threads.

Nota

In the Python 2.x series, this module contained `camelCase` names for some methods and functions. These are deprecated as of Python 3.10, but they are still supported for compatibility with Python 2.5 and lower.

Detalhes da implementação do CPython: In CPython, due to the *Global Interpreter Lock*, only one thread can execute Python code at once (even though certain performance-oriented libraries might overcome this limitation). If you want your application to make better use of the computational resources of multi-core machines, you are advised to use *multiprocessing* or *concurrent.futures.ProcessPoolExecutor*. However, threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously.

18.1.2 GIL and performance considerations

Unlike the *multiprocessing* module, which uses separate processes to bypass the *global interpreter lock* (GIL), the threading module operates within a single process, meaning that all threads share the same memory space. However, the GIL limits the performance gains of threading when it comes to CPU-bound tasks, as only one thread can execute Python bytecode at a time. Despite this, threads remain a useful tool for achieving concurrency in many scenarios.

As of Python 3.13, *free-threaded* builds can disable the GIL, enabling true parallel execution of threads, but this feature is not available by default (see **PEP 703**).

18.1.3 Referência

Este módulo define as seguintes funções:

`threading.`**`active_count`**`()`

Retorna o número de objetos *`Thread`* atualmente ativos. A quantidade retornada é igual ao tamanho da lista retornada por *`enumerate()`*.

The function `activeCount` is a deprecated alias for this function.

`threading.`**`current_thread`**`()`

Return the current *`Thread`* object, corresponding to the caller's thread of control. If the caller's thread of control was not created through the `threading` module, a dummy thread object with limited functionality is returned.

The function `currentThread` is a deprecated alias for this function.

`threading.`**`excepthook`**`(`*`args`*`, /)`

Handle uncaught exception raised by *`Thread.run()`*.

O argumento *args* tem os seguintes atributos:

- *exc_type*: Tipo de exceção..
- *exc_value*: Valor da exceção, pode ser `None`.
- *exc_traceback*: Pilha de execução da exceção, pode ser `None`.
- *thread*: Thread que levantou a exceção, pode ser `None`.

Se *exc_type* é *`SystemExit`*, a exceção é silenciosamente ignorada. Caso contrário, a exceção é exibida em *`sys.stderr`*.

Se esta função levantar uma exceção, *`sys.excepthook()`* é chamada para tratá-la.

`threading.excepthook()` pode ser substituída para controlar como exceções não capturadas levantadas por *`Thread.run()`* são tratadas.

Storing *exc_value* using a custom hook can create a reference cycle. It should be cleared explicitly to break the reference cycle when the exception is no longer needed.

Storing *thread* using a custom hook can resurrect it if it is set to an object which is being finalized. Avoid storing *thread* after the custom hook completes to avoid resurrecting objects.

> **Ver também**
>
> *`sys.excepthook()`* trata exceções não capturadas.

Adicionado na versão 3.8.

`threading.`**`__excepthook__`**

Holds the original value of *`threading.excepthook()`*. It is saved so that the original value can be restored in case they happen to get replaced with broken or alternative objects.

Adicionado na versão 3.10.

`threading.`**`get_ident`**`()`

Retorna o 'identificador de thread' do thread atual. Este é um número inteiro diferente de zero. Seu valor não tem significado direto; pretende-se que seja um cookie mágico para ser usado, por exemplo, para indexar um dicionário de dados específicos do thread. identificadores de thread podem ser reciclados quando um thread sai e outro é criado.

Adicionado na versão 3.3.

threading.**get_native_id**()

Retorna a ID de thread integral nativa da thread atual atribuída pelo kernel. Este é um número inteiro não negativo. Seu valor pode ser usado para identificar exclusivamente essa thread específica em todo o sistema (até que a thread termine, após o que o valor poderá ser reciclado pelo sistema operacional).

Disponibilidade: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX, DragonFlyBSD, GNU/kFreeBSD.

Adicionado na versão 3.8.

Alterado na versão 3.13: Adicionado suporte a GNU/kFreeBSD.

threading.**enumerate**()

Return a list of all *Thread* objects currently active. The list includes daemonic threads and dummy thread objects created by *current_thread()*. It excludes terminated threads and threads that have not yet been started. However, the main thread is always part of the result, even when terminated.

threading.**main_thread**()

Return the main *Thread* object. In normal conditions, the main thread is the thread from which the Python interpreter was started.

Adicionado na versão 3.4.

threading.**settrace**(*func*)

Set a trace function for all threads started from the `threading` module. The *func* will be passed to *sys.settrace()* for each thread, before its *run()* method is called.

threading.**settrace_all_threads**(*func*)

Set a trace function for all threads started from the `threading` module and all Python threads that are currently executing.

The *func* will be passed to *sys.settrace()* for each thread, before its *run()* method is called.

Adicionado na versão 3.12.

threading.**gettrace**()

Obtém a função trace conforme definido por *settrace()*.

Adicionado na versão 3.10.

threading.**setprofile**(*func*)

Set a profile function for all threads started from the `threading` module. The *func* will be passed to *sys.setprofile()* for each thread, before its *run()* method is called.

threading.**setprofile_all_threads**(*func*)

Set a profile function for all threads started from the `threading` module and all Python threads that are currently executing.

The *func* will be passed to *sys.setprofile()* for each thread, before its *run()* method is called.

Adicionado na versão 3.12.

threading.**getprofile**()

Obtém a função do criador de perfil conforme definido por *setprofile()*.

Adicionado na versão 3.10.

threading.**stack_size**([*size*])

Retorna o tamanho da pilha de threads usado ao criar novos threads. O argumento opcional *size* especifica o tamanho da pilha a ser usado para threads criados posteriormente e deve ser 0 (usar plataforma ou padrão configurado) ou um valor inteiro positivo de pelo menos 32.768 (32 KiB). Se *size* não for especificado, 0 será usado. Se a alteração do tamanho da pilha de threads não for suportada, uma *RuntimeError* será levantada. Se o tamanho da pilha especificado for inválido, uma *ValueError* será levantada e o tamanho da pilha não será modificado. Atualmente, 0 KiB é o valor mínimo de tamanho de pilha suportado para garantir espaço suficiente para o próprio interpretador. Observe que algumas plataformas podem ter restrições específicas sobre valores

para o tamanho da pilha, como exigir um tamanho mínimo de pilha > 32 KiB ou exigir alocação em múltiplos do tamanho da página de memória do sistema – a documentação da plataforma deve ser consultada para obter mais informações (4 páginas KiB são comuns; usar múltiplos de 4096 para o tamanho da pilha é a abordagem sugerida na ausência de informações mais específicas).

Disponibilidade: Windows, pthreads.

Plataformas Unix com suporte a threads POSIX.

This module also defines the following constant:

`threading.`**`TIMEOUT_MAX`**

The maximum value allowed for the *timeout* parameter of blocking functions (`Lock.acquire()`, `RLock.acquire()`, `Condition.wait()`, etc.). Specifying a timeout greater than this value will raise an `OverflowError`.

Adicionado na versão 3.2.

This module defines a number of classes, which are detailed in the sections below.

The design of this module is loosely based on Java's threading model. However, where Java makes locks and condition variables basic behavior of every object, they are separate objects in Python. Python's `Thread` class supports a subset of the behavior of Java's Thread class; currently, there are no priorities, no thread groups, and threads cannot be destroyed, stopped, suspended, resumed, or interrupted. The static methods of Java's Thread class, when implemented, are mapped to module-level functions.

All of the methods described below are executed atomically.

Thread-local data

Thread-local data is data whose values are thread specific. If you have data that you want to be local to a thread, create a `local` object and use its attributes:

```
>>> mydata = local()
>>> mydata.number = 42
>>> mydata.number
42
```

You can also access the `local`-object's dictionary:

```
>>> mydata.__dict__
{'number': 42}
>>> mydata.__dict__.setdefault('widgets', [])
[]
>>> mydata.widgets
[]
```

If we access the data in a different thread:

```
>>> log = []
>>> def f():
...     items = sorted(mydata.__dict__.items())
...     log.append(items)
...     mydata.number = 11
...     log.append(mydata.number)

>>> import threading
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
>>> log
[[], 11]
```

we get different data. Furthermore, changes made in the other thread don't affect data seen in this thread:

```
>>> mydata.number
42
```

Of course, values you get from a *local* object, including their `__dict__` attribute, are for whatever thread was current at the time the attribute was read. For that reason, you generally don't want to save these values across threads, as they apply only to the thread they came from.

You can create custom *local* objects by subclassing the *local* class:

```
>>> class MyLocal(local):
...     number = 2
...     def __init__(self, /, **kw):
...         self.__dict__.update(kw)
...     def squared(self):
...         return self.number ** 2
```

This can be useful to support default values, methods and initialization. Note that if you define an `__init__()` method, it will be called each time the *local* object is used in a separate thread. This is necessary to initialize each thread's dictionary.

Now if we create a *local* object:

```
>>> mydata = MyLocal(color='red')
```

we have a default number:

```
>>> mydata.number
2
```

an initial color:

```
>>> mydata.color
'red'
>>> del mydata.color
```

And a method that operates on the data:

```
>>> mydata.squared()
4
```

As before, we can access the data in a separate thread:

```
>>> log = []
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
>>> log
[[('color', 'red')], 11]
```

without affecting this thread's data:

```
>>> mydata.number
2
>>> mydata.color
Traceback (most recent call last):
...
AttributeError: 'MyLocal' object has no attribute 'color'
```

Note that subclasses can define *__slots__*, but they are not thread local. They are shared across threads:

```
>>> class MyLocal(local):
...     __slots__ = 'number'

>>> mydata = MyLocal()
>>> mydata.number = 42
>>> mydata.color = 'red'
```

So, the separate thread:

```
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
```

affects what we see:

```
>>> mydata.number
11
```

class threading.**local**

A class that represents thread-local data.

Thread objects

The *Thread* class represents an activity that is run in a separate thread of control. There are two ways to specify the activity: by passing a callable object to the constructor, or by overriding the *run()* method in a subclass. No other methods (except for the constructor) should be overridden in a subclass. In other words, *only* override the `__init__()` and *run()* methods of this class.

Once a thread object is created, its activity must be started by calling the thread's *start()* method. This invokes the *run()* method in a separate thread of control.

Once the thread's activity is started, the thread is considered 'alive'. It stops being alive when its *run()* method terminates – either normally, or by raising an unhandled exception. The *is_alive()* method tests whether the thread is alive.

Other threads can call a thread's *join()* method. This blocks the calling thread until the thread whose *join()* method is called is terminated.

A thread has a name. The name can be passed to the constructor, and read or changed through the *name* attribute.

If the *run()* method raises an exception, *threading.excepthook()* is called to handle it. By default, *threading.excepthook()* ignores silently *SystemExit*.

A thread can be flagged as a "daemon thread". The significance of this flag is that the entire Python program exits when only daemon threads are left. The initial value is inherited from the creating thread. The flag can be set through the *daemon* property or the *daemon* constructor argument.

Nota

Daemon threads are abruptly stopped at shutdown. Their resources (such as open files, database transactions, etc.) may not be released properly. If you want your threads to stop gracefully, make them non-daemonic and use a suitable signalling mechanism such as an *Event*.

There is a "main thread" object; this corresponds to the initial thread of control in the Python program. It is not a daemon thread.

There is the possibility that "dummy thread objects" are created. These are thread objects corresponding to "alien threads", which are threads of control started outside the threading module, such as directly from C code. Dummy thread objects have limited functionality; they are always considered alive and daemonic, and cannot be *joined*. They are never deleted, since it is impossible to detect the termination of alien threads.

class threading.**Thread**(*group=None*, *target=None*, *name=None*, *args=()*, *kwargs={}*, *, *daemon=None*, *context=None*)

This constructor should always be called with keyword arguments. Arguments are:

group should be `None`; reserved for future extension when a `ThreadGroup` class is implemented.

target is the callable object to be invoked by the *run()* method. Defaults to `None`, meaning nothing is called.

name is the thread name. By default, a unique name is constructed of the form "Thread-*N*" where *N* is a small decimal number, or "Thread-*N* (target)" where "target" is `target.__name__` if the *target* argument is specified.

args is a list or tuple of arguments for the target invocation. Defaults to `()`.

kwargs is a dictionary of keyword arguments for the target invocation. Defaults to `{}`.

If not `None`, *daemon* explicitly sets whether the thread is daemonic. If `None` (the default), the daemonic property is inherited from the current thread.

context is the *Context* value to use when starting the thread. The default value is `None` which indicates that the *sys.flags.thread_inherit_context* flag controls the behaviour. If the flag is true, threads will start with a copy of the context of the caller of *start()*. If false, they will start with an empty context. To explicitly start with an empty context, pass a new instance of *Context()*. To explicitly start with a copy of the current context, pass the value from *copy_context()*. The flag defaults true on free-threaded builds and false otherwise.

If the subclass overrides the constructor, it must make sure to invoke the base class constructor (`Thread.__init__()`) before doing anything else to the thread.

Alterado na versão 3.3: Adicionado o parâmetro *daemon*.

Alterado na versão 3.10: Use the *target* name if *name* argument is omitted.

Alterado na versão 3.14: Adicionado o parâmetro *context*.

start()

Start the thread's activity.

It must be called at most once per thread object. It arranges for the object's *run()* method to be invoked in a separate thread of control.

This method will raise a *RuntimeError* if called more than once on the same thread object.

If supported, set the operating system thread name to *threading.Thread.name*. The name can be truncated depending on the operating system thread name limits.

Alterado na versão 3.14: Set the operating system thread name.

run()

Method representing the thread's activity.

You may override this method in a subclass. The standard *run()* method invokes the callable object passed to the object's constructor as the *target* argument, if any, with positional and keyword arguments taken from the *args* and *kwargs* arguments, respectively.

Using list or tuple as the *args* argument which passed to the *Thread* could achieve the same effect.

Exemplo:

```
>>> from threading import Thread
>>> t = Thread(target=print, args=[1])
>>> t.run()
1
>>> t = Thread(target=print, args=(1,))
>>> t.run()
1
```

join(*timeout=None*)

Wait until the thread terminates. This blocks the calling thread until the thread whose *join()* method is called terminates – either normally or through an unhandled exception – or until the optional timeout occurs.

When the *timeout* argument is present and not `None`, it should be a floating-point number specifying a timeout for the operation in seconds (or fractions thereof). As *join()* always returns `None`, you must call *is_alive()* after *join()* to decide whether a timeout happened – if the thread is still alive, the *join()* call timed out.

When the *timeout* argument is not present or `None`, the operation will block until the thread terminates.

A thread can be joined many times.

join() raises a *RuntimeError* if an attempt is made to join the current thread as that would cause a deadlock. It is also an error to *join()* a thread before it has been started and attempts to do so raise the same exception.

If an attempt is made to join a running daemonic thread in late stages of *Python finalization* `join()` raises a *PythonFinalizationError*.

Alterado na versão 3.14: May raise *PythonFinalizationError*.

name

A string used for identification purposes only. It has no semantics. Multiple threads may be given the same name. The initial name is set by the constructor.

On some platforms, the thread name is set at the operating system level when the thread starts, so that it is visible in task managers. This name may be truncated to fit in a system-specific limit (for example, 15 bytes on Linux or 63 bytes on macOS).

Changes to *name* are only reflected at the OS level when the currently running thread is renamed. (Setting the *name* attribute of a different thread only updates the Python Thread object.)

getName()

setName()

Deprecated getter/setter API for *name*; use it directly as a property instead.

Descontinuado desde a versão 3.10.

ident

The 'thread identifier' of this thread or `None` if the thread has not been started. This is a nonzero integer. See the *get_ident()* function. Thread identifiers may be recycled when a thread exits and another thread is created. The identifier is available even after the thread has exited.

native_id

The Thread ID (`TID`) of this thread, as assigned by the OS (kernel). This is a non-negative integer, or `None` if the thread has not been started. See the *get_native_id()* function. This value may be used to uniquely identify this particular thread system-wide (until the thread terminates, after which the value may be recycled by the OS).

> **Nota**
>
> Similar to Process IDs, Thread IDs are only valid (guaranteed unique system-wide) from the time the thread is created until the thread has been terminated.

Disponibilidade: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX, DragonFlyBSD.

Adicionado na versão 3.8.

is_alive()

Return whether the thread is alive.

This method returns `True` just before the *`run()`* method starts until just after the *`run()`* method terminates. The module function *`enumerate()`* returns a list of all alive threads.

daemon

A boolean value indicating whether this thread is a daemon thread (`True`) or not (`False`). This must be set before *`start()`* is called, otherwise *`RuntimeError`* is raised. Its initial value is inherited from the creating thread; the main thread is not a daemon thread and therefore all threads created in the main thread default to *`daemon`* = `False`.

The entire Python program exits when no alive non-daemon threads are left.

isDaemon()

setDaemon()

Deprecated getter/setter API for *`daemon`*; use it directly as a property instead.

Descontinuado desde a versão 3.10.

Lock objects

A primitive lock is a synchronization primitive that is not owned by a particular thread when locked. In Python, it is currently the lowest level synchronization primitive available, implemented directly by the *`_thread`* extension module.

A primitive lock is in one of two states, "locked" or "unlocked". It is created in the unlocked state. It has two basic methods, *`acquire()`* and *`release()`*. When the state is unlocked, *`acquire()`* changes the state to locked and returns immediately. When the state is locked, *`acquire()`* blocks until a call to *`release()`* in another thread changes it to unlocked, then the *`acquire()`* call resets it to locked and returns. The *`release()`* method should only be called in the locked state; it changes the state to unlocked and returns immediately. If an attempt is made to release an unlocked lock, a *`RuntimeError`* will be raised.

Locks also support the *context management protocol*.

When more than one thread is blocked in *`acquire()`* waiting for the state to turn to unlocked, only one thread proceeds when a *`release()`* call resets the state to unlocked; which one of the waiting threads proceeds is not defined, and may vary across implementations.

All methods are executed atomically.

class threading.**Lock**

The class implementing primitive lock objects. Once a thread has acquired a lock, subsequent attempts to acquire it block, until it is released; any thread may release it.

Alterado na versão 3.13: `Lock` is now a class. In earlier Pythons, `Lock` was a factory function which returned an instance of the underlying private lock type.

acquire(*blocking=True*, *timeout=-1*)

Adquire uma trava, bloqueante ou não.

When invoked with the *blocking* argument set to `True` (the default), block until the lock is unlocked, then set it to locked and return `True`.

When invoked with the *blocking* argument set to `False`, do not block. If a call with *blocking* set to `True` would block, return `False` immediately; otherwise, set the lock to locked and return `True`.

When invoked with the floating-point *timeout* argument set to a positive value, block for at most the number of seconds specified by *timeout* and as long as the lock cannot be acquired. A *timeout* argument of `-1` specifies an unbounded wait. It is forbidden to specify a *timeout* when *blocking* is `False`.

The return value is `True` if the lock is acquired successfully, `False` if not (for example if the *timeout* expired).

Alterado na versão 3.2: O parâmetro *timeout* é novo.

Alterado na versão 3.2: Lock acquisition can now be interrupted by signals on POSIX if the underlying threading implementation supports it.

Alterado na versão 3.14: Lock acquisition can now be interrupted by signals on Windows.

release()

Release a lock. This can be called from any thread, not only the thread which has acquired the lock.

When the lock is locked, reset it to unlocked, and return. If any other threads are blocked waiting for the lock to become unlocked, allow exactly one of them to proceed.

When invoked on an unlocked lock, a *RuntimeError* is raised.

There is no return value.

locked()

Return `True` if the lock is acquired.

RLock objects

A reentrant lock is a synchronization primitive that may be acquired multiple times by the same thread. Internally, it uses the concepts of "owning thread" and "recursion level" in addition to the locked/unlocked state used by primitive locks. In the locked state, some thread owns the lock; in the unlocked state, no thread owns it.

Threads call a lock's *acquire()* method to lock it, and its *release()* method to unlock it.

Nota

Reentrant locks support the *context management protocol*, so it is recommended to use `with` instead of manually calling *acquire()* and *release()* to handle acquiring and releasing the lock for a block of code.

RLock's *acquire()*/*release()* call pairs may be nested, unlike Lock's *acquire()*/*release()*. Only the final *release()* (the *release()* of the outermost pair) resets the lock to an unlocked state and allows another thread blocked in *acquire()* to proceed.

acquire()/*release()* must be used in pairs: each acquire must have a release in the thread that has acquired the lock. Failing to call release as many times the lock has been acquired can lead to deadlock.

class threading.**RLock**

This class implements reentrant lock objects. A reentrant lock must be released by the thread that acquired it. Once a thread has acquired a reentrant lock, the same thread may acquire it again without blocking; the thread must release it once for each time it has acquired it.

Note that `RLock` is actually a factory function which returns an instance of the most efficient version of the concrete RLock class that is supported by the platform.

acquire(*blocking=True*, *timeout=-1*)

Adquire uma trava, bloqueante ou não.

Ver também

Using RLock as a context manager
Recommended over manual `acquire()` and *release()* calls whenever practical.

When invoked with the *blocking* argument set to `True` (the default):

- If no thread owns the lock, acquire the lock and return immediately.
- If another thread owns the lock, block until we are able to acquire lock, or *timeout*, if set to a positive float value.
- If the same thread owns the lock, acquire the lock again, and return immediately. This is the difference between *Lock* and `RLock`; *Lock* handles this case the same as the previous, blocking until the lock can be acquired.

When invoked with the *blocking* argument set to `False`:

- If no thread owns the lock, acquire the lock and return immediately.
- If another thread owns the lock, return immediately.
- If the same thread owns the lock, acquire the lock again and return immediately.

In all cases, if the thread was able to acquire the lock, return `True`. If the thread was unable to acquire the lock (i.e. if not blocking or the timeout was reached) return `False`.

If called multiple times, failing to call *release()* as many times may lead to deadlock. Consider using `RLock` as a context manager rather than calling acquire/release directly.

Alterado na versão 3.2: O parâmetro *timeout* é novo.

release()

Release a lock, decrementing the recursion level. If after the decrement it is zero, reset the lock to unlocked (not owned by any thread), and if any other threads are blocked waiting for the lock to become unlocked, allow exactly one of them to proceed. If after the decrement the recursion level is still nonzero, the lock remains locked and owned by the calling thread.

Only call this method when the calling thread owns the lock. A *RuntimeError* is raised if this method is called when the lock is not acquired.

There is no return value.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

Condition objects

A condition variable is always associated with some kind of lock; this can be passed in or one will be created by default. Passing one in is useful when several condition variables must share the same lock. The lock is part of the condition object: you don't have to track it separately.

A condition variable obeys the *context management protocol*: using the `with` statement acquires the associated lock for the duration of the enclosed block. The *acquire()* and *release()* methods also call the corresponding methods of the associated lock.

Other methods must be called with the associated lock held. The *wait()* method releases the lock, and then blocks until another thread awakens it by calling *notify()* or *notify_all()*. Once awakened, *wait()* re-acquires the lock and returns. It is also possible to specify a timeout.

The *notify()* method wakes up one of the threads waiting for the condition variable, if any are waiting. The *notify_all()* method wakes up all threads waiting for the condition variable.

Note: the *notify()* and *notify_all()* methods don't release the lock; this means that the thread or threads awakened will not return from their *wait()* call immediately, but only when the thread that called *notify()* or *notify_all()* finally relinquishes ownership of the lock.

The typical programming style using condition variables uses the lock to synchronize access to some shared state; threads that are interested in a particular change of state call *wait()* repeatedly until they see the desired state, while threads that modify the state call *notify()* or *notify_all()* when they change the state in such a way that it could possibly be a desired state for one of the waiters. For example, the following code is a generic producer-consumer situation with unlimited buffer capacity:

```
# Consume one item
with cv:
    while not an_item_is_available():
        cv.wait()
    get_an_available_item()
```

(continua na próxima página)

(continuação da página anterior)

```
# Produce one item
with cv:
    make_an_item_available()
    cv.notify()
```

The `while` loop checking for the application's condition is necessary because *`wait()`* can return after an arbitrary long time, and the condition which prompted the *`notify()`* call may no longer hold true. This is inherent to multi--threaded programming. The *`wait_for()`* method can be used to automate the condition checking, and eases the computation of timeouts:

```
# Consume an item
with cv:
    cv.wait_for(an_item_is_available)
    get_an_available_item()
```

To choose between *`notify()`* and *`notify_all()`*, consider whether one state change can be interesting for only one or several waiting threads. E.g. in a typical producer-consumer situation, adding one item to the buffer only needs to wake up one consumer thread.

class threading.**Condition**(*lock=None*)

This class implements condition variable objects. A condition variable allows one or more threads to wait until they are notified by another thread.

If the *lock* argument is given and not `None`, it must be a *`Lock`* or *`RLock`* object, and it is used as the underlying lock. Otherwise, a new *`RLock`* object is created and used as the underlying lock.

Alterado na versão 3.3: changed from a factory function to a class.

acquire(**args*)

Acquire the underlying lock. This method calls the corresponding method on the underlying lock; the return value is whatever that method returns.

release()

Release the underlying lock. This method calls the corresponding method on the underlying lock; there is no return value.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

wait(*timeout=None*)

Wait until notified or until a timeout occurs. If the calling thread has not acquired the lock when this method is called, a *`RuntimeError`* is raised.

This method releases the underlying lock, and then blocks until it is awakened by a *`notify()`* or *`notify_all()`* call for the same condition variable in another thread, or until the optional timeout occurs. Once awakened or timed out, it re-acquires the lock and returns.

When the *timeout* argument is present and not `None`, it should be a floating-point number specifying a timeout for the operation in seconds (or fractions thereof).

When the underlying lock is an *`RLock`*, it is not released using its *`release()`* method, since this may not actually unlock the lock when it was acquired multiple times recursively. Instead, an internal interface of the *`RLock`* class is used, which really unlocks it even when it has been recursively acquired several times. Another internal interface is then used to restore the recursion level when the lock is reacquired.

The return value is `True` unless a given *timeout* expired, in which case it is `False`.

Alterado na versão 3.2: Previously, the method always returned `None`.

wait_for(*predicate*, *timeout=None*)

Wait until a condition evaluates to true. *predicate* should be a callable which result will be interpreted as a boolean value. A *timeout* may be provided giving the maximum time to wait.

This utility method may call *wait()* repeatedly until the predicate is satisfied, or until a timeout occurs. The return value is the last return value of the predicate and will evaluate to `False` if the method timed out.

Ignoring the timeout feature, calling this method is roughly equivalent to writing:

```
while not predicate():
    cv.wait()
```

Therefore, the same rules apply as with *wait()*: The lock must be held when called and is re-acquired on return. The predicate is evaluated with the lock held.

Adicionado na versão 3.2.

notify(*n=1*)

By default, wake up one thread waiting on this condition, if any. If the calling thread has not acquired the lock when this method is called, a *RuntimeError* is raised.

This method wakes up at most *n* of the threads waiting for the condition variable; it is a no-op if no threads are waiting.

The current implementation wakes up exactly *n* threads, if at least *n* threads are waiting. However, it's not safe to rely on this behavior. A future, optimized implementation may occasionally wake up more than *n* threads.

Note: an awakened thread does not actually return from its *wait()* call until it can reacquire the lock. Since *notify()* does not release the lock, its caller should.

notify_all()

Wake up all threads waiting on this condition. This method acts like *notify()*, but wakes up all waiting threads instead of one. If the calling thread has not acquired the lock when this method is called, a *RuntimeError* is raised.

The method `notifyAll` is a deprecated alias for this method.

Semaphore objects

This is one of the oldest synchronization primitives in the history of computer science, invented by the early Dutch computer scientist Edsger W. Dijkstra (he used the names `P()` and `V()` instead of *acquire()* and *release()*).

A semaphore manages an internal counter which is decremented by each *acquire()* call and incremented by each *release()* call. The counter can never go below zero; when *acquire()* finds that it is zero, it blocks, waiting until some other thread calls *release()*.

Semaphores also support the *context management protocol*.

class threading.**Semaphore**(*value=1*)

This class implements semaphore objects. A semaphore manages an atomic counter representing the number of *release()* calls minus the number of *acquire()* calls, plus an initial value. The *acquire()* method blocks if necessary until it can return without making the counter negative. If not given, *value* defaults to 1.

The optional argument gives the initial *value* for the internal counter; it defaults to `1`. If the *value* given is less than 0, *ValueError* is raised.

Alterado na versão 3.3: changed from a factory function to a class.

acquire(*blocking=True*, *timeout=None*)

Acquire a semaphore.

When invoked without arguments:

- If the internal counter is larger than zero on entry, decrement it by one and return `True` immediately.

- If the internal counter is zero on entry, block until awoken by a call to *release()*. Once awoken (and the counter is greater than 0), decrement the counter by 1 and return `True`. Exactly one thread will be awoken by each call to *release()*. The order in which threads are awoken should not be relied on.

When invoked with *blocking* set to `False`, do not block. If a call without an argument would block, return `False` immediately; otherwise, do the same thing as when called without arguments, and return `True`.

When invoked with a *timeout* other than `None`, it will block for at most *timeout* seconds. If acquire does not complete successfully in that interval, return `False`. Return `True` otherwise.

Alterado na versão 3.2: O parâmetro *timeout* é novo.

release(*n=1*)

Release a semaphore, incrementing the internal counter by *n*. When it was zero on entry and other threads are waiting for it to become larger than zero again, wake up *n* of those threads.

Alterado na versão 3.9: Added the *n* parameter to release multiple waiting threads at once.

class threading.**BoundedSemaphore**(*value=1*)

Class implementing bounded semaphore objects. A bounded semaphore checks to make sure its current value doesn't exceed its initial value. If it does, *ValueError* is raised. In most situations semaphores are used to guard resources with limited capacity. If the semaphore is released too many times it's a sign of a bug. If not given, *value* defaults to 1.

Alterado na versão 3.3: changed from a factory function to a class.

`Semaphore` example

Semaphores are often used to guard resources with limited capacity, for example, a database server. In any situation where the size of the resource is fixed, you should use a bounded semaphore. Before spawning any worker threads, your main thread would initialize the semaphore:

```
maxconnections = 5
# ...
pool_sema = BoundedSemaphore(value=maxconnections)
```

Once spawned, worker threads call the semaphore's acquire and release methods when they need to connect to the server:

```
with pool_sema:
    conn = connectdb()
    try:
        # ... use connection ...
    finally:
        conn.close()
```

The use of a bounded semaphore reduces the chance that a programming error which causes the semaphore to be released more than it's acquired will go undetected.

Event objects

This is one of the simplest mechanisms for communication between threads: one thread signals an event and other threads wait for it.

An event object manages an internal flag that can be set to true with the *set()* method and reset to false with the *clear()* method. The *wait()* method blocks until the flag is true.

class threading.**Event**

Class implementing event objects. An event manages a flag that can be set to true with the *set()* method and reset to false with the *clear()* method. The *wait()* method blocks until the flag is true. The flag is initially false.

Alterado na versão 3.3: changed from a factory function to a class.

is_set()

Return `True` if and only if the internal flag is true.

The method `isSet` is a deprecated alias for this method.

set()

Set the internal flag to true. All threads waiting for it to become true are awakened. Threads that call *wait()* once the flag is true will not block at all.

clear()

Reset the internal flag to false. Subsequently, threads calling *wait()* will block until *set()* is called to set the internal flag to true again.

wait(*timeout=None*)

Block as long as the internal flag is false and the timeout, if given, has not expired. The return value represents the reason that this blocking method returned; `True` if returning because the internal flag is set to true, or `False` if a timeout is given and the internal flag did not become true within the given wait time.

When the timeout argument is present and not `None`, it should be a floating-point number specifying a timeout for the operation in seconds, or fractions thereof.

Alterado na versão 3.1: Previously, the method always returned `None`.

Timer objects

This class represents an action that should be run only after a certain amount of time has passed — a timer. *Timer* is a subclass of *Thread* and as such also functions as an example of creating custom threads.

Timers are started, as with threads, by calling their *Timer.start* method. The timer can be stopped (before its action has begun) by calling the *cancel()* method. The interval the timer will wait before executing its action may not be exactly the same as the interval specified by the user.

Por exemplo:

```
def hello():
    print("hello, world")

t = Timer(30.0, hello)
t.start()  # after 30 seconds, "hello, world" will be printed
```

class threading.**Timer**(*interval*, *function*, *args=None*, *kwargs=None*)

Create a timer that will run *function* with arguments *args* and keyword arguments *kwargs*, after *interval* seconds have passed. If *args* is `None` (the default) then an empty list will be used. If *kwargs* is `None` (the default) then an empty dict will be used.

Alterado na versão 3.3: changed from a factory function to a class.

cancel()

Stop the timer, and cancel the execution of the timer's action. This will only work if the timer is still in its waiting stage.

Barrier objects

Adicionado na versão 3.2.

This class provides a simple synchronization primitive for use by a fixed number of threads that need to wait for each other. Each of the threads tries to pass the barrier by calling the *wait()* method and will block until all of the threads have made their *wait()* calls. At this point, the threads are released simultaneously.

The barrier can be reused any number of times for the same number of threads.

As an example, here is a simple way to synchronize a client and server thread:

```
b = Barrier(2, timeout=5)

def server():
    start_server()
    b.wait()
    while True:
        connection = accept_connection()
        process_server_connection(connection)

def client():
    b.wait()
    while True:
        connection = make_connection()
        process_client_connection(connection)
```

class threading.**Barrier**(*parties*, *action=None*, *timeout=None*)

Create a barrier object for *parties* number of threads. An *action*, when provided, is a callable to be called by one of the threads when they are released. *timeout* is the default timeout value if none is specified for the *wait()* method.

wait(*timeout=None*)

Pass the barrier. When all the threads party to the barrier have called this function, they are all released simultaneously. If a *timeout* is provided, it is used in preference to any that was supplied to the class constructor.

The return value is an integer in the range 0 to *parties* – 1, different for each thread. This can be used to select a thread to do some special housekeeping, e.g.:

```
i = barrier.wait()
if i == 0:
    # Only one thread needs to print this
    print("passed the barrier")
```

If an *action* was provided to the constructor, one of the threads will have called it prior to being released. Should this call raise an error, the barrier is put into the broken state.

If the call times out, the barrier is put into the broken state.

This method may raise a *BrokenBarrierError* exception if the barrier is broken or reset while a thread is waiting.

reset()

Return the barrier to the default, empty state. Any threads waiting on it will receive the *BrokenBarrierError* exception.

Note that using this function may require some external synchronization if there are other threads whose state is unknown. If a barrier is broken it may be better to just leave it and create a new one.

abort()

Put the barrier into a broken state. This causes any active or future calls to *wait()* to fail with the *BrokenBarrierError*. Use this for example if one of the threads needs to abort, to avoid deadlocking the application.

It may be preferable to simply create the barrier with a sensible *timeout* value to automatically guard against one of the threads going awry.

parties

The number of threads required to pass the barrier.

n_waiting

The number of threads currently waiting in the barrier.

broken

A boolean that is `True` if the barrier is in the broken state.

exception threading.**BrokenBarrierError**

This exception, a subclass of *RuntimeError*, is raised when the *Barrier* object is reset or broken.

18.1.4 Using locks, conditions, and semaphores in the `with` statement

All of the objects provided by this module that have `acquire` and `release` methods can be used as context managers for a `with` statement. The `acquire` method will be called when the block is entered, and `release` will be called when the block is exited. Hence, the following snippet:

```
with some_lock:
    # do something...
```

é equivalente a:

```
some_lock.acquire()
try:
    # do something...
finally:
    some_lock.release()
```

Currently, *Lock*, *RLock*, *Condition*, *Semaphore*, and *BoundedSemaphore* objects may be used as `with` statement context managers.

18.2 `multiprocessing` — Paralelismo baseado em processo

Código-fonte: Lib/multiprocessing/

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em *plataformas móveis* ou *plataformas WebAssembly*.

18.2.1 Introdução

multiprocessing é um pacote que oferece suporte à invocação de processos utilizando uma API semelhante ao módulo *threading*. O pacote *multiprocessing* oferece simultaneamente concorrência local e remota, efetivamente contornando a *trava global do interpretador*, ao utilizar subprocessos ao invés de threads. Devido a isso, o módulo *multiprocessing* permite ao programador aproveitar totalmente os múltiplos processadores de uma máquina. Ele funciona tanto em POSIX como em Windows.

O módulo *multiprocessing* também introduz APIs que não têm análogos no módulo *threading*. Um exemplo principal disso é o objeto *Pool* que oferece um meio conveniente de paralelizar a execução de uma função em vários valores de entrada, distribuindo os dados de entrada entre processos (paralelismo de dados). O exemplo a seguir demonstra a prática comum de definir tais funções em um módulo para que os processos filhos possam importar esse módulo com sucesso. Este exemplo básico de paralelismo de dados usando *Pool*,

```
from multiprocessing import Pool

def f(x):
    return x*x

if __name__ == '__main__':
    with Pool(5) as p:
        print(p.map(f, [1, 2, 3]))
```

vai exibir na saída padrão

```
[1, 4, 9]
```

Ver também

concurrent.futures.ProcessPoolExecutor oferece uma interface de nível mais alto para enviar tarefas para um processo em segundo plano sem bloquear a execução do processo de chamada. Comparado ao uso direto da interface *Pool*, a API *concurrent.futures* permite mais prontamente que o envio de trabalho para o pool de processos subjacente seja separado da espera pelos resultados.

A classe `Process`

Em *multiprocessing*, os processos são gerados criando um objeto *Process* e então chamando seu método *start()*. *Process* segue a API de *threading.Thread*. Um exemplo trivial de um programa multiprocesso é

```
from multiprocessing import Process

def f(name):
    print('hello', name)

if __name__ == '__main__':
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()
```

Para mostrar os IDs de processo individuais envolvidos, aqui está um exemplo expandido:

```
from multiprocessing import Process
import os

def info(title):
    print(title)
    print('module name:', __name__)
    print('parent process:', os.getppid())
    print('process id:', os.getpid())

def f(name):
    info('function f')
    print('hello', name)

if __name__ == '__main__':
    info('main line')
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()
```

Para uma explicação do porquê a parte `if __name__ == '__main__'` é necessária, veja *Programming guidelines*.

The arguments to *Process* usually need to be unpickleable from within the child process. If you tried typing the above example directly into a REPL it could lead to an *AttributeError* in the child process trying to locate the *f* function in the __main__ module.

Contextos e métodos de inicialização

Dependendo da plataforma, *multiprocessing* provê três maneiras de iniciar um processo. Estes *métodos de inicialização* são

spawn
: O processo pai inicia um novo processo do interpretador Python. O processo filho herdará apenas

os recursos necessários para executar o método *run()* do objeto do processo. Em particular, descritores de arquivo e identificadores desnecessários do processo pai não serão herdados. Iniciar um processo usando esse método é bem lento comparado a usar *fork* ou *forkserver*.

Disponível em plataformas POSIX e Windows. O padrão no Windows e macOS.

fork

O processo pai usa *os.fork()* para criar um fork do interpretador Python. O processo filho, quando começa, é efetivamente idêntico ao processo pai. Todos os recursos do pai são herdados pelo processo filho. Observe que criar um fork com segurança de um processo multithread é problemático.

Disponível em sistemas POSIX.

Alterado na versão 3.14: Esse não é mais o método de inicialização padrão em nenhuma plataforma. O código que requer *fork* deve especificar explicitamente isso por meio de *get_context()* ou *set_start_method()*.

Alterado na versão 3.12: Se o Python for capaz de detectar que seu processo tem várias threads, a função *os.fork()* que esse método de inicialização chama internamente levantará *DeprecationWarning*. Use um método de inicialização diferente. Veja a documentação de *os.fork()* para mais explicações.

forkserver

Quando o programa é inicializado e seleciona o método *forkserver*, um processo de servidor é gerado. A partir disso, sempre que um novo processo é necessário, o processo pai conecta-se ao servidor e solicita que um novo processo seja feito um fork. O processo fork do servidor é de thread única, a menos que bibliotecas do sistema ou importações pré-carregadas gerem threads como um efeito colateral; neste sentido, geralmente é seguro usar *os.fork()*. Nenhum recurso desnecessário é herdado.

Disponível em plataformas POSIX que suportem a passagem de arquivo descritor para encadeamentos Unix, como Linux. O padrão nessas plataformas.

Alterado na versão 3.14: Isso se tornou o método de inicialização padrão nas plataformas POSIX.

Alterado na versão 3.4: *spawn* adicionado em todas as plataformas POSIX, e *forkserver* adicionado para algumas plataformas POSIX. Processos filhos não herdam mais todos os identificadores herdáveis dos pais no Windows.

Alterado na versão 3.8: No macOS, o método de inicialização *spawn* agora é o padrão. O método *fork* deve ser considerado inseguro, pois pode levar a travamentos do subprocesso, uma vez que as bibliotecas do sistema macOS podem iniciar threads. Consulte bpo-33725.

Alterado na versão 3.14: Nas plataformas POSIX, o método de inicialização padrão foi alterado de *fork* para *forkserver* para manter o desempenho, mas evitar incompatibilidades comuns de processos multithreads. Consulte gh-84559.

No POSIX, usar os métodos de inicialização *spawn* ou *forkserver* também iniciará um processo *rastreador de recursos* que rastreia os recursos de sistema nomeados não vinculados (como semáforos nomeados ou objetos *SharedMemory*) criados por processos do programa. Quando todos os processos tiverem saído, o rastreador de recursos desvincula qualquer objeto rastreado restante. Normalmente, não deve haver nenhum, mas se um processo foi morto por um sinal, pode haver alguns recursos "vazados". (Nem os semáforos vazados nem os segmentos de memória compartilhada serão desvinculados automaticamente até a próxima reinicialização do sistema. Isso é problemático para ambos os objetos porque o sistema permite apenas um número limitado de semáforos nomeados, e os segmentos de memória compartilhada ocupam algum espaço na memória principal.)

Para selecionar um método de inicialização, você usa *set_start_method()* na cláusula `if __name__ == '__main__'` do módulo principal. Por exemplo:

```
import multiprocessing as mp

def foo(q):
    q.put('hello')
```

(continua na próxima página)

(continuação da página anterior)

```
if __name__ == '__main__':
    mp.set_start_method('spawn')
    q = mp.Queue()
    p = mp.Process(target=foo, args=(q,))
    p.start()
    print(q.get())
    p.join()
```

set_start_method() não deve ser usada mais de uma vez no programa.

Alternativamente, você pode usar *get_context()* para obter um objeto de contexto. Objetos de contexto têm a mesma API que o módulo multiprocessing e permitem que se usem vários métodos de inicialização no mesmo programa.

```
import multiprocessing as mp

def foo(q):
    q.put('hello')

if __name__ == '__main__':
    ctx = mp.get_context('spawn')
    q = ctx.Queue()
    p = ctx.Process(target=foo, args=(q,))
    p.start()
    print(q.get())
    p.join()
```

Note que objetos relacionados a um contexto podem não ser compatíveis com processos para um contexto diferente. Em particular, travas criadas usando o contexto *fork* não podem ser passados para processos iniciados usando os métodos de inicialização *spawn* ou *forkserver*.

Libraries using *multiprocessing* or *ProcessPoolExecutor* should be designed to allow their users to provide their own multiprocessing context. Using a specific context of your own within a library can lead to incompatibilities with the rest of the library user's application. Always document if your library requires a specific start method.

Aviso

Os métodos de inicialização `'spawn'` e `'forkserver'` geralmente não podem ser usadas com executáveis "congelados" (por exemplo, binários produzidos por pacotes como **PyInstaller** e **cx_Freeze**) em sistemas POSIX. O método de inicialização `'fork'` pode funcionar se o código não usar threads.

Trocando objetos entre processos

multiprocessing tem suporte a dois tipos de canal de comunicação entre processos:

Filas

A classe *Queue* é quase um clone de *queue.Queue*. Por exemplo:

```
from multiprocessing import Process, Queue

def f(q):
    q.put([42, None, 'hello'])

if __name__ == '__main__':
    q = Queue()
    p = Process(target=f, args=(q,))
    p.start()
```

(continua na próxima página)

(continuação da página anterior)

```
    print(q.get())    # prints "[42, None, 'hello']"
    p.join()
```

As filas são seguras para threads e processos. Qualquer objeto colocado em uma fila *multiprocessing* será serializado.

Encadeamentos

A função *Pipe()* retorna um par de objetos de conexão conectados por um encadeamento que por padrão é duplex (bidirecional). Por exemplo:

```
from multiprocessing import Process, Pipe

def f(conn):
    conn.send([42, None, 'hello'])
    conn.close()

if __name__ == '__main__':
    parent_conn, child_conn = Pipe()
    p = Process(target=f, args=(child_conn,))
    p.start()
    print(parent_conn.recv())   # exibe "[42, None, 'hello']"
    p.join()
```

Os dois objetos de conexão retornados por *Pipe()* representam as duas extremidades do encadeamento. Cada objeto de conexão tem os métodos `send()` e `recv()` (entre outros). Observe que os dados em um encadeamento podem ser corrompidos se dois processos (ou threads) tentarem ler ou gravar na *mesma* extremidade do encadeamento ao mesmo tempo. Claro que não há risco de corrupção de processos usando extremidades diferentes do encadeamento ao mesmo tempo.

O método `send()` serializa o objeto e `recv()` recria o objeto.

Sincronização entre processos

multiprocessing contém equivalentes de todas as primitivas de sincronização de *threading*. Por exemplo, pode-se usar uma trava para garantir que apenas um processo exiba na saída padrão por vez:

```
from multiprocessing import Process, Lock

def f(l, i):
    l.acquire()
    try:
        print('hello world', i)
    finally:
        l.release()

if __name__ == '__main__':
    lock = Lock()

    for num in range(10):
        Process(target=f, args=(lock, num)).start()
```

Sem utilizar a saída da trava dos diferentes processos, é possível que tudo fique confuso.

Compartilhando estado entre processos

Conforme mencionado acima, ao fazer programação concorrente, geralmente é melhor evitar o uso de estado compartilhado, tanto quanto possível. Isso é particularmente verdadeiro ao utilizar múltiplos processos.

No entanto, se você realmente precisa utilizar algum compartilhamento de dados, então *multiprocessing* fornece algumas maneiras de se fazer isso.

Memória compartilhada

Os dados podem ser armazenados em um mapa de memória compartilhado utilizando *Value* ou *Array*. Por exemplo, o código a seguir

```
from multiprocessing import Process, Value, Array

def f(n, a):
    n.value = 3.1415927
    for i in range(len(a)):
        a[i] = -a[i]

if __name__ == '__main__':
    num = Value('d', 0.0)
    arr = Array('i', range(10))

    p = Process(target=f, args=(num, arr))
    p.start()
    p.join()

    print(num.value)
    print(arr[:])
```

vai exibir

```
3.1415927
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
```

Os argumentos `'d'` e `'i'` usados ao criar `num` e `arr` são typecodes do tipo usado pelo módulo *array*: `'d'` indica um ponto flutuante de precisão dupla e `'i'` indica um inteiro com sinal. Esses objetos compartilhados serão seguros para thread e processo.

Para mais flexibilidade no uso de memória compartilhada, pode-se utilizar o módulo *multiprocessing.sharedctypes*, que suporta a criação de objetos ctypes arbitrários alocados da memória compartilhada.

Processo de servidor

Um objeto gerenciador retornado por *Manager()* controla um processo de servidor que contém objetos Python e permite que outros processos os manipulem usando proxies.

A manager returned by *Manager()* will support types *list*, *dict*, *set*, *Namespace*, *Lock*, *RLock*, *Semaphore*, *BoundedSemaphore*, *Condition*, *Event*, *Barrier*, *Queue*, *Value* and *Array*. For example,

```
from multiprocessing import Process, Manager

def f(d, l, s):
    d[1] = '1'
    d['2'] = 2
    d[0.25] = None
    l.reverse()
    s.add('a')
    s.add('b')
```

(continua na próxima página)

(continuação da página anterior)

```
if __name__ == '__main__':
    with Manager() as manager:
        d = manager.dict()
        l = manager.list(range(10))
        s = manager.set()

        p = Process(target=f, args=(d, l, s))
        p.start()
        p.join()

        print(d)
        print(l)
        print(s)
```

vai exibir

```
{0.25: None, 1: '1', '2': 2}
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
{'a', 'b'}
```

Os gerenciadores de processos de servidor são mais flexíveis do que usar objetos de memória compartilhada porque podem ser feitos para oferecer suporte a tipos de objetos arbitrários. Além disso, um único gerenciador pode ser compartilhado por processos em diferentes computadores em uma rede. Eles são, no entanto, mais lentos do que usar memória compartilhada.

Usando um pool de workers

A classe *Pool* representa um pool de processos de worker. Ela tem métodos que permitem que tarefas sejam descarregadas para os processos de worker de algumas maneiras diferentes.

Por exemplo:

```
from multiprocessing import Pool, TimeoutError
import time
import os

def f(x):
    return x*x

if __name__ == '__main__':
    # inicia 4 processos de trabalhador
    with Pool(processes=4) as pool:

        # exibe "[0, 1, 4,..., 81]"
        print(pool.map(f, range(10)))

        # exibe mesmo números em ordem arbitrária
        for i in pool.imap_unordered(f, range(10)):
            print(i)

        # calcula "f(20)" assincronamente
        res = pool.apply_async(f, (20,))      # executa em *apenas* um processo
        print(res.get(timeout=1))             # exibe "400"

        # calcula "os.getpid()" assincronamente
        res = pool.apply_async(os.getpid, ()) # executa em *apenas* um procsso
```

(continua na próxima página)

(continuação da página anterior)

```
        print(res.get(timeout=1))             # exibe o PID daquele processo

        # iniciando vários cálculos de forma assíncrona *pod* usar mais procssos
        multiple_results = [pool.apply_async(os.getpid, ()) for i in range(4)]
        print([res.get(timeout=1) for res in multiple_results])

        # faz um único worker dormir por 10 segundos
        res = pool.apply_async(time.sleep, (10,))
        try:
            print(res.get(timeout=1))
        except TimeoutError:
            print("We lacked patience and got a multiprocessing.TimeoutError")

        print("For the moment, the pool remains available for more work")

    # saindo o bloco 'with' parou o pool
    print("Now the pool is closed and no longer available")
```

Observe que os métodos de um pool só devem ser usados pelo processo que o criou.

Nota

A funcionalidade dentro deste pacote requer que o módulo `__main__` seja importável pelos filhos. Isso é abordado em *Programming guidelines*, mas vale a pena apontar aqui. Isso significa que alguns exemplos, como os exemplos `multiprocessing.pool.Pool` não funcionarão no interpretador interativo. Por exemplo:

```
>>> from multiprocessing import Pool
>>> p = Pool(5)
>>> def f(x):
...     return x*x
...
>>> with p:
...     p.map(f, [1,2,3])
Process PoolWorker-1:
Process PoolWorker-2:
Process PoolWorker-3:
Traceback (most recent call last):
Traceback (most recent call last):
Traceback (most recent call last):
AttributeError: Can't get attribute 'f' on <module '__main__' (<class '_frozen_
↪importlib.BuiltinImporter'>)>
AttributeError: Can't get attribute 'f' on <module '__main__' (<class '_frozen_
↪importlib.BuiltinImporter'>)>
AttributeError: Can't get attribute 'f' on <module '__main__' (<class '_frozen_
↪importlib.BuiltinImporter'>)>
```

(Se você tentar isso, na verdade, serão gerados três tracebacks completos intercalados de forma semi-aleatória, e então você pode ter que interromper o processo pai de alguma forma.)

18.2.2 Referência

O pacote *multiprocessing* replica principalmente a API do módulo *threading*.

`Process` e exceções

class multiprocessing.**Process**(*group=None*, *target=None*, *name=None*, *args=()*, *kwargs={}*, ***, *daemon=None*)

Objetos processo representam atividades que são executadas em um processo separado. A classe *Process* possui equivalentes de todos os métodos de *threading.Thread*.

O construtor deve sempre ser chamado com argumentos nomeados. *group* deve sempre ser `None`; ele existe somente para compatibilidade com *threading.Thread*. *target* é o objeto chamável a ser invocado pelo método *run()*. O padrão é `None`, o que significa que nada é chamado. *name* é o nome do processo (veja *name* para mais detalhes). *args* é a tupla de argumento para a invocação de destino. *kwargs* é um dicionário de argumentos nomeados para a invocação de destino. Se fornecido, o argumento somente-nomeados *daemon* define o sinalizador do processo *daemon* como `True` ou `False`. Se `None` (o padrão), este sinalizador será herdado do processo de criação.

Por padrão, nenhum argumento é passado para *target*. O argumento *args*, que tem como padrão `()`, pode ser usado para especificar uma lista ou tupla de argumentos a serem passados para *target*.

If a subclass overrides the constructor, it must make sure it invokes the base class constructor (`super().__init__()`) before doing anything else to the process.

Nota

In general, all arguments to *Process* must be picklable. This is frequently observed when trying to create a *Process* or use a *concurrent.futures.ProcessPoolExecutor* from a REPL with a locally defined *target* function.

Passing a callable object defined in the current REPL session causes the child process to die via an uncaught *AttributeError* exception when starting as *target* must have been defined within an importable module in order to be loaded during unpickling.

Example of this uncatchable error from the child:

```
>>> import multiprocessing as mp
>>> def knigit():
...     print("Ni!")
...
>>> process = mp.Process(target=knigit)
>>> process.start()
>>> Traceback (most recent call last):
  File ".../multiprocessing/spawn.py", line ..., in spawn_main
  File ".../multiprocessing/spawn.py", line ..., in _main
AttributeError: module '__main__' has no attribute 'knigit'
>>> process
<SpawnProcess name='SpawnProcess-1' pid=379473 parent=378707 stopped␣
↪exitcode=1>
```

See *The spawn and forkserver start methods*. While this restriction is not true if using the `"fork"` start method, as of Python `3.14` that is no longer the default on any platform. See *Contextos e métodos de inicialização*. See also gh-132898.

Alterado na versão 3.3: Adicionado o parâmetro *daemon*.

run()

Método que representa a atividade do processo.

Você pode substituir esse método em uma subclasse. O método padrão *run()* invoca o objeto chamável passado ao construtor do objeto como o argumento alvo, se houver, com argumentos nomeados e

sequenciais retirados dos argumentos *args* e *kwargs*, respectivamente.

Usar uma lista ou tupla como argumento *args* passado para `Process` obtém o mesmo efeito.

Exemplo:

```
>>> from multiprocessing import Process
>>> p = Process(target=print, args=[1])
>>> p.run()
1
>>> p = Process(target=print, args=(1,))
>>> p.run()
1
```

start()

Inicia a atividade do processo.

Isso deve ser chamado no máximo uma vez por objeto processo. Ele organiza para que o método `run()` do objeto seja invocado em um processo separado.

join([*timeout*])

Se o argumento opcional *timeout* for `None` (o padrão), o método bloqueia até que o processo cujo método `join()` é chamado termine. Se *timeout* for um número positivo, ele bloqueia no máximo *timeout* segundos. Observe que o método retorna `None` se seu processo terminar ou se o método tiver tempo limite. Verifique o `exitcode` do processo para determinar se ele terminou.

Um processo pode ser usar "join" muitas vezes.

Um processo não pode se unir porque isso causaria um impasse. É um erro tentar se unir a um processo antes que ele tenha sido iniciado.

name

O nome do processo. O nome é uma string usada apenas para fins de identificação. Não tem semântica. Vários processos podem receber o mesmo nome.

O nome inicial é definido pelo construtor. Se nenhum nome explícito for fornecido ao construtor, um nome do formato 'Processo-N_1:N_2:…:N_k' é construído, onde cada N_k é o N-ésimo filho de seu pai.

is_alive()

Retorna se o processo está ativo.

Em termos gerais, um objeto processo está ativo desde o momento em que o método `start()` retorna até o término do processo filho.

daemon

O sinalizador daemon do processo, um valor Booleano. Isso deve ser definido antes de `start()` ser chamado.

O valor inicial é herdado do processo de criação.

Quando um processo sai, ele tenta encerrar todos os seus processos filhos daemônicos.

Note que um processo daemônico não tem permissão para criar processos filhos. Caso contrário, um processo daemônico deixaria seus filhos órfãos se ele fosse encerrado quando seu processo pai saísse. Além disso, esses **não** são daemons ou serviços Unix, eles são processos normais que serão encerrados (e em vez de usar "join") se processos não daemônicos tiverem saído.

Além da API `threading.Thread`, os objetos `Process` também oferecem suporte aos seguintes atributos e métodos:

pid

Retorna o ID do processo. Antes do processo ser gerado, este será `None`.

exitcode

O código de saída da criança. Este será `None` se o processo ainda não tiver terminado.

Se o método *run()* da criança retornar normalmente, o código de saída será 0. Se ele terminar via *sys.exit()* com um argumento inteiro *N*, o código de saída será *N*.

Se a criança for encerrada devido a uma exceção não capturada em *run()*, o código de saída será 1. Se ela for encerrada pelo sinal *N*, o código de saída será o valor negativo *-N*.

authkey

A chave de autenticação do processo (uma string de bytes).

Quando *multiprocessing* é inicializado, o processo principal recebe uma string aleatória usando *os.urandom()*.

Quando um objeto *Process* é criado, ele herda a chave de autenticação do seu processo pai, embora isso possa ser alterado definindo *authkey* para outra sequência de bytes.

Veja *Authentication keys*.

sentinel

Um identificador numérico de um objeto do sistema que ficará "pronto" quando o processo terminar.

Você pode usar esse valor se quiser esperar por vários eventos ao mesmo tempo usando *multiprocessing.connection.wait()*. Caso contrário, chamar *join()* é mais simples.

No Windows, este é um identificador de sistema operacional utilizável com a família de chamadas de API `WaitForSingleObject` e `WaitForMultipleObjects`. No POSIX, este é um descritor de arquivo utilizável com primitivos do módulo *select*.

Adicionado na versão 3.3.

interrupt()

Encerra o processo. Funciona em POSIX usando o sinal *SIGINT*. O comportamento no Windows é indefinido.

Por padrão, isso encerra o processo filho levantando *KeyboardInterrupt*. Esse comportamento pode ser alterado com a configuração do respectivo sinal manipulador no processo filho *signal.signal()* para *SIGINT*.

Observação: se o processo filho capturar e descartar *KeyboardInterrupt*, o processo não será encerrado.

Observação: o comportamento padrão também definirá *exitcode* para `1` como se um exceção não-capturada fosse levantada no filho processo. Para ter um *exitcode* diferente, você pode simplesmente capturar *KeyboardInterrupt* e chamar `exit(your_code)`.

Adicionado na versão 3.14.

terminate()

Encerra o processo. No POSIX isso é feito usando o sinal *SIGTERM*; no Windows é usado `TerminateProcess()`. Note que os manipuladores de saída e cláusulas finally, etc., não serão executados.

Observe que os processos descendentes do processo *não* serão encerrados — eles simplesmente ficarão órfãos.

> **Aviso**
>
> Se esse método for usado quando o processo associado estiver usando um encadeamento ou fila, então o encadeamento ou fila é passível de ser corrompido e pode se tornar inutilizável por outro processo. Similarmente, se o processo adquiriu um trava ou semáforo etc., então encerrá-lo é passível de causar impasse em outros processos.

kill()

O mesmo que *terminate()*, mas usando o sinal SIGKILL no POSIX.

Adicionado na versão 3.7.

close()

Fecha o objeto *Process*, liberando todos os recursos associados a ele. *ValueError* é levantado se o processo subjacente ainda estiver em execução. Uma vez que *close()* retorne com sucesso, a maioria dos outros métodos e atributos do objeto *Process* levantará *ValueError*.

Adicionado na versão 3.7.

Observe que os métodos *start()*, *join()*, *is_alive()*, *terminate()* e *exitcode* devem ser chamados somente pelo processo que criou o objeto processo.

Exemplo de uso de alguns dos métodos de *Process*:

```
>>> import multiprocessing, time, signal
>>> mp_context = multiprocessing.get_context('spawn')
>>> p = mp_context.Process(target=time.sleep, args=(1000,))
>>> print(p, p.is_alive())
<...Process ... initial> False
>>> p.start()
>>> print(p, p.is_alive())
<...Process ... started> True
>>> p.terminate()
>>> time.sleep(0.1)
>>> print(p, p.is_alive())
<...Process ... stopped exitcode=-SIGTERM> False
>>> p.exitcode == -signal.SIGTERM
True
```

exception multiprocessing.**ProcessError**

A classe base de todas as exceções de *multiprocessing*.

exception multiprocessing.**BufferTooShort**

Exceção levantada por Connection.recv_bytes_into() quando o objeto buffer fornecido é muito pequeno para a mensagem lida.

Se e for uma instância de *BufferTooShort*, então e.args[0] retornará a mensagem como uma string de bytes.

exception multiprocessing.**AuthenticationError**

Levantada quando há um erro de autenticação.

exception multiprocessing.**TimeoutError**

Levantada por métodos com um tempo limite quando o tempo limite expira.

Encadeamentos e filas

Ao usar vários processos, geralmente é usada a passagem de mensagens para comunicação entre processos e evita-se ter que usar quaisquer primitivas de sincronização, como travas.

Para passar mensagens, pode-se usar *Pipe()* (para uma conexão entre dois processos) ou uma fila (que permite múltiplos produtores e consumidores).

Os tipos *Queue*, *SimpleQueue* e *JoinableQueue* são filas FIFO multiprodutoras e multiconsumidoras modeladas na classe *queue.Queue* da biblioteca padrão. Elas diferem porque *Queue* não tem os métodos *task_done()* e *join()* introduzidos na classe *queue.Queue* do Python 2.5.

Se você usar *JoinableQueue*, então você **deve** chamar *JoinableQueue.task_done()* para cada tarefa removida da fila, caso contrário, o semáforo usado para contar o número de tarefas não concluídas pode eventualmente transbordar, levantando uma exceção.

Uma diferença de outras implementações de filas no Python é que as filas do *multiprocessing* serializam todos os objetos que são colocados nelas usando *pickle*. O objeto retornado pelo método get é um objeto recriado que não compartilha memória com o objeto original.

Observe que também é possível criar uma fila compartilhada usando um objeto gerenciador — veja *Gerenciadores*.

Nota

multiprocessing usa as exceções usuais *queue.Empty* e *queue.Full* para sinalizar um tempo limite. Elas não estão disponíveis no espaço de nomes do *multiprocessing*, então você precisa importá-las de *queue*.

Nota

Quando um objeto é colocado em uma fila, o objeto é serializado com pickle e uma thread em segundo plano depois descarrega os dados serializados com pickle para um encadeamento subjacente. Isso tem algumas consequências que são um pouco surpreendentes, mas não devem causar nenhuma dificuldade prática – se elas realmente o incomodam, então você pode usar uma fila criada com um *gerenciador*.

(1) Depois de colocar um objeto em uma fila vazia, pode haver um atraso infinitesimal antes que o método *empty()* da fila retorne *False* e *get_nowait()* possa retornar sem levantar *queue.Empty*.

(2) Se vários processos estiverem enfileirando objetos, é possível que os objetos sejam recebidos na outra extremidade fora de ordem. No entanto, objetos enfileirados pelo mesmo processo sempre estarão na ordem esperada em relação uns aos outros.

Aviso

Se um processo for morto usando *Process.terminate()* ou *os.kill()* enquanto estiver tentando usar uma *Queue*, os dados na fila provavelmente serão corrompidos. Isso pode fazer com que qualquer outro processo obtenha uma exceção quando tentar usar a fila mais tarde.

Aviso

Conforme mencionado acima, se um processo filho tiver colocado itens em uma fila (e não tiver usado *JoinableQueue.cancel_join_thread*), esse processo não será encerrado até que todos os itens armazenados em buffer tenham sido liberados para o encadeamento.

Isso significa que se você tentar juntar esse processo, poderá obter um impasse, a menos que tenha certeza de que todos os itens que foram colocados na fila foram consumidos. Da mesma forma, se o processo filho não for daemônico, o processo pai pode travar na saída quando tentar juntar todos os seus filhos não daemônicos.

Note que uma fila criada usando um gerenciador não tem esse problema. Veja *Programming guidelines*.

Para um exemplo do uso de filas para comunicação entre processos, veja *Exemplos*.

`multiprocessing.`**`Pipe`**`(`[*duplex*]`)`

Retorna um par `(conn1, conn2)` de objetos *Connection* representando as extremidades de um encadeamento.

Se *duplex* for `True` (o padrão), então o encadeamento é bidirecional. Se *duplex* for `False`, então o encadeamento é unidirecional: `conn1` pode ser usado somente para receber mensagens e `conn2` pode ser usado somente para enviar mensagens.

O método `send()` serializa o objeto usando *pickle* e `recv()` recria o objeto.

class multiprocessing.**Queue**([*maxsize*])

Retorna uma fila compartilhada de processo implementada usando um encadeamento e algumas travas/semáforos. Quando um processo coloca um item na fila pela primeira vez, uma thread alimentadora é iniciada, a qual transfere objetos de um buffer para o encadeamento.

As exceções usuais *queue.Empty* e *queue.Full* do módulo *queue* da biblioteca padrão são levantadas para sinalizar tempos limite.

Queue implementa todos os métodos de *queue.Queue*, exceto *task_done()* e *join()*.

qsize()

Retorna o tamanho aproximado da fila. Devido à semântica de multithreading/multiprocessamento, esse número não é confiável.

Observe que isso pode levantar *NotImplementedError* em plataformas como macOS, onde sem_getvalue() não está implementado.

empty()

Retorna True se a fila estiver vazia, False caso contrário. Devido à semântica de multithreading/multiprocessamento, isso não é confiável.

Pode levantar uma exceção *OSError* em filas fechadas. (não garantido)

full()

Retorna True se a fila estiver cheia, False caso contrário. Devido à semântica de multithreading/multiprocessamento, isso não é confiável.

put(*obj*[, *block*[, *timeout*]])

Coloca *obj* na fila. Se o argumento opcional *block* for True (o padrão) e *timeout* for None (o padrão), bloqueia se necessário até que um slot livre esteja disponível. Se *timeout* for um número positivo, ele bloqueia no máximo *timeout* segundos e levanta a exceção *queue.Full* se nenhum slot livre estiver disponível dentro desse tempo. Caso contrário (*block* é False), coloca um item na fila se um slot livre estiver imediatamente disponível, senão levanta a exceção *queue.Full* (*timeout* é ignorado nesse caso).

Alterado na versão 3.8: Se a fila for fechada, *ValueError* será levantada em vez de *AssertionError*.

put_nowait(*obj*)

Equivalente a put(obj, False).

get([*block*[, *timeout*]])

Remove e retorna um item da fila. Se os argumentos opcionais *block* forem True (o padrão) e *timeout* forem None (o padrão), bloqueia se necessário até que um item esteja disponível. Se *timeout* for um número positivo, ele bloqueia no máximo *timeout* segundos e levantada a exceção *queue.Empty* se nenhum item estiver disponível dentro desse tempo. Caso contrário (block for False), retorna um item se um estiver imediatamente disponível, senão levantada a exceção *queue.Empty* (*timeout* é ignorado nesse caso).

Alterado na versão 3.8: Se a fila for fechada, *ValueError* será levantada em vez de *OSError*.

get_nowait()

Equivalente a get(False).

multiprocessing.Queue tem alguns métodos adicionais não encontrados em *queue.Queue*. Esses métodos geralmente são desnecessários para a maioria dos códigos:

close()

Fecha a fila: libera recursos internos.

A queue must not be used anymore after it is closed. For example, *get()*, *put()* and *empty()* methods must no longer be called.

The background thread will quit once it has flushed all buffered data to the pipe. This is called automatically when the queue is garbage collected.

join_thread()

Junta a thread de segundo plano. Isso só pode ser usado depois que *close()* for chamado. Isso bloqueia até que a thread de segundo plano saia, garantindo que todos os dados no buffer tenham sido liberados para o encadeamento.

Por padrão, se um processo não for o criador da fila, ao sair, ele tentará se juntar ao thread de segundo plano da fila. O processo pode chamar *cancel_join_thread()* para fazer *join_thread()* não fazer nada.

cancel_join_thread()

Impede que *join_thread()* bloqueie. Em particular, isso impede que o thread de segundo plano seja unido automaticamente quando o processo sai – veja *join_thread()*.

Um nome melhor para esse método pode ser `allow_exit_without_flush()`. É provável que ele faça com que dados enfileirados sejam perdidos, e você quase certamente não precisará usá-lo. Ele está realmente lá somente se você precisar que o processo atual saia imediatamente sem esperar para liberar dados enfileirados para o encadeamento subjacente, e você não se importa com dados perdidos.

Nota

A funcionalidade desta classe requer uma implementação de semáforo compartilhado funcional no sistema operacional host. Sem uma, a funcionalidade nesta classe será desabilitada, e as tentativas de instanciar uma *Queue* resultarão em um *ImportError*. Veja bpo-3770 para informações adicionais. O mesmo vale para qualquer um dos tipos de fila especializados listados abaixo.

class multiprocessing.**SimpleQueue**

É um tipo *Queue* simplificado, muito próximo de um *Pipe* travado.

close()

Fecha a fila: libera recursos internos.

Uma fila não deve mais ser usada depois de fechada. Por exemplo, os métodos *get()*, *put()* e *empty()* não devem mais ser chamados.

Adicionado na versão 3.9.

empty()

Retorna `True` se a fila estiver vazia, `False` caso contrário.

Sempre levanta *OSError* se a SimpleQueue estiver fechada.

get()

Remove e retorna um item da fila.

put(*item*)

Coloca *item* na fila.

class multiprocessing.**JoinableQueue**([*maxsize*])

JoinableQueue, uma subclasse de *Queue*, é uma fila que também possui os métodos *task_done()* e *join()*.

task_done()

Indica que a tarefa anteriormente enfileirado está concluída. Para cada *get()* usado para buscar uma tarefa, uma chamada subsequente para *task_done()* avisa à fila, que o processamento na tarefa está concluído.

Se um *join()* estiver causando bloqueio no momento, ele irá continuar quando todos os itens tiverem sido processados (significando que uma chamada *task_done()* foi recebida para cada item que foi chamado o método *put()* para colocar na fila).

Levanta *ValueError* se for chamado mais vezes do que o número de itens colocados na fila.

join()

Bloqueia até que todos os itens na fila tenham sido obtidos e processados.

A contagem de tarefas inacabadas aumenta sempre que um item é adicionado à fila. A contagem diminui sempre que uma consumidora chama *task_done()* para indicar que o item foi recuperado e todo o trabalho nele foi concluído. Quando a contagem de tarefas inacabadas chega a zero, *join()* desbloqueia.

Diversos

multiprocessing.**active_children**()

Retorna a lista de todos os filhos ativos do processo atual.

Chamar isso tem o efeito colateral de "juntar" quaisquer processos que já tenham sido concluídos.

multiprocessing.**cpu_count**()

Retorna o número de CPUs no sistema.

Este número não equivale ao número de CPUs que o processo atual pode usar. O número de CPUs utilizáveis pode ser obtido com *os.process_cpu_count()* (ou `len(os.sched_getaffinity(0))`).

Quando o número de CPUs não pode ser determinado, uma *NotImplementedError* é levantada.

Ver também

os.cpu_count() os.process_cpu_count()

Alterado na versão 3.13: O valor de retorno também pode ser substituído usando o sinalizador `-X cpu_count` ou `PYTHON_CPU_COUNT`, pois este é apenas um invólucro em torno das APIs de contagem de CPU do *os*.

multiprocessing.**current_process**()

Retorna o objeto *Process* correspondente ao processo atual.

Um análogo de *threading.current_thread()*.

multiprocessing.**parent_process**()

Retorna o objeto *Process* correspondente ao processo pai do *current_process()*. Para o processo principal, `parent_process` será `None`.

Adicionado na versão 3.8.

multiprocessing.**freeze_support**()

Add support for when a program which uses *multiprocessing* has been frozen to produce an executable. (Has been tested with **py2exe**, **PyInstaller** and **cx_Freeze**.)

É preciso chamar esta função logo após a linha `if __name__ == '__main__'` do módulo principal. Por exemplo:

```
from multiprocessing import Process, freeze_support

def f():
    print('hello world!')

if __name__ == '__main__':
    freeze_support()
    Process(target=f).start()
```

Se a linha `freeze_support()` for omitida, tentar executar o executável congelado levantará *RuntimeError*.

Calling `freeze_support()` has no effect when the start method is not *spawn*. In addition, if the module is being run normally by the Python interpreter (the program has not been frozen), then `freeze_support()` has no effect.

multiprocessing.**get_all_start_methods**()

Retorna uma lista dos métodos de inicialização suportados, o primeiro dos quais é o padrão. Os métodos possíveis são `'fork'`, `'spawn'` e `'forkserver'`. Nem todas as plataformas suportam todos os métodos. Consulte *Contextos e métodos de inicialização*.

Adicionado na versão 3.4.

multiprocessing.**get_context**(*method=None*)

Retorna um objeto de contexto que possui os mesmos atributos do módulo *multiprocessing*.

Se *method* for `None`, então o contexto padrão será retornado. Observe que, se o método de inicialização global não tiver sido definido, isso o definirá como método padrão. Caso contrário, *method* deve ser `'fork'`, `'spawn'`, `'forkserver'`. *ValueError* é levantada se o método de inicialização não estiver disponível. Consulte *Contextos e métodos de inicialização*.

Adicionado na versão 3.4.

multiprocessing.**get_start_method**(*allow_none=False*)

Retorna o nome do método de inicialização usado para iniciar processos.

If the global start method has not been set and *allow_none* is `False`, then the start method is set to the default and the name is returned. If the start method has not been set and *allow_none* is `True` then `None` is returned.

O valor de retorno pode ser `'fork'`, `'spawn'`, `'forkserver'` ou `None`. Veja *Contextos e métodos de inicialização*.

Adicionado na versão 3.4.

Alterado na versão 3.8: No macOS, o método de inicialização *spawn* agora é o padrão. O método de inicialização *fork* deve ser considerado inseguro, pois pode levar a travamentos do subprocesso. Consulte bpo-33725.

multiprocessing.**set_executable**(*executable*)

Define o caminho do interpretador Python a ser usado ao iniciar um processo filho. (Por padrão, *sys.executable* é usado). Os incorporadores provavelmente precisarão fazer algo como

```
set_executable(os.path.join(sys.exec_prefix, 'pythonw.exe'))
```

antes que eles possam criar processos filho.

Alterado na versão 3.4: Agora com suporte no POSIX quando o método de inicialização `'spawn'` é usado.

Alterado na versão 3.11: Aceita um *objeto caminho ou similar*.

multiprocessing.**set_forkserver_preload**(*module_names*)

Define uma lista de nomes de módulos para o processo principal do forkserver tentar importar, de modo que seu estado já importado seja herdado por processos forks. Qualquer *ImportError* ao fazer isso é ignorado silenciosamente. Isso pode ser usado como um aprimoramento de desempenho para evitar trabalho repetido em cada processo.

Para que isso aconteça, trabalho deve ser chamado antes que o forkserver processar tenha sido iniciado (antes de criar um `Pool` ou iniciar um *Process*).

Só é significativo ao usar o método de inicialização `'forkserver'`. Consulte *Contextos e métodos de inicialização*.

Adicionado na versão 3.4.

multiprocessing.**set_start_method**(*method*, *force=False*)

Define o método que deve ser usado para iniciar processos filhos. O argumento *method* pode ser `'fork'`, `'spawn'` ou `'forkserver'`. Levanta *RuntimeError* se o método de inicialização já tiver sido definido e *force* não for `True`. Se *method* for `None` e *force* for `True`, o método de inicialização será definido como `None`. Se *method* for `None` e *force* for `False`, o contexto será definido como o contexto padrão.

Observe que isso deve ser chamado no máximo uma vez e deve ser protegido dentro da cláusula `if __name__ == '__main__'` do módulo principal.

Veja *Contextos e métodos de inicialização*.

Adicionado na versão 3.4.

Nota

multiprocessing não contém equivalentes a *threading.active_count()*, *threading.enumerate()*, *threading.settrace()*, *threading.setprofile()*, *threading.Timer* ou *threading.local*.

Objetos de conexão

Objetos de conexão permitem o envio e recebimento de strings e objetos que podem ser serializados com pickle. Eles podem ser pensados como sockets conectados orientados a mensagens.

Objetos de conexão geralmente são criados usando *Pipe* – veja também *Listeners and Clients*.

class multiprocessing.connection.**Connection**

send(*obj*)

Envia um objeto para a outra extremidade da conexão que deve ser lido usando *recv()*.

O objeto deve poder ser serializado com pickle. Serializações muito grandes com pickles (aproximadamente 32 MiB+, embora isso dependa do sistema operacional) podem levantar uma exceção *ValueError*.

recv()

Retorna um objeto enviado a partir da outra extremidade da conexão usando *send()*. Bloqueia até que haja algo para receber. Levanta *EOFError* se não houver mais nada para receber e a outra extremidade tenha sido fechada.

fileno()

Retorna o descritor de arquivo ou identificador usado pela conexão.

close()

Fecha a conexão.

Isso é chamado automaticamente quando a conexão é coletada como lixo.

poll([*timeout*])

Retorna se há algum dado disponível para leitura.

Se *timeout* não for especificado, ele retornará imediatamente. Se *timeout* for um número, isso especificará o tempo máximo em segundos para bloquear. Se *timeout* for `None`, um tempo limite infinito será usado.

Observe que vários objetos de conexão podem ser pesquisados ao mesmo tempo usando *multiprocessing.connection.wait()*.

send_bytes(*buffer*[, *offset*[, *size*]])

Enviar dados de bytes de um *objeto bytes ou similar* como uma mensagem completa.

Se *offset* for fornecido, os dados serão lidos daquela posição em *buffer*. Se *size* for fornecido, essa quantidade de bytes será lida do buffer. Buffers muito grandes (aproximadamente 32 MiB+, embora isso dependa do sistema operacional) podem levantar uma exceção *ValueError*

recv_bytes([*maxlength*])

Retorna uma mensagem completa de dados como bytes enviados a partir da outra extremidade da conexão como uma string. Bloqueia até que haja algo para receber. Levanta *EOFError* se não houver mais nada para receber e a outra extremidade tenha sido fechada.

Se *maxlength* for especificado e a mensagem for maior que *maxlength*, *OSError* será levantada e a conexão não será mais legível.

Alterado na versão 3.3: Esta função costumava levantada *IOError*, que agora é um apelido de *OSError*.

recv_bytes_into(*buffer*[, *offset*])

Lê para *buffer* uma mensagem completa de dados como bytes enviados a partir da outra extremidade da conexão e retorna o número de bytes na mensagem. Bloqueia até que haja algo para receber. Levanta *EOFError* se não houver mais nada para receber e a outra extremidade tenha sido fechada.

buffer deve ser um *objeto bytes ou similar* gravável. Se *offset* for fornecido, a mensagem será escrita no buffer a partir dessa posição. A posição deve ser um inteiro não negativo menor que o comprimento de *buffer* (em bytes).

Se o buffer for muito curto, uma exceção `BufferTooShort` será levantada e a mensagem completa estará disponível como `e.args[0]`, onde `e` é a instância da exceção.

Alterado na versão 3.3: Os próprios objetos de conexão agora podem ser transferidos entre processos usando *Connection.send()* e *Connection.recv()*.

Objetos de conexão agora também oferecem suporte ao protocolo de gerenciamento de contexto — veja *Tipos de Gerenciador de Contexto*. *__enter__()* retorna o objeto de conexão, e *__exit__()* chama *close()*.

Por exemplo:

```
>>> from multiprocessing import Pipe
>>> a, b = Pipe()
>>> a.send([1, 'hello', None])
>>> b.recv()
[1, 'hello', None]
>>> b.send_bytes(b'thank you')
>>> a.recv_bytes()
b'thank you'
>>> import array
>>> arr1 = array.array('i', range(5))
>>> arr2 = array.array('i', [0] * 10)
>>> a.send_bytes(arr1)
>>> count = b.recv_bytes_into(arr2)
>>> assert count == len(arr1) * arr1.itemsize
>>> arr2
array('i', [0, 1, 2, 3, 4, 0, 0, 0, 0, 0])
```

Aviso

O método *Connection.recv()* deserializa com picke automaticamente os dados recebidos, o que pode ser um risco à segurança, a menos que você possa confiar no processo que enviou a mensagem.

Portanto, a menos que o objeto de conexão tenha sido produzido usando `Pipe()`, você deve usar apenas os métodos *recv()* e *send()* após executar algum tipo de autenticação. Veja *Authentication keys*.

Aviso

Se um processo for encerrado enquanto estiver tentando ler ou escrever em um encadeamento, os dados no encadeamento provavelmente serão corrompidos, porque pode se tornar impossível ter certeza de onde estão os limites do encadeamento da mensagem.

Primitivas de sincronização

Geralmente, primitivas de sincronização não são tão necessárias em um programa multiprocesso quanto em um programa multithread. Veja a documentação do módulo *threading*.

Observe que também é possível criar primitivas de sincronização usando um objeto gerenciador — veja *Gerenciadores*.

class multiprocessing.**Barrier**(*parties*[, *action*[, *timeout*]])

Um objeto de barreira: um clone de *threading.Barrier*.

Adicionado na versão 3.3.

class multiprocessing.**BoundedSemaphore**([*value*])

Um objeto semáforo delimitado: um análogo próximo de *threading.BoundedSemaphore*.

Existe uma única diferença em relação ao seu análogo próximo: o primeiro argumento do método `acquire` é chamado *block*, como é consistente com *Lock.acquire()*.

> **Nota**
>
> No macOS, isso é indistinguível de *Semaphore* porque `sem_getvalue()` não está implementado nessa plataforma.

class multiprocessing.**Condition**([*lock*])

Uma variável de condição: um apelido para *threading.Condition*.

Se *lock* for especificado, ele deverá ser um objeto *Lock* ou *RLock* de *multiprocessing*.

Alterado na versão 3.3: O método *wait_for()* foi adicionado.

class multiprocessing.**Event**

Um clone de *threading.Event*.

class multiprocessing.**Lock**

Um objeto de trava não recursivo: um análogo próximo de *threading.Lock*. Uma vez que um processo ou thread tenha adquirido um trava, tentativas subsequentes de adquiri-la de qualquer processo ou thread bloquearão até que ele seja liberada; qualquer processo ou thread pode liberá-la. Os conceitos e comportamentos de *threading.Lock* conforme se aplica a threads são replicados aqui em *multiprocessing.Lock* conforme se aplica a processos ou threads, exceto conforme observado.

Observe que *Lock* é na verdade uma função de fábrica que retorna uma instância de `multiprocessing.synchronize.Lock` inicializada com um contexto padrão.

Lock oferece suporte ao protocolo *gerenciador de contexto* e, portanto, pode ser usado em instruções `with`.

acquire(*block=True*, *timeout=None*)

Adquire uma trava, bloqueante ou não.

Com o argumento *block* definido como `True` (o padrão), a chamada do método bloqueará até que a trava esteja em um estado destravado, então o definirá como travada e retornará `True`. Observe que o nome deste primeiro argumento difere daquele em *threading.Lock.acquire()*.

Com o argumento *block* definido como `False`, a chamada do método não bloqueia. Se a trava estiver atualmente em um estado travado, retorna `False`; caso contrário, defina a trava para um estado travado e retorna `True`.

Quando invocado com um valor positivo de ponto flutuante para *timeout*, bloqueie por no máximo o número de segundos especificado por *timeout* enquanto a trava não puder ser adquirida. Invocações com um valor negativo para *timeout* são equivalentes a um *timeout* de zero. Invocações com um valor *timeout* de `None` (o padrão) definem o período de tempo limite como infinito. Observe que o tratamento de valores negativos ou `None` para *timeout* difere do comportamento implementado em *threading.Lock.acquire()*. O argumento *timeout* não tem implicações práticas se o argumento *block* for definido como `False` e, portanto, for ignorado. Retorna `True` se a trava tiver sido adquirida ou `False` se o período de tempo limite tiver decorrido.

release()

Libera uma trava. Isso pode ser chamado de qualquer processo ou thread, não apenas do processo ou thread que originalmente adquiriu a trava.

O comportamento é o mesmo de *threading.Lock.release()*, exceto que quando invocado em uma trava desatravada, uma *ValueError* é levantada.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

class multiprocessing.**RLock**

Um objeto de trava recursiva: um análogo próximo de *threading.RLock*. Uma trava recursiva deve ser liberada pelo processo ou thread que o adquiriu. Uma vez que um processo ou thread tenha adquirido uma trava recursiva, o mesmo processo ou thread pode adquiri-la novamente sem trava; esse processo ou thread deve liberá-la uma vez para cada vez que ela foi adquirida.

Observe que *RLock* é na verdade uma função de fábrica que retorna uma instância de `multiprocessing.synchronize.RLock` inicializada com um contexto padrão.

RLock oferece suporte ao protocolo *gerenciador de contexto* e, portanto, pode ser usado em instruções `with`.

acquire(*block=True*, *timeout=None*)

Adquire uma trava, bloqueante ou não.

Quando invocado com o argumento *block* definido como `True`, bloqueia até que a trava esteja em um estado destravado (não pertencente a nenhum processo ou thread), a menos que a trava já seja de propriedade do processo ou thread atual. O processo ou thread atual então assume a propriedade da trava (se ainda não tiver propriedade) e o nível de recursão dentro de incrementos por um da trava, resultando em um valor de retorno de `True`. Observe que há várias diferenças no comportamento deste primeiro argumento em comparação com a implementação de *threading.RLock.acquire()*, começando pelo nome do argumento em si.

Quando invocado com o argumento *block* definido como `False`, não bloqueie. Se a trava já tiver sido adquirida (e, portanto, for de propriedade) por outro processo ou thread, o processo ou thread atual não assume a propriedade e o nível de recursão dentro da trava não é alterada, resultando em um valor de retorno de `False`. Se a trava estiver em um estado destravado, o processo ou thread atual assume a propriedade e o nível de recursão é incrementado, resultando em um valor de retorno de `True`.

O uso e os comportamentos do argumento *timeout* são os mesmos que em *Lock.acquire()*. Observe que alguns desses comportamentos de *timeout* diferem dos comportamentos implementados em *threading.RLock.acquire()*.

release()

Libera uma trava, decrementando o nível de recursão. Se após o decremento o nível de recursão for zero, redefine a trava para destravada (não pertencente a nenhum processo ou thread) e se quaisquer outros processos ou threads estiverem bloqueados esperando a trava ser destravada, permita que exatamente um deles prossiga. Se após o decremento o nível de recursão ainda for diferente de zero, o trava permanece travada e pertencente ao processo ou thread de chamada.

Somente chame esse método quando o processo ou thread de chamada for proprietária da trava. Uma *AssertionError* é levantada se esse método for chamado por um processo ou thread diferente do proprietário ou se a trava estiver em um estado destravado (sem proprietário). Observe que o tipo de exceção levantada nessa situação difere do comportamento implementado em *threading.RLock.release()*.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

class multiprocessing.**Semaphore**([*value*])

Um objeto semáforo: um análogo próximo de *threading.Semaphore*.

Existe uma única diferença em relação ao seu análogo próximo: o primeiro argumento do método `acquire` é chamado *block*, como é consistente com *Lock.acquire()*.

Nota

No macOS, não há suporte a `sem_timedwait`, então chamar `acquire()` com um tempo limite emulará o comportamento dessa função usando um laço de suspensão.

Nota

Algumas das funcionalidades deste pacote exibem uma implementação de semáforo compartilhado funcional no sistema operacional host. Sem uma, o módulo `multiprocessing.synchronize` será desabilitado e as tentativas de importação dele resultarão em um *`ImportError`*. Veja bpo-3770 para informações adicionais.

Objetos `ctypes` compartilhados

É possível criar objetos compartilhados usando memória compartilhada que pode ser herdada por processos filhos.

`multiprocessing.`**`Value`**(*typecode_or_type*, **args*, *lock=True*)

Retorna um objeto *`ctypes`* alocado da memória compartilhada. Por padrão, o valor de retorno é, na verdade, um invólucro sincronizado para o objeto. O objeto em si pode ser acessado por meio do atributo *value* de um *`Value`*.

typecode_or_type determina o tipo do objeto retornado: é um tipo ctypes ou um typecode de um caractere do tipo usado pelo módulo *`array`*. **args* é passado para o construtor para o tipo.

Se *lock* for `True` (o padrão), então um novo objeto de trava recursiva é criado para sincronizar o acesso ao valor. Se *lock* for um objeto *`Lock`* ou *`RLock`*, então ele será usado para sincronizar o acesso ao valor. Se *lock* for `False`, então o acesso ao objeto retornado não será protegido automaticamente por uma trava, então ele não será necessariamente "seguro para processo".

Operações como += que envolvem uma leitura e escrita não são atômicas. Então se, por exemplo, você quiser incrementar atomicamente um valor compartilhado, não é suficiente apenas fazer

```
counter.value += 1
```

Supondo que a trava associada seja recursiva (o que é por padrão), você pode fazer

```
with counter.get_lock():
    counter.value += 1
```

Observe que *lock* é um argumento somente-nomeado.

`multiprocessing.`**`Array`**(*typecode_or_type*, *size_or_initializer*, ***, *lock=True*)

Retorna um vetor de ctypes alocado da memória compartilhada. Por padrão, o valor de retorno é, na verdade, um invólucro sincronizado para o vetor.

typecode_or_type determina o tipo dos elementos do vetor retornado: é um tipo ctypes ou um typecode de um caractere do tipo usado pelo módulo *`array`*. Se *size_or_initializer* for um inteiro, então ele determina o comprimento do vetor, e o vetor será inicialmente zerado. Caso contrário, *size_or_initializer* é uma sequência que é usada para inicializar o vetor e cujo comprimento determina o comprimento do vetor.

Se *lock* for `True` (o padrão), então um novo objeto de trava é criado para sincronizar o acesso ao valor. Se *lock* for um objeto *`Lock`* ou *`RLock`*, então ele será usado para sincronizar o acesso ao valor. Se *lock* for `False`, então o acesso ao objeto retornado não será protegido automaticamente por uma trava, então ele não será necessariamente "seguro para processo".

Observe que *lock* é um argumento somente-nomeado.

Observe que um vetor de *`ctypes.c_char`* tem atributos *value* e *raw* que permitem usá-lo para armazenar e recuperar strings.

O módulo `multiprocessing.sharedctypes`

O módulo *multiprocessing.sharedctypes* fornece funções para alocar objetos *ctypes* da memória compartilhada que podem ser herdados por processos filhos.

> **Nota**
>
> Embora seja possível armazenar um ponteiro na memória compartilhada, lembre-se de que isso se referirá a um local no espaço de endereço de um processo específico. No entanto, é bem provável que o ponteiro seja inválido no contexto de um segundo processo e tentar desreferenciar o ponteiro do segundo processo pode causar um travamento.

`multiprocessing.sharedctypes.`**`RawArray`**(*typecode_or_type*, *size_or_initializer*)

Retorna um vetor de ctypes alocado da memória compartilhada.

typecode_or_type determina o tipo dos elementos do vetor retornado: é um tipo ctypes ou um typecode de um caractere do tipo usado pelo módulo *array*. Se *size_or_initializer* for um inteiro, então ele determina o comprimento do vetor, e o vetor será inicialmente zerado. Caso contrário, *size_or_initializer* é uma sequência que é usada para inicializar o vetor e cujo comprimento determina o comprimento do vetor.

Observe que definir e obter um elemento é potencialmente não atômico – use *Array()* para garantir que o acesso seja sincronizado automaticamente usando uma trava.

`multiprocessing.sharedctypes.`**`RawValue`**(*typecode_or_type*, **args*)

Retorna um objeto ctypes alocado da memória compartilhada.

typecode_or_type determina o tipo do objeto retornado: é um tipo ctypes ou um typecode de um caractere do tipo usado pelo módulo *array*. **args* é passado para o construtor para o tipo.

Observe que definir e obter o valor é potencialmente não atômico – use *Value()* para garantir que o acesso seja sincronizado automaticamente usando uma trava.

Observe que um vetor de *ctypes.c_char* tem atributos `value` e `raw` que permitem usá-la para armazenar e recuperar strings – veja a documentação de *ctypes*.

`multiprocessing.sharedctypes.`**`Array`**(*typecode_or_type*, *size_or_initializer*, *, *lock=True*)

O mesmo que *RawArray()*, exceto que dependendo do valor de *lock*, um invólucro de sincronização seguro para processo pode ser retornado em vez de um vetor de ctypes brutos.

Se *lock* for `True` (o padrão), então um novo objeto de trava é criado para sincronizar o acesso ao valor. Se *lock* for um objeto *Lock* ou *RLock*, então ele será usado para sincronizar o acesso ao valor. Se *lock* for `False`, então o acesso ao objeto retornado não será protegido automaticamente por uma trava, então ele não será necessariamente "seguro para processo".

Observe que *lock* é um argumento somente-nomeado.

`multiprocessing.sharedctypes.`**`Value`**(*typecode_or_type*, **args*, *lock=True*)

O mesmo que *RawValue()*, exceto que dependendo do valor de *lock*, um invólucro de sincronização seguro para processo pode ser retornado em vez de um objeto ctypes brutos.

Se *lock* for `True` (o padrão), então um novo objeto de trava é criado para sincronizar o acesso ao valor. Se *lock* for um objeto *Lock* ou *RLock*, então ele será usado para sincronizar o acesso ao valor. Se *lock* for `False`, então o acesso ao objeto retornado não será protegido automaticamente por uma trava, então ele não será necessariamente "seguro para processo".

Observe que *lock* é um argumento somente-nomeado.

`multiprocessing.sharedctypes.`**`copy`**(*obj*)

Retorna um objeto ctypes alocado da memória compartilhada que é uma cópia do objeto ctypes *obj*.

`multiprocessing.sharedctypes.`**`synchronized`**(*obj*[, *lock*])

Retorna um objeto invólucro seguro para o processo para um objeto ctypes que usa *lock* para sincronizar o acesso. Se *lock* for `None` (o padrão), então um objeto *multiprocessing.RLock* é criado automaticamente.

A synchronized wrapper will have two methods in addition to those of the object it wraps: `get_obj()` returns the wrapped object and `get_lock()` returns the lock object used for synchronization.

Note that accessing the ctypes object through the wrapper can be a lot slower than accessing the raw ctypes object.

Alterado na versão 3.5: Synchronized objects support the *context manager* protocol.

The table below compares the syntax for creating shared ctypes objects from shared memory with the normal ctypes syntax. (In the table `MyStruct` is some subclass of *ctypes.Structure*.)

ctypes	sharedctypes using type	sharedctypes using typecode
c_double(2.4)	RawValue(c_double, 2.4)	RawValue('d', 2.4)
MyStruct(4, 6)	RawValue(MyStruct, 4, 6)	
(c_short * 7)()	RawArray(c_short, 7)	RawArray('h', 7)
(c_int * 3)(9, 2, 8)	RawArray(c_int, (9, 2, 8))	RawArray('i', (9, 2, 8))

Below is an example where a number of ctypes objects are modified by a child process:

```
from multiprocessing import Process, Lock
from multiprocessing.sharedctypes import Value, Array
from ctypes import Structure, c_double

class Point(Structure):
    _fields_ = [('x', c_double), ('y', c_double)]

def modify(n, x, s, A):
    n.value **= 2
    x.value **= 2
    s.value = s.value.upper()
    for a in A:
        a.x **= 2
        a.y **= 2

if __name__ == '__main__':
    lock = Lock()

    n = Value('i', 7)
    x = Value(c_double, 1.0/3.0, lock=False)
    s = Array('c', b'hello world', lock=lock)
    A = Array(Point, [(1.875,-6.25), (-5.75,2.0), (2.375,9.5)], lock=lock)

    p = Process(target=modify, args=(n, x, s, A))
    p.start()
    p.join()

    print(n.value)
    print(x.value)
    print(s.value)
    print([(a.x, a.y) for a in A])
```

The results printed are

```
49
0.1111111111111111
HELLO WORLD
[(3.515625, 39.0625), (33.0625, 4.0), (5.640625, 90.25)]
```

Gerenciadores

Managers provide a way to create data which can be shared between different processes, including sharing over a network between processes running on different machines. A manager object controls a server process which manages *shared objects*. Other processes can access the shared objects by using proxies.

`multiprocessing.`**`Manager`**`()`

Retorna um objeto *`SyncManager`* inicializado que pode ser usado para compartilhar objetos entre processos. O objeto gerenciador retornado corresponde a um processo filho gerado e tem métodos que criarão objetos compartilhados e retornarão proxies correspondentes.

Manager processes will be shutdown as soon as they are garbage collected or their parent process exits. The manager classes are defined in the *`multiprocessing.managers`* module:

`class` `multiprocessing.managers.`**`BaseManager`**`(`*address=None*, *authkey=None*, *serializer='pickle'*, *ctx=None*, ***, *shutdown_timeout=1.0*`)`

Criando um objeto BaseManager.

Once created one should call *`start()`* or `get_server().serve_forever()` to ensure that the manager object refers to a started manager process.

address is the address on which the manager process listens for new connections. If *address* is `None` then an arbitrary one is chosen.

authkey is the authentication key which will be used to check the validity of incoming connections to the server process. If *authkey* is `None` then `current_process().authkey` is used. Otherwise *authkey* is used and it must be a byte string.

serializer must be `'pickle'` (use *`pickle`* serialization) or `'xmlrpclib'` (use *`xmlrpc.client`* serialization).

ctx is a context object, or `None` (use the current context). See the `get_context()` function.

shutdown_timeout is a timeout in seconds used to wait until the process used by the manager completes in the *`shutdown()`* method. If the shutdown times out, the process is terminated. If terminating the process also times out, the process is killed.

Alterado na versão 3.11: Added the *shutdown_timeout* parameter.

`start``(`[*initializer*[, *initargs*]]`)`

Start a subprocess to start the manager. If *initializer* is not `None` then the subprocess will call `initializer(*initargs)` when it starts.

`get_server``()`

Returns a `Server` object which represents the actual server under the control of the Manager. The `Server` object supports the `serve_forever()` method:

```
>>> from multiprocessing.managers import BaseManager
>>> manager = BaseManager(address=('', 50000), authkey=b'abc')
>>> server = manager.get_server()
>>> server.serve_forever()
```

`Server` additionally has an *`address`* attribute.

`connect``()`

Connect a local manager object to a remote manager process:

```
>>> from multiprocessing.managers import BaseManager
>>> m = BaseManager(address=('127.0.0.1', 50000), authkey=b'abc')
>>> m.connect()
```

`shutdown``()`

Stop the process used by the manager. This is only available if *`start()`* has been used to start the server process.

This can be called multiple times.

register(*typeid*[, *callable*[, *proxytype*[, *exposed*[, *method_to_typeid*[, *create_method*]]]]])

A classmethod which can be used for registering a type or callable with the manager class.

typeid is a "type identifier" which is used to identify a particular type of shared object. This must be a string.

callable is a callable used for creating objects for this type identifier. If a manager instance will be connected to the server using the *connect()* method, or if the *create_method* argument is `False` then this can be left as `None`.

proxytype is a subclass of *BaseProxy* which is used to create proxies for shared objects with this *typeid*. If `None` then a proxy class is created automatically.

exposed is used to specify a sequence of method names which proxies for this typeid should be allowed to access using *BaseProxy._callmethod()*. (If *exposed* is `None` then `proxytype._exposed_` is used instead if it exists.) In the case where no exposed list is specified, all "public methods" of the shared object will be accessible. (Here a "public method" means any attribute which has a `__call__()` method and whose name does not begin with `'_'`.)

method_to_typeid is a mapping used to specify the return type of those exposed methods which should return a proxy. It maps method names to typeid strings. (If *method_to_typeid* is `None` then `proxytype._method_to_typeid_` is used instead if it exists.) If a method's name is not a key of this mapping or if the mapping is `None` then the object returned by the method will be copied by value.

create_method determines whether a method should be created with name *typeid* which can be used to tell the server process to create a new shared object and return a proxy for it. By default it is `True`.

BaseManager instances also have one read-only property:

address

The address used by the manager.

Alterado na versão 3.3: Manager objects support the context management protocol – see *Tipos de Gerenciador de Contexto*. *__enter__()* starts the server process (if it has not already started) and then returns the manager object. *__exit__()* calls *shutdown()*.

In previous versions *__enter__()* did not start the manager's server process if it was not already started.

class multiprocessing.managers.**SyncManager**

A subclass of *BaseManager* which can be used for the synchronization of processes. Objects of this type are returned by *multiprocessing.Manager()*.

Its methods create and return *Objetos proxies* for a number of commonly used data types to be synchronized across processes. This notably includes shared lists and dictionaries.

Barrier(*parties*[, *action*[, *timeout*]])

Create a shared *threading.Barrier* object and return a proxy for it.

Adicionado na versão 3.3.

BoundedSemaphore([*value*])

Create a shared *threading.BoundedSemaphore* object and return a proxy for it.

Condition([*lock*])

Create a shared *threading.Condition* object and return a proxy for it.

If *lock* is supplied then it should be a proxy for a *threading.Lock* or *threading.RLock* object.

Alterado na versão 3.3: O método *wait_for()* foi adicionado.

Event()

Create a shared *threading.Event* object and return a proxy for it.

Lock()

Create a shared *threading.Lock* object and return a proxy for it.

Namespace()

Create a shared *Namespace* object and return a proxy for it.

Queue([*maxsize*])

Create a shared *queue.Queue* object and return a proxy for it.

RLock()

Create a shared *threading.RLock* object and return a proxy for it.

Semaphore([*value*])

Create a shared *threading.Semaphore* object and return a proxy for it.

Array(*typecode*, *sequence*)

Create an array and return a proxy for it.

Value(*typecode*, *value*)

Create an object with a writable `value` attribute and return a proxy for it.

dict()

dict(*mapping*)

dict(*sequence*)

Create a shared *dict* object and return a proxy for it.

list()

list(*sequence*)

Create a shared *list* object and return a proxy for it.

set()

set(*sequence*)

set(*mapping*)

Cria um objeto *set* compartilhado e retorna um proxy para ele.

Adicionado na versão 3.14: Foi adicionado suporte para *set*.

Alterado na versão 3.6: Objetos compartilhados podem ser aninhados. Por exemplo, em um objeto contêiner compartilhado, como uma lista compartilhada, é possível conter outros objeto compartilhados que serão gerenciados e sincronizados pelo *SyncManager*.

class multiprocessing.managers.**Namespace**

A type that can register with *SyncManager*.

A namespace object has no public methods, but does have writable attributes. Its representation shows the values of its attributes.

However, when using a proxy for a namespace object, an attribute beginning with `'_'` will be an attribute of the proxy and not an attribute of the referent:

```
>>> mp_context = multiprocessing.get_context('spawn')
>>> manager = mp_context.Manager()
>>> Global = manager.Namespace()
>>> Global.x = 10
>>> Global.y = 'hello'
>>> Global._z = 12.3    # this is an attribute of the proxy
>>> print(Global)
Namespace(x=10, y='hello')
```

Customized managers

To create one's own manager, one creates a subclass of *BaseManager* and uses the *register()* classmethod to register new types or callables with the manager class. For example:

```
from multiprocessing.managers import BaseManager

class MathsClass:
    def add(self, x, y):
        return x + y
    def mul(self, x, y):
        return x * y

class MyManager(BaseManager):
    pass

MyManager.register('Maths', MathsClass)

if __name__ == '__main__':
    with MyManager() as manager:
        maths = manager.Maths()
        print(maths.add(4, 3))         # prints 7
        print(maths.mul(7, 8))         # prints 56
```

Using a remote manager

It is possible to run a manager server on one machine and have clients use it from other machines (assuming that the firewalls involved allow it).

Running the following commands creates a server for a single shared queue which remote clients can access:

```
>>> from multiprocessing.managers import BaseManager
>>> from queue import Queue
>>> queue = Queue()
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue', callable=lambda:queue)
>>> m = QueueManager(address=('', 50000), authkey=b'abracadabra')
>>> s = m.get_server()
>>> s.serve_forever()
```

One client can access the server as follows:

```
>>> from multiprocessing.managers import BaseManager
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue')
>>> m = QueueManager(address=('foo.bar.org', 50000), authkey=b'abracadabra')
>>> m.connect()
>>> queue = m.get_queue()
>>> queue.put('hello')
```

Another client can also use it:

```
>>> from multiprocessing.managers import BaseManager
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue')
>>> m = QueueManager(address=('foo.bar.org', 50000), authkey=b'abracadabra')
>>> m.connect()
>>> queue = m.get_queue()
```

(continua na próxima página)

(continuação da página anterior)

```
>>> queue.get()
'hello'
```

Local processes can also access that queue, using the code from above on the client to access it remotely:

```
>>> from multiprocessing import Process, Queue
>>> from multiprocessing.managers import BaseManager
>>> class Worker(Process):
...     def __init__(self, q):
...         self.q = q
...         super().__init__()
...     def run(self):
...         self.q.put('local hello')
...
>>> queue = Queue()
>>> w = Worker(queue)
>>> w.start()
>>> class QueueManager(BaseManager): pass
...
>>> QueueManager.register('get_queue', callable=lambda: queue)
>>> m = QueueManager(address=('', 50000), authkey=b'abracadabra')
>>> s = m.get_server()
>>> s.serve_forever()
```

Objetos proxies

Um proxy é um objeto que *refere-se* a um objeto compartilhado que reside (presumivelmente) em um processo diferente. Dizemos que o objeto compartilhado é o *referente* do proxy. Vários objetos proxies podem ter o mesmo referente.

Um proxy objeto tem métodos que invocam métodos correspondes de seu referente (embora nem todo método do referente esteja necessariamente disponível por meio do proxy). Dessa forma, um proxy pode ser usado da mesma forma que seu referente:

```
>>> mp_context = multiprocessing.get_context('spawn')
>>> manager = mp_context.Manager()
>>> l = manager.list([i*i for i in range(10)])
>>> print(l)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> print(repr(l))
<ListProxy object, typeid 'list' at 0x...>
>>> l[4]
16
>>> l[2:5]
[4, 9, 16]
```

Observe que, ao aplicar *str()* a um proxy, o retorno representará o referente, enquanto que ao aplicar o *repr()*, o retorno representará o proxy.

Um aspecto importante dos objetos proxies é o fato de que eles podem ser serializados com pickle, portanto podem ser transferidos entre processos. Dessa forma, um referente pode conter *Objetos proxies*. Isso permite o aninhamento dessas listas e dicionários gerenciados e outros *Objetos proxies*:

```
>>> a = manager.list()
>>> b = manager.list()
>>> a.append(b)         # referente de a agora contém referente de b
>>> print(a, b)
```

(continua na próxima página)

(continuação da página anterior)

```
[<ListProxy object, typeid 'list' at ...>] []
>>> b.append('hello')
>>> print(a[0], b)
['hello'] ['hello']
```

Da mesma forma, os proxies de dicionários e listas podem ser aninhados um dentro do outro:

```
>>> l_outer = manager.list([ manager.dict() for i in range(2) ])
>>> d_first_inner = l_outer[0]
>>> d_first_inner['a'] = 1
>>> d_first_inner['b'] = 2
>>> l_outer[1]['c'] = 3
>>> l_outer[1]['z'] = 26
>>> print(l_outer[0])
{'a': 1, 'b': 2}
>>> print(l_outer[1])
{'c': 3, 'z': 26}
```

Se objetos padrões (não-proxy) de *list* ou *dict* estiverem contidos em um referente, as modificações nesses mutáveis não serão propagadas pelo gerente porque o proxy não tem como saber quando os valores contidos nele são modificados. No entanto, o armazenamento de um valor em um proxy contêiner (que aciona um `__setitem__` no objeto proxy) se propaga pelo gerenciador e, portanto, para modificar efetivamente esse item, é possível reatribuir o valor modificado ao proxy contêiner:

```
# cria um proxy para uma lista e adicionar um objeto mutável (um dicionário)
lproxy = manager.list()
lproxy.append({})
# agora modifica o dicionário
d = lproxy[0]
d['a'] = 1
d['b'] = 2
# neste ponto, as mudanças em d não estão ainda sincronizadas, mas
# atualizando o dicionário, o proxy é notificado da mudança
lproxy[0] = d
```

Essa abordagem talvez seja menos conveniente do que usar *Objetos proxies* aninhados para a maioria dos casos de uso, mas também demonstra um nível de controle sobre a sincronização.

Nota

Os tipos de proxies em *multiprocessing* não fazem nada com as comparações de valor. Portanto, por exemplo, temos:

```
>>> manager.list([1,2,3]) == [1,2,3]
False
```

Em vez disso, deve-se usar uma cópia do referente ao fazer comparações.

class multiprocessing.managers.**BaseProxy**

Proxy objects are instances of subclasses of *BaseProxy*.

_callmethod(*methodname*[, *args*[, *kwds*]])

Call and return the result of a method of the proxy's referent.

If `proxy` is a proxy whose referent is `obj` then the expression

```
proxy._callmethod(methodname, args, kwds)
```

will evaluate the expression

```
getattr(obj, methodname)(*args, **kwds)
```

in the manager's process.

The returned value will be a copy of the result of the call or a proxy to a new shared object – see documentation for the *method_to_typeid* argument of *BaseManager.register()*.

If an exception is raised by the call, then is re-raised by *_callmethod()*. If some other exception is raised in the manager's process then this is converted into a `RemoteError` exception and is raised by *_callmethod()*.

Note in particular that an exception will be raised if *methodname* has not been *exposed*.

An example of the usage of *_callmethod()*:

```
>>> l = manager.list(range(10))
>>> l._callmethod('__len__')
10
>>> l._callmethod('__getitem__', (slice(2, 7),)) # equivalent to l[2:7]
[2, 3, 4, 5, 6]
>>> l._callmethod('__getitem__', (20,))          # equivalent to l[20]
Traceback (most recent call last):
...
IndexError: list index out of range
```

_getvalue()

Return a copy of the referent.

If the referent is unpicklable then this will raise an exception.

__repr__()

Return a representation of the proxy object.

__str__()

Return the representation of the referent.

Limpeza

A proxy object uses a weakref callback so that when it gets garbage collected it deregisters itself from the manager which owns its referent.

A shared object gets deleted from the manager process when there are no longer any proxies referring to it.

Process Pools

One can create a pool of processes which will carry out tasks submitted to it with the *Pool* class.

class multiprocessing.pool.**Pool**([*processes*[, *initializer*[, *initargs*[, *maxtasksperchild*[, *context*]]]]])

A process pool object which controls a pool of worker processes to which jobs can be submitted. It supports asynchronous results with timeouts and callbacks and has a parallel map implementation.

processes is the number of worker processes to use. If *processes* is `None` then the number returned by *os.process_cpu_count()* is used.

If *initializer* is not `None` then each worker process will call `initializer(*initargs)` when it starts.

maxtasksperchild is the number of tasks a worker process can complete before it will exit and be replaced with a fresh worker process, to enable unused resources to be freed. The default *maxtasksperchild* is `None`, which means worker processes will live as long as the pool.

context can be used to specify the context used for starting the worker processes. Usually a pool is created using the function `multiprocessing.Pool()` or the *Pool()* method of a context object. In both cases *context* is set appropriately.

Note that the methods of the pool object should only be called by the process which created the pool.

Aviso

multiprocessing.pool objects have internal resources that need to be properly managed (like any other resource) by using the pool as a context manager or by calling *close()* and *terminate()* manually. Failure to do this can lead to the process hanging on finalization.

Note that it is **not correct** to rely on the garbage collector to destroy the pool as CPython does not assure that the finalizer of the pool will be called (see `object.__del__()` for more information).

Alterado na versão 3.2: Added the *maxtasksperchild* parameter.

Alterado na versão 3.4: Adicionado o parâmetro *context*.

Alterado na versão 3.13: *processes* uses *os.process_cpu_count()* by default, instead of *os.cpu_count()*.

Nota

Worker processes within a *Pool* typically live for the complete duration of the Pool's work queue. A frequent pattern found in other systems (such as Apache, mod_wsgi, etc) to free resources held by workers is to allow a worker within a pool to complete only a set amount of work before being exiting, being cleaned up and a new process spawned to replace the old one. The *maxtasksperchild* argument to the *Pool* exposes this ability to the end user.

apply(*func*[, *args*[, *kwds*]])

Call *func* with arguments *args* and keyword arguments *kwds*. It blocks until the result is ready. Given this blocks, *apply_async()* is better suited for performing work in parallel. Additionally, *func* is only executed in one of the workers of the pool.

apply_async(*func*[, *args*[, *kwds*[, *callback*[, *error_callback*]]]])

A variant of the *apply()* method which returns a *AsyncResult* object.

If *callback* is specified then it should be a callable which accepts a single argument. When the result becomes ready *callback* is applied to it, that is unless the call failed, in which case the *error_callback* is applied instead.

If *error_callback* is specified then it should be a callable which accepts a single argument. If the target function fails, then the *error_callback* is called with the exception instance.

Callbacks should complete immediately since otherwise the thread which handles the results will get blocked.

map(*func*, *iterable*[, *chunksize*])

A parallel equivalent of the *map()* built-in function (it supports only one *iterable* argument though, for multiple iterables see *starmap()*). It blocks until the result is ready.

This method chops the iterable into a number of chunks which it submits to the process pool as separate tasks. The (approximate) size of these chunks can be specified by setting *chunksize* to a positive integer.

Note that it may cause high memory usage for very long iterables. Consider using *imap()* or *imap_unordered()* with explicit *chunksize* option for better efficiency.

map_async(*func*, *iterable*[, *chunksize*[, *callback*[, *error_callback*]]])

A variant of the *map()* method which returns a *AsyncResult* object.

If *callback* is specified then it should be a callable which accepts a single argument. When the result becomes ready *callback* is applied to it, that is unless the call failed, in which case the *error_callback* is applied instead.

If *error_callback* is specified then it should be a callable which accepts a single argument. If the target function fails, then the *error_callback* is called with the exception instance.

Callbacks should complete immediately since otherwise the thread which handles the results will get blocked.

imap(*func*, *iterable*[, *chunksize*])

A lazier version of `map()`.

The *chunksize* argument is the same as the one used by the `map()` method. For very long iterables using a large value for *chunksize* can make the job complete **much** faster than using the default value of `1`.

Also if *chunksize* is `1` then the `next()` method of the iterator returned by the `imap()` method has an optional *timeout* parameter: `next(timeout)` will raise `multiprocessing.TimeoutError` if the result cannot be returned within *timeout* seconds.

imap_unordered(*func*, *iterable*[, *chunksize*])

The same as `imap()` except that the ordering of the results from the returned iterator should be considered arbitrary. (Only when there is only one worker process is the order guaranteed to be "correct".)

starmap(*func*, *iterable*[, *chunksize*])

Like `map()` except that the elements of the *iterable* are expected to be iterables that are unpacked as arguments.

Hence an *iterable* of `[(1,2), (3, 4)]` results in `[func(1,2), func(3,4)]`.

Adicionado na versão 3.3.

starmap_async(*func*, *iterable*[, *chunksize*[, *callback*[, *error_callback*]]])

A combination of `starmap()` and `map_async()` that iterates over *iterable* of iterables and calls *func* with the iterables unpacked. Returns a result object.

Adicionado na versão 3.3.

close()

Prevents any more tasks from being submitted to the pool. Once all the tasks have been completed the worker processes will exit.

terminate()

Stops the worker processes immediately without completing outstanding work. When the pool object is garbage collected `terminate()` will be called immediately.

join()

Wait for the worker processes to exit. One must call `close()` or `terminate()` before using `join()`.

Alterado na versão 3.3: Pool objects now support the context management protocol – see *Tipos de Gerenciador de Contexto*. `__enter__()` returns the pool object, and `__exit__()` calls `terminate()`.

class multiprocessing.pool.**AsyncResult**

The class of the result returned by `Pool.apply_async()` and `Pool.map_async()`.

get([*timeout*])

Return the result when it arrives. If *timeout* is not `None` and the result does not arrive within *timeout* seconds then `multiprocessing.TimeoutError` is raised. If the remote call raised an exception then that exception will be reraised by `get()`.

wait([*timeout*])

Wait until the result is available or until *timeout* seconds pass.

ready()

Return whether the call has completed.

successful()

Return whether the call completed without raising an exception. Will raise *ValueError* if the result is not ready.

Alterado na versão 3.7: If the result is not ready, *ValueError* is raised instead of *AssertionError*.

The following example demonstrates the use of a pool:

```
from multiprocessing import Pool
import time

def f(x):
    return x*x

if __name__ == '__main__':
    with Pool(processes=4) as pool:          # start 4 worker processes
        result = pool.apply_async(f, (10,)) # evaluate "f(10)" asynchronously in a␣
↪single process
        print(result.get(timeout=1))         # prints "100" unless your computer is␣
↪*very* slow

        print(pool.map(f, range(10)))        # prints "[0, 1, 4,..., 81]"

        it = pool.imap(f, range(10))
        print(next(it))                      # prints "0"
        print(next(it))                      # prints "1"
        print(it.next(timeout=1))            # prints "4" unless your computer is␣
↪*very* slow

        result = pool.apply_async(time.sleep, (10,))
        print(result.get(timeout=1))         # raises multiprocessing.TimeoutError
```

Listeners and Clients

Usually message passing between processes is done using queues or by using *Connection* objects returned by *Pipe()*.

However, the *multiprocessing.connection* module allows some extra flexibility. It basically gives a high level message oriented API for dealing with sockets or Windows named pipes. It also has support for *digest authentication* using the *hmac* module, and for polling multiple connections at the same time.

multiprocessing.connection.**deliver_challenge**(*connection*, *authkey*)

Send a randomly generated message to the other end of the connection and wait for a reply.

If the reply matches the digest of the message using *authkey* as the key then a welcome message is sent to the other end of the connection. Otherwise *AuthenticationError* is raised.

multiprocessing.connection.**answer_challenge**(*connection*, *authkey*)

Receive a message, calculate the digest of the message using *authkey* as the key, and then send the digest back.

If a welcome message is not received, then *AuthenticationError* is raised.

multiprocessing.connection.**Client**(*address*[, *family*[, *authkey*]])

Attempt to set up a connection to the listener which is using address *address*, returning a *Connection*.

The type of the connection is determined by *family* argument, but this can generally be omitted since it can usually be inferred from the format of *address*. (See *Formatos de Endereços*)

If *authkey* is given and not `None`, it should be a byte string and will be used as the secret key for an HMAC-based authentication challenge. No authentication is done if *authkey* is `None`. *AuthenticationError* is raised if authentication fails. See *Authentication keys*.

class multiprocessing.connection.**Listener**([*address*[, *family*[, *backlog*[, *authkey*]]]])

A wrapper for a bound socket or Windows named pipe which is 'listening' for connections.

address is the address to be used by the bound socket or named pipe of the listener object.

> **Nota**
>
> If an address of '0.0.0.0' is used, the address will not be a connectable end point on Windows. If you require a connectable end-point, you should use '127.0.0.1'.

family is the type of socket (or named pipe) to use. This can be one of the strings `'AF_INET'` (for a TCP socket), `'AF_UNIX'` (for a Unix domain socket) or `'AF_PIPE'` (for a Windows named pipe). Of these only the first is guaranteed to be available. If *family* is `None` then the family is inferred from the format of *address*. If *address* is also `None` then a default is chosen. This default is the family which is assumed to be the fastest available. See *Formatos de Endereços*. Note that if *family* is `'AF_UNIX'` and address is `None` then the socket will be created in a private temporary directory created using *tempfile.mkstemp()*.

If the listener object uses a socket then *backlog* (1 by default) is passed to the *listen()* method of the socket once it has been bound.

If *authkey* is given and not `None`, it should be a byte string and will be used as the secret key for an HMAC-based authentication challenge. No authentication is done if *authkey* is `None`. *AuthenticationError* is raised if authentication fails. See *Authentication keys*.

accept()

Accept a connection on the bound socket or named pipe of the listener object and return a *Connection* object. If authentication is attempted and fails, then *AuthenticationError* is raised.

close()

Close the bound socket or named pipe of the listener object. This is called automatically when the listener is garbage collected. However it is advisable to call it explicitly.

Listener objects have the following read-only properties:

address

The address which is being used by the Listener object.

last_accepted

The address from which the last accepted connection came. If this is unavailable then it is `None`.

Alterado na versão 3.3: Listener objects now support the context management protocol – see *Tipos de Gerenciador de Contexto*. *__enter__()* returns the listener object, and *__exit__()* calls *close()*.

multiprocessing.connection.**wait**(*object_list*, *timeout=None*)

Wait till an object in *object_list* is ready. Returns the list of those objects in *object_list* which are ready. If *timeout* is a float then the call blocks for at most that many seconds. If *timeout* is `None` then it will block for an unlimited period. A negative timeout is equivalent to a zero timeout.

For both POSIX and Windows, an object can appear in *object_list* if it is

- a readable *Connection* object;
- a connected and readable *socket.socket* object; or
- the *sentinel* attribute of a *Process* object.

A connection or socket object is ready when there is data available to be read from it, or the other end has been closed.

POSIX: `wait(object_list, timeout)` almost equivalent `select.select(object_list, [], [], timeout)`. The difference is that, if *select.select()* is interrupted by a signal, it can raise *OSError* with an error number of `EINTR`, whereas *wait()* will not.

Windows: An item in *object_list* must either be an integer handle which is waitable (according to the definition used by the documentation of the Win32 function `WaitForMultipleObjects()`) or it can be an object with a `fileno()` method which returns a socket handle or pipe handle. (Note that pipe handles and socket handles are **not** waitable handles.)

Adicionado na versão 3.3.

Examples

The following server code creates a listener which uses `'secret password'` as an authentication key. It then waits for a connection and sends some data to the client:

```
from multiprocessing.connection import Listener
from array import array

address = ('localhost', 6000)     # family is deduced to be 'AF_INET'

with Listener(address, authkey=b'secret password') as listener:
    with listener.accept() as conn:
        print('connection accepted from', listener.last_accepted)

        conn.send([2.25, None, 'junk', float])

        conn.send_bytes(b'hello')

        conn.send_bytes(array('i', [42, 1729]))
```

The following code connects to the server and receives some data from the server:

```
from multiprocessing.connection import Client
from array import array

address = ('localhost', 6000)

with Client(address, authkey=b'secret password') as conn:
    print(conn.recv())                  # => [2.25, None, 'junk', float]

    print(conn.recv_bytes())            # => 'hello'

    arr = array('i', [0, 0, 0, 0, 0])
    print(conn.recv_bytes_into(arr))    # => 8
    print(arr)                          # => array('i', [42, 1729, 0, 0, 0])
```

The following code uses `wait()` to wait for messages from multiple processes at once:

```
from multiprocessing import Process, Pipe, current_process
from multiprocessing.connection import wait

def foo(w):
    for i in range(10):
        w.send((i, current_process().name))
    w.close()

if __name__ == '__main__':
    readers = []

    for i in range(4):
        r, w = Pipe(duplex=False)
        readers.append(r)
```

(continua na próxima página)

(continuação da página anterior)

```
        p = Process(target=foo, args=(w,))
        p.start()
        # We close the writable end of the pipe now to be sure that
        # p is the only process which owns a handle for it.  This
        # ensures that when p closes its handle for the writable end,
        # wait() will promptly report the readable end as being ready.
        w.close()

    while readers:
        for r in wait(readers):
            try:
                msg = r.recv()
            except EOFError:
                readers.remove(r)
            else:
                print(msg)
```

Formatos de Endereços

- Um endereço `'AF_INET'` é uma tupla na forma de `(hostname, port)` sendo *hostname* uma string e *port* um inteiro.
- An `'AF_UNIX'` address is a string representing a filename on the filesystem.
- An `'AF_PIPE'` address is a string of the form `r'\\.\pipe\`*PipeName*`'`. To use *Client()* to connect to a named pipe on a remote computer called *ServerName* one should use an address of the form `r'\\`*ServerName*`\pipe\`*PipeName*`'` instead.

Note that any string beginning with two backslashes is assumed by default to be an `'AF_PIPE'` address rather than an `'AF_UNIX'` address.

Authentication keys

When one uses *Connection.recv*, the data received is automatically unpickled. Unfortunately unpickling data from an untrusted source is a security risk. Therefore *Listener* and *Client()* use the *hmac* module to provide digest authentication.

An authentication key is a byte string which can be thought of as a password: once a connection is established both ends will demand proof that the other knows the authentication key. (Demonstrating that both ends are using the same key does **not** involve sending the key over the connection.)

If authentication is requested but no authentication key is specified then the return value of `current_process().authkey` is used (see *Process*). This value will be automatically inherited by any *Process* object that the current process creates. This means that (by default) all processes of a multi-process program will share a single authentication key which can be used when setting up connections between themselves.

Suitable authentication keys can also be generated by using *os.urandom()*.

Gerando logs

Some support for logging is available. Note, however, that the *logging* package does not use process shared locks so it is possible (depending on the handler type) for messages from different processes to get mixed up.

`multiprocessing.`**`get_logger`**`()`

Returns the logger used by *multiprocessing*. If necessary, a new one will be created.

When first created the logger has level *logging.NOTSET* and no default handler. Messages sent to this logger will not by default propagate to the root logger.

Note that on Windows child processes will only inherit the level of the parent process's logger – any other customization of the logger will not be inherited.

multiprocessing.**log_to_stderr**(*level=None*)

This function performs a call to *get_logger()* but in addition to returning the logger created by get_logger, it adds a handler which sends output to *sys.stderr* using format `'[%(levelname)s/%(processName)s] %(message)s'`. You can modify `levelname` of the logger by passing a `level` argument.

Below is an example session with logging turned on:

```
>>> import multiprocessing, logging
>>> logger = multiprocessing.log_to_stderr()
>>> logger.setLevel(logging.INFO)
>>> logger.warning('doomed')
[WARNING/MainProcess] doomed
>>> m = multiprocessing.Manager()
[INFO/SyncManager-...] child process calling self.run()
[INFO/SyncManager-...] created temp directory /.../pymp-...
[INFO/SyncManager-...] manager serving at '/.../listener-...'
>>> del m
[INFO/MainProcess] sending shutdown message to manager
[INFO/SyncManager-...] manager exiting with exitcode 0
```

For a full table of logging levels, see the *logging* module.

The `multiprocessing.dummy` module

multiprocessing.dummy replicates the API of *multiprocessing* but is no more than a wrapper around the *threading* module.

In particular, the `Pool` function provided by *multiprocessing.dummy* returns an instance of *ThreadPool*, which is a subclass of *Pool* that supports all the same method calls but uses a pool of worker threads rather than worker processes.

class multiprocessing.pool.**ThreadPool**([*processes*[, *initializer*[, *initargs*]]])

A thread pool object which controls a pool of worker threads to which jobs can be submitted. *ThreadPool* instances are fully interface compatible with *Pool* instances, and their resources must also be properly managed, either by using the pool as a context manager or by calling *close()* and *terminate()* manually.

processes is the number of worker threads to use. If *processes* is `None` then the number returned by *os.process_cpu_count()* is used.

If *initializer* is not `None` then each worker process will call `initializer(*initargs)` when it starts.

Unlike *Pool*, *maxtasksperchild* and *context* cannot be provided.

> **Nota**
>
> A *ThreadPool* shares the same interface as *Pool*, which is designed around a pool of processes and predates the introduction of the *concurrent.futures* module. As such, it inherits some operations that don't make sense for a pool backed by threads, and it has its own type for representing the status of asynchronous jobs, *AsyncResult*, that is not understood by any other libraries.
>
> Users should generally prefer to use *concurrent.futures.ThreadPoolExecutor*, which has a simpler interface that was designed around threads from the start, and which returns *concurrent.futures.Future* instances that are compatible with many other libraries, including *asyncio*.

18.2.3 Programming guidelines

There are certain guidelines and idioms which should be adhered to when using *multiprocessing*.

All start methods

The following applies to all start methods.

Avoid shared state

As far as possible one should try to avoid shifting large amounts of data between processes.

It is probably best to stick to using queues or pipes for communication between processes rather than using the lower level synchronization primitives.

Picklability

Ensure that the arguments to the methods of proxies are picklable.

Thread safety of proxies

Do not use a proxy object from more than one thread unless you protect it with a lock.

(There is never a problem with different processes using the *same* proxy.)

Joining zombie processes

On POSIX when a process finishes but has not been joined it becomes a zombie. There should never be very many because each time a new process starts (or `active_children()` is called) all completed processes which have not yet been joined will be joined. Also calling a finished process's `Process.is_alive` will join the process. Even so it is probably good practice to explicitly join all the processes that you start.

Better to inherit than pickle/unpickle

When using the *spawn* or *forkserver* start methods many types from `multiprocessing` need to be picklable so that child processes can use them. However, one should generally avoid sending shared objects to other processes using pipes or queues. Instead you should arrange the program so that a process which needs access to a shared resource created elsewhere can inherit it from an ancestor process.

Avoid terminating processes

Using the `Process.terminate` method to stop a process is liable to cause any shared resources (such as locks, semaphores, pipes and queues) currently being used by the process to become broken or unavailable to other processes.

Therefore it is probably best to only consider using `Process.terminate` on processes which never use any shared resources.

Joining processes that use queues

Bear in mind that a process that has put items in a queue will wait before terminating until all the buffered items are fed by the "feeder" thread to the underlying pipe. (The child process can call the `Queue.cancel_join_thread` method of the queue to avoid this behaviour.)

This means that whenever you use a queue you need to make sure that all items which have been put on the queue will eventually be removed before the process is joined. Otherwise you cannot be sure that processes which have put items on the queue will terminate. Remember also that non-daemonic processes will be joined automatically.

An example which will deadlock is the following:

```
from multiprocessing import Process, Queue

def f(q):
    q.put('X' * 1000000)

if __name__ == '__main__':
    queue = Queue()
    p = Process(target=f, args=(queue,))
    p.start()
```

(continua na próxima página)

(continuação da página anterior)

```
    p.join()                    # this deadlocks
    obj = queue.get()
```

A fix here would be to swap the last two lines (or simply remove the `p.join()` line).

Explicitly pass resources to child processes

On POSIX using the *fork* start method, a child process can make use of a shared resource created in a parent process using a global resource. However, it is better to pass the object as an argument to the constructor for the child process.

Apart from making the code (potentially) compatible with Windows and the other start methods this also ensures that as long as the child process is still alive the object will not be garbage collected in the parent process. This might be important if some resource is freed when the object is garbage collected in the parent process.

So for instance

```
from multiprocessing import Process, Lock

def f():
    ... do something using "lock" ...

if __name__ == '__main__':
    lock = Lock()
    for i in range(10):
        Process(target=f).start()
```

should be rewritten as

```
from multiprocessing import Process, Lock

def f(l):
    ... do something using "l" ...

if __name__ == '__main__':
    lock = Lock()
    for i in range(10):
        Process(target=f, args=(lock,)).start()
```

Beware of replacing *sys.stdin* with a "file like object"

multiprocessing originally unconditionally called:

```
os.close(sys.stdin.fileno())
```

in the `multiprocessing.Process._bootstrap()` method — this resulted in issues with processes-in-processes. This has been changed to:

```
sys.stdin.close()
sys.stdin = open(os.open(os.devnull, os.O_RDONLY), closefd=False)
```

Which solves the fundamental issue of processes colliding with each other resulting in a bad file descriptor error, but introduces a potential danger to applications which replace *sys.stdin()* with a "file-like object" with output buffering. This danger is that if multiple processes call *close()* on this file-like object, it could result in the same data being flushed to the object multiple times, resulting in corruption.

If you write a file-like object and implement your own caching, you can make it fork-safe by storing the pid whenever you append to the cache, and discarding the cache when the pid changes. For example:

```
@property
def cache(self):
    pid = os.getpid()
    if pid != self._pid:
        self._pid = pid
        self._cache = []
    return self._cache
```

For more information, see bpo-5155, bpo-5313 and bpo-5331

The *spawn* and *forkserver* start methods

There are a few extra restrictions which don't apply to the *fork* start method.

More picklability

Ensure that all arguments to *Process* are picklable. Also, if you subclass `Process.__init__`, you must make sure that instances will be picklable when the *Process.start* method is called.

Global variables

Bear in mind that if code run in a child process tries to access a global variable, then the value it sees (if any) may not be the same as the value in the parent process at the time that *Process.start* was called.

However, global variables which are just module level constants cause no problems.

Safe importing of main module

Make sure that the main module can be safely imported by a new Python interpreter without causing unintended side effects (such as starting a new process).

For example, using the *spawn* or *forkserver* start method running the following module would fail with a *RuntimeError*:

```
from multiprocessing import Process

def foo():
    print('hello')

p = Process(target=foo)
p.start()
```

Instead one should protect the "entry point" of the program by using `if __name__ == '__main__':` as follows:

```
from multiprocessing import Process, freeze_support, set_start_method

def foo():
    print('hello')

if __name__ == '__main__':
    freeze_support()
    set_start_method('spawn')
    p = Process(target=foo)
    p.start()
```

(The `freeze_support()` line can be omitted if the program will be run normally instead of frozen.)

This allows the newly spawned Python interpreter to safely import the module and then run the module's `foo()` function.

Similar restrictions apply if a pool or manager is created in the main module.

18.2.4 Exemplos

Demonstration of how to create and use customized managers and proxies:

```
from multiprocessing import freeze_support
from multiprocessing.managers import BaseManager, BaseProxy
import operator

##

class Foo:
    def f(self):
        print('you called Foo.f()')
    def g(self):
        print('you called Foo.g()')
    def _h(self):
        print('you called Foo._h()')

# A simple generator function
def baz():
    for i in range(10):
        yield i*i

# Proxy type for generator objects
class GeneratorProxy(BaseProxy):
    _exposed_ = ['__next__']
    def __iter__(self):
        return self
    def __next__(self):
        return self._callmethod('__next__')

# Function to return the operator module
def get_operator_module():
    return operator

##

class MyManager(BaseManager):
    pass

# register the Foo class; make `f()` and `g()` accessible via proxy
MyManager.register('Foo1', Foo)

# register the Foo class; make `g()` and `_h()` accessible via proxy
MyManager.register('Foo2', Foo, exposed=('g', '_h'))

# register the generator function baz; use `GeneratorProxy` to make proxies
MyManager.register('baz', baz, proxytype=GeneratorProxy)

# register get_operator_module(); make public functions accessible via proxy
MyManager.register('operator', get_operator_module)

##

def test():
    manager = MyManager()
    manager.start()
```

(continua na próxima página)

(continuação da página anterior)

```
    print('-' * 20)

    f1 = manager.Foo1()
    f1.f()
    f1.g()
    assert not hasattr(f1, '_h')
    assert sorted(f1._exposed_) == sorted(['f', 'g'])

    print('-' * 20)

    f2 = manager.Foo2()
    f2.g()
    f2._h()
    assert not hasattr(f2, 'f')
    assert sorted(f2._exposed_) == sorted(['g', '_h'])

    print('-' * 20)

    it = manager.baz()
    for i in it:
        print('<%d>' % i, end=' ')
    print()

    print('-' * 20)

    op = manager.operator()
    print('op.add(23, 45) =', op.add(23, 45))
    print('op.pow(2, 94) =', op.pow(2, 94))
    print('op._exposed_ =', op._exposed_)

##

if __name__ == '__main__':
    freeze_support()
    test()
```

Using *Pool*:

```
import multiprocessing
import time
import random
import sys

#
# Functions used by test code
#

def calculate(func, args):
    result = func(*args)
    return '%s says that %s%s = %s' % (
        multiprocessing.current_process().name,
        func.__name__, args, result
        )

def calculatestar(args):
    return calculate(*args)
```

(continua na próxima página)

(continuação da página anterior)

```
def mul(a, b):
    time.sleep(0.5 * random.random())
    return a * b

def plus(a, b):
    time.sleep(0.5 * random.random())
    return a + b

def f(x):
    return 1.0 / (x - 5.0)

def pow3(x):
    return x ** 3

def noop(x):
    pass

#
# Test code
#

def test():
    PROCESSES = 4
    print('Creating pool with %d processes\n' % PROCESSES)

    with multiprocessing.Pool(PROCESSES) as pool:
        #
        # Tests
        #

        TASKS = [(mul, (i, 7)) for i in range(10)] + \
                [(plus, (i, 8)) for i in range(10)]

        results = [pool.apply_async(calculate, t) for t in TASKS]
        imap_it = pool.imap(calculatestar, TASKS)
        imap_unordered_it = pool.imap_unordered(calculatestar, TASKS)

        print('Ordered results using pool.apply_async():')
        for r in results:
            print('\t', r.get())
        print()

        print('Ordered results using pool.imap():')
        for x in imap_it:
            print('\t', x)
        print()

        print('Unordered results using pool.imap_unordered():')
        for x in imap_unordered_it:
            print('\t', x)
        print()

        print('Ordered results using pool.map() --- will block till complete:')
        for x in pool.map(calculatestar, TASKS):
            print('\t', x)
```

(continua na próxima página)

(continuação da página anterior)

```
        print()

        #
        # Test error handling
        #

        print('Testing error handling:')

        try:
            print(pool.apply(f, (5,)))
        except ZeroDivisionError:
            print('\tGot ZeroDivisionError as expected from pool.apply()')
        else:
            raise AssertionError('expected ZeroDivisionError')

        try:
            print(pool.map(f, list(range(10))))
        except ZeroDivisionError:
            print('\tGot ZeroDivisionError as expected from pool.map()')
        else:
            raise AssertionError('expected ZeroDivisionError')

        try:
            print(list(pool.imap(f, list(range(10)))))
        except ZeroDivisionError:
            print('\tGot ZeroDivisionError as expected from list(pool.imap())')
        else:
            raise AssertionError('expected ZeroDivisionError')

        it = pool.imap(f, list(range(10)))
        for i in range(10):
            try:
                x = next(it)
            except ZeroDivisionError:
                if i == 5:
                    pass
            except StopIteration:
                break
            else:
                if i == 5:
                    raise AssertionError('expected ZeroDivisionError')

        assert i == 9
        print('\tGot ZeroDivisionError as expected from IMapIterator.next()')
        print()

        #
        # Testing timeouts
        #

        print('Testing ApplyResult.get() with timeout:', end=' ')
        res = pool.apply_async(calculate, TASKS[0])
        while 1:
            sys.stdout.flush()
            try:
                sys.stdout.write('\n\t%s' % res.get(0.02))
```

(continua na próxima página)

(continuação da página anterior)

```
                break
            except multiprocessing.TimeoutError:
                sys.stdout.write('.')
        print()
        print()

        print('Testing IMapIterator.next() with timeout:', end=' ')
        it = pool.imap(calculatestar, TASKS)
        while 1:
            sys.stdout.flush()
            try:
                sys.stdout.write('\n\t%s' % it.next(0.02))
            except StopIteration:
                break
            except multiprocessing.TimeoutError:
                sys.stdout.write('.')
        print()
        print()


if __name__ == '__main__':
    multiprocessing.freeze_support()
    test()
```

An example showing how to use queues to feed tasks to a collection of worker processes and collect the results:

```
import time
import random

from multiprocessing import Process, Queue, current_process, freeze_support

#
# Function run by worker processes
#

def worker(input, output):
    for func, args in iter(input.get, 'STOP'):
        result = calculate(func, args)
        output.put(result)

#
# Function used to calculate result
#

def calculate(func, args):
    result = func(*args)
    return '%s says that %s%s = %s' % \
        (current_process().name, func.__name__, args, result)

#
# Functions referenced by tasks
#

def mul(a, b):
    time.sleep(0.5*random.random())
    return a * b
```

(continua na próxima página)

(continuação da página anterior)

```
def plus(a, b):
    time.sleep(0.5*random.random())
    return a + b

#
#
#

def test():
    NUMBER_OF_PROCESSES = 4
    TASKS1 = [(mul, (i, 7)) for i in range(20)]
    TASKS2 = [(plus, (i, 8)) for i in range(10)]

    # Create queues
    task_queue = Queue()
    done_queue = Queue()

    # Submit tasks
    for task in TASKS1:
        task_queue.put(task)

    # Start worker processes
    for i in range(NUMBER_OF_PROCESSES):
        Process(target=worker, args=(task_queue, done_queue)).start()

    # Get and print results
    print('Unordered results:')
    for i in range(len(TASKS1)):
        print('\t', done_queue.get())

    # Add more tasks using `put()`
    for task in TASKS2:
        task_queue.put(task)

    # Get and print some more results
    for i in range(len(TASKS2)):
        print('\t', done_queue.get())

    # Tell child processes to stop
    for i in range(NUMBER_OF_PROCESSES):
        task_queue.put('STOP')


if __name__ == '__main__':
    freeze_support()
    test()
```

18.3 `multiprocessing.shared_memory` — Memória compartilhada para acesso direto entre processos

Código-fonte: Lib/multiprocessing/shared_memory.py

Adicionado na versão 3.8.

Este módulo fornece uma classe, *SharedMemory*, para a alocação e gerenciamento da memória compartilhada a ser acessada por um ou mais processos em uma máquina multicore ou de multiprocessamento simétrico (SMP). Para ajudar com o gerenciamento do ciclo de vida da memória compartilhada especialmente entre processos distintos, uma subclasse de *BaseManager*, *SharedMemoryManager*, também é fornecida no módulo *multiprocessing.managers*.

Neste módulo, memória compartilhada refere-se a blocos de memória compartilhada no "estilo POSIX" (embora não seja necessariamente implementado explicitamente como tal) e não se refere a "memória compartilhada distribuída". Este estilo de memória compartilhada permite que processos distintos potencialmente leiam e escrevam em uma região comum (ou compartilhada) de memória volátil. Os processos são convencionalmente limitados a ter acesso somente ao próprio espaço de memória de processo mas a memória compartilhada permite o compartilhamento de dados entre processos, evitando a necessidade de enviar mensagens entre processos contendo estes dados. Compartilhar dados diretamente via memória pode fornecer ganhos de desempenho significativos comparado ao compartilhamento de dados via disco ou soquete ou outras comunicações que requerem a serialização/desserialização e cópia dos dados.

class multiprocessing.shared_memory.**SharedMemory**(*name=None*, *create=False*, *size=0*, ***, *track=True*)

Cria uma instância da classe `SharedMemory` para criar um novo bloco de memória compartilhada ou anexar a um bloco de memória compartilhada existente. A cada bloco de memória compartilhada é atribuído um nome único. Desta forma, um processo pode criar um bloco de memória compartilhada com um nome particular e um processo diferente pode ser anexado a esse mesmo bloco de memória compartilhada usando este mesmo nome.

Como um recurso para compartilhar dados entre processos, os blocos de memória compartilhada podem sobreviver ao processo original que os criou. Quando um processo não precisa mais acessar um bloco de memória compartilhada que ainda pode ser necessário para outros processos, o método *close()* deve ser chamado. Quando um bloco de memória compartilhada não é mais necessário para nenhum processo, o método *unlink()* deve ser chamado para garantir a limpeza apropriada.

Parâmetros

- **name** (`str | None`) – O nome único para a memória compartilhada requisitada, especificado como uma string. Ao criar um novo bloco de memória compartilhada, se `None` (o padrão) é fornecido para o nome, um novo nome será gerado.
- **create** (`bool`) – Controla quando um novo bloco de memória compartilhada é criado (`True`) ou um bloco de memória compartilhada existente é anexado (`False`).
- **size** (`int`) – O número de bytes requeridos ao criar um novo bloco de memória compartilhada. Como algumas plataformas optam por alocar pedaços de memória com base no tamanho da página de memória da própria plataforma, o tamanho exato do bloco de memória compartilhada pode ser maior ou igual ao tamanho requerido. Ao anexar a um bloco de memória compartilhada existente, o parâmetro *size* é ignorado.
- **track** (`bool`) – Quando `True`, registra o bloco de memória compartilhada com um processo de rastreador de recursos em plataformas onde o sistema operacional não faz isso automaticamente. O rastreador de recursos garante a limpeza adequada da memória compartilhada, mesmo se todos os outros processos com acesso à memória saírem sem fazer isso. Os processos Python criados a partir de um ancestral comum usando os recursos de *multiprocessing* compartilham um único processo de rastreador de recursos, e o tempo de vida dos segmentos de memória compartilhada é manipulado automaticamente entre esses processos. Os processos Python criados de qualquer outra forma receberão seu próprio rastreador de recursos ao acessar a memória compartilhada com *track* habilitado. Isso fará com que a memória compartilhada seja excluída pelo rastreador de recursos do primeiro processo que terminar. Para evitar esse problema, os usuários de *subprocess* ou processos Python autônomos devem definir *track* como `False` quando já houver outro processo em vigor que faça a contabilidade. *track* é ignorado no Windows, que tem seu próprio rastreamento e exclui automaticamente a memória compartilhada quando todos os identificadores para ele foram fechados.

Alterado na versão 3.13: Adicionado o parâmetro *track*.

close()

Fecha o identificador/descritor de arquivo para a memória compartilhada desta instância. *close()* deve ser chamado quando o acesso ao bloco de memória compartilhada desta instância não for mais necessário. Dependendo do sistema operacional, a memória subjacente pode ou não ser liberada, mesmo que todos os identificadores para ela tenham sido fechados. Para garantir uma limpeza adequada, use o método *unlink()*.

unlink()

Exclui o bloco de memória compartilhada subjacente. Isso deve ser chamado apenas uma vez por bloco de memória compartilhada, independentemente do número de handles para ele, mesmo em outros processos. *unlink()* e *close()* podem ser chamados em qualquer ordem, mas tentar acessar dados dentro de um bloco de memória compartilhada após *unlink()* pode resultar em erros de acesso à memória, dependendo da plataforma.

Este método não tem efeito no Windows, onde a única maneira de excluir um bloco de memória compartilhada é fechar todos os identificadores.

buf

Uma visualização de memória do conteúdo do bloco de memória compartilhada.

name

Acesso somente leitura ao nome único do bloco de memória compartilhada.

size

Acesso somente leitura ao tamanho em bytes do bloco de memória compartilhada.

O exemplo a seguir demonstra um uso baixo nível de instâncias de *SharedMemory*:

```
>>> from multiprocessing import shared_memory
>>> shm_a = shared_memory.SharedMemory(create=True, size=10)
>>> type(shm_a.buf)
<class 'memoryview'>
>>> buffer = shm_a.buf
>>> len(buffer)
10
>>> buffer[:4] = bytearray([22, 33, 44, 55])  # Modifica vários de uma só vez
>>> buffer[4] = 100                           # Modifica um único byte de cada vez
>>> # Anexa a um bloco de memória compartilhada existente
>>> shm_b = shared_memory.SharedMemory(shm_a.name)
>>> import array
>>> array.array('b', shm_b.buf[:5])  # Copia os dados para um novo array.array
array('b', [22, 33, 44, 55, 100])
>>> shm_b.buf[:5] = b'howdy'  # Modifica via shm_b usando bytes
>>> bytes(shm_a.buf[:5])      # Acessa via shm_a
b'howdy'
>>> shm_b.close()   # Fecha cada instância de SharedMemory
>>> shm_a.close()
>>> shm_a.unlink()  # Chama unlink uma vez para liberar a memória Compartilhada
```

O exemplo a seguir demonstra um uso prático da classe *SharedMemory* com arrays do NumPy, acessando o mesmo `numpy.ndarray` de dois consoles Python distintos.

```
>>> # No primeiro console interativo do Python
>>> import numpy as np
>>> a = np.array([1, 1, 2, 3, 5, 8])  # Inicia com um array de NumPy existente
>>> from multiprocessing import shared_memory
>>> shm = shared_memory.SharedMemory(create=True, size=a.nbytes)
>>> # Agora cria um array de NumPy suportado por memória compartilhada
>>> b = np.ndarray(a.shape, dtype=a.dtype, buffer=shm.buf)
>>> b[:] = a[:]  # Copia os dados originais para memória compartilhada
```

(continua na próxima página)

(continuação da página anterior)

```
>>> b
array([1, 1, 2, 3, 5, 8])
>>> type(b)
<class 'numpy.ndarray'>
>>> type(a)
<class 'numpy.ndarray'>
>>> shm.name  # Não especificamos um nome, então um foi escolhido por nós
'psm_21467_46075'

>>> # No mesmo shell ou em um novo console do Python na mesma máquina
>>> import numpy as np
>>> from multiprocessing import shared_memory
>>> # Anexa ao bloco de memória compartilhada existente
>>> existing_shm = shared_memory.SharedMemory(name='psm_21467_46075')
>>> # Observe que a.shape é (6,) e a.dtype é np.int64 neste exemplo
>>> c = np.ndarray((6,), dtype=np.int64, buffer=existing_shm.buf)
>>> c
array([1, 1, 2, 3, 5, 8])
>>> c[-1] = 888
>>> c
array([  1,   1,   2,   3,   5, 888])

>>> # De volta ao primeiro console interativo do Python, b reflete essa mudança
>>> b
array([  1,   1,   2,   3,   5, 888])

>>> # Limpeza de dentro do segundo console Python
>>> del c  # Desnecessário; apenas enfatizar a array não é mais usado
>>> existing_shm.close()

>>> # Limpeza de dentro do primeiro console Python
>>> del b  # Desnecessário; apenas enfatizar a array não é mais usado
>>> shm.close()
>>> shm.unlink()  # Libera o bloco de memória compartilhada no final
```

class multiprocessing.managers.**SharedMemoryManager**([*address*[, *authkey*]])

Uma subclasse de *multiprocessing.managers.BaseManager* que pode ser usada para o gerenciamento dos blocos de memória compartilhada entre processos.

Uma chamada ao método *start()* em uma instância de `SharedMemoryManager` faz com que um novo processo seja iniciado. A única finalidade desse novo processo é gerenciar o ciclo de vida de todos os blocos de memória criados através dele. Para acionar a liberação de todos os blocos de memória gerenciados por este processo, invoque o método *shutdown()* na instância. Isso aciona uma chamada de *unlink()* em todos os objetos *SharedMemory* gerenciados por esse processo e então para o processo em si. Ao criar instâncias de `SharedMemory` através de um `SharedMemoryManager`, evitamos a necessidade de rastrear e acionar manualmente a liberação dos recursos de memória compartilhada.

Esta classe fornece métodos para criar e retornar instâncias de *SharedMemory* e para criar um objeto lista ou similar (*ShareableList*) apoiado por memória compartilhada.

Consulte *BaseManager* para obter uma descrição dos argumentos de entrada opcionais herdados *address* e *authkey* e como eles podem ser usados para conectar-se a um serviço `SharedMemoryManager` existente de outros processos.

SharedMemory(*size*)

Cria e retorna um novo objeto *SharedMemory* com o *size* especificado em bytes.

ShareableList(*sequence*)

Cria e retorna um novo objeto *ShareableList*, inicializado pelos valores da entrada *sequence*.

O exemplo a seguir demonstra os mecanismos básicos de um *SharedMemoryManager*:

```
>>> from multiprocessing.managers import SharedMemoryManager
>>> smm = SharedMemoryManager()
>>> smm.start()  # Inicia o procsso que gerencia os blocos de memória compartilhada
>>> sl = smm.ShareableList(range(4))
>>> sl
ShareableList([0, 1, 2, 3], name='psm_6572_7512')
>>> raw_shm = smm.SharedMemory(size=128)
>>> another_sl = smm.ShareableList('alpha')
>>> another_sl
ShareableList(['a', 'l', 'p', 'h', 'a'], name='psm_6572_12221')
>>> smm.shutdown()  # Chama unlink() em sl, raw_shm, e another_sl
```

O exemplo a seguir retrata um padrão potencialmente mais conveniente para usar objetos *SharedMemoryManager* através da instrução `with` para garantir que todos os blocos de memória compartilhada são liberados depois que não são mais necessários.

```
>>> with SharedMemoryManager() as smm:
...     sl = smm.ShareableList(range(2000))
...     # Divide o trabalho entre dois processos, armazenando resultados parciais␣
↪em sl
...     p1 = Process(target=do_work, args=(sl, 0, 1000))
...     p2 = Process(target=do_work, args=(sl, 1000, 2000))
...     p1.start()
...     p2.start()  # Um multiprocessing.Pool pode ser mais eficiente
...     p1.join()
...     p2.join()   # Espera todo trabalho ser concluído nos dois procssos
...     total_result = sum(sl)  # Consolida os resutlados parciais agora em sl
```

Ao usar um *SharedMemoryManager* em uma instrução `with`, os blocos de memória compartilhada criados utilizando este gerenciador são todos liberados quando o bloco de código com a instrução `with` termina sua execução.

class multiprocessing.shared_memory.**ShareableList**(*sequence=None*, *, *name=None*)

Fornece um objeto mutável do tipo lista onde todos os valores armazenados dentro são armazenados em um bloco de memória compartilhada. Isso restringe valores armazenáveis aos seguintes tipos de dados embutidos:

- *int* (com sinal, 64-bit)
- *float*
- *bool*
- *str* (menos de 10M bytes cada quando codificado como UTF-8)
- *bytes* (menos de 10M bytes cada)
- `None`

Ele também difere notavelmente do tipo embutido *list*, pois essas listas não podem alterar seu comprimento geral (ou seja, sem `append()`, `insert()`, etc.) e não oferecem suporte à criação dinâmica de novas instâncias `ShareableList` por meio de fatiamento.

sequence é usado para preencher um `ShareableList` com valores. Defina como `None` para anexar a uma `ShareableList` já existente pelo seu nome único de memória compartilhada.

name é um nome único para a memória compartilhada requerida, como descrito na definição de *SharedMemory*. Ao anexar a uma `ShareableList` já existente, deve-se especificar o nome único do bloco de memória compartilhada e definir *sequence* como `None`.

> **Nota**
>
> Existe um problema conhecido para os valores *bytes* e *str*. Se eles terminarem com bytes ou caracteres nulos `\x00`, eles podem ser *silenciosamente removidos* ao buscá-los pelo índice da `ShareableList`. Esse comportamento `.rstrip(b'\x00')` é considerado um bug e pode desaparecer no futuro. Veja gh-106939.

Para aplicações onde a remoção de nulos finais é um problema, contorne-o sempre anexando incondicionalmente um byte extra diferente de 0 ao final de tais valores ao armazená-los e removendo-os incondicionalmente ao buscá-los:

```
>>> from multiprocessing import shared_memory
>>> nul_bug_demo = shared_memory.ShareableList(['?\x00', b'\x03\x02\x01\x00\
↪x00\x00'])
>>> nul_bug_demo[0]
'?'
>>> nul_bug_demo[1]
b'\x03\x02\x01'
>>> nul_bug_demo.shm.unlink()
>>> padded = shared_memory.ShareableList(['?\x00\x07', b'\x03\x02\x01\x00\x00\
↪x00\x07'])
>>> padded[0][:-1]
'?\x00'
>>> padded[1][:-1]
b'\x03\x02\x01\x00\x00\x00'
>>> padded.shm.unlink()
```

count(*value*)

Retorna o número de ocorrências de *value*.

index(*value*)

Retorna a primeira posição do índice de *value*. Levanta *ValueError* se *value* não estiver presente.

format

Atributo somente leitura contendo o formato de empacotamento *struct* usado por todos os valores armazenados atualmente.

shm

A instância de *SharedMemory* onde os valores são armazenados.

O exemplo a seguir demonstra o uso básico de uma instância de *ShareableList*:

```
>>> from multiprocessing import shared_memory
>>> a = shared_memory.ShareableList(['howdy', b'HoWdY', -273.154, 100, None, True,␣
↪42])
>>> [ type(entry) for entry in a ]
[<class 'str'>, <class 'bytes'>, <class 'float'>, <class 'int'>, <class 'NoneType'>
↪, <class 'bool'>, <class 'int'>]
>>> a[2]
-273.154
>>> a[2] = -78.5
>>> a[2]
-78.5
>>> a[2] = 'dry ice'  # Changing data types is supported as well
>>> a[2]
'dry ice'
>>> a[2] = 'larger than previously allocated storage space'
Traceback (most recent call last):
```

(continua na próxima página)

(continuação da página anterior)

```
  ...
ValueError: exceeds available storage for existing str
>>> a[2]
'dry ice'
>>> len(a)
7
>>> a.index(42)
6
>>> a.count(b'howdy')
0
>>> a.count(b'HoWdY')
1
>>> a.shm.close()
>>> a.shm.unlink()
>>> del a  # Use of a ShareableList after call to unlink() is unsupported
```

O exemplo a seguir retrata como um, dois ou mais processos podem acessar a mesma *ShareableList* fornecendo o nome do bloco de memória compartilhada por trás dela:

```
>>> b = shared_memory.ShareableList(range(5))          # In a first process
>>> c = shared_memory.ShareableList(name=b.shm.name)   # In a second process
>>> c
ShareableList([0, 1, 2, 3, 4], name='...')
>>> c[-1] = -999
>>> b[-1]
-999
>>> b.shm.close()
>>> c.shm.close()
>>> c.shm.unlink()
```

Os exemplos a seguir demonstram que os objetos *ShareableList* (e o subjacente *SharedMemory*) podem ser serializados e desserializados com pickle, se preciso. Note, ainda será o mesmo objeto compartilhado. Isto acontece pois o objeto desserializado tem o mesmo nome único e é somente anexado ao objeto existente com o mesmo nome (se o objeto ainda existe):

```
>>> import pickle
>>> from multiprocessing import shared_memory
>>> sl = shared_memory.ShareableList(range(10))
>>> list(sl)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> deserialized_sl = pickle.loads(pickle.dumps(sl))
>>> list(deserialized_sl)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> sl[0] = -1
>>> deserialized_sl[1] = -2
>>> list(sl)
[-1, -2, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(deserialized_sl)
[-1, -2, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> sl.shm.close()
>>> sl.shm.unlink()
```

18.4 O pacote `concurrent`

Este pacote contém os seguintes módulos:

- *`concurrent.futures`* – Iniciando tarefas em paralelo
- *`concurrent.interpreters`* — Múltiplos interpretadores no mesmo processo

18.5 `concurrent.futures` — Launching parallel tasks

Adicionado na versão 3.2.

Source code: Lib/concurrent/futures/thread.py, Lib/concurrent/futures/process.py, and Lib/concurrent/futures/interpreter.py

---

The *`concurrent.futures`* module provides a high-level interface for asynchronously executing callables.

The asynchronous execution can be performed with threads, using *`ThreadPoolExecutor`* or *`InterpreterPoolExecutor`*, or separate processes, using *`ProcessPoolExecutor`*. Each implements the same interface, which is defined by the abstract *`Executor`* class.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

18.5.1 Executor Objects

class concurrent.futures.**Executor**

An abstract class that provides methods to execute calls asynchronously. It should not be used directly, but through its concrete subclasses.

submit(*fn*, /, **args*, ***kwargs*)

Schedules the callable, *fn*, to be executed as `fn(*args, **kwargs)` and returns a *`Future`* object representing the execution of the callable.

```
with ThreadPoolExecutor(max_workers=1) as executor:
    future = executor.submit(pow, 323, 1235)
    print(future.result())
```

map(*fn*, **iterables*, *timeout=None*, *chunksize=1*, *buffersize=None*)

Similar to *`map(fn, *iterables)`* except:

- The *iterables* are collected immediately rather than lazily, unless a *buffersize* is specified to limit the number of submitted tasks whose results have not yet been yielded. If the buffer is full, iteration over the *iterables* pauses until a result is yielded from the buffer.
- *fn* is executed asynchronously and several calls to *fn* may be made concurrently.

The returned iterator raises a *`TimeoutError`* if *`__next__()`* is called and the result isn't available after *timeout* seconds from the original call to *`Executor.map()`*. *timeout* can be an int or a float. If *timeout* is not specified or `None`, there is no limit to the wait time.

If a *fn* call raises an exception, then that exception will be raised when its value is retrieved from the iterator.

When using *`ProcessPoolExecutor`*, this method chops *iterables* into a number of chunks which it submits to the pool as separate tasks. The (approximate) size of these chunks can be specified by setting *chunksize* to a positive integer. For very long iterables, using a large value for *chunksize* can significantly improve performance compared to the default size of 1. With *`ThreadPoolExecutor`* and *`InterpreterPoolExecutor`*, *chunksize* has no effect.

Alterado na versão 3.5: Added the *chunksize* parameter.

Alterado na versão 3.14: Added the *buffersize* parameter.

shutdown(*wait=True*, *, *cancel_futures=False*)

Signal the executor that it should free any resources that it is using when the currently pending futures are done executing. Calls to *Executor.submit()* and *Executor.map()* made after shutdown will raise *RuntimeError*.

If *wait* is `True` then this method will not return until all the pending futures are done executing and the resources associated with the executor have been freed. If *wait* is `False` then this method will return immediately and the resources associated with the executor will be freed when all pending futures are done executing. Regardless of the value of *wait*, the entire Python program will not exit until all pending futures are done executing.

If *cancel_futures* is `True`, this method will cancel all pending futures that the executor has not started running. Any futures that are completed or running won't be cancelled, regardless of the value of *cancel_futures*.

If both *cancel_futures* and *wait* are `True`, all futures that the executor has started running will be completed prior to this method returning. The remaining futures are cancelled.

You can avoid having to call this method explicitly if you use the executor as a *context manager* via the `with` statement, which will shutdown the *Executor* (waiting as if *Executor.shutdown()* were called with *wait* set to `True`):

```
import shutil
with ThreadPoolExecutor(max_workers=4) as e:
    e.submit(shutil.copy, 'src1.txt', 'dest1.txt')
    e.submit(shutil.copy, 'src2.txt', 'dest2.txt')
    e.submit(shutil.copy, 'src3.txt', 'dest3.txt')
    e.submit(shutil.copy, 'src4.txt', 'dest4.txt')
```

Alterado na versão 3.9: Adicionado *cancel_futures*.

18.5.2 ThreadPoolExecutor

ThreadPoolExecutor is an *Executor* subclass that uses a pool of threads to execute calls asynchronously.

Deadlocks can occur when the callable associated with a *Future* waits on the results of another *Future*. For example:

```
import time
def wait_on_b():
    time.sleep(5)
    print(b.result())  # b will never complete because it is waiting on a.
    return 5

def wait_on_a():
    time.sleep(5)
    print(a.result())  # a will never complete because it is waiting on b.
    return 6


executor = ThreadPoolExecutor(max_workers=2)
a = executor.submit(wait_on_b)
b = executor.submit(wait_on_a)
```

And:

```
def wait_on_future():
    f = executor.submit(pow, 5, 2)
    # This will never complete because there is only one worker thread and
    # it is executing this function.
    print(f.result())

executor = ThreadPoolExecutor(max_workers=1)
executor.submit(wait_on_future)
```

class concurrent.futures.**ThreadPoolExecutor**(*max_workers=None*, *thread_name_prefix=''*, *initializer=None*, *initargs=()*)

An *Executor* subclass that uses a pool of at most *max_workers* threads to execute calls asynchronously.

All threads enqueued to `ThreadPoolExecutor` will be joined before the interpreter can exit. Note that the exit handler which does this is executed *before* any exit handlers added using `atexit`. This means exceptions in the main thread must be caught and handled in order to signal threads to exit gracefully. For this reason, it is recommended that `ThreadPoolExecutor` not be used for long-running tasks.

initializer is an optional callable that is called at the start of each worker thread; *initargs* is a tuple of arguments passed to the initializer. Should *initializer* raise an exception, all currently pending jobs will raise a *BrokenThreadPool*, as well as any attempt to submit more jobs to the pool.

Alterado na versão 3.5: If *max_workers* is `None` or not given, it will default to the number of processors on the machine, multiplied by `5`, assuming that *ThreadPoolExecutor* is often used to overlap I/O instead of CPU work and the number of workers should be higher than the number of workers for *ProcessPoolExecutor*.

Alterado na versão 3.6: Added the *thread_name_prefix* parameter to allow users to control the *threading.Thread* names for worker threads created by the pool for easier debugging.

Alterado na versão 3.7: Added the *initializer* and *initargs* arguments.

Alterado na versão 3.8: Default value of *max_workers* is changed to `min(32, os.cpu_count() + 4)`. This default value preserves at least 5 workers for I/O bound tasks. It utilizes at most 32 CPU cores for CPU bound tasks which release the GIL. And it avoids using very large resources implicitly on many-core machines.

ThreadPoolExecutor now reuses idle worker threads before starting *max_workers* worker threads too.

Alterado na versão 3.13: Default value of *max_workers* is changed to `min(32, (os.process_cpu_count() or 1) + 4)`.

Exemplo de ThreadPoolExecutor

```
import concurrent.futures
import urllib.request

URLS = ['http://www.foxnews.com/',
        'http://www.cnn.com/',
        'http://europe.wsj.com/',
        'http://www.bbc.co.uk/',
        'http://nonexistent-subdomain.python.org/']

# Retrieve a single page and report the URL and contents
def load_url(url, timeout):
    with urllib.request.urlopen(url, timeout=timeout) as conn:
        return conn.read()

# We can use a with statement to ensure threads are cleaned up promptly
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    # Start the load operations and mark each future with its URL
    future_to_url = {executor.submit(load_url, url, 60): url for url in URLS}
```

(continua na próxima página)

(continuação da página anterior)

```
    for future in concurrent.futures.as_completed(future_to_url):
        url = future_to_url[future]
        try:
            data = future.result()
        except Exception as exc:
            print('%r generated an exception: %s' % (url, exc))
        else:
            print('%r page is %d bytes' % (url, len(data)))
```

18.5.3 InterpreterPoolExecutor

The *InterpreterPoolExecutor* class uses a pool of interpreters to execute calls asynchronously. It is a *ThreadPoolExecutor* subclass, which means each worker is running in its own thread. The difference here is that each worker has its own interpreter, and runs each task using that interpreter.

The biggest benefit to using interpreters instead of only threads is true multi-core parallelism. Each interpreter has its own *Global Interpreter Lock*, so code running in one interpreter can run on one CPU core, while code in another interpreter runs unblocked on a different core.

The tradeoff is that writing concurrent code for use with multiple interpreters can take extra effort. However, this is because it forces you to be deliberate about how and when interpreters interact, and to be explicit about what data is shared between interpreters. This results in several benefits that help balance the extra effort, including true multi-core parallelism, For example, code written this way can make it easier to reason about concurrency. Another major benefit is that you don't have to deal with several of the big pain points of using threads, like race conditions.

Each worker's interpreter is isolated from all the other interpreters. "Isolated" means each interpreter has its own runtime state and operates completely independently. For example, if you redirect *sys.stdout* in one interpreter, it will not be automatically redirected to any other interpreter. If you import a module in one interpreter, it is not automatically imported in any other. You would need to import the module separately in interpreter where you need it. In fact, each module imported in an interpreter is a completely separate object from the same module in a different interpreter, including *sys*, *builtins*, and even *__main__*.

Isolation means a mutable object, or other data, cannot be used by more than one interpreter at the same time. That effectively means interpreters cannot actually share such objects or data. Instead, each interpreter must have its own copy, and you will have to synchronize any changes between the copies manually. Immutable objects and data, like the builtin singletons, strings, and tuples of immutable objects, don't have these limitations.

Communicating and synchronizing between interpreters is most effectively done using dedicated tools, like those proposed in **PEP 734**. One less efficient alternative is to serialize with *pickle* and then send the bytes over a shared *socket* or *pipe*.

class concurrent.futures.**InterpreterPoolExecutor**(*max_workers=None*, *thread_name_prefix=''*, *initializer=None*, *initargs=()*)

A *ThreadPoolExecutor* subclass that executes calls asynchronously using a pool of at most *max_workers* threads. Each thread runs tasks in its own interpreter. The worker interpreters are isolated from each other, which means each has its own runtime state and that they can't share any mutable objects or other data. Each interpreter has its own *Global Interpreter Lock*, which means code run with this executor has true multi-core parallelism.

The optional *initializer* and *initargs* arguments have the same meaning as for ThreadPoolExecutor: the initializer is run when each worker is created, though in this case it is run in the worker's interpreter. The executor serializes the *initializer* and *initargs* using *pickle* when sending them to the worker's interpreter.

> **Nota**
>
> The executor may replace uncaught exceptions from *initializer* with *ExecutionFailed*.

Other caveats from parent *ThreadPoolExecutor* apply here.

submit() and *map()* work like normal, except the worker serializes the callable and arguments using *pickle* when sending them to its interpreter. The worker likewise serializes the return value when sending it back.

When a worker's current task raises an uncaught exception, the worker always tries to preserve the exception as-is. If that is successful then it also sets the __cause__ to a corresponding *ExecutionFailed* instance, which contains a summary of the original exception. In the uncommon case that the worker is not able to preserve the original as-is then it directly preserves the corresponding *ExecutionFailed* instance instead.

18.5.4 `ProcessPoolExecutor`

The *ProcessPoolExecutor* class is an *Executor* subclass that uses a pool of processes to execute calls asynchronously. *ProcessPoolExecutor* uses the *multiprocessing* module, which allows it to side-step the *Global Interpreter Lock* but also means that only picklable objects can be executed and returned.

The __main__ module must be importable by worker subprocesses. This means that *ProcessPoolExecutor* will not work in the interactive interpreter.

Calling *Executor* or *Future* methods from a callable submitted to a *ProcessPoolExecutor* will result in deadlock.

Note that the restrictions on functions and arguments needing to picklable as per *multiprocessing.Process* apply when using *submit()* and *map()* on a *ProcessPoolExecutor*. A function defined in a REPL or a lambda should not be expected to work.

class concurrent.futures.**ProcessPoolExecutor**(*max_workers=None*, *mp_context=None*, *initializer=None*, *initargs=()*, *max_tasks_per_child=None*)

An *Executor* subclass that executes calls asynchronously using a pool of at most *max_workers* processes. If *max_workers* is None or not given, it will default to *os.process_cpu_count()*. If *max_workers* is less than or equal to 0, then a *ValueError* will be raised. On Windows, *max_workers* must be less than or equal to 61. If it is not then *ValueError* will be raised. If *max_workers* is None, then the default chosen will be at most 61, even if more processors are available. *mp_context* can be a *multiprocessing* context or None. It will be used to launch the workers. If *mp_context* is None or not given, the default *multiprocessing* context is used. See *Contextos e métodos de inicialização*.

initializer is an optional callable that is called at the start of each worker process; *initargs* is a tuple of arguments passed to the initializer. Should *initializer* raise an exception, all currently pending jobs will raise a *BrokenProcessPool*, as well as any attempt to submit more jobs to the pool.

max_tasks_per_child is an optional argument that specifies the maximum number of tasks a single process can execute before it will exit and be replaced with a fresh worker process. By default *max_tasks_per_child* is None which means worker processes will live as long as the pool. When a max is specified, the "spawn" multiprocessing start method will be used by default in absence of a *mp_context* parameter. This feature is incompatible with the "fork" start method.

Alterado na versão 3.3: When one of the worker processes terminates abruptly, a *BrokenProcessPool* error is now raised. Previously, behaviour was undefined but operations on the executor or its futures would often freeze or deadlock.

Alterado na versão 3.7: The *mp_context* argument was added to allow users to control the start_method for worker processes created by the pool.

Added the *initializer* and *initargs* arguments.

Alterado na versão 3.11: The *max_tasks_per_child* argument was added to allow users to control the lifetime of workers in the pool.

Alterado na versão 3.12: On POSIX systems, if your application has multiple threads and the *multiprocessing* context uses the "fork" start method: The *os.fork()* function called internally to spawn workers may raise a *DeprecationWarning*. Pass a *mp_context* configured to use a different start method. See the *os.fork()* documentation for further explanation.

Alterado na versão 3.13: *max_workers* uses *os.process_cpu_count()* by default, instead of *os.cpu_count()*.

Alterado na versão 3.14: The default process start method (see *Contextos e métodos de inicialização*) changed away from *fork*. If you require the *fork* start method for `ProcessPoolExecutor` you must explicitly pass `mp_context=multiprocessing.get_context("fork")`.

terminate_workers()

Attempt to terminate all living worker processes immediately by calling `Process.terminate` on each of them. Internally, it will also call `Executor.shutdown()` to ensure that all other resources associated with the executor are freed.

After calling this method the caller should no longer submit tasks to the executor.

Adicionado na versão 3.14.

kill_workers()

Attempt to kill all living worker processes immediately by calling `Process.kill` on each of them. Internally, it will also call `Executor.shutdown()` to ensure that all other resources associated with the executor are freed.

After calling this method the caller should no longer submit tasks to the executor.

Adicionado na versão 3.14.

ProcessPoolExecutor Example

```
import concurrent.futures
import math

PRIMES = [
    112272535095293,
    112582705942171,
    112272535095293,
    115280095190773,
    115797848077099,
    1099726899285419]

def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    sqrt_n = int(math.floor(math.sqrt(n)))
    for i in range(3, sqrt_n + 1, 2):
        if n % i == 0:
            return False
    return True

def main():
    with concurrent.futures.ProcessPoolExecutor() as executor:
        for number, prime in zip(PRIMES, executor.map(is_prime, PRIMES)):
            print('%d is prime: %s' % (number, prime))

if __name__ == '__main__':
    main()
```

18.5.5 Future Objects

The *Future* class encapsulates the asynchronous execution of a callable. *Future* instances are created by *Executor.submit()*.

class concurrent.futures.**Future**

Encapsulates the asynchronous execution of a callable. *Future* instances are created by *Executor.submit()* and should not be created directly except for testing.

cancel()

Attempt to cancel the call. If the call is currently being executed or finished running and cannot be cancelled then the method will return `False`, otherwise the call will be cancelled and the method will return `True`.

cancelled()

Return `True` if the call was successfully cancelled.

running()

Return `True` if the call is currently being executed and cannot be cancelled.

done()

Return `True` if the call was successfully cancelled or finished running.

result(*timeout=None*)

Return the value returned by the call. If the call hasn't yet completed then this method will wait up to *timeout* seconds. If the call hasn't completed in *timeout* seconds, then a *TimeoutError* will be raised. *timeout* can be an int or float. If *timeout* is not specified or `None`, there is no limit to the wait time.

If the future is cancelled before completing then *CancelledError* will be raised.

If the call raised an exception, this method will raise the same exception.

exception(*timeout=None*)

Return the exception raised by the call. If the call hasn't yet completed then this method will wait up to *timeout* seconds. If the call hasn't completed in *timeout* seconds, then a *TimeoutError* will be raised. *timeout* can be an int or float. If *timeout* is not specified or `None`, there is no limit to the wait time.

If the future is cancelled before completing then *CancelledError* will be raised.

If the call completed without raising, `None` is returned.

add_done_callback(*fn*)

Attaches the callable *fn* to the future. *fn* will be called, with the future as its only argument, when the future is cancelled or finishes running.

Added callables are called in the order that they were added and are always called in a thread belonging to the process that added them. If the callable raises an *Exception* subclass, it will be logged and ignored. If the callable raises a *BaseException* subclass, the behavior is undefined.

If the future has already completed or been cancelled, *fn* will be called immediately.

The following *Future* methods are meant for use in unit tests and *Executor* implementations.

set_running_or_notify_cancel()

This method should only be called by *Executor* implementations before executing the work associated with the *Future* and by unit tests.

If the method returns `False` then the *Future* was cancelled, i.e. *Future.cancel()* was called and returned `True`. Any threads waiting on the *Future* completing (i.e. through *as_completed()* or *wait()*) will be woken up.

If the method returns `True` then the *Future* was not cancelled and has been put in the running state, i.e. calls to *Future.running()* will return `True`.

This method can only be called once and cannot be called after *Future.set_result()* or *Future.set_exception()* have been called.

set_result(*result*)

Sets the result of the work associated with the `Future` to *result*.

This method should only be used by `Executor` implementations and unit tests.

Alterado na versão 3.8: This method raises `concurrent.futures.InvalidStateError` if the `Future` is already done.

set_exception(*exception*)

Sets the result of the work associated with the `Future` to the `Exception` *exception*.

This method should only be used by `Executor` implementations and unit tests.

Alterado na versão 3.8: This method raises `concurrent.futures.InvalidStateError` if the `Future` is already done.

18.5.6 Module Functions

concurrent.futures.**wait**(*fs*, *timeout=None*, *return_when=ALL_COMPLETED*)

Wait for the `Future` instances (possibly created by different `Executor` instances) given by *fs* to complete. Duplicate futures given to *fs* are removed and will be returned only once. Returns a named 2-tuple of sets. The first set, named `done`, contains the futures that completed (finished or cancelled futures) before the wait completed. The second set, named `not_done`, contains the futures that did not complete (pending or running futures).

timeout can be used to control the maximum number of seconds to wait before returning. *timeout* can be an int or float. If *timeout* is not specified or `None`, there is no limit to the wait time.

return_when indica quando esta função deve retornar. Ele deve ser uma das seguintes constantes:

Constante	Descrição
concurrent.futures.**FIRST_COMPLETED**	A função irá retornar quando qualquer futuro terminar ou for cancelado.
concurrent.futures.**FIRST_EXCEPTION**	The function will return when any future finishes by raising an exception. If no future raises an exception then it is equivalent to `ALL_COMPLETED`.
concurrent.futures.**ALL_COMPLETED**	A função irá retornar quando todos os futuros encerrarem ou forem cancelados.

concurrent.futures.**as_completed**(*fs*, *timeout=None*)

Returns an iterator over the `Future` instances (possibly created by different `Executor` instances) given by *fs* that yields futures as they complete (finished or cancelled futures). Any futures given by *fs* that are duplicated will be returned once. Any futures that completed before `as_completed()` is called will be yielded first. The returned iterator raises a `TimeoutError` if `__next__()` is called and the result isn't available after *timeout* seconds from the original call to `as_completed()`. *timeout* can be an int or float. If *timeout* is not specified or `None`, there is no limit to the wait time.

Ver também

PEP 3148 – **futures - execute computations asynchronously**
The proposal which described this feature for inclusion in the Python standard library.

18.5.7 Exception classes

exception concurrent.futures.**CancelledError**

Raised when a future is cancelled.

exception concurrent.futures.**TimeoutError**

A deprecated alias of *TimeoutError*, raised when a future operation exceeds the given timeout.

Alterado na versão 3.11: Esta classe foi feita como um apelido de *TimeoutError*.

exception concurrent.futures.**BrokenExecutor**

Derived from *RuntimeError*, this exception class is raised when an executor is broken for some reason, and cannot be used to submit or execute new tasks.

Adicionado na versão 3.7.

exception concurrent.futures.**InvalidStateError**

Raised when an operation is performed on a future that is not allowed in the current state.

Adicionado na versão 3.8.

exception concurrent.futures.thread.**BrokenThreadPool**

Derived from *BrokenExecutor*, this exception class is raised when one of the workers of a *ThreadPoolExecutor* has failed initializing.

Adicionado na versão 3.7.

exception concurrent.futures.interpreter.**BrokenInterpreterPool**

Derived from *BrokenThreadPool*, this exception class is raised when one of the workers of a *InterpreterPoolExecutor* has failed initializing.

Adicionado na versão 3.14.

exception concurrent.futures.interpreter.**ExecutionFailed**

Raised from *InterpreterPoolExecutor* when the given initializer fails or from *submit()* when there's an uncaught exception from the submitted task.

Adicionado na versão 3.14.

exception concurrent.futures.process.**BrokenProcessPool**

Derived from *BrokenExecutor* (formerly *RuntimeError*), this exception class is raised when one of the workers of a *ProcessPoolExecutor* has terminated in a non-clean fashion (for example, if it was killed from the outside).

Adicionado na versão 3.3.

18.6 concurrent.interpreters — Múltiplos interpretadores no mesmo processo

Adicionado na versão 3.14.

Source code: Lib/concurrent/interpreters

O módulo concurrent.interpreters constrói interfaces de nível mais alto sobre o módulo de mais baixo nível _interpreters.

The module is primarily meant to provide a basic API for managing interpreters (AKA "subinterpreters") and running things in them. Running mostly involves switching to an interpreter (in the current thread) and calling a function in that execution context.

For concurrency, interpreters themselves (and this module) don't provide much more than isolation, which on its own isn't useful. Actual concurrency is available separately through *threads* See *below*

Ver também

`InterpreterPoolExecutor`
combines threads with interpreters in a familiar interface.

isolating-extensions-howto
como atualizar um módulo de extensão para oferecer suporte a vários interpretadores

PEP 554

PEP 734

PEP 684

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

18.6.1 Detalhes-chave

Before we dive in further, there are a small number of details to keep in mind about using multiple interpreters:

- *isolated*, by default
- nenhuma thread implícita
- nem todos os pacotes PyPI oferecem suporte ao uso em múltiplos interpretadores ainda

18.6.2 Introdução

An "interpreter" is effectively the execution context of the Python runtime. It contains all of the state the runtime needs to execute a program. This includes things like the import state and builtins. (Each thread, even if there's only the main thread, has some extra runtime state, in addition to the current interpreter, related to the current exception and the bytecode eval loop.)

The concept and functionality of the interpreter have been a part of Python since version 2.2, but the feature was only available through the C-API and not well known, and the *isolation* was relatively incomplete until version 3.12.

Multiple Interpreters and Isolation

A Python implementation may support using multiple interpreters in the same process. CPython has this support. Each interpreter is effectively isolated from the others (with a limited number of carefully managed process-global exceptions to the rule).

That isolation is primarily useful as a strong separation between distinct logical components of a program, where you want to have careful control of how those components interact.

Nota

Interpreters in the same process can technically never be strictly isolated from one another since there are few restrictions on memory access within the same process. The Python runtime makes a best effort at isolation but extension modules may easily violate that. Therefore, do not use multiple interpreters in security-sensitive situations, where they shouldn't have access to each other's data.

Running in an Interpreter

Running in a different interpreter involves switching to it in the current thread and then calling some function. The runtime will execute the function using the current interpreter's state. The `concurrent.interpreters` module provides a basic API for creating and managing interpreters, as well as the switch-and-call operation.

No other threads are automatically started for the operation. There is *a helper* for that though. There is another dedicated helper for calling the builtin *exec()* in an interpreter.

When *exec()* (or *eval()*) are called in an interpreter, they run using the interpreter's `__main__` module as the "globals" namespace. The same is true for functions that aren't associated with any module. This is the same as how scripts invoked from the command-line run in the `__main__` module.

Concurrency and Parallelism

As noted earlier, interpreters do not provide any concurrency on their own. They strictly represent the isolated execution context the runtime will use *in the current thread*. That isolation makes them similar to processes, but they still enjoy in-process efficiency, like threads.

All that said, interpreters do naturally support certain flavors of concurrency. There's a powerful side effect of that isolation. It enables a different approach to concurrency than you can take with async or threads. It's a similar concurrency model to CSP or the actor model, a model which is relatively easy to reason about.

You can take advantage of that concurrency model in a single thread, switching back and forth between interpreters, Stackless-style. However, this model is more useful when you combine interpreters with multiple threads. This mostly involves starting a new thread, where you switch to another interpreter and run what you want there.

Each actual thread in Python, even if you're only running in the main thread, has its own *current* execution context. Multiple threads can use the same interpreter or different ones.

At a high level, you can think of the combination of threads and interpreters as threads with opt-in sharing.

As a significant bonus, interpreters are sufficiently isolated that they do not share the *GIL*, which means combining threads with multiple interpreters enables full multi-core parallelism. (This has been the case since Python 3.12.)

Communication Between Interpreters

In practice, multiple interpreters are useful only if we have a way to communicate between them. This usually involves some form of message passing, but can even mean sharing data in some carefully managed way.

With this in mind, the `concurrent.interpreters` module provides a *queue.Queue* implementation, available through *create_queue()*.

"Sharing" Objects

Any data actually shared between interpreters loses the thread-safety provided by the *GIL*. There are various options for dealing with this in extension modules. However, from Python code the lack of thread-safety means objects can't actually be shared, with a few exceptions. Instead, a copy must be created, which means mutable objects won't stay in sync.

By default, most objects are copied with *pickle* when they are passed to another interpreter. Nearly all of the immutable builtin objects are either directly shared or copied efficiently. For example:

- *None*
- *bool* (*True* and *False*)
- *bytes*
- *str*
- *int*
- *float*
- *tuple* (of similarly supported objects)

There is a small number of Python types that actually share mutable data between interpreters:

- *memoryview*
- *Queue*

18.6.3 Referência

Este módulo define as seguintes funções:

concurrent.interpreters.**list_all**()

Retorna *list* de objetos *Interpreter*, um para cada interpretador existente.

concurrent.interpreters.**get_current**()

Retorna um objeto *Interpreter* para o interpretador em execução no momento.

concurrent.interpreters.**get_main**()

Return an *Interpreter* object for the main interpreter. This is the interpreter the runtime created to run the *REPL* or the script given at the command-line. It is usually the only one.

concurrent.interpreters.**create**()

Inicializa um novo interpretador Python (ocioso) e retorna um objeto *Interpreter* para ele.

concurrent.interpreters.**create_queue**()

Initialize a new cross-interpreter queue and return a *Queue* object for it.

Objetos interpretador

class concurrent.interpreters.**Interpreter**(*id*)

Um único interpretador no processo atual.

Geralmente, *Interpreter* não deve ser chamado diretamente. Em vez disso, use *create()* ou uma das outras funções do módulo.

id

(somente leitura)

The underlying interpreter's ID.

whence

(somente leitura)

Uma string descrevendo de onde o interpretador veio.

is_running()

Retorna `True` se o interpretador estiver executando código em seu módulo `__main__` e `False` caso contrário.

close()

Finaliza e destrói o interpretador.

prepare_main(*ns=None*, ***kwargs*)

Bind objects in the interpreter's `__main__` module.

Some objects are actually shared and some are copied efficiently, but most are copied via *pickle*. See *"Sharing" Objects*.

exec(*code*, */*, *dedent=True*)

Executa o código-fonte fornecido no interpretador (na thread atual).

call(*callable*, */*, **args*, ***kwargs*)

Retorna o resultado da chamada da execução da função fornecida no interpretador (na thread atual).

call_in_thread(*callable*, */*, **args*, ***kwargs*)

Executa a função fornecida no interpretador (em uma nova thread).

Exceções

exception concurrent.interpreters.**InterpreterError**

Esta exceção, uma subclasse de *Exception*, é levantada quando ocorre um erro relacionado ao interpretador.

exception concurrent.interpreters.**InterpreterNotFoundError**

Esta exceção, uma subclasse de *InterpreterError*, é levantada quando o interpretador de destino não existe mais.

exception concurrent.interpreters.**ExecutionFailed**

Esta exceção, uma subclasse de *InterpreterError*, é levantada quando o código em execução levanta uma exceção não capturada.

excinfo

Um snapshot básico da exceção levantada no outro interpretador.

exception concurrent.interpreters.**NotShareableError**

Esta exceção, uma subclasse de *TypeError*, é levantada quando um objeto não pode ser enviado para outro interpretador.

Communicating Between Interpreters

class concurrent.interpreters.**Queue**(*id*)

A wrapper around a low-level, cross-interpreter queue, which implements the *queue.Queue* interface. The underlying queue can only be created through *create_queue()*.

Some objects are actually shared and some are copied efficiently, but most are copied via *pickle*. See *"Sharing" Objects*.

id

(somente leitura)

The queue's ID.

exception concurrent.interpreters.**QueueEmptyError**

This exception, a subclass of *queue.Empty*, is raised from `Queue.get()` and `Queue.get_nowait()` when the queue is empty.

exception concurrent.interpreters.**QueueFullError**

This exception, a subclass of *queue.Full*, is raised from `Queue.put()` and `Queue.put_nowait()` when the queue is full.

18.6.4 Uso básico

Criando um interpretador e executando código nele:

```
from concurrent import interpreters

interp = interpreters.create()

# Run in the current OS thread.

interp.exec('print("spam!")')

interp.exec("""if True:
    print('spam!')
    """)

from textwrap import dedent
interp.exec(dedent("""
    print('spam!')
```

(continua na próxima página)

(continuação da página anterior)

```
    """))

def run(arg):
    return arg

res = interp.call(run, 'spam!')
print(res)

def run():
    print('spam!')

interp.call(run)

# Run in new OS thread.

t = interp.call_in_thread(run)
t.join()
```

18.7 `subprocess` — Gerenciamento de subprocessos

Código-fonte: Lib/subprocess.py

O módulo `subprocess` permite que você crie novos processos, conecte-se aos seus canais de entrada/saída/erro e obtenha seus códigos de retorno. Este módulo pretende substituir vários módulos e funções mais antigos:

```
os.system
os.spawn*
```

Informações sobre como o módulo `subprocess` pode ser usado para substituir esses módulos e funções podem ser encontradas nas seções a seguir.

Ver também

PEP 324 – PEP propondo o módulo subprocess

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em *plataformas móveis* ou *plataformas WebAssembly*.

18.7.1 Usando o módulo `subprocess`

A abordagem recomendada para invocar subprocessos é usar a função `run()` para todos os casos de uso que ela pode manipular. Para casos de uso mais avançados, a interface subjacente `Popen` pode ser usada diretamente.

subprocess.**run**(*args*, *, *stdin=None*, *input=None*, *stdout=None*, *stderr=None*, *capture_output=False*, *shell=False*, *cwd=None*, *timeout=None*, *check=False*, *encoding=None*, *errors=None*, *text=None*, *env=None*, *universal_newlines=None*, **other_popen_kwargs*)

Executa o comando descrito por *args*. Aguarda a conclusão do comando e retorna uma instância de `CompletedProcess`.

Os argumentos mostrados acima são meramente os mais comuns, descritos abaixo em *Argumentos usados frequentemente* (daí o uso de notação somente-nomeado na assinatura abreviada). A assinatura da função completa é basicamente a mesma do construtor `Popen` - a maioria dos argumentos para esta função são passados para aquela interface. (*timeout*, *input*, *check* e *capture_output* não são.)

Se *capture_output* for verdadeiro, *stdout* e *stderr* serão capturados. Quando usado, o objeto interno *Popen* é automaticamente criado com *stdout* and *stderr* definidos como *PIPE*. Os argumentos *stdout * e *stderr* não podem ser fornecidos ao mesmo tempo que *capture_output*. Se você deseja capturar e combinar ambos os fluxos em um, defina *stdout* para *PIPE* e *stderr* para *STDOUT*, ao invés de usar *capture_output*.

Um *timeout* pode ser especificado em segundos, ele é passado internamente para *Popen.communicate()*. Se o tempo limite expirar, o processo filho será eliminado e aguardado. A exceção *TimeoutExpired* será levantada novamente após o término do processo filho. A criação inicial do processo em si não pode ser interrompida em muitas APIs de plataforma, portanto, não há garantia de que você verá uma exceção de tempo limite até, pelo menos, o tempo que a criação do processo demorar.

O argumento *input* é passado para *Popen.communicate()* e, portanto, para o stdin do subprocesso. Se usado, deve ser uma sequência de bytes ou uma string se *encoding* ou *errors* forem especificados ou *text* for verdadeiro. Quando usado, o objeto internal *Popen* é automaticamente criado com *PIPE*, e o argumento *stdin* também não pode ser usado.

Se *check* for verdadeiro, e o processo sair com um código de saída diferente de zero, uma exceção *CalledProcessError* será levantada. Os atributos dessa exceção contêm os argumentos, o código de saída e stdout e stderr se eles foram capturados.

Se *encoding* ou *errors* forem especificados, ou *text* for verdadeiro, os objetos arquivo para stdin, stdout e stderr são abertos em modo de texto usando a *encoding* e *errors* especificados ou o *io.TextIOWrapper* padrão. O argumento *universal_newlines* é equivalente a *text* e é fornecido para compatibilidade com versões anteriores. Por padrão, os objetos arquivo são abertos no modo binário.

Se *env* não for `None`, deve ser um mapeamento que defina as variáveis de ambiente para o novo processo; elas são usadas em vez do comportamento padrão de herdar o ambiente do processo atual. É passado diretamente para *Popen*. Este mapeamento pode ser str para str em qualquer plataforma ou bytes para bytes em plataformas POSIX como *os.environ* ou *os.environb*.

Exemplos:

```
>>> subprocess.run(["ls", "-l"])  # não captura a saída
CompletedProcess(args=['ls', '-l'], returncode=0)

>>> subprocess.run("exit 1", shell=True, check=True)
Traceback (most recent call last):
  ...
subprocess.CalledProcessError: Command 'exit 1' returned non-zero exit status 1

>>> subprocess.run(["ls", "-l", "/dev/null"], capture_output=True)
CompletedProcess(args=['ls', '-l', '/dev/null'], returncode=0,
stdout=b'crw-rw-rw- 1 root root 1, 3 Jan 23 16:23 /dev/null\n', stderr=b'')
```

Adicionado na versão 3.5.

Alterado na versão 3.6: Adicionados os parâmetros *encoding* e *errors*.

Alterado na versão 3.7: Adicionado o parâmetro *text*, como um apelido mais compreensível que *universal_newlines*. Adicionado o parâmetro *capture_output*.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

class subprocess.**CompletedProcess**

O valor de retorno de *run()*, representando um processo que foi concluído.

args

Os argumentos usados para iniciar o processo. Pode ser uma lista ou uma string.

returncode

Status de saída do processo filho. Normalmente, um status de saída 0 indica que foi executado com êxito.

Um valor negativo `-N` indica que o filho foi terminado pelo sinal `N` (POSIX apenas).

stdout

Stdout capturado do processo filho. Uma sequência de bytes ou uma string se *run()* foi chamada com uma codificação, erros ou text=True. `None` se stdout não foi capturado.

Se você executou o processo com `stderr=subprocess.STDOUT`, stdout e stderr serão combinados neste atributo, e *stderr* será `None`.

stderr

Stderr capturado do processo filho. Uma sequência de bytes ou uma string se *run()* foi chamada com uma codificação, erros ou text=True. `None` se stderr não foi capturado.

check_returncode()

Se *returncode* é diferente de zero, levantar um *CalledProcessError*.

Adicionado na versão 3.5.

subprocess.**DEVNULL**

Valor especial que pode ser usado como o argumento *stdin*, *stdout* ou *stderr* para *Popen* e indica que o arquivo especial *os.devnull* será usado.

Adicionado na versão 3.3.

subprocess.**PIPE**

Valor especial que pode ser usado como o argumento *stdin*, *stdout* ou *stderr* para *Popen* e indica que um encadeamento para o fluxo padrão deve ser aberto. Mais útil com *Popen.communicate()*.

subprocess.**STDOUT**

Valor especial que pode ser usado como o argumento *stderr* para *Popen* e indica que o erro padrão deve ir para o mesmo manipulador que a saída padrão.

exception subprocess.**SubprocessError**

Classe base para todas as outras exceções deste módulo.

Adicionado na versão 3.3.

exception subprocess.**TimeoutExpired**

Subclasse de *SubprocessError*, levantada quando um tempo limite expira enquanto espera por um processo filho.

cmd

Comando que foi usado para gerar o processo filho.

timeout

Tempo limite em segundos.

output

Saída do processo filho se ele foi capturado por *run()* ou *check_output()*. Caso contrário, `None`. Isto é sempre *bytes* quando qualquer saída foi capturada independente da configuração `text=True`. Pode permanecer `None` em vez de `b''` quando nenhuma saída foi observada.

stdout

Apelido para a saída, para simetria com *stderr*.

stderr

Saída de stderr do processo filho se ele foi capturado por *run()*. Caso contrário, `None`. Isto é sempre *bytes* quando a saída de stderr foi capturada independente da configuração `text=True`. Pode permanecer `None` em vez de `b''` quando nenhuma saída de stderr foi observada.

Adicionado na versão 3.3.

Alterado na versão 3.5: Adicionados os atributos *stdout* e *stderr*

exception subprocess.**CalledProcessError**

Subclasse de *SubprocessError*, levantada quando um processo executado por *check_call()*, *check_output()* ou *run()* (com `check=True`) retorna um status de saída diferente de zero.

returncode

Status de saída do processo filho. Se o processo foi encerrado devido a um sinal, este será o número do sinal negativo.

cmd

Comando que foi usado para gerar o processo filho.

output

Saída do processo filho se ele foi capturado por *run()* ou *check_output()*. Caso contrário, `None`.

stdout

Apelido para a saída, para simetria com *stderr*.

stderr

Saída stderr do processo filho se ele foi capturado por *run()*. Caso contrário, `None`.

Alterado na versão 3.5: Adicionados os atributos *stdout* e *stderr*

Argumentos usados frequentemente

Para suportar uma ampla variedade de casos de uso, o construtor *Popen* (e as funções de conveniência) aceita muitos argumentos opcionais. Para a maioria dos casos de uso típicos, muitos desses argumentos podem ser seguramente deixados em seus valores padrão. Os argumentos mais comumente necessários são:

args é necessário para todas as chamadas e deve ser uma string ou uma sequência de argumentos do programa. Fornecer uma sequência de argumentos geralmente é preferível, pois permite que o módulo cuide de qualquer escape e aspas em argumentos que forem necessários (por exemplo, para permitir espaços em nomes de arquivo). Se for passada uma única string, *shell* deve ser *True* (veja abaixo) ou então a string deve apenas nomear o programa a ser executado sem especificar nenhum argumento.

stdin, *stdout* e *stderr* especificam a entrada padrão, a saída padrão e a saída de erro do programa executado, respectivamente. Os valores válidos são `None`, *PIPE*, *DEVNULL*, um descritor de arquivo existente (um número inteiro positivo) e um *objeto arquivo* existente com um descritor de arquivo válido. Com as configurações padrão de `None`, nenhum redirecionamento ocorrerá. *PIPE* indica que um novo canal para o filho deve ser criado. *DEVNULL* indica que o arquivo especial *os.devnull* será usado. Além disso, *stderr* pode ser *STDOUT*, o que indica que os dados de stderr do processo filho devem ser capturados no mesmo identificador de arquivo que para *stdout*.

Se *encoding* ou *errors* forem especificados, ou *text* (também conhecido como *universal_newlines*) for verdadeiro, os objetos arquivo *stdin*, *stdout* e *stderr* serão abertos em modo de texto usando a *encoding* e *errors* especificados na chamada ou os valores padrão para *io.TextIOWrapper*.

Para *stdin*, os caracteres de fim de linha `'\n'` na entrada serão convertidos para o separador de linha padrão *os.linesep*. Para *stdout* e *stderr*, todas as terminações de linha na saída serão convertidas para `'\n'`. Para obter mais informações, consulte a documentação da classe *io.TextIOWrapper* quando o argumento *newline* para seu construtor é `None`.

Se o modo de texto não for usado, *stdin*, *stdout* e *stderr* serão abertos como fluxos binários. Nenhuma codificação ou conversão de final de linha é executada.

Alterado na versão 3.6: Adicionados os parâmetros *encoding* e *errors*.

Alterado na versão 3.7: Adicionado o parâmetro *text* como um apelido para *universal_newlines*.

> **Nota**
>
> O atributo newlines dos objetos arquivo *Popen.stdin*, *Popen.stdout* e *Popen.stderr* não são atualizados pelo método *Popen.communicate()*.

Se *shell* for `True`, o comando especificado será executado através do shell. Isso pode ser útil se você estiver usando Python principalmente para o fluxo de controle aprimorado que ele oferece sobre a maioria dos shells do sistema e ainda deseja acesso conveniente a outros recursos do shell, como canais de shell, caracteres curinga de nome de arquivo, expansão de variável de ambiente e expansão de ~ para o diretório inicial de um usuário. No entanto, observe que o próprio Python oferece implementações de muitos recursos semelhantes a shell (em particular, *glob*, *fnmatch*, *os.walk()*, *os.path.expandvars()*, *os.path.expanduser()* e *shutil*).

Alterado na versão 3.3: Quando *universal_newlines* é `True`, a classe usa a codificação *locale.getpreferredencoding(False)* em vez de `locale.getpreferredencoding()`. Veja a classe *io.TextIOWrapper* para mais informações sobre esta alteração.

Nota

Leia a seção *Security Considerations* antes de usar `shell=True`.

Essas opções, junto com todas as outras opções, são descritas em mais detalhes na documentação do construtor *Popen*.

Construtor Popen

A criação e gerenciamento do processo subjacente neste módulo é manipulado pela classe *Popen*. Ela oferece muita flexibilidade para que os desenvolvedores sejam capazes de lidar com os casos menos comuns não cobertos pelas funções de conveniência.

class subprocess.**Popen**(*args*, *bufsize=-1*, *executable=None*, *stdin=None*, *stdout=None*, *stderr=None*, *preexec_fn=None*, *close_fds=True*, *shell=False*, *cwd=None*, *env=None*, *universal_newlines=None*, *startupinfo=None*, *creationflags=0*, *restore_signals=True*, *start_new_session=False*, *pass_fds=()*, ***, *group=None*, *extra_groups=None*, *user=None*, *umask=-1*, *encoding=None*, *errors=None*, *text=None*, *pipesize=-1*, *process_group=None*)

Executa um programa filho em um novo processo. No POSIX, a classe usa o comportamento de *os.execvpe()* para executar o programa filho. No Windows, a classe usa a função Windows `CreateProcess()`. Os argumentos para *Popen* são os seguintes.

args deve ser uma sequência de argumentos do programa ou então uma única string ou *objeto caminho ou similar*. Por padrão, o programa a ser executado é o primeiro item em *args* se *args* for uma sequência. Se *args* for uma string, a interpretação depende da plataforma e é descrita abaixo. Consulte os argumentos *shell* e *executable* para obter diferenças adicionais em relação ao comportamento padrão. Salvo indicação em contrário, é recomendado passar *args* como uma sequência.

Aviso

Para confiabilidade máxima, use um caminho totalmente qualificado para o executável. Para procurar um nome não qualificado no `PATH`, use *shutil.which()*. Em todas as plataformas, passar *sys.executable* é a maneira recomendada de relançar o interpretador Python atual, e usar o formato de linha de comando `-m` para iniciar um módulo instalado.

Resolver o caminho do executável (ou o primeiro item de *args*) depende da plataforma. Para POSIX, veja *os.execvpe()*, e note que ao resolver ou procurar o caminho do executável, *cwd* substitui o diretório de trabalho atual e *env* pode substituir a variável de ambiente `PATH`. Para Windows, veja a documentação dos parâmetros `lpApplicationName` e `lpCommandLine` do WinAPI `CreateProcess`, e note que ao resolver ou procurar o caminho do executável com `shell=False`, *cwd* não substitui o diretório de trabalho atual e *env* não pode substituir a variável de ambiente `PATH`. Usar um caminho completo evita todas essas variações.

Um exemplo de passagem de alguns argumentos para um programa externo como uma sequência é:

```
Popen(["/usr/bin/git", "commit", "-m", "Fixes a bug."])
```

No POSIX, se *args* for uma string, a string é interpretada como o nome ou caminho do programa a ser executado. No entanto, isso só pode ser feito se não forem passados argumentos para o programa.

Nota

Pode não ser óbvio como quebrar um comando shell em uma sequência de argumentos, especialmente em casos complexos. *shlex.split()* pode ilustrar como determinar a tokenização correta para *args*:

```
>>> import shlex, subprocess
>>> command_line = input()
/bin/vikings -input eggs.txt -output "spam spam.txt" -cmd "echo '$MONEY'"
>>> args = shlex.split(command_line)
>>> print(args)
['/bin/vikings', '-input', 'eggs.txt', '-output', 'spam spam.txt', '-cmd',
↪"echo '$MONEY'"]
>>> p = subprocess.Popen(args) # Successo!
```

Observe em particular que as opções (como *-input*) e argumentos (como *eggs.txt*) que são separados por espaços em branco no shell vão em elementos de lista separados, enquanto os argumentos que precisam de aspas ou escape de contrabarra quando usados no shell (como nomes de arquivos contendo espaços ou o comando *echo* mostrado acima) são um único elemento de lista.

No Windows, se *args* for uma sequência, será convertido em uma string da maneira descrita em *Converter uma sequência de argumentos em uma string no Windows*. Isso ocorre porque o `CreateProcess()` subjacente opera em strings.

Alterado na versão 3.6: O parâmetro *args* aceita um *objeto caminho ou similar* se *shell* é `False` e uma sequência contendo objetos caminho ou similares no POSIX.

Alterado na versão 3.8: O parâmetro *args* aceita um *objeto caminho ou similar* se *shell* for `False` e uma sequência contendo bytes e objetos caminho ou similares no Windows.

O argumento *shell* (que por padrão é `False`) especifica se deve usar o *shell* como o programa a ser executado. Se *shell* for `True`, é recomendado passar *args* como uma string em vez de uma sequência.

Em POSIX com `shell=True`, o *shell* padrão é `/bin/sh`. Se *args* for uma string, a *string* especifica o comando a ser executado através do *shell*. Isso significa que a *string* deve ser formatada exatamente como seria ao ser digitada no prompt do *shell*. Isso inclui, por exemplo, citar ou escapar com contrabarra nomes de arquivos que contenham espaços. Se *args* for uma sequência, o primeiro item especifica a *string* do comando, e quaisquer itens adicionais serão tratados como argumentos adicionais para o próprio *shell*. Ou seja, *Popen* faz o equivalente a:

```
Popen(['/bin/sh', '-c', args[0], args[1], ...])
```

No Windows com `shell=True`, a variável de ambiente `COMSPEC` especifica o shell padrão. A única vez que você precisa especificar `shell=True` no Windows é quando o comando que você deseja executar é construído no shell (por exemplo, **`dir`** ou **`copy`**). Você não precisa de `shell=True` para executar um arquivo em lote ou executável baseado em console.

Nota

Leia a seção *Security Considerations* antes de usar `shell=True`.

bufsize será fornecido como o argumento correspondente à função *open()* ao criar os objetos arquivo de encadeamento stdin/stdout/stderr:

- `0` significa sem buffer (leitura e gravação são uma chamada de sistema e podem retornar curto)

- `1` significa buffer de linha (utilizável somente se `text=True` ou `universal_newlines=True`)
- qualquer outro valor positivo significa usar um buffer de aproximadamente esse tamanho
- bufsize negativo (o padrão) significa que o padrão do sistema io.DEFAULT_BUFFER_SIZE será usado.

Alterado na versão 3.3.1: *bufsize* agora assume o padrão -1 para habilitar o buffer por padrão para corresponder ao comportamento que a maioria dos códigos espera. Em versões anteriores ao Python 3.2.4 e 3.3.1, ele assumia incorretamente o padrão `0`, que não tinha buffer e permitia leituras curtas. Isso não foi intencional e não correspondeu ao comportamento do Python 2, como a maioria dos códigos esperava.

O argumento *executable* especifica um programa de substituição para executar. Ele é muito raramente necessário. Quando `shell=False`, *executable* substitui o programa a ser executado especificado por *args*. No entanto, o *args* original ainda é passado para o programa. A maioria dos programas trata o programa especificado por *args* como o nome do comando, que pode então ser diferente do programa realmente executado. No POSIX, o nome *args* se torna o nome de exibição para o executável em utilitários como **`ps`**. Se `shell=True`, no POSIX o argumento *executable* especifica um shell de substituição para o padrão `/bin/sh`.

Alterado na versão 3.6: O parâmetro *executable* aceita um *objeto caminho ou similar* no POSIX.

Alterado na versão 3.8: O parâmetro *executable* aceita um *objeto caminho ou similar* bytes no Windows.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

stdin, *stdout* e *stderr* especificam a entrada padrão, a saída padrão e a saída de erro do programa executado, respectivamente. Os valores válidos são `None`, *`PIPE`*, *`DEVNULL`*, um descritor de arquivo existente (um número inteiro positivo) e um *objeto arquivo* existente com um descritor de arquivo válido. Com as configurações padrão de `None`, nenhum redirecionamento ocorrerá. *`PIPE`* indica que um novo canal para o filho deve ser criado. *`DEVNULL`* indica que o arquivo especial *`os.devnull`* será usado. Além disso, *stderr* pode ser *`STDOUT`*, o que indica que os dados de stderr da aplicação devem ser capturados no mesmo identificador de arquivo que para *stdout*.

Se *preexec_fn* for definido como um objeto chamável, esse objeto será chamado no processo filho pouco antes de o filho ser executado. (Somente POSIX)

Aviso

O parâmetro *preexec_fn* NÃO É SEGURO para uso na presença de threads em sua aplicação. O processo filho pode travar antes de exec ser chamado.

Nota

Se você precisar modificar o ambiente para o filho, use o parâmetro *env* em vez de fazer isso em um *preexec_fn*. Os parâmetros *start_new_session* e *process_group* devem tomar o lugar do código usando *preexec_fn* para chamar *`os.setsid()`* ou *`os.setpgid()`* no filho.

Alterado na versão 3.8: O parâmetro *preexec_fn* não é mais suportado em subinterpretadores. O uso do parâmetro em um subinterpretador levanta *`RuntimeError`*. A nova restrição pode afetar aplicações que são implantadas em mod_wsgi, uWSGI e outros ambientes incorporados.

If *close_fds* is true, all file descriptors except `0`, `1` and `2` will be closed before the child process is executed. Otherwise when *close_fds* is false, file descriptors obey their inheritable flag as described in *Herança de descritores de arquivos*.

On Windows, if *close_fds* is true then no handles will be inherited by the child process unless explicitly passed in the `handle_list` element of *`STARTUPINFO.lpAttributeList`*, or by standard handle redirection.

Alterado na versão 3.2: The default for *close_fds* was changed from *`False`* to what is described above.

Alterado na versão 3.7: On Windows the default for *close_fds* was changed from `False` to `True` when redirecting the standard handles. It's now possible to set *close_fds* to `True` when redirecting the standard handles.

pass_fds is an optional sequence of file descriptors to keep open between the parent and child. Providing any *pass_fds* forces *close_fds* to be `True`. (POSIX only)

Alterado na versão 3.2: O parâmetro *pass_fds* foi adicionado.

If *cwd* is not `None`, the function changes the working directory to *cwd* before executing the child. *cwd* can be a string, bytes or *path-like* object. On POSIX, the function looks for *executable* (or for the first item in *args*) relative to *cwd* if the executable path is a relative path.

Alterado na versão 3.6: *cwd* parameter accepts a *path-like object* on POSIX.

Alterado na versão 3.7: *cwd* parameter accepts a *path-like object* on Windows.

Alterado na versão 3.8: *cwd* parameter accepts a bytes object on Windows.

If *restore_signals* is true (the default) all signals that Python has set to SIG_IGN are restored to SIG_DFL in the child process before the exec. Currently this includes the SIGPIPE, SIGXFZ and SIGXFSZ signals. (POSIX only)

Alterado na versão 3.2: *restore_signals* foi adicionado.

If *start_new_session* is true the `setsid()` system call will be made in the child process prior to the execution of the subprocess.

Disponibilidade: POSIX

Alterado na versão 3.2: *start_new_session* foi adicionado.

If *process_group* is a non-negative integer, the `setpgid(0, value)` system call will be made in the child process prior to the execution of the subprocess.

Disponibilidade: POSIX

Alterado na versão 3.11: *process_group* was added.

If *group* is not `None`, the setregid() system call will be made in the child process prior to the execution of the subprocess. If the provided value is a string, it will be looked up via `grp.getgrnam()` and the value in `gr_gid` will be used. If the value is an integer, it will be passed verbatim. (POSIX only)

Disponibilidade: POSIX

Adicionado na versão 3.9.

If *extra_groups* is not `None`, the setgroups() system call will be made in the child process prior to the execution of the subprocess. Strings provided in *extra_groups* will be looked up via `grp.getgrnam()` and the values in `gr_gid` will be used. Integer values will be passed verbatim. (POSIX only)

Disponibilidade: POSIX

Adicionado na versão 3.9.

If *user* is not `None`, the setreuid() system call will be made in the child process prior to the execution of the subprocess. If the provided value is a string, it will be looked up via `pwd.getpwnam()` and the value in `pw_uid` will be used. If the value is an integer, it will be passed verbatim. (POSIX only)

Disponibilidade: POSIX

Adicionado na versão 3.9.

If *umask* is not negative, the umask() system call will be made in the child process prior to the execution of the subprocess.

Disponibilidade: POSIX

Adicionado na versão 3.9.

If *env* is not `None`, it must be a mapping that defines the environment variables for the new process; these are used instead of the default behavior of inheriting the current process' environment. This mapping can be str to str on any platform or bytes to bytes on POSIX platforms much like `os.environ` or `os.environb`.

Nota

If specified, *env* must provide any variables required for the program to execute. On Windows, in order to run a side-by-side assembly the specified *env* **must** include a valid `SystemRoot`.

If *encoding* or *errors* are specified, or *text* is true, the file objects *stdin*, *stdout* and *stderr* are opened in text mode with the specified *encoding* and *errors*, as described above in *Argumentos usados frequentemente*. The *universal_newlines* argument is equivalent to *text* and is provided for backwards compatibility. By default, file objects are opened in binary mode.

Adicionado na versão 3.6: *encoding* e *errors* foram adicionados.

Adicionado na versão 3.7: *text* foi adicionado como um atalho mais legível para *universal_newlines*.

If given, *startupinfo* will be a *STARTUPINFO* object, which is passed to the underlying `CreateProcess` function.

If given, *creationflags*, can be one or more of the following flags:

- *CREATE_NEW_CONSOLE*
- *CREATE_NEW_PROCESS_GROUP*
- *ABOVE_NORMAL_PRIORITY_CLASS*
- *BELOW_NORMAL_PRIORITY_CLASS*
- *HIGH_PRIORITY_CLASS*
- *IDLE_PRIORITY_CLASS*
- *NORMAL_PRIORITY_CLASS*
- *REALTIME_PRIORITY_CLASS*
- *CREATE_NO_WINDOW*
- *DETACHED_PROCESS*
- *CREATE_DEFAULT_ERROR_MODE*
- *CREATE_BREAKAWAY_FROM_JOB*

pipesize can be used to change the size of the pipe when *PIPE* is used for *stdin*, *stdout* or *stderr*. The size of the pipe is only changed on platforms that support this (only Linux at this time of writing). Other platforms will ignore this parameter.

Alterado na versão 3.10: Added the *pipesize* parameter.

Popen objects are supported as context managers via the `with` statement: on exit, standard file descriptors are closed, and the process is waited for.

```
with Popen(["ifconfig"], stdout=PIPE) as proc:
    log.write(proc.stdout.read())
```

Popen and the other functions in this module that use it raise an *auditing event* `subprocess.Popen` with arguments `executable`, `args`, `cwd`, and `env`. The value for `args` may be a single string or a list of strings, depending on platform.

Alterado na versão 3.2: Adicionado suporte a gerenciador de contexto.

Alterado na versão 3.6: O destruidor Popen agora emite um aviso *ResourceWarning* se o processo filho ainda estiver em execução.

Alterado na versão 3.8: Popen can use *os.posix_spawn()* in some cases for better performance. On Windows Subsystem for Linux and QEMU User Emulation, Popen constructor using *os.posix_spawn()* no longer raise an exception on errors like missing program, but the child process fails with a non-zero *returncode*.

Exceções

Exceções levantadas no processo filho, antes que o novo programa comece a ser executado, serão levantadas novamente no pai.

The most common exception raised is *OSError*. This occurs, for example, when trying to execute a non-existent file. Applications should prepare for *OSError* exceptions. Note that, when `shell=True`, *OSError* will be raised by the child only if the selected shell itself was not found. To determine if the shell failed to find the requested application, it is necessary to check the return code or output from the subprocess.

A exceção *ValueError* será levantada se *Popen* for chamado com argumentos inválidos.

check_call() e *check_output()* levantarão *CalledProcessError* se o processo chamado retornar um código de retorno diferente de zero.

All of the functions and methods that accept a *timeout* parameter, such as *run()* and *Popen.communicate()* will raise *TimeoutExpired* if the timeout expires before the process exits.

Todas as exceções definidas neste módulo herdam de *SubprocessError*.

Adicionado na versão 3.3: A classe base *SubprocessError* foi adicionada.

18.7.2 Considerações de Segurança

Unlike some other popen functions, this library will not implicitly choose to call a system shell. This means that all characters, including shell metacharacters, can safely be passed to child processes. If the shell is invoked explicitly, via `shell=True`, it is the application's responsibility to ensure that all whitespace and metacharacters are quoted appropriately to avoid shell injection vulnerabilities. On *some platforms*, it is possible to use *shlex.quote()* for this escaping.

On Windows, batch files (`*.bat` or `*.cmd`) may be launched by the operating system in a system shell regardless of the arguments passed to this library. This could result in arguments being parsed according to shell rules, but without any escaping added by Python. If you are intentionally launching a batch file with arguments from untrusted sources, consider passing `shell=True` to allow Python to escape special characters. See gh-114539 for additional discussion.

18.7.3 Objetos Popen

Instâncias da classe *Popen* têm os seguintes métodos:

`Popen.`**`poll`**`()`

Check if child process has terminated. Set and return *returncode* attribute. Otherwise, returns `None`.

`Popen.`**`wait`**`(`*timeout=None*`)`

Wait for child process to terminate. Set and return *returncode* attribute.

If the process does not terminate after *timeout* seconds, raise a *TimeoutExpired* exception. It is safe to catch this exception and retry the wait.

> **Nota**
>
> This will deadlock when using `stdout=PIPE` or `stderr=PIPE` and the child process generates enough output to a pipe such that it blocks waiting for the OS pipe buffer to accept more data. Use *Popen.communicate()* when using pipes to avoid that.

> **Nota**
>
> When the `timeout` parameter is not `None`, then (on POSIX) the function is implemented using a busy loop (non-blocking call and short sleeps). Use the *asyncio* module for an asynchronous wait: see *asyncio.create_subprocess_exec*.

Alterado na versão 3.3: *timeout* foi adicionado.

`Popen.`**`communicate`**(*input=None*, *timeout=None*)

Interact with process: Send data to stdin. Read data from stdout and stderr, until end-of-file is reached. Wait for process to terminate and set the *returncode* attribute. The optional *input* argument should be data to be sent to the child process, or `None`, if no data should be sent to the child. If streams were opened in text mode, *input* must be a string. Otherwise, it must be bytes.

communicate() returns a tuple `(stdout_data, stderr_data)`. The data will be strings if streams were opened in text mode; otherwise, bytes.

Note that if you want to send data to the process's stdin, you need to create the Popen object with `stdin=PIPE`. Similarly, to get anything other than `None` in the result tuple, you need to give `stdout=PIPE` and/or `stderr=PIPE` too.

If the process does not terminate after *timeout* seconds, a *TimeoutExpired* exception will be raised. Catching this exception and retrying communication will not lose any output.

The child process is not killed if the timeout expires, so in order to cleanup properly a well-behaved application should kill the child process and finish communication:

```
proc = subprocess.Popen(...)
try:
    outs, errs = proc.communicate(timeout=15)
except TimeoutExpired:
    proc.kill()
    outs, errs = proc.communicate()
```

Nota

The data read is buffered in memory, so do not use this method if the data size is large or unlimited.

Alterado na versão 3.3: *timeout* foi adicionado.

`Popen.`**`send_signal`**(*signal*)

Envia o sinal *signal* para o filho.

Não faz nada se o processo for concluído.

Nota

On Windows, SIGTERM is an alias for *terminate()*. CTRL_C_EVENT and CTRL_BREAK_EVENT can be sent to processes started with a *creationflags* parameter which includes `CREATE_NEW_PROCESS_GROUP`.

`Popen.`**`terminate`**()

Stop the child. On POSIX OSs the method sends *SIGTERM* to the child. On Windows the Win32 API function `TerminateProcess()` is called to stop the child.

`Popen.`**`kill`**()

Kills the child. On POSIX OSs the function sends SIGKILL to the child. On Windows *kill()* is an alias for *terminate()*.

The following attributes are also set by the class for you to access. Reassigning them to new values is unsupported:

`Popen.`**`args`**

O argumento *args* conforme foi passado para *Popen* - uma sequência de argumentos do programa ou então uma única string.

Adicionado na versão 3.3.

Popen.**stdin**

If the *stdin* argument was *PIPE*, this attribute is a writeable stream object as returned by *open()*. If the *encoding* or *errors* arguments were specified or the *text* or *universal_newlines* argument was True, the stream is a text stream, otherwise it is a byte stream. If the *stdin* argument was not *PIPE*, this attribute is None.

Popen.**stdout**

If the *stdout* argument was *PIPE*, this attribute is a readable stream object as returned by *open()*. Reading from the stream provides output from the child process. If the *encoding* or *errors* arguments were specified or the *text* or *universal_newlines* argument was True, the stream is a text stream, otherwise it is a byte stream. If the *stdout* argument was not *PIPE*, this attribute is None.

Popen.**stderr**

If the *stderr* argument was *PIPE*, this attribute is a readable stream object as returned by *open()*. Reading from the stream provides error output from the child process. If the *encoding* or *errors* arguments were specified or the *text* or *universal_newlines* argument was True, the stream is a text stream, otherwise it is a byte stream. If the *stderr* argument was not *PIPE*, this attribute is None.

Aviso

Use *communicate()* rather than *.stdin.write*, *.stdout.read* or *.stderr.read* to avoid deadlocks due to any of the other OS pipe buffers filling up and blocking the child process.

Popen.**pid**

O ID de processo do processo filho.

Observe que se você definir o argumento *shell* como True, este é o ID do processo do shell gerado.

Popen.**returncode**

The child return code. Initially None, *returncode* is set by a call to the *poll()*, *wait()*, or *communicate()* methods if they detect that the process has terminated.

A None value indicates that the process hadn't yet terminated at the time of the last method call.

Um valor negativo -N indica que o filho foi terminado pelo sinal N (POSIX apenas).

18.7.4 Windows Popen Helpers

The *STARTUPINFO* class and following constants are only available on Windows.

class subprocess.**STARTUPINFO**(**, dwFlags=0, hStdInput=None, hStdOutput=None, hStdError=None, wShowWindow=0, lpAttributeList=None*)

Partial support of the Windows STARTUPINFO structure is used for *Popen* creation. The following attributes can be set by passing them as keyword-only arguments.

Alterado na versão 3.7: Keyword-only argument support was added.

dwFlags

A bit field that determines whether certain *STARTUPINFO* attributes are used when the process creates a window.

```
si = subprocess.STARTUPINFO()
si.dwFlags = subprocess.STARTF_USESTDHANDLES | subprocess.STARTF_
↪USESHOWWINDOW
```

hStdInput

If *dwFlags* specifies *STARTF_USESTDHANDLES*, this attribute is the standard input handle for the process. If *STARTF_USESTDHANDLES* is not specified, the default for standard input is the keyboard buffer.

hStdOutput

If *dwFlags* specifies *STARTF_USESTDHANDLES*, this attribute is the standard output handle for the process. Otherwise, this attribute is ignored and the default for standard output is the console window's buffer.

hStdError

If *dwFlags* specifies *STARTF_USESTDHANDLES*, this attribute is the standard error handle for the process. Otherwise, this attribute is ignored and the default for standard error is the console window's buffer.

wShowWindow

If *dwFlags* specifies *STARTF_USESHOWWINDOW*, this attribute can be any of the values that can be specified in the `nCmdShow` parameter for the ShowWindow function, except for `SW_SHOWDEFAULT`. Otherwise, this attribute is ignored.

SW_HIDE is provided for this attribute. It is used when *Popen* is called with `shell=True`.

lpAttributeList

A dictionary of additional attributes for process creation as given in `STARTUPINFOEX`, see UpdateProcThreadAttribute.

Atributos suportados:

handle_list
Sequence of handles that will be inherited. *close_fds* must be true if non-empty.

The handles must be temporarily made inheritable by *os.set_handle_inheritable()* when passed to the *Popen* constructor, else *OSError* will be raised with Windows error `ERROR_INVALID_PARAMETER` (87).

> **Aviso**
>
> In a multithreaded process, use caution to avoid leaking handles that are marked inheritable when combining this feature with concurrent calls to other process creation functions that inherit all handles such as *os.system()*. This also applies to standard handle redirection, which temporarily creates inheritable handles.

Adicionado na versão 3.7.

Constantes do Windows

The *subprocess* module exposes the following constants.

`subprocess.`**STD_INPUT_HANDLE**
The standard input device. Initially, this is the console input buffer, `CONIN$`.

`subprocess.`**STD_OUTPUT_HANDLE**
The standard output device. Initially, this is the active console screen buffer, `CONOUT$`.

`subprocess.`**STD_ERROR_HANDLE**
The standard error device. Initially, this is the active console screen buffer, `CONOUT$`.

`subprocess.`**SW_HIDE**
Oculta a janela. Outra janela será ativada.

`subprocess.`**STARTF_USESTDHANDLES**
Specifies that the *STARTUPINFO.hStdInput*, *STARTUPINFO.hStdOutput*, and *STARTUPINFO.hStdError* attributes contain additional information.

`subprocess.`**STARTF_USESHOWWINDOW**
Specifies that the *STARTUPINFO.wShowWindow* attribute contains additional information.

subprocess.**STARTF_FORCEONFEEDBACK**

A *STARTUPINFO.dwFlags* parameter to specify that the *Working in Background* mouse cursor will be displayed while a process is launching. This is the default behavior for GUI processes.

Adicionado na versão 3.13.

subprocess.**STARTF_FORCEOFFFEEDBACK**

A *STARTUPINFO.dwFlags* parameter to specify that the mouse cursor will not be changed when launching a process.

Adicionado na versão 3.13.

subprocess.**CREATE_NEW_CONSOLE**

The new process has a new console, instead of inheriting its parent's console (the default).

subprocess.**CREATE_NEW_PROCESS_GROUP**

A *Popen* `creationflags` parameter to specify that a new process group will be created. This flag is necessary for using *os.kill()* on the subprocess.

This flag is ignored if *CREATE_NEW_CONSOLE* is specified.

subprocess.**ABOVE_NORMAL_PRIORITY_CLASS**

A *Popen* `creationflags` parameter to specify that a new process will have an above average priority.

Adicionado na versão 3.7.

subprocess.**BELOW_NORMAL_PRIORITY_CLASS**

A *Popen* `creationflags` parameter to specify that a new process will have a below average priority.

Adicionado na versão 3.7.

subprocess.**HIGH_PRIORITY_CLASS**

A *Popen* `creationflags` parameter to specify that a new process will have a high priority.

Adicionado na versão 3.7.

subprocess.**IDLE_PRIORITY_CLASS**

A *Popen* `creationflags` parameter to specify that a new process will have an idle (lowest) priority.

Adicionado na versão 3.7.

subprocess.**NORMAL_PRIORITY_CLASS**

A *Popen* `creationflags` parameter to specify that a new process will have a normal priority. (default)

Adicionado na versão 3.7.

subprocess.**REALTIME_PRIORITY_CLASS**

A *Popen* `creationflags` parameter to specify that a new process will have realtime priority. You should almost never use REALTIME_PRIORITY_CLASS, because this interrupts system threads that manage mouse input, keyboard input, and background disk flushing. This class can be appropriate for applications that "talk" directly to hardware or that perform brief tasks that should have limited interruptions.

Adicionado na versão 3.7.

subprocess.**CREATE_NO_WINDOW**

A *Popen* `creationflags` parameter to specify that a new process will not create a window.

Adicionado na versão 3.7.

subprocess.**DETACHED_PROCESS**

A *Popen* `creationflags` parameter to specify that a new process will not inherit its parent's console. This value cannot be used with CREATE_NEW_CONSOLE.

Adicionado na versão 3.7.

subprocess.**CREATE_DEFAULT_ERROR_MODE**

A *Popen* `creationflags` parameter to specify that a new process does not inherit the error mode of the calling process. Instead, the new process gets the default error mode. This feature is particularly useful for multithreaded shell applications that run with hard errors disabled.

Adicionado na versão 3.7.

subprocess.**CREATE_BREAKAWAY_FROM_JOB**

A *Popen* `creationflags` parameter to specify that a new process is not associated with the job.

Adicionado na versão 3.7.

18.7.5 API de alto nível mais antiga

Antes do Python 3.5, essas três funções constituíam a API de alto nível para subprocesso. Agora você pode usar *run()* em muitos casos, mas muitos códigos existentes chamam essas funções.

subprocess.**call**(*args*, *, *stdin=None*, *stdout=None*, *stderr=None*, *shell=False*, *cwd=None*, *timeout=None*, ***other_popen_kwargs*)

Run the command described by *args*. Wait for command to complete, then return the *returncode* attribute.

Code needing to capture stdout or stderr should use *run()* instead:

```
run(...).returncode
```

To suppress stdout or stderr, supply a value of *DEVNULL*.

The arguments shown above are merely some common ones. The full function signature is the same as that of the *Popen* constructor - this function passes all supplied arguments other than *timeout* directly through to that interface.

Nota

Do not use `stdout=PIPE` or `stderr=PIPE` with this function. The child process will block if it generates enough output to a pipe to fill up the OS pipe buffer as the pipes are not being read from.

Alterado na versão 3.3: *timeout* foi adicionado.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

subprocess.**check_call**(*args*, *, *stdin=None*, *stdout=None*, *stderr=None*, *shell=False*, *cwd=None*, *timeout=None*, ***other_popen_kwargs*)

Run command with arguments. Wait for command to complete. If the return code was zero then return, otherwise raise *CalledProcessError*. The *CalledProcessError* object will have the return code in the *returncode* attribute. If *check_call()* was unable to start the process it will propagate the exception that was raised.

Code needing to capture stdout or stderr should use *run()* instead:

```
run(..., check=True)
```

To suppress stdout or stderr, supply a value of *DEVNULL*.

The arguments shown above are merely some common ones. The full function signature is the same as that of the *Popen* constructor - this function passes all supplied arguments other than *timeout* directly through to that interface.

> **Nota**
>
> Do not use `stdout=PIPE` or `stderr=PIPE` with this function. The child process will block if it generates enough output to a pipe to fill up the OS pipe buffer as the pipes are not being read from.

Alterado na versão 3.3: *timeout* foi adicionado.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

`subprocess.`**`check_output`**(*args*, ***, *stdin=None*, *stderr=None*, *shell=False*, *cwd=None*, *encoding=None*, *errors=None*, *universal_newlines=None*, *timeout=None*, *text=None*, ***other_popen_kwargs*)

Executa o comando com argumentos e retorna sua saída.

If the return code was non-zero it raises a *CalledProcessError*. The *CalledProcessError* object will have the return code in the *returncode* attribute and any output in the *output* attribute.

Isso equivale a:

```
run(..., check=True, stdout=PIPE).stdout
```

The arguments shown above are merely some common ones. The full function signature is largely the same as that of *run()* - most arguments are passed directly through to that interface. One API deviation from *run()* behavior exists: passing `input=None` will behave the same as `input=b''` (or `input=''`, depending on other arguments) rather than using the parent's standard input file handle.

By default, this function will return the data as encoded bytes. The actual encoding of the output data may depend on the command being invoked, so the decoding to text will often need to be handled at the application level.

This behaviour may be overridden by setting *text*, *encoding*, *errors*, or *universal_newlines* to `True` as described in *Argumentos usados frequentemente* and *run()*.

To also capture standard error in the result, use `stderr=subprocess.STDOUT`:

```
>>> subprocess.check_output(
...     "ls non_existent_file; exit 0",
...     stderr=subprocess.STDOUT,
...     shell=True)
'ls: non_existent_file: No such file or directory\n'
```

Adicionado na versão 3.1.

Alterado na versão 3.3: *timeout* foi adicionado.

Alterado na versão 3.4: Support for the *input* keyword argument was added.

Alterado na versão 3.6: *encoding* and *errors* were added. See *run()* for details.

Adicionado na versão 3.7: *text* foi adicionado como um atalho mais legível para *universal_newlines*.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

18.7.6 Replacing Older Functions with the `subprocess` Module

In this section, "a becomes b" means that b can be used as a replacement for a.

Nota

All “a” functions in this section fail (more or less) silently if the executed program cannot be found; the “b” replacements raise *OSError* instead.

In addition, the replacements using *check_output()* will fail with a *CalledProcessError* if the requested operation produces a non-zero return code. The output is still available as the *output* attribute of the raised exception.

In the following examples, we assume that the relevant functions have already been imported from the *subprocess* module.

Replacing `/bin/sh` shell command substitution

```
output=$(mycmd myarg)
```

torna-se:

```
output = check_output(["mycmd", "myarg"])
```

Replacing shell pipeline

```
output=$(dmesg | grep hda)
```

torna-se:

```
p1 = Popen(["dmesg"], stdout=PIPE)
p2 = Popen(["grep", "hda"], stdin=p1.stdout, stdout=PIPE)
p1.stdout.close()  # Allow p1 to receive a SIGPIPE if p2 exits.
output = p2.communicate()[0]
```

The `p1.stdout.close()` call after starting the p2 is important in order for p1 to receive a SIGPIPE if p2 exits before p1.

Alternatively, for trusted input, the shell's own pipeline support may still be used directly:

```
output=$(dmesg | grep hda)
```

torna-se:

```
output = check_output("dmesg | grep hda", shell=True)
```

Substituindo `os.system()`

```
sts = os.system("mycmd" + " myarg")
# becomes
retcode = call("mycmd" + " myarg", shell=True)
```

Notas:

- Calling the program through the shell is usually not required.
- The *call()* return value is encoded differently to that of *os.system()*.
- The *os.system()* function ignores SIGINT and SIGQUIT signals while the command is running, but the caller must do this separately when using the *subprocess* module.

Um exemplo mais realista ficaria assim:

```
try:
    retcode = call("mycmd" + " myarg", shell=True)
    if retcode < 0:
        print("Child was terminated by signal", -retcode, file=sys.stderr)
    else:
        print("Child returned", retcode, file=sys.stderr)
except OSError as e:
    print("Execution failed:", e, file=sys.stderr)
```

Replacing the `os.spawn` family

Exemplo P_NOWAIT:

```
pid = os.spawnlp(os.P_NOWAIT, "/bin/mycmd", "mycmd", "myarg")
==>
pid = Popen(["/bin/mycmd", "myarg"]).pid
```

Exemplo P_WAIT:

```
retcode = os.spawnlp(os.P_WAIT, "/bin/mycmd", "mycmd", "myarg")
==>
retcode = call(["/bin/mycmd", "myarg"])
```

Exemplo de vetor:

```
os.spawnvp(os.P_NOWAIT, path, args)
==>
Popen([path] + args[1:])
```

Exemplo de ambiente:

```
os.spawnlpe(os.P_NOWAIT, "/bin/mycmd", "mycmd", "myarg", env)
==>
Popen(["/bin/mycmd", "myarg"], env={"PATH": "/usr/bin"})
```

Replacing `os.popen()`

Return code handling translates as follows:

```
pipe = os.popen(cmd, 'w')
...
rc = pipe.close()
if rc is not None and rc >> 8:
    print("There were some errors")
==>
process = Popen(cmd, stdin=PIPE)
...
process.stdin.close()
if process.wait() != 0:
    print("There were some errors")
```

18.7.7 Legacy Shell Invocation Functions

This module also provides the following legacy functions from the 2.x `commands` module. These operations implicitly invoke the system shell and none of the guarantees described above regarding security and exception handling consistency are valid for these functions.

subprocess.**getstatusoutput**(*cmd*, *, *encoding=None*, *errors=None*)

Return `(exitcode, output)` of executing *cmd* in a shell.

Execute the string *cmd* in a shell with `Popen.check_output()` and return a 2-tuple `(exitcode, output)`. *encoding* and *errors* are used to decode output; see the notes on *Argumentos usados frequentemente* for more details.

A trailing newline is stripped from the output. The exit code for the command can be interpreted as the return code of subprocess. Example:

```
>>> subprocess.getstatusoutput('ls /bin/ls')
(0, '/bin/ls')
>>> subprocess.getstatusoutput('cat /bin/junk')
(1, 'cat: /bin/junk: No such file or directory')
>>> subprocess.getstatusoutput('/bin/junk')
(127, 'sh: /bin/junk: not found')
>>> subprocess.getstatusoutput('/bin/kill $$')
(-15, '')
```

Disponibilidade: Unix, Windows.

Alterado na versão 3.3.4: Suporte ao Windows foi adicionado.

The function now returns (exitcode, output) instead of (status, output) as it did in Python 3.3.3 and earlier. exitcode has the same value as *returncode*.

Alterado na versão 3.11: Adicionados os parâmetros *encoding* e *errors*.

subprocess.**getoutput**(*cmd*, *, *encoding=None*, *errors=None*)

Return output (stdout and stderr) of executing *cmd* in a shell.

Like *getstatusoutput()*, except the exit code is ignored and the return value is a string containing the command's output. Example:

```
>>> subprocess.getoutput('ls /bin/ls')
'/bin/ls'
```

Disponibilidade: Unix, Windows.

Alterado na versão 3.3.4: Suporte para Windows adicionado.

Alterado na versão 3.11: Adicionados os parâmetros *encoding* e *errors*.

18.7.8 Notas

Timeout Behavior

When using the `timeout` parameter in functions like *run()*, *Popen.wait()*, or *Popen.communicate()*, users should be aware of the following behaviors:

1. **Process Creation Delay**: The initial process creation itself cannot be interrupted on many platform APIs. This means that even when specifying a timeout, you are not guaranteed to see a timeout exception until at least after however long process creation takes.
2. **Extremely Small Timeout Values**: Setting very small timeout values (such as a few milliseconds) may result in almost immediate *TimeoutExpired* exceptions because process creation and system scheduling inherently require time.

Converter uma sequência de argumentos em uma string no Windows

On Windows, an *args* sequence is converted to a string that can be parsed using the following rules (which correspond to the rules used by the MS C runtime):

1. Arguments are delimited by white space, which is either a space or a tab.

2. A string surrounded by double quotation marks is interpreted as a single argument, regardless of white space contained within. A quoted string can be embedded in an argument.
3. A double quotation mark preceded by a backslash is interpreted as a literal double quotation mark.
4. Backslashes are interpreted literally, unless they immediately precede a double quotation mark.
5. If backslashes immediately precede a double quotation mark, every pair of backslashes is interpreted as a literal backslash. If the number of backslashes is odd, the last backslash escapes the next double quotation mark as described in rule 3.

Ver também

shlex
Module which provides function to parse and escape command lines.

Disable use of `posix_spawn()`

On Linux, *subprocess* defaults to using the `vfork()` system call internally when it is safe to do so rather than `fork()`. This greatly improves performance.

```
subprocess._USE_POSIX_SPAWN = False  # See CPython issue gh-NNNNNN.
```

It is safe to set this to false on any Python version. It will have no effect on older or newer versions where unsupported. Do not assume the attribute is available to read. Despite the name, a true value does not indicate the corresponding function will be used, only that it may be.

Please file issues any time you have to use these private knobs with a way to reproduce the issue you were seeing. Link to that issue from a comment in your code.

Adicionado na versão 3.8: `_USE_POSIX_SPAWN`

18.8 `sched` — Agendador de eventos

Código-fonte: Lib/sched.py

O módulo *sched* define uma classe que implementa um agendador de eventos de uso geral:

class sched.**scheduler**(*timefunc=time.monotonic*, *delayfunc=time.sleep*)

A classe *scheduler* define uma interface genérica para agendar eventos. Ela precisa de duas funções para lidar com o "mundo externo" — *timefunc* deve ser chamável sem argumentos e retornar um número (o "tempo", em quaisquer unidades). A função *delayfunc* deve ser chamável com um argumento, compatível com a saída de *timefunc*, e deve atrasar a mesma quantidade de unidades de tempo. *delayfunc* também será chamada com o argumento `0` após a execução de cada evento para permitir que outras threads executem em aplicações multithread.

Alterado na versão 3.3: Os parâmetros *timefunc* e *delayfunc* são opcionais.

Alterado na versão 3.3: A classe *scheduler* pode ser usada com segurança em ambientes multithread.

Exemplo:

```
>>> import sched, time
>>> s = sched.scheduler(time.time, time.sleep)
>>> def print_time(a='default'):
...     print("From print_time", time.time(), a)
...
>>> def print_some_times():
...     print(time.time())
```

(continua na próxima página)

(continuação da página anterior)

```
...     s.enter(10, 1, print_time)
...     s.enter(5, 2, print_time, argument=('positional',))
...     # apesar de ter prioridade mais alta, 'keyword' é executado depois de
↪'positional', pois enter() é relativo
...     s.enter(5, 1, print_time, kwargs={'a': 'keyword'})
...     s.enterabs(1_650_000_000, 10, print_time, argument=("first enterabs",))
...     s.enterabs(1_650_000_000, 5, print_time, argument=("second enterabs",))
...     s.run()
...     print(time.time())
...
>>> print_some_times()
1652342830.3640375
From print_time 1652342830.3642538 second enterabs
From print_time 1652342830.3643398 first enterabs
From print_time 1652342835.3694863 positional
From print_time 1652342835.3696074 keyword
From print_time 1652342840.369612 default
1652342840.3697174
```

18.8.1 Objetos Scheduler

As instâncias *scheduler* têm os seguintes métodos e atributos:

scheduler.**enterabs**(*time*, *priority*, *action*, *argument=()*, *kwargs={}*)

Agenda um novo evento. O argumento *time* deve ser um tipo numérico compatível com o valor de retorno da função *timefunc* passada ao construtor. Eventos agendados para o mesmo *time* serão executados na ordem de sua prioridade *priority*. Um número menor representa uma prioridade maior.

Executa o evento significa executar `action(*argument, **kwargs)`. *argument* é uma sequência que contém os argumentos posicionais para *action*. *kwargs* é um dicionário que contém os argumentos nomeados para *action*.

O valor de retorno é um evento que pode ser usado para cancelamento posterior do evento (veja *cancel()*).

Alterado na versão 3.3: o parâmetro *argument* é opcional.

Alterado na versão 3.3: o parâmetro *kwargs* foi adicionado.

scheduler.**enter**(*delay*, *priority*, *action*, *argument=()*, *kwargs={}*)

Agenda um evento para atrasar *delay* mais unidades de tempo. Além do tempo relativo, os outros argumentos, o efeito e o valor de retorno são os mesmos de *enterabs()*.

Alterado na versão 3.3: o parâmetro *argument* é opcional.

Alterado na versão 3.3: o parâmetro *kwargs* foi adicionado.

scheduler.**cancel**(*event*)

Remove o evento da fila. Se *event* não for um evento atualmente na fila, este método levantará uma *ValueError*.

scheduler.**empty**()

Retorna `True` se a fila de eventos estiver vazia.

scheduler.**run**(*blocking=True*)

Executa todos os eventos agendados. Este método aguardará (usando a função *delayfunc* passada ao construtor) o próximo evento, depois o executará e assim por diante até que não haja mais eventos agendados.

Se *blocking* for falso, executa os eventos agendados que expirarão mais cedo (se houver) e então retorna o prazo da próxima chamada agendada no agendador (se houver).

Tanto *action* quanto *delayfunc* podem levantar uma exceção. Em ambos os casos, o agendador manterá um estado consistente e propagará a exceção. Se uma exceção for gerada por *action*, o evento não será tentado em chamadas futuras para `run()`.

Se uma sequência de eventos demorar mais para ser executada do que o tempo disponível antes do próximo evento, o agendador simplesmente ficará para trás. Nenhum evento será descartado; o código de chamada é responsável por cancelar eventos que não são mais pertinentes.

Alterado na versão 3.3: O argumento *blocking* foi adicionado.

`scheduler.`**`queue`**

Atributo somente leitura que retorna uma lista de eventos futuros na ordem em que serão executados. Cada evento é exibido como uma *tupla nomeada* com os seguintes campos: time, priority, action, argument, kwargs.

18.9 `queue` — A synchronized queue class

Código-fonte: Lib/queue.py

The `queue` module implements multi-producer, multi-consumer queues. It is especially useful in threaded programming when information must be exchanged safely between multiple threads. The `Queue` class in this module implements all the required locking semantics.

The module implements three types of queue, which differ only in the order in which the entries are retrieved. In a FIFO queue, the first tasks added are the first retrieved. In a LIFO queue, the most recently added entry is the first retrieved (operating like a stack). With a priority queue, the entries are kept sorted (using the `heapq` module) and the lowest valued entry is retrieved first.

Internally, those three types of queues use locks to temporarily block competing threads; however, they are not designed to handle reentrancy within a thread.

In addition, the module implements a "simple" FIFO queue type, `SimpleQueue`, whose specific implementation provides additional guarantees in exchange for the smaller functionality.

The `queue` module defines the following classes and exceptions:

`class` `queue.`**`Queue`**(*maxsize=0*)

Constructor for a FIFO queue. *maxsize* is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue items are consumed. If *maxsize* is less than or equal to zero, the queue size is infinite.

`class` `queue.`**`LifoQueue`**(*maxsize=0*)

Constructor for a LIFO queue. *maxsize* is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue items are consumed. If *maxsize* is less than or equal to zero, the queue size is infinite.

`class` `queue.`**`PriorityQueue`**(*maxsize=0*)

Constructor for a priority queue. *maxsize* is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue items are consumed. If *maxsize* is less than or equal to zero, the queue size is infinite.

The lowest valued entries are retrieved first (the lowest valued entry is the one that would be returned by `min(entries)`). A typical pattern for entries is a tuple in the form: `(priority_number, data)`.

If the *data* elements are not comparable, the data can be wrapped in a class that ignores the data item and only compares the priority number:

```
from dataclasses import dataclass, field
from typing import Any

@dataclass(order=True)
class PrioritizedItem:
```

(continua na próxima página)

(continuação da página anterior)

```
    priority: int
    item: Any=field(compare=False)
```

class queue.**SimpleQueue**

Constructor for an unbounded FIFO queue. Simple queues lack advanced functionality such as task tracking.

Adicionado na versão 3.7.

exception queue.**Empty**

Exception raised when non-blocking *get()* (or *get_nowait()*) is called on a *Queue* object which is empty.

exception queue.**Full**

Exception raised when non-blocking *put()* (or *put_nowait()*) is called on a *Queue* object which is full.

exception queue.**ShutDown**

Exception raised when *put()* or *get()* is called on a *Queue* object which has been shut down.

Adicionado na versão 3.13.

18.9.1 Objetos Queue

Queue objects (*Queue*, *LifoQueue*, or *PriorityQueue*) provide the public methods described below.

Queue.**qsize**()

Return the approximate size of the queue. Note, qsize() > 0 doesn't guarantee that a subsequent get() will not block, nor will qsize() < maxsize guarantee that put() will not block.

Queue.**empty**()

Return `True` if the queue is empty, `False` otherwise. If empty() returns `True` it doesn't guarantee that a subsequent call to put() will not block. Similarly, if empty() returns `False` it doesn't guarantee that a subsequent call to get() will not block.

Queue.**full**()

Return `True` if the queue is full, `False` otherwise. If full() returns `True` it doesn't guarantee that a subsequent call to get() will not block. Similarly, if full() returns `False` it doesn't guarantee that a subsequent call to put() will not block.

Queue.**put**(*item*, *block=True*, *timeout=None*)

Put *item* into the queue. If optional args *block* is true and *timeout* is `None` (the default), block if necessary until a free slot is available. If *timeout* is a positive number, it blocks at most *timeout* seconds and raises the *Full* exception if no free slot was available within that time. Otherwise (*block* is false), put an item on the queue if a free slot is immediately available, else raise the *Full* exception (*timeout* is ignored in that case).

Raises *ShutDown* if the queue has been shut down.

Queue.**put_nowait**(*item*)

Equivalent to `put(item, block=False)`.

Queue.**get**(*block=True*, *timeout=None*)

Remove and return an item from the queue. If optional args *block* is true and *timeout* is `None` (the default), block if necessary until an item is available. If *timeout* is a positive number, it blocks at most *timeout* seconds and raises the *Empty* exception if no item was available within that time. Otherwise (*block* is false), return an item if one is immediately available, else raise the *Empty* exception (*timeout* is ignored in that case).

Prior to 3.0 on POSIX systems, and for all versions on Windows, if *block* is true and *timeout* is `None`, this operation goes into an uninterruptible wait on an underlying lock. This means that no exceptions can occur, and in particular a SIGINT will not trigger a *KeyboardInterrupt*.

Raises *ShutDown* if the queue has been shut down and is empty, or if the queue has been shut down immediately.

Queue.**get_nowait**()

Equivalente a `get(False)`.

Two methods are offered to support tracking whether enqueued tasks have been fully processed by daemon consumer threads.

Queue.**task_done**()

Indicate that a formerly enqueued task is complete. Used by queue consumer threads. For each *get()* used to fetch a task, a subsequent call to *task_done()* tells the queue that the processing on the task is complete.

If a *join()* is currently blocking, it will resume when all items have been processed (meaning that a *task_done()* call was received for every item that had been *put()* into the queue).

`shutdown(immediate=True)` chama *task_done()* para cada item restante na fila.

Levanta *ValueError* se for chamado mais vezes do que o número de itens colocados na fila.

Queue.**join**()

Blocks until all items in the queue have been gotten and processed.

The count of unfinished tasks goes up whenever an item is added to the queue. The count goes down whenever a consumer thread calls *task_done()* to indicate that the item was retrieved and all work on it is complete. When the count of unfinished tasks drops to zero, *join()* unblocks.

Example of how to wait for enqueued tasks to be completed:

```
import threading
import queue

q = queue.Queue()

def worker():
    while True:
        item = q.get()
        print(f'Working on {item}')
        print(f'Finished {item}')
        q.task_done()

# Turn-on the worker thread.
threading.Thread(target=worker, daemon=True).start()

# Send thirty task requests to the worker.
for item in range(30):
    q.put(item)

# Block until all tasks are done.
q.join()
print('All work completed')
```

Terminating queues

Queue objects can be made to prevent further interaction by shutting them down.

Queue.**shutdown**(*immediate=False*)

Shut down the queue, making *get()* and *put()* raise *ShutDown*.

Por padrão, *get()* em uma fila de desligamento vai levantar quando a fila estiver vazia. Defina *immediate* como verdadeiro para fazer *get()* levantar imediatamente.

Todos os chamadores bloqueados de *put()* e *get()* serão desbloqueados. Se *immediate* for verdadeiro, uma tarefa será marcada como concluída para cada item restante na fila, o que pode desbloquear chamadores de *join()*.

Adicionado na versão 3.13.

18.9.2 Objetos SimpleQueue

SimpleQueue objects provide the public methods described below.

SimpleQueue.**qsize**()

Return the approximate size of the queue. Note, qsize() > 0 doesn't guarantee that a subsequent get() will not block.

SimpleQueue.**empty**()

Return `True` if the queue is empty, `False` otherwise. If empty() returns `False` it doesn't guarantee that a subsequent call to get() will not block.

SimpleQueue.**put**(*item*, *block=True*, *timeout=None*)

Put *item* into the queue. The method never blocks and always succeeds (except for potential low-level errors such as failure to allocate memory). The optional args *block* and *timeout* are ignored and only provided for compatibility with *Queue.put()*.

Detalhes da implementação do CPython: This method has a C implementation which is reentrant. That is, a `put()` or `get()` call can be interrupted by another `put()` call in the same thread without deadlocking or corrupting internal state inside the queue. This makes it appropriate for use in destructors such as `__del__` methods or *weakref* callbacks.

SimpleQueue.**put_nowait**(*item*)

Equivalent to `put(item, block=False)`, provided for compatibility with *Queue.put_nowait()*.

SimpleQueue.**get**(*block=True*, *timeout=None*)

Remove and return an item from the queue. If optional args *block* is true and *timeout* is `None` (the default), block if necessary until an item is available. If *timeout* is a positive number, it blocks at most *timeout* seconds and raises the *Empty* exception if no item was available within that time. Otherwise (*block* is false), return an item if one is immediately available, else raise the *Empty* exception (*timeout* is ignored in that case).

SimpleQueue.**get_nowait**()

Equivalente a `get(False)`.

Ver também

Class *multiprocessing.Queue*
A queue class for use in a multi-processing (rather than multi-threading) context.

collections.deque is an alternative implementation of unbounded queues with fast atomic *append()* and *popleft()* operations that do not require locking and also support indexing.

18.10 `contextvars` — Variáveis de contexto

Este módulo fornece APIs para gerenciar, armazenar e acessar o estado local do contexto. A classe *ContextVar* é usada para declarar e trabalhar com *Variáveis de Contexto*. A função *copy_context()* e a classe *Context* devem ser usadas para gerenciar o contexto atual em frameworks assíncronos.

Os gerenciadores de contexto que possuem estado devem usar Variáveis de Contexto ao invés de *threading.local()* para evitar que seu estado vaze para outro código inesperadamente, quando usado em código concorrente.

Veja também a **PEP 567** para detalhes adicionais.

Adicionado na versão 3.7.

18.10.1 Variáveis de contexto

class contextvars.**ContextVar**(*name*[, *, *default*])

Esta classe é usada para declarar uma nova variável de contexto, como, por exemplo:

```
var: ContextVar[int] = ContextVar('var', default=42)
```

O parâmetro obrigatório *name* é usado para fins de introspecção e depuração.

O parâmetro somente-nomeado opcional *default* é retornado por `ContextVar.get()` quando nenhum valor para a variável é encontrado no contexto atual.

Importante: Variáveis de Contexto devem ser criadas no nível do módulo superior e nunca em fechamentos. Os objetos `Context` contêm referências fortes a variáveis de contexto que evitam que as variáveis de contexto sejam coletadas como lixo corretamente.

name

O nome da variável. Esta é uma propriedade somente leitura.

Adicionado na versão 3.7.1.

get([*default*])

Retorna um valor para a variável de contexto para o contexto atual.

Se não houver valor para a variável no contexto atual, o método vai:

- retornar o valor do argumento *default* do método, se fornecido; ou
- retornar o valor padrão para a variável de contexto, se ela foi criada com uma; ou
- levantar uma `LookupError`.

set(*value*)

Chame para definir um novo valor para a variável de contexto no contexto atual.

O argumento *value* obrigatório é o novo valor para a variável de contexto.

Retorna um objeto `Token` que pode ser usado para restaurar a variável ao seu valor anterior através do método `ContextVar.reset()`.

reset(*token*)

Redefine a variável de contexto para o valor que tinha antes de `ContextVar.set()`. que criou o *token*, ser usado.

Por exemplo:

```
var = ContextVar('var')

token = var.set('novo valor')
# código que usa 'var'; var.get() retorna 'novo valor'.
var.reset(token)

# Após uma chamada de redefinição, var não tem mais valor,
# então var.get() levantaria uma exceção LookupError.
```

class contextvars.**Token**

Objetos *token* são retornados pelo método `ContextVar.set()`. Eles podem ser passados para o método `ContextVar.reset()` para reverter o valor da variável para o que era antes do *set* correspondente.

O token oferece suporte a protocolo do gerenciador de contexto para restaurar o valor da variável de contexto correspondente na saída do bloco `with`:

```
var = ContextVar('var', default='default value')

with var.set('new value'):
    assert var.get() == 'new value'

assert var.get() == 'default value'
```

Adicionado na versão 3.14: Adicionado suporte para uso como gerenciador de contexto.

var

Uma propriedade somente leitura. Aponta para o objeto *ContextVar* que criou o token.

old_value

Uma propriedade somente leitura. Defina como o valor que a variável tinha antes da chamada do método *ContextVar.set()* que criou o token. Aponta para *Token.MISSING* se a variável não foi definida antes da chamada.

MISSING

Um objeto marcador usado por *Token.old_value*.

18.10.2 Gerenciamento de contexto manual

contextvars.**copy_context**()

Retorna uma cópia do objeto *Context* atual.

O trecho a seguir obtém uma cópia do contexto atual e imprime todas as variáveis e seus valores que são definidos nele:

```
ctx: Context = copy_context()
print(list(ctx.items()))
```

A função tem uma complexidade *O*(1) , ou seja, funciona igualmente rápida para contextos com algumas variáveis de contexto e para contextos que têm muitas delas.

class contextvars.**Context**

Um mapeamento de *ContextVars* para seus valores.

`Context()` cria um contexto vazio sem valores nele. Para obter uma cópia do contexto atual, use a função *copy_context()*.

Cada thread tem sua própria pilha efetiva de objetos `Context`. O *contexto atual* é o objeto `Context` no topo da pilha da thread atual. Todos os objetos `Context` nas pilhas são considerados como *inseridos*.

Inserir um contexto, o que pode ser feito chamando seu método *run()*, torna o contexto o contexto atual, colocando-o no topo da pilha de contexto da thread atual.

Sair do contexto atual, o que pode ser feito retornando do retorno de chamada passado para o método *run()*, restaura o contexto atual para o que era antes de o contexto ser inserido, retirando o contexto do topo da pilha de contextos.

Como cada thread tem sua própria pilha de contexto, os objetos *ContextVar* se comportam de maneira semelhante a *threading.local()* quando valores são atribuídos em threads diferentes.

Tentar entrar em um contexto já inserido, incluindo contextos inseridos em outras threads, levanta uma exceção *RuntimeError*.

Após sair de um contexto, ele pode ser acessado novamente (de qualquer thread).

Quaisquer alterações nos valores de *ContextVar* por meio do método *ContextVar.set()* são registradas no contexto atual. O método *ContextVar.get()* retorna o valor associado ao contexto atual. Sair de um contexto efetivamente reverte quaisquer alterações feitas nas variáveis de contexto enquanto o contexto foi inserido (se necessário, os valores podem ser restaurados ao inserir novamente o contexto).

Context implementa a interface *collections.abc.Mapping*.

run(*callable*, **args*, ***kwargs*)

Entra no Context, executa `callable(*args, **kwargs)` e sai do Context. Retorna o valor de retorno de *callable* ou propaga uma exceção, se ocorrer uma.

Exemplo:

```
import contextvars

var = contextvars.ContextVar('var')
var.set('spam')
print(var.get())  # 'spam'

ctx = contextvars.copy_context()

def main():
    # 'var' foi definida para 'spam' antes de
    # chamar 'copy_context()' e 'ctx.run(main)', então:
    print(var.get())  # 'spam'
    print(ctx[var])  # 'spam'

    var.set('ham')

    # Agora, após definir 'var' para 'ham':
    print(var.get())  # 'ham'
    print(ctx[var])  # 'ham'

# Qualquer alteração que a função 'main' feitas a 'var'
# serão contidos em 'ctx'.
ctx.run(main)

# A função 'main()' era executada no contexto 'ctx',
# então alterações a 'var' são contidas nele:
print(ctx[var])  # 'ham'

# No entanto, fora de 'ctx', 'var' ainda está definida para 'spam':
print(var.get())  # 'spam'
```

copy()

Retorna uma cópia rasa do objeto contexto.

var in context

Retorna `True` se *context* tem uma variável para *var* definida; do contrário, retorna `False`.

context[var]

Retorna o valor da variável *ContextVar* *var*. Se a variável não for definida no objeto contexto, uma *KeyError* é levantada.

get(*var*[, *default*])

Retorna o valor para *var* se *var* tiver o valor no objeto contexto. Caso contrário, retorna *default*. Se *default* não for fornecido, retorna `None`.

iter(context)

Retorna um iterador sobre as variáveis armazenadas no objeto contexto.

len(proxy)

Retorna o número das variáveis definidas no objeto contexto.

keys()

Retorna uma lista de todas as variáveis no objeto contexto.

values()

Retorna uma lista dos valores de todas as variáveis no objeto contexto.

items()

Retorna uma lista de tuplas de 2 elementos contendo todas as variáveis e seus valores no objeto contexto.

18.10.3 Suporte a asyncio

Variáveis de contexto encontram suporte nativo em *asyncio* e estão prontas para serem usadas sem qualquer configuração extra. Por exemplo, aqui está um servidor simples de eco, que usa uma variável de contexto para disponibilizar o endereço de um cliente remoto na Task que lida com esse cliente:

```
import asyncio
import contextvars

client_addr_var = contextvars.ContextVar('client_addr')

def render_goodbye():
    # O endereço do cliente atualmente manipulado pode ser acessado
    # sem passá-lo explicitamente para esta função.

    client_addr = client_addr_var.get()
    return f'Good bye, client @ {client_addr}\r\n'.encode()

async def handle_request(reader, writer):
    addr = writer.transport.get_extra_info('socket').getpeername()
    client_addr_var.set(addr)

    # Em qualquer código que chamamos agora é possível obter
    # o endereço do cliente chamando 'client_addr_var.get()'.
    while True:
        line = await reader.readline()
        print(line)
        if not line.strip():
            break

    writer.write(b'HTTP/1.1 200 OK\r\n')  # linha de status
    writer.write(b'\r\n')  # cabeçalhos
    writer.write(render_goodbye())  # corpo
    writer.close()

async def main():
    srv = await asyncio.start_server(
        handle_request, '127.0.0.1', 8081)

    async with srv:
        await srv.serve_forever()

asyncio.run(main())

# Para testá-lo, você pode usar telnet ou curl:
#     telnet 127.0.0.1 8081
#     curl 127.0.0.1:8081
```

A seguir, os módulos de suporte para alguns dos serviços acima:

18.11 _thread— API de segmentação de baixo nível

Este módulo fornece primitivos de baixo nível para trabalhar com vários threads (também chamados *processos leves* ou *tarefas*) — vários threads de controle compartilhando seu espaço de dados global. Para sincronização, travas simples (também chamadas de *mutexes*, *exclusão mútua* ou *semáforos binários*) são fornecidas. O módulo `threading` fornece uma API de segmentação mais fácil de usar e de nível mais alto, construída sobre este módulo.

Alterado na versão 3.7: Este módulo costumava ser opcional, agora está sempre disponível.

Este módulo define as seguintes constantes e funções:

exception _thread.**error**

Gerado em erros específicos de segmento.

Alterado na versão 3.3: Agora este é um sinônimo da exceção embutida `RuntimeError`.

_thread.**LockType**

Este é o tipo de objetos de trava.

_thread.**start_new_thread**(*function*, *args*[, *kwargs*])

Começa uma nova thread e retorna seu identificador. A thread executa a função *function* com a lista de argumentos *args* (que deve ser uma tupla). O argumento opcional *kwargs* especifica um dicionário de argumentos nomeados.

Quando a função retorna, a thread termina silenciosamente.

Quando a função termina com uma exceção não processada, `sys.unraisablehook()` é chamada para lidar com a exceção. O atributo *object* do argumento do hook é *function*. Por padrão, um stack trace (situação da pilha de execução) é exibido e, em seguida, a thread termina (mas outras threads continuam a ser executadas).

Quando a função gera uma exceção `SystemExit`, ela é ignorada.

Levanta um *evento de auditoria* `_thread.start_new_thread` com os argumentos `function`, `args`, `kwargs`.

Alterado na versão 3.8: `sys.unraisablehook()` agora é usada para tratar exceções não tratadas.

_thread.**interrupt_main**(*signum=signal.SIGINT*, */*)

Simula o efeito de um sinal chegando na thread principal. Uma thread pode usar esta função para interromper a thread principal, embora não haja garantia de que a interrupção ocorrerá imediatamente.

Se fornecido, *signum* é o número do sinal a ser simulado. Se *signum* não for fornecido, `signal.SIGINT` será simulado.

Se o sinal fornecido não for tratado pelo Python (o sinal foi definido como `signal.SIG_DFL` ou `signal.SIG_IGN`), esta função faz nada.

Alterado na versão 3.10: O argumento *signum* é adicionado para personalizar o sinal de número.

> **Nota**
>
> Isso não emite o sinal correspondente, mas agenda uma chamada para o tratador associado (se existir). Se você quer realmente emitir o sinal, use `signal.raise_signal()`.

_thread.**exit**()

Levanta a exceção `SystemExit`. Quando não for detectada, a thread terminará silenciosamente.

_thread.**allocate_lock**()

Retorna um novo objeto de trava. Métodos de trava são descritos abaixo. A trava é desativada inicialmente.

_thread.**get_ident**()

Retorna o 'identificador de thread' da thread atual. Este é um número inteiro diferente de zero. Seu valor não tem significado direto; pretende-se que seja um cookie mágico para ser usado, por exemplo, para indexar um dicionário de dados específicos do thread. Identificadores de thread podem ser reciclados quando uma thread termina e outra é criada.

_thread.**get_native_id**()

Retorna a ID de thread integral nativa da thread atual atribuída pelo kernel. Este é um número inteiro não negativo. Seu valor pode ser usado para identificar exclusivamente essa thread específica em todo o sistema (até que a thread termine, após o que o valor poderá ser reciclado pelo sistema operacional).

Disponibilidade: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX, DragonFlyBSD, GNU/kFreeBSD.

Adicionado na versão 3.8.

Alterado na versão 3.13: Adicionado suporte a GNU/kFreeBSD.

_thread.**stack_size**([*size*])

Retorna o tamanho da pilha de threads usado ao criar novas threads. O argumento opcional *size* especifica o tamanho da pilha a ser usado para threads criadas posteriormente e deve ser 0 (usar plataforma ou padrão configurado) ou um valor inteiro positivo de pelo menos 32.768 (32 KiB). Se *size* não for especificado, 0 será usado. Se a alteração do tamanho da pilha de threads não for permitida, uma *RuntimeError* será levantada. Se o tamanho da pilha especificado for inválido, um *ValueError* será levantado e o tamanho da pilha não será modificado. Atualmente, 0 KiB é o valor mínimo de tamanho de pilha suportado para garantir espaço suficiente para o próprio interpretador. Observe que algumas plataformas podem ter restrições específicas sobre valores para o tamanho da pilha, como exigir um tamanho mínimo de pilha > 32 KiB ou exigir alocação em múltiplos do tamanho da página de memória do sistema – a documentação da plataforma deve ser consultada para obter mais informações (4 páginas KiB são comuns; usar múltiplos de 4096 para o tamanho da pilha é a abordagem sugerida na ausência de informações mais específicas).

Disponibilidade: Windows, pthreads.

Plataformas Unix com suporte a threads POSIX.

_thread.**TIMEOUT_MAX**

O valor máximo permitido para o parâmetro *timeout* de *Lock.acquire*. A especificação de um tempo limite maior que esse valor vai levantar um *OverflowError*.

Adicionado na versão 3.2.

Os objetos de trava têm os seguintes métodos:

lock.**acquire**(*blocking=True*, *timeout=-1*)

Sem nenhum argumento opcional, esse método adquire a trava incondicionalmente, se necessário, aguardando até que seja liberada por outra thread (apenas uma thread por vez pode adquirir uma trava — esse é o motivo da sua existência).

Se o argumento inteiro *blocking* estiver presente, a ação dependerá do seu valor: se for falso, a trava será adquirida apenas se puder ser adquirida imediatamente sem aguardar, enquanto se for verdadeiro, a trava será adquirida incondicionalmente, conforme acima.

Se o argumento de ponto flutuante *timeout* estiver presente e positivo, ele especificará o tempo máximo de espera em segundos antes de retornar. Um argumento negativo *timeout* especifica uma espera ilimitada. Você não pode especificar um *timeout* se *blocking* for falso.

O valor de retorno é `True` se a trava for adquirida com sucesso, se não `False`.

Alterado na versão 3.2: O parâmetro *timeout* é novo.

Alterado na versão 3.2: As aquisições de trava agora podem ser interrompidas por sinais no POSIX.

Alterado na versão 3.14: As aquisições de trava agora podem ser interrompidas por sinais no Windows.

lock.**release**()

Libera a trava. A trava deve ter sido adquirido anteriormente, mas não necessariamente pela mesma thread.

lock.**locked**()

Retorna o status da trava: `True` se tiver sido adquirida por alguma thread, `False` se não for o caso.

Além desses métodos, os objetos de trava também podem ser usados através da instrução `with`, por exemplo:

```
import _thread

uma_trava = _thread.allocate_lock()

with uma_trava:
    print("uma_trava está travada enquanto isto executa")
```

Ressalvas:

- Interrupções sempre vão para a thread principal (a exceção *KeyboardInterrupt* será recebida por essa thread).
- Chamar *sys.exit()* ou levantar a exceção *SystemExit* é o equivalente a chamar *_thread.exit()*.
- Quando a thread principal se encerra, o fato de outras threads sobreviverem depende do sistema. Na maioria dos sistemas, elas são eliminadas sem executar cláusulas `try` … `finally` ou destruidores de objetos.

CAPÍTULO 19

Comunicação em Rede e Interprocesso

Os módulos descritos neste capítulo fornecem mecanismos para a comunicação em rede e entre processos.

Alguns módulos funcionam apenas para dois processos que estão na mesma máquina como, por exemplo, *signal* e *mmap*. Outros módulos possuem suporte a protocolos de rede que dois ou mais processos podem usar para se comunicar entre máquinas.

A lista de módulos descritos neste capítulo é:

19.1 `asyncio` — E/S assíncrona

Olá Mundo!

```
import asyncio

async def main():
    print('Olá ...')
    await asyncio.sleep(1)
    print('... Mundo!')

asyncio.run(main())
```

asyncio é uma biblioteca para escrever código **simultâneo** usando a sintaxe **async/await**.

O asyncio é usado como uma base para várias estruturas assíncronas do Python que fornecem rede e servidores web de alto desempenho, bibliotecas de conexão de banco de dados, filas de tarefas distribuídas etc.

asyncio geralmente serve perfeitamente para código de rede **estruturado** de alto nível e vinculado a E/S.

Ver também

a-conceptual-overview-of-asyncio
Explicação dos fundamentos do asyncio.

asyncio fornece um conjunto de APIs de **alto nível** para:

- *executar corrotinas do Python* simultaneamente e ter controle total sobre sua execução;
- realizar *IPC e E/S de rede*;
- controlar *subprocessos*;
- distribuir tarefas por meio de *filas*;
- *sincronizar* código simultâneo;

Além disso, há APIs de **baixo nível** para *desenvolvedores de biblioteca e framework* para:

- criar e gerenciar *laços de eventos*, que fornecem APIs assíncronas para *rede*, execução de *subprocessos*, tratamento de *sinais de sistemas operacionais* etc;
- implementar protocolos eficientes usando *transportes*;
- *fazer uma ponte* sobre bibliotecas baseadas em chamadas e codificar com a sintaxe de async/await.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

REPL de asyncio

Você pode experimentar um contexto atual `asyncio` no *REPL*:

```
$ python -m asyncio
asyncio REPL ...
Use "await" directly instead of "asyncio.run()".
Type "help", "copyright", "credits" or "license" for more information.
>>> import asyncio
>>> await asyncio.sleep(10, result='hello')
'hello'
```

Levanta um *evento de auditoria* `cpython.run_stdin` sem argumentos.

Alterado na versão 3.12.5: (também 3.11.10, 3.10.15, 3.9.20 e 3.8.20) Emite eventos de auditoria.

Alterado na versão 3.13: Usa PyrePL, se possível, nesse caso `PYTHONSTARTUP` também é executado. Emite eventos de auditoria.

Referência

19.1.1 Runners

Source code: Lib/asyncio/runners.py

This section outlines high-level asyncio primitives to run asyncio code.

They are built on top of an *event loop* with the aim to simplify async code usage for common wide-spread scenarios.

- *Executando um programa asyncio*
- *Runner context manager*
- *Handling Keyboard Interruption*

Executando um programa asyncio

asyncio.**run**(*coro*, *, *debug=None*, *loop_factory=None*)

Execute *coro* in an asyncio event loop and return the result.

The argument can be any awaitable object.

This function runs the awaitable, taking care of managing the asyncio event loop, *finalizing asynchronous generators*, and closing the executor.

Esta função não pode ser chamada quando outro laço de eventos asyncio está executando na mesma thread.

If *debug* is `True`, the event loop will be run in debug mode. `False` disables debug mode explicitly. `None` is used to respect the global *Modo de Depuração* settings.

If *loop_factory* is not `None`, it is used to create a new event loop; otherwise *asyncio.new_event_loop()* is used. The loop is closed at the end. This function should be used as a main entry point for asyncio programs, and should ideally only be called once. It is recommended to use *loop_factory* to configure the event loop instead of policies. Passing *asyncio.EventLoop* allows running asyncio without the policy system.

The executor is given a timeout duration of 5 minutes to shutdown. If the executor hasn't finished within that duration, a warning is emitted and the executor is closed.

Exemplo:

```
async def main():
    await asyncio.sleep(1)
    print('hello')

asyncio.run(main())
```

Adicionado na versão 3.7.

Alterado na versão 3.9: Atualizado para usar *loop.shutdown_default_executor()*.

Alterado na versão 3.10: *debug* is `None` by default to respect the global debug mode settings.

Alterado na versão 3.12: Added *loop_factory* parameter.

Alterado na versão 3.14: *coro* can be any awaitable object.

Nota

The `asyncio` policy system is deprecated and will be removed in Python 3.16; from there on, an explicit *loop_factory* is needed to configure the event loop.

Runner context manager

class asyncio.**Runner**(*, *debug=None*, *loop_factory=None*)

A context manager that simplifies *multiple* async function calls in the same context.

Sometimes several top-level async functions should be called in the same *event loop* and *contextvars.Context*.

If *debug* is `True`, the event loop will be run in debug mode. `False` disables debug mode explicitly. `None` is used to respect the global *Modo de Depuração* settings.

loop_factory could be used for overriding the loop creation. It is the responsibility of the *loop_factory* to set the created loop as the current one. By default *asyncio.new_event_loop()* is used and set as current event loop with *asyncio.set_event_loop()* if *loop_factory* is `None`.

Basically, *asyncio.run()* example can be rewritten with the runner usage:

```
async def main():
    await asyncio.sleep(1)
    print('hello')
```

(continua na próxima página)

(continuação da página anterior)

```
with asyncio.Runner() as runner:
    runner.run(main())
```

Adicionado na versão 3.11.

run(*coro*, *, *context=None*)

Execute *coro* in the embedded event loop.

The argument can be any awaitable object.

If the argument is a coroutine, it is wrapped in a Task.

An optional keyword-only *context* argument allows specifying a custom *contextvars.Context* for the code to run in. The runner's default context is used if context is `None`.

Returns the awaitable's result or raises an exception.

Esta função não pode ser chamada quando outro laço de eventos asyncio está executando na mesma thread.

Alterado na versão 3.14: *coro* can be any awaitable object.

close()

Close the runner.

Finalize asynchronous generators, shutdown default executor, close the event loop and release embedded *contextvars.Context*.

get_loop()

Return the event loop associated with the runner instance.

Nota

Runner uses the lazy initialization strategy, its constructor doesn't initialize underlying low-level structures.

Embedded *loop* and *context* are created at the `with` body entering or the first call of *run()* or *get_loop()*.

Handling Keyboard Interruption

Adicionado na versão 3.11.

When *signal.SIGINT* is raised by `Ctrl-C`, *KeyboardInterrupt* exception is raised in the main thread by default. However this doesn't work with *asyncio* because it can interrupt asyncio internals and can hang the program from exiting.

To mitigate this issue, *asyncio* handles *signal.SIGINT* as follows:

1. *asyncio.Runner.run()* installs a custom *signal.SIGINT* handler before any user code is executed and removes it when exiting from the function.
2. The *Runner* creates the main task for the passed coroutine for its execution.
3. When *signal.SIGINT* is raised by `Ctrl-C`, the custom signal handler cancels the main task by calling *asyncio.Task.cancel()* which raises *asyncio.CancelledError* inside the main task. This causes the Python stack to unwind, `try/except` and `try/finally` blocks can be used for resource cleanup. After the main task is cancelled, *asyncio.Runner.run()* raises *KeyboardInterrupt*.
4. A user could write a tight loop which cannot be interrupted by *asyncio.Task.cancel()*, in which case the second following `Ctrl-C` immediately raises the *KeyboardInterrupt* without cancelling the main task.

19.1.2 Corrotinas e Tarefas

Esta seção descreve APIs assíncronas de alto nível para trabalhar com corrotinas e tarefas.

- *Corrotinas*
- *Aguardáveis*
- *Criando Tarefas*
- *Task Cancellation*
- *Task Groups*
- *Dormindo*
- *Executando tarefas concorrentemente*
- *Eager Task Factory*
- *Protegendo contra cancelamento*
- *Tempo limite*
- *Primitivas de Espera*
- *Executando em Threads*
- *Agendando a partir de outras Threads*
- *Introspecção*
- *Objeto Task*

Corrotinas

Source code: Lib/asyncio/coroutines.py

Corrotinas declaradas com a sintaxe async/await é a forma preferida de escrever aplicações assíncronas. Por exemplo, o seguinte trecho de código imprime "hello", espera 1 segundo, e então imprime "world":

```
>>> import asyncio

>>> async def main():
...     print('hello')
...     await asyncio.sleep(1)
...     print('world')

>>> asyncio.run(main())
hello
world
```

Perceba que simplesmente chamar uma corrotina não irá agendá-la para ser executada:

```
>>> main()
<coroutine object main at 0x1053bb7c8>
```

To actually run a coroutine, asyncio provides the following mechanisms:

- A função *asyncio.run()* para executar a função "main()" do ponto de entrada no nível mais alto (veja o exemplo acima.)
- Aguardando uma corrotina. O seguinte trecho de código exibirá "hello" após esperar por 1 segundo e, em seguida, exibirá "world" após esperar por *outros* 2 segundos:

```
import asyncio
import time

async def say_after(delay, what):
    await asyncio.sleep(delay)
    print(what)

async def main():
    print(f"started at {time.strftime('%X')}")

    await say_after(1, 'hello')
    await say_after(2, 'world')

    print(f"finished at {time.strftime('%X')}")

asyncio.run(main())
```

Resultado esperado:

```
started at 17:13:52
hello
world
finished at 17:13:55
```

- A função *asyncio.create_task()* para executar corrotinas concorrentemente como *Tasks* asyncio.

  Vamos modificar o exemplo acima e executar duas corrotinas `say_after` *concorrentemente*:

```
async def main():
    task1 = asyncio.create_task(
        say_after(1, 'hello'))

    task2 = asyncio.create_task(
        say_after(2, 'world'))

    print(f"started at {time.strftime('%X')}")

    # Wait until both tasks are completed (should take
    # around 2 seconds.)
    await task1
    await task2

    print(f"finished at {time.strftime('%X')}")
```

  Perceba que a saída esperada agora mostra que o trecho de código é executado 1 segundo mais rápido do que antes:

```
started at 17:14:32
hello
world
finished at 17:14:34
```

- The *asyncio.TaskGroup* class provides a more modern alternative to *create_task()*. Using this API, the last example becomes:

```
async def main():
    async with asyncio.TaskGroup() as tg:
        task1 = tg.create_task(
```

(continua na próxima página)

(continuação da página anterior)

```
            say_after(1, 'hello'))

        task2 = tg.create_task(
            say_after(2, 'world'))

        print(f"started at {time.strftime('%X')}")

    # The await is implicit when the context manager exits.

    print(f"finished at {time.strftime('%X')}")
```

The timing and output should be the same as for the previous version.

Adicionado na versão 3.11: *asyncio.TaskGroup*.

Aguardáveis

Dizemos que um objeto é um objeto **aguardável** se ele pode ser usado em uma expressão `await`. Muitas APIs asyncio são projetadas para aceitar aguardáveis.

Existem três tipos principais de objetos *aguardáveis*: **corrotinas**, **Tarefas**, e **Futuros**.

Corrotinas

Corrotinas Python são *aguardáveis* e portanto podem ser aguardadas a partir de outras corrotinas:

```
import asyncio

async def nested():
    return 42

async def main():
    # Nothing happens if we just call "nested()".
    # A coroutine object is created but not awaited,
    # so it *won't run at all*.
    nested()  # will raise a "RuntimeWarning".

    # Let's do it differently now and await it:
    print(await nested())  # will print "42".

asyncio.run(main())
```

Importante

Nesta documentação, o termo "corrotina" pode ser usado para dois conceitos intimamente relacionados:

- uma *função de corrotina*: uma função `async def`;
- um *objeto de corrotina*: um objeto retornado ao chamar uma *função de corrotina*.

Tarefas

Tarefas são usadas para agendar corrotinas *concorrentemente*.

Quando uma corrotina é envolta em uma *tarefa* com funções como *asyncio.create_task()*, a corrotina é automaticamente agendada para executar em breve:

```
import asyncio

async def nested():
    return 42

async def main():
    # Schedule nested() to run soon concurrently
    # with "main()".
    task = asyncio.create_task(nested())

    # "task" can now be used to cancel "nested()", or
    # can simply be awaited to wait until it is complete:
    await task

asyncio.run(main())
```

Futuros

Um *Future* é um objeto aguardável especial de **baixo nível** que representa um **resultado eventual** de uma operação assíncrona.

Quando um objeto Future é *aguardado* isso significa que a corrotina irá esperar até que o Future seja resolvido em algum outro local.

Objetos Future em asyncio são necessários para permitir que código baseado em função de retorno seja utilizado com async/await.

Normalmente **não existe necessidade** em criar objetos Future no nível de código da aplicação.

Objetos Future, algumas vezes expostos por bibliotecas e algumas APIs asyncio, podem ser aguardados:

```
async def main():
    await function_that_returns_a_future_object()

    # this is also valid:
    await asyncio.gather(
        function_that_returns_a_future_object(),
        some_python_coroutine()
    )
```

Um bom exemplo de uma função de baixo nível que retorna um objeto Future é *loop.run_in_executor()*.

Criando Tarefas

Source code: Lib/asyncio/tasks.py

---

asyncio.**create_task**(*coro*, *, *name=None*, *context=None*, *eager_start=None*, ***kwargs*)

Envolva a *corrotina* *coro* em uma *Task* e agende sua execução. Retorne o objeto Task.

The full function signature is largely the same as that of the *Task* constructor (or factory) - all of the keyword arguments to this function are passed through to that interface.

An optional keyword-only *context* argument allows specifying a custom *contextvars.Context* for the *coro* to run in. The current context copy is created when no *context* is provided.

An optional keyword-only *eager_start* argument allows specifying if the task should execute eagerly during the call to create_task, or be scheduled later. If *eager_start* is not passed the mode set by *loop.set_task_factory()* will be used.

A tarefa é executada no laço e retornada por *get_running_loop()*, *RuntimeError* é levantado se não existir nenhum loop na thread atual.

Nota

asyncio.TaskGroup.create_task() is a new alternative leveraging structural concurrency; it allows for waiting for a group of related tasks with strong safety guarantees.

Importante

Mantenha uma referência para o resultado dessa função, evitando assim que uma tarefa desapareça durante a execução. O laço de eventos mantém apenas referências fracas para as tarefas. Uma tarefa que não é referenciada por nada mais pode ser removida pelo coletor de lixo a qualquer momento, antes mesmo da função ser finalizada. Para tarefas de segundo plano "atire-e-esqueça", junte-as em uma coleção:

```
background_tasks = set()

for i in range(10):
    task = asyncio.create_task(some_coro(param=i))

    # Add task to the set. This creates a strong reference.
    background_tasks.add(task)

    # To prevent keeping references to finished tasks forever,
    # make each task remove its own reference from the set after
    # completion:
    task.add_done_callback(background_tasks.discard)
```

Adicionado na versão 3.7.

Alterado na versão 3.8: Adicionado o parâmetro *name*.

Alterado na versão 3.11: Adicionado o parâmetro *context*.

Alterado na versão 3.14: Added the *eager_start* parameter by passing on all *kwargs*.

Task Cancellation

Tasks can easily and safely be cancelled. When a task is cancelled, *asyncio.CancelledError* will be raised in the task at the next opportunity.

It is recommended that coroutines use `try/finally` blocks to robustly perform clean-up logic. In case *asyncio.CancelledError* is explicitly caught, it should generally be propagated when clean-up is complete. *asyncio.CancelledError* directly subclasses *BaseException* so most code will not need to be aware of it.

The asyncio components that enable structured concurrency, like *asyncio.TaskGroup* and *asyncio.timeout()*, are implemented using cancellation internally and might misbehave if a coroutine swallows *asyncio.CancelledError*. Similarly, user code should not generally call *uncancel*. However, in cases when suppressing *asyncio.CancelledError* is truly desired, it is necessary to also call `uncancel()` to completely remove the cancellation state.

Task Groups

Task groups combine a task creation API with a convenient and reliable way to wait for all tasks in the group to finish.

class asyncio.**TaskGroup**

An asynchronous context manager holding a group of tasks. Tasks can be added to the group using *create_task()*. All tasks are awaited when the context manager exits.

Adicionado na versão 3.11.

create_task(*coro*, *, *name=None*, *context=None*, *eager_start=None*, ***kwargs*)

Create a task in this task group. The signature matches that of *asyncio.create_task()*. If the task group is inactive (e.g. not yet entered, already finished, or in the process of shutting down), we will close the given `coro`.

Alterado na versão 3.13: Close the given coroutine if the task group is not active.

Alterado na versão 3.14: Passes on all *kwargs* to *loop.create_task()*

Exemplo:

```
async def main():
    async with asyncio.TaskGroup() as tg:
        task1 = tg.create_task(some_coro(...))
        task2 = tg.create_task(another_coro(...))
    print(f"Both tasks have completed now: {task1.result()}, {task2.result()}")
```

The `async with` statement will wait for all tasks in the group to finish. While waiting, new tasks may still be added to the group (for example, by passing `tg` into one of the coroutines and calling `tg.create_task()` in that coroutine). Once the last task has finished and the `async with` block is exited, no new tasks may be added to the group.

The first time any of the tasks belonging to the group fails with an exception other than *asyncio.CancelledError*, the remaining tasks in the group are cancelled. No further tasks can then be added to the group. At this point, if the body of the `async with` statement is still active (i.e., `__aexit__()` hasn't been called yet), the task directly containing the `async with` statement is also cancelled. The resulting *asyncio.CancelledError* will interrupt an `await`, but it will not bubble out of the containing `async with` statement.

Once all tasks have finished, if any tasks have failed with an exception other than *asyncio.CancelledError*, those exceptions are combined in an *ExceptionGroup* or *BaseExceptionGroup* (as appropriate; see their documentation) which is then raised.

Two base exceptions are treated specially: If any task fails with *KeyboardInterrupt* or *SystemExit*, the task group still cancels the remaining tasks and waits for them, but then the initial *KeyboardInterrupt* or *SystemExit* is re-raised instead of *ExceptionGroup* or *BaseExceptionGroup*.

If the body of the `async with` statement exits with an exception (so `__aexit__()` is called with an exception set), this is treated the same as if one of the tasks failed: the remaining tasks are cancelled and then waited for, and non-cancellation exceptions are grouped into an exception group and raised. The exception passed into `__aexit__()`, unless it is *asyncio.CancelledError*, is also included in the exception group. The same special case is made for *KeyboardInterrupt* and *SystemExit* as in the previous paragraph.

Task groups are careful not to mix up the internal cancellation used to "wake up" their `__aexit__()` with cancellation requests for the task in which they are running made by other parties. In particular, when one task group is syntactically nested in another, and both experience an exception in one of their child tasks simultaneously, the inner task group will process its exceptions, and then the outer task group will receive another cancellation and process its own exceptions.

In the case where a task group is cancelled externally and also must raise an *ExceptionGroup*, it will call the parent task's *cancel()* method. This ensures that a *asyncio.CancelledError* will be raised at the next `await`, so the cancellation is not lost.

Task groups preserve the cancellation count reported by *asyncio.Task.cancelling()*.

Alterado na versão 3.13: Improved handling of simultaneous internal and external cancellations and correct preservation of cancellation counts.

Terminating a Task Group

While terminating a task group is not natively supported by the standard library, termination can be achieved by adding an exception-raising task to the task group and ignoring the raised exception:

```
import asyncio
from asyncio import TaskGroup

class TerminateTaskGroup(Exception):
    """Exception raised to terminate a task group."""

async def force_terminate_task_group():
    """Used to force termination of a task group."""
    raise TerminateTaskGroup()

async def job(task_id, sleep_time):
    print(f'Task {task_id}: start')
    await asyncio.sleep(sleep_time)
    print(f'Task {task_id}: done')

async def main():
    try:
        async with TaskGroup() as group:
            # spawn some tasks
            group.create_task(job(1, 0.5))
            group.create_task(job(2, 1.5))
            # sleep for 1 second
            await asyncio.sleep(1)
            # add an exception-raising task to force the group to terminate
            group.create_task(force_terminate_task_group())
    except* TerminateTaskGroup:
        pass

asyncio.run(main())
```

Expected output:

```
Task 1: start
Task 2: start
Task 1: done
```

Dormindo

async asyncio.**sleep**(*delay*, *result=None*)

Bloqueia por *delay* segundos.

Se *result* é fornecido, é retornado para o autor da chamada quando a corrotina termina.

`sleep()` sempre suspende a tarefa atual, permitindo que outras tarefas sejam executadas.

Configurando o delay para 0 fornece um caminho otimizado para permitir que outras tarefas executem. Isto pode ser usado por funções de longa execução para evitar que bloqueiem o laço de eventos por toda a duração da chamada da função.

Exemplo de uma corrotina exibindo a data atual a cada segundo durante 5 segundos:

```
import asyncio
import datetime

async def display_date():
    loop = asyncio.get_running_loop()
    end_time = loop.time() + 5.0
    while True:
        print(datetime.datetime.now())
```

(continua na próxima página)

(continuação da página anterior)

```
        if (loop.time() + 1.0) >= end_time:
            break
        await asyncio.sleep(1)

asyncio.run(display_date())
```

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.13: Raises `ValueError` if *delay* is `nan`.

Executando tarefas concorrentemente

awaitable asyncio.**gather**(**aws*, *return_exceptions=False*)

Executa *objetos aguardáveis* na sequência *aws* de forma *concorrente*.

Se qualquer aguardável em *aws* é uma corrotina, ele é automaticamente agendado como uma Tarefa.

Se todos os aguardáveis forem concluídos com sucesso, o resultado é uma lista agregada de valores retornados. A ordem dos valores resultantes corresponde a ordem dos aguardáveis em *aws*.

Se *return_exceptions* for `False` (valor padrão), a primeira exceção levantada é imediatamente propagada para a tarefa que espera em `gather()`. Outros aguardáveis na sequência *aws* **não serão cancelados** e irão continuar a executar.

Se *return_exceptions* for `True`, exceções são tratadas da mesma forma que resultados com sucesso, e agregadas na lista de resultados.

Se `gather()` for *cancelado*, todos os aguardáveis que foram submetidos (que não foram concluídos ainda) também são *cancelados*.

Se qualquer Tarefa ou Futuro da sequência *aws* for *cancelado*, ele é tratado como se tivesse levantado `CancelledError` – a chamada para `gather()` **não** é cancelada neste caso. Isso existe para prevenir que o cancelamento de uma Tarefa/Futuro submetida ocasione outras Tarefas/Futuros a serem cancelados.

> **Nota**
>
> A new alternative to create and run tasks concurrently and wait for their completion is `asyncio.TaskGroup`. *TaskGroup* provides stronger safety guarantees than *gather* for scheduling a nesting of subtasks: if a task (or a subtask, a task scheduled by a task) raises an exception, *TaskGroup* will, while *gather* will not, cancel the remaining scheduled tasks).

Exemplo:

```
import asyncio

async def factorial(name, number):
    f = 1
    for i in range(2, number + 1):
        print(f"Task {name}: Compute factorial({number}), currently i={i}...")
        await asyncio.sleep(1)
        f *= i
    print(f"Task {name}: factorial({number}) = {f}")
    return f

async def main():
    # Schedule three calls *concurrently*:
    L = await asyncio.gather(
        factorial("A", 2),
        factorial("B", 3),
```

(continua na próxima página)

(continuação da página anterior)

```
        factorial("C", 4),
    )
    print(L)

asyncio.run(main())

# Expected output:
#
#     Task A: Compute factorial(2), currently i=2...
#     Task B: Compute factorial(3), currently i=2...
#     Task C: Compute factorial(4), currently i=2...
#     Task A: factorial(2) = 2
#     Task B: Compute factorial(3), currently i=3...
#     Task C: Compute factorial(4), currently i=3...
#     Task B: factorial(3) = 6
#     Task C: Compute factorial(4), currently i=4...
#     Task C: factorial(4) = 24
#     [2, 6, 24]
```

Nota

If *return_exceptions* is false, cancelling gather() after it has been marked done won't cancel any submitted awaitables. For instance, gather can be marked done after propagating an exception to the caller, therefore, calling `gather.cancel()` after catching an exception (raised by one of the awaitables) from gather won't cancel any other awaitables.

Alterado na versão 3.7: Se *gather* por si mesmo for cancelado, o cancelamento é propagado independente de *return_exceptions*.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se nenhum argumento posicional for fornecido, ou nem todos os argumentos posicionais são objetos similar a Futuro, e não existe nenhum laço de eventos em execução.

Eager Task Factory

`asyncio.`**`eager_task_factory`**(*loop*, *coro*, *, *name=None*, *context=None*)

A task factory for eager task execution.

When using this factory (via `loop.set_task_factory(asyncio.eager_task_factory)`), coroutines begin execution synchronously during `Task` construction. Tasks are only scheduled on the event loop if they block. This can be a performance improvement as the overhead of loop scheduling is avoided for coroutines that complete synchronously.

A common example where this is beneficial is coroutines which employ caching or memoization to avoid actual I/O when possible.

Nota

Immediate execution of the coroutine is a semantic change. If the coroutine returns or raises, the task is never scheduled to the event loop. If the coroutine execution blocks, the task is scheduled to the event loop. This change may introduce behavior changes to existing applications. For example, the application's task execution order is likely to change.

Adicionado na versão 3.12.

asyncio.**create_eager_task_factory**(*custom_task_constructor*)

Create an eager task factory, similar to *eager_task_factory()*, using the provided *custom_task_constructor* when creating a new task instead of the default *Task*.

custom_task_constructor must be a *callable* with the signature matching the signature of *Task.__init__*. The callable must return a *asyncio.Task*-compatible object.

This function returns a *callable* intended to be used as a task factory of an event loop via *loop.set_task_factory(factory)*).

Adicionado na versão 3.12.

Protegendo contra cancelamento

awaitable asyncio.**shield**(*aw*)

Protege um *objeto aguardável* de ser *cancelado*.

Se *aw* é uma corrotina, ela é automaticamente agendada como uma Tarefa.

A instrução:

```
task = asyncio.create_task(something())
res = await shield(task)
```

é equivalente a:

```
res = await something()
```

exceto que se a corrotina contendo-a for cancelada, a Tarefa executando em `something()` não é cancelada. Do ponto de vista de `something()`, o cancelamento não aconteceu. Apesar do autor da chamada ainda estar cancelado, então a expressão "await" ainda levanta um *CancelledError*.

Se `something()` é cancelada por outros meios (isto é, dentro ou a partir de si mesma) isso também iria cancelar `shield()`.

Se for desejado ignorar completamente os cancelamentos (não recomendado) a função `shield()` deve ser combinada com uma cláusula try/except, conforme abaixo:

```
task = asyncio.create_task(something())
try:
    res = await shield(task)
except CancelledError:
    res = None
```

Importante

Mantenha uma referência para as tarefas passadas para essa função função, evitando assim que uma tarefa desapareça durante a execução. O laço de eventos mantém apenas referências fracas para as tarefas. Uma tarefa que não é referenciada por nada mais pode ser removida pelo coletor de lixo a qualquer momento, antes mesmo da função ser finalizada.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se *aw* não é um objeto similar a Futuro, e não existe nenhum laço de eventos em execução.

Tempo limite

asyncio.**timeout**(*delay*)

Return an asynchronous context manager that can be used to limit the amount of time spent waiting on something.

delay can either be `None`, or a float/int number of seconds to wait. If *delay* is `None`, no time limit will be applied; this can be useful if the delay is unknown when the context manager is created.

In either case, the context manager can be rescheduled after creation using *Timeout.reschedule()*.

Exemplo:

```
async def main():
    async with asyncio.timeout(10):
        await long_running_task()
```

If `long_running_task` takes more than 10 seconds to complete, the context manager will cancel the current task and handle the resulting *asyncio.CancelledError* internally, transforming it into a *TimeoutError* which can be caught and handled.

Nota

The *asyncio.timeout()* context manager is what transforms the *asyncio.CancelledError* into a *TimeoutError*, which means the *TimeoutError* can only be caught *outside* of the context manager.

Example of catching *TimeoutError*:

```
async def main():
    try:
        async with asyncio.timeout(10):
            await long_running_task()
    except TimeoutError:
        print("The long operation timed out, but we've handled it.")

    print("This statement will run regardless.")
```

The context manager produced by *asyncio.timeout()* can be rescheduled to a different deadline and inspected.

class asyncio.**Timeout**(*when*)

An asynchronous context manager for cancelling overdue coroutines.

`when` should be an absolute time at which the context should time out, as measured by the event loop's clock:

- If `when` is `None`, the timeout will never trigger.
- If `when < loop.time()`, the timeout will trigger on the next iteration of the event loop.

when() → *float* | *None*

Return the current deadline, or `None` if the current deadline is not set.

reschedule(*when:* float | None)

Reschedule the timeout.

expired() → *bool*

Return whether the context manager has exceeded its deadline (expired).

Exemplo:

```
async def main():
    try:
        # We do not know the timeout when starting, so we pass ``None``.
        async with asyncio.timeout(None) as cm:
            # We know the timeout now, so we reschedule it.
            new_deadline = get_running_loop().time() + 10
            cm.reschedule(new_deadline)

            await long_running_task()
    except TimeoutError:
        pass

    if cm.expired():
        print("Looks like we haven't finished on time.")
```

Timeout context managers can be safely nested.

Adicionado na versão 3.11.

asyncio.**timeout_at**(*when*)

Similar to *asyncio.timeout()*, except *when* is the absolute time to stop waiting, or None.

Exemplo:

```
async def main():
    loop = get_running_loop()
    deadline = loop.time() + 20
    try:
        async with asyncio.timeout_at(deadline):
            await long_running_task()
    except TimeoutError:
        print("The long operation timed out, but we've handled it.")

    print("This statement will run regardless.")
```

Adicionado na versão 3.11.

async asyncio.**wait_for**(*aw*, *timeout*)

Espera o *aguardável* *aw* concluir sem ultrapassar o tempo limite "timeout".

Se *aw* é uma corrotina, ela é automaticamente agendada como uma Tarefa.

timeout pode ser None, ou um ponto flutuante, ou um número inteiro de segundos para aguardar. Se *timeout* é None, aguarda até o future encerrar.

If a timeout occurs, it cancels the task and raises *TimeoutError*.

Para evitar o *cancelamento* da tarefa, envolva-a com *shield()*.

A função irá aguardar até o future ser realmente cancelado, então o tempo total de espera pode exceder o tempo limite *timeout*. Se uma exceção ocorrer durante o cancelamento, ela será propagada.

Se ele for cancelado, o future *aw* também é cancelado.

Exemplo:

```
async def eternity():
    # Sleep for one hour
    await asyncio.sleep(3600)
    print('yay!')

async def main():
```

(continua na próxima página)

(continuação da página anterior)

```
    # Wait for at most 1 second
    try:
        await asyncio.wait_for(eternity(), timeout=1.0)
    except TimeoutError:
        print('timeout!')

asyncio.run(main())

# Expected output:
#
#     timeout!
```

Alterado na versão 3.7: When *aw* is cancelled due to a timeout, `wait_for` waits for *aw* to be cancelled. Previously, it raised *TimeoutError* immediately.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Raises *TimeoutError* instead of *asyncio.TimeoutError*.

Primitivas de Espera

async asyncio.**wait**(*aws*, *, *timeout=None*, *return_when=ALL_COMPLETED*)

Run *Future* and *Task* instances in the *aws* iterable concurrently and block until the condition specified by *return_when*.

O iterável *aws* não deve ser vazio.

Retorna dois conjuntos de Tarefas/Futuros: `(done, pending)`.

Uso:

```
done, pending = await asyncio.wait(aws)
```

timeout (um ponto flutuante ou inteiro), se especificado, pode ser usado para controlar o número máximo de segundos para aguardar antes de retornar.

Note that this function does not raise *TimeoutError*. Futures or Tasks that aren't done when the timeout occurs are simply returned in the second set.

return_when indica quando esta função deve retornar. Ele deve ser uma das seguintes constantes:

Constante	Descrição
asyncio.**FIRST_COMPLETED**	A função irá retornar quando qualquer futuro terminar ou for cancelado.
asyncio.**FIRST_EXCEPTION**	The function will return when any future finishes by raising an exception. If no future raises an exception then it is equivalent to *ALL_COMPLETED*.
asyncio.**ALL_COMPLETED**	A função irá retornar quando todos os futuros encerrarem ou forem cancelados.

Diferente de *wait_for()*, `wait()` não cancela os futuros quando um tempo limite é atingido.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Passing coroutine objects to `wait()` directly is forbidden.

Alterado na versão 3.12: Added support for generators yielding tasks.

asyncio.**as_completed**(*aws*, *, *timeout=None*)

Run *awaitable objects* in the *aws* iterable concurrently. The returned object can be iterated to obtain the results of the awaitables as they finish.

The object returned by `as_completed()` can be iterated as an *asynchronous iterator* or a plain *iterator*. When asynchronous iteration is used, the originally-supplied awaitables are yielded if they are tasks or futures. This makes it easy to correlate previously-scheduled tasks with their results. Example:

```
ipv4_connect = create_task(open_connection("127.0.0.1", 80))
ipv6_connect = create_task(open_connection("::1", 80))
tasks = [ipv4_connect, ipv6_connect]

async for earliest_connect in as_completed(tasks):
    # earliest_connect is done. The result can be obtained by
    # awaiting it or calling earliest_connect.result()
    reader, writer = await earliest_connect

    if earliest_connect is ipv6_connect:
        print("IPv6 connection established.")
    else:
        print("IPv4 connection established.")
```

During asynchronous iteration, implicitly-created tasks will be yielded for supplied awaitables that aren't tasks or futures.

When used as a plain iterator, each iteration yields a new coroutine that returns the result or raises the exception of the next completed awaitable. This pattern is compatible with Python versions older than 3.13:

```
ipv4_connect = create_task(open_connection("127.0.0.1", 80))
ipv6_connect = create_task(open_connection("::1", 80))
tasks = [ipv4_connect, ipv6_connect]

for next_connect in as_completed(tasks):
    # next_connect is not one of the original task objects. It must be
    # awaited to obtain the result value or raise the exception of the
    # awaitable that finishes next.
    reader, writer = await next_connect
```

A *TimeoutError* is raised if the timeout occurs before all awaitables are done. This is raised by the `async for` loop during asynchronous iteration or by the coroutines yielded during plain iteration.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se nem todos os objetos aguardáveis no iterável *aws* forem objetos similar a Futuro, e não existe nenhum laço de eventos em execução.

Alterado na versão 3.12: Added support for generators yielding tasks.

Alterado na versão 3.13: The result can now be used as either an *asynchronous iterator* or as a plain *iterator* (previously it was only a plain iterator).

Executando em Threads

async asyncio.**to_thread**(*func*, /, **args*, ***kwargs*)

Executa a função *func* assincronamente em uma thread separada.

Quaisquer *args e **kwargs fornecidos para esta função são diretamente passados para *func*. Além disso, o *contextvars.Context* atual é propagado, permitindo que variáveis de contexto da thread do laço de eventos sejam acessadas na thread separada.

Retorna uma corrotina que pode ser aguardada para obter o resultado eventual de *func*.

This coroutine function is primarily intended to be used for executing IO-bound functions/methods that would otherwise block the event loop if they were run in the main thread. For example:

```
def blocking_io():
    print(f"start blocking_io at {time.strftime('%X')}")
    # Note that time.sleep() can be replaced with any blocking
    # IO-bound operation, such as file operations.
    time.sleep(1)
    print(f"blocking_io complete at {time.strftime('%X')}")

async def main():
    print(f"started main at {time.strftime('%X')}")

    await asyncio.gather(
        asyncio.to_thread(blocking_io),
        asyncio.sleep(1))

    print(f"finished main at {time.strftime('%X')}")


asyncio.run(main())

# Expected output:
#
# started main at 19:50:53
# start blocking_io at 19:50:53
# blocking_io complete at 19:50:54
# finished main at 19:50:54
```

Chamar diretamente `blocking_io()` em qualquer corrotina iria bloquear o laço de eventos durante a sua duração, resultando em 1 segundo adicional no tempo de execução. Ao invés disso, ao utilizar `asyncio.to_thread()`, nós podemos executá-la em uma thread separada sem bloquear o laço de eventos.

Nota

Devido à *GIL*, `asyncio.to_thread()` pode tipicamente ser usado apenas para fazer funções vinculadas a IO não-bloqueantes. Entretanto, para módulos de extensão que liberam o GIL ou implementações alternativas do Python que não tem um, `asyncio.to_thread()` também pode ser usado para funções vinculadas à CPU.

Adicionado na versão 3.9.

Agendando a partir de outras Threads

`asyncio.`**`run_coroutine_threadsafe`**(*coro*, *loop*)

Envia uma corrotina para o laço de eventos fornecido. Seguro para thread.

Retorna um *concurrent.futures.Future* para aguardar pelo resultado de outra thread do sistema operacional.

Esta função destina-se a ser chamada partir de uma thread diferente do sistema operacional, da qual o laço de eventos está executando. Exemplo:

```
def in_thread(loop: asyncio.AbstractEventLoop) -> None:
    # Run some blocking IO
    pathlib.Path("example.txt").write_text("hello world", encoding="utf8")

    # Create a coroutine
```

(continua na próxima página)

(continuação da página anterior)

```
    coro = asyncio.sleep(1, result=3)

    # Submit the coroutine to a given loop
    future = asyncio.run_coroutine_threadsafe(coro, loop)

    # Wait for the result with an optional timeout argument
    assert future.result(timeout=2) == 3

async def amain() -> None:
    # Get the running loop
    loop = asyncio.get_running_loop()

    # Run something in a thread
    await asyncio.to_thread(in_thread, loop)
```

It's also possible to run the other way around. Example:

```
@contextlib.contextmanager
def loop_in_thread() -> Generator[asyncio.AbstractEventLoop]:
    loop_fut = concurrent.futures.Future[asyncio.AbstractEventLoop]()
    stop_event = asyncio.Event()

    async def main() -> None:
        loop_fut.set_result(asyncio.get_running_loop())
        await stop_event.wait()

    with concurrent.futures.ThreadPoolExecutor(1) as tpe:
        complete_fut = tpe.submit(asyncio.run, main())
        for fut in concurrent.futures.as_completed((loop_fut, complete_fut)):
            if fut is loop_fut:
                loop = loop_fut.result()
                try:
                    yield loop
                finally:
                    loop.call_soon_threadsafe(stop_event.set)
            else:
                fut.result()

# Create a loop in another thread
with loop_in_thread() as loop:
    # Create a coroutine
    coro = asyncio.sleep(1, result=3)

    # Submit the coroutine to a given loop
    future = asyncio.run_coroutine_threadsafe(coro, loop)

    # Wait for the result with an optional timeout argument
    assert future.result(timeout=2) == 3
```

Se uma exceção for levantada na corrotina, o Futuro retornado será notificado. Isso também pode ser usado para cancelar a tarefa no laço de eventos:

```
try:
    result = future.result(timeout)
except TimeoutError:
    print('The coroutine took too long, cancelling the task...')
    future.cancel()
```

(continua na próxima página)

(continuação da página anterior)

```
except Exception as exc:
    print(f'The coroutine raised an exception: {exc!r}')
else:
    print(f'The coroutine returned: {result!r}')
```

Veja a seção *concorrência e multithreading* da documentação.

Ao contrário de outras funções asyncio, esta função requer que o argumento *loop* seja passado explicitamente.

Adicionado na versão 3.5.1.

Introspecção

`asyncio.`**`current_task`**(*loop=None*)

Retorna a instância `Task` atualmente em execução, ou `None` se nenhuma tarefa estiver executando.

Se *loop* for `None`, então `get_running_loop()` é usado para obter o laço atual.

Adicionado na versão 3.7.

`asyncio.`**`all_tasks`**(*loop=None*)

Retorna um conjunto de objetos `Task` ainda não concluídos a serem executados pelo laço.

Se *loop* for `None`, então `get_running_loop()` é usado para obter o laço atual.

Adicionado na versão 3.7.

`asyncio.`**`iscoroutine`**(*obj*)

Return `True` if *obj* is a coroutine object.

Adicionado na versão 3.4.

Objeto Task

`class` `asyncio.`**`Task`**(*coro*, ***, *loop=None*, *name=None*, *context=None*, *eager_start=False*)

Um objeto `similar a Futuro` que executa uma *corrotina* Python. Não é seguro para thread.

Tarefas são usadas para executar corrotinas em laços de eventos. Se uma corrotina espera por um Futuro, a Tarefa suspende a execução da corrotina e aguarda a conclusão do Futuro. Quando o Futuro é *concluído*, a execução da corrotina contida é retomada.

Laço de eventos usam agendamento cooperativo: um ciclo de evento executa uma Tarefa de cada vez. Enquanto uma Tarefa aguarda a conclusão de um Futuro, o laço de eventos executa outras Tarefas, funções de retorno, ou executa operações de IO.

Use a função de alto nível `asyncio.create_task()` para criar Tarefas, ou as funções de baixo nível `loop.create_task()` ou `ensure_future()`. Instanciação manual de Tarefas é desencorajado.

Para cancelar uma Tarefa em execução, use o método `cancel()`. Chamar ele fará com que a Tarefa levante uma exceção `CancelledError` dentro da corrotina contida. Se a corrotina estiver esperando por um objeto Future durante o cancelamento, o objeto Future será cancelado.

`cancelled()` pode ser usado para verificar se a Tarefa foi cancelada. O método retorna `True` se a corrotina envolta não suprimiu a exceção `CancelledError` e foi na verdade cancelada.

`asyncio.Task` herda de `Future` todas as suas APIs exceto `Future.set_result()` e `Future.set_exception()`.

An optional keyword-only *context* argument allows specifying a custom `contextvars.Context` for the *coro* to run in. If no *context* is provided, the Task copies the current context and later runs its coroutine in the copied context.

An optional keyword-only *eager_start* argument allows eagerly starting the execution of the `asyncio.Task` at task creation time. If set to `True` and the event loop is running, the task will start executing the coroutine

immediately, until the first time the coroutine blocks. If the coroutine returns or raises without blocking, the task will be finished eagerly and will skip scheduling to the event loop.

Alterado na versão 3.7: Adicionado suporte para o módulo *contextvars*.

Alterado na versão 3.8: Adicionado o parâmetro *name*.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se *loop* não é especificado, e não existe nenhum laço de eventos em execução.

Alterado na versão 3.11: Adicionado o parâmetro *context*.

Alterado na versão 3.12: Added the *eager_start* parameter.

done()

Retorna `True` se a Tarefa estiver *concluída*.

Uma Tarefa está *concluída* quando a corrotina contida retornou um valor, ou levantou uma exceção, ou a Tarefa foi cancelada.

result()

Retorna o resultado da Tarefa.

Se a Tarefa estiver *concluída*, o resultado da corrotina contida é retornado (ou se a corrotina levantou uma exceção, essa exceção é re-levantada.)

Se a Tarefa foi *cancelada*, este método levanta uma exceção *CancelledError*.

If the Task's result isn't yet available, this method raises an *InvalidStateError* exception.

exception()

Retorna a exceção de uma Tarefa.

Se a corrotina contida levantou uma exceção, essa exceção é retornada. Se a corrotina contida retornou normalmente, este método retorna `None`.

Se a Tarefa foi *cancelada*, este método levanta uma exceção *CancelledError*.

Se a Tarefa não estiver *concluída* ainda, este método levanta uma exceção *InvalidStateError*.

add_done_callback(*callback*, *, *context=None*)

Adiciona uma função de retorno para ser executada quando a Tarefa estiver *concluída*.

Este método deve ser usado apenas em código de baixo nível baseado em funções de retorno.

Veja a documentação para *Future.add_done_callback()* para mais detalhes.

remove_done_callback(*callback*)

Remove *callback* da lista de funções de retorno.

Este método deve ser usado apenas em código de baixo nível baseado em funções de retorno.

Veja a documentação do método *Future.remove_done_callback()* para mais detalhes.

get_stack(*, *limit=None*)

Retorna a lista de frames da pilha para esta Tarefa.

Se a corrotina contida não estiver concluída, isto retorna a pilha onde ela foi suspensa. Se a corrotina foi concluída com sucesso ou foi cancelada, isto retorna uma lista vazia. Se a corrotina foi terminada por uma exceção, isto retorna a lista de frames do traceback (situação da pilha de execução).

Os quadros são sempre ordenados dos mais antigos para os mais recentes.

Apenas um frame da pilha é retornado para uma corrotina suspensa.

O argumento opcional *limit* define o o número de frames máximo para retornar; por padrão todos os frames disponíveis são retornados. O ordenamento da lista retornada é diferente dependendo se uma pilha ou um traceback (situação da pilha de execução) é retornado: os frames mais recentes de uma pilha são retornados, mas os frames mais antigos de um traceback são retornados. (Isso combina com o comportamento do módulo traceback.)

print_stack(*, *limit=None*, *file=None*)

Exibe a pilha ou situação da pilha de execução para esta Tarefa.

Isto produz uma saída similar a do módulo traceback para frames recuperados por *get_stack()*.

O argumento *limit* é passado para *get_stack()* diretamente.

The *file* argument is an I/O stream to which the output is written; by default output is written to *sys.stdout*.

get_coro()

Retorna o objeto corrotina contido pela *Task*.

> **Nota**
>
> This will return `None` for Tasks which have already completed eagerly. See the *Eager Task Factory*.

Adicionado na versão 3.8.

Alterado na versão 3.12: Newly added eager task execution means result may be `None`.

get_context()

Return the *contextvars.Context* object associated with the task.

Adicionado na versão 3.12.

get_name()

Retorna o nome da Tarefa.

Se nenhum nome foi explicitamente designado para a Tarefa, a implementação padrão asyncio da classe Task gera um nome padrão durante a instanciação.

Adicionado na versão 3.8.

set_name(*value*)

Define o nome da Tarefa.

O argumento *value* pode ser qualquer objeto, o qual é então convertido para uma string.

Na implementação padrão da Tarefa, o nome será visível na *repr()* de saída de um objeto task.

Adicionado na versão 3.8.

cancel(*msg=None*)

Solicita o cancelamento da Tarefa.

If the Task is already *done* or *cancelled*, return `False`, otherwise, return `True`.

The method arranges for a *CancelledError* exception to be thrown into the wrapped coroutine on the next cycle of the event loop.

The coroutine then has a chance to clean up or even deny the request by suppressing the exception with a `try` ... ... `except CancelledError` ... `finally` block. Therefore, unlike *Future.cancel()*, *Task.cancel()* does not guarantee that the Task will be cancelled, although suppressing cancellation completely is not common and is actively discouraged. Should the coroutine nevertheless decide to suppress the cancellation, it needs to call *Task.uncancel()* in addition to catching the exception.

Alterado na versão 3.9: Adicionado o parâmetro *msg*.

Alterado na versão 3.11: The `msg` parameter is propagated from cancelled task to its awaiter. O seguinte exemplo ilustra como corrotinas podem interceptar o cancelamento de requisições:

```
async def cancel_me():
    print('cancel_me(): before sleep')
```

(continua na próxima página)

(continuação da página anterior)

```
    try:
        # Wait for 1 hour
        await asyncio.sleep(3600)
    except asyncio.CancelledError:
        print('cancel_me(): cancel sleep')
        raise
    finally:
        print('cancel_me(): after sleep')

async def main():
    # Create a "cancel_me" Task
    task = asyncio.create_task(cancel_me())

    # Wait for 1 second
    await asyncio.sleep(1)

    task.cancel()
    try:
        await task
    except asyncio.CancelledError:
        print("main(): cancel_me is cancelled now")

asyncio.run(main())

# Expected output:
#
#     cancel_me(): before sleep
#     cancel_me(): cancel sleep
#     cancel_me(): after sleep
#     main(): cancel_me is cancelled now
```

cancelled()

Retorna `True` se a Tarefa for *cancelada*.

A Tarefa é *cancelada* quando o cancelamento foi requisitado com *cancel()* e a corrotina contida propagou a exceção *CancelledError* gerada nela.

uncancel()

Decrement the count of cancellation requests to this Task.

Returns the remaining number of cancellation requests.

Note that once execution of a cancelled task completed, further calls to *uncancel()* are ineffective.

Adicionado na versão 3.11.

This method is used by asyncio's internals and isn't expected to be used by end-user code. In particular, if a Task gets successfully uncancelled, this allows for elements of structured concurrency like *Task Groups* and *asyncio.timeout()* to continue running, isolating cancellation to the respective structured block. For example:

```
async def make_request_with_timeout():
    try:
        async with asyncio.timeout(1):
            # Structured block affected by the timeout:
            await make_request()
            await make_another_request()
    except TimeoutError:
        log("There was a timeout")
```

(continua na próxima página)

(continuação da página anterior)

```
    # Outer code not affected by the timeout:
    await unrelated_code()
```

While the block with `make_request()` and `make_another_request()` might get cancelled due to the timeout, `unrelated_code()` should continue running even in case of the timeout. This is implemented with *`uncancel()`*. *`TaskGroup`* context managers use *`uncancel()`* in a similar fashion.

If end-user code is, for some reason, suppressing cancellation by catching *`CancelledError`*, it needs to call this method to remove the cancellation state.

When this method decrements the cancellation count to zero, the method checks if a previous *`cancel()`* call had arranged for *`CancelledError`* to be thrown into the task. If it hasn't been thrown yet, that arrangement will be rescinded (by resetting the internal `_must_cancel` flag).

Alterado na versão 3.13: Changed to rescind pending cancellation requests upon reaching zero.

`cancelling`()

Return the number of pending cancellation requests to this Task, i.e., the number of calls to *`cancel()`* less the number of *`uncancel()`* calls.

Note that if this number is greater than zero but the Task is still executing, *`cancelled()`* will still return `False`. This is because this number can be lowered by calling *`uncancel()`*, which can lead to the task not being cancelled after all if the cancellation requests go down to zero.

This method is used by asyncio's internals and isn't expected to be used by end-user code. See *`uncancel()`* for more details.

Adicionado na versão 3.11.

19.1.3 Streams

Código-fonte: Lib/asyncio/streams.py

Streams são conexões de rede de alto-nível assincronias/espera-pronta. Streams permitem envios e recebimentos de dados sem usar retornos de chamadas ou protocolos de baixo nível.

Aqui está um exemplo de um cliente TCP realizando eco, escrito usando streams asyncio:

```
import asyncio

async def tcp_echo_client(message):
    reader, writer = await asyncio.open_connection(
        '127.0.0.1', 8888)

    print(f'Send: {message!r}')
    writer.write(message.encode())
    await writer.drain()

    data = await reader.read(100)
    print(f'Received: {data.decode()!r}')

    print('Close the connection')
    writer.close()
    await writer.wait_closed()

asyncio.run(tcp_echo_client('Hello World!'))
```

Veja também a seção *Exemplos* abaixo.

Funções Stream

As seguintes funções asyncio de alto nível podem ser usadas para criar e trabalhar com streams:

async asyncio.**open_connection**(*host=None*, *port=None*, ***, *limit=None*, *ssl=None*, *family=0*, *proto=0*, *flags=0*, *sock=None*, *local_addr=None*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *happy_eyeballs_delay=None*, *interleave=None*)

Estabelece uma conexão de rede e retorna um par de objetos `(reader, writer)`.

Os objetos *reader* e *writer* retornados são instâncias das classes `StreamReader` e `StreamWriter`.

limit determina o tamanho limite do buffer usado pela instância `StreamReader` retornada. Por padrão, *limit* é definido em 64 KiB.

O resto dos argumentos é passado diretamente para `loop.create_connection()`.

> **Nota**
>
> The *sock* argument transfers ownership of the socket to the `StreamWriter` created. To close the socket, call its `close()` method.

Alterado na versão 3.7: Adicionado o parâmetro *ssl_handshake_timeout*.

Alterado na versão 3.8: Adicionados os parâmetros *happy_eyeballs_delay* e *interleave*.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

async asyncio.**start_server**(*client_connected_cb*, *host=None*, *port=None*, ***, *limit=None*, *family=socket.AF_UNSPEC*, *flags=socket.AI_PASSIVE*, *sock=None*, *backlog=100*, *ssl=None*, *reuse_address=None*, *reuse_port=None*, *keep_alive=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *start_serving=True*)

Inicia um soquete no servidor.

A função de retorno *client_connected_cb* é chamada sempre que uma nova conexão de um cliente é estabelecida. Ela recebe um par `(reader, writer)` como dois argumentos, instâncias das classes `StreamReader` e `StreamWriter`.

client_connected_cb pode ser simplesmente algo chamável ou uma *função de corrotina*; se ele for uma função de corrotina, ele será automaticamente agendado como uma `Task`.

limit determina o tamanho limite do buffer usado pela instância `StreamReader` retornada. Por padrão, *limit* é definido em 64 KiB.

O resto dos argumentos são passados diretamente para `loop.create_server()`.

> **Nota**
>
> The *sock* argument transfers ownership of the socket to the server created. To close the socket, call the server's `close()` method.

Alterado na versão 3.7: Added the *ssl_handshake_timeout* and *start_serving* parameters.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.13: Added the *keep_alive* parameter.

Soquetes Unix

async asyncio.**open_unix_connection**(*path=None*, ***, *limit=None*, *ssl=None*, *sock=None*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Estabelece uma conexão de soquete Unix e retorna um par com `(reader, writer)`.

Similar a *open_connection()*, mas opera em soquetes Unix.

Veja também a documentação do método *loop.create_unix_connection()*.

Nota

The *sock* argument transfers ownership of the socket to the *StreamWriter* created. To close the socket, call its *close()* method.

Disponibilidade: Unix.

Alterado na versão 3.7: Added the *ssl_handshake_timeout* parameter. The *path* parameter can now be a *path-like object*

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

async asyncio.**start_unix_server**(*client_connected_cb*, *path=None*, ***, *limit=None*, *sock=None*, *backlog=100*, *ssl=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *start_serving=True*, *cleanup_socket=True*)

Inicia um servidor com soquete Unix.

Similar a *start_server()*, mas funciona com soquetes Unix.

If *cleanup_socket* is true then the Unix socket will automatically be removed from the filesystem when the server is closed, unless the socket has been replaced after the server has been created.

Veja também a documentação do método *loop.create_unix_server()*.

Nota

The *sock* argument transfers ownership of the socket to the server created. To close the socket, call the server's *close()* method.

Disponibilidade: Unix.

Alterado na versão 3.7: Added the *ssl_handshake_timeout* and *start_serving* parameters. The *path* parameter can now be a *path-like object*.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.13: Added the *cleanup_socket* parameter.

StreamReader

class asyncio.**StreamReader**

Represents a reader object that provides APIs to read data from the IO stream. As an *asynchronous iterable*, the object supports the `async for` statement.

Não é recomendado instanciar objetos *StreamReader* diretamente; use *open_connection()* e *start_server()* ao invés disso.

feed_eof()

Acknowledge the EOF.

async read(*n=-1*)

Read up to *n* bytes from the stream.

If *n* is not provided or set to `-1`, read until EOF, then return all read *bytes*. If EOF was received and the internal buffer is empty, return an empty `bytes` object.

If *n* is `0`, return an empty `bytes` object immediately.

If *n* is positive, return at most *n* available `bytes` as soon as at least 1 byte is available in the internal buffer. If EOF is received before any byte is read, return an empty `bytes` object.

async readline()

Lê uma linha, onde "line" é uma sequência de bytes encerrando com `\n`.

Se EOF é recebido e `\n` não foi encontrado, o método retorna os dados parcialmente lidos.

Se EOF for recebido e o buffer interno estiver vazio, retorna um objeto `bytes` vazio.

async readexactly(*n*)

Lê exatamente *n* bytes.

Levanta um *IncompleteReadError* se EOF é atingido antes que *n* sejam lidos. Use o atributo *IncompleteReadError.partial* para obter os dados parcialmente lidos.

async readuntil(*separator=b'\n'*)

Lê dados a partir do stream até que *separator* seja encontrado.

Ao ter sucesso, os dados e o separador serão removidos do buffer interno (consumido). Dados retornados irão incluir o separador no final.

Se a quantidade de dados lidos excede o limite configurado para o stream, uma exceção *LimitOverrunError* é levantada, e os dados são deixados no buffer interno e podem ser lidos novamente.

Se EOF for atingido antes que o separador completo seja encontrado, uma exceção *IncompleteReadError* é levantada, e o buffer interno é redefinido. O atributo *IncompleteReadError.partial* pode conter uma parte do separador.

The *separator* may also be a tuple of separators. In this case the return value will be the shortest possible that has any separator as the suffix. For the purposes of *LimitOverrunError*, the shortest possible separator is considered to be the one that matched.

Adicionado na versão 3.5.2.

Alterado na versão 3.13: The *separator* parameter may now be a *tuple* of separators.

at_eof()

Retorna `True` se o buffer estiver vazio e *feed_eof()* foi chamado.

StreamWriter

class asyncio.**StreamWriter**

Representa um objeto de escrita que fornece APIs para escrever dados para o stream de IO.

Não é recomendado instanciar objetos *StreamWriter* diretamente; use *open_connection()* e *start_server()* ao invés.

write(*data*)

O método tenta escrever *data* para o soquete subjacente imediatamente. Se isso falhar, data é enfileirado em um buffer interno de escrita, até que possa ser enviado.

O método deve ser usado juntamente com o método `drain()`:

```
stream.write(data)
await stream.drain()
```

Nota

The *data* buffer should be a C contiguous one-dimensional *bytes-like object*.

writelines(*data*)

O método escreve imediatamente a lista (ou qualquer iterável) de bytes para o soquete subjacente. Se isso falhar, os dados são enfileirados em um buffer de escrita interno até que possam ser enviados.

O método deve ser usado juntamente com o método `drain()`:

```
stream.writelines(lines)
await stream.drain()
```

close()

O método fecha o stream e o soquete subjacente.

The method should be used, though not mandatory, along with the `wait_closed()` method:

```
stream.close()
await stream.wait_closed()
```

can_write_eof()

Retorna `True` se o transporte subjacente suporta o método *`write_eof()`*, `False` caso contrário.

write_eof()

Fecha o extremo de escrita do stream após os dados no buffer de escrita terem sido descarregados.

transport

Retorna o transporte asyncio subjacente.

get_extra_info(*name*, *default=None*)

Acessa informações de transporte opcionais; veja *`BaseTransport.get_extra_info()`* para detalhes.

async drain()

Aguarda até que seja apropriado continuar escrevendo no stream. Exemplo:

```
writer.write(data)
await writer.drain()
```

Este é um método de controle de fluxo que interage com o buffer de entrada e saída de escrita subjacente. Quando o tamanho do buffer atinge a marca d'água alta, *drain()* bloqueia até que o tamanho do buffer seja drenado para a marca d'água baixa, e a escrita possa continuar. Quando não existe nada que cause uma espera, o método *`drain()`* retorna imediatamente.

async start_tls(*sslcontext*, ***, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Upgrade an existing stream-based connection to TLS.

Parâmetros:

- *sslcontext*: uma instância configurada de *`SSLContext`*.
- *server_hostname*: define ou substitui o nome do host no qual o servidor alvo do certificado será comparado.

- *ssl_handshake_timeout* is the time in seconds to wait for the TLS handshake to complete before aborting the connection. `60.0` seconds if `None` (default).
- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0` seconds if `None` (default).

Adicionado na versão 3.11.

Alterado na versão 3.12: Added the *ssl_shutdown_timeout* parameter.

is_closing()

Retorna `True` se o stream estiver fechado ou em processo de ser fechado.

Adicionado na versão 3.7.

async wait_closed()

Aguarda até que o stream seja fechado.

Should be called after *close()* to wait until the underlying connection is closed, ensuring that all data has been flushed before e.g. exiting the program.

Adicionado na versão 3.7.

Exemplos

Cliente para eco TCP usando streams

Cliente de eco TCP usando a função *asyncio.open_connection()*:

```
import asyncio

async def tcp_echo_client(message):
    reader, writer = await asyncio.open_connection(
        '127.0.0.1', 8888)

    print(f'Send: {message!r}')
    writer.write(message.encode())
    await writer.drain()

    data = await reader.read(100)
    print(f'Received: {data.decode()!r}')

    print('Close the connection')
    writer.close()
    await writer.wait_closed()

asyncio.run(tcp_echo_client('Hello World!'))
```

Ver também

O exemplo de *protocolo do cliente para eco TCP* usa o método de baixo nível *loop.create_connection()*.

Servidor eco TCP usando streams

Servidor eco TCP usando a função *asyncio.start_server()*:

```
import asyncio

async def handle_echo(reader, writer):
    data = await reader.read(100)
```

(continua na próxima página)

(continuação da página anterior)

```
    message = data.decode()
    addr = writer.get_extra_info('peername')

    print(f"Received {message!r} from {addr!r}")

    print(f"Send: {message!r}")
    writer.write(data)
    await writer.drain()

    print("Close the connection")
    writer.close()
    await writer.wait_closed()

async def main():
    server = await asyncio.start_server(
        handle_echo, '127.0.0.1', 8888)

    addrs = ', '.join(str(sock.getsockname()) for sock in server.sockets)
    print(f'Serving on {addrs}')

    async with server:
        await server.serve_forever()

asyncio.run(main())
```

Ver também

O exemplo de *protocolo eco de servidor TCP* utiliza o método `loop.create_server()`.

Obtém headers HTTP

Exemplo simples consultando cabeçalhos HTTP da URL passada na linha de comando:

```
import asyncio
import urllib.parse
import sys

async def print_http_headers(url):
    url = urllib.parse.urlsplit(url)
    if url.scheme == 'https':
        reader, writer = await asyncio.open_connection(
            url.hostname, 443, ssl=True)
    else:
        reader, writer = await asyncio.open_connection(
            url.hostname, 80)

    query = (
        f"HEAD {url.path or '/'} HTTP/1.0\r\n"
        f"Host: {url.hostname}\r\n"
        f"\r\n"
    )

    writer.write(query.encode('latin-1'))
    while True:
        line = await reader.readline()
```

(continua na próxima página)

(continuação da página anterior)

```
        if not line:
            break

        line = line.decode('latin1').rstrip()
        if line:
            print(f'HTTP header> {line}')

    # Ignore the body, close the socket
    writer.close()
    await writer.wait_closed()

url = sys.argv[1]
asyncio.run(print_http_headers(url))
```

Uso:

```
python example.py http://example.com/path/page.html
```

ou com HTTPS:

```
python example.py https://example.com/path/page.html
```

Registra um soquete aberto para aguardar por dados usando streams

Corrotina aguardando até que um soquete receba dados usando a função *open_connection()*:

```
import asyncio
import socket

async def wait_for_data():
    # Get a reference to the current event loop because
    # we want to access low-level APIs.
    loop = asyncio.get_running_loop()

    # Create a pair of connected sockets.
    rsock, wsock = socket.socketpair()

    # Register the open socket to wait for data.
    reader, writer = await asyncio.open_connection(sock=rsock)

    # Simulate the reception of data from the network
    loop.call_soon(wsock.send, 'abc'.encode())

    # Wait for data
    data = await reader.read(100)

    # Got data, we are done: close the socket
    print("Received:", data.decode())
    writer.close()
    await writer.wait_closed()

    # Close the second socket
    wsock.close()

asyncio.run(wait_for_data())
```

Ver também

O exemplo de *registro de um soquete aberto para aguardar por dados usando um protocolo* utiliza um protocolo de baixo nível e o método `loop.create_connection()`.

O exemplo para *monitorar um descritor de arquivo para leitura de eventos* utiliza o método de baixo nível `loop.add_reader()` para monitorar um descritor de arquivo.

19.1.4 Synchronization Primitives

Código-fonte: Lib/asyncio/locks.py

---

asyncio synchronization primitives are designed to be similar to those of the `threading` module with two important caveats:

- asyncio primitives are not thread-safe, therefore they should not be used for OS thread synchronization (use `threading` for that);
- methods of these synchronization primitives do not accept the *timeout* argument; use the `asyncio.wait_for()` function to perform operations with timeouts.

asyncio has the following basic synchronization primitives:

- `Lock`
- `Event`
- `Condition`
- `Semaphore`
- `BoundedSemaphore`
- `Barrier`

Lock

class asyncio.**Lock**

Implements a mutex lock for asyncio tasks. Not thread-safe.

An asyncio lock can be used to guarantee exclusive access to a shared resource.

The preferred way to use a Lock is an `async with` statement:

```
lock = asyncio.Lock()

# ... later
async with lock:
    # access shared state
```

which is equivalent to:

```
lock = asyncio.Lock()

# ... later
await lock.acquire()
try:
    # access shared state
finally:
    lock.release()
```

Alterado na versão 3.10: Removido o parâmetro *loop*.

async acquire()

Acquire the lock.

This method waits until the lock is *unlocked*, sets it to *locked* and returns `True`.

When more than one coroutine is blocked in *acquire()* waiting for the lock to be unlocked, only one coroutine eventually proceeds.

Acquiring a lock is *fair*: the coroutine that proceeds will be the first coroutine that started waiting on the lock.

release()

Release the lock.

When the lock is *locked*, reset it to *unlocked* and return.

If the lock is *unlocked*, a *RuntimeError* is raised.

locked()

Return `True` if the lock is *locked*.

Evento

class asyncio.**Event**

An event object. Not thread-safe.

An asyncio event can be used to notify multiple asyncio tasks that some event has happened.

An Event object manages an internal flag that can be set to *true* with the *set()* method and reset to *false* with the *clear()* method. The *wait()* method blocks until the flag is set to *true*. The flag is set to *false* initially.

Alterado na versão 3.10: Removido o parâmetro *loop*. Exemplo:

```
async def waiter(event):
    print('waiting for it ...')
    await event.wait()
    print('... got it!')

async def main():
    # Create an Event object.
    event = asyncio.Event()

    # Spawn a Task to wait until 'event' is set.
    waiter_task = asyncio.create_task(waiter(event))

    # Sleep for 1 second and set the event.
    await asyncio.sleep(1)
    event.set()

    # Wait until the waiter task is finished.
    await waiter_task

asyncio.run(main())
```

async wait()

Wait until the event is set.

If the event is set, return `True` immediately. Otherwise block until another task calls *set()*.

set()

Set the event.

All tasks waiting for event to be set will be immediately awakened.

clear()

Clear (unset) the event.

Subsequent tasks awaiting on *wait()* will now block until the *set()* method is called again.

is_set()

Return `True` if the event is set.

Condição

class asyncio.**Condition**(*lock=None*)

A Condition object. Not thread-safe.

An asyncio condition primitive can be used by a task to wait for some event to happen and then get exclusive access to a shared resource.

In essence, a Condition object combines the functionality of an *Event* and a *Lock*. It is possible to have multiple Condition objects share one Lock, which allows coordinating exclusive access to a shared resource between different tasks interested in particular states of that shared resource.

The optional *lock* argument must be a *Lock* object or `None`. In the latter case a new Lock object is created automatically.

Alterado na versão 3.10: Removido o parâmetro *loop*.

The preferred way to use a Condition is an `async with` statement:

```
cond = asyncio.Condition()

# ... later
async with cond:
    await cond.wait()
```

which is equivalent to:

```
cond = asyncio.Condition()

# ... later
await cond.acquire()
try:
    await cond.wait()
finally:
    cond.release()
```

async acquire()

Acquire the underlying lock.

This method waits until the underlying lock is *unlocked*, sets it to *locked* and returns `True`.

notify(*n=1*)

Wake up *n* tasks (1 by default) waiting on this condition. If fewer than *n* tasks are waiting they are all awakened.

The lock must be acquired before this method is called and released shortly after. If called with an *unlocked* lock a *RuntimeError* error is raised.

locked()

Return `True` if the underlying lock is acquired.

notify_all()

Wake up all tasks waiting on this condition.

This method acts like *notify()*, but wakes up all waiting tasks.

The lock must be acquired before this method is called and released shortly after. If called with an *unlocked* lock a *RuntimeError* error is raised.

release()

Release the underlying lock.

When invoked on an unlocked lock, a *RuntimeError* is raised.

async wait()

Wait until notified.

If the calling task has not acquired the lock when this method is called, a *RuntimeError* is raised.

This method releases the underlying lock, and then blocks until it is awakened by a *notify()* or *notify_all()* call. Once awakened, the Condition re-acquires its lock and this method returns `True`.

Note that a task *may* return from this call spuriously, which is why the caller should always re-check the state and be prepared to *wait()* again. For this reason, you may prefer to use *wait_for()* instead.

async wait_for(*predicate*)

Wait until a predicate becomes *true*.

The predicate must be a callable which result will be interpreted as a boolean value. The method will repeatedly *wait()* until the predicate evaluates to *true*. The final value is the return value.

Semaphore

class asyncio.**Semaphore**(*value=1*)

A Semaphore object. Not thread-safe.

A semaphore manages an internal counter which is decremented by each *acquire()* call and incremented by each *release()* call. The counter can never go below zero; when *acquire()* finds that it is zero, it blocks, waiting until some task calls *release()*.

The optional *value* argument gives the initial value for the internal counter (`1` by default). If the given value is less than `0` a *ValueError* is raised.

Alterado na versão 3.10: Removido o parâmetro *loop*.

The preferred way to use a Semaphore is an `async with` statement:

```
sem = asyncio.Semaphore(10)

# ... later
async with sem:
    # work with shared resource
```

which is equivalent to:

```
sem = asyncio.Semaphore(10)

# ... later
await sem.acquire()
try:
    # work with shared resource
finally:
    sem.release()
```

async acquire()

Acquire a semaphore.

If the internal counter is greater than zero, decrement it by one and return `True` immediately. If it is zero, wait until a *release()* is called and return `True`.

locked()

Returns `True` if semaphore can not be acquired immediately.

release()

Release a semaphore, incrementing the internal counter by one. Can wake up a task waiting to acquire the semaphore.

Unlike *BoundedSemaphore*, *Semaphore* allows making more `release()` calls than `acquire()` calls.

BoundedSemaphore

class asyncio.**BoundedSemaphore**(*value=1*)

A bounded semaphore object. Not thread-safe.

Bounded Semaphore is a version of *Semaphore* that raises a *ValueError* in *release()* if it increases the internal counter above the initial *value*.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Barrier

class asyncio.**Barrier**(*parties*)

A barrier object. Not thread-safe.

A barrier is a simple synchronization primitive that allows to block until *parties* number of tasks are waiting on it. Tasks can wait on the *wait()* method and would be blocked until the specified number of tasks end up waiting on *wait()*. At that point all of the waiting tasks would unblock simultaneously.

`async with` can be used as an alternative to awaiting on *wait()*.

The barrier can be reused any number of times.

Exemplo:

```
async def example_barrier():
    # barrier with 3 parties
    b = asyncio.Barrier(3)

    # create 2 new waiting tasks
    asyncio.create_task(b.wait())
    asyncio.create_task(b.wait())

    await asyncio.sleep(0)
    print(b)

    # The third .wait() call passes the barrier
    await b.wait()
    print(b)
    print("barrier passed")

    await asyncio.sleep(0)
    print(b)

asyncio.run(example_barrier())
```

Result of this example is:

```
<asyncio.locks.Barrier object at 0x... [filling, waiters:2/3]>
<asyncio.locks.Barrier object at 0x... [draining, waiters:0/3]>
barrier passed
<asyncio.locks.Barrier object at 0x... [filling, waiters:0/3]>
```

Adicionado na versão 3.11.

async wait()

Pass the barrier. When all the tasks party to the barrier have called this function, they are all unblocked simultaneously.

When a waiting or blocked task in the barrier is cancelled, this task exits the barrier which stays in the same state. If the state of the barrier is "filling", the number of waiting task decreases by 1.

The return value is an integer in the range of 0 to `parties-1`, different for each task. This can be used to select a task to do some special housekeeping, e.g.:

```
...
async with barrier as position:
    if position == 0:
        # Only one task prints this
        print('End of *draining phase*')
```

This method may raise a *BrokenBarrierError* exception if the barrier is broken or reset while a task is waiting. It could raise a *CancelledError* if a task is cancelled.

async reset()

Return the barrier to the default, empty state. Any tasks waiting on it will receive the *BrokenBarrierError* exception.

If a barrier is broken it may be better to just leave it and create a new one.

async abort()

Put the barrier into a broken state. This causes any active or future calls to *wait()* to fail with the *BrokenBarrierError*. Use this for example if one of the tasks needs to abort, to avoid infinite waiting tasks.

parties

The number of tasks required to pass the barrier.

n_waiting

The number of tasks currently waiting in the barrier while filling.

broken

A boolean that is `True` if the barrier is in the broken state.

exception asyncio.**BrokenBarrierError**

This exception, a subclass of *RuntimeError*, is raised when the *Barrier* object is reset or broken.

Alterado na versão 3.9: Acquiring a lock using `await lock` or `yield from lock` and/or `with` statement (`with await lock`, `with (yield from lock)`) was removed. Use `async with lock` instead.

19.1.5 Subprocessos

Código-fonte: Lib/asyncio/subprocess.py, Lib/asyncio/base_subprocess.py

---

Esta seção descreve APIs async/await asyncio de alto nível para criar e gerenciar subprocessos.

Aqui está um exemplo de como asyncio pode executar um comando shell e obter o seu resultado:

```
import asyncio

async def run(cmd):
    proc = await asyncio.create_subprocess_shell(
        cmd,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE)

    stdout, stderr = await proc.communicate()

    print(f'[{cmd!r} exited with {proc.returncode}]')
    if stdout:
        print(f'[stdout]\n{stdout.decode()}')
    if stderr:
        print(f'[stderr]\n{stderr.decode()}')

asyncio.run(run('ls /zzz'))
```

irá exibir:

```
['ls /zzz' exited with 1]
[stderr]
ls: /zzz: No such file or directory
```

Devido ao fato que todas as funções de subprocessos asyncio são assíncronas e asyncio fornece muitas ferramentas para trabalhar com tais funções, é fácil executar e monitorar múltiplos subprocessos em paralelo. É na verdade trivial modificar o exemplo acima para executar diversos comandos simultaneamente:

```
async def main():
    await asyncio.gather(
        run('ls /zzz'),
        run('sleep 1; echo "hello"'))

asyncio.run(main())
```

Veja também a subseção *Exemplos*.

Criando subprocessos

async asyncio.**create_subprocess_exec**(*program*, **args*, *stdin=None*, *stdout=None*, *stderr=None*, *limit=None*, ***kwds*)

Cria um subprocesso.

O argumento *limit* define o limite do buffer para os invólucros `StreamReader` para `stdout` e `stderr` (se `subprocess.PIPE` for passado para os argumentos *stdout* e *stderr*).

Retorna uma instância de `Process`.

Veja a documentação de `loop.subprocess_exec()` para outros parâmetros.

Alterado na versão 3.10: Removido o parâmetro *loop*.

async asyncio.**create_subprocess_shell**(*cmd*, *stdin=None*, *stdout=None*, *stderr=None*, *limit=None*, ***kwds*)

Executa o comando *cmd* no shell.

O argumento *limit* define o limite do buffer para os invólucros `StreamReader` para `stdout` e `stderr` (se `subprocess.PIPE` for passado para os argumentos *stdout* e *stderr*).

Retorna uma instância de `Process`.

Veja a documentação de `loop.subprocess_shell()` para outros parâmetros.

Importante

É responsabilidade da aplicação garantir que todos os espaços em branco e caracteres especiais tenham aspas apropriadamente para evitar vulnerabilidades de injeção de shell. A função `shlex.quote()` pode ser usada para escapar espaços em branco e caracteres especiais de shell apropriadamente em strings que serão usadas para construir comandos shell.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Nota

Subprocessos estão disponíveis para Windows se uma `ProactorEventLoop` for usada. Veja *Suporte para subprocesso para Windows* para detalhes.

Ver também

asyncio também tem as seguintes APIs *de baixo nível* para trabalhar com subprocessos: `loop.subprocess_exec()`, `loop.subprocess_shell()`, `loop.connect_read_pipe()`, `loop.connect_write_pipe()`, assim como os *Transportes de Subprocesso* e *Protocolos de Subprocesso*.

Constantes

`asyncio.subprocess.`**`PIPE`**

Pode ser passado para os parâmetros *stdin*, *stdout* ou *stderr*.

Se *PIPE* for passado para o argumento *stdin*, o atributo `Process.stdin` irá apontar para uma instância `StreamWriter`.

Se *PIPE* for passado para os argumentos *stdout* ou *stderr*, os atributos `Process.stdout` e `Process.stderr` irão apontar para instâncias `StreamReader`.

`asyncio.subprocess.`**`STDOUT`**

Valor especial que pode ser usado como o argumento *stderr* e indica que a saída de erro padrão deve ser redirecionada para a saída padrão.

`asyncio.subprocess.`**`DEVNULL`**

Valor especial que pode ser usado como argumento *stdin*, *stdout* ou *stderr* para funções de criação de processos. Ele indica que o arquivo especial `os.devnull` será usado para o fluxo de subprocesso correspondente.

Interagindo com subprocessos

Ambas as funções `create_subprocess_exec()` e `create_subprocess_shell()` retornam instâncias da classe *Process*. *Process* é um invólucro de alto nível que permite a comunicação com subprocessos e observar eles serem completados.

`class` `asyncio.subprocess.`**`Process`**

Um objeto que envolve processos do sistema operacional criados pelas funções `create_subprocess_exec()` e `create_subprocess_shell()`.

Esta classe é projetada para ter uma API similar a classe `subprocess.Popen`, mas existem algumas diferenças notáveis:

- ao contrário de Popen, instâncias de Process não têm um equivalente ao método `poll()`;
- os métodos `communicate()` e `wait()` não têm um parâmetro *timeout*: utilize a função `wait_for()`;
- o método `Process.wait()` é assíncrono, enquanto que o método `subprocess.Popen.wait()` é implementado como um laço bloqueante para indicar que está ocupado;

- o parâmetro *universal_newlines* não é suportado.

Esta classe *não é segura para thread*.

Veja também a seção *Subprocesso e Threads*.

async wait()

Aguarda o processo filho encerrar.

Define e retorna o atributo `returncode`.

> **Nota**
>
> Este método pode entrar em deadlock ao usar `stdout=PIPE` ou `stderr=PIPE` e o processo filho gera tantas saídas que ele bloqueia a espera pelo encadeamento de buffer do sistema operacional para aceitar mais dados. Use o método `communicate()` ao usar encadeamentos para evitar essa condição.

async communicate(*input=None*)

Interage com processo:

1. envia dados para *stdin* (se *input* for diferente de `None`);
2. fecha *stdin*;
3. lê dados a partir de *stdout* e *stderr*, até que EOF (fim do arquivo) seja atingido;
4. aguarda o processo encerrar.

O argumento opcional *input* é a informação (objeto `bytes`) que será enviada para o processo filho.

Retorna uma tupla `(stdout_data, stderr_data)`.

Se qualquer exceção `BrokenPipeError` ou `ConnectionResetError` for levantada ao escrever *input* em *stdin*, a exceção é ignorada. Esta condição ocorre quando o processo encerra antes de todos os dados serem escritos em *stdin*.

Se for desejado enviar dados para o *stdin* do processo, o mesmo precisa ser criado com `stdin=PIPE`. De forma similar, para obter qualquer coisa além de `None` na tupla resultante, o processo precisa ser criado com os argumentos `stdout=PIPE` e/ou `stderr=PIPE`.

Perceba que, os dados lidos são armazenados em um buffer na memória, então não use este método se o tamanho dos dados é grande ou ilimitado.

Alterado na versão 3.12: *stdin* é fechado quando `input=None` é também.

send_signal(*signal*)

Envia o sinal *signal* para o processo filho.

> **Nota**
>
> No Windows, `SIGTERM` é um apelido para `terminate()`. `CTRL_C_EVENT` e `CTRL_BREAK_EVENT` podem ser enviados para processos iniciados com um parâmetro *creationflags*, o qual inclui `CREATE_NEW_PROCESS_GROUP`.

terminate()

Interrompe o processo filho.

Em sistemas POSIX este método envia `SIGTERM` para o processo filho.

No Windows a função `TerminateProcess()` da API Win32 é chamada para interromper o processo filho.

kill()

Mata o processo filho.

Em sistemas POSIX este método envia *SIGKILL* para o processo filho.

No Windows, este método é um atalho para *terminate()*.

stdin

Fluxo de entrada padrão (*StreamWriter*) ou `None` se o processo foi criado com `stdin=None`.

stdout

Fluxo de saída padrão (*StreamReader*) ou `None` se o processo foi criado com `stdout=None`.

stderr

Erro de fluxo padrão (*StreamReader*) ou `None` se o processo foi criado com `stderr=None`.

Aviso

Use o método *communicate()* ao invés de *process.stdin.write()*, *await process.stdout.read()* ou *await process.stderr.read()*. Isso evita deadlocks devido a fluxos pausando a leitura ou escrita, e bloqueando o processo filho.

pid

Número de identificação do processo (PID).

Perceba que para processos criados pela função *create_subprocess_shell()*, este atributo é o PID do console gerado.

returncode

Retorna o código do processo quando o mesmo terminar.

Um valor `None` indica que o processo ainda não terminou.

Um valor negativo `-N` indica que o filho foi terminado pelo sinal `N` (POSIX apenas).

Subprocesso e Threads

Laço de eventos padrão do asyncio suporta a execução de subprocessos a partir de diferentes threads por padrão.

No Windows, subprocessos são fornecidos pela classe *ProactorEventLoop* apenas (por padrão), a classe *SelectorEventLoop* não tem suporte a subprocesso.

Perceba que implementações alternativas do laço de eventos podem ter limitações próprias; por favor, verifique a sua documentação.

Ver também

A seção *Concorrência e multithreading em asyncio*.

Exemplos

Um exemplo de uso da classe *Process* para controlar um subprocesso e a classe *StreamReader* para ler a partir da sua saída padrão.

O subprocesso é criado pela função *create_subprocess_exec()*:

```
import asyncio
import sys

async def get_date():
```

(continua na próxima página)

(continuação da página anterior)

```
    code = 'import datetime; print(datetime.datetime.now())'

    # Cria um subprocesso; redireciona a saída padrão
    # para o encadeamento.
    proc = await asyncio.create_subprocess_exec(
        sys.executable, '-c', code,
        stdout=asyncio.subprocess.PIPE)

    # Lê uma linha da saída.
    data = await proc.stdout.readline()
    line = data.decode('ascii').rstrip()

    # Espera o subprocesso sair.
    await proc.wait()
    return line

date = asyncio.run(get_date())
print(f"Current date: {date}")
```

Veja também o *mesmo exemplo* escrito usando APIs de baixo nível.

19.1.6 Filas

Código-fonte: Lib/asyncio/queues.py

Filas asyncio são projetadas para serem similar a classes do módulo `queue`. Apesar de filas asyncio não serem seguras para thread, elas são projetadas para serem usadas especificamente em código async/await.

Perceba que métodos de filas asyncio não possuem um parâmetro *timeout*; use a função `asyncio.wait_for()` para realizar operações de fila com um tempo limite timeout.

Veja também a seção *Exemplos* abaixo.

Queue

class asyncio.**Queue**(*maxsize=0*)

Uma fila onde o primeiro a entrar, é o primeiro a sair (FIFO - First In First Out).

Se *maxsize* for menor que ou igual a zero, o tamanho da fila é infinito. Se ele for um inteiro maior que `0`, então `await put()` bloqueia quando a fila atingir *maxsize* até que um item seja removido por `get()`.

Ao contrário da biblioteca padrão de threading `queue`, o tamanho da fila é sempre conhecido e pode ser obtido através da chamada do método `qsize()`.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Esta classe *não é segura para thread*.

maxsize

Número de itens permitidos na fila.

empty()

Retorna `True` se a fila estiver vazia, `False` caso contrário.

full()

Retorna `True` se existem `maxsize` itens na fila.

Se a fila foi inicializada com `maxsize=0` (o padrão), então `full()` nunca retorna `True`.

async get()

Remove e retorna um item da fila. Se a fila estiver vazia, aguarda até que um item esteja disponível.

Levanta *QueueShutDown* se a fila foi encerrada e está vazia, ou se a fila foi encerrada imediatamente.

get_nowait()

Retorna um item se houver um imediatamente disponível, caso contrário levanta *QueueEmpty*.

async join()

Bloqueia até que todos os itens na fila tenham sido recebidos e processados.

A contagem de tarefas inacabadas aumenta sempre que um item é adicionado à fila. A contagem diminui sempre que uma corrotina consumidora chama *task_done()* para indicar que o item foi recuperado e todo o trabalho nele foi concluído. Quando a contagem de tarefas inacabadas chega a zero, *join()* desbloqueia.

async put(*item*)

Coloca um item na fila. Se a fila estiver cheia, aguarda até que uma posição livre esteja disponível antes de adicionar o item.

Levanta *QueueShutDown* se a fila foi encerrada.

put_nowait(*item*)

Coloca um item na fila sem bloqueá-la.

Se nenhuma posição livre estiver imediatamente disponível, levanta *QueueFull*.

qsize()

Retorna o número de itens na fila.

shutdown(*immediate=False*)

Desliga a fila, fazendo com que *get()* e *put()* levantem *QueueShutDown*.

Por padrão, *get()* em uma fila de desligamento vai levantar quando a fila estiver vazia. Defina *immediate* como verdadeiro para fazer *get()* levantar imediatamente.

Todos os chamadores bloqueados de *put()* e *get()* serão desbloqueados. Se *immediate* for verdadeiro, uma tarefa será marcada como concluída para cada item restante na fila, o que pode desbloquear chamadores de *join()*.

Adicionado na versão 3.13.

task_done()

Indica que o item de trabalho anteriormente enfileirado está concluído.

Usada por consumidores de fila. Para cada *get()* usado para buscar um item de trabalho, uma chamada subsequente para *task_done()* avisa à fila, que o processamento no item de trabalho está concluído.

Se um *join()* estiver sendo bloqueado no momento, ele irá continuar quando todos os itens tiverem sido processados (significando que uma chamada *task_done()* foi recebida para cada item que foi chamado o método *put()* para colocar na fila).

`shutdown(immediate=True)` chama *task_done()* para cada item restante na fila.

Levanta *ValueError* se chamada mais vezes do que a quantidade de itens existentes na fila.

Fila de prioridade

class asyncio.**PriorityQueue**

Uma variante de *Queue*; recupera entradas em ordem de prioridade (mais baixas primeiro).

Entradas são tipicamente tuplas no formato `(priority_number, data)`.

Filas LIFO (último a entrar, primeiro a sair)

class asyncio.**LifoQueue**

Uma variante de *Queue* que recupera as entradas adicionadas mais recentemente primeiro (último a entrar, primeiro a sair).

Exceções

exception asyncio.**QueueEmpty**

Esta exceção é levantada quando o método *get_nowait()* é chamado em uma fila vazia.

exception asyncio.**QueueFull**

Exceção levantada quando o método *put_nowait()* é chamado em uma fila que atingiu seu *maxsize*.

exception asyncio.**QueueShutDown**

Exceção levantada quando *put()* ou *get()* é chamado em uma fila que foi desligada.

Adicionado na versão 3.13.

Exemplos

Filas podem ser usadas para distribuir cargas de trabalho entre diversas tarefas concorrentes:

```
import asyncio
import random
import time


async def worker(nome, fila):
    while True:
        # Retira um "item de trabalho" da fila.
        dormir_por = await queue.get()

        # Dorme por "dormir_por" segundos.
        await asyncio.sleep(dormir_por)

        # Notifica a fila de que o "item de trabalho" foi processado.
        fila.task_done()

        print(f'{nome} dormiu por {dormir_por:.2f} segundos')


async def main():
    # Cria uma fila para armazenar nossa "carga de trabalho".
    fila = asyncio.Queue()

    # Gera tempos aleatórios e os insere na fila.
    tempo_total_de_sono = 0
    for _ in range(20):
        dormir_por = random.uniform(0.05, 1.0)
        tempo_total_de_sono += dormir_por
        fila.put_nowait(dormir_por)

    # Cria três tarefas "worker" para processarem a fila concorrentemente.
    tarefas = []
    for i in range(3):
        tarefa = asyncio.create_task(worker(f'worker-{i}', fila))
        tarefas.append(tarefa)
```

(continua na próxima página)

(continuação da página anterior)

```
    # Espera até a fila ser completamente processada.
    comecou_em = time.monotonic()
    await fila.join()
    tempo_dormido = time.monotonic() - comecou_em

    # Cancela nossas tarefas.
    for tarefa in tarefas:
        tarefa.cancel()
    # Espera até todas as tarefas serem canceladas.
    await asyncio.gather(*tarefas, return_exceptions=True)

    print('====')
    print(f'3 trabalhadoras dormiram em paralelo por {tempo_dormido:.2f} segundos')
    print(f'Tempo total de sono esperado: {tempo_total_de_sono:.2f} segundos')


asyncio.run(main())
```

19.1.7 Exceções

Código-fonte: Lib/asyncio/exceptions.py

---

exception asyncio.**TimeoutError**

Um apelido descontinuado de *TimeoutError*, levantado quando a operação excedeu o prazo determinado.

Alterado na versão 3.11: Esta classe foi feita como um apelido de *TimeoutError*.

exception asyncio.**CancelledError**

A operação foi cancelada.

Esta exceção pode ser capturada para executar operações personalizadas quando as tarefas assíncronas são canceladas. Em quase todas as situações, a exceção deve ser levantada novamente.

Alterado na versão 3.8: *CancelledError* é agora uma subclasse de *BaseException* em vez de *Exception*.

exception asyncio.**InvalidStateError**

Estado interno inválido de *Task* ou *Future*.

Pode ser levantada em situações como definir um valor de resultado para um objeto *Future* que já tem um valor de resultado definido.

exception asyncio.**SendfileNotAvailableError**

A *syscall* "sendfile" não está disponível para o soquete ou tipo de arquivo fornecido.

Uma subclasse de *RuntimeError*.

exception asyncio.**IncompleteReadError**

A operação de leitura solicitada não foi totalmente concluída.

Levantada pelas *APIs de fluxo de asyncio*.

Esta exceção é uma subclasse de *EOFError*.

expected

O número total (*int*) de bytes esperados.

partial

Uma string de *bytes* lida antes do final do fluxo ser alcançado.

exception asyncio.**LimitOverrunError**

Atingiu o limite de tamanho do buffer ao procurar um separador.

Levantada pelas *APIs de fluxo de asyncio*.

consumed

O número total de bytes a serem consumidos.

19.1.8 Call Graph Introspection

Source code: Lib/asyncio/graph.py

---

asyncio has powerful runtime call graph introspection utilities to trace the entire call graph of a running *coroutine* or *task*, or a suspended *future*. These utilities and the underlying machinery can be used from within a Python program or by external profilers and debuggers.

Adicionado na versão 3.14.

asyncio.**print_call_graph**(*future=None*, */*, ***, *file=None*, *depth=1*, *limit=None*)

Print the async call graph for the current task or the provided *Task* or *Future*.

This function prints entries starting from the top frame and going down towards the invocation point.

The function receives an optional *future* argument. If not passed, the current running task will be used.

If the function is called on *the current task*, the optional keyword-only *depth* argument can be used to skip the specified number of frames from top of the stack.

If the optional keyword-only *limit* argument is provided, each call stack in the resulting graph is truncated to include at most `abs(limit)` entries. If *limit* is positive, the entries left are the closest to the invocation point. If *limit* is negative, the topmost entries are left. If *limit* is omitted or `None`, all entries are present. If *limit* is `0`, the call stack is not printed at all, only "awaited by" information is printed.

If *file* is omitted or `None`, the function will print to *sys.stdout*.

Example:

The following Python code:

```
import asyncio

async def test():
    asyncio.print_call_graph()

async def main():
    async with asyncio.TaskGroup() as g:
        g.create_task(test(), name='test')

asyncio.run(main())
```

irá exibir:

```
* Task(name='test', id=0x1039f0fe0)
+ Call stack:
|   File 't2.py', line 4, in async test()
+ Awaited by:
   * Task(name='Task-1', id=0x103a5e060)
     + Call stack:
     |   File 'taskgroups.py', line 107, in async TaskGroup.__aexit__()
     |   File 't2.py', line 7, in async main()
```

asyncio.**format_call_graph**(*future=None*, */*, ***, *depth=1*, *limit=None*)

Like *print_call_graph()*, but returns a string. If *future* is None and there's no current task, the function returns an empty string.

asyncio.**capture_call_graph**(*future=None*, */*, ***, *depth=1*, *limit=None*)

Capture the async call graph for the current task or the provided *Task* or *Future*.

The function receives an optional *future* argument. If not passed, the current running task will be used. If there's no current task, the function returns None.

If the function is called on *the current task*, the optional keyword-only *depth* argument can be used to skip the specified number of frames from top of the stack.

Returns a FutureCallGraph data class object:

- FutureCallGraph(future, call_stack, awaited_by)

  Where *future* is a reference to a *Future* or a *Task* (or their subclasses.)

  call_stack is a tuple of FrameCallGraphEntry objects.

  awaited_by is a tuple of FutureCallGraph objects.

- FrameCallGraphEntry(frame)

  Where *frame* is a frame object of a regular Python function in the call stack.

Low level utility functions

To introspect an async call graph asyncio requires cooperation from control flow structures, such as *shield()* or *TaskGroup*. Any time an intermediate *Future* object with low-level APIs like *Future.add_done_callback()* is involved, the following two functions should be used to inform asyncio about how exactly such intermediate future objects are connected with the tasks they wrap or control.

asyncio.**future_add_to_awaited_by**(*future*, *waiter*, */*)

Record that *future* is awaited on by *waiter*.

Both *future* and *waiter* must be instances of *Future* or *Task* or their subclasses, otherwise the call would have no effect.

A call to future_add_to_awaited_by() must be followed by an eventual call to the *future_discard_from_awaited_by()* function with the same arguments.

asyncio.**future_discard_from_awaited_by**(*future*, *waiter*, */*)

Record that *future* is no longer awaited on by *waiter*.

Both *future* and *waiter* must be instances of *Future* or *Task* or their subclasses, otherwise the call would have no effect.

19.1.9 Laço de Eventos

Código-fonte: Lib/asyncio/events.py, Lib/asyncio/base_events.py

---

Prefácio

O laço de eventos é o núcleo de toda aplicação asyncio. Laços de eventos executam tarefas e funções de retorno assíncronas, realizam operações de entrada e saída e executam subprocessos.

Os desenvolvedores de aplicação normalmente devem usar as funções asyncio de alto nível, como *asyncio.run()*, e devem raramente precisar fazer referência ao objeto de loop ou chamar seus métodos. Esta seção destina-se principalmente a autores de código de baixo nível, bibliotecas e frameworks, que precisam de um controle mais preciso sobre o comportamento do laço de evento.

Obtendo o laço de eventos

As seguintes funções baixo nível podem ser usadas para obter, definir, ou criar um laço de eventos:

asyncio.**get_running_loop**()

Retorna o laço de eventos em execução na thread atual do sistema operacional.

Levanta uma *RuntimeError* se não houver nenhum laço de eventos em execução.

Esta função só pode ser chamada a partir de uma corrotina ou de um retorno de chamada.

Adicionado na versão 3.7.

asyncio.**get_event_loop**()

Obtém o laço de eventos atual.

Quando chamada de uma corrotina ou função de retorno (por exemplo, agendada com call_soon ou API semelhante), esta função sempre retornará o laço de eventos em execução.

Se não houver nenhum laço de eventos em execução definido, a função retornará o resultado da chamada `get_event_loop_policy().get_event_loop()`.

Devido ao fato desta função ter um comportamento particularmente complexo (especialmente quando políticas de laço de eventos customizadas estão sendo usadas), usar a função *get_running_loop()* é preferido ao invés de *get_event_loop()* em corrotinas e funções de retorno.

Como observado acima, considere usar também a função de alto nível *asyncio.run()* ao invés de usar funções de baixo nível para manualmente criar e fechar um laço de eventos.

Alterado na versão 3.14: Raises a *RuntimeError* if there is no current event loop.

Nota

The `asyncio` policy system is deprecated and will be removed in Python 3.16; from there on, this function will return the current running event loop if present else it will return the loop set by *set_event_loop()*.

asyncio.**set_event_loop**(*loop*)

Define *loop* como o laço de eventos atual para a thread atual do sistema operacional.

asyncio.**new_event_loop**()

Cria e retorna um novo objeto de laço de eventos.

Perceba que o comportamento das funções *get_event_loop()*, *set_event_loop()*, e *new_event_loop()* podem ser alteradas *definindo uma política de laço de eventos customizada*.

Conteúdo

Esta página de documentação contém as seguintes seções:

- A seção *Métodos do laço de eventos* é a documentação de referência das APIs de laço de eventos;
- A seção *Tratadores de função de retorno* documenta as instâncias das classes *Handle* e *TimerHandle*, que são retornadas por métodos de agendamento tais como *loop.call_soon()* e *loop.call_later()*;
- A seção *Objetos Server* documenta tipos retornados a partir de métodos de laço de eventos, como *loop.create_server()*;
- A seção *Implementações do Laço de Eventos* documenta as classes *SelectorEventLoop* e *ProactorEventLoop*;
- A seção *Exemplos* demonstra como trabalhar com algumas APIs do laço de eventos APIs.

Métodos do laço de eventos

Laços de eventos possuem APIs de **baixo nível** para as seguintes situações:

- *Executar e interromper o laço*
- *Agendando funções de retorno*
- *Agendando funções de retorno atrasadas*
- *Criando Futures e Tasks*
- *Abrindo conexões de rede*
- *Criando servidores de rede*
- *Transferindo arquivos*
- *Atualizando TLS*
- *Observando descritores de arquivo*
- *Trabalhando com objetos soquete diretamente*
- *DNS*
- *Trabalhando com encadeamentos*
- *Sinais Unix*
- *Executando código em conjuntos de threads ou processos*
- *Tratando erros da API*
- *Habilitando o modo de debug*
- *Executando Subprocessos*

Executar e interromper o laço

`loop.`**`run_until_complete`**`(`*future*`)`

Executar até que o *future* (uma instância da classe `Future`) seja completada.

Se o argumento é um *objeto corrotina*, ele é implicitamente agendado para executar como uma `asyncio.Task`.

Retorna o resultado do Future ou levanta sua exceção.

`loop.`**`run_forever`**`()`

Executa o laço de eventos até que `stop()` seja chamado.

If `stop()` is called before `run_forever()` is called, the loop will poll the I/O selector once with a timeout of zero, run all callbacks scheduled in response to I/O events (and those that were already scheduled), and then exit.

Se `stop()` for chamado enquanto `run_forever()` estiver executando, o laço irá executar o lote atual de funções de retorno e então sair. Perceba que novas funções de retorno agendadas por funções de retorno não serão executadas neste caso; ao invés disso, elas serão executadas na próxima vez que `run_forever()` ou `run_until_complete()` forem chamadas.

`loop.`**`stop`**`()`

Para o laço de eventos.

`loop.`**`is_running`**`()`

Retorna `True` se o laço de eventos estiver em execução.

loop.**is_closed**()

Retorna `True` se o laço de eventos foi fechado.

loop.**close**()

Fecha o laço de eventos.

O laço não deve estar em execução quando esta função for chamada. Qualquer função de retorno pendente será descartada.

Este método limpa todas as filas e desliga o executor, mas não aguarda pelo encerramento do executor.

Este método é idempotente e irreversível. Nenhum outro método deve ser chamado depois que o laço de eventos esteja fechado.

async loop.**shutdown_asyncgens**()

Schedule all currently open *asynchronous generator* objects to close with an `aclose()` call. After calling this method, the event loop will issue a warning if a new asynchronous generator is iterated. This should be used to reliably finalize all scheduled asynchronous generators.

Perceba que não é necessário chamar esta função quando *`asyncio.run()`* for usado.

Exemplo:

```
try:
    loop.run_forever()
finally:
    loop.run_until_complete(loop.shutdown_asyncgens())
    loop.close()
```

Adicionado na versão 3.6.

async loop.**shutdown_default_executor**(*timeout=None*)

Agenda o encerramento do executor padrão e aguarda ele se juntar a todas as threads no *`ThreadPoolExecutor`*. Uma vez que este método tenha sido chamado, usar o executor padrão com *`loop.run_in_executor()`* levantará um *`RuntimeError`*.

O parâmetro *timeout* especifica a quantidade de tempo (em *`float`* segundos) que o executor terá para terminar a junção. Com o padrão `None`, o executor tem permissão para um período de tempo ilimitado.

Se o *timeout* for atingido, uma exceção *`RuntimeWarning`* será emitida e o executor padrão será finalizado sem esperar que suas threads terminem a junção.

Nota

Do not call this method when using *`asyncio.run()`*, as the latter handles default executor shutdown automatically.

Adicionado na versão 3.9.

Alterado na versão 3.12: Added the *timeout* parameter.

Agendando funções de retorno

loop.**call_soon**(*callback*, **args*, *context=None*)

Agenda a *função de retorno* *callback* para ser chamada com argumentos *args* na próxima iteração do laço de eventos.

Return an instance of *`asyncio.Handle`*, which can be used later to cancel the callback.

Funções de retorno são chamadas na ordem em que elas foram registradas. Cada função de retorno será chamada exatamente uma vez.

The optional keyword-only *context* argument specifies a custom `contextvars.Context` for the *callback* to run in. Callbacks use the current context when no *context* is provided.

Unlike `call_soon_threadsafe()`, this method is not thread-safe.

`loop.`**`call_soon_threadsafe`**(*callback*, **args*, *context=None*)

A thread-safe variant of `call_soon()`. When scheduling callbacks from another thread, this function *must* be used, since `call_soon()` is not thread-safe.

This function is safe to be called from a reentrant context or signal handler, however, it is not safe or fruitful to use the returned handle in such contexts.

Levanta `RuntimeError` se chamado em um laço que já foi fechado. Isto pode acontecer em uma thread secundária quando a aplicação principal estiver desligando.

Veja a seção *concorrência e multithreading* da documentação.

Alterado na versão 3.7: O parâmetro somente-nomeado *context* foi adicionado. Veja **PEP 567** para mais detalhes.

Nota

Maior parte das funções de agendamento `asyncio` não permite passar argumentos nomeados. Para fazer isso, use `functools.partial()`:

```
# will schedule "print("Hello", flush=True)"
loop.call_soon(
    functools.partial(print, "Hello", flush=True))
```

Usar objetos parciais é usualmente mais conveniente que usar lambdas, pois asyncio pode renderizar objetos parciais melhor durante debug e mensagens de erro.

Agendando funções de retorno atrasadas

Laço de eventos fornece mecanismos para agendar funções de retorno para serem chamadas em algum ponto no futuro. Laço de eventos usa relógios monotônico para acompanhar o tempo.

`loop.`**`call_later`**(*delay*, *callback*, **args*, *context=None*)

Agenda *callback* para ser chamada após o *delay* número de segundos fornecido (pode ser um inteiro ou um ponto flutuante).

Uma instância de `asyncio.TimerHandle` é retornada, a qual pode ser usada para cancelar a função de retorno.

callback será chamada exatamente uma vez. Se duas funções de retorno são agendadas para exatamente o mesmo tempo, a ordem na qual elas são chamadas é indefinida.

O *args* posicional opcional será passado para a função de retorno quando ela for chamada. Se você quiser que a função de retorno seja chamada com argumentos nomeados, use `functools.partial()`.

Um argumento opcional somente-nomeado *context* permite especificar um `contextvars.Context` customizado para executar na *função de retorno*. O contexto atual é usado quando nenhum *context* é fornecido.

Nota

For performance, callbacks scheduled with `loop.call_later()` may run up to one clock-resolution early (see `time.get_clock_info('monotonic').resolution`).

Alterado na versão 3.7: O parâmetro somente-nomeado *context* foi adicionado. Veja **PEP 567** para mais detalhes.

Alterado na versão 3.8: No Python 3.7 e anterior, com a implementação padrão do laço de eventos, o *delay* não poderia exceder um dia. Isto foi ajustado no Python 3.8.

`loop.`**`call_at`**`(`*when*, *callback*, **args*, *context=None*`)`

Agenda *callback* para ser chamada no timestamp absoluto fornecido *when* (um inteiro ou um ponto flutuante), usando o mesmo horário de referência que `loop.time()`.

O comportamento deste método é o mesmo que `call_later()`.

Uma instância de `asyncio.TimerHandle` é retornada, a qual pode ser usada para cancelar a função de retorno.

Nota

For performance, callbacks scheduled with `loop.call_at()` may run up to one clock-resolution early (see `time.get_clock_info('monotonic').resolution`).

Alterado na versão 3.7: O parâmetro somente-nomeado *context* foi adicionado. Veja **PEP 567** para mais detalhes.

Alterado na versão 3.8: No Python 3.7 e anterior, com a implementação padrão do laço de eventos, a diferença entre *when* e o horário atual não poderia exceder um dia. Isto foi ajustado no Python 3.8.

`loop.`**`time`**`()`

Retorna o horário atual, como um valor `float`, de acordo com o relógio monotônico interno do laço de eventos.

Nota

Alterado na versão 3.8: No Python 3.7 e anterior, tempos limites (*delay* relativo ou *when* absoluto) não poderiam exceder um dia. Isto foi ajustado no Python 3.8.

Ver também

A função `asyncio.sleep()`.

Criando Futures e Tasks

`loop.`**`create_future`**`()`

Cria um objeto `asyncio.Future` anexado ao laço de eventos.

Este é o modo preferido para criar Futures em asyncio. Isto permite que laços de eventos de terceiros forneçam implementações alternativas do objeto Future (com melhor desempenho ou instrumentação).

Adicionado na versão 3.5.2.

`loop.`**`create_task`**`(`*coro*, ***, *name=None*, *context=None*, *eager_start=None*, ***kwargs*`)`

Schedule the execution of *coroutine* *coro*. Return a `Task` object.

Laços de eventos de terceiros podem usar suas próprias subclasses de `Task` para interoperabilidade. Neste caso, o tipo do resultado é uma subclasse de `Task`.

The full function signature is largely the same as that of the `Task` constructor (or factory) - all of the keyword arguments to this function are passed through to that interface.

Se o argumento *name* for fornecido e não é `None`, ele é definido como o nome da tarefa, usando `Task.set_name()`.

An optional keyword-only *context* argument allows specifying a custom `contextvars.Context` for the *coro* to run in. The current context copy is created when no *context* is provided.

An optional keyword-only *eager_start* argument allows specifying if the task should execute eagerly during the call to create_task, or be scheduled later. If *eager_start* is not passed the mode set by *loop.set_task_factory()* will be used.

Alterado na versão 3.8: Adicionado o parâmetro *name*.

Alterado na versão 3.11: Adicionado o parâmetro *context*.

Alterado na versão 3.13.3: Added `kwargs` which passes on arbitrary extra parameters, including `name` and `context`.

Alterado na versão 3.13.4: Rolled back the change that passes on *name* and *context* (if it is None), while still passing on other arbitrary keyword arguments (to avoid breaking backwards compatibility with 3.13.3).

Alterado na versão 3.14: All *kwargs* are now passed on. The *eager_start* parameter works with eager task factories.

`loop.`**`set_task_factory`**(*factory*)

Define a factory da tarefa que será usada por *loop.create_task()*.

If *factory* is `None` the default task factory will be set. Otherwise, *factory* must be a *callable* with the signature matching `(loop, coro, **kwargs)`, where *loop* is a reference to the active event loop, and *coro* is a coroutine object. The callable must pass on all *kwargs*, and return a *asyncio.Task*-compatible object.

Alterado na versão 3.13.3: Required that all *kwargs* are passed on to *asyncio.Task*.

Alterado na versão 3.13.4: *name* is no longer passed to task factories. *context* is no longer passed to task factories if it is `None`.

Alterado na versão 3.14: *name* and *context* are now unconditionally passed on to task factories again.

`loop.`**`get_task_factory`**()

Retorna uma factory de tarefa ou `None` se a factory padrão estiver em uso.

Abrindo conexões de rede

`async` `loop.`**`create_connection`**(*protocol_factory*, *host=None*, *port=None*, *, *ssl=None*, *family=0*, *proto=0*, *flags=0*, *sock=None*, *local_addr=None*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *happy_eyeballs_delay=None*, *interleave=None*, *all_errors=False*)

Abre uma conexão de transporte para streaming, para um endereço fornecido, especificado por *host* e *port*.

The socket family can be either *AF_INET* or *AF_INET6* depending on *host* (or the *family* argument, if provided).

The socket type will be *SOCK_STREAM*.

protocol_factory deve ser um chamável que retorne uma implementação do *protocolo asyncio*.

Este método tentará estabelecer a conexão em segundo plano. Quando tiver sucesso, ele retorna um par `(transport, protocol)`.

A sinopse cronológica da operação subjacente é conforme abaixo:

1. A conexão é estabelecida e um *transporte* é criado para ela.
2. *protocol_factory* é chamada sem argumentos e é esperada que retorne uma instância de *protocolo*.
3. A instância de protocolo é acoplada com o transporte, através da chamada do seu método *connection_made()*.
4. Uma tupla `(transport, protocol)` é retornada ao ter sucesso.

O transporte criado é um stream bi-direcional dependente de implementação.

Outros argumentos:

- *ssl*: se fornecido e não for falso, um transporte SSL/TLS é criado (por padrão um transporte TCP simples é criado). Se *ssl* for um objeto `ssl.SSLContext`, este contexto é usado para criar o transporte; se *ssl* for `True`, um contexto padrão retornado de `ssl.create_default_context()` é usado.

  Ver também

  Considerações de segurança sobre SSL/TLS

- *server_hostname* define ou substitui o hostname que o certificado do servidor de destino será pareado contra. Deve ser passado apenas se *ssl* não for `None`. Por padrão o valor do argumento *host* é usado. Se *host* for vazio, não existe valor padrão e você deve passar um valor para *server_hostname*. Se *server_hostname* for uma string vazia, o pareamento de hostname é desabilitado (o que é um risco de segurança sério, permitindo ataques potenciais man-in-the-middle).
- *family*, *proto*, *flags* são os endereços familiares, protocolos e sinalizadores opcionais a serem passados por getaddrinfo() para resolução do *host*. Se fornecidos, eles devem ser todos inteiros e constantes correspondentes do módulo `socket`.
- *happy_eyeballs_delay*, se fornecido, habilita Happy Eyeballs para esta conexão. Ele deve ser um número de ponto flutuante representando o tempo em segundos para aguardar uma tentativa de conexão encerrar, antes de começar a próxima tentativa em paralelo. Este é o "Atraso na tentativa de conexão" conforme definido na **RFC 8305**. Um valor padrão sensível recomendado pela RFC é `0.25` (250 millissegundos).
- *interleave* controla o reordenamento de endereços quando um nome de servidor resolve para múltiplos endereços IP. Se `0` ou não especificado, nenhum reordenamento é feito, e endereços são tentados na ordem retornada por `getaddrinfo()`. Se um inteiro positivo for especificado, os endereços são intercalados por um endereço familiar, e o inteiro fornecido é interpretado como "Contagem da família do primeiro endereço" conforme definido na **RFC 8305**. O padrão é `0` se *happy_eyeballs_delay* não for especificado, e `1` se ele for.
- *sock*, se fornecido, deve ser um objeto `socket.socket` já existente, já conectado, para ser usado por transporte. Se *sock* é fornecido, *host*, *port*, *family*, *proto*, *flags*, *happy_eyeballs_delay*, *interleave* e *local_addr* não devem ser especificados.

  Nota

  The *sock* argument transfers ownership of the socket to the transport created. To close the socket, call the transport's `close()` method.

- *local_addr*, se fornecido, é uma tupla de `(local_host, local_port)` usada para se ligar ao soquete localmente. O *local_host* e a *local_port* são procurados usando `getaddrinfo()`, de forma similar para *host* e *port*.
- *ssl_handshake_timeout* é (para uma conexão TLS) o tempo em segundos para aguardar pelo encerramento do aperto de mão TLS, antes de abortar a conexão. `60.0` segundos se for `None` (valor padrão).
- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0` seconds if `None` (default).
- *all_errors* determines what exceptions are raised when a connection cannot be created. By default, only a single `Exception` is raised: the first exception if there is only one or all errors have same message, or a single `OSError` with the error messages combined. When `all_errors` is `True`, an `ExceptionGroup` will be raised containing all exceptions (even if there is only one).

Alterado na versão 3.5: Adicionado suporte para SSL/TLS na `ProactorEventLoop`.

Alterado na versão 3.6: The socket option *socket.TCP_NODELAY* is set by default for all TCP connections.

Alterado na versão 3.7: Adicionado o parâmetro *ssl_handshake_timeout*.

Alterado na versão 3.8: Adicionados os parâmetros *happy_eyeballs_delay* e *interleave*.

Happy Eyeballs Algorithm: Success with Dual-Stack Hosts. When a server's IPv4 path and protocol are working, but the server's IPv6 path and protocol are not working, a dual-stack client application experiences significant connection delay compared to an IPv4-only client. This is undesirable because it causes the dual-stack client to have a worse user experience. This document specifies requirements for algorithms that reduce this user-visible delay and provides an algorithm.

For more information: https://datatracker.ietf.org/doc/html/rfc6555

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.12: *all_errors* was added.

Ver também

A função `open_connection()` é uma API alternativa de alto nível. Ela retorna um par de (`StreamReader`, `StreamWriter`) que pode ser usado diretamente em código async/await.

async loop.**create_datagram_endpoint**(*protocol_factory*, *local_addr=None*, *remote_addr=None*, ***, *family=0*, *proto=0*, *flags=0*, *reuse_port=None*, *allow_broadcast=None*, *sock=None*)

Cria uma conexão de datagrama.

The socket family can be either `AF_INET`, `AF_INET6`, or `AF_UNIX`, depending on *host* (or the *family* argument, if provided).

The socket type will be `SOCK_DGRAM`.

protocol_factory deve ser algo chamável, retornando uma implementação de *protocolo*.

Uma tupla de `(transport, protocol)` é retornada em caso de sucesso.

Outros argumentos:

- *local_addr*, se fornecido, é uma tupla de `(local_host, local_port)` usada para ligar o soquete localmente. O *local_host* e a *local_port* são procurados usando `getaddrinfo()`.

  Nota

  On Windows, when using the proactor event loop with `local_addr=None`, an `OSError` with `errno.WSAEINVAL` will be raised when running it.

- *remote_addr*, se fornecido, é uma tupla de `(remote_host, remote_port)` usada para conectar o soquete a um endereço remoto. O *remote_host* e a *remote_port* são procurados usando `getaddrinfo()`.
- *family*, *proto*, *flags* são os endereços familiares, protocolo e flags opcionais a serem passados para `getaddrinfo()` para resolução do *host*. Se fornecido, esses devem ser todos inteiros do módulo de constantes `socket` correspondente.
- *reuse_port* tells the kernel to allow this endpoint to be bound to the same port as other existing endpoints are bound to, so long as they all set this flag when being created. This option is not supported on Windows and some Unixes. If the *socket.SO_REUSEPORT* constant is not defined then this capability is unsupported.
- *allow_broadcast* avisa o kernel para permitir que este endpoint envie mensagens para o endereço de broadcast.
- *sock* pode opcionalmente ser especificado em ordem para usar um objeto `socket.socket` pre-existente, já conectado, para ser usado pelo transporte. Se especificado, *local_addr* e *remote_addr* devem ser omitidos (devem ser `None`).

> **Nota**
>
> The *sock* argument transfers ownership of the socket to the transport created. To close the socket, call the transport's `close()` method.

Veja *protocolo UDP eco cliente* e *protocolo UDP eco servidor* para exemplos.

Alterado na versão 3.4.4: The *family*, *proto*, *flags*, *reuse_address*, *reuse_port*, *allow_broadcast*, and *sock* parameters were added.

Alterado na versão 3.8: Adicionado suporte para Windows.

Alterado na versão 3.8.1: The *reuse_address* parameter is no longer supported, as using *socket.SO_REUSEADDR* poses a significant security concern for UDP. Explicitly passing `reuse_address=True` will raise an exception.

Quando múltiplos processos com diferentes UIDs atribuem soquetes para um endereço de soquete UDP idêntico com `SO_REUSEADDR`, pacotes recebidos podem ser distribuídos aleatoriamente entre os soquetes.

For supported platforms, *reuse_port* can be used as a replacement for similar functionality. With *reuse_port*, *socket.SO_REUSEPORT* is used instead, which specifically prevents processes with differing UIDs from assigning sockets to the same socket address.

Alterado na versão 3.11: The *reuse_address* parameter, disabled since Python 3.8.1, 3.7.6 and 3.6.10, has been entirely removed.

async loop.**create_unix_connection**(*protocol_factory*, *path=None*, ***, *ssl=None*, *sock=None*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Cria uma conexão Unix.

The socket family will be `AF_UNIX`; socket type will be `SOCK_STREAM`.

Uma tupla de `(transport, protocol)` é retornada em caso de sucesso.

path é o nome de um soquete de domínio Unix e é obrigatório, a não ser que um parâmetro *sock* seja especificado. Soquetes Unix abstratos, `str`, `bytes`, e caminhos `Path` são suportados.

Veja a documentação do método `loop.create_connection()` para informações a respeito de argumentos para este método.

Disponibilidade: Unix.

Alterado na versão 3.7: Adicionado o parâmetro *ssl_handshake_timeout*. O parâmetro *path* agora pode ser um *path-like object*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Criando servidores de rede

async loop.**create_server**(*protocol_factory*, *host=None*, *port=None*, ***, *family=socket.AF_UNSPEC*, *flags=socket.AI_PASSIVE*, *sock=None*, *backlog=100*, *ssl=None*, *reuse_address=None*, *reuse_port=None*, *keep_alive=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *start_serving=True*)

Create a TCP server (socket type `SOCK_STREAM`) listening on *port* of the *host* address.

Retorna um objeto `Server`.

Argumentos:

- *protocol_factory* deve ser algo chamável, retornando uma implementação de *protocolo*.
- O parâmetro *host* pode ser definido para diversos tipos, o qual determina onde o servidor deve escutar:
  - Se *host* for uma string, o servidor TCP está vinculado a apenas uma interface de rede, especificada por *host*.

  - Se *host* é uma sequência de strings, o servidor TCP está vinculado a todas as interfaces de rede especificadas pela sequência.
  - Se *host* é uma string vazia ou `None`, todas as interfaces são presumidas e uma lista de múltiplos soquetes será retornada (muito provavelmente um para IPv4 e outro para IPv6).

- O parâmetro *port* pode ser definido para especificar qual porta o servidor deve escutar. Se `0` ou `None` (o padrão), uma porta aleatória disponível será selecionada (note que se *host* resolve para múltiplas interfaces de rede, uma porta aleatória diferente será selecionada para cada interface).
- *family* can be set to either *socket.AF_INET* or *AF_INET6* to force the socket to use IPv4 or IPv6. If not set, the *family* will be determined from host name (defaults to *AF_UNSPEC*).
- *flags* é uma máscara de bits para *getaddrinfo()*.
- *sock* pode opcionalmente ser especificado para usar um objeto soquete pré-existente. Se especificado, *host* e *port* não devem ser especificados.

> **Nota**
>
> The *sock* argument transfers ownership of the socket to the server created. To close the socket, call the server's *close()* method.

- *backlog* é o número máximo de conexões enfileiradas passadas para *listen()* (padrão é 100).
- *ssl* pode ser definido para uma instância de *SSLContext* para habilitar TLS sobre as conexões aceitas.
- *reuse_address* diz ao kernel para reusar um soquete local em estado `TIME_WAIT`, sem aguardar pela expiração natural do seu tempo limite. Se não especificado, será automaticamente definida como `True` no Unix.
- *reuse_port* diz ao kernel para permitir que este endpoint seja vinculado a mesma porta que outros endpoints existentes estão vinculados, contanto que todos eles definam este sinalizador quando forem criados. Esta opção não é suportada no Windows.
- *keep_alive* set to `True` keeps connections active by enabling the periodic transmission of messages.

Alterado na versão 3.13: Added the *keep_alive* parameter.

- *ssl_handshake_timeout* é (para um servidor TLS) o tempo em segundos para aguardar pelo aperto de mão TLS ser concluído, antes de abortar a conexão. `60.0` segundos se `None` (valor padrão).
- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0` seconds if `None` (default).
- Definir *start_serving* para `True` (o valor padrão) faz o servidor criado começar a aceitar conexões imediatamente. Quando definido para `False`, o usuário deve aguardar com *Server.start_serving()* ou *Server.serve_forever()* para fazer o servidor começar a aceitar conexões.

Alterado na versão 3.5: Adicionado suporte para SSL/TLS na *ProactorEventLoop*.

Alterado na versão 3.5.1: O parâmetro *host* pode ser uma sequência de strings.

Alterado na versão 3.6: Added *ssl_handshake_timeout* and *start_serving* parameters. The socket option *socket.TCP_NODELAY* is set by default for all TCP connections.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

> **Ver também**
>
> A função *start_server()* é uma API alternativa de alto nível que retorna um par de *StreamReader* e *StreamWriter* que pode ser usado em um código async/await.

async loop.**create_unix_server**(*protocol_factory*, *path=None*, ***, *sock=None*, *backlog=100*, *ssl=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *start_serving=True*, *cleanup_socket=True*)

Similar to `loop.create_server()` but works with the `AF_UNIX` socket family.

path é o nome de um soquete de domínio Unix, e é obrigatório, a não ser que um argumento *sock* seja fornecido. Soquetes Unix abstratos, `str`, `bytes`, e caminhos `Path` são suportados.

If *cleanup_socket* is true then the Unix socket will automatically be removed from the filesystem when the server is closed, unless the socket has been replaced after the server has been created.

Veja a documentação do método `loop.create_server()` para informações sobre argumentos para este método.

Disponibilidade: Unix.

Alterado na versão 3.7: Adicionados os parâmetros *ssl_handshake_timeout* e *start_serving*. O parâmetros *path* agora pode ser um objeto da classe `Path`.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.13: Added the *cleanup_socket* parameter.

async loop.**connect_accepted_socket**(*protocol_factory*, *sock*, ***, *ssl=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Envolve uma conexão já aceita em um par transporte/protocolo.

Este método pode ser usado por servidores que aceitam conexões fora do asyncio, mas que usam asyncio para manipulá-las.

Parâmetros:

- *protocol_factory* deve ser algo chamável, retornando uma implementação de *protocolo*.
- *sock* é um objeto soquete pré-existente retornado a partir de `socket.accept`.

> **Nota**
>
> The *sock* argument transfers ownership of the socket to the transport created. To close the socket, call the transport's `close()` method.

- *ssl* pode ser definido para um `SSLContext` para habilitar SSL sobre as conexões aceitas.
- *ssl_handshake_timeout* é (para uma conexão SSL) o tempo em segundos para aguardar pelo aperto de mão SSL ser concluído, antes de abortar a conexão. `60.0` segundos se `None` (valor padrão).
- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0` seconds if `None` (default).

Retorna um par `(transport, protocol)`.

Adicionado na versão 3.5.3.

Alterado na versão 3.7: Adicionado o parâmetro *ssl_handshake_timeout*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Transferindo arquivos

async loop.**sendfile**(*transport*, *file*, *offset=0*, *count=None*, ***, *fallback=True*)

Envia um *file* sobre um *transport*. Retorna o número total de bytes enviados.

O método usa `os.sendfile()` de alto desempenho, se disponível.

file deve ser um objeto arquivo regular aberto em modo binário.

offset indica a partir de onde deve iniciar a leitura do arquivo. Se especificado, *count* é o número total de bytes para transmitir, ao contrário de transmitir o arquivo até que EOF seja atingido. A posição do arquivo é sempre atualizada, mesmo quando este método levanta um erro, e `file.tell()` pode ser usado para obter o número atual de bytes enviados.

fallback definido para `True` faz o asyncio manualmente ler e enviar o arquivo quando a plataforma não suporta a chamada de sistema sendfile (por exemplo Windows ou soquete SSL no Unix).

Levanta `SendfileNotAvailableError` se o sistema não suporta a chamada de sistema *sendfile* e *fallback* é `False`.

Adicionado na versão 3.7.

Atualizando TLS

async loop.**start_tls**(*transport*, *protocol*, *sslcontext*, *, *server_side=False*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Atualiza um conexão baseada em transporte existente para TLS.

Create a TLS coder/decoder instance and insert it between the *transport* and the *protocol*. The coder/decoder implements both *transport*-facing protocol and *protocol*-facing transport.

Return the created two-interface instance. After *await*, the *protocol* must stop using the original *transport* and communicate with the returned object only because the coder caches *protocol*-side data and sporadically exchanges extra TLS session packets with *transport*.

In some situations (e.g. when the passed transport is already closing) this may return `None`.

Parâmetros:

- instâncias de *transport* e *protocol*, que métodos como `create_server()` e `create_connection()` retornam.
- *sslcontext*: uma instância configurada de `SSLContext`.
- *server_side* informe `True` quando uma conexão no lado do servidor estiver sendo atualizada (como a que é criada por `create_server()`).
- *server_hostname*: define ou substitui o nome do host no qual o servidor alvo do certificado será comparado.
- *ssl_handshake_timeout* é (para uma conexão TLS) o tempo em segundos para aguardar pelo encerramento do aperto de mão TLS, antes de abortar a conexão. `60.0` segundos se for `None` (valor padrão).
- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0` seconds if `None` (default).

Adicionado na versão 3.7.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Observando descritores de arquivo

loop.**add_reader**(*fd*, *callback*, **args*)

Começa a monitorar o descritor de arquivo *fd* para disponibilidade de leitura e invoca a *callback* com os argumentos especificados assim que *fd* esteja disponível para leitura.

Any preexisting callback registered for *fd* is cancelled and replaced by *callback*.

loop.**remove_reader**(*fd*)

Stop monitoring the *fd* file descriptor for read availability. Returns `True` if *fd* was previously being monitored for reads.

loop.**add_writer**(*fd*, *callback*, **args*)

Começa a monitorar o descritor de arquivo *fd* para disponibilidade de escrita e invoca a *callback* com os argumentos especificados assim que *fd* esteja disponível para escrita.

Any preexisting callback registered for *fd* is cancelled and replaced by *callback*.

Use *functools.partial() para passar argumentos nomeados* para a *callback*.

loop.**remove_writer**(*fd*)

Stop monitoring the *fd* file descriptor for write availability. Returns `True` if *fd* was previously being monitored for writes.

Veja também a seção de *Suporte a Plataformas* para algumas limitações desses métodos.

Trabalhando com objetos soquete diretamente

Em geral, implementações de protocolo que usam APIs baseadas em transporte, tais como *loop.create_connection()* e *loop.create_server()* são mais rápidas que implementações que trabalham com soquetes diretamente. Entretanto, existem alguns casos de uso quando o desempenho não é crítica, e trabalhar com objetos *socket* diretamente é mais conveniente.

async loop.**sock_recv**(*sock*, *nbytes*)

Recebe até *nbytes* do *sock*. Versão assíncrona de *socket.recv()*.

Retorna os dados recebidos como um objeto de bytes.

sock deve ser um soquete não bloqueante.

Alterado na versão 3.7: Apesar deste método sempre ter sido documentado como um método de corrotina, versões anteriores ao Python 3.7 retornavam um *Future*. Desde o Python 3.7 este é um método `async def`.

async loop.**sock_recv_into**(*sock*, *buf*)

Dados recebidos do *sock* no buffer *buf*. Modelado baseado no método bloqueante *socket.recv_into()*.

Retorna o número de bytes escritos no buffer.

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.7.

async loop.**sock_recvfrom**(*sock*, *bufsize*)

Receive a datagram of up to *bufsize* from *sock*. Asynchronous version of *socket.recvfrom()*.

Return a tuple of (received data, remote address).

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.11.

async loop.**sock_recvfrom_into**(*sock*, *buf*, *nbytes=0*)

Receive a datagram of up to *nbytes* from *sock* into *buf*. Asynchronous version of *socket.recvfrom_into()*.

Return a tuple of (number of bytes received, remote address).

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.11.

async loop.**sock_sendall**(*sock*, *data*)

Envia *data* para o soquete *sock*. Versão assíncrona de *socket.sendall()*.

Este método continua a enviar para o soquete até que todos os dados em *data* tenham sido enviados ou um erro ocorra. `None` é retornado em caso de sucesso. Ao ocorrer um erro, uma exceção é levantada. Adicionalmente, não existe nenhuma forma de determinar quantos dados, se algum, foram processados com sucesso pelo destinatário na conexão.

sock deve ser um soquete não bloqueante.

Alterado na versão 3.7: Apesar deste método sempre ter sido documentado como um método de corrotina, antes do Python 3.7 ele retornava um *Future*. Desde o Python 3.7, este é um método `async def`.

async `loop.`**sock_sendto**(*sock*, *data*, *address*)

Send a datagram from *sock* to *address*. Asynchronous version of *socket.sendto()*.

Return the number of bytes sent.

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.11.

async `loop.`**sock_connect**(*sock*, *address*)

Conecta o *sock* em um endereço *address* remoto.

Versão assíncrona de *socket.connect()*.

sock deve ser um soquete não bloqueante.

Alterado na versão 3.5.2: `address` não precisa mais ser resolvido. `sock_connect` irá tentar verificar se *address* já está resolvido chamando *socket.inet_pton()*. Se não estiver, *loop.getaddrinfo()* será usado para resolver *address*.

Ver também

loop.create_connection() e *asyncio.open_connection()*.

async `loop.`**sock_accept**(*sock*)

Aceita uma conexão. Modelado baseado no método bloqueante *socket.accept()*.

O soquete deve estar vinculado a um endereço e escutando por conexões. O valor de retorno é um par `(conn, address)` onde *conn* é um *novo* objeto de soquete usável para enviar e receber dados na conexão, e *address* é o endereço vinculado ao soquete no outro extremo da conexão.

sock deve ser um soquete não bloqueante.

Alterado na versão 3.7: Apesar deste método sempre ter sido documentado como um método de corrotina, antes do Python 3.7 ele retornava um *Future*. Desde o Python 3.7, este é um método `async def`.

Ver também

loop.create_server() e *start_server()*.

async `loop.`**sock_sendfile**(*sock*, *file*, *offset=0*, *count=None*, *, *fallback=True*)

Envia um arquivo usando *os.sendfile* de alto desempenho se possível. Retorna o número total de bytes enviados.

Versão assíncrona de *socket.sendfile()*.

sock deve ser um *socket socket.SOCK_STREAM* não bloqueante.

file deve ser um objeto arquivo regular aberto em modo binário.

offset indica a partir de onde deve iniciar a leitura do arquivo. Se especificado, *count* é o número total de bytes para transmitir, ao contrário de transmitir o arquivo até que EOF seja atingido. A posição do arquivo é sempre atualizada, mesmo quando este método levanta um erro, e *file.tell()* pode ser usado para obter o número atual de bytes enviados.

fallback, quando definido para `True`, faz asyncio ler e enviar manualmente o arquivo, quando a plataforma não suporta a chamada de sistema sendfile (por exemplo Windows ou soquete SSL no Unix).

Levanta *SendfileNotAvailableError* se o sistema não suporta chamadas de sistema *sendfile* e *fallback* é `False`.

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.7.

DNS

async loop.**getaddrinfo**(*host*, *port*, ***, *family=0*, *type=0*, *proto=0*, *flags=0*)

Versão assíncrona de `socket.getaddrinfo()`.

async loop.**getnameinfo**(*sockaddr*, *flags=0*)

Versão assíncrona de `socket.getnameinfo()`.

> **Nota**
>
> Both *getaddrinfo* and *getnameinfo* internally utilize their synchronous versions through the loop's default thread pool executor. When this executor is saturated, these methods may experience delays, which higher-level networking libraries may report as increased timeouts. To mitigate this, consider using a custom executor for other user tasks, or setting a default executor with a larger number of workers.

Alterado na versão 3.7: Ambos os métodos *getaddrinfo* e *getnameinfo* sempre foram documentados para retornar uma corrotina, mas antes do Python 3.7 eles estavam, na verdade, retornando objetos `asyncio.Future`. A partir do Python 3.7, ambos os métodos são corrotinas.

Trabalhando com encadeamentos

async loop.**connect_read_pipe**(*protocol_factory*, *pipe*)

Registra o extremo da leitura de um *pipe* no laço de eventos.

protocol_factory deve ser um chamável que retorne uma implementação do *protocolo asyncio*.

pipe é um *objeto arquivo ou similar*.

Retorna um par `(transport, protocol)`, onde *transport* suporta a interface `ReadTransport` e *protocol* é um objeto instanciado pelo *protocol_factory*.

Com o `SelectorEventLoop` do laço de eventos, o *pipe* é definido para modo não bloqueante.

async loop.**connect_write_pipe**(*protocol_factory*, *pipe*)

Registra o extremo de escrita do *pipe* no laço de eventos.

protocol_factory deve ser um chamável que retorne uma implementação do *protocolo asyncio*.

pipe é um *objeto arquivo ou similar*.

Retorna um part `(transport, protocol)`, onde *transport* suporta a interface `WriteTransport` e *protocol* é um objeto instanciado pelo *protocol_factory*.

Com o `SelectorEventLoop` do laço de eventos, o *pipe* é definido para modo não bloqueante.

> **Nota**
>
> `SelectorEventLoop` não suporta os métodos acima no Windows. Use `ProactorEventLoop` ao invés para Windows.

> **Ver também**
>
> Os métodos `loop.subprocess_exec()` e `loop.subprocess_shell()`.

Sinais Unix

`loop.`**`add_signal_handler`**(*signum*, *callback*, **args*)

Define *callback* como o tratador para o sinal *signum*.

A função de retorno será invocada pelo *loop*, juntamente com outras funções de retorno enfileiradas e corrotinas executáveis daquele laço de eventos. Ao contrário de tratadores de sinal registrados usando *signal.signal()*, uma função de retorno registrada com esta função tem autorização para interagir com o laço de eventos.

Levanta *ValueError* se o número do sinal é inválido ou impossível de capturar. Levanta *RuntimeError* se existe um problema definindo o tratador.

Use *functools.partial() para passar argumentos nomeados* para a *callback*.

Assim como *signal.signal()*, esta função deve ser invocada na thread principal.

`loop.`**`remove_signal_handler`**(*sig*)

Remove o tratador para o sinal *sig*.

Retorna `True` se o tratador de sinal foi removido, ou `False` se nenhum tratador foi definido para o sinal fornecido.

Disponibilidade: Unix.

Ver também

O módulo *signal*.

Executando código em conjuntos de threads ou processos

awaitable `loop.`**`run_in_executor`**(*executor*, *func*, **args*)

Providencia para a *func* ser chamada no executor especificado.

The *executor* argument should be an *concurrent.futures.Executor* instance. The default executor is used if *executor* is `None`. The default executor can be set by *loop.set_default_executor()*, otherwise, a *concurrent.futures.ThreadPoolExecutor* will be lazy-initialized and used by *run_in_executor()* if needed.

Exemplo:

```
import asyncio
import concurrent.futures

def blocking_io():
    # File operations (such as logging) can block the
    # event loop: run them in a thread pool.
    with open('/dev/urandom', 'rb') as f:
        return f.read(100)

def cpu_bound():
    # CPU-bound operations will block the event loop:
    # in general it is preferable to run them in a
    # process pool.
    return sum(i * i for i in range(10 ** 7))

async def main():
    loop = asyncio.get_running_loop()

    ## Options:
```

(continua na próxima página)

(continuação da página anterior)

```
    # 1. Run in the default loop's executor:
    result = await loop.run_in_executor(
        None, blocking_io)
    print('default thread pool', result)

    # 2. Run in a custom thread pool:
    with concurrent.futures.ThreadPoolExecutor() as pool:
        result = await loop.run_in_executor(
            pool, blocking_io)
        print('custom thread pool', result)

    # 3. Run in a custom process pool:
    with concurrent.futures.ProcessPoolExecutor() as pool:
        result = await loop.run_in_executor(
            pool, cpu_bound)
        print('custom process pool', result)

    # 4. Run in a custom interpreter pool:
    with concurrent.futures.InterpreterPoolExecutor() as pool:
        result = await loop.run_in_executor(
            pool, cpu_bound)
        print('custom interpreter pool', result)

if __name__ == '__main__':
    asyncio.run(main())
```

Note that the entry point guard (`if __name__ == '__main__'`) is required for option 3 due to the peculiarities of *multiprocessing*, which is used by *ProcessPoolExecutor*. See *Safe importing of main module*.

Este método retorna um objeto *asyncio.Future*.

Use *functools.partial() para passar argumentos nomeados* para *func*.

Alterado na versão 3.5.3: *loop.run_in_executor()* não configura mais o atributo `max_workers` do executor do conjunto de thread que ele cria, ao invés disso ele deixa para o executor do conjunto de thread (*ThreadPoolExecutor*) para setar o valor padrão.

`loop.`**`set_default_executor`**`(`*executor*`)`

Set *executor* as the default executor used by *run_in_executor()*. *executor* must be an instance of *ThreadPoolExecutor*, which includes *InterpreterPoolExecutor*.

Alterado na versão 3.11: *executor* must be an instance of *ThreadPoolExecutor*.

Tratando erros da API

Permite customizar como exceções são tratadas no laço de eventos.

`loop.`**`set_exception_handler`**`(`*handler*`)`

Define *handler* como o novo tratador de exceções do laço de eventos.

Se *handler* for `None`, o tratador de exceções padrão será definido. Caso contrário, *handler* deve ser um chamável com a assinatura combinando `(loop, context)`, onde `loop` é a referência para o laço de eventos ativo, e `context` é um objeto `dict` contendo os detalhes da exceção (veja a documentação *call_exception_handler()* para detalhes a respeito do contexto).

If the handler is called on behalf of a *Task* or *Handle*, it is run in the *contextvars.Context* of that task or callback handle.

Alterado na versão 3.12: The handler may be called in the *Context* of the task or handle where the exception originated.

loop.**get_exception_handler**()

Retorna o tratador de exceção atual, ou None se nenhum tratador de exceção customizado foi definido.

Adicionado na versão 3.5.2.

loop.**default_exception_handler**(*context*)

Tratador de exceção padrão.

Isso é chamado quando uma exceção ocorre e nenhum tratador de exceção foi definido. Isso pode ser chamado por um tratador de exceção customizado que quer passar adiante para o comportamento do tratador padrão.

parâmetro *context* tem o mesmo significado que em *call_exception_handler()*.

loop.**call_exception_handler**(*context*)

Chama o tratador de exceção do laço de eventos atual.

context é um objeto dict contendo as seguintes chaves (novas chaves podem ser introduzidas em versões futuras do Python):

- 'message': Mensagem de erro;
- 'exception' (opcional): Objeto Exception;
- 'future' (opcional): instância de *asyncio.Future*;
- 'task' (opcional): instância de *asyncio.Task*;
- 'handle' (opcional): instância de *asyncio.Handle*;
- 'protocol' (opcional): instância de *Protocol*;
- 'transport' (opcional): instância de *Transport*;
- 'socket' (opcional): instância de *socket.socket*;
- 'source_traceback' (optional): Traceback of the source;
- 'handle_traceback' (optional): Traceback of the handle;
- **'asyncgen' (opcional): Gerador assíncrono que causou** a exceção.

> **Nota**
>
> This method should not be overloaded in subclassed event loops. For custom exception handling, use the *set_exception_handler()* method.

Habilitando o modo de debug

loop.**get_debug**()

Obtém o modo de debug (*bool*) do laço de eventos.

O valor padrão é True se a variável de ambiente PYTHONASYNCIODEBUG estiver definida para uma string não vazia, False caso contrário.

loop.**set_debug**(*enabled:* bool)

Define o modo de debug do laço de eventos.

Alterado na versão 3.7: O novo *Modo de Desenvolvimento do Python* agora também pode ser usado para habilitar o modo de debug.

`loop.`**`slow_callback_duration`**

This attribute can be used to set the minimum execution duration in seconds that is considered "slow". When debug mode is enabled, "slow" callbacks are logged.

Default value is 100 milliseconds.

Ver também

O *modo de debug de asyncio*.

Executando Subprocessos

Métodos descritos nestas sub-seções são de baixo nível. Em código async/await regular, considere usar as funções convenientes de alto nível `asyncio.create_subprocess_shell()` e `asyncio.create_subprocess_exec()` ao invés.

Nota

No Windows, o laço de eventos padrão `ProactorEventLoop` oferece suporte pra subprocessos, enquanto `SelectorEventLoop`, não. Veja *Suporte para subprocessos no Windows* para detalhes.

async `loop.`**`subprocess_exec`**(*protocol_factory*, **args*, *stdin=subprocess.PIPE*, *stdout=subprocess.PIPE*, *stderr=subprocess.PIPE*, ***kwargs*)

Cria um subprocesso a partir de um ou mais argumentos de string especificados por *args*.

args deve ser uma lista de strings representada por:

- `str`;
- ou `bytes`, encodados na *codificação do sistema de arquivos*.

A primeira string especifica o programa executável, e as strings remanescentes especificam os argumentos. Juntas, argumentos em string formam o `argv` do programa.

Isto é similar a classe `subprocess.Popen` da biblioteca padrão ser chamada com `shell=False` e a lista de strings ser passada como o primeiro argumento; entretanto, onde `Popen` recebe apenas um argumento no qual é uma lista de strings, *subprocess_exec* recebe múltiplos argumentos string.

O *protocol_factory* deve ser um chamável que retorne uma subclasse da classe `asyncio.SubprocessProtocol`.

Outros parâmetros:

- *stdin* pode ser qualquer um destes:
  - a file-like object
  - an existing file descriptor (a positive integer), for example those created with `os.pipe()`
  - a constante `subprocess.PIPE` (padrão), a qual criará um novo encadeamento e conectar a ele,
  - o valor `None`, o qual fará o subprocesso herdar o descritor de arquivo deste processo
  - a constante `subprocess.DEVNULL`, a qual indica que o arquivo especial `os.devnull` será usado
- *stdout* pode ser qualquer um destes:
  - a file-like object
  - a constante `subprocess.PIPE` (padrão), a qual criará um novo encadeamento e conectar a ele,
  - o valor `None`, o qual fará o subprocesso herdar o descritor de arquivo deste processo
  - a constante `subprocess.DEVNULL`, a qual indica que o arquivo especial `os.devnull` será usado

- *stderr* pode ser qualquer um destes:
  - a file-like object
  - a constante *subprocess.PIPE* (padrão), a qual criará um novo encadeamento e conectar a ele,
  - o valor None, o qual fará o subprocesso herdar o descritor de arquivo deste processo
  - a constante *subprocess.DEVNULL*, a qual indica que o arquivo especial *os.devnull* será usado
  - a constante *subprocess.STDOUT*, a qual irá conectar o stream de erro padrão ao stream de saída padrão do processo
- Todos os outros argumentos nomeados são passados para *subprocess.Popen* sem interpretação, exceto *bufsize*, *universal_newlines*, *shell*, *text*, *encoding* e *errors*, os quais não devem ser especificados de forma alguma.

  A API de subprocesso asyncio não suporta decodificar os streams como texto. *bytes.decode()* pode ser usado para converter os bytes retornados do stream para texto.

If a file-like object passed as *stdin*, *stdout* or *stderr* represents a pipe, then the other side of this pipe should be registered with *connect_write_pipe()* or *connect_read_pipe()* for use with the event loop.

Veja o construtor da classe *subprocess.Popen* para documentação sobre outros argumentos.

Retorna um par (transport, protocol), onde *transport* conforma com a classe base *asyncio.SubprocessTransport* e *protocol* é um objeto instanciado pelo *protocol_factory*.

async loop.**subprocess_shell**(*protocol_factory*, *cmd*, *, *stdin=subprocess.PIPE*, *stdout=subprocess.PIPE*, *stderr=subprocess.PIPE*, ***kwargs*)

Cria um subprocesso a partir do *cmd*, o qual pode ser um *str* ou uma string de *bytes* codificada na *codificação do sistema de arquivos*, usando a sintaxe "shell" da plataforma.

Isto é similar a classe *subprocess.Popen* da biblioteca padrão sendo chamada com shell=True.

O argumento *protocol_factory* deve ser um chamável que retorna uma subclasse da classe *SubprocessProtocol*.

Veja *subprocess_exec()* para mais detalhes sobre os argumentos remanescentes.

Retorna um par (transport, protocol), onde *transport* conforma com a classe base *SubprocessTransport* e *protocol* é um objeto instanciado pelo *protocol_factory*.

Nota

É responsabilidade da aplicação garantir que todos os espaços em branco e caracteres especiais sejam tratados apropriadamente para evitar vulnerabilidades de injeção shell . A função *shlex.quote()* pode ser usada para escapar espaços em branco e caracteres especiais apropriadamente em strings que serão usadas para construir comandos shell.

Tratadores de função de retorno

class asyncio.**Handle**

Um objeto invólucro de função de retorno retornado por *loop.call_soon()*, *loop.call_soon_threadsafe()*.

get_context()

Return the *contextvars.Context* object associated with the handle.

Adicionado na versão 3.12.

cancel()

Cancela a função de retorno. Se a função de retorno já tiver sido cancelada ou executada, este método não tem efeito.

cancelled()

Retorna `True` se a função de retorno foi cancelada.

Adicionado na versão 3.7.

class asyncio.**TimerHandle**

Um objeto invólucro de função de retorno retornado por *loop.call_later()*, e *loop.call_at()*.

Esta classe é uma subclasse de *Handle*.

when()

Retorna o tempo de uma função de retorno agendada como *float* segundos.

O tempo é um timestamp absoluto, usando a mesma referência de tempo que *loop.time()*.

Adicionado na versão 3.7.

Objetos Server

Objetos Server são criados pelas funções *loop.create_server()*, *loop.create_unix_server()*, *start_server()*, e *start_unix_server()*.

Do not instantiate the *Server* class directly.

class asyncio.**Server**

Objetos *Server* são gerenciadores de contexto assíncronos. Quando usados em uma instrução `async with`, é garantido que o objeto Server está fechado e não está aceitando novas conexões quando a instrução `async with` estiver completa:

```
srv = await loop.create_server(...)

async with srv:
    # algum código

# Neste ponto, srv é fechado e não mais aceita novas conexões.
```

Alterado na versão 3.7: Objeto Server é um gerenciador de contexto assíncrono desde o Python 3.7.

Alterado na versão 3.11: This class was exposed publicly as `asyncio.Server` in Python 3.9.11, 3.10.3 and 3.11.

close()

Para de servir: fecha soquetes que estavam ouvindo e define o atributo *sockets* para `None`.

Os soquetes que representam conexões de clientes existentes que estão chegando são deixados em aberto.

The server is closed asynchronously; use the *wait_closed()* coroutine to wait until the server is closed (and no more connections are active).

close_clients()

Close all existing incoming client connections.

Calls *close()* on all associated transports.

close() should be called before *close_clients()* when closing the server to avoid races with new clients connecting.

Adicionado na versão 3.13.

abort_clients()

Close all existing incoming client connections immediately, without waiting for pending operations to complete.

Calls *abort()* on all associated transports.

close() should be called before *abort_clients()* when closing the server to avoid races with new clients connecting.

Adicionado na versão 3.13.

get_loop()

Retorna o laço de eventos associado com o objeto server.

Adicionado na versão 3.7.

async start_serving()

Começa a aceitar conexões.

This method is idempotent, so it can be called when the server is already serving.

O parâmetro somente-nomeado *start_serving* para *loop.create_server()* e *asyncio.start_server()* permite criar um objeto Server que não está aceitando conexões inicialmente. Neste caso `Server.start_serving()`, ou *Server.serve_forever()* podem ser usados para fazer o Server começar a aceitar conexões.

Adicionado na versão 3.7.

async serve_forever()

Começa a aceitar conexões até que a corrotina seja cancelada. Cancelamento da task `serve_forever` causa o fechamento do servidor.

Este método pode ser chamado se o servidor já estiver aceitando conexões. Apenas uma task `serve_forever` pode existir para cada objeto *Server*.

Exemplo:

```
async def client_connected(reader, writer):
    # Communicate with the client with
    # reader/writer streams.  For example:
    await reader.readline()

async def main(host, port):
    srv = await asyncio.start_server(
        client_connected, host, port)
    await srv.serve_forever()

asyncio.run(main('127.0.0.1', 0))
```

Adicionado na versão 3.7.

is_serving()

Retorna `True` se o servidor estiver aceitando novas conexões.

Adicionado na versão 3.7.

async wait_closed()

Wait until the *close()* method completes and all active connections have finished.

sockets

List of socket-like objects, `asyncio.trsock.TransportSocket`, which the server is listening on.

Alterado na versão 3.7: Antes do Python 3.7 `Server.sockets` era usado para retornar uma lista interna de soquetes do server diretamente. No uma cópia dessa lista é retornada.

Implementações do Laço de Eventos

asyncio vem com duas implementações de laço de eventos diferente: *SelectorEventLoop* e *ProactorEventLoop*.

By default asyncio is configured to use *EventLoop*.

class asyncio.**SelectorEventLoop**

A subclass of *AbstractEventLoop* based on the *selectors* module.

Usa o *seletor* mais eficiente disponível para a plataforma fornecida. Também é possível configurar manualmente a implementação exata do seletor a ser utilizada:

```
import asyncio
import selectors

async def main():
    ...

loop_factory = lambda: asyncio.SelectorEventLoop(selectors.SelectSelector())
asyncio.run(main(), loop_factory=loop_factory)
```

Disponibilidade: Unix, Windows.

class asyncio.**ProactorEventLoop**

A subclass of *AbstractEventLoop* for Windows that uses "I/O Completion Ports" (IOCP).

Disponibilidade: Windows.

Ver também

MSDN documentation on I/O Completion Ports.

class asyncio.**EventLoop**

An alias to the most efficient available subclass of *AbstractEventLoop* for the given platform.

It is an alias to *SelectorEventLoop* on Unix and *ProactorEventLoop* on Windows.

Adicionado na versão 3.13.

class asyncio.**AbstractEventLoop**

Classe base abstrata para laços de eventos compatíveis com asyncio.

The *Métodos do laço de eventos* section lists all methods that an alternative implementation of AbstractEventLoop should have defined.

Exemplos

Perceba que todos os exemplos nesta seção **propositalmente** mostram como usar as APIs de baixo nível do laço de eventos, tais como *loop.run_forever()* e *loop.call_soon()*. Aplicações asyncio modernas raramente precisam ser escritas desta forma; considere usar as funções de alto nível como *asyncio.run()*.

Hello World com call_soon()

Um exemplo usando o método *loop.call_soon()* para agendar uma função de retorno. A função de retorno exibe "Hello World" e então para o laço de eventos:

```
import asyncio

def hello_world(loop):
    """A callback to print 'Hello World' and stop the event loop"""
    print('Hello World')
    loop.stop()

loop = asyncio.new_event_loop()
```

(continua na próxima página)

(continuação da página anterior)

```
# Schedule a call to hello_world()
loop.call_soon(hello_world, loop)

# Blocking call interrupted by loop.stop()
try:
    loop.run_forever()
finally:
    loop.close()
```

Ver também

Um exemplo similar a *Hello World* criado com uma corrotina e a função `run()`.

Exibe a data atual com call_later()

Um exemplo de uma função de retorno mostrando a data atual a cada segundo. A função de retorno usa o método `loop.call_later()` para reagendar a si mesma depois de 5 segundos, e então para o laço de eventos:

```
import asyncio
import datetime

def display_date(end_time, loop):
    print(datetime.datetime.now())
    if (loop.time() + 1.0) < end_time:
        loop.call_later(1, display_date, end_time, loop)
    else:
        loop.stop()

loop = asyncio.new_event_loop()

# Schedule the first call to display_date()
end_time = loop.time() + 5.0
loop.call_soon(display_date, end_time, loop)

# Blocking call interrupted by loop.stop()
try:
    loop.run_forever()
finally:
    loop.close()
```

Ver também

Um exemplo similar a *data atual* criado com uma corrotina e a função `run()`.

Observa um descritor de arquivo por eventos de leitura

Aguarda até que um descritor de arquivo tenha recebido alguns dados usando o método `loop.add_reader()` e então fecha o laço de eventos:

```
import asyncio
from socket import socketpair

# Create a pair of connected file descriptors
```

(continua na próxima página)

(continuação da página anterior)

```
rsock, wsock = socketpair()

loop = asyncio.new_event_loop()

def reader():
    data = rsock.recv(100)
    print("Received:", data.decode())

    # We are done: unregister the file descriptor
    loop.remove_reader(rsock)

    # Stop the event loop
    loop.stop()

# Register the file descriptor for read event
loop.add_reader(rsock, reader)

# Simulate the reception of data from the network
loop.call_soon(wsock.send, 'abc'.encode())

try:
    # Run the event loop
    loop.run_forever()
finally:
    # We are done. Close sockets and the event loop.
    rsock.close()
    wsock.close()
    loop.close()
```

Ver também

- Um *exemplo* similar usando transportes, protocolos, e o método `loop.create_connection()`.
- Outro *exemplo* similar usando a função de alto nível `asyncio.open_connection()` e streams.

Define tratadores de sinais para SIGINT e SIGTERM

(Este exemplo de `signals` apenas funciona no Unix.)

Register handlers for signals `SIGINT` and `SIGTERM` using the `loop.add_signal_handler()` method:

```
import asyncio
import functools
import os
import signal

def ask_exit(signame, loop):
    print("got signal %s: exit" % signame)
    loop.stop()

async def main():
    loop = asyncio.get_running_loop()

    for signame in {'SIGINT', 'SIGTERM'}:
        loop.add_signal_handler(
            getattr(signal, signame),
```

(continua na próxima página)

(continuação da página anterior)

```
            functools.partial(ask_exit, signame, loop))

    await asyncio.sleep(3600)

print("Event loop running for 1 hour, press Ctrl+C to interrupt.")
print(f"pid {os.getpid()}: send SIGINT or SIGTERM to exit.")

asyncio.run(main())
```

19.1.10 Futuros

Source code: Lib/asyncio/futures.py, Lib/asyncio/base_futures.py

---

Future objects are used to bridge **low-level callback-based code** with high-level async/await code.

Future Functions

`asyncio.`**`isfuture`**(*obj*)

Return `True` if *obj* is either of:

- an instance of *`asyncio.Future`*,
- an instance of *`asyncio.Task`*,
- a Future-like object with a `_asyncio_future_blocking` attribute.

Adicionado na versão 3.5.

`asyncio.`**`ensure_future`**(*obj*, *, *loop=None*)

Retorna:

- *obj* argument as is, if *obj* is a *`Future`*, a *`Task`*, or a Future-like object (*`isfuture()`* is used for the test.)
- a *`Task`* object wrapping *obj*, if *obj* is a coroutine (*`iscoroutine()`* is used for the test); in this case the coroutine will be scheduled by `ensure_future()`.
- a *`Task`* object that would await on *obj*, if *obj* is an awaitable (*`inspect.isawaitable()`* is used for the test.)

If *obj* is neither of the above a *`TypeError`* is raised.

> **Importante**
>
> Save a reference to the result of this function, to avoid a task disappearing mid-execution.
>
> See also the *`create_task()`* function which is the preferred way for creating new tasks or use *`asyncio.TaskGroup`* which keeps reference to the task internally.

Alterado na versão 3.5.1: The function accepts any *awaitable* object.

Descontinuado desde a versão 3.10: Deprecation warning is emitted if *obj* is not a Future-like object and *loop* is not specified and there is no running event loop.

`asyncio.`**`wrap_future`**(*future*, *, *loop=None*)

Wrap a *`concurrent.futures.Future`* object in a *`asyncio.Future`* object.

Descontinuado desde a versão 3.10: Deprecation warning is emitted if *future* is not a Future-like object and *loop* is not specified and there is no running event loop.

Future Object

class asyncio.**Future**(*, *loop=None*)

A Future represents an eventual result of an asynchronous operation. Not thread-safe.

Future is an *awaitable* object. Coroutines can await on Future objects until they either have a result or an exception set, or until they are cancelled. A Future can be awaited multiple times and the result is same.

Typically Futures are used to enable low-level callback-based code (e.g. in protocols implemented using asyncio *transports*) to interoperate with high-level async/await code.

The rule of thumb is to never expose Future objects in user-facing APIs, and the recommended way to create a Future object is to call `loop.create_future()`. This way alternative event loop implementations can inject their own optimized implementations of a Future object.

Alterado na versão 3.7: Adicionado suporte para o módulo `contextvars`.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se *loop* não é especificado, e não existe nenhum laço de eventos em execução.

result()

Return the result of the Future.

If the Future is *done* and has a result set by the `set_result()` method, the result value is returned.

If the Future is *done* and has an exception set by the `set_exception()` method, this method raises the exception.

Se o futuro foi *cancelled*, este método levanta uma exceção `CancelledError`.

If the Future's result isn't yet available, this method raises an `InvalidStateError` exception.

set_result(*result*)

Mark the Future as *done* and set its result.

Raises an `InvalidStateError` error if the Future is already *done*.

set_exception(*exception*)

Mark the Future as *done* and set an exception.

Raises an `InvalidStateError` error if the Future is already *done*.

done()

Return `True` if the Future is *done*.

A Future is *done* if it was *cancelled* or if it has a result or an exception set with `set_result()` or `set_exception()` calls.

cancelled()

Return `True` if the Future was *cancelled*.

The method is usually used to check if a Future is not *cancelled* before setting a result or an exception for it:

```
if not fut.cancelled():
    fut.set_result(42)
```

add_done_callback(*callback*, *, *context=None*)

Add a callback to be run when the Future is *done*.

The *callback* is called with the Future object as its only argument.

If the Future is already *done* when this method is called, the callback is scheduled with `loop.call_soon()`.

Um argumento opcional somente-nomeado *context* permite especificar um `contextvars.Context` customizado para executar na *função de retorno*. O contexto atual é usado quando nenhum *context* é fornecido.

functools.partial() can be used to pass parameters to the callback, e.g.:

```
# Call 'print("Future:", fut)' when "fut" is done.
fut.add_done_callback(
    functools.partial(print, "Future:"))
```

Alterado na versão 3.7: O parâmetro somente-nomeado *context* foi adicionado. Veja **PEP 567** para mais detalhes.

remove_done_callback(*callback*)

Remove *callback* da lista de funções de retorno.

Returns the number of callbacks removed, which is typically 1, unless a callback was added more than once.

cancel(*msg=None*)

Cancel the Future and schedule callbacks.

If the Future is already *done* or *cancelled*, return `False`. Otherwise, change the Future's state to *cancelled*, schedule the callbacks, and return `True`.

Alterado na versão 3.9: Adicionado o parâmetro *msg*.

exception()

Return the exception that was set on this Future.

The exception (or `None` if no exception was set) is returned only if the Future is *done*.

Se o futuro foi *cancelled*, este método levanta uma exceção *CancelledError*.

If the Future isn't *done* yet, this method raises an *InvalidStateError* exception.

get_loop()

Return the event loop the Future object is bound to.

Adicionado na versão 3.7.

This example creates a Future object, creates and schedules an asynchronous Task to set result for the Future, and waits until the Future has a result:

```
async def set_after(fut, delay, value):
    # Sleep for *delay* seconds.
    await asyncio.sleep(delay)

    # Set *value* as a result of *fut* Future.
    fut.set_result(value)

async def main():
    # Get the current event loop.
    loop = asyncio.get_running_loop()

    # Create a new Future object.
    fut = loop.create_future()

    # Run "set_after()" coroutine in a parallel Task.
    # We are using the low-level "loop.create_task()" API here because
    # we already have a reference to the event loop at hand.
    # Otherwise we could have just used "asyncio.create_task()".
    loop.create_task(
        set_after(fut, 1, '... world'))

    print('hello ...')
```

(continua na próxima página)

(continuação da página anterior)

```
    # Wait until *fut* has a result (1 second) and print it.
    print(await fut)

asyncio.run(main())
```

Importante

The Future object was designed to mimic `concurrent.futures.Future`. Key differences include:

- unlike asyncio Futures, `concurrent.futures.Future` instances cannot be awaited.
- `asyncio.Future.result()` and `asyncio.Future.exception()` do not accept the *timeout* argument.
- `asyncio.Future.result()` and `asyncio.Future.exception()` raise an `InvalidStateError` exception when the Future is not *done*.
- Callbacks registered with `asyncio.Future.add_done_callback()` are not called immediately. They are scheduled with `loop.call_soon()` instead.
- asyncio Future is not compatible with the `concurrent.futures.wait()` and `concurrent.futures.as_completed()` functions.
- `asyncio.Future.cancel()` accepts an optional `msg` argument, but `concurrent.futures.Future.cancel()` does not.

19.1.11 Transports and Protocols

Prefácio

Transports and Protocols are used by the **low-level** event loop APIs such as `loop.create_connection()`. They use callback-based programming style and enable high-performance implementations of network or IPC protocols (e.g. HTTP).

Essentially, transports and protocols should only be used in libraries and frameworks and never in high-level asyncio applications.

This documentation page covers both *Transports* and *Protocols*.

Introdução

At the highest level, the transport is concerned with *how* bytes are transmitted, while the protocol determines *which* bytes to transmit (and to some extent when).

A different way of saying the same thing: a transport is an abstraction for a socket (or similar I/O endpoint) while a protocol is an abstraction for an application, from the transport's point of view.

Yet another view is the transport and protocol interfaces together define an abstract interface for using network I/O and interprocess I/O.

There is always a 1:1 relationship between transport and protocol objects: the protocol calls transport methods to send data, while the transport calls protocol methods to pass it data that has been received.

Most of connection oriented event loop methods (such as `loop.create_connection()`) usually accept a *protocol_factory* argument used to create a *Protocol* object for an accepted connection, represented by a *Transport* object. Such methods usually return a tuple of `(transport, protocol)`.

Conteúdo

Esta página de documentação contém as seguintes seções:

- The *Transports* section documents asyncio `BaseTransport`, `ReadTransport`, `WriteTransport`, `Transport`, `DatagramTransport`, and `SubprocessTransport` classes.
- The *Protocols* section documents asyncio `BaseProtocol`, `Protocol`, `BufferedProtocol`, `DatagramProtocol`, and `SubprocessProtocol` classes.
- The *Examples* section showcases how to work with transports, protocols, and low-level event loop APIs.

Transportes

Source code: Lib/asyncio/transports.py

Transports are classes provided by `asyncio` in order to abstract various kinds of communication channels.

Transport objects are always instantiated by an *asyncio event loop*.

asyncio implements transports for TCP, UDP, SSL, and subprocess pipes. The methods available on a transport depend on the transport's kind.

The transport classes are *not thread safe*.

Transports Hierarchy

class asyncio.**BaseTransport**

Base class for all transports. Contains methods that all asyncio transports share.

class asyncio.**WriteTransport**(*BaseTransport*)

A base transport for write-only connections.

Instances of the *WriteTransport* class are returned from the `loop.connect_write_pipe()` event loop method and are also used by subprocess-related methods like `loop.subprocess_exec()`.

class asyncio.**ReadTransport**(*BaseTransport*)

A base transport for read-only connections.

Instances of the *ReadTransport* class are returned from the `loop.connect_read_pipe()` event loop method and are also used by subprocess-related methods like `loop.subprocess_exec()`.

class asyncio.**Transport**(*WriteTransport*, *ReadTransport*)

Interface representing a bidirectional transport, such as a TCP connection.

The user does not instantiate a transport directly; they call a utility function, passing it a protocol factory and other information necessary to create the transport and protocol.

Instances of the *Transport* class are returned from or used by event loop methods like `loop.create_connection()`, `loop.create_unix_connection()`, `loop.create_server()`, `loop.sendfile()`, etc.

class asyncio.**DatagramTransport**(*BaseTransport*)

A transport for datagram (UDP) connections.

Instances of the *DatagramTransport* class are returned from the `loop.create_datagram_endpoint()` event loop method.

class asyncio.**SubprocessTransport**(*BaseTransport*)

An abstraction to represent a connection between a parent and its child OS process.

Instances of the *SubprocessTransport* class are returned from event loop methods `loop.subprocess_shell()` and `loop.subprocess_exec()`.

Base Transport

BaseTransport.**close**()

Fecha o transporte.

If the transport has a buffer for outgoing data, buffered data will be flushed asynchronously. No more data will be received. After all buffered data is flushed, the protocol's *protocol.connection_lost()* method will be called with *None* as its argument. The transport should not be used once it is closed.

BaseTransport.**is_closing**()

Retorna `True` se o transporte estiver fechando ou estiver fechado.

BaseTransport.**get_extra_info**(*name*, *default=None*)

Return information about the transport or underlying resources it uses.

name is a string representing the piece of transport-specific information to get.

default is the value to return if the information is not available, or if the transport does not support querying it with the given third-party event loop implementation or on the current platform.

For example, the following code attempts to get the underlying socket object of the transport:

```
sock = transport.get_extra_info('socket')
if sock is not None:
    print(sock.getsockopt(...))
```

Categories of information that can be queried on some transports:

- socket:
  - `'peername'`: the remote address to which the socket is connected, result of *socket.socket.getpeername()* (`None` on error)
  - `'socket'`: *socket.socket* instance
  - `'sockname'`: the socket's own address, result of *socket.socket.getsockname()*
- SSL socket:
  - `'compression'`: the compression algorithm being used as a string, or `None` if the connection isn't compressed; result of *ssl.SSLSocket.compression()*
  - `'cipher'`: a three-value tuple containing the name of the cipher being used, the version of the SSL protocol that defines its use, and the number of secret bits being used; result of *ssl.SSLSocket.cipher()*
  - `'peercert'`: peer certificate; result of *ssl.SSLSocket.getpeercert()*
  - `'sslcontext'`: *ssl.SSLContext* instance
  - `'ssl_object'`: *ssl.SSLObject* or *ssl.SSLSocket* instance
- pipe:
  - `'pipe'`: pipe object
- subprocess:
  - `'subprocess'`: *subprocess.Popen* instance

BaseTransport.**set_protocol**(*protocol*)

Define um novo protocolo.

Switching protocol should only be done when both protocols are documented to support the switch.

BaseTransport.**get_protocol**()

Retorna o protocolo atual.

Read-only Transports

ReadTransport.**is_reading**()

Return `True` if the transport is receiving new data.

Adicionado na versão 3.7.

ReadTransport.**pause_reading**()

Pause the receiving end of the transport. No data will be passed to the protocol's *protocol.data_received()* method until *resume_reading()* is called.

Alterado na versão 3.7: The method is idempotent, i.e. it can be called when the transport is already paused or closed.

ReadTransport.**resume_reading**()

Resume the receiving end. The protocol's *protocol.data_received()* method will be called once again if some data is available for reading.

Alterado na versão 3.7: The method is idempotent, i.e. it can be called when the transport is already reading.

Write-only Transports

WriteTransport.**abort**()

Close the transport immediately, without waiting for pending operations to complete. Buffered data will be lost. No more data will be received. The protocol's *protocol.connection_lost()* method will eventually be called with *None* as its argument.

WriteTransport.**can_write_eof**()

Return *True* if the transport supports *write_eof()*, *False* if not.

WriteTransport.**get_write_buffer_size**()

Return the current size of the output buffer used by the transport.

WriteTransport.**get_write_buffer_limits**()

Get the *high* and *low* watermarks for write flow control. Return a tuple `(low, high)` where *low* and *high* are positive number of bytes.

Use *set_write_buffer_limits()* to set the limits.

Adicionado na versão 3.4.2.

WriteTransport.**set_write_buffer_limits**(*high=None*, *low=None*)

Set the *high* and *low* watermarks for write flow control.

These two values (measured in number of bytes) control when the protocol's *protocol.pause_writing()* and *protocol.resume_writing()* methods are called. If specified, the low watermark must be less than or equal to the high watermark. Neither *high* nor *low* can be negative.

pause_writing() is called when the buffer size becomes greater than or equal to the *high* value. If writing has been paused, *resume_writing()* is called when the buffer size becomes less than or equal to the *low* value.

The defaults are implementation-specific. If only the high watermark is given, the low watermark defaults to an implementation-specific value less than or equal to the high watermark. Setting *high* to zero forces *low* to zero as well, and causes *pause_writing()* to be called whenever the buffer becomes non-empty. Setting *low* to zero causes *resume_writing()* to be called only once the buffer is empty. Use of zero for either limit is generally sub-optimal as it reduces opportunities for doing I/O and computation concurrently.

Use *get_write_buffer_limits()* to get the limits.

WriteTransport.**write**(*data*)

Write some *data* bytes to the transport.

This method does not block; it buffers the data and arranges for it to be sent out asynchronously.

`WriteTransport.`**`writelines`**(*list_of_data*)

Write a list (or any iterable) of data bytes to the transport. This is functionally equivalent to calling *write()* on each element yielded by the iterable, but may be implemented more efficiently.

`WriteTransport.`**`write_eof`**()

Close the write end of the transport after flushing all buffered data. Data may still be received.

This method can raise *NotImplementedError* if the transport (e.g. SSL) doesn't support half-closed connections.

Transportes de datagrama

`DatagramTransport.`**`sendto`**(*data*, *addr=None*)

Send the *data* bytes to the remote peer given by *addr* (a transport-dependent target address). If *addr* is *None*, the data is sent to the target address given on transport creation.

This method does not block; it buffers the data and arranges for it to be sent out asynchronously.

Alterado na versão 3.13: This method can be called with an empty bytes object to send a zero-length datagram. The buffer size calculation used for flow control is also updated to account for the datagram header.

`DatagramTransport.`**`abort`**()

Close the transport immediately, without waiting for pending operations to complete. Buffered data will be lost. No more data will be received. The protocol's *protocol.connection_lost()* method will eventually be called with *None* as its argument.

Transportes de Subprocesso

`SubprocessTransport.`**`get_pid`**()

Return the subprocess process id as an integer.

`SubprocessTransport.`**`get_pipe_transport`**(*fd*)

Return the transport for the communication pipe corresponding to the integer file descriptor *fd*:

- `0`: writable streaming transport of the standard input (*stdin*), or *None* if the subprocess was not created with `stdin=PIPE`
- `1`: readable streaming transport of the standard output (*stdout*), or *None* if the subprocess was not created with `stdout=PIPE`
- `2`: readable streaming transport of the standard error (*stderr*), or *None* if the subprocess was not created with `stderr=PIPE`
- other *fd*: *None*

`SubprocessTransport.`**`get_returncode`**()

Return the subprocess return code as an integer or *None* if it hasn't returned, which is similar to the *subprocess.Popen.returncode* attribute.

`SubprocessTransport.`**`kill`**()

Mata o subprocesso.

On POSIX systems, the function sends SIGKILL to the subprocess. On Windows, this method is an alias for *terminate()*.

See also *subprocess.Popen.kill()*.

`SubprocessTransport.`**`send_signal`**(*signal*)

Send the *signal* number to the subprocess, as in *subprocess.Popen.send_signal()*.

SubprocessTransport.**terminate**()

Interrompe o subprocesso.

On POSIX systems, this method sends *SIGTERM* to the subprocess. On Windows, the Windows API function TerminateProcess() is called to stop the subprocess.

See also *subprocess.Popen.terminate()*.

SubprocessTransport.**close**()

Kill the subprocess by calling the *kill()* method.

If the subprocess hasn't returned yet, and close transports of *stdin*, *stdout*, and *stderr* pipes.

Protocolos

Source code: Lib/asyncio/protocols.py

---

asyncio provides a set of abstract base classes that should be used to implement network protocols. Those classes are meant to be used together with *transports*.

Subclasses of abstract base protocol classes may implement some or all methods. All these methods are callbacks: they are called by transports on certain events, for example when some data is received. A base protocol method should be called by the corresponding transport.

Protocolos de Base

class asyncio.**BaseProtocol**

Base protocol with methods that all protocols share.

class asyncio.**Protocol**(*BaseProtocol*)

The base class for implementing streaming protocols (TCP, Unix sockets, etc).

class asyncio.**BufferedProtocol**(*BaseProtocol*)

A base class for implementing streaming protocols with manual control of the receive buffer.

class asyncio.**DatagramProtocol**(*BaseProtocol*)

The base class for implementing datagram (UDP) protocols.

class asyncio.**SubprocessProtocol**(*BaseProtocol*)

The base class for implementing protocols communicating with child processes (unidirectional pipes).

Base Protocol

All asyncio protocols can implement Base Protocol callbacks.

Connection Callbacks

Connection callbacks are called on all protocols, exactly once per a successful connection. All other protocol callbacks can only be called between those two methods.

BaseProtocol.**connection_made**(*transport*)

Chamado quando uma conexão é estabelecida.

The *transport* argument is the transport representing the connection. The protocol is responsible for storing the reference to its transport.

BaseProtocol.**connection_lost**(*exc*)

Chamado quanto a conexão é perdida ou fechada.

The argument is either an exception object or *None*. The latter means a regular EOF is received, or the connection was aborted or closed by this side of the connection.

Flow Control Callbacks

Flow control callbacks can be called by transports to pause or resume writing performed by the protocol.

See the documentation of the *set_write_buffer_limits()* method for more details.

BaseProtocol.**pause_writing**()
: Called when the transport's buffer goes over the high watermark.

BaseProtocol.**resume_writing**()
: Called when the transport's buffer drains below the low watermark.

If the buffer size equals the high watermark, *pause_writing()* is not called: the buffer size must go strictly over.

Conversely, *resume_writing()* is called when the buffer size is equal or lower than the low watermark. These end conditions are important to ensure that things go as expected when either mark is zero.

Streaming Protocols

Event methods, such as *loop.create_server()*, *loop.create_unix_server()*, *loop.create_connection()*, *loop.create_unix_connection()*, *loop.connect_accepted_socket()*, *loop.connect_read_pipe()*, and *loop.connect_write_pipe()* accept factories that return streaming protocols.

Protocol.**data_received**(*data*)
: Called when some data is received. *data* is a non-empty bytes object containing the incoming data.

  Whether the data is buffered, chunked or reassembled depends on the transport. In general, you shouldn't rely on specific semantics and instead make your parsing generic and flexible. However, data is always received in the correct order.

  The method can be called an arbitrary number of times while a connection is open.

  However, *protocol.eof_received()* is called at most once. Once `eof_received()` is called, `data_received()` is not called anymore.

Protocol.**eof_received**()
: Called when the other end signals it won't send any more data (for example by calling *transport.write_eof()*, if the other end also uses asyncio).

  This method may return a false value (including `None`), in which case the transport will close itself. Conversely, if this method returns a true value, the protocol used determines whether to close the transport. Since the default implementation returns `None`, it implicitly closes the connection.

  Some transports, including SSL, don't support half-closed connections, in which case returning true from this method will result in the connection being closed.

State machine:

```
start -> connection_made
    [-> data_received]*
    [-> eof_received]?
-> connection_lost -> end
```

Protocolos de Streaming Bufferizados

Adicionado na versão 3.7.

Buffered Protocols can be used with any event loop method that supports *Streaming Protocols*.

`BufferedProtocol` implementations allow explicit manual allocation and control of the receive buffer. Event loops can then use the buffer provided by the protocol to avoid unnecessary data copies. This can result in noticeable performance improvement for protocols that receive big amounts of data. Sophisticated protocol implementations can significantly reduce the number of buffer allocations.

The following callbacks are called on *BufferedProtocol* instances:

BufferedProtocol.**get_buffer**(*sizehint*)

Chamada para alocar um novo buffer para recebimento.

sizehint is the recommended minimum size for the returned buffer. It is acceptable to return smaller or larger buffers than what *sizehint* suggests. When set to -1, the buffer size can be arbitrary. It is an error to return a buffer with a zero size.

`get_buffer()` must return an object implementing the buffer protocol.

BufferedProtocol.**buffer_updated**(*nbytes*)

Chamado quando o buffer foi atualizado com os dados recebidos.

nbytes is the total number of bytes that were written to the buffer.

BufferedProtocol.**eof_received**()

See the documentation of the *protocol.eof_received()* method.

get_buffer() can be called an arbitrary number of times during a connection. However, *protocol.eof_received()* is called at most once and, if called, *get_buffer()* and *buffer_updated()* won't be called after it.

State machine:

```
start -> connection_made
    [-> get_buffer
        [-> buffer_updated]?
    ]*
    [-> eof_received]?
-> connection_lost -> end
```

Protocolos de Datagramas

Datagram Protocol instances should be constructed by protocol factories passed to the *loop.create_datagram_endpoint()* method.

DatagramProtocol.**datagram_received**(*data*, *addr*)

Called when a datagram is received. *data* is a bytes object containing the incoming data. *addr* is the address of the peer sending the data; the exact format depends on the transport.

DatagramProtocol.**error_received**(*exc*)

Called when a previous send or receive operation raises an *OSError*. *exc* is the *OSError* instance.

This method is called in rare conditions, when the transport (e.g. UDP) detects that a datagram could not be delivered to its recipient. In many conditions though, undeliverable datagrams will be silently dropped.

Nota

On BSD systems (macOS, FreeBSD, etc.) flow control is not supported for datagram protocols, because there is no reliable way to detect send failures caused by writing too many packets.

The socket always appears 'ready' and excess packets are dropped. An *OSError* with `errno` set to *errno.ENOBUFS* may or may not be raised; if it is raised, it will be reported to *DatagramProtocol.error_received()* but otherwise ignored.

Protocolos de Subprocesso

Subprocess Protocol instances should be constructed by protocol factories passed to the *loop.subprocess_exec()* and *loop.subprocess_shell()* methods.

SubprocessProtocol.**pipe_data_received**(*fd*, *data*)

Called when the child process writes data into its stdout or stderr pipe.

fd is the integer file descriptor of the pipe.

data is a non-empty bytes object containing the received data.

SubprocessProtocol.**pipe_connection_lost**(*fd*, *exc*)

Chamado quando um dos encadeamentos comunicando com o processo filho é fechado.

fd is the integer file descriptor that was closed.

SubprocessProtocol.**process_exited**()

Chamado quando o processo filho encerrou.

It can be called before *pipe_data_received()* and *pipe_connection_lost()* methods.

Exemplos

TCP Echo Server

Create a TCP echo server using the *loop.create_server()* method, send back received data, and close the connection:

```
import asyncio


class EchoServerProtocol(asyncio.Protocol):
    def connection_made(self, transport):
        peername = transport.get_extra_info('peername')
        print('Connection from {}'.format(peername))
        self.transport = transport

    def data_received(self, data):
        message = data.decode()
        print('Data received: {!r}'.format(message))

        print('Send: {!r}'.format(message))
        self.transport.write(data)

        print('Close the client socket')
        self.transport.close()


async def main():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    server = await loop.create_server(
        EchoServerProtocol,
        '127.0.0.1', 8888)

    async with server:
        await server.serve_forever()


asyncio.run(main())
```

Ver também

The *TCP echo server using streams* example uses the high-level `asyncio.start_server()` function.

TCP Echo Client

A TCP echo client using the `loop.create_connection()` method, sends data, and waits until the connection is closed:

```
import asyncio


class EchoClientProtocol(asyncio.Protocol):
    def __init__(self, message, on_con_lost):
        self.message = message
        self.on_con_lost = on_con_lost

    def connection_made(self, transport):
        transport.write(self.message.encode())
        print('Data sent: {!r}'.format(self.message))

    def data_received(self, data):
        print('Data received: {!r}'.format(data.decode()))

    def connection_lost(self, exc):
        print('The server closed the connection')
        self.on_con_lost.set_result(True)


async def main():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    on_con_lost = loop.create_future()
    message = 'Hello World!'

    transport, protocol = await loop.create_connection(
        lambda: EchoClientProtocol(message, on_con_lost),
        '127.0.0.1', 8888)

    # Wait until the protocol signals that the connection
    # is lost and close the transport.
    try:
        await on_con_lost
    finally:
        transport.close()


asyncio.run(main())
```

Ver também

The *TCP echo client using streams* example uses the high-level `asyncio.open_connection()` function.

UDP Echo Server

A UDP echo server, using the *loop.create_datagram_endpoint()* method, sends back received data:

```
import asyncio


class EchoServerProtocol:
    def connection_made(self, transport):
        self.transport = transport

    def datagram_received(self, data, addr):
        message = data.decode()
        print('Received %r from %s' % (message, addr))
        print('Send %r to %s' % (message, addr))
        self.transport.sendto(data, addr)


async def main():
    print("Starting UDP server")

    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    # One protocol instance will be created to serve all
    # client requests.
    transport, protocol = await loop.create_datagram_endpoint(
        EchoServerProtocol,
        local_addr=('127.0.0.1', 9999))

    try:
        await asyncio.sleep(3600)  # Serve for 1 hour.
    finally:
        transport.close()


asyncio.run(main())
```

UDP Echo Client

A UDP echo client, using the *loop.create_datagram_endpoint()* method, sends data and closes the transport when it receives the answer:

```
import asyncio


class EchoClientProtocol:
    def __init__(self, message, on_con_lost):
        self.message = message
        self.on_con_lost = on_con_lost
        self.transport = None

    def connection_made(self, transport):
        self.transport = transport
        print('Send:', self.message)
        self.transport.sendto(self.message.encode())
```

(continua na próxima página)

(continuação da página anterior)

```
    def datagram_received(self, data, addr):
        print("Received:", data.decode())

        print("Close the socket")
        self.transport.close()

    def error_received(self, exc):
        print('Error received:', exc)

    def connection_lost(self, exc):
        print("Connection closed")
        self.on_con_lost.set_result(True)


async def main():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    on_con_lost = loop.create_future()
    message = "Hello World!"

    transport, protocol = await loop.create_datagram_endpoint(
        lambda: EchoClientProtocol(message, on_con_lost),
        remote_addr=('127.0.0.1', 9999))

    try:
        await on_con_lost
    finally:
        transport.close()


asyncio.run(main())
```

Connecting Existing Sockets

Wait until a socket receives data using the *loop.create_connection()* method with a protocol:

```
import asyncio
import socket


class MyProtocol(asyncio.Protocol):

    def __init__(self, on_con_lost):
        self.transport = None
        self.on_con_lost = on_con_lost

    def connection_made(self, transport):
        self.transport = transport

    def data_received(self, data):
        print("Received:", data.decode())

        # We are done: close the transport;
```

(continua na próxima página)

(continuação da página anterior)

```
        # connection_lost() will be called automatically.
        self.transport.close()

    def connection_lost(self, exc):
        # The socket has been closed
        self.on_con_lost.set_result(True)


async def main():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()
    on_con_lost = loop.create_future()

    # Create a pair of connected sockets
    rsock, wsock = socket.socketpair()

    # Register the socket to wait for data.
    transport, protocol = await loop.create_connection(
        lambda: MyProtocol(on_con_lost), sock=rsock)

    # Simulate the reception of data from the network.
    loop.call_soon(wsock.send, 'abc'.encode())

    try:
        await protocol.on_con_lost
    finally:
        transport.close()
        wsock.close()

asyncio.run(main())
```

Ver também

The *watch a file descriptor for read events* example uses the low-level `loop.add_reader()` method to register an FD.

The *register an open socket to wait for data using streams* example uses high-level streams created by the `open_connection()` function in a coroutine.

loop.subprocess_exec() and SubprocessProtocol

An example of a subprocess protocol used to get the output of a subprocess and to wait for the subprocess exit.

The subprocess is created by the `loop.subprocess_exec()` method:

```
import asyncio
import sys

class DateProtocol(asyncio.SubprocessProtocol):
    def __init__(self, exit_future):
        self.exit_future = exit_future
        self.output = bytearray()
        self.pipe_closed = False
        self.exited = False
```

(continua na próxima página)

(continuação da página anterior)

```
    def pipe_connection_lost(self, fd, exc):
        self.pipe_closed = True
        self.check_for_exit()

    def pipe_data_received(self, fd, data):
        self.output.extend(data)

    def process_exited(self):
        self.exited = True
        # process_exited() method can be called before
        # pipe_connection_lost() method: wait until both methods are
        # called.
        self.check_for_exit()

    def check_for_exit(self):
        if self.pipe_closed and self.exited:
            self.exit_future.set_result(True)

async def get_date():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    code = 'import datetime; print(datetime.datetime.now())'
    exit_future = asyncio.Future(loop=loop)

    # Create the subprocess controlled by DateProtocol;
    # redirect the standard output into a pipe.
    transport, protocol = await loop.subprocess_exec(
        lambda: DateProtocol(exit_future),
        sys.executable, '-c', code,
        stdin=None, stderr=None)

    # Wait for the subprocess exit using the process_exited()
    # method of the protocol.
    await exit_future

    # Close the stdout pipe.
    transport.close()

    # Read the output which was collected by the
    # pipe_data_received() method of the protocol.
    data = bytes(protocol.output)
    return data.decode('ascii').rstrip()

date = asyncio.run(get_date())
print(f"Current date: {date}")
```

See also the *same example* written using high-level APIs.

19.1.12 Políticas

Aviso

Policies are deprecated and will be removed in Python 3.16. Users are encouraged to use the `asyncio.run()`

function or the *asyncio.Runner* with *loop_factory* to use the desired loop implementation.

An event loop policy is a global object used to get and set the current *event loop*, as well as create new event loops. The default policy can be *replaced* with *built-in alternatives* to use different event loop implementations, or substituted by a *custom policy* that can override these behaviors.

The *policy object* gets and sets a separate event loop per *context*. This is per-thread by default, though custom policies could define *context* differently.

Custom event loop policies can control the behavior of *get_event_loop()*, *set_event_loop()*, and *new_event_loop()*.

Policy objects should implement the APIs defined in the *AbstractEventLoopPolicy* abstract base class.

Getting and Setting the Policy

The following functions can be used to get and set the policy for the current process:

asyncio.**get_event_loop_policy**()

Retorna a política de todo o processo atual.

Descontinuado desde a versão 3.14: The *get_event_loop_policy()* function is deprecated and will be removed in Python 3.16.

asyncio.**set_event_loop_policy**(*policy*)

Set the current process-wide policy to *policy*.

If *policy* is set to None, the default policy is restored.

Descontinuado desde a versão 3.14: The *set_event_loop_policy()* function is deprecated and will be removed in Python 3.16.

Policy Objects

The abstract event loop policy base class is defined as follows:

class asyncio.**AbstractEventLoopPolicy**

An abstract base class for asyncio policies.

get_event_loop()

Obtém o laço de eventos para o contexto atual.

Return an event loop object implementing the *AbstractEventLoop* interface.

This method should never return None.

Alterado na versão 3.6.

set_event_loop(*loop*)

Define o laço de eventos do contexto atual como *loop*.

new_event_loop()

Cria e retorna um novo objeto de laço de eventos.

This method should never return None.

Descontinuado desde a versão 3.14: The *AbstractEventLoopPolicy* class is deprecated and will be removed in Python 3.16.

asyncio ships with the following built-in policies:

class asyncio.**DefaultEventLoopPolicy**

The default asyncio policy. Uses *SelectorEventLoop* on Unix and *ProactorEventLoop* on Windows.

There is no need to install the default policy manually. asyncio is configured to use the default policy automatically.

Alterado na versão 3.8: On Windows, *ProactorEventLoop* is now used by default.

Alterado na versão 3.14: The *get_event_loop()* method of the default asyncio policy now raises a *RuntimeError* if there is no set event loop.

Descontinuado desde a versão 3.14: The *DefaultEventLoopPolicy* class is deprecated and will be removed in Python 3.16.

class asyncio.**WindowsSelectorEventLoopPolicy**

An alternative event loop policy that uses the *SelectorEventLoop* event loop implementation.

Disponibilidade: Windows.

Descontinuado desde a versão 3.14: The *WindowsSelectorEventLoopPolicy* class is deprecated and will be removed in Python 3.16.

class asyncio.**WindowsProactorEventLoopPolicy**

An alternative event loop policy that uses the *ProactorEventLoop* event loop implementation.

Disponibilidade: Windows.

Descontinuado desde a versão 3.14: The *WindowsProactorEventLoopPolicy* class is deprecated and will be removed in Python 3.16.

Custom Policies

To implement a new event loop policy, it is recommended to subclass *DefaultEventLoopPolicy* and override the methods for which custom behavior is wanted, e.g.:

```
class MyEventLoopPolicy(asyncio.DefaultEventLoopPolicy):

    def get_event_loop(self):
        """Get the event loop.

        This may be None or an instance of EventLoop.
        """
        loop = super().get_event_loop()
        # Do something with loop ...
        return loop

asyncio.set_event_loop_policy(MyEventLoopPolicy())
```

19.1.13 Suporte a plataformas

The *asyncio* module is designed to be portable, but some platforms have subtle differences and limitations due to the platforms' underlying architecture and capabilities.

Todas as plataformas

- *loop.add_reader()* and *loop.add_writer()* cannot be used to monitor file I/O.

Windows

Source code: Lib/asyncio/proactor_events.py, Lib/asyncio/windows_events.py, Lib/asyncio/windows_utils.py

---

Alterado na versão 3.8: On Windows, *ProactorEventLoop* is now the default event loop.

All event loops on Windows do not support the following methods:

- `loop.create_unix_connection()` and `loop.create_unix_server()` are not supported. The `socket.AF_UNIX` socket family is specific to Unix.
- `loop.add_signal_handler()` and `loop.remove_signal_handler()` are not supported.

`SelectorEventLoop` has the following limitations:

- `SelectSelector` is used to wait on socket events: it supports sockets and is limited to 512 sockets.
- `loop.add_reader()` and `loop.add_writer()` only accept socket handles (e.g. pipe file descriptors are not supported).
- Pipes are not supported, so the `loop.connect_read_pipe()` and `loop.connect_write_pipe()` methods are not implemented.
- *Subprocesses* are not supported, i.e. `loop.subprocess_exec()` and `loop.subprocess_shell()` methods are not implemented.

`ProactorEventLoop` has the following limitations:

- The `loop.add_reader()` and `loop.add_writer()` methods are not supported.

The resolution of the monotonic clock on Windows is usually around 15.6 milliseconds. The best resolution is 0.5 milliseconds. The resolution depends on the hardware (availability of HPET) and on the Windows configuration.

Suporte para subprocesso no Windows

On Windows, the default event loop `ProactorEventLoop` supports subprocesses, whereas `SelectorEventLoop` does not.

macOS

Modern macOS versions are fully supported.

macOS <= 10.8

On macOS 10.6, 10.7 and 10.8, the default event loop uses `selectors.KqueueSelector`, which does not support character devices on these versions. The `SelectorEventLoop` can be manually configured to use `SelectSelector` or `PollSelector` to support character devices on these older versions of macOS. Example:

```
import asyncio
import selectors

selector = selectors.SelectSelector()
loop = asyncio.SelectorEventLoop(selector)
asyncio.set_event_loop(loop)
```

19.1.14 Extensão

A direção principal para estender o módulo `asyncio` é escrever classes de *laços de eventos* personalizados. O Asyncio tem funções auxiliares que podem ser usadas para simplificar essa tarefa.

Nota

Terceiros devem reutilizar o código assíncrono existente com cuidado. Uma versão nova de Python pode quebrar a compatibilidade com versões anteriores da parte *interna* da API.

Escrevendo um laço de eventos personalizado

asyncio.AbstractEventLoop declara muitos métodos. Implementar todos eles do zero é um trabalho tedioso.

Um laço de eventos pode herdar várias implementações de métodos da classe `asyncio.BaseEventLoop`.

Por sua vez, o sucessor deve implementar um conjunto de métodos *privados* declarados, porém não implementados, em `asyncio.BaseEventLoop`.

Por exemplo, `loop.create_connection()``verifica os argumentos, resolve endereços DNS, e chama a função ``loop._make_socket_transport()`, que deve ser implementada pela classe herdada. O método"_make_socket_transport()" não está documentado, e é considerado parte de uma API *interna*.

Construtores privados de Future e Task

As classes *asyncio.Future* e *asyncio.Task* nunca deverão ser criadas diretamente. Por favor, as substitua pelas *factories* correspondentes: *loop.create_future()* e *loop.create_task()* ou *asyncio.create_task()*.

Porém, *laços de eventos* de terceiros podem *reutilizar* as implementações embutidas de Future e Task, obtendo um código complexo e altamente otimizado.

Por isso, segue a listagem de construtores *privados*:

`Future.`**`__init__`**(*, *loop=None*)

Cria uma instância de Future embutida.

loop é uma instância opcional do laço de eventos.

`Task.`**`__init__`**(*coro*, *, *loop=None*, *name=None*, *context=None*)

Cria uma instância embutida de Task.

loop é uma instância opcional do laço de eventos. Os outros argumentos estão descritos em *loop.create_task()*.

Alterado na versão 3.11: o argumento *context* foi adicionado

Suporte ao ciclo de vida de tarefas

Implementações de Task de terceiros devem chamar as seguintes funções para manter uma tarefa visível para *asyncio.all_tasks()* e *asyncio.current_task()*:

`asyncio.`**`_register_task`**(*task*)

Registra uma nova *task* como gerenciada pelo *asyncio*.

Chame esta função no construtor de uma tarefa.

`asyncio.`**`_unregister_task`**(*task*)

Cancela o registro de *task* nas estruturas internas do *asyncio*.

Chame esta função quando a tarefa estiver terminando.

`asyncio.`**`_enter_task`**(*loop*, *task*)

Troca a tarefa atual para o argumento *task*.

Chame esta função logo antes de executar uma porção da *corrotina* embutida (`coroutine.send()` ou `coroutine.throw()`).

`asyncio.`**`_leave_task`**(*loop*, *task*)

Troca a tarefa atual de *task* para `None`.

Chame esta função logo após executar `coroutine.send()` ou `coroutine.throw()`.

19.1.15 Índice da API de alto nível

Esta página lista todas as APIs asyncio de alto nível habilitadas por async/await.

Tarefas

Utilitários para executar programas asyncio, criar Tarefas, e esperar por múltiplas coisas com tempos limites.

run()	Cria um laço de eventos, roda uma corrotina, fecha o laço.
Runner	Um gerenciador de contexto que simplifica várias chamadas assíncronas de funções.
Task	Objeto Task.
TaskGroup	Um gerenciador de contexto que mantém um grupo de tarefas. Oferece uma maneira conveniente e confiável de aguardar a conclusão de todas as tarefas do grupo.
create_task()	Inicia uma Task asyncio e a retorna.
current_task()	Retorna para a Tarefa atual.
all_tasks()	Retorna todas as tarefas que ainda não foram concluídas em um laço de eventos.
await *sleep()*	Dorme for um número de segundos.
await *gather()*	Agenda e espera por coisas concorrentemente.
await *wait_for()*	Executa com um tempo limite.
await *shield()*	Protege contra cancelamento.
await *wait()*	Monitora para conclusão.
timeout()	Executa com um tempo limite. Útil nos casos em que o `wait_for` não é adequado.
to_thread()	Executa uma função assincronamente em uma thread separada.
run_coroutine_threadsafe()	Agenda uma corrotina a partir de outra thread do sistema operacional.
`for` `in` *as_completed()*	Monitora a conclusão com um loop `for`.

Exemplos

- *Usando asyncio.gather() para executar coisas em paralelo*.
- *Usando asyncio.wait_for() para forçar um tempo limite de execução*.
- *Cancelamento*.
- *Usando asyncio.sleep()*.
- Veja também a *página principal de documentação sobre Tarefas*.

Filas

Filas devem ser usadas para distribuir trabalho entre múltiplas Tarefas asyncio, implementar pools de conexão, e padrões pub/sub.

Queue	Uma fila FIFO - Primeiro que entra, é o primeiro que sai.
PriorityQueue	Uma fila de prioridade.
LifoQueue	Uma fila LIFO - Último que entra, é o primeiro que sai.

Exemplos

- *Usando asyncio.Queue para distribuir cargas de trabalho entre diversas Tasks*.
- Veja também a *Página de documentação da classe Queue*.

Subprocessos

Utilitários para iniciar subprocessos e executar comandos no console.

`await create_subprocess_exec()`	Cria um subprocesso.
`await create_subprocess_shell()`	Executa um comando no console.

Exemplos

- *Executando um comando no console*.
- Veja também a *documentação de subprocessos de APIs*.

Streams

APIs de alto nível para trabalhar com entrada e saída de rede.

`await open_connection()`	Estabelece uma conexão TCP.
`await open_unix_connection()`	Estabelece uma conexão com soquete Unix.
`await start_server()`	Inicia um servidor TCP.
`await start_unix_server()`	Inicia um servidor com soquete Unix.
`StreamReader`	Objeto async/await de alto nível para receber dados de rede.
`StreamWriter`	Objeto async/await de alto nível para enviar dados pela rede.

Exemplos

- *Exemplo de cliente TCP*.
- Veja também a documentação das *APIs de streams*.

Sincronização

Primitivas de sincronização similares a threads, que podem ser usadas em tarefas.

`Lock`	Uma trava mutex.
`Event`	Um objeto de evento.
`Condition`	Um objeto de condição.
`Semaphore`	Um semáforo.
`BoundedSemaphore`	Um semáforo limitado.
`Barrier`	Um objeto barreira.

Exemplos

- *Usando asyncio.Event*.
- *Usando asyncio.Barrier*.
- Veja também a documentação das *primitivas de sincronização de asyncio*.

Exceções

asyncio.CancelledError	Levantado quanto a Tarefa é cancelada. Veja também *Task.cancel()*.
asyncio.BrokenBarrierError	Levantado quando um Barreira é quebrada. veja também *Barrier.wait()* .

Exemplos

- *Manipulando CancelledError para executar código no cancelamento de uma requisição*.
- Veja também a lista completa de *exceções específicas de asyncio*.

19.1.16 Índice de APIs de baixo nível

Esta página lista todas as APIs de baixo nível do asyncio.

Obtendo o laço de eventos

asyncio.get_running_loop()	A função **preferida** para obter o laço de eventos em execução.
asyncio.get_event_loop()	Obtém uma instância do laço de eventos (em execução ou atual por meio da política atual).
asyncio.set_event_loop()	Define o laço de eventos como atual através da política atual.
asyncio.new_event_loop()	Cria um novo laço de eventos.

Exemplos

- *Usando asyncio.get_running_loop()*.

Métodos do laço de eventos

Veja também a seção principal da documentação sobre os *Métodos do laço de eventos*.

Ciclo de vida

loop.run_until_complete()	Executa um Future/Task/aguardável até que esteja completo.
loop.run_forever()	Executa o laço de eventos para sempre.
loop.stop()	Para o laço de eventos.
loop.close()	Fecha o laço de eventos.
loop.is_running()	Retorna `True` se o laço de eventos estiver rodando.
loop.is_closed()	Retorna `True` se o laço de eventos estiver fechado.
`await` *loop.shutdown_asyncgens()*	Fecha geradores assíncronos.

Depuração

loop.set_debug()	Habilita ou desabilita o modo de debug.
loop.get_debug()	Obtém o modo de debug atual.

Agendando funções de retorno (callbacks)

loop.call_soon()	Invoca uma função de retorno brevemente.
loop.call_soon_threadsafe()	Uma variante segura para thread de *loop.call_soon()*.
loop.call_later()	Invoca uma função de retorno *após* o tempo especificado.
loop.call_at()	Invoca uma função de retorno *no* instante especificado.

Grupo de Thread/Interpretador/Processo

await *loop.run_in_executor()*	Executa uma função vinculada à CPU ou outra que seja bloqueante em um executor *concurrent.futures*.
loop.set_default_executor()	Define o executor padrão para *loop.run_in_executor()*.

Tasks e Futures

loop.create_future()	Cria um objeto *Future*.
loop.create_task()	Agenda corrotina como uma *Task*.
loop.set_task_factory()	Define uma factory usada por *loop.create_task()* para criar *Tasks*.
loop.get_task_factory()	Obtém o factory *loop.create_task()* usado para criar *Tasks*.

DNS

await *loop.getaddrinfo()*	Versão assíncrona de *socket.getaddrinfo()*.
await *loop.getnameinfo()*	Versão assíncrona de *socket.getnameinfo()*.

Rede e IPC

await *loop.create_connection()*	Abre uma conexão TCP.
await *loop.create_server()*	Cria um servidor TCP.
await *loop.create_unix_connection()*	Abre uma conexão soquete Unix.
await *loop.create_unix_server()*	Cria um servidor soquete Unix.
await *loop.connect_accepted_socket()*	Envolve um *socket* em um par `(transport, protocol)`.
await *loop.create_datagram_endpoint()*	Abre uma conexão por datagrama (UDP).
await *loop.sendfile()*	Envia um arquivo por meio de um transporte.
await *loop.start_tls()*	Atualiza uma conexão existente para TLS.
await *loop.connect_read_pipe()*	Envolve a leitura final de um encadeamento em um par `(transport, protocol)`.
await *loop.connect_write_pipe()*	Envolve a escrita final de um encadeamento em um par `(transport, protocol)`.

Soquetes

`await loop.sock_recv()`	Recebe dados do *socket*.
`await loop.sock_recv_into()`	Recebe dados do *socket* em um buffer.
`await loop.sock_recvfrom()`	Recebe um datagrama do *socket*.
`await loop.sock_recvfrom_into()`	Recebe um datagrama do *socket* em um buffer.
`await loop.sock_sendall()`	Envia dados para o *socket*.
`await loop.sock_sendto()`	Envia um datagrama por meio de *socket* para o endereço dado.
`await loop.sock_connect()`	Conecta ao *socket*.
`await loop.sock_accept()`	Aceita uma conexão do *socket*.
`await loop.sock_sendfile()`	Envia um arquivo usando o *socket*.
`loop.add_reader()`	Começa a observar um descritor de arquivo, aguardando por disponibilidade de leitura.
`loop.remove_reader()`	Interrompe o monitoramento de um descritor de arquivo, que aguarda disponibilidade de leitura.
`loop.add_writer()`	Começa a observar um descritor de arquivo, aguardando por disponibilidade para escrita.
`loop.remove_writer()`	Interrompe o monitoramento de um descritor de arquivo, que aguarda disponibilidade para escrita.

Sinais Unix

`loop.add_signal_handler()`	Adiciona um tratador para um *signal*.
`loop.remove_signal_handler()`	Remove um tratador para um *signal*.

Subprocessos

`loop.subprocess_exec()`	Inicia um subprocesso.
`loop.subprocess_shell()`	Inicia um subprocesso a partir de um comando shell.

Tratamento de erros

`loop.call_exception_handler()`	Chama o tratamento de exceção.
`loop.set_exception_handler()`	Define um novo tratador de exceção.
`loop.get_exception_handler()`	Obtém o tratador de exceção atual.
`loop.default_exception_handler()`	A implementação padrão do tratador de exceção.

Exemplos

- *Usando asyncio.new_event_loop() e loop.run_forever()*.
- *Usando loop.call_later()*.
- Usando `loop.create_connection()` para implementar *um cliente-eco*.
- Usando `loop.create_connection()` para *conectar a um soquete*.
- *Usando add_reader() para monitorar um descritor de arquivo para eventos de leitura*.
- *Usando loop.add_signal_handler()*.
- *Usando loop.subprocess_exec()*.

Transportes

Todos os transportes implementam os seguintes métodos:

transport.close()	Fecha o transporte.
transport.is_closing()	Retorna `True` se o transporte estiver fechando ou estiver fechado.
transport.get_extra_info()	Solicita informação a respeito do transporte.
transport.set_protocol()	Define um novo protocolo.
transport.get_protocol()	Retorna o protocolo atual.

Transportes que podem receber dados (TCP e conexões Unix, encadeamentos, etc). Retornado a partir de métodos como *loop.create_connection()*, *loop.create_unix_connection()*, *loop.connect_read_pipe()*, etc:

Realiza leitura de Transportes

transport.is_reading()	Retorna `True` se o transporte estiver recebendo.
transport.pause_reading()	Pausa o recebimento.
transport.resume_reading()	Continua o recebimento.

Transportes que podem enviar dados (TCP e conexões Unix, encadeamentos, etc). Retornado a partir de métodos como *loop.create_connection()*, *loop.create_unix_connection()*, *loop.connect_write_pipe()*, etc:

Realiza escrita de Transportes

transport.write()	Escreve dados para o transporte.
transport.writelines()	Escreve buffers para o transporte.
transport.can_write_eof()	Retorna *True* se o transporte suporta o envio de EOF.
transport.write_eof()	Fecha e envia EOF após descarregar dados que estavam no buffer.
transport.abort()	Fecha o transporte imediatamente.
transport.get_write_buffer_size()	Retorna o tamanho atual do buffer de saída.
transport.get_write_buffer_limits()	Retorna marcas d'água alta e baixa para controle do fluxo de escrita.
transport.set_write_buffer_limits()	Define novas marcas d'água alta e baixa para controle do fluxo de escrita.

Transporte retornado por *loop.create_datagram_endpoint()*:

Transportes de datagrama

transport.sendto()	Envia dados para o par remoto.
transport.abort()	Fecha o transporte imediatamente.

Abstração de transporte de baixo nível sobre subprocessos. Retornado por *loop.subprocess_exec()* e *loop.subprocess_shell()*:

Transportes de Subprocesso

transport.get_pid()	Retorna o process id do subprocesso.
transport.get_pipe_transport()	Retorna o transporte para o encadeamento de comunicação requisitada (*stdin*, *stdout*, ou *stderr*).
transport.get_returncode()	Retorna o código de retorno do subprocesso.
transport.kill()	Mata o subprocesso.
transport.send_signal()	Envia um sinal para o subprocesso.
transport.terminate()	Interrompe o subprocesso.
transport.close()	Mata o subprocesso e fecha todos os encadeamentos.

Protocolos

Classes de protocolos podem implementar os seguintes **métodos de função de retorno**:

callback *connection_made()*	Chamado quando uma conexão é estabelecida.
callback *connection_lost()*	Chamado quanto a conexão é perdida ou fechada.
callback *pause_writing()*	Chamado quando o buffer de transporte ultrapassa a marca de nível alto d'água.
callback *resume_writing()*	Chamado quando o buffer de transporte drena abaixo da marca de nível baixo d'agua.

Protocolos de Streaming (TCP, Soquetes Unix, Encadeamentos)

callback *data_received()*	Chamado quando algum dado é recebido.
callback *eof_received()*	Chamado quando um EOF é recebido.

Protocolos de Streaming Bufferizados

callback *get_buffer()*	Chamada para alocar um novo buffer para recebimento.
callback *buffer_updated()*	Chamado quando o buffer foi atualizado com os dados recebidos.
callback *eof_received()*	Chamado quando um EOF é recebido.

Protocolos de Datagramas

callback *datagram_received()*	Chamado quando um datagrama é recebido.
callback *error_received()*	Chamado quando uma operação de envio ou recebimento anterior levanta um *OSError*.

Protocolos de Subprocesso

callback *pipe_data_received()*	Chamado quando o processo filho escreve dados no seu encadeamento *stdout* ou *stderr*.
callback *pipe_connection_lost()*	Chamado quando um dos encadeamentos comunicando com o processo filho é fechado.
callback *process_exited()*	Chamado quando o processo filho sai. Isso pode ser chamado antes dos métodos *pipe_data_received()* e *pipe_connection_lost()*.

Políticas de laço de eventos

Política é um mecanismo de baixo nível para alterar o comportamento de funções, similar a *asyncio.get_event_loop()*. Veja também a *seção principal de políticas* para mais detalhes.

Acessando Políticas

asyncio.get_event_loop_policy()	Retorna a política de todo o processo atual.
asyncio.set_event_loop_policy()	Define uma nova política para todo o processo.
AbstractEventLoopPolicy	Classe base para objetos de política.

19.1.17 Desenvolvendo com asyncio

Asynchronous programming is different from classic "sequential" programming.

This page lists common mistakes and traps and explains how to avoid them.

Modo de Depuração

By default asyncio runs in production mode. In order to ease the development asyncio has a *debug mode*.

There are several ways to enable asyncio debug mode:

- Setting the `PYTHONASYNCIODEBUG` environment variable to `1`.
- Usando o *Modo de Desenvolvimento do Python*.
- Passando `debug=True` para *asyncio.run()*.
- Chamando *loop.set_debug()*.

Além de habilitar o modo de depuração, considere também:

- setting the log level of the *asyncio logger* to *logging.DEBUG*, for example the following snippet of code can be run at startup of the application:

```
logging.basicConfig(level=logging.DEBUG)
```

- configuring the *warnings* module to display *ResourceWarning* warnings. One way of doing that is by using the `-W default` command line option.

Quando o modo de depuração está habilitado:

- Many non-threadsafe asyncio APIs (such as *loop.call_soon()* and *loop.call_at()* methods) raise an exception if they are called from a wrong thread.
- O tempo de execução de um seletor de E/S é registrado se demorar muito para executar a operação E/S.
- Funções de retorno demorando mais do que 100 milissegundos são registradas. O atributo *loop.slow_callback_duration* pode ser usado para definir a duração de execução mínima em segundos para se considerada "devagar".

Concorrência e Múltiplas Threads

An event loop runs in a thread (typically the main thread) and executes all callbacks and Tasks in its thread. While a Task is running in the event loop, no other Tasks can run in the same thread. When a Task executes an `await` expression, the running Task gets suspended, and the event loop executes the next Task.

Para agendar uma *callback* de outra thread do SO, o método *loop.call_soon_threadsafe()* deve ser usado. Exemplo:

```
loop.call_soon_threadsafe(callback, *args)
```

Almost all asyncio objects are not thread safe, which is typically not a problem unless there is code that works with them from outside of a Task or a callback. If there's a need for such code to call a low-level asyncio API, the *loop.call_soon_threadsafe()* method should be used, e.g.:

```
loop.call_soon_threadsafe(fut.cancel)
```

To schedule a coroutine object from a different OS thread, the *run_coroutine_threadsafe()* function should be used. It returns a *concurrent.futures.Future* to access the result:

```
async def coro_func():
    return await asyncio.sleep(1, 42)

# Later in another OS thread:

future = asyncio.run_coroutine_threadsafe(coro_func(), loop)
# Wait for the result:
result = future.result()
```

To handle signals the event loop must be run in the main thread.

The *loop.run_in_executor()* method can be used with a *concurrent.futures.ThreadPoolExecutor* or *InterpreterPoolExecutor* to execute blocking code in a different OS thread without blocking the OS thread that the event loop runs in.

There is currently no way to schedule coroutines or callbacks directly from a different process (such as one started with *multiprocessing*). The *Métodos do laço de eventos* section lists APIs that can read from pipes and watch file descriptors without blocking the event loop. In addition, asyncio's *Subprocess* APIs provide a way to start a process and communicate with it from the event loop. Lastly, the aforementioned *loop.run_in_executor()* method can also be used with a *concurrent.futures.ProcessPoolExecutor* to execute code in a different process.

Executando código bloqueante

Blocking (CPU-bound) code should not be called directly. For example, if a function performs a CPU-intensive calculation for 1 second, all concurrent asyncio Tasks and IO operations would be delayed by 1 second.

An executor can be used to run a task in a different thread, including in a different interpreter, or even in a different process to avoid blocking the OS thread with the event loop. See the *loop.run_in_executor()* method for more details.

Gerando logs

asyncio usa o módulo *logging* e todo registro é feito via o registrador `"asyncio"`.

The default log level is *logging.INFO*, which can be easily adjusted:

```
logging.getLogger("asyncio").setLevel(logging.WARNING)
```

Network logging can block the event loop. It is recommended to use a separate thread for handling logs or use non-blocking IO. For example, see blocking-handlers.

Detect never-awaited coroutines

When a coroutine function is called, but not awaited (e.g. `coro()` instead of `await coro()`) or the coroutine is not scheduled with *asyncio.create_task()*, asyncio will emit a *RuntimeWarning*:

```
import asyncio

async def test():
    print("never scheduled")

async def main():
```

(continua na próxima página)

(continuação da página anterior)

```
    test()

asyncio.run(main())
```

Saída:

```
test.py:7: RuntimeWarning: coroutine 'test' was never awaited
  test()
```

Output in debug mode:

```
test.py:7: RuntimeWarning: coroutine 'test' was never awaited
Coroutine created at (most recent call last)
  File "../t.py", line 9, in <module>
    asyncio.run(main(), debug=True)

  < .. >

  File "../t.py", line 7, in main
    test()
  test()
```

The usual fix is to either await the coroutine or call the *asyncio.create_task()* function:

```
async def main():
    await test()
```

Detect never-retrieved exceptions

If a *Future.set_exception()* is called but the Future object is never awaited on, the exception would never be propagated to the user code. In this case, asyncio would emit a log message when the Future object is garbage collected.

Exemplo de uma exceção não tratada:

```
import asyncio

async def bug():
    raise Exception("not consumed")

async def main():
    asyncio.create_task(bug())

asyncio.run(main())
```

Saída:

```
Task exception was never retrieved
future: <Task finished coro=<bug() done, defined at test.py:3>
  exception=Exception('not consumed')>

Traceback (most recent call last):
  File "test.py", line 4, in bug
    raise Exception("not consumed")
Exception: not consumed
```

Enable the debug mode to get the traceback where the task was created:

```
asyncio.run(main(), debug=True)
```

Output in debug mode:

```
Task exception was never retrieved
future: <Task finished coro=<bug() done, defined at test.py:3>
    exception=Exception('not consumed') created at asyncio/tasks.py:321>

source_traceback: Object created at (most recent call last):
  File "../t.py", line 9, in <module>
    asyncio.run(main(), debug=True)

< .. >

Traceback (most recent call last):
  File "../t.py", line 4, in bug
    raise Exception("not consumed")
Exception: not consumed
```

Nota

O código-fonte para o asyncio pode ser encontrado em Lib/asyncio/.

19.2 `socket` — Interface de rede de baixo nível

Código-fonte: Lib/socket.py

Este módulo provê acesso à interface de *soquete* do BSD. Está disponível em todos os sistemas modernos Unix, Windows, MacOS e provavelmente outras plataformas.

Nota

Algum comportamento pode depender da plataforma, pois as chamadas são feitas para as APIs de soquete do sistema operacional.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

A interface Python é uma transliteração direta da chamada de sistema e interface de biblioteca de soquetes do Unix para o estilo orientado a objetos do Python: a função `socket()` retorna um *objeto socket* cujos métodos implementam as diversas chamadas de sistema de soquetes. Os tipos de parâmetro são consideravelmente de nível mais alto que os da interface em C: assim como as operações `read()` e `write()` em arquivos Python, a alocação de buffer em operações de recebimento é automática, e o comprimento do buffer é implícito em operações de envio.

Ver também

Módulo *socketserver*
Classes que simplificam a escrita de servidores de rede.

Módulo *ssl*
Um invólucro de TLS/SSL para objetos socket.

19.2.1 Famílias de soquete

Dependendo do sistema e das opções de construção, várias famílias de soquetes são suportadas por este módulo.

O formato de endereço requerido por um objeto socket em particular é selecionado automaticamente com base na família de endereços especificada quando o objeto socket foi criado. Endereços de socket são representados da seguinte forma:

- O endereço de um socket *AF_UNIX* vinculado a um nó do sistema de arquivos é representado como uma string, usando a codificação do sistema de arquivos e o tratador de erros `'surrogateescape'` (veja **PEP 383**). Um endereço no espaço de nomes abstrato do Linux é retornado como um *objeto bytes ou similar* com um byte nulo inicial; observe que os sockets neste espaço de nomes podem se comunicar com sockets normais do sistema de arquivos, então programas destinados a rodar no Linux podem precisar lidar com ambos os tipos de endereço. Uma string ou um objeto bytes ou similar pode ser usado para qualquer tipo de endereço ao passá-lo como um argumento.

  Alterado na versão 3.3: Previously, *AF_UNIX* socket paths were assumed to use UTF-8 encoding.

  Alterado na versão 3.5: Writable *bytes-like object* is now accepted.

- A pair `(host, port)` is used for the *AF_INET* address family, where *host* is a string representing either a hostname in internet domain notation like `'daring.cwi.nl'` or an IPv4 address like `'100.50.200.5'`, and *port* is an integer.

  - For IPv4 addresses, two special forms are accepted instead of a host address: `''` represents `INADDR_ANY`, which is used to bind to all interfaces, and the string `'<broadcast>'` represents `INADDR_BROADCAST`. This behavior is not compatible with IPv6, therefore, you may want to avoid these if you intend to support IPv6 with your Python programs.

- For *AF_INET6* address family, a four-tuple `(host, port, flowinfo, scope_id)` is used, where *flowinfo* and *scope_id* represent the `sin6_flowinfo` and `sin6_scope_id` members in `struct sockaddr_in6` in C. For *socket* module methods, *flowinfo* and *scope_id* can be omitted just for backward compatibility. Note, however, omission of *scope_id* can cause problems in manipulating scoped IPv6 addresses.

  Alterado na versão 3.7: For multicast addresses (with *scope_id* meaningful) *address* may not contain `%scope_id` (or `zone id`) part. This information is superfluous and may be safely omitted (recommended).

- `AF_NETLINK` sockets are represented as pairs `(pid, groups)`.

- Linux-only support for TIPC is available using the `AF_TIPC` address family. TIPC is an open, non-IP based networked protocol designed for use in clustered computer environments. Addresses are represented by a tuple, and the fields depend on the address type. The general tuple form is `(addr_type, v1, v2, v3 [, scope])`, where:

  - *addr_type* is one of `TIPC_ADDR_NAMESEQ`, `TIPC_ADDR_NAME`, or `TIPC_ADDR_ID`.
  - *scope* is one of `TIPC_ZONE_SCOPE`, `TIPC_CLUSTER_SCOPE`, and `TIPC_NODE_SCOPE`.
  - If *addr_type* is `TIPC_ADDR_NAME`, then *v1* is the server type, *v2* is the port identifier, and *v3* should be 0.

    If *addr_type* is `TIPC_ADDR_NAMESEQ`, then *v1* is the server type, *v2* is the lower port number, and *v3* is the upper port number.

    If *addr_type* is `TIPC_ADDR_ID`, then *v1* is the node, *v2* is the reference, and *v3* should be set to 0.

- A tuple `(interface, )` is used for the *AF_CAN* address family, where *interface* is a string representing a network interface name like `'can0'`. The network interface name `''` can be used to receive packets from all network interfaces of this family.

  - *CAN_ISOTP* protocol require a tuple `(interface, rx_addr, tx_addr)` where both additional parameters are unsigned long integer that represent a CAN identifier (standard or extended).
  - *CAN_J1939* protocol require a tuple `(interface, name, pgn, addr)` where additional parameters are 64-bit unsigned integer representing the ECU name, a 32-bit unsigned integer representing the Parameter Group Number (PGN), and an 8-bit integer representing the address.

- A string or a tuple `(id, unit)` is used for the `SYSPROTO_CONTROL` protocol of the `PF_SYSTEM` family. The string is the name of a kernel control using a dynamically assigned ID. The tuple can be used if ID and unit number of the kernel control are known or if a registered ID is used.

  Adicionado na versão 3.3.

- *AF_BLUETOOTH* supports the following protocols and address formats:

  - *BTPROTO_L2CAP* accepts a tuple `(bdaddr, psm[, cid[, bdaddr_type]])` where:

    * `bdaddr` is a string specifying the Bluetooth address.
    * `psm` is an integer specifying the Protocol/Service Multiplexer.
    * `cid` is an optional integer specifying the Channel Identifier. If not given, defaults to zero.
    * `bdaddr_type` is an optional integer specifying the address type; one of *BDADDR_BREDR* (default), *BDADDR_LE_PUBLIC*, *BDADDR_LE_RANDOM*.

    Alterado na versão 3.14: Added `cid` and `bdaddr_type` fields.

  - *BTPROTO_RFCOMM* accepts `(bdaddr, channel)` where `bdaddr` is the Bluetooth address as a string and `channel` is an integer.

  - *BTPROTO_HCI* accepts a format that depends on your OS.

    * On Linux it accepts an integer `device_id` or a tuple `(device_id, [channel])` where `device_id` specifies the number of the Bluetooth device, and `channel` is an optional integer specifying the HCI channel (*HCI_CHANNEL_RAW* by default).
    * On FreeBSD, NetBSD and DragonFly BSD it accepts `bdaddr` where `bdaddr` is the Bluetooth address as a string.

    Alterado na versão 3.2: NetBSD and DragonFlyBSD support added.

    Alterado na versão 3.13.3: FreeBSD support added.

    Alterado na versão 3.14: Added `channel` field. `device_id` not packed in a tuple is now accepted.

  - *BTPROTO_SCO* accepts `bdaddr` where `bdaddr` is the Bluetooth address as a string or a *bytes* object. (ex. `'12:23:34:45:56:67'` or `b'12:23:34:45:56:67'`)

    Alterado na versão 3.14: FreeBSD support added.

- *AF_ALG* is a Linux-only socket based interface to Kernel cryptography. An algorithm socket is configured with a tuple of two to four elements `(type, name [, feat [, mask]])`, where:

  - *type* is the algorithm type as string, e.g. `aead`, `hash`, `skcipher` or `rng`.
  - *name* is the algorithm name and operation mode as string, e.g. `sha256`, `hmac(sha256)`, `cbc(aes)` or `drbg_nopr_ctr_aes256`.
  - *feat* and *mask* are unsigned 32bit integers.

  Disponibilidade: Linux >= 2.6.38.

  Some algorithm types require more recent Kernels.

  Adicionado na versão 3.6.

- *AF_VSOCK* allows communication between virtual machines and their hosts. The sockets are represented as a `(CID, port)` tuple where the context ID or CID and port are integers.

  Disponibilidade: Linux >= 3.9

  See *vsock(7)*

  Adicionado na versão 3.7.

- *AF_PACKET* is a low-level interface directly to network devices. The addresses are represented by the tuple `(ifname, proto[, pkttype[, hatype[, addr]]])` where:

  - *ifname* - String specifying the device name.

  - *proto* - The Ethernet protocol number. May be *ETH_P_ALL* to capture all protocols, one of the *ETHERTYPE_* constants* or any other Ethernet protocol number.
  - *pkttype* - Optional integer specifying the packet type:
    * `PACKET_HOST` (the default) - Packet addressed to the local host.
    * `PACKET_BROADCAST` - Physical-layer broadcast packet.
    * `PACKET_MULTICAST` - Packet sent to a physical-layer multicast address.
    * `PACKET_OTHERHOST` - Packet to some other host that has been caught by a device driver in promiscuous mode.
    * `PACKET_OUTGOING` - Packet originating from the local host that is looped back to a packet socket.
  - *hatype* - Optional integer specifying the ARP hardware address type.
  - *addr* - Optional bytes-like object specifying the hardware physical address, whose interpretation depends on the device.

  Disponibilidade: Linux >= 2.2.

- *AF_QIPCRTR* is a Linux-only socket based interface for communicating with services running on co-processors in Qualcomm platforms. The address family is represented as a `(node, port)` tuple where the *node* and *port* are non-negative integers.

  Disponibilidade: Linux >= 4.7.

  Adicionado na versão 3.8.

- `IPPROTO_UDPLITE` is a variant of UDP which allows you to specify what portion of a packet is covered with the checksum. It adds two socket options that you can change. `self.setsockopt(IPPROTO_UDPLITE, UDPLITE_SEND_CSCOV, length)` will change what portion of outgoing packets are covered by the checksum and `self.setsockopt(IPPROTO_UDPLITE, UDPLITE_RECV_CSCOV, length)` will filter out packets which cover too little of their data. In both cases `length` should be in `range(8, 2**16, 8)`.

  Such a socket should be constructed with `socket(AF_INET, SOCK_DGRAM, IPPROTO_UDPLITE)` for IPv4 or `socket(AF_INET6, SOCK_DGRAM, IPPROTO_UDPLITE)` for IPv6.

  Disponibilidade: Linux >= 2.6.20, FreeBSD >= 10.1

  Adicionado na versão 3.9.

- *AF_HYPERV* is a Windows-only socket based interface for communicating with Hyper-V hosts and guests. The address family is represented as a `(vm_id, service_id)` tuple where the `vm_id` and `service_id` are UUID strings.

  The `vm_id` is the virtual machine identifier or a set of known VMID values if the target is not a specific virtual machine. Known VMID constants defined on `socket` are:

  - `HV_GUID_ZERO`
  - `HV_GUID_BROADCAST`
  - `HV_GUID_WILDCARD` - Used to bind on itself and accept connections from all partitions.
  - `HV_GUID_CHILDREN` - Used to bind on itself and accept connection from child partitions.
  - `HV_GUID_LOOPBACK` - Used as a target to itself.
  - `HV_GUID_PARENT` - When used as a bind accepts connection from the parent partition. When used as an address target it will connect to the parent partition.

  The `service_id` is the service identifier of the registered service.

  Adicionado na versão 3.12.

If you use a hostname in the *host* portion of IPv4/v6 socket address, the program may show a nondeterministic behavior, as Python uses the first address returned from the DNS resolution. The socket address will be resolved differently into an actual IPv4/v6 address, depending on the results from DNS resolution and/or the host configuration. For deterministic behavior use a numeric address in *host* portion.

All errors raise exceptions. The normal exceptions for invalid argument types and out-of-memory conditions can be raised. Errors related to socket or address semantics raise *OSError* or one of its subclasses.

Non-blocking mode is supported through *setblocking()*. A generalization of this based on timeouts is supported through *settimeout()*.

19.2.2 Conteúdo do módulo

The module *socket* exports the following elements.

Exceções

exception socket.**error**

Um apelido descontinuado de *OSError*.

Alterado na versão 3.3: Seguindo a **PEP 3151**, esta classe foi transformada em um apelido de *OSError*.

exception socket.**herror**

A subclass of *OSError*, this exception is raised for address-related errors, i.e. for functions that use *h_errno* in the POSIX C API, including *gethostbyname_ex()* and *gethostbyaddr()*. The accompanying value is a pair `(h_errno, string)` representing an error returned by a library call. *h_errno* is a numeric value, while *string* represents the description of *h_errno*, as returned by the `hstrerror()` C function.

Alterado na versão 3.3: This class was made a subclass of *OSError*.

exception socket.**gaierror**

A subclass of *OSError*, this exception is raised for address-related errors by *getaddrinfo()* and *getnameinfo()*. The accompanying value is a pair `(error, string)` representing an error returned by a library call. *string* represents the description of *error*, as returned by the `gai_strerror()` C function. The numeric *error* value will match one of the `EAI_*` constants defined in this module.

Alterado na versão 3.3: This class was made a subclass of *OSError*.

exception socket.**timeout**

A deprecated alias of *TimeoutError*.

A subclass of *OSError*, this exception is raised when a timeout occurs on a socket which has had timeouts enabled via a prior call to *settimeout()* (or implicitly through *setdefaulttimeout()*). The accompanying value is a string whose value is currently always “timed out”.

Alterado na versão 3.3: This class was made a subclass of *OSError*.

Alterado na versão 3.10: Esta classe foi feita como um apelido de *TimeoutError*.

Constantes

The AF_* and SOCK_* constants are now `AddressFamily` and `SocketKind` *IntEnum* collections.

Adicionado na versão 3.4.

socket.**AF_UNIX**

socket.**AF_INET**

socket.**AF_INET6**

These constants represent the address (and protocol) families, used for the first argument to *socket()*. If the *AF_UNIX* constant is not defined then this protocol is unsupported. More constants may be available depending on the system.

socket.**AF_UNSPEC**

AF_UNSPEC means that *getaddrinfo()* should return socket addresses for any address family (either IPv4, IPv6, or any other) that can be used.

socket.**SOCK_STREAM**

socket.**SOCK_DGRAM**

socket.**SOCK_RAW**

socket.**SOCK_RDM**

socket.**SOCK_SEQPACKET**

These constants represent the socket types, used for the second argument to *socket()*. More constants may be available depending on the system. (Only *SOCK_STREAM* and *SOCK_DGRAM* appear to be generally useful.)

socket.**SOCK_CLOEXEC**

socket.**SOCK_NONBLOCK**

These two constants, if defined, can be combined with the socket types and allow you to set some flags atomically (thus avoiding possible race conditions and the need for separate calls).

> **Ver também**
>
> Secure File Descriptor Handling for a more thorough explanation.

Disponibilidade: Linux >= 2.6.27.

Adicionado na versão 3.2.

SO_*

socket.**SOMAXCONN**

MSG_*

SOL_*

SCM_*

IPPROTO_*

IPPORT_*

INADDR_*

IP_*

IPV6_*

EAI_*

AI_*

NI_*

TCP_*

Many constants of these forms, documented in the Unix documentation on sockets and/or the IP protocol, are also defined in the socket module. They are generally used in arguments to the *setsockopt()* and *getsockopt()* methods of socket objects. In most cases, only those symbols that are defined in the Unix header files are defined; for a few symbols, default values are provided.

Alterado na versão 3.6: `SO_DOMAIN`, `SO_PROTOCOL`, `SO_PEERSEC`, `SO_PASSSEC`, `TCP_USER_TIMEOUT`, `TCP_CONGESTION` were added.

Alterado na versão 3.6.5: Added support for `TCP_FASTOPEN`, `TCP_KEEPCNT` on Windows platforms when available.

Alterado na versão 3.7: `TCP_NOTSENT_LOWAT` was added.

Added support for `TCP_KEEPIDLE`, `TCP_KEEPINTVL` on Windows platforms when available.

Alterado na versão 3.10: `IP_RECVTOS` was added. Added `TCP_KEEPALIVE`. On MacOS this constant can be used in the same way that `TCP_KEEPIDLE` is used on Linux.

Alterado na versão 3.11: Added `TCP_CONNECTION_INFO`. On MacOS this constant can be used in the same way that `TCP_INFO` is used on Linux and BSD.

Alterado na versão 3.12: Added `SO_RTABLE` and `SO_USER_COOKIE`. On OpenBSD and FreeBSD respectively those constants can be used in the same way that `SO_MARK` is used on Linux. Also added missing TCP socket options from Linux: `TCP_MD5SIG`, `TCP_THIN_LINEAR_TIMEOUTS`, `TCP_THIN_DUPACK`, `TCP_REPAIR`, `TCP_REPAIR_QUEUE`, `TCP_QUEUE_SEQ`, `TCP_REPAIR_OPTIONS`, `TCP_TIMESTAMP`, `TCP_CC_INFO`,

TCP_SAVE_SYN, TCP_SAVED_SYN, TCP_REPAIR_WINDOW, TCP_FASTOPEN_CONNECT, TCP_ULP, TCP_MD5SIG_EXT, TCP_FASTOPEN_KEY, TCP_FASTOPEN_NO_COOKIE, TCP_ZEROCOPY_RECEIVE, TCP_INQ, TCP_TX_DELAY. Added IP_PKTINFO, IP_UNBLOCK_SOURCE, IP_BLOCK_SOURCE, IP_ADD_SOURCE_MEMBERSHIP, IP_DROP_SOURCE_MEMBERSHIP.

Alterado na versão 3.13: Added SO_BINDTOIFINDEX. On Linux this constant can be used in the same way that SO_BINDTODEVICE is used, but with the index of a network interface instead of its name.

Alterado na versão 3.14: Added missing IP_FREEBIND, IP_RECVERR, IPV6_RECVERR, IP_RECVTTL, and IP_RECVORIGDSTADDR on Linux.

Alterado na versão 3.14: Added support for TCP_QUICKACK on Windows platforms when available.

socket.**AF_CAN**

socket.**PF_CAN**

SOL_CAN_*

CAN_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module.

Disponibilidade: Linux >= 2.6.25, NetBSD >= 8.

Adicionado na versão 3.3.

Alterado na versão 3.11: NetBSD support was added.

Alterado na versão 3.14: Restored missing CAN_RAW_ERR_FILTER on Linux.

socket.**CAN_BCM**

CAN_BCM_*

CAN_BCM, in the CAN protocol family, is the broadcast manager (BCM) protocol. Broadcast manager constants, documented in the Linux documentation, are also defined in the socket module.

Disponibilidade: Linux >= 2.6.25.

> **Nota**
>
> The CAN_BCM_CAN_FD_FRAME flag is only available on Linux >= 4.8.

Adicionado na versão 3.4.

socket.**CAN_RAW_FD_FRAMES**

Enables CAN FD support in a CAN_RAW socket. This is disabled by default. This allows your application to send both CAN and CAN FD frames; however, you must accept both CAN and CAN FD frames when reading from the socket.

This constant is documented in the Linux documentation.

Disponibilidade: Linux >= 3.6.

Adicionado na versão 3.5.

socket.**CAN_RAW_JOIN_FILTERS**

Joins the applied CAN filters such that only CAN frames that match all given CAN filters are passed to user space.

This constant is documented in the Linux documentation.

Disponibilidade: Linux >= 4.1.

Adicionado na versão 3.9.

socket.**CAN_ISOTP**

CAN_ISOTP, in the CAN protocol family, is the ISO-TP (ISO 15765-2) protocol. ISO-TP constants, documented in the Linux documentation.

Disponibilidade: Linux >= 2.6.25.

Adicionado na versão 3.7.

socket.**CAN_J1939**

CAN_J1939, in the CAN protocol family, is the SAE J1939 protocol. J1939 constants, documented in the Linux documentation.

Disponibilidade: Linux >= 5.4.

Adicionado na versão 3.9.

socket.**AF_DIVERT**

socket.**PF_DIVERT**

These two constants, documented in the FreeBSD divert(4) manual page, are also defined in the socket module.

Disponibilidade: FreeBSD >= 14.0.

Adicionado na versão 3.12.

socket.**AF_PACKET**

socket.**PF_PACKET**

PACKET_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module.

Disponibilidade: Linux >= 2.2.

socket.**ETH_P_ALL**

ETH_P_ALL can be used in the *socket* constructor as *proto* for the *AF_PACKET* family in order to capture every packet, regardless of protocol.

For more information, see the *packet(7)* manpage.

Disponibilidade: Linux.

Adicionado na versão 3.12.

socket.**AF_RDS**

socket.**PF_RDS**

socket.**SOL_RDS**

RDS_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module.

Disponibilidade: Linux >= 2.6.30.

Adicionado na versão 3.3.

socket.**SIO_RCVALL**

socket.**SIO_KEEPALIVE_VALS**

socket.**SIO_LOOPBACK_FAST_PATH**

RCVALL_*

Constants for Windows' WSAIoctl(). The constants are used as arguments to the *ioctl()* method of socket objects.

Alterado na versão 3.6: SIO_LOOPBACK_FAST_PATH was added.

TIPC_*

TIPC related constants, matching the ones exported by the C socket API. See the TIPC documentation for more information.

socket.**AF_ALG**

socket.**SOL_ALG**

ALG_*

Constants for Linux Kernel cryptography.

Disponibilidade: Linux >= 2.6.38.

Adicionado na versão 3.6.

socket.**AF_VSOCK**

socket.**IOCTL_VM_SOCKETS_GET_LOCAL_CID**

VMADDR*

SO_VM*

Constants for Linux host/guest communication.

Disponibilidade: Linux >= 4.8.

Adicionado na versão 3.7.

socket.**AF_LINK**

Disponibilidade: BSD, macOS.

Adicionado na versão 3.4.

socket.**has_ipv6**

This constant contains a boolean value which indicates if IPv6 is supported on this platform.

socket.**AF_BLUETOOTH**

socket.**BTPROTO_L2CAP**

socket.**BTPROTO_RFCOMM**

socket.**BTPROTO_HCI**

socket.**BTPROTO_SCO**

Integer constants for use with Bluetooth addresses.

socket.**BDADDR_ANY**

socket.**BDADDR_LOCAL**

These are string constants containing Bluetooth addresses with special meanings. For example, *BDADDR_ANY* can be used to indicate any address when specifying the binding socket with *BTPROTO_RFCOMM*.

socket.**BDADDR_BREDR**

socket.**BDADDR_LE_PUBLIC**

socket.**BDADDR_LE_RANDOM**

These constants describe the Bluetooth address type when binding or connecting a *BTPROTO_L2CAP* socket.

Disponibilidade: Linux, FreeBSD

Adicionado na versão 3.14.

socket.**SOL_RFCOMM**

socket.**SOL_L2CAP**

socket.**SOL_HCI**

socket.**SOL_SCO**

socket.**SOL_BLUETOOTH**

Used in the level argument to the *setsockopt()* and *getsockopt()* methods of Bluetooth socket objects.

SOL_BLUETOOTH is only available on Linux. Other constants are available if the corresponding protocol is supported.

SO_L2CAP_*

socket.**L2CAP_LM**

L2CAP_LM_*

SO_RFCOMM_*

RFCOMM_LM_*

SO_SCO_*

SO_BTH_*

BT_*

Used in the option name and value argument to the *setsockopt()* and *getsockopt()* methods of Bluetooth socket objects.

BT_* and *L2CAP_LM* are only available on Linux. SO_BTH_* are only available on Windows. Other constants may be available on Linux and various BSD platforms.

Adicionado na versão 3.14.

socket.**HCI_FILTER**

socket.**HCI_TIME_STAMP**

socket.**HCI_DATA_DIR**

socket.**SO_HCI_EVT_FILTER**

socket.**SO_HCI_PKT_FILTER**

Option names for use with *BTPROTO_HCI*. Availability and format of the option values depend on platform.

Alterado na versão 3.14: Added SO_HCI_EVT_FILTER and SO_HCI_PKT_FILTER on NetBSD and DragonFly BSD. Added HCI_DATA_DIR on FreeBSD, NetBSD and DragonFly BSD.

socket.**HCI_DEV_NONE**

The device_id value used to create an HCI socket that isn't specific to a single Bluetooth adapter.

Disponibilidade: Linux

Adicionado na versão 3.14.

socket.**HCI_CHANNEL_RAW**

socket.**HCI_CHANNEL_USER**

socket.**HCI_CHANNEL_MONITOR**

socket.**HCI_CHANNEL_CONTROL**

socket.**HCI_CHANNEL_LOGGING**

Possible values for channel field in the *BTPROTO_HCI* address.

Disponibilidade: Linux

Adicionado na versão 3.14.

socket.**AF_QIPCRTR**

Constant for Qualcomm's IPC router protocol, used to communicate with service providing remote processors.

Disponibilidade: Linux >= 4.7.

socket.**SCM_CREDS2**

socket.**LOCAL_CREDS**

socket.**LOCAL_CREDS_PERSISTENT**

LOCAL_CREDS and LOCAL_CREDS_PERSISTENT can be used with SOCK_DGRAM, SOCK_STREAM sockets, equivalent to Linux/DragonFlyBSD SO_PASSCRED, while LOCAL_CREDS sends the credentials at first read, LOCAL_CREDS_PERSISTENT sends for each read, SCM_CREDS2 must be then used for the latter for the message type.

Adicionado na versão 3.11.

Disponibilidade: FreeBSD.

socket.**SO_INCOMING_CPU**

Constant to optimize CPU locality, to be used in conjunction with SO_REUSEPORT.

Adicionado na versão 3.11.

Disponibilidade: Linux >= 3.9

socket.**SO_REUSEPORT_LB**

Constant to enable duplicate address and port bindings with load balancing.

Adicionado na versão 3.14.

Disponibilidade: FreeBSD >= 12.0

socket.**AF_HYPERV**

socket.**HV_PROTOCOL_RAW**

socket.**HVSOCKET_CONNECT_TIMEOUT**

socket.**HVSOCKET_CONNECT_TIMEOUT_MAX**

socket.**HVSOCKET_CONNECTED_SUSPEND**

socket.**HVSOCKET_ADDRESS_FLAG_PASSTHRU**

socket.**HV_GUID_ZERO**

socket.**HV_GUID_WILDCARD**

socket.**HV_GUID_BROADCAST**

socket.**HV_GUID_CHILDREN**

socket.**HV_GUID_LOOPBACK**

socket.**HV_GUID_PARENT**

Constants for Windows Hyper-V sockets for host/guest communications.

Disponibilidade: Windows.

Adicionado na versão 3.12.

socket.**ETHERTYPE_ARP**

socket.**ETHERTYPE_IP**

socket.**ETHERTYPE_IPV6**

socket.**ETHERTYPE_VLAN**

IEEE 802.3 protocol number. constants.

Disponibilidade: Linux, FreeBSD, macOS.

Adicionado na versão 3.12.

socket.**SHUT_RD**

socket.**SHUT_WR**

socket.**SHUT_RDWR**

These constants are used by the *shutdown()* method of socket objects.

Disponibilidade: not WASI.

Funções

Criação de sockets

The following functions all create *socket objects*.

class socket.**socket**(*family=AF_INET*, *type=SOCK_STREAM*, *proto=0*, *fileno=None*)

Create a new socket using the given address family, socket type and protocol number. The address family should be *AF_INET* (the default), *AF_INET6*, *AF_UNIX*, *AF_CAN*, *AF_PACKET*, or *AF_RDS*. The socket type should be *SOCK_STREAM* (the default), *SOCK_DGRAM*, *SOCK_RAW* or perhaps one of the other SOCK_ constants. The

protocol number is usually zero and may be omitted or in the case where the address family is *AF_CAN* the protocol should be one of `CAN_RAW`, *CAN_BCM*, *CAN_ISOTP* or *CAN_J1939*.

If *fileno* is specified, the values for *family*, *type*, and *proto* are auto-detected from the specified file descriptor. Auto-detection can be overruled by calling the function with explicit *family*, *type*, or *proto* arguments. This only affects how Python represents e.g. the return value of *socket.getpeername()* but not the actual OS resource. Unlike *socket.fromfd()*, *fileno* will return the same socket and not a duplicate. This may help close a detached socket using *socket.close()*.

The newly created socket is *non-inheritable*.

Levanta um *evento de auditoria* `socket.__new__` com os argumentos `self`, `family`, `type`, `protocol`.

Alterado na versão 3.3: The AF_CAN family was added. The AF_RDS family was added.

Alterado na versão 3.4: The CAN_BCM protocol was added.

Alterado na versão 3.4: The returned socket is now non-inheritable.

Alterado na versão 3.7: The CAN_ISOTP protocol was added.

Alterado na versão 3.7: When *SOCK_NONBLOCK* or *SOCK_CLOEXEC* bit flags are applied to *type* they are cleared, and *socket.type* will not reflect them. They are still passed to the underlying system `socket()` call. Therefore,

```
sock = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM | socket.SOCK_NONBLOCK)
```

will still create a non-blocking socket on OSes that support `SOCK_NONBLOCK`, but `sock.type` will be set to `socket.SOCK_STREAM`.

Alterado na versão 3.9: The CAN_J1939 protocol was added.

Alterado na versão 3.10: The IPPROTO_MPTCP protocol was added.

`socket.`**`socketpair`**`(`[*family*[, *type*[, *proto*]]]`)`

Build a pair of connected socket objects using the given address family, socket type, and protocol number. Address family, socket type, and protocol number are as for the *socket()* function above. The default family is *AF_UNIX* if defined on the platform; otherwise, the default is *AF_INET*.

The newly created sockets are *non-inheritable*.

Alterado na versão 3.2: The returned socket objects now support the whole socket API, rather than a subset.

Alterado na versão 3.4: The returned sockets are now non-inheritable.

Alterado na versão 3.5: Windows support added.

`socket.`**`create_connection`**`(`*address*, *timeout=GLOBAL_DEFAULT*, *source_address=None*, *, *all_errors=False*`)`

Connect to a TCP service listening on the internet *address* (a 2-tuple `(host, port)`), and return the socket object. This is a higher-level function than *socket.connect()*: if *host* is a non-numeric hostname, it will try to resolve it for both *AF_INET* and *AF_INET6*, and then try to connect to all possible addresses in turn until a connection succeeds. This makes it easy to write clients that are compatible to both IPv4 and IPv6.

Passing the optional *timeout* parameter will set the timeout on the socket instance before attempting to connect. If no *timeout* is supplied, the global default timeout setting returned by *getdefaulttimeout()* is used.

If supplied, *source_address* must be a 2-tuple `(host, port)` for the socket to bind to as its source address before connecting. If host or port are '' or 0 respectively the OS default behavior will be used.

When a connection cannot be created, an exception is raised. By default, it is the exception from the last address in the list. If *all_errors* is `True`, it is an *ExceptionGroup* containing the errors of all attempts.

Alterado na versão 3.2: *source_address* foi adicionado.

Alterado na versão 3.11: *all_errors* was added.

socket.**create_server**(*address*, *, *family=AF_INET*, *backlog=None*, *reuse_port=False*, *dualstack_ipv6=False*)

Convenience function which creates a TCP socket bound to *address* (a 2-tuple `(host, port)`) and returns the socket object.

family should be either *AF_INET* or *AF_INET6*. *backlog* is the queue size passed to *socket.listen()*; if not specified , a default reasonable value is chosen. *reuse_port* dictates whether to set the `SO_REUSEPORT` socket option.

If *dualstack_ipv6* is true, *family* is *AF_INET6* and the platform supports it the socket will be able to accept both IPv4 and IPv6 connections, else it will raise *ValueError*. Most POSIX platforms and Windows are supposed to support this functionality. When this functionality is enabled the address returned by *socket.getpeername()* when an IPv4 connection occurs will be an IPv6 address represented as an IPv4-mapped IPv6 address. If *dualstack_ipv6* is false it will explicitly disable this functionality on platforms that enable it by default (e.g. Linux). This parameter can be used in conjunction with *has_dualstack_ipv6()*:

```
import socket

addr = ("", 8080)  # all interfaces, port 8080
if socket.has_dualstack_ipv6():
    s = socket.create_server(addr, family=socket.AF_INET6, dualstack_ipv6=True)
else:
    s = socket.create_server(addr)
```

Nota

On POSIX platforms the `SO_REUSEADDR` socket option is set in order to immediately reuse previous sockets which were bound on the same *address* and remained in TIME_WAIT state.

Adicionado na versão 3.8.

socket.**has_dualstack_ipv6**()

Return `True` if the platform supports creating a TCP socket which can handle both IPv4 and IPv6 connections.

Adicionado na versão 3.8.

socket.**fromfd**(*fd*, *family*, *type*, *proto=0*)

Duplicate the file descriptor *fd* (an integer as returned by a file object's *fileno()* method) and build a socket object from the result. Address family, socket type and protocol number are as for the *socket()* function above. The file descriptor should refer to a socket, but this is not checked — subsequent operations on the object may fail if the file descriptor is invalid. This function is rarely needed, but can be used to get or set socket options on a socket passed to a program as standard input or output (such as a server started by the Unix inet daemon). The socket is assumed to be in blocking mode.

The newly created socket is *non-inheritable*.

Alterado na versão 3.4: The returned socket is now non-inheritable.

socket.**fromshare**(*data*)

Instantiate a socket from data obtained from the *socket.share()* method. The socket is assumed to be in blocking mode.

Disponibilidade: Windows.

Adicionado na versão 3.3.

socket.**SocketType**

This is a Python type object that represents the socket object type. It is the same as `type(socket(...))`.

Outras funções

The *socket* module also offers various network-related services:

socket.**close**(*fd*)

Close a socket file descriptor. This is like *os.close()*, but for sockets. On some platforms (most noticeable Windows) *os.close()* does not work for socket file descriptors.

Adicionado na versão 3.7.

socket.**getaddrinfo**(*host*, *port*, *family=AF_UNSPEC*, *type=0*, *proto=0*, *flags=0*)

This function wraps the C function `getaddrinfo` of the underlying system.

Translate the *host*/*port* argument into a sequence of 5-tuples that contain all the necessary arguments for creating a socket connected to that service. *host* is a domain name, a string representation of an IPv4/v6 address or `None`. *port* is a string service name such as `'http'`, a numeric port number or `None`. By passing `None` as the value of *host* and *port*, you can pass `NULL` to the underlying C API.

The *family*, *type* and *proto* arguments can be optionally specified in order to provide options and limit the list of addresses returned. Pass their default values (*AF_UNSPEC*, 0, and 0, respectively) to not limit the results. See the note below for details.

The *flags* argument can be one or several of the `AI_*` constants, and will influence how results are computed and returned. For example, `AI_NUMERICHOST` will disable domain name resolution and will raise an error if *host* is a domain name.

The function returns a list of 5-tuples with the following structure:

```
(family, type, proto, canonname, sockaddr)
```

In these tuples, *family*, *type*, *proto* are all integers and are meant to be passed to the *socket()* function. *canonname* will be a string representing the canonical name of the *host* if `AI_CANONNAME` is part of the *flags* argument; else *canonname* will be empty. *sockaddr* is a tuple describing a socket address, whose format depends on the returned *family* (a `(address, port)` 2-tuple for *AF_INET*, a `(address, port, flowinfo, scope_id)` 4-tuple for *AF_INET6*), and is meant to be passed to the *socket.connect()* method.

> **Nota**
>
> If you intend to use results from `getaddrinfo()` to create a socket (rather than, for example, retrieve *canonname*), consider limiting the results by *type* (e.g. *SOCK_STREAM* or *SOCK_DGRAM*) and/or *proto* (e.g. `IPPROTO_TCP` or `IPPROTO_UDP`) that your application can handle.
>
> The behavior with default values of *family*, *type*, *proto* and *flags* is system-specific.
>
> Many systems (for example, most Linux configurations) will return a sorted list of all matching addresses. These addresses should generally be tried in order until a connection succeeds (possibly tried in parallel, for example, using a Happy Eyeballs algorithm). In these cases, limiting the *type* and/or *proto* can help eliminate unsuccessful or unusable connection attempts.
>
> Some systems will, however, only return a single address. (For example, this was reported on Solaris and AIX configurations.) On these systems, limiting the *type* and/or *proto* helps ensure that this address is usable.

Levanta um *evento de auditoria* `socket.getaddrinfo` com os argumentos `host`, `port`, `family`, `type`, `protocol`.

The following example fetches address information for a hypothetical TCP connection to `example.org` on port 80 (results may differ on your system if IPv6 isn't enabled):

```
>>> socket.getaddrinfo("example.org", 80, proto=socket.IPPROTO_TCP)
[(socket.AF_INET6, socket.SOCK_STREAM,
 6, '', ('2606:2800:220:1:248:1893:25c8:1946', 80, 0, 0)),
```

(continua na próxima página)

(continuação da página anterior)

```
(socket.AF_INET, socket.SOCK_STREAM,
6, '', ('93.184.216.34', 80))]
```

Alterado na versão 3.2: parameters can now be passed using keyword arguments.

Alterado na versão 3.7: for IPv6 multicast addresses, string representing an address will not contain `%scope_id` part.

`socket.`**`getfqdn`**`(`[*name*]`)`

Return a fully qualified domain name for *name*. If *name* is omitted or empty, it is interpreted as the local host. To find the fully qualified name, the hostname returned by *gethostbyaddr()* is checked, followed by aliases for the host, if available. The first name which includes a period is selected. In case no fully qualified domain name is available and *name* was provided, it is returned unchanged. If *name* was empty or equal to `'0.0.0.0'`, the hostname from *gethostname()* is returned.

`socket.`**`gethostbyname`**`(`*hostname*`)`

Translate a host name to IPv4 address format. The IPv4 address is returned as a string, such as `'100.50.200.5'`. If the host name is an IPv4 address itself it is returned unchanged. See *gethostbyname_ex()* for a more complete interface. *gethostbyname()* does not support IPv6 name resolution, and *getaddrinfo()* should be used instead for IPv4/v6 dual stack support.

Levanta um *evento de auditoria* `socket.gethostbyname` com o argumento `hostname`.

Disponibilidade: not WASI.

`socket.`**`gethostbyname_ex`**`(`*hostname*`)`

Translate a host name to IPv4 address format, extended interface. Return a 3-tuple `(hostname, aliaslist, ipaddrlist)` where *hostname* is the host's primary host name, *aliaslist* is a (possibly empty) list of alternative host names for the same address, and *ipaddrlist* is a list of IPv4 addresses for the same interface on the same host (often but not always a single address). *gethostbyname_ex()* does not support IPv6 name resolution, and *getaddrinfo()* should be used instead for IPv4/v6 dual stack support.

Levanta um *evento de auditoria* `socket.gethostbyname` com o argumento `hostname`.

Disponibilidade: not WASI.

`socket.`**`gethostname`**`()`

Return a string containing the hostname of the machine where the Python interpreter is currently executing.

Levanta um *evento de auditoria* `socket.gethostname` sem argumentos.

Note: *gethostname()* doesn't always return the fully qualified domain name; use *getfqdn()* for that.

Disponibilidade: not WASI.

`socket.`**`gethostbyaddr`**`(`*ip_address*`)`

Return a 3-tuple `(hostname, aliaslist, ipaddrlist)` where *hostname* is the primary host name responding to the given *ip_address*, *aliaslist* is a (possibly empty) list of alternative host names for the same address, and *ipaddrlist* is a list of IPv4/v6 addresses for the same interface on the same host (most likely containing only a single address). To find the fully qualified domain name, use the function *getfqdn()*. *gethostbyaddr()* supports both IPv4 and IPv6.

Levanta um *evento de auditoria* `socket.gethostbyaddr` com o argumento `ip_address`.

Disponibilidade: not WASI.

`socket.`**`getnameinfo`**`(`*sockaddr*, *flags*`)`

Translate a socket address *sockaddr* into a 2-tuple `(host, port)`. Depending on the settings of *flags*, the result can contain a fully qualified domain name or numeric address representation in *host*. Similarly, *port* can contain a string port name or a numeric port number.

For IPv6 addresses, `%scope_id` is appended to the host part if *sockaddr* contains meaningful *scope_id*. Usually this happens for multicast addresses.

For more information about *flags* you can consult *getnameinfo(3)*.

Levanta um *evento de auditoria* `socket.getnameinfo` com o argumento `sockaddr`.

Disponibilidade: not WASI.

`socket.`**`getprotobyname`**(*protocolname*)

Translate an internet protocol name (for example, `'icmp'`) to a constant suitable for passing as the (optional) third argument to the *`socket()`* function. This is usually only needed for sockets opened in "raw" mode (*`SOCK_RAW`*); for the normal socket modes, the correct protocol is chosen automatically if the protocol is omitted or zero.

Disponibilidade: not WASI.

`socket.`**`getservbyname`**(*servicename*[, *protocolname*])

Translate an internet service name and protocol name to a port number for that service. The optional protocol name, if given, should be `'tcp'` or `'udp'`, otherwise any protocol will match.

Levanta um *evento de auditoria* `socket.getservbyname` com os argumentos `servicename`, `protocolname`.

Disponibilidade: not WASI.

`socket.`**`getservbyport`**(*port*[, *protocolname*])

Translate an internet port number and protocol name to a service name for that service. The optional protocol name, if given, should be `'tcp'` or `'udp'`, otherwise any protocol will match.

Levanta um *evento de auditoria* `socket.getservbyport` com os argumentos `port` e `protocolname`.

Disponibilidade: not WASI.

`socket.`**`ntohl`**(*x*)

Convert 32-bit positive integers from network to host byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 4-byte swap operation.

`socket.`**`ntohs`**(*x*)

Convert 16-bit positive integers from network to host byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 2-byte swap operation.

Alterado na versão 3.10: Raises *`OverflowError`* if *x* does not fit in a 16-bit unsigned integer.

`socket.`**`htonl`**(*x*)

Convert 32-bit positive integers from host to network byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 4-byte swap operation.

`socket.`**`htons`**(*x*)

Convert 16-bit positive integers from host to network byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 2-byte swap operation.

Alterado na versão 3.10: Raises *`OverflowError`* if *x* does not fit in a 16-bit unsigned integer.

`socket.`**`inet_aton`**(*ip_string*)

Convert an IPv4 address from dotted-quad string format (for example, '123.45.67.89') to 32-bit packed binary format, as a bytes object four characters in length. This is useful when conversing with a program that uses the standard C library and needs objects of type `in_addr`, which is the C type for the 32-bit packed binary this function returns.

`inet_aton()` also accepts strings with less than three dots; see the Unix manual page *`inet(3)`* for details.

If the IPv4 address string passed to this function is invalid, *`OSError`* will be raised. Note that exactly what is valid depends on the underlying C implementation of `inet_aton()`.

`inet_aton()` does not support IPv6, and *`inet_pton()`* should be used instead for IPv4/v6 dual stack support.

socket.**inet_ntoa**(*packed_ip*)

Convert a 32-bit packed IPv4 address (a *bytes-like object* four bytes in length) to its standard dotted-quad string representation (for example, '123.45.67.89'). This is useful when conversing with a program that uses the standard C library and needs objects of type `in_addr`, which is the C type for the 32-bit packed binary data this function takes as an argument.

If the byte sequence passed to this function is not exactly 4 bytes in length, *OSError* will be raised. *inet_ntoa()* does not support IPv6, and *inet_ntop()* should be used instead for IPv4/v6 dual stack support.

Alterado na versão 3.5: Writable *bytes-like object* is now accepted.

socket.**inet_pton**(*address_family*, *ip_string*)

Convert an IP address from its family-specific string format to a packed, binary format. *inet_pton()* is useful when a library or network protocol calls for an object of type `in_addr` (similar to *inet_aton()*) or `in6_addr`.

Supported values for *address_family* are currently *AF_INET* and *AF_INET6*. If the IP address string *ip_string* is invalid, *OSError* will be raised. Note that exactly what is valid depends on both the value of *address_family* and the underlying implementation of `inet_pton()`.

Disponibilidade: Unix, Windows.

Alterado na versão 3.4: Suporte para Windows adicionado.

socket.**inet_ntop**(*address_family*, *packed_ip*)

Convert a packed IP address (a *bytes-like object* of some number of bytes) to its standard, family-specific string representation (for example, `'7.10.0.5'` or `'5aef:2b::8'`). *inet_ntop()* is useful when a library or network protocol returns an object of type `in_addr` (similar to *inet_ntoa()*) or `in6_addr`.

Supported values for *address_family* are currently *AF_INET* and *AF_INET6*. If the bytes object *packed_ip* is not the correct length for the specified address family, *ValueError* will be raised. *OSError* is raised for errors from the call to *inet_ntop()*.

Disponibilidade: Unix, Windows.

Alterado na versão 3.4: Suporte para Windows adicionado.

Alterado na versão 3.5: Writable *bytes-like object* is now accepted.

socket.**CMSG_LEN**(*length*)

Return the total length, without trailing padding, of an ancillary data item with associated data of the given *length*. This value can often be used as the buffer size for *recvmsg()* to receive a single item of ancillary data, but **RFC 3542** requires portable applications to use *CMSG_SPACE()* and thus include space for padding, even when the item will be the last in the buffer. Raises *OverflowError* if *length* is outside the permissible range of values.

Disponibilidade: Unix, not WASI.

Most Unix platforms.

Adicionado na versão 3.3.

socket.**CMSG_SPACE**(*length*)

Return the buffer size needed for *recvmsg()* to receive an ancillary data item with associated data of the given *length*, along with any trailing padding. The buffer space needed to receive multiple items is the sum of the *CMSG_SPACE()* values for their associated data lengths. Raises *OverflowError* if *length* is outside the permissible range of values.

Note that some systems might support ancillary data without providing this function. Also note that setting the buffer size using the results of this function may not precisely limit the amount of ancillary data that can be received, since additional data may be able to fit into the padding area.

Disponibilidade: Unix, not WASI.

most Unix platforms.

Adicionado na versão 3.3.

socket.**getdefaulttimeout**()

Return the default timeout in seconds (float) for new socket objects. A value of None indicates that new socket objects have no timeout. When the socket module is first imported, the default is None.

socket.**setdefaulttimeout**(*timeout*)

Set the default timeout in seconds (float) for new socket objects. When the socket module is first imported, the default is None. See *settimeout()* for possible values and their respective meanings.

socket.**sethostname**(*name*)

Set the machine's hostname to *name*. This will raise an *OSError* if you don't have enough rights.

Levanta um *evento de auditoria* socket.sethostname com o argumento name.

Disponibilidade: Unix, not Android.

Adicionado na versão 3.3.

socket.**if_nameindex**()

Return a list of network interface information (index int, name string) tuples. *OSError* if the system call fails.

Disponibilidade: Unix, Windows, not WASI.

Adicionado na versão 3.3.

Alterado na versão 3.8: Suporte ao Windows foi adicionado.

Nota

On Windows network interfaces have different names in different contexts (all names are examples):

- UUID: {FB605B73-AAC2-49A6-9A2F-25416AEA0573}
- name: ethernet_32770
- friendly name: vEthernet (nat)
- description: Hyper-V Virtual Ethernet Adapter

This function returns names of the second form from the list, ethernet_32770 in this example case.

socket.**if_nametoindex**(*if_name*)

Return a network interface index number corresponding to an interface name. *OSError* if no interface with the given name exists.

Disponibilidade: Unix, Windows, not WASI.

Adicionado na versão 3.3.

Alterado na versão 3.8: Suporte ao Windows foi adicionado.

Ver também

"Interface name" is a name as documented in *if_nameindex()*.

socket.**if_indextoname**(*if_index*)

Return a network interface name corresponding to an interface index number. *OSError* if no interface with the given index exists.

Disponibilidade: Unix, Windows, not WASI.

Adicionado na versão 3.3.

Alterado na versão 3.8: Suporte ao Windows foi adicionado.

Ver também

"Interface name" is a name as documented in `if_nameindex()`.

`socket.`**`send_fds`**(*sock*, *buffers*, *fds*[, *flags*[, *address*]])

Send the list of file descriptors *fds* over an `AF_UNIX` socket *sock*. The *fds* parameter is a sequence of file descriptors. Consult `sendmsg()` for the documentation of these parameters.

Disponibilidade: Unix, not WASI.

Unix platforms supporting `sendmsg()` and `SCM_RIGHTS` mechanism.

Adicionado na versão 3.9.

`socket.`**`recv_fds`**(*sock*, *bufsize*, *maxfds*[, *flags*])

Receive up to *maxfds* file descriptors from an `AF_UNIX` socket *sock*. Return `(msg, list(fds), flags, addr)`. Consult `recvmsg()` for the documentation of these parameters.

Disponibilidade: Unix, not WASI.

Unix platforms supporting `recvmsg()` and `SCM_RIGHTS` mechanism.

Adicionado na versão 3.9.

Nota

Any truncated integers at the end of the list of file descriptors.

19.2.3 Socket Objects

Socket objects have the following methods. Except for `makefile()`, these correspond to Unix system calls applicable to sockets.

Alterado na versão 3.2: Support for the *context manager* protocol was added. Exiting the context manager is equivalent to calling `close()`.

`socket.`**`accept`**()

Accept a connection. The socket must be bound to an address and listening for connections. The return value is a pair `(conn, address)` where *conn* is a *new* socket object usable to send and receive data on the connection, and *address* is the address bound to the socket on the other end of the connection.

The newly created socket is *non-inheritable*.

Alterado na versão 3.4: The socket is now non-inheritable.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** for the rationale).

`socket.`**`bind`**(*address*)

Bind the socket to *address*. The socket must not already be bound. (The format of *address* depends on the address family — see above.)

Levanta um *evento de auditoria* `socket.bind` com os argumentos `self`, `address`.

Disponibilidade: not WASI.

`socket.`**`close`**()

Mark the socket closed. The underlying system resource (e.g. a file descriptor) is also closed when all file objects from `makefile()` are closed. Once that happens, all future operations on the socket object will fail. The remote end will receive no more data (after queued data is flushed).

Sockets are automatically closed when they are garbage-collected, but it is recommended to `close()` them explicitly, or to use a `with` statement around them.

Alterado na versão 3.6: *OSError* is now raised if an error occurs when the underlying `close()` call is made.

> **Nota**
>
> *close()* releases the resource associated with a connection but does not necessarily close the connection immediately. If you want to close the connection in a timely fashion, call *shutdown()* before *close()*.

`socket.`**`connect`**`(`*address*`)`

Connect to a remote socket at *address*. (The format of *address* depends on the address family — see above.)

If the connection is interrupted by a signal, the method waits until the connection completes, or raise a *TimeoutError* on timeout, if the signal handler doesn't raise an exception and the socket is blocking or has a timeout. For non-blocking sockets, the method raises an *InterruptedError* exception if the connection is interrupted by a signal (or the exception raised by the signal handler).

Levanta um *evento de auditoria* `socket.connect` com os argumentos `self`, `address`.

Alterado na versão 3.5: The method now waits until the connection completes instead of raising an *InterruptedError* exception if the connection is interrupted by a signal, the signal handler doesn't raise an exception and the socket is blocking or has a timeout (see the **PEP 475** for the rationale).

Disponibilidade: not WASI.

`socket.`**`connect_ex`**`(`*address*`)`

Like `connect(address)`, but return an error indicator instead of raising an exception for errors returned by the C-level `connect()` call (other problems, such as "host not found," can still raise exceptions). The error indicator is `0` if the operation succeeded, otherwise the value of the `errno` variable. This is useful to support, for example, asynchronous connects.

Levanta um *evento de auditoria* `socket.connect` com os argumentos `self`, `address`.

Disponibilidade: not WASI.

`socket.`**`detach`**`()`

Put the socket object into closed state without actually closing the underlying file descriptor. The file descriptor is returned, and can be reused for other purposes.

Adicionado na versão 3.2.

`socket.`**`dup`**`()`

Duplicate the socket.

The newly created socket is *non-inheritable*.

Alterado na versão 3.4: The socket is now non-inheritable.

Disponibilidade: not WASI.

`socket.`**`fileno`**`()`

Return the socket's file descriptor (a small integer), or -1 on failure. This is useful with *select.select()*.

Under Windows the small integer returned by this method cannot be used where a file descriptor can be used (such as *os.fdopen()*). Unix does not have this limitation.

`socket.`**`get_inheritable`**`()`

Get the *inheritable flag* of the socket's file descriptor or socket's handle: `True` if the socket can be inherited in child processes, `False` if it cannot.

Adicionado na versão 3.4.

`socket.`**`getpeername`**`()`

Return the remote address to which the socket is connected. This is useful to find out the port number of a remote IPv4/v6 socket, for instance. (The format of the address returned depends on the address family — see above.) On some systems this function is not supported.

socket.**getsockname**()

Return the socket's own address. This is useful to find out the port number of an IPv4/v6 socket, for instance. (The format of the address returned depends on the address family — see above.)

socket.**getsockopt**(*level*, *optname*[, *buflen*])

Return the value of the given socket option (see the Unix man page *getsockopt(2)*). The needed symbolic constants (*SO_* etc.*) are defined in this module. If *buflen* is absent, an integer option is assumed and its integer value is returned by the function. If *buflen* is present, it specifies the maximum length of the buffer used to receive the option in, and this buffer is returned as a bytes object. It is up to the caller to decode the contents of the buffer (see the optional built-in module *struct* for a way to decode C structures encoded as byte strings).

Disponibilidade: not WASI.

socket.**getblocking**()

Return `True` if socket is in blocking mode, `False` if in non-blocking.

This is equivalent to checking `socket.gettimeout() != 0`.

Adicionado na versão 3.7.

socket.**gettimeout**()

Return the timeout in seconds (float) associated with socket operations, or `None` if no timeout is set. This reflects the last call to *setblocking()* or *settimeout()*.

socket.**ioctl**(*control*, *option*)

The *ioctl()* method is a limited interface to the WSAIoctl system interface. Please refer to the Win32 documentation for more information.

On other platforms, the generic *fcntl.fcntl()* and *fcntl.ioctl()* functions may be used; they accept a socket object as their first argument.

Currently only the following control codes are supported: `SIO_RCVALL`, `SIO_KEEPALIVE_VALS`, and `SIO_LOOPBACK_FAST_PATH`.

Disponibilidade: Windows

Alterado na versão 3.6: `SIO_LOOPBACK_FAST_PATH` was added.

socket.**listen**([*backlog*])

Enable a server to accept connections. If *backlog* is specified, it must be at least 0 (if it is lower, it is set to 0); it specifies the number of unaccepted connections that the system will allow before refusing new connections. If not specified, a default reasonable value is chosen.

Disponibilidade: not WASI.

Alterado na versão 3.5: The *backlog* parameter is now optional.

socket.**makefile**(*mode='r'*, *buffering=None*, ***, *encoding=None*, *errors=None*, *newline=None*)

Return a *file object* associated with the socket. The exact returned type depends on the arguments given to *makefile()*. These arguments are interpreted the same way as by the built-in *open()* function, except the only supported *mode* values are `'r'` (default), `'w'`, `'b'`, or a combination of those.

The socket must be in blocking mode; it can have a timeout, but the file object's internal buffer may end up in an inconsistent state if a timeout occurs.

Closing the file object returned by *makefile()* won't close the original socket unless all other file objects have been closed and *socket.close()* has been called on the socket object.

Nota

On Windows, the file-like object created by *makefile()* cannot be used where a file object with a file descriptor is expected, such as the stream arguments of *subprocess.Popen()*.

socket.**recv**(*bufsize*[, *flags*])

Receive data from the socket. The return value is a bytes object representing the data received. The maximum amount of data to be received at once is specified by *bufsize*. A returned empty bytes object indicates that the client has disconnected. See the Unix manual page *recv(2)* for the meaning of the optional argument *flags*; it defaults to zero.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an *InterruptedError* exception (see **PEP 475** for the rationale).

socket.**recvfrom**(*bufsize*[, *flags*])

Receive data from the socket. The return value is a pair `(bytes, address)` where *bytes* is a bytes object representing the data received and *address* is the address of the socket sending the data. See the Unix manual page *recv(2)* for the meaning of the optional argument *flags*; it defaults to zero. (The format of *address* depends on the address family — see above.)

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an *InterruptedError* exception (see **PEP 475** for the rationale).

Alterado na versão 3.7: For multicast IPv6 address, first item of *address* does not contain `%scope_id` part anymore. In order to get full IPv6 address use *getnameinfo()*.

socket.**recvmsg**(*bufsize*[, *ancbufsize*[, *flags*]])

Receive normal data (up to *bufsize* bytes) and ancillary data from the socket. The *ancbufsize* argument sets the size in bytes of the internal buffer used to receive the ancillary data; it defaults to 0, meaning that no ancillary data will be received. Appropriate buffer sizes for ancillary data can be calculated using *CMSG_SPACE()* or *CMSG_LEN()*, and items which do not fit into the buffer might be truncated or discarded. The *flags* argument defaults to 0 and has the same meaning as for *recv()*.

The return value is a 4-tuple: `(data, ancdata, msg_flags, address)`. The *data* item is a *bytes* object holding the non-ancillary data received. The *ancdata* item is a list of zero or more tuples `(cmsg_level, cmsg_type, cmsg_data)` representing the ancillary data (control messages) received: *cmsg_level* and *cmsg_type* are integers specifying the protocol level and protocol-specific type respectively, and *cmsg_data* is a *bytes* object holding the associated data. The *msg_flags* item is the bitwise OR of various flags indicating conditions on the received message; see your system documentation for details. If the receiving socket is unconnected, *address* is the address of the sending socket, if available; otherwise, its value is unspecified.

On some systems, *sendmsg()* and *recvmsg()* can be used to pass file descriptors between processes over an *AF_UNIX* socket. When this facility is used (it is often restricted to *SOCK_STREAM* sockets), *recvmsg()* will return, in its ancillary data, items of the form `(socket.SOL_SOCKET, socket.SCM_RIGHTS, fds)`, where *fds* is a *bytes* object representing the new file descriptors as a binary array of the native C `int` type. If *recvmsg()* raises an exception after the system call returns, it will first attempt to close any file descriptors received via this mechanism.

Some systems do not indicate the truncated length of ancillary data items which have been only partially received. If an item appears to extend beyond the end of the buffer, *recvmsg()* will issue a *RuntimeWarning*, and will return the part of it which is inside the buffer provided it has not been truncated before the start of its associated data.

On systems which support the `SCM_RIGHTS` mechanism, the following function will receive up to *maxfds* file descriptors, returning the message data and a list containing the descriptors (while ignoring unexpected conditions such as unrelated control messages being received). See also *sendmsg()*.

```
import socket, array

def recv_fds(sock, msglen, maxfds):
    fds = array.array("i")   # Array of ints
    msg, ancdata, flags, addr = sock.recvmsg(msglen, socket.CMSG_LEN(maxfds *␣
↪fds.itemsize))
    for cmsg_level, cmsg_type, cmsg_data in ancdata:
```

(continua na próxima página)

(continuação da página anterior)

```
        if cmsg_level == socket.SOL_SOCKET and cmsg_type == socket.SCM_RIGHTS:
            # Append data, ignoring any truncated integers at the end.
            fds.frombytes(cmsg_data[:len(cmsg_data) - (len(cmsg_data) % fds.
↪itemsize)])
    return msg, list(fds)
```

Disponibilidade: Unix.

Most Unix platforms.

Adicionado na versão 3.3.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** for the rationale).

`socket.`**`recvmsg_into`**(*buffers*[, *ancbufsize*[, *flags*]])

Receive normal data and ancillary data from the socket, behaving as `recvmsg()` would, but scatter the non-ancillary data into a series of buffers instead of returning a new bytes object. The *buffers* argument must be an iterable of objects that export writable buffers (e.g. `bytearray` objects); these will be filled with successive chunks of the non-ancillary data until it has all been written or there are no more buffers. The operating system may set a limit (`sysconf()` value `SC_IOV_MAX`) on the number of buffers that can be used. The *ancbufsize* and *flags* arguments have the same meaning as for `recvmsg()`.

The return value is a 4-tuple: `(nbytes, ancdata, msg_flags, address)`, where *nbytes* is the total number of bytes of non-ancillary data written into the buffers, and *ancdata*, *msg_flags* and *address* are the same as for `recvmsg()`.

Exemplo:

```
>>> import socket
>>> s1, s2 = socket.socketpair()
>>> b1 = bytearray(b'----')
>>> b2 = bytearray(b'0123456789')
>>> b3 = bytearray(b'--------------')
>>> s1.send(b'Mary had a little lamb')
22
>>> s2.recvmsg_into([b1, memoryview(b2)[2:9], b3])
(22, [], 0, None)
>>> [b1, b2, b3]
[bytearray(b'Mary'), bytearray(b'01 had a 9'), bytearray(b'little lamb---')]
```

Disponibilidade: Unix.

Most Unix platforms.

Adicionado na versão 3.3.

`socket.`**`recvfrom_into`**(*buffer*[, *nbytes*[, *flags*]])

Receive data from the socket, writing it into *buffer* instead of creating a new bytestring. The return value is a pair `(nbytes, address)` where *nbytes* is the number of bytes received and *address* is the address of the socket sending the data. See the Unix manual page `recv(2)` for the meaning of the optional argument *flags*; it defaults to zero. (The format of *address* depends on the address family — see above.)

`socket.`**`recv_into`**(*buffer*[, *nbytes*[, *flags*]])

Receive up to *nbytes* bytes from the socket, storing the data into a buffer rather than creating a new bytestring. If *nbytes* is not specified (or 0), receive up to the size available in the given buffer. Returns the number of bytes received. See the Unix manual page `recv(2)` for the meaning of the optional argument *flags*; it defaults to zero.

socket.**send**(*bytes*[, *flags*])

Send data to the socket. The socket must be connected to a remote socket. The optional *flags* argument has the same meaning as for `recv()` above. Returns the number of bytes sent. Applications are responsible for checking that all data has been sent; if only some of the data was transmitted, the application needs to attempt delivery of the remaining data. For further information on this topic, consult the socket-howto.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** for the rationale).

socket.**sendall**(*bytes*[, *flags*])

Send data to the socket. The socket must be connected to a remote socket. The optional *flags* argument has the same meaning as for `recv()` above. Unlike `send()`, this method continues to send data from *bytes* until either all data has been sent or an error occurs. `None` is returned on success. On error, an exception is raised, and there is no way to determine how much data, if any, was successfully sent.

Alterado na versão 3.5: The socket timeout is no longer reset each time data is sent successfully. The socket timeout is now the maximum total duration to send all data.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** for the rationale).

socket.**sendto**(*bytes*, *address*)

socket.**sendto**(*bytes*, *flags*, *address*)

Send data to the socket. The socket should not be connected to a remote socket, since the destination socket is specified by *address*. The optional *flags* argument has the same meaning as for `recv()` above. Return the number of bytes sent. (The format of *address* depends on the address family — see above.)

Levanta um *evento de auditoria* `socket.sendto` com os argumentos `self`, `address`.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** for the rationale).

socket.**sendmsg**(*buffers*[, *ancdata*[, *flags*[, *address*]]])

Send normal and ancillary data to the socket, gathering the non-ancillary data from a series of buffers and concatenating it into a single message. The *buffers* argument specifies the non-ancillary data as an iterable of *bytes-like objects* (e.g. `bytes` objects); the operating system may set a limit (`sysconf()` value `SC_IOV_MAX`) on the number of buffers that can be used. The *ancdata* argument specifies the ancillary data (control messages) as an iterable of zero or more tuples `(cmsg_level, cmsg_type, cmsg_data)`, where *cmsg_level* and *cmsg_type* are integers specifying the protocol level and protocol-specific type respectively, and *cmsg_data* is a bytes-like object holding the associated data. Note that some systems (in particular, systems without `CMSG_SPACE()`) might support sending only one control message per call. The *flags* argument defaults to 0 and has the same meaning as for `send()`. If *address* is supplied and not `None`, it sets a destination address for the message. The return value is the number of bytes of non-ancillary data sent.

The following function sends the list of file descriptors *fds* over an `AF_UNIX` socket, on systems which support the `SCM_RIGHTS` mechanism. See also `recvmsg()`.

```
import socket, array

def send_fds(sock, msg, fds):
    return sock.sendmsg([msg], [(socket.SOL_SOCKET, socket.SCM_RIGHTS, array.
↪array("i", fds))])
```

Disponibilidade: Unix, not WASI.

Most Unix platforms.

Levanta um *evento de auditoria* `socket.sendmsg` com os argumentos `self`, `address`.

Adicionado na versão 3.3.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** for the rationale).

socket.**sendmsg_afalg**([*msg*,]*, *op*[, *iv*[, *assoclen*[, *flags*]]])

Specialized version of `sendmsg()` for `AF_ALG` socket. Set mode, IV, AEAD associated data length and flags for `AF_ALG` socket.

Disponibilidade: Linux >= 2.6.38.

Adicionado na versão 3.6.

socket.**sendfile**(*file*, *offset=0*, *count=None*)

Send a file until EOF is reached by using high-performance `os.sendfile` and return the total number of bytes which were sent. *file* must be a regular file object opened in binary mode. If `os.sendfile` is not available (e.g. Windows) or *file* is not a regular file `send()` will be used instead. *offset* tells from where to start reading the file. If specified, *count* is the total number of bytes to transmit as opposed to sending the file until EOF is reached. File position is updated on return or also in case of error in which case `file.tell()` can be used to figure out the number of bytes which were sent. The socket must be of `SOCK_STREAM` type. Non-blocking sockets are not supported.

Adicionado na versão 3.5.

socket.**set_inheritable**(*inheritable*)

Set the *inheritable flag* of the socket's file descriptor or socket's handle.

Adicionado na versão 3.4.

socket.**setblocking**(*flag*)

Set blocking or non-blocking mode of the socket: if *flag* is false, the socket is set to non-blocking, else to blocking mode.

This method is a shorthand for certain `settimeout()` calls:

- `sock.setblocking(True)` is equivalent to `sock.settimeout(None)`
- `sock.setblocking(False)` is equivalent to `sock.settimeout(0.0)`

Alterado na versão 3.7: The method no longer applies `SOCK_NONBLOCK` flag on `socket.type`.

socket.**settimeout**(*value*)

Set a timeout on blocking socket operations. The *value* argument can be a nonnegative floating-point number expressing seconds, or `None`. If a non-zero value is given, subsequent socket operations will raise a `timeout` exception if the timeout period *value* has elapsed before the operation has completed. If zero is given, the socket is put in non-blocking mode. If `None` is given, the socket is put in blocking mode.

For further information, please consult the *notes on socket timeouts*.

Alterado na versão 3.7: The method no longer toggles `SOCK_NONBLOCK` flag on `socket.type`.

socket.**setsockopt**(*level*, *optname*, *value:* int)

socket.**setsockopt**(*level*, *optname*, *value: buffer*)

socket.**setsockopt**(*level*, *optname*, *None*, *optlen: int*)

Set the value of the given socket option (see the Unix manual page `setsockopt(2)`). The needed symbolic constants are defined in this module (SO_* etc. <socket-unix-constants>). The value can be an integer, `None` or a *bytes-like object* representing a buffer. In the later case it is up to the caller to ensure that the bytestring contains the proper bits (see the optional built-in module `struct` for a way to encode C structures as bytestrings). When *value* is set to `None`, *optlen* argument is required. It's equivalent to call `setsockopt()` C function with `optval=NULL` and `optlen=optlen`.

Alterado na versão 3.5: Writable *bytes-like object* is now accepted.

Alterado na versão 3.6: setsockopt(level, optname, None, optlen: int) form added.

Disponibilidade: not WASI.

socket.**shutdown**(*how*)

Shut down one or both halves of the connection. If *how* is `SHUT_RD`, further receives are disallowed. If *how* is `SHUT_WR`, further sends are disallowed. If *how* is `SHUT_RDWR`, further sends and receives are disallowed.

Disponibilidade: not WASI.

socket.**share**(*process_id*)

Duplicate a socket and prepare it for sharing with a target process. The target process must be provided with *process_id*. The resulting bytes object can then be passed to the target process using some form of interprocess communication and the socket can be recreated there using `fromshare()`. Once this method has been called, it is safe to close the socket since the operating system has already duplicated it for the target process.

Disponibilidade: Windows.

Adicionado na versão 3.3.

Note that there are no methods `read()` or `write()`; use `recv()` and `send()` without *flags* argument instead.

Socket objects also have these (read-only) attributes that correspond to the values given to the `socket` constructor.

socket.**family**

The socket family.

socket.**type**

The socket type.

socket.**proto**

The socket protocol.

19.2.4 Notas sobre tempo limite de soquete

Um objeto soquete pode estar em um dos três modos: bloqueio, não-bloqueio ou tempo limite. Por padrão, os soquetes sempre são criados no modo de bloqueio, mas isso pode ser alterado chamando `setdefaulttimeout()`.

- In *blocking mode*, operations block until complete or the system returns an error (such as connection timed out).
- In *non-blocking mode*, operations fail (with an error that is unfortunately system-dependent) if they cannot be completed immediately: functions from the `select` module can be used to know when and whether a socket is available for reading or writing.
- In *timeout mode*, operations fail if they cannot be completed within the timeout specified for the socket (they raise a `timeout` exception) or if the system returns an error.

Nota

At the operating system level, sockets in *timeout mode* are internally set in non-blocking mode. Also, the blocking and timeout modes are shared between file descriptors and socket objects that refer to the same network endpoint. This implementation detail can have visible consequences if e.g. you decide to use the `fileno()` of a socket.

Timeouts and the `connect` method

The `connect()` operation is also subject to the timeout setting, and in general it is recommended to call `settimeout()` before calling `connect()` or pass a timeout parameter to `create_connection()`. However, the system network stack may also return a connection timeout error of its own regardless of any Python socket timeout setting.

Timeouts and the `accept` method

If `getdefaulttimeout()` is not `None`, sockets returned by the `accept()` method inherit that timeout. Otherwise, the behaviour depends on settings of the listening socket:

- if the listening socket is in *blocking mode* or in *timeout mode*, the socket returned by `accept()` is in *blocking mode*;
- if the listening socket is in *non-blocking mode*, whether the socket returned by `accept()` is in blocking or non-blocking mode is operating system-dependent. If you want to ensure cross-platform behaviour, it is recommended you manually override this setting.

19.2.5 Exemplo

Here are four minimal example programs using the TCP/IP protocol: a server that echoes all data that it receives back (servicing only one client), and a client using it. Note that a server must perform the sequence `socket()`, `bind()`, `listen()`, `accept()` (possibly repeating the `accept()` to service more than one client), while a client only needs the sequence `socket()`, `connect()`. Also note that the server does not `sendall()`/`recv()` on the socket it is listening on but on the new socket returned by `accept()`.

The first two examples support IPv4 only.

```
# Echo server program
import socket

HOST = ''                 # Symbolic name meaning all available interfaces
PORT = 50007              # Arbitrary non-privileged port
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen(1)
    conn, addr = s.accept()
    with conn:
        print('Connected by', addr)
        while True:
            data = conn.recv(1024)
            if not data: break
            conn.sendall(data)
```

```
# Echo client program
import socket

HOST = 'daring.cwi.nl'    # The remote host
PORT = 50007              # The same port as used by the server
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b'Hello, world')
    data = s.recv(1024)
print('Received', repr(data))
```

The next two examples are identical to the above two, but support both IPv4 and IPv6. The server side will listen to the first address family available (it should listen to both instead). On most of IPv6-ready systems, IPv6 will take precedence and the server may not accept IPv4 traffic. The client side will try to connect to all the addresses returned as a result of the name resolution, and sends traffic to the first one connected successfully.

```
# Echo server program
import socket
import sys

HOST = None               # Symbolic name meaning all available interfaces
PORT = 50007              # Arbitrary non-privileged port
s = None
for res in socket.getaddrinfo(HOST, PORT, socket.AF_UNSPEC,
                              socket.SOCK_STREAM, 0, socket.AI_PASSIVE):
    af, socktype, proto, canonname, sa = res
```

(continua na próxima página)

(continuação da página anterior)

```
    try:
        s = socket.socket(af, socktype, proto)
    except OSError as msg:
        s = None
        continue
    try:
        s.bind(sa)
        s.listen(1)
    except OSError as msg:
        s.close()
        s = None
        continue
    break
if s is None:
    print('could not open socket')
    sys.exit(1)
conn, addr = s.accept()
with conn:
    print('Connected by', addr)
    while True:
        data = conn.recv(1024)
        if not data: break
        conn.send(data)
```

```
# Echo client program
import socket
import sys

HOST = 'daring.cwi.nl'    # The remote host
PORT = 50007              # The same port as used by the server
s = None
for res in socket.getaddrinfo(HOST, PORT, socket.AF_UNSPEC, socket.SOCK_STREAM):
    af, socktype, proto, canonname, sa = res
    try:
        s = socket.socket(af, socktype, proto)
    except OSError as msg:
        s = None
        continue
    try:
        s.connect(sa)
    except OSError as msg:
        s.close()
        s = None
        continue
    break
if s is None:
    print('could not open socket')
    sys.exit(1)
with s:
    s.sendall(b'Hello, world')
    data = s.recv(1024)
print('Received', repr(data))
```

The next example shows how to write a very simple network sniffer with raw sockets on Windows. The example requires administrator privileges to modify the interface:

```
import socket

# the public network interface
HOST = socket.gethostbyname(socket.gethostname())

# create a raw socket and bind it to the public interface
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
s.bind((HOST, 0))

# Include IP headers
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)

# receive all packets
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

# receive a packet
print(s.recvfrom(65565))

# disabled promiscuous mode
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
```

The next example shows how to use the socket interface to communicate to a CAN network using the raw socket protocol. To use CAN with the broadcast manager protocol instead, open a socket with:

```
socket.socket(socket.AF_CAN, socket.SOCK_DGRAM, socket.CAN_BCM)
```

After binding (`CAN_RAW`) or connecting (*`CAN_BCM`*) the socket, you can use the *`socket.send()`* and *`socket.recv()`* operations (and their counterparts) on the socket object as usual.

This last example might require special privileges:

```
import socket
import struct


# CAN frame packing/unpacking (see 'struct can_frame' in <linux/can.h>)

can_frame_fmt = "=IB3x8s"
can_frame_size = struct.calcsize(can_frame_fmt)

def build_can_frame(can_id, data):
    can_dlc = len(data)
    data = data.ljust(8, b'\x00')
    return struct.pack(can_frame_fmt, can_id, can_dlc, data)

def dissect_can_frame(frame):
    can_id, can_dlc, data = struct.unpack(can_frame_fmt, frame)
    return (can_id, can_dlc, data[:can_dlc])


# create a raw socket and bind it to the 'vcan0' interface
s = socket.socket(socket.AF_CAN, socket.SOCK_RAW, socket.CAN_RAW)
s.bind(('vcan0',))

while True:
    cf, addr = s.recvfrom(can_frame_size)

    print('Received: can_id=%x, can_dlc=%x, data=%s' % dissect_can_frame(cf))
```

(continua na próxima página)

(continuação da página anterior)

```
    try:
        s.send(cf)
    except OSError:
        print('Error sending CAN frame')

    try:
        s.send(build_can_frame(0x01, b'\x01\x02\x03'))
    except OSError:
        print('Error sending CAN frame')
```

Running an example several times with too small delay between executions, could lead to this error:

```
OSError: [Errno 98] Address already in use
```

This is because the previous execution has left the socket in a `TIME_WAIT` state, and can't be immediately reused.

There is a *socket* flag to set, in order to prevent this, `socket.SO_REUSEADDR`:

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind((HOST, PORT))
```

the `SO_REUSEADDR` flag tells the kernel to reuse a local socket in `TIME_WAIT` state, without waiting for its natural timeout to expire.

Ver também

Para uma introdução à programação de socket (em C), veja os seguintes artigos:

- *An Introductory 4.3BSD Interprocess Communication Tutorial*, by Stuart Sechrest
- *An Advanced 4.3BSD Interprocess Communication Tutorial*, by Samuel J. Leffler et al,

both in the UNIX Programmer's Manual, Supplementary Documents 1 (sections PS1:7 and PS1:8). The platform-specific reference material for the various socket-related system calls are also a valuable source of information on the details of socket semantics. For Unix, refer to the manual pages; for Windows, see the WinSock (or Winsock 2) specification. For IPv6-ready APIs, readers may want to refer to **RFC 3493** titled Basic Socket Interface Extensions for IPv6.

19.3 `ssl` — Invólucro de TLS/SSL para objetos socket

Código-fonte: Lib/ssl.py

Este módulo fornece acesso à criptografia Transport Layer Security (frequentemente conhecida como "Secure Sockets Layer") e aos recursos de autenticação de pares para sockets de rede, tanto do lado do cliente quanto do lado do servidor. Este módulo usa a biblioteca OpenSSL. Ela está disponível em todos os sistemas Unix modernos, Windows, macOS e provavelmente plataformas adicionais, desde que o OpenSSL esteja instalado nessa plataforma.

Nota

Alguns comportamentos podem depender da plataforma, já que chamadas são feitas para as APIs de socket do sistema operacional. A versão instalada do OpenSSL também pode causar variações no comportamento. Por exemplo, o TLSv1.3 vem com a versão 1.1.1 do OpenSSL.

Aviso

Não use este módulo sem ler o *Considerações de segurança*. Fazer isso pode levar a uma falsa sensação de segurança, pois as configurações padrão do módulo ssl não são necessariamente apropriadas para sua aplicação.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

Esta seção documenta os objetos e funções no módulo `ssl`; para obter mais informações gerais sobre TLS, SSL e certificados, o leitor pode consultar os documentos na seção "Consulte também" na parte inferior.

Este módulo fornece uma classe, *`ssl.SSLSocket`*, que é derivada do tipo *`socket.socket`*, e fornece um wrapper semelhante a um socket que também criptografa e descriptografa os dados que passam pelo socket com SSL. Ele suporta métodos adicionais como `getpeercert()`, que recupera o certificado do outro lado da conexão, `cipher()`, que recupera a cifra sendo usada para a conexão segura ou `get_verified_chain()`, `get_unverified_chain()` que recupera a cadeia de certificados.

Para aplicações mais sofisticadas, a classe *`ssl.SSLContext`* ajuda a gerenciar configurações e certificados, que podem ser herdados por soquetes SSL criados por meio do método *`SSLContext.wrap_socket()`*.

Alterado na versão 3.5.3: Atualizado para oferecer suporte à vinculação com OpenSSL 1.1.0

Alterado na versão 3.6: OpenSSL 0.9.8, 1.0.0 e 1.0.1 estão descontinuado e não são mais suportados. No futuro, o módulo ssl exigirá pelo menos OpenSSL 1.0.2 ou 1.1.0.

Alterado na versão 3.10: **PEP 644** foi implementado. O módulo ssl requer OpenSSL 1.1.1 ou mais recente.

O uso de constantes e funções descontinuados resulta em avisos de descontinuação.

19.3.1 Funções, constantes e exceções.

Criação de socket

Instâncias de *`SSLSocket`* devem ser criadas usando o método *`SSLContext.wrap_socket()`*. A função auxiliar *`create_default_context()`* retorna um novo contexto com configurações padrão seguras.

Exemplo de soquete de cliente com contexto padrão e pilha dupla IPv4/IPv6:

```
import socket
import ssl

hostname = 'www.python.org'
context = ssl.create_default_context()

with socket.create_connection((hostname, 443)) as sock:
    with context.wrap_socket(sock, server_hostname=hostname) as ssock:
        print(ssock.version())
```

Exemplo de soquete de cliente com contexto personalizado e IPv4:

```
hostname = 'www.python.org'
# PROTOCOL_TLS_CLIENT requer hostname e cadeia de certificados
context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
context.load_verify_locations('path/to/cabundle.pem')

with socket.socket(socket.AF_INET, socket.SOCK_STREAM, 0) as sock:
    with context.wrap_socket(sock, server_hostname=hostname) as ssock:
        print(ssock.version())
```

Exemplo de soquete de servidor escutando no localhost IPv4:

```
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain('/caminho/para/cadeia-certicado.pem', '/caminho/para/
↪privado.key')

with socket.socket(socket.AF_INET, socket.SOCK_STREAM, 0) as sock:
    sock.bind(('127.0.0.1', 8443))
    sock.listen(5)
    with context.wrap_socket(sock, server_side=True) as ssock:
        conn, addr = ssock.accept()
        ...
```

Criação de contexto

A convenience function helps create *SSLContext* objects for common purposes.

ssl.**create_default_context**(*purpose=Purpose.SERVER_AUTH*, *, *cafile=None*, *capath=None*, *cadata=None*)

Return a new *SSLContext* object with default settings for the given *purpose*. The settings are chosen by the *ssl* module, and usually represent a higher security level than when calling the *SSLContext* constructor directly.

cafile, *capath*, *cadata* represent optional CA certificates to trust for certificate verification, as in *SSLContext.load_verify_locations()*. If all three are *None*, this function can choose to trust the system's default CA certificates instead.

The settings are: *PROTOCOL_TLS_CLIENT* or *PROTOCOL_TLS_SERVER*, *OP_NO_SSLv2*, and *OP_NO_SSLv3* with high encryption cipher suites without RC4 and without unauthenticated cipher suites. Passing *SERVER_AUTH* as *purpose* sets *verify_mode* to *CERT_REQUIRED* and either loads CA certificates (when at least one of *cafile*, *capath* or *cadata* is given) or uses *SSLContext.load_default_certs()* to load default CA certificates.

When *keylog_filename* is supported and the environment variable SSLKEYLOGFILE is set, *create_default_context()* enables key logging.

The default settings for this context include *VERIFY_X509_PARTIAL_CHAIN* and *VERIFY_X509_STRICT*. These make the underlying OpenSSL implementation behave more like a conforming implementation of **RFC 5280**, in exchange for a small amount of incompatibility with older X.509 certificates.

Nota

The protocol, options, cipher and other settings may change to more restrictive values anytime without prior deprecation. The values represent a fair balance between compatibility and security.

If your application needs specific settings, you should create a *SSLContext* and apply the settings yourself.

Nota

If you find that when certain older clients or servers attempt to connect with a *SSLContext* created by this function that they get an error stating "Protocol or cipher suite mismatch", it may be that they only support SSL3.0 which this function excludes using the *OP_NO_SSLv3*. SSL3.0 is widely considered to be completely broken. If you still wish to continue to use this function but still allow SSL 3.0 connections you can re-enable them using:

```
ctx = ssl.create_default_context(Purpose.CLIENT_AUTH)
ctx.options &= ~ssl.OP_NO_SSLv3
```

Nota

This context enables *VERIFY_X509_STRICT* by default, which may reject pre-**RFC 5280** or malformed certificates that the underlying OpenSSL implementation otherwise would accept. While disabling this is not recommended, you can do so using:

```
ctx = ssl.create_default_context()
ctx.verify_flags &= ~ssl.VERIFY_X509_STRICT
```

Adicionado na versão 3.4.

Alterado na versão 3.4.4: RC4 was dropped from the default cipher string.

Alterado na versão 3.6: ChaCha20/Poly1305 was added to the default cipher string.

3DES was dropped from the default cipher string.

Alterado na versão 3.8: Support for key logging to `SSLKEYLOGFILE` was added.

Alterado na versão 3.10: The context now uses *PROTOCOL_TLS_CLIENT* or *PROTOCOL_TLS_SERVER* protocol instead of generic *PROTOCOL_TLS*.

Alterado na versão 3.13: The context now uses *VERIFY_X509_PARTIAL_CHAIN* and *VERIFY_X509_STRICT* in its default verify flags.

Exceções

exception ssl.**SSLError**

Raised to signal an error from the underlying SSL implementation (currently provided by the OpenSSL library). This signifies some problem in the higher-level encryption and authentication layer that's superimposed on the underlying network connection. This error is a subtype of *OSError*. The error code and message of *SSLError* instances are provided by the OpenSSL library.

Alterado na versão 3.3: *SSLError* used to be a subtype of *socket.error*.

library

A string mnemonic designating the OpenSSL submodule in which the error occurred, such as `SSL`, `PEM` or `X509`. The range of possible values depends on the OpenSSL version.

Adicionado na versão 3.3.

reason

A string mnemonic designating the reason this error occurred, for example `CERTIFICATE_VERIFY_FAILED`. The range of possible values depends on the OpenSSL version.

Adicionado na versão 3.3.

exception ssl.**SSLZeroReturnError**

A subclass of *SSLError* raised when trying to read or write and the SSL connection has been closed cleanly. Note that this doesn't mean that the underlying transport (read TCP) has been closed.

Adicionado na versão 3.3.

exception ssl.**SSLWantReadError**

A subclass of *SSLError* raised by a *non-blocking SSL socket* when trying to read or write data, but more data needs to be received on the underlying TCP transport before the request can be fulfilled.

Adicionado na versão 3.3.

exception ssl.**SSLWantWriteError**

A subclass of *SSLError* raised by a *non-blocking SSL socket* when trying to read or write data, but more data needs to be sent on the underlying TCP transport before the request can be fulfilled.

Adicionado na versão 3.3.

exception ssl.**SSLSyscallError**

A subclass of *SSLError* raised when a system error was encountered while trying to fulfill an operation on a SSL socket. Unfortunately, there is no easy way to inspect the original errno number.

Adicionado na versão 3.3.

exception ssl.**SSLEOFError**

A subclass of *SSLError* raised when the SSL connection has been terminated abruptly. Generally, you shouldn't try to reuse the underlying transport when this error is encountered.

Adicionado na versão 3.3.

exception ssl.**SSLCertVerificationError**

A subclass of *SSLError* raised when certificate validation has failed.

Adicionado na versão 3.7.

verify_code

A numeric error number that denotes the verification error.

verify_message

A human readable string of the verification error.

exception ssl.**CertificateError**

An alias for *SSLCertVerificationError*.

Alterado na versão 3.7: The exception is now an alias for *SSLCertVerificationError*.

Random generation

ssl.**RAND_bytes**(*num*, /)

Return *num* cryptographically strong pseudo-random bytes. Raises an *SSLError* if the PRNG has not been seeded with enough data or if the operation is not supported by the current RAND method. *RAND_status()* can be used to check the status of the PRNG and *RAND_add()* can be used to seed the PRNG.

For almost all applications *os.urandom()* is preferable.

Read the Wikipedia article, Cryptographically secure pseudorandom number generator (CSPRNG), to get the requirements of a cryptographically strong generator.

Adicionado na versão 3.3.

ssl.**RAND_status**()

Return `True` if the SSL pseudo-random number generator has been seeded with 'enough' randomness, and `False` otherwise. You can use `ssl.RAND_egd()` and *ssl.RAND_add()* to increase the randomness of the pseudo-random number generator.

ssl.**RAND_add**(*bytes*, *entropy*, /)

Mix the given *bytes* into the SSL pseudo-random number generator. The parameter *entropy* (a float) is a lower bound on the entropy contained in string (so you can always use `0.0`). See **RFC 1750** for more information on sources of entropy.

Alterado na versão 3.5: Writable *bytes-like object* is now accepted.

Manipulação de certificados

ssl.**cert_time_to_seconds**(*cert_time*)

Return the time in seconds since the Epoch, given the `cert_time` string representing the "notBefore" or "notAfter" date from a certificate in `"%b %d %H:%M:%S %Y %Z"` strptime format (C locale).

Here's an example:

```
>>> import ssl
>>> timestamp = ssl.cert_time_to_seconds("Jan  5 09:34:43 2018 GMT")
>>> timestamp
1515144883
>>> from datetime import datetime
>>> print(datetime.utcfromtimestamp(timestamp))
2018-01-05 09:34:43
```

"notBefore" or "notAfter" dates must use GMT (**RFC 5280**).

Alterado na versão 3.5: Interpret the input time as a time in UTC as specified by 'GMT' timezone in the input string. Local timezone was used previously. Return an integer (no fractions of a second in the input format)

`ssl.`**`get_server_certificate`**(*addr*, *ssl_version=PROTOCOL_TLS_CLIENT*, *ca_certs=None*[, *timeout*])

Given the address `addr` of an SSL-protected server, as a (*hostname*, *port-number*) pair, fetches the server's certificate, and returns it as a PEM-encoded string. If `ssl_version` is specified, uses that version of the SSL protocol to attempt to connect to the server. If *ca_certs* is specified, it should be a file containing a list of root certificates, the same format as used for the *cafile* parameter in *`SSLContext.load_verify_locations()`*. The call will attempt to validate the server certificate against that set of root certificates, and will fail if the validation attempt fails. A timeout can be specified with the `timeout` parameter.

Alterado na versão 3.3: This function is now IPv6-compatible.

Alterado na versão 3.5: The default *ssl_version* is changed from *`PROTOCOL_SSLv3`* to *`PROTOCOL_TLS`* for maximum compatibility with modern servers.

Alterado na versão 3.10: The *timeout* parameter was added.

`ssl.`**`DER_cert_to_PEM_cert`**(*der_cert_bytes*)

Given a certificate as a DER-encoded blob of bytes, returns a PEM-encoded string version of the same certificate.

`ssl.`**`PEM_cert_to_DER_cert`**(*pem_cert_string*)

Given a certificate as an ASCII PEM string, returns a DER-encoded sequence of bytes for that same certificate.

`ssl.`**`get_default_verify_paths`**()

Returns a named tuple with paths to OpenSSL's default cafile and capath. The paths are the same as used by *`SSLContext.set_default_verify_paths()`*. The return value is a *named tuple* `DefaultVerifyPaths`:

- `cafile` - resolved path to cafile or `None` if the file doesn't exist,
- `capath` - resolved path to capath or `None` if the directory doesn't exist,
- `openssl_cafile_env` - OpenSSL's environment key that points to a cafile,
- `openssl_cafile` - hard coded path to a cafile,
- `openssl_capath_env` - OpenSSL's environment key that points to a capath,
- `openssl_capath` - hard coded path to a capath directory

Adicionado na versão 3.4.

`ssl.`**`enum_certificates`**(*store_name*)

Retrieve certificates from Windows' system cert store. *store_name* may be one of `CA`, `ROOT` or `MY`. Windows may provide additional cert stores, too.

The function returns a list of (cert_bytes, encoding_type, trust) tuples. The encoding_type specifies the encoding of cert_bytes. It is either `x509_asn` for X.509 ASN.1 data or `pkcs_7_asn` for PKCS#7 ASN.1 data. Trust specifies the purpose of the certificate as a set of OIDS or exactly `True` if the certificate is trustworthy for all purposes.

Exemplo:

```
>>> ssl.enum_certificates("CA")
[(b'data...', 'x509_asn', {'1.3.6.1.5.5.7.3.1', '1.3.6.1.5.5.7.3.2'}),
 (b'data...', 'x509_asn', True)]
```

Disponibilidade: Windows.

Adicionado na versão 3.4.

ssl.**enum_crls**(*store_name*)

Retrieve CRLs from Windows' system cert store. *store_name* may be one of `CA`, `ROOT` or `MY`. Windows may provide additional cert stores, too.

The function returns a list of (cert_bytes, encoding_type, trust) tuples. The encoding_type specifies the encoding of cert_bytes. It is either `x509_asn` for X.509 ASN.1 data or `pkcs_7_asn` for PKCS#7 ASN.1 data.

Disponibilidade: Windows.

Adicionado na versão 3.4.

Constantes

All constants are now *enum.IntEnum* or *enum.IntFlag* collections.

Adicionado na versão 3.6.

ssl.**CERT_NONE**

Possible value for *SSLContext.verify_mode*. Except for *PROTOCOL_TLS_CLIENT*, it is the default mode. With client-side sockets, just about any cert is accepted. Validation errors, such as untrusted or expired cert, are ignored and do not abort the TLS/SSL handshake.

In server mode, no certificate is requested from the client, so the client does not send any for client cert authentication.

See the discussion of *Considerações de segurança* below.

ssl.**CERT_OPTIONAL**

Possible value for *SSLContext.verify_mode*. In client mode, *CERT_OPTIONAL* has the same meaning as *CERT_REQUIRED*. It is recommended to use *CERT_REQUIRED* for client-side sockets instead.

In server mode, a client certificate request is sent to the client. The client may either ignore the request or send a certificate in order perform TLS client cert authentication. If the client chooses to send a certificate, it is verified. Any verification error immediately aborts the TLS handshake.

Use of this setting requires a valid set of CA certificates to be passed to *SSLContext.load_verify_locations()*.

ssl.**CERT_REQUIRED**

Possible value for *SSLContext.verify_mode*. In this mode, certificates are required from the other side of the socket connection; an *SSLError* will be raised if no certificate is provided, or if its validation fails. This mode is **not** sufficient to verify a certificate in client mode as it does not match hostnames. *check_hostname* must be enabled as well to verify the authenticity of a cert. *PROTOCOL_TLS_CLIENT* uses *CERT_REQUIRED* and enables *check_hostname* by default.

With server socket, this mode provides mandatory TLS client cert authentication. A client certificate request is sent to the client and the client must provide a valid and trusted certificate.

Use of this setting requires a valid set of CA certificates to be passed to *SSLContext.load_verify_locations()*.

class ssl.**VerifyMode**

enum.IntEnum collection of CERT_* constants.

Adicionado na versão 3.6.

ssl.**VERIFY_DEFAULT**

Possible value for *SSLContext.verify_flags*. In this mode, certificate revocation lists (CRLs) are not checked. By default OpenSSL does neither require nor verify CRLs.

Adicionado na versão 3.4.

ssl.**VERIFY_CRL_CHECK_LEAF**

Possible value for *SSLContext.verify_flags*. In this mode, only the peer cert is checked but none of the intermediate CA certificates. The mode requires a valid CRL that is signed by the peer cert's issuer (its direct ancestor CA). If no proper CRL has been loaded with *SSLContext.load_verify_locations*, validation will fail.

Adicionado na versão 3.4.

ssl.**VERIFY_CRL_CHECK_CHAIN**

Possible value for *SSLContext.verify_flags*. In this mode, CRLs of all certificates in the peer cert chain are checked.

Adicionado na versão 3.4.

ssl.**VERIFY_X509_STRICT**

Possible value for *SSLContext.verify_flags* to disable workarounds for broken X.509 certificates.

Adicionado na versão 3.4.

ssl.**VERIFY_ALLOW_PROXY_CERTS**

Possible value for *SSLContext.verify_flags* to enables proxy certificate verification.

Adicionado na versão 3.10.

ssl.**VERIFY_X509_TRUSTED_FIRST**

Possible value for *SSLContext.verify_flags*. It instructs OpenSSL to prefer trusted certificates when building the trust chain to validate a certificate. This flag is enabled by default.

Adicionado na versão 3.4.4.

ssl.**VERIFY_X509_PARTIAL_CHAIN**

Possible value for *SSLContext.verify_flags*. It instructs OpenSSL to accept intermediate CAs in the trust store to be treated as trust-anchors, in the same way as the self-signed root CA certificates. This makes it possible to trust certificates issued by an intermediate CA without having to trust its ancestor root CA.

Adicionado na versão 3.10.

class ssl.**VerifyFlags**

enum.IntFlag collection of VERIFY_* constants.

Adicionado na versão 3.6.

ssl.**PROTOCOL_TLS**

Selects the highest protocol version that both the client and server support. Despite the name, this option can select both "SSL" and "TLS" protocols.

Adicionado na versão 3.6.

Descontinuado desde a versão 3.10: TLS clients and servers require different default settings for secure communication. The generic TLS protocol constant is deprecated in favor of *PROTOCOL_TLS_CLIENT* and *PROTOCOL_TLS_SERVER*.

ssl.**PROTOCOL_TLS_CLIENT**

Auto-negotiate the highest protocol version that both the client and server support, and configure the context client-side connections. The protocol enables *CERT_REQUIRED* and *check_hostname* by default.

Adicionado na versão 3.6.

ssl.**PROTOCOL_TLS_SERVER**

Auto-negotiate the highest protocol version that both the client and server support, and configure the context server-side connections.

Adicionado na versão 3.6.

ssl.**PROTOCOL_SSLv23**

Alias for *PROTOCOL_TLS*.

Descontinuado desde a versão 3.6: Use *PROTOCOL_TLS* instead.

ssl.**PROTOCOL_SSLv3**

Selects SSL version 3 as the channel encryption protocol.

This protocol is not available if OpenSSL is compiled with the `no-ssl3` option.

> **Aviso**
>
> SSL version 3 is insecure. Its use is highly discouraged.

Descontinuado desde a versão 3.6: OpenSSL has deprecated all version specific protocols. Use the default protocol *PROTOCOL_TLS_SERVER* or *PROTOCOL_TLS_CLIENT* with *SSLContext.minimum_version* and *SSLContext.maximum_version* instead.

ssl.**PROTOCOL_TLSv1**

Selects TLS version 1.0 as the channel encryption protocol.

Descontinuado desde a versão 3.6: OpenSSL has deprecated all version specific protocols.

ssl.**PROTOCOL_TLSv1_1**

Selects TLS version 1.1 as the channel encryption protocol. Available only with openssl version 1.0.1+.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.6: OpenSSL has deprecated all version specific protocols.

ssl.**PROTOCOL_TLSv1_2**

Selects TLS version 1.2 as the channel encryption protocol. Available only with openssl version 1.0.1+.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.6: OpenSSL has deprecated all version specific protocols.

ssl.**OP_ALL**

Enables workarounds for various bugs present in other SSL implementations. This option is set by default. It does not necessarily set the same flags as OpenSSL's `SSL_OP_ALL` constant.

Adicionado na versão 3.2.

ssl.**OP_NO_SSLv2**

Prevents an SSLv2 connection. This option is only applicable in conjunction with *PROTOCOL_TLS*. It prevents the peers from choosing SSLv2 as the protocol version.

Adicionado na versão 3.2.

Descontinuado desde a versão 3.6: SSLv2 is deprecated

ssl.**OP_NO_SSLv3**

Prevents an SSLv3 connection. This option is only applicable in conjunction with *PROTOCOL_TLS*. It prevents the peers from choosing SSLv3 as the protocol version.

Adicionado na versão 3.2.

Descontinuado desde a versão 3.6: SSLv3 is deprecated

ssl.**OP_NO_TLSv1**

Prevents a TLSv1 connection. This option is only applicable in conjunction with *PROTOCOL_TLS*. It prevents the peers from choosing TLSv1 as the protocol version.

Adicionado na versão 3.2.

Descontinuado desde a versão 3.7: The option is deprecated since OpenSSL 1.1.0, use the new *SSLContext.minimum_version* and *SSLContext.maximum_version* instead.

ssl.**OP_NO_TLSv1_1**

Prevents a TLSv1.1 connection. This option is only applicable in conjunction with *PROTOCOL_TLS*. It prevents the peers from choosing TLSv1.1 as the protocol version. Available only with openssl version 1.0.1+.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.7: The option is deprecated since OpenSSL 1.1.0.

ssl.**OP_NO_TLSv1_2**

Prevents a TLSv1.2 connection. This option is only applicable in conjunction with *PROTOCOL_TLS*. It prevents the peers from choosing TLSv1.2 as the protocol version. Available only with openssl version 1.0.1+.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.7: The option is deprecated since OpenSSL 1.1.0.

ssl.**OP_NO_TLSv1_3**

Prevents a TLSv1.3 connection. This option is only applicable in conjunction with *PROTOCOL_TLS*. It prevents the peers from choosing TLSv1.3 as the protocol version. TLS 1.3 is available with OpenSSL 1.1.1 or later. When Python has been compiled against an older version of OpenSSL, the flag defaults to *0*.

Adicionado na versão 3.6.3.

Descontinuado desde a versão 3.7: The option is deprecated since OpenSSL 1.1.0. It was added to 2.7.15 and 3.6.3 for backwards compatibility with OpenSSL 1.0.2.

ssl.**OP_NO_RENEGOTIATION**

Disable all renegotiation in TLSv1.2 and earlier. Do not send HelloRequest messages, and ignore renegotiation requests via ClientHello.

This option is only available with OpenSSL 1.1.0h and later.

Adicionado na versão 3.7.

ssl.**OP_CIPHER_SERVER_PREFERENCE**

Use the server's cipher ordering preference, rather than the client's. This option has no effect on client sockets and SSLv2 server sockets.

Adicionado na versão 3.3.

ssl.**OP_SINGLE_DH_USE**

Prevents reuse of the same DH key for distinct SSL sessions. This improves forward secrecy but requires more computational resources. This option only applies to server sockets.

Adicionado na versão 3.3.

ssl.**OP_SINGLE_ECDH_USE**

Prevents reuse of the same ECDH key for distinct SSL sessions. This improves forward secrecy but requires more computational resources. This option only applies to server sockets.

Adicionado na versão 3.3.

ssl.**OP_ENABLE_MIDDLEBOX_COMPAT**

Send dummy Change Cipher Spec (CCS) messages in TLS 1.3 handshake to make a TLS 1.3 connection look more like a TLS 1.2 connection.

This option is only available with OpenSSL 1.1.1 and later.

Adicionado na versão 3.8.

ssl.**OP_NO_COMPRESSION**

Disable compression on the SSL channel. This is useful if the application protocol supports its own compression scheme.

Adicionado na versão 3.3.

class ssl.**Options**

enum.IntFlag collection of OP_* constants.

ssl.**OP_NO_TICKET**

Prevent client side from requesting a session ticket.

Adicionado na versão 3.6.

ssl.**OP_IGNORE_UNEXPECTED_EOF**

Ignore unexpected shutdown of TLS connections.

This option is only available with OpenSSL 3.0.0 and later.

Adicionado na versão 3.10.

ssl.**OP_ENABLE_KTLS**

Enable the use of the kernel TLS. To benefit from the feature, OpenSSL must have been compiled with support for it, and the negotiated cipher suites and extensions must be supported by it (a list of supported ones may vary by platform and kernel version).

Note that with enabled kernel TLS some cryptographic operations are performed by the kernel directly and not via any available OpenSSL Providers. This might be undesirable if, for example, the application requires all cryptographic operations to be performed by the FIPS provider.

This option is only available with OpenSSL 3.0.0 and later.

Adicionado na versão 3.12.

ssl.**OP_LEGACY_SERVER_CONNECT**

Allow legacy insecure renegotiation between OpenSSL and unpatched servers only.

Adicionado na versão 3.12.

ssl.**HAS_ALPN**

Whether the OpenSSL library has built-in support for the *Application-Layer Protocol Negotiation* TLS extension as described in **RFC 7301**.

Adicionado na versão 3.5.

ssl.**HAS_NEVER_CHECK_COMMON_NAME**

Whether the OpenSSL library has built-in support not checking subject common name and *SSLContext.hostname_checks_common_name* is writeable.

Adicionado na versão 3.7.

ssl.**HAS_ECDH**

Whether the OpenSSL library has built-in support for the Elliptic Curve-based Diffie-Hellman key exchange. This should be true unless the feature was explicitly disabled by the distributor.

Adicionado na versão 3.3.

ssl.**HAS_SNI**

Whether the OpenSSL library has built-in support for the *Server Name Indication* extension (as defined in **RFC 6066**).

Adicionado na versão 3.2.

ssl.**HAS_NPN**

Whether the OpenSSL library has built-in support for the *Next Protocol Negotiation* as described in the Application Layer Protocol Negotiation. When true, you can use the *SSLContext.set_npn_protocols()* method to advertise which protocols you want to support.

Adicionado na versão 3.3.

ssl.**HAS_SSLv2**

Whether the OpenSSL library has built-in support for the SSL 2.0 protocol.

Adicionado na versão 3.7.

ssl.**HAS_SSLv3**

Whether the OpenSSL library has built-in support for the SSL 3.0 protocol.

Adicionado na versão 3.7.

ssl.**HAS_TLSv1**

Whether the OpenSSL library has built-in support for the TLS 1.0 protocol.

Adicionado na versão 3.7.

ssl.**HAS_TLSv1_1**

Whether the OpenSSL library has built-in support for the TLS 1.1 protocol.

Adicionado na versão 3.7.

ssl.**HAS_TLSv1_2**

Whether the OpenSSL library has built-in support for the TLS 1.2 protocol.

Adicionado na versão 3.7.

ssl.**HAS_TLSv1_3**

Whether the OpenSSL library has built-in support for the TLS 1.3 protocol.

Adicionado na versão 3.7.

ssl.**HAS_PSK**

Whether the OpenSSL library has built-in support for TLS-PSK.

Adicionado na versão 3.13.

ssl.**HAS_PHA**

Whether the OpenSSL library has built-in support for TLS-PHA.

Adicionado na versão 3.14.

ssl.**CHANNEL_BINDING_TYPES**

List of supported TLS channel binding types. Strings in this list can be used as arguments to *SSLSocket.get_channel_binding()*.

Adicionado na versão 3.3.

ssl.**OPENSSL_VERSION**

The version string of the OpenSSL library loaded by the interpreter:

```
>>> ssl.OPENSSL_VERSION
'OpenSSL 1.0.2k  26 Jan 2017'
```

Adicionado na versão 3.2.

ssl.**OPENSSL_VERSION_INFO**

A tuple of five integers representing version information about the OpenSSL library:

```
>>> ssl.OPENSSL_VERSION_INFO
(1, 0, 2, 11, 15)
```

Adicionado na versão 3.2.

ssl.**OPENSSL_VERSION_NUMBER**

The raw version number of the OpenSSL library, as a single integer:

```
>>> ssl.OPENSSL_VERSION_NUMBER
268443839
>>> hex(ssl.OPENSSL_VERSION_NUMBER)
'0x100020bf'
```

Adicionado na versão 3.2.

ssl.**ALERT_DESCRIPTION_HANDSHAKE_FAILURE**

ssl.**ALERT_DESCRIPTION_INTERNAL_ERROR**

ALERT_DESCRIPTION_*

Alert Descriptions from **RFC 5246** and others. The IANA TLS Alert Registry contains this list and references to the RFCs where their meaning is defined.

Used as the return value of the callback function in *SSLContext.set_servername_callback()*.

Adicionado na versão 3.4.

class ssl.**AlertDescription**

enum.IntEnum collection of ALERT_DESCRIPTION_* constants.

Adicionado na versão 3.6.

Purpose.**SERVER_AUTH**

Option for *create_default_context()* and *SSLContext.load_default_certs()*. This value indicates that the context may be used to authenticate web servers (therefore, it will be used to create client-side sockets).

Adicionado na versão 3.4.

Purpose.**CLIENT_AUTH**

Option for *create_default_context()* and *SSLContext.load_default_certs()*. This value indicates that the context may be used to authenticate web clients (therefore, it will be used to create server-side sockets).

Adicionado na versão 3.4.

class ssl.**SSLErrorNumber**

enum.IntEnum collection of SSL_ERROR_* constants.

Adicionado na versão 3.6.

class ssl.**TLSVersion**

enum.IntEnum collection of SSL and TLS versions for *SSLContext.maximum_version* and *SSLContext.minimum_version*.

Adicionado na versão 3.7.

TLSVersion.**MINIMUM_SUPPORTED**

TLSVersion.**MAXIMUM_SUPPORTED**

The minimum or maximum supported SSL or TLS version. These are magic constants. Their values don't reflect the lowest and highest available TLS/SSL versions.

TLSVersion.**SSLv3**

TLSVersion.**TLSv1**

TLSVersion.**TLSv1_1**

TLSVersion.**TLSv1_2**

TLSVersion.**TLSv1_3**

SSL 3.0 to TLS 1.3.

Descontinuado desde a versão 3.10: All *TLSVersion* members except *TLSVersion.TLSv1_2* and *TLSVersion.TLSv1_3* are deprecated.

19.3.2 SSL Sockets

class ssl.**SSLSocket**(*socket.socket*)

SSL sockets provide the following methods of *Socket Objects*:

- *accept()*
- *bind()*
- *close()*
- *connect()*
- *detach()*
- *fileno()*
- *getpeername()*, *getsockname()*
- *getsockopt()*, *setsockopt()*
- *gettimeout()*, *settimeout()*, *setblocking()*
- *listen()*
- *makefile()*
- *recv()*, *recv_into()* (but passing a non-zero `flags` argument is not allowed)
- *send()*, *sendall()* (with the same limitation)
- *sendfile()* (but *os.sendfile* will be used for plain-text sockets only, else *send()* will be used)
- *shutdown()*

However, since the SSL (and TLS) protocol has its own framing atop of TCP, the SSL sockets abstraction can, in certain respects, diverge from the specification of normal, OS-level sockets. See especially the *notes on non-blocking sockets*.

Instances of *SSLSocket* must be created using the *SSLContext.wrap_socket()* method.

Alterado na versão 3.5: The `sendfile()` method was added.

Alterado na versão 3.5: The `shutdown()` does not reset the socket timeout each time bytes are received or sent. The socket timeout is now the maximum total duration of the shutdown.

Descontinuado desde a versão 3.6: It is deprecated to create a *SSLSocket* instance directly, use *SSLContext.wrap_socket()* to wrap a socket.

Alterado na versão 3.7: *SSLSocket* instances must to created with *wrap_socket()*. In earlier versions, it was possible to create instances directly. This was never documented or officially supported.

Alterado na versão 3.10: Python now uses `SSL_read_ex` and `SSL_write_ex` internally. The functions support reading and writing of data larger than 2 GB. Writing zero-length data no longer fails with a protocol violation error.

SSL sockets also have the following additional methods and attributes:

SSLSocket.**read**(*len=1024*, *buffer=None*)

Read up to *len* bytes of data from the SSL socket and return the result as a `bytes` instance. If *buffer* is specified, then read into the buffer instead, and return the number of bytes read.

Raise *SSLWantReadError* or *SSLWantWriteError* if the socket is *non-blocking* and the read would block.

As at any time a re-negotiation is possible, a call to *read()* can also cause write operations.

Alterado na versão 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum total duration to read up to *len* bytes.

Descontinuado desde a versão 3.6: Use `recv()` instead of *read()*.

SSLSocket.**write**(*data*)

Write *data* to the SSL socket and return the number of bytes written. The *data* argument must be an object supporting the buffer interface.

Raise *SSLWantReadError* or *SSLWantWriteError* if the socket is *non-blocking* and the write would block.

As at any time a re-negotiation is possible, a call to *write()* can also cause read operations.

Alterado na versão 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum total duration to write *data*.

Descontinuado desde a versão 3.6: Use `send()` instead of *write()*.

Nota

The *read()* and *write()* methods are the low-level methods that read and write unencrypted, application-level data and decrypt/encrypt it to encrypted, wire-level data. These methods require an active SSL connection, i.e. the handshake was completed and *SSLSocket.unwrap()* was not called.

Normally you should use the socket API methods like *recv()* and *send()* instead of these methods.

SSLSocket.**do_handshake**(*block=False*)

Perform the SSL setup handshake.

If *block* is true and the timeout obtained by *gettimeout()* is zero, the socket is set in blocking mode until the handshake is performed.

Alterado na versão 3.4: The handshake method also performs `match_hostname()` when the *check_hostname* attribute of the socket's *context* is true.

Alterado na versão 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum total duration of the handshake.

Alterado na versão 3.7: Hostname or IP address is matched by OpenSSL during handshake. The function `match_hostname()` is no longer used. In case OpenSSL refuses a hostname or IP address, the handshake is aborted early and a TLS alert message is sent to the peer.

SSLSocket.**getpeercert**(*binary_form=False*)

If there is no certificate for the peer on the other end of the connection, return `None`. If the SSL handshake hasn't been done yet, raise *ValueError*.

If the `binary_form` parameter is *False*, and a certificate was received from the peer, this method returns a *dict* instance. If the certificate was not validated, the dict is empty. If the certificate was validated, it returns a dict with several keys, amongst them `subject` (the principal for which the certificate was issued) and `issuer` (the principal issuing the certificate). If a certificate contains an instance of the *Subject Alternative Name* extension (see **RFC 3280**), there will also be a `subjectAltName` key in the dictionary.

The `subject` and `issuer` fields are tuples containing the sequence of relative distinguished names (RDNs) given in the certificate's data structure for the respective fields, and each RDN is a sequence of name-value pairs. Here is a real-world example:

```
{'issuer': ((('countryName', 'IL'),),
            (('organizationName', 'StartCom Ltd.'),),
            (('organizationalUnitName',
              'Secure Digital Certificate Signing'),),
            (('commonName',
              'StartCom Class 2 Primary Intermediate Server CA'),)),
 'notAfter': 'Nov 22 08:15:19 2013 GMT',
 'notBefore': 'Nov 21 03:09:52 2011 GMT',
 'serialNumber': '95F0',
 'subject': ((('description', '571208-SLe257oHY9fVQ07Z'),),
             (('countryName', 'US'),),
             (('stateOrProvinceName', 'California'),),
             (('localityName', 'San Francisco'),),
             (('organizationName', 'Electronic Frontier Foundation, Inc.'),),
             (('commonName', '*.eff.org'),),
             (('emailAddress', 'hostmaster@eff.org'),)),
 'subjectAltName': (('DNS', '*.eff.org'), ('DNS', 'eff.org')),
 'version': 3}
```

If the `binary_form` parameter is *True*, and a certificate was provided, this method returns the DER-encoded form of the entire certificate as a sequence of bytes, or *None* if the peer did not provide a certificate. Whether the peer provides a certificate depends on the SSL socket's role:

- for a client SSL socket, the server will always provide a certificate, regardless of whether validation was required;
- for a server SSL socket, the client will only provide a certificate when requested by the server; therefore *getpeercert()* will return *None* if you used *CERT_NONE* (rather than *CERT_OPTIONAL* or *CERT_REQUIRED*).

See also *SSLContext.check_hostname*.

Alterado na versão 3.2: The returned dictionary includes additional items such as `issuer` and `notBefore`.

Alterado na versão 3.4: *ValueError* is raised when the handshake isn't done. The returned dictionary includes additional X509v3 extension items such as `crlDistributionPoints`, `caIssuers` and `OCSP` URIs.

Alterado na versão 3.9: IPv6 address strings no longer have a trailing new line.

`SSLSocket.`**`get_verified_chain`**`()`

Returns verified certificate chain provided by the other end of the SSL channel as a list of DER-encoded bytes. If certificate verification was disabled method acts the same as *get_unverified_chain()*.

Adicionado na versão 3.13.

`SSLSocket.`**`get_unverified_chain`**`()`

Returns raw certificate chain provided by the other end of the SSL channel as a list of DER-encoded bytes.

Adicionado na versão 3.13.

`SSLSocket.`**`cipher`**`()`

Returns a three-value tuple containing the name of the cipher being used, the version of the SSL protocol that defines its use, and the number of secret bits being used. If no connection has been established, returns `None`.

`SSLSocket.`**`shared_ciphers`**`()`

Return the list of ciphers available in both the client and server. Each entry of the returned list is a three-value tuple containing the name of the cipher, the version of the SSL protocol that defines its use, and the number of secret bits the cipher uses. *shared_ciphers()* returns `None` if no connection has been established or the socket is a client socket.

Adicionado na versão 3.5.

SSLSocket.**compression**()

Return the compression algorithm being used as a string, or `None` if the connection isn't compressed.

If the higher-level protocol supports its own compression mechanism, you can use *OP_NO_COMPRESSION* to disable SSL-level compression.

Adicionado na versão 3.3.

SSLSocket.**get_channel_binding**(*cb_type='tls-unique'*)

Get channel binding data for current connection, as a bytes object. Returns `None` if not connected or the handshake has not been completed.

The *cb_type* parameter allow selection of the desired channel binding type. Valid channel binding types are listed in the *CHANNEL_BINDING_TYPES* list. Currently only the 'tls-unique' channel binding, defined by **RFC 5929**, is supported. *ValueError* will be raised if an unsupported channel binding type is requested.

Adicionado na versão 3.3.

SSLSocket.**selected_alpn_protocol**()

Return the protocol that was selected during the TLS handshake. If *SSLContext.set_alpn_protocols()* was not called, if the other party does not support ALPN, if this socket does not support any of the client's proposed protocols, or if the handshake has not happened yet, `None` is returned.

Adicionado na versão 3.5.

SSLSocket.**selected_npn_protocol**()

Return the higher-level protocol that was selected during the TLS/SSL handshake. If *SSLContext.set_npn_protocols()* was not called, or if the other party does not support NPN, or if the handshake has not yet happened, this will return `None`.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.10: NPN has been superseded by ALPN

SSLSocket.**unwrap**()

Performs the SSL shutdown handshake, which removes the TLS layer from the underlying socket, and returns the underlying socket object. This can be used to go from encrypted operation over a connection to unencrypted. The returned socket should always be used for further communication with the other side of the connection, rather than the original socket.

SSLSocket.**verify_client_post_handshake**()

Requests post-handshake authentication (PHA) from a TLS 1.3 client. PHA can only be initiated for a TLS 1.3 connection from a server-side socket, after the initial TLS handshake and with PHA enabled on both sides, see *SSLContext.post_handshake_auth*.

The method does not perform a cert exchange immediately. The server-side sends a CertificateRequest during the next write event and expects the client to respond with a certificate on the next read event.

If any precondition isn't met (e.g. not TLS 1.3, PHA not enabled), an *SSLError* is raised.

> **Nota**
>
> Only available with OpenSSL 1.1.1 and TLS 1.3 enabled. Without TLS 1.3 support, the method raises *NotImplementedError*.

Adicionado na versão 3.8.

SSLSocket.**version**()

Return the actual SSL protocol version negotiated by the connection as a string, or `None` if no secure connection is established. As of this writing, possible return values include `"SSLv2"`, `"SSLv3"`, `"TLSv1"`, `"TLSv1.1"` and `"TLSv1.2"`. Recent OpenSSL versions may define more return values.

Adicionado na versão 3.5.

SSLSocket.**pending**()

Returns the number of already decrypted bytes available for read, pending on the connection.

SSLSocket.**context**

The *SSLContext* object this SSL socket is tied to.

Adicionado na versão 3.2.

SSLSocket.**server_side**

A boolean which is `True` for server-side sockets and `False` for client-side sockets.

Adicionado na versão 3.2.

SSLSocket.**server_hostname**

Hostname of the server: *str* type, or `None` for server-side socket or if the hostname was not specified in the constructor.

Adicionado na versão 3.2.

Alterado na versão 3.7: The attribute is now always ASCII text. When `server_hostname` is an internationalized domain name (IDN), this attribute now stores the A-label form (`"xn--pythn-mua.org"`), rather than the U-label form (`"pythön.org"`).

SSLSocket.**session**

The *SSLSession* for this SSL connection. The session is available for client and server side sockets after the TLS handshake has been performed. For client sockets the session can be set before *do_handshake()* has been called to reuse a session.

Adicionado na versão 3.6.

SSLSocket.**session_reused**

Adicionado na versão 3.6.

19.3.3 SSL Contexts

Adicionado na versão 3.2.

An SSL context holds various data longer-lived than single SSL connections, such as SSL configuration options, certificate(s) and private key(s). It also manages a cache of SSL sessions for server-side sockets, in order to speed up repeated connections from the same clients.

class ssl.**SSLContext**(*protocol=None*)

Create a new SSL context. You may pass *protocol* which must be one of the `PROTOCOL_*` constants defined in this module. The parameter specifies which version of the SSL protocol to use. Typically, the server chooses a particular protocol version, and the client must adapt to the server's choice. Most of the versions are not interoperable with the other versions. If not specified, the default is *PROTOCOL_TLS*; it provides the most compatibility with other versions.

Here's a table showing which versions in a client (down the side) can connect to which versions in a server (along the top):

client / **server**	**SSLv2**	**SSLv3**	**TLS**[3]	**TLSv1**	**TLSv1.1**	**TLSv1.2**
SSLv2	sim	não	no[1]	não	não	não
SSLv3	não	sim	no[2]	não	não	não
TLS (SSLv23)[3]	no[1]	no[2]	sim	sim	sim	sim
TLSv1	não	não	sim	sim	não	não
TLSv1.1	não	não	sim	não	sim	não
TLSv1.2	não	não	sim	não	não	sim

[3] TLS 1.3 protocol will be available with *PROTOCOL_TLS* in OpenSSL >= 1.1.1. There is no dedicated PROTOCOL constant for just TLS 1.3.

[1] *SSLContext* disables SSLv2 with *OP_NO_SSLv2* by default.

[2] *SSLContext* disables SSLv3 with *OP_NO_SSLv3* by default.

Ver também

create_default_context() lets the *ssl* module choose security settings for a given purpose.

Alterado na versão 3.6: The context is created with secure default values. The options *OP_NO_COMPRESSION*, *OP_CIPHER_SERVER_PREFERENCE*, *OP_SINGLE_DH_USE*, *OP_SINGLE_ECDH_USE*, *OP_NO_SSLv2*, and *OP_NO_SSLv3* (except for *PROTOCOL_SSLv3*) are set by default. The initial cipher suite list contains only `HIGH` ciphers, no `NULL` ciphers and no `MD5` ciphers.

Descontinuado desde a versão 3.10: *SSLContext* without protocol argument is deprecated. The context class will either require *PROTOCOL_TLS_CLIENT* or *PROTOCOL_TLS_SERVER* protocol in the future.

Alterado na versão 3.10: The default cipher suites now include only secure AES and ChaCha20 ciphers with forward secrecy and security level 2. RSA and DH keys with less than 2048 bits and ECC keys with less than 224 bits are prohibited. *PROTOCOL_TLS*, *PROTOCOL_TLS_CLIENT*, and *PROTOCOL_TLS_SERVER* use TLS 1.2 as minimum TLS version.

Nota

SSLContext only supports limited mutation once it has been used by a connection. Adding new certificates to the internal trust store is allowed, but changing ciphers, verification settings, or mTLS certificates may result in surprising behavior.

Nota

SSLContext is designed to be shared and used by multiple connections. Thus, it is thread-safe as long as it is not reconfigured after being used by a connection.

SSLContext objects have the following methods and attributes:

SSLContext.**cert_store_stats**()

Get statistics about quantities of loaded X.509 certificates, count of X.509 certificates flagged as CA certificates and certificate revocation lists as dictionary.

Example for a context with one CA cert and one other cert:

```
>>> context.cert_store_stats()
{'crl': 0, 'x509_ca': 1, 'x509': 2}
```

Adicionado na versão 3.4.

SSLContext.**load_cert_chain**(*certfile*, *keyfile=None*, *password=None*)

Load a private key and the corresponding certificate. The *certfile* string must be the path to a single file in PEM format containing the certificate as well as any number of CA certificates needed to establish the certificate's authenticity. The *keyfile* string, if present, must point to a file containing the private key. Otherwise the private key will be taken from *certfile* as well. See the discussion of *Certificados* for more information on how the certificate is stored in the *certfile*.

The *password* argument may be a function to call to get the password for decrypting the private key. It will only be called if the private key is encrypted and a password is necessary. It will be called with no arguments, and it should return a string, bytes, or bytearray. If the return value is a string it will be encoded as UTF-8 before using it to decrypt the key. Alternatively a string, bytes, or bytearray value may be supplied directly as the *password* argument. It will be ignored if the private key is not encrypted and no password is needed.

If the *password* argument is not specified and a password is required, OpenSSL's built-in password prompting mechanism will be used to interactively prompt the user for a password.

An *SSLError* is raised if the private key doesn't match with the certificate.

Alterado na versão 3.3: New optional argument *password*.

`SSLContext.`**`load_default_certs`**(*purpose=Purpose.SERVER_AUTH*)

Load a set of default "certification authority" (CA) certificates from default locations. On Windows it loads CA certs from the `CA` and `ROOT` system stores. On all systems it calls *SSLContext.set_default_verify_paths()*. In the future the method may load CA certificates from other locations, too.

The *purpose* flag specifies what kind of CA certificates are loaded. The default settings *Purpose.SERVER_AUTH* loads certificates, that are flagged and trusted for TLS web server authentication (client side sockets). *Purpose.CLIENT_AUTH* loads CA certificates for client certificate verification on the server side.

Adicionado na versão 3.4.

`SSLContext.`**`load_verify_locations`**(*cafile=None*, *capath=None*, *cadata=None*)

Load a set of "certification authority" (CA) certificates used to validate other peers' certificates when *verify_mode* is other than *CERT_NONE*. At least one of *cafile* or *capath* must be specified.

This method can also load certification revocation lists (CRLs) in PEM or DER format. In order to make use of CRLs, *SSLContext.verify_flags* must be configured properly.

The *cafile* string, if present, is the path to a file of concatenated CA certificates in PEM format. See the discussion of *Certificados* for more information about how to arrange the certificates in this file.

The *capath* string, if present, is the path to a directory containing several CA certificates in PEM format, following an OpenSSL specific layout.

The *cadata* object, if present, is either an ASCII string of one or more PEM-encoded certificates or a *bytes-like object* of DER-encoded certificates. Like with *capath* extra lines around PEM-encoded certificates are ignored but at least one certificate must be present.

Alterado na versão 3.4: New optional argument *cadata*

`SSLContext.`**`get_ca_certs`**(*binary_form=False*)

Get a list of loaded "certification authority" (CA) certificates. If the `binary_form` parameter is *False* each list entry is a dict like the output of *SSLSocket.getpeercert()*. Otherwise the method returns a list of DER-encoded certificates. The returned list does not contain certificates from *capath* unless a certificate was requested and loaded by a SSL connection.

> **Nota**
>
> Certificates in a capath directory aren't loaded unless they have been used at least once.

Adicionado na versão 3.4.

`SSLContext.`**`get_ciphers`**()

Get a list of enabled ciphers. The list is in order of cipher priority. See *SSLContext.set_ciphers()*.

Exemplo:

```
>>> ctx = ssl.SSLContext(ssl.PROTOCOL_SSLv23)
>>> ctx.set_ciphers('ECDHE+AESGCM:!ECDSA')
>>> ctx.get_ciphers()
[{'aead': True,
  'alg_bits': 256,
  'auth': 'auth-rsa',
  'description': 'ECDHE-RSA-AES256-GCM-SHA384 TLSv1.2 Kx=ECDH     Au=RSA  '
                 'Enc=AESGCM(256) Mac=AEAD',
  'digest': None,
  'id': 50380848,
  'kea': 'kx-ecdhe',
  'name': 'ECDHE-RSA-AES256-GCM-SHA384',
```

(continua na próxima página)

(continuação da página anterior)

```
  'protocol': 'TLSv1.2',
  'strength_bits': 256,
  'symmetric': 'aes-256-gcm'},
 {'aead': True,
  'alg_bits': 128,
  'auth': 'auth-rsa',
  'description': 'ECDHE-RSA-AES128-GCM-SHA256 TLSv1.2 Kx=ECDH     Au=RSA  '
                 'Enc=AESGCM(128) Mac=AEAD',
  'digest': None,
  'id': 50380847,
  'kea': 'kx-ecdhe',
  'name': 'ECDHE-RSA-AES128-GCM-SHA256',
  'protocol': 'TLSv1.2',
  'strength_bits': 128,
  'symmetric': 'aes-128-gcm'}]
```

Adicionado na versão 3.6.

SSLContext.**set_default_verify_paths**()

Load a set of default "certification authority" (CA) certificates from a filesystem path defined when building the OpenSSL library. Unfortunately, there's no easy way to know whether this method succeeds: no error is returned if no certificates are to be found. When the OpenSSL library is provided as part of the operating system, though, it is likely to be configured properly.

SSLContext.**set_ciphers**(*ciphers*, */*)

Set the available ciphers for sockets created with this context. It should be a string in the OpenSSL cipher list format. If no cipher can be selected (because compile-time options or other configuration forbids use of all the specified ciphers), an *SSLError* will be raised.

> **Nota**
>
> when connected, the *SSLSocket.cipher()* method of SSL sockets will give the currently selected cipher.
>
> TLS 1.3 cipher suites cannot be disabled with *set_ciphers()*.

SSLContext.**set_alpn_protocols**(*alpn_protocols*)

Specify which protocols the socket should advertise during the SSL/TLS handshake. It should be a list of ASCII strings, like `['http/1.1', 'spdy/2']`, ordered by preference. The selection of a protocol will happen during the handshake, and will play out according to **RFC 7301**. After a successful handshake, the *SSLSocket.selected_alpn_protocol()* method will return the agreed-upon protocol.

This method will raise *NotImplementedError* if *HAS_ALPN* is `False`.

Adicionado na versão 3.5.

SSLContext.**set_npn_protocols**(*npn_protocols*)

Specify which protocols the socket should advertise during the SSL/TLS handshake. It should be a list of strings, like `['http/1.1', 'spdy/2']`, ordered by preference. The selection of a protocol will happen during the handshake, and will play out according to the Application Layer Protocol Negotiation. After a successful handshake, the *SSLSocket.selected_npn_protocol()* method will return the agreed-upon protocol.

This method will raise *NotImplementedError* if *HAS_NPN* is `False`.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.10: NPN has been superseded by ALPN

SSLContext.**sni_callback**

Register a callback function that will be called after the TLS Client Hello handshake message has been received by the SSL/TLS server when the TLS client specifies a server name indication. The server name indication mechanism is specified in **RFC 6066** section 3 - Server Name Indication.

Only one callback can be set per `SSLContext`. If *sni_callback* is set to `None` then the callback is disabled. Calling this function a subsequent time will disable the previously registered callback.

The callback function will be called with three arguments; the first being the *ssl.SSLSocket*, the second is a string that represents the server name that the client is intending to communicate (or *None* if the TLS Client Hello does not contain a server name) and the third argument is the original *SSLContext*. The server name argument is text. For internationalized domain name, the server name is an IDN A-label (`"xn--pythn-mua.org"`).

A typical use of this callback is to change the *ssl.SSLSocket*'s *SSLSocket.context* attribute to a new object of type *SSLContext* representing a certificate chain that matches the server name.

Due to the early negotiation phase of the TLS connection, only limited methods and attributes are usable like *SSLSocket.selected_alpn_protocol()* and *SSLSocket.context*. The *SSLSocket.getpeercert()*, *SSLSocket.get_verified_chain()*, *SSLSocket.get_unverified_chain()* *SSLSocket.cipher()* and *SSLSocket.compression()* methods require that the TLS connection has progressed beyond the TLS Client Hello and therefore will not return meaningful values nor can they be called safely.

The *sni_callback* function must return `None` to allow the TLS negotiation to continue. If a TLS failure is required, a constant *ALERT_DESCRIPTION_** can be returned. Other return values will result in a TLS fatal error with *ALERT_DESCRIPTION_INTERNAL_ERROR*.

If an exception is raised from the *sni_callback* function the TLS connection will terminate with a fatal TLS alert message *ALERT_DESCRIPTION_HANDSHAKE_FAILURE*.

This method will raise *NotImplementedError* if the OpenSSL library had OPENSSL_NO_TLSEXT defined when it was built.

Adicionado na versão 3.7.

SSLContext.**set_servername_callback**(*server_name_callback*)

This is a legacy API retained for backwards compatibility. When possible, you should use *sni_callback* instead. The given *server_name_callback* is similar to *sni_callback*, except that when the server hostname is an IDN-encoded internationalized domain name, the *server_name_callback* receives a decoded U-label (`"pythön.org"`).

If there is a decoding error on the server name, the TLS connection will terminate with an *ALERT_DESCRIPTION_INTERNAL_ERROR* fatal TLS alert message to the client.

Adicionado na versão 3.4.

SSLContext.**load_dh_params**(*dhfile*, /)

Load the key generation parameters for Diffie-Hellman (DH) key exchange. Using DH key exchange improves forward secrecy at the expense of computational resources (both on the server and on the client). The *dhfile* parameter should be the path to a file containing DH parameters in PEM format.

This setting doesn't apply to client sockets. You can also use the *OP_SINGLE_DH_USE* option to further improve security.

Adicionado na versão 3.3.

SSLContext.**set_ecdh_curve**(*curve_name*, /)

Set the curve name for Elliptic Curve-based Diffie-Hellman (ECDH) key exchange. ECDH is significantly faster than regular DH while arguably as secure. The *curve_name* parameter should be a string describing a well-known elliptic curve, for example `prime256v1` for a widely supported curve.

This setting doesn't apply to client sockets. You can also use the *OP_SINGLE_ECDH_USE* option to further improve security.

This method is not available if *HAS_ECDH* is `False`.

Adicionado na versão 3.3.

Ver também

SSL/TLS & Perfect Forward Secrecy
Vincent Bernat.

`SSLContext.`**`wrap_socket`**(*sock*, *server_side=False*, *do_handshake_on_connect=True*, *suppress_ragged_eofs=True*, *server_hostname=None*, *session=None*)

Wrap an existing Python socket *sock* and return an instance of *SSLContext.sslsocket_class* (default *SSLSocket*). The returned SSL socket is tied to the context, its settings and certificates. *sock* must be a *SOCK_STREAM* socket; other socket types are unsupported.

The parameter `server_side` is a boolean which identifies whether server-side or client-side behavior is desired from this socket.

For client-side sockets, the context construction is lazy; if the underlying socket isn't connected yet, the context construction will be performed after `connect()` is called on the socket. For server-side sockets, if the socket has no remote peer, it is assumed to be a listening socket, and the server-side SSL wrapping is automatically performed on client connections accepted via the `accept()` method. The method may raise *SSLError*.

On client connections, the optional parameter *server_hostname* specifies the hostname of the service which we are connecting to. This allows a single server to host multiple SSL-based services with distinct certificates, quite similarly to HTTP virtual hosts. Specifying *server_hostname* will raise a *ValueError* if *server_side* is true.

The parameter `do_handshake_on_connect` specifies whether to do the SSL handshake automatically after doing a `socket.connect()`, or whether the application program will call it explicitly, by invoking the *SSLSocket.do_handshake()* method. Calling *SSLSocket.do_handshake()* explicitly gives the program control over the blocking behavior of the socket I/O involved in the handshake.

The parameter `suppress_ragged_eofs` specifies how the `SSLSocket.recv()` method should signal unexpected EOF from the other end of the connection. If specified as *True* (the default), it returns a normal EOF (an empty bytes object) in response to unexpected EOF errors raised from the underlying socket; if *False*, it will raise the exceptions back to the caller.

session, see *session*.

To wrap an *SSLSocket* in another *SSLSocket*, use *SSLContext.wrap_bio()*.

Alterado na versão 3.5: Always allow a server_hostname to be passed, even if OpenSSL does not have SNI.

Alterado na versão 3.6: o argumento *session* foi adicionado.

Alterado na versão 3.7: The method returns an instance of *SSLContext.sslsocket_class* instead of hard-coded *SSLSocket*.

`SSLContext.`**`sslsocket_class`**

The return type of *SSLContext.wrap_socket()*, defaults to *SSLSocket*. The attribute can be assigned to on instances of *SSLContext* in order to return a custom subclass of *SSLSocket*.

Adicionado na versão 3.7.

`SSLContext.`**`wrap_bio`**(*incoming*, *outgoing*, *server_side=False*, *server_hostname=None*, *session=None*)

Wrap the BIO objects *incoming* and *outgoing* and return an instance of *SSLContext.sslobject_class* (default *SSLObject*). The SSL routines will read input data from the incoming BIO and write data to the outgoing BIO.

The *server_side*, *server_hostname* and *session* parameters have the same meaning as in *SSLContext.wrap_socket()*.

Alterado na versão 3.6: o argumento *session* foi adicionado.

Alterado na versão 3.7: The method returns an instance of *SSLContext.sslobject_class* instead of hard-coded *SSLObject*.

SSLContext.**sslobject_class**

The return type of *SSLContext.wrap_bio()*, defaults to *SSLObject*. The attribute can be overridden on instance of class in order to return a custom subclass of *SSLObject*.

Adicionado na versão 3.7.

SSLContext.**session_stats**()

Get statistics about the SSL sessions created or managed by this context. A dictionary is returned which maps the names of each piece of information to their numeric values. For example, here is the total number of hits and misses in the session cache since the context was created:

```
>>> stats = context.session_stats()
>>> stats['hits'], stats['misses']
(0, 0)
```

SSLContext.**check_hostname**

Whether to match the peer cert's hostname in *SSLSocket.do_handshake()*. The context's *verify_mode* must be set to *CERT_OPTIONAL* or *CERT_REQUIRED*, and you must pass *server_hostname* to *wrap_socket()* in order to match the hostname. Enabling hostname checking automatically sets *verify_mode* from *CERT_NONE* to *CERT_REQUIRED*. It cannot be set back to *CERT_NONE* as long as hostname checking is enabled. The *PROTOCOL_TLS_CLIENT* protocol enables hostname checking by default. With other protocols, hostname checking must be enabled explicitly.

Exemplo:

```
import socket, ssl

context = ssl.SSLContext(ssl.PROTOCOL_TLSv1_2)
context.verify_mode = ssl.CERT_REQUIRED
context.check_hostname = True
context.load_default_certs()

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
ssl_sock = context.wrap_socket(s, server_hostname='www.verisign.com')
ssl_sock.connect(('www.verisign.com', 443))
```

Adicionado na versão 3.4.

Alterado na versão 3.7: *verify_mode* is now automatically changed to *CERT_REQUIRED* when hostname checking is enabled and *verify_mode* is *CERT_NONE*. Previously the same operation would have failed with a *ValueError*.

SSLContext.**keylog_filename**

Write TLS keys to a keylog file, whenever key material is generated or received. The keylog file is designed for debugging purposes only. The file format is specified by NSS and used by many traffic analyzers such as Wireshark. The log file is opened in append-only mode. Writes are synchronized between threads, but not between processes.

Adicionado na versão 3.8.

SSLContext.**maximum_version**

A *TLSVersion* enum member representing the highest supported TLS version. The value defaults to *TLSVersion.MAXIMUM_SUPPORTED*. The attribute is read-only for protocols other than *PROTOCOL_TLS*, *PROTOCOL_TLS_CLIENT*, and *PROTOCOL_TLS_SERVER*.

The attributes *maximum_version*, *minimum_version* and *SSLContext.options* all affect the supported SSL and TLS versions of the context. The implementation does not prevent invalid combination. For example a context with *OP_NO_TLSv1_2* in *options* and *maximum_version* set to *TLSVersion.TLSv1_2* will not be able to establish a TLS 1.2 connection.

Adicionado na versão 3.7.

SSLContext.**minimum_version**

Like *SSLContext.maximum_version* except it is the lowest supported version or *TLSVersion.MINIMUM_SUPPORTED*.

Adicionado na versão 3.7.

SSLContext.**num_tickets**

Control the number of TLS 1.3 session tickets of a *PROTOCOL_TLS_SERVER* context. The setting has no impact on TLS 1.0 to 1.2 connections.

Adicionado na versão 3.8.

SSLContext.**options**

An integer representing the set of SSL options enabled on this context. The default value is *OP_ALL*, but you can specify other options such as *OP_NO_SSLv2* by ORing them together.

Alterado na versão 3.6: *SSLContext.options* returns *Options* flags:

```
>>> ssl.create_default_context().options
<Options.OP_ALL|OP_NO_SSLv3|OP_NO_SSLv2|OP_NO_COMPRESSION: 2197947391>
```

Descontinuado desde a versão 3.7: All `OP_NO_SSL*` and `OP_NO_TLS*` options have been deprecated since Python 3.7. Use *SSLContext.minimum_version* and *SSLContext.maximum_version* instead.

SSLContext.**post_handshake_auth**

Enable TLS 1.3 post-handshake client authentication. Post-handshake auth is disabled by default and a server can only request a TLS client certificate during the initial handshake. When enabled, a server may request a TLS client certificate at any time after the handshake.

When enabled on client-side sockets, the client signals the server that it supports post-handshake authentication.

When enabled on server-side sockets, *SSLContext.verify_mode* must be set to *CERT_OPTIONAL* or *CERT_REQUIRED*, too. The actual client cert exchange is delayed until *SSLSocket.verify_client_post_handshake()* is called and some I/O is performed.

Adicionado na versão 3.8.

SSLContext.**protocol**

The protocol version chosen when constructing the context. This attribute is read-only.

SSLContext.**hostname_checks_common_name**

Whether *check_hostname* falls back to verify the cert's subject common name in the absence of a subject alternative name extension (default: true).

Adicionado na versão 3.7.

Alterado na versão 3.10: The flag had no effect with OpenSSL before version 1.1.1l. Python 3.8.9, 3.9.3, and 3.10 include workarounds for previous versions.

SSLContext.**security_level**

An integer representing the security level for the context. This attribute is read-only.

Adicionado na versão 3.10.

SSLContext.**verify_flags**

The flags for certificate verification operations. You can set flags like *VERIFY_CRL_CHECK_LEAF* by ORing them together. By default OpenSSL does neither require nor verify certificate revocation lists (CRLs).

Adicionado na versão 3.4.

Alterado na versão 3.6: *SSLContext.verify_flags* returns *VerifyFlags* flags:

```
>>> ssl.create_default_context().verify_flags
<VerifyFlags.VERIFY_X509_TRUSTED_FIRST: 32768>
```

SSLContext.**verify_mode**

Whether to try to verify other peers' certificates and how to behave if verification fails. This attribute must be one of *CERT_NONE*, *CERT_OPTIONAL* or *CERT_REQUIRED*.

Alterado na versão 3.6: *SSLContext.verify_mode* returns *VerifyMode* enum:

```
>>> ssl.create_default_context().verify_mode
<VerifyMode.CERT_REQUIRED: 2>
```

SSLContext.**set_psk_client_callback**(*callback*)

Enables TLS-PSK (pre-shared key) authentication on a client-side connection.

In general, certificate based authentication should be preferred over this method.

The parameter `callback` is a callable object with the signature: `def callback(hint: str | None) -> tuple[str | None, bytes]`. The `hint` parameter is an optional identity hint sent by the server. The return value is a tuple in the form (client-identity, psk). Client-identity is an optional string which may be used by the server to select a corresponding PSK for the client. The string must be less than or equal to `256` octets when UTF-8 encoded. PSK is a *bytes-like object* representing the pre-shared key. Return a zero length PSK to reject the connection.

Setting `callback` to *None* removes any existing callback.

Nota

When using TLS 1.3:

- the `hint` parameter is always *None*.
- client-identity must be a non-empty string.

Exemplo de uso:

```
context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
context.check_hostname = False
context.verify_mode = ssl.CERT_NONE
context.maximum_version = ssl.TLSVersion.TLSv1_2
context.set_ciphers('PSK')

# A simple lambda:
psk = bytes.fromhex('c0ffee')
context.set_psk_client_callback(lambda hint: (None, psk))

# A table using the hint from the server:
psk_table = { 'ServerId_1': bytes.fromhex('c0ffee'),
              'ServerId_2': bytes.fromhex('facade')
}
def callback(hint):
    return 'ClientId_1', psk_table.get(hint, b'')
context.set_psk_client_callback(callback)
```

This method will raise *NotImplementedError* if *HAS_PSK* is `False`.

Adicionado na versão 3.13.

SSLContext.**set_psk_server_callback**(*callback*, *identity_hint=None*)

Enables TLS-PSK (pre-shared key) authentication on a server-side connection.

In general, certificate based authentication should be preferred over this method.

The parameter `callback` is a callable object with the signature: `def callback(identity: str | None) -> bytes`. The `identity` parameter is an optional identity sent by the client which can be used

to select a corresponding PSK. The return value is a *bytes-like object* representing the pre-shared key. Return a zero length PSK to reject the connection.

Setting `callback` to *None* removes any existing callback.

The parameter `identity_hint` is an optional identity hint string sent to the client. The string must be less than or equal to `256` octets when UTF-8 encoded.

> **Nota**
>
> When using TLS 1.3 the `identity_hint` parameter is not sent to the client.

Exemplo de uso:

```
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.maximum_version = ssl.TLSVersion.TLSv1_2
context.set_ciphers('PSK')

# A simple lambda:
psk = bytes.fromhex('c0ffee')
context.set_psk_server_callback(lambda identity: psk)

# A table using the identity of the client:
psk_table = { 'ClientId_1': bytes.fromhex('c0ffee'),
              'ClientId_2': bytes.fromhex('facade')
}
def callback(identity):
    return psk_table.get(identity, b'')
context.set_psk_server_callback(callback, 'ServerId_1')
```

This method will raise *NotImplementedError* if *HAS_PSK* is `False`.

Adicionado na versão 3.13.

19.3.4 Certificados

Certificates in general are part of a public-key / private-key system. In this system, each *principal*, (which may be a machine, or a person, or an organization) is assigned a unique two-part encryption key. One part of the key is public, and is called the *public key*; the other part is kept secret, and is called the *private key*. The two parts are related, in that if you encrypt a message with one of the parts, you can decrypt it with the other part, and **only** with the other part.

A certificate contains information about two principals. It contains the name of a *subject*, and the subject's public key. It also contains a statement by a second principal, the *issuer*, that the subject is who they claim to be, and that this is indeed the subject's public key. The issuer's statement is signed with the issuer's private key, which only the issuer knows. However, anyone can verify the issuer's statement by finding the issuer's public key, decrypting the statement with it, and comparing it to the other information in the certificate. The certificate also contains information about the time period over which it is valid. This is expressed as two fields, called "notBefore" and "notAfter".

In the Python use of certificates, a client or server can use a certificate to prove who they are. The other side of a network connection can also be required to produce a certificate, and that certificate can be validated to the satisfaction of the client or server that requires such validation. The connection attempt can be set to raise an exception if the validation fails. Validation is done automatically, by the underlying OpenSSL framework; the application need not concern itself with its mechanics. But the application does usually need to provide sets of certificates to allow this process to take place.

Python uses files to contain certificates. They should be formatted as "PEM" (see **RFC 1422**), which is a base-64 encoded form wrapped with a header line and a footer line:

```
-----BEGIN CERTIFICATE-----
... (certificate in base64 PEM encoding) ...
-----END CERTIFICATE-----
```

Certificate chains

The Python files which contain certificates can contain a sequence of certificates, sometimes called a *certificate chain*. This chain should start with the specific certificate for the principal who "is" the client or server, and then the certificate for the issuer of that certificate, and then the certificate for the issuer of *that* certificate, and so on up the chain till you get to a certificate which is *self-signed*, that is, a certificate which has the same subject and issuer, sometimes called a *root certificate*. The certificates should just be concatenated together in the certificate file. For example, suppose we had a three certificate chain, from our server certificate to the certificate of the certification authority that signed our server certificate, to the root certificate of the agency which issued the certification authority's certificate:

```
-----BEGIN CERTIFICATE-----
... (certificate for your server)...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
... (the certificate for the CA)...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
... (the root certificate for the CA's issuer)...
-----END CERTIFICATE-----
```

CA certificates

If you are going to require validation of the other side of the connection's certificate, you need to provide a "CA certs" file, filled with the certificate chains for each issuer you are willing to trust. Again, this file just contains these chains concatenated together. For validation, Python will use the first chain it finds in the file which matches. The platform's certificates file can be used by calling *SSLContext.load_default_certs()*, this is done automatically with *create_default_context()*.

Combined key and certificate

Often the private key is stored in the same file as the certificate; in this case, only the `certfile` parameter to *SSLContext.load_cert_chain()* needs to be passed. If the private key is stored with the certificate, it should come before the first certificate in the certificate chain:

```
-----BEGIN RSA PRIVATE KEY-----
... (private key in base64 encoding) ...
-----END RSA PRIVATE KEY-----
-----BEGIN CERTIFICATE-----
... (certificate in base64 PEM encoding) ...
-----END CERTIFICATE-----
```

Self-signed certificates

If you are going to create a server that provides SSL-encrypted connection services, you will need to acquire a certificate for that service. There are many ways of acquiring appropriate certificates, such as buying one from a certification authority. Another common practice is to generate a self-signed certificate. The simplest way to do this is with the OpenSSL package, using something like the following:

```
% openssl req -new -x509 -days 365 -nodes -out cert.pem -keyout cert.pem
Generating a 1024 bit RSA private key
.......++++++
.............................++++++
writing new private key to 'cert.pem'
```

(continua na próxima página)

(continuação da página anterior)

```
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:US
State or Province Name (full name) [Some-State]:MyState
Locality Name (eg, city) []:Some City
Organization Name (eg, company) [Internet Widgits Pty Ltd]:My Organization, Inc.
Organizational Unit Name (eg, section) []:My Group
Common Name (eg, YOUR name) []:myserver.mygroup.myorganization.com
Email Address []:ops@myserver.mygroup.myorganization.com
%
```

The disadvantage of a self-signed certificate is that it is its own root certificate, and no one else will have it in their cache of known (and trusted) root certificates.

19.3.5 Exemplos

Testing for SSL support

To test for the presence of SSL support in a Python installation, user code should use the following idiom:

```
try:
    import ssl
except ImportError:
    pass
else:
    ...  # do something that requires SSL support
```

Client-side operation

This example creates a SSL context with the recommended security settings for client sockets, including automatic certificate verification:

```
>>> context = ssl.create_default_context()
```

If you prefer to tune security settings yourself, you might create a context from scratch (but beware that you might not get the settings right):

```
>>> context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
>>> context.load_verify_locations("/etc/ssl/certs/ca-bundle.crt")
```

(this snippet assumes your operating system places a bundle of all CA certificates in `/etc/ssl/certs/ca-bundle.crt`; if not, you'll get an error and have to adjust the location)

The *PROTOCOL_TLS_CLIENT* protocol configures the context for cert validation and hostname verification. *verify_mode* is set to *CERT_REQUIRED* and *check_hostname* is set to `True`. All other protocols create SSL contexts with insecure defaults.

When you use the context to connect to a server, *CERT_REQUIRED* and *check_hostname* validate the server certificate: it ensures that the server certificate was signed with one of the CA certificates, checks the signature for correctness, and verifies other properties like validity and identity of the hostname:

```
>>> conn = context.wrap_socket(socket.socket(socket.AF_INET),
...                            server_hostname="www.python.org")
>>> conn.connect(("www.python.org", 443))
```

You may then fetch the certificate:

```
>>> cert = conn.getpeercert()
```

Visual inspection shows that the certificate does identify the desired service (that is, the HTTPS host `www.python.org`):

```
>>> pprint.pprint(cert)
{'OCSP': ('http://ocsp.digicert.com',),
 'caIssuers': ('http://cacerts.digicert.com/DigiCertSHA2ExtendedValidationServerCA.
↪crt',),
 'crlDistributionPoints': ('http://crl3.digicert.com/sha2-ev-server-g1.crl',
                           'http://crl4.digicert.com/sha2-ev-server-g1.crl'),
 'issuer': ((('countryName', 'US'),),
            (('organizationName', 'DigiCert Inc'),),
            (('organizationalUnitName', 'www.digicert.com'),),
            (('commonName', 'DigiCert SHA2 Extended Validation Server CA'),)),
 'notAfter': 'Sep  9 12:00:00 2016 GMT',
 'notBefore': 'Sep  5 00:00:00 2014 GMT',
 'serialNumber': '01BB6F00122B177F36CAB49CEA8B6B26',
 'subject': ((('businessCategory', 'Private Organization'),),
             (('1.3.6.1.4.1.311.60.2.1.3', 'US'),),
             (('1.3.6.1.4.1.311.60.2.1.2', 'Delaware'),),
             (('serialNumber', '3359300'),),
             (('streetAddress', '16 Allen Rd'),),
             (('postalCode', '03894-4801'),),
             (('countryName', 'US'),),
             (('stateOrProvinceName', 'NH'),),
             (('localityName', 'Wolfeboro'),),
             (('organizationName', 'Python Software Foundation'),),
             (('commonName', 'www.python.org'),)),
 'subjectAltName': (('DNS', 'www.python.org'),
                    ('DNS', 'python.org'),
                    ('DNS', 'pypi.org'),
                    ('DNS', 'docs.python.org'),
                    ('DNS', 'testpypi.org'),
                    ('DNS', 'bugs.python.org'),
                    ('DNS', 'wiki.python.org'),
                    ('DNS', 'hg.python.org'),
                    ('DNS', 'mail.python.org'),
                    ('DNS', 'packaging.python.org'),
                    ('DNS', 'pythonhosted.org'),
                    ('DNS', 'www.pythonhosted.org'),
                    ('DNS', 'test.pythonhosted.org'),
                    ('DNS', 'us.pycon.org'),
                    ('DNS', 'id.python.org')),
 'version': 3}
```

Now the SSL channel is established and the certificate verified, you can proceed to talk with the server:

```
>>> conn.sendall(b"HEAD / HTTP/1.0\r\nHost: linuxfr.org\r\n\r\n")
>>> pprint.pprint(conn.recv(1024).split(b"\r\n"))
[b'HTTP/1.1 200 OK',
```

(continua na próxima página)

(continuação da página anterior)

```
 b'Date: Sat, 18 Oct 2014 18:27:20 GMT',
 b'Server: nginx',
 b'Content-Type: text/html; charset=utf-8',
 b'X-Frame-Options: SAMEORIGIN',
 b'Content-Length: 45679',
 b'Accept-Ranges: bytes',
 b'Via: 1.1 varnish',
 b'Age: 2188',
 b'X-Served-By: cache-lcy1134-LCY',
 b'X-Cache: HIT',
 b'X-Cache-Hits: 11',
 b'Vary: Cookie',
 b'Strict-Transport-Security: max-age=63072000; includeSubDomains',
 b'Connection: close',
 b'',
 b'']
```

See the discussion of *Considerações de segurança* below.

Server-side operation

For server operation, typically you'll need to have a server certificate, and private key, each in a file. You'll first create a context holding the key and the certificate, so that clients can check your authenticity. Then you'll open a socket, bind it to a port, call `listen()` on it, and start waiting for clients to connect:

```
import socket, ssl

context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile="mycertfile", keyfile="mykeyfile")

bindsocket = socket.socket()
bindsocket.bind(('myaddr.example.com', 10023))
bindsocket.listen(5)
```

When a client connects, you'll call `accept()` on the socket to get the new socket from the other end, and use the context's *`SSLContext.wrap_socket()`* method to create a server-side SSL socket for the connection:

```
while True:
    newsocket, fromaddr = bindsocket.accept()
    connstream = context.wrap_socket(newsocket, server_side=True)
    try:
        deal_with_client(connstream)
    finally:
        connstream.shutdown(socket.SHUT_RDWR)
        connstream.close()
```

Then you'll read data from the `connstream` and do something with it till you are finished with the client (or the client is finished with you):

```
def deal_with_client(connstream):
    data = connstream.recv(1024)
    # empty data means the client is finished with us
    while data:
        if not do_something(connstream, data):
            # we'll assume do_something returns False
            # when we're finished with client
            break
```

(continua na próxima página)

(continuação da página anterior)

```
        data = connstream.recv(1024)
    # finished with client
```

And go back to listening for new client connections (of course, a real server would probably handle each client connection in a separate thread, or put the sockets in *non-blocking mode* and use an event loop).

19.3.6 Notes on non-blocking sockets

SSL sockets behave slightly different than regular sockets in non-blocking mode. When working with non-blocking sockets, there are thus several things you need to be aware of:

- Most *SSLSocket* methods will raise either *SSLWantWriteError* or *SSLWantReadError* instead of *BlockingIOError* if an I/O operation would block. *SSLWantReadError* will be raised if a read operation on the underlying socket is necessary, and *SSLWantWriteError* for a write operation on the underlying socket. Note that attempts to *write* to an SSL socket may require *reading* from the underlying socket first, and attempts to *read* from the SSL socket may require a prior *write* to the underlying socket.

  Alterado na versão 3.5: In earlier Python versions, the `SSLSocket.send()` method returned zero instead of raising *SSLWantWriteError* or *SSLWantReadError*.

- Calling *select()* tells you that the OS-level socket can be read from (or written to), but it does not imply that there is sufficient data at the upper SSL layer. For example, only part of an SSL frame might have arrived. Therefore, you must be ready to handle `SSLSocket.recv()` and `SSLSocket.send()` failures, and retry after another call to *select()*.

- Conversely, since the SSL layer has its own framing, a SSL socket may still have data available for reading without *select()* being aware of it. Therefore, you should first call `SSLSocket.recv()` to drain any potentially available data, and then only block on a *select()* call if still necessary.

  (of course, similar provisions apply when using other primitives such as *poll()*, or those in the *selectors* module)

- The SSL handshake itself will be non-blocking: the *SSLSocket.do_handshake()* method has to be retried until it returns successfully. Here is a synopsis using *select()* to wait for the socket's readiness:

```
while True:
    try:
        sock.do_handshake()
        break
    except ssl.SSLWantReadError:
        select.select([sock], [], [])
    except ssl.SSLWantWriteError:
        select.select([], [sock], [])
```

Ver também

The *asyncio* module supports *non-blocking SSL sockets* and provides a higher level *Streams API*. It polls for events using the *selectors* module and handles *SSLWantWriteError*, *SSLWantReadError* and *BlockingIOError* exceptions. It runs the SSL handshake asynchronously as well.

19.3.7 Підтримка BIO пам'яті

Adicionado na versão 3.5.

Ever since the SSL module was introduced in Python 2.6, the *SSLSocket* class has provided two related but distinct areas of functionality:

- SSL protocol handling
- Network IO

The network IO API is identical to that provided by *socket.socket*, from which *SSLSocket* also inherits. This allows an SSL socket to be used as a drop-in replacement for a regular socket, making it very easy to add SSL support to an existing application.

Combining SSL protocol handling and network IO usually works well, but there are some cases where it doesn't. An example is async IO frameworks that want to use a different IO multiplexing model than the "select/poll on a file descriptor" (readiness based) model that is assumed by *socket.socket* and by the internal OpenSSL socket IO routines. This is mostly relevant for platforms like Windows where this model is not efficient. For this purpose, a reduced scope variant of *SSLSocket* called *SSLObject* is provided.

class ssl.**SSLObject**

A reduced-scope variant of *SSLSocket* representing an SSL protocol instance that does not contain any network IO methods. This class is typically used by framework authors that want to implement asynchronous IO for SSL through memory buffers.

This class implements an interface on top of a low-level SSL object as implemented by OpenSSL. This object captures the state of an SSL connection but does not provide any network IO itself. IO needs to be performed through separate "BIO" objects which are OpenSSL's IO abstraction layer.

This class has no public constructor. An *SSLObject* instance must be created using the *wrap_bio()* method. This method will create the *SSLObject* instance and bind it to a pair of BIOs. The *incoming* BIO is used to pass data from Python to the SSL protocol instance, while the *outgoing* BIO is used to pass data the other way around.

The following methods are available:

- *context*
- *server_side*
- *server_hostname*
- *session*
- *session_reused*
- *read()*
- *write()*
- *getpeercert()*
- *get_verified_chain()*
- *get_unverified_chain()*
- *selected_alpn_protocol()*
- *selected_npn_protocol()*
- *cipher()*
- *shared_ciphers()*
- *compression()*
- *pending()*
- *do_handshake()*
- *verify_client_post_handshake()*
- *unwrap()*
- *get_channel_binding()*
- *version()*

When compared to *SSLSocket*, this object lacks the following features:

- Any form of network IO; `recv()` and `send()` read and write only to the underlying *MemoryBIO* buffers.

- There is no *do_handshake_on_connect* machinery. You must always manually call `do_handshake()` to start the handshake.
- There is no handling of *suppress_ragged_eofs*. All end-of-file conditions that are in violation of the protocol are reported via the `SSLEOFError` exception.
- The method `unwrap()` call does not return anything, unlike for an SSL socket where it returns the underlying socket.
- The *server_name_callback* callback passed to `SSLContext.set_servername_callback()` will get an `SSLObject` instance instead of a `SSLSocket` instance as its first parameter.

Some notes related to the use of `SSLObject`:

- All IO on an `SSLObject` is *non-blocking*. This means that for example `read()` will raise an `SSLWantReadError` if it needs more data than the incoming BIO has available.

Alterado na versão 3.7: `SSLObject` instances must be created with `wrap_bio()`. In earlier versions, it was possible to create instances directly. This was never documented or officially supported.

An SSLObject communicates with the outside world using memory buffers. The class `MemoryBIO` provides a memory buffer that can be used for this purpose. It wraps an OpenSSL memory BIO (Basic IO) object:

class ssl.**MemoryBIO**

A memory buffer that can be used to pass data between Python and an SSL protocol instance.

pending

Return the number of bytes currently in the memory buffer.

eof

A boolean indicating whether the memory BIO is current at the end-of-file position.

read(*n=-1*, /)

Read up to *n* bytes from the memory buffer. If *n* is not specified or negative, all bytes are returned.

write(*buf*, /)

Write the bytes from *buf* to the memory BIO. The *buf* argument must be an object supporting the buffer protocol.

The return value is the number of bytes written, which is always equal to the length of *buf*.

write_eof()

Write an EOF marker to the memory BIO. After this method has been called, it is illegal to call `write()`. The attribute `eof` will become true after all data currently in the buffer has been read.

19.3.8 SSL session

Adicionado na versão 3.6.

class ssl.**SSLSession**

Session object used by `session`.

id

time

timeout

ticket_lifetime_hint

has_ticket

19.3.9 Considerações de segurança

Best defaults

For **client use**, if you don't have any special requirements for your security policy, it is highly recommended that you use the *create_default_context()* function to create your SSL context. It will load the system's trusted CA certificates, enable certificate validation and hostname checking, and try to choose reasonably secure protocol and cipher settings.

For example, here is how you would use the *smtplib.SMTP* class to create a trusted, secure connection to a SMTP server:

```
>>> import ssl, smtplib
>>> smtp = smtplib.SMTP("mail.python.org", port=587)
>>> context = ssl.create_default_context()
>>> smtp.starttls(context=context)
(220, b'2.0.0 Ready to start TLS')
```

If a client certificate is needed for the connection, it can be added with *SSLContext.load_cert_chain()*.

By contrast, if you create the SSL context by calling the *SSLContext* constructor yourself, it will not have certificate validation nor hostname checking enabled by default. If you do so, please read the paragraphs below to achieve a good security level.

Manual settings

Verifying certificates

When calling the *SSLContext* constructor directly, *CERT_NONE* is the default. Since it does not authenticate the other peer, it can be insecure, especially in client mode where most of the time you would like to ensure the authenticity of the server you're talking to. Therefore, when in client mode, it is highly recommended to use *CERT_REQUIRED*. However, it is in itself not sufficient; you also have to check that the server certificate, which can be obtained by calling *SSLSocket.getpeercert()*, matches the desired service. For many protocols and applications, the service can be identified by the hostname. This common check is automatically performed when *SSLContext.check_hostname* is enabled.

Alterado na versão 3.7: Hostname matchings is now performed by OpenSSL. Python no longer uses `match_hostname()`.

In server mode, if you want to authenticate your clients using the SSL layer (rather than using a higher-level authentication mechanism), you'll also have to specify *CERT_REQUIRED* and similarly check the client certificate.

Protocol versions

SSL versions 2 and 3 are considered insecure and are therefore dangerous to use. If you want maximum compatibility between clients and servers, it is recommended to use *PROTOCOL_TLS_CLIENT* or *PROTOCOL_TLS_SERVER* as the protocol version. SSLv2 and SSLv3 are disabled by default.

```
>>> client_context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
>>> client_context.minimum_version = ssl.TLSVersion.TLSv1_3
>>> client_context.maximum_version = ssl.TLSVersion.TLSv1_3
```

The SSL context created above will only allow TLSv1.3 and later (if supported by your system) connections to a server. *PROTOCOL_TLS_CLIENT* implies certificate validation and hostname checks by default. You have to load certificates into the context.

Cipher selection

If you have advanced security requirements, fine-tuning of the ciphers enabled when negotiating a SSL session is possible through the *SSLContext.set_ciphers()* method. Starting from Python 3.2.3, the ssl module disables certain weak ciphers by default, but you may want to further restrict the cipher choice. Be sure to read OpenSSL's

documentation about the cipher list format. If you want to check which ciphers are enabled by a given cipher list, use *SSLContext.get_ciphers()* or the `openssl ciphers` command on your system.

Multi-processing

If using this module as part of a multi-processed application (using, for example the *multiprocessing* or *concurrent.futures* modules), be aware that OpenSSL's internal random number generator does not properly handle forked processes. Applications must change the PRNG state of the parent process if they use any SSL feature with *os.fork()*. Any successful call of *RAND_add()* or *RAND_bytes()* is sufficient.

19.3.10 TLS 1.3

Adicionado na versão 3.7.

The TLS 1.3 protocol behaves slightly differently than previous version of TLS/SSL. Some new TLS 1.3 features are not yet available.

- TLS 1.3 uses a disjunct set of cipher suites. All AES-GCM and ChaCha20 cipher suites are enabled by default. The method *SSLContext.set_ciphers()* cannot enable or disable any TLS 1.3 ciphers yet, but *SSLContext.get_ciphers()* returns them.
- Session tickets are no longer sent as part of the initial handshake and are handled differently. *SSLSocket.session* and *SSLSession* are not compatible with TLS 1.3.
- Client-side certificates are also no longer verified during the initial handshake. A server can request a certificate at any time. Clients process certificate requests while they send or receive application data from the server.
- TLS 1.3 features like early data, deferred TLS client cert request, signature algorithm configuration, and re-keying are not supported yet.

Ver também

Class *socket.socket*
Documentation of underlying *socket* class

SSL/TLS Strong Encryption: An Introduction
Intro from the Apache HTTP Server documentation

RFC 1422: Privacy Enhancement for Internet Electronic Mail: Part II: Certificate-Based Key Management
Steve Kent

RFC 4086: Randomness Requirements for Security
Donald E., Jeffrey I. Schiller

RFC 5280: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile
D. Cooper

RFC 5246: The Transport Layer Security (TLS) Protocol Version 1.2
T. Dierks et. al.

RFC 6066: Transport Layer Security (TLS) Extensions
D. Eastlake

IANA TLS: Transport Layer Security (TLS) Parameters
IANA

RFC 7525: Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)
IETF

Mozilla's Server Side TLS recommendations
Mozilla

19.4 `select` — Waiting for I/O completion

Este módulo fornece acesso às funções `select()` e `poll()` disponíveis na maioria dos sistemas operacionais, `devpoll()` disponível no Solaris e derivados, `epoll()` disponível no Linux 2.5+ e `kqueue()` disponível na maioria dos BSD. Observe que no Windows, ele funciona apenas para sockets; em outros sistemas operacionais, ele também funciona para outros tipos de arquivo (em particular, no Unix, ele funciona em pipes). Ele não pode ser usado em arquivos regulares para determinar se um arquivo cresceu desde a última leitura.

Nota

O módulo *selectors* permite multiplexação de E/S de alto nível e eficiente, construída sobre as primitivas do módulo *select*. Os usuários são encorajados a usar o módulo *selectors* em vez disso, a menos que queiram controle preciso sobre as primitivas de nível de SO usadas.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

O módulo define o seguinte:

exception `select.error`

Um apelido descontinuado de *OSError*.

Alterado na versão 3.3: Seguindo a **PEP 3151**, esta classe foi transformada em um apelido de *OSError*.

`select.`**`devpoll`**`()`

(Suportado somente no Solaris e derivados.) Retorna um objeto de pesquisa `/dev/poll`; consulte a seção */dev/poll Polling Objects* abaixo para os métodos suportados pelos objetos devpoll.

Objetos `devpoll()` são vinculados ao número de descritores de arquivo permitidos no momento da instanciação. Se seu programa reduzir esse valor, `devpoll()` falhará. Se seu programa aumentar esse valor, `devpoll()` pode retornar uma lista incompleta de descritores de arquivo ativos.

O novo descritor de arquivo é *não herdável*.

Adicionado na versão 3.3.

Alterado na versão 3.4: O novo descritor de arquivo agora é não-hereditário.

`select.`**`epoll`**`(`*sizehint=-1*, *flags=0*`)`

(Only supported on Linux 2.5.44 and newer.) Return an edge polling object, which can be used as Edge or Level Triggered interface for I/O events.

sizehint informs epoll about the expected number of events to be registered. It must be positive, or `-1` to use the default. It is only used on older systems where `epoll_create1()` is not available; otherwise it has no effect (though its value is still checked).

flags is deprecated and completely ignored. However, when supplied, its value must be `0` or `select.EPOLL_CLOEXEC`, otherwise `OSError` is raised.

See the *Edge and Level Trigger Polling (epoll) Objects* section below for the methods supported by epolling objects.

`epoll` objects support the context management protocol: when used in a `with` statement, the new file descriptor is automatically closed at the end of the block.

O novo descritor de arquivo é *não herdável*.

Alterado na versão 3.3: Added the *flags* parameter.

Alterado na versão 3.4: Support for the `with` statement was added. The new file descriptor is now non-inheritable.

Descontinuado desde a versão 3.4: The *flags* parameter. `select.EPOLL_CLOEXEC` is used by default now. Use `os.set_inheritable()` to make the file descriptor inheritable.

`select.poll()`

(Not supported by all operating systems.) Returns a polling object, which supports registering and unregistering file descriptors, and then polling them for I/O events; see section *Polling Objects* below for the methods supported by polling objects.

`select.kqueue()`

(Only supported on BSD.) Returns a kernel queue object; see section *Kqueue Objects* below for the methods supported by kqueue objects.

O novo descritor de arquivo é *não herdável*.

Alterado na versão 3.4: O novo descritor de arquivo agora é não-hereditário.

`select.kevent`(*ident*, *filter=KQ_FILTER_READ*, *flags=KQ_EV_ADD*, *fflags=0*, *data=0*, *udata=0*)

(Only supported on BSD.) Returns a kernel event object; see section *Kevent Objects* below for the methods supported by kevent objects.

`select.select`(*rlist*, *wlist*, *xlist*[, *timeout*])

This is a straightforward interface to the Unix `select()` system call. The first three arguments are iterables of 'waitable objects': either integers representing file descriptors or objects with a parameterless method named `fileno()` returning such an integer:

- *rlist*: wait until ready for reading
- *wlist*: wait until ready for writing
- *xlist*: wait for an "exceptional condition" (see the manual page for what your system considers such a condition)

Empty iterables are allowed, but acceptance of three empty iterables is platform-dependent. (It is known to work on Unix but not on Windows.) The optional *timeout* argument specifies a time-out as a floating-point number in seconds. When the *timeout* argument is omitted the function blocks until at least one file descriptor is ready. A time-out value of zero specifies a poll and never blocks.

The return value is a triple of lists of objects that are ready: subsets of the first three arguments. When the time-out is reached without a file descriptor becoming ready, three empty lists are returned.

Among the acceptable object types in the iterables are Python *file objects* (e.g. `sys.stdin`, or objects returned by `open()` or `os.popen()`), socket objects returned by `socket.socket()`. You may also define a *wrapper* class yourself, as long as it has an appropriate `fileno()` method (that really returns a file descriptor, not just a random integer).

> **Nota**
>
> File objects on Windows are not acceptable, but sockets are. On Windows, the underlying `select()` function is provided by the WinSock library, and does not handle file descriptors that don't originate from WinSock.

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** for the rationale), instead of raising `InterruptedError`.

`select.PIPE_BUF`

The minimum number of bytes which can be written without blocking to a pipe when the pipe has been reported as ready for writing by `select()`, `poll()` or another interface in this module. This doesn't apply to other kind of file-like objects such as sockets.

This value is guaranteed by POSIX to be at least 512.

Disponibilidade: Unix

Adicionado na versão 3.2.

19.4.1 `/dev/poll` Polling Objects

Solaris and derivatives have `/dev/poll`. While `select()` is *O*(*highest file descriptor*) and `poll()` is *O*(*number of file descriptors*), `/dev/poll` is *O*(*active file descriptors*).

`/dev/poll` behaviour is very close to the standard `poll()` object.

`devpoll.close()`

Close the file descriptor of the polling object.

Adicionado na versão 3.4.

`devpoll.closed`

`True` if the polling object is closed.

Adicionado na versão 3.4.

`devpoll.fileno()`

Return the file descriptor number of the polling object.

Adicionado na versão 3.4.

`devpoll.register`(*fd*[, *eventmask*])

Register a file descriptor with the polling object. Future calls to the *poll()* method will then check whether the file descriptor has any pending I/O events. *fd* can be either an integer, or an object with a *fileno()* method that returns an integer. File objects implement `fileno()`, so they can also be used as the argument.

eventmask is an optional bitmask describing the type of events you want to check for. The constants are the same that with `poll()` object. The default value is a combination of the constants `POLLIN`, `POLLPRI`, and `POLLOUT`.

> **Aviso**
>
> Registering a file descriptor that's already registered is not an error, but the result is undefined. The appropriate action is to unregister or modify it first. This is an important difference compared with `poll()`.

`devpoll.modify`(*fd*[, *eventmask*])

This method does an *unregister()* followed by a *register()*. It is (a bit) more efficient that doing the same explicitly.

`devpoll.unregister`(*fd*)

Remove a file descriptor being tracked by a polling object. Just like the *register()* method, *fd* can be an integer or an object with a *fileno()* method that returns an integer.

Attempting to remove a file descriptor that was never registered is safely ignored.

`devpoll.poll`([*timeout*])

Polls the set of registered file descriptors, and returns a possibly empty list containing `(fd, event)` 2-tuples for the descriptors that have events or errors to report. *fd* is the file descriptor, and *event* is a bitmask with bits set for the reported events for that descriptor — `POLLIN` for waiting input, `POLLOUT` to indicate that the descriptor can be written to, and so forth. An empty list indicates that the call timed out and no file descriptors had any events to report. If *timeout* is given, it specifies the length of time in milliseconds which the system will wait for events before returning. If *timeout* is omitted, -1, or *None*, the call will block until there is an event for this poll object.

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** for the rationale), instead of raising *InterruptedError*.

19.4.2 Edge and Level Trigger Polling (epoll) Objects

https://linux.die.net/man/4/epoll

eventmask

Constante	Significado
EPOLLIN	Disponível para leitura
EPOLLOUT	Disponível para escrita
EPOLLPRI	Urgent data for read
EPOLLERR	Error condition happened on the assoc. fd
EPOLLHUP	Hang up happened on the assoc. fd
EPOLLET	Set Edge Trigger behavior, the default is Level Trigger behavior
EPOLLONESH	Set one-shot behavior. After one event is pulled out, the fd is internally disabled
EPOLLEXCLU	Wake only one epoll object when the associated fd has an event. The default (if this flag is not set) is to wake all epoll objects polling on a fd.
EPOLLRDHUP	Stream socket peer closed connection or shut down writing half of connection.
EPOLLRDNORM	Equivalent to EPOLLIN
EPOLLRDBAND	Priority data band can be read.
EPOLLWRNORM	Equivalente a EPOLLOUT
EPOLLWRBAND	Priority data may be written.
EPOLLMSG	Ignorado.
EPOLLWAKEUP	Prevents sleep during event waiting.

Adicionado na versão 3.6: EPOLLEXCLUSIVE was added. It's only supported by Linux Kernel 4.5 or later.

Adicionado na versão 3.14: EPOLLWAKEUP was added. It's only supported by Linux Kernel 3.5 or later.

epoll.**close**()

Close the control file descriptor of the epoll object.

epoll.**closed**

True if the epoll object is closed.

epoll.**fileno**()

Return the file descriptor number of the control fd.

epoll.**fromfd**(*fd*)

Create an epoll object from a given file descriptor.

epoll.**register**(*fd*[, *eventmask*])

Register a fd descriptor with the epoll object.

epoll.**modify**(*fd*, *eventmask*)

Modify a registered file descriptor.

epoll.**unregister**(*fd*)

Remove a registered file descriptor from the epoll object.

Alterado na versão 3.9: The method no longer ignores the *EBADF* error.

epoll.**poll**(*timeout=None*, *maxevents=-1*)

Wait for events. timeout in seconds (float)

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** for the rationale), instead of raising *InterruptedError*.

19.4.3 Polling Objects

The `poll()` system call, supported on most Unix systems, provides better scalability for network servers that service many, many clients at the same time. `poll()` scales better because the system call only requires listing the file descriptors of interest, while `select()` builds a bitmap, turns on bits for the fds of interest, and then afterward the whole bitmap has to be linearly scanned again. `select()` is *O*(*highest file descriptor*), while `poll()` is *O*(*number of file descriptors*).

`poll.`**`register`**(*fd*[, *eventmask*])

Register a file descriptor with the polling object. Future calls to the *poll()* method will then check whether the file descriptor has any pending I/O events. *fd* can be either an integer, or an object with a *fileno()* method that returns an integer. File objects implement `fileno()`, so they can also be used as the argument.

eventmask is an optional bitmask describing the type of events you want to check for, and can be a combination of the constants `POLLIN`, `POLLPRI`, and `POLLOUT`, described in the table below. If not specified, the default value used will check for all 3 types of events.

Constante	Significado
`POLLIN`	There is data to read
`POLLPRI`	There is urgent data to read
`POLLOUT`	Ready for output: writing will not block
`POLLERR`	Error condition of some sort
`POLLHUP`	Hung up
`POLLRDHUP`	Stream socket peer closed connection, or shut down writing half of connection
`POLLNVAL`	Invalid request: descriptor not open

Registering a file descriptor that's already registered is not an error, and has the same effect as registering the descriptor exactly once.

`poll.`**`modify`**(*fd*, *eventmask*)

Modifies an already registered fd. This has the same effect as `register(fd, eventmask)`. Attempting to modify a file descriptor that was never registered causes an *OSError* exception with errno `ENOENT` to be raised.

`poll.`**`unregister`**(*fd*)

Remove a file descriptor being tracked by a polling object. Just like the *register()* method, *fd* can be an integer or an object with a *fileno()* method that returns an integer.

Attempting to remove a file descriptor that was never registered causes a *KeyError* exception to be raised.

`poll.`**`poll`**([*timeout*])

Polls the set of registered file descriptors, and returns a possibly empty list containing `(fd, event)` 2-tuples for the descriptors that have events or errors to report. *fd* is the file descriptor, and *event* is a bitmask with bits set for the reported events for that descriptor — `POLLIN` for waiting input, `POLLOUT` to indicate that the descriptor can be written to, and so forth. An empty list indicates that the call timed out and no file descriptors had any events to report. If *timeout* is given, it specifies the length of time in milliseconds which the system will wait for events before returning. If *timeout* is omitted, negative, or *None*, the call will block until there is an event for this poll object.

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** for the rationale), instead of raising *InterruptedError*.

19.4.4 Kqueue Objects

`kqueue.`**`close`**()

Close the control file descriptor of the kqueue object.

kqueue.**closed**

`True` if the kqueue object is closed.

kqueue.**fileno**()

Return the file descriptor number of the control fd.

kqueue.**fromfd**(*fd*)

Create a kqueue object from a given file descriptor.

kqueue.**control**(*changelist*, *max_events*[, *timeout*]) → eventlist

Low level interface to kevent

- changelist must be an iterable of kevent objects or `None`
- max_events must be 0 or a positive integer
- timeout in seconds (floats possible); the default is `None`, to wait forever

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** for the rationale), instead of raising *InterruptedError*.

19.4.5 Kevent Objects

https://man.freebsd.org/cgi/man.cgi?query=kqueue&sektion=2

kevent.**ident**

Value used to identify the event. The interpretation depends on the filter but it's usually the file descriptor. In the constructor ident can either be an int or an object with a *fileno()* method. kevent stores the integer internally.

kevent.**filter**

Name of the kernel filter.

Constante	Significado
KQ_FILTER_READ	Takes a descriptor and returns whenever there is data available to read
KQ_FILTER_WRITE	Takes a descriptor and returns whenever there is data available to write
KQ_FILTER_AIO	AIO requests
KQ_FILTER_VNODE	Returns when one or more of the requested events watched in *fflag* occurs
KQ_FILTER_PROC	Watch for events on a process id
KQ_FILTER_NETDEV	Watch for events on a network device [not available on macOS]
KQ_FILTER_SIGNAL	Returns whenever the watched signal is delivered to the process
KQ_FILTER_TIMER	Establishes an arbitrary timer

kevent.**flags**

Filter action.

Constante	Significado
KQ_EV_ADD	Adds or modifies an event
KQ_EV_DELETE	Removes an event from the queue
KQ_EV_ENABLE	Permitscontrol() to returns the event
KQ_EV_DISABLE	Disablesevent
KQ_EV_ONESHOT	Removes event after first occurrence
KQ_EV_CLEAR	Reset the state after an event is retrieved
KQ_EV_SYSFLAGS	internal event
KQ_EV_FLAG1	internal event
KQ_EV_EOF	Filter specific EOF condition
KQ_EV_ERROR	See return values

kevent.**fflags**

Filter specific flags.

KQ_FILTER_READ and KQ_FILTER_WRITE filter flags:

Constante	Significado
KQ_NOTE_LOWAT	low water mark of a socket buffer

KQ_FILTER_VNODE filter flags:

Constante	Significado
KQ_NOTE_DELETE	*unlink()* was called
KQ_NOTE_WRITE	a write occurred
KQ_NOTE_EXTEND	the file was extended
KQ_NOTE_ATTRIB	an attribute was changed
KQ_NOTE_LINK	the link count has changed
KQ_NOTE_RENAME	the file was renamed
KQ_NOTE_REVOKE	access to the file was revoked

KQ_FILTER_PROC filter flags:

Constante	Significado
KQ_NOTE_EXIT	the process has exited
KQ_NOTE_FORK	the process has called *fork()*
KQ_NOTE_EXEC	the process has executed a new process
KQ_NOTE_PCTRLMASK	internal filter flag
KQ_NOTE_PDATAMASK	internal filter flag
KQ_NOTE_TRACK	follow a process across *fork()*
KQ_NOTE_CHILD	returned on the child process for *NOTE_TRACK*
KQ_NOTE_TRACKERR	unable to attach to a child

KQ_FILTER_NETDEV filter flags (not available on macOS):

Constante	Significado
KQ_NOTE_LINKUP	link is up
KQ_NOTE_LINKDOWN	link is down
KQ_NOTE_LINKINV	estado do link é inválido

kevent.**data**

Filter specific data.

kevent.**udata**

User defined value.

19.5 selectors — High-level I/O multiplexing

Adicionado na versão 3.4.

Código-fonte: Lib/selectors.py

19.5.1 Introdução

This module allows high-level and efficient I/O multiplexing, built upon the `select` module primitives. Users are encouraged to use this module instead, unless they want precise control over the OS-level primitives used.

It defines a `BaseSelector` abstract base class, along with several concrete implementations (`KqueueSelector`, `EpollSelector`...), that can be used to wait for I/O readiness notification on multiple file objects. In the following, "file object" refers to any object with a `fileno()` method, or a raw file descriptor. See *file object*.

`DefaultSelector` is an alias to the most efficient implementation available on the current platform: this should be the default choice for most users.

Nota

The type of file objects supported depends on the platform: on Windows, sockets are supported, but not pipes, whereas on Unix, both are supported (some other types may be supported as well, such as fifos or special file devices).

Ver também

select
Low-level I/O multiplexing module.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

19.5.2 Classes

Classes hierarchy:

```
BaseSelector
+-- SelectSelector
+-- PollSelector
+-- EpollSelector
+-- DevpollSelector
+-- KqueueSelector
```

In the following, *events* is a bitwise mask indicating which I/O events should be waited for on a given file object. It can be a combination of the modules constants below:

Constante	Significado
`selectors.`**`EVENT_READ`**	Disponível para leitura
`selectors.`**`EVENT_WRITE`**	Disponível para escrita

class selectors.**SelectorKey**

A `SelectorKey` is a `namedtuple` used to associate a file object to its underlying file descriptor, selected event mask and attached data. It is returned by several `BaseSelector` methods.

fileobj

File object registered.

fd

O descritor de arquivo subjacente.

events

Events that must be waited for on this file object.

data

Optional opaque data associated to this file object: for example, this could be used to store a per-client session ID.

class selectors.**BaseSelector**

A *BaseSelector* is used to wait for I/O event readiness on multiple file objects. It supports file stream registration, unregistration, and a method to wait for I/O events on those streams, with an optional timeout. It's an abstract base class, so cannot be instantiated. Use *DefaultSelector* instead, or one of *SelectSelector*, *KqueueSelector* etc. if you want to specifically use an implementation, and your platform supports it. *BaseSelector* and its concrete implementations support the *context manager* protocol.

abstractmethod register(*fileobj*, *events*, *data=None*)

Registra um objeto arquivo para seleção, monitorando-o para eventos de I/O

fileobj is the file object to monitor. It may either be an integer file descriptor or an object with a `fileno()` method. *events* is a bitwise mask of events to monitor. *data* is an opaque object.

This returns a new *SelectorKey* instance, or raises a *ValueError* in case of invalid event mask or file descriptor, or *KeyError* if the file object is already registered.

abstractmethod unregister(*fileobj*)

Unregister a file object from selection, removing it from monitoring. A file object shall be unregistered prior to being closed.

fileobj must be a file object previously registered.

This returns the associated *SelectorKey* instance, or raises a *KeyError* if *fileobj* is not registered. It will raise *ValueError* if *fileobj* is invalid (e.g. it has no `fileno()` method or its `fileno()` method has an invalid return value).

modify(*fileobj*, *events*, *data=None*)

Change a registered file object's monitored events or attached data.

This is equivalent to `BaseSelector.unregister(fileobj)` followed by `BaseSelector.register(fileobj, events, data)`, except that it can be implemented more efficiently.

This returns a new *SelectorKey* instance, or raises a *ValueError* in case of invalid event mask or file descriptor, or *KeyError* if the file object is not registered.

abstractmethod select(*timeout=None*)

Wait until some registered file objects become ready, or the timeout expires.

If `timeout > 0`, this specifies the maximum wait time, in seconds. If `timeout <= 0`, the call won't block, and will report the currently ready file objects. If *timeout* is `None`, the call will block until a monitored file object becomes ready.

This returns a list of `(key, events)` tuples, one for each ready file object.

key is the *SelectorKey* instance corresponding to a ready file object. *events* is a bitmask of events ready on this file object.

> **Nota**
>
> This method can return before any file object becomes ready or the timeout has elapsed if the current process receives a signal: in this case, an empty list will be returned.

Alterado na versão 3.5: The selector is now retried with a recomputed timeout when interrupted by a signal if the signal handler did not raise an exception (see **PEP 475** for the rationale), instead of returning an empty list of events before the timeout.

close()

Close the selector.

This must be called to make sure that any underlying resource is freed. The selector shall not be used once it has been closed.

get_key(*fileobj*)

Return the key associated with a registered file object.

This returns the *SelectorKey* instance associated to this file object, or raises *KeyError* if the file object is not registered.

abstractmethod get_map()

Return a mapping of file objects to selector keys.

This returns a *Mapping* instance mapping registered file objects to their associated *SelectorKey* instance.

class selectors.**DefaultSelector**

The default selector class, using the most efficient implementation available on the current platform. This should be the default choice for most users.

class selectors.**SelectSelector**

select.select()-based selector.

class selectors.**PollSelector**

select.poll()-based selector.

class selectors.**EpollSelector**

select.epoll()-based selector.

fileno()

This returns the file descriptor used by the underlying *select.epoll()* object.

class selectors.**DevpollSelector**

select.devpoll()-based selector.

fileno()

This returns the file descriptor used by the underlying *select.devpoll()* object.

Adicionado na versão 3.5.

class selectors.**KqueueSelector**

select.kqueue()-based selector.

fileno()

This returns the file descriptor used by the underlying *select.kqueue()* object.

19.5.3 Exemplos

Here is a simple echo server implementation:

```
import selectors
import socket

sel = selectors.DefaultSelector()

def accept(sock, mask):
    conn, addr = sock.accept()  # Should be ready
```

(continua na próxima página)

(continuação da página anterior)

```
    print('accepted', conn, 'from', addr)
    conn.setblocking(False)
    sel.register(conn, selectors.EVENT_READ, read)

def read(conn, mask):
    data = conn.recv(1000)  # Should be ready
    if data:
        print('echoing', repr(data), 'to', conn)
        conn.send(data)  # Hope it won't block
    else:
        print('closing', conn)
        sel.unregister(conn)
        conn.close()

sock = socket.socket()
sock.bind(('localhost', 1234))
sock.listen(100)
sock.setblocking(False)
sel.register(sock, selectors.EVENT_READ, accept)

while True:
    events = sel.select()
    for key, mask in events:
        callback = key.data
        callback(key.fileobj, mask)
```

19.6 `signal` — Define manipuladores para eventos assíncronos

Código-fonte: Lib/signal.py

Este módulo fornece mecanismos para usar manipuladores de sinal em Python.

19.6.1 Regras gerais

A função *signal.signal()* permite definir manipuladores personalizados a serem executados quando um sinal é recebido. Um pequeno número de manipuladores padrão são instalados: *SIGPIPE* é ignorado (então erros de gravação em encadeamentos e sockets podem ser relatados como exceções comuns do Python) e *SIGINT* é traduzido em uma exceção *KeyboardInterrupt* se o processo pai não a tiver alterado.

Um manipulador para um sinal específico, uma vez definido, permanece instalado até ser explicitamente redefinido (o Python emula a interface de estilo BSD independentemente da implementação subjacente), com exceção do manipulador para *SIGCHLD*, que segue a implementação subjacente.

Em plataformas WebAssembly, os sinais são emulados e, portanto, se comportam de forma diferente. Várias funções e sinais não estão disponíveis nessas plataformas.

Execução de manipuladores de sinais Python

Um manipulador de sinal Python não é executado dentro do manipulador de sinal de baixo nível (C). Em vez disso, o manipulador de sinal de baixo nível define um sinalizador que diz à *máquina virtual* para executar o manipulador de sinal Python correspondente em um ponto posterior (por exemplo, na próxima instrução *bytecode*). Isso tem consequências:

- Faz pouco sentido capturar erros síncronos como *SIGFPE* ou *SIGSEGV* que são causados por uma operação inválida no código C. O Python retornará do manipulador de sinais para o código C, o que provavelmente levantará o mesmo sinal novamente, fazendo com que o Python aparentemente trave. Do Python 3.3 em diante, você pode usar o módulo *faulthandler* para relatar erros síncronos.

- Um cálculo de longa duração implementado puramente em C (como correspondência de expressão regular em um grande corpo de texto) pode ser executado ininterruptamente por um período de tempo arbitrário, independentemente de quaisquer sinais recebidos. Os manipuladores de sinal do Python serão chamados quando o cálculo terminar.
- Se o manipulador levantar uma exceção, ela será levantada "do nada" na thread principal. Veja a *nota abaixo* para uma discussão.

Sinais e threads

Os manipuladores de sinais Python são sempre executados na thread principal do Python do interpretador principal, mesmo se o sinal foi recebido em outra thread. Isso significa que os sinais não podem ser usados como um meio de comunicação entre threads. Você pode usar as primitivas de sincronização do módulo *threading* em vez disso.

Além disso, somente a thread principal do interpretador principal tem permissão para definir um novo manipulador de sinal.

19.6.2 Conteúdo do módulo

Alterado na versão 3.5: As constantes relacionadas a sinal (SIG*), manipulador (*SIG_DFL*, *SIG_IGN*) e sigmask (*SIG_BLOCK*, *SIG_UNBLOCK*, *SIG_SETMASK*) listadas abaixo foram transformadas em *enums* (*Signals*, *Handlers* e *Sigmasks* respectivamente). As funções *getsignal()*, *pthread_sigmask()*, *sigpending()* e *sigwait()* retornam *enums* legíveis por humanos como objetos *Signals*.

O módulo de sinal define três enumerações:

class signal.**Signals**

Coleção de *enum.IntEnum* de constantes SIG* e constantes CTRL_*.

Adicionado na versão 3.5.

class signal.**Handlers**

Coleção de *enum.IntEnum* das constantes *SIG_DFL* e *SIG_IGN*.

Adicionado na versão 3.5.

class signal.**Sigmasks**

Coleção de *enum.IntEnum* das constantes *SIG_BLOCK*, *SIG_UNBLOCK* e *SIG_SETMASK*.

Disponibilidade: Unix.

Veja a página man *sigprocmask(2)* e *pthread_sigmask(3)* para mais informações.

Adicionado na versão 3.5.

As variáveis definidas no módulo *signal* são:

signal.**SIG_DFL**

Esta é uma das duas opções de manipulação de sinal padrão; ela simplesmente executará a função padrão para o sinal. Por exemplo, na maioria dos sistemas, a ação padrão para SIGQUIT é despejar o núcleo e sair, enquanto a ação padrão para *SIGCHLD* é simplesmente ignorá-lo.

signal.**SIG_IGN**

Este é outro manipulador de sinal padrão, que simplesmente ignorará o sinal fornecido.

signal.**SIGABRT**

Sinal de abortar de *abort(3)*.

signal.**SIGALRM**

Sinal do temporizador de *alarm(2)*.

Disponibilidade: Unix.

signal.**SIGBREAK**

Interrupção do teclado (CTRL + BREAK).

Disponibilidade: Windows.

signal.**SIGBUS**

Erro de barramento (acesso incorreto à memória).

Disponibilidade: Unix.

signal.**SIGCHLD**

Processo filho interrompido ou encerrado.

Disponibilidade: Unix.

signal.**SIGCLD**

Apelido para *SIGCHLD*.

Disponibilidade: not macOS.

signal.**SIGCONT**

Continua o processo se ele estiver parado no momento

Disponibilidade: Unix.

signal.**SIGFPE**

Exceção de ponto flutuante. Por exemplo, divisão por zero.

> **Ver também**
>
> *ZeroDivisionError* é levatanda quando o segundo argumento de uma operação de divisão ou módulo é zero.

signal.**SIGHUP**

Travamento detectado no terminal de controle ou morte do processo de controle.

Disponibilidade: Unix.

signal.**SIGILL**

Instrução ilegal.

signal.**SIGINT**

Interrupção do teclado (CTRL + C).

A ação padrão é levantar *KeyboardInterrupt*.

signal.**SIGKILL**

Sinal para encerrar forçadamente.

Ele não pode ser capturado, bloqueado ou ignorado.

Disponibilidade: Unix.

signal.**SIGPIPE**

Encadeamento quebrado: escreve no encadeamento sem leitores.

A ação padrão é ignorar o sinal.

Disponibilidade: Unix.

signal.**SIGSEGV**

Falha de segmentação: referência de memória inválida.

signal.**SIGSTKFLT**

Falha de pilha no coprocessador. O kernel Linux não levanta esse sinal: ele só pode ser emitido no espaço do usuário.

Disponibilidade: Linux.

Em arquiteturas onde o sinal está disponível. Veja a página man *signal(7)* para mais informações.

Adicionado na versão 3.11.

signal.**SIGTERM**

Sinal de encerramento.

signal.**SIGUSR1**

Sinal definido pelo usuário 1.

Disponibilidade: Unix.

signal.**SIGUSR2**

Sinal definido pelo usuário 2.

Disponibilidade: Unix.

signal.**SIGWINCH**

Sinal de redimensionamento do Windows.

Disponibilidade: Unix.

SIG*

Todos os números de sinal são definidos simbolicamente. Por exemplo, o sinal de desligamento é definido como *signal.SIGHUP*; os nomes das variáveis são idênticos aos usados em programas C, como encontrados em `<signal.h>`. A página man do Unix para '`signal()`' lista os sinais existentes (em alguns sistemas, é *signal(2)*, em outros, a lista está em *signal(7)*). Observe que nem todos os sistemas definem o mesmo conjunto de nomes de sinais; apenas os nomes definidos pelo sistema são definidos por este módulo.

signal.**CTRL_C_EVENT**

O sinal correspondente ao evento de pressionamento de tecla `Ctrl+C`. Este sinal só pode ser usado com *os.kill()*.

Disponibilidade: Windows.

Adicionado na versão 3.2.

signal.**CTRL_BREAK_EVENT**

O sinal correspondente ao evento de pressionamento de tecla `Ctrl+Break`. Este sinal só pode ser usado com *os.kill()*.

Disponibilidade: Windows.

Adicionado na versão 3.2.

signal.**NSIG**

Um a mais que o número do sinal mais alto. Use *valid_signals()* para obter números de sinais válidos.

signal.**ITIMER_REAL**

Diminui o intervalo do temporizador em tempo real e entrega *SIGALRM* ao expirar.

signal.**ITIMER_VIRTUAL**

Diminui o intervalo do temporizador somente quando o processo está em execução e entrega SIGVTALRM após a expiração.

signal.**ITIMER_PROF**

Diminui o temporizador de intervalo tanto quando o processo é executado quanto quando o sistema está executando em nome do processo. Juntamente com o ITIMER_VIRTUAL, este temporizador é geralmente usado para criar um perfil do tempo gasto pelo aplicativo nos espaços do usuário e do kernel. O SIGPROF é fornecido após a expiração.

signal.**SIG_BLOCK**

Um possível valor para o parâmetro *how* para *pthread_sigmask()* indicando que os sinais devem ser bloqueados.

Adicionado na versão 3.3.

signal.**SIG_UNBLOCK**

Um possível valor para o parâmetro *how* para *pthread_sigmask()* indicando que os sinais devem ser desbloqueados.

Adicionado na versão 3.3.

signal.**SIG_SETMASK**

Um possível valor para o parâmetro *how* para *pthread_sigmask()* indicando que a máscara de sinal deve ser substituída.

Adicionado na versão 3.3.

O módulo *signal* define uma exceção:

exception signal.**ItimerError**

Levantada para sinalizar um erro da implementação subjacente de *setitimer()* ou *getitimer()*. Espere este erro se um temporizador de intervalo inválido ou um tempo negativo for passado para *setitimer()*. Este erro é um subtipo de *OSError*.

Adicionado na versão 3.3: Este erro costumava ser um subtipo de *IOError*, que agora é um apelido de *OSError*.

O módulo *signal* define as seguintes funções:

signal.**alarm**(*time*)

Se *time* for diferente de zero, esta função solicita que um sinal *SIGALRM* seja enviado ao processo em *time* segundos. Qualquer alarme previamente agendado será cancelado (apenas um alarme pode ser agendado por vez). O valor retornado será o número de segundos antes de qualquer alarme previamente configurado ter sido emitido. Se *time* for zero, nenhum alarme será agendado e qualquer alarme agendado será cancelado. Se o valor retornado for zero, nenhum alarme será agendado no momento.

Disponibilidade: Unix.

Veja a página man *alarm(2)* para mais informações.

signal.**getsignal**(*signalnum*)

Retorna o manipulador de sinal atual para o sinal *signalnum*. O valor retornado pode ser um objeto Python invocável ou um dos valores especiais *signal.SIG_IGN*, *signal.SIG_DFL* ou *None*. Aqui, *signal.SIG_IGN* significa que o sinal foi ignorado anteriormente, *signal.SIG_DFL* significa que a maneira padrão de manipular o sinal estava em uso anteriormente e `None` significa que o manipulador de sinal anterior não foi instalado a partir do Python.

signal.**strsignal**(*signalnum*)

Retorna a descrição do sinal *signalnum*, como "Interrupt" para *SIGINT*. Retorna *None* se *signalnum* não tiver descrição. Levanta *ValueError* se *signalnum* for inválido.

Adicionado na versão 3.8.

signal.**valid_signals**()

Retorna o conjunto de números de sinais válidos nesta plataforma. Pode ser menor que `range(1, NSIG)` se alguns sinais forem reservados pelo sistema para uso interno.

Adicionado na versão 3.8.

signal.**pause**()

Faz o processo hibernar até que um sinal seja recebido; o manipulador apropriado será então chamado. Não retorna nada.

Disponibilidade: Unix.

Veja a página man *signal(2)* para mais informações.

veja também *sigwait()*, *sigwaitinfo()*, *sigtimedwait()* e *sigpending()*.

signal.**raise_signal**(*signum*)

Envia um sinal para o processo de chamada. Não retorna nada.

Adicionado na versão 3.8.

signal.**pidfd_send_signal**(*pidfd*, *sig*, *siginfo=None*, *flags=0*)

Envia o sinal *sig* para o processo referenciado pelo descritor de arquivo *pidfd*. Atualmente, o Python não suporta o parâmetro *siginfo*; ele deve ser `None`. O argumento *flags* é fornecido para futuras extensões; nenhum valor de sinalizador está definido no momento.

Veja a página man *pidfd_send_signal(2)* para mais informações.

Disponibilidade: Linux >= 5.1, Android >= *build-time* API level 31

Adicionado na versão 3.9.

signal.**pthread_kill**(*thread_id*, *signalnum*)

Envia o sinal *signalnum* para a thread *thread_id*, outra thread no mesmo processo que a chamadora. A thread alvo pode estar executando qualquer código (Python ou não). No entanto, se a thread alvo estiver executando o interpretador Python, os manipuladores de sinal Python serão *executados pela thread principal do interpretador principal*. Portanto, o único objetivo de enviar um sinal para uma thread Python específica seria forçar uma chamada de sistema em execução a falhar com *InterruptedError*.

Utilize a *threading.get_ident()* ou o atributo *ident* dos objetos *threading.Thread* para obter um valor adequado para *thread_id*.

Se *signalnum* for 0, nenhum sinal será enviado, mas a verificação de erros ainda será realizada; isso pode ser usado para verificar se a thread de destino ainda está em execução.

Levanta um *evento de auditoria* `signal.pthread_kill` com os argumentos `thread_id`, `signalnum`.

Disponibilidade: Unix.

Veja a página man *pthread_kill(3)* para mais informações.

Veja também *os.kill()*.

Adicionado na versão 3.3.

signal.**pthread_sigmask**(*how*, *mask*)

Busca e/ou altera a máscara de sinal da thread chamadora. A máscara de sinal é o conjunto de sinais cuja entrega está atualmente bloqueada para o chamador. Retorna a máscara de sinal antiga como um conjunto de sinais.

O comportamento da chamada depende do valor de *how*, como segue.

- *SIG_BLOCK*: O conjunto de sinais bloqueados é a união do conjunto atual e do argumento *mask*.
- *SIG_UNBLOCK*: Os sinais em *mask* são removidos do conjunto atual de sinais bloqueados. É permitido tentar desbloquear um sinal que não esteja bloqueado.
- *SIG_SETMASK*: O conjunto de sinais bloqueados é definido como o argumento *mask*.

mask é um conjunto de números de sinais (ex.: {*signal.SIGINT*, *signal.SIGTERM*}). Use *valid_signals()* para uma máscara completa, incluindo todos os sinais.

Por exemplo, `signal.pthread_sigmask(signal.SIG_BLOCK, [])` lê a máscara de sinal da thread de chamada.

SIGKILL e `SIGSTOP` não podem ser bloqueados.

Disponibilidade: Unix.

Veja a página man *sigprocmask(2)* e *pthread_sigmask(3)* para mais informações.

Veja também *pause()*, *sigpending()* e *sigwait()*.

Adicionado na versão 3.3.

signal.**setitimer**(*which*, *seconds*, *interval=0.0*)

Define o temporizador de intervalo fornecido (um dos seguintes: *signal.ITIMER_REAL*, *signal.ITIMER_VIRTUAL* ou *signal.ITIMER_PROF*) especificado por *which* para disparar após *seconds* (float é aceito, diferente de *alarm()*) e, depois disso, a cada *interval* segundos (se *interval* for diferente de zero). O temporizador de intervalo especificado por *which* pode ser zerado definindo *seconds* como zero.

Quando um temporizador de intervalo dispara, um sinal é enviado ao processo. O sinal enviado depende do temporizador utilizado; *signal.ITIMER_REAL* enviará *SIGALRM*, *signal.ITIMER_VIRTUAL* enviará SIGVTALRM e *signal.ITIMER_PROF* enviará SIGPROF.

Os valores antigos são retornados como uma tupla: (atraso, intervalo).

Tentar passar um intervalo de tempo inválido causará uma *ItimerError*.

Disponibilidade: Unix.

signal.**getitimer**(*which*)

Retorna o valor atual de um intervalo de tempo especificado por *which*.

Disponibilidade: Unix.

signal.**set_wakeup_fd**(*fd*, *, *warn_on_full_buffer=True*)

Define o descritor de arquivo de ativação como *fd*. Quando um sinal para o qual seu programa registrou um manipulador de sinais é recebido, o número do sinal é escrito como um único byte no fd. Se você não registrou um manipulador de sinais para os sinais de seu interesse, nada será escrito no fd de ativação. Isso pode ser usado por uma biblioteca para ativar uma chamada de poll ou seleção, permitindo que o sinal seja totalmente processado.

O antigo fd de ativação é retornado (ou -1 se a ativação do descritor de arquivo não estava habilitada). Se *fd* for -1, a ativação do descritor de arquivo está desabilitada. Se não for -1, *fd* deve ser não bloqueante. Cabe à biblioteca remover quaisquer bytes de *fd* antes de chamar poll ou select novamente.

Quando threads estão habilitadas, esta função só pode ser chamada da *thread principal do interpretador principal*; tentar chamá-la de outras threads levantará uma exceção *ValueError*.

Há duas maneiras comuns de usar esta função. Em ambas as abordagens, você usa o fd para despertar quando um sinal chega, mas elas diferem na forma como determinam *which* sinal ou sinais chegaram.

Na primeira abordagem, lemos os dados do buffer do fd, e os valores dos bytes fornecem os números dos sinais. Isso é simples, mas em casos raros pode apresentar um problema: geralmente, o fd terá um espaço de buffer limitado e, se muitos sinais chegarem muito rápido, o buffer pode ficar cheio e alguns sinais podem ser perdidos. Se você usar essa abordagem, defina `warn_on_full_buffer=True`, o que pelo menos fará com que um aviso seja impresso no stderr quando os sinais forem perdidos.

Na segunda abordagem, usamos o fd de ativação *apenas* para ativações e ignoramos os valores de bytes reais. Nesse caso, tudo o que importa é se o buffer do fd está vazio ou não; um buffer cheio não indica nenhum problema. Se você usar essa abordagem, defina `warn_on_full_buffer=False` para que seus usuários não sejam confundidos por mensagens de aviso falsas.

Alterado na versão 3.5: No Windows, a função agora também suporta manipuladores de socket.

Alterado na versão 3.7: Adiciona o parâmetro `warn_on_full_buffer`.

signal.**siginterrupt**(*signalnum*, *flag*)

Altera o comportamento de reinicialização de chamadas de sistema: se *flag* for *False*, as chamadas de sistema serão reiniciadas quando interrompidas pelo sinal *signalnum*; caso contrário, as chamadas de sistema serão interrompidas. Não retorna nada.

Disponibilidade: Unix.

Veja a página man *siginterrupt(3)* para mais informações.

Observe que a instalação de um manipulador de sinal com *signal()* redefinirá o comportamento de reinicialização para interrompível ao chamar implicitamente `siginterrupt()` com *flag* contendo valor verdadeiro para o sinal fornecido.

signal.**signal**(*signalnum*, *handler*)

Define o manipulador do sinal *signalnum* para a função *handler*. *handler* pode ser um objeto Python invocável que recebe dois argumentos (veja abaixo), ou um dos valores especiais *signal.SIG_IGN* ou *signal.SIG_DFL*. O manipulador de sinal anterior será retornado (veja a descrição de *getsignal()* acima). (Consulte a página man do Unix *signal(2)* para mais informações.)

Quando threads estão habilitadas, esta função só pode ser chamada da *thread principal do interpretador principal*; tentar chamá-la de outras threads levantará uma exceção *ValueError*.

O *handler* é chamado com dois argumentos: o número do sinal e o quadro de pilha atual (`None` ou um objeto quadro; para uma descrição dos objetos quadro, consulte a descrição na hierarquia de tipo ou consulte as descrições de atributo no módulo *inspect*).

No Windows, *signal()* só pode ser chamado com *SIGABRT*, *SIGFPE*, *SIGILL*, *SIGINT*, *SIGSEGV*, *SIGTERM* ou *SIGBREAK*. Uma exceção *ValueError* será levantada em qualquer outro caso. Observe que nem todos os sistemas definem o mesmo conjunto de nomes de sinais; uma exceção *AttributeError* será levantada se um nome de sinal não for definido como constante de nível de módulo `SIG*`.

signal.**sigpending**()

Examine o conjunto de sinais pendentes para entrega ao thread de chamada (ou seja, os sinais que foram gerados enquanto bloqueados). Retorne o conjunto de sinais pendentes.

Disponibilidade: Unix.

Veja a página man *sigpending(2)* para mais informações.

Veja também *pause()*, *pthread_sigmask()* e *sigwait()*.

Adicionado na versão 3.3.

signal.**sigwait**(*sigset*)

Suspende a execução do thread de chamada até a entrega de um dos sinais especificados no conjunto de sinais *sigset*. A função aceita o sinal (remove-o da lista de sinais pendentes) e retorna o número do sinal.

Disponibilidade: Unix.

Veja a página man *sigwait(3)* para mais informações.

Veja também *pause()*, *pthread_sigmask()*, *sigpending()*, *sigwaitinfo()* e *sigtimedwait()*.

Adicionado na versão 3.3.

signal.**sigwaitinfo**(*sigset*)

Suspende a execução da thread chamadora até a entrega de um dos sinais especificados no conjunto de sinais *sigset*. A função aceita o sinal e o remove da lista de sinais pendentes. Se um dos sinais em *sigset* já estiver pendente para a thread chamadora, a função retornará imediatamente com informações sobre esse sinal. O manipulador de sinais não é chamado para o sinal entregue. A função levanta *InterruptedError* se for interrompida por um sinal que não esteja em *sigset*.

O valor de retorno é um objeto que representa os dados contidos na estrutura de `siginfo_t`, a saber: `si_signo`, `si_code`, `si_errno`, `si_pid`, `si_uid`, `si_status`, `si_band`.

Disponibilidade: Unix.

Veja a página man *sigwaitinfo(2)* para mais informações.

Veja também *pause()*, *sigwait()* e *sigtimedwait()*.

Adicionado na versão 3.3.

Alterado na versão 3.5: A função agora é tentada novamente se interrompida por um sinal que não esteja em *sigset* e o manipulador de sinal não levanta uma exceção (veja **PEP 475** para a justificativa).

signal.**sigtimedwait**(*sigset*, *timeout*)

Semelhante a *sigwaitinfo()*, mas recebe um argumento *timeout* adicional que especifica um tempo limite. Se *timeout* for especificado como `0`, uma consulta será realizada. Retorna *None* se ocorrer um tempo limite.

Disponibilidade: Unix.

Veja a página man *sigtimedwait(2)* para mais informações.

Veja também *pause()*, *sigwait()* e *sigwaitinfo()*.

Adicionado na versão 3.3.

Alterado na versão 3.5: A função agora é tentada novamente com o *timeout* recalculado se interrompida por um sinal que não esteja em *sigset* e o manipulador de sinal não levanta uma exceção (veja **PEP 475** para a justificativa).

19.6.3 Exemplos

Aqui está um programa de exemplo mínimo. Ele usa a função *alarm()* para limitar o tempo gasto esperando para abrir um arquivo; isso é útil se o arquivo for para um dispositivo serial que pode não estar ligado, o que normalmente faria com que o *os.open()* travasse indefinidamente. A solução é definir um alarme de 5 segundos antes de abrir o arquivo; se a operação demorar muito, o sinal de alarme será enviado e o manipulador levantará uma exceção.

```
import signal, os

def handler(signum, frame):
    signame = signal.Signals(signum).name
    print(f'Signal handler called with signal {signame} ({signum})')
    raise OSError("Couldn't open device!")

# Define o manipulador de sinal e um alarme de 5 segundos
signal.signal(signal.SIGALRM, handler)
signal.alarm(5)

# Este open() pode travar indefinidamente
fd = os.open('/dev/ttyS0', os.O_RDWR)

signal.alarm(0)          # Desabilita o alarme
```

19.6.4 Nota sobre SIGPIPE

Canalizar a saída do seu programa para ferramentas como *head(1)* fará com que um sinal *SIGPIPE* seja enviado ao seu processo quando o receptor da saída padrão fechar antes do tempo. Isso resulta em uma exceção como `BrokenPipeError: [Errno 32] Broken pipe`. Para lidar com esse caso, envolva seu ponto de entrada para capturar essa exceção da seguinte maneira:

```
import os
import sys

def main():
    try:
        # simula uma saída grande (seu código substitui este loop)
        for x in range(10000):
            print("y")
        # limpa a saída aqui para forçar o acionamento do SIGPIPE
        # enquanto estiver dentro deste bloco try.
        sys.stdout.flush()
    except BrokenPipeError:
        # Python libera fluxos padrão na saída; redireciona a saída restante
        # para devnull para evitar outro BrokenPipeError no desligamento
        devnull = os.open(os.devnull, os.O_WRONLY)
        os.dup2(devnull, sys.stdout.fileno())
        sys.exit(1)  # Python exits with error code 1 on EPIPE
```

(continua na próxima página)

(continuação da página anterior)

```
if __name__ == '__main__':
    main()
```

Não defina a disposição de *SIGPIPE* como *SIG_DFL* para evitar *BrokenPipeError*. Isso faria com que seu programa encerrasse inesperadamente sempre que qualquer conexão de soquete fosse interrompida enquanto o programa ainda estivesse escrevendo nele.

19.6.5 Nota sobre manipuladores de sinais e exceções

Se um manipulador de sinais levantar uma exceção, a exceção será propagada para a thread principal e poderá ser levantada após qualquer instrução *bytecode*. Mais notavelmente, uma *KeyboardInterrupt* pode aparecer a qualquer momento durante a execução. A maioria dos códigos Python, incluindo a biblioteca padrão, não pode ser robusta contra isso, e, portanto, uma *KeyboardInterrupt* (ou qualquer outra exceção resultante de um manipulador de sinais) pode, em raras ocasiões, colocar o programa em um estado inesperado.

Para ilustrar esse problema, considere o seguinte código:

```
class SpamContext:
    def __init__(self):
        self.lock = threading.Lock()

    def __enter__(self):
        # Se ocorrer KeyboardInterrupt aqui, está tudo bem
        self.lock.acquire()
        # Se KeyboardInterrupt ocorrer aqui, __exit__ não será chamado
        ...
        # KeyboardInterrupt pode ocorrer logo antes do retorno da função

    def __exit__(self, exc_type, exc_val, exc_tb):
        ...
        self.lock.release()
```

Para muitos programas, especialmente aqueles que simplesmente desejam encerrar em *KeyboardInterrupt*, isso não é um problema, mas aplicações complexos ou que exigem alta confiabilidade devem evitar levantar exceções de manipuladores de sinal. Eles também devem evitar capturar *KeyboardInterrupt* como forma de encerrar o programa sem problemas. Em vez disso, devem instalar seu próprio manipulador *SIGINT*. Abaixo está um exemplo de um servidor HTTP que evita *KeyboardInterrupt*:

```
import signal
import socket
from selectors import DefaultSelector, EVENT_READ
from http.server import HTTPServer, SimpleHTTPRequestHandler

interrupt_read, interrupt_write = socket.socketpair()

def handler(signum, frame):
    print('Signal handler called with signal', signum)
    interrupt_write.send(b'\0')
signal.signal(signal.SIGINT, handler)

def serve_forever(httpd):
    sel = DefaultSelector()
    sel.register(interrupt_read, EVENT_READ)
    sel.register(httpd, EVENT_READ)

    while True:
        for key, _ in sel.select():
```

(continua na próxima página)

(continuação da página anterior)

```
            if key.fileobj == interrupt_read:
                interrupt_read.recv(1)
                return
            if key.fileobj == httpd:
                httpd.handle_request()

print("Serving on port 8000")
httpd = HTTPServer(('', 8000), SimpleHTTPRequestHandler)
serve_forever(httpd)
print("Shutdown...")
```

19.7 mmap — Suporte a arquivos mapeados na memória

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

Objetos arquivo mapeados na memória se comportam como *bytearray* e como *objetos arquivo*. Você pode usar objetos mmap na maioria dos lugares onde *bytearray* é esperado; por exemplo, você pode usar o módulo *re* para pesquisar um arquivo mapeado na memória. Você também pode alterar um único byte executando `obj[index] = 97` ou alterar uma subsequência atribuindo a uma fatia: `obj[i1:i2] = b'...'`. Você também pode ler e gravar dados começando na posição atual do arquivo e `seek()` através do arquivo para diferentes posições.

Um arquivo mapeado na memória é criado pelo construtor *mmap*, que é diferente no Unix e no Windows. Nos dois casos, você deve fornecer um descritor de arquivo para um arquivo aberto para atualização. Se você deseja mapear um objeto arquivo Python existente, use o método *fileno()* para obter o valor correto para o parâmetro *fileno*. Caso contrário, você pode abrir o arquivo usando a função *os.open()*, que retorna um descritor de arquivo diretamente (o arquivo ainda precisa ser fechado quando terminar).

Nota

Se você deseja criar um mapeamento de memória para um arquivo gravável e armazenado em buffer, deve usar *flush()* no arquivo primeiro. Isso é necessário para garantir que as modificações locais nos buffers estejam realmente disponíveis para o mapeamento.

Para as versões Unix e Windows do construtor, *access* pode ser especificado como um parâmetro opcional de palavra-chave. *access* aceita um dos quatro valores: `ACCESS_READ`, `ACCESS_WRITE` ou `ACCESS_COPY` para especificar memória somente leitura, gravação ou cópia na gravação, respectivamente `ACCESS_DEFAULT` para adiar para *prot*. *access* pode ser usado no Unix e no Windows. Se *access* não for especificado, o mmap do Windows retornará um mapeamento de gravação. Os valores iniciais da memória para todos os três tipos de acesso são obtidos do arquivo especificado. A atribuição a um mapa de memória `ACCESS_READ` gera uma exceção *TypeError*. A atribuição a um mapa de memória `ACCESS_WRITE` afeta a memória e o arquivo subjacente. A atribuição a um mapa de memória `ACCESS_COPY` afeta a memória, mas não atualiza o arquivo subjacente.

Alterado na versão 3.7: Adicionada a constante `ACCESS_DEFAULT`.

Para mapear a memória anônima, -1 deve ser passado como o fileno junto com o comprimento.

class mmap.**mmap**(*fileno*, *length*, *tagname=None*, *access=ACCESS_DEFAULT*, *offset=0*)

(Versão Windows) Mapeia *length* bytes do arquivo especificado pelo identificador de arquivo *fileno* e cria um objeto mmap. Se *length* for maior que o tamanho atual do arquivo, o arquivo será estendido para conter *length* bytes. Se *length* for `0`, o tamanho máximo do mapa será o tamanho atual do arquivo, exceto que, se o arquivo estiver vazio, o Windows levantará uma exceção (você não poderá criar um mapeamento vazio no Windows).

tagname, se especificado e não `None`, é uma string que fornece um nome de tag para o mapeamento. O Windows permite que você tenha muitos mapeamentos diferentes no mesmo arquivo. Se você especificar o nome de uma marca existente, essa marca será aberta; caso contrário, uma nova marca com esse nome será criada. Se este parâmetro for omitido ou `None`, o mapeamento será criado sem um nome. Evitar o uso do parâmetro *tagname* ajudará a manter seu código portátil entre o Unix e o Windows.

offset pode ser especificado como um deslocamento inteiro não negativo. As referências mmap serão relativas ao deslocamento do início do arquivo. *offset* tem como padrão 0. *offset* deve ser um múltiplo de `ALLOCATIONGRANULARITY`.

Levanta um *evento de auditoria* `mmap.__new__` com os argumentos `fileno`, `length`, `access`, `offset`.

class `mmap.`**mmap**(*fileno*, *length*, *flags=MAP_SHARED*, *prot=PROT_WRITE | PROT_READ*, *access=ACCESS_DEFAULT*, *offset=0*, *, *trackfd=True*)

(Versão Unix) Mapeia *length* bytes do arquivo especificado pelo descritor de arquivo *fileno* e retorna um objeto mmap. Se *length* for `0`, o comprimento máximo do mapa será o tamanho atual do arquivo quando *mmap* for chamado.

flags especifica a natureza do mapeamento. *MAP_PRIVATE* cria um mapeamento privado de cópia na gravação, de modo que alterações no conteúdo do objeto mmap serão privadas para este processo, e *MAP_SHARED* cria um mapeamento compartilhado com todos os outros processos que mapeiam as mesmas áreas do arquivo. O valor padrão é *MAP_SHARED*. Alguns sistemas têm sinalizadores adicionais possíveis, com a lista completa especificada em *constantes MAP_**.

prot, se especificado, fornece a proteção de memória desejada; os dois valores mais úteis são `PROT_READ` e `PROT_WRITE`, para especificar que as páginas podem ser lidas ou gravadas. *prot* tem como padrão `PROT_READ | PROT_WRITE`.

access pode ser especificado em vez de *flags* e *prot* como um parâmetro nomeado opcional. É um erro especificar *flags*, *prot* e *access*. Consulte a descrição de *access* acima para obter informações sobre como usar este parâmetro.

offset pode ser especificado como um deslocamento inteiro não negativo. As referências mmap serão relativas ao deslocamento do início do arquivo. *offset* tem como padrão 0. *offset* deve ser um múltiplo de `ALLOCATIONGRANULARITY`, que é igual a `PAGESIZE` em sistemas Unix.

Se *trackfd* for `False`, o descritor de arquivo especificado por *fileno* não será duplicado e o objeto `mmap` resultante não será associado ao arquivo subjacente do mapa. Isso significa que os métodos *size()* e *resize()* falharão. Este modo é útil para limitar o número de descritores de arquivo abertos.

Para garantir a validade do mapeamento de memória criado, o arquivo especificado pelo descritor *fileno* é sincronizado internamente de forma automática com o armazenamento de suporte físico no macOS.

Alterado na versão 3.13: The *trackfd* parameter was added.

This example shows a simple way of using *mmap*:

```
import mmap

# write a simple example file
with open("hello.txt", "wb") as f:
    f.write(b"Hello Python!\n")

with open("hello.txt", "r+b") as f:
    # memory-map the file, size 0 means whole file
    mm = mmap.mmap(f.fileno(), 0)
    # read content via standard file methods
    print(mm.readline())  # prints b"Hello Python!\n"
    # read content via slice notation
    print(mm[:5])  # prints b"Hello"
    # update content using slice notation;
    # note that new content must have same size
    mm[6:] = b" world!\n"
```

(continua na próxima página)

(continuação da página anterior)

```
    # ... and read again using standard file methods
    mm.seek(0)
    print(mm.readline())  # prints b"Hello  world!\n"
    # close the map
    mm.close()
```

mmap can also be used as a context manager in a `with` statement:

```
import mmap

with mmap.mmap(-1, 13) as mm:
    mm.write(b"Hello world!")
```

Adicionado na versão 3.2: Suporte a gerenciador de contexto.

The next example demonstrates how to create an anonymous map and exchange data between the parent and child processes:

```
import mmap
import os

mm = mmap.mmap(-1, 13)
mm.write(b"Hello world!")

pid = os.fork()

if pid == 0:  # In a child process
    mm.seek(0)
    print(mm.readline())

    mm.close()
```

Levanta um *evento de auditoria* `mmap.__new__` com os argumentos `fileno`, `length`, `access`, `offset`.

Memory-mapped file objects support the following methods:

close()

Closes the mmap. Subsequent calls to other methods of the object will result in a ValueError exception being raised. This will not close the open file.

closed

`True` if the file is closed.

Adicionado na versão 3.2.

find(*sub*[, *start*[, *end*]])

Returns the lowest index in the object where the subsequence *sub* is found, such that *sub* is contained in the range [*start*, *end*]. Optional arguments *start* and *end* are interpreted as in slice notation. Returns `-1` on failure.

Alterado na versão 3.5: Writable *bytes-like object* is now accepted.

flush()

flush(*offset*, *size*, /)

Flushes changes made to the in-memory copy of a file back to disk. Without use of this call there is no guarantee that changes are written back before the object is destroyed. If *offset* and *size* are specified, only changes to the given range of bytes will be flushed to disk; otherwise, the whole extent of the mapping is flushed. *offset* must be a multiple of the `PAGESIZE` or `ALLOCATIONGRANULARITY`.

`None` is returned to indicate success. An exception is raised when the call failed.

Alterado na versão 3.8: Previously, a nonzero value was returned on success; zero was returned on error under Windows. A zero value was returned on success; an exception was raised on error under Unix.

madvise(*option*[, *start*[, *length*]])

Send advice *option* to the kernel about the memory region beginning at *start* and extending *length* bytes. *option* must be one of the *MADV_* constants* available on the system. If *start* and *length* are omitted, the entire mapping is spanned. On some systems (including Linux), *start* must be a multiple of the `PAGESIZE`.

Availability: Systems with the `madvise()` system call.

Adicionado na versão 3.8.

move(*dest*, *src*, *count*)

Copy the *count* bytes starting at offset *src* to the destination index *dest*. If the mmap was created with `ACCESS_READ`, then calls to move will raise a *`TypeError`* exception.

read([*n*])

Return a *`bytes`* containing up to *n* bytes starting from the current file position. If the argument is omitted, `None` or negative, return all bytes from the current file position to the end of the mapping. The file position is updated to point after the bytes that were returned.

Alterado na versão 3.3: Argument can be omitted or `None`.

read_byte()

Returns a byte at the current file position as an integer, and advances the file position by 1.

readline()

Returns a single line, starting at the current file position and up to the next newline. The file position is updated to point after the bytes that were returned.

resize(*newsize*)

Resizes the map and the underlying file, if any.

Resizing a map created with *access* of `ACCESS_READ` or `ACCESS_COPY`, will raise a *`TypeError`* exception. Resizing a map created with *trackfd* set to `False`, will raise a *`ValueError`* exception.

On Windows: Resizing the map will raise an *`OSError`* if there are other maps against the same named file. Resizing an anonymous map (ie against the pagefile) will silently create a new map with the original data copied over up to the length of the new size.

Alterado na versão 3.11: Correctly fails if attempting to resize when another map is held Allows resize against an anonymous map on Windows

rfind(*sub*[, *start*[, *end*]])

Returns the highest index in the object where the subsequence *sub* is found, such that *sub* is contained in the range [*start*, *end*]. Optional arguments *start* and *end* are interpreted as in slice notation. Returns `-1` on failure.

Alterado na versão 3.5: Writable *bytes-like object* is now accepted.

seek(*pos*[, *whence*])

Set the file's current position. *whence* argument is optional and defaults to `os.SEEK_SET` or `0` (absolute file positioning); other values are `os.SEEK_CUR` or `1` (seek relative to the current position) and `os.SEEK_END` or `2` (seek relative to the file's end).

Alterado na versão 3.13: Return the new absolute position instead of `None`.

seekable()

Return whether the file supports seeking, and the return value is always `True`.

Adicionado na versão 3.13.

size()

Return the length of the file, which can be larger than the size of the memory-mapped area.

tell()

Returns the current position of the file pointer.

write(*bytes*)

Write the bytes in *bytes* into memory at the current position of the file pointer and return the number of bytes written (never less than `len(bytes)`, since if the write fails, a *ValueError* will be raised). The file position is updated to point after the bytes that were written. If the mmap was created with `ACCESS_READ`, then writing to it will raise a *TypeError* exception.

Alterado na versão 3.5: Writable *bytes-like object* is now accepted.

Alterado na versão 3.6: The number of bytes written is now returned.

write_byte(*byte*)

Write the integer *byte* into memory at the current position of the file pointer; the file position is advanced by `1`. If the mmap was created with `ACCESS_READ`, then writing to it will raise a *TypeError* exception.

19.7.1 Constantes MADV_*

mmap.**MADV_NORMAL**

mmap.**MADV_RANDOM**

mmap.**MADV_SEQUENTIAL**

mmap.**MADV_WILLNEED**

mmap.**MADV_DONTNEED**

mmap.**MADV_REMOVE**

mmap.**MADV_DONTFORK**

mmap.**MADV_DOFORK**

mmap.**MADV_HWPOISON**

mmap.**MADV_MERGEABLE**

mmap.**MADV_UNMERGEABLE**

mmap.**MADV_SOFT_OFFLINE**

mmap.**MADV_HUGEPAGE**

mmap.**MADV_NOHUGEPAGE**

mmap.**MADV_DONTDUMP**

mmap.**MADV_DODUMP**

mmap.**MADV_FREE**

mmap.**MADV_NOSYNC**

mmap.**MADV_AUTOSYNC**

mmap.**MADV_NOCORE**

mmap.**MADV_CORE**

mmap.**MADV_PROTECT**

mmap.**MADV_FREE_REUSABLE**

mmap.**MADV_FREE_REUSE**

These options can be passed to *mmap.madvise()*. Not every option will be present on every system.

Availability: Systems with the madvise() system call.

Adicionado na versão 3.8.

19.7.2 Constantes MAP_*

mmap.**MAP_SHARED**

mmap.**MAP_PRIVATE**

mmap.**MAP_32BIT**

mmap.**MAP_ALIGNED_SUPER**

mmap.**MAP_ANON**

mmap.**MAP_ANONYMOUS**

mmap.**MAP_CONCEAL**

mmap.**MAP_DENYWRITE**

mmap.**MAP_EXECUTABLE**

mmap.**MAP_HASSEMAPHORE**

mmap.**MAP_JIT**

mmap.**MAP_NOCACHE**

mmap.**MAP_NOEXTEND**

mmap.**MAP_NORESERVE**

mmap.**MAP_POPULATE**

mmap.**MAP_RESILIENT_CODESIGN**

mmap.**MAP_RESILIENT_MEDIA**

mmap.**MAP_STACK**

mmap.**MAP_TPRO**

mmap.**MAP_TRANSLATED_ALLOW_EXECUTE**

mmap.**MAP_UNIX03**

These are the various flags that can be passed to *mmap.mmap()*. *MAP_ALIGNED_SUPER* is only available at FreeBSD and *MAP_CONCEAL* is only available at OpenBSD. Note that some options might not be present on some systems.

Alterado na versão 3.10: Added *MAP_POPULATE* constant.

Adicionado na versão 3.11: Added *MAP_STACK* constant.

Adicionado na versão 3.12: Added *MAP_ALIGNED_SUPER* and *MAP_CONCEAL* constants.

Adicionado na versão 3.13: Added *MAP_32BIT*, *MAP_HASSEMAPHORE*, *MAP_JIT*, *MAP_NOCACHE*, *MAP_NOEXTEND*, *MAP_NORESERVE*, *MAP_RESILIENT_CODESIGN*, *MAP_RESILIENT_MEDIA*, *MAP_TPRO*, *MAP_TRANSLATED_ALLOW_EXECUTE*, and *MAP_UNIX03* constants.

CAPÍTULO 20

Manuseio de Dados na Internet

Este capítulo descreve módulos que suportam a manipulação de formatos de dados comumente usados na Internet.

20.1 `email` — Um e-mail e um pacote MIME manipulável

Código-fonte: Lib/email/__init__.py

O pacote `email` é uma biblioteca para gerenciar mensagens de e-mail. Ela foi especificamente *não* projetada para enviar mensagens de e-mail para SMTP (**RFC 2821**), NNTP ou outros servidores; essas são funções de módulos como `smtplib`. O pacote `email` tenta ser o mais compatível possível com RFC, suportando **RFC 5322** e **RFC 6532**, bem como os RFCs relacionados ao MIME como **RFC 2045**, **RFC 2046**, **RFC 2047**, **RFC 2183** e **RFC 2231**.

No geral a estrutura do pacote de e-mail pode ser dividida em três componentes principais, mais um quarto componente que controla o comportamento dos outros componentes.

O componente central do pacote é um "modelo de objeto" que representa mensagens de e-mail. Uma aplicação interage com o pacote principalmente através da interface do modelo de objeto definida no submódulo `message`. A aplicação pode usar essa API para fazer perguntas sobre um e-mail existente, construir um novo e-mail ou adicionar ou remover subcomponentes de e-mail que usam a mesma interface de modelo de objeto. Ou seja, seguindo a natureza das mensagens de e-mail e seus subcomponentes MIME, o modelo de objeto de e-mail é uma estrutura em árvore de objetos que fornecem a API `EmailMessage`.

Os outros dois componentes principais do pacote são `parser` e `generator`. O analisador sintático pega a versão serializada de uma mensagem de e-mail (um fluxo de bytes) e a converte em uma árvore de objetos `EmailMessage`. O gerador pega um `EmailMessage` e o transforma novamente em um fluxo de bytes serializado. (O analisador sintático e o gerador também lidam com fluxos de caracteres de texto, mas esse uso é desencorajado, pois é muito fácil terminar com mensagens que não são válidas de uma maneira ou de outra.)

O componente de controle é o módulo `policy`. Cada `EmailMessage`, cada `generator` e cada `parser` tem um objeto associado `policy` que controla seu comportamento. Normalmente, uma aplicação precisa especificar a política apenas quando uma `EmailMessage` é criada, instanciando diretamente uma `EmailMessage` para criar um novo e-mail ou analisando um fluxo de entrada usando um `parser`. Mas a política pode ser alterada quando a mensagem é serializada usando um `generator`. Isso permite, por exemplo, analisar uma mensagem de e-mail genérica do disco, mas serializá-la usando as configurações SMTP padrão ao enviá-la para um servidor de e-mail.

O pacote de e-mail faz o possível para ocultar os detalhes das várias RFCs em vigor da aplicação. Conceitualmente, a aplicação deve tratar a mensagem de e-mail como uma árvore estruturada de texto unicode e anexos binários, sem

ter que se preocupar com a forma como eles são representados quando serializados. Na prática, no entanto, muitas vezes é necessário estar ciente de pelo menos algumas das regras que regem as mensagens MIME e sua estrutura, especificamente os nomes e a natureza dos "tipos de conteúdo" MIME e como eles identificam documentos com várias partes. Na maioria das vezes, esse conhecimento só deve ser necessário para aplicações mais complexos e, mesmo assim, deve ser apenas a estrutura de alto nível em questão, e não os detalhes de como essas estruturas são representadas. Como os tipos de conteúdo MIME são amplamente utilizados no software moderno da Internet (não apenas no e-mail), este será um conceito familiar para muitos programadores.

As seções a seguir descrevem a funcionalidade do pacote `email`. Começamos com o modelo de objeto `message`, que é a interface principal que uma aplicação usará, e seguimos com os componentes de `parser` e `generator`. Em seguida, abordamos os controles `policy`, que concluem o tratamento dos principais componentes da biblioteca.

As próximas três seções cobrem as exceções que o pacote pode apresentar e os defeitos (não conformidade com as RFCs) que o `parser` pode detectar. A seguir, abordamos os subcomponentes `headerregistry` e os subcomponentes `contentmanager`, que fornecem ferramentas para manipulação mais detalhada de cabeçalhos e cargas úteis, respectivamente. Ambos os componentes contêm recursos relevantes para consumir e produzir mensagens não triviais, mas também documentam suas APIs de extensibilidade, que serão de interesse para aplicações avançadas.

A seguir, é apresentado um conjunto de exemplos de uso das partes fundamentais das APIs abordadas nas seções anteriores.

O exposto acima representa a API moderna (compatível com unicode) do pacote de e-mail. As seções restantes, começando com a classe `Message`, cobrem a API legada `compat32` que lida muito mais diretamente com os detalhes de como as mensagens de e-mail são representadas. A API `compat32` *não* oculta os detalhes dos RFCs da aplicação, mas para aplicações que precisam operar nesse nível, eles podem ser ferramentas úteis. Esta documentação também é relevante para aplicações que ainda estão usando a API `compat32` por motivos de compatibilidade com versões anteriores.

Alterado na versão 3.6: Documentos reorganizados e reescritos para promover a nova API `EmailMessage`/`EmailPolicy`.

Conteúdos da documentação do pacote `email`:

20.1.1 `email.message`: Representing an email message

Código-fonte: Lib/email/message.py

Adicionado na versão 3.6:[1]

The central class in the `email` package is the `EmailMessage` class, imported from the `email.message` module. It is the base class for the `email` object model. `EmailMessage` provides the core functionality for setting and querying header fields, for accessing message bodies, and for creating or modifying structured messages.

An email message consists of *headers* and a *payload* (which is also referred to as the *content*). Headers are **RFC 5322** or **RFC 6532** style field names and values, where the field name and value are separated by a colon. The colon is not part of either the field name or the field value. The payload may be a simple text message, or a binary object, or a structured sequence of sub-messages each with their own set of headers and their own payload. The latter type of payload is indicated by the message having a MIME type such as `multipart/*` or `message/rfc822`.

The conceptual model provided by an `EmailMessage` object is that of an ordered dictionary of headers coupled with a *payload* that represents the **RFC 5322** body of the message, which might be a list of sub-`EmailMessage` objects. In addition to the normal dictionary methods for accessing the header names and values, there are methods for accessing specialized information from the headers (for example the MIME content type), for operating on the payload, for generating a serialized version of the message, and for recursively walking over the object tree.

The `EmailMessage` dictionary-like interface is indexed by the header names, which must be ASCII values. The values of the dictionary are strings with some extra methods. Headers are stored and returned in case-preserving form, but field names are matched case-insensitively. The keys are ordered, but unlike a real dict, there can be duplicates. Additional methods are provided for working with headers that have duplicate keys.

[1] Originally added in 3.4 as a *provisional module*. Docs for legacy message class moved to *email.message.Message: Representing an email message using the compat32 API*.

The *payload* is either a string or bytes object, in the case of simple message objects, or a list of *EmailMessage* objects, for MIME container documents such as *multipart/** and *message/rfc822* message objects.

class email.message.**EmailMessage**(*policy=default*)

If *policy* is specified use the rules it specifies to update and serialize the representation of the message. If *policy* is not set, use the *default* policy, which follows the rules of the email RFCs except for line endings (instead of the RFC mandated \r\n, it uses the Python standard \n line endings). For more information see the *policy* documentation.

as_string(*unixfrom=False*, *maxheaderlen=None*, *policy=None*)

Return the entire message flattened as a string. When optional *unixfrom* is true, the envelope header is included in the returned string. *unixfrom* defaults to False. For backward compatibility with the base *Message* class *maxheaderlen* is accepted, but defaults to None, which means that by default the line length is controlled by the *max_line_length* of the policy. The *policy* argument may be used to override the default policy obtained from the message instance. This can be used to control some of the formatting produced by the method, since the specified *policy* will be passed to the *Generator*.

Flattening the message may trigger changes to the *EmailMessage* if defaults need to be filled in to complete the transformation to a string (for example, MIME boundaries may be generated or modified).

Note that this method is provided as a convenience and may not be the most useful way to serialize messages in your application, especially if you are dealing with multiple messages. See *email.generator.Generator* for a more flexible API for serializing messages. Note also that this method is restricted to producing messages serialized as "7 bit clean" when *utf8* is False, which is the default.

Alterado na versão 3.6: the default behavior when *maxheaderlen* is not specified was changed from defaulting to 0 to defaulting to the value of *max_line_length* from the policy.

__str__()

Equivalent to as_string(policy=self.policy.clone(utf8=True)). Allows str(msg) to produce a string containing the serialized message in a readable format.

Alterado na versão 3.4: the method was changed to use utf8=True, thus producing an **RFC 6531**-like message representation, instead of being a direct alias for *as_string()*.

as_bytes(*unixfrom=False*, *policy=None*)

Return the entire message flattened as a bytes object. When optional *unixfrom* is true, the envelope header is included in the returned string. *unixfrom* defaults to False. The *policy* argument may be used to override the default policy obtained from the message instance. This can be used to control some of the formatting produced by the method, since the specified *policy* will be passed to the *BytesGenerator*.

Flattening the message may trigger changes to the *EmailMessage* if defaults need to be filled in to complete the transformation to a string (for example, MIME boundaries may be generated or modified).

Note that this method is provided as a convenience and may not be the most useful way to serialize messages in your application, especially if you are dealing with multiple messages. See *email.generator.BytesGenerator* for a more flexible API for serializing messages.

__bytes__()

Equivalent to *as_bytes()*. Allows bytes(msg) to produce a bytes object containing the serialized message.

is_multipart()

Return True if the message's payload is a list of sub-*EmailMessage* objects, otherwise return False. When *is_multipart()* returns False, the payload should be a string object (which might be a CTE encoded binary payload). Note that *is_multipart()* returning True does not necessarily mean that "msg.get_content_maintype() == 'multipart'" will return the True. For example, is_multipart will return True when the *EmailMessage* is of type message/rfc822.

set_unixfrom(*unixfrom*)

Set the message's envelope header to *unixfrom*, which should be a string. (See *mboxMessage* for a brief description of this header.)

get_unixfrom()

Return the message's envelope header. Defaults to `None` if the envelope header was never set.

The following methods implement the mapping-like interface for accessing the message's headers. Note that there are some semantic differences between these methods and a normal mapping (i.e. dictionary) interface. For example, in a dictionary there are no duplicate keys, but here there may be duplicate message headers. Also, in dictionaries there is no guaranteed order to the keys returned by *keys()*, but in an *EmailMessage* object, headers are always returned in the order they appeared in the original message, or in which they were added to the message later. Any header deleted and then re-added is always appended to the end of the header list.

These semantic differences are intentional and are biased toward convenience in the most common use cases.

Note that in all cases, any envelope header present in the message is not included in the mapping interface.

__len__()

Return the total number of headers, including duplicates.

__contains__(*name*)

Return `True` if the message object has a field named *name*. Matching is done without regard to case and *name* does not include the trailing colon. Used for the `in` operator. For example:

```
if 'message-id' in myMessage:
   print('Message-ID:', myMessage['message-id'])
```

__getitem__(*name*)

Return the value of the named header field. *name* does not include the colon field separator. If the header is missing, `None` is returned; a *KeyError* is never raised.

Note that if the named field appears more than once in the message's headers, exactly which of those field values will be returned is undefined. Use the *get_all()* method to get the values of all the extant headers named *name*.

Using the standard (non-`compat32`) policies, the returned value is an instance of a subclass of *email.headerregistry.BaseHeader*.

__setitem__(*name*, *val*)

Add a header to the message with field name *name* and value *val*. The field is appended to the end of the message's existing headers.

Note that this does *not* overwrite or delete any existing header with the same name. If you want to ensure that the new header is the only one present in the message with field name *name*, delete the field first, e.g.:

```
del msg['subject']
msg['subject'] = 'Python roolz!'
```

If the *policy* defines certain headers to be unique (as the standard policies do), this method may raise a *ValueError* when an attempt is made to assign a value to such a header when one already exists. This behavior is intentional for consistency's sake, but do not depend on it as we may choose to make such assignments do an automatic deletion of the existing header in the future.

__delitem__(*name*)

Delete all occurrences of the field with name *name* from the message's headers. No exception is raised if the named field isn't present in the headers.

keys()

Return a list of all the message's header field names.

values()

Return a list of all the message's field values.

items()

Return a list of 2-tuples containing all the message's field headers and values.

get(*name*, *failobj=None*)

Return the value of the named header field. This is identical to `__getitem__()` except that optional *failobj* is returned if the named header is missing (*failobj* defaults to `None`).

Here are some additional useful header related methods:

get_all(*name*, *failobj=None*)

Return a list of all the values for the field named *name*. If there are no such named headers in the message, *failobj* is returned (defaults to `None`).

add_header(*_name*, *_value*, ***_params*)

Extended header setting. This method is similar to *__setitem__()* except that additional header parameters can be provided as keyword arguments. *_name* is the header field to add and *_value* is the *primary* value for the header.

For each item in the keyword argument dictionary *_params*, the key is taken as the parameter name, with underscores converted to dashes (since dashes are illegal in Python identifiers). Normally, the parameter will be added as `key="value"` unless the value is `None`, in which case only the key will be added.

If the value contains non-ASCII characters, the charset and language may be explicitly controlled by specifying the value as a three tuple in the format `(CHARSET, LANGUAGE, VALUE)`, where `CHARSET` is a string naming the charset to be used to encode the value, `LANGUAGE` can usually be set to `None` or the empty string (see **RFC 2231** for other possibilities), and `VALUE` is the string value containing non-ASCII code points. If a three tuple is not passed and the value contains non-ASCII characters, it is automatically encoded in **RFC 2231** format using a `CHARSET` of `utf-8` and a `LANGUAGE` of `None`.

Aqui está um exemplo:

```
msg.add_header('Content-Disposition', 'attachment', filename='bud.gif')
```

This will add a header that looks like

```
Content-Disposition: attachment; filename="bud.gif"
```

An example of the extended interface with non-ASCII characters:

```
msg.add_header('Content-Disposition', 'attachment',
               filename=('iso-8859-1', '', 'Fußballer.ppt'))
```

replace_header(*_name*, *_value*)

Replace a header. Replace the first header found in the message that matches *_name*, retaining header order and field name case of the original header. If no matching header is found, raise a *KeyError*.

get_content_type()

Return the message's content type, coerced to lower case of the form *maintype/subtype*. If there is no *Content-Type* header in the message return the value returned by *get_default_type()*. If the *Content-Type* header is invalid, return `text/plain`.

(According to **RFC 2045**, messages always have a default type, *get_content_type()* will always return a value. **RFC 2045** defines a message's default type to be *text/plain* unless it appears inside a *multipart/digest* container, in which case it would be *message/rfc822*. If the *Content-Type* header has an invalid type specification, **RFC 2045** mandates that the default type be *text/plain*.)

get_content_maintype()

Return the message's main content type. This is the *maintype* part of the string returned by *get_content_type()*.

get_content_subtype()

Return the message's sub-content type. This is the *subtype* part of the string returned by *get_content_type()*.

get_default_type()

Return the default content type. Most messages have a default content type of *text/plain*, except for messages that are subparts of *multipart/digest* containers. Such subparts have a default content type of *message/rfc822*.

set_default_type(*ctype*)

Set the default content type. *ctype* should either be *text/plain* or *message/rfc822*, although this is not enforced. The default content type is not stored in the *Content-Type* header, so it only affects the return value of the get_content_type methods when no *Content-Type* header is present in the message.

set_param(*param*, *value*, *header='Content-Type'*, *requote=True*, *charset=None*, *language=''*, *replace=False*)

Set a parameter in the *Content-Type* header. If the parameter already exists in the header, replace its value with *value*. When *header* is Content-Type (the default) and the header does not yet exist in the message, add it, set its value to *text/plain*, and append the new parameter value. Optional *header* specifies an alternative header to *Content-Type*.

If the value contains non-ASCII characters, the charset and language may be explicitly specified using the optional *charset* and *language* parameters. Optional *language* specifies the **RFC 2231** language, defaulting to the empty string. Both *charset* and *language* should be strings. The default is to use the utf8 *charset* and None for the *language*.

If *replace* is False (the default) the header is moved to the end of the list of headers. If *replace* is True, the header will be updated in place.

Uso do parâmetro *requote* com objetos *EmailMessage* está descontinuado.

Note that existing parameter values of headers may be accessed through the *params* attribute of the header value (for example, msg['Content-Type'].params['charset']).

Alterado na versão 3.4: Palavra-chave replace foi adicionada.

del_param(*param*, *header='content-type'*, *requote=True*)

Remove the given parameter completely from the *Content-Type* header. The header will be re-written in place without the parameter or its value. Optional *header* specifies an alternative to *Content-Type*.

Uso do parâmetro *requote* com objetos *EmailMessage* está descontinuado.

get_filename(*failobj=None*)

Return the value of the filename parameter of the *Content-Disposition* header of the message. If the header does not have a filename parameter, this method falls back to looking for the name parameter on the *Content-Type* header. If neither is found, or the header is missing, then *failobj* is returned. The returned string will always be unquoted as per *email.utils.unquote()*.

get_boundary(*failobj=None*)

Return the value of the boundary parameter of the *Content-Type* header of the message, or *failobj* if either the header is missing, or has no boundary parameter. The returned string will always be unquoted as per *email.utils.unquote()*.

set_boundary(*boundary*)

Set the boundary parameter of the *Content-Type* header to *boundary*. *set_boundary()* will always quote *boundary* if necessary. A *HeaderParseError* is raised if the message object has no *Content-Type* header.

Note that using this method is subtly different from deleting the old *Content-Type* header and adding a new one with the new boundary via *add_header()*, because *set_boundary()* preserves the order of the *Content-Type* header in the list of headers.

get_content_charset(*failobj=None*)

Return the charset parameter of the *Content-Type* header, coerced to lower case. If there is no *Content-Type* header, or if that header has no charset parameter, *failobj* is returned.

get_charsets(*failobj=None*)

Return a list containing the character set names in the message. If the message is a *multipart*, then the list will contain one element for each subpart in the payload, otherwise, it will be a list of length 1.

Each item in the list will be a string which is the value of the `charset` parameter in the *Content-Type* header for the represented subpart. If the subpart has no *Content-Type* header, no `charset` parameter, or is not of the *text* main MIME type, then that item in the returned list will be *failobj*.

is_attachment()

Return `True` if there is a *Content-Disposition* header and its (case insensitive) value is `attachment`, `False` otherwise.

Alterado na versão 3.4.2: is_attachment is now a method instead of a property, for consistency with *is_multipart()*.

get_content_disposition()

Return the lowercased value (without parameters) of the message's *Content-Disposition* header if it has one, or `None`. The possible values for this method are *inline*, *attachment* or `None` if the message follows **RFC 2183**.

Adicionado na versão 3.5.

The following methods relate to interrogating and manipulating the content (payload) of the message.

walk()

The *walk()* method is an all-purpose generator which can be used to iterate over all the parts and subparts of a message object tree, in depth-first traversal order. You will typically use *walk()* as the iterator in a `for` loop; each iteration returns the next subpart.

Here's an example that prints the MIME type of every part of a multipart message structure:

```
>>> for part in msg.walk():
...     print(part.get_content_type())
multipart/report
text/plain
message/delivery-status
text/plain
text/plain
message/rfc822
text/plain
```

`walk` iterates over the subparts of any part where *is_multipart()* returns `True`, even though `msg.get_content_maintype() == 'multipart'` may return `False`. We can see this in our example by making use of the `_structure` debug helper function:

```
>>> from email.iterators import _structure
>>> for part in msg.walk():
...     print(part.get_content_maintype() == 'multipart',
...           part.is_multipart())
True True
False False
False True
False False
False False
False True
False False
>>> _structure(msg)
multipart/report
    text/plain
    message/delivery-status
        text/plain
```

(continua na próxima página)

(continuação da página anterior)

```
        text/plain
    message/rfc822
        text/plain
```

Here the `message` parts are not `multiparts`, but they do contain subparts. `is_multipart()` returns `True` and `walk` descends into the subparts.

get_body(*preferencelist=('related', 'html', 'plain')*)

Return the MIME part that is the best candidate to be the "body" of the message.

preferencelist must be a sequence of strings from the set `related`, `html`, and `plain`, and indicates the order of preference for the content type of the part returned.

Start looking for candidate matches with the object on which the `get_body` method is called.

If `related` is not included in *preferencelist*, consider the root part (or subpart of the root part) of any related encountered as a candidate if the (sub-)part matches a preference.

When encountering a `multipart/related`, check the `start` parameter and if a part with a matching *`Content-ID`* is found, consider only it when looking for candidate matches. Otherwise consider only the first (default root) part of the `multipart/related`.

If a part has a *`Content-Disposition`* header, only consider the part a candidate match if the value of the header is `inline`.

If none of the candidates matches any of the preferences in *preferencelist*, return `None`.

Notes: (1) For most applications the only *preferencelist* combinations that really make sense are `('plain',)`, `('html', 'plain')`, and the default `('related', 'html', 'plain')`. (2) Because matching starts with the object on which `get_body` is called, calling `get_body` on a `multipart/related` will return the object itself unless *preferencelist* has a non-default value. (3) Messages (or message parts) that do not specify a *`Content-Type`* or whose *`Content-Type`* header is invalid will be treated as if they are of type `text/plain`, which may occasionally cause `get_body` to return unexpected results.

iter_attachments()

Return an iterator over all of the immediate sub-parts of the message that are not candidate "body" parts. That is, skip the first occurrence of each of `text/plain`, `text/html`, `multipart/related`, or `multipart/alternative` (unless they are explicitly marked as attachments via *`Content-Disposition: attachment`*), and return all remaining parts. When applied directly to a `multipart/related`, return an iterator over the all the related parts except the root part (ie: the part pointed to by the `start` parameter, or the first part if there is no `start` parameter or the `start` parameter doesn't match the *`Content-ID`* of any of the parts). When applied directly to a `multipart/alternative` or a non-`multipart`, return an empty iterator.

iter_parts()

Return an iterator over all of the immediate sub-parts of the message, which will be empty for a non-`multipart`. (See also *`walk()`*.)

get_content(**args*, *content_manager=None*, ***kw*)

Call the *`get_content()`* method of the *content_manager*, passing self as the message object, and passing along any other arguments or keywords as additional arguments. If *content_manager* is not specified, use the `content_manager` specified by the current *`policy`*.

set_content(**args*, *content_manager=None*, ***kw*)

Call the *`set_content()`* method of the *content_manager*, passing self as the message object, and passing along any other arguments or keywords as additional arguments. If *content_manager* is not specified, use the `content_manager` specified by the current *`policy`*.

make_related(*boundary=None*)

Convert a non-`multipart` message into a `multipart/related` message, moving any existing *`Content-`* headers and payload into a (new) first part of the `multipart`. If *boundary* is specified,

use it as the boundary string in the multipart, otherwise leave the boundary to be automatically created when it is needed (for example, when the message is serialized).

make_alternative(*boundary=None*)

Convert a non-`multipart` or a `multipart/related` into a `multipart/alternative`, moving any existing `Content-` headers and payload into a (new) first part of the `multipart`. If *boundary* is specified, use it as the boundary string in the multipart, otherwise leave the boundary to be automatically created when it is needed (for example, when the message is serialized).

make_mixed(*boundary=None*)

Convert a non-`multipart`, a `multipart/related`, or a `multipart-alternative` into a `multipart/mixed`, moving any existing `Content-` headers and payload into a (new) first part of the `multipart`. If *boundary* is specified, use it as the boundary string in the multipart, otherwise leave the boundary to be automatically created when it is needed (for example, when the message is serialized).

add_related(**args*, *content_manager=None*, ***kw*)

If the message is a `multipart/related`, create a new message object, pass all of the arguments to its `set_content()` method, and `attach()` it to the `multipart`. If the message is a non-`multipart`, call `make_related()` and then proceed as above. If the message is any other type of `multipart`, raise a `TypeError`. If *content_manager* is not specified, use the `content_manager` specified by the current `policy`. If the added part has no `Content-Disposition` header, add one with the value `inline`.

add_alternative(**args*, *content_manager=None*, ***kw*)

If the message is a `multipart/alternative`, create a new message object, pass all of the arguments to its `set_content()` method, and `attach()` it to the `multipart`. If the message is a non-`multipart` or `multipart/related`, call `make_alternative()` and then proceed as above. If the message is any other type of `multipart`, raise a `TypeError`. If *content_manager* is not specified, use the `content_manager` specified by the current `policy`.

add_attachment(**args*, *content_manager=None*, ***kw*)

If the message is a `multipart/mixed`, create a new message object, pass all of the arguments to its `set_content()` method, and `attach()` it to the `multipart`. If the message is a non-`multipart`, `multipart/related`, or `multipart/alternative`, call `make_mixed()` and then proceed as above. If *content_manager* is not specified, use the `content_manager` specified by the current `policy`. If the added part has no `Content-Disposition` header, add one with the value `attachment`. This method can be used both for explicit attachments (`Content-Disposition: attachment`) and `inline` attachments (`Content-Disposition: inline`), by passing appropriate options to the `content_manager`.

clear()

Remove the payload and all of the headers.

clear_content()

Remove the payload and all of the `!Content-` headers, leaving all other headers intact and in their original order.

`EmailMessage` objects have the following instance attributes:

preamble

The format of a MIME document allows for some text between the blank line following the headers, and the first multipart boundary string. Normally, this text is never visible in a MIME-aware mail reader because it falls outside the standard MIME armor. However, when viewing the raw text of the message, or when viewing the message in a non-MIME aware reader, this text can become visible.

The *preamble* attribute contains this leading extra-armor text for MIME documents. When the `Parser` discovers some text after the headers but before the first boundary string, it assigns this text to the message's *preamble* attribute. When the `Generator` is writing out the plain text representation of a MIME message, and it finds the message has a *preamble* attribute, it will write this text in the area between the headers and the first boundary. See `email.parser` and `email.generator` for details.

Note that if the message object has no preamble, the *preamble* attribute will be `None`.

epilogue

The *epilogue* attribute acts the same way as the *preamble* attribute, except that it contains text that appears between the last boundary and the end of the message. As with the `preamble`, if there is no epilog text this attribute will be `None`.

defects

The *defects* attribute contains a list of all the problems found when parsing this message. See `email.errors` for a detailed description of the possible parsing defects.

class email.message.**MIMEPart**(*policy=default*)

This class represents a subpart of a MIME message. It is identical to `EmailMessage`, except that no `MIME-Version` headers are added when `set_content()` is called, since sub-parts do not need their own `MIME-Version` headers.

20.1.2 `email.parser`: Parsing email messages

Código-fonte: Lib/email/parser.py

Message object structures can be created in one of two ways: they can be created from whole cloth by creating an `EmailMessage` object, adding headers using the dictionary interface, and adding payload(s) using `set_content()` and related methods, or they can be created by parsing a serialized representation of the email message.

The `email` package provides a standard parser that understands most email document structures, including MIME documents. You can pass the parser a bytes, string or file object, and the parser will return to you the root `EmailMessage` instance of the object structure. For simple, non-MIME messages the payload of this root object will likely be a string containing the text of the message. For MIME messages, the root object will return `True` from its `is_multipart()` method, and the subparts can be accessed via the payload manipulation methods, such as `get_body()`, `iter_parts()`, and `walk()`.

There are actually two parser interfaces available for use, the `Parser` API and the incremental `FeedParser` API. The `Parser` API is most useful if you have the entire text of the message in memory, or if the entire message lives in a file on the file system. `FeedParser` is more appropriate when you are reading the message from a stream which might block waiting for more input (such as reading an email message from a socket). The `FeedParser` can consume and parse the message incrementally, and only returns the root object when you close the parser.

Note that the parser can be extended in limited ways, and of course you can implement your own parser completely from scratch. All of the logic that connects the `email` package's bundled parser and the `EmailMessage` class is embodied in the `Policy` class, so a custom parser can create message object trees any way it finds necessary by implementing custom versions of the appropriate `Policy` methods.

API do FeedParser

The `BytesFeedParser`, imported from the `email.feedparser` module, provides an API that is conducive to incremental parsing of email messages, such as would be necessary when reading the text of an email message from a source that can block (such as a socket). The `BytesFeedParser` can of course be used to parse an email message fully contained in a *bytes-like object*, string, or file, but the `BytesParser` API may be more convenient for such use cases. The semantics and results of the two parser APIs are identical.

The `BytesFeedParser`'s API is simple; you create an instance, feed it a bunch of bytes until there's no more to feed it, then close the parser to retrieve the root message object. The `BytesFeedParser` is extremely accurate when parsing standards-compliant messages, and it does a very good job of parsing non-compliant messages, providing information about how a message was deemed broken. It will populate a message object's `defects` attribute with a list of any problems it found in a message. See the `email.errors` module for the list of defects that it can find.

Here is the API for the `BytesFeedParser`:

class email.parser.**BytesFeedParser**(*_factory=None*, *, *policy=policy.compat32*)

Create a `BytesFeedParser` instance. Optional *_factory* is a no-argument callable; if not specified use the `message_factory` from the *policy*. Call *_factory* whenever a new message object is needed.

If *policy* is specified use the rules it specifies to update the representation of the message. If *policy* is not set, use the *compat32* policy, which maintains backward compatibility with the Python 3.2 version of the email package and provides *Message* as the default factory. All other policies provide *EmailMessage* as the default *_factory*. For more information on what else *policy* controls, see the *policy* documentation.

Note: **The policy keyword should always be specified**; The default will change to *email.policy.default* in a future version of Python.

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionada o argumento nomeado *policy*.

Alterado na versão 3.6: *_factory* defaults to the policy `message_factory`.

feed(*data*)

Feed the parser some more data. *data* should be a *bytes-like object* containing one or more lines. The lines can be partial and the parser will stitch such partial lines together properly. The lines can have any of the three common line endings: carriage return, newline, or carriage return and newline (they can even be mixed).

close()

Complete the parsing of all previously fed data and return the root message object. It is undefined what happens if *feed()* is called after this method has been called.

class email.parser.**FeedParser**(*_factory=None*, *, *policy=policy.compat32*)

Works like *BytesFeedParser* except that the input to the *feed()* method must be a string. This is of limited utility, since the only way for such a message to be valid is for it to contain only ASCII text or, if *utf8* is `True`, no binary attachments.

Alterado na versão 3.3: Adicionada o argumento nomeado *policy*.

Parser API

The *BytesParser* class, imported from the *email.parser* module, provides an API that can be used to parse a message when the complete contents of the message are available in a *bytes-like object* or file. The *email.parser* module also provides *Parser* for parsing strings, and header-only parsers, *BytesHeaderParser* and *HeaderParser*, which can be used if you're only interested in the headers of the message. *BytesHeaderParser* and *HeaderParser* can be much faster in these situations, since they do not attempt to parse the message body, instead setting the payload to the raw body.

class email.parser.**BytesParser**(*_class=None*, *, *policy=policy.compat32*)

Create a *BytesParser* instance. The *_class* and *policy* arguments have the same meaning and semantics as the *_factory* and *policy* arguments of *BytesFeedParser*.

Note: **The policy keyword should always be specified**; The default will change to *email.policy.default* in a future version of Python.

Alterado na versão 3.3: Removed the *strict* argument that was deprecated in 2.4. Added the *policy* keyword.

Alterado na versão 3.6: *_class* defaults to the policy `message_factory`.

parse(*fp*, *headersonly=False*)

Read all the data from the binary file-like object *fp*, parse the resulting bytes, and return the message object. *fp* must support both the *readline()* and the `read()` methods.

The bytes contained in *fp* must be formatted as a block of **RFC 5322** (or, if *utf8* is `True`, **RFC 6532**) style headers and header continuation lines, optionally preceded by an envelope header. The header block is terminated either by the end of the data or by a blank line. Following the header block is the body of the message (which may contain MIME-encoded subparts, including subparts with a *Content-Transfer-Encoding* of `8bit`).

Optional *headersonly* is a flag specifying whether to stop parsing after reading the headers or not. The default is `False`, meaning it parses the entire contents of the file.

parsebytes(*bytes*, *headersonly=False*)

Similar to the `parse()` method, except it takes a *bytes-like object* instead of a file-like object. Calling this method on a *bytes-like object* is equivalent to wrapping *bytes* in a `BytesIO` instance first and calling `parse()`.

Optional *headersonly* is as with the `parse()` method.

Adicionado na versão 3.2.

class email.parser.**BytesHeaderParser**(*_class=None*, ***, *policy=policy.compat32*)

Exactly like `BytesParser`, except that *headersonly* defaults to `True`.

Adicionado na versão 3.3.

class email.parser.**Parser**(*_class=None*, ***, *policy=policy.compat32*)

This class is parallel to `BytesParser`, but handles string input.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

Alterado na versão 3.6: *_class* defaults to the policy `message_factory`.

parse(*fp*, *headersonly=False*)

Read all the data from the text-mode file-like object *fp*, parse the resulting text, and return the root message object. *fp* must support both the `readline()` and the `read()` methods on file-like objects.

Other than the text mode requirement, this method operates like `BytesParser.parse()`.

parsestr(*text*, *headersonly=False*)

Similar to the `parse()` method, except it takes a string object instead of a file-like object. Calling this method on a string is equivalent to wrapping *text* in a `StringIO` instance first and calling `parse()`.

Optional *headersonly* is as with the `parse()` method.

class email.parser.**HeaderParser**(*_class=None*, ***, *policy=policy.compat32*)

Exactly like `Parser`, except that *headersonly* defaults to `True`.

Since creating a message object structure from a string or a file object is such a common task, four functions are provided as a convenience. They are available in the top-level `email` package namespace.

email.**message_from_bytes**(*s*, *_class=None*, ***, *policy=policy.compat32*)

Return a message object structure from a *bytes-like object*. This is equivalent to `BytesParser().parsebytes(s)`. Optional *_class* and *policy* are interpreted as with the `BytesParser` class constructor.

Adicionado na versão 3.2.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

email.**message_from_binary_file**(*fp*, *_class=None*, ***, *policy=policy.compat32*)

Return a message object structure tree from an open binary *file object*. This is equivalent to `BytesParser().parse(fp)`. *_class* and *policy* are interpreted as with the `BytesParser` class constructor.

Adicionado na versão 3.2.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

email.**message_from_string**(*s*, *_class=None*, ***, *policy=policy.compat32*)

Return a message object structure from a string. This is equivalent to `Parser().parsestr(s)`. *_class* and *policy* are interpreted as with the `Parser` class constructor.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

email.**message_from_file**(*fp*, *_class=None*, ***, *policy=policy.compat32*)

Return a message object structure tree from an open *file object*. This is equivalent to `Parser().parse(fp)`. *_class* and *policy* are interpreted as with the `Parser` class constructor.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

Alterado na versão 3.6: *_class* defaults to the policy `message_factory`.

Here's an example of how you might use *message_from_bytes()* at an interactive Python prompt:

```
>>> import email
>>> msg = email.message_from_bytes(myBytes)
```

Additional notes

Here are some notes on the parsing semantics:

- Most non-*multipart* type messages are parsed as a single message object with a string payload. These objects will return `False` for *is_multipart()*, and *iter_parts()* will yield an empty list.
- All *multipart* type messages will be parsed as a container message object with a list of sub-message objects for their payload. The outer container message will return `True` for *is_multipart()*, and *iter_parts()* will yield a list of subparts.
- Most messages with a content type of *message/** (such as *message/delivery-status* and *message/rfc822*) will also be parsed as container object containing a list payload of length 1. Their *is_multipart()* method will return `True`. The single element yielded by *iter_parts()* will be a sub-message object.
- Some non-standards-compliant messages may not be internally consistent about their *multipart*-edness. Such messages may have a *Content-Type* header of type *multipart*, but their *is_multipart()* method may return `False`. If such messages were parsed with the *FeedParser*, they will have an instance of the *MultipartInvariantViolationDefect* class in their *defects* attribute list. See *email.errors* for details.

20.1.3 `email.generator`: Geração de documentos MIME

Código-fonte: Lib/email/generator.py

Uma das tarefas mais comuns é gerar a versão plana (serializada) da mensagem de e-mail representada por uma estrutura de objeto de mensagem. Você precisará fazer isso se quiser enviar sua mensagem via *smtplib.SMTP.sendmail()* ou exibir a mensagem no console. A tarefa das classes geradoras é pegar uma estrutura de objeto de mensagem e produzir uma representação serializada.

Assim como no módulo *email.parser*, você não está limitado à funcionalidade do gerador integrado; você mesmo pode criar um do zero. No entanto, o gerador integrado sabe como gerar a maioria dos e-mails em conformidade com os padrões, deve lidar perfeitamente com mensagens de e-mail MIME e não MIME e foi projetado para que as operações de análise e geração orientadas a bytes sejam inversas, presumindo que a mesma *policy* não transformadora seja usada para ambas. Ou seja, analisar o fluxo de bytes serializado por meio da classe *BytesParser* e, em seguida, regenerar o fluxo de bytes serializado usando *BytesGenerator* deve produzir uma saída idêntica à entrada[1]. (Por outro lado, usar o gerador em um *EmailMessage* construído pelo programa pode resultar em alterações no objeto *EmailMessage* à medida que os padrões são preenchidos.)

A classe *Generator* pode ser usada para simplificar uma mensagem em uma representação serializada de texto (em oposição a binária), mas como o Unicode não pode representar dados binários diretamente, a mensagem é necessariamente transformada em algo que contém apenas caracteres ASCII, usando as técnicas de codificação de transferência de conteúdo RFC de e-mail padrão para codificar mensagens de e-mail para transporte em canais que não são "limpos de 8 bits".

Para acomodar o processamento reproduzível de mensagens assinadas por SMIME, *Generator* desabilita a dobragem de cabeçalho para partes de mensagens do tipo `multipart/signed` e todas as subpartes.

class email.generator.**BytesGenerator**(*outfp*, *mangle_from_=None*, *maxheaderlen=None*, ***, *policy=None*)

Retorna um objeto *BytesGenerator* que gravará qualquer mensagem fornecida ao método *flatten()*, ou

[1] Esta instrução presume que você usa a configuração apropriada para `unixfrom` e que não haja configurações *email.policy* que exijam ajustes automáticos (por exemplo, *refold_source* deve ser `none`, que *não* é o padrão). Também não é 100% verdadeiro, pois, se a mensagem não estiver em conformidade com os padrões RFC, ocasionalmente informações sobre o texto original exato são perdidas durante a recuperação de erros de análise. O objetivo é corrigir esses últimos casos extremos sempre que possível.

qualquer texto codificado surrogateescape fornecido ao método *write()*, no *objeto arquivo ou similar* *outfp*. *outfp* deve oferecer suporte a um método `write` que aceite dados binários.

Se o *mangle_from_* opcional for `True`, coloca um caractere > na frente de qualquer linha no corpo que comece com a string exata `"From "`, ou seja, `From` seguido por um espaço no início de uma linha. *mangle_from_* assume por padrão o valor da configuração *mangle_from_* da *policy* (que é `True` para a política *compat32* e `False` para todas as outras). *mangle_from_* deve ser usado quando as mensagens são armazenadas no formato mbox do Unix (consulte *mailbox* e WHY THE CONTENT-LENGTH FORMAT IS BAD).

Se *maxheaderlen* não for `None`, redobra quaisquer linhas de cabeçalho maiores que *maxheaderlen* ou, se for `0`, não redobra nenhum cabeçalho. Se *manheaderlen* for `None` (o padrão), redobra os cabeçalhos e outras linhas de mensagem de acordo com as configurações da *policy*.

Se *policy* for especificado, usa essa política para controlar a geração de mensagens. Se *policy* for `None` (o padrão), usa a política associada ao objeto *Message* ou *EmailMessage* passado para `flatten` para controlar a geração de mensagens. Consulte *email.policy* para obter detalhes sobre o que *policy* controla.

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionada o argumento nomeado *policy*.

Alterado na versão 3.6: O comportamento padrão dos parâmetros *mangle_from_* e *maxheaderlen* é seguir a política.

flatten(*msg*, *unixfrom=False*, *linesep=None*)

Exibe a representação textual da estrutura do objeto de mensagem com raiz em *msg* no arquivo de saída especificado quando a instância *BytesGenerator* foi criada.

Se a opção *policy cte_type* for `8bit` (o padrão), copia todos os cabeçalhos da mensagem original analisada que não tenham sido modificados para a saída, com todos os bytes com o bit mais alto definido reproduzidos como no original, e preserva a *Content-Transfer-Encoding* não ASCII de quaisquer partes do corpo que os contenham. Se `cte_type` for `7bit`, converte os bytes com o bit mais alto definido, conforme necessário, usando uma *Content-Transfer-Encoding* compatível com ASCII. Ou seja, transforma partes com *Content-Transfer-Encoding* não ASCII (*Content-Transfer-Encoding: 8bit*) em um *Content-Transfer-Encoding* compatível com ASCII e codifica bytes não ASCII inválidos para RFC em cabeçalhos usando o conjunto de caracteres MIME `unknown-8bit`, tornando-os compatíveis com RFC.

Se *unixfrom* for `True`, exibe o delimitador de cabeçalho de envelope usado pelo formato de caixa de correio Unix (consulte *mailbox*) antes do primeiro dos cabeçalhos **RFC 5322** do objeto de mensagem raiz. Se o objeto raiz não tiver cabeçalho de envelope, crie um padrão. O padrão é `False`. Observe que, para subpartes, nenhum cabeçalho de envelope é exibido.

Se *linesep* não for `None`, usa-o como caractere separador entre todas as linhas da mensagem simplificada. Se *linesep* for `None` (o padrão), usa o valor especificado na *policy*.

clone(*fp*)

Retorna um clone independente desta instância *BytesGenerator* com exatamente as mesmas configurações de opção e *fp* como o novo *outfp*.

write(*s*)

Codifica *s* usando o codec `ASCII` e o tratador de erros `surrogateescape` e passa-o para o método *write* do *outfp* passado ao construtor *BytesGenerator*.

Para facilitar, *EmailMessage* fornece os métodos *as_bytes()* e `bytes(aMessage)` (também conhecidos como *__bytes__()*), que simplificam a geração de uma representação binária serializada de um objeto de mensagem. Para mais detalhes, consulte *email.message*.

Como strings não podem representar dados binários, a classe *Generator* deve converter quaisquer dados binários em qualquer mensagem que ela nivele para um formato compatível com ASCII, convertendo-os para um *Content-Transfer_Encoding* compatível com ASCII. Usando a terminologia dos RFCs de e-mail, você pode pensar nisso como *Generator* serializando para um fluxo de E/S que não é "limpo em 8 bits". Em outras palavras, a maioria das aplicações usará *BytesGenerator*, e não *Generator*.

class email.generator.**Generator**(*outfp*, *mangle_from_=None*, *maxheaderlen=None*, *, *policy=None*)

Retorna um objeto *Generator* que gravará qualquer mensagem fornecida ao método *flatten()*, ou qualquer texto fornecido ao método *write()*, no *objeto arquivo ou similar outfp*. *outfp* deve oferecer suporte a um método `write` que aceite dados string.

Se o *mangle_from_* opcional for `True`, coloca um caractere > na frente de qualquer linha no corpo que comece com a string exata `"From "`, ou seja, `From` seguido por um espaço no início de uma linha. *mangle_from_* assume por padrão o valor da configuração *mangle_from_* da *policy* (que é `True` para a política *compat32* e `False` para todas as outras). *mangle_from_* deve ser usado quando as mensagens são armazenadas no formato mbox do Unix (consulte *mailbox* e WHY THE CONTENT-LENGTH FORMAT IS BAD).

Se *maxheaderlen* não for `None`, redobra quaisquer linhas de cabeçalho maiores que *maxheaderlen* ou, se for `0`, não redobra nenhum cabeçalho. Se *manheaderlen* for `None` (o padrão), redobra os cabeçalhos e outras linhas de mensagem de acordo com as configurações da *policy*.

Se *policy* for especificado, usa essa política para controlar a geração de mensagens. Se *policy* for `None` (o padrão), usa a política associada ao objeto *Message* ou *EmailMessage* passado para `flatten` para controlar a geração de mensagens. Consulte *email.policy* para obter detalhes sobre o que *policy* controla.

Alterado na versão 3.3: Adicionada o argumento nomeado *policy*.

Alterado na versão 3.6: O comportamento padrão dos parâmetros *mangle_from_* e *maxheaderlen* é seguir a política.

flatten(*msg*, *unixfrom=False*, *linesep=None*)

Exibe a representação textual da estrutura do objeto de mensagem com raiz em *msg* no arquivo de saída especificado quando a instância *Generator* foi criada.

Se a opção *policy cte_type* for `8bit`, gera a mensagem como se a opção estivesse definida como `7bit`. (Isso é necessário porque strings não podem representar bytes não ASCII.) Converte quaisquer bytes com o bit mais alto definido, conforme necessário, usando uma *Content-Transfer-Encoding* compatível com ASCII. Ou seja, transforma partes com *Content-Transfer-Encoding* não ASCII (*Content-Transfer-Encoding: 8bit*) em um *Content-Transfer-Encoding* compatível com ASCII e codifica bytes não ASCII inválidos para RFC em cabeçalhos usando o conjunto de caracteres MIME `unknown-8bit`, tornando-os compatíveis com RFC.

Se *unixfrom* for `True`, exibe o delimitador de cabeçalho de envelope usado pelo formato de caixa de correio Unix (consulte *mailbox*) antes do primeiro dos cabeçalhos **RFC 5322** do objeto de mensagem raiz. Se o objeto raiz não tiver cabeçalho de envelope, crie um padrão. O padrão é `False`. Observe que, para subpartes, nenhum cabeçalho de envelope é exibido.

Se *linesep* não for `None`, usa-o como caractere separador entre todas as linhas da mensagem simplificada. Se *linesep* for `None` (o padrão), usa o valor especificado na *policy*.

Alterado na versão 3.2: Adicionado suporte para recodificação de corpos de mensagens `8bit` e o argumento *linesep*.

clone(*fp*)

Retorna um clone independente desta instância *Generator* com exatamente as mesmas opções e *fp* como o novo *outfp*.

write(*s*)

Escreve *s* no método *write* do *outfp* passado ao construtor de *Generator*. Isso fornece API arquivo ou similar suficiente para que instâncias de *Generator* sejam usadas na função *print()*.

Para facilitar, *EmailMessage* fornece os métodos *as_string()* e `str(aMessage)` (também conhecidos como *__str__()*), que simplificam a geração de uma representação de string formatada de um objeto de mensagem. Para mais detalhes, consulte *email.message*.

O módulo *email.generator* também fornece uma classe derivada, *DecodedGenerator*, que é como a classe base *Generator*, exceto que as partes não-*text* não são serializadas, mas são representadas no fluxo de saída por uma string derivada de um modelo preenchido com informações sobre a parte.

class email.generator.**DecodedGenerator**(*outfp*, *mangle_from_=None*, *maxheaderlen=None*, *fmt=None*, ***, *policy=None*)

Age como *Generator*, exceto que para qualquer subparte da mensagem passada para *Generator.flatten()*, se a subparte for do tipo principal *text*, exibe a carga decodificada da subparte e, se o tipo principal não for *text*, em vez de exibi-la, preenche a string *fmt* usando informações da parte e exibe a string preenchida resultante.

Para preencher *fmt*, executa `fmt % part_info`, onde `part_info` é um dicionário composto pelas seguintes chaves e valores:

- `type` – Tipo MIME completo da parte não-*text*
- `maintype` – Tipo MIME principal da parte não-*text*
- `subtype` – Tipo sub-MIME da parte não-*text*
- `filename` – Nome de arquivo da parte não-*text*
- `description` – Descrição associada à parte não-*text*
- `encoding` – Codificação de transferência de conteúdo da parte não-*text*

Se *fmt* for `None`, usa o seguinte *fmt* padrão:

"[Non-text (%(type)s) part of message omitted, filename %(filename)s]"

Os opcionais *_mangle_from_* e *maxheaderlen* são como os da classe base *Generator*.

20.1.4 email.policy: Policy Objects

Adicionado na versão 3.3.

Código-fonte: Lib/email/policy.py

The *email* package's prime focus is the handling of email messages as described by the various email and MIME RFCs. However, the general format of email messages (a block of header fields each consisting of a name followed by a colon followed by a value, the whole block followed by a blank line and an arbitrary 'body'), is a format that has found utility outside of the realm of email. Some of these uses conform fairly closely to the main email RFCs, some do not. Even when working with email, there are times when it is desirable to break strict compliance with the RFCs, such as generating emails that interoperate with email servers that do not themselves follow the standards, or that implement extensions you want to use in ways that violate the standards.

Policy objects give the email package the flexibility to handle all these disparate use cases.

A *Policy* object encapsulates a set of attributes and methods that control the behavior of various components of the email package during use. *Policy* instances can be passed to various classes and methods in the email package to alter the default behavior. The settable values and their defaults are described below.

There is a default policy used by all classes in the email package. For all of the *parser* classes and the related convenience functions, and for the *Message* class, this is the *Compat32* policy, via its corresponding pre-defined instance *compat32*. This policy provides for complete backward compatibility (in some cases, including bug compatibility) with the pre-Python3.3 version of the email package.

This default value for the *policy* keyword to *EmailMessage* is the *EmailPolicy* policy, via its pre-defined instance *default*.

When a *Message* or *EmailMessage* object is created, it acquires a policy. If the message is created by a *parser*, a policy passed to the parser will be the policy used by the message it creates. If the message is created by the program, then the policy can be specified when it is created. When a message is passed to a *generator*, the generator uses the policy from the message by default, but you can also pass a specific policy to the generator that will override the one stored on the message object.

The default value for the *policy* keyword for the *email.parser* classes and the parser convenience functions **will be changing** in a future version of Python. Therefore you should **always specify explicitly which policy you want to use** when calling any of the classes and functions described in the *parser* module.

The first part of this documentation covers the features of *Policy*, an *abstract base class* that defines the features that are common to all policy objects, including *compat32*. This includes certain hook methods that are called internally by the email package, which a custom policy could override to obtain different behavior. The second part describes the concrete classes *EmailPolicy* and *Compat32*, which implement the hooks that provide the standard behavior and the backward compatible behavior and features, respectively.

Policy instances are immutable, but they can be cloned, accepting the same keyword arguments as the class constructor and returning a new *Policy* instance that is a copy of the original but with the specified attributes values changed.

As an example, the following code could be used to read an email message from a file on disk and pass it to the system `sendmail` program on a Unix system:

```
>>> from email import message_from_binary_file
>>> from email.generator import BytesGenerator
>>> from email import policy
>>> from subprocess import Popen, PIPE
>>> with open('mymsg.txt', 'rb') as f:
...     msg = message_from_binary_file(f, policy=policy.default)
...
>>> p = Popen(['sendmail', msg['To'].addresses[0]], stdin=PIPE)
>>> g = BytesGenerator(p.stdin, policy=msg.policy.clone(linesep='\r\n'))
>>> g.flatten(msg)
>>> p.stdin.close()
>>> rc = p.wait()
```

Here we are telling *BytesGenerator* to use the RFC correct line separator characters when creating the binary string to feed into `sendmail's stdin`, where the default policy would use `\n` line separators.

Some email package methods accept a *policy* keyword argument, allowing the policy to be overridden for that method. For example, the following code uses the *as_bytes()* method of the *msg* object from the previous example and writes the message to a file using the native line separators for the platform on which it is running:

```
>>> import os
>>> with open('converted.txt', 'wb') as f:
...     f.write(msg.as_bytes(policy=msg.policy.clone(linesep=os.linesep)))
17
```

Policy objects can also be combined using the addition operator, producing a policy object whose settings are a combination of the non-default values of the summed objects:

```
>>> compat_SMTP = policy.compat32.clone(linesep='\r\n')
>>> compat_strict = policy.compat32.clone(raise_on_defect=True)
>>> compat_strict_SMTP = compat_SMTP + compat_strict
```

This operation is not commutative; that is, the order in which the objects are added matters. To illustrate:

```
>>> policy100 = policy.compat32.clone(max_line_length=100)
>>> policy80 = policy.compat32.clone(max_line_length=80)
>>> apolicy = policy100 + policy80
>>> apolicy.max_line_length
80
>>> apolicy = policy80 + policy100
>>> apolicy.max_line_length
100
```

class email.policy.**Policy**(***kw*)

This is the *abstract base class* for all policy classes. It provides default implementations for a couple of trivial methods, as well as the implementation of the immutability property, the *clone()* method, and the constructor semantics.

The constructor of a policy class can be passed various keyword arguments. The arguments that may be specified are any non-method properties on this class, plus any additional non-method properties on the concrete class. A value specified in the constructor will override the default value for the corresponding attribute.

This class defines the following properties, and thus values for the following may be passed in the constructor of any policy class:

max_line_length

The maximum length of any line in the serialized output, not counting the end of line character(s). Default is 78, per **RFC 5322**. A value of `0` or *None* indicates that no line wrapping should be done at all.

linesep

The string to be used to terminate lines in serialized output. The default is `\n` because that's the internal end-of-line discipline used by Python, though `\r\n` is required by the RFCs.

cte_type

Controls the type of Content Transfer Encodings that may be or are required to be used. The possible values are:

`7bit`	all data must be "7 bit clean" (ASCII-only). This means that where necessary data will be encoded using either quoted-printable or base64 encoding.
`8bit`	data is not constrained to be 7 bit clean. Data in headers is still required to be ASCII-only and so will be encoded (see *fold_binary()* and *utf8* below for exceptions), but body parts may use the `8bit` CTE.

A `cte_type` value of `8bit` only works with `BytesGenerator`, not `Generator`, because strings cannot contain binary data. If a `Generator` is operating under a policy that specifies `cte_type=8bit`, it will act as if `cte_type` is `7bit`.

raise_on_defect

If *True*, any defects encountered will be raised as errors. If *False* (the default), defects will be passed to the *register_defect()* method.

mangle_from_

If *True*, lines starting with *"From "* in the body are escaped by putting a > in front of them. This parameter is used when the message is being serialized by a generator. Default: *False*.

Adicionado na versão 3.5.

message_factory

A factory function for constructing a new empty message object. Used by the parser when building messages. Defaults to `None`, in which case *Message* is used.

Adicionado na versão 3.6.

verify_generated_headers

If `True` (the default), the generator will raise *HeaderWriteError* instead of writing a header that is improperly folded or delimited, such that it would be parsed as multiple headers or joined with adjacent data. Such headers can be generated by custom header classes or bugs in the `email` module.

As it's a security feature, this defaults to `True` even in the *Compat32* policy. For backwards compatible, but unsafe, behavior, it must be set to `False` explicitly.

Adicionado na versão 3.13.

The following *Policy* method is intended to be called by code using the email library to create policy instances with custom settings:

clone(***kw*)

Return a new *Policy* instance whose attributes have the same values as the current instance, except where those attributes are given new values by the keyword arguments.

The remaining *Policy* methods are called by the email package code, and are not intended to be called by an application using the email package. A custom policy must implement all of these methods.

handle_defect(*obj*, *defect*)

Handle a *defect* found on *obj*. When the email package calls this method, *defect* will always be a subclass of *MessageDefect*.

The default implementation checks the *raise_on_defect* flag. If it is `True`, *defect* is raised as an exception. If it is `False` (the default), *obj* and *defect* are passed to *register_defect()*.

register_defect(*obj*, *defect*)

Register a *defect* on *obj*. In the email package, *defect* will always be a subclass of *MessageDefect*.

The default implementation calls the `append` method of the `defects` attribute of *obj*. When the email package calls *handle_defect*, *obj* will normally have a `defects` attribute that has an `append` method. Custom object types used with the email package (for example, custom `Message` objects) should also provide such an attribute, otherwise defects in parsed messages will raise unexpected errors.

header_max_count(*name*)

Return the maximum allowed number of headers named *name*.

Called when a header is added to an *EmailMessage* or *Message* object. If the returned value is not `0` or `None`, and there are already a number of headers with the name *name* greater than or equal to the value returned, a *ValueError* is raised.

Because the default behavior of `Message.__setitem__` is to append the value to the list of headers, it is easy to create duplicate headers without realizing it. This method allows certain headers to be limited in the number of instances of that header that may be added to a `Message` programmatically. (The limit is not observed by the parser, which will faithfully produce as many headers as exist in the message being parsed.)

The default implementation returns `None` for all header names.

header_source_parse(*sourcelines*)

The email package calls this method with a list of strings, each string ending with the line separation characters found in the source being parsed. The first line includes the field header name and separator. All whitespace in the source is preserved. The method should return the `(name, value)` tuple that is to be stored in the `Message` to represent the parsed header.

If an implementation wishes to retain compatibility with the existing email package policies, *name* should be the case preserved name (all characters up to the ':' separator), while *value* should be the unfolded value (all line separator characters removed, but whitespace kept intact), stripped of leading whitespace.

sourcelines may contain surrogateescaped binary data.

There is no default implementation

header_store_parse(*name*, *value*)

The email package calls this method with the name and value provided by the application program when the application program is modifying a `Message` programmatically (as opposed to a `Message` created by a parser). The method should return the `(name, value)` tuple that is to be stored in the `Message` to represent the header.

If an implementation wishes to retain compatibility with the existing email package policies, the *name* and *value* should be strings or string subclasses that do not change the content of the passed in arguments.

There is no default implementation

header_fetch_parse(*name*, *value*)

The email package calls this method with the *name* and *value* currently stored in the `Message` when that header is requested by the application program, and whatever the method returns is what is passed back to the application as the value of the header being retrieved. Note that there may be more than one header with the same name stored in the `Message`; the method is passed the specific name and value of the header destined to be returned to the application.

value may contain surrogateescaped binary data. There should be no surrogateescaped binary data in the value returned by the method.

There is no default implementation

fold(*name*, *value*)

The email package calls this method with the *name* and *value* currently stored in the `Message` for a given header. The method should return a string that represents that header "folded" correctly (according to the policy settings) by composing the *name* with the *value* and inserting *linesep* characters at the appropriate places. See **RFC 5322** for a discussion of the rules for folding email headers.

value may contain surrogateescaped binary data. There should be no surrogateescaped binary data in the string returned by the method.

fold_binary(*name*, *value*)

The same as *fold()*, except that the returned value should be a bytes object rather than a string.

value may contain surrogateescaped binary data. These could be converted back into binary data in the returned bytes object.

class email.policy.**EmailPolicy**(***kw*)

This concrete *Policy* provides behavior that is intended to be fully compliant with the current email RFCs. These include (but are not limited to) **RFC 5322**, **RFC 2047**, and the current MIME RFCs.

This policy adds new header parsing and folding algorithms. Instead of simple strings, headers are `str` subclasses with attributes that depend on the type of the field. The parsing and folding algorithm fully implement **RFC 2047** and **RFC 5322**.

The default value for the *message_factory* attribute is *EmailMessage*.

In addition to the settable attributes listed above that apply to all policies, this policy adds the following additional attributes:

Adicionado na versão 3.6:[1]

utf8

If `False`, follow **RFC 5322**, supporting non-ASCII characters in headers by encoding them as "encoded words". If `True`, follow **RFC 6532** and use `utf-8` encoding for headers. Messages formatted in this way may be passed to SMTP servers that support the `SMTPUTF8` extension (**RFC 6531**).

refold_source

If the value for a header in the `Message` object originated from a *parser* (as opposed to being set by a program), this attribute indicates whether or not a generator should refold that value when transforming the message back into serialized form. The possible values are:

`none`	all source values use original folding
`long`	source values that have any line that is longer than `max_line_length` will be refolded
`all`	todos os valores são redobrados.

O padrão é `long`.

header_factory

A callable that takes two arguments, `name` and `value`, where `name` is a header field name and `value` is an unfolded header field value, and returns a string subclass that represents that header. A default `header_factory` (see *headerregistry*) is provided that supports custom parsing for the various address and date **RFC 5322** header field types, and the major MIME header field stypes. Support for additional custom parsing will be added in the future.

[1] Originally added in 3.3 as a *provisional feature*.

content_manager

An object with at least two methods: get_content and set_content. When the *get_content()* or *set_content()* method of an *EmailMessage* object is called, it calls the corresponding method of this object, passing it the message object as its first argument, and any arguments or keywords that were passed to it as additional arguments. By default `content_manager` is set to *raw_data_manager*.

Adicionado na versão 3.4.

The class provides the following concrete implementations of the abstract methods of *Policy*:

header_max_count(*name*)

Returns the value of the *max_count* attribute of the specialized class used to represent the header with the given name.

header_source_parse(*sourcelines*)

The name is parsed as everything up to the ':' and returned unmodified. The value is determined by stripping leading whitespace off the remainder of the first line, joining all subsequent lines together, and stripping any trailing carriage return or linefeed characters.

header_store_parse(*name*, *value*)

The name is returned unchanged. If the input value has a `name` attribute and it matches *name* ignoring case, the value is returned unchanged. Otherwise the *name* and *value* are passed to `header_factory`, and the resulting header object is returned as the value. In this case a `ValueError` is raised if the input value contains CR or LF characters.

header_fetch_parse(*name*, *value*)

If the value has a `name` attribute, it is returned to unmodified. Otherwise the *name*, and the *value* with any CR or LF characters removed, are passed to the `header_factory`, and the resulting header object is returned. Any surrogateescaped bytes get turned into the unicode unknown-character glyph.

fold(*name*, *value*)

Header folding is controlled by the *refold_source* policy setting. A value is considered to be a 'source value' if and only if it does not have a `name` attribute (having a `name` attribute means it is a header object of some sort). If a source value needs to be refolded according to the policy, it is converted into a header object by passing the *name* and the *value* with any CR and LF characters removed to the `header_factory`. Folding of a header object is done by calling its `fold` method with the current policy.

Source values are split into lines using *splitlines()*. If the value is not to be refolded, the lines are rejoined using the `linesep` from the policy and returned. The exception is lines containing non-ascii binary data. In that case the value is refolded regardless of the `refold_source` setting, which causes the binary data to be CTE encoded using the `unknown-8bit` charset.

fold_binary(*name*, *value*)

The same as *fold()* if *cte_type* is `7bit`, except that the returned value is bytes.

If *cte_type* is `8bit`, non-ASCII binary data is converted back into bytes. Headers with binary data are not refolded, regardless of the `refold_header` setting, since there is no way to know whether the binary data consists of single byte characters or multibyte characters.

The following instances of *EmailPolicy* provide defaults suitable for specific application domains. Note that in the future the behavior of these instances (in particular the `HTTP` instance) may be adjusted to conform even more closely to the RFCs relevant to their domains.

email.policy.**default**

An instance of `EmailPolicy` with all defaults unchanged. This policy uses the standard Python `\n` line endings rather than the RFC-correct `\r\n`.

email.policy.**SMTP**

Suitable for serializing messages in conformance with the email RFCs. Like `default`, but with `linesep` set to `\r\n`, which is RFC compliant.

email.policy.**SMTPUTF8**

The same as SMTP except that *utf8* is True. Useful for serializing messages to a message store without using encoded words in the headers. Should only be used for SMTP transmission if the sender or recipient addresses have non-ASCII characters (the *smtplib.SMTP.send_message()* method handles this automatically).

email.policy.**HTTP**

Suitable for serializing headers with for use in HTTP traffic. Like SMTP except that max_line_length is set to None (unlimited).

email.policy.**strict**

Convenience instance. The same as default except that raise_on_defect is set to True. This allows any policy to be made strict by writing:

```
somepolicy + policy.strict
```

With all of these *EmailPolicies*, the effective API of the email package is changed from the Python 3.2 API in the following ways:

- Setting a header on a *Message* results in that header being parsed and a header object created.
- Fetching a header value from a *Message* results in that header being parsed and a header object created and returned.
- Any header object, or any header that is refolded due to the policy settings, is folded using an algorithm that fully implements the RFC folding algorithms, including knowing where encoded words are required and allowed.

From the application view, this means that any header obtained through the *EmailMessage* is a header object with extra attributes, whose string value is the fully decoded unicode value of the header. Likewise, a header may be assigned a new value, or a new header created, using a unicode string, and the policy will take care of converting the unicode string into the correct RFC encoded form.

The header objects and their attributes are described in *headerregistry*.

class email.policy.**Compat32**(***kw*)

This concrete *Policy* is the backward compatibility policy. It replicates the behavior of the email package in Python 3.2. The *policy* module also defines an instance of this class, *compat32*, that is used as the default policy. Thus the default behavior of the email package is to maintain compatibility with Python 3.2.

The following attributes have values that are different from the *Policy* default:

mangle_from_

O padrão é True.

The class provides the following concrete implementations of the abstract methods of *Policy*:

header_source_parse(*sourcelines*)

The name is parsed as everything up to the ':' and returned unmodified. The value is determined by stripping leading whitespace off the remainder of the first line, joining all subsequent lines together, and stripping any trailing carriage return or linefeed characters.

header_store_parse(*name*, *value*)

The name and value are returned unmodified.

header_fetch_parse(*name*, *value*)

If the value contains binary data, it is converted into a *Header* object using the unknown-8bit charset. Otherwise it is returned unmodified.

fold(*name*, *value*)

Headers are folded using the *Header* folding algorithm, which preserves existing line breaks in the value, and wraps each resulting line to the max_line_length. Non-ASCII binary data are CTE encoded using the unknown-8bit charset.

fold_binary(*name*, *value*)

Headers are folded using the *Header* folding algorithm, which preserves existing line breaks in the value, and wraps each resulting line to the `max_line_length`. If `cte_type` is `7bit`, non-ascii binary data is CTE encoded using the `unknown-8bit` charset. Otherwise the original source header is used, with its existing line breaks and any (RFC invalid) binary data it may contain.

`email.policy.`**compat32**

An instance of *Compat32*, providing backward compatibility with the behavior of the email package in Python 3.2.

20.1.5 `email.errors`: Classes de Exceção e Defeito.

Código-fonte: Lib/email/errors.py

A seguinte classe de exceção é definida no modulo *email.errors*.

exception `email.errors.`**MessageError**

Essa é a classe base para todas as exceções que o pacote *email* pode levantar. Ela é derivada da classe padrão *Exception* e não define métodos adicionais.

exception `email.errors.`**MessageParseError**

Essa é a classe base para exceções levantadas pela classe *Parser*. Ela é derivada do *MessageError*. Essa classe pode ser usada internamente pelo analisador sintático usado pelo *headerregistry*.

exception `email.errors.`**HeaderParseError**

Raised under some error conditions when parsing the **RFC 5322** headers of a message, this class is derived from *MessageParseError*. The *set_boundary()* method will raise this error if the content type is unknown when the method is called. *Header* may raise this error for certain base64 decoding errors, and when an attempt is made to create a header that appears to contain an embedded header (that is, there is what is supposed to be a continuation line that has no leading whitespace and looks like a header).

exception `email.errors.`**BoundaryError**

Descontinuado e não mais usado.

exception `email.errors.`**MultipartConversionError**

Raised if the *attach()* method is called on an instance of a class derived from *MIMENonMultipart* (e.g. *MIMEImage*). *MultipartConversionError* multiply inherits from *MessageError* and the built-in *TypeError*.

exception `email.errors.`**HeaderWriteError**

Raised when an error occurs when the *generator* outputs headers.

exception `email.errors.`**MessageDefect**

This is the base class for all defects found when parsing email messages. It is derived from *ValueError*.

exception `email.errors.`**HeaderDefect**

This is the base class for all defects found when parsing email headers. It is derived from *MessageDefect*.

Here is the list of the defects that the *FeedParser* can find while parsing messages. Note that the defects are added to the message where the problem was found, so for example, if a message nested inside a *multipart/alternative* had a malformed header, that nested message object would have a defect, but the containing messages would not.

All defect classes are subclassed from *email.errors.MessageDefect*.

exception `email.errors.`**NoBoundaryInMultipartDefect**

A message claimed to be a multipart, but had no *boundary* parameter.

exception `email.errors.`**StartBoundaryNotFoundDefect**

The start boundary claimed in the *Content-Type* header was never found.

exception email.errors.**CloseBoundaryNotFoundDefect**

A start boundary was found, but no corresponding close boundary was ever found.

Adicionado na versão 3.3.

exception email.errors.**FirstHeaderLineIsContinuationDefect**

The message had a continuation line as its first header line.

exception email.errors.**MisplacedEnvelopeHeaderDefect**

A "Unix From" header was found in the middle of a header block.

exception email.errors.**MissingHeaderBodySeparatorDefect**

A line was found while parsing headers that had no leading white space but contained no ':'. Parsing continues assuming that the line represents the first line of the body.

Adicionado na versão 3.3.

exception email.errors.**MalformedHeaderDefect**

A header was found that was missing a colon, or was otherwise malformed.

Descontinuado desde a versão 3.3: This defect has not been used for several Python versions.

exception email.errors.**MultipartInvariantViolationDefect**

A message claimed to be a *multipart*, but no subparts were found. Note that when a message has this defect, its *is_multipart()* method may return False even though its content type claims to be *multipart*.

exception email.errors.**InvalidBase64PaddingDefect**

When decoding a block of base64 encoded bytes, the padding was not correct. Enough padding is added to perform the decode, but the resulting decoded bytes may be invalid.

exception email.errors.**InvalidBase64CharactersDefect**

When decoding a block of base64 encoded bytes, characters outside the base64 alphabet were encountered. The characters are ignored, but the resulting decoded bytes may be invalid.

exception email.errors.**InvalidBase64LengthDefect**

When decoding a block of base64 encoded bytes, the number of non-padding base64 characters was invalid (1 more than a multiple of 4). The encoded block was kept as-is.

exception email.errors.**InvalidDateDefect**

When decoding an invalid or unparsable date field. The original value is kept as-is.

20.1.6 email.headerregistry: Custom Header Objects

Código-fonte: Lib/email/headerregistry.py

Adicionado na versão 3.6:[1]

Headers are represented by customized subclasses of *str*. The particular class used to represent a given header is determined by the *header_factory* of the *policy* in effect when the headers are created. This section documents the particular header_factory implemented by the email package for handling **RFC 5322** compliant email messages, which not only provides customized header objects for various header types, but also provides an extension mechanism for applications to add their own custom header types.

When using any of the policy objects derived from *EmailPolicy*, all headers are produced by *HeaderRegistry* and have *BaseHeader* as their last base class. Each header class has an additional base class that is determined by the type of the header. For example, many headers have the class *UnstructuredHeader* as their other base class. The specialized second class for a header is determined by the name of the header, using a lookup table stored in the *HeaderRegistry*. All of this is managed transparently for the typical application program, but interfaces are provided for modifying the default behavior for use by more complex applications.

[1] Originally added in 3.3 as a *provisional module*

The sections below first document the header base classes and their attributes, followed by the API for modifying the behavior of *HeaderRegistry*, and finally the support classes used to represent the data parsed from structured headers.

class email.headerregistry.**BaseHeader**(*name*, *value*)

name and *value* are passed to BaseHeader from the *header_factory* call. The string value of any header object is the *value* fully decoded to unicode.

This base class defines the following read-only properties:

name

The name of the header (the portion of the field before the ':'). This is exactly the value passed in the *header_factory* call for *name*; that is, case is preserved.

defects

A tuple of *HeaderDefect* instances reporting any RFC compliance problems found during parsing. The email package tries to be complete about detecting compliance issues. See the *errors* module for a discussion of the types of defects that may be reported.

max_count

The maximum number of headers of this type that can have the same name. A value of None means unlimited. The BaseHeader value for this attribute is None; it is expected that specialized header classes will override this value as needed.

BaseHeader also provides the following method, which is called by the email library code and should not in general be called by application programs:

fold(*, *policy*)

Return a string containing *linesep* characters as required to correctly fold the header according to *policy*. A *cte_type* of 8bit will be treated as if it were 7bit, since headers may not contain arbitrary binary data. If *utf8* is False, non-ASCII data will be **RFC 2047** encoded.

BaseHeader by itself cannot be used to create a header object. It defines a protocol that each specialized header cooperates with in order to produce the header object. Specifically, BaseHeader requires that the specialized class provide a *classmethod()* named parse. This method is called as follows:

```
parse(string, kwds)
```

kwds is a dictionary containing one pre-initialized key, defects. defects is an empty list. The parse method should append any detected defects to this list. On return, the kwds dictionary *must* contain values for at least the keys decoded and defects. decoded should be the string value for the header (that is, the header value fully decoded to unicode). The parse method should assume that *string* may contain content-transfer-encoded parts, but should correctly handle all valid unicode characters as well so that it can parse un-encoded header values.

BaseHeader's __new__ then creates the header instance, and calls its init method. The specialized class only needs to provide an init method if it wishes to set additional attributes beyond those provided by BaseHeader itself. Such an init method should look like this:

```
def init(self, /, *args, **kw):
    self._myattr = kw.pop('myattr')
    super().init(*args, **kw)
```

That is, anything extra that the specialized class puts in to the kwds dictionary should be removed and handled, and the remaining contents of kw (and args) passed to the BaseHeader init method.

class email.headerregistry.**UnstructuredHeader**

An "unstructured" header is the default type of header in **RFC 5322**. Any header that does not have a specified syntax is treated as unstructured. The classic example of an unstructured header is the *Subject* header.

In **RFC 5322**, an unstructured header is a run of arbitrary text in the ASCII character set. **RFC 2047**, however, has an **RFC 5322** compatible mechanism for encoding non-ASCII text as ASCII characters within a header value. When a *value* containing encoded words is passed to the constructor, the UnstructuredHeader parser

converts such encoded words into unicode, following the **RFC 2047** rules for unstructured text. The parser uses heuristics to attempt to decode certain non-compliant encoded words. Defects are registered in such cases, as well as defects for issues such as invalid characters within the encoded words or the non-encoded text.

This header type provides no additional attributes.

class email.headerregistry.**DateHeader**

RFC 5322 specifies a very specific format for dates within email headers. The `DateHeader` parser recognizes that date format, as well as recognizing a number of variant forms that are sometimes found "in the wild".

This header type provides the following additional attributes:

datetime

If the header value can be recognized as a valid date of one form or another, this attribute will contain a *datetime* instance representing that date. If the timezone of the input date is specified as `-0000` (indicating it is in UTC but contains no information about the source timezone), then *datetime* will be a naive *datetime*. If a specific timezone offset is found (including `+0000`), then *datetime* will contain an aware `datetime` that uses *datetime.timezone* to record the timezone offset.

The `decoded` value of the header is determined by formatting the `datetime` according to the **RFC 5322** rules; that is, it is set to:

```
email.utils.format_datetime(self.datetime)
```

When creating a `DateHeader`, *value* may be *datetime* instance. This means, for example, that the following code is valid and does what one would expect:

```
msg['Date'] = datetime(2011, 7, 15, 21)
```

Because this is a naive `datetime` it will be interpreted as a UTC timestamp, and the resulting value will have a timezone of `-0000`. Much more useful is to use the *localtime()* function from the *utils* module:

```
msg['Date'] = utils.localtime()
```

This example sets the date header to the current time and date using the current timezone offset.

class email.headerregistry.**AddressHeader**

Address headers are one of the most complex structured header types. The `AddressHeader` class provides a generic interface to any address header.

This header type provides the following additional attributes:

groups

A tuple of *Group* objects encoding the addresses and groups found in the header value. Addresses that are not part of a group are represented in this list as single-address `Groups` whose *display_name* is `None`.

addresses

A tuple of *Address* objects encoding all of the individual addresses from the header value. If the header value contains any groups, the individual addresses from the group are included in the list at the point where the group occurs in the value (that is, the list of addresses is "flattened" into a one dimensional list).

The `decoded` value of the header will have all encoded words decoded to unicode. *idna* encoded domain names are also decoded to unicode. The `decoded` value is set by *joining* the *str* value of the elements of the `groups` attribute with `', '`.

A list of *Address* and *Group* objects in any combination may be used to set the value of an address header. `Group` objects whose `display_name` is `None` will be interpreted as single addresses, which allows an address list to be copied with groups intact by using the list obtained from the `groups` attribute of the source header.

class email.headerregistry.**SingleAddressHeader**

Uma subclasse de *AddressHeader* que adiciona um atributo adicional:

address

The single address encoded by the header value. If the header value actually contains more than one address (which would be a violation of the RFC under the default *policy*), accessing this attribute will result in a *ValueError*.

Many of the above classes also have a `Unique` variant (for example, `UniqueUnstructuredHeader`). The only difference is that in the `Unique` variant, *max_count* is set to 1.

class email.headerregistry.**MIMEVersionHeader**

There is really only one valid value for the *MIME-Version* header, and that is `1.0`. For future proofing, this header class supports other valid version numbers. If a version number has a valid value per **RFC 2045**, then the header object will have non-`None` values for the following attributes:

version

The version number as a string, with any whitespace and/or comments removed.

major

The major version number as an integer

minor

The minor version number as an integer

class email.headerregistry.**ParameterizedMIMEHeader**

MIME headers all start with the prefix 'Content-'. Each specific header has a certain value, described under the class for that header. Some can also take a list of supplemental parameters, which have a common format. This class serves as a base for all the MIME headers that take parameters.

params

A dictionary mapping parameter names to parameter values.

class email.headerregistry.**ContentTypeHeader**

A *ParameterizedMIMEHeader* class that handles the *Content-Type* header.

content_type

The content type string, in the form `maintype/subtype`.

maintype

subtype

class email.headerregistry.**ContentDispositionHeader**

Uma classe *ParameterizedMIMEHeader* que lida com o cabeçalho *Content-Disposition*.

content_disposition

`inline` and `attachment` are the only valid values in common use.

class email.headerregistry.**ContentTransferEncoding**

Handles the *Content-Transfer-Encoding* header.

cte

Valid values are `7bit`, `8bit`, `base64`, and `quoted-printable`. See **RFC 2045** for more information.

class email.headerregistry.**HeaderRegistry**(*base_class=BaseHeader*, *default_class=UnstructuredHeader*, *use_default_map=True*)

This is the factory used by *EmailPolicy* by default. `HeaderRegistry` builds the class used to create a header instance dynamically, using *base_class* and a specialized class retrieved from a registry that it holds. When a given header name does not appear in the registry, the class specified by *default_class* is used as the specialized class. When *use_default_map* is `True` (the default), the standard mapping of header names to classes is copied in to the registry during initialization. *base_class* is always the last class in the generated class's `__bases__` list.

The default mappings are:

subject
UniqueUnstructuredHeader

date
UniqueDateHeader

resent-date
DateHeader

orig-date
UniqueDateHeader

sender
UniqueSingleAddressHeader

resent-sender
SingleAddressHeader

to
UniqueAddressHeader

resent-to
AddressHeader

cc
UniqueAddressHeader

resent-cc
AddressHeader

bcc
UniqueAddressHeader

resent-bcc
AddressHeader

from
UniqueAddressHeader

resent-from
AddressHeader

reply-to
UniqueAddressHeader

mime-version
MIMEVersionHeader

content-type
ContentTypeHeader

content-disposition
ContentDispositionHeader

content-transfer-encoding
ContentTransferEncodingHeader

message-id
MessageIDHeader

`HeaderRegistry` has the following methods:

`map_to_type`(*self*, *name*, *cls*)

name is the name of the header to be mapped. It will be converted to lower case in the registry. *cls* is the specialized class to be used, along with *base_class*, to create the class used to instantiate headers that match *name*.

__getitem__(*name*)

Construct and return a class to handle creating a *name* header.

__call__(*name*, *value*)

Retrieves the specialized header associated with *name* from the registry (using *default_class* if *name* does not appear in the registry) and composes it with *base_class* to produce a class, calls the constructed class's constructor, passing it the same argument list, and finally returns the class instance created thereby.

The following classes are the classes used to represent data parsed from structured headers and can, in general, be used by an application program to construct structured values to assign to specific headers.

class email.headerregistry.**Address**(*display_name=''*, *username=''*, *domain=''*, *addr_spec=None*)

The class used to represent an email address. The general form of an address is:

```
[display_name] <username@domain>
```

ou:

```
username@domain
```

where each part must conform to specific syntax rules spelled out in **RFC 5322**.

As a convenience *addr_spec* can be specified instead of *username* and *domain*, in which case *username* and *domain* will be parsed from the *addr_spec*. An *addr_spec* must be a properly RFC quoted string; if it is not `Address` will raise an error. Unicode characters are allowed and will be property encoded when serialized. However, per the RFCs, unicode is *not* allowed in the username portion of the address.

display_name

The display name portion of the address, if any, with all quoting removed. If the address does not have a display name, this attribute will be an empty string.

username

The `username` portion of the address, with all quoting removed.

domain

The `domain` portion of the address.

addr_spec

The `username@domain` portion of the address, correctly quoted for use as a bare address (the second form shown above). This attribute is not mutable.

__str__()

The `str` value of the object is the address quoted according to **RFC 5322** rules, but with no Content Transfer Encoding of any non-ASCII characters.

To support SMTP (**RFC 5321**), `Address` handles one special case: if `username` and `domain` are both the empty string (or `None`), then the string value of the `Address` is `<>`.

class email.headerregistry.**Group**(*display_name=None*, *addresses=None*)

The class used to represent an address group. The general form of an address group is:

```
display_name: [address-list];
```

As a convenience for processing lists of addresses that consist of a mixture of groups and single addresses, a `Group` may also be used to represent single addresses that are not part of a group by setting *display_name* to `None` and providing a list of the single address as *addresses*.

display_name

The `display_name` of the group. If it is `None` and there is exactly one `Address` in `addresses`, then the `Group` represents a single address that is not in a group.

addresses

A possibly empty tuple of *Address* objects representing the addresses in the group.

__str__()

The `str` value of a `Group` is formatted according to **RFC 5322**, but with no Content Transfer Encoding of any non-ASCII characters. If `display_name` is none and there is a single `Address` in the `addresses` list, the `str` value will be the same as the `str` of that single `Address`.

20.1.7 `email.contentmanager`: Gerenciando conteúdo MIME

Código-fonte: Lib/email/contentmanager.py

Adicionado na versão 3.6:[1]

class email.contentmanager.**ContentManager**

Classe base para gerenciadores de conteúdo. Fornece os mecanismos de registro padrão para registrar conversores entre conteúdo MIME e outras representações, bem como os métodos de despacho `get_content` e `set_content`.

get_content(*msg*, **args*, ***kw*)

Procura uma função manipuladora baseada no `mimetype` de *msg* (veja o próximo parágrafo), chama-a, passando por todos os argumentos, e retorna o resultado da chamada. A expectativa é que a função manipuladora extraia o payload de *msg* e retorne um objeto que codifique informações sobre os dados extraídos.

Para encontrar o manipulador, procura as seguintes chaves no registro, parando na primeira encontrada:

- a string que representa o tipo MIME completo (`maintype/subtype`)
- the string representing the `maintype`
- the empty string

If none of these keys produce a handler, raise a *KeyError* for the full MIME type.

set_content(*msg*, *obj*, **args*, ***kw*)

If the `maintype` is `multipart`, raise a *TypeError*; otherwise look up a handler function based on the type of *obj* (see next paragraph), call *clear_content()* on the *msg*, and call the handler function, passing through all arguments. The expectation is that the handler will transform and store *obj* into *msg*, possibly making other changes to *msg* as well, such as adding various MIME headers to encode information needed to interpret the stored data.

To find the handler, obtain the type of *obj* (`typ = type(obj)`), and look for the following keys in the registry, stopping with the first one found:

- the type itself (`typ`)
- the type's fully qualified name (`typ.__module__ + '.' + typ.__qualname__`).
- the type's `qualname` (`typ.__qualname__`)
- the type's `name` (`typ.__name__`).

If none of the above match, repeat all of the checks above for each of the types in the *MRO* (`typ.__mro__`). Finally, if no other key yields a handler, check for a handler for the key `None`. If there is no handler for `None`, raise a *KeyError* for the fully qualified name of the type.

Also add a *MIME-Version* header if one is not present (see also *MIMEPart*).

add_get_handler(*key*, *handler*)

Record the function *handler* as the handler for *key*. For the possible values of *key*, see *get_content()*.

[1] Originally added in 3.4 as a *provisional module*

add_set_handler(*typekey*, *handler*)

Record *handler* as the function to call when an object of a type matching *typekey* is passed to `set_content()`. For the possible values of *typekey*, see `set_content()`.

Content Manager Instances

Currently the email package provides only one concrete content manager, `raw_data_manager`, although more may be added in the future. `raw_data_manager` is the `content_manager` provided by `EmailPolicy` and its derivatives.

email.contentmanager.**raw_data_manager**

This content manager provides only a minimum interface beyond that provided by `Message` itself: it deals only with text, raw byte strings, and `Message` objects. Nevertheless, it provides significant advantages compared to the base API: `get_content` on a text part will return a unicode string without the application needing to manually decode it, `set_content` provides a rich set of options for controlling the headers added to a part and controlling the content transfer encoding, and it enables the use of the various `add_` methods, thereby simplifying the creation of multipart messages.

email.contentmanager.**get_content**(*msg*, *errors='replace'*)

Return the payload of the part as either a string (for `text` parts), an `EmailMessage` object (for `message/rfc822` parts), or a `bytes` object (for all other non-multipart types). Raise a `KeyError` if called on a `multipart`. If the part is a `text` part and *errors* is specified, use it as the error handler when decoding the payload to unicode. The default error handler is `replace`.

email.contentmanager.**set_content**(*msg*, *<'str'>*, *subtype="plain"*, *charset='utf-8'*, *cte=None*, *disposition=None*, *filename=None*, *cid=None*, *params=None*, *headers=None*)

email.contentmanager.**set_content**(*msg*, *<'bytes'>*, *maintype*, *subtype*, *cte="base64"*, *disposition=None*, *filename=None*, *cid=None*, *params=None*, *headers=None*)

email.contentmanager.**set_content**(*msg*, *<'EmailMessage'>*, *cte=None*, *disposition=None*, *filename=None*, *cid=None*, *params=None*, *headers=None*)

Adicione headers e payload à *msg*:

Add a `Content-Type` header with a `maintype/subtype` value.

- For `str`, set the MIME `maintype` to `text`, and set the subtype to *subtype* if it is specified, or `plain` if it is not.
- For `bytes`, use the specified *maintype* and *subtype*, or raise a `TypeError` if they are not specified.
- For `EmailMessage` objects, set the maintype to `message`, and set the subtype to *subtype* if it is specified or `rfc822` if it is not. If *subtype* is `partial`, raise an error (`bytes` objects must be used to construct `message/partial` parts).

If *charset* is provided (which is valid only for `str`), encode the string to bytes using the specified character set. The default is `utf-8`. If the specified *charset* is a known alias for a standard MIME charset name, use the standard charset instead.

If *cte* is set, encode the payload using the specified content transfer encoding, and set the `Content-Transfer-Encoding` header to that value. Possible values for *cte* are `quoted-printable`, `base64`, `7bit`, `8bit`, and `binary`. If the input cannot be encoded in the specified encoding (for example, specifying a *cte* of `7bit` for an input that contains non-ASCII values), raise a `ValueError`.

- For `str` objects, if *cte* is not set use heuristics to determine the most compact encoding. Prior to encoding, `str.splitlines()` is used to normalize all line boundaries, ensuring that each line of the payload is terminated by the current policy's `linesep` property (even if the original string did not end with one).
- For `bytes` objects, *cte* is taken to be base64 if not set, and the aforementioned newline translation is not performed.

- For *EmailMessage*, per **RFC 2046**, raise an error if a *cte* of `quoted-printable` or `base64` is requested for *subtype* `rfc822`, and for any *cte* other than `7bit` for *subtype* `external-body`. For `message/rfc822`, use `8bit` if *cte* is not specified. For all other values of *subtype*, use `7bit`.

> **Nota**
>
> A *cte* of `binary` does not actually work correctly yet. The `EmailMessage` object as modified by `set_content` is correct, but *BytesGenerator* does not serialize it correctly.

If *disposition* is set, use it as the value of the *Content-Disposition* header. If not specified, and *filename* is specified, add the header with the value `attachment`. If *disposition* is not specified and *filename* is also not specified, do not add the header. The only valid values for *disposition* are `attachment` and `inline`.

If *filename* is specified, use it as the value of the `filename` parameter of the *Content-Disposition* header.

If *cid* is specified, add a *Content-ID* header with *cid* as its value.

If *params* is specified, iterate its `items` method and use the resulting `(key, value)` pairs to set additional parameters on the *Content-Type* header.

If *headers* is specified and is a list of strings of the form `headername: headervalue` or a list of `header` objects (distinguished from strings by having a `name` attribute), add the headers to *msg*.

20.1.8 `email`: Exemplos

Aqui estão alguns exemplos de como usar o pacote *email* para ler, escrever e enviar mensagens de e-mail simples, bem como mensagens MIME mais complexas.

Primeiro, vamos ver como criar e enviar uma mensagem de texto simples (tanto o conteúdo do texto quanto os endereços podem conter caracteres Unicode):

```
# Importa smtplib para a função de envio
import smtplib

# Importa os módulos de email que precisamos
from email.message import EmailMessage

# Abre o arquivo de texto simples cujo nome está em arquivo texto para leitura.
with open(textfile) as fp:
    # Create a text/plain message
    msg = EmailMessage()
    msg.set_content(fp.read())

# me == o endereço de e-mail do remetente
# you == o endereço de e-mail do destinatário
msg['Subject'] = f'The contents of {textfile}'
msg['From'] = me
msg['To'] = you

# Envia a mensagem via nosso próprio servidor SMTP.
s = smtplib.SMTP('localhost')
s.send_message(msg)
s.quit()
```

A análise dos cabeçalhos **RFC 822** pode ser feita facilmente usando as classes do módulo *parser*:

```
# Importa os módulos de email que precisamos
#from email.parser import BytesParser
from email.parser import Parser
from email.policy import default

# Se os cabeçalhos de e-mal estão em um arquivo, descomente essas duas linhas:
# with open(messagefile, 'rb') as fp:
#     headers = BytesParser(policy=default).parse(fp)

#  Ou para análise de cabeçalhos em uma string (essa é uma operação comum), use:
headers = Parser(policy=default).parsestr(
        'From: Foo Bar <user@example.com>\n'
        'To: <someone_else@example.com>\n'
        'Subject: Test message\n'
        '\n'
        'Body would go here\n')

#  Agora os itens de cabeçalho podem ser acessadas como um dicionário:
print('To: {}'.format(headers['to']))
print('From: {}'.format(headers['from']))
print('Subject: {}'.format(headers['subject']))

# Você também pode acessar as partes dos endereços:
print('Recipient username: {}'.format(headers['to'].addresses[0].username))
print('Sender name: {}'.format(headers['from'].addresses[0].display_name))
```

Aqui está um exemplo de como enviar uma mensagem MIME contendo várias fotos de família que podem estar em um diretório:

```
# Importa smtplib para a função de envio.
import smtplib

# Aqui estão os módulos de pacotes de e-mail que precisamos.
from email.message import EmailMessage

# Cria a contêiner da mensagem de e-mail.
msg = EmailMessage()
msg['Subject'] = 'Our family reunion'
# me == o endereço de e-mail do remetente
# family = a lista dos endereços de e-mail de todos os destinatários
msg['From'] = me
msg['To'] = ', '.join(family)
msg.preamble = 'You will not see this in a MIME-aware mail reader.\n'

# Abre os arquivos no modo binário. Você também pode omitir
# o subtipo se você quiser que MIMEImage adivinhe.
for file in pngfiles:
    with open(file, 'rb') as fp:
        img_data = fp.read()
    msg.add_attachment(img_data, maintype='image',
                               subtype='png')

# Envia o e-mail por meio de nosso próprio servidor SMTP.
with smtplib.SMTP('localhost') as s:
    s.send_message(msg)
```

Aqui está um exemplo de como enviar todo o conteúdo de um diretório como uma mensagem de e-mail:[1]

```
#!/usr/bin/env python3

"""Envia o conteúdo de um diretório como uma mensagem MIME."""

import os
import smtplib
# Para adivinhar o tipo MIME com base na extensão do nome do arquivo
import mimetypes

from argparse import ArgumentParser

from email.message import EmailMessage
from email.policy import SMTP


def main():
    parser = ArgumentParser(description="""\
Envia o conteúdo de um diretório como uma mensagem MIME.
A menos que a opção -o seja informada, o e-mail será enviado por
meio de encaminhamento para o seu servidor SMTP local, que então
realiza o processo de entrega normal. Sua máquina local deve estar
executando um servidor SMTP.
""")
    parser.add_argument('-d', '--directory',
                        help="""Mail the contents of the specified directory,
                        otherwise use the current directory.  Only the regular
                        files in the directory are sent, and we don't recurse to
                        subdirectories.""")
    parser.add_argument('-o', '--output',
                        metavar='FILE',
                        help="""Print the composed message to FILE instead of
                        sending the message to the SMTP server.""")
    parser.add_argument('-s', '--sender', required=True,
                        help='The value of the From: header (required)')
    parser.add_argument('-r', '--recipient', required=True,
                        action='append', metavar='RECIPIENT',
                        default=[], dest='recipients',
                        help='A To: header value (at least one required)')
    args = parser.parse_args()
    directory = args.directory
    if not directory:
        directory = '.'
    # Cria a mensagem
    msg = EmailMessage()
    msg['Subject'] = f'Contents of directory {os.path.abspath(directory)}'
    msg['To'] = ', '.join(args.recipients)
    msg['From'] = args.sender
    msg.preamble = 'You will not see this in a MIME-aware mail reader.\n'

    for filename in os.listdir(directory):
        path = os.path.join(directory, filename)
        if not os.path.isfile(path):
            continue
        # Adivinhe o tipo de conteúdo com base na extensão do arquivo.
```

(continua na próxima página)

[1] Obrigado a Matthew Dixon Cowles pela inspiração original e pelos exemplos.

(continuação da página anterior)

```
        # A codificação será ignorada, embora devamos verificar coisas
        # simples como arquivos compactados ou compactados com gzip.
        ctype, encoding = mimetypes.guess_file_type(path)
        if ctype is None or encoding is not None:
            # Não foi possível fazer nenhuma suposição, ou o arquivo está
            # codificado (compactado), então use um tipo genérico de
            # conjunto de bits.
            ctype = 'application/octet-stream'
        maintype, subtype = ctype.split('/', 1)
        with open(path, 'rb') as fp:
            msg.add_attachment(fp.read(),
                               maintype=maintype,
                               subtype=subtype,
                               filename=filename)
    # Agora envia ou armazena a mensagem
    if args.output:
        with open(args.output, 'wb') as fp:
            fp.write(msg.as_bytes(policy=SMTP))
    else:
        with smtplib.SMTP('localhost') as s:
            s.send_message(msg)


if __name__ == '__main__':
    main()
```

Aqui está um exemplo de como desempacotar uma mensagem MIME, como a acima, para um diretório de arquivos:

```
#!/usr/bin/env python3

"""Desempacota uma mensagem MIME em um diretório de arquivos."""

import os
import email
import mimetypes

from email.policy import default

from argparse import ArgumentParser


def main():
    parser = ArgumentParser(description="""\
Desempacota uma mensagem MIME em um diretório de arquivos.
""")
    parser.add_argument('-d', '--directory', required=True,
                        help="""Unpack the MIME message into the named
                        directory, which will be created if it doesn't already
                        exist.""")
    parser.add_argument('msgfile')
    args = parser.parse_args()

    with open(args.msgfile, 'rb') as fp:
        msg = email.message_from_binary_file(fp, policy=default)

    try:
```

(continua na próxima página)

(continuação da página anterior)

```
        os.mkdir(args.directory)
    except FileExistsError:
        pass

    counter = 1
    for part in msg.walk():
        # multipart/* são apenas contêineres
        if part.get_content_maintype() == 'multipart':
            continue
        # Os aplicativos realmente devem higienizar o nome de arquivo
        # fornecido para que uma mensagem de e-mail não possa ser
        # usada para substituir arquivos importantes
        filename = part.get_filename()
        if not filename:
            ext = mimetypes.guess_extension(part.get_content_type())
            if not ext:
                # Usa uma extensão genérica de conjunto de bits
                ext = '.bin'
            filename = f'part-{counter:03d}{ext}'
        counter += 1
        with open(os.path.join(args.directory, filename), 'wb') as fp:
            fp.write(part.get_payload(decode=True))


if __name__ == '__main__':
    main()
```

Aqui está um exemplo de como criar uma mensagem HTML com uma versão alternativa em texto simples. Para tornar as coisas um pouco mais interessantes, incluímos uma imagem relacionada na parte html e salvamos uma cópia do que vamos enviar para o disco, assim como enviamos.

```
#!/usr/bin/env python3

import smtplib

from email.message import EmailMessage
from email.headerregistry import Address
from email.utils import make_msgid

# Cria a mensagem texto base.
msg = EmailMessage()
msg['Subject'] = "Pourquoi pas des asperges pour ce midi ?"
msg['From'] = Address("Pepé Le Pew", "pepe", "example.com")
msg['To'] = (Address("Penelope Pussycat", "penelope", "example.com"),
             Address("Fabrette Pussycat", "fabrette", "example.com"))
msg.set_content("""\
Salut!

Cette recette [1] sera sûrement un très bon repas.

[1] http://www.yummly.com/recipe/Roasted-Asparagus-Epicurious-203718

--Pepé
""")

# Adiciona a versão HTML. Isso converte a mensagem em um contêiner
```

(continua na próxima página)

(continuação da página anterior)

```
# multiparte/alternativo, com a mensagem de texto original como a primeira
# parte e a nova mensagem HTML como a segunda parte.
asparagus_cid = make_msgid()
msg.add_alternative("""\
<html>
  <head></head>
  <body>
    <p>Salut!</p>
    <p>Cette
        <a href="http://www.yummly.com/recipe/Roasted-Asparagus-Epicurious-203718">
            recette
        </a> sera sûrement un très bon repas.
    </p>
    <img src="cid:{asparagus_cid}">
  </body>
</html>
""".format(asparagus_cid=asparagus_cid[1:-1]), subtype='html')
# observe que precisamos remover o <> do msgid para usar no html.

# Agora adiciona a imagem relacionada à parte html.
with open("roasted-asparagus.jpg", 'rb') as img:
    msg.get_payload()[1].add_related(img.read(), 'image', 'jpeg',
                                     cid=asparagus_cid)

# Agora adiciona a imagem relacionada à parte html.
with open('outgoing.msg', 'wb') as f:
    f.write(bytes(msg))

# Envia a mensagem via servidor SMTP local.
with smtplib.SMTP('localhost') as s:
    s.send_message(msg)
```

Se nos fosse enviada a mensagem do último exemplo, aqui está uma maneira de processá-la:

```
import os
import sys
import tempfile
import mimetypes
import webbrowser

# Importa os módulos de e-mail que precisaremos
from email import policy
from email.parser import BytesParser


def magic_html_parser(html_text, partfiles):
    """Retorna HTML com segurança e link para partfiles.

    Reescreve os atributos href="cid:...." para apontar aos arquivos em partfiles.
    Embora não seja trivial, isso deve ser possível usando html.parser.
    """
    raise NotImplementedError("Add the magic needed")


# Em um programa real, você pegaria o nome de arquivo a partir dos argumentos.
with open('outgoing.msg', 'rb') as fp:
```

(continua na próxima página)

(continuação da página anterior)

```
    msg = BytesParser(policy=policy.default).parse(fp)

# Agora os itens de cabeçalho podem ser acessados como um dicionário e
# qualquer caractere não-ASCII será convertido para unicode:
print('To:', msg['to'])
print('From:', msg['from'])
print('Subject:', msg['subject'])

# Se quisermos imprimir uma prévia do conteúdo da mensagem, podemos
# extrair o conteúdo menos formatado e imprimir as três primeiras linhas.
# É claro que, se a mensagem não tiver uma parte de texto simples,
# imprimir as três primeiras linhas de HTML provavelmente será inútil,
# mas este é apenas um exemplo conceitual.
simplest = msg.get_body(preferencelist=('plain', 'html'))
print()
print(''.join(simplest.get_content().splitlines(keepends=True)[:3]))

ans = input("View full message?")
if ans.lower()[0] == 'n':
    sys.exit()

# Podemos extrair a alternativa richest para exibi-la:
richest = msg.get_body()
partfiles = {}
if richest['content-type'].maintype == 'text':
    if richest['content-type'].subtype == 'plain':
        for line in richest.get_content().splitlines():
            print(line)
        sys.exit()
    elif richest['content-type'].subtype == 'html':
        body = richest
    else:
        print("Don't know how to display {}".format(richest.get_content_type()))
        sys.exit()
elif richest['content-type'].content_type == 'multipart/related':
    body = richest.get_body(preferencelist=('html'))
    for part in richest.iter_attachments():
        fn = part.get_filename()
        if fn:
            extension = os.path.splitext(part.get_filename())[1]
        else:
            extension = mimetypes.guess_extension(part.get_content_type())
        with tempfile.NamedTemporaryFile(suffix=extension, delete=False) as f:
            f.write(part.get_content())
            # remove novamente o <> para ir da forma email de cid para html.
            partfiles[part['content-id'][1:-1]] = f.name
else:
    print("Don't know how to display {}".format(richest.get_content_type()))
    sys.exit()
with tempfile.NamedTemporaryFile(mode='w', delete=False) as f:
    f.write(magic_html_parser(body.get_content(), partfiles))
webbrowser.open(f.name)
os.remove(f.name)
for fn in partfiles.values():
    os.remove(fn)
```

(continua na próxima página)

(continuação da página anterior)

```
# É claro que há muitas mensagens de e-mail que podem quebrar
# esse simples programa, mas ele lidará com as mais comuns.
```

Até o prompt, a saída do comando acima é:

```
To: Penelope Pussycat <penelope@example.com>, Fabrette Pussycat <fabrette@example.
↪com>
From: Pepé Le Pew <pepe@example.com>
Subject: Pourquoi pas des asperges pour ce midi ?

Salut!

Cette recette [1] sera sûrement un très bon repas.
```

API legada

20.1.9 `email.message.Message`: Representing an email message using the `compat32` API

The `Message` class is very similar to the `EmailMessage` class, without the methods added by that class, and with the default behavior of certain other methods being slightly different. We also document here some methods that, while supported by the `EmailMessage` class, are not recommended unless you are dealing with legacy code.

The philosophy and structure of the two classes is otherwise the same.

This document describes the behavior under the default (for `Message`) policy `Compat32`. If you are going to use another policy, you should be using the `EmailMessage` class instead.

An email message consists of *headers* and a *payload*. Headers must be **RFC 5322** style names and values, where the field name and value are separated by a colon. The colon is not part of either the field name or the field value. The payload may be a simple text message, or a binary object, or a structured sequence of sub-messages each with their own set of headers and their own payload. The latter type of payload is indicated by the message having a MIME type such as `multipart/*` or `message/rfc822`.

The conceptual model provided by a `Message` object is that of an ordered dictionary of headers with additional methods for accessing both specialized information from the headers, for accessing the payload, for generating a serialized version of the message, and for recursively walking over the object tree. Note that duplicate headers are supported but special methods must be used to access them.

The `Message` pseudo-dictionary is indexed by the header names, which must be ASCII values. The values of the dictionary are strings that are supposed to contain only ASCII characters; there is some special handling for non-ASCII input, but it doesn't always produce the correct results. Headers are stored and returned in case-preserving form, but field names are matched case-insensitively. There may also be a single envelope header, also known as the *Unix-From* header or the `From_` header. The *payload* is either a string or bytes, in the case of simple message objects, or a list of `Message` objects, for MIME container documents (e.g. `multipart/*` and `message/rfc822`).

Here are the methods of the `Message` class:

class `email.message.`**Message**(*policy=compat32*)

If *policy* is specified (it must be an instance of a `policy` class) use the rules it specifies to update and serialize the representation of the message. If *policy* is not set, use the `compat32` policy, which maintains backward compatibility with the Python 3.2 version of the email package. For more information see the `policy` documentation.

Alterado na versão 3.3: The *policy* keyword argument was added.

as_string(*unixfrom=False*, *maxheaderlen=0*, *policy=None*)

Return the entire message flattened as a string. When optional *unixfrom* is true, the envelope header is included in the returned string. *unixfrom* defaults to `False`. For backward compatibility reasons, *maxheaderlen* defaults to `0`, so if you want a different value you must override it explicitly (the value specified for *max_line_length* in the policy will be ignored by this method). The *policy* argument may be

used to override the default policy obtained from the message instance. This can be used to control some of the formatting produced by the method, since the specified *policy* will be passed to the `Generator`.

Flattening the message may trigger changes to the *Message* if defaults need to be filled in to complete the transformation to a string (for example, MIME boundaries may be generated or modified).

Note that this method is provided as a convenience and may not always format the message the way you want. For example, by default it does not do the mangling of lines that begin with `From` that is required by the Unix mbox format. For more flexibility, instantiate a *Generator* instance and use its *flatten()* method directly. For example:

```
from io import StringIO
from email.generator import Generator
fp = StringIO()
g = Generator(fp, mangle_from_=True, maxheaderlen=60)
g.flatten(msg)
text = fp.getvalue()
```

If the message object contains binary data that is not encoded according to RFC standards, the non-compliant data will be replaced by unicode "unknown character" code points. (See also *as_bytes()* and *BytesGenerator*.)

Alterado na versão 3.4: the *policy* keyword argument was added.

__str__()

Equivalent to *as_string()*. Allows `str(msg)` to produce a string containing the formatted message.

as_bytes(*unixfrom=False*, *policy=None*)

Return the entire message flattened as a bytes object. When optional *unixfrom* is true, the envelope header is included in the returned string. *unixfrom* defaults to `False`. The *policy* argument may be used to override the default policy obtained from the message instance. This can be used to control some of the formatting produced by the method, since the specified *policy* will be passed to the `BytesGenerator`.

Flattening the message may trigger changes to the *Message* if defaults need to be filled in to complete the transformation to a string (for example, MIME boundaries may be generated or modified).

Note that this method is provided as a convenience and may not always format the message the way you want. For example, by default it does not do the mangling of lines that begin with `From` that is required by the Unix mbox format. For more flexibility, instantiate a *BytesGenerator* instance and use its *flatten()* method directly. For example:

```
from io import BytesIO
from email.generator import BytesGenerator
fp = BytesIO()
g = BytesGenerator(fp, mangle_from_=True, maxheaderlen=60)
g.flatten(msg)
text = fp.getvalue()
```

Adicionado na versão 3.4.

__bytes__()

Equivalent to *as_bytes()*. Allows `bytes(msg)` to produce a bytes object containing the formatted message.

Adicionado na versão 3.4.

is_multipart()

Return `True` if the message's payload is a list of sub-*Message* objects, otherwise return `False`. When *is_multipart()* returns `False`, the payload should be a string object (which might be a CTE encoded binary payload). (Note that *is_multipart()* returning `True` does not necessarily mean that "msg.get_content_maintype() == 'multipart'" will return the `True`. For example, `is_multipart` will return `True` when the *Message* is of type `message/rfc822`.)

set_unixfrom(*unixfrom*)

Set the message's envelope header to *unixfrom*, which should be a string.

get_unixfrom()

Return the message's envelope header. Defaults to `None` if the envelope header was never set.

attach(*payload*)

Add the given *payload* to the current payload, which must be `None` or a list of *Message* objects before the call. After the call, the payload will always be a list of *Message* objects. If you want to set the payload to a scalar object (e.g. a string), use *set_payload()* instead.

This is a legacy method. On the *EmailMessage* class its functionality is replaced by *set_content()* and the related `make` and `add` methods.

get_payload(*i=None*, *decode=False*)

Return the current payload, which will be a list of *Message* objects when *is_multipart()* is `True`, or a string when *is_multipart()* is `False`. If the payload is a list and you mutate the list object, you modify the message's payload in place.

With optional argument *i*, *get_payload()* will return the *i*-th element of the payload, counting from zero, if *is_multipart()* is `True`. An *IndexError* will be raised if *i* is less than 0 or greater than or equal to the number of items in the payload. If the payload is a string (i.e. *is_multipart()* is `False`) and *i* is given, a *TypeError* is raised.

Optional *decode* is a flag indicating whether the payload should be decoded or not, according to the *Content-Transfer-Encoding* header. When `True` and the message is not a multipart, the payload will be decoded if this header's value is `quoted-printable` or `base64`. If some other encoding is used, or *Content-Transfer-Encoding* header is missing, the payload is returned as-is (undecoded). In all cases the returned value is binary data. If the message is a multipart and the *decode* flag is `True`, then `None` is returned. If the payload is base64 and it was not perfectly formed (missing padding, characters outside the base64 alphabet), then an appropriate defect will be added to the message's defect property (*InvalidBase64PaddingDefect* or *InvalidBase64CharactersDefect*, respectively).

When *decode* is `False` (the default) the body is returned as a string without decoding the *Content-Transfer-Encoding*. However, for a *Content-Transfer-Encoding* of 8bit, an attempt is made to decode the original bytes using the `charset` specified by the *Content-Type* header, using the `replace` error handler. If no `charset` is specified, or if the `charset` given is not recognized by the email package, the body is decoded using the default ASCII charset.

This is a legacy method. On the *EmailMessage* class its functionality is replaced by *get_content()* and *iter_parts()*.

set_payload(*payload*, *charset=None*)

Set the entire message object's payload to *payload*. It is the client's responsibility to ensure the payload invariants. Optional *charset* sets the message's default character set; see *set_charset()* for details.

This is a legacy method. On the *EmailMessage* class its functionality is replaced by *set_content()*.

set_charset(*charset*)

Set the character set of the payload to *charset*, which can either be a *Charset* instance (see *email.charset*), a string naming a character set, or `None`. If it is a string, it will be converted to a *Charset* instance. If *charset* is `None`, the `charset` parameter will be removed from the *Content-Type* header (the message will not be otherwise modified). Anything else will generate a *TypeError*.

If there is no existing *MIME-Version* header one will be added. If there is no existing *Content-Type* header, one will be added with a value of *text/plain*. Whether the *Content-Type* header already exists or not, its `charset` parameter will be set to *charset.output_charset*. If *charset.input_charset* and *charset.output_charset* differ, the payload will be re-encoded to the *output_charset*. If there is no existing *Content-Transfer-Encoding* header, then the payload will be transfer-encoded, if needed, using the specified *Charset*, and a header with the appropriate value will be added. If a *Content-Transfer-Encoding* header already exists, the payload is assumed to already be correctly encoded using that *Content-Transfer-Encoding* and is not modified.

This is a legacy method. On the *EmailMessage* class its functionality is replaced by the *charset* parameter of the *email.message.EmailMessage.set_content()* method.

get_charset()

Return the *Charset* instance associated with the message's payload.

This is a legacy method. On the *EmailMessage* class it always returns None.

The following methods implement a mapping-like interface for accessing the message's **RFC 2822** headers. Note that there are some semantic differences between these methods and a normal mapping (i.e. dictionary) interface. For example, in a dictionary there are no duplicate keys, but here there may be duplicate message headers. Also, in dictionaries there is no guaranteed order to the keys returned by *keys()*, but in a *Message* object, headers are always returned in the order they appeared in the original message, or were added to the message later. Any header deleted and then re-added are always appended to the end of the header list.

These semantic differences are intentional and are biased toward maximal convenience.

Note that in all cases, any envelope header present in the message is not included in the mapping interface.

In a model generated from bytes, any header values that (in contravention of the RFCs) contain non-ASCII bytes will, when retrieved through this interface, be represented as *Header* objects with a charset of unknown-8bit.

__len__()

Return the total number of headers, including duplicates.

__contains__(*name*)

Return True if the message object has a field named *name*. Matching is done case-insensitively and *name* should not include the trailing colon. Used for the in operator, e.g.:

```
if 'message-id' in myMessage:
   print('Message-ID:', myMessage['message-id'])
```

__getitem__(*name*)

Return the value of the named header field. *name* should not include the colon field separator. If the header is missing, None is returned; a *KeyError* is never raised.

Note that if the named field appears more than once in the message's headers, exactly which of those field values will be returned is undefined. Use the *get_all()* method to get the values of all the extant named headers.

__setitem__(*name*, *val*)

Add a header to the message with field name *name* and value *val*. The field is appended to the end of the message's existing fields.

Note that this does *not* overwrite or delete any existing header with the same name. If you want to ensure that the new header is the only one present in the message with field name *name*, delete the field first, e.g.:

```
del msg['subject']
msg['subject'] = 'Python roolz!'
```

__delitem__(*name*)

Delete all occurrences of the field with name *name* from the message's headers. No exception is raised if the named field isn't present in the headers.

keys()

Return a list of all the message's header field names.

values()

Return a list of all the message's field values.

items()

Return a list of 2-tuples containing all the message's field headers and values.

get(*name*, *failobj=None*)

Return the value of the named header field. This is identical to `__getitem__()` except that optional *failobj* is returned if the named header is missing (defaults to `None`).

Here are some additional useful methods:

get_all(*name*, *failobj=None*)

Return a list of all the values for the field named *name*. If there are no such named headers in the message, *failobj* is returned (defaults to `None`).

add_header(*_name*, *_value*, ***_params*)

Extended header setting. This method is similar to *__setitem__()* except that additional header parameters can be provided as keyword arguments. *_name* is the header field to add and *_value* is the *primary* value for the header.

For each item in the keyword argument dictionary *_params*, the key is taken as the parameter name, with underscores converted to dashes (since dashes are illegal in Python identifiers). Normally, the parameter will be added as `key="value"` unless the value is `None`, in which case only the key will be added. If the value contains non-ASCII characters, it can be specified as a three tuple in the format `(CHARSET, LANGUAGE, VALUE)`, where `CHARSET` is a string naming the charset to be used to encode the value, `LANGUAGE` can usually be set to `None` or the empty string (see **RFC 2231** for other possibilities), and `VALUE` is the string value containing non-ASCII code points. If a three tuple is not passed and the value contains non-ASCII characters, it is automatically encoded in **RFC 2231** format using a `CHARSET` of `utf-8` and a `LANGUAGE` of `None`.

Aqui está um exemplo:

```
msg.add_header('Content-Disposition', 'attachment', filename='bud.gif')
```

This will add a header that looks like

```
Content-Disposition: attachment; filename="bud.gif"
```

Um exemplo com caracteres não-ASCII:

```
msg.add_header('Content-Disposition', 'attachment',
               filename=('iso-8859-1', '', 'Fußballer.ppt'))
```

Que produz

```
Content-Disposition: attachment; filename*="iso-8859-1''Fu%DFballer.ppt"
```

replace_header(*_name*, *_value*)

Replace a header. Replace the first header found in the message that matches *_name*, retaining header order and field name case. If no matching header was found, a *KeyError* is raised.

get_content_type()

Return the message's content type. The returned string is coerced to lower case of the form *maintype/subtype*. If there was no *Content-Type* header in the message the default type as given by *get_default_type()* will be returned. Since according to **RFC 2045**, messages always have a default type, *get_content_type()* will always return a value.

RFC 2045 defines a message's default type to be *text/plain* unless it appears inside a *multipart/digest* container, in which case it would be *message/rfc822*. If the *Content-Type* header has an invalid type specification, **RFC 2045** mandates that the default type be *text/plain*.

get_content_maintype()

Return the message's main content type. This is the *maintype* part of the string returned by *get_content_type()*.

get_content_subtype()

Return the message's sub-content type. This is the *subtype* part of the string returned by *get_content_type()*.

get_default_type()

Return the default content type. Most messages have a default content type of *text/plain*, except for messages that are subparts of *multipart/digest* containers. Such subparts have a default content type of *message/rfc822*.

set_default_type(*ctype*)

Set the default content type. *ctype* should either be *text/plain* or *message/rfc822*, although this is not enforced. The default content type is not stored in the *Content-Type* header.

get_params(*failobj=None*, *header='content-type'*, *unquote=True*)

Return the message's *Content-Type* parameters, as a list. The elements of the returned list are 2-tuples of key/value pairs, as split on the `'='` sign. The left hand side of the `'='` is the key, while the right hand side is the value. If there is no `'='` sign in the parameter the value is the empty string, otherwise the value is as described in *get_param()* and is unquoted if optional *unquote* is `True` (the default).

Optional *failobj* is the object to return if there is no *Content-Type* header. Optional *header* is the header to search instead of *Content-Type*.

This is a legacy method. On the *EmailMessage* class its functionality is replaced by the *params* property of the individual header objects returned by the header access methods.

get_param(*param*, *failobj=None*, *header='content-type'*, *unquote=True*)

Return the value of the *Content-Type* header's parameter *param* as a string. If the message has no *Content-Type* header or if there is no such parameter, then *failobj* is returned (defaults to `None`).

Optional *header* if given, specifies the message header to use instead of *Content-Type*.

Parameter keys are always compared case insensitively. The return value can either be a string, or a 3-tuple if the parameter was **RFC 2231** encoded. When it's a 3-tuple, the elements of the value are of the form `(CHARSET, LANGUAGE, VALUE)`. Note that both `CHARSET` and `LANGUAGE` can be `None`, in which case you should consider `VALUE` to be encoded in the `us-ascii` charset. You can usually ignore `LANGUAGE`.

If your application doesn't care whether the parameter was encoded as in **RFC 2231**, you can collapse the parameter value by calling *email.utils.collapse_rfc2231_value()*, passing in the return value from *get_param()*. This will return a suitably decoded Unicode string when the value is a tuple, or the original string unquoted if it isn't. For example:

```
rawparam = msg.get_param('foo')
param = email.utils.collapse_rfc2231_value(rawparam)
```

In any case, the parameter value (either the returned string, or the `VALUE` item in the 3-tuple) is always unquoted, unless *unquote* is set to `False`.

This is a legacy method. On the *EmailMessage* class its functionality is replaced by the *params* property of the individual header objects returned by the header access methods.

set_param(*param*, *value*, *header='Content-Type'*, *requote=True*, *charset=None*, *language=''*, *replace=False*)

Set a parameter in the *Content-Type* header. If the parameter already exists in the header, its value will be replaced with *value*. If the *Content-Type* header as not yet been defined for this message, it will be set to *text/plain* and the new parameter value will be appended as per **RFC 2045**.

Optional *header* specifies an alternative header to *Content-Type*, and all parameters will be quoted as necessary unless optional *requote* is `False` (the default is `True`).

If optional *charset* is specified, the parameter will be encoded according to **RFC 2231**. Optional *language* specifies the RFC 2231 language, defaulting to the empty string. Both *charset* and *language* should be strings.

If *replace* is `False` (the default) the header is moved to the end of the list of headers. If *replace* is `True`, the header will be updated in place.

Alterado na versão 3.4: Palavra-chave `replace` foi adicionada.

del_param(*param*, *header='content-type'*, *requote=True*)

Remove the given parameter completely from the *Content-Type* header. The header will be re-written in place without the parameter or its value. All values will be quoted as necessary unless *requote* is `False` (the default is `True`). Optional *header* specifies an alternative to *Content-Type*.

set_type(*type*, *header='Content-Type'*, *requote=True*)

Set the main type and subtype for the *Content-Type* header. *type* must be a string in the form *maintype/subtype*, otherwise a *ValueError* is raised.

This method replaces the *Content-Type* header, keeping all the parameters in place. If *requote* is `False`, this leaves the existing header's quoting as is, otherwise the parameters will be quoted (the default).

An alternative header can be specified in the *header* argument. When the *Content-Type* header is set a *MIME-Version* header is also added.

This is a legacy method. On the *EmailMessage* class its functionality is replaced by the `make_` and `add_` methods.

get_filename(*failobj=None*)

Return the value of the `filename` parameter of the *Content-Disposition* header of the message. If the header does not have a `filename` parameter, this method falls back to looking for the `name` parameter on the *Content-Type* header. If neither is found, or the header is missing, then *failobj* is returned. The returned string will always be unquoted as per *email.utils.unquote()*.

get_boundary(*failobj=None*)

Return the value of the `boundary` parameter of the *Content-Type* header of the message, or *failobj* if either the header is missing, or has no `boundary` parameter. The returned string will always be unquoted as per *email.utils.unquote()*.

set_boundary(*boundary*)

Set the `boundary` parameter of the *Content-Type* header to *boundary*. *set_boundary()* will always quote *boundary* if necessary. A *HeaderParseError* is raised if the message object has no *Content-Type* header.

Note that using this method is subtly different than deleting the old *Content-Type* header and adding a new one with the new boundary via *add_header()*, because *set_boundary()* preserves the order of the *Content-Type* header in the list of headers. However, it does *not* preserve any continuation lines which may have been present in the original *Content-Type* header.

get_content_charset(*failobj=None*)

Return the `charset` parameter of the *Content-Type* header, coerced to lower case. If there is no *Content-Type* header, or if that header has no `charset` parameter, *failobj* is returned.

Note that this method differs from *get_charset()* which returns the *Charset* instance for the default encoding of the message body.

get_charsets(*failobj=None*)

Return a list containing the character set names in the message. If the message is a *multipart*, then the list will contain one element for each subpart in the payload, otherwise, it will be a list of length 1.

Each item in the list will be a string which is the value of the `charset` parameter in the *Content-Type* header for the represented subpart. However, if the subpart has no *Content-Type* header, no `charset` parameter, or is not of the *text* main MIME type, then that item in the returned list will be *failobj*.

get_content_disposition()

Return the lowercased value (without parameters) of the message's *Content-Disposition* header if it has one, or `None`. The possible values for this method are *inline*, *attachment* or `None` if the message follows **RFC 2183**.

Adicionado na versão 3.5.

walk()

The *walk()* method is an all-purpose generator which can be used to iterate over all the parts and subparts of a message object tree, in depth-first traversal order. You will typically use *walk()* as the iterator in a `for` loop; each iteration returns the next subpart.

Here's an example that prints the MIME type of every part of a multipart message structure:

```
>>> for part in msg.walk():
...     print(part.get_content_type())
multipart/report
text/plain
message/delivery-status
text/plain
text/plain
message/rfc822
text/plain
```

`walk` iterates over the subparts of any part where *is_multipart()* returns `True`, even though `msg.get_content_maintype() == 'multipart'` may return `False`. We can see this in our example by making use of the `_structure` debug helper function:

```
>>> for part in msg.walk():
...     print(part.get_content_maintype() == 'multipart',
...           part.is_multipart())
True True
False False
False True
False False
False False
False True
False False
>>> _structure(msg)
multipart/report
    text/plain
    message/delivery-status
        text/plain
        text/plain
    message/rfc822
        text/plain
```

Here the `message` parts are not `multiparts`, but they do contain subparts. `is_multipart()` returns `True` and `walk` descends into the subparts.

Message objects can also optionally contain two instance attributes, which can be used when generating the plain text of a MIME message.

preamble

The format of a MIME document allows for some text between the blank line following the headers, and the first multipart boundary string. Normally, this text is never visible in a MIME-aware mail reader because it falls outside the standard MIME armor. However, when viewing the raw text of the message, or when viewing the message in a non-MIME aware reader, this text can become visible.

The *preamble* attribute contains this leading extra-armor text for MIME documents. When the *Parser* discovers some text after the headers but before the first boundary string, it assigns this text to the message's *preamble* attribute. When the *Generator* is writing out the plain text representation of a MIME message, and it finds the message has a *preamble* attribute, it will write this text in the area between the headers and the first boundary. See *email.parser* and *email.generator* for details.

Note that if the message object has no preamble, the *preamble* attribute will be `None`.

epilogue

The *epilogue* attribute acts the same way as the *preamble* attribute, except that it contains text that appears between the last boundary and the end of the message.

You do not need to set the epilogue to the empty string in order for the `Generator` to print a newline at the end of the file.

defects

The *defects* attribute contains a list of all the problems found when parsing this message. See `email.errors` for a detailed description of the possible parsing defects.

20.1.10 `email.mimeemail.mime`: Criando e-mail e objetos MIME do zero

Código-fonte: Lib/email/mime/

Este módulo faz parte da API de e-mail legada (`Compat32`). Sua funcionalidade é parcialmente substituída por `contentmanager` na nova API, mas em certos aplicativos essas classes ainda podem ser úteis, mesmo em código não legado.

Normalmente, você obtém uma estrutura de objeto de mensagem passando um arquivo ou algum texto para um analisador sintático, que analisa o texto e retorna o objeto de mensagem raiz. No entanto, você também pode criar uma estrutura de mensagem completa do zero, ou até objetos individuais de `Message` manualmente. De fato, você também pode pegar uma estrutura existente e adicionar novos objetos `Message`, movê-los, etc. Isso cria uma interface muito conveniente para fatiar e cortar dados de mensagens MIME.

Você pode criar uma nova estrutura de objeto criando instâncias de `Message`, adicionando anexos e todos os cabeçalhos apropriados manualmente. Porém, para mensagens MIME, o pacote `email` fornece algumas subclasses convenientes para facilitar as coisas.

Arquivo estão as classes:

class email.mime.base.**MIMEBase**(*_maintype*, *_subtype*, *, *policy=compat32*, ***_params*)

Módulo: `email.mime.base`

Esta é a classe base para todas as subclasses específicas de MIME de `Message`. Normalmente você não criará instâncias especificamente de `MIMEBase`, embora possa. A `MIMEBase` é fornecida principalmente como uma classe base conveniente para subclasses mais específicas para MIME.

_maintype é o tipo principal de `Content-Type` (ex., `text` ou `image`) e *_subtype* é o tipo principal de `Content-Type` (ex., `plain` ou `gif`). *_params* é um dicionário de parâmetros chave/valor e é passado diretamente para `Message.add_header`.

Se *policy* for especificado, (o padrão é a política `compat32`) será passado para `Message`.

A classe `MIMEBase` sempre adiciona um cabeçalho `Content-Type` (com base em *_maintype*, *_subtype* e *_params*) e um cabeçalho `MIME-Version` (sempre definido como `1.0`).

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.nonmultipart.**MIMENonMultipart**

Módulo: `email.mime.nonmultipart`

Uma subclasse de `MIMEBase`, esta é uma classe base intermediária para mensagens MIME que não são `multipart`. O principal objetivo desta classe é impedir o uso do método `attach()`, que só faz sentido para mensagens `multipart`. Se `attach()` for chamado, uma exceção `MultipartConversionError` será levantada.

class email.mime.multipart.**MIMEMultipart**(*_subtype='mixed'*, *boundary=None*, *_subparts=None*, *, *policy=compat32*, ***_params*)

Módulo: `email.mime.multipart`

Uma subclasse de *MIMEBase*, esta é uma classe base intermediária para mensagens MIME que são *multipart*. O *_subtype* opcional é padronizado como *mixed*, mas pode ser usado para especificar o subtipo da mensagem. Um cabeçalho *Content-Type* de *multipart/_subtype* será adicionado ao objeto da mensagem. Um cabeçalho *MIME-Version* também será adicionado.

O *boundary* opcional é a string de limites de várias partes. Quando None (o padrão), o limite é calculado quando necessário (por exemplo, quando a mensagem é serializada).

_subparts é uma sequência de subpartes iniciais para a carga. Deve ser possível converter essa sequência em uma lista. Você sempre pode anexar novas subpartes à mensagem usando o método *Message.attach*.

O argumento opcional *policy* tem como padrão *compat32*.

Parâmetros adicionais para o cabeçalho *Content-Type* são retirados dos argumentos nomeados ou passados para o argumento *_params*, que é um dicionário.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.application.**MIMEApplication**(*_data*, *_subtype='octet-stream'*, *_encoder=email.encoders.encode_base64*, ***, *policy=compat32*, ***_params*)

Módulo: *email.mime.application*

Uma subclasse de *MIMENonMultipart*, a classe *MIMEApplication* é usada para representar objetos de mensagem MIME do tipo principal *application*. *_data* contém os bytes para os dados brutos da aplicação. O *_subtype* opcional especifica o subtipo MIME e o padrão é *octet-stream*.

O *_encoder* opcional é um chamável (isto é, função) que executará a codificação real dos dados para transporte. Esse chamável requer um argumento, que é a instância *MIMEApplication*. Ele deve usar *get_payload()* e *set_payload()* para alterar a carga útil para o formulário codificado. Também deve adicionar *Content-Transfer-Encoding* ou outros cabeçalhos ao objeto de mensagem, conforme necessário. A codificação padrão é base64. Veja o módulo *email.encoders* para obter uma lista dos codificadores embutidos.

O argumento opcional *policy* tem como padrão *compat32*.

_params são passados diretamente para o construtor da classe base.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.audio.**MIMEAudio**(*_audiodata*, *_subtype=None*, *_encoder=email.encoders.encode_base64*, ***, *policy=compat32*, ***_params*)

Módulo: *email.mime.audio*

Uma subclasse de *MIMENonMultipart*, a classe *MIMEAudio* é usada para criar objetos de mensagem MIME do tipo principal *audio*. *_audiodata* contém os bytes para os dados brutos de áudio. Se esses dados puderem ser decodificados como au, wav, aiff ou aifc, então o subtipo será automaticamente incluído no cabeçalho *Content-Type*. Caso contrário, você pode especificar explicitamente o subtipo de áudio por meio do argumento *_subtype*. Se o tipo menor não pôde ser adivinhado e *_subtype* não foi fornecido, então um exceção *TypeError* é levantada.

O *_encoder* opcional é um chamável (ou seja, função) que executará a codificação real dos dados de áudio para transporte. Esse chamável requer um argumento, que é a instância *MIMEAudio*. Ele deve usar *get_payload()* e *set_payload()* para alterar a carga útil para a forma codificada. Também deve adicionar *Content-Transfer-Encoding* ou outros cabeçalhos ao objeto de mensagem, conforme necessário. A codificação padrão é base64. Veja o módulo *email.encoders* para obter uma lista dos codificadores embutidos.

O argumento opcional *policy* tem como padrão *compat32*.

_params são passados diretamente para o construtor da classe base.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.image.**MIMEImage**(*_imagedata*, *_subtype=None*, *_encoder=email.encoders.encode_base64*, ***, *policy=compat32*, ***_params*)

Módulo: *email.mime.image*

Uma subclasse de *MIMENonMultipart*, a classe *MIMEImage* é usada para criar objetos de mensagem MIME do tipo principal *image*. *_imagedata* contém os bytes para os dados de imagem bruta. Se esse tipo de dados puder ser detectado (jpeg, png, gif, tiff, rgb, pbm, pgm, ppm, rast, xbm, bmp, webp e exr tentados), então o subtipo será automaticamente incluído no cabeçalho *Content-Type*. Caso contrário, você pode especificar explicitamente o subtipo de imagem por meio do argumento *_subtype*. Se o tipo secundário não puder ser adivinhado e *_subtype* não for fornecido, então *TypeError* será levantada.

O *_encoder* opcional é um chamável (ou seja, função) que executará a codificação real dos dados de imagem para transporte. Esse chamável requer um argumento, que é a instância *MIMEImage*. Ele deve usar *get_payload()* e *set_payload()* para alterar a carga útil para a forma codificada. Também deve adicionar *Content-Transfer-Encoding* ou outros cabeçalhos ao objeto de mensagem, conforme necessário. A codificação padrão é base64. Veja o módulo *email.encoders* para obter uma lista dos codificadores embutidos.

O argumento opcional *policy* tem como padrão *compat32*.

_params são passados diretamente para o construtor *MIMEBase*.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.message.**MIMEMessage**(*_msg*, *_subtype='rfc822'*, ***, *policy=compat32*)

Módulo: *email.mime.message*

Uma subclasse de *MIMENonMultipart*, a classe *MIMEMessage* é usada para criar objetos MIME do tipo principal *message*. *_msg* é usado como carga útil e deve ser uma instância da classe *Message* (ou uma subclasse dela), caso contrário, uma *TypeError* é levantada.

O opcional *_subtype* define o subtipo da mensagem; o padrão é *rfc822*.

O argumento opcional *policy* tem como padrão *compat32*.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.text.**MIMEText**(*_text*, *_subtype='plain'*, *_charset=None*, ***, *policy=compat32*)

Módulo: *email.mime.text*

Uma subclasse de *MIMENonMultipart*, a classe *MIMEText* é usada para criar objetos MIME do tipo principal *text*. *_text* é a string para a carga útil. *_subtype* é o tipo secundário e o padrão é *plain*. *_charset* é o conjunto de caracteres do texto e é passado como um argumento para o construtor *MIMENonMultipart*; o padrão é `us-ascii` se a string contiver apenas pontos de código `ascii` e `utf-8` caso contrário. O parâmetro *_charset* aceita uma string ou uma instância *Charset*.

A menos que o argumento *_charset* seja explicitamente definido como `None`, o objeto MIMEText criado terá um cabeçalho *Content-Type* com um parâmetro `charset` e um cabeçalho *Content-Transfer-Encoding*. Isso significa que uma chamada `set_payload` subsequente não resultará em uma carga útil codificada, mesmo se um charset for passado no comando `set_payload`. Você pode "redefinir" esse comportamento excluindo o cabeçalho `Content-Transfer-Encoding`, após o qual uma chamada `set_payload` codificará automaticamente a nova carga útil (e adicionará um novo cabeçalho *Content-Transfer-Encoding*).

O argumento opcional *policy* tem como padrão *compat32*.

Alterado na versão 3.5: *_charset* também aceita instâncias de *Charset*.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

20.1.11 `email.header`: Internationalized headers

Código-fonte: Lib/email/header.py

This module is part of the legacy (`Compat32`) email API. In the current API encoding and decoding of headers is handled transparently by the dictionary-like API of the *`EmailMessage`* class. In addition to uses in legacy code, this module can be useful in applications that need to completely control the character sets used when encoding headers.

O texto restante nesta seção é a documentação original do módulo.

RFC 2822 is the base standard that describes the format of email messages. It derives from the older **RFC 822** standard which came into widespread use at a time when most email was composed of ASCII characters only. **RFC 2822** is a specification written assuming email contains only 7-bit ASCII characters.

Of course, as email has been deployed worldwide, it has become internationalized, such that language specific character sets can now be used in email messages. The base standard still requires email messages to be transferred using only 7-bit ASCII characters, so a slew of RFCs have been written describing how to encode email containing non-ASCII characters into **RFC 2822**-compliant format. These RFCs include **RFC 2045**, **RFC 2046**, **RFC 2047**, and **RFC 2231**. The *`email`* package supports these standards in its *`email.header`* and *`email.charset`* modules.

If you want to include non-ASCII characters in your email headers, say in the *`Subject`* or *`To`* fields, you should use the *`Header`* class and assign the field in the *`Message`* object to an instance of *`Header`* instead of using a string for the header value. Import the *`Header`* class from the *`email.header`* module. For example:

```
>>> from email.message import Message
>>> from email.header import Header
>>> msg = Message()
>>> h = Header('p\xf6stal', 'iso-8859-1')
>>> msg['Subject'] = h
>>> msg.as_string()
'Subject: =?iso-8859-1?q?p=F6stal?=\n\n'
```

Notice here how we wanted the *`Subject`* field to contain a non-ASCII character? We did this by creating a *`Header`* instance and passing in the character set that the byte string was encoded in. When the subsequent *`Message`* instance was flattened, the *`Subject`* field was properly **RFC 2047** encoded. MIME-aware mail readers would show this header using the embedded ISO-8859-1 character.

Here is the *`Header`* class description:

class `email.header.`**Header**(*s=None*, *charset=None*, *maxlinelen=None*, *header_name=None*, *continuation_ws=' '*, *errors='strict'*)

Create a MIME-compliant header that can contain strings in different character sets.

Optional *s* is the initial header value. If `None` (the default), the initial header value is not set. You can later append to the header with *`append()`* method calls. *s* may be an instance of *`bytes`* or *`str`*, but see the *`append()`* documentation for semantics.

Optional *charset* serves two purposes: it has the same meaning as the *charset* argument to the *`append()`* method. It also sets the default character set for all subsequent *`append()`* calls that omit the *charset* argument. If *charset* is not provided in the constructor (the default), the `us-ascii` character set is used both as *s*'s initial charset and as the default for subsequent *`append()`* calls.

The maximum line length can be specified explicitly via *maxlinelen*. For splitting the first line to a shorter value (to account for the field header which isn't included in *s*, e.g. *`Subject`*) pass in the name of the field in *header_name*. The default *maxlinelen* is 78, and the default value for *header_name* is `None`, meaning it is not taken into account for the first line of a long, split header.

Optional *continuation_ws* must be **RFC 2822**-compliant folding whitespace, and is usually either a space or a hard tab character. This character will be prepended to continuation lines. *continuation_ws* defaults to a single space character.

Optional *errors* is passed straight through to the *`append()`* method.

append(*s*, *charset=None*, *errors='strict'*)

Append the string *s* to the MIME header.

Optional *charset*, if given, should be a `Charset` instance (see `email.charset`) or the name of a character set, which will be converted to a `Charset` instance. A value of `None` (the default) means that the *charset* given in the constructor is used.

s may be an instance of `bytes` or `str`. If it is an instance of `bytes`, then *charset* is the encoding of that byte string, and a `UnicodeError` will be raised if the string cannot be decoded with that character set.

If *s* is an instance of `str`, then *charset* is a hint specifying the character set of the characters in the string.

In either case, when producing an **RFC 2822**-compliant header using **RFC 2047** rules, the string will be encoded using the output codec of the charset. If the string cannot be encoded using the output codec, a UnicodeError will be raised.

Optional *errors* is passed as the errors argument to the decode call if *s* is a byte string.

encode(*splitchars=';, \t'*, *maxlinelen=None*, *linesep='\n'*)

Encode a message header into an RFC-compliant format, possibly wrapping long lines and encapsulating non-ASCII parts in base64 or quoted-printable encodings.

Optional *splitchars* is a string containing characters which should be given extra weight by the splitting algorithm during normal header wrapping. This is in very rough support of **RFC 2822**'s 'higher level syntactic breaks': split points preceded by a splitchar are preferred during line splitting, with the characters preferred in the order in which they appear in the string. Space and tab may be included in the string to indicate whether preference should be given to one over the other as a split point when other split chars do not appear in the line being split. Splitchars does not affect **RFC 2047** encoded lines.

maxlinelen, if given, overrides the instance's value for the maximum line length.

linesep specifies the characters used to separate the lines of the folded header. It defaults to the most useful value for Python application code (`\n`), but `\r\n` can be specified in order to produce headers with RFC-compliant line separators.

Alterado na versão 3.2: Adicionado o argumento *linesep*.

The `Header` class also provides a number of methods to support standard operators and built-in functions.

__str__()

Returns an approximation of the `Header` as a string, using an unlimited line length. All pieces are converted to unicode using the specified encoding and joined together appropriately. Any pieces with a charset of `'unknown-8bit'` are decoded as ASCII using the `'replace'` error handler.

Alterado na versão 3.2: Added handling for the `'unknown-8bit'` charset.

__eq__(*other*)

This method allows you to compare two `Header` instances for equality.

__ne__(*other*)

This method allows you to compare two `Header` instances for inequality.

The `email.header` module also provides the following convenient functions.

email.header.**decode_header**(*header*)

Decode a message header value without converting the character set. The header value is in *header*.

For historical reasons, this function may return either:

1. A list of pairs containing each of the decoded parts of the header, `(decoded_bytes, charset)`, where *decoded_bytes* is always an instance of `bytes`, and *charset* is either:
   - A lower case string containing the name of the character set specified.
   - `None` for non-encoded parts of the header.
2. A list of length 1 containing a pair `(string, None)`, where *string* is always an instance of `str`.

An *email.errors.HeaderParseError* may be raised when certain decoding errors occur (e.g. a base64 decoding exception).

Here are examples:

```
>>> from email.header import decode_header
>>> decode_header('=?iso-8859-1?q?p=F6stal?=')
[(b'p\xf6stal', 'iso-8859-1')]
>>> decode_header('unencoded_string')
[('unencoded_string', None)]
>>> decode_header('bar =?utf-8?B?ZsOzbw==?=')
[(b'bar ', None), (b'f\xc3\xb3o', 'utf-8')]
```

Nota

This function exists for backwards compatibility only. For new code, we recommend using *email.headerregistry.HeaderRegistry*.

email.header.**make_header**(*decoded_seq*, *maxlinelen=None*, *header_name=None*, *continuation_ws=' '*)

Create a *Header* instance from a sequence of pairs as returned by *decode_header()*.

decode_header() takes a header value string and returns a sequence of pairs of the format `(decoded_string, charset)` where *charset* is the name of the character set.

This function takes one of those sequence of pairs and returns a *Header* instance. Optional *maxlinelen*, *header_name*, and *continuation_ws* are as in the *Header* constructor.

Nota

This function exists for backwards compatibility only, and is not recommended for use in new code.

20.1.12 email.charset: Representando conjuntos de caracteres

Código-fonte: Lib/email/charset.py

Este módulo faz parte da API de e-mail legada (`Compat32`). Na nova API, apenas a tabela de apelidos é usada.

O texto restante nesta seção é a documentação original do módulo.

Este módulo fornece uma classe *Charset* para representar conjuntos de caracteres e conversões de conjuntos de caracteres em mensagens de e-mail, bem como um registro de conjuntos de caracteres e vários métodos práticos para manipular esse registro. Instâncias de *Charset* são usadas em vários outros módulos do pacote *email*.

Importa esta classe do módulo *email.charset*.

class email.charset.**Charset**(*input_charset=DEFAULT_CHARSET*)

Mapeia conjuntos de caracteres para suas propriedades de e-mail.

Esta classe fornece informações sobre os requisitos impostos ao e-mail para um conjunto de caracteres específico. Também fornece rotinas práticas para conversão entre conjuntos de caracteres, considerando a disponibilidade dos codecs aplicáveis. Dado um conjunto de caracteres, ela fará o possível para fornecer informações sobre como usá-lo em uma mensagem de e-mail de forma compatível com RFC.

Certos conjuntos de caracteres devem ser codificados com quoted-printable ou base64 quando usados em cabeçalhos ou corpos de e-mail. Certos conjuntos de caracteres devem ser convertidos diretamente e não são permitidos em e-mails.

O *input_charset* opcional é descrito abaixo; ele é sempre convertido para letras minúsculas. Após ser normalizado por apelido, ele também é usado como uma consulta no registro de conjuntos de caracteres para

descobrir a codificação do cabeçalho, a codificação do corpo e o codec de conversão de saída a serem usados para o conjunto de caracteres. Por exemplo, se *input_charset* for `iso-8859-1`, os cabeçalhos e corpos serão codificados usando quoted-printable e nenhum codec de conversão de saída será necessário. Se *input_charset* for `euc-jp`, os cabeçalhos serão codificados em base64, os corpos não serão codificados, mas o texto de saída será convertido do conjunto de caracteres `euc-jp` para o conjunto de caracteres `iso-2022-jp`.

Instâncias *Charset* têm os seguintes atributos de dados:

input_charset

O conjunto de caracteres inicial especificado. Apelidos comuns são convertidos em seus nomes de e-mail *oficiais* (por exemplo, `latin_1` é convertido para `iso-8859-1`). O padrão é `us-ascii` de 7 bits.

header_encoding

Se o conjunto de caracteres precisar ser codificado antes de ser usado em um cabeçalho de e-mail, este atributo será definido como `charset.QP` (para quoted-printable), `charset.BASE64` (para codificação base64) ou `charset.SHORTEST` para a codificação mais curta entre QP ou BASE64. Caso contrário, será `None`.

body_encoding

O mesmo que *header_encoding*, mas descreve a codificação do corpo da mensagem de e-mail, que pode ser diferente da codificação do cabeçalho. `charset.SHORTEST` não é permitido para *body_encoding*.

output_charset

Alguns conjuntos de caracteres precisam ser convertidos antes de serem usados em cabeçalhos ou corpos de e-mail. Se *input_charset* for um deles, este atributo conterá o nome do conjunto de caracteres para o qual a saída será convertida. Caso contrário, será `None`.

input_codec

O nome do codec Python usado para converter o *input_charset* para Unicode. Se nenhum codec de conversão for necessário, este atributo será `None`.

output_codec

O nome do codec Python usado para converter Unicode para o *output_charset*. Se nenhum codec de conversão for necessário, este atributo terá o mesmo valor que o *input_codec*.

Instâncias *Charset* também têm os seguintes métodos:

get_body_encoding()

Retorna a codificação de transferência de conteúdo usada para codificação do corpo.

Esta é a string `quoted-printable` ou `base64`, dependendo da codificação usada, ou é uma função, caso em que você deve chamar a função com um único argumento, sendo o objeto Message codificado. A função deve então definir o próprio cabeçalho *Content-Transfer-Encoding* com o valor apropriado.

Retorna a string `quoted-printable` se *body_encoding* for `QP`, retorna a string `base64` se *body_encoding* for `BASE64` e retorna a string `7bit` caso contrário.

get_output_charset()

Retorna o conjunto de caracteres de saída.

Este é o atributo *output_charset* se não for `None`, caso contrário, é *input_charset*.

header_encode(*string*)

Codifica o cabeçalho com a string *string*.

O tipo de codificação (base64 ou quoted-printable) será baseado no atributo *header_encoding*.

header_encode_lines(*string*, *maxlengths*)

Codifica um cabeçalho de uma *string* convertendo-a primeiro em bytes.

Isso é semelhante a *header_encode()*, exceto que a string é ajustada aos comprimentos máximos de linha, conforme fornecido pelo argumento *maxlengths*, que deve ser um iterador: cada elemento retornado deste iterador fornecerá o próximo comprimento máximo de linha.

body_encode(*string*)

Codifica o corpo com a string *string*.

O tipo de codificação (base64 ou quoted-printable) será baseado no atributo *body_encoding*.

A classe *Charset* também fornece vários métodos para dar suporte a operações padrão e funções embutidas.

__str__()

Retorna *input_charset* como uma string convertida para minúsculas. `__repr__()` é um apelido para `__str__()`.

__eq__(*other*)

Este método permite que você compare duas instâncias *Charset* para verificar a igualdade.

__ne__(*other*)

Este método permite que você compare duas instâncias *Charset* para verificar a desigualdade.

O módulo *email.charset* também fornece as seguintes funções para adicionar novas entradas ao conjunto global de caracteres, apelidos e registros de codec:

email.charset.**add_charset**(*charset*, *header_enc=None*, *body_enc=None*, *output_charset=None*)

Adiciona propriedades de caracteres ao registro global.

charset é o conjunto de caracteres de entrada e deve ser o nome canônico de um conjunto de caracteres.

header_enc e *body_enc* opcionais são `charset.QP` para quoted-printable, `charset.BASE64` para codificação base64, `charset.SHORTEST` para a codificação mais curta entre quoted-printable ou base64, ou `None` para nenhuma codificação. `SHORTEST` é válido apenas para *header_enc*. O padrão é `None` para nenhuma codificação.

output_charset opcional é o conjunto de caracteres que a saída deve conter. As conversões prosseguirão do conjunto de caracteres de entrada para Unicode e para o conjunto de caracteres de saída quando o método `Charset.convert()` for chamado. O padrão é gerar a saída no mesmo conjunto de caracteres da entrada.

Tanto *input_charset* quanto *output_charset* devem ter entradas de codec Unicode no mapeamento de conjunto de caracteres para codec do módulo; use *add_codec()* para adicionar codecs que o módulo não conhece. Consulte a documentação do módulo *codecs* para obter mais informações.

O registro do conjunto de caracteres global é mantido no dicionário global do módulo `CHARSETS`.

email.charset.**add_alias**(*alias*, *canonical*)

Adicione um apelido para o conjunto de caracteres. *alias* é o nome do alias, por exemplo, `latin-1`. *canonical* é o nome canônico do conjunto de caracteres, por exemplo, `iso-8859-1`.

O registro global de apelido de conjunto de caracteres é mantido no dicionário global do módulo `ALIASES`.

email.charset.**add_codec**(*charset*, *codecname*)

Adiciona um codec que mapeia caracteres no conjunto de caracteres fornecido para e a partir do Unicode.

charset é o nome canônico de um conjunto de caracteres. *codecname* é o nome de um codec Python, conforme apropriado para o segundo argumento do método *encode()* do *str*.

20.1.13 email.encoders: Codificadores

Código-fonte: Lib/email/encoders.py

Este módulo faz parte da API legada de e-mail (`Compat32`). Na nova API, a funcionalidade é fornecida pelo parâmetro *cte* do método *set_content()*.

Este módulo está descontinuado no Python 3. As funções fornecidas aqui não devem ser chamadas explicitamente, pois a classe *MIMEText* define o tipo de conteúdo e o cabeçalho CTE usando os valores *_subtype* e *_charset* passados durante a instanciação dessa classe.

O texto restante nesta seção é a documentação original do módulo.

Ao criar objetos *Message* do zero, você frequentemente precisa codificar as cargas úteis para transporte por meio de servidores de e-mail compatíveis. Isso é especialmente verdadeiro para mensagens do tipo *image/** e *text/** contendo dados binários.

O pacote *email* fornece alguns codificadores convenientes em seu módulo *encoders*. Esses codificadores são realmente usados pelos construtores de classe *MIMEAudio* e *MIMEImage* para fornecer codificações padrão. Todas as funções de codificador recebem exatamente um argumento, o objeto de mensagem a ser codificado. Eles geralmente extraem a carga útil, codificam-na e redefinem a carga útil para esse valor recém-codificado. Eles também devem definir o cabeçalho *Content-Transfer-Encoding* conforme apropriado.

Observe que essas funções não são significativas para uma mensagem multiparte. Elas devem ser aplicadas a subpartes individuais, e levantarão um *TypeError* se for passada uma mensagem cujo tipo seja multiparte.

Aqui estão as funções de codificação fornecidas:

email.encoders.**encode_quopri**(*msg*)

Codifica a carga útil em formato quoted-printable e define o cabeçalho *Content-Transfer-Encoding* como `quoted-printable`[1]. Esta é uma boa codificação para usar quando a maior parte da sua carga útil é de dados imprimíveis normais, mas contém alguns caracteres não imprimíveis.

email.encoders.**encode_base64**(*msg*)

Codifica a carga útil em formato base64 e define o cabeçalho *Content-Transfer-Encoding* para `base64`. Esta é uma boa codificação para usar quando a maior parte da sua carga útil é de dados não imprimíveis, pois é um formato mais compacto do que quoted-printable. A desvantagem da codificação base64 é que ela torna o texto não legível por humanos.

email.encoders.**encode_7or8bit**(*msg*)

Na verdade, isso não modifica a carga útil da mensagem, mas define o cabeçalho *Content-Transfer-Encoding* para `7bit` ou `8bit`, conforme apropriado, com base nos dados da carga útil.

email.encoders.**encode_noop**(*msg*)

Isso não faz nada; nem mesmo define o cabeçalho *Content-Transfer-Encoding*.

20.1.14 email.utils: Utilitários diversos

Código-fonte: Lib/email/utils.py

Existem alguns utilitários úteis fornecidos no *email.utils* module:

email.utils.**localtime**(*dt=None*)

Retorna a hora local como um objeto datetime consciente. Se chamado sem argumentos, retorna a hora atual. Caso contrário, o argumento *dt* deve ser uma instância *datetime* e é convertido para o fuso horário local de acordo com o banco de dados de fuso horário do sistema. Se *dt* for ingênuo (ou seja, `dt.tzinfo` for `None`), será presumido como estando no horário local.

Adicionado na versão 3.3.

Deprecated since version 3.12, removed in version 3.14: O parâmetro *isdst*.

email.utils.**make_msgid**(*idstring=None*, *domain=None*)

Retorna uma string adequada para um cabeçalho *Message-ID* compatível com **RFC 2822**. O *idstring* opcional, se fornecido, é uma string usada para fortalecer a exclusividade do ID da mensagem. O *domain* Opcional, se dado, fornece a porção do msgid após o '@'. O padrão é o nome do host local. Normalmente, não é necessário substituir esse padrão, mas pode ser útil em alguns casos, como um sistema distribuído de construção que usa um nome de domínio consistente em vários hosts.

Alterado na versão 3.2: Adicionada a palavra-chave *domain*.

[1] Observe que a codificação com *encode_quopri()* também codifica todos os caracteres de tabulação e espaço nos dados.

As funções restantes fazem parte da API de e-mail herdada (`Compat32`). Não há necessidade de usá-las diretamente com a nova API, pois a análise e a formatação fornecidas são feitas automaticamente pelo mecanismo de análise de cabeçalhos da nova API.

`email.utils.`**`quote`**(*str*)

Devolve uma nova string com barras invertidas em *str* substituídas por duas barras invertidas e aspas duplas substituídas por aspas duplas invertidas.

`email.utils.`**`unquote`**(*str*)

Retorna uma nova string que é uma versão sem aspas de *str*. Se *str* terminar e começar com aspas duplas, elas serão removidas. Da mesma forma, se *str* termina e começa com colchetes angulares, eles são removidos.

`email.utils.`**`parseaddr`**(*address*, *, *strict=True*)

Analisa o endereço – que deve ser o valor de algum campo contendo endereço, como *To* ou *Cc* – em suas partes constituinte *realname* e *email address*. Retorna uma tupla daquela informação, a menos que a análise falhe, caso em que uma tupla de 2 de `('', '')` é retornada.

Se *strict* for verdadeiro, usa um analisador sintático estrito que rejeite entradas malformadas.

Alterado na versão 3.13: Adiciona o parâmetro opcional *strict* e rejeita entradas malformadas por padrão.

`email.utils.`**`formataddr`**(*pair*, *charset='utf-8'*)

O inverso de *parseaddr()*, isto leva uma tupla de 2 do forma `(realname, email_address)` e retorna o valor de string adequado para um cabeçalho *To* ou *Cc* . Se o primeiro elemento de *pair* for falso, o segundo elemento será retornado sem modificações.

O *charset* opcional é o conjunto de caracteres que será usado na codificação **RFC 2047** do `realname` se o `realname` contiver caracteres não-ASCII. Pode ser uma instância de *str* ou a *Charset*. O padrão é `utf-8`.

Alterado na versão 3.3: Adicionada a opção *charset*.

`email.utils.`**`getaddresses`**(*fieldvalues*, *, *strict=True*)

Este método retorna uma lista de tuplas 2 do formulário retornado por `parseaddr()`. *fieldvalues* é uma sequência de valores do campo de cabeçalho que pode ser retornada por *Message.get_all*.

Se *strict* for verdadeiro, usa um analisador sintático estrito que rejeite entradas malformadas.

Aqui está um exemplo simples que recebe todos os destinatários de uma mensagem:

```
from email.utils import getaddresses

tos = msg.get_all('to', [])
ccs = msg.get_all('cc', [])
resent_tos = msg.get_all('resent-to', [])
resent_ccs = msg.get_all('resent-cc', [])
all_recipients = getaddresses(tos + ccs + resent_tos + resent_ccs)
```

Alterado na versão 3.13: Adiciona o parâmetro opcional *strict* e rejeita entradas malformadas por padrão.

`email.utils.`**`parsedate`**(*date*)

Tenta analisar uma data de acordo com as regras em **RFC 2822**. no entanto, alguns mailers não seguem esse formato conforme especificado, portanto *parsedate()* tenta adivinhar corretamente em tais casos. *date* é uma string contendo uma data **RFC 2822**, como `"Mon, 20 Nov 1995 19:12:08 -0500"`. Se conseguir analisar a data, *parsedate()* retorna uma 9-tupla que pode ser passada diretamente para *time.mktime()*; caso contrário, `None` será retornado. Observe que os índices 6, 7 e 8 da tupla de resultados não são utilizáveis.

`email.utils.`**`parsedate_tz`**(*date*)

Executa a mesma função que *parsedate()*, mas retorna `None` ou uma tupla de 10; os primeiros 9 elementos formam uma tupla que pode ser passada diretamente para *time.mktime()*, e o décimo é o deslocamento do fuso horário da data do UTC (que é o termo oficial para o horário de Greenwich)[1]. Se a string de entrada não

[1] Observa que o sinal do deslocamento de fuso horário é o oposto do sinal da variável `time.timezone` para o mesmo fuso horário; a última variável segue o padrão POSIX enquanto este módulo segue **RFC 2822**.

tem fuso horário, o último elemento da tupla retornado é `0`, que representa UTC. Observe que os índices 6, 7 e 8 da tupla de resultado não podem ser usados.

`email.utils.`**`parsedate_to_datetime`**(*date*)

O inverso de *format_datetime()*. Desempenha a mesma função que *parsedate()*, mas em caso de sucesso retorna um *datetime*; caso contrário, `ValueError` é levantada se *date* contiver um valor inválido, como uma hora maior que 23 ou uma diferença de fuso horário não entre -24 e 24 horas. Se a data de entrada tem um fuso horário de `-0000`, o `datetime` será um `datetime` ingênuo, e se a data estiver em conformidade com os RFCs representará um horário em UTC, mas sem indicação do fuso horário de origem real da mensagem de onde vem a data. Se a data de entrada tiver qualquer outro deslocamento de fuso horário válido, o `datetime` será um `datetime` consciente com o correspondente a *timezone tzinfo*.

Adicionado na versão 3.3.

`email.utils.`**`mktime_tz`**(*tuple*)

Transforma uma tupla de 10 conforme retornado por *parsedate_tz()* em um timestamp UTC (segundos desde a Era Unix). Se o item de fuso horário na tupla for `None`, considera a hora local.

`email.utils.`**`formatdate`**(*timeval=None*, *localtime=False*, *usegmt=False*)

Retorna uma string de data conforme **RFC 2822**. Por exemplo:

```
Fri, 09 Nov 2001 01:08:47 -0000
```

O *timeval* opcional, se fornecido, é um valor de tempo de ponto flutuante, conforme aceito por *time.gmtime()* e *time.localtime()*, caso contrário, o tempo atual é usado.

Há um sinalizador opcional *localtime*, que, quando é `True`, interpreta *timeval* e retorna uma data relativa ao fuso horário local em vez do UTC, levando em consideração o horário de verão. O padrão é `False`, o que significa que o UTC é usado.

O *usegmt* opcional é um sinalizador que quando `True`, produz uma string de data com o fuso horário como uma string ascii `GMT`, ao invés de um numérico `-0000`. Isso é necessário para alguns protocolos (como HTTP). Isso se aplica apenas quando *localtime* for `False`. O padrão é `False`.

`email.utils.`**`format_datetime`**(*dt*, *usegmt=False*)

Como `formatdate`, mas a entrada é uma instância de *datetime*. Se for uma data e hora ingênua, será considerado "UTC sem informações sobre o fuso horário de origem" e o convencional `-0000` será usado para o fuso horário. Se for um `datetime` ciente, então o deslocamento de fuso horário numérico é usado. Se for um fuso horário ciente com deslocamento zero, então *usegmt* pode ser definido como `True`, caso em que a string `GMT` é usada em vez do deslocamento numérico do fuso horário. Isso fornece uma maneira de gerar cabeçalhos de data HTTP em conformidade com os padrões.

Adicionado na versão 3.3.

`email.utils.`**`decode_rfc2231`**(*s*)

Decodifica a string *s* de acordo com **RFC 2231**.

`email.utils.`**`encode_rfc2231`**(*s*, *charset=None*, *language=None*)

Codifica a string *s* de acordo com **RFC 2231**. *charset* e *language* opcionais, se fornecido, são o nome do conjunto de caracteres e o nome do idioma a ser usado. Se nenhum deles for fornecido, *s* é retornado como está. Se *charset* for fornecido, mas *language* não, a string será codificada usando a string vazia para *language*.

`email.utils.`**`collapse_rfc2231_value`**(*value*, *errors='replace'*, *fallback_charset='us-ascii'*)

Quando um parâmetro de cabeçalho é codificado no formato **RFC 2231**, *Message.get_param* pode retornar uma tupla de 3 contendo o conjunto de caracteres, idioma e valor. *collapse_rfc2231_value()* transforma isso em uma string Unicode. *errors* opcionais são passados para o argumento *errors* do método *encode()* de *str*; o padrão é `'replace'`. *fallback_charset* opcional especifica o conjunto de caracteres a ser usado se aquele no cabeçalho **RFC 2231** não for conhecido pelo Python; o padrão é `'us-ascii'`.

Por conveniência, se *value* passado para *collapse_rfc2231_value()* não for uma tupla, deve ser uma string e é retornado sem aspas.

email.utils.**decode_params**(*params*)

Decodifica a lista de parâmetros de acordo com **RFC 2231**. *params* é uma sequência de 2 tuplas contendo elementos do formulário `(content-type, string-value)`.

20.1.15 `email.iterators`: Iteradores

Código-fonte: Lib/email/iterators.py

A iteração sobre uma árvore de objetos de mensagem é bastante fácil com o método `Message.walk`. O módulo `email.iterators` fornece algumas iterações úteis de nível mais alto sobre as árvores de objetos de mensagens.

email.iterators.**body_line_iterator**(*msg*, *decode=False*)

Isso itera sobre todas as cargas úteis em todas as subpartes de *msg*, retornando as cargas úteis das strings de linhas por linha. Ele pula todos os cabeçalhos da subparte e pula qualquer subparte com uma carga útil que não seja uma string Python. Isso é um pouco equivalente à leitura da representação de texto simples da mensagem de um arquivo usando `readline()`, pulando todos os cabeçalhos intermediários.

decode opcional é passado por meio do `Message.get_payload`.

email.iterators.**typed_subpart_iterator**(*msg*, *maintype='text'*, *subtype=None*)

Isso itera sobre todas as subpartes de *msg*, retornando apenas as subpartes que correspondem ao tipo MIME especificado por *maintype* e *subtype*.

Observe que *subtipo* é opcional; se omitido, a correspondência de tipo MIME da subparte é feita apenas com o tipo principal. *maintype* também é opcional; o padrão é `text`.

Assim, por padrão `typed_subpart_iterator()` retorna cada subparte que possui um tipo MIME de `text/*`.

A seguinte função foi adicionada como uma ferramenta de depuração útil. *Não* deve ser considerado parte da interface pública suportada para o pacote.

email.iterators.**_structure**(*msg*, *fp=None*, *level=0*, *include_default=False*)

Imprime uma representação recuada dos tipos de conteúdo da estrutura do objeto de mensagem. Por exemplo:

```
>>> msg = email.message_from_file(algum_arquivo)
>>> _structure(msg)
multipart/mixed
    text/plain
    text/plain
    multipart/digest
        message/rfc822
            text/plain
        message/rfc822
            text/plain
        message/rfc822
            text/plain
        message/rfc822
            text/plain
        message/rfc822
            text/plain
    text/plain
```

O *fp* opcional é um objeto arquivo ou similar para o qual deve-se imprimir a saída. Deve ser adequado para a função Python `print()`. *level* usado internamente. *include_default*, se verdadeiro, também imprime o tipo padrão.

Ver também

Módulo *smtplib*
Cliente SMTP (Protocolo Simples de Envio de E-mail)

Módulo *poplib*
Cliente POP (Post Office Protocol)

Módulo *imaplib*
Cliente IMAP (Internet Message Access Protocol)

Módulo *mailbox*
Ferramentas para criar, ler, e gerenciar coleções de mensagem em disco usando vários formatos padrão.

20.2 `json` — Codificador e decodificador JSON

Código-fonte: Lib/json/__init__.py

JSON (JavaScript Object Notation), especificado pelo **RFC 7159** (que tornou o **RFC 4627** obsoleta) e pelo ECMA-404, é um formato leve de troca de dados inspirado pela sintaxe de objeto JavaScript (embora não seja um subconjunto estrito de JavaScript[1]).

Nota

O termo "objeto" no contexto do processamento JSON em Python pode ser ambíguo. Todos os valores em Python são objetos. Em JSON, um objeto se refere a qualquer dado entre chaves, semelhante a um dicionário Python.

Aviso

Tenha cuidado quando estiver analisando dados JSON de fontes não-confiáveis. Uma string JSON maliciosa pode fazer o decodificador consumir recursos consideráveis de CPU e memória. É recomendado limitar o tamanho do dado a ser analisado.

Este módulo expõe uma API familiar para pessoas usuárias dos módulos *marshal* e *pickle* da biblioteca padrão.

Codificação de hierarquias básicas de objetos Python:

```
>>> import json
>>> json.dumps(['foo', {'bar': ('baz', None, 1.0, 2)}])
'["foo", {"bar": ["baz", null, 1.0, 2]}]'
>>> print(json.dumps("\"foo\bar"))
"\"foo\bar"
>>> print(json.dumps('\u1234'))
"\u1234"
>>> print(json.dumps('\\'))
"\\"
>>> print(json.dumps({"c": 0, "b": 0, "a": 0}, sort_keys=True))
{"a": 0, "b": 0, "c": 0}
>>> from io import StringIO
>>> io = StringIO()
>>> json.dump(['streaming API'], io)
>>> io.getvalue()
'["streaming API"]'
```

[1] Como apresentado na errata para RFC 7159, JSON permite os caracteres literais U+2028 (SEPARADOR DE LINHA) e U+2029 (SEPARADOR DE PARÁGRAFO) em strings, enquanto que JavaScript (ECMAScript Edition 5.1) não.

Codificação compacta:

```
>>> import json
>>> json.dumps([1, 2, 3, {'4': 5, '6': 7}], separators=(',', ':'))
'[1,2,3,{"4":5,"6":7}]'
```

Saída bonita:

```
>>> import json
>>> print(json.dumps({'6': 7, '4': 5}, sort_keys=True, indent=4))
{
    "4": 5,
    "6": 7
}
```

Personalizando em codificação de objeto JSON:

```
>>> import json
>>> def custom_json(obj):
...     if isinstance(obj, complex):
...         return {'__complex__': True, 'real': obj.real, 'imag': obj.imag}
...     raise TypeError(f'Cannot serialize object of {type(obj)}')
...
>>> json.dumps(1 + 2j, default=custom_json)
'{"__complex__": true, "real": 1.0, "imag": 2.0}'
```

Decodificando JSON:

```
>>> import json
>>> json.loads('["foo", {"bar":["baz", null, 1.0, 2]}]')
['foo', {'bar': ['baz', None, 1.0, 2]}]
>>> json.loads('"\\"foo\\bar"')
'"foo\x08ar'
>>> from io import StringIO
>>> io = StringIO('["streaming API"]')
>>> json.load(io)
['streaming API']
```

Personalizando em decodificação de objeto JSON:

```
>>> import json
>>> def as_complex(dct):
...     if '__complex__' in dct:
...         return complex(dct['real'], dct['imag'])
...     return dct
...
>>> json.loads('{"__complex__": true, "real": 1, "imag": 2}',
...     object_hook=as_complex)
(1+2j)
>>> import decimal
>>> json.loads('1.1', parse_float=decimal.Decimal)
Decimal('1.1')
```

Estendendo *JSONEncoder*:

```
>>> import json
>>> class ComplexEncoder(json.JSONEncoder):
...     def default(self, obj):
...         if isinstance(obj, complex):
```

(continua na próxima página)

(continuação da página anterior)

```
...             return [obj.real, obj.imag]
...         # Let the base class default method raise the TypeError
...         return super().default(obj)
...
>>> json.dumps(2 + 1j, cls=ComplexEncoder)
'[2.0, 1.0]'
>>> ComplexEncoder().encode(2 + 1j)
'[2.0, 1.0]'
>>> list(ComplexEncoder().iterencode(2 + 1j))
['[2.0', ', 1.0', ']']
```

Usando *json* para validar a partir do console e exibir formatado:

```
$ echo '{"json":"obj"}' | python -m json
{
    "json": "obj"
}
$ echo '{1.2:3.4}' | python -m json
Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
```

Veja *Interface de linha de comando* para a documentação detalhada.

Nota

JSON é um subconjunto do YAML 1.2. O JSON gerado pelas definições padrões desse módulo (particularmente, o valor padrão dos *separadores*) é também um subconjunto do YAML 1.0 e 1.1. Esse módulo pode, portanto, também ser usado como serializador YAML.

Nota

O codificador e o decodificador do módulo preservam a ordem de entrada e saída por padrão. A ordem só é perdida se os contêineres subjacentes estão desordenados.

20.2.1 Uso básico

`json.`**`dump`**(*obj*, *fp*, ***, *skipkeys=False*, *ensure_ascii=True*, *check_circular=True*, *allow_nan=True*, *cls=None*, *indent=None*, *separators=None*, *default=None*, *sort_keys=False*, ***kw*)

Serializa um *obj* como um stream JSON formatado para *fp* (um *objeto arquivo ou similar* com suporte a `.write()`) usando essa *tabela de conversão de Python para JSON*.

Nota

Diferente de *pickle* e *marshal*, JSON não é um protocolo com datagrama, assim tentar serializar múltiplos objetos com chamadas repetidas para *dump()* usando o mesmo *fp* resultará em um arquivo JSON inválido.

Parâmetros

- **obj** (*object*) – O objeto Python a ser serializado.
- **fp** (*file-like object*) – O objeto arquivo ou similar *obj* para o qual será serializado. O módulo `json` sempre produz objetos *str*, e não objetos *bytes*. Dessa forma, `fp.write()` precisa ter suporte para entradas *str*.

- **skipkeys** (bool) – Se True, as chaves que não forem de um tipo básico (*str*, *int*, *float*, *bool*, None) serão ignoradas ao invés de levantar uma exceção *TypeError*. Padrão False.
- **ensure_ascii** (bool) – Se True (o padrão), será garantido que a saída terá todos os caracteres não ASCII que chegam escapados. Se False, esses caracteres serão emitidos como estão.
- **check_circular** (bool) – Se False, a verificação de referência circular for de tipos contêineres é ignorada e uma referência circular resultará em uma exceção *RecursionError* (ou pior). Padrão True.
- **allow_nan** (bool) – Se False, serialização de valores *float* fora de intervalo (nan, inf, -inf) vai resultar em um *ValueError* em estrita conformidade com a especificação. Se True (o padrão), seus equivalentes JavaScript (NaN, Infinity, -Infinity) serão usados.
- **cls** (a *JSONEncoder* subclass) – Se definido, um codificador JSON personalizado com o método *default()* substituído, para serializar em tipos de dados personalizados. Se None (o padrão), JSONEncoder é usado.
- **indent** (int | str | *None*) – Se for um inteiro positivo ou uma string, os elementos da vetores JSON e os membros do objeto serão impressos de forma bonita com esse nível de indentação. Um inteiro positivo recua essa quantidade de espaços por nível; uma string (como "\t") é usada para indentar cada nível. Se for zero, negativo ou "" (a string vazia), somente novas linhas serão inseridas. Se for None (o padrão), a representação mais compacta será usada.
- **separators** (tuple | *None*) – Uma tupla dupla: (item_separator, key_separator). Se None (o padrão), *separators* assume como padrão (', ', ': ') se *indent* for None, e (',', ': ') caso contrário. Para o JSON mais compacto, especifique (',', ':') para eliminar espaços em branco.
- **default** (*callable* | None) – Uma função que é chamada para objetos que não podem ser serializados de outra forma. Ela deve retornar uma versão JSON codificável do objeto ou gerar um *TypeError*. Se None (o padrão), TypeError é levantada.
- **sort_keys** (bool) – Se True, os dicionários serão emitidos classificados por chave. Padrão False.

Alterado na versão 3.2: Permite strings para *indent*, além de inteiros.

Alterado na versão 3.4: Usa (',', ': ') como padrão se *indent* não for None.

Alterado na versão 3.6: Todos os parâmetros opcionais agora são *somente-nomeados*.

json.**dumps**(*obj*, *, *skipkeys=False*, *ensure_ascii=True*, *check_circular=True*, *allow_nan=True*, *cls=None*, *indent=None*, *separators=None*, *default=None*, *sort_keys=False*, **kw*)

Serializa *obj* para uma *str* com formato JSON usando essa *tabela de conversão*. Os argumentos têm o mesmo significado que na função *dump()*.

Nota

Chaves nos pares chave/valor de JSON são sempre do tipo *str*. Quando um dicionário é convertido para JSON, todas as chaves são convertidas para strings. Como resultado disso, se um dicionário é convertido para JSON e depois de volta para um dicionário, o dicionário pode não ser igual ao original. Isto é, loads(dumps(x)) != x se x tem chaves não strings.

json.**load**(*fp*, *, *cls=None*, *object_hook=None*, *parse_float=None*, *parse_int=None*, *parse_constant=None*, *object_pairs_hook=None*, **kw*)

Desserializa *fp* para um objeto Python usando a *tabela de conversão JSON para Python*.

Parâmetros

- **fp** (*file-like object*) – Um *arquivo texto* ou *arquivo binário* com suporte a `.read()` contendo o documento JSON a ser desserializado.
- **cls** (a *JSONDecoder* subclass) – Se definido, um decodificador JSON personalizado. Argumentos nomeados adicionais para `load()` serão passados para o construtor de *cls*. Se `None` (o padrão), `JSONDecoder` é usado.
- **object_hook** (*callable* | None) – Se definido, uma função que é chamada com o resultado de qualquer literal de objeto JSON decodificado (um *dict*). O valor de retorno desta função será usado em vez do *dict*. Este recurso pode ser usado para implementar decodificadores personalizados, por exemplo dica de classes JSON-RPC. O padrão é `None`.
- **object_pairs_hook** (*callable* | None) – Se definido, uma função que é chamada com o resultado de qualquer literal de objeto JSON decodificado com uma lista ordenada de pares. O valor de retorno desta função será usado em vez de *dict*. Este recurso pode ser usado para implementar decodificadores personalizados. Se *object_hook* também for definido, *object_pairs_hook* terá prioridade. O padrão é `None`.
- **parse_float** (*callable* | None) – Se definido, uma função que é chamada com a string de cada ponto flutuante JSON a ser decodificada. Se `None` (o padrão), é equivalente a `float(num_str)`. Isso pode ser usado para analisar pontos flutuantes JSON em tipos de dados personalizados, por exemplo *decimal.Decimal*.
- **parse_int** (*callable* | None) – Se definido, uma função que é chamada com a string de cada int JSON a ser decodificado. Se `None` (o padrão), é equivalente a `int(num_str)`. Isso pode ser usado para analisar inteiros JSON em tipos de dados personalizados, por exemplo *float*.
- **parse_constant** (*callable* | None) – Se definido, uma função que é chamada com uma das seguintes strings: `'-Infinity'`, `'Infinity'` ou `'NaN'`. Isso pode ser usado para levantar uma exceção se números JSON inválidos forem encontrados. Padrão `None`.

Levanta

- ***JSONDecodeError*** – Quando os dados a serem desserializados não forem um documento JSON válido.
- ***UnicodeDecodeError*** – Quando os dados que estão sendo desserializados não contêm dados codificados em UTF-8, UTF-16 ou UTF-32.

Alterado na versão 3.1:

- Adicionado o parâmetro opcional *object_pairs_hook*.
- *parse_constant* não é mais chamada para 'null', 'true', 'false'.

Alterado na versão 3.6:

- Todos os parâmetros opcionais agora são *somente-nomeados*.
- *fp* agora pode ser um *arquivo binário*. A entrada deve estar codificada como UTF-8, UTF-16 ou UTF-32.

Alterado na versão 3.11: O *parse_int* padrão para *int()* agora limita o tamanho máximo da string de inteiro via *limitação de comprimento de string na conversão para inteiro* do interpretador para ajudar a evitar ataques por negação de serviço.

`json.`**loads**(*s*, ***, *cls=None*, *object_hook=None*, *parse_float=None*, *parse_int=None*, *parse_constant=None*, *object_pairs_hook=None*, ***kw*)

Idêntico a *load()*, mas em vez de um objeto arquivo ou similar, desserializa *s* (uma instância de *str*, *bytes* ou *bytearray* contendo um documento JSON) para um objeto Python usando essa *tabela de conversão*.

Alterado na versão 3.6: *s* agora pode ser um do tipo *bytes* ou *bytearray*. A entrada deve estar codificado como UTF-8, UTF-16 ou UTF-32.

Alterado na versão 3.9: O argumento nomeado *encoding* foi removido.

20.2.2 Codificadores e decodificadores

class json.**JSONDecoder**(*, *object_hook=None*, *parse_float=None*, *parse_int=None*, *parse_constant=None*, *strict=True*, *object_pairs_hook=None*)

Decodificador JSON simples.

Executa as seguintes traduções na decodificação por padrão:

JSON	Python
objeto	dicionário
array	lista
string	str
número (inteiro)	int
número (real)	float
verdadeiro	True
falso	False
nulo	None

Ele também entende `NaN`, `Infinity` e `-Infinity` como seus valores `float` correspondentes, que estão fora da especificação JSON.

object_hook é uma função opcional que será chamada com o resultado de cada objeto JSON decodificado e seu valor de retorno será usado no lugar do dado *dict*. Isso pode ser usado para fornecer desserializações personalizadas (por exemplo, para oferecer suporte a dicas de classe JSON-RPC)

object_pairs_hook é uma função opcional que será chamada com o resultado de todo objeto JSON decodificado com uma lista ordenada de pares. O valor de retorno de *object_pairs_hook* será usado no lugar do *dict*. Este recurso pode ser usado para implementar decodificadores personalizados. Se *object_pairs_hook* também for definido, o *object_pairs_hook* terá prioridade.

Alterado na versão 3.1: Adicionado suporte para *object_pairs_hook*.

parse_float é uma função opcional que será chamada com a string de cada ponto flutuante JSON para ser decodificado. Por padrão, equivale a `float(num_str)`. Pode ser usado para qualquer outro tipo de dado ou analisador de pontos flutuante JSON (por exemplo, *decimal.Decimal*).

parse_int é uma função opcional que será chamada com a string de cada inteiro JSON para ser decodificado. Por padrão, equivale a `int(num_str)`. Pode ser usado para qualquer outro tipo de dado ou analisador de inteiros JSON (por exemplo, *float*).

parse_constant é uma função opcional que será chamada para cada um das seguintes strings: `'-Infinity'`, `'Infinity'`, `'NaN'`. Isso pode ser usado para levantar uma exceção se forem encontrados números JSON inválidos.

Se *strict* for falso (`True` é o padrão), os caracteres de controle serão permitidos dentro das strings. Os caracteres de controle neste contexto são aqueles com códigos de caracteres no intervalo 0–31, incluindo `'\t'` (tab), `'\n'`, `'\r'` e `'\0'`.

Se os dados a serem desserializados não forem um documento JSON válido, será levantada uma exceção *JSONDecodeError*.

Alterado na versão 3.6: Todos os parâmetros agora são *somente-nomeado*.

decode(*s*)

Retorna a representação Python de *s* (uma instância *str* contendo um documento JSON).

Uma exceção *JSONDecodeError* será levantada se o documento JSON fornecido não for válido.

raw_decode(*s*)

Decodifica um documento JSON a partir de *s* (uma *str* iniciando com um documento JSON) e retornando uma tupla de 2 elementos, a representação Python e o índice em *s* onde o documento finaliza.

Isso pode ser usado para decodificar um documento JSON a partir de uma string que possa ter dados extras ao final.

class json.**JSONEncoder**(*, *skipkeys=False*, *ensure_ascii=True*, *check_circular=True*, *allow_nan=True*, *sort_keys=False*, *indent=None*, *separators=None*, *default=None*)

Codificador JSON extensível para estruturas de dados Python.

Por padrão, possui suporte para os seguintes objetos e tipos:

Python	JSON
dicionário	objeto
list, tuple	array
str	string
int, float e Enums derivados de int e float	number
True	verdadeiro
False	falso
None	nulo

Alterado na versão 3.4: Adicionado suporte para classes Enum derivadas de int e float.

Para estender isso para reconhecer outros objetos, crie uma subclasse e implemente o método *default()* com outro método que retorne um objeto serializável para `o` se possível, caso contrário deveria chamar a implementação da superclasse (para levantar *TypeError*).

Se *skipkeys* é falso (o padrão), então sluma *TypeError* será levantada ao tentar codificar as chaves que não são *str*, *int*, *float*, *bool* ou `None`. Se *skipkeys* é verdadeiro, esses itens são simplesmente pulados.

Se *ensure_ascii* for verdadeiro (o padrão), será garantido que a saída terá todos os caracteres não ASCII que chegam escapados. Se *ensure_ascii* for falso, a saída desses caracteres ficará como está.

Se *check_circular* é verdadeiro (o padrão), então listas, dicionários e objetos codificados personalizados serão verificados por referências circulares durante a codificação para prevenir uma recursão infinita (que iria causar uma *RecursionError*). Caso contrário, nenhuma verificação será feita.

Se *allow_nan* for verdadeiro (o padrão), então `NaN`, `Infinity`, e `-Infinity` serão codificados como tal. Esse comportamento não é compatível com a especificação do JSON, mas é consistente com a maioria dos codificadores e decodificadores baseados em JavaScript. Caso contrário, será levantada uma *ValueError* para tais pontos flutuantes.

Se *sort_keys* for verdadeiro (padrão: `False`), então a saída dos dicionários serão ordenados pela chave; isto é útil para testes de regressão para certificar-se que as serializações de JSON possam ser comparadas com uma base no dia a dia.

Se *indent* for um inteiro não negativo ou uma string, então elementos de um vetor JSON e membros de objetos terão uma saída formatada com este nível de indentação. Um nível de indentação 0, negativo ou `""` apenas colocará novas linhas. `None` (o padrão) seleciona a representação mais compacta. Usando um inteiro positivo a indentação terá alguns espaços por nível. Se *indent* for uma string (como `"\t"`), essa string será usada para indentar cada nível.

Alterado na versão 3.2: Permite strings para *indent*, além de inteiros.

Se especificado, *separators* deve ser uma tupla `(item_separator, key_separator)`. O padrão é `(', ', ': ')` se *indent* for `None` e `(',', ': ')` caso contrário. Para pegar representação JSON mais compacta, você deve especificar `(',', ':')` para eliminar espaços em branco.

Alterado na versão 3.4: Usa `(',', ': ')` como padrão se *indent* não for `None`.

Se especificado, *default* deve ser uma função para ser chamada para objetos que não podem ser serializados de outra forma. Deve retornar uma versão codificável JSON do objeto ou levantar uma exceção *TypeError*. Se não for especificada, *TypeError* é levantada.

Alterado na versão 3.6: Todos os parâmetros agora são *somente-nomeado*.

default(*o*)

Implemente este método em uma subclasse que retorna um objeto serializável para *o* ou que chame a implementação base (para levantar uma *TypeError*).

Por exemplo, para suporte a iteradores arbitrários, você poderia implementar *default()* dessa forma:

```
def default(self, o):
   try:
       iterable = iter(o)
   except TypeError:
       pass
   else:
       return list(iterable)
   # Faz o método padrão de classe base levantar a TypeError
   return super().default(o)
```

encode(*o*)

Retorna uma string representando um JSON a partir da estrutura de dados Python, *o*. Por exemplo:

```
>>> json.JSONEncoder().encode({"foo": ["bar", "baz"]})
'{"foo": ["bar", "baz"]}'
```

iterencode(*o*)

Codifica o objeto dado, *o*, e produz cada representação em string assim que disponível. Por exemplo:

```
for chunk in json.JSONEncoder().iterencode(bigobject):
    mysocket.write(chunk)
```

20.2.3 Exceções

exception json.**JSONDecodeError**(*msg*, *doc*, *pos*)

Subclasse de *ValueError* com os seguintes atributos adicionais:

msg

A mensagem de erro não formatada.

doc

O documento JSON sendo analisado.

pos

O índice inicial de *doc* em que a análise falhou.

lineno

A linha correspondente a *pos*.

colno

A coluna correspondente a *pos*.

Adicionado na versão 3.5.

20.2.4 Conformidade e interoperabilidade entre padrões

O formato JSON é especificado pela **RFC 7159** e por ECMA-404. Esta seção detalha o nível de conformidade deste módulo com a RFC. Para simplificar, as subclasses *JSONEncoder* e *JSONDecoder*, e outros parâmetros além daqueles explicitamente mencionados, não são considerados.

Este módulo não está em conformidade com a RFC de forma estrita, implementando algumas extensões que são JavaScript válidas, mas não JSON válido. Em particular:

- Os valores de números infinitos e NaN são aceitos e produzidos;

- Nomes repetidos em um objeto são aceitos e apenas o valor do último par nome-valor é usado.

Uma vez que a RFC permite que os analisadores compatíveis com a RFC aceitem textos de entrada que não sejam compatíveis com a RFC, o desserializador deste módulo é tecnicamente compatível com a RFC nas configurações padrão.

Codificações de caracteres

A RFC requer que JSON seja representado usando UTF-8, UTF-16 ou UTF-32, com UTF-8 sendo o padrão recomendado para interoperabilidade máxima.

Conforme permitido, embora não exigido, pela RFC, o serializador deste módulo define *ensure_ascii=True* por padrão, escapando a saída para que as strings resultantes contenham apenas caracteres ASCII.

Além do parâmetro *ensure_ascii*, este módulo é definido estritamente em termos de conversão entre objetos Python e *strings Unicode* e, portanto, não aborda diretamente o problema de codificação de caracteres.

A RFC proíbe adicionar uma marca de ordem de byte (do inglês *byte order mark* - BOM) ao início de um texto JSON, e o serializador deste módulo não adiciona um BOM à sua saída. A RFC permite, mas não exige, que os desserializadores JSON ignorem um BOM inicial em sua entrada. O desserializador deste módulo levanta uma *ValueError* quando um BOM inicial está presente.

A RFC não proíbe explicitamente as strings JSON que contêm sequências de bytes que não correspondem a caracteres Unicode válidos (por exemplo, substitutos UTF-16 não emparelhados), mas observa que podem causar problemas de interoperabilidade. Por padrão, este módulo aceita e produz (quando presente no original *str*) pontos de código para tais sequências.

Valores numéricos infinitos e NaN

A RFC não permite a representação de valores infinitos ou numéricos NaN. Apesar disso, por padrão, este módulo aceita e produz `Infinity`, `-Infinity` e `NaN` como se fossem valores literais de número JSON válidos:

```
>>> # Nenhuma dessas chamadas levanta uma exceção, mas os resultados não são JSON␣
↪válidos
>>> json.dumps(float('-inf'))
'-Infinity'
>>> json.dumps(float('nan'))
'NaN'
>>> # Same when deserializing
>>> json.loads('-Infinity')
-inf
>>> json.loads('NaN')
nan
```

No serializador, o parâmetro *allow_nan* pode ser usado para alterar esse comportamento. No desserializador, o parâmetro *parse_constant* pode ser usado para alterar esse comportamento.

Nomes repetidos dentro de um objeto

A RFC especifica que os nomes em um objeto JSON devem ser exclusivos, mas não determina como os nomes repetidos em objetos JSON devem ser tratados. Por padrão, este módulo não levanta uma exceção; em vez disso, ele ignora tudo, exceto o último par nome-valor para um determinado nome:

```
>>> weird_json = '{"x": 1, "x": 2, "x": 3}'
>>> json.loads(weird_json)
{'x': 3}
```

O parâmetro *object_pairs_hook* pode ser usado para alterar este comportamento.

Valores não objeto e não vetor de nível superior

A versão antiga de JSON especificada pela obsoleta **RFC 4627** exige que o valor de nível superior do texto JSON deve ser do tipo object ou array (*dict* ou *list* Python), e não poderia ser dos tipos null, boolean, number ou string. **RFC 7159** removeu essa restrição, e esse módulo não tem nenhuma implementação que faça essa restrição, seja em seus serializadores, sejam nos desserializadores.

Independentemente, para máxima interoperabilidade, você pode querer aderir voluntariamente à restrição.

Limitações de implementação

Algumas implementações de desserializadores JSON podem definir limites em:

- o tamanho de textos JSON aceitos
- o nível máximo de aninhamento de objetos e vetores JSON
- o intervalo e a precisão de números JSON
- o conteúdo e o tamanho máximo de strings JSON

Esse módulo não impõe nenhum limite além daqueles já colocados pelas estruturas de dados Python ou pelo interpretador Python em si.

Quando serializando para JSON, tenha cuidado com qualquer limitação nas aplicações que irão consumir seu JSON. Em particular, é comum para números JSON serem desserializados com números de precisão dupla definida em IEEE 754 e, portanto, sujeito a limitações de precisão e de intervalo da representação. Isso é especialmente relevante quando serializando valores Python *int* de magnitude extremamente grande, ou quando serializando instâncias de tipos numéricos "exóticos" como *decimal.Decimal*.

20.2.5 Interface de linha de comando

Código-fonte: Lib/json/tool.py

O módulo *json* pode ser invocado como um script via `python -m json` para validar e fazer uma impressão bem formatada de objetos JSON. O submódulo *json.tool* implementa essa interface.

Se os argumentos opcionais `infile` e `outfile` não forem especificados, *sys.stdin* e *sys.stdout* serão usados respectivamente:

```
$ echo '{"json": "obj"}' | python -m json
{
    "json": "obj"
}
$ echo '{1.2:3.4}' | python -m json
Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
```

Alterado na versão 3.5: A saída agora está na mesma ordem da entrada. Use a opção *--sort-keys* para ordenar a saída de dicionários alfabeticamente pela chave.

Alterado na versão 3.14: O módulo *json* agora pode ser executado diretamente como `python -m json`. Para retrocompatibilidade, a invocação da CLI como `python -m json.tool` continua sendo aceita.

Opções de linha de comando

infile

O arquivo JSON para ser validado ou saída embelezada:

```
$ python -m json mp_films.json
[
    {
        "title": "And Now for Something Completely Different",
```

(continua na próxima página)

(continuação da página anterior)

```
        "year": 1971
    },
    {
        "title": "Monty Python and the Holy Grail",
        "year": 1975
    }
]
```

Se *infile* não é especificado, lê de `sys.stdin`.

outfile

Escreve a saída de *infile* para o *outfile* dado. Caso contrário, escreve em `sys.stdout`.

--sort-keys

Ordena a saída de dicionários alfabeticamente pela chave.

Adicionado na versão 3.5.

--no-ensure-ascii

Desabilita escape de caracteres não-ascii, veja `json.dumps()` para mais informações.

Adicionado na versão 3.9.

--json-lines

Analisa cada linha da entrada como um objeto JSON separado.

Adicionado na versão 3.8.

--indent, --tab, --no-indent, --compact

Opções mutualmente exclusivas para controle de espaços em branco.

Adicionado na versão 3.9.

-h, --help

Exibe a mensagem de ajuda.

20.3 `mailbox` — Manipulate mailboxes in various formats

Código-fonte: Lib/mailbox.py

This module defines two classes, `Mailbox` and `Message`, for accessing and manipulating on-disk mailboxes and the messages they contain. `Mailbox` offers a dictionary-like mapping from keys to messages. `Message` extends the `email.message` module's `Message` class with format-specific state and behavior. Supported mailbox formats are Maildir, mbox, MH, Babyl, and MMDF.

Ver também

Module ***email***
Represent and manipulate messages.

20.3.1 `Mailbox` objects

class mailbox.**Mailbox**

A mailbox, which may be inspected and modified.

The `Mailbox` class defines an interface and is not intended to be instantiated. Instead, format-specific subclasses should inherit from `Mailbox` and your code should instantiate a particular subclass.

The `Mailbox` interface is dictionary-like, with small keys corresponding to messages. Keys are issued by the `Mailbox` instance with which they will be used and are only meaningful to that `Mailbox` instance. A key continues to identify a message even if the corresponding message is modified, such as by replacing it with another message.

Messages may be added to a `Mailbox` instance using the set-like method *`add()`* and removed using a `del` statement or the set-like methods *`remove()`* and *`discard()`*.

`Mailbox` interface semantics differ from dictionary semantics in some noteworthy ways. Each time a message is requested, a new representation (typically a *`Message`* instance) is generated based upon the current state of the mailbox. Similarly, when a message is added to a `Mailbox` instance, the provided message representation's contents are copied. In neither case is a reference to the message representation kept by the `Mailbox` instance.

The default `Mailbox` *iterator* iterates over message representations, not keys as the default *`dictionary`* iterator does. Moreover, modification of a mailbox during iteration is safe and well-defined. Messages added to the mailbox after an iterator is created will not be seen by the iterator. Messages removed from the mailbox before the iterator yields them will be silently skipped, though using a key from an iterator may result in a *`KeyError`* exception if the corresponding message is subsequently removed.

Aviso

Be very cautious when modifying mailboxes that might be simultaneously changed by some other process. The safest mailbox format to use for such tasks is *`Maildir`*; try to avoid using single-file formats such as *`mbox`* for concurrent writing. If you're modifying a mailbox, you *must* lock it by calling the *`lock()`* and *`unlock()`* methods *before* reading any messages in the file or making any changes by adding or deleting a message. Failing to lock the mailbox runs the risk of losing messages or corrupting the entire mailbox.

`Mailbox` instances have the following methods:

add(*message*)

Add *message* to the mailbox and return the key that has been assigned to it.

Parameter *message* may be a *`Message`* instance, an *`email.message.Message`* instance, a string, a byte string, or a file-like object (which should be open in binary mode). If *message* is an instance of the appropriate format-specific *`Message`* subclass (e.g., if it's an *`mboxMessage`* instance and this is an *`mbox`* instance), its format-specific information is used. Otherwise, reasonable defaults for format-specific information are used.

Alterado na versão 3.2: Support for binary input was added.

remove(*key*)

__delitem__(*key*)

discard(*key*)

Delete the message corresponding to *key* from the mailbox.

If no such message exists, a *`KeyError`* exception is raised if the method was called as *`remove()`* or *`__delitem__()`* but no exception is raised if the method was called as *`discard()`*. The behavior of *`discard()`* may be preferred if the underlying mailbox format supports concurrent modification by other processes.

__setitem__(*key*, *message*)

Replace the message corresponding to *key* with *message*. Raise a *`KeyError`* exception if no message already corresponds to *key*.

As with *`add()`*, parameter *message* may be a *`Message`* instance, an *`email.message.Message`* instance, a string, a byte string, or a file-like object (which should be open in binary mode). If *message* is an instance of the appropriate format-specific *`Message`* subclass (e.g., if it's an *`mboxMessage`* instance and this is an *`mbox`* instance), its format-specific information is used. Otherwise, the format-specific information of the message that currently corresponds to *key* is left unchanged.

iterkeys()

Return an *iterator* over all keys

keys()

The same as `iterkeys()`, except that a `list` is returned rather than an *iterator*

itervalues()

__iter__()

Return an *iterator* over representations of all messages. The messages are represented as instances of the appropriate format-specific `Message` subclass unless a custom message factory was specified when the `Mailbox` instance was initialized.

> **Nota**
>
> The behavior of `__iter__()` is unlike that of dictionaries, which iterate over keys.

values()

The same as `itervalues()`, except that a `list` is returned rather than an *iterator*

iteritems()

Return an *iterator* over (*key*, *message*) pairs, where *key* is a key and *message* is a message representation. The messages are represented as instances of the appropriate format-specific `Message` subclass unless a custom message factory was specified when the `Mailbox` instance was initialized.

items()

The same as `iteritems()`, except that a `list` of pairs is returned rather than an *iterator* of pairs.

get(*key*, *default=None*)

__getitem__(*key*)

Return a representation of the message corresponding to *key*. If no such message exists, *default* is returned if the method was called as `get()` and a `KeyError` exception is raised if the method was called as `__getitem__()`. The message is represented as an instance of the appropriate format-specific `Message` subclass unless a custom message factory was specified when the `Mailbox` instance was initialized.

get_message(*key*)

Return a representation of the message corresponding to *key* as an instance of the appropriate format--specific `Message` subclass, or raise a `KeyError` exception if no such message exists.

get_bytes(*key*)

Return a byte representation of the message corresponding to *key*, or raise a `KeyError` exception if no such message exists.

Adicionado na versão 3.2.

get_string(*key*)

Return a string representation of the message corresponding to *key*, or raise a `KeyError` exception if no such message exists. The message is processed through `email.message.Message` to convert it to a 7bit clean representation.

get_file(*key*)

Return a *file-like* representation of the message corresponding to *key*, or raise a `KeyError` exception if no such message exists. The file-like object behaves as if open in binary mode. This file should be closed once it is no longer needed.

Alterado na versão 3.2: The file object really is a *binary file*; previously it was incorrectly returned in text mode. Also, the *file-like object* now supports the *context manager* protocol: you can use a `with` statement to automatically close it.

> **Nota**
>
> Unlike other representations of messages, *file-like* representations are not necessarily independent of the `Mailbox` instance that created them or of the underlying mailbox. More specific documentation is provided by each subclass.

__contains__(*key*)

Return `True` if *key* corresponds to a message, `False` otherwise.

__len__()

Return a count of messages in the mailbox.

clear()

Delete all messages from the mailbox.

pop(*key*, *default=None*)

Return a representation of the message corresponding to *key* and delete the message. If no such message exists, return *default*. The message is represented as an instance of the appropriate format-specific *Message* subclass unless a custom message factory was specified when the `Mailbox` instance was initialized.

popitem()

Return an arbitrary (*key*, *message*) pair, where *key* is a key and *message* is a message representation, and delete the corresponding message. If the mailbox is empty, raise a *KeyError* exception. The message is represented as an instance of the appropriate format-specific *Message* subclass unless a custom message factory was specified when the `Mailbox` instance was initialized.

update(*arg*)

Parameter *arg* should be a *key*-to-*message* mapping or an iterable of (*key*, *message*) pairs. Updates the mailbox so that, for each given *key* and *message*, the message corresponding to *key* is set to *message* as if by using *__setitem__()*. As with *__setitem__()*, each *key* must already correspond to a message in the mailbox or else a *KeyError* exception will be raised, so in general it is incorrect for *arg* to be a `Mailbox` instance.

> **Nota**
>
> Unlike with dictionaries, keyword arguments are not supported.

flush()

Write any pending changes to the filesystem. For some *Mailbox* subclasses, changes are always written immediately and `flush()` does nothing, but you should still make a habit of calling this method.

lock()

Acquire an exclusive advisory lock on the mailbox so that other processes know not to modify it. An *ExternalClashError* is raised if the lock is not available. The particular locking mechanisms used depend upon the mailbox format. You should *always* lock the mailbox before making any modifications to its contents.

unlock()

Release the lock on the mailbox, if any.

close()

Flush the mailbox, unlock it if necessary, and close any open files. For some `Mailbox` subclasses, this method does nothing.

`Maildir` objects

class mailbox.**Maildir**(*dirname*, *factory=None*, *create=True*)

A subclass of *Mailbox* for mailboxes in Maildir format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, *MaildirMessage* is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

If *create* is `True` and the *dirname* path exists, it will be treated as an existing maildir without attempting to verify its directory layout.

It is for historical reasons that *dirname* is named as such rather than *path*.

Maildir is a directory-based mailbox format invented for the qmail mail transfer agent and now widely supported by other programs. Messages in a Maildir mailbox are stored in separate files within a common directory structure. This design allows Maildir mailboxes to be accessed and modified by multiple unrelated programs without data corruption, so file locking is unnecessary.

Maildir mailboxes contain three subdirectories, namely: `tmp`, `new`, and `cur`. Messages are created momentarily in the `tmp` subdirectory and then moved to the `new` subdirectory to finalize delivery. A mail user agent may subsequently move the message to the `cur` subdirectory and store information about the state of the message in a special "info" section appended to its file name.

Folders of the style introduced by the Courier mail transfer agent are also supported. Any subdirectory of the main mailbox is considered a folder if `'.'` is the first character in its name. Folder names are represented by `Maildir` without the leading `'.'`. Each folder is itself a Maildir mailbox but should not contain other folders. Instead, a logical nesting is indicated using `'.'` to delimit levels, e.g., "Archived.2005.07".

colon

The Maildir specification requires the use of a colon (`':'`) in certain message file names. However, some operating systems do not permit this character in file names, If you wish to use a Maildir-like format on such an operating system, you should specify another character to use instead. The exclamation point (`'!'`) is a popular choice. For example:

```
import mailbox
mailbox.Maildir.colon = '!'
```

The `colon` attribute may also be set on a per-instance basis.

Alterado na versão 3.13: *Maildir* now ignores files with a leading dot.

`Maildir` instances have all of the methods of *Mailbox* in addition to the following:

list_folders()

Return a list of the names of all folders.

get_folder(*folder*)

Return a `Maildir` instance representing the folder whose name is *folder*. A *NoSuchMailboxError* exception is raised if the folder does not exist.

add_folder(*folder*)

Create a folder whose name is *folder* and return a `Maildir` instance representing it.

remove_folder(*folder*)

Delete the folder whose name is *folder*. If the folder contains any messages, a *NotEmptyError* exception will be raised and the folder will not be deleted.

clean()

Delete temporary files from the mailbox that have not been accessed in the last 36 hours. The Maildir specification says that mail-reading programs should do this occasionally.

get_flags(*key*)

Return as a string the flags that are set on the message corresponding to *key*. This is the same as `get_message(key).get_flags()` but much faster, because it does not open the message file. Use this method when iterating over the keys to determine which messages are interesting to get.

If you do have a *MaildirMessage* object, use its *get_flags()* method instead, because changes made by the message's *set_flags()*, *add_flag()* and *remove_flag()* methods are not reflected here until the mailbox's *__setitem__()* method is called.

Adicionado na versão 3.13.

set_flags(*key*, *flags*)

On the message corresponding to *key*, set the flags specified by *flags* and unset all others. Calling `some_mailbox.set_flags(key, flags)` is similar to

```
one_message = some_mailbox.get_message(key)
one_message.set_flags(flags)
some_mailbox[key] = one_message
```

but faster, because it does not open the message file.

If you do have a *MaildirMessage* object, use its *set_flags()* method instead, because changes made with this mailbox method will not be visible to the message object's method, *get_flags()*.

Adicionado na versão 3.13.

add_flag(*key*, *flag*)

On the message corresponding to *key*, set the flags specified by *flag* without changing other flags. To add more than one flag at a time, *flag* may be a string of more than one character.

Considerations for using this method versus the message object's *add_flag()* method are similar to those for *set_flags()*; see the discussion there.

Adicionado na versão 3.13.

remove_flag(*key*, *flag*)

On the message corresponding to *key*, unset the flags specified by *flag* without changing other flags. To remove more than one flag at a time, *flag* may be a string of more than one character.

Considerations for using this method versus the message object's *remove_flag()* method are similar to those for *set_flags()*; see the discussion there.

Adicionado na versão 3.13.

get_info(*key*)

Return a string containing the info for the message corresponding to *key*. This is the same as `get_message(key).get_info()` but much faster, because it does not open the message file. Use this method when iterating over the keys to determine which messages are interesting to get.

If you do have a *MaildirMessage* object, use its *get_info()* method instead, because changes made by the message's *set_info()* method are not reflected here until the mailbox's *__setitem__()* method is called.

Adicionado na versão 3.13.

set_info(*key*, *info*)

Set the info of the message corresponding to *key* to *info*. Calling `some_mailbox.set_info(key, flags)` is similar to

```
one_message = some_mailbox.get_message(key)
one_message.set_info(info)
some_mailbox[key] = one_message
```

but faster, because it does not open the message file.

If you do have a *MaildirMessage* object, use its *set_info()* method instead, because changes made with this mailbox method will not be visible to the message object's method, *get_info()*.

Adicionado na versão 3.13.

Some *Mailbox* methods implemented by `Maildir` deserve special remarks:

add(*message*)

__setitem__(*key*, *message*)

update(*arg*)

> **Aviso**
>
> These methods generate unique file names based upon the current process ID. When using multiple threads, undetected name clashes may occur and cause corruption of the mailbox unless threads are coordinated to avoid using these methods to manipulate the same mailbox simultaneously.

flush()

All changes to Maildir mailboxes are immediately applied, so this method does nothing.

lock()

unlock()

Maildir mailboxes do not support (or require) locking, so these methods do nothing.

close()

`Maildir` instances do not keep any open files and the underlying mailboxes do not support locking, so this method does nothing.

get_file(*key*)

Depending upon the host platform, it may not be possible to modify or remove the underlying message while the returned file remains open.

> **Ver também**
>
> **maildir man page from Courier**
> A specification of the format. Describes a common extension for supporting folders.
>
> **Using maildir format**
> Notes on Maildir by its inventor. Includes an updated name-creation scheme and details on "info" semantics.

`mbox` objects

class `mailbox.`**mbox**(*path*, *factory=None*, *create=True*)

A subclass of *Mailbox* for mailboxes in mbox format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, *mboxMessage* is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

The mbox format is the classic format for storing mail on Unix systems. All messages in an mbox mailbox are stored in a single file with the beginning of each message indicated by a line whose first five characters are "From ".

Several variations of the mbox format exist to address perceived shortcomings in the original. In the interest of compatibility, `mbox` implements the original format, which is sometimes referred to as *mboxo*. This means that the *Content-Length* header, if present, is ignored and that any occurrences of "From " at the beginning

of a line in a message body are transformed to ">From " when storing the message, although occurrences of ">From " are not transformed to "From " when reading the message.

Some *Mailbox* methods implemented by mbox deserve special remarks:

get_bytes(*key*, *from_=False*)

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix "From " line. If *from_* is False, the first line of the file is dropped.

get_file(*key*, *from_=False*)

Using the file after calling *flush()* or *close()* on the mbox instance may yield unpredictable results or raise an exception.

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix "From " line. If *from_* is False, the first line of the file is dropped.

get_string(*key*, *from_=False*)

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix "From " line. If *from_* is False, the first line of the file is dropped.

lock()

unlock()

Three locking mechanisms are used—dot locking and, if available, the flock() and lockf() system calls.

Ver também

mbox man page from tin
A specification of the format, with details on locking.

Configuring Netscape Mail on Unix: Why The Content-Length Format is Bad
An argument for using the original mbox format rather than a variation.

"mbox" is a family of several mutually incompatible mailbox formats
A history of mbox variations.

MH objects

class mailbox.**MH**(*path*, *factory=None*, *create=True*)

A subclass of *Mailbox* for mailboxes in MH format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is None, *MHMessage* is used as the default message representation. If *create* is True, the mailbox is created if it does not exist.

MH is a directory-based mailbox format invented for the MH Message Handling System, a mail user agent. Each message in an MH mailbox resides in its own file. An MH mailbox may contain other MH mailboxes (called *folders*) in addition to messages. Folders may be nested indefinitely. MH mailboxes also support *sequences*, which are named lists used to logically group messages without moving them to sub-folders. Sequences are defined in a file called .mh_sequences in each folder.

The MH class manipulates MH mailboxes, but it does not attempt to emulate all of **mh**'s behaviors. In particular, it does not modify and is not affected by the context or .mh_profile files that are used by **mh** to store its state and configuration.

MH instances have all of the methods of *Mailbox* in addition to the following:

Alterado na versão 3.13: Supported folders that don't contain a .mh_sequences file.

list_folders()

Return a list of the names of all folders.

get_folder(*folder*)

Return an `MH` instance representing the folder whose name is *folder*. A `NoSuchMailboxError` exception is raised if the folder does not exist.

add_folder(*folder*)

Create a folder whose name is *folder* and return an `MH` instance representing it.

remove_folder(*folder*)

Delete the folder whose name is *folder*. If the folder contains any messages, a `NotEmptyError` exception will be raised and the folder will not be deleted.

get_sequences()

Return a dictionary of sequence names mapped to key lists. If there are no sequences, the empty dictionary is returned.

set_sequences(*sequences*)

Re-define the sequences that exist in the mailbox based upon *sequences*, a dictionary of names mapped to key lists, like returned by `get_sequences()`.

pack()

Rename messages in the mailbox as necessary to eliminate gaps in numbering. Entries in the sequences list are updated correspondingly.

> **Nota**
>
> Already-issued keys are invalidated by this operation and should not be subsequently used.

Some `Mailbox` methods implemented by `MH` deserve special remarks:

remove(*key*)

__delitem__(*key*)

discard(*key*)

These methods immediately delete the message. The MH convention of marking a message for deletion by prepending a comma to its name is not used.

lock()

unlock()

Three locking mechanisms are used—dot locking and, if available, the `flock()` and `lockf()` system calls. For MH mailboxes, locking the mailbox means locking the `.mh_sequences` file and, only for the duration of any operations that affect them, locking individual message files.

get_file(*key*)

Depending upon the host platform, it may not be possible to remove the underlying message while the returned file remains open.

flush()

All changes to MH mailboxes are immediately applied, so this method does nothing.

close()

`MH` instances do not keep any open files, so this method is equivalent to `unlock()`.

> **Ver também**
>
> **nmh - Message Handling System**
> Home page of **nmh**, an updated version of the original **mh**.
>
> **MH & nmh: Email for Users & Programmers**
> A GPL-licensed book on **mh** and **nmh**, with some information on the mailbox format.

`Babyl` objects

class mailbox.**Babyl**(*path*, *factory=None*, *create=True*)

A subclass of *Mailbox* for mailboxes in Babyl format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, *BabylMessage* is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

Babyl is a single-file mailbox format used by the Rmail mail user agent included with Emacs. The beginning of a message is indicated by a line containing the two characters Control-Underscore (`'\037'`) and Control-L (`'\014'`). The end of a message is indicated by the start of the next message or, in the case of the last message, a line containing a Control-Underscore (`'\037'`) character.

Messages in a Babyl mailbox have two sets of headers, original headers and so-called visible headers. Visible headers are typically a subset of the original headers that have been reformatted or abridged to be more attractive. Each message in a Babyl mailbox also has an accompanying list of *labels*, or short strings that record extra information about the message, and a list of all user-defined labels found in the mailbox is kept in the Babyl options section.

`Babyl` instances have all of the methods of *Mailbox* in addition to the following:

get_labels()

Return a list of the names of all user-defined labels used in the mailbox.

> **Nota**
>
> The actual messages are inspected to determine which labels exist in the mailbox rather than consulting the list of labels in the Babyl options section, but the Babyl section is updated whenever the mailbox is modified.

Some *Mailbox* methods implemented by `Babyl` deserve special remarks:

get_file(*key*)

In Babyl mailboxes, the headers of a message are not stored contiguously with the body of the message. To generate a file-like representation, the headers and body are copied together into an *io.BytesIO* instance, which has an API identical to that of a file. As a result, the file-like object is truly independent of the underlying mailbox but does not save memory compared to a string representation.

lock()

unlock()

Three locking mechanisms are used—dot locking and, if available, the `flock()` and `lockf()` system calls.

> **Ver também**
>
> **Format of Version 5 Babyl Files**
> A specification of the Babyl format.
>
> **Reading Mail with Rmail**
> The Rmail manual, with some information on Babyl semantics.

`MMDF` objects

class mailbox.**MMDF**(*path*, *factory=None*, *create=True*)

A subclass of *Mailbox* for mailboxes in MMDF format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, *MMDFMessage* is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

MMDF is a single-file mailbox format invented for the Multichannel Memorandum Distribution Facility, a mail transfer agent. Each message is in the same form as an mbox message but is bracketed before and after by lines containing four Control-A (`'\001'`) characters. As with the mbox format, the beginning of each message is indicated by a line whose first five characters are "From ", but additional occurrences of "From " are not transformed to ">From " when storing messages because the extra message separator lines prevent mistaking such occurrences for the starts of subsequent messages.

Some *Mailbox* methods implemented by MMDF deserve special remarks:

get_bytes(*key*, *from_=False*)

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix "From " line. If *from_* is False, the first line of the file is dropped.

get_file(*key*, *from_=False*)

Using the file after calling *flush()* or *close()* on the MMDF instance may yield unpredictable results or raise an exception.

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix "From " line. If *from_* is False, the first line of the file is dropped.

lock()

unlock()

Three locking mechanisms are used—dot locking and, if available, the `flock()` and `lockf()` system calls.

Ver também

mmdf man page from tin
A specification of MMDF format from the documentation of tin, a newsreader.

MMDF
A Wikipedia article describing the Multichannel Memorandum Distribution Facility.

20.3.2 `Message` objects

class mailbox.**Message**(*message=None*)

A subclass of the *email.message* module's *Message*. Subclasses of `mailbox.Message` add mailbox-format-specific state and behavior.

If *message* is omitted, the new instance is created in a default, empty state. If *message* is an *email.message.Message* instance, its contents are copied; furthermore, any format-specific information is converted insofar as possible if *message* is a `Message` instance. If *message* is a string, a byte string, or a file, it should contain an **RFC 2822**-compliant message, which is read and parsed. Files should be open in binary mode, but text mode files are accepted for backward compatibility.

The format-specific state and behaviors offered by subclasses vary, but in general it is only the properties that are not specific to a particular mailbox that are supported (although presumably the properties are specific to a particular mailbox format). For example, file offsets for single-file mailbox formats and file names for directory-based mailbox formats are not retained, because they are only applicable to the original mailbox. But state such as whether a message has been read by the user or marked as important is retained, because it applies to the message itself.

There is no requirement that `Message` instances be used to represent messages retrieved using *Mailbox* instances. In some situations, the time and memory required to generate `Message` representations might not be acceptable. For such situations, `Mailbox` instances also offer string and file-like representations, and a custom message factory may be specified when a `Mailbox` instance is initialized.

`MaildirMessage` objects

class `mailbox.`**MaildirMessage**(*message=None*)

A message with Maildir-specific behaviors. Parameter *message* has the same meaning as with the `Message` constructor.

Typically, a mail user agent application moves all of the messages in the `new` subdirectory to the `cur` subdirectory after the first time the user opens and closes the mailbox, recording that the messages are old whether or not they've actually been read. Each message in `cur` has an "info" section added to its file name to store information about its state. (Some mail readers may also add an "info" section to messages in `new`.) The "info" section may take one of two forms: it may contain "2," followed by a list of standardized flags (e.g., "2,FR") or it may contain "1," followed by so-called experimental information. Standard flags for Maildir messages are as follows:

Sinalizador	Significado	Explicação
D	Draft	Under composition
F	Flagged	Marked as important
P	Passed	Forwarded, resent, or bounced
R	Replied	Replied to
S	Seen	Read
T	Trashed	Marked for subsequent deletion

`MaildirMessage` instances offer the following methods:

get_subdir()

Return either "new" (if the message should be stored in the `new` subdirectory) or "cur" (if the message should be stored in the `cur` subdirectory).

> **Nota**
>
> A message is typically moved from `new` to `cur` after its mailbox has been accessed, whether or not the message has been read. A message `msg` has been read if `"S" in msg.get_flags()` is `True`.

set_subdir(*subdir*)

Set the subdirectory the message should be stored in. Parameter *subdir* must be either "new" or "cur".

get_flags()

Return a string specifying the flags that are currently set. If the message complies with the standard Maildir format, the result is the concatenation in alphabetical order of zero or one occurrence of each of `'D'`, `'F'`, `'P'`, `'R'`, `'S'`, and `'T'`. The empty string is returned if no flags are set or if "info" contains experimental semantics.

set_flags(*flags*)

Set the flags specified by *flags* and unset all others.

add_flag(*flag*)

Set the flag(s) specified by *flag* without changing other flags. To add more than one flag at a time, *flag* may be a string of more than one character. The current "info" is overwritten whether or not it contains experimental information rather than flags.

remove_flag(*flag*)

Unset the flag(s) specified by *flag* without changing other flags. To remove more than one flag at a time, *flag* maybe a string of more than one character. If "info" contains experimental information rather than flags, the current "info" is not modified.

get_date()

Return the delivery date of the message as a floating-point number representing seconds since the epoch.

set_date(*date*)

Set the delivery date of the message to *date*, a floating-point number representing seconds since the epoch.

get_info()

Return a string containing the "info" for a message. This is useful for accessing and modifying "info" that is experimental (i.e., not a list of flags).

set_info(*info*)

Set "info" to *info*, which should be a string.

When a `MaildirMessage` instance is created based upon an *mboxMessage* or *MMDFMessage* instance, the *Status* and *X-Status* headers are omitted and the following conversions take place:

Resulting state	*mboxMessage* or *MMDFMessage* state
"cur" subdirectory	O flag
F flag	F flag
R flag	A flag
S flag	R flag
T flag	D flag

When a `MaildirMessage` instance is created based upon an *MHMessage* instance, the following conversions take place:

Resulting state	*MHMessage* state
"cur" subdirectory	"unseen" sequence
"cur" subdirectory and S flag	no "unseen" sequence
F flag	"flagged" sequence
R flag	"replied" sequence

When a `MaildirMessage` instance is created based upon a *BabylMessage* instance, the following conversions take place:

Resulting state	*BabylMessage* state
"cur" subdirectory	"unseen" label
"cur" subdirectory and S flag	no "unseen" label
P flag	"forwarded" or "resent" label
R flag	"answered" label
T flag	"deleted" label

mboxMessage objects

class mailbox.**mboxMessage**(*message=None*)

A message with mbox-specific behaviors. Parameter *message* has the same meaning as with the *Message* constructor.

Messages in an mbox mailbox are stored together in a single file. The sender's envelope address and the time of delivery are typically stored in a line beginning with "From " that is used to indicate the start of a message, though there is considerable variation in the exact format of this data among mbox implementations. Flags that indicate the state of the message, such as whether it has been read or marked as important, are typically stored in *Status* and *X-Status* headers.

Conventional flags for mbox messages are as follows:

Sinalizador	Significado	Explicação
R	Read	Read
O	Old	Previously detected by MUA
D	Deleted	Marked for subsequent deletion
F	Flagged	Marked as important
A	Answered	Replied to

The "R" and "O" flags are stored in the `Status` header, and the "D", "F", and "A" flags are stored in the `X-Status` header. The flags and headers typically appear in the order mentioned.

`mboxMessage` instances offer the following methods:

get_from()

Return a string representing the "From " line that marks the start of the message in an mbox mailbox. The leading "From " and the trailing newline are excluded.

set_from(*from_*, *time_*=*None*)

Set the "From " line to *from_*, which should be specified without a leading "From " or trailing newline. For convenience, *time_* may be specified and will be formatted appropriately and appended to *from_*. If *time_* is specified, it should be a `time.struct_time` instance, a tuple suitable for passing to `time.strftime()`, or `True` (to use `time.gmtime()`).

get_flags()

Return a string specifying the flags that are currently set. If the message complies with the conventional format, the result is the concatenation in the following order of zero or one occurrence of each of `'R'`, `'O'`, `'D'`, `'F'`, and `'A'`.

set_flags(*flags*)

Set the flags specified by *flags* and unset all others. Parameter *flags* should be the concatenation in any order of zero or more occurrences of each of `'R'`, `'O'`, `'D'`, `'F'`, and `'A'`.

add_flag(*flag*)

Set the flag(s) specified by *flag* without changing other flags. To add more than one flag at a time, *flag* may be a string of more than one character.

remove_flag(*flag*)

Unset the flag(s) specified by *flag* without changing other flags. To remove more than one flag at a time, *flag* maybe a string of more than one character.

When an `mboxMessage` instance is created based upon a `MaildirMessage` instance, a "From " line is generated based upon the `MaildirMessage` instance's delivery date, and the following conversions take place:

Resulting state	`MaildirMessage` state
R flag	S flag
O flag	"cur" subdirectory
D flag	T flag
F flag	F flag
A flag	R flag

When an `mboxMessage` instance is created based upon an `MHMessage` instance, the following conversions take place:

Resulting state	`MHMessage` state
R flag and O flag	no "unseen" sequence
O flag	"unseen" sequence
F flag	"flagged" sequence
A flag	"replied" sequence

When an `mboxMessage` instance is created based upon a *`BabylMessage`* instance, the following conversions take place:

Resulting state	*`BabylMessage`* state
R flag and O flag	no “unseen” label
O flag	“unseen” label
D flag	“deleted” label
A flag	“answered” label

When a `mboxMessage` instance is created based upon an *`MMDFMessage`* instance, the “From “ line is copied and all flags directly correspond:

Resulting state	*`MMDFMessage`* state
R flag	R flag
O flag	O flag
D flag	D flag
F flag	F flag
A flag	A flag

`MHMessage` objects

class mailbox.**MHMessage**(*message=None*)

A message with MH-specific behaviors. Parameter *message* has the same meaning as with the *`Message`* constructor.

MH messages do not support marks or flags in the traditional sense, but they do support sequences, which are logical groupings of arbitrary messages. Some mail reading programs (although not the standard **mh** and **nmh**) use sequences in much the same way flags are used with other formats, as follows:

Sequence	Explicação
unseen	Not read, but previously detected by MUA
replied	Replied to
flagged	Marked as important

`MHMessage` instances offer the following methods:

get_sequences()

Return a list of the names of sequences that include this message.

set_sequences(*sequences*)

Set the list of sequences that include this message.

add_sequence(*sequence*)

Add *sequence* to the list of sequences that include this message.

remove_sequence(*sequence*)

Remove *sequence* from the list of sequences that include this message.

When an `MHMessage` instance is created based upon a *`MaildirMessage`* instance, the following conversions take place:

Resulting state	*`MaildirMessage`* state
“unseen” sequence	no S flag
“replied” sequence	R flag
“flagged” sequence	F flag

When an `MHMessage` instance is created based upon an *`mboxMessage`* or *`MMDFMessage`* instance, the *`Status`* and *`X-Status`* headers are omitted and the following conversions take place:

Resulting state	*`mboxMessage`* or *`MMDFMessage`* state
“unseen” sequence	no R flag
“replied” sequence	A flag
“flagged” sequence	F flag

When an `MHMessage` instance is created based upon a *`BabylMessage`* instance, the following conversions take place:

Resulting state	*`BabylMessage`* state
“unseen” sequence	“unseen” label
“replied” sequence	“answered” label

`BabylMessage` objects

class mailbox.**BabylMessage**(*message=None*)

A message with Babyl-specific behaviors. Parameter *message* has the same meaning as with the *`Message`* constructor.

Certain message labels, called *attributes*, are defined by convention to have special meanings. The attributes are as follows:

Label	Explicação
unseen	Not read, but previously detected by MUA
deleted	Marked for subsequent deletion
filed	Copied to another file or mailbox
answered	Replied to
forwarded	Forwarded
edited	Modified by the user
resent	Resent

By default, Rmail displays only visible headers. The `BabylMessage` class, though, uses the original headers because they are more complete. Visible headers may be accessed explicitly if desired.

`BabylMessage` instances offer the following methods:

get_labels()

Return a list of labels on the message.

set_labels(*labels*)

Set the list of labels on the message to *labels*.

add_label(*label*)

Add *label* to the list of labels on the message.

remove_label(*label*)

Remove *label* from the list of labels on the message.

get_visible()

Return a *`Message`* instance whose headers are the message’s visible headers and whose body is empty.

set_visible(*visible*)

Set the message’s visible headers to be the same as the headers in *message*. Parameter *visible* should be a *`Message`* instance, an *`email.message.Message`* instance, a string, or a file-like object (which should be open in text mode).

update_visible()

When a BabylMessage instance's original headers are modified, the visible headers are not automatically modified to correspond. This method updates the visible headers as follows: each visible header with a corresponding original header is set to the value of the original header, each visible header without a corresponding original header is removed, and any of *Date*, *From*, *Reply-To*, *To*, *CC*, and *Subject* that are present in the original headers but not the visible headers are added to the visible headers.

When a BabylMessage instance is created based upon a *MaildirMessage* instance, the following conversions take place:

Resulting state	*MaildirMessage* state
"unseen" label	no S flag
"deleted" label	T flag
"answered" label	R flag
"forwarded" label	P flag

When a BabylMessage instance is created based upon an *mboxMessage* or *MMDFMessage* instance, the *Status* and *X-Status* headers are omitted and the following conversions take place:

Resulting state	*mboxMessage* or *MMDFMessage* state
"unseen" label	no R flag
"deleted" label	D flag
"answered" label	A flag

When a BabylMessage instance is created based upon an *MHMessage* instance, the following conversions take place:

Resulting state	*MHMessage* state
"unseen" label	"unseen" sequence
"answered" label	"replied" sequence

MMDFMessage objects

class mailbox.**MMDFMessage**(*message=None*)

A message with MMDF-specific behaviors. Parameter *message* has the same meaning as with the *Message* constructor.

As with message in an mbox mailbox, MMDF messages are stored with the sender's address and the delivery date in an initial line beginning with "From ". Likewise, flags that indicate the state of the message are typically stored in *Status* and *X-Status* headers.

Conventional flags for MMDF messages are identical to those of mbox message and are as follows:

Sinalizador	Significado	Explicação
R	Read	Read
O	Old	Previously detected by MUA
D	Deleted	Marked for subsequent deletion
F	Flagged	Marked as important
A	Answered	Replied to

The "R" and "O" flags are stored in the *Status* header, and the "D", "F", and "A" flags are stored in the *X-Status* header. The flags and headers typically appear in the order mentioned.

MMDFMessage instances offer the following methods, which are identical to those offered by *mboxMessage*:

get_from()

Return a string representing the "From " line that marks the start of the message in an mbox mailbox. The leading "From " and the trailing newline are excluded.

set_from(*from_*, *time_*=*None*)

Set the "From " line to *from_*, which should be specified without a leading "From " or trailing newline. For convenience, *time_* may be specified and will be formatted appropriately and appended to *from_*. If *time_* is specified, it should be a *time.struct_time* instance, a tuple suitable for passing to *time.strftime()*, or `True` (to use *time.gmtime()*).

get_flags()

Return a string specifying the flags that are currently set. If the message complies with the conventional format, the result is the concatenation in the following order of zero or one occurrence of each of `'R'`, `'O'`, `'D'`, `'F'`, and `'A'`.

set_flags(*flags*)

Set the flags specified by *flags* and unset all others. Parameter *flags* should be the concatenation in any order of zero or more occurrences of each of `'R'`, `'O'`, `'D'`, `'F'`, and `'A'`.

add_flag(*flag*)

Set the flag(s) specified by *flag* without changing other flags. To add more than one flag at a time, *flag* may be a string of more than one character.

remove_flag(*flag*)

Unset the flag(s) specified by *flag* without changing other flags. To remove more than one flag at a time, *flag* maybe a string of more than one character.

When an `MMDFMessage` instance is created based upon a *MaildirMessage* instance, a "From " line is generated based upon the *MaildirMessage* instance's delivery date, and the following conversions take place:

Resulting state	*MaildirMessage* state
R flag	S flag
O flag	"cur" subdirectory
D flag	T flag
F flag	F flag
A flag	R flag

When an `MMDFMessage` instance is created based upon an *MHMessage* instance, the following conversions take place:

Resulting state	*MHMessage* state
R flag and O flag	no "unseen" sequence
O flag	"unseen" sequence
F flag	"flagged" sequence
A flag	"replied" sequence

When an `MMDFMessage` instance is created based upon a *BabylMessage* instance, the following conversions take place:

Resulting state	*BabylMessage* state
R flag and O flag	no "unseen" label
O flag	"unseen" label
D flag	"deleted" label
A flag	"answered" label

When an `MMDFMessage` instance is created based upon an *`mboxMessage`* instance, the "From " line is copied and all flags directly correspond:

Resulting state	*`mboxMessage`* state
R flag	R flag
O flag	O flag
D flag	D flag
F flag	F flag
A flag	A flag

20.3.3 Exceções

The following exception classes are defined in the `mailbox` module:

exception `mailbox.`**Error**

The based class for all other module-specific exceptions.

exception `mailbox.`**NoSuchMailboxError**

Raised when a mailbox is expected but is not found, such as when instantiating a *`Mailbox`* subclass with a path that does not exist (and with the *create* parameter set to `False`), or when opening a folder that does not exist.

exception `mailbox.`**NotEmptyError**

Raised when a mailbox is not empty but is expected to be, such as when deleting a folder that contains messages.

exception `mailbox.`**ExternalClashError**

Raised when some mailbox-related condition beyond the control of the program causes it to be unable to proceed, such as when failing to acquire a lock that another program already holds a lock, or when a uniquely generated file name already exists.

exception `mailbox.`**FormatError**

Raised when the data in a file cannot be parsed, such as when an *`MH`* instance attempts to read a corrupted `.mh_sequences` file.

20.3.4 Exemplos

A simple example of printing the subjects of all messages in a mailbox that seem interesting:

```
import mailbox
for message in mailbox.mbox('~/mbox'):
    subject = message['subject']       # Could possibly be None.
    if subject and 'python' in subject.lower():
        print(subject)
```

To copy all mail from a Babyl mailbox to an MH mailbox, converting all of the format-specific information that can be converted:

```
import mailbox
destination = mailbox.MH('~/Mail')
destination.lock()
for message in mailbox.Babyl('~/RMAIL'):
    destination.add(mailbox.MHMessage(message))
destination.flush()
destination.unlock()
```

This example sorts mail from several mailing lists into different mailboxes, being careful to avoid mail corruption due to concurrent modification by other programs, mail loss due to interruption of the program, or premature termination due to malformed messages in the mailbox:

```
import mailbox
import email.errors

list_names = ('python-list', 'python-dev', 'python-bugs')

boxes = {name: mailbox.mbox('~/email/%s' % name) for name in list_names}
inbox = mailbox.Maildir('~/Maildir', factory=None)

for key in inbox.iterkeys():
    try:
        message = inbox[key]
    except email.errors.MessageParseError:
        continue                # The message is malformed. Just leave it.

    for name in list_names:
        list_id = message['list-id']
        if list_id and name in list_id:
            # Get mailbox to use
            box = boxes[name]

            # Write copy to disk before removing original.
            # If there's a crash, you might duplicate a message, but
            # that's better than losing a message completely.
            box.lock()
            box.add(message)
            box.flush()
            box.unlock()

            # Remove original message
            inbox.lock()
            inbox.discard(key)
            inbox.flush()
            inbox.unlock()
            break               # Found destination, so stop looking.

for box in boxes.itervalues():
    box.close()
```

20.4 `mimetypes` — Mapeia nomes de arquivos para tipos MIME

Código-fonte: Lib/mimetypes.py

O módulo *mimetypes* converte entre um nome de arquivo ou URL e o tipo MIME associado à extensão do arquivo. As conversões são fornecidas do nome do arquivo para o tipo MIME e da extensão do tipo MIME para o nome do arquivo; codificações não são suportadas para a última conversão.

O módulo fornece uma classe e várias funções convenientes. As funções são a interface normal para este módulo, mas algumas aplicações também podem estar interessadas na classe.

As funções descritas abaixo fornecem a interface principal para este módulo. Se o módulo não foi inicializado, eles chamarão *init()* se confiarem nas informações configuradas por *init()*.

mimetypes.**guess_type**(*url*, *strict=True*)

Adivinha o tipo de arquivo com base em seu nome de arquivo, caminho ou URL, fornecido por *url*. A URL pode ser uma string ou um *objeto caminho ou similar*.

O valor de retorno é uma tupla `(type, encoding)` onde o *tipo* é `None` se o tipo não puder ser ser adivinhado (sufixo ausente ou desconhecido) ou uma string no formato `'type/subtype'`, utilizável para um cabeçalho MIME *content-type*.

encoding é `None` para nenhuma codificação ou o nome do programa usado para codificar (por exemplo **compress** ou **gzip**). A codificação é adequada para uso como cabeçalho *Content-Encoding* , **não** como cabeçalho *Content-Transfer-Encoding*. Os mapeamentos são orientados por tabela. Os sufixos de codificação diferenciam maiúsculas de minúsculas; os sufixos de tipo são testados primeiro considerando maiúsculas e minúsculas e depois sem considerar.

O argumento opcional *strict* é um sinalizador que especifica se a lista de tipos MIME conhecidos é limitada apenas aos tipos oficiais registrados na IANA. No entanto, o comportamento deste módulo também depende do sistema operacional subjacente. Somente tipos de arquivo reconhecidos pelo sistema operacional ou registrados explicitamente no banco de dados interno do Python podem ser identificados. Quando *strict* é `True` (o padrão), apenas os tipos IANA são suportados; quando *strict* é `False`, alguns tipos MIME adicionais não padronizados, mas geralmente usados, também são reconhecidos.

Alterado na versão 3.8: Adicionado suporte para que a *url* seja um *objeto caminho ou similar*.

Descontinuado desde a versão 3.13: Passar um caminho de arquivo em vez de URL está *suavemente descontinuado*. Use *guess_file_type()* para isso.

`mimetypes.`**guess_file_type**(*path*, ***, *strict=True*)

Adivinha o tipo de um arquivo com base em seu caminho, dado por *path*. Semelhante à função *guess_type()*, mas aceita um caminho em vez de URL. O caminho pode ser uma string, um objeto bytes ou um *objeto caminho ou similar*.

Adicionado na versão 3.13.

`mimetypes.`**guess_all_extensions**(*type*, *strict=True*)

Adivinha as extensões para um arquivo com base em seu tipo MIME, fornecido por *type*. O valor de retorno é uma lista de strings que fornecem todas as extensões possíveis de nome de arquivo, incluindo o ponto (`'.'`). Não é garantido que as extensões tenham sido associadas a qualquer fluxo de dados específico, mas seriam mapeadas para o tipo MIME *type* por *guess_type()* e *guess_file_type()*.

O argumento opcional *strict* tem o mesmo significado que usado com a função *guess_type()*.

`mimetypes.`**guess_extension**(*type*, *strict=True*)

Adivinha a extensão para um arquivo com base em seu tipo MIME, fornecido por *type*. O valor de retorno é uma string que fornece uma extensão de nome de arquivo, incluindo o ponto (`'.'`). Não é garantido que a extensão tenha sido associada a qualquer fluxo de dados específico, mas seria mapeada para o tipo MIME *type* por *guess_type()* e *guess_file_type()*. Se nenhuma extensão puder ser adivinhada para *type*, `None` é retornado.

O argumento opcional *strict* tem o mesmo significado que usado com a função *guess_type()*.

Algumas funções e dados adicionais estão disponíveis para controlar o comportamento do módulo.

`mimetypes.`**init**(*files=None*)

Inicializa as estruturas de dados internas. Se fornecido, *files* deverá ser uma sequência de nomes de arquivos que devem ser usados para aumentar o mapa de tipos padrão. Se for omitido, os nomes de arquivos a serem usados serão retirados de *knownfiles*; no Windows, as configurações atuais do registro são carregadas. Cada arquivo nomeado em *files* ou *knownfiles* tem precedência sobre os definidos anteriormente. É permitido chamar *init()* repetidamente.

Especificar uma lista vazia para *files* impedirá que os padrões do sistema sejam carregados: apenas os valores bem conhecidos estarão presentes em uma lista interna.

Se *files* for `None`, a estrutura de dados interna será completamente reconstruída com seu valor inicial padrão. Essa é uma operação estável e produzirá os mesmos resultados quando chamada várias vezes.

Alterado na versão 3.2: Anteriormente, as configurações do registro do Windows eram ignoradas.

mimetypes.**read_mime_types**(*filename*)

Carrega o mapa de tipos fornecido no arquivo *filename*, se existir. O mapa de tipos é retornado como um dicionário que faz o mapeamento de extensões de nome de arquivo, incluindo o ponto inicial (`'.'`), para strings no formato `'tipo/subtipo'`. Se o arquivo *filename* não existir ou não puder ser lido, `None` é retornado.

mimetypes.**add_type**(*type*, *ext*, *strict=True*)

Adiciona um mapeamento do tipo MIME *type* para a extensão *ext*. Quando a extensão já for conhecida, o novo tipo substituirá o antigo. Quando o tipo já for conhecido, a extensão será adicionada à lista de extensões conhecidas.

Quando *strict* for `True` (o padrão), o mapeamento será adicionado aos tipos MIME oficiais, caso contrário, aos não-padrão.

mimetypes.**inited**

Sinalizador indicando se as estruturas de dados globais foram inicializadas ou não. Isso é definido para `True` por *init()*.

mimetypes.**knownfiles**

Lista de nomes de arquivos de mapas de tipos comumente instalados. Esses arquivos são normalmente nomeado como `mime.types` e são instalados em diferentes locais por diferentes pacote.

mimetypes.**suffix_map**

Dicionário mapeando sufixos para sufixos. Isso é usado para permitir o reconhecimento de arquivos codificados para os quais a codificação e o tipo são indicados pela mesma extensão. Por exemplo, em um arquivo codificado, a extensão `.tgz` é mapeada para `.tar.gz` para permitir que a codificação e o tipo sejam reconhecidos separadamente.

mimetypes.**encodings_map**

Dicionário mapeando extensões de nome de arquivo para os tipos de codificação.

mimetypes.**types_map**

Dicionário mapeando extensões de nome do arquivo para tipos MIME.

mimetypes.**common_types**

Dicionário mapeando extensões de nome de arquivo para tipos MIME não padronizados, mas comumente encontrados.

Um exemplo de uso do módulo:

```
>>> import mimetypes
>>> mimetypes.init()
>>> mimetypes.knownfiles
['/etc/mime.types', '/etc/httpd/mime.types', ... ]
>>> mimetypes.suffix_map['.tgz']
'.tar.gz'
>>> mimetypes.encodings_map['.gz']
'gzip'
>>> mimetypes.types_map['.tgz']
'application/x-tar-gz'
```

20.4.1 MimeTypes objects

The *MimeTypes* class may be useful for applications which may want more than one MIME-type database; it provides an interface similar to the one of the *mimetypes* module.

class mimetypes.**MimeTypes**(*filenames=()*, *strict=True*)

This class represents a MIME-types database. By default, it provides access to the same database as the rest of this module. The initial database is a copy of that provided by the module, and may be extended by loading additional `mime.types`-style files into the database using the *read()* or *readfp()* methods. The mapping dictionaries may also be cleared before loading additional data if the default data is not desired.

The optional *filenames* parameter can be used to cause additional files to be loaded "on top" of the default database.

suffix_map

Dictionary mapping suffixes to suffixes. This is used to allow recognition of encoded files for which the encoding and the type are indicated by the same extension. For example, the `.tgz` extension is mapped to `.tar.gz` to allow the encoding and type to be recognized separately. This is initially a copy of the global *suffix_map* defined in the module.

encodings_map

Dictionary mapping filename extensions to encoding types. This is initially a copy of the global *encodings_map* defined in the module.

types_map

Tuple containing two dictionaries, mapping filename extensions to MIME types: the first dictionary is for the non-standards types and the second one is for the standard types. They are initialized by *common_types* and *types_map*.

types_map_inv

Tuple containing two dictionaries, mapping MIME types to a list of filename extensions: the first dictionary is for the non-standards types and the second one is for the standard types. They are initialized by *common_types* and *types_map*.

guess_extension(*type*, *strict=True*)

Similar to the *guess_extension()* function, using the tables stored as part of the object.

guess_type(*url*, *strict=True*)

Similar to the *guess_type()* function, using the tables stored as part of the object.

guess_file_type(*path*, *, *strict=True*)

Similar to the *guess_file_type()* function, using the tables stored as part of the object.

Adicionado na versão 3.13.

guess_all_extensions(*type*, *strict=True*)

Similar to the *guess_all_extensions()* function, using the tables stored as part of the object.

read(*filename*, *strict=True*)

Load MIME information from a file named *filename*. This uses *readfp()* to parse the file.

If *strict* is `True`, information will be added to list of standard types, else to the list of non-standard types.

readfp(*fp*, *strict=True*)

Carrega informações do tipo MIME de um arquivo aberto *fp*. O arquivo precisa estar no formato padrão dos arquivos `mime.types`.

If *strict* is `True`, information will be added to the list of standard types, else to the list of non-standard types.

read_windows_registry(*strict=True*)

Carrega informações do tipo MIME a partir do registro do Windows.

Disponibilidade: Windows.

If *strict* is `True`, information will be added to the list of standard types, else to the list of non-standard types.

Adicionado na versão 3.2.

add_type(*type*, *ext*, *strict=True*)

Add a mapping from the MIME type *type* to the extension *ext*. Valid extensions start with a '.' or are empty. When the extension is already known, the new type will replace the old one. When the type is already known the extension will be added to the list of known extensions.

Quando *strict* for `True` (o padrão), o mapeamento será adicionado aos tipos MIME oficiais, caso contrário, aos não-padrão.

Deprecated since version 3.14, will be removed in version 3.16: Invalid, undotted extensions will raise a *ValueError* in Python 3.16.

20.4.2 Uso na linha de comando

The `mimetypes` module can be executed as a script from the command line.

```
python -m mimetypes [-h] [-e] [-l] type [type ...]
```

As seguintes opções são aceitas:

-h

--help

Mostra a mensagem de ajuda e sai.

-e

--extension

Guess extension instead of type.

-l

--lenient

Additionally search for some common, but non-standard types.

By default the script converts MIME types to file extensions. However, if `--extension` is specified, it converts file extensions to MIME types.

For each `type` entry, the script writes a line into the standard output stream. If an unknown type occurs, it writes an error message into the standard error stream and exits with the return code `1`.

20.4.3 Exemplos na linha de comando

Here are some examples of typical usage of the `mimetypes` command-line interface:

```
$ # get a MIME type by a file name
$ python -m mimetypes filename.png
type: image/png encoding: None

$ # get a MIME type by a URL
$ python -m mimetypes https://example.com/filename.txt
type: text/plain encoding: None

$ # get a complex MIME type
$ python -m mimetypes filename.tar.gz
type: application/x-tar encoding: gzip

$ # get a MIME type for a rare file extension
$ python -m mimetypes filename.pict
error: unknown extension of filename.pict

$ # now look in the extended database built into Python
$ python -m mimetypes --lenient filename.pict
type: image/pict encoding: None

$ # get a file extension by a MIME type
$ python -m mimetypes --extension text/javascript
.js
```

(continua na próxima página)

(continuação da página anterior)

```
$ # get a file extension by a rare MIME type
$ python -m mimetypes --extension text/xul
error: unknown type text/xul

$ # now look in the extended database again
$ python -m mimetypes --extension --lenient text/xul
.xul

$ # try to feed an unknown file extension
$ python -m mimetypes filename.sh filename.nc filename.xxx filename.txt
type: application/x-sh encoding: None
type: application/x-netcdf encoding: None
error: unknown extension of filename.xxx

$ # try to feed an unknown MIME type
$ python -m mimetypes --extension audio/aac audio/opus audio/future audio/x-wav
.aac
.opus
error: unknown type audio/future
```

20.5 `base64` — Base16, Base32, Base64, Base85 Data Encodings

Código-fonte: Lib/base64.py

This module provides functions for encoding binary data to printable ASCII characters and decoding such encodings back to binary data. This includes the *encodings specified in* **RFC 4648** (Base64, Base32 and Base16) and the non-standard *Base85 encodings*.

Existem duas interfaces fornecidas por este módulo. A interface moderna oferece suporte a codificar *objetos bytes ou similares* para `bytes` ASCII, e decodificar *objetos bytes ou similares* ou strings contendo ASCII para `bytes`. Ambos os alfabetos de base 64 definidos em **RFC 4648** (normal e seguro para URL e sistema de arquivos) são suportados.

The *legacy interface* does not support decoding from strings, but it does provide functions for encoding and decoding to and from *file objects*. It only supports the Base64 standard alphabet, and it adds newlines every 76 characters as per **RFC 2045**. Note that if you are looking for **RFC 2045** support you probably want to be looking at the `email` package instead.

Alterado na versão 3.3: Strings Unicode exclusivamente ASCII agora são aceitas pelas funções de decodificação da interface moderna.

Alterado na versão 3.4: Quaisquer *objetos bytes ou similares* agora são aceitos por todas as funções de codificação e decodificação neste módulo. Adicionado suporte a ASCII85/Base85.

20.5.1 RFC 4648 Encodings

The **RFC 4648** encodings are suitable for encoding binary data so that it can be safely sent by email, used as parts of URLs, or included as part of an HTTP POST request.

`base64.`**`b64encode`**(*s*, *altchars=None*)

Codifica o *objeto bytes ou similar* *s* usando Base64 e retorna o `bytes` codificado.

Optional *altchars* must be a *bytes-like object* of length 2 which specifies an alternative alphabet for the `+` and `/` characters. This allows an application to e.g. generate URL or filesystem safe Base64 strings. The default is `None`, for which the standard Base64 alphabet is used.

May assert or raise a `ValueError` if the length of *altchars* is not 2. Raises a `TypeError` if *altchars* is not a *bytes-like object*.

base64.**b64decode**(*s*, *altchars=None*, *validate=False*)

Decodifica o *objeto bytes ou similar* ou string ASCII *s* codificada em Base64 e retorna o *bytes* decodificado.

Optional *altchars* must be a *bytes-like object* or ASCII string of length 2 which specifies the alternative alphabet used instead of the + and / characters.

A *binascii.Error* exception is raised if *s* is incorrectly padded.

If *validate* is `False` (the default), characters that are neither in the normal base-64 alphabet nor the alternative alphabet are discarded prior to the padding check. If *validate* is `True`, these non-alphabet characters in the input result in a *binascii.Error*.

For more information about the strict base64 check, see *binascii.a2b_base64()*

May assert or raise a *ValueError* if the length of *altchars* is not 2.

base64.**standard_b64encode**(*s*)

Encode *bytes-like object* *s* using the standard Base64 alphabet and return the encoded *bytes*.

base64.**standard_b64decode**(*s*)

Decode *bytes-like object* or ASCII string *s* using the standard Base64 alphabet and return the decoded *bytes*.

base64.**urlsafe_b64encode**(*s*)

Encode *bytes-like object* *s* using the URL- and filesystem-safe alphabet, which substitutes - instead of + and _ instead of / in the standard Base64 alphabet, and return the encoded *bytes*. The result can still contain =.

base64.**urlsafe_b64decode**(*s*)

Decode *bytes-like object* or ASCII string *s* using the URL- and filesystem-safe alphabet, which substitutes - instead of + and _ instead of / in the standard Base64 alphabet, and return the decoded *bytes*.

base64.**b32encode**(*s*)

Encode the *bytes-like object* *s* using Base32 and return the encoded *bytes*.

base64.**b32decode**(*s*, *casefold=False*, *map01=None*)

Decode the Base32 encoded *bytes-like object* or ASCII string *s* and return the decoded *bytes*.

casefold opcional é uma flag especificando se um alfabeto minúsculo é aceitável como entrada. Por razões de segurança, o padrão é `False`.

RFC 4648 allows for optional mapping of the digit 0 (zero) to the letter O (oh), and for optional mapping of the digit 1 (one) to either the letter I (eye) or letter L (el). The optional argument *map01* when not `None`, specifies which letter the digit 1 should be mapped to (when *map01* is not `None`, the digit 0 is always mapped to the letter O). For security purposes the default is `None`, so that 0 and 1 are not allowed in the input.

A *binascii.Error* is raised if *s* is incorrectly padded or if there are non-alphabet characters present in the input.

base64.**b32hexencode**(*s*)

Similar to *b32encode()* but uses the Extended Hex Alphabet, as defined in **RFC 4648**.

Adicionado na versão 3.10.

base64.**b32hexdecode**(*s*, *casefold=False*)

Similar to *b32decode()* but uses the Extended Hex Alphabet, as defined in **RFC 4648**.

This version does not allow the digit 0 (zero) to the letter O (oh) and digit 1 (one) to either the letter I (eye) or letter L (el) mappings, all these characters are included in the Extended Hex Alphabet and are not interchangeable.

Adicionado na versão 3.10.

base64.**b16encode**(*s*)

Encode the *bytes-like object* *s* using Base16 and return the encoded *bytes*.

base64.**b16decode**(*s*, *casefold=False*)

Decode the Base16 encoded *bytes-like object* or ASCII string *s* and return the decoded `bytes`.

casefold opcional é uma flag especificando se um alfabeto minúsculo é aceitável como entrada. Por razões de segurança, o padrão é `False`.

A `binascii.Error` is raised if *s* is incorrectly padded or if there are non-alphabet characters present in the input.

20.5.2 Base85 Encodings

Base85 encoding is not formally specified but rather a de facto standard, thus different systems perform the encoding differently.

The `a85encode()` and `b85encode()` functions in this module are two implementations of the de facto standard. You should call the function with the Base85 implementation used by the software you intend to work with.

The two functions present in this module differ in how they handle the following:

- Whether to include enclosing <~ and ~> markers
- Whether to include newline characters
- The set of ASCII characters used for encoding
- Handling of null bytes

Refer to the documentation of the individual functions for more information.

base64.**a85encode**(*b*, ***, *foldspaces=False*, *wrapcol=0*, *pad=False*, *adobe=False*)

Encode the *bytes-like object* *b* using Ascii85 and return the encoded `bytes`.

foldspaces is an optional flag that uses the special short sequence 'y' instead of 4 consecutive spaces (ASCII 0x20) as supported by 'btoa'. This feature is not supported by the "standard" Ascii85 encoding.

wrapcol controls whether the output should have newline (`b'\n'`) characters added to it. If this is non-zero, each output line will be at most this many characters long, excluding the trailing newline.

pad controls whether the input is padded to a multiple of 4 before encoding. Note that the `btoa` implementation always pads.

adobe controls whether the encoded byte sequence is framed with <~ and ~>, which is used by the Adobe implementation.

Adicionado na versão 3.4.

base64.**a85decode**(*b*, ***, *foldspaces=False*, *adobe=False*, *ignorechars=b' \t\n\r\x0b'*)

Decode the Ascii85 encoded *bytes-like object* or ASCII string *b* and return the decoded `bytes`.

foldspaces is a flag that specifies whether the 'y' short sequence should be accepted as shorthand for 4 consecutive spaces (ASCII 0x20). This feature is not supported by the "standard" Ascii85 encoding.

adobe controla se a entrada está no formato Adobe Ascii85 (ou seja, cercada por <~ e ~>).

ignorechars should be a *bytes-like object* or ASCII string containing characters to ignore from the input. This should only contain whitespace characters, and by default contains all whitespace characters in ASCII.

Adicionado na versão 3.4.

base64.**b85encode**(*b*, *pad=False*)

Encode the *bytes-like object* *b* using base85 (as used in e.g. git-style binary diffs) and return the encoded `bytes`.

If *pad* is true, the input is padded with `b'\0'` so its length is a multiple of 4 bytes before encoding.

Adicionado na versão 3.4.

base64.**b85decode**(*b*)

Decode the base85-encoded *bytes-like object* or ASCII string *b* and return the decoded `bytes`. Padding is implicitly removed, if necessary.

Adicionado na versão 3.4.

base64.**z85encode**(*s*)

Encode the *bytes-like object* *s* using Z85 (as used in ZeroMQ) and return the encoded `bytes`. See Z85 specification for more information.

Adicionado na versão 3.13.

base64.**z85decode**(*s*)

Decode the Z85-encoded *bytes-like object* or ASCII string *s* and return the decoded `bytes`. See Z85 specification for more information.

Adicionado na versão 3.13.

20.5.3 Legacy Interface

base64.**decode**(*input*, *output*)

Decode the contents of the binary *input* file and write the resulting binary data to the *output* file. *input* and *output* must be *file objects*. *input* will be read until `input.readline()` returns an empty bytes object.

base64.**decodebytes**(*s*)

Decode the *bytes-like object* *s*, which must contain one or more lines of base64 encoded data, and return the decoded `bytes`.

Adicionado na versão 3.1.

base64.**encode**(*input*, *output*)

Encode the contents of the binary *input* file and write the resulting base64 encoded data to the *output* file. *input* and *output* must be *file objects*. *input* will be read until `input.read()` returns an empty bytes object. `encode()` inserts a newline character (`b'\n'`) after every 76 bytes of the output, as well as ensuring that the output always ends with a newline, as per **RFC 2045** (MIME).

base64.**encodebytes**(*s*)

Encode the *bytes-like object* *s*, which can contain arbitrary binary data, and return `bytes` containing the base64-encoded data, with newlines (`b'\n'`) inserted after every 76 bytes of output, and ensuring that there is a trailing newline, as per **RFC 2045** (MIME).

Adicionado na versão 3.1.

Um exemplo de uso do módulo:

```
>>> import base64
>>> encoded = base64.b64encode(b'data to be encoded')
>>> encoded
b'ZGF0YSBoYBiZSBlbmNvZGVk'
>>> data = base64.b64decode(encoded)
>>> data
b'data to be encoded'
```

20.5.4 Considerações de Segurança

A new security considerations section was added to **RFC 4648** (section 12); it's recommended to review the security section for any code deployed to production.

Ver também

Módulo `binascii`
Módulo de suporte contendo conversões ASCII para binário e binário para ASCII.

RFC 1521 - MIME (Multipurpose Internet Mail Extensions) Part One: Mechanisms for Specifying and Describing the Format of Internet Message Bodies
Section 5.2, "Base64 Content-Transfer-Encoding," provides the definition of the base64 encoding.

20.6 `binascii` — Converte entre binário e ASCII

O módulo `binascii` contém vários métodos para converter entre binário e várias representações binárias codificadas em ASCII. Normalmente, você não usará essas funções diretamente, mas usará módulos invólucros como `base64` em vez disso. O módulo `binascii` contém funções de baixo nível escritas em C para maior velocidade que são usadas pelos módulos de nível mais alto.

Nota

Funções `a2b_*` aceitam strings Unicode contendo apenas caracteres ASCII. Outras funções aceitam apenas *objetos bytes ou similares* (como `bytes`, `bytearray` e outros objetos que suportam o protocolo buffer).

Alterado na versão 3.3: Strings unicode exclusivamente ASCII agora são aceitas pelas funções `a2b_*`.

O módulo `binascii` define as seguintes funções:

`binascii.`**`a2b_uu`**(*string*)

Converte uma única linha de dados uuencoded de volta para binário e retorna os dados binários. As linhas normalmente contêm 45 bytes (binários), exceto a última linha. Os dados da linha podem ser seguidos por espaços em branco.

`binascii.`**`b2a_uu`**(*data*, *, *backtick=False*)

Converte dados binários para uma linha de caracteres ASCII, o valor de retorno é a linha convertida, incluindo um caractere de nova linha. O comprimento de *data* deve ser no máximo 45. Se *backtick* for true, zeros são representados por `` '`' `` em vez de espaços.

Alterado na versão 3.7: Adicionado o parâmetro *backtick*.

`binascii.`**`a2b_base64`**(*string*, /, *, *strict_mode=False*)

Converte um bloco de dados base64 de volta para binário e retorna os dados binários. Mais de uma linha pode ser passada por vez.

Se *strict_mode* for true, somente dados base64 válidos serão convertidos. Dados base64 inválidos levantarão `binascii.Error`.

base64 válido:

- Em conformidade com **RFC 3548**.
- Contém apenas caracteres do alfabeto base64.
- Não contém dados excedentes após o preenchimento (incluindo excesso de preenchimento, novas linhas, etc.).
- Não começa com um preenchimento.

Alterado na versão 3.11: Adicionado o parâmetro *strict_mode*.

`binascii.`**`b2a_base64`**(*data*, *, *newline=True*)

Converte dados binários para uma linha de caracteres ASCII em codificação base64. O valor de retorno é a linha convertida, incluindo um caractere de nova linha se *newline* for verdadeiro. A saída desta função está em conformidade com **RFC 3548**.

Alterado na versão 3.6: Adicionado o parâmetro *newline*.

binascii.**a2b_qp**(*data*, *header=False*)

Convert a block of quoted-printable data back to binary and return the binary data. More than one line may be passed at a time. If the optional argument *header* is present and true, underscores will be decoded as spaces.

binascii.**b2a_qp**(*data*, *quotetabs=False*, *istext=True*, *header=False*)

Convert binary data to a line(s) of ASCII characters in quoted-printable encoding. The return value is the converted line(s). If the optional argument *quotetabs* is present and true, all tabs and spaces will be encoded. If the optional argument *istext* is present and true, newlines are not encoded but trailing whitespace will be encoded. If the optional argument *header* is present and true, spaces will be encoded as underscores per RFC 1522. If the optional argument *header* is present and false, newline characters will be encoded as well; otherwise linefeed conversion might corrupt the binary data stream.

binascii.**crc_hqx**(*data*, *value*)

Compute a 16-bit CRC value of *data*, starting with *value* as the initial CRC, and return the result. This uses the CRC-CCITT polynomial $x^{16} + x^{12} + x^5 + 1$, often represented as 0x1021. This CRC is used in the binhex4 format.

binascii.**crc32**(*data*[, *value*])

Compute CRC-32, the unsigned 32-bit checksum of *data*, starting with an initial CRC of *value*. The default initial CRC is zero. The algorithm is consistent with the ZIP file checksum. Since the algorithm is designed for use as a checksum algorithm, it is not suitable for use as a general hash algorithm. Use as follows:

```
print(binascii.crc32(b"hello world"))
# Or, in two pieces:
crc = binascii.crc32(b"hello")
crc = binascii.crc32(b" world", crc)
print('crc32 = {:#010x}'.format(crc))
```

Alterado na versão 3.0: O resultado é sempre sem sinal.

binascii.**b2a_hex**(*data*[, *sep*[, *bytes_per_sep=1*]])

binascii.**hexlify**(*data*[, *sep*[, *bytes_per_sep=1*]])

Return the hexadecimal representation of the binary *data*. Every byte of *data* is converted into the corresponding 2-digit hex representation. The returned bytes object is therefore twice as long as the length of *data*.

Similar functionality (but returning a text string) is also conveniently accessible using the *bytes.hex()* method.

If *sep* is specified, it must be a single character str or bytes object. It will be inserted in the output after every *bytes_per_sep* input bytes. Separator placement is counted from the right end of the output by default, if you wish to count from the left, supply a negative *bytes_per_sep* value.

```
>>> import binascii
>>> binascii.b2a_hex(b'\xb9\x01\xef')
b'b901ef'
>>> binascii.hexlify(b'\xb9\x01\xef', '-')
b'b9-01-ef'
>>> binascii.b2a_hex(b'\xb9\x01\xef', b'_', 2)
b'b9_01ef'
>>> binascii.b2a_hex(b'\xb9\x01\xef', b' ', -2)
b'b901 ef'
```

Alterado na versão 3.8: The *sep* and *bytes_per_sep* parameters were added.

binascii.**a2b_hex**(*hexstr*)

binascii.**unhexlify**(*hexstr*)

Return the binary data represented by the hexadecimal string *hexstr*. This function is the inverse of *b2a_hex()*. *hexstr* must contain an even number of hexadecimal digits (which can be upper or lower case), otherwise an *Error* exception is raised.

Similar functionality (accepting only text string arguments, but more liberal towards whitespace) is also accessible using the `bytes.fromhex()` class method.

exception binascii.**Error**

Exception raised on errors. These are usually programming errors.

exception binascii.**Incomplete**

Exception raised on incomplete data. These are usually not programming errors, but may be handled by reading a little more data and trying again.

Ver também

Módulo ***base64***
Support for RFC compliant base64-style encoding in base 16, 32, 64, and 85.

Módulo ***quopri***
Support for quoted-printable encoding used in MIME email messages.

20.7 `quopri` — Codifica e decodifica dados MIME imprimidos entre aspas

Código-fonte: Lib/quopri.py

Este módulo realiza codificação e decodificação de transporte imprimida entre aspas, como definido em **RFC 1521**: "MIME (Multipurpose Internet Mail Extensions) Part One: Mechanisms for Specifying and Describing the Format of Internet Message Bodies". A codificação imprimida entre aspas é projetada para dados em que há relativamente poucos caracteres não imprimíveis; o esquema de codificação base64 disponível através do módulo `base64` é mais compacto se existirem muitos desses caracteres, como no envio de um arquivo gráfico.

quopri.**decode**(*input*, *output*, *header=False*)

Decodifica o conteúdo do arquivo *input* e escreve os dados binários decodificados resultantes no arquivo *output*. *input* e *output* devem ser *objetos arquivos binários*. Se o argumento opcional *header* estiver presente e for true, o sublinhado será decodificado como espaço. Isso é usado para decodificar cabeçalhos codificados em "Q", conforme descrito em **RFC 1522**: "MIME (Multipurpose Internet Mail Extensions) Part Two: Message Header Extensions for Non-ASCII Text".

quopri.**encode**(*input*, *output*, *quotetabs*, *header=False*)

Codifica o conteúdo do arquivo *input* e grava os dados imprimíveis entre aspas resultantes no arquivo *output*. *input* e *output* devem ser *objetos arquivos binários*. *quotetabs*, um sinalizador não opcional que controla a codificação de espaços e tabulações incorporados; quando true, codifica esses espaços em branco incorporados e, quando false, os deixa sem codificação. Observe que os espaços e tabulações que aparecem no final das linhas são sempre codificados, conforme **RFC 1521**. *header* é um sinalizador que controla se os espaços são codificados como sublinhados, conforme **RFC 1522**.

quopri.**decodestring**(*s*, *header=False*)

Como `decode()`, exceto pelo fato de aceitar uma fonte `bytes` e retornar o correspondente decodificado `bytes`.

quopri.**encodestring**(*s*, *quotetabs=False*, *header=False*)

Como `encode()`, exceto pelo fato de aceitar uma fonte `bytes` e retornar o `bytes` codificado correspondente. Por padrão, envia um valor `False` para o parâmetro *quotetabs* da função `encode()`.

Ver também

Módulo ***base64***
Codifica e decodifica dados de base64 MIME

CAPÍTULO 21

Ferramentas de Processamento de Markup Estruturado

O Python suporta uma variedade de módulos para trabalhar com vários formatos de marcação de dados estruturados. Isso inclui módulos para trabalhar com o Standard Generalized Markup Language (SGML) e o Hypertext Markup Language (HTML) e várias interfaces para trabalhar com o XML (Extensible Markup Language).

21.1 `html` — Suporte HTML (HyperText Markup Language)

Código-fonte: Lib/html/__init__.py

Este módulo define utilitários para manipular HTML.

`html.`**`escape`**(*s*, *quote=True*)

Converte os caracteres `&`, < e > na string *s* para sequências seguras em HTML. Use se necessitar exibir texto que possa conter estes caracteres no HTML. Se o flag opcional *quote* é verdadeiro (padrão), os caracteres (`"`) e (`'`) também são convertidos; isso auxilia na inclusão de valores delimitados por aspas num atributo HTML, como em `<a href="...">`. Se *quote* for definido para falso, os caracteres (`"`) e (`'`) não são traduzidos.

Adicionado na versão 3.2.

`html.`**`unescape`**(*s*)

Converte todas as referências de caracteres numéricos e nomeados (ex. `>`, `>`, `>`) na string *s* para caracteres Unicode correspondentes. Essa função usa as regras definidas pelo padrão HTML 5 para referências de caracteres, sejam válidas ou inválidas, e a *lista de referência de caracteres nomeados do HTML 5*.

Adicionado na versão 3.4.

Sub módulos no pacote `html` são:

- *`html.parser`* – analisador HTML/XHTML com modo de análise branda
- *`html.entities`* – definições das entidade HTML

21.2 `html.parser` — Simple HTML and XHTML parser

Código-fonte: Lib/html/parser.py

This module defines a class *HTMLParser* which serves as the basis for parsing text files formatted in HTML (HyperText Mark-up Language) and XHTML.

class `html.parser.`**HTMLParser**(*, *convert_charrefs=True*)

Create a parser instance able to parse invalid markup.

If *convert_charrefs* is `True` (the default), all character references (except the ones in `script`/`style` elements) are automatically converted to the corresponding Unicode characters.

An *HTMLParser* instance is fed HTML data and calls handler methods when start tags, end tags, text, comments, and other markup elements are encountered. The user should subclass *HTMLParser* and override its methods to implement the desired behavior.

This parser does not check that end tags match start tags or call the end-tag handler for elements which are closed implicitly by closing an outer element.

Alterado na versão 3.4: *convert_charrefs* keyword argument added.

Alterado na versão 3.5: The default value for argument *convert_charrefs* is now `True`.

21.2.1 Example HTML Parser Application

As a basic example, below is a simple HTML parser that uses the *HTMLParser* class to print out start tags, end tags, and data as they are encountered:

```
from html.parser import HTMLParser

class MyHTMLParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print("Encountered a start tag:", tag)

    def handle_endtag(self, tag):
        print("Encountered an end tag :", tag)

    def handle_data(self, data):
        print("Encountered some data  :", data)

parser = MyHTMLParser()
parser.feed('<html><head><title>Test</title></head>'
            '<body><h1>Parse me!</h1></body></html>')
```

The output will then be:

```
Encountered a start tag: html
Encountered a start tag: head
Encountered a start tag: title
Encountered some data  : Test
Encountered an end tag : title
Encountered an end tag : head
Encountered a start tag: body
Encountered a start tag: h1
Encountered some data  : Parse me!
Encountered an end tag : h1
Encountered an end tag : body
Encountered an end tag : html
```

21.2.2 `HTMLParser` Methods

HTMLParser instances have the following methods:

HTMLParser.**feed**(*data*)

Feed some text to the parser. It is processed insofar as it consists of complete elements; incomplete data is buffered until more data is fed or *close()* is called. *data* must be *str*.

HTMLParser.**close**()

Force processing of all buffered data as if it were followed by an end-of-file mark. This method may be redefined by a derived class to define additional processing at the end of the input, but the redefined version should always call the *HTMLParser* base class method *close()*.

HTMLParser.**reset**()

Reset the instance. Loses all unprocessed data. This is called implicitly at instantiation time.

HTMLParser.**getpos**()

Return current line number and offset.

HTMLParser.**get_starttag_text**()

Return the text of the most recently opened start tag. This should not normally be needed for structured processing, but may be useful in dealing with HTML "as deployed" or for re-generating input with minimal changes (whitespace between attributes can be preserved, etc.).

The following methods are called when data or markup elements are encountered and they are meant to be overridden in a subclass. The base class implementations do nothing (except for *handle_startendtag()*):

HTMLParser.**handle_starttag**(*tag*, *attrs*)

This method is called to handle the start tag of an element (e.g. `<div id="main">`).

The *tag* argument is the name of the tag converted to lower case. The *attrs* argument is a list of `(name, value)` pairs containing the attributes found inside the tag's <> brackets. The *name* will be translated to lower case, and quotes in the *value* have been removed, and character and entity references have been replaced.

For instance, for the tag `<A HREF="https://www.cwi.nl/">`, this method would be called as `handle_starttag('a', [('href', 'https://www.cwi.nl/')])`.

All entity references from *html.entities* are replaced in the attribute values.

HTMLParser.**handle_endtag**(*tag*)

This method is called to handle the end tag of an element (e.g. `</div>`).

The *tag* argument is the name of the tag converted to lower case.

HTMLParser.**handle_startendtag**(*tag*, *attrs*)

Similar to *handle_starttag()*, but called when the parser encounters an XHTML-style empty tag (`<img ... />`). This method may be overridden by subclasses which require this particular lexical information; the default implementation simply calls *handle_starttag()* and *handle_endtag()*.

HTMLParser.**handle_data**(*data*)

This method is called to process arbitrary data (e.g. text nodes and the content of `<script>...</script>` and `<style>...</style>`).

HTMLParser.**handle_entityref**(*name*)

This method is called to process a named character reference of the form `&name;` (e.g. `>`), where *name* is a general entity reference (e.g. `'gt'`). This method is never called if *convert_charrefs* is `True`.

HTMLParser.**handle_charref**(*name*)

This method is called to process decimal and hexadecimal numeric character references of the form `&#NNN;` and `&#xNNN;`. For example, the decimal equivalent for `>` is `>`, whereas the hexadecimal is `>`; in this case the method will receive `'62'` or `'x3E'`. This method is never called if *convert_charrefs* is `True`.

HTMLParser.**handle_comment**(*data*)

This method is called when a comment is encountered (e.g. `<!--comment-->`).

For example, the comment `<!-- comment -->` will cause this method to be called with the argument `' comment '`.

The content of Internet Explorer conditional comments (condcoms) will also be sent to this method, so, for `<!--[if IE 9]>IE9-specific content<![endif]-->`, this method will receive `'[if IE 9]>IE9-specific content<![endif]'`.

HTMLParser.**handle_decl**(*decl*)

This method is called to handle an HTML doctype declaration (e.g. `<!DOCTYPE html>`).

The *decl* parameter will be the entire contents of the declaration inside the `<!...>` markup (e.g. `'DOCTYPE html'`).

HTMLParser.**handle_pi**(*data*)

Method called when a processing instruction is encountered. The *data* parameter will contain the entire processing instruction. For example, for the processing instruction `<?proc color='red'>`, this method would be called as `handle_pi("proc color='red'")`. It is intended to be overridden by a derived class; the base class implementation does nothing.

> **Nota**
>
> The *HTMLParser* class uses the SGML syntactic rules for processing instructions. An XHTML processing instruction using the trailing `'?'` will cause the `'?'` to be included in *data*.

HTMLParser.**unknown_decl**(*data*)

This method is called when an unrecognized declaration is read by the parser.

The *data* parameter will be the entire contents of the declaration inside the `<![...]>` markup. It is sometimes useful to be overridden by a derived class. The base class implementation does nothing.

21.2.3 Exemplos

The following class implements a parser that will be used to illustrate more examples:

```
from html.parser import HTMLParser
from html.entities import name2codepoint

class MyHTMLParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print("Start tag:", tag)
        for attr in attrs:
            print("     attr:", attr)

    def handle_endtag(self, tag):
        print("End tag  :", tag)

    def handle_data(self, data):
        print("Data     :", data)

    def handle_comment(self, data):
        print("Comment  :", data)

    def handle_entityref(self, name):
        c = chr(name2codepoint[name])
        print("Named ent:", c)
```

(continua na próxima página)

(continuação da página anterior)

```
    def handle_charref(self, name):
        if name.startswith('x'):
            c = chr(int(name[1:], 16))
        else:
            c = chr(int(name))
        print("Num ent  :", c)

    def handle_decl(self, data):
        print("Decl     :", data)

parser = MyHTMLParser()
```

Parsing a doctype:

```
>>> parser.feed('<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" '
...             '"http://www.w3.org/TR/html4/strict.dtd">')
Decl     : DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/
↪html4/strict.dtd"
```

Parsing an element with a few attributes and a title:

```
>>> parser.feed('<img src="python-logo.png" alt="The Python logo">')
Start tag: img
     attr: ('src', 'python-logo.png')
     attr: ('alt', 'The Python logo')
>>>
>>> parser.feed('<h1>Python</h1>')
Start tag: h1
Data     : Python
End tag  : h1
```

The content of `script` and `style` elements is returned as is, without further parsing:

```
>>> parser.feed('<style type="text/css">#python { color: green }</style>')
Start tag: style
     attr: ('type', 'text/css')
Data     : #python { color: green }
End tag  : style

>>> parser.feed('<script type="text/javascript">'
...             'alert("<strong>hello!</strong>");</script>')
Start tag: script
     attr: ('type', 'text/javascript')
Data     : alert("<strong>hello!</strong>");
End tag  : script
```

Parsing comments:

```
>>> parser.feed('<!--a comment-->'
...             '<!--[if IE 9]>IE-specific content<![endif]-->')
Comment  : a comment
Comment  : [if IE 9]>IE-specific content<![endif]
```

Parsing named and numeric character references and converting them to the correct char (note: these 3 references are all equivalent to `'>'`):

```
>>> parser = MyHTMLParser()
>>> parser.feed('&gt;&#62;&#x3E;')
Data     : >>>

>>> parser = MyHTMLParser(convert_charrefs=False)
>>> parser.feed('&gt;&#62;&#x3E;')
Named ent: >
Num ent  : >
Num ent  : >
```

Feeding incomplete chunks to *feed()* works, but *handle_data()* might be called more than once (unless *convert_charrefs* is set to `True`):

```
>>> for chunk in ['<sp', 'an>buff', 'ered', ' text</s', 'pan>']:
...     parser.feed(chunk)
...
Start tag: span
Data     : buff
Data     : ered
Data     :  text
End tag  : span
```

Parsing invalid HTML (e.g. unquoted attributes) also works:

```
>>> parser.feed('<p><a class=link href=#main>tag soup</p ></a>')
Start tag: p
Start tag: a
     attr: ('class', 'link')
     attr: ('href', '#main')
Data     : tag soup
End tag  : p
End tag  : a
```

21.3 `html.entities` — Definições de entidades gerais de HTML

Código-fonte: Lib/html/entities.py

Esse módulo define quatro dicionários, *html5*, *name2codepoint*, *codepoint2name* e *entitydefs*.

`html.entities.`**`html5`**

Um dicionário que mapeia referências de caracteres nomeados em HTML5[1] para os caracteres Unicode equivalentes, por exemplo, `html5['gt;'] == '>'`. Note que o caractere de ponto e vírgula final está incluído no nome (por exemplo, `'gt;'`), entretanto alguns dos nomes são aceitos pelo padrão mesmo sem o ponto e vírgula: neste caso o nome está presente com e sem o `';'`. Veja também *html.unescape()*.

Adicionado na versão 3.3.

`html.entities.`**`entitydefs`**

Um dicionário que mapeia as definições de entidade XHTML 1.0 para seu texto substituto em ISO Latin-1.

`html.entities.`**`name2codepoint`**

Um dicionário que mapeia nomes de entidades HTML4 para os pontos de código Unicode.

`html.entities.`**`codepoint2name`**

Um dicionário que mapeia pontos de código Unicode para nomes de entidades HTML4.

[1] Veja https://html.spec.whatwg.org/multipage/named-characters.html#named-character-references

21.4 Módulos de Processamento de XML

Código-fonte: Lib/xml/

As interfaces do Python para processar XML estão agrupadas no pacote `xml`.

> **Nota**
>
> Se você precisar analisar dados não confiáveis ou não autenticados, consulte *Segurança no XML*.

É importante observar que os módulos no pacote `xml` exigem que está disponível pelo menos um analisador sintático XML compatível com SAX. O analisador sintático Expat está incluído no Python, então o módulo `xml.parsers.expat` estará sempre disponível.

A documentação para os pacotes `xml.dom` e `xml.sax` são a definição das ligações Python para as interfaces DOM e SAX.

Os submódulos de manipulação XML são:

- `xml.etree.ElementTree`: a API de ElementTree, um processador XML simples e leve
- `xml.dom`: a definição da API de DOM
- `xml.dom.minidom`: uma implementação mínima do DOM
- suporte para construir árvores parciais de DOM no `xml.dom.pulldom`
- `xml.sax`: Classe base SAX2 e funções de conveniência
- `xml.parsers.expat`: a ligação do analisador sintático Expat

21.4.1 Segurança no XML

Um atacante pode abusar dos recursos XML para realizar ataques de negação de serviço, acessar arquivos locais, gerar conexões de rede com outras máquinas ou contornar firewalls.

Versões do Expat anteriores à 2.6.0 podem ser vulneráveis a “billion laughs”, “quadratic blowup” e “large tokens”. O Python pode ser vulnerável se usar versões mais antigas do Expat como biblioteca fornecida pelo sistema. Verifique `pyexpat.EXPAT_VERSION`.

`xmlrpc` é **vulnerável** ao ataque “decompression bomb”.

billion laughs / exponential entity expansion
: O ataque Billion Laughs (bilhões de risadas, em uma tradução livre) - também conhecido como “exponential entity expansion” (expansão exponencial de entidades, em uma tradução livre) - usa vários níveis de entidades aninhadas. Cada entidade se refere a outra entidade várias vezes, e a definição final da entidade contém uma pequena string. A expansão exponencial resulta em vários gigabytes de texto e consome muita memória e tempo de CPU.

quadratic blowup entity expansion
: Um ataque “quadratic blowup” (explosão quadrática, em uma tradução livre) português) é semelhante a um ataque Billion Laughs; ele abusa da expansão de entidades também. Em vez de entidades aninhadas, ele repete uma grande entidade com alguns milhares de caracteres repetidamente. O ataque não é tão eficiente quanto o caso exponencial, mas evita acionar contramedidas do analisador sintático que proíbem entidades profundamente aninhadas.

decompression bomb
: “Bombas de descompressão” em uma tradução livre. Também conhecidas como ZIP bomb, se aplicam a todas as bibliotecas XML que podem analisar fluxos XML comprimidos, como fluxos de HTTP compactados com gzip ou arquivos comprimidos com LZMA. Para um atacante, isso pode reduzir a quantidade de dados transmitidos em três magnitudes ou mais.

large tokens
: Expat precisa analisar novamente os símbolos não finalizados; sem a proteção introduzida no Expat 2.6.0, isso pode levar a um tempo de execução quadrático que pode ser usado para causar negação de serviço na aplicação ao analisar XML. O problema é conhecido como **CVE 2023-52425**.

21.5 `xml.etree.ElementTree` — A API XML ElementTree

Código-fonte: Lib/xml/etree/ElementTree.py

O módulo *`xml.etree.ElementTree`* implementa uma API simples e eficiente para análise e criação de dados XML.

Alterado na versão 3.3: Este módulo usará uma implementação rápida sempre que disponível.

Descontinuado desde a versão 3.3: O módulo `xml.etree.cElementTree` foi descontinuado.

Nota

Se você precisar analisar dados não confiáveis ou não autenticados, consulte *Segurança no XML*.

21.5.1 Tutorial

Esse é um tutorial curto para usar *`xml.etree.ElementTree`* (`ET` na versão resumida). O objetivo é demonstrar alguns conceitos básicos e trechos de códigos do módulo.

Árvore e elementos XML

XML é um formato de dados estritamente hierárquico, e a maneira mais natural de representá-lo é como uma árvore. `ET` possui duas classes para esse propósito - *`ElementTree`* representa todo o documento XML como uma árvore e *`Element`* representa um único nó desta árvore. Interações com o documento inteiro (ler e escrever de/para arquivos) são frequentemente feitos em nível de *`ElementTree`*. Interações com um único elemento XML e seus subelementos são feitos a nível de *`Element`* .

Analisando XML

Usaremos o documento XML fictício `country_data.xml` como dados de amostra para esta seção:

```
<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank>1</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank>4</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
    <country name="Panama">
        <rank>68</rank>
        <year>2011</year>
        <gdppc>13600</gdppc>
```

(continua na próxima página)

(continuação da página anterior)

```
        <neighbor name="Costa Rica" direction="W"/>
        <neighbor name="Colombia" direction="E"/>
    </country>
</data>
```

Nós podemos importar esses dados lendo de um arquivo:

```
import xml.etree.ElementTree as ET
tree = ET.parse('country_data.xml')
root = tree.getroot()
```

Ou diretamente de uma string:

```
root = ET.fromstring(country_data_as_string)
```

fromstring() obtém o XML de uma string e armazena em um *Element*, que será o elemento raiz dessa árvore. Outras funções de análise sintática podem criar um *ElementTree*. Cheque a documentação para se certificar sobre qual dado será retornado.

Assim como um *Element*, `root` tem uma tag e um dicionário de atributos:

```
>>> root.tag
'data'
>>> root.attrib
{}
```

Ele também tem nós filhos sobre os quais nós podemos iterar:

```
>>> for child in root:
...     print(child.tag, child.attrib)
...
country {'name': 'Liechtenstein'}
country {'name': 'Singapore'}
country {'name': 'Panama'}
```

Nós filhos são os mais próximos, e nós podemos acessar nós específicos por índices:

```
>>> root[0][1].text
'2008'
```

Nota

Nem todos os elementos da entrada XML acabarão como elementos da árvore analisada. Atualmente, este módulo ignora quaisquer comentários XML, instruções de processamento e declarações de tipo de documento na entrada. No entanto, árvores construídas usando a API deste módulo, em vez de serem analisadas a partir de texto XML, podem conter comentários e instruções de processamento; eles serão incluídos ao gerar a saída XML. Uma declaração de tipo de documento pode ser acessada passando uma instância personalizada *TreeBuilder* para o construtor *XMLParser*.

A API de pull para análise sem bloqueio

A maioria das funções de análise fornecidas por este módulo exigem que todo o documento seja lido de uma só vez antes de retornar qualquer resultado. É possível usar um *XMLParser* e alimentar dados nele de forma incremental, mas é uma API de push que chama métodos em um destino da função de retorno, o que é de muito baixo nível e inconveniente para a maioria das necessidades. Às vezes, o que o usuário realmente deseja é ser capaz de analisar XML de forma incremental, sem operações bloqueantes, enquanto desfruta da conveniência de objetos *Element* totalmente construídos.

A ferramenta mais poderosa para fazer isso é *XMLPullParser*. Ela não requer uma leitura bloqueante para obter os dados XML e, em vez disso, é alimentada com dados de forma incremental com chamadas de *XMLPullParser.feed()*. Para obter os elementos XML analisados, chame *XMLPullParser.read_events()*. Aqui está um exemplo:

```
>>> parser = ET.XMLPullParser(['start', 'end'])
>>> parser.feed('<mytag>sometext')
>>> list(parser.read_events())
[('start', <Element 'mytag' at 0x7fa66db2be58>)]
>>> parser.feed(' more text</mytag>')
>>> for event, elem in parser.read_events():
...     print(event)
...     print(elem.tag, 'text=', elem.text)
...
end
mytag text= sometext more text
```

O caso de uso óbvio são aplicações que operam sem bloqueio, onde os dados XML são recebidos de um soquete ou lidos de forma incremental de algum dispositivo de armazenamento. Nesses casos, leituras bloqueantes são inaceitáveis.

Por ser tão flexível, *XMLPullParser* pode ser inconveniente de usar em casos de uso mais simples. Se você não se importa que sua aplicação bloqueie a leitura de dados XML, mas ainda assim gostaria de ter recursos de análise incremental, dê uma olhada em *iterparse()*. Pode ser útil quando você está lendo um documento XML grande e não deseja mantê-lo totalmente na memória.

Onde o feedback *imediato* através de eventos é desejado, chamar o método *XMLPullParser.flush()* pode ajudar a reduzir o atraso; certifique-se de estudar as notas de segurança relacionadas.

Encontrando elementos interessantes

Element possui alguns métodos úteis que ajudam a iterar recursivamente sobre toda a subárvore abaixo dele (seus filhos, seus filhos e assim por diante). Por exemplo, *Element.iter()*:

```
>>> for neighbor in root.iter('neighbor'):
...     print(neighbor.attrib)
...
{'name': 'Austria', 'direction': 'E'}
{'name': 'Switzerland', 'direction': 'W'}
{'name': 'Malaysia', 'direction': 'N'}
{'name': 'Costa Rica', 'direction': 'W'}
{'name': 'Colombia', 'direction': 'E'}
```

Element.findall() encontra apenas elementos com uma tag que são filhos diretos do elemento atual. *Element.find()* encontra o *primeiro* filho com uma tag específica, e *Element.text* acessa o conteúdo de texto do elemento. *Element.get()* acessa os atributos do elemento:

```
>>> for country in root.findall('country'):
...     rank = country.find('rank').text
...     name = country.get('name')
...     print(name, rank)
...
Liechtenstein 1
Singapore 4
Panama 68
```

Uma especificação mais sofisticada de quais elementos procurar é possível usando *XPath*.

Modificando um arquivo XML

ElementTree fornece uma maneira simples de construir documentos XML e escrevê-los em arquivos. O método *ElementTree.write()* serve para esse propósito.

Uma vez criado, um objeto *Element* pode ser manipulado alterando diretamente seus campos (como *Element.text*), adicionando e modificando atributos (método *Element.set()*), bem como como adicionar novos filhos (por exemplo, com *Element.append()*).

Digamos que queremos adicionar um à classificação de cada país e adicionar um atributo `updated` ao elemento de classificação:

```
>>> for rank in root.iter('rank'):
...     new_rank = int(rank.text) + 1
...     rank.text = str(new_rank)
...     rank.set('updated', 'yes')
...
>>> tree.write('output.xml')
```

Nosso XML agora se parece com isto:

```
<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank updated="yes">2</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank updated="yes">5</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
    <country name="Panama">
        <rank updated="yes">69</rank>
        <year>2011</year>
        <gdppc>13600</gdppc>
        <neighbor name="Costa Rica" direction="W"/>
        <neighbor name="Colombia" direction="E"/>
    </country>
</data>
```

Podemos remover elementos usando *Element.remove()*. Digamos que queremos remover todos os países com classificação superior a 50:

```
>>> for country in root.findall('country'):
...     # using root.findall() to avoid removal during traversal
...     rank = int(country.find('rank').text)
...     if rank > 50:
...         root.remove(country)
...
>>> tree.write('output.xml')
```

Observe que a modificação simultânea durante a iteração pode levar a problemas, assim como ao iterar e modificar listas ou dicionários do Python. Portanto, o exemplo primeiro coleta todos os elementos correspondentes com `root.findall()`, e só então itera sobre a lista de correspondências.

Nosso XML agora se parece com isto:

```
<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank updated="yes">2</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank updated="yes">5</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
</data>
```

Construindo documentos XML

A função *SubElement()* também fornece uma maneira conveniente de criar novos subelementos para um determinado elemento:

```
>>> a = ET.Element('a')
>>> b = ET.SubElement(a, 'b')
>>> c = ET.SubElement(a, 'c')
>>> d = ET.SubElement(c, 'd')
>>> ET.dump(a)
<a><b /><c><d /></c></a>
```

Analisando XML com espaços de nomes

Se a entrada XML tiver espaços de nomes, tags e atributos com prefixos no formato `prefixo:algumatag` serão expandidos para `{uri}algumatag` onde *prefixo* é substituído pelo *URI* completo. Além disso, se houver um espaço de nomes padrão, esse URI completo será anexado a todas as tags não prefixadas.

Aqui está um exemplo XML que incorpora dois espaços de nomes, um com o prefixo "fictional" e outro servindo como espaço de nomes padrão:

```
<?xml version="1.0"?>
<actors xmlns:fictional="http://characters.example.com"
        xmlns="http://people.example.com">
    <actor>
        <name>John Cleese</name>
        <fictional:character>Lancelot</fictional:character>
        <fictional:character>Archie Leach</fictional:character>
    </actor>
    <actor>
        <name>Eric Idle</name>
        <fictional:character>Sir Robin</fictional:character>
        <fictional:character>Gunther</fictional:character>
        <fictional:character>Commander Clement</fictional:character>
    </actor>
</actors>
```

Uma maneira de pesquisar e explorar este exemplo XML é adicionar manualmente o URI a cada tag ou atributo no xpath de um *find()* ou *findall()*:

```
root = fromstring(xml_text)
for actor in root.findall('{http://people.example.com}actor'):
    name = actor.find('{http://people.example.com}name')
    print(name.text)
    for char in actor.findall('{http://characters.example.com}character'):
        print(' |-->', char.text)
```

A melhor maneira de pesquisar o exemplo XML com espaço de nomes é criar um dicionário com seus próprios prefixos e usá-los nas funções de pesquisa:

```
ns = {'real_person': 'http://people.example.com',
      'role': 'http://characters.example.com'}

for actor in root.findall('real_person:actor', ns):
    name = actor.find('real_person:name', ns)
    print(name.text)
    for char in actor.findall('role:character', ns):
        print(' |-->', char.text)
```

Essas duas abordagens resultam no seguinte:

```
John Cleese
 |--> Lancelot
 |--> Archie Leach
Eric Idle
 |--> Sir Robin
 |--> Gunther
 |--> Commander Clement
```

21.5.2 Suporte a XPath

Este módulo fornece suporte limitado para expressões XPath para localizar elementos em uma árvore. O objetivo é oferecer suporte a um pequeno subconjunto da sintaxe abreviada; um mecanismo XPath completo está fora do escopo do módulo.

Exemplo

Aqui está um exemplo que demonstra alguns dos recursos XPath do módulo. Estaremos usando o documento XML `countrydata` da seção *Analisando XML*:

```
import xml.etree.ElementTree as ET

root = ET.fromstring(countrydata)

# Top-level elements
root.findall(".")

# All 'neighbor' grand-children of 'country' children of the top-level
# elements
root.findall("./country/neighbor")

# Nodes with name='Singapore' that have a 'year' child
root.findall(".//year/..[@name='Singapore']")

# 'year' nodes that are children of nodes with name='Singapore'
root.findall(".//*[@name='Singapore']/year")
```

(continua na próxima página)

(continuação da página anterior)

```
# All 'neighbor' nodes that are the second child of their parent
root.findall(".//neighbor[2]")
```

Para XML com espaços de nomes, use a notação qualificada usual `{espaço-de-nomes}tag`:

```
# All dublin-core "title" tags in the document
root.findall(".//{http://purl.org/dc/elements/1.1/}title")
```

Supported XPath syntax

Sintaxe	Significado
`tag`	Selects all child elements with the given tag. For example, `spam` selects all child elements named `spam`, and `spam/egg` selects all grandchildren named `egg` in all children named `spam`. `{namespace}*` selects all tags in the given namespace, `{*}spam` selects tags named `spam` in any (or no) namespace, and `{}*` only selects tags that are not in a namespace. Alterado na versão 3.8: Support for star-wildcards was added.
`*`	Selects all child elements, including comments and processing instructions. For example, `*/egg` selects all grandchildren named `egg`.
`.`	Selects the current node. This is mostly useful at the beginning of the path, to indicate that it's a relative path.
`//`	Selects all subelements, on all levels beneath the current element. For example, `.//egg` selects all `egg` elements in the entire tree.
`..`	Selects the parent element. Returns `None` if the path attempts to reach the ancestors of the start element (the element `find` was called on).
`[@attrib]`	Selects all elements that have the given attribute.
`[@attrib='value']`	Selects all elements for which the given attribute has the given value. The value cannot contain quotes.
`[@attrib!='value']`	Selects all elements for which the given attribute does not have the given value. The value cannot contain quotes. Adicionado na versão 3.10.
`[tag]`	Selects all elements that have a child named `tag`. Only immediate children are supported.
`[.='text']`	Selects all elements whose complete text content, including descendants, equals the given `text`. Adicionado na versão 3.7.
`[.!='text']`	Selects all elements whose complete text content, including descendants, does not equal the given `text`. Adicionado na versão 3.10.
`[tag='text']`	Selects all elements that have a child named `tag` whose complete text content, including descendants, equals the given `text`.
`[tag!='text']`	Selects all elements that have a child named `tag` whose complete text content, including descendants, does not equal the given `text`. Adicionado na versão 3.10.
`[position]`	Selects all elements that are located at the given position. The position can be either an integer (1 is the first position), the expression `last()` (for the last position), or a position relative to the last position (e.g. `last()-1`).

Predicates (expressions within square brackets) must be preceded by a tag name, an asterisk, or another predicate. `position` predicates must be preceded by a tag name.

21.5.3 Referência

Funções

xml.etree.ElementTree.**canonicalize**(*xml_data=None*, *, *out=None*, *from_file=None*, **options*)

C14N 2.0 transformation function.

Canonicalization is a way to normalise XML output in a way that allows byte-by-byte comparisons and digital signatures. It reduces the freedom that XML serializers have and instead generates a more constrained XML representation. The main restrictions regard the placement of namespace declarations, the ordering of attributes, and ignorable whitespace.

This function takes an XML data string (*xml_data*) or a file path or file-like object (*from_file*) as input, converts it to the canonical form, and writes it out using the *out* file(-like) object, if provided, or returns it as a text string if not. The output file receives text, not bytes. It should therefore be opened in text mode with `utf-8` encoding.

Typical uses:

```
xml_data = "<root>...</root>"
print(canonicalize(xml_data))

with open("c14n_output.xml", mode='w', encoding='utf-8') as out_file:
    canonicalize(xml_data, out=out_file)

with open("c14n_output.xml", mode='w', encoding='utf-8') as out_file:
    canonicalize(from_file="inputfile.xml", out=out_file)
```

As *options* de configuração são as seguintes:

- *with_comments*: set to true to include comments (default: false)
- ***strip_text*: set to true to strip whitespace before and after text content**
  (default: false)
- ***rewrite_prefixes*: set to true to replace namespace prefixes by "n{number}"**
  (default: false)
- ***qname_aware_tags*: a set of qname aware tag names in which prefixes**
  should be replaced in text content (default: empty)
- ***qname_aware_attrs*: a set of qname aware attribute names in which prefixes**
  should be replaced in text content (default: empty)
- *exclude_attrs*: a set of attribute names that should not be serialised
- *exclude_tags*: a set of tag names that should not be serialised

In the option list above, "a set" refers to any collection or iterable of strings, no ordering is expected.

Adicionado na versão 3.8.

xml.etree.ElementTree.**Comment**(*text=None*)

Comment element factory. This factory function creates a special element that will be serialized as an XML comment by the standard serializer. The comment string can be either a bytestring or a Unicode string. *text* is a string containing the comment string. Returns an element instance representing a comment.

Note that *XMLParser* skips over comments in the input instead of creating comment objects for them. An *ElementTree* will only contain comment nodes if they have been inserted into to the tree using one of the *Element* methods.

xml.etree.ElementTree.**dump**(*elem*)

Writes an element tree or element structure to sys.stdout. This function should be used for debugging only.

The exact output format is implementation dependent. In this version, it's written as an ordinary XML file.

elem is an element tree or an individual element.

Alterado na versão 3.8: The *dump()* function now preserves the attribute order specified by the user.

xml.etree.ElementTree.**fromstring**(*text*, *parser=None*)

Parses an XML section from a string constant. Same as *XML()*. *text* is a string containing XML data. *parser* is an optional parser instance. If not given, the standard *XMLParser* parser is used. Returns an *Element* instance.

xml.etree.ElementTree.**fromstringlist**(*sequence*, *parser=None*)

Parses an XML document from a sequence of string fragments. *sequence* is a list or other sequence containing XML data fragments. *parser* is an optional parser instance. If not given, the standard *XMLParser* parser is used. Returns an *Element* instance.

Adicionado na versão 3.2.

xml.etree.ElementTree.**indent**(*tree*, *space=' '*, *level=0*)

Appends whitespace to the subtree to indent the tree visually. This can be used to generate pretty-printed XML output. *tree* can be an Element or ElementTree. *space* is the whitespace string that will be inserted for each indentation level, two space characters by default. For indenting partial subtrees inside of an already indented tree, pass the initial indentation level as *level*.

Adicionado na versão 3.9.

xml.etree.ElementTree.**iselement**(*element*)

Check if an object appears to be a valid element object. *element* is an element instance. Return `True` if this is an element object.

xml.etree.ElementTree.**iterparse**(*source*, *events=None*, *parser=None*)

Parses an XML section into an element tree incrementally, and reports what's going on to the user. *source* is a filename or *file object* containing XML data. *events* is a sequence of events to report back. The supported events are the strings `"start"`, `"end"`, `"comment"`, `"pi"`, `"start-ns"` and `"end-ns"` (the "ns" events are used to get detailed namespace information). If *events* is omitted, only `"end"` events are reported. *parser* is an optional parser instance. If not given, the standard *XMLParser* parser is used. *parser* must be a subclass of *XMLParser* and can only use the default *TreeBuilder* as a target. Returns an *iterator* providing `(event, elem)` pairs; it has a `root` attribute that references the root element of the resulting XML tree once *source* is fully read. The iterator has the `close()` method that closes the internal file object if *source* is a filename.

Note that while *iterparse()* builds the tree incrementally, it issues blocking reads on *source* (or the file it names). As such, it's unsuitable for applications where blocking reads can't be made. For fully non-blocking parsing, see *XMLPullParser*.

Nota

iterparse() only guarantees that it has seen the ">" character of a starting tag when it emits a "start" event, so the attributes are defined, but the contents of the text and tail attributes are undefined at that point. The same applies to the element children; they may or may not be present.

If you need a fully populated element, look for "end" events instead.

Descontinuado desde a versão 3.4: O argumento *parser*.

Alterado na versão 3.8: Os eventos `comment` e `pi` foram adicionados.

Alterado na versão 3.13: Added the `close()` method.

xml.etree.ElementTree.**parse**(*source*, *parser=None*)

Parses an XML section into an element tree. *source* is a filename or file object containing XML data. *parser* is an optional parser instance. If not given, the standard *XMLParser* parser is used. Returns an *ElementTree* instance.

xml.etree.ElementTree.**ProcessingInstruction**(*target*, *text=None*)

PI element factory. This factory function creates a special element that will be serialized as an XML processing

instruction. *target* is a string containing the PI target. *text* is a string containing the PI contents, if given. Returns an element instance, representing a processing instruction.

Note that *XMLParser* skips over processing instructions in the input instead of creating PI objects for them. An *ElementTree* will only contain processing instruction nodes if they have been inserted into to the tree using one of the *Element* methods.

xml.etree.ElementTree.**register_namespace**(*prefix*, *uri*)

Registers a namespace prefix. The registry is global, and any existing mapping for either the given prefix or the namespace URI will be removed. *prefix* is a namespace prefix. *uri* is a namespace uri. Tags and attributes in this namespace will be serialized with the given prefix, if at all possible.

Adicionado na versão 3.2.

xml.etree.ElementTree.**SubElement**(*parent*, *tag*, *attrib={}*, ***extra*)

Subelement factory. This function creates an element instance, and appends it to an existing element.

The element name, attribute names, and attribute values can be either bytestrings or Unicode strings. *parent* is the parent element. *tag* is the subelement name. *attrib* is an optional dictionary, containing element attributes. *extra* contains additional attributes, given as keyword arguments. Returns an element instance.

xml.etree.ElementTree.**tostring**(*element*, *encoding='us-ascii'*, *method='xml'*, ***, *xml_declaration=None*, *default_namespace=None*, *short_empty_elements=True*)

Generates a string representation of an XML element, including all subelements. *element* is an *Element* instance. *encoding*[1] is the output encoding (default is US-ASCII). Use `encoding="unicode"` to generate a Unicode string (otherwise, a bytestring is generated). *method* is either `"xml"`, `"html"` or `"text"` (default is `"xml"`). *xml_declaration*, *default_namespace* and *short_empty_elements* has the same meaning as in *ElementTree.write()*. Returns an (optionally) encoded string containing the XML data.

Alterado na versão 3.4: Foi adicionado o parâmetro *short_empty_elements*.

Alterado na versão 3.8: Added the *xml_declaration* and *default_namespace* parameters.

Alterado na versão 3.8: The *tostring()* function now preserves the attribute order specified by the user.

xml.etree.ElementTree.**tostringlist**(*element*, *encoding='us-ascii'*, *method='xml'*, ***, *xml_declaration=None*, *default_namespace=None*, *short_empty_elements=True*)

Generates a string representation of an XML element, including all subelements. *element* is an *Element* instance. *encoding*[1] is the output encoding (default is US-ASCII). Use `encoding="unicode"` to generate a Unicode string (otherwise, a bytestring is generated). *method* is either `"xml"`, `"html"` or `"text"` (default is `"xml"`). *xml_declaration*, *default_namespace* and *short_empty_elements* has the same meaning as in *ElementTree.write()*. Returns a list of (optionally) encoded strings containing the XML data. It does not guarantee any specific sequence, except that `b"".join(tostringlist(element)) == tostring(element)`.

Adicionado na versão 3.2.

Alterado na versão 3.4: Foi adicionado o parâmetro *short_empty_elements*.

Alterado na versão 3.8: Added the *xml_declaration* and *default_namespace* parameters.

Alterado na versão 3.8: The *tostringlist()* function now preserves the attribute order specified by the user.

xml.etree.ElementTree.**XML**(*text*, *parser=None*)

Parses an XML section from a string constant. This function can be used to embed "XML literals" in Python code. *text* is a string containing XML data. *parser* is an optional parser instance. If not given, the standard *XMLParser* parser is used. Returns an *Element* instance.

[1] The encoding string included in XML output should conform to the appropriate standards. For example, "UTF-8" is valid, but "UTF8" is not. See https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-EncodingDecl and https://www.iana.org/assignments/character-sets/character-sets.xhtml.

xml.etree.ElementTree.**XMLID**(*text*, *parser=None*)

Parses an XML section from a string constant, and also returns a dictionary which maps from element id:s to elements. *text* is a string containing XML data. *parser* is an optional parser instance. If not given, the standard *XMLParser* parser is used. Returns a tuple containing an *Element* instance and a dictionary.

21.5.4 Suporte a XInclude

This module provides limited support for XInclude directives, via the *xml.etree.ElementInclude* helper module. This module can be used to insert subtrees and text strings into element trees, based on information in the tree.

Exemplo

Here's an example that demonstrates use of the XInclude module. To include an XML document in the current document, use the `{http://www.w3.org/2001/XInclude}include` element and set the **parse** attribute to `"xml"`, and use the **href** attribute to specify the document to include.

```
<?xml version="1.0"?>
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  <xi:include href="source.xml" parse="xml" />
</document>
```

By default, the **href** attribute is treated as a file name. You can use custom loaders to override this behaviour. Also note that the standard helper does not support XPointer syntax.

To process this file, load it as usual, and pass the root element to the *xml.etree.ElementTree* module:

```
from xml.etree import ElementTree, ElementInclude

tree = ElementTree.parse("document.xml")
root = tree.getroot()

ElementInclude.include(root)
```

The ElementInclude module replaces the `{http://www.w3.org/2001/XInclude}include` element with the root element from the **source.xml** document. The result might look something like this:

```
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  <para>This is a paragraph.</para>
</document>
```

If the **parse** attribute is omitted, it defaults to "xml". The href attribute is required.

To include a text document, use the `{http://www.w3.org/2001/XInclude}include` element, and set the **parse** attribute to "text":

```
<?xml version="1.0"?>
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  Copyright (c) <xi:include href="year.txt" parse="text" />.
</document>
```

The result might look something like:

```
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  Copyright (c) 2003.
</document>
```

21.5.5 Referência

Funções

xml.etree.ElementInclude.**default_loader**(*href*, *parse*, *encoding=None*)

Default loader. This default loader reads an included resource from disk. *href* is a URL. *parse* is for parse mode either "xml" or "text". *encoding* is an optional text encoding. If not given, encoding is `utf-8`. Returns the expanded resource. If the parse mode is `"xml"`, this is an *Element* instance. If the parse mode is `"text"`, this is a string. If the loader fails, it can return `None` or raise an exception.

xml.etree.ElementInclude.**include**(*elem*, *loader=None*, *base_url=None*, *max_depth=6*)

This function expands XInclude directives in-place in tree pointed by *elem*. *elem* is either the root *Element* or an *ElementTree* instance to find such element. *loader* is an optional resource loader. If omitted, it defaults to *default_loader()*. If given, it should be a callable that implements the same interface as *default_loader()*. *base_url* is base URL of the original file, to resolve relative include file references. *max_depth* is the maximum number of recursive inclusions. Limited to reduce the risk of malicious content explosion. Pass `None` to disable the limitation.

Alterado na versão 3.9: Added the *base_url* and *max_depth* parameters.

Element Objects

class xml.etree.ElementTree.**Element**(*tag*, *attrib={}*, ***extra*)

Element class. This class defines the Element interface, and provides a reference implementation of this interface.

The element name, attribute names, and attribute values can be either bytestrings or Unicode strings. *tag* is the element name. *attrib* is an optional dictionary, containing element attributes. *extra* contains additional attributes, given as keyword arguments.

tag

A string identifying what kind of data this element represents (the element type, in other words).

text

tail

These attributes can be used to hold additional data associated with the element. Their values are usually strings but may be any application-specific object. If the element is created from an XML file, the *text* attribute holds either the text between the element's start tag and its first child or end tag, or `None`, and the *tail* attribute holds either the text between the element's end tag and the next tag, or `None`. For the XML data

```
<a><b>1<c>2<d/>3</c></b>4</a>
```

the *a* element has `None` for both *text* and *tail* attributes, the *b* element has *text* `"1"` and *tail* `"4"`, the *c* element has *text* `"2"` and *tail* `None`, and the *d* element has *text* `None` and *tail* `"3"`.

To collect the inner text of an element, see *itertext()*, for example `"".join(element.itertext())`.

Applications may store arbitrary objects in these attributes.

attrib

A dictionary containing the element's attributes. Note that while the *attrib* value is always a real mutable Python dictionary, an ElementTree implementation may choose to use another internal representation, and create the dictionary only if someone asks for it. To take advantage of such implementations, use the dictionary methods below whenever possible.

The following dictionary-like methods work on the element attributes.

clear()

Resets an element. This function removes all subelements, clears all attributes, and sets the text and tail attributes to `None`.

get(*key*, *default=None*)

Gets the element attribute named *key*.

Returns the attribute value, or *default* if the attribute was not found.

items()

Returns the element attributes as a sequence of (name, value) pairs. The attributes are returned in an arbitrary order.

keys()

Returns the elements attribute names as a list. The names are returned in an arbitrary order.

set(*key*, *value*)

Set the attribute *key* on the element to *value*.

Os métodos a seguir funcionam nos filhos do elemento (subelementos).

append(*subelement*)

Adds the element *subelement* to the end of this element's internal list of subelements. Raises `TypeError` if *subelement* is not an `Element`.

extend(*subelements*)

Appends *subelements* from an iterable of elements. Raises `TypeError` if a subelement is not an `Element`.

Adicionado na versão 3.2.

find(*match*, *namespaces=None*)

Finds the first subelement matching *match*. *match* may be a tag name or a *path*. Returns an element instance or `None`. *namespaces* is an optional mapping from namespace prefix to full name. Pass `''` as prefix to move all unprefixed tag names in the expression into the given namespace.

findall(*match*, *namespaces=None*)

Finds all matching subelements, by tag name or *path*. Returns a list containing all matching elements in document order. *namespaces* is an optional mapping from namespace prefix to full name. Pass `''` as prefix to move all unprefixed tag names in the expression into the given namespace.

findtext(*match*, *default=None*, *namespaces=None*)

Finds text for the first subelement matching *match*. *match* may be a tag name or a *path*. Returns the text content of the first matching element, or *default* if no element was found. Note that if the matching element has no text content an empty string is returned. *namespaces* is an optional mapping from namespace prefix to full name. Pass `''` as prefix to move all unprefixed tag names in the expression into the given namespace.

insert(*index*, *subelement*)

Inserts *subelement* at the given position in this element. Raises `TypeError` if *subelement* is not an `Element`.

iter(*tag=None*)

Creates a tree *iterator* with the current element as the root. The iterator iterates over this element and all elements below it, in document (depth first) order. If *tag* is not `None` or `'*'`, only elements whose tag equals *tag* are returned from the iterator. If the tree structure is modified during iteration, the result is undefined.

Adicionado na versão 3.2.

iterfind(*match*, *namespaces=None*)

Finds all matching subelements, by tag name or *path*. Returns an iterable yielding all matching elements in document order. *namespaces* is an optional mapping from namespace prefix to full name.

Adicionado na versão 3.2.

itertext()

Creates a text iterator. The iterator loops over this element and all subelements, in document order, and returns all inner text.

Adicionado na versão 3.2.

makeelement(*tag*, *attrib*)

Creates a new element object of the same type as this element. Do not call this method, use the *SubElement()* factory function instead.

remove(*subelement*)

Removes *subelement* from the element. Unlike the find* methods this method compares elements based on the instance identity, not on tag value or contents.

Element objects also support the following sequence type methods for working with subelements: `__delitem__()`, `__getitem__()`, `__setitem__()`, `__len__()`.

Caution: Elements with no subelements will test as `False`. In a future release of Python, all elements will test as `True` regardless of whether subelements exist. Instead, prefer explicit `len(elem)` or `elem is not None` tests.:

```
element = root.find('foo')

if not element:  # careful!
    print("element not found, or element has no subelements")

if element is None:
    print("element not found")
```

Alterado na versão 3.12: Testing the truth value of an Element emits *DeprecationWarning*.

Prior to Python 3.8, the serialisation order of the XML attributes of elements was artificially made predictable by sorting the attributes by their name. Based on the now guaranteed ordering of dicts, this arbitrary reordering was removed in Python 3.8 to preserve the order in which attributes were originally parsed or created by user code.

In general, user code should try not to depend on a specific ordering of attributes, given that the XML Information Set explicitly excludes the attribute order from conveying information. Code should be prepared to deal with any ordering on input. In cases where deterministic XML output is required, e.g. for cryptographic signing or test data sets, canonical serialisation is available with the *canonicalize()* function.

In cases where canonical output is not applicable but a specific attribute order is still desirable on output, code should aim for creating the attributes directly in the desired order, to avoid perceptual mismatches for readers of the code. In cases where this is difficult to achieve, a recipe like the following can be applied prior to serialisation to enforce an order independently from the Element creation:

```
def reorder_attributes(root):
    for el in root.iter():
        attrib = el.attrib
        if len(attrib) > 1:
            # adjust attribute order, e.g. by sorting
            attribs = sorted(attrib.items())
            attrib.clear()
            attrib.update(attribs)
```

ElementTree Objects

class xml.etree.ElementTree.**ElementTree**(*element=None*, *file=None*)

ElementTree wrapper class. This class represents an entire element hierarchy, and adds some extra support for serialization to and from standard XML.

element is the root element. The tree is initialized with the contents of the XML *file* if given.

_setroot(*element*)

Replaces the root element for this tree. This discards the current contents of the tree, and replaces it with the given element. Use with care. *element* is an element instance.

find(*match*, *namespaces=None*)

Same as `Element.find()`, starting at the root of the tree.

findall(*match*, *namespaces=None*)

Same as `Element.findall()`, starting at the root of the tree.

findtext(*match*, *default=None*, *namespaces=None*)

Same as `Element.findtext()`, starting at the root of the tree.

getroot()

Returns the root element for this tree.

iter(*tag=None*)

Creates and returns a tree iterator for the root element. The iterator loops over all elements in this tree, in section order. *tag* is the tag to look for (default is to return all elements).

iterfind(*match*, *namespaces=None*)

Same as `Element.iterfind()`, starting at the root of the tree.

Adicionado na versão 3.2.

parse(*source*, *parser=None*)

Loads an external XML section into this element tree. *source* is a file name or *file object*. *parser* is an optional parser instance. If not given, the standard `XMLParser` parser is used. Returns the section root element.

write(*file*, *encoding='us-ascii'*, *xml_declaration=None*, *default_namespace=None*, *method='xml'*, ***, *short_empty_elements=True*)

Writes the element tree to a file, as XML. *file* is a file name, or a *file object* opened for writing. *encoding*[Página 1449, 1] is the output encoding (default is US-ASCII). *xml_declaration* controls if an XML declaration should be added to the file. Use `False` for never, `True` for always, `None` for only if not US-ASCII or UTF-8 or Unicode (default is `None`). *default_namespace* sets the default XML namespace (for "xmlns"). *method* is either `"xml"`, `"html"` or `"text"` (default is `"xml"`). The keyword-only *short_empty_elements* parameter controls the formatting of elements that contain no content. If `True` (the default), they are emitted as a single self-closed tag, otherwise they are emitted as a pair of start/end tags.

The output is either a string (`str`) or binary (`bytes`). This is controlled by the *encoding* argument. If *encoding* is `"unicode"`, the output is a string; otherwise, it's binary. Note that this may conflict with the type of *file* if it's an open *file object*; make sure you do not try to write a string to a binary stream and vice versa.

Alterado na versão 3.4: Foi adicionado o parâmetro *short_empty_elements*.

Alterado na versão 3.8: The `write()` method now preserves the attribute order specified by the user.

This is the XML file that is going to be manipulated:

```
<html>
    <head>
        <title>Example page</title>
    </head>
    <body>
        <p>Moved to <a href="http://example.org/">example.org</a>
        or <a href="http://example.com/">example.com</a>.</p>
    </body>
</html>
```

Example of changing the attribute "target" of every link in first paragraph:

```
>>> from xml.etree.ElementTree import ElementTree
>>> tree = ElementTree()
>>> tree.parse("index.xhtml")
<Element 'html' at 0xb77e6fac>
>>> p = tree.find("body/p")     # Finds first occurrence of tag p in body
>>> p
<Element 'p' at 0xb77ec26c>
>>> links = list(p.iter("a"))   # Returns list of all links
>>> links
[<Element 'a' at 0xb77ec2ac>, <Element 'a' at 0xb77ec1cc>]
>>> for i in links:             # Iterates through all found links
...     i.attrib["target"] = "blank"
...
>>> tree.write("output.xhtml")
```

QName Objects

class xml.etree.ElementTree.**QName**(*text_or_uri*, *tag=None*)

QName wrapper. This can be used to wrap a QName attribute value, in order to get proper namespace handling on output. *text_or_uri* is a string containing the QName value, in the form {uri}local, or, if the tag argument is given, the URI part of a QName. If *tag* is given, the first argument is interpreted as a URI, and this argument is interpreted as a local name. *QName* instances are opaque.

TreeBuilder Objects

class xml.etree.ElementTree.**TreeBuilder**(*element_factory=None*, *, *comment_factory=None*, *pi_factory=None*, *insert_comments=False*, *insert_pis=False*)

Generic element structure builder. This builder converts a sequence of start, data, end, comment and pi method calls to a well-formed element structure. You can use this class to build an element structure using a custom XML parser, or a parser for some other XML-like format.

element_factory, when given, must be a callable accepting two positional arguments: a tag and a dict of attributes. It is expected to return a new element instance.

The *comment_factory* and *pi_factory* functions, when given, should behave like the *Comment()* and *ProcessingInstruction()* functions to create comments and processing instructions. When not given, the default factories will be used. When *insert_comments* and/or *insert_pis* is true, comments/pis will be inserted into the tree if they appear within the root element (but not outside of it).

close()

Flushes the builder buffers, and returns the toplevel document element. Returns an *Element* instance.

data(*data*)

Adds text to the current element. *data* is a string. This should be either a bytestring, or a Unicode string.

end(*tag*)

Closes the current element. *tag* is the element name. Returns the closed element.

start(*tag*, *attrs*)

Opens a new element. *tag* is the element name. *attrs* is a dictionary containing element attributes. Returns the opened element.

comment(*text*)

Creates a comment with the given *text*. If `insert_comments` is true, this will also add it to the tree.

Adicionado na versão 3.8.

pi(*target*, *text*)

Creates a process instruction with the given *target* name and *text*. If `insert_pis` is true, this will also add it to the tree.

Adicionado na versão 3.8.

In addition, a custom `TreeBuilder` object can provide the following methods:

doctype(*name*, *pubid*, *system*)

Handles a doctype declaration. *name* is the doctype name. *pubid* is the public identifier. *system* is the system identifier. This method does not exist on the default `TreeBuilder` class.

Adicionado na versão 3.2.

start_ns(*prefix*, *uri*)

Is called whenever the parser encounters a new namespace declaration, before the `start()` callback for the opening element that defines it. *prefix* is `''` for the default namespace and the declared namespace prefix name otherwise. *uri* is the namespace URI.

Adicionado na versão 3.8.

end_ns(*prefix*)

Is called after the `end()` callback of an element that declared a namespace prefix mapping, with the name of the *prefix* that went out of scope.

Adicionado na versão 3.8.

class xml.etree.ElementTree.**C14NWriterTarget**(*write*, *, *with_comments=False*, *strip_text=False*, *rewrite_prefixes=False*, *qname_aware_tags=None*, *qname_aware_attrs=None*, *exclude_attrs=None*, *exclude_tags=None*)

A C14N 2.0 writer. Arguments are the same as for the `canonicalize()` function. This class does not build a tree but translates the callback events directly into a serialised form using the *write* function.

Adicionado na versão 3.8.

Objetos XMLParser

class xml.etree.ElementTree.**XMLParser**(*, *target=None*, *encoding=None*)

This class is the low-level building block of the module. It uses `xml.parsers.expat` for efficient, event-based parsing of XML. It can be fed XML data incrementally with the `feed()` method, and parsing events are translated to a push API - by invoking callbacks on the *target* object. If *target* is omitted, the standard `TreeBuilder` is used. If *encoding*[Página 1449, 1] is given, the value overrides the encoding specified in the XML file.

Alterado na versão 3.8: Parameters are now *keyword-only*. The *html* argument is no longer supported.

close()

Finishes feeding data to the parser. Returns the result of calling the `close()` method of the *target* passed during construction; by default, this is the toplevel document element.

feed(*data*)

Feeds data to the parser. *data* is encoded data.

flush()

Triggers parsing of any previously fed unparsed data, which can be used to ensure more immediate feedback, in particular with Expat >=2.6.0. The implementation of `flush()` temporarily disables reparse deferral with Expat (if currently enabled) and triggers a reparse. Disabling reparse deferral has security consequences; please see `xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()` for details.

Note that `flush()` has been backported to some prior releases of CPython as a security fix. Check for availability of `flush()` using `hasattr()` if used in code running across a variety of Python versions.

Adicionado na versão 3.13.

`XMLParser.feed()` calls *target*'s `start(tag, attrs_dict)` method for each opening tag, its `end(tag)` method for each closing tag, and data is processed by method `data(data)`. For further supported callback

methods, see the *TreeBuilder* class. *XMLParser.close()* calls *target*'s method `close()`. *XMLParser* can be used not only for building a tree structure. This is an example of counting the maximum depth of an XML file:

```
>>> from xml.etree.ElementTree import XMLParser
>>> class MaxDepth:                     # The target object of the parser
...     maxDepth = 0
...     depth = 0
...     def start(self, tag, attrib):   # Called for each opening tag.
...         self.depth += 1
...         if self.depth > self.maxDepth:
...             self.maxDepth = self.depth
...     def end(self, tag):             # Called for each closing tag.
...         self.depth -= 1
...     def data(self, data):
...         pass            # We do not need to do anything with data.
...     def close(self):    # Called when all data has been parsed.
...         return self.maxDepth
...
>>> target = MaxDepth()
>>> parser = XMLParser(target=target)
>>> exampleXml = """
... <a>
...   <b>
...   </b>
...   <b>
...     <c>
...       <d>
...       </d>
...     </c>
...   </b>
... </a>"""
>>> parser.feed(exampleXml)
>>> parser.close()
4
```

XMLPullParser Objects

class xml.etree.ElementTree.**XMLPullParser**(*events=None*)

A pull parser suitable for non-blocking applications. Its input-side API is similar to that of *XMLParser*, but instead of pushing calls to a callback target, *XMLPullParser* collects an internal list of parsing events and lets the user read from it. *events* is a sequence of events to report back. The supported events are the strings `"start"`, `"end"`, `"comment"`, `"pi"`, `"start-ns"` and `"end-ns"` (the "ns" events are used to get detailed namespace information). If *events* is omitted, only `"end"` events are reported.

feed(*data*)

Feed the given bytes data to the parser.

flush()

Triggers parsing of any previously fed unparsed data, which can be used to ensure more immediate feedback, in particular with Expat >=2.6.0. The implementation of *flush()* temporarily disables reparse deferral with Expat (if currently enabled) and triggers a reparse. Disabling reparse deferral has security consequences; please see *xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()* for details.

Note that *flush()* has been backported to some prior releases of CPython as a security fix. Check for availability of *flush()* using *hasattr()* if used in code running across a variety of Python versions.

Adicionado na versão 3.13.

close()

Signal the parser that the data stream is terminated. Unlike *XMLParser.close()*, this method always returns *None*. Any events not yet retrieved when the parser is closed can still be read with *read_events()*.

read_events()

Return an iterator over the events which have been encountered in the data fed to the parser. The iterator yields `(event, elem)` pairs, where *event* is a string representing the type of event (e.g. `"end"`) and *elem* is the encountered *Element* object, or other context value as follows.

- `start`, `end`: O Element atual.
- `comment`, `pi`: the current comment / processing instruction
- `start-ns`: a tuple `(prefix, uri)` naming the declared namespace mapping.
- `end-ns`: *None* (this may change in a future version)

Events provided in a previous call to *read_events()* will not be yielded again. Events are consumed from the internal queue only when they are retrieved from the iterator, so multiple readers iterating in parallel over iterators obtained from *read_events()* will have unpredictable results.

Nota

XMLPullParser only guarantees that it has seen the ">" character of a starting tag when it emits a "start" event, so the attributes are defined, but the contents of the text and tail attributes are undefined at that point. The same applies to the element children; they may or may not be present.

If you need a fully populated element, look for "end" events instead.

Adicionado na versão 3.4.

Alterado na versão 3.8: Os eventos `comment` e `pi` foram adicionados.

Exceções

class xml.etree.ElementTree.**ParseError**

XML parse error, raised by the various parsing methods in this module when parsing fails. The string representation of an instance of this exception will contain a user-friendly error message. In addition, it will have the following attributes available:

code

A numeric error code from the expat parser. See the documentation of *xml.parsers.expat* for the list of error codes and their meanings.

position

A tuple of *line*, *column* numbers, specifying where the error occurred.

21.6 xml.dom — The Document Object Model API

Código-fonte: Lib/xml/dom/__init__.py

The Document Object Model, or "DOM," is a cross-language API from the World Wide Web Consortium (W3C) for accessing and modifying XML documents. A DOM implementation presents an XML document as a tree structure, or allows client code to build such a structure from scratch. It then gives access to the structure through a set of objects which provided well-known interfaces.

The DOM is extremely useful for random-access applications. SAX only allows you a view of one bit of the document at a time. If you are looking at one SAX element, you have no access to another. If you are looking at a text node, you have no access to a containing element. When you write a SAX application, you need to keep track of your

program's position in the document somewhere in your own code. SAX does not do it for you. Also, if you need to look ahead in the XML document, you are just out of luck.

Some applications are simply impossible in an event driven model with no access to a tree. Of course you could build some sort of tree yourself in SAX events, but the DOM allows you to avoid writing that code. The DOM is a standard tree representation for XML data.

The Document Object Model is being defined by the W3C in stages, or "levels" in their terminology. The Python mapping of the API is substantially based on the DOM Level 2 recommendation.

DOM applications typically start by parsing some XML into a DOM. How this is accomplished is not covered at all by DOM Level 1, and Level 2 provides only limited improvements: There is a `DOMImplementation` object class which provides access to `Document` creation methods, but no way to access an XML reader/parser/Document builder in an implementation-independent way. There is also no well-defined way to access these methods without an existing `Document` object. In Python, each DOM implementation will provide a function *`getDOMImplementation()`*. DOM Level 3 adds a Load/Store specification, which defines an interface to the reader, but this is not yet available in the Python standard library.

Once you have a DOM document object, you can access the parts of your XML document through its properties and methods. These properties are defined in the DOM specification; this portion of the reference manual describes the interpretation of the specification in Python.

The specification provided by the W3C defines the DOM API for Java, ECMAScript, and OMG IDL. The Python mapping defined here is based in large part on the IDL version of the specification, but strict compliance is not required (though implementations are free to support the strict mapping from IDL). See section *Conformance* for a detailed discussion of mapping requirements.

Ver também

Document Object Model (DOM) Level 2 Specification
The W3C recommendation upon which the Python DOM API is based.

Document Object Model (DOM) Level 1 Specification
The W3C recommendation for the DOM supported by *`xml.dom.minidom`*.

Python Language Mapping Specification
This specifies the mapping from OMG IDL to Python.

21.6.1 Conteúdo do módulo

The *`xml.dom`* contains the following functions:

`xml.dom.`**`registerDOMImplementation`**(*name*, *factory*)

Register the *factory* function with the name *name*. The factory function should return an object which implements the `DOMImplementation` interface. The factory function can return the same object every time, or a new one for each call, as appropriate for the specific implementation (e.g. if that implementation supports some customization).

`xml.dom.`**`getDOMImplementation`**(*name=None*, *features=()*)

Return a suitable DOM implementation. The *name* is either well-known, the module name of a DOM implementation, or `None`. If it is not `None`, imports the corresponding module and returns a `DOMImplementation` object if the import succeeds. If no name is given, and if the environment variable `PYTHON_DOM` is set, this variable is used to find the implementation.

If name is not given, this examines the available implementations to find one with the required feature set. If no implementation can be found, raise an *`ImportError`*. The features list must be a sequence of `(feature, version)` pairs which are passed to the `hasFeature()` method on available `DOMImplementation` objects.

Some convenience constants are also provided:

xml.dom.**EMPTY_NAMESPACE**

The value used to indicate that no namespace is associated with a node in the DOM. This is typically found as the `namespaceURI` of a node, or used as the *namespaceURI* parameter to a namespaces-specific method.

xml.dom.**XML_NAMESPACE**

The namespace URI associated with the reserved prefix `xml`, as defined by Namespaces in XML (section 4).

xml.dom.**XMLNS_NAMESPACE**

The namespace URI for namespace declarations, as defined by Document Object Model (DOM) Level 2 Core Specification (section 1.1.8).

xml.dom.**XHTML_NAMESPACE**

The URI of the XHTML namespace as defined by XHTML 1.0: The Extensible HyperText Markup Language (section 3.1.1).

In addition, *xml.dom* contains a base `Node` class and the DOM exception classes. The `Node` class provided by this module does not implement any of the methods or attributes defined by the DOM specification; concrete DOM implementations must provide those. The `Node` class provided as part of this module does provide the constants used for the `nodeType` attribute on concrete `Node` objects; they are located within the class rather than at the module level to conform with the DOM specifications.

21.6.2 Objects in the DOM

The definitive documentation for the DOM is the DOM specification from the W3C.

Note that DOM attributes may also be manipulated as nodes instead of as simple strings. It is fairly rare that you must do this, however, so this usage is not yet documented.

Interface	Seção	Propósito
`DOMImplementation`	*DOMImplementation Objects*	Interface to the underlying implementation.
`Node`	*Objetos Node*	Base interface for most objects in a document.
`NodeList`	*Objetos NodeList*	Interface for a sequence of nodes.
`DocumentType`	*DocumentType Objects*	Information about the declarations needed to process a document.
`Document`	*Document Objects*	Object which represents an entire document.
`Element`	*Element Objects*	Element nodes in the document hierarchy.
`Attr`	*Attr Objects*	Attribute value nodes on element nodes.
`Comment`	*Comment Objects*	Representation of comments in the source document.
`Text`	*Text and CDATASection Objects*	Nodes containing textual content from the document.
`ProcessingInstructi`	*Objetos ProcessingInstruction*	Processing instruction representation.

An additional section describes the exceptions defined for working with the DOM in Python.

DOMImplementation Objects

The `DOMImplementation` interface provides a way for applications to determine the availability of particular features in the DOM they are using. DOM Level 2 added the ability to create new `Document` and `DocumentType` objects using the `DOMImplementation` as well.

`DOMImplementation`.**hasFeature**(*feature*, *version*)

Return `True` if the feature identified by the pair of strings *feature* and *version* is implemented.

`DOMImplementation`.**createDocument**(*namespaceUri*, *qualifiedName*, *doctype*)

Return a new `Document` object (the root of the DOM), with a child `Element` object having the given *namespaceUri* and *qualifiedName*. The *doctype* must be a `DocumentType` object created by *createDocumentType()*, or `None`. In the Python DOM API, the first two arguments can also be `None` in order to indicate that no `Element` child is to be created.

DOMImplementation.**createDocumentType**(*qualifiedName*, *publicId*, *systemId*)

Return a new DocumentType object that encapsulates the given *qualifiedName*, *publicId*, and *systemId* strings, representing the information contained in an XML document type declaration.

Objetos Node

All of the components of an XML document are subclasses of Node.

Node.**nodeType**

An integer representing the node type. Symbolic constants for the types are on the Node object: ELEMENT_NODE, ATTRIBUTE_NODE, TEXT_NODE, CDATA_SECTION_NODE, ENTITY_NODE, PROCESSING_INSTRUCTION_NODE, COMMENT_NODE, DOCUMENT_NODE, DOCUMENT_TYPE_NODE, NOTATION_NODE. This is a read-only attribute.

Node.**parentNode**

The parent of the current node, or None for the document node. The value is always a Node object or None. For Element nodes, this will be the parent element, except for the root element, in which case it will be the Document object. For Attr nodes, this is always None. This is a read-only attribute.

Node.**attributes**

A NamedNodeMap of attribute objects. Only elements have actual values for this; others provide None for this attribute. This is a read-only attribute.

Node.**previousSibling**

The node that immediately precedes this one with the same parent. For instance the element with an end-tag that comes just before the *self* element's start-tag. Of course, XML documents are made up of more than just elements so the previous sibling could be text, a comment, or something else. If this node is the first child of the parent, this attribute will be None. This is a read-only attribute.

Node.**nextSibling**

The node that immediately follows this one with the same parent. See also *previousSibling*. If this is the last child of the parent, this attribute will be None. This is a read-only attribute.

Node.**childNodes**

A list of nodes contained within this node. This is a read-only attribute.

Node.**firstChild**

The first child of the node, if there are any, or None. This is a read-only attribute.

Node.**lastChild**

The last child of the node, if there are any, or None. This is a read-only attribute.

Node.**localName**

The part of the tagName following the colon if there is one, else the entire tagName. The value is a string.

Node.**prefix**

The part of the tagName preceding the colon if there is one, else the empty string. The value is a string, or None.

Node.**namespaceURI**

The namespace associated with the element name. This will be a string or None. This is a read-only attribute.

Node.**nodeName**

This has a different meaning for each node type; see the DOM specification for details. You can always get the information you would get here from another property such as the tagName property for elements or the name property for attributes. For all node types, the value of this attribute will be either a string or None. This is a read-only attribute.

Node.**nodeValue**

This has a different meaning for each node type; see the DOM specification for details. The situation is similar to that with *nodeName*. The value is a string or None.

Node.**hasAttributes**()
: Return `True` if the node has any attributes.

Node.**hasChildNodes**()
: Return `True` if the node has any child nodes.

Node.**isSameNode**(*other*)
: Return `True` if *other* refers to the same node as this node. This is especially useful for DOM implementations which use any sort of proxy architecture (because more than one object can refer to the same node).

> **Nota**
>
> This is based on a proposed DOM Level 3 API which is still in the "working draft" stage, but this particular interface appears uncontroversial. Changes from the W3C will not necessarily affect this method in the Python DOM interface (though any new W3C API for this would also be supported).

Node.**appendChild**(*newChild*)
: Add a new child node to this node at the end of the list of children, returning *newChild*. If the node was already in the tree, it is removed first.

Node.**insertBefore**(*newChild*, *refChild*)
: Insert a new child node before an existing child. It must be the case that *refChild* is a child of this node; if not, *ValueError* is raised. *newChild* is returned. If *refChild* is `None`, it inserts *newChild* at the end of the children's list.

Node.**removeChild**(*oldChild*)
: Remove a child node. *oldChild* must be a child of this node; if not, *ValueError* is raised. *oldChild* is returned on success. If *oldChild* will not be used further, its `unlink()` method should be called.

Node.**replaceChild**(*newChild*, *oldChild*)
: Replace an existing node with a new node. It must be the case that *oldChild* is a child of this node; if not, *ValueError* is raised.

Node.**normalize**()
: Join adjacent text nodes so that all stretches of text are stored as single `Text` instances. This simplifies processing text from a DOM tree for many applications.

Node.**cloneNode**(*deep*)
: Clone this node. Setting *deep* means to clone all child nodes as well. This returns the clone.

Objetos NodeList

A `NodeList` represents a sequence of nodes. These objects are used in two ways in the DOM Core recommendation: an `Element` object provides one as its list of child nodes, and the `getElementsByTagName()` and `getElementsByTagNameNS()` methods of `Node` return objects with this interface to represent query results.

The DOM Level 2 recommendation defines one method and one attribute for these objects:

NodeList.**item**(*i*)
: Return the *i*'th item from the sequence, if there is one, or `None`. The index *i* is not allowed to be less than zero or greater than or equal to the length of the sequence.

NodeList.**length**
: The number of nodes in the sequence.

In addition, the Python DOM interface requires that some additional support is provided to allow `NodeList` objects to be used as Python sequences. All `NodeList` implementations must include support for `__len__()` and `__getitem__()`; this allows iteration over the `NodeList` in `for` statements and proper support for the *len()* built-in function.

If a DOM implementation supports modification of the document, the `NodeList` implementation must also support the `__setitem__()` and `__delitem__()` methods.

DocumentType Objects

Information about the notations and entities declared by a document (including the external subset if the parser uses it and can provide the information) is available from a `DocumentType` object. The `DocumentType` for a document is available from the `Document` object's `doctype` attribute; if there is no `DOCTYPE` declaration for the document, the document's `doctype` attribute will be set to `None` instead of an instance of this interface.

`DocumentType` is a specialization of `Node`, and adds the following attributes:

`DocumentType.`**`publicId`**

The public identifier for the external subset of the document type definition. This will be a string or `None`.

`DocumentType.`**`systemId`**

The system identifier for the external subset of the document type definition. This will be a URI as a string, or `None`.

`DocumentType.`**`internalSubset`**

A string giving the complete internal subset from the document. This does not include the brackets which enclose the subset. If the document has no internal subset, this should be `None`.

`DocumentType.`**`name`**

The name of the root element as given in the `DOCTYPE` declaration, if present.

`DocumentType.`**`entities`**

This is a `NamedNodeMap` giving the definitions of external entities. For entity names defined more than once, only the first definition is provided (others are ignored as required by the XML recommendation). This may be `None` if the information is not provided by the parser, or if no entities are defined.

`DocumentType.`**`notations`**

This is a `NamedNodeMap` giving the definitions of notations. For notation names defined more than once, only the first definition is provided (others are ignored as required by the XML recommendation). This may be `None` if the information is not provided by the parser, or if no notations are defined.

Document Objects

A `Document` represents an entire XML document, including its constituent elements, attributes, processing instructions, comments etc. Remember that it inherits properties from `Node`.

`Document.`**`documentElement`**

The one and only root element of the document.

`Document.`**`createElement`**(*tagName*)

Create and return a new element node. The element is not inserted into the document when it is created. You need to explicitly insert it with one of the other methods such as `insertBefore()` or `appendChild()`.

`Document.`**`createElementNS`**(*namespaceURI*, *tagName*)

Create and return a new element with a namespace. The *tagName* may have a prefix. The element is not inserted into the document when it is created. You need to explicitly insert it with one of the other methods such as `insertBefore()` or `appendChild()`.

`Document.`**`createTextNode`**(*data*)

Create and return a text node containing the data passed as a parameter. As with the other creation methods, this one does not insert the node into the tree.

`Document.`**`createComment`**(*data*)

Create and return a comment node containing the data passed as a parameter. As with the other creation methods, this one does not insert the node into the tree.

`Document.`**`createProcessingInstruction`**(*target*, *data*)

Create and return a processing instruction node containing the *target* and *data* passed as parameters. As with the other creation methods, this one does not insert the node into the tree.

Document.**createAttribute**(*name*)

Create and return an attribute node. This method does not associate the attribute node with any particular element. You must use `setAttributeNode()` on the appropriate `Element` object to use the newly created attribute instance.

Document.**createAttributeNS**(*namespaceURI*, *qualifiedName*)

Create and return an attribute node with a namespace. The *tagName* may have a prefix. This method does not associate the attribute node with any particular element. You must use `setAttributeNode()` on the appropriate `Element` object to use the newly created attribute instance.

Document.**getElementsByTagName**(*tagName*)

Search for all descendants (direct children, children's children, etc.) with a particular element type name.

Document.**getElementsByTagNameNS**(*namespaceURI*, *localName*)

Search for all descendants (direct children, children's children, etc.) with a particular namespace URI and localname. The localname is the part of the namespace after the prefix.

Element Objects

`Element` is a subclass of `Node`, so inherits all the attributes of that class.

Element.**tagName**

The element type name. In a namespace-using document it may have colons in it. The value is a string.

Element.**getElementsByTagName**(*tagName*)

Same as equivalent method in the `Document` class.

Element.**getElementsByTagNameNS**(*namespaceURI*, *localName*)

Same as equivalent method in the `Document` class.

Element.**hasAttribute**(*name*)

Return `True` if the element has an attribute named by *name*.

Element.**hasAttributeNS**(*namespaceURI*, *localName*)

Return `True` if the element has an attribute named by *namespaceURI* and *localName*.

Element.**getAttribute**(*name*)

Return the value of the attribute named by *name* as a string. If no such attribute exists, an empty string is returned, as if the attribute had no value.

Element.**getAttributeNode**(*attrname*)

Return the `Attr` node for the attribute named by *attrname*.

Element.**getAttributeNS**(*namespaceURI*, *localName*)

Return the value of the attribute named by *namespaceURI* and *localName* as a string. If no such attribute exists, an empty string is returned, as if the attribute had no value.

Element.**getAttributeNodeNS**(*namespaceURI*, *localName*)

Return an attribute value as a node, given a *namespaceURI* and *localName*.

Element.**removeAttribute**(*name*)

Remove an attribute by name. If there is no matching attribute, a `NotFoundErr` is raised.

Element.**removeAttributeNode**(*oldAttr*)

Remove and return *oldAttr* from the attribute list, if present. If *oldAttr* is not present, `NotFoundErr` is raised.

Element.**removeAttributeNS**(*namespaceURI*, *localName*)

Remove an attribute by name. Note that it uses a localName, not a qname. No exception is raised if there is no matching attribute.

Element.**setAttribute**(*name*, *value*)

Set an attribute value from a string.

Element.**setAttributeNode**(*newAttr*)

Add a new attribute node to the element, replacing an existing attribute if necessary if the `name` attribute matches. If a replacement occurs, the old attribute node will be returned. If *newAttr* is already in use, *InuseAttributeErr* will be raised.

Element.**setAttributeNodeNS**(*newAttr*)

Add a new attribute node to the element, replacing an existing attribute if necessary if the `namespaceURI` and `localName` attributes match. If a replacement occurs, the old attribute node will be returned. If *newAttr* is already in use, *InuseAttributeErr* will be raised.

Element.**setAttributeNS**(*namespaceURI*, *qname*, *value*)

Set an attribute value from a string, given a *namespaceURI* and a *qname*. Note that a qname is the whole attribute name. This is different than above.

Attr Objects

`Attr` inherits from `Node`, so inherits all its attributes.

Attr.**name**

The attribute name. In a namespace-using document it may include a colon.

Attr.**localName**

The part of the name following the colon if there is one, else the entire name. This is a read-only attribute.

Attr.**prefix**

The part of the name preceding the colon if there is one, else the empty string.

Attr.**value**

The text value of the attribute. This is a synonym for the `nodeValue` attribute.

NamedNodeMap Objects

`NamedNodeMap` does *not* inherit from `Node`.

NamedNodeMap.**length**

The length of the attribute list.

NamedNodeMap.**item**(*index*)

Return an attribute with a particular index. The order you get the attributes in is arbitrary but will be consistent for the life of a DOM. Each item is an attribute node. Get its value with the `value` attribute.

There are also experimental methods that give this class more mapping behavior. You can use them or you can use the standardized `getAttribute*()` family of methods on the `Element` objects.

Comment Objects

`Comment` represents a comment in the XML document. It is a subclass of `Node`, but cannot have child nodes.

Comment.**data**

The content of the comment as a string. The attribute contains all characters between the leading `<!--` and trailing `-->`, but does not include them.

Text and CDATASection Objects

The `Text` interface represents text in the XML document. If the parser and DOM implementation support the DOM's XML extension, portions of the text enclosed in CDATA marked sections are stored in `CDATASection` objects. These two interfaces are identical, but provide different values for the `nodeType` attribute.

These interfaces extend the `Node` interface. They cannot have child nodes.

`Text`.**data**

The content of the text node as a string.

Nota

The use of a `CDATASection` node does not indicate that the node represents a complete CDATA marked section, only that the content of the node was part of a CDATA section. A single CDATA section may be represented by more than one node in the document tree. There is no way to determine whether two adjacent `CDATASection` nodes represent different CDATA marked sections.

Objetos ProcessingInstruction

Represents a processing instruction in the XML document; this inherits from the `Node` interface and cannot have child nodes.

`ProcessingInstruction`.**target**

The content of the processing instruction up to the first whitespace character. This is a read-only attribute.

`ProcessingInstruction`.**data**

The content of the processing instruction following the first whitespace character.

Exceções

The DOM Level 2 recommendation defines a single exception, *`DOMException`*, and a number of constants that allow applications to determine what sort of error occurred. *`DOMException`* instances carry a *`code`* attribute that provides the appropriate value for the specific exception.

The Python DOM interface provides the constants, but also expands the set of exceptions so that a specific exception exists for each of the exception codes defined by the DOM. The implementations must raise the appropriate specific exception, each of which carries the appropriate value for the *`code`* attribute.

exception `xml.dom`.**DOMException**

Base exception class used for all specific DOM exceptions. This exception class cannot be directly instantiated.

exception `xml.dom`.**DomstringSizeErr**

Raised when a specified range of text does not fit into a string. This is not known to be used in the Python DOM implementations, but may be received from DOM implementations not written in Python.

exception `xml.dom`.**HierarchyRequestErr**

Raised when an attempt is made to insert a node where the node type is not allowed.

exception `xml.dom`.**IndexSizeErr**

Raised when an index or size parameter to a method is negative or exceeds the allowed values.

exception `xml.dom`.**InuseAttributeErr**

Raised when an attempt is made to insert an `Attr` node that is already present elsewhere in the document.

exception `xml.dom`.**InvalidAccessErr**

Raised if a parameter or an operation is not supported on the underlying object.

exception `xml.dom`.**InvalidCharacterErr**

This exception is raised when a string parameter contains a character that is not permitted in the context it's being used in by the XML 1.0 recommendation. For example, attempting to create an `Element` node with a space in the element type name will cause this error to be raised.

exception `xml.dom`.**InvalidModificationErr**

Raised when an attempt is made to modify the type of a node.

exception `xml.dom`.**InvalidStateErr**

Raised when an attempt is made to use an object that is not defined or is no longer usable.

exception xml.dom.**NamespaceErr**

If an attempt is made to change any object in a way that is not permitted with regard to the Namespaces in XML recommendation, this exception is raised.

exception xml.dom.**NotFoundErr**

Exception when a node does not exist in the referenced context. For example, `NamedNodeMap.removeNamedItem()` will raise this if the node passed in does not exist in the map.

exception xml.dom.**NotSupportedErr**

Raised when the implementation does not support the requested type of object or operation.

exception xml.dom.**NoDataAllowedErr**

This is raised if data is specified for a node which does not support data.

exception xml.dom.**NoModificationAllowedErr**

Raised on attempts to modify an object where modifications are not allowed (such as for read-only nodes).

exception xml.dom.**SyntaxErr**

Raised when an invalid or illegal string is specified.

exception xml.dom.**WrongDocumentErr**

Raised when a node is inserted in a different document than it currently belongs to, and the implementation does not support migrating the node from one document to the other.

The exception codes defined in the DOM recommendation map to the exceptions described above according to this table:

Constante	Exception
DOMSTRING_SIZE_ERR	*DomstringSizeErr*
HIERARCHY_REQUEST_ERR	*HierarchyRequestErr*
INDEX_SIZE_ERR	*IndexSizeErr*
INUSE_ATTRIBUTE_ERR	*InuseAttributeErr*
INVALID_ACCESS_ERR	*InvalidAccessErr*
INVALID_CHARACTER_ERR	*InvalidCharacterErr*
INVALID_MODIFICATION_ERR	*InvalidModificationErr*
INVALID_STATE_ERR	*InvalidStateErr*
NAMESPACE_ERR	*NamespaceErr*
NOT_FOUND_ERR	*NotFoundErr*
NOT_SUPPORTED_ERR	*NotSupportedErr*
NO_DATA_ALLOWED_ERR	*NoDataAllowedErr*
NO_MODIFICATION_ALLOWED_ERR	*NoModificationAllowedErr*
SYNTAX_ERR	*SyntaxErr*
WRONG_DOCUMENT_ERR	*WrongDocumentErr*

21.6.3 Conformance

This section describes the conformance requirements and relationships between the Python DOM API, the W3C DOM recommendations, and the OMG IDL mapping for Python.

Type Mapping

The IDL types used in the DOM specification are mapped to Python types according to the following table.

Tipo IDL	Tipo em Python
`boolean`	`bool` or `int`
`int`	`int`
`long int`	`int`
`unsigned int`	`int`
`DOMString`	`str` or `bytes`
`null`	`None`

Accessor Methods

The mapping from OMG IDL to Python defines accessor functions for IDL `attribute` declarations in much the way the Java mapping does. Mapping the IDL declarations

```
readonly attribute string someValue;
         attribute string anotherValue;
```

yields three accessor functions: a "get" method for `someValue` (`_get_someValue()`), and "get" and "set" methods for `anotherValue` (`_get_anotherValue()` and `_set_anotherValue()`). The mapping, in particular, does not require that the IDL attributes are accessible as normal Python attributes: `object.someValue` is *not* required to work, and may raise an *AttributeError*.

The Python DOM API, however, *does* require that normal attribute access work. This means that the typical surrogates generated by Python IDL compilers are not likely to work, and wrapper objects may be needed on the client if the DOM objects are accessed via CORBA. While this does require some additional consideration for CORBA DOM clients, the implementers with experience using DOM over CORBA from Python do not consider this a problem. Attributes that are declared `readonly` may not restrict write access in all DOM implementations.

In the Python DOM API, accessor functions are not required. If provided, they should take the form defined by the Python IDL mapping, but these methods are considered unnecessary since the attributes are accessible directly from Python. "Set" accessors should never be provided for `readonly` attributes.

The IDL definitions do not fully embody the requirements of the W3C DOM API, such as the notion of certain objects, such as the return value of `getElementsByTagName()`, being "live". The Python DOM API does not require implementations to enforce such requirements.

21.7 `xml.dom.minidom` — Minimal DOM implementation

Código-fonte: Lib/xml/dom/minidom.py

---

xml.dom.minidom is a minimal implementation of the Document Object Model interface, with an API similar to that in other languages. It is intended to be simpler than the full DOM and also significantly smaller. Users who are not already proficient with the DOM should consider using the *xml.etree.ElementTree* module for their XML processing instead.

> **Nota**
>
> Se você precisar analisar dados não confiáveis ou não autenticados, consulte *Segurança no XML*.

DOM applications typically start by parsing some XML into a DOM. With *xml.dom.minidom*, this is done through the parse functions:

```
from xml.dom.minidom import parse, parseString

dom1 = parse('c:\\temp\\mydata.xml')  # parse an XML file by name
```

(continua na próxima página)

(continuação da página anterior)

```
datasource = open('c:\\temp\\mydata.xml')
dom2 = parse(datasource)  # parse an open file

dom3 = parseString('<myxml>Some data<empty/> some more data</myxml>')
```

The *parse()* function can take either a filename or an open file object.

xml.dom.minidom.**parse**(*filename_or_file*, *parser=None*, *bufsize=None*)

Return a Document from the given input. *filename_or_file* may be either a file name, or a file-like object. *parser*, if given, must be a SAX2 parser object. This function will change the document handler of the parser and activate namespace support; other parser configuration (like setting an entity resolver) must have been done in advance.

Se você tiver XML em uma string, poderá usar a função *parseString()*:

xml.dom.minidom.**parseString**(*string*, *parser=None*)

Return a Document that represents the *string*. This method creates an *io.StringIO* object for the string and passes that on to *parse()*.

Both functions return a Document object representing the content of the document.

What the *parse()* and *parseString()* functions do is connect an XML parser with a "DOM builder" that can accept parse events from any SAX parser and convert them into a DOM tree. The name of the functions are perhaps misleading, but are easy to grasp when learning the interfaces. The parsing of the document will be completed before these functions return; it's simply that these functions do not provide a parser implementation themselves.

You can also create a Document by calling a method on a "DOM Implementation" object. You can get this object either by calling the getDOMImplementation() function in the *xml.dom* package or the *xml.dom.minidom* module. Once you have a Document, you can add child nodes to it to populate the DOM:

```
from xml.dom.minidom import getDOMImplementation

impl = getDOMImplementation()

newdoc = impl.createDocument(None, "some_tag", None)
top_element = newdoc.documentElement
text = newdoc.createTextNode('Some textual content.')
top_element.appendChild(text)
```

Once you have a DOM document object, you can access the parts of your XML document through its properties and methods. These properties are defined in the DOM specification. The main property of the document object is the documentElement property. It gives you the main element in the XML document: the one that holds all others. Here is an example program:

```
dom3 = parseString("<myxml>Some data</myxml>")
assert dom3.documentElement.tagName == "myxml"
```

When you are finished with a DOM tree, you may optionally call the unlink() method to encourage early cleanup of the now-unneeded objects. unlink() is an *xml.dom.minidom*-specific extension to the DOM API that renders the node and its descendants essentially useless. Otherwise, Python's garbage collector will eventually take care of the objects in the tree.

Ver também

Document Object Model (DOM) Level 1 Specification
The W3C recommendation for the DOM supported by *xml.dom.minidom*.

21.7.1 Objetos DOM

The definition of the DOM API for Python is given as part of the *xml.dom* module documentation. This section lists the differences between the API and *xml.dom.minidom*.

Node.**unlink**()

Break internal references within the DOM so that it will be garbage collected on versions of Python without cyclic GC. Even when cyclic GC is available, using this can make large amounts of memory available sooner, so calling this on DOM objects as soon as they are no longer needed is good practice. This only needs to be called on the `Document` object, but may be called on child nodes to discard children of that node.

You can avoid calling this method explicitly by using the `with` statement. The following code will automatically unlink *dom* when the `with` block is exited:

```
with xml.dom.minidom.parse(datasource) as dom:
    ... # Work with dom.
```

Node.**writexml**(*writer*, *indent=''*, *addindent=''*, *newl=''*, *encoding=None*, *standalone=None*)

Write XML to the writer object. The writer receives texts but not bytes as input, it should have a `write()` method which matches that of the file object interface. The *indent* parameter is the indentation of the current node. The *addindent* parameter is the incremental indentation to use for subnodes of the current one. The *newl* parameter specifies the string to use to terminate newlines.

For the `Document` node, an additional keyword argument *encoding* can be used to specify the encoding field of the XML header.

Similarly, explicitly stating the *standalone* argument causes the standalone document declarations to be added to the prologue of the XML document. If the value is set to `True`, `standalone="yes"` is added, otherwise it is set to `"no"`. Not stating the argument will omit the declaration from the document.

Alterado na versão 3.8: The *writexml()* method now preserves the attribute order specified by the user.

Alterado na versão 3.9: The *standalone* parameter was added.

Node.**toxml**(*encoding=None*, *standalone=None*)

Return a string or byte string containing the XML represented by the DOM node.

With an explicit *encoding*[1] argument, the result is a byte string in the specified encoding. With no *encoding* argument, the result is a Unicode string, and the XML declaration in the resulting string does not specify an encoding. Encoding this string in an encoding other than UTF-8 is likely incorrect, since UTF-8 is the default encoding of XML.

The *standalone* argument behaves exactly as in *writexml()*.

Alterado na versão 3.8: The *toxml()* method now preserves the attribute order specified by the user.

Alterado na versão 3.9: The *standalone* parameter was added.

Node.**toprettyxml**(*indent='\t'*, *newl='\n'*, *encoding=None*, *standalone=None*)

Return a pretty-printed version of the document. *indent* specifies the indentation string and defaults to a tabulator; *newl* specifies the string emitted at the end of each line and defaults to `\n`.

The *encoding* argument behaves like the corresponding argument of *toxml()*.

The *standalone* argument behaves exactly as in *writexml()*.

Alterado na versão 3.8: The *toprettyxml()* method now preserves the attribute order specified by the user.

Alterado na versão 3.9: The *standalone* parameter was added.

[1] The encoding name included in the XML output should conform to the appropriate standards. For example, "UTF-8" is valid, but "UTF8" is not valid in an XML document's declaration, even though Python accepts it as an encoding name. See https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-EncodingDecl and https://www.iana.org/assignments/character-sets/character-sets.xhtml.

21.7.2 DOM Example

This example program is a fairly realistic example of a simple program. In this particular case, we do not take much advantage of the flexibility of the DOM.

```
import xml.dom.minidom

document = """\
<slideshow>
<title>Demo slideshow</title>
<slide><title>Slide title</title>
<point>This is a demo</point>
<point>Of a program for processing slides</point>
</slide>

<slide><title>Another demo slide</title>
<point>It is important</point>
<point>To have more than</point>
<point>one slide</point>
</slide>
</slideshow>
"""

dom = xml.dom.minidom.parseString(document)

def getText(nodelist):
    rc = []
    for node in nodelist:
        if node.nodeType == node.TEXT_NODE:
            rc.append(node.data)
    return ''.join(rc)

def handleSlideshow(slideshow):
    print("<html>")
    handleSlideshowTitle(slideshow.getElementsByTagName("title")[0])
    slides = slideshow.getElementsByTagName("slide")
    handleToc(slides)
    handleSlides(slides)
    print("</html>")

def handleSlides(slides):
    for slide in slides:
        handleSlide(slide)

def handleSlide(slide):
    handleSlideTitle(slide.getElementsByTagName("title")[0])
    handlePoints(slide.getElementsByTagName("point"))

def handleSlideshowTitle(title):
    print(f"<title>{getText(title.childNodes)}</title>")

def handleSlideTitle(title):
    print(f"<h2>{getText(title.childNodes)}</h2>")

def handlePoints(points):
    print("<ul>")
    for point in points:
        handlePoint(point)
```

(continua na próxima página)

(continuação da página anterior)

```
    print("</ul>")

def handlePoint(point):
    print(f"<li>{getText(point.childNodes)}</li>")

def handleToc(slides):
    for slide in slides:
        title = slide.getElementsByTagName("title")[0]
        print(f"<p>{getText(title.childNodes)}</p>")

handleSlideshow(dom)
```

21.7.3 minidom e o padrão DOM

The *xml.dom.minidom* module is essentially a DOM 1.0-compatible DOM with some DOM 2 features (primarily namespace features).

Usage of the DOM interface in Python is straight-forward. The following mapping rules apply:

- Interfaces are accessed through instance objects. Applications should not instantiate the classes themselves; they should use the creator functions available on the `Document` object. Derived interfaces support all operations (and attributes) from the base interfaces, plus any new operations.
- Operations are used as methods. Since the DOM uses only `in` parameters, the arguments are passed in normal order (from left to right). There are no optional arguments. `void` operations return `None`.
- IDL attributes map to instance attributes. For compatibility with the OMG IDL language mapping for Python, an attribute `foo` can also be accessed through accessor methods `_get_foo()` and `_set_foo()`. `readonly` attributes must not be changed; this is not enforced at runtime.
- The types `short int`, `unsigned int`, `unsigned long long`, and `boolean` all map to Python integer objects.
- The type `DOMString` maps to Python strings. *xml.dom.minidom* supports either bytes or strings, but will normally produce strings. Values of type `DOMString` may also be `None` where allowed to have the IDL `null` value by the DOM specification from the W3C.
- `const` declarations map to variables in their respective scope (e.g. `xml.dom.minidom.Node.PROCESSING_INSTRUCTION_NODE`); they must not be changed.
- `DOMException` is currently not supported in *xml.dom.minidom*. Instead, *xml.dom.minidom* uses standard Python exceptions such as *TypeError* and *AttributeError*.
- `NodeList` objects are implemented using Python's built-in list type. These objects provide the interface defined in the DOM specification, but with earlier versions of Python they do not support the official API. They are, however, much more "Pythonic" than the interface defined in the W3C recommendations.

The following interfaces have no implementation in *xml.dom.minidom*:

- `DOMTimeStamp`
- `EntityReference`

Most of these reflect information in the XML document that is not of general utility to most DOM users.

21.8 `xml.dom.pulldom` — Suporte para construir árvores parciais de DOM

Código-fonte: Lib/xml/dom/pulldom.py

O módulo *xml.dom.pulldom* fornece um "analisador sintático de pull" que também pode ser solicitado a produzir fragmentos do documento acessíveis via DOM, quando necessário. O conceito básico envolve extrair "eventos" de um fluxo de XML recebido e processá-los. Ao contrário do SAX, que também emprega um modelo de processamento orientado a eventos juntamente com retornos de chamada, o usuário de um analisador sintático pull é responsável por extrair explicitamente os eventos do fluxo, repetindo esses eventos até que o processamento seja concluído ou ocorra uma condição de erro.

Nota

Se você precisar analisar dados não confiáveis ou não autenticados, consulte *Segurança no XML*.

Alterado na versão 3.7.1: O analisador sintático de SAX não processa mais entidades externas gerais por padrão para aumentar a segurança. Para habilitar o processamento de entidades externas, passe uma instância personalizada do analisador sintático em:

```
from xml.dom.pulldom import parse
from xml.sax import make_parser
from xml.sax.handler import feature_external_ges

parser = make_parser()
parser.setFeature(feature_external_ges, True)
parse(filename, parser=parser)
```

Exemplo:

```
from xml.dom import pulldom

doc = pulldom.parse('sales_items.xml')
for event, node in doc:
    if event == pulldom.START_ELEMENT and node.tagName == 'item':
        if int(node.getAttribute('price')) > 50:
            doc.expandNode(node)
            print(node.toxml())
```

`event` é uma constante e pode ser um de:

- `START_ELEMENT`
- `END_ELEMENT`
- `COMMENT`
- `START_DOCUMENT`
- `END_DOCUMENT`
- `CHARACTERS`
- `PROCESSING_INSTRUCTION`
- `IGNORABLE_WHITESPACE`

`node` é um objeto do tipo `xml.dom.minidom.Document`, `xml.dom.minidom.Element` ou `xml.dom.minidom.Text`.

Como o documento é tratado como um fluxo "plano" de eventos, a "árvore" do documento é implicitamente percorrida e os elementos desejados são encontrados independentemente de sua profundidade na árvore. Em outras palavras, não é necessário considerar questões hierárquicas, como a busca recursiva dos nós do documento, embora, se o contexto dos elementos fosse importante, seria necessário manter algum estado relacionado ao contexto (ou seja, lembrar onde se está no documento em um determinado ponto) ou utilizar o método *DOMEventStream.expandNode()* e alternar para o processamento relacionado ao DOM.

class xml.dom.pulldom.**PullDom**(*documentFactory=None*)

Subclasse de *xml.sax.handler.ContentHandler*.

class xml.dom.pulldom.**SAX2DOM**(*documentFactory=None*)

Subclasse de *xml.sax.handler.ContentHandler*.

xml.dom.pulldom.**parse**(*stream_or_string*, *parser=None*, *bufsize=None*)

Retorna um *DOMEventStream* da entrada fornecida. *stream_or_string* pode ser um nome de arquivo ou um objeto arquivo ou similar. *parser*, se fornecido, deve ser um objeto *XMLReader*. Esta função alterará o manipulador de documentos do analisador e ativará o suporte a espaços de nomes; outras configurações do analisador sintático (como a definição de um resolvedor de entidades) devem ter sido feitas previamente.

Se você tiver XML em uma string, poderá usar a função *parseString()*:

xml.dom.pulldom.**parseString**(*string*, *parser=None*)

Retorna um *DOMEventStream* que representa a *string* (Unicode).

xml.dom.pulldom.**default_bufsize**

Valor padrão para o parâmetro *bufsize* para *parse()*.

O valor desta variável pode ser alterado antes de chamar *parse()* e o novo valor entrará em vigor.

21.8.1 Objetos DOMEventStream

class xml.dom.pulldom.**DOMEventStream**(*stream*, *parser*, *bufsize*)

Alterado na versão 3.11: O suporte para o método `__getitem__()` foi removido

getEvent()

Retorna uma tupla contendo evento *event* e o nó *node* atual como `xml.dom.minidom.Document` se o evento for igual a `START_DOCUMENT`, `xml.dom.minidom.Element` se o evento for igual a `START_ELEMENT` ou `END_ELEMENT` ou `xml.dom.minidom.Text` se o evento for igual a `CHARACTERS`. O nó atual não contém informações sobre seus filhos, a menos que *expandNode()* seja chamado.

expandNode(*node*)

Expande todos os filhos de *node* para *node*. Exemplo:

```
from xml.dom import pulldom

xml = '<html><title>Foo</title> <p>Algum texto <div>e mais</div></p> </
↪html>'
doc = pulldom.parseString(xml)
for event, node in doc:
    if event == pulldom.START_ELEMENT and node.tagName == 'p':
        # A seguinte instrução só exibe '<p/>'
        print(node.toxml())
        doc.expandNode(node)
        # A seguinte instrução exibe o nó com todos os seus filhos '<p>
↪Algum texto <div>e mais</div></p>'
        print(node.toxml())
```

reset()

21.9 xml.sax — Support for SAX2 parsers

Código-fonte: Lib/xml/sax/__init__.py

The *xml.sax* package provides a number of modules which implement the Simple API for XML (SAX) interface for Python. The package itself provides the SAX exceptions and the convenience functions which will be most used by users of the SAX API.

Nota

Se você precisar analisar dados não confiáveis ou não autenticados, consulte *Segurança no XML*.

Alterado na versão 3.7.1: The SAX parser no longer processes general external entities by default to increase security. Before, the parser created network connections to fetch remote files or loaded local files from the file system for DTD and entities. The feature can be enabled again with method `setFeature()` on the parser object and argument `feature_external_ges`.

As funções de conveniência são:

`xml.sax.`**`make_parser`**(*parser_list=[]*)

Create and return a SAX `XMLReader` object. The first parser found will be used. If *parser_list* is provided, it must be an iterable of strings which name modules that have a function named `create_parser()`. Modules listed in *parser_list* will be used before modules in the default list of parsers.

Alterado na versão 3.8: The *parser_list* argument can be any iterable, not just a list.

`xml.sax.`**`parse`**(*filename_or_stream*, *handler*, *error_handler=handler.ErrorHandler()*)

Create a SAX parser and use it to parse a document. The document, passed in as *filename_or_stream*, can be a filename or a file object. The *handler* parameter needs to be a SAX `ContentHandler` instance. If *error_handler* is given, it must be a SAX `ErrorHandler` instance; if omitted, `SAXParseException` will be raised on all errors. There is no return value; all work must be done by the *handler* passed in.

`xml.sax.`**`parseString`**(*string*, *handler*, *error_handler=handler.ErrorHandler()*)

Similar to `parse()`, but parses from a buffer *string* received as a parameter. *string* must be a `str` instance or a *bytes-like object*.

Alterado na versão 3.5: Added support of `str` instances.

A typical SAX application uses three kinds of objects: readers, handlers and input sources. "Reader" in this context is another term for parser, i.e. some piece of code that reads the bytes or characters from the input source, and produces a sequence of events. The events then get distributed to the handler objects, i.e. the reader invokes a method on the handler. A SAX application must therefore obtain a reader object, create or open the input sources, create the handlers, and connect these objects all together. As the final step of preparation, the reader is called to parse the input. During parsing, methods on the handler objects are called based on structural and syntactic events from the input data.

For these objects, only the interfaces are relevant; they are normally not instantiated by the application itself. Since Python does not have an explicit notion of interface, they are formally introduced as classes, but applications may use implementations which do not inherit from the provided classes. The `InputSource`, `Locator`, `Attributes`, `AttributesNS`, and `XMLReader` interfaces are defined in the module `xml.sax.xmlreader`. The handler interfaces are defined in `xml.sax.handler`. For convenience, `InputSource` (which is often instantiated directly) and the handler classes are also available from `xml.sax`. These interfaces are described below.

In addition to these classes, `xml.sax` provides the following exception classes.

`exception` `xml.sax.`**`SAXException`**(*msg*, *exception=None*)

Encapsulate an XML error or warning. This class can contain basic error or warning information from either the XML parser or the application: it can be subclassed to provide additional functionality or to add localization. Note that although the handlers defined in the `ErrorHandler` interface receive instances of this exception, it is not required to actually raise the exception — it is also useful as a container for information.

When instantiated, *msg* should be a human-readable description of the error. The optional *exception* parameter, if given, should be `None` or an exception that was caught by the parsing code and is being passed along as information.

This is the base class for the other SAX exception classes.

`exception` `xml.sax.`**`SAXParseException`**(*msg*, *exception*, *locator*)

Subclass of `SAXException` raised on parse errors. Instances of this class are passed to the methods of the SAX `ErrorHandler` interface to provide information about the parse error. This class supports the SAX `Locator` interface as well as the `SAXException` interface.

exception xml.sax.**SAXNotRecognizedException**(*msg*, *exception=None*)

Subclass of *SAXException* raised when a SAX *XMLReader* is confronted with an unrecognized feature or property. SAX applications and extensions may use this class for similar purposes.

exception xml.sax.**SAXNotSupportedException**(*msg*, *exception=None*)

Subclass of *SAXException* raised when a SAX *XMLReader* is asked to enable a feature that is not supported, or to set a property to a value that the implementation does not support. SAX applications and extensions may use this class for similar purposes.

Ver também

SAX: The Simple API for XML
This site is the focal point for the definition of the SAX API. It provides a Java implementation and online documentation. Links to implementations and historical information are also available.

Module ***xml.sax.handler***
Definitions of the interfaces for application-provided objects.

Module ***xml.sax.saxutils***
Convenience functions for use in SAX applications.

Module ***xml.sax.xmlreader***
Definitions of the interfaces for parser-provided objects.

21.9.1 SAXException Objects

The *SAXException* exception class supports the following methods:

SAXException.**getMessage**()

Return a human-readable message describing the error condition.

SAXException.**getException**()

Return an encapsulated exception object, or None.

21.10 xml.sax.handler — Base classes for SAX handlers

Código-fonte: Lib/xml/sax/handler.py

The SAX API defines five kinds of handlers: content handlers, DTD handlers, error handlers, entity resolvers and lexical handlers. Applications normally only need to implement those interfaces whose events they are interested in; they can implement the interfaces in a single object or in multiple objects. Handler implementations should inherit from the base classes provided in the module *xml.sax.handler*, so that all methods get default implementations.

class xml.sax.handler.**ContentHandler**

This is the main callback interface in SAX, and the one most important to applications. The order of events in this interface mirrors the order of the information in the document.

class xml.sax.handler.**DTDHandler**

Manipular eventos DTD.

This interface specifies only those DTD events required for basic parsing (unparsed entities and attributes).

class xml.sax.handler.**EntityResolver**

Basic interface for resolving entities. If you create an object implementing this interface, then register the object with your Parser, the parser will call the method in your object to resolve all external entities.

class xml.sax.handler.**ErrorHandler**

Interface used by the parser to present error and warning messages to the application. The methods of this object control whether errors are immediately converted to exceptions or are handled in some other way.

class xml.sax.handler.**LexicalHandler**

Interface used by the parser to represent low frequency events which may not be of interest to many applications.

In addition to these classes, *xml.sax.handler* provides symbolic constants for the feature and property names.

xml.sax.handler.**feature_namespaces**

valor: "http://xml.org/sax/features/namespaces"
true: Executa o processamento do espaço de nomes.
false: Optionally do not perform Namespace processing (implies namespace-prefixes; default).
access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.**feature_namespace_prefixes**

valor: "http://xml.org/sax/features/namespace-prefixes"
true: Report the original prefixed names and attributes used for Namespace declarations.
false: Do not report attributes used for Namespace declarations, and optionally do not report original prefixed names (default).
access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.**feature_string_interning**

valor: "http://xml.org/sax/features/string-interning"
true: All element names, prefixes, attribute names, Namespace URIs, and local names are interned using the built-in intern function.
false: Names are not necessarily interned, although they may be (default).
access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.**feature_validation**

valor: "http://xml.org/sax/features/validation"
true: Report all validation errors (implies external-general-entities and external-parameter-entities).
false: Do not report validation errors.
access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.**feature_external_ges**

valor: "http://xml.org/sax/features/external-general-entities"
true: Include all external general (text) entities.
false: Do not include external general entities.
access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.**feature_external_pes**

valor: "http://xml.org/sax/features/external-parameter-entities"
true: Include all external parameter entities, including the external DTD subset.
false: Do not include any external parameter entities, even the external DTD subset.
access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.**all_features**

Lista de todos os recursos.

xml.sax.handler.**property_lexical_handler**

valor: "http://xml.org/sax/properties/lexical-handler"
data type: xml.sax.handler.LexicalHandler (not supported in Python 2)
descrição: Um tratador de extensão opcional para eventos lexicais como comentários.
acesso: leitura/gravação

xml.sax.handler.**property_declaration_handler**

valor: "http://xml.org/sax/properties/declaration-handler"

data type: xml.sax.sax2lib.DeclHandler (not supported in Python 2)

description: An optional extension handler for DTD-related events other than notations and unparsed entities.

acesso: leitura/gravação

xml.sax.handler.**property_dom_node**

valor: "http://xml.org/sax/properties/dom-node"

data type: org.w3c.dom.Node (not supported in Python 2)

description: When parsing, the current DOM node being visited if this is a DOM iterator; when not parsing, the root DOM node for iteration.

access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.**property_xml_string**

valor: "http://xml.org/sax/properties/xml-string"

data type: Bytes

description: The literal string of characters that was the source for the current event.

acesso: somente leitura

xml.sax.handler.**all_properties**

List of all known property names.

21.10.1 ContentHandler Objects

Users are expected to subclass *ContentHandler* to support their application. The following methods are called by the parser on the appropriate events in the input document:

ContentHandler.**setDocumentLocator**(*locator*)

Called by the parser to give the application a locator for locating the origin of document events.

SAX parsers are strongly encouraged (though not absolutely required) to supply a locator: if it does so, it must supply the locator to the application by invoking this method before invoking any of the other methods in the DocumentHandler interface.

The locator allows the application to determine the end position of any document-related event, even if the parser is not reporting an error. Typically, the application will use this information for reporting its own errors (such as character content that does not match an application's business rules). The information returned by the locator is probably not sufficient for use with a search engine.

Note that the locator will return correct information only during the invocation of the events in this interface. The application should not attempt to use it at any other time.

ContentHandler.**startDocument**()

Receive notification of the beginning of a document.

The SAX parser will invoke this method only once, before any other methods in this interface or in DTDHandler (except for *setDocumentLocator()*).

ContentHandler.**endDocument**()

Receive notification of the end of a document.

The SAX parser will invoke this method only once, and it will be the last method invoked during the parse. The parser shall not invoke this method until it has either abandoned parsing (because of an unrecoverable error) or reached the end of input.

ContentHandler.**startPrefixMapping**(*prefix*, *uri*)

Begin the scope of a prefix-URI Namespace mapping.

The information from this event is not necessary for normal Namespace processing: the SAX XML reader will automatically replace prefixes for element and attribute names when the `feature_namespaces` feature is enabled (the default).

There are cases, however, when applications need to use prefixes in character data or in attribute values, where they cannot safely be expanded automatically; the *startPrefixMapping()* and *endPrefixMapping()* events supply the information to the application to expand prefixes in those contexts itself, if necessary.

Note that *startPrefixMapping()* and *endPrefixMapping()* events are not guaranteed to be properly nested relative to each-other: all *startPrefixMapping()* events will occur before the corresponding *startElement()* event, and all *endPrefixMapping()* events will occur after the corresponding *endElement()* event, but their order is not guaranteed.

`ContentHandler.`**`endPrefixMapping`**(*prefix*)

End the scope of a prefix-URI mapping.

See *startPrefixMapping()* for details. This event will always occur after the corresponding *endElement()* event, but the order of *endPrefixMapping()* events is not otherwise guaranteed.

`ContentHandler.`**`startElement`**(*name*, *attrs*)

Signals the start of an element in non-namespace mode.

The *name* parameter contains the raw XML 1.0 name of the element type as a string and the *attrs* parameter holds an object of the *Attributes* interface containing the attributes of the element. The object passed as *attrs* may be re-used by the parser; holding on to a reference to it is not a reliable way to keep a copy of the attributes. To keep a copy of the attributes, use the *copy()* method of the *attrs* object.

`ContentHandler.`**`endElement`**(*name*)

Signals the end of an element in non-namespace mode.

The *name* parameter contains the name of the element type, just as with the *startElement()* event.

`ContentHandler.`**`startElementNS`**(*name*, *qname*, *attrs*)

Signals the start of an element in namespace mode.

The *name* parameter contains the name of the element type as a `(uri, localname)` tuple, the *qname* parameter contains the raw XML 1.0 name used in the source document, and the *attrs* parameter holds an instance of the *AttributesNS* interface containing the attributes of the element. If no namespace is associated with the element, the *uri* component of *name* will be `None`. The object passed as *attrs* may be re-used by the parser; holding on to a reference to it is not a reliable way to keep a copy of the attributes. To keep a copy of the attributes, use the *copy()* method of the *attrs* object.

Parsers may set the *qname* parameter to `None`, unless the `feature_namespace_prefixes` feature is activated.

`ContentHandler.`**`endElementNS`**(*name*, *qname*)

Signals the end of an element in namespace mode.

O parâmetro *name* contém o nome do tipo de elemento, assim como no método *startElementNS()*, da mesma forma o parâmetro *qname*.

`ContentHandler.`**`characters`**(*content*)

Receive notification of character data.

The Parser will call this method to report each chunk of character data. SAX parsers may return all contiguous character data in a single chunk, or they may split it into several chunks; however, all of the characters in any single event must come from the same external entity so that the Locator provides useful information.

content may be a string or bytes instance; the `expat` reader module always produces strings.

Nota

The earlier SAX 1 interface provided by the Python XML Special Interest Group used a more Java-like interface for this method. Since most parsers used from Python did not take advantage of the older interface, the simpler signature was chosen to replace it. To convert old code to the new interface, use *content* instead of slicing content with the old *offset* and *length* parameters.

`ContentHandler.`**`ignorableWhitespace`**(*whitespace*)

Receive notification of ignorable whitespace in element content.

Validating Parsers must use this method to report each chunk of ignorable whitespace (see the W3C XML 1.0 recommendation, section 2.10): non-validating parsers may also use this method if they are capable of parsing and using content models.

SAX parsers may return all contiguous whitespace in a single chunk, or they may split it into several chunks; however, all of the characters in any single event must come from the same external entity, so that the Locator provides useful information.

`ContentHandler.`**`processingInstruction`**(*target*, *data*)

Receive notification of a processing instruction.

The Parser will invoke this method once for each processing instruction found: note that processing instructions may occur before or after the main document element.

A SAX parser should never report an XML declaration (XML 1.0, section 2.8) or a text declaration (XML 1.0, section 4.3.1) using this method.

`ContentHandler.`**`skippedEntity`**(*name*)

Receive notification of a skipped entity.

The Parser will invoke this method once for each entity skipped. Non-validating processors may skip entities if they have not seen the declarations (because, for example, the entity was declared in an external DTD subset). All processors may skip external entities, depending on the values of the `feature_external_ges` and the `feature_external_pes` properties.

21.10.2 DTDHandler Objects

As instâncias *DTDHandler* fornecem os seguintes métodos:

`DTDHandler.`**`notationDecl`**(*name*, *publicId*, *systemId*)

Handle a notation declaration event.

`DTDHandler.`**`unparsedEntityDecl`**(*name*, *publicId*, *systemId*, *ndata*)

Handle an unparsed entity declaration event.

21.10.3 EntityResolver Objects

`EntityResolver.`**`resolveEntity`**(*publicId*, *systemId*)

Resolve the system identifier of an entity and return either the system identifier to read from as a string, or an InputSource to read from. The default implementation returns *systemId*.

21.10.4 ErrorHandler Objects

Objects with this interface are used to receive error and warning information from the *XMLReader*. If you create an object that implements this interface, then register the object with your *XMLReader*, the parser will call the methods in your object to report all warnings and errors. There are three levels of errors available: warnings, (possibly) recoverable errors, and unrecoverable errors. All methods take a *SAXParseException* as the only parameter. Errors and warnings may be converted to an exception by raising the passed-in exception object.

`ErrorHandler.`**`error`**(*exception*)

Called when the parser encounters a recoverable error. If this method does not raise an exception, parsing may continue, but further document information should not be expected by the application. Allowing the parser to continue may allow additional errors to be discovered in the input document.

ErrorHandler.**fatalError**(*exception*)

Called when the parser encounters an error it cannot recover from; parsing is expected to terminate when this method returns.

ErrorHandler.**warning**(*exception*)

Called when the parser presents minor warning information to the application. Parsing is expected to continue when this method returns, and document information will continue to be passed to the application. Raising an exception in this method will cause parsing to end.

21.10.5 LexicalHandler Objects

Optional SAX2 handler for lexical events.

This handler is used to obtain lexical information about an XML document. Lexical information includes information describing the document encoding used and XML comments embedded in the document, as well as section boundaries for the DTD and for any CDATA sections. The lexical handlers are used in the same manner as content handlers.

Set the LexicalHandler of an XMLReader by using the setProperty method with the property identifier `'http://xml.org/sax/properties/lexical-handler'`.

LexicalHandler.**comment**(*content*)

Reports a comment anywhere in the document (including the DTD and outside the document element).

LexicalHandler.**startDTD**(*name*, *public_id*, *system_id*)

Reports the start of the DTD declarations if the document has an associated DTD.

LexicalHandler.**endDTD**()

Reports the end of DTD declaration.

LexicalHandler.**startCDATA**()

Reports the start of a CDATA marked section.

The contents of the CDATA marked section will be reported through the characters handler.

LexicalHandler.**endCDATA**()

Reports the end of a CDATA marked section.

21.11 `xml.sax.saxutils` — Utilitários de SAX

Código-fonte: Lib/xml/sax/saxutils.py

O módulo *`xml.sax.saxutils`* contém uma série de classes e funções que são comumente úteis ao criar aplicações SAX, seja em uso direto ou como classes base.

xml.sax.saxutils.**escape**(*data*, *entities={}*)

Escapa `'&'`, `'<'` e `'>'` em uma string de dados.

Você pode escapar outras strings de dados passando um dicionário como o parâmetro opcional *entities*. As chaves e os valores devem ser strings; cada chave será substituída pelo seu valor correspondente. Os caracteres `'&'`, `'<'` e `'>'` são sempre escapados, mesmo que *entities* seja fornecido.

> **Nota**
>
> Esta função deve ser usada apenas para escapar caracteres que não podem ser usados diretamente em XML. Não a utilize como uma função geral de tradução de strings.

xml.sax.saxutils.**unescape**(*data*, *entities={}*)

Remove escape de `'&'`, `'<'` e `'>'` em uma string de dados.

Você pode remover escape de outras strings de dados passando um dicionário como o parâmetro opcional *entities*. As chaves e os valores devem ser strings; cada chave será substituída pelo seu valor correspondente. `'&amp'`, `'<'` e `'>'` têm sempre o escape removido, mesmo que *entities* seja fornecido.

xml.sax.saxutils.**quoteattr**(*data*, *entities={}*)

Semelhante a *escape()*, mas também prepara *data* para ser usado como um valor de atributo. O valor de retorno é uma versão entre aspas de *data* com quaisquer substituições adicionais necessárias. *quoteattr()* selecionará um caractere de aspas com base no conteúdo de *data*, tentando evitar a codificação de quaisquer caracteres de aspas na string. Se já houver caracteres de aspas simples e duplas em *data*, os caracteres de aspas duplas serão codificados e *data* será colocado entre aspas duplas. A string resultante pode ser usada diretamente como um valor de atributo:

```
>>> print("<element attr=%s>" % quoteattr("ab ' cd \" ef"))
<element attr="ab ' cd &quot; ef">
```

Esta função é útil ao gerar valores de atributos para HTML ou qualquer SGML usando a sintaxe concreta de referência.

class xml.sax.saxutils.**XMLGenerator**(*out=None*, *encoding='iso-8859-1'*, *short_empty_elements=False*)

Esta classe implementa a interface *ContentHandler* escrevendo eventos SAX de volta em um documento XML. Em outras palavras, usar um *XMLGenerator* como manipulador de conteúdo reproduzirá o documento original que está sendo analisado. *out* deve ser um objeto arquivo ou similar, cujo padrão é *sys.stdout*. *encoding* é a codificação do fluxo de saída, cujo padrão é `'iso-8859-1'`. *short_empty_elements* controla a formatação de elementos sem conteúdo: se `False` (o padrão), eles são emitidos como um par de tags de início/fim; se definido como `True`, eles são emitidos como uma única tag autofechada.

Alterado na versão 3.2: Foi adicionado o parâmetro *short_empty_elements*.

class xml.sax.saxutils.**XMLFilterBase**(*base*)

Esta classe foi projetada para ficar entre um *XMLReader* e os manipuladores de eventos da aplicação cliente. Por padrão, ela não faz nada além de passar requisições para o leitor e eventos para os manipuladores sem modificações, mas subclasses podem substituir métodos específicos para modificar o fluxo de eventos ou as requisições de configuração à medida que passam.

xml.sax.saxutils.**prepare_input_source**(*source*, *base=''*)

Esta função recebe uma fonte de entrada e uma URL base opcional e retorna um objeto *InputSource* totalmente resolvido, pronto para leitura. A fonte de entrada pode ser fornecida como uma string, um objeto arquivo ou similar ou um objeto *InputSource*; os analisadores usarão esta função para implementar o argumento polimórfico *source* em seu método *parse()*.

21.12 xml.sax.xmlreader — Interface for XML parsers

Código-fonte: Lib/xml/sax/xmlreader.py

SAX parsers implement the *XMLReader* interface. They are implemented in a Python module, which must provide a function `create_parser()`. This function is invoked by *xml.sax.make_parser()* with no arguments to create a new parser object.

class xml.sax.xmlreader.**XMLReader**

Base class which can be inherited by SAX parsers.

class xml.sax.xmlreader.**IncrementalParser**

In some cases, it is desirable not to parse an input source at once, but to feed chunks of the document as they get available. Note that the reader will normally not read the entire file, but read it in chunks as well; still `parse()` won't return until the entire document is processed. So these interfaces should be used if the blocking behaviour of `parse()` is not desirable.

When the parser is instantiated it is ready to begin accepting data from the feed method immediately. After parsing has been finished with a call to close the reset method must be called to make the parser ready to accept new data, either from feed or using the parse method.

Note that these methods must *not* be called during parsing, that is, after parse has been called and before it returns.

By default, the class also implements the parse method of the XMLReader interface using the feed, close and reset methods of the IncrementalParser interface as a convenience to SAX 2.0 driver writers.

class xml.sax.xmlreader.**Locator**

Interface for associating a SAX event with a document location. A locator object will return valid results only during calls to DocumentHandler methods; at any other time, the results are unpredictable. If information is not available, methods may return `None`.

class xml.sax.xmlreader.**InputSource**(*system_id=None*)

Encapsulation of the information needed by the *XMLReader* to read entities.

This class may include information about the public identifier, system identifier, byte stream (possibly with character encoding information) and/or the character stream of an entity.

Applications will create objects of this class for use in the *XMLReader.parse()* method and for returning from EntityResolver.resolveEntity.

An *InputSource* belongs to the application, the *XMLReader* is not allowed to modify *InputSource* objects passed to it from the application, although it may make copies and modify those.

class xml.sax.xmlreader.**AttributesImpl**(*attrs*)

This is an implementation of the `Attributes` interface (see section *The Attributes Interface*). This is a dictionary-like object which represents the element attributes in a `startElement()` call. In addition to the most useful dictionary operations, it supports a number of other methods as described by the interface. Objects of this class should be instantiated by readers; *attrs* must be a dictionary-like object containing a mapping from attribute names to attribute values.

class xml.sax.xmlreader.**AttributesNSImpl**(*attrs*, *qnames*)

Namespace-aware variant of *AttributesImpl*, which will be passed to `startElementNS()`. It is derived from *AttributesImpl*, but understands attribute names as two-tuples of *namespaceURI* and *localname*. In addition, it provides a number of methods expecting qualified names as they appear in the original document. This class implements the `AttributesNS` interface (see section *The AttributesNS Interface*).

21.12.1 XMLReader Objects

The *XMLReader* interface supports the following methods:

XMLReader.**parse**(*source*)

Process an input source, producing SAX events. The *source* object can be a system identifier (a string identifying the input source – typically a file name or a URL), a *pathlib.Path* or *path-like* object, or an *InputSource* object. When *parse()* returns, the input is completely processed, and the parser object can be discarded or reset.

Alterado na versão 3.5: Added support of character streams.

Alterado na versão 3.8: Added support of path-like objects.

XMLReader.**getContentHandler**()

Return the current *ContentHandler*.

XMLReader.**setContentHandler**(*handler*)

Set the current *ContentHandler*. If no *ContentHandler* is set, content events will be discarded.

XMLReader.**getDTDHandler**()

Return the current *DTDHandler*.

XMLReader.**setDTDHandler**(*handler*)

Set the current *DTDHandler*. If no *DTDHandler* is set, DTD events will be discarded.

XMLReader.**getEntityResolver**()

Return the current *EntityResolver*.

XMLReader.**setEntityResolver**(*handler*)

Set the current *EntityResolver*. If no *EntityResolver* is set, attempts to resolve an external entity will result in opening the system identifier for the entity, and fail if it is not available.

XMLReader.**getErrorHandler**()

Return the current *ErrorHandler*.

XMLReader.**setErrorHandler**(*handler*)

Set the current error handler. If no *ErrorHandler* is set, errors will be raised as exceptions, and warnings will be printed.

XMLReader.**setLocale**(*locale*)

Allow an application to set the locale for errors and warnings.

SAX parsers are not required to provide localization for errors and warnings; if they cannot support the requested locale, however, they must raise a SAX exception. Applications may request a locale change in the middle of a parse.

XMLReader.**getFeature**(*featurename*)

Return the current setting for feature *featurename*. If the feature is not recognized, `SAXNotRecognizedException` is raised. The well-known featurenames are listed in the module *xml.sax.handler*.

XMLReader.**setFeature**(*featurename*, *value*)

Set the *featurename* to *value*. If the feature is not recognized, `SAXNotRecognizedException` is raised. If the feature or its setting is not supported by the parser, *SAXNotSupportedException* is raised.

XMLReader.**getProperty**(*propertyname*)

Return the current setting for property *propertyname*. If the property is not recognized, a `SAXNotRecognizedException` is raised. The well-known propertynames are listed in the module *xml.sax.handler*.

XMLReader.**setProperty**(*propertyname*, *value*)

Set the *propertyname* to *value*. If the property is not recognized, `SAXNotRecognizedException` is raised. If the property or its setting is not supported by the parser, *SAXNotSupportedException* is raised.

21.12.2 IncrementalParser Objects

Instances of *IncrementalParser* offer the following additional methods:

IncrementalParser.**feed**(*data*)

Process a chunk of *data*.

IncrementalParser.**close**()

Assume the end of the document. That will check well-formedness conditions that can be checked only at the end, invoke handlers, and may clean up resources allocated during parsing.

IncrementalParser.**reset**()

This method is called after close has been called to reset the parser so that it is ready to parse new documents. The results of calling parse or feed after close without calling reset are undefined.

21.12.3 Locator Objects

Instances of *Locator* provide these methods:

`Locator.`**`getColumnNumber`**`()`

Return the column number where the current event begins.

`Locator.`**`getLineNumber`**`()`

Return the line number where the current event begins.

`Locator.`**`getPublicId`**`()`

Return the public identifier for the current event.

`Locator.`**`getSystemId`**`()`

Return the system identifier for the current event.

21.12.4 InputSource Objects

`InputSource.`**`setPublicId`**`(`*id*`)`

Sets the public identifier of this *InputSource*.

`InputSource.`**`getPublicId`**`()`

Returns the public identifier of this *InputSource*.

`InputSource.`**`setSystemId`**`(`*id*`)`

Sets the system identifier of this *InputSource*.

`InputSource.`**`getSystemId`**`()`

Returns the system identifier of this *InputSource*.

`InputSource.`**`setEncoding`**`(`*encoding*`)`

Sets the character encoding of this *InputSource*.

The encoding must be a string acceptable for an XML encoding declaration (see section 4.3.3 of the XML recommendation).

The encoding attribute of the *InputSource* is ignored if the *InputSource* also contains a character stream.

`InputSource.`**`getEncoding`**`()`

Obtém a codificação de caracteres deste InputSource.

`InputSource.`**`setByteStream`**`(`*bytefile*`)`

Set the byte stream (a *binary file*) for this input source.

The SAX parser will ignore this if there is also a character stream specified, but it will use a byte stream in preference to opening a URI connection itself.

If the application knows the character encoding of the byte stream, it should set it with the setEncoding method.

`InputSource.`**`getByteStream`**`()`

Get the byte stream for this input source.

The getEncoding method will return the character encoding for this byte stream, or `None` if unknown.

`InputSource.`**`setCharacterStream`**`(`*charfile*`)`

Set the character stream (a *text file*) for this input source.

If there is a character stream specified, the SAX parser will ignore any byte stream and will not attempt to open a URI connection to the system identifier.

`InputSource.`**`getCharacterStream`**`()`

Get the character stream for this input source.

21.12.5 The `Attributes` Interface

`Attributes` objects implement a portion of the *mapping protocol*, including the methods `copy()`, `get()`, `__contains__()`, `items()`, `keys()`, and `values()`. The following methods are also provided:

`Attributes.`**`getLength`**`()`

Return the number of attributes.

`Attributes.`**`getNames`**`()`

Return the names of the attributes.

`Attributes.`**`getType`**`(`*name*`)`

Returns the type of the attribute *name*, which is normally `'CDATA'`.

`Attributes.`**`getValue`**`(`*name*`)`

Return the value of attribute *name*.

21.12.6 The `AttributesNS` Interface

This interface is a subtype of the `Attributes` interface (see section *The Attributes Interface*). All methods supported by that interface are also available on `AttributesNS` objects.

The following methods are also available:

`AttributesNS.`**`getValueByQName`**`(`*name*`)`

Return the value for a qualified name.

`AttributesNS.`**`getNameByQName`**`(`*name*`)`

Return the `(namespace, localname)` pair for a qualified *name*.

`AttributesNS.`**`getQNameByName`**`(`*name*`)`

Return the qualified name for a `(namespace, localname)` pair.

`AttributesNS.`**`getQNames`**`()`

Return the qualified names of all attributes.

21.13 `xml.parsers.expat` — Fast XML parsing using Expat

Nota

Se você precisar analisar dados não confiáveis ou não autenticados, consulte *Segurança no XML*.

O módulo *`xml.parsers.expat`* é uma interface Python para o analisador XML sem validação do Expat. O módulo fornece um único tipo de extensão, `xmlparser`, que representa o estado atual de um analisador XML. Após um objeto `xmlparser` ter sido criado, vários atributos do objeto podem ser configurados para funções manipuladoras. Quando um documento XML é alimentado no analisador, as funções de tratamento são chamadas para os dados de caracteres e marcação no documento XML.

Este módulo usa o módulo `pyexpat` para fornecer acesso ao analisador sintático Expat. O uso direto do módulo `pyexpat` foi descontinuado.

Este módulo fornece uma exceção e um objeto de tipo:

`exception` `xml.parsers.expat.`**`ExpatError`**

A exceção levantada quando o Expat relata um erro. Veja a seção *Exceções ExpatError* para mais informações sobre como interpretar erros do Expat.

exception xml.parsers.expat.**error**

Apelido para *ExpatError*.

xml.parsers.expat.**XMLParserType**

O tipo dos valores de retorno da função *ParserCreate()*.

O módulo *xml.parsers.expat* contém duas funções:

xml.parsers.expat.**ErrorString**(*errno*)

Retorna uma string explicativa para um determinado número de erro *errno*.

xml.parsers.expat.**ParserCreate**(*encoding=None*, *namespace_separator=None*)

Cria e retorna um novo objeto xmlparser. *encoding*, se especificado, deve ser uma string que nomeia a codificação usada pelos dados XML. Expat não provê tantas codificações quanto Python e seu repertório de codificações não pode ser estendido; provê UTF-8, UTF-16, ISO-8859-1 (Latin1) e ASCII. Se *encoding*[1] for fornecido, ele substituirá a codificação implícita ou explícita do documento.

Opcionalmente, o Expat pode fazer o processamento de espaço de nomes XML para você, habilitado ao fornecer um valor para *namespace_separator*. O valor deve ser uma string de um caractere; uma exceção *ValueError* será levantada se a string tiver um comprimento ilegal (None é considerado o mesmo que omissão). Quando o processamento de espaço de nomes estiver ativado, os nomes de tipos de elementos e nomes de atributos que pertencem a um espaço de nomes serão expandidos. O nome do elemento passado para os manipuladores de elemento StartElementHandler e EndElementHandler será a concatenação do URI do espaço de nomes, o caractere separador do espaço de nomes e a parte local do nome. Se o separador do espaço de nomes for um byte zero (chr(0)), então o URI do espaço de nomes e a parte local serão concatenados sem qualquer separador.

Por exemplo, se *namespace_separator* for definido como um caractere de espaço (' ') e o seguinte documento for analisado:

```
<?xml version="1.0"?>
<root xmlns    = "http://default-namespace.org/"
      xmlns:py = "http://www.python.org/ns/">
  <py:elem1 />
  <elem2 xmlns="" />
</root>
```

StartElementHandler receberá as seguintes strings para cada elemento:

```
http://default-namespace.org/ root
http://www.python.org/ns/ elem1
elem2
```

Devido a limitações na biblioteca Expat usada por pyexpat, a instância xmlparser retornada só pode ser usada para analisar um único documento XML. Chame ParserCreate para cada documento para fornecer instâncias exclusivas do analisador sintático.

Ver também

Expat, analisador sintático de XML
Site do projeto Expat

[1] The encoding string included in XML output should conform to the appropriate standards. For example, "UTF-8" is valid, but "UTF8" is not. See https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-EncodingDecl and https://www.iana.org/assignments/character-sets/character-sets.xhtml.

21.13.1 Objetos XMLParser

`xmlparser` objects have the following methods:

`xmlparser.`**`Parse`**`(`*data*[, *isfinal*]`)`

Parses the contents of the string *data*, calling the appropriate handler functions to process the parsed data. *isfinal* must be true on the final call to this method; it allows the parsing of a single file in fragments, not the submission of multiple files. *data* can be the empty string at any time.

`xmlparser.`**`ParseFile`**`(`*file*`)`

Parse XML data reading from the object *file*. *file* only needs to provide the `read(nbytes)` method, returning the empty string when there's no more data.

`xmlparser.`**`SetBase`**`(`*base*`)`

Sets the base to be used for resolving relative URIs in system identifiers in declarations. Resolving relative identifiers is left to the application: this value will be passed through as the *base* argument to the *`ExternalEntityRefHandler()`*, *`NotationDeclHandler()`*, and *`UnparsedEntityDeclHandler()`* functions.

`xmlparser.`**`GetBase`**`()`

Returns a string containing the base set by a previous call to *`SetBase()`*, or `None` if *`SetBase()`* hasn't been called.

`xmlparser.`**`GetInputContext`**`()`

Returns the input data that generated the current event as a string. The data is in the encoding of the entity which contains the text. When called while an event handler is not active, the return value is `None`.

`xmlparser.`**`ExternalEntityParserCreate`**`(`*context*[, *encoding*]`)`

Create a "child" parser which can be used to parse an external parsed entity referred to by content parsed by the parent parser. The *context* parameter should be the string passed to the *`ExternalEntityRefHandler()`* handler function, described below. The child parser is created with the *`ordered_attributes`* and *`specified_attributes`* set to the values of this parser.

`xmlparser.`**`SetParamEntityParsing`**`(`*flag*`)`

Control parsing of parameter entities (including the external DTD subset). Possible *flag* values are `XML_PARAM_ENTITY_PARSING_NEVER`, `XML_PARAM_ENTITY_PARSING_UNLESS_STANDALONE` and `XML_PARAM_ENTITY_PARSING_ALWAYS`. Return true if setting the flag was successful.

`xmlparser.`**`UseForeignDTD`**`(`[*flag*]`)`

Calling this with a true value for *flag* (the default) will cause Expat to call the *`ExternalEntityRefHandler`* with *`None`* for all arguments to allow an alternate DTD to be loaded. If the document does not contain a document type declaration, the *`ExternalEntityRefHandler`* will still be called, but the *`StartDoctypeDeclHandler`* and *`EndDoctypeDeclHandler`* will not be called.

Passing a false value for *flag* will cancel a previous call that passed a true value, but otherwise has no effect.

This method can only be called before the *`Parse()`* or *`ParseFile()`* methods are called; calling it after either of those have been called causes *`ExpatError`* to be raised with the *`code`* attribute set to `errors.codes[errors.XML_ERROR_CANT_CHANGE_FEATURE_ONCE_PARSING]`.

`xmlparser.`**`SetReparseDeferralEnabled`**`(`*enabled*`)`

> **Aviso**
>
> Calling `SetReparseDeferralEnabled(False)` has security implications, as detailed below; please make sure to understand these consequences prior to using the `SetReparseDeferralEnabled` method.

Expat 2.6.0 introduced a security mechanism called "reparse deferral" where instead of causing denial of service through quadratic runtime from reparsing large tokens, reparsing of unfinished tokens is now delayed by default until a sufficient amount of input is reached. Due to this delay, registered handlers may — depending of the sizing of input chunks pushed to Expat — no longer be called right after pushing new input to the parser.

Where immediate feedback and taking over responsibility of protecting against denial of service from large tokens are both wanted, calling `SetReparseDeferralEnabled(False)` disables reparse deferral for the current Expat parser instance, temporarily or altogether. Calling `SetReparseDeferralEnabled(True)` allows re-enabling reparse deferral.

Note that *`SetReparseDeferralEnabled()`* has been backported to some prior releases of CPython as a security fix. Check for availability of *`SetReparseDeferralEnabled()`* using *`hasattr()`* if used in code running across a variety of Python versions.

Adicionado na versão 3.13.

`xmlparser.`**`GetReparseDeferralEnabled`**`()`

Returns whether reparse deferral is currently enabled for the given Expat parser instance.

Adicionado na versão 3.13.

`xmlparser` objects have the following attributes:

`xmlparser.`**`buffer_size`**

The size of the buffer used when *`buffer_text`* is true. A new buffer size can be set by assigning a new integer value to this attribute. When the size is changed, the buffer will be flushed.

`xmlparser.`**`buffer_text`**

Setting this to true causes the `xmlparser` object to buffer textual content returned by Expat to avoid multiple calls to the *`CharacterDataHandler()`* callback whenever possible. This can improve performance substantially since Expat normally breaks character data into chunks at every line ending. This attribute is false by default, and may be changed at any time. Note that when it is false, data that does not contain newlines may be chunked too.

`xmlparser.`**`buffer_used`**

If *`buffer_text`* is enabled, the number of bytes stored in the buffer. These bytes represent UTF-8 encoded text. This attribute has no meaningful interpretation when *`buffer_text`* is false.

`xmlparser.`**`ordered_attributes`**

Setting this attribute to a non-zero integer causes the attributes to be reported as a list rather than a dictionary. The attributes are presented in the order found in the document text. For each attribute, two list entries are presented: the attribute name and the attribute value. (Older versions of this module also used this format.) By default, this attribute is false; it may be changed at any time.

`xmlparser.`**`specified_attributes`**

If set to a non-zero integer, the parser will report only those attributes which were specified in the document instance and not those which were derived from attribute declarations. Applications which set this need to be especially careful to use what additional information is available from the declarations as needed to comply with the standards for the behavior of XML processors. By default, this attribute is false; it may be changed at any time.

The following attributes contain values relating to the most recent error encountered by an `xmlparser` object, and will only have correct values once a call to `Parse()` or `ParseFile()` has raised an *`xml.parsers.expat.ExpatError`* exception.

`xmlparser.`**`ErrorByteIndex`**

Byte index at which an error occurred.

`xmlparser.`**`ErrorCode`**

Numeric code specifying the problem. This value can be passed to the *`ErrorString()`* function, or compared to one of the constants defined in the `errors` object.

`xmlparser.`**`ErrorColumnNumber`**

Column number at which an error occurred.

`xmlparser.`**`ErrorLineNumber`**

Line number at which an error occurred.

The following attributes contain values relating to the current parse location in an `xmlparser` object. During a callback reporting a parse event they indicate the location of the first of the sequence of characters that generated the event. When called outside of a callback, the position indicated will be just past the last parse event (regardless of whether there was an associated callback).

`xmlparser.`**`CurrentByteIndex`**

Current byte index in the parser input.

`xmlparser.`**`CurrentColumnNumber`**

Current column number in the parser input.

`xmlparser.`**`CurrentLineNumber`**

Current line number in the parser input.

Here is the list of handlers that can be set. To set a handler on an `xmlparser` object *o*, use `o.handlername = func`. *handlername* must be taken from the following list, and *func* must be a callable object accepting the correct number of arguments. The arguments are all strings, unless otherwise stated.

`xmlparser.`**`XmlDeclHandler`**(*version*, *encoding*, *standalone*)

Called when the XML declaration is parsed. The XML declaration is the (optional) declaration of the applicable version of the XML recommendation, the encoding of the document text, and an optional "standalone" declaration. *version* and *encoding* will be strings, and *standalone* will be `1` if the document is declared standalone, `0` if it is declared not to be standalone, or `-1` if the standalone clause was omitted. This is only available with Expat version 1.95.0 or newer.

`xmlparser.`**`StartDoctypeDeclHandler`**(*doctypeName*, *systemId*, *publicId*, *has_internal_subset*)

Called when Expat begins parsing the document type declaration (`<!DOCTYPE ...`). The *doctypeName* is provided exactly as presented. The *systemId* and *publicId* parameters give the system and public identifiers if specified, or `None` if omitted. *has_internal_subset* will be true if the document contains and internal document declaration subset. This requires Expat version 1.2 or newer.

`xmlparser.`**`EndDoctypeDeclHandler`**()

Called when Expat is done parsing the document type declaration. This requires Expat version 1.2 or newer.

`xmlparser.`**`ElementDeclHandler`**(*name*, *model*)

Called once for each element type declaration. *name* is the name of the element type, and *model* is a representation of the content model.

`xmlparser.`**`AttlistDeclHandler`**(*elname*, *attname*, *type*, *default*, *required*)

Called for each declared attribute for an element type. If an attribute list declaration declares three attributes, this handler is called three times, once for each attribute. *elname* is the name of the element to which the declaration applies and *attname* is the name of the attribute declared. The attribute type is a string passed as *type*; the possible values are `'CDATA'`, `'ID'`, `'IDREF'`, ... *default* gives the default value for the attribute used when the attribute is not specified by the document instance, or `None` if there is no default value (`#IMPLIED` values). If the attribute is required to be given in the document instance, *required* will be true. This requires Expat version 1.95.0 or newer.

`xmlparser.`**`StartElementHandler`**(*name*, *attributes*)

Called for the start of every element. *name* is a string containing the element name, and *attributes* is the element attributes. If *ordered_attributes* is true, this is a list (see *ordered_attributes* for a full description). Otherwise it's a dictionary mapping names to values.

`xmlparser.`**`EndElementHandler`**(*name*)

Called for the end of every element.

`xmlparser.`**`ProcessingInstructionHandler`**(*target*, *data*)

Called for every processing instruction.

`xmlparser.`**`CharacterDataHandler`**(*data*)

Called for character data. This will be called for normal character data, CDATA marked content, and ignorable whitespace. Applications which must distinguish these cases can use the *StartCdataSectionHandler*, *EndCdataSectionHandler*, and *ElementDeclHandler* callbacks to collect the required information.

Note that the character data may be chunked even if it is short and so you may receive more than one call to *CharacterDataHandler()*. Set the *buffer_text* instance attribute to `True` to avoid that.

`xmlparser.`**`UnparsedEntityDeclHandler`**(*entityName*, *base*, *systemId*, *publicId*, *notationName*)

Called for unparsed (NDATA) entity declarations. This is only present for version 1.2 of the Expat library; for more recent versions, use *EntityDeclHandler* instead. (The underlying function in the Expat library has been declared obsolete.)

`xmlparser.`**`EntityDeclHandler`**(*entityName*, *is_parameter_entity*, *value*, *base*, *systemId*, *publicId*, *notationName*)

Called for all entity declarations. For parameter and internal entities, *value* will be a string giving the declared contents of the entity; this will be `None` for external entities. The *notationName* parameter will be `None` for parsed entities, and the name of the notation for unparsed entities. *is_parameter_entity* will be true if the entity is a parameter entity or false for general entities (most applications only need to be concerned with general entities). This is only available starting with version 1.95.0 of the Expat library.

`xmlparser.`**`NotationDeclHandler`**(*notationName*, *base*, *systemId*, *publicId*)

Called for notation declarations. *notationName*, *base*, and *systemId*, and *publicId* are strings if given. If the public identifier is omitted, *publicId* will be `None`.

`xmlparser.`**`StartNamespaceDeclHandler`**(*prefix*, *uri*)

Called when an element contains a namespace declaration. Namespace declarations are processed before the *StartElementHandler* is called for the element on which declarations are placed.

`xmlparser.`**`EndNamespaceDeclHandler`**(*prefix*)

Called when the closing tag is reached for an element that contained a namespace declaration. This is called once for each namespace declaration on the element in the reverse of the order for which the *StartNamespaceDeclHandler* was called to indicate the start of each namespace declaration's scope. Calls to this handler are made after the corresponding *EndElementHandler* for the end of the element.

`xmlparser.`**`CommentHandler`**(*data*)

Called for comments. *data* is the text of the comment, excluding the leading `'<!--'` and trailing `'-->'`.

`xmlparser.`**`StartCdataSectionHandler`**()

Called at the start of a CDATA section. This and *EndCdataSectionHandler* are needed to be able to identify the syntactical start and end for CDATA sections.

`xmlparser.`**`EndCdataSectionHandler`**()

Called at the end of a CDATA section.

`xmlparser.`**`DefaultHandler`**(*data*)

Called for any characters in the XML document for which no applicable handler has been specified. This means characters that are part of a construct which could be reported, but for which no handler has been supplied.

`xmlparser.`**`DefaultHandlerExpand`**(*data*)

This is the same as the *DefaultHandler()*, but doesn't inhibit expansion of internal entities. The entity reference will not be passed to the default handler.

`xmlparser.`**`NotStandaloneHandler`**()

Called if the XML document hasn't been declared as being a standalone document. This happens when there is an external subset or a reference to a parameter entity, but the XML declaration does not set standalone to `yes` in an XML declaration. If this handler returns `0`, then the parser will raise an `XML_ERROR_NOT_STANDALONE` error. If this handler is not set, no exception is raised by the parser for this condition.

`xmlparser.`**`ExternalEntityRefHandler`**(*context*, *base*, *systemId*, *publicId*)

Called for references to external entities. *base* is the current base, as set by a previous call to *SetBase()*. The public and system identifiers, *systemId* and *publicId*, are strings if given; if the public identifier is not given, *publicId* will be `None`. The *context* value is opaque and should only be used as described below.

For external entities to be parsed, this handler must be implemented. It is responsible for creating the sub-parser using `ExternalEntityParserCreate(context)`, initializing it with the appropriate callbacks,

and parsing the entity. This handler should return an integer; if it returns `0`, the parser will raise an `XML_ERROR_EXTERNAL_ENTITY_HANDLING` error, otherwise parsing will continue.

If this handler is not provided, external entities are reported by the *`DefaultHandler`* callback, if provided.

21.13.2 Exceções ExpatError

`ExpatError` exceptions have a number of interesting attributes:

`ExpatError.`**`code`**

Expat's internal error number for the specific error. The *`errors.messages`* dictionary maps these error numbers to Expat's error messages. For example:

```
from xml.parsers.expat import ParserCreate, ExpatError, errors

p = ParserCreate()
try:
    p.Parse(some_xml_document)
except ExpatError as err:
    print("Error:", errors.messages[err.code])
```

The *`errors`* module also provides error message constants and a dictionary *`codes`* mapping these messages back to the error codes, see below.

`ExpatError.`**`lineno`**

Line number on which the error was detected. The first line is numbered `1`.

`ExpatError.`**`offset`**

Character offset into the line where the error occurred. The first column is numbered `0`.

21.13.3 Exemplo

The following program defines three handlers that just print out their arguments.

```
import xml.parsers.expat

# 3 handler functions
def start_element(name, attrs):
    print('Start element:', name, attrs)
def end_element(name):
    print('End element:', name)
def char_data(data):
    print('Character data:', repr(data))

p = xml.parsers.expat.ParserCreate()

p.StartElementHandler = start_element
p.EndElementHandler = end_element
p.CharacterDataHandler = char_data

p.Parse("""<?xml version="1.0"?>
<parent id="top"><child1 name="paul">Text goes here</child1>
<child2 name="fred">More text</child2>
</parent>""", 1)
```

The output from this program is:

```
Start element: parent {'id': 'top'}
Start element: child1 {'name': 'paul'}
```

(continua na próxima página)

(continuação da página anterior)

```
Character data: 'Text goes here'
End element: child1
Character data: '\n'
Start element: child2 {'name': 'fred'}
Character data: 'More text'
End element: child2
Character data: '\n'
End element: parent
```

21.13.4 Content Model Descriptions

Content models are described using nested tuples. Each tuple contains four values: the type, the quantifier, the name, and a tuple of children. Children are simply additional content model descriptions.

The values of the first two fields are constants defined in the *xml.parsers.expat.model* module. These constants can be collected in two groups: the model type group and the quantifier group.

The constants in the model type group are:

xml.parsers.expat.model.**XML_CTYPE_ANY**

The element named by the model name was declared to have a content model of ANY.

xml.parsers.expat.model.**XML_CTYPE_CHOICE**

The named element allows a choice from a number of options; this is used for content models such as (A | B | C).

xml.parsers.expat.model.**XML_CTYPE_EMPTY**

Elementos declarados como EMPTY têm este tipo de modelo.

xml.parsers.expat.model.**XML_CTYPE_MIXED**

xml.parsers.expat.model.**XML_CTYPE_NAME**

xml.parsers.expat.model.**XML_CTYPE_SEQ**

Models which represent a series of models which follow one after the other are indicated with this model type. This is used for models such as (A, B, C).

The constants in the quantifier group are:

xml.parsers.expat.model.**XML_CQUANT_NONE**

No modifier is given, so it can appear exactly once, as for A.

xml.parsers.expat.model.**XML_CQUANT_OPT**

The model is optional: it can appear once or not at all, as for A?.

xml.parsers.expat.model.**XML_CQUANT_PLUS**

The model must occur one or more times (like A+).

xml.parsers.expat.model.**XML_CQUANT_REP**

The model must occur zero or more times, as for A*.

21.13.5 Constantes de erro de expatriados

The following constants are provided in the *xml.parsers.expat.errors* module. These constants are useful in interpreting some of the attributes of the ExpatError exception objects raised when an error has occurred. Since for backwards compatibility reasons, the constants' value is the error *message* and not the numeric error *code*, you do this by comparing its *code* attribute with errors.codes[errors.XML_ERROR_*CONSTANT_NAME*].

The errors module has the following attributes:

xml.parsers.expat.errors.**codes**

A dictionary mapping string descriptions to their error codes.

Adicionado na versão 3.2.

xml.parsers.expat.errors.**messages**

A dictionary mapping numeric error codes to their string descriptions.

Adicionado na versão 3.2.

xml.parsers.expat.errors.**XML_ERROR_ASYNC_ENTITY**

xml.parsers.expat.errors.**XML_ERROR_ATTRIBUTE_EXTERNAL_ENTITY_REF**

An entity reference in an attribute value referred to an external entity instead of an internal entity.

xml.parsers.expat.errors.**XML_ERROR_BAD_CHAR_REF**

A character reference referred to a character which is illegal in XML (for example, character `0`, or '`&#0;`').

xml.parsers.expat.errors.**XML_ERROR_BINARY_ENTITY_REF**

An entity reference referred to an entity which was declared with a notation, so cannot be parsed.

xml.parsers.expat.errors.**XML_ERROR_DUPLICATE_ATTRIBUTE**

An attribute was used more than once in a start tag.

xml.parsers.expat.errors.**XML_ERROR_INCORRECT_ENCODING**

xml.parsers.expat.errors.**XML_ERROR_INVALID_TOKEN**

Raised when an input byte could not properly be assigned to a character; for example, a NUL byte (value `0`) in a UTF-8 input stream.

xml.parsers.expat.errors.**XML_ERROR_JUNK_AFTER_DOC_ELEMENT**

Something other than whitespace occurred after the document element.

xml.parsers.expat.errors.**XML_ERROR_MISPLACED_XML_PI**

An XML declaration was found somewhere other than the start of the input data.

xml.parsers.expat.errors.**XML_ERROR_NO_ELEMENTS**

The document contains no elements (XML requires all documents to contain exactly one top-level element)..

xml.parsers.expat.errors.**XML_ERROR_NO_MEMORY**

Expat was not able to allocate memory internally.

xml.parsers.expat.errors.**XML_ERROR_PARAM_ENTITY_REF**

A parameter entity reference was found where it was not allowed.

xml.parsers.expat.errors.**XML_ERROR_PARTIAL_CHAR**

An incomplete character was found in the input.

xml.parsers.expat.errors.**XML_ERROR_RECURSIVE_ENTITY_REF**

An entity reference contained another reference to the same entity; possibly via a different name, and possibly indirectly.

xml.parsers.expat.errors.**XML_ERROR_SYNTAX**

Some unspecified syntax error was encountered.

xml.parsers.expat.errors.**XML_ERROR_TAG_MISMATCH**

An end tag did not match the innermost open start tag.

xml.parsers.expat.errors.**XML_ERROR_UNCLOSED_TOKEN**

Some token (such as a start tag) was not closed before the end of the stream or the next token was encountered.

xml.parsers.expat.errors.**XML_ERROR_UNDEFINED_ENTITY**

A reference was made to an entity which was not defined.

xml.parsers.expat.errors.**XML_ERROR_UNKNOWN_ENCODING**

The document encoding is not supported by Expat.

xml.parsers.expat.errors.**XML_ERROR_UNCLOSED_CDATA_SECTION**

A CDATA marked section was not closed.

xml.parsers.expat.errors.**XML_ERROR_EXTERNAL_ENTITY_HANDLING**

xml.parsers.expat.errors.**XML_ERROR_NOT_STANDALONE**

The parser determined that the document was not "standalone" though it declared itself to be in the XML declaration, and the `NotStandaloneHandler` was set and returned `0`.

xml.parsers.expat.errors.**XML_ERROR_UNEXPECTED_STATE**

xml.parsers.expat.errors.**XML_ERROR_ENTITY_DECLARED_IN_PE**

xml.parsers.expat.errors.**XML_ERROR_FEATURE_REQUIRES_XML_DTD**

An operation was requested that requires DTD support to be compiled in, but Expat was configured without DTD support. This should never be reported by a standard build of the *xml.parsers.expat* module.

xml.parsers.expat.errors.**XML_ERROR_CANT_CHANGE_FEATURE_ONCE_PARSING**

A behavioral change was requested after parsing started that can only be changed before parsing has started. This is (currently) only raised by `UseForeignDTD()`.

xml.parsers.expat.errors.**XML_ERROR_UNBOUND_PREFIX**

An undeclared prefix was found when namespace processing was enabled.

xml.parsers.expat.errors.**XML_ERROR_UNDECLARING_PREFIX**

The document attempted to remove the namespace declaration associated with a prefix.

xml.parsers.expat.errors.**XML_ERROR_INCOMPLETE_PE**

A parameter entity contained incomplete markup.

xml.parsers.expat.errors.**XML_ERROR_XML_DECL**

The document contained no document element at all.

xml.parsers.expat.errors.**XML_ERROR_TEXT_DECL**

There was an error parsing a text declaration in an external entity.

xml.parsers.expat.errors.**XML_ERROR_PUBLICID**

Characters were found in the public id that are not allowed.

xml.parsers.expat.errors.**XML_ERROR_SUSPENDED**

The requested operation was made on a suspended parser, but isn't allowed. This includes attempts to provide additional input or to stop the parser.

xml.parsers.expat.errors.**XML_ERROR_NOT_SUSPENDED**

An attempt to resume the parser was made when the parser had not been suspended.

xml.parsers.expat.errors.**XML_ERROR_ABORTED**

This should not be reported to Python applications.

xml.parsers.expat.errors.**XML_ERROR_FINISHED**

The requested operation was made on a parser which was finished parsing input, but isn't allowed. This includes attempts to provide additional input or to stop the parser.

xml.parsers.expat.errors.**XML_ERROR_SUSPEND_PE**

xml.parsers.expat.errors.**XML_ERROR_RESERVED_PREFIX_XML**

An attempt was made to undeclare reserved namespace prefix `xml` or to bind it to another namespace URI.

xml.parsers.expat.errors.**XML_ERROR_RESERVED_PREFIX_XMLNS**

An attempt was made to declare or undeclare reserved namespace prefix `xmlns`.

xml.parsers.expat.errors.**XML_ERROR_RESERVED_NAMESPACE_URI**

An attempt was made to bind the URI of one the reserved namespace prefixes `xml` and `xmlns` to another namespace prefix.

xml.parsers.expat.errors.**XML_ERROR_INVALID_ARGUMENT**

This should not be reported to Python applications.

xml.parsers.expat.errors.**XML_ERROR_NO_BUFFER**

This should not be reported to Python applications.

xml.parsers.expat.errors.**XML_ERROR_AMPLIFICATION_LIMIT_BREACH**

The limit on input amplification factor (from DTD and entities) has been breached.

xml.parsers.expat.errors.**XML_ERROR_NOT_STARTED**

The parser was tried to be stopped or suspended before it started.

Adicionado na versão 3.14.

CAPÍTULO 22

Protocolos de Internet e Suporte

Os módulos descritos neste capítulo implementam protocolos de internet e suporte para tecnologia relacionada. Todos eles são implementados em Python. A maioria desses módulos requer a presença do módulo dependente do sistema *socket*, que atualmente é suportado na maioria das plataformas populares. Segue uma visão geral:

22.1 webbrowser — Controlador de navegador web conveniente

Código-fonte: Lib/webbrowser.py

O módulo *webbrowser* fornece uma interface de alto nível para permitir a exibição de documentos baseados na web para os usuários. Na maioria das circunstâncias, simplesmente chamar a função *open()* deste módulo fará a coisa certa.

No Unix, navegadores gráficos são preferidos no X11, mas navegadores em modo texto serão usados se navegadores gráficos não estiverem disponíveis ou um display X11 não estiver disponível. Se navegadores em modo texto forem usados, o processo de chamada será bloqueado até que o usuário saia do navegador.

Se a variável de ambiente BROWSER existir, ela será interpretada como a lista separada por *os.pathsep* de navegadores a serem tentados antes dos padrões da plataforma. Quando o valor de uma parte da lista contém a string %s, então ela é interpretada como uma linha de comando literal do navegador a ser usada com o argumento URL substituído por %s; se o valor for uma única palavra que se refere a um dos navegadores já registrados, este navegador será adicionado à frente da lista de pesquisa; se a parte não contiver %s, ela será simplesmente interpretada como o nome do navegador a ser iniciado.[1]

Alterado na versão 3.14: A variável BROWSER agora também pode ser usada para reordenar a lista de padrões da plataforma. Isso é particularmente útil no macOS, onde os padrões da plataforma não se referem às ferramentas de linha de comando em PATH.

Para plataformas não-Unix, ou quando um navegador remoto estiver disponível no Unix, o processo de controle não esperará que o usuário termine de usar o navegador, mas permitirá que o navegador remoto mantenha suas próprias janelas na tela. Se os navegadores remotos não estiverem disponíveis no Unix, o processo de controle iniciará um novo navegador e aguardará.

No iOS, a variável de ambiente BROWSER, bem como quaisquer argumentos que controlem direitos autorais, preferência do navegador e criação de nova aba/janela serão ignorados. As páginas web *sempre* serão abertas no navegador preferido do usuário, em uma nova aba, com o navegador sendo trazido para o primeiro plano. O uso do módulo

[1] Executáveis nomeados aqui sem um caminho completo serão pesquisados nos diretórios fornecidos na variável de ambiente PATH.

webbrowser no iOS requer o módulo *ctypes*. Se *ctypes* não estiver disponível, as chamadas para *open()* falharão.

O script **webbrowser** pode ser usado como uma interface de linha de comando para o módulo. Ele aceita uma URL como argumento. Ele aceita os seguintes parâmetros opcionais:

-n, --new-window

Abre a URL em uma nova janela do navegador, se possível.

-t, --new-tab

Abre a URL em uma nova aba do navegador.

As opções são, naturalmente, mutuamente exclusivas. Exemplo de uso:

```
python -m webbrowser -t "https://www.python.org"
```

Disponibilidade: not WASI, not Android.

A seguinte exceção é definida:

exception webbrowser.**Error**

Exceção levantada quando ocorre um erro de controle do navegador.

As seguintes funções estão definidas:

webbrowser.**open**(*url*, *new=0*, *autoraise=True*)

Exibe *url* usando o navegador padrão. Se *new* for 0, o *url* será aberto na mesma janela do navegador, se possível. Se *new* for 1, uma nova janela do navegador será aberta, se possível. Se *new* for 2, uma nova página do navegador ("aba" ou "guia") será aberta, se possível. Se *autoraise* for `True`, a janela será levantada, se possível (observe que, em muitos gerenciadores de janelas, isso ocorrerá independentemente da configuração dessa variável).

Retorna `True` se um navegador foi iniciado com sucesso, `False` caso contrário.

Note que em algumas plataformas, tentar abrir um nome de arquivo usando esta função pode funcionar e iniciar o programa associado do sistema operacional. No entanto, isso não é suportado nem portátil.

Levanta um *evento de auditoria* `webbrowser.open` com o argumento `url`.

webbrowser.**open_new**(*url*)

Abre *url* em uma nova janela do navegador padrão, se possível; caso contrário, abre *url* na única janela do navegador.

Retorna `True` se um navegador foi iniciado com sucesso, `False` caso contrário.

webbrowser.**open_new_tab**(*url*)

Abre *url* em uma nova página ("aba" ou "guia") do navegador padrão, se possível, caso contrário, equivalente a *open_new()*.

Retorna `True` se um navegador foi iniciado com sucesso, `False` caso contrário.

webbrowser.**get**(*using=None*)

Retorna um objeto controlador para o tipo de navegador *using*. Se *using* for `None`, retorna um controlador para um navegador padrão apropriado para o ambiente do chamador.

webbrowser.**register**(*name*, *constructor*, *instance=None*, *, *preferred=False*)

Registra o tipo de navegador *name*. Uma vez que um tipo de navegador é registrado, a função *get()* pode retornar um controlador para esse tipo de navegador. Se *instance* não for fornecido, ou for `None`, *constructor* será chamado sem parâmetros para criar uma instância quando necessário. Se *instance* for fornecido, *constructor* nunca será chamado, e pode ser `None`.

Definir *preferred* como `True` torna este navegador um resultado preferencial para uma chamada *get()* sem argumento. Caso contrário, este ponto de entrada só é útil se você planeja definir a variável `BROWSER` ou chamar *get()* com um argumento não vazio que corresponda ao nome de um manipulador que você declarar.

Alterado na versão 3.7: Parâmetro somente-nomeado *preferred* foi adicionado.

Vários tipos de navegadores são predefinidos. Esta tabela fornece os nomes de tipos que podem ser passados para a função `get()` e as instanciações correspondentes para as classes do controlador, todas definidas neste módulo.

Nome do tipo	Nome da classe	Notas
`'mozilla'`	`Mozilla('mozilla')`	
`'firefox'`	`Mozilla('mozilla')`	
`'epiphany'`	`Epiphany('epiphany')`	
`'kfmclient'`	`Konqueror()`	(1)
`'konqueror'`	`Konqueror()`	(1)
`'kfm'`	`Konqueror()`	(1)
`'opera'`	`Opera()`	
`'links'`	`GenericBrowser('links')`	
`'elinks'`	`Elinks('elinks')`	
`'lynx'`	`GenericBrowser('lynx')`	
`'w3m'`	`GenericBrowser('w3m')`	
`'windows-default'`	`WindowsDefault`	(2)
`'macosx'`	`MacOSXOSAScript('default')`	(3)
`'safari'`	`MacOSXOSAScript('safari')`	(3)
`'google-chrome'`	`Chrome('google-chrome')`	
`'chrome'`	`Chrome('chrome')`	
`'chromium'`	`Chromium('chromium')`	
`'chromium-browser'`	`Chromium('chromium-browser')`	
`'iosbrowser'`	`IOSBrowser`	(4)

Notas:

(1) "Konqueror" é o gerenciador de arquivos para o ambiente de desktop KDE para Unix, e só faz sentido usar se estiver no KDE. Alguma maneira de detectar o KDE de forma confiável seria legal; a variável `KDEDIR` não é suficiente. Note também que o nome "kfm" é usado mesmo ao usar o comando **`konqueror`** com o KDE 2 — a implementação seleciona a melhor estratégia para executar o Konqueror.

(2) Somente em plataformas Windows.

(3) Somente no macOS.

(4) Somente no iOS.

Adicionado na versão 3.2: Uma nova classe `MacOSXOSAScript` foi adicionada e é usada no Mac em vez da classe anterior `MacOSX`. Isso adiciona suporte para abrir navegadores que não estão definidos como padrão do sistema operacional no momento.

Adicionado na versão 3.3: Foi adicionado suporte para Chrome/Chromium.

Alterado na versão 3.12: O suporte para vários navegadores obsoletos foi removido. Os navegadores removidos incluem Grail, Mosaic, Netscape, Galeon, Skipstone, Iceape e Firefox versões 35 e abaixo.

Alterado na versão 3.13: Foi adicionado suporte para iOS.

Aqui estão alguns exemplos simples:

```
url = 'https://docs.python.org/'

# Abre a URL em uma nova aba, se uma janela do navegador já estiver aberta.
webbrowser.open_new_tab(url)

# Abre a URL em uma nova janela, aumentando a janela se possível.
webbrowser.open_new(url)
```

22.1.1 Objetos controladores de navegador

Os controladores de navegador fornecem o atributo *name* e os três métodos a seguir que são paralelos às funções de conveniência em nível de módulo:

controller.**name**

Nome dependente do sistema para o navegador.

controller.**open**(*url*, *new=0*, *autoraise=True*)

Exibe *url* usando o navegador manipulado por este controlador. Se *new* for 1, uma nova janela do navegador será aberta, se possível. Se *new* for 2, uma nova página do navegador ("aba" ou "guia") será aberta, se possível.

controller.**open_new**(*url*)

Abre *url* em uma nova janela do navegador manipulado por este controlador, se possível, caso contrário, abra *url* na única janela do navegador. Apelido para *open_new()*.

controller.**open_new_tab**(*url*)

Abre *url* em uma nova página ("aba" ou "guia") do navegador manipulado por este controlador, se possível, caso contrário, equivalente a *open_new()*.

22.2 wsgiref — Implementação de referência e utilitários WSGI

Código-fonte: Lib/wsgiref

A Interface de Gateway para Servidor Web (em inglês, Web Server Gateway Interface - WSGI) é uma interface padrão localizada entre software de servidores web e aplicações web escritas em Python. Ter um interface padrão torna mais fácil a utilização de uma aplicação que suporta WSGI com inúmeros diferentes servidores web.

Apenas autores de servidores web e frameworks de programação necessitam saber todos os detalhes e especificidades do design WSGI. Você não precisa entender todos os detalhes do WSGI para apenas instalar uma aplicação WSGI ou para escrever uma aplicação web usando um framework existente.

wsgiref é uma implementação de referência da especificação WSGI que pode ser utilizada para adicionar suporte WSGI em um servidor web ou framework. Ele provê utilidades para manipulação de variáveis de ambiente WSGI e cabeçalhos de respostas, classes base para implementação de servidores WSGI, uma demo de um servidor HTTP que serve aplicações WSGI, tipos para verificação de tipos estáticos e uma ferramenta de validação que confere se servidores WSGI e aplicações estão de acordo com a especificação WSGI (**PEP 3333**).

Veja wsgi.readthedocs.io para mais informações sobre WSGI, além de links para tutoriais e outros recursos.

22.2.1 wsgiref.util – Utilidades do ambiente WSGI

Este módulo fornece uma variedade de funções de utilidade para trabalhar com ambientes WSGI. Um ambiente WSGI é um dicionário contendo variáveis de solicitação HTTP, conforme descrito na **PEP 3333**. Todas as funções que recebem um parâmetro *environ* esperam que um dicionário compatível com WSGI seja fornecido; consulte a **PEP 3333** para uma especificação detalhada e *WSGIEnvironment* para um apelido de tipo que pode ser usado em anotações de tipos.

wsgiref.util.**guess_scheme**(*environ*)

Retorna uma sugestão sobre `wsgi.url_scheme` ser "http" ou "https" buscando por uma variável de ambiente `HTTPS` dentro do dicionário *environ*. O valor de retorno é uma string.

Esta função é útil ao criar um gateway que encapsula CGI ou um protocolo semelhante a CGI, como FastCGI. Normalmente, servidores que fornecem tais protocolos incluirão uma variável `HTTPS` com um valor de "1", "yes" ou "on" quando uma solicitação é recebida via SSL. Então, esta função retorna "https" se tal valor for encontrado, e "http" caso contrário.

wsgiref.util.**request_uri**(*environ*, *include_query=True*)

Retorna o URI de solicitação completo, opcionalmente incluindo a string de consulta, usando o algoritmo encontrado na seção "URL Reconstruction" da **PEP 3333**. Se *include_query* for falso, a string de consulta não será incluída no URI resultante.

wsgiref.util.**application_uri**(*environ*)

Semelhante a *request_uri()*, exceto que as variáveis PATH_INFO e QUERY_STRING são ignoradas. O resultado é o URI base do objeto da aplicação endereçado pela solicitação.

wsgiref.util.**shift_path_info**(*environ*)

Desloca um único nome de PATH_INFO para SCRIPT_NAME e retorna o nome. O dicionário *environ* é *modificado* no local; use uma cópia se precisar manter o PATH_INFO ou SCRIPT_NAME original intacto.

Se não houver segmentos de caminho restantes em PATH_INFO, None será retornado.

Normalmente, essa rotina é usada para processar cada porção de um caminho de URI de solicitação, por exemplo, para tratar o caminho como uma série de chaves de dicionário. Essa rotina modifica o ambiente passado para torná-lo adequado para invocar outra aplicação WSGI localizada no URI de destino. Por exemplo, se houver uma aplicação WSGI em /foo, e o caminho de URI de solicitação for /foo/bar/baz, e a aplicação WSGI em /foo chamar *shift_path_info()*, ele receberá a string "bar", e o ambiente será atualizado para ser adequado para passar para uma aplicação WSGI em /foo/bar. Ou seja, SCRIPT_NAME mudará de /foo para /foo/bar, e PATH_INFO mudará de /bar/baz para /baz.

Quando PATH_INFO é apenas um "/", esta rotina retorna uma string vazia e acrescenta uma barra final a SCRIPT_NAME, embora segmentos de caminho vazios sejam normalmente ignorados, e SCRIPT_NAME normalmente não termine em uma barra. Este é um comportamento intencional, para garantir que uma aplicação possa diferenciar URIs terminando em /x daqueles terminando em /x/ ao usar esta rotina para fazer travessia de objetos.

wsgiref.util.**setup_testing_defaults**(*environ*)

Atualiza *environ* com padrões triviais para fins de teste.

Esta rotina adiciona vários parâmetros necessários para WSGI, incluindo HTTP_HOST, SERVER_NAME, SERVER_PORT, REQUEST_METHOD, SCRIPT_NAME, PATH_INFO e todas as variáveis wsgi.* definidas pela **PEP 3333**. Ela fornece apenas valores padrão e não substitui nenhuma configuração existente para essas variáveis.

Esta rotina tem como objetivo facilitar que testes unitários de servidores e aplicações WSGI configurem ambientes fictícios. Ela NÃO deve ser usada por servidores ou aplicações WSGI reais, pois os dados são falsos!

Exemplo de uso (veja também *demo_app()* para outro exemplo):

```
from wsgiref.util import setup_testing_defaults
from wsgiref.simple_server import make_server

# Uma aplicação WSGI relativamente simples. Vai exibir o dicionário
# do ambiente após ser atualizada por setup_testing_defaults
def simple_app(environ, start_response):
    setup_testing_defaults(environ)

    status = '200 OK'
    headers = [('Content-type', 'text/plain; charset=utf-8')]

    start_response(status, headers)

    ret = [("%s: %s\n" % (key, value)).encode("utf-8")
           for key, value in environ.items()]
    return ret

with make_server('', 8000, simple_app) as httpd:
    print("Serving on port 8000...")
    httpd.serve_forever()
```

Além das funções de ambiente acima, o módulo *wsgiref.util* também fornece estes utilitários diversos:

wsgiref.util.**is_hop_by_hop**(*header_name*)

Retorna `True` se 'header_name' for um cabeçalho HTTP/1.1 "Hop-by-Hop", conforme definido por **RFC 2616**.

class wsgiref.util.**FileWrapper**(*filelike*, *blksize=8192*)

Uma implementação concreta do protocolo *wsgiref.types.FileWrapper* usado para converter um objeto arquivo ou similar em um *iterador*. Os objetos resultantes são *iteráveis*. Conforme o objeto é iterado, o parâmetro opcional *blksize* será repetidamente passado para o método `read()` do objeto *filelike* para obter bytestrings para produzir. Quando `read()` retorna uma string de bytes vazia, a iteração é encerrada e não pode ser retomada.

Se *filelike* tiver um método `close()`, o objeto retornado também terá um método `close()` e invocará o método `close()` do objeto *filelike* quando chamado.

Exemplo de uso:

```
from io import StringIO
from wsgiref.util import FileWrapper

# Estamos usando um StringIO-buffer como objeto arquivo ou similar
filelike = StringIO("This is an example file-like object"*10)
wrapper = FileWrapper(filelike, blksize=5)

for chunk in wrapper:
    print(chunk)
```

Alterado na versão 3.11: O suporte para o método `__getitem__()` foi removido

22.2.2 `wsgiref.headers` – Ferramentas de cabeçalho de resposta WSGI

Este módulo fornece uma única classe, *Headers*, para manipulação conveniente de cabeçalhos de resposta WSGI usando uma interface semelhante a mapeamento.

class wsgiref.headers.**Headers**([*headers*])

Cria um objeto mapeamento envolvendo *headers*, que deve ser uma lista de tuplas de nome/valor de cabeçalho, conforme descrito na **PEP 3333**. O valor padrão de *headers* é uma lista vazia.

Os objetos *Headers* oferecem suporte a operações de mapeamento típicas, incluindo `__getitem__()`, *get()*, `__setitem__()`, *setdefault()*, `__delitem__()` e `__contains__()`. Para cada um desses métodos, a chave é o nome do cabeçalho (tratado sem distinção entre maiúsculas e minúsculas) e o valor é o primeiro valor associado a esse nome de cabeçalho. Definir um cabeçalho exclui quaisquer valores existentes para esse cabeçalho e, em seguida, adiciona um novo valor ao final da lista de cabeçalhos encapsulados. A ordem existente dos cabeçalhos geralmente é mantida, com novos cabeçalhos adicionados ao final da lista encapsulada.

Diferentemente de um dicionário, objetos *Headers* não levantam um erro quando você tenta obter ou excluir uma chave que não está na lista de cabeçalhos encapsulados. Obter um cabeçalho inexistente retorna apenas `None`, e excluir um cabeçalho inexistente não faz nada.

Os objetos *Headers* também oferecem suporte aos métodos `keys()`, `values()` e `items()`. As listas retornadas por `keys()` e `items()` podem incluir a mesma chave mais de uma vez se houver um cabeçalho multivalorado. O `len()` de um objeto *Headers* é o mesmo que o comprimento de seus `items()`, que é o mesmo que o comprimento da lista de cabeçalhos encapsulados. Na verdade, o método `items()` apenas retorna uma cópia da lista de cabeçalhos encapsulados.

Chamar `bytes()` em um objeto *Headers* retorna uma bytestring formatada adequada para transmissão como cabeçalhos de resposta HTTP. Cada cabeçalho é colocado em uma linha com seu valor, separado por dois pontos e um espaço. Cada linha é terminada por um retorno de carro e uma quebra de linha, e a bytestring é terminada com uma linha em branco.

Além de sua interface de mapeamento e recursos de formatação, os objetos *Headers* também têm os seguintes métodos para consultar e adicionar cabeçalhos multivalorados e para adicionar cabeçalhos com parâmetros

MIME:

get_all(*name*)

Retorna uma lista de todos os valores para o cabeçalho nomeado.

A lista retornada será classificada na ordem em que apareceu na lista de cabeçalho original ou foi adicionada a esta instância, e pode conter duplicatas. Quaisquer campos excluídos e reinseridos são sempre anexados à lista de cabeçalho. Se não houver campos com o nome fornecido, retorna uma lista vazia.

add_header(*name*, *value*, ***_params*)

Adiciona um cabeçalho (possivelmente multivalorado), com parâmetros MIME opcionais especificados por meio de argumentos nomeados.

name é o campo de cabeçalho a ser adicionado. Argumentos nomeados podem ser usados para definir parâmetros MIME para o campo de cabeçalho. Cada parâmetro deve ser uma string ou `None`. Sublinhados em nomes de parâmetros são convertidos em traços, já que traços são ilegais em identificadores Python, mas muitos nomes de parâmetros MIME incluem traços. Se o valor do parâmetro for uma string, ele será adicionado aos parâmetros de valor do cabeçalho no formato `name="value"`. Se for `None`, somente o nome do parâmetro será adicionado. (Isso é usado para parâmetros MIME sem um valor.) Exemplo de uso:

```
h.add_header('content-disposition', 'attachment', filename='bud.gif')
```

O exemplo acima adicionará um cabeçalho parecido com este:

```
Content-Disposition: attachment; filename="bud.gif"
```

Alterado na versão 3.5: o parâmetro *headers* é opcional.

22.2.3 `wsgiref.simple_server` – um servidor HTTP WSGI simples

Este módulo implementa um servidor HTTP simples (com base em *`http.server`*) que serve aplicações WSGI. Cada instância de servidor serve uma única aplicação WSGI em um host e porta fornecidos. Se você quiser servir várias aplicações em um único host e porta, você deve criar uma aplicação WSGI que analise `PATH_INFO` para selecionar qual aplicação invocar para cada solicitação. (Por exemplo, usando a função `shift_path_info()` de *`wsgiref.util`*.)

`wsgiref.simple_server.`**make_server**(*host*, *port*, *app*, *server_class=WSGIServer*, *handler_class=WSGIRequestHandler*)

Cria um novo servidor WSGI escutando em *host* e *port*, aceitando conexões para *app*. O valor de retorno é uma instância da *server_class* fornecida e processará solicitações usando a *handler_class* especificada. *app* deve ser um objeto de aplicação WSGI, conforme definido pela **PEP 3333**.

Exemplo de uso:

```
from wsgiref.simple_server import make_server, demo_app

with make_server('', 8000, demo_app) as httpd:
    print("Serving HTTP on port 8000...")

    # Responde a solicitações até o processo ser encerrado
    httpd.serve_forever()

    # Alternativa: serve uma solicitação e então sai
    httpd.handle_request()
```

`wsgiref.simple_server.`**demo_app**(*environ*, *start_response*)

Esta função é uma aplicação WSGI pequeno, mas completo, que retorna uma página de texto contendo a mensagem "Hello world!" e uma lista dos pares chave/valor fornecidos no parâmetro *environ*. É útil para verificar se um servidor WSGI (como *`wsgiref.simple_server`*) é capaz de executar uma aplicação WSGI simples corretamente.

O chamável *start_response* deve seguir o protocolo *StartResponse*.

class wsgiref.simple_server.**WSGIServer**(*server_address*, *RequestHandlerClass*)

Cria uma instância *WSGIServer*. *server_address* deve ser uma tupla (host,port) e *RequestHandlerClass* deve ser a subclasse de *http.server.BaseHTTPRequestHandler* que será usada para processar solicitações.

Normalmente você não precisa chamar esse construtor, pois a função *make_server()* pode cuidar de todos os detalhes para você.

WSGIServer é uma subclasse de *http.server.HTTPServer*, então todos os seus métodos (como serve_forever() e handle_request()) estão disponíveis. *WSGIServer* também fornece estes métodos específicos do WSGI:

set_app(*application*)

Define o chamável *application* como a aplicação WSGI que receberá solicitações.

get_app()

Retorna o chamável da aplicação definido atualmente.

Normalmente, no entanto, você não precisa usar esses métodos adicionais, pois *set_app()* é normalmente chamado por *make_server()*, e *get_app()* existe principalmente para o benefício de instâncias do manipulador de solicitações.

class wsgiref.simple_server.**WSGIRequestHandler**(*request*, *client_address*, *server*)

Cria um manipulador HTTP para *request* fornecido (isto é, um soquete), *client_address* (uma tupla (host, port)) e *server* (instância *WSGIServer*).

Você não precisa criar instâncias desta classe diretamente; elas são criadas automaticamente conforme necessário pelos objetos *WSGIServer*. Você pode, no entanto, estender esta classe e fornecê-la como uma *handler_class* para a função *make_server()*. Alguns métodos possivelmente relevantes para substituir em subclasses:

get_environ()

Retorna um dicionário *WSGIEnvironment* para uma solicitação. A implementação padrão copia o conteúdo do atributo de dicionário base_environ do objeto *WSGIServer* e então adiciona vários cabeçalhos derivados da solicitação HTTP. Cada chamada para esse método deve retornar um novo dicionário contendo todas as variáveis de ambiente CGI relevantes conforme especificado na **PEP 3333**.

get_stderr()

Retorna o objeto que deve ser usado como o fluxo wsgi.errors. A implementação padrão retorna apenas sys.stderr.

handle()

Processa a solicitação HTTP. A implementação padrão cria uma instância manipuladora usando uma classe *wsgiref.handlers* para implementar a interface da aplicação WSGI.

22.2.4 wsgiref.validate — Verificador de conformidade WSGI

Ao criar novos objetos de aplicação WSGI, frameworks, servidores ou middleware, pode ser útil validar a conformidade do novo código usando *wsgiref.validate*. Este módulo fornece uma função que cria objetos de aplicação WSGI que validam comunicações entre um servidor ou gateway WSGI e um objeto de aplicação WSGI, para verificar ambos os lados quanto à conformidade do protocolo.

Note que este utilitário não garante a conformidade completa com **PEP 3333**; uma ausência de erros deste módulo não significa necessariamente que os erros não existam. No entanto, se este módulo produzir um erro, então é virtualmente certo que o servidor ou a aplicação não está 100% em conformidade.

Este módulo é baseado no módulo paste.lint da biblioteca "Python Paste" de Ian Bicking.

wsgiref.validate.**validator**(*application*)

Encapsula *application* e retorna um novo objeto de aplicação WSGI. A aplicação retornada encaminhará todas

as solicitações para a *application* original e verificará se tanto a *application* quanto o servidor que o invoca estão em conformidade com a especificação WSGI e com **RFC 2616**.

Qualquer falta de conformidade detectada resulta em uma *AssertionError* sendo levantado; note, entretanto, que a forma como esses erros são manipulados depende do servidor. Por exemplo, *wsgiref.simple_server* e outros servidores baseados em *wsgiref.handlers* (que não substituem os métodos de manipulação de erros para fazer outra coisa) simplesmente emitirão uma mensagem de que ocorreu um erro e despejarão o traceback para `sys.stderr` ou algum outro fluxo de erro.

Este invólucro também pode gerar saída usando o módulo *warnings* para indicar comportamentos que são questionáveis, mas que podem não ser realmente proibidos por **PEP 3333**. A menos que sejam suprimidos usando opções de linha de comando do Python ou a API *warnings*, quaisquer avisos serão gravados em `sys.stderr` (*não* `wsgi.errors`, a menos que sejam o mesmo objeto).

Exemplo de uso:

```
from wsgiref.validate import validator
from wsgiref.simple_server import make_server

# Nosso objeto chamável que intencionalmente não está em
# conformidade com o padrão, então o validador irá quebrar
def simple_app(environ, start_response):
    status = '200 OK'  # HTTP Status
    headers = [('Content-type', 'text/plain')]  # HTTP Headers
    start_response(status, headers)

    # Isso vai quebrar porque precisamos retornar uma lista,
    # e o validador vai nos informar
    return b"Hello World"

# Este é a aplicação encapsulada em um validador
validator_app = validator(simple_app)

with make_server('', 8000, validator_app) as httpd:
    print("Listening on port 8000....")
    httpd.serve_forever()
```

22.2.5 `wsgiref.handlers` – classes base de servidor/gateway

Este módulo fornece classes de manipulador base para implementar servidores e gateways WSGI. Essas classes base lidam com a maior parte do trabalho de comunicação com uma aplicação WSGI, desde que recebam um ambiente semelhante ao CGI, junto com fluxos de entrada, saída e erro.

class wsgiref.handlers.**CGIHandler**

Invocação baseada em CGI via `sys.stdin`, `sys.stdout`, `sys.stderr` e `os.environ`. Isso é útil quando você tem uma aplicação WSGI e quer executá-lo como um script CGI. Basta invocar `CGIHandler().run(app)`, onde `app` é o objeto da aplicação WSGI que você deseja invocar.

Esta classe é uma subclasse de *BaseCGIHandler* que define `wsgi.run_once` como true, `wsgi.multithread` como false e `wsgi.multiprocess` como true, e sempre usa *sys* e *os* para obter os fluxos CGI e o ambiente necessários.

class wsgiref.handlers.**IISCGIHandler**

Uma alternativa especializada para *CGIHandler*, para uso ao implantar no servidor web IIS da Microsoft, sem ter definido a opção de configuração allowPathInfo (IIS>=7) ou metabase allowPathInfoForScriptMappings (IIS<7).

Por padrão, o IIS fornece um `PATH_INFO` que duplica o `SCRIPT_NAME` na frente, causando problemas para aplicações WSGI que desejam implementar roteamento. Este manipulador remove qualquer caminho duplicado.

O IIS pode ser configurado para passar o `PATH_INFO` correto, mas isso causa outro bug onde `PATH_TRANSLATED` está errado. Felizmente, essa variável raramente é usada e não é garantida pelo WSGI. No IIS<7, no entanto, a configuração só pode ser feita em um nível de vhost, afetando todos os outros mapeamentos de script, muitos dos quais quebram quando expostos ao bug `PATH_TRANSLATED`. Por esse motivo, o IIS<7 quase nunca é implantado com a correção (mesmo o IIS7 raramente a usa porque ainda não há uma UI para ela).

Não há como o código CGI dizer se a opção foi definida, então uma classe de manipulador separada é fornecida. Ela é usada da mesma forma que *CGIHandler*, ou seja, chamando `IISCGIHandler().run(app)`, onde `app` é o objeto de aplicação WSGI que você deseja invocar.

Adicionado na versão 3.2.

class wsgiref.handlers.**BaseCGIHandler**(*stdin*, *stdout*, *stderr*, *environ*, *multithread=True*, *multiprocess=False*)

Similar to *CGIHandler*, but instead of using the *sys* and *os* modules, the CGI environment and I/O streams are specified explicitly. The *multithread* and *multiprocess* values are used to set the `wsgi.multithread` and `wsgi.multiprocess` flags for any applications run by the handler instance.

This class is a subclass of *SimpleHandler* intended for use with software other than HTTP "origin servers". If you are writing a gateway protocol implementation (such as CGI, FastCGI, SCGI, etc.) that uses a `Status:` header to send an HTTP status, you probably want to subclass this instead of *SimpleHandler*.

class wsgiref.handlers.**SimpleHandler**(*stdin*, *stdout*, *stderr*, *environ*, *multithread=True*, *multiprocess=False*)

Similar to *BaseCGIHandler*, but designed for use with HTTP origin servers. If you are writing an HTTP server implementation, you will probably want to subclass this instead of *BaseCGIHandler*.

This class is a subclass of *BaseHandler*. It overrides the `__init__()`, *get_stdin()*, *get_stderr()*, *add_cgi_vars()*, *_write()*, and *_flush()* methods to support explicitly setting the environment and streams via the constructor. The supplied environment and streams are stored in the `stdin`, `stdout`, `stderr`, and `environ` attributes.

The *write()* method of *stdout* should write each chunk in full, like *io.BufferedIOBase*.

class wsgiref.handlers.**BaseHandler**

This is an abstract base class for running WSGI applications. Each instance will handle a single HTTP request, although in principle you could create a subclass that was reusable for multiple requests.

BaseHandler instances have only one method intended for external use:

run(*app*)

Run the specified WSGI application, *app*.

All of the other *BaseHandler* methods are invoked by this method in the process of running the application, and thus exist primarily to allow customizing the process.

The following methods MUST be overridden in a subclass:

_write(*data*)

Buffer the bytes *data* for transmission to the client. It's okay if this method actually transmits the data; *BaseHandler* just separates write and flush operations for greater efficiency when the underlying system actually has such a distinction.

_flush()

Force buffered data to be transmitted to the client. It's okay if this method is a no-op (i.e., if *_write()* actually sends the data).

get_stdin()

Return an object compatible with *InputStream* suitable for use as the `wsgi.input` of the request currently being processed.

get_stderr()

Return an object compatible with *ErrorStream* suitable for use as the `wsgi.errors` of the request currently being processed.

add_cgi_vars()

Insert CGI variables for the current request into the `environ` attribute.

Here are some other methods and attributes you may wish to override. This list is only a summary, however, and does not include every method that can be overridden. You should consult the docstrings and source code for additional information before attempting to create a customized *BaseHandler* subclass.

Attributes and methods for customizing the WSGI environment:

wsgi_multithread

The value to be used for the `wsgi.multithread` environment variable. It defaults to true in *BaseHandler*, but may have a different default (or be set by the constructor) in the other subclasses.

wsgi_multiprocess

The value to be used for the `wsgi.multiprocess` environment variable. It defaults to true in *BaseHandler*, but may have a different default (or be set by the constructor) in the other subclasses.

wsgi_run_once

The value to be used for the `wsgi.run_once` environment variable. It defaults to false in *BaseHandler*, but *CGIHandler* sets it to true by default.

os_environ

The default environment variables to be included in every request's WSGI environment. By default, this is a copy of `os.environ` at the time that *wsgiref.handlers* was imported, but subclasses can either create their own at the class or instance level. Note that the dictionary should be considered read-only, since the default value is shared between multiple classes and instances.

server_software

If the *origin_server* attribute is set, this attribute's value is used to set the default `SERVER_SOFTWARE` WSGI environment variable, and also to set a default `Server:` header in HTTP responses. It is ignored for handlers (such as *BaseCGIHandler* and *CGIHandler*) that are not HTTP origin servers.

Alterado na versão 3.3: O termo "Python" é substituído por termos específicos de implementação, como "CPython", "Jython" etc.

get_scheme()

Return the URL scheme being used for the current request. The default implementation uses the `guess_scheme()` function from *wsgiref.util* to guess whether the scheme should be "http" or "https", based on the current request's `environ` variables.

setup_environ()

Set the `environ` attribute to a fully populated WSGI environment. The default implementation uses all of the above methods and attributes, plus the *get_stdin()*, *get_stderr()*, and *add_cgi_vars()* methods and the *wsgi_file_wrapper* attribute. It also inserts a `SERVER_SOFTWARE` key if not present, as long as the *origin_server* attribute is a true value and the *server_software* attribute is set.

Methods and attributes for customizing exception handling:

log_exception(*exc_info*)

Log the *exc_info* tuple in the server log. *exc_info* is a `(type, value, traceback)` tuple. The default implementation simply writes the traceback to the request's `wsgi.errors` stream and flushes it. Subclasses can override this method to change the format or retarget the output, mail the traceback to an administrator, or whatever other action may be deemed suitable.

traceback_limit

The maximum number of frames to include in tracebacks output by the default *log_exception()* method. If `None`, all frames are included.

error_output(*environ*, *start_response*)

This method is a WSGI application to generate an error page for the user. It is only invoked if an error occurs before headers are sent to the client.

This method can access the current error using `sys.exception()`, and should pass that information to *start_response* when calling it (as described in the "Error Handling" section of **PEP 3333**). In particular, the *start_response* callable should follow the *StartResponse* protocol.

The default implementation just uses the *error_status*, *error_headers*, and *error_body* attributes to generate an output page. Subclasses can override this to produce more dynamic error output.

Note, however, that it's not recommended from a security perspective to spit out diagnostics to any old user; ideally, you should have to do something special to enable diagnostic output, which is why the default implementation doesn't include any.

error_status

The HTTP status used for error responses. This should be a status string as defined in **PEP 3333**; it defaults to a 500 code and message.

error_headers

The HTTP headers used for error responses. This should be a list of WSGI response headers (`(name, value)` tuples), as described in **PEP 3333**. The default list just sets the content type to `text/plain`.

error_body

The error response body. This should be an HTTP response body bytestring. It defaults to the plain text, "A server error occurred. Please contact the administrator."

Methods and attributes for **PEP 3333**'s "Optional Platform-Specific File Handling" feature:

wsgi_file_wrapper

A `wsgi.file_wrapper` factory, compatible with *wsgiref.types.FileWrapper*, or `None`. The default value of this attribute is the *wsgiref.util.FileWrapper* class.

sendfile()

Override to implement platform-specific file transmission. This method is called only if the application's return value is an instance of the class specified by the *wsgi_file_wrapper* attribute. It should return a true value if it was able to successfully transmit the file, so that the default transmission code will not be executed. The default implementation of this method just returns a false value.

Miscellaneous methods and attributes:

origin_server

This attribute should be set to a true value if the handler's *_write()* and *_flush()* are being used to communicate directly to the client, rather than via a CGI-like gateway protocol that wants the HTTP status in a special `Status:` header.

This attribute's default value is true in *BaseHandler*, but false in *BaseCGIHandler* and *CGIHandler*.

http_version

If *origin_server* is true, this string attribute is used to set the HTTP version of the response set to the client. It defaults to `"1.0"`.

wsgiref.handlers.**read_environ**()

Transcode CGI variables from `os.environ` to **PEP 3333** "bytes in unicode" strings, returning a new dictionary. This function is used by *CGIHandler* and *IISCGIHandler* in place of directly using `os.environ`, which is not necessarily WSGI-compliant on all platforms and web servers using Python 3 – specifically, ones where the OS's actual environment is Unicode (i.e. Windows), or ones where the environment is bytes, but the system encoding used by Python to decode it is anything other than ISO-8859-1 (e.g. Unix systems using UTF-8).

If you are implementing a CGI-based handler of your own, you probably want to use this routine instead of just copying values out of `os.environ` directly.

Adicionado na versão 3.2.

22.2.6 `wsgiref.types` – WSGI types for static type checking

This module provides various types for static type checking as described in **PEP 3333**.

Adicionado na versão 3.11.

class wsgiref.types.**StartResponse**

A *typing.Protocol* describing **start_response()** callables (**PEP 3333**).

wsgiref.types.**WSGIEnvironment**

A type alias describing a WSGI environment dictionary.

wsgiref.types.**WSGIApplication**

A type alias describing a WSGI application callable.

class wsgiref.types.**InputStream**

A *typing.Protocol* describing a **WSGI Input Stream**.

class wsgiref.types.**ErrorStream**

A *typing.Protocol* describing a **WSGI Error Stream**.

class wsgiref.types.**FileWrapper**

A *typing.Protocol* describing a **file wrapper**. See *wsgiref.util.FileWrapper* for a concrete implementation of this protocol.

22.2.7 Exemplos

This is a working "Hello World" WSGI application, where the *start_response* callable should follow the *StartResponse* protocol:

```
"""
Every WSGI application must have an application object - a callable
object that accepts two arguments. For that purpose, we're going to
use a function (note that you're not limited to a function, you can
use a class for example). The first argument passed to the function
is a dictionary containing CGI-style environment variables and the
second variable is the callable object.
"""
from wsgiref.simple_server import make_server


def hello_world_app(environ, start_response):
    status = "200 OK"  # HTTP Status
    headers = [("Content-type", "text/plain; charset=utf-8")]  # HTTP Headers
    start_response(status, headers)

    # The returned object is going to be printed
    return [b"Hello World"]

with make_server("", 8000, hello_world_app) as httpd:
    print("Serving on port 8000...")

    # Serve until process is killed
    httpd.serve_forever()
```

Example of a WSGI application serving the current directory, accept optional directory and port number (default: 8000) on the command line:

```
"""
Small wsgiref based web server. Takes a path to serve from and an
```

(continua na próxima página)

(continuação da página anterior)

```
optional port number (defaults to 8000), then tries to serve files.
MIME types are guessed from the file names, 404 errors are raised
if the file is not found.
"""
import mimetypes
import os
import sys
from wsgiref import simple_server, util


def app(environ, respond):
    # Get the file name and MIME type
    fn = os.path.join(path, environ["PATH_INFO"][1:])
    if "." not in fn.split(os.path.sep)[-1]:
        fn = os.path.join(fn, "index.html")
    mime_type = mimetypes.guess_file_type(fn)[0]

    # Return 200 OK if file exists, otherwise 404 Not Found
    if os.path.exists(fn):
        respond("200 OK", [("Content-Type", mime_type)])
        return util.FileWrapper(open(fn, "rb"))
    else:
        respond("404 Not Found", [("Content-Type", "text/plain")])
        return [b"not found"]


if __name__ == "__main__":
    # Get the path and port from command-line arguments
    path = sys.argv[1] if len(sys.argv) > 1 else os.getcwd()
    port = int(sys.argv[2]) if len(sys.argv) > 2 else 8000

    # Make and start the server until control-c
    httpd = simple_server.make_server("", port, app)
    print(f"Serving {path} on port {port}, control-C to stop")
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("Shutting down.")
        httpd.server_close()
```

22.3 `urllib` — Módulos de manipulação de URL

Código-fonte: Lib/urllib/

`urllib` é um pacote que coleciona vários módulos para trabalhar com URLs:

- *`urllib.request`* para abrir e ler URLs
- *`urllib.error`* contendo as exceções levantadas por *`urllib.request`*
- *`urllib.parse`* para analisar URLs
- *`urllib.robotparser`* para analisar arquivos `robots.txt`

22.4 `urllib.request` — Biblioteca extensível para abrir URLs

Código-fonte: Lib/urllib/request.py

O módulo `urllib.request` define funções e classes que ajudam a abrir URLs (principalmente HTTP) em um mundo complexo — autenticação básica ou por digest, redirecionamentos, cookies e muito mais.

Ver também

O pacote Requests é recomendado para uma interface alto nível de cliente HTTP.

Aviso

No macOS, não é seguro usar este módulo em programas que utilizam `os.fork()` porque a implementação `getproxies()` para macOS utiliza uma API de sistema de nível mais alto. Defina a variável de ambiente `no_proxy` como `*` para evitar esse problema (por exemplo, `os.environ["no_proxy"] = "*"`).

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

O módulo `urllib.request` define as seguintes funções:

`urllib.request.`**`urlopen`**(*url*, *data=None*, [*timeout*,]*, *context=None*)

Abre *url*, que pode ser uma string contendo uma URL válida e devidamente codificada, ou um objeto `Request`.

data deve ser um objeto que especifique dados adicionais a serem enviados ao servidor ou `None`, se nenhum dado for necessário. Veja `Request` para detalhes.

O módulo urllib.request usa HTTP/1.1 e inclui o cabeçalho `Connection:close` em suas solicitações HTTP.

O parâmetro opcional *timeout* especifica um tempo limite em segundos para bloquear operações como a tentativa de conexão (se não for especificado, a configuração de tempo limite padrão global será usada). Na verdade, isso só funciona para conexões HTTP, HTTPS e FTP.

Se *context* for especificado, deve ser uma instância de `ssl.SSLContext` descrevendo as várias opções SSL. Veja `HTTPSConnection` para mais detalhes.

Esta função sempre retorna um objeto que pode funcionar como um *gerenciador de contexto* e tem as propriedades *url*, *headers* e *status*. Veja `urllib.response.addinfourl` para mais detalhes sobre essas propriedades.

Para URLs HTTP e HTTPS, esta função retorna um objeto `http.client.HTTPResponse` ligeiramente modificado. Além dos três novos métodos acima, o atributo msg contém as mesmas informações que o atributo `reason` — a frase de razão retornada pelo servidor — em vez dos cabeçalhos de resposta como é especificado na documentação para `HTTPResponse`.

Para URLs de FTP, arquivo e dados, esta função retorna um objeto `urllib.response.addinfourl`.

Levanta `URLError` quando ocorrer erros de protocolo.

Observe que `None` pode ser retornado se nenhum manipulador lidar com a solicitação (embora a `OpenerDirector` global instalada padrão use `UnknownHandler` para garantir que isso nunca aconteça).

Além disso, se as configurações de proxy forem detectadas (por exemplo, quando uma variável de ambiente `*_proxy` como `http_proxy` é definida), `ProxyHandler` é instalado por padrão e garante que as solicitações sejam manipuladas pelo proxy.

A função legada `urllib.urlopen` do Python 2.6 e anteriores foi descontinuada; *urllib.request.urlopen()* corresponde à antiga `urllib2.urlopen`. O tratamento de proxy, que foi feito passando um parâmetro de dicionário para `urllib.urlopen`, pode ser obtido usando objetos *ProxyHandler*.

O abridor padrão levanta um *evento de auditoria* `urllib.Request` com os argumentos `fullurl`, `data`, `headers`, `method` obtidos do objeto de requisição.

Alterado na versão 3.2: *cafile* e *capath* foram adicionados.

Hosts virtuais HTTPS agora são suportados, se possível (ou seja, se *ssl.HAS_SNI* for verdadeiro).

data pode ser um objeto iterável.

Alterado na versão 3.3: *cadefault* foi adicionado.

Alterado na versão 3.4.3: *context* foi adicionado.

Alterado na versão 3.10: A conexão HTTPS agora envia uma extensão ALPN com o indicador de protocolo `http/1.1` quando nenhum *context* é fornecido. *context* personalizado deve definir protocolos ALPN com *set_alpn_protocols()*.

Alterado na versão 3.13: Remove os parâmetros *cafile*, *capath* e *cadefault*: use o parâmetro *context* em seu lugar.

`urllib.request.`**`install_opener`**(*opener*)

Instala uma instância de *OpenerDirector* como o abridor global padrão. A instalação de um abridor só é necessária se você quiser que o urlopen o utilize; caso contrário, basta chamar *OpenerDirector.open()* em vez de *urlopen()*. O código não verifica se há um *OpenerDirector* real, e qualquer classe com a interface apropriada funcionará.

`urllib.request.`**`build_opener`**([*handler*, ...])

Return an *OpenerDirector* instance, which chains the handlers in the order given. *handler*s can be either instances of *BaseHandler*, or subclasses of *BaseHandler* (in which case it must be possible to call the constructor without any parameters). Instances of the following classes will be in front of the *handler*s, unless the *handler*s contain them, instances of them or subclasses of them: *ProxyHandler* (if proxy settings are detected), *UnknownHandler*, *HTTPHandler*, *HTTPDefaultErrorHandler*, *HTTPRedirectHandler*, *FTPHandler*, *FileHandler*, *HTTPErrorProcessor*.

If the Python installation has SSL support (i.e., if the *ssl* module can be imported), *HTTPSHandler* will also be added.

A *BaseHandler* subclass may also change its `handler_order` attribute to modify its position in the handlers list.

`urllib.request.`**`pathname2url`**(*path*, *, *add_scheme=False*)

Convert the given local path to a `file:` URL. This function uses *quote()* function to encode the path.

If *add_scheme* is false (the default), the return value omits the `file:` scheme prefix. Set *add_scheme* to true to return a complete URL.

This example shows the function being used on Windows:

```
>>> from urllib.request import pathname2url
>>> path = 'C:\\Program Files'
>>> pathname2url(path, add_scheme=True)
'file:///C:/Program%20Files'
```

Alterado na versão 3.14: Windows drive letters are no longer converted to uppercase, and `:` characters not following a drive letter no longer cause an *OSError* exception to be raised on Windows.

Alterado na versão 3.14: Paths beginning with a slash are converted to URLs with authority sections. For example, the path `/etc/hosts` is converted to the URL `///etc/hosts`.

Alterado na versão 3.14: The *add_scheme* parameter was added.

urllib.request.**url2pathname**(*url*, *, *require_scheme=False*, *resolve_host=False*)

Convert the given `file:` URL to a local path. This function uses *unquote()* to decode the URL.

If *require_scheme* is false (the default), the given value should omit a `file:` scheme prefix. If *require_scheme* is set to true, the given value should include the prefix; a *URLError* is raised if it doesn't.

The URL authority is discarded if it is empty, `localhost`, or the local hostname. Otherwise, if *resolve_host* is set to true, the authority is resolved using *socket.gethostbyname()* and discarded if it matches a local IP address (as per **RFC 8089 §3**). If the authority is still unhandled, then on Windows a UNC path is returned, and on other platforms a *URLError* is raised.

This example shows the function being used on Windows:

```
>>> from urllib.request import url2pathname
>>> url = 'file:///C:/Program%20Files'
>>> url2pathname(url, require_scheme=True)
'C:\\Program Files'
```

Alterado na versão 3.14: Windows drive letters are no longer converted to uppercase, and `:` characters not following a drive letter no longer cause an *OSError* exception to be raised on Windows.

Alterado na versão 3.14: The URL authority is discarded if it matches the local hostname. Otherwise, if the authority isn't empty or `localhost`, then on Windows a UNC path is returned (as before), and on other platforms a *URLError* is raised.

Alterado na versão 3.14: The URL query and fragment components are discarded if present.

Alterado na versão 3.14: The *require_scheme* and *resolve_host* parameters were added.

urllib.request.**getproxies**()

This helper function returns a dictionary of scheme to proxy server URL mappings. It scans the environment for variables named `<scheme>_proxy`, in a case insensitive approach, for all operating systems first, and when it cannot find it, looks for proxy information from System Configuration for macOS and Windows Systems Registry for Windows. If both lowercase and uppercase environment variables exist (and disagree), lowercase is preferred.

> **Nota**
>
> If the environment variable `REQUEST_METHOD` is set, which usually indicates your script is running in a CGI environment, the environment variable `HTTP_PROXY` (uppercase `_PROXY`) will be ignored. This is because that variable can be injected by a client using the "Proxy:" HTTP header. If you need to use an HTTP proxy in a CGI environment, either use `ProxyHandler` explicitly, or make sure the variable name is in lowercase (or at least the `_proxy` suffix).

As seguintes classes são disponibilizadas:

class urllib.request.**Request**(*url*, *data=None*, *headers={}*, *origin_req_host=None*, *unverifiable=False*, *method=None*)

This class is an abstraction of a URL request.

url should be a string containing a valid, properly encoded URL.

data must be an object specifying additional data to send to the server, or `None` if no such data is needed. Currently HTTP requests are the only ones that use *data*. The supported object types include bytes, file-like objects, and iterables of bytes-like objects. If no `Content-Length` nor `Transfer-Encoding` header field has been provided, *HTTPHandler* will set these headers according to the type of *data*. `Content-Length` will be used to send bytes objects, while `Transfer-Encoding: chunked` as specified in **RFC 7230**, Section 3.3.1 will be used to send files and other iterables.

For an HTTP POST request method, *data* should be a buffer in the standard *application/x-www-form-urlencoded* format. The *urllib.parse.urlencode()* function takes a mapping or se-

quence of 2-tuples and returns an ASCII string in this format. It should be encoded to bytes before being used as the *data* parameter.

headers should be a dictionary, and will be treated as if *add_header()* was called with each key and value as arguments. This is often used to "spoof" the `User-Agent` header value, which is used by a browser to identify itself – some HTTP servers only allow requests coming from common browsers as opposed to scripts. For example, Mozilla Firefox may identify itself as `"Mozilla/5.0 (X11; U; Linux i686) Gecko/20071127 Firefox/2.0.0.11"`, while *urllib*'s default user agent string is `"Python-urllib/2.6"` (on Python 2.6). All header keys are sent in camel case.

An appropriate `Content-Type` header should be included if the *data* argument is present. If this header has not been provided and *data* is not `None`, `Content-Type: application/x-www-form-urlencoded` will be added as a default.

The next two arguments are only of interest for correct handling of third-party HTTP cookies:

origin_req_host should be the request-host of the origin transaction, as defined by **RFC 2965**. It defaults to `http.cookiejar.request_host(self)`. This is the host name or IP address of the original request that was initiated by the user. For example, if the request is for an image in an HTML document, this should be the request-host of the request for the page containing the image.

unverifiable should indicate whether the request is unverifiable, as defined by **RFC 2965**. It defaults to `False`. An unverifiable request is one whose URL the user did not have the option to approve. For example, if the request is for an image in an HTML document, and the user had no option to approve the automatic fetching of the image, this should be true.

method should be a string that indicates the HTTP request method that will be used (e.g. `'HEAD'`). If provided, its value is stored in the *method* attribute and is used by *get_method()*. The default is `'GET'` if *data* is `None` or `'POST'` otherwise. Subclasses may indicate a different default method by setting the *method* attribute in the class itself.

> **Nota**
>
> The request will not work as expected if the data object is unable to deliver its content more than once (e.g. a file or an iterable that can produce the content only once) and the request is retried for HTTP redirects or authentication. The *data* is sent to the HTTP server right away after the headers. There is no support for a 100-continue expectation in the library.

Alterado na versão 3.3: *Request.method* argument is added to the Request class.

Alterado na versão 3.4: Default *Request.method* may be indicated at the class level.

Alterado na versão 3.6: Do not raise an error if the `Content-Length` has not been provided and *data* is neither `None` nor a bytes object. Fall back to use chunked transfer encoding instead.

class `urllib.request.`**OpenerDirector**

The *OpenerDirector* class opens URLs via *BaseHandler*s chained together. It manages the chaining of handlers, and recovery from errors.

class `urllib.request.`**BaseHandler**

This is the base class for all registered handlers — and handles only the simple mechanics of registration.

class `urllib.request.`**HTTPDefaultErrorHandler**

A class which defines a default handler for HTTP error responses; all responses are turned into *HTTPError* exceptions.

class `urllib.request.`**HTTPRedirectHandler**

Uma classe para lidar com redirecionamentos.

class `urllib.request.`**HTTPCookieProcessor**(*cookiejar=None*)

Classe para lidar com Cookies HTTP.

class urllib.request.**ProxyHandler**(*proxies=None*)

Cause requests to go through a proxy. If *proxies* is given, it must be a dictionary mapping protocol names to URLs of proxies. The default is to read the list of proxies from the environment variables `<protocol>_proxy`. If no proxy environment variables are set, then in a Windows environment proxy settings are obtained from the registry's Internet Settings section, and in a macOS environment proxy information is retrieved from the System Configuration Framework.

To disable autodetected proxy pass an empty dictionary.

The `no_proxy` environment variable can be used to specify hosts which shouldn't be reached via proxy; if set, it should be a comma-separated list of hostname suffixes, optionally with `:port` appended, for example `cern.ch,ncsa.uiuc.edu,some.host:8080`.

> **Nota**
>
> `HTTP_PROXY` será ignorado se uma variável `REQUEST_METHOD` estiver definida; veja a documentação em *getproxies()*.

class urllib.request.**HTTPPasswordMgr**

Keep a database of `(realm, uri) -> (user, password)` mappings.

class urllib.request.**HTTPPasswordMgrWithDefaultRealm**

Keep a database of `(realm, uri) -> (user, password)` mappings. A realm of `None` is considered a catch-all realm, which is searched if no other realm fits.

class urllib.request.**HTTPPasswordMgrWithPriorAuth**

A variant of *HTTPPasswordMgrWithDefaultRealm* that also has a database of `uri -> is_authenticated` mappings. Can be used by a BasicAuth handler to determine when to send authentication credentials immediately instead of waiting for a `401` response first.

Adicionado na versão 3.5.

class urllib.request.**AbstractBasicAuthHandler**(*password_mgr=None*)

This is a mixin class that helps with HTTP authentication, both to the remote host and to a proxy. *password_mgr*, if given, should be something that is compatible with *HTTPPasswordMgr*; refer to section *HTTPPasswordMgr Objects* for information on the interface that must be supported. If *passwd_mgr* also provides `is_authenticated` and `update_authenticated` methods (see *HTTPPasswordMgrWithPriorAuth Objects*), then the handler will use the `is_authenticated` result for a given URI to determine whether or not to send authentication credentials with the request. If `is_authenticated` returns `True` for the URI, credentials are sent. If `is_authenticated` is `False`, credentials are not sent, and then if a `401` response is received the request is re-sent with the authentication credentials. If authentication succeeds, `update_authenticated` is called to set `is_authenticated` `True` for the URI, so that subsequent requests to the URI or any of its super-URIs will automatically include the authentication credentials.

Adicionado na versão 3.5: Added `is_authenticated` support.

class urllib.request.**HTTPBasicAuthHandler**(*password_mgr=None*)

Handle authentication with the remote host. *password_mgr*, if given, should be something that is compatible with *HTTPPasswordMgr*; refer to section *HTTPPasswordMgr Objects* for information on the interface that must be supported. HTTPBasicAuthHandler will raise a *ValueError* when presented with a wrong Authentication scheme.

class urllib.request.**ProxyBasicAuthHandler**(*password_mgr=None*)

Handle authentication with the proxy. *password_mgr*, if given, should be something that is compatible with *HTTPPasswordMgr*; refer to section *HTTPPasswordMgr Objects* for information on the interface that must be supported.

class urllib.request.**AbstractDigestAuthHandler**(*password_mgr=None*)

This is a mixin class that helps with HTTP authentication, both to the remote host and to a proxy. *password_mgr*, if given, should be something that is compatible with *HTTPPasswordMgr*; refer to section *HTTPPasswordMgr Objects* for information on the interface that must be supported.

Alterado na versão 3.14: Added support for HTTP digest authentication algorithm `SHA-256`.

class urllib.request.**HTTPDigestAuthHandler**(*password_mgr=None*)

Handle authentication with the remote host. *password_mgr*, if given, should be something that is compatible with *HTTPPasswordMgr*; refer to section *HTTPPasswordMgr Objects* for information on the interface that must be supported. When both Digest Authentication Handler and Basic Authentication Handler are both added, Digest Authentication is always tried first. If the Digest Authentication returns a 40x response again, it is sent to Basic Authentication handler to Handle. This Handler method will raise a *ValueError* when presented with an authentication scheme other than Digest or Basic.

Alterado na versão 3.3: Raise *ValueError* on unsupported Authentication Scheme.

class urllib.request.**ProxyDigestAuthHandler**(*password_mgr=None*)

Handle authentication with the proxy. *password_mgr*, if given, should be something that is compatible with *HTTPPasswordMgr*; refer to section *HTTPPasswordMgr Objects* for information on the interface that must be supported.

class urllib.request.**HTTPHandler**

A class to handle opening of HTTP URLs.

class urllib.request.**HTTPSHandler**(*debuglevel=0*, *context=None*, *check_hostname=None*)

A class to handle opening of HTTPS URLs. *context* and *check_hostname* have the same meaning as in *http.client.HTTPSConnection*.

Alterado na versão 3.2: *context* and *check_hostname* were added.

class urllib.request.**FileHandler**

Abre arquivos locais.

class urllib.request.**DataHandler**

Abre dados das URLs.

Adicionado na versão 3.4.

class urllib.request.**FTPHandler**

Abre URLs de FTP.

class urllib.request.**CacheFTPHandler**

Open FTP URLs, keeping a cache of open FTP connections to minimize delays.

class urllib.request.**UnknownHandler**

A catch-all class to handle unknown URLs.

class urllib.request.**HTTPErrorProcessor**

Process HTTP error responses.

22.4.1 Objeto Request

The following methods describe *Request*'s public interface, and so all may be overridden in subclasses. It also defines several public attributes that can be used by clients to inspect the parsed request.

Request.**full_url**

The original URL passed to the constructor.

Alterado na versão 3.4.

Request.full_url is a property with setter, getter and a deleter. Getting *full_url* returns the original request URL with the fragment, if it was present.

Request.**type**

The URI scheme.

Request.**host**

The URI authority, typically a host, but may also contain a port separated by a colon.

Request.**origin_req_host**

The original host for the request, without port.

Request.**selector**

The URI path. If the *Request* uses a proxy, then selector will be the full URL that is passed to the proxy.

Request.**data**

The entity body for the request, or None if not specified.

Alterado na versão 3.4: Changing value of *Request.data* now deletes "Content-Length" header if it was previously set or calculated.

Request.**unverifiable**

boolean, indicates whether the request is unverifiable as defined by **RFC 2965**.

Request.**method**

The HTTP request method to use. By default its value is *None*, which means that *get_method()* will do its normal computation of the method to be used. Its value can be set (thus overriding the default computation in *get_method()*) either by providing a default value by setting it at the class level in a *Request* subclass, or by passing a value in to the *Request* constructor via the *method* argument.

Adicionado na versão 3.3.

Alterado na versão 3.4: A default value can now be set in subclasses; previously it could only be set via the constructor argument.

Request.**get_method**()

Return a string indicating the HTTP request method. If *Request.method* is not None, return its value, otherwise return 'GET' if *Request.data* is None, or 'POST' if it's not. This is only meaningful for HTTP requests.

Alterado na versão 3.3: get_method now looks at the value of *Request.method*.

Request.**add_header**(*key*, *val*)

Add another header to the request. Headers are currently ignored by all handlers except HTTP handlers, where they are added to the list of headers sent to the server. Note that there cannot be more than one header with the same name, and later calls will overwrite previous calls in case the *key* collides. Currently, this is no loss of HTTP functionality, since all headers which have meaning when used more than once have a (header-specific) way of gaining the same functionality using only one header. Note that headers added using this method are also added to redirected requests.

Request.**add_unredirected_header**(*key*, *header*)

Add a header that will not be added to a redirected request.

Request.**has_header**(*header*)

Return whether the instance has the named header (checks both regular and unredirected).

Request.**remove_header**(*header*)

Remove o cabeçalho nomeado da instância de solicitação (tanto de cabeçalhos regulares como de cabeçalhos não-redirecionados).

Adicionado na versão 3.4.

Request.**get_full_url**()

Return the URL given in the constructor.

Alterado na versão 3.4.

Returns *Request.full_url*

Request.**set_proxy**(*host*, *type*)

Prepare the request by connecting to a proxy server. The *host* and *type* will replace those of the instance, and the instance's selector will be the original URL given in the constructor.

Request.**get_header**(*header_name*, *default=None*)

Return the value of the given header. If the header is not present, return the default value.

Request.**header_items**()

Return a list of tuples (header_name, header_value) of the Request headers.

Alterado na versão 3.4: The request methods add_data, has_data, get_data, get_type, get_host, get_selector, get_origin_req_host and is_unverifiable that were deprecated since 3.3 have been removed.

22.4.2 OpenerDirector Objects

OpenerDirector instances have the following methods:

OpenerDirector.**add_handler**(*handler*)

handler should be an instance of *BaseHandler*. The following methods are searched, and added to the possible chains (note that HTTP errors are a special case). Note that, in the following, *protocol* should be replaced with the actual protocol to handle, for example `http_response()` would be the HTTP protocol response handler. Also *type* should be replaced with the actual HTTP code, for example `http_error_404()` would handle HTTP 404 errors.

- `<protocol>_open()` — signal that the handler knows how to open *protocol* URLs.

  Veja *BaseHandler.<protocol>_open()* para maiores informações.

- `http_error_<type>()` — signal that the handler knows how to handle HTTP errors with HTTP error code *type*.

  Veja *BaseHandler.http_error_<nnn>()* para maiores informações.

- `<protocol>_error()` — signal that the handler knows how to handle errors from (non-`http`) *protocol*.

- `<protocol>_request()` — signal that the handler knows how to pre-process *protocol* requests.

  Veja *BaseHandler.<protocol>_request()* para maiores informações.

- `<protocol>_response()` — signal that the handler knows how to post-process *protocol* responses.

  Veja *BaseHandler.<protocol>_response()* para maiores informações.

OpenerDirector.**open**(*url*, *data=None*[, *timeout*])

Open the given *url* (which can be a request object or a string), optionally passing the given *data*. Arguments, return values and exceptions raised are the same as those of *urlopen()* (which simply calls the *open()* method on the currently installed global *OpenerDirector*). The optional *timeout* parameter specifies a timeout in seconds for blocking operations like the connection attempt (if not specified, the global default timeout setting will be used). The timeout feature actually works only for HTTP, HTTPS and FTP connections.

OpenerDirector.**error**(*proto*, **args*)

Handle an error of the given protocol. This will call the registered error handlers for the given protocol with the given arguments (which are protocol specific). The HTTP protocol is a special case which uses the HTTP response code to determine the specific error handler; refer to the `http_error_<type>()` methods of the handler classes.

Return values and exceptions raised are the same as those of *urlopen()*.

OpenerDirector objects open URLs in three stages:

The order in which these methods are called within each stage is determined by sorting the handler instances.

1. Every handler with a method named like `<protocol>_request()` has that method called to pre-process the request.

2. Handlers with a method named like `<protocol>_open()` are called to handle the request. This stage ends when a handler either returns a non-*None* value (ie. a response), or raises an exception (usually *URLError*). Exceptions are allowed to propagate.

In fact, the above algorithm is first tried for methods named *default_open()*. If all such methods return *None*, the algorithm is repeated for methods named like `<protocol>_open()`. If all such methods return *None*, the algorithm is repeated for methods named *unknown_open()*.

Note that the implementation of these methods may involve calls of the parent *OpenerDirector* instance's *open()* and *error()* methods.

3. Every handler with a method named like `<protocol>_response()` has that method called to post-process the response.

22.4.3 BaseHandler Objects

BaseHandler objects provide a couple of methods that are directly useful, and others that are meant to be used by derived classes. These are intended for direct use:

`BaseHandler.`**`add_parent`**`(`*director*`)`

Add a director as parent.

`BaseHandler.`**`close`**`()`

Remove any parents.

The following attribute and methods should only be used by classes derived from *BaseHandler*.

Nota

The convention has been adopted that subclasses defining `<protocol>_request()` or `<protocol>_response()` methods are named `*Processor`; all others are named `*Handler`.

`BaseHandler.`**`parent`**

A valid *OpenerDirector*, which can be used to open using a different protocol, or handle errors.

`BaseHandler.`**`default_open`**`(`*req*`)`

This method is *not* defined in *BaseHandler*, but subclasses should define it if they want to catch all URLs.

This method, if implemented, will be called by the parent *OpenerDirector*. It should return a file-like object as described in the return value of the *open()* method of *OpenerDirector*, or `None`. It should raise *URLError*, unless a truly exceptional thing happens (for example, *MemoryError* should not be mapped to *URLError*).

This method will be called before any protocol-specific open method.

`BaseHandler.<protocol>_open(req)`

This method is *not* defined in *BaseHandler*, but subclasses should define it if they want to handle URLs with the given protocol.

This method, if defined, will be called by the parent *OpenerDirector*. Return values should be the same as for *default_open()*.

`BaseHandler.`**`unknown_open`**`(`*req*`)`

This method is *not* defined in *BaseHandler*, but subclasses should define it if they want to catch all URLs with no specific registered handler to open it.

This method, if implemented, will be called by the *parent* *OpenerDirector*. Return values should be the same as for *default_open()*.

`BaseHandler.`**`http_error_default`**`(`*req*, *fp*, *code*, *msg*, *hdrs*`)`

This method is *not* defined in *BaseHandler*, but subclasses should override it if they intend to provide a catch-all for otherwise unhandled HTTP errors. It will be called automatically by the *OpenerDirector* getting the error, and should not normally be called in other circumstances.

OpenerDirector will call this method with five positional arguments:

1. a *Request* object,

2. a file-like object with the HTTP error body,
3. the three-digit code of the error, as a string,
4. the user-visible explanation of the code, as a string, and
5. the headers of the error, as a mapping object.

Return values and exceptions raised should be the same as those of *urlopen()*.

BaseHandler.http_error_<nnn>(req, fp, code, msg, hdrs)

nnn should be a three-digit HTTP error code. This method is also not defined in *BaseHandler*, but will be called, if it exists, on an instance of a subclass, when an HTTP error with code *nnn* occurs.

Subclasses should override this method to handle specific HTTP errors.

Arguments, return values and exceptions raised should be the same as for *http_error_default()*.

BaseHandler.<protocol>_request(req)

This method is *not* defined in *BaseHandler*, but subclasses should define it if they want to pre-process requests of the given protocol.

This method, if defined, will be called by the parent *OpenerDirector*. *req* will be a *Request* object. The return value should be a *Request* object.

BaseHandler.<protocol>_response(req, response)

This method is *not* defined in *BaseHandler*, but subclasses should define it if they want to post-process responses of the given protocol.

This method, if defined, will be called by the parent *OpenerDirector*. *req* will be a *Request* object. *response* will be an object implementing the same interface as the return value of *urlopen()*. The return value should implement the same interface as the return value of *urlopen()*.

22.4.4 HTTPRedirectHandler Objects

Nota

Some HTTP redirections require action from this module's client code. If this is the case, *HTTPError* is raised. See **RFC 2616** for details of the precise meanings of the various redirection codes.

An *HTTPError* exception raised as a security consideration if the HTTPRedirectHandler is presented with a redirected URL which is not an HTTP, HTTPS or FTP URL.

HTTPRedirectHandler.**redirect_request**(*req*, *fp*, *code*, *msg*, *hdrs*, *newurl*)

Return a *Request* or `None` in response to a redirect. This is called by the default implementations of the `http_error_30*()` methods when a redirection is received from the server. If a redirection should take place, return a new *Request* to allow `http_error_30*()` to perform the redirect to *newurl*. Otherwise, raise *HTTPError* if no other handler should try to handle this URL, or return `None` if you can't but another handler might.

Nota

The default implementation of this method does not strictly follow **RFC 2616**, which says that 301 and 302 responses to `POST` requests must not be automatically redirected without confirmation by the user. In reality, browsers do allow automatic redirection of these responses, changing the POST to a `GET`, and the default implementation reproduces this behavior.

HTTPRedirectHandler.**http_error_301**(*req*, *fp*, *code*, *msg*, *hdrs*)

Redirect to the `Location:` or `URI:` URL. This method is called by the parent *OpenerDirector* when getting an HTTP 'moved permanently' response.

HTTPRedirectHandler.**http_error_302**(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as *http_error_301()*, but called for the 'found' response.

HTTPRedirectHandler.**http_error_303**(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as *http_error_301()*, but called for the 'see other' response.

HTTPRedirectHandler.**http_error_307**(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as *http_error_301()*, but called for the 'temporary redirect' response. It does not allow changing the request method from `POST` to `GET`.

HTTPRedirectHandler.**http_error_308**(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as *http_error_301()*, but called for the 'permanent redirect' response. It does not allow changing the request method from `POST` to `GET`.

Adicionado na versão 3.11.

22.4.5 HTTPCookieProcessor Objects

HTTPCookieProcessor instances have one attribute:

HTTPCookieProcessor.**cookiejar**

The *http.cookiejar.CookieJar* in which cookies are stored.

22.4.6 ProxyHandler Objects

ProxyHandler.<protocol>_open(request)

The *ProxyHandler* will have a method `<protocol>_open()` for every *protocol* which has a proxy in the *proxies* dictionary given in the constructor. The method will modify requests to go through the proxy, by calling `request.set_proxy()`, and call the next handler in the chain to actually execute the protocol.

22.4.7 HTTPPasswordMgr Objects

These methods are available on *HTTPPasswordMgr* and *HTTPPasswordMgrWithDefaultRealm* objects.

HTTPPasswordMgr.**add_password**(*realm*, *uri*, *user*, *passwd*)

uri can be either a single URI, or a sequence of URIs. *realm*, *user* and *passwd* must be strings. This causes `(user, passwd)` to be used as authentication tokens when authentication for *realm* and a super-URI of any of the given URIs is given.

HTTPPasswordMgr.**find_user_password**(*realm*, *authuri*)

Get user/password for given realm and URI, if any. This method will return `(None, None)` if there is no matching user/password.

For *HTTPPasswordMgrWithDefaultRealm* objects, the realm `None` will be searched if the given *realm* has no matching user/password.

22.4.8 HTTPPasswordMgrWithPriorAuth Objects

This password manager extends *HTTPPasswordMgrWithDefaultRealm* to support tracking URIs for which authentication credentials should always be sent.

HTTPPasswordMgrWithPriorAuth.**add_password**(*realm*, *uri*, *user*, *passwd*, *is_authenticated=False*)

realm, *uri*, *user*, *passwd* are as for *HTTPPasswordMgr.add_password()*. *is_authenticated* sets the initial value of the `is_authenticated` flag for the given URI or list of URIs. If *is_authenticated* is specified as `True`, *realm* is ignored.

HTTPPasswordMgrWithPriorAuth.**find_user_password**(*realm*, *authuri*)

Same as for *HTTPPasswordMgrWithDefaultRealm* objects

HTTPPasswordMgrWithPriorAuth.**update_authenticated**(*self*, *uri*, *is_authenticated=False*)

Update the `is_authenticated` flag for the given *uri* or list of URIs.

HTTPPasswordMgrWithPriorAuth.**is_authenticated**(*self*, *authuri*)

Returns the current state of the `is_authenticated` flag for the given URI.

22.4.9 AbstractBasicAuthHandler Objects

AbstractBasicAuthHandler.**http_error_auth_reqed**(*authreq*, *host*, *req*, *headers*)

Handle an authentication request by getting a user/password pair, and re-trying the request. *authreq* should be the name of the header where the information about the realm is included in the request, *host* specifies the URL and path to authenticate for, *req* should be the (failed) `Request` object, and *headers* should be the error headers.

host is either an authority (e.g. `"python.org"`) or a URL containing an authority component (e.g. `"http://python.org/"`). In either case, the authority must not contain a userinfo component (so, `"python.org"` and `"python.org:80"` are fine, `"joe:password@python.org"` is not).

22.4.10 HTTPBasicAuthHandler Objects

HTTPBasicAuthHandler.**http_error_401**(*req*, *fp*, *code*, *msg*, *hdrs*)

Retry the request with authentication information, if available.

22.4.11 ProxyBasicAuthHandler Objects

ProxyBasicAuthHandler.**http_error_407**(*req*, *fp*, *code*, *msg*, *hdrs*)

Retry the request with authentication information, if available.

22.4.12 AbstractDigestAuthHandler Objects

AbstractDigestAuthHandler.**http_error_auth_reqed**(*authreq*, *host*, *req*, *headers*)

authreq should be the name of the header where the information about the realm is included in the request, *host* should be the host to authenticate to, *req* should be the (failed) `Request` object, and *headers* should be the error headers.

22.4.13 HTTPDigestAuthHandler Objects

HTTPDigestAuthHandler.**http_error_401**(*req*, *fp*, *code*, *msg*, *hdrs*)

Retry the request with authentication information, if available.

22.4.14 ProxyDigestAuthHandler Objects

ProxyDigestAuthHandler.**http_error_407**(*req*, *fp*, *code*, *msg*, *hdrs*)

Retry the request with authentication information, if available.

22.4.15 HTTPHandler Objects

HTTPHandler.**http_open**(*req*)

Send an HTTP request, which can be either GET or POST, depending on `req.data`.

22.4.16 Objetos HTTPSHandler

HTTPSHandler.**https_open**(*req*)

Send an HTTPS request, which can be either GET or POST, depending on `req.data`.

22.4.17 FileHandler Objects

`FileHandler.`**`file_open`**(*req*)

Open the file locally, if there is no host name, or the host name is `'localhost'`.

Alterado na versão 3.2: This method is applicable only for local hostnames. When a remote hostname is given, a *URLError* is raised.

22.4.18 DataHandler Objects

`DataHandler.`**`data_open`**(*req*)

Read a data URL. This kind of URL contains the content encoded in the URL itself. The data URL syntax is specified in **RFC 2397**. This implementation ignores white spaces in base64 encoded data URLs so the URL may be wrapped in whatever source file it comes from. But even though some browsers don't mind about a missing padding at the end of a base64 encoded data URL, this implementation will raise a *ValueError* in that case.

22.4.19 FTPHandler Objects

`FTPHandler.`**`ftp_open`**(*req*)

Open the FTP file indicated by *req*. The login is always done with empty username and password.

22.4.20 CacheFTPHandler Objects

CacheFTPHandler objects are *FTPHandler* objects with the following additional methods:

`CacheFTPHandler.`**`setTimeout`**(*t*)

Set timeout of connections to *t* seconds.

`CacheFTPHandler.`**`setMaxConns`**(*m*)

Set maximum number of cached connections to *m*.

22.4.21 Objetos UnknownHandler

`UnknownHandler.`**`unknown_open`**()

Raise a *URLError* exception.

22.4.22 HTTPErrorProcessor Objects

`HTTPErrorProcessor.`**`http_response`**(*request*, *response*)

Process HTTP error responses.

For 200 error codes, the response object is returned immediately.

For non-200 error codes, this simply passes the job on to the `http_error_<type>()` handler methods, via *OpenerDirector.error()*. Eventually, *HTTPDefaultErrorHandler* will raise an *HTTPError* if no other handler handles the error.

`HTTPErrorProcessor.`**`https_response`**(*request*, *response*)

Process HTTPS error responses.

The behavior is same as *http_response()*.

22.4.23 Exemplos

In addition to the examples below, more examples are given in urllib-howto.

This example gets the python.org main page and displays the first 300 bytes of it:

```
>>> import urllib.request
>>> with urllib.request.urlopen('http://www.python.org/') as f:
...     print(f.read(300))
...
b'<!doctype html>\n<!--[if lt IE 7]>   <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9
↪">   <![endif]-->\n<!--[if IE 7]>      <html class="no-js ie7 lt-ie8 lt-ie9">   ␣
↪      <![endif]-->\n<!--[if IE 8]>      <html class="no-js ie8 lt-ie9">
```

Note that urlopen returns a bytes object. This is because there is no way for urlopen to automatically determine the encoding of the byte stream it receives from the HTTP server. In general, a program will decode the returned bytes object to string once it determines or guesses the appropriate encoding.

The following HTML spec document, https://html.spec.whatwg.org/#charset, lists the various ways in which an HTML or an XML document could have specified its encoding information.

For additional information, see the W3C document: https://www.w3.org/International/questions/qa-html-encoding-declarations.

As the python.org website uses *utf-8* encoding as specified in its meta tag, we will use the same for decoding the bytes object:

```
>>> with urllib.request.urlopen('http://www.python.org/') as f:
...     print(f.read(100).decode('utf-8'))
...
<!doctype html>
<!--[if lt IE 7]>   <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">   <![endif]-->
<!-
```

It is also possible to achieve the same result without using the *context manager* approach:

```
>>> import urllib.request
>>> f = urllib.request.urlopen('http://www.python.org/')
>>> try:
...     print(f.read(100).decode('utf-8'))
... finally:
...     f.close()
...
<!doctype html>
<!--[if lt IE 7]>   <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">   <![endif]-->
<!--
```

In the following example, we are sending a data-stream to the stdin of a CGI and reading the data it returns to us. Note that this example will only work when the Python installation supports SSL.

```
>>> import urllib.request
>>> req = urllib.request.Request(url='https://localhost/cgi-bin/test.cgi',
...                       data=b'This data is passed to stdin of the CGI')
>>> with urllib.request.urlopen(req) as f:
...     print(f.read().decode('utf-8'))
...
Got Data: "This data is passed to stdin of the CGI"
```

The code for the sample CGI used in the above example is:

```
#!/usr/bin/env python
import sys
data = sys.stdin.read()
print('Content-type: text/plain\n\nGot Data: "%s"' % data)
```

Here is an example of doing a `PUT` request using *Request*:

```
import urllib.request
DATA = b'some data'
req = urllib.request.Request(url='http://localhost:8080', data=DATA, method='PUT')
with urllib.request.urlopen(req) as f:
    pass
print(f.status)
print(f.reason)
```

Use of Basic HTTP Authentication:

```
import urllib.request
# Create an OpenerDirector with support for Basic HTTP Authentication...
auth_handler = urllib.request.HTTPBasicAuthHandler()
auth_handler.add_password(realm='PDQ Application',
                          uri='https://mahler:8092/site-updates.py',
                          user='klem',
                          passwd='kadidd!ehopper')
opener = urllib.request.build_opener(auth_handler)
# ...and install it globally so it can be used with urlopen.
urllib.request.install_opener(opener)
with urllib.request.urlopen('http://www.example.com/login.html') as f:
    print(f.read().decode('utf-8'))
```

build_opener() provides many handlers by default, including a *ProxyHandler*. By default, *ProxyHandler* uses the environment variables named <scheme>_proxy, where <scheme> is the URL scheme involved. For example, the http_proxy environment variable is read to obtain the HTTP proxy's URL.

This example replaces the default *ProxyHandler* with one that uses programmatically supplied proxy URLs, and adds proxy authorization support with *ProxyBasicAuthHandler*.

```
proxy_handler = urllib.request.ProxyHandler({'http': 'http://www.example.com:3128/
↪'})
proxy_auth_handler = urllib.request.ProxyBasicAuthHandler()
proxy_auth_handler.add_password('realm', 'host', 'username', 'password')

opener = urllib.request.build_opener(proxy_handler, proxy_auth_handler)
# This time, rather than install the OpenerDirector, we use it directly:
with opener.open('http://www.example.com/login.html') as f:
   print(f.read().decode('utf-8'))
```

Adicionando cabeçalhos HTTP:

Use the *headers* argument to the *Request* constructor, or:

```
import urllib.request
req = urllib.request.Request('http://www.example.com/')
req.add_header('Referer', 'http://www.python.org/')
# Customize the default User-Agent header value:
req.add_header('User-Agent', 'urllib-example/0.1 (Contact: . . .)')
with urllib.request.urlopen(req) as f:
    print(f.read().decode('utf-8'))
```

OpenerDirector automatically adds a *User-Agent* header to every *Request*. To change this:

```
import urllib.request
opener = urllib.request.build_opener()
opener.addheaders = [('User-agent', 'Mozilla/5.0')]
with opener.open('http://www.example.com/') as f:
   print(f.read().decode('utf-8'))
```

Also, remember that a few standard headers (*Content-Length*, *Content-Type* and *Host*) are added when the *Request* is passed to *urlopen()* (or *OpenerDirector.open()*).

Here is an example session that uses the GET method to retrieve a URL containing parameters:

```
>>> import urllib.request
>>> import urllib.parse
>>> params = urllib.parse.urlencode({'spam': 1, 'eggs': 2, 'bacon': 0})
>>> url = "http://www.musi-cal.com/cgi-bin/query?%s" % params
>>> with urllib.request.urlopen(url) as f:
...     print(f.read().decode('utf-8'))
...
```

The following example uses the POST method instead. Note that params output from urlencode is encoded to bytes before it is sent to urlopen as data:

```
>>> import urllib.request
>>> import urllib.parse
>>> data = urllib.parse.urlencode({'spam': 1, 'eggs': 2, 'bacon': 0})
>>> data = data.encode('ascii')
>>> with urllib.request.urlopen("http://requestb.in/xrbl82xr", data) as f:
...     print(f.read().decode('utf-8'))
...
```

The following example uses an explicitly specified HTTP proxy, overriding environment settings:

```
>>> import urllib.request
>>> proxies = {'http': 'http://proxy.example.com:8080/'}
>>> opener = urllib.request.build_opener(urllib.request.ProxyHandler(proxies))
>>> with opener.open("http://www.python.org") as f:
...     f.read().decode('utf-8')
...
```

The following example uses no proxies at all, overriding environment settings:

```
>>> import urllib.request
>>> opener = urllib.request.build_opener(urllib.request.ProxyHandler({}))
>>> with opener.open("http://www.python.org/") as f:
...     f.read().decode('utf-8')
...
```

22.4.24 Legacy interface

The following functions and classes are ported from the Python 2 module urllib (as opposed to urllib2). They might become deprecated at some point in the future.

urllib.request.**urlretrieve**(*url*, *filename=None*, *reporthook=None*, *data=None*)

Copy a network object denoted by a URL to a local file. If the URL points to a local file, the object will not be copied unless filename is supplied. Return a tuple (filename, headers) where *filename* is the local file name under which the object can be found, and *headers* is whatever the info() method of the object returned by *urlopen()* returned (for a remote object). Exceptions are the same as for *urlopen()*.

The second argument, if present, specifies the file location to copy to (if absent, the location will be a tempfile with a generated name). The third argument, if present, is a callable that will be called once on establishment of the network connection and once after each block read thereafter. The callable will be passed three arguments; a count of blocks transferred so far, a block size in bytes, and the total size of the file. The third argument may be -1 on older FTP servers which do not return a file size in response to a retrieval request.

The following example illustrates the most common usage scenario:

```
>>> import urllib.request
>>> local_filename, headers = urllib.request.urlretrieve('http://python.org/')
>>> html = open(local_filename)
>>> html.close()
```

If the *url* uses the `http:` scheme identifier, the optional *data* argument may be given to specify a `POST` request (normally the request type is `GET`). The *data* argument must be a bytes object in standard *application/x-www-form-urlencoded* format; see the *urllib.parse.urlencode()* function.

urlretrieve() will raise *ContentTooShortError* when it detects that the amount of data available was less than the expected amount (which is the size reported by a *Content-Length* header). This can occur, for example, when the download is interrupted.

The *Content-Length* is treated as a lower bound: if there's more data to read, urlretrieve reads more data, but if less data is available, it raises the exception.

You can still retrieve the downloaded data in this case, it is stored in the `content` attribute of the exception instance.

If no *Content-Length* header was supplied, urlretrieve can not check the size of the data it has downloaded, and just returns it. In this case you just have to assume that the download was successful.

`urllib.request.`**`urlcleanup`**`()`

Cleans up temporary files that may have been left behind by previous calls to *urlretrieve()*.

22.4.25 `urllib.request` Restrictions

- Currently, only the following protocols are supported: HTTP (versions 0.9 and 1.0), FTP, local files, and data URLs.

  Alterado na versão 3.4: Added support for data URLs.

- The caching feature of *urlretrieve()* has been disabled until someone finds the time to hack proper processing of Expiration time headers.
- There should be a function to query whether a particular URL is in the cache.
- For backward compatibility, if a URL appears to point to a local file but the file can't be opened, the URL is re-interpreted using the FTP protocol. This can sometimes cause confusing error messages.
- The *urlopen()* and *urlretrieve()* functions can cause arbitrarily long delays while waiting for a network connection to be set up. This means that it is difficult to build an interactive web client using these functions without using threads.
- The data returned by *urlopen()* or *urlretrieve()* is the raw data returned by the server. This may be binary data (such as an image), plain text or (for example) HTML. The HTTP protocol provides type information in the reply header, which can be inspected by looking at the *Content-Type* header. If the returned data is HTML, you can use the module *html.parser* to parse it.
- The code handling the FTP protocol cannot differentiate between a file and a directory. This can lead to unexpected behavior when attempting to read a URL that points to a file that is not accessible. If the URL ends in a `/`, it is assumed to refer to a directory and will be handled accordingly. But if an attempt to read a file leads to a 550 error (meaning the URL cannot be found or is not accessible, often for permission reasons), then the path is treated as a directory in order to handle the case when a directory is specified by a URL but the trailing `/` has been left off. This can cause misleading results when you try to fetch a file whose read permissions make it inaccessible; the FTP code will try to read it, fail with a 550 error, and then perform a directory listing for the unreadable file. If fine-grained control is needed, consider using the *ftplib* module.

22.5 `urllib.response` — Response classes used by urllib

The *urllib.response* module defines functions and classes which define a minimal file-like interface, including `read()` and `readline()`. Functions defined by this module are used internally by the *urllib.request* module. The typical response object is a *urllib.response.addinfourl* instance:

class urllib.response.**addinfourl**

url
URL of the resource retrieved, commonly used to determine if a redirect was followed.

headers
Returns the headers of the response in the form of an *EmailMessage* instance.

status
Adicionado na versão 3.9.

Status code returned by server.

geturl()
Descontinuado desde a versão 3.9: Deprecated in favor of *url*.

info()
Descontinuado desde a versão 3.9: Deprecated in favor of *headers*.

code
Descontinuado desde a versão 3.9: Deprecated in favor of *status*.

getcode()
Descontinuado desde a versão 3.9: Deprecated in favor of *status*.

22.6 urllib.parse — Analisa URLs para componentes

Código-fonte: Lib/urllib/parse.py

Este módulo define uma interface padrão para quebrar strings de Uniform Resource Locator (URL) em componentes (esquema de endereçamento, local de rede, caminho etc.), para combinar os componentes de volta em uma string de URL e para converter uma "URL relativo" em uma URL absoluta dado uma "URL base".

O módulo foi projetado para corresponder ao RFC da internet sobre Localizadores de Recursos Uniformes Relativos, ou URL relativa. Ele tem suporte aos seguintes esquemas de URL: file, ftp, gopher, hdl, http, https, imap, itms-services, mailto, mms, news, nntp, prospero, rsync, rtsp, rtsps,"rtspu", sftp, shttp, sip, sips, snews, svn, svn+ssh, telnet, wais, ws, wss.

A inclusão do esquema de URL itms-services pode impedir que uma aplicação passe pelo processo de revisão da App Store da Apple para as App Stores do macOS e iOS. O tratamento do esquema itms-services é sempre removido no iOS; no macOS, ele *pode* ser removido se o CPython tiver sido criado com a opção --with-app-store-compliance.

O módulo *urllib.parse* define funções que se enquadram em duas grandes categorias: análise de URL e colocação de aspas na URL. Eles são abordados em detalhes nas seções a seguir.

As funções deste módulo usam o termo descontinuado netloc (ou net_loc), que foi introduzido no **RFC 1808**. No entanto, este termo foi descontinuado pelo **RFC 3986**, que introduziu o termo authority como seu substituto. O uso de netloc continua para compatibilidade com versões anteriores.

22.6.1 Análise de URL

As funções de análise de URL se concentram na divisão de uma string de URL em seus componentes ou na combinação de componentes de URL em uma string de URL.

urllib.parse.**urlparse**(*urlstring*, *scheme=''*, *allow_fragments=True*)
Analisa uma URL em seis componentes, retornando uma *tupla nomeada* de 6 itens. Isso corresponde à estrutura geral de uma URL: scheme://netloc/path;parameters?query#fragment. Cada item da tupla é uma string, possivelmente vazia. Os componentes não são divididos em partes menores (por exemplo, o netloc, ou local da rede, é uma única string) e escapes % não são expandidos. Os delimitadores conforme mostrado acima não fazem parte do resultado, exceto por uma barra inicial no componente *path*, que é retido se estiver presente. Por exemplo:

```
>>> from urllib.parse import urlparse
>>> urlparse("scheme://netloc/path;parameters?query#fragment")
ParseResult(scheme='scheme', netloc='netloc', path='/path;parameters', params='
↪',
            query='query', fragment='fragment')
>>> o = urlparse("http://docs.python.org:80/3/library/urllib.parse.html?"
...              "highlight=params#url-parsing")
>>> o
ParseResult(scheme='http', netloc='docs.python.org:80',
            path='/3/library/urllib.parse.html', params='',
            query='highlight=params', fragment='url-parsing')
>>> o.scheme
'http'
>>> o.netloc
'docs.python.org:80'
>>> o.hostname
'docs.python.org'
>>> o.port
80
>>> o._replace(fragment="").geturl()
'http://docs.python.org:80/3/library/urllib.parse.html?highlight=params'
```

Seguindo as especificações de sintaxe em **RFC 1808**, o urlparse reconhece um netloc apenas se for introduzido apropriadamente por '//'. Caso contrário, presume-se que a entrada seja uma URL relativa e, portanto, comece com um componente de caminho.

```
>>> from urllib.parse import urlparse
>>> urlparse('//www.cwi.nl:80/%7Eguido/Python.html')
ParseResult(scheme='', netloc='www.cwi.nl:80', path='/%7Eguido/Python.html',
            params='', query='', fragment='')
>>> urlparse('www.cwi.nl/%7Eguido/Python.html')
ParseResult(scheme='', netloc='', path='www.cwi.nl/%7Eguido/Python.html',
            params='', query='', fragment='')
>>> urlparse('help/Python.html')
ParseResult(scheme='', netloc='', path='help/Python.html', params='',
            query='', fragment='')
```

O argumento *scheme* fornece o esquema de endereçamento padrão, a ser usado apenas se o URL não especificar um. Deve ser do mesmo tipo (texto ou bytes) que *urlstring*, exceto que o valor padrão `''` é sempre permitido e é automaticamente convertido para `b''` se apropriado.

Se o argumento *allow_fragments* for falso, os identificadores de fragmento não serão reconhecidos. Em vez disso, eles são analisados como parte do caminho, parâmetros ou componente de consulta, e `fragment` é definido como a string vazia no valor de retorno.

O valor de retorno é uma *tupla nomeada*, o que significa que seus itens podem ser acessados por índice ou como atributos nomeados, que são:

Atributo	Índice	Valor	Valor, se não presente
`scheme`	0	Especificador do esquema da URL	parâmetro *scheme*
`netloc`	1	Parte da localização na rede	string vazia
`path`	2	Caminho hierárquico	string vazia
`params`	3	Parâmetros para o último elemento de caminho	string vazia
`query`	4	Componente da consulta	string vazia
`fragment`	5	Identificador do fragmento	string vazia
`username`		Nome do usuário	*None*
`password`		Senha	*None*
`hostname`		Nome de máquina (em minúsculo)	*None*
`port`		Número da porta como inteiro, se presente	*None*

Ler o atributo `port` irá levantar uma *ValueError* se uma porta inválida for especificada no URL. Veja a seção *Structured Parse Results* para mais informações sobre o objeto de resultado.

Colchetes sem correspondência no atributo `netloc` levantará uma *ValueError*.

Caracteres no atributo `netloc` que se decompõem sob a normalização NFKC (como usado pela codificação IDNA) em qualquer um dos `/`, `?`, `#`, `@` ou `:` vai levantar uma *ValueError*. Se a URL for decomposta antes da análise, nenhum erro será levantado.

Como é o caso com todas as tuplas nomeadas, a subclasse tem alguns métodos e atributos adicionais que são particularmente úteis. Um desses métodos é `_replace()`. O método `_replace()` retornará um novo objeto ParseResult substituindo os campos especificados por novos valores.

```
>>> from urllib.parse import urlparse
>>> u = urlparse('//www.cwi.nl:80/%7Eguido/Python.html')
>>> u
ParseResult(scheme='', netloc='www.cwi.nl:80', path='/%7Eguido/Python.html',
            params='', query='', fragment='')
>>> u._replace(scheme='http')
ParseResult(scheme='http', netloc='www.cwi.nl:80', path='/%7Eguido/Python.html
↪',
            params='', query='', fragment='')
```

Aviso

urlparse() não realiza validação. Veja *Segurança ao analisar URLs* para detalhes.

Alterado na versão 3.2: Adicionados recursos de análise de URL IPv6.

Alterado na versão 3.3: O fragmento agora é analisado para todos os esquemas de URL (a menos que *allow_fragments* seja falso), de acordo com a **RFC 3986**. Anteriormente, existia uma lista de permitidos de esquemas que suportam fragmentos.

Alterado na versão 3.6: Números de porta fora do intervalo agora levantam *ValueError*, em vez de retornar *None*.

Alterado na versão 3.8: Os caracteres que afetam a análise de netloc sob normalização NFKC agora levantarão *ValueError*.

`urllib.parse.`**`parse_qs`**(*qs*, *keep_blank_values=False*, *strict_parsing=False*, *encoding='utf-8'*, *errors='replace'*, *max_num_fields=None*, *separator='&'*)

Analisa uma string de consulta fornecida como um argumento de string (dados do tipo *application/x-www-form-urlencoded*). Os dados são retornados como um dicionário. As chaves de dicionário são os nomes de variáveis de consulta exclusivos e os valores são listas de valores para cada nome.

O argumento opcional *keep_blank_values* é um sinalizador que indica se os valores em branco em consultas codificadas por porcentagem devem ser tratados como strings em branco. Um valor verdadeiro indica que os

espaços em branco devem ser mantidos como strings em branco. O valor falso padrão indica que os valores em branco devem ser ignorados e tratados como se não tivessem sido incluídos.

O argumento opcional *strict_parsing* é um sinalizador que indica o que fazer com os erros de análise. Se falso (padrão), os erros são ignorados silenciosamente. Se verdadeiro, os erros levantam uma exceção `ValueError`.

Os parâmetros opcionais *encoding* e *errors* especificam como decodificar sequências codificadas em porcentagem em caracteres Unicode, conforme aceito pelo método `bytes.decode()`.

O argumento opcional *max_num_fields* é o número máximo de campos a serem lidos. Se definido, então levanta um `ValueError` se houver mais de *max_num_fields* campos lidos.

O argumento opcional *separador* é o símbolo a ser usado para separar os argumentos da consulta. O padrão é `&`.

Use a função `urllib.parse.urlencode()` (com o parâmetro `doseq` definido como `True`) para converter esses dicionários em strings de consulta.

Alterado na versão 3.2: Adicionado os parâmetros *encoding* e *errors*.

Alterado na versão 3.8: Adicionado o parâmetro *max_num_fields*.

Alterado na versão 3.10: Adicionado parâmetro *separator* com o valor padrão de `&`. Versões do Python anteriores ao Python 3.10 permitiam o uso de `;` e `&` como separador de parâmetro de consulta. Isso foi alterado para permitir apenas uma única chave separadora, com `&` como o separador padrão.

Descontinuado desde a versão 3.14: Accepting objects with false values (like `0` and `[]`) except empty strings and byte-like objects and `None` is now deprecated.

urllib.parse.**parse_qsl**(*qs*, *keep_blank_values=False*, *strict_parsing=False*, *encoding='utf-8'*, *errors='replace'*, *max_num_fields=None*, *separator='&'*)

Analisa uma string de consulta fornecida como um argumento de string (dados do tipo `application/x-www-form-urlencoded`). Os dados são retornados como uma lista de pares de nome e valor.

O argumento opcional *keep_blank_values* é um sinalizador que indica se os valores em branco em consultas codificadas por porcentagem devem ser tratados como strings em branco. Um valor verdadeiro indica que os espaços em branco devem ser mantidos como strings em branco. O valor falso padrão indica que os valores em branco devem ser ignorados e tratados como se não tivessem sido incluídos.

O argumento opcional *strict_parsing* é um sinalizador que indica o que fazer com os erros de análise. Se falso (padrão), os erros são ignorados silenciosamente. Se verdadeiro, os erros levantam uma exceção `ValueError`.

Os parâmetros opcionais *encoding* e *errors* especificam como decodificar sequências codificadas em porcentagem em caracteres Unicode, conforme aceito pelo método `bytes.decode()`.

O argumento opcional *max_num_fields* é o número máximo de campos a serem lidos. Se definido, então levanta um `ValueError` se houver mais de *max_num_fields* campos lidos.

O argumento opcional *separador* é o símbolo a ser usado para separar os argumentos da consulta. O padrão é `&`.

Use a função `urllib.parse.urlencode()` para converter essas listas pares de strings de consulta.

Alterado na versão 3.2: Adicionado os parâmetros *encoding* e *errors*.

Alterado na versão 3.8: Adicionado o parâmetro *max_num_fields*.

Alterado na versão 3.10: Adicionado parâmetro *separator* com o valor padrão de `&`. Versões do Python anteriores ao Python 3.10 permitiam o uso de `;` e `&` como separador de parâmetro de consulta. Isso foi alterado para permitir apenas uma única chave separadora, com `&` como o separador padrão.

urllib.parse.**urlunparse**(*parts*)

Constrói uma URL a partir de uma tupla conforme retornada por `urlparse()`. O argumento *parts* pode ser qualquer iterável de seis itens. Isso pode resultar em uma URL ligeiramente diferente, mas equivalente, se a URL que foi analisada originalmente tiver delimitadores desnecessários (por exemplo, um `?` com uma consulta vazia; o RFC afirma que eles são equivalentes).

urllib.parse.**urlsplit**(*urlstring*, *scheme=''*, *allow_fragments=True*)

Isto é similar a `urlparse()`, mas não divide os parâmetros da URL. Isto deve ser usado geralmente em vez de `urlparse()` se a sintaxe de URL mais recente permitindo que parâmetros sejam aplicados a cada segmento da porção *path* da URL (veja **RFC 2396**) for desejada. Uma função separada é necessária para separar os segmentos de caminho e parâmetros. Esta função retorna uma *tupla nomeada* de 5 itens:

```
(esquema de endereçamento, local de rede, caminho, consulta, identificador de␣
↪fragmento).
```

O valor de retorno é uma *tupla nomeada*, seus itens podem ser acessados por índice ou como atributos nomeados:

Atributo	Índice	Valor	Valor, se não presente
`scheme`	0	Especificador do esquema da URL	parâmetro *scheme*
`netloc`	1	Parte da localização na rede	string vazia
`path`	2	Caminho hierárquico	string vazia
`query`	3	Componente da consulta	string vazia
`fragment`	4	Identificador do fragmento	string vazia
`username`		Nome do usuário	`None`
`password`		Senha	`None`
`hostname`		Nome de máquina (em minúsculo)	`None`
`port`		Número da porta como inteiro, se presente	`None`

Ler o atributo `port` irá levantar uma `ValueError` se uma porta inválida for especificada no URL. Veja a seção *Structured Parse Results* para mais informações sobre o objeto de resultado.

Colchetes sem correspondência no atributo `netloc` levantará uma `ValueError`.

Caracteres no atributo `netloc` que se decompõem sob a normalização NFKC (como usado pela codificação IDNA) em qualquer um dos `/`, `?`, `#`, `@` ou `:` vai levantar uma `ValueError`. Se a URL for decomposta antes da análise, nenhum erro será levantado.

Seguindo algumas das especificações WHATWG que atualizam o RFC 3986, os caracteres de controle C0 e espaço iniciais são removidos da URL. Os caracteres `\n`, `\r` e tab `\t` são removidos da URL em qualquer posição.

Aviso

`urlsplit()` não executa validação. Veja *Segurança ao analisar URLs* para detalhes.

Alterado na versão 3.6: Números de porta fora do intervalo agora levantam `ValueError`, em vez de retornar `None`.

Alterado na versão 3.8: Os caracteres que afetam a análise de netloc sob normalização NFKC agora levantarão `ValueError`.

Alterado na versão 3.10: Caracteres de nova linha e tabulação ASCII são removidos do URL.

Alterado na versão 3.12: Os caracteres iniciais de espaço e de controle WHATWG C0 são removidos do URL.

urllib.parse.**urlunsplit**(*parts*)

Combina os elementos de uma tupla conforme retornado por `urlsplit()` em uma URL completa como uma string. O argumento *parts* pode ser qualquer iterável de cinco itens. Isso pode resultar em uma URL ligeiramente diferente, mas equivalente, se a URL que foi analisada originalmente tiver delimitadores desnecessários (por exemplo, um ? com uma consulta vazia; o RFC afirma que eles são equivalentes).

urllib.parse.**urljoin**(*base*, *url*, *allow_fragments=True*)

Constrói uma URL completa ("absoluta") combinando uma "URL base" (*base*) com outra URL (*url*). Infor-

malmente, isso usa componentes da URL base, em particular o esquema de endereçamento, o local da rede e (parte do) caminho, para fornecer componentes ausentes na URL relativa. Por exemplo:

```
>>> from urllib.parse import urljoin
>>> urljoin('http://www.cwi.nl/%7Eguido/Python.html', 'FAQ.html')
'http://www.cwi.nl/%7Eguido/FAQ.html'
```

O argumento *allow_fragments* tem o mesmo significado e padrão que *urlparse()*.

Nota

Se *url* for uma URL absoluta (ou seja, começa com `//` ou `scheme://`), o nome do host e/ou esquema da *url* estarão presentes no resultado. Por exemplo:

```
>>> urljoin('http://www.cwi.nl/%7Eguido/Python.html',
...         '//www.python.org/%7Eguido')
'http://www.python.org/%7Eguido'
```

Se você não quiser esse comportamento, pré-processe a *url* com *urlsplit()* e *urlunsplit()*, removendo possíveis partes *scheme* e *netloc*.

Aviso

Como uma URL absoluta pode ser passada como o parâmetro `url`, geralmente **não é seguro** usar `urljoin` com uma `url` controlada pelo invasor. Por exemplo, em `urljoin("https://website.com/users/", username)`, se `username` puder conter uma URL absoluta, o resultado de `urljoin` será a URL absoluta.

Alterado na versão 3.5: Comportamento atualizado para corresponder à semântica definida no **RFC 3986**.

`urllib.parse.`**`urldefrag`**(*url*)

Se *url* contiver um identificador de fragmento, retorna uma versão modificada de *url* sem identificador de fragmento, e o identificador de fragmento como uma string separada. Se não houver identificador de fragmento em *url*, retorna *url* não modificado e uma string vazia.

O valor de retorno é uma *tupla nomeada*, seus itens podem ser acessados por índice ou como atributos nomeados:

Atributo	Índice	Valor	Valor, se não presente
`url`	0	URL com nenhum fragmento	string vazia
`fragment`	1	Identificador do fragmento	string vazia

Veja a seção *Structured Parse Results* para obter mais informações sobre o objeto de resultado.

Alterado na versão 3.2: O resultado é um objeto estruturado em vez de uma simples tupla de 2 elementos.

`urllib.parse.`**`unwrap`**(*url*)

Extrai a URL de uma URL encapsulada (ou seja, uma string formatada como `<URL:esquema://host/caminho>`, `<esquema://host/caminho>`, `URL:esquema://host/caminho` ou `esquema://host/caminho`). Se *url* não for uma URL encapsulada, ela será retornada sem alterações.

22.6.2 Segurança ao analisar URLs

As APIs *urlsplit()* e *urlparse()* não realizam **validação** de entradas. Elas podem não levantar erros em entradas que outras aplicações consideram inválidas. Elas também podem ter sucesso em algumas entradas que podem não ser consideradas URLs em outros lugares. Seu propósito é para funcionalidade prática em vez de pureza.

Em vez de levantar uma exceção em uma entrada incomum, eles podem retornar algumas partes do componente como strings vazias. Ou os componentes podem conter mais do que talvez devessem.

Recomendamos que os usuários dessas APIs, onde os valores podem ser usados em qualquer lugar com implicações de segurança, codifiquem defensivamente. Faça alguma verificação dentro do seu código antes de confiar em uma parte do componente retornada. Esse `esquema` faz sentido? Esse `caminho` sensato? Há algo estranho sobre esse `hostname`? etc.

O que constitui uma URL não é universalmente bem definido. Diferentes aplicações têm diferentes necessidades e restrições desejadas. Por exemplo, a especificação WHATWG descreve o que os clientes web voltados para o usuário, como um navegador web, exigem. Por outro lado, **RFC 3986** é mais geral. Essas funções incorporam alguns aspectos de ambos, mas não podem ser reivindicadas como compatíveis com nenhum deles. As APIs e o código de usuário existente com expectativas sobre comportamentos específicos são anteriores a ambos os padrões, o que nos leva a ser muito cautelosos ao fazer alterações no comportamento da API.

22.6.3 Analisando bytes codificados em ASCII

As funções de análise de URL foram originalmente projetadas para operar somente em strings de caracteres. Na prática, é útil ser capaz de manipular URLs corretamente citadas e codificadas como sequências de bytes ASCII. Consequentemente, todas as funções de análise de URL neste módulo operam em objetos *bytes* e *bytearray* além de objetos *str*.

Se dados *str* forem passados, o resultado também conterá apenas dados *str*. Se dados *bytes* ou *bytearray* forem passados, o resultado conterá apenas dados *bytes*.

Tentar misturar dados *str* com *bytes* ou *bytearray* em uma única chamada de função resultará em uma *TypeError* sendo levantada, enquanto tentar passar valores de bytes não-ASCII acionará *UnicodeDecodeError*.

Para prover uma conversão mais fácil de objetos de resultado entre *str* e *bytes*, todos os valores de retorno das funções de análise de URL fornecem um método `encode()` (quando o resultado contém dados *str*) ou um método `decode()` (quando o resultado contém dados *bytes*). As assinaturas desses métodos correspondem às dos métodos *str* e *bytes* correspondentes (exceto que a codificação padrão é `'ascii'` em vez de `'utf-8'`). Cada um produz um valor de um tipo correspondente que contém dados *bytes* (para métodos `encode()`) ou dados *str* (para métodos `decode()`).

Os aplicações que precisam operar em URLs possivelmente envolta em aspas de forma incorreta, e que podem conter dados não ASCII, precisarão fazer sua própria decodificação de bytes para caracteres antes de invocar os métodos de análise de URL.

O comportamento descrito nesta seção se aplica somente às funções de análise de URL. As funções de aplicação de aspas em URL usam suas próprias regras ao produzir ou consumir sequências de bytes, conforme detalhado na documentação das funções de citação de URL individuais.

Alterado na versão 3.2: As funções de análise de URL agora aceitam sequências de bytes codificadas em ASCII

22.6.4 Structured Parse Results

The result objects from the *urlparse()*, *urlsplit()* and *urldefrag()* functions are subclasses of the *tuple* type. These subclasses add the attributes listed in the documentation for those functions, the encoding and decoding support described in the previous section, as well as an additional method:

`urllib.parse.SplitResult.`**`geturl`**`()`

Return the re-combined version of the original URL as a string. This may differ from the original URL in that the scheme may be normalized to lower case and empty components may be dropped. Specifically, empty parameters, queries, and fragment identifiers will be removed.

For *urldefrag()* results, only empty fragment identifiers will be removed. For *urlsplit()* and *urlparse()* results, all noted changes will be made to the URL returned by this method.

The result of this method remains unchanged if passed back through the original parsing function:

```
>>> from urllib.parse import urlsplit
>>> url = 'HTTP://www.Python.org/doc/#'
```

(continua na próxima página)

(continuação da página anterior)

```
>>> r1 = urlsplit(url)
>>> r1.geturl()
'http://www.Python.org/doc/'
>>> r2 = urlsplit(r1.geturl())
>>> r2.geturl()
'http://www.Python.org/doc/'
```

The following classes provide the implementations of the structured parse results when operating on *str* objects:

class urllib.parse.**DefragResult**(*url*, *fragment*)

Concrete class for *urldefrag()* results containing *str* data. The encode() method returns a *DefragResultBytes* instance.

Adicionado na versão 3.2.

class urllib.parse.**ParseResult**(*scheme*, *netloc*, *path*, *params*, *query*, *fragment*)

Concrete class for *urlparse()* results containing *str* data. The encode() method returns a *ParseResultBytes* instance.

class urllib.parse.**SplitResult**(*scheme*, *netloc*, *path*, *query*, *fragment*)

Concrete class for *urlsplit()* results containing *str* data. The encode() method returns a *SplitResultBytes* instance.

The following classes provide the implementations of the parse results when operating on *bytes* or *bytearray* objects:

class urllib.parse.**DefragResultBytes**(*url*, *fragment*)

Concrete class for *urldefrag()* results containing *bytes* data. The decode() method returns a *DefragResult* instance.

Adicionado na versão 3.2.

class urllib.parse.**ParseResultBytes**(*scheme*, *netloc*, *path*, *params*, *query*, *fragment*)

Concrete class for *urlparse()* results containing *bytes* data. The decode() method returns a *ParseResult* instance.

Adicionado na versão 3.2.

class urllib.parse.**SplitResultBytes**(*scheme*, *netloc*, *path*, *query*, *fragment*)

Concrete class for *urlsplit()* results containing *bytes* data. The decode() method returns a *SplitResult* instance.

Adicionado na versão 3.2.

22.6.5 URL Quoting

The URL quoting functions focus on taking program data and making it safe for use as URL components by quoting special characters and appropriately encoding non-ASCII text. They also support reversing these operations to recreate the original data from the contents of a URL component if that task isn't already covered by the URL parsing functions above.

urllib.parse.**quote**(*string*, *safe='/'*, *encoding=None*, *errors=None*)

Replace special characters in *string* using the `%xx` escape. Letters, digits, and the characters `'_.-~'` are never quoted. By default, this function is intended for quoting the path section of a URL. The optional *safe* parameter specifies additional ASCII characters that should not be quoted — its default value is `'/'`.

string may be either a *str* or a *bytes* object.

Alterado na versão 3.7: Moved from **RFC 2396** to **RFC 3986** for quoting URL strings. "~" is now included in the set of unreserved characters.

The optional *encoding* and *errors* parameters specify how to deal with non-ASCII characters, as accepted by the *str.encode()* method. *encoding* defaults to `'utf-8'`. *errors* defaults to `'strict'`, meaning unsupported

characters raise a *UnicodeEncodeError*. *encoding* and *errors* must not be supplied if *string* is a *bytes*, or a *TypeError* is raised.

Note that `quote(string, safe, encoding, errors)` is equivalent to `quote_from_bytes(string.encode(encoding, errors), safe)`.

Example: `quote('/El Niño/')` yields `'/El%20Ni%C3%B1o/'`.

`urllib.parse.`**`quote_plus`**(*string*, *safe=''*, *encoding=None*, *errors=None*)

Like *quote()*, but also replace spaces with plus signs, as required for quoting HTML form values when building up a query string to go into a URL. Plus signs in the original string are escaped unless they are included in *safe*. It also does not have *safe* default to `'/'`.

Example: `quote_plus('/El Niño/')` yields `'%2FEl+Ni%C3%B1o%2F'`.

`urllib.parse.`**`quote_from_bytes`**(*bytes*, *safe='/'*)

Like *quote()*, but accepts a *bytes* object rather than a *str*, and does not perform string-to-bytes encoding.

Example: `quote_from_bytes(b'a&\xef')` yields `'a%26%EF'`.

`urllib.parse.`**`unquote`**(*string*, *encoding='utf-8'*, *errors='replace'*)

Replace `%xx` escapes with their single-character equivalent. The optional *encoding* and *errors* parameters specify how to decode percent-encoded sequences into Unicode characters, as accepted by the *bytes.decode()* method.

string may be either a *str* or a *bytes* object.

encoding defaults to `'utf-8'`. *errors* defaults to `'replace'`, meaning invalid sequences are replaced by a placeholder character.

Example: `unquote('/El%20Ni%C3%B1o/')` yields `'/El Niño/'`.

Alterado na versão 3.9: *string* parameter supports bytes and str objects (previously only str).

`urllib.parse.`**`unquote_plus`**(*string*, *encoding='utf-8'*, *errors='replace'*)

Like *unquote()*, but also replace plus signs with spaces, as required for unquoting HTML form values.

string must be a *str*.

Example: `unquote_plus('/El+Ni%C3%B1o/')` yields `'/El Niño/'`.

`urllib.parse.`**`unquote_to_bytes`**(*string*)

Replace `%xx` escapes with their single-octet equivalent, and return a *bytes* object.

string may be either a *str* or a *bytes* object.

If it is a *str*, unescaped non-ASCII characters in *string* are encoded into UTF-8 bytes.

Example: `unquote_to_bytes('a%26%EF')` yields `b'a&\xef'`.

`urllib.parse.`**`urlencode`**(*query*, *doseq=False*, *safe=''*, *encoding=None*, *errors=None*, *quote_via=quote_plus*)

Convert a mapping object or a sequence of two-element tuples, which may contain *str* or *bytes* objects, to a percent-encoded ASCII text string. If the resultant string is to be used as a *data* for POST operation with the *urlopen()* function, then it should be encoded to bytes, otherwise it would result in a *TypeError*.

The resulting string is a series of `key=value` pairs separated by `'&'` characters, where both *key* and *value* are quoted using the *quote_via* function. By default, *quote_plus()* is used to quote the values, which means spaces are quoted as a `'+'` character and '/' characters are encoded as `%2F`, which follows the standard for GET requests (`application/x-www-form-urlencoded`). An alternate function that can be passed as *quote_via* is *quote()*, which will encode spaces as `%20` and not encode '/' characters. For maximum control of what is quoted, use `quote` and specify a value for *safe*.

When a sequence of two-element tuples is used as the *query* argument, the first element of each tuple is a key and the second is a value. The value element in itself can be a sequence and in that case, if the optional parameter *doseq* evaluates to `True`, individual `key=value` pairs separated by `'&'` are generated for each element of the value sequence for the key. The order of parameters in the encoded string will match the order of parameter tuples in the sequence.

The *safe*, *encoding*, and *errors* parameters are passed down to *quote_via* (the *encoding* and *errors* parameters are only passed when a query element is a `str`).

To reverse this encoding process, `parse_qs()` and `parse_qsl()` are provided in this module to parse query strings into Python data structures.

Refer to *urllib examples* to find out how the `urllib.parse.urlencode()` method can be used for generating the query string of a URL or data for a POST request.

Alterado na versão 3.2: *query* supports bytes and string objects.

Alterado na versão 3.5: Added the *quote_via* parameter.

Descontinuado desde a versão 3.14: Accepting objects with false values (like `0` and `[]`) except empty strings and byte-like objects and `None` is now deprecated.

Ver também

WHATWG - URL Living standard
Working Group for the URL Standard that defines URLs, domains, IP addresses, the application/x-www-form-urlencoded format, and their API.

RFC 3986 - Uniform Resource Identifiers
This is the current standard (STD66). Any changes to urllib.parse module should conform to this. Certain deviations could be observed, which are mostly for backward compatibility purposes and for certain de-facto parsing requirements as commonly observed in major browsers.

RFC 2732 - Format for Literal IPv6 Addresses in URL's.
This specifies the parsing requirements of IPv6 URLs.

RFC 2396 - Uniform Resource Identifiers (URI): Generic Syntax
Document describing the generic syntactic requirements for both Uniform Resource Names (URNs) and Uniform Resource Locators (URLs).

RFC 2368 - O esquema de URL mailto.
Parsing requirements for mailto URL schemes.

RFC 1808 - Relative Uniform Resource Locators
This Request For Comments includes the rules for joining an absolute and a relative URL, including a fair number of "Abnormal Examples" which govern the treatment of border cases.

RFC 1738 - Uniform Resource Locators (URL)
This specifies the formal syntax and semantics of absolute URLs.

22.7 `urllib.error` — Classes de exceção levantadas por urllib.request

Código-fonte: Lib/urllib/error.py

O módulo `urllib.error` define as classes de exceção para exceções levantadas por `urllib.request`. A classe de exceção base é `URLError`.

As seguintes exceções são levantadas por `urllib.error` conforme apropriado:

exception urllib.error.**URLError**

Os manipuladores levantam essa exceção (ou exceções derivadas) quando encontram um problema. É uma subclasse de `OSError`.

reason

O motivo desse erro. Pode ser uma string de mensagem ou outra instância de exceção.

Alterado na versão 3.3: *URLError* costumava ser um subtipo de *IOError*, que agora é um apelido de *OSError*.

exception urllib.error.**HTTPError**(*url*, *code*, *msg*, *hdrs*, *fp*)

Embora seja uma exceção (uma subclasse de *URLError*), uma *HTTPError* também pode funcionar como um valor de retorno não excepcional do tipo arquivo (a mesma coisa que *urlopen()* retorna). Isso é útil ao lidar com erros de HTTP exóticos, como solicitações de autenticação.

url

Contém a solicitação URL. Um apelido para *nome do arquivo* atributo.

code

Um código de status HTTP conforme definido em **RFC 2616**. Este valor numérico corresponde a um valor encontrado no dicionário de códigos conforme encontrado em *http.server.BaseHTTPRequestHandler.responses*.

reason

Geralmente é uma string explicando o motivo desse erro. Um apelido para o atributo *msg*.

headers

Os cabeçalhos de resposta HTTP para a solicitação HTTP que causou a *HTTPError*. Um apelido para o atributo *hdrs*.

Adicionado na versão 3.4.

fp

Um objeto arquivo ou similar no qual o corpo do HTTP erro pode ser lido.

exception urllib.error.**ContentTooShortError**(*msg*, *content*)

Esta exceção é levantada quando a função *urlretrieve()* detecta que a quantidade de dados baixados é menor que a quantidade esperada (fornecida pelo cabeçalho *Content-Length*).

content

Os dados baixados (e supostamente truncados).

22.8 urllib.robotparser — Analisador sintático de robots.txt

Código-fonte: Lib/urllib/robotparser.py

Este módulo fornece uma única classe, *RobotFileParser*, que responde a perguntas sobre se um agente de usuário específico pode ou não buscar uma URL no site que publicou o arquivo `robots.txt`. Para mais detalhes sobre a estrutura dos arquivos `robots.txt`, consulte http://www.robotstxt.org/orig.html.

class urllib.robotparser.**RobotFileParser**(*url=''*)

Esta classe fornece métodos para ler, analisar e responder perguntas sobre o arquivo `robots.txt` em *url*.

set_url(*url*)

Define a URL referente a um arquivo `robots.txt`.

read()

Lê o URL `robots.txt` e o alimenta para o analisador sintático.

parse(*lines*)

Analisa o argumento de linhas.

can_fetch(*useragent*, *url*)

Retorna `True` se o *useragent* tiver permissão para buscar a *url* de acordo com as regras contidas no arquivo `robots.txt` analisado.

mtime()

Retorna o horário em que o arquivo `robots.txt` foi buscado pela última vez. Isso é útil para web spiders de longa duração que precisam verificar novos arquivos `robots.txt` periodicamente.

modified()

Define a hora em que o arquivo `robots.txt` foi buscado pela última vez para a hora atual.

crawl_delay(*useragent*)

Retorna o valor do parâmetro `Crawl-delay` de `robots.txt` para o *useragent* em questão. Se não houver tal parâmetro ou se ele não se aplicar ao *useragent* especificado ou se a entrada `robots.txt` para este parâmetro tiver sintaxe inválida, retorna `None`.

Adicionado na versão 3.6.

request_rate(*useragent*)

Retorna o conteúdo do parâmetro `Request-rate` de `robots.txt` como uma *tupla nomeada* `RequestRate(requests, seconds)`. Se não houver tal parâmetro ou se ele não se aplicar ao *useragent* especificado ou se a entrada `robots.txt` para este parâmetro tiver sintaxe inválida, retorna `None`.

Adicionado na versão 3.6.

site_maps()

Retorna o conteúdo do parâmetro `Sitemap` de `robots.txt` no formato de uma *list()*. Se não houver tal parâmetro ou a entrada `robots.txt` para este parâmetro tiver sintaxe inválida, retorna `None`.

Adicionado na versão 3.8.

O exemplo a seguir demonstra o uso básico da classe *RobotFileParser*:

```
>>> import urllib.robotparser
>>> rp = urllib.robotparser.RobotFileParser()
>>> rp.set_url("http://www.musi-cal.com/robots.txt")
>>> rp.read()
>>> rrate = rp.request_rate("*")
>>> rrate.requests
3
>>> rrate.seconds
20
>>> rp.crawl_delay("*")
6
>>> rp.can_fetch("*", "http://www.musi-cal.com/cgi-bin/search?city=San+Francisco")
False
>>> rp.can_fetch("*", "http://www.musi-cal.com/")
True
```

22.9 `http` — HTTP modules

Código-fonte: Lib/http/__init__.py

---

http é um pacote que coleta vários módulos para trabalhar com o Protocolo de Transferência de Hipertexto:

- *http.client* é um cliente de protocolo HTTP de baixo nível; para abertura de URL de alto nível, use *urllib.request*
- *http.server* contém classes básicas de servidor HTTP baseadas em *socketserver*
- *http.cookies* tem utilidades para implementar gerenciamento de estado com cookies
- *http.cookiejar* provê persistência de cookies

O módulo *http* também define os seguintes enums que podem ajudar você a trabalhar com os status code http relacionados:

class http.**HTTPStatus**

Adicionado na versão 3.5.

Subclasse de *enum.IntEnum* que define um conjunto de códigos de status HTTP, frases de razão e descrições longas escritas em inglês.

Uso:

```
>>> from http import HTTPStatus
>>> HTTPStatus.OK
HTTPStatus.OK
>>> HTTPStatus.OK == 200
True
>>> HTTPStatus.OK.value
200
>>> HTTPStatus.OK.phrase
'OK'
>>> HTTPStatus.OK.description
'Request fulfilled, document follows'
>>> list(HTTPStatus)
[HTTPStatus.CONTINUE, HTTPStatus.SWITCHING_PROTOCOLS, ...]
```

22.9.1 códigos de status HTTP

Supported, IANA-registered status codes available in *http.HTTPStatus* are:

Código	Nome da Enumeração	Detalhes
100	CONTINUE	HTTP Semantics **RFC 9110**, Section 15.2.1
101	SWITCHING_PROTOCOLS	HTTP Semantics **RFC 9110**, Section 15.2.2
102	PROCESSING	WebDAV **RFC 2518**, Seção 10.1
103	EARLY_HINTS	Um código de status HTTP para indicar dicas **RFC 8297**
200	OK	HTTP Semantics **RFC 9110**, Section 15.3.1
201	CREATED	HTTP Semantics **RFC 9110**, Section 15.3.2
202	ACCEPTED	HTTP Semantics **RFC 9110**, Section 15.3.3
203	NON_AUTHORITATIVE_INFORMATION	HTTP Semantics **RFC 9110**, Section 15.3.4
204	NO_CONTENT	HTTP Semantics **RFC 9110**, Section 15.3.5
205	RESET_CONTENT	HTTP Semantics **RFC 9110**, Section 15.3.6
206	PARTIAL_CONTENT	HTTP Semantics **RFC 9110**, Section 15.3.7
207	MULTI_STATUS	WebDAV **RFC 4918**, Seção 11.1
208	ALREADY_REPORTED	Extensões Vinculadas WebDAV **RFC 5842**, Seção 7.1 (Experimental)
226	IM_USED	Codificador Delta em HTTP **RFC 3229**, Seção 10.4.1
300	MULTIPLE_CHOICES	HTTP Semantics **RFC 9110**, Section 15.4.1
301	MOVED_PERMANENTLY	HTTP Semantics **RFC 9110**, Section 15.4.2
302	FOUND	HTTP Semantics **RFC 9110**, Section 15.4.3
303	SEE_OTHER	HTTP Semantics **RFC 9110**, Section 15.4.4
304	NOT_MODIFIED	HTTP Semantics **RFC 9110**, Section 15.4.5
305	USE_PROXY	HTTP Semantics **RFC 9110**, Section 15.4.6
307	TEMPORARY_REDIRECT	HTTP Semantics **RFC 9110**, Section 15.4.8
308	PERMANENT_REDIRECT	HTTP Semantics **RFC 9110**, Section 15.4.9
400	BAD_REQUEST	HTTP Semantics **RFC 9110**, Section 15.5.1
401	UNAUTHORIZED	HTTP Semantics **RFC 9110**, Section 15.5.2
402	PAYMENT_REQUIRED	HTTP Semantics **RFC 9110**, Section 15.5.3
403	FORBIDDEN	HTTP Semantics **RFC 9110**, Section 15.5.4
404	NOT_FOUND	HTTP Semantics **RFC 9110**, Section 15.5.5
405	METHOD_NOT_ALLOWED	HTTP Semantics **RFC 9110**, Section 15.5.6
406	NOT_ACCEPTABLE	HTTP Semantics **RFC 9110**, Section 15.5.7
407	PROXY_AUTHENTICATION_REQUIRED	HTTP Semantics **RFC 9110**, Section 15.5.8

continua na próx

Tabela 1 – continuação da página anterior

Código	Nome da Enumeração	Detalhes
408	REQUEST_TIMEOUT	HTTP Semantics **RFC 9110**, Section 15.5.9
409	CONFLICT	HTTP Semantics **RFC 9110**, Section 15.5.10
410	GONE	HTTP Semantics **RFC 9110**, Section 15.5.11
411	LENGTH_REQUIRED	HTTP Semantics **RFC 9110**, Section 15.5.12
412	PRECONDITION_FAILED	HTTP Semantics **RFC 9110**, Section 15.5.13
413	CONTENT_TOO_LARGE	HTTP Semantics **RFC 9110**, Section 15.5.14
414	URI_TOO_LONG	HTTP Semantics **RFC 9110**, Section 15.5.15
415	UNSUPPORTED_MEDIA_TYPE	HTTP Semantics **RFC 9110**, Section 15.5.16
416	RANGE_NOT_SATISFIABLE	HTTP Semantics **RFC 9110**, Section 15.5.17
417	EXPECTATION_FAILED	HTTP Semantics **RFC 9110**, Section 15.5.18
418	IM_A_TEAPOT	HTCPCP/1.0 **RFC 2324**, Seção 2.3.2
421	MISDIRECTED_REQUEST	HTTP Semantics **RFC 9110**, Section 15.5.20
422	UNPROCESSABLE_CONTENT	HTTP Semantics **RFC 9110**, Section 15.5.21
423	LOCKED	WebDAV **RFC 4918**, Seção 11.3
424	FAILED_DEPENDENCY	WebDAV **RFC 4918**, Seção 11.4
425	TOO_EARLY	Usando dados antecipados em HTTP **RFC 8470**
426	UPGRADE_REQUIRED	HTTP Semantics **RFC 9110**, Section 15.5.22
428	PRECONDITION_REQUIRED	Códigos de Status HTTP Adicionais **RFC 6585**
429	TOO_MANY_REQUESTS	Códigos de Status HTTP Adicionais **RFC 6585**
431	REQUEST_HEADER_FIELDS_TOO_LARGE	Códigos de Status HTTP Adicionais **RFC 6585**
451	UNAVAILABLE_FOR_LEGAL_REASONS	Um Código de Status HTTP para Relatar Obstáculos Legais **RFC 7725**
500	INTERNAL_SERVER_ERROR	HTTP Semantics **RFC 9110**, Section 15.6.1
501	NOT_IMPLEMENTED	HTTP Semantics **RFC 9110**, Section 15.6.2
502	BAD_GATEWAY	HTTP Semantics **RFC 9110**, Section 15.6.3
503	SERVICE_UNAVAILABLE	HTTP Semantics **RFC 9110**, Section 15.6.4
504	GATEWAY_TIMEOUT	HTTP Semantics **RFC 9110**, Section 15.6.5
505	HTTP_VERSION_NOT_SUPPORTED	HTTP Semantics **RFC 9110**, Section 15.6.6
506	VARIANT_ALSO_NEGOTIATES	Negociação Transparente de Conteúdo em HTTP **RFC 2295**, Seção 8.1 (Ex
507	INSUFFICIENT_STORAGE	WebDAV **RFC 4918**, Seção 11.5
508	LOOP_DETECTED	Extensões de Ligação WebDAV **RFC 5842**, Seção 7.2 (Experimental)
510	NOT_EXTENDED	Um Framework de Extensão HTTP **RFC 2774**, Seção 7 (Experimental)
511	NETWORK_AUTHENTICATION_REQUIRED	Códigos de Status HTTP Adicionais **RFC 6585**, Seção 6

Para preservar compatibilidade anterior, valores de enumerações também estão presentes no módulo *http.client* na forma de constantes. O nome da enumeração é igual ao nome da constante (i.e. `http.HTTPStatus.OK` também está disponível como `http.client.OK`).

Alterado na versão 3.7: Código de status `421 MISDIRECTED_REQUEST` adicionado.

Adicionado na versão 3.8: Código de status `451 UNAVAILABLE_FOR_LEGAL_REASONS` adicionado.

Adicionado na versão 3.9: Adicionados os códigos de status `103 EARLY_HINTS`, `418 IM_A_TEAPOT` e `425 TOO_EARLY`.

Alterado na versão 3.13: Foi implementada a RFC9110 referente a nomeação de constantes de status. Nomes antigos de constantes foram preservados para manter a retrocompatibilidade.

22.9.2 Categorias de HTTP status

Adicionado na versão 3.12.

Os valores enum possuem várias propriedades que indicam a categoria do status HTTP.

Propriedade	Indica que	Detalhes
`is_informational`	`100 <= status <= 199`	HTTP Semantics **RFC 9110**, Section 15
`is_success`	`200 <= status <= 299`	HTTP Semantics **RFC 9110**, Section 15
`is_redirection`	`300 <= status <= 399`	HTTP Semantics **RFC 9110**, Section 15
`is_client_error`	`400 <= status <= 499`	HTTP Semantics **RFC 9110**, Section 15
`is_server_error`	`500 <= status <= 599`	HTTP Semantics **RFC 9110**, Section 15

Uso:

```
>>> from http import HTTPStatus
>>> HTTPStatus.OK.is_success
True
>>> HTTPStatus.OK.is_client_error
False
```

class http.**HTTPMethod**

Adicionado na versão 3.11.

A subclass of *enum.StrEnum* that defines a set of HTTP methods and descriptions written in English.

Uso:

```
>>> from http import HTTPMethod
>>>
>>> HTTPMethod.GET
<HTTPMethod.GET>
>>> HTTPMethod.GET == 'GET'
True
>>> HTTPMethod.GET.value
'GET'
>>> HTTPMethod.GET.description
'Retrieve the target.'
>>> list(HTTPMethod)
[<HTTPMethod.CONNECT>,
 <HTTPMethod.DELETE>,
 <HTTPMethod.GET>,
 <HTTPMethod.HEAD>,
 <HTTPMethod.OPTIONS>,
 <HTTPMethod.PATCH>,
 <HTTPMethod.POST>,
 <HTTPMethod.PUT>,
 <HTTPMethod.TRACE>]
```

22.9.3 Métodos HTTP

Supported, IANA-registered methods available in *http.HTTPMethod* are:

Método	Nome da Enumeração	Detalhes
`GET`	`GET`	HTTP Semantics **RFC 9110**, Section 9.3.1
`HEAD`	`HEAD`	HTTP Semantics **RFC 9110**, Section 9.3.2
`POST`	`POST`	HTTP Semantics **RFC 9110**, Section 9.3.3
`PUT`	`PUT`	HTTP Semantics **RFC 9110**, Section 9.3.4
`DELETE`	`DELETE`	HTTP Semantics **RFC 9110**, Section 9.3.5
`CONNECT`	`CONNECT`	HTTP Semantics **RFC 9110**, Section 9.3.6
`OPTIONS`	`OPTIONS`	HTTP Semantics **RFC 9110**, Section 9.3.7
`TRACE`	`TRACE`	HTTP Semantics **RFC 9110**, Section 9.3.8
`PATCH`	`PATCH`	HTTP/1.1 **RFC 5789**

22.10 `http.client` — HTTP protocol client

Código-fonte: Lib/http/client.py

This module defines classes that implement the client side of the HTTP and HTTPS protocols. It is normally not used directly — the module `urllib.request` uses it to handle URLs that use HTTP and HTTPS.

Ver também

The Requests package is recommended for a higher-level HTTP client interface.

Nota

Suporte HTTPS está disponível somente se Python foi compilado com suporte SSL (através do módulo `ssl`).

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

O módulo fornece as seguintes classes:

class http.client.**HTTPConnection**(*host*, *port=None*, [*timeout*,]*source_address=None*, *blocksize=8192*)

An `HTTPConnection` instance represents one transaction with an HTTP server. It should be instantiated by passing it a host and optional port number. If no port number is passed, the port is extracted from the host string if it has the form `host:port`, else the default HTTP port (80) is used. If the optional *timeout* parameter is given, blocking operations (like connection attempts) will timeout after that many seconds (if it is not given, the global default timeout setting is used). The optional *source_address* parameter may be a tuple of a (host, port) to use as the source address the HTTP connection is made from. The optional *blocksize* parameter sets the buffer size in bytes for sending a file-like message body.

Por exemplo, todas as seguintes chamadas criam instâncias que conectam ao servidor com o mesmo host e porta:

```
>>> h1 = http.client.HTTPConnection('www.python.org')
>>> h2 = http.client.HTTPConnection('www.python.org:80')
>>> h3 = http.client.HTTPConnection('www.python.org', 80)
>>> h4 = http.client.HTTPConnection('www.python.org', 80, timeout=10)
```

Alterado na versão 3.2: *source_address* foi adicionado.

Alterado na versão 3.4: The *strict* parameter was removed. HTTP 0.9-style “Simple Responses” are no longer supported.

Alterado na versão 3.7: O argumento *blocksize* foi adicionado.

class http.client.**HTTPSConnection**(*host*, *port=None*, *, [*timeout*,]*source_address=None*, *context=None*, *blocksize=8192*)

Uma subclasse de `HTTPConnection` que utiliza SSL para comunicação com servidores seguros. A porta padrão é `443`. Se *context* for especificado, ele deve ser uma instância de `ssl.SSLContext` descrevendo as várias opções do SSL.

Por favor leia *Considerações de segurança* para mais informações sobre as melhores práticas.

Alterado na versão 3.2: *source_address*, *context* e *check_hostname* foram adicionados.

Alterado na versão 3.2: This class now supports HTTPS virtual hosts if possible (that is, if `ssl.HAS_SNI` is true).

Alterado na versão 3.4: O argumento *strict* foi removido. "Respostas Simples" HTTP com o estilo 0.9 não são mais suportadas.

Alterado na versão 3.4.3: This class now performs all the necessary certificate and hostname checks by default. To revert to the previous, unverified, behavior `ssl._create_unverified_context()` can be passed to the *context* parameter.

Alterado na versão 3.8: Esta classe agora habilita TLS 1.3 *ssl.SSLContext.post_handshake_auth* para o padrão *context* ou quanto *cert_file* é fornecido com um *context* personalizado.

Alterado na versão 3.10: This class now sends an ALPN extension with protocol indicator `http/1.1` when no *context* is given. Custom *context* should set ALPN protocols with *set_alpn_protocols()*.

Alterado na versão 3.12: The deprecated *key_file*, *cert_file* and *check_hostname* parameters have been removed.

class `http.client.`**HTTPResponse**(*sock*, *debuglevel=0*, *method=None*, *url=None*)

Classe em que instâncias são retornadas mediante de conexão bem-sucedida. Não é instanciável diretamente pelo usuário.

Alterado na versão 3.4: O argumento *strict* foi removido. "Respostas Simples" HTTP com o estilo 0.9 não são mais suportadas.

Este módulo fornece a seguinte função:

`http.client.`**parse_headers**(*fp*)

Parse the headers from a file pointer *fp* representing a HTTP request/response. The file has to be a *BufferedIOBase* reader (i.e. not text) and must provide a valid **RFC 2822** style header.

Esta função retorna uma instância de *http.client.HTTPMessage* que armazena os campos do cabeçalho, mas não o payload (o mesmo que *HTTPResponse.msg* e *http.server.BaseHTTPRequestHandler.headers*). Depois de retornar, o ponteiro de arquivo *fp* está pronto para ler o corpo da requisição HTTP.

> **Nota**
>
> *parse_headers()* não analisa a linha inicial de uma mensagem HTTP; ele apenas analisa as linhas de `Name: value`. O arquivo deve estar pronto para ler essas linhas de campo, então a primeira linha já deve ter sido consumida antes de chamar a função.

As seguintes exceções são levantadas conforme apropriado:

exception `http.client.`**HTTPException**

A classe base das outras exceções neste módulo. É uma subclasse de *Exception*.

exception `http.client.`**NotConnected**

Uma subclasse de *HTTPException*.

exception `http.client.`**InvalidURL**

Uma subclasse de *HTTPException*, levantada se uma porta é fornecida e esta é não-numérica ou vazia.

exception `http.client.`**UnknownProtocol**

Uma subclasse de *HTTPException*.

exception `http.client.`**UnknownTransferEncoding**

Uma subclasse de *HTTPException*.

exception `http.client.`**UnimplementedFileMode**

Uma subclasse de *HTTPException*.

exception `http.client.`**IncompleteRead**

Uma subclasse de *HTTPException*.

exception http.client.**ImproperConnectionState**

Uma subclasse de *HTTPException*.

exception http.client.**CannotSendRequest**

Uma subclasse de *ImproperConnectionState*.

exception http.client.**CannotSendHeader**

Uma subclasse de *ImproperConnectionState*.

exception http.client.**ResponseNotReady**

Uma subclasse de *ImproperConnectionState*.

exception http.client.**BadStatusLine**

Uma subclasse de *HTTPException*. Levantada se um servidor responde com um código de status HTTP que não é entendido.

exception http.client.**LineTooLong**

Uma subclasse de *HTTPException*. Levantada se uma linha excessivamente longa é recebida, do servidor, no protocolo HTTP.

exception http.client.**RemoteDisconnected**

Uma subclasse de *ConnectionResetError* e *BadStatusLine*. Levantada por *HTTPConnection.getresponse()* quando a tentativa de ler a resposta resulta na não leitura dos dados pela conexão, indicando que o fim remoto fechou a conexão.

Adicionado na versão 3.5: Anteriormente, a exceção *BadStatusLine*`('')` foi levantada.

As constantes definidas neste módulo são:

http.client.**HTTP_PORT**

A porta padrão para o protocolo HTTP (sempre `80`).

http.client.**HTTPS_PORT**

A porta padrão para o protocolo HTTPS (sempre `443`).

http.client.**responses**

Este dicionário mapeia os códigos de status HTTP 1.1 para os nomes em W3C.

Exemplo: `http.client.responses[http.client.NOT_FOUND]` é `'Not Found'`.

Ver *códigos de status HTTP* para uma lista de códigos de status HTTP que estão disponíveis neste módulo como constantes.

22.10.1 Objetos de HTTPConnection

Instâncias *HTTPConnection* contêm os seguintes métodos:

HTTPConnection.**request**(*method*, *url*, *body=None*, *headers={}*, ***, *encode_chunked=False*)

This will send a request to the server using the HTTP request method *method* and the request URI *url*. The provided *url* must be an absolute path to conform with **RFC 2616 §5.1.2** (unless connecting to an HTTP proxy server or using the `OPTIONS` or `CONNECT` methods).

Se *body* é especificado, os dados especificados são mandados depois que os cabeçalhos estão prontos. Pode ser um *str*, um *objeto bytes ou similar*, um *objeto arquivo* aberto, ou um iterável de *bytes*. Se *body* é uma string, ele é codificado como ISO-8859-1, o padrão para HTTP. Se é um objeto bytes ou similar, os bytes são enviados como estão. Se é um *objeto arquivo*, o conteúdo do arquivo é enviado; este objeto arquivo deve oferecer suporte a pelo menos o método `read()`. Se o objeto arquivo é uma instância de *io.TextIOBase*, os dados retornados pelo método `read()` será codificado como ISO-8859-1, de outra forma os dados retornados por `read()` são enviados como estão. Se *body* é um iterável, os elementos do iterável são enviados até o iterável se esgotar.

The *headers* argument should be a mapping of extra HTTP headers to send with the request. A **Host header** must be provided to conform with **RFC 2616 §5.1.2** (unless connecting to an HTTP proxy server or using the `OPTIONS` or `CONNECT` methods).

If *headers* contains neither Content-Length nor Transfer-Encoding, but there is a request body, one of those header fields will be added automatically. If *body* is `None`, the Content-Length header is set to `0` for methods that expect a body (`PUT`, `POST`, and `PATCH`). If *body* is a string or a bytes-like object that is not also a *file*, the Content-Length header is set to its length. Any other type of *body* (files and iterables in general) will be chunk-encoded, and the Transfer-Encoding header will automatically be set instead of Content-Length.

The *encode_chunked* argument is only relevant if Transfer-Encoding is specified in *headers*. If *encode_chunked* is `False`, the HTTPConnection object assumes that all encoding is handled by the calling code. If it is `True`, the body will be chunk-encoded.

For example, to perform a `GET` request to `https://docs.python.org/3/`:

```
>>> import http.client
>>> host = "docs.python.org"
>>> conn = http.client.HTTPSConnection(host)
>>> conn.request("GET", "/3/", headers={"Host": host})
>>> response = conn.getresponse()
>>> print(response.status, response.reason)
200 OK
```

Nota

Chunked transfer encoding has been added to the HTTP protocol version 1.1. Unless the HTTP server is known to handle HTTP 1.1, the caller must either specify the Content-Length, or must pass a *str* or bytes-like object that is not also a file as the body representation.

Alterado na versão 3.2: *body* pode agora ser um iterável.

Alterado na versão 3.6: If neither Content-Length nor Transfer-Encoding are set in *headers*, file and iterable *body* objects are now chunk-encoded. The *encode_chunked* argument was added. No attempt is made to determine the Content-Length for file objects.

`HTTPConnection.`**`getresponse`**`()`

Should be called after a request is sent to get the response from the server. Returns an *HTTPResponse* instance.

Nota

Note that you must have read the whole response before you can send a new request to the server.

Alterado na versão 3.5: Se uma *ConnectionError* ou subclasse for levantada, o objeto *HTTPConnection* estará pronto para se reconectar quando uma nova solicitação for enviada.

`HTTPConnection.`**`set_debuglevel`**`(`*level*`)`

Set the debugging level. The default debug level is `0`, meaning no debugging output is printed. Any value greater than `0` will cause all currently defined debug output to be printed to stdout. The `debuglevel` is passed to any new *HTTPResponse* objects that are created.

Adicionado na versão 3.1.

`HTTPConnection.`**`set_tunnel`**`(`*host*, *port=None*, *headers=None*`)`

Set the host and the port for HTTP Connect Tunnelling. This allows running the connection through a proxy server.

The *host* and *port* arguments specify the endpoint of the tunneled connection (i.e. the address included in the CONNECT request, *not* the address of the proxy server).

The *headers* argument should be a mapping of extra HTTP headers to send with the CONNECT request.

As HTTP/1.1 is used for HTTP CONNECT tunnelling request, as per the RFC, a HTTP `Host:` header must be provided, matching the authority-form of the request target provided as the destination for the CONNECT

request. If a HTTP `Host:` header is not provided via the headers argument, one is generated and transmitted automatically.

For example, to tunnel through a HTTPS proxy server running locally on port 8080, we would pass the address of the proxy to the *`HTTPSConnection`* constructor, and the address of the host that we eventually want to reach to the *`set_tunnel()`* method:

```
>>> import http.client
>>> conn = http.client.HTTPSConnection("localhost", 8080)
>>> conn.set_tunnel("www.python.org")
>>> conn.request("HEAD","/index.html")
```

Adicionado na versão 3.2.

Alterado na versão 3.12: HTTP CONNECT tunnelling requests use protocol HTTP/1.1, upgraded from protocol HTTP/1.0. `Host:` HTTP headers are mandatory for HTTP/1.1, so one will be automatically generated and transmitted if not provided in the headers argument.

`HTTPConnection.`**`get_proxy_response_headers`**`()`

Returns a dictionary with the headers of the response received from the proxy server to the CONNECT request.

If the CONNECT request was not sent, the method returns `None`.

Adicionado na versão 3.12.

`HTTPConnection.`**`connect`**`()`

Connect to the server specified when the object was created. By default, this is called automatically when making a request if the client does not already have a connection.

Levanta um *evento de auditoria* `http.client.connect` com os argumentos `self`, `host`, `port`.

`HTTPConnection.`**`close`**`()`

Close the connection to the server.

`HTTPConnection.`**`blocksize`**

Buffer size in bytes for sending a file-like message body.

Adicionado na versão 3.7.

As an alternative to using the *`request()`* method described above, you can also send your request step by step, by using the four functions below.

`HTTPConnection.`**`putrequest`**`(`*method*, *url*, *skip_host=False*, *skip_accept_encoding=False*`)`

This should be the first call after the connection to the server has been made. It sends a line to the server consisting of the *method* string, the *url* string, and the HTTP version (`HTTP/1.1`). To disable automatic sending of `Host:` or `Accept-Encoding:` headers (for example to accept additional content encodings), specify *skip_host* or *skip_accept_encoding* with non-False values.

`HTTPConnection.`**`putheader`**`(`*header*, *argument*`[`, ... `]`)

Send an **RFC 822**-style header to the server. It sends a line to the server consisting of the header, a colon and a space, and the first argument. If more arguments are given, continuation lines are sent, each consisting of a tab and an argument.

`HTTPConnection.`**`endheaders`**`(`*message_body=None*, *, *encode_chunked=False*`)`

Send a blank line to the server, signalling the end of the headers. The optional *message_body* argument can be used to pass a message body associated with the request.

If *encode_chunked* is `True`, the result of each iteration of *message_body* will be chunk-encoded as specified in **RFC 7230**, Section 3.3.1. How the data is encoded is dependent on the type of *message_body*. If *message_body* implements the buffer interface the encoding will result in a single chunk. If *message_body* is a *`collections.abc.Iterable`*, each iteration of *message_body* will result in a chunk. If *message_body* is a *file object*, each call to `.read()` will result in a chunk. The method automatically signals the end of the chunk-encoded data immediately after *message_body*.

> **Nota**
>
> Due to the chunked encoding specification, empty chunks yielded by an iterator body will be ignored by the chunk-encoder. This is to avoid premature termination of the read of the request by the target server due to malformed encoding.

Alterado na versão 3.6: Added chunked encoding support and the *encode_chunked* parameter.

`HTTPConnection.`**`send`**`(`*data*`)`

Send data to the server. This should be used directly only after the `endheaders()` method has been called and before `getresponse()` is called.

Levanta um *evento de auditoria* `http.client.send` com os argumentos `self`, `data`.

22.10.2 Objetos HTTPResponse

An `HTTPResponse` instance wraps the HTTP response from the server. It provides access to the request headers and the entity body. The response is an iterable object and can be used in a with statement.

Alterado na versão 3.5: The `io.BufferedIOBase` interface is now implemented and all of its reader operations are supported.

`HTTPResponse.`**`read`**`(`[*amt*]`)`

Reads and returns the response body, or up to the next *amt* bytes.

`HTTPResponse.`**`readinto`**`(`*b*`)`

Reads up to the next len(b) bytes of the response body into the buffer *b*. Returns the number of bytes read.

Adicionado na versão 3.3.

`HTTPResponse.`**`getheader`**`(`*name*, *default=None*`)`

Return the value of the header *name*, or *default* if there is no header matching *name*. If there is more than one header with the name *name*, return all of the values joined by ', '. If *default* is any iterable other than a single string, its elements are similarly returned joined by commas.

`HTTPResponse.`**`getheaders`**`()`

Return a list of (header, value) tuples.

`HTTPResponse.`**`fileno`**`()`

Return the `fileno` of the underlying socket.

`HTTPResponse.`**`msg`**

A `http.client.HTTPMessage` instance containing the response headers. `http.client.HTTPMessage` is a subclass of `email.message.Message`.

`HTTPResponse.`**`version`**

HTTP protocol version used by server. 10 for HTTP/1.0, 11 for HTTP/1.1.

`HTTPResponse.`**`url`**

URL of the resource retrieved, commonly used to determine if a redirect was followed.

`HTTPResponse.`**`headers`**

Headers of the response in the form of an `email.message.EmailMessage` instance.

`HTTPResponse.`**`status`**

Status code returned by server.

`HTTPResponse.`**`reason`**

Reason phrase returned by server.

HTTPResponse.**debuglevel**

A debugging hook. If *debuglevel* is greater than zero, messages will be printed to stdout as the response is read and parsed.

HTTPResponse.**closed**

Is True if the stream is closed.

HTTPResponse.**geturl**()

Descontinuado desde a versão 3.9: Deprecated in favor of *url*.

HTTPResponse.**info**()

Descontinuado desde a versão 3.9: Deprecated in favor of *headers*.

HTTPResponse.**getcode**()

Descontinuado desde a versão 3.9: Deprecated in favor of *status*.

22.10.3 Exemplos

Here is an example session that uses the GET method:

```
>>> import http.client
>>> conn = http.client.HTTPSConnection("www.python.org")
>>> conn.request("GET", "/")
>>> r1 = conn.getresponse()
>>> print(r1.status, r1.reason)
200 OK
>>> data1 = r1.read()  # This will return entire content.
>>> # The following example demonstrates reading data in chunks.
>>> conn.request("GET", "/")
>>> r1 = conn.getresponse()
>>> while chunk := r1.read(200):
...     print(repr(chunk))
b'<!doctype html>\n<!--[if"...
...
>>> # Example of an invalid request
>>> conn = http.client.HTTPSConnection("docs.python.org")
>>> conn.request("GET", "/parrot.spam")
>>> r2 = conn.getresponse()
>>> print(r2.status, r2.reason)
404 Not Found
>>> data2 = r2.read()
>>> conn.close()
```

Here is an example session that uses the HEAD method. Note that the HEAD method never returns any data.

```
>>> import http.client
>>> conn = http.client.HTTPSConnection("www.python.org")
>>> conn.request("HEAD", "/")
>>> res = conn.getresponse()
>>> print(res.status, res.reason)
200 OK
>>> data = res.read()
>>> print(len(data))
0
>>> data == b''
True
```

Here is an example session that uses the POST method:

```
>>> import http.client, urllib.parse
>>> params = urllib.parse.urlencode({'@number': 12524, '@type': 'issue', '@action
↪': 'show'})
>>> headers = {"Content-type": "application/x-www-form-urlencoded",
...            "Accept": "text/plain"}
>>> conn = http.client.HTTPConnection("bugs.python.org")
>>> conn.request("POST", "", params, headers)
>>> response = conn.getresponse()
>>> print(response.status, response.reason)
302 Found
>>> data = response.read()
>>> data
b'Redirecting to <a href="https://bugs.python.org/issue12524">https://bugs.python.
↪org/issue12524</a>'
>>> conn.close()
```

Client side HTTP `PUT` requests are very similar to `POST` requests. The difference lies only on the server side where HTTP servers will allow resources to be created via `PUT` requests. It should be noted that custom HTTP methods are also handled in *`urllib.request.Request`* by setting the appropriate method attribute. Here is an example session that uses the `PUT` method:

```
>>> # This creates an HTTP request
>>> # with the content of BODY as the enclosed representation
>>> # for the resource http://localhost:8080/file
...
>>> import http.client
>>> BODY = "***filecontents***"
>>> conn = http.client.HTTPConnection("localhost", 8080)
>>> conn.request("PUT", "/file", BODY)
>>> response = conn.getresponse()
>>> print(response.status, response.reason)
200, OK
```

22.10.4 HTTPMessage Objects

class http.client.**HTTPMessage**(*email.message.Message*)

An *`http.client.HTTPMessage`* instance holds the headers from an HTTP response. It is implemented using the *`email.message.Message`* class.

22.11 `ftplib` — FTP protocol client

Código-fonte: Lib/ftplib.py

This module defines the class *`FTP`* and a few related items. The *`FTP`* class implements the client side of the FTP protocol. You can use this to write Python programs that perform a variety of automated FTP jobs, such as mirroring other FTP servers. It is also used by the module *`urllib.request`* to handle URLs that use FTP. For more information on FTP (File Transfer Protocol), see internet **RFC 959**.

The default encoding is UTF-8, following **RFC 2640**.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

Here's a sample session using the *`ftplib`* module:

```
>>> from ftplib import FTP
>>> ftp = FTP('ftp.us.debian.org')  # connect to host, default port
>>> ftp.login()                     # user anonymous, passwd anonymous@
'230 Login successful.'
>>> ftp.cwd('debian')               # change into "debian" directory
'250 Directory successfully changed.'
>>> ftp.retrlines('LIST')           # list directory contents
-rw-rw-r--    1 1176     1176         1063 Jun 15 10:18 README
...
drwxr-sr-x    5 1176     1176         4096 Dec 19  2000 pool
drwxr-sr-x    4 1176     1176         4096 Nov 17  2008 project
drwxr-xr-x    3 1176     1176         4096 Oct 10  2012 tools
'226 Directory send OK.'
>>> with open('README', 'wb') as fp:
>>>     ftp.retrbinary('RETR README', fp.write)
'226 Transfer complete.'
>>> ftp.quit()
'221 Goodbye.'
```

22.11.1 Referência

FTP objects

class ftplib.**FTP**(*host=''*, *user=''*, *passwd=''*, *acct=''*, *timeout=None*, *source_address=None*, ***, *encoding='utf-8'*)

Return a new instance of the *FTP* class.

Parâmetros

- **host** (str) – The hostname to connect to. If given, `connect(host)` is implicitly called by the constructor.
- **user** (str) – The username to log in with (default: `'anonymous'`). If given, `login(host, passwd, acct)` is implicitly called by the constructor.
- **passwd** (str) – The password to use when logging in. If not given, and if *passwd* is the empty string or `"-"`, a password will be automatically generated.
- **acct** (str) – Account information to be used for the `ACCT` FTP command. Few systems implement this. See RFC-959 for more details.
- **timeout** (float | *None*) – A timeout in seconds for blocking operations like *connect()* (default: the global default timeout setting).
- **source_address** (tuple | *None*) – A 2-tuple `(host, port)` for the socket to bind to as its source address before connecting.
- **encoding** (str) – The encoding for directories and filenames (default: `'utf-8'`).

The *FTP* class supports the `with` statement, e.g.:

```
>>> from ftplib import FTP
>>> with FTP("ftp1.at.proftpd.org") as ftp:
...     ftp.login()
...     ftp.dir()
...
'230 Anonymous login ok, restrictions apply.'
dr-xr-xr-x   9 ftp      ftp           154 May  6 10:43 .
dr-xr-xr-x   9 ftp      ftp           154 May  6 10:43 ..
dr-xr-xr-x   5 ftp      ftp          4096 May  6 10:43 CentOS
dr-xr-xr-x   3 ftp      ftp            18 Jul 10  2008 Fedora
>>>
```

Alterado na versão 3.2: Suporte para a instrução `with` foi adicionado.

Alterado na versão 3.3: *source_address* parameter was added.

Alterado na versão 3.9: If the *timeout* parameter is set to be zero, it will raise a *ValueError* to prevent the creation of a non-blocking socket. The *encoding* parameter was added, and the default was changed from Latin-1 to UTF-8 to follow **RFC 2640**.

Several `FTP` methods are available in two flavors: one for handling text files and another for binary files. The methods are named for the command which is used followed by `lines` for the text version or `binary` for the binary version.

FTP instances have the following methods:

set_debuglevel(*level*)

Set the instance's debugging level as an *int*. This controls the amount of debugging output printed. The debug levels are:

- `0` (default): No debug output.
- `1`: Produce a moderate amount of debug output, generally a single line per request.
- `2` or higher: Produce the maximum amount of debugging output, logging each line sent and received on the control connection.

connect(*host=''*, *port=0*, *timeout=None*, *source_address=None*)

Connect to the given host and port. This function should be called only once for each instance; it should not be called if a *host* argument was given when the *FTP* instance was created. All other `FTP` methods can only be called after a connection has successfully been made.

Parâmetros

- **host** (`str`) – The host to connect to.
- **port** (`int`) – The TCP port to connect to (default: `21`, as specified by the FTP protocol specification). It is rarely needed to specify a different port number.
- **timeout** (`float` | *None*) – A timeout in seconds for the connection attempt (default: the global default timeout setting).
- **source_address** (`tuple` | *None*) – A 2-tuple `(host, port)` for the socket to bind to as its source address before connecting.

Levanta um *evento de auditoria* `ftplib.connect` com os argumentos `self`, `host`, `port`.

Alterado na versão 3.3: *source_address* parameter was added.

getwelcome()

Return the welcome message sent by the server in reply to the initial connection. (This message sometimes contains disclaimers or help information that may be relevant to the user.)

login(*user='anonymous'*, *passwd=''*, *acct=''*)

Log on to the connected FTP server. This function should be called only once for each instance, after a connection has been established; it should not be called if the *host* and *user* arguments were given when the *FTP* instance was created. Most FTP commands are only allowed after the client has logged in.

Parâmetros

- **user** (`str`) – The username to log in with (default: `'anonymous'`).
- **passwd** (`str`) – The password to use when logging in. If not given, and if *passwd* is the empty string or `"-"`, a password will be automatically generated.
- **acct** (`str`) – Account information to be used for the `ACCT` FTP command. Few systems implement this. See RFC-959 for more details.

abort()

Abort a file transfer that is in progress. Using this does not always work, but it's worth a try.

sendcmd(*cmd*)

Send a simple command string to the server and return the response string.

Levanta um *evento de auditoria* `ftplib.sendcmd` com os argumentos `self`, `cmd`.

voidcmd(*cmd*)

Send a simple command string to the server and handle the response. Return the response string if the response code corresponds to success (codes in the range 200–299). Raise *error_reply* otherwise.

Levanta um *evento de auditoria* `ftplib.sendcmd` com os argumentos `self`, `cmd`.

retrbinary(*cmd*, *callback*, *blocksize=8192*, *rest=None*)

Retrieve a file in binary transfer mode.

Parâmetros

- **cmd** (`str`) – An appropriate `RETR` command: `"RETR filename"`.
- **callback** (*callable*) – A single parameter callable that is called for each block of data received, with its single argument being the data as *bytes*.
- **blocksize** (`int`) – The maximum chunk size to read on the low-level *socket* object created to do the actual transfer. This also corresponds to the largest size of data that will be passed to *callback*. Defaults to `8192`.
- **rest** (`int`) – A `REST` command to be sent to the server. See the documentation for the *rest* parameter of the *transfercmd()* method.

retrlines(*cmd*, *callback=None*)

Retrieve a file or directory listing in the encoding specified by the *encoding* parameter at initialization. *cmd* should be an appropriate `RETR` command (see *retrbinary()*) or a command such as `LIST` or `NLST` (usually just the string `'LIST'`). `LIST` retrieves a list of files and information about those files. `NLST` retrieves a list of file names. The *callback* function is called for each line with a string argument containing the line with the trailing CRLF stripped. The default *callback* prints the line to *sys.stdout*.

set_pasv(*val*)

Enable "passive" mode if *val* is true, otherwise disable passive mode. Passive mode is on by default.

storbinary(*cmd*, *fp*, *blocksize=8192*, *callback=None*, *rest=None*)

Store a file in binary transfer mode.

Parâmetros

- **cmd** (`str`) – An appropriate `STOR` command: `"STOR filename"`.
- **fp** (*file object*) – A file object (opened in binary mode) which is read until EOF, using its *read()* method in blocks of size *blocksize* to provide the data to be stored.
- **blocksize** (`int`) – The read block size. Defaults to `8192`.
- **callback** (*callable*) – A single parameter callable that is called for each block of data sent, with its single argument being the data as *bytes*.
- **rest** (`int`) – A `REST` command to be sent to the server. See the documentation for the *rest* parameter of the *transfercmd()* method.

Alterado na versão 3.2: The *rest* parameter was added.

storlines(*cmd*, *fp*, *callback=None*)

Store a file in line mode. *cmd* should be an appropriate `STOR` command (see *storbinary()*). Lines are read until EOF from the *file object* *fp* (opened in binary mode) using its *readline()* method to provide the data to be stored. *callback* is an optional single parameter callable that is called on each line after it is sent.

transfercmd(*cmd*, *rest=None*)

Initiate a transfer over the data connection. If the transfer is active, send an `EPRT` or `PORT` command and the transfer command specified by *cmd*, and accept the connection. If the server is passive, send an `EPSV` or `PASV` command, connect to it, and start the transfer command. Either way, return the socket for the connection.

If optional *rest* is given, a `REST` command is sent to the server, passing *rest* as an argument. *rest* is usually a byte offset into the requested file, telling the server to restart sending the file's bytes at the requested offset, skipping over the initial bytes. Note however that the `transfercmd()` method converts *rest* to a string with the *encoding* parameter specified at initialization, but no check is performed on the string's contents. If the server does not recognize the `REST` command, an `error_reply` exception will be raised. If this happens, simply call `transfercmd()` without a *rest* argument.

ntransfercmd(*cmd*, *rest=None*)

Like `transfercmd()`, but returns a tuple of the data connection and the expected size of the data. If the expected size could not be computed, `None` will be returned as the expected size. *cmd* and *rest* means the same thing as in `transfercmd()`.

mlsd(*path=''*, *facts=[]*)

List a directory in a standardized format by using `MLSD` command (**RFC 3659**). If *path* is omitted the current directory is assumed. *facts* is a list of strings representing the type of information desired (e.g. `["type", "size", "perm"]`). Return a generator object yielding a tuple of two elements for every file found in path. First element is the file name, the second one is a dictionary containing facts about the file name. Content of this dictionary might be limited by the *facts* argument but server is not guaranteed to return all requested facts.

Adicionado na versão 3.3.

nlst(*argument*[, ...])

Return a list of file names as returned by the `NLST` command. The optional *argument* is a directory to list (default is the current server directory). Multiple arguments can be used to pass non-standard options to the `NLST` command.

> **Nota**
>
> If your server supports the command, `mlsd()` offers a better API.

dir(*argument*[, ...])

Produce a directory listing as returned by the `LIST` command, printing it to standard output. The optional *argument* is a directory to list (default is the current server directory). Multiple arguments can be used to pass non-standard options to the `LIST` command. If the last argument is a function, it is used as a *callback* function as for `retrlines()`; the default prints to `sys.stdout`. This method returns `None`.

> **Nota**
>
> If your server supports the command, `mlsd()` offers a better API.

rename(*fromname*, *toname*)

Rename file *fromname* on the server to *toname*.

delete(*filename*)

Remove the file named *filename* from the server. If successful, returns the text of the response, otherwise raises `error_perm` on permission errors or `error_reply` on other errors.

cwd(*pathname*)

Set the current directory on the server.

mkd(*pathname*)

Create a new directory on the server.

pwd()

Return the pathname of the current directory on the server.

rmd(*dirname*)

Remove the directory named *dirname* on the server.

size(*filename*)

Request the size of the file named *filename* on the server. On success, the size of the file is returned as an integer, otherwise `None` is returned. Note that the `SIZE` command is not standardized, but is supported by many common server implementations.

quit()

Send a `QUIT` command to the server and close the connection. This is the "polite" way to close a connection, but it may raise an exception if the server responds with an error to the `QUIT` command. This implies a call to the *close()* method which renders the *FTP* instance useless for subsequent calls (see below).

close()

Close the connection unilaterally. This should not be applied to an already closed connection such as after a successful call to *quit()*. After this call the *FTP* instance should not be used any more (after a call to *close()* or *quit()* you cannot reopen the connection by issuing another *login()* method).

FTP_TLS objects

class ftplib.**FTP_TLS**(*host=''*, *user=''*, *passwd=''*, *acct=''*, ***, *context=None*, *timeout=None*, *source_address=None*, *encoding='utf-8'*)

An *FTP* subclass which adds TLS support to FTP as described in **RFC 4217**. Connect to port 21 implicitly securing the FTP control connection before authenticating.

> **Nota**
>
> The user must explicitly secure the data connection by calling the *prot_p()* method.

Parâmetros

- **host** (`str`) – The hostname to connect to. If given, `connect(host)` is implicitly called by the constructor.
- **user** (`str`) – The username to log in with (default: `'anonymous'`). If given, `login(host, passwd, acct)` is implicitly called by the constructor.
- **passwd** (`str`) – The password to use when logging in. If not given, and if *passwd* is the empty string or `"-"`, a password will be automatically generated.
- **acct** (`str`) – Account information to be used for the `ACCT` FTP command. Few systems implement this. See RFC-959 for more details.
- **context** (*ssl.SSLContext*) – An SSL context object which allows bundling SSL configuration options, certificates and private keys into a single, potentially long-lived, structure. Please read *Considerações de segurança* for best practices.
- **timeout** (`float` | *None*) – A timeout in seconds for blocking operations like *connect()* (default: the global default timeout setting).
- **source_address** (`tuple` | *None*) – A 2-tuple `(host, port)` for the socket to bind to as its source address before connecting.
- **encoding** (`str`) – The encoding for directories and filenames (default: `'utf-8'`).

Adicionado na versão 3.2.

Alterado na versão 3.3: Added the *source_address* parameter.

Alterado na versão 3.4: A classe agora oferece suporte a verificação de nome de host com *ssl.SSLContext.check_hostname* e *Indicação de nome de servidor* (veja *ssl.HAS_SNI*).

Alterado na versão 3.9: If the *timeout* parameter is set to be zero, it will raise a *ValueError* to prevent the creation of a non-blocking socket. The *encoding* parameter was added, and the default was changed from Latin-1 to UTF-8 to follow **RFC 2640**.

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

Here's a sample session using the *FTP_TLS* class:

```
>>> ftps = FTP_TLS('ftp.pureftpd.org')
>>> ftps.login()
'230 Anonymous user logged in'
>>> ftps.prot_p()
'200 Data protection level set to "private"'
>>> ftps.nlst()
['6jack', 'OpenBSD', 'antilink', 'blogbench', 'bsdcam', 'clockspeed', 'djbdns-
↪jedi', 'docs', 'eaccelerator-jedi', 'favicon.ico', 'francotone', 'fugu',
↪'ignore', 'libpuzzle', 'metalog', 'minidentd', 'misc', 'mysql-udf-global-
↪user-variables', 'php-jenkins-hash', 'php-skein-hash', 'php-webdav',
↪'phpaudit', 'phpbench', 'pincaster', 'ping', 'posto', 'pub', 'public',
↪'public_keys', 'pure-ftpd', 'qscan', 'qtc', 'sharedance', 'skycache', 'sound
↪', 'tmp', 'ucarp']
```

FTP_TLS class inherits from *FTP*, defining these additional methods and attributes:

ssl_version

The SSL version to use (defaults to *ssl.PROTOCOL_SSLv23*).

auth()

Set up a secure control connection by using TLS or SSL, depending on what is specified in the *ssl_version* attribute.

Alterado na versão 3.4: The method now supports hostname check with *ssl.SSLContext.check_hostname* and *Server Name Indication* (see *ssl.HAS_SNI*).

ccc()

Revert control channel back to plaintext. This can be useful to take advantage of firewalls that know how to handle NAT with non-secure FTP without opening fixed ports.

Adicionado na versão 3.3.

prot_p()

Set up secure data connection.

prot_c()

Set up clear text data connection.

Module variables

exception ftplib.**error_reply**

Exception raised when an unexpected reply is received from the server.

exception ftplib.**error_temp**

Exception raised when an error code signifying a temporary error (response codes in the range 400–499) is received.

exception ftplib.**error_perm**

Exception raised when an error code signifying a permanent error (response codes in the range 500–599) is received.

exception ftplib.**error_proto**

Exception raised when a reply is received from the server that does not fit the response specifications of the File Transfer Protocol, i.e. begin with a digit in the range 1–5.

ftplib.**all_errors**

The set of all exceptions (as a tuple) that methods of *FTP* instances may raise as a result of problems with the FTP connection (as opposed to programming errors made by the caller). This set includes the four exceptions listed above as well as *OSError* and *EOFError*.

Ver também

Módulo *netrc*

Parser for the `.netrc` file format. The file `.netrc` is typically used by FTP clients to load user authentication information before prompting the user.

22.12 poplib — Cliente de protocolo POP3

Source code: Lib/poplib.py

Este módulo define uma classe, *POP3*, que encapsula uma conexão com um servidor POP3 e implementa o protocolo conforme definido em **RFC 1939**. A classe *POP3* oferece suporte aos conjuntos de comandos mínimos e opcionais de **RFC 1939**. A classe *POP3* também oferece suporte ao comando `STLS` introduzido em **RFC 2595** para permitir a comunicação criptografada em uma conexão já estabelecida.

Além disso, este módulo fornece uma classe *POP3_SSL*, que fornece suporte para conexão a servidores POP3 que usam SSL como uma camada de protocolo subjacente.

Observe que o POP3, embora amplamente suportado, está obsoleto. A qualidade de implementação dos servidores POP3 varia muito, e muitos são bastante ruins. Se o seu servidor de e-mail oferece suporte a IMAP, seria melhor usar a classe *imaplib.IMAP4*, pois os servidores IMAP tendem a ter uma implementação melhor.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

O módulo *poplib* fornece duas classes:

class poplib.**POP3**(*host*, *port=POP3_PORT*[, *timeout*])

Esta classe implementa o protocolo POP3 real. A conexão é criada quando a instância é inicializada. Se *port* for omitido, a porta POP3 padrão (110) será usada. O parâmetro opcional *timeout* especifica um tempo limite em segundos para a tentativa de conexão (se não for especificado, a configuração de tempo limite padrão global será usada).

Levanta um *evento de auditoria* `ftplib.connect` com os argumentos `self`, `host`, `port`.

Todos os comandos levantarão um *evento de auditoria* `poplib.putline` com os argumentos `self` e `line`, onde `line` são os bytes prestes a serem enviados ao host remoto.

Alterado na versão 3.9: Se o parâmetro *timeout* for definido como zero, será levantada *ValueError* para impedir a criação de um soquete não bloqueante.

class poplib.**POP3_SSL**(*host*, *port=POP3_SSL_PORT*, *, *timeout=None*, *context=None*)

Esta é uma subclasse de *POP3* que se conecta ao servidor por meio de um soquete criptografado SSL. Se *port* não for especificado, 995, a porta padrão POP3 sobre SSL será usada. *timeout* funciona como no construtor *POP3*. *context* é um objeto opcional *ssl.SSLContext* que permite agrupar opções de configuração SSL,

certificados e chaves privadas em uma única estrutura (potencialmente duradoura). Consulte *Considerações de segurança* para obter as melhores práticas.

Levanta um *evento de auditoria* `ftplib.connect` com os argumentos `self`, `host`, `port`.

Todos os comandos levantarão um *evento de auditoria* `poplib.putline` com os argumentos `self` e `line`, onde `line` são os bytes prestes a serem enviados ao host remoto.

Alterado na versão 3.2: Parâmetro *context* adicionado.

Alterado na versão 3.4: A classe agora oferece suporte a verificação de nome de host com `ssl.SSLContext.check_hostname` e *Indicação de nome de servidor* (veja `ssl.HAS_SNI`).

Alterado na versão 3.9: Se o parâmetro *timeout* for definido como zero, será levantada `ValueError` para impedir a criação de um soquete não bloqueante.

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

Uma exceção é definida como um atributo do módulo `poplib`:

exception poplib.**error_proto**

Exceção levantada em caso de erros deste módulo (erros do módulo `socket` não são detectados). O motivo da exceção é passado ao construtor como uma string.

Ver também

Módulo `imaplib`
The standard Python IMAP module.

Frequently Asked Questions About Fetchmail
The FAQ for the **`fetchmail`** POP/IMAP client collects information on POP3 server variations and RFC noncompliance that may be useful if you need to write an application based on the POP protocol.

22.12.1 Objetos POP3

All POP3 commands are represented by methods of the same name, in lowercase; most return the response text sent by the server.

A `POP3` instance has the following methods:

POP3.**set_debuglevel**(*level*)

Set the instance's debugging level. This controls the amount of debugging output printed. The default, `0`, produces no debugging output. A value of `1` produces a moderate amount of debugging output, generally a single line per request. A value of `2` or higher produces the maximum amount of debugging output, logging each line sent and received on the control connection.

POP3.**getwelcome**()

Returns the greeting string sent by the POP3 server.

POP3.**capa**()

Query the server's capabilities as specified in **RFC 2449**. Returns a dictionary in the form `{'name': ['param'...]}`.

Adicionado na versão 3.4.

POP3.**user**(*username*)

Send user command, response should indicate that a password is required.

POP3.**pass_**(*password*)

Send password, response includes message count and mailbox size. Note: the mailbox on the server is locked until `quit()` is called.

POP3.**apop**(*user*, *secret*)

Use the more secure APOP authentication to log into the POP3 server.

POP3.**rpop**(*user*)

Use RPOP authentication (similar to UNIX r-commands) to log into POP3 server.

POP3.**stat**()

Get mailbox status. The result is a tuple of 2 integers: `(message count, mailbox size)`.

POP3.**list**([*which*])

Request message list, result is in the form `(response, ['mesg_num octets', ...], octets)`. If *which* is set, it is the message to list.

POP3.**retr**(*which*)

Retrieve whole message number *which*, and set its seen flag. Result is in form `(response, ['line', ...], octets)`.

POP3.**dele**(*which*)

Flag message number *which* for deletion. On most servers deletions are not actually performed until QUIT (the major exception is Eudora QPOP, which deliberately violates the RFCs by doing pending deletes on any disconnect).

POP3.**rset**()

Remove any deletion marks for the mailbox.

POP3.**noop**()

Do nothing. Might be used as a keep-alive.

POP3.**quit**()

Signoff: commit changes, unlock mailbox, drop connection.

POP3.**top**(*which*, *howmuch*)

Retrieves the message header plus *howmuch* lines of the message after the header of message number *which*. Result is in form `(response, ['line', ...], octets)`.

The POP3 TOP command this method uses, unlike the RETR command, doesn't set the message's seen flag; unfortunately, TOP is poorly specified in the RFCs and is frequently broken in off-brand servers. Test this method by hand against the POP3 servers you will use before trusting it.

POP3.**uidl**(*which=None*)

Return message digest (unique id) list. If *which* is specified, result contains the unique id for that message in the form `'response mesgnum uid`, otherwise result is list `(response, ['mesgnum uid', ...], octets)`.

POP3.**utf8**()

Try to switch to UTF-8 mode. Returns the server response if successful, raises *`error_proto`* if not. Specified in **RFC 6856**.

Adicionado na versão 3.5.

POP3.**stls**(*context=None*)

Start a TLS session on the active connection as specified in **RFC 2595**. This is only allowed before user authentication

context parameter is a *`ssl.SSLContext`* object which allows bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure. Please read *Considerações de segurança* for best practices.

This method supports hostname checking via *`ssl.SSLContext.check_hostname`* and *Server Name Indication* (see *`ssl.HAS_SNI`*).

Adicionado na versão 3.4.

Instances of *`POP3_SSL`* have no additional methods. The interface of this subclass is identical to its parent.

22.12.2 Exemplo POP3

Here is a minimal example (without error checking) that opens a mailbox and retrieves and prints all messages:

```
import getpass, poplib

M = poplib.POP3('localhost')
M.user(getpass.getuser())
M.pass_(getpass.getpass())
numMessages = len(M.list()[1])
for i in range(numMessages):
    for j in M.retr(i+1)[1]:
        print(j)
```

At the end of the module, there is a test section that contains a more extensive example of usage.

22.13 `imaplib` — IMAP4 protocol client

Código-fonte: Lib/imaplib.py

This module defines three classes, *IMAP4*, *IMAP4_SSL* and *IMAP4_stream*, which encapsulate a connection to an IMAP4 server and implement a large subset of the IMAP4rev1 client protocol as defined in **RFC 2060**. It is backward compatible with IMAP4 (**RFC 1730**) servers, but note that the `STATUS` command is not supported in IMAP4.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

Three classes are provided by the *imaplib* module, *IMAP4* is the base class:

class imaplib.**IMAP4**(*host=''*, *port=IMAP4_PORT*, *timeout=None*)

This class implements the actual IMAP4 protocol. The connection is created and protocol version (IMAP4 or IMAP4rev1) is determined when the instance is initialized. If *host* is not specified, `''` (the local host) is used. If *port* is omitted, the standard IMAP4 port (143) is used. The optional *timeout* parameter specifies a timeout in seconds for the connection attempt. If timeout is not given or is `None`, the global default socket timeout is used.

The *IMAP4* class supports the `with` statement. When used like this, the IMAP4 `LOGOUT` command is issued automatically when the `with` statement exits. E.g.:

```
>>> from imaplib import IMAP4
>>> with IMAP4("domain.org") as M:
...     M.noop()
...
('OK', [b'Nothing Accomplished. d25if65hy903weo.87'])
```

Alterado na versão 3.5: Suporte para a instrução `with` foi adicionado.

Alterado na versão 3.9: O parâmetro opcional *timeout* foi adicionado.

Three exceptions are defined as attributes of the *IMAP4* class:

exception IMAP4.**error**

Exception raised on any errors. The reason for the exception is passed to the constructor as a string.

exception IMAP4.**abort**

IMAP4 server errors cause this exception to be raised. This is a sub-class of *IMAP4.error*. Note that closing the instance and instantiating a new one will usually allow recovery from this exception.

exception IMAP4.**readonly**

This exception is raised when a writable mailbox has its status changed by the server. This is a sub-class of *IMAP4.error*. Some other client now has write permission, and the mailbox will need to be re-opened to re-obtain write permission.

There's also a subclass for secure connections:

class imaplib.**IMAP4_SSL**(*host=''*, *port=IMAP4_SSL_PORT*, ***, *ssl_context=None*, *timeout=None*)

This is a subclass derived from *IMAP4* that connects over an SSL encrypted socket (to use this class you need a socket module that was compiled with SSL support). If *host* is not specified, `''` (the local host) is used. If *port* is omitted, the standard IMAP4-over-SSL port (993) is used. *ssl_context* is a *ssl.SSLContext* object which allows bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure. Please read *Considerações de segurança* for best practices.

The optional *timeout* parameter specifies a timeout in seconds for the connection attempt. If timeout is not given or is `None`, the global default socket timeout is used.

Alterado na versão 3.3: *ssl_context* parameter was added.

Alterado na versão 3.4: A classe agora oferece suporte a verificação de nome de host com *ssl.SSLContext.check_hostname* e *Indicação de nome de servidor* (veja *ssl.HAS_SNI*).

Alterado na versão 3.9: O parâmetro opcional *timeout* foi adicionado.

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

The second subclass allows for connections created by a child process:

class imaplib.**IMAP4_stream**(*command*)

This is a subclass derived from *IMAP4* that connects to the `stdin/stdout` file descriptors created by passing *command* to `subprocess.Popen()`.

The following utility functions are defined:

imaplib.**Internaldate2tuple**(*datestr*)

Parse an IMAP4 `INTERNALDATE` string and return corresponding local time. The return value is a *time.struct_time* tuple or `None` if the string has wrong format.

imaplib.**Int2AP**(*num*)

Converts an integer into a bytes representation using characters from the set [`A` .. `P`].

imaplib.**ParseFlags**(*flagstr*)

Converts an IMAP4 `FLAGS` response to a tuple of individual flags.

imaplib.**Time2Internaldate**(*date_time*)

Convert *date_time* to an IMAP4 `INTERNALDATE` representation. The return value is a string in the form: `"DD-Mmm-YYYY HH:MM:SS +HHMM"` (including double-quotes). The *date_time* argument can be a number (int or float) representing seconds since epoch (as returned by *time.time()*), a 9-tuple representing local time an instance of *time.struct_time* (as returned by *time.localtime()*), an aware instance of *datetime.datetime*, or a double-quoted string. In the last case, it is assumed to already be in the correct format.

Note that IMAP4 message numbers change as the mailbox changes; in particular, after an `EXPUNGE` command performs deletions the remaining messages are renumbered. So it is highly advisable to use UIDs instead, with the UID command.

At the end of the module, there is a test section that contains a more extensive example of usage.

Ver também

Documents describing the protocol, sources for servers implementing it, by the University of Washington's IMAP Information Center can all be found at (**Source Code**) https://github.com/uw-imap/imap (**Not Maintained**).

22.13.1 Objetos IMAP4

All IMAP4rev1 commands are represented by methods of the same name, either uppercase or lowercase.

All arguments to commands are converted to strings, except for `AUTHENTICATE`, and the last argument to `APPEND` which is passed as an IMAP4 literal. If necessary (the string contains IMAP4 protocol-sensitive characters and isn't enclosed with either parentheses or double quotes) each string is quoted. However, the *password* argument to the `LOGIN` command is always quoted. If you want to avoid having an argument string quoted (eg: the *flags* argument to `STORE`) then enclose the string in parentheses (eg: `r'(\Deleted)'`).

Most commands return a tuple: `(type, [data, ...])` where *type* is usually `'OK'` or `'NO'`, and *data* is either the text from the command response, or mandated results from the command. Each *data* is either a `bytes`, or a tuple. If a tuple, then the first part is the header of the response, and the second part contains the data (ie: 'literal' value).

The *message_set* options to commands below is a string specifying one or more messages to be acted upon. It may be a simple message number (`'1'`), a range of message numbers (`'2:4'`), or a group of non-contiguous ranges separated by commas (`'1:3,6:9'`). A range can contain an asterisk to indicate an infinite upper bound (`'3:*'`).

An *IMAP4* instance has the following methods:

IMAP4.**append**(*mailbox*, *flags*, *date_time*, *message*)

Append *message* to named mailbox.

IMAP4.**authenticate**(*mechanism*, *authobject*)

Authenticate command — requires response processing.

mechanism specifies which authentication mechanism is to be used - it should appear in the instance variable `capabilities` in the form `AUTH=mechanism`.

authobject must be a callable object:

```
data = authobject(response)
```

It will be called to process server continuation responses; the *response* argument it is passed will be `bytes`. It should return `bytes` *data* that will be base64 encoded and sent to the server. It should return `None` if the client abort response `*` should be sent instead.

Alterado na versão 3.5: string usernames and passwords are now encoded to `utf-8` instead of being limited to ASCII.

IMAP4.**check**()

Checkpoint mailbox on server.

IMAP4.**close**()

Close currently selected mailbox. Deleted messages are removed from writable mailbox. This is the recommended command before `LOGOUT`.

IMAP4.**copy**(*message_set*, *new_mailbox*)

Copy *message_set* messages onto end of *new_mailbox*.

IMAP4.**create**(*mailbox*)

Create new mailbox named *mailbox*.

IMAP4.**delete**(*mailbox*)

Delete old mailbox named *mailbox*.

IMAP4.**deleteacl**(*mailbox*, *who*)

Delete the ACLs (remove any rights) set for who on mailbox.

IMAP4.**enable**(*capability*)

Enable *capability* (see **RFC 5161**). Most capabilities do not need to be enabled. Currently only the `UTF8=ACCEPT` capability is supported (see **RFC 6855**).

Adicionado na versão 3.5: The *enable()* method itself, and **RFC 6855** support.

IMAP4.**expunge**()

Permanently remove deleted items from selected mailbox. Generates an EXPUNGE response for each deleted message. Returned data contains a list of EXPUNGE message numbers in order received.

IMAP4.**fetch**(*message_set*, *message_parts*)

Fetch (parts of) messages. *message_parts* should be a string of message part names enclosed within parentheses, eg: "(UID BODY[TEXT])". Returned data are tuples of message part envelope and data.

IMAP4.**getacl**(*mailbox*)

Get the ACLs for *mailbox*. The method is non-standard, but is supported by the Cyrus server.

IMAP4.**getannotation**(*mailbox*, *entry*, *attribute*)

Retrieve the specified ANNOTATIONs for *mailbox*. The method is non-standard, but is supported by the Cyrus server.

IMAP4.**getquota**(*root*)

Get the quota *root*'s resource usage and limits. This method is part of the IMAP4 QUOTA extension defined in rfc2087.

IMAP4.**getquotaroot**(*mailbox*)

Get the list of quota roots for the named *mailbox*. This method is part of the IMAP4 QUOTA extension defined in rfc2087.

IMAP4.**idle**(*duration=None*)

Return an Idler: an iterable context manager implementing the IMAP4 IDLE command as defined in **RFC 2177**.

The returned object sends the IDLE command when activated by the with statement, produces IMAP untagged responses via the *iterator* protocol, and sends DONE upon context exit.

All untagged responses that arrive after sending the IDLE command (including any that arrive before the server acknowledges the command) will be available via iteration. Any leftover responses (those not iterated in the with context) can be retrieved in the usual way after IDLE ends, using *IMAP4.response()*.

Responses are represented as (type, [data, ...]) tuples, as described in *IMAP4 Objects*.

The *duration* argument sets a maximum duration (in seconds) to keep idling, after which any ongoing iteration will stop. It can be an *int* or *float*, or None for no time limit. Callers wishing to avoid inactivity timeouts on servers that impose them should keep this at most 29 minutes (1740 seconds). Requires a socket connection; *duration* must be None on *IMAP4_stream* connections.

```
>>> with M.idle(duration=29 * 60) as idler:
...     for typ, data in idler:
...         print(typ, data)
...
EXISTS [b'1']
RECENT [b'1']
```

Idler.**burst**(*interval=0.1*)

Yield a burst of responses no more than *interval* seconds apart (expressed as an *int* or *float*).

This *generator* is an alternative to iterating one response at a time, intended to aid in efficient batch processing. It retrieves the next response along with any immediately available subsequent responses. (For example, a rapid series of EXPUNGE responses after a bulk delete.)

Requires a socket connection; does not work on *IMAP4_stream* connections.

```
>>> with M.idle() as idler:
...     # get a response and any others following by < 0.1 seconds
...     batch = list(idler.burst())
...     print(f'processing {len(batch)} responses...')
...     print(batch)
```

(continua na próxima página)

(continuação da página anterior)

```
...
processing 3 responses...
[('EXPUNGE', [b'2']), ('EXPUNGE', [b'1']), ('RECENT', [b'0'])]
```

Dica

The `IDLE` context's maximum duration, as passed to *IMAP4.idle()*, is respected when waiting for the first response in a burst. Therefore, an expired `Idler` will cause this generator to return immediately without producing anything. Callers should consider this if using it in a loop.

Nota

The iterator returned by *IMAP4.idle()* is usable only within a `with` statement. Before or after that context, unsolicited responses are collected internally whenever a command finishes, and can be retrieved with *IMAP4.response()*.

Nota

The `Idler` class name and structure are internal interfaces, subject to change. Calling code can rely on its context management, iteration, and public method to remain stable, but should not subclass, instantiate, compare, or otherwise directly reference the class.

Adicionado na versão 3.14.

`IMAP4.`**`list`**`(`[*directory*[, *pattern*]]`)`

List mailbox names in *directory* matching *pattern*. *directory* defaults to the top-level mail folder, and *pattern* defaults to match anything. Returned data contains a list of `LIST` responses.

`IMAP4.`**`login`**`(`*user*, *password*`)`

Identify the client using a plaintext password. The *password* will be quoted.

`IMAP4.`**`login_cram_md5`**`(`*user*, *password*`)`

Force use of `CRAM-MD5` authentication when identifying the client to protect the password. Will only work if the server `CAPABILITY` response includes the phrase `AUTH=CRAM-MD5`.

Alterado na versão 3.14: An *IMAP4.error* is raised if MD5 support is not available.

`IMAP4.`**`logout`**`()`

Shutdown connection to server. Returns server `BYE` response.

Alterado na versão 3.8: The method no longer ignores silently arbitrary exceptions.

`IMAP4.`**`lsub`**`(`*directory*=`""`, *pattern*='*'`)`

List subscribed mailbox names in directory matching pattern. *directory* defaults to the top level directory and *pattern* defaults to match any mailbox. Returned data are tuples of message part envelope and data.

`IMAP4.`**`myrights`**`(`*mailbox*`)`

Show my ACLs for a mailbox (i.e. the rights that I have on mailbox).

`IMAP4.`**`namespace`**`()`

Returns IMAP namespaces as defined in **RFC 2342**.

`IMAP4.`**`noop`**`()`

Envia `NOOP` para o servidor.

IMAP4.**open**(*host*, *port*, *timeout=None*)

Opens socket to *port* at *host*. The optional *timeout* parameter specifies a timeout in seconds for the connection attempt. If timeout is not given or is `None`, the global default socket timeout is used. Also note that if the *timeout* parameter is set to be zero, it will raise a *`ValueError`* to reject creating a non-blocking socket. This method is implicitly called by the *`IMAP4`* constructor. The connection objects established by this method will be used in the *`IMAP4.read()`*, *`IMAP4.readline()`*, *`IMAP4.send()`*, and *`IMAP4.shutdown()`* methods. You may override this method.

Levanta um *evento de auditoria* `imaplib.open` com os argumentos `self`, `host`, `port`.

Alterado na versão 3.9: The *timeout* parameter was added.

IMAP4.**partial**(*message_num*, *message_part*, *start*, *length*)

Fetch truncated part of a message. Returned data is a tuple of message part envelope and data.

IMAP4.**proxyauth**(*user*)

Assume authentication as *user*. Allows an authorised administrator to proxy into any user's mailbox.

IMAP4.**read**(*size*)

Reads *size* bytes from the remote server. You may override this method.

IMAP4.**readline**()

Reads one line from the remote server. You may override this method.

IMAP4.**recent**()

Prompt server for an update. Returned data is `None` if no new messages, else value of `RECENT` response.

IMAP4.**rename**(*oldmailbox*, *newmailbox*)

Rename mailbox named *oldmailbox* to *newmailbox*.

IMAP4.**response**(*code*)

Return data for response *code* if received, or `None`. Returns the given code, instead of the usual type.

IMAP4.**search**(*charset*, *criterion*[, ...])

Search mailbox for matching messages. *charset* may be `None`, in which case no `CHARSET` will be specified in the request to the server. The IMAP protocol requires that at least one criterion be specified; an exception will be raised when the server returns an error. *charset* must be `None` if the `UTF8=ACCEPT` capability was enabled using the *`enable()`* command.

Exemplo:

```
# M is a connected IMAP4 instance...
typ, msgnums = M.search(None, 'FROM', '"LDJ"')

# or:
typ, msgnums = M.search(None, '(FROM "LDJ")')
```

IMAP4.**select**(*mailbox='INBOX'*, *readonly=False*)

Select a mailbox. Returned data is the count of messages in *mailbox* (`EXISTS` response). The default *mailbox* is `'INBOX'`. If the *readonly* flag is set, modifications to the mailbox are not allowed.

IMAP4.**send**(*data*)

Sends `data` to the remote server. You may override this method.

Levanta um *evento de auditoria* `imaplib.send` com os argumentos `self`, `data`.

IMAP4.**setacl**(*mailbox*, *who*, *what*)

Set an `ACL` for *mailbox*. The method is non-standard, but is supported by the `Cyrus` server.

IMAP4.**setannotation**(*mailbox*, *entry*, *attribute*[, ...])

Set `ANNOTATION`s for *mailbox*. The method is non-standard, but is supported by the `Cyrus` server.

IMAP4.**setquota**(*root*, *limits*)

Set the `quota` *root*'s resource *limits*. This method is part of the IMAP4 QUOTA extension defined in rfc2087.

IMAP4.**shutdown**()

Close connection established in `open`. This method is implicitly called by *IMAP4.logout()*. You may override this method.

IMAP4.**socket**()

Returns socket instance used to connect to server.

IMAP4.**sort**(*sort_criteria*, *charset*, *search_criterion*[, ...])

The `sort` command is a variant of `search` with sorting semantics for the results. Returned data contains a space separated list of matching message numbers.

Sort has two arguments before the *search_criterion* argument(s); a parenthesized list of *sort_criteria*, and the searching *charset*. Note that unlike `search`, the searching *charset* argument is mandatory. There is also a `uid sort` command which corresponds to `sort` the way that `uid search` corresponds to `search`. The `sort` command first searches the mailbox for messages that match the given searching criteria using the charset argument for the interpretation of strings in the searching criteria. It then returns the numbers of matching messages.

This is an `IMAP4rev1` extension command.

IMAP4.**starttls**(*ssl_context=None*)

Send a `STARTTLS` command. The *ssl_context* argument is optional and should be a *ssl.SSLContext* object. This will enable encryption on the IMAP connection. Please read *Considerações de segurança* for best practices.

Adicionado na versão 3.2.

Alterado na versão 3.4: The method now supports hostname check with *ssl.SSLContext.check_hostname* and *Server Name Indication* (see *ssl.HAS_SNI*).

IMAP4.**status**(*mailbox*, *names*)

Request named status conditions for *mailbox*.

IMAP4.**store**(*message_set*, *command*, *flag_list*)

Alters flag dispositions for messages in mailbox. *command* is specified by section 6.4.6 of **RFC 2060** as being one of "FLAGS", "+FLAGS", or "-FLAGS", optionally with a suffix of ".SILENT".

For example, to set the delete flag on all messages:

```
typ, data = M.search(None, 'ALL')
for num in data[0].split():
   M.store(num, '+FLAGS', '\\Deleted')
M.expunge()
```

Nota

Creating flags containing ']' (for example: "[test]") violates **RFC 3501** (the IMAP protocol). However, imaplib has historically allowed creation of such tags, and popular IMAP servers, such as Gmail, accept and produce such flags. There are non-Python programs which also create such tags. Although it is an RFC violation and IMAP clients and servers are supposed to be strict, imaplib still continues to allow such tags to be created for backward compatibility reasons, and as of Python 3.6, handles them if they are sent from the server, since this improves real-world compatibility.

IMAP4.**subscribe**(*mailbox*)

Subscribe to new mailbox.

IMAP4.**thread**(*threading_algorithm*, *charset*, *search_criterion*[, ...])

The `thread` command is a variant of `search` with threading semantics for the results. Returned data contains a space separated list of thread members.

Thread members consist of zero or more messages numbers, delimited by spaces, indicating successive parent and child.

Thread has two arguments before the *search_criterion* argument(s); a *threading_algorithm*, and the searching *charset*. Note that unlike `search`, the searching *charset* argument is mandatory. There is also a `uid thread` command which corresponds to `thread` the way that `uid search` corresponds to `search`. The `thread` command first searches the mailbox for messages that match the given searching criteria using the *charset* argument for the interpretation of strings in the searching criteria. It then returns the matching messages threaded according to the specified threading algorithm.

This is an `IMAP4rev1` extension command.

`IMAP4.`**`uid`**`(`*command*, *arg*[, ...]`)`

Execute command args with messages identified by UID, rather than message number. Returns response appropriate to command. At least one argument must be supplied; if none are provided, the server will return an error and an exception will be raised.

`IMAP4.`**`unsubscribe`**`(`*mailbox*`)`

Unsubscribe from old mailbox.

`IMAP4.`**`unselect`**`()`

`imaplib.IMAP4.unselect()` frees server's resources associated with the selected mailbox and returns the server to the authenticated state. This command performs the same actions as *`imaplib.IMAP4.close()`*, except that no messages are permanently removed from the currently selected mailbox.

Adicionado na versão 3.9.

`IMAP4.`**`xatom`**`(`*name*[, ...]`)`

Allow simple extension commands notified by server in `CAPABILITY` response.

The following attributes are defined on instances of *`IMAP4`*:

`IMAP4.`**`PROTOCOL_VERSION`**

The most recent supported protocol in the `CAPABILITY` response from the server.

`IMAP4.`**`debug`**

Integer value to control debugging output. The initialize value is taken from the module variable `Debug`. Values greater than three trace each command.

`IMAP4.`**`utf8_enabled`**

Boolean value that is normally `False`, but is set to `True` if an *`enable()`* command is successfully issued for the `UTF8=ACCEPT` capability.

Adicionado na versão 3.5.

22.13.2 Exemplo IMAP4

Here is a minimal example (without error checking) that opens a mailbox and retrieves and prints all messages:

```
import getpass, imaplib

M = imaplib.IMAP4(host='example.org')
M.login(getpass.getuser(), getpass.getpass())
M.select()
typ, data = M.search(None, 'ALL')
for num in data[0].split():
    typ, data = M.fetch(num, '(RFC822)')
    print('Message %s\n%s\n' % (num, data[0][1]))
M.close()
M.logout()
```

22.14 `smtplib` — SMTP protocol client

Código-fonte: Lib/smtplib.py

The *smtplib* module defines an SMTP client session object that can be used to send mail to any internet machine with an SMTP or ESMTP listener daemon. For details of SMTP and ESMTP operation, consult **RFC 821** (Simple Mail Transfer Protocol) and **RFC 1869** (SMTP Service Extensions).

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

class smtplib.**SMTP**(*host=''*, *port=0*, *local_hostname=None*, [*timeout*,]*source_address=None*)

An *SMTP* instance encapsulates an SMTP connection. It has methods that support a full repertoire of SMTP and ESMTP operations. If the optional *host* and *port* parameters are given, the SMTP *connect()* method is called with those parameters during initialization. If specified, *local_hostname* is used as the FQDN of the local host in the HELO/EHLO command. Otherwise, the local hostname is found using *socket.getfqdn()*. If the *connect()* call returns anything other than a success code, an *SMTPConnectError* is raised. The optional *timeout* parameter specifies a timeout in seconds for blocking operations like the connection attempt (if not specified, the global default timeout setting will be used). If the timeout expires, *TimeoutError* is raised. The optional *source_address* parameter allows binding to some specific source address in a machine with multiple network interfaces, and/or to some specific source TCP port. It takes a 2-tuple `(host, port)`, for the socket to bind to as its source address before connecting. If omitted (or if *host* or *port* are `''` and/or `0` respectively) the OS default behavior will be used.

For normal use, you should only require the initialization/connect, *sendmail()*, and *SMTP.quit()* methods. An example is included below.

The *SMTP* class supports the `with` statement. When used like this, the SMTP `QUIT` command is issued automatically when the `with` statement exits. E.g.:

```
>>> from smtplib import SMTP
>>> with SMTP("domain.org") as smtp:
...     smtp.noop()
...
(250, b'Ok')
>>>
```

All commands will raise an *auditing event* `smtplib.SMTP.send` with arguments `self` and `data`, where `data` is the bytes about to be sent to the remote host.

Alterado na versão 3.3: Suporte para a instrução `with` foi adicionado.

Alterado na versão 3.3: *source_address* argument was added.

Adicionado na versão 3.5: A extensão SMTPUTF8 (**RFC 6531**) é suportada agora.

Alterado na versão 3.9: Se o parâmetro *timeout* for definido como zero, será levantada *ValueError* para impedir a criação de um soquete não bloqueante.

class smtplib.**SMTP_SSL**(*host=''*, *port=0*, *local_hostname=None*, *, [*timeout*,]*context=None*, *source_address=None*)

An *SMTP_SSL* instance behaves exactly the same as instances of *SMTP*. *SMTP_SSL* should be used for situations where SSL is required from the beginning of the connection and using `starttls()` is not appropriate. If *host* is not specified, the local host is used. If *port* is zero, the standard SMTP-over-SSL port (465) is used. The optional arguments *local_hostname*, *timeout* and *source_address* have the same meaning as they do in the *SMTP* class. *context*, also optional, can contain a *SSLContext* and allows configuring various aspects of the secure connection. Please read *Considerações de segurança* for best practices.

Alterado na versão 3.3: *context* foi adicionado.

Alterado na versão 3.3: O argumento *source_address* foi adicionado.

Alterado na versão 3.4: A classe agora oferece suporte a verificação de nome de host com `ssl.SSLContext.check_hostname` e *Indicação de nome de servidor* (veja `ssl.HAS_SNI`).

Alterado na versão 3.9: If the *timeout* parameter is set to be zero, it will raise a `ValueError` to prevent the creation of a non-blocking socket

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

class smtplib.**LMTP**(*host=''*, *port=LMTP_PORT*, *local_hostname=None*, *source_address=None*[, *timeout*])

The LMTP protocol, which is very similar to ESMTP, is heavily based on the standard SMTP client. It's common to use Unix sockets for LMTP, so our `connect()` method must support that as well as a regular host:port server. The optional arguments *local_hostname* and *source_address* have the same meaning as they do in the `SMTP` class. To specify a Unix socket, you must use an absolute path for *host*, starting with a '/'.

Authentication is supported, using the regular SMTP mechanism. When using a Unix socket, LMTP generally don't support or require any authentication, but your mileage might vary.

Alterado na versão 3.9: O parâmetro opcional *timeout* foi adicionado.

A nice selection of exceptions is defined as well:

exception smtplib.**SMTPException**

Subclass of `OSError` that is the base exception class for all the other exceptions provided by this module.

Alterado na versão 3.4: SMTPException became subclass of `OSError`

exception smtplib.**SMTPServerDisconnected**

This exception is raised when the server unexpectedly disconnects, or when an attempt is made to use the `SMTP` instance before connecting it to a server.

exception smtplib.**SMTPResponseException**

Base class for all exceptions that include an SMTP error code. These exceptions are generated in some instances when the SMTP server returns an error code. The error code is stored in the `smtp_code` attribute of the error, and the `smtp_error` attribute is set to the error message.

exception smtplib.**SMTPSenderRefused**

Sender address refused. In addition to the attributes set by on all `SMTPResponseException` exceptions, this sets 'sender' to the string that the SMTP server refused.

exception smtplib.**SMTPRecipientsRefused**

All recipient addresses refused. The errors for each recipient are accessible through the attribute `recipients`, which is a dictionary of exactly the same sort as `SMTP.sendmail()` returns.

exception smtplib.**SMTPDataError**

O servidor SMTP recusou aceitar o dados da mensagem.

exception smtplib.**SMTPConnectError**

Ocorreu um erro ao estabelecer uma conexão com o servidor.

exception smtplib.**SMTPHeloError**

O servidor recusou sua mensagem `HELO`.

exception smtplib.**SMTPNotSupportedError**

The command or option attempted is not supported by the server.

Adicionado na versão 3.5.

exception smtplib.**SMTPAuthenticationError**

SMTP authentication went wrong. Most probably the server didn't accept the username/password combination provided.

Ver também

RFC 821 - Simple Mail Transfer Protocol
Protocol definition for SMTP. This document covers the model, operating procedure, and protocol details for SMTP.

RFC 1869 - SMTP Service Extensions
Definition of the ESMTP extensions for SMTP. This describes a framework for extending SMTP with new commands, supporting dynamic discovery of the commands provided by the server, and defines a few additional commands.

22.14.1 Objetos SMTP

An *SMTP* instance has the following methods:

SMTP.**set_debuglevel**(*level*)

Set the debug output level. A value of 1 or `True` for *level* results in debug messages for connection and for all messages sent to and received from the server. A value of 2 for *level* results in these messages being timestamped.

Alterado na versão 3.5: Adicionado o nível de depuração 2.

SMTP.**docmd**(*cmd*, *args=''*)

Send a command *cmd* to the server. The optional argument *args* is simply concatenated to the command, separated by a space.

This returns a 2-tuple composed of a numeric response code and the actual response line (multiline responses are joined into one long line.)

In normal operation it should not be necessary to call this method explicitly. It is used to implement other methods and may be useful for testing private extensions.

If the connection to the server is lost while waiting for the reply, *SMTPServerDisconnected* will be raised.

SMTP.**connect**(*host='localhost'*, *port=0*)

Connect to a host on a given port. The defaults are to connect to the local host at the standard SMTP port (25). If the hostname ends with a colon (`':'`) followed by a number, that suffix will be stripped off and the number interpreted as the port number to use. This method is automatically invoked by the constructor if a host is specified during instantiation. Returns a 2-tuple of the response code and message sent by the server in its connection response.

Levanta um *evento de auditoria* `smtplib.connect` com os argumentos `self`, `host`, `port`.

SMTP.**helo**(*name=''*)

Identify yourself to the SMTP server using `HELO`. The hostname argument defaults to the fully qualified domain name of the local host. The message returned by the server is stored as the `helo_resp` attribute of the object.

In normal operation it should not be necessary to call this method explicitly. It will be implicitly called by the *sendmail()* when necessary.

SMTP.**ehlo**(*name=''*)

Identify yourself to an ESMTP server using `EHLO`. The hostname argument defaults to the fully qualified domain name of the local host. Examine the response for ESMTP option and store them for use by *has_extn()*. Also sets several informational attributes: the message returned by the server is stored as the `ehlo_resp` attribute, `does_esmtp` is set to `True` or `False` depending on whether the server supports ESMTP, and `esmtp_features` will be a dictionary containing the names of the SMTP service extensions this server supports, and their parameters (if any).

Unless you wish to use *has_extn()* before sending mail, it should not be necessary to call this method explicitly. It will be implicitly called by *sendmail()* when necessary.

SMTP.**ehlo_or_helo_if_needed**()

This method calls *ehlo()* and/or *helo()* if there has been no previous EHLO or HELO command this session. It tries ESMTP EHLO first.

SMTPHeloError
The server didn't reply properly to the HELO greeting.

SMTP.**has_extn**(*name*)

Return *True* if *name* is in the set of SMTP service extensions returned by the server, *False* otherwise. Case is ignored.

SMTP.**verify**(*address*)

Check the validity of an address on this server using SMTP VRFY. Returns a tuple consisting of code 250 and a full **RFC 822** address (including human name) if the user address is valid. Otherwise returns an SMTP error code of 400 or greater and an error string.

Nota

Muitos servidores desabilitam o SMTP VRFY para despistar spammers.

SMTP.**login**(*user*, *password*, *, *initial_response_ok=True*)

Log in on an SMTP server that requires authentication. The arguments are the username and the password to authenticate with. If there has been no previous EHLO or HELO command this session, this method tries ESMTP EHLO first. This method will return normally if the authentication was successful, or may raise the following exceptions:

SMTPHeloError
The server didn't reply properly to the HELO greeting.

SMTPAuthenticationError
O servidor não aceitou a combinação de usuário/senha.

SMTPNotSupportedError
The AUTH command is not supported by the server.

SMTPException
No suitable authentication method was found.

Each of the authentication methods supported by *smtplib* are tried in turn if they are advertised as supported by the server. See *auth()* for a list of supported authentication methods. *initial_response_ok* is passed through to *auth()*.

Optional keyword argument *initial_response_ok* specifies whether, for authentication methods that support it, an "initial response" as specified in **RFC 4954** can be sent along with the AUTH command, rather than requiring a challenge/response.

Alterado na versão 3.5: *SMTPNotSupportedError* may be raised, and the *initial_response_ok* parameter was added.

SMTP.**auth**(*mechanism*, *authobject*, *, *initial_response_ok=True*)

Issue an SMTP AUTH command for the specified authentication *mechanism*, and handle the challenge response via *authobject*.

mechanism specifies which authentication mechanism is to be used as argument to the AUTH command; the valid values are those listed in the auth element of esmtp_features.

authobject must be a callable object taking an optional single argument:

```
data = authobject(challenge=None)
```

If optional keyword argument *initial_response_ok* is true, authobject() will be called first with no argument. It can return the **RFC 4954** "initial response" ASCII str which will be encoded and sent with the AUTH command as below. If the authobject() does not support an initial response (e.g. because it requires a

challenge), it should return `None` when called with `challenge=None`. If *initial_response_ok* is false, then `authobject()` will not be called first with `None`.

If the initial response check returns `None`, or if *initial_response_ok* is false, `authobject()` will be called to process the server's challenge response; the *challenge* argument it is passed will be a `bytes`. It should return ASCII `str` *data* that will be base64 encoded and sent to the server.

The `SMTP` class provides `authobjects` for the `CRAM-MD5`, `PLAIN`, and `LOGIN` mechanisms; they are named `SMTP.auth_cram_md5`, `SMTP.auth_plain`, and `SMTP.auth_login` respectively. They all require that the `user` and `password` properties of the `SMTP` instance are set to appropriate values.

User code does not normally need to call `auth` directly, but can instead call the *`login()`* method, which will try each of the above mechanisms in turn, in the order listed. `auth` is exposed to facilitate the implementation of authentication methods not (or not yet) supported directly by *`smtplib`*.

Adicionado na versão 3.5.

`SMTP.`**`starttls`**`(*, context=None)`

Put the SMTP connection in TLS (Transport Layer Security) mode. All SMTP commands that follow will be encrypted. You should then call *`ehlo()`* again.

If *keyfile* and *certfile* are provided, they are used to create an *`ssl.SSLContext`*.

Optional *context* parameter is an *`ssl.SSLContext`* object; This is an alternative to using a keyfile and a certfile and if specified both *keyfile* and *certfile* should be `None`.

If there has been no previous `EHLO` or `HELO` command this session, this method tries ESMTP `EHLO` first.

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

`SMTPHeloError`
The server didn't reply properly to the `HELO` greeting.

`SMTPNotSupportedError`
O servidor não da suporte a extensão STARTTLS.

`RuntimeError`
SSL/TLS support is not available to your Python interpreter.

Alterado na versão 3.3: *context* foi adicionado.

Alterado na versão 3.4: The method now supports hostname check with `SSLContext.check_hostname` and *Server Name Indicator* (see *`HAS_SNI`*).

Alterado na versão 3.5: The error raised for lack of STARTTLS support is now the *`SMTPNotSupportedError`* subclass instead of the base *`SMTPException`*.

`SMTP.`**`sendmail`**`(`*from_addr*, *to_addrs*, *msg*, *mail_options=()*, *rcpt_options=()*`)`

Send mail. The required arguments are an **RFC 822** from-address string, a list of **RFC 822** to-address strings (a bare string will be treated as a list with 1 address), and a message string. The caller may pass a list of ESMTP options (such as `8bitmime`) to be used in `MAIL FROM` commands as *mail_options*. ESMTP options (such as `DSN` commands) that should be used with all `RCPT` commands can be passed as *rcpt_options*. (If you need to use different ESMTP options to different recipients you have to use the low-level methods such as `mail()`, `rcpt()` and `data()` to send the message.)

> **Nota**
>
> The *from_addr* and *to_addrs* parameters are used to construct the message envelope used by the transport agents. `sendmail` does not modify the message headers in any way.

msg may be a string containing characters in the ASCII range, or a byte string. A string is encoded to bytes using the ascii codec, and lone `\r` and `\n` characters are converted to `\r\n` characters. A byte string is not modified.

If there has been no previous EHLO or HELO command this session, this method tries ESMTP EHLO first. If the server does ESMTP, message size and each of the specified options will be passed to it (if the option is in the feature set the server advertises). If EHLO fails, HELO will be tried and ESMTP options suppressed.

This method will return normally if the mail is accepted for at least one recipient. Otherwise it will raise an exception. That is, if this method does not raise an exception, then someone should get your mail. If this method does not raise an exception, it returns a dictionary, with one entry for each recipient that was refused. Each entry contains a tuple of the SMTP error code and the accompanying error message sent by the server.

If SMTPUTF8 is included in *mail_options*, and the server supports it, *from_addr* and *to_addrs* may contain non-ASCII characters.

Esse método deve levantar as seguintes exceções:

SMTPRecipientsRefused
All recipients were refused. Nobody got the mail. The recipients attribute of the exception object is a dictionary with information about the refused recipients (like the one returned when at least one recipient was accepted).

SMTPHeloError
The server didn't reply properly to the HELO greeting.

SMTPSenderRefused
O servidor não aceita o *from_addr*.

SMTPDataError
The server replied with an unexpected error code (other than a refusal of a recipient).

SMTPNotSupportedError
SMTPUTF8 was given in the *mail_options* but is not supported by the server.

Unless otherwise noted, the connection will be open even after an exception is raised.

Alterado na versão 3.2: *msg* may be a byte string.

Alterado na versão 3.5: SMTPUTF8 support added, and *SMTPNotSupportedError* may be raised if SMTPUTF8 is specified but the server does not support it.

SMTP.**send_message**(*msg*, *from_addr=None*, *to_addrs=None*, *mail_options=()*, *rcpt_options=()*)

This is a convenience method for calling *sendmail()* with the message represented by an *email.message.Message* object. The arguments have the same meaning as for *sendmail()*, except that *msg* is a Message object.

If *from_addr* is None or *to_addrs* is None, send_message fills those arguments with addresses extracted from the headers of *msg* as specified in **RFC 5322**: *from_addr* is set to the *Sender* field if it is present, and otherwise to the *From* field. *to_addrs* combines the values (if any) of the *To*, *Cc*, and *Bcc* fields from *msg*. If exactly one set of *Resent-** headers appear in the message, the regular headers are ignored and the *Resent-** headers are used instead. If the message contains more than one set of *Resent-** headers, a *ValueError* is raised, since there is no way to unambiguously detect the most recent set of *Resent-* headers.

send_message serializes *msg* using *BytesGenerator* with \r\n as the *linesep*, and calls *sendmail()* to transmit the resulting message. Regardless of the values of *from_addr* and *to_addrs*, send_message does not transmit any *Bcc* or *Resent-Bcc* headers that may appear in *msg*. If any of the addresses in *from_addr* and *to_addrs* contain non-ASCII characters and the server does not advertise SMTPUTF8 support, an *SMTPNotSupportedError* is raised. Otherwise the Message is serialized with a clone of its *policy* with the *utf8* attribute set to True, and SMTPUTF8 and BODY=8BITMIME are added to *mail_options*.

Adicionado na versão 3.2.

Adicionado na versão 3.5: Support for internationalized addresses (SMTPUTF8).

SMTP.**quit**()

Terminate the SMTP session and close the connection. Return the result of the SMTP QUIT command.

Low-level methods corresponding to the standard SMTP/ESMTP commands HELP, RSET, NOOP, MAIL, RCPT, and DATA are also supported. Normally these do not need to be called directly, so they are not documented here. For details, consult the module code.

22.14.2 Exemplo SMTP

This example prompts the user for addresses needed in the message envelope ('To' and 'From' addresses), and the message to be delivered. Note that the headers to be included with the message must be included in the message as entered; this example doesn't do any processing of the **RFC 822** headers. In particular, the 'To' and 'From' addresses must be included in the message headers explicitly:

```
import smtplib

def prompt(title):
    return input(title).strip()

from_addr = prompt("From: ")
to_addrs  = prompt("To: ").split()
print("Enter message, end with ^D (Unix) or ^Z (Windows):")

# Add the From: and To: headers at the start!
lines = [f"From: {from_addr}", f"To: {', '.join(to_addrs)}", ""]
while True:
    try:
        line = input()
    except EOFError:
        break
    else:
        lines.append(line)

msg = "\r\n".join(lines)
print("Message length is", len(msg))

server = smtplib.SMTP("localhost")
server.set_debuglevel(1)
server.sendmail(from_addr, to_addrs, msg)
server.quit()
```

Nota

In general, you will want to use the `email` package's features to construct an email message, which you can then send via `send_message()`; see *email: Exemplos*.

22.15 `uuid` — Objetos UUID conforme RFC 9562

Código-fonte: Lib/uuid.py

Este módulo fornece objetos `UUID` imutáveis (a classe `UUID`) e *funções* para gerar UUIDs correspondentes a uma versão específica de UUID, conforme especificado em **RFC 9562** (que substitui **RFC 4122**), por exemplo, `uuid1()` para a versão 1 de UUID, `uuid3()` para a versão 3 de UUID e assim por diante. Observe que a versão 2 de UUID foi omitida deliberadamente, pois está fora do escopo da RFC.

Se tudo o que você quer é um ID único, você provavelmente deve chamar `uuid1()` ou `uuid4()`. Note que `uuid1()` pode comprometer a privacidade, pois cria um UUID contendo o endereço de rede do computador. `uuid4()` cria um UUID aleatório.

Dependendo do suporte da plataforma subjacente, `uuid1()` pode ou não retornar um UUID "seguro". Um UUID seguro é aquele que é gerado usando métodos de sincronização que garantem que dois processos não possam obter o mesmo UUID. Todas as instâncias de `UUID` têm um atributo `is_safe` que retransmite qualquer informação sobre a segurança do UUID, usando esta enumeração:

class uuid.**SafeUUID**

Adicionado na versão 3.7.

safe

O UUID foi gerado pela plataforma de forma segura para multiprocessamento.

unsafe

O UUID não foi gerado de forma segura para multiprocessamento.

unknown

A plataforma não fornece informações sobre se o UUID foi gerado com segurança ou não.

class uuid.**UUID**(*hex=None*, *bytes=None*, *bytes_le=None*, *fields=None*, *int=None*, *version=None*, ***, *is_safe=SafeUUID.unknown*)

Cria um UUID a partir de uma sequência de 32 dígitos hexadecimais, uma sequência de 16 bytes em ordem big-endian como o argumento *bytes*, uma sequência de 16 bytes em ordem little-endian como o argumento *bytes_le*, uma tupla de seis inteiros (*time_low* de 32 bits, *time_mid* de 16 bits, *time_hi_version* de 16 bits, *clock_seq_hi_variant* de 8 bits, *clock_seq_low* de 8 bits, *node* de 48 bits) como o argumento *fields* ou um único inteiro de 128 bits como o argumento *int*. Quando uma sequência de dígitos hexadecimais é fornecida, chaves, hifens e um prefixo URN são todos opcionais. Por exemplo, todas essas expressões produzem o mesmo UUID:

```
UUID('{12345678-1234-5678-1234-567812345678}')
UUID('12345678123456781234567812345678')
UUID('urn:uuid:12345678-1234-5678-1234-567812345678')
UUID(bytes=b'\x12\x34\x56\x78'*4)
UUID(bytes_le=b'\x78\x56\x34\x12\x34\x12\x78\x56' +
              b'\x12\x34\x56\x78\x12\x34\x56\x78')
UUID(fields=(0x12345678, 0x1234, 0x5678, 0x12, 0x34, 0x567812345678))
UUID(int=0x12345678123456781234567812345678)
```

Exatamente um de *hex*, *bytes*, *bytes_le*, *fields* ou *int* deve ser fornecido. O argumento *version* é opcional; se fornecido, o UUID resultante terá sua variante e número de versão definidos de acordo com **RFC 9562**, substituindo bits no *hex*, *bytes*, *bytes_le*, *fields* ou *int* fornecido.

A comparação de objetos UUID é feita por meio da comparação de seus atributos *UUID.int*. A comparação com um objeto não UUID levanta uma *TypeError*.

`str(uuid)` retorna uma string no formato `12345678-1234-5678-1234-567812345678` onde os 32 dígitos hexadecimais representam o UUID.

As instâncias *UUID* têm estes atributos de somente leitura:

UUID.**bytes**

O UUID como uma string de 16 bytes (contendo os seis campos inteiros em ordem de bytes big-endian).

UUID.**bytes_le**

O UUID como uma string de 16 bytes (com *time_low*, *time_mid* e *time_hi_version* em ordem de bytes little-endian).

UUID.**fields**

Uma tupla dos seis campos de inteiros do UUID, que também estão disponíveis como seis atributos individuais e dois atributos derivados:

Campo	Significado
UUID.**time_low**	Os primeiros 32 bits do UUID. Relevante apenas para a versão 1.
UUID.**time_mid**	Os próximos 16 bits do UUID. Relevante apenas para a versão 1.
UUID.**time_hi_version**	Os próximos 16 bits do UUID. Relevante apenas para a versão 1.
UUID.**clock_seq_hi_variant**	Os próximos 8 bits do UUID. Relevante apenas para as versões 1 e 6.
UUID.**clock_seq_low**	Os próximos 8 bits do UUID. Relevante apenas para as versões 1 e 6.
UUID.**node**	Os últimos 48 bits do UUID. Relevante apenas para a versão 1.
UUID.**time**	O registro de data e hora de 60 bits como uma contagem de intervalos de 100 nanossegundos desde a época gregoriana (1582-10-15 00:00:00) para as versões 1 e 6, ou o registro de data e hora de 48 bits em milissegundos desde a época Unix (1970-01-01 00:00:00) para a versão 7.
UUID.**clock_seq**	Número de sequência de 14 bits. Relevante apenas para as versões 1 e 6.

UUID.**hex**

O UUID como uma sequência hexadecimal de 32 caracteres minúsculos.

UUID.**int**

O UUID como um inteiro de 128 bits.

UUID.**urn**

O UUID como um URN conforme especificado no **RFC 9562**.

UUID.**variant**

A variante UUID, que determina o layout interno do UUID. Esta será uma das constantes *RESERVED_NCS*, *RFC_4122*, *RESERVED_MICROSOFT* ou *RESERVED_FUTURE*.

UUID.**version**

O número da versão do UUID (1 a 8, significativo somente quando a variante é *RFC_4122*).

Alterado na versão 3.14: Adicionadas versões 6, 7 e 8 do UUID.

UUID.**is_safe**

Uma enumeração de *SafeUUID* que indica se a plataforma gerou o UUID de forma segura para multiprocessamento.

Adicionado na versão 3.7.

O módulo *uuid* define as seguintes funções:

uuid.**getnode**()

Obtém o endereço de hardware como um inteiro positivo de 48 bits. Na primeira vez que isso for executado, ele pode iniciar um programa separado, o que pode ser bem lento. Se todas as tentativas de obter o endereço de hardware falharem, escolhemos um número aleatório de 48 bits com o bit multicast (bit menos significativo

do primeiro octeto) definido como 1, conforme recomendado no **RFC 4122**. "Endereço de hardware" significa o endereço MAC de uma interface de rede. Em uma máquina com várias interfaces de rede, endereços MAC administrados universalmente (ou seja, onde o segundo bit menos significativo do primeiro octeto é *não definido*) serão preferidos em relação a endereços MAC administrados localmente, mas sem nenhuma outra garantia de ordenação.

Alterado na versão 3.7: Endereços MAC administrados universalmente são preferidos em vez de endereços MAC administrados localmente, pois os primeiros têm garantia de serem globalmente exclusivos, enquanto os últimos não.

uuid.**uuid1**(*node=None*, *clock_seq=None*)

Gera um UUID a partir de um ID de host, número de sequência e hora atual de acordo com **RFC 9562, §5.1**.

Quando *node* não é especificado, `getnode()` é usado para obter o endereço de hardware como um inteiro positivo de 48 bits. Quando um número de sequência *clock_seq* não é especificado, um inteiro positivo pseudoaleatório de 14 bits é gerado.

Se *node* ou *clock_seq* excederem a contagem de bits esperada, apenas seus bits menos significativos serão mantidos.

uuid.**uuid3**(*namespace*, *name*)

Gera um UUID com base no hash MD5 de um identificador de espaço de nomes (que é um UUID) e um nome (que é um objeto `bytes` ou uma string que será codificada usando UTF-8) de acordo com **RFC 9562, §5.3**.

uuid.**uuid4**()

Gera um UUID aleatório em um método criptograficamente seguro de acordo com **RFC 9562, §5.4**.

uuid.**uuid5**(*namespace*, *name*)

Gera um UUID com base no hash SHA-1 de um identificador de espaço de nomes (que é um UUID) e um nome (que é um objeto `bytes` ou uma string que será codificada usando UTF-8) de acordo com **RFC 9562, §5.5**.

uuid.**uuid6**(*node=None*, *clock_seq=None*)

Gera um UUID a partir de um número de sequência e hora atual de acordo com **RFC 9562, §5.6**.

Esta é uma alternativa ao `uuid1()` para melhorar a localidade do banco de dados.

Quando *node* não é especificado, `getnode()` é usado para obter o endereço de hardware como um inteiro positivo de 48 bits. Quando um número de sequência *clock_seq* não é especificado, um inteiro positivo pseudoaleatório de 14 bits é gerado.

Se *node* ou *clock_seq* excederem a contagem de bits esperada, apenas seus bits menos significativos serão mantidos.

Adicionado na versão 3.14.

uuid.**uuid7**()

Gera um UUID baseado em tempo de acordo com **RFC 9562, §5.7**.

Para portabilidade entre plataformas sem precisão de submilissegundos, os UUIDs produzidos por esta função incorporam um registro de data e hora de 48 bits e usam um contador de 42 bits para garantir a monotonicidade dentro de um milissegundo.

Adicionado na versão 3.14.

uuid.**uuid8**(*a=None*, *b=None*, *c=None*)

Gera um UUID pseudoaleatório de acordo com **RFC 9562, §5.8**.

Quando especificados, espera-se que os parâmetros *a*, *b* e *c* sejam inteiros positivos de 48, 12 e 62 bits, respectivamente. Se excederem a contagem de bits esperada, apenas os bits menos significativos serão mantidos; argumentos não especificados são substituídos por um inteiro pseudoaleatório de tamanho apropriado.

Por padrão, *a*, *b* e *c* não são gerados por um gerador de números pseudoaleatórios criptograficamente seguro (CSPRNG). Use `uuid4()` quando um UUID precisar ser usado em um contexto de segurança sensível.

Adicionado na versão 3.14.

O módulo *uuid* define os seguintes identificadores de espaço de nomes para uso com *uuid3()* ou *uuid5()*.

uuid.**NAMESPACE_DNS**
: Quando esse espaço de nomes é especificado, a string *name* é um nome de domínio totalmente qualificado.

uuid.**NAMESPACE_URL**
: Quando esse espaço de nomes é especificado, a string *name* é uma URL.

uuid.**NAMESPACE_OID**
: Quando esse espaço de nomes é especificado, a string *name* é um OID ISO.

uuid.**NAMESPACE_X500**
: Quando esse espaço de nomes é especificado, a string *name* é um DN X.500 em DER ou um formato de saída de texto.

O módulo *uuid* define as seguintes constantes para os valores possíveis do atributo *variant*:

uuid.**RESERVED_NCS**
: Reservado para compatibilidade com NCS.

uuid.**RFC_4122**
: Especifica o layout do UUID fornecido em **RFC 4122**. Esta constante é mantida para retrocompatibilidade, mesmo que **RFC 4122** tenha sido substituído por **RFC 9562**.

uuid.**RESERVED_MICROSOFT**
: Reservado para compatibilidade com Microsoft.

uuid.**RESERVED_FUTURE**
: Reservado para definição futura.

O módulo *uuid* define os valores especiais de UUID Nil e Max:

uuid.**NIL**
: Uma forma especial de UUID que é especificada para ter todos os 128 bits definidos como zero de acordo com **RFC 9562, §5.9**.

  Adicionado na versão 3.14.

uuid.**MAX**
: Uma forma especial de UUID que é especificada para ter todos os 128 bits definidos como um, de acordo com **RFC 9562, §5.10**.

  Adicionado na versão 3.14.

Ver também

RFC 9562 - Um espaço de nomes de URN de identificador universalmente exclusivo (UUID)
Esta especificação define um espaço de nomes de Uniform Resource Name para UUIDs, o formato interno de UUIDs e métodos de geração de UUIDs.

22.15.1 Uso na linha de comando

Adicionado na versão 3.12.

O módulo *uuid* pode ser invocado como um script na linha de comando:

```
python -m uuid [-h] [-u {uuid1,uuid3,uuid4,uuid5,uuid6,uuid7,uuid8}] [-n ESPAÇOS-
↪DE-NOME] [-N NOME]
```

As seguintes opções são aceitas:

-h, --help
Mostra a mensagem de ajuda e sai.

-u <uuid>
--uuid <uuid>
Especifica o nome da função a ser usada para gerar o uuid. Por padrão, *uuid4()* é usado.

Alterado na versão 3.14: Permite gerar versões 6, 7 e 8 do UUID.

-n <namespace>
--namespace <namespace>
O espaço de nomes é um UUID, ou `@ns` onde `ns` é um UUID predefinido bem conhecido endereçado pelo nome do espaço de nomes. Como `@dns`, `@url`, `@oid` e `@x500`. Necessário apenas para funções *uuid3()* / *uuid5()*.

-N <name>
--name <name>
O nome usado como parte da geração do uuid. Requerido apenas para funções *uuid3()* / *uuid5()*.

-C <num>
--count <num>
Gera *num* novos UUIDs.

Adicionado na versão 3.14.

22.15.2 Exemplo

Aqui estão alguns exemplos de uso típico do módulo *uuid*:

```
>>> import uuid

>>> # make a UUID based on the host ID and current time
>>> uuid.uuid1()
UUID('a8098c1a-f86e-11da-bd1a-00112444be1e')

>>> # make a UUID using an MD5 hash of a namespace UUID and a name
>>> uuid.uuid3(uuid.NAMESPACE_DNS, 'python.org')
UUID('6fa459ea-ee8a-3ca4-894e-db77e160355e')

>>> # make a random UUID
>>> uuid.uuid4()
UUID('16fd2706-8baf-433b-82eb-8c7fada847da')

>>> # make a UUID using a SHA-1 hash of a namespace UUID and a name
>>> uuid.uuid5(uuid.NAMESPACE_DNS, 'python.org')
UUID('886313e1-3b8a-5372-9b90-0c9aee199e5d')

>>> # make a UUID from a string of hex digits (braces and hyphens ignored)
>>> x = uuid.UUID('{00010203-0405-0607-0809-0a0b0c0d0e0f}')

>>> # convert a UUID to a string of hex digits in standard form
>>> str(x)
'00010203-0405-0607-0809-0a0b0c0d0e0f'

>>> # get the raw 16 bytes of the UUID
>>> x.bytes
b'\x00\x01\x02\x03\x04\x05\x06\x07\x08\t\n\x0b\x0c\r\x0e\x0f'

>>> # make a UUID from a 16-byte string
```

(continua na próxima página)

(continuação da página anterior)

```
>>> uuid.UUID(bytes=x.bytes)
UUID('00010203-0405-0607-0809-0a0b0c0d0e0f')

>>> # get the Nil UUID
>>> uuid.NIL
UUID('00000000-0000-0000-0000-000000000000')

>>> # get the Max UUID
>>> uuid.MAX
UUID('ffffffff-ffff-ffff-ffff-ffffffffffff')

>>> # same as UUIDv1 but with fields reordered to improve DB locality
>>> uuid.uuid6()
UUID('1f0799c0-98b9-62db-92c6-a0d365b91053')

>>> # get UUIDv7 creation (local) time as a timestamp in milliseconds
>>> u = uuid.uuid7()
>>> u.time
1743936859822

>>> # get UUIDv7 creation (local) time as a datetime object
>>> import datetime as dt
>>> dt.datetime.fromtimestamp(u.time / 1000)
datetime.datetime(...)

>>> # make a UUID with custom blocks
>>> uuid.uuid8(0x12345678, 0x9abcdef0, 0x11223344)
UUID('00001234-5678-8ef0-8000-000011223344')
```

22.15.3 Exemplo de linha de comando

Aqui estão alguns exemplos de uso típico da interface de linha de comando do *uuid*:

```
# gera um UUID aleatório - por padrão uuid4() é usado
$ python -m uuid

# gera um UUID usando uuid1()
$ python -m uuid -u uuid1

# gera um UUID usando uuid5
$ python -m uuid -u uuid5 -n @url -N example.com

# gera 42 UUIDs aleatórios
$ python -m uuid -C 42
```

22.16 `socketserver` — Um framework para servidores de rede

Código-fonte: Lib/socketserver.py

O módulo *socketserver* simplifica a tarefa de escrever servidores de rede.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

Existem quatro classes básicas de servidores concretos:

class socketserver.**TCPServer**(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*)

Isso usa o protocolo TCP da internet, que fornece fluxos contínuos de dados entre o cliente e o servidor. Se *bind_and_activate* for true, o construtor tenta automaticamente invocar *server_bind()* e *server_activate()*. Os outros parâmetros são passados para a classe base *BaseServer*.

class socketserver.**UDPServer**(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*)

Isso usa datagramas, que são pacotes discretos de informações que podem chegar fora de ordem ou ser perdidos durante o trânsito. Os parâmetros são os mesmos de *TCPServer*.

class socketserver.**UnixStreamServer**(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*)

class socketserver.**UnixDatagramServer**(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*)

Essas classes usadas com menos frequência são semelhantes às classes TCP e UDP, mas usam soquetes de domínio Unix; elas não estão disponíveis em plataformas não Unix. Os parâmetros são os mesmos de *TCPServer*.

Essas quatro classes processam solicitações *synchronously*; cada solicitação deve ser concluída antes que a próxima solicitação possa ser iniciada. Isso não é adequado se cada solicitação demorar muito para ser concluída, porque exige muita computação ou porque retorna muitos dados que o cliente demora a processar. A solução é criar um processo ou thread separado para lidar com cada solicitação; as classes misturadas *ForkingMixIn* e *ThreadingMixIn* podem ser usadas para oferecer suporte ao comportamento assíncrono.

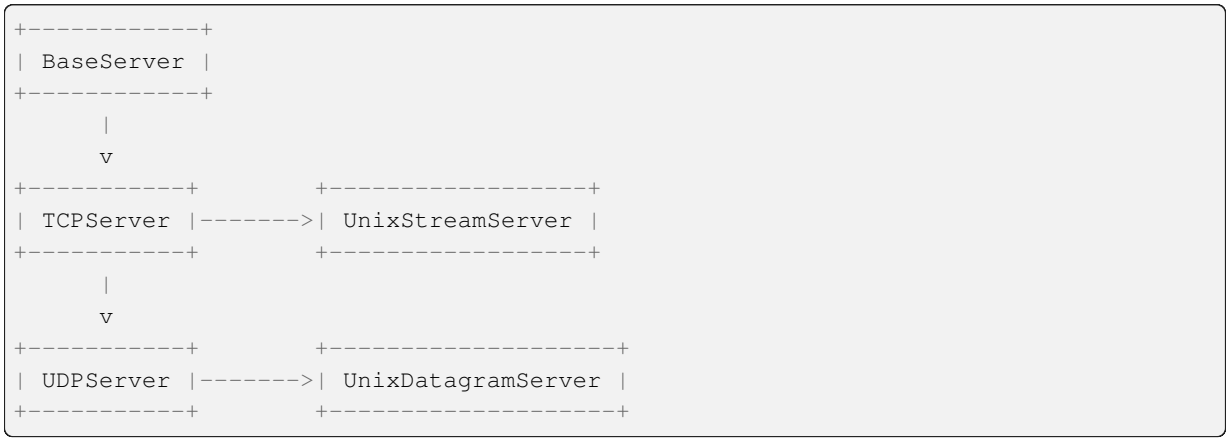
Criar um servidor requer várias etapas. Primeiro, você deve criar uma classe de manipulador de solicitação estendendo a classe *BaseRequestHandler* e substituindo seu método *handle()*; esse método processará as solicitações recebidas. Segundo, você deve instanciar uma das classes de servidor, passando a ela o endereço do servidor e a classe de manipulador de solicitação. É recomendável usar o servidor em uma instrução `with`. Em seguida, chame o método *handle_request()* ou *serve_forever()* do objeto do servidor para processar uma ou muitas solicitações. Finalmente, chame *server_close()* para fechar o soquete (a menos que você tenha usado uma instrução `with`).

Ao herdar de *ThreadingMixIn* para comportamento de conexão em threads, você deve declarar explicitamente como quer que suas threads se comportem em um desligamento abrupto. A classe *ThreadingMixIn* define um atributo *daemon_threads*, que indica se o servidor deve ou não esperar pelo término da thread. Você deve definir o sinalizador explicitamente se quiser que as threads se comportem de forma autônoma; o padrão é *False*, o que significa que o Python não sairá até que todas as threads criadas por *ThreadingMixIn* tenham sido encerradas.

As classes de servidor têm os mesmos métodos e atributos externos, não importa qual protocolo de rede eles usam.

22.16.1 Notas sobre criação do servidor

Existem cinco classes em um diagrama de herança, quatro das quais representam servidores síncronos de quatro tipos:



Observe que *UnixDatagramServer* deriva de *UDPServer*, não de *UnixStreamServer* — a única diferença entre um IP e um servidor Unix é a família de endereços.

class socketserver.**ForkingMixIn**

class socketserver.**ThreadingMixIn**

Versões de fork e threads de cada tipo de servidor podem ser criadas usando essas classes mix-in. Por exemplo, *ThreadingUDPServer* é criado da seguinte forma:

```
class ThreadingUDPServer(ThreadingMixIn, UDPServer):
    pass
```

A classe mix-in vem primeiro, pois ela substitui um método definido em *UDPServer*. Definir os vários atributos também altera o comportamento do mecanismo de servidor subjacente.

ForkingMixIn e as classes Forking mencionadas abaixo estão disponíveis apenas em plataformas POSIX que oferecem suporte a *fork()*.

block_on_close

ForkingMixIn.server_close aguarda até que todos os processos filho sejam concluídos, exceto se o atributo *block_on_close* for `False`.

ThreadingMixIn.server_close aguarda até que todos as threads não daemon sejam concluídos, exceto se o atributo *block_on_close* for `False`.

max_children

Especifica quantos processos filho existirão para processar solicitações simultaneamente para *ForkingMixIn*. Se o limite for atingido, novas solicitações aguardarão até que um processo filho seja concluído.

daemon_threads

Para *ThreadingMixIn*, use threads como daemon definindo *ThreadingMixIn.daemon_threads* como `True` para não esperar até que as threads sejam concluídas.

Alterado na versão 3.7: *ForkingMixIn.server_close* e *ThreadingMixIn.server_close* agora esperam até que todos os processos filhos e threads não daemon sejam concluídos. Adicione um novo atributo de classe *ForkingMixIn.block_on_close* para optar pelo comportamento pré-3.7.

class socketserver.**ForkingTCPServer**

class socketserver.**ForkingUDPServer**

class socketserver.**ThreadingTCPServer**

class socketserver.**ThreadingUDPServer**

class socketserver.**ForkingUnixStreamServer**

class socketserver.**ForkingUnixDatagramServer**

class socketserver.**ThreadingUnixStreamServer**

class socketserver.**ThreadingUnixDatagramServer**

Essas classes são predefinidas usando as classes mix-in.

Adicionado na versão 3.12: As classes `ForkingUnixStreamServer` e `ForkingUnixDatagramServer` foram adicionadas.

Para implementar um serviço, você deve derivar uma classe de *BaseRequestHandler* e redefinir seu método *handle()*. Você pode então executar várias versões do serviço combinando uma das classes de servidor com sua classe de manipulador de solicitação. A classe de manipulador de solicitação deve ser diferente para serviços de datagrama ou fluxo. Isso pode ser ocultado usando as subclasses de manipulador *StreamRequestHandler* ou *DatagramRequestHandler*.

Claro, você ainda tem que usar sua cabeça! Por exemplo, não faz sentido usar um servidor de Forking se o serviço contém estado na memória que pode ser modificado por diferentes solicitações, já que as modificações no processo filho nunca atingiriam o estado inicial mantido no processo pai e passado para cada filho. Neste caso, você pode usar um servidor de Threading, mas provavelmente terá que usar travas para proteger a integridade dos dados compartilhados.

Por outro lado, se você estiver construindo um servidor HTTP onde todos os dados são armazenados externamente (por exemplo, no sistema de arquivos), uma classe síncrona essencialmente tornará o serviço "surdo" enquanto uma

solicitação estiver sendo manipulada – o que pode ser por um tempo muito longo se um cliente for lento para receber todos os dados que solicitou. Aqui, um servidor de Threading ou Forking é apropriado.

Em alguns casos, pode ser apropriado processar parte de uma solicitação de forma síncrona, mas terminar o processamento em um filho bifurcado dependendo dos dados da solicitação. Isso pode ser implementado usando um servidor síncrono e fazendo uma bifurcação explícita no método da classe do manipulador de solicitações *handle()*.

Outra abordagem para lidar com múltiplas requisições simultâneas em um ambiente que não oferece suporte a threads nem *fork()* (ou onde estes são muito caros ou inapropriados para o serviço) é manter uma tabela explícita de requisições parcialmente finalizadas e usar *selectors* para decidir em qual requisição trabalhar em seguida (ou se deve lidar com uma nova requisição recebida). Isto é particularmente importante para serviços de fluxo onde cada cliente pode potencialmente ficar conectado por um longo tempo (se threads ou subprocessos não puderem ser usados).

22.16.2 Objetos Server

class socketserver.**BaseServer**(*server_address*, *RequestHandlerClass*)

Esta é a superclasse de todos os objetos Server no módulo. Ela define a interface, dada abaixo, mas não implementa a maioria dos métodos, o que é feito em subclasses. Os dois parâmetros são armazenados nos respectivos atributos *server_address* e *RequestHandlerClass*.

fileno()

Retorna um descritor de arquivo inteiro para o soquete no qual o servidor está escutando. Esta função é mais comumente passada para *selectors*, para permitir o monitoramento de múltiplos servidores no mesmo processo.

handle_request()

Processa uma única solicitação. Esta função chama os seguintes métodos em ordem: *get_request()*, *verify_request()* e *process_request()*. Se o método *handle()* fornecido pelo usuário da classe manipuladora levantar uma exceção, o método *handle_error()* do servidor será chamado. Se nenhuma solicitação for recebida dentro de *timeout* segundos, *handle_timeout()* será chamado e *handle_request()* retornará.

serve_forever(*poll_interval=0.5*)

Manipula solicitações até uma solicitação explícita *shutdown()*. Sonda para desligamento a cada *poll_interval* segundos. Ignora o atributo *timeout*. Ele também chama *service_actions()*, que pode ser usado por uma subclasse ou mixin para fornecer ações específicas para um determinado serviço. Por exemplo, a classe *ForkingMixIn* usa *service_actions()* para limpar processos filhos zumbis.

Alterado na versão 3.3: Adicionada chamada `service_actions` ao método `serve_forever`.

service_actions()

Isso é chamado no laço *serve_forever()*. Este método pode ser substituído por subclasses ou classes mixin para executar ações específicas para um determinado serviço, como ações de limpeza.

Adicionado na versão 3.3.

shutdown()

Diz ao laço *serve_forever()* para parar e esperar até que isso aconteça. *shutdown()* deve ser chamado enquanto *serve_forever()* estiver em execução em um thread diferente, caso contrário, ocorrerá um impasse.

server_close()

Limpa o servidor. Pode ser substituído.

address_family

A família de protocolos à qual o soquete do servidor pertence. Exemplos comuns são *socket.AF_INET*, *socket.AF_INET6* e *socket.AF_UNIX*. Subclasse as classes de servidor TCP ou UDP neste módulo com o atributo de classe `address_family = AF_INET6` definido se você quiser classes de servidor IPv6.

RequestHandlerClass

A classe de manipulador de solicitações fornecida pelo usuário; uma instância dessa classe é criada para cada solicitação.

server_address

O endereço no qual o servidor está escutando. O formato dos endereços varia dependendo da família do protocolo; veja a documentação do módulo *socket* para detalhes. Para protocolos de internet, esta é uma tupla contendo uma string dando o endereço e um número de porta inteiro: `('127.0.0.1', 80)`, por exemplo.

socket

O objeto soquete no qual o servidor ouve para solicitações recebidas.

As classes de servidor oferecem suporte às seguintes variáveis de classe:

allow_reuse_address

Se o servidor permitirá a reutilização de um endereço. O padrão é *False*, e pode ser definido em subclasses para alterar a política.

request_queue_size

O tamanho da fila de requisições. Se levar muito tempo para processar uma única requisição, todas as requisições que chegarem enquanto o servidor estiver ocupado serão colocadas em uma fila, até *request_queue_size* requisições. Quando a fila estiver cheia, outras requisições de clientes receberão um erro "Conexão negada". O valor padrão é geralmente 5, mas isso pode ser substituído por subclasses.

socket_type

O tipo de soquete usado pelo servidor; *socket.SOCK_STREAM* e *socket.SOCK_DGRAM* são dois valores comuns.

timeout

Duração do tempo limite, medida em segundos, ou *None* se nenhum tempo limite for desejado. Se *handle_request()* não receber nenhuma requisição de entrada dentro do período de tempo limite, o método *handle_timeout()* será chamado.

Existem vários métodos de servidor que podem ser substituídos por subclasses de classes de servidor base, como *TCPServer*; esses métodos não são úteis para usuários externos do objeto de servidor.

finish_request(*request*, *client_address*)

Na verdade, processa a solicitação instanciando *RequestHandlerClass* e chamando seu método *handle()*.

get_request()

Deve aceitar uma solicitação do soquete e retornar uma tupla de 2 elementos contendo o *novo* objeto de soquete a ser usado para se comunicar com o cliente e o endereço do cliente.

handle_error(*request*, *client_address*)

Esta função é chamada se o método *handle()* de uma instância *RequestHandlerClass* levanta uma exceção. A ação padrão é exibir o traceback para erro padrão e continuar manipulando solicitações futuras.

Alterado na versão 3.6: Agora é chamada apenas para exceções derivadas da classe *Exception*.

handle_timeout()

Esta função é chamada quando o atributo *timeout* foi definido para um valor diferente de *None* e o período de timeout passou sem que nenhuma solicitação tenha sido recebida. A ação padrão para servidores de bifurcação é coletar o status de quaisquer processos filhos que tenham saído, enquanto em servidores de threading esse método não faz nada.

process_request(*request*, *client_address*)

Chama *finish_request()* para criar uma instância de *RequestHandlerClass*. Se desejado, esta função pode criar um novo processo ou thread para manipular a solicitação; as classes *ForkingMixIn* e *ThreadingMixIn* fazem isso.

server_activate()

Chamado pelo construtor do servidor para ativar o servidor. O comportamento padrão para um servidor TCP apenas invoca *listen()* no soquete do servidor. Pode ser substituído.

server_bind()

Chamada pelo construtor do servidor para vincular o soquete ao endereço desejado. Pode ser substituída.

verify_request(*request*, *client_address*)

Deve retornar um valor booleano; se o valor for *True*, a solicitação será processada, e se for *False*, a solicitação será negada. Esta função pode ser substituída para implementar controles de acesso para um servidor. A implementação padrão sempre retorna *True*.

Alterado na versão 3.6: Foi adicionado suporte para o protocolo *gerenciador de contexto*. Sair do gerenciador de contexto é equivalente a chamar *server_close()*.

22.16.3 Objetos manipulador de requisições

class socketserver.**BaseRequestHandler**

Esta é a superclasse de todos os objetos manipulador de requisições. Ela define a interface, dada abaixo. Uma subclasse concreta do manipulador de requisições deve definir um novo método *handle()* e pode substituir qualquer um dos outros métodos. Uma nova instância da subclasse é criada para cada requisição.

setup()

Chamada antes do método *handle()* para executar quaisquer ações de inicialização necessárias. A implementação padrão não faz nada.

handle()

Esta função deve fazer todo o trabalho necessário para atender a uma solicitação. A implementação padrão não faz nada. Vários atributos de instância estão disponíveis para ela; a solicitação está disponível como *request*; o endereço do cliente como *client_address*; e a instância do servidor como *server*, caso precise de acesso a informações por servidor.

O tipo de *request* é diferente para serviços de datagrama ou fluxo. Para serviços de fluxo, *request* é um objeto soquete; para serviços de datagrama, *request* é um par de string e soquete.

finish()

Chamada após o método *handle()* para executar quaisquer ações de limpeza necessárias. A implementação padrão não faz nada. Se *setup()* levanta uma exceção, esta função não será chamada.

request

O *novo* objeto *socket.socket* a ser usado para se comunicar com o cliente.

client_address

Endereço do cliente retornado por *BaseServer.get_request()*.

server

Objeto *BaseServer* usado para manipular a solicitação.

class socketserver.**StreamRequestHandler**

class socketserver.**DatagramRequestHandler**

Essas subclasses *BaseRequestHandler* substituem os métodos *setup()* e *finish()* e fornecem os atributos *rfile* e *wfile*.

rfile

Um objeto arquivo do qual recebe a solicitação é lido. Oferece suporte à interface legível *io.BufferedIOBase*.

wfile

Um objeto arquivo no qual a resposta é escrita. Oferece suporte à interface gravável *io.BufferedIOBase*

Alterado na versão 3.6: *wfile* também oferece suporte à interface gravável *io.BufferedIOBase*.

22.16.4 Exemplos

Exemplo de `socketserver.TCPServer`

Este é o lado do servidor:

```
import socketserver

class MyTCPHandler(socketserver.BaseRequestHandler):
    """
    A classe de manipulador de solicitação para nosso servidor.

    Ela é instanciada uma vez por conexão com o servidor e deve
    substituir o método handle() para implementar a comunicação
    com o cliente.
    """

    def handle(self):
        # self.request é o soquete TCP conectado ao cliente
        pieces = [b'']
        total = 0
        while b'\n' not in pieces[-1] and total < 10_000:
            pieces.append(self.request.recv(2000))
            total += len(pieces[-1])
        self.data = b''.join(pieces)
        print(f"Received from {self.client_address[0]}:")
        print(self.data.decode("utf-8"))
        # só envia de volta os mesmos dados, mas em caixa alta
        self.request.sendall(self.data.upper())
        # após retornarmos, o soquete será fechado.

if __name__ == "__main__":
    HOST, PORT = "localhost", 9999

    # Cria o servidor, ligando-o ao localhost na porta 9999
    with socketserver.TCPServer((HOST, PORT), MyTCPHandler) as server:
        # Ativa o servidor; isso vai mantê-lo em execução até você
        # interromper o programa com Ctrl-C
        server.serve_forever()
```

Uma classe de manipulador de solicitações alternativa que faz uso de fluxos (objetos arquivo ou similares que simplificam a comunicação ao fornecer a interface de arquivo padrão):

```
class MyTCPHandler(socketserver.StreamRequestHandler):

    def handle(self):
        # self.rfile é um objeto arquivo ou similar criado pelo manipulador.
        # Agora podemos usar, por exemplo, readline() em vez de chamadas
        # brutas a recv().
        # Nos limitamos a 10000 bytes para evitar abuso pelo remetente.
        self.data = self.rfile.readline(10000).rstrip()
        print(f"{self.client_address[0]} wrote:")
        print(self.data.decode("utf-8"))
        # Da mesma forma, self.wfile é um objeto arquivo ou similar para
        # escrever de volta para o cliente
        self.wfile.write(self.data.upper())
```

A diferença é que a chamada `readline()` no segundo manipulador chamará `recv()` várias vezes até encontrar um caractere de nova linha, enquanto o primeiro manipulador teve que usar um laço `recv()` para acumular dados até

uma nova linha. Se tivesse usado apenas um único `recv()` sem o laço, ele teria retornado apenas o que foi recebido até agora do cliente. O TCP é baseado em fluxo: os dados chegam na ordem em que foram enviados, mas não há correlação entre as chamadas `send()` ou `sendall()` do cliente e o número de chamadas `recv()` no servidor necessárias para recebê-los.

Este é o lado do cliente:

```
import socket
import sys

HOST, PORT = "localhost", 9999
data = " ".join(sys.argv[1:])

# Cria um soquete (SOCK_STREAM significa um soquete TCP)
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
    # Conecta a um servidor e envia dados
    sock.connect((HOST, PORT))
    sock.sendall(bytes(data, "utf-8"))
    sock.sendall(b"\n")

    # Recebe dados do servidor e desliga
    received = str(sock.recv(1024), "utf-8")

print("Sent:     ", data)
print("Received:", received)
```

A saída do exemplo deve se parecer com algo como isso:

Servidor:

```
$ python TCPServer.py
127.0.0.1 wrote:
b'hello world with TCP'
127.0.0.1 wrote:
b'python is nice'
```

Cliente:

```
$ python TCPClient.py hello world with TCP
Sent:     hello world with TCP
Received: HELLO WORLD WITH TCP
$ python TCPClient.py python is nice
Sent:     python is nice
Received: PYTHON IS NICE
```

Exemplo `socketserver.UDPServer`

Este é o lado do servidor:

```
import socketserver

class MyUDPHandler(socketserver.BaseRequestHandler):
    """
    Esta classe funciona de forma semelhante à classe do manipulador TCP,
    exceto que self.request consiste em um par de dados e soquete do cliente
    e, como não há conexão, o endereço do cliente deve ser fornecido
    explicitamente ao enviar dados de volta via sendto().
    """
```

(continua na próxima página)

(continuação da página anterior)

```
    def handle(self):
        data = self.request[0].strip()
        socket = self.request[1]
        print(f"{self.client_address[0]} wrote:")
        print(data)
        socket.sendto(data.upper(), self.client_address)

if __name__ == "__main__":
    HOST, PORT = "localhost", 9999
    with socketserver.UDPServer((HOST, PORT), MyUDPHandler) as server:
        server.serve_forever()
```

Este é o lado do cliente:

```
import socket
import sys

HOST, PORT = "localhost", 9999
data = " ".join(sys.argv[1:])

# SOCK_DGRAM é o tipo de soquete para usar para soquetes UDP
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Como você pode ver, não há uma chamada connect(); UDP não tem conexão.
# Em vez disso, dados são enviados diretamente para o destinatário via sendto().
sock.sendto(bytes(data + "\n", "utf-8"), (HOST, PORT))
received = str(sock.recv(1024), "utf-8")

print("Sent:     ", data)
print("Received:", received)
```

A saída do exemplo deve ser exatamente igual à do exemplo do servidor TCP.

Mixins assíncronos

Para construir manipuladores assíncronos, use as classes *ThreadingMixIn* e *ForkingMixIn*.

Um exemplo para a classe *ThreadingMixIn*:

```
import socket
import threading
import socketserver

class ThreadedTCPRequestHandler(socketserver.BaseRequestHandler):

    def handle(self):
        data = str(self.request.recv(1024), 'ascii')
        cur_thread = threading.current_thread()
        response = bytes("{}: {}".format(cur_thread.name, data), 'ascii')
        self.request.sendall(response)

class ThreadedTCPServer(socketserver.ThreadingMixIn, socketserver.TCPServer):
    pass

def client(ip, port, message):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.connect((ip, port))
```

(continua na próxima página)

(continuação da página anterior)

```
        sock.sendall(bytes(message, 'ascii'))
        response = str(sock.recv(1024), 'ascii')
        print("Received: {}".format(response))

if __name__ == "__main__":
    # Porta 0 significa selecionar uma porta não usada arbitrária
    HOST, PORT = "localhost", 0

    server = ThreadedTCPServer((HOST, PORT), ThreadedTCPRequestHandler)
    with server:
        ip, port = server.server_address

        # Inicia uma thread com o servidor -- essa thread vai então
        # iniciar mais um thread para cada requisição
        server_thread = threading.Thread(target=server.serve_forever)
        # Sai da thread do servidor quando a thread principal é encerrada
        server_thread.daemon = True
        server_thread.start()
        print("Server loop running in thread:", server_thread.name)

        client(ip, port, "Hello World 1")
        client(ip, port, "Hello World 2")
        client(ip, port, "Hello World 3")

        server.shutdown()
```

A saída do exemplo deve se parecer com algo como isso:

```
$ python ThreadedTCPServer.py
Server loop running in thread: Thread-1
Received: Thread-2: Hello World 1
Received: Thread-3: Hello World 2
Received: Thread-4: Hello World 3
```

A classe *ForkingMixIn* é usada da mesma forma, exceto que o servidor gerará um novo processo para cada solicitação. Disponível apenas em plataformas POSIX que oferecem suporte a *fork()*.

22.17 `http.server` — HTTP servers

Código-fonte: Lib/http/server.py

This module defines classes for implementing HTTP servers.

> **Aviso**
>
> *http.server* is not recommended for production. It only implements *basic security checks*.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

One class, *HTTPServer*, is a *socketserver.TCPServer* subclass. It creates and listens at the HTTP socket, dispatching the requests to a handler. Code to create and run the server looks like this:

```
def run(server_class=HTTPServer, handler_class=BaseHTTPRequestHandler):
    server_address = ('', 8000)
    httpd = server_class(server_address, handler_class)
    httpd.serve_forever()
```

class http.server.**HTTPServer**(*server_address*, *RequestHandlerClass*)

This class builds on the *TCPServer* class by storing the server address as instance variables named `server_name` and `server_port`. The server is accessible by the handler, typically through the handler's `server` instance variable.

class http.server.**ThreadingHTTPServer**(*server_address*, *RequestHandlerClass*)

This class is identical to HTTPServer but uses threads to handle requests by using the *ThreadingMixIn*. This is useful to handle web browsers pre-opening sockets, on which *HTTPServer* would wait indefinitely.

Adicionado na versão 3.7.

class http.server.**HTTPSServer**(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*, ***, *certfile*, *keyfile=None*, *password=None*, *alpn_protocols=None*)

Subclass of *HTTPServer* with a wrapped socket using the *ssl* module. If the *ssl* module is not available, instantiating a `HTTPSServer` object fails with a *RuntimeError*.

The *certfile* argument is the path to the SSL certificate chain file, and the *keyfile* is the path to file containing the private key.

A *password* can be specified for files protected and wrapped with PKCS#8, but beware that this could possibly expose hardcoded passwords in clear.

Ver também

See *ssl.SSLContext.load_cert_chain()* for additional information on the accepted values for *certfile*, *keyfile* and *password*.

When specified, the *alpn_protocols* argument must be a sequence of strings specifying the "Application-Layer Protocol Negotiation" (ALPN) protocols supported by the server. ALPN allows the server and the client to negotiate the application protocol during the TLS handshake.

By default, it is set to `["http/1.1"]`, meaning the server supports HTTP/1.1.

Adicionado na versão 3.14.

class http.server.**ThreadingHTTPSServer**(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*, ***, *certfile*, *keyfile=None*, *password=None*, *alpn_protocols=None*)

This class is identical to *HTTPSServer* but uses threads to handle requests by inheriting from *ThreadingMixIn*. This is analogous to *ThreadingHTTPServer* only using *HTTPSServer*.

Adicionado na versão 3.14.

The *HTTPServer*, *ThreadingHTTPServer*, *HTTPSServer* and *ThreadingHTTPSServer* must be given a *RequestHandlerClass* on instantiation, of which this module provides three different variants:

class http.server.**BaseHTTPRequestHandler**(*request*, *client_address*, *server*)

This class is used to handle the HTTP requests that arrive at the server. By itself, it cannot respond to any actual HTTP requests; it must be subclassed to handle each request method (e.g. GET or POST). *BaseHTTPRequestHandler* provides a number of class and instance variables, and methods for use by subclasses.

The handler will parse the request and the headers, then call a method specific to the request type. The method name is constructed from the request. For example, for the request method `SPAM`, the `do_SPAM()` method will be called with no arguments. All of the relevant information is stored in instance variables of the handler. Subclasses should not need to override or extend the `__init__()` method.

BaseHTTPRequestHandler tem os seguintes atributos de instância:

client_address

Contains a tuple of the form `(host, port)` referring to the client's address.

server

Contains the server instance.

close_connection

Boolean that should be set before *handle_one_request()* returns, indicating if another request may be expected, or if the connection should be shut down.

requestline

Contains the string representation of the HTTP request line. The terminating CRLF is stripped. This attribute should be set by *handle_one_request()*. If no valid request line was processed, it should be set to the empty string.

command

Contains the command (request type). For example, `'GET'`.

path

Contains the request path. If query component of the URL is present, then `path` includes the query. Using the terminology of **RFC 3986**, `path` here includes `hier-part` and the `query`.

request_version

Contains the version string from the request. For example, `'HTTP/1.0'`.

headers

Holds an instance of the class specified by the *MessageClass* class variable. This instance parses and manages the headers in the HTTP request. The *parse_headers()* function from *http.client* is used to parse the headers and it requires that the HTTP request provide a valid **RFC 2822** style header.

rfile

An *io.BufferedIOBase* input stream, ready to read from the start of the optional input data.

wfile

Contains the output stream for writing a response back to the client. Proper adherence to the HTTP protocol must be used when writing to this stream in order to achieve successful interoperation with HTTP clients.

Alterado na versão 3.6: This is an *io.BufferedIOBase* stream.

BaseHTTPRequestHandler tem os seguintes atributos:

server_version

Specifies the server software version. You may want to override this. The format is multiple whitespace-separated strings, where each string is of the form name[/version]. For example, `'BaseHTTP/0.2'`.

sys_version

Contains the Python system version, in a form usable by the *version_string* method and the *server_version* class variable. For example, `'Python/1.4'`.

error_message_format

Specifies a format string that should be used by *send_error()* method for building an error response to the client. The string is filled by default with variables from *responses* based on the status code that passed to *send_error()*.

error_content_type

Specifies the Content-Type HTTP header of error responses sent to the client. The default value is `'text/html'`.

protocol_version

Specifies the HTTP version to which the server is conformant. It is sent in responses to let the client know the server's communication capabilities for future requests. If set to `'HTTP/1.1'`, the server will permit HTTP persistent connections; however, your server *must* then include an accurate `Content-Length` header (using *send_header()*) in all of its responses to clients. For backwards compatibility, the setting defaults to `'HTTP/1.0'`.

MessageClass

Specifies an *email.message.Message*-like class to parse HTTP headers. Typically, this is not overridden, and it defaults to *http.client.HTTPMessage*.

responses

This attribute contains a mapping of error code integers to two-element tuples containing a short and long message. For example, `{code: (shortmessage, longmessage)}`. The *shortmessage* is usually used as the *message* key in an error response, and *longmessage* as the *explain* key. It is used by *send_response_only()* and *send_error()* methods.

Uma instância de *BaseHTTPRequestHandler* tem os seguintes métodos:

handle()

Calls *handle_one_request()* once (or, if persistent connections are enabled, multiple times) to handle incoming HTTP requests. You should never need to override it; instead, implement appropriate `do_*()` methods.

handle_one_request()

This method will parse and dispatch the request to the appropriate `do_*()` method. You should never need to override it.

handle_expect_100()

When an HTTP/1.1 conformant server receives an `Expect: 100-continue` request header it responds back with a `100 Continue` followed by `200 OK` headers. This method can be overridden to raise an error if the server does not want the client to continue. For e.g. server can choose to send `417 Expectation Failed` as a response header and `return False`.

Adicionado na versão 3.2.

send_error(*code*, *message=None*, *explain=None*)

Sends and logs a complete error reply to the client. The numeric *code* specifies the HTTP error code, with *message* as an optional, short, human readable description of the error. The *explain* argument can be used to provide more detailed information about the error; it will be formatted using the *error_message_format* attribute and emitted, after a complete set of headers, as the response body. The *responses* attribute holds the default values for *message* and *explain* that will be used if no value is provided; for unknown codes the default value for both is the string `???`. The body will be empty if the method is HEAD or the response code is one of the following: `1xx`, `204 No Content`, `205 Reset Content`, `304 Not Modified`.

Alterado na versão 3.4: The error response includes a Content-Length header. Added the *explain* argument.

send_response(*code*, *message=None*)

Adds a response header to the headers buffer and logs the accepted request. The HTTP response line is written to the internal buffer, followed by *Server* and *Date* headers. The values for these two headers are picked up from the *version_string()* and *date_time_string()* methods, respectively. If the server does not intend to send any other headers using the *send_header()* method, then *send_response()* should be followed by an *end_headers()* call.

Alterado na versão 3.3: Headers are stored to an internal buffer and *end_headers()* needs to be called explicitly.

send_header(*keyword*, *value*)

Adds the HTTP header to an internal buffer which will be written to the output stream when either *end_headers()* or *flush_headers()* is invoked. *keyword* should specify the header keyword, with

value specifying its value. Note that, after the send_header calls are done, *end_headers()* MUST BE called in order to complete the operation.

Alterado na versão 3.2: Headers are stored in an internal buffer.

send_response_only(*code*, *message=None*)

Sends the response header only, used for the purposes when `100 Continue` response is sent by the server to the client. The headers not buffered and sent directly the output stream.If the *message* is not specified, the HTTP message corresponding the response *code* is sent.

Adicionado na versão 3.2.

end_headers()

Adds a blank line (indicating the end of the HTTP headers in the response) to the headers buffer and calls *flush_headers()*.

Alterado na versão 3.2: The buffered headers are written to the output stream.

flush_headers()

Finally send the headers to the output stream and flush the internal headers buffer.

Adicionado na versão 3.3.

log_request(*code='-'*, *size='-'*)

Logs an accepted (successful) request. *code* should specify the numeric HTTP code associated with the response. If a size of the response is available, then it should be passed as the *size* parameter.

log_error(...)

Logs an error when a request cannot be fulfilled. By default, it passes the message to *log_message()*, so it takes the same arguments (*format* and additional values).

log_message(*format*, ...)

Logs an arbitrary message to `sys.stderr`. This is typically overridden to create custom error logging mechanisms. The *format* argument is a standard printf-style format string, where the additional arguments to *log_message()* are applied as inputs to the formatting. The client ip address and current date and time are prefixed to every message logged.

version_string()

Returns the server software's version string. This is a combination of the *server_version* and *sys_version* attributes.

date_time_string(*timestamp=None*)

Returns the date and time given by *timestamp* (which must be `None` or in the format returned by *time.time()*), formatted for a message header. If *timestamp* is omitted, it uses the current date and time.

The result looks like `'Sun, 06 Nov 1994 08:49:37 GMT'`.

log_date_time_string()

Returns the current date and time, formatted for logging.

address_string()

Retorna o endereço do cliente.

Alterado na versão 3.3: Previously, a name lookup was performed. To avoid name resolution delays, it now always returns the IP address.

class http.server.**SimpleHTTPRequestHandler**(*request*, *client_address*, *server*, *directory=None*)

This class serves files from the directory *directory* and below, or the current directory if *directory* is not provided, directly mapping the directory structure to HTTP requests.

Alterado na versão 3.7: Added the *directory* parameter.

Alterado na versão 3.9: The *directory* parameter accepts a *path-like object*.

Muito do trabalho, como analisar o pedido, é feito pela classe base *BaseHTTPRequestHandler*. Esta classe implementa as funções *do_GET()* e *do_HEAD()*.

The following are defined as class-level attributes of *SimpleHTTPRequestHandler*:

server_version

This will be `"SimpleHTTP/" + __version__`, where `__version__` is defined at the module level.

extensions_map

A dictionary mapping suffixes into MIME types, contains custom overrides for the default system mappings. The mapping is used case-insensitively, and so should contain only lower-cased keys.

Alterado na versão 3.9: This dictionary is no longer filled with the default system mappings, but only contains overrides.

The *SimpleHTTPRequestHandler* class defines the following methods:

do_HEAD()

This method serves the `'HEAD'` request type: it sends the headers it would send for the equivalent `GET` request. See the *do_GET()* method for a more complete explanation of the possible headers.

do_GET()

The request is mapped to a local file by interpreting the request as a path relative to the current working directory.

If the request was mapped to a directory, the directory is checked for a file named `index.html` or `index.htm` (in that order). If found, the file's contents are returned; otherwise a directory listing is generated by calling the `list_directory()` method. This method uses *os.listdir()* to scan the directory, and returns a `404` error response if the *listdir()* fails.

If the request was mapped to a file, it is opened. Any *OSError* exception in opening the requested file is mapped to a `404`, `'File not found'` error. If there was an `'If-Modified-Since'` header in the request, and the file was not modified after this time, a `304`, `'Not Modified'` response is sent. Otherwise, the content type is guessed by calling the `guess_type()` method, which in turn uses the *extensions_map* variable, and the file contents are returned.

A `'Content-type:'` header with the guessed content type is output, followed by a `'Content-Length:'` header with the file's size and a `'Last-Modified:'` header with the file's modification time.

Then follows a blank line signifying the end of the headers, and then the contents of the file are output.

For example usage, see the implementation of the `test` function in Lib/http/server.py.

Alterado na versão 3.7: Support of the `'If-Modified-Since'` header.

The *SimpleHTTPRequestHandler* class can be used in the following manner in order to create a very basic webserver serving files relative to the current directory:

```
import http.server
import socketserver

PORT = 8000

Handler = http.server.SimpleHTTPRequestHandler

with socketserver.TCPServer(("", PORT), Handler) as httpd:
    print("serving at port", PORT)
    httpd.serve_forever()
```

SimpleHTTPRequestHandler can also be subclassed to enhance behavior, such as using different index file names by overriding the class attribute `index_pages`.

class http.server.**CGIHTTPRequestHandler**(*request*, *client_address*, *server*)

This class is used to serve either files or output of CGI scripts from the current directory and below. Note that mapping HTTP hierarchic structure to local directory structure is exactly as in *SimpleHTTPRequestHandler*.

> **Nota**
>
> CGI scripts run by the *CGIHTTPRequestHandler* class cannot execute redirects (HTTP code 302), because code 200 (script output follows) is sent prior to execution of the CGI script. This pre-empts the status code.

The class will however, run the CGI script, instead of serving it as a file, if it guesses it to be a CGI script. Only directory-based CGI are used — the other common server configuration is to treat special extensions as denoting CGI scripts.

The `do_GET()` and `do_HEAD()` functions are modified to run CGI scripts and serve the output, instead of serving files, if the request leads to somewhere below the `cgi_directories` path.

The *CGIHTTPRequestHandler* defines the following data member:

cgi_directories

This defaults to `['/cgi-bin', '/htbin']` and describes directories to treat as containing CGI scripts.

The *CGIHTTPRequestHandler* defines the following method:

do_POST()

This method serves the `'POST'` request type, only allowed for CGI scripts. Error 501, "Can only POST to CGI scripts", is output when trying to POST to a non-CGI url.

Note that CGI scripts will be run with UID of user nobody, for security reasons. Problems with the CGI script will be translated to error 403.

Deprecated since version 3.13, will be removed in version 3.15: *CGIHTTPRequestHandler* is being removed in 3.15. CGI has not been considered a good way to do things for well over a decade. This code has been unmaintained for a while now and sees very little practical use. Retaining it could lead to further *security considerations*.

22.17.1 Interface de linha de comando

http.server can also be invoked directly using the `-m` switch of the interpreter. The following example illustrates how to serve files relative to the current directory:

```
python -m http.server [OPTIONS] [port]
```

As seguintes opções são aceitas:

port

The server listens to port 8000 by default. The default can be overridden by passing the desired port number as an argument:

```
python -m http.server 9000
```

-b, --bind <address>

Specifies a specific address to which it should bind. Both IPv4 and IPv6 addresses are supported. By default, the server binds itself to all interfaces. For example, the following command causes the server to bind to localhost only:

```
python -m http.server --bind 127.0.0.1
```

Adicionado na versão 3.4.

Alterado na versão 3.8: Support IPv6 in the `--bind` option.

-d, --directory <dir>

Specifies a directory to which it should serve the files. By default, the server uses the current directory. For example, the following command uses a specific directory:

```
python -m http.server --directory /tmp/
```

Adicionado na versão 3.7.

-p, --protocol <version>

Specifies the HTTP version to which the server is conformant. By default, the server is conformant to HTTP/1.0. For example, the following command runs an HTTP/1.1 conformant server:

```
python -m http.server --protocol HTTP/1.1
```

Adicionado na versão 3.11.

--cgi

CGIHTTPRequestHandler can be enabled in the command line by passing the `--cgi` option:

```
python -m http.server --cgi
```

Deprecated since version 3.13, will be removed in version 3.15: *http.server* command line `--cgi` support is being removed because *CGIHTTPRequestHandler* is being removed.

Aviso

CGIHTTPRequestHandler and the `--cgi` command-line option are not intended for use by untrusted clients and may be vulnerable to exploitation. Always use within a secure environment.

--tls-cert

Specifies a TLS certificate chain for HTTPS connections:

```
python -m http.server --tls-cert fullchain.pem
```

Adicionado na versão 3.14.

--tls-key

Specifies a private key file for HTTPS connections.

This option requires `--tls-cert` to be specified.

Adicionado na versão 3.14.

--tls-password-file

Specifies the password file for password-protected private keys:

```
python -m http.server \
       --tls-cert cert.pem \
       --tls-key key.pem \
       --tls-password-file password.txt
```

This option requires *–tls-cert*‘ to be specified.

Adicionado na versão 3.14.

22.17.2 Considerações de segurança

SimpleHTTPRequestHandler will follow symbolic links when handling requests, this makes it possible for files outside of the specified directory to be served.

Earlier versions of Python did not scrub control characters from the log messages emitted to stderr from `python -m http.server` or the default *BaseHTTPRequestHandler* `.log_message` implementation. This could allow remote clients connecting to your server to send nefarious control codes to your terminal.

Alterado na versão 3.12: Control characters are scrubbed in stderr logs.

22.18 http.cookies — HTTP state management

Código-fonte: Lib/http/cookies.py

The *http.cookies* module defines classes for abstracting the concept of cookies, an HTTP state management mechanism. It supports both simple string-only cookies, and provides an abstraction for having any serializable data-type as cookie value.

The module formerly strictly applied the parsing rules described in the **RFC 2109** and **RFC 2068** specifications. It has since been discovered that MSIE 3.0x didn't follow the character rules outlined in those specs; many current-day browsers and servers have also relaxed parsing rules when it comes to cookie handling. As a result, this module now uses parsing rules that are a bit less strict than they once were.

The character set, *string.ascii_letters*, *string.digits* and `!#$%&'*+-.^_`|~:` denote the set of valid characters allowed by this module in a cookie name (as *key*).

Alterado na versão 3.3: Allowed ':' as a valid cookie name character.

Nota

On encountering an invalid cookie, *CookieError* is raised, so if your cookie data comes from a browser you should always prepare for invalid data and catch *CookieError* on parsing.

exception http.cookies.**CookieError**

Exception failing because of **RFC 2109** invalidity: incorrect attributes, incorrect *Set-Cookie* header, etc.

class http.cookies.**BaseCookie**([*input*])

This class is a dictionary-like object whose keys are strings and whose values are *Morsel* instances. Note that upon setting a key to a value, the value is first converted to a *Morsel* containing the key and the value.

If *input* is given, it is passed to the *load()* method.

class http.cookies.**SimpleCookie**([*input*])

This class derives from *BaseCookie* and overrides *value_decode()* and *value_encode()*. SimpleCookie supports strings as cookie values. When setting the value, SimpleCookie calls the builtin *str()* to convert the value to a string. Values received from HTTP are kept as strings.

Ver também

Módulo *http.cookiejar*
HTTP cookie handling for web *clients*. The *http.cookiejar* and *http.cookies* modules do not depend on each other.

RFC 2109 - HTTP State Management Mechanism
This is the state management specification implemented by this module.

22.18.1 Objetos Cookie

BaseCookie.**value_decode**(*val*)

Return a tuple `(real_value, coded_value)` from a string representation. `real_value` can be any type. This method does no decoding in *BaseCookie* — it exists so it can be overridden.

BaseCookie.**value_encode**(*val*)

Return a tuple `(real_value, coded_value)`. *val* can be any type, but `coded_value` will always be converted to a string. This method does no encoding in *BaseCookie* — it exists so it can be overridden.

In general, it should be the case that *value_encode()* and *value_decode()* are inverses on the range of *value_decode*.

BaseCookie.**output**(*attrs=None*, *header='Set-Cookie:'*, *sep='\r\n'*)

Return a string representation suitable to be sent as HTTP headers. *attrs* and *header* are sent to each *Morsel*'s *output()* method. *sep* is used to join the headers together, and is by default the combination `'\r\n'` (CRLF).

BaseCookie.**js_output**(*attrs=None*)

Return an embeddable JavaScript snippet, which, if run on a browser which supports JavaScript, will act the same as if the HTTP headers was sent.

The meaning for *attrs* is the same as in *output()*.

BaseCookie.**load**(*rawdata*)

If *rawdata* is a string, parse it as an `HTTP_COOKIE` and add the values found there as *Morsel*s. If it is a dictionary, it is equivalent to:

```
for k, v in rawdata.items():
    cookie[k] = v
```

22.18.2 Objetos Morsel

class http.cookies.**Morsel**

Abstract a key/value pair, which has some **RFC 2109** attributes.

Morsels are dictionary-like objects, whose set of keys is constant — the valid **RFC 2109** attributes, which are:

- **expires**
- **path**
- **comment**
- **domain**
- **max-age**
- **secure**
- **version**
- **httponly**
- **samesite**
- **partitioned**

The attribute *httponly* specifies that the cookie is only transferred in HTTP requests, and is not accessible through JavaScript. This is intended to mitigate some forms of cross-site scripting.

The attribute *samesite* controls when the browser sends the cookie with cross-site requests. This helps to mitigate CSRF attacks. Valid values are "Strict" (only sent with same-site requests), "Lax" (sent with same--site requests and top-level navigations), and "None" (sent with same-site and cross-site requests). When using "None", the "secure" attribute must also be set, as required by modern browsers.

The attribute *partitioned* indicates to user agents that these cross-site cookies *should* only be available in the same top-level context that the cookie was first set in. For this to be accepted by the user agent, you **must** also set `Secure`.

In addition, it is recommended to use the `__Host` prefix when setting partitioned cookies to make them bound to the hostname and not the registrable domain. Read CHIPS (Cookies Having Independent Partitioned State) for full details and examples.

The keys are case-insensitive and their default value is `''`.

Alterado na versão 3.5: `__eq__()` now takes *key* and *value* into account.

Alterado na versão 3.7: Attributes *key*, *value* and *coded_value* are read-only. Use *set()* for setting them.

Alterado na versão 3.8: Added support for the *samesite* attribute.

Alterado na versão 3.14: Added support for the *partitioned* attribute.

`Morsel.`**`value`**

O valor do cookie.

`Morsel.`**`coded_value`**

The encoded value of the cookie — this is what should be sent.

`Morsel.`**`key`**

O nome do cookie.

`Morsel.`**`set`**(*key*, *value*, *coded_value*)

Set the *key*, *value* and *coded_value* attributes.

`Morsel.`**`isReservedKey`**(*K*)

Whether *K* is a member of the set of keys of a *Morsel*.

`Morsel.`**`output`**(*attrs=None*, *header='Set-Cookie:'*)

Return a string representation of the Morsel, suitable to be sent as an HTTP header. By default, all the attributes are included, unless *attrs* is given, in which case it should be a list of attributes to use. *header* is by default `"Set-Cookie:"`.

`Morsel.`**`js_output`**(*attrs=None*)

Return an embeddable JavaScript snippet, which, if run on a browser which supports JavaScript, will act the same as if the HTTP header was sent.

The meaning for *attrs* is the same as in *output()*.

`Morsel.`**`OutputString`**(*attrs=None*)

Return a string representing the Morsel, without any surrounding HTTP or JavaScript.

The meaning for *attrs* is the same as in *output()*.

`Morsel.`**`update`**(*values*)

Update the values in the Morsel dictionary with the values in the dictionary *values*. Raise an error if any of the keys in the *values* dict is not a valid **RFC 2109** attribute.

Alterado na versão 3.5: an error is raised for invalid keys.

`Morsel.`**`copy`**(*value*)

Return a shallow copy of the Morsel object.

Alterado na versão 3.5: return a Morsel object instead of a dict.

`Morsel.`**`setdefault`**(*key*, *value=None*)

Raise an error if key is not a valid **RFC 2109** attribute, otherwise behave the same as *dict.setdefault()*.

22.18.3 Exemplo

The following example demonstrates how to use the *http.cookies* module.

```
>>> from http import cookies
>>> C = cookies.SimpleCookie()
>>> C["fig"] = "newton"
>>> C["sugar"] = "wafer"
>>> print(C) # generate HTTP headers
Set-Cookie: fig=newton
Set-Cookie: sugar=wafer
>>> print(C.output()) # same thing
Set-Cookie: fig=newton
Set-Cookie: sugar=wafer
>>> C = cookies.SimpleCookie()
>>> C["rocky"] = "road"
>>> C["rocky"]["path"] = "/cookie"
```

(continua na próxima página)

(continuação da página anterior)

```
>>> print(C.output(header="Cookie:"))
Cookie: rocky=road; Path=/cookie
>>> print(C.output(attrs=[], header="Cookie:"))
Cookie: rocky=road
>>> C = cookies.SimpleCookie()
>>> C.load("chips=ahoy; vienna=finger") # load from a string (HTTP header)
>>> print(C)
Set-Cookie: chips=ahoy
Set-Cookie: vienna=finger
>>> C = cookies.SimpleCookie()
>>> C.load('keebler="E=everybody; L=\\"Loves\\"; fudge=\\012;";')
>>> print(C)
Set-Cookie: keebler="E=everybody; L=\"Loves\"; fudge=\012;"
>>> C = cookies.SimpleCookie()
>>> C["oreo"] = "doublestuff"
>>> C["oreo"]["path"] = "/"
>>> print(C)
Set-Cookie: oreo=doublestuff; Path=/
>>> C = cookies.SimpleCookie()
>>> C["twix"] = "none for you"
>>> C["twix"].value
'none for you'
>>> C = cookies.SimpleCookie()
>>> C["number"] = 7 # equivalent to C["number"] = str(7)
>>> C["string"] = "seven"
>>> C["number"].value
'7'
>>> C["string"].value
'seven'
>>> print(C)
Set-Cookie: number=7
Set-Cookie: string=seven
```

22.19 `http.cookiejar` — Cookie handling for HTTP clients

Código-fonte: Lib/http/cookiejar.py

The *http.cookiejar* module defines classes for automatic handling of HTTP cookies. It is useful for accessing web sites that require small pieces of data – *cookies* – to be set on the client machine by an HTTP response from a web server, and then returned to the server in later HTTP requests.

Both the regular Netscape cookie protocol and the protocol defined by **RFC 2965** are handled. RFC 2965 handling is switched off by default. **RFC 2109** cookies are parsed as Netscape cookies and subsequently treated either as Netscape or RFC 2965 cookies according to the 'policy' in effect. Note that the great majority of cookies on the internet are Netscape cookies. *http.cookiejar* attempts to follow the de-facto Netscape cookie protocol (which differs substantially from that set out in the original Netscape specification), including taking note of the `max-age` and `port` cookie-attributes introduced with RFC 2965.

Nota

The various named parameters found in *Set-Cookie* and *Set-Cookie2* headers (eg. `domain` and `expires`) are conventionally referred to as *attributes*. To distinguish them from Python attributes, the documentation for this module uses the term *cookie-attribute* instead.

The module defines the following exception:

exception http.cookiejar.**LoadError**

Instances of *FileCookieJar* raise this exception on failure to load cookies from a file. *LoadError* is a subclass of *OSError*.

Alterado na versão 3.3: *LoadError* used to be a subtype of *IOError*, which is now an alias of *OSError*.

As seguintes classes são disponibilizadas:

class http.cookiejar.**CookieJar**(*policy=None*)

policy is an object implementing the *CookiePolicy* interface.

The *CookieJar* class stores HTTP cookies. It extracts cookies from HTTP requests, and returns them in HTTP responses. *CookieJar* instances automatically expire contained cookies when necessary. Subclasses are also responsible for storing and retrieving cookies from a file or database.

class http.cookiejar.**FileCookieJar**(*filename=None*, *delayload=None*, *policy=None*)

policy is an object implementing the *CookiePolicy* interface. For the other arguments, see the documentation for the corresponding attributes.

A *CookieJar* which can load cookies from, and perhaps save cookies to, a file on disk. Cookies are **NOT** loaded from the named file until either the *load()* or *revert()* method is called. Subclasses of this class are documented in section *FileCookieJar subclasses and co-operation with web browsers*.

This should not be initialized directly – use its subclasses below instead.

Alterado na versão 3.8: The filename parameter supports a *path-like object*.

class http.cookiejar.**CookiePolicy**

This class is responsible for deciding whether each cookie should be accepted from / returned to the server.

class http.cookiejar.**DefaultCookiePolicy**(*blocked_domains=None*, *allowed_domains=None*, *netscape=True*, *rfc2965=False*, *rfc2109_as_netscape=None*, *hide_cookie2=False*, *strict_domain=False*, *strict_rfc2965_unverifiable=True*, *strict_ns_unverifiable=False*, *strict_ns_domain=DefaultCookiePolicy.DomainLiberal*, *strict_ns_set_initial_dollar=False*, *strict_ns_set_path=False*, *secure_protocols=('https', 'wss')*)

Constructor arguments should be passed as keyword arguments only. *blocked_domains* is a sequence of domain names that we never accept cookies from, nor return cookies to. *allowed_domains* if not *None*, this is a sequence of the only domains for which we accept and return cookies. *secure_protocols* is a sequence of protocols for which secure cookies can be added to. By default *https* and *wss* (secure websocket) are considered secure protocols. For all other arguments, see the documentation for *CookiePolicy* and *DefaultCookiePolicy* objects.

DefaultCookiePolicy implements the standard accept / reject rules for Netscape and **RFC 2965** cookies. By default, **RFC 2109** cookies (ie. cookies received in a *Set-Cookie* header with a version cookie-attribute of 1) are treated according to the RFC 2965 rules. However, if RFC 2965 handling is turned off or *rfc2109_as_netscape* is `True`, RFC 2109 cookies are 'downgraded' by the *CookieJar* instance to Netscape cookies, by setting the `version` attribute of the *Cookie* instance to 0. *DefaultCookiePolicy* also provides some parameters to allow some fine-tuning of policy.

class http.cookiejar.**Cookie**

This class represents Netscape, **RFC 2109** and **RFC 2965** cookies. It is not expected that users of *http.cookiejar* construct their own *Cookie* instances. Instead, if necessary, call `make_cookies()` on a *CookieJar* instance.

Ver também

Módulo ***urllib.request***
URL opening with automatic cookie handling.

Module *http.cookies*
HTTP cookie classes, principally useful for server-side code. The *http.cookiejar* and *http.cookies* modules do not depend on each other.

https://curl.se/rfc/cookie_spec.html
The specification of the original Netscape cookie protocol. Though this is still the dominant protocol, the 'Netscape cookie protocol' implemented by all the major browsers (and *http.cookiejar*) only bears a passing resemblance to the one sketched out in `cookie_spec.html`.

RFC 2109 - HTTP State Management Mechanism
Obsoleted by **RFC 2965**. Uses *Set-Cookie* with version=1.

RFC 2965 - HTTP State Management Mechanism
The Netscape protocol with the bugs fixed. Uses *Set-Cookie2* in place of *Set-Cookie*. Not widely used.

https://kristol.org/cookie/errata.html
Unfinished errata to **RFC 2965**.

RFC 2964 - Use of HTTP State Management

22.19.1 CookieJar and FileCookieJar Objects

CookieJar objects support the *iterator* protocol for iterating over contained *Cookie* objects.

CookieJar has the following methods:

`CookieJar.`**`add_cookie_header`**(*request*)
Add correct *Cookie* header to *request*.

If policy allows (ie. the `rfc2965` and `hide_cookie2` attributes of the *CookieJar*'s *CookiePolicy* instance are true and false respectively), the *Cookie2* header is also added when appropriate.

The *request* object (usually a *urllib.request.Request* instance) must support the methods `get_full_url()`, `has_header()`, `get_header()`, `header_items()`, `add_unredirected_header()` and the attributes `host`, `type`, `unverifiable` and `origin_req_host` as documented by *urllib.request*.

Alterado na versão 3.3: *request* object needs `origin_req_host` attribute. Dependency on a deprecated method `get_origin_req_host()` has been removed.

`CookieJar.`**`extract_cookies`**(*response*, *request*)
Extract cookies from HTTP *response* and store them in the *CookieJar*, where allowed by policy.

The *CookieJar* will look for allowable *Set-Cookie* and *Set-Cookie2* headers in the *response* argument, and store cookies as appropriate (subject to the *CookiePolicy.set_ok()* method's approval).

The *response* object (usually the result of a call to *urllib.request.urlopen()*, or similar) should support an `info()` method, which returns an *email.message.Message* instance.

The *request* object (usually a *urllib.request.Request* instance) must support the method `get_full_url()` and the attributes `host`, `unverifiable` and `origin_req_host`, as documented by *urllib.request*. The request is used to set default values for cookie-attributes as well as for checking that the cookie is allowed to be set.

Alterado na versão 3.3: *request* object needs `origin_req_host` attribute. Dependency on a deprecated method `get_origin_req_host()` has been removed.

`CookieJar.`**`set_policy`**(*policy*)
Set the *CookiePolicy* instance to be used.

`CookieJar.`**`make_cookies`**(*response*, *request*)
Return sequence of *Cookie* objects extracted from *response* object.

See the documentation for *extract_cookies()* for the interfaces required of the *response* and *request* arguments.

CookieJar.**set_cookie_if_ok**(*cookie*, *request*)

Set a *Cookie* if policy says it's OK to do so.

CookieJar.**set_cookie**(*cookie*)

Set a *Cookie*, without checking with policy to see whether or not it should be set.

CookieJar.**clear**([*domain*[, *path*[, *name*]]])

Clear some cookies.

If invoked without arguments, clear all cookies. If given a single argument, only cookies belonging to that *domain* will be removed. If given two arguments, cookies belonging to the specified *domain* and URL *path* are removed. If given three arguments, then the cookie with the specified *domain*, *path* and *name* is removed.

Raises *KeyError* if no matching cookie exists.

CookieJar.**clear_session_cookies**()

Descarta os cookies de sessão.

Discards all contained cookies that have a true `discard` attribute (usually because they had either no `max-age` or `expires` cookie-attribute, or an explicit `discard` cookie-attribute). For interactive browsers, the end of a session usually corresponds to closing the browser window.

Note that the `save()` method won't save session cookies anyway, unless you ask otherwise by passing a true *ignore_discard* argument.

FileCookieJar implements the following additional methods:

FileCookieJar.**save**(*filename=None*, *ignore_discard=False*, *ignore_expires=False*)

Save cookies to a file.

This base class raises *NotImplementedError*. Subclasses may leave this method unimplemented.

filename is the name of file in which to save cookies. If *filename* is not specified, `self.filename` is used (whose default is the value passed to the constructor, if any); if `self.filename` is *None*, *ValueError* is raised.

ignore_discard: save even cookies set to be discarded. *ignore_expires*: save even cookies that have expired

The file is overwritten if it already exists, thus wiping all the cookies it contains. Saved cookies can be restored later using the *load()* or *revert()* methods.

FileCookieJar.**load**(*filename=None*, *ignore_discard=False*, *ignore_expires=False*)

Load cookies from a file.

Old cookies are kept unless overwritten by newly loaded ones.

Arguments are as for *save()*.

The named file must be in the format understood by the class, or *LoadError* will be raised. Also, *OSError* may be raised, for example if the file does not exist.

Alterado na versão 3.3: *IOError* costumava ser levantado, agora ele é um codinome para *OSError*.

FileCookieJar.**revert**(*filename=None*, *ignore_discard=False*, *ignore_expires=False*)

Clear all cookies and reload cookies from a saved file.

revert() can raise the same exceptions as *load()*. If there is a failure, the object's state will not be altered.

FileCookieJar instances have the following public attributes:

FileCookieJar.**filename**

Filename of default file in which to keep cookies. This attribute may be assigned to.

FileCookieJar.**delayload**

If true, load cookies lazily from disk. This attribute should not be assigned to. This is only a hint, since this only affects performance, not behaviour (unless the cookies on disk are changing). A *CookieJar* object may ignore it. None of the *FileCookieJar* classes included in the standard library lazily loads cookies.

22.19.2 FileCookieJar subclasses and co-operation with web browsers

The following *CookieJar* subclasses are provided for reading and writing.

class http.cookiejar.**MozillaCookieJar**(*filename=None*, *delayload=None*, *policy=None*)

A *FileCookieJar* that can load from and save cookies to disk in the Mozilla `cookies.txt` file format (which is also used by curl and the Lynx and Netscape browsers).

> **Nota**
>
> This loses information about **RFC 2965** cookies, and also about newer or non-standard cookie-attributes such as `port`.

> **Aviso**
>
> Back up your cookies before saving if you have cookies whose loss / corruption would be inconvenient (there are some subtleties which may lead to slight changes in the file over a load / save round-trip).

Also note that cookies saved while Mozilla is running will get clobbered by Mozilla.

class http.cookiejar.**LWPCookieJar**(*filename=None*, *delayload=None*, *policy=None*)

A *FileCookieJar* that can load from and save cookies to disk in format compatible with the libwww-perl library's `Set-Cookie3` file format. This is convenient if you want to store cookies in a human-readable file.

Alterado na versão 3.8: The filename parameter supports a *path-like object*.

22.19.3 Objeto CookiePolicy

Objects implementing the *CookiePolicy* interface have the following methods:

CookiePolicy.**set_ok**(*cookie*, *request*)

Return boolean value indicating whether cookie should be accepted from server.

cookie is a *Cookie* instance. *request* is an object implementing the interface defined by the documentation for *CookieJar.extract_cookies()*.

CookiePolicy.**return_ok**(*cookie*, *request*)

Return boolean value indicating whether cookie should be returned to server.

cookie is a *Cookie* instance. *request* is an object implementing the interface defined by the documentation for *CookieJar.add_cookie_header()*.

CookiePolicy.**domain_return_ok**(*domain*, *request*)

Return `False` if cookies should not be returned, given cookie domain.

This method is an optimization. It removes the need for checking every cookie with a particular domain (which might involve reading many files). Returning true from *domain_return_ok()* and *path_return_ok()* leaves all the work to *return_ok()*.

If *domain_return_ok()* returns true for the cookie domain, *path_return_ok()* is called for the cookie path. Otherwise, *path_return_ok()* and *return_ok()* are never called for that cookie domain. If *path_return_ok()* returns true, *return_ok()* is called with the *Cookie* object itself for a full check. Otherwise, *return_ok()* is never called for that cookie path.

Note that *domain_return_ok()* is called for every *cookie* domain, not just for the *request* domain. For example, the function might be called with both ".example.com" and "www.example.com" if the request domain is "www.example.com". The same goes for *path_return_ok()*.

The *request* argument is as documented for *return_ok()*.

CookiePolicy.**path_return_ok**(*path*, *request*)

Return False if cookies should not be returned, given cookie path.

See the documentation for *domain_return_ok()*.

In addition to implementing the methods above, implementations of the *CookiePolicy* interface must also supply the following attributes, indicating which protocols should be used, and how. All of these attributes may be assigned to.

CookiePolicy.**netscape**

Implement Netscape protocol.

CookiePolicy.**rfc2965**

Implement **RFC 2965** protocol.

CookiePolicy.**hide_cookie2**

Don't add *Cookie2* header to requests (the presence of this header indicates to the server that we understand **RFC 2965** cookies).

The most useful way to define a *CookiePolicy* class is by subclassing from *DefaultCookiePolicy* and overriding some or all of the methods above. *CookiePolicy* itself may be used as a 'null policy' to allow setting and receiving any and all cookies (this is unlikely to be useful).

22.19.4 DefaultCookiePolicy Objects

Implements the standard rules for accepting and returning cookies.

Both **RFC 2965** and Netscape cookies are covered. RFC 2965 handling is switched off by default.

The easiest way to provide your own policy is to override this class and call its methods in your overridden implementations before adding your own additional checks:

```
import http.cookiejar
class MyCookiePolicy(http.cookiejar.DefaultCookiePolicy):
    def set_ok(self, cookie, request):
        if not http.cookiejar.DefaultCookiePolicy.set_ok(self, cookie, request):
            return False
        if i_dont_want_to_store_this_cookie(cookie):
            return False
        return True
```

In addition to the features required to implement the *CookiePolicy* interface, this class allows you to block and allow domains from setting and receiving cookies. There are also some strictness switches that allow you to tighten up the rather loose Netscape protocol rules a little bit (at the cost of blocking some benign cookies).

A domain blocklist and allowlist is provided (both off by default). Only domains not in the blocklist and present in the allowlist (if the allowlist is active) participate in cookie setting and returning. Use the *blocked_domains* constructor argument, and blocked_domains() and set_blocked_domains() methods (and the corresponding argument and methods for *allowed_domains*). If you set an allowlist, you can turn it off again by setting it to *None*.

Domains in block or allow lists that do not start with a dot must equal the cookie domain to be matched. For example, "example.com" matches a blocklist entry of "example.com", but "www.example.com" does not. Domains that do start with a dot are matched by more specific domains too. For example, both "www.example.com" and "www.coyote.example.com" match ".example.com" (but "example.com" itself does not). IP addresses are an exception, and must match exactly. For example, if blocked_domains contains "192.168.1.2" and ".168.1.2", 192.168.1.2 is blocked, but 193.168.1.2 is not.

DefaultCookiePolicy implements the following additional methods:

DefaultCookiePolicy.**blocked_domains**()

Return the sequence of blocked domains (as a tuple).

DefaultCookiePolicy.**set_blocked_domains**(*blocked_domains*)

Set the sequence of blocked domains.

DefaultCookiePolicy.**is_blocked**(*domain*)

Return `True` if *domain* is on the blocklist for setting or receiving cookies.

DefaultCookiePolicy.**allowed_domains**()

Return *None*, or the sequence of allowed domains (as a tuple).

DefaultCookiePolicy.**set_allowed_domains**(*allowed_domains*)

Set the sequence of allowed domains, or *None*.

DefaultCookiePolicy.**is_not_allowed**(*domain*)

Return `True` if *domain* is not on the allowlist for setting or receiving cookies.

DefaultCookiePolicy instances have the following attributes, which are all initialised from the constructor arguments of the same name, and which may all be assigned to.

DefaultCookiePolicy.**rfc2109_as_netscape**

If true, request that the *CookieJar* instance downgrade **RFC 2109** cookies (ie. cookies received in a *Set-Cookie* header with a version cookie-attribute of 1) to Netscape cookies by setting the version attribute of the *Cookie* instance to 0. The default value is *None*, in which case RFC 2109 cookies are downgraded if and only if **RFC 2965** handling is turned off. Therefore, RFC 2109 cookies are downgraded by default.

General strictness switches:

DefaultCookiePolicy.**strict_domain**

Don't allow sites to set two-component domains with country-code top-level domains like `.co.uk`, `.gov.uk`, `.co.nz`.etc. This is far from perfect and isn't guaranteed to work!

RFC 2965 protocol strictness switches:

DefaultCookiePolicy.**strict_rfc2965_unverifiable**

Follow **RFC 2965** rules on unverifiable transactions (usually, an unverifiable transaction is one resulting from a redirect or a request for an image hosted on another site). If this is false, cookies are *never* blocked on the basis of verifiability

Netscape protocol strictness switches:

DefaultCookiePolicy.**strict_ns_unverifiable**

Apply **RFC 2965** rules on unverifiable transactions even to Netscape cookies.

DefaultCookiePolicy.**strict_ns_domain**

Flags indicating how strict to be with domain-matching rules for Netscape cookies. See below for acceptable values.

DefaultCookiePolicy.**strict_ns_set_initial_dollar**

Ignore cookies in Set-Cookie: headers that have names starting with `'$'`.

DefaultCookiePolicy.**strict_ns_set_path**

Don't allow setting cookies whose path doesn't path-match request URI.

strict_ns_domain is a collection of flags. Its value is constructed by or-ing together (for example, `DomainStrictNoDots|DomainStrictNonDomain` means both flags are set).

DefaultCookiePolicy.**DomainStrictNoDots**

When setting cookies, the 'host prefix' must not contain a dot (eg. `www.foo.bar.com` can't set a cookie for `.bar.com`, because `www.foo` contains a dot).

`DefaultCookiePolicy.`**`DomainStrictNonDomain`**

Cookies that did not explicitly specify a `domain` cookie-attribute can only be returned to a domain equal to the domain that set the cookie (eg. `spam.example.com` won't be returned cookies from `example.com` that had no `domain` cookie-attribute).

`DefaultCookiePolicy.`**`DomainRFC2965Match`**

When setting cookies, require a full **RFC 2965** domain-match.

The following attributes are provided for convenience, and are the most useful combinations of the above flags:

`DefaultCookiePolicy.`**`DomainLiberal`**

Equivalent to 0 (ie. all of the above Netscape domain strictness flags switched off).

`DefaultCookiePolicy.`**`DomainStrict`**

Equivalente a `DomainStrictNoDots|DomainStrictNonDomain`.

22.19.5 Objetos Cookie

`Cookie` instances have Python attributes roughly corresponding to the standard cookie-attributes specified in the various cookie standards. The correspondence is not one-to-one, because there are complicated rules for assigning default values, because the `max-age` and `expires` cookie-attributes contain equivalent information, and because **RFC 2109** cookies may be 'downgraded' by *`http.cookiejar`* from version 1 to version 0 (Netscape) cookies.

Assignment to these attributes should not be necessary other than in rare circumstances in a *`CookiePolicy`* method. The class does not enforce internal consistency, so you should know what you're doing if you do that.

`Cookie.`**`version`**

Integer or *`None`*. Netscape cookies have *`version`* 0. **RFC 2965** and **RFC 2109** cookies have a `version` cookie-attribute of 1. However, note that *`http.cookiejar`* may 'downgrade' RFC 2109 cookies to Netscape cookies, in which case *`version`* is 0.

`Cookie.`**`name`**

Cookie name (a string).

`Cookie.`**`value`**

Cookie value (a string), or *`None`*.

`Cookie.`**`port`**

String representing a port or a set of ports (eg. '80', or '80,8080'), or *`None`*.

`Cookie.`**`domain`**

Cookie domain (a string).

`Cookie.`**`path`**

Cookie path (a string, eg. `'/acme/rocket_launchers'`).

`Cookie.`**`secure`**

`True` if cookie should only be returned over a secure connection.

`Cookie.`**`expires`**

Integer expiry date in seconds since epoch, or *`None`*. See also the *`is_expired()`* method.

`Cookie.`**`discard`**

`True` if this is a session cookie.

`Cookie.`**`comment`**

String comment from the server explaining the function of this cookie, or *`None`*.

`Cookie.`**`comment_url`**

URL linking to a comment from the server explaining the function of this cookie, or *`None`*.

`Cookie.`**`rfc2109`**

`True` if this cookie was received as an **RFC 2109** cookie (ie. the cookie arrived in a *`Set-Cookie`* header, and the value of the Version cookie-attribute in that header was 1). This attribute is provided because *`http.cookiejar`* may 'downgrade' RFC 2109 cookies to Netscape cookies, in which case *`version`* is 0.

`Cookie.`**`port_specified`**

`True` if a port or set of ports was explicitly specified by the server (in the *`Set-Cookie`* / *`Set-Cookie2`* header).

`Cookie.`**`domain_specified`**

`True` if a domain was explicitly specified by the server.

`Cookie.`**`domain_initial_dot`**

`True` if the domain explicitly specified by the server began with a dot (`'.'`).

Cookies may have additional non-standard cookie-attributes. These may be accessed using the following methods:

`Cookie.`**`has_nonstandard_attr`**(*name*)

Return `True` if cookie has the named cookie-attribute.

`Cookie.`**`get_nonstandard_attr`**(*name*, *default=None*)

If cookie has the named cookie-attribute, return its value. Otherwise, return *default*.

`Cookie.`**`set_nonstandard_attr`**(*name*, *value*)

Set the value of the named cookie-attribute.

The *`Cookie`* class also defines the following method:

`Cookie.`**`is_expired`**(*now=None*)

`True` if cookie has passed the time at which the server requested it should expire. If *now* is given (in seconds since the epoch), return whether the cookie has expired at the specified time.

22.19.6 Exemplos

The first example shows the most common usage of *`http.cookiejar`*:

```
import http.cookiejar, urllib.request
cj = http.cookiejar.CookieJar()
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))
r = opener.open("http://example.com/")
```

This example illustrates how to open a URL using your Netscape, Mozilla, or Lynx cookies (assumes Unix/Netscape convention for location of the cookies file):

```
import os, http.cookiejar, urllib.request
cj = http.cookiejar.MozillaCookieJar()
cj.load(os.path.join(os.path.expanduser("~"), ".netscape", "cookies.txt"))
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))
r = opener.open("http://example.com/")
```

The next example illustrates the use of *`DefaultCookiePolicy`*. Turn on **RFC 2965** cookies, be more strict about domains when setting and returning Netscape cookies, and block some domains from setting cookies or having them returned:

```
import urllib.request
from http.cookiejar import CookieJar, DefaultCookiePolicy
policy = DefaultCookiePolicy(
    rfc2965=True, strict_ns_domain=Policy.DomainStrict,
    blocked_domains=["ads.net", ".ads.net"])
cj = CookieJar(policy)
```

(continua na próxima página)

(continuação da página anterior)

```
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))
r = opener.open("http://example.com/")
```

22.20 xmlrpc — Módulos de servidor e cliente XMLRPC

XML-RPC é um método de chamada de procedimento remoto que usa XML passado via HTTP como um transporte. Com ele, um cliente pode chamar métodos com parâmetros em um servidor remoto (o servidor é nomeado por um URI) e receber dados estruturados.

`xmlrpc` é um pacote que coleta módulos de servidor e de cliente que implementam o XML-RPC. Os módulos são:

- *xmlrpc.client*
- *xmlrpc.server*

22.21 xmlrpc.client — Cliente XML-RPC

Código-fonte: Lib/xmlrpc/client.py

XML-RPC é um método de chamada remota de métodos que usa XML usando HTTP(S) como transporte. Com ele, um cliente pode chamar métodos com parâmetros em um servidor remoto (o servidor é nomeado por um URI) e receber de volta dados estruturados. Este módulo oferece suporte à escrita de código de clientes XML-RPC; ele lida com todos os detalhes da tradução entre Python objetos e XML.

> **Aviso**
>
> O módulo *xmlrpc.client* não é seguro contra dados criados com códigos maliciosos. Se você precisar analisar dados não confiáveis ou não autenticados, consulte *Segurança no XML*.

Alterado na versão 3.5: Para URIs com HTTPS, *xmlrpc.client* agora faz todas as validações de certificado e nome do servidor necessárias por padrão.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

class xmlrpc.client.**ServerProxy**(*uri*, *transport=None*, *encoding=None*, *verbose=False*, *allow_none=False*, *use_datetime=False*, *use_builtin_types=False*, *, *headers=()*, *context=None*)

Uma instância de *ServerProxy* é um objeto que gerencia a comunicação com um servidor XML-RPC remoto. O primeiro argumento obrigatório é uma URI (Uniform Resource Indicator - Indicador de Recurso Uniforme) e normalmente vai ser a URL do servidor. O segundo parâmetro, opcional, é uma instância de um factory de transporte; por padrão é uma instância de `SafeTransport` para URLs https e uma instância de `Transport` caso contrário. O terceiro parâmetro, opcional, é o encoding, por padrão sendo UTF-8. O quarto argumento, opcional, é a flag de debug.

Os seguinte parâmetros coordenam o uso da instância de proxy retornada. Se *alow_none* for verdadeiro, a constante `None` do Python será traduzida para XML; o comportamento padrão é que `None` levante uma *TypeError*, Isso é uma extensão comum da especificação do XML-RPC, mas não é suportado por todos os clientes e servidores, veja http://ontosys.com/xml-rpc/extensions.php para uma descrição. A flag *use_builtin_types* pode ser usada para que valores de data e hora sejam representados como objetos *datetime.datetime* e dados binários representados com objetos *bytes*; essa flag é False por padrão. Objetos *datetime.datetime*, *bytes* e *bytearray* podem ser usados nas chamadas. O parâmetro *headers* é uma sequência opcional de headers a serem enviados em cada requisição, representados por uma sequência de tuplas de dois valores representando o nome do header e seu valor (como `[('Header-Name', 'value')]`).

Se uma URL HTTPS é fornecida, *context* pode ser *ssl.SSLContext* e configura as definições SSL da conexão HTTPS subjacente. A flag *use_datetime* é obsoleta e é similar a *use_builtin_types* mas se aplica somente a valores de data e hora.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

Alterado na versão 3.8: O parâmetro *headers* foi adicionado.

Both the HTTP and HTTPS transports support the URL syntax extension for HTTP Basic Authentication: `http://user:pass@host:port/path`. The `user:pass` portion will be base64-encoded as an HTTP 'Authorization' header, and sent to the remote server as part of the connection process when invoking an XML--RPC method. You only need to use this if the remote server requires a Basic Authentication user and password.

A instância retornada é um objeto proxy com métodos que podem ser usados para invocar a chamada RPC correspondendo no servidor remoto. Se o servidor remoto suportar a API de instrospecção, o proxy também pode ser usado para perguntar ao servidor remoto pelos métodos que ele suporta (descoberta de serviço) e recuperar outros meta-dados associados com o servidor.

Os tipos que são conformáveis (por exemplo, que podem ser convertidos para XML) incluem os seguintes (e exceto onde indicado, eles não são convertidos como o mesmo tipo Python):

Tipo XML-RPC	Tipo em Python
`boolean`	*bool*
`int`, `i1`, `i2`, `i4`, `i8` ou `biginteger`	*int* no intervalo de -2147483648 a 2147483647. Os valores recebem a tag `<int>`.
`double` ou `float`	*float*. Os valores recebem a tag `<double>`.
`string`	*str*
`array`	*list* ou *tuple* contendo elementos conformáveis. As matrizes são retornadas como *lists*.
`struct`	*dict*. Chaves devem ser strings, valores podem ser qualquer tipo conformáveis. Objetos de classes definidas pelo usuário pode ser usadas; apenas o atributo `__dict__` delas é transmitido.
`dateTime.iso8601`	*DateTime* ou *datetime.datetime*. O tipo retornado depende de volumes de sinalizadores *use_builtin_types* e *use_datetime*.
`base64`	*Binary*, *bytes* ou *bytearray*. O tipo retornado depende do valor da flag *use_builtin_types*.
`nil`	A constante `None`. A passagem é permitida somente se *allow_none* for verdadeiro.
`bigdecimal`	*decimal.Decimal*. Somente tipo retornado.

Esta é a lista completa de tipos suportados por XML-RPC. Chamadas de método podem também levantar uma instância de *Fault*, usado para que o servidor XML-RPC indique erros, ou *ProtocolError* para indicar erros na camada HTTP/HTTPS. Tnato *Fault* quanto *ProtocolError* derivam da classe base `Error`. Observe que o módulo de cliente xmlrpc atualmente não converte instância de subclasses dos tipos built-in.

Ao passar strings, os caracteres especiais para XML, como `<`, `>` e `&`, serão automaticamente escapados. No entanto, é responsabilidade do chamador garantir que o string esteja livre de caracteres que não são permitidos em XML, como os caracteres de controle com valores ASCII entre 0 e 31 (exceto, é claro, tabulação, nova linha e retorno de carro); se isso não for feito, resultará em uma solicitação XML-RPC que não é um XML bem formado. Se você precisar passar bytes arbitrários via XML-RPC, use as classes *bytes* ou *bytearray* ou a classe wrapper *Binary* descrito abaixo.

`Server` foi mantido como um apelido para *ServerProxy* para compatibilidade retroativa. Código novo deve usar *ServerProxy*.

Alterado na versão 3.5: Argumento *context* adicionado.

Alterado na versão 3.6: Adicionado suporte para tags com prefixos (por exemplo `ex:nil`) Adicionado suporte para desconversão de tipos adicionados usados pela implementação do Apache XML-RPC para numéricos:

`i1`, `i2`, `i8`, `biginteger`, `float` e `bigdecimal`. Veja https://ws.apache.org/xmlrpc/types.html para uma descrição.

Ver também

XML-RPC HOWTO
Uma boa descrição das operações XML-RPC e software cliente em vários idiomas. Contém praticamente tudo o que um desenvolvedor de clientes XML-RPC precisa saber.

XML-RPC Introspection
Describe a extensão do protocolo XML-RPC para instrospecção.

XML-RPC Specification
A especificação oficial.

22.21.1 Objetos ServerProxy

A *`ServerProxy`* instance has a method corresponding to each remote procedure call accepted by the XML-RPC server. Calling the method performs an RPC, dispatched by both name and argument signature (e.g. the same method name can be overloaded with multiple argument signatures). The RPC finishes by returning a value, which may be either returned data in a conformant type or a *`Fault`* or *`ProtocolError`* object indicating an error.

Servers that support the XML introspection API support some common methods grouped under the reserved `system` attribute:

`ServerProxy.system.`**`listMethods`**`()`
This method returns a list of strings, one for each (non-system) method supported by the XML-RPC server.

`ServerProxy.system.`**`methodSignature`**`(`*name*`)`
This method takes one parameter, the name of a method implemented by the XML-RPC server. It returns an array of possible signatures for this method. A signature is an array of types. The first of these types is the return type of the method, the rest are parameters.

Because multiple signatures (ie. overloading) is permitted, this method returns a list of signatures rather than a singleton.

Signatures themselves are restricted to the top level parameters expected by a method. For instance if a method expects one array of structs as a parameter, and it returns a string, its signature is simply “string, array”. If it expects three integers and returns a string, its signature is “string, int, int, int”.

If no signature is defined for the method, a non-array value is returned. In Python this means that the type of the returned value will be something other than list.

`ServerProxy.system.`**`methodHelp`**`(`*name*`)`
This method takes one parameter, the name of a method implemented by the XML-RPC server. It returns a documentation string describing the use of that method. If no such string is available, an empty string is returned. The documentation string may contain HTML markup.

Alterado na versão 3.5: Instances of *`ServerProxy`* support the *context manager* protocol for closing the underlying transport.

A working example follows. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

def is_even(n):
    return n % 2 == 0

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(is_even, "is_even")
server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client

with xmlrpc.client.ServerProxy("http://localhost:8000/") as proxy:
    print("3 is even: %s" % str(proxy.is_even(3)))
    print("100 is even: %s" % str(proxy.is_even(100)))
```

22.21.2 Objetos DateTime

class xmlrpc.client.**DateTime**

This class may be initialized with seconds since the epoch, a time tuple, an ISO 8601 time/date string, or a *datetime.datetime* instance. It has the following methods, supported mainly for internal use by the marshalling/unmarshalling code:

decode(*string*)

Accept a string as the instance's new time value.

encode(*out*)

Write the XML-RPC encoding of this *DateTime* item to the *out* stream object.

It also supports certain of Python's built-in operators through `rich comparison` and `__repr__()` methods.

A working example follows. The server code:

```
import datetime
from xmlrpc.server import SimpleXMLRPCServer
import xmlrpc.client

def today():
    today = datetime.datetime.today()
    return xmlrpc.client.DateTime(today)

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(today, "today")
server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client
import datetime

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")

today = proxy.today()
# convert the ISO8601 string to a datetime object
converted = datetime.datetime.strptime(today.value, "%Y%m%dT%H:%M:%S")
print("Today: %s" % converted.strftime("%d.%m.%Y, %H:%M"))
```

22.21.3 Objetos Binários

class xmlrpc.client.**Binary**

This class may be initialized from bytes data (which may include NULs). The primary access to the content of a *Binary* object is provided by an attribute:

data

The binary data encapsulated by the *Binary* instance. The data is provided as a *bytes* object.

Binary objects have the following methods, supported mainly for internal use by the marshalling/unmarshalling code:

decode(*bytes*)

Accept a base64 *bytes* object and decode it as the instance's new data.

encode(*out*)

Write the XML-RPC base 64 encoding of this binary item to the *out* stream object.

The encoded data will have newlines every 76 characters as per **RFC 2045 section 6.8**, which was the de facto standard base64 specification when the XML-RPC spec was written.

It also supports certain of Python's built-in operators through `__eq__()` and `__ne__()` methods.

Example usage of the binary objects. We're going to transfer an image over XMLRPC:

```
from xmlrpc.server import SimpleXMLRPCServer
import xmlrpc.client

def python_logo():
    with open("python_logo.jpg", "rb") as handle:
        return xmlrpc.client.Binary(handle.read())

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(python_logo, 'python_logo')

server.serve_forever()
```

The client gets the image and saves it to a file:

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
with open("fetched_python_logo.jpg", "wb") as handle:
    handle.write(proxy.python_logo().data)
```

22.21.4 Objetos Fault

class xmlrpc.client.**Fault**

A *Fault* object encapsulates the content of an XML-RPC fault tag. Fault objects have the following attributes:

faultCode

An int indicating the fault type.

faultString

A string containing a diagnostic message associated with the fault.

In the following example we're going to intentionally cause a *Fault* by returning a complex type object. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

# A marshalling error is going to occur because we're returning a
# complex number
def add(x, y):
    return x+y+0j

server = SimpleXMLRPCServer(("localhost", 8000))
```

(continua na próxima página)

(continuação da página anterior)

```
print("Listening on port 8000...")
server.register_function(add, 'add')

server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
try:
    proxy.add(2, 5)
except xmlrpc.client.Fault as err:
    print("A fault occurred")
    print("Fault code: %d" % err.faultCode)
    print("Fault string: %s" % err.faultString)
```

22.21.5 Objetos ProtocolError

class xmlrpc.client.**ProtocolError**

A *ProtocolError* object describes a protocol error in the underlying transport layer (such as a 404 'not found' error if the server named by the URI does not exist). It has the following attributes:

url

The URI or URL that triggered the error.

errcode

O código do erro.

errmsg

The error message or diagnostic string.

headers

A dict containing the headers of the HTTP/HTTPS request that triggered the error.

In the following example we're going to intentionally cause a *ProtocolError* by providing an invalid URI:

```
import xmlrpc.client

# create a ServerProxy with a URI that doesn't respond to XMLRPC requests
proxy = xmlrpc.client.ServerProxy("http://google.com/")

try:
    proxy.some_method()
except xmlrpc.client.ProtocolError as err:
    print("A protocol error occurred")
    print("URL: %s" % err.url)
    print("HTTP/HTTPS headers: %s" % err.headers)
    print("Error code: %d" % err.errcode)
    print("Error message: %s" % err.errmsg)
```

22.21.6 Objetos MultiCall

The *MultiCall* object provides a way to encapsulate multiple calls to a remote server into a single request[1].

[1] This approach has been first presented in a discussion on xmlrpc.com.

class xmlrpc.client.**MultiCall**(*server*)

Create an object used to boxcar method calls. *server* is the eventual target of the call. Calls can be made to the result object, but they will immediately return `None`, and only store the call name and parameters in the *MultiCall* object. Calling the object itself causes all stored calls to be transmitted as a single `system.multicall` request. The result of this call is a *generator*; iterating over this generator yields the individual results.

A usage example of this class follows. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    return x // y

# A simple server with simple arithmetic functions
server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_multicall_functions()
server.register_function(add, 'add')
server.register_function(subtract, 'subtract')
server.register_function(multiply, 'multiply')
server.register_function(divide, 'divide')
server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
multicall = xmlrpc.client.MultiCall(proxy)
multicall.add(7, 3)
multicall.subtract(7, 3)
multicall.multiply(7, 3)
multicall.divide(7, 3)
result = multicall()

print("7+3=%d, 7-3=%d, 7*3=%d, 7//3=%d" % tuple(result))
```

22.21.7 Convenience Functions

xmlrpc.client.**dumps**(*params*, *methodname=None*, *methodresponse=None*, *encoding=None*, *allow_none=False*)

Convert *params* into an XML-RPC request. or into a response if *methodresponse* is true. *params* can be either a tuple of arguments or an instance of the *Fault* exception class. If *methodresponse* is true, only a single value can be returned, meaning that *params* must be of length 1. *encoding*, if supplied, is the encoding to use in the generated XML; the default is UTF-8. Python's *None* value cannot be used in standard XML-RPC; to allow using it via an extension, provide a true value for *allow_none*.

xmlrpc.client.**loads**(*data*, *use_datetime=False*, *use_builtin_types=False*)

Convert an XML-RPC request or response into Python objects, a `(params, methodname)`. *params* is a tuple of argument; *methodname* is a string, or `None` if no method name is present in the packet. If the XML-RPC packet represents a fault condition, this function will raise a `Fault` exception. The *use_builtin_types* flag can be used to cause date/time values to be presented as `datetime.datetime` objects and binary data to be presented as `bytes` objects; this flag is false by default.

The obsolete *use_datetime* flag is similar to *use_builtin_types* but it applies only to date/time values.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

22.21.8 Exemplo de uso do cliente

```
# simple test program (from the XML-RPC specification)
from xmlrpc.client import ServerProxy, Error

# server = ServerProxy("http://localhost:8000") # local server
with ServerProxy("http://betty.userland.com") as proxy:

    print(proxy)

    try:
        print(proxy.examples.getStateName(41))
    except Error as v:
        print("ERROR", v)
```

To access an XML-RPC server through a HTTP proxy, you need to define a custom transport. The following example shows how:

```
import http.client
import xmlrpc.client

class ProxiedTransport(xmlrpc.client.Transport):

    def set_proxy(self, host, port=None, headers=None):
        self.proxy = host, port
        self.proxy_headers = headers

    def make_connection(self, host):
        connection = http.client.HTTPConnection(*self.proxy)
        connection.set_tunnel(host, headers=self.proxy_headers)
        self._connection = host, connection
        return connection

transport = ProxiedTransport()
transport.set_proxy('proxy-server', 8080)
server = xmlrpc.client.ServerProxy('http://betty.userland.com',␣
↪transport=transport)
print(server.examples.getStateName(41))
```

22.21.9 Example of Client and Server Usage

Veja *Exemplo de SimpleXMLRPCServer*.

22.22 `xmlrpc.server` — Servidores XML-RPC básicos

Código-fonte: Lib/xmlrpc/server.py

O módulo `xmlrpc.server` fornece um framework básico de servidor para servidores XML-RPC escritos em Python. Os servidores podem ser independentes, usando `SimpleXMLRPCServer`, ou incorporados em um ambiente CGI, usando `CGIXMLRPCRequestHandler`.

> **Aviso**
>
> O módulo `xmlrpc.server` não é seguro contra dados criados com códigos maliciosos. Se você precisar analisar dados não confiáveis ou não autenticados, consulte *Segurança no XML*.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

class xmlrpc.server.**SimpleXMLRPCServer**(*addr*, *requestHandler=SimpleXMLRPCRequestHandler*, *logRequests=True*, *allow_none=False*, *encoding=None*, *bind_and_activate=True*, *use_builtin_types=False*)

Cria uma nova instância do servidor. Esta classe fornece métodos para registro de funções que podem ser chamadas pelo protocolo XML-RPC. O parâmetro *requestHandler* deve ser uma fábrica para instâncias do tratador de solicitações; o padrão é `SimpleXMLRPCRequestHandler`. Os parâmetros *addr* e *requestHandler* são passados para o construtor `socketserver.TCPServer`. Se *logRequests* for true (o padrão), as solicitações serão registradas; definir esse parâmetro como false desativará os registros. Os parâmetros *allow_none* e *encoding* são transmitidos para `xmlrpc.client` e controlam as respostas XML-RPC que serão retornadas do servidor. O parâmetro *bind_and_activate* controla se `server_bind()` e `server_activate()` são chamados imediatamente pelo construtor; o padrão é true. A configuração como false permite que o código manipule a variável de classe *allow_reuse_address* antes que o endereço seja vinculado. O parâmetro *use_builtin_types* é passado para a função `loads()` e controla quais tipos são processados quando valores de data/hora ou dados binários são recebidos; o padrão é false.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

class xmlrpc.server.**CGIXMLRPCRequestHandler**(*allow_none=False*, *encoding=None*, *use_builtin_types=False*)

Cria uma nova instância para manipular solicitações XML-RPC em um ambiente CGI. Os parâmetros *allow_none* e *encoding* são transmitidos para `xmlrpc.client` e controlam as respostas XML-RPC que serão retornadas do servidor. O parâmetro *use_builtin_types* é passado para a função `loads()` e controla quais tipos são processados quando valores de data/hora ou dados binários são recebidos; o padrão é false.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

class xmlrpc.server.**SimpleXMLRPCRequestHandler**

Cria uma nova instância do manipulador de solicitações. Este manipulador de solicitação possui suporte a solicitações `POST` e modifica o registro para que o parâmetro *logRequests* para o construtor de `SimpleXMLRPCServer` seja respeitado.

22.22.1 Objetos de SimpleXMLRPCServer

A classe `SimpleXMLRPCServer` é baseada em `socketserver.TCPServer` e fornece um meio de criar servidores XML-RPC simples e independentes.

SimpleXMLRPCServer.**register_function**(*function=None*, *name=None*)

Registra uma função que possa responder às solicitações XML-RPC. Se *name* for fornecido, será o nome do método associado a *function*, caso contrário, `function.__name__` será usado. *name* é uma string e pode conter caracteres ilegais para identificadores Python, incluindo o caractere de ponto.

Este método também pode ser usado como um decorador. Quando usado como decorador, *name* só pode ser fornecido como argumento nomeado para registrar *function* em *name*. Se nenhum *name* for fornecido, `function.__name__` será usado.

Alterado na versão 3.7: *register_function()* pode ser usado como um decorador.

`SimpleXMLRPCServer.`**register_instance**(*instance*, *allow_dotted_names=False*)

Registra um objeto que é usado para expor nomes de métodos que não foram registrados usando *register_function()*. Se *instance* contiver um método `_dispatch()`, ele será chamado com o nome do método solicitado e os parâmetros da solicitação. Sua API é `def _dispatch(self, method, params)` (observe que *params* não representa uma lista de argumentos variáveis). Se ele chama uma função subjacente para executar sua tarefa, essa função é chamada como `func(*params)`, expandindo a lista de parâmetros. O valor de retorno de `_dispatch()` é retornado ao cliente como resultado. Se *instance* não possui o método `_dispatch()`, é procurado por um atributo correspondente ao nome do método solicitado.

Se o argumento opcional *allow_dotted_names* for true e a instância não tiver o método `_dispatch()`, e se o nome do método solicitado contiver pontos, cada componente do nome do método será pesquisado individualmente, com o efeito de que um simples pesquisa hierárquica é realizada. O valor encontrado nessa pesquisa é chamado com os parâmetros da solicitação e o valor retornado é passado de volta ao cliente.

Aviso

A ativação da opção *allow_dotted_names* permite que os invasores acessem as variáveis globais do seu módulo e podem permitir que os invasores executem códigos arbitrários em sua máquina. Use esta opção apenas em uma rede fechada e segura.

`SimpleXMLRPCServer.`**register_introspection_functions**()

Registra as funções de introspecção XML-RPC `system.listMethods`, `system.methodHelp` e `system.methodSignature`.

`SimpleXMLRPCServer.`**register_multicall_functions**()

Registra a função de multichamada XML-RPC system.multicall.

`SimpleXMLRPCRequestHandler.`**rpc_paths**

Um valor de atributo que deve ser uma tupla listando partes do caminho válidas da URL para receber solicitações XML-RPC. Solicitações postadas em outros caminhos resultarão em um erro HTTP 404 "página inexistente". Se esta tupla estiver vazia, todos os caminhos serão considerados válidos. O valor padrão é `('/', '/RPC2')`.

Exemplo de SimpleXMLRPCServer

Código do servidor:

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.server import SimpleXMLRPCRequestHandler

# Restringe a um caminho em particular.
class RequestHandler(SimpleXMLRPCRequestHandler):
    rpc_paths = ('/RPC2',)

# Cria o servidor
with SimpleXMLRPCServer(('localhost', 8000),
                        requestHandler=RequestHandler) as server:
    server.register_introspection_functions()

    # Registra uma função pow(); isso vai usar o valor de
    # pow.__name__ como o nome, que é apenas 'pow'.
    server.register_function(pow)
```

(continua na próxima página)

(continuação da página anterior)

```
    # Registra uma função sob um nome diferente
    def adder_function(x, y):
        return x + y
    server.register_function(adder_function, 'add')

    # Registra uma instância; todos os métodos da instância
    # são publicados como métodos XML-RPC (neste caso,
    # apenas 'mul').
    class MyFuncs:
        def mul(self, x, y):
            return x * y

    server.register_instance(MyFuncs())

    # Executa o laço principal do servidor
    server.serve_forever()
```

O código do cliente a seguir chamará os métodos disponibilizados pelo servidor anterior:

```
import xmlrpc.client

s = xmlrpc.client.ServerProxy('http://localhost:8000')
print(s.pow(2,3))  # Returns 2**3 = 8
print(s.add(2,3))  # Returns 5
print(s.mul(5,2))  # Returns 5*2 = 10

# Exibe uma lista dos métodos disponíveis
print(s.system.listMethods())
```

`register_function()` também pode ser usado como um decorador. O exemplo anterior do servidor pode registrar funções com um decorador:

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.server import SimpleXMLRPCRequestHandler

class RequestHandler(SimpleXMLRPCRequestHandler):
    rpc_paths = ('/RPC2',)

with SimpleXMLRPCServer(('localhost', 8000),
                        requestHandler=RequestHandler) as server:
    server.register_introspection_functions()

    # Registra a função pow(); isso vai usar o valor de
    # pow.__name__ como o nome, que é apenas 'pow'.
    server.register_function(pow)

    # Registra uma função sob um nome diferente, usando
    # register_function como um decorador. *name* só pode
    # ser dado como um argumento nomeado.
    @server.register_function(name='add')
    def adder_function(x, y):
        return x + y

    # Registra uma função sob function.__name__.
    @server.register_function
    def mul(x, y):
```

(continua na próxima página)

(continuação da página anterior)

```
        return x * y

    server.serve_forever()
```

O exemplo a seguir, incluído no módulo `Lib/xmlrpc/server.py`, mostra um servidor que permite nomes com pontos e registra uma função de várias chamadas.

Aviso

A ativação da opção *allow_dotted_names* permite que os invasores acessem as variáveis globais do seu módulo e podem permitir que os invasores executem códigos arbitrários em sua máquina. Use este exemplo apenas em uma rede fechada e segura.

```
import datetime

class ExampleService:
    def getData(self):
        return '42'

    class currentTime:
        @staticmethod
        def getCurrentTime():
            return datetime.datetime.now()

with SimpleXMLRPCServer(("localhost", 8000)) as server:
    server.register_function(pow)
    server.register_function(lambda x,y: x+y, 'add')
    server.register_instance(ExampleService(), allow_dotted_names=True)
    server.register_multicall_functions()
    print('Serving XML-RPC on localhost port 8000')
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nKeyboard interrupt received, exiting.")
        sys.exit(0)
```

Esta demonstração ExampleService pode ser chamada na linha de comando:

```
python -m xmlrpc.server
```

O cliente que interage com o servidor acima está incluído em `Lib/xmlrpc/client.py`:

```
server = ServerProxy("http://localhost:8000")

try:
    print(server.currentTime.getCurrentTime())
except Error as v:
    print("ERROR", v)

multi = MultiCall(server)
multi.getData()
multi.pow(2,9)
multi.add(1,2)
try:
    for response in multi():
```

(continua na próxima página)

(continuação da página anterior)

```
        print(response)
except Error as v:
    print("ERROR", v)
```

Este cliente que interage com o servidor XMLRPC de demonstração pode ser chamado como:

```
python -m xmlrpc.client
```

22.22.2 CGIXMLRPCRequestHandler

A classe *CGIXMLRPCRequestHandler* pode ser usada para manipular solicitações XML-RPC enviadas para scripts CGI Python.

CGIXMLRPCRequestHandler.**register_function**(*function=None*, *name=None*)

Registra uma função que possa responder às solicitações XML-RPC. Se *name* for fornecido, será o nome do método associado a *function*, caso contrário, `function.__name__` será usado. *name* é uma string e pode conter caracteres ilegais para identificadores Python, incluindo o caractere de ponto.

Este método também pode ser usado como um decorador. Quando usado como decorador, *name* só pode ser fornecido como argumento nomeado para registrar *function* em *name*. Se nenhum *name* for fornecido, `function.__name__` será usado.

Alterado na versão 3.7: *register_function()* pode ser usado como um decorador.

CGIXMLRPCRequestHandler.**register_instance**(*instance*)

Registra um objeto que é usado para expor nomes de métodos que não foram registrados usando *register_function()*. Se a instância contiver um método `_dispatch()`, ela será chamada com o nome do método solicitado e os parâmetros da solicitação; o valor retornado é retornado ao cliente como resultado. Se a instância não tiver um método `_dispatch()`, será procurado um atributo correspondente ao nome do método solicitado; se o nome do método solicitado contiver pontos, cada componente do nome do método será pesquisado individualmente, com o efeito de que uma pesquisa hierárquica simples é executada. O valor encontrado nessa pesquisa é chamado com os parâmetros da solicitação e o valor retornado é passado de volta ao cliente.

CGIXMLRPCRequestHandler.**register_introspection_functions**()

Registra as funções de introspecção XML-RPC `system.listMethods`, `system.methodHelp` e `system.methodSignature`.

CGIXMLRPCRequestHandler.**register_multicall_functions**()

Registra a função de multichamada XML-RPC `system.multicall`.

CGIXMLRPCRequestHandler.**handle_request**(*request_text=None*)

Manipula uma solicitação XML-RPC. Se *request_text* for fornecido, devem ser os dados POST fornecidos pelo servidor HTTP, caso contrário, o conteúdo do stdin será usado.

Exemplo:

```
class MyFuncs:
    def mul(self, x, y):
        return x * y


handler = CGIXMLRPCRequestHandler()
handler.register_function(pow)
handler.register_function(lambda x,y: x+y, 'add')
handler.register_introspection_functions()
handler.register_instance(MyFuncs())
handler.handle_request()
```

22.22.3 Documentando servidor XMLRPC

Essas classes estendem as classes acima para servir a documentação HTML em resposta a solicitações HTTP GET. Os servidores podem ser independentes, usando *DocXMLRPCServer* ou incorporados em um ambiente CGI, usando *DocCGIXMLRPCRequestHandler*.

class xmlrpc.server.**DocXMLRPCServer**(*addr*, *requestHandler=DocXMLRPCRequestHandler*, *logRequests=True*, *allow_none=False*, *encoding=None*, *bind_and_activate=True*, *use_builtin_types=True*)

Cria uma nova instância do servidor. Todos os parâmetros têm o mesmo significado que para *SimpleXMLRPCServer*; *requestHandler* assume como padrão *DocXMLRPCRequestHandler*.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

class xmlrpc.server.**DocCGIXMLRPCRequestHandler**

Cria uma nova instância para manipular solicitações XML-RPC em um ambiente CGI.

class xmlrpc.server.**DocXMLRPCRequestHandler**

Cria uma nova instância do manipulador de solicitações. Este manipulador de solicitações possui suporte a solicitações POST de XML-RPC, documenta solicitações GET e modifica o registro para que o parâmetro *logRequests* no parâmetro *DocXMLRPCServer* seja respeitado.

22.22.4 Objetos de DocXMLRPCServer

A classe *DocXMLRPCServer* é derivada de *SimpleXMLRPCServer* e fornece um meio de criar servidores XML-RPC autodocumentados e independentes. Solicitações HTTP POST são tratadas como chamadas de método XML-RPC. As solicitações HTTP GET são tratadas gerando documentação HTML no estilo pydoc. Isso permite que um servidor forneça sua própria documentação baseada na Web.

DocXMLRPCServer.**set_server_title**(*server_title*)

Define o título usado na documentação HTML gerada. Este título será usado dentro do elemento "title" do HTML.

DocXMLRPCServer.**set_server_name**(*server_name*)

Define o nome usado na documentação HTML gerada. Este nome aparecerá na parte superior da documentação gerada dentro de um elemento "h1".

DocXMLRPCServer.**set_server_documentation**(*server_documentation*)

Define a descrição usada na documentação HTML gerada. Esta descrição aparecerá na documentação como um parágrafo, abaixo do nome do servidor.

22.22.5 DocCGIXMLRPCRequestHandler

A classe *DocCGIXMLRPCRequestHandler* é derivada de *CGIXMLRPCRequestHandler* e fornece um meio de criar scripts CGI XML-RPC autodocumentados. Solicitações HTTP POST são tratadas como chamadas de método XML-RPC. As solicitações HTTP GET são tratadas gerando documentação HTML no estilo pydoc. Isso permite que um servidor forneça sua própria documentação baseada na web.

DocCGIXMLRPCRequestHandler.**set_server_title**(*server_title*)

Define o título usado na documentação HTML gerada. Este título será usado dentro do elemento "title" do HTML.

DocCGIXMLRPCRequestHandler.**set_server_name**(*server_name*)

Define o nome usado na documentação HTML gerada. Este nome aparecerá na parte superior da documentação gerada dentro de um elemento "h1".

DocCGIXMLRPCRequestHandler.**set_server_documentation**(*server_documentation*)

Define a descrição usada na documentação HTML gerada. Esta descrição aparecerá na documentação como um parágrafo, abaixo do nome do servidor.

22.23 `ipaddress` — Biblioteca de manipulação de IPv4/IPv6

Código-fonte: Lib/ipaddress.py

---

ipaddress fornece recursos para criar, manipular e operar em endereços e redes IPv4 e IPv6.

As funções e classes neste módulo facilitam o tratamento de várias tarefas relacionadas a endereços IP, incluindo verificar se dois hosts estão ou não na mesma sub-rede, iterar sobre todos os hosts em uma sub-rede específica, verificar se uma string representa ou não um valor válido. Endereço IP ou definição de rede e assim por diante.

Esta é a referência completa da API do módulo – para uma visão geral e introdução, consulte ipaddress-howto.

Adicionado na versão 3.3.

22.23.1 Funções de fábrica de conveniência

O módulo *ipaddress* fornece funções de fábrica para criar endereços IP, redes e interfaces de forma conveniente:

ipaddress.**ip_address**(*address*)

Retorna um objeto *IPv4Address* ou *IPv6Address* dependendo do endereço IP passado como argumento. Podem ser fornecidos endereços IPv4 ou IPv6; números inteiros menores que `2**32` serão considerados IPv4 por padrão. Uma exceção *ValueError* é levantada se *address* não representar um endereço IPv4 ou IPv6 válido.

```
>>> ipaddress.ip_address('192.168.0.1')
IPv4Address('192.168.0.1')
>>> ipaddress.ip_address('2001:db8::')
IPv6Address('2001:db8::')
```

ipaddress.**ip_network**(*address*, *strict=True*)

Retorna um objeto *IPv4Network* ou *IPv6Network* dependendo do endereço IP passado como argumento. *address* é uma string ou número inteiro que representa a rede IP. Podem ser fornecidas redes IPv4 ou IPv6; números inteiros menores que `2**32` serão considerados IPv4 por padrão. *strict* é passado para o construtor *IPv4Network* ou *IPv6Network*. Uma exceção *ValueError* é levantada se *address* não representar um endereço IPv4 ou IPv6 válido, ou se a rede tiver bits de host configurados.

```
>>> ipaddress.ip_network('192.168.0.0/28')
IPv4Network('192.168.0.0/28')
```

ipaddress.**ip_interface**(*address*)

Retorna um objeto *IPv4Interface* ou *IPv6Interface* dependendo do endereço IP passado como argumento. *address* é uma string ou um inteiro representando o endereço IP. Podem ser fornecidos endereços IPv4 ou IPv6; números inteiros menores que `2**32` serão considerados IPv4 por padrão. Uma exceção *ValueError* é levantada se *address* não representar um endereço IPv4 ou IPv6 válido.

Uma desvantagem dessas funções de conveniência é que a necessidade de lidar com os formatos IPv4 e IPv6 significa que as mensagens de erro fornecem informações mínimas sobre o erro exato, pois as funções não sabem se o formato IPv4 ou IPv6 foi pretendido. Relatórios de erros mais detalhados podem ser obtidos chamando diretamente os construtores de classe específicos da versão apropriada.

22.23.2 Endereços IP

Objetos de endereço

Os objetos *IPv4Address* e *IPv6Address* compartilham muitos atributos comuns. Alguns atributos que são significativos apenas para endereços IPv6 também são implementados por objetos *IPv4Address*, para facilitar a escrita de código que lide corretamente com ambas as versões de IP. Os objetos de endereço são *hasheáveis*, portanto podem ser usados como chaves em dicionários.

class ipaddress.**IPv4Address**(*address*)

Constrói um endereço IPv4. Uma exceção *AddressValueError* é levantada se *address* não for um endereço IPv4 válido.

O seguinte constitui um endereço IPv4 válido:

1. Uma string em notação decimal por ponto, consistindo de quatro inteiros decimais em um intervalo inclusivo 0–255 separado por pontos (e.g. `192.168.0.1`). Cada inteiro representa um octeto (byte) no endereço. Zeros à esquerda não são tolerados para evitar confusão com notação octal.
2. Um inteiro que cabe em 32 bits.
3. Um inteiro compactado em um objeto *bytes* de comprimento 4 (octeto mais significativo primeiro).

```
>>> ipaddress.IPv4Address('192.168.0.1')
IPv4Address('192.168.0.1')
>>> ipaddress.IPv4Address(3232235521)
IPv4Address('192.168.0.1')
>>> ipaddress.IPv4Address(b'\xC0\xA8\x00\x01')
IPv4Address('192.168.0.1')
```

Alterado na versão 3.8: Zeros à esquerda são tolerados, mesmo em casos ambíguos que se parecem com notação octal.

Alterado na versão 3.9.5: Zeros à esquerda não são mais tolerados e são tratados como um erro. As strings de endereço IPv4 agora são analisadas de forma tão estrita quanto glibc *inet_pton()*.

version

O número de versão apropriado: `4` para IPv4, `6` para IPv6.

Alterado na versão 3.14: Disponibilizado na classe.

max_prefixlen

O número total de bits na representação de endereço para esta versão: `32` para IPv4, `128` para IPv6.

O prefixo define o número de bits iniciais em um endereço que são comparados para determinar se um endereço faz ou não parte de uma rede.

Alterado na versão 3.14: Disponibilizado na classe.

compressed

exploded

A representação de string em notação decimal pontilhada. Zeros à esquerda nunca são incluídos na representação.

Como o IPv4 não define uma notação abreviada para endereços com octetos definidos como zero, esses dois atributos são sempre iguais a `str(addr)` para endereços IPv4. A exposição desses atributos facilita a gravação de código de exibição que pode tratar endereços IPv4 e IPv6.

packed

A representação binária deste endereço – um objeto *bytes* de comprimento apropriado (primeiro o octeto mais significativo). São 4 bytes para IPv4 e 16 bytes para IPv6.

reverse_pointer

O nome do registro PTR de DNS reverso para o endereço IP, por exemplo:

```
>>> ipaddress.ip_address("127.0.0.1").reverse_pointer
'1.0.0.127.in-addr.arpa'
>>> ipaddress.ip_address("2001:db8::1").reverse_pointer
'1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.8.b.d.0.1.0.0.2.ip6.arpa'
```

Este é o nome que pode ser usado para realizar uma pesquisa PTR, não o nome do host resolvido em si.

Adicionado na versão 3.5.

is_multicast

`True` se o endereço estiver reservado para uso multicast. Consulte **RFC 3171** (para IPv4) ou **RFC 2373** (para IPv6).

is_private

`True` se o endereço for definido como não acessível globalmente por iana-ipv4-special-registry (para IPv4) ou iana-ipv6-special-registry (para IPv6) com as seguintes exceções:

- `is_private` é `False` para o espaço de endereço compartilhado (`100.64.0.0/10`)
- Para endereços IPv6 mapeados em IPv4, o valor `is_private` é determinado pela semântica dos endereços IPv4 subjacentes e a seguinte condição é válida (consulte *`IPv6Address.ipv4_mapped`*):

```
address.is_private == address.ipv4_mapped.is_private
```

`is_private` tem valor oposto a *`is_global`*, exceto para o espaço de endereço compartilhado (intervalo `100.64.0.0/10`) onde ambos são `False`.

Alterado na versão 3.13: Corrigidos alguns falsos positivos e falsos negativos.

- `192.0.0.0/24` é considerado privado com exceção de `192.0.0.9/32` e `192.0.0.10/32` (anteriormente: apenas o sub-intervalo `192.0.0.0/29` foi considerado privado).
- `64:ff9b:1::/48` é considerado privado.
- `2002::/16` é considerado privado.
- Existem exceções em `2001::/23` (de outra forma considerado privado): `2001:1::1/128`, `2001:1::2/128`, `2001:3::/32`, `2001:4:112::/48`, `2001:20::/28`, `2001:30::/28`. As exceções não são consideradas privadas.

is_global

`True` se o endereço for definido como não acessível globalmente por iana-ipv4-special-registry (para IPv4) ou iana-ipv6-special-registry (para IPv6) com a seguinte exceção:

Para endereços IPv6 mapeados em IPv4, o valor `is_private` é determinado pela semântica dos endereços IPv4 subjacentes e a seguinte condição é válida (consulte *`IPv6Address.ipv4_mapped`*):

```
address.is_global == address.ipv4_mapped.is_global
```

`is_global` tem valor oposto a *`is_private`*, exceto para o espaço de endereço compartilhado (intervalo `100.64.0.0/10`) onde ambos são `False`.

Adicionado na versão 3.4.

Alterado na versão 3.13: Corrigidos alguns falsos positivos e falsos negativos, veja *`is_private`* para detalhes.

is_unspecified

`True` se o endereço não estiver especificado. Consulte **RFC 5735** (para IPv4) ou **RFC 2373** (para IPv6).

is_reserved

`True` se o endereço for indicado como reservado pela IETF. Para IPv4, este é apenas `240.0.0.0/4`, o bloco de endereços `Reserved`. Para IPv6, este é todos os endereços alocados como `Reserved by IETF` para uso futuro.

> **Nota**
>
> Para IPv4, `is_reserved` não está relacionado ao valor do bloco de endereço da coluna `Reserved-by-Protocol` em iana-ipv4-special-registry.

> **Cuidado**
>
> Para IPv6, `fec0::/10` um antigo prefixo de endereço de escopo Site-Local está atualmente excluído dessa lista (veja *`is_site_local`* & **RFC 3879**).

is_loopback

`True` se este for um endereço de loopack. Consulte **RFC 3330** (para IPv4) ou **RFC 2373** (para IPv6).

is_link_local

`True` se o endereço está reservado para uso de link local. Veja: **RFC 3927**.

ipv6_mapped

Objeto *`IPv4Address`* representando o endereço IPv6 mapeado para IPv6. Veja **RFC 4291**.

Adicionado na versão 3.13.

`IPv4Address.`**`__format__`**(*fmt*)

Retorna a representação string do endereço de IP, controlado por uma string de formato explícito. *fmt* pode ser um dos seguintes: `'s'`, a opção padrão, equivalente à *`str()`*, `'b'` para uma string binária preenchida com zeros, `'X'` ou `'x'` para uma representação hexadecimal maiúscula ou minúscula, ou `'n'`, que equivale a `'b'` para endereços IPv4 e `'x'` para IPv6. Para representações binárias e hexadecimais, o especificador de forma `'#'` e a opção de agrupamento `'_'` estão disponíveis. `__format__` é utilizado por `format`, `str.format` e f-strings.

```
>>> format(ipaddress.IPv4Address('192.168.0.1'))
'192.168.0.1'
>>> '{:#b}'.format(ipaddress.IPv4Address('192.168.0.1'))
'0b11000000101010000000000000000001'
>>> f'{ipaddress.IPv6Address("2001:db8::1000"):s}'
'2001:db8::1000'
>>> format(ipaddress.IPv6Address('2001:db8::1000'), '_X')
'2001_0DB8_0000_0000_0000_0000_0000_1000'
>>> '{:#_n}'.format(ipaddress.IPv6Address('2001:db8::1000'))
'0x2001_0db8_0000_0000_0000_0000_0000_1000'
```

Adicionado na versão 3.9.

class `ipaddress.`**`IPv6Address`**(*address*)

Constrói um endereço IPv6. Uma exceção *`AddressValueError`* é levantada se *address* não for um endereço IPv6 válido.

O seguinte constitui um endereço IPv6 válido:

1. Uma string constituída de oito grupos de quatro dígitos hexadecimais, cada grupo representando 16 bits. Os grupos são separados por dois pontos. Isto descreve uma notação *explodida* (longa); A string também pode ser *compactada* (notação curta) por vários meios. Ver **RFC 4291** para detalhes. Por exemplo, `"0000:0000:0000:0000:0000:0abc:0007:0def"` pode ser compactada para `"::abc:7:def"`.

   Opcionalmente, a string pode ter um ID de escopo de zona, expressado por um sufixo `%scope_id`. Se presente, o ID de escopo deve ser não vazio, e pode não conter `%`. Ver **RFC 4007** para detalhes. Por exemplo, `fe80::1234%1` pode identificar o endereço `fe80::1234` no primeiro link do nó.

2. Um inteiro que cabe em 128 bits.

3. Um inteiro compactado em um objeto *`bytes`* de comprimento 16, big-endian.

```
>>> ipaddress.IPv6Address('2001:db8::1000')
IPv6Address('2001:db8::1000')
>>> ipaddress.IPv6Address('ff02::5678%1')
IPv6Address('ff02::5678%1')
```

compressed

A forma curta da representação do endereço, com zeros à esquerda em grupos omitidos e a sequência mais longa de grupos consistida inteiramente por zeros colapsada em um grupo vazio único.

Este também é o valor retornado por `str(addr)` para endereços IPv6.

exploded

A forma longa da representação do endereço, com todos zeros à esquerda em grupos consistindo inteiramente de zeros incluídos.

Para os seguintes atributos e métodos, veja a documentação correspondente para a classe *IPv4Address*:

packed

reverse_pointer

version

max_prefixlen

is_multicast

is_private

is_global

Adicionado na versão 3.4.

is_unspecified

is_reserved

is_loopback

is_link_local

is_site_local

`True` se o endereço estiver reservado para uso local do site. Observe que o espaço de endereço local do site foi descontinuado pelo **RFC 3879**. Use *is_private* para testar se este endereço está no espaço de endereços locais exclusivos conforme definido pelo **RFC 4193**.

ipv4_mapped

Para endereços que parecem ser endereços mapeados IPv4 (começando com `::FFFF/96`), esta propriedade reportará o endereço IPv4 incorporado. Para qualquer outro endereço, esta propriedade será `None`.

scope_id

Para endereços com escopo definido pelo **RFC 4007**, esta propriedade identifica a zona específica do escopo do endereço à qual o endereço pertence, como uma string. Quando nenhuma zona de escopo for especificada, esta propriedade será `None`.

sixtofour

Para endereços que parecem ser endereços 6to4 (começando com `2002::/16`), como definido pelo **RFC 3056**, esta propriedade reportará o endereço IPv4 incorporado. Para qualquer outro endereço, esta propriedade será `None`.

teredo

Para endereços que parecem ser endereços Teredo (começando com `2001::/32`), como definido pelo **RFC 4380**, esta propriedade reportará o par de endereços `(server, client)` incorporado. Para qualquer outro endereço, esta propriedade será `None`.

`IPv6Address.`**`__format__`**(*fmt*)

Consulte a documentação do método correspondente em *IPv4Address*.

Adicionado na versão 3.9.

Conversão para strings e inteiros

Para interoperar com interfaces de rede, como o módulo de soquete, os endereços devem ser convertidos em strings ou inteiros. Isso é tratado usando as funções internas *str()* e *int()*:

```
>>> str(ipaddress.IPv4Address('192.168.0.1'))
'192.168.0.1'
>>> int(ipaddress.IPv4Address('192.168.0.1'))
3232235521
>>> str(ipaddress.IPv6Address('::1'))
'::1'
>>> int(ipaddress.IPv6Address('::1'))
1
```

Observe que os endereços com escopo IPv6 são convertidos em números inteiros sem ID de zona de escopo.

Operadores

Os objetos de endereço têm suporte a alguns operadores. Salvo indicação em contrário, os operadores só podem ser aplicados entre objetos compatíveis (ou seja, IPv4 com IPv4, IPv6 com IPv6).

Operadores de comparação

Os objetos de endereço podem ser comparados com o conjunto usual de operadores de comparação. Os mesmos endereços IPv6 com IDs de zona de escopo diferentes não são iguais. Alguns exemplos:

```
>>> IPv4Address('127.0.0.2') > IPv4Address('127.0.0.1')
True
>>> IPv4Address('127.0.0.2') == IPv4Address('127.0.0.1')
False
>>> IPv4Address('127.0.0.2') != IPv4Address('127.0.0.1')
True
>>> IPv6Address('fe80::1234') == IPv6Address('fe80::1234%1')
False
>>> IPv6Address('fe80::1234%1') != IPv6Address('fe80::1234%2')
True
```

Operadores aritméticos

Inteiros podem ser adicionados ou subtraídos de objetos de endereço. Alguns exemplos:

```
>>> IPv4Address('127.0.0.2') + 3
IPv4Address('127.0.0.5')
>>> IPv4Address('127.0.0.2') - 3
IPv4Address('126.255.255.255')
>>> IPv4Address('255.255.255.255') + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ipaddress.AddressValueError: 4294967296 (>= 2**32) is not permitted as an IPv4␣
↪address
```

22.23.3 Definições de rede IP

Os objetos *IPv4Network* e *IPv6Network* fornecem um mecanismo para definir e inspecionar definições de rede IP. Uma definição de rede consiste em uma *máscara* e um *endereço de rede* e, como tal, define um intervalo de endereços IP que é igual ao endereço de rede quando mascarado (E binário) com a máscara. Por exemplo, uma definição de rede com a máscara `255.255.255.0` e o endereço de rede `192.168.1.0` consiste em endereços IP no intervalo inclusivo `192.168.1.0` a `192.168.1.255`.

Prefixo, máscara de rede e máscara de host

Existem várias maneiras equivalentes de especificar máscaras de rede IP. Um *prefixo* `/<nbits>` é uma notação que denota quantos bits de ordem superior estão definidos na máscara de rede. Uma máscara de rede, ou *netmask*, é um endereço IP com um certo número de bits de alta ordem definidos. Assim o prefixo `/24` é equivalente à máscara de rede `255.255.255.0` em IPv4, ou `ffff:ff00::` em IPv6. Além disso, uma máscara de host, ou *host mask*, é o inverso lógico de uma máscara de rede e às vezes é usada (por exemplo, nas listas de controle de acesso da Cisco) para denotar uma máscara de rede. A máscara de host equivalente a `/24` em IPv4 é `0.0.0.255`.

Objetos de rede

Todos os atributos implementados por objetos de endereço também são implementados por objetos de rede. Além disso, os objetos de rede implementam atributos adicionais. Todos estes são comuns entre *IPv4Network* e *IPv6Network*, portanto, para evitar duplicação, eles são documentados apenas para *IPv4Network*. Objetos de rede são *hasheáveis*, portanto podem ser usados como chaves em dicionários.

class ipaddress.**IPv4Network**(*address*, *strict=True*)

Constrói uma definição de rede IPv4. *address* pode ser um dos seguintes:

1. Uma string que consiste em um endereço IP e uma máscara opcional, separados por uma barra (`/`). O endereço IP é o endereço de rede, e a máscara pode ser um único número, o que significa que é um *prefixo*, ou uma representação de string de um endereço IPv4. Se for o último, a máscara será interpretada como uma *máscara de rede* se começar com um campo diferente de zero, ou como uma *máscara de host* se começar com um campo zero, com a única exceção de uma máscara totalmente zero que é tratada como uma *máscara de rede*. Se nenhuma máscara for fornecida, será considerada `/32`.

   Por exemplo, as seguintes especificações de *address* são equivalentes: `192.168.1.0/24`, `192.168.1.0/255.255.255.0` e `192.168.1.0/0.0.0.255`.

2. Um número inteiro que cabe em 32 bits. Isto é equivalente a uma rede de endereço único, com o endereço de rede sendo *address* e a máscara sendo `/32`.

3. Um inteiro compactado em um objeto *bytes* de comprimento 4, big-endian. A interpretação é semelhante a um *address* no formato inteiro.

4. Uma tupla de dois elementos de uma descrição de endereço e uma máscara de rede, onde a descrição do endereço é uma string, um número inteiro de 32 bits, um número inteiro compactado de 4 bytes ou um objeto *IPv4Address* existente; e a máscara de rede é um número inteiro que representa o comprimento do prefixo (por exemplo, `24`) ou uma string que representa a máscara do prefixo (por exemplo, `255.255.255.0`).

Uma exceção *AddressValueError* é levantada se *address* não for um endereço IPv4 válido. Uma exceção *NetmaskValueError* é levantada se a máscara não for válida para um endereço IPv4.

Se *strict* for `True` e os bits do host estiverem definidos no endereço fornecido, então *ValueError* será levantada. Caso contrário, os bits do host serão mascarados para determinar o endereço de rede apropriado.

Salvo indicação em contrário, todos os métodos de rede que aceitam outros objetos rede/endereço irão levantar *TypeError* se a versão IP do argumento for incompatível com `self`.

Alterado na versão 3.5: Adicionado o formulário de tupla de 2 elementos para o parâmetro do construtor *address*.

version

max_prefixlen

Consulte a documentação do atributo correspondente em *IPv4Address*.

is_multicast

is_private

is_unspecified

is_reserved

is_loopback

is_link_local

Esses atributos terão valor verdadeiro para a rede como um todo se forem verdadeiros tanto para o endereço de rede quanto para o endereço de broadcast.

network_address

O endereço de rede da rede. O endereço de rede e o comprimento do prefixo juntos definem exclusivamente uma rede.

broadcast_address

O endereço de broadcast da rede. Os pacotes enviados para o endereço de broadcast devem ser recebidos por todos os hosts da rede.

hostmask

A máscara do host, como um objeto *IPv4Address*.

netmask

A máscara de rede, como um objeto *IPv4Address*.

with_prefixlen

compressed

exploded

Uma representação de string da rede, com a máscara em notação de prefixo.

`with_prefixlen` e `compressed` são sempre iguais a `str(network)`. `exploded` usa a forma explodida do endereço de rede.

with_netmask

Uma representação em string da rede, com a máscara na notação de máscara de rede.

with_hostmask

Uma representação de string da rede, com a máscara na notação de máscara de host.

num_addresses

O número total de endereços na rede.

prefixlen

Comprimento do prefixo de rede, em bits.

hosts()

Retorna um iterador sobre os hosts utilizáveis na rede. Os hosts utilizáveis são todos os endereços IP que pertencem à rede, exceto o próprio endereço de rede e o endereço de transmissão da rede. Para redes com comprimento de máscara 31, o endereço de rede e o endereço de transmissão de rede também são incluídos no resultado. Redes com máscara 32 retornarão uma lista contendo o endereço de host único.

```
>>> list(ip_network('192.0.2.0/29').hosts())
[IPv4Address('192.0.2.1'), IPv4Address('192.0.2.2'),
 IPv4Address('192.0.2.3'), IPv4Address('192.0.2.4'),
 IPv4Address('192.0.2.5'), IPv4Address('192.0.2.6')]
>>> list(ip_network('192.0.2.0/31').hosts())
[IPv4Address('192.0.2.0'), IPv4Address('192.0.2.1')]
>>> list(ip_network('192.0.2.1/32').hosts())
[IPv4Address('192.0.2.1')]
```

overlaps(*other*)

`True` se esta rede estiver parcial ou totalmente contida em *other* ou *other* estiver totalmente contida nesta rede.

address_exclude(*network*)

Calcula as definições de rede resultantes da remoção da *network* fornecida desta. Retorna um iterador de objetos de rede. Levanta `ValueError` se *network* não estiver completamente contida nesta rede.

```
>>> n1 = ip_network('192.0.2.0/28')
>>> n2 = ip_network('192.0.2.1/32')
>>> list(n1.address_exclude(n2))
[IPv4Network('192.0.2.8/29'), IPv4Network('192.0.2.4/30'),
 IPv4Network('192.0.2.2/31'), IPv4Network('192.0.2.0/32')]
```

subnets(*prefixlen_diff=1*, *new_prefix=None*)

As sub-redes que se unem para criar a definição de rede atual, dependendo dos valores dos argumentos. *prefixlen_diff* é o valor pelo qual o comprimento do nosso prefixo deve ser aumentado. *new_prefix* é o novo prefixo desejado das sub-redes; deve ser maior que nosso prefixo. Um, e apenas um, de *prefixlen_diff* e *new_prefix* deve ser definido. Retorna um iterador de objetos de rede.

```
>>> list(ip_network('192.0.2.0/24').subnets())
[IPv4Network('192.0.2.0/25'), IPv4Network('192.0.2.128/25')]
>>> list(ip_network('192.0.2.0/24').subnets(prefixlen_diff=2))
[IPv4Network('192.0.2.0/26'), IPv4Network('192.0.2.64/26'),
 IPv4Network('192.0.2.128/26'), IPv4Network('192.0.2.192/26')]
>>> list(ip_network('192.0.2.0/24').subnets(new_prefix=26))
[IPv4Network('192.0.2.0/26'), IPv4Network('192.0.2.64/26'),
 IPv4Network('192.0.2.128/26'), IPv4Network('192.0.2.192/26')]
>>> list(ip_network('192.0.2.0/24').subnets(new_prefix=23))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    raise ValueError('new prefix must be longer')
ValueError: new prefix must be longer
>>> list(ip_network('192.0.2.0/24').subnets(new_prefix=25))
[IPv4Network('192.0.2.0/25'), IPv4Network('192.0.2.128/25')]
```

supernet(*prefixlen_diff=1*, *new_prefix=None*)

As super-redes contendo esta definição de rede, dependendo dos valores dos argumentos. *prefixlen_diff* é o valor pelo qual o comprimento do nosso prefixo deve ser diminuída. *new_prefix* é o novo prefixo desejado da super-rede; deve ser maior que nosso prefixo. Um, e apenas um, de *prefixlen_diff* e *new_prefix* deve ser definido. Retorna um único objeto rede.

```
>>> ip_network('192.0.2.0/24').supernet()
IPv4Network('192.0.2.0/23')
>>> ip_network('192.0.2.0/24').supernet(prefixlen_diff=2)
IPv4Network('192.0.0.0/22')
>>> ip_network('192.0.2.0/24').supernet(new_prefix=20)
IPv4Network('192.0.0.0/20')
```

subnet_of(*other*)

Retorna `True` se esta rede é uma sub-rede de *other*.

```
>>> a = ip_network('192.168.1.0/24')
>>> b = ip_network('192.168.1.128/30')
>>> b.subnet_of(a)
True
```

Adicionado na versão 3.7.

supernet_of(*other*)

Retorna `True` se esta rede é uma super-rede de *other*.

```
>>> a = ip_network('192.168.1.0/24')
>>> b = ip_network('192.168.1.128/30')
>>> a.supernet_of(b)
True
```

Adicionado na versão 3.7.

compare_networks(*other*)

Compara esta rede com *other*. Nesta comparação são considerados apenas os endereços de rede; bits de host não são. Retorna `-1`, `0` ou `1`.

```
>>> ip_network('192.0.2.1/32').compare_networks(ip_network('192.0.2.2/32'))
-1
>>> ip_network('192.0.2.1/32').compare_networks(ip_network('192.0.2.0/32'))
1
>>> ip_network('192.0.2.1/32').compare_networks(ip_network('192.0.2.1/32'))
0
```

Descontinuado desde a versão 3.7: Ele usa o mesmo algoritmo de ordenação e comparação que "<", "==" e ">"

class ipaddress.**IPv6Network**(*address*, *strict=True*)

Constrói uma definição de rede IPv6. *address* pode ser um dos seguintes:

1. Uma string que consiste em um endereço IP e um comprimento de prefixo opcional, separados por uma barra (`/`). O endereço IP é o endereço de rede e o comprimento do prefixo deve ser um único número, o *prefixo*. Se nenhum comprimento de prefixo for fornecido, será considerado `/128`.

   Observe que as máscaras de rede expandidas atualmente não são suportadas. Isso significa que `2001:db00::0/24` é um argumento válido enquanto `2001:db00::0/ffff:ff00::` não é.

2. Um número inteiro que cabe em 128 bits. Isto é equivalente a uma rede de endereço único, com o endereço de rede sendo *address* e a máscara sendo `/128`.

3. Um inteiro compactado em um objeto *bytes* de comprimento 16, big-endian. A interpretação é semelhante a um *address* no formato inteiro.

4. Uma tupla de dois elementos de uma descrição de endereço e uma máscara de rede, onde a descrição do endereço é uma string, um número inteiro de 128 bits, um número inteiro compactado de 16 bytes ou um objeto *IPv6Address* existente; e a máscara de rede é um número inteiro que representa o comprimento do prefixo.

Uma exceção *AddressValueError* é levantada se *address* não for um endereço IPv6 válido. Uma exceção *NetmaskValueError* é levantada se a máscara não for válida para um endereço IPv6.

Se *strict* for `True` e os bits do host estiverem definidos no endereço fornecido, então *ValueError* será levantada. Caso contrário, os bits do host serão mascarados para determinar o endereço de rede apropriado.

Alterado na versão 3.5: Adicionado o formulário de tupla de 2 elementos para o parâmetro do construtor *address*.

version

max_prefixlen

is_multicast

is_private

is_unspecified

is_reserved

is_loopback

is_link_local

network_address

broadcast_address

hostmask

netmask

with_prefixlen

compressed

exploded

with_netmask

with_hostmask

num_addresses

prefixlen

hosts()

Retorna um iterador sobre os hosts utilizáveis na rede. Os hosts utilizáveis são todos os endereços IP que pertencem à rede, exceto o próprio endereço de anycast do roteador da sub-rede. Para redes com comprimento de máscara 127, o endereço anycast do roteador da sub-rede de rede é também incluído no resultado. Redes com máscara 128 retornarão uma lista contendo o endereço de host único.

overlaps(*other*)

address_exclude(*network*)

subnets(*prefixlen_diff=1*, *new_prefix=None*)

supernet(*prefixlen_diff=1*, *new_prefix=None*)

subnet_of(*other*)

supernet_of(*other*)

compare_networks(*other*)

Consulte a documentação do atributo correspondente em *IPv4Network*.

is_site_local

Este atributo terá valor verdadeiro para a rede como um todo se for verdadeiro tanto para o endereço de rede quanto para o endereço de broadcast.

Operadores

Os objetos de rede têm suporte a alguns operadores. Salvo indicação em contrário, os operadores só podem ser aplicados entre objetos compatíveis (ou seja, IPv4 com IPv4, IPv6 com IPv6).

Operadores lógicos

Os objetos de rede podem ser comparados com o conjunto usual de operadores lógicos. Os objetos de rede são ordenados primeiro por endereço de rede e depois por máscara de rede.

Iteração

Os objetos de rede podem ser iterados para listar todos os endereços pertencentes à rede. Para iteração, *todos* os hosts são retornados, incluindo os hosts inutilizáveis (para hosts utilizáveis, use o método `hosts()`). Um exemplo:

```
>>> for addr in IPv4Network('192.0.2.0/28'):
...     addr
...
IPv4Address('192.0.2.0')
IPv4Address('192.0.2.1')
IPv4Address('192.0.2.2')
IPv4Address('192.0.2.3')
IPv4Address('192.0.2.4')
IPv4Address('192.0.2.5')
IPv4Address('192.0.2.6')
IPv4Address('192.0.2.7')
IPv4Address('192.0.2.8')
IPv4Address('192.0.2.9')
IPv4Address('192.0.2.10')
IPv4Address('192.0.2.11')
IPv4Address('192.0.2.12')
IPv4Address('192.0.2.13')
IPv4Address('192.0.2.14')
IPv4Address('192.0.2.15')
```

Redes como contêineres de endereços

Os objetos de rede podem atuar como contêineres de endereços. Alguns exemplos:

```
>>> IPv4Network('192.0.2.0/28')[0]
IPv4Address('192.0.2.0')
>>> IPv4Network('192.0.2.0/28')[15]
IPv4Address('192.0.2.15')
>>> IPv4Address('192.0.2.6') in IPv4Network('192.0.2.0/28')
True
>>> IPv4Address('192.0.3.6') in IPv4Network('192.0.2.0/28')
False
```

22.23.4 Objetos de interface

Os objetos de interface são *hasheáveis*, portanto podem ser usados como chaves em dicionários.

class ipaddress.**IPv4Interface**(*address*)

Constrói uma interface IPv4. O significado de *address* é o mesmo do construtor de `IPv4Network`, exceto que endereços de host arbitrários são sempre aceitos.

`IPv4Interface` é uma subclasse de `IPv4Address`, portanto herda todos os atributos dessa classe. Além disso, os seguintes atributos estão disponíveis:

ip

O endereço (`IPv4Address`) sem informações de rede.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.ip
IPv4Address('192.0.2.5')
```

network

A rede (`IPv4Network`) à qual esta interface pertence.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.network
IPv4Network('192.0.2.0/24')
```

with_prefixlen

Uma representação de string da interface com a máscara em notação de prefixo.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.with_prefixlen
'192.0.2.5/24'
```

with_netmask

Uma representação de string da interface com a rede como uma máscara de rede.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.with_netmask
'192.0.2.5/255.255.255.0'
```

with_hostmask

Uma representação de string da interface com a rede como uma máscara de host.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.with_hostmask
'192.0.2.5/0.0.0.255'
```

class ipaddress.**IPv6Interface**(*address*)

Constrói uma interface IPv6. O significado de *address* é o mesmo do construtor de *IPv6Network*, exceto que endereços de host arbitrários são sempre aceitos.

IPv6Interface é uma subclasse de *IPv6Address*, portanto herda todos os atributos dessa classe. Além disso, os seguintes atributos estão disponíveis:

ip

network

with_prefixlen

with_netmask

with_hostmask

Consulte a documentação do atributo correspondente em *IPv4Interface*.

Operadores

Os objetos de interface têm suporte a alguns operadores. Salvo indicação em contrário, os operadores só podem ser aplicados entre objetos compatíveis (ou seja, IPv4 com IPv4, IPv6 com IPv6).

Operadores lógicos

Os objetos de interface podem ser comparados com o conjunto usual de operadores lógicos.

Para comparação de igualdade (== e !=), tanto o endereço IP quanto a rede devem ser iguais para que os objetos sejam iguais. Uma interface não será igual a nenhum endereço ou objeto de rede.

Para ordenação (<, >, etc) as regras são diferentes. Objetos de interface e de endereço com a mesma versão IP podem ser comparados e os objetos de endereço sempre serão ordenados antes dos objetos de interface. Dois objetos de interface são primeiro comparados por suas redes e, se forem iguais, então por seus endereços IP.

22.23.5 Outras funções de nível de módulo

O módulo também fornece as seguintes funções de nível de módulo:

ipaddress.**v4_int_to_packed**(*address*)

Representa um endereço como 4 bytes compactados em ordem de rede (big-endian). *address* é uma representação inteira de um endereço IP IPv4. Uma exceção *ValueError* é levantada se o número inteiro for negativo ou muito grande para ser um endereço IP IPv4.

```
>>> ipaddress.ip_address(3221225985)
IPv4Address('192.0.2.1')
>>> ipaddress.v4_int_to_packed(3221225985)
b'\xc0\x00\x02\x01'
```

ipaddress.**v6_int_to_packed**(*address*)

Representa um endereço como 16 bytes compactados em ordem de rede (big-endian). *address* é uma representação inteira de um endereço IP IPv6. Uma exceção *ValueError* é levantada se o número inteiro for negativo ou muito grande para ser um endereço IP IPv6.

ipaddress.**summarize_address_range**(*first*, *last*)

Retorna um iterador do intervalo de rede resumido, considerando o primeiro e o último endereço IP. *first* é o primeiro *IPv4Address* ou *IPv6Address* no intervalo e *last* é o último *IPv4Address* ou *IPv6Address* no intervalo. Uma exceção *TypeError* é levantada se *first* ou *last* não forem endereços IP ou não forem da mesma versão. Uma exceção *ValueError* é levantada se *last* não for maior que *first* ou se a versão do *first* endereço não for 4 ou 6.

```
>>> [ipaddr for ipaddr in ipaddress.summarize_address_range(
...    ipaddress.IPv4Address('192.0.2.0'),
...    ipaddress.IPv4Address('192.0.2.130'))]
[IPv4Network('192.0.2.0/25'), IPv4Network('192.0.2.128/31'), IPv4Network('192.
↪0.2.130/32')]
```

ipaddress.**collapse_addresses**(*addresses*)

Retorna um iterador dos objetos *IPv4Network* ou *IPv6Network* recolhidos. *addresses* é um *iterável* de objetos *IPv4Network* ou *IPv6Network*. Uma exceção *TypeError* é levantada se *addresses* contiver objetos de versão mista.

```
>>> [ipaddr for ipaddr in
... ipaddress.collapse_addresses([ipaddress.IPv4Network('192.0.2.0/25'),
... ipaddress.IPv4Network('192.0.2.128/25')])]
[IPv4Network('192.0.2.0/24')]
```

ipaddress.**get_mixed_type_key**(*obj*)

Retorna uma chave adequada para ordenação entre redes e endereços. Os objetos de endereço e de rede não são ordenáveis por padrão; eles são fundamentalmente diferentes, então a expressão:

```
IPv4Address('192.0.2.0') <= IPv4Network('192.0.2.0/24')
```

não faz sentido. Porém, há alguns momentos em que você pode desejar que *ipaddress* os ordene de qualquer maneira. Se precisar fazer isso, você pode usar esta função como argumento *key* para *sorted()*

obj é um objeto de rede ou de endereço.

22.23.6 Exceções personalizadas

Para oferecer suporte a relatórios de erros mais específicos de construtores de classe, o módulo define as seguintes exceções:

exception ipaddress.**AddressValueError**(*ValueError*)

Qualquer erro de valor relacionado ao endereço.

exception ipaddress.**NetmaskValueError**(*ValueError*)

Qualquer erro de valor relacionado à máscara de rede.

CAPÍTULO 23

Serviços Multimídia

Os módulos descritos neste capítulo implementam vários algoritmos ou interfaces que são bastante úteis em aplicações multimídia. Eles estão disponíveis a critério da instalação. Aqui está uma visão geral:

23.1 `wave` — Read and write WAV files

Código-fonte: Lib/wave.py

The *wave* module provides a convenient interface to the Waveform Audio "WAVE" (or "WAV") file format. Only uncompressed PCM encoded wave files are supported.

Alterado na versão 3.12: Support for `WAVE_FORMAT_EXTENSIBLE` headers was added, provided that the extended format is `KSDATAFORMAT_SUBTYPE_PCM`.

The *wave* module defines the following function and exception:

`wave.`**`open`**(*file*, *mode=None*)

If *file* is a string, open the file by that name, otherwise treat it as a file-like object. *mode* can be:

`'rb'`
Modo somente para leitura.

`'wb'`
Modo somente para escrita.

Note that it does not allow read/write WAV files.

A *mode* of `'rb'` returns a *Wave_read* object, while a *mode* of `'wb'` returns a *Wave_write* object. If *mode* is omitted and a file-like object is passed as *file*, `file.mode` is used as the default value for *mode*.

If you pass in a file-like object, the wave object will not close it when its `close()` method is called; it is the caller's responsibility to close the file object.

The *open()* function may be used in a `with` statement. When the `with` block completes, the *Wave_read.close()* or *Wave_write.close()* method is called.

Alterado na versão 3.4: Added support for unseekable files.

`exception` `wave.`**`Error`**

An error raised when something is impossible because it violates the WAV specification or hits an implementation deficiency.

23.1.1 Objetos Wave_read

class wave.**Wave_read**

Read a WAV file.

Wave_read objects, as returned by *open()*, have the following methods:

close()

Close the stream if it was opened by *wave*, and make the instance unusable. This is called automatically on object collection.

getnchannels()

Returns number of audio channels (`1` for mono, `2` for stereo).

getsampwidth()

Retorna a largura da amostra em bytes.

getframerate()

Retorna a frequência de amostragem.

getnframes()

Retorna o número de quadros de áudio.

getcomptype()

Returns compression type (`'NONE'` is the only supported type).

getcompname()

Human-readable version of *getcomptype()*. Usually `'not compressed'` parallels `'NONE'`.

getparams()

Returns a *namedtuple()* `(nchannels, sampwidth, framerate, nframes, comptype, compname)`, equivalent to output of the `get*()` methods.

readframes(*n*)

Reads and returns at most *n* frames of audio, as a *bytes* object.

rewind()

Volta o ponteiro do arquivo para o início do fluxo de áudio.

The following two methods are defined for compatibility with the old `aifc` module, and don't do anything interesting.

getmarkers()

Retorna `None`.

Deprecated since version 3.13, will be removed in version 3.15: The method only existed for compatibility with the `aifc` module which has been removed in Python 3.13.

getmark(*id*)

Levanta um erro.

Deprecated since version 3.13, will be removed in version 3.15: The method only existed for compatibility with the `aifc` module which has been removed in Python 3.13.

Os dois métodos a seguir definem um termo "posição" que é compatível entre eles e é dependente da implementação.

setpos(*pos*)

Set the file pointer to the specified position.

tell()

Return current file pointer position.

23.1.2 Objetos Wave_write

class wave.**Wave_write**

Write a WAV file.

Wave_write objects, as returned by *open()*.

For seekable output streams, the wave header will automatically be updated to reflect the number of frames actually written. For unseekable streams, the *nframes* value must be accurate when the first frame data is written. An accurate *nframes* value can be achieved either by calling *setnframes()* or *setparams()* with the number of frames that will be written before *close()* is called and then using *writeframesraw()* to write the frame data, or by calling *writeframes()* with all of the frame data to be written. In the latter case *writeframes()* will calculate the number of frames in the data and set *nframes* accordingly before writing the frame data.

Alterado na versão 3.4: Added support for unseekable files.

Wave_write objects have the following methods:

close()

Make sure *nframes* is correct, and close the file if it was opened by *wave*. This method is called upon object collection. It will raise an exception if the output stream is not seekable and *nframes* does not match the number of frames actually written.

setnchannels(*n*)

Define o número de canais.

setsampwidth(*n*)

Set the sample width to *n* bytes.

setframerate(*n*)

Set the frame rate to *n*.

Alterado na versão 3.2: A non-integral input to this method is rounded to the nearest integer.

setnframes(*n*)

Set the number of frames to *n*. This will be changed later if the number of frames actually written is different (this update attempt will raise an error if the output stream is not seekable).

setcomptype(*type*, *name*)

Set the compression type and description. At the moment, only compression type NONE is supported, meaning no compression.

setparams(*tuple*)

The *tuple* should be (nchannels, sampwidth, framerate, nframes, comptype, compname), with values valid for the set*() methods. Sets all parameters.

tell()

Return current position in the file, with the same disclaimer for the *Wave_read.tell()* and *Wave_read.setpos()* methods.

writeframesraw(*data*)

Escreve quadros de áudio, sem corrigir *nframes*.

Alterado na versão 3.4: Todo *objeto bytes ou similar* agora é aceito.

writeframes(*data*)

Write audio frames and make sure *nframes* is correct. It will raise an error if the output stream is not seekable and the total number of frames that have been written after *data* has been written does not match the previously set value for *nframes*.

Alterado na versão 3.4: Todo *objeto bytes ou similar* agora é aceito.

Note that it is invalid to set any parameters after calling *writeframes()* or *writeframesraw()*, and any attempt to do so will raise *wave.Error*.

23.2 `colorsys` — Conversões entre sistemas de cores

Código-fonte: Lib/colorsys.py

O módulo *colorsys* define conversões bidirecionais de valores de cores entre cores expressas no espaço de cores RGB (Red Green Blue) usado em monitores de computador e três outros sistemas de coordenadas: YIQ, HLS (Hue Lightness Saturation) e HSV (Hue Saturation Value). As coordenadas em todos esses espaços de cores são valores de ponto flutuante. No espaço YIQ, a coordenada Y está entre 0 e 1, mas as coordenadas I e Q podem ser positivas ou negativas. Em todos os outros espaços, as coordenadas estão todas entre 0 e 1.

Ver também

Mais informações sobre espaços de cores podem ser encontradas em https://poynton.ca/ColorFAQ.html e https://www.cambridgeincolour.com/tutorials/color-spaces.htm.

O módulo *colorsys* define as seguintes funções:

`colorsys.`**`rgb_to_yiq`**(*r*, *g*, *b*)

Converte a cor de coordenadas RGB para coordenadas YIQ.

`colorsys.`**`yiq_to_rgb`**(*y*, *i*, *q*)

Converte a cor de coordenadas YIQ para coordenadas RGB.

`colorsys.`**`rgb_to_hls`**(*r*, *g*, *b*)

Converte a cor de coordenadas RGB para coordenadas HLS.

`colorsys.`**`hls_to_rgb`**(*h*, *l*, *s*)

Converte a cor de coordenadas HLS para coordenadas RGB.

`colorsys.`**`rgb_to_hsv`**(*r*, *g*, *b*)

Converte a cor de coordenadas RGB para coordenadas HSV.

`colorsys.`**`hsv_to_rgb`**(*h*, *s*, *v*)

Converte a cor de coordenadas HSV para coordenadas RGB.

Exemplo:

```
>>> import colorsys
>>> colorsys.rgb_to_hsv(0.2, 0.4, 0.4)
(0.5, 0.5, 0.4)
>>> colorsys.hsv_to_rgb(0.5, 0.5, 0.4)
(0.2, 0.4, 0.4)
```

CAPÍTULO 24

Internacionalização

Os módulos descritos neste capítulo ajudam você a criar um software que é independente de idioma e localidade, fornecendo mecanismos para selecionar um idioma a ser usado em mensagens de programa ou adaptando a saída para corresponder às convenções locais.

A lista de módulos descritos neste capítulo é:

24.1 `gettext` — Serviços de internacionalização multilíngues

Código-fonte: Lib/gettext.py

O módulo *gettext* fornece serviços de internacionalização (I18N) e localização (L10N) para seus módulos e aplicativos Python. Ele suporta a API do catálogo de mensagens GNU **gettext** e uma API baseada em classes de nível mais alto que podem ser mais apropriadas para arquivos Python. A interface descrita abaixo permite gravar o módulo e as mensagens do aplicativo em um idioma natural e fornecer um catálogo de mensagens traduzidas para execução em diferentes idiomas naturais.

Algumas dicas sobre localização de seus módulos e aplicativos Python também são fornecidas.

24.1.1 API do GNU `gettext`

O módulo *gettext* define a API a seguir, que é muito semelhante à API do GNU **gettext**. Se você usar esta API, você afetará a tradução de todo o seu aplicativo globalmente. Geralmente, é isso que você deseja se o seu aplicativo for monolíngue, com a escolha do idioma dependente da localidade do seu usuário. Se você estiver localizando um módulo Python, ou se seu aplicativo precisar alternar idiomas rapidamente, provavelmente desejará usar a API baseada em classe.

`gettext.`**`bindtextdomain`**(*domain*, *localedir=None*)

Liga o *domain* ao diretório de localidade *localedir*. Mais concretamente, *gettext* procurará arquivos binários `.mo` para o domínio especificado usando o caminho (no Unix): *localedir*`/`*language*`/LC_MESSAGES/`*domain*`.mo`, sendo *language* pesquisado nas variáveis de ambiente `LANGUAGE`, `LC_ALL`, `LC_MESSAGES` e `LANG` respectivamente.

Se *localedir* for omitido ou `None`, a ligação atual para *domain* será retornada.[1]

[1] O diretório de localidade padrão depende do sistema; por exemplo, no Red Hat Linux é `/usr/share/locale`, mas no Solaris é `/usr/lib/locale`. O módulo `gettext` não tenta dar suporte a esses padrões dependentes do sistema; em vez disso, seu padrão é *sys.base_prefix*`/share/locale` (consulte *sys.base_prefix*). Por esse motivo, é sempre melhor chamar *bindtextdomain()* com um caminho absoluto explícito no início da sua aplicação.

gettext.**textdomain**(*domain=None*)

Altera ou consulta o domínio global atual. Se *domain* for `None`, o domínio global atual será retornado; caso contrário, o domínio global será definido como *domain*, o qual será retornado.

gettext.**gettext**(*message*)

Retorna a tradução localizada de *message*, com base no diretório global atual de domínio, idioma e localidade. Essa função geralmente é apelidada como `_()` no espaço de nomes local (veja exemplos abaixo).

gettext.**dgettext**(*domain*, *message*)

Semelhante a *gettext()*, mas procura a mensagem no *domain* especificado.

gettext.**ngettext**(*singular*, *plural*, *n*)

Semelhante a *gettext()*, mas considera formas plurais. Se uma tradução for encontrada, aplica a fórmula do plural a *n* e retorne a mensagem resultante (alguns idiomas têm mais de duas formas no plural). Se nenhuma tradução for encontrada, retorna *singular* se *n* for 1; retorna *plural* caso contrário.

A fórmula de Plural é retirada do cabeçalho do catálogo. É uma expressão C ou Python que possui uma variável livre *n*; a expressão é avaliada para o índice do plural no catálogo. Veja a documentação do gettext GNU para obter a sintaxe precisa a ser usada em arquivos `.po` e as fórmulas para um variedade de idiomas.

gettext.**dngettext**(*domain*, *singular*, *plural*, *n*)

Semelhante a *ngettext()*, mas procura a mensagem no *domain* especificado.

gettext.**pgettext**(*context*, *message*)

gettext.**dpgettext**(*domain*, *context*, *message*)

gettext.**npgettext**(*context*, *singular*, *plural*, *n*)

gettext.**dnpgettext**(*domain*, *context*, *singular*, *plural*, *n*)

Semelhante às funções correspondentes sem o `p` no prefixo (ou seja, *gettext()*, *dgettext()*, *ngettext()*, *dngettext()*), mas a tradução é restrita ao *context* de mensagem fornecido.

Adicionado na versão 3.8.

Note que GNU **gettext** também define um método `dcgettext()`, mas isso não foi considerado útil e, portanto, atualmente não está implementado.

Aqui está um exemplo de uso típico para esta API:

```
import gettext
gettext.bindtextdomain('minhaaplicacao', '/caminho/para/diretório/do/meu/idioma')
gettext.textdomain('minhaaplicacao')
_ = gettext.gettext
# ...
print(_('Esta é uma string traduzível.'))
```

24.1.2 API baseada em classe

A API baseada em classe do módulo *gettext* oferece mais flexibilidade e maior conveniência do que a API do GNU **gettext**. É a maneira recomendada de localizar seus aplicativos e módulos Python. `gettext` define uma classe *GNUTranslations* que implementa a análise de arquivos no formato GNU `.mo` e possui métodos para retornar strings. Instâncias dessa classe também podem se instalar no espaço de nomes embutido como a função `_()`.

gettext.**find**(*domain*, *localedir=None*, *languages=None*, *all=False*)

Esta função implementa o algoritmo de busca de arquivos `.mo` padrão. É necessário um *domain*, idêntico ao que *textdomain()* leva. *localedir* opcional é como em *bindtextdomain()*. *languages* opcional é uma lista de strings, em que cada string é um código de idioma.

Se *localedir* não for fornecido, o diretório local do sistema padrão será usado.[2] Se *languages* não for fornecido, as seguintes variáveis de ambiente serão pesquisadas: `LANGUAGE`, `LC_ALL`, `LC_MESSAGES` e `LANG`. O primeiro

[2] Consulte a nota de rodapé para a *bindtextdomain()* acima.

retornando um valor não vazio é usado para a variável *languages*. As variáveis de ambiente devem conter uma lista de idiomas separada por dois pontos, que será dividida nos dois pontos para produzir a lista esperada de strings de código de idioma.

`find()` expande e normaliza os idiomas e itera através deles, procurando por um arquivo existente construído com esses componentes:

localedir/*language*/LC_MESSAGES/*domain*.mo

O primeiro nome de arquivo existente é retornado por `find()`. Se nenhum desses arquivos for encontrado, será retornado `None`. Se *all* for fornecido, ele retornará uma lista de todos os nomes de arquivos, na ordem em que aparecem na lista de idiomas ou nas variáveis de ambiente.

gettext.**translation**(*domain*, *localedir=None*, *languages=None*, *class_=None*, *fallback=False*)

Retorna uma instância de `*Translations` com base nos *domain*, *localedir* e *languages*, que são passados primeiro para `find()` para obter uma lista dos caminhos de arquivos `.mo` associados. Instâncias com nomes de arquivo idênticos `.mo` são armazenados em cache. A classe atual instanciada é *class_* se fornecida, caso contrário `GNUTranslations`. O construtor da classe deve usar um único argumento *objeto arquivo*.

Se vários arquivos forem encontrados, os arquivos posteriores serão usados como fallbacks para os anteriores. Para permitir a configuração do fallback, `copy.copy()` é usado para clonar cada objeto de conversão do cache; os dados reais da instância ainda são compartilhados com o cache.

Se nenhum arquivo `.mo` for encontrado, essa função levanta `OSError` se *fallback* for falso (que é o padrão) e retorna uma instância `NullTranslations` se *fallback* for verdadeiro.

Alterado na versão 3.3: `IOError` costumava ser levantada, agora ele é um apelido para `OSError`.

Alterado na versão 3.11: O parâmetro *codeset* foi removido.

gettext.**install**(*domain*, *localedir=None*, *, *names=None*)

Isso instala a função `_()` no espaço de nomes interno do Python, com base em *domain* e *localedir* que são passados para a função `translation()`.

Para o parâmetro *names*, por favor, veja a descrição do método `install()` do objeto de tradução.

Como visto abaixo, você normalmente marca as strings candidatas à tradução em sua aplicação, envolvendo-as em uma chamada para a função `_()`, assim:

```
print(_('Esta string será traduzida.'))
```

Por conveniência, você deseja que a função `_()` seja instalada no espaço de nomes interno do Python, para que seja facilmente acessível em todos os módulos do sua aplicação.

Alterado na versão 3.11: *names* é agora um parâmetro somente-nomeado.

A classe `NullTranslations`

As classes de tradução são o que realmente implementa a tradução de strings de mensagens do arquivo-fonte original para strings de mensagens traduzidas. A classe base usada por todas as classes de tradução é `NullTranslations`; isso fornece a interface básica que você pode usar para escrever suas próprias classes de tradução especializadas. Aqui estão os métodos de `NullTranslations`:

class gettext.**NullTranslations**(*fp=None*)

Recebe um *objeto arquivo* opcional *fp*, que é ignorado pela classe base. Inicializa as variáveis de instância "protegidas" *_info* e *_charset*, que são definidas por classes derivadas, bem como *_fallback*, que é definido através de `add_fallback()`. Ele então chama `self._parse(fp)` se *fp* não for `None`.

_parse(*fp*)

No-op na classe base, esse método pega o objeto arquivo *fp* e lê os dados do arquivo, inicializando seu catálogo de mensagens. Se você tiver um formato de arquivo de catálogo de mensagens não suportado, substitua esse método para analisar seu formato.

add_fallback(*fallback*)

Adiciona *fallback* como o objeto reserva para o objeto de tradução atual. Um objeto de tradução deve consultar o fallback se não puder fornecer uma tradução para uma determinada mensagem.

gettext(*message*)

Se um fallback tiver sido definido, encaminha `gettext()` para o fallback. Caso contrário, retorna *message*. Substituído em classes derivadas.

ngettext(*singular*, *plural*, *n*)

Se um fallback tiver sido definido, encaminha `ngettext()` para o fallback. Caso contrário, retorna *singular* se *n* for 1; do contrário, retorna *plural*. Substituído em classes derivadas.

pgettext(*context*, *message*)

Se um fallback tiver sido definido, encaminha *pgettext()* para o fallback. Caso contrário, retorna a mensagem traduzida. Substituído em classes derivadas.

Adicionado na versão 3.8.

npgettext(*context*, *singular*, *plural*, *n*)

Se um fallback tiver sido definido, encaminha *npgettext()* para o fallback. Caso contrário, retorna a mensagem traduzida. Substituído em classes derivadas.

Adicionado na versão 3.8.

info()

Retorna um dicionário que contém os metadados encontrados no arquivo de catálogo de mensagens.

charset()

Retorna a codificação do arquivo de catálogo de mensagens.

install(*names=None*)

Este método instala *gettext()* no espaço de nomes embutido, ligando-o a `_`.

Se o parâmetro *names* for fornecido, deve ser uma sequência contendo os nomes das funções que você deseja instalar no espaço de nomes embutidos, além de `_()`. Há suporte aos nomes `'gettext'`, `'ngettext'`, `'pgettext'` e `'npgettext'`

Observe que esta é apenas uma maneira, embora a maneira mais conveniente, de disponibilizar a função `_()` para sua aplicação. Como afeta a aplicação inteira globalmente, e especificamente o espaço de nomes embutido, os módulos localizados nunca devem instalar `_()`. Em vez disso, eles devem usar este código para disponibilizar `_()` para seu módulo:

```
import gettext
t = gettext.translation('meumódulo', ...)
_ = t.gettext
```

Isso coloca `_()` apenas no espaço de nomes global do módulo e, portanto, afeta apenas as chamadas dentro deste módulo.

Alterado na versão 3.8: Adicionado `'pgettext'` e `'npgettext'`.

A classe `GNUTranslations`

O módulo `gettext` fornece uma classe adicional derivada de *NullTranslations*: *GNUTranslations*. Esta classe substitui `_parse()` para permitir a leitura de arquivos `.mo` do formato GNU **gettext** nos formatos big-endian e little-endian.

GNUTranslations analisa metadados opcionais do catálogo de tradução. É uma convenção com o GNU **gettext** incluir metadados como tradução para a string vazia. Esses metadados estão nos pares `key: value` no estilo **RFC 822** e devem conter a chave `Project-Id-Version`. Se a chave `Content-Type` for encontrada, a propriedade `charset` será usada para inicializar a variável de instância `_charset` "protegida", com o padrão `None` se não for encontrada. Se a codificação de "charset" for especificada, todos os IDs e strings de mensagens lidos no catálogo serão convertidos em Unicode usando essa codificação, caso contrário, o ASCII será presumido.

Como os IDs de mensagens também são lidos como strings Unicode, todos os métodos `*gettext()` presumem os IDs de mensagens como sendo strings Unicode, não como strings de bytes.

Todo o conjunto de pares chave/valor é colocado em um dicionário e definido como a variável de instância `_info` "protegida".

Se o número mágico do arquivo `.mo` for inválido, o número principal da versão é inesperado ou se ocorrerem outros problemas durante a leitura do arquivo, instanciando uma classe *GNUTranslations* pode levantar *OSError*.

class gettext.**GNUTranslations**

Os seguintes métodos são substituídos a partir da implementação da classe base:

gettext(*message*)

Procura o ID da *message* no catálogo e retorna a string de mensagens correspondente, como uma string Unicode. Se não houver entrada no catálogo para o ID da *message* e um fallback tiver sido definido, a pesquisa será encaminhada para o método *gettext()* do fallback. Caso contrário, o ID da *message* é retornado.

ngettext(*singular*, *plural*, *n*)

Faz uma pesquisa de plural-forms de um ID de mensagem. *singular* é usado como o ID da mensagem para fins de pesquisa no catálogo, enquanto *n* é usado para determinar qual forma plural usar. A string de mensagens retornada é uma string Unicode.

Se o ID da mensagem não for encontrado no catálogo e um fallback for especificado, a solicitação será encaminhada para o método do fallback *ngettext()*. Caso contrário, quando *n* for 1, *singular* será retornado e *plural* será retornado em todos os outros casos.

Aqui está um exemplo:

```
n = len(os.listdir('.'))
cat = GNUTranslations(somefile)
message = cat.ngettext(
    'There is %(num)d file in this directory',
    'There are %(num)d files in this directory',
    n) % {'num': n}
```

pgettext(*context*, *message*)

Procura o ID do *context* e da *message* no catálogo e retorna a string de mensagens correspondente, como uma string Unicode. Se não houver entrada no catálogo para o ID do *context* e da *message*, e um fallback tiver sido definido, a pesquisa será encaminhada para o método *pgettext()* do fallback. Caso contrário, o ID da *message* é retornado.

Adicionado na versão 3.8.

npgettext(*context*, *singular*, *plural*, *n*)

Faz uma pesquisa de plural-forms de um ID de mensagem. *singular* é usado como o ID da mensagem para fins de pesquisa no catálogo, enquanto *n* é usado para determinar qual forma plural usar.

Se o ID da mensagem para *context* não for encontrado no catálogo e um fallback for especificado, a solicitação será encaminhada para o método *npgettext()* do fallback. Caso contrário, quando *n* for 1, *singular* será retornado e *plural* será retornado em todos os outros casos.

Adicionado na versão 3.8.

Suporte a catálogo de mensagens do Solaris

O sistema operacional Solaris define seu próprio formato de arquivo binário `.mo`, mas como nenhuma documentação pode ser encontrada nesse formato, ela não é suportada no momento.

O construtor Catalog

O GNOME usa uma versão do módulo *gettext* de James Henstridge, mas esta versão tem uma API um pouco diferente. Seu uso documentado foi:

```
import gettext
cat = gettext.Catalog(domain, localedir)
_ = cat.gettext
print(_('hello world'))
```

Para compatibilidade com este módulo mais antigo, a função `Catalog()` é um apelido para a função *translation()* descrita acima.

Uma diferença entre este módulo e o de Henstridge: seus objetos de catálogo suportavam o acesso por meio de uma API de mapeamento, mas isso parece não ser utilizado e, portanto, não é atualmente suportado.

24.1.3 Internacionalizando seus programas e módulos

Internationalization (I18N), ou internacionalização (I17O) em português, refere-se à operação pela qual um programa é informado sobre vários idiomas. Localization (L10N), ou localização em português, refere-se à adaptação do seu programa, uma vez internacionalizado, aos hábitos culturais e de idioma local. Para fornecer mensagens multilíngues para seus programas Python, você precisa executar as seguintes etapas:

1. preparar seu programa ou módulo especialmente marcando strings traduzíveis
2. executar um conjunto de ferramentas nos arquivos marcados para gerar catálogos de mensagens não tratadas
3. criar traduções específicas do idioma dos catálogos de mensagens
4. usar o módulo *gettext* para que as strings das mensagens sejam traduzidas corretamente

Para preparar seu código para I18N, você precisa examinar todas as strings em seus arquivos. Qualquer string que precise ser traduzida deve ser marcada envolvendo-a em `_('...')` — isto é, uma chamada para a função `_`. Por exemplo:

```
filename = 'meulog.txt'
message = _('escrevendo uma mensagem de log')
with open(filename, 'w') as fp:
    fp.write(message)
```

Neste exemplo, a string `'escrevendo uma mensagem de log'` está marcada como um candidato para tradução, enquanto as strings `'meulog.txt'` e `'w'` não estão.

Existem algumas ferramentas para extrair as strings destinadas à tradução. O GNU **gettext** original tem suporte apenas ao código-fonte C ou C++, mas sua versão estendida **xgettext** varre o código escrito em várias linguagens, incluindo Python, para encontrar strings marcadas como traduzíveis. Babel é uma biblioteca de internacionalização do Python que inclui um script `pybabel` para extrair e compilar catálogos de mensagens. O programa de François Pinard chamado **xpot** faz um trabalho semelhante e está disponível como parte de seu pacote po-utils.

(O Python também inclui versões em Python puro desses programas, chamadas **pygettext.py** e **msgfmt.py**; algumas distribuições do Python as instalam para você. O **pygettext.py** é semelhante ao **xgettext**, mas apenas entende o código-fonte do Python e não consegue lidar com outras linguagens de programação como C ou C++. O **pygettext.py** possui suporte a uma interface de linha de comando semelhante à do **xgettext**; para detalhes sobre seu uso, execute `pygettext.py --help`. O **msgfmt.py** é compatível com binários com GNU **msgfmt**. Com esses dois programas, você pode não precisar do pacote GNU **gettext** para internacionalizar suas aplicações Python.)

xgettext, **pygettext** e ferramentas similares geram `.po` que são catálogos de mensagens. Eles são arquivos legíveis por humanos estruturados que contêm todas as strings marcadas no código-fonte, junto com um espaço reservado para as versões traduzidas dessas strings.

Cópias destes arquivos `.po` são entregues aos tradutores humanos individuais que escrevem traduções para todos os idiomas naturais suportados. Eles enviam de volta as versões completas específicas do idioma como um arquivo `<nome-do-idioma>.po` que é compilado em um arquivo de catálogo binário legível por máquina `.mo` usando o

programa **msgfmt**. Os arquivos `.mo` são usados pelo módulo *gettext* para o processamento de tradução real em tempo de execução.

Como você usa o módulo *gettext* no seu código depende se você está internacionalizando um único módulo ou sua aplicação inteira. As próximas duas seções discutirão cada caso.

Localizando seu módulo

Se você estiver localizando seu módulo, tome cuidado para não fazer alterações globais, por exemplo para o espaço de nomes embutidos. Você não deve usar a API GNU **gettext**, mas a API baseada em classe.

Digamos que seu módulo seja chamado "spam" e as várias traduções do idioma natural do arquivo `.mo` residam em `/usr/share/locale` no formato GNU **gettext**. Aqui está o que você colocaria sobre o seu módulo:

```
import gettext
t = gettext.translation('spam', '/usr/share/locale')
_ = t.gettext
```

Localizando sua aplicação

Se você estiver localizando sua aplicação, poderá instalar a função `_()` globalmente no espaço de nomes embutidos, geralmente no arquivo principal do driver do sua aplicação. Isso permitirá que todos os arquivos específicos de sua aplicação usem `_('...')` sem precisar instalá-la explicitamente em cada arquivo.

No caso simples, você precisa adicionar apenas o seguinte código ao arquivo do driver principal da sua aplicação:

```
import gettext
gettext.install('minhaaplicacao')
```

Se você precisar definir o diretório da localidade, poderá passá-lo para a função *install()*:

```
import gettext
gettext.install('minhaaplicacao', '/usr/share/locale')
```

Alterando os idiomas durante o uso

Se o seu programa precisar oferecer suporte a vários idiomas ao mesmo tempo, convém criar várias instâncias de tradução e alternar entre elas explicitamente, assim:

```
import gettext

lang1 = gettext.translation('minhaaplicacao', languages=['en'])
lang2 = gettext.translation('minhaaplicacao', languages=['fr'])
lang3 = gettext.translation('minhaaplicacao', languages=['de'])

# começa usando o idioma 1
lang1.install()

# ... o tempo passa, o usuário seleciona o idioma 2
lang2.install()

# ... mais tempo passa, o usuário seleciona o idioma 3
lang3.install()
```

Traduções adiadas

Na maioria das situações de codificação, as strings são traduzidas onde são codificadas. Ocasionalmente, no entanto, é necessário marcar strings para tradução, mas adiar a tradução real até mais tarde. Um exemplo clássico é:

```
animals = ['mollusk',
           'albatross',
           'rat',
           'penguin',
           'python', ]
# ...
for a in animals:
    print(a)
```

Aqui, você deseja marcar as strings na lista `animals` como traduzíveis, mas na verdade não deseja traduzi-las até que sejam impressas.

Aqui está uma maneira de lidar com esta situação:

```
def _(message): return message

animals = [_('mollusk'),
           _('albatross'),
           _('rat'),
           _('penguin'),
           _('python'), ]

del _

# ...
for a in animals:
    print(_(a))
```

Isso funciona porque a definição fictícia de `_()` simplesmente retorna a string inalterada. E essa definição fictícia vai substituir temporariamente qualquer definição de `_()` no espaço de nomes embutido (até o comando `del`). Tome cuidado, se você tiver uma definição anterior de `_()` no espaço de nomes local.

Observe que o segundo uso de `_()` não identificará "a" como traduzível para o programa **`gettext`**, porque o parâmetro não é uma string literal.

Outra maneira de lidar com isso é com o seguinte exemplo:

```
def N_(message): return message

animals = [N_('mollusk'),
           N_('albatross'),
           N_('rat'),
           N_('penguin'),
           N_('python'), ]

# ...
for a in animals:
    print(_(a))
```

Nesse caso, você está marcando strings traduzíveis com a função `N_()`, que não entra em conflito com nenhuma definição de `_()`. No entanto, você precisará ensinar seu programa de extração de mensagens a procurar strings traduzíveis marcadas com `N_()`. **`xgettext`**, **`pygettext`**, `pybabel extract` e **`xpot`** possuem suporte a isso através do uso da opção de linha de comando `-k`. A escolha de `N_()` aqui é totalmente arbitrária; poderia facilmente ter sido `MarkThisStringForTranslation()`.

24.1.4 Reconhecimentos

As seguintes pessoas contribuíram com código, feedback, sugestões de design, implementações anteriores e experiência valiosa para a criação deste módulo:

- Peter Funk
- James Henstridge
- Juan David Ibáñez Palomar
- Marc-André Lemburg
- Martin von Löwis
- François Pinard
- Barry Warsaw
- Gustavo Niemeyer

24.2 `locale` — Serviços de internacionalização

Código-fonte: Lib/locale.py

O módulo *`locale`* abre o acesso ao banco de dados de localidade POSIX e funcionalidade. O mecanismo de localidade POSIX permite que os programadores lidem com certas questões culturais em uma aplicação, sem exigir que o programador conheça todas as especificidades de cada país onde o software é executado.

O módulo *`locale`* é implementado em cima do módulo `_locale`, que por sua vez usa uma implementação a localidade ANSI C se disponível.

O módulo *`locale`* define a seguinte exceção e funções:

exception locale.**Error**

Exceção levantada quando a localidade passada para *`setlocale()`* não é reconhecida.

locale.**setlocale**(*category*, *locale=None*)

Se *locale* for fornecido e não `None`, *`setlocale()`* modifica a configuração de locale para a *category*. As categorias disponíveis estão listadas na descrição dos dados abaixo. *locale* pode ser uma *string* ou um par, código de idioma e codificação. Uma string vazia especifica as configurações padrão do usuário. Se a modificação da localidade falhar, a exceção *`Error`* é levantada. Se for bem-sucedido, a nova configuração de localidade será retornada.

Se *locale* for um par, ele será convertido em um nome de localidade usando o mecanismo de apelidamento de localidade. O código de idioma tem o mesmo formato de *nome da localidade*, mas sem codificação e modificador `@`. O código de idioma e a codificação podem ser `None`.

Se *locale* for omitido ou `None`, a configuração atual para *category* é retornada.

`setlocale()` não é seguro para thread na maioria dos sistemas. As aplicações normalmente começam com uma chamada de

```
import locale
locale.setlocale(locale.LC_ALL, '')
```

Isso define a localidade de todas as categorias para a configuração padrão do usuário (normalmente especificada na variável de ambiente `LANG`). Se a localidade não for alterada depois disso, o uso de multithreading não deve causar problemas.

locale.**localeconv**()

Retorna o banco de dados das convenções locais como um dicionário. Este dicionário possui as seguintes strings como chaves:

Categoria	Chave	Significado
LC_NUMERIC	'decimal_point'	Caractere de ponto decimal.
	'grouping'	Sequência de números especificando quais posições relativas os 'thousands_sep' são esperados. Se a sequência for terminada com *CHAR_MAX*, nenhum agrupamento adicional é realizado. Se a sequência termina com um 0, o tamanho do último grupo é usado repetidamente.
	'thousands_sep'	Caractere usado entre grupos.
LC_MONETARY	'int_curr_symbol'	Símbolo internacional de moeda.
	'currency_symbol'	Símbolo local de moeda.
	'p_cs_precedes/n_cs_precedes'	Se o símbolo da moeda precede o valor (para valores positivos ou negativos).
	'p_sep_by_space/n_sep_by_space'	Se o símbolo monetário está separado do valor por um espaço (para valores positivos ou negativos).
	'mon_decimal_point'	Ponto decimal usado para valores monetários.
	'frac_digits'	Número de dígitos fracionários usados na formatação local de valores monetários.
	'int_frac_digits'	Número de dígitos fracionários usados na formatação internacional de valores monetários.
	'mon_thousands_sep'	Separador de grupo usado para valores monetários.
	'mon_grouping'	Equivalente a 'grouping', usado para valores monetários.
	'positive_sign'	Símbolo usado para anotar um valor monetário positivo.
	'negative_sign'	Símbolo usado para anotar um valor monetário negativo.
	'p_sign_posn/n_sign_posn'	A posição do sinal (para valores positivos resp. negativos), veja abaixo.

Todos os valores numéricos podem ser definidos como *CHAR_MAX* para indicar que não há valor especificado nesta localidade.

Os valores possíveis para 'p_sign_posn' e 'n_sign_posn' são dados abaixo.

Valor	Explicação
0	A moeda e o valor estão entre parênteses.
1	O sinal deve preceder o valor e o símbolo da moeda.
2	O sinal deve seguir o valor e o símbolo da moeda.
3	O sinal deve preceder imediatamente o valor.
4	O sinal deve seguir imediatamente o valor.
CHAR_MAX	Nada é especificado nesta localidade.

A função define temporariamente a localidade de LC_CTYPE para a localidade de LC_NUMERIC ou a localidade de LC_MONETARY se as localidades forem diferentes e as strings numéricas ou monetárias não forem ASCII.

Esta mudança temporária afeta outras threads.

Alterado na versão 3.7: A função agora define temporariamente a localidade de `LC_CTYPE` para a localidade de `LC_NUMERIC` em alguns casos.

`locale.`**`nl_langinfo`**(*option*)

Retorna algumas informações específicas da localidade em uma string. Esta função não está disponível em todos os sistemas e o conjunto de opções possíveis também pode variar entre as plataformas. Os valores de argumento possíveis são números, para os quais constantes simbólicas estão disponíveis no módulo da localidade.

A função *nl_langinfo()* aceita uma das seguintes chaves. A maioria das descrições são tiradas da descrição correspondente na biblioteca GNU C.

`locale.`**`CODESET`**

Obtém uma string com o nome da codificação de caracteres usada na localidade selecionado.

`locale.`**`D_T_FMT`**

Obtém uma string que pode ser usada como uma string de formato para *time.strftime()* para representar a data e a hora de uma maneira específica da localidade.

`locale.`**`D_FMT`**

Obtém uma string que pode ser usada como uma string de formato para *time.strftime()* para representar uma data de uma maneira específica da localidade.

`locale.`**`T_FMT`**

Obtém uma string que pode ser usada como uma string de formato para *time.strftime()* para representar uma hora de uma maneira específica da localidade.

`locale.`**`T_FMT_AMPM`**

Obtém uma string de formato para *time.strftime()* para representar a hora no formato am/pm.

`locale.`**`DAY_1`**

`locale.`**`DAY_2`**

`locale.`**`DAY_3`**

`locale.`**`DAY_4`**

`locale.`**`DAY_5`**

`locale.`**`DAY_6`**

`locale.`**`DAY_7`**

Obtém o nome do enésimo dia da semana.

> **Nota**
>
> Isso segue a convenção dos EUA de *DAY_1* ser no domingo, não a convenção internacional (ISO 8601) que segunda-feira é o primeiro dia da semana.

`locale.`**`ABDAY_1`**

`locale.`**`ABDAY_2`**

`locale.`**`ABDAY_3`**

`locale.`**`ABDAY_4`**

`locale.`**`ABDAY_5`**

`locale.`**`ABDAY_6`**

`locale.`**`ABDAY_7`**

Obtém o nome abreviado do enésimo dia da semana.

`locale.`**`MON_1`**

`locale.`**`MON_2`**

locale.**MON_3**
locale.**MON_4**
locale.**MON_5**
locale.**MON_6**
locale.**MON_7**
locale.**MON_8**
locale.**MON_9**
locale.**MON_10**
locale.**MON_11**
locale.**MON_12**

Obtém o nome do enésimo dia do mês.

locale.**ABMON_1**
locale.**ABMON_2**
locale.**ABMON_3**
locale.**ABMON_4**
locale.**ABMON_5**
locale.**ABMON_6**
locale.**ABMON_7**
locale.**ABMON_8**
locale.**ABMON_9**
locale.**ABMON_10**
locale.**ABMON_11**
locale.**ABMON_12**

Obtém o nome abreviado do enésimo dia do mês.

locale.**RADIXCHAR**

Obtém o caractere separador decimal (ponto decimal, vírgula decimal etc.).

locale.**THOUSEP**

Obtém o caractere separador para milhares (grupos de três dígitos).

locale.**YESEXPR**

Obtém uma expressão regular que pode ser usada com a função regex para reconhecer uma resposta positiva a uma pergunta sim/não.

locale.**NOEXPR**

Obtém uma expressão regular que pode ser usada com a função `regex(3)` para reconhecer uma resposta negativa a uma pergunta sim/não.

> **Nota**
>
> As expressões regulares para *YESEXPR* e *NOEXPR* usam uma sintaxe adequada para a função `regex` da biblioteca C, que pode ser diferente da sintaxe usada em *re*.

locale.**CRNCYSTR**

Obtém o símbolo da moeda, precedido por "-" se o símbolo deve aparecer antes do valor, "+" se o símbolo deve aparecer após o valor ou "." se o símbolo deve substituir o caractere separador decimal.

locale.**ERA**

Obtém uma string que descreva como os anos são contados e exibidos para cada era em um localidade.

A maioria das localidades não define esse valor. Um exemplo de localidade que define esse valor é o japonês. No Japão, a representação tradicional de datas inclui o nome da época correspondente ao reinado do então imperador.

Normalmente, não deve ser necessário usar esse valor diretamente. Especificar o modificador `E` em suas strings de formato faz com que a função *time.strftime()* use essas informações. O formato da string retornada é especificado em *The Open Group Base Specifications Issue 8*, parágrafo 7.3.5.2 LC_TIME C-Language Access.

`locale.`**ERA_D_T_FMT**

Obtém uma string de formato para *time.strftime()* para representar a data e a hora de uma forma baseada na era específica da localidade.

`locale.`**ERA_D_FMT**

Obtém uma string de formato para *time.strftime()* para representar uma data em uma forma baseada em era específica da localidade.

`locale.`**ERA_T_FMT**

Obtém uma string de formato para *time.strftime()* para representar uma hora em uma forma baseada em era específica da localidade.

`locale.`**ALT_DIGITS**

Obtém uma string consistindo de até 100 símbolos separados por ponto e vírgula usados para representar os valores de 0 a 99 de uma forma específica de localidade. Na maioria das localidades, esta é uma string vazia.

A função define temporariamente a localidade `LC_CTYPE` para a localidade da categoria que determina o valor solicitado (`LC_TIME`, `LC_NUMERIC`, `LC_MONETARY` ou `LC_MESSAGES`) se as localidades forem diferentes e a string resultante não for ASCII. Essa alteração temporária afeta outras threads.

Alterado na versão 3.14: A função agora define temporariamente a localidade de `LC_CTYPE` em alguns casos.

`locale.`**getdefaultlocale**(*[envvars]*)

Tenta determinar as configurações de localidade padrão e as retorna como uma tupla na forma `(language code, encoding)`.

De acordo com POSIX, um programa que não chamou `setlocale(LC_ALL, '')` executa usando a localidade portátil `'C'`. Chamar `setlocale(LC_ALL, '')` permite que ele use a localidade padrão conforme definido pela variável `LANG`. Como não queremos interferir com a configuração de localidade atual, emulamos o comportamento da maneira descrita acima.

Para manter a compatibilidade com outras plataformas, não apenas a variável `LANG` é testada, mas uma lista de variáveis fornecida como parâmetro envvars. Será utilizado o primeiro encontrado a ser definido. *envvars* padroniza para o caminho de pesquisa usado no GNU gettext; deve sempre conter o nome da variável `'LANG'`. O caminho de pesquisa do GNU gettext contém `'LC_ALL'`, `'LC_CTYPE'`, `'LANG'` e `'LANGUAGE'`, nesta ordem.

O código de idioma tem o mesmo formato de *nome de localidade*, mas sem codificação e modificador `@`. O código de idioma e a codificação podem ser `None` se seus valores não puderem ser determinados. O locale "C" é representado como `(None, None)`.

Deprecated since version 3.11, will be removed in version 3.15.

`locale.`**getlocale**(*category=LC_CTYPE*)

Retorna a configuração atual para a categoria de localidade fornecida como uma tupla contendo o código de idioma e a codificação. *category* pode ser um dos valores `LC_*`, exceto *LC_ALL*. O padrão é *LC_CTYPE*.

O código de idioma tem o mesmo formato de *nome de localidade*, mas sem codificação e modificador `@`. O código de idioma e a codificação podem ser `None` se seus valores não puderem ser determinados. O locale "C" é representado como `(None, None)`.

`locale.`**getpreferredencoding**(*do_setlocale=True*)

Retorna a *codificação da localidade* usada para dados de texto, de acordo com as preferências do usuário. As preferências do usuário são expressas de maneira diferente em sistemas diferentes e podem não estar disponíveis programaticamente em alguns sistemas, portanto, essa função retorna apenas uma estimativa.

Em alguns sistemas, é necessário invocar *setlocale()* para obter as preferências do usuário, portanto, esta função não é segura para thread. Se invocar setlocale não for necessário ou desejado, *do_setlocale* deve ser definido como `False`.

No Android ou se o *Modo UTF-8 do Python* é retornado, sempre retorna `'utf-8'`, a *codificação da localidade* e o argumento *do_setlocale* são ignorados.

A pré-inicialização do Python configura a localidade LC_CTYPE. Veja também *tratador de erros e codificação do sistema de arquivos*.

Alterado na versão 3.7: A função agora sempre retorna `"utf-8"` no Android ou se o *Modo UTF-8 do Python* estiver habilitado.

`locale.`**`getencoding`**`()`

Obtém a atual *codificação da localidade*:

- No Android e no VxWorks, retorna `"utf-8"`.
- No Unix, retorna a codificação da localidade *LC_CTYPE* atual. Retorna `"utf-8"` se `nl_langinfo(CODESET)` retornar uma string vazia: por exemplo, se a localidade LC_CTYPE atual não for compatível.
- No Windows, retorna a página de código ANSI.

A pré-inicialização do Python configura a localidade LC_CTYPE. Veja também *tratador de erros e codificação do sistema de arquivos*.

Este função é semelhante a *getpreferredencoding(False)*, exceto pelo fato de que esta função ignora o *Modo UTF-8 do Python*.

Adicionado na versão 3.11.

`locale.`**`normalize`**`(`*localename*`)`

Retorna um código de localidade normalizado para o nome de localidade fornecido. O código de localidade retornado é formatado para uso com *setlocale()*. Se a normalização falhar, o nome original será retornado inalterado.

Se a codificação fornecida não for conhecida, o padrão da função é a codificação padrão para o código da localidade, assim como *setlocale()*.

`locale.`**`strcoll`**`(`*string1*, *string2*`)`

Compara duas strings de acordo com a configuração atual *LC_COLLATE*. Como qualquer outra função de comparação, retorna um valor negativo ou positivo, ou `0`, dependendo se *string1* agrupa antes ou depois de *string2* ou é igual a ele.

`locale.`**`strxfrm`**`(`*string*`)`

Transforma uma string em uma que pode ser usada em comparações com reconhecimento de localidade. Por exemplo, `strxfrm(s1) < strxfrm(s2)` é equivalente a `strcoll(s1, s2) < 0`. Esta função pode ser usada quando a mesma string é comparada repetidamente, por exemplo, ao agrupar uma sequência de strings.

`locale.`**`format_string`**`(`*format*, *val*, *grouping=False*, *monetary=False*`)`

Formata um número *val* de acordo com a configuração atual do *LC_NUMERIC*. O formato segue as convenções do operador `%`. Para valores de ponto flutuante, o ponto decimal é modificado, se apropriado. Se *grouping* for `True`, também levará em conta o agrupamento.

Se *monetary* for verdadeiro, a conversão usa o separador de milhares monetários e strings de agrupamento.

Processa especificadores de formatação como em `format % val`, mas leva as configurações de localidade atuais em consideração.

Alterado na versão 3.7: O parâmetro nomeado *monetary* foi adicionado.

`locale.`**`currency`**`(`*val*, *symbol=True*, *grouping=False*, *international=False*`)`

Formata um número *val* de acordo com as configurações atuais de *LC_MONETARY*.

A string retornada inclui o símbolo da moeda se *symbol* for verdadeiro, que é o padrão. Se *grouping* for `True` (o que não é o padrão), o agrupamento é feito com o valor. Se *international* for `True` (o que não é o padrão), o símbolo da moeda internacional será usado.

> **Nota**
>
> Esta função não funcionará com a localidade 'C', então você deve definir uma localidade via *setlocale()* primeiro.

`locale.`**`str`**(*float*)

Formata um número de ponto flutuante usando o mesmo formato da função embutida `str(float)`, mas leva o ponto decimal em consideração.

`locale.`**`delocalize`**(*string*)

Converte uma string em uma string numérica normalizada, seguindo as configurações de *LC_NUMERIC*.

Adicionado na versão 3.5.

`locale.`**`localize`**(*string*, *grouping=False*, *monetary=False*)

Converte uma string numérica normalizada em uma string formatada, seguindo as configurações de *LC_NUMERIC*.

Adicionado na versão 3.10.

`locale.`**`atof`**(*string*, *func=float*)

Converte uma string para um número, de acordo com o valor da constante *LC_NUMERIC*, chamando *func* com o resultado da chamada à *delocalize()* em *string*.

`locale.`**`atoi`**(*string*)

Converte uma string em um número inteiro, seguindo as convenções de *LC_NUMERIC*.

`locale.`**`LC_CTYPE`**

Categoria da localidade para as funções de tipo de caracteres. Mais importante ainda, essa categoria define a codificação do texto, ou seja, como os bytes são interpretado como pontos de código Unicode. Consulte **PEP 538** e **PEP 540** para saber como essa variável pode ser automaticamente coagida para `C.UTF-8` para evitar problemas criados por configurações inválidas em contêineres ou configurações incompatíveis passadas por conexões remotas com SSH.

O Python não usa internamente funções de transformação de caracteres dependente de localidade do `ctype.h`. Em vez disso, um `pyctype.h` interno fornece equivalentes independentes de localidade como `Py_TOLOWER`.

`locale.`**`LC_COLLATE`**

Categoria da localidade para classificação de strings. As funções *strcoll()* e *strxfrm()* do módulo *locale* são afetadas.

`locale.`**`LC_TIME`**

Categoria da localidade para a formatação de hora. A função *time.strftime()* segue essas convenções.

`locale.`**`LC_MONETARY`**

Categoria da localidade para formatação de valores monetários. As opções disponíveis estão disponíveis na função *localeconv()*.

`locale.`**`LC_MESSAGES`**

Categoria da localidade para exibição de mensagens. Python atualmente não oferece suporte a mensagens com reconhecimento de localidade específicas da aplicação. Mensagens exibidas pelo sistema operacional, como aquelas retornadas por *os.strerror()* podem ser afetadas por esta categoria.

Esse valor pode não estar disponível em sistemas operacionais que não estejam em conformidade com o padrão POSIX, principalmente o Windows.

locale.**LC_NUMERIC**

Categoria da localidade para formatação de números. As funções *format_string()*, *atoi()*, *atof()* e *str()* do módulo *locale* são afetadas por essa categoria. Todas as outras operações de formatação numérica não são afetadas.

locale.**LC_ALL**

Combinação de todas as configurações da localidade. Se este sinalizador for usado quando a localidade for alterada, a configuração da localidade para todas as categorias será tentada. Se isso falhar para qualquer categoria, nenhuma categoria é alterada. Quando a localidade é recuperada usando este sinalizador, uma string indicando a configuração para todas as categorias é retornada. Esta string pode ser usada posteriormente para restaurar as configurações.

locale.**CHAR_MAX**

Esta é uma constante simbólica usada para diferentes valores retornados por *localeconv()*.

Exemplo:

```
>>> import locale
>>> loc = locale.getlocale()  # obtém localidade atual
# usa localidade de alemão; o nome pode variar conforme plataforma
>>> locale.setlocale(locale.LC_ALL, 'de_DE')
>>> locale.strcoll('f\xe4n', 'foo')  # compara uma string contendo um umlaut
>>> locale.setlocale(locale.LC_ALL, '')    # usa a localidade preferida do usuário
>>> locale.setlocale(locale.LC_ALL, 'C')   # usa a localidade padrão (C)
>>> locale.setlocale(locale.LC_ALL, loc)   # restaura localidade salva
```

24.2.1 Histórico, detalhes, dicas, dicas e advertências

O padrão C define a localidade como uma propriedade de todo o programa que pode ser relativamente cara para alterar. Além disso, algumas implementações são interrompidas de forma que mudanças frequentes de localidade podem causar despejos de memória. Isso torna a localidade um tanto dolorosa de usar corretamente.

Inicialmente, quando um programa é iniciado, a localidade é a localidade `C`, não importa qual a localidade preferida do usuário. Há uma exceção: a categoria *LC_CTYPE* é alterada na inicialização para definir a codificação de localidade atual para a codificação de localidade preferida do usuário. O programa deve dizer explicitamente que deseja as configurações de localidade preferidas do usuário para outras categorias chamando `setlocale(LC_ALL, '')`.

Geralmente é uma má ideia chamar *setlocale()* em alguma rotina de biblioteca, já que como efeito colateral afeta todo o programa. Salvar e restaurar é quase tão ruim: é caro e afeta outras threads que são executadas antes de as configurações serem restauradas.

Se, ao codificar um módulo para uso geral, você precisa de uma versão independente da localidade de uma operação que é afetada pela localidade (como certos formatos usados com *time.strftime()*), você terá que encontrar uma maneira de faça isso sem usar a rotina de biblioteca padrão. Melhor ainda é se convencer de que não há problema em usar as configurações da localidade. Apenas como último recurso, você deve documentar que seu módulo não é compatível com configurações de localidade não-`C`.

A única maneira de realizar operações numéricas de acordo com a localidade é usar as funções especiais definidas por este módulo: *atof()*, *atoi()*, *format_string()*, *str()*.

Não há como realizar conversões de maiúsculas e minúsculas e classificações de caracteres de acordo com a localidade. Para strings de texto (Unicode), isso é feito de acordo com o valor do caractere apenas, enquanto para strings de byte, as conversões e classificações são feitas de acordo com o valor ASCII do byte e bytes cujo bit alto está definido (ou seja, bytes não ASCII) nunca são convertidos ou considerados parte de uma classe de caracteres, como letras ou espaços em branco.

24.2.2 Nomes de localidades

O formato do nome de localidade depende da plataforma, e o conjunto de locais suportados pode depender da configuração do sistema.

Em plataformas Posix, isso geralmente tem o formato[1]:

```
language ["_" territory] ["." charset] ["@" modifier]
```

onde *language* é um código de idioma de duas ou três letras da ISO 639, *territory* é um código de país ou região de duas letras da ISO 3166, *charset* é uma codificação de localidade e *modifier* é um nome de script, uma subtag de idioma, um identificador de ordem de classificação ou outro modificador de localidade (por exemplo, “latin”, “valencia”, “stroke” e “euro”).

No Windows, vários formatos são suportados.[2][3] Um subconjunto de tags IETF BCP 47:

```
language ["-" script] ["-" territory] ["." charset]
language ["-" script] "-" territory "-" modifier
```

onde *language* e *territory* têm o mesmo significado que em Posix, *script* é um código de script de quatro letras da ISO 15924 e *modifier* é uma subtag de idioma, um identificador de ordem de classificação ou modificador personalizado (por exemplo, “valencia”, “stroke” ou “x-python”). São aceitos os separadores hífen (`'-'`) e sublinhado (`'_'`). Somente a codificação UTF-8 é permitida para tags BCP 47.

O Windows também oferece suporte a nomes de localidade no formato:

```
language ["_" territory] ["." charset]
```

onde *language* e *territory* são nomes completos, como “English” e “United States”, e *charset* é um número de página de código (por exemplo, “1252”) ou UTF-8. Só é aceito o separador sublinhado neste formato.

A localidade “C” é aceita em todas as plataformas.

24.2.3 Para escritores de extensão e programas que incorporam Python

Módulos de extensão nunca devem chamar *setlocale()*, exceto para descobrir qual é a localidade atual. Mas uma vez que o valor de retorno só pode ser usado portavelmente para restaurá-lo, isso não é muito útil (exceto talvez para descobrir se a localidade é ou não `C`).

Quando o código Python usa o módulo *locale* para alterar a localidade, isso também afeta a aplicação de incorporação. Se a aplicação de incorporação não quiser que isso aconteça, ele deve remover o módulo de extensão `_locale` (que faz todo o trabalho) da tabela de módulos embutidos no arquivo `config.c` e certificar-se de que o módulo `_locale` não está acessível como uma biblioteca compartilhada.

24.2.4 Acesso a catálogos de mensagens

`locale.`**`gettext`**`(`*msg*`)`

`locale.`**`dgettext`**`(`*domain*, *msg*`)`

`locale.`**`dcgettext`**`(`*domain*, *msg*, *category*`)`

`locale.`**`textdomain`**`(`*domain*`)`

`locale.`**`bindtextdomain`**`(`*domain*, *dir*`)`

`locale.`**`bind_textdomain_codeset`**`(`*domain*, *codeset*`)`

O módulo locale expõe a interface gettext da biblioteca C em sistemas que fornecem essa interface. Consiste nas funções *gettext()*, *dgettext()*, *dcgettext()*, *textdomain()*, *bindtextdomain()* e *bind_textdomain_codeset()*. Elas são semelhantes às mesmas funções no módulo *gettext*, mas usam o formato binário da biblioteca C para catálogos de mensagens e os algoritmos de pesquisa da biblioteca C para localizar catálogos de mensagens.

As aplicações Python normalmente não precisam invocar essas funções e devem usar *gettext* em seu lugar. Uma exceção conhecida a esta regra são os aplicativos que se vinculam a bibliotecas C adicionais que invocam internamente

[1] IEEE Std 1003.1-2024; 8.2 Internationalization Variables
[2] Nomes de localidade, idiomas e strings de país/região do UCRT
[3] Nomes de localidade

`gettext` ou `dcgettext`. Para essas aplicações, pode ser necessário vincular o domínio de texto, para que as bibliotecas possam localizar adequadamente seus catálogos de mensagens.

CAPÍTULO 25

Interfaces gráficas de usuário com Tk

Tk/Tcl tem sido parte integrante do Python. Ele fornece um kit de ferramentas de janela robusto e independente de plataforma, que está disponível para programadores Python usando o pacote *tkinter* e sua extensão, o módulo *tkinter.ttk*.

O pacote *tkinter* é uma fina camada orientada a objetos no topo do Tcl/Tk. Para usar *tkinter*, você não precisa escrever o código Tcl, mas precisará consultar a documentação do Tk e, ocasionalmente, a documentação do Tcl. *tkinter* é um conjunto de envólucros que implementam os widgets Tk como classes Python.

As principais virtudes do *tkinter* são que ele é rápido, e que geralmente vem junto com o Python. Embora sua documentação padrão seja fraca, um bom material está disponível, que inclui: referências, tutoriais, um livro e outros. *tkinter* também é famoso por ter uma aparência desatualizada, que foi amplamente melhorada no Tk 8.5. No entanto, existem muitas outras bibliotecas GUI nas quais você pode estar interessado. A wiki do Python lista várias frameworks e ferramentas GUI alternativas.

25.1 `tkinter` — Interface Python para Tcl/Tk

Código-fonte: Lib/tkinter/__init__.py

O pacote *tkinter* ("Tk interface") é a interface padrão do Python e faz parte do kit de ferramentas Tcl/Tk GUI. Tanto o Tk quanto o *tkinter* estão disponíveis na maioria das plataformas Unix, incluindo macOS, bem como em sistemas Windows.

Executar `python -m tkinter` na linha de comando deve abrir uma janela demonstrando uma interface Tk simples, informando que *tkinter* está apropriadamente instalado em seu sistema, e também mostrando qual versão do Tcl/Tk está instalado, para que você possa ler a documentação do Tcl/Tk específica para essa versão.

Tkinter tem suporte a uma gama de versões Tcl/Tk, compiladas com ou sem suporte a thread. O lançamento oficial do binário Python agrupa Tcl/Tk 8.6 em thread. Veja o código-fonte do módulo *_tkinter* para mais informações sobre as versões suportadas.

O Tkinter não é um invólucro fino, mas adiciona uma boa quantidade de sua própria lógica para tornar a experiência mais pythônica. Esta documentação se concentrará nessas adições e mudanças, e consulte a documentação oficial do Tcl/Tk para detalhes que não foram alterados.

Nota

Tcl/Tk 8.5 (2007) introduziu um conjunto moderno de componentes de interface de usuário para ser usados com a nova API. As APIs antigas e novas ainda estão disponíveis. A maioria da documentação que você encontrará online ainda usa a API antiga e pode estar desatualizada.

Ver também

- **TkDocs**
  Um extenso tutorial sobre como criar interfaces de usuário com o Tkinter. Explica conceitos chave, e ilustra as abordagens recomendadas usando a API moderna.
- **Referência do Tkinter 8.5: uma GUI para Python**
  Documentação de referência para Tkinter 8.5 detalhando classes, métodos e opções disponíveis.

Recursos Tcl/Tk:

- **Comandos Tk**
  Referência abrangente para cada um dos comandos Tcl/Tk subjacentes usados pelo Tkinter.
- **Tcl/Tk Website**
  Documentação adicional e links para o desenvolvimento do core do Tcl/Tk.

Livros:

- **Modern Tkinter for Busy Python Developers**
  Por Mark Roseman. (ISBN 978-1999149567)
- **Python GUI programming with Tkinter**
  por Alan D. Moore. (ISBN 978-1788835886)
- **Programming Python**
  Por Mark Lutz; tem uma excelente cobertura sobre o Tkinter. (ISBN 978-0596158101)
- **Tcl and the Tk Toolkit (2nd edition)**
  Por John Ousterhout, criador do Tcl/Tk, e Ken Jones; não cobre o Tkinter. (ISBN 978-0321336330)

25.1.1 Arquitetura

Tcl/Tk não é uma biblioteca única mas consiste em alguns módulos distintos, cada um com sua própria funcionalidade e sua própria documentação oficial. As versões binárias do Python também incluem um módulo adicional junto com ele.

Tcl
: Tcl é uma linguagem de programação interpretada dinâmica, assim como Python. Embora possa ser usado sozinho como uma linguagem de programação de propósito geral, é mais comumente embutido em aplicativos C como um mecanismo de script ou uma interface para o kit de ferramentas Tk. A biblioteca Tcl tem uma interface C para criar e gerenciar uma ou mais instâncias de um interpretador Tcl, executar comandos e scripts Tcl nessas instâncias e adicionar comandos personalizados implementados em Tcl ou C. Cada interpretador tem uma fila de eventos, e há instalações para enviar eventos e processá-los. Ao contrário do Python, o modelo de execução do Tcl é projetado em torno da multitarefa cooperativa, e o Tkinter faz a ponte entre essa diferença (veja *Modelo de threading* para detalhes).

Tk
: Tk é um pacote Tcl implementado em C que adiciona comandos personalizados para criar e manipular widgets GUI. Cada objeto `Tk` incorpora sua própria instância do interpretador Tcl com Tk carregado nele. Os widgets do Tk são muito personalizáveis, embora ao custo de uma aparência desatualizada. Tk usa a fila de eventos do Tcl para gerar e processar eventos da GUI.

Ttk
: Themed Tk (Ttk) é uma família mais recente de widgets Tk que fornecem uma aparência muito melhor em

diferentes plataformas do que muitos dos widgets Tk clássicos. O Ttk é distribuído como parte do Tk, começando com o Tk versão 8.5. As ligações Python são fornecidas em um módulo separado, *tkinter.ttk*.

Internamente, Tk e Ttk usam recursos do sistema operacional, ou seja, Xlib no Unix/X11, Cocoa no macOS, GDI no Windows.

Quando sua aplicação Python usa uma classe do Tkinter, por exemplo, para criar um widget, o módulo *tkinter* primeiro monta uma sequência de comando Tcl/Tk. Ele passa essa sequência de comando Tcl para um módulo binário interno *_tkinter*, que então chama o interpretador Tcl para avaliá-lo. O interpretador Tcl então chamará os pacotes Tk e/ou Ttk, que por sua vez farão chamadas para Xlib, Cocoa ou GDI.

25.1.2 Módulos Tkinter

O suporte para Tkinter está espalhado por vários módulos. A maioria dos aplicativos precisará do módulo *tkinter* principal, bem como do módulo *tkinter.ttk*, que fornece o conjunto de widgets temáticos modernos e a API:

```
from tkinter import *
from tkinter import ttk
```

class tkinter.**Tk**(*screenName=None*, *baseName=None*, *className='Tk'*, *useTk=True*, *sync=False*, *use=None*)

Construa um Tk widget de alto nível, sendo esse, geralmente, a janela principal de uma aplicação, e inicialize um interpretador Tcl para esse widget. Cada instância tem seu próprio interpretador Tcl associado.

A classe *Tk* normalmente é instanciada usando todos os valores padrão. No entanto, os seguintes argumentos nomeados são reconhecidos atualmente:

screenName
Quando fornecido (como uma string), define a variável de ambiente `DISPLAY`. (Somente X11)

baseName
Nome do arquivo de perfil. Por padrão, *baseName* é derivado do nome do programa (`sys.argv[0]`).

className
Nome da classe widget. Usado como um arquivo de perfil e também como o nome com o qual Tcl é invocado (*argv0* em *interp*).

useTk
Se `True`, inicialize o subsistema Tk. A função *tkinter.Tcl()* define isso como `False`.

sync
Se `True`, execute todos os comandos do servidor X de forma síncrona, para que os erros sejam relatados imediatamente. Pode ser usado para depuração. (Somente X11)

use
Especifica o *id* da janela na qual inserir a aplicação, em vez de criá-la como uma janela de nível superior separada. O *id* deve ser especificado da mesma forma que o valor da opção -use para widgets de nível superior (ou seja, tem um formato como o retornado por `winfo_id()`).

Observe que em algumas plataformas isso só funcionará corretamente se *id* se referir a um frame Tk ou nível superior que tenha sua opção -container habilitada.

Tk lê e interpreta os arquivos de perfil, chamados *.className*`.tcl` e *.baseName*`.tcl`, por meio do interpretador Tcl e chama *exec()* no conteúdo de *.className*`.py` e *.baseName*`.py`. O caminho para os arquivos de perfil encontra-se na variável de ambiente `HOME` ou, se não estiver definida, então *os.curdir*.

tk
Objeto da aplicação Tk criado ao instanciar *Tk*. Isso fornece acesso ao interpretador Tcl. Cada widget anexado à mesma instância de *Tk* tem o mesmo valor para o atributo *tk*.

master
O objeto widget que contém este widget. Para *Tk*, o *master* é *None* porque é a janela principal. Os termos *master* e *parent* são semelhantes e às vezes usados alternadamente como nome de argumento; No entanto, chamar `winfo_parent()` retorna uma string com o nome do widget enquanto *master* retorna o objeto. *parent*/*child* reflete a relação de árvore enquanto *master*/*slave* reflete a estrutura do contêiner.

children

Descendentes diretos deste widget como um *dict* com os nomes do widget filho como a chave e os objetos de instância filho como o valor.

tkinter.**Tcl**(*screenName=None*, *baseName=None*, *className='Tk'*, *useTk=False*)

A função *Tcl()* é uma função fábrica que cria um objeto assim como o criado pela classe *Tk*, exceto por não inicializar o subsistema Tk. Isto é útil quando executando o interpretador Tcl em um ambiente onde não se quer criar janelas de alto nível externas, ou quando não se pode (como em sistemas Unix/Linux sem um servidor X). Um objeto criado pelo objeto *Tcl()* pode ter uma janela de alto nível criada (e o subsistema Tk inicializado) chamando o método `loadtk()`.

Os módulos que fornecem suporte ao Tk incluem:

tkinter
Módulo principal Tkinter.

tkinter.colorchooser
Caixa de diálogo para permitir que o usuário escolha uma cor.

tkinter.commondialog
Classe base para os diálogos definidos nos outros módulos listados aqui.

tkinter.filedialog
Diálogos comuns para permitir ao usuário especificar um arquivo para abrir ou salvar.

tkinter.font
Utilitários para ajudar a trabalhar com fontes.

tkinter.messagebox
Acesso às caixas de diálogo padrão do Tk.

tkinter.scrolledtext
Componente de texto com uma barra de rolagem vertical integrada.

tkinter.simpledialog
Diálogos básicos e funções de conveniência.

tkinter.ttk
Conjunto de widgets temáticos introduzidos no Tk 8.5, fornecendo alternativas modernas para muitos dos widgets clássicos no módulo principal *tkinter*.

Módulos adicionais:

_tkinter
Um módulo binário que contém a interface de baixo nível para Tcl/Tk. Ele é importado automaticamente pelo módulo principal *tkinter* e nunca deve ser usado diretamente. Geralmente é uma biblioteca compartilhada (ou DLL), mas pode, em alguns casos, ser vinculada estaticamente ao interpretador Python.

idlelib
Ambiente Integrado de Desenvolvimento e Aprendizagem do Python (IDLE). Baseado em *tkinter*.

tkinter.constants
Constantes simbólicas que podem ser usadas no lugar de strings ao passar vários parâmetros para chamadas Tkinter. Este módulo é importado automaticamente pelo módulo principal *tkinter*.

tkinter.dnd
(experimental) Suporte de arrastar e soltar para *tkinter*. Será descontinuado quando for substituído pelo Tk DND.

turtle
Gráficos tartaruga em uma janela Tk.

25.1.3 Preservador de vida Tkinter

Esta seção não foi projetada para ser um tutorial exaustivo sobre Tk ou Tkinter. Para isso, consulte um dos recursos externos mencionados anteriormente. Em vez disso, esta seção fornece uma orientação muito rápida sobre como é uma aplicação Tkinter, identifica os conceitos fundamentais do Tk e explica como o wrapper do Tkinter é estruturado.

O restante desta seção ajudará a identificar as classes, métodos e opções que você precisará em uma aplicação Tkinter e onde encontrar documentação mais detalhada sobre isso, inclusive no guia de referência oficial do Tcl/Tk.

Um programa Olá Mundo

Começaremos a explorar o Tkinter através de uma simples aplicação "Hello World". Esta não é a menor aplicação que poderíamos escrever, mas tem o suficiente para ilustrar alguns conceitos-chave que você precisa saber.

```
from tkinter import *
from tkinter import ttk
root = Tk()
frm = ttk.Frame(root, padding=10)
frm.grid()
ttk.Label(frm, text="Hello World!").grid(column=0, row=0)
ttk.Button(frm, text="Quit", command=root.destroy).grid(column=1, row=0)
root.mainloop()
```

Após as importações, a próxima linha será criar a instância da classe `Tk` que inicia Tk e cria o interpretador associado a Tcl. Ele também cria uma janela de nível superior, conhecida como janela raiz, que serve como a janela principal da aplicação.

A linha a seguir cria um frame widget , que neste caso conterá um rótulo e um botão que criaremos a seguir. O frame se encaixa dentro da janela raiz.

A próxima linha cria um rótulo widget contendo uma string de texto estático. O método `grid()` é utilizado para especificar o layout relativo (posição) do rótulo dentro do frame widget, semelhante a como as tabelas em HTML funcionam.

Então um widget de botão é criado, e posicionado à direita do rótulo. Quando pressionado, irá chamar o método `destroy()` da janela raiz.

Por fim, o método `destroy()` coloca tudo no display, e responde à entrada do usuário até que o programa termine.

Conceitos importantes do Tk

Mesmo com este programa simples, os conceitos-chave do Tk podem ser mostrados:

widgets
: A interface de usuário do Tkinter é composta de *widgets* individuais. Cada widget é representado como um objeto Python, instanciado de classes como `ttk.Frame`, `ttk.Label`, e `ttk.Button`.

hierarquia dos widgets
: Os widgets são organizados em uma *hierarquia*. O rótulo e botão estavam contidos em um frame, que por sua vez estava contido na janela raiz. Quando criamos um widget *filho*, seu widget *pai* é passado como primeiro argumento para o construtor do widget.

opções de configuração
: Widgets possuem *opções de configuração*, que modificam sua aparência e comportamento, como o texto a ser exibido em um rótulo ou botão. Diferentes classes de widgets terão diferentes conjuntos de opções.

gerenciamento de geometria
: Os widgets não são automaticamente adicionados à interface de usuário quando eles são criados. Um *gerenciador de geometria* como `grid` controla onde na interface de usuário eles são colocados.

loop de eventos
: O Tkinter reage à entrada do usuário, muda de seu programa, e até mesmo atualiza a tela somente ao executar ativamente um *loop de eventos*. Se o seu programa não estiver executando o loop de eventos, sua interface de usuário não será atualizada.

Entendendo como Tkinter envolve Tcl/Tk

Quando a sua aplicação utiliza as classes e métodos do Tkinter, internamente o Tkinter está montando strings representando comandos Tcl/Tk e executando esses comandos no interpretador Tcl anexado à instância `Tk` de sua aplicação.

Seja tentando navegar na documentação de referência, tentando encontrar o método ou opção certa, adaptando algum código existente ou depurando seu aplicativo Tkinter, há momentos em que será útil entender como são os comandos Tcl/Tk subjacentes.

Para ilustrar, aqui está o equivalente Tcl/Tk da parte principal do script Tkinter acima.

```
ttk::frame .frm -padding 10
grid .frm
grid [ttk::label .frm.lbl -text "Hello World!"] -column 0 -row 0
grid [ttk::button .frm.btn -text "Quit" -command "destroy ."] -column 1 -row 0
```

A sintaxe do Tcl é semelhante a muitas linguagens de shell, onde a primeira palavra é o comando a ser executado, seguido de argumentos para esse comando separados por espaços. Sem entrar em muitos detalhes, observe o seguinte:

- Os comandos usados para criar widgets (como `ttk::frame`) correspondem a classes de widgets no Tkinter.
- As opções de widget Tcl (como `-text`) correspondem a argumentos nomeados no Tkinter.
- Widgets são referenciados por um *pathname* em Tcl (like `.frm.btn`), enquanto o Tkinter não utiliza nomes, mas referências a objetos.
- O lugar de um widget na hierarquia widget é codificado em seu pathname (hierárquico), que utiliza um `.` (ponto) como um separador de caminho. O pathname para a janela raiz é apenas `.` (ponto). No Tkinter, a hierarquia é definida não pelo pathname, mas pela especificação do widget pai ao criar cada widget filho.
- Operações que são implementadas como *comandos* separados em Tcl (como `grid` ou `destroy`) são representadas como *métodos* em objetos de widget Tkinter. Como você verá em breve, em outras ocasiões o Tcl usa o que parecem ser chamadas de método em objetos widget, que espelham mais de perto o que seria usado no Tkinter.

Como é que eu...? Que opção faz...?

Se você não tem certeza de como fazer algo no Tkinter e não consegue encontrá-lo imediatamente no tutorial ou na documentação de referência que está usando, existem algumas estratégias que podem ser úteis.

Primeiro, lembre-se de que os detalhes de como os widgets individuais funcionam podem variar entre diferentes versões tanto do Tkinter quanto do Tcl/Tk. Se você estiver procurando por documentação, certifique-se de que corresponda às versões do Python e Tcl/Tk instaladas em seu sistema.

Ao procurar como usar uma API, é útil saber o nome exato da classe, opção ou método que você está usando. A introspecção, seja em um console Python interativo ou com a *`print()`*, pode ajudar a identificar o que você precisa.

Para descobrir quais opções de configuração estão disponíveis em qualquer widget, chame o método `configure()`, que retorna um dicionário contendo uma variedade de informações sobre cada objeto, incluindo seus valores padrão e atuais. Use `keys()` para obter apenas os nomes de cada opção.

```
btn = ttk.Button(frm, ...)
print(btn.configure().keys())
```

Como a maioria dos widgets tem muitas opções de configuração em comum, pode ser útil descobrir quais são específicas para uma determinada classe de widget. Comparar a lista de opções com a de um widget mais simples, como um frame, é uma maneira de fazer isso.

```
print(set(btn.configure().keys()) - set(frm.configure().keys()))
```

Da mesma forma, você pode encontrar os métodos disponíveis para um objeto widget usando a função padrão *`dir()`*. Se você tentar, verá que existem mais de 200 métodos comuns de widget, então identificar aqueles específicos para uma classe de widget é útil.

```
print(dir(btn))
print(set(dir(btn)) - set(dir(frm)))
```

Navegando no Manual de Referência Tcl/Tk

Como observado, o manual (páginas man) de referência oficial dos comandos Tk é frequentemente a descrição mais precisa do que operações específicas em widgets fazem. Mesmo quando você sabe o nome da opção ou método que você precisa, você ainda pode ter alguns lugares para procurar.

Enquanto todas as operações no Tkinter são implementadas como chamadas de método em objetos de widget, você viu que muitas operações Tcl/Tk aparecem como comandos que recebem um caminho de widget como seu primeiro parâmetro, seguido de parâmetros opcionais, por exemplo.

```
destroy .
grid .frm.btn -column 0 -row 0
```

Outros, no entanto, parecem mais com métodos chamados em um objeto widget (na verdade, quando você cria um widget em Tcl/Tk, ele cria um comando Tcl com o nome do caminho do widget, sendo o primeiro parâmetro desse comando o nome de um método a ser chamado).

```
.frm.btn invoke
.frm.lbl configure -text "Goodbye"
```

Na documentação oficial de referência do Tcl/Tk, você encontrará a maioria das operações que se parecem com chamadas de método na página man de um widget específico (por exemplo, você encontrará o método `invoke()` na página man do ttk::button), enquanto as funções que recebem um widget como parâmetro geralmente têm sua própria página man (por exemplo, grid).

Você encontrará muitas opções e métodos comuns nas páginas man options e ttk::widget, enquanto outros são encontrados na página man para uma classe de widget específica.

Você também encontrará que muitos métodos do Tkinter têm nomes compostos, por exemplo, `winfo_x()`, `winfo_height()`, `winfo_viewable()`. Você encontra documentação para todos eles na página man winfo.

Nota

De forma bem confusa, também existem métodos em todos os widgets do Tkinter que na verdade não operam no widget, mas operam em um escopo global, independente de qualquer widget. Exemplos são métodos para acessar a área de transferência ou o sinal do sistema. (Eles são implementados como métodos na classe base `Widget` da qual todos os widgets do Tkinter herdam).

25.1.4 Modelo de threading

Python e Tcl/Tk têm modelos de threading muito diferentes, que *`tkinter`* tenta fazer a ponte. Se você usa threads, pode precisar estar ciente disso.

Um interpretador Python pode ter muitas threads associados a ele. Em Tcl, várias threads podem ser criadas, mas cada thread tem uma instância de interpretador Tcl separada associada a ele. Threads também podem criar mais de uma instância do interpretador, embora cada instância do interpretador possa ser usada apenas pela thread que a criou.

Cada objeto `Tk` criado por *`tkinter`* contém um interpretador Tcl. Ele também controla qual thread criou aquele interpretador. Chamadas para *`tkinter`* podem ser feitas a partir de qualquer thread do Python. Internamente, se uma chamada vier de um thread diferente daquele que criou o objeto `Tk`, um evento é postado na fila de eventos do interpretador e, quando executado, o resultado é retornado ao thread de chamada do Python.

As aplicações Tcl/Tk são normalmente orientadas a eventos, o que significa que após a inicialização, o interpretador executa um laço de eventos (ou seja `Tk.mainloop()`) e responde aos eventos. Por ser de thread única, os tratadores de eventos devem responder rapidamente, caso contrário, bloquearão o processamento de outros eventos. Para evitar isso, quaisquer cálculos de longa execução não devem ser executados em um tratador de eventos, mas são divididos

em pedaços menores usando temporizadores ou executados em outra thread. Isso é diferente de muitos kits de ferramentas de GUI em que a GUI é executada em um thread completamente separado de todo o código do aplicação, incluindo tratadores de eventos.

Se o interpretador Tcl não estiver executando o laço de eventos e processando eventos, qualquer chamada *tkinter* feita de threads diferentes daquela que está executando o interpretador do Tcl irá falhar.

Existem vários casos especiais:

- Bibliotecas Tcl/Tk podem ser construídas de forma que não reconheçam thread. Neste caso, *tkinter* chama a biblioteca da thread originadora do Python, mesmo que seja diferente da thread que criou o interpretador Tcl. Uma trava global garante que apenas uma chamada ocorra por vez.
- Enquanto *tkinter* permite que você crie mais de uma instância de um objeto `Tk` (com seu próprio interpretador), todos os interpretadores que fazem parte da mesma thread compartilham uma fila de eventos comum, o que fica muito rápido. Na prática, não crie mais de uma instância de `Tk` por vez. Caso contrário, é melhor criá-los em threads separadas e garantir que você esteja executando uma construção Tcl/Tk com reconhecimento de thread.
- Bloquear manipuladores de eventos não é a única maneira de evitar que o interpretador Tcl entre novamente no laço de eventos. É ainda possível executar vários laços de eventos aninhados ou abandonar totalmente o laço de eventos. Se você estiver fazendo algo complicado quando se trata de eventos ou threads, esteja ciente dessas possibilidades.
- Existem algumas funções de seleção do *tkinter* que atualmente funcionam apenas quando chamadas a partir da thread que criou o interpretador Tcl.

25.1.5 Referência Útil

Opções de Definição

As opções controlam coisas como a cor e a largura da borda de um widget. As opções podem ser definidas de três maneiras:

No momento da criação do objeto, usando argumentos nomeados

```
fred = Button(self, fg="red", bg="blue")
```

Após a criação do objeto, tratando o nome da opção como um índice de dicionário

```
fred["fg"] = "red"
fred["bg"] = "blue"
```

Use o método config() para atualizar vários attrs posteriores à criação do objeto

```
fred.config(fg="red", bg="blue")
```

Para uma explicação completa de uma dada opção e seu comportamento, veja as páginas man do Tk para o widget em questão.

Note que as páginas man listam "OPÇÕES PADRÃO" e "OPÇÕES DE WIDGET ESPECÍFICO" para cada widget. A primeira é uma lista de opções que são comuns a vários widgets, e a segunda são opções que são idiossincráticas ao widget em particular. As Opções Padrão estão documentadas na página man *options(3)*.

Nenhuma distinção entre as opções padrão e específicas de widget são feitas neste documento. Algumas opções não se aplicam a alguns tipos de widgets. A resposta de um dado widget a uma opção particular depende da classe do widget; botões possuem a opção `command`, etiquetas não.

As opções suportadas por um dado widget estão listadas na página man daquele widget, ou podem ser consultadas no tempo de execução chamando o método `config()` sem argumentos, ou chamando o método `keys()` daquele widget. O valor retornado por essas chamadas é um dicionário cujas chaves são os nomes das opções como uma string (por exemplo, `'relief'`) e cujos valores são tuplas de 5 elementos.

Algumas opções, como `bg` são sinônimos para opções comuns com nomes mais longos (`bg` é a abreviação de "background", ou plano de fundo em inglês). Passando o nome de uma opção abreviada ao método `config()` irá retornar

uma tupla de 2 elementos, e não uma tupla de 5 elementos. A tupla de 2 elementos devolvida irá conter o nome do sinônimo e a opção "verdadeira" (como por exemplo em `('bg', 'background')`).

Índice	Significado	Exemplo
0	nome da opção	`'relief'`
1	nome da opção para buscas de banco de dados	`'relief'`
2	classe de opção para busca de banco de dados	`'Relief'`
3	valor padrão	`'raised'`
4	valor atual	`'groove'`

Exemplo:

```
>>> print(fred.config())
{'relief': ('relief', 'relief', 'Relief', 'raised', 'groove')}
```

É claro que o dicionário exibido irá incluir todas as opções disponíveis e seus valores. Isso foi apenas um exemplo.

O Empacotador

O empacotador é um dos mecanismos de gerenciamento de geometria do Tk. Gerenciadores de geometria são utilizados para especificar as posições relativas dos widgets dentro dos seus contêineres - os seus *processos pai* mútuos. Em contraste ao mais incômodo *posicionador* (que é usado com menos frequência, e não falaremos dele aqui), o empacotador recebe a especificação de relacionamento qualitativo - *acima*, *à esquerda de*, *preenchimento*, etc. - e determina as coordenadas de posição exatas para você.

O tamanho de qualquer widget *mestre* é determinado pelo tamanho dos "widgets escravos" internos. O empacotador é usado para controlar onde os widgets escravos aparecem dentro do mestre no qual são empacotados. Você pode empacotar widgets em quadros e quadros em outros quadros, a fim de obter o tipo de layout que deseja. Além disso, o arranjo é ajustado dinamicamente para acomodar alterações incrementais na configuração, uma vez que é empacotado.

Note que os widgets não aparecem até que eles tenham sua geometria especificada pelo gerenciador de geometria. É um erro comum de iniciantes deixar a geometria de fora da especificação, então se surpreender que o widget é criado sendo que nada apareceu. O widget irá aparecer apenas quando tiver, por exemplo, o método método `pack()` do empacotador aplicado a ele.

O método pack() pode ser chamado com pares de palavra reservada-opção/valor que controlam onde o widget deverá aparecer dentro do seu contêiner, e como deverá se comportar quando a janela da aplicação principal for redimensionada. Aqui estão alguns exemplos:

```
fred.pack()                     # tem como padrão side = "top"
fred.pack(side="left")
fred.pack(expand=1)
```

Opções do Empacotador

Para uma informação mais completa do empacotador e as opções que pode receber, veja as páginas man e a página 183 do livro do John Ousterhout.

anchor
: Tipo âncora. Denota onde o packer deverá posicionar cada ajudante em seu pacote.

expand
: Booleano, `0` ou `1`.

fill
: Valores legais: `'x'`, `'y'`, `'both'`, `'none'`.

ipadx e ipady
: Uma distância - designando o deslocamento interno de cada lado do widget ajudante.

padx e pady
: Uma distância - designando o deslocamento externo de cada lado do widget ajudante.

side
: Valores legais são: `'left'`, `'right'`, `'top'`, `'bottom'`.

Acoplando Variáveis de Widgets

A definição do valor atual de alguns widgets (como widgets de entrada de texto) podem ser conectadas diretamente às variáveis da aplicação utilizando métodos especiais. Estas opções são `variable`, `textvariable`, `onvalue`, `offvalue`, e `value`. A conexão funciona por ambos os lados: Se por algum motivo a variável se altera, o widget ao qual ela está conectada irá se atualizar para refletir este novo valor.

Infelizmente, na atual implementação do *tkinter* não é possível passar uma variável Python arbitrária para um widget por uma opção `variable` ou `textvariable`. Os únicos tipos de variáveis para as quais isso funciona são variáveis que são subclasses da classe chamada Variable, definida em *tkinter*.

Há muitas subclasses de Variable úteis já definidas: `StringVar`, `IntVar`, `DoubleVar`, e `BooleanVar`. Para ler o atual valor de uma variável, chame o método `get()` nelas, e para alterar o valor você deve chamar o método `set()`. Se você seguir este protocolo, o widget irá sempre acompanhar os valores da variável, sem nenhuma intervenção da sua parte.

Por exemplo:

```
import tkinter as tk

class App(tk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.pack()

        self.entrythingy = tk.Entry()
        self.entrythingy.pack()

        # Cria a variável da aplicação.
        self.contents = tk.StringVar()
        # Define-a com algum valor.
        self.contents.set("this is a variable")
        # Fala para o widget de entrada para monitorar essa variável.
        self.entrythingy["textvariable"] = self.contents

        # Define um retorno de chamada para quando o usuário pressionar Enter.
        # Isso imprime o valor atual da variável.
        self.entrythingy.bind('<Key-Return>',
                              self.print_contents)

    def print_contents(self, event):
        print("Hi. The current entry content is:",
              self.contents.get())

root = tk.Tk()
myapp = App(root)
myapp.mainloop()
```

O Gerenciador de Janela

No Tk, existe um comando utilitário, `wm`, para interagir com o gerenciador de janelas. As opções do comando `wm` permitem que você controle coisas como títulos, localização, ícones bitmap, entre outros. No *tkinter*, estes comandos foram implementados como métodos da classe `Wm` class. Widgets de mais alto nível são subclasses da classe `Wm` e portanto podem chamar os métodos `Wm` diretamente.

Para chegar à janela de nível superior que contém um determinado widget, geralmente você pode apenas consultar o mestre do widget. Claro, se o widget foi compactado dentro de um quadro, o mestre não representará uma janela de nível superior. Para chegar à janela de nível superior que contém um widget arbitrário, você pode chamar o método `_root()`. Este método começa com um sublinhado para denotar o fato de que esta função é parte da implementação, e não uma interface para a funcionalidade Tk.

Aqui estão alguns exemplos de uso típico:

```
import tkinter as tk

class App(tk.Frame):
    def __init__(self, master=None):
        super().__init__(master)
        self.pack()

# Cria a aplicação
myapp = App()

#
# aqui estão as chamadas de método para a classe do gerenciador de janelas
#
myapp.master.title("Meu Aplicativo Que Faz Nada")
myapp.master.maxsize(1000, 400)

# inicia o programa
myapp.mainloop()
```

Opções de Tipos de Dados do Tk

anchor
: Valores legais são os pontos cardeais: `"n"`, `"ne"`, `"e"`, `"se"`, `"s"`, `"sw"`, `"w"`, `"nw"`, e também `"center"`.

bitmap
: Há 8 bitmaps embutidos nomeados: `'error'`, `'gray25'`, `'gray50'`, `'hourglass'`, `'info'`, `'questhead'`, `'question'`, `'warning'`. Para especificar um nome de arquivo do bitmap X, passe o caminho completo do arquivo, precedido pelo caractere `@`, como em `"@/usr/contrib/bitmap/gumby.bit"`.

booleano
: Você pode passar os inteiros 0 ou 1 ou as strings `"yes"` ou `"no"`.

função de retorno
: Esta é uma função Python que não aceita argumentos. Por exemplo:

```
def print_it():
    print("hi there")
fred["command"] = print_it
```

cor
: As cores podem ser fornecidas como nomes de cores X no arquivo rgb.txt ou como strings representando valores RGB em intervalos de 4 bits: `"#RGB"`, 8 bit: `"#RRGGBB"`, 12 bit: `"#RRRGGGBBB"` ou 16 bit: `"#RRRRGGGGBBBB"`, onde R,G,B aqui representam qualquer dígito hexadecimal legal. Veja a página 160 do livro de Ousterhout para detalhes.

cursor
: Os nomes de cursor X padrão de `cursorfont.h` podem ser usados, sem o prefixo `XC_`. Por exemplo, para obter um cursor de mão (`XC_hand2`), use a string `"hand2"`. Você também pode especificar um bitmap e um arquivo de máscara de sua preferência. Veja a página 179 do livro de Ousterhout.

distance
: As distâncias da tela podem ser especificadas em pixels ou distâncias absolutas. Pixels são dados como números e distâncias absolutas como strings, com os caracteres finais denotando unidades: `c` para centímetros, `i` para

polegadas, `m` para milímetros, `p` para os pontos da impressora. Por exemplo, 3,5 polegadas é expresso como `"3.5i"`.

font
Tk usa um formato de nome de fonte de lista, como `{courier 10 bold}`. Tamanhos de fonte com números positivos são medidos em pontos; tamanhos com números negativos são medidos em pixels.

geometria
Esta é uma string no formato `widthxheight`, onde largura e altura são medidas em pixels para a maioria dos widgets (em caracteres para widgets que exibem texto). Por exemplo: `fred["geometry"] = "200x100"`.

justify
Valores aceitos são as strings: `"left"`, `"center"`, `"right"` e `"fill"`.

region
Esta é uma string com quatro elementos delimitados por espaço, cada um dos quais a uma distância legal (veja acima). Por exemplo: `"2 3 4 5"` e `"3i 2i 4.5i 2i"` e `"3c 2c 4c 10.43c"` são todas regiões legais.

relief
Determina qual será o estilo da borda de um widget. Os valores legais são: `"raised"`, `"sunken"`, `"flat"`, `"groove"` e `"ridge"`.

scrollcommand
Este é quase sempre o método `set()` de algum widget da barra de rolagem, mas pode ser qualquer método de widget que receba um único argumento.

wrap
Deve ser um de: `"none"`, `"char"` ou `"word"`.

Ligações e Eventos

O método de ligação do comando widget permite que você observe certos eventos e tenha um acionamento de função de retorno de chamada quando esse tipo de evento ocorrer. A forma do método de ligação é:

```
def bind(self, sequence, func, add=''):
```

sendo que:

sequence
é uma string que denota o tipo de evento de destino. (Veja a página man do *bind(3tk)* e a página 201 do livro de John Ousterhout, *Tcl and the Tk Toolkit (2nd edition)*, para mais detalhes).

func
é uma função Python, tendo um argumento, a ser invocada quando o evento ocorre. Uma instância de evento será passada como argumento. (As funções implantadas dessa forma são comumente conhecidas como *funções de retorno*.)

add
é opcional, tanto `''` ou `'+'`. Passar uma string vazia indica que essa ligação substitui todas as outras ligações às quais esse evento está associado. Passar um `'+'` significa que esta função deve ser adicionada à lista de funções ligadas a este tipo de evento.

Por exemplo:

```
def turn_red(self, event):
    event.widget["activeforeground"] = "red"

self.button.bind("<Enter>", self.turn_red)
```

Observe como o campo widget do evento está sendo acessado na função de retorno `turn_red()`. Este campo contém o widget que capturou o evento X. A tabela a seguir lista os outros campos de evento que você pode acessar e como eles são denotados no Tk, o que pode ser útil ao se referir às páginas man do Tk.

Tk	Campo de Eventos do Tkinter	Tk	Campo de Eventos do Tkinter
%f	focus	%A	char
%h	height	%E	send_event
%k	keycode	%K	keysym
%s	state	%N	keysym_num
%t	time	%T	tipo
%w	width	%W	widget
%x	x	%X	x_root
%y	y	%Y	y_root

O Parâmetro index

Vários widgets requerem que parâmetros de "indx" sejam passados. Eles são usados para apontar para um local específico em um widget Text, ou para caracteres específicos em um widget Entry, ou para itens de menu específicos em um widget Menu.

Índices de widget de entrada (índice, índice de visualização, etc.)
Os widgets de entrada têm opções que se referem às posições dos caracteres no texto exibido. Você pode acessar esses pontos especiais em um widget de texto usando as seguintes funções *tkinter*:

Índice de widget de texto
A notação de índice do widget de texto é muito rica e é melhor detalhada nas páginas do manual Tk.

Índices de menu (menu.invoke(), menu.entryconfig(), etc.)
Algumas opções e métodos de menus manipulam entradas de menu específicas. Sempre que um índice de menu é necessário para uma opção ou parâmetro, você pode passar:

- um número inteiro que se refere à posição numérica da entrada no widget, contada do topo, começando com 0;
- a string `"active"`, que se refere à posição do menu que está atualmente sob o cursor;
- a string `"last"` que se refere ao último item do menu;
- Um inteiro precedido por `@`, como em `@6`, onde o inteiro é interpretado como uma coordenada de pixel y no sistema de coordenadas do menu;
- a string `"none"`, que indica nenhuma entrada de menu, mais frequentemente usada com menu.activate() para desativar todas as entradas e, finalmente,
- uma string de texto cujo padrão corresponde ao rótulo da entrada do menu, conforme varrido da parte superior do menu para a parte inferior. Observe que este tipo de índice é considerado após todos os outros, o que significa que as correspondências para itens de menu rotulados como `last`, `active` ou `none` podem ser interpretadas como os literais acima, em vez disso.

Imagens

Imagens de diferentes formatos podem ser criadas por meio da subclasse correspondente de `tkinter.Image`:

- `BitmapImage` para imagens no formato XBM.
- `PhotoImage` para imagens nos formatos PGM, PPM, GIF e PNG. O suporte ao último foi adicionado a partir do Tk 8.6.

Qualquer tipo de imagem é criado através da opção `file` ou `data` (outras opções também estão disponíveis).

Alterado na versão 3.13: Adicionado o método `copy_replace()` à classe `PhotoImage` para copiar uma região de uma imagem para outra imagem, possivelmente com ampliação e subamostragem de pixels. Adiciona parâmetro *from_coords* para os métodos `copy()`, `zoom()` e `subsample()` da classe `PhotoImage`. Adiciona parâmetros *zoom* e *subsample* para o método `copy()` da classe `PhotoImage`.

O objeto imagem pode então ser usado sempre que uma opção `image` há suporte em algum widget (por exemplo: rótulos, botões, menus). Nestes casos, o Tk não guardará uma referência à imagem. Quando a última referência

Python ao objeto imagem for excluída, os dados da imagem também serão excluídos e o Tk exibirá uma caixa vazia onde quer que a imagem tenha sido usada.

Ver também

O pacote Pillow adiciona suporte para formatos como BMP, JPEG, TIFF e WebP, entre outros.

25.1.6 Tratadores de arquivos

O Tk permite que você registre e cancele o registro de uma função de retorno que será chamada a partir do laço central do Tk quando uma E/S for possível em um descritor de arquivo. Apenas um tratador pode ser registrado por descritor de arquivo. Código de exemplo:

```
import tkinter
widget = tkinter.Tk()
mask = tkinter.READABLE | tkinter.WRITABLE
widget.tk.createfilehandler(file, mask, callback)
...
widget.tk.deletefilehandler(file)
```

Este recurso não está disponível no Windows.

Já que você não sabe quantos bytes estão disponíveis para leitura, você pode não querer usar os métodos *read()* ou *readline()* de *BufferedIOBase* ou *TextIOBase*, já que eles insistirão em ler uma quantidade predefinida de bytes. Para sockets, os métodos *recv()* ou *recvfrom()* vão servir; para outros arquivos, use dados brutos ou `os.read(file.fileno(), maxbytecount)`.

`Widget.tk.`**`createfilehandler`**(*file*, *mask*, *func*)

Registra a função de retorno do tratador de arquivo *func*. O argumento *file* pode ser um objeto com um método *fileno()* (como um arquivo ou objeto soquete) ou um descritor de arquivo de inteiro. O argumento *mask* é uma combinação OR de qualquer uma das três constantes abaixo. O retorno de chamada é chamado da seguinte maneira:

```
callback(file, mask)
```

`Widget.tk.`**`deletefilehandler`**(*file*)

Cancela o registro de um tratador de arquivo.

`_tkinter.`**`READABLE`**

`_tkinter.`**`WRITABLE`**

`_tkinter.`**`EXCEPTION`**

Constantes usadas nos argumentos *mask*.

25.2 `tkinter.colorchooser` — Diálogo de escolha de cor

Código-fonte: Lib/tkinter/colorchooser.py

O módulo *tkinter.colorchooser* fornece a classe *Chooser* como uma interface para o diálogo do seletor de cores nativo. `Chooser` implementa uma janela de diálogo de escolha de cores modal. A classe `Chooser` herda da classe *Dialog*.

`class` `tkinter.colorchooser.`**`Chooser`**(*master=None*, ***options*)

`tkinter.colorchooser.`**`askcolor`**(*color=None*, ***options*)

Cria um diálogo de escolha de cores. Uma chamada para esse método mostrará a janela, aguardará o usuário fazer uma seleção e retornará a cor selecionada (ou `None`) ao chamador.

Ver também

Módulo *tkinter.commondialog*
Módulo de diálogo padrão do Tkinter

25.3 `tkinter.font` — Invólucro de fontes Tkinter

Código-fonte: Lib/tkinter/font.py

O módulo *tkinter.font* apresenta a classe *Font* para criação e utilização de fontes nomeadas.

Os diferentes pesos e inclinações de fontes são:

`tkinter.font.`**`NORMAL`**

`tkinter.font.`**`BOLD`**

`tkinter.font.`**`ITALIC`**

`tkinter.font.`**`ROMAN`**

class `tkinter.font.`**Font**(*root=None*, *font=None*, *name=None*, *exists=False*, ***options*)

A classe *Font* representa uma fonte nomeada. Instâncias *font* recebem nomes únicos e podem ser especificadas por família, tamanho e configuração de estilo. Fontes nomeadas são os métodos Tk para criação e identificação de fontes de um objeto singular, ao invés de especificar a fonte por seus atributos em cada ocorrência.

argumentos:

font - tupla de especificação da fonte (família, tamanho, opções)
name - nome único da fonte
exists - aponta para uma fonte nomeada existente se for verdadeiro

palavras reservadas opcionais (ignoradas caso *font* seja especificado):

family - a família da fonte e.x. Courier, Times
size - tamanho da fonte
Se *size* for positivo, ele é interpretado como tamanho em pontos.
Caso *size* seja um número negativo, seu valor absoluto é tratado como tamanho em pixels.
weight - ênfase da fonte (normal,negrito)
slant - ROMAN, ITALIC
underline - sublinhamento da fonte (0 - sem sublinhamento, 1 - sublinhado)
overstrike - fonte riscada (0 - sem risco, 1 - letras riscadas)

actual(*option=None*, *displayof=None*)

Retorna os atributos da fonte.

cget(*option*)

Recupera um atributo da fonte.

config(***options*)

Modifica atributos da fonte.

copy()

Retorna uma nova instância da fonte atual.

measure(*text*, *displayof=None*)

Retorna a quantidade de espaço que o texto ocuparia na exibição especificada ao ser formatado na fonte atual. Caso nenhuma exibição seja especificada, a exibição presumida será a da janela principal da aplicação.

metrics(**options*, ***kw*)

Retorna dados específicos da fonte. Opções inclusas:

***ascent* - distância entre a linha de base e o ponto mais alto que um** caractere da fonte pode ocupar

***descent* - distância entre a linha de base e o ponto mais baixo que um** caractere da fonte pode ocupar

***linespace* - separação vertical mínima necessária entre dois** caracteres da fonte que garante que não ocorra sobreposição entre as linhas.

fixed - 1 caso a fonte seja de largura fixa, 0 caso contrário

tkinter.font.**families**(*root=None*, *displayof=None*)

Retorna as diferentes famílias de fontes.

tkinter.font.**names**(*root=None*)

Retorna os nomes das fontes definidas.

tkinter.font.**nametofont**(*name*, *root=None*)

Retorna a representação *Font* de uma fonte nomeada do tk.

Alterado na versão 3.10: O parâmetro *root* foi adicionado.

25.4 Diálogos Tkinter

25.4.1 tkinter.simpledialog — Diálogos de entrada padrão do Tkinter

Código-fonte: Lib/tkinter/simpledialog.py

O módulo *tkinter.simpledialog* contém classes de conveniência e funções para criar diálogos modais simples para obter um valor do usuário.

tkinter.simpledialog.**askfloat**(*title*, *prompt*, ***kw*)

tkinter.simpledialog.**askinteger**(*title*, *prompt*, ***kw*)

tkinter.simpledialog.**askstring**(*title*, *prompt*, ***kw*)

As três funções acima fornecem caixas de diálogo que solicitam que o usuário insira um valor do tipo desejado.

class tkinter.simpledialog.**Dialog**(*parent*, *title=None*)

A classe base para diálogos personalizados.

body(*master*)

Substitui para construir a interface da caixa de diálogo e retornar o widget que deve ter foco inicial.

buttonbox()

O comportamento padrão adiciona botões OK e Cancelar. Substitua para layouts de botão personalizados.

25.4.2 tkinter.filedialog — Caixas de diálogo de seleção de arquivo

Código-fonte: Lib/tkinter/filedialog.py

The módulo *tkinter.filedialog* fornece classes e funções de fábrica para criar janelas de seleção de arquivo/diretório.

Caixas de diálogo nativos de carregar/salvar

As seguintes classes e funções fornecem janelas de diálogo de arquivo que combinam uma aparência nativa com opções de configuração para personalizar o comportamento. Os seguintes argumentos nomeados são aplicáveis às classes e funções listado abaixo:

parent - a janela para colocar a caixa de diálogo no topo

title - o título da janela

initialdir - o diretório no qual a caixa de diálogo começa

initialfile - o arquivo selecionado ao abrir a caixa de diálogo

filetypes - uma sequência de tuplas (rótulo, padrão), o caractere curinga '*' é permitido

defaultextension - extensão padrão para anexar ao arquivo (caixas de diálogo para salvar)

multiple - quando verdadeiro, a seleção de vários itens é permitida

Fábrica de funções estáticas

As funções a seguir, quando chamadas, criam uma caixa de diálogo modal e nativa, aguardam a seleção do usuário e, em seguida, retornam o(s) valor(es) selecionado(s) ou `None` para o chamador.

`tkinter.filedialog.`**`askopenfile`**(*mode='r'*, ***options*)

`tkinter.filedialog.`**`askopenfiles`**(*mode='r'*, ***options*)

As duas funções acima criam uma caixa de diálogo `Open` e retornam a caixa de diálogo com um ou mais objetos arquivo abertos em modo somente leitura.

`tkinter.filedialog.`**`asksaveasfile`**(*mode='w'*, ***options*)

Cria uma caixa de diálogo `SaveAs` e retorna um objeto arquivo aberto em modo somente escrita.

`tkinter.filedialog.`**`askopenfilename`**(***options*)

`tkinter.filedialog.`**`askopenfilenames`**(***options*)

As duas funções acima criam uma caixa de diálogo `Open` e retornam um ou mais nomes de arquivos selecionados que correspondem aos arquivos existentes.

`tkinter.filedialog.`**`asksaveasfilename`**(***options*)

Cria uma caixa de diálogo `SaveAs` e retorna o nome do arquivo selecionado.

`tkinter.filedialog.`**`askdirectory`**(***options*)

Solicita ao usuário que selecione um diretório.

Opção de palavra reservada adicional:

mustexist - determina se a seleção deve ser um diretório existente.

`class` `tkinter.filedialog.`**`Open`**(*master=None*, ***options*)

`class` `tkinter.filedialog.`**`SaveAs`**(*master=None*, ***options*)

As duas classes acima fornecem janelas de diálogo nativas para salvar e carregar files.

Classes de conveniência

As classes abaixo são usadas para criar janelas de arquivos/diretórios desde o início. Elas não emulam a aparência nativa da plataforma.

class tkinter.filedialog.**Directory**(*master=None*, ***options*)

Cria uma caixa de diálogo solicitando que o usuário selecione um diretório.

Nota

A classe *FileDialog* deve ser uma subclasse para manipulação e comportamento de eventos personalizados.

class tkinter.filedialog.**FileDialog**(*master*, *title=None*)

Cria uma caixa de diálogo básica de seleção de arquivo.

cancel_command(*event=None*)

Aciona o encerramento da janela de diálogo.

dirs_double_event(*event*)

Manipulador de eventos para evento de clique duplo no diretório.

dirs_select_event(*event*)

Manipulador de eventos para evento de clique no diretório.

files_double_event(*event*)

Manipulador de eventos para evento de clique duplo no arquivo.

files_select_event(*event*)

Manipulador de eventos para evento de clique único no arquivo.

filter_command(*event=None*)

Filtra os arquivos por diretório.

get_filter()

Recupera o filtro de arquivo atualmente em uso.

get_selection()

Recupera o item atualmente selecionado.

go(*dir_or_file=os.curdir*, *pattern='*'*, *default=''*, *key=None*)

Caixa de diálogo de renderização e inicia um laço de eventos.

ok_event(*event*)

Sai da caixa de diálogo retornando a seleção atual.

quit(*how=None*)

Sai da caixa de diálogo retornando o nome do arquivo, se houver.

set_filter(*dir*, *pat*)

Define o filtro de arquivo.

set_selection(*file*)

Atualiza a seleção de arquivo atual para *file*.

class tkinter.filedialog.**LoadFileDialog**(*master*, *title=None*)

Uma subclasse de FileDialog que cria uma janela de diálogo para selecionar um arquivo existente.

ok_command()

Testa se um arquivo é fornecido e se a seleção indica um já arquivo existente.

class tkinter.filedialog.**SaveFileDialog**(*master*, *title=None*)

Uma subclasse de FileDialog que cria uma janela de diálogo para selecionar um arquivo de destino.

ok_command()

Testa se a seleção aponta ou não para um arquivo válido que não é um diretório. A confirmação é necessária se um arquivo já existente for selecionado.

25.4.3 `tkinter.commondialog` — Modelos de janela de diálogo

Código-fonte: Lib/tkinter/commondialog.py

O módulo *tkinter.commondialog* fornece a a classe *Dialog*, que é a classe base para diálogos definidos em outros módulos de suporte.

class tkinter.commondialog.**Dialog**(*master=None*, ***options*)

show(***options*)

Renderiza a janela de diálogo.

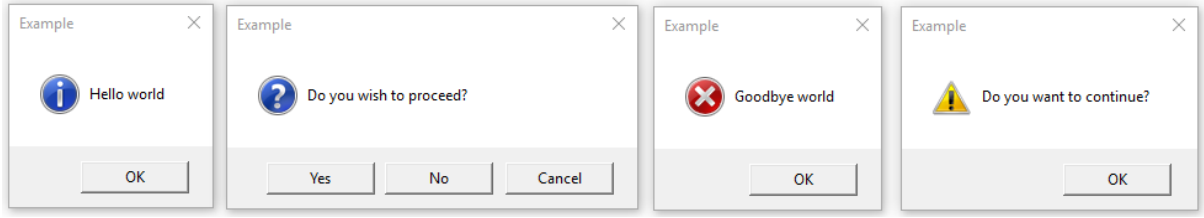
Ver também

Módulos *tkinter.messagebox*, tut-files

25.5 `tkinter.messagebox` — Prompts de mensagem do Tkinter

Código-fonte: Lib/tkinter/messagebox.py

O módulo *tkinter.messagebox* provê uma classe base template e uma variedade de métodos de conveniência para as configurações mais comumente usadas. As caixas de mensagem são modais e vão retornar um conjunto de (`True`, `False`, `None`, *OK*, *CANCEL*, *YES*, *NO*) baseado na seleção do usuário. Estilos e formatos de janelas comuns estão incluídos, mas não estão limitados a:



class tkinter.messagebox.**Message**(*master=None*, ***options*)

Criar uma janela de mensagem com uma mensagem específica da aplicação, um ícone e um conjunto de botões. Cada botão na janela é identificado com um nome simbólico único (veja as opções de "type").

As seguintes opções são suportadas:

"command"
Especifica a função que será invocada quando o usuário fecha a janela de diálogo. O nome do botão clicado pelo usuário para fechar a janela é passada como um argumento. Esta funcionalidade está disponível apenas no macOS.

"default"
Fornece o *nome simbólico* do botão padrão para essa janela de mensagem (*OK*, *CANCEL*, e assim por diante). Se essa opção não for especificada, o primeiro botão da caixa de diálogo será o padrão.

"detail"
Especifica uma mensagem auxiliar para a mensagem principal fornecida pela opção *message*. Esta mensagem será apresentado abaixo da mensagem principal e, quando suportado pelo sistema operacional, em uma fonte menos enfatizada do que a mensagem principal.

"icon"
Especifica um *icon* a ser apresentado. Se essa opção não for especificada, o ícone *INFO* será exibido.

"message"
Especifica a mensagem para mostrar nessa caixa de mensagem. O padrão valor é uma string vazia.

"parent"
Torna a janela especificada a janela pai da caixa de mensagem. A caixa de mensagem é exibida na parte superior de sua janela pai.

"title"
Especifica uma string para mostrar como o título da caixa de mensagem. Essa opção é ignorada no macOS, onde a plataforma proíbe o uso de um título nesse tipo de caixa de diálogo.

"type"
Organiza um *conjunto predefinido de botões* a serem mostrados.

show(***options*)

Mostra uma janela de mensagem e aguarda que o usuário selecione um dos botões. Em seguida, retorna o nome simbólico do botão selecionado. Argumentos nomeados pode substituir opções especificadas no construtor.

Caixa de mensagem de informação

tkinter.messagebox.**showinfo**(*title=None*, *message=None*, ***options*)

Cria e exibe uma caixa de mensagem informativa com o título e a mensagem especificados.

Caixas de mensagem de atenção

tkinter.messagebox.**showwarning**(*title=None*, *message=None*, ***options*)

Cria e exibe uma caixa de mensagem de alerta com o título e a mensagem especificados.

tkinter.messagebox.**showerror**(*title=None*, *message=None*, ***options*)

Cria e exibe uma caixa de mensagem de erro com o título e a mensagem especificados.

Caixas de mensagem de dúvida

tkinter.messagebox.**askquestion**(*title=None*, *message=None*, ***, *type=YESNO*, ***options*)

Faz uma pergunta. Por padrão mostra os botões *YES* e *NO*. Retorna o nome simbólico do botão selecionado.

tkinter.messagebox.**askokcancel**(*title=None*, *message=None*, ***options*)

Pergunta se a operação deve prosseguir. Mostra os botões *OK* e *CANCEL*. Retorna `True` se a resposta for "OK" e `False` caso contrário.

tkinter.messagebox.**askretrycancel**(*title=None*, *message=None*, ***options*)

Pergunta se a operação deve ser tentada novamente. Mostra os botões *RETRY* e *CANCEL*. Retorna `True` se a resposta for sim e `False` caso contrário.

tkinter.messagebox.**askyesno**(*title=None*, *message=None*, ***options*)

Faz uma pergunta. Mostra os botões *YES* e *NO*. Retorna `True` se a resposta for sim e `False` caso contrário.

tkinter.messagebox.**askyesnocancel**(*title=None*, *message=None*, ***options*)

Faz uma pergunta. Mostra os botões *YES*, *NO* e *CANCEL*. Retorna `True` se a resposta for sim, `None` se cancelado, e `False` caso contrário.

Nomes simbólicos dos botões:

tkinter.messagebox.**ABORT = 'abort'**

tkinter.messagebox.**RETRY = 'retry'**

tkinter.messagebox.**IGNORE = 'ignore'**

tkinter.messagebox.**OK = 'ok'**

tkinter.messagebox.**CANCEL = 'cancel'**

tkinter.messagebox.**YES = 'yes'**

tkinter.messagebox.**NO = 'no'**

Conjunto predefinido de botões:

tkinter.messagebox.**ABORTRETRYIGNORE = 'abortretryignore'**

Exibe três botões cujos nomes simbólicos são *ABORT*, *RETRY* e *IGNORE*.

tkinter.messagebox.**OK = 'ok'**

Exibe um botão cujo nome simbólico é *OK*.

tkinter.messagebox.**OKCANCEL = 'okcancel'**

Exibe dois botões cujos nomes simbólicos são *OK* e *CANCEL*.

tkinter.messagebox.**RETRYCANCEL = 'retrycancel'**

Exibe dois botões cujos nomes simbólicos são *RETRY* e *CANCEL*.

tkinter.messagebox.**YESNO = 'yesno'**

Exibe dois botões cujos nomes simbólicos são *YES* e *NO*.

tkinter.messagebox.**YESNOCANCEL = 'yesnocancel'**

Exibe três botões cujos nomes simbólicos são *YES*, *NO* e *CANCEL*.

Imagens de ícones:

tkinter.messagebox.**ERROR = 'error'**

tkinter.messagebox.**INFO = 'info'**

tkinter.messagebox.**QUESTION = 'question'**

tkinter.messagebox.**WARNING = 'warning'**

25.6 `tkinter.scrolledtext` — Widget de texto de rolado

Código-fonte: Lib/tkinter/scrolledtext.py

O módulo *tkinter.scrolledtext* fornece uma classe com o mesmo nome que implementa um widget de texto básico que possui uma barra de rolagem vertical configurada para fazer a "coisa certa". Usar a classe *ScrolledText* é muito mais fácil do que configurar um widget de texto e barra de rolagem diretamente.

O widget de texto e a barra de rolagem são agrupados em `Frame`, e os métodos dos gerenciadores de geometria `Grid` e `Pack` são adquiridos do objeto `Frame`. Isso permite que o widget *ScrolledText* seja usado diretamente para obter o comportamento mais normal de gerenciamento de geometria.

Se um controle mais específico for necessário, os seguintes atributos estarão disponíveis:

class tkinter.scrolledtext.**ScrolledText**(*master=None*, ***kw*)

frame

O quadro que envolve os widgets de barra de rolagem e texto.

vbar

O widget da barra de rolagem.

25.7 `tkinter.dnd` — Suporte para arrastar e soltar

Código-fonte: Lib/tkinter/dnd.py

Nota

Isto é experimental e deverá ser descontinuado quando for substituído pelo Tk DND.

O módulo `tkinter.dnd` fornece suporte de arrastar e soltar para objetos dentro de uma única aplicação, dentro da mesma janela ou entre janelas. Para permitir que um objeto seja arrastado, você deve criar uma associação de evento para ele que inicie o processo de arrastar e soltar. Normalmente, você associa um evento ButtonPress a uma função de retorno que você escreve (consulte *Ligações e Eventos*). A função deve chamar `dnd_start()`, onde 'source' é o objeto a ser arrastado e 'event' é o evento que invocou a chamada (o argumento para sua função de retorno).

A seleção de um objeto de destino ocorre da seguinte forma:

1. Pesquisa de cima para baixo da área sob o mouse para o widget alvo
   - O widget alvo deve ter um atributo chamável *dnd_accept*
   - Se *dnd_accept* não estiver presente ou retornar `None`, a pesquisa vai para o widget pai
   - Se nenhum widget de destino for encontrado, o objeto alvo é `None`
2. Chamada para *<old_target>.dnd_leave(source, event)*
3. Chamada para *<new_target>.dnd_enter(source, event)*
4. Chamada para *<target>.dnd_commit(source, event)* para notificar que soltou
5. Chamada para *<source>.dnd_end(target, event)* para sinalizar o fim do arrastar e soltar

class `tkinter.dnd.`**DndHandler**(*source*, *event*)

A classe *DndHandler* trata de eventos de arrastar e soltar que rastreiam eventos Motion e ButtonRelease na raiz do widget de evento.

cancel(*event=None*)

Cancela o processo de arrastar e soltar.

finish(*event*, *commit=0*)

Executa o fim das funções de arrastar e soltar.

on_motion(*event*)

Inspeciona a área abaixo do mouse para objetos alvos enquanto o arrasto é executado.

on_release(*event*)

Sinaliza o fim do arrasto quando o padrão de liberação for acionado.

`tkinter.dnd.`**dnd_start**(*source*, *event*)

Função de fábrica para processo de arrastar e soltar.

Ver também

Ligações e Eventos

25.8 `tkinter.ttk` — Widgets temáticos do Tk

Código-fonte: Lib/tkinter/ttk.py

O módulo *tkinter.ttk* introduzido no Tk 8.5, fornece acesso ao conjunto de widgets temáticos do Tk, Alguns dos benefícios incluídos são a renderização de fontes com suavização de serrilhamento no x11 e transparência de janelas (requer um gerenciador de composição de janelas no x11).

A ideia básica para *tkinter.ttk* é separar, na medida do possível, o código que implementa o comportamento de widget do código que implementa sua aparência.

Ver também

Tk Widget Styling Support
Um documento que apresenta o suporte de temas para Tk

25.8.1 Usando Ttk

Para começar a usar Ttk, importe seu módulo:

```
from tkinter import ttk
```

Para substituir os widgets básicos do Tk, a importação deve seguir a importação do Tk:

```
from tkinter import *
from tkinter.ttk import *
```

Esse código faz com que vários widgets *tkinter.ttk* (`Button`, `Checkbutton`, `Entry`, `Frame`, `Label`, `LabelFrame`, `Menubutton`, `PanedWindow`, `Radiobutton`, `Scale` e `Scrollbar`) substituam automaticamente os widgets Tk.

Isso traz o benefício direto de utilizar os widgets novos que possuem melhor aparência e sensação de uso entre as plataformas, contudo, os widgets substitutos não são completamente compatíveis. A maior diferença é que as opções como "fg", "bg" e outras relacionadas à estilização do widget não estão mais presentes nos widgets Ttk. Em vez disso, use a classe `ttk.Style` para melhores efeitos de estilização.

Ver também

Convertendo aplicações existentes para utilização dos widgets Tile
Uma monografia (utilizando a terminologia Tcl) sobre diferenças tipicamente encontradas ao mover aplicações para utilização de novos widgets.

25.8.2 Widgets Ttk

Ttk traz 18 widgets, das quais doze já existiam no tkinter: `Button`, `Checkbutton`, `Entry`, `Frame`, `Label`, `LabelFrame`, `Menubutton`, `PanedWindow`, `Radiobutton`, `Scale`, `Scrollbar`, e *Spinbox*. As outras seis são novas: *Combobox*, *Notebook*, *Progressbar*, `Separator`, `Sizegrip` e *Treeview*. E todas elas são subclasses de *Widget*.

Utilizar os widgets Ttk melhora a experiência de uso e o visual da aplicação. Conforme discutido anteriormente, existem diferenças em como a estilização é codificada.

Código Tk:

```
l1 = tkinter.Label(text="Test", fg="black", bg="white")
l2 = tkinter.Label(text="Test", fg="black", bg="white")
```

Código Ttk:

```
style = ttk.Style()
style.configure("BW.TLabel", foreground="black", background="white")
```

(continua na próxima página)

(continuação da página anterior)

```
l1 = ttk.Label(text="Test", style="BW.TLabel")
l2 = ttk.Label(text="Test", style="BW.TLabel")
```

Para mais informações sobre *TtkStyling*, veja a documentação da classe `Style`.

25.8.3 Widget

`ttk.Widget` define opções padrão e métodos suportados pelos widgets temáticos Tk e não deve ser instanciada diretamente.

Opções padrão

Todos os Widgets `ttk` aceitam as seguintes opções:

Opção	Descrição
class	Specifies the window class. The class is used when querying the option database for the window's other options, to determine the default bindtags for the window, and to select the widget's default layout and style. This option is read-only, and may only be specified when the window is created.
cursor	Especifica o cursor do mouse a ser utilizado pelo widget. Caso seja definido como uma string vazia (padrão), o cursor é herdado do widget pai.
takefocus	Determina se a janela aceitará foco durante a navegação com o teclado. Retorna 0, 1 ou uma string vazia. Caso retorne 0, significa que a janela deve ser completamente ignorada durante a navegação com o teclado. Caso retorne 1, significa que a janela receberá as entradas do teclado enquanto estiver visível. E a string vazia significa que os scripts de navegação determinam quando a janela recebe foco ou não.
style	Pode ser utilizado para especificar um estilo customizado do widget.

Opções de ferramenta rolável

As opções a seguir são suportadas pelos widgets que são controlados por uma barra de rolagem.

Opção	Descrição
xscrollcommand	Utilizado para comunicação com barras de rolagem horizontais. Quando a visualização na janela do widget mudar, o widget irá gerar um comando Tcl baseado no comando da barra de rolagem. Geralmente essa opção consiste no método `Scrollbar.set()` de alguma barra de rolagem. Isso fará com que a barra de rolagem seja atualizada sempre que a visualização da janela mude.
yscrollcommand	Utilizado para comunicação com barras de rolagem verticais. Para mais informações, veja acima.

Opções de rótulo

As opções a seguir são suportadas por rótulos, botões e outros widgets semelhantes.

Opção	Descrição
text	Especifica uma string para exibição dentro do widget.
textvariable	Specifies a name whose value will be used in place of the text option resource.
underline	Caso definido, especifica o índice (base-0) de um caractere para sublinhamento na string de texto. O caractere sublinhado é utilizado para ativação mnemônica.
image	Define uma imagem para exibição. Isso é uma lista de 1 ou mais elementos. O primeiro elemento é o nome da imagem padrão. O resto da lista é uma sequência de pares estado/valor conforme definido por `Style.map()`, definindo imagens diferentes para utilização quando o widget estiver em um determinado estado ou em uma combinação de estados. Todas as imagens da lista devem ter o mesmo tamanho.
compound	Define como exibir a imagem em relação ao texto, no caso em que ambas as opções de texto e imagem estejam presentes. Valores válidos são: • texto: exibir apenas texto • imagem: exibir apenas a imagem • top, bottom, left, right: exibem a imagem acima, abaixo, à esquerda, ou à direita do texto, respectivamente. • none: o padrão, exibe a imagem caso ela esteja presente, caso contrário , exibe o texto.
width	Caso maior que zero, especifica quanto espaço, em largura de caracteres, alocar para o rótulo do texto, se menor que zero, especifica a largura mínima. Se zero ou inespecífico, é utilizada a largura normal do rótulo de texto.

Opções de compatibilidade

Opção	Descrição
state	May be set to “normal” or “disabled” to control the “disabled” state bit. This is a write-only option: setting it changes the widget state, but the `Widget.state()` method does not affect this option.

Estados de Widget

O estado do widget é um bitmap de sinalizadores de estado independentes.

Sinalizador	Descrição
active	O cursor do mouse está sobre o widget e pressionar um botão do mouse causará alguma ação
disabled	Widget está desabilitado sob controle do programa.
focus	Widget tem foco do teclado
pressed	Widget está sendo pressionado
selected	“On”, “true”, ou “current” para coisas como botões de confirmação e botões de opção.
background	Windows e Mac possuem uma noção de janela “ativa” ou em primeiro plano. O estado de “plano de fundo” é definido para widgets em uma janela no plano de fundo, e limpo para aqueles na janela de primeiro plano
readonly	Widget não deve permitir modificação do usuário
alternate	Um formato de exibição específico do widget
invalid	O valor do widget é inválido

Uma especificação de estado é uma sequência de nomes de estado, opcionalmente prefixada por um ponto de exclamação indicando que o bit está desligado.

ttk.Widget

Além dos métodos descritos abaixo, a classe `ttk.Widget` oferece suporte aos métodos `tkinter.Widget.cget()` e `tkinter.Widget.configure()`.

class tkinter.ttk.**Widget**

identify(*x*, *y*)

Retorna o nome do elemento na posição *x y*, ou a string vazia caso o ponto não esteja dentro de algum elemento.

x e *y* são coordenadas de pixel relativas ao widget.

instate(*statespec*, *callback=None*, **args*, ***kw*)

Test the widget's state. If a callback is not specified, returns `True` if the widget state matches *statespec* and `False` otherwise. If callback is specified then it is called with args if widget state matches *statespec*.

state(*statespec=None*)

Modify or inquire widget state. If *statespec* is specified, sets the widget state according to it and return a new *statespec* indicating which flags were changed. If *statespec* is not specified, returns the currently enabled state flags.

statespec will usually be a list or a tuple.

25.8.4 Combobox

O widget `ttk.Combobox` combina um campo de texto com uma lista suspensa de valores. Esse widget é uma subclasse de `Entry`.

Besides the methods inherited from *Widget*: `Widget.cget()`, `Widget.configure()`, *Widget.identify()*, *Widget.instate()* and *Widget.state()*, and the following inherited from `Entry`: `Entry.bbox()`, `Entry.delete()`, `Entry.icursor()`, `Entry.index()`, `Entry.insert()`, `Entry.selection()`, `Entry.xview()`, it has some other methods, described at `ttk.Combobox`.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
exportselection	Valor booleano. Caso definido, a seleção do widget é vinculada à seleção do Gerenciador de Janelas (que pode ser retornada ao chamar Misc.selection_get, por exemplo).
justify	Especifica como o texto é alinhado dentro do widget. Pode ser "left", "center" ou "right".
height	Especifica a altura, em linhas, da lista suspensa.
postcommand	Um script (possivelmente registrado com Misc.register) que é chamado imediatamente antes da exibição dos valores. Ele pode especificar quais valores exibir.
state	Um dos estados: "normal", "readonly" ou "disabled". No estado "readonly", o valor não pode ser editado diretamente, e o usuário só pode selecionar valores contidos na lista suspensa. No estado "normal", o campo de texto é diretamente editável. No estado "disabled", nenhuma interação é possível.
textvariable	Especifica um nome que terá seu valor vinculado ao valor do widget. Sempre que o valor associado a esse nome mudar, o valor do widget será atualizado, e vice versa. Veja `tkinter.StringVar`.
values	Especifica a lista de valores para exibição na lista suspensa.
width	Specifies an integer value indicating the desired width of the entry window, in average-size characters of the widget's font.

Eventos virtuais

Os widgets combobox geram um evento virtual **«ComboboxSelected»** quando o usuário seleciona um elemento da lista de valores.

ttk.Combobox

class tkinter.ttk.**Combobox**

current(*newindex=None*)

If *newindex* is specified, sets the combobox value to the element position *newindex*. Otherwise, returns the index of the current value or -1 if the current value is not in the values list.

get()

Retorna o valor atual do combobox.

set(*value*)

Sets the value of the combobox to *value*.

25.8.5 Spinbox

The `ttk.Spinbox` widget is a `ttk.Entry` enhanced with increment and decrement arrows. It can be used for numbers or lists of string values. This widget is a subclass of `Entry`.

Besides the methods inherited from *`Widget`*: `Widget.cget()`, `Widget.configure()`, *`Widget.identify()`*, *`Widget.instate()`* and *`Widget.state()`*, and the following inherited from `Entry`: `Entry.bbox()`, `Entry.delete()`, `Entry.icursor()`, `Entry.index()`, `Entry.insert()`, `Entry.xview()`, it has some other methods, described at `ttk.Spinbox`.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
from	Float value. If set, this is the minimum value to which the decrement button will decrement. Must be spelled as `from_` when used as an argument, since `from` is a Python keyword.
to	Float value. If set, this is the maximum value to which the increment button will increment.
increment	Float value. Specifies the amount which the increment/decrement buttons change the value. Defaults to 1.0.
values	Sequence of string or float values. If specified, the increment/decrement buttons will cycle through the items in this sequence rather than incrementing or decrementing numbers.
wrap	Boolean value. If `True`, increment and decrement buttons will cycle from the `to` value to the `from` value or the `from` value to the `to` value, respectively.
format	String value. This specifies the format of numbers set by the increment/decrement buttons. It must be in the form "%W.Pf", where W is the padded width of the value, P is the precision, and '%' and 'f' are literal.
command	Python callable. Will be called with no arguments whenever either of the increment or decrement buttons are pressed.

Eventos virtuais

The spinbox widget generates an **«Increment»** virtual event when the user presses <Up>, and a **«Decrement»** virtual event when the user presses <Down>.

ttk.Spinbox

class tkinter.ttk.**Spinbox**

get()

Returns the current value of the spinbox.

set(*value*)

Sets the value of the spinbox to *value*.

25.8.6 Notebook

Ttk Notebook widget manages a collection of windows and displays a single one at a time. Each child window is associated with a tab, which the user may select to change the currently displayed window.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
height	If present and greater than zero, specifies the desired height of the pane area (not including internal padding or tabs). Otherwise, the maximum height of all panes is used.
padding	Specifies the amount of extra space to add around the outside of the notebook. The padding is a list up to four length specifications left top right bottom. If fewer than four elements are specified, bottom defaults to top, right defaults to left, and top defaults to left.
width	If present and greater than zero, specified the desired width of the pane area (not including internal padding). Otherwise, the maximum width of all panes is used.

Tab Options

There are also specific options for tabs:

Opção	Descrição
state	Either “normal”, “disabled” or “hidden”. If “disabled”, then the tab is not selectable. If “hidden”, then the tab is not shown.
sticky	Specifies how the child window is positioned within the pane area. Value is a string containing zero or more of the characters “n”, “s”, “e” or “w”. Each letter refers to a side (north, south, east or west) that the child window will stick to, as per the `grid()` geometry manager.
padding	Specifies the amount of extra space to add between the notebook and this pane. Syntax is the same as for the option padding used by this widget.
text	Specifies a text to be displayed in the tab.
image	Specifies an image to display in the tab. See the option image described in *`Widget`*.
compound	Specifies how to display the image relative to the text, in the case both options text and image are present. See *Label Options* for legal values.
underline	Specifies the index (0-based) of a character to underline in the text string. The underlined character is used for mnemonic activation if *`Notebook.enable_traversal()`* is called.

Tab Identifiers

The tab_id present in several methods of `ttk.Notebook` may take any of the following forms:

- An integer between zero and the number of tabs
- The name of a child window
- A positional specification of the form “@x,y”, which identifies the tab
- The literal string “current”, which identifies the currently selected tab
- The literal string “end”, which returns the number of tabs (only valid for *`Notebook.index()`*)

Virtual Events

This widget generates a **«NotebookTabChanged»** virtual event after a new tab is selected.

ttk.Notebook

class tkinter.ttk.**Notebook**

add(*child*, ***kw*)

Adds a new tab to the notebook.

If window is currently managed by the notebook but hidden, it is restored to its previous position.

See *Tab Options* for the list of available options.

forget(*tab_id*)

Removes the tab specified by *tab_id*, unmaps and unmanages the associated window.

hide(*tab_id*)

Hides the tab specified by *tab_id*.

The tab will not be displayed, but the associated window remains managed by the notebook and its configuration remembered. Hidden tabs may be restored with the `add()` command.

identify(*x*, *y*)

Returns the name of the tab element at position *x*, *y*, or the empty string if none.

index(*tab_id*)

Returns the numeric index of the tab specified by *tab_id*, or the total number of tabs if *tab_id* is the string "end".

insert(*pos*, *child*, ***kw*)

Inserts a pane at the specified position.

pos is either the string "end", an integer index, or the name of a managed child. If *child* is already managed by the notebook, moves it to the specified position.

See *Tab Options* for the list of available options.

select(*tab_id=None*)

Selects the specified *tab_id*.

The associated child window will be displayed, and the previously selected window (if different) is unmapped. If *tab_id* is omitted, returns the widget name of the currently selected pane.

tab(*tab_id*, *option=None*, ***kw*)

Query or modify the options of the specific *tab_id*.

If *kw* is not given, returns a dictionary of the tab option values. If *option* is specified, returns the value of that *option*. Otherwise, sets the options to the corresponding values.

tabs()

Returns a list of windows managed by the notebook.

enable_traversal()

Enable keyboard traversal for a toplevel window containing this notebook.

This will extend the bindings for the toplevel window containing the notebook as follows:

- `Control-Tab`: selects the tab following the currently selected one.
- `Shift-Control-Tab`: selects the tab preceding the currently selected one.
- `Alt-K`: where *K* is the mnemonic (underlined) character of any tab, will select that tab.

Multiple notebooks in a single toplevel may be enabled for traversal, including nested notebooks. However, notebook traversal only works properly if all panes have the notebook they are in as master.

25.8.7 Progressbar

The `ttk.Progressbar` widget shows the status of a long-running operation. It can operate in two modes: 1) the determinate mode which shows the amount completed relative to the total amount of work to be done and 2) the indeterminate mode which provides an animated display to let the user know that work is progressing.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
orient	One of “horizontal” or “vertical”. Specifies the orientation of the progress bar.
length	Specifies the length of the long axis of the progress bar (width if horizontal, height if vertical).
mode	One of “determinate” or “indeterminate”.
maximum	A number specifying the maximum value. Defaults to 100.
value	The current value of the progress bar. In “determinate” mode, this represents the amount of work completed. In “indeterminate” mode, it is interpreted as modulo *maximum*; that is, the progress bar completes one “cycle” when its value increases by *maximum*.
variable	A name which is linked to the option value. If specified, the value of the progress bar is automatically set to the value of this name whenever the latter is modified.
phase	Read-only option. The widget periodically increments the value of this option whenever its value is greater than 0 and, in determinate mode, less than maximum. This option may be used by the current theme to provide additional animation effects.

ttk.Progressbar

class tkinter.ttk.**Progressbar**

start(*interval=None*)

Begin autoincrement mode: schedules a recurring timer event that calls *Progressbar.step()* every *interval* milliseconds. If omitted, *interval* defaults to 50 milliseconds.

step(*amount=None*)

Increments the progress bar’s value by *amount*.

amount defaults to 1.0 if omitted.

stop()

Stop autoincrement mode: cancels any recurring timer event initiated by *Progressbar.start()* for this progress bar.

25.8.8 Separator

The `ttk.Separator` widget displays a horizontal or vertical separator bar.

It has no other methods besides the ones inherited from `ttk.Widget`.

Opções

This widget accepts the following specific option:

Opção	Descrição
orient	One of “horizontal” or “vertical”. Specifies the orientation of the separator.

25.8.9 Sizegrip

The `ttk.Sizegrip` widget (also known as a grow box) allows the user to resize the containing toplevel window by pressing and dragging the grip.

This widget has neither specific options nor specific methods, besides the ones inherited from `ttk.Widget`.

Platform-specific notes

- On macOS, toplevel windows automatically include a built-in size grip by default. Adding a `Sizegrip` is harmless, since the built-in grip will just mask the widget.

Bugs

- If the containing toplevel's position was specified relative to the right or bottom of the screen (e.g. `....`), the `Sizegrip` widget will not resize the window.
- This widget supports only "southeast" resizing.

25.8.10 Treeview

The `ttk.Treeview` widget displays a hierarchical collection of items. Each item has a textual label, an optional image, and an optional list of data values. The data values are displayed in successive columns after the tree label.

The order in which data values are displayed may be controlled by setting the widget option `displaycolumns`. The tree widget can also display column headings. Columns may be accessed by number or symbolic names listed in the widget option columns. See *Column Identifiers*.

Each item is identified by a unique name. The widget will generate item IDs if they are not supplied by the caller. There is a distinguished root item, named `{}`. The root item itself is not displayed; its children appear at the top level of the hierarchy.

Each item also has a list of tags, which can be used to associate event bindings with individual items and control the appearance of the item.

The Treeview widget supports horizontal and vertical scrolling, according to the options described in *Scrollable Widget Options* and the methods *`Treeview.xview()`* and *`Treeview.yview()`*.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
columns	A list of column identifiers, specifying the number of columns and their names.
displaycolumns	A list of column identifiers (either symbolic or integer indices) specifying which data columns are displayed and the order in which they appear, or the string "#all".
height	Specifies the number of rows which should be visible. Note: the requested width is determined from the sum of the column widths.
padding	Specifies the internal padding for the widget. The padding is a list of up to four length specifications.
selectmode	Controls how the built-in class bindings manage the selection. One of "extended", "browse" or "none". If set to "extended" (the default), multiple items may be selected. If "browse", only a single item will be selected at a time. If "none", the selection will not be changed. Note that the application code and tag bindings can set the selection however they wish, regardless of the value of this option.
show	A list containing zero or more of the following values, specifying which elements of the tree to display. • tree: display tree labels in column #0. • headings: display the heading row. The default is "tree headings", i.e., show all elements. **Note**: Column #0 always refers to the tree column, even if show="tree" is not specified.

Item Options

The following item options may be specified for items in the insert and item widget commands.

Opção	Descrição
text	The textual label to display for the item.
image	A Tk Image, displayed to the left of the label.
values	The list of values associated with the item. Each item should have the same number of values as the widget option columns. If there are fewer values than columns, the remaining values are assumed empty. If there are more values than columns, the extra values are ignored.
open	`True`/`False` value indicating whether the item's children should be displayed or hidden.
tags	A list of tags associated with this item.

Tag Options

The following options may be specified on tags:

Opção	Descrição
foreground	Specifies the text foreground color.
background	Specifies the cell or item background color.
font	Specifies the font to use when drawing text.
image	Specifies the item image, in case the item's image option is empty.

Column Identifiers

Column identifiers take any of the following forms:

- A symbolic name from the list of columns option.
- An integer n, specifying the nth data column.
- A string of the form #n, where n is an integer, specifying the nth display column.

Notas:

- Item's option values may be displayed in a different order than the order in which they are stored.
- Column #0 always refers to the tree column, even if show="tree" is not specified.

A data column number is an index into an item's option values list; a display column number is the column number in the tree where the values are displayed. Tree labels are displayed in column #0. If option displaycolumns is not set, then data column n is displayed in column #n+1. Again, **column #0 always refers to the tree column**.

Virtual Events

The Treeview widget generates the following virtual events.

Evento	Descrição
«TreeviewSelect»	Generated whenever the selection changes.
«TreeviewOpen»	Generated just before settings the focus item to open=True.
«TreeviewClose»	Generated just after setting the focus item to open=False.

The `Treeview.focus()` and `Treeview.selection()` methods can be used to determine the affected item or items.

ttk.Treeview

class tkinter.ttk.**Treeview**

bbox(*item*, *column=None*)

Returns the bounding box (relative to the treeview widget's window) of the specified *item* in the form (x, y, width, height).

If *column* is specified, returns the bounding box of that cell. If the *item* is not visible (i.e., if it is a descendant of a closed item or is scrolled offscreen), returns an empty string.

get_children(*item=None*)

Returns the list of children belonging to *item*.

If *item* is not specified, returns root children.

set_children(*item*, **newchildren*)

Replaces *item*'s child with *newchildren*.

Children present in *item* that are not present in *newchildren* are detached from the tree. No items in *newchildren* may be an ancestor of *item*. Note that not specifying *newchildren* results in detaching *item*'s children.

column(*column*, *option=None*, ***kw*)

Query or modify the options for the specified *column*.

If *kw* is not given, returns a dict of the column option values. If *option* is specified then the value for that *option* is returned. Otherwise, sets the options to the corresponding values.

The valid options/values are:

id
Returns the column name. This is a read-only option.

***anchor*: One of the standard Tk anchor values.**
Specifies how the text in this column should be aligned with respect to the cell.

***minwidth*: width**
The minimum width of the column in pixels. The treeview widget will not make the column any smaller than specified by this option when the widget is resized or the user drags a column.

***stretch*: `True/False`**
Specifies whether the column's width should be adjusted when the widget is resized.

***width*: width**
The width of the column in pixels.

To configure the tree column, call this with column = "#0"

`delete`(**items*)

Delete all specified *items* and all their descendants.

The root item may not be deleted.

`detach`(**items*)

Unlinks all of the specified *items* from the tree.

The items and all of their descendants are still present, and may be reinserted at another point in the tree, but will not be displayed.

The root item may not be detached.

`exists`(*item*)

Returns `True` if the specified *item* is present in the tree.

`focus`(*item=None*)

If *item* is specified, sets the focus item to *item*. Otherwise, returns the current focus item, or '' if there is none.

`heading`(*column*, *option=None*, ***kw*)

Query or modify the heading options for the specified *column*.

If *kw* is not given, returns a dict of the heading option values. If *option* is specified then the value for that *option* is returned. Otherwise, sets the options to the corresponding values.

The valid options/values are:

***text*: text**
The text to display in the column heading.

***image*: imageName**
Specifies an image to display to the right of the column heading.

***anchor*: anchor**
Specifies how the heading text should be aligned. One of the standard Tk anchor values.

***command*: callback**
A callback to be invoked when the heading label is pressed.

To configure the tree column heading, call this with column = "#0".

`identify`(*component*, *x*, *y*)

Returns a description of the specified *component* under the point given by *x* and *y*, or the empty string if no such *component* is present at that position.

`identify_row`(*y*)

Returns the item ID of the item at position *y*.

`identify_column`(*x*)

Returns the data column identifier of the cell at position *x*.

The tree column has ID #0.

`identify_region`(*x*, *y*)

Returns one of:

region	meaning
heading	Tree heading area.
separator	Space between two columns headings.
tree	The tree area.
cell	A data cell.

Availability: Tk 8.6.

identify_element(*x*, *y*)

Returns the element at position *x*, *y*.

Availability: Tk 8.6.

index(*item*)

Returns the integer index of *item* within its parent's list of children.

insert(*parent*, *index*, *iid=None*, ***kw*)

Creates a new item and returns the item identifier of the newly created item.

parent is the item ID of the parent item, or the empty string to create a new top-level item. *index* is an integer, or the value "end", specifying where in the list of parent's children to insert the new item. If *index* is less than or equal to zero, the new node is inserted at the beginning; if *index* is greater than or equal to the current number of children, it is inserted at the end. If *iid* is specified, it is used as the item identifier; *iid* must not already exist in the tree. Otherwise, a new unique identifier is generated.

See *Item Options* for the list of available options.

item(*item*, *option=None*, ***kw*)

Query or modify the options for the specified *item*.

If no options are given, a dict with options/values for the item is returned. If *option* is specified then the value for that option is returned. Otherwise, sets the options to the corresponding values as given by *kw*.

move(*item*, *parent*, *index*)

Moves *item* to position *index* in *parent*'s list of children.

It is illegal to move an item under one of its descendants. If *index* is less than or equal to zero, *item* is moved to the beginning; if greater than or equal to the number of children, it is moved to the end. If *item* was detached it is reattached.

next(*item*)

Returns the identifier of *item*'s next sibling, or '' if *item* is the last child of its parent.

parent(*item*)

Returns the ID of the parent of *item*, or '' if *item* is at the top level of the hierarchy.

prev(*item*)

Returns the identifier of *item*'s previous sibling, or '' if *item* is the first child of its parent.

reattach(*item*, *parent*, *index*)

An alias for `Treeview.move()`.

see(*item*)

Ensure that *item* is visible.

Sets all of *item*'s ancestors open option to `True`, and scrolls the widget if necessary so that *item* is within the visible portion of the tree.

selection()

Returns a tuple of selected items.

Alterado na versão 3.8: `selection()` no longer takes arguments. For changing the selection state use the following selection methods.

selection_set(**items*)

items becomes the new selection.

Alterado na versão 3.6: *items* can be passed as separate arguments, not just as a single tuple.

selection_add(**items*)

Add *items* to the selection.

Alterado na versão 3.6: *items* can be passed as separate arguments, not just as a single tuple.

selection_remove(**items*)

Remove *items* from the selection.

Alterado na versão 3.6: *items* can be passed as separate arguments, not just as a single tuple.

selection_toggle(**items*)

Toggle the selection state of each item in *items*.

Alterado na versão 3.6: *items* can be passed as separate arguments, not just as a single tuple.

set(*item*, *column=None*, *value=None*)

With one argument, returns a dictionary of column/value pairs for the specified *item*. With two arguments, returns the current value of the specified *column*. With three arguments, sets the value of given *column* in given *item* to the specified *value*.

tag_bind(*tagname*, *sequence=None*, *callback=None*)

Bind a callback for the given event *sequence* to the tag *tagname*. When an event is delivered to an item, the callbacks for each of the item's tags option are called.

tag_configure(*tagname*, *option=None*, ***kw*)

Query or modify the options for the specified *tagname*.

If *kw* is not given, returns a dict of the option settings for *tagname*. If *option* is specified, returns the value for that *option* for the specified *tagname*. Otherwise, sets the options to the corresponding values for the given *tagname*.

tag_has(*tagname*, *item=None*)

If *item* is specified, returns 1 or 0 depending on whether the specified *item* has the given *tagname*. Otherwise, returns a list of all items that have the specified tag.

Availability: Tk 8.6

xview(**args*)

Query or modify horizontal position of the treeview.

yview(**args*)

Query or modify vertical position of the treeview.

25.8.11 Ttk Styling

Each widget in `ttk` is assigned a style, which specifies the set of elements making up the widget and how they are arranged, along with dynamic and default settings for element options. By default the style name is the same as the widget's class name, but it may be overridden by the widget's style option. If you don't know the class name of a widget, use the method `Misc.winfo_class()` (somewidget.winfo_class()).

Ver também

Tcl'2004 conference presentation
This document explains how the theme engine works

class tkinter.ttk.**Style**

This class is used to manipulate the style database.

configure (*style*, *query_opt=None*, ***kw*)

Query or set the default value of the specified option(s) in *style*.

Each key in *kw* is an option and each value is a string identifying the value for that option.

For example, to change every default button to be a flat button with some padding and a different background color:

```
from tkinter import ttk
import tkinter

root = tkinter.Tk()

ttk.Style().configure("TButton", padding=6, relief="flat",
   background="#ccc")

btn = ttk.Button(text="Sample")
btn.pack()

root.mainloop()
```

map (*style*, *query_opt=None*, ***kw*)

Query or sets dynamic values of the specified option(s) in *style*.

Each key in *kw* is an option and each value should be a list or a tuple (usually) containing statespecs grouped in tuples, lists, or some other preference. A statespec is a compound of one or more states and then a value.

An example may make it more understandable:

```
import tkinter
from tkinter import ttk

root = tkinter.Tk()

style = ttk.Style()
style.map("C.TButton",
    foreground=[('pressed', 'red'), ('active', 'blue')],
    background=[('pressed', '!disabled', 'black'), ('active', 'white')]
    )

colored_btn = ttk.Button(text="Test", style="C.TButton").pack()

root.mainloop()
```

Note that the order of the (states, value) sequences for an option does matter, if the order is changed to `[('active', 'blue'), ('pressed', 'red')]` in the foreground option, for example, the result would be a blue foreground when the widget were in active or pressed states.

lookup (*style*, *option*, *state=None*, *default=None*)

Returns the value specified for *option* in *style*.

If *state* is specified, it is expected to be a sequence of one or more states. If the *default* argument is set, it is used as a fallback value in case no specification for option is found.

To check what font a Button uses by default:

```
from tkinter import ttk

print(ttk.Style().lookup("TButton", "font"))
```

layout (*style*, *layoutspec=None*)

Define the widget layout for given *style*. If *layoutspec* is omitted, return the layout specification for given style.

layoutspec, if specified, is expected to be a list or some other sequence type (excluding strings), where each item should be a tuple and the first item is the layout name and the second item should have the format described in *Layouts*.

To understand the format, see the following example (it is not intended to do anything useful):

```
from tkinter import ttk
import tkinter

root = tkinter.Tk()

style = ttk.Style()
style.layout("TMenubutton", [
    ("Menubutton.background", None),
    ("Menubutton.button", {"children":
        [("Menubutton.focus", {"children":
            [("Menubutton.padding", {"children":
                [("Menubutton.label", {"side": "left", "expand": 1})]
            })]
        })]
    }),
])

mbtn = ttk.Menubutton(text='Text')
mbtn.pack()
root.mainloop()
```

element_create (*elementname*, *etype*, **args*, ***kw*)

Create a new element in the current theme, of the given *etype* which is expected to be either "image", "from" or "vsapi". The latter is only available in Tk 8.6 on Windows.

If "image" is used, *args* should contain the default image name followed by statespec/value pairs (this is the imagespec), and *kw* may have the following options:

border=padding
: padding is a list of up to four integers, specifying the left, top, right, and bottom borders, respectively.

height=height
: Specifies a minimum height for the element. If less than zero, the base image's height is used as a default.

padding=padding
: Specifies the element's interior padding. Defaults to border's value if not specified.

sticky=spec
: Specifies how the image is placed within the final parcel. spec contains zero or more characters "n", "s", "w", or "e".

width=width
: Specifies a minimum width for the element. If less than zero, the base image's width is used as a default.

Exemplo:

```
img1 = tkinter.PhotoImage(master=root, file='button.png')
img1 = tkinter.PhotoImage(master=root, file='button-pressed.png')
img1 = tkinter.PhotoImage(master=root, file='button-active.png')
style = ttk.Style(root)
```

(continua na próxima página)

(continuação da página anterior)

```
style.element_create('Button.button', 'image',
                     img1, ('pressed', img2), ('active', img3),
                     border=(2, 4), sticky='we')
```

If "from" is used as the value of *etype*, `element_create()` will clone an existing element. *args* is expected to contain a themename, from which the element will be cloned, and optionally an element to clone from. If this element to clone from is not specified, an empty element will be used. *kw* is discarded.

Exemplo:

```
style = ttk.Style(root)
style.element_create('plain.background', 'from', 'default')
```

If "vsapi" is used as the value of *etype*, `element_create()` will create a new element in the current theme whose visual appearance is drawn using the Microsoft Visual Styles API which is responsible for the themed styles on Windows XP and Vista. *args* is expected to contain the Visual Styles class and part as given in the Microsoft documentation followed by an optional sequence of tuples of ttk states and the corresponding Visual Styles API state value. *kw* may have the following options:

padding=padding
: Specify the element's interior padding. *padding* is a list of up to four integers specifying the left, top, right and bottom padding quantities respectively. If fewer than four elements are specified, bottom defaults to top, right defaults to left, and top defaults to left. In other words, a list of three numbers specify the left, vertical, and right padding; a list of two numbers specify the horizontal and the vertical padding; a single number specifies the same padding all the way around the widget. This option may not be mixed with any other options.

margins=padding
: Specifies the elements exterior padding. *padding* is a list of up to four integers specifying the left, top, right and bottom padding quantities respectively. This option may not be mixed with any other options.

width=width
: Specifies the width for the element. If this option is set then the Visual Styles API will not be queried for the recommended size or the part. If this option is set then *height* should also be set. The *width* and *height* options cannot be mixed with the *padding* or *margins* options.

height=height
: Specifies the height of the element. See the comments for *width*.

Exemplo:

```
style = ttk.Style(root)
style.element_create('pin', 'vsapi', 'EXPLORERBAR', 3, [
                     ('pressed', '!selected', 3),
                     ('active', '!selected', 2),
                     ('pressed', 'selected', 6),
                     ('active', 'selected', 5),
                     ('selected', 4),
                     ('', 1)])
style.layout('Explorer.Pin',
             [('Explorer.Pin.pin', {'sticky': 'news'})])
pin = ttk.Checkbutton(style='Explorer.Pin')
pin.pack(expand=True, fill='both')
```

Alterado na versão 3.13: Added support of the "vsapi" element factory.

element_names()
: Returns the list of elements defined in the current theme.

element_options(*elementname*)

Returns the list of *elementname*'s options.

theme_create(*themename*, *parent=None*, *settings=None*)

Create a new theme.

It is an error if *themename* already exists. If *parent* is specified, the new theme will inherit styles, elements and layouts from the parent theme. If *settings* are present they are expected to have the same syntax used for *theme_settings()*.

theme_settings(*themename*, *settings*)

Temporarily sets the current theme to *themename*, apply specified *settings* and then restore the previous theme.

Each key in *settings* is a style and each value may contain the keys 'configure', 'map', 'layout' and 'element create' and they are expected to have the same format as specified by the methods *Style.configure()*, *Style.map()*, *Style.layout()* and *Style.element_create()* respectively.

As an example, let's change the Combobox for the default theme a bit:

```
from tkinter import ttk
import tkinter

root = tkinter.Tk()

style = ttk.Style()
style.theme_settings("default", {
   "TCombobox": {
       "configure": {"padding": 5},
       "map": {
           "background": [("active", "green2"),
                          ("!disabled", "green4")],
           "fieldbackground": [("!disabled", "green3")],
           "foreground": [("focus", "OliveDrab1"),
                          ("!disabled", "OliveDrab2")]
       }
   }
})

combo = ttk.Combobox().pack()

root.mainloop()
```

theme_names()

Returns a list of all known themes.

theme_use(*themename=None*)

If *themename* is not given, returns the theme in use. Otherwise, sets the current theme to *themename*, refreshes all widgets and emits a «ThemeChanged» event.

Layouts

A layout can be just `None`, if it takes no options, or a dict of options specifying how to arrange the element. The layout mechanism uses a simplified version of the pack geometry manager: given an initial cavity, each element is allocated a parcel.

The valid options/values are:

***side*: whichside**
Specifies which side of the cavity to place the element; one of top, right, bottom or left. If omitted, the element occupies the entire cavity.

***sticky*: nswe**
Specifies where the element is placed inside its allocated parcel.

***unit*: 0 ou 1**
If set to 1, causes the element and all of its descendants to be treated as a single element for the purposes of `Widget.identify()` et al. It's used for things like scrollbar thumbs with grips.

***children*: [sublayout…]**
Specifies a list of elements to place inside the element. Each element is a tuple (or other sequence type) where the first item is the layout name, and the other is a *Layout*.

25.9 IDLE — editor e console Python

Código-fonte: Lib/idlelib/

IDLE é o Ambiente Integrado de Desenvolvimento e Aprendizagem (em inglês, Integrated Development and Learning Environment) do Python.

O IDLE tem os seguintes recursos:

- multiplataforma: funciona basicamente da mesma forma no Windows, Unix e macOS
- Janela de console do Python (interpretador interativo) com colorização de entrada de código, saída e mensagens de erro
- editor de texto multi-janela com múltiplos desfazer, colorização Python, indentação inteligente, dicas de chamada, preenchimento automático e outros recursos
- pesquisa em qualquer janela, substituição nas janelas do editor e pesquisa em vários arquivos (grep)
- depurador com pontos de interrupção persistentes, etapas e visualização de espaço de nomes global e local
- configuração, navegadores e outros diálogos

25.9.1 Menus

O IDLE possui dois tipos principais de janelas: a janela do console e a janela do Editor. É possível ter várias janelas do Editor simultaneamente. No Windows e no Linux, cada uma tem seu próprio menu superior. Cada menu documentado abaixo indica a qual tipo de janela está associado.

Output windows, such as used for Edit => Find in Files, are a subtype of editor window. They currently have the same top menu but a different default title and context menu.

No macOS, há um menu de aplicações. Ele muda dinamicamente de acordo com a janela selecionada no momento. Ele possui um menu de IDLE, e algumas entradas descritas abaixo foram movidas para atender às diretrizes da Apple.

Menu Arquivo (Console e Editor)

Novo Arquivo
Create a new file editing window.

Abrir…
Open an existing file with an Open dialog.

Open Module…
Open an existing module (searches sys.path).

Arquivos Recentes
Open a list of recent files. Click one to open it.

Module Browser
Show functions, classes, and methods in the current Editor file in a tree structure. In the shell, open a module first.

Path Browser
Show sys.path directories, modules, functions, classes and methods in a tree structure.

Salvar
Save the current window to the associated file, if there is one. Windows that have been changed since being opened or last saved have a * before and after the window title. If there is no associated file, do Save As instead.

Salvar como…
Save the current window with a Save As dialog. The file saved becomes the new associated file for the window. (If your file namager is set to hide extensions, the current extension will be omitted in the file name box. If the new filename has no '.', '.py' and '.txt' will be added for Python and text files, except that on macOS Aqua,'.py' is added for all files.)

Save Copy As…
Save the current window to different file without changing the associated file. (See Save As note above about filename extensions.)

Print Window
Print the current window to the default printer.

Close Window
Close the current window (if an unsaved editor, ask to save; if an unsaved Shell, ask to quit execution). Calling `exit()` or `close()` in the Shell window also closes Shell. If this is the only window, also exit IDLE.

Exit IDLE
Close all windows and quit IDLE (ask to save unsaved edit windows).

Edit menu (Shell and Editor)

Desfazer
Desfaz a última alteração na janela atual. Um máximo de 1000 alterações podem ser desfeitas.

Redo
Redo the last undone change to the current window.

Select All
Select the entire contents of the current window.

Cut
Copy selection into the system-wide clipboard; then delete the selection.

Copy
Copy selection into the system-wide clipboard.

Paste
Insert contents of the system-wide clipboard into the current window.

The clipboard functions are also available in context menus.

Find…
Open a search dialog with many options

Find Again
Repeat the last search, if there is one.

Find Selection
Search for the currently selected string, if there is one.

Find in Files…
Open a file search dialog. Put results in a new output window.

Replace…
Open a search-and-replace dialog.

Go to Line
Move the cursor to the beginning of the line requested and make that line visible. A request past the end of the file goes to the end. Clear any selection and update the line and column status.

Show Completions
Open a scrollable list allowing selection of existing names. See *Completions* in the Editing and navigation section below.

Expand Word
Expand a prefix you have typed to match a full word in the same window; repeat to get a different expansion.

Show Call Tip
After an unclosed parenthesis for a function, open a small window with function parameter hints. See *Calltips* in the Editing and navigation section below.

Show Surrounding Parens
Highlight the surrounding parenthesis.

Format menu (Editor window only)

Format Paragraph
Reformat the current blank-line-delimited paragraph in comment block or multiline string or selected line in a string. All lines in the paragraph will be formatted to less than N columns, where N defaults to 72.

Indent Region
Shift selected lines right by the indent width (default 4 spaces).

Dedent Region
Shift selected lines left by the indent width (default 4 spaces).

Comment Out Region
Insert ## in front of selected lines.

Uncomment Region
Remove leading # or ## from selected lines.

Tabify Region
Turn *leading* stretches of spaces into tabs. (Note: We recommend using 4 space blocks to indent Python code.)

Untabify Region
Turn *all* tabs into the correct number of spaces.

Toggle Tabs
Open a dialog to switch between indenting with spaces and tabs.

New Indent Width
Open a dialog to change indent width. The accepted default by the Python community is 4 spaces.

Strip Trailing Chitespace
Remove trailing space and other whitespace characters after the last non-whitespace character of a line by applying str.rstrip to each line, including lines within multiline strings. Except for Shell windows, remove extra newlines at the end of the file.

Run menu (Editor window only)

Run Module
Do *Check Module*. If no error, restart the shell to clean the environment, then execute the module. Output is displayed in the Shell window. Note that output requires use of `print` or `write`. When execution is complete, the Shell retains focus and displays a prompt. At this point, one may interactively explore the result of execution. This is similar to executing a file with `python -i file` at a command line.

Run… Customized
Same as *Run Module*, but run the module with customized settings. *Command Line Arguments* extend `sys.argv` as if passed on a command line. The module can be run in the Shell without restarting.

Check Module
Check the syntax of the module currently open in the Editor window. If the module has not been saved IDLE will either prompt the user to save or autosave, as selected in the General tab of the Idle Settings dialog. If there is a syntax error, the approximate location is indicated in the Editor window.

Python Shell
Open or wake up the Python Shell window.

Shell menu (Shell window only)

View Last Restart
Scroll the shell window to the last Shell restart.

Restart Shell
Restart the shell to clean the environment and reset display and exception handling.

Previous History
Cycle through earlier commands in history which match the current entry.

Next History
Cycle through later commands in history which match the current entry.

Interrupt Execution
Stop a running program.

Debug menu (Shell window only)

Go to File/Line
Look on the current line. with the cursor, and the line above for a filename and line number. If found, open the file if not already open, and show the line. Use this to view source lines referenced in an exception traceback and lines found by Find in Files. Also available in the context menu of the Shell window and Output windows.

Debugger (toggle)
When activated, code entered in the Shell or run from an Editor will run under the debugger. In the Editor, breakpoints can be set with the context menu. This feature is still incomplete and somewhat experimental.

Stack Viewer
Show the stack traceback of the last exception in a tree widget, with access to locals and globals.

Auto-open Stack Viewer
Toggle automatically opening the stack viewer on an unhandled exception.

Options menu (Shell and Editor)

Configure IDLE
Open a configuration dialog and change preferences for the following: fonts, indentation, keybindings, text color themes, startup windows and size, additional help sources, and extensions. On macOS, open the configuration dialog by selecting Preferences in the application menu. For more details, see *Setting preferences* under Help and preferences.

Most configuration options apply to all windows or all future windows. The option items below only apply to the active window.

Show/Hide Code Context (Editor Window only)
Open a pane at the top of the edit window which shows the block context of the code which has scrolled above the top of the window. See *Code Context* in the Editing and Navigation section below.

Show/Hide Line Numbers (Editor Window only)
Open a column to the left of the edit window which shows the number of each line of text. The default is off, which may be changed in the preferences (see *Setting preferences*).

Zoom/Restore Height
Toggles the window between normal size and maximum height. The initial size defaults to 40 lines by 80 chars unless changed on the General tab of the Configure IDLE dialog. The maximum height for a screen is determined by momentarily maximizing a window the first time one is zoomed on the screen. Changing screen settings may invalidate the saved height. This toggle has no effect when a window is maximized.

Window menu (Shell and Editor)

Lists the names of all open windows; select one to bring it to the foreground (deiconifying it if necessary).

Help menu (Shell and Editor)

About IDLE
Display version, copyright, license, credits, and more.

IDLE Help
Display this IDLE document, detailing the menu options, basic editing and navigation, and other tips.

Documentação do Python
Access local Python documentation, if installed, or start a web browser and open docs.python.org showing the latest Python documentation.

Demonstração com o Turtle
Run the turtledemo module with example Python code and turtle drawings.

Additional help sources may be added here with the Configure IDLE dialog under the General tab. See the *Help sources* subsection below for more on Help menu choices.

Context menus

Open a context menu by right-clicking in a window (Control-click on macOS). Context menus have the standard clipboard functions also on the Edit menu.

Cut
Copy selection into the system-wide clipboard; then delete the selection.

Copy
Copy selection into the system-wide clipboard.

Paste
Insert contents of the system-wide clipboard into the current window.

Editor windows also have breakpoint functions. Lines with a breakpoint set are specially marked. Breakpoints only have an effect when running under the debugger. Breakpoints for a file are saved in the user's `.idlerc` directory.

Set Breakpoint
Set a breakpoint on the current line.

Clear Breakpoint
Clear the breakpoint on that line.

Shell and Output windows also have the following.

Go to file/line
Same as in Debug menu.

The Shell window also has an output squeezing facility explained in the *Python Shell window* subsection below.

Squeeze
If the cursor is over an output line, squeeze all the output between the code above and the prompt below down to a 'Squeezed text' label.

25.9.2 Editing and Navigation

Editor windows

IDLE may open editor windows when it starts, depending on settings and how you start IDLE. Thereafter, use the File menu. There can be only one open editor window for a given file.

The title bar contains the name of the file, the full path, and the version of Python and IDLE running the window. The status bar contains the line number ('Ln') and column number ('Col'). Line numbers start with 1; column numbers with 0.

IDLE assumes that files with a known .py* extension contain Python code and that other files do not. Run Python code with the Run menu.

Teclas de atalho

The IDLE insertion cursor is a thin vertical bar between character positions. When characters are entered, the insertion cursor and everything to its right moves right one character and the new character is entered in the new space.

Several non-character keys move the cursor and possibly delete characters. Deletion does not puts text on the clipboard, but IDLE has an undo list. Wherever this doc discusses keys, 'C' refers to the `Control` key on Windows and Unix and the `Command` key on macOS. (And all such discussions assume that the keys have not been re-bound to something else.)

- Arrow keys move the cursor one character or line.
- `C-LeftArrow` and `C-RightArrow` moves left or right one word.
- `Home` and `End` go to the beginning or end of the line.
- `Page Up` and `Page Down` go up or down one screen.
- `C-Home` and `C-End` go to beginning or end of the file.
- `Backspace` and `Del` (or `C-d`) delete the previous or next character.
- `C-Backspace` and `C-Del` delete one word left or right.
- `C-k` deletes ('kills') everything to the right.

Standard keybindings (like `C-c` to copy and `C-v` to paste) may work. Keybindings are selected in the Configure IDLE dialog.

Automatic indentation

After a block-opening statement, the next line is indented by 4 spaces (in the Python Shell window by one tab). After certain keywords (break, return etc.) the next line is dedented. In leading indentation, `Backspace` deletes up to 4 spaces if they are there. `Tab` inserts spaces (in the Python Shell window one tab), number depends on Indent width. Currently, tabs are restricted to four spaces due to Tcl/Tk limitations.

See also the indent/dedent region commands on the *Format menu*.

Busca e Substituição

Any selection becomes a search target. However, only selections within a line work because searches are only performed within lines with the terminal newline removed. If `[x] Regular expression` is checked, the target is interpreted according to the Python re module.

Completions

Completions are supplied, when requested and available, for module names, attributes of classes or functions, or filenames. Each request method displays a completion box with existing names. (See tab completions below for an exception.) For any box, change the name being completed and the item highlighted in the box by typing and deleting characters; by hitting `Up`, `Down`, `PageUp`, `PageDown`, `Home`, and `End` keys; and by a single click within the box. Close the box with `Escape`, `Enter`, and double `Tab` keys or clicks outside the box. A double click within the box selects and closes.

One way to open a box is to type a key character and wait for a predefined interval. This defaults to 2 seconds; customize it in the settings dialog. (To prevent auto popups, set the delay to a large number of milliseconds, such as 100000000.) For imported module names or class or function attributes, type '.'. For filenames in the root directory, type `os.sep` or `os.altsep` immediately after an opening quote. (On Windows, one can specify a drive first.) Move into subdirectories by typing a directory name and a separator.

Instead of waiting, or after a box is closed, open a completion box immediately with Show Completions on the Edit menu. The default hot key is `C-space`. If one types a prefix for the desired name before opening the box, the first

match or near miss is made visible. The result is the same as if one enters a prefix after the box is displayed. Show Completions after a quote completes filenames in the current directory instead of a root directory.

Hitting `Tab` after a prefix usually has the same effect as Show Completions. (With no prefix, it indents.) However, if there is only one match to the prefix, that match is immediately added to the editor text without opening a box.

Invoking 'Show Completions', or hitting `Tab` after a prefix, outside of a string and without a preceding '.' opens a box with keywords, builtin names, and available module-level names.

When editing code in an editor (as oppose to Shell), increase the available module-level names by running your code and not restarting the Shell thereafter. This is especially useful after adding imports at the top of a file. This also increases possible attribute completions.

Completion boxes initially exclude names beginning with '_' or, for modules, not included in '__all__'. The hidden names can be accessed by typing '_' after '.', either before or after the box is opened.

Calltips

A calltip is shown automatically when one types `(` after the name of an *accessible* function. A function name expression may include dots and subscripts. A calltip remains until it is clicked, the cursor is moved out of the argument area, or `)` is typed. Whenever the cursor is in the argument part of a definition, select Edit and "Show Call Tip" on the menu or enter its shortcut to display a calltip.

The calltip consists of the function's signature and docstring up to the latter's first blank line or the fifth non-blank line. (Some builtin functions lack an accessible signature.) A '/' or '*' in the signature indicates that the preceding or following arguments are passed by position or name (keyword) only. Details are subject to change.

In Shell, the accessible functions depends on what modules have been imported into the user process, including those imported by Idle itself, and which definitions have been run, all since the last restart.

For example, restart the Shell and enter `itertools.count(`. A calltip appears because Idle imports itertools into the user process for its own use. (This could change.) Enter `turtle.write(` and nothing appears. Idle does not itself import turtle. The menu entry and shortcut also do nothing. Enter `import turtle`. Thereafter, `turtle.write(` will display a calltip.

In an editor, import statements have no effect until one runs the file. One might want to run a file after writing import statements, after adding function definitions, or after opening an existing file.

Code Context

Within an editor window containing Python code, code context can be toggled in order to show or hide a pane at the top of the window. When shown, this pane freezes the opening lines for block code, such as those beginning with `class`, `def`, or `if` keywords, that would have otherwise scrolled out of view. The size of the pane will be expanded and contracted as needed to show the all current levels of context, up to the maximum number of lines defined in the Configure IDLE dialog (which defaults to 15). If there are no current context lines and the feature is toggled on, a single blank line will display. Clicking on a line in the context pane will move that line to the top of the editor.

The text and background colors for the context pane can be configured under the Highlights tab in the Configure IDLE dialog.

Shell window

In IDLE's Shell, enter, edit, and recall complete statements. (Most consoles and terminals only work with a single physical line at a time).

Submit a single-line statement for execution by hitting `Return` with the cursor anywhere on the line. If a line is extended with Backslash (`\`), the cursor must be on the last physical line. Submit a multi-line compound statement by entering a blank line after the statement.

When one pastes code into Shell, it is not compiled and possibly executed until one hits `Return`, as specified above. One may edit pasted code first. If one pastes more than one statement into Shell, the result will be a *SyntaxError* when multiple statements are compiled as if they were one.

Lines containing RESTART mean that the user execution process has been re-started. This occurs when the user execution process has crashed, when one requests a restart on the Shell menu, or when one runs code in an editor window.

The editing features described in previous subsections work when entering code interactively. IDLE's Shell window also responds to the following:

- C-c attempts to interrupt statement execution (but may fail).
- C-d closes Shell if typed at a >>> prompt.
- Alt-p and Alt-n (C-p and C-n on macOS) retrieve to the current prompt the previous or next previously entered statement that matches anything already typed.
- Return while the cursor is on any previous statement appends the latter to anything already typed at the prompt.

Text colors

Idle defaults to black on white text, but colors text with special meanings. For the shell, these are shell output, shell error, user output, and user error. For Python code, at the shell prompt or in an editor, these are keywords, builtin class and function names, names following class and def, strings, and comments. For any text window, these are the cursor (when present), found text (when possible), and selected text.

IDLE also highlights the soft keywords match, case, and _ in pattern-matching statements. However, this highlighting is not perfect and will be incorrect in some rare cases, including some _-s in case patterns.

Text coloring is done in the background, so uncolorized text is occasionally visible. To change the color scheme, use the Configure IDLE dialog Highlighting tab. The marking of debugger breakpoint lines in the editor and text in popups and dialogs is not user-configurable.

25.9.3 Startup and Code Execution

Upon startup with the -s option, IDLE will execute the file referenced by the environment variables IDLESTARTUP or PYTHONSTARTUP. IDLE first checks for IDLESTARTUP; if IDLESTARTUP is present the file referenced is run. If IDLESTARTUP is not present, IDLE checks for PYTHONSTARTUP. Files referenced by these environment variables are convenient places to store functions that are used frequently from the IDLE shell, or for executing import statements to import common modules.

In addition, Tk also loads a startup file if it is present. Note that the Tk file is loaded unconditionally. This additional file is .Idle.py and is looked for in the user's home directory. Statements in this file will be executed in the Tk namespace, so this file is not useful for importing functions to be used from IDLE's Python shell.

Uso na linha de comando

IDLE can be invoked from the command line with various options. The general syntax is:

```
python -m idlelib [options] [file ...]
```

The following options are available:

-c <command>

Run the specified Python command in the shell window. For example, pass -c "print('Hello, World!')". On Windows, the outer quotes must be double quotes as shown.

-d

Enable the debugger and open the shell window.

-e

Open an editor window.

-h

Print a help message with legal combinations of options and exit.

-i

Open a shell window.

-r <file>

Run the specified file in the shell window.

-s

Run the startup file (as defined by the environment variables `IDLESTARTUP` or `PYTHONSTARTUP`) before opening the shell window.

-t <title>

Set the title of the shell window.

-

Read and execute standard input in the shell window. This option must be the last one before any arguments.

If arguments are provided:

- If `-`, `-c`, or `-r` is used, all arguments are placed in `sys.argv[1:]`, and `sys.argv[0]` is set to `''`, `'-c'`, or `'-r'` respectively. No editor window is opened, even if that is the default set in the *Options* dialog.
- Otherwise, arguments are treated as files to be opened for editing, and `sys.argv` reflects the arguments passed to IDLE itself.

Startup failure

IDLE uses a socket to communicate between the IDLE GUI process and the user code execution process. A connection must be established whenever the Shell starts or restarts. (The latter is indicated by a divider line that says 'RESTART'). If the user process fails to connect to the GUI process, it usually displays a `Tk` error box with a 'cannot connect' message that directs the user here. It then exits.

One specific connection failure on Unix systems results from misconfigured masquerading rules somewhere in a system's network setup. When IDLE is started from a terminal, one will see a message starting with `** Invalid host:`. The valid value is `127.0.0.1 (idlelib.rpc.LOCALHOST)`. One can diagnose with `tcpconnect -irv 127.0.0.1 6543` in one terminal window and `tcplisten <same args>` in another.

A common cause of failure is a user-written file with the same name as a standard library module, such as *random.py* and *tkinter.py*. When such a file is located in the same directory as a file that is about to be run, IDLE cannot import the stdlib file. The current fix is to rename the user file.

Though less common than in the past, an antivirus or firewall program may stop the connection. If the program cannot be taught to allow the connection, then it must be turned off for IDLE to work. It is safe to allow this internal connection because no data is visible on external ports. A similar problem is a network mis-configuration that blocks connections.

Python installation issues occasionally stop IDLE: multiple versions can clash, or a single installation might need admin access. If one undo the clash, or cannot or does not want to run as admin, it might be easiest to completely remove Python and start over.

A zombie pythonw.exe process could be a problem. On Windows, use Task Manager to check for one and stop it if there is. Sometimes a restart initiated by a program crash or Keyboard Interrupt (control-C) may fail to connect. Dismissing the error box or using Restart Shell on the Shell menu may fix a temporary problem.

When IDLE first starts, it attempts to read user configuration files in `~/.idlerc/` (~ is one's home directory). If there is a problem, an error message should be displayed. Leaving aside random disk glitches, this can be prevented by never editing the files by hand. Instead, use the configuration dialog, under Options. Once there is an error in a user configuration file, the best solution may be to delete it and start over with the settings dialog.

If IDLE quits with no message, and it was not started from a console, try starting it from a console or terminal (`python -m idlelib`) and see if this results in an error message.

On Unix-based systems with tcl/tk older than `8.6.11` (see `About IDLE`) certain characters of certain fonts can cause a tk failure with a message to the terminal. This can happen either if one starts IDLE to edit a file with such a character or later when entering such a character. If one cannot upgrade tcl/tk, then re-configure IDLE to use a font that works better.

Running user code

With rare exceptions, the result of executing Python code with IDLE is intended to be the same as executing the same code by the default method, directly with Python in a text-mode system console or terminal window. However, the different interface and operation occasionally affect visible results. For instance, `sys.modules` starts with more entries, and `threading.active_count()` returns 2 instead of 1.

By default, IDLE runs user code in a separate OS process rather than in the user interface process that runs the shell and editor. In the execution process, it replaces `sys.stdin`, `sys.stdout`, and `sys.stderr` with objects that get input from and send output to the Shell window. The original values stored in `sys.__stdin__`, `sys.__stdout__`, and `sys.__stderr__` are not touched, but may be `None`.

Sending print output from one process to a text widget in another is slower than printing to a system terminal in the same process. This has the most effect when printing multiple arguments, as the string for each argument, each separator, the newline are sent separately. For development, this is usually not a problem, but if one wants to print faster in IDLE, format and join together everything one wants displayed together and then print a single string. Both format strings and *`str.join()`* can help combine fields and lines.

IDLE's standard stream replacements are not inherited by subprocesses created in the execution process, whether directly by user code or by modules such as multiprocessing. If such subprocess use `input` from sys.stdin or `print` or `write` to sys.stdout or sys.stderr, IDLE should be started in a command line window. (On Windows, use `python` or `py` rather than `pythonw` or `pyw`.) The secondary subprocess will then be attached to that window for input and output.

If `sys` is reset by user code, such as with `importlib.reload(sys)`, IDLE's changes are lost and input from the keyboard and output to the screen will not work correctly.

When Shell has the focus, it controls the keyboard and screen. This is normally transparent, but functions that directly access the keyboard and screen will not work. These include system-specific functions that determine whether a key has been pressed and if so, which.

The IDLE code running in the execution process adds frames to the call stack that would not be there otherwise. IDLE wraps `sys.getrecursionlimit` and `sys.setrecursionlimit` to reduce the effect of the additional stack frames.

When user code raises SystemExit either directly or by calling sys.exit, IDLE returns to a Shell prompt instead of exiting.

User output in Shell

When a program outputs text, the result is determined by the corresponding output device. When IDLE executes user code, `sys.stdout` and `sys.stderr` are connected to the display area of IDLE's Shell. Some of its features are inherited from the underlying Tk Text widget. Others are programmed additions. Where it matters, Shell is designed for development rather than production runs.

For instance, Shell never throws away output. A program that sends unlimited output to Shell will eventually fill memory, resulting in a memory error. In contrast, some system text windows only keep the last n lines of output. A Windows console, for instance, keeps a user-settable 1 to 9999 lines, with 300 the default.

A Tk Text widget, and hence IDLE's Shell, displays characters (codepoints) in the BMP (Basic Multilingual Plane) subset of Unicode. Which characters are displayed with a proper glyph and which with a replacement box depends on the operating system and installed fonts. Tab characters cause the following text to begin after the next tab stop. (They occur every 8 'characters'). Newline characters cause following text to appear on a new line. Other control characters are ignored or displayed as a space, box, or something else, depending on the operating system and font. (Moving the text cursor through such output with arrow keys may exhibit some surprising spacing behavior.)

```
>>> s = 'a\tb\a<\x02><\r>\bc\nd'  # Enter 22 chars.
>>> len(s)
14
>>> s  # Display repr(s)
'a\tb\x07<\x02><\r>\x08c\nd'
>>> print(s, end='')  # Display s as is.
# Result varies by OS and font.  Try it.
```

The `repr` function is used for interactive echo of expression values. It returns an altered version of the input string in which control codes, some BMP codepoints, and all non-BMP codepoints are replaced with escape codes. As demonstrated above, it allows one to identify the characters in a string, regardless of how they are displayed.

Normal and error output are generally kept separate (on separate lines) from code input and each other. They each get different highlight colors.

For SyntaxError tracebacks, the normal '^' marking where the error was detected is replaced by coloring the text with an error highlight. When code run from a file causes other exceptions, one may right click on a traceback line to jump to the corresponding line in an IDLE editor. The file will be opened if necessary.

Shell has a special facility for squeezing output lines down to a 'Squeezed text' label. This is done automatically for output over N lines (N = 50 by default). N can be changed in the PyShell section of the General page of the Settings dialog. Output with fewer lines can be squeezed by right clicking on the output. This can be useful lines long enough to slow down scrolling.

Squeezed output is expanded in place by double-clicking the label. It can also be sent to the clipboard or a separate view window by right-clicking the label.

Developing tkinter applications

IDLE is intentionally different from standard Python in order to facilitate development of tkinter programs. Enter `import tkinter as tk; root = tk.Tk()` in standard Python and nothing appears. Enter the same in IDLE and a tk window appears. In standard Python, one must also enter `root.update()` to see the window. IDLE does the equivalent in the background, about 20 times a second, which is about every 50 milliseconds. Next enter `b = tk.Button(root, text='button'); b.pack()`. Again, nothing visibly changes in standard Python until one enters `root.update()`.

Most tkinter programs run `root.mainloop()`, which usually does not return until the tk app is destroyed. If the program is run with `python -i` or from an IDLE editor, a `>>>` shell prompt does not appear until `mainloop()` returns, at which time there is nothing left to interact with.

When running a tkinter program from an IDLE editor, one can comment out the mainloop call. One then gets a shell prompt immediately and can interact with the live application. One just has to remember to re-enable the mainloop call when running in standard Python.

Running without a subprocess

By default, IDLE executes user code in a separate subprocess via a socket, which uses the internal loopback interface. This connection is not externally visible and no data is sent to or received from the internet. If firewall software complains anyway, you can ignore it.

If the attempt to make the socket connection fails, Idle will notify you. Such failures are sometimes transient, but if persistent, the problem may be either a firewall blocking the connection or misconfiguration of a particular system. Until the problem is fixed, one can run Idle with the -n command line switch.

If IDLE is started with the -n command line switch it will run in a single process and will not create the subprocess which runs the RPC Python execution server. This can be useful if Python cannot create the subprocess or the RPC socket interface on your platform. However, in this mode user code is not isolated from IDLE itself. Also, the environment is not restarted when Run/Run Module (F5) is selected. If your code has been modified, you must reload() the affected modules and re-import any specific items (e.g. from foo import baz) if the changes are to take effect. For these reasons, it is preferable to run IDLE with the default subprocess if at all possible.

Descontinuado desde a versão 3.4.

25.9.4 Help and Preferences

Help sources

Help menu entry "IDLE Help" displays a formatted html version of the IDLE chapter of the Library Reference. The result, in a read-only tkinter text window, is close to what one sees in a web browser. Navigate through the text with a mousewheel, the scrollbar, or up and down arrow keys held down. Or click the TOC (Table of Contents) button and select a section header in the opened box.

Help menu entry "Python Docs" opens the extensive sources of help, including tutorials, available at `docs.python.org/x.y`, where 'x.y' is the currently running Python version. If your system has an off-line copy of the docs (this may be an installation option), that will be opened instead.

Selected URLs can be added or removed from the help menu at any time using the General tab of the Configure IDLE dialog.

Setting preferences

The font preferences, highlighting, keys, and general preferences can be changed via Configure IDLE on the Option menu. Non-default user settings are saved in a `.idlerc` directory in the user's home directory. Problems caused by bad user configuration files are solved by editing or deleting one or more of the files in `.idlerc`.

On the Font tab, see the text sample for the effect of font face and size on multiple characters in multiple languages. Edit the sample to add other characters of personal interest. Use the sample to select monospaced fonts. If particular characters have problems in Shell or an editor, add them to the top of the sample and try changing first size and then font.

On the Highlights and Keys tab, select a built-in or custom color theme and key set. To use a newer built-in color theme or key set with older IDLEs, save it as a new custom theme or key set and it well be accessible to older IDLEs.

IDLE on macOS

Under System Preferences: Dock, one can set "Prefer tabs when opening documents" to "Always". This setting is not compatible with the tk/tkinter GUI framework used by IDLE, and it breaks a few IDLE features.

Extensions

IDLE contains an extension facility. Preferences for extensions can be changed with the Extensions tab of the preferences dialog. See the beginning of config-extensions.def in the idlelib directory for further information. The only current default extension is zzdummy, an example also used for testing.

25.9.5 idlelib — implementation of IDLE application

Source code: Lib/idlelib

The Lib/idlelib package implements the IDLE application. See the rest of this page for how to use IDLE.

The files in idlelib are described in idlelib/README.txt. Access it either in idlelib or click Help => About IDLE on the IDLE menu. This file also maps IDLE menu items to the code that implements the item. Except for files listed under 'Startup', the idlelib code is 'private' in sense that feature changes can be backported (see **PEP 434**).

25.10 `turtle` — Gráficos tartaruga

Código-fonte: Lib/turtle.py

25.10.1 Introdução

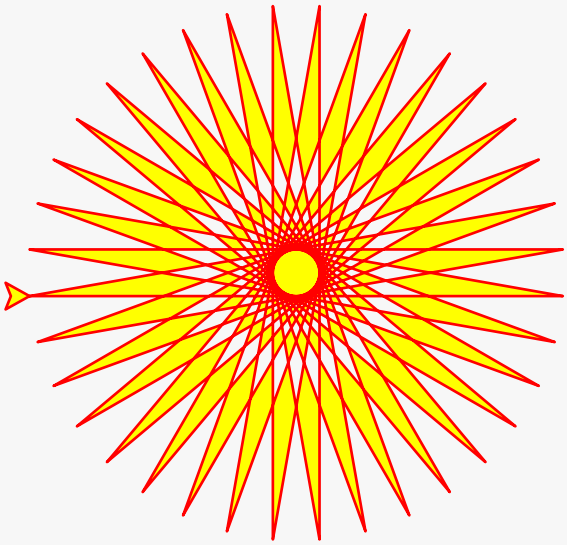
Gráficos tartaruga são uma implementação das populares ferramentas de desenho geométrico introduzidas em Logo, desenvolvido por Wally Feurzeig, Seymour Papert e Cynthia Solomon em 1967.

25.10.2 Começando

Imagine uma tartaruga robótica começando em (0, 0) no plano x-y. Depois de um `import turtle`, execute o comando `turtle.forward(15)`, e ela moverá (na tela!) 15 pixels na direção em que está virada (para frente), desenhando uma linha à medida que se move. Execute o comando `turtle.right(25)`, e ela fará uma rotação, de 25 graus no sentido horário, sem sair do lugar.

Turtle star

A tartaruga pode desenhar formas intrincadas usando programas que repetem movimentos simples.



Em Python, os gráficos de tartaruga fornecem uma representação de uma "tartaruga" física (um pequeno robô com uma caneta) que desenha em uma folha de papel no chão.

É uma maneira eficaz e comprovada de os alunos conhecerem os conceitos de programação e a interação com o software, pois oferece resposta instantânea e visível. Também oferece acesso conveniente a resultados gráficos em geral.

Desenho de tartaruga foi originalmente criado como uma ferramenta educacional, para ser usado por professores em sala de aula. Para a pessoa que precisa produzir algum resultado gráfico, essa pode ser uma maneira de fazer isso sem a sobrecarga de introduzir bibliotecas mais complexas ou externas em seu trabalho.

25.10.3 Tutorial

As pessoas recém-chegadas devem começar por aqui. Neste tutorial, exploraremos alguns dos conceitos básicos do desenho de tartarugas.

Iniciando um ambiente da tartaruga

Em um console do Python, faça a importação de todos os objetos do módulo `turtle`:

```
from turtle import *
```

Se você se deparar com a seguinte mensagem de erro `No module named '_tkinter'`, será necessário instalar o *pacote de interface gráfica Tk* em seu sistema.

Desenho básico

Envie a tartaruga para frente 100 passos:

```
forward(100)
```

Você deverá ver (provavelmente, em uma nova janela na sua tela) uma linha desenhada pela tartaruga, em direção ao leste (→) . Mude a direção da tartaruga, de modo que ela gire 120 graus para a esquerda (sentido anti-horário):

```
left(120)
```

Vamos continuar desenhando um triângulo:

```
forward(100)
left(120)
forward(100)
```

Observe como a tartaruga, representada por uma seta, aponta em diferentes direções à medida que você a guia.

Faça experiências com esses comandos e também com `backward()` e `right()`.

Controle da Caneta

Tente alterar a cor - por exemplo, `color('blue')` - e a largura da linha - por exemplo, `width(3)` - e, em seguida, desenhe novamente.

Você também pode mover a tartaruga sem desenhar, levantando a caneta: `up()` antes de mover. Para começar a desenhar novamente, use `down()`.

A posição da tartaruga

Envie sua tartaruga de volta ao ponto de partida (útil se ela tiver desaparecido da tela):

```
home()
```

A posição inicial da tartaruga está no centro da tela. Se você precisar conhecê-la, obtenha as coordenadas x-y da tartaruga com:

```
pos()
```

A a posição inicial fica em `(0, 0)`.

E, depois de um tempo, provavelmente vai querer ajudar a limpar a janela aberta para que possamos começar de novo:

```
clearscreen()
```

Criação de padrões algorítmicos

Usando laços de repetição, é possível construir criar padrões geométricos:

```
for steps in range(100):
    for c in ('blue', 'red', 'green'):
        color(c)
        forward(steps)
        right(30)
```

- que, é claro, são limitados apenas pela imaginação!

Vamos desenhar o formato de estrela na parte superior da página. Queremos linhas de contorno vermelhas, com preenchimento em amarelo:

```
color('red')
fillcolor('yellow')
```

Assim como `up()` e `down()` determinam se as linhas serão desenhadas, o preenchimento pode ser ativado e desativado:

```
begin_fill()
```

Em seguida, vamos criar um laço de repetição:

```
while True:
    forward(200)
    left(170)
    if abs(pos()) < 1:
        break
```

`abs(pos()) < 1` é uma boa maneira de saber quando a tartaruga está de volta à sua posição inicial.

Por fim, complete o preenchimento:

```
end_fill()
```

(Observe que o preenchimento só ocorre de fato quando você fornece o comando `end_fill()` .)

25.10.4 Como fazer...

Esta seção aborda alguns casos de uso e abordagens típicos do módulo "turtle".

Comece o mais rápido possível

Uma dos prazeres dos gráficos de tartaruga é o feedback visual imediato disponível a partir de comandos simples. É uma excelente maneira de apresentar ideias de programação às crianças, com um mínimo de sofrimento (não apenas às crianças, é claro).

O módulo tartaruga torna isso possível ao expor toda a sua funcionalidade básica como funções, disponíveis com `from turtle import *`. O tutorial gráficos de *turtle* cobre essa abordagem.

Vale a pena observar que muitos dos comandos da tartaruga também possuem equivalentes ainda mais concisos, como usar `fd()` para obter o mesmo resultado de *`forward()`*. Esses comandos são especialmente úteis quando se trabalha com alunos para os quais a digitação não é uma habilidade.

> Você precisará ter o pacote de interface *`Tk`* instalado em seu sistema para que os gráficos do módulo "turtle" funcionem. Esteja ciente de que isso nem sempre é simples, portanto, verifique isso com antecedência se estiver planejando usar os gráficos de tartaruga com um aluno.

Inicie e encerre o preenchimento automaticamente

A partir do Python 3.14, você pode usar o *gerenciador de contexto* *`fill()`* em vez de *`begin_fill()`* e *`end_fill()`* para iniciar e encerrar o preenchimento automaticamente. Aqui está um exemplo:

```
with fill():
    for i in range(4):
        forward(100)
        right(90)

forward(200)
```

O código acima equivale a:

```
begin_fill()
for i in range(4):
    forward(100)
    right(90)
end_fill()

forward(200)
```

Use o espaço de nomes do módulo `turtle`

Usar `from turtle import *` é conveniente, mas lembre-se de que ele importa um coleção bastante grande de objetos e, se você estiver fazendo qualquer coisa no seu código que não esteja relacionada com o módulo gráficos de tartaruga, corre o risco de um conflito de nomes de objetos (isso se torna um problema ainda maior se você estiver usando o módulo de gráficos de tartaruga em um script em que outros módulos possam ser também importados).

A solução é usar o identificador/domínio do próprio módulo `import turtle` sem o asterisco no final da importação - assim a chamada do objeto/função `fd()` torna-se `turtle.fd()`, da mesma forma como `width()` torna-se `turtle.width()` e assim por diante. (Se digitar "turtle" várias vezes se tornar tedioso, use `import turtle as t`, por exemplo, para fornecer um apelido mais conciso e objetivo ao identificador/domínio dentro do seu código).

Use o módulo gráfico de tartaruga dentro de um script com código específico

Recomenda-se usar o módulo `turtle` com uma apelido para identificador/domínio, conforme descrito acima, por exemplo:

```
import turtle as t
from random import random

for i in range(100):
    steps = int(random() * 100)
    angle = int(random() * 360)
    t.right(angle)
    t.fd(steps)
```

No entanto, outra etapa também é necessária ou o Python também fechará a janela da tartaruga assim que o código acima terminar de ser executado. Adicione:

```
t.mainloop()
```

ao final do código anterior. Assim, o código anterior agora conterá uma instrução que aguardará ser dispensado e não sairá até que seja encerrado, por exemplo, fechando a janela aberta pelo módulo gráficos da tartaruga.

Use o módulo gráficos da tartaruga orientado a objetos

Ver também

Explicação de interface orientada a objetos

Exceto para fins introdutórios muito básicos ou para experimentar coisas o mais rápido possível, é mais comum e muito mais eficiente usar a abordagem orientada a objetos para gráficos de tartarugas. Por exemplo, isso permite várias tartarugas na tela ao mesmo tempo.

Nessa abordagem, os vários comandos do módulo turtle são métodos de objetos (principalmente objetos de `Turtle`). Você *pode* usar a abordagem orientada a objetos em console, mas ela seria mais típica em um script Python.

O exemplo acima se torna então:

```
from turtle import Turtle
from random import random

t = Turtle()
for i in range(100):
    steps = int(random() * 100)
    angle = int(random() * 360)
    t.right(angle)
    t.fd(steps)
```

(continua na próxima página)

(continuação da página anterior)

```
t.screen.mainloop()
```

Observe a última linha. `t.screen` é um instância da subclasse *Screen* que existe numa instância da classe tartaruga ou "turtle"; ela é criada automaticamente junto com a instância de tartaruga.

A tela da tartaruga pode ser personalizada, por exemplo:

```
t.screen.title('Object-oriented turtle demo')
t.screen.bgcolor("orange")
```

25.10.5 Referência Gráficos de Tartaruga

Nota

Na documentação a seguir, a lista de argumentos para funções é fornecida. Os métodos, é claro, têm o primeiro argumento adicional *self* que é omitido aqui.

Métodos de Turtle

Movimentos de Turtle

Movimento e desenho

forward() | *fd()*
backward() | *bk()* | *back()*
right() | *rt()*
left() | *lt()*
goto() | *setpos()* | *setposition()*
teleport()
setx()
sety()
setheading() | *seth()*
home()
circle()
dot()
stamp()
clearstamp()
clearstamps()
undo()
speed()

Fala o estado de Turtle

position() | *pos()*
towards()
xcor()
ycor()
heading()
distance()

Configuração e Medidas

degrees()
radians()

Controle da Caneta

Estado do Desenho

pendown() | *pd()* | *down()*
penup() | *pu()* | *up()*
pensize() | *width()*
pen()
isdown()

Controle da Cor

color()
pencolor()
fillcolor()

Preenchimento

filling()
fill()
begin_fill()
end_fill()

Mais sobre o Controle do Desenho

reset()
clear()
write()

Estado da tartaruga

Visibilidade

showturtle() | *st()*
hideturtle() | *ht()*
isvisible()

Aparência

shape()
resizemode()
shapesize() | *turtlesize()*
shearfactor()
tiltangle()
tilt()
shapetransform()
get_shapepoly()

Eventos Utilizados

onclick()
onrelease()
ondrag()

Métodos Especiais da Tartaruga

poly()
begin_poly()
end_poly()
get_poly()
clone()
getturtle() | *getpen()*

getscreen()
setundobuffer()
undobufferentries()

Métodos de TurtleScreen/Screen

Controle da Janela

bgcolor()
bgpic()
clearscreen()
resetscreen()
screensize()
setworldcoordinates()

Controle da animação

no_animation()
delay()
tracer()
update()

Usando os eventos de tela

listen()
onkey() | *onkeyrelease()*
onkeypress()
onclick() | *onscreenclick()*
ontimer()
mainloop() | *done()*

Configurações e métodos especiais

mode()
colormode()
getcanvas()
getshapes()
register_shape() | *addshape()*
turtles()
window_height()
window_width()

Métodos de entrada

textinput()
numinput()

Métodos específicos para Screen

bye()
exitonclick()
save()
setup()
title()

25.10.6 Métodos de RawTurtle/Turtle e funções correspondentes

A maioria dos exemplos desta seção referem-se a uma instância Turtle chamada `turtle`.

Movimentos de Turtle

`turtle.`**`forward`**(*distance*)

`turtle.`**`fd`**(*distance*)

Parâmetros
`distance` – um número (inteiro ou ponto flutuante)

Move a tartaruga para frente pela *distance* especificada, na direção em que a tartaruga está indo.

```
>>> turtle.position()
(0.00,0.00)
>>> turtle.forward(25)
>>> turtle.position()
(25.00,0.00)
>>> turtle.forward(-75)
>>> turtle.position()
(-50.00,0.00)
```

`turtle.`**`back`**(*distance*)

`turtle.`**`bk`**(*distance*)

`turtle.`**`backward`**(*distance*)

Parâmetros
`distance` – um número

Move a tartaruga para trás por *distance*, na direção oposta à direção em que a tartaruga está indo. Não muda o rumo da tartaruga.

```
>>> turtle.position()
(0.00,0.00)
>>> turtle.backward(30)
>>> turtle.position()
(-30.00,0.00)
```

`turtle.`**`right`**(*angle*)

`turtle.`**`rt`**(*angle*)

Parâmetros
`angle` – um número (inteiro ou ponto flutuante)

Vira a tartaruga à direita por unidades de *angle*. (As unidades são por padrão graus, mas podem ser definidas através das funções *`degrees()`* e *`radians()`*.) A orientação do ângulo depende do modo tartaruga, veja *`mode()`*.

```
>>> turtle.heading()
22.0
>>> turtle.right(45)
>>> turtle.heading()
337.0
```

`turtle.`**`left`**(*angle*)

`turtle.`**`lt`**(*angle*)

Parâmetros
`angle` – um número (inteiro ou ponto flutuante)

Vira a tartaruga à esquerda por unidades de *angle*. (As unidades são por padrão graus, mas podem ser definidas através das funções `degrees()` e `radians()`.) A orientação do ângulo depende do modo tartaruga, veja `mode()`.

```
>>> turtle.heading()
22.0
>>> turtle.left(45)
>>> turtle.heading()
67.0
```

`turtle.`**`goto`**(*x*, *y=None*)

`turtle.`**`setpos`**(*x*, *y=None*)

`turtle.`**`setposition`**(*x*, *y=None*)

Parâmetros

- **x** – um número ou um par/vetor de números
- **y** – um número ou `None`

Se *y* for `None`, *x* deve ser um par de coordenadas ou uma classe `Vec2D` (por exemplo, como retornado pela função `pos()`).

Move a tartaruga para uma posição absoluta. Caso a caneta esteja virada para baixo, traça a linha. Não altera a orientação da tartaruga.

```
>>> tp = turtle.pos()
>>> tp
(0.00,0.00)
>>> turtle.setpos(60,30)
>>> turtle.pos()
(60.00,30.00)
>>> turtle.setpos((20,80))
>>> turtle.pos()
(20.00,80.00)
>>> turtle.setpos(tp)
>>> turtle.pos()
(0.00,0.00)
```

`turtle.`**`teleport`**(*x*, *y=None*, *, *fill_gap=False*)

Parâmetros

- **x** – um número ou `None`
- **y** – um número ou `None`
- **fill_gap** – um valor booleano

Move a tartaruga para uma posição absoluta. Ao contrário de goto(x, y), uma linha não será desenhada. A orientação da tartaruga não muda. Se estiver sendo preenchido no momento, o(s) polígono(s) teletransportado(s) será(ão) preenchido(s) após sair, e o preenchimento começará novamente após o teletransporte. Isso pode ser desabilitado com fill_gap=True, o que faz com que a linha imaginária percorrida durante o teletransporte atue como uma barreira de preenchimento como em goto(x, y).

```
>>> tp = turtle.pos()
>>> tp
(0.00,0.00)
>>> turtle.teleport(60)
>>> turtle.pos()
(60.00,0.00)
>>> turtle.teleport(y=10)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> turtle.pos()
(60.00,10.00)
>>> turtle.teleport(20, 30)
>>> turtle.pos()
(20.00,30.00)
```

Adicionado na versão 3.12.

turtle.**setx**(*x*)

Parâmetros
x – um número (inteiro ou ponto flutuante)

Define a primeira coordenada da tartaruga para *x*, deixa a segunda coordenada inalterada.

```
>>> turtle.position()
(0.00,240.00)
>>> turtle.setx(10)
>>> turtle.position()
(10.00,240.00)
```

turtle.**sety**(*y*)

Parâmetros
y – um número (inteiro ou ponto flutuante)

Defina a segunda coordenada da tartaruga para *y*, deixa a primeira coordenada inalterada.

```
>>> turtle.position()
(0.00,40.00)
>>> turtle.sety(-10)
>>> turtle.position()
(0.00,-10.00)
```

turtle.**setheading**(*to_angle*)

turtle.**seth**(*to_angle*)

Parâmetros
to_angle – um número (inteiro ou ponto flutuante)

Define a orientação da tartaruga para *to_angle*. Aqui estão algumas direções mais comuns em graus:

modo padrão	modo logo
0 - leste	0 - norte
90 - norte	90 - leste
180 - oeste	180 - sul
270 - sul	270 - oeste

```
>>> turtle.setheading(90)
>>> turtle.heading()
90.0
```

turtle.**home**()

Move a tartaruga para a origem – coordenadas (0,0) – e define seu rumo para sua orientação inicial (que depende do modo, veja *mode()*).

```
>>> turtle.heading()
90.0
>>> turtle.position()
(0.00,-10.00)
>>> turtle.home()
>>> turtle.position()
(0.00,0.00)
>>> turtle.heading()
0.0
```

`turtle.`**`circle`**(*radius*, *extent=None*, *steps=None*)

Parâmetros

- **`radius`** – um número
- **`extent`** – um número (ou `None`)
- **`steps`** – um inteiro (ou `None`)

Desenha um círculo com dado *radius*. O centro são as unidades de *radius* à esquerda da tartaruga; *extent* – um ângulo – determina qual parte do círculo é desenhada. Se *extent* não for fornecida, desenha o círculo inteiro. Se *extent* não for um círculo completo, uma extremidade do arco será a posição atual da caneta. Desenha o arco no sentido anti-horário se *radius* for positivo, caso contrário, no sentido horário. Finalmente, a direção da tartaruga é alterada pela quantidade de *extent*.

Como o círculo é aproximado por um polígono regular inscrito, *steps* determina o número de passos a serem usados. Caso não seja informado, será calculado automaticamente. Pode ser usado para desenhar polígonos regulares.

```
>>> turtle.home()
>>> turtle.position()
(0.00,0.00)
>>> turtle.heading()
0.0
>>> turtle.circle(50)
>>> turtle.position()
(-0.00,0.00)
>>> turtle.heading()
0.0
>>> turtle.circle(120, 180)  # draw a semicircle
>>> turtle.position()
(0.00,240.00)
>>> turtle.heading()
180.0
```

`turtle.`**`dot`**(*size=None*, **color*)

Parâmetros

- **`size`** – um inteiro >= 1 (caso seja fornecido)
- **`color`** – uma string de cores ou uma tupla de cores numéricas

Desenha um ponto circular com diâmetro *size*, usando *color*. Se *size* não for fornecido, o máximo de pensize+4 e 2*pensize será usado.

```
>>> turtle.home()
>>> turtle.dot()
>>> turtle.fd(50); turtle.dot(20, "blue"); turtle.fd(50)
>>> turtle.position()
(100.00,-0.00)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> turtle.heading()
0.0
```

turtle.**stamp**()

Carimba uma cópia da forma da tartaruga na tela na posição atual da tartaruga. Retorna um stamp_id para esse carimbo, que pode ser usado para excluí-lo chamando `clearstamp(stamp_id)`.

```
>>> turtle.color("blue")
>>> stamp_id = turtle.stamp()
>>> turtle.fd(50)
```

turtle.**clearstamp**(*stampid*)

Parâmetros
stampid – um inteiro, deve ser o valor de retorno da chamada de *stamp()* anterior

Exclui o carimbo com o *stamp* fornecido.

```
>>> turtle.position()
(150.00,-0.00)
>>> turtle.color("blue")
>>> astamp = turtle.stamp()
>>> turtle.fd(50)
>>> turtle.position()
(200.00,-0.00)
>>> turtle.clearstamp(astamp)
>>> turtle.position()
(200.00,-0.00)
```

turtle.**clearstamps**(*n=None*)

Parâmetros
n – um inteiro (ou `None`)

Exclui todos ou o primeiro/último *n* dos selos da tartaruga. Se *n* for `None`, exclui todos os carimbos, se $n > 0$ exclui os primeiros *n* carimbos, senão se $n < 0$ exclui os últimos *n* carimbos.

```
>>> for i in range(8):
...     unused_stamp_id = turtle.stamp()
...     turtle.fd(30)
>>> turtle.clearstamps(2)
>>> turtle.clearstamps(-2)
>>> turtle.clearstamps()
```

turtle.**undo**()

Desfaz (repetidamente) a(s) última(s) ação(ões) da tartaruga. O número de ações de desfazer disponíveis é determinado pelo tamanho do buffer de desfazer.

```
>>> for i in range(4):
...     turtle.fd(50); turtle.lt(80)
...
>>> for i in range(8):
...     turtle.undo()
```

turtle.**speed**(*speed=None*)

Parâmetros
speed – um inteiro no intervalo 0..10 ou uma string de velocidade (veja abaixo)

Define a velocidade da tartaruga para um valor inteiro no intervalo 0..10. Se nenhum argumento for fornecido, retorna a velocidade atual.

Se a entrada for um número maior que 10 ou menor que 0,5, a velocidade é definida como 0. As strings de velocidade são mapeadas para valores de velocidade da seguinte forma:

- "fastest": 0
- "fast": 10
- "normal": 6
- "slow": 3
- "slowest": 1

Velocidades de 1 a 10 tornam a animação cada vez mais rápida, tanto para o desenho da linha como para a rotação da tartaruga.

Atenção: *speed* = 0 significa que *nenhuma* animação ocorre. forward/back faz a tartaruga pular e da mesma forma para left/right faz a tartaruga girar instantaneamente.

```
>>> turtle.speed()
3
>>> turtle.speed('normal')
>>> turtle.speed()
6
>>> turtle.speed(9)
>>> turtle.speed()
9
```

Fala o estado de Turtle

`turtle.`**`position`**`()`

`turtle.`**`pos`**`()`

Retorna a localização atual da tartaruga (x,y) (como um vetor *Vec2D*).

```
>>> turtle.pos()
(440.00,-0.00)
```

`turtle.`**`towards`**`(`*x*, *y=None*`)`

Parâmetros

- **x** – um número ou um par/vetor de números ou uma instância de tartaruga
- **y** – um número caso *x* seja um número, senão `None`

Retorna o ângulo entre a linha da posição da tartaruga para a posição especificada por (x,y), o vetor ou a outra tartaruga. Isso depende da orientação inicial da tartaruga, que depende do modo - "standard"/"world" ou "logo".

```
>>> turtle.goto(10, 10)
>>> turtle.towards(0,0)
225.0
```

`turtle.`**`xcor`**`()`

Retorna a coordenada X da tartaruga.

```
>>> turtle.home()
>>> turtle.left(50)
>>> turtle.forward(100)
>>> turtle.pos()
```

(continua na próxima página)

(continuação da página anterior)

```
(64.28,76.60)
>>> print(round(turtle.xcor(), 5))
64.27876
```

turtle.**ycor**()

Retorna a coordenada Y da tartaruga.

```
>>> turtle.home()
>>> turtle.left(60)
>>> turtle.forward(100)
>>> print(turtle.pos())
(50.00,86.60)
>>> print(round(turtle.ycor(), 5))
86.60254
```

turtle.**heading**()

Retorna o título atual da tartaruga (o valor depende do modo da tartaruga, veja *mode()*).

```
>>> turtle.home()
>>> turtle.left(67)
>>> turtle.heading()
67.0
```

turtle.**distance**(*x*, *y=None*)

Parâmetros

- **x** – um número ou um par/vetor de números ou uma instância de tartaruga
- **y** – um número caso *x* seja um número, senão `None`

Retorna a distância da tartaruga para (x,y), o vetor dado, ou a outra tartaruga dada, em unidades de passo de tartaruga.

```
>>> turtle.home()
>>> turtle.distance(30,40)
50.0
>>> turtle.distance((30,40))
50.0
>>> joe = Turtle()
>>> joe.forward(77)
>>> turtle.distance(joe)
77.0
```

Configurações de medida

turtle.**degrees**(*fullcircle=360.0*)

Parâmetros

fullcircle – um número

Define as unidades de medição do ângulo, ou seja, defina o número de "graus" para um círculo completo. O valor padrão é 360 graus.

```
>>> turtle.home()
>>> turtle.left(90)
>>> turtle.heading()
90.0
```

(continua na próxima página)

(continuação da página anterior)

```
>>> # Alterar unidade de medida de ângulo para grad (também
>>> # conhecido como gon, grau ou grado e é igual a 1/100
>>> # do ângulo reto).
>>> turtle.degrees(400.0)
>>> turtle.heading()
100.0
>>> turtle.degrees(360)
>>> turtle.heading()
90.0
```

turtle.**radians**()

Define as unidades de medida de ângulo para radianos. Equivalente a `degrees(2*math.pi)`.

```
>>> turtle.home()
>>> turtle.left(90)
>>> turtle.heading()
90.0
>>> turtle.radians()
>>> turtle.heading()
1.5707963267948966
```

Controle da Caneta

Estado do Desenho

turtle.**pendown**()

turtle.**pd**()

turtle.**down**()

Desce a caneta - desenha ao se mover.

turtle.**penup**()

turtle.**pu**()

turtle.**up**()

Levanta a caneta – sem qualquer desenho ao se mover.

turtle.**pensize**(*width=None*)

turtle.**width**(*width=None*)

Parâmetros

width – um número positivo

Define a espessura da linha para *width* ou retorna-a. Se resizemode estiver definido como "auto" e a forma de tartaruga for um polígono, esse polígono será desenhado com a mesma espessura de linha. Se nenhum argumento for fornecido, a espessura atual da caneta será retornada.

```
>>> turtle.pensize()
1
>>> turtle.pensize(10)   # from here on lines of width 10 are drawn
```

turtle.**pen**(*pen=None*, ***pendict*)

Parâmetros

- **pen** – um dicionário com algumas ou todas as chaves listadas abaixo
- **pendict** – um ou mais argumentos nomeados com as chaves listadas abaixo como palavras-chave

Retorna ou define os atributos da caneta em um "dicionário da caneta" com os seguintes pares de chave/valor:

- “shown”: True/False
- “pendown”: True/False
- “pencolor”: string de cores ou tupla de cores
- “fillcolor”: string de cores ou tupla de cores
- “pensize”: número positivo
- “speed”: número na faixa de 0..10.
- “resizemode”: “auto”, “user” ou “noresize”
- “stretchfactor”: (número positivo, número positivo)
- “outline”: número positivo
- “tilt”: número

Este dicionário pode ser usado como argumento para uma chamada subsequente para *pen()* para restaurar o estado da caneta anterior. Além disso, um ou mais desses atributos podem ser fornecidos como argumentos nomeados. Isso pode ser usado para definir vários atributos de caneta em uma instrução.

```
>>> turtle.pen(fillcolor="black", pencolor="red", pensize=10)
>>> sorted(turtle.pen().items())
[('fillcolor', 'black'), ('outline', 1), ('pencolor', 'red'),
 ('pendown', True), ('pensize', 10), ('resizemode', 'noresize'),
 ('shearfactor', 0.0), ('shown', True), ('speed', 9),
 ('stretchfactor', (1.0, 1.0)), ('tilt', 0.0)]
>>> penstate=turtle.pen()
>>> turtle.color("yellow", "")
>>> turtle.penup()
>>> sorted(turtle.pen().items())[:3]
[('fillcolor', ''), ('outline', 1), ('pencolor', 'yellow')]
>>> turtle.pen(penstate, fillcolor="green")
>>> sorted(turtle.pen().items())[:3]
[('fillcolor', 'green'), ('outline', 1), ('pencolor', 'red')]
```

turtle.**isdown**()

Retorna `True` se a caneta estiver abaixada, `False` se estiver levantada.

```
>>> turtle.penup()
>>> turtle.isdown()
False
>>> turtle.pendown()
>>> turtle.isdown()
True
```

Controle da Cor

turtle.**pencolor**(**args*)

Retorna ou define a cor da caneta ou pencolor.

São permitidos quatro formatos de entrada:

pencolor()
Retorna a cor da caneta atual como string de especificação de cor ou como uma tupla (veja o exemplo). Pode ser usado como entrada para outra chamada color/pencolor/fillcolor.

pencolor(colorstring)
Define pencolor como *colorstring*, que é uma string de especificação de cor Tk, como `"red"`, `"yellow"` ou `"#33cc8c"`.

pencolor((r, g, b))
Define a cor da caneta como a cor RGB representada pela tupla *r*, *g*, e *b*. Os valores de *r*, *g*, and *b* precisam estar na faixa 0..colormode, onde colormode é 1.0 ou 255 (ver *colormode()*).

pencolor(r, g, b)
Define a cor da caneta como a cor RGB representada por *r*, *g*, e *b*. Os valores de *r*, *g*, and *b* precisam estar na faixa 0..colormode.

Se a forma da tartaruga for um polígono, o contorno desse polígono será desenhado com a nova cor de caneta definida.

```
>>> colormode()
1.0
>>> turtle.pencolor()
'red'
>>> turtle.pencolor("brown")
>>> turtle.pencolor()
'brown'
>>> tup = (0.2, 0.8, 0.55)
>>> turtle.pencolor(tup)
>>> turtle.pencolor()
(0.2, 0.8, 0.5490196078431373)
>>> colormode(255)
>>> turtle.pencolor()
(51.0, 204.0, 140.0)
>>> turtle.pencolor('#32c18f')
>>> turtle.pencolor()
(50.0, 193.0, 143.0)
```

turtle.**fillcolor**(**args*)
Retorna ou define o fillcolor.

São permitidos quatro formatos de entrada:

fillcolor()
Retorna a cor de preenchimento atual como string, ou em formato de tupla (veja o exemplo). Pode ser usado como entrada para outra chamada color/pencolor/fillcolor.

fillcolor(colorstring)
Define fillcolor como *colorstring*, que é uma especificação de cor do Tk em formato de string, como `"red"`, `"yellow"`, ou `"#33cc8c"`.

fillcolor((r, g, b))
Define fillcolor como a cor no padrão RGB representada por tupla de *r*, *g* e *b*. Cada um de *r*, *g* e *b* deve estar no limite 0..colormode, em que colormode é 1,0 ou 255 (consulte *colormode()*).

fillcolor(r, g, b)
Define fillcolor como uma cor no formato RGB representada por *r*, *g* e *b*. Cada um dos elementos *r*, *g* e *b* devem estar no limite 0..colormode.

Se turtleshape for um polígono, o interior desse polígono será desenhado com a nova cor de preenchimento definida.

```
>>> turtle.fillcolor("violet")
>>> turtle.fillcolor()
'violet'
>>> turtle.pencolor()
(50.0, 193.0, 143.0)
>>> turtle.fillcolor((50, 193, 143))  # Inteiros, não pontos flutuantes
>>> turtle.fillcolor()
(50.0, 193.0, 143.0)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> turtle.fillcolor('#ffffff')
>>> turtle.fillcolor()
(255.0, 255.0, 255.0)
```

turtle.**color**(**args*)

Retorna ou define os valores de pencolor e fillcolor.

Diversos formatos de entrada são permitidos. Eles usam de 0 a 3 argumentos da seguinte forma:

color()
: Retorna os valores atuais de pencolor e fillcollor como um par de cores como strings ou tuplas conforme retornado por *pencolor()* e *fillcolor()*.

color(colorstring), color((r,g,b)), color(r,g,b)
: Entradas como em *pencolor()*, define ambos, fillcolor e pencolor, para o valor informado.

color(colorstring1, colorstring2), color((r1,g1,b1), (r2,g2,b2))
: Equivalente a `pencolor(colorstring1)` e `fillcolor(colorstring2)` e de forma análoga se a outra formatação de entrada for usada.

Se o turtleshape for um polígono, o contorno e o interior desse polígono serão desenhados com as cores recém-definidas.

```
>>> turtle.color("red", "green")
>>> turtle.color()
('red', 'green')
>>> color("#285078", "#a0c8f0")
>>> color()
((40.0, 80.0, 120.0), (160.0, 200.0, 240.0))
```

Veja também: Método da tela *colormode()*.

Preenchimento

turtle.**filling**()

Retorna fillstate (`True` se estiver preenchido, `False` caso contrário).

```
>>> turtle.begin_fill()
>>> if turtle.filling():
...     turtle.pensize(5)
... else:
...     turtle.pensize(3)
```

turtle.**fill**()

Preenche a forma desenhada no bloco `with turtle.fill():`.

```
>>> turtle.color("black", "red")
>>> with turtle.fill():
...     turtle.circle(80)
```

Usar `fill()` equivale a adicionar *begin_fill()* antes do bloco de preenchimento e *end_fill()* depois do bloco de preenchimento:

```
>>> turtle.color("black", "red")
>>> turtle.begin_fill()
>>> turtle.circle(80)
>>> turtle.end_fill()
```

Adicionado na versão 3.14.

turtle.**begin_fill**()

Deve ser chamado antes de desenhar uma forma a ser preenchida.

turtle.**end_fill**()

Preenche a forma desenhada após a última chamada para *begin_fill()*.

O preenchimento ou não de regiões sobrepostas para polígonos que se cruzam ou formas múltiplas depende dos gráficos do sistema operacional, do tipo de sobreposição e do número de sobreposições. Por exemplo, a estrela da Tartaruga acima pode ser toda amarela ou ter algumas regiões brancas.

```
>>> turtle.color("black", "red")
>>> turtle.begin_fill()
>>> turtle.circle(80)
>>> turtle.end_fill()
```

Mais sobre o Controle do Desenho

turtle.**reset**()

Remove os desenhos da tartaruga da tela, centraliza novamente a tartaruga e define as variáveis para os valores padrões.

```
>>> turtle.goto(0,-22)
>>> turtle.left(100)
>>> turtle.position()
(0.00,-22.00)
>>> turtle.heading()
100.0
>>> turtle.reset()
>>> turtle.position()
(0.00,0.00)
>>> turtle.heading()
0.0
```

turtle.**clear**()

Exclui os desenhos da tartaruga da tela e não move a tartaruga. O atual estado e posição da tartaruga, bem como os desenhos de outras tartarugas, não são afetados.

turtle.**write**(*arg*, *move=False*, *align='left'*, *font=('Arial', 8, 'normal')*)

Parâmetros

- **arg** – objeto a ser escrito na TurtleScreen
- **move** – True/False
- **align** – uma das Strings "left", "center" ou right"
- **font** – a triple (fontname, fontsize, fonttype)

Escreve o texto - a representação string de *arg* - na posição atual da tartaruga de acordo com *align* ("left", "center" ou "right") e com a fonte fornecida. Se *move* for verdadeiro, a caneta será movida para o canto inferior direito do texto. Por padrão, *move* é `False`.

```
>>> turtle.write("Home = ", True, align="center")
>>> turtle.write((0,0), True)
```

Estado da tartaruga

Visibilidade

turtle.**hideturtle**()

turtle.**ht**()

Torna a tartaruga invisível. É uma boa ideia fazer isso enquanto estiver fazendo algum desenho complexo, pois ocultar a tartaruga acelera o desenho de forma visível.

```
>>> turtle.hideturtle()
```

turtle.**showturtle**()

turtle.**st**()

Tornar a tartaruga visível.

```
>>> turtle.showturtle()
```

turtle.**isvisible**()

Retorna `True` se a Tartaruga for exibida, `False` se estiver oculta.

```
>>> turtle.hideturtle()
>>> turtle.isvisible()
False
>>> turtle.showturtle()
>>> turtle.isvisible()
True
```

Aparência

turtle.**shape**(*name=None*)

Parâmetros
name – uma string que é um shapename válido

Define a forma da tartaruga para a forma com o *nome* fornecido ou, se o nome não for fornecido, retorna nome da forma atual. A forma com *name* deve existir no dicionário TurtleScreen. Inicialmente, há as seguintes formas de polígono: "arrow" (seta), "turtle" (tartaruga), "circle" (círculo), "square" (quadrado), "triangle" (triângulo), "classic" (clássico). Para saber mais sobre como lidar com formas, consulte o método Screen *register_shape()* .

```
>>> turtle.shape()
'classic'
>>> turtle.shape("turtle")
>>> turtle.shape()
'turtle'
```

turtle.**resizemode**(*rmode=None*)

Parâmetros
rmode – uma das Strings "auto", "user", "noresize"

Define resizemode como um dos valores: "auto", "user", "noresize". Se *rmode* não for fornecido, retorna o modo de redimensionamento atual. Os diferentes modos de redimensionamento têm os seguintes efeitos:

- "auto": adapta a aparência da tartaruga correspondente ao valor do pensize.
- "user": adapta a aparência da tartaruga de acordo com os valores de stretchfactor e outlinewidth (outline), que são definidos por *shapesize()*.
- "noresize": não acontece nenhuma adaptação na aparência da tartaruga.

`resizemode("user")` é chamado por *shapesize()* quando usado com argumentos.

```
>>> turtle.resizemode()
'noresize'
```

(continua na próxima página)

(continuação da página anterior)

```
>>> turtle.resizemode("auto")
>>> turtle.resizemode()
'auto'
```

turtle.**shapesize**(*stretch_wid=None*, *stretch_len=None*, *outline=None*)

turtle.**turtlesize**(*stretch_wid=None*, *stretch_len=None*, *outline=None*)

Parâmetros

- **stretch_wid** – número positivo
- **stretch_len** – número positivo
- **outline** – número positivo

Retorna ou define os atributos x/y-stretchfactors e/ou contorno da caneta. Define o modo de redimensionamento como "usuário". Se e somente se resizemode estiver definido como "user", a tartaruga será exibida esticada de acordo com seus stretchfactors: *stretch_wid* é stretchfactor perpendicular à sua orientação, *stretch_len* é stretchfactor na direção de sua orientação, *outline* determina a largura do contorno da forma.

```
>>> turtle.shapesize()
(1.0, 1.0, 1)
>>> turtle.resizemode("user")
>>> turtle.shapesize(5, 5, 12)
>>> turtle.shapesize()
(5, 5, 12)
>>> turtle.shapesize(outline=8)
>>> turtle.shapesize()
(5, 5, 8)
```

turtle.**shearfactor**(*shear=None*)

Parâmetros

shear – número (opcional)

Define ou retorna o fator de shearfactor atual. Corta a forma de tartaruga de acordo com o fator de shearfactor fornecido, que é a tangente do ângulo de shearfactor. *Não* muda a direção da tartaruga (direção do movimento). Se o shearfactor não for fornecido: retorna o fator de shearfactor atual, i. Isso é. a tangente do ângulo de shearfactor, pelo qual as linhas paralelas à direção da tartaruga são cortadas.

```
>>> turtle.shape("circle")
>>> turtle.shapesize(5,2)
>>> turtle.shearfactor(0.5)
>>> turtle.shearfactor()
0.5
```

turtle.**tilt**(*angle*)

Parâmetros

angle – um número

Gira a forma de tartaruga em um *ângulo* a partir de seu ângulo de inclinação atual, mas *não* altera o rumo da tartaruga (direção do movimento).

```
>>> turtle.reset()
>>> turtle.shape("circle")
>>> turtle.shapesize(5,2)
>>> turtle.tilt(30)
>>> turtle.fd(50)
>>> turtle.tilt(30)
>>> turtle.fd(50)
```

turtle.**tiltangle**(*angle=None*)

Parâmetros
angle – um número (opcional)

Define ou retorna o ângulo de inclinação atual. Se o ângulo for fornecido, gira a forma de tartaruga para apontar na direção especificada pelo ângulo, independentemente do seu ângulo de inclinação atual. *Não* muda a direção da tartaruga (direção do movimento). Se o ângulo não for fornecido: retorna o ângulo de inclinação atual, que é o ângulo entre a orientação da forma da tartaruga e a direção da tartaruga (sua direção de movimento).

```
>>> turtle.reset()
>>> turtle.shape("circle")
>>> turtle.shapesize(5,2)
>>> turtle.tilt(45)
>>> turtle.tiltangle()
45.0
```

turtle.**shapetransform**(*t11=None*, *t12=None*, *t21=None*, *t22=None*)

Parâmetros

- **t11** – um número (opcional)
- **t12** – um número (opcional)
- **t21** – um número (opcional)
- **t12** – um número (opcional)

Define ou retorna a matriz de transformação atual da forma da tartaruga.

Se nenhum dos elementos da matriz for fornecido, retorna a matriz de transformação como uma tupla de 4 elementos. Caso contrário, define os elementos fornecidos e transforma a forma da tartaruga de acordo com a matriz que consiste na primeira linha t11, t12 e na segunda linha t21, t22. O resultado de t11 * t22 - t12 * t21 não deve ser zero, caso contrário será gerado um erro. Modifica stretchfactor, shearfactor e tiltangle de acordo com a matriz fornecida.

```
>>> turtle = Turtle()
>>> turtle.shape("square")
>>> turtle.shapesize(4,2)
>>> turtle.shearfactor(-0.5)
>>> turtle.shapetransform()
(4.0, -1.0, -0.0, 2.0)
```

turtle.**get_shapepoly**()

Retorna o polígono da forma atual como uma tupla de pares de coordenadas. Isto pode ser usado para definir uma nova forma ou componentes de uma forma composta.

```
>>> turtle.shape("square")
>>> turtle.shapetransform(4, -1, 0, 2)
>>> turtle.get_shapepoly()
((50, -20), (30, 20), (-50, 20), (-30, -20))
```

Eventos Utilizados

turtle.**onclick**(*fun*, *btn=1*, *add=None*)

Parâmetros

- **fun** – um função com dois argumento que serão chamados com as coordenadas do ponto clicado na tela
- **btn** – número do botão do mouse, o padrão é 1 (botão esquerdo do mouse)

- **add** – `True` ou `False` – se `True`, uma nova ligação será adicionada; caso contrário, ela substituirá uma ligação antiga

Vincula *fun* a eventos de clique do mouse nessa tartaruga. Se *fun* for `None`, as associações existentes serão removidas. Exemplo para a tartaruga anônima, ou seja, a forma processual:

```
>>> def turn(x, y):
...     left(180)
...
>>> onclick(turn)  # Agora, clicar na tartaruga irá girá-la.
>>> onclick(None)  # a associação de eventos será removida
```

`turtle.`**`onrelease`**(*fun*, *btn=1*, *add=None*)

Parâmetros

- **fun** – um função com dois argumento que serão chamados com as coordenadas do ponto clicado na tela
- **btn** – número do botão do mouse, o padrão é 1 (botão esquerdo do mouse)
- **add** – `True` ou `False` – se `True`, uma nova ligação será adicionada; caso contrário, ela substituirá uma ligação antiga

Vincule *fun* aos eventos de liberação do botão do mouse nesta tartaruga. Se *fun* for `None`, as associações existentes serão removidas.

```
>>> class MyTurtle(Turtle):
...     def glow(self,x,y):
...         self.fillcolor("red")
...     def unglow(self,x,y):
...         self.fillcolor("")
...
>>> turtle = MyTurtle()
>>> turtle.onclick(turtle.glow)      # clicar na tartaruga torna o␣
↪preenchimento vermelho,
>>> turtle.onrelease(turtle.unglow) # .liberar a transforma em transparente
```

`turtle.`**`ondrag`**(*fun*, *btn=1*, *add=None*)

Parâmetros

- **fun** – um função com dois argumento que serão chamados com as coordenadas do ponto clicado na tela
- **btn** – número do botão do mouse, o padrão é 1 (botão esquerdo do mouse)
- **add** – `True` ou `False` – se `True`, uma nova ligação será adicionada; caso contrário, ela substituirá uma ligação antiga

Vincule *fun* a eventos de movimentação do mouse nessa tartaruga. Se *fun* for `None`, as associações existentes serão removidas.

Observação: Cada sequência de eventos de movimento do mouse em uma tartaruga é precedida por um evento de clique do mouse naquela tartaruga.

```
>>> turtle.ondrag(turtle.goto)
```

Posteriormente, clicar e arrastar a Tartaruga a moverá pela tela, produzindo desenhos à mão (se a caneta estiver deitada).

Métodos Especiais da Tartaruga

turtle.**poly**()

Registra os vértices de um polígono desenhado no bloco `with turtle.poly():`. O primeiro e o último vértices serão conectados.

```
>>> with turtle.poly():
...     turtle.forward(100)
...     turtle.right(60)
...     turtle.forward(100)
```

Adicionado na versão 3.14.

turtle.**begin_poly**()

Começa a registrar os vértices de um polígono. A posição atual da tartaruga é o primeiro vértice do polígono.

turtle.**end_poly**()

Para de registrar os vértices de um polígono. A posição atual da tartaruga é o último vértice do polígono. Isto será conectado ao primeiro vértice.

turtle.**get_poly**()

Retorna o último polígono registrado.

```
>>> turtle.home()
>>> turtle.begin_poly()
>>> turtle.fd(100)
>>> turtle.left(20)
>>> turtle.fd(30)
>>> turtle.left(60)
>>> turtle.fd(50)
>>> turtle.end_poly()
>>> p = turtle.get_poly()
>>> register_shape("myFavouriteShape", p)
```

turtle.**clone**()

Cria e retorna um clone da tartaruga com a mesma posição, direção e propriedades da tartaruga original.

```
>>> mick = Turtle()
>>> joe = mick.clone()
```

turtle.**getturtle**()

turtle.**getpen**()

Retorna o próprio objeto Turtle. Uso recomendado: como função para retornar a "tartaruga anônima":

```
>>> pet = getturtle()
>>> pet.fd(50)
>>> pet
<turtle.Turtle object at 0x...>
```

turtle.**getscreen**()

Retorna o objeto *TurtleScreen* no qual a tartaruga está sendo desenhada. Os métodos de TurtleScreen podem então ser chamados para este objeto.

```
>>> ts = turtle.getscreen()
>>> ts
<turtle._Screen object at 0x...>
>>> ts.bgcolor("pink")
```

turtle.**setundobuffer**(*size*)

Parâmetros
size – um inteiro ou None

Define ou desativa o undobuffer. Se *size* for um inteiro, um undobuffer vazio de determinado tamanho será instalado. *size* fornece o número máximo de ações da tartaruga que podem ser desfeitas pelo método/função *undo()*. Se *size* for None, o undobuffer será desativado.

```
>>> turtle.setundobuffer(42)
```

turtle.**undobufferentries**()

Retorna o número de entradas no undobuffer.

```
>>> while undobufferentries():
...     undo()
```

Formas compostas

Para usar formas de tartaruga compostas, que consistem em vários polígonos de cores diferentes, você deve usar a classe auxiliar *Shape* explicitamente, conforme descrito abaixo:

1. Crie um objeto Shape vazio do tipo "compound".
2. Adicione quantos componentes desejar a esse objeto, usando o método *addcomponent()*.

   Por exemplo:

```
>>> s = Shape("compound")
>>> poly1 = ((0,0),(10,-5),(0,10),(-10,-5))
>>> s.addcomponent(poly1, "red", "blue")
>>> poly2 = ((0,0),(10,-5),(-10,-5))
>>> s.addcomponent(poly2, "blue", "red")
```

3. Agora, adicione o Shape à lista de formas da tela e use-o:

```
>>> register_shape("myshape", s)
>>> shape("myshape")
```

Nota

A classe *Shape* é usada internamente pelo método *register_shape()* de diferentes maneiras. O programador da aplicação precisa lidar com a classe Shape *somente* ao usar formas compostas como as mostradas acima!

25.10.7 métodos do TurtleScreen/Screen e as funções correspondentes

A maioria dos exemplos desta seção se refere a uma instância de TurtleScreen chamada screen.

Controle da Janela

turtle.**bgcolor**(**args*)

Parâmetros
args – uma sequência de cores ou três números no intervalo 0..colormode ou uma tupla de 3 elementos desses números

Define ou retorna a cor de fundo do TurtleScreen.

```
>>> screen.bgcolor("orange")
>>> screen.bgcolor()
'orange'
>>> screen.bgcolor("#800080")
>>> screen.bgcolor()
(128.0, 0.0, 128.0)
```

turtle.**bgpic**(*picname=None*)

Parâmetros

picname – um string, um nome de um arquivo de imagem (PNG, GIF, PGM e PPM) ou `"nopic"`, ou `None`

Configura a imagem de fundo ou retorna nome da imagem de fundo atual. Se *picname* for um nome de arquivo, definirá a imagem correspondente como plano de fundo. Se *picname* for `"nopic"`, exclui a imagem de fundo, caso haja. Se *picname* for `None`, retorna o nome do arquivo da imagem de fundo atual:

```
>>> screen.bgpic()
'nopic'
>>> screen.bgpic("landscape.gif")
>>> screen.bgpic()
"landscape.gif"
```

turtle.**clear**()

Nota

Este método de TurtleScreen está disponível como uma função global somente sob o nome `clearscreen`. A função global `clear` é derivada de forma diferente do método de Turtle `clear` .

turtle.**clearscreen**()

Remove todos os desenhos e todas as tartarugas da TurtleScreen. Redefine a TurtleScreen, agora vazia, para seu estado inicial: fundo branco, sem imagem de fundo, sem vínculos de eventos e rastreamento ativado.

turtle.**reset**()

Nota

Este método de TurtleScreen está disponível como uma função global somente com o nome `resetscreen`. A função global `reset` é outra derivada do método de Turtle `reset` .

turtle.**resetscreen**()

Reinicia todas as Turtles em Screen para seu estado inicial.

turtle.**screensize**(*canvwidth=None*, *canvheight=None*, *bg=None*)

Parâmetros

- **canvwidth** – positivo inteiro, a nova largura da tela em pixels
- **canvheight** – positivo inteiro, a nova altura da tela em pixels
- **bg** – Uma string com a cor ou uma tupla, a nova cor de fundo

Se nenhum argumento for fornecido, retorna o atual (canvaswidth, canvasheight). Caso contrário, redimensiona a tela em que as tartarugas estão desenhadas. Não altera a janela de desenho. Para observar as partes ocultas da tela, use as barras de rolagem. Com este método, é possível tornar visíveis as partes de um desenho que antes estavam fora da tela.

```
>>> screen.screensize()
(400, 300)
>>> screen.screensize(2000,1500)
>>> screen.screensize()
(2000, 1500)
```

Por exemplo, para procurar uma tartaruga que escapou por engano ;-)

turtle.**setworldcoordinates**(*llx*, *lly*, *urx*, *ury*)

Parâmetros

- **llx** – um número representando a coordenada x do canto inferior esquerdo da tela
- **lly** – um número representando a coordenada y do canto inferior esquerdo da tela
- **urx** – um número representando a coordenada x do canto superior direito da tela
- **ury** – um número representando a coordenada y do canto superior direito da tela

Configura o sistema de coordenadas definido pelo usuário e muda para o modo "world", se necessário. Isso executa um `screen.reset()`. Se o modo "world" já estiver ativo, todos os desenhos atuais serão redesenhados de acordo com as novas coordenadas.

ATENÇÃO: em sistemas de coordenadas definidos pelo usuário, os ângulos podem aparecer distorcidos.

```
>>> screen.reset()
>>> screen.setworldcoordinates(-50,-7.5,50,7.5)
>>> for _ in range(72):
...     left(10)
...
>>> for _ in range(8):
...     left(45); fd(2)   # a regular octagon
```

Controle da animação

turtle.**no_animation**()

Desabilita temporariamente a animação da tartaruga. O código escrito dentro do bloco `no_animation` não será animado; assim que o bloco de código for fechado, o desenho aparecerá.

```
>>> with screen.no_animation():
...     for dist in range(2, 400, 2):
...         fd(dist)
...         rt(90)
```

Adicionado na versão 3.14.

turtle.**delay**(*delay=None*)

Parâmetros

delay – um número inteiro positivo

Define ou retorna o *delay* do desenho em milissegundos. (Esse é aproximadamente o intervalo de tempo entre duas atualizações consecutivas da tela). Quanto maior o atraso do desenho, mais lenta será a animação.

Argumentos opcionais:

```
>>> screen.delay()
10
>>> screen.delay(5)
>>> screen.delay()
5
```

turtle.**tracer**(*n=None*, *delay=None*)

Parâmetros

- **n** – inteiro não-negativo
- **delay** – inteiro não-negativo

Ative ou desative a animação da tartaruga e defina o atraso para atualização dos desenhos. Se *n* for fornecido, apenas cada n-ésima atualização regular da tela será realmente executada. (Pode ser usado para acelerar o desenho de gráficos complexos.) Quando chamado sem argumentos, retorna o valor atualmente armazenado de n. O segundo argumento define o valor do atraso (consulte *delay()*).

```
>>> screen.tracer(8, 25)
>>> dist = 2
>>> for i in range(200):
...     fd(dist)
...     rt(90)
...     dist += 2
```

turtle.**update**()

Executa uma atualização do TurtleScreen. Para ser usado quando o rastreador estiver desligado.

Veja também o método RawTurtle/Turtle *speed()*.

Usando os eventos de tela

turtle.**listen**(*xdummy=None*, *ydummy=None*)

Define o foco no TurtleScreen (para coletar eventos-chave). Argumentos fictícios são fornecidos para que seja possível passar *listen()* para o método onclick.

turtle.**onkey**(*fun*, *key*)

turtle.**onkeyrelease**(*fun*, *key*)

Parâmetros

- **fun** – uma função sem argumentos ou `None`
- **key** – uma string: uma tecla (por exemplo, "a") ou o nome de uma tecla (por exemplo, "space")

Vincula *fun* ao evento de liberação da tecla. Se *fun* for `None`, as ligações de eventos serão removidas. Observação: para poder registrar eventos-chave, o TurtleScreen deve ter o foco. (Veja o método *listen()*.)

```
>>> def f():
...     fd(50)
...     lt(60)
...
>>> screen.onkey(f, "Up")
>>> screen.listen()
```

turtle.**onkeypress**(*fun*, *key=None*)

Parâmetros

- **fun** – uma função sem argumentos ou `None`
- **key** – uma string: uma tecla (por exemplo, "a") ou o nome de uma tecla (por exemplo, "space")

Vincula *fun* ao evento de pressionamento de tecla se a tecla for fornecida, ou a qualquer evento de pressionamento de tecla se nenhuma tecla for fornecida. Observação: para poder registrar eventos-chave, o TurtleScreen deve ter foco. (Veja o método *listen()*.)

```
>>> def f():
...     fd(50)
...
>>> screen.onkey(f, "Up")
>>> screen.listen()
```

turtle.**onclick**(*fun*, *btn=1*, *add=None*)

turtle.**onscreenclick**(*fun*, *btn=1*, *add=None*)

Parâmetros

- **fun** – um função com dois argumento que serão chamados com as coordenadas do ponto clicado na tela
- **btn** – número do botão do mouse, o padrão é 1 (botão esquerdo do mouse)
- **add** – `True` ou `False` – se `True`, uma nova ligação será adicionada; caso contrário, ela substituirá uma ligação antiga

Vincula *fun* a eventos de clique do mouse na tela. Se *fun* for `None`, as associações existentes serão removidas.

Exemplo de uma instância de TurtleScreen chamada `screen` e uma instância de Turtle chamada `turtle`:

```
>>> screen.onclick(turtle.goto) # Em seguida, clicar no TurtleScreen
>>>                             # fará a tartaruga se mover até o ponto␣
↪clicado.
>>> screen.onclick(None)        # remove associação de evento novamente
```

Nota

Este método TurtleScreen está disponível como uma função global somente com o nome `onscreenclick`. A função global `onclick` é outra derivada do método de Turtle `onclick` .

turtle.**ontimer**(*fun*, *t=0*)

Parâmetros

- **fun** – um função sem nenhum argumento
- **t** – um número >= 0

Instala um cronômetro que chama *fun* após *t* milissegundos.

```
>>> running = True
>>> def f():
...     if running:
...         fd(50)
...         lt(60)
...         screen.ontimer(f, 250)
>>> f()   ### faz a tartaruga marchar
>>> running = False
```

turtle.**mainloop**()

turtle.**done**()

Inicia o loop de eventos - chamando a função principal do loop do Tkinter. Deve ser a última instrução em um programa gráfico de tartaruga. *Não* deve ser usado se um script for executado no IDLE no modo -n (sem subprocesso) - para uso interativo de gráficos de tartaruga.

```
>>> screen.mainloop()
```

Métodos de entrada

`turtle.`**`textinput`**(*title*, *prompt*)

Parâmetros

- **`title`** – string
- **`prompt`** – string

Abre uma janela de diálogo para a entrada de uma string. O parâmetro *title* é o título da janela de diálogo, prompt é um texto que descreve as informações a serem inseridas. Se a caixa de diálogo for preenchida, retorna a string que foi informada. Se a caixa de diálogo for cancelada, retorna `None`.

```
>>> screen.textinput("NIM", "Name of first player:")
```

`turtle.`**`numinput`**(*title*, *prompt*, *default=None*, *minval=None*, *maxval=None*)

Parâmetros

- **`title`** – string
- **`prompt`** – string
- **`default`** – número (opcional)
- **`minval`** – número (opcional)
- **`maxval`** – número (opcional)

Abre uma janela de diálogo para a entrada de um número. O parâmetro *title* é o título da janela de diálogo, prompt é um texto que descreve principalmente quais informações numéricas devem ser inseridas. default: valor padrão , minval: mínimo valor para entrada, maxval: máximo valor para entrada. O número inserido deve estar no intervalo minval .. maxval, se este for fornecido. Caso contrário, será emitida uma dica e a caixa de diálogo permanecerá aberta para correção. Se a caixa de diálogo for preenchida, retorna o número informado. Se a caixa de diálogo for cancelada, retorna `None`.

```
>>> screen.numinput("Poker", "Your stakes:", 1000, minval=10, maxval=10000)
```

Configurações e métodos especiais

`turtle.`**`mode`**(*mode=None*)

Parâmetros

`mode` – uma das strings “standard”, “logo” ou “world”

Define o modo da tartaruga (“padrão”, “logotipo” ou “mundo”) e redefine a posição. Se o modo não for fornecido, o modo atual será retornado.

O modo “standard” é compatível com o antigo *`turtle`*. O modo “logo” é compatível com a maioria dos gráficos de tartaruga. O modo “world” usa “coordenadas mundiais” definidas pelo usuário. **Atenção**: nesse modo, os ângulos aparecem distorcidos se a proporção de `x/y` não for igual a 1.

Modo	Título inicial da tartaruga	ângulos positivos
“standard”	para a direita (east)	counterclockwise
“logo”	upward (north)	sentido horário

```
>>> mode("logo")   # resets turtle heading to north
>>> mode()
'logo'
```

turtle.**colormode**(*cmode=None*)

Parâmetros

cmode – um dos valores 1.0 ou 255

Retorna o modo de cor ou defini-lo como 1,0 ou 255. Posteriormente, os valores *r*, *g*, *b* das triplas de cores devem estar no limite 0..*cmode*.

```
>>> screen.colormode(1)
>>> turtle.pencolor(240, 160, 80)
Traceback (most recent call last):
     ...
TurtleGraphicsError: bad color sequence: (240, 160, 80)
>>> screen.colormode()
1.0
>>> screen.colormode(255)
>>> screen.colormode()
255
>>> turtle.pencolor(240,160,80)
```

turtle.**getcanvas**()

Retorna a tela desse TurtleScreen. Útil para pessoas que sabem o que fazer com uma tela do Tkinter.

```
>>> cv = screen.getcanvas()
>>> cv
<turtle.ScrolledCanvas object ...>
```

turtle.**getshapes**()

Retorna uma lista dos nomes de todas as formas de tartarugas disponíveis no momento.

```
>>> screen.getshapes()
['arrow', 'blank', 'circle', ..., 'turtle']
```

turtle.**register_shape**(*name*, *shape=None*)

turtle.**addshape**(*name*, *shape=None*)

Há quatro maneiras diferentes de chamar esta função:

(1) *name* é o nome de um arquivo de imagem (PNG, GIF, PGM e PPM) e *shape* é `None`: Instala a forma de imagem correspondente.

```
>>> screen.register_shape("turtle.gif")
```

Nota

As formas de imagem *não* giram ao girar a tartaruga, portanto, elas não exibem o rumo da tartaruga!

(2) *name* é uma string arbitrária e *shape* é o nome de um arquivo de imagem (PNG, GIF, PGM e PPM): Instala a forma de imagem correspondente.

```
>>> screen.register_shape("turtle", "turtle.gif")
```

Nota

As formas de imagem *não* giram ao girar a tartaruga, portanto, elas não exibem o rumo da tartaruga!

(3) *name* é uma string arbitrária e *shape* é um tupla de pares de coordenadas: Instala a forma de polígono correspondente.

```
>>> screen.register_shape("triangle", ((5,-3), (0,5), (-5,-3)))
```

(4) *name* é uma string arbitrária e *shape* é um objeto (composto) *Shape*: Instala a forma composta correspondente.

Adiciona uma forma de tartaruga à lista de formas do TurtleScreen. Somente as formas registradas dessa forma podem ser usadas com o comando `shape(shapename)` .

Alterado na versão 3.14: Adicionado suporte para os formatos de imagem PNG, PGM e PPM. É possível especificar tanto o nome da forma quanto o nome do arquivo de imagem.

`turtle.`**`turtles`**`()`

Retorne uma lista de tartarugas na tela.

```
>>> for turtle in screen.turtles():
...     turtle.color("red")
```

`turtle.`**`window_height`**`()`

Retorna a altura da janela da tartaruga.

```
>>> screen.window_height()
480
```

`turtle.`**`window_width`**`()`

Retorna a largura da janela da tartaruga.

```
>>> screen.window_width()
640
```

Métodos específico do Screen, estes métodos não são herdados do TurtleScreen

`turtle.`**`bye`**`()`

Fecha a janela do gráficos de tartaruga.

`turtle.`**`exitonclick`**`()`

Vincula o método `bye()` aos cliques do mouse na tela.

Se o valor "using_IDLE" na configuração dicionário for `False` (valor padrão), entra também no loop principal. Observação: Se IDLE for executado com a chave `-n` (sem subprocesso), esse valor deverá ser definido como `True` em `turtle.cfg`. Nesse caso, o loop principal do próprio IDLE também estará ativo para o script do cliente.

`turtle.`**`save`**`(`*filename*, *overwrite=False*`)`

Salva o desenho atual da tartaruga (e tartarugas) como um arquivo PostScript.

Parâmetros

- **`filename`** – o caminho do arquivo PostScript salvo
- **`overwrite`** – Se for `False` e já existir um arquivo com o nome fornecido, a função levantará uma `FileExistsError`. Se for `True`, o arquivo será sobrescrito.

```
>>> screen.save("my_drawing.ps")
>>> screen.save("my_drawing.ps", overwrite=True)
```

Adicionado na versão 3.14.

`turtle.`**`setup`**`(`*width=_CFG['width']*, *height=_CFG['height']*, *startx=_CFG['leftright']*, *starty=_CFG['topbottom']*`)`

Define o tamanho e a posição da janela principal. Os valores padrões dos argumentos são armazenados na configuração dicionário e podem ser alterados por meio de um arquivo `turtle.cfg`.

Parâmetros

- **width** – se for um inteiro, o tamanho em pixels; se for um float, uma fração da tela; o padrão é 50% da tela
- **height** – se for um inteiro, a altura em pixels; se for um float, uma fração da tela; o padrão é 75% da tela
- **startx** – se positivo, este parâmetro define a posição inicial em pixels a partir da borda esquerda da tela; se negativo, define a partir da borda direita; se `None`, centraliza a janela horizontalmente
- **starty** – se positivo, este parâmetro define a posição inicial, em pixels, a partir da borda superior da tela; se negativo, define a partir da borda inferior; se for `None`, centraliza a janela verticalmente

```
>>> screen.setup (width=200, height=200, startx=0, starty=0)
>>>              # define a janela para 200x200 pixels, no canto superior␣
↪esquerdo da tela
>>> screen.setup(width=.75, height=0.5, startx=None, starty=None)
>>>              # define a janela para 75% da tela por 50% da tela e␣
↪centraliza
```

`turtle.`**`title`**(*titlestring*)

Parâmetros

titlestring – uma string que é exibida na barra de título da janela de gráficos da tartaruga

Define o título da janela da tartaruga como *titlestring*.

```
>>> screen.title("Welcome to the turtle zoo!")
```

25.10.8 Classes Públicas

class `turtle.`**`RawTurtle`**(*canvas*)

class `turtle.`**`RawPen`**(*canvas*)

Parâmetros

canvas – um `tkinter.Canvas`, um *ScrolledCanvas* ou um *TurtleScreen*

Cria uma tartaruga. A tartaruga tem todos os métodos descritos acima como "métodos de Turtle/RawTurtle".

class `turtle.`**`Turtle`**

Subclasse do RawTurtle, tem a mesma interface, mas desenha em um objeto padrão *Screen*, que é criado automaticamente, quando necessário, pela primeira vez.

class `turtle.`**`TurtleScreen`**(*cv*)

Parâmetros

cv – uma `tkinter.Canvas`

Fornece métodos orientado para a tela, como *bgcolor()* etc., que são descritos acima.

class `turtle.`**`Screen`**

Subclasse do TurtleScreen, com *quatro métodos adicionados*.

class `turtle.`**`ScrolledCanvas`**(*master*)

Parâmetros

master – algum widget do Tkinter para conter o ScrolledCanvas. Exemplo: um Tkinter--canvas com barras de rolagem adicionadas

Usado pela classe Screen, que, portanto, fornece automaticamente um ScrolledCanvas como playground para as tartarugas.

class turtle.**Shape**(*type_*, *data*)

Parâmetros
type_ – uma das strings “polygon”, “image”, “compound”

Estrutura de dados para modelar a forma. O par `(type_, data)` deve seguir essa especificação:

*type_*	*data*
“polygon”	Uma tupla com o par de coordenadas do polígono
“image”	uma imagem (nesse formato, usada apenas internamente!)
“compound”	`None` (uma forma composta deve ser construída usando o método *addcomponent()*)

addcomponent(*poly*, *fill*, *outline=None*)

Parâmetros

- **poly** – um polígono, ou seja, um tupla de pares de números
- **fill** – uma cor com a qual o *poly* será preenchido
- **outline** – uma cor para o contorno do polígono (se fornecido)

Exemplo:

```
>>> poly = ((0,0),(10,-5),(0,10),(-10,-5))
>>> s = Shape("compound")
>>> s.addcomponent(poly, "red", "blue")
>>> # ... adicione mais componentes e então use register_shape()
```

Veja *Formas compostas*.

class turtle.**Vec2D**(*x*, *y*)

Uma classe de vetor bidimensional, usada como classe auxiliar para implementar gráficos de tartaruga. Pode ser útil também para programas de gráficos de tartaruga. Derivado de uma tupla, portanto, um vetor é um tupla!

Fornece (para vetores *a*, *b*, número *k*):

- `a + b` vetor adicional
- `a - b` subtração de vetor
- `a * b` produto interno
- `k * a` e `a * k` multiplicação com escalar
- `abs(a)` valor absoluto de um
- rotação `a.rotate(angle)`

25.10.9 Explicação

Um objeto tartaruga é baseado em um objeto canvas, e há uma série de classes principais na interface orientada a objetos da tartaruga que podem ser usadas para criá-los e relacioná-los entre si.

Uma instância da classe *Turtle* cria automaticamente uma nova instância de *Screen*, caso esta não esteja presente.

`Turtle` é um subclasse de *RawTurtle*, que *não* cria automaticamente uma superfície de desenho - uma *tela* precisará ser fornecida ou criada para ele. A *tela* pode ser uma `tkinter.Canvas`, *ScrolledCanvas* ou *TurtleScreen*.

TurtleScreen é a superfície de desenho básica para uma tartaruga. *Screen* é uma subclasse de `TurtleScreen` e inclui *alguns métodos adicionais* para gerenciar sua aparência (incluindo tamanho e título) e comportamento. O construtor `TurtleScreen` precisa de um `tkinter.Canvas` ou um *ScrolledCanvas* como um argumento.

A interface funcional para gráficos de tartaruga usa os vários métodos de `Turtle` e `TurtleScreen`/`Screen`. Nos bastidores, um objeto de tela é criado automaticamente sempre que um função derivada de um método de `Screen` é

chamado. Da mesma forma, um objeto de tartaruga é criado automaticamente sempre que qualquer uma das funções derivadas de um método de tartaruga é chamado.

Para usar várias tartarugas em uma tela, a interface orientada a objetos deve ser usada.

25.10.10 Ajuda e Configuração

Como usar a Ajuda

Os métodos públicos das classes Screen e Turtle estão amplamente documentado por meio de docstrings. Eles podem ser usados como ajuda on-line por meio dos recursos de ajuda do Python:

- Ao usar o IDLE, as dicas de ferramentas mostram as assinaturas e as primeiras linhas das docstrings das chamadas de função/método digitadas.
- Chamar *help()* em métodos ou funções exibe o seguinte docstring:

```
>>> help(Screen.bgcolor)
Help on method bgcolor in module turtle:

bgcolor(self, *args) unbound turtle.Screen method
    Set or return backgroundcolor of the TurtleScreen.

    Arguments (if given): a color string or three numbers
    in the range 0..colormode or a 3-tuple of such numbers.


    >>> screen.bgcolor("orange")
    >>> screen.bgcolor()
    "orange"
    >>> screen.bgcolor(0.5,0,0.5)
    >>> screen.bgcolor()
    "#800080"

>>> help(Turtle.penup)
Help on method penup in module turtle:

penup(self) unbound turtle.Turtle method
    Pull the pen up -- no drawing when moving.

    Aliases: penup | pu | up

    No argument

    >>> turtle.penup()
```

- Os docstrings de funções que são derivados do métodos têm um formato diferente:

```
>>> help(bgcolor)
Help on function bgcolor in module turtle:

bgcolor(*args)
    Set or return backgroundcolor of the TurtleScreen.

    Arguments (if given): a color string or three numbers
    in the range 0..colormode or a 3-tuple of such numbers.

    Example::

      >>> bgcolor("orange")
```

(continua na próxima página)

(continuação da página anterior)

```
      >>> bgcolor()
      "orange"
      >>> bgcolor(0.5,0,0.5)
      >>> bgcolor()
      "#800080"

>>> help(penup)
Help on function penup in module turtle:

penup()
    Pull the pen up -- no drawing when moving.

    Aliases: penup | pu | up

    No argument

    Example:
    >>> penup()
```

Esses docstrings modificados são criados automaticamente junto com as definições de função que são derivadas de métodos ao importar.

Tradução de docstrings em diferentes idiomas

Há um utilitário para criar a dicionário cujas chaves são os nomes de método e cujos valores são os docstrings dos métodos públicos das classes Screen e Turtle.

`turtle.`**`write_docstringdict`**(*filename='turtle_docstringdict'*)

Parâmetros
`filename` – uma string, usado como nome de arquivo

Cria e escreve um dicionário de docstring em um script Python com o nome de arquivo fornecido. Esta função deve ser chamada explicitamente (ela não é usada pelas classes gráficas da tartaruga). O dicionário docstring será gravado no script Python *`filename`*`.py`. O objetivo é servir como modelo para traduzir os documentos para diferentes idiomas.

Se você (ou seus alunos) quiserem usar o *`turtle`* com a ajuda on-line em seu idioma nativo, será necessário traduzir os docstrings e salvar o arquivo resultante como, por exemplo, `turtle_docstringdict_german.py`.

Se você tiver uma entrada específica em seu arquivo `turtle.cfg`, esse dicionário será lido no momento do importar e substituirá as documentações originais em inglês.

No momento da redação deste texto, há docstring dicionários em alemão e italiano. (Pedidos devem ser enviados para glingl@aon.at.)

Como configurar Screen and Turtles

A configuração padrão integrada imita a aparência e o comportamento do antigo módulo Turtle para manter a melhor compatibilidade possível com ele.

Se quiser usar uma configuração diferente que reflita melhor o recurso deste módulo ou que se adapte melhor às suas necessidades, por exemplo, para uso em uma sala de aula, você pode criar um arquivo de configuração `turtle.cfg` que será lido no momento de importar e modifica a configuração de acordo com suas definições.

A configuração que definida no arquivo `turtle.cfg`:

```
width = 0.5
height = 0.75
leftright = None
topbottom = None
```

(continua na próxima página)

(continuação da página anterior)

```
canvwidth = 400
canvheight = 300
mode = standard
colormode = 1.0
delay = 10
undobuffersize = 1000
shape = classic
pencolor = black
fillcolor = black
resizemode = noresize
visible = True
language = english
exampleturtle = turtle
examplescreen = screen
title = Python Turtle Graphics
using_IDLE = False
```

Breve explicação das entradas selecionadas:

- As quatro primeiras linhas correspondem aos argumentos do método *Screen.setup*.
- As linhas 5 e 6 correspondem a argumento do método *Screen.screensize*.
- *shape* pode ser qualquer uma das formas pré-definidas, como por exemplo: seta, tartaruga, etc. Para obter mais informações, consulte `help(shape)`.
- Se você não quiser usar nenhuma cor de preenchimento (ou seja, tornar a tartaruga transparente), você deve usar `fillcolor = ""` (todas as strings não vazias não devem ter aspas no arquivo cfg).
- Se você quiser refletir o estado da tartaruga, você deve usar `resizemode = auto`.
- Se você definir, por exemplo, `language = italian`, o docstringdict `turtle_docstringdict_italian.py` será carregado ao importar (se estiver presente no caminho de importação, por exemplo, no mesmo diretório que *turtle*).
- As entradas *exampleturtle* e *examplescreen* definem os nomes desses objetos conforme ocorrem nas docstrings. A transformação de "método-docstrings" para "função-docstrings" excluirá esses nomes dos docstrings.
- *using_IDLE*: Defina isso como `True` se você trabalha regularmente com IDLE e sua chave `-n` ("sem subprocesso"). Isso impedirá que o *exitonclick()* entre no loop principal.

Pode haver um arquivo `turtle.cfg` no diretório em que o *turtle* está armazenado e um arquivo adicional no diretório de trabalho atual. O arquivo armazenado no diretório de trabalho atual tem preferência e vai substituir as configurações do primeiro.

O diretório `Lib/turtledemo` contém um arquivo chamado `turtle.cfg`. Você pode usá-lo como um exemplo e ver seus efeitos ao executar as demonstrações (de preferência, não utilize visualizador de demonstrações).

25.10.11 `turtledemo` — Scripts de Demonstração

O pacote *turtledemo* inclui um conjunto de scripts de demonstração. Esses scripts podem ser executados e visualizados usando o visualizador de demonstração fornecido da seguinte forma:

```
python -m turtledemo
```

Como alternativa, você pode executar os scripts de demonstração individualmente. Por exemplo,

```
python -m turtledemo.bytedesign
```

O diretório do pacote *turtledemo* contém:

- Um arquivo de exemplo `__main__.py` que pode ser usado para visualizar o código-fonte dos scripts e executá-los ao mesmo tempo.

- Vários scripts que demonstram diferentes recursos do módulo *turtle*. Os exemplos podem ser acessados no menu Exemplos. Eles também podem ser executados de forma isolada.
- Um arquivo `turtle.cfg` que serve como exemplo de como escrever e usar esse tipo de arquivos.

Os scripts de demonstração são:

Nome	Descrição	Recursos
bytedesign	Padrão de gráficos de tartaruga clássico complexo	*tracer()*, delay, *update()*
chaos	gráficos do modelo de Verhulst, mostram que os cálculos do computador podem gerar resultados às vezes contra as expectativas do bom senso	coordenadas mundiais
relógio	Relógio analógico que mostra o horário do seu computador	tartarugas como as mãos do relógio, ontimer
colormixer	experimento com r, g, b	*ondrag()*
forest	3 breadth-first trees	randomization
fractalcurves	Curvas de Hilbert & Koch	recursão
lindenmayer	ethnomathematics (indian kolams)	L-System
minimal_hanoi	Torres de Hanoi	Tartarugas retângulos como discos de Hanói (shape, shapesize)
nim	jogue o clássico jogo nim com três montes de gravetos contra o computador.	tartarugas como gravetos, acionadas por eventos (mouse, teclado)
paint	programa de desenho super minimalista	*onclick()*
peça	elementar	tartaruga: aparência e animação
penrose	ladrilhos irregulares com pipas e dardos	*stamp()*
planet_and_moon	simulação do sistema gravitacional	formas compostas, *Vec2D*
rosette	um padrão do artigo Wikipédia sobre gráficos de tartaruga	*clone()*, *undo()*
round_dance	tartarugas dançantes girando em pares na direção oposta	formas compostas, clonar o shapesize, inclinação, get_shapepoly, atualizar
sorting_animate	demonstração visual de diferentes tipos de métodos de ordenação	alinhamento simples, randomização
tree	uma árvore (gráfica) da largura inicial (usando geradores)	*clone()*
two_canvases	desenho simples	tartarugas em duas telas
yinyang	outro exemplo elementar	*circle()*

Diverta-se!

25.10.12 Modificações desde a versão do Python 2.6

- O métodos *Turtle.tracer*, *Turtle.window_width* e *Turtle.window_height* foram removidos. O métodos com esses nomes e funcionalidades agora estão disponíveis apenas como métodos da classe *Screen*. As funções derivadas desses métodos continuam disponíveis. (No Python 2.6 esses métodos eram duplicações dos métodos correspondentes das classes *TurtleScreen*/*Screen*).
- O método `Turtle.fill()` foi eliminado. O comportamento de *begin_fill()* e *end_fill()* foi ligeiramente alterado: agora, todo processo de preenchimento deve ser concluído com uma chamada para `end_fill()`.
- Um método *Turtle.filling* foi adicionado. Ela retorna um valor booleano: `True` se um processo de preenchimento estiver em andamento, `False` caso contrário. Esse comportamento corresponde a uma chamada `fill()` sem argumento em Python 2.6.

25.10.13 Modificações desde a versão do Python 3.0

- Foram adicionados a class *Turtle* os métodos *shearfactor()* , *shapetransform()* e *get_shapepoly()*. Assim, o limite completo de transformações lineares regulares está agora disponível para transformar as formas de tartarugas. O *tiltangle()* foi melhorado em termos de funcionalidade: agora pode ser usado para obter ou definir o ângulo de inclinação.
- Na classe *Screen*, o método *onkeypress()* foi adicionado como um complemento ao *onkey()*. Como esse último vincula ações ao liberar uma tecla pressionada, um apelido: *onkeyrelease()* também foi adicionado a ele.
- O método *Screen.mainloop* foi adicionado. Não é mais necessário utilizar a função *mainloop()* quando estiver trabalhando com os objetos *Screen* e *Turtle*.
- Dois novos métodos para informar dados foram adicionados: *Screen.textinput* e *Screen.numinput*. Estes métodos abrem caixas de diálogo para digitação e retornam strings e números, respectivamente.

CAPÍTULO 26

Ferramentas de Desenvolvimento

Os módulos descritos neste capítulo ajudam você a escrever softwares. Por exemplo, o módulo *pydoc* recebe um módulo e gera documentação com base no conteúdo do módulo. Os módulos *doctest* e *unittest* contêm frameworks para escrever testes unitários que automaticamente exercitam código e verificam se a saída esperada é produzida.

A lista de módulos descritos neste capítulo é:

26.1 `typing` — Suporte para dicas de tipo

Adicionado na versão 3.5.

Código-fonte: Lib/typing.py

> **Nota**
>
> O ambiente de execução do Python não força anotações de tipos de variáveis e funções. Elas podem ser usadas por ferramentas de terceiros como *verificadores de tipo*, IDEs, linters, etc.

Este módulo fornece suporte em tempo de execução para dicas de tipo.

Considere a função abaixo:

```
def surface_area_of_cube(edge_length: float) -> str:
    return f"A área superficial de um cubo é {6 * edge_length ** 2}."
```

A função `surface_area_of_cube` recebe um argumento que se espera ser uma instância de *float*, conforme indicado pela *dica de tipo* `edge_length: float`. Espera-se que a função retorne uma instância de *str*, conforme indicado pela dica `-> str`.

Embora as dicas de tipo possam ser classes simples como *float* ou *str*, elas também podem ser mais complexas. O módulo *typing* fornece um vocabulário de dicas de tipo mais avançadas.

Novos recursos são frequentemente adicionados ao módulo `typing`. O pacote typing_extensions fornece backports desses novos recursos para versões mais antigas do Python.

Ver também

Folha de dicas sobre tipagem
Uma visão geral das dicas de tipo (hospedado por mypy docs, em inglês).

Seção de referência do sistema de tipos da documentação do mypy
O sistema de tipagem do Python é padronizado pelas PEPs, portanto esta referência deve se aplicar a maioria do verificadores de tipo do Python. (Alguns trechos podem se referir especificamente ao mypy. Documento em inglês).

Static Typing with Python
Documentação independente de verificador de tipo escrita pela comunidade, detalhando os recursos do sistema de tipo, ferramentas úteis de tipagem e melhores práticas.

26.1.1 Especificação para o sistema de tipos do Python

A especificação canônica e atualizada do sistema de tipos Python pode ser encontrada em Specification for the Python type system.

26.1.2 Apelidos de tipo

Um apelido de tipo é definido utilizando a instrução `type`, que por sua vez cria uma instância da classe *`TypeAliasType`*. Neste exemplo, `Vector` e `list[float]` serão tratados de maneira equivalente pelos verificadores de tipo estático:

```
type Vector = list[float]

def scale(scalar: float, vector: Vector) -> Vector:
    return [scalar * num for num in vector]

# passa na verificação de tipos; uma lista de floats é qualificada como Vector.
new_vector = scale(2.0, [1.0, -4.2, 5.4])
```

Apelidos de tipo são úteis para simplificar assinaturas de tipo complexas. Por exemplo:

```
from collections.abc import Sequence

type ConnectionOptions = dict[str, str]
type Address = tuple[str, int]
type Server = tuple[Address, ConnectionOptions]

def broadcast_message(message: str, servers: Sequence[Server]) -> None:
    ...

# O verificador de tipo tratará a assinatura de tipo anterior
# como exatamente equivalente à assinatura abaixo.
def broadcast_message(
    message: str,
    servers: Sequence[tuple[tuple[str, int], dict[str, str]]]
) -> None:
    ...
```

A instrução `type` é nova no Python 3.12. Para manter retrocompatibilidade, apelidos de tipo também podem ser criados usando atribuição:

```
Vector = list[float]
```

Ou marcado com *`TypeAlias`* para tornar explícito que se trata de um apelido de tipo e não uma atribuição de variável comum:

```
from typing import TypeAlias

Vector: TypeAlias = list[float]
```

26.1.3 NewType

Utilize o auxiliar *NewType* para criar tipos únicos:

```
from typing import NewType

UserId = NewType('UserId', int)
some_id = UserId(524313)
```

O verificador de tipo estático tratará o novo tipo como se fosse uma subclasse do tipo original. Isso é útil para ajudar a encontrar erros de lógica:

```
def get_user_name(user_id: UserId) -> str:
    ...

# passa na verificação de tipo
user_a = get_user_name(UserId(42351))

# falha na verificação de tipo: um int não é um UserId
user_b = get_user_name(-1)
```

Você ainda pode executar todas as operações `int` em uma variável do tipo `UserId`, mas o resultado sempre será do tipo `int`. Isso permite que você passe um `UserId` em qualquer ocasião que `int` possa ser esperado, mas previne que você acidentalmente crie um `UserId` de uma forma inválida:

```
# 'output' é do tipo 'int', em vez de 'UserId'
output = UserId(23413) + UserId(54341)
```

Note que essas verificações são aplicadas apenas pelo verificador de tipo estático. Em tempo de execução, a instrução `Derived = NewType('Derived', Base)` irá tornar `Derived` um chamável que retornará imediatamente qualquer parâmetro que você passar. Isso significa que a expressão `Derived(some_value)` não cria uma nova classe ou introduz sobrecarga além de uma chamada regular de função.instrução

Mais precisamente, a expressão `some_value is Derived(some_value)` é sempre verdadeira em tempo de execução.

É inválido criar um subtipo de `Derived`:

```
from typing import NewType

UserId = NewType('UserId', int)

# Falha em tempo de execução e não passa na verificação de tipos
class AdminUserId(UserId): pass
```

No entanto, é possível criar um *NewType* baseado em um 'derivado' `NewType`:

```
from typing import NewType

UserId = NewType('UserId', int)

ProUserId = NewType('ProUserId', UserId)
```

e a verificação de tipos para `ProUserId` funcionará como esperado.

Veja **PEP 484** para mais detalhes.

Nota

Lembre-se que o uso de um apelido de tipo declara que dois tipos serão *equivalentes* entre si. Efetuar `type Alias = Original` fará o verificador de tipo estático tratar `Alias` como sendo *exatamente equivalente* a `Original` em todos os casos. Isso é útil quando você deseja simplificar assinaturas de tipo complexas.

Em contraste, `NewType` declara que um tipo será *subtipo* de outro. Efetuando `Derived = NewType('Derived', Original)` irá fazer o verificador de tipo estático tratar `Derived` como uma *subclasse* de `Original`, o que significa que um valor do tipo `Original` não pode ser utilizado onde um valor do tipo `Derived` é esperado. Isso é útil quando você deseja evitar erros de lógica com custo mínimo de tempo de execução.

Adicionado na versão 3.5.2.

Alterado na versão 3.10: `NewType` agora é uma classe em vez de uma função. Consequentemente, existem alguns custos em tempo de execução ao chamar `NewType` em vez de uma função comum.

Alterado na versão 3.11: O desempenho de chamar `NewType` voltou ao mesmo nível da versão Python 3.9.

26.1.4 Anotações de objetos chamáveis

Funções – ou outros objetos *chamáveis* – podem ser anotados com *collections.abc.Callable* ou com o tipo descontinuado *typing.Callable*. `Callable[[int], str]` expressa uma função que recebe um único parâmetro de tipo *int* e retorna uma *str*.

Por exemplo:

```
from collections.abc import Callable, Awaitable

def feeder(get_next_item: Callable[[], str]) -> None:
    ...  # Corpo

def async_query(on_success: Callable[[int], None],
                on_error: Callable[[int, Exception], None]) -> None:
    ...  # Corpo

async def on_update(value: str) -> None:
    ...  # Corpo

callback: Callable[[str], Awaitable[None]] = on_update
```

A sintaxe de subscrição deve sempre ser usada com exatamente dois valores: a lista de argumentos e o tipo de retorno. A lista de argumentos deve ser uma lista de tipos, uma *ParamSpec*, *Concatenate* ou reticências (`...`). O tipo de retorno deve ser um tipo único.

Se uma reticências literal `...` é passada no lugar de uma lista de argumentos, ela indica que um chamável com uma lista de qualquer parâmetro arbitrário seria aceita.

```
def concat(x: str, y: str) -> str:
    return x + y

x: Callable[..., str]
x = str     # OK
x = concat  # OK também
```

`Callable` não pode representar assinaturas complexas, como funções que aceitam um número variado de argumentos, *funções sobrecarregadas*, or funções que recebem apenas parâmetros somente-nomeados. No entanto, essas assinaturas podem ser expressas ao se definir uma *Protocol* com um método `__call__()`:

```
from collections.abc import Iterable
from typing import Protocol

class Combiner(Protocol):
    def __call__(self, *vals: bytes, maxlen: int | None = None) -> list[bytes]: ...

def batch_proc(data: Iterable[bytes], cb_results: Combiner) -> bytes:
    for item in data:
        ...

def good_cb(*vals: bytes, maxlen: int | None = None) -> list[bytes]:
    ...
def bad_cb(*vals: bytes, maxitems: int | None) -> list[bytes]:
    ...

batch_proc([], good_cb)  # OK
batch_proc([], bad_cb)   # Erro! Argumento 2 tem tipo incompatível por ter
                         # nome e tipo diferente do callback
```

Chamáveis que recebem outros chamáveis como argumentos podem indicar que seus tipos de parâmetro dependem uns dos outros usando *ParamSpec*. Além disso, se esse chamável adiciona ou remove argumentos de outros chamáveis, o operador *Concatenate* pode ser usado. Eles assumem a forma de `Callable[ParamSpecVariable, ReturnType]` e `Callable[Concatenate[Arg1Type, Arg2Type, ..., ParamSpecVariable], ReturnType]`, respectivamente.

Alterado na versão 3.10: `Callable` agora oferece suporte a *ParamSpec* e *Concatenate*. Veja **PEP 612** para mais detalhes.

Ver também

A documentação para *ParamSpec* e *Concatenate* contém exemplos de uso em `Callable`.

26.1.5 Genéricos

Como a informação de tipo sobre objetos mantidos em contêineres não pode ser inferida estaticamente de uma maneira genérica, muitas classes de contêineres na biblioteca padrão permitem usar subscrição para denotar o tipo esperado dos elementos dos contêineres.

```
from collections.abc import Mapping, Sequence

class Employee: ...

# Sequence[Employee] indica que todos os elementos na sequência
# devem ser instâncias de "Employee".
# Mapping[str, str] indica que todas as chaves e valores no mapeamento
# devem ser strings.
def notify_by_email(employees: Sequence[Employee],
                    overrides: Mapping[str, str]) -> None: ...
```

Funções e classes genéricas podem ser parametrizadas utilizando a sintaxe de parâmetro de tipos:

```
from collections.abc import Sequence

def first[T](l: Sequence[T]) -> T:  # Função é genérica sobre TypeVar "T"
    return l[0]
```

Ou utilizando a fábrica *TypeVar* diretamente:

```
from collections.abc import Sequence
from typing import TypeVar

U = TypeVar('U')                  # Declara tipo variável "U"

def second(l: Sequence[U]) -> U:  # Função é genérica sobre a TypeVar "U"
    return l[1]
```

Alterado na versão 3.12: O suporte sintático para genéricos é novo no Python 3.12.

26.1.6 Anotando tuplas

Para a maior parte dos tipos contêineres em Python, o sistema de tipagem presume que todos os elementos do contêiner são do mesmo tipo. Por exemplo:

```
from collections.abc import Mapping

# Verificador de tipos irá inferir que todos os elementos em ``x`` devem ser ints
x: list[int] = []

# Erro no verificador: ``list`` aceita um único tipo de argumento
y: list[int, str] = [1, 'foo']

# Verificador de tipos irá inferir que todas as chaves em ``z`` devem ser strings,
# e todos os valores em ``z`` devem ser strings ou ints
z: Mapping[str, str | int] = {}
```

`list` aceita apenas um tipo de argumento, e assim o verificador de tipos emitirá um erro na atribuição `y` acima. Da mesma forma, `Mapping` aceita apenas dois tipos de argumento: O primeiro indica o tipo das chaves, e o segundo indica o tipo dos valores.

Ao contrário da maioria dos outros contêineres Python, é comum no código Python idiomático que as tuplas tenham elementos que não sejam todos do mesmo tipo. Por esse motivo, as tuplas têm um caso especial no sistema de tipagem do Python. `tuple` aceita *qualquer quantidade* de argumentos de tipo:

```
# OK: ``x`` foi atribuída a uma tupla de comprimento 1, onde o único elemento é um
↪int
x: tuple[int] = (5,)

# OK: ``y`` foi atribuída a uma tupla de comprimento 2;
# o elemento 1 é um int, o elemento 2 é uma str
y: tuple[int, str] = (5, "foo")

# Erro: a anotação de tipo indica uma tupla de comprimento 1,
# mas ``z`` foi atribuída a uma tupla de comprimento 3
z: tuple[int] = (1, 2, 3)
```

Para denotar uma tupla que pode ter *qualquer* comprimento e na qual todos os elementos são do mesmo tipo `T`, use a literal de reticências `...`: `tuple[T, ...]`. Para denotar uma tupla vazia, use `tuple[()]`. Usar `tuple` simples como anotação equivale a usar `tuple[Any, ...]`:

```
x: tuple[int, ...] = (1, 2)
# Essas reatribuições são OK: ``tuple[int, ...]`` indica que x pode ter qualquer
↪comprimento
x = (1, 2, 3)
x = ()
# Essa reatribuição é um erro: todos os elementos de ``x`` devem ser ints
x = ("foo", "bar")
```

(continua na próxima página)

(continuação da página anterior)

```
# ``y`` só pode ser atribuída a uma tupla vazia
y: tuple[()] = ()

z: tuple = ("foo", "bar")
# Essas reatribuições são OK: o tipo ``tuple`` equivale a ``tuple[Any, ...]``
z = (1, 2, 3)
z = ()
```

26.1.7 O tipo de objetos de classe

Uma variável anotada com `C` pode aceitar um valor de tipo `C`. Por outro lado, uma variável anotada com `type[C]` (ou o tipo descontinuado *typing.Type[C]*) pode aceitar valores que são as próprias classes – mais especificamente, ela aceitará o *objeto classe* de `C`. Por exemplo:

```
a = 3         # Tem tipo ``int``
b = int       # Tem tipo ``type[int]``
c = type(a)   # Também tem tipo ``type[int]``
```

Observe que `type[C]` é covariante:

```
class User: ...
class ProUser(User): ...
class TeamUser(User): ...

def make_new_user(user_class: type[User]) -> User:
    # ...
    return user_class()

make_new_user(User)      # OK
make_new_user(ProUser)   # Também OK: ``type[ProUser]`` é subtipo de ``type[User]``
make_new_user(TeamUser)  # Tudo bem
make_new_user(User())    # Erro: esperava-se ``type[User]`` mas retornou ``User``
make_new_user(int)       # Erro: ``type[int]`` não é subtipo de ``type[User]``
```

Os únicos parâmetros válidos para *type* são classes, *Any*, *type variables* e uniões de qualquer um desses tipos. Por exemplo:

```
def new_non_team_user(user_class: type[BasicUser | ProUser]): ...

new_non_team_user(BasicUser)  # OK
new_non_team_user(ProUser)    # OK
new_non_team_user(TeamUser)   # Erro: ``type[TeamUser]`` não é subtipo
                              # de ``type[BasicUser | ProUser]``
new_non_team_user(User)       # Também é um erro
```

`type[Any]` é equivalente a *type*, que é a raiz da hierarquia de metaclasses do Python.

26.1.8 Anotando geradores e corrotinas

Um gerador pode ser anotado usando um tipo genérico *Generator[YieldType, SendType, ReturnType]*. Por exemplo:

```
def echo_round() -> Generator[int, float, str]:
    sent = yield 0
    while sent >= 0:
```

(continua na próxima página)

(continuação da página anterior)

```
        sent = yield round(sent)
    return 'Done'
```

Note que, diferentemente de outras classes genéricas na biblioteca padrão, o `SendType` da classe *`Generator`* se comporta contravariantemente, em vez de covariantemente ou invariantemente.

Os parâmetros `SendType` e `ReturnType` são `None` por padrão:

```
def infinite_stream(start: int) -> Generator[int]:
    while True:
        yield start
        start += 1
```

Também é possível definir esses tipos explicitamente:

```
def infinite_stream(start: int) -> Generator[int, None, None]:
    while True:
        yield start
        start += 1
```

Geradores simples que só produzem valores também podem ser anotados com tipo de retorno *`Iterable[YieldType]`* ou *`Iterator[YieldType]`*:

```
def infinite_stream(start: int) -> Iterator[int]:
    while True:
        yield start
        start += 1
```

Geradores assíncronos são manipulados similarmente, mas não espere um argumento de tipo `ReturnType` (*`AsyncGenerator[YieldType, SendType]`*). O argumento `SendType` é `None` por padrão, então as seguintes definições são equivalentes:

```
async def infinite_stream(start: int) -> AsyncGenerator[int]:
    while True:
        yield start
        start = await increment(start)

async def infinite_stream(start: int) -> AsyncGenerator[int, None]:
    while True:
        yield start
        start = await increment(start)
```

Como no caso síncrono, *`AsyncIterable[YieldType]`* e *`AsyncIterator[YieldType]`* também estão disponíveis:

```
async def infinite_stream(start: int) -> AsyncIterator[int]:
    while True:
        yield start
        start = await increment(start)
```

Corrotinas podem ser anotadas usando *`Coroutine[YieldType, SendType, ReturnType]`*. Argumentos genéricos correspondem àqueles da classe *`Generator`*, por exemplo:

```
from collections.abc import Coroutine
c: Coroutine[list[str], str, int]  # Uma corrotina defina em outro lugar
x = c.send('hi')                   # Tipo inferido de 'x' é list[str]
async def bar() -> None:
    y = await c                    # Tipo inferido de 'y' é int
```

26.1.9 Tipos genéricos definidos pelo usuário

Uma classe definida pelo usuário pode ser definida como uma classe genérica.

```
from logging import Logger

class LoggedVar[T]:
    def __init__(self, value: T, name: str, logger: Logger) -> None:
        self.name = name
        self.logger = logger
        self.value = value

    def set(self, new: T) -> None:
        self.log('Set ' + repr(self.value))
        self.value = new

    def get(self) -> T:
        self.log('Get ' + repr(self.value))
        return self.value

    def log(self, message: str) -> None:
        self.logger.info('%s: %s', self.name, message)
```

Esta sintaxe indica que o classe `LoggedVar` é parametrizada em torno de uma única *type variable* `T`. Isso também torna `T` válido como um tipo dentro do corpo da classe.

Classes genéricas implicitamente herdar de *`Generic`*. Para compatibilidade com Python 3.11 e versões inferiores, também é possível herdar explicitamente de *`Generic`* para indicar uma classe genérica:

```
from typing import TypeVar, Generic

T = TypeVar('T')

class LoggedVar(Generic[T]):
    ...
```

Classes genéricas têm métodos `__class_getitem__()`, o que significa que podem ser parametrizadas em tempo de execução (por exemplo, `LoggedVar[int]` abaixo):

```
from collections.abc import Iterable

def zero_all_vars(vars: Iterable[LoggedVar[int]]) -> None:
    for var in vars:
        var.set(0)
```

Um tipo genérico pode ter qualquer número de tipos de variáveis. Todas as variedades de *`TypeVar`* são permitidas como parâmetros para um tipo genérico:

```
from typing import TypeVar, Generic, Sequence

class WeirdTrio[T, B: Sequence[bytes], S: (int, str)]:
    ...

OldT = TypeVar('OldT', contravariant=True)
OldB = TypeVar('OldB', bound=Sequence[bytes], covariant=True)
OldS = TypeVar('OldS', int, str)

class OldWeirdTrio(Generic[OldT, OldB, OldS]):
    ...
```

Cada tipo dos argumentos para *Generic* devem ser distintos. Assim, os seguintes exemplos são inválidos:

```
from typing import TypeVar, Generic
...

class Pair[M, M]:  # SyntaxError
    ...

T = TypeVar('T')

class Pair(Generic[T, T]):   # INVÁLIDO
    ...
```

Classes genéricas podem também herdar de outras classes:

```
from collections.abc import Sized

class LinkedList[T](Sized):
    ...
```

Ao herdar das classes genérico, alguns tipos podem ser fixos:

```
from collections.abc import Mapping

class MyDict[T](Mapping[str, T]):
    ...
```

Neste caso `MyDict` possui um único parâmetro, `T`.

O uso de uma classe genérica sem especificar tipos pressupõe *Any* para cada posição. No exemplo a seguir, `MyIterable` não é genérico, mas herda implicitamente de `Iterable[Any]`:

```
from collections.abc import Iterable

class MyIterable(Iterable): # Igual a Iterable[Any]
    ...
```

Também há suporte para tipos genéricos definidos pelo usuário. Exemplos:

```
from collections.abc import Iterable

type Response[S] = Iterable[S] | int

# Tipo de retorno aqui é igual a Iterable[str] | int
def response(query: str) -> Response[str]:
    ...

type Vec[T] = Iterable[tuple[T, T]]

def inproduct[T: (int, float, complex)](v: Vec[T]) -> T: # Igual a␣
↪Iterable[tuple[T, T]]
    return sum(x*y for x, y in v)
```

Para manter retrocompatibilidade, os apelidos de tipos genéricos também podem ser criados usando atribuição:

```
from collections.abc import Iterable
from typing import TypeVar

S = TypeVar("S")
Response = Iterable[S] | int
```

Alterado na versão 3.7: *Generic* não possui mais uma metaclasse personalizada.

Alterado na versão 3.12: Suporte sintático para apelidos de tipo e genéricos é novo na versão 3.12. Anteriormente, as classes genéricas precisavam explicitamente herdar de *Generic* ou conter um tipo de variável em uma de suas bases.

Genéricos definidos pelo usuário para expressões de parâmetros também oferecem suporte por meio de variáveis de especificação de parâmetros no formato `[**P]`. O comportamento é consistente com as variáveis de tipo descritas acima, pois as variáveis de especificação de parâmetro são tratadas pelo módulo `typing` como uma variável de tipo especializada. A única exceção a isso é que uma lista de tipos pode ser usada para substituir um *ParamSpec*:

```
>>> class Z[T, **P]: ...  # T é um TypeVar; P é um ParamSpec
...
>>> Z[int, [dict, float]]
__main__.Z[int, [dict, float]]
```

Classes genéricas sobre um *ParamSpec* também podem ser criadas usando herança explícita de *Generic*. Neste caso, `**` não é usado:

```
from typing import ParamSpec, Generic

P = ParamSpec('P')

class Z(Generic[P]):
    ...
```

Outra diferença entre *TypeVar* e *ParamSpec* é que um genérico com apenas uma variável de especificação de parâmetro aceitará listas de parâmetros nos formatos `X[[Type1, Type2, ...]]` e também `X[Type1, Type2, ...]` por razões estéticas. Internamente, o último é convertido no primeiro, portanto são equivalentes:

```
>>> class X[**P]: ...
...
>>> X[int, str]
__main__.X[[int, str]]
>>> X[[int, str]]
__main__.X[[int, str]]
```

Observe que genéricos com *ParamSpec* podem não ter `__parameters__` corretos após a substituição em alguns casos porque eles são destinados principalmente à verificação de tipo estático.

Alterado na versão 3.10: *Generic* agora pode ser parametrizado através de expressões de parâmetros. Veja *ParamSpec* e **PEP 612** para mais detalhes.

Uma classe genérica definida pelo usuário pode ter ABCs como classes base sem conflito de metaclasse. Não há suporte a metaclasses genéricas. O resultado da parametrização de genéricos é armazenado em cache, e a maioria dos tipos no módulo `typing` são *hasheáveis* e comparáveis em termos de igualdade.

26.1.10 O tipo `Any`

Um tipo especial de tipo é *Any*. Um verificador de tipo estático tratará cada tipo como sendo compatível com *Any* e *Any* como sendo compatível com todos os tipos.

Isso significa que é possível realizar qualquer operação ou chamada de método sobre um valor do tipo *Any* e atribuí-lo a qualquer variável:

```
from typing import Any

a: Any = None
a = []          # OK
a = 2           # OK
```

(continua na próxima página)

(continuação da página anterior)

```
s: str = ''
s = a           # OK

def foo(item: Any) -> int:
    # Passa na verificação de tipos; 'item' pode ser qualquer tipo,
    # e esse tipo pode ter um método 'bar'
    item.bar()
    ...
```

Observe que nenhuma verificação de tipo é realizada ao atribuir um valor do tipo *Any* a um tipo mais preciso. Por exemplo, o verificador de tipo estático não relatou um erro ao atribuir `a` a `s` mesmo que `s` tenha sido declarado como sendo do tipo *str* e receba um valor *int* em tempo de execução!

Além disso, todas as funções sem um tipo de retorno ou tipos de parâmetro terão como padrão implicitamente o uso de *Any*:

```
def legacy_parser(text):
    ...
    return data

# Um verificador de tipo estático tratará a função acima
# como tendo a mesma assinatura da função abaixo:
def legacy_parser(text: Any) -> Any:
    ...
    return data
```

Este comportamento permite que *Any* seja usado como uma *saída de emergência* quando você precisar misturar código tipado dinamicamente e estaticamente.

Compare o comportamento de *Any* com o comportamento de *object*. Semelhante a *Any*, todo tipo é um subtipo de *object*. No entanto, ao contrário de *Any*, o inverso não é verdadeiro: *object* *não* é um subtipo de qualquer outro tipo.

Isso significa que quando o tipo de um valor é *object*, um verificador de tipo rejeitará quase todas as operações nele, e atribuí-lo a uma variável (ou usá-la como valor de retorno) de um tipo mais especializado é um tipo erro. Por exemplo:

```
def hash_a(item: object) -> int:
    # Falha na verificação de tipo; um 'object' não tem um método 'magic'.
    item.magic()
    ...

def hash_b(item: Any) -> int:
    # Passa na verificação de tipo
    item.magic()
    ...

# Passa na verificação de tipo, já que ints e strs são subclasses de 'object'
hash_a(42)
hash_a("foo")

# Passa na verificação de tipo, já que Any é compatível com todos os tipos
hash_b(42)
hash_b("foo")
```

Use *object* para indicar que um valor pode ser de qualquer tipo de maneira segura. Use *Any* para indicar que um valor é tipado dinamicamente.

26.1.11 Subtipagem nominal vs estrutural

Inicialmente a PEP 484 definiu o sistema de tipos estáticos do Python como usando *subtipagem nominal*. Isto significa que uma classe `A` é permitida onde uma classe `B` é esperada se e somente se `A` for uma subclasse de `B`.

Este requisito anteriormente também se aplicava a classes base abstratas, como *`Iterable`*. O problema com essa abordagem é que uma classe teve que ser marcada explicitamente para suportá-los, o que não é pythônico e diferente do que normalmente seria feito em código Python de tipo dinamicamente idiomático. Por exemplo, isso está em conformidade com PEP 484:

```
from collections.abc import Sized, Iterable, Iterator

class Bucket(Sized, Iterable[int]):
    ...
    def __len__(self) -> int: ...
    def __iter__(self) -> Iterator[int]: ...
```

PEP 544 permite resolver este problema permitindo que os usuários escrevam o código acima sem classes base explícitas na definição de classe, permitindo que `Bucket` seja implicitamente considerado um subtipo de `Sized` e `Iterable[int]` por verificador de tipo estático. Isso é conhecido como *subtipagem estrutural* (ou tipagem pato estática):

```
from collections.abc import Iterator, Iterable

class Bucket:  # Nota: sem classes base
    ...
    def __len__(self) -> int: ...
    def __iter__(self) -> Iterator[int]: ...

def collect(items: Iterable[int]) -> int: ...
result = collect(Bucket())  # Passa na verificação de tipo
```

Além disso, ao criar uma subclasse de uma classe especial *`Protocol`*, um usuário pode definir novos protocolos personalizados para aproveitar ao máximo a subtipagem estrutural (veja exemplos abaixo).

26.1.12 Conteúdo do módulo

O módulo `typing` define as seguintes classes, funções e decoracores.

Tipos primitivos especiais

Tipos especiais

Eles podem ser usados como tipos em anotações. Eles não oferecem suporte a subscrição usando `[]`.

`typing.`**`Any`**

Tipo especial que indica um tipo irrestrito.

- Todos os tipos são compatíveis com *`Any`*.
- *`Any`* é compatível com todos os tipos.

Alterado na versão 3.11: *`Any`* agora pode ser usado como classe base. Isso pode ser útil para evitar erros do verificador de tipo com classes que podem digitar em qualquer lugar ou são altamente dinâmicas.

`typing.`**`AnyStr`**

Uma *tipo variável restrito*.

Definição:

```
AnyStr = TypeVar('AnyStr', str, bytes)
```

`AnyStr` deve ser usado para funções que podem aceitar argumentos *`str`* ou *`bytes`* mas não podem permitir que os dois se misturem.

Por exemplo:

```
def concat(a: AnyStr, b: AnyStr) -> AnyStr:
    return a + b

concat("foo", "bar")    # OK: saída tem tipo 'str'
concat(b"foo", b"bar")  # OK: saída tem tipo 'bytes'
concat("foo", b"bar")   # Erro: não se mistura str com bytes
```

Note que, apesar do nome, `AnyStr` não tem nada a ver com o tipo *`Any`*, nem significa "qualquer string". Em particular, `AnyStr` e `str | bytes` são diferentes entre si e têm casos de uso diferentes:

```
# Uso inválido de AnyStr:
# A variável de tipo é usada somente uma vez na assinatura da função,
# então não pode ser "resolvida" pelo verificador de tipo
def greet_bad(cond: bool) -> AnyStr:
    return "hi there!" if cond else b"greetings!"

# O jeito melhor de anotar esta função
def greet_proper(cond: bool) -> str | bytes:
    return "hi there!" if cond else b"greetings!"
```

Deprecated since version 3.13, will be removed in version 3.18: Descontinuado em favor da nova sintaxe de parâmetro de tipo. Use `class A[T: (str, bytes)]: ...` em vez de importar `AnyStr`. Veja a **PEP 695** para mais detalhes.

No Python 3.16, `AnyStr` será removido de `typing.__all__`, e avisos de descontinuação serão emitidos em tempo de execução quando ele for acessado ou importado de `typing`. `AnyStr` será removido do `typing` no Python 3.18.

`typing.`**`LiteralString`**

Tipo especial que inclui apenas strings literais.

Qualquer literal de string é compatível com `LiteralString`, assim como outro `LiteralString`. Entretanto, um objeto digitado apenas `str` não é. Uma string criada pela composição de objetos do tipo `LiteralString` também é aceitável como uma `LiteralString`.

Exemplo:

```
def run_query(sql: LiteralString) -> None:
    ...

def caller(arbitrary_string: str, literal_string: LiteralString) -> None:
    run_query("SELECT * FROM students")  # OK
    run_query(literal_string)  # OK
    run_query("SELECT * FROM " + literal_string)  # OK
    run_query(arbitrary_string)  # erro no verificador
    run_query(  # erro no verificador
        f"SELECT * FROM students WHERE name = {arbitrary_string}"
    )
```

`LiteralString` é útil para APIs sensíveis onde strings arbitrárias geradas pelo usuário podem gerar problemas. Por exemplo, os dois casos acima que geram erros no verificador de tipo podem ser vulneráveis a um ataque de injeção de SQL.

Veja **PEP 675** para mais detalhes.

Adicionado na versão 3.11.

`typing.`**`Never`**

typing.**NoReturn**

`Never` e `NoReturn` representam o tipo inferior, um tipo que não possui membros.

Eles podem ser usados para indicar que uma função nunca retorna, como *sys.exit()*:

```
from typing import Never  # ou NoReturn

def stop() -> Never:
    raise RuntimeError('no way')
```

Ou para definir uma função que nunca deve ser chamada, pois não existem argumentos válidos, como *assert_never()*:

```
from typing import Never  # or NoReturn

def never_call_me(arg: Never) -> None:
    pass

def int_or_str(arg: int | str) -> None:
    never_call_me(arg)  # erro no verificador
    match arg:
        case int():
            print("It's an int")
        case str():
            print("It's a str")
        case _:
            never_call_me(arg)  # OK, arg é do tipo Never (ou NoReturn)
```

`Never` e `NoReturn` têm o mesmo significado no sistema de tipos e os verificadores de tipo estático tratam ambos de forma equivalente.

Adicionado na versão 3.6.2: Adicionado *NoReturn*.

Adicionado na versão 3.11: Adicionado *Never*.

typing.**Self**

Tipo especial para representar a classe atual inclusa.

Por exemplo:

```
from typing import Self, reveal_type

class Foo:
    def return_self(self) -> Self:
        ...
        return self

class SubclassOfFoo(Foo): pass

reveal_type(Foo().return_self())  # Tipo revelado é "Foo"
reveal_type(SubclassOfFoo().return_self())  # Tipo revelado é "SubclassOfFoo"
```

Esta anotação é semanticamente equivalente à seguinte, embora de forma mais sucinta:

```
from typing import TypeVar

Self = TypeVar("Self", bound="Foo")

class Foo:
    def return_self(self: Self) -> Self:
```

(continua na próxima página)

(continuação da página anterior)

```
        ...
        return self
```

Em geral, se algo retorna `self`, como nos exemplos acima, você deve usar `Self` como anotação de retorno. Se `Foo.return_self` foi anotado como retornando `"Foo"`, então o verificador de tipo inferiria o objeto retornado de `SubclassOfFoo.return_self` como sendo do tipo `Foo` em vez de `SubclassOfFoo`.

Outros casos de uso comuns incluem:

- *classmethod*s que são usados como construtores alternativos e retornam instâncias do parâmetro `cls`.
- Anotando um método `__enter__()` que retorna self.

Você não deveria usar `Self` como a anotação de retorno se não for garantido que o método retorne uma instância de uma subclasse quando a classe for subclassificada:

```
class Eggs:
    # Self seria uma anotação de retorno incorreta aqui,
    # já que o objeto retornado sempre é instância de Eggs,
    # inclusive em subclasses
    def returns_eggs(self) -> "Eggs":
        return Eggs()
```

Veja **PEP 673** para mais detalhes.

Adicionado na versão 3.11.

typing.**TypeAlias**

Anotações especiais para declarar explicitamente um *apelido de tipo*.

Por exemplo:

```
from typing import TypeAlias

Factors: TypeAlias = list[int]
```

`TypeAlias` é particularmente útil em versões mais antigas do Python para anotar apelidos que fazem uso de referências futuras, pois pode ser difícil para os verificadores de tipo distingui-los das atribuições normais de variáveis:

```
from typing import Generic, TypeAlias, TypeVar

T = TypeVar("T")

# "Box" não existe ainda,
# então temos que usar aspas para a referência futura em Python <3.12.
# Usar ``TypeAlias`` diz ao verificador de tipo que isso é uma declaração de␣
↪apelido de tipo,
# e não uma atribuição de variável a string.
BoxOfStrings: TypeAlias = "Box[str]"

class Box(Generic[T]):
    @classmethod
    def make_box_of_strings(cls) -> BoxOfStrings: ...
```

Veja **PEP 613** para mais detalhes.

Adicionado na versão 3.10.

Descontinuado desde a versão 3.12: *TypeAlias* foi descontinuado em favor da instrução `type`, que cria instâncias de *TypeAliasType* e oferece suporte nativo a referências futuras. Observe que embora *TypeAlias* e *TypeAliasType* sirvam propósitos semelhantes e tenham nomes semelhantes, eles são distintos e o último

não é o tipo do primeiro. A remoção de *TypeAlias* não está planejada atualmente, mas os usuários são encorajados a migrar para instruções `type`.

Formas especiais

Eles podem ser usados como tipos em anotações. Todos eles oferecem suporte a subscrição usando `[]`, mas cada um tem uma sintaxe única.

class typing.**Union**

Tipo de união; `Union[X, Y]` é equivalente a `X | Y` e significa X ou Y.

Para definir uma união, use, por exemplo. `Union[int, str]` ou a abreviatura `int | str`. Usar essa abreviação é recomendado. Detalhes:

- Os argumentos devem ser tipos e deve haver pelo menos um.
- As uniões de uniões são achatadas, por exemplo:

```
Union[Union[int, str], float] == Union[int, str, float]
```

  Entretanto, isso não se aplica a uniões referenciadas por meio de um apelido de tipo, para evitar forçar a avaliação da *TypeAliasType* subjacente:

```
type A = Union[int, str]
Union[A, float] != Union[int, str, float]
```

- As uniões de um único argumento desaparecem, por exemplo:

```
Union[int] == int  # O construtor retorna um int
```

- Argumento redundantes são pulados, e.g.:

```
Union[int, str, int] == Union[int, str] == int | str
```

- Ao comparar uniões, a ordem de argumentos é ignorada. Por exemplo:

```
Union[int, str] == Union[str, int]
```

- Você não pode estender ou instanciar uma `Union`
- Você não pode escrever `Union[X][Y]`.

Alterado na versão 3.7: Não remova subclasses explícitas de uniões em tempo de execução.

Alterado na versão 3.10: Uniões agora podem ser escritas com `X | Y`. Veja *expressões de união de tipos*.

Alterado na versão 3.14: *types.UnionType* agora é um apelido para *Union*, e tanto `Union[int, str]` quanto `int | str` criam instâncias da mesma classe. Para verificar se um objeto é um `Union` em tempo de execução, use `isinstance(obj, Union)`. Para compatibilidade com versões anteriores do Python, use `get_origin(obj) is typing.Union or get_origin(obj) is types.UnionType`.

typing.**Optional**

`Optional[X]` equivale a `X | None` (ou `Union[X, None]`).

Note que isso não é o mesmo conceito de um argumento opcional, que possui um valor por padrão. Um argumento opcional com padrão não requer o qualificador `Optional` em sua anotação de tipo só por ser opcional. Por exemplo:

```
def foo(arg: int = 0) -> None:
    ...
```

Por outro lado, se um valor explícito de `None` for permitido, o uso de `Optional` é apropriado, seja o argumento opcional ou não. Por exemplo:

```
def foo(arg: Optional[int] = None) -> None:
    ...
```

Alterado na versão 3.10: Optional agora pode ser escrito como `X | None`. Veja *expressões de união de tipos*.

typing.**Concatenate**

Forma especial para anotar funções de ordem superior.

`Concatenate` pode ser usado em conjunção com *Callable* e *ParamSpec* para anotar um chamável de maior ordem que adiciona, remove ou transforma parâmetros de outro chamável. Seu uso é feito na forma `Concatenate[Arg1Type, Arg2Type, ..., ParamSpecVariable]`, `Concatenate` atualmente só é válido quando usado como primeiro argumento de um *Callable*. O último parâmetro de `Concatenate` deve ser um *ParamSpec* ou reticências (`...`).

Por exemplo, para anotar um decorador `with_lock` que oferece uma instância de *threading.Lock* para a função decorada, `Concatenate` pode ser usado para indicar que `with_lock` espera um chamável cujo primeiro argumento tem tipo `Lock`, e retorna um chamável com uma assinatura de tipos diferente. Neste caso, o *ParamSpec* indica que os tipos dos parâmetros do chamável retornado dependem dos tipos dos parâmetros do chamável de entrada:

```
from collections.abc import Callable
from threading import Lock
from typing import Concatenate

# Use esta trava para se certificar de que
# uma única thread a executa a qualquer momento.
my_lock = Lock()

def with_lock[**P, R](f: Callable[Concatenate[Lock, P], R]) -> Callable[P, R]:
    '''Um decorador com segurança de tipos que oferece uma trava.'''
    def inner(*args: P.args, **kwargs: P.kwargs) -> R:
        # Oferece a trava como primeiro argumento.
        return f(my_lock, *args, **kwargs)
    return inner

@with_lock
def sum_threadsafe(lock: Lock, numbers: list[float]) -> float:
    '''Somatório de uma lista de números de modo seguro para threads.'''
    with lock:
        return sum(numbers)

# We don't need to pass in the lock ourselves thanks to the decorator.
sum_threadsafe([1.1, 2.2, 3.3])
```

Adicionado na versão 3.10.

Ver também

- **PEP 612** – Variáveis de especificação de parâmetro (a PEP que introduz `ParamSpec` e `Concatenate`)
- *ParamSpec*
- *Anotações de objetos chamáveis*

typing.**Literal**

Forma especial de tipagem para definir “tipos literais”.

`Literal` pode ser usado para indicar aos verificadores de tipo que o objeto anotado tem valor equivalente a algum dos literais oferecidos.

Por exemplo:

```
def validate_simple(data: Any) -> Literal[True]:  # sempre retorna True
    ...

type Mode = Literal['r', 'rb', 'w', 'wb']
def open_helper(file: str, mode: Mode) -> str:
    ...

open_helper('/some/path', 'r')      # Passa verificação de tipos
open_helper('/other/path', 'typo')  # Erro em verificação de tipos
```

`Literal[...]` não é subclasse. Em tempo de execução, permite-se um valor arbitrário como argumento de tipo para `Literal[...]`, mas verificadores de tipo podem impor restrições. Veja **PEP 586** para mais detalhes sobre tipos literais.

Detalhes adicionais:

- Os argumentos devem ser valores literais e deve haver pelo menos um.
- Tipos `Literal` aninhados são achatados, por exemplo:

  ```
  assert Literal[Literal[1, 2], 3] == Literal[1, 2, 3]
  ```

  Entretanto, isso não se aplica a tipos `Literal` referenciadas por meio de um apelido de tipo, para evitar forçar a avaliação da *`TypeAliasType`* subjacente:

  ```
  type A = Literal[1, 2]
  assert Literal[A, 3] != Literal[1, 2, 3]
  ```

- Argumento redundantes são pulados, e.g.:

  ```
  assert Literal[1, 2, 1] == Literal[1, 2]
  ```

- Ao comparar literais, a ordem de argumentos é ignorada. Por exemplo:

  ```
  assert Literal[1, 2] == Literal[2, 1]
  ```

- Você não pode estender ou instanciar uma `Literal`.
- Você não pode escrever `Literal[X][Y]`.

Adicionado na versão 3.8.

Alterado na versão 3.9.1: `Literal` agora remove parâmetros duplicados. Comparações de igualdade entre objetos `Literal` não dependem da ordem. Objetos `Literal` agora levantam uma exceção *`TypeError`* durante comparações de igualdade se um de seus parâmetros não for *hasheável*.

`typing.`**`ClassVar`**

Uma construção especial de tipagem para marcar variáveis de classe.

Como introduzido na **PEP 526**, uma variável cuja anotação de tipo tem um invólucro ClassVar indica que um dado atributo deve ser usado como uma variável de classe, e que ele não deve ser definido em instâncias dessa classe. Modo de usar:

```
class Starship:
    stats: ClassVar[dict[str, int]] = {} # variável de classe
    damage: int = 10                     # variável de instância
```

`ClassVar` aceita apenas tipos e não pode ser subscrita posteriormente.

ClassVar não é uma classe, e não deve ser usada com *isinstance()* ou *issubclass()*. *ClassVar* não muda com o comportamento do Python em tempo de execução, mas pode ser usada por verificadores de tipos de terceiros. Por exemplo, um verificador de tipos pode sinalizar que o seguinte código é errado:

```
enterprise_d = Starship(3000)
enterprise_d.stats = {} # Erro: atribuindo valor a variável de classe de uma␣
↪instância
Starship.stats = {}     # Isso é OK
```

Adicionado na versão 3.5.3.

Alterado na versão 3.13: *ClassVar* agora pode ser aninhada em *Final* e vice-versa.

typing.**Final**

Uma construção especial de tipagem para indicar nomes finais a verificadores de tipos

Nomes finais não podem ser reatribuídos em qualquer escopo. Nomes finais declarados em escopos de classe não podem ser substituídos em subclasses.

Por exemplo:

```
MAX_SIZE: Final = 9000
MAX_SIZE += 1  # Erro reportado pelo verificador de tipos

class Connection:
    TIMEOUT: Final[int] = 10

class FastConnector(Connection):
    TIMEOUT = 1  # Error reportado pelo verificador de tipos
```

Não há verificação em tempo de execução dessas propriedades. Veja **PEP 591** para mais detalhes.

Adicionado na versão 3.8.

Alterado na versão 3.13: *Final* agora pode ser aninhada em *ClassVar* e vice-versa.

typing.**Required**

Uma construção especial de tipagem para marcar uma chave de *TypedDict* como necessária.

Isso é útil principalmente para TypedDicts com `total=False`. Veja *TypedDict* e **PEP 655** para obter mais detalhes.

Adicionado na versão 3.11.

typing.**NotRequired**

Uma construção especial de tipagem para marcar uma chave de *TypedDict* como potencialmente ausente.

Veja *TypedDict* e **PEP 655** para obter mais detalhes.

Adicionado na versão 3.11.

typing.**ReadOnly**

Uma construção especial de tipagem para marcar um item de um *TypedDict* como somente leitura.

Por exemplo:

```
class Movie(TypedDict):
   title: ReadOnly[str]
   year: int

def mutate_movie(m: Movie) -> None:
   m["year"] = 1999  # permitido
   m["title"] = "The Matrix"  # erro no verificador de tipos
```

Não há verificação em tempo de execução para esta propriedade.

Consulte *TypedDict* e **PEP 705** para obter mais detalhes.

Adicionado na versão 3.13.

typing.**Annotated**

Forma especial de tipagem para adicionar metadados específicos de contexto para uma anotação.

Adiciona metadados `x` a um determinado tipo `T` usando a anotação `Annotated[T, x]`. Os metadados adicionados usando `Annotated` podem ser usados por ferramentas de análise estática ou em tempo de execução. Em tempo de execução, os metadados são armazenados em um atributo `__metadata__`.

Se uma biblioteca ou ferramenta encontrar uma anotação `Annotated[T, x]` e ela não tiver lógica especial para os metadados, ela deverá ignorar os metadados e simplesmente tratar a anotação como `T`. Dessa forma, `Annotated` pode ser útil em códigos que desejarem usar a anotação para propósitos fora do sistema de tipagem estática do Python.

Usar `Annotated[T, x]` como um anotação ainda permite a verificação estática do tipo de `T`, pois os verificadores de tipo simplesmente ignoram os metadados em `x`. Dessa forma, o `Annotated` difere do decorador *@no_type_check*, que também pode ser usado para adicionar anotações fora do escopo do sistema de tipagem, mas desativa completamente a verificação de tipos para uma função ou classe.

A responsabilidade de como interpretar os metadados é da ferramenta ou da biblioteca que encontrar uma anotação `Annotated`. Uma ferramenta ou biblioteca que encontrar um tipo `Annotated` poderá examinar os elementos de metadados para determinar se eles são de interesse (por exemplo, usando *isinstance()*).

Annotated[<type>, <metadata>]

Aqui está um exemplo de como você pode usar `Annotated` para adicionar metadados a anotações de tipos se estiver fazendo uma análise de intervalos:

```
@dataclass
class ValueRange:
    lo: int
    hi: int

T1 = Annotated[int, ValueRange(-10, 5)]
T2 = Annotated[T1, ValueRange(-20, 3)]
```

O primeiro argumento para `Annotated` deve ser um tipo válido. Vários elementos de metadados podem ser fornecidos, pois `Annotated` suporta argumentos variádicos. A ordem dos elementos de metadados é preservada e importa para verificações de igualdade:

```
@dataclass
class ctype:
     kind: str

a1 = Annotated[int, ValueRange(3, 10), ctype("char")]
a2 = Annotated[int, ctype("char"), ValueRange(3, 10)]

assert a1 != a2  # a ordem importa
```

Cabe à ferramenta que consome as anotações decidir se o cliente tem permissão de adicionar vários elementos de metadados a uma anotação e como mesclar essas anotações.

Os tipos `Annotated` aninhados são achatados. A ordem dos elementos de metadados começa com a anotação mais interna:

```
assert Annotated[Annotated[int, ValueRange(3, 10)], ctype("char")] ==␣
↪Annotated[
    int, ValueRange(3, 10), ctype("char")
]
```

Entretanto, isso não se aplica a tipos `Annotated` referenciadas por meio de um apelido de tipo, para evitar forçar a avaliação da *`TypeAliasType`* subjacente:

```
type From3To10[T] = Annotated[T, ValueRange(3, 10)]
assert Annotated[From3To10[int], ctype("char")] != Annotated[
   int, ValueRange(3, 10), ctype("char")
]
```

Elementos duplicados de metadata não são removidos:

```
assert Annotated[int, ValueRange(3, 10)] != Annotated[
    int, ValueRange(3, 10), ValueRange(3, 10)
]
```

`Annotated` pode ser usado com apelidos aninhados e apelidos genéricos:

```
@dataclass
class MaxLen:
    value: int

type Vec[T] = Annotated[list[tuple[T, T]], MaxLen(10)]

# Quando usado em uma anotação de tipo, um verificador de tipos␣
↪tratará "V" da mesma forma que
# ``Annotated[list[tuple[int, int]], MaxLen(10)]``:
type V = Vec[int]
```

`Annotated` não pode ser usado com um *`TypeVarTuple`* desempacotado:

```
type Variadic[*Ts] = Annotated[*Ts, Ann1] = Annotated[T1, T2, T3, ..., Ann1]
↪# NÃO válido
```

onde `T1`, `T2`, … são *`TypeVars`*. Isso é inválido, pois somente um tipo deve ser passado para Annotated.

Por padrão, *`get_type_hints()`* retira os metadados de anotações. Passe `include_extras=True` para preservar os metadados:

```
>>> from typing import Annotated, get_type_hints
>>> def func(x: Annotated[int, "metadata"]) -> None: pass
...
>>> get_type_hints(func)
{'x': <class 'int'>, 'return': <class 'NoneType'>}
>>> get_type_hints(func, include_extras=True)
{'x': typing.Annotated[int, 'metadata'], 'return': <class 'NoneType'>}
```

Em tempo de execução, os metadados associados a um tipo `Annotated` podem ser recuperados por meio do atributo `__metadata__`:

```
>>> from typing import Annotated
>>> X = Annotated[int, "very", "important", "metadata"]
>>> X
typing.Annotated[int, 'very', 'important', 'metadata']
>>> X.__metadata__
('very', 'important', 'metadata')
```

Se você quiser recuperar o tipo original envolvido por `Annotated`, use o atributo `__origin__`:

```
>>> from typing import Annotated, get_origin
>>> Password = Annotated[str, "secret"]
```

(continua na próxima página)

(continuação da página anterior)

```
>>> Password.__origin__
<class 'str'>
```

Note que usar *get_origin()* retornará o próprio `Annotated`:

```
>>> get_origin(Password)
typing.Annotated
```

Ver também

PEP 593 - Anotações flexíveis para funções e variáveis
A PEP que introduz `Annotated` à biblioteca padrão.

Adicionado na versão 3.9.

`typing.`**`TypeIs`**

Uma construção especial de tipagem para marcar funções de predicado de tipo definidas pelo usuário.

`TypeIs` pode ser usado para anotar o tipo de retorno de uma função de predicado de tipo definida pelo usuário. `TypeIs` aceita apenas um único argumento de tipo. Em tempo de execução, funções marcadas dessa forma devem retornar um booleano e receber, no mínimo, um argumento posicional.

`TypeIs` visa beneficiar *estreitamento de tipo* – uma técnica usada por verificador de tipo estático para determinar um tipo mais preciso de expressão dentro do fluxo de código de um programa. Normalmente, o estreitamento de tipo é feito por meio da análise do fluxo de código condicional e da aplicação do estreitamento a um bloquear código. A condicional expressão aqui é às vezes chamada de "predicado de tipo":

```
def is_str(val: str | float):
    # predicado de tipos "isinstance"
    if isinstance(val, str):
        # Tipo de ``val`` é restrito a ``str``
        ...
    else:
        # Senão, tipo de ``val`` é restrito a ``float``.
        ...
```

Às vezes, seria conveniente usar uma função booleana definida pelo usuário como um predicado de tipo. Essa função deveria usar `TypeIs[...]` ou *TypeGuard* como seu tipo de retorno para alertar o verificador de tipo estático sobre essa intenção. `TypeIs` geralmente tem um comportamento mais intuitivo do que `TypeGuard`, mas não pode ser usado quando os tipos de entrada e saída são incompatíveis (por exemplo, de `list[object]` para `list[int]`) ou quando a função não retorna `True` para todas as instâncias do tipo restrito.

Usar `-> TypeIs[NarrowedType]` informa ao verificador de tipo estático que, para uma determinada função:

1. O valor de retorno é um booleano.
2. Se o valor de retorno for `True`, o tipo de seu argumento será a interseção entre o tipo original do argumento e o `NarrowedType`.
3. Se o valor de retorno for `False`, o tipo de seu argumento será restrito para excluir `NarrowedType`.

Por exemplo:

```
from typing import assert_type, final, TypeIs

class Parent: pass
class Child(Parent): pass
```

(continua na próxima página)

(continuação da página anterior)

```
@final
class Unrelated: pass

def is_parent(val: object) -> TypeIs[Parent]:
    return isinstance(val, Parent)

def run(arg: Child | Unrelated):
    if is_parent(arg):
        # Tipo de ``arg`` é restrito à interseção
        # de ``Parent`` e ``Child``, o que equivale a
        # ``Child``.
        assert_type(arg, Child)
    else:
        # Tipo de ``arg`` é restrito para excluir ``Parent``,
        # então sobrou o tipo ``Unrelated``.
        assert_type(arg, Unrelated)
```

O tipo dentro de `TypeIs` deve ser consistente com o tipo do argumento da função; caso contrário, o verificador de tipo estático levantará um erro. Uma função `TypeIs` escrita incorretamente pode ocasionar um comportamento insensato pelo sistema de tipos; é de responsabilidade do usuário escrever tais funções de forma segura para tipos.

Se uma função `TypeIs` for um método de classe ou instância, então o tipo em `TypeIs` mapeia para o tipo do segundo parâmetro (depois de `cls` ou `self`).

Em resumo, a forma `def foo(arg: TypeA) -> TypeIs[TypeB]: ...`, significa que se `foo(arg)` retorna `True`, então `arg` é um instância de `TypeB`, e se retorna `False`, não é um instância de `TypeB`.

`TypeIs` também funciona com variáveis de tipo. Para mais informações, consulte a **PEP 742** (Restringindo tipos com `TypeIs`).

Adicionado na versão 3.13.

typing.**TypeGuard**

Uma construção especial de tipagem para marcar funções de predicado de tipo definidas pelo usuário.

Funções de predicados de tipo são funções definidas pelo usuário que retornam se seu argumento é uma instância de um tipo específico. O `TypeGuard` funciona como *`TypeIs`*, mas tem efeitos sutilmente diferentes sobre o comportamento da verificação de tipos (veja abaixo).

Usar `-> TypeGuard` informa ao verificador de tipo estático que, para uma determinada função:

1. O valor de retorno é um booleano.
2. Se o valor de retorno for `True`, o tipo de seu argumento é o tipo dentro de `TypeGuard`.

`TypeGuard` também funciona com tipos variáveis. Consulte a **PEP 647** para obter mais detalhes.

Por exemplo:

```
def is_str_list(val: list[object]) -> TypeGuard[list[str]]:
    '''Determina se todos os objetos na lista são strings'''
    return all(isinstance(x, str) for x in val)

def func1(val: list[object]):
    if is_str_list(val):
        # Tipo de ``val`` é restrito a ``list[str]``.
        print(" ".join(val))
    else:
        # Tipo de ``val`` continua sendo ``list[object]``.
        print("Not a list of strings!")
```

`TypeIs` e `TypeGuard` diferem das seguintes maneiras:

- `TypeIs` exige que o tipo restrito seja um subtipo do tipo de entrada, enquanto o `TypeGuard`, não. O motivo principal é permitir coisas como restringir `list[object]` a `list[str]`, embora este não seja um subtipo daquele, já que `list` é invariante.
- Quando uma função `TypeGuard` retorna `True`, os verificadores de tipo restringem o tipo da variável exatamente ao tipo `TypeGuard`. Quando uma função `TypeIs` retorna `True`, os verificadores de tipo podem inferir um tipo mais preciso combinando o tipo previamente conhecido da variável com o tipo `TypeIs`. (Tecnicamente, isso é conhecido como um tipo de interseção).
- Quando uma função `TypeGuard` retorna `False`, os verificadores de tipo não podem restringir o tipo da variável. Quando uma função `TypeIs` retorna `False`, os verificadores de tipo podem restringir o tipo da variável para excluir o tipo `TypeIs`.

Adicionado na versão 3.10.

`typing.`**`Unpack`**

Operador de tipagem para marcar conceitualmente um objeto como tendo sido desempacotado.

Por exemplo, usar o operador de desempacotamento `*` em uma *tupla de tipos variáveis* equivale a usar o `Unpack` para marcar a tupla de tipos variáveis como tendo sido desempacotada:

```
Ts = TypeVarTuple('Ts')
tup: tuple[*Ts]
# Efetivamente faz:
tup: tuple[Unpack[Ts]]
```

Na verdade, `Unpack` pode ser usado de forma intercambiável com `*` no contexto dos tipos *typing.TypeVarTuple* e *builtins.tuple*. Você pode ver `Unpack` sendo usado explicitamente em versões mais antigas do Python, onde `*` não podia ser usado em certos lugares:

```
# Em versões mais antigas do Python, TypeVarTuple e Unpack
# estão localizados no pacote de backports `typing_extensions`.
from typing_extensions import TypeVarTuple, Unpack

Ts = TypeVarTuple('Ts')
tup: tuple[*Ts]         # Erro de sintaxe no Python <= 3.10!
tup: tuple[Unpack[Ts]]  # Semanticamente equivalente e compatível com versões␣
↪anteriores
```

`Unpack` também pode ser usado com *typing.TypedDict* para a tipagem de `**kwargs` em uma assinatura de função:

```
from typing import TypedDict, Unpack

class Movie(TypedDict):
    name: str
    year: int

# Esta função espera dois argumentos nomeados: `name` de tipo `str`
# e `year` de tipo `int`.
def foo(**kwargs: Unpack[Movie]): ...
```

Consulte **PEP 692** para obter mais detalhes sobre como usar `Unpack` para tipagem de `**kwargs`.

Adicionado na versão 3.11.

Criando tipos genéricos e apelidos de tipo

As classes a seguir não devem ser usadas diretamente como anotações. propósito O objetivo delas é construir bloquear para criar genérico e digitar apelido.

Esses objetos podem ser criados por meio de sintaxe especial (listas de parâmetros de tipo e a instrução `type`). Para compatibilidade com Python 3.11 e versões anteriores, eles também podem ser criados sem a sintaxe dedicada, como documentado abaixo.

class typing.**Generic**

Classe base abstrata para tipos genéricos

Normalmente, um tipo genérico é declarado adicionando-se uma lista de parâmetros de tipo após o nome da classe:

```
class Mapping[KT, VT]:
    def __getitem__(self, key: KT) -> VT:
        ...
        # Etc.
```

Essa classe herda implicitamente de `Generic`. A semântica em tempo de execução dessa sintaxe é discutida na Referência da Linguagem.

Esta classe pode ser utilizada como segue:

```
def lookup_name[X, Y](mapping: Mapping[X, Y], key: X, default: Y) -> Y:
    try:
        return mapping[key]
    except KeyError:
        return default
```

Aqui os colchetes depois no nome da função indica uma função genérica.

Para manter retrocompatibilidade, as classes genéricas também podem ser declaradas herdando explicitamente de `Generic`. Nesse caso, os parâmetros de tipo devem ser declarados separadamente:

```
KT = TypeVar('KT')
VT = TypeVar('VT')

class Mapping(Generic[KT, VT]):
    def __getitem__(self, key: KT) -> VT:
        ...
        # Etc.
```

class typing.**TypeVar**(*name*, **constraints*, *bound=None*, *covariant=False*, *contravariant=False*, *infer_variance=False*, *default=typing.NoDefault*)

Tipo variável.

A maneira preferida de construir um tipo variável é por meio da sintaxe dedicada para funções genéricas, classes genéricas e apelidos de tipo genérico:

```
class Sequence[T]:  # T é um TypeVar
    ...
```

Essa sintaxe também pode ser usada para criar tipos variáveis delimitados e tipos variáveis restritos:

```
class StrSequence[S: str]:  # S é a TypeVar com como limite superior em `str`;
    ...                     # podemos dizer que S é "delimitada por `str`"


class StrOrBytesSequence[A: (str, bytes)]:  # A é uma TypeVar restrita a str␣
```

(continua na próxima página)

(continuação da página anterior)

```
↪ou bytes
    ...
```

No entanto, se desejado, tipos variáveis reutilizáveis também podem ser construídas manualmente, como:

```
T = TypeVar('T') # Pode ser qualquer coisa
S = TypeVar('S', bound=str) # Pode ser qualquer subtipo de str
A = TypeVar('A', str, bytes) # Deve ser exatamente str ou bytes
```

Tipos variáveis existem principalmente para o benefício de verificadores de tipo estático. Eles servem como parâmetros para tipos genéricos, bem como para definições de função genérica e apelidos de tipo. Consulte *Generic* para obter mais informações sobre genérico. Funções genéricas funcionam da seguinte forma:

```
def repeat[T](x: T, n: int) -> Sequence[T]:
    """Retorna uma lista contendo n referências para x."""
    return [x]*n


def print_capitalized[S: str](x: S) -> S:
    """Exibe x capitalizado, e retorna x."""
    print(x.capitalize())
    return x


def concatenate[A: (str, bytes)](x: A, y: A) -> A:
    """Concatena dois objetos string ou bytes."""
    return x + y
```

Observe que tipos variáveis podem ser *delimitados*, *restritos*, nenhum dos dois, mas não podem ser *ambos*.

A variância de tipos variáveis é inferida pelos verificadores de tipo quando eles são criados por meio do sintaxe de parâmetro de tipo ou quando `infer_variance=True` é passado. Os tipos variáveis criados manualmente podem ser explicitamente marcados como covariante ou contravariantes ao passar `covariant=True` ou `contravariant=True`. Por padrão, os tipos variáveis criados manualmente são invariante. Consulte **PEP 484** e **PEP 695** para obter mais detalhes.

Tipos variáveis delimitados e tipos variáveis restritos têm semânticas diferentes de várias formas importantes. Usar um tipo variável *delimitado* significa que a `TypeVar` será resolvido usando o tipo mais específico possível:

```
x = print_capitalized('a string')
reveal_type(x)  # tipo revelado é str

class StringSubclass(str):
    pass

y = print_capitalized(StringSubclass('another string'))
reveal_type(y)  # tipo revelado é StringSubclass

z = print_capitalized(45)  # erro: int não é subtipo de str
```

O limite superior de um tipo variável pode ser um tipo concreto, um tipo abstrato (classe base abstrata ou protocolo) ou até mesmo uma união de tipos:

```
# Pode ser qualquer coisa com um método __abs__
def print_abs[T: SupportsAbs](arg: T) -> None:
    print("Absolute value:", abs(arg))

U = TypeVar('U', bound=str|bytes)  # Pode ser qualquer subtipo com a união str␣
```

(continua na próxima página)

(continuação da página anterior)

```
↪| bytes
V = TypeVar('V', bound=SupportsAbs)  # Pode ser qualquer coisa com um método __
↪abs__
```

Porém, usar um tipo variável *restrito* significa que a `TypeVar` só poderá ser resolvida como sendo exatamente uma das restrições dadas:

```
a = concatenate('one', 'two')
reveal_type(a) # o tipo revelado é str

b = concatenate(StringSubclass('one'), StringSubclass('two'))
reveal_type(b) # o tipo revelado é str, apesar de StringSubclass ter sido␣
↪passado

c = concatenate('one', b'two') # erro: tipo variável 'A' pode ser str ou bytes␣
↪em uma chamada de função, mas não ambos
```

Em tempo de execução, `isinstance(x, T)` levantará *TypeError*.

__name__

O nome do tipo variável.

__covariant__

Se o tipo variável foi explicitamente marcado como covariante.

__contravariant__

Se o tipo variável foi explicitamente marcado como contravariante.

__infer_variance__

Se a variância de tipo variável deve ser inferida por verificadores de tipo.

Adicionado na versão 3.12.

__bound__

A limite superior do tipo variável, se houver.

Alterado na versão 3.12: Para tipos variáveis criado por meio da sintaxe de parâmetro de tipo , a limite é avaliado somente quando o atributo é acessado, não quando o tipo variável é criado (consulte lazy--evaluation).

evaluate_bound()

Uma *função de avaliação* correspondente ao atributo *__bound__*. Quando chamado diretamente, este método oferece suporte apenas ao formato *VALUE*, que equivale a acessar o atributo *__bound__* diretamente, mas o objeto do método pode ser passado para *annotationlib.call_evaluate_function()* para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

__constraints__

Um tupla contendo as restrições do tipo variável, se houver.

Alterado na versão 3.12: Para tipos variáveis criado por meio da sintaxe de parâmetro de tipo , a restrição é avaliada somente quando o atributo é acessado, não quando o tipo variável é criado (consulte lazy--evaluation).

evaluate_constraints()

Uma *função de avaliação* correspondente ao atributo *__constraints__*. Quando chamado diretamente, este método oferece suporte apenas ao formato *VALUE*, que equivale a acessar o atributo *__constraints__* diretamente, mas o objeto do método pode ser passado para *annotationlib.call_evaluate_function()* para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

__default__

O padrão valor do tipo variável, ou *typing.NoDefault* se ele não tiver padrão.

Adicionado na versão 3.13.

evaluate_default()

Uma *função de avaliação* correspondente ao atributo *__default__*. Quando chamado diretamente, este método oferece suporte apenas ao formato *VALUE*, que equivale a acessar o atributo *__default__* diretamente, mas o objeto do método pode ser passado para *annotationlib.call_evaluate_function()* para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

has_default()

Retorna se o tipo variável tem ou não um valor padrão. Isso equivale a verificar se *__default__* não é o singleton *typing.NoDefault*, exceto por não forçar a avaliação instantânea do valor padrão que é avaliado preguiçosamente.

Adicionado na versão 3.13.

Alterado na versão 3.12: Tipos variáveis agora podem ser declarados usando a sintaxe de parâmetro de tipo introduzida por **PEP 695**. O parâmetro `infer_variance` foi adicionado.

Alterado na versão 3.13: Adiciona suporte a valores padrão.

class typing.**TypeVarTuple**(*name*, *, *default=typing.NoDefault*)

Tupla de tipo variável. Uma forma especializada de *tipo variável* que permite genéricos *variádicos*.

Tuplas de tipo variável podem ser declarada em listas de parâmetros de tipo usando um único asterisco (`*`) antes do nome:

```
def move_first_element_to_last[T, *Ts](tup: tuple[T, *Ts]) -> tuple[*Ts, T]:
    return (*tup[1:], tup[0])
```

Ou invocando o construtor `TypeVarTuple` explicitamente:

```
T = TypeVar("T")
Ts = TypeVarTuple("Ts")

def move_first_element_to_last(tup: tuple[T, *Ts]) -> tuple[*Ts, T]:
    return (*tup[1:], tup[0])
```

Um tipo variável normal permite a parametrização com um único tipo. Uma tupla de tipos variáveis, por outro lado, permite a parametrização com um número *arbitrário* de tipos, agindo como um número *arbitrário* de tipos variáveis envolvidos em um tupla. Por exemplo:

```
# T é limitado a int, Ts é limitado a ()
# Valor retornado é (1,), que tem tipo tuple[int]
move_first_element_to_last(tup=(1,))

# T é limitado a int, Ts é limitado a (str,)
# Valor retornado é ('spam', 1), que tem tipo tuple[str, int]
move_first_element_to_last(tup=(1, 'spam'))

# T é limitado a int, Ts é limitado a (str, float)
# Valor retornado é ('spam', 3.0, 1), que tem tipo tuple[str, float, int]
move_first_element_to_last(tup=(1, 'spam', 3.0))

# Isso é um erro para o verificador de tipos (e falha em tempo de execução)
# pois tuple[()] não é compatível com tuple[T, *Ts]
# (pelo menos um elemento é necessário)
move_first_element_to_last(tup=())
```

Observe o uso do operador de desempacotamento `*` em `tuple[T, *Ts]`. Conceitualmente, você pode interpretar `Ts` como uma tupla de tipos variáveis `(T1, T2, ...)`. Então o tipo `tuple[T, *Ts]` se tornaria `tuple[T, *(T1, T2, ...)]`, que equivale a `tuple[T, T1, T2, ...]`. (Note que, em versões mais antigas de Python, você pode encontrar isso escrito com *`Unpack`* em vez de `Unpack[Ts]`.)

Tuplas de tipos variáveis devem *sempre* ser desempacotadas. Isso ajuda a distinguir entre tuplas de tipos variáveis e tipos variáveis normais:

```
x: Ts          # Inválido
x: tuple[Ts]   # Inválido
x: tuple[*Ts]  # O jeito correto
```

As tuplas de tipos variáveis podem ser usadas no mesmo contexto que tipos variáveis normais. Por exemplo, em argumentos, tipos de retorno e definições de classes:

```
class Array[*Shape]:
    def __getitem__(self, key: tuple[*Shape]) -> float: ...
    def __abs__(self) -> "Array[*Shape]": ...
    def get_shape(self) -> tuple[*Shape]: ...
```

As tuplas de tipos variáveis podem ser combinadas com tipos variáveis normais:

```
class Array[DType, *Shape]:  # Tudo certo aqui
    pass

class Array2[*Shape, DType]:  # Tudo certo aqui também
    pass

class Height: ...
class Width: ...

float_array_1d: Array[float, Height] = Array()      # Totalmente certo
int_array_2d: Array[int, Height, Width] = Array()   # Sim, certo também
```

No entanto, observe que no máximo uma tupla de tipos variáveis pode aparecer em uma única lista de argumentos de tipo ou parâmetros de tipo:

```
x: tuple[*Ts, *Ts]              # Inválido
class Array[*Shape, *Shape]:  # Inválido
    pass
```

Por fim, uma tupla de tipos variáveis desempacotada pode ser usada como anotação de tipo de `*args`:

```
def call_soon[*Ts](
    callback: Callable[[*Ts], None],
    *args: *Ts
) -> None:
    ...
    callback(*args)
```

Ao contrário do que acontece com anotações não desempacotadas de `*args` - por exemplo, `*args: int`, que especificaria que *todos* os argumentos são `int` -, `*args: *Ts` permite referenciar os tipos de cada argumento em `*args` *individualmente*. Isso nos permite garantir que os tipos em `*args` passados para `call_soon` correspondem aos tipos dos argumentos (posicionais) de `callback`.

Consulte a **PEP 646** para mais detalhes sobre tuplas de tipos variáveis.

__name__

O nome da tupla de tipos variáveis.

__default__

O valor padrão da tupla de tipos variáveis, ou *typing.NoDefault* se não existir padrão.

Adicionado na versão 3.13.

evaluate_default()

Uma *função de avaliação* correspondente ao atributo *__default__*. Quando chamado diretamente, este método oferece suporte apenas ao formato *VALUE*, que equivale a acessar o atributo *__default__* diretamente, mas o objeto do método pode ser passado para *annotationlib.call_evaluate_function()* para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

has_default()

Retorna se a tupla de tipos variáveis tem ou não um valor padrão. Isso equivale a verificar se *__default__* não é o singleton *typing.NoDefault*, exceto por não forçar a avaliação instantânea do valor padrão que é avaliado preguiçosamente.

Adicionado na versão 3.13.

Adicionado na versão 3.11.

Alterado na versão 3.12: Tuplas de tipos variáveis agora podem ser declaradas usando a sintaxe de parâmetros de tipo introduzido pela **PEP 695**.

Alterado na versão 3.13: Adiciona suporte a valores padrão.

class typing.**ParamSpec**(*name*, *, *bound=None*, *covariant=False*, *contravariant=False*, *default=typing.NoDefault*)

Variável de especificação de parâmetro. Uma versão especializada de *tipos variáveis*.

Em listas de parâmetros de tipo as especificações de parâmetros podem ser declaradas com dois asteriscos (`**`):

```
type IntFunc[**P] = Callable[P, int]
```

Para compatibilidade com Python 3.11 e versões anteriores, os objetos `ParamSpec` também podem ser criados da seguinte forma:

```
P = ParamSpec('P')
```

Variáveis de especificação de parâmetro existem principalmente para o benefício de verificadores de tipo estático. São usadas para encaminhar os tipos de parâmetros de um chamável para outro chamável – um padrão comumente encontrado em funções e decoradores de ordem superior. Só são válidas quando usados em `Concatenate`, ou como o primeiro argumento para `Callable`, ou como parâmetro para genéricos definidos pelo usuário. Consulte *Generic* para obter mais informações sobre tipos genéricos.

Por exemplo, para adicionar um registro básico de eventos a uma função, é possível criar um decorador `add_logging` para registrar chamadas de função. A variável de especificação de parâmetro informa ao verificador de tipos que o chamável passado para o decorador e o novo chamável retornado por ele têm parâmetros de tipo interdependentes:

```
from collections.abc import Callable
import logging

def add_logging[T, **P](f: Callable[P, T]) -> Callable[P, T]:
    '''Um decorador com segurança de tipos que acrescenta registros de eventos
↪(logs) a uma função.'''
    def inner(*args: P.args, **kwargs: P.kwargs) -> T:
        logging.info(f'{f.__name__} was called')
        return f(*args, **kwargs)
    return inner
```

(continua na próxima página)

(continuação da página anterior)

```
@add_logging
def add_two(x: float, y: float) -> float:
    '''Soma dois números.'''
    return x + y
```

Sem o `ParamSpec`, a maneira mais simples de anotar isso anteriormente era usar um *`TypeVar`* com o limite superior `Callable[..., Any]`. No entanto, isso causa dois problemas:

1. O verificador de tipos não consegue verificar a função `inner`, porque `*args` e `**kwargs` precisam ter tipo *`Any`*.
2. *`cast()`* pode ser exigida no corpo do decorador `add_logging` ao retornar a função `inner`, ou o verificador de tipo estático deverá ser instruído a ignorar o `return inner`.

args

kwargs

Como `ParamSpec` captura tanto parâmetros posicionais quanto parâmetros nomeados, `P.args` e `P.kwargs` podem ser usados para dividir um `ParamSpec` em seus componentes. `P.args` representa a tupla de parâmetros posicionais em uma determinada chamada e só deve ser usada para anotar `*args`. `P.kwargs` representa o mapeamento de parâmetros nomeados para seus valores em uma determinada chamada, e só deve ser usado para anotar `**kwargs`. Ambos os atributos exigem que o parâmetro anotado esteja em escopo. Em tempo de execução, `P.args` e `P.kwargs` são instâncias, respectivamente, de *`ParamSpecArgs`* e *`ParamSpecKwargs`*.

__name__

O nome da especificação de parâmetros.

__default__

O valor padrão da especificação de parâmetro, ou *`typing.NoDefault`* se não tiver padrão.

Adicionado na versão 3.13.

evaluate_default()

Uma *função de avaliação* correspondente ao atributo *`__default__`*. Quando chamado diretamente, este método oferece suporte apenas ao formato *`VALUE`*, que equivale a acessar o atributo *`__default__`* diretamente, mas o objeto do método pode ser passado para *`annotationlib.call_evaluate_function()`* para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

has_default()

Retorna se a especificação de parâmetros tem ou não um valor padrão. Isso equivale a verificar se *`__default__`* não é o singleton *`typing.NoDefault`*, exceto por não forçar a avaliação instantânea do valor padrão que é avaliado preguiçosamente.

Adicionado na versão 3.13.

Variáveis de especificação de parâmetros criadas com `covariant=True` ou `contravariant=True` podem ser usadas para declarar tipos genéricos covariantes ou contravariantes. O argumento `bound` também é aceito, semelhante ao *`TypeVar`*. Porém, a semântica real dessas palavras reservadas ainda não foi decidida.

Adicionado na versão 3.10.

Alterado na versão 3.12: Especificações de parâmetros agora podem ser declaradas usando a sintaxe de parâmetros de tipo introduzido pela **PEP 695**.

Alterado na versão 3.13: Adiciona suporte a valores padrão.

Nota

Somente variáveis de especificação de parâmetro definidas em escopo global podem ser serializadas com pickle.

Ver também

- **PEP 612** – Variáveis de especificação de parâmetro (a PEP que introduz `ParamSpec` e `Concatenate`)
- *`Concatenate`*
- *Anotações de objetos chamáveis*

`typing.`**`ParamSpecArgs`**

`typing.`**`ParamSpecKwargs`**

Tipos dos argumentos e dos argumentos nomeados de um *`ParamSpec`*. O atributo `P.args` de um `ParamSpec` é uma instância de `ParamSpecArgs`, e o atributo `P.kwargs` é uma instância de `ParamSpecKwargs`. São destinados à introspecção em tempo de execução, e não têm nenhum significado especial para o verificador de tipo estático.

Chamar *`get_origin()`* em qualquer um desses objetos retornará o `ParamSpec` original:

```
>>> from typing import ParamSpec, get_origin
>>> P = ParamSpec("P")
>>> get_origin(P.args) is P
True
>>> get_origin(P.kwargs) is P
True
```

Adicionado na versão 3.10.

class `typing.`**`TypeAliasType`**(*name*, *value*, *, *type_params=()*)

O tipo de apelidos de tipo criados pela instrução `type`.

Exemplo:

```
>>> type Alias = int
>>> type(Alias)
<class 'typing.TypeAliasType'>
```

Adicionado na versão 3.12.

`__name__`

O nome do apelido de tipo:

```
>>> type Alias = int
>>> Alias.__name__
'Alias'
```

`__module__`

O módulo no qual o apelido de tipo foi definido:

```
>>> type Alias = int
>>> Alias.__module__
'__main__'
```

__type_params__

Os parâmetros de tipo do apelido de tipo, ou uma tupla vazia se o apelido não for genérico:

```
>>> type ListOrSet[T] = list[T] | set[T]
>>> ListOrSet.__type_params__
(T,)
>>> type NotGeneric = int
>>> NotGeneric.__type_params__
()
```

__value__

O valor do apelido de tipo. Isso é uma avaliação preguiçosa, portanto, os nomes usados na definição do apelido não são resolvidos até que o atributo `__value__` seja acessado:

```
>>> type Mutually = Recursive
>>> type Recursive = Mutually
>>> Mutually
Mutually
>>> Recursive
Recursive
>>> Mutually.__value__
Recursive
>>> Recursive.__value__
Mutually
```

evaluate_value()

Uma *função de avaliação* correspondente ao atributo *__value__*. Quando chamado diretamente, este método oferece suporte apenas ao formato *VALUE*, que equivale a acessar o atributo *__value__* diretamente, mas o objeto do método pode ser passado para *annotationlib.call_evaluate_function()* para avaliar o valor em um formato diferente.

```
>>> type Alias = undefined
>>> Alias.__value__
Traceback (most recent call last):
...
NameError: name 'undefined' is not defined
>>> from annotationlib import Format, call_evaluate_function
>>> Alias.evaluate_value(Format.VALUE)
Traceback (most recent call last):
...
NameError: name 'undefined' is not defined
>>> call_evaluate_function(Alias.evaluate_value, Format.FORWARDREF)
ForwardRef('undefined')
```

Adicionado na versão 3.14.

Descompactando

Apelidos de tipo oferecem suporte a desempacotamento em estrela usando a sintaxe `*Alias`. Isso equivale a usar `Unpack[Alias]` diretamente:

```
>>> type Alias = tuple[int, str]
>>> type Unpacked = tuple[bool, *Alias]
>>> Unpacked.__value__
tuple[bool, typing.Unpack[Alias]]
```

Adicionado na versão 3.14.

Outras diretivas especiais

Essas funções e classes não devem ser usadas diretamente como anotações. O objetivo é que sejam blocos de construção para criar e declarar tipos.

class typing.**NamedTuple**

Versão tipada de *collections.namedtuple()*.

Uso:

```
class Employee(NamedTuple):
    name: str
    id: int
```

Isso equivale a:

```
Employee = collections.namedtuple('Employee', ['name', 'id'])
```

Para dar um valor padrão a um campo, você pode atribuir um valor a ele no corpo da classe:

```
class Employee(NamedTuple):
    name: str
    id: int = 3

employee = Employee('Guido')
assert employee.id == 3
```

Campos com valor padrão devem vir depois de quaisquer campos sem valor padrão.

A classe resultante tem um atributo extra `__annotations__` que fornece um dicionário que mapeia os nomes de campos para os tipos de campos. (Os nomes de campos estão no atributo `_fields` e os valores padrões estão no atributo `_field_defaults`, e ambos fazem parte da API de *namedtuple()*.)

Subclasses de `NamedTuple` também podem ter strings de documentação e métodos:

```
class Employee(NamedTuple):
    """Representa um funcionário."""
    name: str
    id: int = 3

    def __repr__(self) -> str:
        return f'<Employee {self.name}, id={self.id}>'
```

Subclasses de `NamedTuple` podem ser genéricas:

```
class Group[T](NamedTuple):
    key: T
    group: list[T]
```

Uso retrocompatível:

```
# Para criar uma NamedTuple genérica em Python 3.11
T = TypeVar("T")

class Group(NamedTuple, Generic[T]):
    key: T
    group: list[T]

# Também há suporte para a sintaxe funcional
Employee = NamedTuple('Employee', [('name', str), ('id', int)])
```

Alterado na versão 3.6: Adiciona suporte à sintaxe de anotação de variáveis da **PEP 526**.

Alterado na versão 3.6.1: Adiciona suporte a valores padrões, métodos, e docstrings.

Alterado na versão 3.8: Os atributos `_field_types` e `__annotations__` agora são dicionários regulares em vez de instâncias de `OrderedDict`.

Alterado na versão 3.9: Remove o atributo `_field_types` em favor do atributo mais padronizado `__annotations__` que tem as mesmas informações.

Alterado na versão 3.11: Adiciona suporte a tuplas nomeadas genéricas.

Alterado na versão 3.14: O uso de *`super()`* (e a *variável de clausura* `__class__`) em métodos de subclasses de `NamedTuple` não é suportado e causa um *`TypeError`*.

Deprecated since version 3.13, will be removed in version 3.15: A sintaxe de argumentos nomeados não documentados para criar classes NamedTuple (`NT = NamedTuple("NT", x=int)`) está descontinuado e será proibido na versão 3.15. Em vez disso, use a sintaxe baseada em classes ou a sintaxe funcional.

Deprecated since version 3.13, will be removed in version 3.15: Ao usar a sintaxe funcional para criar uma classe NamedTuple, não passar um valor para o parâmetro 'fields' (`NT = NamedTuple("NT")`) está descontinuado. Passar `None` para o parâmetro 'fields' (`NT = NamedTuple("NT", None)`) também está descontinuado. Ambos não serão permitidos em Python 3.15. Para criar uma classe NamedTuple com 0 campos, use `class NT(NamedTuple): pass` ou `NT = NamedTuple("NT", [])`.

class typing.**NewType**(*name*, *tp*)

Classe auxiliar para criar *tipos únicos* com baixo custo.

Um tipo `NewType` é considerado um tipo distinto por um verificador de tipos. Porém, em tempo de execução, chamar `NewType` retorna seu argumento inalterado.

Uso:

```
UserId = NewType('UserId', int) # Declare o NewType "UserId"
first_user = UserId(1) # "UserId" retorna o argumento inalterado em tempo de␣
↪execução
```

__module__

O módulo no qual o novo tipo está definido.

__name__

O nome do novo tipo.

__supertype__

O tipo na qual o novo tipo é baseado.

Adicionado na versão 3.5.2.

Alterado na versão 3.10: `NewType` agora é uma classe em vez de uma função.

class typing.**Protocol**(*Generic*)

Classe base para classes de protocolo.

Classes de protocolo são definidas assim:

```
class Proto(Protocol):
    def meth(self) -> int:
        ...
```

Essas classes são usadas principalmente com verificadores de tipo estático que reconhecem a subtipagem estrutural (tipagem pato estática). Por exemplo:

```
class C:
    def meth(self) -> int:
        return 0
```

(continua na próxima página)

(continuação da página anterior)

```
def func(x: Proto) -> int:
    return x.meth()

func(C())  # Passa na verificação de tipos
```

Consulte a **PEP 544** para obter mais detalhes. Classes de protocolo decoradas com *runtime_checkable()* (descritas posteriormente) funcionam como protocolos em tempo de execução simples, somente verificando a presença de determinados atributos, e ignorando suas assinaturas de tipo. Classes de protocolo sem este decorador não podem ser usadas como o segundo argumento para *isinstance()* ou *issubclass()*.

Classes de protocolo podem ser genéricas. Por exemplo:

```
class GenProto[T](Protocol):
    def meth(self) -> T:
        ...
```

Em códigos que precisam ser compatíveis com Python 3.11 ou versões anteriores, protocolos genéricos podem ser escritos da seguinte forma:

```
T = TypeVar("T")

class GenProto(Protocol[T]):
    def meth(self) -> T:
        ...
```

Adicionado na versão 3.8.

@typing.**runtime_checkable**

Marca uma classe de protocolo como um protocolo de tempo de execução.

Esse protocolo pode ser usado com *isinstance()* e *issubclass()*. Isso permite uma verificação estrutural simples, muito semelhante a "pôneis de um truque só" em *collections.abc*, como *Iterable*. Por exemplo:

```
@runtime_checkable
class Closable(Protocol):
    def close(self): ...

assert isinstance(open('/some/file'), Closable)

@runtime_checkable
class Named(Protocol):
    name: str

import threading
assert isinstance(threading.Thread(name='Bob'), Named)
```

Este decorador levanta *TypeError* quando aplicado a uma classe não-protocolo.

Nota

`runtime_checkable()` verificará apenas a presença dos métodos ou atributos obrigatórios, em vez de tipos ou assinaturas de tipos. Por exemplo, o *ssl.SSLObject* é uma classe e, portanto, passa por uma verificação *issubclass()* em relação a *Callable*. No entanto, o método `ssl.SSLObject.__init__` existe apenas para levantar um *TypeError* com uma mensagem mais informativa, o que impossibilita chamar (instanciar) o *ssl.SSLObject*.

> **Nota**
>
> uma verificação com *isinstance()* sobre um protocolo verificável em tempo de execução pode ser surpreendentemente lenta se comparada a uma verificação `isinstance()` sobre outros tipos de classe. Considere usar expressões alternativas, como chamar a função *hasattr()* para realizar verificações estruturais em código sensível a desempenho.

Adicionado na versão 3.8.

Alterado na versão 3.12: A implementação interna do *isinstance()* agora verifica os protocolos verificáveis em tempo de execução usando *inspect.getattr_static()* para procurar o atributo (anteriormente, era usado *hasattr()*). Como resultado, alguns objetos que costumavam ser considerados instâncias de um protoloco verifiável em tempo de execução podem não ser mais considerados instâncias desse protocolo em Python 3.12+, e vice-versa. É improvável que a maioria dos usuários seja afetada por essa alteração.

Alterado na versão 3.12: Os membros de um protocolo verificável em tempo de execução agora serão considerados "congelados" em tempo de execução assim que a classe for criada. A alteração dinâmica de atributos de um protocolo em tempo de execução ainda funcionará, mas não terá impacto nas verificações de *isinstance()* ao comparar objetos com o protocolo. Consulte o O que há de novo no Python 3.12 para mais detalhes.

class typing.**TypedDict**(*dict*)

Uma construção especial para adicionar dicas de tipo a um dicionário. Em tempo de execução, é um simples *dict*.

`TypedDict` declara um tipo dicionário que espera que todas as suas instâncias tenham um determinado conjunto de chaves, onde cada chave está associada a um valor de um tipo consistente. Essa expectativa não é verificada em tempo de execução, mas é imposta apenas por verificadores de tipos. Modo de usar:

```
class Point2D(TypedDict):
    x: int
    y: int
    label: str

a: Point2D = {'x': 1, 'y': 2, 'label': 'good'}  # OK
b: Point2D = {'z': 3, 'label': 'bad'}           # Falha na verificação de tipos

assert Point2D(x=1, y=2, label='first') == dict(x=1, y=2, label='first')
```

Um modo alternativo de criar um `TypedDict` é usando a sintaxe de chamada de função. O segundo argumento deve ser um *dict*:

```
Point2D = TypedDict('Point2D', {'x': int, 'y': int, 'label': str})
```

Esta sintaxe funcional permite definirmos chaves usando identificadores inválidos, por exemplo, por serem palavras-chave ou conterem hífens, ou quando os nomes das chaves não devem ser desconfigurados como nomes privados comuns:

```
# levanta SyntaxError
class Point2D(TypedDict):
    in: int  # 'in' é uma palavra reservada
    x-y: int  # nome com hífen

class Definition(TypedDict):
    __schema: str  # desconfigurado para `_Definition__schema`

# OK, sintaxe funcional
Point2D = TypedDict('Point2D', {'in': int, 'x-y': int})
Definition = TypedDict('Definition', {'__schema': str})  # não desconfigurado
```

Por padrão, todas as chaves devem estar presentes em um `TypedDict`. É possível marcar chaves individuais como não-obrigatórias usando *`NotRequired`*:

```
class Point2D(TypedDict):
    x: int
    y: int
    label: NotRequired[str]

# Sintaxe alternativa
Point2D = TypedDict('Point2D', {'x': int, 'y': int, 'label': NotRequired[str]})
```

Isso significa que um `TypedDict` `Point2D` pode ter a chave `label` omitida.

Também é possível marcar todas as chaves como não necessárias por padrão, especificando a totalidade como `False`:

```
class Point2D(TypedDict, total=False):
    x: int
    y: int

# Sintaxe alternativa
Point2D = TypedDict('Point2D', {'x': int, 'y': int}, total=False)
```

Isso significa que um `TypedDict` `Point2D` pode ter qualquer uma de suas chaves omitidas. Espera-se que um verificador de tipos apenas permita os literais `False` ou `True` como valores do argumento `total`. `True` é o padrão, e todos os itens definidos no corpo da classe tornam-se obrigatórios.

As chaves individuais de um `TypedDict` com `total=False` podem ser marcadas como obrigatórias usando *`Required`*:

```
class Point2D(TypedDict, total=False):
    x: Required[int]
    y: Required[int]
    label: str

# Sintaxe alternativa
Point2D = TypedDict('Point2D', {
    'x': Required[int],
    'y': Required[int],
    'label': str
}, total=False)
```

É possível que um tipo `TypedDict` herde de um ou mais tipos `TypedDict` usando a sintaxe baseada em classes. Modo de usar:

```
class Point3D(Point2D):
    z: int
```

`Point3D` tem três itens: `x`, `y` e `z`. Equivale a esta definição:

```
class Point3D(TypedDict):
    x: int
    y: int
    z: int
```

Um `TypedDict` não pode herdar de uma classe não `TypedDict`, exceto *`Generic`*. Por exemplo:

```
class X(TypedDict):
    x: int
```

(continua na próxima página)

(continuação da página anterior)

```
class Y(TypedDict):
    y: int

class Z(object): pass  # Uma classe não TypedDict

class XY(X, Y): pass  # OK

class XZ(X, Z): pass  # levanta TypeError
```

Um `TypedDict` pode ser genérico:

```
class Group[T](TypedDict):
    key: T
    group: list[T]
```

Para criar um `TypedDict` genérico que seja compatível com Python 3.11 ou inferior, herde de *Generic* explicitamente:

```
T = TypeVar("T")

class Group(TypedDict, Generic[T]):
    key: T
    group: list[T]
```

Um `TypedDict` pode ser inspecionado por meio de dicionários de anotações (consulte annotations-howto para obter mais informações sobre as melhores práticas de anotações), *__total__*, *__required_keys__* e *__optional_keys__*.

__total__

`Point2D.__total__` fornece o valor do argumento `total`. Exemplo:

```
>>> from typing import TypedDict
>>> class Point2D(TypedDict): pass
>>> Point2D.__total__
True
>>> class Point2D(TypedDict, total=False): pass
>>> Point2D.__total__
False
>>> class Point3D(Point2D): pass
>>> Point3D.__total__
True
```

Esse atributo reflete *apenas* o valor do argumento `total` para a classe `TypedDict` atual, e não que a classe é semanticamente total. Por exemplo, um `TypedDict` com `__total__` definido como `True` pode ter chaves marcadas com *NotRequired*, ou pode herdar de outro `TypedDict` com `total=False`. Portanto, geralmente é melhor usar *__required_keys__* e *__optional_keys__* para introspecção.

__required_keys__

Adicionado na versão 3.9.

__optional_keys__

`Point2D.__required_keys__` e `Point2D.__optional_keys__` retornam objetos *frozenset* contendo chaves obrigatórias e opcionais, respectivamente.

As chaves marcadas com *Required* sempre aparecerão em `__required_keys__` e as chaves marcadas com *NotRequired* sempre aparecerão em `__optional_keys__`.

Para manter a retrocompatibilidade com Python 3.10 e versões anteriores, também é possível usar herança para declarar chaves obrigatórias e opcionais no mesmo `TypedDict`. Isso é feito declarando um

`TypedDict` com um valor para o argumento `total` e então herdando-a em outro `TypedDict` usando um valor `total` diferente:

```
>>> class Point2D(TypedDict, total=False):
...     x: int
...     y: int
...
>>> class Point3D(Point2D):
...     z: int
...
>>> Point3D.__required_keys__ == frozenset({'z'})
True
>>> Point3D.__optional_keys__ == frozenset({'x', 'y'})
True
```

Adicionado na versão 3.9.

Nota

Se `from __future__ import annotations` for usado ou se anotações forem fornecidas como strings, as anotações não serão avaliadas quando o `TypedDict` for definido. Portanto, a introspecção em tempo de execução da qual `__required_keys__` e `__optional_keys__` dependem pode não funcionar corretamente, e os valores dos atributos podem estar incorretos.

O suporte para *ReadOnly* está refletido nos seguintes atributos:

__readonly_keys__

Um *frozenset* contendo os nomes de todas as chaves somente para leitura. As chaves são somente para leitura se tiverem o qualificador *ReadOnly*.

Adicionado na versão 3.13.

__mutable_keys__

Um *frozenset* contendo os nomes de todas as chaves mutáveis. As chaves são mutáveis se não tiverem o qualificador *ReadOnly*.

Adicionado na versão 3.13.

Consulte a seção TypedDict na documentação de tipagem para obter mais exemplos e regras detalhadas.

Adicionado na versão 3.8.

Alterado na versão 3.11: Adicionado suporte para marcar chaves individuais como *Required* ou *NotRequired*. Consulte **PEP 655**.

Alterado na versão 3.11: Adicionado suporte para `TypedDict`s genéricos.

Alterado na versão 3.13: Removeu suporte para o método de argumento nomeado para criar `TypedDict`s.

Alterado na versão 3.13: Adiciona suporte ao qualificador *ReadOnly*.

Deprecated since version 3.13, will be removed in version 3.15: Ao usar a sintaxe funcional para criar uma classe TypedDict, não passar um valor para o parâmetro 'fields' (`TD = TypedDict("TD")`) está descontinuado. Passar `None` para o parâmetro 'fields' (`TD = TypedDict("TD", None)`) também está descontinuado. Ambos não serão permitidos em Python 3.15. Para criar uma classe TypedDict com 0 campos, use `class TD(TypedDict): pass` ou `TD = TypedDict("TD", {})`.

Protocolos

Os protocolos a seguir são fornecidos pelo módulo `typing`. Todos são decorados com *@runtime_checkable*.

class typing.**SupportsAbs**

Um ABC com um método abstrato `__abs__` que é covariante em seu tipo de retorno.

class typing.**SupportsBytes**

Um ABC com um método abstrato `__bytes__`.

class typing.**SupportsComplex**

Um ABC com um método abstrato `__complex__`.

class typing.**SupportsFloat**

Um ABC com um método abstrato `__float__`.

class typing.**SupportsIndex**

Um ABC com um método abstrato `__index__`.

Adicionado na versão 3.8.

class typing.**SupportsInt**

Um ABC com um método abstrato `__int__`.

class typing.**SupportsRound**

Uma ABC com um método abstrato `__round__` que é covariante em seu tipo de retorno.

ABCs e protocolos para trabalhar com E/S

class typing.**IO**[*AnyStr*]()

class typing.**TextIO**[*AnyStr*]()

class typing.**BinaryIO**[*AnyStr*]()

A classe genérica `IO[AnyStr]` e suas subclasses `TextIO(IO[str])` e `BinaryIO(IO[bytes])` representam os tipos de fluxos de E/S, como os retornados por *open()*. Note que essas classes não são protocolos e sua interface é bem ampla.

Os protocolos *io.Reader* e *io.Writer* oferecem uma alternativa mais simples para tipos de argumentos, quando apenas os métodos `read()` ou `write()` são acessados, respectivamente:

```
def read_and_write(reader: Reader[str], writer: Writer[bytes]):
    data = reader.read()
    writer.write(data.encode())
```

Considere também usar *collections.abc.Iterable* para iterar sobre as linhas de um fluxo de entrada:

```
def read_config(stream: Iterable[str]):
    for line in stream:
        ...
```

Funções e decoradores

typing.**cast**(*typ*, *val*)

Converta um valor em um tipo.

Isso retorna o valor inalterado. Para o verificador de tipos, isso indica que o valor de retorno tem o tipo designado, mas em tempo de execução não verificamos nada intencionalmente (queremos que isso seja o mais rápido possível).

typing.**assert_type**(*val*, *typ*, */*)

Pede para um verificador de tipo estático confirmar se *val* tem tipo inferido *typ*.

Essa função faz nada em tempo de execução: ela retorna o primeiro argumento inalterado, sem verificações ou efeitos colaterais, independentemente do tipo real do argumento.

Quando um verificador de tipo estático encontra uma chamada para `assert_type()`, ele emite um erro se o valor não for do tipo especificado:

```
def greet(name: str) -> None:
    assert_type(name, str) # OK, o tipo inferido de `name` é `str`
    assert_type(name, int) # erro no verificador de tipos
```

Esse função é útil para garantir que o verificador de tipos entende um script conforme as intenções do desenvolvedor:

```
def complex_function(arg: object):
    # Realiza alguma lógica complexa de restrição de tipos
    # após a qual esperamos que o tipo seja `int`
    ...
    # Testa se o verificador de tipos entendeu nossa função corretamente
    assert_type(arg, int)
```

Adicionado na versão 3.11.

typing.**assert_never**(*arg*, /)

Pede ao verificador de tipo estático para confirmar se uma linha de código é inalcançável.

Exemplo:

```
def int_or_str(arg: int | str) -> None:
    match arg:
        case int():
            print("É um int")
        case str():
            print("É um str")
        case _ as unreachable:
            assert_never(unreachable)
```

Aqui, as anotações permitem que o verificador de tipos deduza que o último caso nunca será executado, pois `arg` é um *int* ou uma *str*, e ambas as opções são cobertas por casos anteriores.

Se um verificador de tipos descobrir que uma chamada para `assert_never()` é alcançável, ele emitirá um erro. Por exemplo, se a anotação de tipo para `arg` fosse `int | str | float`, o verificador de tipos emitiria um erro indicando que `unreachable` é do tipo *float*. Para que uma chamada para `assert_never` passe verificação de tipos, o tipo inferido do argumento passado deve ser o tipo inferior, *Never*, e nada mais.

Em tempo de execução, essa função levanta uma exceção quando chamada.

Ver também

Unreachable Code and Exhaustiveness Checking tem mais informações sobre a verificação de exaustividade com tipagem estática.

Adicionado na versão 3.11.

typing.**reveal_type**(*obj*, /)

Pede para um verificador de tipo estático revelar o tipo inferido de uma expressão.

Quando um verificador de tipo estático encontra uma chamada para essa função, ele emite um diagnóstico com o tipo inferido do argumento. Por exemplo:

```
x: int = 1
reveal_type(x) # O tipo revelado é "builtins.int"
```

Isso pode ser útil quando você deseja depurar como o verificador de tipos lida com um determinado trecho de código.

Em tempo de execução, esta função envia o tipo de seu argumento para *sys.stderr* e retorna o argumento inalterado (permitindo que a chamada seja usada em uma expressão):

```
x = reveal_type(1)  # exibe "Runtime type is int"
print(x)  # exibe "1"
```

Observe que o tipo em tempo de execução pode ser diferente (mais ou menos específico) do tipo inferido estaticamente por um verificador de tipos.

A maioria dos verificadores de tipos dá suporte a `reveal_type()` em qualquer lugar, mesmo que o nome não seja importado de `typing`. Porém, importar o nome de `typing` permite que o código seja executado sem erros em tempo de execução e comunica a intenção com mais clareza.

Adicionado na versão 3.11.

`@typing.`**`dataclass_transform`**(*, *eq_default=True*, *order_default=False*, *kw_only_default=False*, *frozen_default=False*, *field_specifiers=()*, ***kwargs*)

Um decorador que marca um objeto como tendo comportamento similar a uma *dataclass*.

`dataclass_transform` pode ser usado para decorar uma classe, metaclasse, ou função decoradora. A presença de `@dataclass_transform()` informa a um verificador de tipo estático que o objeto decorado executa "mágica" em tempo de execução que transforma uma classe de maneira semelhante a *@dataclasses.dataclass*.

Exemplo de uso com a função decoradora:

```
@dataclass_transform()
def create_model[T](cls: type[T]) -> type[T]:
    ...
    return cls

@create_model
class CustomerModel:
    id: int
    name: str
```

Em uma classe base:

```
@dataclass_transform()
class ModelBase: ...

class CustomerModel(ModelBase):
    id: int
    name: str
```

Em uma metaclasse:

```
@dataclass_transform()
class ModelMeta(type): ...

class ModelBase(metaclass=ModelMeta): ...

class CustomerModel(ModelBase):
    id: int
    name: str
```

As classes `CustomerModel` definidas acima serão tratadas pelos verificadores de tipos semelhantemente às classes criadas com *@dataclasses.dataclass*. Por exemplo, os verificadores de tipo presumirão que essas classes têm métodos `__init__` que aceitam `id` e `name`.

A classe, metaclasse, ou função decorada pode aceitar os seguintes argumentos booleanos tal que verificadores de tipo presumirão ter o mesmo efeito que teriam no decorador *@dataclasses.dataclass*: `init`,

`eq`, `order`, `unsafe_hash`, `frozen`, `match_args`, `kw_only`, e `slots`. É necessário que o valor desses argumentos (`True` ou `False`) seja avaliado estaticamente.

Os argumentos do decorador `dataclass_transform` podem ser usados para personalizar o comportamento padrão da classe, metaclasse, ou função decorada:

Parâmetros

- **`eq_default`** (`bool`) – Indica se o parâmetro `eq` é presumido como `True` ou `False` se for omitido pelo chamador. O padrão é `True`.
- **`order_default`** (`bool`) – Indica se o parâmetro `order` é presumido como `True` ou `False` se for omitido pelo chamador. O padrão é `False`.
- **`kw_only_default`** (`bool`) – Indica se o parâmetro `kw_only` é presumido como `True` ou `False` se for omitido pelo chamador. O padrão é `False`.
- **`frozen_default`** (`bool`) – Indica se o parâmetro `frozen` é presumido como `True` ou `False` se for omitido pelo chamador. O padrão é `False`. Added in version 3.12
- **`field_specifiers`** (*`tuple[Callable[..., Any], ...]`*) – Especifica uma lista estática de classes ou funções compatíveis que descrevem campos, semelhante a *`dataclasses.field()`*. O valor padrão é `()`.
- **`**kwargs`** (*`Any`*) – Outros argumentos nomeados arbitrários são aceitos para permitir possíveis futuras extensões.

Verificadores de tipo reconhecem os seguintes parâmetros opcionais em especificadores de campos:

Tabela1: **Parâmetros reconhecidos para especificadores de campos**

Nome do parâmetro	Descrição
`init`	Indica se o campo deve ser incluído no método `__init__` sintetizado. Se não for especificado, o valor padrão de `init` é `True`.
`default`	Fornece o valor padrão do campo.
`default_factory`	Fornece uma função de retorno de tempo de execução que retorna o valor padrão do campo. Se nem `default` nem `default_factory` forem especificados, o campo é presumido como sem valor padrão e deverá receber um valor quando a classe for instanciada.
`factory`	Um apelido para o parâmetro `default_factory` em especificadores de campos.
`kw_only`	Indica se o campo deve ser marcado como somente-nomeado. Se verdadeiro, o campo será somente-nomeado. Se falso, não será somente-nomeado. Se não for especificado, será usado o valor do parâmetro `kw_only` do objeto decorado com `dataclass_transform` ou, se este não for especificado, será usado o valor de `kw_only_default` no `dataclass_transform`.
`alias`	Fornece um nome alternativo para o campo. Esse nome alternativo é usado no método `__init__` sintetizado.

Em tempo de execução, esse decorador registra seu argumento no atributo `__dataclass_transform__` no objeto decorado. Ele não tem nenhum outro efeito em tempo de execução.

Consulte **PEP 681** para mais detalhes.

Adicionado na versão 3.11.

`@typing.`**`overload`**

Decorador para criar funções e métodos sobrecarregados.

O decorador `@overload` permite descrever funções e métodos com suporte a várias combinações de tipos de argumento. Uma sequência de definições decoradas com `@overload` deve preceder uma única definição não decorada por `@overload` (para a mesma função/método).

Definições decoradas com `@overload` são usadas somente para benefício do verificador de tipos, já que serão sobrescritas por definições sem decoração de `@overload`. Enquanto isso, definições sem decoração de `@overload` serão usadas em tempo de execução, mas devem ser ignoradas pelo verificador de tipos. Em tempo de execução, chamar uma função decorada por `@overload` diretamente levantará *NotImplementedError*.

Um exemplo de sobrecarga que fornece um tipo mais preciso do que o expressar o tipo expresso por uma união ou um tipo variável:

```
@overload
def process(response: None) -> None:
    ...
@overload
def process(response: int) -> tuple[int, str]:
    ...
@overload
def process(response: bytes) -> str:
    ...
def process(response):
    ...  # a implementação real vem aqui
```

Consulte **PEP 484** para mais detalhes e uma comparação com outras semânticas de tipagem.

Alterado na versão 3.11: Funções sobrecarregadas agora podem ser inspecionadas em tempo de execução usando *get_overloads()*.

`typing.`**`get_overloads`**(*func*)

Retorna uma sequência de definições de *func* decoradas com *@overload*.

func é o objeto função para a implementação da função sobrecarregada. Por exemplo, dada a definição de `process` na documentação de *@overload*, `get_overloads(process)` retornará uma sequência de três objetos função para as três sobrecargas definidas. Se for chamada em uma função sem sobrecargas, `get_overloads()` retornará uma sequência vazia.

A função `get_overloads()` pode ser usada em tempo de execução para introspecção de uma função sobrecarregada.

Adicionado na versão 3.11.

`typing.`**`clear_overloads`**()

Apaga todas as sobrecargas registradas no registro interno.

Isso pode ser usado para recuperar a memória usada pelo registro.

Adicionado na versão 3.11.

`@typing.`**`final`**

Um decorador para indicar métodos e classes finais.

Decorar um método com `@final` indica a um verificador de tipos que o método não pode ser substituído em uma subclasse. Decorar uma classe com `@final` indica que ela não pode ser herdada.

Por exemplo:

```
class Base:
    @final
    def done(self) -> None:
        ...
class Sub(Base):
    def done(self) -> None:  # Erro reportado pelo verificador de tipos
        ...

@final
class Leaf:
```

(continua na próxima página)

(continuação da página anterior)

```
    ...
class Other(Leaf):  # Erro reportado pelo verificador de tipos
    ...
```

Não há verificação em tempo de execução dessas propriedades. Veja **PEP 591** para mais detalhes.

Adicionado na versão 3.8.

Alterado na versão 3.11: O decorador tentará definir um atributo `__final__` como `True` no objeto decorado. Assim, uma verificação como `if getattr(obj, "__final__", False)` pode ser usada em tempo de execução para determinar se um objeto `obj` foi marcado como final. Se o objeto decorado não tem suporte a definição de atributos, o decorador retorna o objeto inalterado sem levantar uma exceção.

`@typing.`**`no_type_check`**

Decorador para indicar que anotações não são dicas de tipo.

Isso funciona como *decorator* de uma classe ou função. Com uma classe, ele se aplica recursivamente a todos os métodos e classes definidos nessa classe (mas não a métodos definidos em suas superclasses ou subclasses). Os verificadores de tipos ignorarão todas as anotações em uma função ou classe com este decorador.

`@no_type_check` modifica o objeto decorado internamente.

`@typing.`**`no_type_check_decorator`**

Decorador para dar a outro decorador o efeito *`no_type_check()`*.

Isso envolve o decorador com algo que envolve a função decorada em *`no_type_check()`*.

Deprecated since version 3.13, will be removed in version 3.15: Nenhum verificador de tipos jamais adicionou suporte para `@no_type_check_decorator`. Portanto, ele está descontinuado e será removido no Python 3.15.

`@typing.`**`override`**

Decorador para indicar que um método em uma subclasse deve substituir um método ou atributo em uma superclasse.

Os verificadores de tipo devem emitir um erro se um método decorado com `@override` substitui nada. Isso ajuda a evitar erros que podem ocorrer quando uma classe base muda sem uma mudança equivalente em uma classe filha.

Por exemplo:

```
class Base:
    def log_status(self) -> None:
        ...

class Sub(Base):
    @override
    def log_status(self) -> None:  # OK: substitui Base.log_status
        ...

    @override
    def done(self) -> None:  # Erro reportado pelo verificador de tipos
        ...
```

Não há verificação desta propriedade em tempo de execução.

O decorador tentará definir um atributo `__override__` como `True` no objeto decorado. Assim, uma verificação como `if getattr(obj, "__override__", False)` poderá ser usada em tempo de execução para determinar se um objeto `obj` foi marcado como uma substituição. Se o objeto decorado não der suporte à atribuição de atributos, o decorador retorna o objeto inalterado sem levantar uma exceção.

Consulte **PEP 698** para obter mais detalhes.

Adicionado na versão 3.12.

@typing.**type_check_only**

Decorador para marcar uma classe ou função como indisponível em tempo de execução.

Este decorador em si não está disponível em tempo de execução. Seu objetivo principal é marcar classes definidas em arquivos de tipo stub se uma implementação retornar uma instância de uma classe privada:

```
@type_check_only
class Response:  # privado ou indisponível em tempo de execução
    code: int
    def get_header(self, name: str) -> str: ...

def fetch_response() -> Response: ...
```

Observe que retornar instâncias de classes privadas não é recomendado. Normalmente, é preferível tornar essas classes públicas.

Auxiliares de introspecção

typing.**get_type_hints**(*obj*, *globalns=None*, *localns=None*, *include_extras=False*)

Retorna um dicionário contendo dicas de tipo para uma função, método, módulo ou objeto classe.

Geralmente é o mesmo que `obj.__annotations__`, mas esta função faz as seguintes alterações no dicionário de anotações:

- Referências futuras codificadas como literais de string ou objetos *ForwardRef* são tratados avaliando-as em *globalns*, *localns* e (quando aplicável) no espaço de nomes de parâmetro de tipo de *obj*. Se *globalns* ou *localns* não forem fornecidos, os dicionários de espaço de nomes apropriados serão inferidos de *obj*.
- `None` é substituído por *types.NoneType*.
- Se *@no_type_check* tiver sido aplicada a *obj*, será retornado um dicionário vazio.
- Se *obj* for uma classe `C`, a função retorna um dicionário que mescla anotações das classes base de `C` com as de `C` diretamente. Isso é feito percorrendo `C.__mro__` e combinando iterativamente os dicionários `__annotations__`. Anotações em classes que aparecem antes na *ordem de resolução de métodos* sempre tem precedência sobre anotações em classes que aparecem depois no ordem de resolução de métodos.
- A função substitui recursivamente todas as ocorrências de `Annotated[T, ...]` por `T`, a menos que *include_extras* seja definido como `True` (consulte *Annotated* para obter mais informações).

Veja também *annotationlib.get_annotations()*, uma função de nível inferior que retorna anotações mais diretamente.

Cuidado

Esta função pode executar código arbitrário contido em anotações. Consulte *Security implications of introspecting annotations* para obter mais informações.

Nota

Se alguma referência futura nas anotações de *obj* não for resolvível ou não for código Python válido, esta função levantará uma exceção como *NameError*. Por exemplo, isso pode acontecer com *apelidos de tipo* importados que incluem referências futuras, ou com nomes importados em *if TYPE_CHECKING*.

Alterado na versão 3.9: Adiciona o parâmetro `include_extras` como parte da **PEP 593**. Consulte a documentação em *Annotated* para mais informações.

Alterado na versão 3.11: Anteriormente, `Optional[t]` era adicionado a anotações de funções e métodos se um valor padrão igual a `None` fosse definido. Agora, a anotação é retornada inalterada.

typing.**get_origin**(*tp*)

Obtenha o versão sem subscrição de um tipo: para um objeto cujo tipo tem a forma `X[Y, Z, ...]`, retorna `X`.

Se `X` for um apelido do módulo typing para uma classe embutida ou classe de *collections*, ele será normalizado para a classe original. Se `X` for uma instância de *ParamSpecArgs* ou *ParamSpecKwargs*, retorna o *ParamSpec* subjacente. Retorna `None` para objetos incompatíveis.

Exemplos:

```
assert get_origin(str) is None
assert get_origin(Dict[str, int]) is dict
assert get_origin(Union[int, str]) is Union
assert get_origin(Annotated[str, "metadata"]) is Annotated
P = ParamSpec('P')
assert get_origin(P.args) is P
assert get_origin(P.kwargs) is P
```

Adicionado na versão 3.8.

typing.**get_args**(*tp*)

Obtenha os argumentos de tipos com todas as substituições realizadas: para um objeto cujo tipo tem a forma `X[Y, Z, ...]`, retorna `(Y, Z, ...)`.

Se `X` for uma união ou um *Literal*, e estiver contido em outro tipo genérico, a ordem de `(Y, Z, ...)` poderá ser diferente da ordem dos argumentos originais `[Y, Z, ...]` devido ao armazenamento dos tipos em cache. Retorna `()` para objetos sem suporte.

Exemplos:

```
assert get_args(int) == ()
assert get_args(Dict[int, str]) == (int, str)
assert get_args(Union[int, str]) == (int, str)
```

Adicionado na versão 3.8.

typing.**get_protocol_members**(*tp*)

Retorna o conjunto de membros definidos em um *Protocol*.

```
>>> from typing import Protocol, get_protocol_members
>>> class P(Protocol):
...     def a(self) -> str: ...
...     b: int
>>> get_protocol_members(P) == frozenset({'a', 'b'})
True
```

Levanta *TypeError* para argumentos que não são protocolos.

Adicionado na versão 3.13.

typing.**is_protocol**(*tp*)

Determina se um tipo é um *Protocol*.

Por exemplo:

```
class P(Protocol):
    def a(self) -> str: ...
    b: int

is_protocol(P)     # => True
is_protocol(int)   # => False
```

Adicionado na versão 3.13.

typing.**is_typeddict**(*tp*)

Verifica se um tipo é um *TypedDict*.

Por exemplo:

```
class Film(TypedDict):
    title: str
    year: int

assert is_typeddict(Film)
assert not is_typeddict(list | str)

# TypedDict é uma fábrica para criar dicionários tipados
# e não um dicionário tipado em si.
assert not is_typeddict(TypedDict)
```

Adicionado na versão 3.10.

class typing.**ForwardRef**

Classe usada para representação de tipagem interna de referências futuras como strings.

Por exemplo, `List["AlgumaClasse"]` é implicitamente transformado em `List[ForwardRef("AlgumaClasse")]`. `ForwardRef` não deve ser instanciada por um usuário, mas pode ser usada por ferramentas de introspecção.

Nota

Tipos genéricos da **PEP 585** como `list["SomeClass"]` não serão transformados implicitamente em `list[ForwardRef("SomeClass")]` e, portanto, não serão resolvidos automaticamente para `list[SomeClass]`.

Adicionado na versão 3.7.4.

Alterado na versão 3.14: Este agora é um apelido para *annotationlib.ForwardRef*. Vários comportamentos não documentados desta classe foram alterados; por exemplo, após a avaliação de um `ForwardRef`, o valor avaliado não é mais armazenado em cache.

typing.**evaluate_forward_ref**(*forward_ref*, *, *owner=None*, *globals=None*, *locals=None*, *type_params=None*, *format=annotationlib.Format.VALUE*)

Avalia um *annotationlib.ForwardRef* como um *dica de tipo*.

Isso é semelhante a chamar *annotationlib.ForwardRef.evaluate()*, mas diferentemente desse método, `evaluate_forward_ref()` também avalia recursivamente referências futuras aninhadas dentro da dica de tipo.

Consulte a documentação de *annotationlib.ForwardRef.evaluate()* para saber o significado dos parâmetros *owner*, *globals*, *locals*, *type_params* e *format*.

Cuidado

Esta função pode executar código arbitrário contido em anotações. Consulte *Security implications of introspecting annotations* para obter mais informações.

Adicionado na versão 3.14.

typing.**NoDefault**

Um objeto sinalizador usado para indicar que um parâmetro de tipo não possui valor-padrão. Por exemplo:

```
>>> T = TypeVar("T")
>>> T.__default__ is typing.NoDefault
True
>>> S = TypeVar("S", default=None)
>>> S.__default__ is None
True
```

Adicionado na versão 3.13.

Constante

typing.**TYPE_CHECKING**

Uma constante especial presumida ser `True` por verificadores de tipo estático de terceiros. É `False` em tempo de execução.

Um módulo cuja importação é cara e que contém apenas tipos usados para anotações de tipagem pode ser importado com segurança dentro de um bloco `if TYPE_CHECKING:`. Isso impede que o módulo seja importado em tempo de execução; anotações não são avaliadas de forma ansiosa (veja **PEP 649**), portanto, usar símbolos indefinidos em anotações é inofensivo – desde que você não os examine posteriormente. Sua ferramenta de análise de tipos estáticos definirá `TYPE_CHECKING` como `True` durante a análise de tipos estáticos, o que significa que o módulo será importado e os tipos serão verificados corretamente durante essa análise.

Uso:

```
if TYPE_CHECKING:
    import expensive_mod

def fun(arg: expensive_mod.SomeType) -> None:
    local_var: expensive_mod.AnotherType = other_fun()
```

Se você ocasionalmente precisar examinar anotações de tipo em tempo de execução que podem conter símbolos indefinidos, use *annotationlib.get_annotations()* com um parâmetro `format` de *annotationlib.Format.STRING* ou *annotationlib.Format.FORWARDREF* para recuperar as anotações com segurança sem levantar *NameError*.

Adicionado na versão 3.5.2.

Apelidos descontinuados

Este módulo define vários apelidos descontinuados de classes pré-existentes da biblioteca padrão. Originalmente, elas eram incluídas no módulo `typing` para permitir a parametrização dessas classes genéricas usando `[]`. Porém, os apelidos se tornaram redundantes no Python 3.9, quando as classes pré-existentes correspondentes passaram a dar suporte a `[]` (consulte a **PEP 585**).

Os tipos redundantes estão descontinuados desde Python 3.9. No entanto, enquanto os apelidos podem ser removidos em algum momento, essa remoção desses apelidos não está planejada. Assim, nenhum aviso de descontinuação será enviado pelo interpretador para esses apelidos.

Se em algum momento decidirem remover esses apelidos descontinuados, o interpretador emitirá um aviso de descontinuação por, no mínimo, duas versões de lançamento antes da remoção. Os apelidos são garantidos a permanecerem no módulo `typing` sem avisos de descontinuação até, no mínimo, Python 3.14.

Verificadores de tipos são encorajados a sinalizar o uso de tipos descontinuados se o programa que estão verificando respeita uma versão mínima de Python 3.9 ou mais nova.

Apelidos de tipos embutidos

class typing.**Dict**(*dict, MutableMapping[KT, VT]*)

Apelido descontinuado de *dict*.

Note que, para anotar argumentos, prefere-se usar um tipo de coleção abstrata como *Mapping* em vez de *dict* ou `typing.Dict`.

Descontinuado desde a versão 3.9: *builtins.dict* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class `typing.`**List**(*list, MutableSequence[T]*)

Apelido descontinuado de *list*.

Note que, para anotar argumentos, é preferível usar um tipo de coleção abstrata, como *Sequence* ou *Iterable*, em vez de *list* ou `typing.List`.

Descontinuado desde a versão 3.9: *builtins.list* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class `typing.`**Set**(*set, MutableSet[T]*)

Apelido descontinuado de *builtins.set*.

Note que, para anotar argumentos, prefere-se usar um tipo de coleção abstrata como *collections.abc.Set* em vez de *set* ou *typing.Set*.

Descontinuado desde a versão 3.9: *builtins.set* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class `typing.`**FrozenSet**(*frozenset, AbstractSet[T_co]*)

Apelido descontinuado de *builtins.frozenset*.

Descontinuado desde a versão 3.9: *builtins.frozenset* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

`typing.`**Tuple**

Apelido descontinuado para *tuple*.

tuple e `Tuple` são casos especiais no sistema de tipos; consulte *Anotando tuplas* para obter mais detalhes.

Descontinuado desde a versão 3.9: *builtins.tuple* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class `typing.`**Type**(*Generic[CT_co]*)

Apelido descontinuado de *type*.

Consulte *O tipo de objetos de classe* para obter detalhes sobre como usar *type* ou `typing.Type` em anotações de tipos.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: *builtins.type* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

Apelidos para tipos em `collections`

class `typing.`**DefaultDict**(*collections.defaultdict, MutableMapping[KT, VT]*)

Apelido descontinuado de *collections.defaultdict*.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: *collections.defaultdict* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class `typing.`**OrderedDict**(*collections.OrderedDict, MutableMapping[KT, VT]*)

Apelido descontinuado de *collections.OrderedDict*.

Adicionado na versão 3.7.2.

Descontinuado desde a versão 3.9: *collections.OrderedDict* agora oferece suporte a subscrição (`[]`). consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**ChainMap**(*collections.ChainMap, MutableMapping[KT, VT]*)

Apelido descontinuado de *collections.ChainMap*.

Adicionado na versão 3.6.1.

Descontinuado desde a versão 3.9: *collections.ChainMap* agora oferece suporte a subscrição (`[]`). consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Counter**(*collections.Counter, Dict[T, int]*)

Apelido descontinuado de *collections.Counter*.

Adicionado na versão 3.6.1.

Descontinuado desde a versão 3.9: *collections.Counter* agora oferece suporte a subscrição (`[]`). consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Deque**(*deque, MutableSequence[T]*)

Apelido descontinuado de *collections.deque*.

Adicionado na versão 3.6.1.

Descontinuado desde a versão 3.9: *collections.deque* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

Apelidos de outros tipos concretos

class typing.**Pattern**

class typing.**Match**

Apelidos descontinuados correspondem aos tipos de retorno de *re.compile()* e *re.match()*.

Esses tipos (e as funções correspondentes) são genéricas sobre *AnyStr*. `Pattern` pode ser especializado como `Pattern[str]` ou `Pattern[bytes]`; `Match` pode ser especializado como `Match[str]` ou `Match[bytes]`.

Descontinuado desde a versão 3.9: Classes `Pattern` e `Match` de *re* agora suporte `[]`. Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Text**

Apelido descontinuado de *str*.

`Text` é fornecida para servir como um caminho compatível com versões futuras para código em Python 2: no Python 2, `Text` é um apelido para `unicode`.

Use `Text` para indicar que um valor deve conter uma string unicode de forma compatível com Python 2 e Python 3:

```
def add_unicode_checkmark(text: Text) -> Text:
    return text + u' \u2713'
```

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.11: Python 2 deixou de receber suporte, e a maioria dos verificadores de tipos também não oferece suporte à verificação de tipos de código de Python 2. A remoção do apelido não está planejada no momento, mas os usuários são incentivados a usar *str* em vez de `Text`.

Apelidos de contêineres ABC em `collections.abc`

class typing.**AbstractSet**(*Collection[T_co]*)

Apelido descontinuado de *collections.abc.Set*.

Descontinuado desde a versão 3.9: *collections.abc.Set* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**ByteString**(*Sequence[int]*)

Apelido descontinuado para *collections.abc.ByteString*.

Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use *Buffer* ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

`ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de *bytes* e *bytearray*. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de `ByteString` nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como *memoryview*, também nunca foram entendidos como subtipos de `ByteString` (seja em tempo de execução ou por verificadores de tipo estáticos).

See **PEP 688** for more details.

Deprecated since version 3.9, will be removed in version 3.17.

class typing.**Collection**(*Sized, Iterable[T_co], Container[T_co]*)

Apelido descontinuado de *collections.abc.Collection*.

Adicionado na versão 3.6.

Descontinuado desde a versão 3.9: *collections.abc.Collection* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Container**(*Generic[T_co]*)

Apelido descontinuado de *collections.abc.Container*.

Descontinuado desde a versão 3.9: *collections.abc.Container* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**ItemsView**(*MappingView, AbstractSet[tuple[KT_co, VT_co]]*)

Apelido descontinuado de *collections.abc.ItemsView*.

Descontinuado desde a versão 3.9: *collections.abc.ItemsView* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**KeysView**(*MappingView, AbstractSet[KT_co]*)

Apelido descontinuado de *collections.abc.KeysView*.

Descontinuado desde a versão 3.9: *collections.abc.KeysView* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Mapping**(*Collection[KT], Generic[KT, VT_co]*)

Apelido descontinuado de *collections.abc.Mapping*.

Descontinuado desde a versão 3.9: *collections.abc.Mapping* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**MappingView**(*Sized*)

Apelido descontinuado de *collections.abc.MappingView*.

Descontinuado desde a versão 3.9: *collections.abc.MappingView* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**MutableMapping**(*Mapping[KT, VT]*)

Apelido descontinuado de *collections.abc.MutableMapping*.

Descontinuado desde a versão 3.9: *collections.abc.MutableMapping* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**MutableSequence**(*Sequence[T]*)

Apelido descontinuado de *collections.abc.MutableSequence*.

Descontinuado desde a versão 3.9: *collections.abc.MutableSequence* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**MutableSet**(*AbstractSet[T]*)

Apelido descontinuado de *collections.abc.MutableSet*.

Descontinuado desde a versão 3.9: *collections.abc.MutableSet* agora oferece suporte a subscrição ([]). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Sequence**(*Reversible[T_co], Collection[T_co]*)

Apelido descontinuado de *collections.abc.Sequence*.

Descontinuado desde a versão 3.9: *collections.abc.Sequence* agora oferece suporte a subscrição ([]). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**ValuesView**(*MappingView, Collection[_VT_co]*)

Apelido descontinuado de *collections.abc.ValuesView*.

Descontinuado desde a versão 3.9: *collections.abc.ValuesView* agora oferece suporte a subscrição ([]). Consulte **PEP 585** e *Tipo Generic Alias*.

Apelidos para ABCs assíncronas em collections.abc

class typing.**Coroutine**(*Awaitable[ReturnType], Generic[YieldType, SendType, ReturnType]*)

Apelido descontinuado de *collections.abc.Coroutine*.

Consulte *Anotando geradores e corrotinas* para obter detalhes sobre como usar *collections.abc.Coroutine* e typing.Coroutine em anotações de tipos.

Adicionado na versão 3.5.3.

Descontinuado desde a versão 3.9: *collections.abc.Coroutine* agora oferece suporte a subscrição ([]). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**AsyncGenerator**(*AsyncIterator[YieldType], Generic[YieldType, SendType]*)

Apelido descontinuado de *collections.abc.AsyncGenerator*.

Consulte *Anotando geradores e corrotinas* para obter detalhes sobre como usar *collections.abc.AsyncGenerator* e typing.AsyncGenerator em anotações de tipos.

Adicionado na versão 3.6.1.

Descontinuado desde a versão 3.9: *collections.abc.AsyncGenerator* agora oferece suporte a subscrição ([]). Consulte **PEP 585** e *Tipo Generic Alias*.

Alterado na versão 3.13: O parâmetro SendType agora tem um valor-padrão.

class typing.**AsyncIterable**(*Generic[T_co]*)

Apelido descontinuado de *collections.abc.AsyncIterable*.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: *collections.abc.AsyncIterable* agora oferece suporte a subscrição ([]). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**AsyncIterator**(*AsyncIterable[T_co]*)

Apelido descontinuado de *collections.abc.AsyncIterator*.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: *collections.abc.AsyncIterator* agora oferece suporte a subscrição ([]). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Awaitable**(*Generic[T_co]*)

Apelido descontinuado de *collections.abc.Awaitable*.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: *collections.abc.Awaitable* agora oferece suporte a subscrição ([]). Consulte **PEP 585** e *Tipo Generic Alias*.

Apelidos para outros ABCs em `collections.abc`.

class typing.**Iterable**(*Generic[T_co]*)

Apelido descontinuado de *collections.abc.Iterable*.

Descontinuado desde a versão 3.9: *collections.abc.Iterable* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Iterator**(*Iterable[T_co]*)

Apelido descontinuado de *collections.abc.Iterator*.

Descontinuado desde a versão 3.9: *collections.abc.Iterator* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

typing.**Callable**

Apelido descontinuado de *collections.abc.Callable*.

Veja *Anotações de objetos chamáveis* para detalhes sobre como usar *collections.abc.Callable* e `typing.Callable` em anotações de tipo.

Descontinuado desde a versão 3.9: *collections.abc.Callable* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

Alterado na versão 3.10: `Callable` agora oferece suporte a *ParamSpec* e *Concatenate*. Veja **PEP 612** para mais detalhes.

class typing.**Generator**(*Iterator[YieldType], Generic[YieldType, SendType, ReturnType]*)

Apelido descontinuado de *collections.abc.Generator*.

Consulte *Anotando geradores e corrotinas* para obter detalhes sobre como usar *collections.abc.Generator* e `typing.Generator` em anotações de tipos.

Descontinuado desde a versão 3.9: *collections.abc.Generator* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

Alterado na versão 3.13: Adiciona valores-padrão para os tipos de envio e retorno.

class typing.**Hashable**

Apelido descontinuado de *collections.abc.Hashable*.

Descontinuado desde a versão 3.12: Use *collections.abc.Hashable* diretamente.

class typing.**Reversible**(*Iterable[T_co]*)

Apelido descontinuado de *collections.abc.Reversible*.

Descontinuado desde a versão 3.9: *collections.abc.Reversible* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

class typing.**Sized**

Apelido descontinuado de *collections.abc.Sized*.

Descontinuado desde a versão 3.12: Use *collections.abc.Sized* diretamente.

Apelidos de ABCs da `contextlib`

class typing.**ContextManager**(*Generic[T_co, ExitT_co]*)

Apelido descontinuado de *contextlib.AbstractContextManager*.

O primeiro parâmetro de tipo, `T_co`, representa o tipo retornado pelo método `__enter__()`. O segundo parâmetro opcional de tipo, `ExitT_co`, cujo valor-padrão é `bool | None`, representa o tipo retornado pelo método `__exit__()`.

Adicionado na versão 3.5.4.

Descontinuado desde a versão 3.9: *contextlib.AbstractContextManager* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

Alterado na versão 3.13: Adiciona o segundo parâmetro opcional de tipo, `ExitT_co`.

class typing.**AsyncContextManager**(*Generic[T_co, AExitT_co]*)

Apelido descontinuado de *contextlib.AbstractAsyncContextManager*.

O primeiro parâmetro de tipo, `T_co`, representa o tipo retornado pelo método `__aenter__()`. O segundo parâmetro opcional de tipo, `AExitT_co`, cujo valor-padrão é `bool | None`, representa o tipo retornado pelo método `__aexit__()`.

Adicionado na versão 3.6.2.

Descontinuado desde a versão 3.9: *contextlib.AbstractAsyncContextManager* agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** e *Tipo Generic Alias*.

Alterado na versão 3.13: Adiciona o segundo parâmetro opcional de tipo, `AExitT_co`.

26.1.13 Cronograma de descontinuação dos principais recursos

Alguns recursos em `typing` estão descontinuados e podem ser removidos em uma versão futura de Python. A tabela a seguir resume as principais descontinuações para sua conveniência. Ela está sujeita a alterações, e nem todas as descontinuações estão listadas.

Recurso	Descontinuado em	Remoção planejada	PEP/issue
versões `typing` de coleções-padrão	3.9	Não definido (consulte *apelidos descontinuados* para mais informações)	**PEP 585**
typing.ByteString	3.9	3.17	gh-91896
typing.Text	3.11	Não definido	gh-92332
typing.Hashable e *typing.Sized*	3.12	Não definido	gh-94309
typing.TypeAlias	3.12	Não definido	**PEP 695**
@typing.no_type_check_decorator	3.13	3.15	gh-106309
typing.AnyStr	3.13	3.18	gh-105578

26.2 pydoc — Gerador de documentação e sistema de ajuda online

Código-fonte: Lib/pydoc.py

O módulo `pydoc` gera automaticamente a documentação dos módulos Python. A documentação pode ser apresentada como páginas de texto no console, servida em um navegador web ou salva em arquivos HTML.

Para módulos, classes, funções e métodos, a documentação exibida é derivada da docstring (ou seja, o atributo *__doc__*) do objeto, e recursivamente de seus membros documentáveis. Se não houver docstring, `pydoc` tenta obter uma descrição do bloco de linhas de comentário logo acima da definição da classe, função ou método no arquivo fonte, ou no topo do módulo (consulte *inspect.getcomments()*).

A função embutida *help()* invoca o sistema de ajuda online no interpretador interativo, que usa `pydoc` para gerar sua documentação como texto no console. A mesma documentação de texto também pode ser vista de fora do interpretador Python executando **pydoc** como um script no prompt de comando do sistema operacional. Por exemplo, executar

```
python -m pydoc sys
```

em um prompt de console exibirá a documentação do módulo *sys*, em um estilo semelhante às páginas de manual mostradas pelo comando Unix **man**. O argumento para **pydoc** pode ser o nome de uma função, módulo ou pacote, ou uma referência pontilhada a uma classe, método ou função dentro de um módulo ou módulo em um pacote. Se o argumento para **pydoc** parecer um caminho (ou seja, ele contém o separador de caminho para o seu sistema operacional, como uma barra no Unix) e se refere a um arquivo fonte Python existente, então a documentação é produzida para esse arquivo.

Nota

Para encontrar objetos e sua documentação, `pydoc` importa os módulos a serem documentados. Portanto, qualquer código no nível do módulo será executado nessa ocasião. Use uma proteção `if __name__ == '__main__':` para executar código apenas quando um arquivo é chamado como um script e não apenas importado.

Ao imprimir a saída para o console, **pydoc** tenta paginar a saída para facilitar a leitura. Se a variável de ambiente `MANPAGER` ou a `PAGER` estiver definida, **pydoc** usará seu valor como um programa de paginação. Quando ambas estiverem definidas, `MANPAGER` é usada.

Especificar um sinalizador `-w` antes do argumento fará com que a documentação HTML seja escrita em um arquivo no diretório atual, ao invés de exibir texto no console.

Especificar um sinalizador `-k` antes do argumento irá pesquisar as linhas de sinopse de todos os módulos disponíveis para a palavra reservada fornecida como o argumento, novamente de uma maneira semelhante ao comando Unix **man**. A linha de sinopse de um módulo é a primeira linha de sua string de documentação.

Você também pode usar **pydoc** para iniciar um servidor HTTP na máquina local que servirá a documentação para os navegadores web visitantes. **python -m pydoc -p 1234** irá iniciar um servidor HTTP na porta 1234, permitindo que você navegue pela documentação em `http://localhost:1234/` em seu navegador preferido. Especificar `0` como o número da porta irá selecionar uma porta não utilizada arbitrária.

python -m pydoc -n <hostname> irá iniciar o servidor ouvindo no nome de host fornecido. Por padrão, o nome de host é "localhost", mas se você deseja que o servidor seja acessado por outras máquinas, você pode alterar o nome de host ao qual o servidor responde. Durante o desenvolvimento, isso é especialmente útil se você deseja executar o pydoc de dentro de um contêiner.

python -m pydoc -b irá iniciar o servidor e, adicionalmente, abrir um navegador da web para uma página de índice do módulo. Cada página exibida tem uma barra de navegação na parte superior onde você pode escolher *Get* para obter ajuda em um item individual, *Search* para pesquisar todos os módulos com uma palavra reservada em sua linha de sinopse e ir para as páginas de índice do módulo em *Module index*, tópicos em *Topics* e palavras reservadas em *Keywords*.

Quando **pydoc** gera documentação, ele usa o ambiente atual e o caminho para localizar os módulos. Assim, invocar **pydoc spam** documenta precisamente a versão do módulo que você obteria se iniciasse o interpretador Python e digitasse `import spam`.

Os documentos do módulo para os módulos principais são presumidos como residindo em `https://docs.python.org/X.Y/library/`, sendo `X` e `Y` os números de versão principal e secundária do interpretador Python. Isso pode ser substituído definindo a variável de ambiente `PYTHONDOCS` para uma URL diferente ou para um diretório local contendo as páginas do Manual de Referência da Biblioteca.

Alterado na versão 3.2: Adicionada a opção `-b`.

Alterado na versão 3.3: A opção de linha de comando `-g` foi removida.

Alterado na versão 3.4: `pydoc` agora usa *inspect.signature()* em vez de *inspect.getfullargspec()* para extrair informações de assinatura de chamáveis.

Alterado na versão 3.7: Adicionada a opção `-n`.

26.3 Modo de Desenvolvimento do Python

Adicionado na versão 3.7.

O Modo de Desenvolvimento do Python introduz verificações de tempo de execução adicionais que são muito custosas para serem ativadas por padrão. Não deve ser mais detalhado que o padrão se o código estiver correto; novos avisos são emitidos somente quando um problema é detectado.

Ele pode ser ativado usando a opção de linha de comando `-X dev` ou configurando a variável de ambiente `PYTHONDEVMODE` como `1`.

Veja também a compilação de depuração do Python.

26.3.1 Efeitos do Modo de Desenvolvimento do Python

A ativação do Modo de Desenvolvimento do Python é semelhante ao comando a seguir, mas com efeitos adicionais descritos abaixo:

```
PYTHONMALLOC=debug PYTHONASYNCIODEBUG=1 python -W default -X faulthandler
```

Efeitos do Modo de Desenvolvimento do Python:

- Adiciona o *filtro de avisos* `default`. Os seguintes avisos são exibidos:
  - *DeprecationWarning*
  - *ImportWarning*
  - *PendingDeprecationWarning*
  - *ResourceWarning*

  Normalmente, os avisos acima são filtrados pelos *filtros de avisos* padrão.

  Ele se comporta como se a opção de linha de comando `-W default` fosse usada.

  Use a opção de linha de comando `-W error` ou defina a variável de ambiente `PYTHONWARNINGS` com `error` para tratar avisos como erros.
- Instala ganchos de depuração nos alocadores de memória para verificar por:
  - Estouro negativo de buffer
  - Estouro de buffer
  - Violação de API de alocador de memória
  - Uso inseguro da GIL

  Consulte a função C `PyMem_SetupDebugHooks()`.

  Se comporta como se a variável de ambiente `PYTHONMALLOC` estivesse definida com `debug`.

  Para habilitar o Modo de Desenvolvimento do Python sem instalar ganchos de depuração nos alocadores de memória, defina a variável de ambiente `PYTHONMALLOC` como `default`.
- Chama *faulthandler.enable()* na inicialização do Python para instalar manipuladores para os sinais *SIGSEGV*, *SIGFPE*, *SIGABRT*, *SIGBUS* e *SIGILL* para despejar o traceback (situação da pilha de execução) do Python no caso de travamento.

  Ele se comporta como se a opção de linha de comando `-X faulthandler` fosse usada ou se a variável de ambiente `PYTHONFAULTHANDLER` estivesse definida como `1`.
- Ativa o *modo de depuração de asyncio*. Por exemplo, *asyncio* verifica as corrotinas que não foram aguardadas (*await*) e as registra.

  Ele se comporta como se a variável de ambiente `PYTHONASYNCIODEBUG` estivesse definida como `1`.

- Verifica os argumentos *encoding* e *errors* para operações de codificação e decodificação de strings. Exemplos: `open()`, `str.encode()` e `bytes.decode()`.

  Por padrão, para obter o melhor desempenho, o argumento *errors* é verificado apenas no primeiro erro de codificação/decodificação, e o argumento *encoding* às vezes é ignorado para strings vazias.

- O destrutor de `io.IOBase` registra exceções `close()`.
- Define o atributo `dev_mode` de `sys.flags` como `True`.

O Modo de Desenvolvimento do Python não ativa o módulo `tracemalloc` por padrão porque o custo adicional (para desempenho e memória) seria muito grande. A ativação do módulo `tracemalloc` fornece informações adicionais sobre a origem de alguns erros. Por exemplo, `ResourceWarning` registra o retorno ao local onde o recurso foi alocado, e um erro de estouro de buffer registra o retorno ao local onde o bloco de memória foi alocado.

O Modo de Desenvolvimento do Python não impede que a opção de linha de comando `-O` remova as instruções `assert` nem configure `__debug__` como `False`.

O Modo de Desenvolvimento do Python só pode ser ativado na inicialização do Python. Seu valor pode ser lido de `sys.flags.dev_mode`.

Alterado na versão 3.8: O destrutor de `io.IOBase` agora registra exceções `close()`.

Alterado na versão 3.9: Os argumentos *encoding* e *errors* agora são verificados para operações de codificação e decodificação de strings.

26.3.2 Exemplo de ResourceWarning

Exemplo de um script que conta o número de linhas do arquivo texto especificado na linha de comando:

```
import sys

def main():
    fp = open(sys.argv[1])
    nlines = len(fp.readlines())
    print(nlines)
    # O arquivo é fechado implicitamente

if __name__ == "__main__":
    main()
```

O script não fecha o arquivo explicitamente. Por padrão, o Python não emite nenhum aviso. Exemplo usando README.txt, que possui 269 linhas:

```
$ python script.py README.txt
269
```

A ativação do Modo de Desenvolvimento do Python exibe um aviso `ResourceWarning`:

```
$ python -X dev script.py README.txt
269
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper name='README.rst'␣
↪mode='r' encoding='UTF-8'>
  main()
ResourceWarning: Enable tracemalloc to get the object allocation traceback
```

Além disso, ativar `tracemalloc` mostra a linha em que o arquivo foi aberto:

```
$ python -X dev -X tracemalloc=5 script.py README.rst
269
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper name='README.rst'␣
↪mode='r' encoding='UTF-8'>
```

(continua na próxima página)

(continuação da página anterior)

```
  main()
Object allocated at (most recent call last):
  File "script.py", lineno 10
    main()
  File "script.py", lineno 4
    fp = open(sys.argv[1])
```

A correção é fechar explicitamente o arquivo. Exemplo usando um gerenciador de contexto:

```
def main():
    # Fecha o arquivo explicitamente ao sair do bloco with
    with open(sys.argv[1]) as fp:
        nlines = len(fp.readlines())
    print(nlines)
```

Não fechar um recurso explicitamente pode deixá-lo aberto por muito mais tempo do que o esperado; isso pode causar problemas graves ao sair do Python. É ruim no CPython, mas é ainda pior no PyPy. Fechar recursos explicitamente torna uma aplicação mais determinística e mais confiável.

26.3.3 Exemplo de erro de descritor de arquivo inválido

Script exibindo sua própria primeira linha:

```
import os

def main():
    fp = open(__file__)
    firstline = fp.readline()
    print(firstline.rstrip())
    os.close(fp.fileno())
    # O arquivo é fechado explicitamente

main()
```

Por padrão, o Python não emite qualquer aviso:

```
$ python script.py
import os
```

O Modo de Desenvolvimento do Python mostra uma *ResourceWarning* e registra um erro "Bad file descriptor" ao finalizar o objeto arquivo:

```
$ python -X dev script.py
import os
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper name='script.py'␣
↪mode='r' encoding='UTF-8'>
  main()
ResourceWarning: Enable tracemalloc to get the object allocation traceback
Exception ignored in: <_io.TextIOWrapper name='script.py' mode='r' encoding='UTF-8
↪'>
Traceback (most recent call last):
  File "script.py", line 10, in <module>
    main()
OSError: [Errno 9] Bad file descriptor
```

`os.close(fp.fileno())` fecha o descritor de arquivo. Quando o finalizador de objeto arquivo tenta fechar o descritor de arquivo novamente, ele falha com o erro `Bad file descriptor`. Um descritor de arquivo deve ser

fechado apenas uma vez. Na pior das hipóteses, fechá-lo duas vezes pode causar um acidente (consulte bpo-18748 para um exemplo).

A correção é remover a linha `os.close(fp.fileno())` ou abrir o arquivo com `closefd=False`.

26.4 `doctest` — Teste exemplos interativos de Python

Código-fonte: Lib/doctest.py

O módulo *doctest* busca partes de texto que se parecem com sessões interativas do Python e, em seguida, executa essas sessões para verificar se elas funcionam exatamente como mostrado. Existem várias maneiras comuns de usar o doctest:

- Para verificar se as docstrings de um módulo estão atualizadas, verificando que todos os exemplos interativos ainda funcionam conforme documentado.
- Para executar testes de regressão, verificando que os exemplos interativos de um arquivo de teste ou um objeto teste funcionam como esperado.
- Para escrever a documentação do tutorial para um pacote, ilustrado de forma liberal com exemplos de entrada e saída. Dependendo se os exemplos ou o texto expositivo são enfatizados, isso tem o sabor do "teste ilustrado" ou "documentação executável".

Aqui temos um pequeno exemplo, porém, completo:

```
"""
This is the "example" module.

The example module supplies one function, factorial().  For example,

>>> factorial(5)
120
"""

def factorial(n):
    """Return the factorial of n, an exact integer >= 0.

    >>> [factorial(n) for n in range(6)]
    [1, 1, 2, 6, 24, 120]
    >>> factorial(30)
    265252859812191058636308480000000
    >>> factorial(-1)
    Traceback (most recent call last):
        ...
    ValueError: n must be >= 0

    Factorials of floats are OK, but the float must be an exact integer:
    >>> factorial(30.1)
    Traceback (most recent call last):
        ...
    ValueError: n must be exact integer
    >>> factorial(30.0)
    265252859812191058636308480000000

    It must also not be ridiculously large:
    >>> factorial(1e100)
    Traceback (most recent call last):
        ...
```

(continua na próxima página)

(continuação da página anterior)

```
    OverflowError: n too large
    """

    import math
    if not n >= 0:
        raise ValueError("n must be >= 0")
    if math.floor(n) != n:
        raise ValueError("n must be exact integer")
    if n+1 == n:  # catch a value like 1e300
        raise OverflowError("n too large")
    result = 1
    factor = 2
    while factor <= n:
        result *= factor
        factor += 1
    return result


if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

Se executar diretamente `example.py` desde a linha de comando, *doctest* a mágica funcionará:

```
$ python example.py
$
```

Observe que nada foi impresso na saída padrão! Isso é normal, e isso significa que todos os exemplos funcionaram. Passe `-v` para o script e *doctest* imprimirá um registro detalhado do que está sendo testando imprimindo ainda um resumo no final:

```
$ python example.py -v
Trying:
    factorial(5)
Expecting:
    120
ok
Trying:
    [factorial(n) for n in range(6)]
Expecting:
    [1, 1, 2, 6, 24, 120]
ok
```

E assim por diante, eventualmente terminando com:

```
Trying:
    factorial(1e100)
Expecting:
    Traceback (most recent call last):
        ...
    OverflowError: n too large
ok
2 items passed all tests:
   1 test in __main__
   6 tests in __main__.factorial
7 tests in 2 items.
7 passed.
```

(continua na próxima página)

(continuação da página anterior)

```
Test passed.
$
```

Isso é tudo o que precisas saber para começar a fazer uso produtivo do módulo *doctest*! Pule! As seções a seguir fornecem detalhes completos. Observe que há muitos exemplos de doctests no conjunto de testes padrão Python e bibliotecas. Exemplos especialmente úteis podem ser encontrados no arquivo de teste padrão `Lib/test/test_doctest/test_doctest.py`.

Adicionado na versão 3.13: A saída é colorizada por padrão e ela pode ser controlada usando variáveis de ambiente.

26.4.1 Uso simples: verificando exemplos em Docstrings

A maneira mais simples de começar a usar o doctest (mas não necessariamente a maneira como você continuará fazendo isso) é encerrar cada módulo `M` com:

```
if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

`doctest` e então examine a docstrings no módulo `M`.

Executar o módulo como um script faz com que os exemplos nas docstrings sejam executados e verificados:

```
python M.py
```

Isso não exibirá nada, a menos que um exemplo falhe, caso em que o(s) exemplo(s) falhando(s) e a(s) causa(s) da(s) falha(s) são impressas em stdout e a linha final de saída será `***Test Failed*** N failures.`, onde *N* é o número de exemplos que falharam.

Ao invés disso, execute agora com a opção `-v`:

```
python M.py -v
```

e um relatório detalhado de todos os exemplos testados é impresso na saída padrão, junto com diversos resumos no final.

Você pode forçar o modo verboso passando `verbose=True` para *testmod()*, ou proibi-lo passando `verbose=False`. Em qualquer um desses casos, *sys.argv* não é examinado por *testmod()* (então passar `-v` ou não não tem efeito).

Há também um atalho de linha de comando para executar *testmod()*, veja a seção *Uso na linha de comando*.

Para mais informações sobre *testmod()*, veja a seção *Basic API*.

26.4.2 Utilização comum: Verificando exemplos em um arquivo texto

Outra aplicação simples do doctest é testar exemplos interativos em um arquivo texto. Isso pode ser feito com a função *testfile()*:

```
import doctest
doctest.testfile("example.txt")
```

Esse pequeno script executa e verifica quaisquer exemplos interativos do Python contidos no arquivo `example.txt`. O conteúdo do arquivo é tratado como se fosse uma única docstring gigante; o arquivo não precisa conter um programa Python! Por exemplo, talvez `example.txt` contenha isto:

```
The ``example`` module
======================

Using ``factorial``
```

(continua na próxima página)

(continuação da página anterior)

```
------------------

This is an example text file in reStructuredText format.  First import
``factorial`` from the ``example`` module:

    >>> from example import factorial

Now use it:

    >>> factorial(6)
    120
```

Executar `doctest.testfile("example.txt")` então encontra o erro nesta documentação:

```
File "./example.txt", line 14, in example.txt
Failed example:
    factorial(6)
Expected:
    120
Got:
    720
```

Assim como *testmod()*, *testfile()* não vai exibir nada a menos que um exemplo falhe. Se um exemplo falhar, então o(s) exemplo(s) com falha e a(s) causa(s) da(s) falha(s) são impressos em stdout, usando o mesmo formato que `testmod()`.

Por padrão, *testfile()* procura por arquivos no diretório do módulo chamador. Veja a seção *Basic API* para uma descrição dos argumentos opcionais que podem ser usados para dizer para procurar por arquivos em outros locais.

Assim como *testmod()*, a verbosidade de *testfile()* pode ser definida com a opção de linha de comando `-v` ou com o argumento nomeado opcional *verbose*.

Há também um atalho de linha de comando para executar *testfile()*, veja a seção *Uso na linha de comando*.

Para maiores informações em *testfile()*, veja a seção *Basic API*.

26.4.3 Uso na linha de comando

O módulo *doctest* pode ser invocado como um script na linha de comando:

```
python -m doctest [-v] [-o OPÇÃO] [-f] arquivo [arquivo ...]
```

-v, --verbose

Relatório detalhado de todos os exemplos testados é impresso na saída padrão, junto com diversos resumos no final.

```
python -m doctest -v example.py
```

Isso importará `example.py` como um módulo independente e executará *testmod()* nele. Observe que isso pode não funcionar corretamente se o arquivo fizer parte de um pacote e importar outros submódulos desse pacote.

Se o nome do arquivo não termina com `.py`, `doctest` infere que ele deve ser executado com *testfile()*:

```
python -m doctest -v example.txt
```

-o, --option <option>

Os sinalizadores de opção controlam vários aspectos do comportamento do doctest, consulte a seção *Flags opcionais*.

Adicionado na versão 3.4.

-f, --fail-fast

Este é um atalho para `-o FAIL_FAST`.

Adicionado na versão 3.4.

26.4.4 Como ele funciona

Esta seção examina detalhadamente como o doctest funciona: quais docstrings ele analisa, como encontra exemplos interativos, qual contexto de execução ele usa, como ele lida com exceções e como sinalizadores de opção podem ser usados para controlar seu comportamento. Esta é a informação que você precisa saber para escrever exemplos de doctest; para obter informações sobre como realmente executar o doctest nesses exemplos, consulte as seções a seguir.

Quais docstrings são examinadas?

A docstring do módulo e todas as docstrings de funções, classes e métodos são pesquisadas. Os objetos importados para o módulo não são pesquisados. Além disso, há casos em que você deseja que os testes façam parte de um módulo, mas não do texto de ajuda, o que exige que os testes não sejam incluídos na documentação. Doctest procura uma variável em nível de módulo chamada `__test__` e a utiliza para localizar outros testes. Se `M.__test__` existir, deve ser um dicionário, e cada entrada mapeia um nome (string) para um objeto função, objeto classe ou string. As docstrings de funções e objetos classe encontradas em `M.__test__` são pesquisadas e as strings são tratadas como se fossem docstrings. Na saída, uma chave `K` em `M.__test__` aparece com o nome `M.__test__.K`.

Por exemplo, coloque este bloco de código no topo de `example.py`:

```
__test__ = {
    'numbers': """
>>> factorial(6)
720

>>> [factorial(n) for n in range(6)]
[1, 1, 2, 6, 24, 120]
"""
}
```

O valor de `example.__test__["numbers"]` será tratado como uma docstring e todos os testes dentro dele serão executados. É importante observar que o valor pode ser mapeado para uma função, objeto classe ou módulo; se sim, `doctest` pesquisa recursivamente em busca de docstrings, que são então escaneados em busca de testes.

Quaisquer classes encontradas são pesquisadas recursivamente de forma semelhante, para testar docstrings em seus métodos contidos e classes aninhadas.

Nota

`doctest` can only automatically discover classes and functions that are defined at the module level or inside other classes.

Since nested classes and functions only exist when an outer function is called, they cannot be discovered. Define them outside to make them visible.

Como os exemplos de docstrings são reconhecidos?

Na maioria dos casos, copiar e colar de uma sessão de console interativo funciona bem, mas o doctest não está tentando fazer uma emulação exata de qualquer shell Python específico.

```
>>> # comentários são ignorados
>>> x = 12
>>> x
12
```

(continua na próxima página)

(continuação da página anterior)

```
>>> if x == 13:
...     print("yes")
... else:
...     print("no")
...     print("NO")
...     print("NO!!!")
...
no
NO
NO!!!
>>>
```

Qualquer saída esperada deve seguir imediatamente a linha final `'>>> '` ou `'... '` contendo o código, e a saída esperada (se houver) se estende até a próxima `'>>> '` ou linha com apenas espaços em branco.

A saída formatada:

- A saída esperada não pode conter uma linha com apenas espaços em branco, uma vez que tal linha é usada para sinalizar o fim da saída esperada. Se a saída esperada contiver uma linha vazia, coloque `<BLANKLINE>` em seu exemplo doctest em cada local onde uma linha em branco é esperada.
- Todos os caracteres de tabulação rígidos são expandidos para espaços, usando paradas de tabulação de 8 colunas. As guias na saída gerada pelo código testado não são modificadas. Como quaisquer tabulações rígidas na saída de amostra *são* expandidas, isso significa que se a saída do código incluir tabulações rígidas, a única maneira de o doctest passar é se a opção *NORMALIZE_WHITESPACE* ou a *diretiva* estiver em vigor. Alternativamente, o teste pode ser reescrito para capturar a saída e compará-la com um valor esperado como parte do teste. Esse tratamento das guias na fonte foi obtido por tentativa e erro e provou ser a maneira menos propensa a erros de lidar com elas. É possível usar um algoritmo diferente para lidar com guias escrevendo uma classe *DocTestParser* personalizada.
- A saída para stdout é capturada, mas não para stderr (os tracebacks de exceção são capturados por um meio diferente).
- Se você continuar uma linha através de barra invertida em uma sessão interativa, ou por qualquer outro motivo usar uma barra invertida, você deverá usar uma docstring bruta, que preservará suas barras invertidas exatamente como você as digita:

  ```
  >>> def f(x):
  ...     r'''Backslashes in a raw docstring: m\n'''
  ...
  >>> print(f.__doc__)
  Backslashes in a raw docstring: m\n
  ```

  Caso contrário, a barra invertida será interpretada como parte da string. Por exemplo, o `\n` acima seria interpretado como um caractere de nova linha. Alternativamente, você pode duplicar cada barra invertida na versão doctest (e não usar uma string bruta):

  ```
  >>> def f(x):
  ...     '''Backslashes in a raw docstring: m\\n'''
  ...
  >>> print(f.__doc__)
  Backslashes in a raw docstring: m\n
  ```

- A coluna inicial não importa:

  ```
  >>> assert "Easy!"
        >>> import math
            >>> math.floor(1.9)
            1
  ```

e tantos caracteres de espaço em branco iniciais são removidos da saída esperada quantos apareceram na linha inicial `'>>> '` que iniciou o exemplo.

Qual é o contexto de execução?

Por padrão, cada vez que *doctest* encontra uma docstring para testar, ele usa uma *cópia superficial* dos globais de `M`, para que a execução de testes não altere os globais reais do módulo, e para que um teste em `M` não possa deixar migalhas que acidentalmente permitam que outro teste funcione. Isso significa que os exemplos podem usar livremente quaisquer nomes definidos no nível superior em `M` e nomes definidos anteriormente na docstring que está sendo executada. Os exemplos não podem ver nomes definidos em outras docstrings.

Você pode forçar o uso de seu próprio dicionário como contexto de execução passando `globs=seu_dicionario` para *testmod()* ou *testfile()*.

E quanto às exceções?

Não tem problema, desde que o traceback seja a única saída produzida pelo exemplo: basta colar o traceback.[1] Como os tracebacks contêm detalhes que provavelmente mudarão rapidamente (por exemplo, caminhos exatos de arquivos e números de linha), este é um caso em que o doctest trabalha duro para ser flexível no que aceita.

Exemplo simples:

```
>>> [1, 2, 3].remove(42)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

Esse doctest será bem-sucedido se *ValueError* for levantada, com o detalhe `list.remove(x): x not in list`, conforme mostrado.

A saída esperada para uma exceção deve começar com um cabeçalho do traceback, que pode ser qualquer uma das duas linhas a seguir, indentadas da mesma forma que a primeira linha do exemplo:

```
Traceback (most recent call last):
Traceback (innermost last):
```

The traceback header is followed by an optional traceback stack, whose contents are ignored by doctest. The traceback stack is typically omitted, or copied verbatim from an interactive session.

The traceback stack is followed by the most interesting part: the line(s) containing the exception type and detail. This is usually the last line of a traceback, but can extend across multiple lines if the exception has a multi-line detail:

```
>>> raise ValueError('multi\n    line\ndetail')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: multi
    line
detail
```

The last three lines (starting with *ValueError*) are compared against the exception's type and detail, and the rest are ignored.

Best practice is to omit the traceback stack, unless it adds significant documentation value to the example. So the last example is probably better as:

```
>>> raise ValueError('multi\n    line\ndetail')
Traceback (most recent call last):
    ...
ValueError: multi
```

(continua na próxima página)

[1] Examples containing both expected output and an exception are not supported. Trying to guess where one ends and the other begins is too error-prone, and that also makes for a confusing test.

(continuação da página anterior)

```
    line
detail
```

Note that tracebacks are treated very specially. In particular, in the rewritten example, the use of `...` is independent of doctest's *ELLIPSIS* option. The ellipsis in that example could be left out, or could just as well be three (or three hundred) commas or digits, or an indented transcript of a Monty Python skit.

Some details you should read once, but won't need to remember:

- Doctest can't guess whether your expected output came from an exception traceback or from ordinary printing. So, e.g., an example that expects `ValueError: 42 is prime` will pass whether *ValueError* is actually raised or if the example merely prints that traceback text. In practice, ordinary output rarely begins with a traceback header line, so this doesn't create real problems.
- Each line of the traceback stack (if present) must be indented further than the first line of the example, *or* start with a non-alphanumeric character. The first line following the traceback header indented the same and starting with an alphanumeric is taken to be the start of the exception detail. Of course this does the right thing for genuine tracebacks.
- When the *IGNORE_EXCEPTION_DETAIL* doctest option is specified, everything following the leftmost colon and any module information in the exception name is ignored.
- The interactive shell omits the traceback header line for some *SyntaxError*s. But doctest uses the traceback header line to distinguish exceptions from non-exceptions. So in the rare case where you need to test a `SyntaxError` that omits the traceback header, you will need to manually add the traceback header line to your test example.
- For some exceptions, Python displays the position of the error using `^` markers and tildes:

  ```
  >>> 1 + None
    File "<stdin>", line 1
      1 + None
      ~~^~~~~
  TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
  ```

  Since the lines showing the position of the error come before the exception type and detail, they are not checked by doctest. For example, the following test would pass, even though it puts the `^` marker in the wrong location:

  ```
  >>> 1 + None
    File "<stdin>", line 1
      1 + None
      ^~~~~~~
  TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
  ```

Flags opcionais

A number of option flags control various aspects of doctest's behavior. Symbolic names for the flags are supplied as module constants, which can be bitwise ORed together and passed to various functions. The names can also be used in *doctest directives*, and may be passed to the doctest command line interface via the `-o` option.

The first group of options define test semantics, controlling aspects of how doctest decides whether actual output matches an example's expected output:

`doctest.`**`DONT_ACCEPT_TRUE_FOR_1`**

By default, if an expected output block contains just `1`, an actual output block containing just `1` or just `True` is considered to be a match, and similarly for `0` versus `False`. When *DONT_ACCEPT_TRUE_FOR_1* is specified, neither substitution is allowed. The default behavior caters to that Python changed the return type of many functions from integer to boolean; doctests expecting "little integer" output still work in these cases. This option will probably go away, but not for several years.

doctest.**DONT_ACCEPT_BLANKLINE**

By default, if an expected output block contains a line containing only the string `<BLANKLINE>`, then that line will match a blank line in the actual output. Because a genuinely blank line delimits the expected output, this is the only way to communicate that a blank line is expected. When *DONT_ACCEPT_BLANKLINE* is specified, this substitution is not allowed.

doctest.**NORMALIZE_WHITESPACE**

When specified, all sequences of whitespace (blanks and newlines) are treated as equal. Any sequence of whitespace within the expected output will match any sequence of whitespace within the actual output. By default, whitespace must match exactly. *NORMALIZE_WHITESPACE* is especially useful when a line of expected output is very long, and you want to wrap it across multiple lines in your source.

doctest.**ELLIPSIS**

When specified, an ellipsis marker (`...`) in the expected output can match any substring in the actual output. This includes substrings that span line boundaries, and empty substrings, so it's best to keep usage of this simple. Complicated uses can lead to the same kinds of "oops, it matched too much!" surprises that `.*` is prone to in regular expressions.

doctest.**IGNORE_EXCEPTION_DETAIL**

When specified, doctests expecting exceptions pass so long as an exception of the expected type is raised, even if the details (message and fully qualified exception name) don't match.

For example, an example expecting `ValueError: 42` will pass if the actual exception raised is `ValueError: 3*14`, but will fail if, say, a *TypeError* is raised instead. It will also ignore any fully qualified name included before the exception class, which can vary between implementations and versions of Python and the code/libraries in use. Hence, all three of these variations will work with the flag specified:

```
>>> raise Exception('message')
Traceback (most recent call last):
Exception: message

>>> raise Exception('message')
Traceback (most recent call last):
builtins.Exception: message

>>> raise Exception('message')
Traceback (most recent call last):
__main__.Exception: message
```

Note that *ELLIPSIS* can also be used to ignore the details of the exception message, but such a test may still fail based on whether the module name is present or matches exactly.

Alterado na versão 3.2: *IGNORE_EXCEPTION_DETAIL* now also ignores any information relating to the module containing the exception under test.

doctest.**SKIP**

When specified, do not run the example at all. This can be useful in contexts where doctest examples serve as both documentation and test cases, and an example should be included for documentation purposes, but should not be checked. E.g., the example's output might be random; or the example might depend on resources which would be unavailable to the test driver.

The SKIP flag can also be used for temporarily "commenting out" examples.

doctest.**COMPARISON_FLAGS**

A bitmask or'ing together all the comparison flags above.

The second group of options controls how test failures are reported:

doctest.**REPORT_UDIFF**

When specified, failures that involve multi-line expected and actual outputs are displayed using a unified diff.

doctest.**REPORT_CDIFF**

When specified, failures that involve multi-line expected and actual outputs will be displayed using a context diff.

doctest.**REPORT_NDIFF**

When specified, differences are computed by `difflib.Differ`, using the same algorithm as the popular `ndiff.py` utility. This is the only method that marks differences within lines as well as across lines. For example, if a line of expected output contains digit `1` where actual output contains letter `l`, a line is inserted with a caret marking the mismatching column positions.

doctest.**REPORT_ONLY_FIRST_FAILURE**

When specified, display the first failing example in each doctest, but suppress output for all remaining examples. This will prevent doctest from reporting correct examples that break because of earlier failures; but it might also hide incorrect examples that fail independently of the first failure. When *REPORT_ONLY_FIRST_FAILURE* is specified, the remaining examples are still run, and still count towards the total number of failures reported; only the output is suppressed.

doctest.**FAIL_FAST**

When specified, exit after the first failing example and don't attempt to run the remaining examples. Thus, the number of failures reported will be at most 1. This flag may be useful during debugging, since examples after the first failure won't even produce debugging output.

doctest.**REPORTING_FLAGS**

A bitmask or'ing together all the reporting flags above.

There is also a way to register new option flag names, though this isn't useful unless you intend to extend *doctest* internals via subclassing:

doctest.**register_optionflag**(*name*)

Create a new option flag with a given name, and return the new flag's integer value. *register_optionflag()* can be used when subclassing *OutputChecker* or *DocTestRunner* to create new options that are supported by your subclasses. *register_optionflag()* should always be called using the following idiom:

```
MY_FLAG = register_optionflag('MY_FLAG')
```

Directives

Doctest directives may be used to modify the *option flags* for an individual example. Doctest directives are special Python comments following an example's source code:

```
directive:             "#" "doctest:" directive_options
directive_options:     directive_option ("," directive_option)*
directive_option:      on_or_off directive_option_name
on_or_off:             "+" | "-"
directive_option_name: "DONT_ACCEPT_BLANKLINE" | "NORMALIZE_WHITESPACE" | ...
```

Whitespace is not allowed between the `+` or `-` and the directive option name. The directive option name can be any of the option flag names explained above.

An example's doctest directives modify doctest's behavior for that single example. Use `+` to enable the named behavior, or `-` to disable it.

Por exemplo, este teste é aprovado:

```
>>> print(list(range(20)))  # doctest: +NORMALIZE_WHITESPACE
[0,   1,  2,  3,  4,  5,  6,  7,  8,  9,
10,  11, 12, 13, 14, 15, 16, 17, 18, 19]
```

Without the directive it would fail, both because the actual output doesn't have two blanks before the single-digit list elements, and because the actual output is on a single line. This test also passes, and also requires a directive to do so:

```
>>> print(list(range(20)))  # doctest: +ELLIPSIS
[0, 1, ..., 18, 19]
```

Multiple directives can be used on a single physical line, separated by commas:

```
>>> print(list(range(20)))  # doctest: +ELLIPSIS, +NORMALIZE_WHITESPACE
[0,    1, ...,   18,    19]
```

If multiple directive comments are used for a single example, then they are combined:

```
>>> print(list(range(20)))  # doctest: +ELLIPSIS
...                         # doctest: +NORMALIZE_WHITESPACE
[0,    1, ...,   18,    19]
```

As the previous example shows, you can add `...` lines to your example containing only directives. This can be useful when an example is too long for a directive to comfortably fit on the same line:

```
>>> print(list(range(5)) + list(range(10, 20)) + list(range(30, 40)))
... # doctest: +ELLIPSIS
[0, ..., 4, 10, ..., 19, 30, ..., 39]
```

Note that since all options are disabled by default, and directives apply only to the example they appear in, enabling options (via `+` in a directive) is usually the only meaningful choice. However, option flags can also be passed to functions that run doctests, establishing different defaults. In such cases, disabling an option via `-` in a directive can be useful.

Avisos

doctest is serious about requiring exact matches in expected output. If even a single character doesn't match, the test fails. This will probably surprise you a few times, as you learn exactly what Python does and doesn't guarantee about output. For example, when printing a set, Python doesn't guarantee that the element is printed in any particular order, so a test like

```
>>> foo()
{"spam", "eggs"}
```

is vulnerable! One workaround is to do

```
>>> foo() == {"spam", "eggs"}
True
```

instead. Another is to do

```
>>> d = sorted(foo())
>>> d
['eggs', 'spam']
```

There are others, but you get the idea.

Another bad idea is to print things that embed an object address, like

```
>>> id(1.0)  # certain to fail some of the time
7948648
>>> class C: pass
>>> C()  # the default repr() for instances embeds an address
<C object at 0x00AC18F0>
```

The *ELLIPSIS* directive gives a nice approach for the last example:

```
>>> C()   # doctest: +ELLIPSIS
<C object at 0x...>
```

Floating-point numbers are also subject to small output variations across platforms, because Python defers to the platform C library for some floating-point calculations, and C libraries vary widely in quality here.

```
>>> 1000**0.1  # risky
1.9952623149688797
>>> round(1000**0.1, 9) # safer
1.995262315
>>> print(f'{1000**0.1:.4f}') # much safer
1.9953
```

Numbers of the form `I/2.**J` are safe across all platforms, and I often contrive doctest examples to produce numbers of that form:

```
>>> 3./4  # utterly safe
0.75
```

Simple fractions are also easier for people to understand, and that makes for better documentation.

26.4.5 Basic API

The functions *testmod()* and *testfile()* provide a simple interface to doctest that should be sufficient for most basic uses. For a less formal introduction to these two functions, see sections *Uso simples: verificando exemplos em Docstrings* and *Utilização comum: Verificando exemplos em um arquivo texto*.

doctest.**testfile**(*filename*, *module_relative=True*, *name=None*, *package=None*, *globs=None*, *verbose=None*, *report=True*, *optionflags=0*, *extraglobs=None*, *raise_on_error=False*, *parser=DocTestParser()*, *encoding=None*)

All arguments except *filename* are optional, and should be specified in keyword form.

Test examples in the file named *filename*. Return `(failure_count, test_count)`.

Optional argument *module_relative* specifies how the filename should be interpreted:

- If *module_relative* is `True` (the default), then *filename* specifies an OS-independent module-relative path. By default, this path is relative to the calling module's directory; but if the *package* argument is specified, then it is relative to that package. To ensure OS-independence, *filename* should use `/` characters to separate path segments, and may not be an absolute path (i.e., it may not begin with `/`).
- If *module_relative* is `False`, then *filename* specifies an OS-specific path. The path may be absolute or relative; relative paths are resolved with respect to the current working directory.

Optional argument *name* gives the name of the test; by default, or if `None`, `os.path.basename(filename)` is used.

Optional argument *package* is a Python package or the name of a Python package whose directory should be used as the base directory for a module-relative filename. If no package is specified, then the calling module's directory is used as the base directory for module-relative filenames. It is an error to specify *package* if *module_relative* is `False`.

Optional argument *globs* gives a dict to be used as the globals when executing examples. A new shallow copy of this dict is created for the doctest, so its examples start with a clean slate. By default, or if `None`, a new empty dict is used.

Optional argument *extraglobs* gives a dict merged into the globals used to execute examples. This works like *dict.update()*: if *globs* and *extraglobs* have a common key, the associated value in *extraglobs* appears in the combined dict. By default, or if `None`, no extra globals are used. This is an advanced feature that allows parameterization of doctests. For example, a doctest can be written for a base class, using a generic name for the class, then reused to test any number of subclasses by passing an *extraglobs* dict mapping the generic name to the subclass to be tested.

Optional argument *verbose* prints lots of stuff if true, and prints only failures if false; by default, or if `None`, it's true if and only if `'-v'` is in *sys.argv*.

Optional argument *report* prints a summary at the end when true, else prints nothing at the end. In verbose mode, the summary is detailed, else the summary is very brief (in fact, empty if all tests passed).

Optional argument *optionflags* (default value `0`) takes the bitwise OR of option flags. See section *Flags opcionais*.

Optional argument *raise_on_error* defaults to false. If true, an exception is raised upon the first failure or unexpected exception in an example. This allows failures to be post-mortem debugged. Default behavior is to continue running examples.

Optional argument *parser* specifies a *DocTestParser* (or subclass) that should be used to extract tests from the files. It defaults to a normal parser (i.e., `DocTestParser()`).

Optional argument *encoding* specifies an encoding that should be used to convert the file to unicode.

doctest.**testmod**(*m=None*, *name=None*, *globs=None*, *verbose=None*, *report=True*, *optionflags=0*, *extraglobs=None*, *raise_on_error=False*, *exclude_empty=False*)

All arguments are optional, and all except for *m* should be specified in keyword form.

Test examples in docstrings in functions and classes reachable from module *m* (or module *__main__* if *m* is not supplied or is `None`), starting with `m.__doc__`.

Also test examples reachable from dict `m.__test__`, if it exists. `m.__test__` maps names (strings) to functions, classes and strings; function and class docstrings are searched for examples; strings are searched directly, as if they were docstrings.

Only docstrings attached to objects belonging to module *m* are searched.

Return `(failure_count, test_count)`.

Optional argument *name* gives the name of the module; by default, or if `None`, `m.__name__` is used.

Optional argument *exclude_empty* defaults to false. If true, objects for which no doctests are found are excluded from consideration. The default is a backward compatibility hack, so that code still using *doctest.master.summarize* in conjunction with *testmod()* continues to get output for objects with no tests. The *exclude_empty* argument to the newer *DocTestFinder* constructor defaults to true.

Optional arguments *extraglobs*, *verbose*, *report*, *optionflags*, *raise_on_error*, and *globs* are the same as for function *testfile()* above, except that *globs* defaults to `m.__dict__`.

doctest.**run_docstring_examples**(*f*, *globs*, *verbose=False*, *name='NoName'*, *compileflags=None*, *optionflags=0*)

Test examples associated with object *f*; for example, *f* may be a string, a module, a function, or a class object.

A shallow copy of dictionary argument *globs* is used for the execution context.

Optional argument *name* is used in failure messages, and defaults to `"NoName"`.

If optional argument *verbose* is true, output is generated even if there are no failures. By default, output is generated only in case of an example failure.

Optional argument *compileflags* gives the set of flags that should be used by the Python compiler when running the examples. By default, or if `None`, flags are deduced corresponding to the set of future features found in *globs*.

Optional argument *optionflags* works as for function *testfile()* above.

26.4.6 API do Unittest

As your collection of doctest'ed modules grows, you'll want a way to run all their doctests systematically. *doctest* provides two functions that can be used to create *unittest* test suites from modules and text files containing doctests. To integrate with *unittest* test discovery, include a *load_tests* function in your test module:

```
import unittest
import doctest
import my_module_with_doctests

def load_tests(loader, tests, ignore):
    tests.addTests(doctest.DocTestSuite(my_module_with_doctests))
    return tests
```

There are two main functions for creating *unittest.TestSuite* instances from text files and modules with doctests:

doctest.**DocFileSuite**(**paths*, *module_relative=True*, *package=None*, *setUp=None*, *tearDown=None*, *globs=None*, *optionflags=0*, *parser=DocTestParser()*, *encoding=None*)

Convert doctest tests from one or more text files to a *unittest.TestSuite*.

The returned *unittest.TestSuite* is to be run by the unittest framework and runs the interactive examples in each file. If an example in any file fails, then the synthesized unit test fails, and a *failureException* exception is raised showing the name of the file containing the test and a (sometimes approximate) line number. If all the examples in a file are skipped, then the synthesized unit test is also marked as skipped.

Pass one or more paths (as strings) to text files to be examined.

Options may be provided as keyword arguments:

Optional argument *module_relative* specifies how the filenames in *paths* should be interpreted:

- If *module_relative* is `True` (the default), then each filename in *paths* specifies an OS-independent module-relative path. By default, this path is relative to the calling module's directory; but if the *package* argument is specified, then it is relative to that package. To ensure OS-independence, each filename should use `/` characters to separate path segments, and may not be an absolute path (i.e., it may not begin with `/`).
- If *module_relative* is `False`, then each filename in *paths* specifies an OS-specific path. The path may be absolute or relative; relative paths are resolved with respect to the current working directory.

Optional argument *package* is a Python package or the name of a Python package whose directory should be used as the base directory for module-relative filenames in *paths*. If no package is specified, then the calling module's directory is used as the base directory for module-relative filenames. It is an error to specify *package* if *module_relative* is `False`.

Optional argument *setUp* specifies a set-up function for the test suite. This is called before running the tests in each file. The *setUp* function will be passed a *DocTest* object. The *setUp* function can access the test globals as the *globs* attribute of the test passed.

Optional argument *tearDown* specifies a tear-down function for the test suite. This is called after running the tests in each file. The *tearDown* function will be passed a *DocTest* object. The *tearDown* function can access the test globals as the *globs* attribute of the test passed.

Optional argument *globs* is a dictionary containing the initial global variables for the tests. A new copy of this dictionary is created for each test. By default, *globs* is a new empty dictionary.

Optional argument *optionflags* specifies the default doctest options for the tests, created by or-ing together individual option flags. See section *Flags opcionais*. See function *set_unittest_reportflags()* below for a better way to set reporting options.

Optional argument *parser* specifies a *DocTestParser* (or subclass) that should be used to extract tests from the files. It defaults to a normal parser (i.e., `DocTestParser()`).

Optional argument *encoding* specifies an encoding that should be used to convert the file to unicode.

The global `__file__` is added to the globals provided to doctests loaded from a text file using *DocFileSuite()*.

doctest.**DocTestSuite**(*module=None*, *globs=None*, *extraglobs=None*, *test_finder=None*, *setUp=None*, *tearDown=None*, *optionflags=0*, *checker=None*)

Convert doctest tests for a module to a *unittest.TestSuite*.

The returned *unittest.TestSuite* is to be run by the unittest framework and runs each doctest in the module. Each docstring is run as a separate unit test. If any of the doctests fail, then the synthesized unit test fails, and a *unittest.TestCase.failureException* exception is raised showing the name of the file containing the test and a (sometimes approximate) line number. If all the examples in a docstring are skipped, then the

Optional argument *module* provides the module to be tested. It can be a module object or a (possibly dotted) module name. If not specified, the module calling this function is used.

Optional argument *globs* is a dictionary containing the initial global variables for the tests. A new copy of this dictionary is created for each test. By default, *globs* is the module's `__dict__`.

Optional argument *extraglobs* specifies an extra set of global variables, which is merged into *globs*. By default, no extra globals are used.

Optional argument *test_finder* is the *DocTestFinder* object (or a drop-in replacement) that is used to extract doctests from the module.

Optional arguments *setUp*, *tearDown*, and *optionflags* are the same as for function *DocFileSuite()* above, but they are called for each docstring.

This function uses the same search technique as *testmod()*.

Alterado na versão 3.5: *DocTestSuite()* returns an empty *unittest.TestSuite* if *module* contains no docstrings instead of raising *ValueError*.

Under the covers, *DocTestSuite()* creates a *unittest.TestSuite* out of `doctest.DocTestCase` instances, and `DocTestCase` is a subclass of *unittest.TestCase*. `DocTestCase` isn't documented here (it's an internal detail), but studying its code can answer questions about the exact details of *unittest* integration.

Similarly, *DocFileSuite()* creates a *unittest.TestSuite* out of `doctest.DocFileCase` instances, and `DocFileCase` is a subclass of `DocTestCase`.

So both ways of creating a *unittest.TestSuite* run instances of `DocTestCase`. This is important for a subtle reason: when you run *doctest* functions yourself, you can control the `doctest` options in use directly, by passing option flags to `doctest` functions. However, if you're writing a *unittest* framework, `unittest` ultimately controls when and how tests get run. The framework author typically wants to control `doctest` reporting options (perhaps, e.g., specified by command line options), but there's no way to pass options through `unittest` to `doctest` test runners.

For this reason, *doctest* also supports a notion of `doctest` reporting flags specific to *unittest* support, via this function:

`doctest.`**`set_unittest_reportflags`**(*flags*)

Set the *doctest* reporting flags to use.

Argument *flags* takes the bitwise OR of option flags. See section *Flags opcionais*. Only "reporting flags" can be used.

This is a module-global setting, and affects all future doctests run by module *unittest*: the `runTest()` method of `DocTestCase` looks at the option flags specified for the test case when the `DocTestCase` instance was constructed. If no reporting flags were specified (which is the typical and expected case), `doctest`'s `unittest` reporting flags are bitwise ORed into the option flags, and the option flags so augmented are passed to the *DocTestRunner* instance created to run the doctest. If any reporting flags were specified when the `DocTestCase` instance was constructed, `doctest`'s `unittest` reporting flags are ignored.

The value of the *unittest* reporting flags in effect before the function was called is returned by the function.

26.4.7 Advanced API

The basic API is a simple wrapper that's intended to make doctest easy to use. It is fairly flexible, and should meet most users' needs; however, if you require more fine-grained control over testing, or wish to extend doctest's capabilities, then you should use the advanced API.

The advanced API revolves around two container classes, which are used to store the interactive examples extracted from doctest cases:

- *Example*: A single Python *statement*, paired with its expected output.
- *DocTest*: A collection of *Example*s, typically extracted from a single docstring or text file.

Additional processing classes are defined to find, parse, and run, and check doctest examples:

- *DocTestFinder*: Finds all docstrings in a given module, and uses a *DocTestParser* to create a *DocTest* from every docstring that contains interactive examples.
- *DocTestParser*: Creates a *DocTest* object from a string (such as an object's docstring).
- *DocTestRunner*: Executes the examples in a *DocTest*, and uses an *OutputChecker* to verify their output.
- *OutputChecker*: Compares the actual output from a doctest example with the expected output, and decides whether they match.

The relationships among these processing classes are summarized in the following diagram:

```
                            list of:
+------+                   +---------+
|module| --DocTestFinder-> | DocTest | --DocTestRunner-> results
+------+    |        ^     +---------+     |        ^     (printed)
            |        |     | Example |     |        |
            v        |     |   ...   |     v        |
          DocTestParser    | Example |   OutputChecker
                           +---------+
```

DocTest Objects

class doctest.**DocTest**(*examples*, *globs*, *name*, *filename*, *lineno*, *docstring*)

A collection of doctest examples that should be run in a single namespace. The constructor arguments are used to initialize the attributes of the same names.

DocTest defines the following attributes. They are initialized by the constructor, and should not be modified directly.

examples

A list of *Example* objects encoding the individual interactive Python examples that should be run by this test.

globs

The namespace (aka globals) that the examples should be run in. This is a dictionary mapping names to values. Any changes to the namespace made by the examples (such as binding new variables) will be reflected in *globs* after the test is run.

name

A string name identifying the *DocTest*. Typically, this is the name of the object or file that the test was extracted from.

filename

The name of the file that this *DocTest* was extracted from; or `None` if the filename is unknown, or if the `DocTest` was not extracted from a file.

lineno

The line number within *filename* where this *DocTest* begins, or `None` if the line number is unavailable. This line number is zero-based with respect to the beginning of the file.

docstring

The string that the test was extracted from, or `None` if the string is unavailable, or if the test was not extracted from a string.

Example Objects

class doctest.**Example**(*source*, *want*, *exc_msg=None*, *lineno=0*, *indent=0*, *options=None*)

A single interactive example, consisting of a Python statement and its expected output. The constructor arguments are used to initialize the attributes of the same names.

Example defines the following attributes. They are initialized by the constructor, and should not be modified directly.

source

A string containing the example's source code. This source code consists of a single Python statement, and always ends with a newline; the constructor adds a newline when necessary.

want

The expected output from running the example's source code (either from stdout, or a traceback in case of exception). *want* ends with a newline unless no output is expected, in which case it's an empty string. The constructor adds a newline when necessary.

exc_msg

The exception message generated by the example, if the example is expected to generate an exception; or `None` if it is not expected to generate an exception. This exception message is compared against the return value of *traceback.format_exception_only()*. *exc_msg* ends with a newline unless it's `None`. The constructor adds a newline if needed.

lineno

The line number within the string containing this example where the example begins. This line number is zero-based with respect to the beginning of the containing string.

indent

The example's indentation in the containing string, i.e., the number of space characters that precede the example's first prompt.

options

A dictionary mapping from option flags to `True` or `False`, which is used to override default options for this example. Any option flags not contained in this dictionary are left at their default value (as specified by the *DocTestRunner*'s *optionflags*). By default, no options are set.

DocTestFinder objects

class doctest.**DocTestFinder**(*verbose=False*, *parser=DocTestParser()*, *recurse=True*, *exclude_empty=True*)

A processing class used to extract the *DocTest*s that are relevant to a given object, from its docstring and the docstrings of its contained objects. *DocTest*s can be extracted from modules, classes, functions, methods, staticmethods, classmethods, and properties.

The optional argument *verbose* can be used to display the objects searched by the finder. It defaults to `False` (no output).

The optional argument *parser* specifies the *DocTestParser* object (or a drop-in replacement) that is used to extract doctests from docstrings.

If the optional argument *recurse* is false, then *DocTestFinder.find()* will only examine the given object, and not any contained objects.

If the optional argument *exclude_empty* is false, then *DocTestFinder.find()* will include tests for objects with empty docstrings.

DocTestFinder defines the following method:

find(*obj[, name][, module][, globs][, extraglobs]*)

Return a list of the *DocTest*s that are defined by *obj*'s docstring, or by any of its contained objects' docstrings.

The optional argument *name* specifies the object's name; this name will be used to construct names for the returned *DocTest*s. If *name* is not specified, then `obj.__name__` is used.

The optional parameter *module* is the module that contains the given object. If the module is not specified or is `None`, then the test finder will attempt to automatically determine the correct module. The object's module is used:

- As a default namespace, if *globs* is not specified.
- To prevent the DocTestFinder from extracting DocTests from objects that are imported from other modules. (Contained objects with modules other than *module* are ignored.)
- To find the name of the file containing the object.
- To help find the line number of the object within its file.

If *module* is `False`, no attempt to find the module will be made. This is obscure, of use mostly in testing doctest itself: if *module* is `False`, or is `None` but cannot be found automatically, then all objects are considered to belong to the (non-existent) module, so all contained objects will (recursively) be searched for doctests.

The globals for each *DocTest* is formed by combining *globs* and *extraglobs* (bindings in *extraglobs* override bindings in *globs*). A new shallow copy of the globals dictionary is created for each `DocTest`. If *globs* is not specified, then it defaults to the module's `__dict__`, if specified, or `{}` otherwise. If *extraglobs* is not specified, then it defaults to `{}`.

DocTestParser objects

class doctest.**DocTestParser**

A processing class used to extract interactive examples from a string, and use them to create a *DocTest* object.

DocTestParser defines the following methods:

get_doctest(*string*, *globs*, *name*, *filename*, *lineno*)

Extract all doctest examples from the given string, and collect them into a *DocTest* object.

globs, *name*, *filename*, and *lineno* are attributes for the new `DocTest` object. See the documentation for *DocTest* for more information.

get_examples(*string*, *name='<string>'*)

Extract all doctest examples from the given string, and return them as a list of *Example* objects. Line numbers are 0-based. The optional argument *name* is a name identifying this string, and is only used for error messages.

parse(*string*, *name='<string>'*)

Divide the given string into examples and intervening text, and return them as a list of alternating *Example*s and strings. Line numbers for the `Example`s are 0-based. The optional argument *name* is a name identifying this string, and is only used for error messages.

TestResults objects

class doctest.**TestResults**(*failed*, *attempted*)

failed

Number of failed tests.

attempted

Number of attempted tests.

skipped

Number of skipped tests.

Adicionado na versão 3.13.

DocTestRunner objects

class doctest.**DocTestRunner**(*checker=None*, *verbose=None*, *optionflags=0*)

A processing class used to execute and verify the interactive examples in a *DocTest*.

The comparison between expected outputs and actual outputs is done by an *OutputChecker*. This comparison may be customized with a number of option flags; see section *Flags opcionais* for more information. If the option flags are insufficient, then the comparison may also be customized by passing a subclass of `OutputChecker` to the constructor.

The test runner's display output can be controlled in two ways. First, an output function can be passed to *run()*; this function will be called with strings that should be displayed. It defaults to `sys.stdout.write`. If capturing the output is not sufficient, then the display output can be also customized by subclassing DocTestRunner, and overriding the methods *report_start()*, *report_success()*, *report_unexpected_exception()*, and *report_failure()*.

The optional keyword argument *checker* specifies the *OutputChecker* object (or drop-in replacement) that should be used to compare the expected outputs to the actual outputs of doctest examples.

The optional keyword argument *verbose* controls the *DocTestRunner*'s verbosity. If *verbose* is `True`, then information is printed about each example, as it is run. If *verbose* is `False`, then only failures are printed. If *verbose* is unspecified, or `None`, then verbose output is used iff the command-line switch `-v` is used.

The optional keyword argument *optionflags* can be used to control how the test runner compares expected output to actual output, and how it displays failures. For more information, see section *Flags opcionais*.

The test runner accumulates statistics. The aggregated number of attempted, failed and skipped examples is also available via the *tries*, *failures* and *skips* attributes. The *run()* and *summarize()* methods return a *TestResults* instance.

DocTestRunner defines the following methods:

report_start(*out*, *test*, *example*)

Report that the test runner is about to process the given example. This method is provided to allow subclasses of *DocTestRunner* to customize their output; it should not be called directly.

example is the example about to be processed. *test* is the test containing *example*. *out* is the output function that was passed to *DocTestRunner.run()*.

report_success(*out*, *test*, *example*, *got*)

Report that the given example ran successfully. This method is provided to allow subclasses of *DocTestRunner* to customize their output; it should not be called directly.

example is the example about to be processed. *got* is the actual output from the example. *test* is the test containing *example*. *out* is the output function that was passed to *DocTestRunner.run()*.

report_failure(*out*, *test*, *example*, *got*)

Report that the given example failed. This method is provided to allow subclasses of *DocTestRunner* to customize their output; it should not be called directly.

example is the example about to be processed. *got* is the actual output from the example. *test* is the test containing *example*. *out* is the output function that was passed to *DocTestRunner.run()*.

report_unexpected_exception(*out*, *test*, *example*, *exc_info*)

Report that the given example raised an unexpected exception. This method is provided to allow subclasses of *DocTestRunner* to customize their output; it should not be called directly.

example is the example about to be processed. *exc_info* is a tuple containing information about the unexpected exception (as returned by *sys.exc_info()*). *test* is the test containing *example*. *out* is the output function that was passed to *DocTestRunner.run()*.

run(*test*, *compileflags=None*, *out=None*, *clear_globs=True*)

Run the examples in *test* (a *DocTest* object), and display the results using the writer function *out*. Return a *TestResults* instance.

The examples are run in the namespace `test.globs`. If *clear_globs* is true (the default), then this namespace will be cleared after the test runs, to help with garbage collection. If you would like to examine the namespace after the test completes, then use *clear_globs=False*.

compileflags gives the set of flags that should be used by the Python compiler when running the examples. If not specified, then it will default to the set of future-import flags that apply to *globs*.

The output of each example is checked using the *DocTestRunner*'s output checker, and the results are formatted by the `DocTestRunner.report_*()` methods.

summarize(*verbose=None*)

Print a summary of all the test cases that have been run by this DocTestRunner, and return a *TestResults* instance.

The optional *verbose* argument controls how detailed the summary is. If the verbosity is not specified, then the *DocTestRunner*'s verbosity is used.

DocTestParser has the following attributes:

tries

Number of attempted examples.

failures

Number of failed examples.

skips

Number of skipped examples.

Adicionado na versão 3.13.

OutputChecker objects

class doctest.**OutputChecker**

A class used to check the whether the actual output from a doctest example matches the expected output. *OutputChecker* defines two methods: *check_output()*, which compares a given pair of outputs, and returns `True` if they match; and *output_difference()*, which returns a string describing the differences between two outputs.

OutputChecker define os seguintes métodos:

check_output(*want*, *got*, *optionflags*)

Return `True` iff the actual output from an example (*got*) matches the expected output (*want*). These strings are always considered to match if they are identical; but depending on what option flags the test runner is using, several non-exact match types are also possible. See section *Flags opcionais* for more information about option flags.

output_difference(*example*, *got*, *optionflags*)

Return a string describing the differences between the expected output for a given example (*example*) and the actual output (*got*). *optionflags* is the set of option flags used to compare *want* and *got*.

26.4.8 Depuração

Doctest provides several mechanisms for debugging doctest examples:

- Several functions convert doctests to executable Python programs, which can be run under the Python debugger, *pdb*.
- The *DebugRunner* class is a subclass of *DocTestRunner* that raises an exception for the first failing example, containing information about that example. This information can be used to perform post-mortem debugging on the example.

- The *unittest* cases generated by *DocTestSuite()* support the *debug()* method defined by *unittest.TestCase*.
- You can add a call to *pdb.set_trace()* in a doctest example, and you'll drop into the Python debugger when that line is executed. Then you can inspect current values of variables, and so on. For example, suppose `a.py` contains just this module docstring:

```
"""
>>> def f(x):
...     g(x*2)
>>> def g(x):
...     print(x+3)
...     import pdb; pdb.set_trace()
>>> f(3)
9
"""
```

Then an interactive Python session may look like this:

```
>>> import a, doctest
>>> doctest.testmod(a)
--Return--
> <doctest a[1]>(3)g()->None
-> import pdb; pdb.set_trace()
(Pdb) list
  1     def g(x):
  2         print(x+3)
  3  ->     import pdb; pdb.set_trace()
[EOF]
(Pdb) p x
6
(Pdb) step
--Return--
> <doctest a[0]>(2)f()->None
-> g(x*2)
(Pdb) list
  1     def f(x):
  2  ->     g(x*2)
[EOF]
(Pdb) p x
3
(Pdb) step
--Return--
> <doctest a[2]>(1)?()->None
-> f(3)
(Pdb) cont
(0, 3)
>>>
```

Functions that convert doctests to Python code, and possibly run the synthesized code under the debugger:

`doctest.`**`script_from_examples`**`(`*s*`)`

Convert text with examples to a script.

Argument *s* is a string containing doctest examples. The string is converted to a Python script, where doctest examples in *s* are converted to regular code, and everything else is converted to Python comments. The generated script is returned as a string. For example,

```
import doctest
print(doctest.script_from_examples(r"""
    Set x and y to 1 and 2.
    >>> x, y = 1, 2

    Print their sum:
    >>> print(x+y)
    3
"""))
```

displays:

```
# Set x and y to 1 and 2.
x, y = 1, 2
#
# Print their sum:
print(x+y)
# Expected:
## 3
```

This function is used internally by other functions (see below), but can also be useful when you want to transform an interactive Python session into a Python script.

`doctest.`**`testsource`**(*module*, *name*)

Convert the doctest for an object to a script.

Argument *module* is a module object, or dotted name of a module, containing the object whose doctests are of interest. Argument *name* is the name (within the module) of the object with the doctests of interest. The result is a string, containing the object's docstring converted to a Python script, as described for *script_from_examples()* above. For example, if module `a.py` contains a top-level function `f()`, then

```
import a, doctest
print(doctest.testsource(a, "a.f"))
```

prints a script version of function `f()`'s docstring, with doctests converted to code, and the rest placed in comments.

`doctest.`**`debug`**(*module*, *name*, *pm=False*)

Debug the doctests for an object.

The *module* and *name* arguments are the same as for function *testsource()* above. The synthesized Python script for the named object's docstring is written to a temporary file, and then that file is run under the control of the Python debugger, *pdb*.

A shallow copy of `module.__dict__` is used for both local and global execution context.

Optional argument *pm* controls whether post-mortem debugging is used. If *pm* has a true value, the script file is run directly, and the debugger gets involved only if the script terminates via raising an unhandled exception. If it does, then post-mortem debugging is invoked, via *pdb.post_mortem()*, passing the traceback object from the unhandled exception. If *pm* is not specified, or is false, the script is run under the debugger from the start, via passing an appropriate *exec()* call to *pdb.run()*.

`doctest.`**`debug_src`**(*src*, *pm=False*, *globs=None*)

Debug the doctests in a string.

This is like function *debug()* above, except that a string containing doctest examples is specified directly, via the *src* argument.

Optional argument *pm* has the same meaning as in function *debug()* above.

Optional argument *globs* gives a dictionary to use as both local and global execution context. If not specified, or `None`, an empty dictionary is used. If specified, a shallow copy of the dictionary is used.

The *DebugRunner* class, and the special exceptions it may raise, are of most interest to testing framework authors, and will only be sketched here. See the source code, and especially *DebugRunner*'s docstring (which is a doctest!) for more details:

class doctest.**DebugRunner**(*checker=None*, *verbose=None*, *optionflags=0*)

A subclass of *DocTestRunner* that raises an exception as soon as a failure is encountered. If an unexpected exception occurs, an *UnexpectedException* exception is raised, containing the test, the example, and the original exception. If the output doesn't match, then a *DocTestFailure* exception is raised, containing the test, the example, and the actual output.

For information about the constructor parameters and methods, see the documentation for *DocTestRunner* in section *Advanced API*.

There are two exceptions that may be raised by *DebugRunner* instances:

exception doctest.**DocTestFailure**(*test*, *example*, *got*)

An exception raised by *DocTestRunner* to signal that a doctest example's actual output did not match its expected output. The constructor arguments are used to initialize the attributes of the same names.

DocTestFailure defines the following attributes:

DocTestFailure.**test**

The *DocTest* object that was being run when the example failed.

DocTestFailure.**example**

The *Example* that failed.

DocTestFailure.**got**

The example's actual output.

exception doctest.**UnexpectedException**(*test*, *example*, *exc_info*)

An exception raised by *DocTestRunner* to signal that a doctest example raised an unexpected exception. The constructor arguments are used to initialize the attributes of the same names.

UnexpectedException define os seguintes atributos:

UnexpectedException.**test**

The *DocTest* object that was being run when the example failed.

UnexpectedException.**example**

The *Example* that failed.

UnexpectedException.**exc_info**

A tuple containing information about the unexpected exception, as returned by *sys.exc_info()*.

26.4.9 Soapbox

As mentioned in the introduction, *doctest* has grown to have three primary uses:

1. Checking examples in docstrings.
2. Regression testing.
3. Executable documentation / literate testing.

These uses have different requirements, and it is important to distinguish them. In particular, filling your docstrings with obscure test cases makes for bad documentation.

When writing a docstring, choose docstring examples with care. There's an art to this that needs to be learned—it may not be natural at first. Examples should add genuine value to the documentation. A good example can often be worth many words. If done with care, the examples will be invaluable for your users, and will pay back the time it takes to collect them many times over as the years go by and things change. I'm still amazed at how often one of my *doctest* examples stops working after a "harmless" change.

Doctest also makes an excellent tool for regression testing, especially if you don't skimp on explanatory text. By interleaving prose and examples, it becomes much easier to keep track of what's actually being tested, and why.

When a test fails, good prose can make it much easier to figure out what the problem is, and how it should be fixed. It's true that you could write extensive comments in code-based testing, but few programmers do. Many have found that using doctest approaches instead leads to much clearer tests. Perhaps this is simply because doctest makes writing prose a little easier than writing code, while writing comments in code is a little harder. I think it goes deeper than just that: the natural attitude when writing a doctest-based test is that you want to explain the fine points of your software, and illustrate them with examples. This in turn naturally leads to test files that start with the simplest features, and logically progress to complications and edge cases. A coherent narrative is the result, instead of a collection of isolated functions that test isolated bits of functionality seemingly at random. It's a different attitude, and produces different results, blurring the distinction between testing and explaining.

Regression testing is best confined to dedicated objects or files. There are several options for organizing tests:

- Write text files containing test cases as interactive examples, and test the files using *testfile()* or *DocFileSuite()*. This is recommended, although is easiest to do for new projects, designed from the start to use doctest.
- Define functions named `_regrtest_topic` that consist of single docstrings, containing test cases for the named topics. These functions can be included in the same file as the module, or separated out into a separate test file.
- Define a *__test__* dictionary mapping from regression test topics to docstrings containing test cases.

When you have placed your tests in a module, the module can itself be the test runner. When a test fails, you can arrange for your test runner to re-run only the failing doctest while you debug the problem. Here is a minimal example of such a test runner:

```
if __name__ == '__main__':
    import doctest
    flags = doctest.REPORT_NDIFF|doctest.FAIL_FAST
    if len(sys.argv) > 1:
        name = sys.argv[1]
        if name in globals():
            obj = globals()[name]
        else:
            obj = __test__[name]
        doctest.run_docstring_examples(obj, globals(), name=name,
                                       optionflags=flags)
    else:
        fail, total = doctest.testmod(optionflags=flags)
        print(f"{fail} failures out of {total} tests")
```

26.5 `unittest` — Unit testing framework

Código-fonte: Lib/unittest/__init__.py

(Caso já estejas familiarizado com os conceitos básicos de testes, poderás querer ignorar *a lista de métodos assertivos*.)

O framework de testes unitários *unittest* foi originalmente inspirado no JUnit e tem um sabor semelhante contendo as principais estruturas de teste de unidades existentes em outras linguagens. Ele suporta a automação de testes, compartilhamento de configuração e código de desligamento para testes, agregação de testes em coleções e independência dos testes do framework de relatórios.

Para conseguir isso, o módulo *unittest* suporta alguns conceitos importantes de forma orientada a objetos:

definição de contexto de teste
: Uma *definição de contexto de teste* representa a preparação necessária pra performar um ou mais testes, além de quaisquer ações de limpeza relacionadas. Isso pode envolver, por exemplo, criar bancos de dados proxy ou temporários, diretórios ou iniciar um processo de servidor.

caso de teste
: Um *test case* é uma unidade de teste individual. O mesmo verifica uma resposta específica a um determinado conjunto de entradas. O `unittest` fornece uma classe base, `TestCase`, que pode ser usada para criar novos casos de teste.

Suíte de Testes
: Uma *test suite* é uma coleção de casos de teste, conjuntos de teste ou ambos. O mesmo é usado para agregar testes que devem ser executados juntos.

test runner
: Um *test runner* é um componente que orquestra a execução de testes e fornece o resultado para o usuário. O runner pode usar uma interface gráfica, uma interface textual ou retornar um valor especial para indicar os resultados da execução dos testes.

Ver também

Módulo `doctest`
: Outro módulo de suporte a testes com um sabor muito diferente.

Simple Smalltalk Testing: With Patterns
: O documento original de Kent Beck sobre estruturas de teste usando o padrão compartilhado por `unittest`.

pytest
: É um framework externo do unittest com uma sintaxe mais leve para escrever testes. Por exemplo, `assert func(10) == 42`.

The Python Testing Tools Taxonomy
: Uma extensa lista de ferramentas para testar código Python, incluindo estruturas de teste funcionais e bibliotecas de objetos simulados.

Testing in Python Mailing List
: Um grupo de interesse especial para discussão de testes e ferramentas de teste, em Python.

The script `Tools/unittestgui/unittestgui.py` in the Python source distribution is a GUI tool for test discovery and execution. This is intended largely for ease of use for those new to unit testing. For production environments it is recommended that tests be driven by a continuous integration system such as Buildbot, Jenkins, GitHub Actions, or AppVeyor.

26.5.1 Exemplo Básico

O módulo `unittest` fornece um conjunto amplo de ferramentas para a construção e execução de testes. Esta seção demonstra que um pequeno subconjunto das ferramentas é suficiente para atender às necessidades da maioria dos usuários.

Aqui temos um simples Script para testar três métodos de String:

```
import unittest

class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')

    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())

    def test_split(self):
        s = 'hello world'
```

(continua na próxima página)

(continuação da página anterior)

```
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a string
        with self.assertRaises(TypeError):
            s.split(2)

if __name__ == '__main__':
    unittest.main()
```

A test case is created by subclassing *unittest.TestCase*. The three individual tests are defined with methods whose names start with the letters `test`. This naming convention informs the test runner about which methods represent tests.

O cerne de cada teste é a invocação de um método *assertEqual()* para verificar se há um resultado esperado; *assertTrue()* ou *assertFalse()* para verificar uma condição; ou *assertRaises()* para verificar se uma exceção específica será levantada. Esses métodos são usados ao invés de utilizar a expressão `assert` para que o runner de teste possa acumular todos os resultados do teste e produzir um relatório.

Os métodos *setUp()* e *tearDown()* permitem que você defina instruções que serão executadas antes e depois de cada método de teste. Eles são abordados em mais detalhes na seção *Organizando código teste*.

O bloco final mostra uma maneira simples de executar os testes. A função *unittest.main()* fornece uma interface de linha de comando para o Script de teste. Quando executado a partir da linha de comando, o Script acima produz uma saída que se parece com isso:

```
...
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
```

Passando a opção `-v` para o nosso Script de teste instruirá a função *unittest.main()* a habilitar um nível mais alto de verbosidade e produzirá a seguinte saída:

```
test_isupper (__main__.TestStringMethods.test_isupper) ... ok
test_split (__main__.TestStringMethods.test_split) ... ok
test_upper (__main__.TestStringMethods.test_upper) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

Os exemplos acima mostram os recursos mais utilizados *unittest* que são suficientes para atender a muitas necessidades de testes diários. O restante da documentação explora o conjunto completo de recursos desde os primeiros princípios.

Alterado na versão 3.11: The behavior of returning a value from a test method (other than the default `None` value), is now deprecated.

26.5.2 Interface de Linha de Comando

O módulo unittest pode ser usado diretamente da linha de comando para executar testes de módulos, classes ou mesmo testes de métodos individuais:

```
python -m unittest test_module1 test_module2
python -m unittest test_module.TestClass
python -m unittest test_module.TestClass.test_method
```

Você pode passar uma lista com qualquer combinação de nomes de módulos e nomes de classes ou métodos totalmente qualificados.

Os módulos de teste podem ser especificados por caminhos de arquivo também:

```
python -m unittest tests/test_something.py
```

Isso permite que você use o auto-completar do console/shell para especificar o módulo de teste. O arquivo especificado precisa ser "importável" como um módulo. O caminho é convertido para um nome de módulo ao remover a extensão *.py* e conversando os separadores do caminho em .. Se você quer executar um arquivo de teste que não é importável como um módulo, você deve executar o arquivo diretamente.

Você pode executar os testes com mais detalhes (maior verbosidade) ao usar o sinalizador -v:

```
python -m unittest -v test_module
```

Quando executado sem argumentos *Test Discovery* é iniciado:

```
python -m unittest
```

Para uma lista de todas as opções de linha de comando:

```
python -m unittest -h
```

Alterado na versão 3.2: Em versões mais antigas, era possível de executar apenas testes de métodos individuais e não de módulos ou classes.

Adicionado na versão 3.14: A saída é colorizada por padrão e ela pode ser controlada usando variáveis de ambiente.

Opções de linha de comando

unittest suporta as seguintes opções de linha de comando:

-b, --buffer

Os streams da saída padrão e do erro padrão são carregados durante a execução do teste. A saída de um teste que passou é descartada. A saída geralmente é mostrada quando um teste falha e é adicionada às mensagens de falha.

-c, --catch

`Control-C` durante a execução do teste aguarda até que o teste corrente termine e a partir disso mostra todos os resultados até o momento. Um segundo `Control-C` invoca uma exceção *`KeyboardInterrupt`* comum.

Veja *Signal Handling* para as funções que provêm essa funcionalidade.

-f, --failfast

Parar a execução do teste no primeiro erro ou falha.

-k

Only run test methods and classes that match the pattern or substring. This option may be used multiple times, in which case all test cases that match any of the given patterns are included.

Padrões que contém um caractere curinga (`*`) são combinados com os testes pelo método *`fnmatch.fnmatchcase()`*; caso contrário, é utilizada uma combinação simples de substrings, diferenciando-se letras maiúsculas e minúsculas.

Padrões são combinados com o nome completo qualificado do método de teste no formato que ele é importado pelo carregador.

Por exemplo, `-k foo` combina com `foo_tests.SomeTest.test_something`, `bar_tests.SomeTest.test_foo`, mas não com `bar_tests.FooTest.test_something`.

--locals

Mostra variáveis locais no traceback.

--durations `N`

Show the N slowest test cases (N=0 for all).

Adicionado na versão 3.2: As opções de linha de comando `-b`, `-c` e `-f` foram adicionadas.

Adicionado na versão 3.5: A opção de linha de comando `--locals`.

Adicionado na versão 3.7: A opção de linha de comando `-k`.

Adicionado na versão 3.12: The command-line option `--durations`.

A linha de comando também pode ser usada para descobrir testes, para executar todos os testes de um projeto ou apenas de um subconjunto.

26.5.3 Test Discovery

Adicionado na versão 3.2.

Unittest supports simple test discovery. In order to be compatible with test discovery, all of the test files must be modules or packages importable from the top-level directory of the project (this means that their filenames must be valid identifiers).

O descobrimento de testes é implementado no *`TestLoader.discover()`*, mas também pode ser utilizado a partir da linha de comando. O comando básico para uso é:

```
cd project_directory
python -m unittest discover
```

Nota

Como um atalho, `python -m unittest` é o equivalente a `python -m unittest discover`. Se você deseja passar argumentos para a descoberta de testes, o subcomando `discover` deve ser usado explicitamente.

O sub-comando `discover` (descubra) tem as seguintes opções:

`-v`, `--verbose`
: Saída verbosa

`-s`, `--start-directory` `directory`
: Diretório no qual se inicia o descobrimento (`.` por padrão)

`-p`, `--pattern` `pattern`
: Padrão de texto para se descobrir os arquivos de teste (`test*.py` por padrão)

`-t`, `--top-level-directory` `directory`
: Diretório raiz do projeto (diretório de início por padrão)

As opções *`-s`*, *`-p`* e *`-t`* podem ser passadas como argumentos posicionais nessa ordem. As duas linhas de comando seguintes são equivalentes:

```
python -m unittest discover -s project_directory -p "*_test.py"
python -m unittest discover project_directory "*_test.py"
```

Além de aceitar caminhos, também é possível passar o nome de um pacote, como `myproject.subpackage.test`, como diretório de início. O nome do pacote que for passado será importado e sua localização no sistema de arquivos será utilizada como diretório de início.

Cuidado

O descobridor de testes importa os testes para carregá-los. Uma vez que o descobridor tiver encontrado todos os arquivos de teste a partir do diretório de início especificado, ele transforma os caminhos em nomes de pacotes para conseguir importá-los. Por exemplo, `foo/bar/baz.py` será importado como `foo.bar.baz`.

Se você possuir um pacote instalado globalmente e tentar executar o descobrimento em uma versão diferente deste mesmo pacote, a importação *pode* acontecer do lugar errado. Se isso acontecer, o descobridor de testes irá emitir um alerta e encerrar a execução.

Se você configurar o diretório de início como sendo um nome de pacote, não um caminho para um diretório, o descobridor irá presumir que qualquer local do qual ele importar é o local correto. Neste caso, nenhum alerta será emitido.

Módulos de testes e pacotes podem conter customizações no carregamento de testes utilizando *load_tests protocol*.

Alterado na versão 3.4: O descobrimento de testes possui suporte para *pacotes de espaço de nomes*.

Alterado na versão 3.11: Test discovery dropped the *namespace packages* support. It has been broken since Python 3.7. Start directory and its subdirectories containing tests must be regular package that have `__init__.py` file.

If the start directory is the dotted name of the package, the ancestor packages can be namespace packages.

Alterado na versão 3.14: Test discovery supports *namespace package* as start directory again. To avoid scanning directories unrelated to Python, tests are not searched in subdirectories that do not contain `__init__.py`.

26.5.4 Organizando código teste

O bloco básico de construção dos testes unitários são os *casos de teste* — cenários únicos que devem ser configurados e avaliados em sua correção. No *unittest*, casos de teste são representados por instâncias *unittest.TestCase*. Para criar seus próprios casos de teste, você deve escrever subclasses de *TestCase* ou utilizar *FunctionTestCase*.

O código de teste em uma instância da classe *TestCase* deve ser completamente auto-contido, de maneira que ele possa ser executado isoladamente ou combinado, de forma arbitrária, com quaisquer outros casos de teste.

A mais simples subclasse de *TestCase* irá implementar um método de teste (i.e. um método cujo nome começa com `test`) para executar um teste:

```
import unittest

class DefaultWidgetSizeTestCase(unittest.TestCase):
    def test_default_widget_size(self):
        widget = Widget('The widget')
        self.assertEqual(widget.size(), (50, 50))
```

Note that in order to test something, we use one of the *assert* methods* provided by the *TestCase* base class. If the test fails, an exception will be raised with an explanatory message, and *unittest* will identify the test case as a *failure*. Any other exceptions will be treated as *errors*.

Os testes podem ser muitos, e as configurações podem ser repetitivas. Por sorte, temos como reaproveitar estas configurações implementando um método chamado *setUp()*, o qual será automaticamente chamado pelo framework de teste para cada teste único que executarmos:

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def test_default_widget_size(self):
        self.assertEqual(self.widget.size(), (50,50),
                         'incorrect default size')

    def test_widget_resize(self):
        self.widget.resize(100,150)
        self.assertEqual(self.widget.size(), (100,150),
                         'wrong size after resize')
```

> **Nota**
>
> A ordem de execução dos testes será feita com base na ordenação dos nomes dos métodos de teste respeitando os critérios da ordenação de strings embutida.

Se o método *setUp()* levantar uma exceção durante a sua execução, o framework irá considerar que o teste sofreu um erro e o método de teste não será executado.

De maneira similar, pode-se definir um método *tearDown()* que limpa o ambiente após a execução do método de teste:

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def tearDown(self):
        self.widget.dispose()
```

Se o método *setUp()* for bem sucedido, o método *tearDown()* será executado, independente do resultado do método de teste.

Tal ambiente de trabalho do código de teste é chamado de *definição de contexto de teste*. Uma nova instância TestCase é criada como uma única definição de contexto de teste usada para executar cada método de teste. Portanto, os métodos *setUp()*, *tearDown()*, e `__init__()` serão chamados uma vez por teste.

É recomendado utilizar implementações de TestCase para agrupar testes de acordo com as funcionalidades que eles testam. *unittest* disponibiliza um mecanismo para isso: a *suíte de testes*, representada pela classe *TestSuite* do módulo *unittest*. Em grande parte dos casos, chamar *unittest.main()* é suficiente para coletar todos os casos de teste do módulo e executá-los para você.

Entretanto, caso você queira customizar a construção da sua suíte de testes, é possível fazê-la desta forma:

```
def suite():
    suite = unittest.TestSuite()
    suite.addTest(WidgetTestCase('test_default_widget_size'))
    suite.addTest(WidgetTestCase('test_widget_resize'))
    return suite

if __name__ == '__main__':
    runner = unittest.TextTestRunner()
    runner.run(suite())
```

Você pode colocar as definições dos casos de teste e suíte de testes em um mesmo módulo que contém o código a ser testado (tal como `widget.py`), mas existem várias vantagens ao colocar o código dos testes em um módulo separado, como `test_widget.py`:

- O módulo de teste pode ser executado de maneira isolada a partir da linha de comando.
- O código de teste pode ser mais facilmente separado do código a ser entregue.
- Há uma menor tentação para modificar o código de teste para que ele se adéque ao código testado sem que haja uma boa razão.
- O código de teste deve ser modificado muito menos frequentemente do que o código que ele testa.
- Código testado pode ser reformulado mais facilmente.
- Testes para módulos escritos em C devem ser, obrigatoriamente, colocados em módulos separados, então por que não manter a consistência?
- Se as estratégias de teste mudarem, não há a necessidade de mudar o código-fonte.

26.5.5 Reutilizando códigos de teste antigos

Alguns usuários irão encontrar antigos códigos de teste que eles gostariam de executar com *unittest* sem converter cada função antiga para uma subclasse de *TestCase*.

Por isso, *unittest* contém uma classe *FunctionTestCase*. Esta subclasse de *TestCase* pode ser utilizada para englobar funções de teste existentes. Funções de definição de estado inicial e final (set-up e tear-down) também podem ser fornecidas.

Dadas as seguintes funções de teste:

```
def testSomething():
    something = makeSomething()
    assert something.name is not None
    # ...
```

é possível criar uma instância de caso de teste, com métodos opcionais de configuração inicial e final (set-up e tear-down), como mostrado a seguir:

```
testcase = unittest.FunctionTestCase(testSomething,
                                     setUp=makeSomethingDB,
                                     tearDown=deleteSomethingDB)
```

Nota

Apesar da classe *FunctionTestCase* poder ser utilizada para converter rapidamente um teste existente em um teste do módulo *unittest*, esta abordagem não é recomendada. Investir tempo para configurar corretamente uma subclasse de *TestCase* irá tornar futuras refatorações infinitamente mais fáceis.

Em alguns casos, os testes existentes podem ter sido escritos utilizando o módulo *doctest*. Se for o caso, *doctest* contém a classe `DocTestSuite` que pode construir, automaticamente, instâncias *unittest.TestSuite* a partir dos testes baseados em *doctest*.

26.5.6 Ignorando testes e falhas esperadas

Adicionado na versão 3.1.

Unittest suporta ignorar métodos de teste individuais e, até mesmo, classes de teste inteiras. Além disso, há suporte para a marcação de um teste como uma "falha esperada", um teste que está incorreto e irá falhar, mas não deve ser considerado como uma falha no *TestResult*.

Ignorar um teste é simplesmente uma questão de utilizar o *decorador* *skip()* ou uma de suas variantes condicionais, chamando *TestCase.skipTest()* em um *setUp()* ou método de teste, ou levantando *SkipTest* diretamente.

Ignorar se parece basicamente com:

```
class MyTestCase(unittest.TestCase):

    @unittest.skip("demonstrating skipping")
    def test_nothing(self):
        self.fail("shouldn't happen")

    @unittest.skipIf(mylib.__version__ < (1, 3),
                     "not supported in this library version")
    def test_format(self):
        # Tests that work for only a certain version of the library.
        pass

    @unittest.skipUnless(sys.platform.startswith("win"), "requires Windows")
```

(continua na próxima página)

(continuação da página anterior)

```
    def test_windows_support(self):
        # windows specific testing code
        pass

    def test_maybe_skipped(self):
        if not external_resource_available():
            self.skipTest("external resource not available")
        # test code that depends on the external resource
        pass
```

Esta é a saída da execução do exemplo acima em modo verboso:

```
test_format (__main__.MyTestCase.test_format) ... skipped 'not supported in this␣
↪library version'
test_nothing (__main__.MyTestCase.test_nothing) ... skipped 'demonstrating skipping
↪'
test_maybe_skipped (__main__.MyTestCase.test_maybe_skipped) ... skipped 'external␣
↪resource not available'
test_windows_support (__main__.MyTestCase.test_windows_support) ... skipped
↪'requires Windows'

----------------------------------------------------------------------
Ran 4 tests in 0.005s

OK (skipped=4)
```

Classes podem ser puladas assim como métodos:

```
@unittest.skip("showing class skipping")
class MySkippedTestCase(unittest.TestCase):
    def test_not_run(self):
        pass
```

TestCase.setUp() também pode ignorar o teste. Isso é útil quando um recurso que precisa ser configurado não está disponível.

Falhas esperadas usam o decorador *expectedFailure()*:

```
class ExpectedFailureTestCase(unittest.TestCase):
    @unittest.expectedFailure
    def test_fail(self):
        self.assertEqual(1, 0, "broken")
```

É simples ignorar com decoradores customizados, basta fazer um decorador que chama *skip()* no teste quando ele deve ser ignorado. Este decorador ignora o teste a não ser que o objeto fornecido tenha um determinado atributo:

```
def skipUnlessHasattr(obj, attr):
    if hasattr(obj, attr):
        return lambda func: func
    return unittest.skip("{!r} doesn't have {!r}".format(obj, attr))
```

Os decoradores e exceção seguintes ignoram testes e falhas esperadas:

`@unittest.`**`skip`**`(`*reason*`)`

Ignora incondicionalmente o teste decorado. *reason* deve descrever a razão pela qual o teste está sendo ignorado.

`@unittest.`**`skipIf`**`(`*condition*, *reason*`)`

Ignora o teste decorado se *condition* for verdadeiro.

@unittest.**skipUnless**(*condition*, *reason*)

Ignora o teste decorado a não ser que *condition* seja verdadeiro.

@unittest.**expectedFailure**

Marca o teste como uma falha ou erro esperado. Se o teste falhar ou ocorrerem erros na função de teste mesmo (ao invés de em um dos métodos *test fixture*), então ele será considerado como executado com sucesso. Se o teste passar, ele será considerado como uma falha.

exception unittest.**SkipTest**(*reason*)

Esta exceção é levantada para ignorar um teste.

Normalmente, você pode utilizar `TestCase.skipTest()` ou um dos decoradores para ignorar sem ter de levantar esta exceção diretamente.

Testes ignorados não terão seus métodos `setUp()` ou `tearDown()` executados. Classes ignoradas não terão seus métodos `setUpClass()` ou `tearDownClass()` executados. Módulos ignorados não terão seus métodos `setUpModule()` ou `tearDownModule()` executados.

26.5.7 Distinguindo iterações de teste utilizando subtestes

Adicionado na versão 3.4.

Quando existem pequenas diferenças entre os seus testes, por exemplo alguns parâmetros, unittest permite que você distinga-os dentro do corpo de um método de teste utilizando o gerenciador de contexto `subTest()`.

Por exemplo, o teste seguinte:

```
class NumbersTest(unittest.TestCase):

    def test_even(self):
        """
        Test that numbers between 0 and 5 are all even.
        """
        for i in range(0, 6):
            with self.subTest(i=i):
                self.assertEqual(i % 2, 0)
```

irá produzir a seguinte saída:

```
======================================================================
FAIL: test_even (__main__.NumbersTest.test_even) (i=1)
Test that numbers between 0 and 5 are all even.
----------------------------------------------------------------------
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0

======================================================================
FAIL: test_even (__main__.NumbersTest.test_even) (i=3)
Test that numbers between 0 and 5 are all even.
----------------------------------------------------------------------
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0

======================================================================
```

(continua na próxima página)

(continuação da página anterior)

```
FAIL: test_even (__main__.NumbersTest.test_even) (i=5)
Test that numbers between 0 and 5 are all even.
----------------------------------------------------------------------
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0
```

Sem usar um subteste, a execução para depois da primeira falha e o erro será menos fácil de ser diagnosticado porque o valor de `i` não será mostrado:

```
======================================================================
FAIL: test_even (__main__.NumbersTest.test_even)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "subtests.py", line 32, in test_even
    self.assertEqual(i % 2, 0)
AssertionError: 1 != 0
```

26.5.8 Classes e funções

Esta seção descreve de maneira aprofundada a API do módulo *unittest*.

Casos de teste

class unittest.**TestCase**(*methodName='runTest'*)

Instâncias da classe *TestCase* representam unidades lógicas de teste no universo do *unittest*. Esta classe deve ser utilizada como classe base, com testes específicos sendo implementados por subclasses concretas. Esta classe implementa a interface requerida pelo executor de testes, para permitir o controle dos testes, e métodos que o código de teste pode utilizar para checar e reportar diversos tipos de falhas.

Cada instância da classe *TestCase* irá executar um único método base: o método chamado *methodName*. Em muitos casos de uso da classe *TestCase*, você não precisará modificar o *methodName* ou reimplementar o método `runTest()` padrão.

Alterado na versão 3.2: *TestCase* pode ser instanciada com sucesso sem fornecer um *methodName*. Isso torna mais fácil experimentar com *TestCase* a partir de um interpretador interativo.

Instâncias *TestCase* possuem três grupos de métodos: um grupo utilizado para executar o teste, outro utilizado pela implementação do teste para checar as condições e reportar falhas, e alguns métodos de investigação para permitir coletar dados sobre o teste em si.

Os métodos do primeiro grupo (que executam os testes) são:

setUp()

Método chamado para preparar a definição de contexto de teste. Chamado imediatamente após chamar o método de teste; exceto pelo *AssertionError* ou *SkipTest*, qualquer exceção levantada por este método será considerada um erro além de uma simples falha de teste. A implementação padrão não faz nada.

tearDown()

Método chamado imediatamente após o método de teste ter sido chamado e o resultado registrado. Este método é chamado mesmo se o método de teste tiver levantado uma exceção, então a implementação em subclasses deve ser feita com cautela ao verificar o estado interno. Qualquer exceção além de *AssertionError* ou *SkipTest* levantada por este método será considerado um erro adicional, e não uma simples falha de teste (que incrementaria o número total de erros no relatório final de testes). Este método será executado apenas se o método *setUp()* for bem sucedido, independente do resultado do método de teste. A implementação padrão não faz nada.

setUpClass()

Um método de classe chamado antes da execução dos testes de uma classe específica. O método `setUpClass` é chamado com a classe sendo o único argumento e deve ser decorada como um *classmethod()*:

```
@classmethod
def setUpClass(cls):
    ...
```

Veja *Classes e Módulos de Definição de Contexto* para mais detalhes.

Adicionado na versão 3.2.

tearDownClass()

Um método de classe chamado depois da execução dos testes de uma classe específica. O método `tearDownClass` é chamado com a classe sendo o único argumento e deve ser decorada como um *classmethod()*:

```
@classmethod
def tearDownClass(cls):
    ...
```

Veja *Classes e Módulos de Definição de Contexto* para mais detalhes.

Adicionado na versão 3.2.

run(*result=None*)

Executa o teste, registrando as informações no objeto resultado *TestResult*, passado como *result*. Se *result* for omitido, ou definido como `None`, um objeto resultado temporário é criado (chamando o método *defaultTestResult()*) e utilizado. O objeto resultado é retornado para quem chamou o método *run()*.

O mesmo efeito pode ser obtido ao chamar uma instância da classe *TestCase*.

Alterado na versão 3.3: Versões anteriores de `run` não retornavam o resultado. Nem chamando por uma instância.

skipTest(*reason*)

Ao se executar durante um método de teste ou *setUp()*, pula o teste em execução. Veja *Ignorando testes e falhas esperadas* para mais informações.

Adicionado na versão 3.1.

subTest(*msg=None*, ***params*)

Retorna um gerenciador de contexto que executa, como subteste, o bloco de código englobado. *msg* e *params* são valores opcionais e arbitrários que são mostrados sempre quando um subteste falha, permitindo identificá-los claramente.

Um caso de teste pode conter inúmeras declarações de subteste e elas podem ser aninhadas de forma arbitrária.

Veja *Distinguindo iterações de teste utilizando subtestes* para mais informações.

Adicionado na versão 3.4.

debug()

Executa o teste sem coletar o resultado. Permite propagar exceções levantadas pelo teste e pode ser utilizado para oferecer suporte aos testes sob um depurador.

A classe *TestCase* oferece diversos métodos de asserção para checar e reportar falhas. A tabela a seguir lista os métodos mais utilizados (veja as tabelas abaixo para mais métodos de asserção).

Método	Avalia se	Novo em
assertEqual(a, b)	`a == b`	
assertNotEqual(a, b)	`a != b`	
assertTrue(x)	`bool(x) is True`	
assertFalse(x)	`bool(x) is False`	
assertIs(a, b)	`a is b`	3.1
assertIsNot(a, b)	`a is not b`	3.1
assertIsNone(x)	`x is None`	3.1
assertIsNotNone(x)	`x is not None`	3.1
assertIn(a, b)	`a in b`	3.1
assertNotIn(a, b)	`a not in b`	3.1
assertIsInstance(a, b)	`isinstance(a, b)`	3.2
assertNotIsInstance(a, b)	`not isinstance(a, b)`	3.2
assertIsSubclass(a, b)	`issubclass(a, b)`	3.14
assertNotIsSubclass(a, b)	`not issubclass(a, b)`	3.14

Todos os métodos de asserção aceitam um argumento *msg* que, se especificado, é utilizado como a mensagem de erro em caso de falha (veja também *longMessage*). Note que o argumento nomeado *msg* pode ser passado para *assertRaises()*, *assertRaisesRegex()*, *assertWarns()* e *assertWarnsRegex()* apenas quando eles são utilizados como gerenciador de contexto.

assertEqual(*first*, *second*, *msg=None*)

Testa se *first* e *second* são iguais. Se o resultado da comparação não indicar igualdade, o teste irá falhar.

Além disso, se *first* e *second* são exatamente do mesmo tipo e são uma lista, tupla, dict, set, frozenset, str, ou qualquer outro tipo que é registrado na subclasse com *addTypeEqualityFunc()*, a função de igualdade específica do tipo é será chamada para gerar uma mensagem de erro mais útil (veja também *lista de métodos específicos por tipo*)

Alterado na versão 3.1: Adição da chamada automática da função específica por tipo.

Alterado na versão 3.2: método *assertMultiLineEqual()* adicionado como função de igualdade padrão para o tipo string.

assertNotEqual(*first*, *second*, *msg=None*)

Testa se *first* e *second* não são iguais. Se o resultado da comparação indicar igualdade, o teste irá falhar.

assertTrue(*expr*, *msg=None*)

assertFalse(*expr*, *msg=None*)

Testa se a expressão *expr* é verdadeira (ou falsa).

Note que isso é equivalente a `bool(expr) is True` e não a `expr is True` (use `assertIs(expr, True)` neste último caso). Este método também deve ser evitado quando outros métodos mais específicos estão disponíveis (e.g. `assertEqual(a, b)` no lugar de `assertTrue(a == b)`), já que estes podem ter uma melhor mensagem de erro em caso de falha.

assertIs(*first*, *second*, *msg=None*)

assertIsNot(*first*, *second*, *msg=None*)

Testa que *first* e *second* são (ou não são) o mesmo objeto.

Adicionado na versão 3.1.

assertIsNone(*expr*, *msg=None*)

assertIsNotNone(*expr*, *msg=None*)

Testa se *expr* é (ou não é) `None`.

Adicionado na versão 3.1.

assertIn(*member*, *container*, *msg=None*)

assertNotIn(*member*, *container*, *msg=None*)

Testa se *member* está (ou não está) em *container*.

Adicionado na versão 3.1.

assertIsInstance(*obj*, *cls*, *msg=None*)

assertNotIsInstance(*obj*, *cls*, *msg=None*)

Testa se *obj* é (ou não é) uma instância de *cls* (que pode ser uma classe ou uma tupla de classes, como suportado pela função `isinstance()`). Para checar pelo tipo exato, use `assertIs(type(obj), cls)`.

Adicionado na versão 3.2.

assertIsSubclass(*cls*, *superclass*, *msg=None*)

assertNotIsSubclass(*cls*, *superclass*, *msg=None*)

Test that *cls* is (or is not) a subclass of *superclass* (which can be a class or a tuple of classes, as supported by `issubclass()`). To check for the exact type, use `assertIs(cls, superclass)`.

Adicionado na versão 3.14.

Também é possível checar a produção de exceções, avisos e mensagens de log usando os seguintes métodos:

Método	Avalia se	Novo em
`assertRaises(exc, fun, *args, **kwds)`	`fun(*args, **kwds)` levanta *exc*	
`assertRaisesRegex(exc, r, fun, *args, **kwds)`	`fun(*args, **kwds)` levanta *exc* e a mensagem casa com a expressão regular *r*	3.1
`assertWarns(warn, fun, *args, **kwds)`	`fun(*args, **kwds)` levanta *warn*	3.2
`assertWarnsRegex(warn, r, fun, *args, **kwds)`	`fun(*args, **kwds)` levanta *warn* e a mensagem casa com a expressão regular *r*	3.2
`assertLogs(logger, level)`	O bloco `with` registra logs no *logger* com pelo menos um nível *level*	Módulos para processamento de XML
`assertNoLogs(logger, level)`	**O bloco `with` não loga em** *logger* com *level* mínimo	3.10

assertRaises(*exception*, *callable*, **args*, ***kwds*)

assertRaises(*exception*, *, *msg=None*)

Testa se uma exceção é levantada quando *callable* é chamado com quaisquer argumentos nomeados ou posicionais que também são passados para `assertRaises()`. O teste passa se *exception* for levantada e falha se outra exceção ou nenhuma exceção for levantada.

Se somente os argumentos *exception* e, possivelmente, *msg* forem passados, retorna um gerenciador de contexto para que o código sob teste possa ser escrito de forma embutida ao invés de ser definido como uma função:

```
with self.assertRaises(SomeException):
    do_something()
```

Quando utilizado como gerenciador de contexto, `assertRaises()` aceita o argumento nomeado adicional *msg*.

O gerenciador de contexto irá armazenar o objeto exceção capturado no seu atributo `exception`. Isso pode ser útil se a intenção for realizar testes adicionais na exceção levantada:

```
with self.assertRaises(SomeException) as cm:
    do_something()

the_exception = cm.exception
self.assertEqual(the_exception.error_code, 3)
```

Alterado na versão 3.1: Adicionada a possibilidade de se utilizar *assertRaises()* como gerenciador de contexto.

Alterado na versão 3.2: Adicionado o atributo `exception`.

Alterado na versão 3.3: Adicionado o argumento nomeado *msg* quando utilizado como gerenciador de contexto.

assertRaisesRegex(*exception*, *regex*, *callable*, **args*, ***kwds*)

assertRaisesRegex(*exception*, *regex*, ***, *msg=None*)

Similar ao *assertRaises()* mas também testa se *regex* casa com a representação em string da exceção levantada. *regex* pode ser um objeto expressão regular ou uma string contendo uma expressão regular compatível com o uso pela função *re.search()*. Exemplos:

```
self.assertRaisesRegex(ValueError, "invalid literal for.*XYZ'$",
                       int, 'XYZ')
```

ou:

```
with self.assertRaisesRegex(ValueError, 'literal'):
   int('XYZ')
```

Adicionado na versão 3.1: Adicionado com o nome `assertRaisesRegexp`.

Alterado na versão 3.2: Renomeado para *assertRaisesRegex()*.

Alterado na versão 3.3: Adicionado o argumento nomeado *msg* quando utilizado como gerenciador de contexto.

assertWarns(*warning*, *callable*, **args*, ***kwds*)

assertWarns(*warning*, ***, *msg=None*)

Testa se o aviso *warning* é acionado quando *callable* é chamado com quaisquer argumentos nomeados ou posicionais que também são passados para *assertWarns()*. O teste passa se *warning* for acionado e falha se não for. Qualquer exceção é considerada como falha. Para capturar qualquer aviso presente em um grupo de avisos, uma tupla contendo as classes de aviso podem ser passadas como *warning*.

Se apenas os argumentos *warning* e, possivelmente, *msg* forem passados, retorna um gerenciador de contexto para que o código sob teste possa ser escrito de forma embutida ao invés de ser definido como uma função:

```
with self.assertWarns(SomeWarning):
    do_something()
```

Quando usado como gerenciador de contexto, *assertWarns()* aceita o argumento nomeado adicional *msg*.

O gerenciador de contexto irá armazenar o objeto de aviso capturado no atributo `warning` e a linha de código fonte que acionou o aviso nos atributos `filename` e `lineno`. Isso pode ser útil se a intenção for executar verificações adicionais no aviso capturado:

```
with self.assertWarns(SomeWarning) as cm:
    do_something()

self.assertIn('myfile.py', cm.filename)
self.assertEqual(320, cm.lineno)
```

Este método funciona independente dos filtros de aviso configurados no momento em que é chamado.

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionado o argumento nomeado *msg* quando utilizado como gerenciador de contexto.

assertWarnsRegex(*warning*, *regex*, *callable*, **args*, ***kwds*)

assertWarnsRegex(*warning*, *regex*, ***, *msg=None*)

Similar ao `assertWarns()` mas também testa se *regex* casa com a mensagem do aviso acionado. *regex* pode ser um objeto expressão regular ou uma string contendo uma expressão regular compatível para uso pela função `re.search()`. Exemplo:

```
self.assertWarnsRegex(DeprecationWarning,
                      r'legacy_function\(\) is deprecated',
                      legacy_function, 'XYZ')
```

ou:

```
with self.assertWarnsRegex(RuntimeWarning, 'unsafe frobnicating'):
    frobnicate('/etc/passwd')
```

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionado o argumento nomeado *msg* quando utilizado como gerenciador de contexto.

assertLogs(*logger=None*, *level=None*)

Um gerenciador de contexto para testar se pelo menos uma mensagem é inserida em *logger* ou em um de seus filhos, com pelo menos um nível *level*.

Se fornecido, *logger* deve ser um objeto `logging.Logger` ou uma `str` fornecendo o nome de um logger. O padrão é o logger root, o qual irá capturar todas as mensagens que não foram bloqueadas por um logger descendente não-propagante.

If given, *level* should be either a numeric logging level or its string equivalent (for example either `"ERROR"` or `logging.ERROR`). The default is `logging.INFO`.

O teste passa se pelo menos uma das mensagens emitidas dentro do bloco `with` casa com as condições dadas por *logger* e *level*, falhando caso contrário.

O objeto retornado pelo gerenciador de contexto é um registro auxiliar que mantém os rastros das mensagens de log capturadas de acordo com os critérios dados. Ele possui dois atributos:

records

Uma lista de objetos de log da classe `logging.LogRecord` que foram compatíveis com os critérios dados.

output

Uma lista de objetos da classe `str` com a saída formatada das mensagens de log compatíveis com os critérios dados.

Exemplo:

```
with self.assertLogs('foo', level='INFO') as cm:
    logging.getLogger('foo').info('first message')
    logging.getLogger('foo.bar').error('second message')
self.assertEqual(cm.output, ['INFO:foo:first message',
                             'ERROR:foo.bar:second message'])
```

Adicionado na versão 3.4.

assertNoLogs(*logger=None*, *level=None*)

Um gerenciador de contexto para testar que nenhuma mensagem é logada no *logger* ou um dos seus filhos, com pelo menos o *level* fornecido.

Se passado, *logger* deve ser um objeto da classe `logging.Logger` ou um objeto da classe `str` com o nome do registrador de logs. O padrão é o registrador de log raiz, que irá capturar todas as mensagens.

If given, *level* should be either a numeric logging level or its string equivalent (for example either `"ERROR"` or `logging.ERROR`). The default is `logging.INFO`.

Ao contrário de `assertLogs()`, nada será retornado pelo gerenciador de contexto.

Adicionado na versão 3.10.

Existem também outros métodos usados para executar verificações mais específicas, como:

Método	Avalia se	Novo em
`assertAlmostEqual(a, b)`	`round(a-b, 7) == 0`	
`assertNotAlmostEqual b)`	`round(a-b, 7) != 0`	
`assertGreater(a, b)`	`a > b`	3.1
`assertGreaterEqual(a, b)`	`a >= b`	3.1
`assertLess(a, b)`	`a < b`	3.1
`assertLessEqual(a, b)`	`a <= b`	3.1
`assertRegex(s, r)`	`r.search(s)`	3.1
`assertNotRegex(s, r)`	`not r.search(s)`	3.2
`assertCountEqual(a, b)`	*a* e *b* possuem os mesmos elementos na mesma quantidade, independente da sua ordem.	3.2
`assertStartsWith(a, b)`	`a.startswith(b)`	3.14
`assertNotStartsWith(a b)`	`not a.startswith(b)`	3.14
`assertEndsWith(a, b)`	`a.endswith(b)`	3.14
`assertNotEndsWith(a, b)`	`not a.endswith(b)`	3.14
`assertHasAttr(a, b)`	`hastattr(a, b)`	3.14
`assertNotHasAttr(a, b)`	`not hastattr(a, b)`	3.14

assertAlmostEqual(*first*, *second*, *places=7*, *msg=None*, *delta=None*)

assertNotAlmostEqual(*first*, *second*, *places=7*, *msg=None*, *delta=None*)

Testa se *first* e *second* são (ou não são) aproximadamente iguais considerando a diferença entre eles, arredondando para o número de casas decimais dado (7 por padrão), e comparando a zero. Note que estes métodos arredondam os valores considerando o número de casas decimais (i.e. como a função `round()`) e não o número de algarismos significativos.

Se *delta* é fornecido no lugar de *places*, a diferença entre *first* e *second* deve ser menor ou igual a (ou maior que) *delta*.

Passar *delta* e *places* ao mesmo tempo levanta a exceção `TypeError`.

Alterado na versão 3.2: `assertAlmostEqual()` considera, automaticamente, objetos quase iguais que possuem a comparação de igualdade dada como verdadeira. `assertNotAlmostEqual()` falha auto-

maticamente se os objetos possuem a comparação de igualdade dada como verdadeira. Adicionado o argumento nomeado *delta*.

assertGreater(*first*, *second*, *msg=None*)

assertGreaterEqual(*first*, *second*, *msg=None*)

assertLess(*first*, *second*, *msg=None*)

assertLessEqual(*first*, *second*, *msg=None*)

Testa que *first* é respectivamente >, >=, < ou <= que *second*, dependendo do nome do método. Se não for, o teste irá falhar:

```
>>> self.assertGreaterEqual(3, 4)
AssertionError: "3" unexpectedly not greater than or equal to "4"
```

Adicionado na versão 3.1.

assertRegex(*text*, *regex*, *msg=None*)

assertNotRegex(*text*, *regex*, *msg=None*)

Testa que uma procura por *regex* corresponde (ou não corresponde) a *text*. Em caso de falha, a mensagem de erro irá incluir o padrão e o *text* (ou o padrão e a parte do *text* que inesperadamente correspondeu). *regex* pode ser um objeto de expressão regular ou uma string contendo uma expressão regular adequada para uso por *re.search()*.

Adicionado na versão 3.1: Adicionada abaixo do nome `assertRegexpMatches`.

Alterado na versão 3.2: O método `assertRegexpMatches()` foi renomeado para *assertRegex()*.

Adicionado na versão 3.2: *assertNotRegex()*.

assertCountEqual(*first*, *second*, *msg=None*)

Test that sequence *first* contains the same elements as *second*, regardless of their order. When they don't, an error message listing the differences between the sequences will be generated.

Elementos duplicados *não* são ignorados ao comparar *first* e *second*. É verificado se cada elemento tem a mesma contagem em ambas sequências. Equivalente a: `assertEqual(Counter(list(first)), Counter(list(second)))`, mas também funciona com sequências de objetos não hasheáveis.

Adicionado na versão 3.2.

assertStartsWith(*s*, *prefix*, *msg=None*)

assertNotStartsWith(*s*, *prefix*, *msg=None*)

Test that the Unicode or byte string *s* starts (or does not start) with a *prefix*. *prefix* can also be a tuple of strings to try.

Adicionado na versão 3.14.

assertEndsWith(*s*, *suffix*, *msg=None*)

assertNotEndsWith(*s*, *suffix*, *msg=None*)

Test that the Unicode or byte string *s* ends (or does not end) with a *suffix*. *suffix* can also be a tuple of strings to try.

Adicionado na versão 3.14.

assertHasAttr(*obj*, *name*, *msg=None*)

assertNotHasAttr(*obj*, *name*, *msg=None*)

Test that the object *obj* has (or has not) an attribute *name*.

Adicionado na versão 3.14.

The *assertEqual()* method dispatches the equality check for objects of the same type to different type-specific methods. These methods are already implemented for most of the built-in types, but it's also possible to register new methods using *addTypeEqualityFunc()*:

addTypeEqualityFunc(*typeobj*, *function*)

Registers a type-specific method called by `assertEqual()` to check if two objects of exactly the same *typeobj* (not subclasses) compare equal. *function* must take two positional arguments and a third msg=None keyword argument just as `assertEqual()` does. It must raise `self.failureException(msg)` when inequality between the first two parameters is detected – possibly providing useful information and explaining the inequalities in details in the error message.

Adicionado na versão 3.1.

The list of type-specific methods automatically used by `assertEqual()` are summarized in the following table. Note that it's usually not necessary to invoke these methods directly.

Método	Usado para comparar	Novo em
`assertMultiLineEqual(a, b)`	strings	3.1
`assertSequenceEqual(a, b)`	sequências	3.1
`assertListEqual(a, b)`	listas	3.1
`assertTupleEqual(a, b)`	tuplas	3.1
`assertSetEqual(a, b)`	sets ou frozensets	3.1
`assertDictEqual(a, b)`	dicionários	3.1

assertMultiLineEqual(*first*, *second*, *msg=None*)

Test that the multiline string *first* is equal to the string *second*. When not equal a diff of the two strings highlighting the differences will be included in the error message. This method is used by default when comparing strings with `assertEqual()`.

Adicionado na versão 3.1.

assertSequenceEqual(*first*, *second*, *msg=None*, *seq_type=None*)

Tests that two sequences are equal. If a *seq_type* is supplied, both *first* and *second* must be instances of *seq_type* or a failure will be raised. If the sequences are different an error message is constructed that shows the difference between the two.

This method is not called directly by `assertEqual()`, but it's used to implement `assertListEqual()` and `assertTupleEqual()`.

Adicionado na versão 3.1.

assertListEqual(*first*, *second*, *msg=None*)

assertTupleEqual(*first*, *second*, *msg=None*)

Tests that two lists or tuples are equal. If not, an error message is constructed that shows only the differences between the two. An error is also raised if either of the parameters are of the wrong type. These methods are used by default when comparing lists or tuples with `assertEqual()`.

Adicionado na versão 3.1.

assertSetEqual(*first*, *second*, *msg=None*)

Tests that two sets are equal. If not, an error message is constructed that lists the differences between the sets. This method is used by default when comparing sets or frozensets with `assertEqual()`.

Fails if either of *first* or *second* does not have a `set.difference()` method.

Adicionado na versão 3.1.

assertDictEqual(*first*, *second*, *msg=None*)

Test that two dictionaries are equal. If not, an error message is constructed that shows the differences in the dictionaries. This method will be used by default to compare dictionaries in calls to `assertEqual()`.

Adicionado na versão 3.1.

Finally the `TestCase` provides the following methods and attributes:

fail(*msg=None*)

Signals a test failure unconditionally, with *msg* or `None` for the error message.

failureException

This class attribute gives the exception raised by the test method. If a test framework needs to use a specialized exception, possibly to carry additional information, it must subclass this exception in order to "play fair" with the framework. The initial value of this attribute is *AssertionError*.

longMessage

This class attribute determines what happens when a custom failure message is passed as the msg argument to an assertXYY call that fails. `True` is the default value. In this case, the custom message is appended to the end of the standard failure message. When set to `False`, the custom message replaces the standard message.

The class setting can be overridden in individual test methods by assigning an instance attribute, self.longMessage, to `True` or `False` before calling the assert methods.

The class setting gets reset before each test call.

Adicionado na versão 3.1.

maxDiff

This attribute controls the maximum length of diffs output by assert methods that report diffs on failure. It defaults to 80*8 characters. Assert methods affected by this attribute are *assertSequenceEqual()* (including all the sequence comparison methods that delegate to it), *assertDictEqual()* and *assertMultiLineEqual()*.

Setting `maxDiff` to `None` means that there is no maximum length of diffs.

Adicionado na versão 3.2.

Frameworks de teste podem usar os seguintes métodos para coletar informações sobre o teste:

countTestCases()

Retorna o número de testes representados por esse objeto de teste. Para instâncias *TestCase* será sempre `1`.

defaultTestResult()

Return an instance of the test result class that should be used for this test case class (if no other result instance is provided to the *run()* method).

For *TestCase* instances, this will always be an instance of *TestResult*; subclasses of *TestCase* should override this as necessary.

id()

Retorna uma string identificando o caso de teste específico. Geralmente é o nome completo do método do teste, incluindo o módulo e o nome da classe.

shortDescription()

Retorna uma descrição do teste ou `None` se nenhuma descrição for fornecida. A implementação padrão desse método retorna a primeira linha da docstring do método do teste, se disponível, ou `None`.

Alterado na versão 3.1: In 3.1 this was changed to add the test name to the short description even in the presence of a docstring. This caused compatibility issues with unittest extensions and adding the test name was moved to the *TextTestResult* in Python 3.2.

addCleanup(*function*, /, **args*, ***kwargs*)

Add a function to be called after *tearDown()* to cleanup resources used during the test. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into *addCleanup()* when they are added.

If *setUp()* fails, meaning that *tearDown()* is not called, then any cleanup functions added will still be called.

Adicionado na versão 3.1.

enterContext(*cm*)

Enter the supplied *context manager*. If successful, also add its `__exit__()` method as a cleanup function by `addCleanup()` and return the result of the `__enter__()` method.

Adicionado na versão 3.11.

doCleanups()

This method is called unconditionally after `tearDown()`, or after `setUp()` if `setUp()` raises an exception.

It is responsible for calling all the cleanup functions added by `addCleanup()`. If you need cleanup functions to be called *prior* to `tearDown()` then you can call `doCleanups()` yourself.

`doCleanups()` pops methods off the stack of cleanup functions one at a time, so it can be called at any time.

Adicionado na versão 3.1.

classmethod addClassCleanup(*function*, */*, **args*, ***kwargs*)

Add a function to be called after `tearDownClass()` to cleanup resources used during the test class. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into `addClassCleanup()` when they are added.

If `setUpClass()` fails, meaning that `tearDownClass()` is not called, then any cleanup functions added will still be called.

Adicionado na versão 3.8.

classmethod enterClassContext(*cm*)

Enter the supplied *context manager*. If successful, also add its `__exit__()` method as a cleanup function by `addClassCleanup()` and return the result of the `__enter__()` method.

Adicionado na versão 3.11.

classmethod doClassCleanups()

This method is called unconditionally after `tearDownClass()`, or after `setUpClass()` if `setUpClass()` raises an exception.

It is responsible for calling all the cleanup functions added by `addClassCleanup()`. If you need cleanup functions to be called *prior* to `tearDownClass()` then you can call `doClassCleanups()` yourself.

`doClassCleanups()` pops methods off the stack of cleanup functions one at a time, so it can be called at any time.

Adicionado na versão 3.8.

class unittest.**IsolatedAsyncioTestCase**(*methodName='runTest'*)

This class provides an API similar to `TestCase` and also accepts coroutines as test functions.

Adicionado na versão 3.8.

loop_factory

The *loop_factory* passed to `asyncio.Runner`. Override in subclasses with `asyncio.EventLoop` to avoid using the asyncio policy system.

Adicionado na versão 3.13.

async asyncSetUp()

Method called to prepare the test fixture. This is called after `setUp()`. This is called immediately before calling the test method; other than `AssertionError` or `SkipTest`, any exception raised by this method will be considered an error rather than a test failure. The default implementation does nothing.

async asyncTearDown()

Method called immediately after the test method has been called and the result recorded. This is called before `tearDown()`. This is called even if the test method raised an exception, so the implementation in subclasses may need to be particularly careful about checking internal state. Any exception, other than

AssertionError or *SkipTest*, raised by this method will be considered an additional error rather than a test failure (thus increasing the total number of reported errors). This method will only be called if the *asyncSetUp()* succeeds, regardless of the outcome of the test method. The default implementation does nothing.

addAsyncCleanup(*function*, */*, **args*, ***kwargs*)

This method accepts a coroutine that can be used as a cleanup function.

async enterAsyncContext(*cm*)

Enter the supplied *asynchronous context manager*. If successful, also add its `__aexit__()` method as a cleanup function by *addAsyncCleanup()* and return the result of the `__aenter__()` method.

Adicionado na versão 3.11.

run(*result=None*)

Sets up a new event loop to run the test, collecting the result into the *TestResult* object passed as *result*. If *result* is omitted or `None`, a temporary result object is created (by calling the `defaultTestResult()` method) and used. The result object is returned to *run()*'s caller. At the end of the test all the tasks in the event loop are cancelled.

An example illustrating the order:

```
from unittest import IsolatedAsyncioTestCase

events = []


class Test(IsolatedAsyncioTestCase):


    def setUp(self):
        events.append("setUp")

    async def asyncSetUp(self):
        self._async_connection = await AsyncConnection()
        events.append("asyncSetUp")

    async def test_response(self):
        events.append("test_response")
        response = await self._async_connection.get("https://example.com")
        self.assertEqual(response.status_code, 200)
        self.addAsyncCleanup(self.on_cleanup)

    def tearDown(self):
        events.append("tearDown")

    async def asyncTearDown(self):
        await self._async_connection.close()
        events.append("asyncTearDown")

    async def on_cleanup(self):
        events.append("cleanup")

if __name__ == "__main__":
    unittest.main()
```

After running the test, `events` would contain `["setUp", "asyncSetUp", "test_response", "asyncTearDown", "tearDown", "cleanup"]`.

class unittest.**FunctionTestCase**(*testFunc*, *setUp=None*, *tearDown=None*, *description=None*)

This class implements the portion of the *TestCase* interface which allows the test runner to drive the test, but

does not provide the methods which test code can use to check and report errors. This is used to create test cases using legacy test code, allowing it to be integrated into a *unittest*-based test framework.

Grouping tests

class unittest.**TestSuite**(*tests=()*)

This class represents an aggregation of individual test cases and test suites. The class presents the interface needed by the test runner to allow it to be run as any other test case. Running a *TestSuite* instance is the same as iterating over the suite, running each test individually.

If *tests* is given, it must be an iterable of individual test cases or other test suites that will be used to build the suite initially. Additional methods are provided to add test cases and suites to the collection later on.

TestSuite objects behave much like *TestCase* objects, except they do not actually implement a test. Instead, they are used to aggregate tests into groups of tests that should be run together. Some additional methods are available to add tests to *TestSuite* instances:

addTest(*test*)

Add a *TestCase* or *TestSuite* to the suite.

addTests(*tests*)

Add all the tests from an iterable of *TestCase* and *TestSuite* instances to this test suite.

This is equivalent to iterating over *tests*, calling *addTest()* for each element.

TestSuite shares the following methods with *TestCase*:

run(*result*)

Run the tests associated with this suite, collecting the result into the test result object passed as *result*. Note that unlike *TestCase.run()*, *TestSuite.run()* requires the result object to be passed in.

debug()

Run the tests associated with this suite without collecting the result. This allows exceptions raised by the test to be propagated to the caller and can be used to support running tests under a debugger.

countTestCases()

Return the number of tests represented by this test object, including all individual tests and sub-suites.

__iter__()

Tests grouped by a *TestSuite* are always accessed by iteration. Subclasses can lazily provide tests by overriding `__iter__()`. Note that this method may be called several times on a single suite (for example when counting tests or comparing for equality) so the tests returned by repeated iterations before *TestSuite.run()* must be the same for each call iteration. After *TestSuite.run()*, callers should not rely on the tests returned by this method unless the caller uses a subclass that overrides `TestSuite._removeTestAtIndex()` to preserve test references.

Alterado na versão 3.2: In earlier versions the *TestSuite* accessed tests directly rather than through iteration, so overriding `__iter__()` wasn't sufficient for providing tests.

Alterado na versão 3.4: In earlier versions the *TestSuite* held references to each *TestCase* after *TestSuite.run()*. Subclasses can restore that behavior by overriding `TestSuite._removeTestAtIndex()`.

In the typical usage of a *TestSuite* object, the *run()* method is invoked by a `TestRunner` rather than by the end-user test harness.

Carregando e executando testes

class unittest.**TestLoader**

The *TestLoader* class is used to create test suites from classes and modules. Normally, there is no need to create an instance of this class; the *unittest* module provides an instance that can be shared as *unittest.defaultTestLoader*. Using a subclass or instance, however, allows customization of some configurable properties.

Objetos da classe *TestLoader* possuem os seguintes atributos:

errors

A list of the non-fatal errors encountered while loading tests. Not reset by the loader at any point. Fatal errors are signalled by the relevant method raising an exception to the caller. Non-fatal errors are also indicated by a synthetic test that will raise the original error when run.

Adicionado na versão 3.5.

Objetos da classe *TestLoader* possuem os seguintes métodos:

loadTestsFromTestCase(*testCaseClass*)

Return a suite of all test cases contained in the *TestCase*-derived `testCaseClass`.

A test case instance is created for each method named by *getTestCaseNames()*. By default these are the method names beginning with `test`. If *getTestCaseNames()* returns no methods, but the `runTest()` method is implemented, a single test case is created for that method instead.

loadTestsFromModule(*module*, *, *pattern=None*)

Return a suite of all test cases contained in the given module. This method searches *module* for classes derived from *TestCase* and creates an instance of the class for each test method defined for the class.

> **Nota**
>
> While using a hierarchy of *TestCase*-derived classes can be convenient in sharing fixtures and helper functions, defining test methods on base classes that are not intended to be instantiated directly does not play well with this method. Doing so, however, can be useful when the fixtures are different and defined in subclasses.

If a module provides a `load_tests` function it will be called to load the tests. This allows modules to customize test loading. This is the *load_tests protocol*. The *pattern* argument is passed as the third argument to `load_tests`.

Alterado na versão 3.2: Suporte para `load_tests` adicionado.

Alterado na versão 3.5: Support for a keyword-only argument *pattern* has been added.

Alterado na versão 3.12: The undocumented and unofficial *use_load_tests* parameter has been removed.

loadTestsFromName(*name*, *module=None*)

Return a suite of all test cases given a string specifier.

The specifier *name* is a "dotted name" that may resolve either to a module, a test case class, a test method within a test case class, a *TestSuite* instance, or a callable object which returns a *TestCase* or *TestSuite* instance. These checks are applied in the order listed here; that is, a method on a possible test case class will be picked up as "a test method within a test case class", rather than "a callable object".

For example, if you have a module `SampleTests` containing a *TestCase*-derived class `SampleTestCase` with three test methods (`test_one()`, `test_two()`, and `test_three()`), the specifier `'SampleTests.SampleTestCase'` would cause this method to return a suite which will run all three test methods. Using the specifier `'SampleTests.SampleTestCase.test_two'` would cause it to return a test suite which will run only the `test_two()` test method. The specifier can refer to modules and packages which have not been imported; they will be imported as a side-effect.

O método opcionalmente resolve o *nome* relativo ao *módulo* dado.

Alterado na versão 3.5: If an *ImportError* or *AttributeError* occurs while traversing *name* then a synthetic test that raises that error when run will be returned. These errors are included in the errors accumulated by self.errors.

loadTestsFromNames(*names*, *module=None*)

Similar to *loadTestsFromName()*, but takes a sequence of names rather than a single name. The return value is a test suite which supports all the tests defined for each name.

getTestCaseNames(*testCaseClass*)

Return a sorted sequence of method names found within *testCaseClass*; this should be a subclass of `TestCase`.

discover(*start_dir*, *pattern='test*.py'*, *top_level_dir=None*)

Find all the test modules by recursing into subdirectories from the specified start directory, and return a TestSuite object containing them. Only test files that match *pattern* will be loaded. (Using shell style pattern matching.) Only module names that are importable (i.e. are valid Python identifiers) will be loaded.

All test modules must be importable from the top level of the project. If the start directory is not the top level directory then *top_level_dir* must be specified separately.

If importing a module fails, for example due to a syntax error, then this will be recorded as a single error and discovery will continue. If the import failure is due to `SkipTest` being raised, it will be recorded as a skip instead of an error.

If a package (a directory containing a file named `__init__.py`) is found, the package will be checked for a `load_tests` function. If this exists then it will be called `package.load_tests(loader, tests, pattern)`. Test discovery takes care to ensure that a package is only checked for tests once during an invocation, even if the load_tests function itself calls `loader.discover`.

If `load_tests` exists then discovery does *not* recurse into the package, `load_tests` is responsible for loading all tests in the package.

The pattern is deliberately not stored as a loader attribute so that packages can continue discovery themselves.

top_level_dir is stored internally, and used as a default to any nested calls to `discover()`. That is, if a package's `load_tests` calls `loader.discover()`, it does not need to pass this argument.

start_dir can be a dotted module name as well as a directory.

Adicionado na versão 3.2.

Alterado na versão 3.4: Modules that raise `SkipTest` on import are recorded as skips, not errors.

start_dir can be a *namespace packages*.

Paths are sorted before being imported so that execution order is the same even if the underlying file system's ordering is not dependent on file name.

Alterado na versão 3.5: Found packages are now checked for `load_tests` regardless of whether their path matches *pattern*, because it is impossible for a package name to match the default pattern.

Alterado na versão 3.11: *start_dir* can not be a *namespace packages*. It has been broken since Python 3.7, and Python 3.11 officially removes it.

Alterado na versão 3.13: *top_level_dir* is only stored for the duration of *discover* call.

Alterado na versão 3.14: *start_dir* can once again be a *namespace package*.

The following attributes of a `TestLoader` can be configured either by subclassing or assignment on an instance:

testMethodPrefix

String giving the prefix of method names which will be interpreted as test methods. The default value is `'test'`.

This affects `getTestCaseNames()` and all the `loadTestsFrom*` methods.

sortTestMethodsUsing

Function to be used to compare method names when sorting them in `getTestCaseNames()` and all the `loadTestsFrom*` methods.

suiteClass

Callable object that constructs a test suite from a list of tests. No methods on the resulting object are needed. The default value is the *TestSuite* class.

This affects all the `loadTestsFrom*` methods.

testNamePatterns

List of Unix shell-style wildcard test name patterns that test methods have to match to be included in test suites (see `-k` option).

If this attribute is not `None` (the default), all test methods to be included in test suites must match one of the patterns in this list. Note that matches are always performed using *fnmatch.fnmatchcase()*, so unlike patterns passed to the `-k` option, simple substring patterns will have to be converted using `*` wildcards.

This affects all the `loadTestsFrom*` methods.

Adicionado na versão 3.7.

class unittest.**TestResult**

This class is used to compile information about which tests have succeeded and which have failed.

A *TestResult* object stores the results of a set of tests. The *TestCase* and *TestSuite* classes ensure that results are properly recorded; test authors do not need to worry about recording the outcome of tests.

Testing frameworks built on top of *unittest* may want access to the *TestResult* object generated by running a set of tests for reporting purposes; a *TestResult* instance is returned by the `TestRunner.run()` method for this purpose.

TestResult instances have the following attributes that will be of interest when inspecting the results of running a set of tests:

errors

A list containing 2-tuples of *TestCase* instances and strings holding formatted tracebacks. Each tuple represents a test which raised an unexpected exception.

failures

A list containing 2-tuples of *TestCase* instances and strings holding formatted tracebacks. Each tuple represents a test where a failure was explicitly signalled using the *assert* methods*.

skipped

A list containing 2-tuples of *TestCase* instances and strings holding the reason for skipping the test.

Adicionado na versão 3.1.

expectedFailures

A list containing 2-tuples of *TestCase* instances and strings holding formatted tracebacks. Each tuple represents an expected failure or error of the test case.

unexpectedSuccesses

A list containing *TestCase* instances that were marked as expected failures, but succeeded.

collectedDurations

A list containing 2-tuples of test case names and floats representing the elapsed time of each test which was run.

Adicionado na versão 3.12.

shouldStop

Set to `True` when the execution of tests should stop by *stop()*.

testsRun

The total number of tests run so far.

buffer

If set to true, `sys.stdout` and `sys.stderr` will be buffered in between *startTest()* and *stopTest()* being called. Collected output will only be echoed onto the real `sys.stdout` and `sys.stderr` if the test fails or errors. Any output is also attached to the failure / error message.

Adicionado na versão 3.2.

failfast

Se definido como true (verdadeiro) *stop()* será chamado na primeira falha ou erro, interrompendo a execução do teste.

Adicionado na versão 3.2.

tb_locals

If set to true then local variables will be shown in tracebacks.

Adicionado na versão 3.5.

wasSuccessful()

Return `True` if all tests run so far have passed, otherwise returns `False`.

Alterado na versão 3.4: Returns `False` if there were any *unexpectedSuccesses* from tests marked with the *expectedFailure()* decorator.

stop()

This method can be called to signal that the set of tests being run should be aborted by setting the *shouldStop* attribute to `True`. `TestRunner` objects should respect this flag and return without running any additional tests.

For example, this feature is used by the *TextTestRunner* class to stop the test framework when the user signals an interrupt from the keyboard. Interactive tools which provide `TestRunner` implementations can use this in a similar manner.

The following methods of the *TestResult* class are used to maintain the internal data structures, and may be extended in subclasses to support additional reporting requirements. This is particularly useful in building tools which support interactive reporting while tests are being run.

startTest(*test*)

Called when the test case *test* is about to be run.

stopTest(*test*)

Called after the test case *test* has been executed, regardless of the outcome.

startTestRun()

Called once before any tests are executed.

Adicionado na versão 3.1.

stopTestRun()

Called once after all tests are executed.

Adicionado na versão 3.1.

addError(*test*, *err*)

Called when the test case *test* raises an unexpected exception. *err* is a tuple of the form returned by *sys.exc_info()*: `(type, value, traceback)`.

The default implementation appends a tuple `(test, formatted_err)` to the instance's *errors* attribute, where *formatted_err* is a formatted traceback derived from *err*.

addFailure(*test*, *err*)

Called when the test case *test* signals a failure. *err* is a tuple of the form returned by *sys.exc_info()*: `(type, value, traceback)`.

The default implementation appends a tuple `(test, formatted_err)` to the instance's *failures* attribute, where *formatted_err* is a formatted traceback derived from *err*.

addSuccess(*test*)

Called when the test case *test* succeeds.

The default implementation does nothing.

addSkip(*test*, *reason*)

Called when the test case *test* is skipped. *reason* is the reason the test gave for skipping.

The default implementation appends a tuple `(test, reason)` to the instance's *skipped* attribute.

addExpectedFailure(*test*, *err*)

Called when the test case *test* fails or errors, but was marked with the *expectedFailure()* decorator.

The default implementation appends a tuple `(test, formatted_err)` to the instance's *expectedFailures* attribute, where *formatted_err* is a formatted traceback derived from *err*.

addUnexpectedSuccess(*test*)

Called when the test case *test* was marked with the *expectedFailure()* decorator, but succeeded.

The default implementation appends the test to the instance's *unexpectedSuccesses* attribute.

addSubTest(*test*, *subtest*, *outcome*)

Called when a subtest finishes. *test* is the test case corresponding to the test method. *subtest* is a custom *TestCase* instance describing the subtest.

If *outcome* is *None*, the subtest succeeded. Otherwise, it failed with an exception where *outcome* is a tuple of the form returned by *sys.exc_info()*: `(type, value, traceback)`.

The default implementation does nothing when the outcome is a success, and records subtest failures as normal failures.

Adicionado na versão 3.4.

addDuration(*test*, *elapsed*)

Called when the test case finishes. *elapsed* is the time represented in seconds, and it includes the execution of cleanup functions.

Adicionado na versão 3.12.

class unittest.**TextTestResult**(*stream*, *descriptions*, *verbosity*, *, *durations=None*)

A concrete implementation of *TestResult* used by the *TextTestRunner*. Subclasses should accept `**kwargs` to ensure compatibility as the interface changes.

Adicionado na versão 3.2.

Alterado na versão 3.12: Added the *durations* keyword parameter.

unittest.**defaultTestLoader**

Instance of the *TestLoader* class intended to be shared. If no customization of the *TestLoader* is needed, this instance can be used instead of repeatedly creating new instances.

class unittest.**TextTestRunner**(*stream=None*, *descriptions=True*, *verbosity=1*, *failfast=False*, *buffer=False*, *resultclass=None*, *warnings=None*, *, *tb_locals=False*, *durations=None*)

A basic test runner implementation that outputs results to a stream. If *stream* is `None`, the default, *sys.stderr* is used as the output stream. This class has a few configurable parameters, but is essentially very simple. Graphical applications which run test suites should provide alternate implementations. Such implementations should accept `**kwargs` as the interface to construct runners changes when features are added to unittest.

By default this runner shows *DeprecationWarning*, *PendingDeprecationWarning*, *ResourceWarning* and *ImportWarning* even if they are *ignored by default*. This behavior can be overridden using Python's `-Wd` or `-Wa` options (see Warning control) and leaving *warnings* to `None`.

Alterado na versão 3.2: Added the *warnings* parameter.

Alterado na versão 3.2: The default stream is set to *sys.stderr* at instantiation time rather than import time.

Alterado na versão 3.5: Added the *tb_locals* parameter.

Alterado na versão 3.12: Added the *durations* parameter.

_makeResult()

This method returns the instance of `TestResult` used by *run()*. It is not intended to be called directly, but can be overridden in subclasses to provide a custom `TestResult`.

`_makeResult()` instantiates the class or callable passed in the `TextTestRunner` constructor as the `resultclass` argument. It defaults to *TextTestResult* if no `resultclass` is provided. The result class is instantiated with the following arguments:

```
stream, descriptions, verbosity
```

run(*test*)

This method is the main public interface to the `TextTestRunner`. This method takes a *TestSuite* or *TestCase* instance. A *TestResult* is created by calling *_makeResult()* and the test(s) are run and the results printed to stdout.

unittest.**main**(*module='__main__'*, *defaultTest=None*, *argv=None*, *testRunner=None*, *testLoader=unittest.defaultTestLoader*, *exit=True*, *verbosity=1*, *failfast=None*, *catchbreak=None*, *buffer=None*, *warnings=None*)

A command-line program that loads a set of tests from *module* and runs them; this is primarily for making test modules conveniently executable. The simplest use for this function is to include the following line at the end of a test script:

```
if __name__ == '__main__':
    unittest.main()
```

You can run tests with more detailed information by passing in the verbosity argument:

```
if __name__ == '__main__':
    unittest.main(verbosity=2)
```

The *defaultTest* argument is either the name of a single test or an iterable of test names to run if no test names are specified via *argv*. If not specified or `None` and no test names are provided via *argv*, all tests found in *module* are run.

O argumento *argv* pode ser uma lista de opções passada para o programa, com o primeiro elemento sendo o nome do programa. Se não for especificado ou for `None`, os valores de *sys.argv* são usados.

The *testRunner* argument can either be a test runner class or an already created instance of it. By default `main` calls *sys.exit()* with an exit code indicating success (0) or failure (1) of the tests run. An exit code of 5 indicates that no tests were run or skipped.

The *testLoader* argument has to be a *TestLoader* instance, and defaults to *defaultTestLoader*.

`main` supports being used from the interactive interpreter by passing in the argument `exit=False`. This displays the result on standard output without calling *sys.exit()*:

```
>>> from unittest import main
>>> main(module='test_module', exit=False)
```

The *failfast*, *catchbreak* and *buffer* parameters have the same effect as the same-name *command-line options*.

The *warnings* argument specifies the *warning filter* that should be used while running the tests. If it's not specified, it will remain `None` if a `-W` option is passed to **python** (see Warning control), otherwise it will be set to `'default'`.

Calling `main` returns an object with the `result` attribute that contains the result of the tests run as a *unittest.TestResult*.

Alterado na versão 3.1: The *exit* parameter was added.

Alterado na versão 3.2: The *verbosity*, *failfast*, *catchbreak*, *buffer* and *warnings* parameters were added.

Alterado na versão 3.4: The *defaultTest* parameter was changed to also accept an iterable of test names.

load_tests Protocol

Adicionado na versão 3.2.

Modules or packages can customize how tests are loaded from them during normal test runs or test discovery by implementing a function called `load_tests`.

If a test module defines `load_tests` it will be called by *TestLoader.loadTestsFromModule()* with the following arguments:

```
load_tests(loader, standard_tests, pattern)
```

where *pattern* is passed straight through from `loadTestsFromModule`. It defaults to `None`.

It should return a *TestSuite*.

loader is the instance of *TestLoader* doing the loading. *standard_tests* are the tests that would be loaded by default from the module. It is common for test modules to only want to add or remove tests from the standard set of tests. The third argument is used when loading packages as part of test discovery.

A typical `load_tests` function that loads tests from a specific set of *TestCase* classes may look like:

```
test_cases = (TestCase1, TestCase2, TestCase3)

def load_tests(loader, tests, pattern):
    suite = TestSuite()
    for test_class in test_cases:
        tests = loader.loadTestsFromTestCase(test_class)
        suite.addTests(tests)
    return suite
```

If discovery is started in a directory containing a package, either from the command line or by calling *TestLoader.discover()*, then the package `__init__.py` will be checked for `load_tests`. If that function does not exist, discovery will recurse into the package as though it were just another directory. Otherwise, discovery of the package's tests will be left up to `load_tests` which is called with the following arguments:

```
load_tests(loader, standard_tests, pattern)
```

This should return a *TestSuite* representing all the tests from the package. (`standard_tests` will only contain tests collected from `__init__.py`.)

Because the pattern is passed into `load_tests` the package is free to continue (and potentially modify) test discovery. A 'do nothing' `load_tests` function for a test package would look like:

```
def load_tests(loader, standard_tests, pattern):
    # top level directory cached on loader instance
    this_dir = os.path.dirname(__file__)
    package_tests = loader.discover(start_dir=this_dir, pattern=pattern)
    standard_tests.addTests(package_tests)
    return standard_tests
```

Alterado na versão 3.5: Discovery no longer checks package names for matching *pattern* due to the impossibility of package names matching the default pattern.

26.5.9 Classes e Módulos de Definição de Contexto

Definições de contexto em um nível de classe e módulo são implementadas na classe *TestSuite*. Quando a suíte de testes encontrar um teste de uma nova classe, o método `tearDownClass()` da classe anterior (se houver alguma) é chamado logo antes da chamada do método `setUpClass()` da nova classe.

Similarly if a test is from a different module from the previous test then `tearDownModule` from the previous module is run, followed by `setUpModule` from the new module.

Após executar todos os testes, haverá a execução final do `tearDownClass` e do `tearDownModule`.

Note that shared fixtures do not play well with [potential] features like test parallelization and they break test isolation. They should be used with care.

The default ordering of tests created by the unittest test loaders is to group all tests from the same modules and classes together. This will lead to `setUpClass` / `setUpModule` (etc) being called exactly once per class and module. If you randomize the order, so that tests from different modules and classes are adjacent to each other, then these shared fixture functions may be called multiple times in a single test run.

Shared fixtures are not intended to work with suites with non-standard ordering. A `BaseTestSuite` still exists for frameworks that don't want to support shared fixtures.

If there are any exceptions raised during one of the shared fixture functions the test is reported as an error. Because there is no corresponding test instance an `_ErrorHolder` object (that has the same interface as a *`TestCase`*) is created to represent the error. If you are just using the standard unittest test runner then this detail doesn't matter, but if you are a framework author it may be relevant.

setUpClass and tearDownClass

These must be implemented as class methods:

```
import unittest

class Test(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls._connection = createExpensiveConnectionObject()

    @classmethod
    def tearDownClass(cls):
        cls._connection.destroy()
```

If you want the `setUpClass` and `tearDownClass` on base classes called then you must call up to them yourself. The implementations in *`TestCase`* are empty.

If an exception is raised during a `setUpClass` then the tests in the class are not run and the `tearDownClass` is not run. Skipped classes will not have `setUpClass` or `tearDownClass` run. If the exception is a *`SkipTest`* exception then the class will be reported as having been skipped instead of as an error.

setUpModule and tearDownModule

These should be implemented as functions:

```
def setUpModule():
    createConnection()

def tearDownModule():
    closeConnection()
```

If an exception is raised in a `setUpModule` then none of the tests in the module will be run and the `tearDownModule` will not be run. If the exception is a *`SkipTest`* exception then the module will be reported as having been skipped instead of as an error.

To add cleanup code that must be run even in the case of an exception, use `addModuleCleanup`:

`unittest.`**`addModuleCleanup`**(*function*, */*, **args*, ***kwargs*)

Add a function to be called after *`tearDownModule()`* to cleanup resources used during the test class. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into *`addModuleCleanup()`* when they are added.

If *setUpModule()* fails, meaning that *tearDownModule()* is not called, then any cleanup functions added will still be called.

Adicionado na versão 3.8.

unittest.**enterModuleContext**(*cm*)

Enter the supplied *context manager*. If successful, also add its `__exit__()` method as a cleanup function by *addModuleCleanup()* and return the result of the `__enter__()` method.

Adicionado na versão 3.11.

unittest.**doModuleCleanups**()

This function is called unconditionally after *tearDownModule()*, or after *setUpModule()* if *setUpModule()* raises an exception.

It is responsible for calling all the cleanup functions added by *addModuleCleanup()*. If you need cleanup functions to be called *prior* to *tearDownModule()* then you can call *doModuleCleanups()* yourself.

doModuleCleanups() pops methods off the stack of cleanup functions one at a time, so it can be called at any time.

Adicionado na versão 3.8.

26.5.10 Tratamento de sinal

Adicionado na versão 3.2.

The *-c/--catch* command-line option to unittest, along with the `catchbreak` parameter to *unittest.main()*, provide more friendly handling of control-C during a test run. With catch break behavior enabled control-C will allow the currently running test to complete, and the test run will then end and report all the results so far. A second control-c will raise a *KeyboardInterrupt* in the usual way.

The control-c handling signal handler attempts to remain compatible with code or tests that install their own *signal.SIGINT* handler. If the `unittest` handler is called but *isn't* the installed *signal.SIGINT* handler, i.e. it has been replaced by the system under test and delegated to, then it calls the default handler. This will normally be the expected behavior by code that replaces an installed handler and delegates to it. For individual tests that need `unittest` control-c handling disabled the *removeHandler()* decorator can be used.

There are a few utility functions for framework authors to enable control-c handling functionality within test frameworks.

unittest.**installHandler**()

Install the control-c handler. When a *signal.SIGINT* is received (usually in response to the user pressing control-c) all registered results have *stop()* called.

unittest.**registerResult**(*result*)

Register a *TestResult* object for control-c handling. Registering a result stores a weak reference to it, so it doesn't prevent the result from being garbage collected.

Registering a *TestResult* object has no side-effects if control-c handling is not enabled, so test frameworks can unconditionally register all results they create independently of whether or not handling is enabled.

unittest.**removeResult**(*result*)

Remove um resultado registrado. Dado que um resultado for removido, então *stop()* não será mais chamado no objeto resultado em resposta a um Ctrl+C

unittest.**removeHandler**(*function=None*)

When called without arguments this function removes the control-c handler if it has been installed. This function can also be used as a test decorator to temporarily remove the handler while the test is being executed:

```
@unittest.removeHandler
def test_signal_handling(self):
    ...
```

26.6 `unittest.mock` — mock object library

Adicionado na versão 3.3.

Código-fonte: Lib/unittest/mock.py

---

unittest.mock é uma biblioteca para teste em Python. Que permite substituir partes do seu sistema em teste por objetos simulados e fazer afirmações sobre como elas foram usadas.

unittest.mock fornece uma classe core *Mock* removendo a necessidade de criar uma série de stubs em todo o seu conjunto de testes. Depois de executar uma ação, você pode fazer afirmações sobre quais métodos / atributos foram usados e com quais argumentos foram chamados. Você também pode especificar valores de retorno e definir os atributos necessários da maneira normal.

Adicionalmente, o mock fornece um decorador *patch()* que lida com os atributos do módulo de patch e do nível de classe no escopo de um teste, junto com *sentinel* para criar objetos únicos. Veja o *guia rápido* para alguns exemplos de como usar *Mock*, *MagicMock* e *patch()*.

Mock foi projetado para uso com *unittest* e é baseado no padrão 'ação -> asserção' em vez de 'gravar -> reproduzir' usado por muitas estruturas de simulação.

There is a backport of *unittest.mock* for earlier versions of Python, available as mock on PyPI.

26.6.1 Guia Rápido

Os objetos *Mock* e *MagicMock* criam todos os atributos e métodos à medida que você os acessa e armazena detalhes de como eles foram usados. Você pode configurá-los, especificar valores de retorno ou limitar quais atributos estão disponíveis e, em seguida, fazer afirmações sobre como eles foram usados:

```
>>> from unittest.mock import MagicMock
>>> thing = ProductionClass()
>>> thing.method = MagicMock(return_value=3)
>>> thing.method(3, 4, 5, key='value')
3
>>> thing.method.assert_called_with(3, 4, 5, key='value')
```

side_effect allows you to perform side effects, including raising an exception when a mock is called:

```
>>> from unittest.mock import Mock
>>> mock = Mock(side_effect=KeyError('foo'))
>>> mock()
Traceback (most recent call last):
 ...
KeyError: 'foo'
```

```
>>> values = {'a': 1, 'b': 2, 'c': 3}
>>> def side_effect(arg):
...     return values[arg]
...
>>> mock.side_effect = side_effect
>>> mock('a'), mock('b'), mock('c')
(1, 2, 3)
>>> mock.side_effect = [5, 4, 3, 2, 1]
>>> mock(), mock(), mock()
(5, 4, 3)
```

O Mock tem muitas outras maneiras de configurá-lo e controlar seu comportamento. Por exemplo, o argumento *spec* configura o mock para obter sua especificação de outro objeto. Tentar acessar atributos ou métodos no mock que não existem no spec falhará com um *AttributeError*.

O gerenciador de contexto / decorador *patch()* facilita a simulação de classes ou objetos em um módulo em teste. O objeto que você especificar será substituído por um mock (ou outro objeto) durante o teste e restaurado quando o teste terminar:

```
>>> from unittest.mock import patch
>>> @patch('module.ClassName2')
... @patch('module.ClassName1')
... def test(MockClass1, MockClass2):
...     module.ClassName1()
...     module.ClassName2()
...     assert MockClass1 is module.ClassName1
...     assert MockClass2 is module.ClassName2
...     assert MockClass1.called
...     assert MockClass2.called
...
>>> test()
```

Nota

Quando você aninha decoradores de patches, as simulações são passadas para a função decorada na mesma ordem em que foram aplicadas (a ordem normal *Python* em que os decoradores são aplicados). Isso significa de baixo para cima, portanto, no exemplo acima, a simulação para `module.ClassName1` é passada primeiro.

Com *patch()*, é importante que você faça o patch de objetos no espaço de nomes onde eles são procurados. Normalmente, isso é simples, mas para um guia rápido, leia *onde fazer o patch*.

Assim como um decorador *patch()* pode ser usado como um gerenciador de contexto em uma instrução with:

```
>>> with patch.object(ProductionClass, 'method', return_value=None) as mock_method:
...     thing = ProductionClass()
...     thing.method(1, 2, 3)
...
>>> mock_method.assert_called_once_with(1, 2, 3)
```

Também existe *patch.dict()* para definir valores em um dicionário apenas durante um escopo e restaurar o dicionário ao seu estado original quando o teste termina:

```
>>> foo = {'key': 'value'}
>>> original = foo.copy()
>>> with patch.dict(foo, {'newkey': 'newvalue'}, clear=True):
...     assert foo == {'newkey': 'newvalue'}
...
>>> assert foo == original
```

Mock possui suporte a simulação de *métodos mágicos* de Python. A maneira mais fácil de usar métodos mágicos é com a classe *MagicMock*. Ele permite que você faça coisas como:

```
>>> mock = MagicMock()
>>> mock.__str__.return_value = 'foobarbaz'
>>> str(mock)
'foobarbaz'
>>> mock.__str__.assert_called_with()
```

Mock permite atribuir funções (ou outras instâncias do Mock) a métodos mágicos e elas serão chamadas apropriadamente. A classe *MagicMock* é apenas uma variante do Mock que possui todos os métodos mágicos pré-criados para você (bem, todos os úteis de qualquer maneira).

A seguir, é apresentado um exemplo do uso de métodos mágicos com a classe Mock comum:

```
>>> mock = Mock()
>>> mock.__str__ = Mock(return_value='wheeeeee')
>>> str(mock)
'wheeeeee'
```

Para garantir que os objetos mock em seus testes tenham a mesmo API que os objetos que eles estão substituindo, você pode usar *especificação automática*. A especificação automática pode ser feita por meio do argumento *autospec* para fazer patch ou pela função `create_autospec()`. A especificação automática cria objetos mock que têm os mesmos atributos e métodos que os objetos que estão substituindo, e qualquer funções e métodos (incluindo construtores) têm a mesma assinatura de chamada que o objeto real.

Isso garante que seus mocks falharão da mesma forma que o código de produção se forem usados incorretamente:

```
>>> from unittest.mock import create_autospec
>>> def function(a, b, c):
...     pass
...
>>> mock_function = create_autospec(function, return_value='fishy')
>>> mock_function(1, 2, 3)
'fishy'
>>> mock_function.assert_called_once_with(1, 2, 3)
>>> mock_function('wrong arguments')
Traceback (most recent call last):
  ...
TypeError: missing a required argument: 'b'
```

`create_autospec()` também pode ser usada com classes, onde copia a assinatura do método `__init__`, e com objetos chamáveis onde copia a assinatura do método `__call__`.

26.6.2 A classe Mock

`Mock` é um objeto simulado flexível destinado a substituir o uso de *stubs* e dublês de teste em todo o seu código. Os mocks são chamáveis e cria atributos como novos mocks à medida que você os acessa[1]. Acessar o mesmo atributo sempre retorna o mesmo mock. Os mocks registram como você os utiliza, permitindo que você faça asserções sobre o que o seu código fez com eles.

`MagicMock` é uma subclasse de `Mock` com todos os métodos mágicos pré-criados e prontos para uso. Existem também variantes não chamáveis, úteis quando você está simulando objetos que não são chamáveis: `NonCallableMock` e `NonCallableMagicMock`

The `patch()` decorators makes it easy to temporarily replace classes in a particular module with a `Mock` object. By default `patch()` will create a `MagicMock` for you. You can specify an alternative class of `Mock` using the *new_callable* argument to `patch()`.

class unittest.mock.**Mock**(*spec=None*, *side_effect=None*, *return_value=DEFAULT*, *wraps=None*, *name=None*, *spec_set=None*, *unsafe=False*, ***kwargs*)

Create a new `Mock` object. `Mock` takes several optional arguments that specify the behaviour of the Mock object:

- *spec*: This can be either a list of strings or an existing object (a class or instance) that acts as the specification for the mock object. If you pass in an object then a list of strings is formed by calling dir on the object (excluding unsupported magic attributes and methods). Accessing any attribute not in this list will raise an `AttributeError`.

  If *spec* is an object (rather than a list of strings) then `__class__` returns the class of the spec object. This allows mocks to pass `isinstance()` tests.

[1] The only exceptions are magic methods and attributes (those that have leading and trailing double underscores). Mock doesn't create these but instead raises an `AttributeError`. This is because the interpreter will often implicitly request these methods, and gets *very* confused to get a new Mock object when it expects a magic method. If you need magic method support see *magic methods*.

- *spec_set*: A stricter variant of *spec*. If used, attempting to *set* or get an attribute on the mock that isn't on the object passed as *spec_set* will raise an *AttributeError*.
- *side_effect*: A function to be called whenever the Mock is called. See the *side_effect* attribute. Useful for raising exceptions or dynamically changing return values. The function is called with the same arguments as the mock, and unless it returns *DEFAULT*, the return value of this function is used as the return value.

  Alternatively *side_effect* can be an exception class or instance. In this case the exception will be raised when the mock is called.

  If *side_effect* is an iterable then each call to the mock will return the next value from the iterable.

  A *side_effect* can be cleared by setting it to `None`.
- *return_value*: The value returned when the mock is called. By default this is a new Mock (created on first access). See the *return_value* attribute.
- *unsafe*: By default, accessing any attribute whose name starts with *assert*, *assret*, *asert*, *aseert* or *assrt* will raise an *AttributeError*. Passing `unsafe=True` will allow access to these attributes.

  Adicionado na versão 3.5.
- *wraps*: Item for the mock object to wrap. If *wraps* is not `None` then calling the Mock will pass the call through to the wrapped object (returning the real result). Attribute access on the mock will return a Mock object that wraps the corresponding attribute of the wrapped object (so attempting to access an attribute that doesn't exist will raise an *AttributeError*).

  If the mock has an explicit *return_value* set then calls are not passed to the wrapped object and the *return_value* is returned instead.
- *name*: If the mock has a name then it will be used in the repr of the mock. This can be useful for debugging. The name is propagated to child mocks.

Mocks can also be called with arbitrary keyword arguments. These will be used to set attributes on the mock after it is created. See the *configure_mock()* method for details.

assert_called()

Afirmar que o mock foi chamado pelo menos uma vez.

```
>>> mock = Mock()
>>> mock.method()
<Mock name='mock.method()' id='...'>
>>> mock.method.assert_called()
```

Adicionado na versão 3.6.

assert_called_once()

Afirma que o mock foi chamado exatamente uma vez.

```
>>> mock = Mock()
>>> mock.method()
<Mock name='mock.method()' id='...'>
>>> mock.method.assert_called_once()
>>> mock.method()
<Mock name='mock.method()' id='...'>
>>> mock.method.assert_called_once()
Traceback (most recent call last):
...
AssertionError: Expected 'method' to have been called once. Called 2 times.
Calls: [call(), call()].
```

Adicionado na versão 3.6.

assert_called_with(**args*, ***kwargs*)

This method is a convenient way of asserting that the last call has been made in a particular way:

```
>>> mock = Mock()
>>> mock.method(1, 2, 3, test='wow')
<Mock name='mock.method()' id='...'>
>>> mock.method.assert_called_with(1, 2, 3, test='wow')
```

assert_called_once_with(**args*, ***kwargs*)

Assert that the mock was called exactly once and that call was with the specified arguments.

```
>>> mock = Mock(return_value=None)
>>> mock('foo', bar='baz')
>>> mock.assert_called_once_with('foo', bar='baz')
>>> mock('other', bar='values')
>>> mock.assert_called_once_with('other', bar='values')
Traceback (most recent call last):
  ...
AssertionError: Expected 'mock' to be called once. Called 2 times.
Calls: [call('foo', bar='baz'), call('other', bar='values')].
```

assert_any_call(**args*, ***kwargs*)

assert the mock has been called with the specified arguments.

The assert passes if the mock has *ever* been called, unlike *assert_called_with()* and *assert_called_once_with()* that only pass if the call is the most recent one, and in the case of *assert_called_once_with()* it must also be the only call.

```
>>> mock = Mock(return_value=None)
>>> mock(1, 2, arg='thing')
>>> mock('some', 'thing', 'else')
>>> mock.assert_any_call(1, 2, arg='thing')
```

assert_has_calls(*calls*, *any_order=False*)

assert the mock has been called with the specified calls. The *mock_calls* list is checked for the calls.

If *any_order* is false then the calls must be sequential. There can be extra calls before or after the specified calls.

If *any_order* is true then the calls can be in any order, but they must all appear in *mock_calls*.

```
>>> mock = Mock(return_value=None)
>>> mock(1)
>>> mock(2)
>>> mock(3)
>>> mock(4)
>>> calls = [call(2), call(3)]
>>> mock.assert_has_calls(calls)
>>> calls = [call(4), call(2), call(3)]
>>> mock.assert_has_calls(calls, any_order=True)
```

assert_not_called()

Afirma que o mock nunca foi chamado.

```
>>> m = Mock()
>>> m.hello.assert_not_called()
>>> obj = m.hello()
>>> m.hello.assert_not_called()
Traceback (most recent call last):
```

(continua na próxima página)

(continuação da página anterior)

```
  ...
AssertionError: Expected 'hello' to not have been called. Called 1 times.
Calls: [call()].
```

Adicionado na versão 3.5.

reset_mock(*, *return_value=False*, *side_effect=False*)

The reset_mock method resets all the call attributes on a mock object:

```
>>> mock = Mock(return_value=None)
>>> mock('hello')
>>> mock.called
True
>>> mock.reset_mock()
>>> mock.called
False
```

This can be useful where you want to make a series of assertions that reuse the same object.

return_value parameter when set to `True` resets *`return_value`*:

```
>>> mock = Mock(return_value=5)
>>> mock('hello')
5
>>> mock.reset_mock(return_value=True)
>>> mock('hello')
<Mock name='mock()' id='...'>
```

side_effect parameter when set to `True` resets *`side_effect`*:

```
>>> mock = Mock(side_effect=ValueError)
>>> mock('hello')
Traceback (most recent call last):
  ...
ValueError
>>> mock.reset_mock(side_effect=True)
>>> mock('hello')
<Mock name='mock()' id='...'>
```

Note that *`reset_mock()`* *doesn't* clear the *`return_value`*, *`side_effect`* or any child attributes you have set using normal assignment by default.

Child mocks are reset as well.

Alterado na versão 3.6: Foram adicionados dois argumentos somente-nomeado à função reset_mock.

mock_add_spec(*spec*, *spec_set=False*)

Add a spec to a mock. *spec* can either be an object or a list of strings. Only attributes on the *spec* can be fetched as attributes from the mock.

If *spec_set* is true then only attributes on the spec can be set.

attach_mock(*mock*, *attribute*)

Attach a mock as an attribute of this one, replacing its name and parent. Calls to the attached mock will be recorded in the *`method_calls`* and *`mock_calls`* attributes of this one.

configure_mock(***kwargs*)

Define atributos no mock por meio de argumentos nomeados.

Attributes plus return values and side effects can be set on child mocks using standard dot notation and unpacking a dictionary in the method call:

```
>>> mock = Mock()
>>> attrs = {'method.return_value': 3, 'other.side_effect': KeyError}
>>> mock.configure_mock(**attrs)
>>> mock.method()
3
>>> mock.other()
Traceback (most recent call last):
  ...
KeyError
```

The same thing can be achieved in the constructor call to mocks:

```
>>> attrs = {'method.return_value': 3, 'other.side_effect': KeyError}
>>> mock = Mock(some_attribute='eggs', **attrs)
>>> mock.some_attribute
'eggs'
>>> mock.method()
3
>>> mock.other()
Traceback (most recent call last):
  ...
KeyError
```

configure_mock() exists to make it easier to do configuration after the mock has been created.

__dir__()

Mock objects limit the results of `dir(some_mock)` to useful results. For mocks with a *spec* this includes all the permitted attributes for the mock.

See *FILTER_DIR* for what this filtering does, and how to switch it off.

_get_child_mock(***kw*)

Create the child mocks for attributes and return value. By default child mocks will be the same type as the parent. Subclasses of Mock may want to override this to customize the way child mocks are made.

For non-callable mocks the callable variant will be used (rather than any custom subclass).

called

A boolean representing whether or not the mock object has been called:

```
>>> mock = Mock(return_value=None)
>>> mock.called
False
>>> mock()
>>> mock.called
True
```

call_count

An integer telling you how many times the mock object has been called:

```
>>> mock = Mock(return_value=None)
>>> mock.call_count
0
>>> mock()
>>> mock()
>>> mock.call_count
2
```

return_value

Set this to configure the value returned by calling the mock:

```
>>> mock = Mock()
>>> mock.return_value = 'fish'
>>> mock()
'fish'
```

The default return value is a mock object and you can configure it in the normal way:

```
>>> mock = Mock()
>>> mock.return_value.attribute = sentinel.Attribute
>>> mock.return_value()
<Mock name='mock()()' id='...'>
>>> mock.return_value.assert_called_with()
```

return_value também pode ser definido no construtor:

```
>>> mock = Mock(return_value=3)
>>> mock.return_value
3
>>> mock()
3
```

side_effect

This can either be a function to be called when the mock is called, an iterable or an exception (class or instance) to be raised.

If you pass in a function it will be called with same arguments as the mock and unless the function returns the *DEFAULT* singleton the call to the mock will then return whatever the function returns. If the function returns *DEFAULT* then the mock will return its normal value (from the *return_value*).

If you pass in an iterable, it is used to retrieve an iterator which must yield a value on every call. This value can either be an exception instance to be raised, or a value to be returned from the call to the mock (*DEFAULT* handling is identical to the function case).

An example of a mock that raises an exception (to test exception handling of an API):

```
>>> mock = Mock()
>>> mock.side_effect = Exception('Boom!')
>>> mock()
Traceback (most recent call last):
  ...
Exception: Boom!
```

Usando *side_effect* para retornar um sequência de valores:

```
>>> mock = Mock()
>>> mock.side_effect = [3, 2, 1]
>>> mock(), mock(), mock()
(3, 2, 1)
```

Usando um chamável:

```
>>> mock = Mock(return_value=3)
>>> def side_effect(*args, **kwargs):
...     return DEFAULT
...
>>> mock.side_effect = side_effect
>>> mock()
3
```

side_effect can be set in the constructor. Here's an example that adds one to the value the mock is called with and returns it:

```
>>> side_effect = lambda value: value + 1
>>> mock = Mock(side_effect=side_effect)
>>> mock(3)
4
>>> mock(-8)
-7
```

Configuração *side_effect* para `None` limpa isso:

```
>>> m = Mock(side_effect=KeyError, return_value=3)
>>> m()
Traceback (most recent call last):
 ...
KeyError
>>> m.side_effect = None
>>> m()
3
```

call_args

This is either `None` (if the mock hasn't been called), or the arguments that the mock was last called with. This will be in the form of a tuple: the first member, which can also be accessed through the `args` property, is any positional arguments the mock was called with (or an empty tuple) and the second member, which can also be accessed through the `kwargs` property, is any keyword arguments (or an empty dictionary).

```
>>> mock = Mock(return_value=None)
>>> print(mock.call_args)
None
>>> mock()
>>> mock.call_args
call()
>>> mock.call_args == ()
True
>>> mock(3, 4)
>>> mock.call_args
call(3, 4)
>>> mock.call_args == ((3, 4),)
True
>>> mock.call_args.args
(3, 4)
>>> mock.call_args.kwargs
{}
>>> mock(3, 4, 5, key='fish', next='w00t!')
>>> mock.call_args
call(3, 4, 5, key='fish', next='w00t!')
>>> mock.call_args.args
(3, 4, 5)
>>> mock.call_args.kwargs
{'key': 'fish', 'next': 'w00t!'}
```

call_args, along with members of the lists *call_args_list*, *method_calls* and *mock_calls* are *call* objects. These are tuples, so they can be unpacked to get at the individual arguments and make more complex assertions. See *calls as tuples*.

Alterado na versão 3.8: Adicionadas propriedades `args` e `kwargs`.

call_args_list

This is a list of all the calls made to the mock object in sequence (so the length of the list is the number of times it has been called). Before any calls have been made it is an empty list. The *call* object can be used for conveniently constructing lists of calls to compare with *call_args_list*.

```
>>> mock = Mock(return_value=None)
>>> mock()
>>> mock(3, 4)
>>> mock(key='fish', next='w00t!')
>>> mock.call_args_list
[call(), call(3, 4), call(key='fish', next='w00t!')]
>>> expected = [(), ((3, 4),), ({'key': 'fish', 'next': 'w00t!'},)]
>>> mock.call_args_list == expected
True
```

Members of *call_args_list* are *call* objects. These can be unpacked as tuples to get at the individual arguments. See *calls as tuples*.

method_calls

As well as tracking calls to themselves, mocks also track calls to methods and attributes, and *their* methods and attributes:

```
>>> mock = Mock()
>>> mock.method()
<Mock name='mock.method()' id='...'>
>>> mock.property.method.attribute()
<Mock name='mock.property.method.attribute()' id='...'>
>>> mock.method_calls
[call.method(), call.property.method.attribute()]
```

Members of *method_calls* are *call* objects. These can be unpacked as tuples to get at the individual arguments. See *calls as tuples*.

mock_calls

mock_calls records *all* calls to the mock object, its methods, magic methods *and* return value mocks.

```
>>> mock = MagicMock()
>>> result = mock(1, 2, 3)
>>> mock.first(a=3)
<MagicMock name='mock.first()' id='...'>
>>> mock.second()
<MagicMock name='mock.second()' id='...'>
>>> int(mock)
1
>>> result(1)
<MagicMock name='mock()()' id='...'>
>>> expected = [call(1, 2, 3), call.first(a=3), call.second(),
... call.__int__(), call()(1)]
>>> mock.mock_calls == expected
True
```

Members of *mock_calls* are *call* objects. These can be unpacked as tuples to get at the individual arguments. See *calls as tuples*.

Nota

The way *mock_calls* are recorded means that where nested calls are made, the parameters of ancestor calls are not recorded and so will always compare equal:

```
>>> mock = MagicMock()
>>> mock.top(a=3).bottom()
<MagicMock name='mock.top().bottom()' id='...'>
>>> mock.mock_calls
[call.top(a=3), call.top().bottom()]
>>> mock.mock_calls[-1] == call.top(a=-1).bottom()
True
```

__class__

Normally the __class__ attribute of an object will return its type. For a mock object with a `spec`, __class__ returns the spec class instead. This allows mock objects to pass *isinstance()* tests for the object they are replacing / masquerading as:

```
>>> mock = Mock(spec=3)
>>> isinstance(mock, int)
True
```

__class__ is assignable to, this allows a mock to pass an *isinstance()* check without forcing you to use a spec:

```
>>> mock = Mock()
>>> mock.__class__ = dict
>>> isinstance(mock, dict)
True
```

class unittest.mock.**NonCallableMock**(*spec=None*, *wraps=None*, *name=None*, *spec_set=None*, ***kwargs*)

A non-callable version of *Mock*. The constructor parameters have the same meaning of *Mock*, with the exception of *return_value* and *side_effect* which have no meaning on a non-callable mock.

Mock objects that use a class or an instance as a `spec` or `spec_set` are able to pass *isinstance()* tests:

```
>>> mock = Mock(spec=SomeClass)
>>> isinstance(mock, SomeClass)
True
>>> mock = Mock(spec_set=SomeClass())
>>> isinstance(mock, SomeClass)
True
```

The *Mock* classes have support for mocking magic methods. See *magic methods* for the full details.

The mock classes and the *patch()* decorators all take arbitrary keyword arguments for configuration. For the *patch()* decorators the keywords are passed to the constructor of the mock being created. The keyword arguments are for configuring attributes of the mock:

```
>>> m = MagicMock(attribute=3, other='fish')
>>> m.attribute
3
>>> m.other
'fish'
```

The return value and side effect of child mocks can be set in the same way, using dotted notation. As you can't use dotted names directly in a call you have to create a dictionary and unpack it using `**`:

```
>>> attrs = {'method.return_value': 3, 'other.side_effect': KeyError}
>>> mock = Mock(some_attribute='eggs', **attrs)
>>> mock.some_attribute
'eggs'
```

(continua na próxima página)

(continuação da página anterior)

```
>>> mock.method()
3
>>> mock.other()
Traceback (most recent call last):
  ...
KeyError
```

A callable mock which was created with a *spec* (or a *spec_set*) will introspect the specification object's signature when matching calls to the mock. Therefore, it can match the actual call's arguments regardless of whether they were passed positionally or by name:

```
>>> def f(a, b, c): pass
...
>>> mock = Mock(spec=f)
>>> mock(1, 2, c=3)
<Mock name='mock()' id='140161580456576'>
>>> mock.assert_called_with(1, 2, 3)
>>> mock.assert_called_with(a=1, b=2, c=3)
```

This applies to *assert_called_with()*, *assert_called_once_with()*, *assert_has_calls()* and *assert_any_call()*. When *Especificação automática*, it will also apply to method calls on the mock object.

Alterado na versão 3.4: Adicionada introspecção de assinatura em objetos mock especificados e auto-especificados.

class unittest.mock.**PropertyMock**(**args*, ***kwargs*)

A mock intended to be used as a *property*, or other *descriptor*, on a class. *PropertyMock* provides `__get__()` and `__set__()` methods so you can specify a return value when it is fetched.

Fetching a *PropertyMock* instance from an object calls the mock, with no args. Setting it calls the mock with the value being set.

```
>>> class Foo:
...     @property
...     def foo(self):
...         return 'something'
...     @foo.setter
...     def foo(self, value):
...         pass
...
>>> with patch('__main__.Foo.foo', new_callable=PropertyMock) as mock_foo:
...     mock_foo.return_value = 'mockity-mock'
...     this_foo = Foo()
...     print(this_foo.foo)
...     this_foo.foo = 6
...
mockity-mock
>>> mock_foo.mock_calls
[call(), call(6)]
```

Because of the way mock attributes are stored you can't directly attach a *PropertyMock* to a mock object. Instead you can attach it to the mock type object:

```
>>> m = MagicMock()
>>> p = PropertyMock(return_value=3)
>>> type(m).foo = p
>>> m.foo
3
>>> p.assert_called_once_with()
```

Cuidado

If an *AttributeError* is raised by *PropertyMock*, it will be interpreted as a missing descriptor and `__getattr__()` will be called on the parent mock:

```
>>> m = MagicMock()
>>> no_attribute = PropertyMock(side_effect=AttributeError)
>>> type(m).my_property = no_attribute
>>> m.my_property
<MagicMock name='mock.my_property' id='140165240345424'>
```

See `__getattr__()` for details.

class unittest.mock.**AsyncMock**(*spec=None*, *side_effect=None*, *return_value=DEFAULT*, *wraps=None*, *name=None*, *spec_set=None*, *unsafe=False*, ***kwargs*)

An asynchronous version of *MagicMock*. The *AsyncMock* object will behave so the object is recognized as an async function, and the result of a call is an awaitable.

```
>>> mock = AsyncMock()
>>> inspect.iscoroutinefunction(mock)
True
>>> inspect.isawaitable(mock())
True
```

The result of `mock()` is an async function which will have the outcome of `side_effect` or `return_value` after it has been awaited:

- if `side_effect` is a function, the async function will return the result of that function,
- if `side_effect` is an exception, the async function will raise the exception,
- if `side_effect` is an iterable, the async function will return the next value of the iterable, however, if the sequence of result is exhausted, `StopAsyncIteration` is raised immediately,
- if `side_effect` is not defined, the async function will return the value defined by `return_value`, hence, by default, the async function returns a new *AsyncMock* object.

Setting the *spec* of a *Mock* or *MagicMock* to an async function will result in a coroutine object being returned after calling.

```
>>> async def async_func(): pass
...
>>> mock = MagicMock(async_func)
>>> mock
<MagicMock spec='function' id='...'>
>>> mock()
<coroutine object AsyncMockMixin._mock_call at ...>
```

Setting the *spec* of a *Mock*, *MagicMock*, or *AsyncMock* to a class with asynchronous and synchronous functions will automatically detect the synchronous functions and set them as *MagicMock* (if the parent mock is *AsyncMock* or *MagicMock*) or *Mock* (if the parent mock is *Mock*). All asynchronous functions will be *AsyncMock*.

```
>>> class ExampleClass:
...     def sync_foo():
...         pass
...     async def async_foo():
...         pass
...
>>> a_mock = AsyncMock(ExampleClass)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> a_mock.sync_foo
<MagicMock name='mock.sync_foo' id='...'>
>>> a_mock.async_foo
<AsyncMock name='mock.async_foo' id='...'>
>>> mock = Mock(ExampleClass)
>>> mock.sync_foo
<Mock name='mock.sync_foo' id='...'>
>>> mock.async_foo
<AsyncMock name='mock.async_foo' id='...'>
```

Adicionado na versão 3.8.

assert_awaited()

Assert that the mock was awaited at least once. Note that this is separate from the object having been called, the `await` keyword must be used:

```
>>> mock = AsyncMock()
>>> async def main(coroutine_mock):
...     await coroutine_mock
...
>>> coroutine_mock = mock()
>>> mock.called
True
>>> mock.assert_awaited()
Traceback (most recent call last):
...
AssertionError: Expected mock to have been awaited.
>>> asyncio.run(main(coroutine_mock))
>>> mock.assert_awaited()
```

assert_awaited_once()

Afirme que o mock foi aguardado exatamente uma vez.

```
>>> mock = AsyncMock()
>>> async def main():
...     await mock()
...
>>> asyncio.run(main())
>>> mock.assert_awaited_once()
>>> asyncio.run(main())
>>> mock.assert_awaited_once()
Traceback (most recent call last):
...
AssertionError: Expected mock to have been awaited once. Awaited 2 times.
```

assert_awaited_with(**args*, ***kwargs*)

Assert that the last await was with the specified arguments.

```
>>> mock = AsyncMock()
>>> async def main(*args, **kwargs):
...     await mock(*args, **kwargs)
...
>>> asyncio.run(main('foo', bar='bar'))
>>> mock.assert_awaited_with('foo', bar='bar')
>>> mock.assert_awaited_with('other')
Traceback (most recent call last):
...
```

(continua na próxima página)

(continuação da página anterior)

```
AssertionError: expected await not found.
Expected: mock('other')
Actual: mock('foo', bar='bar')
```

assert_awaited_once_with(**args*, ***kwargs*)

Assert that the mock was awaited exactly once and with the specified arguments.

```
>>> mock = AsyncMock()
>>> async def main(*args, **kwargs):
...     await mock(*args, **kwargs)
...
>>> asyncio.run(main('foo', bar='bar'))
>>> mock.assert_awaited_once_with('foo', bar='bar')
>>> asyncio.run(main('foo', bar='bar'))
>>> mock.assert_awaited_once_with('foo', bar='bar')
Traceback (most recent call last):
...
AssertionError: Expected mock to have been awaited once. Awaited 2 times.
```

assert_any_await(**args*, ***kwargs*)

Assert the mock has ever been awaited with the specified arguments.

```
>>> mock = AsyncMock()
>>> async def main(*args, **kwargs):
...     await mock(*args, **kwargs)
...
>>> asyncio.run(main('foo', bar='bar'))
>>> asyncio.run(main('hello'))
>>> mock.assert_any_await('foo', bar='bar')
>>> mock.assert_any_await('other')
Traceback (most recent call last):
...
AssertionError: mock('other') await not found
```

assert_has_awaits(*calls*, *any_order=False*)

Assert the mock has been awaited with the specified calls. The *await_args_list* list is checked for the awaits.

If *any_order* is false then the awaits must be sequential. There can be extra calls before or after the specified awaits.

If *any_order* is true then the awaits can be in any order, but they must all appear in *await_args_list*.

```
>>> mock = AsyncMock()
>>> async def main(*args, **kwargs):
...     await mock(*args, **kwargs)
...
>>> calls = [call("foo"), call("bar")]
>>> mock.assert_has_awaits(calls)
Traceback (most recent call last):
...
AssertionError: Awaits not found.
Expected: [call('foo'), call('bar')]
Actual: []
>>> asyncio.run(main('foo'))
>>> asyncio.run(main('bar'))
>>> mock.assert_has_awaits(calls)
```

assert_not_awaited()

Afirma que o mock nunca foi aguardado.

```
>>> mock = AsyncMock()
>>> mock.assert_not_awaited()
```

reset_mock(**args*, ***kwargs*)

See *Mock.reset_mock()*. Also sets *await_count* to 0, *await_args* to None, and clears the *await_args_list*.

await_count

An integer keeping track of how many times the mock object has been awaited.

```
>>> mock = AsyncMock()
>>> async def main():
...     await mock()
...
>>> asyncio.run(main())
>>> mock.await_count
1
>>> asyncio.run(main())
>>> mock.await_count
2
```

await_args

This is either None (if the mock hasn't been awaited), or the arguments that the mock was last awaited with. Functions the same as *Mock.call_args*.

```
>>> mock = AsyncMock()
>>> async def main(*args):
...     await mock(*args)
...
>>> mock.await_args
>>> asyncio.run(main('foo'))
>>> mock.await_args
call('foo')
>>> asyncio.run(main('bar'))
>>> mock.await_args
call('bar')
```

await_args_list

This is a list of all the awaits made to the mock object in sequence (so the length of the list is the number of times it has been awaited). Before any awaits have been made it is an empty list.

```
>>> mock = AsyncMock()
>>> async def main(*args):
...     await mock(*args)
...
>>> mock.await_args_list
[]
>>> asyncio.run(main('foo'))
>>> mock.await_args_list
[call('foo')]
>>> asyncio.run(main('bar'))
>>> mock.await_args_list
[call('foo'), call('bar')]
```

class unittest.mock.**ThreadingMock**(*spec=None*, *side_effect=None*, *return_value=DEFAULT*, *wraps=None*, *name=None*, *spec_set=None*, *unsafe=False*, ***, *timeout=UNSET*, ***kwargs*)

A version of *MagicMock* for multithreading tests. The *ThreadingMock* object provides extra methods to wait for a call to be invoked, rather than assert on it immediately.

The default timeout is specified by the `timeout` argument, or if unset by the *ThreadingMock.DEFAULT_TIMEOUT* attribute, which defaults to blocking (`None`).

You can configure the global default timeout by setting *ThreadingMock.DEFAULT_TIMEOUT*.

wait_until_called(***, *timeout=UNSET*)

Waits until the mock is called.

If a timeout was passed at the creation of the mock or if a timeout argument is passed to this function, the function raises an *AssertionError* if the call is not performed in time.

```
>>> mock = ThreadingMock()
>>> thread = threading.Thread(target=mock)
>>> thread.start()
>>> mock.wait_until_called(timeout=1)
>>> thread.join()
```

wait_until_any_call_with(**args*, ***kwargs*)

Waits until the mock is called with the specified arguments.

If a timeout was passed at the creation of the mock the function raises an *AssertionError* if the call is not performed in time.

```
>>> mock = ThreadingMock()
>>> thread = threading.Thread(target=mock, args=("arg1", "arg2",), kwargs={
↪"arg": "thing"})
>>> thread.start()
>>> mock.wait_until_any_call_with("arg1", "arg2", arg="thing")
>>> thread.join()
```

DEFAULT_TIMEOUT

Global default timeout in seconds to create instances of *ThreadingMock*.

Adicionado na versão 3.13.

Fazendo chamadas

Mock objects are callable. The call will return the value set as the *return_value* attribute. The default return value is a new Mock object; it is created the first time the return value is accessed (either explicitly or by calling the Mock) - but it is stored and the same one returned each time.

Calls made to the object will be recorded in the attributes like *call_args* and *call_args_list*.

If *side_effect* is set then it will be called after the call has been recorded, so if `side_effect` raises an exception the call is still recorded.

The simplest way to make a mock raise an exception when called is to make *side_effect* an exception class or instance:

```
>>> m = MagicMock(side_effect=IndexError)
>>> m(1, 2, 3)
Traceback (most recent call last):
  ...
IndexError
>>> m.mock_calls
[call(1, 2, 3)]
```

(continua na próxima página)

(continuação da página anterior)

```
>>> m.side_effect = KeyError('Bang!')
>>> m('two', 'three', 'four')
Traceback (most recent call last):
  ...
KeyError: 'Bang!'
>>> m.mock_calls
[call(1, 2, 3), call('two', 'three', 'four')]
```

If *side_effect* is a function then whatever that function returns is what calls to the mock return. The `side_effect` function is called with the same arguments as the mock. This allows you to vary the return value of the call dynamically, based on the input:

```
>>> def side_effect(value):
...     return value + 1
...
>>> m = MagicMock(side_effect=side_effect)
>>> m(1)
2
>>> m(2)
3
>>> m.mock_calls
[call(1), call(2)]
```

If you want the mock to still return the default return value (a new mock), or any set return value, then there are two ways of doing this. Either return *return_value* from inside *side_effect*, or return *DEFAULT*:

```
>>> m = MagicMock()
>>> def side_effect(*args, **kwargs):
...     return m.return_value
...
>>> m.side_effect = side_effect
>>> m.return_value = 3
>>> m()
3
>>> def side_effect(*args, **kwargs):
...     return DEFAULT
...
>>> m.side_effect = side_effect
>>> m()
3
```

To remove a *side_effect*, and return to the default behaviour, set the `side_effect` to `None`:

```
>>> m = MagicMock(return_value=6)
>>> def side_effect(*args, **kwargs):
...     return 3
...
>>> m.side_effect = side_effect
>>> m()
3
>>> m.side_effect = None
>>> m()
6
```

The *side_effect* can also be any iterable object. Repeated calls to the mock will return values from the iterable (until the iterable is exhausted and a *StopIteration* is raised):

```
>>> m = MagicMock(side_effect=[1, 2, 3])
>>> m()
1
>>> m()
2
>>> m()
3
>>> m()
Traceback (most recent call last):
  ...
StopIteration
```

If any members of the iterable are exceptions they will be raised instead of returned:

```
>>> iterable = (33, ValueError, 66)
>>> m = MagicMock(side_effect=iterable)
>>> m()
33
>>> m()
Traceback (most recent call last):
 ...
ValueError
>>> m()
66
```

Deletando Atributos

Mock objects create attributes on demand. This allows them to pretend to be objects of any type.

You may want a mock object to return `False` to a *`hasattr()`* call, or raise an *`AttributeError`* when an attribute is fetched. You can do this by providing an object as a `spec` for a mock, but that isn't always convenient.

You "block" attributes by deleting them. Once deleted, accessing an attribute will raise an *`AttributeError`*.

```
>>> mock = MagicMock()
>>> hasattr(mock, 'm')
True
>>> del mock.m
>>> hasattr(mock, 'm')
False
>>> del mock.f
>>> mock.f
Traceback (most recent call last):
    ...
AttributeError: f
```

Nomes de Mock e o atributo *name*

Since "name" is an argument to the *`Mock`* constructor, if you want your mock object to have a "name" attribute you can't just pass it in at creation time. There are two alternatives. One option is to use *`configure_mock()`*:

```
>>> mock = MagicMock()
>>> mock.configure_mock(name='my_name')
>>> mock.name
'my_name'
```

A simpler option is to simply set the "name" attribute after mock creation:

```
>>> mock = MagicMock()
>>> mock.name = "foo"
```

Anexando Mocks como Atributos

When you attach a mock as an attribute of another mock (or as the return value) it becomes a "child" of that mock. Calls to the child are recorded in the *method_calls* and *mock_calls* attributes of the parent. This is useful for configuring child mocks and then attaching them to the parent, or for attaching mocks to a parent that records all calls to the children and allows you to make assertions about the order of calls between mocks:

```
>>> parent = MagicMock()
>>> child1 = MagicMock(return_value=None)
>>> child2 = MagicMock(return_value=None)
>>> parent.child1 = child1
>>> parent.child2 = child2
>>> child1(1)
>>> child2(2)
>>> parent.mock_calls
[call.child1(1), call.child2(2)]
```

The exception to this is if the mock has a name. This allows you to prevent the "parenting" if for some reason you don't want it to happen.

```
>>> mock = MagicMock()
>>> not_a_child = MagicMock(name='not-a-child')
>>> mock.attribute = not_a_child
>>> mock.attribute()
<MagicMock name='not-a-child()' id='...'>
>>> mock.mock_calls
[]
```

Mocks created for you by *patch()* are automatically given names. To attach mocks that have names to a parent you use the *attach_mock()* method:

```
>>> thing1 = object()
>>> thing2 = object()
>>> parent = MagicMock()
>>> with patch('__main__.thing1', return_value=None) as child1:
...     with patch('__main__.thing2', return_value=None) as child2:
...         parent.attach_mock(child1, 'child1')
...         parent.attach_mock(child2, 'child2')
...         child1('one')
...         child2('two')
...
>>> parent.mock_calls
[call.child1('one'), call.child2('two')]
```

26.6.3 Os criadores de patches

The patch decorators are used for patching objects only within the scope of the function they decorate. They automatically handle the unpatching for you, even if exceptions are raised. All of these functions can also be used in with statements or as class decorators.

patch

> **Nota**
>
> The key is to do the patching in the right namespace. See the section *where to patch*.

unittest.mock.**patch**(*target*, *new=DEFAULT*, *spec=None*, *create=False*, *spec_set=None*, *autospec=None*, *new_callable=None*, ***kwargs*)

`patch()` acts as a function decorator, class decorator or a context manager. Inside the body of the function or with statement, the *target* is patched with a *new* object. When the function/with statement exits the patch is undone.

If *new* is omitted, then the target is replaced with an `AsyncMock` if the patched object is an async function or a `MagicMock` otherwise. If `patch()` is used as a decorator and *new* is omitted, the created mock is passed in as an extra argument to the decorated function. If `patch()` is used as a context manager the created mock is returned by the context manager.

target should be a string in the form `'package.module.ClassName'`. The *target* is imported and the specified object replaced with the *new* object, so the *target* must be importable from the environment you are calling `patch()` from. The target is imported when the decorated function is executed, not at decoration time.

The *spec* and *spec_set* keyword arguments are passed to the `MagicMock` if patch is creating one for you.

In addition you can pass `spec=True` or `spec_set=True`, which causes patch to pass in the object being mocked as the spec/spec_set object.

new_callable allows you to specify a different class, or callable object, that will be called to create the *new* object. By default `AsyncMock` is used for async functions and `MagicMock` for the rest.

A more powerful form of *spec* is *autospec*. If you set `autospec=True` then the mock will be created with a spec from the object being replaced. All attributes of the mock will also have the spec of the corresponding attribute of the object being replaced. Methods and functions being mocked will have their arguments checked and will raise a `TypeError` if they are called with the wrong signature. For mocks replacing a class, their return value (the 'instance') will have the same spec as the class. See the `create_autospec()` function and *Especificação automática*.

Instead of `autospec=True` you can pass `autospec=some_object` to use an arbitrary object as the spec instead of the one being replaced.

By default `patch()` will fail to replace attributes that don't exist. If you pass in `create=True`, and the attribute doesn't exist, patch will create the attribute for you when the patched function is called, and delete it again after the patched function has exited. This is useful for writing tests against attributes that your production code creates at runtime. It is off by default because it can be dangerous. With it switched on you can write passing tests against APIs that don't actually exist!

> **Nota**
>
> Alterado na versão 3.5: If you are patching builtins in a module then you don't need to pass `create=True`, it will be added by default.

Patch can be used as a `TestCase` class decorator. It works by decorating each test method in the class. This reduces the boilerplate code when your test methods share a common patchings set. `patch()` finds tests by looking for method names that start with `patch.TEST_PREFIX`. By default this is `'test'`, which matches the way `unittest` finds tests. You can specify an alternative prefix by setting `patch.TEST_PREFIX`.

Patch can be used as a context manager, with the with statement. Here the patching applies to the indented block after the with statement. If you use "as" then the patched object will be bound to the name after the "as"; very useful if `patch()` is creating a mock object for you.

`patch()` takes arbitrary keyword arguments. These will be passed to `AsyncMock` if the patched object is asynchronous, to `MagicMock` otherwise or to *new_callable* if specified.

`patch.dict(...)`, `patch.multiple(...)` e `patch.object(...)` estão disponíveis para casos de uso alternativos.

patch() as function decorator, creating the mock for you and passing it into the decorated function:

```
>>> @patch('__main__.SomeClass')
... def function(normal_argument, mock_class):
...     print(mock_class is SomeClass)
...
>>> function(None)
True
```

Patching a class replaces the class with a *MagicMock* *instance*. If the class is instantiated in the code under test then it will be the *return_value* of the mock that will be used.

If the class is instantiated multiple times you could use *side_effect* to return a new mock each time. Alternatively you can set the *return_value* to be anything you want.

To configure return values on methods of *instances* on the patched class you must do this on the *return_value*. For example:

```
>>> class Class:
...     def method(self):
...         pass
...
>>> with patch('__main__.Class') as MockClass:
...     instance = MockClass.return_value
...     instance.method.return_value = 'foo'
...     assert Class() is instance
...     assert Class().method() == 'foo'
...
```

If you use *spec* or *spec_set* and *patch()* is replacing a *class*, then the return value of the created mock will have the same spec.

```
>>> Original = Class
>>> patcher = patch('__main__.Class', spec=True)
>>> MockClass = patcher.start()
>>> instance = MockClass()
>>> assert isinstance(instance, Original)
>>> patcher.stop()
```

The *new_callable* argument is useful where you want to use an alternative class to the default *MagicMock* for the created mock. For example, if you wanted a *NonCallableMock* to be used:

```
>>> thing = object()
>>> with patch('__main__.thing', new_callable=NonCallableMock) as mock_thing:
...     assert thing is mock_thing
...     thing()
...
Traceback (most recent call last):
  ...
TypeError: 'NonCallableMock' object is not callable
```

Another use case might be to replace an object with an *io.StringIO* instance:

```
>>> from io import StringIO
>>> def foo():
...     print('Something')
...
```

(continua na próxima página)

(continuação da página anterior)

```
>>> @patch('sys.stdout', new_callable=StringIO)
... def test(mock_stdout):
...     foo()
...     assert mock_stdout.getvalue() == 'Something\n'
...
>>> test()
```

When *patch()* is creating a mock for you, it is common that the first thing you need to do is to configure the mock. Some of that configuration can be done in the call to patch. Any arbitrary keywords you pass into the call will be used to set attributes on the created mock:

```
>>> patcher = patch('__main__.thing', first='one', second='two')
>>> mock_thing = patcher.start()
>>> mock_thing.first
'one'
>>> mock_thing.second
'two'
```

As well as attributes on the created mock attributes, like the *return_value* and *side_effect*, of child mocks can also be configured. These aren't syntactically valid to pass in directly as keyword arguments, but a dictionary with these as keys can still be expanded into a *patch()* call using **:

```
>>> config = {'method.return_value': 3, 'other.side_effect': KeyError}
>>> patcher = patch('__main__.thing', **config)
>>> mock_thing = patcher.start()
>>> mock_thing.method()
3
>>> mock_thing.other()
Traceback (most recent call last):
  ...
KeyError
```

By default, attempting to patch a function in a module (or a method or an attribute in a class) that does not exist will fail with *AttributeError*:

```
>>> @patch('sys.non_existing_attribute', 42)
... def test():
...     assert sys.non_existing_attribute == 42
...
>>> test()
Traceback (most recent call last):
  ...
AttributeError: <module 'sys' (built-in)> does not have the attribute 'non_
↪existing_attribute'
```

but adding `create=True` in the call to *patch()* will make the previous example work as expected:

```
>>> @patch('sys.non_existing_attribute', 42, create=True)
... def test(mock_stdout):
...     assert sys.non_existing_attribute == 42
...
>>> test()
```

Alterado na versão 3.8: *patch()* now returns an *AsyncMock* if the target is an async function.

patch.object

patch.**object**(*target*, *attribute*, *new=DEFAULT*, *spec=None*, *create=False*, *spec_set=None*, *autospec=None*, *new_callable=None*, ***kwargs*)

patch the named member (*attribute*) on an object (*target*) with a mock object.

`patch.object()` can be used as a decorator, class decorator or a context manager. Arguments *new*, *spec*, *create*, *spec_set*, *autospec* and *new_callable* have the same meaning as for `patch()`. Like `patch()`, `patch.object()` takes arbitrary keyword arguments for configuring the mock object it creates.

When used as a class decorator `patch.object()` honours `patch.TEST_PREFIX` for choosing which methods to wrap.

You can either call `patch.object()` with three arguments or two arguments. The three argument form takes the object to be patched, the attribute name and the object to replace the attribute with.

When calling with the two argument form you omit the replacement object, and a mock is created for you and passed in as an extra argument to the decorated function:

```
>>> @patch.object(SomeClass, 'class_method')
... def test(mock_method):
...     SomeClass.class_method(3)
...     mock_method.assert_called_with(3)
...
>>> test()
```

spec, *create* and the other arguments to `patch.object()` have the same meaning as they do for `patch()`.

patch.dict

patch.**dict**(*in_dict*, *values=()*, *clear=False*, ***kwargs*)

Patch a dictionary, or dictionary like object, and restore the dictionary to its original state after the test, where the restored dictionary is a copy of the dictionary as it was before the test.

in_dict can be a dictionary or a mapping like container. If it is a mapping then it must at least support getting, setting and deleting items plus iterating over keys.

in_dict can also be a string specifying the name of the dictionary, which will then be fetched by importing it.

values can be a dictionary of values to set in the dictionary. *values* can also be an iterable of `(key, value)` pairs.

If *clear* is true then the dictionary will be cleared before the new values are set.

`patch.dict()` can also be called with arbitrary keyword arguments to set values in the dictionary.

Alterado na versão 3.8: `patch.dict()` now returns the patched dictionary when used as a context manager.

`patch.dict()` can be used as a context manager, decorator or class decorator:

```
>>> foo = {}
>>> @patch.dict(foo, {'newkey': 'newvalue'})
... def test():
...     assert foo == {'newkey': 'newvalue'}
...
>>> test()
>>> assert foo == {}
```

When used as a class decorator `patch.dict()` honours `patch.TEST_PREFIX` (default to `'test'`) for choosing which methods to wrap:

```
>>> import os
>>> import unittest
>>> from unittest.mock import patch
```

(continua na próxima página)

(continuação da página anterior)

```
>>> @patch.dict('os.environ', {'newkey': 'newvalue'})
... class TestSample(unittest.TestCase):
...     def test_sample(self):
...         self.assertEqual(os.environ['newkey'], 'newvalue')
```

If you want to use a different prefix for your test, you can inform the patchers of the different prefix by setting `patch.TEST_PREFIX`. For more details about how to change the value of see *TEST_PREFIX*.

patch.dict() can be used to add members to a dictionary, or simply let a test change a dictionary, and ensure the dictionary is restored when the test ends.

```
>>> foo = {}
>>> with patch.dict(foo, {'newkey': 'newvalue'}) as patched_foo:
...     assert foo == {'newkey': 'newvalue'}
...     assert patched_foo == {'newkey': 'newvalue'}
...     # You can add, update or delete keys of foo (or patched_foo, it's the same␣
↪dict)
...     patched_foo['spam'] = 'eggs'
...
>>> assert foo == {}
>>> assert patched_foo == {}
```

```
>>> import os
>>> with patch.dict('os.environ', {'newkey': 'newvalue'}):
...     print(os.environ['newkey'])
...
newvalue
>>> assert 'newkey' not in os.environ
```

Keywords can be used in the *patch.dict()* call to set values in the dictionary:

```
>>> mymodule = MagicMock()
>>> mymodule.function.return_value = 'fish'
>>> with patch.dict('sys.modules', mymodule=mymodule):
...     import mymodule
...     mymodule.function('some', 'args')
...
'fish'
```

patch.dict() can be used with dictionary like objects that aren't actually dictionaries. At the very minimum they must support item getting, setting, deleting and either iteration or membership test. This corresponds to the magic methods `__getitem__()`, `__setitem__()`, `__delitem__()` and either *__iter__()* or `__contains__()`.

```
>>> class Container:
...     def __init__(self):
...         self.values = {}
...     def __getitem__(self, name):
...         return self.values[name]
...     def __setitem__(self, name, value):
...         self.values[name] = value
...     def __delitem__(self, name):
...         del self.values[name]
...     def __iter__(self):
...         return iter(self.values)
...
>>> thing = Container()
>>> thing['one'] = 1
```

(continua na próxima página)

(continuação da página anterior)

```
>>> with patch.dict(thing, one=2, two=3):
...     assert thing['one'] == 2
...     assert thing['two'] == 3
...
>>> assert thing['one'] == 1
>>> assert list(thing) == ['one']
```

patch.multiple

patch.**multiple**(*target*, *spec=None*, *create=False*, *spec_set=None*, *autospec=None*, *new_callable=None*, ***kwargs*)

Perform multiple patches in a single call. It takes the object to be patched (either as an object or a string to fetch the object by importing) and keyword arguments for the patches:

```
with patch.multiple(settings, FIRST_PATCH='one', SECOND_PATCH='two'):
    ...
```

Use *DEFAULT* as the value if you want *patch.multiple()* to create mocks for you. In this case the created mocks are passed into a decorated function by keyword, and a dictionary is returned when *patch.multiple()* is used as a context manager.

patch.multiple() can be used as a decorator, class decorator or a context manager. The arguments *spec*, *spec_set*, *create*, *autospec* and *new_callable* have the same meaning as for *patch()*. These arguments will be applied to *all* patches done by *patch.multiple()*.

When used as a class decorator *patch.multiple()* honours `patch.TEST_PREFIX` for choosing which methods to wrap.

If you want *patch.multiple()* to create mocks for you, then you can use *DEFAULT* as the value. If you use *patch.multiple()* as a decorator then the created mocks are passed into the decorated function by keyword.

```
>>> thing = object()
>>> other = object()

>>> @patch.multiple('__main__', thing=DEFAULT, other=DEFAULT)
... def test_function(thing, other):
...     assert isinstance(thing, MagicMock)
...     assert isinstance(other, MagicMock)
...
>>> test_function()
```

patch.multiple() can be nested with other `patch` decorators, but put arguments passed by keyword *after* any of the standard arguments created by *patch()*:

```
>>> @patch('sys.exit')
... @patch.multiple('__main__', thing=DEFAULT, other=DEFAULT)
... def test_function(mock_exit, other, thing):
...     assert 'other' in repr(other)
...     assert 'thing' in repr(thing)
...     assert 'exit' in repr(mock_exit)
...
>>> test_function()
```

If *patch.multiple()* is used as a context manager, the value returned by the context manager is a dictionary where created mocks are keyed by name:

```
>>> with patch.multiple('__main__', thing=DEFAULT, other=DEFAULT) as values:
...     assert 'other' in repr(values['other'])
```

(continua na próxima página)

(continuação da página anterior)

```
...     assert 'thing' in repr(values['thing'])
...     assert values['thing'] is thing
...     assert values['other'] is other
...
```

métodos do patch: *start* e *stop*

All the patchers have `start()` and `stop()` methods. These make it simpler to do patching in `setUp` methods or where you want to do multiple patches without nesting decorators or with statements.

To use them call *patch()*, *patch.object()* or *patch.dict()* as normal and keep a reference to the returned `patcher` object. You can then call `start()` to put the patch in place and `stop()` to undo it.

If you are using *patch()* to create a mock for you then it will be returned by the call to `patcher.start`.

```
>>> patcher = patch('package.module.ClassName')
>>> from package import module
>>> original = module.ClassName
>>> new_mock = patcher.start()
>>> assert module.ClassName is not original
>>> assert module.ClassName is new_mock
>>> patcher.stop()
>>> assert module.ClassName is original
>>> assert module.ClassName is not new_mock
```

A typical use case for this might be for doing multiple patches in the `setUp` method of a *TestCase*:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         self.patcher1 = patch('package.module.Class1')
...         self.patcher2 = patch('package.module.Class2')
...         self.MockClass1 = self.patcher1.start()
...         self.MockClass2 = self.patcher2.start()
...
...     def tearDown(self):
...         self.patcher1.stop()
...         self.patcher2.stop()
...
...     def test_something(self):
...         assert package.module.Class1 is self.MockClass1
...         assert package.module.Class2 is self.MockClass2
...
>>> MyTest('test_something').run()
```

Cuidado

If you use this technique you must ensure that the patching is "undone" by calling `stop`. This can be fiddlier than you might think, because if an exception is raised in the `setUp` then `tearDown` is not called. *unittest.TestCase.addCleanup()* makes this easier:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         patcher = patch('package.module.Class')
...         self.MockClass = patcher.start()
...         self.addCleanup(patcher.stop)
...
...     def test_something(self):
...         assert package.module.Class is self.MockClass
...
```

As an added bonus you no longer need to keep a reference to the `patcher` object.

It is also possible to stop all patches which have been started by using *patch.stopall()*.

patch.**stopall**()

Stop all active patches. Only stops patches started with `start`.

patch de embutidos

You can patch any builtins within a module. The following example patches builtin *ord()*:

```
>>> @patch('__main__.ord')
... def test(mock_ord):
...     mock_ord.return_value = 101
...     print(ord('c'))
...
>>> test()
101
```

TEST_PREFIX

All of the patchers can be used as class decorators. When used in this way they wrap every test method on the class. The patchers recognise methods that start with `'test'` as being test methods. This is the same way that the *unittest.TestLoader* finds test methods by default.

It is possible that you want to use a different prefix for your tests. You can inform the patchers of the different prefix by setting `patch.TEST_PREFIX`:

```
>>> patch.TEST_PREFIX = 'foo'
>>> value = 3
>>>
>>> @patch('__main__.value', 'not three')
... class Thing:
...     def foo_one(self):
...         print(value)
...     def foo_two(self):
...         print(value)
...
>>>
>>> Thing().foo_one()
not three
>>> Thing().foo_two()
not three
>>> value
3
```

Aninhando Decoradores Patch

If you want to perform multiple patches then you can simply stack up the decorators.

You can stack up multiple patch decorators using this pattern:

```
>>> @patch.object(SomeClass, 'class_method')
... @patch.object(SomeClass, 'static_method')
... def test(mock1, mock2):
...     assert SomeClass.static_method is mock1
...     assert SomeClass.class_method is mock2
...     SomeClass.static_method('foo')
```

(continua na próxima página)

(continuação da página anterior)

```
...     SomeClass.class_method('bar')
...     return mock1, mock2
...
>>> mock1, mock2 = test()
>>> mock1.assert_called_once_with('foo')
>>> mock2.assert_called_once_with('bar')
```

Note that the decorators are applied from the bottom upwards. This is the standard way that Python applies decorators. The order of the created mocks passed into your test function matches this order.

Onde fazer patch

patch() works by (temporarily) changing the object that a *name* points to with another one. There can be many names pointing to any individual object, so for patching to work you must ensure that you patch the name used by the system under test.

The basic principle is that you patch where an object is *looked up*, which is not necessarily the same place as where it is defined. A couple of examples will help to clarify this.

Imagine we have a project that we want to test with the following structure:

```
a.py
    -> Defines SomeClass

b.py
    -> from a import SomeClass
    -> some_function instantiates SomeClass
```

Now we want to test `some_function` but we want to mock out `SomeClass` using *patch()*. The problem is that when we import module b, which we will have to do when it imports `SomeClass` from module a. If we use *patch()* to mock out `a.SomeClass` then it will have no effect on our test; module b already has a reference to the *real* `SomeClass` and it looks like our patching had no effect.

The key is to patch out `SomeClass` where it is used (or where it is looked up). In this case `some_function` will actually look up `SomeClass` in module b, where we have imported it. The patching should look like:

```
@patch('b.SomeClass')
```

However, consider the alternative scenario where instead of `from a import SomeClass` module b does `import a` and `some_function` uses `a.SomeClass`. Both of these import forms are common. In this case the class we want to patch is being looked up in the module and so we have to patch `a.SomeClass` instead:

```
@patch('a.SomeClass')
```

Patching Descriptors and Proxy Objects

Both *patch* and *patch.object* correctly patch and restore descriptors: class methods, static methods and properties. You should patch these on the *class* rather than an instance. They also work with *some* objects that proxy attribute access, like the django settings object.

26.6.4 MagicMock and magic method support

Simulando Métodos Mágicos

Mock supports mocking the Python protocol methods, also known as *"magic methods"*. This allows mock objects to replace containers or other objects that implement Python protocols.

Because magic methods are looked up differently from normal methods[2], this support has been specially implemented.

[2] Magic methods *should* be looked up on the class rather than the instance. Different versions of Python are inconsistent about applying this rule. The supported protocol methods should work with all supported versions of Python.

This means that only specific magic methods are supported. The supported list includes *almost* all of them. If there are any missing that you need please let us know.

You mock magic methods by setting the method you are interested in to a function or a mock instance. If you are using a function then it *must* take `self` as the first argument[3].

```
>>> def __str__(self):
...     return 'fooble'
...
>>> mock = Mock()
>>> mock.__str__ = __str__
>>> str(mock)
'fooble'
```

```
>>> mock = Mock()
>>> mock.__str__ = Mock()
>>> mock.__str__.return_value = 'fooble'
>>> str(mock)
'fooble'
```

```
>>> mock = Mock()
>>> mock.__iter__ = Mock(return_value=iter([]))
>>> list(mock)
[]
```

One use case for this is for mocking objects used as context managers in a `with` statement:

```
>>> mock = Mock()
>>> mock.__enter__ = Mock(return_value='foo')
>>> mock.__exit__ = Mock(return_value=False)
>>> with mock as m:
...     assert m == 'foo'
...
>>> mock.__enter__.assert_called_with()
>>> mock.__exit__.assert_called_with(None, None, None)
```

Calls to magic methods do not appear in *method_calls*, but they are recorded in *mock_calls*.

Nota

If you use the *spec* keyword argument to create a mock then attempting to set a magic method that isn't in the spec will raise an *AttributeError*.

A lista completa de métodos mágicos compatíveis é:

- `__hash__`, `__sizeof__`, `__repr__` e `__str__`
- `__dir__`, `__format__` e `__subclasses__`
- `__round__`, `__floor__`, `__trunc__` e `__ceil__`
- Comparações: `__lt__`, `__gt__`, `__le__`, `__ge__`, `__eq__` e `__ne__`
- Container methods: `__getitem__`, `__setitem__`, `__delitem__`, `__contains__`, `__len__`, `__iter__`, `__reversed__` and `__missing__`
- Gerenciador de contexto: `__enter__`, `__exit__`, `__aenter__` e `__aexit__`
- Métodos numéricos unários: `__neg__`, `__pos__` e `__invert__`

[3] The function is basically hooked up to the class, but each `Mock` instance is kept isolated from the others.

- The numeric methods (including right hand and in-place variants): `__add__`, `__sub__`, `__mul__`, `__matmul__`, `__truediv__`, `__floordiv__`, `__mod__`, `__divmod__`, `__lshift__`, `__rshift__`, `__and__`, `__xor__`, `__or__`, and `__pow__`
- Métodos de conversão numérica: `__complex__`, `__int__`, `__float__` e `__index__`
- Métodos descritores: `__get__`, `__set__` e `__delete__`
- Pickling: `__reduce__`, `__reduce_ex__`, `__getinitargs__`, `__getnewargs__`, `__getstate__` e `__setstate__`
- File system path representation: `__fspath__`
- Métodos de iteração assíncrona: `__aiter__` e `__anext__`

Alterado na versão 3.8: Adicionado suporte para *os.PathLike.__fspath__()*.

Alterado na versão 3.8: Adicionado suporte para `__aenter__`, `__aexit__`, `__aiter__` e `__anext__`.

The following methods exist but are *not* supported as they are either in use by mock, can't be set dynamically, or can cause problems:

- `__getattr__`, `__setattr__`, `__init__` e `__new__`
- `__prepare__`, `__instancecheck__`, `__subclasscheck__`, `__del__`

Magic Mock

Existem duas variantes de `MagicMock`: *MagicMock* e *NonCallableMagicMock*.

class unittest.mock.**MagicMock**(**args*, ***kw*)

`MagicMock` is a subclass of *Mock* with default implementations of most of the *magic methods*. You can use `MagicMock` without having to configure the magic methods yourself.

The constructor parameters have the same meaning as for *Mock*.

If you use the *spec* or *spec_set* arguments then *only* magic methods that exist in the spec will be created.

class unittest.mock.**NonCallableMagicMock**(**args*, ***kw*)

Uma versão não-chamável de *MagicMock*.

The constructor parameters have the same meaning as for *MagicMock*, with the exception of *return_value* and *side_effect* which have no meaning on a non-callable mock.

The magic methods are setup with *MagicMock* objects, so you can configure them and use them in the usual way:

```
>>> mock = MagicMock()
>>> mock[3] = 'fish'
>>> mock.__setitem__.assert_called_with(3, 'fish')
>>> mock.__getitem__.return_value = 'result'
>>> mock[2]
'result'
```

By default many of the protocol methods are required to return objects of a specific type. These methods are preconfigured with a default return value, so that they can be used without you having to do anything if you aren't interested in the return value. You can still *set* the return value manually if you want to change the default.

Métodos e seus padrões:

- `__lt__`: *NotImplemented*
- `__gt__`: `NotImplemented`
- `__le__`: `NotImplemented`
- `__ge__`: `NotImplemented`
- `__int__`: `1`
- `__contains__`: `False`

- `__len__`: `0`
- `__iter__`: `iter([])`
- `__exit__`: `False`
- `__aexit__`: `False`
- `__complex__`: `1j`
- `__float__`: `1.0`
- `__bool__`: `True`
- `__index__`: `1`
- `__hash__`: hash padrão para o mock
- `__str__`: *str* padrão para o mock
- `__sizeof__`: *sizeof* padrão para o mock

Por exemplo:

```
>>> mock = MagicMock()
>>> int(mock)
1
>>> len(mock)
0
>>> list(mock)
[]
>>> object() in mock
False
```

The two equality methods, `__eq__()` and `__ne__()`, are special. They do the default equality comparison on identity, using the *side_effect* attribute, unless you change their return value to return something else:

```
>>> MagicMock() == 3
False
>>> MagicMock() != 3
True
>>> mock = MagicMock()
>>> mock.__eq__.return_value = True
>>> mock == 3
True
```

The return value of `MagicMock.__iter__()` can be any iterable object and isn't required to be an iterator:

```
>>> mock = MagicMock()
>>> mock.__iter__.return_value = ['a', 'b', 'c']
>>> list(mock)
['a', 'b', 'c']
>>> list(mock)
['a', 'b', 'c']
```

If the return value *is* an iterator, then iterating over it once will consume it and subsequent iterations will result in an empty list:

```
>>> mock.__iter__.return_value = iter(['a', 'b', 'c'])
>>> list(mock)
['a', 'b', 'c']
>>> list(mock)
[]
```

`MagicMock` has all of the supported magic methods configured except for some of the obscure and obsolete ones. You can still set these up if you want.

Magic methods that are supported but not setup by default in `MagicMock` are:

- `__subclasses__`
- `__dir__`
- `__format__`
- `__get__`, `__set__` e `__delete__`
- `__reversed__` e `__missing__`
- `__reduce__`, `__reduce_ex__`, `__getinitargs__`, `__getnewargs__`, `__getstate__` e `__setstate__`
- `__getformat__`

26.6.5 Ajudantes

sentinel

`unittest.mock.`**`sentinel`**

The `sentinel` object provides a convenient way of providing unique objects for your tests.

Attributes are created on demand when you access them by name. Accessing the same attribute will always return the same object. The objects returned have a sensible repr so that test failure messages are readable.

Alterado na versão 3.7: The `sentinel` attributes now preserve their identity when they are *`copied`* or *`pickled`*.

Sometimes when testing you need to test that a specific object is passed as an argument to another method, or returned. It can be common to create named sentinel objects to test this. *`sentinel`* provides a convenient way of creating and testing the identity of objects like this.

In this example we monkey patch `method` to return `sentinel.some_object`:

```
>>> real = ProductionClass()
>>> real.method = Mock(name="method")
>>> real.method.return_value = sentinel.some_object
>>> result = real.method()
>>> assert result is sentinel.some_object
>>> result
sentinel.some_object
```

DEFAULT

`unittest.mock.`**`DEFAULT`**

The *`DEFAULT`* object is a pre-created sentinel (actually `sentinel.DEFAULT`). It can be used by *`side_effect`* functions to indicate that the normal return value should be used.

chamada

`unittest.mock.`**`call`**(**args*, ***kwargs*)

`call()` is a helper object for making simpler assertions, for comparing with *`call_args`*, *`call_args_list`*, *`mock_calls`* and *`method_calls`*. *`call()`* can also be used with *`assert_has_calls()`*.

```
>>> m = MagicMock(return_value=None)
>>> m(1, 2, a='foo', b='bar')
>>> m()
>>> m.call_args_list == [call(1, 2, a='foo', b='bar'), call()]
True
```

call.**call_list**()

For a call object that represents multiple calls, *call_list()* returns a list of all the intermediate calls as well as the final call.

call_list is particularly useful for making assertions on "chained calls". A chained call is multiple calls on a single line of code. This results in multiple entries in *mock_calls* on a mock. Manually constructing the sequence of calls can be tedious.

call_list() can construct the sequence of calls from the same chained call:

```
>>> m = MagicMock()
>>> m(1).method(arg='foo').other('bar')(2.0)
<MagicMock name='mock().method().other()()' id='...'>
>>> kall = call(1).method(arg='foo').other('bar')(2.0)
>>> kall.call_list()
[call(1),
 call().method(arg='foo'),
 call().method().other('bar'),
 call().method().other()(2.0)]
>>> m.mock_calls == kall.call_list()
True
```

A call object is either a tuple of (positional args, keyword args) or (name, positional args, keyword args) depending on how it was constructed. When you construct them yourself this isn't particularly interesting, but the call objects that are in the *Mock.call_args*, *Mock.call_args_list* and *Mock.mock_calls* attributes can be introspected to get at the individual arguments they contain.

The call objects in *Mock.call_args* and *Mock.call_args_list* are two-tuples of (positional args, keyword args) whereas the call objects in *Mock.mock_calls*, along with ones you construct yourself, are three-tuples of (name, positional args, keyword args).

You can use their "tupleness" to pull out the individual arguments for more complex introspection and assertions. The positional arguments are a tuple (an empty tuple if there are no positional arguments) and the keyword arguments are a dictionary:

```
>>> m = MagicMock(return_value=None)
>>> m(1, 2, 3, arg='one', arg2='two')
>>> kall = m.call_args
>>> kall.args
(1, 2, 3)
>>> kall.kwargs
{'arg': 'one', 'arg2': 'two'}
>>> kall.args is kall[0]
True
>>> kall.kwargs is kall[1]
True
```

```
>>> m = MagicMock()
>>> m.foo(4, 5, 6, arg='two', arg2='three')
<MagicMock name='mock.foo()' id='...'>
>>> kall = m.mock_calls[0]
>>> name, args, kwargs = kall
>>> name
'foo'
>>> args
(4, 5, 6)
>>> kwargs
{'arg': 'two', 'arg2': 'three'}
>>> name is m.mock_calls[0][0]
True
```

create_autospec

unittest.mock.**create_autospec**(*spec*, *spec_set=False*, *instance=False*, ***kwargs*)

Create a mock object using another object as a spec. Attributes on the mock will use the corresponding attribute on the *spec* object as their spec.

Functions or methods being mocked will have their arguments checked to ensure that they are called with the correct signature.

If *spec_set* is `True` then attempting to set attributes that don't exist on the spec object will raise an *AttributeError*.

If a class is used as a spec then the return value of the mock (the instance of the class) will have the same spec. You can use a class as the spec for an instance object by passing `instance=True`. The returned mock will only be callable if instances of the mock are callable.

create_autospec() also takes arbitrary keyword arguments that are passed to the constructor of the created mock.

See *Especificação automática* for examples of how to use auto-speccing with *create_autospec()* and the *autospec* argument to *patch()*.

Alterado na versão 3.8: *create_autospec()* now returns an *AsyncMock* if the target is an async function.

ANY

unittest.mock.**ANY**

Sometimes you may need to make assertions about *some* of the arguments in a call to mock, but either not care about some of the arguments or want to pull them individually out of *call_args* and make more complex assertions on them.

To ignore certain arguments you can pass in objects that compare equal to *everything*. Calls to *assert_called_with()* and *assert_called_once_with()* will then succeed no matter what was passed in.

```
>>> mock = Mock(return_value=None)
>>> mock('foo', bar=object())
>>> mock.assert_called_once_with('foo', bar=ANY)
```

ANY can also be used in comparisons with call lists like *mock_calls*:

```
>>> m = MagicMock(return_value=None)
>>> m(1)
>>> m(1, 2)
>>> m(object())
>>> m.mock_calls == [call(1), call(1, 2), ANY]
True
```

ANY is not limited to comparisons with call objects and so can also be used in test assertions:

```
class TestStringMethods(unittest.TestCase):

    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', ANY])
```

FILTER_DIR

unittest.mock.**FILTER_DIR**

FILTER_DIR is a module level variable that controls the way mock objects respond to *dir()*. The default is `True`, which uses the filtering described below, to only show useful members. If you dislike this filtering, or need to switch it off for diagnostic purposes, then set `mock.FILTER_DIR = False`.

With filtering on, `dir(some_mock)` shows only useful attributes and will include any dynamically created attributes that wouldn't normally be shown. If the mock was created with a *spec* (or *autospec* of course) then all the attributes from the original are shown, even if they haven't been accessed yet:

```
>>> dir(Mock())
['assert_any_call',
 'assert_called',
 'assert_called_once',
 'assert_called_once_with',
 'assert_called_with',
 'assert_has_calls',
 'assert_not_called',
 'attach_mock',
 ...
>>> from urllib import request
>>> dir(Mock(spec=request))
['AbstractBasicAuthHandler',
 'AbstractDigestAuthHandler',
 'AbstractHTTPHandler',
 'BaseHandler',
 ...
```

Many of the not-very-useful (private to *Mock* rather than the thing being mocked) underscore and double underscore prefixed attributes have been filtered from the result of calling *dir()* on a *Mock*. If you dislike this behaviour you can switch it off by setting the module level switch *FILTER_DIR*:

```
>>> from unittest import mock
>>> mock.FILTER_DIR = False
>>> dir(mock.Mock())
['_NonCallableMock__get_return_value',
 '_NonCallableMock__get_side_effect',
 '_NonCallableMock__return_value_doc',
 '_NonCallableMock__set_return_value',
 '_NonCallableMock__set_side_effect',
 '__call__',
 '__class__',
 ...
```

Alternatively you can just use `vars(my_mock)` (instance members) and `dir(type(my_mock))` (type members) to bypass the filtering irrespective of *FILTER_DIR*.

mock_open

`unittest.mock.`**`mock_open`**(*mock=None*, *read_data=None*)

A helper function to create a mock to replace the use of *open()*. It works for *open()* called directly or used as a context manager.

The *mock* argument is the mock object to configure. If `None` (the default) then a *MagicMock* will be created for you, with the API limited to methods or attributes available on standard file handles.

read_data is a string for the *read()*, *readline()*, and *readlines()* methods of the file handle to return. Calls to those methods will take data from *read_data* until it is depleted. The mock of these methods is pretty simplistic: every time the *mock* is called, the *read_data* is rewound to the start. If you need more control over the data that you are feeding to the tested code you will need to customize this mock for yourself. When that is insufficient, one of the in-memory filesystem packages on PyPI can offer a realistic filesystem for testing.

Alterado na versão 3.4: Added *readline()* and *readlines()* support. The mock of *read()* changed to consume *read_data* rather than returning it on each call.

Alterado na versão 3.5: *read_data* is now reset on each call to the *mock*.

Alterado na versão 3.8: Added `__iter__()` to implementation so that iteration (such as in for loops) correctly consumes *read_data*.

Using `open()` as a context manager is a great way to ensure your file handles are closed properly and is becoming common:

```
with open('/some/path', 'w') as f:
    f.write('something')
```

The issue is that even if you mock out the call to `open()` it is the *returned object* that is used as a context manager (and has `__enter__()` and `__exit__()` called).

Mocking context managers with a `MagicMock` is common enough and fiddly enough that a helper function is useful.

```
>>> m = mock_open()
>>> with patch('__main__.open', m):
...     with open('foo', 'w') as h:
...         h.write('some stuff')
...
>>> m.mock_calls
[call('foo', 'w'),
 call().__enter__(),
 call().write('some stuff'),
 call().__exit__(None, None, None)]
>>> m.assert_called_once_with('foo', 'w')
>>> handle = m()
>>> handle.write.assert_called_once_with('some stuff')
```

E para ler arquivos:

```
>>> with patch('__main__.open', mock_open(read_data='bibble')) as m:
...     with open('foo') as h:
...         result = h.read()
...
>>> m.assert_called_once_with('foo')
>>> assert result == 'bibble'
```

Especificação automática

Autospeccing is based on the existing `spec` feature of mock. It limits the api of mocks to the api of an original object (the spec), but it is recursive (implemented lazily) so that attributes of mocks only have the same api as the attributes of the spec. In addition mocked functions / methods have the same call signature as the original so they raise a `TypeError` if they are called incorrectly.

Before I explain how auto-speccing works, here's why it is needed.

`Mock` is a very powerful and flexible object, but it suffers from a flaw which is general to mocking. If you refactor some of your code, rename members and so on, any tests for code that is still using the *old api* but uses mocks instead of the real objects will still pass. This means your tests can all pass even though your code is broken.

Alterado na versão 3.5: Before 3.5, tests with a typo in the word assert would silently pass when they should raise an error. You can still achieve this behavior by passing `unsafe=True` to Mock.

Note that this is another reason why you need integration tests as well as unit tests. Testing everything in isolation is all fine and dandy, but if you don't test how your units are "wired together" there is still lots of room for bugs that tests might have caught.

`unittest.mock` already provides a feature to help with this, called speccing. If you use a class or instance as the `spec` for a mock then you can only access attributes on the mock that exist on the real class:

```
>>> from urllib import request
>>> mock = Mock(spec=request.Request)
>>> mock.assret_called_with  # Intentional typo!
Traceback (most recent call last):
  ...
AttributeError: Mock object has no attribute 'assret_called_with'
```

The spec only applies to the mock itself, so we still have the same issue with any methods on the mock:

```
>>> mock.header_items()
<mock.Mock object at 0x...>
>>> mock.header_items.assret_called_with()  # Intentional typo!
```

Auto-speccing solves this problem. You can either pass `autospec=True` to *patch()* / *patch.object()* or use the *create_autospec()* function to create a mock with a spec. If you use the `autospec=True` argument to *patch()* then the object that is being replaced will be used as the spec object. Because the speccing is done "lazily" (the spec is created as attributes on the mock are accessed) you can use it with very complex or deeply nested objects (like modules that import modules that import modules) without a big performance hit.

Aqui está um exemplo disso em uso:

```
>>> from urllib import request
>>> patcher = patch('__main__.request', autospec=True)
>>> mock_request = patcher.start()
>>> request is mock_request
True
>>> mock_request.Request
<MagicMock name='request.Request' spec='Request' id='...'>
```

You can see that `request.Request` has a spec. `request.Request` takes two arguments in the constructor (one of which is *self*). Here's what happens if we try to call it incorrectly:

```
>>> req = request.Request()
Traceback (most recent call last):
  ...
TypeError: <lambda>() takes at least 2 arguments (1 given)
```

The spec also applies to instantiated classes (i.e. the return value of specced mocks):

```
>>> req = request.Request('foo')
>>> req
<NonCallableMagicMock name='request.Request()' spec='Request' id='...'>
```

`Request` objects are not callable, so the return value of instantiating our mocked out `request.Request` is a non-callable mock. With the spec in place any typos in our asserts will raise the correct error:

```
>>> req.add_header('spam', 'eggs')
<MagicMock name='request.Request().add_header()' id='...'>
>>> req.add_header.assret_called_with  # Intentional typo!
Traceback (most recent call last):
  ...
AttributeError: Mock object has no attribute 'assret_called_with'
>>> req.add_header.assert_called_with('spam', 'eggs')
```

In many cases you will just be able to add `autospec=True` to your existing *patch()* calls and then be protected against bugs due to typos and api changes.

As well as using *autospec* through *patch()* there is a *create_autospec()* for creating autospecced mocks directly:

```
>>> from urllib import request
>>> mock_request = create_autospec(request)
>>> mock_request.Request('foo', 'bar')
<NonCallableMagicMock name='mock.Request()' spec='Request' id='...'>
```

This isn't without caveats and limitations however, which is why it is not the default behaviour. In order to know what attributes are available on the spec object, autospec has to introspect (access attributes) the spec. As you traverse attributes on the mock a corresponding traversal of the original object is happening under the hood. If any of your specced objects have properties or descriptors that can trigger code execution then you may not be able to use autospec. On the other hand it is much better to design your objects so that introspection is safe[4].

A more serious problem is that it is common for instance attributes to be created in the `__init__()` method and not to exist on the class at all. *autospec* can't know about any dynamically created attributes and restricts the api to visible attributes.

```
>>> class Something:
...   def __init__(self):
...     self.a = 33
...
>>> with patch('__main__.Something', autospec=True):
...   thing = Something()
...   thing.a
...
Traceback (most recent call last):
  ...
AttributeError: Mock object has no attribute 'a'
```

There are a few different ways of resolving this problem. The easiest, but not necessarily the least annoying, way is to simply set the required attributes on the mock after creation. Just because *autospec* doesn't allow you to fetch attributes that don't exist on the spec it doesn't prevent you setting them:

```
>>> with patch('__main__.Something', autospec=True):
...   thing = Something()
...   thing.a = 33
...
```

There is a more aggressive version of both *spec* and *autospec* that *does* prevent you setting non-existent attributes. This is useful if you want to ensure your code only *sets* valid attributes too, but obviously it prevents this particular scenario:

```
>>> with patch('__main__.Something', autospec=True, spec_set=True):
...   thing = Something()
...   thing.a = 33
...
Traceback (most recent call last):
  ...
AttributeError: Mock object has no attribute 'a'
```

Probably the best way of solving the problem is to add class attributes as default values for instance members initialised in `__init__()`. Note that if you are only setting default attributes in `__init__()` then providing them via class attributes (shared between instances of course) is faster too. e.g.

```
class Something:
    a = 33
```

This brings up another issue. It is relatively common to provide a default value of `None` for members that will later be an object of a different type. `None` would be useless as a spec because it wouldn't let you access *any* attributes or

[4] This only applies to classes or already instantiated objects. Calling a mocked class to create a mock instance *does not* create a real instance. It is only attribute lookups - along with calls to *dir()* - that are done.

methods on it. As `None` is *never* going to be useful as a spec, and probably indicates a member that will normally of some other type, autospec doesn't use a spec for members that are set to `None`. These will just be ordinary mocks (well - MagicMocks):

```
>>> class Something:
...     member = None
...
>>> mock = create_autospec(Something)
>>> mock.member.foo.bar.baz()
<MagicMock name='mock.member.foo.bar.baz()' id='...'>
```

If modifying your production classes to add defaults isn't to your liking then there are more options. One of these is simply to use an instance as the spec rather than the class. The other is to create a subclass of the production class and add the defaults to the subclass without affecting the production class. Both of these require you to use an alternative object as the spec. Thankfully *patch()* supports this - you can simply pass the alternative object as the *autospec* argument:

```
>>> class Something:
...   def __init__(self):
...     self.a = 33
...
>>> class SomethingForTest(Something):
...   a = 33
...
>>> p = patch('__main__.Something', autospec=SomethingForTest)
>>> mock = p.start()
>>> mock.a
<NonCallableMagicMock name='Something.a' spec='int' id='...'>
```

Vedando mocks

unittest.mock.**seal**(*mock*)

Seal will disable the automatic creation of mocks when accessing an attribute of the mock being sealed or any of its attributes that are already mocks recursively.

If a mock instance with a name or a spec is assigned to an attribute it won't be considered in the sealing chain. This allows one to prevent seal from fixing part of the mock object.

```
>>> mock = Mock()
>>> mock.submock.attribute1 = 2
>>> mock.not_submock = mock.Mock(name="sample_name")
>>> seal(mock)
>>> mock.new_attribute  # This will raise AttributeError.
>>> mock.submock.attribute2  # This will raise AttributeError.
>>> mock.not_submock.attribute2  # This won't raise.
```

Adicionado na versão 3.7.

26.6.6 Order of precedence of `side_effect`, `return_value` and *wraps*

The order of their precedence is:

1. *side_effect*
2. *return_value*
3. *wraps*

If all three are set, mock will return the value from *side_effect*, ignoring *return_value* and the wrapped object altogether. If any two are set, the one with the higher precedence will return the value. Regardless of the order of which was set first, the order of precedence remains unchanged.

```
>>> from unittest.mock import Mock
>>> class Order:
...     @staticmethod
...     def get_value():
...         return "third"
...
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.get_value.side_effect = ["first"]
>>> order_mock.get_value.return_value = "second"
>>> order_mock.get_value()
'first'
```

As `None` is the default value of *side_effect*, if you reassign its value back to `None`, the order of precedence will be checked between *return_value* and the wrapped object, ignoring *side_effect*.

```
>>> order_mock.get_value.side_effect = None
>>> order_mock.get_value()
'second'
```

If the value being returned by *side_effect* is *DEFAULT*, it is ignored and the order of precedence moves to the successor to obtain the value to return.

```
>>> from unittest.mock import DEFAULT
>>> order_mock.get_value.side_effect = [DEFAULT]
>>> order_mock.get_value()
'second'
```

When *Mock* wraps an object, the default value of *return_value* will be *DEFAULT*.

```
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.return_value
sentinel.DEFAULT
>>> order_mock.get_value.return_value
sentinel.DEFAULT
```

The order of precedence will ignore this value and it will move to the last successor which is the wrapped object.

As the real call is being made to the wrapped object, creating an instance of this mock will return the real instance of the class. The positional arguments, if any, required by the wrapped object must be passed.

```
>>> order_mock_instance = order_mock()
>>> isinstance(order_mock_instance, Order)
True
>>> order_mock_instance.get_value()
'third'
```

```
>>> order_mock.get_value.return_value = DEFAULT
>>> order_mock.get_value()
'third'
```

```
>>> order_mock.get_value.return_value = "second"
>>> order_mock.get_value()
'second'
```

But if you assign `None` to it, this will not be ignored as it is an explicit assignment. So, the order of precedence will not move to the wrapped object.

```
>>> order_mock.get_value.return_value = None
>>> order_mock.get_value() is None
True
```

Even if you set all three at once when initializing the mock, the order of precedence remains the same:

```
>>> order_mock = Mock(spec=Order, wraps=Order,
...                   **{"get_value.side_effect": ["first"],
...                      "get_value.return_value": "second"}
...                   )
...
>>> order_mock.get_value()
'first'
>>> order_mock.get_value.side_effect = None
>>> order_mock.get_value()
'second'
>>> order_mock.get_value.return_value = DEFAULT
>>> order_mock.get_value()
'third'
```

If *side_effect* is exhausted, the order of precedence will not cause a value to be obtained from the successors. Instead, `StopIteration` exception is raised.

```
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.get_value.side_effect = ["first side effect value",
...                                     "another side effect value"]
>>> order_mock.get_value.return_value = "second"
```

```
>>> order_mock.get_value()
'first side effect value'
>>> order_mock.get_value()
'another side effect value'
```

```
>>> order_mock.get_value()
Traceback (most recent call last):
 ...
StopIteration
```

26.7 `unittest.mock` — primeiros passos

Adicionado na versão 3.3.

26.7.1 Usando Mock

Métodos de aplicação de patches em Mock

Usos comuns para objetos *Mock* incluem:

- Métodos de aplicação de patches
- Método de gravação que invoca objetos

Talvez você queira substituir um método em um objeto para verificar se ele foi chamado com os argumentos corretos por outra parte do sistema:

```
>>> real = SomeClass()
>>> real.method = MagicMock(name='method')
```

(continua na próxima página)

(continuação da página anterior)

```
>>> real.method(3, 4, 5, key='value')
<MagicMock name='method()' id='...'>
```

Depois que nosso mock for usado (`real.method` neste exemplo), ele terá métodos e atributos que permitem que você faça afirmações sobre como ele foi usado.

Nota

Na maioria desses exemplos, as classes *`Mock`* e *`MagicMock`* são intercambiáveis. Como a `MagicMock` é a classe mais capaz, ela faz sentido para ser usada por padrão.

Uma vez que o mock foi chamado, seu atributo *`called`* é definido como `True`. Mais importante, podemos usar o método *`assert_called_with()`* ou *`assert_called_once_with()`* para verificar se ele foi chamado com os argumentos corretos.

Este exemplo testa se a chamada de `ProductionClass().method` resulta em uma chamada para o método `something`:

```
>>> class ProductionClass:
...     def method(self):
...         self.something(1, 2, 3)
...     def something(self, a, b, c):
...         pass
...
>>> real = ProductionClass()
>>> real.something = MagicMock()
>>> real.method()
>>> real.something.assert_called_once_with(1, 2, 3)
```

Mock de chamadas de métodos em um objeto

No último exemplo, aplicamos um patch em um método diretamente em um objeto para verificar se ele foi chamado corretamente. Outro caso de uso comum é passar um objeto para um método (ou alguma parte do sistema em teste) e então verificar se ele é usado da maneira correta.

O `ProductionClass` simples abaixo tem um método `closer`. Se ele for chamado com um objeto, então ele chama `close` nele.

```
>>> class ProductionClass:
...     def closer(self, something):
...         something.close()
...
```

Então, para testá-lo, precisamos passar um objeto com um método `close` e verificar se ele foi chamado corretamente.

```
>>> real = ProductionClass()
>>> mock = Mock()
>>> real.closer(mock)
>>> mock.close.assert_called_with()
```

Não precisamos fazer nenhum trabalho para fornecer o método 'close' em nosso mock. Acessar close o cria. Então, se 'close' ainda não foi chamado, acessá-lo no teste o criará, mas *`assert_called_with()`* levantará uma exceção de falha.

Mocking Classes

A common use case is to mock out classes instantiated by your code under test. When you patch a class, then that class is replaced with a mock. Instances are created by *calling the class*. This means you access the "mock instance" by looking at the return value of the mocked class.

In the example below we have a function `some_function` that instantiates `Foo` and calls a method on it. The call to *`patch()`* replaces the class `Foo` with a mock. The `Foo` instance is the result of calling the mock, so it is configured by modifying the mock *`return_value`*.

```
>>> def some_function():
...     instance = module.Foo()
...     return instance.method()
...
>>> with patch('module.Foo') as mock:
...     instance = mock.return_value
...     instance.method.return_value = 'the result'
...     result = some_function()
...     assert result == 'the result'
```

Nomeando os mocks

It can be useful to give your mocks a name. The name is shown in the repr of the mock and can be helpful when the mock appears in test failure messages. The name is also propagated to attributes or methods of the mock:

```
>>> mock = MagicMock(name='foo')
>>> mock
<MagicMock name='foo' id='...'>
>>> mock.method
<MagicMock name='foo.method' id='...'>
```

Rastreando todas as chamadas

Frequentemente você quer rastrear mais de uma chamada para um método. O atributo *`mock_calls`* registra todas as chamadas para atributos filhos do mock - e também para seus filhos.

```
>>> mock = MagicMock()
>>> mock.method()
<MagicMock name='mock.method()' id='...'>
>>> mock.attribute.method(10, x=53)
<MagicMock name='mock.attribute.method()' id='...'>
>>> mock.mock_calls
[call.method(), call.attribute.method(10, x=53)]
```

If you make an assertion about `mock_calls` and any unexpected methods have been called, then the assertion will fail. This is useful because as well as asserting that the calls you expected have been made, you are also checking that they were made in the right order and with no additional calls:

You use the *`call`* object to construct lists for comparing with `mock_calls`:

```
>>> expected = [call.method(), call.attribute.method(10, x=53)]
>>> mock.mock_calls == expected
True
```

However, parameters to calls that return mocks are not recorded, which means it is not possible to track nested calls where the parameters used to create ancestors are important:

```
>>> m = Mock()
>>> m.factory(important=True).deliver()
```

(continua na próxima página)

(continuação da página anterior)

```
<Mock name='mock.factory().deliver()' id='...'>
>>> m.mock_calls[-1] == call.factory(important=False).deliver()
True
```

Setting Return Values and Attributes

Setting the return values on a mock object is trivially easy:

```
>>> mock = Mock()
>>> mock.return_value = 3
>>> mock()
3
```

Of course you can do the same for methods on the mock:

```
>>> mock = Mock()
>>> mock.method.return_value = 3
>>> mock.method()
3
```

The return value can also be set in the constructor:

```
>>> mock = Mock(return_value=3)
>>> mock()
3
```

If you need an attribute setting on your mock, just do it:

```
>>> mock = Mock()
>>> mock.x = 3
>>> mock.x
3
```

Sometimes you want to mock up a more complex situation, like for example `mock.connection.cursor().execute("SELECT 1")`. If we wanted this call to return a list, then we have to configure the result of the nested call.

We can use *call* to construct the set of calls in a "chained call" like this for easy assertion afterwards:

```
>>> mock = Mock()
>>> cursor = mock.connection.cursor.return_value
>>> cursor.execute.return_value = ['foo']
>>> mock.connection.cursor().execute("SELECT 1")
['foo']
>>> expected = call.connection.cursor().execute("SELECT 1").call_list()
>>> mock.mock_calls
[call.connection.cursor(), call.connection.cursor().execute('SELECT 1')]
>>> mock.mock_calls == expected
True
```

It is the call to `.call_list()` that turns our call object into a list of calls representing the chained calls.

Raising exceptions with mocks

A useful attribute is *side_effect*. If you set this to an exception class or instance then the exception will be raised when the mock is called.

```
>>> mock = Mock(side_effect=Exception('Boom!'))
>>> mock()
Traceback (most recent call last):
  ...
Exception: Boom!
```

Side effect functions and iterables

`side_effect` can also be set to a function or an iterable. The use case for `side_effect` as an iterable is where your mock is going to be called several times, and you want each call to return a different value. When you set `side_effect` to an iterable every call to the mock returns the next value from the iterable:

```
>>> mock = MagicMock(side_effect=[4, 5, 6])
>>> mock()
4
>>> mock()
5
>>> mock()
6
```

For more advanced use cases, like dynamically varying the return values depending on what the mock is called with, `side_effect` can be a function. The function will be called with the same arguments as the mock. Whatever the function returns is what the call returns:

```
>>> vals = {(1, 2): 1, (2, 3): 2}
>>> def side_effect(*args):
...     return vals[args]
...
>>> mock = MagicMock(side_effect=side_effect)
>>> mock(1, 2)
1
>>> mock(2, 3)
2
```

Mocking asynchronous iterators

Since Python 3.8, `AsyncMock` and `MagicMock` have support to mock async-iterators through `__aiter__`. The *`return_value`* attribute of `__aiter__` can be used to set the return values to be used for iteration.

```
>>> mock = MagicMock()  # AsyncMock also works here
>>> mock.__aiter__.return_value = [1, 2, 3]
>>> async def main():
...     return [i async for i in mock]
...
>>> asyncio.run(main())
[1, 2, 3]
```

Mocking asynchronous context manager

Since Python 3.8, `AsyncMock` and `MagicMock` have support to mock async-context-managers through `__aenter__` and `__aexit__`. By default, `__aenter__` and `__aexit__` are `AsyncMock` instances that return an async function.

```
>>> class AsyncContextManager:
...     async def __aenter__(self):
...         return self
...     async def __aexit__(self, exc_type, exc, tb):
...         pass
```

(continua na próxima página)

(continuação da página anterior)

```
...
>>> mock_instance = MagicMock(AsyncContextManager())  # AsyncMock also works here
>>> async def main():
...     async with mock_instance as result:
...         pass
...
>>> asyncio.run(main())
>>> mock_instance.__aenter__.assert_awaited_once()
>>> mock_instance.__aexit__.assert_awaited_once()
```

Creating a Mock from an Existing Object

One problem with over use of mocking is that it couples your tests to the implementation of your mocks rather than your real code. Suppose you have a class that implements `some_method`. In a test for another class, you provide a mock of this object that *also* provides `some_method`. If later you refactor the first class, so that it no longer has `some_method` - then your tests will continue to pass even though your code is now broken!

Mock allows you to provide an object as a specification for the mock, using the *spec* keyword argument. Accessing methods / attributes on the mock that don't exist on your specification object will immediately raise an attribute error. If you change the implementation of your specification, then tests that use that class will start failing immediately without you having to instantiate the class in those tests.

```
>>> mock = Mock(spec=SomeClass)
>>> mock.old_method()
Traceback (most recent call last):
   ...
AttributeError: Mock object has no attribute 'old_method'. Did you mean: 'class_
↪method'?
```

Using a specification also enables a smarter matching of calls made to the mock, regardless of whether some parameters were passed as positional or named arguments:

```
>>> def f(a, b, c): pass
...
>>> mock = Mock(spec=f)
>>> mock(1, 2, 3)
<Mock name='mock()' id='140161580456576'>
>>> mock.assert_called_with(a=1, b=2, c=3)
```

If you want this smarter matching to also work with method calls on the mock, you can use *auto-speccing*.

If you want a stronger form of specification that prevents the setting of arbitrary attributes as well as the getting of them then you can use *spec_set* instead of *spec*.

Using side_effect to return per file content

mock_open() is used to patch *open()* method. *side_effect* can be used to return a new Mock object per call. This can be used to return different contents per file stored in a dictionary:

```
DEFAULT = "default"
data_dict = {"file1": "data1",
             "file2": "data2"}

def open_side_effect(name):
    return mock_open(read_data=data_dict.get(name, DEFAULT))()

with patch("builtins.open", side_effect=open_side_effect):
    with open("file1") as file1:
```

(continua na próxima página)

(continuação da página anterior)

```
        assert file1.read() == "data1"

    with open("file2") as file2:
        assert file2.read() == "data2"

    with open("file3") as file2:
        assert file2.read() == "default"
```

26.7.2 Patch Decorators

Nota

Com *patch()*, é importante que você faça o patch de objetos no espaço de nomes onde eles são procurados. Normalmente, isso é simples, mas para um guia rápido, leia *onde fazer o patch*.

A common need in tests is to patch a class attribute or a module attribute, for example patching a builtin or patching a class in a module to test that it is instantiated. Modules and classes are effectively global, so patching on them has to be undone after the test or the patch will persist into other tests and cause hard to diagnose problems.

mock provides three convenient decorators for this: *patch()*, *patch.object()* and *patch.dict()*. `patch` takes a single string, of the form `package.module.Class.attribute` to specify the attribute you are patching. It also optionally takes a value that you want the attribute (or class or whatever) to be replaced with. 'patch.object' takes an object and the name of the attribute you would like patched, plus optionally the value to patch it with.

`patch.object`:

```
>>> original = SomeClass.attribute
>>> @patch.object(SomeClass, 'attribute', sentinel.attribute)
... def test():
...     assert SomeClass.attribute == sentinel.attribute
...
>>> test()
>>> assert SomeClass.attribute == original

>>> @patch('package.module.attribute', sentinel.attribute)
... def test():
...     from package.module import attribute
...     assert attribute is sentinel.attribute
...
>>> test()
```

If you are patching a module (including *builtins*) then use *patch()* instead of *patch.object()*:

```
>>> mock = MagicMock(return_value=sentinel.file_handle)
>>> with patch('builtins.open', mock):
...     handle = open('filename', 'r')
...
>>> mock.assert_called_with('filename', 'r')
>>> assert handle == sentinel.file_handle, "incorrect file handle returned"
```

The module name can be 'dotted', in the form `package.module` if needed:

```
>>> @patch('package.module.ClassName.attribute', sentinel.attribute)
... def test():
...     from package.module import ClassName
...     assert ClassName.attribute == sentinel.attribute
```

(continua na próxima página)

(continuação da página anterior)

```
...
>>> test()
```

A nice pattern is to actually decorate test methods themselves:

```
>>> class MyTest(unittest.TestCase):
...     @patch.object(SomeClass, 'attribute', sentinel.attribute)
...     def test_something(self):
...         self.assertEqual(SomeClass.attribute, sentinel.attribute)
...
>>> original = SomeClass.attribute
>>> MyTest('test_something').test_something()
>>> assert SomeClass.attribute == original
```

If you want to patch with a Mock, you can use *patch()* with only one argument (or *patch.object()* with two arguments). The mock will be created for you and passed into the test function / method:

```
>>> class MyTest(unittest.TestCase):
...     @patch.object(SomeClass, 'static_method')
...     def test_something(self, mock_method):
...         SomeClass.static_method()
...         mock_method.assert_called_with()
...
>>> MyTest('test_something').test_something()
```

You can stack up multiple patch decorators using this pattern:

```
>>> class MyTest(unittest.TestCase):
...     @patch('package.module.ClassName1')
...     @patch('package.module.ClassName2')
...     def test_something(self, MockClass2, MockClass1):
...         self.assertIs(package.module.ClassName1, MockClass1)
...         self.assertIs(package.module.ClassName2, MockClass2)
...
>>> MyTest('test_something').test_something()
```

When you nest patch decorators the mocks are passed in to the decorated function in the same order they applied (the normal *Python* order that decorators are applied). This means from the bottom up, so in the example above the mock for `test_module.ClassName2` is passed in first.

Também existe *patch.dict()* para definir valores em um dicionário apenas durante um escopo e restaurar o dicionário ao seu estado original quando o teste termina:

```
>>> foo = {'key': 'value'}
>>> original = foo.copy()
>>> with patch.dict(foo, {'newkey': 'newvalue'}, clear=True):
...     assert foo == {'newkey': 'newvalue'}
...
>>> assert foo == original
```

`patch`, `patch.object` and `patch.dict` can all be used as context managers.

Where you use *patch()* to create a mock for you, you can get a reference to the mock using the "as" form of the with statement:

```
>>> class ProductionClass:
...     def method(self):
...         pass
```

(continua na próxima página)

(continuação da página anterior)

```
...
>>> with patch.object(ProductionClass, 'method') as mock_method:
...     mock_method.return_value = None
...     real = ProductionClass()
...     real.method(1, 2, 3)
...
>>> mock_method.assert_called_with(1, 2, 3)
```

As an alternative `patch`, `patch.object` and `patch.dict` can be used as class decorators. When used in this way it is the same as applying the decorator individually to every method whose name starts with "test".

26.7.3 Further Examples

Here are some more examples for some slightly more advanced scenarios.

Mocking chained calls

Mocking chained calls is actually straightforward with mock once you understand the *return_value* attribute. When a mock is called for the first time, or you fetch its `return_value` before it has been called, a new *Mock* is created.

This means that you can see how the object returned from a call to a mocked object has been used by interrogating the `return_value` mock:

```
>>> mock = Mock()
>>> mock().foo(a=2, b=3)
<Mock name='mock().foo()' id='...'>
>>> mock.return_value.foo.assert_called_with(a=2, b=3)
```

From here it is a simple step to configure and then make assertions about chained calls. Of course another alternative is writing your code in a more testable way in the first place…

So, suppose we have some code that looks a little bit like this:

```
>>> class Something:
...     def __init__(self):
...         self.backend = BackendProvider()
...     def method(self):
...         response = self.backend.get_endpoint('foobar').create_call('spam',
↪'eggs').start_call()
...         # more code
```

Assuming that `BackendProvider` is already well tested, how do we test `method()`? Specifically, we want to test that the code section `# more code` uses the response object in the correct way.

As this chain of calls is made from an instance attribute we can monkey patch the `backend` attribute on a `Something` instance. In this particular case we are only interested in the return value from the final call to `start_call` so we don't have much configuration to do. Let's assume the object it returns is 'file-like', so we'll ensure that our response object uses the builtin *open()* as its `spec`.

To do this we create a mock instance as our mock backend and create a mock response object for it. To set the response as the return value for that final `start_call` we could do this:

```
mock_backend.get_endpoint.return_value.create_call.return_value.start_call.return_
↪value = mock_response
```

We can do that in a slightly nicer way using the *configure_mock()* method to directly set the return value for us:

```
>>> something = Something()
>>> mock_response = Mock(spec=open)
>>> mock_backend = Mock()
>>> config = {'get_endpoint.return_value.create_call.return_value.start_call.
↪return_value': mock_response}
>>> mock_backend.configure_mock(**config)
```

With these we monkey patch the "mock backend" in place and can make the real call:

```
>>> something.backend = mock_backend
>>> something.method()
```

Using *mock_calls* we can check the chained call with a single assert. A chained call is several calls in one line of code, so there will be several entries in `mock_calls`. We can use *call.call_list()* to create this list of calls for us:

```
>>> chained = call.get_endpoint('foobar').create_call('spam', 'eggs').start_call()
>>> call_list = chained.call_list()
>>> assert mock_backend.mock_calls == call_list
```

Partial mocking

In some tests I wanted to mock out a call to *datetime.date.today()* to return a known date, but I didn't want to prevent the code under test from creating new date objects. Unfortunately *datetime.date* is written in C, and so I couldn't just monkey-patch out the static *datetime.date.today()* method.

I found a simple way of doing this that involved effectively wrapping the date class with a mock, but passing through calls to the constructor to the real class (and returning real instances).

The *patch decorator* is used here to mock out the `date` class in the module under test. The *side_effect* attribute on the mock date class is then set to a lambda function that returns a real date. When the mock date class is called a real date will be constructed and returned by `side_effect`.

```
>>> from datetime import date
>>> with patch('mymodule.date') as mock_date:
...     mock_date.today.return_value = date(2010, 10, 8)
...     mock_date.side_effect = lambda *args, **kw: date(*args, **kw)
...
...     assert mymodule.date.today() == date(2010, 10, 8)
...     assert mymodule.date(2009, 6, 8) == date(2009, 6, 8)
```

Note that we don't patch *datetime.date* globally, we patch `date` in the module that *uses* it. See *where to patch*.

When `date.today()` is called a known date is returned, but calls to the `date(...)` constructor still return normal dates. Without this you can find yourself having to calculate an expected result using exactly the same algorithm as the code under test, which is a classic testing anti-pattern.

Calls to the date constructor are recorded in the `mock_date` attributes (`call_count` and friends) which may also be useful for your tests.

Uma forma alternativa de lidar com datas de mock, ou outras classes embutidas, é discutida nesta entrada de blog.

Mocking a Generator Method

A Python generator is a function or method that uses the `yield` statement to return a series of values when iterated over[1].

A generator method / function is called to return the generator object. It is the generator object that is then iterated over. The protocol method for iteration is *__iter__()*, so we can mock this using a *MagicMock*.

[1] There are also generator expressions and more advanced uses of generators, but we aren't concerned about them here. A very good introduction to generators and how powerful they are is: Generator Tricks for Systems Programmers.

Here's an example class with an "iter" method implemented as a generator:

```
>>> class Foo:
...     def iter(self):
...         for i in [1, 2, 3]:
...             yield i
...
>>> foo = Foo()
>>> list(foo.iter())
[1, 2, 3]
```

How would we mock this class, and in particular its "iter" method?

To configure the values returned from the iteration (implicit in the call to *list*), we need to configure the object returned by the call to `foo.iter()`.

```
>>> mock_foo = MagicMock()
>>> mock_foo.iter.return_value = iter([1, 2, 3])
>>> list(mock_foo.iter())
[1, 2, 3]
```

Applying the same patch to every test method

If you want several patches in place for multiple test methods the obvious way is to apply the patch decorators to every method. This can feel like unnecessary repetition. Instead, you can use *patch()* (in all its various forms) as a class decorator. This applies the patches to all test methods on the class. A test method is identified by methods whose names start with `test`:

```
>>> @patch('mymodule.SomeClass')
... class MyTest(unittest.TestCase):
...
...     def test_one(self, MockSomeClass):
...         self.assertIs(mymodule.SomeClass, MockSomeClass)
...
...     def test_two(self, MockSomeClass):
...         self.assertIs(mymodule.SomeClass, MockSomeClass)
...
...     def not_a_test(self):
...         return 'something'
...
>>> MyTest('test_one').test_one()
>>> MyTest('test_two').test_two()
>>> MyTest('test_two').not_a_test()
'something'
```

An alternative way of managing patches is to use the *métodos do patch: start e stop*. These allow you to move the patching into your `setUp` and `tearDown` methods.

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         self.patcher = patch('mymodule.foo')
...         self.mock_foo = self.patcher.start()
...
...     def test_foo(self):
...         self.assertIs(mymodule.foo, self.mock_foo)
...
...     def tearDown(self):
...         self.patcher.stop()
```

(continua na próxima página)

(continuação da página anterior)

```
...
>>> MyTest('test_foo').run()
```

If you use this technique you must ensure that the patching is "undone" by calling `stop`. This can be fiddlier than you might think, because if an exception is raised in the setUp then tearDown is not called. *unittest.TestCase.addCleanup()* makes this easier:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         patcher = patch('mymodule.foo')
...         self.addCleanup(patcher.stop)
...         self.mock_foo = patcher.start()
...
...     def test_foo(self):
...         self.assertIs(mymodule.foo, self.mock_foo)
...
>>> MyTest('test_foo').run()
```

Mocking Unbound Methods

Whilst writing tests today I needed to patch an *unbound method* (patching the method on the class rather than on the instance). I needed self to be passed in as the first argument because I want to make asserts about which objects were calling this particular method. The issue is that you can't patch with a mock for this, because if you replace an unbound method with a mock it doesn't become a bound method when fetched from the instance, and so it doesn't get self passed in. The workaround is to patch the unbound method with a real function instead. The *patch()* decorator makes it so simple to patch out methods with a mock that having to create a real function becomes a nuisance.

If you pass `autospec=True` to patch then it does the patching with a *real* function object. This function object has the same signature as the one it is replacing, but delegates to a mock under the hood. You still get your mock auto-created in exactly the same way as before. What it means though, is that if you use it to patch out an unbound method on a class the mocked function will be turned into a bound method if it is fetched from an instance. It will have `self` passed in as the first argument, which is exactly what I wanted:

```
>>> class Foo:
...   def foo(self):
...     pass
...
>>> with patch.object(Foo, 'foo', autospec=True) as mock_foo:
...   mock_foo.return_value = 'foo'
...   foo = Foo()
...   foo.foo()
...
'foo'
>>> mock_foo.assert_called_once_with(foo)
```

If we don't use `autospec=True` then the unbound method is patched out with a Mock instance instead, and isn't called with `self`.

Checking multiple calls with mock

mock has a nice API for making assertions about how your mock objects are used.

```
>>> mock = Mock()
>>> mock.foo_bar.return_value = None
>>> mock.foo_bar('baz', spam='eggs')
>>> mock.foo_bar.assert_called_with('baz', spam='eggs')
```

If your mock is only being called once you can use the *assert_called_once_with()* method that also asserts that the *call_count* is one.

```
>>> mock.foo_bar.assert_called_once_with('baz', spam='eggs')
>>> mock.foo_bar()
>>> mock.foo_bar.assert_called_once_with('baz', spam='eggs')
Traceback (most recent call last):
    ...
AssertionError: Expected 'foo_bar' to be called once. Called 2 times.
Calls: [call('baz', spam='eggs'), call()].
```

Both `assert_called_with` and `assert_called_once_with` make assertions about the *most recent* call. If your mock is going to be called several times, and you want to make assertions about *all* those calls you can use *call_args_list*:

```
>>> mock = Mock(return_value=None)
>>> mock(1, 2, 3)
>>> mock(4, 5, 6)
>>> mock()
>>> mock.call_args_list
[call(1, 2, 3), call(4, 5, 6), call()]
```

The *call* helper makes it easy to make assertions about these calls. You can build up a list of expected calls and compare it to `call_args_list`. This looks remarkably similar to the repr of the `call_args_list`:

```
>>> expected = [call(1, 2, 3), call(4, 5, 6), call()]
>>> mock.call_args_list == expected
True
```

Coping with mutable arguments

Another situation is rare, but can bite you, is when your mock is called with mutable arguments. `call_args` and `call_args_list` store *references* to the arguments. If the arguments are mutated by the code under test then you can no longer make assertions about what the values were when the mock was called.

Here's some example code that shows the problem. Imagine the following functions defined in 'mymodule':

```
def frob(val):
    pass

def grob(val):
    "First frob and then clear val"
    frob(val)
    val.clear()
```

When we try to test that `grob` calls `frob` with the correct argument look what happens:

```
>>> with patch('mymodule.frob') as mock_frob:
...     val = {6}
...     mymodule.grob(val)
...
>>> val
set()
>>> mock_frob.assert_called_with({6})
Traceback (most recent call last):
    ...
AssertionError: Expected: (({6},), {})
Called with: ((set(),), {})
```

One possibility would be for mock to copy the arguments you pass in. This could then cause problems if you do assertions that rely on object identity for equality.

Here's one solution that uses the *side_effect* functionality. If you provide a `side_effect` function for a mock then `side_effect` will be called with the same args as the mock. This gives us an opportunity to copy the arguments and store them for later assertions. In this example I'm using *another* mock to store the arguments so that I can use the mock methods for doing the assertion. Again a helper function sets this up for me.

```
>>> from copy import deepcopy
>>> from unittest.mock import Mock, patch, DEFAULT
>>> def copy_call_args(mock):
...     new_mock = Mock()
...     def side_effect(*args, **kwargs):
...         args = deepcopy(args)
...         kwargs = deepcopy(kwargs)
...         new_mock(*args, **kwargs)
...         return DEFAULT
...     mock.side_effect = side_effect
...     return new_mock
...
>>> with patch('mymodule.frob') as mock_frob:
...     new_mock = copy_call_args(mock_frob)
...     val = {6}
...     mymodule.grob(val)
...
>>> new_mock.assert_called_with({6})
>>> new_mock.call_args
call({6})
```

`copy_call_args` is called with the mock that will be called. It returns a new mock that we do the assertion on. The `side_effect` function makes a copy of the args and calls our `new_mock` with the copy.

Nota

If your mock is only going to be used once there is an easier way of checking arguments at the point they are called. You can simply do the checking inside a `side_effect` function.

```
>>> def side_effect(arg):
...     assert arg == {6}
...
>>> mock = Mock(side_effect=side_effect)
>>> mock({6})
>>> mock(set())
Traceback (most recent call last):
    ...
AssertionError
```

An alternative approach is to create a subclass of *Mock* or *MagicMock* that copies (using *copy.deepcopy()*) the arguments. Here's an example implementation:

```
>>> from copy import deepcopy
>>> class CopyingMock(MagicMock):
...     def __call__(self, /, *args, **kwargs):
...         args = deepcopy(args)
...         kwargs = deepcopy(kwargs)
...         return super().__call__(*args, **kwargs)
...
```

(continua na próxima página)

(continuação da página anterior)

```
>>> c = CopyingMock(return_value=None)
>>> arg = set()
>>> c(arg)
>>> arg.add(1)
>>> c.assert_called_with(set())
>>> c.assert_called_with(arg)
Traceback (most recent call last):
    ...
AssertionError: expected call not found.
Expected: mock({1})
Actual: mock(set())
>>> c.foo
<CopyingMock name='mock.foo' id='...'>
```

When you subclass `Mock` or `MagicMock` all dynamically created attributes, and the `return_value` will use your subclass automatically. That means all children of a `CopyingMock` will also have the type `CopyingMock`.

Nesting Patches

Using patch as a context manager is nice, but if you do multiple patches you can end up with nested with statements indenting further and further to the right:

```
>>> class MyTest(unittest.TestCase):
...
...     def test_foo(self):
...         with patch('mymodule.Foo') as mock_foo:
...             with patch('mymodule.Bar') as mock_bar:
...                 with patch('mymodule.Spam') as mock_spam:
...                     assert mymodule.Foo is mock_foo
...                     assert mymodule.Bar is mock_bar
...                     assert mymodule.Spam is mock_spam
...
>>> original = mymodule.Foo
>>> MyTest('test_foo').test_foo()
>>> assert mymodule.Foo is original
```

With unittest `cleanup` functions and the *métodos do patch: start e stop* we can achieve the same effect without the nested indentation. A simple helper method, `create_patch`, puts the patch in place and returns the created mock for us:

```
>>> class MyTest(unittest.TestCase):
...
...     def create_patch(self, name):
...         patcher = patch(name)
...         thing = patcher.start()
...         self.addCleanup(patcher.stop)
...         return thing
...
...     def test_foo(self):
...         mock_foo = self.create_patch('mymodule.Foo')
...         mock_bar = self.create_patch('mymodule.Bar')
...         mock_spam = self.create_patch('mymodule.Spam')
...
...         assert mymodule.Foo is mock_foo
...         assert mymodule.Bar is mock_bar
...         assert mymodule.Spam is mock_spam
...
```

(continua na próxima página)

(continuação da página anterior)

```
>>> original = mymodule.Foo
>>> MyTest('test_foo').run()
>>> assert mymodule.Foo is original
```

Mocking a dictionary with MagicMock

You may want to mock a dictionary, or other container object, recording all access to it whilst having it still behave like a dictionary.

We can do this with *MagicMock*, which will behave like a dictionary, and using *side_effect* to delegate dictionary access to a real underlying dictionary that is under our control.

When the `__getitem__()` and `__setitem__()` methods of our `MagicMock` are called (normal dictionary access) then `side_effect` is called with the key (and in the case of `__setitem__` the value too). We can also control what is returned.

After the `MagicMock` has been used we can use attributes like *call_args_list* to assert about how the dictionary was used:

```
>>> my_dict = {'a': 1, 'b': 2, 'c': 3}
>>> def getitem(name):
...      return my_dict[name]
...
>>> def setitem(name, val):
...     my_dict[name] = val
...
>>> mock = MagicMock()
>>> mock.__getitem__.side_effect = getitem
>>> mock.__setitem__.side_effect = setitem
```

Nota

An alternative to using `MagicMock` is to use `Mock` and *only* provide the magic methods you specifically want:

```
>>> mock = Mock()
>>> mock.__getitem__ = Mock(side_effect=getitem)
>>> mock.__setitem__ = Mock(side_effect=setitem)
```

A *third* option is to use `MagicMock` but passing in `dict` as the *spec* (or *spec_set*) argument so that the `MagicMock` created only has dictionary magic methods available:

```
>>> mock = MagicMock(spec_set=dict)
>>> mock.__getitem__.side_effect = getitem
>>> mock.__setitem__.side_effect = setitem
```

With these side effect functions in place, the `mock` will behave like a normal dictionary but recording the access. It even raises a *KeyError* if you try to access a key that doesn't exist.

```
>>> mock['a']
1
>>> mock['c']
3
>>> mock['d']
Traceback (most recent call last):
    ...
KeyError: 'd'
>>> mock['b'] = 'fish'
```

(continua na próxima página)

(continuação da página anterior)

```
>>> mock['d'] = 'eggs'
>>> mock['b']
'fish'
>>> mock['d']
'eggs'
```

After it has been used you can make assertions about the access using the normal mock methods and attributes:

```
>>> mock.__getitem__.call_args_list
[call('a'), call('c'), call('d'), call('b'), call('d')]
>>> mock.__setitem__.call_args_list
[call('b', 'fish'), call('d', 'eggs')]
>>> my_dict
{'a': 1, 'b': 'fish', 'c': 3, 'd': 'eggs'}
```

Mock subclasses and their attributes

There are various reasons why you might want to subclass *Mock*. One reason might be to add helper methods. Here's a silly example:

```
>>> class MyMock(MagicMock):
...     def has_been_called(self):
...         return self.called
...
>>> mymock = MyMock(return_value=None)
>>> mymock
<MyMock id='...'>
>>> mymock.has_been_called()
False
>>> mymock()
>>> mymock.has_been_called()
True
```

The standard behaviour for `Mock` instances is that attributes and the return value mocks are of the same type as the mock they are accessed on. This ensures that `Mock` attributes are `Mocks` and `MagicMock` attributes are `MagicMocks`[2]. So if you're subclassing to add helper methods then they'll also be available on the attributes and return value mock of instances of your subclass.

```
>>> mymock.foo
<MyMock name='mock.foo' id='...'>
>>> mymock.foo.has_been_called()
False
>>> mymock.foo()
<MyMock name='mock.foo()' id='...'>
>>> mymock.foo.has_been_called()
True
```

Sometimes this is inconvenient. For example, one user is subclassing mock to created a Twisted adaptor. Having this applied to attributes too actually causes errors.

`Mock` (in all its flavours) uses a method called `_get_child_mock` to create these "sub-mocks" for attributes and return values. You can prevent your subclass being used for attributes by overriding this method. The signature is that it takes arbitrary keyword arguments (`**kwargs`) which are then passed onto the mock constructor:

[2] An exception to this rule are the non-callable mocks. Attributes use the callable variant because otherwise non-callable mocks couldn't have callable methods.

```
>>> class Subclass(MagicMock):
...     def _get_child_mock(self, /, **kwargs):
...         return MagicMock(**kwargs)
...
>>> mymock = Subclass()
>>> mymock.foo
<MagicMock name='mock.foo' id='...'>
>>> assert isinstance(mymock, Subclass)
>>> assert not isinstance(mymock.foo, Subclass)
>>> assert not isinstance(mymock(), Subclass)
```

Mocking imports with patch.dict

One situation where mocking can be hard is where you have a local import inside a function. These are harder to mock because they aren't using an object from the module namespace that we can patch out.

Generally local imports are to be avoided. They are sometimes done to prevent circular dependencies, for which there is *usually* a much better way to solve the problem (refactor the code) or to prevent "up front costs" by delaying the import. This can also be solved in better ways than an unconditional local import (store the module as a class or module attribute and only do the import on first use).

That aside there is a way to use `mock` to affect the results of an import. Importing fetches an *object* from the *sys.modules* dictionary. Note that it fetches an *object*, which need not be a module. Importing a module for the first time results in a module object being put in `sys.modules`, so usually when you import something you get a module back. This need not be the case however.

This means you can use *patch.dict()* to *temporarily* put a mock in place in *sys.modules*. Any imports whilst this patch is active will fetch the mock. When the patch is complete (the decorated function exits, the with statement body is complete or `patcher.stop()` is called) then whatever was there previously will be restored safely.

Here's an example that mocks out the 'fooble' module.

```
>>> import sys
>>> mock = Mock()
>>> with patch.dict('sys.modules', {'fooble': mock}):
...     import fooble
...     fooble.blob()
...
<Mock name='mock.blob()' id='...'>
>>> assert 'fooble' not in sys.modules
>>> mock.blob.assert_called_once_with()
```

As you can see the `import fooble` succeeds, but on exit there is no 'fooble' left in *sys.modules*.

This also works for the `from module import name` form:

```
>>> mock = Mock()
>>> with patch.dict('sys.modules', {'fooble': mock}):
...     from fooble import blob
...     blob.blip()
...
<Mock name='mock.blob.blip()' id='...'>
>>> mock.blob.blip.assert_called_once_with()
```

With slightly more work you can also mock package imports:

```
>>> mock = Mock()
>>> modules = {'package': mock, 'package.module': mock.module}
>>> with patch.dict('sys.modules', modules):
...     from package.module import fooble
```

(continua na próxima página)

(continuação da página anterior)

```
...     fooble()
...
<Mock name='mock.module.fooble()' id='...'>
>>> mock.module.fooble.assert_called_once_with()
```

Tracking order of calls and less verbose call assertions

The *Mock* class allows you to track the *order* of method calls on your mock objects through the *method_calls* attribute. This doesn't allow you to track the order of calls between separate mock objects, however we can use *mock_calls* to achieve the same effect.

Because mocks track calls to child mocks in `mock_calls`, and accessing an arbitrary attribute of a mock creates a child mock, we can create our separate mocks from a parent one. Calls to those child mock will then all be recorded, in order, in the `mock_calls` of the parent:

```
>>> manager = Mock()
>>> mock_foo = manager.foo
>>> mock_bar = manager.bar
```

```
>>> mock_foo.something()
<Mock name='mock.foo.something()' id='...'>
>>> mock_bar.other.thing()
<Mock name='mock.bar.other.thing()' id='...'>
```

```
>>> manager.mock_calls
[call.foo.something(), call.bar.other.thing()]
```

We can then assert about the calls, including the order, by comparing with the `mock_calls` attribute on the manager mock:

```
>>> expected_calls = [call.foo.something(), call.bar.other.thing()]
>>> manager.mock_calls == expected_calls
True
```

If `patch` is creating, and putting in place, your mocks then you can attach them to a manager mock using the *attach_mock()* method. After attaching calls will be recorded in `mock_calls` of the manager.

```
>>> manager = MagicMock()
>>> with patch('mymodule.Class1') as MockClass1:
...     with patch('mymodule.Class2') as MockClass2:
...         manager.attach_mock(MockClass1, 'MockClass1')
...         manager.attach_mock(MockClass2, 'MockClass2')
...         MockClass1().foo()
...         MockClass2().bar()
<MagicMock name='mock.MockClass1().foo()' id='...'>
<MagicMock name='mock.MockClass2().bar()' id='...'>
>>> manager.mock_calls
[call.MockClass1(),
call.MockClass1().foo(),
call.MockClass2(),
call.MockClass2().bar()]
```

If many calls have been made, but you're only interested in a particular sequence of them then an alternative is to use the *assert_has_calls()* method. This takes a list of calls (constructed with the *call* object). If that sequence of calls are in *mock_calls* then the assert succeeds.

```
>>> m = MagicMock()
>>> m().foo().bar().baz()
<MagicMock name='mock().foo().bar().baz()' id='...'>
>>> m.one().two().three()
<MagicMock name='mock.one().two().three()' id='...'>
>>> calls = call.one().two().three().call_list()
>>> m.assert_has_calls(calls)
```

Even though the chained call `m.one().two().three()` aren't the only calls that have been made to the mock, the assert still succeeds.

Sometimes a mock may have several calls made to it, and you are only interested in asserting about *some* of those calls. You may not even care about the order. In this case you can pass `any_order=True` to `assert_has_calls`:

```
>>> m = MagicMock()
>>> m(1), m.two(2, 3), m.seven(7), m.fifty('50')
(...)
>>> calls = [call.fifty('50'), call(1), call.seven(7)]
>>> m.assert_has_calls(calls, any_order=True)
```

More complex argument matching

Using the same basic concept as *ANY* we can implement matchers to do more complex assertions on objects used as arguments to mocks.

Suppose we expect some object to be passed to a mock that by default compares equal based on object identity (which is the Python default for user defined classes). To use *assert_called_with()* we would need to pass in the exact same object. If we are only interested in some of the attributes of this object then we can create a matcher that will check these attributes for us.

You can see in this example how a 'standard' call to `assert_called_with` isn't sufficient:

```
>>> class Foo:
...     def __init__(self, a, b):
...         self.a, self.b = a, b
...
>>> mock = Mock(return_value=None)
>>> mock(Foo(1, 2))
>>> mock.assert_called_with(Foo(1, 2))
Traceback (most recent call last):
    ...
AssertionError: expected call not found.
Expected: mock(<__main__.Foo object at 0x...>)
Actual: mock(<__main__.Foo object at 0x...>)
```

A comparison function for our `Foo` class might look something like this:

```
>>> def compare(self, other):
...     if not type(self) == type(other):
...         return False
...     if self.a != other.a:
...         return False
...     if self.b != other.b:
...         return False
...     return True
...
```

And a matcher object that can use comparison functions like this for its equality operation would look something like this:

```
>>> class Matcher:
...     def __init__(self, compare, some_obj):
...         self.compare = compare
...         self.some_obj = some_obj
...     def __eq__(self, other):
...         return self.compare(self.some_obj, other)
...
```

Putting all this together:

```
>>> match_foo = Matcher(compare, Foo(1, 2))
>>> mock.assert_called_with(match_foo)
```

The `Matcher` is instantiated with our compare function and the `Foo` object we want to compare against. In `assert_called_with` the `Matcher` equality method will be called, which compares the object the mock was called with against the one we created our matcher with. If they match then `assert_called_with` passes, and if they don't an *AssertionError* is raised:

```
>>> match_wrong = Matcher(compare, Foo(3, 4))
>>> mock.assert_called_with(match_wrong)
Traceback (most recent call last):
    ...
AssertionError: Expected: ((<Matcher object at 0x...>,), {})
Called with: ((<Foo object at 0x...>,), {})
```

With a bit of tweaking you could have the comparison function raise the *AssertionError* directly and provide a more useful failure message.

As of version 1.5, the Python testing library PyHamcrest provides similar functionality, that may be useful here, in the form of its equality matcher (hamcrest.library.integration.match_equality).

26.8 `test` — Pacote de Testes de Regressão do Python

Nota

O pacote *test* é apenas para uso interno do Python. O mesmo está sendo documentado para o benefício dos principais desenvolvedores do Python. Qualquer uso deste pacote fora da biblioteca padrão do Python é desencorajado, pois, o código mencionado aqui pode ser alterado ou removido sem aviso prévio entre as versões do Python.

O pacote *test* contém todos os testes de regressão do Python, bem como os módulos *test.support* e *test.regrtest*. *test.support* é utilizado para aprimorar seus testes enquanto o *test.regrtest* direciona a suite de testes.

Cada módulo no pacote *test* cujo nome começa com `test_` é um conjunto de testes para um módulo ou recurso específico. Todos os novos testes devem ser escritos usando o módulo *unittest* ou *doctest*. Alguns testes mais antigos são escritos usando um estilo de teste "tradicional" que compara a saída impressa a `sys.stdout`. Este estilo de teste foi considerado descontinuado.

Ver também

Módulo *unittest*
Escrevendo testes de regressão PyUnit.

Módulo *doctest*
Testes embutidos em Strings da documentação.

26.8.1 Escrever testes unitários para o pacote `test`

É preferível que os testes que usam o módulo *unittest* sigam algumas diretrizes. Uma é nomear o módulo de teste iniciando-o com `test_` e termine com o nome do módulo que está sendo testado. Os métodos de teste no módulo de teste deve começar com `test_` e terminar com uma descrição do que o método está testando. Isso é necessário para que os métodos sejam reconhecidos pelo driver de teste como métodos de teste. Além disso, na string de documentação para o método deve ser incluído. Um comentário (como os `# Tests function returns only True or False`) deve ser usado para fornecer documentação para testar métodos. Isso é feito porque as strings de documentação são impressas se existem e, portanto, qual teste está sendo executado não é indicado.

Um boilerplate básico é muitas vezes usado:

```
import unittest
from test import support

class MyTestCase1(unittest.TestCase):

    # Só use setUp() e tearDown() se necessário

    def setUp(self):
        ... código para executar na preparação de testes ...

    def tearDown(self):
        ... código para executar a limpeza após os testes ...

    def test_feature_one(self):
        # Testa o recurso um.
        ... código de teste ...

    def test_feature_two(self):
        # Testa o recurso dois.
        ... código de teste ...

    ... mais métodos de teste ...

class MyTestCase2(unittest.TestCase):
    ... mesma estrutura que MyTestCase1 ...

... mais classes de teste ...

if __name__ == '__main__':
    unittest.main()
```

Este padrão de código permite que o conjunto de testes seja executado pelo *test.regrtest*, por conta própria, como um script que suporte o *unittest* CLI, ou através do `python -m unittest` CLI.

O objetivo do teste de regressão é tentar quebrar o código. Isso leva a algumas diretrizes que devemos seguir:

- O conjunto de testes deve exercitar todas as classes, funções e constantes. Isso inclui não apenas a API externa que deve ser apresentada ao mundo exterior, mas também o código "privado".
- Os testes de Whitebox (que examinam o código que está sendo testado quando os testes estão sendo gravados) são preferidos. O teste Blackbox (que testa apenas a interface do público de usuário) não é completo o suficiente para garantir que todos os casos de limite e extremos sejam testados.
- Certifique-se de que todos os valores possíveis sejam testados, incluindo os inválidos. Isso garante que não

apenas todos os valores válidos são aceitos, mas também, que os valores impróprios são tratados corretamente.

- Esgote o maior número possível de caminhos de código. Teste onde ocorre a ramificação e, assim, personalize a entrada para garantir que tantos caminhos diferentes pelo código sejam tomados.
- Adicione um teste explícito para quaisquer bugs descobertos ao código testado. Isso garantirá que o erro não apareça novamente se o código for alterado no futuro.
- Certifique-se de limpar após seus testes (como fechar e remover todos os arquivos temporários).
- Se um teste depende de uma condição específica do sistema operacional, então verifica se a condição já existe antes de tentar o teste.
- Importa o menor número de módulos possível e faça isso o mais rápido possível. Isso minimiza dependências externas de testes, e também minimiza possíveis comportamento anômalo dos efeitos colaterais da importação de um módulo.
- Tente maximizar a reutilização de código. Ocasionalmente, os testes variam em algo tão pequeno quanto o tipo de entrada é usado. Minimize a duplicação de código criando uma subclasse básica de testes com uma classe que especifica o input:

```
class TestFuncAcceptsSequencesMixin:

    func = mySuperWhammyFunction

    def test_func(self):
        self.func(self.arg)

class AcceptLists(TestFuncAcceptsSequencesMixin, unittest.TestCase):
    arg = [1, 2, 3]

class AcceptStrings(TestFuncAcceptsSequencesMixin, unittest.TestCase):
    arg = 'abc'

class AcceptTuples(TestFuncAcceptsSequencesMixin, unittest.TestCase):
    arg = (1, 2, 3)
```

Ao usar este padrão, lembre-se que todas as classes que herdam de *unittest.TestCase* são executadas como testes. A classe `TestFuncAcceptsSequencesMixin` no exemplo acima não tem nenhum dado e, portanto, não pode ser executada sozinha, portanto, ela não herda de *unittest.TestCase*.

Ver também

Test Driven Development
Um livro de Kent Beck sobre escrita de testes antes do código.

26.8.2 Executando testes usando a interface de linha de comando

O pacote *test* pode ser executado como um script para conduzir o conjunto de testes de regressão do Python, graças à opção `-m`: **`python -m test`**. Nos bastidores, ele usa *test.regrtest*; a chamada **`python -m test.regrtest`** usado nas versões anteriores do Python ainda funciona. Executar o script por si só começa a executar todos os testes de regressão no pacote *test*. Ele faz isso encontrando todos os módulos no pacote cujo nome começa com `test_`, importando-os e executando a função `test_main()` se presente ou carregando os testes via unittest.TestLoader.loadTestsFromModule se `test_main` não existir. Os nomes dos testes a serem executados também podem ser passados para o script. Especificando um teste de regressão simples (**`python -m test test_spam`**) minimizará saiba e imprima apenas se o teste passou ou falhou.

A execução de *test* permite definir diretamente quais recursos estão disponíveis para os testes usarem. Você faz isso usando a opção de linha de comando `-u`. Especificar `all` como o valor para a opção `-u` ativa todos os recursos possíveis: **`python -m test -uall`**. Se todos menos um recurso for desejado (um caso mais comum), uma lista separada por vírgulas de recursos que não são desejados pode ser listada após `all`. O comando **`python -m test`**

-uall,-audio,-largefile executará *test* com todos os recursos, exceto os recursos `audio` e `largefile`. Para obter uma lista de todos os recursos e mais opções de linha de comando, execute **python -m test -h**.

Alguns outros meios para executar os testes de regressão dependem em qual plataforma os testes estão sendo executados. No Unix, você pode executar: programa: ‘ make test’ no diretório de mais alto nível onde o Python foi construído. No Windows, executar: programa ‘rt.bat’ do seu diretório: file: ‘PCbuild’ executará todos os testes de regressão.

Adicionado na versão 3.14: A saída é colorizada por padrão e ela pode ser controlada usando variáveis de ambiente.

26.9 `test.support` — Utilitários para o conjunto de teste do Python

O módulo *test.support* fornece suporte para a suíte de testes de regressão do Python.

Nota

test.support is not a public module. It is documented here to help Python developers write tests. The API of this module is subject to change without backwards compatibility concerns between releases.

Esse módulo define as seguintes exceções:

exception test.support.**TestFailed**

Exceção a ser levantada quando um teste falha. Isto foi descontinuado em favor dos testes baseados em *unittest* e métodos de asserção de *unittest.TestCase*.

exception test.support.**ResourceDenied**

Subclasse de *unittest.SkipTest*. Levantada quando um recurso (como uma conexão de rede) não está disponível. Levantada pela função *requires()*.

O módulo *test.support* define as seguintes constantes:

test.support.**verbose**

`True` quando a saída detalhada está habilitada. Deve ser verificado quando informações mais detalhadas são desejadas sobre um teste em execução. *verbose* é definido por *test.regrtest*.

test.support.**is_jython**

`True` se o interpretador em execução for Jython.

test.support.**is_android**

`True` if `sys.platform` is `android`.

test.support.**is_emscripten**

`True` if `sys.platform` is `emscripten`.

test.support.**is_wasi**

`True` if `sys.platform` is `wasi`.

test.support.**is_apple_mobile**

`True` if `sys.platform` is `ios`, `tvos`, or `watchos`.

test.support.**is_apple**

`True` if `sys.platform` is `darwin` or `is_apple_mobile` is `True`.

test.support.**unix_shell**

Caminho para o console se não estiver no Windows; por outro lado `None`

test.support.**LOOPBACK_TIMEOUT**

Tempo limite em segundos para testes usando um servidor de rede escutando na interface de loopback local da rede como `127.0.0.1`.

O tempo limite é longo o suficiente para evitar a falha do teste: leva em consideração que o cliente e o servidor podem ser executados em diferentes threads ou mesmo processos diferentes.

O tempo limite deve ser longo o suficiente para os métodos *connect()*, *recv()* e *send()* de *socket.socket*.

Seu valor padrão é 5 segundos.

Veja também *INTERNET_TIMEOUT*.

test.support.**INTERNET_TIMEOUT**

Tempo limite em segundos para solicitações de rede indo para a Internet.

O tempo limite é curto o suficiente para evitar que um teste espere muito tempo se a requisição da Internet for bloqueada por qualquer motivo.

Normalmente, um tempo limite usando *INTERNET_TIMEOUT* não deve marcar um teste como falhado, mas ignorá-lo: veja *transient_internet()*.

Seu valor padrão é 1 minuto.

Veja também *LOOPBACK_TIMEOUT*.

test.support.**SHORT_TIMEOUT**

Tempo limite em segundos para marcar um teste como falho se o teste demorar "muito tempo".

O valor do tempo limite depende da opção da linha de comando regrtest `--timeout`.

Se um teste usando *SHORT_TIMEOUT* começar a falhar aleatoriamente em buildbots lentos, use *LONG_TIMEOUT*.

Seu valor padrão é 30 segundos.

test.support.**LONG_TIMEOUT**

Tempo limite em segundos para detectar quando um teste trava.

É longo o suficiente para reduzir o risco de falha de teste nos buildbots Python mais lentos. Não deve ser usado para marcar um teste como reprovado se o teste demorar "muito tempo". O valor do tempo limite depende da opção de linha de comando regrtest `--timeout`.

Seu valor padrão é 5 minutos.

Veja também *LOOPBACK_TIMEOUT*, *INTERNET_TIMEOUT* e *SHORT_TIMEOUT*.

test.support.**PGO**

Definido quando os testes podem ser ignorados quando não são úteis para PGO.

test.support.**PIPE_MAX_SIZE**

Uma constante que provavelmente é maior que o tamanho do buffer de canal do sistema operacional subjacente, para fazer o bloqueio de escritas.

test.support.**Py_DEBUG**

`True` se o Python foi criado com a macro `Py_DEBUG` definida, ou seja, se o Python foi criado no modo de depuração.

Adicionado na versão 3.12.

test.support.**SOCK_MAX_SIZE**

Uma constante que provavelmente é maior que o tamanho do buffer de soquete do sistema operacional subjacente, para fazer o bloqueio de escritas.

test.support.**TEST_SUPPORT_DIR**

Define o diretório de mais alto nível que contém *test.support*.

test.support.**TEST_HOME_DIR**

Define o diretório de mais alto nível para o pacote de teste.

test.support.**TEST_DATA_DIR**

Define o diretório `data` de dentro do pacote de teste.

test.support.**MAX_Py_ssize_t**

Define *sys.maxsize* para grandes testes de memória.

test.support.**max_memuse**

Set by *set_memlimit()* as the memory limit for big memory tests. Limited by *MAX_Py_ssize_t*.

test.support.**real_max_memuse**

Set by *set_memlimit()* as the memory limit for big memory tests. Not limited by *MAX_Py_ssize_t*.

test.support.**MISSING_C_DOCSTRINGS**

Set to `True` if Python is built without docstrings (the `WITH_DOC_STRINGS` macro is not defined). See the `configure --without-doc-strings` option.

See also the *HAVE_DOCSTRINGS* variable.

test.support.**HAVE_DOCSTRINGS**

Set to `True` if function docstrings are available. See the `python -OO` option, which strips docstrings of functions implemented in Python.

See also the *MISSING_C_DOCSTRINGS* variable.

test.support.**TEST_HTTP_URL**

Define the URL of a dedicated HTTP server for the network tests.

test.support.**ALWAYS_EQ**

Object that is equal to anything. Used to test mixed type comparison.

test.support.**NEVER_EQ**

Object that is not equal to anything (even to *ALWAYS_EQ*). Used to test mixed type comparison.

test.support.**LARGEST**

Object that is greater than anything (except itself). Used to test mixed type comparison.

test.support.**SMALLEST**

Object that is less than anything (except itself). Used to test mixed type comparison.

O módulo *test.support* define as seguintes funções:

test.support.**busy_retry**(*timeout*, *err_msg=None*, */*, ***, *error=True*)

Run the loop body until `break` stops the loop.

After *timeout* seconds, raise an *AssertionError* if *error* is true, or just stop the loop if *error* is false.

Exemplo:

```
for _ in support.busy_retry(support.SHORT_TIMEOUT):
    if check():
        break
```

Example of error=False usage:

```
for _ in support.busy_retry(support.SHORT_TIMEOUT, error=False):
    if check():
        break
else:
    raise RuntimeError('my custom error')
```

test.support.**sleeping_retry**(*timeout*, *err_msg=None*, */*, ***, *init_delay=0.010*, *max_delay=1.0*, *error=True*)

Wait strategy that applies exponential backoff.

Run the loop body until `break` stops the loop. Sleep at each loop iteration, but not at the first iteration. The sleep delay is doubled at each iteration (up to *max_delay* seconds).

See *busy_retry()* documentation for the parameters usage.

Example raising an exception after SHORT_TIMEOUT seconds:

```
for _ in support.sleeping_retry(support.SHORT_TIMEOUT):
    if check():
        break
```

Example of error=False usage:

```
for _ in support.sleeping_retry(support.SHORT_TIMEOUT, error=False):
    if check():
        break
else:
    raise RuntimeError('my custom error')
```

test.support.**is_resource_enabled**(*resource*)

Return `True` if *resource* is enabled and available. The list of available resources is only set when *test.regrtest* is executing the tests.

test.support.**python_is_optimized**()

Return `True` if Python was not built with `-O0` or `-Og`.

test.support.**with_pymalloc**()

Return `_testcapi.WITH_PYMALLOC`.

test.support.**requires**(*resource*, *msg=None*)

Raise *ResourceDenied* if *resource* is not available. *msg* is the argument to *ResourceDenied* if it is raised. Always returns `True` if called by a function whose `__name__` is `'__main__'`. Used when tests are executed by *test.regrtest*.

test.support.**sortdict**(*dict*)

Return a repr of *dict* with keys sorted.

test.support.**findfile**(*filename*, *subdir=None*)

Return the path to the file named *filename*. If no match is found *filename* is returned. This does not equal a failure since it could be the path to the file.

Setting *subdir* indicates a relative path to use to find the file rather than looking directly in the path directories.

test.support.**get_pagesize**()

Get size of a page in bytes.

Adicionado na versão 3.12.

test.support.**setswitchinterval**(*interval*)

Set the *sys.setswitchinterval()* to the given *interval*. Defines a minimum interval for Android systems to prevent the system from hanging.

test.support.**check_impl_detail**(***guards*)

Use this check to guard CPython's implementation-specific tests or to run them only on the implementations guarded by the arguments. This function returns `True` or `False` depending on the host platform. Example usage:

```
check_impl_detail()               # Only on CPython (default).
check_impl_detail(jython=True)    # Only on Jython.
check_impl_detail(cpython=False)  # Everywhere except CPython.
```

test.support.**set_memlimit**(*limit*)

Define os valores para *max_memuse* e *real_max_memuse* para grandes testes de memória.

test.support.**record_original_stdout**(*stdout*)

Armazena o valor de *stdout*. Destina-se a manter o stdout no momento em que o registro começou.

test.support.**get_original_stdout**()

Retorna o stdout original definido por *record_original_stdout()* ou `sys.stdout` se não estiver definido.

test.support.**args_from_interpreter_flags**()

Retorna uma lista de argumentos de linha de comando reproduzindo as configurações em `sys.flags` e `sys.warnoptions`.

test.support.**optim_args_from_interpreter_flags**()

Retorna a lista de argumentos da linha de comando reproduzindo as configurações de otimização atuais em `sys.flags`.

test.support.**captured_stdin**()

test.support.**captured_stdout**()

test.support.**captured_stderr**()

Um gerenciador de contexto que substitui temporariamente o fluxo nomeado pelo objeto *io.StringIO*.

Exemplo do uso com fluxos de saída:

```
with captured_stdout() as stdout, captured_stderr() as stderr:
    print("hello")
    print("error", file=sys.stderr)
assert stdout.getvalue() == "hello\n"
assert stderr.getvalue() == "error\n"
```

Exemplo de uso com fluxo de entrada:

```
with captured_stdin() as stdin:
    stdin.write('hello\n')
    stdin.seek(0)
    # call test code that consumes from sys.stdin
    captured = input()
self.assertEqual(captured, "hello")
```

test.support.**disable_faulthandler**()

Um gerenciador de contexto que desativa temporariamente *faulthandler*.

test.support.**gc_collect**()

Força a coleta de tantos objetos quanto possível. Isso é necessário porque a desalocação oportuna não é garantida pelo coletor de lixo. Isso significa que os métodos `__del__` podem ser chamados mais tarde do que o esperado e as referência fraca podem permanecer ativas por mais tempo do que o esperado.

test.support.**disable_gc**()

A context manager that disables the garbage collector on entry. On exit, the garbage collector is restored to its prior state.

test.support.**swap_attr**(*obj*, *attr*, *new_val*)

Context manager to swap out an attribute with a new object.

Uso:

```
with swap_attr(obj, "attr", 5):
    ...
```

This will set `obj.attr` to 5 for the duration of the `with` block, restoring the old value at the end of the block. If `attr` doesn't exist on `obj`, it will be created and then deleted at the end of the block.

The old value (or `None` if it doesn't exist) will be assigned to the target of the "as" clause, if there is one.

test.support.**swap_item**(*obj*, *attr*, *new_val*)

Context manager to swap out an item with a new object.

Uso:

```
with swap_item(obj, "item", 5):
    ...
```

This will set `obj["item"]` to 5 for the duration of the `with` block, restoring the old value at the end of the block. If `item` doesn't exist on `obj`, it will be created and then deleted at the end of the block.

The old value (or `None` if it doesn't exist) will be assigned to the target of the "as" clause, if there is one.

test.support.**flush_std_streams**()

Call the `flush()` method on *sys.stdout* and then on *sys.stderr*. It can be used to make sure that the logs order is consistent before writing into stderr.

Adicionado na versão 3.11.

test.support.**print_warning**(*msg*)

Print a warning into *sys.__stderr__*. Format the message as: `f"Warning -- {msg}"`. If *msg* is made of multiple lines, add `"Warning -- "` prefix to each line.

Adicionado na versão 3.9.

test.support.**wait_process**(*pid*, *, *exitcode*, *timeout=None*)

Wait until process *pid* completes and check that the process exit code is *exitcode*.

Raise an *AssertionError* if the process exit code is not equal to *exitcode*.

If the process runs longer than *timeout* seconds (*SHORT_TIMEOUT* by default), kill the process and raise an *AssertionError*. The timeout feature is not available on Windows.

Adicionado na versão 3.9.

test.support.**calcobjsize**(*fmt*)

Return the size of the `PyObject` whose structure members are defined by *fmt*. The returned value includes the size of the Python object header and alignment.

test.support.**calcvobjsize**(*fmt*)

Return the size of the `PyVarObject` whose structure members are defined by *fmt*. The returned value includes the size of the Python object header and alignment.

test.support.**checksizeof**(*test*, *o*, *size*)

For testcase *test*, assert that the `sys.getsizeof` for *o* plus the GC header size equals *size*.

@test.support.**anticipate_failure**(*condition*)

A decorator to conditionally mark tests with *unittest.expectedFailure()*. Any use of this decorator should have an associated comment identifying the relevant tracker issue.

test.support.**system_must_validate_cert**(*f*)

A decorator that skips the decorated test on TLS certification validation failures.

@test.support.**run_with_locale**(*catstr*, **locales*)

A decorator for running a function in a different locale, correctly resetting it after it has finished. *catstr* is the locale category as a string (for example `"LC_ALL"`). The *locales* passed will be tried sequentially, and the first valid locale will be used.

@test.support.**run_with_tz**(*tz*)

A decorator for running a function in a specific timezone, correctly resetting it after it has finished.

@test.support.**requires_freebsd_version**(**min_version*)

Decorator for the minimum version when running test on FreeBSD. If the FreeBSD version is less than the minimum, the test is skipped.

@test.support.**requires_linux_version**(**min_version*)

Decorator for the minimum version when running test on Linux. If the Linux version is less than the minimum, the test is skipped.

@test.support.**requires_mac_version**(**min_version*)

Decorator for the minimum version when running test on macOS. If the macOS version is less than the minimum, the test is skipped.

@test.support.**requires_gil_enabled**

Decorador para pular testes na construção com threads livres. Se a *GIL* estiver desabilitada, o teste será ignorado.

@test.support.**requires_IEEE_754**

Decorator for skipping tests on non-IEEE 754 platforms.

@test.support.**requires_zlib**

Decorator for skipping tests if *zlib* doesn't exist.

@test.support.**requires_gzip**

Decorator for skipping tests if *gzip* doesn't exist.

@test.support.**requires_bz2**

Decorator for skipping tests if *bz2* doesn't exist.

@test.support.**requires_lzma**

Decorator for skipping tests if *lzma* doesn't exist.

@test.support.**requires_resource**(*resource*)

Decorator for skipping tests if *resource* is not available.

@test.support.**requires_docstrings**

Decorator for only running the test if *HAVE_DOCSTRINGS*.

@test.support.**requires_limited_api**

Decorator for only running the test if Limited C API is available.

@test.support.**cpython_only**

Decorator for tests only applicable to CPython.

@test.support.**impl_detail**(*msg=None*, ***guards*)

Decorator for invoking *check_impl_detail()* on *guards*. If that returns `False`, then uses *msg* as the reason for skipping the test.

@test.support.**thread_unsafe**(*reason=None*)

Decorator for marking tests as thread-unsafe. This test always runs in one thread even when invoked with `--parallel-threads`.

@test.support.**no_tracing**

Decorator to temporarily turn off tracing for the duration of the test.

@test.support.**refcount_test**

Decorator for tests which involve reference counting. The decorator does not run the test if it is not run by CPython. Any trace function is unset for the duration of the test to prevent unexpected refcounts caused by the trace function.

@test.support.**bigmemtest**(*size*, *memuse*, *dry_run=True*)

Decorator for bigmem tests.

size is a requested size for the test (in arbitrary, test-interpreted units.) *memuse* is the number of bytes per unit for the test, or a good estimate of it. For example, a test that needs two byte buffers, of 4 GiB each, could be decorated with `@bigmemtest(size=_4G, memuse=2)`.

The *size* argument is normally passed to the decorated test method as an extra argument. If *dry_run* is `True`, the value passed to the test method may be less than the requested value. If *dry_run* is `False`, it means the test doesn't support dummy runs when `-M` is not specified.

`@test.support.`**`bigaddrspacetest`**

Decorador para testes que preenche o espaço do endereço.

`test.support.`**`linked_to_musl`**`()`

Return `False` if there is no evidence the interpreter was compiled with `musl`, otherwise return a version triple, either `(0, 0, 0)` if the version is unknown, or the actual version if it is known. Intended for use in `skip` decorators. `emscripten` and `wasi` are assumed to be compiled with `musl`; otherwise `platform.libc_ver` is checked.

`test.support.`**`check_syntax_error`**`(`*testcase*, *statement*, *errtext=''*, ***, *lineno=None*, *offset=None*`)`

Test for syntax errors in *statement* by attempting to compile *statement*. *testcase* is the *unittest* instance for the test. *errtext* is the regular expression which should match the string representation of the raised *SyntaxError*. If *lineno* is not `None`, compares to the line of the exception. If *offset* is not `None`, compares to the offset of the exception.

`test.support.`**`open_urlresource`**`(`*url*, **args*, ***kw*`)`

Abre *url*. Se falhar em abrir, levanta *TestFailed*.

`test.support.`**`reap_children`**`()`

Use this at the end of `test_main` whenever sub-processes are started. This will help ensure that no extra children (zombies) stick around to hog resources and create problems when looking for refleaks.

`test.support.`**`get_attribute`**`(`*obj*, *name*`)`

Get an attribute, raising *unittest.SkipTest* if *AttributeError* is raised.

`test.support.`**`catch_unraisable_exception`**`()`

Context manager catching unraisable exception using *sys.unraisablehook()*.

Storing the exception value (`cm.unraisable.exc_value`) creates a reference cycle. The reference cycle is broken explicitly when the context manager exits.

Storing the object (`cm.unraisable.object`) can resurrect it if it is set to an object which is being finalized. Exiting the context manager clears the stored object.

Uso:

```
with support.catch_unraisable_exception() as cm:
    # code creating an "unraisable exception"
    ...

    # check the unraisable exception: use cm.unraisable
    ...

# cm.unraisable attribute no longer exists at this point
# (to break a reference cycle)
```

Adicionado na versão 3.8.

`test.support.`**`load_package_tests`**`(`*pkg_dir*, *loader*, *standard_tests*, *pattern*`)`

Generic implementation of the *unittest* `load_tests` protocol for use in test packages. *pkg_dir* is the root directory of the package; *loader*, *standard_tests*, and *pattern* are the arguments expected by `load_tests`. In simple cases, the test package's `__init__.py` can be the following:

```
import os
from test.support import load_package_tests

def load_tests(*args):
    return load_package_tests(os.path.dirname(__file__), *args)
```

test.support.**detect_api_mismatch**(*ref_api*, *other_api*, *, *ignore=()*)

Returns the set of attributes, functions or methods of *ref_api* not found on *other_api*, except for a defined list of items to be ignored in this check specified in *ignore*.

By default this skips private attributes beginning with '_' but includes all magic methods, i.e. those starting and ending in '__'.

Adicionado na versão 3.5.

test.support.**patch**(*test_instance*, *object_to_patch*, *attr_name*, *new_value*)

Override *object_to_patch.attr_name* with *new_value*. Also add cleanup procedure to *test_instance* to restore *object_to_patch* for *attr_name*. The *attr_name* should be a valid attribute for *object_to_patch*.

test.support.**run_in_subinterp**(*code*)

Run *code* in subinterpreter. Raise *unittest.SkipTest* if *tracemalloc* is enabled.

test.support.**check_free_after_iterating**(*test*, *iter*, *cls*, *args=()*)

Assert instances of *cls* are deallocated after iterating.

test.support.**missing_compiler_executable**(*cmd_names=[]*)

Check for the existence of the compiler executables whose names are listed in *cmd_names* or all the compiler executables when *cmd_names* is empty and return the first missing executable or `None` when none is found missing.

test.support.**check__all__**(*test_case*, *module*, *name_of_module=None*, *extra=()*, *not_exported=()*)

Assert that the `__all__` variable of *module* contains all public names.

The module's public names (its API) are detected automatically based on whether they match the public name convention and were defined in *module*.

The *name_of_module* argument can specify (as a string or tuple thereof) what module(s) an API could be defined in order to be detected as a public API. One case for this is when *module* imports part of its public API from other modules, possibly a C backend (like `csv` and its `_csv`).

The *extra* argument can be a set of names that wouldn't otherwise be automatically detected as "public", like objects without a proper *__module__* attribute. If provided, it will be added to the automatically detected ones.

The *not_exported* argument can be a set of names that must not be treated as part of the public API even though their names indicate otherwise.

Exemplo de uso:

```
import bar
import foo
import unittest
from test import support

class MiscTestCase(unittest.TestCase):
    def test__all__(self):
        support.check__all__(self, foo)

class OtherTestCase(unittest.TestCase):
    def test__all__(self):
        extra = {'BAR_CONST', 'FOO_CONST'}
        not_exported = {'baz'}  # Undocumented name.
        # bar imports part of its API from _bar.
        support.check__all__(self, bar, ('bar', '_bar'),
                             extra=extra, not_exported=not_exported)
```

Adicionado na versão 3.6.

test.support.**skip_if_broken_multiprocessing_synchronize**()

Skip tests if the `multiprocessing.synchronize` module is missing, if there is no available semaphore implementation, or if creating a lock raises an *OSError*.

Adicionado na versão 3.10.

test.support.**check_disallow_instantiation**(*test_case*, *tp*, **args*, ***kwds*)

Assert that type *tp* cannot be instantiated using *args* and *kwds*.

Adicionado na versão 3.10.

test.support.**adjust_int_max_str_digits**(*max_digits*)

This function returns a context manager that will change the global *sys.set_int_max_str_digits()* setting for the duration of the context to allow execution of test code that needs a different limit on the number of digits when converting between an integer and string.

Adicionado na versão 3.11.

The *test.support* module defines the following classes:

class test.support.**SuppressCrashReport**

A context manager used to try to prevent crash dialog popups on tests that are expected to crash a subprocess.

No Windows, desativa as caixas de diálogo Relatório de Erros do Windows usando SetErrorMode <https://msdn.microsoft.com/en-us/library/windows/desktop/ms680621.aspx>_.

On UNIX, *resource.setrlimit()* is used to set *resource.RLIMIT_CORE*'s soft limit to 0 to prevent coredump file creation.

On both platforms, the old value is restored by `__exit__()`.

class test.support.**SaveSignals**

Class to save and restore signal handlers registered by the Python signal handler.

save(*self*)

Save the signal handlers to a dictionary mapping signal numbers to the current signal handler.

restore(*self*)

Set the signal numbers from the *save()* dictionary to the saved handler.

class test.support.**Matcher**

matches(*self*, *d*, ***kwargs*)

Tenta corresponder um único dicionário com os argumentos fornecidos.

match_value(*self*, *k*, *dv*, *v*)

Tente combinar um único valor armazenado (*dv*) com um valor fornecido (*v*).

26.10 `test.support.socket_helper` — Utilities for socket tests

The *test.support.socket_helper* module provides support for socket tests.

Adicionado na versão 3.9.

test.support.socket_helper.**IPV6_ENABLED**

Set to `True` if IPv6 is enabled on this host, `False` otherwise.

test.support.socket_helper.**find_unused_port**(*family=socket.AF_INET*, *socktype=socket.SOCK_STREAM*)

Returns an unused port that should be suitable for binding. This is achieved by creating a temporary socket with the same family and type as the `sock` parameter (default is *AF_INET*, *SOCK_STREAM*), and binding it to the specified host address (defaults to `0.0.0.0`) with the port set to 0, eliciting an unused ephemeral port from the OS. The temporary socket is then closed and deleted, and the ephemeral port is returned.

Either this method or *bind_port()* should be used for any tests where a server socket needs to be bound to a particular port for the duration of the test. Which one to use depends on whether the calling code is creating a Python socket, or if an unused port needs to be provided in a constructor or passed to an external program (i.e. the `-accept` argument to openssl's s_server mode). Always prefer *bind_port()* over *find_unused_port()* where possible. Using a hard coded port is discouraged since it can make multiple instances of the test impossible to run simultaneously, which is a problem for buildbots.

`test.support.socket_helper.`**bind_port**(*sock*, *host=HOST*)

Bind the socket to a free port and return the port number. Relies on ephemeral ports in order to ensure we are using an unbound port. This is important as many tests may be running simultaneously, especially in a buildbot environment. This method raises an exception if the `sock.family` is *AF_INET* and `sock.type` is *SOCK_STREAM*, and the socket has `SO_REUSEADDR` or `SO_REUSEPORT` set on it. Tests should never set these socket options for TCP/IP sockets. The only case for setting these options is testing multicasting via multiple UDP sockets.

Additionally, if the `SO_EXCLUSIVEADDRUSE` socket option is available (i.e. on Windows), it will be set on the socket. This will prevent anyone else from binding to our host/port for the duration of the test.

`test.support.socket_helper.`**bind_unix_socket**(*sock*, *addr*)

Bind a Unix socket, raising *unittest.SkipTest* if *PermissionError* is raised.

`@test.support.socket_helper.`**skip_unless_bind_unix_socket**

A decorator for running tests that require a functional `bind()` for Unix sockets.

`test.support.socket_helper.`**transient_internet**(*resource_name*, ***, *timeout=30.0*, *errnos=()*)

A context manager that raises *ResourceDenied* when various issues with the internet connection manifest themselves as exceptions.

26.11 `test.support.script_helper` — Utilities for the Python execution tests

The *test.support.script_helper* module provides support for Python's script execution tests.

`test.support.script_helper.`**interpreter_requires_environment**()

Return `True` if `sys.executable interpreter` requires environment variables in order to be able to run at all.

This is designed to be used with `@unittest.skipIf()` to annotate tests that need to use an `assert_python*()` function to launch an isolated mode (`-I`) or no environment mode (`-E`) sub-interpreter process.

A normal build & test does not run into this situation but it can happen when trying to run the standard library test suite from an interpreter that doesn't have an obvious home with Python's current home finding logic.

Setting `PYTHONHOME` is one way to get most of the testsuite to run in that situation. `PYTHONPATH` or `PYTHONUSERSITE` are other common environment variables that might impact whether or not the interpreter can start.

`test.support.script_helper.`**run_python_until_end**(**args*, ***env_vars*)

Set up the environment based on *env_vars* for running the interpreter in a subprocess. The values can include `__isolated`, `__cleanenv`, `__cwd`, and `TERM`.

Alterado na versão 3.9: The function no longer strips whitespaces from *stderr*.

`test.support.script_helper.`**assert_python_ok**(**args*, ***env_vars*)

Assert that running the interpreter with *args* and optional environment variables *env_vars* succeeds (`rc == 0`) and return a `(return code, stdout, stderr)` tuple.

If the *__cleanenv* keyword-only parameter is set, *env_vars* is used as a fresh environment.

Python is started in isolated mode (command line option `-I`), except if the *__isolated* keyword-only parameter is set to `False`.

Alterado na versão 3.9: The function no longer strips whitespaces from *stderr*.

test.support.script_helper.**assert_python_failure**(**args*, ***env_vars*)

Assert that running the interpreter with *args* and optional environment variables *env_vars* fails (`rc != 0`) and return a `(return code, stdout, stderr)` tuple.

See *assert_python_ok()* for more options.

Alterado na versão 3.9: The function no longer strips whitespaces from *stderr*.

test.support.script_helper.**spawn_python**(**args*, *stdout=subprocess.PIPE*, *stderr=subprocess.STDOUT*, ***kw*)

Run a Python subprocess with the given arguments.

kw is extra keyword args to pass to *subprocess.Popen()*. Returns a *subprocess.Popen* object.

test.support.script_helper.**kill_python**(*p*)

Run the given *subprocess.Popen* process until completion and return stdout.

test.support.script_helper.**make_script**(*script_dir*, *script_basename*, *source*, *omit_suffix=False*)

Cria um script contendo *source* no caminho *script_dir* e *script_basename*. Se *omit_suffix* `False`, acrescente `.py` ao nome. Retorna o caminho completo do script.

test.support.script_helper.**make_zip_script**(*zip_dir*, *zip_basename*, *script_name*, *name_in_zip=None*)

Cria um arquivo zip em *zip_dir* e *zip_basename* com a extensão `zip` que contém os arquivos em *script_name*. *name_in_zip* é o nome do arquivo. Retorna uma tupla contendo `(full path, full path of archive name)`.

test.support.script_helper.**make_pkg**(*pkg_dir*, *init_source=''*)

Cria um diretório nomeado *pkg_dir* contendo um arquivo `__init__` com *init_source* como seus conteúdos.

test.support.script_helper.**make_zip_pkg**(*zip_dir*, *zip_basename*, *pkg_name*, *script_basename*, *source*, *depth=1*, *compiled=False*)

Create a zip package directory with a path of *zip_dir* and *zip_basename* containing an empty `__init__` file and a file *script_basename* containing the *source*. If *compiled* is `True`, both source files will be compiled and added to the zip package. Return a tuple of the full zip path and the archive name for the zip file.

26.12 `test.support.bytecode_helper` — Ferramentas de suporte para testar a geração correta de bytecode

The *test.support.bytecode_helper* module provides support for testing and inspecting bytecode generation.

Adicionado na versão 3.9.

O módulo define a seguinte classe:

class test.support.bytecode_helper.**BytecodeTestCase**(*unittest.TestCase*)

This class has custom assertion methods for inspecting bytecode.

BytecodeTestCase.**get_disassembly_as_string**(*co*)

Return the disassembly of *co* as string.

BytecodeTestCase.**assertInBytecode**(*x*, *opname*, *argval=_UNSPECIFIED*)

Return instr if *opname* is found, otherwise throws *AssertionError*.

BytecodeTestCase.**assertNotInBytecode**(*x*, *opname*, *argval=_UNSPECIFIED*)

Throws *AssertionError* if *opname* is found.

26.13 test.support.threading_helper — Utilities for threading tests

The *test.support.threading_helper* module provides support for threading tests.

Adicionado na versão 3.10.

test.support.threading_helper.**join_thread**(*thread*, *timeout=None*)

Join a *thread* within *timeout*. Raise an *AssertionError* if thread is still alive after *timeout* seconds.

@test.support.threading_helper.**reap_threads**

Decorator to ensure the threads are cleaned up even if the test fails.

test.support.threading_helper.**start_threads**(*threads*, *unlock=None*)

Context manager to start *threads*, which is a sequence of threads. *unlock* is a function called after the threads are started, even if an exception was raised; an example would be *threading.Event.set()*. start_threads will attempt to join the started threads upon exit.

test.support.threading_helper.**threading_cleanup**(**original_values*)

Cleanup up threads not specified in *original_values*. Designed to emit a warning if a test leaves running threads in the background.

test.support.threading_helper.**threading_setup**()

Return current thread count and copy of dangling threads.

test.support.threading_helper.**wait_threads_exit**(*timeout=None*)

Context manager to wait until all threads created in the with statement exit.

test.support.threading_helper.**catch_threading_exception**()

Context manager catching *threading.Thread* exception using *threading.excepthook()*.

Attributes set when an exception is caught:

- exc_type
- exc_value
- exc_traceback
- thread

See *threading.excepthook()* documentation.

These attributes are deleted at the context manager exit.

Uso:

```
with threading_helper.catch_threading_exception() as cm:
    # code spawning a thread which raises an exception
    ...

    # check the thread exception, use cm attributes:
    # exc_type, exc_value, exc_traceback, thread
    ...

# exc_type, exc_value, exc_traceback, thread attributes of cm no longer
# exists at this point
# (to avoid reference cycles)
```

Adicionado na versão 3.8.

test.support.threading_helper.**run_concurrently**(*worker_func*, *nthreads*, *args=()*, *kwargs={}*)

Run the worker function concurrently in multiple threads. Re-raises an exception if any thread raises one, after all threads have finished.

26.14 `test.support.os_helper` — Utilities for os tests

The *test.support.os_helper* module provides support for os tests.

Adicionado na versão 3.10.

`test.support.os_helper.`**`FS_NONASCII`**

A non-ASCII character encodable by *os.fsencode()*.

`test.support.os_helper.`**`SAVEDCWD`**

Set to *os.getcwd()*.

`test.support.os_helper.`**`TESTFN`**

Set to a name that is safe to use as the name of a temporary file. Any temporary file that is created should be closed and unlinked (removed).

`test.support.os_helper.`**`TESTFN_NONASCII`**

Set to a filename containing the *FS_NONASCII* character, if it exists. This guarantees that if the filename exists, it can be encoded and decoded with the default filesystem encoding. This allows tests that require a non-ASCII filename to be easily skipped on platforms where they can't work.

`test.support.os_helper.`**`TESTFN_UNENCODABLE`**

Define o nome de arquivo (tipo str) que não pode ser codificado pela codificação do sistema de arquivos no modo estrito. Ele pode ser `None` se não for possível gerar como um nome de arquivo.

`test.support.os_helper.`**`TESTFN_UNDECODABLE`**

Define o nome de arquivo (tipo str) que não pode ser codificado pela codificação do sistema de arquivos no modo estrito. Ele pode ser `None` se não for possível ser gerado com um nome de arquivo.

`test.support.os_helper.`**`TESTFN_UNICODE`**

Define um nome não-ASCII para o arquivo temporário.

`class` `test.support.os_helper.`**`EnvironmentVarGuard`**

Class used to temporarily set or unset environment variables. Instances can be used as a context manager and have a complete dictionary interface for querying/modifying the underlying `os.environ`. After exit from the context manager all changes to environment variables done through this instance will be rolled back.

Alterado na versão 3.1: Adicionada uma interface para dicionário.

`class` `test.support.os_helper.`**`FakePath`**(*path*)

Simple *path-like object*. It implements the *__fspath__()* method which just returns the *path* argument. If *path* is an exception, it will be raised in `__fspath__()`.

`EnvironmentVarGuard.`**`set`**(*envvar*, *value*)

Temporariamente define a variável de ambiente `envvar` para o valor `value`.

`EnvironmentVarGuard.`**`unset`**(*envvar*, **others*)

Temporarily unset one or more environment variables.

Alterado na versão 3.14: More than one environment variable can be unset.

`test.support.os_helper.`**`can_symlink`**()

Return `True` if the OS supports symbolic links, `False` otherwise.

`test.support.os_helper.`**`can_xattr`**()

Return `True` if the OS supports xattr, `False` otherwise.

`test.support.os_helper.`**`change_cwd`**(*path*, *quiet=False*)

A context manager that temporarily changes the current working directory to *path* and yields the directory.

If *quiet* is `False`, the context manager raises an exception on error. Otherwise, it issues only a warning and keeps the current working directory the same.

test.support.os_helper.**create_empty_file**(*filename*)

Create an empty file with *filename*. If it already exists, truncate it.

test.support.os_helper.**fd_count**()

Conta o número de descritores de arquivos abertos.

test.support.os_helper.**fs_is_case_insensitive**(*directory*)

Return `True` if the file system for *directory* is case-insensitive.

test.support.os_helper.**make_bad_fd**()

Cria um descritor de arquivo inválido abrindo e fechando um arquivo temporário e retornando seu descritor.

test.support.os_helper.**rmdir**(*filename*)

Call *os.rmdir()* on *filename*. On Windows platforms, this is wrapped with a wait loop that checks for the existence of the file, which is needed due to antivirus programs that can hold files open and prevent deletion.

test.support.os_helper.**rmtree**(*path*)

Call *shutil.rmtree()* on *path* or call *os.lstat()* and *os.rmdir()* to remove a path and its contents. As with *rmdir()*, on Windows platforms this is wrapped with a wait loop that checks for the existence of the files.

@test.support.os_helper.**skip_unless_symlink**

Um decorador para executar testes que requerem suporte para links simbólicos.

@test.support.os_helper.**skip_unless_xattr**

Um decorador para execução de testes que requerem suporte para xattr.

test.support.os_helper.**temp_cwd**(*name='tempcwd'*, *quiet=False*)

Um gerenciador de contexto que cria temporariamente um novo diretório e altera o diretório de trabalho atual (CWD).

The context manager creates a temporary directory in the current directory with name *name* before temporarily changing the current working directory. If *name* is `None`, the temporary directory is created using *tempfile.mkdtemp()*.

Se *quiet* é `False` e ele não possibilita criar ou alterar o CWD, um erro é levantado. Por outro lado, somente um aviso surge e o CWD original é utilizado.

test.support.os_helper.**temp_dir**(*path=None*, *quiet=False*)

Um gerenciador de contexto que cria um diretório temporário no *path* e produz o diretório.

If *path* is `None`, the temporary directory is created using *tempfile.mkdtemp()*. If *quiet* is `False`, the context manager raises an exception on error. Otherwise, if *path* is specified and cannot be created, only a warning is issued.

test.support.os_helper.**temp_umask**(*umask*)

Um gerenciador de contexto que temporariamente define o umask do processo.

test.support.os_helper.**unlink**(*filename*)

Call *os.unlink()* on *filename*. As with *rmdir()*, on Windows platforms, this is wrapped with a wait loop that checks for the existence of the file.

26.15 test.support.import_helper — Utilities for import tests

The *test.support.import_helper* module provides support for import tests.

Adicionado na versão 3.10.

test.support.import_helper.**forget**(*module_name*)

Remove the module named *module_name* from `sys.modules` and delete any byte-compiled files of the module.

test.support.import_helper.**import_fresh_module**(*name*, *fresh=()*, *blocked=()*, *deprecated=False*)

This function imports and returns a fresh copy of the named Python module by removing the named module from `sys.modules` before doing the import. Note that unlike `reload()`, the original module is not affected by this operation.

fresh is an iterable of additional module names that are also removed from the `sys.modules` cache before doing the import.

blocked is an iterable of module names that are replaced with `None` in the module cache during the import to ensure that attempts to import them raise *ImportError*.

The named module and any modules named in the *fresh* and *blocked* parameters are saved before starting the import and then reinserted into `sys.modules` when the fresh import is complete.

Module and package deprecation messages are suppressed during this import if *deprecated* is `True`.

This function will raise *ImportError* if the named module cannot be imported.

Exemplo de uso:

```
# Get copies of the warnings module for testing without affecting the
# version being used by the rest of the test suite. One copy uses the
# C implementation, the other is forced to use the pure Python fallback
# implementation
py_warnings = import_fresh_module('warnings', blocked=['_warnings'])
c_warnings = import_fresh_module('warnings', fresh=['_warnings'])
```

Adicionado na versão 3.1.

test.support.import_helper.**import_module**(*name*, *deprecated=False*, ***, *required_on=()*)

This function imports and returns the named module. Unlike a normal import, this function raises *unittest.SkipTest* if the module cannot be imported.

Module and package deprecation messages are suppressed during this import if *deprecated* is `True`. If a module is required on a platform but optional for others, set *required_on* to an iterable of platform prefixes which will be compared against *sys.platform*.

Adicionado na versão 3.1.

test.support.import_helper.**modules_setup**()

Retorna a cópia de *sys.modules*.

test.support.import_helper.**modules_cleanup**(*oldmodules*)

Remove modules except for *oldmodules* and `encodings` in order to preserve internal cache.

test.support.import_helper.**unload**(*name*)

Exclui o *name* de `sys.modules`.

test.support.import_helper.**make_legacy_pyc**(*source*)

Move a **PEP 3147/PEP 488** pyc file to its legacy pyc location and return the file system path to the legacy pyc file. The *source* value is the file system path to the source file. It does not need to exist, however the PEP 3147/488 pyc file must exist.

class test.support.import_helper.**CleanImport**(**module_names*)

A context manager to force import to return a new module reference. This is useful for testing module-level behaviors, such as the emission of a *DeprecationWarning* on import. Example usage:

```
with CleanImport('foo'):
    importlib.import_module('foo')  # New reference.
```

class test.support.import_helper.**DirsOnSysPath**(**paths*)

A context manager to temporarily add directories to *sys.path*.

This makes a copy of *sys.path*, appends any directories given as positional arguments, then reverts *sys.path* to the copied settings when the context ends.

Note that *all* *sys.path* modifications in the body of the context manager, including replacement of the object, will be reverted at the end of the block.

26.16 test.support.warnings_helper — Utilities for warnings tests

The *test.support.warnings_helper* module provides support for warnings tests.

Adicionado na versão 3.10.

test.support.warnings_helper.**ignore_warnings**(*, *category*)

Suppress warnings that are instances of *category*, which must be *Warning* or a subclass. Roughly equivalent to *warnings.catch_warnings()* with *warnings.simplefilter('ignore', category=category)*. For example:

```
@warning_helper.ignore_warnings(category=DeprecationWarning)
def test_suppress_warning():
    # do something
```

Adicionado na versão 3.8.

test.support.warnings_helper.**check_no_resource_warning**(*testcase*)

Context manager to check that no *ResourceWarning* was raised. You must remove the object which may emit *ResourceWarning* before the end of the context manager.

test.support.warnings_helper.**check_syntax_warning**(*testcase*, *statement*, *errtext=''*, *, *lineno=1*, *offset=None*)

Test for syntax warning in *statement* by attempting to compile *statement*. Test also that the *SyntaxWarning* is emitted only once, and that it will be converted to a *SyntaxError* when turned into error. *testcase* is the *unittest* instance for the test. *errtext* is the regular expression which should match the string representation of the emitted *SyntaxWarning* and raised *SyntaxError*. If *lineno* is not `None`, compares to the line of the warning and exception. If *offset* is not `None`, compares to the offset of the exception.

Adicionado na versão 3.8.

test.support.warnings_helper.**check_warnings**(**filters*, *quiet=True*)

A convenience wrapper for *warnings.catch_warnings()* that makes it easier to test that a warning was correctly raised. It is approximately equivalent to calling `warnings.catch_warnings(record=True)` with *warnings.simplefilter()* set to `always` and with the option to automatically validate the results that are recorded.

`check_warnings` accepts 2-tuples of the form `("message regexp", WarningCategory)` as positional arguments. If one or more *filters* are provided, or if the optional keyword argument *quiet* is `False`, it checks to make sure the warnings are as expected: each specified filter must match at least one of the warnings raised by the enclosed code or the test fails, and if any warnings are raised that do not match any of the specified filters the test fails. To disable the first of these checks, set *quiet* to `True`.

Se nenhum argumento é especificado, o padrão é:

```
check_warnings(("", Warning), quiet=True)
```

Nesse caso, todos os avisos são capturados e nenhum erro é gerado.

On entry to the context manager, a `WarningRecorder` instance is returned. The underlying warnings list from *catch_warnings()* is available via the recorder object's *warnings* attribute. As a convenience, the attributes of the object representing the most recent warning can also be accessed directly through the recorder object (see example below). If no warning has been raised, then any of the attributes that would otherwise be expected on an object representing a warning will return `None`.

The recorder object also has a `reset()` method, which clears the warnings list.

O gerenciador de contexto é desenhado para ser utilizado dessa forma:

```
with check_warnings(("assertion is always true", SyntaxWarning),
                    ("", UserWarning)):
    exec('assert(False, "Hey!")')
    warnings.warn(UserWarning("Hide me!"))
```

No caso, se um aviso não foi levantado, ou algum outro aviso não foi levantado, *check_warnings()* deveria aparecer como um erro.

When a test needs to look more deeply into the warnings, rather than just checking whether or not they occurred, code like this can be used:

```
with check_warnings(quiet=True) as w:
    warnings.warn("foo")
    assert str(w.args[0]) == "foo"
    warnings.warn("bar")
    assert str(w.args[0]) == "bar"
    assert str(w.warnings[0].args[0]) == "foo"
    assert str(w.warnings[1].args[0]) == "bar"
    w.reset()
    assert len(w.warnings) == 0
```

Aqui todos os avisos serão capturados e o código de teste testa os avisos diretamente capturados.

Alterado na versão 3.2: Novos argumentos opcionais *filters* e *quiet*.

class test.support.warnings_helper.**WarningsRecorder**

Class used to record warnings for unit tests. See documentation of *check_warnings()* above for more details.

CAPÍTULO 27

Depuração e perfilamento

Essas bibliotecas ajudam no desenvolvimento do Python: o depurador permite que você percorra o código, analise os quadros de pilha e defina pontos de interrupção etc., e os criadores de perfil executam o código e fornecem uma análise detalhada dos tempos de execução, permitindo identificar gargalos em seus programas. Os eventos de auditoria fornecem visibilidade dos comportamentos de tempo de execução que, de outra forma, exigiriam depuração ou correção intrusiva.

27.1 Tabela de eventos de auditoria

Esta tabela contém todos os eventos levantados por chamadas de *sys.audit()* ou `PySys_Audit()` durante todo o tempo de execução do CPython e da biblioteca padrão. Essas chamadas foram adicionadas na versão 3.8 ou posterior (veja **PEP 578**).

Veja *sys.addaudithook()* e `PySys_AddAuditHook()` para informações sobre como tratar estes eventos.

Esta tabela é gerada a partir da documentação do CPython e pode não representar eventos levantados por outras implementações. Consulte a documentação específica de tempo de execução para obter eventos reais levantados.

Audit event	Arguments
_thread.start_new_thread	`function`, `args`, `kwargs`
array.__new__	`typecode`, `initializer`
builtins.breakpoint	`breakpointhook`
builtins.id	`id`
builtins.input	`prompt`
builtins.input/result	`result`
code.__new__	`code`, `filename`, `name`, `argcount`, `posonlyargcount`, `kwonlyargcount`, `nlocals`, `st`
compile	`source`, `filename`
cpython.PyConfig_Set	`name`, `value`
cpython.PyInterpreterState_Clear	
cpython.PyInterpreterState_New	
cpython._PySys_ClearAuditHooks	
cpython.remote_debugger_script	`script_path`
cpython.run_command	`command`
cpython.run_file	`filename`
cpython.run_interactivehook	`hook`
cpython.run_module	`module-name`

Tabela 1 – continuação da página anterior

Audit event	Arguments
cpython.run_startup	filename
cpython.run_stdin	
ctypes.addressof	obj
ctypes.call_function	func_pointer, arguments
ctypes.cdata	address
ctypes.cdata/buffer	pointer, size, offset
ctypes.create_string_buffer	init, size
ctypes.create_unicode_buffer	init, size
ctypes.dlopen	name
ctypes.dlsym	library, name
ctypes.dlsym/handle	handle, name
ctypes.get_errno	
ctypes.get_last_error	
ctypes.memoryview_at	address, size, readonly
ctypes.set_errno	errno
ctypes.set_exception	code
ctypes.set_last_error	error
ctypes.string_at	ptr, size
ctypes.wstring_at	ptr, size
ensurepip.bootstrap	root
exec	code_object
fcntl.fcntl	fd, cmd, arg
fcntl.flock	fd, operation
fcntl.ioctl	fd, request, arg
fcntl.lockf	fd, cmd, len, start, whence
ftplib.connect	self, host, port
ftplib.sendcmd	self, cmd
function.__new__	code
gc.get_objects	generation
gc.get_referents	objs
gc.get_referrers	objs
glob.glob	pathname, recursive
glob.glob/2	pathname, recursive, root_dir, dir_fd
http.client.connect	self, host, port
http.client.send	self, data
imaplib.open	self, host, port
imaplib.send	self, data
import	module, filename, sys.path, sys.meta_path, sys.path_hooks
marshal.dumps	value, version
marshal.load	
marshal.loads	bytes
mmap.__new__	fileno, length, access, offset
msvcrt.get_osfhandle	fd
msvcrt.locking	fd, mode, nbytes
msvcrt.open_osfhandle	handle, flags
object.__delattr__	obj, name
object.__getattr__	obj, name
object.__setattr__	obj, name, value
open	path, mode, flags
os.add_dll_directory	path
os.chdir	path
os.chflags	path, flags
os.chmod	path, mode, dir_fd
os.chown	path, uid, gid, dir_fd
os.exec	path, args, env

Tabela 1 – continuação da página anterior

Audit event	Arguments
os.fork	
os.forkpty	
os.fwalk	top, topdown, onerror, follow_symlinks, dir_fd
os.getxattr	path, attribute
os.kill	pid, sig
os.killpg	pgid, sig
os.link	src, dst, src_dir_fd, dst_dir_fd
os.listdir	path
os.listdrives	
os.listmounts	volume
os.listvolumes	
os.listxattr	path
os.lockf	fd, cmd, len
os.mkdir	path, mode, dir_fd
os.posix_spawn	path, argv, env
os.putenv	key, value
os.remove	path, dir_fd
os.removexattr	path, attribute
os.rename	src, dst, src_dir_fd, dst_dir_fd
os.rmdir	path, dir_fd
os.scandir	path
os.setxattr	path, attribute, value, flags
os.spawn	mode, path, args, env
os.startfile	path, operation
os.startfile/2	path, operation, arguments, cwd, show_cmd
os.symlink	src, dst, dir_fd
os.system	command
os.truncate	fd, length
os.unsetenv	key
os.utime	path, times, ns, dir_fd
os.walk	top, topdown, onerror, followlinks
pathlib.Path.glob	self, pattern
pathlib.Path.rglob	self, pattern
pdb.Pdb	
pickle.find_class	module, name
poplib.connect	self, host, port
poplib.putline	self, line
pty.spawn	argv
resource.prlimit	pid, resource, limits
resource.setrlimit	resource, limits
setopencodehook	
shutil.chown	path, user, group
shutil.copyfile	src, dst
shutil.copymode	src, dst
shutil.copystat	src, dst
shutil.copytree	src, dst
shutil.make_archive	base_name, format, root_dir, base_dir
shutil.move	src, dst
shutil.rmtree	path, dir_fd
shutil.unpack_archive	filename, extract_dir, format
signal.pthread_kill	thread_id, signalnum
smtplib.connect	self, host, port
smtplib.send	self, data
socket.__new__	self, family, type, protocol
socket.bind	self, address

Tabela 1 – continuação da página anterior

Audit event	Arguments
socket.connect	self, address
socket.getaddrinfo	host, port, family, type, protocol
socket.gethostbyaddr	ip_address
socket.gethostbyname	hostname
socket.gethostname	
socket.getnameinfo	sockaddr
socket.getservbyname	servicename, protocolname
socket.getservbyport	port, protocolname
socket.sendmsg	self, address
socket.sendto	self, address
socket.sethostname	name
sqlite3.connect	database
sqlite3.connect/handle	connection_handle
sqlite3.enable_load_extension	connection, enabled
sqlite3.load_extension	connection, path
subprocess.Popen	executable, args, cwd, env
sys._current_exceptions	
sys._current_frames	
sys._getframe	frame
sys._getframemodulename	depth
sys.addaudithook	
sys.excepthook	hook, type, value, traceback
sys.monitoring.register_callback	func
sys.remote_exec	pid
sys.set_asyncgen_hooks_finalizer	
sys.set_asyncgen_hooks_firstiter	
sys.setprofile	
sys.settrace	
sys.unraisablehook	hook, unraisable
syslog.closelog	
syslog.openlog	ident, logoption, facility
syslog.setlogmask	maskpri
syslog.syslog	priority, message
tempfile.mkdtemp	fullpath
tempfile.mkstemp	fullpath
time.sleep	secs
urllib.Request	fullurl, data, headers, method
webbrowser.open	url
winreg.ConnectRegistry	computer_name, key
winreg.CreateKey	key, sub_key, access
winreg.DeleteKey	key, sub_key, access
winreg.DeleteValue	key, value
winreg.DisableReflectionKey	key
winreg.EnableReflectionKey	key
winreg.EnumKey	key, index
winreg.EnumValue	key, index
winreg.ExpandEnvironmentStrings	str
winreg.LoadKey	key, sub_key, file_name
winreg.OpenKey	key, sub_key, access
winreg.OpenKey/result	key
winreg.PyHKEY.Detach	key
winreg.QueryInfoKey	key
winreg.QueryReflectionKey	key
winreg.QueryValue	key, sub_key, value_name
winreg.SaveKey	key, file_name

Tabela 1 – continuação da página anterior

Audit event	Arguments
winreg.SetValue	`key`, `sub_key`, `type`, `value`

Os eventos a seguir são levantados internamente e não correspondem a nenhuma API pública de CPython:

Evento de auditoria	Argumentos
_winapi.CreateFile	`file_name`, `desired_access`, `share_mode`, `creation_disposition`, `flags_and_attributes`
_winapi.CreateJunction	`src_path`, `dst_path`
_winapi.CreateNamedPipe	`name`, `open_mode`, `pipe_mode`
_winapi.CreatePipe	
_winapi.CreateProcess	`application_name`, `command_line`, `current_directory`
_winapi.OpenProcess	`process_id`, `desired_access`
_winapi.TerminateProcess	`handle`, `exit_code`
_posixsubprocess.fork_exec	`exec_list`, `args`, `env`
ctypes.PyObj_FromPtr	`obj`

Adicionado na versão 3.14: O evento de auditoria interno `_posixsubprocess.fork_exec`.

27.2 bdb — Framework do depurador

Código-fonte: Lib/bdb.py

O módulo *bdb* lida com funções básicas do depurador, como definir pontos de interrupção ou gerenciar a execução por meio do depurador.

A seguinte exceção é definida:

exception bdb.**BdbQuit**

Exceção levantada pela classe *Bdb* para sair do depurador.

O módulo *bdb* também define duas classes:

class bdb.**Breakpoint**(*self*, *file*, *line*, *temporary=False*, *cond=None*, *funcname=None*)

This class implements temporary breakpoints, ignore counts, disabling and (re-)enabling, and conditionals.

Breakpoints are indexed by number through a list called *bpbynumber* and by `(file, line)` pairs through *bplist*. The former points to a single instance of class *Breakpoint*. The latter points to a list of such instances since there may be more than one breakpoint per line.

When creating a breakpoint, its associated *file name* should be in canonical form. If a *funcname* is defined, a breakpoint *hit* will be counted when the first line of that function is executed. A *conditional* breakpoint always counts a *hit*.

Breakpoint instances have the following methods:

deleteMe()

Delete the breakpoint from the list associated to a file/line. If it is the last breakpoint in that position, it also deletes the entry for the file/line.

enable()
Mark the breakpoint as enabled.

disable()
Mark the breakpoint as disabled.

bpformat()
Return a string with all the information about the breakpoint, nicely formatted:

- Breakpoint number.
- Temporary status (del or keep).
- File/line position.
- Break condition.
- Número de vezes para ignorar.
- Number of times hit.

Adicionado na versão 3.2.

bpprint(*out=None*)
Print the output of *bpformat()* to the file *out*, or if it is None, to standard output.

Breakpoint instances have the following attributes:

file
Nome do arquivo do *Breakpoint*.

line
Line number of the *Breakpoint* within *file*.

temporary
True if a *Breakpoint* at (file, line) is temporary.

cond
Condition for evaluating a *Breakpoint* at (file, line).

funcname
Function name that defines whether a *Breakpoint* is hit upon entering the function.

enabled
True if *Breakpoint* is enabled.

bpbynumber
Numeric index for a single instance of a *Breakpoint*.

bplist
Dictionary of *Breakpoint* instances indexed by (*file*, *line*) tuples.

ignore
Number of times to ignore a *Breakpoint*.

hits
Count of the number of times a *Breakpoint* has been hit.

class bdb.**Bdb**(*skip=None*, *backend='settrace'*)
The *Bdb* class acts as a generic Python debugger base class.

This class takes care of the details of the trace facility; a derived class should implement user interaction. The standard debugger class (*pdb.Pdb*) is an example.

The *skip* argument, if given, must be an iterable of glob-style module name patterns. The debugger will not step into frames that originate in a module that matches one of these patterns. Whether a frame is considered to originate in a certain module is determined by the __name__ in the frame globals.

The *backend* argument specifies the backend to use for *Bdb*. It can be either `'settrace'` or `'monitoring'`. `'settrace'` uses *sys.settrace()* which has the best backward compatibility. The `'monitoring'` backend uses the new *sys.monitoring* that was introduced in Python 3.12, which can be much more efficient because it can disable unused events. We are trying to keep the exact interfaces for both backends, but there are some differences. The debugger developers are encouraged to use the `'monitoring'` backend to achieve better performance.

Alterado na versão 3.1: Adicionado o parâmetro *skip*.

Alterado na versão 3.14: Added the *backend* parameter.

The following methods of *Bdb* normally don't need to be overridden.

canonic(*filename*)

Return canonical form of *filename*.

For real file names, the canonical form is an operating-system-dependent, *case-normalized absolute path*. A *filename* with angle brackets, such as `"<stdin>"` generated in interactive mode, is returned unchanged.

start_trace(*self*)

Start tracing. For `'settrace'` backend, this method is equivalent to `sys.settrace(self.trace_dispatch)`

Adicionado na versão 3.14.

stop_trace(*self*)

Stop tracing. For `'settrace'` backend, this method is equivalent to `sys.settrace(None)`

Adicionado na versão 3.14.

reset()

Set the `botframe`, `stopframe`, `returnframe` and *quitting* attributes with values ready to start debugging.

trace_dispatch(*frame*, *event*, *arg*)

This function is installed as the trace function of debugged frames. Its return value is the new trace function (in most cases, that is, itself).

The default implementation decides how to dispatch a frame, depending on the type of event (passed as a string) that is about to be executed. *event* can be one of the following:

- `"line"`: A new line of code is going to be executed.
- `"call"`: A function is about to be called, or another code block entered.
- `"return"`: A function or other code block is about to return.
- `"exception"`: An exception has occurred.
- `"c_call"`: A C function is about to be called.
- `"c_return"`: A C function has returned.
- `"c_exception"`: A C function has raised an exception.

For the Python events, specialized functions (see below) are called. For the C events, no action is taken.

The *arg* parameter depends on the previous event.

See the documentation for *sys.settrace()* for more information on the trace function. For more information on code and frame objects, refer to types.

dispatch_line(*frame*)

If the debugger should stop on the current line, invoke the *user_line()* method (which should be overridden in subclasses). Raise a *BdbQuit* exception if the *quitting* flag is set (which can be set from *user_line()*). Return a reference to the *trace_dispatch()* method for further tracing in that scope.

dispatch_call(*frame*, *arg*)

If the debugger should stop on this function call, invoke the *user_call()* method (which should be overridden in subclasses). Raise a *BdbQuit* exception if the *quitting* flag is set (which can be set from *user_call()*). Return a reference to the *trace_dispatch()* method for further tracing in that scope.

dispatch_return(*frame*, *arg*)

If the debugger should stop on this function return, invoke the *user_return()* method (which should be overridden in subclasses). Raise a *BdbQuit* exception if the *quitting* flag is set (which can be set from *user_return()*). Return a reference to the *trace_dispatch()* method for further tracing in that scope.

dispatch_exception(*frame*, *arg*)

If the debugger should stop at this exception, invokes the *user_exception()* method (which should be overridden in subclasses). Raise a *BdbQuit* exception if the *quitting* flag is set (which can be set from *user_exception()*). Return a reference to the *trace_dispatch()* method for further tracing in that scope.

Normally derived classes don't override the following methods, but they may if they want to redefine the definition of stopping and breakpoints.

is_skipped_line(*module_name*)

Return `True` if *module_name* matches any skip pattern.

stop_here(*frame*)

Return `True` if *frame* is below the starting frame in the stack.

break_here(*frame*)

Return `True` if there is an effective breakpoint for this line.

Check whether a line or function breakpoint exists and is in effect. Delete temporary breakpoints based on information from *effective()*.

break_anywhere(*frame*)

Return `True` if any breakpoint exists for *frame*'s filename.

Derived classes should override these methods to gain control over debugger operation.

user_call(*frame*, *argument_list*)

Called from *dispatch_call()* if a break might stop inside the called function.

argument_list is not used anymore and will always be `None`. The argument is kept for backwards compatibility.

user_line(*frame*)

Called from *dispatch_line()* when either *stop_here()* or *break_here()* returns `True`.

user_return(*frame*, *return_value*)

Called from *dispatch_return()* when *stop_here()* returns `True`.

user_exception(*frame*, *exc_info*)

Called from *dispatch_exception()* when *stop_here()* returns `True`.

do_clear(*arg*)

Handle how a breakpoint must be removed when it is a temporary one.

This method must be implemented by derived classes.

Derived classes and clients can call the following methods to affect the stepping state.

set_step()

Stop after one line of code.

set_next(*frame*)

Stop on the next line in or below the given frame.

set_return(*frame*)

Stop when returning from the given frame.

set_until(*frame*, *lineno=None*)

Stop when the line with the *lineno* greater than the current one is reached or when returning from current frame.

set_trace([*frame*])

Start debugging from *frame*. If *frame* is not specified, debugging starts from caller's frame.

Alterado na versão 3.13: *set_trace()* entrará no depurador imediatamente, em vez de na próxima linha de código a ser executada.

set_continue()

Stop only at breakpoints or when finished. If there are no breakpoints, set the system trace function to `None`.

set_quit()

Set the `quitting` attribute to `True`. This raises *BdbQuit* in the next call to one of the `dispatch_*()` methods.

Derived classes and clients can call the following methods to manipulate breakpoints. These methods return a string containing an error message if something went wrong, or `None` if all is well.

set_break(*filename*, *lineno*, *temporary=False*, *cond=None*, *funcname=None*)

Set a new breakpoint. If the *lineno* line doesn't exist for the *filename* passed as argument, return an error message. The *filename* should be in canonical form, as described in the *canonic()* method.

clear_break(*filename*, *lineno*)

Delete the breakpoints in *filename* and *lineno*. If none were set, return an error message.

clear_bpbynumber(*arg*)

Delete the breakpoint which has the index *arg* in the *Breakpoint.bpbynumber*. If *arg* is not numeric or out of range, return an error message.

clear_all_file_breaks(*filename*)

Delete all breakpoints in *filename*. If none were set, return an error message.

clear_all_breaks()

Delete all existing breakpoints. If none were set, return an error message.

get_bpbynumber(*arg*)

Return a breakpoint specified by the given number. If *arg* is a string, it will be converted to a number. If *arg* is a non-numeric string, if the given breakpoint never existed or has been deleted, a *ValueError* is raised.

Adicionado na versão 3.2.

get_break(*filename*, *lineno*)

Return `True` if there is a breakpoint for *lineno* in *filename*.

get_breaks(*filename*, *lineno*)

Return all breakpoints for *lineno* in *filename*, or an empty list if none are set.

get_file_breaks(*filename*)

Return all breakpoints in *filename*, or an empty list if none are set.

get_all_breaks()

Return all breakpoints that are set.

Derived classes and clients can call the following methods to disable and restart events to achieve better performance. These methods only work when using the `'monitoring'` backend.

disable_current_event()

Disable the current event until the next time *restart_events()* is called. This is helpful when the debugger is not interested in the current line.

Adicionado na versão 3.14.

restart_events()

Restart all the disabled events. This function is automatically called in `dispatch_*` methods after `user_*` methods are called. If the `dispatch_*` methods are not overridden, the disabled events will be restarted after each user interaction.

Adicionado na versão 3.14.

Derived classes and clients can call the following methods to get a data structure representing a stack trace.

get_stack(*f*, *t*)

Return a list of (frame, lineno) tuples in a stack trace, and a size.

The most recently called frame is last in the list. The size is the number of frames below the frame where the debugger was invoked.

format_stack_entry(*frame_lineno*, *lprefix=': '*)

Return a string with information about a stack entry, which is a `(frame, lineno)` tuple. The return string contains:

- The canonical filename which contains the frame.
- O nome da função ou `"<lambda>"`.
- O argumento de entrada.
- The return value.
- The line of code (if it exists).

Os dois métodos a seguir podem ser chamados pelos clientes para usar um depurador e depurar uma *instrução*, fornecida como uma string.

run(*cmd*, *globals=None*, *locals=None*)

Debug a statement executed via the *exec()* function. *globals* defaults to `__main__.__dict__`, *locals* defaults to *globals*.

runeval(*expr*, *globals=None*, *locals=None*)

Debug an expression executed via the *eval()* function. *globals* and *locals* have the same meaning as in *run()*.

runctx(*cmd*, *globals*, *locals*)

For backwards compatibility. Calls the *run()* method.

runcall(*func*, */*, **args*, ***kwds*)

Debug a single function call, and return its result.

Finally, the module defines the following functions:

bdb.**checkfuncname**(*b*, *frame*)

Return `True` if we should break here, depending on the way the *Breakpoint* *b* was set.

If it was set via line number, it checks if *b.line* is the same as the one in *frame*. If the breakpoint was set via *function name*, we have to check we are in the right *frame* (the right function) and if we are on its first executable line.

bdb.**effective**(*file*, *line*, *frame*)

Return `(active breakpoint, delete temporary flag)` or `(None, None)` as the breakpoint to act upon.

The *active breakpoint* is the first entry in *bplist* for the (*file*, *line*) (which must exist) that is *enabled*, for which *checkfuncname()* is true, and that has neither a false *condition* nor positive *ignore* count. The *flag*, meaning that a temporary breakpoint should be deleted, is `False` only when the *cond* cannot be evaluated (in which case, *ignore* count is ignored).

If no such entry exists, then `(None, None)` is returned.

bdb.**set_trace**()

Start debugging with a *Bdb* instance from caller's frame.

27.3 `faulthandler` — Dump the Python traceback

Adicionado na versão 3.3.

This module contains functions to dump Python tracebacks explicitly, on a fault, after a timeout, or on a user signal. Call *faulthandler.enable()* to install fault handlers for the *SIGSEGV*, *SIGFPE*, *SIGABRT*, *SIGBUS*, and *SIGILL* signals. You can also enable them at startup by setting the `PYTHONFAULTHANDLER` environment variable or by using the `-X faulthandler` command line option.

The fault handler is compatible with system fault handlers like Apport or the Windows fault handler. The module uses an alternative stack for signal handlers if the `sigaltstack()` function is available. This allows it to dump the traceback even on a stack overflow.

The fault handler is called on catastrophic cases and therefore can only use signal-safe functions (e.g. it cannot allocate memory on the heap). Because of this limitation traceback dumping is minimal compared to normal Python tracebacks:

- Only ASCII is supported. The `backslashreplace` error handler is used on encoding.
- Each string is limited to 500 characters.
- Only the filename, the function name and the line number are displayed. (no source code)
- It is limited to 100 frames and 100 threads.
- The order is reversed: the most recent call is shown first.

By default, the Python traceback is written to *sys.stderr*. To see tracebacks, applications must be run in the terminal. A log file can alternatively be passed to *faulthandler.enable()*.

The module is implemented in C, so tracebacks can be dumped on a crash or when Python is deadlocked.

The *Python Development Mode* calls *faulthandler.enable()* at Python startup.

Ver também

Module *pdb*
Interactive source code debugger for Python programs.

Módulo *traceback*
Interface padrão para extrair, formatar e imprimir rastreamentos de pilha de programas Python.

27.3.1 Dumping the traceback

faulthandler.**dump_traceback**(*file=sys.stderr*, *all_threads=True*)

Dump the tracebacks of all threads into *file*. If *all_threads* is `False`, dump only the current thread.

> **Ver também**
>
> *traceback.print_tb()*, which can be used to print a traceback object.

Alterado na versão 3.5: Added support for passing file descriptor to this function.

27.3.2 Dumping the C stack

Adicionado na versão 3.14.

faulthandler.**dump_c_stack**(*file=sys.stderr*)

Dump the C stack trace of the current thread into *file*.

If the Python build does not support it or the operating system does not provide a stack trace, then this prints an error in place of a dumped C stack.

C Stack Compatibility

If the system does not support the C-level *backtrace(3)* or *dladdr1(3)*, then C stack dumps will not work. An error will be printed instead of the stack.

Additionally, some compilers do not support *CPython's* implementation of C stack dumps. As a result, a different error may be printed instead of the stack, even if the operating system supports dumping stacks.

> **Nota**
>
> Dumping C stacks can be arbitrarily slow, depending on the DWARF level of the binaries in the call stack.

27.3.3 Fault handler state

faulthandler.**enable**(*file=sys.stderr*, *all_threads=True*, *c_stack=True*)

Enable the fault handler: install handlers for the *SIGSEGV*, *SIGFPE*, *SIGABRT*, *SIGBUS* and *SIGILL* signals to dump the Python traceback. If *all_threads* is `True`, produce tracebacks for every running thread. Otherwise, dump only the current thread.

The *file* must be kept open until the fault handler is disabled: see *issue with file descriptors*.

If *c_stack* is `True`, then the C stack trace is printed after the Python traceback, unless the system does not support it. See *dump_c_stack()* for more information on compatibility.

Alterado na versão 3.5: Added support for passing file descriptor to this function.

Alterado na versão 3.6: On Windows, a handler for Windows exception is also installed.

Alterado na versão 3.10: The dump now mentions if a garbage collector collection is running if *all_threads* is true.

Alterado na versão 3.14: Only the current thread is dumped if the *GIL* is disabled to prevent the risk of data races.

Alterado na versão 3.14: The dump now displays the C stack trace if *c_stack* is true.

faulthandler.**disable**()

Disable the fault handler: uninstall the signal handlers installed by *enable()*.

faulthandler.**is_enabled**()

Check if the fault handler is enabled.

27.3.4 Dumping the tracebacks after a timeout

faulthandler.**dump_traceback_later**(*timeout*, *repeat=False*, *file=sys.stderr*, *exit=False*)

Dump the tracebacks of all threads, after a timeout of *timeout* seconds, or every *timeout* seconds if *repeat* is `True`. If *exit* is `True`, call `_exit()` with status=1 after dumping the tracebacks. (Note `_exit()` exits the process immediately, which means it doesn't do any cleanup like flushing file buffers.) If the function is called twice, the new call replaces previous parameters and resets the timeout. The timer has a sub-second resolution.

The *file* must be kept open until the traceback is dumped or *cancel_dump_traceback_later()* is called: see *issue with file descriptors*.

This function is implemented using a watchdog thread.

Alterado na versão 3.5: Added support for passing file descriptor to this function.

Alterado na versão 3.7: This function is now always available.

faulthandler.**cancel_dump_traceback_later**()

Cancel the last call to *dump_traceback_later()*.

27.3.5 Dumping the traceback on a user signal

faulthandler.**register**(*signum*, *file=sys.stderr*, *all_threads=True*, *chain=False*)

Register a user signal: install a handler for the *signum* signal to dump the traceback of all threads, or of the current thread if *all_threads* is `False`, into *file*. Call the previous handler if chain is `True`.

The *file* must be kept open until the signal is unregistered by *unregister()*: see *issue with file descriptors*.

Não disponível no Windows.

Alterado na versão 3.5: Added support for passing file descriptor to this function.

faulthandler.**unregister**(*signum*)

Unregister a user signal: uninstall the handler of the *signum* signal installed by *register()*. Return `True` if the signal was registered, `False` otherwise.

Não disponível no Windows.

27.3.6 Issue with file descriptors

enable(), *dump_traceback_later()* and *register()* keep the file descriptor of their *file* argument. If the file is closed and its file descriptor is reused by a new file, or if *os.dup2()* is used to replace the file descriptor, the traceback will be written into a different file. Call these functions again each time that the file is replaced.

27.3.7 Exemplo

Example of a segmentation fault on Linux with and without enabling the fault handler:

```
$ python -c "import ctypes; ctypes.string_at(0)"
Segmentation fault

$ python -q -X faulthandler
>>> import ctypes
>>> ctypes.string_at(0)
Fatal Python error: Segmentation fault

Current thread 0x00007fb899f39700 (most recent call first):
  File "/opt/python/Lib/ctypes/__init__.py", line 486 in string_at
  File "<stdin>", line 1 in <module>

Current thread's C stack trace (most recent call first):
  Binary file "/opt/python/python", at _Py_DumpStack+0x42 [0x5b27f7d7147e]
```

(continua na próxima página)

(continuação da página anterior)

```
  Binary file "/opt/python/python", at +0x32dcbd [0x5b27f7d85cbd]
  Binary file "/opt/python/python", at +0x32df8a [0x5b27f7d85f8a]
  Binary file "/usr/lib/libc.so.6", at +0x3def0 [0x77b73226bef0]
  Binary file "/usr/lib/libc.so.6", at +0x17ef9c [0x77b7323acf9c]
  Binary file "/opt/python/build/lib.linux-x86_64-3.14/_ctypes.cpython-314d-x86_64-
↪linux-gnu.so", at +0xcdf6 [0x77b7315dddf6]
  Binary file "/usr/lib/libffi.so.8", at +0x7976 [0x77b73158f976]
  Binary file "/usr/lib/libffi.so.8", at +0x413c [0x77b73158c13c]
  Binary file "/usr/lib/libffi.so.8", at ffi_call+0x12e [0x77b73158ef0e]
  Binary file "/opt/python/build/lib.linux-x86_64-3.14/_ctypes.cpython-314d-x86_64-
↪linux-gnu.so", at +0x15a33 [0x77b7315e6a33]
  Binary file "/opt/python/build/lib.linux-x86_64-3.14/_ctypes.cpython-314d-x86_64-
↪linux-gnu.so", at +0x164fa [0x77b7315e74fa]
  Binary file "/opt/python/build/lib.linux-x86_64-3.14/_ctypes.cpython-314d-x86_64-
↪linux-gnu.so", at +0xc624 [0x77b7315dd624]
  Binary file "/opt/python/python", at _PyObject_MakeTpCall+0xce [0x5b27f7b73883]
  Binary file "/opt/python/python", at +0x11bab6 [0x5b27f7b73ab6]
  Binary file "/opt/python/python", at PyObject_Vectorcall+0x23 [0x5b27f7b73b04]
  Binary file "/opt/python/python", at _PyEval_EvalFrameDefault+0x490c␣
↪[0x5b27f7cbb302]
  Binary file "/opt/python/python", at +0x2818e6 [0x5b27f7cd98e6]
  Binary file "/opt/python/python", at +0x281aab [0x5b27f7cd9aab]
  Binary file "/opt/python/python", at PyEval_EvalCode+0xc5 [0x5b27f7cd9ba3]
  Binary file "/opt/python/python", at +0x255957 [0x5b27f7cad957]
  Binary file "/opt/python/python", at +0x255ab4 [0x5b27f7cadab4]
  Binary file "/opt/python/python", at _PyEval_EvalFrameDefault+0x6c3e␣
↪[0x5b27f7cbd634]
  Binary file "/opt/python/python", at +0x2818e6 [0x5b27f7cd98e6]
  Binary file "/opt/python/python", at +0x281aab [0x5b27f7cd9aab]
  Binary file "/opt/python/python", at +0x11b6e1 [0x5b27f7b736e1]
  Binary file "/opt/python/python", at +0x11d348 [0x5b27f7b75348]
  Binary file "/opt/python/python", at +0x11d626 [0x5b27f7b75626]
  Binary file "/opt/python/python", at PyObject_Call+0x20 [0x5b27f7b7565e]
  Binary file "/opt/python/python", at +0x32a67a [0x5b27f7d8267a]
  Binary file "/opt/python/python", at +0x32a7f8 [0x5b27f7d827f8]
  Binary file "/opt/python/python", at +0x32ac1b [0x5b27f7d82c1b]
  Binary file "/opt/python/python", at Py_RunMain+0x31 [0x5b27f7d82ebe]
  <truncated rest of calls>
Segmentation fault
```

27.4 pdb — O depurador do Python

Código-fonte: Lib/pdb.py

O módulo *pdb* define um depurador de código-fonte interativo para programas Python. Ele possui suporte a definição de pontos de interrupção (condicionais) e passo único no nível da linha de origem, inspeção de quadros de pilha, listagem de código-fonte e avaliação de código Python arbitrário no contexto de qualquer quadro de pilha. Ele também tem suporte a depuração *post-mortem* e pode ser chamado sob controle do programa.

O depurador é extensível – na verdade, ele é definido como a classe *Pdb*. Atualmente, isso não está documentado, mas é facilmente compreendido pela leitura do código-fonte. A interface de extensão usa os módulos *bdb* e *cmd*.

Ver também

Módulo *faulthandler*
: Usado para despejar tracebacks (situação da pilha de execução) do Python explicitamente, em uma falha, após um tempo limite ou em um sinal do usuário.

Módulo *traceback*
: Interface padrão para extrair, formatar e imprimir rastreamentos de pilha de programas Python.

O uso típico para entrar no depurador é inserir:

```
import pdb; pdb.set_trace()
```

ou:

```
breakpoint()
```

no local que você deseja entrar o depurador e, em seguida, executar o programa. Você pode percorrer o código seguindo esta instrução e continuar executando sem o depurador usando o comando *continue*.

Alterado na versão 3.7: A função embutida *breakpoint()*, quando chamada com valores padrão, pode ser usada em vez de `import pdb; pdb.set_trace()`.

```
def double(x):
   breakpoint()
   return x * 2
val = 3
print(f"{val} * 2 is {double(val)}")
```

O prompt do depurador é `(Pdb)`, que é o indicador de que você está no modo de depuração:

```
> ...(2)double()
-> breakpoint()
(Pdb) p x
3
(Pdb) continue
3 * 2 is 6
```

Alterado na versão 3.3: O preenchimento por tabulação através do módulo *readline* está disponível para comandos e argumentos de comando, por exemplo os nomes globais e locais atuais são oferecidos como argumentos do comando `p`. Você também pode invocar *pdb* na linha de comando para depurar outros scripts. Por exemplo:

```
python -m pdb [-c comando] (-m módulo | -p pid | arquivo_python) [argumentos ...]
```

Quando invocado como um módulo, o pdb entra automaticamente na depuração *post-mortem* se o programa que está sendo depurado for encerrado de forma anormal. Após a depuração *post-mortem* (ou após a saída normal do programa), o pdb reiniciará o programa. A reinicialização automática preserva o estado do pdb (p.ex., pontos de interrupção) e, na maioria dos casos, é mais útil do que encerrar o depurador na saída do programa.

-c, --command <command>
: Para executar comandos como se fossem dados em um arquivo `.pdbrc`. Veja *Comandos de depuração*.

  Alterado na versão 3.2: Adicionada a opção `-c`.

-m <module>
: Para executar módulos de maneira similar ao `python -m`. Assim como em um script, o depurador pausará a execução logo antes da primeira linha do módulo.

  Alterado na versão 3.7: Adicionada a opção `-m`.

-p, --pid <pid>
: Anexa ao processo com o PID especificado.

Adicionado na versão 3.14.

Para anexar a um processo Python em execução para depuração remota, use a opção `-p` ou `--pid` com o PID do processo de destino:

```
python -m pdb -p 1234
```

Nota

Anexar a um processo remoto bloqueado em uma chamada de sistema ou aguardando E/S só funcionará quando a próxima instrução de bytecode for executada ou quando o processo receber um sinal.

O uso típico para executar uma instrução sob o controle do depurador é:

```
>>> import pdb
>>> def f(x):
...     print(1 / x)
>>> pdb.run("f(2)")
> <string>(1)<module>()
(Pdb) continue
0.5
>>>
```

O uso típico para inspecionar um programa com falha é:

```
>>> import pdb
>>> def f(x):
...     print(1 / x)
...
>>> f(0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in f
ZeroDivisionError: division by zero
>>> pdb.pm()
> <stdin>(2)f()
(Pdb) p x
0
(Pdb)
```

Alterado na versão 3.13: A implementação da **PEP 667** significa que atribuições de nomes feitas via `pdb` afetarão imediatamente o escopo ativo, mesmo quando executado dentro de um *escopo otimizado*.

O módulo define as seguintes funções; cada uma entra no depurador de uma maneira ligeiramente diferente:

`pdb.`**`run`**(*statement*, *globals=None*, *locals=None*)

Executa a instrução *statement* (fornecida como uma string ou um objeto código) sob controle do depurador. O prompt do depurador aparece antes de qualquer código ser executado; você pode definir pontos de interrupção e digitar `continue` ou pode percorrer a instrução usando `step` ou `next` (todos esses comandos são explicados abaixo). Os argumentos opcionais *globals* e *locals* especificam o ambiente em que o código é executado; por padrão, o dicionário do módulo `__main__` é usado. (Veja a explicação das funções embutidas `exec()` ou `eval()`.)

`pdb.`**`runeval`**(*expression*, *globals=None*, *locals=None*)

Avalia a expressão *expression* (fornecida como uma string ou um objeto código) sob controle do depurador. Quando `runeval()` retorna, ele retorna o valor de *expression*. Caso contrário, esta função é semelhante a `run()`.

pdb.**runcall**(*function*, **args*, ***kwds*)

Chama a função *function* (um objeto função ou método, não uma string) com os argumentos fornecidos. Quando *runcall()* retorna, ele retorna qualquer que seja a chamada de função retornada. O prompt do depurador aparece assim que a função é inserida.

pdb.**set_trace**(*, *header=None*, *commands=None*)

Entra no depurador no quadro da pilha de chamada. Isso é útil para codificar um ponto de interrupção em um determinado ponto de um programa, mesmo que o código não esteja sendo depurado de outra forma (por exemplo, quando uma asserção falha). Se fornecido, *header* é impresso no console imediatamente antes do início da depuração. O argumento *commands*, se fornecido, é uma lista de comandos para executar quando o depurador é iniciado.

Alterado na versão 3.7: O argumento somente-nomeado *header*.

Alterado na versão 3.13: *set_trace()* entrará no depurador imediatamente, em vez da próxima linha de código a ser executada.

Adicionado na versão 3.14: O argumento *commands*.

awaitable pdb.**set_trace_async**(*, *header=None*, *commands=None*)

Versão async de *set_trace()*. Esta função deve ser usada dentro de uma função async com `await`.

```
async def f():
    await pdb.set_trace_async()
```

As instruções `await` são suportadas se o depurador for invocado por esta função.

Adicionado na versão 3.14.

pdb.**post_mortem**(*t=None*)

Entra em depuração post-mortem da exceção fornecida ou objeto traceback. Se nenhum valor for fornecido, usa a exceção que está sendo tratada no momento ou levanta `ValueError` se não houver nenhuma.

Alterado na versão 3.13: Adiciona suporte a objetos exceção.

pdb.**pm**()

Entra a depuração post-mortem da exceção encontrada em *sys.last_exc*.

pdb.**set_default_backend**(*backend*)

Há dois backends suportados para pdb: `'settrace'` e `'monitoring'`. Consulte *bdb.Bdb* para obter detalhes. O usuário pode definir o backend padrão a ser usado caso nenhum seja especificado ao instanciar *Pdb*. Se nenhum backend for especificado, o padrão será `'settrace'`.

> **Nota**
>
> *breakpoint()* e *set_trace()* não serão afetados por esta função. Eles sempre usam o backend `'monitoring'`.

Adicionado na versão 3.14.

pdb.**get_default_backend**()

Retorna o backend padrão para pdb.

Adicionado na versão 3.14.

As funções `run*` e a *set_trace()* são aliases, ou apelidos, para instanciar a classe *Pdb* e chamar o método com o mesmo nome. Se você deseja acessar outros recursos, faça você mesmo:

class pdb.**Pdb**(*completekey='tab'*, *stdin=None*, *stdout=None*, *skip=None*, *nosigint=False*, *readrc=True*, *mode=None*, *backend=None*, *colorize=False*)

Pdb é a classe do depurador.

Os argumentos *completekey*, *stdin* e *stdout* são passados para a classe subjacente *cmd.Cmd*; veja a descrição lá.

O argumento *skip*, se fornecido, deve ser um iterável de padrões de nome de módulo no estilo glob. O depurador não entrará nos quadros que se originam em um módulo que corresponde a um desses padrões.[1]

Por padrão, o Pdb define um manipulador para o sinal SIGINT (que é enviado quando o usuário pressiona `Ctrl-C` no console) quando você dá um comando *continue*. Isso permite que você entre no depurador novamente pressionando `Ctrl-C`. Se você deseja que o Pdb não toque no manipulador SIGINT, defina *nosigint* como verdadeiro.

O argumento *readrc* é padronizado como verdadeiro e controla se o Pdb carregará arquivos .pdbrc do sistema de arquivos.

O argumento *mode* especifica como o depurador foi invocado. Ele afeta o funcionamento de alguns comandos do depurador. Os valores válidos são `'inline'` (usado pelo comando embutido breakpoint()), `'cli'` (usado pela invocação da linha de comando) ou `None` (para comportamento retrocompatível, como antes da adição do argumento *mode*).

O argumento *backend* especifica o backend a ser usado pelo depurador. Se `None` for passado, o backend padrão será usado. Consulte *set_default_backend()*. Caso contrário, os backends implementados são `'settrace'` e `'monitoring'`.

O argumento *colorize*, se definido como `True`, habilitará a saída colorida no depurador, se houver suporte para cores. Isso destacará o código-fonte exibido no pdb.

Exemplo de chamada para habilitar rastreamento com *skip*:

```
import pdb; pdb.Pdb(skip=['django.*']).set_trace()
```

Levanta um *evento de auditoria* `pdb.Pdb` sem argumentos.

Alterado na versão 3.1: Adicionado o parâmetro *skip*.

Alterado na versão 3.2: Adicionado o parâmetro *nosigint*. Anteriormente, um manipulador de SIGINT nunca era definido por Pdb.

Alterado na versão 3.6: O argumento *readrc*.

Adicionado na versão 3.14: Adicionado o argumento *mode*.

Adicionado na versão 3.14: Adicionado o argumento *backend*.

Adicionado na versão 3.14: Adicionado o argumento *colorize*.

Alterado na versão 3.14: Pontos de interrupção em linha como *breakpoint()* ou *pdb.set_trace()* sempre interromperão o programa ao chamar o quadro, ignorando o padrão *skip* (se houver).

run(*statement*, *globals=None*, *locals=None*)

runeval(*expression*, *globals=None*, *locals=None*)

runcall(*function*, **args*, ***kwds*)

set_trace()

Consulte a documentação para as funções explicadas acima.

27.4.1 Comandos de depuração

Os comandos reconhecidos pelo depurador estão listados abaixo. A maioria dos comandos pode ser abreviada para uma ou duas letras, conforme indicado; por exemplo, `h(elp)` significa que `h` ou `help` podem ser usados para inserir o comando de ajuda (mas não `he` ou `hel`, nem `H` ou `Help` ou `HELP`). Os argumentos para os comandos devem ser separados por espaços em branco (espaços ou tabulações). Os argumentos opcionais estão entre colchetes (`[]`) na sintaxe do comando; os colchetes não devem ser digitados. As alternativas na sintaxe de comando são separadas por uma barra vertical (`|`).

Digitar uma linha em branco repete o último comando digitado. Exceção: se o último comando foi um comando *list*, as próximas 11 linhas serão listadas.

[1] Se um quadro é considerado originário de um determinado módulo é determinado pelo `__name__` nos globais do quadro.

Os comandos que o depurador não reconhece são presumidos como instruções Python e são executados no contexto do programa que está sendo depurado. As instruções Python também podem ser prefixadas com um ponto de exclamação (`!`). Essa é uma maneira poderosa de inspecionar o programa que está sendo depurado; é até possível alterar uma variável ou chamar uma função. Quando ocorre uma exceção em uma instrução, o nome da exceção é impresso, mas o estado do depurador não é alterado.

Alterado na versão 3.13: Expressões/instruções, cujo prefixo é um comando pdb, agora são identificadas e executadas corretamente.

O depurador possui suporte a *aliases*. Os aliases podem ter parâmetros que permitem um certo nível de adaptabilidade ao contexto em exame.

Vários comandos podem ser inseridos em uma única linha, separados por `;;`. (Um único `;` não é usado, pois é o separador para vários comandos em uma linha que é passada para o analisador Python.) Nenhuma inteligência é aplicada para separar os comandos; a entrada é dividida no primeiro par `;;`, mesmo que esteja no meio de uma string entre aspas. Uma solução alternativa para strings com caractere de ponto e vírgula duplo é usar a concatenação de string implícita `';'';'` ou `";"";"`.

Para definir uma variável global temporária, use uma *variável de conveniência*. Uma *variável de conveniência* é uma variável cujo nome começa com `$`. Por exemplo, `$foo = 1` define uma variável global `$foo` que você pode usar na sessão do depurador. As *variáveis de conveniência* são limpas quando o programa retoma a execução, portanto é menos provável que interfira em seu programa em comparação ao uso de variáveis normais como `foo = 1`.

Existem quatro *variáveis de conveniência* predefinidas:

- `$_frame`: o quadro atual que você está depurando
- `$_retval`: o valor de retorno se o quadro estiver retornando
- `$_exception`: a exceção se o quadro estiver levantando uma exceção
- `$_asynctask`: a tarefa asyncio se o pdb parar em uma função assíncrona

Adicionado na versão 3.12: Adicionado o recurso de *variável de conveniência*.

Adicionado na versão 3.14: Adicionada a variável de conveniência `$_asynctask`.

Se um arquivo `.pdbrc` existe no diretório inicial do usuário ou no diretório atual, ele é lido com a codificação `'utf-8'` e executado como se tivesse sido digitado no prompt do depurador, com a exceção de que linhas vazias e linhas iniciando com `#` são ignoradas. Isso é particularmente útil para aliases. Se ambos os arquivos existirem, aquele no diretório inicial será lido primeiro e os aliases definidos poderão ser substituídos pelo arquivo local.

Alterado na versão 3.2: `.pdbrc` agora pode conter comandos que continuam a depuração, como *`continue`* ou *`next`*. Anteriormente, esses comandos não tinham efeito.

Alterado na versão 3.11: `.pdbrc` agora é lido com a codificação `'utf-8'`. Anteriormente, ele era lido com a codificação da localidade do sistema.

h(elp) `[command]`

Sem argumento, imprime a lista de comandos disponíveis. Com um *command* como argumento, imprime ajuda sobre esse comando. `help pdb` exibe a documentação completa (a docstring do módulo *`pdb`*). Como o argumento *command* deve ser um identificador, `help exec` deve ser inserido para obter ajuda sobre o comando `!`.

w(here) `[count]`

Exibe um stack trace (situação da pilha de execução), com o quadro mais recente na parte inferior. Se *count* for 0, exibe a entrada do quadro atual. Se *count* for negativo, exibe os quadros menos recentes - *count*. Se *count* for positivo, exibe os quadros *count* mais recentes. Uma seta (>) indica o quadro atual, que determina o contexto da maioria dos comandos.

Alterado na versão 3.14: o argumento *count* foi adicionado

d(own) `[count]`

Move os níveis do quadro atual *count* (padrão 1) para baixo no stack trace (para um quadro mais recente).

u(p) [count]

Move os níveis do quadro atual na *count* (padrão 1) para cima no stack trace (para um quadro mais antigo).

b(reak) [([filename:]lineno | function) [, condition]]

Com um argumento *lineno*, define uma quebra na linha *lineno* no arquivo atual. O número da linha pode ser prefixado com *filename* e dois pontos, para especificar um ponto de interrupção em outro arquivo (possivelmente um que ainda não tenha sido carregado). O arquivo é pesquisado em *sys.path*. Formas aceitáveis de *filename* são `/caminhoabsoluto/para/arquivo.py`, `caminhorelativo/do/arquivo.py`, `módulo` e `pacote.módulo`.

Com um argumento *function*, define uma interrupção na primeira instrução executável dentro dessa função. *function* pode ser qualquer expressão avaliada como uma função no espaço de nomes atual.

Se um segundo argumento estiver presente, é uma expressão que deve ser avaliada como verdadeira antes que o ponto de interrupção seja respeitado.

Sem argumento, lista todas as quebras, inclusive para cada ponto de interrupção, o número de vezes que o ponto de interrupção foi atingido, a contagem atual de ignorados e a condição associada, se houver.

Cada ponto de interrupção recebe um número ao qual todos os outros comandos de ponto de interrupção se referem.

tbreak [([filename:]lineno | function) [, condition]]

Ponto de interrupção temporário, que é removido automaticamente quando é atingido pela primeira vez. Os argumentos são os mesmos que para *break*.

cl(ear) [filename:lineno | bpnumber ...]

Com um argumento *filename:lineno*, limpa todos os pontos de interrupção nessa linha. Com uma lista separada por espaços de números de ponto de interrupção, limpa esses pontos de interrupção. Sem argumento, limpa todas as quebras (mas primeiro pede a confirmação).

disable bpnumber [bpnumber ...]

Desativa os pontos de interrupção fornecidos como uma lista separada por espaços de números de pontos de interrupção. Desabilitar um ponto de interrupção significa que ele não pode interromper a execução do programa, mas, ao contrário de limpar um ponto de interrupção, ele permanece na lista de pontos de interrupção e pode ser (re)ativado.

enable bpnumber [bpnumber ...]

Ativa o ponto de interrupção especificado.

ignore bpnumber [count]

Define a contagem de ignorados para o número do ponto de interrupção especificado. Se *count* for omitida, a contagem de ignorados será definida como 0. Um ponto de interrupção se torna ativo quando a contagem de ignorados é zero. Quando diferente de zero, a contagem é decrementada cada vez que o ponto de interrupção é atingido e o ponto de interrupção não é desativado e qualquer condição associada é avaliada como verdadeira.

condition bpnumber [condition]

Define uma nova *condition* para o ponto de interrupção, uma expressão que deve ser avaliada como verdadeira antes que o ponto de interrupção seja respeitado. Se *condition* for omitida, qualquer condição existente será removida; isto é, o ponto de interrupção é tornado incondicional.

commands [bpnumber]

Especifica uma lista de comandos para o número do ponto de interrupção *bpnumber*. Os próprios comandos aparecem nas linhas seguintes. Digite em uma linha contendo apenas `end` para finalizar os comandos. Um exemplo:

```
(Pdb) commands 1
(com) p alguma_variável
(com) end
(Pdb)
```

Para remover todos os comandos de um ponto de interrupção, digite `commands` e siga-o imediatamente com `end`; isto é, não dê comandos.

Sem argumento *bpnumber*, `commands` refere-se ao último conjunto de pontos de interrupção.

Você pode usar comandos de ponto de interrupção para iniciar seu programa novamente. Simplesmente use o comando *`continue`*, ou *`step`*, ou qualquer outro comando que reinicie a execução.

Especificar qualquer comando que retome a execução (atualmente *`continue`*, *`step`*, *`next`*, *`return`*, *`until`*, *`jump`*, *`quit`* e suas abreviações) finaliza a lista de comandos (como se esse comando fosse imediatamente seguido pelo end). Isso ocorre porque sempre que você retoma a execução (mesmo com um simples next ou step), você pode encontrar outro ponto de interrupção — que pode ter sua própria lista de comandos, levando a ambiguidades sobre qual lista executar.

Se a lista de comandos contiver o comando `silent` ou um comando que retome a execução, a mensagem de ponto de interrupção contendo informações sobre o quadro não será exibida.

Alterado na versão 3.14: As informações do quadro não serão exibidas se um comando que retome a execução estiver presente na lista de comandos.

s(tep)

Executa a linha atual, interrompe na primeira ocasião possível (em uma função chamada ou na próxima linha na função atual).

n(ext)

Continua a execução até que a próxima linha na função atual seja atingida ou ela retorne. (A diferença entre *`next`* e *`step`* é que *`step`* para dentro de uma função chamada, enquanto *`next`* executa funções chamadas em (quase) velocidade máxima, parando apenas na próxima linha da função atual.)

unt(il) `[lineno]`

Sem argumento, continua a execução até que a linha com um número maior que o atual seja atingida.

Com *lineno*, continua a execução até que uma linha com um número maior ou igual a *lineno* ser alcançada. Nos dois casos, também interrompe quando o quadro atual retornar.

Alterado na versão 3.2: Permite fornecer um número de linha explícito.

r(eturn)

Continua a execução até que a função atual retorne.

c(ont(inue))

Continua a execução, interrompe apenas quando um ponto de interrupção for encontrado.

j(ump) `lineno`

Define a próxima linha que será executada. Disponível apenas no quadro mais inferior. Isso permite voltar e executar o código novamente ou avançar para pular o código que você não deseja executar.

Deve-se notar que nem todos os saltos são permitidos – por exemplo, não é possível pular para o meio de um loop de `for` ou sair de uma cláusula `finally`.

l(ist) `[first[, last]]`

Lista o código-fonte do arquivo atual. Sem argumentos, lista 11 linhas ao redor da linha atual ou continue a listagem anterior. Com `.` como argumento, lista 11 linhas ao redor da linha atual. Com um argumento, lista 11 linhas nessa linha. Com dois argumentos, lista o intervalo especificado; se o segundo argumento for menor que o primeiro, ele será interpretado como uma contagem.

A linha atual no quadro atual é indicada por `->`. Se uma exceção estiver sendo depurada, a linha em que a exceção foi originalmente gerada ou propagada é indicada por `>>`, se for diferente da linha atual.

Alterado na versão 3.2: Adicionado o marcador `>>`.

ll | `longlist`

Lista todo o código-fonte da função ou quadro atual. As linhas interessantes estão marcadas como para *`list`*.

Adicionado na versão 3.2.

a(rgs)

Imprime os argumentos da função atual e seus valores atuais.

p `expression`

Avalia *expression* no contexto atual e imprima seu valor.

> **Nota**
>
> `print()` também pode ser usado, mas não é um comando de depuração — isso executa a função Python *print()*.

pp `expression`

Como o comando *p*, exceto que o valor de *expression* é bastante impresso usando o módulo *pprint*.

whatis `expression`

Exibe o tipo de *expression*.

source `expression`

Tenta obter o código-fonte de *expression* e exibe-o.

Adicionado na versão 3.2.

display `[expression]`

Exibe o valor de *expression* caso ela tenha sido alterada, sempre que a execução for interrompida no quadro atual.

Sem *expression*, lista todas as expressões de exibição para o quadro atual.

> **Nota**
>
> Display avalia *expression* e compara com o resultado da avaliação anterior de *expression*, portanto, quando o resultado é mutável, display pode não ser capaz de captar as alterações.

Exemplo:

```
lst = []
breakpoint()
pass
lst.append(1)
print(lst)
```

Display não perceberá que `lst` foi alterado porque o resultado da avaliação foi modificado internamente por `lst.append(1)` antes de ser comparado:

```
> example.py(3)<module>()
-> pass
(Pdb) display lst
display lst: []
(Pdb) n
> example.py(4)<module>()
-> lst.append(1)
(Pdb) n
> example.py(5)<module>()
-> print(lst)
(Pdb)
```

Você pode fazer alguns truques com o mecanismo de cópia para fazê-lo funcionar:

```
> example.py(3)<module>()
-> pass
(Pdb) display lst[:]
display lst[:]: []
(Pdb) n
> example.py(4)<module>()
-> lst.append(1)
(Pdb) n
> example.py(5)<module>()
-> print(lst)
display lst[:]: [1]  [old: []]
(Pdb)
```

Adicionado na versão 3.2.

undisplay [expression]

Não exibe mais *expression* no quadro atual. Sem expressão, limpa todas as expressões de exibição para o quadro atual.

Adicionado na versão 3.2.

interact

Inicia um interpretador interativo (usando o módulo *code*) em um novo espaço de nomes global inicializado a partir dos espaços de nomes locais e globais para o escopo atual. Use `exit()` ou `quit()` para sair do interpretador e retornar ao depurador.

> **Nota**
>
> Como `interact` cria um novo espaço de nomes dedicado para execução de código, atribuições a variáveis não afetarão os espaços de nomes originais. No entanto, as modificações em quaisquer objetos mutáveis referenciados serão refletidas nos espaços de nomes originais, como de costume.

Adicionado na versão 3.2.

Alterado na versão 3.13: `exit()` e `quit()` podem ser usados para sair do comando *interact*.

Alterado na versão 3.13: *interact* direciona sua saída para o canal de saída do depurador em vez de *sys.stderr*.

alias [name [command]]

Cria um apelido chamado *name* que executa *command*. O comando *command não* deve ser colocado entre aspas. Parâmetros substituíveis podem ser indicados por `%1`, `%2`, … e `%9`, enquanto `%*` é substituído por todos os parâmetros. Se *command* for omitido, o apelido atual para *name* será mostrado. Se nenhum argumento for fornecido, todos os apelidos serão listados.

Os aliases podem ser aninhados e podem conter qualquer coisa que possa ser digitada legalmente no prompt do pdb. Observe que os comandos internos do pdb *podem* ser substituídos por aliases. Esse comando é oculto até que o alias seja removido. O alias é aplicado recursivamente à primeira palavra da linha de comando; todas as outras palavras da linha são deixadas em paz.

Como exemplo, aqui estão dois aliases úteis (especialmente quando colocados no arquivo `.pdbrc`):

```
# Exibe variáveis de instância (use em "pi classInst")
alias pi for k in %1.__dict__.keys(): print(f"%1.{k} = {%1.__dict__[k]}")
# Exibe variáveis de instância em self
alias ps pi self
```

unalias name

Executa o alias *name* especificado.

! `statement`

Executa a instrução *statement* (de uma só linha) no contexto do quadro de pilha atual. O ponto de exclamação pode ser omitido, a menos que a primeira palavra da instrução seja semelhante a um comando de depuração, por exemplo:

```
(Pdb) ! n=42
(Pdb)
```

Para definir uma variável global, você pode prefixar o comando de atribuição com uma instrução `global` na mesma linha, por exemplo:

```
(Pdb) global list_options; list_options = ['-l']
(Pdb)
```

run `[args ...]`

restart `[args ...]`

Reinicia o programa Python depurado. Se *args* for fornecido, ele é dividido com *shlex* e o resultado é usado como o novo *sys.argv*. Histórico, pontos de interrupção, ações e opções do depurador são preservados. *restart* é um apelido para *run*.

Alterado na versão 3.14: Os comandos *run* e *restart* são desabilitados quando o depurador é invocado no modo `'inline'`.

q(uit)

Sai do depurador. O programa que está sendo executado é abortado. Uma entrada com fim de arquivo equivale a *quit*.

Um prompt de confirmação será exibido se o depurador for invocado no modo `'inline'`. `y`, `Y`, `<Enter>` ou `EOF` confirmarão o encerramento.

Alterado na versão 3.14: Um prompt de confirmação será exibido se o depurador for invocado no modo `'inline'`. Após a confirmação, o depurador chamará *sys.exit()* imediatamente, em vez de levantar *bdb.BdbQuit* no próximo evento de rastreamento.

debug `code`

Entra em um depurador recursivo que percorre *code* (que é uma expressão ou instrução arbitrária a ser executada no ambiente atual).

retval

Exibe o valor de retorno para o último retorno de a função atual.

exceptions `[excnumber]`

Lista ou salta entre exceções encadeadas.

Ao usar `pdb.pm()` ou `Pdb.post_mortem(...)` com uma exceção encadeada em vez de um traceback, permite ao usuário mover entre as exceções encadeadas usando o comando `exceptions` para listar exceções e `exceptions <número>` para mudar para essa exceção.

Exemplo:

```
def out():
    try:
        middle()
    except Exception as e:
        raise ValueError("reraise middle() error") from e

def middle():
    try:
        return inner(0)
    except Exception as e:
        raise ValueError("Middle fail")
```

(continua na próxima página)

(continuação da página anterior)

```
def inner(x):
    1 / x

 out()
```

chamar `pdb.pm()` permitirá mover entre exceções:

```
> example.py(5)out()
-> raise ValueError("reraise middle() error") from e

(Pdb) exceptions
  0 ZeroDivisionError('division by zero')
  1 ValueError('Middle fail')
> 2 ValueError('reraise middle() error')

(Pdb) exceptions 0
> example.py(16)inner()
-> 1 / x

(Pdb) up
> example.py(10)middle()
-> return inner(0)
```

Adicionado na versão 3.13.

27.5 Os Profilers do Python

Código-fonte: Lib/profile.py and Lib/pstats.py

27.5.1 Introdução aos profilers

cProfile and *profile* provide *deterministic profiling* of Python programs. A *profile* is a set of statistics that describes how often and for how long various parts of the program executed. These statistics can be formatted into reports via the *pstats* module.

A biblioteca padrão do Python tem duas implementações diferentes para a mesma interface de profiling:

1. *cProfile* is recommended for most users; it's a C extension with reasonable overhead that makes it suitable for profiling long-running programs. Based on `lsprof`, contributed by Brett Rosen and Ted Czotter.
2. *profile*, a pure Python module whose interface is imitated by *cProfile*, but which adds significant overhead to profiled programs. If you're trying to extend the profiler in some way, the task might be easier with this module. Originally designed and written by Jim Roskind.

Nota

The profiler modules are designed to provide an execution profile for a given program, not for benchmarking purposes (for that, there is *timeit* for reasonably accurate results). This particularly applies to benchmarking Python code against C code: the profilers introduce overhead for Python code, but not for C-level functions, and so the C code would seem faster than any Python one.

27.5.2 Instant User's Manual

This section is provided for users that "don't want to read the manual." It provides a very brief overview, and allows a user to rapidly perform profiling on an existing application.

Para realizar profile de uma função que recebe apenas um argumento, você pode fazer:

```
import cProfile
import re
cProfile.run('re.compile("foo|bar")')
```

(Use *profile* instead of *cProfile* if the latter is not available on your system.)

The above action would run *re.compile()* and print profile results like the following:

```
      214 function calls (207 primitive calls) in 0.002 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
     1    0.000    0.000    0.002    0.002 {built-in method builtins.exec}
     1    0.000    0.000    0.001    0.001 <string>:1(<module>)
     1    0.000    0.000    0.001    0.001 __init__.py:250(compile)
     1    0.000    0.000    0.001    0.001 __init__.py:289(_compile)
     1    0.000    0.000    0.000    0.000 _compiler.py:759(compile)
     1    0.000    0.000    0.000    0.000 _parser.py:937(parse)
     1    0.000    0.000    0.000    0.000 _compiler.py:598(_code)
     1    0.000    0.000    0.000    0.000 _parser.py:435(_parse_sub)
```

The first line indicates that 214 calls were monitored. Of those calls, 207 were *primitive*, meaning that the call was not induced via recursion. The next line: `Ordered by: cumulative time` indicates the output is sorted by the `cumtime` values. The column headings include:

ncalls
: for the number of calls.

tottime
: for the total time spent in the given function (and excluding time made in calls to sub-functions)

percall
: is the quotient of `tottime` divided by `ncalls`

cumtime
: is the cumulative time spent in this and all subfunctions (from invocation till exit). This figure is accurate *even* for recursive functions.

percall
: is the quotient of `cumtime` divided by primitive calls

filename:lineno(function)
: provides the respective data of each function

When there are two numbers in the first column (for example `3/1`), it means that the function recursed. The second value is the number of primitive calls and the former is the total number of calls. Note that when the function does not recurse, these two values are the same, and only the single figure is printed.

Instead of printing the output at the end of the profile run, you can save the results to a file by specifying a filename to the `run()` function:

```
import cProfile
import re
cProfile.run('re.compile("foo|bar")', 'restats')
```

The *pstats.Stats* class reads profile results from a file and formats them in various ways.

The files *cProfile* and *profile* can also be invoked as a script to profile another script. For example:

```
python -m cProfile [-o output_file] [-s sort_order] (-m module | myscript.py)
```

-o <output_file>

Writes the profile results to a file instead of to stdout.

-s <sort_order>

Specifies one of the *sort_stats()* sort values to sort the output by. This only applies when *-o* is not supplied.

-m <module>

Specifies that a module is being profiled instead of a script.

Adicionado na versão 3.7: Added the `-m` option to *cProfile*.

Adicionado na versão 3.8: Added the `-m` option to *profile*.

The *pstats* module's *Stats* class has a variety of methods for manipulating and printing the data saved into a profile results file:

```
import pstats
from pstats import SortKey
p = pstats.Stats('restats')
p.strip_dirs().sort_stats(-1).print_stats()
```

The *strip_dirs()* method removed the extraneous path from all the module names. The *sort_stats()* method sorted all the entries according to the standard module/line/name string that is printed. The *print_stats()* method printed out all the statistics. You might try the following sort calls:

```
p.sort_stats(SortKey.NAME)
p.print_stats()
```

The first call will actually sort the list by function name, and the second call will print out the statistics. The following are some interesting calls to experiment with:

```
p.sort_stats(SortKey.CUMULATIVE).print_stats(10)
```

This sorts the profile by cumulative time in a function, and then only prints the ten most significant lines. If you want to understand what algorithms are taking time, the above line is what you would use.

If you were looking to see what functions were looping a lot, and taking a lot of time, you would do:

```
p.sort_stats(SortKey.TIME).print_stats(10)
```

to sort according to time spent within each function, and then print the statistics for the top ten functions.

Você pode tentar também:

```
p.sort_stats(SortKey.FILENAME).print_stats('__init__')
```

This will sort all the statistics by file name, and then print out statistics for only the class init methods (since they are spelled with `__init__` in them). As one final example, you could try:

```
p.sort_stats(SortKey.TIME, SortKey.CUMULATIVE).print_stats(.5, 'init')
```

This line sorts statistics with a primary key of time, and a secondary key of cumulative time, and then prints out some of the statistics. To be specific, the list is first culled down to 50% (re: `.5`) of its original size, then only lines containing `init` are maintained, and that sub-sub-list is printed.

If you wondered what functions called the above functions, you could now (`p` is still sorted according to the last criteria) do:

```
p.print_callers(.5, 'init')
```

and you would get a list of callers for each of the listed functions.

If you want more functionality, you're going to have to read the manual, or guess what the following functions do:

```
p.print_callees()
p.add('restats')
```

Invoked as a script, the *pstats* module is a statistics browser for reading and examining profile dumps. It has a simple line-oriented interface (implemented using *cmd*) and interactive help.

27.5.3 `profile` and `cProfile` Module Reference

Both the *profile* and *cProfile* modules provide the following functions:

`profile.`**`run`**(*command*, *filename=None*, *sort=-1*)

This function takes a single argument that can be passed to the *exec()* function, and an optional file name. In all cases this routine executes:

```
exec(command, __main__.__dict__, __main__.__dict__)
```

and gathers profiling statistics from the execution. If no file name is present, then this function automatically creates a *Stats* instance and prints a simple profiling report. If the sort value is specified, it is passed to this *Stats* instance to control how the results are sorted.

`profile.`**`runctx`**(*command*, *globals*, *locals*, *filename=None*, *sort=-1*)

This function is similar to *run()*, with added arguments to supply the globals and locals mappings for the *command* string. This routine executes:

```
exec(command, globals, locals)
```

and gathers profiling statistics as in the *run()* function above.

`class` `profile.`**`Profile`**(*timer=None*, *timeunit=0.0*, *subcalls=True*, *builtins=True*)

This class is normally only used if more precise control over profiling is needed than what the `cProfile.run()` function provides.

A custom timer can be supplied for measuring how long code takes to run via the *timer* argument. This must be a function that returns a single number representing the current time. If the number is an integer, the *timeunit* specifies a multiplier that specifies the duration of each unit of time. For example, if the timer returns times measured in thousands of seconds, the time unit would be `.001`.

Directly using the *Profile* class allows formatting profile results without writing the profile data to a file:

```
import cProfile, pstats, io
from pstats import SortKey
pr = cProfile.Profile()
pr.enable()
# ... do something ...
pr.disable()
s = io.StringIO()
sortby = SortKey.CUMULATIVE
ps = pstats.Stats(pr, stream=s).sort_stats(sortby)
ps.print_stats()
print(s.getvalue())
```

The *Profile* class can also be used as a context manager (supported only in *cProfile* module. see *Tipos de Gerenciador de Contexto*):

```
import cProfile

with cProfile.Profile() as pr:
    # ... do something ...

    pr.print_stats()
```

Alterado na versão 3.8: Adicionado suporte a gerenciador de contexto.

enable()

Inicia a coleta de dados de profilling. Apenas em *cProfile*.

disable()

Para de coletar dados de profilling. Apenas em *cProfile*.

create_stats()

Stop collecting profiling data and record the results internally as the current profile.

print_stats(*sort=-1*)

Cria um objeto *Stats* com base no perfil atual e imprime os resultados para stdout.

The *sort* parameter specifies the sorting order of the displayed statistics. It accepts a single key or a tuple of keys to enable multi-level sorting, as in *Stats.sort_stats*.

Adicionado na versão 3.13: *print_stats()* now accepts a tuple of keys.

dump_stats(*filename*)

Escreve os resultados do profile atual para *filename*.

run(*cmd*)

Profile the cmd via *exec()*.

runctx(*cmd*, *globals*, *locals*)

Profile the cmd via *exec()* with the specified global and local environment.

runcall(*func*, */*, **args*, ***kwargs*)

Profile `func(*args, **kwargs)`

Note that profiling will only work if the called command/function actually returns. If the interpreter is terminated (e.g. via a *sys.exit()* call during the called command/function execution) no profiling results will be printed.

27.5.4 A classe `Stats`

Analysis of the profiler data is done using the *Stats* class.

class pstats.**Stats**(**filenames or profile*, *stream=sys.stdout*)

This class constructor creates an instance of a "statistics object" from a *filename* (or list of filenames) or from a `Profile` instance. Output will be printed to the stream specified by *stream*.

The file selected by the above constructor must have been created by the corresponding version of *profile* or *cProfile*. To be specific, there is *no* file compatibility guaranteed with future versions of this profiler, and there is no compatibility with files produced by other profilers, or the same profiler run on a different operating system. If several files are provided, all the statistics for identical functions will be coalesced, so that an overall view of several processes can be considered in a single report. If additional files need to be combined with data in an existing *Stats* object, the *add()* method can be used.

Instead of reading the profile data from a file, a `cProfile.Profile` or *profile.Profile* object can be used as the profile data source.

Os objetos de *Stats* tem os seguintes métodos:

strip_dirs()

This method for the *Stats* class removes all leading path information from file names. It is very useful in reducing the size of the printout to fit within (close to) 80 columns. This method modifies the object, and the stripped information is lost. After performing a strip operation, the object is considered to have its entries in a "random" order, as it was just after object initialization and loading. If *strip_dirs()* causes two function names to be indistinguishable (they are on the same line of the same filename, and have the same function name), then the statistics for these two entries are accumulated into a single entry.

add(**filenames*)

This method of the *Stats* class accumulates additional profiling information into the current profiling object. Its arguments should refer to filenames created by the corresponding version of *profile.run()* or `cProfile.run()`. Statistics for identically named (re: file, line, name) functions are automatically accumulated into single function statistics.

dump_stats(*filename*)

Save the data loaded into the *Stats* object to a file named *filename*. The file is created if it does not exist, and is overwritten if it already exists. This is equivalent to the method of the same name on the *profile.Profile* and `cProfile.Profile` classes.

sort_stats(**keys*)

This method modifies the *Stats* object by sorting it according to the supplied criteria. The argument can be either a string or a SortKey enum identifying the basis of a sort (example: `'time'`, `'name'`, `SortKey.TIME` or `SortKey.NAME`). The SortKey enums argument have advantage over the string argument in that it is more robust and less error prone.

When more than one key is provided, then additional keys are used as secondary criteria when there is equality in all keys selected before them. For example, `sort_stats(SortKey.NAME, SortKey.FILE)` will sort all the entries according to their function name, and resolve all ties (identical function names) by sorting by file name.

For the string argument, abbreviations can be used for any key names, as long as the abbreviation is unambiguous.

The following are the valid string and SortKey:

Valid String Arg	Valid enum Arg	Significado
`'calls'`	SortKey.CALLS	call count
`'cumulative'`	SortKey.CUMULATIVE	cumulative time
`'cumtime'`	N/D	cumulative time
`'file'`	N/D	file name
`'filename'`	SortKey.FILENAME	file name
`'module'`	N/D	file name
`'ncalls'`	N/D	call count
`'pcalls'`	SortKey.PCALLS	primitive call count
`'line'`	SortKey.LINE	line number
`'name'`	SortKey.NAME	function name
`'nfl'`	SortKey.NFL	name/file/line
`'stdname'`	SortKey.STDNAME	standard name
`'time'`	SortKey.TIME	internal time
`'tottime'`	N/D	internal time

Note that all sorts on statistics are in descending order (placing most time consuming items first), where as name, file, and line number searches are in ascending order (alphabetical). The subtle distinction between `SortKey.NFL` and `SortKey.STDNAME` is that the standard name is a sort of the name as printed, which means that the embedded line numbers get compared in an odd way. For example, lines 3, 20, and 40 would (if the file names were the same) appear in the string order 20, 3 and 40. In contrast, `SortKey.NFL` does a numeric compare of the line numbers. In fact, `sort_stats(SortKey.NFL)` is the same as `sort_stats(SortKey.NAME, SortKey.FILENAME, SortKey.LINE)`.

For backward-compatibility reasons, the numeric arguments `-1`, `0`, `1`, and `2` are permitted. They are interpreted as `'stdname'`, `'calls'`, `'time'`, and `'cumulative'` respectively. If this old style format (numeric) is used, only one sort key (the numeric key) will be used, and additional arguments will be silently ignored.

Adicionado na versão 3.7: Added the SortKey enum.

reverse_order()

This method for the *Stats* class reverses the ordering of the basic list within the object. Note that by default ascending vs descending order is properly selected based on the sort key of choice.

print_stats(**restrictions*)

This method for the *Stats* class prints out a report as described in the *profile.run()* definition.

The order of the printing is based on the last *sort_stats()* operation done on the object (subject to caveats in *add()* and *strip_dirs()*).

The arguments provided (if any) can be used to limit the list down to the significant entries. Initially, the list is taken to be the complete set of profiled functions. Each restriction is either an integer (to select a count of lines), or a decimal fraction between 0.0 and 1.0 inclusive (to select a percentage of lines), or a string that will be interpreted as a regular expression (to pattern match the standard name that is printed). If several restrictions are provided, then they are applied sequentially. For example:

```
print_stats(.1, 'foo:')
```

would first limit the printing to first 10% of list, and then only print functions that were part of filename `.*foo:`. In contrast, the command:

```
print_stats('foo:', .1)
```

would limit the list to all functions having file names `.*foo:`, and then proceed to only print the first 10% of them.

print_callers(**restrictions*)

This method for the *Stats* class prints a list of all functions that called each function in the profiled database. The ordering is identical to that provided by *print_stats()*, and the definition of the restricting argument is also identical. Each caller is reported on its own line. The format differs slightly depending on the profiler that produced the stats:

- With *profile*, a number is shown in parentheses after each caller to show how many times this specific call was made. For convenience, a second non-parenthesized number repeats the cumulative time spent in the function at the right.
- With *cProfile*, each caller is preceded by three numbers: the number of times this specific call was made, and the total and cumulative times spent in the current function while it was invoked by this specific caller.

print_callees(**restrictions*)

This method for the *Stats* class prints a list of all function that were called by the indicated function. Aside from this reversal of direction of calls (re: called vs was called by), the arguments and ordering are identical to the *print_callers()* method.

get_stats_profile()

This method returns an instance of StatsProfile, which contains a mapping of function names to instances of FunctionProfile. Each FunctionProfile instance holds information related to the function's profile such as how long the function took to run, how many times it was called, etc...

Adicionado na versão 3.9: Added the following dataclasses: StatsProfile, FunctionProfile. Added the following function: get_stats_profile.

27.5.5 What Is Deterministic Profiling?

Deterministic profiling is meant to reflect the fact that all *function call*, *function return*, and *exception* events are monitored, and precise timings are made for the intervals between these events (during which time the user's code is executing). In contrast, *statistical profiling* (which is not done by this module) randomly samples the effective instruction pointer, and deduces where time is being spent. The latter technique traditionally involves less overhead (as the code does not need to be instrumented), but provides only relative indications of where time is being spent.

In Python, since there is an interpreter active during execution, the presence of instrumented code is not required in order to do deterministic profiling. Python automatically provides a *hook* (optional callback) for each event. In addition, the interpreted nature of Python tends to add so much overhead to execution, that deterministic profiling tends to only add small processing overhead in typical applications. The result is that deterministic profiling is not that expensive, yet provides extensive run time statistics about the execution of a Python program.

Call count statistics can be used to identify bugs in code (surprising counts), and to identify possible inline-expansion points (high call counts). Internal time statistics can be used to identify "hot loops" that should be carefully optimized. Cumulative time statistics should be used to identify high level errors in the selection of algorithms. Note that the unusual handling of cumulative times in this profiler allows statistics for recursive implementations of algorithms to be directly compared to iterative implementations.

27.5.6 Limitações

One limitation has to do with accuracy of timing information. There is a fundamental problem with deterministic profilers involving accuracy. The most obvious restriction is that the underlying "clock" is only ticking at a rate (typically) of about .001 seconds. Hence no measurements will be more accurate than the underlying clock. If enough measurements are taken, then the "error" will tend to average out. Unfortunately, removing this first error induces a second source of error.

The second problem is that it "takes a while" from when an event is dispatched until the profiler's call to get the time actually *gets* the state of the clock. Similarly, there is a certain lag when exiting the profiler event handler from the time that the clock's value was obtained (and then squirreled away), until the user's code is once again executing. As a result, functions that are called many times, or call many functions, will typically accumulate this error. The error that accumulates in this fashion is typically less than the accuracy of the clock (less than one clock tick), but it *can* accumulate and become very significant.

The problem is more important with `profile` than with the lower-overhead `cProfile`. For this reason, `profile` provides a means of calibrating itself for a given platform so that this error can be probabilistically (on the average) removed. After the profiler is calibrated, it will be more accurate (in a least square sense), but it will sometimes produce negative numbers (when call counts are exceptionally low, and the gods of probability work against you :-).) Do *not* be alarmed by negative numbers in the profile. They should *only* appear if you have calibrated your profiler, and the results are actually better than without calibration.

27.5.7 Calibration

The profiler of the `profile` module subtracts a constant from each event handling time to compensate for the overhead of calling the time function, and socking away the results. By default, the constant is 0. The following procedure can be used to obtain a better constant for a given platform (see *Limitações*).

```
import profile
pr = profile.Profile()
for i in range(5):
    print(pr.calibrate(10000))
```

The method executes the number of Python calls given by the argument, directly and again under the profiler, measuring the time for both. It then computes the hidden overhead per profiler event, and returns that as a float. For example, on a 1.8Ghz Intel Core i5 running macOS, and using Python's time.process_time() as the timer, the magical number is about 4.04e-6.

The object of this exercise is to get a fairly consistent result. If your computer is *very* fast, or your timer function has poor resolution, you might have to pass 100000, or even 1000000, to get consistent results.

When you have a consistent answer, there are three ways you can use it:

```
import profile

# 1. Apply computed bias to all Profile instances created hereafter.
profile.Profile.bias = your_computed_bias

# 2. Apply computed bias to a specific Profile instance.
pr = profile.Profile()
pr.bias = your_computed_bias

# 3. Specify computed bias in instance constructor.
pr = profile.Profile(bias=your_computed_bias)
```

If you have a choice, you are better off choosing a smaller constant, and then your results will "less often" show up as negative in profile statistics.

27.5.8 Using a custom timer

If you want to change how current time is determined (for example, to force use of wall-clock time or elapsed process time), pass the timing function you want to the `Profile` class constructor:

```
pr = profile.Profile(your_time_func)
```

The resulting profiler will then call `your_time_func`. Depending on whether you are using *profile.Profile* or `cProfile.Profile`, `your_time_func`'s return value will be interpreted differently:

profile.Profile

`your_time_func` should return a single number, or a list of numbers whose sum is the current time (like what *os.times()* returns). If the function returns a single time number, or the list of returned numbers has length 2, then you will get an especially fast version of the dispatch routine.

Be warned that you should calibrate the profiler class for the timer function that you choose (see *Calibration*). For most machines, a timer that returns a lone integer value will provide the best results in terms of low overhead during profiling. (*os.times()* is *pretty* bad, as it returns a tuple of floating-point values). If you want to substitute a better timer in the cleanest fashion, derive a class and hardwire a replacement dispatch method that best handles your timer call, along with the appropriate calibration constant.

cProfile.Profile

`your_time_func` should return a single number. If it returns integers, you can also invoke the class constructor with a second argument specifying the real duration of one unit of time. For example, if `your_integer_time_func` returns times measured in thousands of seconds, you would construct the `Profile` instance as follows:

```
pr = cProfile.Profile(your_integer_time_func, 0.001)
```

As the `cProfile.Profile` class cannot be calibrated, custom timer functions should be used with care and should be as fast as possible. For the best results with a custom timer, it might be necessary to hard-code it in the C source of the internal `_lsprof` module.

Python 3.3 adds several new functions in *time* that can be used to make precise measurements of process or wall-clock time. For example, see *time.perf_counter()*.

27.6 `timeit` — Mede o tempo de execução de pequenos trechos de código

Código-fonte: Lib/timeit.py

Este módulo fornece uma maneira simples de cronometrar pequenos trechos do código Python. Ele tem uma *Interface de Linha de Comando* e um *chamável*. Ele evita uma série de armadilhas comuns para medir os tempos de execução

de um script. Veja também o capítulo "Algorithms" de Tim Peters, na segunda edição do *Python Cookbook*, publicado pela O'Reilly.

27.6.1 Exemplos básicos

O exemplo a seguir mostra como a *Interface de Linha de Comando* pode ser usado para comparar três expressões diferentes:

```
$ python -m timeit "'-'.join(str(n) for n in range(100))"
10000 loops, best of 5: 30.2 usec per loop
$ python -m timeit "'-'.join([str(n) for n in range(100)])"
10000 loops, best of 5: 27.5 usec per loop
$ python -m timeit "'-'.join(map(str, range(100)))"
10000 loops, best of 5: 23.2 usec per loop
```

Isso pode ser obtido da interface *Interface em Python*

```
>>> import timeit
>>> timeit.timeit('"-".join(str(n) for n in range(100))', number=10000)
0.3018611848820001
>>> timeit.timeit('"-".join([str(n) for n in range(100)])', number=10000)
0.2727368790656328
>>> timeit.timeit('"-".join(map(str, range(100)))', number=10000)
0.23702679807320237
```

Um chamável também pode ser passado para a *Interface em Python*:

```
>>> timeit.timeit(lambda: "-".join(map(str, range(100))), number=10000)
0.19665591977536678
```

Observe, entretanto, que `timeit()` determinará automaticamente o número de repetições somente quando a interface de linha de comando for usada. Na seção *Exemplos* você encontrará exemplos mais avançados.

27.6.2 Interface em Python

Este módulo define três funções e uma classe pública:

`timeit.`**`timeit`**(*stmt='pass'*, *setup='pass'*, *timer=<default timer>*, *number=1000000*, *globals=None*)

Cria uma instância de `Timer` com o código de *setup* e a função *timer* função para executar o método `timeit()` com o total de execuções informado em *number*. O argumento opcional *globals* especifica um espaço de nomes no qual o código será executado.

Alterado na versão 3.5: O parâmetro opcional *globals* foi adicionado.

`timeit.`**`repeat`**(*stmt='pass'*, *setup='pass'*, *timer=<default timer>*, *repeat=5*, *number=1000000*, *globals=None*)

Cria uma instância `Timer` com a instrução fornecida, o código de *setup* e a função *timer* e para executar o método `repeat()` com o total de execuções informando em *repeat* e o *número* de execuções fornecidos. O argumento opcional *globals* especifica um espaço de nomes no qual executar o código.

Alterado na versão 3.5: O parâmetro opcional *globals* foi adicionado.

Alterado na versão 3.7: Valor padrão de *repetição* mudou de 3 para 5.

`timeit.`**`default_timer`**()

O cronômetro padrão, que é sempre time.perf_counter(), retorna segundos com ponto flutuante. Uma alternativa, time.perf_counter_ns, retorna um inteiro em nanossegundos.

Alterado na versão 3.3: `time.perf_counter()` é o cronômetro padrão agora.

class timeit.**Timer**(*stmt='pass'*, *setup='pass'*, *timer=<timer function>*, *globals=None*)

Classe para cronometrar a velocidade de execução de pequenos trechos de código.

O construtor recebe uma instrução a ser cronometrada, uma instrução adicional usada para configuração e uma função de timer. Ambas as instruções têm como padrão `'pass'`; a função de timer é dependente da plataforma (veja a string do documento do módulo). *stmt* e *setup* também podem conter múltiplas instruções separadas por `;` ou novas linhas, desde que não contenham literais de string multilinhas. A instrução será, por padrão, executada dentro do espaço de nomes do *timeit*; esse comportamento pode ser controlado passando um espaço de nomes para *globals*.

Para medir o tempo de execução da primeira instrução use o método `timeit()`. Os métodos `repeat()` e `autorange()` são convenientes para chamar `timeit()` várias vezes.

O tempo de execução de *setup* é excluído do tempo total de execução cronometrado.

Os parâmetros *stmt* e *setup* também podem receber objetos que podem ser chamados sem argumentos. Isso incorporará chamadas a eles em uma função de timer que será executada por `timeit()`. Observe que a sobrecarga de temporização é um pouco maior neste caso por causa das chamadas de função extras.

Alterado na versão 3.5: O parâmetro opcional *globals* foi adicionado.

timeit(*number=1000000*)

Mede o tempo para a quantidade *number* de execuções da instrução principal. Isso executa a instrução setup uma vez e, em seguida, retorna o tempo necessário para executar a instrução principal várias vezes. O temporizador padrão retorna segundos como um float. O argumento é o número de vezes que o laço passa, com padrão de um milhão. A instrução principal, a instrução de configuração e a função de timer a ser usada são passadas para o construtor.

> **Nota**
>
> Por padrão, `timeit()` desativa temporariamente *coleta de lixo* durante a temporização. A vantagem dessa abordagem é que ela torna temporizações independentes mais comparáveis. A desvantagem é que o GC pode ser um componente importante do desempenho da função que está sendo medida. Se for assim, o GC pode ser reativado como a primeira instrução na string *setup*. Por exemplo:
>
> ```
> timeit.Timer('for i in range(10): oct(i)', 'gc.enable()').timeit()
> ```

autorange(*callback=None*)

Determina automaticamente quantas vezes chamar `timeit()`.

Esta é uma função de conveniência que chama `timeit()` repetidamente para que o tempo total seja >= 0,2 segundos, retornando o eventual (número de voltas, tempo gasto para esse número de voltas). Ela chama `timeit()` com números crescentes da sequência 1, 2, 5, 10, 20, 50, … até que o tempo gasto seja de pelo menos 0,2 segundos.

Se *callback* for fornecido e não for `None`, ele será chamado após cada tentativa e tem dois argumento: `callback(number, time_taken)`.

Adicionado na versão 3.6.

repeat(*repeat=5*, *number=1000000*)

Chama `timeit()` algumas vezes.

Esse é um função de conveniência que chama o `timeit()` repetidamente e retorna uma lista de resultados. O primeiro argumento especifica quantas vezes deve chamar o `timeit()`. O segundo argumento especifica o argumento *number* para `timeit()`.

> **Nota**
>
> É tentador calcular a média e o desvio padrão do vetor de resultados e relatá-los. No entanto, isso não é muito útil. Em um caso típico, o menor valor fornece um limite inferior para a velocidade com que

sua máquina pode executar o trecho de código fornecido; valores mais altos no vetor de resultados normalmente não são causados pela variabilidade na velocidade do Python, mas por outros processos que interferem na precisão do seu tempo. Portanto, o `min()` do resultado é provavelmente o único número no qual você deve se interessar. Depois disso, você deve analisar o vetor inteiro e aplicar o bom senso em vez de estatística.

Alterado na versão 3.7: Valor padrão de *repetição* mudou de 3 para 5.

print_exc(*file=None*)

Função auxiliar para imprimir um traceback do código cronometrado.

Uso típico:

```
t = Timer(...)       # fora do try/except
try:
    t.timeit(...)    # ou t.repeat(...)
except Exception:
    t.print_exc()
```

A vantagem em relação ao traceback padrão é que as linhas de origem no modelo compilado serão exibidas. O argumento opcional *file* direciona para onde o traceback é enviado; o padrão é `sys.stderr`.

27.6.3 Interface de Linha de Comando

Quando chamado como um programa a partir da linha de comando, as seguintes opções estão disponíveis:

```
python -m timeit [-n N] [-r N] [-u U] [-s S] [-p] [-v] [-h] [instrução ...]
```

As seguintes opções são permitidas

-n N, **--number**=N

Quantas vezes deve executar 'statement'

-r N, **--repeat**=N

Quantidade de vezes para repetir o cronômetro (o valor padrão é 5)

-s S, **--setup**=S

instrução a ser executada apenas uma vez e quando iniciada (padrão `pass`)

-p, **--process**

mede apenas o tempo de processamento, e não o tempo total de execução, usando `time.process_time()` em vez de `time.perf_counter()`, que é o padrão

Adicionado na versão 3.3.

-u, **--unit**=U

especifique uma unidade de tempo para a saída do cronômetro; pode selecionar `nsec`, `usec`, `msec`, ou `sec`

Adicionado na versão 3.5.

-v, **--verbose**

imprime resultados brutos de tempo; repetir para obter mais precisão de dígitos

-h, **--help**

imprime uma mensagem curta de uso e sai

Uma instrução multilinha pode ser fornecida especificando cada linha como um argumento de instrução separado; linhas indentadas são possíveis colocando um argumento entre aspas e usando espaços à esquerda. Múltiplas opções `-s` são tratadas de forma semelhante.

Se `-n` não for informada, um número adequado de loops será calculado tentando adicionar números numa sequência como 1, 2, 5, 10, 20, 50, … até que o tempo total seja de pelo menos 0,2 segundos.

As medições de *default_timer()* podem ser afetadas por outros programas em execução na mesma máquina, portanto, a melhor coisa a fazer quando uma cronometragem precisa é repeti-la algumas vezes e usar o melhor tempo. A opção *-r* é boa para isso; o padrão de 5 repetições provavelmente é suficiente na maioria dos casos. Você pode usar *time.process_time()* para medir o tempo de CPU.

Nota

Há uma certa sobrecarga padrão associada à execução de uma instrução *pass*. O código aqui não tenta ocultá-lo, mas você deve estar ciente disso. A sobrecarga padrão pode ser medida invocando pelo programa sem argumento, e pode ser diferente entre diferentes versões Python.

27.6.4 Exemplos

É possível fornecer uma instrução de configuração que é executada apenas uma vez no início:

```
$ python -m timeit -s "text = 'sample string'; char = 'g'" "char in text"
5000000 loops, best of 5: 0.0877 usec per loop
$ python -m timeit -s "text = 'sample string'; char = 'g'" "text.find(char)"
1000000 loops, best of 5: 0.342 usec per loop
```

Na saída, existem três campos. A contagem de laços, que informa quantas vezes o corpo da instrução foi executado por repetição do laço de temporização. A contagem de repetições ('melhor de 5') que informa quantas vezes o laço de temporização foi repetido e, finalmente, o tempo que o corpo da instrução levou, em média, na melhor repetição do laço de temporização. Ou seja, o tempo necessário para a repetição mais rápida dividido pela contagem de interações.

```
>>> import timeit
>>> timeit.timeit('char in text', setup='text = "sample string"; char = "g"')
0.41440500499993504
>>> timeit.timeit('text.find(char)', setup='text = "sample string"; char = "g"')
1.7246671520006203
```

O mesmo pode ser feito usando a classe *Timer* e seus métodos:

```
>>> import timeit
>>> t = timeit.Timer('char in text', setup='text = "sample string"; char = "g"')
>>> t.timeit()
0.3955516149999312
>>> t.repeat()
[0.40183617287970225, 0.37027556854118704, 0.38344867356679524, 0.3712595970846668,
↪ 0.37866875250654886]
```

Os exemplos a seguir mostram como cronometrar expressões que contêm várias linhas. Aqui comparamos o custo de usar *hasattr()* vs. `try`/`except` para testar atributos de objetos presentes e ausentes:

```
$ python -m timeit "try:" "  str.__bool__" "except AttributeError:" "  pass"
20000 loops, best of 5: 15.7 usec per loop
$ python -m timeit "if hasattr(str, '__bool__'): pass"
50000 loops, best of 5: 4.26 usec per loop

$ python -m timeit "try:" "  int.__bool__" "except AttributeError:" "  pass"
200000 loops, best of 5: 1.43 usec per loop
$ python -m timeit "if hasattr(int, '__bool__'): pass"
100000 loops, best of 5: 2.23 usec per loop
```

```
>>> import timeit
>>> # atributo está em falta
>>> s = """\
```

(continua na próxima página)

(continuação da página anterior)

```
... try:
...     str.__bool__
... except AttributeError:
...     pass
... """
>>> timeit.timeit(stmt=s, number=100000)
0.9138244460009446
>>> s = "if hasattr(str, '__bool__'): pass"
>>> timeit.timeit(stmt=s, number=100000)
0.5829014980008651
>>>
>>> # atributo está presente
>>> s = """\
... try:
...     int.__bool__
... except AttributeError:
...     pass
... """
>>> timeit.timeit(stmt=s, number=100000)
0.04215312199994514
>>> s = "if hasattr(int, '__bool__'): pass"
>>> timeit.timeit(stmt=s, number=100000)
0.08588060699912603
```

Para dar ao módulo `timeit` acesso as funções que você definiu, você pode passar o parâmetro *setup*, que contém um instrução de importar:

```
def test():
    """Stupid test function"""
    L = [i for i in range(100)]

if __name__ == '__main__':
    import timeit
    print(timeit.timeit("test()", setup="from __main__ import test"))
```

Outra opção é passar `globals()` para o parâmetro *globals*, o que fará com que o código seja executado em seu espaço de nomes global. Isso pode ser mais conveniente do que especificar individualmente imports:

```
def f(x):
    return x**2
def g(x):
    return x**4
def h(x):
    return x**8

import timeit
print(timeit.timeit('[func(42) for func in (f,g,h)]', globals=globals()))
```

27.7 `trace` — Rastreia ou acompanha a execução de instruções Python

Código-fonte: Lib/trace.py

O módulo `trace` permite que você rastreie a execução do programa, gere listagens de cobertura de instrução anotada,

imprima relações de chamador/receptor e funções de lista executadas durante a execução de um programa. Ele pode ser usado em outro programa ou na linha de comando.

Ver também

Coverage.py
Uma popular ferramenta de cobertura de terceiros que fornece saída HTML junto com recursos avançados, como cobertura de ramificações.

27.7.1 Uso na linha de comando

O módulo *trace* pode ser chamado a partir da linha de comando. Pode ser tão simples quanto:

```
python -m trace --count -C . algumarquivo.py ...
```

O comando acima irá executar `algumarquivo.py` e gerar listagens anotadas de todos os módulos Python importados durante a execução para o diretório atual.

--help
Exibe o modo de uso e sai.

--version
Exibe a versão do módulo e sai.

Adicionado na versão 3.8: Adicionada a opção `--module` que permite executar um módulo executável.

Opções principais

Pelo menos uma das seguintes opções deve ser especificada ao invocar *trace*. A opção *--listfuncs* é mutuamente exclusiva com as opções *--trace* e *--count*. Quando *--listfuncs* é fornecida, nem *--count* nem *--trace* são aceitas, e vice-versa.

-c, --count
Produz um conjunto de arquivos de listagem anotada após a conclusão do programa que mostra quantas vezes cada instrução foi executada. Veja também *--coverdir*, *--file* e *--no-report* abaixo.

-t, --trace
Exibe linhas como elas são executadas.

-l, --listfuncs
Exibe as funções executadas executando o programa.

-r, --report
Produz uma lista anotada de uma execução de programa anterior que usava a opção *--count* e *--file*. Isso não executa nenhum código.

-T, --trackcalls
Exibe os relacionamentos de chamada expostos ao executar o programa.

Modificadores

-f, --file=<file>
Nome de um arquivo para acumular contagens em várias execuções de rastreamento. Deve ser usado com a opção *--count*.

-C, --coverdir=<dir>
Diretório para onde vão os arquivos de relatório. O relatório de cobertura para `pacote.módulo` é escrito em arquivo *dir/pacote/módulo*`.cover`.

-m, --missing

Ao gerar listagens anotadas, marca as linhas que não foram executadas com `>>>>>>`.

-s, --summary

Ao usar `--count` ou `--report`, escreve um breve resumo no stdout para cada arquivo processado.

-R, --no-report

Não gera listagens anotadas. Isso é útil se você pretende fazer várias execuções com `--count`, e então produzir um único conjunto de listagens anotadas no final.

-g, --timing

Prefixa cada linha com o tempo desde o início do programa. Usado apenas durante o rastreamento.

Filtros

Essas opções podem ser repetidas várias vezes.

--ignore-module=<mod>

Ignora cada um dos nomes de módulo fornecidos e seus submódulos (se for um pacote). O argumento pode ser uma lista de nomes separados por uma vírgula.

--ignore-dir=<dir>

Ignora todos os módulos e pacotes no diretório e subdiretórios nomeados. O argumento pode ser uma lista de diretórios separados por `os.pathsep`.

27.7.2 Interface programática

class trace.**Trace**(*count=1*, *trace=1*, *countfuncs=0*, *countcallers=0*, *ignoremods=()*, *ignoredirs=()*, *infile=None*, *outfile=None*, *timing=False*)

Cria um objeto para rastrear a execução de uma única instrução ou expressão. Todos os parâmetros são opcionais. *count* ativa a contagem de números de linha. *trace* ativa o rastreamento de execução de linha. *countfuncs* ativa a listagem das funções chamadas durante a execução. *countcallers* ativa o rastreamento de relacionamento de chamada. *ignoremods* é uma lista de módulos ou pacotes a serem ignorados. *ignoreirs* é uma lista de diretórios cujos módulos ou pacotes devem ser ignorados. *infile* é o nome do arquivo do qual deve ler as informações de contagem armazenadas. *outfile* é o nome do arquivo no qual deve escrever as informações de contagem atualizadas. *timing* ativa a exibição de um carimbo de data/hora relativo ao momento em que o rastreamento foi iniciado.

run(*cmd*)

Executa o comando e reúne estatísticas da execução com os parâmetros de rastreamento atuais. *cmd* deve ser uma string ou objeto código, adequado para passar para `exec()`.

runctx(*cmd*, *globals=None*, *locals=None*)

Executa o comando e reúne estatísticas da execução com os parâmetros de rastreamento atuais, nos ambientes global e local definidos. Se não for definido, *globals* e *locals* usam como padrão dicionários vazios.

runfunc(*func*, */*, **args*, ***kwds*)

Chama *func* com os argumentos fornecidos sob controle do objeto `Trace` com os parâmetros de rastreamento atuais.

results()

Retorna um objeto `CoverageResults` que contém os resultados cumulativos de todas as chamadas anteriores para `run`, `runctx` e `runfunc` para a instância `Trace` fornecida. Não redefine os resultados de rastreamento acumulados.

class trace.**CoverageResults**

Um contêiner para resultados de cobertura, criado por `Trace.results()`. Não deve ser criado diretamente pelo usuário.

update(*other*)

Mescla dados de outro objeto *CoverageResults*.

write_results(*show_missing=True*, *summary=False*, *coverdir=None*, ***, *ignore_missing_files=False*)

Escreve os resultados da cobertura. Defina *show_missing* para mostrar as linhas que não tiveram ocorrências. Defina o *summary* para incluir na saída o resumo da cobertura por módulo. *coverdir* especifica o diretório no qual os arquivos de resultados de cobertura serão enviados. Se for `None`, os resultados de cada arquivo de origem são colocados em seu diretório.

Se *ignore_missing_files* for `True`, as contagens de cobertura para arquivos que não existem mais serão ignoradas silenciosamente. Caso contrário, um arquivo ausente vai levantar *FileNotFoundError*.

Alterado na versão 3.13: Adicionado o parâmetro *ignore_missing_files*.

Um exemplo simples que demonstra o uso da interface programática:

```
import sys
import trace

# cria um objeto Trace, dizendo a ele o que ignorar e se deve
# fazer rastreamento ou contagem de linhas ou ambos.
tracer = trace.Trace(
    ignoredirs=[sys.prefix, sys.exec_prefix],
    trace=0,
    count=1)

# executa o novo comando usando o rastreador fornecido
tracer.run('main()')

# faz um relatório, colocando a saída no diretório atual
r = tracer.results()
r.write_results(show_missing=True, coverdir=".")
```

27.8 `tracemalloc` — Trace memory allocations

Adicionado na versão 3.4.

Código-fonte: Lib/tracemalloc.py

The tracemalloc module is a debug tool to trace memory blocks allocated by Python. It provides the following information:

- Traceback where an object was allocated
- Estatísticas sobre blocos de memória alocados por nome de arquivo e por número de linha: tamanho total, número e tamanho médio dos blocos de memória alocados
- Calcula as diferenças entre dois instantâneos para detectar vazamentos de memória

To trace most memory blocks allocated by Python, the module should be started as early as possible by setting the `PYTHONTRACEMALLOC` environment variable to `1`, or by using `-X tracemalloc` command line option. The *tracemalloc.start()* function can be called at runtime to start tracing Python memory allocations.

By default, a trace of an allocated memory block only stores the most recent frame (1 frame). To store 25 frames at startup: set the `PYTHONTRACEMALLOC` environment variable to `25`, or use the `-X tracemalloc=25` command line option.

27.8.1 Exemplos

Exibe o top 10

Display the 10 files allocating the most memory:

```
import tracemalloc

tracemalloc.start()

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')

print("[ Top 10 ]")
for stat in top_stats[:10]:
    print(stat)
```

Example of output of the Python test suite:

```
[ Top 10 ]
<frozen importlib._bootstrap>:716: size=4855 KiB, count=39328, average=126 B
<frozen importlib._bootstrap>:284: size=521 KiB, count=3199, average=167 B
/usr/lib/python3.4/collections/__init__.py:368: size=244 KiB, count=2315,␣
↪average=108 B
/usr/lib/python3.4/unittest/case.py:381: size=185 KiB, count=779, average=243 B
/usr/lib/python3.4/unittest/case.py:402: size=154 KiB, count=378, average=416 B
/usr/lib/python3.4/abc.py:133: size=88.7 KiB, count=347, average=262 B
<frozen importlib._bootstrap>:1446: size=70.4 KiB, count=911, average=79 B
<frozen importlib._bootstrap>:1454: size=52.0 KiB, count=25, average=2131 B
<string>:5: size=49.7 KiB, count=148, average=344 B
/usr/lib/python3.4/sysconfig.py:411: size=48.0 KiB, count=1, average=48.0 KiB
```

We can see that Python loaded `4855 KiB` data (bytecode and constants) from modules and that the *collections* module allocated `244 KiB` to build *namedtuple* types.

See *Snapshot.statistics()* for more options.

Compute differences

Take two snapshots and display the differences:

```
import tracemalloc
tracemalloc.start()
# ... start your application ...

snapshot1 = tracemalloc.take_snapshot()
# ... call the function leaking memory ...
snapshot2 = tracemalloc.take_snapshot()

top_stats = snapshot2.compare_to(snapshot1, 'lineno')

print("[ Top 10 differences ]")
for stat in top_stats[:10]:
    print(stat)
```

Example of output before/after running some tests of the Python test suite:

```
[ Top 10 differences ]
<frozen importlib._bootstrap>:716: size=8173 KiB (+4428 KiB), count=71332 (+39369),
↪ average=117 B
/usr/lib/python3.4/linecache.py:127: size=940 KiB (+940 KiB), count=8106 (+8106),␣
↪average=119 B
/usr/lib/python3.4/unittest/case.py:571: size=298 KiB (+298 KiB), count=589 (+589),
↪ average=519 B
<frozen importlib._bootstrap>:284: size=1005 KiB (+166 KiB), count=7423 (+1526),␣
↪average=139 B
/usr/lib/python3.4/mimetypes.py:217: size=112 KiB (+112 KiB), count=1334 (+1334),␣
↪average=86 B
/usr/lib/python3.4/http/server.py:848: size=96.0 KiB (+96.0 KiB), count=1 (+1),␣
↪average=96.0 KiB
/usr/lib/python3.4/inspect.py:1465: size=83.5 KiB (+83.5 KiB), count=109 (+109),␣
↪average=784 B
/usr/lib/python3.4/unittest/mock.py:491: size=77.7 KiB (+77.7 KiB), count=143␣
↪(+143), average=557 B
/usr/lib/python3.4/urllib/parse.py:476: size=71.8 KiB (+71.8 KiB), count=969␣
↪(+969), average=76 B
/usr/lib/python3.4/contextlib.py:38: size=67.2 KiB (+67.2 KiB), count=126 (+126),␣
↪average=546 B
```

We can see that Python has loaded `8173 KiB` of module data (bytecode and constants), and that this is `4428 KiB` more than had been loaded before the tests, when the previous snapshot was taken. Similarly, the *linecache* module has cached `940 KiB` of Python source code to format tracebacks, all of it since the previous snapshot.

If the system has little free memory, snapshots can be written on disk using the *Snapshot.dump()* method to analyze the snapshot offline. Then use the *Snapshot.load()* method reload the snapshot.

Get the traceback of a memory block

Code to display the traceback of the biggest memory block:

```
import tracemalloc

# Store 25 frames
tracemalloc.start(25)

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('traceback')

# pick the biggest memory block
stat = top_stats[0]
print("%s memory blocks: %.1f KiB" % (stat.count, stat.size / 1024))
for line in stat.traceback.format():
    print(line)
```

Example of output of the Python test suite (traceback limited to 25 frames):

```
903 memory blocks: 870.1 KiB
  File "<frozen importlib._bootstrap>", line 716
  File "<frozen importlib._bootstrap>", line 1036
  File "<frozen importlib._bootstrap>", line 934
  File "<frozen importlib._bootstrap>", line 1068
  File "<frozen importlib._bootstrap>", line 619
  File "<frozen importlib._bootstrap>", line 1581
```

(continua na próxima página)

(continuação da página anterior)

```
  File "<frozen importlib._bootstrap>", line 1614
  File "/usr/lib/python3.4/doctest.py", line 101
    import pdb
  File "<frozen importlib._bootstrap>", line 284
  File "<frozen importlib._bootstrap>", line 938
  File "<frozen importlib._bootstrap>", line 1068
  File "<frozen importlib._bootstrap>", line 619
  File "<frozen importlib._bootstrap>", line 1581
  File "<frozen importlib._bootstrap>", line 1614
  File "/usr/lib/python3.4/test/support/__init__.py", line 1728
    import doctest
  File "/usr/lib/python3.4/test/test_pickletools.py", line 21
    support.run_doctest(pickletools)
  File "/usr/lib/python3.4/test/regrtest.py", line 1276
    test_runner()
  File "/usr/lib/python3.4/test/regrtest.py", line 976
    display_failure=not verbose)
  File "/usr/lib/python3.4/test/regrtest.py", line 761
    match_tests=ns.match_tests)
  File "/usr/lib/python3.4/test/regrtest.py", line 1563
    main()
  File "/usr/lib/python3.4/test/__main__.py", line 3
    regrtest.main_in_temp_cwd()
  File "/usr/lib/python3.4/runpy.py", line 73
    exec(code, run_globals)
  File "/usr/lib/python3.4/runpy.py", line 160
    "__main__", fname, loader, pkg_name)
```

We can see that the most memory was allocated in the *importlib* module to load data (bytecode and constants) from modules: `870.1 KiB`. The traceback is where the *importlib* loaded data most recently: on the `import pdb` line of the *doctest* module. The traceback may change if a new module is loaded.

Pretty top

Code to display the 10 lines allocating the most memory with a pretty output, ignoring `<frozen importlib._bootstrap>` and `<unknown>` files:

```
import linecache
import os
import tracemalloc

def display_top(snapshot, key_type='lineno', limit=10):
    snapshot = snapshot.filter_traces((
        tracemalloc.Filter(False, "<frozen importlib._bootstrap>"),
        tracemalloc.Filter(False, "<unknown>"),
    ))
    top_stats = snapshot.statistics(key_type)

    print("Top %s lines" % limit)
    for index, stat in enumerate(top_stats[:limit], 1):
        frame = stat.traceback[0]
        print("#%s: %s:%s: %.1f KiB"
              % (index, frame.filename, frame.lineno, stat.size / 1024))
        line = linecache.getline(frame.filename, frame.lineno).strip()
        if line:
            print('    %s' % line)
```

(continua na próxima página)

(continuação da página anterior)

```
    other = top_stats[limit:]
    if other:
        size = sum(stat.size for stat in other)
        print("%s other: %.1f KiB" % (len(other), size / 1024))
    total = sum(stat.size for stat in top_stats)
    print("Total allocated size: %.1f KiB" % (total / 1024))

tracemalloc.start()

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
display_top(snapshot)
```

Example of output of the Python test suite:

```
Top 10 lines
#1: Lib/base64.py:414: 419.8 KiB
    _b85chars2 = [(a + b) for a in _b85chars for b in _b85chars]
#2: Lib/base64.py:306: 419.8 KiB
    _a85chars2 = [(a + b) for a in _a85chars for b in _a85chars]
#3: collections/__init__.py:368: 293.6 KiB
    exec(class_definition, namespace)
#4: Lib/abc.py:133: 115.2 KiB
    cls = super().__new__(mcls, name, bases, namespace)
#5: unittest/case.py:574: 103.1 KiB
    testMethod()
#6: Lib/linecache.py:127: 95.4 KiB
    lines = fp.readlines()
#7: urllib/parse.py:476: 71.8 KiB
    for a in _hexdig for b in _hexdig}
#8: <string>:5: 62.0 KiB
#9: Lib/_weakrefset.py:37: 60.0 KiB
    self.data = set()
#10: Lib/base64.py:142: 59.8 KiB
    _b32tab2 = [a + b for a in _b32tab for b in _b32tab]
6220 other: 3602.8 KiB
Total allocated size: 5303.1 KiB
```

See *Snapshot.statistics()* for more options.

Record the current and peak size of all traced memory blocks

The following code computes two sums like `0 + 1 + 2 + ...` inefficiently, by creating a list of those numbers. This list consumes a lot of memory temporarily. We can use *get_traced_memory()* and *reset_peak()* to observe the small memory usage after the sum is computed as well as the peak memory usage during the computations:

```
import tracemalloc

tracemalloc.start()

# Example code: compute a sum with a large temporary list
large_sum = sum(list(range(100000)))

first_size, first_peak = tracemalloc.get_traced_memory()

tracemalloc.reset_peak()
```

(continua na próxima página)

(continuação da página anterior)

```
# Example code: compute a sum with a small temporary list
small_sum = sum(list(range(1000)))

second_size, second_peak = tracemalloc.get_traced_memory()

print(f"{first_size=}, {first_peak=}")
print(f"{second_size=}, {second_peak=}")
```

Saída:

```
first_size=664, first_peak=3592984
second_size=804, second_peak=29704
```

Using *reset_peak()* ensured we could accurately record the peak during the computation of `small_sum`, even though it is much smaller than the overall peak size of memory blocks since the *start()* call. Without the call to *reset_peak()*, `second_peak` would still be the peak from the computation `large_sum` (that is, equal to `first_peak`). In this case, both peaks are much higher than the final memory usage, and which suggests we could optimise (by removing the unnecessary call to *list*, and writing `sum(range(...))`).

27.8.2 API

Funções

`tracemalloc.`**`clear_traces`**`()`

Clear traces of memory blocks allocated by Python.

See also *stop()*.

`tracemalloc.`**`get_object_traceback`**`(`*obj*`)`

Get the traceback where the Python object *obj* was allocated. Return a *Traceback* instance, or `None` if the *tracemalloc* module is not tracing memory allocations or did not trace the allocation of the object.

See also *gc.get_referrers()* and *sys.getsizeof()* functions.

`tracemalloc.`**`get_traceback_limit`**`()`

Get the maximum number of frames stored in the traceback of a trace.

The *tracemalloc* module must be tracing memory allocations to get the limit, otherwise an exception is raised.

The limit is set by the *start()* function.

`tracemalloc.`**`get_traced_memory`**`()`

Get the current size and peak size of memory blocks traced by the *tracemalloc* module as a tuple: `(current: int, peak: int)`.

`tracemalloc.`**`reset_peak`**`()`

Set the peak size of memory blocks traced by the *tracemalloc* module to the current size.

Do nothing if the *tracemalloc* module is not tracing memory allocations.

This function only modifies the recorded peak size, and does not modify or clear any traces, unlike *clear_traces()*. Snapshots taken with *take_snapshot()* before a call to *reset_peak()* can be meaningfully compared to snapshots taken after the call.

See also *get_traced_memory()*.

Adicionado na versão 3.9.

`tracemalloc.`**`get_tracemalloc_memory`**`()`

Get the memory usage in bytes of the *tracemalloc* module used to store traces of memory blocks. Return an *int*.

tracemalloc.**is_tracing**()

`True` if the *tracemalloc* module is tracing Python memory allocations, `False` otherwise.

See also *start()* and *stop()* functions.

tracemalloc.**start**(*nframe:* int = *1*)

Start tracing Python memory allocations: install hooks on Python memory allocators. Collected tracebacks of traces will be limited to *nframe* frames. By default, a trace of a memory block only stores the most recent frame: the limit is `1`. *nframe* must be greater or equal to `1`.

You can still read the original number of total frames that composed the traceback by looking at the *Traceback.total_nframe* attribute.

Storing more than `1` frame is only useful to compute statistics grouped by `'traceback'` or to compute cumulative statistics: see the *Snapshot.compare_to()* and *Snapshot.statistics()* methods.

Storing more frames increases the memory and CPU overhead of the *tracemalloc* module. Use the *get_tracemalloc_memory()* function to measure how much memory is used by the *tracemalloc* module.

The `PYTHONTRACEMALLOC` environment variable (`PYTHONTRACEMALLOC=NFRAME`) and the `-X tracemalloc=NFRAME` command line option can be used to start tracing at startup.

See also *stop()*, *is_tracing()* and *get_traceback_limit()* functions.

tracemalloc.**stop**()

Stop tracing Python memory allocations: uninstall hooks on Python memory allocators. Also clears all previously collected traces of memory blocks allocated by Python.

Call *take_snapshot()* function to take a snapshot of traces before clearing them.

See also *start()*, *is_tracing()* and *clear_traces()* functions.

tracemalloc.**take_snapshot**()

Take a snapshot of traces of memory blocks allocated by Python. Return a new *Snapshot* instance.

The snapshot does not include memory blocks allocated before the *tracemalloc* module started to trace memory allocations.

Tracebacks of traces are limited to *get_traceback_limit()* frames. Use the *nframe* parameter of the *start()* function to store more frames.

The *tracemalloc* module must be tracing memory allocations to take a snapshot, see the *start()* function.

See also the *get_object_traceback()* function.

DomainFilter

class tracemalloc.**DomainFilter**(*inclusive:* bool, *domain:* int)

Filter traces of memory blocks by their address space (domain).

Adicionado na versão 3.6.

inclusive

If *inclusive* is `True` (include), match memory blocks allocated in the address space *domain*.

If *inclusive* is `False` (exclude), match memory blocks not allocated in the address space *domain*.

domain

Address space of a memory block (`int`). Read-only property.

Filter

class tracemalloc.**Filter**(*inclusive:* bool, *filename_pattern:* str, *lineno:* int = *None*, *all_frames:* bool = *False*, *domain:* int = *None*)

Filter on traces of memory blocks.

See the *fnmatch.fnmatch()* function for the syntax of *filename_pattern*. The `'.pyc'` file extension is replaced with `'.py'`.

Exemplos:

- `Filter(True, subprocess.__file__)` only includes traces of the *subprocess* module
- `Filter(False, tracemalloc.__file__)` excludes traces of the *tracemalloc* module
- `Filter(False, "<unknown>")` excludes empty tracebacks

Alterado na versão 3.5: The `'.pyo'` file extension is no longer replaced with `'.py'`.

Alterado na versão 3.6: Added the *domain* attribute.

domain

Address space of a memory block (`int` or `None`).

tracemalloc uses the domain `0` to trace memory allocations made by Python. C extensions can use other domains to trace other resources.

inclusive

If *inclusive* is `True` (include), only match memory blocks allocated in a file with a name matching *filename_pattern* at line number *lineno*.

If *inclusive* is `False` (exclude), ignore memory blocks allocated in a file with a name matching *filename_pattern* at line number *lineno*.

lineno

Line number (`int`) of the filter. If *lineno* is `None`, the filter matches any line number.

filename_pattern

Filename pattern of the filter (`str`). Read-only property.

all_frames

If *all_frames* is `True`, all frames of the traceback are checked. If *all_frames* is `False`, only the most recent frame is checked.

This attribute has no effect if the traceback limit is `1`. See the *get_traceback_limit()* function and *Snapshot.traceback_limit* attribute.

Frame

class tracemalloc.**Frame**

Frame de um Traceback

The *Traceback* class is a sequence of *Frame* instances.

filename

Filename (`str`).

lineno

Número da linha (`int`).

Snapshot

class tracemalloc.**Snapshot**

Snapshot of traces of memory blocks allocated by Python.

The *take_snapshot()* function creates a snapshot instance.

compare_to(*old_snapshot:* Snapshot, *key_type:* str, *cumulative:* bool = *False*)

Compute the differences with an old snapshot. Get statistics as a sorted list of `StatisticDiff` instances grouped by *key_type*.

See the `Snapshot.statistics()` method for *key_type* and *cumulative* parameters.

The result is sorted from the biggest to the smallest by: absolute value of `StatisticDiff.size_diff`, `StatisticDiff.size`, absolute value of `StatisticDiff.count_diff`, `Statistic.count` and then by `StatisticDiff.traceback`.

dump(*filename*)

Write the snapshot into a file.

Use `load()` to reload the snapshot.

filter_traces(*filters*)

Create a new `Snapshot` instance with a filtered `traces` sequence, *filters* is a list of `DomainFilter` and `Filter` instances. If *filters* is an empty list, return a new `Snapshot` instance with a copy of the traces.

All inclusive filters are applied at once, a trace is ignored if no inclusive filters match it. A trace is ignored if at least one exclusive filter matches it.

Alterado na versão 3.6: `DomainFilter` instances are now also accepted in *filters*.

classmethod load(*filename*)

Load a snapshot from a file.

See also `dump()`.

statistics(*key_type:* str, *cumulative:* bool = *False*)

Get statistics as a sorted list of `Statistic` instances grouped by *key_type*:

key_type	description
`'filename'`	filename
`'lineno'`	filename and line number
`'traceback'`	traceback

If *cumulative* is `True`, cumulate size and count of memory blocks of all frames of the traceback of a trace, not only the most recent frame. The cumulative mode can only be used with *key_type* equals to `'filename'` and `'lineno'`.

The result is sorted from the biggest to the smallest by: `Statistic.size`, `Statistic.count` and then by `Statistic.traceback`.

traceback_limit

Maximum number of frames stored in the traceback of `traces`: result of the `get_traceback_limit()` when the snapshot was taken.

traces

Traces of all memory blocks allocated by Python: sequence of `Trace` instances.

The sequence has an undefined order. Use the `Snapshot.statistics()` method to get a sorted list of statistics.

Statistic

class tracemalloc.**Statistic**

Statistic on memory allocations.

`Snapshot.statistics()` returns a list of `Statistic` instances.

See also the `StatisticDiff` class.

count

Number of memory blocks (`int`).

size

Total size of memory blocks in bytes (`int`).

traceback

Traceback where the memory block was allocated, *Traceback* instance.

StatisticDiff

class tracemalloc.**StatisticDiff**

Statistic difference on memory allocations between an old and a new *Snapshot* instance.

Snapshot.compare_to() returns a list of *StatisticDiff* instances. See also the *Statistic* class.

count

Number of memory blocks in the new snapshot (`int`): `0` if the memory blocks have been released in the new snapshot.

count_diff

Difference of number of memory blocks between the old and the new snapshots (`int`): `0` if the memory blocks have been allocated in the new snapshot.

size

Total size of memory blocks in bytes in the new snapshot (`int`): `0` if the memory blocks have been released in the new snapshot.

size_diff

Difference of total size of memory blocks in bytes between the old and the new snapshots (`int`): `0` if the memory blocks have been allocated in the new snapshot.

traceback

Traceback where the memory blocks were allocated, *Traceback* instance.

Trace

class tracemalloc.**Trace**

Trace of a memory block.

The *Snapshot.traces* attribute is a sequence of *Trace* instances.

Alterado na versão 3.6: Added the *domain* attribute.

domain

Address space of a memory block (`int`). Read-only property.

tracemalloc uses the domain `0` to trace memory allocations made by Python. C extensions can use other domains to trace other resources.

size

Size of the memory block in bytes (`int`).

traceback

Traceback where the memory block was allocated, *Traceback* instance.

Traceback

class tracemalloc.**Traceback**

Sequence of *Frame* instances sorted from the oldest frame to the most recent frame.

A traceback contains at least `1` frame. If the `tracemalloc` module failed to get a frame, the filename `"<unknown>"` at line number `0` is used.

When a snapshot is taken, tracebacks of traces are limited to *get_traceback_limit()* frames. See the *take_snapshot()* function. The original number of frames of the traceback is stored in the *Traceback.total_nframe* attribute. That allows to know if a traceback has been truncated by the traceback limit.

The *Trace.traceback* attribute is an instance of *Traceback* instance.

Alterado na versão 3.7: Frames are now sorted from the oldest to the most recent, instead of most recent to oldest.

total_nframe

Total number of frames that composed the traceback before truncation. This attribute can be set to `None` if the information is not available.

Alterado na versão 3.9: The *Traceback.total_nframe* attribute was added.

format(*limit=None*, *most_recent_first=False*)

Format the traceback as a list of lines. Use the *linecache* module to retrieve lines from the source code. If *limit* is set, format the *limit* most recent frames if *limit* is positive. Otherwise, format the `abs(limit)` oldest frames. If *most_recent_first* is `True`, the order of the formatted frames is reversed, returning the most recent frame first instead of last.

Similar to the *traceback.format_tb()* function, except that *format()* does not include newlines.

Exemplo:

```
print("Traceback (most recent call first):")
for line in traceback:
    print(line)
```

Saída:

```
Traceback (most recent call first):
  File "test.py", line 9
    obj = Object()
  File "test.py", line 12
    tb = tracemalloc.get_object_traceback(f())
```

CAPÍTULO 28

Empacotamento e Distribuição de Software

Essas bibliotecas ajudam você a publicar e instalar software do Python. Embora esses módulos sejam projetados para funcionar em conjunto com o Python Package Index, eles também podem ser usados com um servidor de indexação local ou sem qualquer servidor de indexação.

28.1 `ensurepip` — Inicialização do instalador do `pip`

Adicionado na versão 3.4.

Código-fonte: Lib/ensurepip

O pacote *ensurepip* fornece suporte a fazer bootstrapping, ou seja, inicializar o instalador do `pip` em uma instalação existente do Python ou em um ambiente virtual. Essa abordagem de bootstrapping reflete o fato de que `pip` é um projeto independente com seu próprio ciclo de lançamento, e a última versão estável disponível é fornecida com manutenção e lançamentos de recursos do interpretador de referência CPython.

Na maioria dos casos, os usuários finais do Python não precisam invocar esse módulo diretamente (como `pip` deve ser inicializado por padrão), mas pode ser necessário se a instalação do `pip` foi ignorada ao instalar o Python (ou ao criar um ambiente virtual) ou após desinstalar explicitamente `pip`.

Nota

Este módulo *não* acessa a Internet. Todos os componentes necessários para iniciar o `pip` estão incluídos como partes internas do pacote.

Ver também

installing-index
O guia do usuário final para instalar pacotes Python

PEP 453: Inicialização explícita de pip em instalações Python
A justificativa e especificação originais para este módulo.

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em *plataformas móveis* ou *plataformas WebAssembly*.

28.1.1 Interface de linha de comando

A interface da linha de comando é chamada usando a opção `-m` do interpretador.

A invocação mais simples possível é:

```
python -m ensurepip
```

Essa invocação instalará `pip` se ainda não estiver instalada, mas, caso contrário, não fará nada. Para garantir que a versão instalada do `pip` seja pelo menos tão recente quanto a disponível do `ensurepip`, passe a opção `--upgrade`:

```
python -m ensurepip --upgrade
```

Por padrão, `pip` é instalado no ambiente virtual atual (se houver um ativo) ou nos pacotes de sites do sistema (se não houver um ambiente virtual ativo). O local da instalação pode ser controlado através de duas opções adicionais de linha de comando:

--root <dir>

Instala `pip` em relação ao diretório raiz fornecido, em vez da raiz do ambiente virtual atualmente ativo (se houver) ou a raiz padrão da instalação atual do Python.

--user

Instala `pip` no diretório de pacotes do site do usuário em vez de globalmente para a instalação atual do Python (essa opção não é permitida dentro de um ambiente virtual ativo).

Por padrão, os scripts `pipX` e `pipX.Y` serão instalados (onde X.Y representa a versão do Python usada para invocar `ensurepip`). Os scripts instalados podem ser controlados através de duas opções adicionais de linha de comando:

--altinstall

Se uma instalação alternativa for solicitada, o script `pipX` *não* será instalado.

--default-pip

Se uma instalação "pip padrão" for solicitada, o script `pip` será instalado junto com os dois scripts comuns.

Fornecer as duas opções de seleção de script acionará uma exceção.

28.1.2 API do módulo

O *ensurepip* expõe duas funções para uso programático:

ensurepip.**version**()

Retorna uma string que especifica a versão disponível do pip que será instalado ao inicializar um ambiente.

ensurepip.**bootstrap**(*root=None*, *upgrade=False*, *user=False*, *altinstall=False*, *default_pip=False*, *verbosity=0*)

Inicializa `pip` no ambiente atual ou designado.

root especifica um diretório raiz alternativo para instalar em relação a. Se *root* for `None`, a instalação utilizará o local de instalação padrão para o ambiente atual.

upgrade indica se deve ou não atualizar uma instalação existente de uma versão anterior do `pip` para a versão disponível.

user indica se é necessário usar o esquema do usuário em vez de instalar globalmente.

Por padrão, os scripts `pipX` e `pipX.Y` serão instalados (onde X.Y significa a versão atual do Python).

Se *altinstall* estiver definido, o `pipX` *não* será instalado.

Se *default_pip* estiver definido, o `pip` será instalado além dos dois scripts comuns.

Definir *altinstall* e *default_pip* acionará *ValueError*.

verbosity controla o nível de saída para *sys.stdout* da operação de inicialização.

Levanta um *evento de auditoria* `ensurepip.bootstrap` com o argumento `root`.

Nota

O processo de inicialização tem efeitos colaterais em `sys.path` e `os.environ`. Invocar a interface da linha de comando em um subprocesso permite que esses efeitos colaterais sejam evitados.

Nota

O processo de inicialização pode instalar módulos adicionais exigidos pelo `pip`, mas outro software não deve presumir que essas dependências sempre estarão presentes por padrão (como as dependências podem ser removidas em uma versão futura do `pip`).

28.2 `venv` — Criação de ambientes virtuais

Adicionado na versão 3.3.

Código-fonte: Lib/venv/

O módulo `venv` oferece suporte à criação de "ambientes virtuais" leves, cada um com seu próprio conjunto independente de pacotes Python instalados em seus diretórios `site`. Um ambiente virtual é criado sobre uma instalação Python existente, conhecido como o Python "base" do ambiente virtual, e por padrão é isolado dos pacotes no ambiente base, de forma que apenas aqueles explicitamente instalados no ambiente virtual estejam disponíveis. Consulte *Ambientes virtuais* e a *documentação sobre ambientes virtuais* do `site` para obter mais informações.

Quando usadas em um ambiente virtual, ferramentas de instalação comuns, como pip, instalarão pacotes Python em um ambiente virtual sem precisar ser instruído a fazê-lo explicitamente.

Um ambiente virtual é (dentre outras coisas):

- Usado para conter um interpretador Python específico e bibliotecas de software e binários necessários para dar suporte a um projeto (biblioteca ou aplicação). Eles são, por padrão, isolados de software em outros ambientes virtuais e de interpretadores e bibliotecas Python instalados no sistema operacional.
- Contido em um diretório, convencionalmente denominado `.venv` ou `venv` no diretório do projeto, ou em um diretório contêiner para vários ambientes virtuais, como `~/.virtualenvs`.
- Não inserido em sistemas de controle de código-fonte, como Git.
- Considerado descartável – deve ser simples excluí-lo e recriá-lo do zero. Você não coloca nenhum código de projeto no ambiente.
- Não é considerado móvel ou copiável – basta recriar o mesmo ambiente no local de destino.

Veja **PEP 405** para mais informações sobre ambientes virtuais do Python.

Ver também

Python Packaging User Guide: Criar e usar ambientes virtuais

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em *plataformas móveis* ou *plataformas WebAssembly*.

28.2.1 Criando ambientes virtuais

As *ambiente virtuais* são criadas executando o módulo `venv`:

```
python -m venv /caminho/para/novo/ambiente/virtual
```

Isso cria o diretório de destino (incluindo diretórios pais conforme necessário) e coloca um arquivo `pyvenv.cfg` nele com uma chave `home` apontando para a instalação do Python da qual o comando foi executado. Ele também cria um subdiretório `bin` (ou `Scripts` no Windows) contendo uma cópia ou link simbólico do executável Python (conforme apropriado para a plataforma ou argumentos usados no momento da criação do ambiente). Ele também cria um subdiretório `lib/pythonX.Y/site-packages` (no Windows, este é `Lib\site-packages`). Se um diretório existente for especificado, ele será reutilizado.

Alterado na versão 3.5: O uso de `venv` agora é recomendado para a criação de ambientes virtuais.

Deprecated since version 3.6, removed in version 3.8: **pyvenv** era a ferramenta recomendada para criar ambientes virtuais para Python 3.3 e 3.4, e foi substituída na versão 3.5 pela execução direta de `venv`.

No Windows, invoque o comando `venv` da seguinte forma:

```
PS> python -m venv C:\caminho\para\novo\ambiente\virtual
```

O comando, se executado com `-h`, mostrará as opções disponíveis:

```
usage: venv [-h] [--system-site-packages] [--symlinks | --copies] [--clear]
            [--upgrade] [--without-pip] [--prompt PROMPT] [--upgrade-deps]
            [--without-scm-ignore-files]
            ENV_DIR [ENV_DIR ...]

Creates virtual Python environments in one or more target directories.

Once an environment has been created, you may wish to activate it, e.g. by
sourcing an activate script in its bin directory.
```

ENV_DIR

Um argumento obrigatório que especifica o diretório para criar o ambiente.

--system-site-packages

Fornece ao ambiente virtual acesso ao diretório site-packages do sistema.

--symlinks

Tenta usar links simbólicos em vez de cópias, quando links simbólicos não forem o padrão para a plataforma.

--copies

Tenta usar cópias em vez de links simbólicos, mesmo quando os links simbólicos forem o padrão para a plataforma.

--clear

Exclui o conteúdo do diretório do ambiente, se ele já existir, antes da criação do ambiente.

--upgrade

Atualiza o diretório do ambiente para usar esta versão do Python, supondo que o Python tenha sido atualizado no local.

--without-pip

Ignora a instalação ou atualização do pip no ambiente virtual (o pip é inicializado por padrão).

--prompt `<PROMPT>`

Fornece um prefixo de prompt alternativo para este ambiente.

--upgrade-deps

Atualiza as dependências principais (pip) para a versão mais recente no PyPI.

--without-scm-ignore-files

Ignora a adição de arquivos para ignorar do SCM ao diretório do ambiente (o Git é suportado por padrão).

Alterado na versão 3.4: Instala o pip por padrão, adicionadas as opções `--without-pip` e `--copies`.

Alterado na versão 3.4: Nas versões anteriores, se o diretório de destino já existia, era levantado um erro, a menos que a opção `--clear` ou `--upgrade` fosse fornecida.

Alterado na versão 3.9: Adiciona a opção `--upgrade-deps` para atualizar pip + setuptools para a última no PyPI.

Alterado na versão 3.12: `setuptools` não é mais uma dependência central do venv.

Alterado na versão 3.13: Adicionada a opção `--without-scm-ignore-files`.

Alterado na versão 3.13: `venv` agora cria um arquivo `.gitignore` para Git por padrão.

Nota

Embora haja suporte a links simbólicos no Windows, eles não são recomendados. É importante notar que clicar duas vezes em `python.exe` no Explorador de Arquivos resolverá o link simbólico com entusiasmo e ignorará o ambiente virtual.

Nota

No Microsoft Windows, pode ser necessário ativar o script `Activate.ps1`, definindo a política de execução para o usuário. Você pode fazer isso executando o seguinte comando do PowerShell:

```
PS C:\> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Consulte About Execution Policies para mais informações.

O arquivo `pyvenv.cfg` criado também inclui a chave `include-system-site-packages`, definida como `true` se `venv` for executado com a opção `--system-site-packages`; caso contrário, `false`.

A menos que a opção `--without-pip` seja dada, *ensurepip* será chamado para inicializar o `pip` no ambiente virtual.

Vários caminhos podem ser dados para `venv`, caso em que um ambiente virtual idêntico será criado, de acordo com as opções fornecidas, em cada caminho fornecido.

28.2.2 Como funcionam os venvs

Quando um interpretador Python está sendo executado a partir de um ambiente virtual, *sys.prefix* e *sys.exec_prefix* apontam para os diretórios do ambiente virtual, enquanto *sys.base_prefix* e *sys.base_exec_prefix* apontam para aqueles do Python base usado para criar o ambiente. É suficiente verificar `sys.prefix != sys.base_prefix` para determinar se o interpretador atual está sendo executado a partir de um ambiente virtual.

Um ambiente virtual pode ser "ativado" usando um script em seu diretório binário (`bin` no POSIX; `Scripts` no Windows). Isso precederá esse diretório ao seu `PATH`, de modo que a execução de **python** invoque o interpretador Python do ambiente e você possa executar os scripts instalados sem precisar usar o caminho completo. A invocação do script de ativação é específica da plataforma (*<venv>* deve ser substituído pelo caminho para o diretório que contém o ambiente virtual):

Plataforma	Shell	Comando para ativar o ambiente virtual
POSIX	bash/zsh	`$ source <venv>/bin/activate`
	fish	`$ source <venv>/bin/activate.fish`
	csh/tcsh	`$ source <venv>/bin/activate.csh`
	pwsh	`$ <venv>/bin/Activate.ps1`
Windows	cmd.exe	`C:\> <venv>\Scripts\activate.bat`
	PowerShell	`PS C:\> <venv>\Scripts\Activate.ps1`

Adicionado na versão 3.4: Os scripts de ativação **`fish`** e **`csh`**.

Adicionado na versão 3.8: Scripts de ativação de PowerShell instalados sob POSIX para suporte a PowerShell Core.

Você não *precisa* especificamente ativar um ambiente virtual, pois pode apenas especificar o caminho completo para o interpretador Python desse ambiente ao invocar o Python. Além disso, todos os scripts instalados no ambiente devem ser executáveis sem ativá-lo.

Para isso, os scripts instalados em ambientes virtuais possuem uma linha "shebang" que aponta para o interpretador Python do ambiente `#!/<caminho-para-venv>/bin/python`. Isso significa que o script será executado com esse interpretador independentemente do valor de `PATH`. No Windows, o processamento de linha "shebang" é suportado se você tiver o launcher instalado. Assim, clicar duas vezes em um script instalado em uma janela do Windows Explorer deve executá-lo com o interpretador correto sem que o ambiente precise ser ativado ou no `PATH`.

Quando um ambiente virtual é ativado, a variável de ambiente `VIRTUAL_ENV` é definida como o caminho do ambiente. Como não é necessário ativar explicitamente um ambiente virtual para usá-lo, `VIRTUAL_ENV` não pode ser usado para determinar se um ambiente virtual está sendo usado.

Aviso

Como os scripts instalados em ambientes não devem esperar que o ambiente seja ativado, suas linhas shebang contêm os caminhos absolutos para os interpretadores de seus ambientes. Por causa disso, os ambientes são inerentemente não portáteis, no caso geral. Você sempre deve ter um meio simples de recriar um ambiente (por exemplo, se você tiver um arquivo de requisitos `requirements.txt`, você pode invocar `pip install -r requirements.txt` usando o `pip` do ambiente para instalar todos os pacotes necessários para o ambiente). Se por algum motivo você precisar mover o ambiente para um novo local, deverá recriá-lo no local desejado e excluir o do local antigo. Se você mover um ambiente porque moveu um diretório pai dele, deverá recriar o ambiente em seu novo local. Caso contrário, o software instalado no ambiente pode não funcionar conforme o esperado.

Você pode desativar um ambiente virtual digitando `deactivate` em seu shell. O mecanismo exato é específico da plataforma e é um detalhe de implementação interna (normalmente, um script ou função de shell será usado).

28.2.3 API

O método de alto nível descrito acima utiliza uma API simples que fornece mecanismos para que criadores de ambientes virtuais de terceiros personalizem a criação do ambiente de acordo com suas necessidades, a classe *`EnvBuilder`*.

class venv.**EnvBuilder**(*system_site_packages=False*, *clear=False*, *symlinks=False*, *upgrade=False*, *with_pip=False*, *prompt=None*, *upgrade_deps=False*, ***, *scm_ignore_files=frozenset()*)

A classe *`EnvBuilder`* aceita os seguintes argumentos nomeados na instanciação:

- *system_site_packages* – um valor booleano indicando que os pacotes de sites do sistema Python devem estar disponíveis para o ambiente (o padrão é `False`).
- *clear* – um valor booleano que, se verdadeiro, excluirá o conteúdo de qualquer diretório de destino existente, antes de criar o ambiente.
- *symlinks* – um valor booleano que indica se você deseja vincular o binário Python ao invés de copiar.

- *upgrade* – um valor booleano que, se verdadeiro, atualizará um ambiente existente com o Python em execução - para uso quando o Python tiver sido atualizado localmente (o padrão é `False`).
- *with_pip* – um valor booleano que, se verdadeiro, garante que o pip seja instalado no ambiente virtual. Isso usa *ensurepip* com a opção `--default-pip`.
- *prompt* – uma string a ser usada após o ambiente virtual ser ativado (o padrão é `None`, o que significa que o nome do diretório do ambiente seria usado). Se a string especial `"."` for fornecida, o nome da base do diretório atual será usado como prompt.
- *upgrade_deps* – Atualiza os módulos base do venv para os mais recentes no PyPI
- *scm_ignore_files* – Cria arquivos com lista de ignorados com base nos gerenciadores de controle de fonte (SCM) especificados no iterável. O suporte é definido por ter um método chamado `create_{scm}_ignore_file`. O único valor suportado por padrão é `"git"` via *create_git_ignore_file()*.

Alterado na versão 3.4: Adicionado o parâmetro `with_pip`

Alterado na versão 3.6: Adicionado o parâmetro `prompt`

Alterado na versão 3.9: Adicionado o parâmetro `upgrade_deps`

Alterado na versão 3.13: Adicionado o parâmetro `scm_ignore_files`

EnvBuilder pode ser usado como uma classe base.

create(*env_dir*)

Cria um ambiente virtual especificando o diretório de destino (absoluto ou relativo ao diretório atual) que deve conter o ambiente virtual. O método `create` cria o ambiente no diretório especificado ou levanta uma exceção apropriada.

O método `create` da classe *EnvBuilder* ilustra os ganchos disponíveis para personalização de subclasse:

```
def create(self, env_dir):
    """
    Cria um ambiente Python virtualizado em um diretório.
    env_dir é o diretório de destino para criar um ambiente.
    """
    env_dir = os.path.abspath(env_dir)
    context = self.ensure_directories(env_dir)
    self.create_configuration(context)
    self.setup_python(context)
    self.setup_scripts(context)
    self.post_setup(context)
```

Cada um dos métodos *ensure_directories()*, *create_configuration()*, *setup_python()*, *setup_scripts()* e *post_setup()* pode ser substituído.

ensure_directories(*env_dir*)

Cria o diretório do ambiente e todos os subdiretórios necessários que ainda não existem e retorna um objeto de contexto. Este objeto de contexto é apenas um detentor de atributos (como caminhos) para uso pelos outros métodos. Se o *EnvBuilder* for criado com o argumento `clear=True`, o conteúdo do diretório do ambiente será limpo e todos os subdiretórios necessários serão recriados.

O objeto de contexto retornado é um *types.SimpleNamespace* com os seguintes atributos:

- `env_dir` - A localização do ambiente virtual. Usado para `__VENV_DIR__` em scripts de ativação (veja *install_scripts()*).
- `env_name` - O nome do ambiente virtual. Usado para `__VENV_NAME__` em scripts de ativação (veja *install_scripts()*).
- `prompt` - O prompt a ser usado pelos scripts de ativação. Usado para `__VENV_PROMPT__` em scripts de ativação (veja *install_scripts()*).

- `executable` - O executável Python subjacente usado pelo ambiente virtual. Isso leva em consideração o caso em que um ambiente virtual é criado a partir de outro ambiente virtual.
- `inc_path` - O caminho de inclusão para o ambiente virtual.
- `lib_path` - O caminho purelib para o ambiente virtual.
- `bin_path` - O caminho do script para o ambiente virtual.
- `bin_name` - O nome do caminho do script relativo ao local do ambiente virtual. Usado para `__VENV_BIN_NAME__` em scripts de ativação (consulte *`install_scripts()`*).
- `env_exe` - O nome do interpretador Python no ambiente virtual. Usado para `__VENV_PYTHON__` em scripts de ativação (veja *`install_scripts()`*).
- `env_exec_cmd` - O nome do interpretador Python, levando em consideração os redirecionamentos do sistema de arquivos. Isso pode ser usado para executar o Python no ambiente virtual.

Alterado na versão 3.11: O *esquema de instalação de sysconfig* do *venv* é usado para construir os caminhos dos diretórios criados.

Alterado na versão 3.12: O atributo `lib_path` foi adicionado ao contexto, e o objeto de contexto foi documentado.

`create_configuration`(*context*)

Cria o arquivo de configuração `pyvenv.cfg` no ambiente.

`setup_python`(*context*)

Cria uma cópia ou link simbólico para o executável Python no ambiente. Nos sistemas POSIX, se um executável específico `python3.x` foi usado, links simbólicos para `python` e `python3` serão criados apontando para esse executável, a menos que já existam arquivos com esses nomes.

`setup_scripts`(*context*)

Instala scripts de ativação apropriados para a plataforma no ambiente virtual.

`upgrade_dependencies`(*context*)

Atualiza os principais pacotes de dependência do venv (atualmente pip) no ambiente. Isso é feito através da distribuição do executável `pip` no ambiente.

Adicionado na versão 3.9.

Alterado na versão 3.12: setuptools não é mais uma dependência central do venv.

`post_setup`(*context*)

Um método de espaço reservado que pode ser substituído em implementações de terceiros para pré-instalar pacotes no ambiente virtual ou executar outras etapas pós-criação.

`install_scripts`(*context*, *path*)

Este método utilitário que pode ser chamado de *`setup_scripts()`* ou *`post_setup()`* nas subclasses para ajudar na instalação de scripts personalizados no ambiente virtual.

path é o caminho para um diretório que deve conter subdiretórios `common`, `posix`, `nt`; cada um contendo scripts destinados ao diretório `bin` no ambiente. O conteúdo de `common` e o diretório correspondente a *`os.name`* são copiados após alguma substituição de texto dos espaços reservados:

- `__VENV_DIR__` é substituído pelo caminho absoluto do diretório do ambiente.
- `__VENV_NAME__` é substituído pelo nome do ambiente (segmento do caminho final do diretório do ambiente).
- `__VENV_PROMPT__` é substituído pelo prompt (o nome do ambiente entre parênteses e com o seguinte espaço)
- `__VENV_BIN_NAME__` é substituído pelo nome do diretório bin (`bin` ou `Scripts`).
- `__VENV_PYTHON__` é substituído pelo caminho absoluto do executável do ambiente.

É permitido que os diretórios existam (para quando um ambiente existente estiver sendo atualizado).

create_git_ignore_file(*context*)

Cria um arquivo `.gitignore` dentro do ambiente virtual que faz com que todo o diretório seja ignorado pelo gerenciador de controle de fonte Git.

Adicionado na versão 3.13.

Alterado na versão 3.7.2: O Windows agora usa scripts redirecionadores para `python[w].exe` em vez de copiar os binários reais. No 3.7.2, somente *setup_python()* não faz nada a menos que seja executado a partir de uma construção na árvore de origem.

Alterado na versão 3.7.3: O Windows copia os scripts redirecionadores como parte do *setup_python()* em vez de *setup_scripts()*. Este não foi o caso em 3.7.2. Ao usar links simbólicos, será feito link para os executáveis originais.

Há também uma função de conveniência no nível do módulo:

venv.**create**(*env_dir*, *system_site_packages=False*, *clear=False*, *symlinks=False*, *with_pip=False*, *prompt=None*, *upgrade_deps=False*, ***, *scm_ignore_files=frozenset()*)

Cria um *EnvBuilder* com os argumentos nomeados fornecidos e chame seu método *create()* com o argumento *env_dir*.

Adicionado na versão 3.3.

Alterado na versão 3.4: Adicionado o parâmetro *with_pip*

Alterado na versão 3.6: Adicionado o parâmetro *prompt*

Alterado na versão 3.9: Adicionado o parâmetro *upgrade_deps*

Alterado na versão 3.13: Adicionado o parâmetro *scm_ignore_files*

28.2.4 Um exemplo de extensão de `EnvBuilder`

O script a seguir mostra como estender *EnvBuilder* implementando uma subclasse que instala setuptools e pip em um ambiente virtual criado:

```
import os
import os.path
from subprocess import Popen, PIPE
import sys
from threading import Thread
from urllib.parse import urlparse
from urllib.request import urlretrieve
import venv

class ExtendedEnvBuilder(venv.EnvBuilder):
    """
    This builder installs setuptools and pip so that you can pip or
    easy_install other packages into the created virtual environment.

    :param nodist: If true, setuptools and pip are not installed into the
                   created virtual environment.
    :param nopip: If true, pip is not installed into the created
                  virtual environment.
    :param progress: If setuptools or pip are installed, the progress of the
                     installation can be monitored by passing a progress
                     callable. If specified, it is called with two
                     arguments: a string indicating some progress, and a
                     context indicating where the string is coming from.
                     The context argument can have one of three values:
                     'main', indicating that it is called from virtualize()
                     itself, and 'stdout' and 'stderr', which are obtained
```

(continua na próxima página)

(continuação da página anterior)

```
                    by reading lines from the output streams of a subprocess
                    which is used to install the app.

                    If a callable is not specified, default progress
                    information is output to sys.stderr.
    """

    def __init__(self, *args, **kwargs):
        self.nodist = kwargs.pop('nodist', False)
        self.nopip = kwargs.pop('nopip', False)
        self.progress = kwargs.pop('progress', None)
        self.verbose = kwargs.pop('verbose', False)
        super().__init__(*args, **kwargs)

    def post_setup(self, context):
        """
        Set up any packages which need to be pre-installed into the
        virtual environment being created.

        :param context: The information for the virtual environment
                        creation request being processed.
        """
        os.environ['VIRTUAL_ENV'] = context.env_dir
        if not self.nodist:
            self.install_setuptools(context)
        # Can't install pip without setuptools
        if not self.nopip and not self.nodist:
            self.install_pip(context)

    def reader(self, stream, context):
        """
        Read lines from a subprocess' output stream and either pass to a progress
        callable (if specified) or write progress information to sys.stderr.
        """
        progress = self.progress
        while True:
            s = stream.readline()
            if not s:
                break
            if progress is not None:
                progress(s, context)
            else:
                if not self.verbose:
                    sys.stderr.write('.')
                else:
                    sys.stderr.write(s.decode('utf-8'))
                sys.stderr.flush()
        stream.close()

    def install_script(self, context, name, url):
        _, _, path, _, _, _ = urlparse(url)
        fn = os.path.split(path)[-1]
        binpath = context.bin_path
        distpath = os.path.join(binpath, fn)
        # Download script into the virtual environment's binaries folder
        urlretrieve(url, distpath)
```

(continua na próxima página)

(continuação da página anterior)

```
        progress = self.progress
        if self.verbose:
            term = '\n'
        else:
            term = ''
        if progress is not None:
            progress('Installing %s ...%s' % (name, term), 'main')
        else:
            sys.stderr.write('Installing %s ...%s' % (name, term))
            sys.stderr.flush()
        # Install in the virtual environment
        args = [context.env_exe, fn]
        p = Popen(args, stdout=PIPE, stderr=PIPE, cwd=binpath)
        t1 = Thread(target=self.reader, args=(p.stdout, 'stdout'))
        t1.start()
        t2 = Thread(target=self.reader, args=(p.stderr, 'stderr'))
        t2.start()
        p.wait()
        t1.join()
        t2.join()
        if progress is not None:
            progress('done.', 'main')
        else:
            sys.stderr.write('done.\n')
        # Clean up - no longer needed
        os.unlink(distpath)

    def install_setuptools(self, context):
        """
        Install setuptools in the virtual environment.

        :param context: The information for the virtual environment
                        creation request being processed.
        """
        url = "https://bootstrap.pypa.io/ez_setup.py"
        self.install_script(context, 'setuptools', url)
        # clear up the setuptools archive which gets downloaded
        pred = lambda o: o.startswith('setuptools-') and o.endswith('.tar.gz')
        files = filter(pred, os.listdir(context.bin_path))
        for f in files:
            f = os.path.join(context.bin_path, f)
            os.unlink(f)

    def install_pip(self, context):
        """
        Install pip in the virtual environment.

        :param context: The information for the virtual environment
                        creation request being processed.
        """
        url = 'https://bootstrap.pypa.io/get-pip.py'
        self.install_script(context, 'pip', url)


def main(args=None):
    import argparse
```

(continua na próxima página)

(continuação da página anterior)

```
    parser = argparse.ArgumentParser(prog=__name__,
                                     description='Creates virtual Python '
                                                 'environments in one or '
                                                 'more target '
                                                 'directories.')
    parser.add_argument('dirs', metavar='ENV_DIR', nargs='+',
                        help='A directory in which to create the '
                             'virtual environment.')
    parser.add_argument('--no-setuptools', default=False,
                        action='store_true', dest='nodist',
                        help="Don't install setuptools or pip in the "
                             "virtual environment.")
    parser.add_argument('--no-pip', default=False,
                        action='store_true', dest='nopip',
                        help="Don't install pip in the virtual "
                             "environment.")
    parser.add_argument('--system-site-packages', default=False,
                        action='store_true', dest='system_site',
                        help='Give the virtual environment access to the '
                             'system site-packages dir.')
    if os.name == 'nt':
        use_symlinks = False
    else:
        use_symlinks = True
    parser.add_argument('--symlinks', default=use_symlinks,
                        action='store_true', dest='symlinks',
                        help='Try to use symlinks rather than copies, '
                             'when symlinks are not the default for '
                             'the platform.')
    parser.add_argument('--clear', default=False, action='store_true',
                        dest='clear', help='Delete the contents of the '
                                           'virtual environment '
                                           'directory if it already '
                                           'exists, before virtual '
                                           'environment creation.')
    parser.add_argument('--upgrade', default=False, action='store_true',
                        dest='upgrade', help='Upgrade the virtual '
                                             'environment directory to '
                                             'use this version of '
                                             'Python, assuming Python '
                                             'has been upgraded '
                                             'in-place.')
    parser.add_argument('--verbose', default=False, action='store_true',
                        dest='verbose', help='Display the output '
                                             'from the scripts which '
                                             'install setuptools and pip.')
    options = parser.parse_args(args)
    if options.upgrade and options.clear:
        raise ValueError('you cannot supply --upgrade and --clear together.')
    builder = ExtendedEnvBuilder(system_site_packages=options.system_site,
                                 clear=options.clear,
                                 symlinks=options.symlinks,
                                 upgrade=options.upgrade,
                                 nodist=options.nodist,
                                 nopip=options.nopip,
```

(continua na próxima página)

(continuação da página anterior)

```
                                    verbose=options.verbose)
    for d in options.dirs:
        builder.create(d)

if __name__ == '__main__':
    rc = 1
    try:
        main()
        rc = 0
    except Exception as e:
        print('Error: %s' % e, file=sys.stderr)
    sys.exit(rc)
```

Esse script também está disponível para download online.

28.3 `zipapp` — Gerencia arquivos zip executáveis do Python

Adicionado na versão 3.5.

Código-fonte: Lib/zipapp.py

Este módulo fornece ferramentas para gerenciar a criação de arquivos zip contendo código Python, que pode ser executado diretamente pelo interpretador Python. O módulo fornece uma *Interface de linha de comando* e uma *API do Python*.

28.3.1 Exemplo básico

O exemplo a seguir mostra como a *Interface de linha de comando* pode ser usada para criar um arquivo executável a partir de um diretório contendo código Python. Quando executado, o arquivo executará a função `main` do módulo `myapp` no arquivo.

```
$ python -m zipapp myapp -m "myapp:main"
$ python myapp.pyz
<output from myapp>
```

28.3.2 Interface de linha de comando

Quando chamado como um programa a partir da linha de comando, as seguintes opções estão disponíveis:

```
$ python -m zipapp source [options]
```

Se *source* for um diretório, isso criará um arquivo a partir do conteúdo de *source*. Se *source* for um arquivo, ele deve ser um arquivo, e será copiado para o arquivo de destino (ou o conteúdo de sua linha shebang será exibido se a opção –info for especificada).

As seguintes opções são compreendidas:

-o <output>, **--output**=<output>

Escreve a saída em um arquivo chamado *output*. Se essa opção não for especificada, o nome do arquivo de saída será o mesmo que o *source* de entrada, com a extensão `.pyz` adicionada. Se um nome de arquivo explícito for fornecido, ele será usado como está (então uma extensão `.pyz` deve ser incluída se necessário).

Um nome de arquivo de saída deve ser especificado se *source* for um arquivo (e nesse caso, *output* não deve ser o mesmo que *source*).

-p <interpreter>, **--python**=<interpreter>

Adiciona uma linha `#!` ao arquivo especificando *interpreter* como o comando a ser executado. Além disso, no POSIX, torne o arquivo executável. O padrão é não escrever nenhuma linha `#!` e não tornar o arquivo executável.

-m <mainfn>, **--main**=<mainfn>

Escreve um arquivo `__main__.py` no arquivo que executa *mainfn*. O argumento *mainfn* deve ter o formato "pkg.mod:fn", onde "pkg.mod" é um pacote/módulo no arquivo, e "fn" é um chamável no módulo fornecido. O arquivo `__main__.py` executará esse chamável.

--main não pode ser especificado ao copiar um arquivo.

-c, --compress

Compacta arquivos com o método deflate, reduzindo o tamanho do arquivo de saída. Por padrão, os arquivos são armazenados descompactados no arquivo.

--compress não tem efeito ao copiar um arquivo.

Adicionado na versão 3.7.

--info

Exibe o interpretador incorporado no arquivo, para fins de diagnóstico. Neste caso, quaisquer outras opções são ignoradas e SOURCE deve ser um arquivo, não um diretório.

-h, --help

Exibe uma mensagem curta de uso e sai

28.3.3 API do Python

O módulo define duas funções de conveniência:

zipapp.**create_archive**(*source*, *target=None*, *interpreter=None*, *main=None*, *filter=None*, *compressed=False*)

Cria um arquivo de aplicação a partir de *source*. A fonte pode ser qualquer um dos seguintes:

- O nome de um diretório, ou um *objeto caminho ou similar* referente a um diretório, caso em que um novo arquivo de aplicação será criado a partir do conteúdo desse diretório.
- O nome de um arquivo de aplicação existente, ou um *objeto caminho ou similar* referindo-se a tal arquivo, em cujo caso o arquivo é copiado para o alvo (modificando-o para refletir o valor dado para o argumento *interpreter*). O nome do arquivo deve incluir a extensão `.pyz`, se necessário.
- Um objeto arquivo aberto para leitura no modo bytes. O conteúdo do arquivo deve ser um arquivo de aplicação, e o objeto arquivo é presumido como estando posicionado no início do arquivo.

O argumento *target* determina onde o arquivo resultante será escrito:

- Se for o nome de um arquivo ou um *objeto caminho ou similar*, o arquivo será escrito nesse arquivo.
- Se for um objeto arquivo aberto, o arquivo será escrito naquele objeto arquivo, que deve estar aberto para escrita no modo bytes.
- Se o destino for omitido (ou `Nenhum`), a fonte deverá ser um diretório e o destino será um arquivo com o mesmo nome da fonte, com uma extensão `.pyz` adicionada.

O argumento *interpreter* especifica o nome do interpretador Python com o qual o arquivo será executado. Ele é escrito como uma linha "shebang" no início do arquivo. No POSIX, isso será interpretado pelo sistema operacional, e no Windows será manipulado pelo inicializador Python. Omitir o *interpreter* resulta em nenhuma linha shebang sendo escrita. Se um interpretador for especificado, e o alvo for um nome de arquivo, o bit executável do arquivo alvo será definido.

O argumento *main* especifica o nome de um chamável que será usado como o programa principal para o arquivo. Ele só pode ser especificado se a fonte for um diretório, e a fonte ainda não contiver um arquivo `__main__.py`. O argumento *main* deve ter o formato "pkg.módulo:chamável" e o arquivo será executado importando "pkg.módulo" e executando o chamável fornecido sem argumentos. É um erro omitir *main* se a

fonte for um diretório e não contiver um arquivo `__main__.py`, pois, caso contrário, o arquivo resultante não seria executável.

O argumento opcional *filter* especifica uma função de retorno de chamada que recebe um objeto Path representando o caminho para o arquivo que está sendo adicionado (relativo ao diretório da fonte). Ele deve retornar `True` se o arquivo for adicionado.

O argumento opcional *compressed* determina se os arquivos são compactados. Se definido como `True`, os arquivos no arquivo são compactados com o método deflate; caso contrário, os arquivos são armazenados descompactados. Este argumento não tem efeito ao copiar um arquivo existente.

Se um objeto arquivo for especificado para *source* ou *target*, é responsabilidade do chamador fechá-lo após chamar create_archive.

Ao copiar um arquivo existente, os objetos arquivo fornecidos precisam apenas dos métodos `read` e `readline` ou `write`. Ao criar um arquivo de um diretório, se o alvo for um objeto arquivo, ele será passado para a classe `zipfile.ZipFile` e deve fornecer os métodos necessários para essa classe.

Alterado na versão 3.7: Adicionados os parâmetros *filter* e *compressed*.

`zipapp.`**`get_interpreter`**(*archive*)

Retorna o interpretador especificado na linha `#!` no início do arquivo. Se não houver nenhuma linha `#!`, retorna *None*. O argumento *archive* pode ser um nome de arquivo ou um objeto arquivo ou similar aberto para leitura no modo bytes. É presumido esteja no início do arquivo.

28.3.4 Exemplos

Compacte um diretório em um arquivo e execute-o.

```
$ python -m zipapp myapp
$ python myapp.pyz
<output from myapp>
```

O mesmo pode ser feito usando a função *create_archive()*:

```
>>> import zipapp
>>> zipapp.create_archive('myapp', 'myapp.pyz')
```

Para tornar o aplicativo diretamente executável no POSIX, especifique um interpretador a ser usado.

```
$ python -m zipapp myapp -p "/usr/bin/env python"
$ ./myapp.pyz
<saída de myapp>
```

Para substituir a linha shebang em um arquivo existente, crie um arquivo modificado usando a função *create_archive()*:

```
>>> import zipapp
>>> zipapp.create_archive('old_archive.pyz', 'new_archive.pyz', '/usr/bin/python3')
```

Para atualizar o arquivo no local, faça a substituição na memória usando um objeto *BytesIO* e, em seguida, sobrescreva a fonte depois. Observe que há um risco ao sobrescrever um arquivo no local de que um erro resultará na perda do arquivo original. Este código não protege contra tais erros, mas o código de produção deve fazê-lo. Além disso, este método só funcionará se o arquivo couber na memória:

```
>>> import zipapp
>>> import io
>>> temp = io.BytesIO()
>>> zipapp.create_archive('myapp.pyz', temp, '/usr/bin/python2')
>>> with open('myapp.pyz', 'wb') as f:
>>>     f.write(temp.getvalue())
```

28.3.5 Especificando o interpretador

Observe que se você especificar um interpretador e então distribuir seu arquivo de aplicação, você precisa garantir que o interpretador usado seja portátil. O inicializador Python para Windows oferece suporte às formas mais comuns da linha POSIX `#!`, mas há outras questões a serem consideradas:

- Se você usar "/usr/bin/env python" (ou outras formas do comando "python", como "/usr/bin/python"), você precisa considerar que seus usuários podem ter Python 2 ou Python 3 como padrão, e escrever seu código para funcionar em ambas as versões.
- Se você usar uma versão explícita, por exemplo "/usr/bin/env python3", sua aplicação não funcionará para usuários que não tenham essa versão. (Isso pode ser o que você quer se não tiver tornado seu código compatível com Python 2).
- Não há como dizer "python X.Y ou posterior", então tome cuidado ao usar uma versão exata como "/usr/bin/env python3.4", pois você precisará alterar sua linha shebang para usuários do Python 3.5, por exemplo.

Normalmente, você deve usar "/usr/bin/env python2" ou "/usr/bin/env python3", dependendo se seu código foi escrito para Python 2 ou 3.

28.3.6 Criando aplicações autônomas com zipapp

Usando o módulo *zipapp*, é possível criar programas Python autocontidos, que podem ser distribuídos para usuários finais que precisam apenas ter uma versão adequada do Python instalada em seu sistema. A chave para fazer isso é agrupar todas as dependências da aplicação no arquivo, junto com o código da aplicação.

As etapas para criar um arquivo autônomo são as seguintes:

1. Crie sua aplicação em um diretório normalmente, para que você tenha um diretório `myapp` contendo um arquivo `__main__.py` e qualquer código de aplicação de suporte.
2. Instale todas as dependências da sua aplicação no diretório `myapp`, usando pip:

   ```
   $ python -m pip install -r requirements.txt --target myapp
   ```

   (isso presume que você tenha os requisitos do seu projeto em um arquivo `requirements.txt` - caso contrário, você pode simplesmente listar as dependências manualmente na linha de comando do pip).
3. Empacote a aplicação usando:

   ```
   $ python -m zipapp -p "interpreter" myapp
   ```

Isso produzirá um executável autônomo, que pode ser executado em qualquer máquina com o interpretador apropriado disponível. Veja *Especificando o interpretador* para detalhes. Ele pode ser enviado aos usuários como um único arquivo.

No Unix, o arquivo `myapp.pyz` é executável como está. Você pode renomear o arquivo para remover a extensão `.pyz` se preferir um nome de comando "simples". No Windows, o arquivo `myapp.pyz[w]` é executável em virtude do fato de que o interpretador Python registra as extensões de arquivo `.pyz` e `.pyzw` quando instalado.

Advertência

Se sua aplicação depende de um pacote que inclui uma extensão C, esse pacote não pode ser executado a partir de um arquivo zip (essa é uma limitação do sistema operacional, pois o código executável deve estar presente no sistema de arquivos para que o carregador do sistema operacional o carregue). Nesse caso, você pode excluir essa dependência do arquivo zip e exigir que seus usuários o tenham instalado ou enviá-lo junto com seu arquivo zip e adicionar código ao seu `__main__.py` para incluir o diretório que contém o módulo descompactado em `sys.path`. Nesse caso, você precisará certificar-se de enviar binários apropriados para sua(s) arquitetura(s) de destino (e potencialmente escolher a versão correta para adicionar ao `sys.path` no tempo de execução, com base na máquina do usuário).

28.3.7 O formato de arquivo da aplicação zip Python

O Python tem sido capaz de executar arquivos zip que contêm um arquivo `__main__.py` desde a versão 2.6. Para ser executado pelo Python, um arquivo de aplicação simplesmente tem que ser um arquivo zip padrão contendo um arquivo `__main__.py` que será executado como o ponto de entrada para o aplicativo. Como de costume para qualquer script Python, o pai do script (neste caso, o arquivo zip) será colocado em *sys.path* e, portanto, outros módulos podem ser importados do arquivo zip.

O formato de arquivo zip permite que dados arbitrários sejam prefixados a um arquivo zip. O formato de aplicação zip usa essa habilidade para prefixar uma linha "shebang" padrão POSIX ao arquivo (`#!/caminho/para/interpretador`).

Formalmente, o formato de aplicação zip Python é:

1. Uma linha shebang opcional, contendo os caracteres `b'#!'` seguidos por um nome de interpretador e, em seguida, um caractere de nova linha (`b'\n'`). O nome do interpretador pode ser qualquer coisa aceitável para o processamento "shebang" do sistema operacional ou o inicializador Python no Windows. O interpretador deve ser codificado em UTF-8 no Windows e em *sys.getfilesystemencoding()* no POSIX.
2. Dados padrão do zipfile, conforme gerados pelo módulo *zipfile*. O conteúdo do zipfile *deve* incluir um arquivo chamado `__main__.py` (que deve estar na "raiz" do zipfile - ou seja, não pode estar em um subdiretório). Os dados do zipfile podem ser compactados ou descompactados.

Se um arquivo de aplicação tiver uma linha shebang, ele poderá ter o bit executável definido em sistemas POSIX, para permitir que seja executado diretamente.

Não há exigência de que as ferramentas neste módulo sejam usadas para criar arquivos de aplicações - o módulo é uma conveniência, mas arquivos no formato acima criados por qualquer meio são aceitáveis para Python.

CAPÍTULO 29

Serviços de Tempo de Execução Python

Os módulos descritos neste capítulo oferecem uma ampla gama de serviços relacionados ao interpretador Python e sua interação com o ambiente. Aqui está uma visão geral:

29.1 `sys` — Parâmetros e funções específicas do sistema

Este módulo fornece acesso a algumas variáveis usadas ou mantidas pelo interpretador e a funções que interagem fortemente com o interpretador. Sempre disponível. A menos que explicitamente indicado o contrário, todas as variáveis são somente leitura.

`sys.`**`abiflags`**

Em sistemas POSIX onde Python foi construído com o script `configure` padrão, ele contém os sinalizadores ABI conforme especificado por **PEP 3149**.

Adicionado na versão 3.2.

Alterado na versão 3.8: Os sinalizadores padrões se tornaram uma string vazia (o sinalizador `m` para pymalloc foi removido).

Disponibilidade: Unix.

`sys.`**`addaudithook`**(*hook*)

Anexa o *hook* chamável à lista de ganchos de auditoria ativos para o (sub)interpretador atual.

Quando um evento de auditoria é levantado através da função *sys.audit()*, cada gancho será chamado na ordem em que foi adicionado com o nome do evento e a tupla de argumentos. Ganchos nativos adicionados por `PySys_AddAuditHook()` são chamados primeiro, seguidos por ganchos adicionados no (sub)interpretador atual. Os ganchos podem registrar o evento, levantar uma exceção para cancelar a operação ou encerrar o processo completamente.

Observe que os ganchos de auditoria são principalmente para coletar informações sobre ações internas ou não observáveis, seja por Python ou bibliotecas escritas em Python. Eles não são adequados para implementar um ambiente isolado estilo "sandbox". Em particular, o código malicioso pode desabilitar ou ignorar os ganchos adicionados usando esta função. No mínimo, quaisquer ganchos sensíveis à segurança devem ser adicionados usando a API C `PySys_AddAuditHook()` antes de inicializar o tempo de execução, e quaisquer módulos que permitam modificação arbitrária da memória (como *ctypes*) devem ser completamente removidos ou monitorados de perto.

Chamar *sys.addaudithook()* levantará um evento de auditoria denominado `sys.addaudithook` sem argumentos. Se qualquer gancho existente levantar uma exceção derivada de *RuntimeError*, o novo gancho não será adicionado e a exceção será suprimida. Como resultado, os chamadores não podem presumir que seu gancho foi adicionado, a menos que controlem todos os ganchos existentes.

Veja *tabela de eventos de auditoria* para todos os eventos levantados pelo CPython, e **PEP 578** para a discussão do projeto original.

Adicionado na versão 3.8.

Alterado na versão 3.8.1: Exceções derivadas de *Exception*, mas não de *RuntimeError*, não são mais suprimidas.

Quando o rastreamento está ativado (consulte *settrace()*), os ganchos do Python são rastreados apenas se o chamável tiver um membro `__cantrace__` definido como um valor verdadeiro. Caso contrário, as funções de rastreamento ignorarão o gancho.

`sys.`**`argv`**

A lista de argumentos de linha de comando passados para um script Python. `argv[0]` é o nome do script (depende do sistema operacional se este é um nome de caminho completo ou não). Se o comando foi executado usando a opção de linha de comando `-c` para o interpretador, `argv[0]` é definido como a string `'-c'`. Se nenhum nome de script foi passado para o interpretador Python, `argv[0]` é a string vazia.

Para percorrer a entrada padrão ou a lista de arquivos fornecida na linha de comando, consulte o módulo *fileinput*.

Veja também *sys.orig_argv*.

> **Nota**
>
> No Unix, os argumentos da linha de comando são passados por bytes do sistema operacional. O Python os decodifica com a codificação do sistema de arquivos e o tratador de erros "surrogateescape". Quando você precisar de bytes originais, você pode obtê-los por `[os.fsencode(arg) for arg in sys.argv]`.

`sys.`**`audit`**(*event*, **args*)

Levanta um evento de auditoria e aciona quaisquer ganchos de auditoria ativos. *event* é uma string que identifica o evento e *args* pode conter argumentos opcionais com mais informações sobre o evento. O número e os tipos de argumentos para um determinado evento são considerados uma API pública e estável e não devem ser modificados entre as versões.

Por exemplo, um evento de auditoria é denominado `os.chdir`. Este evento tem um argumento chamado *path* que conterá o novo diretório de trabalho solicitado.

sys.audit() chamará os ganchos de auditoria existentes, passando o nome do evento e os argumentos, e levantará novamente a primeira exceção de qualquer gancho. Em geral, se uma exceção for levantada, ela não deve ser tratada e o processo deve ser encerrado o mais rápido possível. Isso permite que as implementações de gancho decidam como responder a eventos específicos: elas podem simplesmente registrar o evento ou abortar a operação levantando uma exceção.

Ganchos são adicionados usando as funções *sys.addaudithook()* ou `PySys_AddAuditHook()`.

O equivalente nativo desta função é `PySys_Audit()`. Usar a função nativa é preferível quando possível.

Veja *tabela de eventos de auditoria* para todos os eventos levantados pelo CPython.

Adicionado na versão 3.8.

`sys.`**`base_exec_prefix`**

Equivalente a *exec_prefix*, mas referindo-se à instalação base do Python.

Ao executar em *Ambientes virtuais*, *exec_prefix* é substituído pelo prefixo do ambiente virtual. *base_exec_prefix*, por outro lado, não muda e sempre aponta para a instalação base do Python. Consulte *Ambientes virtuais* para obter mais informações.

Adicionado na versão 3.3.

sys.**base_prefix**

Equivalente a *prefix*, mas referindo-se à instalação base do Python.

Ao executar em *ambiente virtual*, *prefix* é substituído pelo prefixo do ambiente virtual. *base_prefix*, por outro lado, não muda e sempre aponta para a instalação base do Python. Consulte *Ambientes virtuais* para obter mais informações.

Adicionado na versão 3.3.

sys.**byteorder**

Um indicador da ordem nativa de bytes. Isso terá o valor `'big'` em plataformas big endian (byte mais significativo primeiro) e `'little'` em plataformas little endian (byte menos significativo primeiro).

sys.**builtin_module_names**

Uma tupla de strings contendo os nomes de todos os módulos compilados neste interpretador Python. (Esta informação não está disponível de nenhuma outra forma — `modules.keys()` apenas lista os módulos importados.)

Veja também a lista de *sys.stdlib_module_names*.

sys.**call_tracing**(*func*, *args*)

Chama `func(*args)`, enquanto o rastreamento está habilitado. O estado de rastreamento é salvo e restaurado posteriormente. Isso deve ser chamado de um depurador de um ponto de verificação, para depurar ou analisar o desempenho recursivamente algum outro código.

O rastreamento é suspenso ao chamar uma função de rastreamento definida por *settrace()* ou *setprofile()* para evitar recursão infinita. `call_tracing()` habilita a recursão explícita da função de rastreamento.

sys.**copyright**

Uma string contendo os direitos autorais pertencentes ao interpretador Python.

sys.**_clear_type_cache**()

Limpa o cache de tipo interno. O cache de tipo é usado para acelerar pesquisas de atributos e métodos. Use a função *apenas* para descartar referências desnecessárias durante a depuração de vazamento de referência.

Esta função deve ser usada apenas para fins internos e especializados.

Descontinuado desde a versão 3.13: Em vez disso, use a função mais geral *_clear_internal_caches()*.

sys.**_clear_internal_caches**()

Limpa todos os caches internos relacionados ao desempenho. Use esta função *somente* para liberar referências e blocos de memória desnecessários ao procurar por vazamentos.

Adicionado na versão 3.13.

sys.**_current_frames**()

Retorna um dicionário mapeando o identificador de cada encadeamento (*thread*) para o quadro de pilha mais alto atualmente ativo nesse encadeamento no momento em que a função é chamada. Observe que as funções no módulo *traceback* podem construir a pilha de chamadas dado tal quadro.

Isso é mais útil para fazer a depuração de impasses: esta função não requer a cooperação das threads em impasse e as pilhas de chamadas de tais threads são congeladas enquanto permanecerem em impasse (*deadlock*). O quadro retornado para uma thread sem impasse pode não ter nenhuma relação com a atividade atual da thread no momento em que o código de chamada examina o quadro.

Esta função deve ser usada apenas para fins internos e especializados.

Levanta um *evento de auditoria* `sys._current_frames` sem argumentos.

sys.**_current_exceptions**()

Retorna um dicionário mapeando o identificador de cada encadeamento (*thread*) para a exceção mais alta atualmente ativo nesse encadeamento no momento em que a função é chamada. Se um encadeamento não estiver manipulando uma exceção no momento, ele não será incluído no dicionário de resultados.

Isso é mais útil para criação de perfil estatístico.

Esta função deve ser usada apenas para fins internos e especializados.

Levanta um *evento de auditoria* `sys._current_exceptions` sem argumentos.

Alterado na versão 3.12: Cada valor no dicionário agora é uma única instância de exceção, em vez de uma tupla de 3 elementos, como retornado de `sys.exc_info()`.

`sys.`**`breakpointhook`**`()`

Esta função de gancho é chamada pela função embutida *`breakpoint()`*. Por padrão, ela leva você ao depurador *`pdb`*, mas pode ser configurado para qualquer outra função para que você possa escolher qual depurador será usado.

A assinatura dessa função depende do que ela chama. Por exemplo, a ligação padrão (por exemplo, `pdb.set_trace()`) não espera nenhum argumento, mas você pode vinculá-la a uma função que espera argumentos adicionais (posicionais e/ou nomeados). A função embutida `breakpoint()` passa seus `*args` e `**kws` diretamente. O que quer que `breakpointhooks()` retorne é retornado de `breakpoint()`.

A implementação padrão primeiro consulta a variável de ambiente `PYTHONBREAKPOINT`. Se for definido como `"0"`, então esta função retorna imediatamente; ou seja, é um no-op. Se a variável de ambiente não for definida, ou for definida como uma string vazia, `pdb.set_trace()` será chamado. Caso contrário, essa variável deve nomear uma função a ser executada, usando a nomenclatura de importação pontilhada do Python, por exemplo `package.subpackage.module.function`. Neste caso, `package.subpackage.module` seria importado e o módulo resultante deve ter um chamável chamado `function()`. Isso é executado, passando `*args` e `**kws`, e qualquer que seja o retorno de `function()`, `sys.breakpointhook()` retorna para a função embutida *`breakpoint()`*.

Observe que se algo der errado ao importar o chamável nomeado por `PYTHONBREAKPOINT`, uma *`RuntimeWarning`* é relatado e o ponto de interrupção é ignorado.

Observe também que se `sys.breakpointhook()` for sobrescrito programaticamente, `PYTHONBREAKPOINT` *não* será consultado.

Adicionado na versão 3.7.

`sys.`**`_debugmallocstats`**`()`

Imprima informações de baixo nível para stderr sobre o estado do alocador de memória do CPython.

Se o Python for compilado no modo de depuração (`opção --with-pydebug do configure`), ele também executa algumas verificações de consistência interna custosas.

Adicionado na versão 3.3.

Esta função é específica para CPython. O formato de saída exato não é definido aqui e pode mudar.

`sys.`**`dllhandle`**

Número inteiro que especifica o identificador da DLL do Python.

Disponibilidade: Windows.

`sys.`**`displayhook`**`(`*`value`*`)`

Se *value* não for `None`, esta função imprime `repr(value)` em `sys.stdout`, e salva *value* em `builtins._`. Se `repr(value)` não for codificável para `sys.stdout.encoding` com o tratador de erros `sys.stdout.errors` (que provavelmente é `'strict'`), codifica-o para `sys.stdout.encoding` com tratador de erros `'backslashreplace'`.

`sys.displayhook` é chamado no resultado da avaliação de uma *expressão* inserida em uma sessão interativa do Python. A exibição desses valores pode ser personalizada atribuindo outra função de um argumento a `sys.displayhook`.

Pseudocódigo:

```
def displayhook(value):
    if value is None:
        return
    # Define '_' como None para evitar recursão
```

(continua na próxima página)

(continuação da página anterior)

```
    builtins._ = None
    text = repr(value)
    try:
        sys.stdout.write(text)
    except UnicodeEncodeError:
        bytes = text.encode(sys.stdout.encoding, 'backslashreplace')
        if hasattr(sys.stdout, 'buffer'):
            sys.stdout.buffer.write(bytes)
        else:
            text = bytes.decode(sys.stdout.encoding, 'strict')
            sys.stdout.write(text)
    sys.stdout.write("\n")
    builtins._ = value
```

Alterado na versão 3.2: Usa o tratador de erros `'backslashreplace'` ao ser levantada *UnicodeEncodeError*.

sys.**dont_write_bytecode**

Se isso for true, o Python não tentará escrever arquivos `.pyc` na importação de módulos fonte. Este valor é inicialmente definido como `True` ou `False` dependendo da opção de linha de comando `-B` e da variável de ambiente `PYTHONDONTWRITEBYTECODE`, mas você mesmo pode configurá-lo para controlar geração de arquivo bytecode.

sys.**_emscripten_info**

Um *tupla nomeada* contendo informações sobre o ambiente na plataforma *wasm32-emscripten*. A tupla nomeada é provisória e pode mudar no futuro.

_emscripten_info.**emscripten_version**

Versão emscripten como tupla de ints (maior, menor, micro), por exemplo `(3, 1, 8)`.

_emscripten_info.**runtime**

String em tempo de execução; por exemplo, user-agent do navegador, `'Node.js v14.18.2'`, ou `'UNKNOWN'`.

_emscripten_info.**pthreads**

`True` se o Python for compilado com suporte a pthreads do Emscripten.

_emscripten_info.**shared_memory**

`True` se o Python for compilado com suporte a memória compartilhada.

Disponibilidade: Emscripten.

Adicionado na versão 3.11.

sys.**pycache_prefix**

Se for definido (não `None`), o Python escreverá arquivos de cache de bytecode `.pyc` para (e os lerá de) uma árvore de diretórios paralela com raiz neste diretório, em vez de diretórios `__pycache__` na árvore de código-fonte. Quaisquer diretórios `__pycache__` na árvore de código-fonte serão ignorados e novos arquivos `.pyc` gravados dentro do prefixo pycache. Portanto, se você usar *compileall* como uma etapa de pré-compilação, certifique-se de executá-lo com o mesmo prefixo pycache (se houver) que usará em tempo de execução.

Um caminho relativo é interpretado em relação ao diretório de trabalho atual.

Este valor é definido inicialmente com base no valor da opção de linha de comando `-X pycache_prefix=PATH` ou na variável de ambiente `PYTHONPYCACHEPREFIX` (a linha de comando tem precedência). Se nenhum estiver definido, é `None`.

Adicionado na versão 3.8.

sys.**excepthook**(*type*, *value*, *traceback*)

Esta função imprime um determinado traceback (situação da pilha de execução) e exceção para `sys.stderr`.

Quando uma exceção além da *SystemExit* é levantada e não capturada, o interpretador chama `sys.excepthook` com três argumentos, a classe de exceção, a instância de exceção e um objeto traceback. Em uma sessão interativa, isso acontece logo antes de o controle retornar ao prompt; em um programa Python, isso acontece pouco antes de o programa ser encerrado. A manipulação de tais exceções de nível superior pode ser personalizada atribuindo outra função de três argumentos a `sys.excepthook`.

Levanta um evento de auditoria `sys.excepthook` com os argumentos `hook`, `type`, `value`, `traceback` quando ocorre uma exceção não capturada. Se nenhum gancho foi definido, `hook` pode ser `None`. Se algum gancho gerar uma exceção derivada de *RuntimeError*, a chamada para o gancho será suprimida. Caso contrário, a exceção de gancho de auditoria será relatada como não aumentável e `sys.excepthook` será chamado.

Ver também

A função *sys.unraisablehook()* lida com exceções que não podem ser levantadas e a função *threading.excepthook()* trata exceções levantadas por *threading.Thread.run()*.

`sys.`**`__breakpointhook__`**

`sys.`**`__displayhook__`**

`sys.`**`__excepthook__`**

`sys.`**`__unraisablehook__`**

Esses objetos contêm os valores originais de `breakpointhook`, `displayhook`, `excepthook` e `unraisablehook` no início do programa. Eles são salvos para que `breakpointhook`, `displayhook` e `excepthook`, `unraisablehook` possam ser restaurados caso sejam substituídos por objetos quebrados ou alternativos.

Adicionado na versão 3.7: __breakpointhook__

Adicionado na versão 3.8: __unraisablehook__

`sys.`**`exception`**`()`

Esta função, quando chamada enquanto um tratador de exceção está em execução (como uma cláusula `except` ou `except*`), retorna a instância da exceção que foi capturada por este tratador. Quando os manipuladores de exceção estão aninhados um dentro do outro, apenas a exceção tratada pelo tratador mais interno é acessível.

Se nenhum tratador de exceções estiver em execução, esta função retornará `None`.

Adicionado na versão 3.11.

`sys.`**`exc_info`**`()`

Essa função retorna a representação de estilo antigo da exceção tratada. Se uma exceção `e` é tratada atualmente (de forma que *exception()* retornaria `e`), *exc_info()* retorna a tupla `(type(e), e, e.__traceback__)`. Ou seja, uma tupla contendo o tipo da exceção (uma subclasse de *BaseException*), a própria exceção e um objeto de traceback que normalmente encapsula a pilha de chamadas no ponto em que a última exceção ocorreu.

Se nenhuma exceção estiver sendo tratada em qualquer lugar da pilha, esta função retornará uma tupla contendo três valores `None`.

Alterado na versão 3.11: Os campos `type` e `traceback` agora são derivados do `value` (a instância da exceção), portanto, quando uma exceção é modificada enquanto está sendo tratada, as alterações são refletidas nos resultados das subsequentes chamadas para *exc_info()*.

`sys.`**`exec_prefix`**

Uma string que fornece o prefixo do diretório específico do site onde os arquivos Python dependentes da plataforma são instalados; por padrão, também é `'/usr/local'`. Isso pode ser definido em tempo de compilação com o argumento `--exec-prefix` para o script **`configure`**. Especificamente, todos os arquivos de configuração (por exemplo, o arquivo de cabeçalho `pyconfig.h`) são instalados no diretório *exec_prefix*`/lib/python`*X.Y*`/config`, e os módulos da biblioteca compartilhada são instalados em *exec_prefix*`/lib/python`*X.Y*`/lib-dynload`, onde *X.Y* é o número da versão do Python, por exemplo `3.2`.

> **Nota**
>
> Se um *ambiente virtual* estiver em vigor, este `exec_prefix` vai apontar para o ambiente virtual. O valor para a instalação do Python ainda estará disponível, via `base_exec_prefix`. Confira *Ambientes virtuais* para mais informações.

Alterado na versão 3.14: Ao executar em um *ambiente virtual*, `prefix` e `exec_prefix` agora são definidos como o prefixo do ambiente virtual pela *inicialização do caminho*, em vez de `site`. Isso significa que `prefix` e `exec_prefix` sempre apontam para o ambiente virtual, mesmo quando `site` está desabilitado (`-S`).

`sys.`**`executable`**

Uma string que fornece o caminho absoluto do binário executável para o interpretador Python, em sistemas onde isso faz sentido. Se o Python não conseguir recuperar o caminho real para seu executável, `sys.executable` será uma string vazia ou `None`.

`sys.`**`exit`**`(`[*arg*]`)`

Levanta uma exceção `SystemExit`, sinalizando a intenção de sair do interpretador.

O argumento opcional *arg* pode ser um número inteiro dando o status de saída (o padrão é zero) ou outro tipo de objeto. Se for um número inteiro, zero é considerado "terminação bem-sucedida" e qualquer valor diferente de zero é considerado "terminação anormal" por shells e similares. A maioria dos sistemas exige que ele esteja no intervalo de 0 a 127 e, caso contrário, produz resultados indefinidos. Alguns sistemas têm uma convenção para atribuir significados específicos a códigos de saída específicos, mas estes são geralmente pouco desenvolvidos; programas Unix geralmente usam 2 para erros de sintaxe de linha de comando e 1 para todos os outros tipos de erros. Se outro tipo de objeto for passado, `None` é equivalente a passar zero, e qualquer outro objeto é impresso em `stderr` e resulta em um código de saída de 1. Em particular, `sys.exit( "alguma mensagem de erro")` é uma maneira rápida de sair de um programa quando ocorre um erro.

Uma vez que `exit()` em última análise, "apenas" gera uma exceção, ele só sairá do processo quando chamado da thread principal e a exceção não é interceptada. As ações de limpeza especificadas pelas cláusulas finally de instruções `try` são honradas e é possível interceptar a tentativa de saída em um nível externo.

Alterado na versão 3.6: Se ocorrer um erro na limpeza após o interpretador Python ter capturado `SystemExit` (como um erro ao liberar dados em buffer nos fluxos padrão), o status de saída é alterado para 120.

`sys.`**`flags`**

A *tupla nomeada* *flags* expõe o status dos sinalizadores de linha de comando. Os sinalizadores devem ser acessados somente pelo nome e não pelo índice. Os atributos são somente leitura.

flags.**debug**	`-d`
flags.**inspect**	`-i`
flags.**interactive**	`-i`
flags.**isolated**	`-I`
flags.**optimize**	`-O` ou `-OO`
flags.**dont_write_bytecode**	`-B`
flags.**no_user_site**	`-s`
flags.**no_site**	`-S`
flags.**ignore_environment**	`-E`
flags.**verbose**	`-v`
flags.**bytes_warning**	`-b`
flags.**quiet**	`-q`
flags.**hash_randomization**	`-R`
flags.**dev_mode**	`-X dev` (*Modo de Desenvolvimento do Python*)
flags.**utf8_mode**	`-X utf8`
flags.**safe_path**	`-P`
flags.**int_max_str_digits**	`-X int_max_str_digits` (*limitação de comprimento de string na conversão para inteiro*)
flags.**warn_default_encoding**	`-X warn_default_encoding`
flags.**gil**	`-X gil` e `PYTHON_GIL`
flags.**thread_inherit_context**	`-X thread_inherit_context` e `PYTHON_THREAD_INHERIT_CONTEXT`
flags.**context_aware_warnings**	`-X context_aware_warnings` e `PYTHON_CONTEXT_AWARE_WARNINGS`

Alterado na versão 3.2: Adicionado o atributo `quiet` para o novo sinalizador `-q`.

Adicionado na versão 3.2.3: O atributo `hash_randomization`.

Alterado na versão 3.3: Removido o atributo obsoleto `division_warning`.

Alterado na versão 3.4: Adicionado o atributo `isolated` para o sinalizador `-I isolated`.

Alterado na versão 3.7: Adicionado o atributo `dev_mode` para o novo *Modo de Desenvolvimento do Python* e o atributo `utf8_mode` para o novo sinalizador `-X utf8`.

Alterado na versão 3.10: Adicionado o atributo `warn_default_encoding` para o sinalizador `-X warn_default_encoding`.

Alterado na versão 3.11: Adicionado o atributo `safe_path` para a opção `-P`.

Alterado na versão 3.11: Adicionado o atributo `int_max_str_digits`.

Alterado na versão 3.13: Adicionado o atributo `gil`.

Alterado na versão 3.14: Adicionado o atributo `thread_inherit_context`.

Alterado na versão 3.14: Adicionado o atributo `context_aware_warnings`.

`sys.`**`float_info`**

Uma *tupla nomeada* contendo informações sobre o tipo float, ponto flutuante. Ele contém informações de baixo nível sobre a precisão e a representação interna. Os valores correspondem às várias constantes de ponto flutuante definidas no arquivo de cabeçalho padrão `float.h` para a linguagem de programação 'C'; consulte a seção 5.2.4.2.2 do padrão ISO/IEC C de 1999 [C99], 'Características dos tipos flutuantes', para obter detalhes.

Tabela1: Atributos da tupla nomeada `float_info`

atributo	macro em float.h	explicação
`float_info.`**`epsilon`**	`DBL_EPSILON`	diferença entre 1,0 e o menor valor maior que 1,0 que pode ser representado como ponto flutuante. Veja também `math.ulp.()`
`float_info.`**`dig`**	`DBL_DIG`	O número máximo de dígitos decimais que podem ser fielmente representados em um ponto flutuante; veja abaixo.
`float_info.`**`mant_dig`**	`DBL_MANT_DIG`	Precisão do ponto flutuante: o número de dígitos de base `radix` no significando de um ponto flutuante.
`float_info.`**`max`**	`DBL_MAX`	O ponto flutuante finito positivo máximo representável.
`float_info.`**`max_exp`**	`DBL_MAX_EXP`	O inteiro máximo *e* de tal modo que `radix**(e-1)` é um ponto flutuante finito representável.
`float_info.`**`max_10_exp`**	`DBL_MAX_10_EXP`	O inteiro máximo *e* de tal modo que `10**e` é um intervalo de pontos flutuantes finitos representáveis.
`float_info.`**`min`**	`DBL_MIN`	O ponto flutuante *normalizado* positivo mínimo representável. Use *`math.ulp(0.0)`* para obter o menor ponto flutuante representável positivo *desnormalizado*.
`float_info.`**`min_exp`**	`DBL_MIN_EXP`	O inteiro mínimo *e* de tal modo que `radix**(e-1)` é um ponto flutuante normalizado.
`float_info.`**`min_10_exp`**	`DBL_MIN_10_EXP`	O inteiro mínimo *e* de tal modo que `10**e` é um ponto flutuante normalizado.
`float_info.`**`radix`**	`FLT_RADIX`	A raiz da representação do expoente.
`float_info.`**`rounds`**	`FLT_ROUNDS`	Um inteiro representando o modo de arredondamento para aritmética de ponto flutuante. Isso reflete o valor da macro do sistema `FLT_ROUNDS` no momento da inicialização do interpretador: • `-1`: indeterminável • `0`: em direção a zero • `1`: para o mais próximo • `2`: em direção ao infinito positivo • `3`: em direção ao infinito negativo Todos os outros valores para `FLT_ROUNDS` caracterizam o comportamento de arredondamento definido pela implementação.

O atributo *sys.float_info.dig* precisa de mais explicações. Se `s` for qualquer string representando um número decimal com no máximo `sys.float_info.dig` dígitos significativos, então converter `s` para ponto flutuante e vice-versa recuperará uma string representando o mesmo decimal valor:

```
>>> import sys
>>> sys.float_info.dig
15
>>> s = '3.14159265358979'    # string decimal com 15 dígitos significativos
>>> format(float(s), '.15g')  # converter para float e desconverter retorna␣
↪mesmo valor
'3.14159265358979'
```

Mas para strings com mais de *sys.float_info.dig* dígitos significativos, isso nem sempre é verdade:

```
>>> s = '9876543211234567'    # 16 dígitos significativos é muito!
>>> format(float(s), '.16g')  # a conversão muda o valor
'9876543211234568'
```

`sys.`**`float_repr_style`**

Uma string indicando como a função *repr()* se comporta para floats. Se a string tem valor `'short'` então para um ponto flutuante finito `x`, `repr(x)` visa produzir uma string curta com a propriedade que `float(repr(x)) == x`. Este é o comportamento usual no Python 3.1 e posterior. Caso contrário, `float_repr_style` tem valor `'legacy'` e `repr(x)` se comporta da mesma forma que nas versões do Python anteriores a 3.1.

Adicionado na versão 3.1.

`sys.`**`getallocatedblocks`**`()`

Retorna a quantidade de blocos de memória alocados atualmente pelo interpretador, independentemente do tamanho. Esta função é útil principalmente ao acompanhar e depurar vazamentos de memória. Devido aos caches internos do interpretador, o resultado pode variar de chamada a chamada; você pode ter que chamar *_clear_internal_caches()* e *gc.collect()* para obter resultados mais previsíveis.

Se uma construção ou implementação do Python não puder computar razoavelmente essas informações, *getallocatedblocks()* poderá retornar 0 em seu lugar.

Adicionado na versão 3.4.

`sys.`**`getunicodeinternedsize`**`()`

Retorna o número de objetos unicode que foram internalizados.

Adicionado na versão 3.12.

`sys.`**`getandroidapilevel`**`()`

Retorna o nível de API de tempo de construção do Android como um inteiro. Isso representa a versão mínima do Android em que esta construção do Python pode ser executada. Para obter informações sobre a versão de tempo de execução, consulte *platform.android_ver()*.

Disponibilidade: Android.

Adicionado na versão 3.7.

`sys.`**`getdefaultencoding`**`()`

Retorna `'utf-8'`. Este é o nome da codificação de string padrão, usada em métodos como *str.encode()*.

`sys.`**`getdlopenflags`**`()`

Retorna o valor atual dos sinalizadores que são usados para chamadas `dlopen()`. Nomes simbólicos para os valores dos sinalizadores podem ser encontrados no módulo *os* (constantes `RTLD_`*xxx*, por exemplo *os.RTLD_LAZY*).

Disponibilidade: Unix.

sys.**getfilesystemencoding**()

Obtém o *codificação do sistema de arquivos*: a codificação usada com o *manipulador de erros do sistema de arquivos* para converter entre nomes de arquivos Unicode e nomes de arquivos de bytes. O manipulador de erros do sistema de arquivos é retornado de `getfilesystemencodeerrors()`.

Para melhor compatibilidade, str deve ser usado para nomes de arquivos em todos os casos, embora a representação de nomes de arquivos como bytes também seja suportada. As funções que aceitam ou retornam nomes de arquivo devem suportar str ou bytes e converter internamente para a representação preferencial do sistema.

`os.fsencode()` e `os.fsdecode()` devem ser usados para garantir que a codificação correta e o modo de erros sejam usados.

O *tratador de erros e codificação do sistema de arquivos* são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Alterado na versão 3.2: O resultado de `getfilesystemencoding()` não pode mais ser `None`.

Alterado na versão 3.6: O Windows não tem mais garantia de retornar `'mbcs'`. Veja **PEP 529** e `_enablelegacywindowsfsencoding()` para mais informações.

Alterado na versão 3.7: Retorna `'utf-8'` se o *Modo UTF-8 do Python* estiver ativado.

sys.**getfilesystemencodeerrors**()

Obtém o *manipulador de erros do sistema de arquivos*: o manipulador de erros usado com a *codificação do sistema de arquivos* para converter entre nomes de arquivos Unicode e nomes de arquivos de bytes. A codificação do sistema de arquivos é retornado de `getfilesystemencoding()`.

`os.fsencode()` e `os.fsdecode()` devem ser usados para garantir que a codificação correta e o modo de erros sejam usados.

O *tratador de erros e codificação do sistema de arquivos* são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Adicionado na versão 3.6.

sys.**get_int_max_str_digits**()

Retorna o valor atual para a *limitação de comprimento de string na conversão para inteiro*. Veja também `set_int_max_str_digits()`.

Adicionado na versão 3.11.

sys.**getrefcount**(*object*)

Retorna a contagem de referências do *object*. A contagem retornada é geralmente um valor maior do que o esperado, porque inclui a referência (temporária) passada como argumento para `getrefcount()`.

Observe que o valor retornado pode não refletir realmente quantas referências ao objeto são realmente mantidas. Por exemplo, alguns objetos são *imortais* e têm uma contagem de referências muito alta, que não reflete o número real de referências. Consequentemente, não confie no valor retornado para ser preciso, exceto um valor de 0 ou 1.

Objetos *imortais* com uma grande contagem de referências podem ser identificados via `_is_immortal()`.

Alterado na versão 3.12: Objetos imortais têm contagens de referência muito grandes que não correspondem ao número real de referências ao objeto.

sys.**getrecursionlimit**()

Retorna o valor atual do limite de recursão, a profundidade máxima da pilha do interpretador Python. Esse limite evita que a recursão infinita cause um estouro da pilha C e falhe o Python. Pode ser definido por `setrecursionlimit()`.

sys.**getsizeof**(*object*[, *default*])

Retorna o tamanho de um objeto em bytes. O objeto pode ser qualquer tipo de objeto. Todos os objetos embutidos retornarão resultados corretos, mas isso não precisa ser verdadeiro para extensões de terceiros, pois é específico da implementação.

Apenas o consumo de memória diretamente atribuído ao objeto é contabilizado, não o consumo de memória dos objetos a que ele se refere.

Se fornecido, *default* será retornado se o objeto não fornecer meios para recuperar o tamanho. Caso contrário, uma exceção *TypeError* será levantada.

getsizeof() chama o método __sizeof__ do objeto e adiciona uma sobrecarga adicional do coletor de lixo se o objeto for gerenciado pelo coletor de lixo.

Consulte receita de tamanho recursivo para obter um exemplo de uso de *getsizeof()* recursivamente para encontrar o tamanho dos contêineres e todo o seu conteúdo.

sys.**getswitchinterval**()

Retorna o "intervalo de troca de thread" do interpretador em segundos; veja *setswitchinterval()*.

Adicionado na versão 3.2.

sys.**_getframe**([*depth*])

Retorna um objeto quadro da pilha de chamadas. Se o inteiro opcional *depth* for fornecido, retorna o objeto de quadro que muitos chamam abaixo do topo da pilha. Se for mais profundo do que a pilha de chamadas, *ValueError* é levantada. O padrão para *depth* é zero, retornando o quadro no topo da pilha de chamadas.

Levanta um *evento de auditoria* `sys._getframe` com o argumento `frame`.

Esta função deve ser usada apenas para fins internos e especializados. Não é garantido que exista em todas as implementações do Python.

sys.**_getframemodulename**([*depth*])

Retorna o nome de um módulo da pilha de chamadas. Se o inteiro opcional *depth* for fornecido, retorna o módulo que muitas chamadas abaixo do topo da pilha. Se for mais profundo do que a pilha de chamadas, ou se o módulo não for identificável, `None` é retornado. O padrão para *depth* é zero, retornando o módulo no topo da pilha de chamadas.

Levanta um *evento de auditoria* `sys._getframemodulename` com o argumento `depth`.

Esta função deve ser usada apenas para fins internos e especializados. Não é garantido que exista em todas as implementações do Python.

Adicionado na versão 3.12.

sys.**getobjects**(*limit*[, *type*])

Esta função só existe se o CPython foi construído usando a opção de configuração especializada `--with-trace-refs`. Ela é destinada apenas para depurar problemas de coleta de lixo.

Retorna uma lista de até *limit* objetos Python alocados dinamicamente. Se *type* for fornecido, somente objetos daquele tipo exato (não subtipos) serão incluídos.

Objetos da lista não são seguros para uso. Especificamente, o resultado incluirá objetos de todos os interpretadores que compartilham seu estado de alocador de objetos (ou seja, aqueles criados com `PyInterpreterConfig.use_main_obmalloc` definido como 1 ou usando `Py_NewInterpreter()` e o interpretador principal). Misturar objetos de diferentes iinterpretadores pode levar a travamentos ou outros comportamentos inesperados.

Esta função deve ser usada apenas para fins especializados. Não é garantido que exista em todas as implementações do Python.

Alterado na versão 3.14: O resultado pode incluir objetos de outros interpretadores.

sys.**getprofile**()

Obtém a função do criador de perfil conforme definido por *setprofile()*.

sys.**gettrace**()

Obtém a função trace conforme definido por *settrace()*.

A função *gettrace()* destina-se apenas à implementação de depuradores, perfiladores, ferramentas de cobertura e similares. Seu comportamento faz parte da plataforma de implementação, e não da definição da linguagem e, portanto, pode não estar disponível em todas as implementações do Python.

sys.**getwindowsversion**()

Retorna uma tupla nomeada que descreve a versão do Windows em execução no momento. Os elementos nomeados são *major*, *minor*, *build*, *platform*, *service_pack*, *service_pack_minor*, *service_pack_major*, *suite_mask*, *product_type* e *platform_version*. *service_pack* contém uma string, *platform_version* uma tupla de 3 elementos e todos os outros valores são inteiros. Os componentes também podem ser acessados pelo nome, então `sys.getwindowsversion()[0]` é equivalente a `sys.getwindowsversion().major`. Para compatibilidade com versões anteriores, apenas os primeiros 5 elementos são recuperáveis por indexação.

platform será `2` (VER_PLATFORM_WIN32_NT).

product_type pode ser um dos seguintes valores:

Constante	Significado
`1` (VER_NT_WORKSTATION)	O sistema é uma estação de trabalho
`2` (VER_NT_DOMAIN_CONTROLLER)	O sistema é um controlador de domínio.
`3` (VER_NT_SERVER)	O sistema é um servidor, mas não um controlador de domínio.

Esta função atua como um envólucro em volta da função Win32 `GetVersionEx()`; consulte a documentação da Microsoft em `OSVERSIONINFOEX()` para obter mais informações sobre esses campos.

platform_version retorna a versão principal, a versão secundária e o número de compilação do sistema operacional atual, em vez da versão que está sendo emulada para o processo. Destina-se a ser usado no registro, e não na detecção de recursos.

Nota

platform_version deriva a versão de kernel32.dll que pode ser de uma versão diferente da versão do sistema operacional. Use o módulo `platform` para obter a versão precisa do sistema operacional.

Disponibilidade: Windows.

Alterado na versão 3.2: Alterado para uma tupla nomeada e adicionado *service_pack_minor*, *service_pack_major*, *suite_mask* e *product_type*.

Alterado na versão 3.6: Adicionado *platform_version*

sys.**get_asyncgen_hooks**()

Retorna um objeto *asyncgen_hooks*, que é semelhante a um `namedtuple` no formato `(firstiter, finalizer)`, onde *firstiter* e *finalizer* devem ser `None` ou funções que recebem um *iterador gerador assíncrono* como argumento e são usadas para agendar a finalização de um gerador assíncrono por um laço de eventos.

Adicionado na versão 3.6: Veja **PEP 525** para mais detalhes.

Nota

Esta função foi adicionada provisoriamente (veja **PEP 411** para detalhes.)

sys.**get_coroutine_origin_tracking_depth**()

Obtém a profundidade de rastreamento da origem da corrotina atual, conforme definido por `set_coroutine_origin_tracking_depth()`.

Adicionado na versão 3.7.

Nota

Esta função foi adicionada provisoriamente (veja **PEP 411** para detalhes.) Use-a apenas para propósitos de depuração.

sys.**hash_info**

Uma *tupla nomeada* fornecendo parâmetros da implementação de hash numérico. Para mais detalhes sobre hashing de tipos numéricos, veja *Hashing de tipos numéricos*.

hash_info.**width**

A largura em bits usada para fazer hash de valores

hash_info.**modulus**

O módulo primo P usado para esquema de hash numérico

hash_info.**inf**

O valor de hash retornado para um infinito positivo

hash_info.**nan**

(Este atributo não é mais usado)

hash_info.**imag**

O multiplicador usado para a parte imaginária de um número complexo

hash_info.**algorithm**

O nome do algoritmo para hash de str, bytes e memoryview

hash_info.**hash_bits**

O tamanho da saída interna do algoritmo de hash

hash_info.**seed_bits**

O tamanho da chave semente do algoritmo hash

Adicionado na versão 3.2.

Alterado na versão 3.4: Adicionado *algorithm*, *hash_bits* e *seed_bits*

sys.**hexversion**

O número da versão codificado como um único inteiro. Isso é garantido para aumentar com cada versão, incluindo suporte adequado para lançamentos de não produção. Por exemplo, para testar se o interpretador Python é pelo menos a versão 1.5.2, use:

```
if sys.hexversion >= 0x010502F0:
    # usa algum recurso avançado
    ...
else:
    # usa uma implementação alternativa ou adverte o usuário
    ...
```

Isso é chamado `hexversion` já que só parece realmente significativo quando visto como o resultado de passá-lo para a função embutida `hex()`. A *tupla nomeada* `sys.version_info` pode ser usada para uma codificação mais amigável das mesmas informações.

Mais detalhes sobre `hexversion` podem ser encontrados em apiabiversion.

sys.**implementation**

Um objeto que contém informações sobre a implementação do interpretador Python em execução no momento. Os atributos a seguir devem existir em todas as implementações do Python.

name é o identificador da implementação, por exemplo `'cpython'`. A string real é definida pela implementação do Python, mas é garantido que seja minúscula.

version é uma tupla nomeada, no mesmo formato que `sys.version_info`. Ela representa a versão da implementação Python em *implementation*. Isso tem um significado distinto da versão específica da *linguagem*

Python à qual o interpretador em execução no momento está em conformidade, que `sys.version_info` representa. Por exemplo, para PyPy 1.8 `sys.implementation.version` pode ser `sys.version_info(1, 8, 0, 'final', 0)`, enquanto `sys.version_info` seria `sys.version_info(2, 7, 2, 'final', 0)`. Para CPython, eles são o mesmo valor, pois é a implementação de referência.

hexversion é a versão de implementação em formato hexadecimal, como *sys.hexversion*.

cache_tag é a tag usada pelo mecanismo de importação nos nomes de arquivo dos módulos em cache. Por convenção, seria um composto do nome e da versão da implementação, como `'cpython-33'`. No entanto, uma implementação Python pode usar algum outro valor, se apropriado. Se `cache_tag` for definido como `None`, isso indica que o cache do módulo deve ser desabilitado.

supports_isolated_interpreters é um valor booleano que indica se esta implementação oferece suporte a múltiplos interpretadores isolados. É `True` para CPython na maioria das plataformas. Plataformas com este suporte implementam o módulo de baixo nível `_interpreters`.

Ver também

PEP 684, **PEP 734** e *concurrent.interpreters*.

sys.implementation pode conter atributos adicionais específicos para a implementação do Python. Esses atributos não padrão devem começar com um sublinhado e não são descritos aqui. Independentemente do seu conteúdo, *sys.implementation* não mudará durante uma execução do interpretador, nem entre versões de implementação. (No entanto, ele pode mudar entre versões da linguagem Python.) Veja a **PEP 421** para mais informações.

Adicionado na versão 3.3.

Alterado na versão 3.14: Adicionado o campo `supports_isolated_interpreters`.

Nota

A adição de novos atributos obrigatórios deve passar pelo processo normal de PEPs. Veja a **PEP 421** para mais informações.

`sys.`**`int_info`**

Uma *tupla nomeada* que contém informações sobre a representação interna de inteiros do Python. Os atributos são somente leitura.

`int_info.`**`bits_per_digit`**

O número de bits mantidos em cada dígito. Os inteiros do Python são armazenados internamente na base `2**int_info.bits_per_digit`.

`int_info.`**`sizeof_digit`**

O tamanho em bytes do tipo C usado para representar um dígito.

`int_info.`**`default_max_str_digits`**

O valor padrão para *sys.get_int_max_str_digits()* quando não estiver explicitamente configurado.

`int_info.`**`str_digits_check_threshold`**

O valor mínimo diferente de zero para *sys.set_int_max_str_digits()*, `PYTHONINTMAXSTRDIGITS` ou `-X int_max_str_digits`.

Adicionado na versão 3.1.

Alterado na versão 3.11: Adicionados *default_max_str_digits* e *str_digits_check_threshold*.

`sys.`**`__interactivehook__`**

Quando esse atributo existe, seu valor é chamado automaticamente (sem argumentos) quando o interpretador é

iniciado em modo interativo. Isso é feito após o arquivo `PYTHONSTARTUP` ser lido, para que você possa definir esse gancho lá. O módulo `site` *define isso*.

Levanta um *evento de auditoria* `cpython.run_interactivehook` com o objeto gancho como o argumento quando o gancho é chamado na inicialização.

Adicionado na versão 3.4.

`sys.`**`intern`**`(`*string*`)`

Insere *string* na tabela de strings "internalizadas" e retorna a string internalizada – que é a própria *string* ou uma cópia. Strings internalizadas são úteis para obter um pouco de desempenho na pesquisa de dicionário – se as chaves em um dicionário foram internalizadas, e a chave de pesquisa para internalizada, já que comparações de chaves (após o hash) podem ser feitas por uma comparação de ponteiros em vez de uma comparação de strings. Normalmente, os nomes usados em programas Python são internalizados automaticamente, e os dicionários usados para armazenar atributos de módulo, classe ou instância têm chaves internalizadas.

Strings internalizadas não são *imortais*; você deve manter uma referência ao valor de retorno de `intern()` para se beneficiar dela.

`sys.`**`_is_gil_enabled`**`()`

Retorna `True` se a *GIL* estiver habiltada e `False` caso contrário.

Adicionado na versão 3.13.

Não há garantia de que exista em todas as implementações do Python.

`sys.`**`is_finalizing`**`()`

Retorna `True` se o interpretador Python principal estiver *desligando*. Retorna `False` caso contrário.

Veja também a exceção `PythonFinalizationError`.

Adicionado na versão 3.5.

`sys.`**`_jit`**

Utilitários para observar compilação just-in-time.

A compilação JIT é um *detalhe de implementação experimental* do CPython. Não há garantia de que `sys._jit` exista ou se comporte da mesma maneira em todas as implementações, versões ou configurações de construção do Python.

Adicionado na versão 3.14.

`_jit.`**`is_available`**`()`

Retorna `True` se o executável Python atual oferecer suporte à compilação JIT e `False` caso contrário. Isso pode ser controlado compilando o CPython com a opção `--experimental-jit` no Windows e a opção `--enable-experimental-jit` em todas as outras plataformas.

`_jit.`**`is_enabled`**`()`

Retorna `True` se a compilação JIT estiver habilitada para o processo Python atual (implica `sys._jit.is_available()`), e `False` caso contrário. Se a compilação JIT estiver disponível, isso pode ser controlado definindo a variável de ambiente `PYTHON_JIT` como `0` (desabilitada) ou `1` (habilitada) na inicialização do interpretador.

`_jit.`**`is_active`**`()`

Retorna `True` se o quadro Python mais alto estiver executando código JIT (implica `sys._jit.is_enabled()`), e `False` caso contrário.

> **Nota**
>
> Esta função destina-se a testar e depurar o próprio JIT. Deve ser evitada para qualquer outro propósito.

Nota

Devido à natureza do rastreamento de compiladores JIT, chamadas repetidas a esta função podem gerar resultados surpreendentes. Por exemplo, ramificar seu valor de retorno provavelmente levará a um comportamento inesperado (se isso fizer com que o código JIT seja inserido ou encerrado):

```
>>> for warmup in range(BIG_NUMBER):
...     # Esta linha está "quente" e, eventualmente, é compilada JIT:
...     if sys._jit.is_active():
...         # Esta linha está "fria" e é executada no interpretador:
...         assert sys._jit.is_active()
...
Traceback (most recent call last):
  File "<stdin>", line 5, in <module>
    assert sys._jit.is_active()
           ~~~~~~~~~~~~~~~~~~^^
AssertionError
```

sys.**last_exc**

Esta variável nem sempre está definida; ela é definida para a instância de exceção quando uma exceção não é tratada e o interpretador imprime uma mensagem de erro e um stack traceback. Seu uso pretendido é permitir que um usuário interativo importe um módulo depurador e se envolva na depuração post-mortem sem ter que reexecutar o comando que causou o erro. (O uso típico é `import pdb; pdb.pm()` para entrar no depurador post-mortem; veja o módulo *pdb* para mais informações.)

Adicionado na versão 3.12.

sys.**_is_immortal**(*op*)

Retorna *True* se o objeto fornecido for *imortal*, *False* caso contrário.

Nota

Objetos que são imortais (e, portanto, retornam `True` ao serem passados para esta função) não têm garantia de serem imortais em versões futuras, e vice-versa para objetos mortais.

Adicionado na versão 3.14.

Esta função deve ser usada apenas para fins especializados. Não é garantido que exista em todas as implementações do Python.

sys.**_is_interned**(*string*)

Retorna *True* se a string fornecida for "internalizada", *False* caso contrário.

Adicionado na versão 3.13.

Não há garantia de que exista em todas as implementações do Python.

sys.**last_type**

sys.**last_value**

sys.**last_traceback**

Essas três variáveis estão descontinuadas; use *sys.last_exc* em vez disso. Elas mantêm a representação legada de `sys.last_exc`, conforme retornada de *exc_info()* acima.

sys.**maxsize**

Um inteiro que fornece o valor máximo que uma variável do tipo `Py_ssize_t` pode assumir. Geralmente é `2**31 - 1` em uma plataforma de 32 bits e `2**63 - 1` em uma plataforma de 64 bits.

sys.**maxunicode**

Um inteiro que fornece o valor do maior ponto de código Unicode, ou seja, `1114111` (`0x10FFFF` em hexadecimal).

Alterado na versão 3.3: Antes da **PEP 393**, `sys.maxunicode` costumava ser `0xFFFF` ou `0x10FFFF`, dependendo da opção de configuração que especificava se os caracteres Unicode eram armazenados como UCS-2 ou UCS-4.

sys.**meta_path**

Uma lista de objetos *localizadores de metacaminho* que têm seus métodos *`find_spec()`* chamados para que algum deles localize o módulo a ser importado. Por padrão, armazena entradas que implementam a semântica de importação padrão de Python. O método *`find_spec()`* é chamado com pelo menos o nome absoluto do módulo a ser importado. Se o módulo a ser importado estiver contido num pacote, o atributo `__path__` do pacote pai é passado como segundo argumento. O método retorna um *spec de módulo*, ou `None` se o módulo não for encontrado.

Ver também

importlib.abc.MetaPathFinder
A classe base abstrata que define a interface dos objetos localizadores em *`meta_path`*.

importlib.machinery.ModuleSpec
A classe concreta da qual *`find_spec()`* deve retornar instâncias.

Alterado na versão 3.4: *Especificações de módulo* foram introduzidas no Python 3.4, pela **PEP 451**.

Alterado na versão 3.12: Foi removido o alternativa que procurava um método `find_module()` se uma entrada *`meta_path`* não tivesse um método *`find_spec()`*.

sys.**modules**

Este é um dicionário que mapeia nomes de módulos para módulos que já foram carregados. Isso pode ser manipulado para forçar o recarregamento de módulos e outros truques. No entanto, substituir o dicionário não vai funcionar necessariamente como esperado e excluir itens essenciais do dicionário pode fazer com que o Python falhe. Se você quiser iterar sobre este dicionário global, sempre use `sys.modules.copy()` ou `tuple(sys.modules)` para evitar exceções, pois seu tamanho pode mudar durante a iteração como um efeito colateral do código ou atividade em outros threads.

sys.**orig_argv**

A lista dos argumentos originais da linha de comando passados para o executável Python.

Os elementos de *`sys.orig_argv`* são os argumentos para o interpretador Python, enquanto os elementos de *`sys.argv`* são os argumentos para o programa do usuário. Os argumentos consumidos pelo próprio interpretador estarão presentes em *`sys.orig_argv`* e ausentes em *`sys.argv`*.

Adicionado na versão 3.10.

sys.**path**

Uma lista de strings que especifica o caminho de pesquisa para módulos. Inicializado a partir da variável de ambiente `PYTHONPATH`, mais um padrão dependente da instalação.

Por padrão, conforme inicializado na inicialização do programa, um caminho potencialmente inseguro é adicionado ao *`sys.path`* (*antes* das entradas inseridas como resultado de `PYTHONPATH`):

- A linha de comando `python -m módulo`: adiciona o diretório de trabalho atual
- A linha de comando `python script.py`: prefixa o diretório do script. Se for um link simbólico, resolve links simbólicos.
- As linhas de comando `python -c código` e `python` (REPL): adiciona uma string vazia, que indica o diretório de trabalho atual.

Para não acrescentar esse caminho potencialmente inseguro, use a opção de linha de comando `-P` ou a variável de ambiente `PYTHONSAFEPATH`.

Um programa é livre para modificar esta lista para seus próprios propósitos. Somente strings devem ser adicionadas a *sys.path*; todos os outros tipos de dados são ignorados durante a importação.

Ver também

- Módulo *site* Descreve como usar arquivos .pth para estender *sys.path*.

sys.path_hooks

Uma lista de chamáveis que recebem um argumento de caminho para tentar criar um *localizador* para o caminho. Se um localizador puder ser criado, ele deve ser retornado pelo chamável, senão, ele levanta *ImportError*.

Originalmente especificado na **PEP 302**.

sys.path_importer_cache

Um dicionário atuando como um cache para objetos do *localizador*. As chaves são caminhos que foram passados para *sys.path_hooks* e os valores são os localizadores que são encontrados. Se um caminho for um caminho de sistema de arquivo válido, mas nenhum localizador for encontrado em *sys.path_hooks*, então `None` é armazenado.

Originalmente especificado na **PEP 302**.

sys.platform

Uma string contendo um identificador da plataforma. Valores conhecidos são:

Sistema	Valor de `platform`
AIX	`'aix'`
Android	`'android'`
Emscripten	`'emscripten'`
FreeBSD	`'freebsd'`
iOS	`'ios'`
Linux	`'linux'`
macOS	`'darwin'`
Windows	`'win32'`
Windows/Cygwin	`'cygwin'`
WASI	`'wasi'`

Em sistemas Unix não listados na tabela, o valor é o nome do sistema operacional em letras minúsculas, conforme retornado por `uname -s`, com a primeira parte da versão, conforme retornado por `uname -r` anexada, por exemplo, `'sunos5'`, *no momento em que o Python foi construído*. A menos que você queira testar uma versão específica do sistema, é recomendável usar o seguinte idioma:

```
if sys.platform.startswith('sunos'):
    # código específico do SunOS aqui...
```

Alterado na versão 3.3: No Linux, *sys.platform* não contém mais a versão principal. É sempre `'linux'`, em vez de `'linux2'` ou `'linux3'`.

Alterado na versão 3.8: No AIX, *sys.platform* não contém mais a versão principal. É sempre `'aix'`, em vez de `'aix5'` ou `'aix7'`.

Alterado na versão 3.13: No Android, *sys.platform* agora retorna `'android'` em vez de `'linux'`.

Alterado na versão 3.14: No FreeBSD, *sys.platform* não contém mais a versão principal. É sempre `'freebsd'`, em vez de `'freebsd13'` ou `'freebsd14'`.

> **Ver também**
>
> *os.name* tem uma granularidade mais substancial. *os.uname()* fornece informações de versão dependentes do sistema.
>
> O módulo *platform* fornece verificações detalhadas sobre a identificação do sistema.

sys.**platlibdir**

Nome do diretório da biblioteca específica da plataforma. É usado para construir o caminho da biblioteca padrão e os caminhos dos módulos de extensão instalados.

É igual a `"lib"` na maioria das plataformas. No Fedora e SuSE, é igual a `"lib64"` em plataformas de 64 bits, o que fornece os seguintes caminhos `sys.path` (onde `X.Y` é a versão `major.minor` do Python):

- `/usr/lib64/pythonX.Y/`: Biblioteca padrão (como `os.py` do módulo *os*)
- `/usr/lib64/pythonX.Y/lib-dynload/`: Módulos de extensão C da biblioteca padrão (como o módulo *errno*, o nome exato do arquivo é específico da plataforma)
- `/usr/lib/pythonX.Y/site-packages/` (sempre usa `lib`, não *sys.platlibdir*): Módulos de terceiros
- `/usr/lib64/pythonX.Y/site-packages/`: Módulos de extensão C de pacotes de terceiros

Adicionado na versão 3.9.

sys.**prefix**

Uma string que fornece o prefixo do diretório específico do site onde os arquivos Python independentes de plataforma são instalados; no Unix, o padrão é `/usr/local`. Isso pode ser definido no momento da construção com o argumento `--prefix` para o script **configure**. Veja *Caminhos de instalação* para caminhos derivados.

> **Nota**
>
> Se um *ambiente virtual* estiver em vigor, este *prefix* vai apontar para o ambiente virtual. O valor para a instalação do Python ainda estará disponível, via *base_prefix*. Confira *Ambientes virtuais* para mais informações.

Alterado na versão 3.14: Ao executar em um *ambiente virtual*, *prefix* e *exec_prefix* agora são definidos como o prefixo do ambiente virtual pela *inicialização do caminho*, em vez de *site*. Isso significa que *prefix* e *exec_prefix* sempre apontam para o ambiente virtual, mesmo quando *site* está desabilitado (`-S`).

sys.**ps1**

sys.**ps2**

Strings especificando o prompt primário e secundário do interpretador. Elas são definidas somente se o interpretador estiver no modo interativo. Seus valores iniciais neste caso são `'>>> '` e `'... '`. Se um objeto não string for atribuído a qualquer variável, seu *str()* é reavaliado cada vez que o interpretador se prepara para ler um novo comando interativo; isso pode ser usado para implementar um prompt dinâmico.

sys.**setdlopenflags**(*n*)

Define os sinalizadores usados pelo interpretador para chamadas `dlopen()`, como quando o interpretador carrega módulos de extensão. Entre outras coisas, isso habilitará uma resolução preguiçosa de símbolos ao importar um módulo, se chamado como `sys.setdlopenflags(0)`. Para compartilhar símbolos entre módulos de extensão, chame como `sys.setdlopenflags(os.RTLD_GLOBAL)`. Nomes simbólicos para os valores de sinalizadores podem ser encontrados no módulo *os* (constantes `RTLD_xxx`, por exemplo, *os.RTLD_LAZY*).

Disponibilidade: Unix.

sys.**set_int_max_str_digits**(*maxdigits*)

Define a *limitação de comprimento de string na conversão para inteiro* usada por este interpretador. Veja também *get_int_max_str_digits()*.

Adicionado na versão 3.11.

sys.**setprofile**(*profilefunc*)

Define a função de perfil do sistema, que permite implementar um perfilador de código-fonte Python em Python. Veja o capítulo *Os Profilers do Python* para mais informações sobre o perfilador Python. A função de perfil do sistema é chamada de forma semelhante à função de rastreamento do sistema (veja *settrace()*), mas é chamada com eventos diferentes, por exemplo, não é chamada para cada linha de código executada (apenas na chamada e retorno, mas o evento de retorno é relatado mesmo quando uma exceção foi definida). A função é específica do thread, mas não há como o perfilador saber sobre trocas de contexto entre threads, então não faz sentido usá-la na presença de vários threads. Além disso, seu valor de retorno não é usado, então ele pode simplesmente retornar `None`. Um erro na função de perfil fará com que sua própria definição seja removida.

> **Nota**
>
> O mesmo mecanismo de rastreamento é usado para `setprofile()` como *settrace()*. Para rastrear chamadas com `setprofile()` dentro de uma função de rastreamento (por exemplo, em um ponto de interrupção do depurador), consulte *call_tracing()*.

As funções de perfil devem ter três argumentos: *frame*, *event* e *arg*. *frame* é o quadro de pilha atual. *event* é uma string: `'call'`, `'return'`, `'c_call'`, `'c_return'` ou `'c_exception'`. *arg* depende do tipo de evento.

Os eventos têm o seguinte significado:

'call'
: Uma função é chamada (ou algum outro bloco de código é inserido). A função de perfil é chamada; *arg* é `None`.

'return'
: Uma função (ou outro bloco de código) está prestes a retornar. A função de perfilação é chamada; *arg* é o valor que será retornado, ou `None` se o evento for causado por uma exceção sendo levantada.

'c_call'
: Uma função C está prestes a ser chamada. Pode ser uma função de extensão ou uma embutida. *arg* é o objeto da função C.

'c_return'
: Uma função C foi retornada. *arg* é o objeto da função C.

'c_exception'
: Uma função C levantou uma exceção. *arg* é o objeto da função C.

Levanta um *evento de auditoria* `sys.setprofile` sem argumentos.

sys.**setrecursionlimit**(*limit*)

Define a profundidade máxima da pilha do interpretador Python para *limit*. Esse limite impede que a recursão infinita cause um estouro da pilha C e trave o Python.

O limite mais alto possível depende da plataforma. Um usuário pode precisar definir o limite mais alto quando tiver um programa que exija recursão profunda e uma plataforma que dê suporte a um limite mais alto. Isso deve ser feito com cuidado, porque um limite muito alto pode levar a uma falha.

Se o novo limite for muito baixo na profundidade de recursão atual, uma exceção *RecursionError* será levantada.

Alterado na versão 3.5.1: Uma exceção *RecursionError* agora é levantada se o novo limite for muito baixo na profundidade de recursão atual.

sys.**setswitchinterval**(*interval*)

Define o intervalo de troca de thread do interpretador (em segundos). Este valor de ponto flutuante determina a duração ideal das "fatias de tempo" alocadas para threads Python em execução simultânea. Observe que o valor real pode ser maior, especialmente se funções ou métodos internos de execução longa forem usados.

Além disso, qual thread se torna agendada no final do intervalo é uma decisão do sistema operacional. O interpretador não tem seu próprio escalonador.

Adicionado na versão 3.2.

sys.**settrace**(*tracefunc*)

Define a função trace do sistema, que permite que você implemente um depurador de código-fonte Python em Python. A função é específica de thread; para um depurador suportar múltiplas threads, ele deve registrar uma função trace usando *settrace()* para cada thread que está sendo depurada ou usar *threading.settrace()*.

As funções de rastreamento devem ter três argumentos: *frame*, *event* e *arg*. *frame* é o quadro de pilha atual. *event* é uma string: `'call'`, `'line'`, `'return'`, `'exception'` ou `'opcode'`. *arg* depende do tipo de evento.

A função de rastreamento é invocada (com *event* definido como `'call'`) sempre que um novo escopo local é inserido; ela deve retornar uma referência a uma função de rastreamento local a ser usada para o novo escopo, ou `None` se o escopo não deve ser rastreado.

A função de rastreamento local deve retornar uma referência a si mesma ou a outra função que seria então usada como a função de rastreamento local para o escopo.

Se ocorrer algum erro na função de rastreamento, ela será desativada, assim como `settrace(None)` é chamado.

> **Nota**
>
> O rastreamento é desabilitado ao chamar a função de rastreamento (por exemplo, uma função definida por `settrace()`). Para rastreamento recursivo, veja *call_tracing()*.

Os eventos têm o seguinte significado:

'call'
Uma função é chamada (ou algum outro bloco de código é inserido). A função de rastreamento global é chamada; *arg* é `None`; o valor de retorno especifica a função de rastreamento local.

'line'
O interpretador está prestes a executar uma nova linha de código ou reexecutar a condição de um laço. A função de rastreamento local é chamada; *arg* é `None`; o valor de retorno especifica a nova função de rastreamento local. Veja `Objects/lnotab_notes.txt` para uma explicação detalhada de como isso funciona. Eventos por linha podem ser desabilitados para um quadro definindo `f_trace_lines` como *False* naquele quadro.

'return'
Uma função (ou outro bloco de código) está prestes a retornar. A função de rastreamento local é chamada; *arg* é o valor que será retornado, ou `None` se o evento for causado por uma exceção sendo levantada. O valor de retorno da função de rastreamento é ignorado.

'exception'
Ocorreu uma exceção. A função de rastreamento local é chamada; *arg* é uma tupla `(exception, value, traceback)`; o valor de retorno especifica a nova função de rastreamento local.

'opcode'
O interpretador está prestes a executar um novo opcode (veja *dis* para detalhes do opcode). A função de rastreamento local é chamada; *arg* é `None`; o valor de retorno especifica a nova função de rastreamento local. Eventos por opcode não são emitidos por padrão: eles devem ser explicitamente solicitados definindo `f_trace_opcodes` como *True* no quadro.

Observe que, à medida que uma exceção é propagada pela cadeia de chamadores, um evento `'exception'` é gerado em cada nível.

Para um uso mais refinado, é possível definir uma função de rastreamento atribuindo `frame.f_trace = tracefunc` explicitamente, em vez de depender de ser definida indiretamente por meio do valor de retorno

de uma função de rastreamento já instalada. Isso também é necessário para ativar a função de rastreamento no quadro atual, o que *settrace()* não faz. Observe que, para que isso funcione, uma função de rastreamento global deve ter sido instalada com *settrace()* para habilitar o maquinário de rastreamento em tempo de execução, mas não precisa ser a mesma função de rastreamento (por exemplo, pode ser uma função de rastreamento de baixa sobrecarga que simplesmente retorna `None` para se desabilitar imediatamente em cada quadro).

Para mais informações sobre objetos de código e quadro, consulte types.

Levanta um *evento de auditoria* `sys.settrace` sem argumentos.

A função *settrace()* destina-se apenas à implementação de depuradores, perfiladores, ferramentas de cobertura e similares. Seu comportamento faz parte da plataforma de implementação, e não da definição da linguagem e, portanto, pode não estar disponível em todas as implementações do Python.

Alterado na versão 3.7: Tipo de evento `'opcode'` adicionado; atributos `f_trace_lines` e `f_trace_opcodes` adicionados a quadros

`sys.`**`set_asyncgen_hooks`**(*[firstiter] [, finalizer]*)

Aceita dois argumentos nomeados opcionais que são chamáveis que aceitam um *iterador gerador assíncrono* como argumento. O chamável *firstiter* será chamado quando um gerador assíncrono for iterado pela primeira vez. O *finalizer* será chamado quando um gerador assíncrono estiver prestes a ser coletado como lixo.

Levanta um *evento de auditoria* `sys.set_asyncgen_hooks_firstiter` sem argumentos.

Levanta um *evento de auditoria* `sys.set_asyncgen_hooks_finalizer` sem argumentos.

Dois eventos de auditoria são gerados porque a API subjacente consiste em duas chamadas, cada uma das quais deve levantar seu próprio evento.

Adicionado na versão 3.6: Veja a **PEP 525** para mais detalhes, e para um exemplo de referência de um método *finalizer* veja a implementação de `asyncio.Loop.shutdown_asyncgens` em Lib/asyncio/base_events.py

> **Nota**
>
> Esta função foi adicionada provisoriamente (veja **PEP 411** para detalhes.)

`sys.`**`set_coroutine_origin_tracking_depth`**(*depth*)

Permite habilitar ou desabilitar o rastreamento de origem de corrotina. Quando habilitado, o atributo `cr_origin` em objetos de corrotina conterá uma tupla de tuplas (nome do arquivo, número da linha, nome da função) descrevendo o traceback onde o objeto corrotina foi criado, com a chamada mais recente primeiro. Quando desabilitado, `cr_origin` será `None`.

Para habilitar, passe um valor *depth* maior que zero; isso define o número de quadros cujas informações serão capturadas. Para desabilitar, defina *depth* para zero.

Esta configuração é específica do thread.

Adicionado na versão 3.7.

> **Nota**
>
> Esta função foi adicionada provisoriamente (veja **PEP 411** para detalhes.) Use-a apenas para propósitos de depuração.

`sys.`**`activate_stack_trampoline`**(*backend*, */*)

Ativa o trampolim do perfilador de pilha *backend*. O único backend suportado é `"perf"`.

Os trampolins empilháveis não podem ser ativados se o JIT estiver ativo.

Disponibilidade: Linux.

Adicionado na versão 3.12.

Ver também

- perf_profiling
- https://perf.wiki.kernel.org

`sys.`**`deactivate_stack_trampoline`**`()`

Desativa o backend trampolim do perfilador de pilha atual.

Se nenhum perfilador de pilha estiver ativado, esta função não terá efeito.

Disponibilidade: Linux.

Adicionado na versão 3.12.

`sys.`**`is_stack_trampoline_active`**`()`

Retorna `True` se um trampolim de perfilador de pilha estiver ativo.

Disponibilidade: Linux.

Adicionado na versão 3.12.

`sys.`**`remote_exec`**`(`*pid*, *script*`)`

Executa *script*, um arquivo contendo código Python no processo remoto com o *pid* fornecido.

Esta função retorna imediatamente, e o código será executado pela thread principal do processo de destino na próxima oportunidade disponível, de forma semelhante à manipulação de sinais. Não há interface para determinar quando o código foi executado. O chamador é responsável por garantir que o arquivo ainda exista sempre que o processo remoto tentar lê-lo e que ele não tenha sido sobrescrito.

O processo remoto deve estar executando um interpretador CPython da mesma versão principal e secundária que o processo local. Se o interpretador local ou remoto for de pré-lançamento (alfa, beta ou candidato a lançamento), os interpretadores local e remoto devem ser exatamente da mesma versão.

Quando o código é executado no processo remoto, um *evento de auditoria* `sys.remote_exec` é gerado com o *pid* e o caminho para o arquivo de script. Este evento é levantado no processo que chamou *`sys.remote_exec()`*.

Quando o script é executado no processo remoto, um *evento de auditoria* `cpython.remote_debugger_script` é levantado com o caminho no processo remoto. Este evento é levantado no processo remoto, não naquele que chamou *`sys.remote_exec()`*.

Disponibilidade: Unix, Windows.

Adicionado na versão 3.14.

`sys.`**`_enablelegacywindowsfsencoding`**`()`

Altera *tratador de erros e codificação do sistema de arquivos* para 'replace' e 'mbcs' respectivamente, para consistência com versões do Python anteriores à 3.6.

Isso é equivalente a definir a variável de ambiente `PYTHONLEGACYWINDOWSFSENCODING` antes de iniciar o Python.

Veja também *`sys.getfilesystemencoding()`* e *`sys.getfilesystemencodeerrors()`*.

Disponibilidade: Windows.

Nota

Alterar a codificação do sistema de arquivos após a inicialização do Python é arriscado porque o fsencoding antigo ou os caminhos codificados pelo fsencoding antigo podem estar armazenados em cache em algum lugar. Use `PYTHONLEGACYWINDOWSFSENCODING` em vez disso.

Adicionado na versão 3.6: Veja **PEP 529** para mais detalhes.

Deprecated since version 3.13, will be removed in version 3.16: Use `PYTHONLEGACYWINDOWSFSENCODING`.

`sys.`**`stdin`**

`sys.`**`stdout`**

`sys.`**`stderr`**

Objetos arquivo usados pelo interpretador para entrada padrão, saída e erros:

- `stdin` é usado para todas as entradas interativas (incluindo chamadas para *`input()`*);
- `stdout` é usado para a saída das instruções *`print()`* e *expressões* e para os prompts de *`input()`*;
- Os prompts do próprio interpretador e suas mensagens de erro vão para `stderr`.

Esses fluxos são *arquivos texto* regulares como aqueles retornados pela função *`open()`*. Seus parâmetros são escolhidos da seguinte forma:

- A codificação e o tratamento de erros são inicializados a partir de `PyConfig.stdio_encoding` e `PyConfig.stdio_errors`.

  No Windows, o UTF-8 é usado para o dispositivo de console. Dispositivos não-caracteres, como arquivos de disco e pipes, usam a codificação da localidade do sistema (ou seja, a página de código ANSI). Dispositivos de caracteres não-console, como NUL (ou seja, onde `isatty()` retorna `True`) usam o valor das páginas de código de entrada e saída do console na inicialização, respectivamente para stdin e stdout/stderr. Isso assume como padrão a *codificação da localidade* do sistema se o processo não for inicialmente anexado a um console.

  O comportamento especial do console pode ser substituído definindo a variável de ambiente PYTHONLEGACYWINDOWSSTDIO antes de iniciar o Python. Nesse caso, as páginas de código do console são usadas como para qualquer outro dispositivo de caractere.

  Em todas as plataformas, você pode substituir a codificação de caracteres definindo a variável de ambiente `PYTHONIOENCODING` antes de iniciar o Python ou usando a nova opção de linha de comando `-X utf8` e a variável de ambiente `PYTHONUTF8`. No entanto, para o console do Windows, isso só se aplica quando `PYTHONLEGACYWINDOWSSTDIO` também está definido.

- Quando interativo, o fluxo `stdout` é armazenado em buffer de linha. Caso contrário, ele é armazenado em buffer de bloco como arquivos texto comuns. O fluxo `stderr` é armazenado em buffer de linha em ambos os casos. Você pode tornar ambos os fluxos sem buffer passando a opção de linha de comando `-u` ou definindo a variável de ambiente `PYTHONUNBUFFERED`.

Alterado na versão 3.9: `stderr` não interativo agora é armazenado em buffer de linha em vez de totalmente armazenado em buffer.

> **Nota**
>
> Para escrever ou ler dados binários de/para os fluxos padrão, use o objeto binário subjacente *`buffer`*. Por exemplo, para escrever bytes em *`stdout`*, use `sys.stdout.buffer.write(b'abc')`.
>
> Entretanto, se você estiver escrevendo uma biblioteca (e não controlar em qual contexto seu código será executado), esteja ciente de que os fluxos padrão podem ser substituídos por objetos arquivo ou similares, como *`io.StringIO`*, que não oferecem suporte ao atributo `buffer`.

`sys.`**`__stdin__`**

`sys.`**`__stdout__`**

`sys.`**`__stderr__`**

Esses objetos contêm os valores originais de `stdin`, `stderr` e `stdout` no início do programa. Eles são usados durante a finalização e podem ser úteis para imprimir no fluxo padrão real, não importa se o objeto `sys.std*` foi redirecionado.

Ele também pode ser usado para restaurar os arquivos reais para objetos arquivo de trabalho conhecidos, caso tenham sido substituídos por um objeto quebrado. No entanto, a maneira preferida de fazer isso é salvar explicitamente o fluxo anterior antes de substituí-lo e restaurar o objeto salvo.

> **Nota**
>
> Sob algumas condições, `stdin`, `stdout` e `stderr`, bem como os valores originais de `__stdin__`, `__stdout__` e `__stderr__` podem ser `None`. Geralmente é o caso de aplicativos GUI do Windows que não estão conectados a um console e aplicativos Python iniciados com **`pythonw`**.

`sys.`**`stdlib_module_names`**

Um frozenset de strings contendo os nomes dos módulos da biblioteca padrão.

É o mesmo em todas as plataformas. Módulos que não estão disponíveis em algumas plataformas e módulos desabilitados na construção do Python também são listados. Todos os tipos de módulos são listados: Python puro, módulos embutidos, congelados e de extensão. Módulos de teste são excluídos.

Para pacotes, somente o pacote principal é listado: subpacotes e submódulos não são listados. Por exemplo, o pacote `email` é listado, mas o subpacote `email.mime` e o submódulo `email.message` não são listados.

Veja também a lista de *`sys.builtin_module_names`*.

Adicionado na versão 3.10.

`sys.`**`thread_info`**

Uma *tupla nomeada* contendo informações sobre a implementação de threads.

`thread_info.`**`name`**

O nome da implementação da thread:

- `"nt"`: Threads do Windows
- `"pthread"`: Threads do POSIX
- `"pthread-stubs"`: stubs de threads POSIX (em plataformas WebAssembly que não implementam threads).
- `"solaris"`: Threads do Solaris

`thread_info.`**`lock`**

O nome da implementação de trava.

- `"semaphore"`: uma trava usa um semáforo
- `"mutex+cond"`: uma trava usa um mutex e uma variável de condição
- `None` se essa informação for desconhecida

`thread_info.`**`version`**

O nome e a versão da biblioteca de threads. É uma string, ou `None` se essa informação for desconhecida.

Adicionado na versão 3.3.

`sys.`**`tracebacklimit`**

Quando essa variável é definida como um valor inteiro, ela determina o número máximo de níveis de informações de traceback impressas quando ocorre uma exceção não tratada. O padrão é `1000`. Quando definida como `0` ou menos, todas as informações de traceback são suprimidas e apenas o tipo e o valor da exceção são impressos.

`sys.`**`unraisablehook`**(*unraisable*, */*)

Manipula uma exceção não levantada

Chamada quando uma exceção ocorreu, mas não há como o Python manipulá-la. Por exemplo, quando um destrutor levanta uma exceção ou durante a coleta de lixo (*`gc.collect()`*).

O argumento *unraisable* tem os seguintes atributos:

- `exc_type`: Tipo de exceção.
- `exc_value`: Valor da exceção, pode ser `None`.
- `exc_traceback`: Traceback da exceção, pode ser `None`.
- `err_msg`: Mensagem de erro, pode ser `None`.
- `object`: O objeto que causa a exceção pode ser `None`.

Os formatos de gancho padrão `err_msg` e `object` como: `f'{err_msg}: {object!r}'`; usa a mensagem de erro "Exception ignored in" se `err_msg` for `None`.

`sys.unraisablehook()` pode ser substituída para controlar como exceções não levantáveis são manipuladas.

Ver também

`excepthook()`, que manipula exceções não capturadas.

Aviso

Armazena `exc_value` usando um gancho personalizado pode criar um ciclo de referência. Ele deve ser limpo explicitamente para quebrar o ciclo de referência quando a exceção não for mais necessária.

Armazenar `object` usando um gancho personalizado pode ressuscitá-lo se ele for definido como um objeto que está sendo finalizado. Evite armazenar `object` após o gancho personalizado ser concluído para evitar ressuscitar objetos.

Levanta um evento de auditoria `sys.unraisablehook` com argumentos *hook* e *unraisable* quando ocorrer uma exceção que não puder ser manipulada. O objeto *unraisable* é o mesmo que será passado para o gancho. Se nenhum gancho tiver sido definido, *hook* pode ser `None`.

Adicionado na versão 3.8.

`sys.`**`version`**

Uma string contendo o número da versão do interpretador Python mais informações adicionais sobre o número da construção e o compilador usado. Esta string é exibida quando o interpretador interativo é iniciado. Não extraia informações de versão dela, em vez disso, use *`version_info`* e as funções fornecidas pelo módulo *`platform`*.

`sys.`**`api_version`**

A versão da API C, equivalente à macro C `PYTHON_API_VERSION`. Definido para retrocompatibilidade.

Atualmente, essa constante não é atualizada nas novas versões de Python e não é útil para controle de versão. Isso pode mudar no futuro.

`sys.`**`version_info`**

Uma tupla contendo os cinco componentes do número da versão: *major*, *minor*, *micro*, *releaselevel* e *serial*. Todos os valores, exceto *releaselevel*, são inteiros; o nível de versão é `'alpha'`, `'beta'`, `'candidate'` ou `'final'`. O valor `version_info` correspondente à versão 2.0 do Python é `(2, 0, 0, 'final', 0)`. Os componentes também podem ser acessados por nome, então `sys.version_info[0]` é equivalente a `sys.version_info.major` e assim por diante.

Alterado na versão 3.1: Adiciona componentes nomeados como atributos.

`sys.`**`warnoptions`**

Este é um detalhe de implementação do framework de avisos; não modifique este valor. Consulte o módulo *`warnings`* para obter mais informações sobre o framework de avisos.

sys.**winver**

O número da versão usado para formar chaves de registro em plataformas Windows. Isso é armazenado como recurso de string 1000 na DLL do Python. O valor normalmente é a versão principal e secundária do interpretador Python em execução. Ele é fornecido no módulo *sys* para fins informativos; modificar esse valor não tem efeito nas chaves de registro usadas pelo Python.

Disponibilidade: Windows.

sys.**monitoring**

Espaço de nomes contendo funções e constantes para função de retorno de registro e controle de eventos de monitoramento. Veja *sys.monitoring* para detalhes.

sys.**_xoptions**

Um dicionário dos vários sinalizadores específicos de implementação passados pela opção de linha de comando `-X`. Os nomes de opção são mapeados para seus valores, se fornecidos explicitamente, ou para *True*. Exemplo:

```
$ ./python -Xa=b -Xc
Python 3.2a3+ (py3k, Oct 16 2010, 20:14:50)
[GCC 4.4.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import sys
>>> sys._xoptions
{'a': 'b', 'c': True}
```

Esta é uma maneira específica do CPython de acessar opções passadas por `-X`. Outras implementações podem exportá-las por outros meios, ou não exportá-las.

Adicionado na versão 3.2.

Citações

29.2 `sys.monitoring` — Monitoramento de eventos de execução

Adicionado na versão 3.12.

Nota

sys.monitoring é um espaço de nomes dentro do módulo *sys*, não um módulo independente, então não há necessidade de importá-lo com `import sys.monitoring`, bastando usar `import sys` e então usar `sys.monitoring`.

Esse espaço de nomes fornece acesso às funções e constantes necessárias para ativar e controlar o monitoramento de eventos.

À medida que os programas são executados, ocorrem eventos que podem ser de interesse para as ferramentas que monitoram a execução. O espaço de nomes *sys.monitoring* fornece meios para receber retornos de chamada quando ocorrem eventos de interesse.

A API de monitoramento consiste em três componentes:

- *Identificadores de ferramenta*
- *Eventos*
- *Funções de retorno de chamadas*

29.2.1 Identificadores de ferramenta

Um identificador de ferramenta é um número inteiro e o nome associado. Os identificadores de ferramenta são usados para evitar que as ferramentas interfiram umas nas outras e para permitir que várias ferramentas operem ao mesmo tempo. Atualmente, as ferramentas são totalmente independentes e não podem ser usadas para monitorar umas às outras. Essa restrição poderá ser suspensa no futuro.

Antes de registrar ou ativar eventos, uma ferramenta deve escolher um identificador. Os identificadores são números inteiros no intervalo de 0 a 5, inclusive.

Registro e uso de ferramentas

sys.monitoring.**use_tool_id**(*tool_id:* int, *name:* str, /) → *None*

Deve ser chamado antes que *tool_id* possa ser usado. *tool_id* deve estar no intervalo de 0 a 5, inclusive. Levanta um `ValueError` se *tool_id* estiver em uso.

sys.monitoring.**clear_tool_id**(*tool_id:* int, /) → *None*

Cancela o registro de todos os eventos e funções de retorno de chamada associados a *tool_id*.

sys.monitoring.**free_tool_id**(*tool_id:* int, /) → *None*

Deve ser chamado quando uma ferramenta não precisar mais de *tool_id*. Chamará `clear_tool_id()` antes de liberar *tool_id*.

sys.monitoring.**get_tool**(*tool_id:* int, /) → *str* | *None*

Retorna o nome da ferramenta se *tool_id* estiver em uso; caso contrário, retorna `None`. *tool_id* deve estar no intervalo de 0 a 5, inclusive.

Todas as IDs são tratadas da mesma forma pela VM com relação aos eventos, mas as seguintes IDs são predefinidas para facilitar a cooperação entre as ferramentas:

```
sys.monitoring.DEBUGGER_ID = 0
sys.monitoring.COVERAGE_ID = 1
sys.monitoring.PROFILER_ID = 2
sys.monitoring.OPTIMIZER_ID = 5
```

29.2.2 Eventos

Os seguintes eventos são suportados:

sys.monitoring.events.**BRANCH_LEFT**

Uma ramificação condicional vai para a esquerda.

Cabe à ferramenta determinar como apresentar os ramos "esquerdo" e "direito". Não há garantia de qual ramo é "esquerdo" e qual é "direito", exceto que será consistente durante toda a duração do programa.

sys.monitoring.events.**BRANCH_RIGHT**

Uma ramificação condicional vai para a direita.

sys.monitoring.events.**CALL**

Uma chamada no código Python (o evento ocorre antes da chamada).

sys.monitoring.events.**C_RAISE**

Uma exceção levantada por qualquer chamável, exceto funções Python (o evento ocorre após a saída).

sys.monitoring.events.**C_RETURN**

Retorno de qualquer chamável, exceto por funções Python (o evento ocorre após o retorno).

sys.monitoring.events.**EXCEPTION_HANDLED**

Uma exceção é tratada.

sys.monitoring.events.**INSTRUCTION**

Uma instrução VM está prestes a ser executada.

sys.monitoring.events.**JUMP**

É feito um salto incondicional no gráfico do fluxo de controle.

sys.monitoring.events.**LINE**

Está prestes a ser executada uma instrução que tem um número de linha diferente da instrução anterior.

sys.monitoring.events.**PY_RESUME**

Retomada de uma função Python (para funções geradoras e de corrotina), exceto para chamadas `throw()`.

sys.monitoring.events.**PY_RETURN**

Retorno de uma função Python (ocorre imediatamente antes do retorno, o quadro do receptor estará na pilha).

sys.monitoring.events.**PY_START**

Início de uma função Python (ocorre imediatamente após a chamada, o quadro do receptor da chamada estará na pilha)

sys.monitoring.events.**PY_THROW**

Uma função Python é retomada por uma chamada `throw()`.

sys.monitoring.events.**PY_UNWIND**

Saída de uma função Python durante o desenrolamento de uma exceção. Isso inclui exceções levantadas diretamente na função e que podem continuar a se propagar.

sys.monitoring.events.**PY_YIELD**

Produz de uma função Python (ocorre imediatamente antes do yield, o quadro do receptor estará na pilha).

sys.monitoring.events.**RAISE**

Uma exceção é levantada, exceto aquelas que causam um evento *STOP_ITERATION*.

sys.monitoring.events.**RERAISE**

Uma exceção é levantada novamente, por exemplo, no final de um bloco `finally`.

sys.monitoring.events.**STOP_ITERATION**

Uma exceção artificial *StopIteration* é levantada; consulte *o evento STOP_ITERATION*.

Mais eventos poderão ser adicionados no futuro.

Esses eventos são atributos do espaço de nomes `sys.monitoring.events`. Cada evento é representado como uma constante de potência de 2 inteiros. Para definir um conjunto de eventos, basta aplicar OU (OR) bit a bit nos eventos individuais juntos. Por exemplo, para especificar os eventos *PY_RETURN* e *PY_START*, use a expressão `PY_RETURN | PY_START`.

sys.monitoring.events.**NO_EVENTS**

Um apelido para `0` para que os usuários possam fazer comparações explícitas como:

```
if get_events(DEBUGGER_ID) == NO_EVENTS:
    ...
```

Definir este evento desativa todos os eventos.

Eventos locais

Os eventos locais estão associados à execução normal do programa e ocorrem em locais claramente definidos. Todos os eventos locais podem ser desativados. Os eventos locais são:

- *PY_START*
- *PY_RESUME*
- *PY_RETURN*
- *PY_YIELD*
- *CALL*

- *LINE*
- *INSTRUCTION*
- *JUMP*
- *BRANCH_LEFT*
- *BRANCH_RIGHT*
- *STOP_ITERATION*

Evento descontinuado

- BRANCH

O evento BRANCH foi descontinuado na versão 3.14. Usar os eventos *BRANCH_LEFT* e *BRANCH_RIGHT* proporcionará um desempenho muito melhor, pois podem ser desabilitados independentemente.

Eventos auxiliares

Os eventos auxiliares podem ser monitorados como outros eventos, mas são controlados por outro evento:

- *C_RAISE*
- *C_RETURN*

Os eventos *C_RETURN* e *C_RAISE* são controlados pelo evento *CALL*. Os eventos *C_RETURN* e *C_RAISE* só serão vistos se o evento *CALL* correspondente estiver sendo monitorado.

Outros eventos

Outros eventos não estão necessariamente vinculados a um local específico no programa e não podem ser desativados individualmente.

Os outros eventos que podem ser monitorados são:

- *PY_THROW*
- *PY_UNWIND*
- *RAISE*
- *EXCEPTION_HANDLED*

O evento STOP_ITERATION

PEP 380 especifica que uma exceção *StopIteration* é levantada ao retornar um valor de um gerador ou corrotina. No entanto, essa é uma maneira muito ineficiente de retornar um valor, portanto, algumas implementações do Python, especialmente o CPython 3.12+, não levantam uma exceção, a menos que ela seja visível para outro código.

Para permitir que as ferramentas monitorem exceções reais sem reduzir a velocidade dos geradores e das corrotinas, o evento *STOP_ITERATION* é fornecido. O *STOP_ITERATION* pode ser desativado localmente, ao contrário do *RAISE*.

Observe que o evento *STOP_ITERATION* e o evento *RAISE* para uma exceção *StopIteration* são equivalentes e tratados como intercambiáveis ao gerar eventos. As implementações favorecerão *STOP_ITERATION* por questões de desempenho, mas podem gerar um evento *RAISE* com uma *StopIteration*.

29.2.3 Ativação e desativação de eventos

Para monitorar um evento, ele deve ser ativado e uma função de retorno correspondente deve ser registrada. Os eventos podem ser ativados ou desativados definindo-os globalmente e/ou para um objeto código específico. Um evento vai acionar apenas uma vez, mesmo se ele for ligado globalmente e localmente.

Definir eventos globalmente

Os eventos podem ser controlados globalmente, modificando o conjunto de eventos que estão sendo monitorados.

sys.monitoring.**get_events**(*tool_id:* int, /) → *int*

Retorna o endereço `int` que representa todos os eventos ativos.

sys.monitoring.**set_events**(*tool_id:* int, *event_set:* int, /) → *None*

Ativa todos os eventos definidos em *event_set*. Levanta um *ValueError* se *tool_id* não estiver em uso.

Nenhum evento está ativo por padrão.

Eventos por objeto código

Os eventos também podem ser controlados por objeto código. As funções definidas abaixo que aceitam um *types.CodeType* devem estar preparadas para aceitar um objeto semelhante de funções que não são definidas no Python (veja c-api-monitoring).

sys.monitoring.**get_local_events**(*tool_id:* int, *code:* CodeType, /) → *int*

Retorna todos os eventos locais para *code*

sys.monitoring.**set_local_events**(*tool_id:* int, *code:* CodeType, *event_set:* int, /) → *None*

Ativa todos os eventos locais para *code* que estão definidos em *event_set*. Levanta a *ValueError* se *tool_id* não estiver em uso.

Desativação de eventos

sys.monitoring.**DISABLE**

Um valor especial que pode ser retornado de um função de retorno função para desativar eventos para o local do código atual.

Os eventos locais podem ser desativados para um local de código específico, retornando *sys.monitoring.DISABLE* de uma função de retorno de chamada. Isso não altera quais eventos são definidos ou quaisquer outros locais de código para o mesmo evento.

A desativação de eventos para locais específicos é muito importante para o monitoramento de alto desempenho. Por exemplo, um programa pode ser executado em um depurador sem sobrecarga se o depurador desativar todo o monitoramento, exceto alguns pontos de interrupção.

sys.monitoring.**restart_events**() → *None*

Habilita todos os eventos que foram desabilitados por *sys.monitoring.DISABLE* para todas as ferramentas.

29.2.4 Registro de funções de retorno de chamada

sys.monitoring.**register_callback**(*tool_id:* int, *event:* int, *func:* Callable | None, /) → *Callable* | *None*

Registra o chamável *func* para o *event* com o *tool_id* fornecido

Se outra função de retorno tiver sido registrada para o *tool_id* e o *event* fornecidos, ela será cancelada e retornada. Caso contrário, *register_callback()* retorna `None`.

Levanta um *evento de auditoria* `sys.monitoring.register_callback` com o argumento `func`.

As funções podem ser canceladas chamando `sys.monitoring.register_callback(tool_id, event, None)`.

As funções de retorno de chamada podem ser registradas e canceladas a qualquer momento.

Os retornos de chamada são chamados apenas uma vez, independentemente de o evento estar ativado global e localmente. Portanto, se um evento puder ser ativado tanto para eventos globais quanto locais pelo seu código, o retorno de chamada precisa ser escrito para lidar com qualquer um dos gatilhos.

Argumentos da função de retorno de chamada

sys.monitoring.**MISSING**

Um valor especial que é passado para uma função de retorno para indicar que não há argumento para a chamada.

Quando ocorre um evento ativo, a função de retorno de chamada registrada é chamada. Funções de retorno retornando um objeto diferente de *DISABLE* vai ter nenhum efeito. Eventos diferentes fornecerão à função de retorno de chamada argumentos diferentes, como segue:

- *PY_START* e *PY_RESUME*:

```
func(code: CodeType, instruction_offset: int) -> object
```

- *PY_RETURN* e *PY_YIELD*:

```
func(code: CodeType, instruction_offset: int, retval: object) -> object
```

- *CALL*, *C_RAISE* e *C_RETURN* (*arg0* pode ser *MISSING* especificamente):

```
func(code: CodeType, instruction_offset: int, callable: object, arg0: object) -
↪> object
```

code representa o objeto de código onde a chamada está sendo feita, enquanto *callable* é o objeto que está prestes a ser chamado (e, portanto, acionou o evento). Se não houver argumentos, *arg0* é definido como *sys.monitoring.MISSING*.

Para métodos de instância, *callable* será o objeto função encontrado na classe com *arg0* definido para a instância (ou seja, o argumento `self` para o método).

- *RAISE*, *RERAISE*, *EXCEPTION_HANDLED*, *PY_UNWIND*, *PY_THROW* e *STOP_ITERATION*:

```
func(code: CodeType, instruction_offset: int, exception: BaseException) ->␣
↪object
```

- *LINE*:

```
func(code: CodeType, line_number: int) -> object
```

- *BRANCH_LEFT*, *BRANCH_RIGHT* e *JUMP*:

```
func(code: CodeType, instruction_offset: int, destination_offset: int) ->␣
↪object
```

Observe que o *destination_offset* é onde o código será executado em seguida.

- *INSTRUCTION*:

```
func(code: CodeType, instruction_offset: int) -> object
```

29.3 `sysconfig` — Fornece acesso às informações de configuração do Python

Adicionado na versão 3.2.

Código-fonte: Lib/sysconfig

O módulo *sysconfig* fornece acesso às informações de configuração do Python, como a lista de caminhos de instalação e as variáveis de configuração relevantes para a plataforma atual.

29.3.1 Variáveis de configuração

Uma distribuição do Python contém um arquivo de cabeçalho `Makefile` e `pyconfig.h` que são necessários para construir o próprio binário do Python e extensões C de terceiros compiladas usando `setuptools`.

sysconfig coloca todas as variáveis encontradas nestes arquivos em um dicionário que pode ser acessado usando *get_config_vars()* ou *get_config_var()*.

Note que no Windows, é um conjunto muito menor.

`sysconfig.`**`get_config_vars`**`(`**args*`)`

Sem argumentos, retorna um dicionário de todas as variáveis de configuração relevantes para a plataforma atual.

Com argumentos, retorna uma lista de valores resultantes da pesquisa de cada argumento no dicionário de variáveis de configuração.

Para cada argumento, se o valor não for encontrado, retorna `None`.

`sysconfig.`**`get_config_var`**`(`*name*`)`

Retorna o valor de uma única variável *nome*. Equivalente a `get_config_vars().get(name)`.

Se *name* não for encontrado, retorna `None`.

Exemplo de uso:

```
>>> import sysconfig
>>> sysconfig.get_config_var('Py_ENABLE_SHARED')
0
>>> sysconfig.get_config_var('LIBDIR')
'/usr/local/lib'
>>> sysconfig.get_config_vars('AR', 'CXX')
['ar', 'g++']
```

29.3.2 Caminhos de instalação

O Python usa um esquema de instalação que difere dependendo da plataforma e das opções de instalação. Esses esquemas são armazenados em *sysconfig* sob identificadores únicos baseados no valor retornado por *os.name*. Os esquemas são usados por instaladores de pacote para determinar para onde copiar arquivos.

Python atualmente oferece suporte a nove esquemas:

- *posix_prefix*: esquema para plataformas POSIX como Linux ou macOS. Este é o esquema padrão usado quando o Python ou um componente é instalado.
- *posix_home*: esquema para plataformas POSIX, quando a opção *home* é usada. Esse esquema define caminhos localizados sob um prefixo inicial específico.
- *posix_user*: esquema para plataformas POSIX, quando a opção *user* é usada. Esse esquema define caminhos localizados sob o diretório pessoal (home) do usuário (*site.USER_BASE*).
- *posix_venv*: esquema para *ambientes virtuais do Python* em plataformas POSIX; por padrão, é o mesmo que *posix_prefix*.
- *nt*: esquema para Windows. Este é o esquema padrão usado quando o Python ou um componente é instalado.
- *nt_user*: esquema para Windows, quando utilizada a opção *user*.
- *nt_venv*: esquema para *ambientes virtuais do Python* no Windows; por padrão, é o mesmo que *nt_prefix*.
- *venv*: um esquema com valores de *posix_venv* ou *nt_venv* dependendo da plataforma em que o Python é executado.
- *osx_framework_user*: esquema para plataformas macOS, quando utilizada a opção *user*.

Cada esquema é composto por uma série de caminhos e cada caminho possui um identificador único. Python atualmente usa oito caminhos:

- *stdlib*: diretório que contém os arquivos da biblioteca Python padrão que não são específicos da plataforma.
- *platstdlib*: diretório que contém os arquivos da biblioteca Python padrão que são específicos da plataforma.
- *platlib*: diretório para arquivos específicos do site e específicos da plataforma.
- *purelib*: diretório para arquivos específicos do site e não específicos da plataforma (Python 'pure').
- *include*: diretório para arquivos de cabeçalho não específicos da plataforma para a API C do Python.
- *platinclude*: diretório para arquivos de cabeçalho específicos da plataforma para a API C do Python.
- *scripts*: diretório para arquivos de script.
- *data*: diretório para arquivos de dados.

29.3.3 Esquema de usuário

Este esquema foi projetado para ser a solução mais conveniente para usuários que não têm permissão de escrita no diretório global de pacotes de sites ou não desejam instalar nele.

Os arquivos serão instalados em subdiretórios de *site.USER_BASE* (escrito como *userbase* daqui em diante). Este esquema instala módulos Python puros e módulos de extensão no mesmo local (também conhecido como *site.USER_SITE*).

posix_user

Caminho	Diretório de instalação
stdlib	*userbase*/lib/python*X.Y*
platstdlib	*userbase*/lib/python*X.Y*
platlib	*userbase*/lib/python*X.Y*/site-packages
purelib	*userbase*/lib/python*X.Y*/site-packages
include	*userbase*/include/python*X.Y*
scripts	*userbase*/bin
data	*userbase*

nt_user

Caminho	Diretório de instalação
stdlib	*userbase*\Python*XY*
platstdlib	*userbase*\Python*XY*
platlib	*userbase*\Python*XY*\site-packages
purelib	*userbase*\Python*XY*\site-packages
include	*userbase*\Python*XY*\Include
scripts	*userbase*\Python*XY*\Scripts
data	*userbase*

osx_framework_user

Caminho	Diretório de instalação
stdlib	*userbase*/lib/python
platstdlib	*userbase*/lib/python
platlib	*userbase*/lib/python/site-packages
purelib	*userbase*/lib/python/site-packages
include	*userbase*/include/python*X.Y*
scripts	*userbase*/bin
data	*userbase*

29.3.4 Esquema de home

A ideia por trás do "esquema home" é que você construa e mantenha um estoque pessoal de módulos Python. O nome deste esquema é derivado da ideia de um diretório "home" no Unix, uma vez que não é incomum para um usuário Unix fazer seu diretório home ter um layout semelhante a `/usr/` ou `/usr/local/`. Este esquema pode ser usado por qualquer pessoa, independentemente do sistema operacional para o qual está instalando.

posix_home

Caminho	Diretório de instalação
stdlib	*home*/lib/python
platstdlib	*home*/lib/python
platlib	*home*/lib/python
purelib	*home*/lib/python
include	*home*/include/python
platinclude	*home*/include/python
scripts	*home*/bin
data	*home*

29.3.5 Esquema de prefixo

O "esquema prefixo" é útil quando você deseja usar uma instalação Python para realizar a compilação/instalação (ou seja, para executar o script de configuração), mas instalar módulos no diretório de módulo de terceiros de uma instalação Python diferente (ou algo que parece uma instalação diferente do Python). Se isso parece um pouco incomum, então é – é por isso que os esquemas usuário e home vêm antes. No entanto, existem pelo menos dois casos conhecidos em que o esquema prefixo será útil.

Primeiro, considere que muitas distribuições Linux colocam Python em `/usr`, ao invés do mais tradicional `/usr/local`. Isso é totalmente apropriado, já que, nesses casos, o Python é parte do "sistema" em vez de um complemento local. No entanto, se você estiver instalando módulos Python a partir do código-fonte, provavelmente deseja que eles entrem em `/usr/local/lib/python2.`*X* em vez de `/usr/lib/python2.`*X*.

Outra possibilidade é um sistema de arquivos de rede onde o nome usado para escrever em um diretório remoto é diferente do nome usado para lê-lo: por exemplo, o interpretador Python acessado como `/usr/local/bin/python` pode pesquisar por módulos em `/usr/local/lib/python2.`*X*, mas esses módulos teriam que ser instalados em, digamos, `/mnt/`*@server*`/export/lib/python2.`*X*.

posix_prefix

Caminho	Diretório de instalação
stdlib	*prefix*/lib/python*X.Y*
platstdlib	*prefix*/lib/python*X.Y*
platlib	*prefix*/lib/python*X.Y*/site-packages
purelib	*prefix*/lib/python*X.Y*/site-packages
include	*prefix*/include/python*X.Y*
platinclude	*prefix*/include/python*X.Y*
scripts	*prefix*/bin
data	*prefix*

nt

Caminho	Diretório de instalação
stdlib	*prefix*\Lib
platstdlib	*prefix*\Lib
platlib	*prefix*\Lib\site-packages
purelib	*prefix*\Lib\site-packages
include	*prefix*\Include
platinclude	*prefix*\Include
scripts	*prefix*\Scripts
data	*prefix*

29.3.6 Funções de caminho de instalação

sysconfig fornece algumas funções para determinar esses caminhos de instalação.

sysconfig.**get_scheme_names**()

Retorna uma tupla contendo todos os esquemas atualmente suportados em *sysconfig*.

sysconfig.**get_default_scheme**()

Retorna o nome do esquema padrão para a plataforma atual.

Adicionado na versão 3.10: Esta função era chamada de `_get_default_scheme()` e considerada um detalhe de implementação.

Alterado na versão 3.11: Quando o Python é executado em um ambiente virtual, o esquema *venv* é retornado.

sysconfig.**get_preferred_scheme**(*key*)

Retorna um nome de esquema preferido para um layout de instalação especificado por *key*.

key deve ser `"prefix"`, `"home"` ou `"user"`.

O valor de retorno é um nome de esquema listado em *get_scheme_names()*. Ele pode ser passado para as funções *sysconfig* que recebem um argumento *scheme*, como *get_paths()*.

Adicionado na versão 3.10.

Alterado na versão 3.11: Quando o Python é executado em um ambiente virtual e `key="prefix"`, o esquema *venv* é retornado.

sysconfig.**_get_preferred_schemes**()

Retorna um dict contendo nomes de esquema preferidos na plataforma atual. Os implementadores e redistribuidores do Python podem adicionar seus esquemas preferidos ao valor global de nível de módulo `_INSTALL_SCHEMES` e modificar esta função para retornar esses nomes de esquema. Por exemplo, fornecer esquemas diferentes para os gerenciadores de pacotes de sistema e idioma usarem, de modo que os pacotes instalados por um não se misturem com os do outro.

Os usuários finais não devem usar esta função, mas *get_default_scheme()* e *get_preferred_scheme()*.

Adicionado na versão 3.10.

sysconfig.**get_path_names**()

Retorna uma tupla contendo todos os nomes de caminhos atualmente suportados em *sysconfig*.

sysconfig.**get_path**(*name*[, *scheme*[, *vars*[, *expand*]]])

Retorna um caminho de instalação correspondente ao caminho *name*, do esquema de instalação denominado *scheme*.

name deve ser um valor da lista retornada por *get_path_names()*.

sysconfig armazena os caminhos de instalação correspondentes a cada nome de caminho, para cada plataforma, com variáveis a serem expandidas. Por exemplo, o caminho *stdlib* para o esquema *nt* é: `{base}/Lib`.

get_path() usará as variáveis retornadas por *get_config_vars()* para expandir o caminho. Todas as variáveis possuem valores padrão para cada plataforma, portanto, pode-se chamar esta função e obter o valor padrão.

Se *scheme* for fornecido, deve ser um valor da lista retornada por *get_scheme_names()*. Caso contrário, o esquema padrão para a plataforma atual é usado.

Se *vars* for fornecido, deve ser um dicionário de variáveis que atualizará o dicionário retornado por *get_config_vars()*.

Se *expand* for definido como `False`, o caminho não será expandido usando as variáveis.

Se *name* não for encontrado, levanta uma *KeyError*.

sysconfig.**get_paths**([*scheme*[, *vars*[, *expand*]]])

Retorna um dicionário contendo todos os caminhos de instalação correspondentes a um esquema de instalação. Veja *get_path()* para mais informações.

Se *esquema* não for fornecido, usará o esquema padrão para a plataforma atual.

Se *vars* for fornecido, deve ser um dicionário de variáveis que atualizará o dicionário usado para expandir os caminhos.

Se *expand* for definido como falso, os caminhos não serão expandidos.

Se *scheme* não for um esquema existente, *get_paths()* vai levantar uma *KeyError*.

29.3.7 Outras funções

sysconfig.**get_python_version**()

Retorna o número da versão Python `MAJOR.MINOR` como uma string. Semelhante a `'%d.%d' % sys.version_info[:2]`.

sysconfig.**get_platform**()

Retorna uma string que identifica a plataforma atual.

Isso é usado principalmente para distinguir diretórios de construção específicos da plataforma e distribuições construídas específicas da plataforma. Geralmente inclui o nome e a versão do sistema operacional e a arquitetura (conforme fornecido por *os.uname()*), embora as informações exatas incluídas dependam do sistema operacional; por exemplo, no Linux, a versão do kernel não é particularmente importante.

Exemplos de valores retornados:

- linux-i586
- linux-alpha (?)
- solaris-2.6-sun4u

Windows vai retornar um entre:

- win-amd64 (Windows 64 bits no AMD64, isto é, x86_64, Intel64 e EM64T)
- win-arm64 (Windows 64 bits no ARM64, isto é, AArch64)
- win32 (todos os demais - especificamente, sys.platform é retornado)

macOS pode retornar:

- macosx-10.6-ppc
- macosx-10.4-ppc64
- macosx-10.3-i386
- macosx-10.4-fat

Para outras plataformas não POSIX, é retornado apenas *sys.platform*.

sysconfig.**is_python_build**()

Retorna `True` se o interpretador Python em execução foi construído a partir do código-fonte e está sendo executado a partir de seu local de construção, e não de um local resultante de, por exemplo, executando `make install` ou instalando através de um instalador binário.

sysconfig.**parse_config_h**(*fp*[, *vars*])

Analisa um arquivo no estilo `config.h`.

fp é um objeto arquivo ou similar apontando para o arquivo `config.h` ou similar.

Um dicionário contendo pares nome/valor é retornado. Se um dicionário opcional for passado como segundo argumento, ele será usado no lugar de um novo dicionário e atualizado com os valores lidos no arquivo.

sysconfig.**get_config_h_filename**()

Retorna o caminho do `pyconfig.h`.

sysconfig.**get_makefile_filename**()

Retorna o caminho do `Makefile`.

29.3.8 Uso na linha de comando

Você pode usar *sysconfig* como um script com a opção *-m* do Python:

```
$ python -m sysconfig
Platform: "macosx-10.4-i386"
Python version: "3.2"
Current installation scheme: "posix_prefix"

Paths:
        data = "/usr/local"
        include = "/Users/tarek/Dev/svn.python.org/py3k/Include"
        platinclude = "."
        platlib = "/usr/local/lib/python3.2/site-packages"
        platstdlib = "/usr/local/lib/python3.2"
        purelib = "/usr/local/lib/python3.2/site-packages"
        scripts = "/usr/local/bin"
        stdlib = "/usr/local/lib/python3.2"

Variables:
        AC_APPLE_UNIVERSAL_BUILD = "0"
        AIX_GENUINE_CPLUSPLUS = "0"
        AR = "ar"
        ARFLAGS = "rc"
        ...
```

Esta chamada imprimirá na saída padrão as informações retornadas por *get_platform()*, *get_python_version()*, *get_path()* e *get_config_vars()*.

29.4 builtins — Objetos embutidos

Este módulo fornece acesso direto a todos os identificadores embutidos do Python; Por exemplo, builtins.open é o nome completo para a função embutida *open()*.

Este módulo normalmente não é acessado explicitamente pela maioria dos aplicativos, mas pode ser útil em módulos que fornecem objetos com o mesmo nome como um valor embutido, mas em que o objeto embutido desse nome também é necessário. Por exemplo, em um módulo que deseja implementar uma função *open()* que envolve o embutido *open()*, este módulo pode ser usado diretamente:

```
import builtins

def open(path):
    f = builtins.open(path, 'r')
    return UpperCaser(f)

class UpperCaser:
    '''Envoltório em volta de um arquivo que converte saída para maiúsculo.'''

    def __init__(self, f):
        self._f = f

    def read(self, count=-1):
        return self._f.read(count).upper()

    # ...
```

Como um detalhe de implementação, a maioria dos módulos tem o nome __builtins__ disponibilizados como parte de seus globais. O valor de __builtins__ normalmente, este é o módulo ou o valor desse módulo __dict__ atributo. Uma vez que este é um detalhe de implementação, ele não pode ser usado por implementações alternativas do Python.

Ver também

- *Constantes embutidas*
- *Exceções embutidas*
- *Funções embutidas*
- *Tipos embutidos*

29.5 __main__ — Ambiente de código principal

Em Python, o nome especial __main__ é usado em duas construções importantes:

1. O nome do ambiente principal do programa, que pode ser verificado usando a expressão __name__ == '__main__'; e
2. o arquivo __main__.py em pacotes Python.

Ambas as formas estão relacionadas aos módulos Python; como os usuários interagem com eles e como eles interagem entre si. Eles serão explicados em detalhes abaixo. Se você ainda não conhece módulos Python, veja a seção tut--modules para uma introdução.

29.5.1 `__name__ == '__main__'`

Quando um pacote ou módulo Python é importado, `__name__` é definido como o nome do módulo. Normalmente, este é o nome do próprio arquivo Python sem a extensão `.py`:

```
>>> import configparser
>>> configparser.__name__
'configparser'
```

Caso o arquivo seja parte de um pacote, `__name__` também incluirá o nome da pasta raiz do pacote.

```
>>> from concurrent.futures import process
>>> process.__name__
'concurrent.futures.process'
```

Porém, se o módulo for executado como o código principal, `__name__` passa a ser definido como a string `'__main__'`

O que é o "ambiente de código principal"?

`__main__` é o nome do ambiente principal no qual o código é executado. "Ambiente de código principal" é o primeiro módulo Python definido pelo usuário que começa a ser executado. É considerado principal porque ele importa todos os outros módulos que o programa precisa. As vezes, "ambiente principal" também pode ser chamado de "ponto de entrada" da aplicação.

O ambiente de código principal pode ser:

- o escopo de um prompt de comando interativo:

```
>>> __name__
'__main__'
```

- o módulo Python passado ao interpretador do Python como um argumento correspondente ao nome do arquivo:

```
$ python helloworld.py
Hello, world!
```

- o módulo ou pacote Python passado ao interpretador do Python com o argumento `-m`:

```
$ python -m tarfile
usage: tarfile.py [-h] [-v] (...)
```

- código Python lido pelo interpretador através da entrada padrão de linha de comando:

```
$ echo "import this" | python
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
...
```

- código Python passado ao interpretador do Python com o argumento `-c`:

```
$ python -c "import this"
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
...
```

Em cada uma destas situações, a variável especial `__name__` passa a ser definida como `'__main__'`.

Como resultado, um módulo pode saber se está ou não sendo executado no ambiente principal verificando seu próprio `__name__`, que habilita um termo comum para executar código condicionalmente quando o módulo não é inicializado a partir de uma instrução de importação:

```
if __name__ == '__main__':
    # Executa quando o módulo não é inicializado por uma instrução de importação.
    ...
```

Ver também

Para uma visão mais detalhada sobre como `__name__` é definido em todas as situações, veja a sessão tut-modules.

Uso idiomático

Alguns módulos contém códigos que são específicos para serem usados como scripts, como análise de argumentos através de linha de comando ou leitura de dados da entrada padrão. Se um módulo como o citado for importado em um outro módulo diferente, por exemplo em testes unitários, o código do script também seria executado de forma indesejada.

É aqui onde o uso do trecho de código `if __name__ == '__main__'` revela-se útil. Os códigos dentro desta condicional não rodarão ao não ser que o módulo seja executado através do ambiente principal.

Colocar o mínimo de instruções possível no bloco abaixo de `if __name__ == '__main__'` pode melhorar a clareza e a precisão do código. Na maioria das vezes, uma função chamada de `main` encapsula o comportamento principal do programa:

```
# echo.py

import shlex
import sys

def echo(phrase: str) -> None:
   """Um invólucro fictício sobre print."""
   # com propósito de demonstração, imagine que existe
   # uma lógica reutilizável valiosa nesta função
   print(phrase)

def main() -> int:
    """Ecoa os argumentos de entrada para a saída padrão."""
    phrase = shlex.join(sys.argv)
    echo(phrase)
    return 0

if __name__ == '__main__':
    sys.exit(main())  # a próxima seção explica o uso de sys.exit
```

Repare quem se o módulo, ao invés de ter encapsulado o código dentro da função `main`, fosse colocado direto dentro do bloco `if __name__ == '__main__'`, a variável `phrase` seria global para todo o módulo. Isto é suscetível à erros pois outras funções dentro do módulo poderiam inadvertidamente usar a variável global ao invés da local. A função `main` resolve este problema.

O uso da função `main` tem o benefício adicional de a própria função `echo` ser isolada e importável em outro lugar. Quando `echo.py` é importado, as funções `echo` e `main` serão definidas, mas nenhuma delas será chamada por conta do bloco `__name__ != '__main__'`.

Considerações sobre pacotes

`main` são funções frequentemente usadas para criar ferramentas de linha de comando especificando-as como pontos de entrada para scripts de console. Quando isto é feito, pip insere a chamada da função em um modelo de script, onde o valor de retorno de `main` é passado para *`sys.exit()`*. Por exemplo:

```
sys.exit(main())
```

Uma vez que a chamada à `main` está embutida dentro de *`sys.exit()`*, a expectativa é que a sua função retorne somente valores aceitável para *`sys.exit()`*; normalmente um inteiro ou `None` (retornado de forma implícita, caso a sua função não tenha uma instrução de retorno).

Seguindo proativamente essa convenção, nosso módulo terá o mesmo comportamento quando executado diretamente (ou seja, `python echo.py`) como terá também se posteriormente criarmos um pacote como um ponto de entrada de script de console em um pacote instalável via pip.

Em particular, tenha cuidado ao retornar strings de sua função `main`. *`sys.exit()`* interpretará um argumento de string como uma mensagem de falha, então seu programa terá um código de saída `1`, indicando falha, e a string será escrita em *`sys.stderr`*. O exemplo anterior de `echo.py` exemplifica o uso da convenção `sys.exit(main())`.

Ver também

O Guia de Usuário para Empacotamento de Python contém uma coleção de tutoriais e referências sobre como distribuir e instalar pacotes Python com ferramentas modernas.

29.5.2 `__main__.py` em pacotes Python

Se você não estiver familiarizado com pacotes Python, veja a seção tut-packages do tutorial. Mais comumente, o arquivo `__main__.py` é usado para fornecer uma interface de linha de comando para um pacote. Considere o seguinte pacote hipotético, "bandclass":

```
bandclass
  ├── __init__.py
  ├── __main__.py
  └── student.py
```

`__main__.py` será executado quando o próprio pacote for invocado diretamente da linha de comando usando o sinalizador `-m`. Por exemplo:

```
$ python -m bandclass
```

Este comando fará com que `__main__.py` seja executado. Como você utiliza esse mecanismo dependerá da natureza do pacote que você está escrevendo, mas neste caso hipotético, pode fazer sentido permitir que o professor procure alunos:

```
# bandclass/__main__.py

import sys
from .student import search_students

student_name = sys.argv[1] if len(sys.argv) >= 2 else ''
print(f'Estudante encontrado(a): {search_students(student_name)}')
```

Observe que `from .student import search_students` é um exemplo de importação relativa. Esse estilo de importação pode ser usado ao fazer referência a módulos em um pacote. Para mais detalhes, veja intra-package--references na seção tut-modules do tutorial.

Uso idiomático

O conteúdo do `__main__.py` normalmente não é protegido por um bloco `if __name__ == '__main__'`. Em vez disso, esses arquivos são mantidos curtos e importa funções para serem executados a partir de outros módulos. Esses outros módulos podem então ter suas unidades facilmente testadas e são adequadamente reutilizáveis.

Se usado, o bloco condicional `if __name__ == '__main__'` ainda funcionará como esperado para o arquivo `__main__.py` dentro de um pacote, pois seu atributo `__name__` incluirá o caminho do pacote se importado:

```
>>> import asyncio.__main__
>>> asyncio.__main__.__name__
'asyncio.__main__'
```

Porém, isso não funcionará para arquivos `__main__.py` no diretório raiz de um arquivo `.zip`. Portanto, para consistência, é preferível um `__main__.py` mínimo sem uma verificação de `__name__`.

Ver também

Veja *venv*, da biblioteca padrão, para um exemplo de pacote com um `__main__.py` minimalista. Ele não contém um bloco `if __name__ == '__main__'`. Você pode invocá-lo com `python -m venv [directory]`.

Veja *runpy* para mais detalhes sobre o sinalizador `-m` para o executável do interpretador.

Veja *zipapp* para saber como executar aplicativos compactados como arquivos *.zip*. Nesse caso, o Python procura um arquivo `__main__.py` no diretório raiz do arquivo.

29.5.3 `import __main__`

Independentemente do módulo com o qual um programa Python foi iniciado, outros módulos executados no mesmo programa podem importar o escopo do ambiente principal (*espaço de nomes*) importando o módulo `__main__`. Isso não faz a importação de um arquivo `__main__.py`, mas sim qualquer módulo que recebeu o nome especial `'__main__'`.

Aqui está um módulo de exemplo que consome o espaço de nomes `__main__`:

```
# namely.py

import __main__

def did_user_define_their_name():
    return 'my_name' in dir(__main__)

def print_user_name():
    if not did_user_define_their_name():
        raise ValueError('Defina a variável `my_name`!')

    print(__main__.my_name)
```

Exemplo de uso deste módulo pode ser como abaixo:

```
# start.py

import sys

from namely import print_user_name

# my_name = "Dinsdale"

def main():
```

(continua na próxima página)

(continuação da página anterior)

```
    try:
        print_user_name()
    except ValueError as ve:
        return str(ve)

if __name__ == "__main__":
    sys.exit(main())
```

Agora, se iniciarmos nosso programa, o resultado seria assim:

```
$ python start.py
Defina a variável `my_name`!
```

O código de saída do programa seria 1, indicando um erro. Descomentar a linha com `my_name = "Dinsdale"` corrige o programa e agora ele sai com o código de status 0, indicando sucesso:

```
$ python start.py
Dinsdale
```

Observe que a importação de `__main__` não causa nenhum problema com a execução involuntária de código principal destinado ao uso de script que é colocado no bloco `if __name__ == "__main__"` do módulo `start`. Por que isso funciona?

O Python insere um módulo `__main__` vazio em *`sys.modules`* na inicialização do interpretador e o preenche executando o código principal. Em nosso exemplo, este é o módulo `start` que executa linha por linha e importa `namely`. Por sua vez, `namely` importa `__main__` (que é realmente `start`). Isso é um ciclo de importação! Felizmente, como o módulo `__main__` parcialmente preenchido está presente em *`sys.modules`*, o Python o passa para `namely`. Veja Considerações especiais sobre __main__ na referência do sistema de importação para detalhes sobre como isso funciona.

O REPL do Python é outro exemplo de um "ambiente principal", então qualquer coisa definida no REPL se torna parte do escopo do `__main__`:

```
>>> import namely
>>> namely.did_user_define_their_name()
False
>>> namely.print_user_name()
Traceback (most recent call last):
...
ValueError: Defina a variável `my_name`!
>>> my_name = 'Jabberwocky'
>>> namely.did_user_define_their_name()
True
>>> namely.print_user_name()
Jabberwocky
```

O escopo `__main__` é usado na implementação de *`pdb`* e *`rlcompleter`*.

29.6 `warnings` — Controle de avisos

Código-fonte: Lib/warnings.py

As mensagens de aviso são normalmente emitidas em situações em que é útil alertar o usuário sobre alguma condição em um programa, onde essa condição (normalmente) não garante o levantamento de uma exceção e o encerramento do programa. Por exemplo, pode-se querer emitir um aviso quando um programa usa um módulo obsoleto.

Os programadores Python emitem avisos chamando a função *warn()* definida neste módulo. (Os programadores C usam `PyErr_WarnEx()`; veja exceptionhandling para detalhes).

Mensagens de aviso são normalmente escritas no *sys.stderr*, mas sua disposição pode ser alterada de forma flexível, desde ignorar todos os avisos até transformá-los em exceções. A disposição dos avisos pode variar de acordo com *categoria de aviso*, o texto da mensagem de aviso e o local de origem onde ela é emitida. As repetições de um aviso específico para o mesmo local de origem são normalmente suprimidas.

Existem duas etapas no controle de avisos: primeiro, cada vez que um aviso é emitido, é feita uma determinação se uma mensagem deve ser emitida ou não; a seguir, se uma mensagem deve ser emitida, ela é formatada e impressa usando um gancho configurável pelo usuário.

A determinação de emitir ou não uma mensagem de aviso é controlada pelo *filtro de aviso*, que é uma sequência de regras e ações correspondentes. As regras podem ser adicionadas ao filtro chamando *filterwarnings()* e redefinidas para seu estado padrão chamando *resetwarnings()*.

A exibição de mensagens de aviso é feita chamando *showwarning()*, que pode ser substituída; a implementação padrão desta função formata a mensagem chamando *formatwarning()*, que também está disponível para uso por implementações personalizadas.

Ver também

logging.captureWarnings() permite que você manipule todos os avisos com a infraestrutura de registro padrão.

29.6.1 Categorias de avisos

Existem várias exceções embutidas que representam categorias de aviso. Essa categorização é útil para filtrar grupos de avisos.

Embora sejam tecnicamente *exceções embutidas*, elas são documentadas aqui, porque conceitualmente pertencem ao mecanismo de avisos.

O código do usuário pode definir categorias de aviso adicionais criando uma subclasse de uma das categorias de aviso padrão. Uma categoria de aviso deve ser sempre uma subclasse da classe *Warning*.

As seguintes classes de categorias de avisos estão definidas atualmente:

Classe	Descrição
Warning	Esta é a classe base de todas as classes de categoria de aviso. É uma subclasse de *Exception*.
UserWarning	A categoria padrão para *warn()*.
DeprecationWarning	Categoria base para avisos sobre recursos descontinuados quando esses avisos são destinados a outros desenvolvedores Python (ignorado por padrão, a menos que acionado por código em __main__).
SyntaxWarning	Base category for warnings about dubious syntactic features (typically emitted when compiling Python source code, and hence may not be suppressed by runtime filters)
RuntimeWarning	Categoria base para avisos sobre recursos duvidosos de tempo de execução.
FutureWarning	Categoria base para avisos sobre recursos descontinuados quando esses avisos se destinam a usuários finais de aplicações escritas em Python.
PendingDeprecationWarning	Categoria base para avisos sobre recursos que serão descontinuados no futuro (ignorados por padrão).
ImportWarning	Categoria base para avisos acionados durante o processo de importação de um módulo (ignorado por padrão).
UnicodeWarning	Categoria base para avisos relacionados a Unicode.
BytesWarning	Categoria base para avisos relacionados a *bytes* e *bytearray*.
ResourceWarning	Categoria base para avisos relacionados a uso de recursos (ignorado por padrão).

Alterado na versão 3.7: Anteriormente, *DeprecationWarning* e *FutureWarning* eram diferenciadas com base em se um recurso estava sendo removido completamente ou mudando seu comportamento. Elas agora são diferenciadas com base em seu público-alvo e na maneira como são tratadas pelos filtros de avisos padrão.

29.6.2 O filtro de avisos

O filtro de avisos controla se os avisos são ignorados, exibidos ou transformados em erros (levantando uma exceção).

Conceitualmente, o filtro de avisos mantém uma lista ordenada de especificações de filtro; qualquer aviso específico é comparado com cada especificação de filtro na lista, por sua vez, até que uma correspondência seja encontrada; o filtro determina a disposição da correspondência. Cada entrada é uma tupla no formato (*action*, *message*, *category*, *module*, *lineno*), sendo:

- *action* é uma das seguintes strings:

Valor	Disposição
"default"	exibe a primeira ocorrência de avisos correspondentes para cada local (módulo + número da linha) onde o aviso é emitido
"error"	transforma avisos correspondentes em exceções
"ignore"	nunca exibe avisos correspondentes
"always"	sempre exibe avisos correspondentes
"all"	alias to "always"
"module"	exibe a primeira ocorrência de avisos correspondentes para cada módulo onde o aviso é emitido (independentemente do número da linha)
"once"	exibe apenas a primeira ocorrência de avisos correspondentes, independentemente da localização

- *message* é uma string contendo uma expressão regular que o início da mensagem de aviso deve corresponder, sem distinção entre maiúsculas e minúsculas. Em `-W` e `PYTHONWARNINGS`, *message* é uma string literal que o início da mensagem de aviso deve conter (sem distinção entre maiúsculas e minúsculas), ignorando qualquer espaço em branco no início ou no fim de *message*.

- *category* é uma classe (uma subclasse de `Warning`) da qual a categoria de aviso deve ser uma subclasse para corresponder.
- *module* é uma string contendo uma expressão regular que o início do nome do módulo totalmente qualificado deve corresponder, diferenciando maiúsculas de minúsculas. Em `-W` e `PYTHONWARNINGS`, *module* é uma string literal que o nome do módulo totalmente qualificado deve ser igual a (diferenciando maiúsculas de minúsculas), ignorando qualquer espaço em branco no início ou no fim de *module*.
- *lineno* é um número inteiro que deve corresponder ao número da linha onde ocorreu o aviso, ou `0` para corresponder a todos os números de linha.

Como a classe `Warning` é derivada da classe embutida `Exception`, para transformar um aviso em um erro, simplesmente levantamos `category(message)`.

Se um aviso for relatado e não corresponder a nenhum filtro registrado, a ação "padrão" será aplicada (daí seu nome).

Critérios de supressão de avisos repetidos

Os filtros que suprimem avisos repetidos aplicam os seguintes critérios para determinar se um aviso é considerado uma repetição:

- `"default"`: Um aviso é considerado uma repetição somente se (*message*, *category*, *module*, *lineno*) forem todos iguais.
- `"module"`: Um aviso é considerado uma repetição se (*message*, *category*, *module*) forem os mesmos, ignorando o número da linha.
- `"once"`: Um aviso é considerado uma repetição se (*message*, *category*) forem os mesmos, ignorando o módulo e o número da linha.

Descrevendo filtros de aviso

The warnings filter is initialized by `-W` options passed to the Python interpreter command line and the `PYTHONWARNINGS` environment variable. The interpreter saves the arguments for all supplied entries without interpretation in `sys.warnoptions`; the `warnings` module parses these when it is first imported (invalid options are ignored, after printing a message to `sys.stderr`).

Individual warnings filters are specified as a sequence of fields separated by colons:

```
action:message:category:module:line
```

The meaning of each of these fields is as described in *O filtro de avisos*. When listing multiple filters on a single line (as for `PYTHONWARNINGS`), the individual filters are separated by commas and the filters listed later take precedence over those listed before them (as they're applied left-to-right, and the most recently applied filters take precedence over earlier ones).

Commonly used warning filters apply to either all warnings, warnings in a particular category, or warnings raised by particular modules or packages. Some examples:

```
default                       # Show all warnings (even those ignored by default)
ignore                        # Ignore all warnings
error                         # Convert all warnings to errors
error::ResourceWarning        # Treat ResourceWarning messages as errors
default::DeprecationWarning   # Show DeprecationWarning messages
ignore,default:::mymodule     # Only report warnings triggered by "mymodule"
error:::mymodule              # Convert warnings to errors in "mymodule"
```

Filtro de avisos padrão

By default, Python installs several warning filters, which can be overridden by the `-W` command-line option, the `PYTHONWARNINGS` environment variable and calls to `filterwarnings()`.

In regular release builds, the default warning filter has the following entries (in order of precedence):

```
default::DeprecationWarning:__main__
ignore::DeprecationWarning
ignore::PendingDeprecationWarning
ignore::ImportWarning
ignore::ResourceWarning
```

In a debug build, the list of default warning filters is empty.

Alterado na versão 3.2: *DeprecationWarning* is now ignored by default in addition to *PendingDeprecationWarning*.

Alterado na versão 3.7: *DeprecationWarning* is once again shown by default when triggered directly by code in `__main__`.

Alterado na versão 3.7: *BytesWarning* no longer appears in the default filter list and is instead configured via *sys.warnoptions* when `-b` is specified twice.

Overriding the default filter

Developers of applications written in Python may wish to hide *all* Python level warnings from their users by default, and only display them when running tests or otherwise working on the application. The *sys.warnoptions* attribute used to pass filter configurations to the interpreter can be used as a marker to indicate whether or not warnings should be disabled:

```
import sys

if not sys.warnoptions:
    import warnings
    warnings.simplefilter("ignore")
```

Developers of test runners for Python code are advised to instead ensure that *all* warnings are displayed by default for the code under test, using code like:

```
import sys

if not sys.warnoptions:
    import os, warnings
    warnings.simplefilter("default") # Change the filter in this process
    os.environ["PYTHONWARNINGS"] = "default" # Also affect subprocesses
```

Finally, developers of interactive shells that run user code in a namespace other than `__main__` are advised to ensure that *DeprecationWarning* messages are made visible by default, using code like the following (where `user_ns` is the module used to execute code entered interactively):

```
import warnings
warnings.filterwarnings("default", category=DeprecationWarning,
                                   module=user_ns.get("__name__"))
```

29.6.3 Temporarily Suppressing Warnings

If you are using code that you know will raise a warning, such as a deprecated function, but do not want to see the warning (even when warnings have been explicitly configured via the command line), then it is possible to suppress the warning using the *catch_warnings* context manager:

```
import warnings

def fxn():
    warnings.warn("deprecated", DeprecationWarning)
```

(continua na próxima página)

(continuação da página anterior)

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    fxn()
```

While within the context manager all warnings will simply be ignored. This allows you to use known-deprecated code without having to see the warning while not suppressing the warning for other code that might not be aware of its use of deprecated code.

> **Nota**
>
> See *Concurrent safety of Context Managers* for details on the concurrency-safety of the `catch_warnings` context manager when used in programs using multiple threads or async functions.

29.6.4 Testing Warnings

To test warnings raised by code, use the `catch_warnings` context manager. With it you can temporarily mutate the warnings filter to facilitate your testing. For instance, do the following to capture all raised warnings to check:

```
import warnings

def fxn():
    warnings.warn("deprecated", DeprecationWarning)

with warnings.catch_warnings(record=True) as w:
    # Cause all warnings to always be triggered.
    warnings.simplefilter("always")
    # Trigger a warning.
    fxn()
    # Verify some things
    assert len(w) == 1
    assert issubclass(w[-1].category, DeprecationWarning)
    assert "deprecated" in str(w[-1].message)
```

One can also cause all warnings to be exceptions by using `error` instead of `always`. One thing to be aware of is that if a warning has already been raised because of a `once`/`default` rule, then no matter what filters are set the warning will not be seen again unless the warnings registry related to the warning has been cleared.

Once the context manager exits, the warnings filter is restored to its state when the context was entered. This prevents tests from changing the warnings filter in unexpected ways between tests and leading to indeterminate test results.

> **Nota**
>
> See *Concurrent safety of Context Managers* for details on the concurrency-safety of the `catch_warnings` context manager when used in programs using multiple threads or async functions.

When testing multiple operations that raise the same kind of warning, it is important to test them in a manner that confirms each operation is raising a new warning (e.g. set warnings to be raised as exceptions and check the operations raise exceptions, check that the length of the warning list continues to increase after each operation, or else delete the previous entries from the warnings list before each new operation).

29.6.5 Updating Code For New Versions of Dependencies

Warning categories that are primarily of interest to Python developers (rather than end users of applications written in Python) are ignored by default.

Notably, this "ignored by default" list includes *DeprecationWarning* (for every module except `__main__`), which means developers should make sure to test their code with typically ignored warnings made visible in order to receive timely notifications of future breaking API changes (whether in the standard library or third party packages).

In the ideal case, the code will have a suitable test suite, and the test runner will take care of implicitly enabling all warnings when running tests (the test runner provided by the *unittest* module does this).

In less ideal cases, applications can be checked for use of deprecated interfaces by passing `-Wd` to the Python interpreter (this is shorthand for `-W default`) or setting `PYTHONWARNINGS=default` in the environment. This enables default handling for all warnings, including those that are ignored by default. To change what action is taken for encountered warnings you can change what argument is passed to `-W` (e.g. `-W error`). See the `-W` flag for more details on what is possible.

29.6.6 Funções disponíveis

`warnings.`**`warn`**(*message*, *category=None*, *stacklevel=1*, *source=None*, ***, *skip_file_prefixes=()*)

Issue a warning, or maybe ignore it or raise an exception. The *category* argument, if given, must be a *warning category class*; it defaults to *UserWarning*. Alternatively, *message* can be a *Warning* instance, in which case *category* will be ignored and `message.__class__` will be used. In this case, the message text will be `str(message)`. This function raises an exception if the particular warning issued is changed into an error by the *warnings filter*. The *stacklevel* argument can be used by wrapper functions written in Python, like this:

```
def deprecated_api(message):
    warnings.warn(message, DeprecationWarning, stacklevel=2)
```

This makes the warning refer to `deprecated_api`'s caller, rather than to the source of `deprecated_api` itself (since the latter would defeat the purpose of the warning message).

The *skip_file_prefixes* keyword argument can be used to indicate which stack frames are ignored when counting stack levels. This can be useful when you want the warning to always appear at call sites outside of a package when a constant *stacklevel* does not fit all call paths or is otherwise challenging to maintain. If supplied, it must be a tuple of strings. When prefixes are supplied, stacklevel is implicitly overridden to be `max(2, stacklevel)`. To cause a warning to be attributed to the caller from outside of the current package you might write:

```
# example/lower.py
_warn_skips = (os.path.dirname(__file__),)

def one_way(r_luxury_yacht=None, t_wobbler_mangrove=None):
    if r_luxury_yacht:
        warnings.warn("Please migrate to t_wobbler_mangrove=.",
                      skip_file_prefixes=_warn_skips)

# example/higher.py
from . import lower

def another_way(**kw):
    lower.one_way(**kw)
```

This makes the warning refer to both the `example.lower.one_way()` and `example.higher.another_way()` call sites only from calling code living outside of `example` package.

source, if supplied, is the destroyed object which emitted a *ResourceWarning*.

Alterado na versão 3.6: Adicionado o parâmetro *source*.

Alterado na versão 3.12: Added *skip_file_prefixes*.

warnings.**warn_explicit**(*message*, *category*, *filename*, *lineno*, *module=None*, *registry=None*, *module_globals=None*, *source=None*)

This is a low-level interface to the functionality of `warn()`, passing in explicitly the message, category, filename and line number, and optionally the module name and the registry (which should be the `__warningregistry__` dictionary of the module). The module name defaults to the filename with `.py` stripped; if no registry is passed, the warning is never suppressed. *message* must be a string and *category* a subclass of `Warning` or *message* may be a `Warning` instance, in which case *category* will be ignored.

module_globals, if supplied, should be the global namespace in use by the code for which the warning is issued. (This argument is used to support displaying source for modules found in zipfiles or other non-filesystem import sources).

source, if supplied, is the destroyed object which emitted a `ResourceWarning`.

Alterado na versão 3.6: Adiciona o parâmetro *source*.

warnings.**showwarning**(*message*, *category*, *filename*, *lineno*, *file=None*, *line=None*)

Write a warning to a file. The default implementation calls `formatwarning(message, category, filename, lineno, line)` and writes the resulting string to *file*, which defaults to `sys.stderr`. You may replace this function with any callable by assigning to `warnings.showwarning`. *line* is a line of source code to be included in the warning message; if *line* is not supplied, `showwarning()` will try to read the line specified by *filename* and *lineno*.

warnings.**formatwarning**(*message*, *category*, *filename*, *lineno*, *line=None*)

Format a warning the standard way. This returns a string which may contain embedded newlines and ends in a newline. *line* is a line of source code to be included in the warning message; if *line* is not supplied, `formatwarning()` will try to read the line specified by *filename* and *lineno*.

warnings.**filterwarnings**(*action*, *message=''*, *category=Warning*, *module=''*, *lineno=0*, *append=False*)

Insert an entry into the list of *warnings filter specifications*. The entry is inserted at the front by default; if *append* is true, it is inserted at the end. This checks the types of the arguments, compiles the *message* and *module* regular expressions, and inserts them as a tuple in the list of warnings filters. Entries closer to the front of the list override entries later in the list, if both match a particular warning. Omitted arguments default to a value that matches everything.

warnings.**simplefilter**(*action*, *category=Warning*, *lineno=0*, *append=False*)

Insert a simple entry into the list of *warnings filter specifications*. The meaning of the function parameters is as for `filterwarnings()`, but regular expressions are not needed as the filter inserted always matches any message in any module as long as the category and line number match.

warnings.**resetwarnings**()

Reset the warnings filter. This discards the effect of all previous calls to `filterwarnings()`, including that of the `-W` command line options and calls to `simplefilter()`.

@warnings.**deprecated**(*msg*, ***, *category=DeprecationWarning*, *stacklevel=1*)

Decorator to indicate that a class, function or overload is deprecated.

When this decorator is applied to an object, deprecation warnings may be emitted at runtime when the object is used. *static type checkers* will also generate a diagnostic on usage of the deprecated object.

Uso:

```
from warnings import deprecated
from typing import overload

@deprecated("Use B instead")
class A:
    pass

@deprecated("Use g instead")
def f():
```

(continua na próxima página)

(continuação da página anterior)

```
    pass

@overload
@deprecated("int support is deprecated")
def g(x: int) -> int: ...
@overload
def g(x: str) -> int: ...
```

The warning specified by *category* will be emitted at runtime on use of deprecated objects. For functions, that happens on calls; for classes, on instantiation and on creation of subclasses. If the *category* is `None`, no warning is emitted at runtime. The *stacklevel* determines where the warning is emitted. If it is `1` (the default), the warning is emitted at the direct caller of the deprecated object; if it is higher, it is emitted further up the stack. Static type checker behavior is not affected by the *category* and *stacklevel* arguments.

The deprecation message passed to the decorator is saved in the `__deprecated__` attribute on the decorated object. If applied to an overload, the decorator must be after the *`@overload`* decorator for the attribute to exist on the overload as returned by *`typing.get_overloads()`*.

Adicionado na versão 3.13: See **PEP 702**.

29.6.7 Gerenciadores de contexto disponíveis

class warnings.**catch_warnings**(*, *record=False*, *module=None*, *action=None*, *category=Warning*, *lineno=0*, *append=False*)

A context manager that copies and, upon exit, restores the warnings filter and the *`showwarning()`* function. If the *record* argument is *`False`* (the default) the context manager returns *`None`* on entry. If *record* is *`True`*, a list is returned that is progressively populated with objects as seen by a custom *`showwarning()`* function (which also suppresses output to `sys.stdout`). Each object in the list has attributes with the same names as the arguments to *`showwarning()`*.

The *module* argument takes a module that will be used instead of the module returned when you import *`warnings`* whose filter will be protected. This argument exists primarily for testing the *`warnings`* module itself.

If the *action* argument is not `None`, the remaining arguments are passed to *`simplefilter()`* as if it were called immediately on entering the context.

See *O filtro de avisos* for the meaning of the *category* and *lineno* parameters.

> **Nota**
>
> See *Concurrent safety of Context Managers* for details on the concurrency-safety of the *`catch_warnings`* context manager when used in programs using multiple threads or async functions.

Alterado na versão 3.11: Added the *action*, *category*, *lineno*, and *append* parameters.

29.6.8 Concurrent safety of Context Managers

The behavior of *`catch_warnings`* context manager depends on the *`sys.flags.context_aware_warnings`* flag. If the flag is true, the context manager behaves in a concurrent-safe fashion and otherwise not. Concurrent--safe means that it is both thread-safe and safe to use within *asyncio coroutines* and tasks. Being thread-safe means that behavior is predictable in a multi-threaded program. The flag defaults to true for free-threaded builds and false otherwise.

If the *`context_aware_warnings`* flag is false, then *`catch_warnings`* will modify the global attributes of the *`warnings`* module. This is not safe if used within a concurrent program (using multiple threads or using asyncio coroutines). For example, if two or more threads use the *`catch_warnings`* class at the same time, the behavior is undefined.

If the flag is true, *catch_warnings* will not modify global attributes and will instead use a *ContextVar* to store the newly established warning filtering state. A context variable provides thread-local storage and it makes the use of *catch_warnings* thread-safe.

The *record* parameter of the context handler also behaves differently depending on the value of the flag. When *record* is true and the flag is false, the context manager works by replacing and then later restoring the module's *showwarning()* function. That is not concurrent-safe.

When *record* is true and the flag is true, the *showwarning()* function is not replaced. Instead, the recording status is indicated by an internal property in the context variable. In this case, the *showwarning()* function will not be restored when exiting the context handler.

The *context_aware_warnings* flag can be set the `-X context_aware_warnings` command-line option or by the `PYTHON_CONTEXT_AWARE_WARNINGS` environment variable.

> **Nota**
>
> It is likely that most programs that desire thread-safe behaviour of the warnings module will also want to set the *thread_inherit_context* flag to true. That flag causes threads created by *threading.Thread* to start with a copy of the context variables from the thread starting it. When true, the context established by *catch_warnings* in one thread will also apply to new threads started by it. If false, new threads will start with an empty warnings context variable, meaning that any filtering that was established by a *catch_warnings* context manager will no longer be active.

Alterado na versão 3.14: Added the *sys.flags.context_aware_warnings* flag and the use of a context variable for *catch_warnings* if the flag is true. Previous versions of Python acted as if the flag was always set to false.

29.7 `dataclasses` — Data Classes

Código-fonte: Lib/dataclasses.py

Este módulo fornece um decorador e funções para adicionar automaticamente *métodos especiais* tais como `__init__()` e `__repr__()` a classes definidas pelo usuário. Foi originalmente descrita em **PEP 557**.

Variáveis-membro a serem usadas nesses métodos gerados são definidas usando as anotações de tipo da **PEP 526**. Por exemplo, esse código:

```
from dataclasses import dataclass

@dataclass
class InventoryItem:
    """Class for keeping track of an item in inventory."""
    name: str
    unit_price: float
    quantity_on_hand: int = 0

    def total_cost(self) -> float:
        return self.unit_price * self.quantity_on_hand
```

adicionará, entre outras coisas, um `__init__()` como esse:

```
def __init__(self, name: str, unit_price: float, quantity_on_hand: int = 0):
    self.name = name
    self.unit_price = unit_price
    self.quantity_on_hand = quantity_on_hand
```

Observe que este método é adicionado automaticamente à classe: ele não é especificado diretamente na definição `InventoryItem` mostrada acima.

Adicionado na versão 3.7.

29.7.1 Conteúdo do módulo

`@dataclasses.dataclass`(*, *init=True*, *repr=True*, *eq=True*, *order=False*, *unsafe_hash=False*, *frozen=False*, *match_args=True*, *kw_only=False*, *slots=False*, *weakref_slot=False*)

Esta função é um *decorador* que é usado para adicionar *métodos especiais* para classes, conforme descrito abaixo.

O decorador `@dataclass` examina a classe para encontrar campos (`field`. Um `field` é definido como uma variável de classe que tem uma *anotação de tipo*. Com duas exceções, descritas mais adiante, `@dataclass` não examina o tipo especificado na anotação de variável.

A ordem dos campos em todos os métodos gerados é a ordem em que eles aparecem na definição de classe.

O decorador `@dataclass` adicionará vários métodos "dunder" à classe, descritos abaixo. Se algum dos métodos adicionados já existir na classe, o comportamento dependerá do parâmetro, conforme documentado abaixo. O decorador retorna a mesma classe decorada; nenhuma nova classe é criada.

Se `@dataclass` for usado apenas como um simples decorador, sem parâmetros, ele age como se tivesse os valores padrão documentados nessa assinatura. Ou seja, esses três usos de `@dataclass` são equivalentes:

```
@dataclass
class C:
    ...

@dataclass()
class C:
    ...

@dataclass(init=True, repr=True, eq=True, order=False, unsafe_hash=False,␣
↪frozen=False,
           match_args=True, kw_only=False, slots=False, weakref_slot=False)
class C:
    ...
```

Os parâmetros do `@dataclass` são:

- *init*: Se verdadeiro (o padrão), o método `__init__()` será gerado.

  Se a classe do usuário definir `__init__()` esse parâmetro é ignorado.

- *repr*: Se verdadeiro (o padrão), um método `__repr__()` será gerado. A sequência de string de representação gerada terá o nome da classe e o nome e representação de cada campo, na ordem em que são definidos na classe. Os campos marcados como excluídos da representação não são incluídos. Por exemplo: `InventoryItem(name='widget', unit_price=3.0, quantity_on_hand=10)`.

  Se a classe do usuário já define `__repr__()` esse parâmetro é ignorado.

- *eq*: Se verdadeiro (o padrão), um método `__eq__()` será gerado. Este método compara a classe como se fosse uma tupla de campos, em ordem. Ambas as instâncias na comparação devem ser de tipo idêntico.

  Se a classe do usuário já define `__eq__()` esse parâmetro é ignorado.

- *order*: Se verdadeiro (o padrão é `False`), os métodos `__lt__()`, `__le__()`, `__gt__()` e `__ge__()` serão gerados. Comparam a classe como se fosse uma tupla de campos, em ordem. Ambas instâncias na comparação devem ser de tipo idêntico. Se *order* é verdadeiro e *eq* é falso, a exceção *`ValueError`* é levantada.

  Se a classe do usuário já define algum dentre `__lt__()`, `__le__()`, `__gt__()` ou `__ge__()`, então *`TypeError`* é levantada.

- *unsafe_hash*: Se verdadeiro, força `dataclasses` a criar um método `__hash__()`, mesmo que não seja seguro fazê-lo. Caso contrário, gera um método `__hash__()` de acordo com a configuração de *eq* e *frozen*. O valor padrão é `False`.

`__hash__()` é usado para prover o método embutido *hash()*, e quando objetos são adicionados a coleções do tipo dicionário ou conjunto. Ter um método `__hash__()` implica que instâncias da classe serão imutáveis. Mutabilidade é uma propriedade complicada, que depende da intenção do programador, da existência e comportamento do método `__eq__()`, e dos valores dos parâmetros *eq* e *frozen* no decorador `@dataclass`.

Por padrão, `@dataclass` não vai adicionar implicitamente um método `__hash__()`, a menos que seja seguro fazê-lo. Nem irá adicionar ou modificar um método `__hash__()` existente, definido explicitamente. Configurar o atributo de classe `__hash__ = None` tem um significado específico para o Python, conforme descrito na documentação de `__hash__()`.

Se `__hash__()` não é definido explicitamente, ou se é configurado como `None`, então `@dataclass` *pode* adicionar um método `__hash__()` implícito. Mesmo que não seja recomendado, pode-se forçar `@dataclass` a criar um método `__hash__()` com `unsafe_hash=True`. Este pode ser o caso se sua classe é logicamente imutável, mas na prática pode ser mudada. Esse é um caso de uso específico e deve ser considerado com muito cuidado.

Essas são as regras governando a criação implícita de um método `__hash__()`. Observe que não pode ter um método `__hash__()` explícito na dataclass e configurar `unsafe_hash=True`; isso resultará em um *TypeError*.

Se *eq* e *frozen* são ambos verdadeiros, por padrão `@dataclass` vai gerar um método `__hash__()`. Se *eq* é verdadeiro e *frozen* é falso, `__hash__()` será configurado para `None`, marcando a classe como não-hasheável (já que é mutável). Se *eq* é falso, `__hash__()` será deixado intocado, o que significa que o método `__hash__()` da superclasse será usado (se a superclasse é *object*, significa que voltará para o hash baseado em id).

- *frozen*: If true (the default is `False`), assigning to fields will generate an exception. This emulates read-only frozen instances. See the *discussion* below.

  If `__setattr__()` or `__delattr__()` is defined in the class and *frozen* is true, then *TypeError* is raised.

- *match_args*: Se verdadeiro (o padrão é `True`), a tupla `__match_args__` será criada a partir da lista de parâmetros não-somente-nomeado para o método `__init__()` gerado (mesmo se `__init__()` não for gerado, veja acima). Se falso, ou se `__match_args__` já estiver definido na classe, então `__match_args__` não será gerado.

  Adicionado na versão 3.10.

- *kw_only*: Se verdadeiro (o valor padrão é `False`), então todos os campos serão marcados como somente-nomeado. Se um campo for marcado como somente-nomeado, então o único efeito é que o parâmetro `__init__()` gerado a partir de um campo somente-nomeado deve ser especificado com um campo quando `__init__()` é chamado. Veja a entrada *parâmetro* do glossário para detalhes. Veja também a seção *KW_ONLY*.

  Campos somente-nomeados não são incluídos em `__match_args__`.

  Adicionado na versão 3.10.

- *slots*: Se true (o padrão é `False`), o atributo `__slots__` será gerado e uma nova classe será retornada no lugar da original. Se `__slots__` já estiver definido na classe, então *TypeError* será levantada.

  Aviso

  Passar parâmetros para uma classe base `__init_subclass__()` ao usar `slots=True` resultará em um *TypeError*. Use `__init_subclass__` sem parâmetros ou use valores padrão como solução alternativa. Consulte gh-91126 para obter detalhes completos.

  Adicionado na versão 3.10.

Alterado na versão 3.11: Se um nome de campo já estiver incluído no `__slots__` de uma classe base, ele não será incluído no `__slots__` gerado para evitar substitui-los. Portanto, não use `__slots__` para recuperar os nomes dos campos de uma classe de dados. Use *`fields()`* em vez disso. Para poder determinar os slots herdados, a classe base `__slots__` pode ser qualquer iterável, mas *não* um iterador.

- *weakref_slot*: Se verdadeiro (o padrão é `False`), adicione um slot chamado "__weakref__", que é necessário para tornar uma instância *`fraca`*. É um erro especificar `weakref_slot=True` sem também especificar `slots=True`.

  Adicionado na versão 3.11.

`fields` pode opcionalmente especificar um valor padrão, usando sintaxe Python normal:

```
@dataclass
class C:
    a: int       # 'a' tem nenhum valor mínimo
    b: int = 0   # atribui um valor padrão para 'b'
```

Nesse exemplo, `a` e `b` serão incluídos no método `__init__()` adicionado, que será definido como:

```
def __init__(self, a: int, b: int = 0):
```

`TypeError` será levantada se um campo sem valor padrão for definido após um campo com valor padrão. Isso é verdadeiro se ocorrer numa classe simples, ou como resultado de uma herança de classe.

`dataclasses.`**`field`**`(*, default=MISSING, default_factory=MISSING, init=True, repr=True, hash=None, compare=True, metadata=None, kw_only=MISSING, doc=None)`

Para casos de uso comuns e simples, nenhuma outra funcionalidade é necessária. Existem, no entanto, alguns recursos que requerem informações adicionais por campo. Para satisfazer essa necessidade de informações adicionais, você pode substituir o valor do campo padrão por uma chamada para a função `field()` fornecida. Por exemplo:

```
@dataclass
class C:
    mylist: list[int] = field(default_factory=list)

c = C()
c.mylist += [1, 2, 3]
```

Como mostrado acima, o valor *`MISSING`* é um objeto sentinela usado para detectar se alguns parâmetros são fornecidos pelo usuário. Este sentinela é usado porque `None` é um valor válido para alguns parâmetros com um significado distinto. Nenhum código deve usar diretamente o valor *`MISSING`*.

Os parâmetros de `field()` são:

- *default*: Se fornecido, este será o valor padrão para este campo. Isso é necessário porque a própria chamada `field()` substitui a posição normal do valor padrão.
- *default_factory*: Se fornecido, deve ser um chamável sem argumento que será chamado quando um valor padrão for necessário para este campo. Entre outras finalidades, isso pode ser usado para especificar campos com valores padrão mutáveis, conforme discutido abaixo. É um erro especificar ambos *default* e *default_factory*.
- *init*: Se verdadeiro (o padrão), este campo é incluído como um parâmetro para o método `__init__()` gerado.
- *repr*: Se verdadeiro (o padrão), este campo é incluído na string retornada pelo método `__repr__()` gerado.
- *hash*: Pode ser um bool ou `None`. Se verdadeiro, este campo é incluído no método `__hash__()` gerado. Se false, este campo é excluído do `__hash__()` gerado. Se `None` (o padrão), usa o valor de *compare*:

este seria normalmente o comportamento esperado, já que um campo deve ser incluído no hash se for usado para comparações. Definir este valor para algo diferente de `None` é desencorajado.

Uma possível razão para definir `hash=False` mas `compare=True` seria se um campo for caro para calcular um valor de hash, esse campo for necessário para teste de igualdade e houver outros campos que contribuem para o valor de hash do tipo. Mesmo que um campo seja excluído do hash, ele ainda será usado para comparações.

- *compare*: Se verdadeiro (o padrão), este campo é incluído nos métodos de igualdade e comparação gerados (`__eq__()`, `__gt__()`, etc.).
- *metadata*: Pode ser um mapeamento ou `None`. `None` é tratado como um dicionário vazio. Este valor é agrupado em *`MappingProxyType()`* para torná-lo somente leitura e exposto no objeto *`Field`*. Ele não é usado por Data Classes e é fornecido como um mecanismo de extensão de terceiros. Vários terceiros podem ter sua própria chave, para usar como um espaço de nomes nos metadados.
- *kw_only*: Se verdadeiro, este campo será marcado como somente-nomeado. Isso é usado quando os parâmetros do método `__init__()` gerados são calculados.

  Campos somente-nomeados também não são incluídos em `__match_args__`.

  Adicionado na versão 3.10.

- *doc*: docstring opcional para este campo.

  Adicionado na versão 3.14.

If the default value of a field is specified by a call to `field()`, then the class attribute for this field will be replaced by the specified *default* value. If *default* is not provided, then the class attribute will be deleted. The intent is that after the *`@dataclass`* decorator runs, the class attributes will all contain the default values for the fields, just as if the default value itself were specified. For example, after:

```
@dataclass
class C:
    x: int
    y: int = field(repr=False)
    z: int = field(repr=False, default=10)
    t: int = 20
```

The class attribute `C.z` will be `10`, the class attribute `C.t` will be `20`, and the class attributes `C.x` and `C.y` will not be set.

class dataclasses.**Field**

`Field` objects describe each defined field. These objects are created internally, and are returned by the *`fields()`* module-level method (see below). Users should never instantiate a `Field` object directly. Its documented attributes are:

- `name`: The name of the field.
- `type`: The type of the field.
- `default`, `default_factory`, `init`, `repr`, `hash`, `compare`, `metadata`, and `kw_only` have the identical meaning and values as they do in the *`field()`* function.

Outros atributos podem existir, mas são privados e não devem ser inspecionados ou confiáveis.

class dataclasses.**InitVar**

`InitVar[T]` type annotations describe variables that are *init-only*. Fields annotated with `InitVar` are considered pseudo-fields, and thus are neither returned by the *`fields()`* function nor used in any way except adding them as parameters to `__init__()` and an optional *`__post_init__()`*.

dataclasses.**fields**(*class_or_instance*)

Retorna uma tupla de objetos *Field* que definem os campos para esta classe de dados. Aceita uma classe de dados ou uma instância de uma classe de dados. Levanta *TypeError* se não for passada uma classe de dados ou instância de uma. Não retorna pseudocampos que são ClassVar ou InitVar.

dataclasses.**asdict**(*obj*, *, *dict_factory=dict*)

Converts the dataclass *obj* to a dict (by using the factory function *dict_factory*). Each dataclass is converted to a dict of its fields, as name: value pairs. dataclasses, dicts, lists, and tuples are recursed into. Other objects are copied with *copy.deepcopy()*.

Example of using asdict() on nested dataclasses:

```
@dataclass
class Point:
     x: int
     y: int

@dataclass
class C:
     mylist: list[Point]

p = Point(10, 20)
assert asdict(p) == {'x': 10, 'y': 20}

c = C([Point(0, 0), Point(10, 4)])
assert asdict(c) == {'mylist': [{'x': 0, 'y': 0}, {'x': 10, 'y': 4}]}
```

Para criar uma cópia rasa, a seguinte solução alternativa pode ser usada:

```
{field.name: getattr(obj, field.name) for field in fields(obj)}
```

asdict() raises *TypeError* if *obj* is not a dataclass instance.

dataclasses.**astuple**(*obj*, *, *tuple_factory=tuple*)

Converts the dataclass *obj* to a tuple (by using the factory function *tuple_factory*). Each dataclass is converted to a tuple of its field values. dataclasses, dicts, lists, and tuples are recursed into. Other objects are copied with *copy.deepcopy()*.

Continuando a partir do exemplo anterior:

```
assert astuple(p) == (10, 20)
assert astuple(c) == ([(0, 0), (10, 4)],)
```

Para criar uma cópia rasa, a seguinte solução alternativa pode ser usada:

```
tuple(getattr(obj, field.name) for field in dataclasses.fields(obj))
```

astuple() raises *TypeError* if *obj* is not a dataclass instance.

dataclasses.**make_dataclass**(*cls_name*, *fields*, *, *bases=()*, *namespace=None*, *init=True*, *repr=True*, *eq=True*, *order=False*, *unsafe_hash=False*, *frozen=False*, *match_args=True*, *kw_only=False*, *slots=False*, *weakref_slot=False*, *module=None*, *decorator=dataclass*)

Creates a new dataclass with name *cls_name*, fields as defined in *fields*, base classes as given in *bases*, and initialized with a namespace as given in *namespace*. *fields* is an iterable whose elements are each either name, (name, type), or (name, type, Field). If just name is supplied, *typing.Any* is used for type. The values of *init*, *repr*, *eq*, *order*, *unsafe_hash*, *frozen*, *match_args*, *kw_only*, *slots*, and *weakref_slot* have the same meaning as they do in *@dataclass*.

If *module* is defined, the __module__ attribute of the dataclass is set to that value. By default, it is set to the module name of the caller.

The *decorator* parameter is a callable that will be used to create the dataclass. It should take the class object as a first argument and the same keyword arguments as *@dataclass*. By default, the *@dataclass* function is used.

This function is not strictly required, because any Python mechanism for creating a new class with `__annotations__` can then apply the *@dataclass* function to convert that class to a dataclass. This function is provided as a convenience. For example:

```
C = make_dataclass('C',
                   [('x', int),
                     'y',
                    ('z', int, field(default=5))],
                   namespace={'add_one': lambda self: self.x + 1})
```

É equivalente a:

```
@dataclass
class C:
    x: int
    y: 'typing.Any'
    z: int = 5

    def add_one(self):
        return self.x + 1
```

Adicionado na versão 3.14: Added the *decorator* parameter.

`dataclasses.`**`replace`**(*obj*, */*, ***changes*)

Creates a new object of the same type as *obj*, replacing fields with values from *changes*. If *obj* is not a Data Class, raises *TypeError*. If keys in *changes* are not field names of the given dataclass, raises *TypeError*.

The newly returned object is created by calling the `__init__()` method of the dataclass. This ensures that *__post_init__()*, if present, is also called.

Init-only variables without default values, if any exist, must be specified on the call to `replace()` so that they can be passed to `__init__()` and *__post_init__()*.

It is an error for *changes* to contain any fields that are defined as having `init=False`. A *ValueError* will be raised in this case.

Be forewarned about how `init=False` fields work during a call to `replace()`. They are not copied from the source object, but rather are initialized in *__post_init__()*, if they're initialized at all. It is expected that `init=False` fields will be rarely and judiciously used. If they are used, it might be wise to have alternate class constructors, or perhaps a custom `replace()` (or similarly named) method which handles instance copying.

Dataclass instances are also supported by generic function *copy.replace()*.

`dataclasses.`**`is_dataclass`**(*obj*)

Return `True` if its parameter is a dataclass (including subclasses of a dataclass) or an instance of one, otherwise return `False`.

Se você precisa saber se a classe é uma instância de dataclass (e não a dataclass de fato), então adicione uma verificação para `not isinstance(obj, type)`:

```
def is_dataclass_instance(obj):
    return is_dataclass(obj) and not isinstance(obj, type)
```

`dataclasses.`**`MISSING`**

A sentinel value signifying a missing default or default_factory.

`dataclasses.`**`KW_ONLY`**

A sentinel value used as a type annotation. Any fields after a pseudo-field with the type of `KW_ONLY` are marked as keyword-only fields. Note that a pseudo-field of type `KW_ONLY` is otherwise completely ignored.

This includes the name of such a field. By convention, a name of _ is used for a `KW_ONLY` field. Keyword-only fields signify `__init__()` parameters that must be specified as keywords when the class is instantiated.

In this example, the fields `y` and `z` will be marked as keyword-only fields:

```
@dataclass
class Point:
    x: float
    _: KW_ONLY
    y: float
    z: float

p = Point(0, y=1.5, z=2.0)
```

In a single dataclass, it is an error to specify more than one field whose type is `KW_ONLY`.

Adicionado na versão 3.10.

exception dataclasses.**FrozenInstanceError**

Raised when an implicitly defined `__setattr__()` or `__delattr__()` is called on a dataclass which was defined with `frozen=True`. It is a subclass of *AttributeError*.

29.7.2 Processamento pós-inicialização

dataclasses.**__post_init__**()

When defined on the class, it will be called by the generated `__init__()`, normally as `self.__post_init__()`. However, if any `InitVar` fields are defined, they will also be passed to `__post_init__()` in the order they were defined in the class. If no `__init__()` method is generated, then `__post_init__()` will not automatically be called.

Among other uses, this allows for initializing field values that depend on one or more other fields. For example:

```
@dataclass
class C:
    a: float
    b: float
    c: float = field(init=False)

    def __post_init__(self):
        self.c = self.a + self.b
```

The `__init__()` method generated by *@dataclass* does not call base class `__init__()` methods. If the base class has an `__init__()` method that has to be called, it is common to call this method in a *__post_init__()* method:

```
class Rectangle:
    def __init__(self, height, width):
        self.height = height
        self.width = width

@dataclass
class Square(Rectangle):
    side: float

    def __post_init__(self):
        super().__init__(self.side, self.side)
```

Note, however, that in general the dataclass-generated `__init__()` methods don't need to be called, since the derived dataclass will take care of initializing all fields of any base class that is a dataclass itself.

See the section below on init-only variables for ways to pass parameters to `__post_init__()`. Also see the warning about how *`replace()`* handles `init=False` fields.

29.7.3 Variáveis de classe

One of the few places where *`@dataclass`* actually inspects the type of a field is to determine if a field is a class variable as defined in **PEP 526**. It does this by checking if the type of the field is *`typing.ClassVar`*. If a field is a `ClassVar`, it is excluded from consideration as a field and is ignored by the dataclass mechanisms. Such `ClassVar` pseudo-fields are not returned by the module-level *`fields()`* function.

29.7.4 Variáveis de inicialização apenas

Another place where *`@dataclass`* inspects a type annotation is to determine if a field is an init-only variable. It does this by seeing if the type of a field is of type *`InitVar`*. If a field is an *`InitVar`*, it is considered a pseudo-field called an init-only field. As it is not a true field, it is not returned by the module-level *`fields()`* function. Init-only fields are added as parameters to the generated `__init__()` method, and are passed to the optional *`__post_init__()`* method. They are not otherwise used by dataclasses.

For example, suppose a field will be initialized from a database, if a value is not provided when creating the class:

```
@dataclass
class C:
    i: int
    j: int | None = None
    database: InitVar[DatabaseType | None] = None

    def __post_init__(self, database):
        if self.j is None and database is not None:
            self.j = database.lookup('j')

c = C(10, database=my_database)
```

In this case, *`fields()`* will return *`Field`* objects for `i` and `j`, but not for `database`.

29.7.5 Frozen instances

It is not possible to create truly immutable Python objects. However, by passing `frozen=True` to the *`@dataclass`* decorator you can emulate immutability. In that case, dataclasses will add `__setattr__()` and `__delattr__()` methods to the class. These methods will raise a *`FrozenInstanceError`* when invoked.

There is a tiny performance penalty when using `frozen=True`: `__init__()` cannot use simple assignment to initialize fields, and must use `object.__setattr__()`.

29.7.6 Herança

When the dataclass is being created by the *`@dataclass`* decorator, it looks through all of the class's base classes in reverse MRO (that is, starting at *`object`*) and, for each dataclass that it finds, adds the fields from that base class to an ordered mapping of fields. After all of the base class fields are added, it adds its own fields to the ordered mapping. All of the generated methods will use this combined, calculated ordered mapping of fields. Because the fields are in insertion order, derived classes override base classes. An example:

```
@dataclass
class Base:
    x: Any = 15.0
    y: int = 0

@dataclass
class C(Base):
```

(continua na próxima página)

(continuação da página anterior)

```
    z: int = 10
    x: int = 15
```

The final list of fields is, in order, `x`, `y`, `z`. The final type of `x` is *int*, as specified in class `C`.

The generated `__init__()` method for `C` will look like:

```
def __init__(self, x: int = 15, y: int = 0, z: int = 10):
```

29.7.7 Re-ordering of keyword-only parameters in `__init__()`

After the parameters needed for `__init__()` are computed, any keyword-only parameters are moved to come after all regular (non-keyword-only) parameters. This is a requirement of how keyword-only parameters are implemented in Python: they must come after non-keyword-only parameters.

In this example, `Base.y`, `Base.w`, and `D.t` are keyword-only fields, and `Base.x` and `D.z` are regular fields:

```
@dataclass
class Base:
    x: Any = 15.0
    _: KW_ONLY
    y: int = 0
    w: int = 1

@dataclass
class D(Base):
    z: int = 10
    t: int = field(kw_only=True, default=0)
```

The generated `__init__()` method for `D` will look like:

```
def __init__(self, x: Any = 15.0, z: int = 10, *, y: int = 0, w: int = 1, t: int =␣
↪0):
```

Note that the parameters have been re-ordered from how they appear in the list of fields: parameters derived from regular fields are followed by parameters derived from keyword-only fields.

The relative ordering of keyword-only parameters is maintained in the re-ordered `__init__()` parameter list.

29.7.8 Funções padrão de fábrica

If a *field()* specifies a *default_factory*, it is called with zero arguments when a default value for the field is needed. For example, to create a new instance of a list, use:

```
mylist: list = field(default_factory=list)
```

If a field is excluded from `__init__()` (using `init=False`) and the field also specifies *default_factory*, then the default factory function will always be called from the generated `__init__()` function. This happens because there is no other way to give the field an initial value.

29.7.9 Valores padrão mutáveis

Python stores default member variable values in class attributes. Consider this example, not using dataclasses:

```
class C:
    x = []
    def add(self, element):
        self.x.append(element)
```

(continua na próxima página)

(continuação da página anterior)

```
o1 = C()
o2 = C()
o1.add(1)
o2.add(2)
assert o1.x == [1, 2]
assert o1.x is o2.x
```

Note that the two instances of class `C` share the same class variable `x`, as expected.

Usando dataclasses, *se* este código fosse válido:

```
@dataclass
class D:
    x: list = []       # This code raises ValueError
    def add(self, element):
        self.x.append(element)
```

Geraria código similar a:

```
class D:
    x = []
    def __init__(self, x=x):
        self.x = x
    def add(self, element):
        self.x.append(element)

assert D().x is D().x
```

This has the same issue as the original example using class `C`. That is, two instances of class `D` that do not specify a value for `x` when creating a class instance will share the same copy of `x`. Because dataclasses just use normal Python class creation they also share this behavior. There is no general way for Data Classes to detect this condition. Instead, the *`@dataclass`* decorator will raise a *`ValueError`* if it detects an unhashable default parameter. The assumption is that if a value is unhashable, it is mutable. This is a partial solution, but it does protect against many common errors.

Using default factory functions is a way to create new instances of mutable types as default values for fields:

```
@dataclass
class D:
    x: list = field(default_factory=list)

assert D().x is not D().x
```

Alterado na versão 3.11: Instead of looking for and disallowing objects of type *`list`*, *`dict`*, or *`set`*, unhashable objects are now not allowed as default values. Unhashability is used to approximate mutability.

29.7.10 Descriptor-typed fields

Fields that are assigned descriptor objects as their default value have the following special behaviors:

- The value for the field passed to the dataclass's `__init__()` method is passed to the descriptor's `__set__()` method rather than overwriting the descriptor object.
- Similarly, when getting or setting the field, the descriptor's `__get__()` or `__set__()` method is called rather than returning or overwriting the descriptor object.
- To determine whether a field contains a default value, *`@dataclass`* will call the descriptor's `__get__()` method using its class access form: `descriptor.__get__(obj=None, type=cls)`. If the descriptor

returns a value in this case, it will be used as the field's default. On the other hand, if the descriptor raises *AttributeError* in this situation, no default value will be provided for the field.

```
class IntConversionDescriptor:
    def __init__(self, *, default):
        self._default = default

    def __set_name__(self, owner, name):
        self._name = "_" + name

    def __get__(self, obj, type):
        if obj is None:
            return self._default

        return getattr(obj, self._name, self._default)

    def __set__(self, obj, value):
        setattr(obj, self._name, int(value))

@dataclass
class InventoryItem:
    quantity_on_hand: IntConversionDescriptor =␣
↪IntConversionDescriptor(default=100)

i = InventoryItem()
print(i.quantity_on_hand)    # 100
i.quantity_on_hand = 2.5     # calls __set__ with 2.5
print(i.quantity_on_hand)    # 2
```

Note that if a field is annotated with a descriptor type, but is not assigned a descriptor object as its default value, the field will act like a normal field.

29.8 `contextlib` — Utilitários para contextos da instrução `with`

Código-fonte: Lib/contextlib.py

Este módulo fornece utilitários para tarefas comuns envolvendo a instrução `with`. Para mais informações, veja também *Tipos de Gerenciador de Contexto* e context-managers.

29.8.1 Utilitários

Funções e classes fornecidas:

class contextlib.**AbstractContextManager**

Uma *classe base abstrata* para classes que implementam `object.__enter__()` e `object.__exit__()`. Uma implementação padrão para `object.__enter__()` é fornecida, que retorna `self`, enquanto `object.__exit__()` é um método abstrato que, por padrão, retorna `None`. Veja também a definição de *Tipos de Gerenciador de Contexto*.

Adicionado na versão 3.6.

class contextlib.**AbstractAsyncContextManager**

Uma *classe base abstrata* para classes que implementam `object.__aenter__()` e `object.__aexit__()`. Uma implementação padrão para `object.__aenter__()` é fornecida, que retorna `self`, enquanto `object.__aexit__()` é um método abstrato que, por padrão, retorna `None`. Veja também a definição de async-context-managers

Adicionado na versão 3.7.

@contextlib.**contextmanager**

Esta função é um *decorador* que pode ser usado para definir uma função de fábrica para gerenciadores de contexto de instrução `with`, sem precisar criar uma classe ou separar os métodos `__enter__()` e `__exit__()`.

While many objects natively support use in with statements, sometimes a resource needs to be managed that isn't a context manager in its own right, and doesn't implement a `close()` method for use with `contextlib.closing`.

Um exemplo abstrato seria o seguinte para garantir o gerenciamento correto dos recursos:

```
from contextlib import contextmanager

@contextmanager
def managed_resource(*args, **kwds):
    # Códigopara obter recursos como, por exemplo:
    resource = acquire_resource(*args, **kwds)
    try:
        yield resource
    finally:
        # Código para liberar recursos como, por exemplo:
        release_resource(resource)
```

A função pode, então, ser usada da seguinte forma:

```
>>> with managed_resource(timeout=3600) as resource:
...     # O recurso é liberado no final deste bloco,
...     # mesmo que o código no bloco levante uma exceção
```

A função que está sendo decorada deve retornar um iterador *gerador* quando chamada. Este iterador deve produzir exatamente um valor, que será vinculado aos alvos na cláusula `as` da instrução `with`, se houver.

No ponto em que o gerador é produzido, o bloco aninhado na instrução `with` é executado. O gerador é então retomado após o bloco ser encerrado. Se uma exceção não tratada ocorrer no bloco, ela será levantada novamente dentro do gerador no ponto em que a produção ocorreu. Assim, você pode usar uma instrução `try…except…finally` para capturar o erro (se houver), ou garantir que alguma limpeza ocorra. Se uma exceção for capturada apenas para registrá-la ou para executar alguma ação (em vez de suprimi-la completamente), o gerador deve levantar novamente essa exceção. Caso contrário, o gerenciador de contexto do gerador indicará à instrução `with` que a exceção foi tratada, e a execução será retomada com a instrução imediatamente após a instrução `with`.

`contextmanager()` usa *`ContextDecorator`* para que os gerenciadores de contexto que ela cria possam ser usados como decoradores, bem como em instruções `with`. Quando usada como um decorador, uma nova instância do gerador é implicitamente criada em cada chamada de função (isso permite que os gerenciadores de contexto criados por *`contextmanager()`* atendam ao requisito de que os gerenciadores de contexto ofereçam suporte a múltiplas invocações para serem usados como decoradores).

Alterado na versão 3.2: Uso de *`ContextDecorator`*.

@contextlib.**asynccontextmanager**

Semelhante a *`contextmanager()`*, mas cria um gerenciador de contexto assíncrono.

Esta função é um *decorador* que pode ser usado para definir uma função de fábrica para gerenciadores de contexto assíncronos de instrução `async with`, sem precisar criar uma classe ou separar os métodos `__aenter__()` e `__aexit__()`. Ela deve ser aplicada a uma função que atua como *gerador assíncrono*.

Um exemplo simples:

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def get_connection():
```

(continua na próxima página)

(continuação da página anterior)

```
    conn = await acquire_db_connection()
    try:
        yield conn
    finally:
        await release_db_connection(conn)

async def get_all_users():
    async with get_connection() as conn:
        return conn.query('SELECT ...')
```

Adicionado na versão 3.7.

Gerenciadores de contexto definidos com *asynccontextmanager()* podem ser usados como decoradores ou com instruções `async with`:

```
import time
from contextlib import asynccontextmanager

@asynccontextmanager
async def timeit():
    now = time.monotonic()
    try:
        yield
    finally:
        print(f'it took {time.monotonic() - now}s to run')

@timeit()
async def main():
    # ... código do async ...
```

Quando usada como um decorador, uma nova instância do gerador é implicitamente criada em cada chamada de função. Isso permite que os gerenciadores de contexto criados por *asynccontextmanager()* atendam ao requisito de que os gerenciadores de contexto ofereçam suporte a múltiplas invocações para serem usados como decoradores.

Alterado na versão 3.10: Gerenciadores de contexto assíncronos criados com *asynccontextmanager()* podem ser usados como decoradores.

`contextlib.`**`closing`**(*thing*)

Retorna um gerenciador de contexto que fecha *thing* após a conclusão do bloco. Isso basicamente equivale a:

```
from contextlib import contextmanager

@contextmanager
def closing(thing):
    try:
        yield thing
    finally:
        thing.close()
```

E permite que você escreva código como isso:

```
from contextlib import closing
from urllib.request import urlopen

with closing(urlopen('https://www.python.org')) as page:
    for line in page:
        print(line)
```

sem precisar fechar explicitamente `page`. Mesmo se ocorrer um erro, `page.close()` será chamado quando o bloco `with` for encerrado.

> **Nota**
>
> A maioria dos tipos que gerenciam recursos suporta o protocolo de *gerenciador de contexto*, que fecha *thing* ao sair da declaração `with`. Como tal, `closing()` é mais útil para tipos de terceiros que não oferecem suporte a gerenciadores de contexto. Este exemplo é puramente para fins ilustrativos, pois *`urlopen()`* normalmente seria usado em um gerenciador de contexto.

`contextlib.`**`aclosing`**(*thing*)

Retorna um gerenciador de contexto async que chama o método `aclose()` de *thing* após a conclusão do bloco. Isso basicamente equivale a:

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def aclosing(thing):
    try:
        yield thing
    finally:
        await thing.aclose()
```

De forma significativa, `aclosing()` oferece suporte a limpeza determinística de geradores assíncronos quando eles são encerrados mais cedo por `break` ou uma exceção. Por exemplo:

```
from contextlib import aclosing

async with aclosing(my_generator()) as values:
    async for value in values:
        if value == 42:
            break
```

Esse padrão garante que o código de saída assíncrono do gerador seja executado no mesmo contexto que suas iterações (para que exceções e variáveis de contexto funcionem conforme o esperado, e o código de saída não seja executado após o tempo de vida de alguma tarefa da qual ele depende).

Adicionado na versão 3.10.

`contextlib.`**`nullcontext`**(*enter_result=None*)

Retorna um gerenciador de contexto que retorna *enter_result* de `__enter__`, mas não faz nada de outra forma. Ele foi criado para ser usado como um substituto para um gerenciador de contexto opcional, por exemplo:

```
def myfunction(arg, ignore_exceptions=False):
    if ignore_exceptions:
        # Usa suppress para ignorar todas as exceções.
        cm = contextlib.suppress(Exception)
    else:
        # Não ignora quaisquer exceções, cm não tem efeito.
        cm = contextlib.nullcontext()
    with cm:
        # Faz algo
```

Um exemplo usando *enter_result*:

```
def process_file(file_or_path):
    if isinstance(file_or_path, str):
        # Se for uma string, abre o arquivo
```

(continua na próxima página)

(continuação da página anterior)

```
        cm = open(file_or_path)
    else:
        # O chamador é responsável por fechar o arquivo
        cm = nullcontext(file_or_path)

    with cm as file:
        # Efetua um processamento no arquivo
```

Também pode ser usado como um substituto para gerenciadores de contexto assíncronos:

```
async def send_http(session=None):
    if not session:
        # Se houver nenhuma sessão http, cria-a com aiohttp
        cm = aiohttp.ClientSession()
    else:
        # O chamador é responsável por fechar a sessão
        cm = nullcontext(session)

    async with cm as session:
        # Envia requisições http com sessão
```

Adicionado na versão 3.7.

Alterado na versão 3.10: Suporte a *gerenciador de contexto assíncrono* foi adicionado.

contextlib.**suppress**(**exceptions*)

Retorna um gerenciador de contexto que suprime qualquer uma das exceções especificadas se elas ocorrerem no corpo de uma instrução `with` e então retoma a execução com a primeira instrução após o final da instrução `with`.

Como qualquer outro mecanismo que suprime completamente exceções, este gerenciador de contexto deve ser usado apenas para cobrir erros muito específicos, onde continuar silenciosamente com a execução do programa é considerado a coisa certa a fazer.

Por exemplo:

```
from contextlib import suppress

with suppress(FileNotFoundError):
    os.remove('algumarquivo.tmp')

with suppress(FileNotFoundError):
    os.remove('outroarquivo.tmp')
```

Este código equivale a:

```
try:
    os.remove('algumarquivo.tmp')
except FileNotFoundError:
    pass

try:
    os.remove('outroarquivo.tmp')
except FileNotFoundError:
    pass
```

O gerenciador de contexto é *reentrante*.

Se o código dentro do bloco `with` levantar uma exceção *BaseExceptionGroup*, exceções suprimidas são removidas do grupo. Quaisquer exceções do grupo que não forem suprimidas são levantadas novamente em

um novo grupo que é criado usando o método *derive()* do grupo original.

Adicionado na versão 3.4.

Alterado na versão 3.12: `suppress` now supports suppressing exceptions raised as part of a *BaseExceptionGroup*.

`contextlib.`**`redirect_stdout`**(*new_target*)

Context manager for temporarily redirecting *sys.stdout* to another file or file-like object.

This tool adds flexibility to existing functions or classes whose output is hardwired to stdout.

For example, the output of *help()* normally is sent to *sys.stdout*. You can capture that output in a string by redirecting the output to an *io.StringIO* object. The replacement stream is returned from the `__enter__` method and so is available as the target of the `with` statement:

```
with redirect_stdout(io.StringIO()) as f:
    help(pow)
s = f.getvalue()
```

To send the output of *help()* to a file on disk, redirect the output to a regular file:

```
with open('help.txt', 'w') as f:
    with redirect_stdout(f):
        help(pow)
```

To send the output of *help()* to *sys.stderr*:

```
with redirect_stdout(sys.stderr):
    help(pow)
```

Note that the global side effect on *sys.stdout* means that this context manager is not suitable for use in library code and most threaded applications. It also has no effect on the output of subprocesses. However, it is still a useful approach for many utility scripts.

O gerenciador de contexto é *reentrante*.

Adicionado na versão 3.4.

`contextlib.`**`redirect_stderr`**(*new_target*)

Similar to *redirect_stdout()* but redirecting *sys.stderr* to another file or file-like object.

O gerenciador de contexto é *reentrante*.

Adicionado na versão 3.5.

`contextlib.`**`chdir`**(*path*)

Non parallel-safe context manager to change the current working directory. As this changes a global state, the working directory, it is not suitable for use in most threaded or async contexts. It is also not suitable for most non-linear code execution, like generators, where the program execution is temporarily relinquished – unless explicitly desired, you should not yield when this context manager is active.

This is a simple wrapper around *chdir()*, it changes the current working directory upon entering and restores the old one on exit.

O gerenciador de contexto é *reentrante*.

Adicionado na versão 3.11.

`class` `contextlib.`**`ContextDecorator`**

A base class that enables a context manager to also be used as a decorator.

Context managers inheriting from `ContextDecorator` have to implement `__enter__` and `__exit__` as normal. `__exit__` retains its optional exception handling even when used as a decorator.

`ContextDecorator` is used by *contextmanager()*, so you get this functionality automatically.

Example of `ContextDecorator`:

```
from contextlib import ContextDecorator

class mycontext(ContextDecorator):
    def __enter__(self):
        print('Starting')
        return self

    def __exit__(self, *exc):
        print('Finishing')
        return False
```

The class can then be used like this:

```
>>> @mycontext()
... def function():
...     print('The bit in the middle')
...
>>> function()
Starting
The bit in the middle
Finishing

>>> with mycontext():
...     print('The bit in the middle')
...
Starting
The bit in the middle
Finishing
```

This change is just syntactic sugar for any construct of the following form:

```
def f():
    with cm():
        # Do stuff
```

`ContextDecorator` lets you instead write:

```
@cm()
def f():
    # Do stuff
```

It makes it clear that the `cm` applies to the whole function, rather than just a piece of it (and saving an indentation level is nice, too).

Existing context managers that already have a base class can be extended by using `ContextDecorator` as a mixin class:

```
from contextlib import ContextDecorator

class mycontext(ContextBaseClass, ContextDecorator):
    def __enter__(self):
        return self

    def __exit__(self, *exc):
        return False
```

> **Nota**
>
> As the decorated function must be able to be called multiple times, the underlying context manager must support use in multiple `with` statements. If this is not the case, then the original construct with the explicit `with` statement inside the function should be used.

Adicionado na versão 3.2.

class contextlib.**AsyncContextDecorator**

Similar to *ContextDecorator* but only for asynchronous functions.

Example of `AsyncContextDecorator`:

```
from asyncio import run
from contextlib import AsyncContextDecorator

class mycontext(AsyncContextDecorator):
    async def __aenter__(self):
        print('Starting')
        return self

    async def __aexit__(self, *exc):
        print('Finishing')
        return False
```

The class can then be used like this:

```
>>> @mycontext()
... async def function():
...     print('The bit in the middle')
...
>>> run(function())
Starting
The bit in the middle
Finishing

>>> async def function():
...     async with mycontext():
...         print('The bit in the middle')
...
>>> run(function())
Starting
The bit in the middle
Finishing
```

Adicionado na versão 3.10.

class contextlib.**ExitStack**

A context manager that is designed to make it easy to programmatically combine other context managers and cleanup functions, especially those that are optional or otherwise driven by input data.

For example, a set of files may easily be handled in a single with statement as follows:

```
with ExitStack() as stack:
    files = [stack.enter_context(open(fname)) for fname in filenames]
    # All opened files will automatically be closed at the end of
    # the with statement, even if attempts to open files later
    # in the list raise an exception
```

The `__enter__()` method returns the *ExitStack* instance, and performs no additional operations.

Each instance maintains a stack of registered callbacks that are called in reverse order when the instance is closed (either explicitly or implicitly at the end of a `with` statement). Note that callbacks are *not* invoked implicitly when the context stack instance is garbage collected.

This stack model is used so that context managers that acquire their resources in their `__init__` method (such as file objects) can be handled correctly.

Since registered callbacks are invoked in the reverse order of registration, this ends up behaving as if multiple nested `with` statements had been used with the registered set of callbacks. This even extends to exception handling - if an inner callback suppresses or replaces an exception, then outer callbacks will be passed arguments based on that updated state.

This is a relatively low level API that takes care of the details of correctly unwinding the stack of exit callbacks. It provides a suitable foundation for higher level context managers that manipulate the exit stack in application specific ways.

Adicionado na versão 3.3.

enter_context(*cm*)

Enters a new context manager and adds its `__exit__()` method to the callback stack. The return value is the result of the context manager's own `__enter__()` method.

These context managers may suppress exceptions just as they normally would if used directly as part of a `with` statement.

Alterado na versão 3.11: Raises *TypeError* instead of *AttributeError* if *cm* is not a context manager.

push(*exit*)

Adds a context manager's `__exit__()` method to the callback stack.

As `__enter__` is *not* invoked, this method can be used to cover part of an `__enter__()` implementation with a context manager's own `__exit__()` method.

If passed an object that is not a context manager, this method assumes it is a callback with the same signature as a context manager's `__exit__()` method and adds it directly to the callback stack.

By returning true values, these callbacks can suppress exceptions the same way context manager `__exit__()` methods can.

The passed in object is returned from the function, allowing this method to be used as a function decorator.

callback(*callback*, */*, **args*, ***kwds*)

Accepts an arbitrary callback function and arguments and adds it to the callback stack.

Unlike the other methods, callbacks added this way cannot suppress exceptions (as they are never passed the exception details).

The passed in callback is returned from the function, allowing this method to be used as a function decorator.

pop_all()

Transfers the callback stack to a fresh *ExitStack* instance and returns it. No callbacks are invoked by this operation - instead, they will now be invoked when the new stack is closed (either explicitly or implicitly at the end of a `with` statement).

For example, a group of files can be opened as an "all or nothing" operation as follows:

```
with ExitStack() as stack:
    files = [stack.enter_context(open(fname)) for fname in filenames]
    # Hold onto the close method, but don't call it yet.
    close_files = stack.pop_all().close
    # If opening any file fails, all previously opened files will be
    # closed automatically. If all files are opened successfully,
```

(continua na próxima página)

(continuação da página anterior)

```
    # they will remain open even after the with statement ends.
    # close_files() can then be invoked explicitly to close them all.
```

close()

Immediately unwinds the callback stack, invoking callbacks in the reverse order of registration. For any context managers and exit callbacks registered, the arguments passed in will indicate that no exception occurred.

class contextlib.**AsyncExitStack**

An asynchronous context manager, similar to *ExitStack*, that supports combining both synchronous and asynchronous context managers, as well as having coroutines for cleanup logic.

The *close()* method is not implemented; *aclose()* must be used instead.

async enter_async_context(*cm*)

Similar to *ExitStack.enter_context()* but expects an asynchronous context manager.

Alterado na versão 3.11: Raises *TypeError* instead of *AttributeError* if *cm* is not an asynchronous context manager.

push_async_exit(*exit*)

Similar to *ExitStack.push()* but expects either an asynchronous context manager or a coroutine function.

push_async_callback(*callback*, /, **args*, ***kwds*)

Similar to *ExitStack.callback()* but expects a coroutine function.

async aclose()

Similar to *ExitStack.close()* but properly handles awaitables.

Continuing the example for *asynccontextmanager()*:

```
async with AsyncExitStack() as stack:
    connections = [await stack.enter_async_context(get_connection())
        for i in range(5)]
    # All opened connections will automatically be released at the end of
    # the async with statement, even if attempts to open a connection
    # later in the list raise an exception.
```

Adicionado na versão 3.7.

29.8.2 Exemplos e receitas

This section describes some examples and recipes for making effective use of the tools provided by *contextlib*.

Supporting a variable number of context managers

The primary use case for *ExitStack* is the one given in the class documentation: supporting a variable number of context managers and other cleanup operations in a single `with` statement. The variability may come from the number of context managers needed being driven by user input (such as opening a user specified collection of files), or from some of the context managers being optional:

```
with ExitStack() as stack:
    for resource in resources:
        stack.enter_context(resource)
    if need_special_resource():
        special = acquire_special_resource()
        stack.callback(release_special_resource, special)
    # Perform operations that use the acquired resources
```

As shown, *ExitStack* also makes it quite easy to use `with` statements to manage arbitrary resources that don't natively support the context management protocol.

Catching exceptions from `__enter__` methods

It is occasionally desirable to catch exceptions from an `__enter__` method implementation, *without* inadvertently catching exceptions from the `with` statement body or the context manager's `__exit__` method. By using *ExitStack* the steps in the context management protocol can be separated slightly in order to allow this:

```
stack = ExitStack()
try:
    x = stack.enter_context(cm)
except Exception:
    # handle __enter__ exception
else:
    with stack:
        # Handle normal case
```

Actually needing to do this is likely to indicate that the underlying API should be providing a direct resource management interface for use with `try`/`except`/`finally` statements, but not all APIs are well designed in that regard. When a context manager is the only resource management API provided, then *ExitStack* can make it easier to handle various situations that can't be handled directly in a `with` statement.

Cleaning up in an `__enter__` implementation

As noted in the documentation of *ExitStack.push()*, this method can be useful in cleaning up an already allocated resource if later steps in the `__enter__()` implementation fail.

Here's an example of doing this for a context manager that accepts resource acquisition and release functions, along with an optional validation function, and maps them to the context management protocol:

```
from contextlib import contextmanager, AbstractContextManager, ExitStack

class ResourceManager(AbstractContextManager):

    def __init__(self, acquire_resource, release_resource, check_resource_ok=None):
        self.acquire_resource = acquire_resource
        self.release_resource = release_resource
        if check_resource_ok is None:
            def check_resource_ok(resource):
                return True
        self.check_resource_ok = check_resource_ok

    @contextmanager
    def _cleanup_on_error(self):
        with ExitStack() as stack:
            stack.push(self)
            yield
            # The validation check passed and didn't raise an exception
            # Accordingly, we want to keep the resource, and pass it
            # back to our caller
            stack.pop_all()

    def __enter__(self):
        resource = self.acquire_resource()
        with self._cleanup_on_error():
            if not self.check_resource_ok(resource):
                msg = "Failed validation for {!r}"
                raise RuntimeError(msg.format(resource))
```

(continua na próxima página)

(continuação da página anterior)

```
        return resource

    def __exit__(self, *exc_details):
        # We don't need to duplicate any of our resource release logic
        self.release_resource()
```

Replacing any use of `try-finally` and flag variables

A pattern you will sometimes see is a `try-finally` statement with a flag variable to indicate whether or not the body of the `finally` clause should be executed. In its simplest form (that can't already be handled just by using an `except` clause instead), it looks something like this:

```
cleanup_needed = True
try:
    result = perform_operation()
    if result:
        cleanup_needed = False
finally:
    if cleanup_needed:
        cleanup_resources()
```

As with any `try` statement based code, this can cause problems for development and review, because the setup code and the cleanup code can end up being separated by arbitrarily long sections of code.

`ExitStack` makes it possible to instead register a callback for execution at the end of a `with` statement, and then later decide to skip executing that callback:

```
from contextlib import ExitStack

with ExitStack() as stack:
    stack.callback(cleanup_resources)
    result = perform_operation()
    if result:
        stack.pop_all()
```

This allows the intended cleanup behaviour to be made explicit up front, rather than requiring a separate flag variable.

If a particular application uses this pattern a lot, it can be simplified even further by means of a small helper class:

```
from contextlib import ExitStack

class Callback(ExitStack):
    def __init__(self, callback, /, *args, **kwds):
        super().__init__()
        self.callback(callback, *args, **kwds)

    def cancel(self):
        self.pop_all()

with Callback(cleanup_resources) as cb:
    result = perform_operation()
    if result:
        cb.cancel()
```

If the resource cleanup isn't already neatly bundled into a standalone function, then it is still possible to use the decorator form of *`ExitStack.callback()`* to declare the resource cleanup in advance:

```
from contextlib import ExitStack

with ExitStack() as stack:
    @stack.callback
    def cleanup_resources():
        ...
    result = perform_operation()
    if result:
        stack.pop_all()
```

Devido à maneira como o protocolo decorador funciona, uma função de retorno de chamada declarada dessa forma não pode receber nenhum parâmetro. Em vez disso, quaisquer recursos a serem liberados devem ser acessados como variáveis de clausura.

Using a context manager as a function decorator

ContextDecorator makes it possible to use a context manager in both an ordinary `with` statement and also as a function decorator.

For example, it is sometimes useful to wrap functions or groups of statements with a logger that can track the time of entry and time of exit. Rather than writing both a function decorator and a context manager for the task, inheriting from *ContextDecorator* provides both capabilities in a single definition:

```
from contextlib import ContextDecorator
import logging

logging.basicConfig(level=logging.INFO)

class track_entry_and_exit(ContextDecorator):
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        logging.info('Entering: %s', self.name)

    def __exit__(self, exc_type, exc, exc_tb):
        logging.info('Exiting: %s', self.name)
```

Instances of this class can be used as both a context manager:

```
with track_entry_and_exit('widget loader'):
    print('Some time consuming activity goes here')
    load_widget()
```

And also as a function decorator:

```
@track_entry_and_exit('widget loader')
def activity():
    print('Some time consuming activity goes here')
    load_widget()
```

Note that there is one additional limitation when using context managers as function decorators: there's no way to access the return value of `__enter__()`. If that value is needed, then it is still necessary to use an explicit `with` statement.

Ver também

PEP 343 - A instrução "with"
A especificação, o histórico e os exemplos para a instrução Python `with`.

29.8.3 Single use, reusable and reentrant context managers

Most context managers are written in a way that means they can only be used effectively in a `with` statement once. These single use context managers must be created afresh each time they're used - attempting to use them a second time will trigger an exception or otherwise not work correctly.

This common limitation means that it is generally advisable to create context managers directly in the header of the `with` statement where they are used (as shown in all of the usage examples above).

Files are an example of effectively single use context managers, since the first `with` statement will close the file, preventing any further IO operations using that file object.

Context managers created using *contextmanager()* are also single use context managers, and will complain about the underlying generator failing to yield if an attempt is made to use them a second time:

```
>>> from contextlib import contextmanager
>>> @contextmanager
... def singleuse():
...     print("Before")
...     yield
...     print("After")
...
>>> cm = singleuse()
>>> with cm:
...     pass
...
Before
After
>>> with cm:
...     pass
...
Traceback (most recent call last):
    ...
RuntimeError: generator didn't yield
```

Reentrant context managers

More sophisticated context managers may be "reentrant". These context managers can not only be used in multiple `with` statements, but may also be used *inside* a `with` statement that is already using the same context manager.

threading.RLock is an example of a reentrant context manager, as are *suppress()*, *redirect_stdout()*, and *chdir()*. Here's a very simple example of reentrant use:

```
>>> from contextlib import redirect_stdout
>>> from io import StringIO
>>> stream = StringIO()
>>> write_to_stream = redirect_stdout(stream)
>>> with write_to_stream:
...     print("This is written to the stream rather than stdout")
...     with write_to_stream:
...         print("This is also written to the stream")
...
>>> print("This is written directly to stdout")
This is written directly to stdout
>>> print(stream.getvalue())
```

(continua na próxima página)

(continuação da página anterior)

```
This is written to the stream rather than stdout
This is also written to the stream
```

Real world examples of reentrancy are more likely to involve multiple functions calling each other and hence be far more complicated than this example.

Note also that being reentrant is *not* the same thing as being thread safe. *redirect_stdout()*, for example, is definitely not thread safe, as it makes a global modification to the system state by binding *sys.stdout* to a different stream.

Gerenciadores de contexto reutilizáveis

Distinct from both single use and reentrant context managers are "reusable" context managers (or, to be completely explicit, "reusable, but not reentrant" context managers, since reentrant context managers are also reusable). These context managers support being used multiple times, but will fail (or otherwise not work correctly) if the specific context manager instance has already been used in a containing with statement.

threading.Lock is an example of a reusable, but not reentrant, context manager (for a reentrant lock, it is necessary to use *threading.RLock* instead).

Another example of a reusable, but not reentrant, context manager is *ExitStack*, as it invokes *all* currently registered callbacks when leaving any with statement, regardless of where those callbacks were added:

```
>>> from contextlib import ExitStack
>>> stack = ExitStack()
>>> with stack:
...     stack.callback(print, "Callback: from first context")
...     print("Leaving first context")
...
Leaving first context
Callback: from first context
>>> with stack:
...     stack.callback(print, "Callback: from second context")
...     print("Leaving second context")
...
Leaving second context
Callback: from second context
>>> with stack:
...     stack.callback(print, "Callback: from outer context")
...     with stack:
...         stack.callback(print, "Callback: from inner context")
...         print("Leaving inner context")
...     print("Leaving outer context")
...
Leaving inner context
Callback: from inner context
Callback: from outer context
Leaving outer context
```

As the output from the example shows, reusing a single stack object across multiple with statements works correctly, but attempting to nest them will cause the stack to be cleared at the end of the innermost with statement, which is unlikely to be desirable behaviour.

Using separate *ExitStack* instances instead of reusing a single instance avoids that problem:

```
>>> from contextlib import ExitStack
>>> with ExitStack() as outer_stack:
...     outer_stack.callback(print, "Callback: from outer context")
...     with ExitStack() as inner_stack:
```

(continua na próxima página)

(continuação da página anterior)

```
...         inner_stack.callback(print, "Callback: from inner context")
...         print("Leaving inner context")
...     print("Leaving outer context")
...
Leaving inner context
Callback: from inner context
Leaving outer context
Callback: from outer context
```

29.9 abc — Classes base abstratas

Código-fonte: Lib/abc.py

Este módulo fornece a infraestrutura para definir *classes base abstratas* (ABCs, do inglês *abstract base class*) em Python, como delineado em **PEP 3119**; veja o PEP para entender o porquê isto foi adicionado ao Python. (Veja também **PEP 3141** e o módulo *numbers* sobre uma hierarquia de tipo para números baseado nas ABCs.)

O módulo *collections* tem algumas classes concretas que derivam de ABCs; essas podem, evidentemente, ser ainda mais derivadas. Além disso, o submódulo *collections.abc* tem algumas ABCs que podem ser usadas para testar se uma classe ou instância oferece uma interface particular, por exemplo, se é *hasheável* ou se é um *mapeamento*.

Este módulo fornece a metaclasse *ABCMeta* para definir ABCs e uma classe auxiliar *ABC* para alternativamente definir ABCs através de herança:

class abc.**ABC**

Uma classe auxiliar que tem *ABCMeta* como sua metaclasse. Com essa classe, uma classe base abstrata pode ser criada simplesmente derivando da ABC evitando às vezes confundir o uso da metaclasse, por exemplo:

```
from abc import ABC

class MyABC(ABC):
    pass
```

Note que o tipo da classe ABC ainda é *ABCMeta*, portanto herdar da ABC requer as precauções usuais a respeito do uso da metaclasse, pois herança múltipla pode levar a conflitos de metaclasse. Pode-se também definir uma classe base abstrata ao passar a palavra reservada metaclasse e usar ABCMeta diretamente, por exemplo:

```
from abc import ABCMeta

class MyABC(metaclass=ABCMeta):
    pass
```

Adicionado na versão 3.4.

class abc.**ABCMeta**

Metaclasse para definir Classe Base Abstrata (ABCs).

Use esta metaclasse para criar uma ABC. Uma ABC pode ser diretamente estendida, e então agir como uma classe misturada. Você também pode registrar classes concretas não relacionadas (até mesmo classes embutidas) e ABCs não relacionadas como "subclasses virtuais" – estas e suas descendentes serão consideradas subclasses da ABC de registro pela função embutida *issubclass()*, mas a ABC de registro não irá aparecer na MRO (Ordem de Resolução do Método) e nem as implementações do método definidas pela ABC de registro será chamável (nem mesmo via *super()*).[1]

Classes criadas com a metaclasse de ABCMeta tem o seguinte método:

[1] Programadores C++ devem notar que o conceito da classe base virtual do Python não é o mesmo que o de C++.

register(*subclass*)

Registra *subclass* como uma "subclasse virtual" desta ABC. Por exemplo:

```
from abc import ABC

class MyABC(ABC):
    pass

MyABC.register(tuple)

assert issubclass(tuple, MyABC)
assert isinstance((), MyABC)
```

Alterado na versão 3.3: Retorna a subclasse registrada, para permitir o uso como um decorador de classe.

Alterado na versão 3.4: Para detectar chamadas para `register()`, você pode usar a função *`get_cache_token()`*.

Você também pode sobrepor este método em uma classe base abstrata:

__subclasshook__(*subclass*)

(Deve obrigatoriamente ser definido como um método de classe.)

Verifica se *subclass* é considerada uma subclasse desta ABC. Isto significa que você pode personalizar ainda mais o comportamento de *`issubclass()`* sem a necessidade de chamar *`register()`* em toda classe que você queira considerar uma subclasse da ABC. (Este método de classe é chamado do método `__subclasscheck__()` da ABC.)

Este método deve retornar `True`, `False` ou *`NotImplemented`*. Se retornar `True`, *subclass* é considerada uma subclasse desta ABC. Se retornar `False`, *subclass* não é considerada uma subclasse desta ABC, mesmo que normalmente seria uma. Se retornar `NotImplemented`, a verificação da subclasse é continuada com o mecanismo usual.

Para uma demonstração destes conceitos, veja este exemplo de definição ABC:

```
class Foo:
    def __getitem__(self, index):
        ...
    def __len__(self):
        ...
    def get_iterator(self):
        return iter(self)

class MyIterable(ABC):

    @abstractmethod
    def __iter__(self):
        while False:
            yield None

    def get_iterator(self):
        return self.__iter__()

    @classmethod
    def __subclasshook__(cls, C):
        if cls is MyIterable:
            if any("__iter__" in B.__dict__ for B in C.__mro__):
                return True
        return NotImplemented

MyIterable.register(Foo)
```

A `MyIterable` da ABC define o método iterável padrão, `__iter__()`, como um método abstrato. A implementação dada aqui pode ainda ser chamada da subclasse. O método `get_iterator()` é também parte da classe base abstrata `MyIterable`, mas não precisa ser substituído nas classes derivadas não abstratas.

O método de classe *`__subclasshook__()`* definido aqui diz que qualquer classe que tenha um método `__iter__()` em seu `__dict__` (ou no de uma de suas classes base, acessados via lista `__mro__`) é considerada uma `MyIterable` também.

Finalmente, a última linha faz de `Foo` uma subclasse virtual da `MyIterable`, apesar de não definir um método `__iter__()` (ela usa o protocolo iterável antigo, definido em termos de `__len__()` e `__getitem__()`). Note que isto não fará o `get_iterator` disponível como um método de `Foo`, então ele é fornecido separadamente.

O módulo `abc` também fornece o seguinte decorador:

`@abc.`**`abstractmethod`**

Um decorador indicando métodos abstratos.

Usar este decorador requer que a metaclasse da classe seja *`ABCMeta`* ou seja derivada desta. Uma classe que tem uma metaclasse derivada de `ABCMeta` não pode ser instanciada, a menos que todos os seus métodos abstratos e propriedades estejam substituídos. Os métodos abstratos podem ser chamados usando qualquer um dos mecanismos normais de chamadas a 'super'. `abstractmethod()` pode ser usado para declarar métodos abstratos para propriedades e descritores.

Adicionar dinamicamente métodos abstratos a uma classe, ou tentar modificar o status de abstração de um método ou classe uma vez que estejam criados, só é suportado usando a função *`update_abstractmethods()`*. A `abstractmethod()` afeta apenas subclasses derivadas usando herança regular; "subclasses virtuais" registradas com o método *`register()`* da ABC não são afetadas.

Quando `abstractmethod()` é aplicado em combinação com outros descritores de método, ele deve ser aplicado como o decorador mais interno, como mostrado nos seguintes exemplos de uso:

```
class C(ABC):
    @abstractmethod
    def my_abstract_method(self, arg1):
        ...
    @classmethod
    @abstractmethod
    def my_abstract_classmethod(cls, arg2):
        ...
    @staticmethod
    @abstractmethod
    def my_abstract_staticmethod(arg3):
        ...

    @property
    @abstractmethod
    def my_abstract_property(self):
        ...
    @my_abstract_property.setter
    @abstractmethod
    def my_abstract_property(self, val):
        ...

    @abstractmethod
    def _get_x(self):
        ...
    @abstractmethod
    def _set_x(self, val):
        ...
    x = property(_get_x, _set_x)
```

Para que interopere corretamente com o maquinário da classe base abstrata, o descritor precisa identificar-se como abstrato usando `__isabstractmethod__`. No geral, este atributo deve ser `True` se algum dos métodos usados para compor o descritor for abstrato. Por exemplo, a *property* embutida do Python faz o equivalente a:

```
class Descriptor:
    ...
    @property
    def __isabstractmethod__(self):
        return any(getattr(f, '__isabstractmethod__', False) for
                   f in (self._fget, self._fset, self._fdel))
```

Nota

Diferente de métodos abstratos Java, esses métodos abstratos podem ter uma implementação. Esta implementação pode ser chamada via mecanismo da *super()* da classe que a substitui. Isto pode ser útil como um ponto final para uma super chamada em um framework que usa herança múltipla cooperativa.

O módulo `abc` também suporta os seguintes decoradores herdados:

`@abc.`**abstractclassmethod**

Adicionado na versão 3.2.

Descontinuado desde a versão 3.3: Agora é possível usar *classmethod* com *abstractmethod()*, tornando redundante este decorador.

Uma subclasse da *classmethod()* embutida, indicando um método de classe abstrato. Caso contrário, é similar à *abstractmethod()*.

Este caso especial está descontinuado, pois o decorador da *classmethod()* está agora corretamente identificado como abstrato quando aplicado a um método abstrato:

```
class C(ABC):
    @classmethod
    @abstractmethod
    def my_abstract_classmethod(cls, arg):
        ...
```

`@abc.`**abstractstaticmethod**

Adicionado na versão 3.2.

Descontinuado desde a versão 3.3: Agora é possível usar *staticmethod* com *abstractmethod()*, tornando redundante este decorador.

Uma subclasse da *staticmethod()* embutida, indicando um método estático abstrato. Caso contrário, ela é similar à *abstractmethod()*.

Este caso especial está descontinuado, pois o decorador da *staticmethod()* está agora corretamente identificado como abstrato quando aplicado a um método abstrato:

```
class C(ABC):
    @staticmethod
    @abstractmethod
    def my_abstract_staticmethod(arg):
        ...
```

`@abc.`**abstractproperty**

Descontinuado desde a versão 3.3: Agora é possível usar *property*, *property.getter()*, *property.setter()* e *property.deleter()* com *abstractmethod()*, tornando redundante este decorador.

Uma subclasse da *property()* embutida, indicando uma propriedade abstrata.

Este caso especial está descontinuado, pois o decorador da *property()* está agora corretamente identificado como abstrato quando aplicado a um método abstrato:

```
class C(ABC):
    @property
    @abstractmethod
    def my_abstract_property(self):
        ...
```

O exemplo acima define uma propriedade somente leitura; você também pode definir uma propriedade abstrata de leitura e escrita marcando apropriadamente um ou mais dos métodos subjacentes como abstratos:

```
class C(ABC):
    @property
    def x(self):
        ...

    @x.setter
    @abstractmethod
    def x(self, val):
        ...
```

Se apenas alguns componentes são abstratos, apenas estes componentes precisam ser atualizados para criar uma propriedade concreta em uma subclasse:

```
class D(C):
    @C.x.setter
    def x(self, val):
        ...
```

O módulo `abc` também fornece as seguintes funções:

`abc.`**`get_cache_token`**`()`

Retorna o token de cache da classe base abstrata atual.

O token é um objeto opaco (que suporta teste de igualdade) identificando a versão atual do cache da classe base abstrata para subclasses virtuais. O token muda a cada chamada ao *ABCMeta.register()* em qualquer CBA.

Adicionado na versão 3.4.

`abc.`**`update_abstractmethods`**`(`*cls*`)`

Uma função para recalcular o status de abstração de uma classe abstrata. Esta função deve ser chamada se os métodos abstratos de uma classe foram implementados ou alterados após sua criação. Normalmente, essa função deve ser chamada de dentro de um decorador de classe.

Retorna *cls*, para permitir o uso como decorador de classe.

Se *cls* não for uma instância de *ABCMeta*, não faz nada.

> **Nota**
>
> Esta função presume que as superclasses de *cls* já estão atualizadas. Ele não atualiza nenhuma subclasse.

Adicionado na versão 3.10.

29.10 atexit — Manipuladores de saída

O módulo *atexit* define funções para registrar e cancelar o registro de funções de limpeza. As funções assim registradas são executadas automaticamente após a conclusão normal do interpretador. O módulo *atexit* executa essas funções na ordem *reversa* na qual foram registradas; se você inscrever A, B e C, no momento do término do interpretador, eles serão executados na ordem C, B, A .

Nota: As funções registradas através deste módulo não são invocadas quando o programa é morto por um sinal não tratado pelo Python, quando um erro interno fatal do Python é detectado ou quando a função *os._exit()* é invocada.

Nota: O efeito de registrar ou cancelar o registro de funções dentro de uma função de limpeza é indefinido.

Alterado na versão 3.7: Quando usadas com os subinterpretadores de C-API, as funções registradas são locais para o interpretador em que foram registradas.

atexit.**register**(*func*, **args*, ***kwargs*)

Registre *func* como uma função a ser executada no término. Qualquer o argumento opcional que deve ser passado para *func* for passado como argumento para *register()*. É possível registrar mais ou menos a mesma função e argumentos.

Na terminação normal do programa (por exemplo, se *sys.exit()* for chamado ou a execução do módulo principal for concluída), todas as funções registradas serão chamadas por último, pela primeira ordem. A suposição é que os módulos de nível inferior normalmente serão importados antes dos módulos de nível mais alto e, portanto, devem ser limpos posteriormente.

Se uma exceção é levantada durante a execução dos manipuladores de saída, um traceback é impresso (a menos que *SystemExit* seja levantada) e as informações de exceção sejam salvas. Depois de todos os manipuladores de saída terem tido a chance de executar a última exceção a ser levantada, é levantada novamente.

Esta função retorna *func*, o que torna possível usá-la como um decorador.

> **Aviso**
>
> Iniciar novas threads ou chamar *os.fork()* de uma função registrada pode levar a uma condição de corrida entre os estados de thread de liberação de thread principal do tempo de execução do Python enquanto as rotinas internas *threading* ou o novo processo tentam usar esse estado. Isso pode levar a travamentos em vez de desligamento normal.

Alterado na versão 3.12: Tentativas de iniciar uma nova thread ou *os.fork()* um novo processo em uma função registrada agora leva a *RuntimeError*.

atexit.**unregister**(*func*)

Remove *func* da lista de funções a serem executadas no desligamento do interpretador. *unregister()* silenciosamente não faz nada se *func* não foi registrado anteriormente. Se *func* foi registrado mais de uma vez, cada ocorrência dessa função na pilha de chamada *atexit* será removida. Comparações de igualdade (==) são usadas internamente durante o cancelamento do registro, portanto, as referências de função não precisam ter identidades correspondentes.

> **Ver também**
>
> **Módulo *readline***
> Exemplo útil de *atexit* para ler e escrever arquivos de histórico de *readline*.

29.10.1 Exemplo do `atexit`

O exemplo simples a seguir demonstra como um módulo pode inicializar um contador de um arquivo quando ele é importado e salvar automaticamente o valor atualizado do contador quando o programa termina, sem depender que a aplicação faça uma chamada explícita nesse módulo na finalização.

```
try:
    with open('counterfile') as infile:
        _count = int(infile.read())
except FileNotFoundError:
    _count = 0

def incrcounter(n):
    global _count
    _count = _count + n

def savecounter():
    with open('counterfile', 'w') as outfile:
        outfile.write('%d' % _count)

import atexit

atexit.register(savecounter)
```

Os argumentos posicional e de palavra reservada também podem ser passados para *register()* para ser passada para a função registrada quando é chamada

```
def goodbye(name, adjective):
    print('Goodbye %s, it was %s to meet you.' % (name, adjective))

import atexit

atexit.register(goodbye, 'Donny', 'nice')
# ou:
atexit.register(goodbye, adjective='nice', name='Donny')
```

Utilizado como um *decorador*:

```
import atexit

@atexit.register
def goodbye():
    print('You are now leaving the Python sector.')
```

Isso só funciona com funções que podem ser invocadas sem argumentos.

29.11 `traceback` — Imprime ou recupera um traceback da pilha

Código-fonte: Lib/traceback.py

This module provides a standard interface to extract, format and print stack traces of Python programs. It is more flexible than the interpreter's default traceback display, and therefore makes it possible to configure certain aspects of the output. Finally, it contains a utility for capturing enough information about an exception to print it later, without the need to save a reference to the actual exception. Since exceptions can be the roots of large objects graph, this utility can significantly improve memory management.

The module uses traceback objects — these are objects of type *types.TracebackType*, which are assigned to the *__traceback__* field of *BaseException* instances.

Ver também

Módulo *faulthandler*
Usado para despejar tracebacks (situação da pilha de execução) do Python explicitamente, em uma falha, após um tempo limite ou em um sinal do usuário.

Module *pdb*
Interactive source code debugger for Python programs.

The module's API can be divided into two parts:

- Module-level functions offering basic functionality, which are useful for interactive inspection of exceptions and tracebacks.
- *TracebackException* class and its helper classes *StackSummary* and *FrameSummary*. These offer both more flexibility in the output generated and the ability to store the information necessary for later formatting without holding references to actual exception and traceback objects.

Adicionado na versão 3.13: A saída é colorizada por padrão e ela pode ser controlada usando variáveis de ambiente.

29.11.1 Funções de nível de módulo

traceback.**print_tb**(*tb*, *limit=None*, *file=None*)

Print up to *limit* stack trace entries from traceback object *tb* (starting from the caller's frame) if *limit* is positive. Otherwise, print the last `abs(limit)` entries. If *limit* is omitted or `None`, all entries are printed. If *file* is omitted or `None`, the output goes to *sys.stderr*; otherwise it should be an open *file* or *file-like object* to receive the output.

Nota

The meaning of the *limit* parameter is different than the meaning of *sys.tracebacklimit*. A negative *limit* value corresponds to a positive value of `sys.tracebacklimit`, whereas the behaviour of a positive *limit* value cannot be achieved with `sys.tracebacklimit`.

Alterado na versão 3.5: Added negative *limit* support.

traceback.**print_exception**(*exc*, /, [*value*, *tb*,]*limit=None*, *file=None*, *chain=True*)

Print exception information and stack trace entries from traceback object *tb* to *file*. This differs from *print_tb()* in the following ways:

- if *tb* is not `None`, it prints a header `Traceback (most recent call last):`
- it prints the exception type and *value* after the stack trace
- if *type(value)* is *SyntaxError* and *value* has the appropriate format, it prints the line where the syntax error occurred with a caret indicating the approximate position of the error.

Since Python 3.10, instead of passing *value* and *tb*, an exception object can be passed as the first argument. If *value* and *tb* are provided, the first argument is ignored in order to provide backwards compatibility.

The optional *limit* argument has the same meaning as for *print_tb()*. If *chain* is true (the default), then chained exceptions (the *__cause__* or *__context__* attributes of the exception) will be printed as well, like the interpreter itself does when printing an unhandled exception.

Alterado na versão 3.5: The *etype* argument is ignored and inferred from the type of *value*.

Alterado na versão 3.10: The *etype* parameter has been renamed to *exc* and is now positional-only.

traceback.**print_exc**(*limit=None*, *file=None*, *chain=True*)

This is a shorthand for `print_exception(sys.exception(), limit=limit, file=file, chain=chain)`.

traceback.**print_last**(*limit=None*, *file=None*, *chain=True*)

This is a shorthand for `print_exception(sys.last_exc, limit=limit, file=file, chain=chain)`. In general it will work only after an exception has reached an interactive prompt (see *sys.last_exc*).

traceback.**print_stack**(*f=None*, *limit=None*, *file=None*)

Print up to *limit* stack trace entries (starting from the invocation point) if *limit* is positive. Otherwise, print the last `abs(limit)` entries. If *limit* is omitted or `None`, all entries are printed. The optional *f* argument can be used to specify an alternate stack frame to start. The optional *file* argument has the same meaning as for *print_tb()*.

Alterado na versão 3.5: Added negative *limit* support.

traceback.**extract_tb**(*tb*, *limit=None*)

Return a *StackSummary* object representing a list of "pre-processed" stack trace entries extracted from the traceback object *tb*. It is useful for alternate formatting of stack traces. The optional *limit* argument has the same meaning as for *print_tb()*. A "pre-processed" stack trace entry is a *FrameSummary* object containing attributes *filename*, *lineno*, *name*, and *line* representing the information that is usually printed for a stack trace.

traceback.**extract_stack**(*f=None*, *limit=None*)

Extract the raw traceback from the current stack frame. The return value has the same format as for *extract_tb()*. The optional *f* and *limit* arguments have the same meaning as for *print_stack()*.

traceback.**print_list**(*extracted_list*, *file=None*)

Print the list of tuples as returned by *extract_tb()* or *extract_stack()* as a formatted stack trace to the given file. If *file* is `None`, the output is written to *sys.stderr*.

traceback.**format_list**(*extracted_list*)

Given a list of tuples or *FrameSummary* objects as returned by *extract_tb()* or *extract_stack()*, return a list of strings ready for printing. Each string in the resulting list corresponds to the item with the same index in the argument list. Each string ends in a newline; the strings may contain internal newlines as well, for those items whose source text line is not `None`.

traceback.**format_exception_only**(*exc*, */*, [*value*,]*, *show_group=False*)

Format the exception part of a traceback using an exception value such as given by *sys.last_value*. The return value is a list of strings, each ending in a newline. The list contains the exception's message, which is normally a single string; however, for *SyntaxError* exceptions, it contains several lines that (when printed) display detailed information about where the syntax error occurred. Following the message, the list contains the exception's *notes*.

Since Python 3.10, instead of passing *value*, an exception object can be passed as the first argument. If *value* is provided, the first argument is ignored in order to provide backwards compatibility.

When *show_group* is `True`, and the exception is an instance of *BaseExceptionGroup*, the nested exceptions are included as well, recursively, with indentation relative to their nesting depth.

Alterado na versão 3.10: The *etype* parameter has been renamed to *exc* and is now positional-only.

Alterado na versão 3.11: The returned list now includes any *notes* attached to the exception.

Alterado na versão 3.13: *show_group* parameter was added.

traceback.**format_exception**(*exc*, */*, [*value*, *tb*,]*limit=None*, *chain=True*)

Format a stack trace and the exception information. The arguments have the same meaning as the corresponding arguments to *print_exception()*. The return value is a list of strings, each ending in a newline and some containing internal newlines. When these lines are concatenated and printed, exactly the same text is printed as does *print_exception()*.

Alterado na versão 3.5: The *etype* argument is ignored and inferred from the type of *value*.

Alterado na versão 3.10: This function's behavior and signature were modified to match *print_exception()*.

traceback.**format_exc**(*limit=None*, *chain=True*)

This is like `print_exc(limit)` but returns a string instead of printing to a file.

traceback.**format_tb**(*tb*, *limit=None*)

A shorthand for `format_list(extract_tb(tb, limit))`.

traceback.**format_stack**(*f=None*, *limit=None*)

A shorthand for `format_list(extract_stack(f, limit))`.

traceback.**clear_frames**(*tb*)

Clears the local variables of all the stack frames in a traceback *tb* by calling the `clear()` method of each frame object.

Adicionado na versão 3.4.

traceback.**walk_stack**(*f*)

Walk a stack following `f.f_back` from the given frame, yielding the frame and line number for each frame. If *f* is `None`, the current stack is used. This helper is used with *`StackSummary.extract()`*.

Adicionado na versão 3.5.

Alterado na versão 3.14: This function previously returned a generator that would walk the stack when first iterated over. The generator returned now is the state of the stack when `walk_stack` is called.

traceback.**walk_tb**(*tb*)

Walk a traceback following `tb_next` yielding the frame and line number for each frame. This helper is used with *`StackSummary.extract()`*.

Adicionado na versão 3.5.

29.11.2 `TracebackException` Objects

Adicionado na versão 3.5.

`TracebackException` objects are created from actual exceptions to capture data for later printing. They offer a more lightweight method of storing this information by avoiding holding references to traceback and frame objects. In addition, they expose more options to configure the output compared to the module-level functions described above.

class traceback.**TracebackException**(*exc_type*, *exc_value*, *exc_traceback*, ***, *limit=None*, *lookup_lines=True*, *capture_locals=False*, *compact=False*, *max_group_width=15*, *max_group_depth=10*)

Capture an exception for later rendering. The meaning of *limit*, *lookup_lines* and *capture_locals* are as for the *`StackSummary`* class.

If *compact* is true, only data that is required by `TracebackException`'s *`format()`* method is saved in the class attributes. In particular, the *`__context__`* field is calculated only if *`__cause__`* is `None` and *`__suppress_context__`* is false.

Note that when locals are captured, they are also shown in the traceback.

max_group_width and *max_group_depth* control the formatting of exception groups (see *`BaseExceptionGroup`*). The depth refers to the nesting level of the group, and the width refers to the size of a single exception group's exceptions array. The formatted output is truncated when either limit is exceeded.

Alterado na versão 3.10: Adicionado o parâmetro *compact*.

Alterado na versão 3.11: Added the *max_group_width* and *max_group_depth* parameters.

__cause__

A `TracebackException` of the original *`__cause__`*.

__context__

A `TracebackException` of the original *`__context__`*.

exceptions

If `self` represents an *ExceptionGroup*, this field holds a list of `TracebackException` instances representing the nested exceptions. Otherwise it is `None`.

Adicionado na versão 3.11.

__suppress_context__

The *__suppress_context__* value from the original exception.

__notes__

The *__notes__* value from the original exception, or `None` if the exception does not have any notes. If it is not `None` is it formatted in the traceback after the exception string.

Adicionado na versão 3.11.

stack

A *StackSummary* representing the traceback.

exc_type

The class of the original traceback.

Descontinuado desde a versão 3.13.

exc_type_str

String display of the class of the original exception.

Adicionado na versão 3.13.

filename

For syntax errors - the file name where the error occurred.

lineno

For syntax errors - the line number where the error occurred.

end_lineno

For syntax errors - the end line number where the error occurred. Can be `None` if not present.

Adicionado na versão 3.10.

text

For syntax errors - the text where the error occurred.

offset

For syntax errors - the offset into the text where the error occurred.

end_offset

For syntax errors - the end offset into the text where the error occurred. Can be `None` if not present.

Adicionado na versão 3.10.

msg

For syntax errors - the compiler error message.

classmethod from_exception(*exc*, ***, *limit=None*, *lookup_lines=True*, *capture_locals=False*)

Capture an exception for later rendering. *limit*, *lookup_lines* and *capture_locals* are as for the *StackSummary* class.

Note that when locals are captured, they are also shown in the traceback.

print(***, *file=None*, *chain=True*)

Print to *file* (default `sys.stderr`) the exception information returned by *format()*.

Adicionado na versão 3.11.

format(*, *chain=True*)

Format the exception.

If *chain* is not `True`, *__cause__* and *__context__* will not be formatted.

The return value is a generator of strings, each ending in a newline and some containing internal newlines. *print_exception()* is a wrapper around this method which just prints the lines to a file.

format_exception_only(*, *show_group=False*)

Format the exception part of the traceback.

The return value is a generator of strings, each ending in a newline.

When *show_group* is `False`, the generator emits the exception's message followed by its notes (if it has any). The exception message is normally a single string; however, for *SyntaxError* exceptions, it consists of several lines that (when printed) display detailed information about where the syntax error occurred.

When *show_group* is `True`, and the exception is an instance of *BaseExceptionGroup*, the nested exceptions are included as well, recursively, with indentation relative to their nesting depth.

Alterado na versão 3.11: The exception's *notes* are now included in the output.

Alterado na versão 3.13: Added the *show_group* parameter.

29.11.3 `StackSummary` Objects

Adicionado na versão 3.5.

`StackSummary` objects represent a call stack ready for formatting.

class traceback.**StackSummary**

classmethod extract(*frame_gen*, *, *limit=None*, *lookup_lines=True*, *capture_locals=False*)

Construct a `StackSummary` object from a frame generator (such as is returned by *walk_stack()* or *walk_tb()*).

If *limit* is supplied, only this many frames are taken from *frame_gen*. If *lookup_lines* is `False`, the returned *FrameSummary* objects will not have read their lines in yet, making the cost of creating the `StackSummary` cheaper (which may be valuable if it may not actually get formatted). If *capture_locals* is `True` the local variables in each `FrameSummary` are captured as object representations.

Alterado na versão 3.12: Exceptions raised from *repr()* on a local variable (when *capture_locals* is `True`) are no longer propagated to the caller.

classmethod from_list(*a_list*)

Construct a `StackSummary` object from a supplied list of *FrameSummary* objects or old-style list of tuples. Each tuple should be a 4-tuple with *filename*, *lineno*, *name*, *line* as the elements.

format()

Returns a list of strings ready for printing. Each string in the resulting list corresponds to a single frame from the stack. Each string ends in a newline; the strings may contain internal newlines as well, for those items with source text lines.

For long sequences of the same frame and line, the first few repetitions are shown, followed by a summary line stating the exact number of further repetitions.

Alterado na versão 3.6: Long sequences of repeated frames are now abbreviated.

format_frame_summary(*frame_summary*)

Returns a string for printing one of the frames involved in the stack. This method is called for each *FrameSummary* object to be printed by *StackSummary.format()*. If it returns `None`, the frame is omitted from the output.

Adicionado na versão 3.11.

29.11.4 `FrameSummary` Objects

Adicionado na versão 3.5.

A `FrameSummary` object represents a single frame in a traceback.

class `traceback.`**FrameSummary**(*filename*, *lineno*, *name*, *, *lookup_line=True*, *locals=None*, *line=None*, *end_lineno=None*, *colno=None*, *end_colno=None*)

Represents a single frame in the traceback or stack that is being formatted or printed. It may optionally have a stringified version of the frame's locals included in it. If *lookup_line* is `False`, the source code is not looked up until the `FrameSummary` has the *line* attribute accessed (which also happens when casting it to a *tuple*). *line* may be directly provided, and will prevent line lookups happening at all. *locals* is an optional local variable mapping, and if supplied the variable representations are stored in the summary for later display.

`FrameSummary` instances have the following attributes:

filename

The filename of the source code for this frame. Equivalent to accessing `f.f_code.co_filename` on a frame object *f*.

lineno

The line number of the source code for this frame.

name

Equivalent to accessing `f.f_code.co_name` on a frame object *f*.

line

A string representing the source code for this frame, with leading and trailing whitespace stripped. If the source is not available, it is `None`.

end_lineno

The last line number of the source code for this frame. By default, it is set to `lineno` and indexation starts from 1.

Alterado na versão 3.13: The default value changed from `None` to `lineno`.

colno

The column number of the source code for this frame. By default, it is `None` and indexation starts from 0.

end_colno

The last column number of the source code for this frame. By default, it is `None` and indexation starts from 0.

29.11.5 Examples of Using the Module-Level Functions

This simple example implements a basic read-eval-print loop, similar to (but less useful than) the standard Python interactive interpreter loop. For a more complete implementation of the interpreter loop, refer to the *code* module.

```
import sys, traceback

def run_user_code(envdir):
    source = input(">>> ")
    try:
        exec(source, envdir)
    except Exception:
        print("Exception in user code:")
        print("-"*60)
        traceback.print_exc(file=sys.stdout)
        print("-"*60)

envdir = {}
```

(continua na próxima página)

(continuação da página anterior)

```
while True:
    run_user_code(envdir)
```

The following example demonstrates the different ways to print and format the exception and traceback:

```
import sys, traceback

def lumberjack():
    bright_side_of_life()

def bright_side_of_life():
    return tuple()[0]

try:
    lumberjack()
except IndexError as exc:
    print("*** print_tb:")
    traceback.print_tb(exc.__traceback__, limit=1, file=sys.stdout)
    print("*** print_exception:")
    traceback.print_exception(exc, limit=2, file=sys.stdout)
    print("*** print_exc:")
    traceback.print_exc(limit=2, file=sys.stdout)
    print("*** format_exc, first and last line:")
    formatted_lines = traceback.format_exc().splitlines()
    print(formatted_lines[0])
    print(formatted_lines[-1])
    print("*** format_exception:")
    print(repr(traceback.format_exception(exc)))
    print("*** extract_tb:")
    print(repr(traceback.extract_tb(exc.__traceback__)))
    print("*** format_tb:")
    print(repr(traceback.format_tb(exc.__traceback__)))
    print("*** tb_lineno:", exc.__traceback__.tb_lineno)
```

The output for the example would look similar to this:

```
*** print_tb:
  File "<doctest...>", line 10, in <module>
    lumberjack()
    ~~~~~~~~~~^^
*** print_exception:
Traceback (most recent call last):
  File "<doctest...>", line 10, in <module>
    lumberjack()
    ~~~~~~~~~~^^
  File "<doctest...>", line 4, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~^^
IndexError: tuple index out of range
*** print_exc:
Traceback (most recent call last):
  File "<doctest...>", line 10, in <module>
    lumberjack()
    ~~~~~~~~~~^^
  File "<doctest...>", line 4, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~^^
```

(continua na próxima página)

(continuação da página anterior)

```
IndexError: tuple index out of range
*** format_exc, first and last line:
Traceback (most recent call last):
IndexError: tuple index out of range
*** format_exception:
['Traceback (most recent call last):\n',
 '  File "<doctest default[0]>", line 10, in <module>\n    lumberjack()\n    ~~~~~~
↪~~~~^^\n',
 '  File "<doctest default[0]>", line 4, in lumberjack\n    bright_side_of_life()\
↪n    ~~~~~~~~~~~~~~~~~~~^^\n',
 '  File "<doctest default[0]>", line 7, in bright_side_of_life\n    return␣
↪tuple()[0]\n           ~~~~~~~^^^\n',
 'IndexError: tuple index out of range\n']
*** extract_tb:
[<FrameSummary file <doctest...>, line 10 in <module>>,
 <FrameSummary file <doctest...>, line 4 in lumberjack>,
 <FrameSummary file <doctest...>, line 7 in bright_side_of_life>]
*** format_tb:
['  File "<doctest default[0]>", line 10, in <module>\n    lumberjack()\n    ~~~~~~
↪~~~~^^\n',
 '  File "<doctest default[0]>", line 4, in lumberjack\n    bright_side_of_life()\
↪n    ~~~~~~~~~~~~~~~~~~~^^\n',
 '  File "<doctest default[0]>", line 7, in bright_side_of_life\n    return␣
↪tuple()[0]\n           ~~~~~~~^^^\n']
*** tb_lineno: 10
```

The following example shows the different ways to print and format the stack:

```
>>> import traceback
>>> def another_function():
...     lumberstack()
...
>>> def lumberstack():
...     traceback.print_stack()
...     print(repr(traceback.extract_stack()))
...     print(repr(traceback.format_stack()))
...
>>> another_function()
  File "<doctest>", line 10, in <module>
    another_function()
  File "<doctest>", line 3, in another_function
    lumberstack()
  File "<doctest>", line 6, in lumberstack
    traceback.print_stack()
[('<doctest>', 10, '<module>', 'another_function()'),
 ('<doctest>', 3, 'another_function', 'lumberstack()'),
 ('<doctest>', 7, 'lumberstack', 'print(repr(traceback.extract_stack()))')]
['  File "<doctest>", line 10, in <module>\n    another_function()\n',
 '  File "<doctest>", line 3, in another_function\n    lumberstack()\n',
 '  File "<doctest>", line 8, in lumberstack\n    print(repr(traceback.format_
↪stack()))\n']
```

This last example demonstrates the final few formatting functions:

```
>>> import traceback
>>> traceback.format_list([('spam.py', 3, '<module>', 'spam.eggs()'),
...                        ('eggs.py', 42, 'eggs', 'return "bacon"')])
```

(continua na próxima página)

(continuação da página anterior)

```
['  File "spam.py", line 3, in <module>\n    spam.eggs()\n',
 '  File "eggs.py", line 42, in eggs\n    return "bacon"\n']
>>> an_error = IndexError('tuple index out of range')
>>> traceback.format_exception_only(an_error)
['IndexError: tuple index out of range\n']
```

29.11.6 Examples of Using `TracebackException`

With the helper class, we have more options:

```
>>> import sys
>>> from traceback import TracebackException
>>>
>>> def lumberjack():
...     bright_side_of_life()
...
>>> def bright_side_of_life():
...     t = "bright", "side", "of", "life"
...     return t[5]
...
>>> try:
...     lumberjack()
... except IndexError as e:
...     exc = e
...
>>> try:
...     try:
...         lumberjack()
...     except:
...         1/0
... except Exception as e:
...     chained_exc = e
...
>>> # limit works as with the module-level functions
>>> TracebackException.from_exception(exc, limit=-2).print()
Traceback (most recent call last):
  File "<python-input-1>", line 6, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~^^
  File "<python-input-1>", line 10, in bright_side_of_life
    return t[5]
           ~^^^
IndexError: tuple index out of range

>>> # capture_locals adds local variables in frames
>>> TracebackException.from_exception(exc, limit=-2, capture_locals=True).print()
Traceback (most recent call last):
  File "<python-input-1>", line 6, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~^^
  File "<python-input-1>", line 10, in bright_side_of_life
    return t[5]
           ~^^^
    t = ("bright", "side", "of", "life")
IndexError: tuple index out of range
```

(continua na próxima página)

(continuação da página anterior)

```
>>> # The *chain* kwarg to print() controls whether chained
>>> # exceptions are displayed
>>> TracebackException.from_exception(chained_exc).print()
Traceback (most recent call last):
  File "<python-input-19>", line 4, in <module>
    lumberjack()
    ~~~~~~~~~~^^
  File "<python-input-8>", line 7, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~~^^
  File "<python-input-8>", line 11, in bright_side_of_life
    return t[5]
           ~^^^
IndexError: tuple index out of range

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "<python-input-19>", line 6, in <module>
    1/0
    ~^~
ZeroDivisionError: division by zero

>>> TracebackException.from_exception(chained_exc).print(chain=False)
Traceback (most recent call last):
  File "<python-input-19>", line 6, in <module>
    1/0
    ~^~
ZeroDivisionError: division by zero
```

29.12 `__future__` — Definições de instruções future

Código-fonte: Lib/__future__.py

Instruções na forma `from __future__ import feature` são chamadas de instruções future. Esses são casos especiais do compilador Python para permitir o uso de novos recursos do Python em módulos que contêm a instrução future antes da versão em que o recurso se torna padrão.

Embora o compilador Python dê um significado especial adicional a essas instruções future, elas ainda são executadas como qualquer outra instrução de importação, o módulo *__future__* existe e é tratado pelo sistema de importação da mesma forma que qualquer outro módulo Python. Isso serve a três propósitos:

- Para evitar confundir as ferramentas existentes que analisam as instruções de importação e esperam encontrar os módulos que estão importando.
- Para documentar quando as mudanças incompatíveis foram introduzidas, e quando elas serão — ou foram — obrigatórias. Esta é uma forma de documentação executável e pode ser inspecionada programaticamente através da importação *__future__* e examinando seus conteúdos.
- Para garantir que instruções future sejam executadas em versões anteriores a Python 2.1, pelo menos, processe exceções de tempo de execução (a importação de *__future__* falhará, porque não havia nenhum módulo desse nome antes de 2.1).

29.12.1 Conteúdo do módulo

Nenhuma descrição de característica será excluída de *__future__*. Desde a sua introdução no Python 2.1, os seguintes recursos encontraram o caminho para o idioma usando esse mecanismo:

característica	opcional em	obrigatório em	efeito
__future__.**nested_scopes**	2.1.0b1	2.2	**PEP 227**: *Statically Nested Scopes*
__future__.**generators**	2.2.0a1	2.3	**PEP 255**: *Simple Generators*
__future__.**division**	2.2.0a2	3.0	**PEP 238**: *Changing the Division Operator*
__future__.**absolute_import**	2.5.0a1	3.0	**PEP 328**: *Imports: Multi-Line and Absolute/Relative*
__future__.**with_statement**	2.5.0a1	2.6	**PEP 343**: *The "with" Statement*
__future__.**print_function**	2.6.0a2	3.0	**PEP 3105**: *Make print a function*
__future__.**unicode_literals**	2.6.0a2	3.0	**PEP 3112**: *Bytes literals in Python 3000*
__future__.**generator_stop**	3.5.0b1	3.7	**PEP 479**: *StopIteration handling inside generators*
__future__.**annotations**	3.7.0b1	Nunca[1]	**PEP 563**: *Avaliação adiada de anotações*, **PEP 649**: *Avaliação diferida de anotações usando descritores*

class __future__.**_Feature**

Cada instrução em `__future__.py` é da forma:

```
FeatureName = _Feature(OptionalRelease, MandatoryRelease,
                       CompilerFlag)
```

Onde, normalmente, *OptionalRelease* é inferior a *MandatoryRelease*, e ambos são tuplas de 5 entradas da mesma forma que *sys.version_info*:

```
(PY_MAJOR_VERSION, # o 2 em 2.1.0a3; um int
 PY_MINOR_VERSION, # o 1; um int
 PY_MICRO_VERSION, # o 0; um int
```

(continua na próxima página)

[1] `from __future__ import annotations` estava programado para se tornar obrigatório no Python 3.10, mas a alteração foi adiada e, por fim, cancelada. Este recurso será eventualmente descontinuado e removido. Veja **PEP 649** e **PEP 749**.

(continuação da página anterior)

```
 PY_RELEASE_LEVEL, # "alpha", "beta", "candidate" ou "final"; uma string
 PY_RELEASE_SERIAL # o 3; um int
)
```

_Feature.**getOptionalRelease**()

OptionalRelease registra o primeiro lançamento no qual o recurso foi aceito.

_Feature.**getMandatoryRelease**()

No caso de um *MandatoryRelease* que ainda não ocorreu, *MandatoryRelease* prevê o lançamento em que o recurso se tornará parte da linguagem.

Senão *MandatoryRelease* registra quando o recurso se tornou parte do idioma; Em versões em ou depois disso, os módulos não precisam mais de uma instrução future para usar o recurso em questão, mas podem continuar a usar essas importações.

MandatoryRelease também pode ser `None`, o que significa que uma característica planejada foi descartada ou que isso ainda não está decidido.

_Feature.**compiler_flag**

CompilerFlag é o sinalizador (bitfield) que deve ser passado no quarto argumento para a função embutida *compile()* para habilitar o recurso no código compilado dinamicamente. Este sinalizador é armazenado no atributo *_Feature.compiler_flag* em instâncias de *_Feature*.

Ver também

future
Como o compilador trata as importações de future.

PEP 236 - De volta ao __future__
A proposta original para o mecanismo do __future__.

29.13 gc — Interface para o coletor de lixo

Este módulo fornece uma interface para o coletor de lixo opcional. Ele disponibiliza a habilidade de desabilitar o coletor, ajustar a frequência da coleção, e configurar as opções de depuração. Ele também fornece acesso para objetos inacessíveis que o coletor encontra mas não pode "limpar". Como o coletor complementa a contagem de referência já usada em Python, você pode desabilitar o coletor se você tem certeza que o seu programa não cria ciclos de referência. A coleta automática pode ser desabilitada pela chamada `gc.disable()`. Para depurar um programa vazando, chame `gc.set_debug(gc.DEBUG_LEAK)`. Perceba que isto inclui `gc.DEBUG_SAVEALL`, fazendo com que objetos coletados pelo coletor de lixo sejam salvos para inspeção em gc.garbage.

O módulo *gc* fornece as seguintes funções:

gc.**enable**()

Habilita a coleta de lixo automática.

gc.**disable**()

Desabilita a coleta de lixo automática.

gc.**isenabled**()

Retorna `True` se a coleta automática estiver habilitada.

gc.**collect**(*generation=2*)

Realiza uma coleta. O argumento opcional *generation* pode ser um inteiro especificando qual geração coletar (de 0 a 2). Uma exceção *ValueError* é levantada se o número de geração for inválido. A soma dos objetos coletados e dos objetos incoletáveis é retornada.

Chamar `gc.collect(0)` executará uma coleta de GC na geração mais jovem.

Chamar `gc.collect(1)` executará uma coleta de GC na geração mais jovem e um incremento na geração antiga.

Chamar `gc.collect(2)` ou `gc.collect()` executa uma coleta completa

As listas livres mantidas para vários tipos embutidos são limpas sempre que uma coleta completa ou coleta da geração mais alta (2) é executada. Nem todos os itens em algumas listas livres podem ser liberados devido à implementação particular, em particular *float*.

O efeito de chamar `gc.collect()` enquanto o interpretador já está realizando uma coleta é indefinido.

Alterado na versão 3.14: `generation=1` executa um incremento de coleta.

`gc.`**`set_debug`**(*flags*)

Define os sinalizadores de depuração da coleta de lixo. As informações de depuração serão gravadas em `sys.stderr`. Veja abaixo uma lista de sinalizadores de depuração que podem ser combinados usando operações de bit para controlar a depuração.

`gc.`**`get_debug`**()

Retorna os sinalizadores de depuração atualmente definidos.

`gc.`**`get_objects`**(*generation=None*)

Retorna uma lista de todos os objetos rastreados pelo coletor, excluindo a lista retornada. Se *generation* não for `None`, retorna apenas os objetos da seguinte forma:

- 0: Todos os objetos da geração jovem
- 1: Nenhum objeto, pois não há geração 1 (desde o Python 3.14)
- 2: Todos os objetos da geração antiga

Alterado na versão 3.8: Novo parâmetro *generation*.

Alterado na versão 3.14: A geração 1 foi removida

Levanta um *evento de auditoria* `gc.get_objects` com o argumento `generation`.

`gc.`**`get_stats`**()

Retorna uma lista de três dicionários por geração contendo estatísticas de coleta desde o início do interpretador. O número de chaves pode mudar no futuro, mas atualmente cada dicionário conterá os seguintes itens:

- `collections` é o número de vezes que esta geração foi coletada;
- `collected` é o número total de objetos coletados nesta geração;
- `uncollectable` é o número total de objetos que foram considerados incobráveis (e, portanto, movidos para a lista *garbage*) dentro desta geração.

Adicionado na versão 3.4.

`gc.`**`set_threshold`**(*threshold0*[, *threshold1*[, *threshold2*]])

Define os limites de coleta de lixo (a frequência de coleta). Definir *threshold0* como zero desativa a coleta.

O GC classifica os objetos em duas gerações, dependendo se sobreviveram ou não a uma coleção. Os novos objetos são colocados na geração jovem. Se um objeto sobreviver a uma coleção, ele é movido para a geração antiga.

Para decidir quando executar, o coletor registra o número de alocações e desalocações de objetos desde a última coleta. Quando o número de alocações menos o número de desalocações excede *threshold0*, a coleta é iniciada. Para cada coleta, todos os objetos da geração jovem e uma fração da geração antiga são coletados.

Na construção com threads livres, o aumento no uso de memória do processo também é verificado antes da execução do coletor. Se o uso de memória não tiver aumentado 10% desde a última coleta e o número líquido de alocações de objetos não tiver excedido 40 vezes o limite 0, a coleta não será executada.

A fração da geração antiga coletada é **inversamente** proporcional a *threshold1*. Quanto maior o *threshold1*, mais lentos os objetos da geração antiga são coletados. Para o valor padrão de 10, 1% da geração antiga é escaneado durante cada coleta.

threshold2 é ignorado.

Consulte o design do coletor de lixo para mais informações.

Alterado na versão 3.14: *threshold2* é ignorado

`gc.get_count()`

Retorna as contagens da coleta atual como uma tupla de `(count0, count1, count2)`.

`gc.get_threshold()`

Retorna os limites da coleta atual como uma tupla de `(threshold0, threshold1, threshold2)`.

`gc.get_referrers(*objs)`

Retorna a lista de objetos que se referem diretamente a qualquer um dos objs. Esta função localizará apenas os contêineres que suportam coleta de lixo; tipos de extensão que se referem a outros objetos, mas não suportam coleta de lixo, não serão encontrados.

Observe que os objetos que já foram desreferenciados, mas que vivem em ciclos e ainda não foram coletados pelo coletor de lixo podem ser listados entre os referenciadores resultantes. Para obter apenas os objetos atualmente ativos, chame *`collect()`* antes de chamar *`get_referrers()`*.

Aviso

Deve-se tomar cuidado ao usar objetos retornados por *`get_referrers()`* porque alguns deles ainda podem estar em construção e, portanto, em um estado temporariamente inválido. Evite usar *`get_referrers()`* para qualquer finalidade que não seja depuração.

Levanta um *evento de auditoria* `gc.get_referrers` com o argumento `objs`.

`gc.get_referents(*objs)`

Retorna uma lista de objetos diretamente referenciados por qualquer um dos argumentos. Os referentes retornados são aqueles objetos visitados pelos métodos a nível do C `tp_traverse` dos argumentos (se houver), e podem não ser todos os objetos diretamente alcançáveis. Os métodos `tp_traverse` são suportados apenas por objetos que suportam coleta de lixo e são necessários apenas para visitar objetos que possam estar envolvidos em um ciclo. Assim, por exemplo, se um número inteiro pode ser acessado diretamente de um argumento, esse objeto inteiro pode ou não aparecer na lista de resultados.

Levanta um *evento de auditoria* `gc.get_referents` com o argumento `objs`.

`gc.is_tracked(obj)`

Retorna `True` se o objeto está atualmente rastreado pelo coletor de lixo, `False` caso contrário. Como regra geral, as instâncias de tipos atômicos não são rastreadas e as instâncias de tipos não atômicos (contêineres, objetos definidos pelo usuário…) são. No entanto, algumas otimizações específicas do tipo podem estar presentes para suprimir a pegada do coletor de lixo de instâncias simples (por exemplo, dicts contendo apenas chaves e valores atômicos):

```
>>> gc.is_tracked(0)
False
>>> gc.is_tracked("a")
False
>>> gc.is_tracked([])
True
>>> gc.is_tracked({})
False
>>> gc.is_tracked({"a": 1})
True
```

Adicionado na versão 3.1.

gc.**is_finalized**(*obj*)

Retorna `True` se o objeto fornecido foi finalizado pelo coletor de lixo, `False` caso contrário.

```
>>> x = None
>>> class Lazarus:
...     def __del__(self):
...         global x
...         x = self
...
>>> lazarus = Lazarus()
>>> gc.is_finalized(lazarus)
False
>>> del lazarus
>>> gc.is_finalized(x)
True
```

Adicionado na versão 3.9.

gc.**freeze**()

Congela todos os objetos rastreados pelo coletor de lixo; move-os para uma geração permanente e ignora-os em todas as coleções futuras.

Se um processo for `fork()` sem `exec()`, evitar cópia em gravação (copy-on-write) desnecessário em processos filho maximizará o compartilhamento de memória e reduzirá o uso geral de memória. Isso requer evitar a criação de "buracos" liberados nas páginas de memória no processo pai e garantir que as coleções GC nos processos filho não toquem no contador `gc_refs` de objetos de vida longa originados no processo pai. Para realizar ambos, chame `gc.disable()` no início do processo pai, `gc.freeze()` logo antes de `fork()` e `gc.enable()` no início em processos filhos.

Adicionado na versão 3.7.

gc.**unfreeze**()

Descongela os objetos na geração permanente, coloca-os de volta na geração mais antiga.

Adicionado na versão 3.7.

gc.**get_freeze_count**()

Retorna o número de objetos na geração permanente.

Adicionado na versão 3.7.

As seguintes variáveis são fornecidas para acesso somente leitura (você pode alterar os valores, mas não deve revinculá-los):

gc.**garbage**

Uma lista de objetos que o coletor considerou inacessíveis, mas não puderam ser liberados (objetos não coletáveis). A partir do Python 3.4, esta lista deve estar vazia na maioria das vezes, exceto ao usar instâncias de tipos de extensão C com um slot `tp_del` não-`NULL`.

Se *DEBUG_SAVEALL* for definido, todos os objetos inacessíveis serão adicionados a esta lista ao invés de liberados.

Alterado na versão 3.2: Se esta lista não estiver vazia no *desligamento do interpretador*, um *ResourceWarning* é emitido, que é silencioso por padrão. Se *DEBUG_UNCOLLECTABLE* for definido, além disso, todos os objetos não coletáveis serão impressos.

Alterado na versão 3.4: Seguindo a **PEP 442**, objetos com um método `__del__()` não vão mais para *gc.garbage*.

gc.**callbacks**

Uma lista de retornos de chamada que serão invocados pelo coletor de lixo antes e depois da coleta. As funções de retorno serão chamadas com dois argumentos, *phase* e *info*.

phase pode ser um dos dois valores:

"start": A coleta de lixo está prestes a começar.

"stop": A coleta de lixo terminou.

info é um ditado que fornece mais informações para a função de retorno. As seguintes chaves estão atualmente definidas:

"generation": A geração mais antiga sendo coletada.

"collected": Quando *phase* é "stop", o número de objetos coletados com sucesso.

"uncollectable": Quando *phase* é "stop", o número de objetos que não puderam ser coletados e foram colocados em `garbage`.

As aplicações podem adicionar suas próprias funções de retorno a essa lista. Os principais casos de uso são:

Reunir estatísticas sobre coleta de lixo, como com que frequência várias gerações são coletadas e quanto tempo leva a coleta.

Permitindo que os aplicativos identifiquem e limpem seus próprios tipos não colecionáveis quando eles aparecem em `garbage`.

Adicionado na versão 3.3.

As seguintes constantes são fornecidas para uso com `set_debug()`:

`gc.`**`DEBUG_STATS`**

Imprimir estatísticas durante a coleta. Esta informação pode ser útil ao sintonizar a frequência de coleta.

`gc.`**`DEBUG_COLLECTABLE`**

Imprimir informações sobre objetos colecionáveis encontrados.

`gc.`**`DEBUG_UNCOLLECTABLE`**

Imprime informações de objetos não colecionáveis encontrados (objetos que não são alcançáveis, mas não podem ser liberados pelo coletor). Esses objetos serão adicionados à lista `garbage`.

Alterado na versão 3.2: Imprime também o conteúdo da lista `garbage` em *desligamento do interpretador*, se não estiver vazia.

`gc.`**`DEBUG_SAVEALL`**

Quando definido, todos os objetos inacessíveis encontrados serão anexados ao *lixo* em vez de serem liberados. Isso pode ser útil para depurar um programa com vazamento.

`gc.`**`DEBUG_LEAK`**

Os sinalizadores de depuração necessários para o coletor imprimir informações sobre um programa com vazamento (igual a `DEBUG_COLLECTABLE | DEBUG_UNCOLLECTABLE | DEBUG_SAVEALL`).

29.14 `inspect` — Inspecione objetos vivos

Código-fonte: Lib/inspect.py

O módulo `inspect` fornece diversas funções úteis para ajudar a obter informações sobre objetos ativos, como módulos, classes, métodos, funções, tracebacks, objetos quadro e objetos código. Por exemplo, ele pode ajudar você a examinar o conteúdo de uma classe, recuperar o código-fonte de um método, extrair e formatar a lista de argumentos de uma função ou obter todas as informações necessárias para exibir um traceback detalhado.

Há quatro tipos principais de serviços fornecidos por este módulo: verificação de tipos, obtenção de código-fonte, inspeção de classes e funções e exame da pilha do interpretador.

29.14.1 Tipos e membros

A função *getmembers()* recupera os membros de um objeto, como uma classe ou módulo. As funções cujos nomes começam com "is" são fornecidas principalmente como opções convenientes para o segundo argumento de *getmembers()*. Elas também ajudam a determinar quando você pode esperar encontrar os seguintes atributos especiais (consulte import-mod-attrs para atributos de módulo):

Tipo	Atributo	Descrição
class	__doc__	string de documentação
	__name__	nome com o qual esta classe foi definida
	__qualname__	nome qualificado
	__module__	nome do módulo no qual esta classe foi definida
	__type_params__	Uma tupla contendo os parâmetros de tipo de uma classe genérica
método	__doc__	string de documentação
	__name__	nome com o qual este método foi definido
	__qualname__	nome qualificado
	__func__	objeto função contendo implementação de método
	__self__	instância para o qual este método está vinculado, ou `None`.
	__module__	nome do módulo no qual este método foi definido
função	__doc__	string de documentação
	__name__	nome com o qual esta função foi definida
	__qualname__	nome qualificado
	__code__	objeto código contendo *bytecode* da função compilada
	__defaults__	tupla de quaisquer valores padrão para parâmetros posicionais ou de parâmetros nomeados
	__kwdefaults__	mapeamento de quaisquer valores padrão para parâmetros somente-nomeados
	__globals__	espaço de nomes global no qual esta função foi definida
	__builtins__	espaço de nomes de métodos e funções embutidas
	__annotations__	mapeamento de nomes de parâmetros para anotações; a chave `"return"` é reservada para
	__type_params__	Uma tupla contendo os parâmetros de tipo de uma função genérica
	__module__	nome do módulo no qual esta função foi definida
traceback	tb_frame	objeto quadro neste nível
	tb_lasti	índice da última instrução tentada no bytecode
	tb_lineno	número da linha atual no código-fonte do Python
	tb_next	próximo objeto de traceback interno (chamado por este nível)
quadro	f_back	próximo objeto de quadro externo (chamador deste quadro)
	f_builtins	espaço de nomes de métodos e funçõess embutidas visto por este quadro
	f_code	objeto código sendo executado neste quadro
	f_globals	espaço de nomes global visto por este quadro
	f_lasti	índice da última instrução tentada no bytecode
	f_lineno	número da linha atual no código-fonte do Python
	f_locals	espaço de nomes local visto por este quadro
	f_generator	retorna o gerador ou objeto de corrotina que possui este quadro, ou `None` se o quadro for d
	f_trace	função de rastreamento para este quadro, ou `None`
	f_trace_lines	indica se um evento de rastreamento é acionado para cada linha de origem
	f_trace_opcodes	indica se eventos por código de operação são solicitados
	clear()	usado para limpar todas as referências a variáveis locais
código	co_argcount	número de argumentos (não incluindo argumentos somente-nomeados, args * ou **)
	co_code	string de bytecode compilado bruto
	co_cellvars	tuple of names of cell variables (referenced by containing scopes)
	co_consts	tuple of constants used in the bytecode
	co_filename	name of file in which this code object was created
	co_firstlineno	number of first line in Python source code
	co_flags	bitmap of `CO_*` flags, read more *here*
	co_lnotab	encoded mapping of line numbers to bytecode indices
	co_freevars	tuple of names of free variables (referenced via a function's closure)
	co_posonlyargcount	number of positional only arguments
	co_kwonlyargcount	number of keyword only arguments (not including ** arg)

continua

Tabela 2 – continuação da página anterior

Tipo	Atributo	Descrição
	co_name	name with which this code object was defined
	co_qualname	fully qualified name with which this code object was defined
	co_names	tuple of names other than arguments and function locals
	co_nlocals	number of local variables
	co_stacksize	virtual machine stack space required
	co_varnames	tuple of names of arguments and local variables
	co_lines()	returns an iterator that yields successive bytecode ranges
	co_positions()	returns an iterator of source code positions for each bytecode instruction
	replace()	returns a copy of the code object with new values
gerador	__name__	nome
	__qualname__	nome qualificado
	gi_frame	quadro
	gi_running	is the generator running?
	gi_suspended	is the generator suspended?
	gi_code	código
	gi_yieldfrom	object being iterated by `yield from`, or `None`
async generator	__name__	nome
	__qualname__	nome qualificado
	ag_await	object being awaited on, or `None`
	ag_frame	quadro
	ag_running	is the generator running?
	ag_code	código
corrotina	__name__	nome
	__qualname__	nome qualificado
	cr_await	object being awaited on, or `None`
	cr_frame	quadro
	cr_running	is the coroutine running?
	cr_code	código
	cr_origin	where coroutine was created, or `None`. See *sys.set_coroutine_origin_tracking_*
builtin	__doc__	string de documentação
	__name__	original name of this function or method
	__qualname__	nome qualificado
	__self__	instance to which a method is bound, or `None`

Alterado na versão 3.5: Add `__qualname__` and `gi_yieldfrom` attributes to generators.

The `__name__` attribute of generators is now set from the function name, instead of the code name, and it can now be modified.

Alterado na versão 3.7: Add `cr_origin` attribute to coroutines.

Alterado na versão 3.10: Add `__builtins__` attribute to functions.

Alterado na versão 3.14: Add `f_generator` attribute to frames.

`inspect.`**`getmembers`**(*object*[, *predicate*])

Return all the members of an object in a list of `(name, value)` pairs sorted by name. If the optional *predicate* argument—which will be called with the `value` object of each member—is supplied, only members for which the predicate returns a true value are included.

> **Nota**
>
> *`getmembers()`* will only return class attributes defined in the metaclass when the argument is a class and those attributes have been listed in the metaclass' custom `__dir__()`.

`inspect.`**`getmembers_static`**(*object*[, *predicate*])

Return all the members of an object in a list of `(name, value)` pairs sorted by name without triggering dynamic lookup via the descriptor protocol, __getattr__ or __getattribute__. Optionally, only return members that satisfy a given predicate.

> **Nota**
>
> *`getmembers_static()`* may not be able to retrieve all members that getmembers can fetch (like dynamically created attributes) and may find members that getmembers can't (like descriptors that raise AttributeError). It can also return descriptor objects instead of instance members in some cases.

Adicionado na versão 3.11.

`inspect.`**`getmodulename`**(*path*)

Return the name of the module named by the file *path*, without including the names of enclosing packages. The file extension is checked against all of the entries in *`importlib.machinery.all_suffixes()`*. If it matches, the final path component is returned with the extension removed. Otherwise, `None` is returned.

Note that this function *only* returns a meaningful name for actual Python modules - paths that potentially refer to Python packages will still return `None`.

Alterado na versão 3.3: The function is based directly on *`importlib`*.

`inspect.`**`ismodule`**(*object*)

Return `True` if the object is a module.

`inspect.`**`isclass`**(*object*)

Return `True` if the object is a class, whether built-in or created in Python code.

`inspect.`**`ismethod`**(*object*)

Return `True` if the object is a bound method written in Python.

`inspect.`**`ispackage`**(*object*)

Return `True` if the object is a *package*.

Adicionado na versão 3.14.

`inspect.`**`isfunction`**(*object*)

Return `True` if the object is a Python function, which includes functions created by a *lambda* expression.

`inspect.`**`isgeneratorfunction`**(*object*)

Return `True` if the object is a Python generator function.

Alterado na versão 3.8: Functions wrapped in *`functools.partial()`* now return `True` if the wrapped function is a Python generator function.

Alterado na versão 3.13: Functions wrapped in *`functools.partialmethod()`* now return `True` if the wrapped function is a Python generator function.

`inspect.`**`isgenerator`**(*object*)

Return `True` if the object is a generator.

`inspect.`**`iscoroutinefunction`**(*object*)

Return `True` if the object is a *coroutine function* (a function defined with an `async def` syntax), a *`functools.partial()`* wrapping a *coroutine function*, or a sync function marked with *`markcoroutinefunction()`*.

Adicionado na versão 3.5.

Alterado na versão 3.8: Functions wrapped in *`functools.partial()`* now return `True` if the wrapped function is a *coroutine function*.

Alterado na versão 3.12: Sync functions marked with *`markcoroutinefunction()`* now return `True`.

Alterado na versão 3.13: Functions wrapped in *functools.partialmethod()* now return `True` if the wrapped function is a *coroutine function*.

inspect.**markcoroutinefunction**(*func*)

Decorator to mark a callable as a *coroutine function* if it would not otherwise be detected by *iscoroutinefunction()*.

This may be of use for sync functions that return a *coroutine*, if the function is passed to an API that requires *iscoroutinefunction()*.

When possible, using an `async def` function is preferred. Also acceptable is calling the function and testing the return with *iscoroutine()*.

Adicionado na versão 3.12.

inspect.**iscoroutine**(*object*)

Return `True` if the object is a *coroutine* created by an `async def` function.

Adicionado na versão 3.5.

inspect.**isawaitable**(*object*)

Return `True` if the object can be used in `await` expression.

Can also be used to distinguish generator-based coroutines from regular generators:

```
import types

def gen():
    yield
@types.coroutine
def gen_coro():
    yield

assert not isawaitable(gen())
assert isawaitable(gen_coro())
```

Adicionado na versão 3.5.

inspect.**isasyncgenfunction**(*object*)

Return `True` if the object is an *asynchronous generator* function, for example:

```
>>> async def agen():
...     yield 1
...
>>> inspect.isasyncgenfunction(agen)
True
```

Adicionado na versão 3.6.

Alterado na versão 3.8: Functions wrapped in *functools.partial()* now return `True` if the wrapped function is an *asynchronous generator* function.

Alterado na versão 3.13: Functions wrapped in *functools.partialmethod()* now return `True` if the wrapped function is a *coroutine function*.

inspect.**isasyncgen**(*object*)

Return `True` if the object is an *asynchronous generator iterator* created by an *asynchronous generator* function.

Adicionado na versão 3.6.

inspect.**istraceback**(*object*)

Return `True` if the object is a traceback.

inspect.**isframe**(*object*)

Return `True` if the object is a frame.

inspect.**iscode**(*object*)

Return `True` if the object is a code.

inspect.**isbuiltin**(*object*)

Return `True` if the object is a built-in function or a bound built-in method.

inspect.**ismethodwrapper**(*object*)

Return `True` if the type of object is a *MethodWrapperType*.

These are instances of *MethodWrapperType*, such as `__str__()`, `__eq__()` and `__repr__()`.

Adicionado na versão 3.11.

inspect.**isroutine**(*object*)

Return `True` if the object is a user-defined or built-in function or method.

inspect.**isabstract**(*object*)

Return `True` if the object is an abstract base class.

inspect.**ismethoddescriptor**(*object*)

Return `True` if the object is a method descriptor, but not if *ismethod()*, *isclass()*, *isfunction()* or *isbuiltin()* are true.

This, for example, is true of `int.__add__`. An object passing this test has a `__get__()` method, but not a `__set__()` method or a `__delete__()` method. Beyond that, the set of attributes varies. A *__name__* attribute is usually sensible, and *__doc__* often is.

Methods implemented via descriptors that also pass one of the other tests return `False` from the *ismethoddescriptor()* test, simply because the other tests promise more – you can, e.g., count on having the `__func__` attribute (etc) when an object passes *ismethod()*.

Alterado na versão 3.13: This function no longer incorrectly reports objects with `__get__()` and `__delete__()`, but not `__set__()`, as being method descriptors (such objects are data descriptors, not method descriptors).

inspect.**isdatadescriptor**(*object*)

Return `True` if the object is a data descriptor.

Data descriptors have a `__set__` or a `__delete__` method. Examples are properties (defined in Python), getsets, and members. The latter two are defined in C and there are more specific tests available for those types, which is robust across Python implementations. Typically, data descriptors will also have *__name__* and `__doc__` attributes (properties, getsets, and members have both of these attributes), but this is not guaranteed.

inspect.**isgetsetdescriptor**(*object*)

Return `True` if the object is a getset descriptor.

Detalhes da implementação do CPython: getsets are attributes defined in extension modules via `PyGetSetDef` structures. For Python implementations without such types, this method will always return `False`.

inspect.**ismemberdescriptor**(*object*)

Return `True` if the object is a member descriptor.

Detalhes da implementação do CPython: Member descriptors are attributes defined in extension modules via `PyMemberDef` structures. For Python implementations without such types, this method will always return `False`.

29.14.2 Retrieving source code

inspect.**getdoc**(*object*)

Get the documentation string for an object, cleaned up with *cleandoc()*. If the documentation string for an object is not provided and the object is a class, a method, a property or a descriptor, retrieve the documentation string from the inheritance hierarchy. Return None if the documentation string is invalid or missing.

Alterado na versão 3.5: Strings de documentação agora são herdadas, se não forem sobrescritas.

inspect.**getcomments**(*object*)

Return in a single string any lines of comments immediately preceding the object's source code (for a class, function, or method), or at the top of the Python source file (if the object is a module). If the object's source code is unavailable, return None. This could happen if the object has been defined in C or the interactive shell.

inspect.**getfile**(*object*)

Retorna o nome do arquivo (texto ou binário) no qual um objeto foi definido. Isso falhará com um *TypeError* se o objeto for um módulo, classe ou função embutidos.

inspect.**getmodule**(*object*)

Try to guess which module an object was defined in. Return None if the module cannot be determined.

inspect.**getsourcefile**(*object*)

Return the name of the Python source file in which an object was defined or None if no way can be identified to get the source. This will fail with a *TypeError* if the object is a built-in module, class, or function.

inspect.**getsourcelines**(*object*)

Return a list of source lines and starting line number for an object. The argument may be a module, class, method, function, traceback, frame, or code object. The source code is returned as a list of the lines corresponding to the object and the line number indicates where in the original source file the first line of code was found. An *OSError* is raised if the source code cannot be retrieved. A *TypeError* is raised if the object is a built-in module, class, or function.

Alterado na versão 3.3: *OSError* is raised instead of *IOError*, now an alias of the former.

inspect.**getsource**(*object*)

Return the text of the source code for an object. The argument may be a module, class, method, function, traceback, frame, or code object. The source code is returned as a single string. An *OSError* is raised if the source code cannot be retrieved. A *TypeError* is raised if the object is a built-in module, class, or function.

Alterado na versão 3.3: *OSError* is raised instead of *IOError*, now an alias of the former.

inspect.**cleandoc**(*doc*)

Clean up indentation from docstrings that are indented to line up with blocks of code.

All leading whitespace is removed from the first line. Any leading whitespace that can be uniformly removed from the second line onwards is removed. Empty lines at the beginning and end are subsequently removed. Also, all tabs are expanded to spaces.

29.14.3 Introspecting callables with the Signature object

Adicionado na versão 3.3.

The *Signature* object represents the call signature of a callable object and its return annotation. To retrieve a Signature object, use the signature() function.

inspect.**signature**(*callable*, ***, *follow_wrapped=True*, *globals=None*, *locals=None*, *eval_str=False*, *annotation_format=Format.VALUE*)

Return a *Signature* object for the given *callable*:

```
>>> from inspect import signature
>>> def foo(a, *, b:int, **kwargs):
...     pass
```

(continua na próxima página)

(continuação da página anterior)

```
>>> sig = signature(foo)

>>> str(sig)
'(a, *, b: int, **kwargs)'

>>> str(sig.parameters['b'])
'b: int'

>>> sig.parameters['b'].annotation
<class 'int'>
```

Accepts a wide range of Python callables, from plain functions and classes to *functools.partial()* objects.

If some of the annotations are strings (e.g., because `from __future__ import annotations` was used), *signature()* will attempt to automatically un-stringize the annotations using *annotationlib.get_annotations()*. The *globals*, *locals*, and *eval_str* parameters are passed into `annotationlib.get_annotations()` when resolving the annotations; see the documentation for `annotationlib.get_annotations()` for instructions on how to use these parameters. A member of the *annotationlib.Format* enum can be passed to the *annotation_format* parameter to control the format of the returned annotations. For example, use `annotation_format=annotationlib.Format.STRING` to return annotations in string format.

Raises *ValueError* if no signature can be provided, and *TypeError* if that type of object is not supported. Also, if the annotations are stringized, and *eval_str* is not false, the `eval()` call(s) to un-stringize the annotations in *annotationlib.get_annotations()* could potentially raise any kind of exception.

A slash (/) in the signature of a function denotes that the parameters prior to it are positional-only. For more info, see the FAQ entry on positional-only parameters.

Alterado na versão 3.5: The *follow_wrapped* parameter was added. Pass `False` to get a signature of *callable* specifically (`callable.__wrapped__` will not be used to unwrap decorated callables.)

Alterado na versão 3.10: The *globals*, *locals*, and *eval_str* parameters were added.

Alterado na versão 3.14: The *annotation_format* parameter was added.

Nota

Some callables may not be introspectable in certain implementations of Python. For example, in CPython, some built-in functions defined in C provide no metadata about their arguments.

Detalhes da implementação do CPython: If the passed object has a `__signature__` attribute, we may use it to create the signature. The exact semantics are an implementation detail and are subject to unannounced changes. Consult the source code for current semantics.

class inspect.**Signature**(*parameters=None*, *, *return_annotation=Signature.empty*)

A `Signature` object represents the call signature of a function and its return annotation. For each parameter accepted by the function it stores a *Parameter* object in its *parameters* collection.

The optional *parameters* argument is a sequence of *Parameter* objects, which is validated to check that there are no parameters with duplicate names, and that the parameters are in the right order, i.e. positional-only first, then positional-or-keyword, and that parameters with defaults follow parameters without defaults.

The optional *return_annotation* argument can be an arbitrary Python object. It represents the "return" annotation of the callable.

`Signature` objects are *immutable*. Use *Signature.replace()* or *copy.replace()* to make a modified copy.

Alterado na versão 3.5: `Signature` objects are now picklable and *hashable*.

empty

A special class-level marker to specify absence of a return annotation.

parameters

An ordered mapping of parameters' names to the corresponding *Parameter* objects. Parameters appear in strict definition order, including keyword-only parameters.

Alterado na versão 3.7: Python only explicitly guaranteed that it preserved the declaration order of keyword-only parameters as of version 3.7, although in practice this order had always been preserved in Python 3.

return_annotation

The "return" annotation for the callable. If the callable has no "return" annotation, this attribute is set to *Signature.empty*.

bind(**args*, ***kwargs*)

Create a mapping from positional and keyword arguments to parameters. Returns *BoundArguments* if `*args` and `**kwargs` match the signature, or raises a *TypeError*.

bind_partial(**args*, ***kwargs*)

Works the same way as *Signature.bind()*, but allows the omission of some required arguments (mimics *functools.partial()* behavior.) Returns *BoundArguments*, or raises a *TypeError* if the passed arguments do not match the signature.

replace(**[, parameters][, return_annotation]*)

Create a new *Signature* instance based on the instance *replace()* was invoked on. It is possible to pass different *parameters* and/or *return_annotation* to override the corresponding properties of the base signature. To remove `return_annotation` from the copied `Signature`, pass in *Signature.empty*.

```
>>> def test(a, b):
...     pass
...
>>> sig = signature(test)
>>> new_sig = sig.replace(return_annotation="new return anno")
>>> str(new_sig)
"(a, b) -> 'new return anno'"
```

Signature objects are also supported by the generic function *copy.replace()*.

format(***, *max_width=None*, *quote_annotation_strings=True*)

Create a string representation of the *Signature* object.

If *max_width* is passed, the method will attempt to fit the signature into lines of at most *max_width* characters. If the signature is longer than *max_width*, all parameters will be on separate lines.

If *quote_annotation_strings* is False, *annotations* in the signature are displayed without opening and closing quotation marks if they are strings. This is useful if the signature was created with the *STRING* format or if `from __future__ import annotations` was used.

Adicionado na versão 3.13.

Alterado na versão 3.14: The *unquote_annotations* parameter was added.

classmethod from_callable(*obj*, ***, *follow_wrapped=True*, *globals=None*, *locals=None*, *eval_str=False*)

Return a *Signature* (or its subclass) object for a given callable *obj*.

This method simplifies subclassing of *Signature*:

```
class MySignature(Signature):
    pass
sig = MySignature.from_callable(sum)
assert isinstance(sig, MySignature)
```

Its behavior is otherwise identical to that of *signature()*.

Adicionado na versão 3.5.

Alterado na versão 3.10: The *globals*, *locals*, and *eval_str* parameters were added.

class inspect.**Parameter**(*name*, *kind*, *, *default=Parameter.empty*, *annotation=Parameter.empty*)

Objetos `Parameter` são *imutáveis*. Em vez de modificar um objeto `Parameter`, você pode usar *Parameter.replace()* ou *copy.replace()* para criar uma cópia modificada.

Alterado na versão 3.5: Objetos Parameter agora são compatíveis com pickle e *hasheável*.

empty

Um marcador de nível de classe especial para especificar a ausência de valor padrão e anotação.

name

O nome do parâmetro como uma string. O nome deve ser um identificador Python válido.

CPython gera nomes de parâmetro implícito na forma `.0` no código objeto usado para implementar compreensão e expressão geradora.

Alterado na versão 3.6: Esses nomes de parâmetro agora são expostos por este módulo como nomes como `implicit0`.

default

O padrão valor para o parâmetro. Se o parâmetro não tiver padrão valor, esse atributo será definido como *Parameter.empty*.

annotation

A anotação para o parâmetro. Se o parâmetro não tiver anotação, esse atributo será definido como *Parameter.empty*.

kind

Descreve como valores de argumento são vinculados ao parâmetro. Os possíveis valores são acessíveis via *Parameter* (como `Parameter.KEYWORD_ONLY`) e tem suporte para comparação e ordenação, na seguinte ordem:

Nome	Significado
POSITIONAL_ONLY	Valor deve ser fornecido como argumento posicional. Parâmetros somente-posicionais são aqueles que aparecem antes de uma entrada `/` (se presente) em uma definição de função Python.
POSITIONAL_OR_KEYWORD	Valor pode ser fornecido como argumento posicional ou nomeado (esse é o comportamento de ligação padrão para funções implementadas em Python).
`*VAR_POSITIONAL*`	Uma tupla de argumentos posicionais que não estão vinculados a nenhum outro parâmetro. Isso corresponde a um parâmetro `*args` em uma definição de função Python.
KEYWORD_ONLY	Valor deve ser fornecido como argumento nomeado. Parâmetros somente-nomeados são aqueles que aparecem após uma entrada `*` ou `*args` em uma definição de função Python.
VAR_KEYWORD	Um dicionário de argumentos nomeados que não estão vinculados a nenhum outro parâmetro. Isso corresponde a um parâmetro `**kwargs` em uma definição de função Python.

Exemplo: exibir todos argumentos somente-nomeados sem padrão valor:

```
>>> def foo(a, b, *, c, d=10):
...     pass

>>> sig = signature(foo)
```

(continua na próxima página)

(continuação da página anterior)

```
>>> for param in sig.parameters.values():
...     if (param.kind == param.KEYWORD_ONLY and
...                         param.default is param.empty):
...         print('Parameter:', param)
Parameter: c
```

kind.**description**

Descreve um valor enumerado de *Parameter.kind*.

Adicionado na versão 3.8.

Exemplo: exibir as descrições de todos os argumento:

```
>>> def foo(a, b, *, c, d=10):
...     pass

>>> sig = signature(foo)
>>> for param in sig.parameters.values():
...     print(param.kind.description)
positional or keyword
positional or keyword
keyword-only
keyword-only
```

replace(**[, name][, kind][, default][, annotation]*)

Cria uma nova instância de *Parameter* com base na instância em que o método foi chamado. Para substituir um atributo de `Parameter`, passe o argumento correspondente. Para remover um valor padrão ou/e uma anotação de um `Parameter`, passe *Parameter.empty*.

```
>>> from inspect import Parameter
>>> param = Parameter('foo', Parameter.KEYWORD_ONLY, default=42)
>>> str(param)
'foo=42'

>>> str(param.replace()) # Criará uma cópia rasa de 'param'
'foo=42'

>>> str(param.replace(default=Parameter.empty, annotation='spam'))
"foo: 'spam'"
```

Objetos *Parameter* também são suportados pela função genérica *copy.replace()*.

Alterado na versão 3.4: Em Python 3.3, era permitido objetos *Parameter* terem `name` definido como `None` se `kind` estivesse definido como `POSITIONAL_ONLY`. Isso não é mais permitido.

class inspect.**BoundArguments**

Result of a *Signature.bind()* or *Signature.bind_partial()* call. Holds the mapping of arguments to the function's parameters.

arguments

A mutable mapping of parameters' names to arguments' values. Contains only explicitly bound arguments. Changes in *arguments* will reflect in *args* and *kwargs*.

Should be used in conjunction with *Signature.parameters* for any argument processing purposes.

> **Nota**
>
> Arguments for which *Signature.bind()* or *Signature.bind_partial()* relied on a default value are skipped. However, if needed, use *BoundArguments.apply_defaults()* to add them.

Alterado na versão 3.9: *arguments* is now of type *dict*. Formerly, it was of type *collections.OrderedDict*.

args

A tuple of positional arguments values. Dynamically computed from the *arguments* attribute.

kwargs

A dict of keyword arguments values. Dynamically computed from the *arguments* attribute. Arguments that can be passed positionally are included in *args* instead.

signature

A reference to the parent *Signature* object.

apply_defaults()

Set default values for missing arguments.

For variable-positional arguments (`*args`) the default is an empty tuple.

For variable-keyword arguments (`**kwargs`) the default is an empty dict.

```
>>> def foo(a, b='ham', *args): pass
>>> ba = inspect.signature(foo).bind('spam')
>>> ba.apply_defaults()
>>> ba.arguments
{'a': 'spam', 'b': 'ham', 'args': ()}
```

Adicionado na versão 3.5.

The *args* and *kwargs* properties can be used to invoke functions:

```
def test(a, *, b):
    ...

sig = signature(test)
ba = sig.bind(10, b=20)
test(*ba.args, **ba.kwargs)
```

Ver também

PEP 362 - Function Signature Object.
The detailed specification, implementation details and examples.

29.14.4 Classes e funções

inspect.**getclasstree**(*classes*, *unique=False*)

Arrange the given list of classes into a hierarchy of nested lists. Where a nested list appears, it contains classes derived from the class whose entry immediately precedes the list. Each entry is a 2-tuple containing a class and a tuple of its base classes. If the *unique* argument is true, exactly one entry appears in the returned structure for each class in the given list. Otherwise, classes using multiple inheritance and their descendants will appear multiple times.

inspect.**getfullargspec**(*func*)

Get the names and default values of a Python function's parameters. A *named tuple* is returned:

```
FullArgSpec(args, varargs, varkw, defaults, kwonlyargs, kwonlydefaults,
annotations)
```

args is a list of the positional parameter names. *varargs* is the name of the `*` parameter or `None` if arbitrary positional arguments are not accepted. *varkw* is the name of the `**` parameter or `None` if arbitrary keyword arguments are not accepted. *defaults* is an *n*-tuple of default argument values corresponding to the last *n*

positional parameters, or `None` if there are no such defaults defined. *kwonlyargs* is a list of keyword-only parameter names in declaration order. *kwonlydefaults* is a dictionary mapping parameter names from *kwonlyargs* to the default values used if no argument is supplied. *annotations* is a dictionary mapping parameter names to annotations. The special key `"return"` is used to report the function return value annotation (if any).

Note that *`signature()`* and *Signature Object* provide the recommended API for callable introspection, and support additional behaviours (like positional-only arguments) that are sometimes encountered in extension module APIs. This function is retained primarily for use in code that needs to maintain compatibility with the Python 2 `inspect` module API.

Alterado na versão 3.4: This function is now based on *`signature()`*, but still ignores `__wrapped__` attributes and includes the already bound first parameter in the signature output for bound methods.

Alterado na versão 3.6: This method was previously documented as deprecated in favour of *`signature()`* in Python 3.5, but that decision has been reversed in order to restore a clearly supported standard interface for single-source Python 2/3 code migrating away from the legacy `getargspec()` API.

Alterado na versão 3.7: Python only explicitly guaranteed that it preserved the declaration order of keyword-only parameters as of version 3.7, although in practice this order had always been preserved in Python 3.

`inspect.`**`getargvalues`**(*frame*)

Get information about arguments passed into a particular frame. A *named tuple* `ArgInfo(args, varargs, keywords, locals)` is returned. *args* is a list of the argument names. *varargs* and *keywords* are the names of the `*` and `**` arguments or `None`. *locals* is the locals dictionary of the given frame.

> **Nota**
>
> This function was inadvertently marked as deprecated in Python 3.5.

`inspect.`**`formatargvalues`**(*args*[, *varargs*, *varkw*, *locals*, *formatarg*, *formatvarargs*, *formatvarkw*, *formatvalue*])

Format a pretty argument spec from the four values returned by *`getargvalues()`*. The format* arguments are the corresponding optional formatting functions that are called to turn names and values into strings.

> **Nota**
>
> This function was inadvertently marked as deprecated in Python 3.5.

`inspect.`**`getmro`**(*cls*)

Return a tuple of class cls's base classes, including cls, in method resolution order. No class appears more than once in this tuple. Note that the method resolution order depends on cls's type. Unless a very peculiar user-defined metatype is in use, cls will be the first element of the tuple.

`inspect.`**`getcallargs`**(*func*, */*, **args*, ***kwds*)

Bind the *args* and *kwds* to the argument names of the Python function or method *func*, as if it was called with them. For bound methods, bind also the first argument (typically named `self`) to the associated instance. A dict is returned, mapping the argument names (including the names of the `*` and `**` arguments, if any) to their values from *args* and *kwds*. In case of invoking *func* incorrectly, i.e. whenever `func(*args, **kwds)` would raise an exception because of incompatible signature, an exception of the same type and the same or similar message is raised. For example:

```
>>> from inspect import getcallargs
>>> def f(a, b=1, *pos, **named):
...     pass
...
>>> getcallargs(f, 1, 2, 3) == {'a': 1, 'named': {}, 'b': 2, 'pos': (3,)}
True
>>> getcallargs(f, a=2, x=4) == {'a': 2, 'named': {'x': 4}, 'b': 1, 'pos': ()}
```

(continua na próxima página)

(continuação da página anterior)

```
True
>>> getcallargs(f)
Traceback (most recent call last):
...
TypeError: f() missing 1 required positional argument: 'a'
```

Adicionado na versão 3.2.

Descontinuado desde a versão 3.5: Use *Signature.bind()* and *Signature.bind_partial()* instead.

inspect.**getclosurevars**(*func*)

Get the mapping of external name references in a Python function or method *func* to their current values. A *named tuple* `ClosureVars(nonlocals, globals, builtins, unbound)` is returned. *nonlocals* maps referenced names to lexical closure variables, *globals* to the function's module globals and *builtins* to the builtins visible from the function body. *unbound* is the set of names referenced in the function that could not be resolved at all given the current module globals and builtins.

TypeError is raised if *func* is not a Python function or method.

Adicionado na versão 3.3.

inspect.**unwrap**(*func*, *, *stop=None*)

Get the object wrapped by *func*. It follows the chain of `__wrapped__` attributes returning the last object in the chain.

stop is an optional callback accepting an object in the wrapper chain as its sole argument that allows the unwrapping to be terminated early if the callback returns a true value. If the callback never returns a true value, the last object in the chain is returned as usual. For example, *signature()* uses this to stop unwrapping if any object in the chain has a `__signature__` attribute defined.

ValueError is raised if a cycle is encountered.

Adicionado na versão 3.4.

inspect.**get_annotations**(*obj*, *, *globals=None*, *locals=None*, *eval_str=False*, *format=annotationlib.Format.VALUE*)

Calcula o dicionário de anotações para um objeto.

This is an alias for *annotationlib.get_annotations()*; see the documentation of that function for more information.

Cuidado

Esta função pode executar código arbitrário contido em anotações. Consulte *Security implications of introspecting annotations* para obter mais informações.

Adicionado na versão 3.10.

Alterado na versão 3.14: This function is now an alias for *annotationlib.get_annotations()*. Calling it as `inspect.get_annotations` will continue to work.

29.14.5 A pilha to interpretador

Some of the following functions return *FrameInfo* objects. For backwards compatibility these objects allow tuple-like operations on all attributes except `positions`. This behavior is considered deprecated and may be removed in the future.

class inspect.**FrameInfo**

frame

The frame object that the record corresponds to.

filename

The file name associated with the code being executed by the frame this record corresponds to.

lineno

The line number of the current line associated with the code being executed by the frame this record corresponds to.

function

The function name that is being executed by the frame this record corresponds to.

code_context

A list of lines of context from the source code that's being executed by the frame this record corresponds to.

index

The index of the current line being executed in the *code_context* list.

positions

A *dis.Positions* object containing the start line number, end line number, start column offset, and end column offset associated with the instruction being executed by the frame this record corresponds to.

Alterado na versão 3.5: Return a *named tuple* instead of a *tuple*.

Alterado na versão 3.11: `FrameInfo` is now a class instance (that is backwards compatible with the previous *named tuple*).

class inspect.**Traceback**

filename

The file name associated with the code being executed by the frame this traceback corresponds to.

lineno

The line number of the current line associated with the code being executed by the frame this traceback corresponds to.

function

The function name that is being executed by the frame this traceback corresponds to.

code_context

A list of lines of context from the source code that's being executed by the frame this traceback corresponds to.

index

The index of the current line being executed in the *code_context* list.

positions

A *dis.Positions* object containing the start line number, end line number, start column offset, and end column offset associated with the instruction being executed by the frame this traceback corresponds to.

Alterado na versão 3.11: `Traceback` is now a class instance (that is backwards compatible with the previous *named tuple*).

Nota

Keeping references to frame objects, as found in the first element of the frame records these functions return, can cause your program to create reference cycles. Once a reference cycle has been created, the lifespan of all objects which can be accessed from the objects which form the cycle can become much longer even if Python's optional cycle detector is enabled. If such cycles must be created, it is important to ensure they are explicitly broken to avoid the delayed destruction of objects and increased memory consumption which occurs.

Though the cycle detector will catch these, destruction of the frames (and local variables) can be made deterministic by removing the cycle in a `finally` clause. This is also important if the cycle detector was disabled when Python was compiled or using *gc.disable()*. For example:

```
def handle_stackframe_without_leak():
    frame = inspect.currentframe()
    try:
        # do something with the frame
    finally:
        del frame
```

If you want to keep the frame around (for example to print a traceback later), you can also break reference cycles by using the `frame.clear()` method.

The optional *context* argument supported by most of these functions specifies the number of lines of context to return, which are centered around the current line.

`inspect.`**`getframeinfo`**(*frame*, *context=1*)

Get information about a frame or traceback object. A *Traceback* object is returned.

Alterado na versão 3.11: A *Traceback* object is returned instead of a named tuple.

`inspect.`**`getouterframes`**(*frame*, *context=1*)

Get a list of *FrameInfo* objects for a frame and all outer frames. These frames represent the calls that lead to the creation of *frame*. The first entry in the returned list represents *frame*; the last entry represents the outermost call on *frame*'s stack.

Alterado na versão 3.5: A list of *named tuples* `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned.

Alterado na versão 3.11: A list of *FrameInfo* objects is returned.

`inspect.`**`getinnerframes`**(*traceback*, *context=1*)

Get a list of *FrameInfo* objects for a traceback's frame and all inner frames. These frames represent calls made as a consequence of *frame*. The first entry in the list represents *traceback*; the last entry represents where the exception was raised.

Alterado na versão 3.5: A list of *named tuples* `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned.

Alterado na versão 3.11: A list of *FrameInfo* objects is returned.

`inspect.`**`currentframe`**()

Return the frame object for the caller's stack frame.

Detalhes da implementação do CPython: This function relies on Python stack frame support in the interpreter, which isn't guaranteed to exist in all implementations of Python. If running in an implementation without Python stack frame support this function returns `None`.

`inspect.`**`stack`**(*context=1*)

Return a list of *FrameInfo* objects for the caller's stack. The first entry in the returned list represents the caller; the last entry represents the outermost call on the stack.

Alterado na versão 3.5: A list of *named tuples* `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned.

Alterado na versão 3.11: A list of *FrameInfo* objects is returned.

`inspect.`**`trace`**(*context=1*)

Return a list of *FrameInfo* objects for the stack between the current frame and the frame in which an exception currently being handled was raised in. The first entry in the list represents the caller; the last entry represents where the exception was raised.

Alterado na versão 3.5: A list of *named tuples* `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned.

Alterado na versão 3.11: A list of *`FrameInfo`* objects is returned.

29.14.6 Fetching attributes statically

Both *`getattr()`* and *`hasattr()`* can trigger code execution when fetching or checking for the existence of attributes. Descriptors, like properties, will be invoked and `__getattr__()` and `__getattribute__()` may be called.

For cases where you want passive introspection, like documentation tools, this can be inconvenient. *`getattr_static()`* has the same signature as *`getattr()`* but avoids executing code when it fetches attributes.

`inspect.`**`getattr_static`**(*obj*, *attr*, *default=None*)

Retrieve attributes without triggering dynamic lookup via the descriptor protocol, `__getattr__()` or `__getattribute__()`.

Note: this function may not be able to retrieve all attributes that getattr can fetch (like dynamically created attributes) and may find attributes that getattr can't (like descriptors that raise AttributeError). It can also return descriptors objects instead of instance members.

If the instance `__dict__` is shadowed by another member (for example a property) then this function will be unable to find instance members.

Adicionado na versão 3.2.

`getattr_static()` does not resolve descriptors, for example slot descriptors or getset descriptors on objects implemented in C. The descriptor object is returned instead of the underlying attribute.

You can handle these with code like the following. Note that for arbitrary getset descriptors invoking these may trigger code execution:

```
# example code for resolving the builtin descriptor types
class _foo:
    __slots__ = ['foo']

slot_descriptor = type(_foo.foo)
getset_descriptor = type(type(open(__file__)).name)
wrapper_descriptor = type(str.__dict__['__add__'])
descriptor_types = (slot_descriptor, getset_descriptor, wrapper_descriptor)

result = getattr_static(some_object, 'foo')
if type(result) in descriptor_types:
    try:
        result = result.__get__()
    except AttributeError:
        # descriptors can raise AttributeError to
        # indicate there is no underlying value
        # in which case the descriptor itself will
        # have to do
        pass
```

29.14.7 Current State of Generators, Coroutines, and Asynchronous Generators

When implementing coroutine schedulers and for other advanced uses of generators, it is useful to determine whether a generator is currently executing, is waiting to start or resume or execution, or has already terminated. *`getgeneratorstate()`* allows the current state of a generator to be determined easily.

`inspect.`**`getgeneratorstate`**(*generator*)

Get current state of a generator-iterator.

Possible states are:

- GEN_CREATED: Waiting to start execution.
- GEN_RUNNING: Currently being executed by the interpreter.
- GEN_SUSPENDED: Currently suspended at a yield expression.
- GEN_CLOSED: Execution has completed.

Adicionado na versão 3.2.

inspect.**getcoroutinestate**(*coroutine*)

Get current state of a coroutine object. The function is intended to be used with coroutine objects created by `async def` functions, but will accept any coroutine-like object that has `cr_running` and `cr_frame` attributes.

Possible states are:

- CORO_CREATED: Waiting to start execution.
- CORO_RUNNING: Currently being executed by the interpreter.
- CORO_SUSPENDED: Currently suspended at an await expression.
- CORO_CLOSED: Execution has completed.

Adicionado na versão 3.5.

inspect.**getasyncgenstate**(*agen*)

Get current state of an asynchronous generator object. The function is intended to be used with asynchronous iterator objects created by `async def` functions which use the `yield` statement, but will accept any asynchronous generator-like object that has `ag_running` and `ag_frame` attributes.

Possible states are:

- AGEN_CREATED: Waiting to start execution.
- AGEN_RUNNING: Currently being executed by the interpreter.
- AGEN_SUSPENDED: Currently suspended at a yield expression.
- AGEN_CLOSED: Execution has completed.

Adicionado na versão 3.12.

The current internal state of the generator can also be queried. This is mostly useful for testing purposes, to ensure that internal state is being updated as expected:

inspect.**getgeneratorlocals**(*generator*)

Get the mapping of live local variables in *generator* to their current values. A dictionary is returned that maps from variable names to values. This is the equivalent of calling *locals()* in the body of the generator, and all the same caveats apply.

If *generator* is a *generator* with no currently associated frame, then an empty dictionary is returned. *TypeError* is raised if *generator* is not a Python generator object.

Detalhes da implementação do CPython: This function relies on the generator exposing a Python stack frame for introspection, which isn't guaranteed to be the case in all implementations of Python. In such cases, this function will always return an empty dictionary.

Adicionado na versão 3.3.

inspect.**getcoroutinelocals**(*coroutine*)

This function is analogous to *getgeneratorlocals()*, but works for coroutine objects created by `async def` functions.

Adicionado na versão 3.5.

inspect.**getasyncgenlocals**(*agen*)

This function is analogous to *getgeneratorlocals()*, but works for asynchronous generator objects created by `async def` functions which use the `yield` statement.

Adicionado na versão 3.12.

29.14.8 Code Objects Bit Flags

Python code objects have a `co_flags` attribute, which is a bitmap of the following flags:

inspect.**CO_OPTIMIZED**

The code object is optimized, using fast locals.

inspect.**CO_NEWLOCALS**

If set, a new dict will be created for the frame's `f_locals` when the code object is executed.

inspect.**CO_VARARGS**

The code object has a variable positional parameter (`*args`-like).

inspect.**CO_VARKEYWORDS**

The code object has a variable keyword parameter (`**kwargs`-like).

inspect.**CO_NESTED**

The flag is set when the code object is a nested function.

inspect.**CO_GENERATOR**

The flag is set when the code object is a generator function, i.e. a generator object is returned when the code object is executed.

inspect.**CO_COROUTINE**

The flag is set when the code object is a coroutine function. When the code object is executed it returns a coroutine object. See **PEP 492** for more details.

Adicionado na versão 3.5.

inspect.**CO_ITERABLE_COROUTINE**

The flag is used to transform generators into generator-based coroutines. Generator objects with this flag can be used in `await` expression, and can `yield from` coroutine objects. See **PEP 492** for more details.

Adicionado na versão 3.5.

inspect.**CO_ASYNC_GENERATOR**

The flag is set when the code object is an asynchronous generator function. When the code object is executed it returns an asynchronous generator object. See **PEP 525** for more details.

Adicionado na versão 3.6.

inspect.**CO_HAS_DOCSTRING**

The flag is set when there is a docstring for the code object in the source code. If set, it will be the first item in `co_consts`.

Adicionado na versão 3.14.

inspect.**CO_METHOD**

The flag is set when the code object is a function defined in class scope.

Adicionado na versão 3.14.

Nota

The flags are specific to CPython, and may not be defined in other Python implementations. Furthermore, the flags are an implementation detail, and can be removed or deprecated in future Python releases. It's recommended to use public APIs from the *inspect* module for any introspection needs.

29.14.9 Buffer flags

class inspect.**BufferFlags**

This is an *enum.IntFlag* that represents the flags that can be passed to the `__buffer__()` method of objects implementing the buffer protocol.

The meaning of the flags is explained at buffer-request-types.

SIMPLE

WRITABLE

FORMAT

ND

STRIDES

C_CONTIGUOUS

F_CONTIGUOUS

ANY_CONTIGUOUS

INDIRECT

CONTIG

CONTIG_RO

STRIDED

STRIDED_RO

RECORDS

RECORDS_RO

FULL

FULL_RO

READ

WRITE

Adicionado na versão 3.12.

29.14.10 Interface de linha de comando

The *inspect* module also provides a basic introspection capability from the command line.

By default, accepts the name of a module and prints the source of that module. A class or function within the module can be printed instead by appended a colon and the qualified name of the target object.

--details

Print information about the specified object rather than the source code

29.15 `annotationlib` — Funcionalidade para introspecção de anotações

Adicionado na versão 3.14.

Código-fonte: Lib/annotationlib.py

O módulo `annotationlib` fornece ferramentas para introspecção de *anotações* em módulos, classes e funções.

As anotações são avaliadas preguiçosamente e frequentemente contêm referências futuras a objetos que ainda não foram definidos quando a anotação é criada. Este módulo fornece um conjunto de ferramentas de baixo nível que podem ser usadas para recuperar anotações de forma confiável, mesmo na presença de referências futuras e outros casos extremos.

Este módulo oferece suporte a recuperação de anotações em três formatos principais (veja `Format`), cada um dos quais funciona melhor para diferentes casos de uso:

- `VALUE` avalia as anotações e retorna seus valores. Isso é mais simples de usar, mas pode levantar erros, por exemplo, se as anotações contiverem referências a nomes indefinidos.
- `FORWARDREF` retorna objetos `ForwardRef` para anotações que não podem ser resolvidas, permitindo que você inspecione as anotações sem avaliá-las. Isso é útil quando você precisa trabalhar com anotações que podem conter referências de encaminhamento não resolvidas.
- `STRING` retorna as anotações como uma string, semelhante a como apareceria no arquivo fonte. Isso é útil para geradores de documentação que desejam exibir anotações de forma legível.

A função `get_annotations()` é o principal ponto de entrada para recuperar anotações. Dada uma função, classe ou módulo, ela retorna um dicionário de anotações no formato solicitado. Este módulo também fornece funcionalidade para trabalhar diretamente com a *função de anotação*, usada para avaliar anotações, como `get_annotate_from_class_namespace()` e `call_annotate_function()`, bem como a função `call_evaluate_function()` para trabalhar com *funções de avaliação*.

Cuidado

Most functionality in this module can execute arbitrary code; see *the security section* for more information.

Ver também

PEP 649 propôs o modelo atual de como as anotações funcionam em Python.

PEP 749 expandiu vários aspectos de **PEP 649** e introduziu o módulo `annotationlib`.

annotations-howto fornece práticas recomendadas para trabalhar com anotações.

typing-extensions fornece um backport de `get_annotations()` que funciona em versões anteriores do Python.

29.15.1 Semânticas de anotação

A forma como as anotações são avaliadas mudou ao longo da história do Python 3 e atualmente ainda depende de uma importação futura. Houve modelos de execução para anotações:

- *Semântica de estoque* (padrão no Python 3.0 a 3.13; veja **PEP 3107** e **PEP 526**): As anotações são avaliadas avidamente, à medida que são encontradas no código-fonte.
- *Anotações em string* (usadas com `from __future__ import annotations` no Python 3.7 e versões mais recentes; veja **PEP 563**): As anotações são armazenadas apenas como strings.
- *Avaliação adiada* (padrão no Python 3.14 e versões mais recentes; veja **PEP 649** e **PEP 749**): As anotações são avaliadas preguiçosamente, somente quando são acessadas.

Como exemplo, considere o seguinte programa:

```
def func(a: Cls) -> None:
    print(a)

class Cls: pass

print(func.__annotations__)
```

Isso se comportará da seguinte maneira:

- De acordo com a semântica de estoque (Python 3.13 e versões anteriores), ele levantará uma *NameError* na linha onde `func` está definido, porque `Cls` é um nome indefinido naquele ponto.
- Em anotações stringificadas (se `from __future__ import annotations` for usado), exibirá `{'a': 'Cls', 'return': 'None'}`.
- Em avaliação adiada (Python 3.14 e posterior), exibirá `{'a': <class 'Cls'>, 'return': None}`.

A semântica de estoque foi usada quando as anotações de função foram introduzidas pela primeira vez no Python 3.0 (por **PEP 3107**) porque esta era a maneira mais simples e óbvia de implementar anotações. O mesmo modelo de execução foi usado quando as anotações de variáveis foram introduzidas no Python 3.6 (por **PEP 526**). No entanto, a semântica de estoque causou problemas ao usar anotações como dicas de tipo, como a necessidade de se referir a nomes que ainda não estavam definidos quando a anotação era encontrada. Além disso, havia problemas de desempenho com a execução de anotações no momento da importação do módulo. Portanto, no Python 3.7, **PEP 563** introduziu a capacidade de armazenar anotações como strings usando a sintaxe `from __future__ import annotations`. O plano na época era eventualmente tornar esse comportamento o padrão, mas um problema surgiu: anotações em string são mais difíceis de processar para aqueles que inspecionam anotações em tempo de execução. Uma proposta alternativa, **PEP 649**, introduziu o terceiro modelo de execução, avaliação adiada, e foi implementada no Python 3.14. Anotações em string ainda são usadas se `from __future__ import annotations` estiver presente, mas esse comportamento será eventualmente removido.

29.15.2 Classes

class annotationlib.**Format**

Uma *IntEnum* que descreve os formatos nos quais as anotações podem ser retornadas. Membros da enumeração, ou seus valores inteiros equivalentes, podem ser passados para *get_annotations()* e outras funções neste módulo, bem como para funções `__annotate__`.

VALUE = 1

Os valores são o resultado da avaliação das expressões de anotação.

VALUE_WITH_FAKE_GLOBALS = 2

Valor especial usado para sinalizar que uma função de anotação está sendo avaliada em um ambiente especial com variáveis globais falsas. Ao receber este valor, as funções de anotação devem retornar o mesmo valor do formato *Format.VALUE* ou levantar *NotImplementedError* para sinalizar que não suportam execução neste ambiente. Este formato é usado apenas internamente e não deve ser passado para as funções deste módulo.

FORWARDREF = 3

Valores são valores de anotação reais (conforme o formato *Format.VALUE*) para valores definidos e proxies *ForwardRef* para valores indefinidos. Objetos reais podem conter referências a objetos proxy *ForwardRef*.

STRING = 4

Valores são a string de texto da anotação como ela aparece no código-fonte, até modificações, incluindo, mas não se limitando a, normalizações de espaços em branco e otimizações de valores constantes.

Os valores exatos dessas strings podem mudar em versões futuras do Python.

Adicionado na versão 3.14.

class annotationlib.**ForwardRef**

Um objeto proxy para referências futuras em anotações.

Instâncias desta classe são retornadas quando o formato *FORWARDREF* é usado e as anotações contêm um nome que não pode ser resolvido. Isso pode acontecer quando uma referência de avanço é usada em uma anotação, como quando uma classe é referenciada antes de ser definida.

__forward_arg__

Uma string contendo o código que foi avaliado para produzir *ForwardRef*. A string pode não ser exatamente equivalente à fonte original.

evaluate(*, *owner=None*, *globals=None*, *locals=None*, *type_params=None*, *format=Format.VALUE*)

Avalia a referência futura, retornando seu valor.

Se o argumento *format* for *VALUE* (o padrão), este método poderá levantar uma exceção, como *NameError*, caso a referência futura se refira a um nome que não pode ser resolvido. Os argumentos deste método podem ser usados para fornecer ligações para nomes que, de outra forma, seriam indefinidos. Se o argumento *format* for *FORWARDREF*, o método nunca levantará uma exceção, mas poderá retornar uma instância de *ForwardRef*. Por exemplo, se o objeto de referência futura contiver o código `list[undefined]`, onde `undefined` é um nome que não está definido, avaliá-lo com o formato *FORWARDREF* retornará `list[ForwardRef('undefined')]`. Se o argumento *format* for *STRING*, o método retornará *__forward_arg__*.

O parâmetro *owner* fornece o mecanismo preferencial para passar informações de escopo para este método. O proprietário de uma *ForwardRef* é o objeto que contém a anotação da qual a *ForwardRef* deriva, como um objeto módulo, objeto tipo ou objeto função.

Os parâmetros *globals*, *locals* e *type_params* fornecem um mecanismo mais preciso para influenciar os nomes disponíveis quando *ForwardRef* é avaliado. *globals* e *locals* são passados para *eval()*, representando os espaços de nomes global e local nos quais o nome é avaliado. O parâmetro *type_params* é relevante para objetos criados usando a sintaxe nativa para classes genéricas e funções. É uma tupla de parâmetros de tipo que estão no escopo enquanto a referência futura está sendo avaliada. Por exemplo, ao avaliar um *ForwardRef* recuperado de uma anotação encontrada no espaço de nomes de classe de uma classe genérica `C`, *type_params* deve ser definido como `C.__type_params__`.

Instâncias de *ForwardRef* retornadas por *get_annotations()* retêm referências a informações sobre o escopo de onde se originaram, portanto, chamar esse método sem argumentos adicionais pode ser suficiente para avaliar tais objetos. Instâncias de *ForwardRef* criadas por outros meios podem não ter nenhuma informação sobre seu escopo, portanto, passar argumentos para esse método pode ser necessário para avaliá-las com sucesso.

Se nenhum entre *owner*, *globals*, *locals* ou *type_params* for fornecido e *ForwardRef* não contiver informações sobre sua origem, dicionários globals e locals vazios serão usados.

Adicionado na versão 3.14.

29.15.3 Funções

annotationlib.**annotations_to_string**(*annotations*)

Converte um dicionário de anotações contendo valores de tempo de execução em um dicionário contendo apenas strings. Se os valores ainda não forem strings, eles serão convertidos usando *type_repr()*. Isso serve como um auxiliar para funções de anotação fornecidas pelo usuário que oferecem suporte ao formato *STRING*, mas não têm acesso ao código que cria as anotações.

Por exemplo, isso é usado para implementar o *STRING* para classes *typing.TypedDict* criadas por meio da sintaxe funcional:

```
>>> from typing import TypedDict
>>> Movie = TypedDict("movie", {"name": str, "year": int})
>>> get_annotations(Movie, format=Format.STRING)
{'name': 'str', 'year': 'int'}
```

Adicionado na versão 3.14.

annotationlib.**call_annotate_function**(*annotate*, *format*, *, *owner=None*)

Chama a *função de anotação* *annotate* com o *format* fornecido, um membro da enumeração `Format` e retorne o dicionário de anotações produzido pela função.

Esta função auxiliar é necessária porque as funções de anotação geradas pelo compilador para funções, classes e módulos oferecem suporte a apenas o formato `VALUE` quando chamadas diretamente. Para oferecer suporte a outros formatos, esta função chama a função de anotação em um ambiente especial que permite a produção de anotações nos outros formatos. Este é um bloco de construção útil ao implementar funcionalidades que precisam avaliar anotações parcialmente enquanto uma classe está sendo construída.

owner é o objeto que possui a função de anotação, geralmente uma função, classe ou módulo. Se fornecido, é usado no formato `FORWARDREF` para produzir um objeto `ForwardRef` que contém mais informações.

Ver também

PEP 649 contém uma explicação da técnica de implementação usada por esta função.

Adicionado na versão 3.14.

annotationlib.**call_evaluate_function**(*evaluate*, *format*, *, *owner=None*)

Chama a *função de avaliação* *evaluate* com o *format* fornecido, um membro da enumeração `Format` e retorna o valor produzido pela função. Isso é semelhante a `call_annotate_function()`, mas esta última sempre retorna um dicionário que mapeia strings para anotações, enquanto esta função retorna um único valor.

Isso se destina ao uso com as funções de avaliação geradas para elementos avaliados preguiçosamente relacionados a apelidos de tipo e parâmetros de tipo:

- `typing.TypeAliasType.evaluate_value()`, o valor dos apelidos de tipo
- `typing.TypeVar.evaluate_bound()`, a delimitação das variáveis de tipo
- `typing.TypeVar.evaluate_constraints()`, as restrições de tipos variáveis
- `typing.TypeVar.evaluate_default()`, o valor padrão de tipos variáveis
- `typing.ParamSpec.evaluate_default()`, o valor padrão de especificações de parâmetros
- `typing.TypeVarTuple.evaluate_default()`, o valor padrão de tuplas de tipos variáveis

owner é o objeto que possui a função de avaliação, como o apelido de tipo ou o objeto de tipo variável.

format pode ser usado para controlar o formato no qual o valor é retornado:

```
>>> type Alias = undefined
>>> call_evaluate_function(Alias.evaluate_value, Format.VALUE)
Traceback (most recent call last):
...
NameError: name 'undefined' is not defined
>>> call_evaluate_function(Alias.evaluate_value, Format.FORWARDREF)
ForwardRef('undefined')
>>> call_evaluate_function(Alias.evaluate_value, Format.STRING)
'undefined'
```

Adicionado na versão 3.14.

annotationlib.**get_annotate_from_class_namespace**(*namespace*)

Recupera a *função de anotação* de um dicionário de espaço de nomes de classe *namespace*. Retorna `None` se o espaço de nomes não contiver uma função de anotação. Isso é útil principalmente antes da classe ser totalmente criada (por exemplo, em uma metaclasse); após a classe existir, a função de anotação pode ser recuperada com `cls.__annotate__`. Veja *abaixo* para um exemplo usando esta função em uma metaclasse.

Adicionado na versão 3.14.

annotationlib.**get_annotations**(*obj*, *, *globals=None*, *locals=None*, *eval_str=False*, *format=Format.VALUE*)

Calcula o dicionário de anotações para um objeto.

obj pode ser um objeto chamável, classe, módulo ou outro objeto com os atributos `__annotate__` ou `__annotations__`. Passar qualquer outro objeto levanta *TypeError*.

O parâmetro *format* controla o formato em que as anotações são retornadas e deve ser um membro da enumeração *Format* ou seu equivalente inteiro. Os diferentes formatos funcionam da seguinte forma:

- VALUE: `object.__annotations__` é tentado primeiro; se não existir, a função `object.__annotate__` é chamada, se existir.
- FORWARDREF: Se `object.__annotations__` existir e puder ser avaliado com sucesso, ele será usado; caso contrário, a função `object.__annotate__` será chamada. Se também não existir, `object.__annotations__` será tentado novamente e qualquer erro ao acessá-lo será levantado novamente.
- STRING: Se `object.__annotate__` existir, ele será chamado primeiro; caso contrário, `object.__annotations__` será usado e transformado em string usando *annotations_to_string()*.

Retorna um dict. `get_annotations()` retorna um novo dict toda vez que é chamado; chamá-lo duas vezes no mesmo objeto retornará dois dicts diferentes, mas equivalentes.

Esta função cuida de vários detalhes para você:

- Se *eval_str* for verdadeiro, valores do tipo `str` serão descodificados usando *eval()*. Isso se destina ao uso com anotações transformadas em string (`from __future__ import annotations`). É um erro definir *eval_str* como true com formatos diferentes de *Format.VALUE*.
- Se *obj* não tiver um dicionário de anotações, retorna um dicionário vazio. (Funções e métodos sempre têm um dicionário de anotações; classes, módulos e outros tipos de chamáveis podem não ter.)
- Ignora anotações herdadas em classes, bem como anotações em metaclasses. Se uma classe não tiver seu próprio dicionário de anotações, retorna um dicionário vazio.
- Todos os acessos aos membros do objeto e valores do dicionário são feitos usando `getattr()` e `dict.get()` por segurança.

eval_str controla se valores do tipo `str` são substituídos ou não pelo resultado da chamada de *eval()* nesses valores:

- Se eval_str for verdadeiro, *eval()* será chamado em valores do tipo `str`. (Observe que `get_annotations()` não captura exceções; se *eval()* levanta uma exceção, ele desenrolará a pilha após a chamada de `get_annotations()`.)
- Se *eval_str* for falso (o padrão), os valores do tipo `str` não serão alterados.

globals e *locals* são passados para *eval()*; consulte a documentação de *eval()* para mais informações. Se *globals* ou *locals* for `None`, esta função pode substituir esse valor por um padrão específico do contexto, dependendo de `type(obj)`:

- Se *obj* for um módulo, *globals* assume como padrão `obj.__dict__`.
- Se *obj* for uma classe, *globals* assume como padrão `sys.modules[obj.__module__].__dict__` e *locals* assume como padrão o espaço de nomes da classe *obj*.
- Se *obj* for um chamável, *globals* assume como padrão `obj.__globals__`, embora se *obj* for uma função encapsulada (usando *functools.update_wrapper()*) ou um objeto *functools.partial*, ela será desencapsulada até que uma função não encapsulada seja encontrada.

Chamar `get_annotations()` é uma boa prática para acessar o dicionário de anotações de qualquer objeto. Consulte annotations-howto para obter mais informações sobre as práticas recomendadas para anotações.

```
>>> def f(a: int, b: str) -> float:
...     pass
```

(continua na próxima página)

(continuação da página anterior)

```
>>> get_annotations(f)
{'a': <class 'int'>, 'b': <class 'str'>, 'return': <class 'float'>}
```

Adicionado na versão 3.14.

annotationlib.**type_repr**(*value*)

Converte um valor Python arbitrário para um formato adequado para uso pelo formato *STRING*. Isso chama *repr()* para a maioria dos objetos, mas tem um tratamento especial para alguns objetos, como objetos tipo.

Isto serve como um auxiliar para funções de anotação fornecidas pelo usuário que oferecem suporte ao formato *STRING*, mas não têm acesso ao código que cria as anotações. Também pode ser usado para fornecer uma representação de string amigável para outros objetos que contêm valores comumente encontrados em anotações.

Adicionado na versão 3.14.

29.15.4 Receitas

Usando anotações em uma metaclasse

Uma metaclasse pode querer inspecionar ou até mesmo modificar as anotações no corpo de uma classe durante a criação da classe. Isso requer a recuperação das anotações do dicionário de espaços de nomes da classe. Para classes criadas com `from __future__ import annotations`, as anotações estarão na chave `__annotations__` do dicionário. Para outras classes com anotações, *get_annotate_from_class_namespace()* pode ser usado para obter a função annotate, e *call_annotate_function()* pode ser usado para chamá-la e recuperar as anotações. Usar o formato *FORWARDREF* geralmente é a melhor opção, pois permite que as anotações se refiram a nomes que ainda não podem ser resolvidos quando a classe é criada.

Para modificar as anotações, é melhor criar uma função de anotação que chame a função de anotação original, faça os ajustes necessários e retorne o resultado.

Abaixo está um exemplo de uma metaclasse que filtra todas as anotações *typing.ClassVar* da classe e as coloca em um atributo separado:

```
import annotationlib
import typing

class ClassVarSeparator(type):
   def __new__(mcls, name, bases, ns):
      if "__annotations__" in ns:  # from __future__ import annotations
         annotations = ns["__annotations__"]
         classvar_keys = {
            key for key, value in annotations.items()
            # Use comparação de strings para simplificar; uma solução
            # mais robusta poderia usar annotationlib.ForwardRef.evaluate
            if value.startswith("ClassVar")
         }
         classvars = {key: annotations[key] for key in classvar_keys}
         ns["__annotations__"] = {
            key: value for key, value in annotations.items()
            if key not in classvar_keys
         }
         wrapped_annotate = None
      elif annotate := annotationlib.get_annotate_from_class_namespace(ns):
         annotations = annotationlib.call_annotate_function(
            annotate, format=annotationlib.Format.FORWARDREF
         )
         classvar_keys = {
            key for key, value in annotations.items()
```

(continua na próxima página)

(continuação da página anterior)

```
            if typing.get_origin(value) is typing.ClassVar
        }
        classvars = {key: annotations[key] for key in classvar_keys}

        def wrapped_annotate(format):
            annos = annotationlib.call_annotate_function(annotate, format,␣
↪owner=typ)
            return {key: value for key, value in annos.items() if key not in␣
↪classvar_keys}

    else:  # nenhuma anotação
        classvars = {}
        wrapped_annotate = None
    typ = super().__new__(mcls, name, bases, ns)

    if wrapped_annotate is not None:
        # Encapsula o __annotate__ original com um invólucro que remove ClassVars
        typ.__annotate__ = wrapped_annotate
    typ.classvars = classvars  # Armazena as ClassVars em um atributo separado
    return typ
```

29.15.5 Limitações do formato `STRING`

O formato *`STRING`* tem como objetivo aproximar o código-fonte da anotação, mas a estratégia de implementação usada significa que nem sempre é possível recuperar o código-fonte exato.

Primeiro, a transformação em string, é claro, não pode recuperar nenhuma informação que não esteja presente no código compilado, incluindo comentários, espaços em branco, parênteses e operações que são simplificadas pelo compilador.

Em segundo lugar, a transformação em string pode interceptar quase todas as operações que envolvem nomes pesquisados em algum escopo, mas não pode interceptar operações que operem totalmente em constantes. Como corolário, isso também significa que não é seguro solicitar o formato `STRING` em código não confiável: Python é poderoso o suficiente para permitir a execução arbitrária de código mesmo sem acesso a nenhuma variável global ou embutida. Por exemplo:

```
>>> def f(x: (1).__class__.__base__.__subclasses__()[-1].__init__.__builtins__[
↪"print"]("Hello world")): pass
...
>>> annotationlib.get_annotations(f, format=annotationlib.Format.STRING)
Hello world
{'x': 'None'}
```

Nota

Este exemplo específico funciona no momento em que este artigo foi escrito, mas depende de detalhes de implementação e não há garantia de que funcionará no futuro.

Entre os diferentes tipos de expressões que existem em Python, conforme representado pelo módulo *`ast`*, algumas expressões são suportadas, o que significa que o formato `STRING` geralmente pode recuperar o código-fonte original; outras não são suportadas, o que significa que podem resultar em saída incorreta ou erro.

Os seguintes são aceitos (às vezes com ressalvas):

- *`ast.BinOp`*
- *`ast.UnaryOp`*

  - *ast.Invert* (~), *ast.UAdd* (+) e *ast.USub* (-) são suportadas
  - *ast.Not* (not) não é suportada
- *ast.Dict* (exceto ao usar desempacotamento **)
- *ast.Set*
- *ast.Compare*
  - *ast.Eq* e *ast.NotEq* são suportadas
  - *ast.Lt*, *ast.LtE*, *ast.Gt* e *ast.GtE* são suportadas, mas o operando pode ser invertido
  - *ast.Is*, *ast.IsNot*, *ast.In* e *ast.NotIn* não são suportadas
- *ast.Call* (exceto ao usar desempacotamento **)
- *ast.Constant* (embora não seja a representação exata da constante; por exemplo, sequências de escape em strings são perdidas; números hexadecimais são convertidos em decimais)
- *ast.Attribute* (presumindo que o valor não é uma constante)
- *ast.Subscript* (presumindo que o valor não é uma constante)
- *ast.Starred* (desempacotamento *)
- *ast.Name*
- *ast.List*
- *ast.Tuple*
- *ast.Slice*

Os seguintes não são suportados, mas geram um erro informativo quando encontrados pela transformação em string:

- *ast.FormattedValue* (f-strings; o erro não é detectado se especificadores de conversões como !r forem usados)
- *ast.JoinedStr* (f-strings)

Os seguintes itens não são suportados e resultam em saída incorreta:

- *ast.BoolOp* (and e or)
- *ast.IfExp*
- *ast.Lambda*
- *ast.ListComp*
- *ast.SetComp*
- *ast.DictComp*
- *ast.GeneratorExp*

Os seguintes itens não são permitidos em escopos de anotação e, portanto, não são relevantes:

- *ast.NamedExpr* (:=)
- *ast.Await*
- *ast.Yield*
- *ast.YieldFrom*

29.15.6 Limitações do formato FORWARDREF

O formato *FORWARDREF* visa produzir valores reais o máximo possível, com tudo o que não puder ser resolvido sendo substituído por objetos *ForwardRef*. Ele é afetado, em linhas gerais, pelas mesmas limitações do formato *STRING*: anotações que realizam operações em literais ou que usam tipos de expressão não suportados podem levantar exceções quando avaliadas usando o formato *FORWARDREF*.

Abaixo estão alguns exemplos do comportamento com expressões não suportadas:

```
>>> from annotationlib import get_annotations, Format
>>> def zerodiv(x: 1 / 0): ...
>>> get_annotations(zerodiv, format=Format.STRING)
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
>>> get_annotations(zerodiv, format=Format.FORWARDREF)
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
>>> def ifexp(x: 1 if y else 0): ...
>>> get_annotations(ifexp, format=Format.STRING)
{'x': '1'}
```

29.15.7 Security implications of introspecting annotations

Much of the functionality in this module involves executing code related to annotations, which can then do arbitrary things. For example, *get_annotations()* may call an arbitrary *annotate function*, and *ForwardRef.evaluate()* may call *eval()* on an arbitrary string. Code contained in an annotation might make arbitrary system calls, enter an infinite loop, or perform any other operation. This is also true for any access of the `__annotations__` attribute, and for various functions in the *typing* module that work with annotations, such as *typing.get_type_hints()*.

Any security issue arising from this also applies immediately after importing code that may contain untrusted annotations: importing code can always cause arbitrary operations to be performed. However, it is unsafe to accept strings or other input from an untrusted source and pass them to any of the APIs for introspecting annotations, for example by editing an `__annotations__` dictionary or directly creating a *ForwardRef* object.

29.16 site — Gancho de configuração específico do site

Código-fonte: Lib/site.py

Este módulo é importado automaticamente durante a inicialização. A importação automática pode ser suprimida usando a opção `-S` do interpretador.

A importação deste módulo normalmente anexa caminhos específicos do site ao caminho de pesquisa de módulos e adiciona *chamáveis*, incluindo *help()* ao espaço de nomes embutido. No entanto, a opção de inicialização do Python `-S` bloqueia isso e este módulo pode ser importado com segurança sem modificações automáticas no caminho de pesquisa de módulos ou adições aos módulos embutidos. Para acionar explicitamente as adições usuais específicas do site, chame a função *main()*.

Alterado na versão 3.3: A importação do módulo usado para acionar a manipulação de caminhos, mesmo ao usar `-S`.

Ele começa construindo até quatro diretórios a partir de uma parte no início e uma no final. Para a parte no início, ele usa `sys.prefix` e `sys.exec_prefix`; inícios vazios são ignorados. Para a parte no final, ele usa a string vazia e então `lib/site-packages` (no Windows) ou `lib/pythonX.Y[t]/site-packages` (no Unix e macOS). (O sufixo opcional "t" indica a construção com *threads livres* e é anexado se `"t"` estiver presente na constante *sys.abiflags*.) Para cada uma das combinações distintas início-fim, ele vê se se refere a um diretório existente e, se sim, adiciona-o a `sys.path` e também inspeciona o caminho recém-adicionado para arquivos de configuração.

Alterado na versão 3.5: Suporte para o diretório "site-python" foi removido.

Alterado na versão 3.13: No Unix, as instalações do Python *threads livres* são identificadas pelo sufixo "t" no nome do diretório específico da versão, como `lib/python3.13t/`.

Alterado na versão 3.14: *site* não é mais responsável por atualizar *sys.prefix* e *sys.exec_prefix* em *Ambientes virtuais*. Isso agora é feito durante a *inicialização do caminho*. Como resultado, em *Ambientes virtuais*, *sys.prefix* e *sys.exec_prefix* não dependem mais da inicialização de *site* e, portanto, não são afetados por `-S`. Ao executar em um *ambiente virtual*, o arquivo `pyvenv.cfg` em *sys.prefix* é verificado em busca de configurações específicas do site. Se a chave `include-system-site-packages` existir e estiver definida como `true` (sem distinção entre maiúsculas e minúsculas), os prefixos de nível de sistema serão pesquisados em busca de site-packages; caso contrário, não serão.

Um arquivo de configuração de caminho é aquele cujo nome tem o formato *name*`.pth` e que existe em um dos quatro diretórios mencionados acima; seu conteúdo são itens adicionais (um por linha) a serem adicionados ao `sys.path`. Itens inexistentes nunca são adicionados ao `sys.path` e não é verificado se o item se refere a um diretório, e não a um arquivo. Nenhum item é adicionado ao `sys.path` mais de uma vez. Linhas em branco e linhas iniciadas com `#` são ignoradas. Linhas iniciadas com `import` (seguidas de espaço ou tabulação) são executadas.

Nota

Uma linha executável em um arquivo `.pth` é executada a cada inicialização do Python, independentemente de um módulo em particular ser realmente usado. Seu impacto deve, portanto, ser reduzido ao mínimo. O objetivo principal das linhas executáveis é tornar o(s) módulo(s) correspondente(s) importável (carregar ganchos de importação de terceiros, ajustar `PATH` etc). Qualquer outra inicialização deve ser feita na importação real de um módulo, se e quando isso acontecer. Limitar um fragmento de código a uma única linha é uma medida deliberada para desencorajar colocar qualquer coisa mais complexa aqui.

Alterado na versão 3.13: Os arquivos `.pth` agora são decodificados primeiro por UTF-8 e depois pela *codificação da localidade* se falhar.

Por exemplo, suponha que `sys.prefix` e `sys.exec_prefix` sejam definidos com `/usr/local`. A biblioteca Python X.Y é instalado em `/usr/local/lib/python`*X.Y*. Suponha que isso tenha um subdiretório `/usr/local/lib/python`*X.Y*`/site-packages` com três subsubdiretórios, `foo`, `bar` e `spam`, e dois caminhos arquivos de configuração, `foo.pth` e `bar.pth`. Presuma que `foo.pth` contém o seguinte:

```
# configuração do pacote foo

foo
bar
bletch
```

e que `bar.pth` contém:

```
# configuração do pacote bar

bar
```

Em seguida, os seguintes diretórios específicos da versão são adicionados a `sys.path`, nesta ordem:

```
/usr/local/lib/pythonX.Y/site-packages/bar
/usr/local/lib/pythonX.Y/site-packages/foo
```

Observe que `bletch` é omitido porque não existe; o diretório `bar` precede o diretório `foo` porque `bar.pth` vem em ordem alfabética antes de `foo.pth`; e `spam` é omitido porque não é mencionado em nenhum dos arquivos de configuração de caminho.

29.16.1 sitecustomize

Após essas manipulações de caminho, é feita uma tentativa de importar um módulo chamado *sitecustomize*, que pode executar personalizações arbitrárias específicas do site. Ele é normalmente criado por um administrador de sistema no diretório site-packages. Se essa importação falhar com uma *ImportError* ou sua exceção de subclasse, e o atributo *name* da exceção for igual a `'sitecustomize'`, ele será ignorado silenciosamente. Se o Python for iniciado sem fluxos de saída disponíveis, como com `pythonw.exe` no Windows (que é usado por padrão para iniciar o IDLE), a tentativa de saída de *sitecustomize* será ignorada. Qualquer outra exceção causa uma falha silenciosa e talvez misteriosa do processo.

29.16.2 usercustomize

Depois disso, é feita uma tentativa de importar um módulo chamado *usercustomize*, que pode executar personalizações arbitrárias específicas do usuário, se *ENABLE_USER_SITE* for true. Este arquivo deve ser criado no diretório site-packages de usuário (veja abaixo), que faz parte de `sys.path` a menos que seja desabilitado por `-s`. Se esta importação falhar com um *ImportError* ou sua exceção de subclasse, e o atributo *name* da exceção for igual a `'usercustomize'`, ele será ignorado silenciosamente.

Observe que, para alguns sistemas não Unix, `sys.prefix` e `sys.exec_prefix` estão vazios, e as manipulações de caminho são ignoradas; no entanto, a importação de *sitecustomize* e *usercustomize* ainda é tentada.

29.16.3 Configuração Readline

Em sistemas que oferecem suporte a *readline*, este módulo também importará e configurará o módulo *rlcompleter*, se o Python for iniciado em modo interativo e sem a opção `-S`. O comportamento padrão é habilitar tab-completion e usar `~/.python_history` como o arquivo de salvamento do histórico. Para desabilitá-lo, exclua (ou substitua) o atributo *sys.__interactivehook__* no seu módulo *sitecustomize* ou *usercustomize* ou no seu arquivo `PYTHONSTARTUP`.

Alterado na versão 3.4: A ativação do rlcompleter e do histórico foi feita automaticamente.

29.16.4 Conteúdo do módulo

site.**PREFIXES**

Uma lista de prefixos para diretórios site-package.

site.**ENABLE_USER_SITE**

Sinalizador mostrando o status do diretório site-packages do usuário. `True` significa que ele está habilitado e foi adicionado a `sys.path`. `False` significa que ele foi desabilitado por solicitação do usuário (com `-s` ou `PYTHONNOUSERSITE`). `None` significa que ele foi desabilitado por motivos de segurança (incompatibilidade entre o ID do usuário ou grupo e o ID efetivo) ou por um administrador.

site.**USER_SITE**

Caminho para o usuário site-packages para o Python em execução. Pode ser `None` se *getusersitepackages()* ainda não tiver sido chamado. O valor padrão é `~/.local/lib/pythonX.Y[t]/site-packages` para construções UNIX e macOS não-framework, `~/Library/Python/X.Y/lib/python/site-packages` para construções macOS framework e `%APPDATA%\Python\PythonXY\site-packages` no Windows. O "t" opcional indica a construção com threads livres. Este diretório é um diretório de site, o que significa que os arquivos `.pth` nele serão processados.

site.**USER_BASE**

Caminho para o diretório base para os pacotes do site do usuário. Pode ser `None` se *getuserbase()* ainda não tiver sido chamado. O valor padrão é `~/.local` para construções não-framework UNIX e macOS, `~/Library/Python/X.Y` para construções de framework macOS e `%APPDATA%\Python` para Windows. Este valor é usado para calcular os diretórios de instalação para scripts, arquivos de dados, módulos Python, etc. para o *esquema de instalação de usuário*. Veja também `PYTHONUSERBASE`.

site.**main**()

Adiciona todos os diretórios padrão específicos do site ao caminho de pesquisa de módulos. Esta função é chamada automaticamente quando este módulo é importado, a menos que o interpretador Python tenha sido iniciado com o sinalizador `-S`.

Alterado na versão 3.3: Esta função costumava ser chamada incondicionalmente.

`site.`**`addsitedir`**`(`*sitedir*, *known_paths=None*`)`

Adiciona um diretório a sys.path e processa seus arquivos `.pth`. Normalmente usado em *sitecustomize* ou *usercustomize* (veja acima).

`site.`**`getsitepackages`**`()`

Retorna uma lista contendo todos os diretórios globais de pacotes de sites.

Adicionado na versão 3.2.

`site.`**`getuserbase`**`()`

Retorna o caminho do diretório base do usuário, *USER_BASE*. Se ele ainda não foi inicializado, esta função também o definirá, respeitando `PYTHONUSERBASE`.

Adicionado na versão 3.2.

`site.`**`getusersitepackages`**`()`

Retorna o caminho do diretório site-packages específico do usuário, *USER_SITE*. Se ele ainda não foi inicializado, esta função também o definirá, respeitando *USER_BASE*. Para determinar se o site-packages específico do usuário foi adicionado a `sys.path`, *ENABLE_USER_SITE* deve ser usado.

Adicionado na versão 3.2.

29.16.5 Interface de linha de comando

O módulo *site* também fornece uma maneira de obter os diretórios do usuário a partir da linha de comando:

```
$ python -m site --user-site
/home/user/.local/lib/python3.11/site-packages
```

Se for chamado sem argumentos, ele vai exibir o conteúdo de *sys.path* na saída padrão, seguido pelo valor de *USER_BASE* e se o diretório existe, depois o mesmo para *USER_SITE* e, finalmente, o valor de *ENABLE_USER_SITE*.

`--user-base`

Exibe o caminho para o diretório base do usuário.

`--user-site`

Exibe o caminho para o diretório site-packages do usuário.

Se ambas as opções forem fornecidas, a base do usuário e o site do usuário serão exibidos (sempre nesta ordem), separados por *os.pathsep*.

Se qualquer opção for fornecida, o script sairá com um destes valores: `0` se o diretório site-packages do usuário estiver habilitado, `1` se ele foi desabilitado pelo usuário, `2` se ele foi desabilitado por motivos de segurança ou por um administrador, e um valor maior que 2 se houver um erro.

Ver também

- **PEP 370** – Diretório site-packages por usuário.
- *A inicialização do caminho de pesquisa de módulos sys.path* – A inicialização de *sys.path*.

Ver também

- Veja o módulo *concurrent.interpreters*, que expõe de forma semelhante a funcionalidade principal do runtime.

CAPÍTULO 30

Interpretadores Python Personalizados

Os módulos descritos neste capítulo permitem a escrita de interfaces semelhantes ao interpretador interativo da Python. Se você quer um interpretador de Python que suporte algum recurso especial, além do idioma de Python, você deve olhar para o módulo *code*. (O módulo *codeop* é de nível inferior, usado para suportar compilação de um pedaço possivelmente incompleto do código Python).

A lista completa de módulos descritos neste capítulo é:

30.1 code — Classes bases do interpretador

Código-fonte: Lib/code.py

O módulo `code` fornece facilidades para implementar laços de leitura-execução-escrita no código Python. São incluídas duas classes e funções de conveniência que podem ser usadas para criar aplicações que fornecem um prompt de interpretador interativo.

class `code.`**InteractiveInterpreter**(*locals=None*)

Esta classe trata de análise e estado do interpretador (espaço de nomes do usuário); o mesmo não lida com buffer de entrada ou solicitação ou nomeação de arquivo de entrada (o nome do arquivo é sempre passado explicitamente). O argumento opcional *locals* especifica um mapeamento para usar como espaço de nomes no qual o código será executado; ele é padrão para um dicionário recém-criado com a chave `'__name__'` definida com `'__console__'` e a chave `'__doc__'` definida com `None`.

Observe que objetos funções e classes criados sob uma instância `InteractiveInterpreter` pertencerão ao espaço de nomes especificado por *locals*. Só serão selecionáveis se *locals* for o espaço de nomes de um módulo existente.

class `code.`**InteractiveConsole**(*locals=None*, *filename='<console>'*, *local_exit=False*)

Emula de forma muito semelhante o comportamento do interpretador Python interativo. Esta classe baseia-se em *InteractiveInterpreter* e adiciona prompts usando os familiares `sys.ps1` e `sys.ps2`, e buffer de entrada. Se *local_exit* for verdadeiro, `exit()` e `quit()` no console não levantarão *SystemExit*, mas retornarão ao código de chamada.

Alterado na versão 3.13: Adicionado o parâmetro *local_exit*.

code.**interact**(*banner=None*, *readfunc=None*, *local=None*, *exitmsg=None*, *local_exit=False*)

Função de conveniência para executar um laço de leitura-execução-impressão. Isto cria uma nova instância de `InteractiveConsole` e define *readfunc* para ser usado como o método `InteractiveConsole.raw_input()`, se fornecido. Se *local* for fornecido, ele será passado para o construtor `InteractiveConsole` para uso como espaço de nomes padrão para o laço do interpretador. Se *local_exit* for fornecido, ele será passado para o construtor `InteractiveConsole`. O método `interact()` da instância é então executado com *banner* e *exitmsg* passados como banner e mensagem de saída a serem usados, se fornecidos. O objeto console é descartado após o uso.

Alterado na versão 3.6: Parâmetro adicionado *exitmsg*.

Alterado na versão 3.13: Adicionado o parâmetro *local_exit*.

code.**compile_command**(*source*, *filename='<input>'*, *symbol='single'*)

Esta função é útil para programas que desejam emular o laço principal do interpretador Python (também conhecido como laço de leitura-execução-impressão). A parte complicada é determinar quando o usuário digitou um comando incompleto que pode ser concluído inserindo mais texto (em vez de um comando completo ou um erro de sintaxe). Esta função *quase* sempre toma a mesma decisão que o laço principal do interpretador real.

source é a string fonte; *filename* é o nome do arquivo opcional do qual a fonte foi lida, sendo o padrão `'<input>'`; e *symbol* é o símbolo opcional de início da gramática, que deve ser `'single'` (o padrão), `'eval'` ou `'exec'`.

Retorna um objeto código (o mesmo que `compile(source, filename, symbol)`) se o comando for completo e válido; `None` se o comando estiver incompleto; levanta `SyntaxError` se o comando estiver completo e contém um erro de sintaxe, ou levanta `OverflowError` ou `ValueError` se o comando contiver um literal inválido.

30.1.1 Objetos de interpretador interativo

InteractiveInterpreter.**runsource**(*source*, *filename='<input>'*, *symbol='single'*)

Compila e executa alguma fonte no interpretador. Os argumentos são os mesmos de `compile_command()`; o padrão para *filename* é `'<input>'`, e para *symbol* é `'single'`. Uma de várias coisas pode acontecer:

- A entrada está incorreta; `compile_command()` levantou uma exceção (`SyntaxError` ou `OverflowError`). Um traceback da sintaxe será impresso chamando o método `showsyntaxerror()`. `runsource()` retorna `False`.
- A entrada está incompleta e são necessárias mais entradas; `compile_command()` retornou `None`. `runsource()` retorna `True`.
- A entrada está completa; `compile_command()` retornou um objeto código. O código é executado chamando `runcode()` (que também lida com exceções de tempo de execução, exceto `SystemExit`). `runsource()` retorna `False`.

O valor de retorno pode ser usado para decidir se usar `sys.ps1` ou `sys.ps2` para solicitar a próxima linha.

InteractiveInterpreter.**runcode**(*code*)

Executa um objeto código. Quando ocorre uma exceção, `showtraceback()` é chamado para exibir um traceback. Todas as exceções são capturadas, exceto `SystemExit`, que pode ser propagada.

Uma observação sobre `KeyboardInterrupt`: esta exceção pode ocorrer em outro lugar neste código e nem sempre pode ser detectada. O chamador deve estar preparado para lidar com isso.

InteractiveInterpreter.**showsyntaxerror**(*filename=None*)

Exibe o erro de sintaxe que acabou de ocorrer. Isso não exibe um stack trace (situação da pilha de execução) porque não há um para erros de sintaxe. Se *filename* for fornecido, ele será inserido na exceção em vez do nome de arquivo padrão fornecido pelo analisador sintático do Python, porque ele sempre usa `'<string>'` ao ler uma string. A saída é escrita pelo método `write()`.

InteractiveInterpreter.**showtraceback**()

Exibe a exceção que acabou de ocorrer. Removemos o primeiro item da pilha porque ele está dentro da implementação do objeto interpretador. A saída é escrita pelo método *write()*.

Alterado na versão 3.5: O traceback encadeado completo é exibido em vez de apenas o traceback primário.

InteractiveInterpreter.**write**(*data*)

Escreve uma string no fluxo de erro padrão (`sys.stderr`). As classes derivadas devem substituir isso para fornecer o tratamento de saída apropriado conforme necessário.

30.1.2 Objetos de console Interativo

A classe *InteractiveConsole* é uma subclasse de *InteractiveInterpreter* e, portanto, oferece todos os métodos dos objetos interpretadores, bem como as seguintes adições.

InteractiveConsole.**interact**(*banner=None*, *exitmsg=None*)

Emula de forma muito semelhante o console interativo do Python. O argumento opcional *banner* especifica o banner a ser impresso antes da primeira interação; por padrão ele imprime um banner semelhante ao impresso pelo interpretador Python padrão, seguido pelo nome da classe do objeto console entre parênteses (para não confundir isso com o interpretador real – já que está tão próximo!).

O argumento opcional *exitmsg* especifica uma mensagem de saída impressa ao sair. Passe a string vazia para suprimir a mensagem de saída. Se *exitmsg* não for fornecido ou `None`, uma mensagem padrão será impressa.

Alterado na versão 3.4: Para suprimir a impressão de qualquer banner, passe uma string vazia.

Alterado na versão 3.6: Imprime uma mensagem de saída ao sair.

InteractiveConsole.**push**(*line*)

Envia uma linha do texto fonte para o interpretador. A linha não deve ter uma nova linha à direita; pode ter novas linhas internas. A linha é anexada a um buffer e o método *runsource()* do interpretador é chamado com o conteúdo concatenado do buffer como fonte. Se isso indicar que o comando foi executado ou é inválido, o buffer será redefinido; caso contrário, o comando estará incompleto e o buffer permanecerá como estava após a linha ser anexada. O valor de retorno é `True` se mais entrada for necessária, `False` se a linha foi tratada de alguma forma (isto é o mesmo que `runsource()`).

InteractiveConsole.**resetbuffer**()

Remove qualquer texto fonte não tratado do buffer de entrada.

InteractiveConsole.**raw_input**(*prompt=''*)

Escreve um prompt e leia uma linha. A linha retornada não inclui a nova linha final. Quando o usuário insere a sequência de teclas de fim de linha, uma exceção *EOFError* é levantada. A implementação base lê `sys.stdin`; uma subclasse pode substituir isso por uma implementação diferente.

30.2 codeop — Compila código Python

Código-fonte: Lib/codeop.py

O módulo *codeop* fornece utilitários sobre os quais o loop de leitura-execução-exibição do Python pode ser emulado, como é feito no módulo *code*. Como resultado, você provavelmente não deseja usar o módulo diretamente; se você deseja incluir tal loop em seu programa, você provavelmente deseja usar o módulo *code*.

Há duas partes para esta tarefa:

1. Ser capaz de dizer se uma linha de entrada completa uma instrução Python: em suma, dizer se deve exibir '>>>' ou '...' em seguida.
2. Lembrar quais instruções futuras o usuário inseriu, para que as entradas subsequentes possam ser compiladas com essas declarações em vigor.

O módulo *codeop* fornece uma maneira de fazer cada uma dessas coisas e uma maneira de fazer as duas coisas.

Para fazer apenas a primeira:

codeop.**compile_command**(*source*, *filename='<input>'*, *symbol='single'*)

Tenta compilar *source*, que deve ser uma string de código Python e retornar um objeto código se *source* for um código Python válido. Nesse caso, o atributo de nome de arquivo do objeto código será *filename*, cujo padrão é `'<input>'`. Retorna `None` se *source não* é um código Python válido, mas é um prefixo de código Python válido.

Se houver um problema com *source*, uma exceção será levantada. *SyntaxError* é levantada se houver sintaxe Python inválida, e *OverflowError* ou *ValueError* se houver um literal inválido.

O argumento *symbol* determina se *source* é compilado como uma instrução (`'single'`, o padrão), como uma sequência de *instruções* (`'exec'`) ou como uma *expressão* (`'eval'`). Qualquer outro valor fará com que *ValueError* seja levantada.

> **Nota**
>
> É possível (mas não provável) que o analisador sintático pare de analisar com um resultado bem-sucedido antes de chegar ao final da fonte; neste caso, os símbolos finais podem ser ignorados em vez de causar um erro. Por exemplo, uma barra invertida seguida por duas novas linhas pode ser seguida por lixo arbitrário. Isso será corrigido quando a API para o analisador for melhor.

class codeop.**Compile**

Instâncias desta classe têm métodos `__call__()` idênticos em assinatura à função embutida *compile()*, mas com a diferença de que se a instância compilar o texto do programa contendo uma instrução *__future__*, a instância se "lembra" e compila todos os textos de programa subsequentes com a instrução em vigor.

class codeop.**CommandCompiler**

Instâncias desta classe têm métodos `__call__()` idênticos em assinatura a *compile_command()*; a diferença é que se a instância compila o texto do programa contendo uma instrução *__future__*, a instância se "lembra" e compila todos os textos do programa subsequentes com a instrução em vigor.

CAPÍTULO 31

Importando módulos

Os módulos descritos neste capítulo fornecem novas maneiras de importar outros módulos Python e hooks para personalizar o processo de importação.

A lista completa de módulos descritos neste capítulo é:

31.1 `zipimport` - Importa módulos de arquivos Zip

Código-fonte: Lib/zipimport.py

Este módulo adiciona a capacidade de importar módulos Python (`*.py`, `*.pyc`) e pacotes de arquivos em formato ZIP. Normalmente, não é necessário usar o módulo *`zipimport`* explicitamente; ele é usado automaticamente pelo mecanismo embutido no `import` para itens *`sys.path`* que são caminhos para arquivos ZIP.

Normalmente, *`sys.path`* é uma lista de nomes de diretórios como strings. Este módulo também permite que um item de *`sys.path`* seja uma string que nomeia um arquivo ZIP. O arquivo ZIP pode conter uma estrutura de subdiretório para suportar importações de pacotes, e um caminho dentro do arquivo pode ser especificado para importar apenas de um subdiretório. Por exemplo, o caminho `example.zip/lib/` importaria apenas do subdiretório `lib/` dentro do arquivo.

Quaisquer arquivos podem estar presentes no arquivo ZIP, mas os importadores são invocados apenas para os arquivos `.py` e `.pyc`. A importação ZIP de módulos dinâmicos (`.pyd`, `.so`) não é permitida. Observe que, se um arquivo contiver apenas arquivos `.py`, o Python não tentará modificá-lo adicionando o arquivo `.pyc` correspondente, o que significa que, se um arquivo ZIP não contiver arquivos `.pyc`, a importação poderá ser bastante lenta.

Alterado na versão 3.13: Suporte para ZIP64.

Alterado na versão 3.8: Anteriormente, não havia suporte para arquivos ZIP com um comentário de arquivo.

Ver também

Nota da aplicação PKZIP
Documentação do formato de arquivo ZIP feita por Phil Katz, criador do formato e dos algoritmos usados.

PEP 273 - Importar módulos de arquivos Zip
Escrito por James C. Ahlstrom, que também forneceu uma implementação. O Python 2.3 segue a especificação em **PEP 273**, mas usa uma implementação escrita por Just van Rossum que utiliza os ganchos de importação descritos em **PEP 302**.

importlib - **A importação do maquinário de importação**
Pacote que fornece os protocolos relevantes para todos os importadores implementarem.

Este módulo define uma exceção:

exception zipimport.**ZipImportError**

Exceção levantada por objetos zipimporter. É uma subclasse de *ImportError*, portanto, também pode ser capturada como *ImportError*.

31.1.1 Objetos zipimporter

zipimporter é a classe para importar arquivos ZIP.

class zipimport.**zipimporter**(*archivepath*)

Cria uma nova instância de zipimporter. *archivepath* deve ser um caminho para um arquivo ZIP ou para um caminho específico dentro de um arquivo ZIP. Por exemplo, um *archivepath* de `foo/bar.zip/lib` procurará módulos no diretório `lib` dentro do arquivo ZIP `foo/bar.zip` (desde que exista).

ZipImportError é levantada se *archivepath* não apontar para um arquivo ZIP válido.

Alterado na versão 3.12: Os métodos `find_loader()` e `find_module()`, descontinuados na versão 3.10, foram removidos. Em vez disso, use *find_spec()*.

create_module(*spec*)

Implementação de *importlib.abc.Loader.create_module()* que retorna *None* para solicitar explicitamente a semântica padrão.

Adicionado na versão 3.10.

exec_module(*module*)

Implementação de *importlib.abc.Loader.exec_module()*.

Adicionado na versão 3.10.

find_spec(*fullname*, *target=None*)

Uma implementação de *importlib.abc.PathEntryFinder.find_spec()*.

Adicionado na versão 3.10.

get_code(*fullname*)

Retorna o objeto código para o módulo especificado. Levanta *ZipImportError* se o módulo não puder ser importado.

get_data(*pathname*)

Retorna os dados associados a *pathname*. Levanta *OSError* se o arquivo não for encontrado.

Alterado na versão 3.3: *IOError* costumava ser levantada, agora ela é um apelido para *OSError*.

get_filename(*fullname*)

Retorna o valor que `__file__` seria definido se o módulo especificado fosse importado. Levanta *ZipImportError* se o módulo não puder ser importado.

Adicionado na versão 3.1.

get_source(*fullname*)

Retorna o código-fonte do módulo especificado. Levanta *ZipImportError* se o módulo não puder ser encontrado, e retorna *None* se o arquivo contiver o módulo, mas não tiver o código-fonte para ele.

is_package(*fullname*)

Retorna `True` se o módulo especificado por *fullname* é um pacote. Levanta *ZipImportError* se o módulo não pode ser localizado.

load_module(*fullname*)

Carrega o módulo especificado por *fullname*. *fullname* deve ser o nome completo (com pontos) do módulo. Retorna o módulo importado em caso de sucesso e levanta *ZipImportError* em caso de falha.

Deprecated since version 3.10, will be removed in version 3.15: Use *exec_module()*.

invalidate_caches()

Limpa o cache interno de informações sobre arquivos encontrados no arquivo ZIP.

Adicionado na versão 3.10.

archive

O nome do arquivo ZIP associado ao importador, sem um possível subcaminho.

prefix

O subcaminho dentro do arquivo ZIP onde os módulos são pesquisados. Esta é a string vazia para objetos zipimporter que apontam para a raiz do arquivo ZIP.

Os atributos *archive* e *prefix*, quando combinados com uma barra, são iguais ao argumento original *archivepath* fornecido ao construtor da *zipimporter*.

31.1.2 Exemplos

Aqui está um exemplo que importa um módulo de um arquivo ZIP - observe que o módulo *zipimport* não é usado explicitamente.

```
$ unzip -l arquivo_exemplo.zip
Archive:  arquivo_exemplo.zip
  Length      Date    Time    Name
 --------    ----    ----    ----
     8467  01-01-00 12:30   exemplo.py
 --------                   -------
     8467                   1 file
```

```
>>> import sys
>>> # Adiciona o arquivo para a frente do caminho de pesquisa de módulos
>>> sys.path.insert(0, 'arquivo_exemplo.zip')
>>> import exemplo
>>> exemplo.__file__
'arquivo_exemplo.zip/exemplo.py'
```

31.2 pkgutil — Utilitário de extensão de pacote

Código-fonte: Lib/pkgutil.py

Este módulo fornece utilitários para o sistema de importação, em particular suporte a pacotes.

class pkgutil.**ModuleInfo**(*module_finder*, *name*, *ispkg*)

Um namedtuple que contém um breve resumo das informações de um módulo.

Adicionado na versão 3.6.

pkgutil.**extend_path**(*path*, *name*)

Estende o caminho de pesquisa para os módulos que compõem um pacote. O uso pretendido é colocar o seguinte código no `__init__.py` de um pacote:

```
from pkgutil import extend_path
__path__ = extend_path(__path__, __name__)
```

Para cada diretório em *sys.path* que tenha um subdiretório que corresponda ao nome do pacote, adiciona o subdiretório ao `__path__` do pacote. Isto é útil se quisermos distribuir diferentes partes de um único pacote lógico como múltiplos diretórios.

Também procura por arquivos `*.pkg` que começam onde `*` corresponde ao argumento *name*. Este recurso é semelhante aos arquivos `*.pth` (consulte o módulo *site* para mais informações), exceto que não considera casos especiais para linhas que começam com `import`. Um arquivo `*.pkg` é confiável por natureza: além de pular linhas em branco e ignorar comentários, todas as entradas encontradas em um arquivo `*.pkg` são adicionadas ao caminho, independentemente de existirem ou não no sistema de arquivos (este é um recurso).

Se o caminho de entrada não for uma lista (como é o caso de pacotes congelados), ele será retornado inalterado. O caminho de entrada não é modificado; uma cópia estendida é retornada. Os itens são anexados à cópia apenas no final.

Presume-se que *sys.path* seja uma sequência. Itens de *sys.path* que não sejam strings referentes a diretórios existentes são ignorados. Itens Unicode em *sys.path* que causam erros quando usados como nomes de arquivo podem fazer com que esta função levante uma exceção (em linha com o comportamento de *os.path.isdir()*).

`pkgutil.`**`get_importer`**(*path_item*)

Recupera um *localizador* para o *path_item* fornecido.

O localizador retornado é armazenado em cache em *sys.path_importer_cache* se ele foi criado recentemente por um gancho de caminho.

O cache (ou parte dele) pode ser limpo manualmente se uma nova varredura de *sys.path_hooks* for necessária.

Alterado na versão 3.3: Atualizado para ser baseado diretamente em *importlib* em vez de depender da emulação de importação interna do pacote da **PEP 302**.

`pkgutil.`**`iter_importers`**(*fullname=''*)

Produz objetos *localizador* para o nome do módulo fornecido.

Se *fullname* contiver `'.'`, os localizadores serão para o pacote que contém *fullname*, caso contrário, serão todos os localizadores de nível superior registrados (ou seja, aqueles em *sys.meta_path* e *sys.path_hooks*).

Se o módulo nomeado estiver em um pacote, esse pacote será importado como um efeito colateral da invocação desta função.

Se nenhum nome de módulo for especificado, todos os localizadores de nível superior serão produzidos.

Alterado na versão 3.3: Atualizado para ser baseado diretamente em *importlib* em vez de depender da emulação de importação interna do pacote da **PEP 302**.

`pkgutil.`**`iter_modules`**(*path=None*, *prefix=''*)

Levanta *ModuleInfo* para todos os submódulos em *path* ou, se *path* for `None`, todos os módulos de nível superior em *sys.path*.

path deve ser `None` ou uma lista de caminhos para procurar módulos.

prefix é uma string a ser exibida na frente de cada nome de módulo na saída.

> **Nota**
>
> Funciona apenas para um *localizador* que define um método `iter_modules()`. Esta interface não é padrão, portanto, o módulo também fornece implementações para *importlib.machinery.FileFinder* e *zipimport.zipimporter*.

Alterado na versão 3.3: Atualizado para ser baseado diretamente em *importlib* em vez de depender da emulação de importação interna do pacote da **PEP 302**.

pkgutil.**walk_packages**(*path=None*, *prefix=''*, *onerror=None*)

Levanta *ModuleInfo* para todos os módulos recursivamente em *path*, ou, se *path* for None, todos os módulos acessíveis.

path deve ser None ou uma lista de caminhos para procurar módulos.

prefix é uma string a ser exibida na frente de cada nome de módulo na saída.

Observe que esta função deve importar todos os *pacotes* (*não* todos os módulos!) no caminho *path* fornecido, para acessar o atributo __path__ para encontrar submódulos.

onerror é uma função que é chamada com um argumento (o nome do pacote que estava sendo importado) caso ocorra alguma exceção durante a tentativa de importação de um pacote. Se nenhuma função *onerror* for fornecida, *ImportError*s são capturadas e ignoradas, enquanto todas as outras exceções são propagadas, encerrando a busca.

Exemplos:

```
# lista todos os módulos que o python consegue acessars
walk_packages()

# lista todos os submódulos de ctypes
walk_packages(ctypes.__path__, ctypes.__name__ + '.')
```

Nota

Funciona apenas para um *localizador* que define um método iter_modules(). Esta interface não é padrão, portanto, o módulo também fornece implementações para *importlib.machinery.FileFinder* e *zipimport.zipimporter*.

Alterado na versão 3.3: Atualizado para ser baseado diretamente em *importlib* em vez de depender da emulação de importação interna do pacote da **PEP 302**.

pkgutil.**get_data**(*package*, *resource*)

Obtém um recurso de um pacote.

Este é um invólucro para a API *get_data* do *carregador*. O argumento *package* deve ser o nome de um pacote, no formato de módulo padrão (foo.bar). O argumento *resource* deve estar no formato de um nome de arquivo relativo, usando / como separador de caminho. O nome do diretório pai .. não é permitido, nem um nome raiz (começando com /).

A função retorna uma string binária que é o conteúdo do recurso especificado.

Para pacotes localizados no sistema de arquivos, que já foram importados, isso é o equivalente aproximado de:

```
d = os.path.dirname(sys.modules[package].__file__)
data = open(os.path.join(d, resource), 'rb').read()
```

Se o pacote não puder ser localizado ou carregado, ou se usar um *carregador* que não oferece suporte a *get_data*, None será retornado. Em particular, o *carregador* para *pacotes de espaço de nomes* não oferece suporte a *get_data*.

pkgutil.**resolve_name**(*name*)

Resolve um nome a um objeto.

Essa funcionalidade é usada em vários lugares na biblioteca padrão (veja bpo-12915) - e funcionalidades equivalentes também estão amplamente utilizadas em pacotes de terceiros, como setuptools, Django e Pyramid.

Espera-se que *name* seja uma string em um dos seguintes formatos, onde W é uma abreviação para um identificador Python válido e ponto representa um ponto literal nessas pseudo-regexes:

- W(.W)*

- `W(.W)*:(W(.W)*)?`

A primeira forma destina-se apenas à retrocompatibilidade. Ela presume que parte do nome pontilhado seja um pacote e o restante seja um objeto em algum lugar dentro desse pacote, possivelmente aninhado dentro de outros objetos. Como o ponto onde o pacote termina e a hierarquia de objetos começa não pode ser inferido por inspeção, tentativas repetidas de importação devem ser feitas com esta forma.

Na segunda forma, o chamador deixa claro o ponto de divisão por meio de um caractere de dois pontos: o nome pontilhado à esquerda do caractere de dois pontos representa um pacote a ser importado, e o nome pontilhado à direita representa a hierarquia de objetos dentro desse pacote. Apenas uma importação é necessária nesta forma. Se terminar com caractere de dois pontos, um objeto de módulo será retornado.

A função retornará um objeto (que pode ser um módulo) ou levantará uma das seguintes exceções:

`ValueError` – se *name* não estiver em um formato reconhecido.

`ImportError` – se uma importação falhou quando não deveria.

`AttributeError` – Se ocorrer uma falha ao percorrer a hierarquia de objetos dentro do pacote importado para chegar ao objeto desejado.

Adicionado na versão 3.9.

31.3 `modulefinder` — Procura módulos usados por um script

Código-fonte: Lib/modulefinder.py

Este módulo fornece uma classe *`ModuleFinder`* que pode ser usada para determinar o conjunto de módulos importados por um script. O `modulefinder.py` também pode ser executado como um script, fornecendo o nome do arquivo de um script Python como seu argumento, após o qual um relatório dos módulos importados será impresso.

`modulefinder.`**`AddPackagePath`**(*pkg_name*, *path*)

Registra que o pacote chamado *pkg_name* pode ser encontrado no caminho especificado em *path*

`modulefinder.`**`ReplacePackage`**(*oldname*, *newname*)

Permite especificar que o módulo chamado *oldname* é de fato o pacote chamado *newname*.

`class` `modulefinder.`**`ModuleFinder`**(*path=None*, *debug=0*, *excludes=[]*, *replace_paths=[]*)

Esta classe fornece os métodos *`run_script()`* e *`report()`* para determinar o conjunto de módulos importados por um script. *path* pode ser uma lista de diretórios para procurar por módulos; se não especificado, `sys.path` é usado. *debug* define o nível de depuração; valores mais altos fazem a classe imprimir mensagens de depuração sobre o que está fazendo. *excludes* é uma lista de nomes de módulos a serem excluídos da análise. *replace_paths* é uma lista de tuplas `(oldpath, newpath)` que serão substituídas nos caminhos dos módulos.

`report`()

Imprime um relatório na saída padrão que lista os módulos importados pelo script e seus caminhos, bem como os módulos que estão faltando ou parecem estar ausentes.

`run_script`(*pathname*)

Analisa o conteúdo do arquivo *pathname*, que deve conter o código Python.

`modules`

Um nome de módulo de mapeamento de dicionário para módulos. Veja *Exemplo de uso de ModuleFinder*.

31.3.1 Exemplo de uso de `ModuleFinder`

O script que será analisado posteriormente (bacon.py):

```
import re, itertools

try:
    import baconhameggs
except ImportError:
    pass

try:
    import guido.python.ham
except ImportError:
    pass
```

O script que irá gerar o relatório de bacon.py:

```
from modulefinder import ModuleFinder

finder = ModuleFinder()
finder.run_script('bacon.py')

print('Loaded modules:')
for name, mod in finder.modules.items():
    print('%s: ' % name, end='')
    print(','.join(list(mod.globalnames.keys())[:3]))

print('-'*50)
print('Modules not imported:')
print('\n'.join(finder.badmodules.keys()))
```

Exemplo de saída (pode variar dependendo da arquitetura):

```
Loaded modules:
_types:
copyreg:  _inverted_registry,_slotnames,__all__
re._compiler:  isstring,_sre,_optimize_unicode
_sre:
re._constants:  REPEAT_ONE,makedict,AT_END_LINE
sys:
re:  __module__,finditer,_expand
itertools:
__main__:  re,itertools,baconhameggs
re._parser:  _PATTERNENDERS,SRE_FLAG_UNICODE
array:
types:  __module__,IntType,TypeType
----------------------------------------------------
Modules not imported:
guido.python.ham
baconhameggs
```

31.4 `runpy` — Localizando e executando módulos Python

Código-fonte: Lib/runpy.py

O módulo *runpy* é usado para localizar e executar módulos Python sem importá-los primeiro. Seu principal uso é implementar a opção de linha de comando `-m` que permite que os scripts sejam localizados usando o espaço de nomes do módulo Python em vez do sistema de arquivos.

Observe que este *não* é um módulo isolado - todo o código é executado no processo atual, e quaisquer efeitos colaterais (como importações em cache de outros módulos) irão permanecer em vigor após o retorno da função.

Além disso, quaisquer funções e classes definidas pelo código executado não têm garantia de funcionar corretamente após o retorno de uma função *runpy*. Se essa limitação não for aceitável para um determinado caso de uso, *importlib* provavelmente será uma escolha mais adequada do que este módulo.

O módulo *runpy* fornece duas funções:

runpy.**run_module**(*mod_name*, *init_globals=None*, *run_name=None*, *alter_sys=False*)

Executa o código do módulo especificado e retorna o dicionário de globals do módulo resultante. O código do módulo é localizado primeiro usando o mecanismo de importação padrão (consulte **PEP 302** para detalhes) e então executado em um novo espaço de nomes de módulo.

O argumento *mod_name* deve ser um nome de módulo absoluto. Se o nome do módulo se referir a um pacote ao invés de um módulo normal, então esse pacote é importado e o submódulo *__main__* dentro desse pacote é então executado e o dicionário global do módulo resultante retornado.

O argumento opcional de dicionário *init_globals* pode ser usado para preencher previamente o dicionário global do módulo antes do código ser executado. *init_globals* não será modificado. Se qualquer uma das variáveis globais especiais abaixo for definida em *init_globals*, estas definições serão substituídas por *run_module()*.

As variáveis globais especiais `__name__`, `__spec__`, `__file__`, `__cached__`, `__loader__` e `__package__` são definidas no dicionário global antes que o código do módulo seja executado (Observe que este é um conjunto mínimo de variáveis - outras variáveis podem ser definidas implicitamente como um detalhe de implementação do interpretador).

`__name__` é definido como *run_name* se este argumento opcional não for *None*, para `mod_name + '.__main__'` se o módulo nomeado for um pacote e para o argumento *mod_name* caso contrário .

`__spec__` será definido adequadamente para o módulo *efetivamente* importado (isto é, `__spec__.name` vai sempre ser *mod_name* ou `mod_name + '.__main__'`, nunca *run_name*).

`__file__`, `__cached__`, `__loader__` e `__package__` são definidos como normal com base no spec de módulo.

Se o argumento *alter_sys* for fornecido e for avaliado como *True*, então `sys.argv[0]` será atualizado com o valor de `__file__` e `sys.modules[__name__]` é atualizado com um objeto de módulo temporário para o módulo que está sendo executado. Ambos `sys.argv[0]` e `sys.modules[__name__]` são restaurados para seus valores originais antes que a função retorne.

Note que esta manipulação de *sys* não é segura para thread. Outras threads podem ver o módulo parcialmente inicializado, bem como a lista alterada de argumentos. É recomendado que o módulo `sys` seja deixado sozinho ao invocar esta função a partir do código encadeado.

Ver também

A opção `-m` oferece funcionalidade equivalente na linha de comando.

Alterado na versão 3.1: Adicionada capacidade de executar pacotes procurando por um submódulo *__main__*.

Alterado na versão 3.2: Adicionada a variável global `__cached__` (veja **PEP 3147**).

Alterado na versão 3.4: Atualizado para aproveitar o recurso de spec de módulo adicionado por **PEP 451**. Isso permite que `__cached__` seja configurado corretamente para módulos executados desta forma, assim como garante que o nome real do módulo esteja sempre acessível como `__spec__.name`.

Alterado na versão 3.12: A definição de `__cached__`, `__loader__` e `__package__` foi descontinuada. Veja *ModuleSpec* para alternativas.

runpy.**run_path**(*path_name*, *init_globals=None*, *run_name=None*)

Executa o código no local do sistema de arquivos nomeado e retorna o dicionário de globals do módulo resultante. Assim como um nome de script fornecido à linha de comando CPython, *file_path* pode se referir a um

arquivo fonte Python, um arquivo compilado em bytecode ou uma entrada válida de *sys.path* contendo um módulo *__main__* (por exemplo, um arquivo zip contendo um arquivo de nível superior __main__.py).

Para um script simples, o código especificado é simplesmente executado em um novo espaço de nomes de módulo. Para uma entrada válida *sys.path* (normalmente um arquivo zip ou diretório), a entrada é primeiro adicionada ao início de sys.path. A função então procura e executa um módulo *__main__* usando o caminho atualizado. Observe que não há proteção especial contra invocar uma entrada __main__ existente localizada em outro lugar em sys.path se não houver tal módulo no local especificado.

O argumento opcional de dicionário *init_globals* pode ser usado para preencher previamente o dicionário de globals do módulo antes do código ser executado. *init_globals* não será modificado. Se qualquer uma das variáveis globais especiais abaixo for definida em *init_globals*, estas definições serão substituídas por *run_path()*.

As variáveis globais especiais __name__, __spec__, __file__, __cached__, __loader__ e __package__ são definidas no dicionário global antes que o código do módulo seja executado (Observe que este é um conjunto mínimo de variáveis - outras variáveis podem ser definidas implicitamente como um detalhe de implementação do interpretador).

__name__ é definido como *run_name* se este argumento opcional não for *None* e como '<run_path>' caso contrário.

Se *file_path* fizer referência diretamente a um arquivo de script (seja como fonte ou como bytecode pré-compilado), então __file__ será definido para *file_path* e __spec__, __cached__, __loader__ e __package__ serão todos definidos como *None*.

Se *file_path* for uma referência a uma entrada válida de *sys.path*, então __spec__ será definido apropriadamente para o módulo *__main__* importado (ou seja, __spec__.name sempre será __main__). __file__, __cached__, __loader__ e __package__ serão definidos como normal com base no spec de módulo.

Várias alterações também são feitas no módulo *sys*. Primeiramente, *sys.path* pode ser alterado conforme descrito acima. sys.argv[0] é atualizado com o valor de *file_path* e sys.modules[__name__] é atualizado com um objeto de módulo temporário para o módulo que está sendo executado. Todas as modificações em itens em *sys* são revertidas antes que a função retorne.

Note que, diferentemente de *run_module()*, as alterações feitas em *sys* não são opcionais nesta função, pois esses ajustes são essenciais para permitir a execução das entradas do *sys.path*. Como as limitações de segurança de threads ainda se aplicam, o uso dessa função no código encadeado deve ser serializado com a trava de importação ou delegado a um processo separado.

Ver também

using-on-interface-options para funcionalidade equivalente na linha de comando (python path/to/script).

Adicionado na versão 3.2.

Alterado na versão 3.4: Atualizado para aproveitar o recurso de spec de módulo adicionado por **PEP 451**. Isso permite que __cached__ seja definido corretamente no caso em que __main__ é importado de uma entrada *sys.path* válida em vez de ser executado diretamente.

Alterado na versão 3.12: A definição de __cached__, __loader__ e __package__ foi descontinuada.

Ver também

PEP 338 – **Executando módulos como scripts**
PEP escrita e implementada por Nick Coghlan.

PEP 366 – **Importações relativas explícitas do módulo principal**
PEP escrita e implementada por Nick Coghlan.

PEP 451 – **Um tipo ModuleSpec para o sistema de importação**
PEP escrita e implementada por Eric Snow

using-on-general - Detalhes da linha de comando do CPython

A função *importlib.import_module()*

31.5 importlib — A implementação de import

Adicionado na versão 3.1.

Código-fonte: Lib/importlib/__init__.py

31.5.1 Introdução

O pacote *importlib* tem três propósitos.

Uma é fornecer a implementação da instrução `import` (e, portanto, por extensão, a função *__import__()*) no código-fonte Python. Isso fornece uma implementação de `import` que é portável para qualquer interpretador Python. Isso também fornece uma implementação que é mais fácil de compreender do que aquela implementada em uma linguagem de programação diferente do Python.

Dois, os componentes para implementar `import` são expostos neste pacote, tornando mais fácil para os usuários criarem seus próprios objetos personalizados (conhecidos genericamente como *importador*) para participar do processo de importação.

Terceiro, o pacote contém módulos que expõem funcionalidades adicionais para gerenciar aspectos de pacotes Python:

- *importlib.metadata* apresenta acesso a metadados de distribuições de terceiros.
- *importlib.resources* fornece rotinas para acessar "recursos" não codificados de pacotes Python.

Ver também

import
A referência da linguagem para a instrução `import`.

Especificação dos pacotes
Especificação original dos pacotes. Algumas semânticas mudaram desde a redação deste documento (por exemplo, redirecionamento baseado em `None` em *sys.modules*).

A função *__import__()*
A instrução `import` é um açúcar sintático para esta função.

A inicialização do caminho de pesquisa de módulos sys.path
A inicialização de *sys.path*.

PEP 235
Importação em plataformas que diferenciam maiúsculo de minúsculo

PEP 263
Definindo codificações do código-fonte do Python

PEP 302
Novos ganchos de importação

PEP 328
Importações: Multilinha e Absoluto/Relativo

PEP 366
Importações relativas explícitas do módulo principal

PEP 420
Pacotes de espaço de nomes implícitos

PEP 451
Um tipo de ModuleSpec para o sistema de importação

PEP 488
Eliminação de arquivos PYO

PEP 489
Inicialização de módulo extensão multifase

PEP 552
pycs determinísticos

PEP 3120
Usando UTF-8 como fonte padrão de codificação

PEP 3147
Diretórios de repositório de PYC

31.5.2 Funções

`importlib.__import__`(*name*, *globals=None*, *locals=None*, *fromlist=()*, *level=0*)

Uma implementação da função embutida *__import__()*.

> **Nota**
>
> A importação programática de módulos deve usar *import_module()* em vez desta função.

`importlib.import_module`(*name*, *package=None*)

Importa um módulo. O argumento *name* especifica qual módulo importar em termos absolutos ou relativos (por exemplo, `pkg.mod` ou `..mod`). Se o nome for especificado em termos relativos, então o argumento *package* deve ser definido como o nome do pacote que atuará como âncora para resolver o nome do pacote (por exemplo, `import_module('..mod', 'pkg.subpkg')` importará `pkg.mod`).

A função *import_module()* atua como um wrapper simplificador em torno de *importlib.__import__()*. Isso significa que toda a semântica da função é derivada de *importlib.__import__()*. A diferença mais importante entre essas duas funções é que *import_module()* retorna o pacote ou módulo especificado (por exemplo, `pkg.mod`), enquanto *__import__()* retorna o pacote ou módulo de nível superior (por exemplo, `pkg`).

Se você estiver importando dinamicamente um módulo que foi criado desde que o interpretador iniciou a execução (por exemplo, criou um arquivo fonte Python), você pode precisar chamar *invalidate_caches()* para que o novo módulo seja notado pelo sistema de importação.

Alterado na versão 3.3: Os pacotes pai são importados automaticamente.

`importlib.invalidate_caches`()

Invalida os caches internos dos localizadores armazenados em *sys.meta_path*. Se um localizador implementar `invalidate_caches()` então ele será chamado para realizar a invalidação. Esta função deve ser chamada se algum módulo for criado/instalado enquanto seu programa estiver em execução para garantir que todos os localizadores notarão a existência do novo módulo.

Adicionado na versão 3.3.

Alterado na versão 3.10: Pacotes de espaço de nomes criados/instalados em um local *sys.path* diferente após o mesmo espaço de nomes já ter sido importado são notados.

`importlib.reload`(*module*)

Recarrega um *module* importado anteriormente. O argumento deve ser um objeto módulo, portanto deve ter sido importado com êxito antes. Isso é útil se você editou o arquivo fonte do módulo usando um editor externo e deseja experimentar a nova versão sem sair do interpretador Python. O valor de retorno é o objeto módulo (que pode ser diferente se a reimportação fizer com que um objeto diferente seja colocado em *sys.modules*).

Quando *reload()* é executado:

- O código do módulo Python é recompilado e o código em nível de módulo é reexecutado, definindo um novo conjunto de objetos que são vinculados a nomes no dicionário do módulo reutilizando o *carregador* que originalmente carregou o módulo. A função `init` dos módulos de extensão não é chamada uma segunda vez.
- Tal como acontece com todos os outros objetos em Python, os objetos antigos só são recuperados depois que suas contagens de referências caem para zero.
- Os nomes no espaço de nomes do módulo são atualizados para apontar para quaisquer objetos novos ou alterados.
- Outras referências aos objetos antigos (como nomes externos ao módulo) não são religadas para se referir aos novos objetos e devem ser atualizadas em cada espaço de nomes onde ocorrem, se isso for desejado.

Existem várias outras ressalvas:

Quando um módulo é recarregado, seu dicionário (contendo as variáveis globais do módulo) é retido. As redefinições de nomes vão substituir as definições antigas, portanto isso geralmente não é um problema. Se a nova versão de um módulo não definir um nome definido pela versão antiga, a definição antiga permanecerá. Este recurso pode ser usado para vantagem do módulo se ele mantiver uma tabela global ou cache de objetos – com uma instrução `try` ele pode testar a presença da tabela e pular sua inicialização se desejar:

```
try:
    cache
except NameError:
    cache = {}
```

Geralmente não é muito útil recarregar módulos embutidos ou carregados dinamicamente. Recarregar *sys*, *__main__*, *builtins* e outros módulos principais não é recomendado. Em muitos casos, os módulos de extensão não são projetados para serem inicializados mais de uma vez e podem falhar de maneiras arbitrárias quando recarregados.

Se um módulo importa objetos de outro módulo usando `from` … `import` …, chamar *reload()* para o outro módulo não redefine os objetos importados dele – uma maneira de contornar isso é executar novamente a instrução `from`, outra é usar `import` e nomes qualificados (*module.name*).

Se um módulo instancia instâncias de uma classe, recarregar o módulo que define a classe não afeta as definições de método das instâncias – elas continuam a usar a definição de classe antiga. O mesmo se aplica às classes derivadas.

Adicionado na versão 3.4.

Alterado na versão 3.7: *ModuleNotFoundError* é levantada quando o módulo que está sendo recarregado não possui um *ModuleSpec*.

Aviso

This function is not thread-safe. Calling it from multiple threads can result in unexpected behavior. It's recommended to use the *threading.Lock* or other synchronization primitives for thread-safe module reloading.

31.5.3 `importlib.abc` – classes base abstratas relacionadas a importação

Código-fonte: Lib/importlib/abc.py

O módulo *importlib.abc* contém todas as principais classes base abstratas usadas por `import`. Algumas subclasses das classes base abstratas principais também são fornecidas para ajudar na implementação dss ABCs principais.

Hierarquia de ABC:

```
object
 +-- MetaPathFinder
 +-- PathEntryFinder
 +-- Loader
      +-- ResourceLoader --------+
      +-- InspectLoader          |
           +-- ExecutionLoader --+
                                 +-- FileLoader
                                 +-- SourceLoader
```

class importlib.abc.**MetaPathFinder**

Uma classe base abstrata que representa um *localizador de metacaminho*.

Adicionado na versão 3.3.

Alterado na versão 3.10: Não é mais uma subclasse de `Finder`.

find_spec(*fullname*, *path*, *target=None*)

Um método abstrato para encontrar um *spec* para o módulo especificado. Se esta for uma importação de nível superior, *path* será `None`. Caso contrário, esta é uma busca por um subpacote ou módulo e *path* será o valor de `__path__` do pacote pai. Se um spec não puder ser encontrado, `None` será retornado. Quando passado, `target` é um objeto de módulo que o localizador pode usar para fazer uma estimativa mais informada sobre qual spec retornar. *importlib.util.spec_from_loader()* pode ser útil para implementar `MetaPathFinders` concretos.

Adicionado na versão 3.4.

invalidate_caches()

Um método opcional que, quando chamado, deve invalidar qualquer cache interno usado pelo localizador. Usado por *importlib.invalidate_caches()* ao invalidar os caches de todos os localizadores em *sys.meta_path*.

Alterado na versão 3.4: Retorna `None` quando chamado em vez de *NotImplemented*.

class importlib.abc.**PathEntryFinder**

Uma classe base abstrata que representa um *localizador de entrada de caminho*. Embora tenha algumas semelhanças com *MetaPathFinder*, `PathEntryFinder` deve ser usado apenas dentro do subsistema de importação baseado em caminho fornecido por *importlib.machinery.PathFinder*.

Adicionado na versão 3.3.

Alterado na versão 3.10: Não é mais uma subclasse de `Finder`.

find_spec(*fullname*, *target=None*)

Um método abstrato para encontrar um *spec* para o módulo especificado. O localizador irá procurar pelo módulo apenas dentro do *entrada de caminho* ao qual ele está atribuído. Se um spec não puder ser encontrado, `None` será retornado. Quando passado, `target` é um objeto de módulo que o localizador pode usar para fazer uma estimativa mais informada sobre qual spec retornar. *importlib.util.spec_from_loader()* pode ser útil para implementar `PathEntryFinders` concretos.

Adicionado na versão 3.4.

invalidate_caches()

Um método opcional que, quando chamado, deve invalidar qualquer cache interno usado pelo localizador. Usado por *importlib.machinery.PathFinder.invalidate_caches()* ao invalidar os caches de todos os localizadores em cache.

class importlib.abc.**Loader**

Uma classe base abstrata para um *carregador*. Veja **PEP 302** para a definição exata de um carregador.

Carregadores que desejam oferecer suporte a leitura de recursos devem implementar um método `get_resource_reader()` conforme especificado por *importlib.resources.abc.ResourceReader*.

Alterado na versão 3.7: Introduzido o método opcional `get_resource_reader()`.

create_module(*spec*)

Um método que retorna o objeto do módulo a ser usado ao importar um módulo. Este método pode retornar `None`, indicando que a semântica padrão de criação do módulo deve ocorrer.

Adicionado na versão 3.4.

Alterado na versão 3.6: Este método não é mais opcional quando *exec_module()* é definido.

exec_module(*module*)

Um método abstrato que executa o módulo em seu próprio espaço de nomes quando um módulo é importado ou recarregado. O módulo já deve estar inicializado quando *exec_module()* for chamado. Quando este método existir, *create_module()* deve ser definido.

Adicionado na versão 3.4.

Alterado na versão 3.6: *create_module()* deve ser definido.

load_module(*fullname*)

Um método legado para carregar um módulo. Se o módulo não puder ser carregado, *ImportError* será levantada, caso contrário, o módulo carregado será retornado.

Se o módulo solicitado já existir em *sys.modules*, esse módulo deverá ser usado e recarregado. Caso contrário, o carregador deve criar um novo módulo e inseri-lo em *sys.modules* antes de qualquer carregamento começar, para evitar a recursão da importação. Se o carregador inseriu um módulo e o carregamento falhar, ele deverá ser removido pelo carregador de *sys.modules*; módulos já em *sys.modules* antes do carregador iniciar a execução devem ser deixados sozinhos.

O carregador deve definir vários atributos no módulo (observe que alguns desses atributos podem mudar quando um módulo é recarregado):

- `module.__name__`
- `module.__file__`
- `module.__cached__` *(descontinuado)*
- `module.__path__`
- `module.__package__` *(descontinuado)*
- `module.__loader__` *(descontinuado)*

Quando *exec_module()* está disponível, então a funcionalidade compatível com versões anteriores é fornecida.

Alterado na versão 3.4: Levanta *ImportError* quando chamado em vez de *NotImplementedError*. Funcionalidade fornecida quando *exec_module()* está disponível.

Deprecated since version 3.4, will be removed in version 3.15: A API recomendada para carregar um módulo é *exec_module()* (e *create_module()*). Os carregadores devem implementá-lo em vez de *load_module()*. O mecanismo de importação cuida de todas as outras responsabilidades de *load_module()* quando *exec_module()* é implementado.

class importlib.abc.**ResourceLoader**

Substituída por TraversableResources

Uma classe base abstrata para um *carregador* que implementa o protocolo opcional **PEP 302** para carregar recursos arbitrários do back-end de armazenamento.

Descontinuado desde a versão 3.7: This ABC is deprecated in favour of supporting resource loading through *importlib.resources.abc.TraversableResources*. This class exists for backwards compatibility only with other ABCs in this module.

abstractmethod get_data(*path*)

Um método abstrato para retornar os bytes dos dados localizados em *path*. Carregadores que possuem um backend de armazenamento arquivo ou similar que permite o armazenamento de dados arbitrários podem implementar esse método abstrato para

fornecer acesso direto aos dados armazenados. *OSError* deve ser levantada se o *path* não puder ser encontrado. Espera-se que o *path* seja construído usando o atributo `__file__` de um módulo ou um item de `__path__` de um pacote.

Alterado na versão 3.4: Levanta *OSError* em vez de *NotImplementedError*.

class importlib.abc.**InspectLoader**

Uma classe base abstrata para um *carregador* que implementa o protocolo opcional da **PEP 302** para carregadores que inspecionam módulos.

get_code(*fullname*)

Retorna o objeto de código para um módulo, ou `None` se o módulo não tiver um objeto código (como seria o caso, por exemplo, para um módulo embutido). Levanta um *ImportError* se o carregador não conseguir encontrar o módulo solicitado.

> **Nota**
>
> Embora o método tenha uma implementação padrão, sugere-se que ele seja substituído, se possível, para desempenho.

Alterado na versão 3.4: Não é mais abstrato e uma implementação concreta é fornecida.

abstractmethod get_source(*fullname*)

Um método abstrato para retornar a fonte de um módulo. Ele é retornado como uma string de texto usando *novas linhas universais*, traduzindo todos os separadores de linha reconhecidos em caracteres `'\n'`. Retorna `None` se nenhuma fonte estiver disponível (por exemplo, um módulo embutido). Levanta *ImportError* se o carregador não puder encontrar o módulo especificado.

Alterado na versão 3.4: Levanta *ImportError* em vez de *NotImplementedError*.

is_package(*fullname*)

Um método opcional para retornar um valor verdadeiro se o módulo for um pacote, um valor falso caso contrário. *ImportError* é levantada se o *carregador* não puder encontrar o módulo.

Alterado na versão 3.4: Levanta *ImportError* em vez de *NotImplementedError*.

static source_to_code(*data*, *path='<string>'*)

Cria um objeto código a partir de código-fonte Python

O argumento *data* pode ser qualquer coisa a qual a função *compile()* tenha suporte (por exemplo, string ou bytes). O argumento *path* deve ser o "caminho" de onde o código-fonte se originou, o que pode ser um conceito abstrato (por exemplo, local em um arquivo zip).

Com o objeto de código subsequente, é possível executá-lo em um módulo executando `exec(code, module.__dict__)`.

Adicionado na versão 3.4.

Alterado na versão 3.5: O método foi tornado estático.

exec_module(*module*)

Implementação de *Loader.exec_module()*.

Adicionado na versão 3.4.

load_module(*fullname*)

Implementação de *Loader.load_module()*.

Deprecated since version 3.4, will be removed in version 3.15: use *exec_module()*.

class importlib.abc.**ExecutionLoader**

Uma classe base abstrata que herda de *InspectLoader* que, quando implementada, ajuda um módulo a ser executado como um script. A ABC representa um protocolo opcional da **PEP 302**.

abstractmethod get_filename(*fullname*)

Um método abstrato que deve retornar o valor de `__file__` para o módulo especificado. Se nenhum caminho estiver disponível, *ImportError* é levantada.

Se o código-fonte estiver disponível, o método deverá retornar o caminho para o arquivo fonte, independentemente de um bytecode ter sido usado para carregar o módulo.

Alterado na versão 3.4: Levanta *ImportError* em vez de *NotImplementedError*.

class importlib.abc.**FileLoader**(*fullname*, *path*)

Uma classe base abstrata que herda de *ResourceLoader* e *ExecutionLoader*, fornecendo implementações concretas de *ResourceLoader.get_data()* e *ExecutionLoader.get_filename()*.

O argumento *fullname* é um nome totalmente resolvido do módulo que o carregador deve manipular. O argumento *path* é o caminho para o arquivo do módulo.

Adicionado na versão 3.3.

name

O nome do módulo que o carregador pode manipular.

path

Caminho para o arquivo do módulo

load_module(*fullname*)

Chama `load_module()` do super.

Deprecated since version 3.4, will be removed in version 3.15: Use *Loader.exec_module()*

abstractmethod get_filename(*fullname*)

Retorna *path*.

abstractmethod get_data(*path*)

Lê *caminho* como um arquivo binário e retorna os bytes dele.

class importlib.abc.**SourceLoader**

Uma classe base abstrata para implementar o carregamento de arquivo fonte (e opcionalmente bytecode). A classe herda de *ResourceLoader* e *ExecutionLoader*, exigindo a implementação de:

- *ResourceLoader.get_data()*
- ***ExecutionLoader.get_filename()***
  Deve retornar apenas o caminho para o arquivo fonte; o carregamento sem fonte não é suportado.

Os métodos abstratos definidos por esta classe são para adicionar suporte opcional ao arquivo bytecode. Não implementar esses métodos opcionais (ou fazer com que eles levantem *NotImplementedError*) faz com que o carregador funcione apenas com código-fonte. Implementar os métodos permite que o carregador funcione com arquivos de código-fonte *e* bytecode; ele não permite carregamento *sem fonte* onde apenas o bytecode é fornecido. Arquivos de bytecode são uma otimização para acelerar o carregamento removendo a etapa de análise do compilador do Python e, portanto, nenhuma API específica de bytecode é exposta.

path_stats(*path*)

Método abstrato opcional que retorna um *dict* contendo metadados sobre o caminho especificado. As chaves de dicionário suportadas são:

- `'mtime'` (obrigatório): um número inteiro ou de ponto flutuante que representa o tempo de modificação do código-fonte;
- `'size'` (opcional): o tamanho em bytes do código-fonte.

Quaisquer outras chaves no dicionário são ignoradas, para permitir extensões futuras. Se o caminho não puder ser manipulado, *OSError* é levantada.

Adicionado na versão 3.3.

Alterado na versão 3.4: Levanta *OSError* em vez de *NotImplementedError*.

path_mtime(*path*)

Método abstrato opcional que retorna o horário de modificação do caminho especificado.

Descontinuado desde a versão 3.3: Este método foi descontinuado em favor de *path_stats()*. Você não precisa implementá-lo, mas ele ainda está disponível para fins de compatibilidade. Levanta *OSError* se o caminho não puder ser manipulado.

Alterado na versão 3.4: Levanta *OSError* em vez de *NotImplementedError*.

set_data(*path*, *data*)

Método abstrato opcional que grava os bytes especificados em um caminho de arquivo. Quaisquer diretórios intermediários que não existam devem ser criados automaticamente.

Quando a gravação no caminho falhar porque o caminho é somente leitura (*errno.EACCES*/*PermissionError*), não propaga a exceção.

Alterado na versão 3.4: Não mais levanta *NotImplementedError* quando chamado.

get_code(*fullname*)

Implementação concreta de *InspectLoader.get_code()*.

exec_module(*module*)

Implementação concreta de *Loader.exec_module()*.

Adicionado na versão 3.4.

load_module(*fullname*)

Implementação concreta de *Loader.load_module()*.

Deprecated since version 3.4, will be removed in version 3.15: Use *exec_module()*.

get_source(*fullname*)

Implementação concreta de *InspectLoader.get_source()*.

is_package(*fullname*)

Implementação concreta de *InspectLoader.is_package()*. Um módulo é determinado como um pacote se seu caminho de arquivo (conforme fornecido por *ExecutionLoader.get_filename()*) for um arquivo chamado `__init__` quando a extensão do arquivo for removida **e** o nome do módulo em si não terminar em `__init__`.

class importlib.abc.**ResourceReader**

Substituída por TraversableResources

Uma *classe base abstrata* para fornecer a capacidade de ler *recursos*.

Da perspectiva deste ABC, um *recurso* é um artefato binário que é enviado dentro de um pacote. Normalmente isso é algo como um arquivo de dados que fica próximo ao arquivo `__init__.py` do pacote. O objetivo desta classe é ajudar a abstrair o acesso a tais arquivos de dados para que não importe se o pacote e seu(s) arquivo(s) de dados estão armazenados, por exemplo, em um arquivo zip versus no sistema de arquivos.

Para qualquer um dos métodos desta classe, espera-se que o argumento *resource* seja um *objeto caminho ou similar* que representa conceitualmente apenas um nome de arquivo. Isso significa que nenhum caminho de subdiretório deve ser incluído no argumento *resource*. Isso ocorre porque a localização do pacote para o qual o leitor se destina, atua como o "diretório". Portanto, a metáfora para diretórios e nomes de arquivos são pacotes e recursos, respectivamente. É também por isso que se espera que as instâncias dessa classe se correlacionem diretamente a um pacote específico (em vez de representar potencialmente vários pacotes ou um módulo).

Carregadores que desejam oferecer suporte à leitura de recursos devem fornecer um método chamado `get_resource_reader(nomecompleto)` que retorna um objeto implementando esta interface ABC. Se o módulo especificado por nomecompleto não for um pacote, este método deve retornar *None*. Um objeto compatível com este ABC só deve ser retornado quando o módulo especificado for um pacote.

Adicionado na versão 3.7.

Deprecated since version 3.12, removed in version 3.14: Use *importlib.resources.abc.TraversableResources*.

abstractmethod open_resource(*resource*)

Retorna um *objeto arquivo ou similar* aberto para leitura binária de *resource*.

Se o recurso não puder ser encontrado, `FileNotFoundError` é levantada.

abstractmethod resource_path(*resource*)

Retorna o caminho do sistema de arquivos para *resource*.

Se o recurso não existir concretamente no sistema de arquivos, levanta `FileNotFoundError`.

abstractmethod is_resource(*name*)

Retorna `True` se o *name* nomeado for considerado um recurso. `FileNotFoundError` é levantada se *name* não existir.

abstractmethod contents()

Retorna um *iterável* de strings sobre o conteúdo do pacote. Observe que não é necessário que todos os nomes retornados pelo iterador sejam recursos reais, por exemplo, é aceitável retornar nomes para os quais `is_resource()` seria falso.

Permitir que nomes que não são recursos sejam retornados é permitir situações em que a forma como um pacote e seus recursos são armazenados é conhecida a priori e os nomes que não são recursos seriam úteis. Por exemplo, o retorno de nomes de subdiretórios é permitido para que, quando se souber que o pacote e os recursos estão armazenados no sistema de arquivos, esses nomes de subdiretórios possam ser usados diretamente.

O método abstrato retorna um iterável sem itens.

class importlib.abc.**Traversable**

Um objeto com um subconjunto de métodos de `pathlib.Path` adequados para percorrer diretórios e abrir arquivos.

Para uma representação do objeto no sistema de arquivos, use `importlib.resources.as_file()`.

Adicionado na versão 3.9.

Deprecated since version 3.12, removed in version 3.14: Use `importlib.resources.abc.Traversable`.

name

Abstrato. O nome base deste objeto sem nenhuma referência pai.

abstractmethod iterdir()

Produz objetos `Traversable` em `self`.

abstractmethod is_dir()

Retorna `True` se `self` for um diretório.

abstractmethod is_file()

Retorna `True` se `self` for um arquivo.

abstractmethod joinpath(*child*)

Retorna o filho Traversable em `self`.

abstractmethod __truediv__(*child*)

Retorna o filho `Traversable` em `self`.

abstractmethod open(*mode='r'*, **args*, ***kwargs*)

mode pode ser 'r' ou 'rb' para abrir como texto ou binário. Retorna um manipulador adequado para leitura (o mesmo que `pathlib.Path.open`).

Ao abrir como texto, aceita parâmetros de codificação como os aceitos por `io.TextIOWrapper`.

read_bytes()

Lê o conteúdo de `self` como bytes.

read_text(*encoding=None*)

Lê o conteúdo de `self` como texto.

class importlib.abc.**TraversableResources**

Uma classe base abstrata para leitores de recursos capaz de servir a interface *importlib.resources.files()*. Subclasse *importlib.resources.abc.ResourceReader* e fornece implementações concretas dos métodos abstratos de *importlib.resources.abc.ResourceReader*. Portanto, qualquer carregador que forneça *importlib.abc.TraversableResources* também fornece ResourceReader.

Espera-se que os carregadores que desejam oferecer suporte à leitura de recursos implementem essa interface.

Adicionado na versão 3.9.

Deprecated since version 3.12, removed in version 3.14: Use *importlib.resources.abc.TraversableResources*.

abstractmethod files()

Retorna um objeto *importlib.resources.abc.Traversable* para o pacote carregado.

31.5.4 importlib.machinery – Importadores e ganchos de caminho

Código-fonte: Lib/importlib/machinery.py

Este módulo contém vários objetos que ajudam o `import` a localizar e carregar módulos.

importlib.machinery.**SOURCE_SUFFIXES**

Uma lista de strings representando os sufixos de arquivo reconhecidos para módulos em código-fonte.

Adicionado na versão 3.3.

importlib.machinery.**DEBUG_BYTECODE_SUFFIXES**

Uma lista de strings representando os sufixos de arquivo para módulos em bytecode não otimizado.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.5: Use *BYTECODE_SUFFIXES* instead.

importlib.machinery.**OPTIMIZED_BYTECODE_SUFFIXES**

Uma lista de strings representando os sufixos de arquivo para módulos em bytecode otimizado.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.5: Use *BYTECODE_SUFFIXES* instead.

importlib.machinery.**BYTECODE_SUFFIXES**

Uma lista de strings representando os sufixos de arquivo reconhecidos para módulos em bytecode (incluindo com ponto no início).

Adicionado na versão 3.3.

Alterado na versão 3.5: O valor não mais depende de `__debug__`.

importlib.machinery.**EXTENSION_SUFFIXES**

Uma lista de strings representando os sufixos de arquivo reconhecidos para módulos de extensão.

Adicionado na versão 3.3.

importlib.machinery.**all_suffixes**()

Retorna uma lista combinada de strings representando todos os sufixos de arquivo para módulos reconhecidos pelo maquinário de importação padrão. Este é um auxiliar para código que simplesmente precisa saber se um caminho de sistema de arquivos potencialmente se refere a um módulo sem precisar de detalhes sobre o tipo de módulo (por exemplo, *inspect.getmodulename()*).

Adicionado na versão 3.3.

class importlib.machinery.**BuiltinImporter**

Um *importador* para módulos embutidos. Todos os módulos embutidos conhecidos estão listados em *sys.builtin_module_names*. Esta classe implementa os ABCs *importlib.abc.MetaPathFinder* e *importlib.abc.InspectLoader*.

Somente métodos de classe são definidos por esta classe para aliviar a necessidade de instanciação.

Alterado na versão 3.5: Como parte da **PEP 489**, o importador embutido agora implementa `Loader.create_module()` e `Loader.exec_module()`

class importlib.machinery.**FrozenImporter**

Um *importador* para módulos congelados. Esta classe implementa os ABCs *importlib.abc.MetaPathFinder* e *importlib.abc.InspectLoader*.

Somente métodos de classe são definidos por esta classe para aliviar a necessidade de instanciação.

Alterado na versão 3.4: Ganhou os métodos `create_module()` e `exec_module()`.

class importlib.machinery.**WindowsRegistryFinder**

Localizador para módulos declarados no registro do Windows. Esta classe implementa a ABC *importlib.abc.MetaPathFinder*.

Somente métodos de classe são definidos por esta classe para aliviar a necessidade de instanciação.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.6: Use a configuração de *site* em vez disso. Versões futuras do Python podem não habilitar esse localizador por padrão.

class importlib.machinery.**PathFinder**

Um *Localizador* para atributos *sys.path* e pacote `__path__`. Esta classe implementa a ABC *importlib.abc.MetaPathFinder*.

Somente métodos de classe são definidos por esta classe para aliviar a necessidade de instanciação.

classmethod find_spec(*fullname*, *path=None*, *target=None*)

Método de classe que tenta encontrar um *spec* para o módulo especificado por *fullname* em *sys.path* ou, se definido, em *path*. Para cada entrada de caminho pesquisada, *sys.path_importer_cache* é verificado. Se um objeto não falso for encontrado, ele será usado como o *localizador de entrada de caminho* para procurar o módulo que está sendo pesquisado. Se nenhuma entrada for encontrada em *sys.path_importer_cache*, então *sys.path_hooks* será pesquisado para um localizador para a entrada de caminho e, se encontrado, será armazenado em *sys.path_importer_cache* junto com a consulta sobre o módulo. Se nenhum localizador for encontrado, então `None` será armazenado no cache e retornado.

Adicionado na versão 3.4.

Alterado na versão 3.5: Se o diretório de trabalho atual — representado por uma string vazia — não for mais válido, `None` será retornado, mas nenhum valor será armazenado em cache em *sys.path_importer_cache*.

classmethod invalidate_caches()

Chama *importlib.abc.PathEntryFinder.invalidate_caches()* em todos os localizadores armazenados em *sys.path_importer_cache* que definem o método. Caso contrário, as entradas em *sys.path_importer_cache* definidas como `None` são excluídas.

Alterado na versão 3.7: Entradas de `None` em *sys.path_importer_cache* são excluídas.

Alterado na versão 3.4: Chama objetos em *sys.path_hooks* com o diretório de trabalho atual para `''` (ou seja, a string vazia).

class importlib.machinery.**FileFinder**(*path*, **loader_details*)

Uma implementação concreta de *importlib.abc.PathEntryFinder* que armazena em cache os resultados do sistema de arquivos.

O argumento *path* é o diretório que o localizador é responsável por pesquisar.

O argumento *loader_details* é um número variável de tuplas de 2 itens, cada uma contendo um carregador e uma sequência de sufixos de arquivo que o carregador reconhece. Espera-se que os carregadores sejam chamáveis que aceitam dois argumentos do nome do módulo e o caminho para o arquivo encontrado.

O localizador armazenará em cache o conteúdo do diretório conforme necessário, fazendo chamadas de "stat" para cada pesquisa de módulo para verificar se o cache não está desatualizado. Como a obsolescência do cache depende da granularidade das informações de estado do sistema operacional do sistema de arquivos, há uma condição de corrida potencial de busca por um módulo, criação de um novo arquivo e, em seguida, busca pelo módulo que o novo arquivo representa. Se as operações ocorrerem rápido o suficiente para caber na granularidade das chamadas de "stat", a pesquisa do módulo falhará. Para evitar que isso aconteça, ao criar um módulo dinamicamente, certifique-se de chamar *importlib.invalidate_caches()*.

Adicionado na versão 3.3.

path

O caminho que o localizador irá procurar.

find_spec(*fullname*, *target=None*)

Tenta encontrar o spec para manipular *fullname* dentro de *path*.

Adicionado na versão 3.4.

invalidate_caches()

Limpa o cache interno.

classmethod path_hook(**loader_details*)

Um método de classe que retorna uma clausura para uso em *sys.path_hooks*. Uma instância de *FileFinder* é retornada pela clausura usando o argumento de caminho dado à clausura diretamente e *loader_details* indiretamente.

Se o argumento para a clausura não for um diretório existente, *ImportError* será levantada.

class importlib.machinery.**SourceFileLoader**(*fullname*, *path*)

Uma implementação concreta de *importlib.abc.SourceLoader* subclassificando *importlib.abc.FileLoader* e fornecendo algumas implementações concretas de outros métodos.

Adicionado na versão 3.3.

name

O nome do módulo que este carregador irá manipular.

path

O caminho para o arquivo fonte.

is_package(*fullname*)

Retorna `True` se *path* parecer ser para um pacote.

path_stats(*path*)

Implementação concreta de *importlib.abc.SourceLoader.path_stats()*.

set_data(*path*, *data*)

Implementação concreta de *importlib.abc.SourceLoader.set_data()*.

load_module(*name=None*)

Implementação concreta de *importlib.abc.Loader.load_module()* onde a especificação do nome do módulo para carregar é opcional

Deprecated since version 3.6, will be removed in version 3.15: Use *importlib.abc.Loader.exec_module()*.

class importlib.machinery.**SourcelessFileLoader**(*fullname*, *path*)

Uma implementação concreta de *importlib.abc.FileLoader* que pode importar arquivos de bytecode (ou seja, não existem arquivos de código-fonte).

Observe que o uso direto de arquivos de bytecode (e, portanto, não de arquivos de código-fonte) impede que seus módulos sejam utilizáveis por todas as implementações do Python ou novas versões do Python que alteram o formato do bytecode.

Adicionado na versão 3.3.

name

O nome do módulo que o carregador vai manipular.

path

O caminho para o arquivo de bytecode.

is_package(*fullname*)

Determina se o módulo é um pacote baseado em *path*.

get_code(*fullname*)

Retorna o objeto código para *name* criado a partir de *path*.

get_source(*fullname*)

Retorna `None`, pois os arquivos de bytecode não têm código-fonte quando este carregador é usado.

load_module(*name=None*)

Implementação concreta de *importlib.abc.Loader.load_module()* onde a especificação do nome do módulo para carregar é opcional

Deprecated since version 3.6, will be removed in version 3.15: Use *importlib.abc.Loader.exec_module()*.

class importlib.machinery.**ExtensionFileLoader**(*fullname*, *path*)

Uma implementação concreta de *importlib.abc.ExecutionLoader* para módulos de extensão.

O argumento *fullname* especifica o nome do módulo ao qual o carregador deve dar suporte. O argumento *path* é o caminho para o arquivo do módulo de extensão.

Observe que, por padrão, a importação de um módulo de extensão falhará em subinterpretadores se ele não implementar inicialização multifásica (consulte a **PEP 489**), mesmo que ele seja importado com sucesso.

Adicionado na versão 3.3.

Alterado na versão 3.12: A inicialização multifásica agora é necessária para uso em subinterpretadores.

name

Nome do módulo ao qual o carregador dá suporte.

path

Caminho para o módulo de extensão.

create_module(*spec*)

Cria o objeto do módulo a partir da especificação fornecida de acordo com a **PEP 489**.

Adicionado na versão 3.5.

exec_module(*module*)

Inicializa o objeto do módulo fornecido de acordo com a **PEP 489**.

Adicionado na versão 3.5.

is_package(*fullname*)

Returns `True` if the file path points to a package's `__init__` module based on *`EXTENSION_SUFFIXES`*.

get_code(*fullname*)

Retorna `None`, pois os módulos de extensão não possuem um objeto código.

get_source(*fullname*)

Retorna `None` pois os módulos de extensão não possuem código-fonte.

get_filename(*fullname*)

Retorna *`path`*.

Adicionado na versão 3.4.

class `importlib.machinery.`**NamespaceLoader**(*name*, *path*, *path_finder*)

Uma implementação concreta de *`importlib.abc.InspectLoader`* para pacotes de espaço de nomes. Este é um apelido para uma classe privada e só é tornado público para introspecção do atributo `__loader__` em pacotes de espaço de nomes:

```
>>> from importlib.machinery import NamespaceLoader
>>> import my_namespace
>>> isinstance(my_namespace.__loader__, NamespaceLoader)
True
>>> import importlib.abc
>>> isinstance(my_namespace.__loader__, importlib.abc.Loader)
True
```

Adicionado na versão 3.11.

class `importlib.machinery.`**ModuleSpec**(*name*, *loader*, ***, *origin=None*, *loader_state=None*, *is_package=None*)

A specification for a module's import-system-related state. This is typically exposed as the module's `__spec__` attribute. Many of these attributes are also available directly on a module: for example, `module.__spec__.origin == module.__file__`. Note, however, that while the *values* are usually equivalent, they can differ since there is no synchronization between the two objects. For example, it is possible to update the module's `__file__` at runtime and this will not be automatically reflected in the module's *`__spec__.origin`*, and vice versa.

Adicionado na versão 3.4.

name

The module's fully qualified name (see `module.__name__`). The *finder* should always set this attribute to a non-empty string.

loader

The *loader* used to load the module (see `module.__loader__`). The *finder* should always set this attribute.

origin

The location the *loader* should use to load the module (see `module.__file__`). For example, for modules loaded from a `.py` file this is the filename. The *finder* should always set this attribute to a meaningful value for the *loader* to use. In the uncommon case that there is not one (like for namespace packages), it should be set to `None`.

submodule_search_locations

A (possibly empty) *sequence* of strings enumerating the locations in which a package's submodules will be found (see `module.__path__`). Most of the time there will only be a single directory in this list.

The *finder* should set this attribute to a sequence, even an empty one, to indicate to the import system that the module is a package. It should be set to `None` for non-package modules. It is set automatically later to a special object for namespace packages.

loader_state

The *finder* may set this attribute to an object containing additional, module-specific data to use when loading the module. Otherwise it should be set to `None`.

cached

The filename of a compiled version of the module's code (see `module.__cached__`). The *finder* should always set this attribute but it may be `None` for modules that do not need compiled code stored.

parent

(Read-only) The fully qualified name of the package the module is in (or the empty string for a top-level module). See `module.__package__`. If the module is a package then this is the same as *name*.

has_location

`True` if the spec's *origin* refers to a loadable location, `False` otherwise. This value impacts how `origin` is interpreted and how the module's `__file__` is populated.

class importlib.machinery.**AppleFrameworkLoader**(*name*, *path*)

A specialization of *importlib.machinery.ExtensionFileLoader* that is able to load extension modules in Framework format.

For compatibility with the iOS App Store, *all* binary modules in an iOS app must be dynamic libraries, contained in a framework with appropriate metadata, stored in the `Frameworks` folder of the packaged app. There can be only a single binary per framework, and there can be no executable binary material outside the Frameworks folder.

To accommodate this requirement, when running on iOS, extension module binaries are *not* packaged as `.so` files on `sys.path`, but as individual standalone frameworks. To discover those frameworks, this loader is be registered against the `.fwork` file extension, with a `.fwork` file acting as a placeholder in the original location of the binary on `sys.path`. The `.fwork` file contains the path of the actual binary in the `Frameworks` folder, relative to the app bundle. To allow for resolving a framework-packaged binary back to the original location, the framework is expected to contain a `.origin` file that contains the location of the `.fwork` file, relative to the app bundle.

Por exemplo, considere o caso de uma importação `from foo.bar import _whiz`, onde `_whiz` é implementado com o módulo binário `sources/foo/bar/_whiz.abi3.so`, com `sources` sendo o local registrado em `sys.path`, relativo ao pacote da aplicação. Este módulo *deve* ser distribuído como `Frameworks/foo.bar._whiz.framework/foo.bar._whiz` (criando o nome do framework a partir do caminho completo de importação do módulo), com um arquivo `Info.plist` no diretório `.framework` identificando o binário como um framework. O módulo `foo.bar._whiz` seria representado no local original com um arquivo marcador `sources/foo/bar/_whiz.abi3.fwork`, contendo o caminho `Frameworks/foo.bar._whiz/foo.bar._whiz`. O framework também conteria `Frameworks/foo.bar._whiz.framework/foo.bar._whiz.origin`, contendo o caminho para o arquivo `.fwork`.

When a module is loaded with this loader, the `__file__` for the module will report as the location of the `.fwork` file. This allows code to use the `__file__` of a module as an anchor for file system traversal. However, the spec origin will reference the location of the *actual* binary in the `.framework` folder.

The Xcode project building the app is responsible for converting any `.so` files from wherever they exist in the `PYTHONPATH` into frameworks in the `Frameworks` folder (including stripping extensions from the module file, the addition of framework metadata, and signing the resulting framework), and creating the `.fwork` and `.origin` files. This will usually be done with a build step in the Xcode project; see the iOS documentation for details on how to construct this build step.

Adicionado na versão 3.13.

Disponibilidade: iOS.

name

Nome do módulo ao qual o carregador dá suporte.

path

Path to the `.fwork` file for the extension module.

31.5.5 `importlib.util` – Utility code for importers

Source code: Lib/importlib/util.py

This module contains the various objects that help in the construction of an *importer*.

`importlib.util.`**`MAGIC_NUMBER`**

The bytes which represent the bytecode version number. If you need help with loading/writing bytecode then consider *`importlib.abc.SourceLoader`*.

Adicionado na versão 3.4.

`importlib.util.`**`cache_from_source`**(*path*, *debug_override=None*, *, *optimization=None*)

Return the **PEP 3147/PEP 488** path to the byte-compiled file associated with the source *path*. For example, if *path* is `/foo/bar/baz.py` the return value would be `/foo/bar/__pycache__/baz.cpython-32.pyc` for Python 3.2. The `cpython-32` string comes from the current magic tag (see `get_tag()`; if `sys.implementation.cache_tag` is not defined then *`NotImplementedError`* will be raised).

The *optimization* parameter is used to specify the optimization level of the bytecode file. An empty string represents no optimization, so `/foo/bar/baz.py` with an *optimization* of `''` will result in a bytecode path of `/foo/bar/__pycache__/baz.cpython-32.pyc`. `None` causes the interpreter's optimization level to be used. Any other value's string representation is used, so `/foo/bar/baz.py` with an *optimization* of `2` will lead to the bytecode path of `/foo/bar/__pycache__/baz.cpython-32.opt-2.pyc`. The string representation of *optimization* can only be alphanumeric, else *`ValueError`* is raised.

The *debug_override* parameter is deprecated and can be used to override the system's value for `__debug__`. A `True` value is the equivalent of setting *optimization* to the empty string. A `False` value is the same as setting *optimization* to `1`. If both *debug_override* an *optimization* are not `None` then *`TypeError`* is raised.

Adicionado na versão 3.4.

Alterado na versão 3.5: The *optimization* parameter was added and the *debug_override* parameter was deprecated.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`importlib.util.`**`source_from_cache`**(*path*)

Given the *path* to a **PEP 3147** file name, return the associated source code file path. For example, if *path* is `/foo/bar/__pycache__/baz.cpython-32.pyc` the returned path would be `/foo/bar/baz.py`. *path* need not exist, however if it does not conform to **PEP 3147** or **PEP 488** format, a *`ValueError`* is raised. If `sys.implementation.cache_tag` is not defined, *`NotImplementedError`* is raised.

Adicionado na versão 3.4.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`importlib.util.`**`decode_source`**(*source_bytes*)

Decode the given bytes representing source code and return it as a string with universal newlines (as required by *`importlib.abc.InspectLoader.get_source()`*).

Adicionado na versão 3.4.

`importlib.util.`**`resolve_name`**(*name*, *package*)

Resolve a relative module name to an absolute one.

If **name** has no leading dots, then **name** is simply returned. This allows for usage such as `importlib.util.resolve_name('sys', __spec__.parent)` without doing a check to see if the **package** argument is needed.

ImportError is raised if **name** is a relative module name but **package** is a false value (e.g. `None` or the empty string). *ImportError* is also raised if a relative name would escape its containing package (e.g. requesting `..bacon` from within the `spam` package).

Adicionado na versão 3.3.

Alterado na versão 3.9: To improve consistency with import statements, raise *ImportError* instead of *ValueError* for invalid relative import attempts.

`importlib.util.`**`find_spec`**(*name*, *package=None*)

Localiza o *spec* para um módulo, opcionalmente relativo ao nome do **package** especificado. Se o módulo estiver em *sys.modules*, então `sys.modules[name].__spec__` será retornado (a menos que o spec seja `None` ou não esteja definido, nesse caso *ValueError* é levantada). Caso contrário, uma pesquisa usando *sys.meta_path* é feita. `None` é retornado se nenhum spec for encontrada.

If **name** is for a submodule (contains a dot), the parent module is automatically imported.

name and **package** work the same as for `import_module()`.

Adicionado na versão 3.4.

Alterado na versão 3.7: Raises *ModuleNotFoundError* instead of *AttributeError* if **package** is in fact not a package (i.e. lacks a `__path__` attribute).

`importlib.util.`**`module_from_spec`**(*spec*)

Create a new module based on **spec** and *spec.loader.create_module*.

If *spec.loader.create_module* does not return `None`, then any pre-existing attributes will not be reset. Also, no *AttributeError* will be raised if triggered while accessing **spec** or setting an attribute on the module.

This function is preferred over using *types.ModuleType* to create a new module as **spec** is used to set as many import-controlled attributes on the module as possible.

Adicionado na versão 3.5.

`importlib.util.`**`spec_from_loader`**(*name*, *loader*, ***, *origin=None*, *is_package=None*)

A factory function for creating a *ModuleSpec* instance based on a loader. The parameters have the same meaning as they do for ModuleSpec. The function uses available *loader* APIs, such as `InspectLoader.is_package()`, to fill in any missing information on the spec.

Adicionado na versão 3.4.

`importlib.util.`**`spec_from_file_location`**(*name*, *location*, ***, *loader=None*, *submodule_search_locations=None*)

A factory function for creating a *ModuleSpec* instance based on the path to a file. Missing information will be filled in on the spec by making use of loader APIs and by the implication that the module will be file-based.

Adicionado na versão 3.4.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

`importlib.util.`**`source_hash`**(*source_bytes*)

Return the hash of *source_bytes* as bytes. A hash-based `.pyc` file embeds the *source_hash()* of the corresponding source file's contents in its header.

Adicionado na versão 3.7.

`importlib.util.`**`_incompatible_extension_module_restrictions`**(***, *disable_check*)

A context manager that can temporarily skip the compatibility check for extension modules. By default the check is enabled and will fail when a single-phase init module is imported in a subinterpreter. It will also fail for a multi-phase init module that doesn't explicitly support a per-interpreter GIL, when imported in an interpreter with its own GIL.

Note that this function is meant to accommodate an unusual case; one which is likely to eventually go away. There's is a pretty good chance this is not what you were looking for.

You can get the same effect as this function by implementing the basic interface of multi-phase init (**PEP 489**) and lying about support for multiple interpreters (or per-interpreter GIL).

> **Aviso**
>
> Using this function to disable the check can lead to unexpected behavior and even crashes. It should only be used during extension module development.

Adicionado na versão 3.12.

class importlib.util.**LazyLoader**(*loader*)

A class which postpones the execution of the loader of a module until the module has an attribute accessed.

This class **only** works with loaders that define *exec_module()* as control over what module type is used for the module is required. For those same reasons, the loader's *create_module()* method must return `None` or a type for which its `__class__` attribute can be mutated along with not using *slots*. Finally, modules which substitute the object placed into *sys.modules* will not work as there is no way to properly replace the module references throughout the interpreter safely; *ValueError* is raised if such a substitution is detected.

> **Nota**
>
> For projects where startup time is critical, this class allows for potentially minimizing the cost of loading a module if it is never used. For projects where startup time is not essential then use of this class is **heavily** discouraged due to error messages created during loading being postponed and thus occurring out of context.

Adicionado na versão 3.5.

Alterado na versão 3.6: Began calling *create_module()*, removing the compatibility warning for *importlib.machinery.BuiltinImporter* and *importlib.machinery.ExtensionFileLoader*.

classmethod factory(*loader*)

A class method which returns a callable that creates a lazy loader. This is meant to be used in situations where the loader is passed by class instead of by instance.

```
suffixes = importlib.machinery.SOURCE_SUFFIXES
loader = importlib.machinery.SourceFileLoader
lazy_loader = importlib.util.LazyLoader.factory(loader)
finder = importlib.machinery.FileFinder(path, (lazy_loader, suffixes))
```

31.5.6 Exemplos

Importing programmatically

To programmatically import a module, use *importlib.import_module()*.

```
import importlib

itertools = importlib.import_module('itertools')
```

Checando se o módulo pode ser importado

If you need to find out if a module can be imported without actually doing the import, then you should use *importlib.util.find_spec()*.

Note that if `name` is a submodule (contains a dot), *importlib.util.find_spec()* will import the parent module.

```
import importlib.util
import sys

# For illustrative purposes.
name = 'itertools'

if name in sys.modules:
    print(f"{name!r} already in sys.modules")
elif (spec := importlib.util.find_spec(name)) is not None:
    # If you chose to perform the actual import ...
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    print(f"{name!r} has been imported")
else:
    print(f"can't find the {name!r} module")
```

Importa o arquivo de origem diretamente

This recipe should be used with caution: it is an approximation of an import statement where the file path is specified directly, rather than *sys.path* being searched. Alternatives should first be considered first, such as modifying *sys.path* when a proper module is required, or using *runpy.run_path()* when the global namespace resulting from running a Python file is appropriate.

To import a Python source file directly from a path, use the following recipe:

```
import importlib.util
import sys


def import_from_path(module_name, file_path):
    spec = importlib.util.spec_from_file_location(module_name, file_path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[module_name] = module
    spec.loader.exec_module(module)
    return module


# For illustrative purposes only (use of `json` is arbitrary).
import json
file_path = json.__file__
module_name = json.__name__

# Similar outcome as `import json`.
json = import_from_path(module_name, file_path)
```

Implementing lazy imports

The example below shows how to implement lazy imports:

```
>>> import importlib.util
>>> import sys
>>> def lazy_import(name):
...     spec = importlib.util.find_spec(name)
...     loader = importlib.util.LazyLoader(spec.loader)
...     spec.loader = loader
...     module = importlib.util.module_from_spec(spec)
```

(continua na próxima página)

(continuação da página anterior)

```
...     sys.modules[name] = module
...     loader.exec_module(module)
...     return module
...
>>> lazy_typing = lazy_import("typing")
>>> #lazy_typing is a real module object,
>>> #but it is not loaded in memory yet.
>>> lazy_typing.TYPE_CHECKING
False
```

Setting up an importer

For deep customizations of import, you typically want to implement an *importer*. This means managing both the *finder* and *loader* side of things. For finders there are two flavours to choose from depending on your needs: a *meta path finder* or a *path entry finder*. The former is what you would put on `sys.meta_path` while the latter is what you create using a *path entry hook* on `sys.path_hooks` which works with `sys.path` entries to potentially create a finder. This example will show you how to register your own importers so that import will use them (for creating an importer for yourself, read the documentation for the appropriate classes defined within this package):

```
import importlib.machinery
import sys

# For illustrative purposes only.
SpamMetaPathFinder = importlib.machinery.PathFinder
SpamPathEntryFinder = importlib.machinery.FileFinder
loader_details = (importlib.machinery.SourceFileLoader,
                  importlib.machinery.SOURCE_SUFFIXES)

# Setting up a meta path finder.
# Make sure to put the finder in the proper location in the list in terms of
# priority.
sys.meta_path.append(SpamMetaPathFinder)

# Setting up a path entry finder.
# Make sure to put the path hook in the proper location in the list in terms
# of priority.
sys.path_hooks.append(SpamPathEntryFinder.path_hook(loader_details))
```

Approximating `importlib.import_module()`

Import itself is implemented in Python code, making it possible to expose most of the import machinery through importlib. The following helps illustrate the various APIs that importlib exposes by providing an approximate implementation of `importlib.import_module()`:

```
import importlib.util
import sys

def import_module(name, package=None):
    """An approximate implementation of import."""
    absolute_name = importlib.util.resolve_name(name, package)
    try:
        return sys.modules[absolute_name]
    except KeyError:
        pass

    path = None
```

(continua na próxima página)

(continuação da página anterior)

```
    if '.' in absolute_name:
        parent_name, _, child_name = absolute_name.rpartition('.')
        parent_module = import_module(parent_name)
        path = parent_module.__spec__.submodule_search_locations
    for finder in sys.meta_path:
        spec = finder.find_spec(absolute_name, path)
        if spec is not None:
            break
    else:
        msg = f'No module named {absolute_name!r}'
        raise ModuleNotFoundError(msg, name=absolute_name)
    module = importlib.util.module_from_spec(spec)
    sys.modules[absolute_name] = module
    spec.loader.exec_module(module)
    if path is not None:
        setattr(parent_module, child_name, module)
    return module
```

31.6 `importlib.resources` – Leitura, abertura e acesso a recursos de pacotes

Código-fonte: Lib/importlib/resources/__init__.py

Adicionado na versão 3.7.

Esse módulo aproveita o sistema de importação do Python para fornecer acesso a *recursos* dentro de *pacotes*.

"Recursos" são recursos de arquivos ou similares associados a um módulo ou pacote em Python. Os recursos podem estar contidos diretamente em um pacote, em um subdiretório contido nesse pacote ou adjacentes a módulos fora de um pacote. Os recursos podem ser de texto ou binários. Como resultado, os códigos-fonte do módulo Python (.py) de um pacote e os artefatos de compilação (pycache) são tecnicamente recursos de fato desse pacote. Na prática, entretanto, os recursos são principalmente os artefatos não-Python expostos especificamente pelo autor do pacote.

Os recursos podem ser abertos ou lidos no modo binário ou de texto.

Os recursos são mais ou menos semelhantes a arquivos dentro de diretórios, embora seja importante ter em mente que isso é apenas uma metáfora. Recursos e pacotes **não** precisam existir como arquivos e diretórios físicos no sistema de arquivos: por exemplo, um pacote e seus recursos podem ser importados de um arquivo zip usando *zipimport*.

> **Nota**
>
> Esse módulo fornece funcionalidade semelhante ao Basic Resource Access do pkg_resources sem a sobrecarga de desempenho desse pacote. Isso facilita a leitura de recursos incluídos em pacotes, com uma semântica mais estável e consistente.
>
> O backport autônomo desse módulo fornece mais informações sobre uso do importlib.resources e migração do pkg_resources para o importlib.resources.

Loaders que desejam oferecer suporte à leitura de recursos devem implementar um método `get_resource_reader(fullname)` conforme especificado por *importlib.resources.abc.ResourceReader*.

class importlib.resources.**Anchor**

Representa uma âncora para recursos, seja um *objeto módulo* ou um nome de módulo como uma string. Definido como `Union[str, ModuleType]`.

importlib.resources.**files**(*anchor:* Anchor | None = *None*)

Retorna um objeto *Traversable* que representa o contêiner de recursos (pense em diretório) e seus recursos (pense em arquivos). Um Traversable pode conter outros contêineres (pense em subdiretórios).

anchor é um *Anchor* opcional. Se a âncora for um pacote, os recursos são resolvidos a partir desse pacote. Se for um módulo, os recursos são resolvidos ao lado desse módulo (no mesmo pacote ou na raiz do pacote). Se a âncora for omitida, o módulo do chamador é usado.

Adicionado na versão 3.9.

Alterado na versão 3.12: O parâmetro *package* foi renomeado para *anchor*. *anchor* agora pode ser um módulo sem ser pacote e, se omitido, será o módulo do chamador por padrão. *package* ainda é aceito por compatibilidade, mas irá levantar um *DeprecationWarning*. Considere passar a posição da âncora de forma posicional ou usar `importlib_resources >= 5.10` para uma interface compatível em versões mais antigas do Python.

importlib.resources.**as_file**(*traversable*)

Dado um objeto *Traversable* que representa um arquivo ou diretório, normalmente de *importlib.resources.files()*, retorna um gerenciador de contexto para uso em uma instrução `with`. O gerenciador de contexto fornece um objeto *pathlib.Path*.

Sair do gerenciador de contexto limpa qualquer arquivo ou diretório temporário criado quando o recurso foi extraído, por exemplo, de um arquivo zip.

Use `as_file` quando os métodos de Traversable (`read_text`, etc.) forem insuficientes e for necessário um arquivo ou diretório real no sistema de arquivos.

Adicionado na versão 3.9.

Alterado na versão 3.12: Adicionado suporte para *traversable* representando um diretório.

31.6.1 API funcional

Um conjunto de auxiliares simplificados e compatíveis com versões anteriores está disponível. Estes permitem operações comuns em uma única chamada de função.

Para todas as funções a seguir:

- *anchor* é um *Anchor*, assim como em *files()*. Ao contrário do comportamento em `files`, *anchor* aqui não pode ser omitido.
- *path_names* são componentes do nome do caminho de um recurso, relativo à âncora. Por exemplo, para obter o texto de um recurso chamado `info.txt`, use:

```
importlib.resources.read_text(my_module, "info.txt")
```

Assim como em *Traversable.joinpath*, os componentes individuais devem usar barras ordinárias (`/`) como separadores do caminho. Por exemplo, as seguintes chamadas são equivalentes:

```
importlib.resources.read_binary(my_module, "pics/painting.png")
importlib.resources.read_binary(my_module, "pics", "painting.png")
```

Por motivos de compatibilidade com versões anteriores, funções que leem texto requerem um argumento *encoding* explícito se múltiplos *path_names* forem fornecidos. Por exemplo, para obter o texto de `info/chapter1.txt`, use:

```
importlib.resources.read_text(my_module, "info", "chapter1.txt",
                              encoding='utf-8')
```

importlib.resources.**open_binary**(*anchor*, **path_names*)

Abre o recurso nomeado para leitura binária.

Veja *a introdução* para detalhes sobre *anchor* e *path_names*.

Esta função retorna um objeto *BinaryIO*, isto é, um fluxo binário aberto para leitura.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).open('rb')
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos.

importlib.resources.**open_text**(*anchor*, **path_names*, *encoding='utf-8'*, *errors='strict'*)

Abre o recurso nomeado para leitura de texto. Por padrão, o conteúdo é lido como UTF-8 estrito.

Veja *a introdução* para detalhes sobre *anchor* e *path_names*. *encoding* e *errors* têm o mesmo significado que na embutida `open()`.

Por motivos de compatibilidade com versões anteriores, o argumento *encoding* precisa ser passado explicitamente se houver múltiplos *path_names*. Essa limitação está agendada para ser removida no Python 3.15.

Esta função retorna um objeto `TextIO`, isto é, um fluxo de texto aberto para leitura.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).open('r', encoding=encoding)
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos. *encoding* e *errors* precisam ser fornecidos como argumentos nomeados.

importlib.resources.**read_binary**(*anchor*, **path_names*)

Lê e retorna o conteúdo do recurso nomeado como `bytes`.

Veja *a introdução* para detalhes sobre *anchor* e *path_names*.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).read_bytes()
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos.

importlib.resources.**read_text**(*anchor*, **path_names*, *encoding='utf-8'*, *errors='strict'*)

Lê e retorna o conteúdo do recurso nomeado como `str`. Por padrão, o conteúdo é lido como UTF-8 estrito.

Veja *a introdução* para detalhes sobre *anchor* e *path_names*. *encoding* e *errors* têm o mesmo significado que na embutida `open()`.

Por motivos de compatibilidade com versões anteriores, o argumento *encoding* precisa ser passado explicitamente se houver múltiplos *path_names*. Essa limitação está agendada para ser removida no Python 3.15.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).read_text(encoding=encoding)
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos. *encoding* e *errors* precisam ser fornecidos como argumentos nomeados.

importlib.resources.**path**(*anchor*, **path_names*)

Fornece o caminho para o *resource* como um caminho real do sistema de arquivos. Essa função retorna um gerenciador de contexto para uso em uma instrução `with`. O gerenciador de contexto fornece um objeto `pathlib.Path`.

Sair do gerenciador de contexto limpa qualquer arquivo temporário criado, por exemplo quando o recurso precisa ser extraído de um arquivo zip.

Por exemplo, o método `stat()` requer um caminho real em um sistema de arquivos; ele pode ser usado assim:

```
with importlib.resources.path(anchor, "resource.txt") as fspath:
    result = fspath.stat()
```

Veja *a introdução* para detalhes sobre *anchor* e *path_names*.

Esta função é mais ou menos equivalente a:

```
as_file(files(anchor).joinpath(*path_names))
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos. *encoding* e *errors* precisam ser fornecidos como argumentos nomeados.

importlib.resources.**is_resource**(*anchor*, **path_names*)

Retorna `True` se o recurso nomeado existe, senão `False`. Esta função não considera diretórios como sendo recursos.

Veja *a introdução* para detalhes sobre *anchor* e *path_names*.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).is_file()
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos.

importlib.resources.**contents**(*anchor*, **path_names*)

Retorna um iterável sobre os itens nomeados no pacote ou caminho. O iterável retorna nomes de recursos (por exemplo, arquivos) e outros itens que não recursos (por exemplo, diretórios) como *str*. O iterável não recorre em subdiretórios.

Veja *a introdução* para detalhes sobre *anchor* e *path_names*.

Esta função é mais ou menos equivalente a:

```
for resource in files(anchor).joinpath(*path_names).iterdir():
    yield resource.name
```

Descontinuado desde a versão 3.11: Prefira `iterdir()` como acima, que oferece mais controle sobre os resultados e funcionalidade mais rica.

31.7 importlib.resources.abc – Classes base abstratas para recursos

Código-fonte: Lib/importlib/resources/abc.py

Adicionado na versão 3.11.

class importlib.resources.abc.**ResourceReader**

Substituída por TraversableResources

Uma *classe base abstrata* para fornecer a capacidade de ler *recursos*.

Da perspectiva deste ABC, um *recurso* é um artefato binário que é enviado dentro de um pacote. Normalmente isso é algo como um arquivo de dados que fica próximo ao arquivo `__init__.py` do pacote. O objetivo desta classe é ajudar a abstrair o acesso a tais arquivos de dados para que não importe se o pacote e seu(s) arquivo(s) de dados estão armazenados, por exemplo, em um arquivo zip versus no sistema de arquivos.

Para qualquer um dos métodos desta classe, espera-se que o argumento *resource* seja um *objeto caminho ou similar* que representa conceitualmente apenas um nome de arquivo. Isso significa que nenhum caminho de subdiretório deve ser incluído no argumento *resource*. Isso ocorre porque a localização do pacote para o qual o leitor se destina, atua como o "diretório". Portanto, a metáfora para diretórios e nomes de arquivos são pacotes e recursos, respectivamente. É também por isso que se espera que as instâncias dessa classe se correlacionem diretamente a um pacote específico (em vez de representar potencialmente vários pacotes ou um módulo).

Carregadores que desejam oferecer suporte à leitura de recursos devem fornecer um método chamado `get_resource_reader(nomecompleto)` que retorna um objeto implementando esta interface ABC. Se

o módulo especificado por nomecompleto não for um pacote, este método deve retornar *None*. Um objeto compatível com este ABC só deve ser retornado quando o módulo especificado for um pacote.

Descontinuado desde a versão 3.12: Use *importlib.resources.abc.TraversableResources*.

abstractmethod open_resource(*resource*)

Retorna um *objeto arquivo ou similar* aberto para leitura binária de *resource*.

Se o recurso não puder ser encontrado, *FileNotFoundError* é levantada.

abstractmethod resource_path(*resource*)

Retorna o caminho do sistema de arquivos para *resource*.

Se o recurso não existir concretamente no sistema de arquivos, levanta *FileNotFoundError*.

abstractmethod is_resource(*name*)

Retorna `True` se o *name* nomeado for considerado um recurso. *FileNotFoundError* é levantada se *name* não existir.

abstractmethod contents()

Retorna um *iterável* de strings sobre o conteúdo do pacote. Observe que não é necessário que todos os nomes retornados pelo iterador sejam recursos reais, por exemplo, é aceitável retornar nomes para os quais *is_resource()* seria falso.

Permitir que nomes que não são recursos sejam retornados é permitir situações em que a forma como um pacote e seus recursos são armazenados é conhecida a priori e os nomes que não são recursos seriam úteis. Por exemplo, o retorno de nomes de subdiretórios é permitido para que, quando se souber que o pacote e os recursos estão armazenados no sistema de arquivos, esses nomes de subdiretórios possam ser usados diretamente.

O método abstrato retorna um iterável sem itens.

class importlib.resources.abc.**Traversable**

Um objeto com um subconjunto de métodos de *pathlib.Path* adequados para percorrer diretórios e abrir arquivos.

Para uma representação do objeto no sistema de arquivos, use *importlib.resources.as_file()*.

name

Abstrato. O nome base deste objeto sem nenhuma referência pai.

abstractmethod iterdir()

Produz objetos Traversable em self.

abstractmethod is_dir()

Retorna `True` se self for um diretório.

abstractmethod is_file()

Retorna `True` se self for um arquivo.

abstractmethod joinpath(**pathsegments*)

Percorre os diretórios de acordo com *pathsegments* e retorna o resultado como `Traversable`.

Cada argumento *pathsegments* pode conter vários nomes separados por barras (`/`, `posixpath.sep`). Por exemplo, os seguintes são equivalentes:

```
files.joinpath('subdir', 'subsuddir', 'arquivo.txt')
files.joinpath('subdir/subsuddir/arquivo.txt')
```

Observe que algumas implementações de `Traversable` podem não ser atualizadas para a versão mais recente do protocolo. Para compatibilidade com tais implementações, forneça um único argumento sem separadores de caminho para cada chamada para `joinpath`. Por exemplo:

```
files.joinpath('subdir').joinpath('subsubdir').joinpath('arquivo.txt')
```

Alterado na versão 3.11: `joinpath` aceita múltiplos *pathsegments*, e estes segmentos podem conter barras como separadores de caminho. Anteriormente, apenas um único argumento *child* era aceito.

abstractmethod **__truediv__**(*child*)

Retorna um filho Traversable em si mesmo. Equivalente a `joinpath(child)`.

abstractmethod **open**(*mode='r'*, **args*, ***kwargs*)

mode pode ser 'r' ou 'rb' para abrir como texto ou binário. Retorna um manipulador adequado para leitura (o mesmo que *pathlib.Path.open*).

Ao abrir como texto, aceita parâmetros de codificação como os aceitos por *io.TextIOWrapper*.

read_bytes()

Lê o conteúdo de self como bytes.

read_text(*encoding=None*)

Lê o conteúdo de self como texto.

class importlib.resources.abc.**TraversableResources**

Uma classe base abstrata para leitores de recursos capaz de servir a interface *importlib.resources.files()*. É uma subclasse de *ResourceReader* e fornece implementações concretas dos métodos abstratos de `ResourceReader`. Portanto, qualquer carregador que forneça `TraversableResources` também fornece `ResourceReader`.

Espera-se que os carregadores que desejam oferecer suporte à leitura de recursos implementem essa interface.

abstractmethod **files**()

Retorna um objeto *importlib.resources.abc.Traversable* para o pacote carregado.

31.8 `importlib.metadata` – Acessando metadados do pacote

Adicionado na versão 3.8.

Alterado na versão 3.10: `importlib.metadata` não é mais provisório.

Código-fonte: Lib/importlib/metadata/__init__.py

`importlib.metadata` é uma biblioteca que fornece acesso aos metadados de um Pacote de Distribuição instalado, como seus pontos de entrada ou seus nomes de nível superior (Pacotes de Importação, módulos, se houver). Construída em parte no sistema de importação do Python, esta biblioteca pretende substituir funcionalidades semelhantes na API de ponto de entrada e API de metadados de `pkg_resources`. Junto com *importlib.resources*, este pacote pode eliminar a necessidade de usar o pacote `pkg_resources` mais antigo e menos eficiente.

`importlib.metadata` opera em *pacotes de distribuição* de terceiros instalados no diretório `site-packages` do Python através de ferramentas como pip. Especificamente, ele funciona com distribuições com diretórios `dist-info` ou `egg-info` detectáveis, e metadados definidos pelas Especificações de metadados principais.

Importante

Eles *não* são necessariamente equivalentes ou correspondem 1:1 aos nomes de *pacotes de importação* de nível superior que podem ser importados dentro do código Python. Um *pacote de distribuição* pode conter vários *pacotes de importação* (e módulos únicos), e um *pacote de importação* de nível superior pode ser mapeado para vários *pacotes de distribuição* se for um pacote de espaço de nomes. Você pode usar *packages_distributions()* para obter um mapeamento entre eles.

Por padrão, os metadados de distribuição podem residir no sistema de arquivos ou em arquivos zip em *sys.path*. Através de um mecanismo de extensão, os metadados podem residir em praticamente qualquer lugar.

Ver também

https://importlib-metadata.readthedocs.io/
A documentação para `importlib_metadata`, que fornece um backport de `importlib.metadata`. Isso inclui uma referência de API para as classes e funções deste módulo, bem como um guia de migração para usuários existentes de `pkg_resources`.

31.8.1 Visão Geral

Digamos que você queira obter a string da versão de um Pacote de Distribuição que você instalou usando `pip`. Começamos criando um ambiente virtual e instalando algo nele:

```
$ python -m venv exemplo
$ source exemplo/bin/activate
(exemplo) $ python -m pip install wheel
```

Você pode obter a string de versão para `wheel` executando o seguinte:

```
(example) $ python
>>> from importlib.metadata import version
>>> version('wheel')
'0.32.3'
```

Você também pode obter uma coleção de pontos de entrada selecionáveis pelas propriedades do EntryPoint (normalmente 'group' ou 'name'), como `console_scripts`, `distutils.commands` e outros. Cada grupo contém uma coleção de objetos *EntryPoint*.

Você pode obter os *metadados para uma distribuição*:

```
>>> list(metadata('wheel'))
['Metadata-Version', 'Name', 'Version', 'Summary', 'Home-page', 'Author', 'Author-
↪email', 'Maintainer', 'Maintainer-email', 'License', 'Project-URL', 'Project-URL
↪', 'Project-URL', 'Keywords', 'Platform', 'Classifier', 'Classifier', 'Classifier
↪', 'Classifier', 'Classifier', 'Classifier', 'Classifier', 'Classifier',
↪'Classifier', 'Classifier', 'Classifier', 'Classifier', 'Requires-Python',
↪'Provides-Extra', 'Requires-Dist', 'Requires-Dist']
```

Você também pode obter uma *número da versão da distribuição*, listar seus *arquivos constituintes* e obter uma lista dos *Requisitos de distribuição* da distribuição.

exception `importlib.metadata.`**PackageNotFoundError**

Subclasse de *`ModuleNotFoundError`* levantada por diversas funções neste módulo quando consultada sobre um pacote de distribuição que não está instalado no ambiente Python atual.

31.8.2 API funcional

Este pacote fornece a seguinte funcionalidade por meio de sua API pública.

Pontos de entrada

`importlib.metadata.`**entry_points**(***select_params*)

Retorna uma instância de *`EntryPoints`* descrevendo pontos de entrada para o ambiente atual. Quaisquer parâmetros nomeados fornecidos são passados para o método `select()` para comparação com os atributos das definições de pontos de entrada individuais.

Observação: atualmente não é possível consultar pontos de entrada com base no atributo `EntryPoint.dist` (já que instâncias diferentes de `Distribution` não são comparadas como iguais, mesmo que tenham os mesmos atributos)

class importlib.metadata.**EntryPoints**

Detalhes de uma coleção de pontos de entrada instalados.

Também fornece um atributo `.groups` que relata todos os grupos de pontos de entrada identificados e um atributo `.names` que relata todos os nomes de pontos de entrada identificados.

class importlib.metadata.**EntryPoint**

Detalhes de um ponto de entrada instalado.

Cada instância de `EntryPoint` tem os atributos `.name`, `.group` e `.value` e um método `.load()` para resolver o valor. Há também os atributos `.module`, `.attr` e `.extras` para obter os componentes do atributo `.value` e `.dist` para obter informações sobre o pacote de distribuição que fornece o ponto de entrada.

Consultar todos os pontos de entrada:

```
>>> eps = entry_points()
```

A função `entry_points()` retorna um objeto `EntryPoints`, uma coleção de todos os objetos `EntryPoint` com atributos `names` e `groups` por conveniência:

```
>>> sorted(eps.groups)
['console_scripts', 'distutils.commands', 'distutils.setup_keywords', 'egg_info.
↪writers', 'setuptools.installation']
```

`EntryPoints` possui um método `select()` para selecionar pontos de entrada correspondendo a propriedades específicas. Selecione pontos de entrada no grupo `console_scripts`:

```
>>> scripts = eps.select(group='console_scripts')
```

Equivalentemente, já que `entry_points()` passa argumentos nomeados para a seleção:

```
>>> scripts = entry_points(group='console_scripts')
```

Escolha um script específico chamado “wheel” (encontrado no projeto wheel):

```
>>> 'wheel' in scripts.names
True
>>> wheel = scripts['wheel']
```

De forma equivalente, consulte esse ponto de entrada durante a seleção:

```
>>> (wheel,) = entry_points(group='console_scripts', name='wheel')
>>> (wheel,) = entry_points().select(group='console_scripts', name='wheel')
```

Inspecione o ponto de entrada resolvido:

```
>>> wheel
EntryPoint(name='wheel', value='wheel.cli:main', group='console_scripts')
>>> wheel.module
'wheel.cli'
>>> wheel.attr
'main'
>>> wheel.extras
[]
>>> main = wheel.load()
>>> main
<function main at 0x103528488>
```

O `group` e `name` são valores arbitrários definidos pelo autor do pacote e normalmente um cliente desejará resolver todos os pontos de entrada para um grupo específico. Leia a documentação do setuptools para obter mais informações sobre pontos de entrada, sua definição e uso.

Alterado na versão 3.12: Os pontos de entrada "selecionáveis" foram introduzidos em `importlib_metadata` 3.6 e Python 3.10. Antes dessas mudanças, `entry_points` não aceitava parâmetros e sempre retornava um dicionário de pontos de entrada, chaveado por grupo. Com `importlib_metadata` 5.0 e Python 3.12, `entry_points` sempre retorna um objeto `EntryPoints`. Veja backports.entry_points_selectable para opções de compatibilidade.

Alterado na versão 3.13: Objetos `EntryPoint` não apresentam mais uma interface tupla ou similar (`__getitem__()`).

Metadados de distribuição

`importlib.metadata.`**`metadata`**(*distribution_name*)

Retorna os metadados de distribuição correspondentes ao pacote de distribuição nomeado como uma instância de *`PackageMetadata`*.

Levanta *`PackageNotFoundError`* se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

`class` `importlib.metadata.`**`PackageMetadata`**

Uma implementação concreta do protocolo PackageMetadata.

Além de fornecer os métodos e atributos de protocolo definidos, subscrever a instância é equivalente a chamar o método `get()`.

Cada Pacote de Distribuição inclui alguns metadados, que você pode extrair usando a função `metadata()`:

```
>>> wheel_metadata = metadata('wheel')
```

As chaves da estrutura de dados retornada nomeiam as palavras reservadas dos metadados, e os valores são retornados sem análise dos metadados de distribuição:

```
>>> wheel_metadata['Requires-Python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
```

`PackageMetadata` também apresenta um atributo `json` que retorna todos os metadados em um formato compatível com JSON pela **PEP 566**:

```
>>> wheel_metadata.json['requires_python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
```

O conjunto completo de metadados disponíveis não é descrito aqui. Consulte a especificação de metadados principais do PyPA para obter detalhes adicionais.

Alterado na versão 3.10: A `Description` agora é incluída nos metadados quando apresentada através do payload. Os caracteres de continuação de linha foram removidos.

O atributo `json` foi adicionado.

Versões de distribuição

`importlib.metadata.`**`version`**(*distribution_name*)

Retorna a versão do pacote de distribuição instalado para o pacote de distribuição nomeado.

Levanta *`PackageNotFoundError`* se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

A função `version()` é a maneira mais rápida de obter o número de versão de um Pacote de Distribuição, como uma string:

```
>>> version('wheel')
'0.32.3'
```

Arquivos de distribuição

importlib.metadata.**files**(*distribution_name*)

Retorna o conjunto completo de arquivos contidos no pacote de distribuição nomeado.

Levanta *PackageNotFoundError* se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

Retorna *None* se a distribuição for encontrada, mas os registros do banco de dados de instalação que relatam os arquivos associados ao pacote de distribuição estiverem ausentes.

class importlib.metadata.**PackagePath**

Um objeto derivado de *pathlib.PurePath* com propriedades adicionais `dist`, `size` e `hash` correspondentes aos metadados de instalação do pacote de distribuição para esse arquivo.

A função `files()` pega um nome Pacote de Distribuição e retorna todos os arquivos instalados por este distribuição. Cada arquivo é é relatado como uma instância de *PackagePath*. Por exemplo:

```
>>> util = [p for p in files('wheel') if 'util.py' in str(p)][0]
>>> util
PackagePath('wheel/util.py')
>>> util.size
859
>>> util.dist
<importlib.metadata._hooks.PathDistribution object at 0x101e0cef0>
>>> util.hash
<FileHash mode: sha256 value: bYkw5oMccfazVCoYQwKkkemoVyMAFoR34mmKBx8R1NI>
```

Uma vez que tenha o arquivo, você também pode ler seu conteúdo:

```
>>> print(util.read_text())
import base64
import sys
...
def as_bytes(s):
    if isinstance(s, text_type):
        return s.encode('utf-8')
    return s
```

Você também pode usar o método `locate()` para obter o caminho absoluto para o arquivo:

```
>>> util.locate()
PosixPath('/home/gustav/example/lib/site-packages/wheel/util.py')
```

No caso em que o arquivo de metadados que lista os arquivos (`RECORD` ou `SOURCES.txt`) estiver faltando, `files()` retornará *None*. O chamador pode querer agrupar chamadas para `files()` em always_iterable ou de outra forma se proteger contra isso condição se a distribuição de destino não for conhecida por ter os metadados presentes.

Requisitos de distribuição

importlib.metadata.**requires**(*distribution_name*)

Retorna os especificadores de dependência declarados para o pacote de distribuição nomeado.

Levanta *PackageNotFoundError* se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

Para obter o conjunto completo de requisitos para um Pacote de Distribuição, use a função `requires()`:

```
>>> requires('wheel')
["pytest (>=3.0.0) ; extra == 'test'", "pytest-cov ; extra == 'test'"]
```

Mapeando importação pra pacotes de distribuição

`importlib.metadata.`**`packages_distributions`**`()`

Retorna um mapeamento do módulo de nível superior e importa os nomes dos pacotes encontrados via *`sys.meta_path`* para os nomes dos pacotes de distribuição (se houver) que fornecem os arquivos correspondentes.

Para permitir pacotes de espaço de nomes (que podem ter membros fornecidos por vários pacotes de distribuição), cada nome de importação de nível superior é mapeado para uma lista de nomes de distribuição em vez de mapear diretamente para um único nome.

Um método conveniente para resolver o nome do Pacote de Distribuição (ou nomes, no caso de um pacote de espaço de nomes) que fornece cada módulo Python de nível superior importável ou Pacote de Importação:

```
>>> packages_distributions()
{'importlib_metadata': ['importlib-metadata'], 'yaml': ['PyYAML'], 'jaraco': [
↪'jaraco.classes', 'jaraco.functools'], ...}
```

Algumas instalações editáveis não fornecem nomes de nível superior e, portanto, esta função não é confiável com tais instalações.

Adicionado na versão 3.10.

31.8.3 Distribuições

`importlib.metadata.`**`distribution`**`(`*distribution_name*`)`

Retorna uma instância de *`Distribution`* descrevendo o pacote de distribuição nomeado.

Levanta *`PackageNotFoundError`* se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

`class` `importlib.metadata.`**`Distribution`**

Detalhes de um pacote de distribuição instalado.

Observação: instâncias diferentes de `Distribution` não são atualmente consideradas iguais em uma comparação, mesmo que estejam relacionadas à mesma distribuição instalada e, portanto, tenham os mesmos atributos.

Embora a API de nível de módulo descrita acima seja o uso mais comum e conveniente, você pode obter todas essas informações da classe `Distribution`. `Distribution` é um objeto abstrato que representa os metadados para um Pacote de Distribuição Python. Você pode obter a instância concreta de subclasse de `Distribution` para um pacote de distribuição instalado chamando a função *`distribution()`*:

```
>>> from importlib.metadata import distribution
>>> dist = distribution('wheel')
>>> type(dist)
<class 'importlib.metadata.PathDistribution'>
```

Assim, uma forma alternativa de obter o número da versão é através da instância `Distribution`:

```
>>> dist.version
'0.32.3'
```

Existem todos os tipos de metadados adicionais disponíveis em instâncias de `Distribution`:

```
>>> dist.metadata['Requires-Python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
>>> dist.metadata['License']
'MIT'
```

Para pacotes editáveis, uma propriedade `origin` pode apresentar metadados conforme a **PEP 610**:

```
>>> dist.origin.url
'file:///caminho/para/wheel-0.32.3.editable-py3-none-any.whl'
```

O conjunto completo de metadados disponíveis não é descrito aqui. Consulte a especificação de metadados principais do PyPA para obter detalhes adicionais.

Adicionado na versão 3.13: A propriedade `.origin` foi adicionada.

31.8.4 Descoberta de distribuição

Por padrão, este pacote fornece suporte embutido para descoberta de metadados para sistema de arquivos e arquivo zip Pacotes de Distribuição. Esta pesquisa do localizador de metadados tem como padrão `sys.path`, mas varia um pouco na maneira como ela interpreta esses valores em relação a outras mecanismo de importação. Em particular:

- `importlib.metadata` não honra objetos *bytes* em `sys.path`.
- `importlib.metadata` irá incidentalmente honrar objetos *pathlib.Path* em `sys.path` mesmo que tais valores sejam ignorados para importações.

31.8.5 Implementando provedores personalizados

`importlib.metadata` aborda duas superfícies de API, uma para *consumidores* e outra para *provedores*. A maioria dos usuários são consumidores, consumindo metadados fornecidos pelos pacotes. Existem outros casos de uso, no entanto, em que os usuários desejam expor metadados por meio de algum outro mecanismo, como junto com um importador personalizado. Esse caso de uso exige um *provedor personalizado*.

Como os metadados de Pacote de Distribuição não estão disponíveis por meio de pesquisas a *sys.path* ou carregadores de pacotes diretamente, os metadados de uma distribuição são encontrados através de localizadores do sistema de importação. Para encontrar os metadados de um pacote de distribuição, `importlib.metadata` consulta a lista de *localizadores de metacaminho* em *sys.meta_path*.

A implementação possui ganchos integrados ao `PathFinder`, servindo metadados para pacotes de distribuição encontrados no sistema de arquivos.

A classe abstrata *importlib.abc.MetaPathFinder* define a interface esperada dos localizadores pelo sistema de importação do Python. `importlib.metadata` estende este protocolo procurando por um chamável `find_distributions` opcional nos localizadores de *sys.meta_path* e apresenta esta interface estendida como a classe base abstrata `DistributionFinder`, que define este método abstrato:

```
@abc.abstractmethod
def find_distributions(context=DistributionFinder.Context()) ->␣
↪Iterable[Distribution]:
    """Return an iterable of all Distribution instances capable of
    loading the metadata for packages for the indicated ``context``.
    """
```

O objeto `DistributionFinder.Context` fornece propriedades `.path` e `.name` indicando o caminho para pesquisar e o nome a ser correspondido e pode fornecer outro contexto relevante buscado pelo consumidor.

Na prática, para prover suporte à localização de metadados de pacotes de distribuição em outros locais fora do sistema de arquivos, crie uma subclasse `Distribution` e implemente os métodos abstratos. Então, a partir de um localizador personalizado, retorne instâncias deste derivado de `Distribution` no método `find_distributions()`.

Exemplo

Imagine um localizador personalizado que carrega módulos Python de um banco de dados:

```
class DatabaseImporter(importlib.abc.MetaPathFinder):
    def __init__(self, db):
        self.db = db
```

(continua na próxima página)

(continuação da página anterior)

```
    def find_spec(self, fullname, target=None) -> ModuleSpec:
        return self.db.spec_from_name(fullname)

sys.meta_path.append(DatabaseImporter(connect_db(...)))
```

Esse importador agora provavelmente fornece módulos importáveis de um banco de dados, mas não fornece metadados ou pontos de entrada. Para que este importador personalizado forneça metadados, ele também precisaria implementar `DistributionFinder`:

```
from importlib.metadata import DistributionFinder

class DatabaseImporter(DistributionFinder):
    ...

    def find_distributions(self, context=DistributionFinder.Context()):
        query = dict(name=context.name) if context.name else {}
        for dist_record in self.db.query_distributions(query):
            yield DatabaseDistribution(dist_record)
```

Desta forma, `query_distributions` retornaria registros para cada distribuição servida pelo banco de dados correspondente à consulta. Por exemplo, se `requests-1.0` estiver no banco de dados, `find_distributions` produziria um `DatabaseDistribution` para `Context(name='requests')` ou `Context(name=None)`.

Por uma questão de simplicidade, este exemplo ignora `context.path`. O atributo `path` tem como padrão `sys.path` e é o conjunto de caminhos de importação a serem considerados na pesquisa. Um `DatabaseImporter` poderia funcionar potencialmente sem qualquer preocupação com um caminho de pesquisa. Supondo que o importador não faça particionamento, o caminho "path" seria irrelevante. Para ilustrar o propósito de `path`, o exemplo precisaria ilustrar um `DatabaseImporter` mais complexo cujo comportamento variasse dependendo de `sys.path`/`PYTHONPATH`. Nesse caso, `find_distributions` deve respeitar o `context.path` e produzir apenas `Distributions` pertinentes a esse caminho.

`DatabaseDistribution`, então, se pareceria com algo como:

```
class DatabaseDistribution(importlib.metadata.Distribution):
    def __init__(self, record):
        self.record = record

    def read_text(self, filename):
        """
        Lê um arquivo como "METADATA" para a atual distribuição.
        """
        if filename == "METADATA":
            return f"""Name: {self.record.name}
Version: {self.record.version}
"""
        if filename == "entry_points.txt":
            return "\n".join(
              f"""[{ep.group}]\n{ep.name}={ep.value}"""
              for ep in self.record.entry_points)

    def locate_file(self, path):
        raise RuntimeError("This distribution has no file system")
```

Esta implementação básica deve fornecer metadados e pontos de entrada para pacotes servidos pelo `DatabaseImporter`, presumindo que o `record` forneça os atributos `.name`, `.version` e `.entry_points` adequados.

O `DatabaseDistribution` também pode fornecer outros arquivos de metadados, como `RECORD` (necessário para `Distribution.files`) ou substituir a implementação de `Distribution.files`. Veja o código-fonte para mais

inspiração.

31.9 A inicialização do caminho de pesquisa de módulos `sys.path`

Um caminho de pesquisa de módulos é inicializado quando o Python é iniciado. Este caminho de pesquisa de módulos pode ser acessado em *sys.path*.

A primeira entrada no caminho de pesquisa de módulos é o diretório que contém o script de entrada, se houver. Caso contrário, a primeira entrada é o diretório atual, que é o caso ao executar o console interativo, um comando com `-c` ou um módulo com `-m`.

A variável de ambiente `PYTHONPATH` é frequentemente usada para adicionar diretórios ao caminho de pesquisa. Se esta variável de ambiente for encontrada, o conteúdo será adicionado ao caminho de pesquisa de módulos.

Nota

`PYTHONPATH` afetará todas as versões/ambientes Python instalados. Tenha cuidado ao definir isso em seu perfil do shell ou em variáveis de ambiente globais. O módulo *site* oferece técnicas mais diferenciadas, conforme mencionado abaixo.

Os próximos itens adicionados são os diretórios contendo módulos padrão do Python, bem como quaisquer *módulos de extensão* dos quais esses módulos dependem. Módulos de extensão são arquivos `.pyd` no Windows e arquivos `.so` em outras plataformas. O diretório com os módulos Python independentes de plataforma é chamado `prefix`. O diretório com os módulos de extensão é chamado `exec_prefix`.

A variável de ambiente `PYTHONHOME` pode ser usada para definir os locais `prefix` e `exec_prefix`. Caso contrário, esses diretórios serão encontrados usando o executável Python como ponto de partida e, em seguida, procurando por vários arquivos e diretórios "de referência". Observe que todos os links simbólicos são seguidos para que o local real do executável do Python seja usado como ponto de partida da pesquisa. O local do executável do Python é chamado `home`.

Uma vez que `home` é determinado, o diretório `prefix` é encontrado procurando primeiro por `python`*majorversion*`minorversion.zip` (`python311.zip`). No Windows, o arquivo zip é procurado em `home` e no Unix espera-se que o arquivo esteja em `lib`. Observe que o local esperado do arquivo zip é adicionado ao caminho de pesquisa de módulos, mesmo que o arquivo não exista. Se nenhum arquivo for encontrado, o Python no Windows continuará a pesquisa por `prefix` procurando por `Lib\os.py`. Python no Unix procurará por `lib/python`*majorversion*`.`*minorversion*`/os.py` (`lib/python3.11/os.py`). No Windows, `prefix` e `exec_prefix` são iguais, porém em outras plataformas `lib/python`*majorversion*`.`*minorversion*`/lib-dynload` (`lib/python3.11/lib-dynload`) é pesquisado e usado como âncora para `exec_prefix`. Em algumas plataformas, `lib` pode ser `lib64` ou outro valor, veja *sys.platlibdir* e `PYTHONPLATLIBDIR`.

Uma vez encontrados, `prefix` e `exec_prefix` estão disponíveis em *sys.base_prefix* e *sys.base_exec_prefix*, respectivamente.

Se `PYTHONHOME` não estiver definido e um arquivo `pyvenv.cfg` for encontrado ao lado do executável principal ou em seu diretório pai, *sys.prefix* e *sys.exec_prefix* serão definidos para o diretório que contém `pyvenv.cfg`; caso contrário, serão definidos com o mesmo valor que *sys.base_prefix* e *sys.base_exec_prefix*, respectivamente. Isso é usado por *Ambientes virtuais*.

Finalmente, o módulo *site* é processado e os diretórios `site-packages` são adicionados ao caminho de pesquisa de módulos. Uma maneira comum de personalizar o caminho de pesquisa é criar módulos *sitecustomize* ou *usercustomize* conforme descrito na documentação do módulo *site*.

Nota

Certas opções de linha de comando podem afetar ainda mais os cálculos de caminho. Veja `-E`, `-I`, `-s` e `-S` para mais detalhes.

Alterado na versão 3.14: *sys.prefix* e *sys.exec_prefix* agora são definidos para o diretório `pyvenv.cfg` durante a inicialização do caminho. Isso era feito anteriormente por *site* e, portanto, afetado por `-S`.

31.9.1 Ambientes virtuais

Ambientes virtuais colocam um arquivo `pyvenv.cfg` em seu prefixo, o que faz com que *sys.prefix* e *sys.exec_prefix* apontem para eles, em vez da instalação base.

Os valores `prefix` e `exec_prefix` da instalação base estão disponíveis em *sys.base_prefix* e *sys.base_exec_prefix*.

Além de ser usado como um marcador para identificar ambientes virtuais, `pyvenv.cfg` também pode ser usado para configurar a inicialização do *site*. Consulte a *documentação de ambientes virtuais* do *site*.

Nota

`PYTHONHOME` substitui a detecção `pyvenv.cfg`.

Nota

Existem outras maneiras de implementar "ambientes virtuais". Esta documentação menciona implementações baseadas no mecanismo `pyvenv.cfg`, como *venv*. A maioria das implementações de ambientes virtuais segue o modelo definido por *venv*, mas pode haver implementações exóticas que divergem dele.

31.9.2 Arquivos _pth

Para substituir completamente *sys.path*, crie um arquivo `._pth` com o mesmo nome da biblioteca compartilhada ou executável (`python._pth` ou `python311._pth`). O caminho da biblioteca compartilhada é sempre conhecido no Windows, mas pode não estar disponível em outras plataformas. No arquivo `._pth`, especifique uma linha para cada caminho a ser adicionado a *sys.path*. O arquivo baseado no nome da biblioteca compartilhada substitui aquele baseado no executável, o que permite que os caminhos sejam restritos para qualquer programa que carregue o tempo de execução, se desejado.

Quando o arquivo existe, todos os registros e variáveis de ambiente são ignorados, o modo isolado é ativado, e *site* não é importado a menos que uma linha do arquivo especifique `import site`. Caminhos em branco e linhas começando com `#` são ignorados. Cada caminho pode ser absoluto ou relativo ao local do arquivo. Instruções de importação diferente de `site` não são permitidas, e código arbitrário não pode ser especificado.

Note que arquivos `.pth` (sem o sublinhado no início) serão processados normalmente pelo módulo *site* quando `import site` tiver sido especificado.

31.9.3 Python embarcado

Se o Python estiver incorporado em outra aplicação, `Py_InitializeFromConfig()` e a estrutura `PyConfig` podem ser usados para inicializar o Python. Os detalhes específicos do caminho estão descritos em init-path-config.

Ver também

- windows_finding_modules para observações detalhadas do Windows.
- using-on-unix para detalhes do Unix.

CAPÍTULO 32

Serviços da Linguagem Python

O Python fornece vários módulos para ajudar a trabalhar com a linguagem Python. Estes módulos suportam tokenizing, parsing, análise de sintaxe, disassembly de bytecode e várias outras facilidades.

Esses módulos incluem:

32.1 `ast` — Árvores de sintaxe abstrata

Código-fonte: Lib/ast.py

O módulo *ast* ajuda as aplicações Python a processar árvores da gramática de sintaxe abstrata do Python. A sintaxe abstrata em si pode mudar em cada lançamento do Python; este módulo ajuda a descobrir programaticamente como é a gramática atual.

Uma árvore de sintaxe abstrata pode ser gerada passando *ast.PyCF_ONLY_AST* como um sinalizador para a função embutida *compile()*, ou usando o auxiliar *parse()* fornecido neste módulo. O resultado será uma árvore de objetos cujas classes herdam de *ast.AST*. Uma árvore de sintaxe abstrata pode ser compilada em um objeto código Python usando a função embutida *compile()*.

32.1.1 Gramática abstrata

A gramática abstrata está atualmente definida da seguinte forma:

```
-- ASDL's 4 builtin types are:
-- identifier, int, string, constant

module Python
{
    mod = Module(stmt* body, type_ignore* type_ignores)
        | Interactive(stmt* body)
        | Expression(expr body)
        | FunctionType(expr* argtypes, expr returns)

    stmt = FunctionDef(identifier name, arguments args,
                       stmt* body, expr* decorator_list, expr? returns,
                       string? type_comment, type_param* type_params)
```

(continua na próxima página)

(continuação da página anterior)

```
          | AsyncFunctionDef(identifier name, arguments args,
                             stmt* body, expr* decorator_list, expr? returns,
                             string? type_comment, type_param* type_params)

          | ClassDef(identifier name,
             expr* bases,
             keyword* keywords,
             stmt* body,
             expr* decorator_list,
             type_param* type_params)
          | Return(expr? value)

          | Delete(expr* targets)
          | Assign(expr* targets, expr value, string? type_comment)
          | TypeAlias(expr name, type_param* type_params, expr value)
          | AugAssign(expr target, operator op, expr value)
          -- 'simple' indicates that we annotate simple name without parens
          | AnnAssign(expr target, expr annotation, expr? value, int simple)

          -- use 'orelse' because else is a keyword in target languages
          | For(expr target, expr iter, stmt* body, stmt* orelse, string? type_
↪comment)
          | AsyncFor(expr target, expr iter, stmt* body, stmt* orelse, string?␣
↪type_comment)
          | While(expr test, stmt* body, stmt* orelse)
          | If(expr test, stmt* body, stmt* orelse)
          | With(withitem* items, stmt* body, string? type_comment)
          | AsyncWith(withitem* items, stmt* body, string? type_comment)

          | Match(expr subject, match_case* cases)

          | Raise(expr? exc, expr? cause)
          | Try(stmt* body, excepthandler* handlers, stmt* orelse, stmt* finalbody)
          | TryStar(stmt* body, excepthandler* handlers, stmt* orelse, stmt*␣
↪finalbody)
          | Assert(expr test, expr? msg)

          | Import(alias* names)
          | ImportFrom(identifier? module, alias* names, int? level)

          | Global(identifier* names)
          | Nonlocal(identifier* names)
          | Expr(expr value)
          | Pass | Break | Continue

          -- col_offset is the byte offset in the utf8 string the parser uses
          attributes (int lineno, int col_offset, int? end_lineno, int? end_col_
↪offset)

          -- BoolOp() can use left & right?
    expr = BoolOp(boolop op, expr* values)
         | NamedExpr(expr target, expr value)
         | BinOp(expr left, operator op, expr right)
         | UnaryOp(unaryop op, expr operand)
         | Lambda(arguments args, expr body)
         | IfExp(expr test, expr body, expr orelse)
```

(continua na próxima página)

(continuação da página anterior)

```
         | Dict(expr?* keys, expr* values)
         | Set(expr* elts)
         | ListComp(expr elt, comprehension* generators)
         | SetComp(expr elt, comprehension* generators)
         | DictComp(expr key, expr value, comprehension* generators)
         | GeneratorExp(expr elt, comprehension* generators)
         -- the grammar constrains where yield expressions can occur
         | Await(expr value)
         | Yield(expr? value)
         | YieldFrom(expr value)
         -- need sequences for compare to distinguish between
         -- x < 4 < 3 and (x < 4) < 3
         | Compare(expr left, cmpop* ops, expr* comparators)
         | Call(expr func, expr* args, keyword* keywords)
         | FormattedValue(expr value, int conversion, expr? format_spec)
         | Interpolation(expr value, constant str, int conversion, expr? format_
↪spec)
         | JoinedStr(expr* values)
         | TemplateStr(expr* values)
         | Constant(constant value, string? kind)

         -- the following expression can appear in assignment context
         | Attribute(expr value, identifier attr, expr_context ctx)
         | Subscript(expr value, expr slice, expr_context ctx)
         | Starred(expr value, expr_context ctx)
         | Name(identifier id, expr_context ctx)
         | List(expr* elts, expr_context ctx)
         | Tuple(expr* elts, expr_context ctx)

         -- can appear only in Subscript
         | Slice(expr? lower, expr? upper, expr? step)

          -- col_offset is the byte offset in the utf8 string the parser uses
          attributes (int lineno, int col_offset, int? end_lineno, int? end_col_
↪offset)

    expr_context = Load | Store | Del

    boolop = And | Or

    operator = Add | Sub | Mult | MatMult | Div | Mod | Pow | LShift
                 | RShift | BitOr | BitXor | BitAnd | FloorDiv

    unaryop = Invert | Not | UAdd | USub

    cmpop = Eq | NotEq | Lt | LtE | Gt | GtE | Is | IsNot | In | NotIn

    comprehension = (expr target, expr iter, expr* ifs, int is_async)

    excepthandler = ExceptHandler(expr? type, identifier? name, stmt* body)
                    attributes (int lineno, int col_offset, int? end_lineno, int?␣
↪end_col_offset)

    arguments = (arg* posonlyargs, arg* args, arg? vararg, arg* kwonlyargs,
                 expr* kw_defaults, arg? kwarg, expr* defaults)
```

(continua na próxima página)

(continuação da página anterior)

```
    arg = (identifier arg, expr? annotation, string? type_comment)
           attributes (int lineno, int col_offset, int? end_lineno, int? end_col_
↪offset)

    -- keyword arguments supplied to call (NULL identifier for **kwargs)
    keyword = (identifier? arg, expr value)
               attributes (int lineno, int col_offset, int? end_lineno, int? end_
↪col_offset)

    -- import name with optional 'as' alias.
    alias = (identifier name, identifier? asname)
             attributes (int lineno, int col_offset, int? end_lineno, int? end_col_
↪offset)

    withitem = (expr context_expr, expr? optional_vars)

    match_case = (pattern pattern, expr? guard, stmt* body)

    pattern = MatchValue(expr value)
            | MatchSingleton(constant value)
            | MatchSequence(pattern* patterns)
            | MatchMapping(expr* keys, pattern* patterns, identifier? rest)
            | MatchClass(expr cls, pattern* patterns, identifier* kwd_attrs,␣
↪pattern* kwd_patterns)

            | MatchStar(identifier? name)
            -- The optional "rest" MatchMapping parameter handles capturing extra␣
↪mapping keys

            | MatchAs(pattern? pattern, identifier? name)
            | MatchOr(pattern* patterns)

             attributes (int lineno, int col_offset, int end_lineno, int end_col_
↪offset)

    type_ignore = TypeIgnore(int lineno, string tag)

    type_param = TypeVar(identifier name, expr? bound, expr? default_value)
               | ParamSpec(identifier name, expr? default_value)
               | TypeVarTuple(identifier name, expr? default_value)
              attributes (int lineno, int col_offset, int end_lineno, int end_col_
↪offset)
}
```

32.1.2 Classes de nós

class ast.**AST**

Esta é a base de todas as classes de nós de AST. As classes de nós reais são derivadas do arquivo `Parser/Python.asdl`, que é reproduzido *acima*. Elas são definidas no módulo C `_ast` e reexportadas no *ast*.

Há uma classe definida para cada símbolo do lado esquerdo na gramática abstrata (por exemplo, `ast.stmt` ou `ast.expr`). Além disso, existe uma classe definida para cada construtor no lado direito; essas classes herdam das classes para as árvores do lado esquerdo. Por exemplo, *ast.BinOp* herda de `ast.expr`. Para regras de produção com alternativas ("somas"), a classe do lado esquerdo é abstrata: apenas instâncias de nós construtores específicos são criadas.

_fields

Cada classe concreta possui um atributo `_fields` que fornece os nomes de todos os nós filhos.

Cada instância de uma classe concreta tem um atributo para cada nó filho, do tipo definido na gramática. Por exemplo, as instâncias *ast.BinOp* possuem um atributo `left` do tipo `ast.expr`.

Se estes atributos estiverem marcados como opcionais na gramática (usando um ponto de interrogação), o valor pode ser `None`. Se os atributos puderem ter valor zero ou mais (marcados com um asterisco), os valores serão representados como listas do Python. Todos os atributos possíveis devem estar presentes e ter valores válidos ao compilar uma AST com *compile()*.

_field_types

O atributo `_field_types` em cada classe concreta é um dicionário que mapeia nomes de campos (como também listado em *_fields*) para seus tipos.

```
>>> ast.TypeVar._field_types
{'name': <class 'str'>, 'bound': ast.expr | None, 'default_value': ast.
↪expr | None}
```

Adicionado na versão 3.13.

lineno

col_offset

end_lineno

end_col_offset

As instâncias das subclasses `ast.expr` e `ast.stmt` possuem os atributos *lineno*, *col_offset*, *end_lineno* e *end_col_offset*. O *lineno* e *end_lineno* são o primeiro e o último número de linha do intervalo do texto de origem (indexado em 1, para que a primeira linha seja a linha 1) e o *col_offset* e *end_col_offset* são os deslocamentos de byte UTF-8 correspondentes do primeiro e do último tokens que geraram o nó. O deslocamento UTF-8 é registrado porque o analisador sintático usa UTF-8 internamente.

Observe que as posições finais não são exigidas pelo compilador e, portanto, são opcionais. O deslocamento final está *após* o último símbolo, por exemplo, é possível obter o segmento de origem de um nó de expressão de uma linha usando `source_line[node.col_offset : node.end_col_offset]`.

O construtor de uma classe `ast.T` analisa seus argumentos da seguinte forma:

- Se houver argumentos posicionais, deve haver tantos quanto houver itens em `T._fields`; eles serão atribuídos como atributos desses nomes.
- Se houver argumentos nomeados, eles definirão os atributos dos mesmos nomes para os valores fornecidos.

Por exemplo, para criar e popular um nó *ast.UnaryOp*, você poderia usar

```
node = ast.UnaryOp(ast.USub(), ast.Constant(5, lineno=0, col_offset=0),
                   lineno=0, col_offset=0)
```

Se um campo que é opcional na gramática for omitido do construtor, o padrão é `None`. Se um campo de lista for omitido, o padrão será a lista vazia. Se um campo do tipo `ast.expr_context` for omitido, o padrão é *Load()*. Se qualquer outro campo for omitido, uma *DeprecationWarning* será levantada e o nó AST não terá este campo. No Python 3.15, esta condição levantará um erro.

Alterado na versão 3.8: A classe *ast.Constant* é agora usada para todas as constantes.

Alterado na versão 3.9: Os índices simples são representados por seus valores, as fatias estendidas são representadas como tuplas.

Alterado na versão 3.14: A saída `__repr__()` dos nós *AST* inclui os valores dos campos dos nós.

Descontinuado desde a versão 3.8: Classes antigas `ast.Num`, `ast.Str`, `ast.Bytes`, `ast.NameConstant` e `ast.Ellipsis` ainda estão disponíveis, mas serão removidos em versões futuras do Python. Enquanto isso, instanciá-las retornará uma instância de uma classe diferente.

Descontinuado desde a versão 3.9: Classes antigas `ast.Index` e `ast.ExtSlice` ainda estão disponíveis, mas serão removidos em versões futuras do Python. Enquanto isso, instanciá-las retornará uma instância de uma classe diferente.

Deprecated since version 3.13, will be removed in version 3.15: Versões anteriores do Python permitiam a criação de nós AST sem campos obrigatórios. Da mesma forma, os construtores do nó AST permitiam argumentos nomeados arbitrários que eram definidos como atributos do nó AST, mesmo que não correspondessem a nenhum dos campos do nó AST. Este comportamento foi descontinuado e será removido no Python 3.15.

Nota

As descrições das classes de nós específicas exibidas aqui foram inicialmente adaptadas do fantástico projeto Green Tree Snakes e de todos os seus contribuidores.

Nós raízes

class `ast.`**Module**(*body*, *type_ignores*)

Um módulo Python, como entrada de arquivo. Tipo de nó gerado por *ast.parse()* com *mode* no padrão `"exec"`.

`body` é uma *list* das *Instruções* do módulo.

`type_ignores` é uma *list* dos comentários de ignorar tipo do módulo; veja *ast.parse()* para mais detalhes.

```
>>> print(ast.dump(ast.parse('x = 1'), indent=4))
Module(
    body=[
        Assign(
            targets=[
                Name(id='x', ctx=Store())],
            value=Constant(value=1))])
```

class `ast.`**Expression**(*body*)

Uma única entrada de expressão Python. Tipo de nó gerado por *ast.parse()* quando *mode* é `"eval"`.

`body` é um nó único, um dos *tipos de expressão*.

```
>>> print(ast.dump(ast.parse('123', mode='eval'), indent=4))
Expression(
    body=Constant(value=123))
```

class `ast.`**Interactive**(*body*)

Uma única entrada interativa, como em tut-interac. Tipo de nó gerado por *ast.parse()* quando *mode* é `"single"`.

`body` é uma *list* de *nós de instrução*.

```
>>> print(ast.dump(ast.parse('x = 1; y = 2', mode='single'), indent=4))
Interactive(
    body=[
        Assign(
            targets=[
                Name(id='x', ctx=Store())],
            value=Constant(value=1)),
        Assign(
            targets=[
                Name(id='y', ctx=Store())],
            value=Constant(value=2))])
```

class ast.**FunctionType**(*argtypes*, *returns*)

Uma representação de comentários de tipo antigo para funções, já que as versões do Python anteriores a 3.5 não davam suporte às anotações da **PEP 484**. Tipo de nó gerado por *ast.parse()* quando *mode* é "func_type".

Esses comentários de tipo ficariam assim:

```
def sum_two_number(a, b):
    # tipo: (int, int) -> int
    return a + b
```

argtypes é uma *list* de *nós de expressão*.

returns é um único *nó de expressão*.

```
>>> print(ast.dump(ast.parse('(int, str) -> List[int]', mode='func_type'),␣
↪indent=4))
FunctionType(
    argtypes=[
        Name(id='int', ctx=Load()),
        Name(id='str', ctx=Load())],
    returns=Subscript(
        value=Name(id='List', ctx=Load()),
        slice=Name(id='int', ctx=Load()),
        ctx=Load()))
```

Adicionado na versão 3.8.

Literais

class ast.**Constant**(*value*)

Um valor constante. O atributo value do literal Constant contém o objeto Python que ele representa. Os valores representados podem ser instâncias de *str*, *bytes*, *int*, *float*, *complex* e *bool*, e as constantes *None* e *Ellipsis*.

```
>>> print(ast.dump(ast.parse('123', mode='eval'), indent=4))
Expression(
    body=Constant(value=123))
```

class ast.**FormattedValue**(*value*, *conversion*, *format_spec*)

Nó que representa um único campo de formatação em uma f-string. Se a string contiver um único campo de formatação e nada mais, o nó poderá ser isolado, caso contrário ele aparecerá em *JoinedStr*.

- value é qualquer nó de expressão (como um literal, uma variável ou uma chamada de função).
- conversion é um inteiro:
  - -1: sem formatação
  - 97 (ord('a')): !a *ASCII* formatting
  - 114 (ord('r')): !r *repr()* formatting
  - 115 (ord('s')): !s *string* formatting
- format_spec é um nó *JoinedStr* que representa a formatação do valor, ou None se nenhum formato foi especificado. Tanto conversion quanto format_spec podem ser configurados ao mesmo tempo.

class ast.**JoinedStr**(*values*)

Uma f-string, compreendendo uma série de nós *FormattedValue* e *Constant*.

```
>>> print(ast.dump(ast.parse('f"sin({a}) is {sin(a):.3}"', mode='eval'),␣
↪indent=4))
Expression(
    body=JoinedStr(
        values=[
            Constant(value='sin('),
            FormattedValue(
                value=Name(id='a', ctx=Load()),
                conversion=-1),
            Constant(value=') is '),
            FormattedValue(
                value=Call(
                    func=Name(id='sin', ctx=Load()),
                    args=[
                        Name(id='a', ctx=Load())]),
                conversion=-1,
                format_spec=JoinedStr(
                    values=[
                        Constant(value='.3')]))])))
```

class ast.**TemplateStr**(*values*, */*)

Adicionado na versão 3.14.

Node representing a template string literal, comprising a series of *Interpolation* and *Constant* nodes. These nodes may be any order, and do not need to be interleaved.

```
>>> expr = ast.parse('t"{name} finished {place:ordinal}"', mode='eval')
>>> print(ast.dump(expr, indent=4))
Expression(
    body=TemplateStr(
        values=[
            Interpolation(
                value=Name(id='name', ctx=Load()),
                str='name',
                conversion=-1),
            Constant(value=' finished '),
            Interpolation(
                value=Name(id='place', ctx=Load()),
                str='place',
                conversion=-1,
                format_spec=JoinedStr(
                    values=[
                        Constant(value='ordinal')]))]))
```

class ast.**Interpolation**(*value*, *str*, *conversion*, *format_spec=None*)

Adicionado na versão 3.14.

Node representing a single interpolation field in a template string literal.

- `value` is any expression node (such as a literal, a variable, or a function call). This has the same meaning as `FormattedValue.value`.
- `str` is a constant containing the text of the interpolation expression.
- `conversion` é um inteiro:
  - -1: no conversion
  - 97 (`ord('a')`): `!a` *ASCII* conversion
  - 114 (`ord('r')`): `!r` *repr()* conversion

- 115 (`ord('s')`): `!s` *string* conversion

This has the same meaning as `FormattedValue.conversion`.

- `format_spec` is a *JoinedStr* node representing the formatting of the value, or `None` if no format was specified. Both `conversion` and `format_spec` can be set at the same time. This has the same meaning as `FormattedValue.format_spec`.

class `ast.`**List**(*elts*, *ctx*)

class `ast.`**Tuple**(*elts*, *ctx*)

Uma lista ou tupla. `elts` contém uma lista de nós que representam os elementos. `ctx` é *Store* se o contêiner for um alvo de atribuição (ou seja, `(x,y)=algumacoisa`), e *Load* caso contrário.

```
>>> print(ast.dump(ast.parse('[1, 2, 3]', mode='eval'), indent=4))
Expression(
    body=List(
        elts=[
            Constant(value=1),
            Constant(value=2),
            Constant(value=3)],
        ctx=Load()))
>>> print(ast.dump(ast.parse('(1, 2, 3)', mode='eval'), indent=4))
Expression(
    body=Tuple(
        elts=[
            Constant(value=1),
            Constant(value=2),
            Constant(value=3)],
        ctx=Load()))
```

class `ast.`**Set**(*elts*)

Um conjunto. `elts` contém uma lista de nós que representam os elementos do conjunto.

```
>>> print(ast.dump(ast.parse('{1, 2, 3}', mode='eval'), indent=4))
Expression(
    body=Set(
        elts=[
            Constant(value=1),
            Constant(value=2),
            Constant(value=3)]))
```

class `ast.`**Dict**(*keys*, *values*)

Um dicionário. `keys` e `values` contêm listas de nós que representam as chaves e os valores respectivamente, em ordem correspondente (o que seria retornado ao chamar `dictionary.keys()` e `dictionary.values()`).

Ao desempacotar o dicionário usando literais de dicionário, a expressão a ser expandida vai para a lista `values`, com um `None` na posição correspondente em `keys`.

```
>>> print(ast.dump(ast.parse('{"a":1, **d}', mode='eval'), indent=4))
Expression(
    body=Dict(
        keys=[
            Constant(value='a'),
            None],
        values=[
            Constant(value=1),
            Name(id='d', ctx=Load())]))
```

Variáveis

class ast.**Name**(*id*, *ctx*)

Um nome de variável. `id` contém o nome como uma string e `ctx` é um dos seguintes tipos.

class ast.**Load**

class ast.**Store**

class ast.**Del**

As referências de variáveis podem ser usadas para carregar o valor de uma variável, para atribuir um novo valor a ela ou para excluí-la. As referências de variáveis recebem um contexto para distinguir esses casos.

```
>>> print(ast.dump(ast.parse('a'), indent=4))
Module(
    body=[
        Expr(
            value=Name(id='a', ctx=Load()))])

>>> print(ast.dump(ast.parse('a = 1'), indent=4))
Module(
    body=[
        Assign(
            targets=[
                Name(id='a', ctx=Store())],
            value=Constant(value=1))])

>>> print(ast.dump(ast.parse('del a'), indent=4))
Module(
    body=[
        Delete(
            targets=[
                Name(id='a', ctx=Del())])])
```

class ast.**Starred**(*value*, *ctx*)

Uma referência de variável `*var`. `value` contém a variável, normalmente um nó *Name*. Este tipo deve ser usado ao construir um nó *Call* com `*args`.

```
>>> print(ast.dump(ast.parse('a, *b = it'), indent=4))
Module(
    body=[
        Assign(
            targets=[
                Tuple(
                    elts=[
                        Name(id='a', ctx=Store()),
                        Starred(
                            value=Name(id='b', ctx=Store()),
                            ctx=Store())],
                    ctx=Store())],
            value=Name(id='it', ctx=Load()))])
```

Expressões

class ast.**Expr**(*value*)

Quando uma expressão, como uma chamada de função, aparece como uma instrução por si só com seu valor de retorno não usado ou armazenado, ela é encapsulada neste contêiner. `value` contém um dos outros nós nesta seção, um nó *Constant*, um *Name*, um *Lambda*, um *Yield* ou *YieldFrom*.

```
>>> print(ast.dump(ast.parse('-a'), indent=4))
Module(
    body=[
        Expr(
            value=UnaryOp(
                op=USub(),
                operand=Name(id='a', ctx=Load()))])
```

class ast.**UnaryOp**(*op*, *operand*)

Uma operação unária. `op` é o operador e `operand` qualquer nó de expressão.

class ast.**UAdd**

class ast.**USub**

class ast.**Not**

class ast.**Invert**

Tokens de operador unário. *Not* é a palavra reservada `not`, *Invert* é o operador `~`.

```
>>> print(ast.dump(ast.parse('not x', mode='eval'), indent=4))
Expression(
    body=UnaryOp(
        op=Not(),
        operand=Name(id='x', ctx=Load())))
```

class ast.**BinOp**(*left*, *op*, *right*)

Uma operação binária (como adição ou divisão). `op` é o operador, e `left` e `right` são quaisquer nós de expressão.

```
>>> print(ast.dump(ast.parse('x + y', mode='eval'), indent=4))
Expression(
    body=BinOp(
        left=Name(id='x', ctx=Load()),
        op=Add(),
        right=Name(id='y', ctx=Load())))
```

class ast.**Add**

class ast.**Sub**

class ast.**Mult**

class ast.**Div**

class ast.**FloorDiv**

class ast.**Mod**

class ast.**Pow**

class ast.**LShift**

class ast.**RShift**

class ast.**BitOr**

class ast.**BitXor**

class ast.**BitAnd**

class ast.**MatMult**

Tokens de operador binário.

class ast.**BoolOp**(*op*, *values*)

Uma operação booleana, 'or' ou 'and'. `op` é *Or* ou *And*. `values` são os valores envolvidos. Operações consecutivas com o mesmo operador, como `a or b or c`, são recolhidas em um nó com vários valores.

Isso não inclui `not`, que é um *UnaryOp*.

```
>>> print(ast.dump(ast.parse('x or y', mode='eval'), indent=4))
Expression(
    body=BoolOp(
        op=Or(),
        values=[
            Name(id='x', ctx=Load()),
            Name(id='y', ctx=Load())]))
```

class ast.**And**

class ast.**Or**

Tokens de operador booleano.

class ast.**Compare**(*left*, *ops*, *comparators*)

Uma comparação de dois ou mais valores. `left` é o primeiro valor na comparação, `ops` a lista de operadores e `comparators` a lista de valores após o primeiro elemento na comparação.

```
>>> print(ast.dump(ast.parse('1 <= a < 10', mode='eval'), indent=4))
Expression(
    body=Compare(
        left=Constant(value=1),
        ops=[
            LtE(),
            Lt()],
        comparators=[
            Name(id='a', ctx=Load()),
            Constant(value=10)]))
```

class ast.**Eq**

class ast.**NotEq**

class ast.**Lt**

class ast.**LtE**

class ast.**Gt**

class ast.**GtE**

class ast.**Is**

class ast.**IsNot**

class ast.**In**

class ast.**NotIn**

Tokens de operador de comparação.

class ast.**Call**(*func*, *args*, *keywords*)

Uma chamada de função. `func` é a função, que geralmente será um objeto *`Name`* ou *`Attribute`*. Dos argumentos:

- `args` contém uma lista dos argumentos passados por posição.
- `keywords` contém uma lista de objetos *`keyword`* representando argumentos passados como nomeados.

Os argumentos `args` e `keywords` são opcionais e padrão para listas vazias.

```
>>> print(ast.dump(ast.parse('func(a, b=c, *d, **e)', mode='eval'), indent=4))
Expression(
    body=Call(
        func=Name(id='func', ctx=Load()),
        args=[
            Name(id='a', ctx=Load()),
            Starred(
```

(continua na próxima página)

(continuação da página anterior)

```
                value=Name(id='d', ctx=Load()),
                ctx=Load())],
        keywords=[
            keyword(
                arg='b',
                value=Name(id='c', ctx=Load())),
            keyword(
                value=Name(id='e', ctx=Load()))]))
```

class ast.**keyword**(*arg*, *value*)

Um argumento nomeado para uma chamada de função ou definição de classe. `arg` é uma string bruta do nome do parâmetro, `value` é um nó para passar.

class ast.**IfExp**(*test*, *body*, *orelse*)

Uma expressão como `a if b else c`. Cada campo contém um único nó, portanto, no exemplo a seguir, todos os três são nós *Name*.

```
>>> print(ast.dump(ast.parse('a if b else c', mode='eval'), indent=4))
Expression(
    body=IfExp(
        test=Name(id='b', ctx=Load()),
        body=Name(id='a', ctx=Load()),
        orelse=Name(id='c', ctx=Load())))
```

class ast.**Attribute**(*value*, *attr*, *ctx*)

Acesso a atributo como, por exemplo, `d.keys`. `value` é um nó, normalmente um *Name*. `attr` é uma string simples fornecendo o nome do atributo, e `ctx` é *Load*, *Store* ou *Del* de acordo com como o atributo é acionado sobre.

```
>>> print(ast.dump(ast.parse('snake.colour', mode='eval'), indent=4))
Expression(
    body=Attribute(
        value=Name(id='snake', ctx=Load()),
        attr='colour',
        ctx=Load()))
```

class ast.**NamedExpr**(*target*, *value*)

Uma expressão nomeada. Este nó de AST é produzido pelo operador de expressões de atribuição (também conhecido como operador morsa). Ao contrário do nó *Assign* no qual o primeiro argumento pode ser múltiplos nós, neste caso ambos `target` e `value` devem ser nós únicos.

```
>>> print(ast.dump(ast.parse('(x := 4)', mode='eval'), indent=4))
Expression(
    body=NamedExpr(
        target=Name(id='x', ctx=Store()),
        value=Constant(value=4)))
```

Adicionado na versão 3.8.

Subscrição

class ast.**Subscript**(*value*, *slice*, *ctx*)

Um subscrito, como `l[1]`. `value` é o objeto subscrito (geralmente sequência ou mapeamento). `slice` é um índice, fatia ou chave. Pode ser uma *Tuple* e conter uma *Slice*. `ctx` é *Load*, *Store* ou *Del* de acordo com a ação realizada com o subscrito.

```
>>> print(ast.dump(ast.parse('l[1:2, 3]', mode='eval'), indent=4))
Expression(
    body=Subscript(
        value=Name(id='l', ctx=Load()),
        slice=Tuple(
            elts=[
                Slice(
                    lower=Constant(value=1),
                    upper=Constant(value=2)),
                Constant(value=3)],
            ctx=Load()),
        ctx=Load()))
```

class ast.**Slice**(*lower*, *upper*, *step*)

Fatiamento regular (no formato `lower:upper` ou `lower:upper:step`). Pode ocorrer apenas dentro do campo *slice* de *Subscript*, diretamente ou como um elemento de *Tuple*.

```
>>> print(ast.dump(ast.parse('l[1:2]', mode='eval'), indent=4))
Expression(
    body=Subscript(
        value=Name(id='l', ctx=Load()),
        slice=Slice(
            lower=Constant(value=1),
            upper=Constant(value=2)),
        ctx=Load()))
```

Compreensões

class ast.**ListComp**(*elt*, *generators*)

class ast.**SetComp**(*elt*, *generators*)

class ast.**GeneratorExp**(*elt*, *generators*)

class ast.**DictComp**(*key*, *value*, *generators*)

Lista e define compreensões, expressões geradoras e compreensões de dicionário. `elt` (ou `key` e `value`) é um único nó que representa a parte que será avaliada para cada item.

`generators` é uma lista de nós de *comprehension*.

```
>>> print(ast.dump(
...     ast.parse('[x for x in numbers]', mode='eval'),
...     indent=4,
... ))
Expression(
    body=ListComp(
        elt=Name(id='x', ctx=Load()),
        generators=[
            comprehension(
                target=Name(id='x', ctx=Store()),
                iter=Name(id='numbers', ctx=Load()),
                is_async=0)]))
>>> print(ast.dump(
...     ast.parse('{x: x**2 for x in numbers}', mode='eval'),
...     indent=4,
... ))
Expression(
    body=DictComp(
        key=Name(id='x', ctx=Load()),
```

(continua na próxima página)

(continuação da página anterior)

```
        value=BinOp(
            left=Name(id='x', ctx=Load()),
            op=Pow(),
            right=Constant(value=2)),
        generators=[
            comprehension(
                target=Name(id='x', ctx=Store()),
                iter=Name(id='numbers', ctx=Load()),
                is_async=0)]))
>>> print(ast.dump(
...     ast.parse('{x for x in numbers}', mode='eval'),
...     indent=4,
... ))
Expression(
    body=SetComp(
        elt=Name(id='x', ctx=Load()),
        generators=[
            comprehension(
                target=Name(id='x', ctx=Store()),
                iter=Name(id='numbers', ctx=Load()),
                is_async=0)]))
```

class ast.**comprehension**(*target*, *iter*, *ifs*, *is_async*)

Uma cláusula `for` em uma compreensão. `target` é a referência a ser usada para cada elemento - normalmente um nó *Name* ou *Tuple*. `iter` é o objeto sobre o qual iterar. `ifs` é uma lista de expressões de teste: cada cláusula `for` pode ter múltiplos `ifs`.

`is_async` indica que uma compreensão é assíncrona (usando um `async for` em vez de `for`). O valor é um número inteiro (0 ou 1).

```
>>> print(ast.dump(ast.parse('[ord(c) for line in file for c in line]', mode=
↪'eval'),
...                 indent=4)) # Várias compreensões em uma.
Expression(
    body=ListComp(
        elt=Call(
            func=Name(id='ord', ctx=Load()),
            args=[
                Name(id='c', ctx=Load())]),
        generators=[
            comprehension(
                target=Name(id='line', ctx=Store()),
                iter=Name(id='file', ctx=Load()),
                is_async=0),
            comprehension(
                target=Name(id='c', ctx=Store()),
                iter=Name(id='line', ctx=Load()),
                is_async=0)]))

>>> print(ast.dump(ast.parse('(n**2 for n in it if n>5 if n<10)', mode='eval'),
...                 indent=4)) # compreensão de gerador
Expression(
    body=GeneratorExp(
        elt=BinOp(
            left=Name(id='n', ctx=Load()),
            op=Pow(),
            right=Constant(value=2)),
```

(continua na próxima página)

(continuação da página anterior)

```
        generators=[
            comprehension(
                target=Name(id='n', ctx=Store()),
                iter=Name(id='it', ctx=Load()),
                ifs=[
                    Compare(
                        left=Name(id='n', ctx=Load()),
                        ops=[
                            Gt()],
                        comparators=[
                            Constant(value=5)]),
                    Compare(
                        left=Name(id='n', ctx=Load()),
                        ops=[
                            Lt()],
                        comparators=[
                            Constant(value=10)])],
                is_async=0)]))

>>> print(ast.dump(ast.parse('[i async for i in soc]', mode='eval'),
...                indent=4)) # compreensão de async
Expression(
    body=ListComp(
        elt=Name(id='i', ctx=Load()),
        generators=[
            comprehension(
                target=Name(id='i', ctx=Store()),
                iter=Name(id='soc', ctx=Load()),
                is_async=1)]))
```

Instruções

class ast.**Assign**(*targets*, *value*, *type_comment*)

Uma atribuição. `targets` é uma lista de nós e `value` é um único nó.

Vários nós em `targets` representam a atribuição do mesmo valor a cada um. O desempacotamento é representada colocando uma *Tuple* ou *List* dentro de `targets`.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário.

```
>>> print(ast.dump(ast.parse('a = b = 1'), indent=4)) # Multiple assignment
Module(
    body=[
        Assign(
            targets=[
                Name(id='a', ctx=Store()),
                Name(id='b', ctx=Store())],
            value=Constant(value=1))])

>>> print(ast.dump(ast.parse('a,b = c'), indent=4)) # Unpacking
Module(
    body=[
        Assign(
            targets=[
                Tuple(
                    elts=[
```

(continua na próxima página)

(continuação da página anterior)

```
                        Name(id='a', ctx=Store()),
                        Name(id='b', ctx=Store())],
                    ctx=Store())],
            value=Name(id='c', ctx=Load()))])
```

class ast.**AnnAssign**(*target*, *annotation*, *value*, *simple*)

Uma atribuição com uma anotação de tipo. `target` é um nó único e pode ser uma classe *Name*, *Attribute* ou *Subscript*. `annotation` é a anotação, como um nó *Constant* ou *Name*. `value` é um único nó opcional.

`simple` é sempre 0 (indicando um alvo "complexo") ou 1 (indicando um alvo "simples"). Um alvo "simples" consiste apenas em um nó *Name* que não aparece entre parênteses; todos os outros alvos são considerados complexos. Apenas alvos simples aparecem no dicionário `__annotations__` de módulos e classes.

```
>>> print(ast.dump(ast.parse('c: int'), indent=4))
Module(
    body=[
        AnnAssign(
            target=Name(id='c', ctx=Store()),
            annotation=Name(id='int', ctx=Load()),
            simple=1)])

>>> print(ast.dump(ast.parse('(a): int = 1'), indent=4)) # Annotation with␣
↪parenthesis
Module(
    body=[
        AnnAssign(
            target=Name(id='a', ctx=Store()),
            annotation=Name(id='int', ctx=Load()),
            value=Constant(value=1),
            simple=0)])

>>> print(ast.dump(ast.parse('a.b: int'), indent=4)) # Attribute annotation
Module(
    body=[
        AnnAssign(
            target=Attribute(
                value=Name(id='a', ctx=Load()),
                attr='b',
                ctx=Store()),
            annotation=Name(id='int', ctx=Load()),
            simple=0)])

>>> print(ast.dump(ast.parse('a[1]: int'), indent=4)) # Subscript annotation
Module(
    body=[
        AnnAssign(
            target=Subscript(
                value=Name(id='a', ctx=Load()),
                slice=Constant(value=1),
                ctx=Store()),
            annotation=Name(id='int', ctx=Load()),
            simple=0)])
```

class ast.**AugAssign**(*target*, *op*, *value*)

Atribuição aumentada, como `a += 1`. No exemplo a seguir, `target` é um nó *Name* para `x` (com o contexto *Store*), `op` é *Add*, e `value` é uma *Constant* com valor para 1.

O atributo `target` não pode ser da classe *Tuple* ou *List*, diferentemente dos alvos de *Assign*.

```
>>> print(ast.dump(ast.parse('x += 2'), indent=4))
Module(
    body=[
        AugAssign(
            target=Name(id='x', ctx=Store()),
            op=Add(),
            value=Constant(value=2))])
```

class ast.**Raise**(*exc*, *cause*)

Uma instrução `raise`. `exc` é o objeto de exceção a ser levantado, normalmente uma *Call* ou *Name*, ou `None` para um `raise` independente. `cause` é a parte opcional para `y` em `raise x from y`.

```
>>> print(ast.dump(ast.parse('raise x from y'), indent=4))
Module(
    body=[
        Raise(
            exc=Name(id='x', ctx=Load()),
            cause=Name(id='y', ctx=Load()))])
```

class ast.**Assert**(*test*, *msg*)

Uma asserção. `test` contém a condição, como um nó *Compare*. `msg` contém a mensagem de falha.

```
>>> print(ast.dump(ast.parse('assert x,y'), indent=4))
Module(
    body=[
        Assert(
            test=Name(id='x', ctx=Load()),
            msg=Name(id='y', ctx=Load()))])
```

class ast.**Delete**(*targets*)

Representa uma instrução `del`. `targets` é uma lista de nós, como nós *Name*, *Attribute* ou *Subscript*.

```
>>> print(ast.dump(ast.parse('del x,y,z'), indent=4))
Module(
    body=[
        Delete(
            targets=[
                Name(id='x', ctx=Del()),
                Name(id='y', ctx=Del()),
                Name(id='z', ctx=Del())])])
```

class ast.**Pass**

Uma instrução `pass`.

```
>>> print(ast.dump(ast.parse('pass'), indent=4))
Module(
    body=[
        Pass()])
```

class ast.**TypeAlias**(*name*, *type_params*, *value*)

Um *apelido de tipo* criado através da instrução `type`. `name` é o nome do apelido, `type_params` é uma lista de *parâmetros de tipo*, e `value` é o valor do apelido do tipo.

```
>>> print(ast.dump(ast.parse('type Alias = int'), indent=4))
Module(
    body=[
        TypeAlias(
```

(continua na próxima página)

(continuação da página anterior)

```
            name=Name(id='Alias', ctx=Store()),
            value=Name(id='int', ctx=Load()))])
```

Adicionado na versão 3.12.

Outras instruções que são aplicáveis apenas dentro de funções ou laços são descritas em outras seções.

Importações

class ast.**Import**(*names*)

Uma instrução de importação. `names` é uma lista de nós de *`alias`*.

```
>>> print(ast.dump(ast.parse('import x,y,z'), indent=4))
Module(
    body=[
        Import(
            names=[
                alias(name='x'),
                alias(name='y'),
                alias(name='z')])])
```

class ast.**ImportFrom**(*module*, *names*, *level*)

Representa `from x import y`. `module` é uma string bruta do nome 'from', sem quaisquer pontos iniciais, ou `None` para instruções como `from . import foo`. `level` é um número inteiro que contém o nível da importação relativa (0 significa importação absoluta).

```
>>> print(ast.dump(ast.parse('from y import x,y,z'), indent=4))
Module(
    body=[
        ImportFrom(
            module='y',
            names=[
                alias(name='x'),
                alias(name='y'),
                alias(name='z')],
            level=0)])
```

class ast.**alias**(*name*, *asname*)

Ambos os parâmetros são strings brutas dos nomes. `asname` pode ser `None` se o nome normal for usado.

```
>>> print(ast.dump(ast.parse('from ..foo.bar import a as b, c'), indent=4))
Module(
    body=[
        ImportFrom(
            module='foo.bar',
            names=[
                alias(name='a', asname='b'),
                alias(name='c')],
            level=2)])
```

Fluxo de controle

Nota

Cláusulas opcionais como `else` são armazenadas como uma lista vazia se não estiverem presentes.

class ast.**If**(*test*, *body*, *orelse*)

Uma instrução `if`. `test` contém um único nó, como um nó *Compare*. `body` e `orelse` contêm, cada um, uma lista de nós.

As cláusulas `elif` não têm uma representação especial no AST, mas aparecem como nós extras de *If* dentro da seção `orelse` da cláusula anterior.

```
>>> print(ast.dump(ast.parse("""
... if x:
...     ...
... elif y:
...     ...
... else:
...     ...
... """), indent=4))
Module(
    body=[
        If(
            test=Name(id='x', ctx=Load()),
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            orelse=[
                If(
                    test=Name(id='y', ctx=Load()),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))],
                    orelse=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

class ast.**For**(*target*, *iter*, *body*, *orelse*, *type_comment*)

Um laço `for`. `target` contém as variáveis às quais o laço atribui, como um único nó de *Name*, *Tuple*, *List*, *Attribute* ou *Subscript*. `iter` contém o item a ser repetido, novamente como um único nó. `body` e `orelse` contêm listas de nós para executar. Aqueles em `orelse` são executados se o laço terminar normalmente, ao invés de através de uma instrução `break`.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário.

```
>>> print(ast.dump(ast.parse("""
... for x in y:
...     ...
... else:
...     ...
... """), indent=4))
Module(
    body=[
        For(
            target=Name(id='x', ctx=Store()),
            iter=Name(id='y', ctx=Load()),
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            orelse=[
                Expr(
                    value=Constant(value=Ellipsis))])])
```

class ast.**While**(*test*, *body*, *orelse*)

Um laço `while`. `test` contém a condição, como um nó de *Compare*.

```
>>> print(ast.dump(ast.parse("""
... while x:
...     ...
... else:
...     ...
... """), indent=4))
Module(
    body=[
        While(
            test=Name(id='x', ctx=Load()),
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            orelse=[
                Expr(
                    value=Constant(value=Ellipsis))])])
```

class ast.**Break**

class ast.**Continue**

As instruções `break` e `continue`.

```
>>> print(ast.dump(ast.parse("""\
... for a in b:
...     if a > 5:
...         break
...     else:
...         continue
...
... """), indent=4))
Module(
    body=[
        For(
            target=Name(id='a', ctx=Store()),
            iter=Name(id='b', ctx=Load()),
            body=[
                If(
                    test=Compare(
                        left=Name(id='a', ctx=Load()),
                        ops=[
                            Gt()],
                        comparators=[
                            Constant(value=5)]),
                    body=[
                        Break()],
                    orelse=[
                        Continue()])])])
```

class ast.**Try**(*body*, *handlers*, *orelse*, *finalbody*)

Blocos `try`. Todos os atributos são uma lista de nós a serem executados, exceto `handlers`, que é uma lista de nós de *ExceptHandler*.

```
>>> print(ast.dump(ast.parse("""
... try:
...     ...
```

(continua na próxima página)

(continuação da página anterior)

```
... except Exception:
...     ...
... except OtherException as e:
...     ...
... else:
...     ...
... finally:
...     ...
... """), indent=4))
Module(
    body=[
        Try(
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            handlers=[
                ExceptHandler(
                    type=Name(id='Exception', ctx=Load()),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]),
                ExceptHandler(
                    type=Name(id='OtherException', ctx=Load()),
                    name='e',
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])],
            orelse=[
                Expr(
                    value=Constant(value=Ellipsis))],
            finalbody=[
                Expr(
                    value=Constant(value=Ellipsis))])])
```

class ast.**TryStar**(*body*, *handlers*, *orelse*, *finalbody*)

Blocos `try` que são seguidos por cláusulas `except*`. Os atributos são os mesmos de *Try*, mas os nós *ExceptHandler* em `handlers` são interpretados como blocos `except*` em vez de `except`.

```
>>> print(ast.dump(ast.parse("""
... try:
...     ...
... except* Exception:
...     ...
... """), indent=4))
Module(
    body=[
        TryStar(
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            handlers=[
                ExceptHandler(
                    type=Name(id='Exception', ctx=Load()),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.11.

class ast.**ExceptHandler**(*type*, *name*, *body*)

Uma única cláusula `except. type` é o tipo de exceção que irá corresponder, normalmente um nó de *Name* (ou `None` para uma cláusula abrangente `except:`). `name` é uma string bruta para o nome conter a exceção, ou `None` se a cláusula não tiver `as foo`. `body` é uma lista de nós.

```
>>> print(ast.dump(ast.parse("""\
... try:
...     a + 1
... except TypeError:
...     pass
... """), indent=4))
Module(
    body=[
        Try(
            body=[
                Expr(
                    value=BinOp(
                        left=Name(id='a', ctx=Load()),
                        op=Add(),
                        right=Constant(value=1)))],
            handlers=[
                ExceptHandler(
                    type=Name(id='TypeError', ctx=Load()),
                    body=[
                        Pass()])])])
```

class ast.**With**(*items*, *body*, *type_comment*)

Um bloco `with. items` é uma lista de nós *withitem* representando os gerenciadores de contexto, e `body` é o bloco indentado dentro do contexto.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário.

class ast.**withitem**(*context_expr*, *optional_vars*)

Um único gerenciador de contexto em um bloco `with. context_expr` é o gerenciador de contexto, geralmente um nó de *Call*. `optional_vars` é um *Name*, *Tuple* ou *List* para a parte `as foo`, ou `None` se não for usado.

```
>>> print(ast.dump(ast.parse("""\
... with a as b, c as d:
...     something(b, d)
... """), indent=4))
Module(
    body=[
        With(
            items=[
                withitem(
                    context_expr=Name(id='a', ctx=Load()),
                    optional_vars=Name(id='b', ctx=Store())),
                withitem(
                    context_expr=Name(id='c', ctx=Load()),
                    optional_vars=Name(id='d', ctx=Store()))],
            body=[
                Expr(
                    value=Call(
                        func=Name(id='something', ctx=Load()),
```

(continua na próxima página)

(continuação da página anterior)

```
                        args=[
                            Name(id='b', ctx=Load()),
                            Name(id='d', ctx=Load())])])])
```

Correspondência de padrões

class ast.**Match**(*subject*, *cases*)

Uma instrução `match`. `subject` contém o assunto da correspondência (o objeto que está sendo comparado com os casos) e `cases` contém um iterável de nós de *match_case* com os diferentes casos.

Adicionado na versão 3.10.

class ast.**match_case**(*pattern*, *guard*, *body*)

Um padrão de caso único em uma instrução `match`. `pattern` contém o padrão de correspondência com o qual o assunto será comparado. Observe que os nós *AST* produzidos para padrões diferem daqueles produzidos para expressões, mesmo quando compartilham a mesma sintaxe.

O atributo `guard` contém uma expressão que será avaliada se o padrão corresponder ao assunto.

`body` contém uma lista de nós a serem executados se o padrão corresponder e o resultado da avaliação da expressão de guarda for verdadeiro.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [x] if x>0:
...         ...
...     case tuple():
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
            cases=[
                match_case(
                    pattern=MatchSequence(
                        patterns=[
                            MatchAs(name='x')]),
                    guard=Compare(
                        left=Name(id='x', ctx=Load()),
                        ops=[
                            Gt()],
                        comparators=[
                            Constant(value=0)]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]),
                match_case(
                    pattern=MatchClass(
                        cls=Name(id='tuple', ctx=Load())),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

class ast.**MatchValue**(*value*)

Um literal de correspondência ou padrão de valor que é comparado por igualdade. `value` é um nó de expressão.

Os nós de valor permitido são restritos conforme descrito na documentação da instrução de correspondência. Este padrão será bem-sucedido se o assunto da correspondência for igual ao valor avaliado.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case "Relevant":
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
            cases=[
                match_case(
                    pattern=MatchValue(
                        value=Constant(value='Relevant')),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

class ast.**MatchSingleton**(*value*)

Um padrão literal de correspondência que compara por identidade. `value` é o singleton a ser comparado com: `None`, `True` ou `False`. Este padrão será bem-sucedido se o assunto da correspondência for a constante fornecida.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case None:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
            cases=[
                match_case(
                    pattern=MatchSingleton(value=None),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

class ast.**MatchSequence**(*patterns*)

Um padrão de sequência de correspondência. `patterns` contém os padrões a serem comparados aos elementos do assunto se o assunto for uma sequência. Corresponde a uma sequência de comprimento variável se um dos subpadrões for um nó `MatchStar`, caso contrário corresponde a uma sequência de comprimento fixo.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [1, 2]:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
```

(continua na próxima página)

(continuação da página anterior)

```
            cases=[
                match_case(
                    pattern=MatchSequence(
                        patterns=[
                            MatchValue(
                                value=Constant(value=1)),
                            MatchValue(
                                value=Constant(value=2))]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

class ast.**MatchStar**(*name*)

Corresponde ao restante da sequência em um padrão de sequência de correspondência de comprimento variável. Se `name` não for `None`, uma lista contendo os elementos restantes da sequência será vinculada a esse nome se o padrão de sequência geral for bem-sucedido.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [1, 2, *rest]:
...         ...
...     case [*_]:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
            cases=[
                match_case(
                    pattern=MatchSequence(
                        patterns=[
                            MatchValue(
                                value=Constant(value=1)),
                            MatchValue(
                                value=Constant(value=2)),
                            MatchStar(name='rest')]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]),
                match_case(
                    pattern=MatchSequence(
                        patterns=[
                            MatchStar()]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

class ast.**MatchMapping**(*keys*, *patterns*, *rest*)

Um padrão de mapeamento de correspondência. `keys` é uma sequência de nós de expressão. `patterns` é uma sequência correspondente de nós padrão. `rest` é um nome opcional que pode ser especificado para capturar os elementos restantes do mapeamento. As expressões-chave permitidas são restritas conforme descrito na documentação da instrução match.

Este padrão será bem-sucedido se o assunto for um mapeamento, todas as expressões-chave avaliadas estiverem presentes no mapeamento e o valor correspondente a cada chave corresponder ao subpadrão correspondente. Se `rest` não for `None`, um dict contendo os elementos de mapeamento restantes será vinculado a esse nome se o padrão de mapeamento geral for bem-sucedido.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case {1: _, 2: _}:
...         ...
...     case {**rest}:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
            cases=[
                match_case(
                    pattern=MatchMapping(
                        keys=[
                            Constant(value=1),
                            Constant(value=2)],
                        patterns=[
                            MatchAs(),
                            MatchAs()]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]),
                match_case(
                    pattern=MatchMapping(rest='rest'),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

class `ast.`**MatchClass**(*cls*, *patterns*, *kwd_attrs*, *kwd_patterns*)

Um padrão de classe de correspondência. `cls` é uma expressão que fornece a classe nominal a ser correspondida. `patterns` é uma sequência de nós padrão a serem comparados com a sequência definida pela classe de atributos de correspondência de padrões. `kwd_attrs` é uma sequência de atributos adicionais a serem correspondidos (especificados como argumentos nomeados no padrão de classe), `kwd_patterns` são os padrões correspondentes (especificados como valores de argumentos nomeados no padrão de classe).

Esse padrão será bem-sucedido se o assunto for uma instância da classe indicada, todos os padrões posicionais corresponderem aos atributos correspondentes definidos pela classe e quaisquer atributos, passados como argumentos nomeados, especificados corresponderem ao seu padrão correspondente.

Nota: as classes podem definir uma propriedade que retorna self para corresponder um nó padrão à instância que está sendo correspondida. Vários tipos internos também são combinados dessa forma, conforme descrito na documentação da instrução match.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case Point2D(0, 0):
...         ...
...     case Point3D(x=0, y=0, z=0):
...         ...
... """), indent=4))
Module(
```

(continua na próxima página)

(continuação da página anterior)

```
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
            cases=[
                match_case(
                    pattern=MatchClass(
                        cls=Name(id='Point2D', ctx=Load()),
                        patterns=[
                            MatchValue(
                                value=Constant(value=0)),
                            MatchValue(
                                value=Constant(value=0))]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]),
                match_case(
                    pattern=MatchClass(
                        cls=Name(id='Point3D', ctx=Load()),
                        kwd_attrs=[
                            'x',
                            'y',
                            'z'],
                        kwd_patterns=[
                            MatchValue(
                                value=Constant(value=0)),
                            MatchValue(
                                value=Constant(value=0)),
                            MatchValue(
                                value=Constant(value=0))]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

class ast.**MatchAs**(*pattern*, *name*)

Uma correspondência "como padrão", padrão de captura ou padrão curinga. `pattern` contém o padrão de correspondência com o qual o assunto será comparado. Se o padrão for `None`, o nó representa um padrão de captura (ou seja, um nome simples) e sempre terá sucesso.

O atributo `name` contém o nome que será vinculado se o padrão for bem-sucedido. Se `name` for `None`, `pattern` também deverá ser `None` e o nó representa o padrão curinga.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [x] as y:
...         ...
...     case _:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
            cases=[
                match_case(
                    pattern=MatchAs(
```

(continua na próxima página)

(continuação da página anterior)

```
                        pattern=MatchSequence(
                            patterns=[
                                MatchAs(name='x')]),
                        name='y'),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]),
                match_case(
                    pattern=MatchAs(),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

class ast.**MatchOr**(*patterns*)

Uma correspondência de "padrão ou". Um "padrão ou" corresponde cada um de seus subpadrões com o assunto, até que um seja bem-sucedido. O "padrão ou" é então considerado bem-sucedido. Se nenhum dos subpadrões for bem-sucedido, o padrão or falhará. O atributo `patterns` contém uma lista de nós de padrões de correspondência que serão comparados com o assunto.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [x] | (y):
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x', ctx=Load()),
            cases=[
                match_case(
                    pattern=MatchOr(
                        patterns=[
                            MatchSequence(
                                patterns=[
                                    MatchAs(name='x')]),
                            MatchAs(name='y')]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])])])
```

Adicionado na versão 3.10.

Anotações de tipos

class ast.**TypeIgnore**(*lineno*, *tag*)

Um comentário `# type: ignore` localizado em *lineno*. *tag* é a tag opcional especificada pelo formato `# type: ignore <tag>`.

```
>>> print(ast.dump(ast.parse('x = 1 # type: ignore', type_comments=True),␣
↪indent=4))
Module(
    body=[
        Assign(
            targets=[
                Name(id='x', ctx=Store())],
```

(continua na próxima página)

(continuação da página anterior)

```
            value=Constant(value=1))],
    type_ignores=[
        TypeIgnore(lineno=1, tag='')])
>>> print(ast.dump(ast.parse('x: bool = 1 # type: ignore[assignment]', type_
↪comments=True), indent=4))
Module(
    body=[
        AnnAssign(
            target=Name(id='x', ctx=Store()),
            annotation=Name(id='bool', ctx=Load()),
            value=Constant(value=1),
            simple=1)],
    type_ignores=[
        TypeIgnore(lineno=1, tag='[assignment]')])
```

Nota

Os nós `TypeIgnore` não são gerados quando o parâmetro *type_comments* está definido como `False` (padrão). Consulte *`ast.parse()`* para obter mais detalhes.

Adicionado na versão 3.8.

Parâmetros de tipo

Parâmetros de tipo podem existir em classes, funções e apelidos de tipo.

class `ast.`**TypeVar**(*name*, *bound*, *default_value*)

Uma *`typing.TypeVar`*. `name` é o nome da variável de tipo. `bound` é o limite ou as restrições, se houver. Se `bound` for uma *`Tuple`*, ele representa restrições; caso contrário, representa o limite. `default_value` é o valor padrão; se `TypeVar` não tiver padrão, este atributo será definido como `None`.

```
>>> print(ast.dump(ast.parse("type Alias[T: int = bool] = list[T]"), indent=4))
Module(
    body=[
        TypeAlias(
            name=Name(id='Alias', ctx=Store()),
            type_params=[
                TypeVar(
                    name='T',
                    bound=Name(id='int', ctx=Load()),
                    default_value=Name(id='bool', ctx=Load()))],
            value=Subscript(
                value=Name(id='list', ctx=Load()),
                slice=Name(id='T', ctx=Load()),
                ctx=Load()))])
```

Adicionado na versão 3.12.

Alterado na versão 3.13: Adicionado o parâmetro *default_value*.

class `ast.`**ParamSpec**(*name*, *default_value*)

Uma *`typing.ParamSpec`*. `name` é o nome da especificação do parâmetro. `default_value` é o valor padrão; se `ParamSpec` não tiver padrão, este atributo será definido como `None`.

```
>>> print(ast.dump(ast.parse("type Alias[**P = [int, str]] = Callable[P, int]
↪"), indent=4))
```

(continua na próxima página)

(continuação da página anterior)

```
Module(
    body=[
        TypeAlias(
            name=Name(id='Alias', ctx=Store()),
            type_params=[
                ParamSpec(
                    name='P',
                    default_value=List(
                        elts=[
                            Name(id='int', ctx=Load()),
                            Name(id='str', ctx=Load())],
                        ctx=Load()))],
            value=Subscript(
                value=Name(id='Callable', ctx=Load()),
                slice=Tuple(
                    elts=[
                        Name(id='P', ctx=Load()),
                        Name(id='int', ctx=Load())],
                    ctx=Load()),
                ctx=Load()))])
```

Adicionado na versão 3.12.

Alterado na versão 3.13: Adicionado o parâmetro *default_value*.

class ast.**TypeVarTuple**(*name*, *default_value*)

Uma *typing.TypeVarTuple*. `name` é o nome da tupla da variável de tipo. `default_value` é o valor padrão; se `TypeVarTuple` não tiver padrão, este atributo será definido como `None`.

```
>>> print(ast.dump(ast.parse("type Alias[*Ts = ()] = tuple[*Ts]"), indent=4))
Module(
    body=[
        TypeAlias(
            name=Name(id='Alias', ctx=Store()),
            type_params=[
                TypeVarTuple(
                    name='Ts',
                    default_value=Tuple(ctx=Load()))],
            value=Subscript(
                value=Name(id='tuple', ctx=Load()),
                slice=Tuple(
                    elts=[
                        Starred(
                            value=Name(id='Ts', ctx=Load()),
                            ctx=Load())],
                    ctx=Load()),
                ctx=Load()))])
```

Adicionado na versão 3.12.

Alterado na versão 3.13: Adicionado o parâmetro *default_value*.

Definições de função e classe

class ast.**FunctionDef**(*name*, *args*, *body*, *decorator_list*, *returns*, *type_comment*, *type_params*)

Uma definição de função

- `name` é uma string bruta do nome da função.

- `args` é um nó *`arguments`*.
- `body` é a lista de nós dentro da função.
- `decorator_list` é a lista de decoradores a serem aplicados, armazenados primeiro na parte externa (ou seja, o primeiro da lista será aplicado por último).
- `returns` é a anotação de retorno.
- `type_params` é uma lista de *parâmetros de tipo*.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário.

Alterado na versão 3.12: Adicionado `type_params`.

class `ast.`**Lambda**(*args*, *body*)

`lambda` é uma definição mínima de função que pode ser usada dentro de uma expressão. Ao contrário de *`FunctionDef`*, `body` contém um único nó.

```
>>> print(ast.dump(ast.parse('lambda x,y: ...'), indent=4))
Module(
    body=[
        Expr(
            value=Lambda(
                args=arguments(
                    args=[
                        arg(arg='x'),
                        arg(arg='y')]),
                body=Constant(value=Ellipsis)))])
```

class `ast.`**arguments**(*posonlyargs*, *args*, *vararg*, *kwonlyargs*, *kw_defaults*, *kwarg*, *defaults*)

Os argumentos para uma função.

- `posonlyargs`, `args` e `kwonlyargs` são listas de nós *`arg`*.
- `vararg` e `kwarg` são nós *`arg`* únicos, referindo-se aos parâmetros `*args, **kwargs`.
- `kw_defaults` é uma lista de valores padrão para argumentos somente-nomeados. Se um for `None`, o argumento correspondente é necessário.
- `defaults` é uma lista de valores padrão para argumentos que podem ser passados posicionalmente. Se houver menos padrões, eles corresponderão aos últimos n argumentos.

class `ast.`**arg**(*arg*, *annotation*, *type_comment*)

Um único argumento em uma lista. `arg` é uma string bruta do nome do argumento; `annotation` é sua anotação, como um nó *`Name`*.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário

```
>>> print(ast.dump(ast.parse("""\
... @decorator1
... @decorator2
... def f(a: 'annotation', b=1, c=2, *d, e, f=3, **g) -> 'return annotation':
...     pass
... """), indent=4))
Module(
    body=[
        FunctionDef(
            name='f',
            args=arguments(
                args=[
```

(continua na próxima página)

(continuação da página anterior)

```
                    arg(
                        arg='a',
                        annotation=Constant(value='annotation')),
                    arg(arg='b'),
                    arg(arg='c')],
                vararg=arg(arg='d'),
                kwonlyargs=[
                    arg(arg='e'),
                    arg(arg='f')],
                kw_defaults=[
                    None,
                    Constant(value=3)],
                kwarg=arg(arg='g'),
                defaults=[
                    Constant(value=1),
                    Constant(value=2)]),
            body=[
                Pass()],
            decorator_list=[
                Name(id='decorator1', ctx=Load()),
                Name(id='decorator2', ctx=Load())],
            returns=Constant(value='return annotation'))])
```

class ast.**Return**(*value*)

Uma instrução `return`.

```
>>> print(ast.dump(ast.parse('return 4'), indent=4))
Module(
    body=[
        Return(
            value=Constant(value=4))])
```

class ast.**Yield**(*value*)

class ast.**YieldFrom**(*value*)

Uma expressão `yield` ou `yield from`. Por serem expressões, elas devem ser agrupadas em um nó *Expr* se o valor enviado de volta não for usado.

```
>>> print(ast.dump(ast.parse('yield x'), indent=4))
Module(
    body=[
        Expr(
            value=Yield(
                value=Name(id='x', ctx=Load()))))])

>>> print(ast.dump(ast.parse('yield from x'), indent=4))
Module(
    body=[
        Expr(
            value=YieldFrom(
                value=Name(id='x', ctx=Load()))))])
```

class ast.**Global**(*names*)

class ast.**Nonlocal**(*names*)

Instruções `global` e `nonlocal`. `names` é uma lista de strings brutas.

```
>>> print(ast.dump(ast.parse('global x,y,z'), indent=4))
Module(
    body=[
        Global(
            names=[
                'x',
                'y',
                'z'])])

>>> print(ast.dump(ast.parse('nonlocal x,y,z'), indent=4))
Module(
    body=[
        Nonlocal(
            names=[
                'x',
                'y',
                'z'])])
```

class ast.**ClassDef**(*name*, *bases*, *keywords*, *body*, *decorator_list*, *type_params*)

Uma definição de classe

- `name` é uma string bruta para o nome da classe
- `bases` é uma lista de nós para classes base especificadas explicitamente.
- `keywords` é uma lista de nós *`keyword`*, principalmente para 'metaclass'. Outras argumentos nomeados serão passadas para a metaclasse, conforme a **PEP 3115**.
- `body` é uma lista de nós que representam o código dentro da definição de classe.
- `decorator_list` é uma lista de nós, como em *`FunctionDef`*.
- `type_params` é uma lista de *parâmetros de tipo*.

```
>>> print(ast.dump(ast.parse("""\
... @decorator1
... @decorator2
... class Foo(base1, base2, metaclass=meta):
...     pass
... """), indent=4))
Module(
    body=[
        ClassDef(
            name='Foo',
            bases=[
                Name(id='base1', ctx=Load()),
                Name(id='base2', ctx=Load())],
            keywords=[
                keyword(
                    arg='metaclass',
                    value=Name(id='meta', ctx=Load()))],
            body=[
                Pass()],
            decorator_list=[
                Name(id='decorator1', ctx=Load()),
                Name(id='decorator2', ctx=Load())])])
```

Alterado na versão 3.12: Adicionado `type_params`.

Async e await

class ast.**AsyncFunctionDef**(*name*, *args*, *body*, *decorator_list*, *returns*, *type_comment*, *type_params*)

Uma definição de função `async def`. Possui os mesmos campos que *FunctionDef*.

Alterado na versão 3.12: Adicionado `type_params`.

class ast.**Await**(*value*)

Uma expressão `await`. `value` é o que ela espera. Válido apenas no corpo de um *AsyncFunctionDef*.

```
>>> print(ast.dump(ast.parse("""\
... async def f():
...     await other_func()
... """), indent=4))
Module(
    body=[
        AsyncFunctionDef(
            name='f',
            args=arguments(),
            body=[
                Expr(
                    value=Await(
                        value=Call(
                            func=Name(id='other_func', ctx=Load())))])])
```

class ast.**AsyncFor**(*target*, *iter*, *body*, *orelse*, *type_comment*)

class ast.**AsyncWith**(*items*, *body*, *type_comment*)

Laços `async for` e gerenciadores de contexto `async with`. Eles têm os mesmos campos que *For* e *With*, respectivamente. Válido apenas no corpo de *AsyncFunctionDef*.

Nota

Quando uma string é analisada por *ast.parse()*, os nós operadores (subclasses de `ast.operator`, `ast.unaryop`, `ast.cmpop`, `ast.boolop` e `ast.expr_context`) na árvore retornada serão singletons. As alterações em um serão refletidas em todas as outras ocorrências do mesmo valor (por exemplo, *ast.Add*).

32.1.3 Auxiliares de `ast`

Além das classes de nós, o módulo *ast* define essas funções e classes utilitárias para percorrer árvores de sintaxe abstrata:

ast.**parse**(*source*, *filename='<unknown>'*, *mode='exec'*, *, *type_comments=False*, *feature_version=None*, *optimize=-1*)

Analisa a fonte como um nó AST. Equivalente a `compile(source, filename, mode, flags=FLAGS_VALUE, optimize=optimize)`, onde `FLAGS_VALUE` é `ast.PyCF_ONLY_AST` se `optimize <= 0` e `ast.PyCF_OPTIMIZED_AST` caso contrário.

Se `type_comments=True` é fornecido, o analisador é modificado para verificar e retornar comentários do tipo, conforme especificado por **PEP 484** e **PEP 526**. Isso é equivalente a adicionar *ast.PyCF_TYPE_COMMENTS* aos sinalizadores passados para *compile()*. Isso relatará erros de sintaxe para comentários do tipo extraviado. Sem esse sinalizador, os comentários do tipo serão ignorados e o campo `type_comment` nos nós AST selecionados sempre será `None`. Além disso, os locais dos comentários `# type: ignore` serão retornados como o atributo `type_ignores` de *Module* (caso contrário, é sempre uma lista vazia).

Além disso, se `mode` for `'func_type'`, a sintaxe de entrada é modificada para corresponder a "comentários de tipo de assinatura" de **PEP 484**, por exemplo, `(str, int) -> List[str]`.

Definir `feature_version` como uma tupla `(major, minor)` resultará em uma tentativa de "melhor esforço" de análise usando a gramática daquela versão do Python. Por exemplo, definir

`feature_version=(3, 9)` tentará proibir a análise de instruções `match`. Atualmente `major` deve ser igual a `3`. A versão mais baixa suportada é `(3, 7)` (e isso pode aumentar em versões futuras do Python); o mais alto é `sys.version_info[0:2]`. A tentativa de "melhor esforço" significa que não há garantia de que a análise (ou sucesso da análise) seja a mesma que quando executada na versão Python correspondente a `feature_version`.

Se a fonte contém um caractere nulo (`\0`), *`ValueError`* é levantada.

> **Aviso**
>
> Observe que a análise bem-sucedida do código-fonte em um objeto AST não garante que o código-fonte fornecido seja um código Python válido que pode ser executado, pois a etapa de compilação pode levantar mais exceções *`SyntaxError`*. Por exemplo, a fonte `return 42` gera um nó AST válido para uma instrução return, mas não pode ser compilado sozinho (precisa estar dentro de um nó de função).
>
> Em particular, *`ast.parse()`* não fará nenhuma verificação de escopo, o que a etapa de compilação faz.

> **Aviso**
>
> É possível travar o interpretador Python com uma string suficientemente grande/complexa devido às limitações de profundidade da pilha no compilador de AST do Python.

Alterado na versão 3.8: Adicionado `type_comments`, `mode='func_type'` e `feature_version`.

Alterado na versão 3.13: A versão mínima suportada para `feature_version` agora é `(3, 7)`. O argumento `optimize` foi adicionado.

`ast.`**`unparse`**(*ast_obj*)

Desfaz análise de um objeto *`ast.AST`* e gera uma string com código que produziria um objeto *`ast.AST`* equivalente se analisado novamente com *`ast.parse()`*.

> **Aviso**
>
> A string de código produzida não será necessariamente igual ao código original que gerou o objeto *`ast.AST`* (sem quaisquer otimizações do compilador, como tuplas/frozensets constantes).

> **Aviso**
>
> Tentar desfazer análise de uma expressão altamente complexa resultaria em *`RecursionError`*.

Adicionado na versão 3.9.

`ast.`**`literal_eval`**(*node_or_string*)

Avalia um nó de expressão ou uma string contendo apenas um literal Python ou uma exibição de contêiner. A string ou nó fornecido pode consistir apenas nas seguintes estruturas literais Python: strings, bytes, números, tuplas, listas, dicionários, conjuntos, booleanos, `None` e `Ellipsis`.

Isso pode ser usado para avaliar strings contendo valores Python sem a necessidade de analisar os valores por conta própria. Não é capaz de avaliar expressões arbitrariamente complexas, por exemplo, envolvendo operadores ou indexação.

Esta função foi documentada como "segura" no passado sem definir o que isso significava. Isso foi enganoso. Isso foi projetado especificamente para não executar código Python, ao contrário do *`eval()`* mais geral. Não há espaço de nomes, pesquisas de nome ou capacidade de chamada. Mas não está livre de ataques: uma entrada relativamente pequena pode levar ao esgotamento da memória ou ao esgotamento da pilha C, travando

o processo. Também existe a possibilidade de negação de serviço por consumo excessivo de CPU em algumas entradas. Portanto, não é recomendado chamá-la em dados não confiáveis.

> **Aviso**
>
> É possível travar o interpretador Python devido às limitações de profundidade da pilha no compilador AST do Python.
>
> Pode levantar *ValueError*, *TypeError*, *SyntaxError*, *MemoryError* e *RecursionError* dependendo da entrada malformada.

Alterado na versão 3.2: Agora permite bytes e literais de conjuntos.

Alterado na versão 3.9: Agora oferece suporte à criação de conjuntos vazios com `'set()'`.

Alterado na versão 3.10: Para entradas de string, os espaços iniciais e tabulações agora são removidos.

`ast.`**`get_docstring`**(*node*, *clean=True*)

Retorna a docstring do *node* dado (que deve ser um nó *FunctionDef*, *AsyncFunctionDef*, *ClassDef* ou *Module*) ou `None` se não tiver uma docstring. Se *clean* for verdadeiro, limpa o recuo da docstring com *inspect.cleandoc()*.

Alterado na versão 3.5: Não há suporte a *AsyncFunctionDef*.

`ast.`**`get_source_segment`**(*source*, *node*, ***, *padded=False*)

Obtém o segmento de código-fonte do *source* que gerou o *node*. Se alguma informação de localização (*lineno*, *end_lineno*, *col_offset* ou *end_col_offset*) está faltando, retorna `None`.

Se *padded* for `True`, a primeira linha de uma instrução multilinha será preenchida com espaços para corresponder à sua posição original.

Adicionado na versão 3.8.

`ast.`**`fix_missing_locations`**(*node*)

Quando você compila uma árvore de nós com *compile()*, o compilador espera atributos *lineno* e *col_offset* para cada nó que os suporta. Isso é tedioso para preencher nós gerados, portanto, esse auxiliar adiciona esses atributos recursivamente, onde ainda não estão definidos, definindo-os para os valores do nó pai. Ele funciona recursivamente a partir do *node*.

`ast.`**`increment_lineno`**(*node*, *n=1*)

Incrementa o número da linhas e o número da linha final de cada nó na árvore começando em *node* em *n*. Isso é útil para "mover código" para um local diferente em um arquivo.

`ast.`**`copy_location`**(*new_node*, *old_node*)

Copia o local de origem (*lineno*, *col_offset*, *end_lineno* e *end_col_offset*) de *old_node* para *new_node* se possível e, então, retorna *new_node*.

`ast.`**`iter_fields`**(*node*)

Produz uma tupla de `(fieldname, value)` para cada campo em `node._fields` que esteja presente em *node*.

`ast.`**`iter_child_nodes`**(*node*)

Produz todos os nós filhos diretos de *node*, ou seja, todos os campos que são nós e todos os itens de campos que são listas de nós.

`ast.`**`walk`**(*node*)

Produz recursivamente todos os nós descendentes na árvore começando em *node* (incluindo o próprio *node*), em nenhuma ordem especificada. Isso é útil se você quiser apenas modificar nós no lugar e não se importar com o contexto.

class ast.**NodeVisitor**

Uma classe base de visitante de nó que percorre a árvore de sintaxe abstrata e chama uma função de visitante para cada nó encontrado. Esta função pode retornar um valor que é encaminhado pelo método *visit()*.

Esta classe deve ser uma subclasse, com a subclasse adicionando métodos visitantes.

visit(*node*)

Visita um nó. A implementação padrão chama o método chamado `self.visit_`*nomedaclasse* sendo *nomedaclasse* o nome da classe do nó, ou *generic_visit()* se aquele método não existir.

generic_visit(*node*)

Este visitante chama *visit()* em todos os filhos do nó.

Observe que nós filhos de nós que possuem um método de visitante personalizado não serão visitados, a menos que o visitante chame *generic_visit()* ou os visite por conta própria.

visit_Constant(*node*)

Manipula todos os nós constantes.

Não use o *NodeVisitor* se você quiser aplicar mudanças nos nós durante a travessia. Para isso existe um visitante especial (*NodeTransformer*) que permite modificações.

Descontinuado desde a versão 3.8: Os métodos `visit_Num()`, `visit_Str()`, `visit_Bytes()`, `visit_NameConstant()` e `visit_Ellipsis()` estão agora descontinuados e não serão chamados em futuras versões do Python. Adicione um método *visit_Constant()* para lidar com nós de constantes.

class ast.**NodeTransformer**

A subclasse *NodeVisitor* que percorre a árvore de sintaxe abstrata e permite a modificação de nós.

O *NodeTransformer* percorrerá a AST e usará o valor de retorno dos métodos do visitante para substituir ou remover o nó antigo. Se o valor de retorno do método visitante for `None`, o nó será removido de seu local, caso contrário, ele será substituído pelo valor de retorno. O valor de retorno pode ser o nó original, caso em que não há substituição.

Aqui está um exemplo de transformador que rescreve todas as ocorrências de procuras por nome (`foo`) para `data['foo']`:

```
class RewriteName(NodeTransformer):

    def visit_Name(self, node):
        return Subscript(
            value=Name(id='data', ctx=Load()),
            slice=Constant(value=node.id),
            ctx=node.ctx
        )
```

Tenha em mente que, se o nó em que você está operando tiver nós filhos, você deve transformar os nós filhos por conta própria ou chamar o método *generic_visit()* para o nó primeiro.

Para nós que faziam parte de uma coleção de instruções (que se aplica a todos os nós de instrução), o visitante também pode retornar uma lista de nós em vez de apenas um único nó.

Se *NodeTransformer* introduz novos nós (que não faziam parte da árvore original) sem fornecer informações de localização (como *lineno*), *fix_missing_locations()* deve ser chamado com o novo subárvore para recalcular as informações de localização:

```
tree = ast.parse('foo', mode='eval')
new_tree = fix_missing_locations(RewriteName().visit(tree))
```

Normalmente você usa o transformador assim:

```
node = YourTransformer().visit(node)
```

ast.**dump**(*node*, *annotate_fields=True*, *include_attributes=False*, *, *indent=None*, *show_empty=False*)

Retorne um despejo formatado da árvore em *node*. Isso é útil principalmente para fins de depuração. Se *annotate_fields* for verdadeiro (por padrão), a sequência retornada mostrará os nomes e os valores para os campos. Se *annotate_fields* for falso, a sequência de resultados será mais compacta ao omitir nomes de campos não ambíguos. Atributos como números de linha e deslocamentos de coluna não são despejados por padrão. Se isso for desejado, *include_attributes* pode ser definido como verdadeiro.

Se *indent* for um inteiro não negativo ou uma string, então a árvore terá uma saída formatada com este nível de indentação. Um nível de indentação 0, negativo ou `""` apenas colocará novas linhas. `None` (o padrão) seleciona a representação de uma única linha. Usando um inteiro positivo a indentação terá alguns espaços por nível. Se *indent* for uma string (como `"\t"`), essa string será usada para indentar cada nível.

Se *show_empty* for falso (o padrão), listas vazias opcionais serão omitidas da saída. Valores opcionais `None` são sempre omitidos.

Alterado na versão 3.9: Adicionada a opção *indent*.

Alterado na versão 3.13: Adicionada a opção *show_empty*.

```
>>> print(ast.dump(ast.parse("""\
... async def f():
...     await other_func()
... """), indent=4, show_empty=True))
Module(
    body=[
        AsyncFunctionDef(
            name='f',
            args=arguments(
                posonlyargs=[],
                args=[],
                kwonlyargs=[],
                kw_defaults=[],
                defaults=[]),
            body=[
                Expr(
                    value=Await(
                        value=Call(
                            func=Name(id='other_func', ctx=Load()),
                            args=[],
                            keywords=[]))))],
            decorator_list=[],
            type_params=[])],
    type_ignores=[])
```

32.1.4 Sinalizadores do compilador

Os seguintes sinalizadores podem ser passados para *compile()* para alterar os efeitos na compilação de um programa:

ast.**PyCF_ALLOW_TOP_LEVEL_AWAIT**

Habilita suporte para `await`, `async for`, `async with` e compreensões assíncronas de nível superior.

Adicionado na versão 3.8.

ast.**PyCF_ONLY_AST**

Gera e retorna uma árvore de sintaxe abstrata em vez de retornar um objeto de código compilado.

ast.**PyCF_OPTIMIZED_AST**

O AST retornado é otimizado de acordo com o argumento *optimize* em *compile()* ou *ast.parse()*.

Adicionado na versão 3.13.

ast.**PyCF_TYPE_COMMENTS**

Habilita suporte para comentários do tipo **PEP 484** e **PEP 526** (`# type: <type>`, `# type: ignore <stuff>`).

Adicionado na versão 3.8.

ast.**compare**(*a*, *b*, */*, ***, *compare_attributes=False*)

Compara recursivamente duas ASTs.

compare_attributes afeta se os atributos AST são considerados na comparação. Se *compare_attributes* for `False` (padrão), os atributos serão ignorados. Caso contrário, todos devem ser iguais. Esta opção é útil para verificar se os ASTs são estruturalmente iguais, mas diferem em espaços em branco ou detalhes semelhantes. Os atributos incluem números de linha e deslocamentos de coluna.

Adicionado na versão 3.14.

32.1.5 Uso na linha de comando

Adicionado na versão 3.9.

O módulo *ast* pode ser executado como um script na linha de comando. É tão simples quanto:

```
python -m ast [-m <mode>] [-a] [infile]
```

As seguintes opções são aceitas:

-h, --help

Mostra a mensagem de ajuda e sai.

-m <mode>

--mode <mode>

Especifica que tipo de código deve ser compilado, como o argumento *mode* em *parse()*.

--no-type-comments

Não analisa comentários de tipo.

-a, --include-attributes

Inclui atributos como números de linha e deslocamentos de colunas.

-i <indent>

--indent <indent>

indentação de nós em AST (número de espaços).

--feature-version <version>

Versão do Python no formato 3.x (por exemplo, 3.10). O padrão é a versão atual do interpretador.

Adicionado na versão 3.14.

-O <level>

--optimize <level>

Nível de otimização do analisador. O padrão é sem otimização.

Adicionado na versão 3.14.

--show-empty

Exibe listas e campos vazios com o valor `None`. O padrão é não exibir objetos vazios.

Adicionado na versão 3.14.

Se `infile` for especificado, seu conteúdo será analisado no AST e despejado no stdout. Caso contrário, o conteúdo será lido em stdin.

Ver também

Green Tree Snakes, um recurso de documentação externo, possui bons detalhes sobre trabalhar com ASTs do Python.

ASTTokens anota ASTs do Python com as posições de tokens e texto no código-fonte que as gerou. Isso é útil para ferramentas que fazem transformações de código-fonte.

leoAst.py unifica as visualizações baseadas em token e em árvore de análise de programas python, inserindo links duas vias entre tokens e nós de ast.

LibCST analisa o código como uma árvore de sintaxe concreta que se parece com uma árvore ast e mantém todos os detalhes de formatação. É útil para construir linters e aplicações de refatoração automatizada (codemod).

Parso é um analisador Python que oferece suporte a recuperação de erros e análise de ida e volta para diferentes versões do Python (em várias versões do Python). Parso também é capaz de listar vários erros de sintaxe em seu arquivo Python.

32.2 `symtable` — Acesso a tabela de símbolos do compilador

Código-fonte: Lib/symtable.py

Symbol tables are generated by the compiler from AST just before bytecode is generated. The symbol table is responsible for calculating the scope of every identifier in the code. *`symtable`* provides an interface to examine these tables.

32.2.1 Generating Symbol Tables

`symtable.`**`symtable`**(*code*, *filename*, *compile_type*)

Return the toplevel *`SymbolTable`* for the Python source *code*. *filename* is the name of the file containing the code. *compile_type* is like the *mode* argument to *`compile()`*.

32.2.2 Examining Symbol Tables

class `symtable.`**`SymbolTableType`**

An enumeration indicating the type of a *`SymbolTable`* object.

`MODULE = "module"`

Used for the symbol table of a module.

`FUNCTION = "function"`

Used for the symbol table of a function.

`CLASS = "class"`

Used for the symbol table of a class.

The following members refer to different flavors of annotation scopes.

`ANNOTATION = "annotation"`

Used for annotations if `from __future__ import annotations` is active.

`TYPE_ALIAS = "type alias"`

Used for the symbol table of `type` constructions.

`TYPE_PARAMETERS = "type parameters"`

Used for the symbol table of generic functions or generic classes.

TYPE_VARIABLE = "type variable"

Used for the symbol table of the bound, the constraint tuple or the default value of a single type variable in the formal sense, i.e., a TypeVar, a TypeVarTuple or a ParamSpec object (the latter two do not support a bound or a constraint tuple).

Adicionado na versão 3.13.

class symtable.**SymbolTable**

A namespace table for a block. The constructor is not public.

get_type()

Return the type of the symbol table. Possible values are members of the *SymbolTableType* enumeration.

Alterado na versão 3.12: Added `'annotation'`, `'TypeVar bound'`, `'type alias'`, and `'type parameter'` as possible return values.

Alterado na versão 3.13: Return values are members of the *SymbolTableType* enumeration.

The exact values of the returned string may change in the future, and thus, it is recommended to use *SymbolTableType* members instead of hard-coded strings.

get_id()

Return the table's identifier.

get_name()

Return the table's name. This is the name of the class if the table is for a class, the name of the function if the table is for a function, or `'top'` if the table is global (*get_type()* returns `'module'`). For type parameter scopes (which are used for generic classes, functions, and type aliases), it is the name of the underlying class, function, or type alias. For type alias scopes, it is the name of the type alias. For *TypeVar* bound scopes, it is the name of the `TypeVar`.

get_lineno()

Return the number of the first line in the block this table represents.

is_optimized()

Return `True` if the locals in this table can be optimized.

is_nested()

Return `True` if the block is a nested class or function.

has_children()

Return `True` if the block has nested namespaces within it. These can be obtained with *get_children()*.

get_identifiers()

Return a view object containing the names of symbols in the table. See the *documentation of view objects*.

lookup(*name*)

Lookup *name* in the table and return a *Symbol* instance.

get_symbols()

Return a list of *Symbol* instances for names in the table.

get_children()

Return a list of the nested symbol tables.

class symtable.**Function**

A namespace for a function or method. This class inherits from *SymbolTable*.

get_parameters()

Return a tuple containing names of parameters to this function.

get_locals()

Return a tuple containing names of locals in this function.

get_globals()

Return a tuple containing names of globals in this function.

get_nonlocals()

Return a tuple containing names of explicitly declared nonlocals in this function.

get_frees()

Return a tuple containing names of *free (closure) variables* in this function.

class symtable.**Class**

A namespace of a class. This class inherits from *SymbolTable*.

get_methods()

Return a tuple containing the names of method-like functions declared in the class.

Here, the term 'method' designates *any* function defined in the class body via `def` or `async def`.

Functions defined in a deeper scope (e.g., in an inner class) are not picked up by *get_methods()*.

Por exemplo:

```
>>> import symtable
>>> st = symtable.symtable('''
... def outer(): pass
...
... class A:
...     def f():
...         def w(): pass
...
...     def g(self): pass
...
...     @classmethod
...     async def h(cls): pass
...
...     global outer
...     def outer(self): pass
... ''', 'test', 'exec')
>>> class_A = st.get_children()[2]
>>> class_A.get_methods()
('f', 'g', 'h')
```

Although `A().f()` raises *TypeError* at runtime, `A.f` is still considered as a method-like function.

Deprecated since version 3.14, will be removed in version 3.16.

class symtable.**Symbol**

An entry in a *SymbolTable* corresponding to an identifier in the source. The constructor is not public.

get_name()

Return the symbol's name.

is_referenced()

Return `True` if the symbol is used in its block.

is_imported()

Return `True` if the symbol is created from an import statement.

is_parameter()

Return `True` if the symbol is a parameter.

is_type_parameter()

Return `True` if the symbol is a type parameter.

Adicionado na versão 3.14.

is_global()

Return `True` if the symbol is global.

is_nonlocal()

Return `True` if the symbol is nonlocal.

is_declared_global()

Return `True` if the symbol is declared global with a global statement.

is_local()

Return `True` if the symbol is local to its block.

is_annotated()

Return `True` if the symbol is annotated.

Adicionado na versão 3.6.

is_free()

Return `True` if the symbol is referenced in its block, but not assigned to.

is_free_class()

Return *True* if a class-scoped symbol is free from the perspective of a method.

Consider the following example:

```
def f():
    x = 1  # function-scoped
    class C:
        x = 2  # class-scoped
        def method(self):
            return x
```

In this example, the class-scoped symbol `x` is considered to be free from the perspective of `C.method`, thereby allowing the latter to return *1* at runtime and not *2*.

Adicionado na versão 3.14.

is_assigned()

Return `True` if the symbol is assigned to in its block.

is_comp_iter()

Return `True` if the symbol is a comprehension iteration variable.

Adicionado na versão 3.14.

is_comp_cell()

Return `True` if the symbol is a cell in an inlined comprehension.

Adicionado na versão 3.14.

is_namespace()

Return `True` if name binding introduces new namespace.

If the name is used as the target of a function or class statement, this will be true.

Por exemplo:

```
>>> table = symtable.symtable("def some_func(): pass", "string", "exec")
>>> table.lookup("some_func").is_namespace()
True
```

Note that a single name can be bound to multiple objects. If the result is `True`, the name may also be bound to other objects, like an int or list, that does not introduce a new namespace.

get_namespaces()

Return a list of namespaces bound to this name.

get_namespace()

Return the namespace bound to this name. If more than one or no namespace is bound to this name, a *ValueError* is raised.

32.2.3 Uso na linha de comando

Adicionado na versão 3.13.

O módulo *symtable* pode ser invocado como um script na linha de comando:

```
python -m symtable [infile...]
```

Symbol tables are generated for the specified Python source files and dumped to stdout. If no input file is specified, the content is read from stdin.

32.3 token — Constantes usadas com árvores de análises do Python

Código-fonte: Lib/token.py

Este módulo fornece constantes que representam os valores numéricos dos nós das folhas da árvore de análise (tokens terminais). Consulte o arquivo `Grammar/Tokens` na distribuição Python para obter as definições dos nomes no contexto da gramática da linguagem. Os valores numéricos específicos para os quais os nomes são mapeados podem mudar entre as versões do Python.

O módulo também fornece um mapeamento de códigos numéricos para nomes e algumas funções. As funções espelham definições nos arquivos de cabeçalho do Python C.

Observe que o valor de um token pode depender das opções do tokenizador. Por exemplo, um token `"+"` pode ser relatado como *PLUS* ou *OP*, ou um token `"match"` pode ser *NAME* ou *SOFT_KEYWORD*.

token.**tok_name**

Dicionário que mapeia os valores numéricos das constantes definidas neste módulo de volta para cadeias de nomes, permitindo que seja gerada uma representação mais legível de árvores de análise.

token.**ISTERMINAL**(*x*)

Retorna `True` para valores de tokens terminais.

token.**ISNONTERMINAL**(*x*)

Retorna `True` para valores de tokens não terminais.

token.**ISEOF**(*x*)

Retorna `True` se *x* for o marcador que indica o final da entrada.

Os constantes de tokens são:

token.**NAME**

Valor de token que indica um identificador. Observe que as palavras reservadas também são inicialmente tokenizadas como tokens `NAME`.

token.**NUMBER**

Valor de token que indica um literal numérico

token.**STRING**

Valor de token que indica um literal de string ou byte, excluindo literais de strings formatadas. A string de token não é interpretada: ela inclui as aspas ao redor e o prefixo (se fornecido); contrabarras são incluídas literalmente, sem processar sequências de escape.

token.**OP**

Um valor de token genérico que indica um operador ou delimitador.

Este valor é relatado apenas pelo módulo *tokenize*. Internamente, o tokenizador usa *tipos exatos de tokens* em vez disso.

token.**COMMENT**

Valor de token usado para indicar um comentário. O analisador sintático ignora tokens COMMENT.

token.**NEWLINE**

Valor de token que indica o fim de uma linha lógica.

token.**NL**

Valor de token usado para indicar uma nova linha não terminante. Os tokens NL são gerados quando uma linha lógica de código é continuada em várias linhas físicas. O analisador sintático ignora os tokens NL.

token.**INDENT**

Valor de token usado no início de uma linha lógica para indicar o início de um bloco indentado.

token.**DEDENT**

Valor de token usado no início de uma linha lógica para indicar o fim de um bloco indentado.

token.**FSTRING_START**

Valor de token usado para indicar o início de um literal de f-string.

A string de token inclui o prefixo e as aspas iniciais, mas nenhum conteúdo do literal.

token.**FSTRING_MIDDLE**

Valor de token usado para texto literal dentro de um literal de f-string, incluindo especificações de formato.

Campos de substituição (ou seja, as partes não literais de f-strings) usam os mesmos tokens que outras expressões e são delimitados pelos tokens *LBRACE*, *RBRACE*, *EXCLAMATION* e *COLON*.

token.**FSTRING_END**

Valor de token usado para indicar o fim de um literal de f-string.

A string do token contém as aspas de fechamento.

token.**TSTRING_START**

Valor de token usado para indicar o início de um literal de string template.

A string de token inclui o prefixo e as aspas iniciais, mas nenhum conteúdo do literal.

Adicionado na versão 3.14.

token.**TSTRING_MIDDLE**

Valor de token usado para texto literal dentro de um literal de string template, incluindo especificações de formato.

Campos de substituição (ou seja, as partes não literais de t-strings) usam os mesmos tokens que outras expressões e são delimitados pelos tokens *LBRACE*, *RBRACE*, *EXCLAMATION* e *COLON*.

Adicionado na versão 3.14.

token.**TSTRING_END**

Valor de token usado para indicar o fim de um literal de string template.

A string do token contém as aspas de fechamento.

Adicionado na versão 3.14.

token.**ENDMARKER**

Valor de token que indica o fim da entrada. Usado em regras de gramática de nível superior.

token.**ENCODING**

Valor de token que indica a codificação usada para decodificar os bytes de origem em texto. O primeiro token retornado por *tokenize.tokenize()* sempre será um token ENCODING.

Este tipo de token não é usado pelo tokenizador do C, mas é necessário para o módulo *tokenize*.

Os seguintes tipos de token não são produzidos pelo módulo *tokenize* e são definidos para usos especiais no tokenizador ou analisador:

token.**TYPE_IGNORE**

Valor de token indicando que um comentário `type: ignore` foi reconhecido. Tais tokens são produzidos em vez de tokens *COMMENT* regulares somente com o sinalizador *PyCF_TYPE_COMMENTS*.

token.**TYPE_COMMENT**

Valor de token indicando que um comentário de tipo foi reconhecido. Tais tokens são produzidos em vez de tokens *COMMENT* regulares somente com o sinalizador *PyCF_TYPE_COMMENTS*.

token.**SOFT_KEYWORD**

Valor de token indicando uma palavra reservada contextual.

O tokenizador nunca produz esse valor. Para verificar uma palavra reservada contextual, passe uma string do token *NAME* para *keyword.issoftkeyword()*.

token.**ERRORTOKEN**

Valor de token usado para indicar uma entrada errada.

O módulo *tokenize* geralmente indica erros levantando exceções em vez de emitir este token. Ele também pode emitir tokens como *OP* ou *NAME* com strings que são rejeitadas posteriormente pelo parser.

Os tokens restantes representam operadores e delimitadores específicos. (O módulo *tokenize* os relata como *OP*; consulte `exact_type` na documentação *tokenize* para obter detalhes.)

Token	Valor
token.**LPAR**	"("
token.**RPAR**	")"
token.**LSQB**	"["
token.**RSQB**	"]"
token.**COLON**	":"
token.**COMMA**	","
token.**SEMI**	";"
token.**PLUS**	"+"

continua na próxima página

Tabela 1 – continuação da página anterior

Token	Valor
token.**MINUS**	"-"
token.**STAR**	"*"
token.**SLASH**	"/"
token.**VBAR**	"|"
token.**AMPER**	"&"
token.**LESS**	"<"
token.**GREATER**	">"
token.**EQUAL**	"="
token.**DOT**	"."
token.**PERCENT**	"%"
token.**LBRACE**	"{"
token.**RBRACE**	"}"
token.**EQEQUAL**	"=="
token.**NOTEQUAL**	"!="
token.**LESSEQUAL**	"<="
token.**GREATEREQUAL**	">="
token.**TILDE**	"~"
token.**CIRCUMFLEX**	"^"

continua na próxima página

Tabela 1 – continuação da página anterior

Token	Valor
token.**LEFTSHIFT**	"<<"
token.**RIGHTSHIFT**	">>"
token.**DOUBLESTAR**	"**"
token.**PLUSEQUAL**	"+="
token.**MINEQUAL**	"-="
token.**STAREQUAL**	"*="
token.**SLASHEQUAL**	"/="
token.**PERCENTEQUAL**	"%="
token.**AMPEREQUAL**	"&="
token.**VBAREQUAL**	"|="
token.**CIRCUMFLEXEQUAL**	"^="
token.**LEFTSHIFTEQUAL**	"<<="
token.**RIGHTSHIFTEQUAL**	">>="
token.**DOUBLESTAREQUAL**	"**="
token.**DOUBLESLASH**	"//"
token.**DOUBLESLASHEQUAL**	"//="
token.**AT**	"@"
token.**ATEQUAL**	"@="

continua na próxima página

Tabela 1 – continuação da página anterior

Token	Valor
token.**RARROW**	"->"
token.**ELLIPSIS**	"..."
token.**COLONEQUAL**	":="
token.**EXCLAMATION**	"!"

As seguintes constantes não-tokens são fornecidas:

token.**N_TOKENS**

O número de tipos de token definidos neste módulo.

token.**EXACT_TOKEN_TYPES**

Um dicionário que mapeia a representação de string de um token para seu código numérico.

Adicionado na versão 3.8.

Alterado na versão 3.5: Adicionados os tokens AWAIT e ASYNC.

Alterado na versão 3.7: Adicionados os tokens *COMMENT*, *NL* e *ENCODING*.

Alterado na versão 3.7: Removido os tokens AWAIT e ASYNC. "async" e "await" são agora tokenizados como tokens *NAME*.

Alterado na versão 3.8: Adicionados *TYPE_COMMENT*, *TYPE_IGNORE*, *COLONEQUAL*. Adicionados os tokens AWAIT e ASYNC (eles são necessários para dar suporte à análise de versões mais antigas do Python para *ast.parse()* com feature_version definido como 6 ou inferior).

Alterado na versão 3.12: Adicionado *EXCLAMATION*.

Alterado na versão 3.13: Removidos novamente os tokens AWAIT e ASYNC.

32.4 keyword — Testando palavras reservadas do Python

Código-fonte: Lib/keyword.py

Este módulo permite a um programa Python determinar se uma string é uma palavra reservada ou palavra reservada contextual.

keyword.**iskeyword**(*s*)

Retorna True se *s* for uma palavra reservada do Python.

keyword.**kwlist**

Sequência contendo todas as palavras reservadas definidas para o interpretador. Se alguma palavra reservada estiver definida para apenas estar ativa quando instruções *__future__* específicas tiverem efeito, estas serão incluídas também.

keyword.**issoftkeyword**(*s*)

Retorna True se *s* for uma palavra reservada contextual do Python.

Adicionado na versão 3.9.

keyword.**softkwlist**

Sequência contendo todas as palavras reservadas contextuais definidas para o interpretador. Se alguma palavra reservada contextual estiver definida para apenas estar ativo quando instruções *__future__* tiverem efeito, estas serão incluídas também.

Adicionado na versão 3.9.

32.5 tokenize — Tokenizador para código-fonte Python

Código-fonte: Lib/tokenize.py

O módulo *tokenize* fornece um scanner léxico para código-fonte Python, implementado em Python. O scanner neste módulo retorna comentários como tokens também, tornando-o útil para implementar "pretty-printers", incluindo colorizadores para exibições na tela.

Para simplificar o tratamento do fluxo de tokens, todos os tokens operadores e delimitadores e *Ellipsis* são retornados usando o tipo de token genérico *OP*. O tipo exato pode ser determinado verificando a propriedade `exact_type` na *tupla nomeada* retornada de *tokenize.tokenize()*.

Aviso

Observe que as funções neste módulo são projetadas apenas para analisar código Python sintaticamente válido (código que não levanta exceção quando analisado usando *ast.parse()*). O comportamento das funções neste módulo é **indefinido** ao fornecer código Python inválido e pode mudar a qualquer momento.

32.5.1 Tokenizando entradas

O ponto de entrada principal é um *gerador*:

tokenize.**tokenize**(*readline*)

O gerador *tokenize()* requer um argumento, *readline*, que deve ser um objeto chamável que fornece a mesma interface que o método *io.IOBase.readline()* de objetos arquivo. Cada chamada para a função deve retornar uma linha de entrada como bytes.

O gerador produz tuplas de 5 elementos com estes membros: o tipo de token; a string de token; uma tupla de 2 elementos `(srow, scol)` de ints especificando a linha e a coluna onde o token começa na fonte; uma tupla de 2 elementos `(erow, ecol)` de ints especificando a linha e a coluna onde o token termina na fonte; e a linha na qual o token foi encontrado. A linha passada (o último item da tupla) é a linha *física*. A tupla 5 elementos é retornada como uma *tupla nomeada* com os nomes dos campos: `tipo string início fim linha`.

A *tupla nomeada* retornada tem uma propriedade adicional chamada `exact_type` que contém o tipo exato do operador para tokens *OP*. Para todos os outros tipos de token, `exact_type` é igual ao campo `type` da tupla nomeada.

Alterado na versão 3.1: Adiciona suporte para tuplas nomeadas.

Alterado na versão 3.3: Adicionado suporte para `exact_type`.

tokenize() determina a codificação de origem do arquivo procurando por um BOM UTF-8 ou cookie de codificação, de acordo com **PEP 263**.

tokenize.**generate_tokens**(*readline*)

Tokeniza uma fonte lendo strings unicode em vez de bytes.

Assim como *tokenize()*, o argumento *readline* é um chamável que retorna uma única linha de entrada. No entanto, *generate_tokens()* espera que *readline* retorne um objeto str em vez de bytes.

O resultado é um iterador produzindo tuplas nomeadas, exatamente como *tokenize()*. Ele não produz um token *ENCODING*.

Todas as constantes do módulo *token* também são exportadas de *tokenize*.

Outra função é fornecida para reverter o processo de tokenização. Isso é útil para criar ferramentas que tokenizam um script, modificam o fluxo de tokens e escrevem de volta o script modificado.

tokenize.**untokenize**(*iterable*)

Converte tokens de volta para o código-fonte Python. O *iterable* deve retornar sequências com pelo menos dois elementos, o tipo de token e a string do token. Quaisquer elementos de sequência adicionais são ignorados.

O resultado é garantido para tokenizar de volta para corresponder à entrada, de modo que a conversão seja sem perdas e as viagens de ida e volta sejam garantidas. A garantia se aplica somente ao tipo de token e à sequência de tokens, pois o espaçamento entre os tokens (posições de coluna) pode mudar.

Retorna bytes, codificados usando o token *ENCODING*, que é a primeira sequência de tokens gerada por *tokenize()*. Se não houver nenhum token de codificação na entrada, ele retorna um str em vez disso.

tokenize() precisa detectar a codificação dos arquivos fonte que ele tokeniza. A função que ele usa para fazer isso está disponível:

tokenize.**detect_encoding**(*readline*)

A função *detect_encoding()* é usada para detectar a codificação que deve ser usada para decodificar um arquivo de origem Python. Ela requer um argumento, readline, da mesma forma que o gerador *tokenize()*.

Ele chamará readline no máximo duas vezes e retornará a codificação usada (como uma string) e uma lista de todas as linhas (não decodificadas de bytes) que ele leu.

Ele detecta a codificação a partir da presença de um BOM UTF-8 ou um cookie de codificação conforme especificado em **PEP 263**. Se um BOM e um cookie estiverem presentes, mas discordarem, uma exceção *SyntaxError* será levantada. Observe que se o BOM for encontrado, `'utf-8-sig'` será retornado como uma codificação.

Se nenhuma codificação for especificada, o padrão `'utf-8'` será retornado.

Use *open()* para abrir arquivos de código-fonte Python: ele usa *detect_encoding()* para detectar a codificação do arquivo.

tokenize.**open**(*filename*)

Abre um arquivo no modo somente leitura usando a codificação detectada por *detect_encoding()*.

Adicionado na versão 3.2.

exception tokenize.**TokenError**

Levantada quando uma docstring ou expressão que pode ser dividida em várias linhas não é concluída em nenhum lugar do arquivo, por exemplo:

```
"""Início da
docstring
```

ou:

```
[1,
 2,
 3
```

32.5.2 Uso na linha de comando

Adicionado na versão 3.3.

O módulo *tokenize* pode ser executado como um script a partir da linha de comando. É tão simples quanto:

```
python -m tokenize [-e] [nome-do-arquivo.py]
```

As seguintes opções são aceitas:

-h, --help
: mostra esta mensagem de ajuda e sai

-e, --exact
: exibe nomes de tokens usando o tipo exato

Se `nome-de-arquivo.py` for especificado, seu conteúdo será tokenizado para stdout. Caso contrário, a tokenização será realizada em stdin.

32.5.3 Exemplos

Exemplo de um reescritor de script que transforma literais float em objetos Decimal:

```
from tokenize import tokenize, untokenize, NUMBER, STRING, NAME, OP
from io import BytesIO

def decistmt(s):
    """Substitui Decimals por floats em uma string de instruções.

    >>> from decimal import Decimal
    >>> s = 'print(+21.3e-5*-.1234/81.7)'
    >>> decistmt(s)
    "print (+Decimal ('21.3e-5')*-Decimal ('.1234')/Decimal ('81.7'))"

    O formato do expoente é herdado da biblioteca da plataforma C. Casos
    conhecidos são "e-007" (Windows) e "e-07" (não Windows). Como estamos
    mostrando apenas 12 dígitos, e o 13º não está próximo de 5, o restante
    da saída deve ser independente da plataforma.

    >>> exec(s)  #doctest: +ELLIPSIS
    -3.21716034272e-0...7

    A saída dos cálculos com Decimal deve ser idêntica em todas as
    plataformas.

    >>> exec(decistmt(s))
    -3.217160342717258261933904529E-7
    """
    result = []
    g = tokenize(BytesIO(s.encode('utf-8')).readline)  # tokenize the string
    for toknum, tokval, _, _, _ in g:
        if toknum == NUMBER and '.' in tokval:  # replace NUMBER tokens
            result.extend([
                (NAME, 'Decimal'),
                (OP, '('),
                (STRING, repr(tokval)),
                (OP, ')')
            ])
        else:
            result.append((toknum, tokval))
    return untokenize(result).decode('utf-8')
```

Exemplo de tokenização a partir da linha de comando. O script:

```
def say_hello():
    print("Hello, World!")

say_hello()
```

será tokenizado para a seguinte saída, onde a primeira coluna é o intervalo das coordenadas da linha/coluna onde o token é encontrado, a segunda coluna é o nome do token e a coluna final é o valor do token (se houver)

```
$ python -m tokenize hello.py
0,0-0,0:            ENCODING       'utf-8'
1,0-1,3:            NAME           'def'
1,4-1,13:           NAME           'say_hello'
1,13-1,14:          OP             '('
1,14-1,15:          OP             ')'
1,15-1,16:          OP             ':'
1,16-1,17:          NEWLINE        '\n'
2,0-2,4:            INDENT         '    '
2,4-2,9:            NAME           'print'
2,9-2,10:           OP             '('
2,10-2,25:          STRING         '"Hello, World!"'
2,25-2,26:          OP             ')'
2,26-2,27:          NEWLINE        '\n'
3,0-3,1:            NL             '\n'
4,0-4,0:            DEDENT         ''
4,0-4,9:            NAME           'say_hello'
4,9-4,10:           OP             '('
4,10-4,11:          OP             ')'
4,11-4,12:          NEWLINE        '\n'
5,0-5,0:            ENDMARKER      ''
```

Os nomes exatos dos tipos de token podem ser exibidos usando a opção `-e`:

```
$ python -m tokenize -e hello.py
0,0-0,0:            ENCODING       'utf-8'
1,0-1,3:            NAME           'def'
1,4-1,13:           NAME           'say_hello'
1,13-1,14:          LPAR           '('
1,14-1,15:          RPAR           ')'
1,15-1,16:          COLON          ':'
1,16-1,17:          NEWLINE        '\n'
2,0-2,4:            INDENT         '    '
2,4-2,9:            NAME           'print'
2,9-2,10:           LPAR           '('
2,10-2,25:          STRING         '"Hello, World!"'
2,25-2,26:          RPAR           ')'
2,26-2,27:          NEWLINE        '\n'
3,0-3,1:            NL             '\n'
4,0-4,0:            DEDENT         ''
4,0-4,9:            NAME           'say_hello'
4,9-4,10:           LPAR           '('
4,10-4,11:          RPAR           ')'
4,11-4,12:          NEWLINE        '\n'
5,0-5,0:            ENDMARKER      ''
```

Exemplo de tokenização de um arquivo programaticamente, lendo strings unicode em vez de bytes com *generate_tokens()*:

```
import tokenize

with tokenize.open('hello.py') as f:
    tokens = tokenize.generate_tokens(f.readline)
    for token in tokens:
        print(token)
```

Ou lendo bytes diretamente com *tokenize()*:

```
import tokenize

with open('hello.py', 'rb') as f:
    tokens = tokenize.tokenize(f.readline)
    for token in tokens:
        print(token)
```

32.6 tabnanny — Detecção de indentação ambígua

Código-fonte: Lib/tabnanny.py

Por enquanto, este módulo deve ser chamado como um script. No entanto, é possível importá-lo para um IDE e usar a função *check()* descrita abaixo.

> **Nota**
>
> A API fornecida por este módulo provavelmente mudará em versões futuras; essas mudanças podem não ser compatíveis com versões anteriores.

tabnanny.**check**(*file_or_dir*)

Se *file_or_dir* for um diretório e não um link simbólico, desce recursivamente a árvore de diretórios nomeada por *file_or_dir*, verificando todos os arquivos `.py` ao longo do caminho. Se *file_or_dir* for um arquivo-fonte comum do Python, ele será verificado quanto a problemas relacionados ao espaço em branco. As mensagens de diagnóstico são gravadas na saída padrão usando a função *print()*.

tabnanny.**verbose**

Sinalizador indicando se as mensagens detalhadas devem ser impressas. Isso é incrementado pela opção `-v` se chamado como um script.

tabnanny.**filename_only**

Sinalizador indicando se os nomes dos arquivos devem ser impressos apenas com problemas relacionados a espaços em branco. Isso é definido como true pela opção `-q` se chamado como um script.

exception tabnanny.**NannyNag**

Levantada por *process_tokens()* se detectar um recuo ambíguo. Capturado e manipulado em *check()*.

tabnanny.**process_tokens**(*tokens*)

Esta função é usada por *check()* para processar os tokens gerados pelo módulo *tokenize*.

> **Ver também**
>
> **Módulo *tokenize***
> Scanner léxico para código-fonte Python.

32.7 pyclbr — Suporte a navegador de módulos do Python

Código-fonte: Lib/pyclbr.py

O módulo *pyclbr* fornece informações limitadas sobre as funções, classes e métodos definidos em um módulo codificado em Python. As informações são suficientes para implementar um navegador de módulos. As informações

são extraídas do código-fonte do Python em vez de importar o módulo, portanto, este módulo é seguro para uso com código não confiável. Essa restrição torna impossível o uso deste módulo com módulos não implementados no Python, incluindo todos os módulos de extensão padrão e opcionais.

pyclbr.**readmodule**(*module*, *path=None*)

Retorna um dicionário que mapeia os nomes de classe no nível do módulo aos descritores de classe. Se possível, descritores para classes base importadas estão incluídos. O parâmetro *module* é uma string com o nome do módulo a ser lido; pode ser o nome de um módulo dentro de um pacote. Se fornecido, *path* é uma sequência de caminhos de diretório anexada a `sys.path`, que é usada para localizar o código-fonte do módulo.

Esta função é a interface original e é mantida apenas para compatibilidade reversa. Ela retorna uma versão filtrada da seguinte.

pyclbr.**readmodule_ex**(*module*, *path=None*)

Retorna uma árvore baseada em dicionário que contém uma função ou descritores de classe para cada função e classe definida no módulo com uma instrução `def` ou `class`. O dicionário retornado mapeia os nomes das funções e das classes no nível do módulo para seus descritores. Objetos aninhados são inseridos no dicionário filho de seus pais. Como em readmodule, *module* nomeia o módulo a ser lido e *path* é anexado ao sys.path. Se o módulo que está sendo lido for um pacote, o dicionário retornado terá uma chave `'__path__'` cujo valor é uma lista que contém o caminho de pesquisa do pacote.

Adicionado na versão 3.7: Descritores para definições aninhadas. Eles são acessados através do novo atributo filho. Cada um tem um novo atributo pai.

Os descritores retornados por essas funções são instâncias das classes Function e Class. Não se espera que os usuários criem instâncias dessas classes.

32.7.1 Objetos Function

class pyclbr.**Function**

Instâncias da classe `Function` descrevem funções definidas por instruções def. Elas têm os seguintes atributos:

file

Nome do arquivo no qual a função está definida.

module

O nome do módulo que define a função descrita.

name

O nome da função.

lineno

O número da linha no arquivo em que a definição é iniciada.

parent

Para funções de nível superior, `None`. Para funções aninhadas, o pai.

Adicionado na versão 3.7.

children

Um *dicionário* que mapeia nomes para descritores para funções e classes aninhadas.

Adicionado na versão 3.7.

is_async

`True` para funções que são definidas com o prefixo `async`. Caso contrário, `False`.

Adicionado na versão 3.10.

32.7.2 Objetos Class

class pyclbr.**Class**

Instâncias da classe `Class` descrevem classes definidas por instruções class. Elas têm os mesmos atributos que *`Functions`* e mais dois.

file

Nome do arquivo no qual a classe está definida.

module

O nome do módulo que define a classe descrita.

name

O nome da classe.

lineno

O número da linha no arquivo em que a definição é iniciada.

parent

Para classes de nível superior, `None`. Para classes aninhadas, o pai.

Adicionado na versão 3.7.

children

Um dicionário que mapeia nomes para descritores para funções e classes aninhadas.

Adicionado na versão 3.7.

super

Uma lista de objetos `Class` que descreve as classes base imediatas da classe que está sendo descrita. Classes nomeadas como superclasses, mas que não podem ser descobertas por *`readmodule_ex()`* são listadas como uma string com o nome da classe em vez de como objetos de `Class`.

methods

Um *`dicionário`* que mapeia nomes de métodos para números de linha. Isso pode ser derivado do dicionário *`children`* mais novo, mas permanece para compatibilidade retroativa.

32.8 `py_compile` — Compila arquivos fonte do Python

Código-fonte: Lib/py_compile.py

O módulo *`py_compile`* fornece uma função para gerar um arquivo de bytecode a partir de um arquivo fonte, e outra função usada quando o arquivo fonte do módulo é chamado como um script.

Embora nem sempre seja necessária, essa função pode ser útil ao instalar módulos para uso compartilhado, especialmente se alguns usuários não tiverem permissão para gravar os arquivos de cache de bytecodes no diretório que contém o código-fonte.

exception py_compile.**PyCompileError**

Exceção levantada quando ocorre um erro ao tentar compilar o arquivo.

py_compile.**compile**(*file*, *cfile=None*, *dfile=None*, *doraise=False*, *optimize=-1*, *invalidation_mode=PycInvalidationMode.TIMESTAMP*, *quiet=0*)

Compila um arquivo fonte para bytecode e grava o arquivo de cache de bytecode. O código-fonte é carregado a partir do arquivo chamado *file*. O bytecode é gravado em *cfile*, cujo padrão é o caminho **PEP 3147**/**PEP 488**, terminando em `.pyc`. Por exemplo, se *file* for `/foo/bar/baz.py`, o *cfile* será padronizado como `/foo/bar/__pycache__/baz.cpython-32.pyc` para o Python 3.2. Se *dfile* for especificado, ele será usado em vez de *file* como o nome do arquivo fonte do qual as linhas de origem são obtidas para exibição em tracebacks de exceção. Se *doraise* for verdadeiro, a *`PyCompileError`* será levantada quando um erro for encontrado durante a compilação de *file*. Se *doraise* for false (o padrão), uma string de erros será gravada em `sys.stderr`, mas

nenhuma exceção será levantada. Essa função retorna o caminho para o arquivo compilado em bytes, ou seja, qualquer valor *cfile* foi usado.

Os argumentos *doraise* e *quiet* determinam como os erros são tratados durante a compilação do arquivo. Se *quiet* for 0 ou 1 e *doraise* for false, o comportamento padrão será ativado: uma string de erros será gravada em `sys.stderr` e a função retornará `None` em vez de um caminho. Se *doraise* for verdadeiro, uma *PyCompileError* será levantada. No entanto, se *quiet* for 2, nenhuma mensagem será escrita e *doraise* não terá efeito.

Se o caminho que *cfile* se tornar (especificado ou computado explicitamente) for um link simbólico ou um arquivo não regular, *FileExistsError* será levantada. Isso serve como um aviso de que a importação transformará esses caminhos em arquivos regulares se for permitido gravar arquivos compilados em bytes nesses caminhos. Esse é um efeito colateral da importação usando a renomeação de arquivo para colocar o arquivo final compilado em bytecode para evitar problemas de gravação simultânea de arquivos.

optimize controla o nível de otimização e é passado para a função embutida *compile()*. O padrão de `-1` seleciona o nível de otimização do interpretador atual.

invalidation_mode deve ser um membro da enum *PycInvalidationMode* e controla como o cache do bytecode gerado é invalidado em tempo de execução. O padrão é *PycInvalidationMode.CHECKED_HASH* se a variável de ambiente `SOURCE_DATE_EPOCH` estiver configurada, caso contrário, o padrão é *PycInvalidationMode.TIMESTAMP*.

Alterado na versão 3.2: Alterado o valor padrão de *cfile* para ficar em conformidade com a **PEP 3147**. O padrão anterior era *file* + `'c'` (`'o'` se a otimização estivesse ativada). Também foi adicionado o parâmetro *optimize*.

Alterado na versão 3.4: Alterado o código para usar *importlib* para a gravação do arquivo de cache do bytecode. Isso significa que a semântica de criação/gravação de arquivo agora corresponde ao que *importlib* faz, por exemplo, permissões, semântica de gravação e movimentação, etc. Também foi adicionada a ressalva de que *FileExistsError* é levantada se *cfile* for um link simbólico ou um arquivo não regular.

Alterado na versão 3.7: O parâmetro *invalidation_mode* foi adicionado conforme especificado em **PEP 552**. Se a variável de ambiente `SOURCE_DATE_EPOCH` estiver configurada, *invalidation_mode* será forçado a *PycInvalidationMode.CHECKED_HASH*.

Alterado na versão 3.7.2: A variável de ambiente `SOURCE_DATE_EPOCH` não substitui mais o valor do argumento *invalidation_mode* e, em vez disso, determina seu valor padrão.

Alterado na versão 3.8: O parâmetro *quiet* foi adicionado.

class py_compile.**PycInvalidationMode**

Uma enumeração de métodos possíveis que o interpretador pode usar para determinar se um arquivo de bytecode está atualizado com um arquivo fonte. O arquivo `.pyc` indica o modo de invalidação desejado em seu cabeçalho. Veja pyc-invalidation para obter mais informações sobre como o Python invalida arquivos `.pyc` em tempo de execução.

Adicionado na versão 3.7.

TIMESTAMP

O arquivo `.pyc` inclui o carimbo de data e hora e o tamanho do arquivo fonte, que o Python comparará com os metadados do arquivo fonte no tempo de execução para determinar se o arquivo `.pyc` precisa ser gerado novamente.

CHECKED_HASH

O arquivo `.pyc` inclui um hash do conteúdo do arquivo fonte, com o qual o Python comparará o fonte em tempo de execução para determinar se o arquivo `.pyc` precisa ser gerado novamente.

UNCHECKED_HASH

Como *CHECKED_HASH*, o arquivo `.pyc` inclui um hash do conteúdo do arquivo fonte. No entanto, em tempo de execução, o Python presumirá que o arquivo `.pyc` está atualizado e não validará o `.pyc` contra o arquivo fonte.

Essa opção é útil quando os `.pycs` são atualizados por algum sistema externo ao Python, como um sistema de compilação.

32.8.1 Interface de Linha de Comando

Este módulo pode ser invocado como um script para compilar vários arquivos fonte. Os arquivos nomeados em *filenames* são compilados e o bytecode resultante é armazenado em cache da maneira normal. Este programa não pesquisa uma estrutura de diretórios para localizar arquivos de origem; ele apenas compila arquivos nomeados explicitamente. O status de saída será diferente de zero se um dos arquivos não puder ser compilado.

<file> ... <fileN>

-

Argumentos posicionais são arquivos para compilar. Se – for o único parâmetro, a lista de arquivos será obtida da entrada padrão.

-q, --quiet

Suprime a saída de erros.

Alterado na versão 3.2: Adicionado suporte a –.

Alterado na versão 3.10: Adicionado suporte a `-q`.

Ver também

Módulo ***compileall***
Utilitários para compilar todos os arquivos fontes Python em uma árvore de diretórios.

32.9 `compileall` — Compilar bibliotecas do Python para bytecode

Código-fonte: Lib/compileall.py

Este módulo fornece algumas funções utilitárias para dar suporte à instalação de bibliotecas Python. Essas funções compilam arquivos fonte Python em uma árvore de diretórios. Este módulo pode ser usado para criar os arquivos de bytecodes em cache no momento da instalação da biblioteca, o que os torna disponíveis para uso mesmo por usuários que não têm permissão de gravação nos diretórios da biblioteca.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja *Plataformas WebAssembly* para mais informações.

32.9.1 Uso na linha de comando

Este módulo pode funcionar como um script (usando **`python -m compileall`**) para compilar fontes do Python.

directory ...

file ...

Argumentos posicionais são arquivos a serem compilados ou diretórios que contêm arquivos de origem, percorridos recursivamente. Se nenhum argumento for fornecido, comporta-se como se a linha de comando fosse `-l <diretórios do sys.path>`.

-l

Não atua recursivamente em subdiretórios, apenas compila arquivos de código-fonte diretamente contidos nos diretórios nomeados ou implícitos.

-f

Força a recompilação, mesmo que os carimbos de data e hora estejam atualizados.

-q

Não imprime a lista de arquivos compilados. Se passado uma vez, as mensagens de erro ainda serão impressas. Se passado duas vezes (`-qq`), toda a saída é suprimida.

-d destdir

Diretório anexado ao caminho de cada arquivo que está sendo compilado. Isso aparecerá nos tracebacks em tempo de compilação e também será compilado no arquivo de bytecode, onde será usado em tracebacks e outras mensagens nos casos em que o arquivo de origem não exista no momento em que o arquivo de bytecode for executado.

-s strip_prefix

Remove o prefixo especificado dos caminhos gravados nos arquivos `.pyc`. Caminhos são tornados relativos ao prefixo.

Esta opção pode ser usada com `-p`, mas não com `-d`.

-p prepend_prefix

Adiciona o prefixo fornecido aos caminhos registrados nos arquivos `.pyc`. Use `-p /` para tornar os caminhos absolutos.

Esta opção pode ser usada com `-s`, mas não com `-d`.

-x regex

A expressão regular `regex` é usada para pesquisar o caminho completo para cada arquivo considerado para compilação e, se a `regex` produzir uma correspondência, o arquivo será ignorado.

-i list

Lê o arquivo `list` e adicione cada linha que ele contém à lista de arquivos e diretórios a serem compilados. Se `list` for `-`, lê as linhas do `stdin`.

-b

Escreve os arquivos de bytecode em seus locais e nomes legados, que podem sobrescrever arquivos de bytecode criados por outra versão do Python. O padrão é gravar arquivos em seus locais e nomes do **PEP 3147**, o que permite que arquivos de bytecode de várias versões do Python coexistam.

-r

Controla o nível máximo de recursão para subdiretórios. Se isso for dado, a opção `-l` não será levada em consideração. **python -m compileall <diretório> -r 0** é equivalente a **python -m compileall <diretório> -l**.

-j N

Usa *N* workers para compilar os arquivos dentro do diretório especificado. Se `0` for usado, o resultado de *os.process_cpu_count()* será usado.

--invalidation-mode [timestamp|checked-hash|unchecked-hash]

Controla como os arquivos de bytecode gerados são invalidados no tempo de execução. O valor `timestamp` significa que os arquivos `.pyc` com o carimbo de data/hora do fonte e o tamanho incorporado serão gerados. Os valores `selected-hash` e `unchecked-hash` fazem com que os pycs baseados em hash sejam gerados. Arquivos pycs baseados em hash incorporam um hash do conteúdo do arquivo fonte em vez de um carimbo de data/hora. Veja pyc-invalidation para obter mais informações sobre como o Python valida os arquivos de cache do bytecode em tempo de execução. O padrão é `timestamp` se a variável de ambiente `SOURCE_DATE_EPOCH` não estiver configurada e `selected-hash` se a variável de ambiente `SOURCE_DATE_EPOCH` estiver configurada.

-o level

Compila com o nível de otimização fornecido. Pode ser usado várias vezes para compilar para vários níveis por vez (por exemplo, `compileall -o 1 -o 2`).

-e dir

Ignora links simbólicos que apontam para fora do diretório especificado.

--hardlink-dupes

Se dois arquivos `.pyc` com nível de otimização diferente tiverem o mesmo conteúdo, usa links físicos para consolidar arquivos duplicados.

Alterado na versão 3.2: Adicionadas as opções `-i`, `-b` e `-h`.

Alterado na versão 3.5: Adicionadas as opções `-j`, `-r` e `-qq`. A opção `-q` foi alterada para um valor multinível. `-b` sempre produzirá um arquivo de bytecodes que termina em `.pyc`, nunca em `.pyo`.

Alterado na versão 3.7: Adicionada a opção `--invalidation-mode`.

Alterado na versão 3.9: Adicionadas as opções `-s`, `-p`, `-e` e `--hardlink-dupes`. Aumentado o limite de recursão padrão de 10 para *sys.getrecursionlimit()*. Adicionada a possibilidade de especificar a opção `-o` várias vezes.

Não há opção na linha de comando para controlar o nível de otimização usado pela função *compile()* porque o próprio interpretador Python já fornece a opção: **`python -O -m compileall`**.

Da mesma forma, a função *compile()* respeita a configuração *sys.pycache_prefix*. O cache do bytecode gerado somente será útil se *compile()* for executado com o mesmo *sys.pycache_prefix* (se houver) que será usado em tempo de execução.

32.9.2 Funções públicas

`compileall.`**`compile_dir`**(*dir*, *maxlevels=sys.getrecursionlimit()*, *ddir=None*, *force=False*, *rx=None*, *quiet=0*, *legacy=False*, *optimize=-1*, *workers=1*, *invalidation_mode=None*, ***, *stripdir=None*, *prependdir=None*, *limit_sl_dest=None*, *hardlink_dupes=False*)

Desce recursivamente a árvore de diretórios nomeada por *dir*, compilando todos os arquivos `.py` ao longo do caminho. Retorna um valor verdadeiro se todos os arquivos forem compilados com êxito e um valor falso caso contrário.

O parâmetro *maxlevels* é usado para limitar a profundidade da recursão; o padrão é `sys.getrecursionlimit()`.

Se *ddir* for fornecido, ele será anexado ao caminho de cada arquivo que está sendo compilado para uso em tracebacks em tempo de compilação e também será compilado no arquivo de bytecode, onde será usado em tracebacks e outras mensagens nos casos em que o arquivo de origem não existe no momento em que o arquivo de bytecode é executado.

Se *force* for verdadeiro, os módulos serão recompilados, mesmo que os carimbos de data e hora estejam atualizados.

Se *rx* for fornecido, seu método `search` será chamado no caminho completo para cada arquivo considerado para compilação e, se retornar um valor verdadeiro, o arquivo será ignorado. Isso pode ser usado para excluir arquivos correspondendo a uma expressão regular, dado como um objeto *re.Pattern*.

Se *quiet* for `False` ou `0` (o padrão), os nomes dos arquivos e outras informações serão impressos com o padrão. Definido como `1`, apenas os erros são impressos. Definido como `2`, toda a saída é suprimida.

Se *legacy* for verdadeiro, os arquivos de bytecodes serão gravados em seus locais e nomes herdados, o que poderá sobrescrever arquivos de bytecodes criados por outra versão do Python. O padrão é gravar arquivos em seus locais e nomes do **PEP 3147**, o que permite que arquivos de bytecodes de várias versões do Python coexistam.

optimize especifica o nível de otimização para o compilador. Ele é passado para a função embutida *compile()*. Aceita também uma sequência de níveis de otimização que levam a várias compilações de um arquivo `.py` em uma chamada.

O argumento *workers* especifica quantos workers são usados para compilar arquivos em paralelo. O padrão é não usar vários workers. Se a plataforma não puder usar vários workers e o argumento *workers* for fornecido, a compilação sequencial será usada como reserva. Se *workers* for 0, o número de núcleos no sistema é usado. Se *workers* for menor que `0`, a *ValueError* será levantada.

invalidation_mode deve ser um membro de enum *py_compile.PycInvalidationMode* e controla como os pycs gerados são invalidados em tempo de execução.

Os argumentos *stripdir*, *prependdir* e *limit_sl_dest* correspondem às opções `-s`, `-p` e `-e` descrita acima. eles podem ser especificados como `str` ou *os.PathLike*.

Se *hardlink_dupes* for verdadeiro e dois arquivos `.pyc` com nível de otimização diferente tiverem o mesmo conteúdo, usa links físicos para consolidar arquivos duplicados.

Alterado na versão 3.2: Adicionado os parâmetros *legacy* e *optimize*.

Alterado na versão 3.5: Adicionado o parâmetro *workers*.

Alterado na versão 3.5: O parâmetro *quiet* foi alterado para um valor multinível.

Alterado na versão 3.5: O parâmetro *legacy* grava apenas arquivos `.pyc`, não os arquivos `.pyo`, independentemente do valor de *optimize*.

Alterado na versão 3.6: Aceita um *objeto caminho ou similar*.

Alterado na versão 3.7: O parâmetro *invalidation_mode* foi adicionado.

Alterado na versão 3.7.2: O valor padrão do parâmetro *invalidation_mode* é atualizado para `None`.

Alterado na versão 3.8: A definição de *workers* como 0 agora escolhe o número ideal de núcleos.

Alterado na versão 3.9: Adicionados os argumentos *stripdir*, *prependdir*, *limit_sl_dest* e *hardlink_dupes*. O valor padrão de *maxlevels* foi alterado de `10` para `sys.getrecursionlimit()`

compileall.**compile_file**(*fullname*, *ddir=None*, *force=False*, *rx=None*, *quiet=0*, *legacy=False*, *optimize=-1*, *invalidation_mode=None*, ***, *stripdir=None*, *prependdir=None*, *limit_sl_dest=None*, *hardlink_dupes=False*)

Compila o arquivo com o caminho *fullname*. Retorna um valor verdadeiro se o arquivo compilado com êxito e um valor falso caso contrário.

Se *ddir* for fornecido, ele será anexado ao caminho do arquivo que está sendo compilado para uso em rastreamentos em tempo de compilação e também será compilado no arquivo de bytecode, onde será usado em tracebacks e outras mensagens nos casos em que o arquivo fonte não existe no momento em que o arquivo de bytecode é executado.

Se *rx* for fornecido, seu método `search` passará o nome do caminho completo para o arquivo que está sendo compilado e, se retornar um valor verdadeiro, o arquivo não será compilado e `True` será retornado. Isso pode ser usado para excluir arquivos correspondendo a uma expressão regular, dado como um objeto *re.Pattern*.

Se *quiet* for `False` ou `0` (o padrão), os nomes dos arquivos e outras informações serão impressos com o padrão. Definido como `1`, apenas os erros são impressos. Definido como `2`, toda a saída é suprimida.

Se *legacy* for verdadeiro, os arquivos de bytecodes serão gravados em seus locais e nomes herdados, o que poderá sobrescrever arquivos de bytecodes criados por outra versão do Python. O padrão é gravar arquivos em seus locais e nomes do **PEP 3147**, o que permite que arquivos de bytecodes de várias versões do Python coexistam.

optimize especifica o nível de otimização para o compilador. Ele é passado para a função embutida *compile()*. Aceita também uma sequência de níveis de otimização que levam a várias compilações de um arquivo `.py` em uma chamada.

invalidation_mode deve ser um membro de enum *py_compile.PycInvalidationMode* e controla como os pycs gerados são invalidados em tempo de execução.

Os argumentos *stripdir*, *prependdir* e *limit_sl_dest* correspondem às opções `-s`, `-p` e `-e` descrita acima. eles podem ser especificados como `str` ou *os.PathLike*.

Se *hardlink_dupes* for verdadeiro e dois arquivos `.pyc` com nível de otimização diferente tiverem o mesmo conteúdo, usa links físicos para consolidar arquivos duplicados.

Adicionado na versão 3.2.

Alterado na versão 3.5: O parâmetro *quiet* foi alterado para um valor multinível.

Alterado na versão 3.5: O parâmetro *legacy* grava apenas arquivos `.pyc`, não os arquivos `.pyo`, independentemente do valor de *optimize*.

Alterado na versão 3.7: O parâmetro *invalidation_mode* foi adicionado.

Alterado na versão 3.7.2: O valor padrão do parâmetro *invalidation_mode* é atualizado para `None`.

Alterado na versão 3.9: Adicionados os argumentos *stripdir*, *prependdir*, *limit_sl_dest* e *hardlink_dupes*.

compileall.**compile_path**(*skip_curdir=True*, *maxlevels=0*, *force=False*, *quiet=0*, *legacy=False*, *optimize=-1*, *invalidation_mode=None*)

Compila Byte para bytecodes todos os arquivos `.py` encontrados ao longo de `sys.path`. Retorna um valor verdadeiro se todos os arquivos forem compilados com êxito e um valor falso caso contrário.

Se *skip_curdir* for verdadeiro (o padrão), o diretório atual não será incluído na pesquisa. Todos os outros parâmetros são passados para a função *compile_dir()*. Note que, ao contrário das outras funções de compilação, `maxlevels` é padronizado como `0`.

Alterado na versão 3.2: Adicionado os parâmetros *legacy* e *optimize*.

Alterado na versão 3.5: O parâmetro *quiet* foi alterado para um valor multinível.

Alterado na versão 3.5: O parâmetro *legacy* grava apenas arquivos `.pyc`, não os arquivos `.pyo`, independentemente do valor de *optimize*.

Alterado na versão 3.7: O parâmetro *invalidation_mode* foi adicionado.

Alterado na versão 3.7.2: O valor padrão do parâmetro *invalidation_mode* é atualizado para `None`.

Para forçar uma recompilação de todos os arquivos `.py` no subdiretório `Lib/` e todos os seus subdiretórios:

```
import compileall

compileall.compile_dir('Lib/', force=True)

# Efetua a mesma compilação, excluindo arquivos em diretórios .svn.
import re
compileall.compile_dir('Lib/', rx=re.compile(r'[/\\][.]svn'), force=True)

# Objetos pathlib.Path também podem ser usados.
import pathlib
compileall.compile_dir(pathlib.Path('Lib/'), force=True)
```

Ver também

Módulo *py_compile*
Compila para bytecode um único arquivo fonte.

32.10 `dis` — Disassembler de bytecode do Python

Código-fonte: Lib/dis.py

O módulo *dis* oferece suporte à análise de *bytecode* do CPython, desmontando-o. O bytecode do CPython que o módulo leva como entrada é definido no arquivo `Include/opcode.h` e usado pelo compilador e pelo interpretador.

O bytecode é um detalhe de implementação do interpretador CPython. Não há garantias de que bytecodes não serão adicionados, removidos ou alterados entre as versões do Python. O uso deste módulo não deve ser considerado que funcionará em todas as VMs do Python ou mesmo versões do Python.

Alterado na versão 3.6: Cada instrução ocupa 2 bytes. Anteriormente, o número de bytes variava de acordo com a instrução.

Alterado na versão 3.10: O argumento para instruções de pulo, tratamento de exceção e laço é agora o deslocamento em instruções, ao invés de em bytes.

Alterado na versão 3.11: Algumas instruções vêm acompanhadas de uma ou mais entradas de cache em linha, as quais assumem a forma de instruções *CACHE*. Tais instruções são escondidas por padrão, mas podem ser visualizadas

passando `show_caches=True` para qualquer utilidade do *dis*. Além disso, o interpretador agora adapta o bytecode para especializá-lo a diferentes condições de tempo de execução. O bytecode adaptativo pode ser visualizado passando `adaptive=True`.

Alterado na versão 3.12: O argumento de um pulo é o deslocamento da instrução alvo relativo à instrução que aparece imediatamente após as entradas *CACHE* da instrução de pulo.

Como consequência, a presença de instruções *CACHE* é transparente para pulos adiante, mas precisa ser considerada ao lidar com pulos para trás.

Alterado na versão 3.13: A saída agora mostra rótulos lógicos ao invés dos deslocamentos das instruções para alvos de pulos e de tratadores de exceções. A opção de linha de comando `-O` e o argumento `show_offsets` foram adicionados.

Alterado na versão 3.14: A opção de linha de comando *-P* e o argumento `show_positions` foram adicionados.

A opção de linha de comando *-S* foi adicionada.

Exemplo: Dada a função `myfunc()`:

```
def myfunc(alist):
    return len(alist)
```

o comando a seguir pode ser usado para mostrar a desconstrução de `myfunc()`:

```
>>> dis.dis(myfunc)
  2           RESUME                   0

  3           LOAD_GLOBAL              1 (len + NULL)
              LOAD_FAST_BORROW         0 (alist)
              CALL                     1
              RETURN_VALUE
```

(O "2" é o número da linha).

32.10.1 Interface de linha de comando

O módulo *dis* pode ser invocado como um script na linha de comando:

```
python -m dis [-h] [-C] [-O] [-P] [-S] [infile]
```

As seguintes opções são aceitas:

-h, --help
: Exibe o modo de uso e sai.

-C, --show-caches
: Mostra caches em linha

  Adicionado na versão 3.13.

-O, --show-offsets
: Mostra os deslocamentos das instruções

  Adicionado na versão 3.13.

-P, --show-positions
: Mostra posições de instruções no código-fonte.

  Adicionado na versão 3.14.

-S, --specialized
: Mostra bytecode especializado.

  Adicionado na versão 3.14.

Se `infile` for especificada, o seu código desmontado será escrito no stdout. Caso contrário, será feito o desmonte da compilação do código-fonte recebido do stdin.

32.10.2 Análise de bytecode

Adicionado na versão 3.4.

A API de análise de bytecode permite que partes do código Python sejam encapsuladas em um objeto *Bytecode* que facilite o acesso aos detalhes do código compilado.

class dis.**Bytecode**(*x*, *, *first_line=None*, *current_offset=None*, *show_caches=False*, *adaptive=False*, *show_offsets=False*, *show_positions=False*)

Analisa o bytecode correspondente a uma função, um gerador, um gerador assíncrono, uma corrotina, um método, uma string de código-fonte, ou um objeto de código (conforme retornado por *compile()*).

Esta é um invólucro de conveniência que encapsula muitas das funções listadas abaixo, principalmente a *get_instructions()*, já que iterar sobre sobre uma instância de *Bytecode* produz operações bytecode como instâncias de *Instruction*.

Se *first_line* não for `None`, ele indica o número de linha que deve ser reportado para a primeira linha de código-fonte no código desmontado. Caso contrário, a informação de linha de código-fonte (se houver) é extraída diretamente da desconstrução do objeto de código.

Se *current_offset* não for `None`, ele é um deslocamento em instruções no código desconstruído. Definir este argumento significa que o *dis()* vai mostrar um marcador de "instrução atual" sobre o opcode especificado.

Se *show_caches* for `True`, o *dis()* vai exibir entradas de cache em linha usadas pelo interpretador para especializar o bytecode.

Se *adaptive* for `True`, o *dis()* vai exibir bytecode especializado que pode ser diferente do bytecode original.

Se *show_offsets* for `True`, o *dis()* vai incluir deslocamentos em instruções na saída.

Se *show_positions* for `True`, *dis()* incluirá posições do código-fonte da instrução na saída.

classmethod from_traceback(*tb*, *, *show_caches=False*)

Constrói uma instância de *Bytecode* a partir do traceback fornecido, definindo *current_offset* apontando para a instrução responsável pela exceção.

codeobj

O objeto de código compilado.

first_line

A primeira linha de código-fonte do objeto de código (caso disponível).

dis()

Retorna uma visualização formatada das operações em bytecode (as mesmas que seriam impressas pela *dis.dis()*, mas retornadas como uma string multilinha).

info()

Retorna uma string multilinha formatada com informação detalhada sobre o objeto de código, como *code_info()*.

Alterado na versão 3.7: Este método agora lida com objetos de corrotina e de gerador assíncrono.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: Adicionado o parâmetro *show_offsets*

Alterado na versão 3.14: Foi adicionado o parâmetro *show_positions*.

Exemplo:

```
>>> bytecode = dis.Bytecode(myfunc)
>>> for instr in bytecode:
...     print(instr.opname)
...
RESUME
LOAD_GLOBAL
LOAD_FAST_BORROW
CALL
RETURN_VALUE
```

32.10.3 Funções de análise

O módulo *dis* também define as seguintes funções que convertem a entrada diretamente para a saída desejada. Elas podem ser úteis se somente uma única operação está sendo feita, de forma que o objeto de análise intermediário não é útil:

dis.**code_info**(*x*)

Retorna uma string multilinha formatada com informação detalhada sobre o objeto de código correspondente à função, gerador, gerador assíncrono, corrotina, método, string de código-fonte ou objeto de código fornecido.

Observe que o conteúdo exato de strings de informação de código são altamente dependentes da implementação e podem mudar de forma arbitrária através de VMs Python ou lançamentos do Python.

Adicionado na versão 3.2.

Alterado na versão 3.7: Este método agora lida com objetos de corrotina e de gerador assíncrono.

dis.**show_code**(*x*, *, *file=None*)

Imprime no arquivo *file* (ou `sys.stdout` caso *file* não seja especificado) informações detalhadas sobre o objeto de código correspondente à função, método, string de código-fonte fornecido.

Este é um atalho conveniente para `print(code_info(x), file=file)`, destinado à exploração interativa no prompt do interpretador.

Adicionado na versão 3.2.

Alterado na versão 3.4: Adicionado o parâmetro *file*.

dis.**dis**(*x=None*, *, *file=None*, *depth=None*, *show_caches=False*, *adaptive=False*, *show_offsets=False*, *show_positions=False*)

Desmonta o objeto *x*. *x* pode denotar um módulo, uma classe, um método, uma função, um gerador, um gerador assíncrono, uma corrotina, um objeto de código, uma string de código-fonte ou uma sequência de bytes contendo bytecode bruto. Para um módulo, são desmontadas todas as funções. Para uma classe, são desmontados todos os métodos (incluindo métodos de classe e estáticos). Para um objeto de código ou sequência de bytecodes brutos, é impressa uma linha para cada instrução de bytecode. Além disso, objetos de código aninhados são desmontados recursivamente. Estes podem incluir expressões geradoras, funções aninhadas, corpos de classes aninhadas, e objetos de código usados para escopos de anotação. Strings são compiladas para objetos de código com a função embutida *compile()* antes de serem desmontadas. Se nenhum objeto for fornecido, o último traceback é desmontado.

O resultado é escrito como texto no arquivo *file* caso tenha sido fornecido como argumento, ou para `sys.stdout` caso contrário.

A profundidade máxima de recursão é limitada por *depth* a menos que seja `None`. `depth=0` significa não fazer recursão.

Se *show_caches* for `True`, essa função vai exibir entradas de cache em linha usadas pelo interpretador para especializar o bytecode.

Se *adaptive* for `True`, essa função vai exibir bytecode especializado que pode ser diferente do bytecode original.

Alterado na versão 3.4: Adicionado o parâmetro *file*.

Alterado na versão 3.7: Foi implementada a desmontagem recursiva, e adicionado o parâmetro *depth*.

Alterado na versão 3.7: Este método agora lida com objetos de corrotina e de gerador assíncrono.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: Foi adicionado o parâmetro *show_offsets*.

Alterado na versão 3.14: Foi adicionado o parâmetro *show_positions*.

dis.**distb**(*tb=None*, ***, *file=None*, *show_caches=False*, *adaptive=False*, *show_offset=False*, *show_positions=False*)

Desmonta a função no topo da pilha de um traceback, usando o último traceback caso nenhum tenha sido passado. A instrução que causou a exceção é indicada.

O resultado é escrito como texto no arquivo *file* caso tenha sido fornecido como argumento, ou para `sys.stdout` caso contrário.

Alterado na versão 3.4: Adicionado o parâmetro *file*.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: Foi adicionado o parâmetro *show_offsets*.

Alterado na versão 3.14: Foi adicionado o parâmetro *show_positions*.

dis.**disassemble**(*code*, *lasti=-1*, ***, *file=None*, *show_caches=False*, *adaptive=False*, *show_offsets=False*, *show_positions=False*)

dis.**disco**(*code*, *lasti=-1*, ***, *file=None*, *show_caches=False*, *adaptive=False*, *show_offsets=False*, *show_positions=False*)

Desmonta um objeto de código, indicando a última instrução se *lasti* tiver sido fornecido. A saída é dividida em colunas da seguinte forma:

1. a localização do código-fonte da instrução. Informações completas sobre a localização são exibidas se *show_positions* for verdadeiro. Caso contrário (o padrão), apenas o número da linha é exibido.
2. a instrução atual, indicada por `-->`,
3. um rótulo da instrução, indicado com `>>`,
4. o endereço da instrução
5. o nome do código da operação,
6. os parâmetros da operação, e
7. a interpretação dos parâmetros, em parênteses.

A interpretação dos parâmetros reconhece nomes de variáveis locais e globais, valores de constantes, alvos de ramificações, e operadores de comparação.

O resultado é escrito como texto no arquivo *file* caso tenha sido fornecido como argumento, ou para `sys.stdout` caso contrário.

Alterado na versão 3.4: Adicionado o parâmetro *file*.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: Foi adicionado o parâmetro *show_offsets*.

Alterado na versão 3.14: Foi adicionado o parâmetro *show_positions*.

dis.**get_instructions**(*x*, ***, *first_line=None*, *show_caches=False*, *adaptive=False*)

Retorna um iterador sobre as instruções na função, método, string de código-fonte ou objeto de código fornecido.

O iterador gera uma série de tuplas nomeadas *Instruction* contendo detalhes de cada operação no código fornecido.

Se *first_line* não for `None`, ele indica o número de linha que deve ser reportado para a primeira linha de código-fonte no código desmontado. Caso contrário, a informação de linha de código-fonte (se houver) é extraída diretamente da desconstrução do objeto de código.

O parâmetro *adaptive* funciona assim como na função `dis()`.

Adicionado na versão 3.4.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: O parâmetro *show_caches* foi descontinuado e não tem mais efeito. O iterador gera as instâncias da `Instruction` com o campo *cache_info* populado (independentemente do valor de *show_caches*) e não gera mais items separados para as entradas de cache.

`dis.`**`findlinestarts`**(*code*)

Essa função geradora usa o método `co_lines()` do objeto de código *code* para encontrar as posições que correspondem aos inícios de cada linha do código-fonte. Elas são geradas em pares `(offset, lineno)`.

Alterado na versão 3.6: Números de linhas podem ser decrescentes. Antes, eles eram sempre crescentes.

Alterado na versão 3.10: O método `co_lines()` da **PEP 626** é usado ao invés dos atributos `co_firstlineno` e `co_lnotab` do objeto de código.

Alterado na versão 3.13: Números de linha podem ser `None` para bytecode que não corresponde a um código-fonte.

`dis.`**`findlabels`**(*code*)

Detecta todas as posições na string de bytecode compilado bruto *code* que são alvos de pulos, e as retorna em uma lista.

`dis.`**`stack_effect`**(*opcode*, *oparg=None*, *, *jump=None*)

Calcula o efeito que o *opcode* com argumento *oparg* tem na pilha.

Se a operação tiver um alvo de pulo e *jump* for `True`, `stack_effect()` vai retornar o efeito na pilha de realizar o pulo. Se *jump* for `False`, ela vai retornar o efeito na pilha de não pular. E se *jump* for `None` (o padrão), vai retornar o efeito máximo na pilha dentre os dois casos.

Adicionado na versão 3.4.

Alterado na versão 3.8: Adicionado o parâmetro *jump*.

Alterado na versão 3.13: Se `oparg` for omitido (ou `None`), agora é retornado o efeito na pilha para o caso `oparg=0`. Anteriormente isso era um erro caso o opcode usasse o seu argumento. Além disso, agora não é mais um erro passar um inteiro como `oparg` quando o `opcode` não o usa; o `oparg` nesse caso é ignorado.

32.10.4 Instruções em bytecode do Python

A função `get_instructions()` e a classe `Bytecode` fornecem detalhes de instruções de bytecode como instâncias de `Instruction`:

`class` `dis.`**`Instruction`**

Detalhes de uma operação em bytecode

`opcode`

código numérico da operação, correspondendo aos valores dos opcodes listados abaixo e aos valores dos bytecodes nas *Opcode collections*.

`opname`

nome legível por humanos para a operação

`baseopcode`

código numérico para a operação base caso a operação seja especializada; caso contrário, igual ao `opcode`

`baseopname`

nome legível por humanos para a operação base caso a operação seja especializada; caso contrário, igual ao `opname`

`arg`

argumento numérico para a operação (se houver), caso contrário `None`

oparg

apelido para *arg*

argval

valor resolvido do argumento (se houver), caso contrário `None`

argrepr

descrição legível por humanos do argumento da operação (se houver), caso contrário uma string vazia.

offset

índice de início da operação dentro da sequência de bytecodes

start_offset

índice de início da operação dentro da sequência de bytecodes, incluindo as operações de `EXTENDED_ARG` prefixadas, caso presentes; caso contrário, igual a *offset*

cache_offset

índice de início das entradas de cache que seguem a operação

end_offset

índice de fim das entradas de cache que seguem a operação

starts_line

`True` se esse opcode inicia uma linha, `False` caso contrário

line_number

linha de código-fonte associada a esse opcode (se houver), senão `None`

is_jump_target

`True` se algum outro código pula para cá, senão `False`

jump_target

índice do bytecode que é alvo do pulo caso essa seja uma operação de pulo, caso contrário `None`

positions

objeto *dis.Positions* contendo os pontos de início e fim cobertos por esta instrução.

cache_info

Informações sobre as entradas de cache desta instrução, como tripletos do formato `(nome, tamanho, dados)`, onde `nome` e `tamanho` descrevem o formato do cache e dados são o conteúdo do cache. `cache_info` é `None` se a instrução não tiver caches.

Adicionado na versão 3.4.

Alterado na versão 3.11: Adicionado o campo `positions`.

Alterado na versão 3.13: Alterado o campo `starts_line`.

Adicionados os campos `start_offset`, `cache_offset`, `end_offset`, `baseopname`, `baseopcode`, `jump_target`, `oparg`, `line_number` e `cache_info`.

class dis.**Positions**

Caso a informação não esteja disponível, alguns campos podem ser `None`.

lineno

end_lineno

col_offset

end_col_offset

Adicionado na versão 3.11.

O compilador de Python atualmente gera as seguintes instruções de bytecode.

Instruções gerais

A seguir, vamos usar `STACK` para nos referirmos à pilha do interpretador, e vamos descrever operações nela como se ela fosse uma lista do Python. Nessa linguagem, `STACK[-1]` é o topo da pilha.

NOP

Código para não fazer nada. Usado como espaço reservado pelo otimizador de bytecode, e para gerar eventos de rastreamento de linha.

NOT_TAKEN

Código sem ação. Usado pelo interpretador para registrar os eventos *BRANCH_LEFT* e *BRANCH_RIGHT* para *sys.monitoring*.

Adicionado na versão 3.14.

POP_ITER

Remove o iterador do topo da pilha.

Adicionado na versão 3.14.

POP_TOP

Remove o item no topo da pilha:

```
STACK.pop()
```

END_FOR

Remove o item no topo da pilha. Equivalente a `POP_TOP`. Usado como limpeza ao final de laços, o que explica o nome.

Adicionado na versão 3.12.

END_SEND

Implementa `del STACK[-2]`. Usado como limpeza quando um gerador termina.

Adicionado na versão 3.12.

COPY (*i*)

Coloca o i-ésimo item no topo da pilha sem removê-lo da sua posição original:

```
assert i > 0
STACK.append(STACK[-i])
```

Adicionado na versão 3.11.

SWAP (*i*)

Troca o topo da pilha de lugar com o i-ésimo elemento.

```
STACK[-i], STACK[-1] = STACK[-1], STACK[-i]
```

Adicionado na versão 3.11.

CACHE

Ao invés de ser uma instrução de fato, este opcode é usado para demarcar espaço extra para o interpretador armazenar dados úteis diretamente no próprio bytecode. É escondido automaticamente por todas as utilidades do `dis`, mas pode ser visualizado com `show_caches=True`.

Do ponto de vista lógico, este espaço faz parte da instrução anterior. Muitos opcodes esperam ser seguidos por um número exato de caches, e instruem o interpretador a pulá-los em tempo de execução.

Caches populados podem se parecer com qualquer instrução, de forma que ler ou modificar bytecode adaptativo bruto contendo dados "quickened" requer muito cuidado.

Adicionado na versão 3.11.

Operações unárias

Operações unárias tiram o topo da pilha, aplicam a operação, e põem o resultado de volta na pilha.

UNARY_NEGATIVE

Implementa `STACK[-1] = -STACK[-1]`.

UNARY_NOT

Implementa `STACK[-1] = not STACK[-1]`.

Alterado na versão 3.13: Essa instrução agora requer que o operando seja exatamente do tipo *bool*.

UNARY_INVERT

Implementa `STACK[-1] = ~STACK[-1]`.

GET_ITER

Implementa `STACK[-1] = iter(STACK[-1])`.

GET_YIELD_FROM_ITER

Se `STACK[-1]` for um *iterador gerador* ou um objeto *corrotina*, nada acontece. Caso contrário, implementa `STACK[-1] = iter(STACK[-1])`.

Adicionado na versão 3.5.

TO_BOOL

Implementa `STACK[-1] = bool(STACK[-1])`.

Adicionado na versão 3.13.

Operações binárias e internas

Operações binárias removem os dois itens no topo da pilha (`STACK[-1]` e `STACK[-2]`). A operação é realizada, e o resultado é colocado de volta na pilha.

As operações internas são como as operações binárias, só que a operação é feita internamente caso suportado por `STACK[-2]`, e o `STACK[-1]` resultante pode ser (mas não necessariamente é) o `STACK[-2]` original.

BINARY_OP (*op*)

Implementa os operadores binários e locais (depende do valor de *op*):

```
rhs = STACK.pop()
lhs = STACK.pop()
STACK.append(lhs op rhs)
```

Adicionado na versão 3.11.

Alterado na versão 3.14: Com oparg :`NB_SUBSCR`, implementa subscrito binário (substitui o opcode `BINARY_SUBSCR`)

STORE_SUBSCR

Implementa:

```
key = STACK.pop()
container = STACK.pop()
value = STACK.pop()
container[key] = value
```

DELETE_SUBSCR

Implementa:

```
key = STACK.pop()
container = STACK.pop()
del container[key]
```

BINARY_SLICE

Implementa:

```
end = STACK.pop()
start = STACK.pop()
container = STACK.pop()
STACK.append(container[start:end])
```

Adicionado na versão 3.12.

STORE_SLICE

Implementa:

```
end = STACK.pop()
start = STACK.pop()
container = STACK.pop()
values = STACK.pop()
container[start:end] = value
```

Adicionado na versão 3.12.

Opcodes para corrotinas

GET_AWAITABLE(*where*)

Implementa `STACK[-1] = get_awaitable(STACK[-1])`, onde `get_awaitable(o)` retorna `o` se `o` for um objeto de corrotina ou um gerador com o sinalizador *CO_ITERABLE_COROUTINE*, ou então resolve `o.__await__`.

Se o operando `where` não for zero, ele indica onde a instrução ocorre:

- `1`: Após uma chamada a `__aenter__`
- `2`: Após uma chamada a `__aexit__`

Adicionado na versão 3.5.

Alterado na versão 3.11: Anteriormente, esta instrução não tinha um oparg.

GET_AITER

Implementa `STACK[-1] = STACK[-1].__aiter__()`.

Adicionado na versão 3.5.

Alterado na versão 3.7: Não é mais aceitado que o `__aiter__` retorne objetos aguardáveis.

GET_ANEXT

Implementa `STACK.append(get_awaitable(STACK[-1].__anext__()))`. Veja `GET_AWAITABLE` para o significado de `get_awaitable`.

Adicionado na versão 3.5.

END_ASYNC_FOR

Termina um laço `async for`. Trata exceções levantadas ao aguardar um item seguinte. A pilha contém o iterável async em `STACK[-2]` e a exceção levantada em `STACK[-1]`. Ambos são retirados. Se a exceção não for *StopAsyncIteration*, ela é re-levantada.

Adicionado na versão 3.8.

Alterado na versão 3.11: A representação da exceção na pilha consiste agora de um item, ao invés de três.

CLEANUP_THROW

Trata uma exceção levantada durante um chamada a `throw()` ou `close()` através do quadro atual. Se `STACK[-1]` for uma instância de *StopIteration*, remove três valores da pilha e põe de volta o seu membro `value`. Caso contrário, re-levanta `STACK[-1]`.

Adicionado na versão 3.12.

Opcodes genéricos

SET_ADD (*i*)

Implementa:

```
item = STACK.pop()
set.add(STACK[-i], item)
```

Usado para implementar compreensões de conjuntos.

LIST_APPEND (*i*)

Implementa:

```
item = STACK.pop()
list.append(STACK[-i], item)
```

Usado para implementar compreensões de lista.

MAP_ADD (*i*)

Implementa:

```
value = STACK.pop()
key = STACK.pop()
dict.__setitem__(STACK[-i], key, value)
```

Usado para implementar compreensões de dicionário.

Adicionado na versão 3.1.

Alterado na versão 3.8: O valor do mapa é `STACK[-1]`, e a sua chave, `STACK[-2]`. Antes, eles estavam ao contrário.

Para as instruções *SET_ADD*, *LIST_APPEND* e *MAP_ADD*, o valor ou par chave/valor é removido da pilha, mas o objeto de contêiner continua na pilha para que ele esteja disponível para as iterações seguintes do laço.

RETURN_VALUE

Retorna `STACK[-1]` para quem chamou a função.

YIELD_VALUE

Gera `STACK.pop()` a partir de um *gerador*.

Alterado na versão 3.11: oparg definido como sendo a profundidade da pilha.

Alterado na versão 3.12: oparg definido como sendo a profundidade do bloco exception, para o fechamento eficiente de geradores.

Alterado na versão 3.13: oparg é `1` caso esta instrução seja parte de um yield-from ou await, e `0` caso contrário.

SETUP_ANNOTATIONS

Verifica se `__annotations__` está definido em `locals()` e, se não estiver, é inicializado como um `dict` vazio. Este opcode é emitido somente se o corpo de uma classe ou módulo contém *anotações de variáveis* estaticamente.

Adicionado na versão 3.6.

POP_EXCEPT

Remove o valor no topo da pilha, o qual é usado para restaurar o estado de exceção.

Alterado na versão 3.11: A representação da exceção na pilha consiste agora de um item, ao invés de três.

RERAISE

Re-levanta a exceção que se encontra no topo da pilha. Se o oparg não for zero, remove um valor adicional do topo da pilha, o qual é atribuído ao `f_lasti` do quadro atual.

Adicionado na versão 3.9.

Alterado na versão 3.11: A representação da exceção na pilha consiste agora de um item, ao invés de três.

PUSH_EXC_INFO

Remove um valor do topo da pilha. Põe a exceção atual no topo da pilha. Põe de volta no topo da pilha o valor que foi removido inicialmente. Usado em tratadores de exceções.

Adicionado na versão 3.11.

CHECK_EXC_MATCH

Verifica correspondências de exceções em `except`. Testa de `STACK[-2]` é uma exceção que corresponde a `STACK[-1]`. Remove `STACK[-1]` do topo da pilha, e põe no seu lugar o resultado booleano do teste.

Adicionado na versão 3.11.

CHECK_EG_MATCH

Verifica correspondências de exceções em `except*`. Aplica `split(STACK[-1])` no grupo de exceções que representa `STACK[-2]`.

No caso de uma correspondência, remove dois itens do topo da pilha e põe nela o subgrupo que falhou a correspondência (`None` caso a correspondência tenha sido total), seguido pelo subgrupo que correspondeu. Quando não há correspondência nenhuma, remove um item (o tipo da correspondência) e põe `None` no seu lugar.

Adicionado na versão 3.11.

WITH_EXCEPT_START

Chama a função na posição 4 da pilha com argumentos (tipo, val, tb) representando a exceção no topo da pilha. Usado para implementar a chamada `context_manager.__exit__(*exc_info())` quando uma exceção ocorreu em uma instrução `with`.

Adicionado na versão 3.9.

Alterado na versão 3.11: A função `__exit__` fica agora na posição 4 pilha, ao invés da 7. A representação da exceção pilha consiste agora de um item, não três.

LOAD_COMMON_CONSTANT

Envia uma constante comum para a pilha. O interpretador contém uma lista codificada de constantes suportadas por esta instrução. Usado pela instrução `assert` para carregar *AssertionError*.

Adicionado na versão 3.14.

LOAD_BUILD_CLASS

Põe a função `builtins.__build_class__()` no topo da pilha. Ela será chamada posteriormente para construir uma classe.

GET_LEN

Executa `STACK.append(len(STACK[-1]))`. Usado em instruções `match` onde a comparação com a estrutura do padrão é necessária.

Adicionado na versão 3.10.

MATCH_MAPPING

Se `STACK[-1]` for uma instância de *collections.abc.Mapping* (ou, de forma mais técnica: se tiver o sinalizador `Py_TPFLAGS_MAPPING` definido no seu `tp_flags`), põe `True` no topo da pilha. Caso contrário, põe `False`.

Adicionado na versão 3.10.

MATCH_SEQUENCE

Se `STACK[-1]` for uma instância de *collections.abc.Sequence* e *não* for uma instância de *str*/*bytes*/*bytearray* (ou, de forma mais técnica: se tiver o sinalizador `Py_TPFLAGS_SEQUENCE` definido no seu `tp_flags`), põe `True` no topo da pilha. Caso contrário, põe `False`.

Adicionado na versão 3.10.

MATCH_KEYS

`STACK[-1]` é uma tupla de chaves de um mapeamento, e `STACK[-2]` é o sujeito de uma correspondência. Se `STACK[-2]` contiver todas as chaves em `STACK[-1]`, põe no topo da pilha um *tuple* contendo os valores correspondentes. Caso contrário, põe `None`.

Adicionado na versão 3.10.

Alterado na versão 3.11: Anteriormente, essa instrução também colocava na pilha um valor booleano indicando sucesso (`True`) ou falha (`False`).

STORE_NAME (*namei*)

Implementa `name = STACK.pop()`. *namei* é o índice de *name* no atributo `co_names` do objeto de código. O compilador tenta usar *STORE_FAST* ou *STORE_GLOBAL* se possível.

DELETE_NAME (*namei*)

Implementa `del name`, onde *namei* é o índice no atributo `co_names` do objeto de código.

UNPACK_SEQUENCE (*count*)

Desempacota `STACK[-1]` em *count* valores individuais, os quais são postos na pilha da direita para a esquerda. Requer que haja exatamente *count* valores:

```
assert(len(STACK[-1]) == count)
STACK.extend(STACK.pop()[:-count-1:-1])
```

UNPACK_EX (*counts*)

Implementa atribuição com um alvo estrelado: desempacota o iterável `STACK[-1]` em valores individuais, sendo que pode haver menos valores do que itens no iterável: um dos novos valores será a lista de todos os itens que sobraram.

A quantidade de valores antes e após o valor que será a lista é limitada a 255.

A quantidade de valores antes do valor lista é passada no argumento do opcode. A quantidade de valores após a lista, se houver, é passada usando um `EXTENDED_ARG`. A consequência é que o argumento pode ser visto como um valor de dois bytes, onde o byte "de baixo" de *counts* é a quantidade de valores antes do valor lista, e o byte "de cima" de *counts*, a quantidade após.

Os valores extraídos são postos na pilha da direita para a esquerda, ou seja, após executar `a, *b, c = d` os valores serão armazenados como `STACK.extend((a, b, c))`.

STORE_ATTR (*namei*)

Implementa:

```
obj = STACK.pop()
value = STACK.pop()
obj.name = value
```

onde *namei* é o índice do nome no `co_names` do objeto de código.

DELETE_ATTR (*namei*)

Implementa:

```
obj = STACK.pop()
del obj.name
```

onde *namei* é o índice do nome no `co_names` do objeto de código.

STORE_GLOBAL (*namei*)

Funciona como o *STORE_NAME*, mas o nome é armazenado com um nome global.

DELETE_GLOBAL (*namei*)

Funciona como o *DELETE_NAME*, mas deleta um nome global.

LOAD_CONST (*consti*)

Põe `co_consts[consti]` no topo da pilha.

LOAD_SMALL_INT (*i*)

Coloca o inteiro `i` na pilha. `i` deve estar em `range(256)`

Adicionado na versão 3.14.

LOAD_NAME (*namei*)

Põe no topo da pilha o valor associado a `co_names[namei]`. O nome é procurado nos locais, nos globais, e então nos embutidos.

LOAD_LOCALS

Envia uma referência ao dicionário local para a pilha. Isso é usado para preparar dicionários de espaço de nomes para *LOAD_FROM_DICT_OR_DEREF* e *LOAD_FROM_DICT_OR_GLOBALS*.

Adicionado na versão 3.12.

LOAD_FROM_DICT_OR_GLOBALS (*i*)

Extrai um mapeamento da pilha e procura o valor de `co_names[namei]`. Se o nome não for encontrado, procura-o nas variáveis globais e, em seguida, nas variáveis embutidas, semelhante a *LOAD_GLOBAL*. Isso é usado para carregar variáveis globais em escopos de anotação dentro dos corpos de classe.

Adicionado na versão 3.12.

BUILD_TEMPLATE

Constrói uma nova instância *Template* a partir de uma tupla de strings e uma tupla de interpolações e envia o objeto resultante para a pilha:

```
interpolations = STACK.pop()
strings = STACK.pop()
STACK.append(_build_template(strings, interpolations))
```

Adicionado na versão 3.14.

BUILD_INTERPOLATION (*format*)

Constrói uma nova instância *Interpolation* a partir de um valor e sua expressão de origem e envia o objeto resultante para a pilha.

Se nenhuma especificação de conversão ou formato estiver presente, `format` será definido como `2`.

If the low bit of `format` is set, it indicates that the interpolation contains a format specification.

If `format >> 2` is non-zero, it indicates that the interpolation contains a conversion. The value of `format >> 2` is the conversion type (`0` for no conversion, `1` for `!s`, `2` for `!r`, and `3` for `!a`):

```
conversion = format >> 2
if format & 1:
    format_spec = STACK.pop()
else:
    format_spec = None
expression = STACK.pop()
value = STACK.pop()
STACK.append(_build_interpolation(value, expression, conversion, format_spec))
```

Adicionado na versão 3.14.

BUILD_TUPLE (*count*)

Creates a tuple consuming *count* items from the stack, and pushes the resulting tuple onto the stack:

```
if count == 0:
    value = ()
else:
```

(continua na próxima página)

(continuação da página anterior)

```
    value = tuple(STACK[-count:])
    STACK = STACK[:-count]

STACK.append(value)
```

BUILD_LIST(*count*)

Works as *BUILD_TUPLE*, but creates a list.

BUILD_SET(*count*)

Works as *BUILD_TUPLE*, but creates a set.

BUILD_MAP(*count*)

Pushes a new dictionary object onto the stack. Pops `2 * count` items so that the dictionary holds *count* entries: `{..., STACK[-4]: STACK[-3], STACK[-2]: STACK[-1]}`.

Alterado na versão 3.5: The dictionary is created from stack items instead of creating an empty dictionary pre-sized to hold *count* items.

BUILD_STRING(*count*)

Concatenates *count* strings from the stack and pushes the resulting string onto the stack.

Adicionado na versão 3.6.

LIST_EXTEND(*i*)

Implementa:

```
seq = STACK.pop()
list.extend(STACK[-i], seq)
```

Used to build lists.

Adicionado na versão 3.9.

SET_UPDATE(*i*)

Implementa:

```
seq = STACK.pop()
set.update(STACK[-i], seq)
```

Used to build sets.

Adicionado na versão 3.9.

DICT_UPDATE(*i*)

Implementa:

```
map = STACK.pop()
dict.update(STACK[-i], map)
```

Used to build dicts.

Adicionado na versão 3.9.

DICT_MERGE(*i*)

Like *DICT_UPDATE* but raises an exception for duplicate keys.

Adicionado na versão 3.9.

LOAD_ATTR(*namei*)

If the low bit of `namei` is not set, this replaces `STACK[-1]` with `getattr(STACK[-1], co_names[namei>>1])`.

If the low bit of `namei` is set, this will attempt to load a method named `co_names[namei>>1]` from the `STACK[-1]` object. `STACK[-1]` is popped. This bytecode distinguishes two cases: if `STACK[-1]` has a method with the correct name, the bytecode pushes the unbound method and `STACK[-1]`. `STACK[-1]` will be used as the first argument (`self`) by *CALL* or *CALL_KW* when calling the unbound method. Otherwise, `NULL` and the object returned by the attribute lookup are pushed.

Alterado na versão 3.12: If the low bit of `namei` is set, then a `NULL` or `self` is pushed to the stack before the attribute or unbound method respectively.

LOAD_SUPER_ATTR (*namei*)

This opcode implements *super()*, both in its zero-argument and two-argument forms (e.g. `super().method()`, `super().attr` and `super(cls, self).method()`, `super(cls, self).attr`).

It pops three values from the stack (from top of stack down):

- `self`: the first argument to the current method
- `cls`: the class within which the current method was defined
- the global `super`

With respect to its argument, it works similarly to *LOAD_ATTR*, except that `namei` is shifted left by 2 bits instead of 1.

The low bit of `namei` signals to attempt a method load, as with *LOAD_ATTR*, which results in pushing `NULL` and the loaded method. When it is unset a single value is pushed to the stack.

The second-low bit of `namei`, if set, means that this was a two-argument call to *super()* (unset means zero-argument).

Adicionado na versão 3.12.

COMPARE_OP (*opname*)

Performs a Boolean operation. The operation name can be found in `cmp_op[opname >> 5]`. If the fifth-lowest bit of `opname` is set (`opname & 16`), the result should be coerced to `bool`.

Alterado na versão 3.13: The fifth-lowest bit of the oparg now indicates a forced conversion to *bool*.

IS_OP (*invert*)

Performs `is` comparison, or `is not` if `invert` is 1.

Adicionado na versão 3.9.

CONTAINS_OP (*invert*)

Performs `in` comparison, or `not in` if `invert` is 1.

Adicionado na versão 3.9.

IMPORT_NAME (*namei*)

Imports the module `co_names[namei]`. `STACK[-1]` and `STACK[-2]` are popped and provide the *fromlist* and *level* arguments of *__import__()*. The module object is pushed onto the stack. The current namespace is not affected: for a proper import statement, a subsequent *STORE_FAST* instruction modifies the namespace.

IMPORT_FROM (*namei*)

Loads the attribute `co_names[namei]` from the module found in `STACK[-1]`. The resulting object is pushed onto the stack, to be subsequently stored by a *STORE_FAST* instruction.

JUMP_FORWARD (*delta*)

Increments bytecode counter by *delta*.

JUMP_BACKWARD (*delta*)

Decrements bytecode counter by *delta*. Checks for interrupts.

Adicionado na versão 3.11.

JUMP_BACKWARD_NO_INTERRUPT (*delta*)

Decrements bytecode counter by *delta*. Does not check for interrupts.

Adicionado na versão 3.11.

POP_JUMP_IF_TRUE (*delta*)

If `STACK[-1]` is true, increments the bytecode counter by *delta*. `STACK[-1]` is popped.

Alterado na versão 3.11: The oparg is now a relative delta rather than an absolute target. This opcode is a pseudo-instruction, replaced in final bytecode by the directed versions (forward/backward).

Alterado na versão 3.12: This is no longer a pseudo-instruction.

Alterado na versão 3.13: Essa instrução agora requer que o operando seja exatamente do tipo *bool*.

POP_JUMP_IF_FALSE (*delta*)

If `STACK[-1]` is false, increments the bytecode counter by *delta*. `STACK[-1]` is popped.

Alterado na versão 3.11: The oparg is now a relative delta rather than an absolute target. This opcode is a pseudo-instruction, replaced in final bytecode by the directed versions (forward/backward).

Alterado na versão 3.12: This is no longer a pseudo-instruction.

Alterado na versão 3.13: Essa instrução agora requer que o operando seja exatamente do tipo *bool*.

POP_JUMP_IF_NOT_NONE (*delta*)

If `STACK[-1]` is not `None`, increments the bytecode counter by *delta*. `STACK[-1]` is popped.

Adicionado na versão 3.11.

Alterado na versão 3.12: This is no longer a pseudo-instruction.

POP_JUMP_IF_NONE (*delta*)

If `STACK[-1]` is `None`, increments the bytecode counter by *delta*. `STACK[-1]` is popped.

Adicionado na versão 3.11.

Alterado na versão 3.12: This is no longer a pseudo-instruction.

FOR_ITER (*delta*)

`STACK[-1]` is an *iterator*. Call its *__next__()* method. If this yields a new value, push it on the stack (leaving the iterator below it). If the iterator indicates it is exhausted then the byte code counter is incremented by *delta*.

Alterado na versão 3.12: Up until 3.11 the iterator was popped when it was exhausted.

LOAD_GLOBAL (*namei*)

Loads the global named `co_names[namei>>1]` onto the stack.

Alterado na versão 3.11: If the low bit of `namei` is set, then a `NULL` is pushed to the stack before the global variable.

LOAD_FAST (*var_num*)

Pushes a reference to the local `co_varnames[var_num]` onto the stack.

Alterado na versão 3.12: This opcode is now only used in situations where the local variable is guaranteed to be initialized. It cannot raise *UnboundLocalError*.

LOAD_FAST_BORROW (*var_num*)

Pushes a borrowed reference to the local `co_varnames[var_num]` onto the stack.

Adicionado na versão 3.14.

LOAD_FAST_LOAD_FAST (*var_nums*)

Pushes references to `co_varnames[var_nums >> 4]` and `co_varnames[var_nums & 15]` onto the stack.

Adicionado na versão 3.13.

LOAD_FAST_BORROW_LOAD_FAST_BORROW (*var_nums*)

Pushes borrowed references to `co_varnames[var_nums >> 4]` and `co_varnames[var_nums & 15]` onto the stack.

Adicionado na versão 3.14.

LOAD_FAST_CHECK (*var_num*)

Pushes a reference to the local `co_varnames[var_num]` onto the stack, raising an *UnboundLocalError* if the local variable has not been initialized.

Adicionado na versão 3.12.

LOAD_FAST_AND_CLEAR (*var_num*)

Pushes a reference to the local `co_varnames[var_num]` onto the stack (or pushes `NULL` onto the stack if the local variable has not been initialized) and sets `co_varnames[var_num]` to `NULL`.

Adicionado na versão 3.12.

STORE_FAST (*var_num*)

Stores `STACK.pop()` into the local `co_varnames[var_num]`.

STORE_FAST_STORE_FAST (*var_nums*)

Stores `STACK[-1]` into `co_varnames[var_nums >> 4]` and `STACK[-2]` into `co_varnames[var_nums & 15]`.

Adicionado na versão 3.13.

STORE_FAST_LOAD_FAST (*var_nums*)

Stores `STACK.pop()` into the local `co_varnames[var_nums >> 4]` and pushes a reference to the local `co_varnames[var_nums & 15]` onto the stack.

Adicionado na versão 3.13.

DELETE_FAST (*var_num*)

Deletes local `co_varnames[var_num]`.

MAKE_CELL (*i*)

Creates a new cell in slot `i`. If that slot is nonempty then that value is stored into the new cell.

Adicionado na versão 3.11.

LOAD_DEREF (*i*)

Loads the cell contained in slot `i` of the "fast locals" storage. Pushes a reference to the object the cell contains on the stack.

Alterado na versão 3.11: `i` is no longer offset by the length of `co_varnames`.

LOAD_FROM_DICT_OR_DEREF (*i*)

Pops a mapping off the stack and looks up the name associated with slot `i` of the "fast locals" storage in this mapping. If the name is not found there, loads it from the cell contained in slot `i`, similar to *LOAD_DEREF*. This is used for loading *closure variables* in class bodies (which previously used `LOAD_CLASSDEREF`) and in annotation scopes within class bodies.

Adicionado na versão 3.12.

STORE_DEREF (*i*)

Stores `STACK.pop()` into the cell contained in slot `i` of the "fast locals" storage.

Alterado na versão 3.11: `i` is no longer offset by the length of `co_varnames`.

DELETE_DEREF (*i*)

Empties the cell contained in slot `i` of the "fast locals" storage. Used by the `del` statement.

Adicionado na versão 3.2.

Alterado na versão 3.11: `i` is no longer offset by the length of `co_varnames`.

COPY_FREE_VARS (*n*)

Copia as `n` *variáveis livres (de clausura)* do fechamento para o quadro. Remove a necessidade de código especial do lado do chamador ao chamar closures.

Adicionado na versão 3.11.

RAISE_VARARGS (*argc*)

Raises an exception using one of the 3 forms of the `raise` statement, depending on the value of *argc*:

- 0: `raise` (re-raise previous exception)
- 1: `raise STACK[-1]` (raise exception instance or type at `STACK[-1]`)
- 2: `raise STACK[-2] from STACK[-1]` (raise exception instance or type at `STACK[-2]` with `__cause__` set to `STACK[-1]`)

CALL (*argc*)

Calls a callable object with the number of arguments specified by `argc`. On the stack are (in ascending order):

- The callable
- `self` or `NULL`
- The remaining positional arguments

`argc` is the total of the positional arguments, excluding `self`.

`CALL` pops all arguments and the callable object off the stack, calls the callable object with those arguments, and pushes the return value returned by the callable object.

Adicionado na versão 3.11.

Alterado na versão 3.13: The callable now always appears at the same position on the stack.

Alterado na versão 3.13: Calls with keyword arguments are now handled by *CALL_KW*.

CALL_KW (*argc*)

Calls a callable object with the number of arguments specified by `argc`, including one or more named arguments. On the stack are (in ascending order):

- The callable
- `self` or `NULL`
- The remaining positional arguments
- The named arguments
- A *tuple* of keyword argument names

`argc` is the total of the positional and named arguments, excluding `self`. The length of the tuple of keyword argument names is the number of named arguments.

`CALL_KW` pops all arguments, the keyword names, and the callable object off the stack, calls the callable object with those arguments, and pushes the return value returned by the callable object.

Adicionado na versão 3.13.

CALL_FUNCTION_EX (*flags*)

Calls a callable object with variable set of positional and keyword arguments. If the lowest bit of *flags* is set, the top of the stack contains a mapping object containing additional keyword arguments. Before the callable is called, the mapping object and iterable object are each "unpacked" and their contents passed in as keyword and positional arguments respectively. `CALL_FUNCTION_EX` pops all arguments and the callable object off the stack, calls the callable object with those arguments, and pushes the return value returned by the callable object.

Adicionado na versão 3.6.

PUSH_NULL

Pushes a `NULL` to the stack. Used in the call sequence to match the `NULL` pushed by `LOAD_METHOD` for non-method calls.

Adicionado na versão 3.11.

MAKE_FUNCTION

Pushes a new function object on the stack built from the code object at `STACK[-1]`.

Alterado na versão 3.10: Flag value `0x04` is a tuple of strings instead of dictionary

Alterado na versão 3.11: Qualified name at `STACK[-1]` was removed.

Alterado na versão 3.13: Extra function attributes on the stack, signaled by oparg flags, were removed. They now use *SET_FUNCTION_ATTRIBUTE*.

SET_FUNCTION_ATTRIBUTE(*flag*)

Sets an attribute on a function object. Expects the function at `STACK[-1]` and the attribute value to set at `STACK[-2]`; consumes both and leaves the function at `STACK[-1]`. The flag determines which attribute to set:

- `0x01` a tuple of default values for positional-only and positional-or-keyword parameters in positional order
- `0x02` a dictionary of keyword-only parameters' default values
- `0x04` a tuple of strings containing parameters' annotations
- `0x08` a tuple containing cells for free variables, making a closure

Adicionado na versão 3.13.

BUILD_SLICE(*argc*)

Pushes a slice object on the stack. *argc* must be 2 or 3. If it is 2, implements:

```
end = STACK.pop()
start = STACK.pop()
STACK.append(slice(start, end))
```

if it is 3, implements:

```
step = STACK.pop()
end = STACK.pop()
start = STACK.pop()
STACK.append(slice(start, end, step))
```

See the *slice()* built-in function for more information.

EXTENDED_ARG(*ext*)

Prefixes any opcode which has an argument too big to fit into the default one byte. *ext* holds an additional byte which act as higher bits in the argument. For each opcode, at most three prefixal `EXTENDED_ARG` are allowed, forming an argument from two-byte to four-byte.

CONVERT_VALUE(*oparg*)

Convert value to a string, depending on `oparg`:

```
value = STACK.pop()
result = func(value)
STACK.append(result)
```

- `oparg == 1`: call *str()* on *value*
- `oparg == 2`: call *repr()* on *value*
- `oparg == 3`: call *ascii()* on *value*

Used for implementing formatted string literals (f-strings).

Adicionado na versão 3.13.

FORMAT_SIMPLE

Formats the value on top of stack:

```
value = STACK.pop()
result = value.__format__("")
STACK.append(result)
```

Used for implementing formatted string literals (f-strings).

Adicionado na versão 3.13.

FORMAT_WITH_SPEC

Formats the given value with the given format spec:

```
spec = STACK.pop()
value = STACK.pop()
result = value.__format__(spec)
STACK.append(result)
```

Used for implementing formatted string literals (f-strings).

Adicionado na versão 3.13.

MATCH_CLASS (*count*)

`STACK[-1]` is a tuple of keyword attribute names, `STACK[-2]` is the class being matched against, and `STACK[-3]` is the match subject. *count* is the number of positional sub-patterns.

Pop `STACK[-1]`, `STACK[-2]`, and `STACK[-3]`. If `STACK[-3]` is an instance of `STACK[-2]` and has the positional and keyword attributes required by *count* and `STACK[-1]`, push a tuple of extracted attributes. Otherwise, push `None`.

Adicionado na versão 3.10.

Alterado na versão 3.11: Anteriormente, essa instrução também colocava na pilha um valor booleano indicando sucesso (`True`) ou falha (`False`).

RESUME (*context*)

A no-op. Performs internal tracing, debugging and optimization checks.

The `context` operand consists of two parts. The lowest two bits indicate where the `RESUME` occurs:

- `0` The start of a function, which is neither a generator, coroutine nor an async generator
- `1` Depois de uma expressão `yield`
- `2` After a `yield from` expression
- `3` Depois de uma expressão `await`

The next bit is `1` if the RESUME is at except-depth `1`, and `0` otherwise.

Adicionado na versão 3.11.

Alterado na versão 3.13: The oparg value changed to include information about except-depth

RETURN_GENERATOR

Create a generator, coroutine, or async generator from the current frame. Used as first opcode of in code object for the above mentioned callables. Clear the current frame and return the newly created generator.

Adicionado na versão 3.11.

SEND (*delta*)

Equivalent to `STACK[-1] = STACK[-2].send(STACK[-1])`. Used in `yield from` and `await` statements.

If the call raises *StopIteration*, pop the top value from the stack, push the exception's `value` attribute, and increment the bytecode counter by *delta*.

Adicionado na versão 3.11.

HAVE_ARGUMENT

This is not really an opcode. It identifies the dividing line between opcodes in the range [0,255] which don't use their argument and those that do (`< HAVE_ARGUMENT` and `>= HAVE_ARGUMENT`, respectively).

If your application uses pseudo instructions or specialized instructions, use the *hasarg* collection instead.

Alterado na versão 3.6: Now every instruction has an argument, but opcodes `< HAVE_ARGUMENT` ignore it. Before, only opcodes `>= HAVE_ARGUMENT` had an argument.

Alterado na versão 3.12: Pseudo instructions were added to the *dis* module, and for them it is not true that comparison with `HAVE_ARGUMENT` indicates whether they use their arg.

Descontinuado desde a versão 3.13: Use *hasarg* instead.

CALL_INTRINSIC_1

Calls an intrinsic function with one argument. Passes `STACK[-1]` as the argument and sets `STACK[-1]` to the result. Used to implement functionality that is not performance critical.

The operand determines which intrinsic function is called:

Operand	Descrição
`INTRINSIC_1_INVAL`	Not valid
`INTRINSIC_PRINT`	Prints the argument to standard out. Used in the REPL.
`INTRINSIC_IMPORT_`	Performs `import *` for the named module.
`INTRINSIC_STOPITE`	Extracts the return value from a `StopIteration` exception.
`INTRINSIC_ASYNC_G`	Wraps an async generator value
`INTRINSIC_UNARY_P`	Performs the unary + operation
`INTRINSIC_LIST_TO_`	Converts a list to a tuple
`INTRINSIC_TYPEVAR`	Cria um *typing.TypeVar*
`INTRINSIC_PARAMSP`	Cria um *typing.ParamSpec*
`INTRINSIC_TYPEVAR`	Cria um *typing.TypeVarTuple*
`INTRINSIC_SUBSCRI`	Returns *typing.Generic* subscripted with the argument
`INTRINSIC_TYPEALI`	Creates a *typing.TypeAliasType*; used in the `type` statement. The argument is a tuple of the type alias's name, type parameters, and value.

Adicionado na versão 3.12.

CALL_INTRINSIC_2

Calls an intrinsic function with two arguments. Used to implement functionality that is not performance critical:

```
arg2 = STACK.pop()
arg1 = STACK.pop()
result = intrinsic2(arg1, arg2)
STACK.append(result)
```

The operand determines which intrinsic function is called:

Operand	Descrição
INTRINSIC_2_INVALID	Not valid
INTRINSIC_PREP_RERAISE_STAR	Calculates the *ExceptionGroup* to raise from a try-except*.
INTRINSIC_TYPEVAR_WITH_BOUND	Creates a *typing.TypeVar* with a bound.
INTRINSIC_TYPEVAR_WITH_CONSTRAIN'	Creates a *typing.TypeVar* with constraints.
INTRINSIC_SET_FUNCTION_TYPE_PARAI	Sets the __type_params__ attribute of a function.

Adicionado na versão 3.12.

LOAD_SPECIAL

Performs special method lookup on STACK[-1]. If type(STACK[-1]).__xxx__ is a method, leave type(STACK[-1]).__xxx__; STACK[-1] on the stack. If type(STACK[-1]).__xxx__ is not a method, leave STACK[-1].__xxx__; NULL on the stack.

Adicionado na versão 3.14.

Pseudo-instructions

These opcodes do not appear in Python bytecode. They are used by the compiler but are replaced by real opcodes or removed before bytecode is generated.

SETUP_FINALLY (*target*)

Set up an exception handler for the following code block. If an exception occurs, the value stack level is restored to its current state and control is transferred to the exception handler at target.

SETUP_CLEANUP (*target*)

Like SETUP_FINALLY, but in case of an exception also pushes the last instruction (lasti) to the stack so that RERAISE can restore it. If an exception occurs, the value stack level and the last instruction on the frame are restored to their current state, and control is transferred to the exception handler at target.

SETUP_WITH (*target*)

Like SETUP_CLEANUP, but in case of an exception one more item is popped from the stack before control is transferred to the exception handler at target.

This variant is used in with and async with constructs, which push the return value of the context manager's __enter__() or __aenter__() to the stack.

POP_BLOCK

Marks the end of the code block associated with the last SETUP_FINALLY, SETUP_CLEANUP or SETUP_WITH.

LOAD_CONST_IMMORTAL (*consti*)

Funciona como *LOAD_CONST*, mas é mais eficiente para objetos imortais.

JUMP

JUMP_NO_INTERRUPT

Undirected relative jump instructions which are replaced by their directed (forward/backward) counterparts by the assembler.

JUMP_IF_TRUE

JUMP_IF_FALSE

Conditional jumps which do not impact the stack. Replaced by the sequence COPY 1, TO_BOOL, POP_JUMP_IF_TRUE/FALSE.

LOAD_CLOSURE (*i*)

Pushes a reference to the cell contained in slot i of the "fast locals" storage.

Note that LOAD_CLOSURE is replaced with LOAD_FAST in the assembler.

Alterado na versão 3.13: This opcode is now a pseudo-instruction.

32.10.5 Opcode collections

These collections are provided for automatic introspection of bytecode instructions:

Alterado na versão 3.12: The collections now contain pseudo instructions and instrumented instructions as well. These are opcodes with values `>= MIN_PSEUDO_OPCODE` and `>= MIN_INSTRUMENTED_OPCODE`.

`dis.`**`opname`**

Sequence of operation names, indexable using the bytecode.

`dis.`**`opmap`**

Dictionary mapping operation names to bytecodes.

`dis.`**`cmp_op`**

Sequence of all compare operation names.

`dis.`**`hasarg`**

Sequence of bytecodes that use their argument.

Adicionado na versão 3.12.

`dis.`**`hasconst`**

Sequence of bytecodes that access a constant.

`dis.`**`hasfree`**

Sequence of bytecodes that access a *free (closure) variable*. 'free' in this context refers to names in the current scope that are referenced by inner scopes or names in outer scopes that are referenced from this scope. It does *not* include references to global or builtin scopes.

`dis.`**`hasname`**

Sequence of bytecodes that access an attribute by name.

`dis.`**`hasjump`**

Sequence of bytecodes that have a jump target. All jumps are relative.

Adicionado na versão 3.13.

`dis.`**`haslocal`**

Sequence of bytecodes that access a local variable.

`dis.`**`hascompare`**

Sequence of bytecodes of Boolean operations.

`dis.`**`hasexc`**

Sequence of bytecodes that set an exception handler.

Adicionado na versão 3.12.

`dis.`**`hasjrel`**

Sequence of bytecodes that have a relative jump target.

Descontinuado desde a versão 3.13: All jumps are now relative. Use *`hasjump`*.

`dis.`**`hasjabs`**

Sequence of bytecodes that have an absolute jump target.

Descontinuado desde a versão 3.13: All jumps are now relative. This list is empty.

32.11 `pickletools` — Ferramentas para desenvolvedores pickle

Código-fonte: Lib/pickletools.py

Este módulo contém várias constantes relacionadas aos detalhes íntimos do módulo *pickle*, alguns comentários extensos sobre a implementação e algumas funções úteis para analisar dados em conserva. O conteúdo deste módulo é útil para desenvolvedores do núcleo Python que estão trabalhando no *pickle*; usuários comuns do módulo *pickle* provavelmente não acharão o módulo *pickletools* relevante.

32.11.1 Uso na linha de comando

Adicionado na versão 3.2.

Quando chamado a partir da linha de comando, `python -m pickletools` irá desmontar o conteúdo de um ou mais arquivos pickle. Note que se você quiser ver o objeto Python armazenado serializado em pickle ao invés dos detalhes do formato pickle, você pode usar `-m pickle`. No entanto, quando o arquivo serializado em pickle que você deseja examinar vem de uma fonte não confiável, `-m pickletools` é uma opção mais segura porque não executa bytecode pickle.

Por exemplo, com uma tupla `(1, 2)` tratada com pickling no arquivo `x.pickle`:

```
$ python -m pickle x.pickle
(1, 2)

$ python -m pickletools x.pickle
    0: \x80 PROTO      3
    2: K    BININT1    1
    4: K    BININT1    2
    6: \x86 TUPLE2
    7: q    BINPUT     0
    9: .    STOP
highest protocol among opcodes = 2
```

Opções de linha de comando

-a, --annotate

Anota cada linha com uma descrição curta do código de operação.

-o, --output=<file>

Nome de um arquivo no qual a saída deve ser escrita.

-l, --indentlevel=<num>

O número de espaços em branco para indentar um novo nível MARK.

-m, --memo

Quando vários objetos são desmontados, preserva memo entre as desmontagens.

-p, --preamble=<preamble>

Quando mais de um arquivo serializado com pickle for especificado, exibe o preâmbulo fornecido antes de cada desmontagem.

pickle_file

Um arquivo serializado com pickle para ler, ou - para indicar leitura da entrada padrão.

32.11.2 Interface programática

`pickletools.`**dis**(*pickle*, *out=None*, *memo=None*, *indentlevel=4*, *annotate=0*)

Produz uma desmontagem simbólica do pickle para o objeto arquivo ou similar *out*, tendo como padrão `sys.stdout`. *pickle* pode ser uma string ou um objeto arquivo ou similar. *memo* pode ser um dicionário Python que será usado como memo do pickle; ele pode ser usado para realizar desmontagens em várias serializações com pickle criadas pelo mesmo pickler. Níveis sucessivos, indicados por códigos de operação `MARK` no fluxo, são indentados por espaços *indentlevel*. Se um valor diferente de zero for fornecido para *annotate*, cada código de operação na saída será anotado com uma breve descrição. O valor de *annotate* é usado como uma dica para a coluna onde a anotação deve começar.

Alterado na versão 3.2: Adicionado o parâmetro *annotate*.

pickletools.**genops**(*pickle*)

Fornece um *iterador* sobre todos os códigos de operação em uma serialização com pickle, retornando uma sequência de triplos `(opcode, arg, pos)`. *opcode* é uma instância de uma classe `OpcodeInfo`; *arg* é o valor decodificado, como um objeto Python, do argumento do opcode; *pos* é a posição em que este código de operação está localizado. *pickle* pode ser uma string ou um objeto arquivo ou similar.

pickletools.**optimize**(*picklestring*)

Retorna uma nova string pickle equivalente após eliminar códigos de operação `PUT` não utilizados. O pickle otimizado é mais curto, leva menos tempo de transmissão, requer menos espaço de armazenamento e efetua unpickling com mais eficiência.

CAPÍTULO 33

Serviços Específicos do MS Windows

Este capítulo descreve módulos que estão disponíveis somente na plataformas MS Windows.

33.1 `msvcrt` — Rotinas úteis do runtime MS VC++

Essas funções fornecem acesso a alguns recursos úteis em plataformas Windows. Alguns módulos de nível superior usam essas funções para construir as implementações de seus serviços no Windows. Por exemplo, o módulo *getpass* usa isso na implementação da função *getpass()*.

Mais documentação sobre essas funções pode ser encontrada na documentação da API da plataforma.

O módulo implementa as variantes normal e ampla de caracteres da API de E/S do console. A API normal lida apenas com caracteres ASCII e é de uso limitado para aplicativos internacionalizados. A API ampla de caracteres deve ser usada sempre que possível.

Alterado na versão 3.3: As operações neste módulo agora levantam *OSError* onde *IOError* foi levantado.

33.1.1 Operações com arquivos

`msvcrt.`**`locking`**(*fd*, *mode*, *nbytes*)

Trava parte de um arquivo com base no descritor de arquivo *fd* do runtime do C. Levanta *OSError* em caso de falha. A região travada do arquivo se estende da posição atual do arquivo por *nbytes* bytes e pode continuar além do final do arquivo. *mode* deve ser uma das constantes `LK_*` listadas abaixo. Várias regiões em um arquivo podem ser travadas ao mesmo tempo, mas não podem se sobrepor. Regiões adjacentes não são mescladas; elas devem ser destravadas individualmente.

Levanta um *evento de auditoria* `msvcrt.locking` com os argumentos `fd`, `mode`, `nbytes`.

`msvcrt.`**`LK_LOCK`**

`msvcrt.`**`LK_RLCK`**

Trava os bytes especificados. Se os bytes não puderem ser travados, o programa tenta novamente imediatamente após 1 segundo. Se, após 10 tentativas, os bytes não puderem ser travados, *OSError* é levantada.

`msvcrt.`**`LK_NBLCK`**

`msvcrt.`**`LK_NBRLCK`**

Trava os bytes especificados. Se os bytes não puderem ser travados, uma exceção *OSError* é levantada.

msvcrt.**LK_UNLCK**

Destrava os bytes especificados, que devem ter sido travados anteriormente.

msvcrt.**setmode**(*fd*, *flags*)

Defina o modo de conversão de final de linha para o descritor de arquivo *fd*. Para configurá-lo no modo de texto, *flags* deve ser `os.O_TEXT`; para binário, deve ser `os.O_BINARY`.

msvcrt.**open_osfhandle**(*handle*, *flags*)

Crie um descritor de arquivo de tempo de execução do C a partir do identificador de arquivo *handle*. O parâmetro *flags* deve ser um OU bit a bit de `os.O_APPEND`, `os.O_RDONLY`, `os.O_TEXT` e `os.O_NOINHERIT`. O descritor de arquivo retornado pode ser usado como parâmetro para `os.fdopen()` para criar um objeto de arquivo.

O descritor de arquivo é herdável por padrão. Passe o sinalizador `os.O_NOINHERIT` para torná-lo não herdável.

Levanta um *evento de auditoria* `msvcrt.open_osfhandle` com os argumentos `handle`, `flags`.

msvcrt.**get_osfhandle**(*fd*)

Retorna o identificador de arquivo para o descritor de arquivo *fd*. Levanta `OSError` se *fd* não for reconhecido.

Levanta um *evento de auditoria* `msvcrt.get_osfhandle` com o argumento `fd`.

33.1.2 E/S de console

msvcrt.**kbhit**()

Returns a nonzero value if a keypress is waiting to be read. Otherwise, return 0.

msvcrt.**getch**()

Read a keypress and return the resulting character as a byte string. Nothing is echoed to the console. This call will block if a keypress is not already available, but will not wait for `Enter` to be pressed. If the pressed key was a special function key, this will return `'\000'` or `'\xe0'`; the next call will return the keycode. The `Control-C` keypress cannot be read with this function.

msvcrt.**getwch**()

Variante com caractere largo de `getch()`, retornando um valor Unicode.

msvcrt.**getche**()

Similar to `getch()`, but the keypress will be echoed if it represents a printable character.

msvcrt.**getwche**()

Variante com caractere largo de `getche()`, retornando um valor Unicode.

msvcrt.**putch**(*char*)

Imprime a string de bytes *char* no console sem armazenar em buffer.

msvcrt.**putwch**(*unicode_char*)

Variante com caractere largo de `putch()`, retornando um valor Unicode.

msvcrt.**ungetch**(*char*)

Faz com que a string de bytes *char* seja "empurrada" para o buffer do console; será o próximo caractere lido por `getch()` ou `getche()`.

msvcrt.**ungetwch**(*unicode_char*)

Variante com caractere largo de `ungetch()`, retornando um valor Unicode.

33.1.3 Outras funções

msvcrt.**heapmin**()

Força o heap `malloc()` a ser limpado e retorna os blocos não utilizados ao sistema operacional. Em caso de falha, isso levanta `OSError`.

msvcrt.**set_error_mode**(*mode*)

Changes the location where the C runtime writes an error message for an error that might end the program. *mode* must be one of the `OUT_*` constants listed below or *REPORT_ERRMODE*. Returns the old setting or -1 if an error occurs. Only available in debug build of Python.

msvcrt.**OUT_TO_DEFAULT**

Error sink is determined by the app's type. Only available in debug build of Python.

msvcrt.**OUT_TO_STDERR**

Error sink is a standard error. Only available in debug build of Python.

msvcrt.**OUT_TO_MSGBOX**

Error sink is a message box. Only available in debug build of Python.

msvcrt.**REPORT_ERRMODE**

Report the current error mode value. Only available in debug build of Python.

msvcrt.**CrtSetReportMode**(*type*, *mode*)

Specifies the destination or destinations for a specific report type generated by `_CrtDbgReport()` in the MS VC++ runtime. *type* must be one of the `CRT_*` constants listed below. *mode* must be one of the `CRTDBG_*` constants listed below. Only available in debug build of Python.

msvcrt.**CrtSetReportFile**(*type*, *file*)

After you use *CrtSetReportMode()* to specify *CRTDBG_MODE_FILE*, you can specify the file handle to receive the message text. *type* must be one of the `CRT_*` constants listed below. *file* should be the file handle your want specified. Only available in debug build of Python.

msvcrt.**CRT_WARN**

Warnings, messages, and information that doesn't need immediate attention.

msvcrt.**CRT_ERROR**

Errors, unrecoverable problems, and issues that require immediate attention.

msvcrt.**CRT_ASSERT**

Assertion failures.

msvcrt.**CRTDBG_MODE_DEBUG**

Writes the message to the debugger's output window.

msvcrt.**CRTDBG_MODE_FILE**

Writes the message to a user-supplied file handle. *CrtSetReportFile()* should be called to define the specific file or stream to use as the destination.

msvcrt.**CRTDBG_MODE_WNDW**

Creates a message box to display the message along with the `Abort`, `Retry`, and `Ignore` buttons.

msvcrt.**CRTDBG_REPORT_MODE**

Returns current *mode* for the specified *type*.

msvcrt.**CRT_ASSEMBLY_VERSION**

A versão do CRT Assembly, do arquivo de cabeçalho `crtassem.h`.

msvcrt.**VC_ASSEMBLY_PUBLICKEYTOKEN**

O token da chave pública do VC Assembly, do arquivo de cabeçalho `crtassem.h`.

msvcrt.**LIBRARIES_ASSEMBLY_NAME_PREFIX**

O prefixo do nome de Libraries Assembly, do arquivo de cabeçalho `crtassem.h`.

33.2 winreg — Acesso aos registros do Windows

Essas funções expõem a API de registro do Windows ao Python. Em vez de usar um inteiro como identificador de registro, um *objeto identificador* é usado para garantir que os identificadores sejam fechados corretamente, mesmo que o programador não os feche explicitamente.

Alterado na versão 3.3: Várias funções neste módulo costumavam levantar um `WindowsError`, que agora é um apelido de `OSError`.

33.2.1 Funções

Este módulo oferece as seguintes funções:

winreg.**CloseKey**(*hkey*)

Fecha uma chave de registro aberta anteriormente. O argumento *hkey* especifica uma chave aberta anteriormente.

> **Nota**
>
> Se *hkey* não for fechado usando este método (ou via `hkey.Close()`), ele será fechado quando o objeto *hkey* for destruído pelo Python.

winreg.**ConnectRegistry**(*computer_name*, *key*)

Estabelece uma conexão com um identificador de registro predefinido em outro computador e retorna um *objeto identificador*.

computer_name é o nome do computador remoto, no formato `r"\\computername"`. Se `None`, o computador local será usado.

key é o identificador predefinido para conexão.

O valor de retorno é o identificador da chave aberta. Se a função falhar, uma exceção `OSError` será levantada.

Levanta um *evento de auditoria* `winreg.ConnectRegistry` com os argumentos `computer_name`, `key`.

Alterado na versão 3.3: Veja *acima*.

winreg.**CreateKey**(*key*, *sub_key*)

Cria ou abre a chave especificada, retornando um *objeto identificador*.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

sub_key é uma string que nomeia a chave que este método abre ou cria.

Se *key* for uma das chaves predefinidas, *sub_key* poderá ser `None`. Nesse caso, o identificador retornado será o mesmo identificador de chave passado para a função.

Se a chave já existir, esta função abre a chave existente.

O valor de retorno é o identificador da chave aberta. Se a função falhar, uma exceção `OSError` será levantada.

Levanta um *evento de auditoria* `winreg.CreateKey` com os argumentos `key`, `sub_key`, `access`.

Levanta um *evento de auditoria* `winreg.OpenKey/result` com o argumento `key`.

Alterado na versão 3.3: Veja *acima*.

winreg.**CreateKeyEx**(*key*, *sub_key*, *reserved=0*, *access=KEY_WRITE*)

Cria ou abre a chave especificada, retornando um *objeto identificador*.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

sub_key é uma string que nomeia a chave que este método abre ou cria.

reserved é um inteiro reservado e deve ser zero. O padrão é zero.

access é um inteiro que especifica uma máscara de acesso que descreve o acesso de segurança desejado para a chave. O padrão é *KEY_WRITE*. Consulte *Direitos de acesso* para outros valores permitidos.

Se *key* for uma das chaves predefinidas, *sub_key* poderá ser `None`. Nesse caso, o identificador retornado será o mesmo identificador de chave passado para a função.

Se a chave já existir, esta função abre a chave existente.

O valor de retorno é o identificador da chave aberta. Se a função falhar, uma exceção *OSError* será levantada.

Levanta um *evento de auditoria* `winreg.CreateKey` com os argumentos `key`, `sub_key`, `access`.

Levanta um *evento de auditoria* `winreg.OpenKey/result` com o argumento `key`.

Adicionado na versão 3.2.

Alterado na versão 3.3: Veja *acima*.

`winreg.`**`DeleteKey`**(*key*, *sub_key*)

Exclui a chave especificada.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

sub_key é uma string que deve ser uma subchave da chave identificada pelo parâmetro *key*. Este valor não pode ser `None`, e a chave não pode ter subchaves.

Este método não pode excluir chaves com subchaves.

Se o método for bem-sucedido, a chave inteira, incluindo todos os seus valores, será removida. Se o método falhar, uma exceção *OSError* será levantada.

Levanta um *evento de auditoria* `winreg.DeleteKey` com os argumentos `key`, `sub_key`, `access`.

Alterado na versão 3.3: Veja *acima*.

`winreg.`**`DeleteKeyEx`**(*key*, *sub_key*, *access=KEY_WOW64_64KEY*, *reserved=0*)

Exclui a chave especificada.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

sub_key é uma string que deve ser uma subchave da chave identificada pelo parâmetro *key*. Este valor não pode ser `None`, e a chave não pode ter subchaves.

reserved é um inteiro reservado e deve ser zero. O padrão é zero.

access é um inteiro que especifica uma máscara de acesso que descreve o acesso de segurança desejado para a chave. O padrão é *KEY_WOW64_64KEY*. No Windows de 32 bits, as constantes WOW64 são ignoradas. Consulte *Direitos de acesso* para outros valores permitidos.

Este método não pode excluir chaves com subchaves.

Se o método for bem-sucedido, a chave inteira, incluindo todos os seus valores, será removida. Se o método falhar, uma exceção *OSError* será levantada.

Em versões do Windows sem suporte, *NotImplementedError* é levantada.

Levanta um *evento de auditoria* `winreg.DeleteKey` com os argumentos `key`, `sub_key`, `access`.

Adicionado na versão 3.2.

Alterado na versão 3.3: Veja *acima*.

`winreg.`**`DeleteValue`**(*key*, *value*)

Remove um valor nomeado de uma chave de registro.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

value é uma string que identifica o valor a ser removido.

Levanta um *evento de auditoria* `winreg.DeleteValue` com os argumentos `key`, `value`.

winreg.**EnumKey**(*key*, *index*)

Enumera subchaves de uma chave de registro aberta, retornando uma string.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

index é um inteiro que identifica o índice da chave a ser recuperada.

A função recupera o nome de uma subchave cada vez que é chamada. Normalmente, ela é chamada repetidamente até que uma exceção *OSError* seja levantada, indicando que não há mais valores disponíveis.

Levanta um *evento de auditoria* `winreg.EnumKey` com os argumentos `key`, `index`.

Alterado na versão 3.3: Veja *acima*.

winreg.**EnumValue**(*key*, *index*)

Enumera valores de uma chave de registro aberta, retornando uma tupla.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

index é um inteiro que identifica o índice do valor a ser recuperado.

A função recupera o nome de uma subchave cada vez que é chamada. Normalmente, ela é chamada repetidamente, até que uma exceção *OSError* seja levantada, indicando que não há mais valores.

O resultado é uma tupla de 3 itens:

Índice	Significado
0	Uma string que identifica o nome do valor
1	Um objeto que contém os dados de valor e cujo tipo depende do tipo de registro subjacente
2	Um inteiro que identifica o tipo dos dados do valor (veja a tabela na documentação para *SetValueEx()*)

Levanta um *evento de auditoria* `winreg.EnumValue` com os argumentos `key`, `index`.

Alterado na versão 3.3: Veja *acima*.

winreg.**ExpandEnvironmentStrings**(*str*)

Expande os espaços reservados de posição de variáveis de ambiente `%NAME%` em strings como *REG_EXPAND_SZ*:

```
>>> ExpandEnvironmentStrings('%windir%')
'C:\\Windows'
```

Levanta um *evento de auditoria* `winreg.ExpandEnvironmentStrings` com o argumento `str`.

winreg.**FlushKey**(*key*)

Grava todos os atributos de uma chave no registro.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

Não é necessário chamar *FlushKey()* para alterar uma chave. As alterações do registro são descarregadas no disco pelo registro usando seu limpador lento. As alterações do registro também são descarregadas no disco no desligamento do sistema. Ao contrário de *CloseKey()*, o método *FlushKey()* retorna somente quando todos os dados foram gravados no registro. Uma aplicação só deve chamar *FlushKey()* se precisar de certeza absoluta de que as alterações do registro estão no disco.

Nota

Se você não sabe se uma chamada *FlushKey()* é necessária, provavelmente não é.

winreg.**LoadKey**(*key*, *sub_key*, *file_name*)

Cria uma subchave sob a chave especificada e armazena informações de registro de um arquivo especificado nessa subchave.

key é um identificador retornado por *ConnectRegistry()* ou uma das constantes *HKEY_USERS* ou *HKEY_LOCAL_MACHINE*.

sub_key é uma string que identifica a subchave a ser carregada.

file_name é o nome do arquivo do qual serão carregados os dados do registro. Este arquivo deve ter sido criado com a função *SaveKey()*. No sistema de arquivos da tabela de alocação de arquivos (FAT), o nome do arquivo não pode ter extensão.

Uma chamada para *LoadKey()* falhará se o processo de chamada não tiver o privilégio SE_RESTORE_PRIVILEGE. Observe que privilégios são diferentes de permissões — consulte a documentação do RegLoadKey para mais detalhes.

Se *key* for um identificador retornado por *ConnectRegistry()*, então o caminho especificado em *file_name* será relativo ao computador remoto.

Levanta um *evento de auditoria* winreg.LoadKey com os argumentos key, sub_key, file_name.

winreg.**OpenKey**(*key*, *sub_key*, *reserved=0*, *access=KEY_READ*)

winreg.**OpenKeyEx**(*key*, *sub_key*, *reserved=0*, *access=KEY_READ*)

Abre a chave especificada, retornando um *objeto identificador*.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

sub_key é uma string que identifica a sub_key a ser aberta.

reserved é um inteiro reservado e deve ser zero. O padrão é zero.

access é um inteiro que especifica uma máscara de acesso que descreve o acesso de segurança desejado para a chave. O padrão é *KEY_READ*. Consulte *Direitos de acesso* para outros valores permitidos.

O resultado é um novo identificador para a chave especificada.

Se a função falhar, *OSError* será levantada.

Levanta um *evento de auditoria* winreg.OpenKey com os argumentos key, sub_key, access.

Levanta um *evento de auditoria* winreg.OpenKey/result com o argumento key.

Alterado na versão 3.2: Permite o uso de argumentos nomeados.

Alterado na versão 3.3: Veja *acima*.

winreg.**QueryInfoKey**(*key*)

Retorna informações sobre uma chave, como uma tupla.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

O resultado é uma tupla de 3 itens:

Índice	Significado
0	Um inteiro que fornece o número de subchaves que esta chave possui.
1	Um inteiro que fornece o número de valores que esta chave possui.
2	Um número inteiro que indica quando a chave foi modificada pela última vez (se disponível) em centenas de nanossegundos desde 1º de janeiro de 1601.

Levanta um *evento de auditoria* winreg.QueryInfoKey com o argumento key.

winreg.**QueryValue**(*key*, *sub_key*)

Recupera o valor sem nome de uma chave, como uma string.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

sub_key é uma string que contém o nome da subchave à qual o valor está associado. Se este parâmetro for `None` ou estiver vazio, a função recupera o valor definido pelo método *`SetValue()`* para a chave identificada por *key*.

Os valores no registro têm componentes de nome, tipo e dados. Este método recupera os dados do primeiro valor de uma chave que possui um nome `NULL`. Mas a chamada de API subjacente não retorna o tipo, portanto, sempre use *`QueryValueEx()`* se possível.

Levanta um *evento de auditoria* `winreg.QueryValue` com os argumentos `key`, `sub_key`, `value_name`.

winreg.**QueryValueEx**(*key*, *value_name*)

Recupera o tipo e os dados de um nome de valor especificado associado a uma chave de registro aberta.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

value_name é uma string que indica o valor a ser consultado.

O resultado é uma tupla de 2 itens:

Índice	Significado
`0`	O valor do item do registro.
`1`	Um inteiro que fornece o tipo de registro para este valor (veja a tabela na documentação para *`SetValueEx()`*)

Levanta um *evento de auditoria* `winreg.QueryValue` com os argumentos `key`, `sub_key`, `value_name`.

winreg.**SaveKey**(*key*, *file_name*)

Salva a chave especificada e todas as suas subchaves no arquivo especificado.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

file_name é o nome do arquivo para salvar os dados do registro. Este arquivo não pode existir. Se este nome de arquivo incluir uma extensão, ele não poderá ser usado em sistemas de arquivos FAT (tabela de alocação de arquivos) pelo método *`LoadKey()`*.

Se *key* representa uma chave em um computador remoto, o caminho descrito por *file_name* é relativo ao computador remoto. O chamador deste método deve possuir o privilégio de segurança **SeBackupPrivilege**. Observe que privilégios são diferentes de permissões — consulte a documentação Conflitos entre Direitos e Permissões do Usuário para obter mais detalhes.

Esta função passa `NULL` para *security_attributes* para a API.

Levanta um *evento de auditoria* `winreg.SaveKey` com os argumentos `key`, `file_name`.

winreg.**SetValue**(*key*, *sub_key*, *type*, *value*)

Associa um valor a uma chave especificada.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

sub_key é uma string que nomeia a subchave à qual o valor está associado.

type é um inteiro que especifica o tipo dos dados. Atualmente, deve ser *`REG_SZ`*, o que significa que apenas strings são suportadas. Use a função *`SetValueEx()`* para suporte a outros tipos de dados.

valor é uma string que especifica o novo valor.

Se a chave especificada pelo parâmetro *sub_key* não existir, a função SetValue a criará.

O comprimento dos valores é limitado pela memória disponível. Valores longos (mais de 2048 bytes) devem ser armazenados como arquivos com os nomes dos arquivos armazenados no registro de configuração. Isso ajuda o registro a funcionar com eficiência.

A chave identificada pelo parâmetro *key* deve ter sido aberta com acesso *KEY_SET_VALUE*.

Levanta um *evento de auditoria* `winreg.SetValue` com os argumentos `key`, `sub_key`, `type`, `value`.

`winreg.`**`SetValueEx`**(*key*, *value_name*, *reserved*, *type*, *value*)

Armazena dados no campo de valor de uma chave de registro aberta.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

value_name é uma string que nomeia a subchave à qual o valor está associado.

reserved pode ser qualquer coisa – zero é sempre passado para a API.

type é um inteiro que especifica o tipo dos dados. Consulte *Tipos de valor* para os tipos disponíveis.

valor é uma string que especifica o novo valor.

Este método também pode definir informações adicionais de valor e tipo para a chave especificada. A chave identificada pelo parâmetro key deve ter sido aberta com o acesso *KEY_SET_VALUE*.

Para abrir a chave, use os métodos *CreateKey()* ou *OpenKey()*.

O comprimento dos valores é limitado pela memória disponível. Valores longos (mais de 2048 bytes) devem ser armazenados como arquivos com os nomes dos arquivos armazenados no registro de configuração. Isso ajuda o registro a funcionar com eficiência.

Levanta um *evento de auditoria* `winreg.SetValue` com os argumentos `key`, `sub_key`, `type`, `value`.

`winreg.`**`DisableReflectionKey`**(*key*)

Desabilita a reflexão do registro para processos de 32 bits em execução em um sistema operacional de 64 bits.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

Geralmente levantará *NotImplementedError* se executado em um sistema operacional de 32 bits.

Se a chave não estiver na lista de reflexão, a função será bem-sucedida, mas não terá efeito. Desabilitar a reflexão para uma chave não afeta a reflexão de nenhuma subchave.

Levanta um *evento de auditoria* `winreg.DisableReflectionKey` com o argumento `key`.

`winreg.`**`EnableReflectionKey`**(*key*)

Restaura a reflexão do registro para a chave desabilitada especificada.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

Geralmente levantará *NotImplementedError* se executado em um sistema operacional de 32 bits.

Restaura a reflexão de uma chave não afeta a reflexão de nenhuma subchave.

Levanta um *evento de auditoria* `winreg.EnableReflectionKey` com o argumento `key`.

`winreg.`**`QueryReflectionKey`**(*key*)

Determina o estado de reflexão para a chave especificada.

key é uma chave já aberta, ou uma das constantes predefinidas *HKEY_**.

Retorna `True` se a reflexão estiver desabilitada.

Geralmente levantará *NotImplementedError* se executado em um sistema operacional de 32 bits.

Levanta um *evento de auditoria* `winreg.QueryReflectionKey` com o argumento `key`.

33.2.2 Constantes

As seguintes constantes são definidas para uso em muitas funções *winreg*.

Constantes HKEY_*

winreg.**HKEY_CLASSES_ROOT**

Entradas de registro subordinadas a esta chave definem tipos (ou classes) de documentos e as propriedades associadas a esses tipos. Aplicações Shell e COM utilizam as informações armazenadas sob esta chave.

winreg.**HKEY_CURRENT_USER**

As entradas de registro subordinadas a esta chave definem as preferências do usuário atual. Essas preferências incluem as configurações de variáveis de ambiente, dados sobre grupos de programas, cores, impressoras, conexões de rede e preferências de aplicações.

winreg.**HKEY_LOCAL_MACHINE**

As entradas de registro subordinadas a esta chave definem o estado físico do computador, incluindo dados sobre o tipo de barramento, memória do sistema e hardware e software instalados.

winreg.**HKEY_USERS**

As entradas de registro subordinadas a esta chave definem a configuração de usuário padrão para novos usuários no computador local e a configuração de usuário para o usuário atual.

winreg.**HKEY_PERFORMANCE_DATA**

Entradas de registro subordinadas a esta chave permitem que você acesse dados de desempenho. Os dados não são armazenados no registro; as funções de registro fazem com que o sistema colete os dados de sua fonte.

winreg.**HKEY_CURRENT_CONFIG**

Contém informações sobre o perfil de hardware atual do sistema de computador local.

winreg.**HKEY_DYN_DATA**

Esta chave não é usada em versões do Windows posteriores à 98.

Direitos de acesso

Para obter mais informações, consulte Segurança e acesso a chaves do registro.

winreg.**KEY_ALL_ACCESS**

Combina os direitos de acesso STANDARD_RIGHTS_REQUIRED, *KEY_QUERY_VALUE*, *KEY_SET_VALUE*, *KEY_CREATE_SUB_KEY*, *KEY_ENUMERATE_SUB_KEYS*, *KEY_NOTIFY* e *KEY_CREATE_LINK*.

winreg.**KEY_WRITE**

Combina os direitos de acesso STANDARD_RIGHTS_WRITE, *KEY_SET_VALUE* e *KEY_CREATE_SUB_KEY*.

winreg.**KEY_READ**

Combina os valores STANDARD_RIGHTS_READ, *KEY_QUERY_VALUE*, *KEY_ENUMERATE_SUB_KEYS* e *KEY_NOTIFY*.

winreg.**KEY_EXECUTE**

Equivalente a *KEY_READ*.

winreg.**KEY_QUERY_VALUE**

Necessário para consultar os valores de uma chave de registro.

winreg.**KEY_SET_VALUE**

Necessário para criar, excluir ou definir um valor de registro.

winreg.**KEY_CREATE_SUB_KEY**

Necessário para criar uma subchave de uma chave de registro.

winreg.**KEY_ENUMERATE_SUB_KEYS**

Necessário para enumerar as subchaves de uma chave de registro.

winreg.**KEY_NOTIFY**

Necessário para solicitar notificações de alteração para uma chave de registro ou para subchaves de uma chave de registro.

winreg.**KEY_CREATE_LINK**

Reservado para uso pelo sistema.

Específico de 64 bits

Para obter mais informações, consulte Acessando uma exibição alternativa do Registro.

winreg.**KEY_WOW64_64KEY**

Indica que uma aplicação no Windows 64 bits deve operar na visualização de registro de 64 bits. No Windows de 32 bits, esta constante é ignorada.

winreg.**KEY_WOW64_32KEY**

Indica que uma aplicação no Windows 64 bits deve operar na visualização de registro de 32 bits. No Windows de 32 bits, esta constante é ignorada.

Tipos de valores

Para obter mais informações, consulte Tipos de valor do Registro.

winreg.**REG_BINARY**

Dados binários em qualquer formato.

winreg.**REG_DWORD**

Número de 32 bits.

winreg.**REG_DWORD_LITTLE_ENDIAN**

Um número de 32 bits no formato little-endian. Equivalente a *REG_DWORD*.

winreg.**REG_DWORD_BIG_ENDIAN**

Um número de 32 bits no formato big-endian.

winreg.**REG_EXPAND_SZ**

String terminada em nulo contendo referências a variáveis de ambiente (`%PATH%`).

winreg.**REG_LINK**

Um link simbólico Unicode.

winreg.**REG_MULTI_SZ**

Uma sequência de strings terminadas em nulo, terminadas por dois caracteres nulos. (O Python lida com essa terminação automaticamente.)

winreg.**REG_NONE**

Nenhum tipo de valor definido.

winreg.**REG_QWORD**

Um número de 64 bits.

Adicionado na versão 3.6.

winreg.**REG_QWORD_LITTLE_ENDIAN**

Um número de 64 bits no formato little-endian. Equivalente a *REG_QWORD*.

Adicionado na versão 3.6.

winreg.**REG_RESOURCE_LIST**

Uma lista de recursos de driver de dispositivo.

winreg.**REG_FULL_RESOURCE_DESCRIPTOR**

Uma configuração de hardware.

winreg.**REG_RESOURCE_REQUIREMENTS_LIST**

Uma lista de recursos de hardware.

winreg.**REG_SZ**

Uma string terminada em nulo.

33.2.3 Objetos identificador de registro

Este objeto encapsula um objeto HKEY do Windows, fechando-o automaticamente quando o objeto é destruído. Para garantir a limpeza, você pode chamar o método *Close()* no objeto ou a função *CloseKey()*.

Todas as funções de registro neste módulo retornam um destes objetos.

Todas as funções de registro neste módulo que aceitam um objeto identificador também aceitam um inteiro; no entanto, o uso do objeto identificador é incentivado.

Objetos identificador fornecem semântica para __bool__() – portanto

```
if handle:
    print("Sim")
```

vai exibir Sim se o identificador for válido no momento (não foi fechado ou desanexado).

O objeto também oferece suporte à semântica de comparação, portanto, os objetos identificador serão comparados como verdadeiros se ambos fizerem referência ao mesmo valor de identificador subjacente do Windows.

Objetos identificador podem ser convertidos em um inteiro (por exemplo, usando a função embutida *int()*), caso em que o valor do identificador do Windows subjacente é retornado. Você também pode usar o método *Detach()* para retornar o identificador inteiro e também desconectar o identificador do Windows do objeto identificador.

PyHKEY.**Close**()

Fecha o identificador subjacente do Windows.

Se o identificador já estiver fechado, nenhum erro será levantado.

PyHKEY.**Detach**()

Separa o identificador do Windows do objeto identificador.

O resultado é um inteiro que contém o valor do identificador antes de ser destacado. Se o identificador já estiver destacado ou fechado, isso retornará zero.

Após chamar esta função, o identificador é efetivamente invalidado, mas não é fechado. Você chamaria esta função quando precisasse que o identificador subjacente de Win32 existisse além do tempo de vida do objeto identificador.

Levanta um *evento de auditoria* winreg.PyHKEY.Detach com o argumento key.

PyHKEY.**__enter__**()

PyHKEY.**__exit__**(**exc_info*)

O objeto HKEY implementa __enter__() e __exit__() e, portanto, oferece suporte ao protocolo de contexto para a instrução with:

```
with OpenKey(HKEY_LOCAL_MACHINE, "foo") as key:
    ...  # trabalha com a chave "key"
```

vai fechar automaticamente *key* quando o controle sair do bloco with.

33.3 `winsound` — Interface de reprodução de som para o Windows

O módulo *winsound* fornece acesso ao mecanismo básico de reprodução de som fornecido pelas plataformas Windows. Inclui funções e várias constantes.

`winsound.`**`Beep`**(*frequency*, *duration*)

Emite um bipe no alto-falante do PC. O parâmetro *frequency* especifica a frequência, em hertz, do som e deve estar no intervalo de 37 a 32.767. O parâmetro *duration* especifica o número de milissegundos que o som deve durar. Se o sistema não conseguir emitir um bipe no alto-falante, *RuntimeError* é levantado.

`winsound.`**`PlaySound`**(*sound*, *flags*)

Chama a função subjacente `PlaySound()` da API da plataforma. O parâmetro *sound* pode ser um nome de arquivo, um apelido de som do sistema, dados de áudio como um *objeto bytes ou similar* ou `None`. Sua interpretação depende do valor de *flags*, que pode ser uma combinação OU bit a bit das constantes descritas abaixo. Se o parâmetro *sound* for `None`, qualquer som de forma de onda em execução no momento será interrompido. Se o sistema indicar um erro, *RuntimeError* será levantada.

`winsound.`**`MessageBeep`**(*type=MB_OK*)

Chama a função subjacente `MessageBeep()` da API da plataforma. Isso reproduz um som conforme especificado no registro. O argumento *type* especifica qual som reproduzir; os valores possíveis são `-1`, `MB_ICONASTERISK`, `MB_ICONEXCLAMATION`, `MB_ICONHAND`, `MB_ICONQUESTION` e `MB_OK`, todos descritos abaixo. O valor `-1` produz um "bipe simples"; este é o fallback final se um som não puder ser reproduzido de outra forma. Se o sistema indicar um erro, *RuntimeError* será levantada.

`winsound.`**`SND_FILENAME`**

O parâmetro *sound* é o nome de um arquivo WAV. Não use com *SND_ALIAS*.

`winsound.`**`SND_ALIAS`**

O parâmetro *sound* é um nome de associação de som do registro. Se o registro não contiver tal nome, reproduza o som padrão do sistema, a menos que *SND_NODEFAULT* também seja especificado. Se nenhum som padrão for registrado, levanta *RuntimeError*. Não use com *SND_FILENAME*.

Todos os sistemas Win32 oferecem suporte a pelo menos o seguinte; a maioria dos sistemas oferecem suporte a muito mais:

name *PlaySound()*	Nome do som correspondente no Painel de Controle
`'SystemAsterisk'`	Asterisco
`'SystemExclamation'`	Exclamação
`'SystemExit'`	Logoff do Windows
`'SystemHand'`	Parada crítica
`'SystemQuestion'`	Pergunta

Por exemplo:

```
import winsound
# Reproduz o som de logoff do Windows.
winsound.PlaySound("SystemExit", winsound.SND_ALIAS)

# Provavelmente reproduz o som padrão do Windows, se algum
# estiver registrado (porque "*" provavelmente não é o nome registrado
# de algum som).
winsound.PlaySound("*", winsound.SND_ALIAS)
```

`winsound.`**`SND_LOOP`**

Reproduz o som repetidamente. O sinalizador *SND_ASYNC* também deve ser usado para evitar bloqueio. Não pode ser usado com *SND_MEMORY*.

winsound.**SND_MEMORY**

O parâmetro *sound* para `PlaySound()` é uma imagem de memória de um arquivo WAV, como um *objeto bytes ou similar*.

> **Nota**
>
> Este módulo não oferece suporte à reprodução de uma imagem de memória de forma assíncrona, portanto, uma combinação deste sinalizador e `SND_ASYNC` levantará `RuntimeError`.

winsound.**SND_PURGE**

Para de reproduzir todas as instâncias do som especificado.

> **Nota**
>
> Não há suporte a este sinalizador em plataformas Windows modernas.

winsound.**SND_ASYNC**

Retorna imediatamente, permitindo que os sons sejam reproduzidos de forma assíncrona.

winsound.**SND_NODEFAULT**

Se o som especificado não for encontrado, o som padrão do sistema não será reproduzido.

winsound.**SND_NOSTOP**

Não interrompe os sons que estão sendo reproduzidos.

winsound.**SND_NOWAIT**

Retorna imediatamente se o driver de som estiver ocupado.

> **Nota**
>
> Não há suporte a este sinalizador em plataformas Windows modernas.

winsound.**SND_APPLICATION**

O parâmetro *sound* é um apelido específico da aplicação no registro. Este sinalizador pode ser combinado com o sinalizador `SND_ALIAS` para especificar um apelido de som definido pela aplicação.

winsound.**SND_SENTRY**

Aciona um evento SoundSentry quando o som é reproduzido.

Adicionado na versão 3.14.

winsound.**SND_SYNC**

O som é reproduzido de forma sincronizada. Este é o comportamento padrão.

Adicionado na versão 3.14.

winsound.**SND_SYSTEM**

Atribui o som à sessão de áudio para sons de notificação do sistema.

Adicionado na versão 3.14.

winsound.**MB_ICONASTERISK**

Reproduz o som `SystemDefault`.

winsound.**MB_ICONEXCLAMATION**

Reproduz o som `SystemExclamation`.

winsound.**MB_ICONHAND**

Reproduz o som `SystemHand`.

winsound.**MB_ICONQUESTION**

Reproduz o som `SystemQuestion`.

winsound.**MB_OK**

Reproduz o som `SystemDefault`.

winsound.**MB_ICONERROR**

Reproduz o som `SystemHand`.

Adicionado na versão 3.14.

winsound.**MB_ICONINFORMATION**

Reproduz o som `SystemDefault`.

Adicionado na versão 3.14.

winsound.**MB_ICONSTOP**

Reproduz o som `SystemHand`.

Adicionado na versão 3.14.

winsound.**MB_ICONWARNING**

Reproduz o som `SystemExclamation`.

Adicionado na versão 3.14.

CAPÍTULO 34

Serviços específicos do Unix

Os módulos descritos neste capítulo fornecem interfaces para recursos que são exclusivos do sistema operacional Unix ou, em alguns casos, para algumas ou várias variantes do mesmo. Aqui está uma visão geral:

34.1 `shlex` — Simple lexical analysis

Código-fonte: Lib/shlex.py

The *shlex* class makes it easy to write lexical analyzers for simple syntaxes resembling that of the Unix shell. This will often be useful for writing minilanguages, (for example, in run control files for Python applications) or for parsing quoted strings.

The *shlex* module defines the following functions:

shlex.**split**(*s*, *comments=False*, *posix=True*)

Split the string *s* using shell-like syntax. If *comments* is *False* (the default), the parsing of comments in the given string will be disabled (setting the *commenters* attribute of the *shlex* instance to the empty string). This function operates in POSIX mode by default, but uses non-POSIX mode if the *posix* argument is false.

Alterado na versão 3.12: Passing None for *s* argument now raises an exception, rather than reading *sys.stdin*.

shlex.**join**(*split_command*)

Concatenate the tokens of the list *split_command* and return a string. This function is the inverse of *split()*.

```
>>> from shlex import join
>>> print(join(['echo', '-n', 'Multiple words']))
echo -n 'Multiple words'
```

The returned value is shell-escaped to protect against injection vulnerabilities (see *quote()*).

Adicionado na versão 3.8.

shlex.**quote**(*s*)

Return a shell-escaped version of the string *s*. The returned value is a string that can safely be used as one token in a shell command line, for cases where you cannot use a list.

> **Aviso**
>
> The `shlex` module is **only designed for Unix shells**.
>
> The *`quote()`* function is not guaranteed to be correct on non-POSIX compliant shells or shells from other operating systems such as Windows. Executing commands quoted by this module on such shells can open up the possibility of a command injection vulnerability.
>
> Consider using functions that pass command arguments with lists such as *`subprocess.run()`* with `shell=False`.

Este idioma seria inseguro:

```
>>> filename = 'somefile; rm -rf ~'
>>> command = 'ls -l {}'.format(filename)
>>> print(command)  # executed by a shell: boom!
ls -l somefile; rm -rf ~
```

`quote()` lets you plug the security hole:

```
>>> from shlex import quote
>>> command = 'ls -l {}'.format(quote(filename))
>>> print(command)
ls -l 'somefile; rm -rf ~'
>>> remote_command = 'ssh home {}'.format(quote(command))
>>> print(remote_command)
ssh home 'ls -l '"'"'somefile; rm -rf ~'"'"''
```

The quoting is compatible with UNIX shells and with *`split()`*:

```
>>> from shlex import split
>>> remote_command = split(remote_command)
>>> remote_command
['ssh', 'home', "ls -l 'somefile; rm -rf ~'"]
>>> command = split(remote_command[-1])
>>> command
['ls', '-l', 'somefile; rm -rf ~']
```

Adicionado na versão 3.3.

The *`shlex`* module defines the following class:

class shlex.**shlex**(*instream=None*, *infile=None*, *posix=False*, *punctuation_chars=False*)

A *`shlex`* instance or subclass instance is a lexical analyzer object. The initialization argument, if present, specifies where to read characters from. It must be a file-/stream-like object with *`read()`* and *`readline()`* methods, or a string. If no argument is given, input will be taken from `sys.stdin`. The second optional argument is a filename string, which sets the initial value of the *`infile`* attribute. If the *instream* argument is omitted or equal to `sys.stdin`, this second argument defaults to "stdin". The *posix* argument defines the operational mode: when *posix* is not true (default), the *`shlex`* instance will operate in compatibility mode. When operating in POSIX mode, *`shlex`* will try to be as close as possible to the POSIX shell parsing rules. The *punctuation_chars* argument provides a way to make the behaviour even closer to how real shells parse. This can take a number of values: the default value, `False`, preserves the behaviour seen under Python 3.5 and earlier. If set to `True`, then parsing of the characters `();<>|&` is changed: any run of these characters (considered punctuation characters) is returned as a single token. If set to a non-empty string of characters, those characters will be used as the punctuation characters. Any characters in the *`wordchars`* attribute that appear in *punctuation_chars* will be removed from *`wordchars`*. See *Improved Compatibility with Shells* for more information. *punctuation_chars* can be set only upon *`shlex`* instance creation and can't be modified later.

Alterado na versão 3.6: The *punctuation_chars* parameter was added.

Ver também

Module *configparser*
Parser for configuration files similar to the Windows `.ini` files.

34.1.1 shlex Objects

A *shlex* instance has the following methods:

`shlex.`**`get_token`**`()`

Return a token. If tokens have been stacked using *push_token()*, pop a token off the stack. Otherwise, read one from the input stream. If reading encounters an immediate end-of-file, *eof* is returned (the empty string (`''`) in non-POSIX mode, and `None` in POSIX mode).

`shlex.`**`push_token`**`(`*str*`)`

Push the argument onto the token stack.

`shlex.`**`read_token`**`()`

Read a raw token. Ignore the pushback stack, and do not interpret source requests. (This is not ordinarily a useful entry point, and is documented here only for the sake of completeness.)

`shlex.`**`sourcehook`**`(`*filename*`)`

When *shlex* detects a source request (see *source* below) this method is given the following token as argument, and expected to return a tuple consisting of a filename and an open file-like object.

Normally, this method first strips any quotes off the argument. If the result is an absolute pathname, or there was no previous source request in effect, or the previous source was a stream (such as `sys.stdin`), the result is left alone. Otherwise, if the result is a relative pathname, the directory part of the name of the file immediately before it on the source inclusion stack is prepended (this behavior is like the way the C preprocessor handles `#include "file.h"`).

The result of the manipulations is treated as a filename, and returned as the first component of the tuple, with *open()* called on it to yield the second component. (Note: this is the reverse of the order of arguments in instance initialization!)

This hook is exposed so that you can use it to implement directory search paths, addition of file extensions, and other namespace hacks. There is no corresponding 'close' hook, but a shlex instance will call the *close()* method of the sourced input stream when it returns EOF.

For more explicit control of source stacking, use the *push_source()* and *pop_source()* methods.

`shlex.`**`push_source`**`(`*newstream*, *newfile=None*`)`

Push an input source stream onto the input stack. If the filename argument is specified it will later be available for use in error messages. This is the same method used internally by the *sourcehook()* method.

`shlex.`**`pop_source`**`()`

Pop the last-pushed input source from the input stack. This is the same method used internally when the lexer reaches EOF on a stacked input stream.

`shlex.`**`error_leader`**`(`*infile=None*, *lineno=None*`)`

This method generates an error message leader in the format of a Unix C compiler error label; the format is `'"%s", line %d: '`, where the `%s` is replaced with the name of the current source file and the `%d` with the current input line number (the optional arguments can be used to override these).

This convenience is provided to encourage *shlex* users to generate error messages in the standard, parseable format understood by Emacs and other Unix tools.

Instances of *shlex* subclasses have some public instance variables which either control lexical analysis or can be used for debugging:

shlex.**commenters**

The string of characters that are recognized as comment beginners. All characters from the comment beginner to end of line are ignored. Includes just `'#'` by default.

shlex.**wordchars**

The string of characters that will accumulate into multi-character tokens. By default, includes all ASCII alphanumerics and underscore. In POSIX mode, the accented characters in the Latin-1 set are also included. If *punctuation_chars* is not empty, the characters `~-./*?=`, which can appear in filename specifications and command line parameters, will also be included in this attribute, and any characters which appear in `punctuation_chars` will be removed from `wordchars` if they are present there. If *whitespace_split* is set to `True`, this will have no effect.

shlex.**whitespace**

Characters that will be considered whitespace and skipped. Whitespace bounds tokens. By default, includes space, tab, linefeed and carriage-return.

shlex.**escape**

Characters that will be considered as escape. This will be only used in POSIX mode, and includes just `'\'` by default.

shlex.**quotes**

Characters that will be considered string quotes. The token accumulates until the same quote is encountered again (thus, different quote types protect each other as in the shell.) By default, includes ASCII single and double quotes.

shlex.**escapedquotes**

Characters in *quotes* that will interpret escape characters defined in *escape*. This is only used in POSIX mode, and includes just `'"'` by default.

shlex.**whitespace_split**

If `True`, tokens will only be split in whitespaces. This is useful, for example, for parsing command lines with *shlex*, getting tokens in a similar way to shell arguments. When used in combination with *punctuation_chars*, tokens will be split on whitespace in addition to those characters.

Alterado na versão 3.8: The *punctuation_chars* attribute was made compatible with the *whitespace_split* attribute.

shlex.**infile**

The name of the current input file, as initially set at class instantiation time or stacked by later source requests. It may be useful to examine this when constructing error messages.

shlex.**instream**

The input stream from which this *shlex* instance is reading characters.

shlex.**source**

This attribute is `None` by default. If you assign a string to it, that string will be recognized as a lexical-level inclusion request similar to the `source` keyword in various shells. That is, the immediately following token will be opened as a filename and input will be taken from that stream until EOF, at which point the *close()* method of that stream will be called and the input source will again become the original input stream. Source requests may be stacked any number of levels deep.

shlex.**debug**

If this attribute is numeric and `1` or more, a *shlex* instance will print verbose progress output on its behavior. If you need to use this, you can read the module source code to learn the details.

shlex.**lineno**

Source line number (count of newlines seen so far plus one).

shlex.**token**

The token buffer. It may be useful to examine this when catching exceptions.

shlex.**eof**

Token used to determine end of file. This will be set to the empty string (''), in non-POSIX mode, and to None in POSIX mode.

shlex.**punctuation_chars**

A read-only property. Characters that will be considered punctuation. Runs of punctuation characters will be returned as a single token. However, note that no semantic validity checking will be performed: for example, '»>' could be returned as a token, even though it may not be recognised as such by shells.

Adicionado na versão 3.6.

34.1.2 Regras de análise

When operating in non-POSIX mode, *shlex* will try to obey to the following rules.

- Quote characters are not recognized within words (Do"Not"Separate is parsed as the single word Do"Not"Separate);
- Escape characters are not recognized;
- Enclosing characters in quotes preserve the literal value of all characters within the quotes;
- Closing quotes separate words ("Do"Separate is parsed as "Do" and Separate);
- If *whitespace_split* is False, any character not declared to be a word character, whitespace, or a quote will be returned as a single-character token. If it is True, *shlex* will only split words in whitespaces;
- EOF is signaled with an empty string ('');
- It's not possible to parse empty strings, even if quoted.

When operating in POSIX mode, *shlex* will try to obey to the following parsing rules.

- Quotes are stripped out, and do not separate words ("Do"Not"Separate" is parsed as the single word DoNotSeparate);
- Non-quoted escape characters (e.g. '\') preserve the literal value of the next character that follows;
- Enclosing characters in quotes which are not part of *escapedquotes* (e.g. "'") preserve the literal value of all characters within the quotes;
- Enclosing characters in quotes which are part of *escapedquotes* (e.g. '"') preserves the literal value of all characters within the quotes, with the exception of the characters mentioned in *escape*. The escape characters retain its special meaning only when followed by the quote in use, or the escape character itself. Otherwise the escape character will be considered a normal character.
- EOF is signaled with a *None* value;
- Quoted empty strings ('') are allowed.

34.1.3 Improved Compatibility with Shells

Adicionado na versão 3.6.

The *shlex* class provides compatibility with the parsing performed by common Unix shells like bash, dash, and sh. To take advantage of this compatibility, specify the punctuation_chars argument in the constructor. This defaults to False, which preserves pre-3.6 behaviour. However, if it is set to True, then parsing of the characters ();<>|& is changed: any run of these characters is returned as a single token. While this is short of a full parser for shells (which would be out of scope for the standard library, given the multiplicity of shells out there), it does allow you to perform processing of command lines more easily than you could otherwise. To illustrate, you can see the difference in the following snippet:

```
>>> import shlex
>>> text = "a && b; c && d || e; f >'abc'; (def \"ghi\")"
>>> s = shlex.shlex(text, posix=True)
>>> s.whitespace_split = True
```

(continua na próxima página)

(continuação da página anterior)

```
>>> list(s)
['a', '&&', 'b;', 'c', '&&', 'd', '||', 'e;', 'f', '>abc;', '(def', 'ghi)']
>>> s = shlex.shlex(text, posix=True, punctuation_chars=True)
>>> s.whitespace_split = True
>>> list(s)
['a', '&&', 'b', ';', 'c', '&&', 'd', '||', 'e', ';', 'f', '>', 'abc', ';',
'(', 'def', 'ghi', ')']
```

Of course, tokens will be returned which are not valid for shells, and you'll need to implement your own error checks on the returned tokens.

Instead of passing `True` as the value for the punctuation_chars parameter, you can pass a string with specific characters, which will be used to determine which characters constitute punctuation. For example:

```
>>> import shlex
>>> s = shlex.shlex("a && b || c", punctuation_chars="|")
>>> list(s)
['a', '&', '&', 'b', '||', 'c']
```

Nota

When `punctuation_chars` is specified, the *wordchars* attribute is augmented with the characters `~-./*?=`. That is because these characters can appear in file names (including wildcards) and command-line arguments (e.g. `--color=auto`). Hence:

```
>>> import shlex
>>> s = shlex.shlex('~/a && b-c --color=auto || d *.py?',
...                 punctuation_chars=True)
>>> list(s)
['~/a', '&&', 'b-c', '--color=auto', '||', 'd', '*.py?']
```

However, to match the shell as closely as possible, it is recommended to always use `posix` and *whitespace_split* when using *punctuation_chars*, which will negate *wordchars* entirely.

For best effect, `punctuation_chars` should be set in conjunction with `posix=True`. (Note that `posix=False` is the default for *shlex*.)

34.2 `posix` — As chamadas de sistema mais comuns do POSIX

Este módulo fornece acesso à funcionalidade do sistema operacional padronizada pelo padrão C e pelo padrão POSIX (uma interface Unix levemente disfarçada).

Disponibilidade: Unix.

Não importe este módulo diretamente. Em vez disso, importe o módulo *os*, que fornece uma versão *portátil* dessa interface. No Unix, o módulo *os* fornece um superconjunto da interface *posix*. Em sistemas operacionais não Unix, o módulo *posix* não está disponível, mas um subconjunto está sempre disponível na interface *os*. Uma vez que *os* é importado, seu uso *não* causa penalidade de desempenho em comparação com *posix*. Além disso, *os* fornece algumas funcionalidades adicionais, como chamar automaticamente *putenv()* quando uma entrada em `os.environ` é alterada.

Erros são relatados como exceções. As exceções usuais são dadas para erros de tipo, enquanto os erros relatados pelas chamadas do sistema levantam *OSError*.

34.2.1 Suporte a arquivos grandes

Vários sistemas operacionais (incluindo AIX e Solaris) fornecem suporte a arquivos maiores que 2 GiB a partir de um modelo de programação C em que `int` e `long` são valores de 32 bits. Isso geralmente é realizado definindo o tamanho relevante e os tipos de deslocamento como valores de 64 bits. Esses arquivos às vezes são chamados de *arquivos grandes*.

O suporte a arquivos grandes é ativado no Python quando o tamanho de um `off_t` é maior que a `long` e `long long` é pelo menos tão grande quanto um `off_t`. Pode ser necessário configurar e compilar o Python com certos sinalizadores do compilador para ativar esse modo. Por exemplo, com o Solaris 2.6 e 2.7, você precisa fazer algo como:

```
CFLAGS="`getconf LFS_CFLAGS`" OPT="-g -O2 $CFLAGS" \
        ./configure
```

Em sistemas Linux com capacidade para arquivos grandes, isso pode funcionar:

```
CFLAGS='-D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64' OPT="-g -O2 $CFLAGS" \
        ./configure
```

34.2.2 Conteúdo notável do módulo

Além de muitas funções descritas na documentação do módulo *os*, *posix* define o seguinte item de dados:

`posix.`**`environ`**

Um dicionário que representa o ambiente de strings no momento em que o interpretador foi iniciado. Chaves e valores são bytes no Unix e str no Windows. Por exemplo, `environ[b'HOME']` (`environ['HOME']` no Windows) é o nome do caminho do diretório inicial, equivalente a `getenv("HOME")` em C .

A modificação deste dicionário não afeta o ambiente de strings passado por *execv()*, *popen()* ou *system()*. Se você precisar alterar o ambiente, passe `environ` para *execve()* ou adicione atribuições de variável e instruções de exportação para a string de comando para *system()* ou *popen()*.

Alterado na versão 3.2: No Unix, chaves e valores são bytes.

> **Nota**
>
> O módulo *os* fornece uma implementação alternativa de `environ` que atualiza o ambiente ao ocorrerem modificações. Observe também que a atualização de *os.environ* tornará este dicionário obsoleto. O uso do módulo *os* é recomendado sobre o acesso direto ao módulo *posix*.

34.3 `pwd` — A senha do banco de dados

Este módulo provê acesso ao banco de dados das contas de usuário do sistema e suas respectivas senhas. Isto está disponível para todas as versões do Unix.

Disponibilidade: Unix, not WASI, not iOS.

As entradas do banco de dados de senhas são reportadas como um objeto do tipo tupla, cujos atributos correspondem aos membros da estrutura `passwd` (Campos dos atributos abaixo, veja `<pwd.h>`):

Índice	Atributo	Significado
0	`pw_name`	Nome de login
1	`pw_passwd`	Senha encriptada opcional
2	`pw_uid`	ID numérico do usuário
3	`pw_gid`	ID numérico do grupo
4	`pw_gecos`	Nome do usuário ou campo de comentário
5	`pw_dir`	Diretório home do usuário
6	`pw_shell`	Interpretador de comandos do usuário

O uid e o gid são números inteiros, e os outros são strings. *KeyError* é levando se o campo requerido não puder ser encontrado.

Nota

Em Unix tradicional, o campo `pw_passwd` geralmente contém uma senha encriptada com um algoritmo derivado de DES. No entanto, a maioria dos sistemas tipo Unix modernos usam o chamado sistema *shadow password*. Nesses Unixes o campo *pw_passwd* só contém um asterisco (`'*'`) ou a letra `'x'` e a senha encriptada é guardada no arquivo `/etc/shadow` o qual não é permitido o acesso irrestrito a leitura. Se o campo *pw_passwd* contém alguma coisa útil dependerá do sistema.

Isto define os seguintes itens

`pwd.`**`getpwuid`**`(`*uid*`)`

Retorna a entrada do banco de dados de senhas para um dado ID de usuário

`pwd.`**`getpwnam`**`(`*name*`)`

Retorna a entrada do banco de dados de senhas para um dado nome de usuário

`pwd.`**`getpwall`**`()`

Retorna uma lista de todos as entradas disponíveis no banco de dados de senhas, em uma ordem arbitrária.

Ver também

Módulo *grp*
Uma interface para o banco de dados do grupo, similar a esta.

34.4 `grp` — O banco de dados de grupos

Este módulo fornece acesso ao banco de dados de grupos Unix. Ele está disponível em todas as versões do Unix.

Disponibilidade: Unix, not WASI, not Android, not iOS.

As entradas do banco de dados de grupos são relatadas como um objeto tupla ou similar, cujos atributos correspondem aos membros da estrutura `group` (campo Atributo abaixo, consulte `<grp.h>`):

Índice	Atributo	Significado
0	gr_name	o nome do grupo
1	gr_passwd	a senha do grupo (criptografada); geralmente vazia
2	gr_gid	O ID numérico do grupo
3	gr_mem	todos os nomes de usuários dos membros do grupo

O gid é um inteiro, nome e senha são strings, e a lista de membros é uma lista de strings. (Observe que a maioria dos usuários não são listados explicitamente como membros do grupo em que estão, de acordo com o banco de dados de senhas. Verifique ambos os bancos de dados para obter informações completas sobre associação. Observe também que um `gr_name` que começa com `+` ou `-` provavelmente é uma referência YP/NIS e pode não ser acessível via *`getgrnam()`* ou *`getgrgid()`*.)

Isto define os seguintes itens

`grp.`**`getgrgid`**(*id*)

Retorna a entrada do banco de dados de grupos para o ID do grupo numérico fornecido. *`KeyError`* é levantada se a entrada solicitada não puder ser encontrada.

Alterado na versão 3.10: *`TypeError`* é levantada para argumentos não inteiros, como pontos flutuantes ou strings.

`grp.`**`getgrnam`**(*name*)

Retorna a entrada do banco de dados de grupos para o nome do grupo fornecido. *`KeyError`* é levantada se a entrada solicitada não puder ser encontrada.

`grp.`**`getgrall`**()

Retorna uma lista de todas as entradas de grupo disponíveis, em ordem arbitrária.

Ver também

Módulo *`pwd`*
Uma interface para o banco de dados de usuários, semelhante a esta.

34.5 `termios` — Controle de tty no estilo POSIX

Este módulo fornece uma interface para as chamadas POSIX para controle de E/S do tty. Para uma descrição completa dessas chamadas, consulte a página de manual Unix *`termios(3)`*. Está disponível apenas para as versões Unix que tenham suporte ao controle de E/S de tty no estilo POSIX do *termios* configurado durante a instalação.

Disponibilidade: Unix.

Todas as funções neste módulo usam um descritor de arquivo *fd* como seu primeiro argumento. Pode ser um descritor de arquivo de tipo inteiro, como retornado por `sys.stdin.fileno()`, ou um *objeto arquivo*, como o próprio `sys.stdin`.

Este módulo também define todas as constantes necessárias para trabalhar com as funções fornecidas aqui; estes têm o mesmo nome de seus equivalentes em C. Consulte a documentação do sistema para mais informações sobre o uso dessas interfaces de controle de terminal.

O módulo define as seguintes funções:

`termios.`**`tcgetattr`**(*fd*)

Retorna uma lista contendo os atributos tty para o descritor de arquivo *fd*, da seguinte forma: `[iflag, oflag, cflag, lflag, ispeed, ospeed, cc]` onde *cc* é uma lista dos caracteres especiais do tty (cada uma string de comprimento 1, exceto os itens com índices `VMIN` e `VTIME`, que são números inteiros quando esses campos são definidos). A interpretação dos sinalizadores e as velocidades, bem como a indexação no vetor *cc*, devem ser feitas usando as constantes simbólicas definidas no módulo *`termios`*.

`termios.`**`tcsetattr`**(*fd*, *when*, *attributes*)

Define os atributos do tty para descritor de arquivo *fd* a partir de *attributes*, que é uma lista como a retornada por *`tcgetattr()`*. O argumento *when* determina quando os atributos são alterados:

`termios.`**`TCSANOW`**

Altera os atributos imediatamente.

termios.**TCSADRAIN**

Altera os atributos depois de transmitir todas as saídas enfileiradas.

termios.**TCSAFLUSH**

Altera os atributos depois de transmitir todas as saídas enfileiradas e descartar todas as entradas enfileiradas.

termios.**tcsendbreak**(*fd*, *duration*)

Envia uma quebra no descritor de arquivo *fd*. Uma duração zero, representada por *duration*, envia uma pausa por 0,25 a 0,5 segundos; *duration* com valor diferente de zero tem um significado dependente do sistema.

termios.**tcdrain**(*fd*)

Aguarda até que toda a saída escrita no descritor de arquivo *fd* seja transmitida.

termios.**tcflush**(*fd*, *queue*)

Descarta dados na fila no descritor de arquivo *fd*. O seletor *queue* especifica qual fila: `TCIFLUSH` para a fila de entrada, `TCOFLUSH` para a fila de saída ou `TCIOFLUSH` para as duas filas.

termios.**tcflow**(*fd*, *action*)

Suspende ou retoma a entrada ou saída no descritor de arquivo *fd*. O argumento *action* pode ser `TCOOFF` para suspender a saída, `TCOON` para reiniciar a saída, `TCIOFF` para suspender a entrada ou `TCION` para reiniciar a entrada.

termios.**tcgetwinsize**(*fd*)

Retorna uma tupla `(ws_row, ws_col)` contendo o tamanho da janela de terminal para o descritor de arquivo *fd*. Requer `termios.TIOCGWINSZ` ou `termios.TIOCGSIZE`.

Adicionado na versão 3.11.

termios.**tcsetwinsize**(*fd*, *winsize*)

Define o tamanho da janela de terminal para o descritor de arquivo *fd* de *winsize*, que é uma tupla de dois elementos `(ws_row, ws_col)` como a retornada por *tcgetwinsize()*. Requer pelo menos um dos pares (`termios.TIOCGWINSZ`, `termios.TIOCSWINSZ`); (`termios.TIOCGSIZE`, `termios.TIOCSSIZE`) a ser definido.

Adicionado na versão 3.11.

Ver também

Módulo *tty*
Funções de conveniência para operações comuns de controle de terminal.

34.5.1 Exemplo

Aqui está uma função que solicita uma senha com o eco desativado. Observe a técnica usando uma chamada separada *tcgetattr()* e uma instrução `try` … `finally` para garantir que os atributos tty antigos sejam restaurados exatamente, aconteça o que acontecer:

```
def getpass(prompt="Senha: "):
    import termios, sys
    fd = sys.stdin.fileno()
    old = termios.tcgetattr(fd)
    new = termios.tcgetattr(fd)
    new[3] = new[3] & ~termios.ECHO          # lflags
    try:
        termios.tcsetattr(fd, termios.TCSADRAIN, new)
        passwd = input(prompt)
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old)
    return passwd
```

34.6 tty — Funções de controle de terminal

Código-fonte: Lib/tty.py

O módulo *tty* define funções para colocar o tty nos modos de cbreak e não tratados (raw).

Disponibilidade: Unix.

Por requerer o módulo *termios*, ele funcionará apenas no Unix.

O módulo *tty* define as seguintes funções:

tty.**cfmakeraw**(*mode*)

Converte a lista de atributos de tty *mode*, que é uma lista como a retornada por *termios.tcgetattr()*, para a de um tty em modo raw.

Adicionado na versão 3.12.

tty.**cfmakecbreak**(*mode*)

Converte a lista de atributos de tty *mode*, que é uma lista como a retornada por *termios.tcgetattr()*, para a de um tty em modo cbreak.

Isto limpa os sinalizadores de modo local ECHO e ICANON em *mode* bem como configura a entrada mínima para 1 byte sem atraso.

Adicionado na versão 3.12.

Alterado na versão 3.12.2: O sinalizador ICRNL não está mais limpo. Isso corresponde ao comportamento stty cbreak do Linux e macOS e ao que *setcbreak()* historicamente fez.

tty.**setraw**(*fd*, *when=termios.TCSAFLUSH*)

Altera o modo do descritor de arquivo *fd* para raw. Se *when* for omitido, o padrão é *termios.TCSAFLUSH*, e é passado para *termios.tcsetattr()*. O retorna valor de *termios.tcgetattr()* é salvo antes de definir *fd* para o modo raw; esse valor é retornado.

Alterado na versão 3.12: O valor de retorno agora é o atributos originais do tty, em vez de None.

tty.**setcbreak**(*fd*, *when=termios.TCSAFLUSH*)

Altera o modo de descritor de arquivo *fd* para cbreak. Se *when* for omitido, o padrão é *termios.TCSAFLUSH*, e é passado para *termios.tcsetattr()*. O retorna valor de *termios.tcgetattr()* é salvo antes de definir *fd* para o modo cbreak; esse valor é retornado.

Isto limpa os sinalizadores de modo local ECHO e ICANON bem como configura a entrada mínima para 1 byte sem atraso.

Alterado na versão 3.12: O valor de retorno agora é o atributos originais do tty, em vez de None.

Alterado na versão 3.12.2: O sinalizador ICRNL não está mais limpo. Isso restaura o comportamento do Python 3.11 e anteriores, além de corresponder ao que Linux, macOS e BSDs descrevem em suas páginas de manual stty(1) em relação ao modo cbreak.

Ver também

Módulo *termios*
Interface baixo nível para controle de terminal.

34.7 pty — Utilitários de pseudoterminal

Código-fonte: Lib/pty.py

O módulo *pty* define operações para lidar com o conceito de pseudoterminal: iniciar outro processo e poder gravar e ler de seu terminal de controle programaticamente.

Disponibilidade: Unix.

O tratamento do pseudoterminal é altamente dependente da plataforma. Esse código foi testado principalmente no Linux, no FreeBSD e no macOS (supõe-se que funcione em outras plataformas POSIX, mas não foi testado exaustivamente).

O módulo *pty* define as seguintes funções:

pty.**fork**()

Faz um fork. Conecta o terminal de controle do filho a um pseudoterminal. O valor de retorno é `(pid, fd)`. Observe que a criança recebe *pid* 0 e o *fd* é *inválido*. O valor de retorno do pai é o *pid* do filho, e o *fd* é um descritor de arquivo conectado ao terminal de controle do filho (e também à entrada e à saída padrão do filho).

> **Aviso**
>
> No macOS, o uso desta função é inseguro quando combinado com o uso de APIs de sistema de nível mais alto, e isso inclui o uso de *urllib.request*.

pty.**openpty**()

Abre um novo par de pseudoterminais, usando *os.openpty()*, se possível, ou código de emulação para sistemas genérico Unix . Retorna um par de descritores de arquivo `(master, slave)`, para a extremidade mestre e escrava, respectivamente.

pty.**spawn**(*argv*[, *master_read*[, *stdin_read*]])

Gera um processo e conecta seu terminal de controle com o E/S padrão do processo atual. Isso é frequentemente usado para confundir programas que insistem em ler no terminal de controle. Espera-se que o processo gerado por trás do pty acabe sendo encerrado e, quando isso acontecer, o *spawn* é retornado.

Um laço copia o STDIN do processo atual para o filho e os dados recebidos do filho para o STDOUT do site processar atual. Não é sinalizado para o filho se o STDIN do processar atual fechar.

As funções *master_read* e o *stdin_read* recebem um descritor de arquivo do qual devem ler e devem sempre retorna uma string de bytes. Para forçar *spawn* retornar antes que o processo filho saia, um byte vazio vetor deve ser retornado para sinalizar o fim do arquivo.

A implementação padrão para ambos os funções vai ler e retornar até 1024 bytes cada vez que a função for chamada. A função de retorno *master_read* é passada para o descritor de arquivo mestre do pseudoterminal para ler a saída do processo filho, e ao *stdin_read* é passado o descritor de arquivo 0, para ler a entrada padrão do processo pai.

Retornar uma string de bytes vazia de qualquer uma das funções de retorno é interpretado como uma condição de fim de vida (EOF), e que a função de retorno não será chamada depois disso. Se *stdin_read* sinalizar EOF, o terminal de controle não poderá mais se comunicar com o processo pai OU o processo filho. A menos que o filho processar seja encerrado sem nenhuma entrada, *spawn* vai então fazer o laço para sempre. Se *master_read* sinalizar EOF, os mesmos comportamento resultados (pelo menos no Linux).

Retorna o valor de status de saída de *os.waitpid()* no processo filho.

os.waitstatus_to_exitcode() pode ser usado para converter o status de saída em um código de saída.

Levanta um *evento de auditoria* `pty.spawn` com o argumento `argv`.

Alterado na versão 3.4: *spawn()* agora retorna o valor de status de *os.waitpid()* no processo filho.

34.7.1 Exemplo

O programa a seguir funciona como o comando Unix *script(1)* , usando um pseudoterminal para registrar todas as entradas e saídas de uma sessão de terminal em um "script".

```
import argparse
import os
import pty
import sys
import time

parser = argparse.ArgumentParser()
parser.add_argument('-a', dest='append', action='store_true')
parser.add_argument('-p', dest='use_python', action='store_true')
parser.add_argument('filename', nargs='?', default='typescript')
options = parser.parse_args()

shell = sys.executable if options.use_python else os.environ.get('SHELL', 'sh')
filename = options.filename
mode = 'ab' if options.append else 'wb'

with open(filename, mode) as script:
    def read(fd):
        data = os.read(fd, 1024)
        script.write(data)
        return data

    print('Script started, file is', filename)
    script.write(('Script started on %s\n' % time.asctime()).encode())

    pty.spawn(shell, read)

    script.write(('Script done on %s\n' % time.asctime()).encode())
    print('Script done, file is', filename)
```

34.8 `fcntl` — as chamadas de sistema `fcntl` e `ioctl`

Este módulo executa o controle de arquivos e de E/S em descritores de arquivos. É uma interface para as rotinas `fcntl()` e `ioctl()` do Unix. Veja as páginas de manual do Unix *fcntl(2)* e *ioctl(2)* para mais detalhes.

Disponibilidade: Unix, not WASI.

Todas as funções neste módulo recebem um descritor de arquivo *fd* como seu primeiro argumento. Este pode ser um descritor de arquivo inteiro, como retornado por `sys.stdin.fileno()`, ou um objeto *io.IOBase*, como o próprio `sys.stdin`, que fornece um *fileno()* que retorna um descritor de arquivo genuíno.

Alterado na versão 3.3: As operações neste módulo costumavam levantar um *IOError*, mas agora levantam um *OSError*.

Alterado na versão 3.8: O módulo `fcntl` agora contém constantes `F_ADD_SEALS`, `F_GET_SEALS` e `F_SEAL_*` para selar descritores de arquivo *os.memfd_create()*.

Alterado na versão 3.9: No macOS, o módulo `fcntl` expõe a constante `F_GETPATH`, que obtém o caminho de um arquivo de um descritor de arquivo. No Linux(>=3.15), o módulo `fcntl` expõe as constantes `F_OFD_GETLK`, `F_OFD_SETLK` e `F_OFD_SETLKW`, que são usadas ao trabalhar com travas de descrição de arquivo aberto.

Alterado na versão 3.10: No Linux >= 2.6.11, o módulo `fcntl` expõe as constantes `F_GETPIPE_SZ` e `F_SETPIPE_SZ`, que permitem verificar e modificar o tamanho de um encadeamento, respectivamente.

Alterado na versão 3.11: No FreeBSD, o módulo `fcntl` expõe as constantes `F_DUP2FD` e `F_DUP2FD_CLOEXEC`, que permitem duplicar um descritor de arquivo, sendo que esta última define também o sinalizador `FD_CLOEXEC`.

Alterado na versão 3.12: No Linux >= 4.5, o módulo *fcntl* expõe as constantes FICLONE e FICLONERANGE, que permitem compartilhar alguns dados de um arquivo com outro arquivo por meio de reflinking em alguns sistemas de arquivos (por exemplo, btrfs, OCFS2 e XFS). Esse comportamento é comumente chamado de "copy-on-write".

Alterado na versão 3.13: No Linux >= 2.6.32, o módulo fcntl expõe as constantes F_GETOWN_EX, F_SETOWN_EX, F_OWNER_TID, F_OWNER_PID, F_OWNER_PGRP, que permitem direcionar sinais de disponibilidade de E/S para uma thread, processo ou grupo de processos específico. No Linux >= 4.13, o módulo fcntl expõe as constantes F_GET_RW_HINT, F_SET_RW_HINT, F_GET_FILE_RW_HINT, F_SET_FILE_RW_HINT e RWH_WRITE_LIFE_*, que permitem informar o kernel sobre o tempo de vida relativo esperado de gravações em um determinado nó-i ou por meio de uma descrição de arquivo aberto específica. No Linux >= 5.1 e NetBSD, o módulo fcntl expõe a constante F_SEAL_FUTURE_WRITE para uso com as operações F_ADD_SEALS e F_GET_SEALS. No FreeBSD, o módulo fcntl expõe as constantes F_READAHEAD, F_ISUNIONSTACK e F_KINFO. No macOS e FreeBSD, o módulo fcntl expõe a constante F_RDAHEAD. No NetBSD e AIX, o módulo fcntl expõe a constante F_CLOSEM. No NetBSD, o módulo fcntl expõe a constante F_MAXFD. No macOS e no NetBSD, o módulo fcntl expõe as constantes F_GETNOSIGPIPE e F_SETNOSIGPIPE.

Alterado na versão 3.14: No Linux >= 6.1, o módulo fcntl expõe o F_DUPFD_QUERY para consultar um descritor de arquivo apontando para o mesmo arquivo.

O módulo define as seguintes funções:

fcntl.**fcntl**(*fd*, *cmd*, *arg=0*, /)

Executa a operação *cmd* no descritor de arquivo *fd* (objetos arquivo que fornecem um método *fileno()* também são aceitos). Os valores usados para *cmd* dependem do sistema operacional e estão disponíveis como constantes no módulo *fcntl*, usando os mesmos nomes usados nos arquivos de cabeçalho C relevantes. O argumento *arg* pode ser um valor inteiro, um *objeto bytes ou similar* ou uma string. O tipo e o tamanho de *arg* devem corresponder ao tipo e ao tamanho do argumento da operação, conforme especificado na documentação C relevante.

Quando *arg* é um inteiro, a função retorna o valor de retorno inteiro da chamada C fcntl().

Quando o argumento é um objeto bytes ou similar, ele representa uma estrutura binária, por exemplo, criada por *struct.pack()*. Um valor de string é codificado em binário usando a codificação UTF-8. Os dados binários são copiados para um buffer cujo endereço é passado para a chamada fcntl() em C. O valor de retorno após uma chamada bem-sucedida é o conteúdo do buffer, convertido em um objeto *bytes*. O comprimento do objeto retornado será igual ao comprimento do argumento *arg*. Este é limitado a 1024 bytes.

Se a chamada a fcntl() falhar, um exceção *OSError* é levantada.

> **Nota**
>
> Se o tipo ou o tamanho de *arg* não corresponder ao tipo ou tamanho do argumento da operação (por exemplo, se um inteiro for passado quando um ponteiro for esperado, ou se as informações retornadas no buffer pelo sistema operacional forem maiores que 1024 bytes), é mais provável que isso resulte em uma violação de segmentação ou em uma corrupção de dados mais sutil.

Levanta um *evento de auditoria* fcntl.fcntl com os argumentos fd, cmd, arg.

Alterado na versão 3.14: Adicione suporte a *objetos bytes ou similares* arbitrários, não apenas *bytes*.

fcntl.**ioctl**(*fd*, *request*, *arg=0*, *mutate_flag=True*, /)

Esta função é idêntica à função *fcntl()*, exceto que o tratamento de argumentos é ainda mais complicado.

O parâmetro *request* é limitado a valores que cabem em 32 ou 64 bits, dependendo da plataforma. Constantes adicionais de interesse para uso como argumento *request* podem ser encontradas no módulo *termios*, com os mesmos nomes usados nos arquivos de cabeçalho C relevantes.

O parâmetro *arg* pode ser um inteiro, um *objeto bytes ou similar* ou uma string. O tipo e o tamanho de *arg* devem corresponder ao tipo e ao tamanho do argumento da operação, conforme especificado na documentação C relevante.

Se *arg* não oferecer suporte à interface de buffer de leitura e escrita ou o *mutate_flag* for falso, o comportamento será o mesmo da função *fcntl()*.

Se *arg* oferece suporte à interface de buffer de leitura e escrita (como *bytearray*) e *mutate_flag* for true (o padrão), o buffer será (na prática) passado para a chamada de sistema subjacente `ioctl()`, o código de retorno desta última será passado de volta para o Python que fez a chamada, e o novo conteúdo do buffer refletirá a ação de `ioctl()`. Esta é uma pequena simplificação, pois se o buffer fornecido tiver menos de 1024 bytes, ele será primeiro copiado para um buffer estático de 1024 bytes, que será então passado para *ioctl()* e copiado de volta para o buffer fornecido.

Se a chamada a `ioctl()` falhar, uma exceção *OSError* é levantada.

Nota

Se o tipo ou o tamanho de *arg* não corresponder ao tipo ou tamanho do argumento da operação (por exemplo, se um inteiro for passado quando um ponteiro for esperado, ou se as informações retornadas no buffer pelo sistema operacional forem maiores que 1024 bytes ou o tamanho do objeto byte ou similar mutável é pequeno demais), é mais provável que isso resulte em uma violação de segmentação ou em uma corrupção de dados mais sutil.

Um exemplo:

```
>>> import array, fcntl, struct, termios, os
>>> os.getpgrp()
13341
>>> struct.unpack('h', fcntl.ioctl(0, termios.TIOCGPGRP, "  "))[0]
13341
>>> buf = array.array('h', [0])
>>> fcntl.ioctl(0, termios.TIOCGPGRP, buf, 1)
0
>>> buf
array('h', [13341])
```

Levanta um *evento de auditoria* `fcntl.ioctl` com os argumentos `fd`, `request`, `arg`.

Alterado na versão 3.14: A GIL é sempre liberada durante uma chamada de sistema. Chamadas de sistema que falham com EINTR são repetidas automaticamente.

`fcntl.`**`flock`**(*fd*, *operation*, */*)

Executa a operação de trava *operation* no descritor de arquivo *fd* (objetos arquivo que fornecem um método *fileno()* também são aceitos). Consulte o manual do Unix *flock(2)* para obter detalhes. (Em alguns sistemas, esta função é emulada usando `fcntl()`.)

Se a chamada `flock()` falhar, uma exceção *OSError* será levantada.

Levanta um *evento de auditoria* `fcntl.flock` com os argumentos `fd` e `operation`.

`fcntl.`**`lockf`**(*fd*, *cmd*, *len=0*, *start=0*, *whence=0*, */*)

Este é essencialmente um invólucro em torno das chamadas de trava *fcntl()*. *fd* é o descritor de arquivo (objetos arquivo que fornecem um método *fileno()* também são aceitos) do arquivo para travar ou destravar, e *cmd* é um dos seguintes valores:

`fcntl.`**`LOCK_UN`**

Libera uma trava existente.

`fcntl.`**`LOCK_SH`**

Adquire uma trava compartilhada.

`fcntl.`**`LOCK_EX`**

Adquire uma trava exclusiva.

`fcntl.`**`LOCK_NB`**

Aplica OU (OR) bit a bit com qualquer uma das outras três constantes `LOCK_*` para tornar a solicitação não bloqueante.

Se LOCK_NB for usado e a trava não puder ser obtido, uma exceção *OSError* será levantada e terá um atributo *errno* definido como *EACCES* ou *EAGAIN* (dependendo do sistema operacional; para portabilidade, verifique ambos os valores). Em pelo menos alguns sistemas, LOCK_EX só pode ser usado se o descritor de arquivo se referir a um arquivo aberto para escrita.

len é o número de bytes para travar, *start* é o deslocamento de bytes em que a trava começa, em relação a *whence*, e *whence* é como em *io.IOBase.seek()*, especificamente:

- 0 – relativo ao início do arquivo (*os.SEEK_SET*)
- 1 – relativo à posição atual do buffer (*os.SEEK_CUR*)
- 2 – relativo ao fim do arquivo (*os.SEEK_END*)

O padrão para *start* é 0, o que significa iniciar no início do arquivo. O padrão para *len* é 0, o que significa travar no final do arquivo. O padrão para *whence* também é 0.

Levanta um *evento de auditoria* `fcntl.lockf` com os argumentos `fd`, `cmd`, `len`, `start`, `whence`.

Exemplos (todos em um sistema compatível com SVR4):

```
import struct, fcntl, os

f = open(...)
rv = fcntl.fcntl(f, fcntl.F_SETFL, os.O_NDELAY)

lockdata = struct.pack('hhllhh', fcntl.F_WRLCK, 0, 0, 0, 0, 0)
rv = fcntl.fcntl(f, fcntl.F_SETLKW, lockdata)
```

Observe que, no primeiro exemplo, a variável de valor de retorno *rv* conterá um valor inteiro; no segundo exemplo, ela conterá um objeto *bytes*. O layout da estrutura da variável *lockdata* depende do sistema — portanto, usar a chamada *flock()* pode ser melhor.

Ver também

Módulo *os*
Se os sinalizadores de trava *O_SHLOCK* e *O_EXLOCK* estiverem presentes no módulo *os* (somente no BSD), a função *os.open()* fornece uma alternativa às funções *lockf()* e *flock()*.

34.9 `resource` — Informação de uso de recursos

Este módulo fornece mecanismos básicos para medir e controlar os recursos do sistema usados por um programa.

Disponibilidade: Unix, not WASI.

Constantes simbólicas são usadas para especificar recursos específicos do sistema e para solicitar informações de uso sobre o processo atual ou seus filhos.

Uma exceção *OSError* é levantada em caso de falha de chamada de sistema.

exception resource.**error**
Um apelido descontinuado de *OSError*.

Alterado na versão 3.3: Seguindo a **PEP 3151**, esta classe foi transformada em um apelido de *OSError*.

34.9.1 Limites de recursos

O uso de recursos pode ser limitado usando a função *setrlimit()* descrita abaixo. Cada recurso é controlado por um par de limites: um limite suave e um limite rígido. O limite suave é o limite atual e pode ser reduzido ou aumentado por um processo ao longo do tempo. O limite suave nunca pode exceder o limite rígido. O limite rígido

pode ser reduzido para qualquer valor maior que o limite suave, mas não aumentado. (Somente processos com o UID efetivo do superusuário podem aumentar um limite rígido.)

Os recursos específicos que podem ser limitados dependem do sistema. Eles são descritos na página man *getrlimit(2)*. Os recursos listados abaixo são suportados quando o sistema operacional subjacente os suporta; recursos que não podem ser verificados ou controlados pelo sistema operacional não são definidos neste módulo para essas plataformas.

resource.**RLIM_INFINITY**

Constante usada para representar o limite de um recurso ilimitado.

resource.**getrlimit**(*resource*)

Retorna uma tupla `(suave, rígido)` com os limites soft e hard atuais de *resource*. Levanta *ValueError* se um recurso inválido for especificado, ou *error* se a chamada do sistema subjacente falhar inesperadamente.

resource.**setrlimit**(*resource*, *limits*)

Define novos limites de consumo de *resource*. O argumento *limits* deve ser uma tupla `(suave, rígido)` de dois inteiros descrevendo os novos limites. Um valor de *RLIM_INFINITY* pode ser usado para solicitar um limite ilimitado.

Levanta *ValueError* se um recurso inválido for especificado, se o novo limite suave exceder o limite rígido ou se um processo tentar aumentar seu limite rígido. Especificar um limite de *RLIM_INFINITY* quando o limite rígido ou do sistema para esse recurso não for ilimitado resultará em uma exceção *ValueError*. Um processo com o UID efetivo de superusuário pode solicitar qualquer valor limite válido, incluindo ilimitado, mas *ValueError* ainda será levantada se o limite solicitado exceder o limite imposto pelo sistema.

`setrlimit` também pode levantar *error* se a chamada do sistema subjacente falhar.

VxWorks só oferece suporte a definir *RLIMIT_NOFILE*.

Levanta um *evento de auditoria* `resource.setrlimit` com os argumentos `resource`, `limits`.

resource.**prlimit**(*pid*, *resource*[, *limits*])

Combina *setrlimit()* e *getrlimit()* em uma única função e permite obter e definir os limites de recursos de um processo arbitrário. Se *pid* for 0, a chamada se aplica ao processo atual. *resource* e *limits* têm o mesmo significado que em *setrlimit()*, exceto que *limits* é opcional.

Quando *limits* não é fornecido, a função retorna o limite de *resource* do processo *pid*. Quando *limits* é fornecido, o limite de *resource* do processo é definido e o limite de recurso anterior é retornado.

Levanta *ProcessLookupError* quando *pid* não pode ser encontrado e *PermissionError* quando o usuário não tem `CAP_SYS_RESOURCE` para o processo.

Levanta um *evento de auditoria* `resource.prlimit` com os argumentos `pid`, `resource`, `limits`.

Disponibilidade: Linux >= 2.6.36 with glibc >= 2.13.

Adicionado na versão 3.4.

Esses símbolos definem recursos cujo consumo pode ser controlado usando as funções *setrlimit()* e *getrlimit()* descritas abaixo. Os valores desses símbolos são exatamente as constantes usadas por programas em C.

A página man do Unix para *getrlimit(2)* lista os recursos disponíveis. Observe que nem todos os sistemas usam o mesmo símbolo ou o mesmo valor para denotar o mesmo recurso. Este módulo não tenta mascarar diferenças de plataforma — símbolos não definidos para uma plataforma não estarão disponíveis neste módulo para essa plataforma.

resource.**RLIMIT_CORE**

O tamanho máximo (em bytes) de um arquivo de núcleo (core file) que o processo atual pode criar. Isso pode resultar na criação de um arquivo de núcleo parcial se um núcleo maior for necessário para conter a imagem completa do processo.

resource.**RLIMIT_CPU**

O tempo máximo de processador (em segundos) que um processo pode usar. Se esse limite for excedido, um sinal `SIGXCPU` é enviado ao processo. (Consulte a documentação do módulo *signal* para obter informações sobre como capturar esse sinal e fazer algo útil, por exemplo, liberar arquivos abertos no disco.)

resource.**RLIMIT_FSIZE**

O tamanho máximo de um arquivo que o processo pode criar.

resource.**RLIMIT_DATA**

O tamanho máximo (em bytes) da heap do processo.

resource.**RLIMIT_STACK**

O tamanho máximo (em bytes) da pilha de chamadas para o processo atual. Isso afeta apenas a pilha da thread principal em um processo multithread.

resource.**RLIMIT_RSS**

O tamanho máximo do conjunto de residentes que deve ser disponibilizado para o processo.

resource.**RLIMIT_NPROC**

O número máximo de processos que o processo atual pode criar.

resource.**RLIMIT_NOFILE**

O número máximo de descritores de arquivo abertos para o processo atual.

resource.**RLIMIT_OFILE**

O nome BSD para *RLIMIT_NOFILE*.

resource.**RLIMIT_MEMLOCK**

O espaço máximo de endereço que pode ser travado na memória.

resource.**RLIMIT_VMEM**

A maior área de memória mapeada que o processo pode ocupar.

Disponibilidade: FreeBSD >= 11.

resource.**RLIMIT_AS**

A área máxima (em bytes) de espaço de endereço que pode ser ocupada pelo processo.

resource.**RLIMIT_MSGQUEUE**

O número de bytes que podem ser alocados para filas de mensagens POSIX.

Disponibilidade: Linux >= 2.6.8.

Adicionado na versão 3.4.

resource.**RLIMIT_NICE**

O teto para o nível agradável do processo (calculado como 20 - rlim_cur).

Disponibilidade: Linux >= 2.6.12.

Adicionado na versão 3.4.

resource.**RLIMIT_RTPRIO**

O teto da prioridade em tempo real.

Disponibilidade: Linux >= 2.6.12.

Adicionado na versão 3.4.

resource.**RLIMIT_RTTIME**

O limite de tempo (em microssegundos) no tempo de CPU que um processo pode gastar sob agendamento em tempo real sem fazer uma chamada de sistema de bloqueio.

Disponibilidade: Linux >= 2.6.25.

Adicionado na versão 3.4.

resource.**RLIMIT_SIGPENDING**

O número de sinais que o processo pode enfileirar.

Disponibilidade: Linux >= 2.6.8.

Adicionado na versão 3.4.

resource.**RLIMIT_SBSIZE**

O tamanho máximo (em bytes) de uso do buffer de soquete para este usuário. Isso limita a quantidade de memória de rede e, portanto, a quantidade de mbufs que este usuário pode armazenar a qualquer momento.

Disponibilidade: FreeBSD.

Adicionado na versão 3.4.

resource.**RLIMIT_SWAP**

O tamanho máximo (em bytes) do espaço de swap que pode ser reservado ou utilizado por todos os processos deste ID de usuário. Este limite é aplicado somente se o bit 1 do sysctl vm.overcommit estiver definido. Consulte tuning(7) para uma descrição completa deste sysctl.

Disponibilidade: FreeBSD.

Adicionado na versão 3.4.

resource.**RLIMIT_NPTS**

O número máximo de pseudoterminais criados por este ID de usuário.

Disponibilidade: FreeBSD.

Adicionado na versão 3.4.

resource.**RLIMIT_KQUEUES**

O número máximo de kqueues que esse ID de usuário tem permissão para criar.

Disponibilidade: FreeBSD >= 11.

Adicionado na versão 3.10.

34.9.2 Uso de recursos

Essas funções são usadas para recuperar informações de uso de recursos:

resource.**getrusage**(*who*)

Esta função retorna um objeto que descreve os recursos consumidos pelo processo atual ou seus filhos, conforme especificado pelo parâmetro *who*. O parâmetro *who* deve ser especificado usando uma das constantes `RUSAGE_*` descritas abaixo.

Um exemplo simples:

```
from resource import *
import time

# uma tarefa não vinculada à CPU
time.sleep(3)
print(getrusage(RUSAGE_SELF))

# uma tarefa vinculada à CPU
for i in range(10 ** 8):
    _ = 1 + 1
print(getrusage(RUSAGE_SELF))
```

Os campos do valor de retorno descrevem como um recurso específico do sistema foi utilizado, por exemplo, a quantidade de tempo gasto em execução no modo de usuário ou o número de vezes que o processo foi trocado da memória principal. Alguns valores dependem do tique do relógio interno, por exemplo, a quantidade de memória que o processo está usando.

Para retrocompatibilidade, o valor de retorno também pode ser acessado como uma tupla de 16 elementos.

Os campos `ru_utime` e `ru_stime` do valor de retorno são valores de ponto flutuante que representam o tempo gasto na execução no modo usuário e o tempo gasto na execução no modo sistema, respectivamente. Os valores restantes são inteiros. Consulte a página man *getrusage(2)* para obter informações detalhadas sobre esses valores. Um breve resumo é apresentado aqui:

Índice	Campo	Recurso
0	`ru_utime`	tempo no modo de usuário (segundos em pontos flutuantes)
1	`ru_stime`	tempo no modo de sistema (segundos em pontos flutuantes)
2	`ru_maxrss`	tamanho máximo do conjunto residente
3	`ru_ixrss`	tamanho da memória compartilhada
4	`ru_idrss`	tamanho da memória não compartilhada
5	`ru_isrss`	tamanho da pilha não compartilhada
6	`ru_minflt`	falhas de página que não requerem E/S
7	`ru_majflt`	falhas de página que requerem E/S
8	`ru_nswap`	número de trocas para a memórias de swap
9	`ru_inblock`	operações de entrada de bloco
10	`ru_oublock`	operações de saída de bloco
11	`ru_msgsnd`	mensagens enviadas
12	`ru_msgrcv`	mensagens recebidas
13	`ru_nsignals`	sinais recebidos
14	`ru_nvcsw`	trocas voluntárias de contexto
15	`ru_nivcsw`	trocas involuntárias de contexto

Esta função levantará uma *ValueError* se um parâmetro *who* inválido for especificado. Também pode levantar uma exceção *error* em circunstâncias incomuns.

`resource.`**`getpagesize`**`()`

Retorna o número de bytes em uma página do sistema. (Não precisa ser o mesmo que o tamanho da página de hardware.)

Os seguintes símbolos `RUSAGE_*` são passados para a função *getrusage()* para especificar quais informações de processos devem ser fornecidas.

`resource.`**`RUSAGE_SELF`**

Passeapara *getrusage()* para solicitar recursos consumidos pelo processo de chamada, que é a soma dos recursos usados por todas as threads no processo.

`resource.`**`RUSAGE_CHILDREN`**

Passa para *getrusage()* para solicitar recursos consumidos por processos filhos do processo de chamada que foram encerrados e aguardados.

`resource.`**`RUSAGE_BOTH`**

Passa para *getrusage()* para solicitar recursos consumidos pelo processo atual e pelos processos filhos. Pode não estar disponível em todos os sistemas.

`resource.`**`RUSAGE_THREAD`**

Passa para *getrusage()* para solicitar recursos consumidos pela thread atual. Pode não estar disponível em todos os sistemas.

Adicionado na versão 3.2.

34.10 `syslog` — Rotinas da biblioteca syslog do Unix

Este módulo fornece uma interface para as rotinas da biblioteca `syslog` do Unix. Consulte as páginas de manual do Unix para uma descrição detalhada do recurso `syslog`.

Disponibilidade: Unix, not WASI, not iOS.

Este módulo é uma camada para a família de rotinas `syslog` do sistema. Uma biblioteca Python pura que pode se comunicar com um servidor syslog está disponível no módulo *`logging.handlers`* como *`SysLogHandler`*.

O módulo define as seguintes funções:

`syslog.`**`syslog`**(*message*)

`syslog.`**`syslog`**(*priority*, *message*)

Envia a string *message* para o registrador do sistema. Uma nova linha final é adicionada, se necessário. Cada mensagem é marcada com uma prioridade composta por um *facility* e um *level*. O argumento opcional *priority*, cujo padrão é *`LOG_INFO`*, determina a prioridade da mensagem. Se a facilidade não está codificada em *priority* usando o OU lógico (`LOG_INFO | LOG_USER`), o valor dado na chamada *`openlog()`* é usado.

Se *`openlog()`* não foi chamado antes da chamada para *`syslog()`*, *`openlog()`* será chamado sem argumentos.

Levanta um *evento de auditoria* `syslog.syslog` com os argumentos `priority`, `message`.

Alterado na versão 3.2: Nas versões anteriores, *`openlog()`* não seria chamado automaticamente se não fosse chamado antes da chamada para *`syslog()`*, adiando a implementação do syslog para chamar `openlog()`.

Alterado na versão 3.12: Esta função é restrita em subinterpretadores. (Apenas o código que executa em vários interpretadores é afetado e a restrição não é relevante para a maioria dos usuários.) *`openlog()`* deve ser chamado no interpretador principal antes de *`syslog()`* pode ser usado em um subinterpretador. Caso contrário, vai levantar *`RuntimeError`*.

`syslog.`**`openlog`**([*ident*[, *logoption*[, *facility*]]])

As opções de log das chamadas subsequentes *`syslog()`* podem ser definidas chamando *`openlog()`*. *`syslog()`* irá chamar *`openlog()`* sem argumentos se o log não estiver aberto no momento.

O argumento nomeado opcional *ident* é uma string que é prefixada a cada mensagem, e o padrão é `sys.argv[0]` com os componentes do caminho inicial removidos. O argumento nomeado opcional *logoption* (o padrão é 0) é um campo de bits – veja abaixo os valores possíveis para combinar. O argumento nomeado opcional *facility* (o padrão é *`LOG_USER`*) define o recurso padrão para mensagens que não possuem um recurso explicitamente codificado.

Levanta um *evento de auditoria* `syslog.openlog` com os argumentos `ident`, `logoption`, `facility`.

Alterado na versão 3.2: Nas versões anteriores, os argumentos nomeados não eram permitidos e *ident* era obrigatório.

Alterado na versão 3.12: Esta função é restrita em subinterpretadores. (Apenas o código que executa em vários interpretadores é afetado e a restrição não é relevante para a maioria dos usuários.) Isso deve ser chamado no interpretador principal.Vai levantar *`RuntimeError`* se chamado em um subinterpretador.

`syslog.`**`closelog`**()

Redefine os valores do módulo syslog e chama a biblioteca de sistema `closelog()`.

Isso faz com que o módulo se comporte como quando importado inicialmente. Por exemplo, *`openlog()`* será chamado na primeira chamada *`syslog()`* (se *`openlog()`* ainda não foi chamado), e *ident* e outro *`openlog()`* os parâmetros são redefinidos para os padrões.

Levanta um *evento de auditoria* `syslog.closelog` sem argumentos.

Alterado na versão 3.12: Esta função é restrita em subinterpretadores. (Apenas o código que executa em vários interpretadores é afetado e a restrição não é relevante para a maioria dos usuários.) Isso deve ser chamado no interpretador principal.Vai levantar *`RuntimeError`* se chamado em um subinterpretador.

`syslog.`**`setlogmask`**(*maskpri*)

Define a máscara de prioridade como *maskpri* e retorna o valor da máscara anterior. Chamadas para *`syslog()`* com um nível de prioridade não definido em *maskpri* são ignoradas. O padrão é registrar todas as prioridades. A função `LOG_MASK(pri)` calcula a máscara para a prioridade individual *pri*. A função `LOG_UPTO(pri)` calcula a máscara para todas as prioridades até e incluindo *pri*.

Levanta um *evento de auditoria* `syslog.setlogmask` com o argumento `maskpri`.

O módulo define as seguintes constantes:

syslog.**LOG_EMERG**

syslog.**LOG_ALERT**

syslog.**LOG_CRIT**

syslog.**LOG_ERR**

syslog.**LOG_WARNING**

syslog.**LOG_NOTICE**

syslog.**LOG_INFO**

syslog.**LOG_DEBUG**

Níveis de prioridade (alto a baixo).

syslog.**LOG_AUTH**

syslog.**LOG_AUTHPRIV**

syslog.**LOG_CRON**

syslog.**LOG_DAEMON**

syslog.**LOG_FTP**

syslog.**LOG_INSTALL**

syslog.**LOG_KERN**

syslog.**LOG_LAUNCHD**

syslog.**LOG_LPR**

syslog.**LOG_MAIL**

syslog.**LOG_NETINFO**

syslog.**LOG_NEWS**

syslog.**LOG_RAS**

syslog.**LOG_REMOTEAUTH**

syslog.**LOG_SYSLOG**

syslog.**LOG_USER**

syslog.**LOG_UUCP**

syslog.**LOG_LOCAL0**

syslog.**LOG_LOCAL1**

syslog.**LOG_LOCAL2**

syslog.**LOG_LOCAL3**

syslog.**LOG_LOCAL4**

syslog.**LOG_LOCAL5**

syslog.**LOG_LOCAL6**

syslog.**LOG_LOCAL7**

Facilidades, dependendo da disponibilidade em <syslog.h> para *LOG_AUTHPRIV*, *LOG_FTP*, *LOG_NETINFO*, *LOG_REMOTEAUTH*, *LOG_INSTALL* e *LOG_RAS*.

Alterado na versão 3.13: Adicionados *LOG_FTP*, *LOG_NETINFO*, *LOG_REMOTEAUTH*, *LOG_INSTALL*, *LOG_RAS* e *LOG_LAUNCHD*.

syslog.**LOG_PID**

syslog.**LOG_CONS**

syslog.**LOG_NDELAY**

syslog.**LOG_ODELAY**

syslog.**LOG_NOWAIT**

syslog.**LOG_PERROR**

Opções de log, dependendo da disponibilidade em <syslog.h> para *LOG_ODELAY*, *LOG_NOWAIT* e *LOG_PERROR*.

34.10.1 Exemplos

Exemplo simples

Um conjunto simples de exemplos:

```
import syslog

syslog.syslog('Processing started')
if error:
    syslog.syslog(syslog.LOG_ERR, 'Processing started')
```

Um exemplo de configuração de algumas opções de log, isso incluiria o ID do processo nas mensagens registradas e escreveria as mensagens no recurso de destino usado para o log de correio:

```
syslog.openlog(logoption=syslog.LOG_PID, facility=syslog.LOG_MAIL)
syslog.syslog('E-mail processing initiated...')
```

CAPÍTULO 35

Interface de linha de comando (CLI) de módulos

Os seguintes módulos têm uma interface de linha de comando.

- *ast*
- *asyncio*
- `base64`
- *calendar*
- `code`
- *compileall*
- `cProfile`: veja *profile*
- *dis*
- *doctest*
- `encodings.rot_13`
- `ensurepip`
- `filecmp`
- `fileinput`
- `ftplib`
- *gzip*
- *http.server*
- `idlelib`
- *inspect*
- *json*
- *mimetypes*
- `pdb`
- *pickle*
- *pickletools*

- *platform*
- `poplib`
- *profile*
- `pstats`
- *py_compile*
- `pyclbr`
- `pydoc`
- `quopri`
- *random*
- `runpy`
- *site*
- *sqlite3*
- *symtable*
- *sysconfig*
- `tabnanny`
- *tarfile*
- `this`
- *timeit*
- *tokenize*
- *trace*
- `turtledemo`
- *unittest*
- *uuid*
- `venv`
- `webbrowser`
- *zipapp*
- *zipfile*

Consulte também a interface de linha de comando do Python.

CAPÍTULO 36

Módulos substituídos

Os módulos descritos neste capítulo foram substituídos por outros módulos para a maioria dos casos de uso e são mantidos principalmente para preservar a retrocompatibilidade.

Os módulos podem aparecer neste capítulo porque eles cobrem apenas um subconjunto limitado de um espaço de problema, e uma solução mais geralmente aplicável está disponível em outro lugar na biblioteca padrão (por exemplo, `getopt` cobre a tarefa muito específica de "imitar a API C `getopt()` no Python", em vez dos recursos mais amplos de análise de opções de linha de comando e análise de argumentos oferecidos por `optparse` e `argparse`).

Alternativamente, módulos podem aparecer neste capítulo porque estão totalmente descontinuados e aguardando remoção em uma versão futura, ou estão *suavemente descontinuados* e seu uso é ativamente desencorajado em novos projetos. Com a remoção de vários módulos obsoletos por meio da **PEP 594**, atualmente não há módulos nesta última categoria.

36.1 `getopt` — Analisador sintático no estilo C para opções de linha de comando

Código-fonte: Lib/getopt.py

Nota

Este módulo é considerado completo em recursos. Uma alternativa mais declarativa e extensível para esta API é fornecida no módulo `optparse`. Outros aprimoramentos funcionais para processamento de parâmetros de linha de comando são fornecidos como módulos de terceiros no PyPI ou como recursos no módulo `argparse`.

Este módulo ajuda os scripts a analisar os argumentos da linha de comando em `sys.argv`. Ele suporta as mesmas convenções da função Unix `getopt()` (incluindo os significados especiais de argumentos da forma '`-`' e '`--`'). Longas opções semelhantes às suportadas pelo software GNU também podem ser usadas por meio de um terceiro argumento opcional.

Usuários que não estão familiarizados com a função Unix `getopt()` devem considerar usar o módulo `argparse` em vez disso. Usuários que estão familiarizados com a função Unix `getopt()`, mas gostariam de obter um comportamento equivalente enquanto escrevem menos código e obtêm melhor ajuda e mensagens de erro devem considerar usar o módulo `optparse`. Veja *Escolhendo uma biblioteca de análise de linha de comando* para detalhes adicionais.

Este módulo fornece duas funções e uma exceção:

getopt.**getopt**(*args*, *shortopts*, *longopts=[]*)

Analisa opções de linha de comando e lista de parâmetros. *args* é a lista de argumentos a ser analisada, sem a referência inicial para o programa em execução. Normalmente, isso significa `sys.argv[1:]`. *shortopts* é a string de letras de opção que o script deseja reconhecer, com opções que requerem um argumento seguido por dois pontos (`':'`) e opções que aceitam o argumento opcional seguido por dois caracteres de dois pontos (`'::'`); ou seja, o mesmo formato que Unix `getopt()` usa.

> **Nota**
>
> Ao contrário do GNU `getopt()`, após um argumento sem opção, todos os argumentos adicionais são considerados também sem opção. Isso é semelhante à maneira como os sistemas Unix não GNU funcionam.

longopts, se especificado, deve ser uma lista de strings com os nomes das opções longas que devem ser suportadas. Os caracteres `'--'` no início não devem ser incluídos no nome da opção. Opções longas que requerem um argumento devem ser seguidas por um sinal de igual (`'='`). Opções longas que aceitam um argumento opcional devem ser seguidas por um sinal de igualdade e ponto de interrogação (`'=?'`). Para aceitar apenas opções longas, *shortopts* deve ser uma string vazia. Opções longas na linha de comando podem ser reconhecidas, desde que forneçam um prefixo do nome da opção que corresponda exatamente a uma das opções aceitas. Por exemplo, se *longopts* for `['foo', 'frob']`, a opção `--fo` irá corresponder a `--foo`, mas `--f` não corresponderá exclusivamente, então *GetoptError* será levantada.

O valor de retorno consiste em dois elementos: o primeiro é uma lista de pares `(option, value)`; a segunda é a lista de argumentos de programa restantes depois que a lista de opções foi removida (esta é uma fatia ao final de *args*). Cada par de opção e valor retornado tem a opção como seu primeiro elemento, prefixado com um hífen para opções curtas (por exemplo, `'-x'`) ou dois hifenes para opções longas (por exemplo, `'--long-option'`), e o argumento da opção como seu segundo elemento, ou uma string vazia se a opção não tiver argumento. As opções ocorrem na lista na mesma ordem em que foram encontradas, permitindo assim múltiplas ocorrências. Opções longas e curtas podem ser misturadas.

Alterado na versão 3.14: Argumentos opcionais são suportados.

getopt.**gnu_getopt**(*args*, *shortopts*, *longopts=[]*)

Esta função funciona como *getopt()*, exceto que o modo de digitalização do estilo GNU é usado por padrão. Isso significa que os argumentos de opção e não opção podem ser misturados. A função *getopt()* interrompe o processamento das opções assim que um argumento não opcional é encontrado.

Se o primeiro caractere da string de opção for `'+'`, ou se a variável de ambiente `POSIXLY_CORRECT` estiver definida, então o processamento da opção para assim que um argumento não opcional for encontrado.

Se o primeiro caractere da string de opção for `'-'`, argumentos não opcionais seguidos por opções serão adicionados à lista de pares opção-valor como um par que tem `None` como primeiro elemento e a lista de argumentos não opcionais como segundo elemento. O segundo elemento do resultado `gnu_getopt()` é uma lista de argumentos do programa após a última opção.

Alterado na versão 3.14: Suporte para retornar opções misturadas e argumentos não opcionais em ordem.

exception getopt.**GetoptError**

Isso é levantado quando uma opção não reconhecida é encontrada na lista de argumentos ou quando uma opção que requer um argumento não é fornecida. O argumento para a exceção é uma string que indica a causa do erro. Para opções longas, um argumento dado a uma opção que não requer uma também fará com que essa exceção seja levantada. Os atributos `msg` e `opt` fornecem a mensagem de erro e a opção relacionada; se não houver uma opção específica à qual a exceção se relaciona, `opt` é uma string vazia.

exception getopt.**error**

Apelido para *GetoptError*; para compatibilidade reversa.

Um exemplo usando apenas opções de estilo Unix:

```
>>> import getopt
>>> args = '-a -b -cfoo -d bar a1 a2'.split()
>>> args
['-a', '-b', '-cfoo', '-d', 'bar', 'a1', 'a2']
>>> optlist, args = getopt.getopt(args, 'abc:d:')
>>> optlist
[('-a', ''), ('-b', ''), ('-c', 'foo'), ('-d', 'bar')]
>>> args
['a1', 'a2']
```

Usar nomes de opções longos é igualmente fácil:

```
>>> s = '--condition=foo --testing --output-file abc.def -x a1 a2'
>>> args = s.split()
>>> args
['--condition=foo', '--testing', '--output-file', 'abc.def', '-x', 'a1', 'a2']
>>> optlist, args = getopt.getopt(args, 'x', [
...     'condition=', 'output-file=', 'testing'])
>>> optlist
[('--condition', 'foo'), ('--testing', ''), ('--output-file', 'abc.def'), ('-x', '
↪')]
>>> args
['a1', 'a2']
```

Argumentos opcionais devem ser especificados explicitamente:

```
>>> s = '-Con -C --color=off --color a1 a2'
>>> args = s.split()
>>> args
['-Con', '-C', '--color=off', '--color', 'a1', 'a2']
>>> optlist, args = getopt.getopt(args, 'C::', ['color=?'])
>>> optlist
[('-C', 'on'), ('-C', ''), ('--color', 'off'), ('--color', '')]
>>> args
['a1', 'a2']
```

A ordem dos argumentos opcionais e não opcionais pode ser preservada:

```
>>> s = 'a1 -x a2 a3 a4 --long a5 a6'
>>> args = s.split()
>>> args
['a1', '-x', 'a2', 'a3', 'a4', '--long', 'a5', 'a6']
>>> optlist, args = getopt.gnu_getopt(args, '-x:', ['long='])
>>> optlist
[(None, ['a1']), ('-x', 'a2'), (None, ['a3', 'a4']), ('--long', 'a5')]
>>> args
['a6']
```

Em um script, o uso típico é algo assim:

```
import getopt, sys

def main():
    try:
        opts, args = getopt.getopt(sys.argv[1:], "ho:v", ["help", "output="])
    except getopt.GetoptError as err:
        # print help information and exit:
        print(err)  # will print something like "option -a not recognized"
```

(continua na próxima página)

(continuação da página anterior)

```
        usage()
        sys.exit(2)
    output = None
    verbose = False
    for o, a in opts:
        if o == "-v":
            verbose = True
        elif o in ("-h", "--help"):
            usage()
            sys.exit()
        elif o in ("-o", "--output"):
            output = a
        else:
            assert False, "unhandled option"
    process(args, output=output, verbose=verbose)

if __name__ == "__main__":
    main()
```

Observe que uma interface de linha de comando equivalente pode ser produzida com menos código e mais mensagens de erro de ajuda e erro informativas usando o módulo *optparse*:

```
import optparse

if __name__ == '__main__':
    parser = optparse.OptionParser()
    parser.add_option('-o', '--output')
    parser.add_option('-v', dest='verbose', action='store_true')
    opts, args = parser.parse_args()
    process(args, output=opts.output, verbose=opts.verbose)
```

Uma interface de linha de comando aproximadamente equivalente para este caso também pode ser produzida usando o módulo *argparse*:

```
import argparse

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('-o', '--output')
    parser.add_argument('-v', dest='verbose', action='store_true')
    parser.add_argument('rest', nargs='*')
    args = parser.parse_args()
    process(args.rest, output=args.output, verbose=args.verbose)
```

Veja *Escolhendo uma biblioteca de análise de linha de comando* para detalhes sobre como a versão `argparse` deste código difere em comportamento da versão `optparse` (e `getopt`).

Ver também

Módulo *optparse*
Análise de opções de linha de comando declarativa.

Módulo *argparse*
Opção de linha de comando e biblioteca de análise de argumentos mais opinativa.

CAPÍTULO 37

Módulos removidos

Os módulos descritos neste capítulo foram removidos da biblioteca padrão do Python. Eles estão documentados aqui para ajudar as pessoas a encontrarem substituições.

37.1 `aifc` — Lê e escreve arquivos AIFF e AIFC

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `aifc` foi o Python 3.12.

37.2 `asynchat` — Manipulador de comando/resposta de socket assíncrono

Deprecated since version 3.6, removed in version 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.6. A remoção foi decidida na **PEP 594**.

As aplicações devem usar o módulo *`asyncio`* em vez disso.

A última versão do Python que forneceu o módulo `asynchat` foi o Python 3.11.

37.3 `asyncore` — Manipulador de socket assíncrono

Deprecated since version 3.6, removed in version 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.6. A remoção foi decidida na **PEP 594**.

As aplicações devem usar o módulo *`asyncio`* em vez disso.

A última versão do Python que forneceu o módulo `asyncore` foi o Python 3.11.

37.4 audioop — Manipulando dados de áudio original

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `audioop` foi o Python 3.12.

37.5 cgi — Suporte a Common Gateway Interface

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

Um fork do módulo no PyPI pode ser usado em vez disso: legacy-cgi. Esta é uma cópia do módulo cgi, não mais mantido ou suportado pela equipe core do Python.

A última versão do Python que forneceu o módulo `cgi` foi o Python 3.12.

37.6 cgitb — Gerenciador de traceback para scripts de CGI

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

Um fork do módulo no PyPI agora pode ser usado em vez disso: legacy-cgi. Esta é uma cópia do módulo cgi, não mais mantido ou suportado pela equipe core do Python.

A última versão do Python que forneceu o módulo `cgitb` foi o Python 3.12.

37.7 chunk — Lê dados em blocos IFF

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `chunk` foi o Python 3.12.

37.8 crypt — Função para verificar senhas Unix

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

As aplicações podem usar o módulo *`hashlib`* da biblioteca padrão. Outras possíveis substituições são bibliotecas de terceiros do PyPI: legacycrypt, bcrypt, argon2-cffi ou passlib. Elas não são suportadas ou mantidas pela equipe core do Python.

A última versão do Python que forneceu o módulo `crypt` foi o Python 3.12.

37.9 distutils — Criação e instalação de módulos do Python

Deprecated since version 3.10, removed in version 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.10. A remoção foi decidida na **PEP 632**, a qual tem um conselho de migração.

A última versão do Python que forneceu o módulo `distutils` foi o Python 3.11.

37.10 `imghdr` — Determina o tipo de uma imagem

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

Possíveis substituições são bibliotecas de terceiros do PyPI: filetype, puremagic ou python-magic. Eles não são suportados ou mantidos pela equipe principal do Python.

A última versão do Python que forneceu o módulo `imghdr` foi o Python 3.12.

37.11 `imp` — Acessa os componentes internos de importação

Deprecated since version 3.4, removed in version 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.4.

O aviso de remoção inclui orientação para migrar código de `imp` para *`importlib`*.

A última versão do Python que forneceu o módulo `imp` foi o Python 3.11.

37.12 `mailcap` — Manipulação de arquivo Mailcap

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `mailcap` foi o Python 3.12.

37.13 `msilib` — Lê e escreve arquivos do Microsoft Installer

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `msilib` foi o Python 3.12.

37.14 `nis` — Interface para NIS da Sun (Yellow Pages)

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `nis` foi o Python 3.12.

37.15 `nntplib` — Cliente de protocolo NNTP

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `nntplib` foi o Python 3.12.

37.16 ossaudiodev — Acesso a dispositivos de áudio compatíveis com OSS

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `ossaudiodev` foi o Python 3.12.

37.17 pipes — Interface para pipelines do shell

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

As aplicações devem usar o módulo *subprocess* em vez disso.

A última versão do Python que forneceu o módulo `pipes` foi o Python 3.12.

37.18 smtpd — Serviços SMTP

Deprecated since version 3.6, removed in version 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.6. A remoção foi decidida na **PEP 594**.

Uma possível substituição é a biblioteca de terceiros aiosmtpd. Esta biblioteca não é mantida ou suportada pela equipe principal do Python.

A última versão do Python que forneceu o módulo `smtpd` foi o Python 3.11.

37.19 sndhdr — Determina o tipo de arquivos de som

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

Possíveis substituições são módulos de terceiros do PyPI: filetype, puremagic ou python-magic. Eles não são suportados ou mantidos pela equipe principal do Python.

A última versão do Python que forneceu o módulo `sndhdr` foi o Python 3.12.

37.20 spwd — O banco de dados de senhas shadow

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

Uma possível substituição é a biblioteca de terceiros python-pam. Esta biblioteca não é suportada ou mantida pela equipe principal do Python.

A última versão do Python que forneceu o módulo `spwd` foi o Python 3.12.

37.21 `sunau` — Lê e escreve arquivos AU da Sun

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `sunau` foi o Python 3.12.

37.22 `telnetlib` — cliente Telnet

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

Possíveis substituições são bibliotecas de terceiros do PyPI: telnetlib3 ou Exscript. Elas não são suportadas ou mantidas pela equipe principal do Python.

A última versão do Python que forneceu o módulo `telnetlib` foi o Python 3.12.

37.23 `uu` — Codifica e decodifica arquivos uuencode

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `uu` foi o Python 3.12.

37.24 `xdrlib` — Codifica e decodifica dados XDR

Deprecated since version 3.11, removed in version 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594**.

A última versão do Python que forneceu o módulo `xdrlib` foi o Python 3.12.

CAPÍTULO 38

Considerações de segurança

Os módulos a seguir têm considerações de segurança específicas:

- `base64`: *considerações de segurança do base64* na **RFC 4648**
- `hashlib`: *todos os construtores usam um argumento somente-nomeado "usedforsecurity", desativando algoritmos conhecidos não seguros e bloqueados*
- `http.server` não é adequado para uso em produção, implementando apenas verificações básicas de segurança. Veja as *considerações de segurança*.
- `logging`: *configuração do Logging usa eval()*
- `multiprocessing`: *Connection.recv() usa pickle*
- `pickle`: *Restringindo globais no pickle*
- `random` não deve ser usado para o propósito de segurança. Em vez disso, use `secrets`
- `shelve`: *shelve é baseado no pickle e, portanto, inadequado para lidar com fontes não confiáveis*
- `ssl`: *Considerações de segurança de SSL/TLS*
- `subprocess`: *Considerações de segurança de Subprocess*
- `tempfile`: *mktemp foi descontinuado em razão de vulnerabilidade de condições de corrida*
- `xml`: *Segurança no XML*
- `zipfile`: *Arquivos .zip preparados de forma maliciosa podem causar esgotamento do volume do disco*

A opção de linha de comando `-I` pode ser usada para executar o Python no modo isolado. Quando não pode ser usado, a opção `-P` ou a variável de ambiente `PYTHONSAFEPATH` pode ser usada para não preceder um caminho potencialmente inseguro para `sys.path` como o diretório atual, o diretório do script ou uma string vazia.

APÊNDICE A

Glossário

>>>
: O prompt padrão do console *interativo* do Python. Normalmente visto em exemplos de código que podem ser executados interativamente no interpretador.

...
: Pode se referir a:

  - O prompt padrão do console *interativo* do Python ao inserir o código para um bloco de código recuado, quando dentro de um par de delimitadores correspondentes esquerdo e direito (parênteses, colchetes, chaves ou aspas triplas) ou após especificar um decorador.
  - Os três pontos formam o objeto *Ellipsis* (reticências).

classe base abstrata
: Classes bases abstratas (ABCs, do inglês *abstract base class*) complementam a *tipagem pato*, fornecendo uma maneira de definir interfaces quando outras técnicas, como `hasattr()`, seriam desajeitadas ou sutilmente erradas (por exemplo, com métodos mágicos). ABCs introduzem subclasses virtuais, classes que não herdam de uma classe mas ainda são reconhecidas por `isinstance()` e `issubclass()`; veja a documentação do módulo `abc`. Python vem com muitas ABCs embutidas para estruturas de dados (no módulo `collections.abc`), números (no módulo `numbers`), fluxos (no módulo `io`), localizadores e carregadores de importação (no módulo `importlib.abc`). Você pode criar suas próprias ABCs com o módulo `abc`.

função de anotação
: Uma função que pode ser chamada para recuperar as *anotações* de um objeto. Esta função é acessível como o atributo `__annotate__` de funções, classes e módulos. Funções de anotação são um subconjunto de *funções de avaliação*.

anotação
: Um rótulo associado a uma variável, um atributo de classe ou um parâmetro de função ou valor de retorno, usado por convenção como *dica de tipo*.

  Anotações de variáveis locais não podem ser acessadas em tempo de execução, mas anotações de variáveis globais, atributos de classe e funções podem ser recuperadas chamando `annotationlib.get_annotations()` em módulos, classes e funções, respectivamente.

  Consulte *anotação de variável*, *anotação de função*, **PEP 484**, **PEP 526** e **PEP 649**, que descrevem essa funcionalidade. Consulte também annotations-howto para obter as melhores práticas sobre como trabalhar com anotações.

argumento
: Um valor passado para uma *função* (ou *método*) ao chamar a função. Existem dois tipos de argumento:

- *argumento nomeado*: um argumento precedido por um identificador (por exemplo, `name=`) na chamada de uma função ou passado como um valor em um dicionário precedido por `**`. Por exemplo, `3` e `5` são ambos argumentos nomeados nas chamadas da função *`complex()`* a seguir:

```
complex(real=3, imag=5)
complex(**{'real': 3, 'imag': 5})
```

- *argumento posicional*: um argumento que não é um argumento nomeado. Argumentos posicionais podem aparecer no início da lista de argumentos e/ou podem ser passados como elementos de um *iterável* precedidos por `*`. Por exemplo, `3` e `5` são ambos argumentos posicionais nas chamadas a seguir:

```
complex(3, 5)
complex(*(3, 5))
```

Argumentos são atribuídos às variáveis locais nomeadas no corpo da função. Veja a seção calls para as regras de atribuição. Sintaticamente, qualquer expressão pode ser usada para representar um argumento; após a expressão ser avaliada, o valor resultante é atribuído à variável local.

Veja também o termo *parâmetro* no glossário, a pergunta no FAQ sobre a diferença entre argumentos e parâmetros e **PEP 362**.

gerenciador de contexto assíncrono
Um objeto que controla o ambiente envolto numa instrução `async with` por meio da definição dos métodos `__aenter__()` e `__aexit__()`. Introduzido pela **PEP 492**.

gerador assíncrono
Uma função que retorna um *iterador gerador assíncrono*. É parecida com uma função de corrotina definida com `async def` exceto pelo fato de conter instruções `yield` para produzir uma série de valores que podem ser usados em um laço `async for`.

Normalmente se refere a uma função geradora assíncrona, mas pode se referir a um *iterador gerador assíncrono* em alguns contextos. Em casos em que o significado não esteja claro, usar o termo completo evita a ambiguidade.

Uma função geradora assíncrona pode conter expressões `await` e também as instruções `async for` e `async with`.

iterador gerador assíncrono
Um objeto criado por uma função *geradora assíncrona*.

Este é um *iterador assíncrono* que, quando chamado usando o método `__anext__()`, retorna um objeto aguardável que executará o corpo da função geradora assíncrona até a próxima expressão `yield`.

Cada `yield` suspende temporariamente o processamento, lembrando o estado de execução (incluindo variáveis locais e instruções `try` pendentes). Quando o *iterador gerador assíncrono* é efetivamente retomado com outro aguardável retornado por `__anext__()`, ele inicia de onde parou. Veja **PEP 492** e **PEP 525**.

iterável assíncrono
Um objeto que pode ser usado em uma instrução `async for`. Deve retornar um *iterador assíncrono* do seu método `__aiter__()`. Introduzido por **PEP 492**.

iterador assíncrono
Um objeto que implementa os métodos `__aiter__()` e `__anext__()`. `__anext__()` deve retornar um objeto *aguardável*. `async for` resolve os aguardáveis retornados por um método `__anext__()` do iterador assíncrono até que ele levante uma exceção *`StopAsyncIteration`*. Introduzido pela **PEP 492**.

estado de thread anexado
Um *estado de thread* que está ativo para a thread atual do sistema operacional.

Quando um *estado de thread* é anexado, a thread do sistema operacional tem acesso à API C completa do Python e pode invocar com segurança o interpretador de bytecode.

A menos que uma função indique explicitamente o contrário, tentar chamar a API C sem um estado de thread anexado resultará em um erro fatal ou comportamento indefinido. Um estado de thread pode ser anexado e

desanexado explicitamente pelo usuário por meio da API C, ou implicitamente pelo ambiente de execução, inclusive durante o bloqueio de chamadas C e pelo interpretador de bytecode entre as chamadas.

Na maioria das construções do Python, ter um estado de thread anexado implica que o chamador mantém a *GIL* para o interpretador atual, portanto, apenas uma thread do SO pode ter um estado de thread anexado em um determinado momento. Em construções do Python com *threads livres*, as threads podem manter simultaneamente um estado de thread anexado, permitindo um verdadeiro paralelismo do interpretador de bytecode.

atributo
Um valor associado a um objeto que é geralmente referenciado pelo nome separado por um ponto. Por exemplo, se um objeto *o* tem um atributo *a* esse seria referenciado como *o.a*.

É possível dar a um objeto um atributo cujo nome não seja um identificador conforme definido por identifiers, por exemplo usando `setattr()`, se o objeto permitir. Tal atributo não será acessível usando uma expressão pontilhada e, em vez disso, precisaria ser recuperado com `getattr()`.

aguardável
Um objeto que pode ser usado em uma expressão `await`. Pode ser uma *corrotina* ou um objeto com um método `__await__()`. Veja também a **PEP 492**.

BDFL
Abreviação da expressão da língua inglesa "Benevolent Dictator for Life" (em português, "Ditador Benevolente Vitalício"), referindo-se a Guido van Rossum, criador do Python.

arquivo binário
Um *objeto arquivo* capaz de ler e gravar em *objetos bytes ou similares*. Exemplos de arquivos binários são arquivos abertos no modo binário (`'rb'`, `'wb'` ou `'rb+'`), `sys.stdin.buffer`, `sys.stdout.buffer`, e instâncias de `io.BytesIO` e `gzip.GzipFile`.

Veja também *arquivo texto* para um objeto arquivo capaz de ler e gravar em objetos `str`.

referência emprestada
Na API C do Python, uma referência emprestada é uma referência a um objeto que não é dona da referência. Ela se torna um ponteiro solto se o objeto for destruído. Por exemplo, uma coleta de lixo pode remover a última *referência forte* para o objeto e assim destruí-lo.

Chamar `Py_INCREF()` na *referência emprestada* é recomendado para convertê-lo, internamente, em uma *referência forte*, exceto quando o objeto não pode ser destruído antes do último uso da referência emprestada. A função `Py_NewRef()` pode ser usada para criar uma nova *referência forte*.

objeto bytes ou similar
Um objeto com suporte ao bufferobjects e que pode exportar um buffer C *contíguo*. Isso inclui todos os objetos `bytes`, `bytearray` e `array.array`, além de muitos objetos `memoryview` comuns. Objetos bytes ou similares podem ser usados para várias operações que funcionam com dados binários; isso inclui compactação, salvamento em um arquivo binário e envio por um soquete.

Algumas operações precisam que os dados binários sejam mutáveis. A documentação geralmente se refere a eles como "objetos bytes ou similares para leitura e escrita". Exemplos de objetos de buffer mutável incluem `bytearray` e um `memoryview` de um `bytearray`. Outras operações exigem que os dados binários sejam armazenados em objetos imutáveis ("objetos bytes ou similares para somente leitura"); exemplos disso incluem `bytes` e a `memoryview` de um objeto `bytes`.

bytecode
O código-fonte Python é compilado para bytecode, a representação interna de um programa em Python no interpretador CPython. O bytecode também é mantido em cache em arquivos `.pyc` e `.pyo`, de forma que executar um mesmo arquivo é mais rápido na segunda vez (a recompilação dos fontes para bytecode não é necessária). Esta "linguagem intermediária" é adequada para execução em uma *máquina virtual*, que executa o código de máquina correspondente para cada bytecode. Tenha em mente que não se espera que bytecodes sejam executados entre máquinas virtuais Python diferentes, nem que se mantenham estáveis entre versões de Python.

Uma lista de instruções bytecode pode ser encontrada na documentação para *o módulo dis*.

chamável

Um chamável é um objeto que pode ser chamado, possivelmente com um conjunto de argumentos (veja *argumento*), com a seguinte sintaxe:

```
chamavel(argumento1, argumento2, argumentoN)
```

Uma *função*, e por extensão um *método*, é um chamável. Uma instância de uma classe que implementa o método `__call__()` também é um chamável.

função de retorno
Também conhecida como callback, é uma função sub-rotina que é passada como um argumento a ser executado em algum ponto no futuro.

classe
Um modelo para criação de objetos definidos pelo usuário. Definições de classe normalmente contém definições de métodos que operam sobre instâncias da classe.

variável de classe
Uma variável definida em uma classe e destinada a ser modificada apenas no nível da classe (ou seja, não em uma instância da classe).

variável de clausura
Uma *variável livre* referenciada de um *escopo aninhado* que é definida em um escopo externo em vez de ser resolvida em tempo de execução a partir dos espaços de nomes embutido ou globais. Pode ser explicitamente definida com a palavra reservada `nonlocal` para permitir acesso de gravação, ou implicitamente definida se a variável estiver sendo somente lida.

Por exemplo, na função `interna` no código a seguir, tanto `x` quanto `print` são *variáveis livres*, mas somente `x` é uma *variável de clausura*:

```
def externa():
    x = 0
    def interna():
        nonlocal x
        x += 1
        print(x)
    return interna
```

Devido ao atributo `codeobject.co_freevars` (que, apesar do nome, inclui apenas os nomes das variáveis de clausura em vez de listar todas as variáveis livres referenciadas), o termo mais geral *variável livre* às vezes é usado mesmo quando o significado pretendido é se referir especificamente às variáveis de clausura.

número complexo
Uma extensão ao familiar sistema de números reais em que todos os números são expressos como uma soma de uma parte real e uma parte imaginária. Números imaginários são múltiplos reais da unidade imaginária (a raiz quadrada de `-1`), normalmente escrita como `i` em matemática ou `j` em engenharia. O Python tem suporte nativo para números complexos, que são escritos com esta última notação; a parte imaginária escrita com um sufixo `j`, p.ex., `3+1j`. Para ter acesso aos equivalentes para números complexos do módulo *`math`*, utilize *`cmath`*. O uso de números complexos é uma funcionalidade matemática bastante avançada. Se você não sabe se irá precisar deles, é quase certo que você pode ignorá-los sem problemas.

contexto
Este termo tem diferentes significados dependendo de onde e como ele é usado. Alguns significados comuns:

- O estado ou ambiente temporário estabelecido por um *gerenciador de contexto* por meio de uma instrução `with`.
- A coleção de ligações de chave-valor associadas a um objeto *`contextvars.Context`* específico e acessadas por meio de objetos *`ContextVar`*. Veja também *variável de contexto*.
- Um objeto *`contextvars.Context`*. Veja também *contexto atual*.

protocolo de gerenciamento de contexto
Os métodos `__enter__()` e `__exit__()` chamados pela instrução `with`. Veja **PEP 343**.

gerenciador de contexto
Um objeto que implementa o *protocolo de gerenciamento de contexto* e controla o ambiente envolto em uma instrução `with`. Veja **PEP 343**.

variável de contexto
Uma variável cujo valor depende de qual contexto é o *contexto atual*. Os valores são acessados por meio de objetos `contextvars.ContextVar`. Variáveis de contexto são usadas principalmente para isolar o estado entre tarefas assíncronas simultâneas.

contíguo
Um buffer é considerado contíguo exatamente se for *contíguo C* ou *contíguo Fortran*. Os buffers de dimensão zero são contíguos C e Fortran. Em vetores unidimensionais, os itens devem ser dispostos na memória próximos um do outro, em ordem crescente de índices, começando do zero. Em vetores multidimensionais contíguos C, o último índice varia mais rapidamente ao visitar itens em ordem de endereço de memória. No entanto, nos vetores contíguos do Fortran, o primeiro índice varia mais rapidamente.

corrotina
Corrotinas são uma forma mais generalizada de sub-rotinas. Sub-rotinas tem a entrada iniciada em um ponto, e a saída em outro ponto. Corrotinas podem entrar, sair, e continuar em muitos pontos diferentes. Elas podem ser implementadas com a instrução `async def`. Veja também **PEP 492**.

função de corrotina
Uma função que retorna um objeto do tipo *corrotina*. Uma função de corrotina pode ser definida com a instrução `async def`, e pode conter as palavras chaves `await`, `async for`, e `async with`. Isso foi introduzido pela **PEP 492**.

CPython
A implementação canônica da linguagem de programação Python, como disponibilizada pelo python.org. O termo "CPython" é usado quando necessário distinguir esta implementação de outras como Jython ou IronPython.

contexto atual
O *contexto* (objeto `contextvars.Context`) que é usado atualmente pelos objetos `ContextVar` para acessar (obter ou definir) os valores de *variáveis de contexto*. Cada thread tem seu próprio contexto atual. Frameworks para executar tarefas assíncronas (veja `asyncio`) associam cada tarefa a um contexto que se torna o contexto atual sempre que a tarefa inicia ou retoma a execução.

isolado cíclico
Um subgrupo de um ou mais objetos que se referenciam em um ciclo de referência, mas não são referenciados por objetos de fora do grupo. O objetivo do *coletor de lixo cíclico* é identificar esses grupos e interromper os ciclos de referência para que a memória possa ser recuperada.

decorador
Uma função que retorna outra função, geralmente aplicada como uma transformação de função usando a sintaxe `@wrapper`. Exemplos comuns para decoradores são `classmethod()` e `staticmethod()`.

A sintaxe do decorador é meramente um açúcar sintático, as duas definições de funções a seguir são semanticamente equivalentes:

```
def f(arg):
    ...
f = staticmethod(f)

@staticmethod
def f(arg):
    ...
```

O mesmo conceito existe para as classes, mas não é comumente utilizado. Veja a documentação de definições de função e definições de classe para obter mais informações sobre decoradores.

descritor
Qualquer objeto que define os métodos `__get__()`, `__set__()` ou `__delete__()`. Quando um atributo de classe é um descritor, seu comportamento de associação especial é acionado no acesso a um atributo.

Normalmente, ao se utilizar *a.b* para se obter, definir ou excluir, um atributo dispara uma busca no objeto chamado *b* no dicionário de classe de *a*, mas se *b* for um descritor, o respectivo método descritor é chamado. Compreender descritores é a chave para um profundo entendimento de Python pois eles são a base de muitas funcionalidades incluindo funções, métodos, propriedades, métodos de classe, métodos estáticos e referências para superclasses.

Para obter mais informações sobre os métodos dos descritores, veja: descriptors ou o Guia de Descritores.

dicionário
Um vetor associativo em que chaves arbitrárias são mapeadas para valores. As chaves podem ser quaisquer objetos que possuam os métodos `__hash__()` e `__eq__()`. Isso é chamado de hash em Perl.

compreensão de dicionário
Uma maneira compacta de processar todos ou parte dos elementos de um iterável e retornar um dicionário com os resultados. `results = {n: n ** 2 for n in range(10)}` gera um dicionário contendo a chave `n` mapeada para o valor `n ** 2`. Veja comprehensions.

visão de dicionário
Os objetos retornados por *`dict.keys()`*, *`dict.values()`* e *`dict.items()`* são chamados de visões de dicionário. Eles fornecem uma visão dinâmica das entradas do dicionário, o que significa que quando o dicionário é alterado, a visão reflete essas alterações. Para forçar a visão de dicionário a se tornar uma lista completa use `list(dictview)`. Veja *Objetos de visão de dicionário*.

docstring
Abreviatura de "documentation string" (string de documentação). Uma string literal que aparece como primeira expressão numa classe, função ou módulo. Ainda que sejam ignoradas quando a suíte é executada, é reconhecida pelo compilador que a coloca no atributo *`__doc__`* da classe, função ou módulo que a encapsula. Como ficam disponíveis por meio de introspecção, docstrings são o lugar canônico para documentação do objeto.

tipagem pato
Também conhecida como *duck-typing*, é um estilo de programação que não verifica o tipo do objeto para determinar se ele possui a interface correta; em vez disso, o método ou atributo é simplesmente chamado ou utilizado ("Se se parece com um pato e grasna como um pato, então deve ser um pato.") Enfatizando interfaces ao invés de tipos específicos, o código bem desenvolvido aprimora sua flexibilidade por permitir substituição polimórfica. Tipagem pato evita necessidade de testes que usem *`type()`* ou *`isinstance()`*. (Note, porém, que a tipagem pato pode ser complementada com o uso de *classes base abstratas*.) Ao invés disso, são normalmente empregados testes *`hasattr()`* ou programação *EAFP*.

dunder
Uma abreviação informal para "double underscore" (em português, sublinhado duplo), usada ao se referir a um *método especial*. Por exemplo, `__init__` é frequentemente pronunciado como "dunder init".

EAFP
Iniciais da expressão em inglês "easier to ask for forgiveness than permission" que significa "é mais fácil pedir perdão que permissão". Este estilo de codificação comum no Python presume a existência de chaves ou atributos válidos e captura exceções caso essa premissa se prove falsa. Este estilo limpo e rápido se caracteriza pela presença de várias instruções `try` e `except`. A técnica diverge do estilo *LBYL*, comum em outras linguagens como C, por exemplo.

função de avaliação
Uma função que pode ser chamada para avaliar um atributo com avaliação preguiçosa de um objeto, como o valor de apelidos de tipo criados com a instrução `type`.

expressão
Uma parte da sintaxe que pode ser avaliada para algum valor. Em outras palavras, uma expressão é a acumulação de elementos de expressão como literais, nomes, atributos de acesso, operadores ou chamadas de funções, todos os quais retornam um valor. Em contraste com muitas outras linguagens, nem todas as construções de linguagem são expressões. Também existem *instruções*, as quais não podem ser usadas como expressões, como, por exemplo, `while`. Atribuições também são instruções, não expressões.

módulo de extensão
Um módulo escrito em C ou C++, usando a API C do Python para interagir tanto com código de usuário quanto do núcleo.

f-string
f-strings
Literais de string prefixados com `f` ou `F` são comumente chamados de "f-strings", abreviação em inglês de literais de strings formatadas. Veja também **PEP 498**.

objeto arquivo
Um objeto que expõe uma API orientada a arquivos (com métodos tais como `read()` ou `write()`) para um recurso subjacente. Dependendo da maneira como foi criado, um objeto arquivo pode mediar o acesso a um arquivo real no disco ou outro tipo de dispositivo de armazenamento ou de comunicação (por exemplo a entrada/saída padrão, buffers em memória, soquetes, pipes, etc.). Objetos arquivo também são chamados de *objetos arquivo ou similares* ou *fluxos*.

Atualmente há três categorias de objetos arquivo: *arquivos binários* brutos, *arquivos binários* em buffer e *arquivos textos*. Suas interfaces estão definidas no módulo `io`. A forma canônica para criar um objeto arquivo é usando a função `open()`.

objeto arquivo ou similar
Um sinônimo do termo *objeto arquivo*.

tratador de erros e codificação do sistema de arquivos
Tratador de erros e codificação usado pelo Python para decodificar bytes do sistema operacional e codificar Unicode para o sistema operacional.

A codificação do sistema de arquivos deve garantir a decodificação bem-sucedida de todos os bytes abaixo de 128. Se a codificação do sistema de arquivos falhar em fornecer essa garantia, as funções da API podem levantar `UnicodeError`.

As funções `sys.getfilesystemencoding()` e `sys.getfilesystemencodeerrors()` podem ser usadas para obter o tratador de erros e codificação do sistema de arquivos.

O *tratador de erros e codificação do sistema de arquivos* são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Veja também *codificação da localidade*.

localizador
Um objeto que tenta encontrar o *carregador* para um módulo que está sendo importado.

Existem dois tipos de localizador: *localizadores de metacaminho* para uso com `sys.meta_path`, e *localizadores de entrada de caminho* para uso com `sys.path_hooks`.

Veja finders-and-loaders e `importlib` para muito mais detalhes.

divisão pelo piso
Divisão matemática que arredonda para baixo para o inteiro mais próximo. O operador de divisão pelo piso é `//`. Por exemplo, a expressão `11 // 4` retorna o valor `2` ao invés de `2.75`, que seria retornado pela divisão de ponto flutuante. Note que `(-11) // 4` é `-3` porque é `-2.75` arredondado *para baixo*. Consulte a **PEP 238**.

threads livres
Um modelo de threads onde múltiplas threads podem simultaneamente executar bytecode Python no mesmo interpretador. Isso está em contraste com a *trava global do interpretador* que permite apenas uma thread por vez executar bytecode Python. Veja **PEP 703**.

variável livre
Formalmente, conforme definido no modelo de execução de linguagem, uma variável livre é qualquer variável usada em um espaço de nomes que não seja uma variável local naquele espaço de nomes. Veja *variável de clausura* para um exemplo. Pragmaticamente, devido ao nome do atributo `codeobject.co_freevars`, o termo também é usado algumas vezes como sinônimo de *variável de clausura*.

função
Uma série de instruções que retorna algum valor para um chamador. Também pode ser passado zero ou mais *argumentos* que podem ser usados na execução do corpo. Veja também *parâmetro*, *método* e a seção function.

anotação de função
Uma *anotação* de um parâmetro de função ou valor de retorno.

Anotações de função são comumente usados por *dicas de tipo*: por exemplo, essa função espera receber dois argumentos `int` e também é esperado que devolva um valor `int`:

```
def soma_dois_numeros(a: int, b: int) -> int:
   return a + b
```

A sintaxe de anotação de função é explicada na seção function.

Veja *anotação de variável* e **PEP 484**, que descrevem esta funcionalidade. Veja também annotations-howto para as melhores práticas sobre como trabalhar com anotações.

__future__
A instrução future, `from __future__ import <feature>`, direciona o compilador a compilar o módulo atual usando sintaxe ou semântica que será padrão em uma versão futura de Python. O módulo `__future__` documenta os possíveis valores de *feature*. Importando esse módulo e avaliando suas variáveis, você pode ver quando um novo recurso foi inicialmente adicionado à linguagem e quando será (ou se já é) o padrão:

```
>>> import __future__
>>> __future__.division
_Feature((2, 2, 0, 'alpha', 2), (3, 0, 0, 'alpha', 0), 8192)
```

coleta de lixo
Também conhecido como *garbage collection*, é o processo de liberar a memória quando ela não é mais utilizada. Python executa a liberação da memória através da contagem de referências e um coletor de lixo cíclico que é capaz de detectar e interromper referências cíclicas. O coletor de lixo pode ser controlado usando o módulo `gc`.

gerador
Uma função que retorna um *iterador gerador*. É parecida com uma função normal, exceto pelo fato de conter expressões `yield` para produzir uma série de valores que podem ser usados em um laço "for" ou que podem ser obtidos um de cada vez com a função `next()`.

Normalmente refere-se a uma função geradora, mas pode referir-se a um *iterador gerador* em alguns contextos. Em alguns casos onde o significado desejado não está claro, usar o termo completo evita ambiguidade.

iterador gerador
Um objeto criado por uma função *geradora*.

Cada `yield` suspende temporariamente o processamento, memorizando o estado da execução (incluindo variáveis locais e instruções try pendentes). Quando o *iterador gerador* retorna, ele se recupera do último ponto onde estava (em contrapartida as funções que iniciam uma nova execução a cada vez que são invocadas).

expressão geradora
Uma *expressão* que retorna um *iterador*. Parece uma expressão normal, seguido de uma cláusula `for` definindo uma variável de laço, um intervalo, e uma cláusula `if` opcional. A expressão combinada gera valores para uma função encapsuladora:

```
>>> sum(i*i for i in range(10))         # soma dos quadrados 0, 1, 4, ... 81
285
```

função genérica
Uma função composta por várias funções implementando a mesma operação para diferentes tipos. Qual implementação deverá ser usada durante a execução é determinada pelo algoritmo de despacho.

Veja também a entrada *despacho único* no glossário, o decorador `functools.singledispatch()`, e a **PEP 443**.

tipo genérico
Um *tipo* que pode ser parametrizado; tipicamente uma classe contêiner tal como `list` ou `dict`. Usado para *dicas de tipo* e *anotações*.

Para mais detalhes, veja *tipo apelido genérico*, **PEP 483**, **PEP 484**, **PEP 585**, e o módulo `typing`.

GIL
Veja *trava global do interpretador*.

trava global do interpretador
O mecanismo utilizado pelo interpretador *CPython* para garantir que apenas uma thread execute o *bytecode* Python por vez. Isto simplifica a implementação do CPython ao fazer com que o modelo de objetos (incluindo tipos embutidos críticos como o `dict`) ganhem segurança implícita contra acesso concorrente. Travar todo o interpretador facilita que o interpretador em si seja multithread, às custas de muito do paralelismo já provido por máquinas com multiprocessadores.

No entanto, alguns módulos de extensão, tanto da biblioteca padrão quanto de terceiros, são desenvolvidos de forma a liberar a GIL ao realizar tarefas computacionalmente muito intensas, como compactação ou cálculos de hash. Além disso, a GIL é sempre liberada nas operações de E/S.

A partir de Python 3.13, o GIL pode ser desabilitado usando a configuração de construção `--disable-gil`. Depois de construir Python com essa opção, o código deve ser executado com a opção `-X gil=0` ou a variável de ambiente `PYTHON_GIL=0` deve estar definida. Esse recurso provê um desempenho melhor para aplicações com múltiplas threads e torna mais fácil o uso eficiente de CPUs com múltiplos núcleos. Para mais detalhes, veja **PEP 703**.

Em versões anteriores da API C do Python, uma função podia declarar que precisava que a GIL fosse mantida para ser usada. Isso se refere a ter um *estado de thread anexado*.

pyc baseado em hash
Um arquivo de cache em bytecode que usa hash ao invés do tempo, no qual o arquivo de código-fonte foi modificado pela última vez, para determinar a sua validade. Veja pyc-invalidation.

hasheável
Um objeto é *hasheável* se tem um valor de hash que nunca muda durante seu ciclo de vida (precisa ter um método `__hash__()`) e pode ser comparado com outros objetos (precisa ter um método `__eq__()`). Objetos hasheáveis que são comparados como iguais devem ter o mesmo valor de hash.

A hasheabilidade faz com que um objeto possa ser usado como uma chave de dicionário e como um membro de conjunto, pois estas estruturas de dados utilizam os valores de hash internamente.

A maioria dos objetos embutidos imutáveis do Python são hasheáveis; contêineres mutáveis (tais como listas ou dicionários) não são; contêineres imutáveis (tais como tuplas e frozensets) são hasheáveis apenas se os seus elementos são hasheáveis. Objetos que são instâncias de classes definidas pelo usuário são hasheáveis por padrão. Todos eles comparam de forma desigual (exceto entre si mesmos), e o seu valor hash é derivado a partir do seu `id()`.

IDLE
Um ambiente de desenvolvimento e aprendizado integrado para Python. *IDLE — editor e console Python* é um editor básico e um ambiente interpretador que vem junto com a distribuição padrão do Python.

imortal
Objetos imortais são um detalhe da implementação do CPython introduzida na **PEP 683**.

Se um objeto é imortal, sua *contagem de referências* nunca é modificada e, portanto, nunca é desalocado enquanto o interpretador está em execução. Por exemplo, `True` e `None` são imortais no CPython.

Objetos imortais podem ser identificados via `sys._is_immortal()` ou via `PyUnstable_IsImmortal()` na API C.

imutável
Um objeto que possui um valor fixo. Objetos imutáveis incluem números, strings e tuplas. Estes objetos não podem ser alterados. Um novo objeto deve ser criado se um valor diferente tiver de ser armazenado. Objetos imutáveis têm um papel importante em lugares onde um valor constante de hash seja necessário, como por exemplo uma chave em um dicionário.

caminho de importação
Uma lista de localizações (ou *entradas de caminho*) que são buscadas pelo *localizador baseado no caminho* por módulos para importar. Durante a importação, esta lista de localizações usualmente vem a partir de `sys.path`, mas para subpacotes ela também pode vir do atributo `__path__` de pacotes-pai.

importação
O processo pelo qual o código Python em um módulo é disponibilizado para o código Python em outro módulo.

importador
Um objeto que localiza e carrega um módulo; Tanto um *localizador* e o objeto *carregador*.

interativo
Python tem um interpretador interativo, o que significa que você pode digitar instruções e expressões no prompt do interpretador, executá-los imediatamente e ver seus resultados. Apenas execute `python` sem argumentos (possivelmente selecionando-o a partir do menu de aplicações de seu sistema operacional). O interpretador interativo é uma maneira poderosa de testar novas ideias ou aprender mais sobre módulos e pacotes (lembre-se do comando `help(x)`). Para saber mais sobre modo interativo, veja tut-interac.

interpretado
Python é uma linguagem interpretada, em oposição àquelas que são compiladas, embora esta distinção possa ser nebulosa devido à presença do compilador de bytecode. Isto significa que os arquivos-fontes podem ser executados diretamente sem necessidade explícita de se criar um arquivo executável. Linguagens interpretadas normalmente têm um ciclo de desenvolvimento/depuração mais curto que as linguagens compiladas, apesar de seus programas geralmente serem executados mais lentamente. Veja também *interativo*.

desligamento do interpretador
Quando solicitado para desligar, o interpretador Python entra em uma fase especial, onde ele gradualmente libera todos os recursos alocados, tais como módulos e várias estruturas internas críticas. Ele também faz diversas chamadas para o *coletor de lixo*. Isto pode disparar a execução de código em destrutores definidos pelo usuário ou função de retorno de referência fraca. Código executado durante a fase de desligamento pode encontrar diversas exceções, pois os recursos que ele depende podem não funcionar mais (exemplos comuns são os módulos de bibliotecas, ou os mecanismos de avisos).

A principal razão para o interpretador desligar, é que o módulo `__main__` ou o script sendo executado terminou sua execução.

iterável
Um objeto capaz de retornar seus membros um de cada vez. Exemplos de iteráveis incluem todos os tipos de sequência (tais como *`list`*, *`str`* e *`tuple`*) e alguns tipos de não-sequência, como o *`dict`* e *objetos arquivo*, além dos objetos de quaisquer classes que você definir com um método `__iter__()` ou `__getitem__()` que implementam a semântica de *sequência*.

Iteráveis podem ser usados em um laço `for` e em vários outros lugares em que uma sequência é necessária (*`zip()`*, *`map()`*, ...). Quando um objeto iterável é passado como argumento para a função embutida *`iter()`*, ela retorna um iterador para o objeto. Este iterador é adequado para se varrer todo o conjunto de valores. Ao usar iteráveis, normalmente não é necessário chamar *`iter()`* ou lidar com os objetos iteradores em si. A instrução `for` faz isso automaticamente para você, criando uma variável temporária para armazenar o iterador durante a execução do laço. Veja também *iterador*, *sequência*, e *gerador*.

iterador
Um objeto que representa um fluxo de dados. Repetidas chamadas ao método *`__next__()`* de um iterador (ou passando o objeto para a função embutida *`next()`*) vão retornar itens sucessivos do fluxo. Quando não houver mais dados disponíveis uma exceção *`StopIteration`* será levantada. Neste ponto, o objeto iterador se esgotou e quaisquer chamadas subsequentes a seu método `__next__()` vão apenas levantar a exceção *`StopIteration`* novamente. Iteradores precisam ter um método *`__iter__()`* que retorne o objeto iterador em si, de forma que todo iterador também é iterável e pode ser usado na maioria dos lugares em que um iterável é requerido. Uma notável exceção é código que tenta realizar passagens em múltiplas iterações. Um objeto contêiner (como uma *`list`*) produz um novo iterador a cada vez que você passá-lo para a função *`iter()`* ou utilizá-lo em um laço `for`. Tentar isso com o mesmo iterador apenas iria retornar o mesmo objeto iterador esgotado já utilizado na iteração anterior, como se fosse um contêiner vazio.

Mais informações podem ser encontradas em *Tipos iteradores*.

O CPython não aplica consistentemente o requisito de que um iterador defina *`__iter__()`*. E também observe que o CPython com threads livres não garante a segurança do thread das operações do iterador.

função chave
Uma função chave ou função colação é um chamável que retorna um valor usado para ordenação ou classifi-

cação. Por exemplo, *locale.strxfrm()* é usada para produzir uma chave de ordenação que leva o locale em consideração para fins de ordenação.

Uma porção de ferramentas no Python aceitam funções chave para controlar como os elementos são ordenados ou agrupados. Algumas delas incluem *min()*, *max()*, *sorted()*, *list.sort()*, *heapq.merge()*, *heapq.nsmallest()*, *heapq.nlargest()* e *itertools.groupby()*.

Há várias maneiras de se criar funções chave. Por exemplo, o método *str.lower()* pode servir como uma função chave para ordenações insensíveis à caixa. Alternativamente, uma função chave ad-hoc pode ser construída a partir de uma expressão `lambda`, como `lambda r: (r[0], r[2])`. Além disso, *operator.attrgetter()*, *operator.itemgetter()* e *operator.methodcaller()* são três construtores de função chave. Consulte o guia de Ordenação para ver exemplos de como criar e utilizar funções chave.

argumento nomeado
Veja *argumento*.

lambda
Uma função de linha anônima consistindo de uma única *expressão*, que é avaliada quando a função é chamada. A sintaxe para criar uma função lambda é `lambda [parameters]: expression`

LBYL
Iniciais da expressão em inglês "look before you leap", que significa algo como "olhe antes de pisar". Este estilo de codificação testa as pré-condições explicitamente antes de fazer chamadas ou buscas. Este estilo contrasta com a abordagem *EAFP* e é caracterizada pela presença de muitas instruções `if`.

Em um ambiente multithread, a abordagem LBYL pode arriscar a introdução de uma condição de corrida entre "o olhar" e "o pisar". Por exemplo, o código `if key in mapping: return mapping[key]` pode falhar se outra thread remover *key* do *mapping* após o teste, mas antes da olhada. Esse problema pode ser resolvido com travas ou usando a abordagem EAFP.

analisador léxico
Nome formal para o *tokenizador*; veja *token*.

lista
Uma *sequência* embutida no Python. Apesar do seu nome, é mais próximo de um vetor em outras linguagens do que uma lista encadeada, como o acesso aos elementos é da ordem $O(1)$.

compreensão de lista
Uma maneira compacta de processar todos ou parte dos elementos de uma sequência e retornar os resultados em uma lista. `result = ['{:#04x}'.format(x) for x in range(256) if x % 2 == 0]` gera uma lista de strings contendo números hexadecimais (0x..) no intervalo de 0 a 255. A cláusula `if` é opcional. Se omitida, todos os elementos no `range(256)` serão processados.

carregador
Um objeto que carrega um módulo. Ele deve definir os métodos `exec_module()` e `create_module()` para implementar a interface *Loader*. Um carregador é normalmente retornado por um *localizador*. Veja também:

- finders-and-loaders
- *importlib.abc.Loader*
- **PEP 302**

codificação da localidade
No Unix, é a codificação da localidade do LC_CTYPE, que pode ser definida com *locale.setlocale(locale.LC_CTYPE, new_locale)*.

No Windows, é a página de código ANSI (ex: `"cp1252"`).

No Android e no VxWorks, o Python usa `"utf-8"` como a codificação da localidade.

locale.getencoding() pode ser usado para obter a codificação da localidade.

Veja também *tratador de erros e codificação do sistema de arquivos*.

método mágico
Um sinônimo informal para um *método especial*.

mapeamento
Um objeto contêiner com suporte a pesquisas de chave arbitrária e que implementa os métodos especificados nas *classes base abstratas* `collections.abc.Mapping` ou `collections.abc.MutableMapping`. Exemplos incluem `dict`, `collections.defaultdict`, `collections.OrderedDict` e `collections.Counter`.

localizador de metacaminho
Um *localizador* retornado por uma busca de `sys.meta_path`. Localizadores de metacaminho são relacionados a, mas diferentes de, *localizadores de entrada de caminho*.

Veja `importlib.abc.MetaPathFinder` para os métodos que localizadores de metacaminho implementam.

metaclasse
A classe de uma classe. Definições de classe criam um nome de classe, um dicionário de classe e uma lista de classes base. A metaclasse é responsável por receber estes três argumentos e criar a classe. A maioria das linguagens de programação orientadas a objetos provê uma implementação default. O que torna o Python especial é o fato de ser possível criar metaclasses personalizadas. A maioria dos usuários nunca vai precisar deste recurso, mas quando houver necessidade, metaclasses possibilitam soluções poderosas e elegantes. Metaclasses têm sido utilizadas para gerar registros de acesso a atributos, para incluir proteção contra acesso concorrente, rastrear a criação de objetos, implementar singletons, dentre muitas outras tarefas.

Mais informações podem ser encontradas em metaclasses.

método
Uma função que é definida dentro do corpo de uma classe. Se chamada como um atributo de uma instância daquela classe, o método receberá a instância do objeto como seu primeiro *argumento* (que comumente é chamado de `self`). Veja *função* e *escopo aninhado*.

ordem de resolução de métodos
Ordem de resolução de métodos é a ordem em que os membros de uma classe base são buscados durante a pesquisa. Veja python_2.3_mro para detalhes do algoritmo usado pelo interpretador do Python desde a versão 2.3.

módulo
Um objeto que serve como uma unidade organizacional de código Python. Os módulos têm um espaço de nomes contendo objetos Python arbitrários. Os módulos são carregados pelo Python através do processo de *importação*.

Veja também *pacote*.

spec de módulo
Um espaço de nomes que contém as informações relacionadas à importação usadas para carregar um módulo. Uma instância de `importlib.machinery.ModuleSpec`.

Veja também module-specs.

MRO
Veja *ordem de resolução de métodos*.

mutável
Objeto mutável é aquele que pode modificar seus valor mas manter seu `id()`. Veja também *imutável*.

tupla nomeada
O termo "tupla nomeada" é aplicado a qualquer tipo ou classe que herda de tupla e cujos elementos indexáveis também são acessíveis usando atributos nomeados. O tipo ou classe pode ter outras funcionalidades também.

Diversos tipos embutidos são tuplas nomeadas, incluindo os valores retornados por `time.localtime()` e `os.stat()`. Outro exemplo é `sys.float_info`:

```
>>> sys.float_info[1]                    # acesso indexado
1024
>>> sys.float_info.max_exp               # acesso a campo nomeado
1024
>>> isinstance(sys.float_info, tuple)    # tipo de tupla
True
```

Algumas tuplas nomeadas são tipos embutidos (tal como os exemplos acima). Alternativamente, uma tupla nomeada pode ser criada a partir de uma definição de classe regular, que herde de *tuple* e que defina campos nomeados. Tal classe pode ser escrita a mão, ou ela pode ser criada herdando *typing.NamedTuple* ou com uma função fábrica *collections.namedtuple()*. As duas últimas técnicas também adicionam alguns métodos extras, que podem não ser encontrados quando foi escrita manualmente, ou em tuplas nomeadas embutidas.

espaço de nomes
O lugar em que uma variável é armazenada. Espaços de nomes são implementados como dicionários. Existem os espaços de nomes local, global e nativo, bem como espaços de nomes aninhados em objetos (em métodos). Espaços de nomes suportam modularidade ao prevenir conflitos de nomes. Por exemplo, as funções `__builtin__.open()` e *os.open()* são diferenciadas por seus espaços de nomes. Espaços de nomes também auxiliam na legibilidade e na manutenibilidade ao torar mais claro quais módulos implementam uma função. Escrever *random.seed()* ou `itertools.izip()`, por exemplo, deixa claro que estas funções são implementadas pelos módulos *random* e *itertools* respectivamente.

pacote de espaço de nomes
Um *pacote* que serve apenas como contêiner para subpacotes. Pacotes de espaços de nomes podem não ter representação física, e especificamente não são como um *pacote regular* porque eles não tem um arquivo `__init__.py`.

Pacotes de espaço de nomes permitem que vários pacotes instaláveis individualmente tenham um pacote pai comum. Caso contrário, é recomendado usar um *pacote regular*.

Para mais informações, veja **PEP 420** e reference-namespace-package.

Veja também *módulo*.

escopo aninhado
A habilidade de referir-se a uma variável em uma definição de fechamento. Por exemplo, uma função definida dentro de outra pode referenciar variáveis da função externa. Perceba que escopos aninhados por padrão funcionam apenas por referência e não por atribuição. Variáveis locais podem ler e escrever no escopo mais interno. De forma similar, variáveis globais podem ler e escrever para o espaço de nomes global. O `nonlocal` permite escrita para escopos externos.

classe estilo novo
Antigo nome para o tipo de classes agora usado para todos os objetos de classes. Em versões anteriores do Python, apenas classes estilo novo podiam usar recursos novos e versáteis do Python, tais como `__slots__`, descritores, propriedades, `__getattribute__()`, métodos de classe e métodos estáticos.

objeto
Qualquer dado que tenha estado (atributos ou valores) e comportamento definidos (métodos). Também a última classe base de qualquer *classe estilo novo*.

escopo otimizado
Um escopo no qual os nomes das variáveis locais de destino são conhecidos de forma confiável pelo compilador quando o código é compilado, permitindo a otimização do acesso de leitura e gravação a esses nomes. Os espaços de nomes locais para funções, geradores, corrotinas, compreensões e expressões geradoras são otimizados desta forma. Nota: a maioria das otimizações de interpretador são aplicadas a todos os escopos, apenas aquelas que dependem de um conjunto conhecido de nomes de variáveis locais e não locais são restritas a escopos otimizados.

pacote
Um *módulo* Python é capaz de conter submódulos ou recursivamente, subpacotes. Tecnicamente, um pacote é um módulo Python com um atributo `__path__`.

Veja também *pacote regular* e *pacote de espaço de nomes*.

parâmetro
Uma entidade nomeada na definição de uma *função* (ou método) que específica um *argumento* (ou em alguns casos, argumentos) que a função pode receber. Existem cinco tipos de parâmetros:

- *posicional-ou-nomeado*: especifica um argumento que pode ser tanto *posicional* quanto *nomeado*. Esse é o tipo padrão de parâmetro, por exemplo *foo* e *bar* a seguir:

```
def func(foo, bar=None): ...
```

- *somente-posicional*: especifica um argumento que pode ser fornecido apenas por posição. Parâmetros somente-posicionais podem ser definidos incluindo o caractere `/` na lista de parâmetros da definição da função após eles, por exemplo *somentepos1* e *somentepos2* a seguir:

```
def func(somentepos1, somentepos2, /, posicional_ou_nomeado): ...
```

- *somente-nomeado*: especifica um argumento que pode ser passado para a função somente por nome. Parâmetros somente-nomeados podem ser definidos com um simples parâmetro var-posicional ou um `*` antes deles na lista de parâmetros na definição da função, por exemplo *somente_nom1* and *somente_nom2* a seguir:

```
def func(arg, *, somente_nom1, somente_nom2): ...
```

- *var-posicional*: especifica que uma sequência arbitrária de argumentos posicionais pode ser fornecida (em adição a qualquer argumento posicional já aceito por outros parâmetros). Tal parâmetro pode ser definido colocando um `*` antes do nome do parâmetro, por exemplo *args* a seguir:

```
def func(*args, **kwargs): ...
```

- *var-nomeado*: especifica que, arbitrariamente, muitos argumentos nomeados podem ser fornecidos (em adição a qualquer argumento nomeado já aceito por outros parâmetros). Tal parâmetro pode ser definido colocando-se `**` antes do nome, por exemplo, *kwargs* no exemplo acima.

Parâmetros podem especificar tanto argumentos opcionais quanto obrigatórios, assim como valores padrão para alguns argumentos opcionais.

Veja também o termo *argumento* no glossário, a pergunta do FAQ sobre a diferença entre argumentos e parâmetros, a classe `inspect.Parameter`, a seção function e a **PEP 362**.

entrada de caminho
: Um local único no *caminho de importação* que o *localizador baseado no caminho* consulta para encontrar módulos a serem importados.

localizador de entrada de caminho
: Um *localizador* retornado por um chamável em `sys.path_hooks` (ou seja, um *gancho de entrada de caminho*) que sabe como localizar os módulos *entrada de caminho*.

  Veja `importlib.abc.PathEntryFinder` para os métodos que localizadores de entrada de caminho implementam.

gancho de entrada de caminho
: Um chamável na lista `sys.path_hooks` que retorna um *localizador de entrada de caminho* caso saiba como localizar módulos em uma *entrada de caminho* específica.

localizador baseado no caminho
: Um dos *localizadores de metacaminho* padrão que procura por um *caminho de importação* de módulos.

objeto caminho ou similar
: Um objeto representando um caminho de sistema de arquivos. Um objeto caminho ou similar é ou um objeto `str` ou `bytes` representando um caminho, ou um objeto implementando o protocolo `os.PathLike`. Um objeto que suporta o protocolo `os.PathLike` pode ser convertido para um arquivo de caminho do sistema `str` ou `bytes`, através da chamada da função `os.fspath()`; `os.fsdecode()` e `os.fsencode()` podem ser usadas para garantir um `str` ou `bytes` como resultado, respectivamente. Introduzido na **PEP 519**.

PEP
: Proposta de melhoria do Python. Uma PEP é um documento de design que fornece informação para a comunidade Python, ou descreve uma nova funcionalidade para o Python ou seus predecessores ou ambientes. PEPs devem prover uma especificação técnica concisa e um racional para funcionalidades propostas.

  PEPs têm a intenção de ser os mecanismos primários para propor novas funcionalidades significativas, para coletar opiniões da comunidade sobre um problema, e para documentar as decisões de design que foram

adicionadas ao Python. O autor da PEP é responsável por construir um consenso dentro da comunidade e documentar opiniões dissidentes.

Veja **PEP 1**.

porção
Um conjunto de arquivos em um único diretório (possivelmente armazenado em um arquivo zip) que contribuem para um pacote de espaço de nomes, conforme definido em **PEP 420**.

argumento posicional
Veja *argumento*.

API provisória
Uma API provisória é uma API que foi deliberadamente excluída das bibliotecas padrões com compatibilidade retroativa garantida. Enquanto mudanças maiores para tais interfaces não são esperadas, contanto que elas sejam marcadas como provisórias, mudanças retroativas incompatíveis (até e incluindo a remoção da interface) podem ocorrer se consideradas necessárias pelos desenvolvedores principais. Tais mudanças não serão feitas gratuitamente – elas irão ocorrer apenas se sérias falhas fundamentais forem descobertas, que foram esquecidas anteriormente a inclusão da API.

Mesmo para APIs provisórias, mudanças retroativas incompatíveis são vistas como uma "solução em último caso" - cada tentativa ainda será feita para encontrar uma resolução retroativa compatível para quaisquer problemas encontrados.

Esse processo permite que a biblioteca padrão continue a evoluir com o passar do tempo, sem se prender em erros de design problemáticos por períodos de tempo prolongados. Veja **PEP 411** para mais detalhes.

pacote provisório
Veja *API provisória*.

Python 3000
Apelido para a linha de lançamento da versão do Python 3.x (cunhada há muito tempo, quando o lançamento da versão 3 era algo em um futuro muito distante.) Esse termo possui a seguinte abreviação: "Py3k".

Pythônico
Uma ideia ou um pedaço de código que segue de perto as formas de escritas mais comuns da linguagem Python, ao invés de implementar códigos usando conceitos comuns a outras linguagens. Por exemplo, um formato comum em Python é fazer um laço sobre todos os elementos de uma iterável usando a instrução `for`. Muitas outras linguagens não têm esse tipo de construção, então as pessoas que não estão familiarizadas com o Python usam um contador numérico:

```
for i in range(len(comida)):
    print(comida[i])
```

Ao contrário do método mais limpo, Pythônico:

```
for parte in comida:
    print(parte)
```

nome qualificado
Um nome pontilhado (quando 2 termos são ligados por um ponto) que mostra o "path" do escopo global de um módulo para uma classe, função ou método definido num determinado módulo, conforme definido pela **PEP 3155**. Para funções e classes de nível superior, o nome qualificado é o mesmo que o nome do objeto:

```
>>> class C:
...     class D:
...         def metodo(self):
...             pass
...
>>> C.__qualname__
'C'
>>> C.D.__qualname__
```

(continua na próxima página)

(continuação da página anterior)

```
'C.D'
>>> C.D.metodo.__qualname__
'C.D.metodo'
```

Quando usado para se referir a módulos, o *nome totalmente qualificado* significa todo o caminho pontilhado para o módulo, incluindo quaisquer pacotes pai, por exemplo: `email.mime.text`:

```
>>> import email.mime.text
>>> email.mime.text.__name__
'email.mime.text'
```

contagem de referências
O número de referências a um objeto. Quando a contagem de referências de um objeto cai para zero, ele é desalocado. Alguns objetos são *imortais* e têm contagens de referências que nunca são modificadas e, portanto, os objetos nunca são desalocados. A contagem de referências geralmente não é visível para o código Python, mas é um elemento-chave da implementação do *CPython*. Os programadores podem chamar a função `sys.getrefcount()` para retornar a contagem de referências para um objeto específico.

Em *CPython*, as contagens de referências não são consideradas valores estáveis ou bem definidos; o número de referências a um objeto e como esse número é afetado pelo código Python pode ser diferente entre as versões.

pacote regular
Um *pacote* tradicional, como um diretório contendo um arquivo `__init__.py`.

Veja também *pacote de espaço de nomes*.

REPL
Um acrônimo para "read–eval–print loop", outro nome para o console *interativo* do interpretador.

__slots__
Uma declaração dentro de uma classe que economiza memória pré-declarando espaço para atributos de instâncias, e eliminando dicionários de instâncias. Apesar de popular, a técnica é um tanto quanto complicada de acertar, e é melhor se for reservada para casos raros, onde existe uma grande quantidade de instâncias em uma aplicação onde a memória é crítica.

sequência
Um *iterável* com suporte para acesso eficiente a seus elementos através de índices inteiros via método especial `__getitem__()` e que define o método `__len__()` que devolve o tamanho da sequência. Alguns tipos de sequência embutidos são: `list`, `str`, `tuple`, e `bytes`. Note que `dict` também tem suporte para `__getitem__()` e `__len__()`, mas é considerado um mapeamento e não uma sequência porque a busca usa uma chave *hasheável* arbitrária em vez de inteiros.

A classe base abstrata `collections.abc.Sequence` define uma interface mais rica que vai além de apenas `__getitem__()` e `__len__()`, adicionando `count()`, `index()`, `__contains__()`, e `__reversed__()`. Tipos que implementam essa interface podem ser explicitamente registrados usando `register()`. Para mais documentação sobre métodos de sequências em geral, veja *Operações comuns de sequências*.

compreensão de conjunto
Uma maneira compacta de processar todos ou parte dos elementos em iterável e retornar um conjunto com os resultados. `results = {c for c in 'abracadabra' if c not in 'abc'}` gera um conjunto de strings `{'r', 'd'}`. Veja comprehensions.

despacho único
Uma forma de despacho de *função genérica* onde a implementação é escolhida com base no tipo de um único argumento.

fatia
Um objeto geralmente contendo uma parte de uma *sequência*. Uma fatia é criada usando a notação de subscrito `[]` pode conter também até dois pontos entre números, como em `variable_name[1:3:5]`. A notação de suporte (subscrito) utiliza objetos `slice` internamente.

suavemente descontinuado
Uma API suavemente descontinuada não deve ser usada em código novo, mas é seguro para código já existente usá-la. A API continua documentada e testada, mas não será melhorada.

A descontinuação suave, diferentemente da descontinuação normal, não planeja remover a API e não emitirá avisos.

Veja PEP 387: Descontinuação suave.

método especial
Um método que é chamado implicitamente pelo Python para executar uma certa operação em um tipo, como uma adição por exemplo. Tais métodos tem nomes iniciando e terminando com dois underscores. Métodos especiais estão documentados em specialnames.

biblioteca padrão
A coleção de *pacotes*, *módulos* e *módulos de extensão* distribuída como parte do pacote do interpretador Python oficial. A relação exata de elementos da coleção pode variar de acordo com a plataforma, bibliotecas de sistema disponíveis ou outros critérios. A documentação pode ser encontrada em *A Biblioteca Padrão do Python*.

Veja também `sys.stdlib_module_names` para obter uma lista de todos os possíveis nomes de módulos da biblioteca padrão.

instrução
Uma instrução é parte de uma suíte (um "bloco" de código). Uma instrução é ou uma *expressão* ou uma de várias construções com uma palavra reservada, tal como `if`, `while` ou `for`.

verificador de tipo estático
Uma ferramenta externa que lê código Python e o analisa, procurando por problemas como tipos incorretos. Consulte também *dicas de tipo* e o módulo `typing`.

stdlib
Uma abreviação para o termo *biblioteca padrão* em inglês (standard library).

referência forte
Na API C do Python, uma referência forte é uma referência a um objeto que pertence ao código que contém a referência. A referência forte é obtida chamando `Py_INCREF()` quando a referência é criada e liberada com `Py_DECREF()` quando a referência é excluída.

A função `Py_NewRef()` pode ser usada para criar uma referência forte para um objeto. Normalmente, a função `Py_DECREF()` deve ser chamada na referência forte antes de sair do escopo da referência forte, para evitar o vazamento de uma referência.

Veja também *referência emprestada*.

t-string
t-strings
Literais de string prefixados com `t` ou `T` são comumente chamados de "t-strings", que é uma abreviação de literais de strings templates.

codificador de texto
Uma string em Python é uma sequência de pontos de código Unicode (no intervalo `U+0000–U+10FFFF`). Para armazenar ou transferir uma string, ela precisa ser serializada como uma sequência de bytes.

A serialização de uma string em uma sequência de bytes é conhecida como "codificação" e a recriação da string a partir de uma sequência de bytes é conhecida como "decodificação".

Há uma variedade de diferentes serializações de texto *codecs*, que são coletivamente chamadas de "codificações de texto".

arquivo texto
Um *objeto arquivo* apto a ler e escrever objetos `str`. Geralmente, um arquivo texto, na verdade, acessa um fluxo de dados de bytes e captura o *codificador de texto* automaticamente. Exemplos de arquivos texto são: arquivos abertos em modo texto (`'r'` or `'w'`), `sys.stdin`, `sys.stdout`, e instâncias de `io.StringIO`.

Veja também *arquivo binário* para um objeto arquivo apto a ler e escrever *objetos bytes ou similares*.

estado de thread
: As informações usadas pelo ambiente de execução do *CPython* para executar em uma thread do sistema operacional. Por exemplo, isso inclui a exceção atual, se houver, e o estado do interpretador de bytecode.

  Cada estado de thread está vinculado a uma única thread do SO, mas as threads podem ter muitos estados de thread disponíveis. No máximo, um deles pode ser *anexado* por vez.

  Um *estado de thread anexado* é necessário para chamar a maioria da API C do Python, a menos que uma função documente explicitamente o contrário. O interpretador de bytecode só é executado em um estado de thread anexado.

  Cada estado de thread pertence a um único interpretador, mas cada interpretador pode ter vários estados de thread, incluindo vários para a mesma thread do sistema operacional. Estados de thread de vários interpretadores podem ser vinculados à mesma thread, mas apenas um pode ser *anexado* nessa thread a qualquer momento.

  Veja Estado de thread e a trava global do interpretador para mais informações.

token
: Uma pequena unidade de código-fonte, gerada pelo analisador léxico (também chamado de *tokenizador*). Nomes, números, strings, operadores, quebras de linha e similares são representados por tokens.

  O módulo `tokenize` expõe o analisador léxico do Python. O módulo `token` contém informações sobre os vários tipos de tokens.

aspas triplas
: Uma string que está definida com três ocorrências de aspas duplas (") ou apóstrofos ('). Enquanto elas não fornecem nenhuma funcionalidade não disponível com strings de aspas simples, elas são úteis para inúmeras razões. Elas permitem que você inclua aspas simples e duplas não escapadas dentro de uma string, e elas podem utilizar múltiplas linhas sem o uso de caractere de continuação, fazendo-as especialmente úteis quando escrevemos documentação em docstrings.

tipo
: O tipo de um objeto Python determina qual classe de objeto ele é; cada objeto tem um tipo. Um tipo de objeto é acessível pelo atributo `__class__` ou pode ser recuperado com `type(obj)`.

apelido de tipo
: Um sinônimo para um tipo, criado através da atribuição do tipo para um identificador.

  Apelidos de tipo são úteis para simplificar *dicas de tipo*. Por exemplo:

```
def remove_tons_de_cinza(
        cores: list[tuple[int, int, int]]) -> list[tuple[int, int, int]]:
    pass
```

  pode tornar-se mais legível desta forma:

```
Cor = tuple[int, int, int]

def remove_tons_de_cinza(cores: list[Cor]) -> list[Cor]:
    pass
```

  Veja `typing` e **PEP 484**, a qual descreve esta funcionalidade.

dica de tipo
: Uma *anotação* que especifica o tipo esperado para uma variável, um atributo de classe, ou um parâmetro de função ou um valor de retorno.

  Dicas de tipo são opcionais e não são forçadas pelo Python, mas elas são úteis para *verificadores de tipo estático*. Eles também ajudam IDEs a completar e refatorar código.

  Dicas de tipos de variáveis globais, atributos de classes, e funções, mas não de variáveis locais, podem ser acessadas usando `typing.get_type_hints()`.

  Veja `typing` e **PEP 484**, a qual descreve esta funcionalidade.

novas linhas universais
: Uma maneira de interpretar fluxos de textos, na qual todos estes são reconhecidos como caracteres de fim de linha: a convenção para fim de linha no Unix `'\n'`, a convenção no Windows `'\r\n'`, e a antiga convenção no Macintosh `'\r'`. Veja **PEP 278** e **PEP 3116**, bem como *`bytes.splitlines()`* para uso adicional.

anotação de variável
: Uma *anotação* de uma variável ou um atributo de classe.

  Ao fazer uma anotação de uma variável ou um atributo de classe, a atribuição é opcional:

```
class C:
    campo: 'anotação'
```

  Anotações de variáveis são normalmente usadas para *dicas de tipo*: por exemplo, espera-se que esta variável receba valores do tipo *`int`*:

```
contagem: int = 0
```

  A sintaxe de anotação de variável é explicada na seção annassign.

  Veja *anotação de função*, **PEP 484** e **PEP 526**, que descrevem esta funcionalidade. Veja também annotations-howto para as melhores práticas sobre como trabalhar com anotações.

ambiente virtual
: Um ambiente de execução isolado que permite usuários Python e aplicações instalarem e atualizarem pacotes Python sem interferir no comportamento de outras aplicações Python em execução no mesmo sistema.

  Veja também *`venv`*.

máquina virtual
: Um computador definido inteiramente em software. A máquina virtual de Python executa o *bytecode* emitido pelo compilador de bytecode.

operador morsa
: Uma maneira bem-humorada de se referir ao operador da expressão de atribuição `:=` porque ele se parece um pouco com uma morsa se você virar a cabeça.

Zen do Python
: Lista de princípios de projeto e filosofias do Python que são úteis para a compreensão e uso da linguagem. A lista é exibida quando se digita "`import this`" no console interativo.

APÊNDICE B

Sobre esta documentação

A documentação do Python é gerada a partir de fontes reStructuredText usando Sphinx, um gerador de documentação criado originalmente para Python e agora mantido como um projeto independente.

O desenvolvimento da documentação e de suas ferramentas é um esforço totalmente voluntário, como Python em si. Se você quer contribuir, por favor dê uma olhada na página reporting-bugs para informações sobre como fazer. Novos voluntários são sempre bem-vindos!

Agradecimentos especiais para:

- Fred L. Drake, Jr., o criador do primeiro conjunto de ferramentas para documentar Python e autor de boa parte do conteúdo;
- O projeto Docutils por criar reStructuredText e o pacote Docutils;
- Fredrik Lundh, pelo seu projeto de referência alternativa em Python, do qual Sphinx pegou muitas boas ideias.

B.1 Contribuidores da documentação do Python

Muitas pessoas tem contribuído para a linguagem Python, sua biblioteca padrão e sua documentação. Veja Misc/ACKS na distribuição do código do Python para ver uma lista parcial de contribuidores.

Tudo isso só foi possível com o esforço e a contribuição da comunidade Python, por isso temos essa maravilhosa documentação – Obrigado a todos!

APÊNDICE C

História e Licença

C.1 História do software

Python foi criado no início dos anos 1990 por Guido van Rossum no Stichting Mathematisch Centrum (CWI, veja https://www.cwi.nl) na Holanda como sucessor de uma linguagem chamada ABC. Guido continua sendo o principal autor do Python, embora inclua muitas contribuições de outros.

Em 1995, Guido continuou seu trabalho em Python na Corporation for National Research Initiatives (CNRI, veja https://www.cnri.reston.va.us) em Reston, Virgínia, onde lançou várias versões do software.

Em maio de 2000, Guido e a equipe de desenvolvimento do núcleo Python mudaram-se para BeOpen.com para formar a equipe BeOpen PythonLabs. Em outubro do mesmo ano, a equipe PythonLabs mudou-se para a Digital Creations, que se tornou Zope Corporation. Em 2001, a Python Software Foundation (PSF, veja https://www.python.org/psf/) foi formada, uma organização sem fins lucrativos criada especificamente para possuir Propriedade Intelectual relacionada ao Python. A Zope Corporation era um membro patrocinador da PSF.

Todas as versões do Python são de código aberto (consulte https://opensource.org para a definição de código aberto). Historicamente, a maioria, mas não todas, versões do Python também são compatíveis com GPL; a tabela abaixo resume os vários lançamentos.

Versão	Derivada de	Ano	Proprietário	Compatível com a GPL? (1)
0.9.0 a 1.2	n/a	1991-1995	CWI	sim
1.3 a 1.5.2	1.2	1995-1999	CNRI	sim
1.6	1.5.2	2000	CNRI	não
2.0	1.6	2000	BeOpen.com	não
1.6.1	1.6	2001	CNRI	sim (2)
2.1	2.0+1.6.1	2001	PSF	não
2.0.1	2.0+1.6.1	2001	PSF	sim
2.1.1	2.1+2.0.1	2001	PSF	sim
2.1.2	2.1.1	2002	PSF	sim
2.1.3	2.1.2	2002	PSF	sim
2.2 e acima	2.1.1	2001-agora	PSF	sim

Nota

(1) Compatível com a GPL não significa que estamos distribuindo Python sob a GPL. Todas as licenças do Python, ao contrário da GPL, permitem distribuir uma versão modificada sem fazer alterações em código aberto. As licenças compatíveis com a GPL possibilitam combinar o Python com outro software lançado sob a GPL; os outros não.

(2) De acordo com Richard Stallman, 1.6.1 não é compatível com GPL, porque sua licença tem uma cláusula de escolha de lei. De acordo com a CNRI, no entanto, o advogado de Stallman disse ao advogado da CNRI que 1.6.1 "não é incompatível" com a GPL.

Graças aos muitos voluntários externos que trabalharam sob a direção de Guido para tornar esses lançamentos possíveis.

C.2 Termos e condições para acessar ou usar Python

O software e a documentação do Python são licenciados sob a Python Software Foundation License Versão 2.

A partir do Python 3.8.6, exemplos, receitas e outros códigos na documentação são licenciados duplamente sob o Licença PSF versão 2 e a *Licença BSD de Zero Cláusula*.

Alguns softwares incorporados ao Python estão sob licenças diferentes. As licenças são listadas com o código abrangido por essa licença. Veja *Licenças e Reconhecimentos para Software Incorporado* para uma lista incompleta dessas licenças.

C.2.1 PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2

```
1. This LICENSE AGREEMENT is between the Python Software Foundation ("PSF"), and
   the Individual or Organization ("Licensee") accessing and otherwise using this
   software ("Python") in source or binary form and its associated documentation.

2. Subject to the terms and conditions of this License Agreement, PSF hereby
   grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce,
   analyze, test, perform and/or display publicly, prepare derivative works,
   distribute, and otherwise use Python alone or in any derivative
   version, provided, however, that PSF's License Agreement and PSF's notice of
   copyright, i.e., "Copyright © 2001 Python Software Foundation; All Rights
   Reserved" are retained in Python alone or in any derivative version
   prepared by Licensee.

3. In the event Licensee prepares a derivative work that is based on or
   incorporates Python or any part thereof, and wants to make the
   derivative work available to others as provided herein, then Licensee hereby
   agrees to include in any such work a brief summary of the changes made to␣
↪Python.

4. PSF is making Python available to Licensee on an "AS IS" basis.
   PSF MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED.  BY WAY OF
   EXAMPLE, BUT NOT LIMITATION, PSF MAKES NO AND DISCLAIMS ANY REPRESENTATION OR
   WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE
   USE OF PYTHON WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. PSF SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PYTHON
   FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF
   MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON, OR ANY DERIVATIVE
   THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material breach of
   its terms and conditions.
```

(continua na próxima página)

(continuação da página anterior)

```
7. Nothing in this License Agreement shall be deemed to create any relationship
   of agency, partnership, or joint venture between PSF and Licensee.  This License
   Agreement does not grant permission to use PSF trademarks or trade name in a
   trademark sense to endorse or promote products or services of Licensee, or any
   third party.

8. By copying, installing or otherwise using Python, Licensee agrees
   to be bound by the terms and conditions of this License Agreement.
```

C.2.2 ACORDO DE LICENCIAMENTO DA BEOPEN.COM PARA PYTHON 2.0

ACORDO DE LICENCIAMENTO DA BEOPEN DE FONTE ABERTA DO PYTHON VERSÃO 1

```
1. This LICENSE AGREEMENT is between BeOpen.com ("BeOpen"), having an office at
   160 Saratoga Avenue, Santa Clara, CA 95051, and the Individual or Organization
   ("Licensee") accessing and otherwise using this software in source or binary
   form and its associated documentation ("the Software").

2. Subject to the terms and conditions of this BeOpen Python License Agreement,
   BeOpen hereby grants Licensee a non-exclusive, royalty-free, world-wide license
   to reproduce, analyze, test, perform and/or display publicly, prepare derivative
   works, distribute, and otherwise use the Software alone or in any derivative
   version, provided, however, that the BeOpen Python License is retained in the
   Software, alone or in any derivative version prepared by Licensee.

3. BeOpen is making the Software available to Licensee on an "AS IS" basis.
   BEOPEN MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED.  BY WAY OF
   EXAMPLE, BUT NOT LIMITATION, BEOPEN MAKES NO AND DISCLAIMS ANY REPRESENTATION OR
   WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE
   USE OF THE SOFTWARE WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

4. BEOPEN SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF THE SOFTWARE FOR
   ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF USING,
   MODIFYING OR DISTRIBUTING THE SOFTWARE, OR ANY DERIVATIVE THEREOF, EVEN IF
   ADVISED OF THE POSSIBILITY THEREOF.

5. This License Agreement will automatically terminate upon a material breach of
   its terms and conditions.

6. This License Agreement shall be governed by and interpreted in all respects
   by the law of the State of California, excluding conflict of law provisions.
   Nothing in this License Agreement shall be deemed to create any relationship of
   agency, partnership, or joint venture between BeOpen and Licensee.  This License
   Agreement does not grant permission to use BeOpen trademarks or trade names in a
   trademark sense to endorse or promote products or services of Licensee, or any
   third party.  As an exception, the "BeOpen Python" logos available at
   http://www.pythonlabs.com/logos.html may be used according to the permissions
   granted on that web page.

7. By copying, installing or otherwise using the software, Licensee agrees to be
   bound by the terms and conditions of this License Agreement.
```

C.2.3 CONTRATO DE LICENÇA DA CNRI PARA O PYTHON 1.6.1

```
1. This LICENSE AGREEMENT is between the Corporation for National Research
   Initiatives, having an office at 1895 Preston White Drive, Reston, VA 20191
   ("CNRI"), and the Individual or Organization ("Licensee") accessing and
   otherwise using Python 1.6.1 software in source or binary form and its
   associated documentation.

2. Subject to the terms and conditions of this License Agreement, CNRI hereby
   grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce,
   analyze, test, perform and/or display publicly, prepare derivative works,
   distribute, and otherwise use Python 1.6.1 alone or in any derivative version,
   provided, however, that CNRI's License Agreement and CNRI's notice of copyright,
   i.e., "Copyright © 1995-2001 Corporation for National Research Initiatives; All
   Rights Reserved" are retained in Python 1.6.1 alone or in any derivative version
   prepared by Licensee.  Alternately, in lieu of CNRI's License Agreement,
   Licensee may substitute the following text (omitting the quotes): "Python 1.6.1
   is made available subject to the terms and conditions in CNRI's License
   Agreement.  This Agreement together with Python 1.6.1 may be located on the
   internet using the following unique, persistent identifier (known as a handle):
   1895.22/1013.  This Agreement may also be obtained from a proxy server on the
   internet using the following URL: http://hdl.handle.net/1895.22/1013".

3. In the event Licensee prepares a derivative work that is based on or
   incorporates Python 1.6.1 or any part thereof, and wants to make the derivative
   work available to others as provided herein, then Licensee hereby agrees to
   include in any such work a brief summary of the changes made to Python 1.6.1.

4. CNRI is making Python 1.6.1 available to Licensee on an "AS IS" basis.  CNRI
   MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED.  BY WAY OF EXAMPLE,
   BUT NOT LIMITATION, CNRI MAKES NO AND DISCLAIMS ANY REPRESENTATION OR WARRANTY
   OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF
   PYTHON 1.6.1 WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. CNRI SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PYTHON 1.6.1 FOR
   ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF
   MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON 1.6.1, OR ANY DERIVATIVE
   THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material breach of
   its terms and conditions.

7. This License Agreement shall be governed by the federal intellectual property
   law of the United States, including without limitation the federal copyright
   law, and, to the extent such U.S. federal law does not apply, by the law of the
   Commonwealth of Virginia, excluding Virginia's conflict of law provisions.
   Notwithstanding the foregoing, with regard to derivative works based on Python
   1.6.1 that incorporate non-separable material that was previously distributed
   under the GNU General Public License (GPL), the law of the Commonwealth of
   Virginia shall govern this License Agreement only as to issues arising under or
   with respect to Paragraphs 4, 5, and 7 of this License Agreement.  Nothing in
   this License Agreement shall be deemed to create any relationship of agency,
   partnership, or joint venture between CNRI and Licensee.  This License Agreement
   does not grant permission to use CNRI trademarks or trade name in a trademark
   sense to endorse or promote products or services of Licensee, or any third
   party.
```

(continua na próxima página)

(continuação da página anterior)

```
8. By clicking on the "ACCEPT" button where indicated, or by copying, installing
   or otherwise using Python 1.6.1, Licensee agrees to be bound by the terms and
   conditions of this License Agreement.
```

C.2.4 ACORDO DE LICENÇA DA CWI PARA PYTHON 0.9.0 A 1.2

```
Copyright © 1991 - 1995, Stichting Mathematisch Centrum Amsterdam, The
Netherlands.  All rights reserved.

Permission to use, copy, modify, and distribute this software and its
documentation for any purpose and without fee is hereby granted, provided that
the above copyright notice appear in all copies and that both that copyright
notice and this permission notice appear in supporting documentation, and that
the name of Stichting Mathematisch Centrum or CWI not be used in advertising or
publicity pertaining to distribution of the software without specific, written
prior permission.

STICHTING MATHEMATISCH CENTRUM DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS
SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS, IN NO
EVENT SHALL STICHTING MATHEMATISCH CENTRUM BE LIABLE FOR ANY SPECIAL, INDIRECT
OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE,
DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS
SOFTWARE.
```

C.2.5 ZERO-CLAUSE BSD LICENSE FOR CODE IN THE PYTHON DOCUMENTATION

```
Permission to use, copy, modify, and/or distribute this software for any
purpose with or without fee is hereby granted.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH
REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY
AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT,
INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM
LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR
OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
PERFORMANCE OF THIS SOFTWARE.
```

C.3 Licenças e Reconhecimentos para Software Incorporado

Esta seção é uma lista incompleta, mas crescente, de licenças e reconhecimentos para softwares de terceiros incorporados na distribuição do Python.

C.3.1 Mersenne Twister

A extensão C `_random` subjacente ao módulo *random* inclui código baseado em um download de http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/MT2002/emt19937ar.html. A seguir estão os comentários literais do código original:

```
A C-program for MT19937, with initialization improved 2002/1/26.
Coded by Takuji Nishimura and Makoto Matsumoto.

Before using, initialize the state by using init_genrand(seed)
```

(continua na próxima página)

(continuação da página anterior)

```
or init_by_array(init_key, key_length).

Copyright (C) 1997 - 2002, Makoto Matsumoto and Takuji Nishimura,
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

 1. Redistributions of source code must retain the above copyright
    notice, this list of conditions and the following disclaimer.

 2. Redistributions in binary form must reproduce the above copyright
    notice, this list of conditions and the following disclaimer in the
    documentation and/or other materials provided with the distribution.

 3. The names of its contributors may not be used to endorse or promote
    products derived from this software without specific prior written
    permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED.  IN NO EVENT SHALL THE COPYRIGHT OWNER OR
CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.


Any feedback is very welcome.
http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/emt.html
email: m-mat @ math.sci.hiroshima-u.ac.jp (remove space)
```

C.3.2 Soquetes

O módulo *socket* usa as funções `getaddrinfo()` e `getnameinfo()`, que são codificadas em arquivos de origem separados do Projeto WIDE, https://www.wide.ad.jp/.

```
Copyright (C) 1995, 1996, 1997, and 1998 WIDE Project.
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:
1. Redistributions of source code must retain the above copyright
   notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.
3. Neither the name of the project nor the names of its contributors
   may be used to endorse or promote products derived from this software
   without specific prior written permission.
```

(continua na próxima página)

(continuação da página anterior)

```
THIS SOFTWARE IS PROVIDED BY THE PROJECT AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
ARE DISCLAIMED.  IN NO EVENT SHALL THE PROJECT OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
SUCH DAMAGE.
```

C.3.3 Serviços de soquete assíncrono

Os módulos `test.support.asynchat` e `test.support.asyncore` contêm o seguinte aviso:

```
Copyright 1996 by Sam Rushing

                        All Rights Reserved

Permission to use, copy, modify, and distribute this software and
its documentation for any purpose and without fee is hereby
granted, provided that the above copyright notice appear in all
copies and that both that copyright notice and this permission
notice appear in supporting documentation, and that the name of Sam
Rushing not be used in advertising or publicity pertaining to
distribution of the software without specific, written prior
permission.

SAM RUSHING DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE,
INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS, IN
NO EVENT SHALL SAM RUSHING BE LIABLE FOR ANY SPECIAL, INDIRECT OR
CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS
OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT,
NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN
CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.
```

C.3.4 Gerenciamento de cookies

O módulo *`http.cookies`* contém o seguinte aviso:

```
Copyright 2000 by Timothy O'Malley <timo@alum.mit.edu>

               All Rights Reserved

Permission to use, copy, modify, and distribute this software
and its documentation for any purpose and without fee is hereby
granted, provided that the above copyright notice appear in all
copies and that both that copyright notice and this permission
notice appear in supporting documentation, and that the name of
Timothy O'Malley  not be used in advertising or publicity
pertaining to distribution of the software without specific, written
prior permission.

Timothy O'Malley DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS
```

(continua na próxima página)

(continuação da página anterior)

```
SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY
AND FITNESS, IN NO EVENT SHALL Timothy O'Malley BE LIABLE FOR
ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
PERFORMANCE OF THIS SOFTWARE.
```

C.3.5 Rastreamento de execução

O módulo *trace* contém o seguinte aviso:

```
portions copyright 2001, Autonomous Zones Industries, Inc., all rights...
err...  reserved and offered to the public under the terms of the
Python 2.2 license.
Author: Zooko O'Whielacronx
http://zooko.com/
mailto:zooko@zooko.com

Copyright 2000, Mojam Media, Inc., all rights reserved.
Author: Skip Montanaro

Copyright 1999, Bioreason, Inc., all rights reserved.
Author: Andrew Dalke

Copyright 1995-1997, Automatrix, Inc., all rights reserved.
Author: Skip Montanaro

Copyright 1991-1995, Stichting Mathematisch Centrum, all rights reserved.


Permission to use, copy, modify, and distribute this Python software and
its associated documentation for any purpose without fee is hereby
granted, provided that the above copyright notice appears in all copies,
and that both that copyright notice and this permission notice appear in
supporting documentation, and that the name of neither Automatrix,
Bioreason or Mojam Media be used in advertising or publicity pertaining to
distribution of the software without specific, written prior permission.
```

C.3.6 Funções UUencode e UUdecode

O codec `uu` contém o seguinte aviso:

```
Copyright 1994 by Lance Ellinghouse
Cathedral City, California Republic, United States of America.
                       All Rights Reserved
Permission to use, copy, modify, and distribute this software and its
documentation for any purpose and without fee is hereby granted,
provided that the above copyright notice appear in all copies and that
both that copyright notice and this permission notice appear in
supporting documentation, and that the name of Lance Ellinghouse
not be used in advertising or publicity pertaining to distribution
of the software without specific, written prior permission.
LANCE ELLINGHOUSE DISCLAIMS ALL WARRANTIES WITH REGARD TO
THIS SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS, IN NO EVENT SHALL LANCE ELLINGHOUSE CENTRUM BE LIABLE
```

(continua na próxima página)

(continuação da página anterior)

```
FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN
ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT
OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.

Modified by Jack Jansen, CWI, July 1995:
- Use binascii module to do the actual line-by-line conversion
  between ascii and binary. This results in a 1000-fold speedup. The C
  version is still 5 times faster, though.
- Arguments more compliant with Python standard
```

C.3.7 Chamadas de procedimento remoto XML

O módulo *xmlrpc.client* contém o seguinte aviso:

```
    The XML-RPC client interface is

Copyright (c) 1999-2002 by Secret Labs AB
Copyright (c) 1999-2002 by Fredrik Lundh

By obtaining, using, and/or copying this software and/or its
associated documentation, you agree that you have read, understood,
and will comply with the following terms and conditions:

Permission to use, copy, modify, and distribute this software and
its associated documentation for any purpose and without fee is
hereby granted, provided that the above copyright notice appears in
all copies, and that both that copyright notice and this permission
notice appear in supporting documentation, and that the name of
Secret Labs AB or the author not be used in advertising or publicity
pertaining to distribution of the software without specific, written
prior permission.

SECRET LABS AB AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH REGARD
TO THIS SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANT-
ABILITY AND FITNESS.  IN NO EVENT SHALL SECRET LABS AB OR THE AUTHOR
BE LIABLE FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY
DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE
OF THIS SOFTWARE.
```

C.3.8 test_epoll

O módulo `test.test_epoll` contém o seguinte aviso:

```
Copyright (c) 2001-2006 Twisted Matrix Laboratories.

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:
```

(continua na próxima página)

(continuação da página anterior)

```
The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

C.3.9 kqueue de seleção

O módulo *select* contém o seguinte aviso para a interface do kqueue:

```
Copyright (c) 2000 Doug White, 2006 James Knight, 2007 Christian Heimes
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:
1. Redistributions of source code must retain the above copyright
   notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
ARE DISCLAIMED.  IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
SUCH DAMAGE.
```

C.3.10 SipHash24

O arquivo `Python/pyhash.c` contém a implementação de Marek Majkowski do algoritmo SipHash24 de Dan Bernstein. Contém a seguinte nota:

```
<MIT License>
Copyright (c) 2013  Marek Majkowski <marek@popcount.org>

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.
```

(continua na próxima página)

(continuação da página anterior)

```
</MIT License>

Original location:
   https://github.com/majek/csiphash/

Solution inspired by code from:
   Samuel Neves (supercop/crypto_auth/siphash24/little)
   djb (supercop/crypto_auth/siphash24/little2)
   Jean-Philippe Aumasson (https://131002.net/siphash/siphash24.c)
```

C.3.11 strtod e dtoa

O arquivo `Python/dtoa.c`, que fornece as funções C dtoa e strtod para conversão de duplas de C para e de strings, é derivado do arquivo com o mesmo nome de David M. Gay, atualmente disponível em https://web.archive.org/web/20220517033456/http://www.netlib.org/fp/dtoa.c. O arquivo original, conforme recuperado em 16 de março de 2009, contém os seguintes avisos de direitos autorais e de licenciamento:

```
/****************************************************************
 *
 * The author of this software is David M. Gay.
 *
 * Copyright (c) 1991, 2000, 2001 by Lucent Technologies.
 *
 * Permission to use, copy, modify, and distribute this software for any
 * purpose without fee is hereby granted, provided that this entire notice
 * is included in all copies of any software which is or includes a copy
 * or modification of this software and in all copies of the supporting
 * documentation for such software.
 *
 * THIS SOFTWARE IS BEING PROVIDED "AS IS", WITHOUT ANY EXPRESS OR IMPLIED
 * WARRANTY.  IN PARTICULAR, NEITHER THE AUTHOR NOR LUCENT MAKES ANY
 * REPRESENTATION OR WARRANTY OF ANY KIND CONCERNING THE MERCHANTABILITY
 * OF THIS SOFTWARE OR ITS FITNESS FOR ANY PARTICULAR PURPOSE.
 *
 ***************************************************************/
```

C.3.12 OpenSSL

Os módulos *hashlib*, *posix* e *ssl* usam a biblioteca OpenSSL para desempenho adicional se forem disponibilizados pelo sistema operacional. Além disso, os instaladores do Windows e do Mac OS X para Python podem incluir uma cópia das bibliotecas do OpenSSL, portanto incluímos uma cópia da licença do OpenSSL aqui: Para o lançamento do OpenSSL 3.0, e lançamentos posteriores derivados deste, se aplica a Apache License v2:

```
                                 Apache License
                           Version 2.0, January 2004
                        https://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

   "License" shall mean the terms and conditions for use, reproduction,
   and distribution as defined by Sections 1 through 9 of this document.

   "Licensor" shall mean the copyright owner or entity authorized by
   the copyright owner that is granting the License.
```

(continua na próxima página)

(continuação da página anterior)

```
   "Legal Entity" shall mean the union of the acting entity and all
   other entities that control, are controlled by, or are under common
   control with that entity. For the purposes of this definition,
   "control" means (i) the power, direct or indirect, to cause the
   direction or management of such entity, whether by contract or
   otherwise, or (ii) ownership of fifty percent (50%) or more of the
   outstanding shares, or (iii) beneficial ownership of such entity.

   "You" (or "Your") shall mean an individual or Legal Entity
   exercising permissions granted by this License.

   "Source" form shall mean the preferred form for making modifications,
   including but not limited to software source code, documentation
   source, and configuration files.

   "Object" form shall mean any form resulting from mechanical
   transformation or translation of a Source form, including but
   not limited to compiled object code, generated documentation,
   and conversions to other media types.

   "Work" shall mean the work of authorship, whether in Source or
   Object form, made available under the License, as indicated by a
   copyright notice that is included in or attached to the work
   (an example is provided in the Appendix below).

   "Derivative Works" shall mean any work, whether in Source or Object
   form, that is based on (or derived from) the Work and for which the
   editorial revisions, annotations, elaborations, or other modifications
   represent, as a whole, an original work of authorship. For the purposes
   of this License, Derivative Works shall not include works that remain
   separable from, or merely link (or bind by name) to the interfaces of,
   the Work and Derivative Works thereof.

   "Contribution" shall mean any work of authorship, including
   the original version of the Work and any modifications or additions
   to that Work or Derivative Works thereof, that is intentionally
   submitted to Licensor for inclusion in the Work by the copyright owner
   or by an individual or Legal Entity authorized to submit on behalf of
   the copyright owner. For the purposes of this definition, "submitted"
   means any form of electronic, verbal, or written communication sent
   to the Licensor or its representatives, including but not limited to
   communication on electronic mailing lists, source code control systems,
   and issue tracking systems that are managed by, or on behalf of, the
   Licensor for the purpose of discussing and improving the Work, but
   excluding communication that is conspicuously marked or otherwise
   designated in writing by the copyright owner as "Not a Contribution."

   "Contributor" shall mean Licensor and any individual or Legal Entity
   on behalf of whom a Contribution has been received by Licensor and
   subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of
   this License, each Contributor hereby grants to You a perpetual,
   worldwide, non-exclusive, no-charge, royalty-free, irrevocable
   copyright license to reproduce, prepare Derivative Works of,
```

(continua na próxima página)

(continuação da página anterior)

```
   publicly display, publicly perform, sublicense, and distribute the
   Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of
   this License, each Contributor hereby grants to You a perpetual,
   worldwide, non-exclusive, no-charge, royalty-free, irrevocable
   (except as stated in this section) patent license to make, have made,
   use, offer to sell, sell, import, and otherwise transfer the Work,
   where such license applies only to those patent claims licensable
   by such Contributor that are necessarily infringed by their
   Contribution(s) alone or by combination of their Contribution(s)
   with the Work to which such Contribution(s) was submitted. If You
   institute patent litigation against any entity (including a
   cross-claim or counterclaim in a lawsuit) alleging that the Work
   or a Contribution incorporated within the Work constitutes direct
   or contributory patent infringement, then any patent licenses
   granted to You under this License for that Work shall terminate
   as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the
   Work or Derivative Works thereof in any medium, with or without
   modifications, and in Source or Object form, provided that You
   meet the following conditions:

   (a) You must give any other recipients of the Work or
       Derivative Works a copy of this License; and

   (b) You must cause any modified files to carry prominent notices
       stating that You changed the files; and

   (c) You must retain, in the Source form of any Derivative Works
       that You distribute, all copyright, patent, trademark, and
       attribution notices from the Source form of the Work,
       excluding those notices that do not pertain to any part of
       the Derivative Works; and

   (d) If the Work includes a "NOTICE" text file as part of its
       distribution, then any Derivative Works that You distribute must
       include a readable copy of the attribution notices contained
       within such NOTICE file, excluding those notices that do not
       pertain to any part of the Derivative Works, in at least one
       of the following places: within a NOTICE text file distributed
       as part of the Derivative Works; within the Source form or
       documentation, if provided along with the Derivative Works; or,
       within a display generated by the Derivative Works, if and
       wherever such third-party notices normally appear. The contents
       of the NOTICE file are for informational purposes only and
       do not modify the License. You may add Your own attribution
       notices within Derivative Works that You distribute, alongside
       or as an addendum to the NOTICE text from the Work, provided
       that such additional attribution notices cannot be construed
       as modifying the License.

   You may add Your own copyright statement to Your modifications and
   may provide additional or different license terms and conditions
   for use, reproduction, or distribution of Your modifications, or
```

(continua na próxima página)

(continuação da página anterior)

```
   for any such Derivative Works as a whole, provided Your use,
   reproduction, and distribution of the Work otherwise complies with
   the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise,
   any Contribution intentionally submitted for inclusion in the Work
   by You to the Licensor shall be under the terms and conditions of
   this License, without any additional terms or conditions.
   Notwithstanding the above, nothing herein shall supersede or modify
   the terms of any separate license agreement you may have executed
   with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade
   names, trademarks, service marks, or product names of the Licensor,
   except as required for reasonable and customary use in describing the
   origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or
   agreed to in writing, Licensor provides the Work (and each
   Contributor provides its Contributions) on an "AS IS" BASIS,
   WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
   implied, including, without limitation, any warranties or conditions
   of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A
   PARTICULAR PURPOSE. You are solely responsible for determining the
   appropriateness of using or redistributing the Work and assume any
   risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory,
   whether in tort (including negligence), contract, or otherwise,
   unless required by applicable law (such as deliberate and grossly
   negligent acts) or agreed to in writing, shall any Contributor be
   liable to You for damages, including any direct, indirect, special,
   incidental, or consequential damages of any character arising as a
   result of this License or out of the use or inability to use the
   Work (including but not limited to damages for loss of goodwill,
   work stoppage, computer failure or malfunction, or any and all
   other commercial damages or losses), even if such Contributor
   has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing
   the Work or Derivative Works thereof, You may choose to offer,
   and charge a fee for, acceptance of support, warranty, indemnity,
   or other liability obligations and/or rights consistent with this
   License. However, in accepting such obligations, You may act only
   on Your own behalf and on Your sole responsibility, not on behalf
   of any other Contributor, and only if You agree to indemnify,
   defend, and hold each Contributor harmless for any liability
   incurred by, or claims asserted against, such Contributor by reason
   of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS
```

C.3.13 expat

A extensão *pyexpat* é construída usando uma cópia incluída das fontes de expatriadas, a menos que a compilação esteja configurada `--with-system-expat`:

```
Copyright (c) 1998, 1999, 2000 Thai Open Source Software Center Ltd
                               and Clark Cooper

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

C.3.14 libffi

A extensão C `_ctypes` subjacente ao módulo *ctypes* é construída usando uma cópia incluída das fontes do libffi, a menos que a construção esteja configurada com `--with-system-libffi`:

```
Copyright (c) 1996-2008  Red Hat, Inc and others.

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT.  IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT
HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN THE SOFTWARE.
```

C.3.15 zlib

A extensão *zlib* é construída usando uma cópia incluída das fontes zlib se a versão do zlib encontrada no sistema for muito antiga para ser usada na construção:

```
Copyright (C) 1995-2011 Jean-loup Gailly and Mark Adler

This software is provided 'as-is', without any express or implied
warranty.  In no event will the authors be held liable for any damages
arising from the use of this software.

Permission is granted to anyone to use this software for any purpose,
including commercial applications, and to alter it and redistribute it
freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not
   claim that you wrote the original software. If you use this software
   in a product, an acknowledgment in the product documentation would be
   appreciated but is not required.

2. Altered source versions must be plainly marked as such, and must not be
   misrepresented as being the original software.

3. This notice may not be removed or altered from any source distribution.

Jean-loup Gailly        Mark Adler
jloup@gzip.org          madler@alumni.caltech.edu
```

C.3.16 cfuhash

A implementação da tabela de hash usada pelo *tracemalloc* é baseada no projeto cfuhash:

```
Copyright (c) 2005 Don Owens
All rights reserved.

This code is released under the BSD license:

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

  * Redistributions of source code must retain the above copyright
    notice, this list of conditions and the following disclaimer.

  * Redistributions in binary form must reproduce the above
    copyright notice, this list of conditions and the following
    disclaimer in the documentation and/or other materials provided
    with the distribution.

  * Neither the name of the author nor the names of its
    contributors may be used to endorse or promote products derived
    from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS
FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE
COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT,
```

(continua na próxima página)

(continuação da página anterior)

```
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED
OF THE POSSIBILITY OF SUCH DAMAGE.
```

C.3.17 libmpdec

A extensão C `_decimal` subjacente ao módulo *decimal* é construída usando uma cópia incluída da biblioteca libmpdec, a menos que a construção esteja configurada com `--with-system-libmpdec`:

```
Copyright (c) 2008-2020 Stefan Krah. All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

1. Redistributions of source code must retain the above copyright
   notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
ARE DISCLAIMED.  IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
SUCH DAMAGE.
```

C.3.18 Conjunto de testes C14N do W3C

O conjunto de testes C14N 2.0 no pacote *test* (`Lib/test/xmltestdata/c14n-20/`) foi recuperado do site do W3C em https://www.w3.org/TR/xml-c14n2-testcases/ e é distribuído sob a licença BSD de 3 cláusulas:

```
Copyright (c) 2013 W3C(R) (MIT, ERCIM, Keio, Beihang),
All Rights Reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

* Redistributions of works must retain the original copyright notice,
  this list of conditions and the following disclaimer.
* Redistributions in binary form must reproduce the original copyright
  notice, this list of conditions and the following disclaimer in the
  documentation and/or other materials provided with the distribution.
* Neither the name of the W3C nor the names of its contributors may be
```

(continua na próxima página)

(continuação da página anterior)

```
  used to endorse or promote products derived from this work without
  specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

C.3.19 mimalloc

Licença MIT:

```
Copyright (c) 2018-2021 Microsoft Corporation, Daan Leijen

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

C.3.20 asyncio

Partes do módulo *asyncio* são incorporadas do uvloop 0.16, que é distribuído sob a licença MIT:

```
Copyright (c) 2015-2021 MagicStack Inc.  http://magic.io

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.
```

(continua na próxima página)

(continuação da página anterior)

```
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

C.3.21 Global Unbounded Sequences (GUS)

O arquivo `Python/qsbr.c` é adaptado do esquema de recuperação de memória segura "Global Unbounded Sequences" do FreeBSD em subr_smr.c. O arquivo é distribuído sob a licença BSD de 2 cláusulas:

```
Copyright (c) 2019,2020 Jeffrey Roberson <jeff@FreeBSD.org>

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:
1. Redistributions of source code must retain the above copyright
   notice unmodified, this list of conditions, and the following
   disclaimer.
2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR "AS IS" AND ANY EXPRESS OR
IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES
OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED.
IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT,
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT
NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF
THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

C.3.22 Ligações Zstandard

As ligações de Zstandard em `Modules/_zstd` e `Lib/compression/zstd` são baseadas no código da biblioteca pyzstd, com direitos autorais de Ma Lin e colaboradores. O código do pyzstd é distribuído sob a Licença BSD de 3 Cláusulas:

```
Copyright (c) 2020-present, Ma Lin and contributors.
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this
   list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice,
   this list of conditions and the following disclaimer in the documentation
   and/or other materials provided with the distribution.

3. Neither the name of the copyright holder nor the names of its
```

(continua na próxima página)

(continuação da página anterior)

```
   contributors may be used to endorse or promote products derived from
   this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY,
OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

APÊNDICE D

Direitos autorais

Python e essa documentação é:

Copyright © 2001 Python Software Foundation. Todos os direitos reservados.

Copyright © 2000 BeOpen.com. Todos os direitos reservados.

Copyright © 1995-2000 Corporation for National Research Initiatives. Todos os direitos reservados.

Copyright © 1991-1995 Stichting Mathematisch Centrum. Todos os direitos reservados.

Veja: *História e Licença* para informações completas de licença e permissões.

Referências Bibliográficas

[Frie09] Friedl, Jeffrey. Mastering Regular Expressions. 3ª ed., O'Reilly Media, 2009. A terceira edição do livro não cobre mais o Python, mas a primeira edição cobriu a escrita de bons padrões de expressão regular em grandes detalhes.

[C99] ISO/IEC 9899:1999. "Programming languages – C." Um rascunho público desta norma está disponível em https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf.

Índice de Módulos Python

_

__future__, 2117
__main__, 2061
_thread, 1133
_tkinter, 1664

a

abc, 2101
aifc, 2337
annotationlib, 2143
argparse, 905
array, 308
ast, 2203
asynchat, 2337
asyncio, 1137
asyncore, 2337
atexit, 2106
audioop, 2338

b

base64, 1425
bdb, 1955
binascii, 1429
bisect, 305
builtins, 2061
bz2, 618

c

calendar, 267
cgi, 2338
cgitb, 2338
chunk, 2338
cmath, 370
cmd, 1015
code, 2155
codecs, 204
codeop, 2157
collections, 275
collections.abc, 294
colorsys, 1642
compileall, 2261
compression.zstd, 599
concurrent.futures, 1091
concurrent.interpreters, 1099
configparser, 667
contextlib, 2086
contextvars, 1128
copy, 326
copyreg, 554
cProfile, 1978
crypt, 2338
csv, 659
ctypes, 865
curses (*Unix*), 983
curses.ascii, 1010
curses.panel, 1014
curses.textpad, 1009

d

dataclasses, 2075
datetime, 223
dbm, 560
dbm.dumb, 565
dbm.gnu (*Unix*), 562
dbm.ndbm (*Unix*), 564
dbm.sqlite3 (*All*), 562
decimal, 374
difflib, 171
dis, 2265
distutils, 2338
doctest, 1814

e

email, 1333
email.charset, 1384
email.contentmanager, 1362
email.encoders, 1386
email.errors, 1355
email.generator, 1345
email.header, 1382
email.headerregistry, 1356
email.iterators, 1390
email.message, 1334
email.mime, 1379
email.mime.application, 1380
email.mime.audio, 1380
email.mime.base, 1379

email.mime.image, 1380
email.mime.message, 1381
email.mime.multipart, 1379
email.mime.nonmultipart, 1379
email.mime.text, 1381
email.parser, 1342
email.policy, 1348
email.utils, 1387
encodings, 220
encodings.idna, 221
encodings.mbcs, 222
encodings.utf_8_sig, 222
ensurepip, 2003
enum, 336
errno, 857

f

faulthandler, 1961
fcntl (*Unix*), 2319
filecmp, 512
fileinput, 981
fnmatch, 522
fractions, 404
ftplib, 1550
functools, 452

g

gc, 2119
getopt, 2333
getpass, 980
gettext, 1643
glob, 520
graphlib, 352
grp (*Unix*), 2314
gzip, 615

h

hashlib, 693
heapq, 300
hmac, 705
html, 1433
html.entities, 1438
html.parser, 1434
http, 1539
http.client, 1543
http.cookiejar, 1600
http.cookies, 1597
http.server, 1589

i

idlelib, 1712
imaplib, 1560
imghdr, 2339
imp, 2339
importlib, 2168
importlib.abc, 2170
importlib.machinery, 2177
importlib.metadata, 2193
importlib.resources, 2188
importlib.resources.abc, 2191
importlib.util, 2183
inspect, 2123
io, 780
ipaddress, 1623
itertools, 435

j

json, 1391
json.tool, 1400

k

keyword, 2252

l

linecache, 524
locale, 1651
logging, 808
logging.config, 826
logging.handlers, 838
lzma, 623

m

mailbox, 1401
mailcap, 2339
marshal, 558
math, 360
mimetypes, 1420
mmap, 1326
modulefinder, 2164
msilib, 2339
msvcrt (*Windows*), 2291
multiprocessing, 1038
multiprocessing.connection, 1071
multiprocessing.dummy, 1075
multiprocessing.managers, 1062
multiprocessing.pool, 1068
multiprocessing.shared_memory, 1084
multiprocessing.sharedctypes, 1060

n

netrc, 688
nis, 2339
nntplib, 2339
numbers, 357

o

operator, 463
optparse, 951
os, 711
os.path, 499
ossaudiodev, 2340

p

pathlib, 471
pathlib.types, 498
pdb, 1964

pickle, 537
pickletools, 2288
pipes, 2340
pkgutil, 2161
platform, 852
plistlib, 689
poplib, 1557
posix (*Unix*), 2312
pprint, 327
profile, 1978
pstats, 1979
pty (*Unix*), 2317
pwd (*Unix*), 2313
py_compile, 2259
pyclbr, 2257
pydoc, 1809

q

queue, 1125
quopri, 1431

r

random, 407
re, 147
readline (*Unix*), 191
reprlib, 333
resource (*Unix*), 2322
rlcompleter, 196
runpy, 2165

s

sched, 1123
secrets, 706
select, 1306
selectors, 1312
shelve, 555
shlex, 2307
shutil, 524
signal, 1316
site, 2151
sitecustomize, 2153
smtpd, 2340
smtplib, 1568
sndhdr, 2340
socket, 1241
socketserver, 1580
spwd, 2340
sqlite3, 566
ssl, 1270
stat, 506
statistics, 418
string, 129
string.templatelib, 142
stringprep, 189
struct, 197
subprocess, 1104
sunau, 2341
symtable, 2243
sys, 2021
sys.monitoring, 2049
sysconfig, 2054
syslog (*Unix*), 2326

t

tabnanny, 2257
tarfile, 641
telnetlib, 2341
tempfile, 514
termios (*Unix*), 2315
test, 1930
test.regrtest, 1932
test.support, 1933
test.support.bytecode_helper, 1944
test.support.import_helper, 1947
test.support.os_helper, 1946
test.support.script_helper, 1943
test.support.socket_helper, 1942
test.support.threading_helper, 1945
test.support.warnings_helper, 1949
textwrap, 183
threading, 1021
time, 795
timeit, 1983
tkinter, 1661
tkinter.colorchooser (*Tk*), 1674
tkinter.commondialog (*Tk*), 1679
tkinter.dnd (*Tk*), 1682
tkinter.filedialog (*Tk*), 1676
tkinter.font (*Tk*), 1675
tkinter.messagebox (*Tk*), 1679
tkinter.scrolledtext (*Tk*), 1681
tkinter.simpledialog (*Tk*), 1676
tkinter.ttk, 1682
token, 2247
tokenize, 2253
tomllib, 686
trace, 1988
traceback, 2107
tracemalloc, 1991
tty (*Unix*), 2317
turtle, 1712
turtledemo, 1749
types, 319
typing, 1753

u

unicodedata, 187
unittest, 1837
unittest.mock, 1869
urllib, 1510
urllib.error, 1537
urllib.parse, 1528
urllib.request, 1511
urllib.response, 1527
urllib.robotparser, 1538
usercustomize, 2153

uu, 2341
uuid, 1574

V

venv, 2005

W

warnings, 2066
wave, 1639
weakref, 311
webbrowser, 1497
winreg (*Windows*), 2294
winsound (*Windows*), 2303
wsgiref, 1500
wsgiref.handlers, 1505
wsgiref.headers, 1502
wsgiref.simple_server, 1503
wsgiref.types, 1509
wsgiref.util, 1500
wsgiref.validate, 1504

X

xdrlib, 2341
xml, 1439
xml.dom, 1458
xml.dom.minidom, 1468
xml.dom.pulldom, 1472
xml.etree.ElementInclude, 1451
xml.etree.ElementTree, 1440
xml.parsers.expat, 1486
xml.parsers.expat.errors, 1493
xml.parsers.expat.model, 1493
xml.sax, 1474
xml.sax.handler, 1476
xml.sax.saxutils, 1481
xml.sax.xmlreader, 1482
xmlrpc, 1609
xmlrpc.client, 1609
xmlrpc.server, 1617

Z

zipapp, 2015
zipfile, 629
zipimport, 2159
zlib, 611
zoneinfo, 262

Não alfabético

?? em expressões regulares, 148
.. em nomes de caminhos, 778
..., **2345** espaço reservado, 187, 329, 333 in doctests, 1822 literal de reticências, 1756, 1758, 1770, 2345 prompt do interpretador, 1819, 2041 reticências literais, 38, 111
! (*exclamação*) caracteres curingas no estilo blog, 520, 522 em um interpretador de comandos, 1016 in struct format strings, 198 na formatação de strings, 131 no módulo curses, 1014
? (*interrogação*) caracteres curingas no estilo blog, 520, 522 em expressões regulares, 148 em gramática de AST, 2206 em instruções SQL, 583 em um interpretador de comandos, 1016 in struct format strings, 200, 201 no módulo argparse, 918 replacement character, 208
- (*menos*) caracteres curingas no estilo blog, 520, 522 em expressões regulares, 149 em formatação no estilo printf, 69, 86 in doctests, 1823 na formatação de strings, 133 operador binário, 41 operador unário, 41
! (*pdb command*), 1973
! (*ponto de exclamação*) em literal de string formatado, 67
. (*ponto*) caracteres curingas no estilo blog, 520 em expressões regulares, 148 em formatação no estilo printf, 69, 85 em nomes de caminhos, 778, 779 na formatação de strings, 131
! f-string, 67
(*cerquilha*) comentário, 2152 em expressões regulares, 156 em formatação no estilo printf, 69, 86 in doctests, 1823 na formatação de strings, 134
$ (*cifrão*) em expressões regulares, 148
$ (*dólar*) em strings template, 139 expansão de variáveis de ambiente, 501 interpolação em arquivos de configuração, 672
% (*porcentagem*) datetime, formato, 258, 800, 802 expansão de variáveis de ambiente (*Windows*), 501, 2296 interpolação em arquivos de configuração, 672 operador, 41 printf, estilo de formatação, 69, 85
& (*e comercial*) operador, 42
(? em expressões regulares, 150
(?! em expressões regulares, 152
(?# em expressões regulares, 152
(?(em expressões regulares, 152
() (*parênteses*) em expressões regulares, 150 em formatação no estilo printf, 69, 85
(?: em expressões regulares, 150
(?<! em expressões regulares, 152

(?<=
    em expressões regulares, 152
(?=
    em expressões regulares, 152
(?P<
    em expressões regulares, 151
(?P=
    em expressões regulares, 151
*?
    em expressões regulares, 148
* (*asterisco*)
    caracteres curingas no estilo blog, 520, 522
    em expressões regulares, 148
    em formatação no estilo printf, 69, 85
    em gramática de AST, 2206
    no módulo argparse, 918
    operador, 41
**
    caracteres curingas no estilo blog, 520
    operador, 41
*+
    em expressões regulares, 148
+?
    em expressões regulares, 148
?+
    em expressões regulares, 148
+ (*mais*)
    em expressões regulares, 148
    em formatação no estilo printf, 69, 86
    in doctests, 1823
    na formatação de strings, 133
    no módulo argparse, 918
    operador binário, 41
    operador unário, 41
++
    em expressões regulares, 148
, (*vírgula*)
    na formatação de strings, 134
-
    idle opção de linha de comando, 1709
    python--m-py_compile opção de linha de comando, 2261
/ (*barra*)
    em nomes de caminhos, 778, 779
    operador, 41
//
    operador, 41
: (*dois pontos*)
    em instruções SQL, 583
    em literal de string formatado, 67
    na formatação de strings, 131
    separador de caminho (*POSIX*), 779
; (*ponto e vírgula*), 779
< (*menor que*)
    in struct format strings, 198
    na formatação de strings, 133
    operador, 40
<<
    operador, 42
<=
    operador, 40
<BLANKLINE>, 1821
<file>
    python--m-py_compile opção de linha de comando, 2261
!=
    operador, 40
= (*igual*)
    in struct format strings, 198
    na formatação de strings, 133
    para ajudar na depuração usando literais de string, 67
==
    operador, 40
> (*maior que*)
    operador, 40
> (*maior*)
    in struct format strings, 198
    na formatação de strings, 133
>=
    operador, 40
>>
    operador, 42
>>>, **2345**
    prompt do interpretador, 1819, 2041
@ (*arroba*)
    in struct format strings, 198
[] (*colchetes*)
    caracteres curingas no estilo blog, 520, 522
    em expressões regulares, 149
    na formatação de strings, 131
\ (*contrabarra*)
    em expressões regulares, 149, 152
    em nomes de caminhos (*Windows*), 778
    sequência de escape, 208
\\
    em expressões regulares, 154
\A
    em expressões regulares, 153
\a
    em expressões regulares, 154
\B
    em expressões regulares, 153
\b
    em expressões regulares, 153, 154
\D
    em expressões regulares, 153
\d
    em expressões regulares, 153
\f
    em expressões regulares, 154
\g
    em expressões regulares, 159

\N
    em expressões regulares, 154
    sequência de escape, 208
\n
    em expressões regulares, 154
\r
    em expressões regulares, 154
\S
    em expressões regulares, 153
\s
    em expressões regulares, 153
\t
    em expressões regulares, 154
\U
    em expressões regulares, 154
    sequência de escape, 208
\u
    em expressões regulares, 154
    sequência de escape, 208
\v
    em expressões regulares, 154
\W
    em expressões regulares, 154
\w
    em expressões regulares, 153
\x
    em expressões regulares, 154
    sequência de escape, 208
\Z
    em expressões regulares, 154
\z
    em expressões regulares, 154
^ (*circunflexo*)
    em expressões regulares, 148, 149
    marker, 1821, 2108
    na formatação de strings, 133
    no módulo curses, 1014
    operador, 42
_ (*sublinhado*)
    gettext, 1644
_ (*underscore*)
    na formatação de strings, 134
__abs__() (*no módulo operator*), 464
__add__() (*no módulo operator*), 464
__and__() (*método enum.Flag*), 346
__and__() (*no módulo operator*), 464
__args__ (*atributo genericalias*), 107
__bound__ (*atributo typing.TypeVar*), 1780
__breakpointhook__ (*no módulo sys*), 2026
__bytes__() (*método email.message.EmailMessage*), 1335
__bytes__() (*método email.message.Message*), 1372
__call__() (*método argparse.Action*), 924
__call__() (*método email.headerregistry.HeaderRegistry*), 1361
__call__() (*método enum.EnumType*), 338
__call__() (*método weakref.finalize*), 315
__call__() (*no módulo operator*), 466
__callback__ (*atributo weakref.ref*), 313
__cause__ (*atributo BaseException*), 115
__cause__ (*atributo de exceção*), 115
__cause__ (*atributo traceback.TracebackException*), 2110
__ceil__() (*método fractions.Fraction*), 406
__class__ (*atributo unittest.mock.Mock*), 1879
__code__ (*atributo de objeto função*), 111
__concat__() (*no módulo operator*), 465
__constraints__ (*atributo typing.TypeVar*), 1780
__contains__() (*método email.message.EmailMessage*), 1336
__contains__() (*método email.message.Message*), 1374
__contains__() (*método enum.EnumType*), 339
__contains__() (*método enum.Flag*), 345
__contains__() (*método mailbox.Mailbox*), 1404
__contains__() (*no módulo operator*), 466
__context__ (*atributo BaseException*), 115
__context__ (*atributo de exceção*), 115
__context__ (*atributo traceback.TracebackException*), 2110
__contravariant__ (*atributo typing.TypeVar*), 1780
__copy__() (*protocolo de cópia*), 327
__covariant__ (*atributo typing.TypeVar*), 1780
__debug__ (*variável interna*), 38
__deepcopy__() (*protocolo de cópia*), 327
__default__ (*atributo typing.ParamSpec*), 1784
__default__ (*atributo typing.TypeVar*), 1780
__default__ (*atributo typing.TypeVarTuple*), 1782
__del__() (*método io.IOBase*), 786
__delitem__() (*método email.message.EmailMessage*), 1336
__delitem__() (*método email.message.Message*), 1374
__delitem__() (*método mailbox.Mailbox*), 1402
__delitem__() (*método mailbox.MH*), 1409
__delitem__() (*no módulo operator*), 466
__dir__() (*método enum.Enum*), 340
__dir__() (*método enum.EnumType*), 339
__dir__() (*método unittest.mock.Mock*), 1875
__displayhook__ (*no módulo sys*), 2026
__doc__ (*atributo definition*), 112
__enter__() (*método contextmanager*), 103
__enter__() (*método winreg.PyHKEY*), 2302
__eq__() (*método de instância*), 40
__eq__() (*método email.charset.Charset*), 1386
__eq__() (*método email.header.Header*), 1383
__eq__() (*método memoryview*), 89
__eq__() (*no módulo operator*), 463
__excepthook__ (*no módulo sys*), 2026
__excepthook__ (*no módulo threading*), 1023
__exit__() (*método contextmanager*), 103
__exit__() (*método winreg.PyHKEY*), 2302
__floor__() (*método fractions.Fraction*), 406
__floordiv__() (*no módulo operator*), 464
__format__, 17
__format__() (*método datetime.date*), 233

__format__() (*método datetime.datetime*), 244
__format__() (*método datetime.time*), 249
__format__() (*método enum.Enum*), 343
__format__() (*método fractions.Fraction*), 406
__format__() (*método ipaddress.IPv4Address*), 1626
__format__() (*método ipaddress.IPv6Address*), 1627
__forward_arg__ (*atributo annotationlib.ForwardRef*), 2145
__fspath__() (*método os.PathLike*), 715
__future__, **2352**
  module, 2117
__ge__() (*método de instância*), 40
__ge__() (*no módulo operator*), 463
__getitem__() (*método email.headerregistry.HeaderRegistry*), 1360
__getitem__() (*método email.message.EmailMessage*), 1336
__getitem__() (*método email.message.Message*), 1374
__getitem__() (*método enum.EnumType*), 339
__getitem__() (*método mailbox.Mailbox*), 1403
__getitem__() (*método re.Match*), 163
__getitem__() (*no módulo operator*), 466
__getnewargs__() (*método object*), 544
__getnewargs_ex__() (*método object*), 544
__getstate__() (*método object*), 544
__getstate__() (*protocolo de cópia*), 549
__gt__() (*método de instância*), 40
__gt__() (*no módulo operator*), 463
__iadd__() (*no módulo operator*), 469
__iand__() (*no módulo operator*), 469
__iconcat__() (*no módulo operator*), 469
__ifloordiv__() (*no módulo operator*), 469
__ilshift__() (*no módulo operator*), 469
__imatmul__() (*no módulo operator*), 469
__imod__() (*no módulo operator*), 469
__import__()
  built-in function, 34
__import__() (*no módulo importlib*), 2169
__imul__() (*no módulo operator*), 469
__index__() (*no módulo operator*), 464
__infer_variance__ (*atributo typing.TypeVar*), 1780
__init__() (*método asyncio.Future*), 1230
__init__() (*método asyncio.Task*), 1230
__init__() (*método difflib.HtmlDiff*), 171
__init__() (*método enum.Enum*), 341
__init__() (*método logging.Handler*), 814
__init_subclass__() (*método enum.Enum*), 341
__interactivehook__ (*no módulo sys*), 2036
__inv__() (*no módulo operator*), 464
__invert__() (*no módulo operator*), 464
__ior__() (*no módulo operator*), 470
__ipow__() (*no módulo operator*), 470
__irshift__() (*no módulo operator*), 470
__isub__() (*no módulo operator*), 470
__iter__() (*método container*), 48
__iter__() (*método enum.EnumType*), 339
__iter__() (*método iterator*), 48
__iter__() (*método mailbox.Mailbox*), 1403
__iter__() (*método unittest.TestSuite*), 1859
__itruediv__() (*no módulo operator*), 470
__ixor__() (*no módulo operator*), 470
__le__() (*método de instância*), 40
__le__() (*no módulo operator*), 463
__len__() (*método email.message.EmailMessage*), 1336
__len__() (*método email.message.Message*), 1374
__len__() (*método enum.EnumType*), 339
__len__() (*método mailbox.Mailbox*), 1404
__lshift__() (*no módulo operator*), 465
__lt__() (*método de instância*), 40
__lt__() (*no módulo operator*), 463
__main__
  module, 2061
  módulo, 2166
__matmul__() (*no módulo operator*), 465
__members__ (*atributo enum.EnumType*), 339
__missing__(), 99
__missing__() (*método collections.defaultdict*), 285
__mod__() (*no módulo operator*), 465
__module__ (*atributo definition*), 112
__module__ (*atributo typing.NewType*), 1788
__module__ (*atributo typing.TypeAliasType*), 1785
__mul__() (*no módulo operator*), 465
__mutable_keys__ (*atributo typing.TypedDict*), 1793
__name__ (*atributo definition*), 112
__name__ (*atributo property*), 28
__name__ (*atributo typing.NewType*), 1788
__name__ (*atributo typing.ParamSpec*), 1784
__name__ (*atributo typing.TypeAliasType*), 1785
__name__ (*atributo typing.TypeVar*), 1780
__name__ (*atributo typing.TypeVarTuple*), 1782
__ne__() (*método de instância*), 40
__ne__() (*método email.charset.Charset*), 1386
__ne__() (*método email.header.Header*), 1383
__ne__() (*no módulo operator*), 463
__neg__() (*no módulo operator*), 465
__new__() (*método enum.Enum*), 342
__new__() (*método string.templatelib.Interpolation*), 146
__new__() (*método string.templatelib.Template*), 144
__next__() (*método csv.csvreader*), 665
__next__() (*método iterator*), 48
__not__() (*no módulo operator*), 464
__notes__ (*atributo BaseException*), 116
__notes__ (*atributo traceback.TracebackException*), 2111
__optional_keys__ (*atributo typing.TypedDict*), 1792
__or__() (*método enum.Flag*), 345
__or__() (*no módulo operator*), 465
__origin__ (*atributo genericalias*), 107
__parameters__ (*atributo genericalias*), 107
__pointer_type__ (*atributo ctypes._CData*), 896
__pos__() (*no módulo operator*), 465

__post_init__() (*no módulo dataclasses*), 2082
__pow__() (*no módulo operator*), 465
__qualname__ (*atributo definition*), 112
__readonly_keys__ (*atributo typing.TypedDict*), 1793
__reduce__() (*método object*), 545
__reduce_ex__() (*método object*), 546
__replace__() (*protocolo de substituição*), 327
__repr__() (*método enum.Enum*), 342
__repr__() (*método multiprocessing.managers.BaseProxy*), 1068
__repr__() (*método netrc.netrc*), 689
__required_keys__ (*atributo typing.TypedDict*), 1792
__reversed__() (*método enum.EnumType*), 340
__round__() (*método fractions.Fraction*), 406
__rshift__() (*no módulo operator*), 465
__setitem__() (*método email.message.EmailMessage*), 1336
__setitem__() (*método email.message.Message*), 1374
__setitem__() (*método mailbox.Mailbox*), 1402
__setitem__() (*método mailbox.Maildir*), 1407
__setitem__() (*no módulo operator*), 466
__setstate__() (*método object*), 545
__setstate__() (*protocolo de cópia*), 549
__slots__, **2360**
__stderr__ (*no módulo sys*), 2046
__stdin__ (*no módulo sys*), 2046
__stdout__ (*no módulo sys*), 2046
__str__() (*método datetime.date*), 233
__str__() (*método datetime.datetime*), 244
__str__() (*método datetime.time*), 249
__str__() (*método email.charset.Charset*), 1386
__str__() (*método email.header.Header*), 1383
__str__() (*método email.headerregistry.Address*), 1361
__str__() (*método email.headerregistry.Group*), 1362
__str__() (*método email.message.EmailMessage*), 1335
__str__() (*método email.message.Message*), 1372
__str__() (*método enum.Enum*), 342
__str__() (*método multiprocessing.managers.BaseProxy*), 1068
__sub__() (*no módulo operator*), 465
__subclasshook__() (*método abc.ABCMeta*), 2102
__supertype__ (*atributo typing.NewType*), 1788
__suppress_context__ (*atributo BaseException*), 115
__suppress_context__ (*atributo de exceção*), 115
__suppress_context__ (*atributo traceback.TracebackException*), 2111
__test__ (*atributo module*), 1818
__total__ (*atributo typing.TypedDict*), 1792
__traceback__ (*atributo BaseException*), 116
__truediv__() (*método importlib.abc.Traversable*), 2176
__truediv__() (*método importlib.resources.abc.Traversable*), 2193
__truediv__() (*no módulo operator*), 465
__type_params__ (*atributo definition*), 112
__type_params__ (*atributo typing.TypeAliasType*), 1785
__unpacked__ (*atributo genericalias*), 107
__unraisablehook__ (*no módulo sys*), 2026
__value__ (*atributo typing.TypeAliasType*), 1786
__version__ (*no módulo curses*), 997
__xor__() (*método enum.Flag*), 346
__xor__() (*no módulo operator*), 465
_add_alias_() (*método enum.Enum*), 343
_add_value_alias_() (*método enum.Enum*), 343
align (*atributo ctypes.Structure*), 900
anonymous (*atributo ctypes.Structure*), 900
_asdict() (*método collections.somenamedtuple*), 288
_b_base_ (*atributo ctypes._CData*), 896
_b_needsfree_ (*atributo ctypes._CData*), 896
_callmethod() (*método multiprocessing.managers.BaseProxy*), 1067
_CData (*classe em ctypes*), 895
_CFuncPtr (*classe em ctypes*), 888
_clear_internal_caches() (*no módulo sys*), 2023
_clear_type_cache() (*no módulo sys*), 2023
_current_exceptions() (*no módulo sys*), 2023
_current_frames() (*no módulo sys*), 2023
_debugmallocstats() (*no módulo sys*), 2024
_emscripten_info (*no módulo sys*), 2025
_enablelegacywindowsfsencoding() (*no módulo sys*), 2045
_enter_task() (*no módulo asyncio*), 1230
_exit() (*no módulo os*), 763
_Feature (*classe em __future__*), 2118
_field_defaults (*atributo collections.somenamedtuple*), 289
_field_types (*atributo ast.AST*), 2207
_fields (*atributo ast.AST*), 2206
_fields (*atributo collections.somenamedtuple*), 289
fields (*atributo ctypes.Structure*), 899
_flush() (*método wsgiref.handlers.BaseHandler*), 1506
_for_archive() (*método zipfile.ZipInfo*), 630
_generate_next_value_() (*método enum.Enum*), 341
_get_child_mock() (*método unittest.mock.Mock*), 1875
_get_preferred_schemes() (*no módulo sysconfig*), 2058
_getframe() (*no módulo sys*), 2033
_getframemodulename() (*no módulo sys*), 2033
_getvalue() (*método multiprocessing.managers.BaseProxy*), 1068
_handle (*atributo ctypes.PyDLL*), 887
ignore (*atributo enum.Enum*), 340
_incompatible_extension_module_restrictions() (*no módulo importlib.util*), 2184
_is_gil_enabled() (*no módulo sys*), 2037

_is_immortal() (*no módulo sys*), 2038
_is_interned() (*no módulo sys*), 2038
_jit (*no módulo sys*), 2037
_jit.is_active() (*no módulo sys*), 2037
_jit.is_available() (*no módulo sys*), 2037
_jit.is_enabled() (*no módulo sys*), 2037
layout (*atributo ctypes.Structure*), 900
_leave_task() (*no módulo asyncio*), 1230
length (*atributo ctypes.Array*), 902
_locale
    módulo, 1651
_log (*atributo logging.LoggerAdapter*), 821
_make() (*método de classe collections.somenamedtuple*), 288
_makeResult() (*método unittest.TextTestRunner*), 1865
missing() (*método enum.Enum*), 341
_name (*atributo ctypes.PyDLL*), 887
name (*atributo enum.Enum*), 340
_numeric_repr_() (*método enum.Flag*), 346
_objects (*atributo ctypes._CData*), 896
order (*atributo enum.Enum*), 340
pack (*atributo ctypes.Structure*), 900
_parse() (*método gettext.NullTranslations*), 1645
_Pointer (*classe em ctypes*), 902
_register_task() (*no módulo asyncio*), 1230
_replace() (*método collections.somenamedtuple*), 288
_setroot() (*método xml.etree.ElementTree.ElementTree*), 1453
_SimpleCData (*classe em ctypes*), 896
_structure() (*no módulo email.iterators*), 1390
_thread
    module, 1133
_tkinter
    module, 1664
type (*atributo ctypes._Pointer*), 903
type (*atributo ctypes.Array*), 902
_unregister_task() (*no módulo asyncio*), 1230
value (*atributo enum.Enum*), 340
_write() (*método wsgiref.handlers.BaseHandler*), 1506
_xoptions (*no módulo sys*), 2049
{} (*chaves*)
    em expressões regulares, 148
    em literal de string formatado, 67
    na formatação de strings, 131
| (*barra vertical*)
    em expressões regulares, 150
    operador, 42
~ (*til*)
    expansão de diretório home, 500
    operador, 42
сертифікати, 1296

A

-a
    ast opção de linha de comando, 2242
    pickletools opção de linha de comando, 2289
A (*no módulo re*), 155
a2b_base64() (*no módulo binascii*), 1429
a2b_hex() (*no módulo binascii*), 1430
a2b_qp() (*no módulo binascii*), 1430
a2b_uu() (*no módulo binascii*), 1429
a85decode() (*no módulo base64*), 1427
a85encode() (*no módulo base64*), 1427
A_ALTCHARSET (*no módulo curses*), 999
A_ATTRIBUTES (*no módulo curses*), 1000
A_BLINK (*no módulo curses*), 999
A_BOLD (*no módulo curses*), 999
A_CHARTEXT (*no módulo curses*), 1000
A_COLOR (*no módulo curses*), 1000
A_DIM (*no módulo curses*), 999
A_HORIZONTAL (*no módulo curses*), 999
A_INVIS (*no módulo curses*), 999
A_ITALIC (*no módulo curses*), 999
A_LEFT (*no módulo curses*), 999
A_LOW (*no módulo curses*), 999
A_NORMAL (*no módulo curses*), 999
A_PROTECT (*no módulo curses*), 999
A_REVERSE (*no módulo curses*), 999
A_RIGHT (*no módulo curses*), 999
A_STANDOUT (*no módulo curses*), 999
A_TOP (*no módulo curses*), 999
A_UNDERLINE (*no módulo curses*), 999
A_VERTICAL (*no módulo curses*), 999
abc
    module, 2101
ABC (*classe em abc*), 2101
ABCMeta (*classe em abc*), 2101
ABDAY_1 (*no módulo locale*), 1653
ABDAY_2 (*no módulo locale*), 1653
ABDAY_3 (*no módulo locale*), 1653
ABDAY_4 (*no módulo locale*), 1653
ABDAY_5 (*no módulo locale*), 1653
ABDAY_6 (*no módulo locale*), 1653
ABDAY_7 (*no módulo locale*), 1653
abiflags (*no módulo sys*), 2021
ABMON_1 (*no módulo locale*), 1654
ABMON_2 (*no módulo locale*), 1654
ABMON_3 (*no módulo locale*), 1654
ABMON_4 (*no módulo locale*), 1654
ABMON_5 (*no módulo locale*), 1654
ABMON_6 (*no módulo locale*), 1654
ABMON_7 (*no módulo locale*), 1654
ABMON_8 (*no módulo locale*), 1654
ABMON_9 (*no módulo locale*), 1654
ABMON_10 (*no módulo locale*), 1654
ABMON_11 (*no módulo locale*), 1654
ABMON_12 (*no módulo locale*), 1654
ABORT (*no módulo tkinter.messagebox*), 1680
abort() (*método asyncio.Barrier*), 1174
abort() (*método asyncio.DatagramTransport*), 1217
abort() (*método asyncio.WriteTransport*), 1216
abort() (*método ftplib.FTP*), 1552

abort() (*método threading.Barrier*), 1037
abort() (*no módulo os*), 762
abort_clients() (*método asyncio.Server*), 1205
ABORTRETRYIGNORE (*no módulo tkinter.messagebox*), 1681
above() (*método curses.panel.Panel*), 1014
ABOVE_NORMAL_PRIORITY_CLASS (*no módulo subprocess*), 1117
abs()
    built-in function, 8
abs() (*método decimal.Context*), 389
abs() (*no módulo operator*), 464
absolute() (*método pathlib.Path*), 485
absolute_import (*no módulo __future__*), 2118
AbsoluteLinkError, 644
AbsolutePathError, 643
abspath() (*no módulo os.path*), 499
AbstractAsyncContextManager (*classe em contextlib*), 2086
AbstractBasicAuthHandler (*classe em urllib.request*), 1515
abstractclassmethod() (*no módulo abc*), 2104
AbstractContextManager (*classe em contextlib*), 2086
AbstractDigestAuthHandler (*classe em urllib.request*), 1515
AbstractEventLoop (*classe em asyncio*), 1207
AbstractEventLoopPolicy (*classe em asyncio*), 1227
abstractmethod() (*no módulo abc*), 2103
abstractproperty() (*no módulo abc*), 2104
AbstractSet (*classe em typing*), 1805
abstractstaticmethod() (*no módulo abc*), 2104
accept() (*método multiprocessing.connection.Listener*), 1072
accept() (*método socket.socket*), 1259
access() (*no módulo os*), 735
accumulate() (*no módulo itertools*), 436
ACK (*no módulo curses.ascii*), 1011
aclose() (*método contextlib.AsyncExitStack*), 2095
aclosing() (*no módulo contextlib*), 2089
acos() (*no módulo cmath*), 372
acos() (*no módulo math*), 367
acosh() (*no módulo cmath*), 372
acosh() (*no módulo math*), 368
acquire() (*método _thread.lock*), 1134
acquire() (*método asyncio.Condition*), 1171
acquire() (*método asyncio.Lock*), 1170
acquire() (*método asyncio.Semaphore*), 1172
acquire() (*método logging.Handler*), 814
acquire() (*método multiprocessing.Lock*), 1057
acquire() (*método multiprocessing.RLock*), 1058
acquire() (*método threading.Condition*), 1033
acquire() (*método threading.Lock*), 1030
acquire() (*método threading.RLock*), 1031
acquire() (*método threading.Semaphore*), 1034
ACS_BBSS (*no módulo curses*), 1006
ACS_BLOCK (*no módulo curses*), 1006
ACS_BOARD (*no módulo curses*), 1006
ACS_BSBS (*no módulo curses*), 1006
ACS_BSSB (*no módulo curses*), 1006
ACS_BSSS (*no módulo curses*), 1006
ACS_BTEE (*no módulo curses*), 1006
ACS_BULLET (*no módulo curses*), 1006
ACS_CKBOARD (*no módulo curses*), 1006
ACS_DARROW (*no módulo curses*), 1006
ACS_DEGREE (*no módulo curses*), 1006
ACS_DIAMOND (*no módulo curses*), 1006
ACS_GEQUAL (*no módulo curses*), 1006
ACS_HLINE (*no módulo curses*), 1006
ACS_LANTERN (*no módulo curses*), 1006
ACS_LARROW (*no módulo curses*), 1006
ACS_LEQUAL (*no módulo curses*), 1006
ACS_LLCORNER (*no módulo curses*), 1006
ACS_LRCORNER (*no módulo curses*), 1007
ACS_LTEE (*no módulo curses*), 1007
ACS_NEQUAL (*no módulo curses*), 1007
ACS_PI (*no módulo curses*), 1007
ACS_PLMINUS (*no módulo curses*), 1007
ACS_PLUS (*no módulo curses*), 1007
ACS_RARROW (*no módulo curses*), 1007
ACS_RTEE (*no módulo curses*), 1007
ACS_S1 (*no módulo curses*), 1007
ACS_S3 (*no módulo curses*), 1007
ACS_S7 (*no módulo curses*), 1007
ACS_S9 (*no módulo curses*), 1007
ACS_SBBS (*no módulo curses*), 1007
ACS_SBSB (*no módulo curses*), 1007
ACS_SBSS (*no módulo curses*), 1007
ACS_SSBB (*no módulo curses*), 1007
ACS_SSBS (*no módulo curses*), 1007
ACS_SSSB (*no módulo curses*), 1007
ACS_SSSS (*no módulo curses*), 1008
ACS_STERLING (*no módulo curses*), 1008
ACS_TTEE (*no módulo curses*), 1008
ACS_UARROW (*no módulo curses*), 1008
ACS_ULCORNER (*no módulo curses*), 1008
ACS_URCORNER (*no módulo curses*), 1008
ACS_VLINE (*no módulo curses*), 1008
action (*atributo optparse.Option*), 966
Action (*classe em argparse*), 924
ACTIONS (*atributo optparse.Option*), 978
activate_stack_trampoline() (*no módulo sys*), 2044
active_children() (*no módulo multiprocessing*), 1053
active_count() (*no módulo threading*), 1023
actual() (*método tkinter.font.Font*), 1675
Add (*classe em ast*), 2213
add() (*método decimal.Context*), 390
add() (*método frozenset*), 97
add() (*método graphlib.TopologicalSorter*), 353
add() (*método mailbox.Mailbox*), 1402
add() (*método mailbox.Maildir*), 1407
add() (*método pstats.Stats*), 1980
add() (*método tarfile.TarFile*), 648

add() (*método tkinter.ttk.Notebook*), 1689
add() (*no módulo operator*), 464
add_alias() (*no módulo email.charset*), 1386
add_alternative() (*método email.message.EmailMessage*), 1341
add_argument() (*método argparse.ArgumentParser*), 914
add_argument_group() (*método argparse.ArgumentParser*), 932
add_attachment() (*método email.message.EmailMessage*), 1341
add_cgi_vars() (*método wsgiref.handlers.BaseHandler*), 1507
add_charset() (*no módulo email.charset*), 1386
add_codec() (*no módulo email.charset*), 1386
add_cookie_header() (*método http.cookiejar.CookieJar*), 1602
add_dll_directory() (*no módulo os*), 762
add_done_callback() (*método asyncio.Future*), 1211
add_done_callback() (*método asyncio.Task*), 1158
add_done_callback() (*método concurrent.futures.Future*), 1097
add_fallback() (*método gettext.NullTranslations*), 1645
add_flag() (*método mailbox.Maildir*), 1406
add_flag() (*método mailbox.MaildirMessage*), 1412
add_flag() (*método mailbox.mboxMessage*), 1414
add_flag() (*método mailbox.MMDFMessage*), 1418
add_folder() (*método mailbox.Maildir*), 1405
add_folder() (*método mailbox.MH*), 1409
add_get_handler() (*método email.contentmanager.ContentManager*), 1362
add_handler() (*método urllib.request.OpenerDirector*), 1518
add_header() (*método email.message.EmailMessage*), 1337
add_header() (*método email.message.Message*), 1375
add_header() (*método urllib.request.Request*), 1517
add_header() (*método wsgiref.headers.Headers*), 1503
add_history() (*no módulo readline*), 193
add_label() (*método mailbox.BabylMessage*), 1416
add_mutually_exclusive_group() (*método argparse.ArgumentParser*), 933
add_note() (*método BaseException*), 116
add_option() (*método optparse.OptionParser*), 964
add_parent() (*método urllib.request.BaseHandler*), 1519
add_password() (*método urllib.request.HTTPPasswordMgr*), 1521
add_password() (*método urllib.request.HTTPPasswordMgrWithPriorAuth*), 1521
add_reader() (*método asyncio.loop*), 1196
add_related() (*método email.message.EmailMessage*), 1341
add_section() (*método configparser.ConfigParser*), 681
add_section() (*método configparser.RawConfigParser*), 685
add_sequence() (*método mailbox.MHMessage*), 1415
add_set_handler() (*método email.contentmanager.ContentManager*), 1362
add_signal_handler() (*método asyncio.loop*), 1200
add_subparsers() (*método argparse.ArgumentParser*), 928
add_type() (*método mimetypes.MimeTypes*), 1423
add_type() (*no módulo mimetypes*), 1422
add_unredirected_header() (*método urllib.request.Request*), 1517
add_writer() (*método asyncio.loop*), 1196
addAsyncCleanup() (*método unittest.IsolatedAsyncioTestCase*), 1858
addaudithook() (*no módulo sys*), 2021
addch() (*método curses.window*), 991
addClassCleanup() (*método de classe unittest.TestCase*), 1857
addCleanup() (*método unittest.TestCase*), 1856
addcomponent() (*método turtle.Shape*), 1746
addDuration() (*método unittest.TestResult*), 1864
addError() (*método unittest.TestResult*), 1863
addExpectedFailure() (*método unittest.TestResult*), 1864
addFailure() (*método unittest.TestResult*), 1863
addfile() (*método tarfile.TarFile*), 648
addFilter() (*método logging.Handler*), 815
addFilter() (*método logging.Logger*), 813
addHandler() (*método logging.Logger*), 813
addinfourl (*classe em urllib.response*), 1527
addLevelName() (*no módulo logging*), 823
addModuleCleanup() (*no módulo unittest*), 1867
addnstr() (*método curses.window*), 991
AddPackagePath() (*no módulo modulefinder*), 2164
addr_spec (*atributo email.headerregistry.Address*), 1361
address (*atributo email.headerregistry.SingleAddressHeader*), 1358
address (*atributo multiprocessing.connection.Listener*), 1072
address (*atributo multiprocessing.managers.BaseManager*), 1063
Address (*classe em email.headerregistry*), 1361
address_exclude() (*método ipaddress.IPv4Network*), 1630
address_exclude() (*método ipaddress.IPv6Network*), 1633
address_family (*atributo socketserver.BaseServer*), 1583
address_string() (*método http.server.BaseHTTPRequestHandler*), 1593
addresses (*atributo email.headerregistry.AddressHeader*), 1358
addresses (*atributo email.headerregistry.Group*), 1361

AddressHeader (*classe em email.headerregistry*), 1358
addressof() (*no módulo ctypes*), 892
AddressValueError, 1636
addshape() (*no módulo turtle*), 1743
addsitedir() (*no módulo site*), 2154
addSkip() (*método unittest.TestResult*), 1864
addstr() (*método curses.window*), 991
addSubTest() (*método unittest.TestResult*), 1864
addSuccess() (*método unittest.TestResult*), 1864
addTest() (*método unittest.TestSuite*), 1859
addTests() (*método unittest.TestSuite*), 1859
addTypeEqualityFunc() (*método unittest.TestCase*), 1854
addUnexpectedSuccess() (*método unittest.TestResult*), 1864
adjust_int_max_str_digits() (*no módulo test.support*), 1942
adjusted() (*método decimal.Decimal*), 380
adler32() (*no módulo zlib*), 611
AF_ALG (*no módulo socket*), 1248
AF_BLUETOOTH (*no módulo socket*), 1249
AF_CAN (*no módulo socket*), 1247
AF_DIVERT (*no módulo socket*), 1248
AF_HYPERV (*no módulo socket*), 1251
AF_INET (*no módulo socket*), 1245
AF_INET6 (*no módulo socket*), 1245
AF_LINK (*no módulo socket*), 1249
AF_PACKET (*no módulo socket*), 1248
AF_QIPCRTR (*no módulo socket*), 1250
AF_RDS (*no módulo socket*), 1248
AF_UNIX (*no módulo socket*), 1245
AF_UNSPEC (*no módulo socket*), 1245
AF_VSOCK (*no módulo socket*), 1249
agendamento de eventos, 1123
aguardável, **2347**
aifc
    module, 2337
aiter()
    built-in function, 8
alarm() (*no módulo signal*), 1320
ALERT_DESCRIPTION_HANDSHAKE_FAILURE (*no módulo ssl*), 1282
ALERT_DESCRIPTION_INTERNAL_ERROR (*no módulo ssl*), 1282
AlertDescription (*classe em ssl*), 1282
algorithm (*atributo sys.hash_info*), 2035
algorithms_available (*no módulo hashlib*), 695
algorithms_guaranteed (*no módulo hashlib*), 695
Alias
    Generic, 104
alias (*classe em ast*), 2221
alias (*pdb command*), 1973
alignment() (*no módulo ctypes*), 892
alive (*atributo weakref.finalize*), 315
all()
    built-in function, 8
ALL_COMPLETED (*no módulo asyncio*), 1153
ALL_COMPLETED (*no módulo concurrent.futures*), 1098
all_errors (*no módulo ftplib*), 1557
all_features (*no módulo xml.sax.handler*), 1477
all_frames (*atributo tracemalloc.Filter*), 1998
all_properties (*no módulo xml.sax.handler*), 1478
all_suffixes() (*no módulo importlib.machinery*), 2177
all_tasks() (*no módulo asyncio*), 1157
allocate_lock() (*no módulo _thread*), 1133
ALLOW_MISSING (*no módulo os.path*), 504
allow_reuse_address (*atributo socketserver.BaseServer*), 1584
allowed_domains() (*método http.cookiejar.DefaultCookiePolicy*), 1606
alt() (*no módulo curses.ascii*), 1014
ALT_DIGITS (*no módulo locale*), 1655
--altinstall
    ensurepip opção de linha de comando, 2004
altsep (*no módulo os*), 779
altzone (*no módulo time*), 807
ALWAYS_EQ (*no módulo test.support*), 1935
ALWAYS_TYPED_ACTIONS (*atributo optparse.Option*), 978
ambiente virtual, **2363**
Ambientes
    virtual, 2005
AmbiguousOptionError, 980
AMPER (*no módulo token*), 2250
AMPEREQUAL (*no módulo token*), 2251
analisador léxico, **2355**
anchor (*atributo pathlib.PurePath*), 477
Anchor (*classe em importlib.resources*), 2188
and
    operador, 39, 40
And (*classe em ast*), 2214
and_() (*no módulo operator*), 464
android_ver() (*no módulo platform*), 856
anext()
    built-in function, 8
AnnAssign (*classe em ast*), 2219
--annotate
    pickletools opção de linha de comando, 2289
Annotated (*no módulo typing*), 1773
annotation (*atributo inspect.Parameter*), 2132
ANNOTATION (*atributo symtable.SymbolTableType*), 2243
annotationlib
    module, 2143
annotations (*no módulo __future__*), 2118
annotations_to_string() (*no módulo annotationlib*), 2145
Ano 2038, 795
Anos de 2 dígitos, 795
anotação, **2345**
    tipo, anotação de; tipo, dica de, 104
anotação de função, **2351**
anotação de variável, **2363**

answer_challenge() (*no módulo multiprocessing.connection*), 1071
anticipate_failure() (*no módulo test.support*), 1938
Any (*no módulo typing*), 1765
ANY (*no módulo unittest.mock*), 1903
any()
    built-in function, 9
ANY_CONTIGUOUS (*atributo inspect.BufferFlags*), 2142
AnyStr (*no módulo typing*), 1765
apelido de tipo, **2362**
API provisória, **2359**
api_version (*no módulo sys*), 2048
apilevel (*no módulo sqlite3*), 571
apop() (*método poplib.POP3*), 1558
append() (*método array.array*), 309
append() (*método collections.deque*), 282
append() (*método email.header.Header*), 1382
append() (*método imaplib.IMAP4*), 1562
append() (*método sequence*), 52
append() (*método xml.etree.ElementTree.Element*), 1452
append_history_file() (*no módulo readline*), 192
appendChild() (*método xml.dom.Node*), 1462
appendleft() (*método collections.deque*), 282
AppleFrameworkLoader (*classe em importlib.machinery*), 2182
application_uri() (*no módulo wsgiref.util*), 1500
apply() (*método multiprocessing.pool.Pool*), 1069
apply_async() (*método multiprocessing.pool.Pool*), 1069
apply_defaults() (*método inspect.BoundArguments*), 2134
APRIL (*no módulo calendar*), 273
architecture() (*no módulo platform*), 853
archive (*atributo zipimport.zipimporter*), 2161
AREGTYPE (*no módulo tarfile*), 644
aRepr (*no módulo reprlib*), 333
arg (*classe em ast*), 2234
argparse
    module, 905
args (*atributo BaseException*), 116
args (*atributo functools.partial*), 463
args (*atributo inspect.BoundArguments*), 2134
args (*atributo subprocess.CompletedProcess*), 1105
args (*atributo subprocess.Popen*), 1114
args (*atributo typing.ParamSpec*), 1784
args (*pdb command*), 1971
args_from_interpreter_flags() (*no módulo test.support*), 1937
argtypes (*atributo ctypes._CFuncPtr*), 888
ArgumentDefaultsHelpFormatter (*classe em argparse*), 909
ArgumentError, 903, 937
argumento, **2345**
argumento nomeado, **2355**
argumento posicional, **2359**
ArgumentParser (*classe em argparse*), 906
arguments (*atributo inspect.BoundArguments*), 2133
arguments (*classe em ast*), 2234
ArgumentTypeError, 937
argv (*no módulo sys*), 2022
ArithmeticError, 117
aritmética, 41
arquivo
    arquivos grandes, 2313
    byte-code, 2259
    caminho configuração, 2152
    configuração, 667
    cópia, 524
    depurador configuração, 1969
    .ini, 667
    mime.types, 1422
    modos, 24
    .pdbrc, 1969
    plist, 689
    temporary, 514
arquivo binário, **2347**
arquivo ini, 667
arquivo ini do Windows, 667
arquivo texto, **2361**
arquivos grandes, 2313
Arquivos zip executáveis, 2015
array
    module, 308
    módulo, 71
array (*classe em array*), 309
Array (*classe em ctypes*), 902
Array() (*método multiprocessing.managers.SyncManager*), 1064
ARRAY() (*no módulo ctypes*), 902
Array() (*no módulo multiprocessing*), 1059
Array() (*no módulo multiprocessing.sharedctypes*), 1060
arraysize (*atributo sqlite3.Cursor*), 585
as_bytes() (*método email.message.EmailMessage*), 1335
as_bytes() (*método email.message.Message*), 1372
as_completed() (*no módulo asyncio*), 1153
as_completed() (*no módulo concurrent.futures*), 1098
as_digested_dict (*atributo compression.zstd.ZstdDict*), 605
as_file() (*no módulo importlib.resources*), 2189
as_integer_ratio(), 404
as_integer_ratio() (*método decimal.Decimal*), 380
as_integer_ratio() (*método float*), 45
as_integer_ratio() (*método fractions.Fraction*), 405
as_integer_ratio() (*método int*), 45
as_posix() (*método pathlib.PurePath*), 479
as_string() (*método email.message.EmailMessage*), 1335
as_string() (*método email.message.Message*), 1371
as_tuple() (*método decimal.Decimal*), 380
as_undigested_dict (*atributo compression.zstd.ZstdDict*), 605

as_uri() (*método pathlib.Path*), 484
ASCII (*no módulo re*), 155
ascii()
    built-in function, 9
ascii() (*no módulo curses.ascii*), 1013
ascii_letters (*no módulo string*), 129
ascii_lowercase (*no módulo string*), 129
ascii_uppercase (*no módulo string*), 129
asctime() (*no módulo time*), 796
asdict() (*no módulo dataclasses*), 2080
asin() (*no módulo cmath*), 372
asin() (*no módulo math*), 367
asinh() (*no módulo cmath*), 372
asinh() (*no módulo math*), 368
askcolor() (*no módulo tkinter.colorchooser*), 1674
askdirectory() (*no módulo tkinter.filedialog*), 1677
askfloat() (*no módulo tkinter.simpledialog*), 1676
askinteger() (*no módulo tkinter.simpledialog*), 1676
askokcancel() (*no módulo tkinter.messagebox*), 1680
askopenfile() (*no módulo tkinter.filedialog*), 1677
askopenfilename() (*no módulo tkinter.filedialog*), 1677
askopenfilenames() (*no módulo tkinter.filedialog*), 1677
askopenfiles() (*no módulo tkinter.filedialog*), 1677
askquestion() (*no módulo tkinter.messagebox*), 1680
askretrycancel() (*no módulo tkinter.messagebox*), 1680
asksaveasfile() (*no módulo tkinter.filedialog*), 1677
asksaveasfilename() (*no módulo tkinter.filedialog*), 1677
askstring() (*no módulo tkinter.simpledialog*), 1676
askyesno() (*no módulo tkinter.messagebox*), 1680
askyesnocancel() (*no módulo tkinter.messagebox*), 1680
aspas triplas, **2362**
assert
    instrução, 117
Assert (*classe em ast*), 2220
assert_any_await() (*método unittest.mock.AsyncMock*), 1883
assert_any_call() (*método unittest.mock.Mock*), 1873
assert_awaited() (*método unittest.mock.AsyncMock*), 1882
assert_awaited_once() (*método unittest.mock.AsyncMock*), 1882
assert_awaited_once_with() (*método unittest.mock.AsyncMock*), 1883
assert_awaited_with() (*método unittest.mock.AsyncMock*), 1882
assert_called() (*método unittest.mock.Mock*), 1872
assert_called_once() (*método unittest.mock.Mock*), 1872
assert_called_once_with() (*método unittest.mock.Mock*), 1873
assert_called_with() (*método unittest.mock.Mock*), 1872
assert_has_awaits() (*método unittest.mock.AsyncMock*), 1883
assert_has_calls() (*método unittest.mock.Mock*), 1873
assert_never() (*no módulo typing*), 1795
assert_not_awaited() (*método unittest.mock.AsyncMock*), 1883
assert_not_called() (*método unittest.mock.Mock*), 1873
assert_python_failure() (*no módulo test.support.script_helper*), 1944
assert_python_ok() (*no módulo test.support.script_helper*), 1943
assert_type() (*no módulo typing*), 1794
assertAlmostEqual() (*método unittest.TestCase*), 1853
assertCountEqual() (*método unittest.TestCase*), 1854
assertDictEqual() (*método unittest.TestCase*), 1855
assertEndsWith() (*método unittest.TestCase*), 1854
assertEqual() (*método unittest.TestCase*), 1849
assertFalse() (*método unittest.TestCase*), 1849
assertGreater() (*método unittest.TestCase*), 1854
assertGreaterEqual() (*método unittest.TestCase*), 1854
assertHasAttr() (*método unittest.TestCase*), 1854
assertIn() (*método unittest.TestCase*), 1849
assertInBytecode() (*método test.support.bytecode_helper.BytecodeTestCase*), 1944
AssertionError, 117
assertIs() (*método unittest.TestCase*), 1849
assertIsInstance() (*método unittest.TestCase*), 1850
assertIsNone() (*método unittest.TestCase*), 1849
assertIsNot() (*método unittest.TestCase*), 1849
assertIsNotNone() (*método unittest.TestCase*), 1849
assertIsSubclass() (*método unittest.TestCase*), 1850
assertLess() (*método unittest.TestCase*), 1854
assertLessEqual() (*método unittest.TestCase*), 1854
assertListEqual() (*método unittest.TestCase*), 1855
assertLogs() (*método unittest.TestCase*), 1852
assertMultiLineEqual() (*método unittest.TestCase*), 1855
assertNoLogs() (*método unittest.TestCase*), 1852
assertNotAlmostEqual() (*método unittest.TestCase*), 1853
assertNotEndsWith() (*método unittest.TestCase*), 1854
assertNotEqual() (*método unittest.TestCase*), 1849
assertNotHasAttr() (*método unittest.TestCase*), 1854
assertNotIn() (*método unittest.TestCase*), 1849
assertNotInBytecode() (*método test.support.bytecode_helper.BytecodeTestCase*), 1944
assertNotIsInstance() (*método unittest.TestCase*),

1850
assertNotIsSubclass() (*método unittest.TestCase*), 1850
assertNotRegex() (*método unittest.TestCase*), 1854
assertNotStartsWith() (*método unittest.TestCase*), 1854
assertRaises() (*método unittest.TestCase*), 1850
assertRaisesRegex() (*método unittest.TestCase*), 1851
assertRegex() (*método unittest.TestCase*), 1854
assertSequenceEqual() (*método unittest.TestCase*), 1855
assertSetEqual() (*método unittest.TestCase*), 1855
assertStartsWith() (*método unittest.TestCase*), 1854
assertTrue() (*método unittest.TestCase*), 1849
assertTupleEqual() (*método unittest.TestCase*), 1855
assertWarns() (*método unittest.TestCase*), 1851
assertWarnsRegex() (*método unittest.TestCase*), 1852
Assign (*classe em ast*), 2218
assume_default_colors() (*no módulo curses*), 984
ast
    module, 2203
AST (*classe em ast*), 2206
ast opção de linha de comando
    -a, 2242
    --feature-version, 2242
    -h, 2242
    --help, 2242
    -i, 2242
    --include-attributes, 2242
    --indent, 2242
    -m, 2242
    --mode, 2242
    --no-type-comments, 2242
    -O, 2242
    --optimize, 2242
    --show-empty, 2242
astimezone() (*método datetime.datetime*), 240
astuple() (*no módulo dataclasses*), 2080
AsyncContextDecorator (*classe em contextlib*), 2093
AsyncContextManager (*classe em typing*), 1809
asynccontextmanager() (*no módulo contextlib*), 2087
AsyncExitStack (*classe em contextlib*), 2095
AsyncFor (*classe em ast*), 2237
AsyncFunctionDef (*classe em ast*), 2237
AsyncGenerator (*classe em collections.abc*), 299
AsyncGenerator (*classe em typing*), 1807
AsyncGeneratorType (*no módulo types*), 321
asynchat
    module, 2337
asyncio
    module, 1137
asyncio.subprocess.DEVNULL (*variável interna*), 1176
asyncio.subprocess.PIPE (*variável interna*), 1176
asyncio.subprocess.Process (*classe interna*), 1176
asyncio.subprocess.STDOUT (*variável interna*), 1176
AsyncIterable (*classe em collections.abc*), 298
AsyncIterable (*classe em typing*), 1807
AsyncIterator (*classe em collections.abc*), 299
AsyncIterator (*classe em typing*), 1807
AsyncMock (*classe em unittest.mock*), 1881
asyncore
    module, 2337
AsyncResult (*classe em multiprocessing.pool*), 1070
asyncSetUp() (*método unittest.IsolatedAsyncioTestCase*), 1857
asyncTearDown() (*método unittest.IsolatedAsyncioTestCase*), 1857
AsyncWith (*classe em ast*), 2237
AT (*no módulo token*), 2251
at_eof() (*método asyncio.StreamReader*), 1164
atan() (*no módulo cmath*), 372
atan() (*no módulo math*), 367
atan2() (*no módulo math*), 367
atanh() (*no módulo cmath*), 372
atanh() (*no módulo math*), 368
ATEQUAL (*no módulo token*), 2251
atexit
    module, 2106
atexit (*atributo weakref.finalize*), 315
atof() (*no módulo locale*), 1657
atoi() (*no módulo locale*), 1657
atribuição
    fatia, 51
    subscrição, 51
atributo, **2347**
attach() (*método email.message.Message*), 1373
attach_mock() (*método unittest.mock.Mock*), 1874
attempted (*atributo doctest.TestResults*), 1831
AttlistDeclHandler() (*método xml.parsers.expat.xmlparser*), 1490
attrgetter() (*no módulo operator*), 466
attrib (*atributo xml.etree.ElementTree.Element*), 1451
Attribute (*classe em ast*), 2215
AttributeError, 117
attributes (*atributo xml.dom.Node*), 1461
AttributesImpl (*classe em xml.sax.xmlreader*), 1483
AttributesNSImpl (*classe em xml.sax.xmlreader*), 1483
attroff() (*método curses.window*), 991
attron() (*método curses.window*), 992
attrset() (*método curses.window*), 992
audioop
    module, 2338
audit() (*no módulo sys*), 2022
auditoria, 2022
AugAssign (*classe em ast*), 2219
AUGUST (*no módulo calendar*), 273
auth() (*método ftplib.FTP_TLS*), 1556

auth() (*método smtplib.SMTP*), 1571
authenticate() (*método imaplib.IMAP4*), 1562
AuthenticationError, 1049
authenticators() (*método netrc.netrc*), 689
authkey (*atributo multiprocessing.Process*), 1048
auto (*classe em enum*), 350
autocommit (*atributo sqlite3.Connection*), 581
autorange() (*método timeit.Timer*), 1985
available_timezones() (*no módulo zoneinfo*), 266
avoids_symlink_attacks (*atributo shutil.rmtree*), 529
Await (*classe em ast*), 2237
await_args (*atributo unittest.mock.AsyncMock*), 1884
await_args_list (*atributo unittest.mock.AsyncMock*), 1884
await_count (*atributo unittest.mock.AsyncMock*), 1884
Awaitable (*classe em collections.abc*), 298
Awaitable (*classe em typing*), 1807

B

-b
    compileall opção de linha de comando, 2262
    http.server opção de linha de comando, 1595
    unittest opção de linha de comando, 1840
b2a_base64() (*no módulo binascii*), 1429
b2a_hex() (*no módulo binascii*), 1430
b2a_qp() (*no módulo binascii*), 1430
b2a_uu() (*no módulo binascii*), 1429
b16decode() (*no módulo base64*), 1426
b16encode() (*no módulo base64*), 1426
b32decode() (*no módulo base64*), 1426
b32encode() (*no módulo base64*), 1426
b32hexdecode() (*no módulo base64*), 1426
b32hexencode() (*no módulo base64*), 1426
b64decode() (*no módulo base64*), 1425
b64encode() (*no módulo base64*), 1425
b85decode() (*no módulo base64*), 1427
b85encode() (*no módulo base64*), 1427
Babyl (*classe em mailbox*), 1410
BabylMessage (*classe em mailbox*), 1416
back() (*no módulo turtle*), 1720
backend (*no módulo readline*), 191
backslashreplace
    error handler's name, 208
backslashreplace_errors() (*no módulo codecs*), 209
backup() (*método sqlite3.Connection*), 579
backward() (*no módulo turtle*), 1720
BadGzipFile, 615
BadOptionError, 980
BadStatusLine, 1545
BadZipFile, 629
BadZipfile, 629
bancos de dados, 565
Barrier (*classe em asyncio*), 1173
Barrier (*classe em multiprocessing*), 1056
Barrier (*classe em threading*), 1037
Barrier() (*método multiprocessing.managers.SyncManager*), 1063
base64
    codificação, 1425
    module, 1425
    módulo, 1429
base_exec_prefix (*no módulo sys*), 2022
base_prefix (*no módulo sys*), 2023
BaseCGIHandler (*classe em wsgiref.handlers*), 1506
BaseCookie (*classe em http.cookies*), 1597
BaseException, 116
BaseExceptionGroup, 125
BaseHandler (*classe em urllib.request*), 1514
BaseHandler (*classe em wsgiref.handlers*), 1506
BaseHeader (*classe em email.headerregistry*), 1357
BaseHTTPRequestHandler (*classe em http.server*), 1590
BaseManager (*classe em multiprocessing.managers*), 1062
basename() (*no módulo os.path*), 500
BaseProtocol (*classe em asyncio*), 1218
BaseProxy (*classe em multiprocessing.managers*), 1067
BaseRequestHandler (*classe em socketserver*), 1585
BaseRotatingHandler (*classe em logging.handlers*), 841
BaseSelector (*classe em selectors*), 1314
BaseServer (*classe em socketserver*), 1583
BaseTransport (*classe em asyncio*), 1214
basicConfig() (*no módulo logging*), 824
BasicContext (*no módulo decimal*), 387
BasicInterpolation (*classe em configparser*), 672
batched() (*no módulo itertools*), 437
baudrate() (*no módulo curses*), 984
bbox() (*método tkinter.ttk.Treeview*), 1693
BDADDR_ANY (*no módulo socket*), 1249
BDADDR_BREDR (*no módulo socket*), 1249
BDADDR_LE_PUBLIC (*no módulo socket*), 1249
BDADDR_LE_RANDOM (*no módulo socket*), 1249
BDADDR_LOCAL (*no módulo socket*), 1249
bdb
    module, 1955
    módulo, 1964
Bdb (*classe em bdb*), 1956
BdbQuit, 1955
BDFL, **2347**
beep() (*no módulo curses*), 984
Beep() (*no módulo winsound*), 2303
begin_fill() (*no módulo turtle*), 1730
begin_poly() (*no módulo turtle*), 1736
BEL (*no módulo curses.ascii*), 1011
below() (*método curses.panel.Panel*), 1014
BELOW_NORMAL_PRIORITY_CLASS (*no módulo subprocess*), 1117
Benchmarking, 1983
benchmarking, 799, 804
--best

gzip opção de linha de comando, 618
betavariate() (*no módulo random*), 411
bgcolor() (*no módulo turtle*), 1737
bgpic() (*no módulo turtle*), 1738
biblioteca padrão, **2361**
bidirectional() (*no módulo unicodedata*), 188
bigaddrspacetest() (*no módulo test.support*), 1940
BigEndianStructure (*classe em ctypes*), 899
BigEndianUnion (*classe em ctypes*), 899
bigmemtest() (*no módulo test.support*), 1939
bin()
built-in function, 9
binário
dados, packing, 197
literais, 41
binários, semáforos, 1133
Binary (*classe em xmlrpc.client*), 1612
BINARY_OP (*opcode*), 2273
BINARY_SLICE (*opcode*), 2273
BinaryIO (*classe em typing*), 1794
binascii
module, 1429
--bind
http.server opção de linha de comando, 1595
bind() (*método inspect.Signature*), 2131
bind() (*método socket.socket*), 1259
bind_partial() (*método inspect.Signature*), 2131
bind_port() (*no módulo test.support.socket_helper*), 1943
bind_textdomain_codeset() (*no módulo locale*), 1659
bind_unix_socket() (*no módulo test.support.socket_helper*), 1943
bindtextdomain() (*no módulo gettext*), 1643
bindtextdomain() (*no módulo locale*), 1659
binomialvariate() (*no módulo random*), 410
BinOp (*classe em ast*), 2213
bisect
module, 305
bisect() (*no módulo bisect*), 305
bisect_left() (*no módulo bisect*), 305
bisect_right() (*no módulo bisect*), 305
bit a bit
operações, 42
bit_count() (*método int*), 43
bit_length() (*método int*), 43
bit_offset (*atributo ctypes.CField*), 902
bit_size (*atributo ctypes.CField*), 902
BitAnd (*classe em ast*), 2213
BitOr (*classe em ast*), 2213
bits_per_digit (*atributo sys.int_info*), 2036
BitXor (*classe em ast*), 2213
bk() (*no módulo turtle*), 1720
bkgd() (*método curses.window*), 992
bkgdset() (*método curses.window*), 992
blake2b() (*no módulo hashlib*), 698
blake2b, blake2s, 698
blake2b.MAX_DIGEST_SIZE (*no módulo hashlib*), 699
blake2b.MAX_KEY_SIZE (*no módulo hashlib*), 699
blake2b.PERSON_SIZE (*no módulo hashlib*), 699
blake2b.SALT_SIZE (*no módulo hashlib*), 699
blake2s() (*no módulo hashlib*), 698
blake2s.MAX_DIGEST_SIZE (*no módulo hashlib*), 699
blake2s.MAX_KEY_SIZE (*no módulo hashlib*), 699
blake2s.PERSON_SIZE (*no módulo hashlib*), 699
blake2s.SALT_SIZE (*no módulo hashlib*), 699
BLKTYPE (*no módulo tarfile*), 644
Blob (*classe em sqlite3*), 586
blobopen() (*método sqlite3.Connection*), 572
block_on_close (*atributo socketserver.ThreadingMixIn*), 1582
block_size (*atributo hmac.HMAC*), 706
blocked_domains() (*método http.cookiejar.DefaultCookiePolicy*), 1605
BlockingIOError, 123, 783
blocksize (*atributo http.client.HTTPConnection*), 1547
body() (*método tkinter.simpledialog.Dialog*), 1676
body_encode() (*método email.charset.Charset*), 1385
body_encoding (*atributo email.charset.Charset*), 1385
body_line_iterator() (*no módulo email.iterators*), 1390
BOLD (*no módulo tkinter.font*), 1675
BOM (*no módulo codecs*), 207
BOM_BE (*no módulo codecs*), 207
BOM_LE (*no módulo codecs*), 207
BOM_UTF8 (*no módulo codecs*), 207
BOM_UTF16 (*no módulo codecs*), 207
BOM_UTF16_BE (*no módulo codecs*), 207
BOM_UTF16_LE (*no módulo codecs*), 207
BOM_UTF32 (*no módulo codecs*), 207
BOM_UTF32_BE (*no módulo codecs*), 207
BOM_UTF32_LE (*no módulo codecs*), 207
bool (*classe interna*), 9
BOOLEAN_STATES (*atributo configparser.ConfigParser*), 677
Booleano
objeto, 40
operações, 39
tipo, 9
valores, 48
BooleanOptionalAction (*classe em argparse*), 925
BoolOp (*classe em ast*), 2213
bootstrap() (*no módulo ensurepip*), 2004
border() (*método curses.window*), 992
bottom() (*método curses.panel.Panel*), 1015
bottom_panel() (*no módulo curses.panel*), 1014
BoundArguments (*classe em inspect*), 2133
BoundaryError, 1355
BoundedSemaphore (*classe em asyncio*), 1173
BoundedSemaphore (*classe em multiprocessing*), 1057
BoundedSemaphore (*classe em threading*), 1035
BoundedSemaphore() (*método multiprocessing.managers.SyncManager*), 1063

bounds() (*método compression.zstd.CompressionParameter*), 606
bounds() (*método compression.zstd.DecompressionParameter*), 609
box() (*método curses.window*), 992
bpbynumber (*atributo bdb.Breakpoint*), 1956
bpformat() (*método bdb.Breakpoint*), 1956
bplist (*atributo bdb.Breakpoint*), 1956
bpprint() (*método bdb.Breakpoint*), 1956
BRANCH_LEFT (*monitoring event*), 2050
BRANCH_RIGHT (*monitoring event*), 2050
Break (*classe em ast*), 2223
break (*pdb command*), 1970
break_anywhere() (*método bdb.Bdb*), 1958
break_here() (*método bdb.Bdb*), 1958
break_long_words (*atributo textwrap.TextWrapper*), 186
break_on_hyphens (*atributo textwrap.TextWrapper*), 186
Breakpoint (*classe em bdb*), 1955
breakpoint()
    built-in function, 9
breakpointhook() (*no módulo sys*), 2024
breakpoints, 1705
broadcast_address (*atributo ipaddress.IPv4Network*), 1630
broadcast_address (*atributo ipaddress.IPv6Network*), 1633
broken (*atributo asyncio.Barrier*), 1174
broken (*atributo threading.Barrier*), 1037
BrokenBarrierError, 1038, 1174
BrokenExecutor, 1099
BrokenInterpreterPool, 1099
BrokenPipeError, 123
BrokenProcessPool, 1099
BrokenThreadPool, 1099
BROWSER, 1497, 1498
BS (*no módulo curses.ascii*), 1011
BsdDbShelf (*classe em shelve*), 557
btlazy2 (*atributo compression.zstd.Strategy*), 609
btopt (*atributo compression.zstd.Strategy*), 609
BTPROTO_HCI (*no módulo socket*), 1249
BTPROTO_L2CAP (*no módulo socket*), 1249
BTPROTO_RFCOMM (*no módulo socket*), 1249
BTPROTO_SCO (*no módulo socket*), 1249
btultra (*atributo compression.zstd.Strategy*), 609
btultra2 (*atributo compression.zstd.Strategy*), 609
buf (*atributo multiprocessing.shared_memory.SharedMemory*), 1086
--buffer
    unittest opção de linha de comando, 1840
buffer (*atributo io.TextIOBase*), 791
buffer (*atributo unittest.TestResult*), 1862
Buffer (*classe em collections.abc*), 299
buffer, tamanho, E/S, 26
buffer_info() (*método array.array*), 309
buffer_size (*atributo xml.parsers.expat.xmlparser*), 1489
buffer_text (*atributo xml.parsers.expat.xmlparser*), 1489
buffer_updated() (*método asyncio.BufferedProtocol*), 1220
buffer_used (*atributo xml.parsers.expat.xmlparser*), 1489
BufferedIOBase (*classe em io*), 786
BufferedProtocol (*classe em asyncio*), 1218
BufferedRandom (*classe em io*), 790
BufferedReader (*classe em io*), 789
BufferedRWPair (*classe em io*), 790
BufferedWriter (*classe em io*), 789
BufferError, 117
BufferFlags (*classe em inspect*), 2142
BufferingFormatter (*classe em logging*), 817
BufferingHandler (*classe em logging.handlers*), 849
BufferTooShort, 1049
BUILD_INTERPOLATION (*opcode*), 2278
BUILD_LIST (*opcode*), 2279
BUILD_MAP (*opcode*), 2279
build_opener() (*no módulo urllib.request*), 1512
BUILD_SET (*opcode*), 2279
BUILD_SLICE (*opcode*), 2284
BUILD_STRING (*opcode*), 2279
BUILD_TEMPLATE (*opcode*), 2278
BUILD_TUPLE (*opcode*), 2278
built-in function
    __import__(), 34
    abs(), 8
    aiter(), 8
    all(), 8
    anext(), 8
    any(), 9
    ascii(), 9
    bin(), 9
    breakpoint(), 9
    callable(), 10
    chr(), 10
    classmethod(), 10
    compile(), 11
    delattr(), 13
    dir(), 13
    divmod(), 14
    enumerate(), 14
    eval(), 14
    exec(), 15
    filter(), 16
    format(), 17
    getattr(), 18
    globals(), 18
    hasattr(), 18
    hash(), 18
    help(), 18
    hex(), 19
    id(), 19
    input(), 19

isinstance(), 21
issubclass(), 21
iter(), 21
len(), 21
locals(), 21
map(), 22
max(), 22
min(), 23
multiprocessing.Manager(), 1062
next(), 23
oct(), 23
open(), 24
ord(), 26
pow(), 26
print(), 27
property.deleter(), 28
property.getter(), 28
property.setter(), 28
repr(), 29
reversed(), 29
round(), 29
setattr(), 29
sorted(), 30
staticmethod(), 30
sum(), 31
vars(), 33
zip(), 33
builtin_module_names (*no módulo sys*), 2023
BuiltinFunctionType (*no módulo types*), 322
BuiltinImporter (*classe em importlib.machinery*), 2178
BuiltinMethodType (*no módulo types*), 322
builtins
module, 2061
módulo, 34
burst() (*método imaplib.IMAP4.Idler*), 1563
busy_retry() (*no módulo test.support*), 1935
BUTTON_ALT (*no módulo curses*), 1008
BUTTON_CTRL (*no módulo curses*), 1008
BUTTON_SHIFT (*no módulo curses*), 1008
buttonbox() (*método tkinter.simpledialog.Dialog*), 1676
BUTTONn_CLICKED (*no módulo curses*), 1008
BUTTONn_DOUBLE_CLICKED (*no módulo curses*), 1008
BUTTONn_PRESSED (*no módulo curses*), 1008
BUTTONn_RELEASED (*no módulo curses*), 1008
BUTTONn_TRIPLE_CLICKED (*no módulo curses*), 1008
bye() (*no módulo turtle*), 1744
byref() (*no módulo ctypes*), 892
byte_offset (*atributo ctypes.CField*), 902
byte_size (*atributo ctypes.CField*), 902
bytearray
formatação, 85
interpolação, 85
métodos, 74
objeto, 51, 71, 72
bytearray (*classe interna*), 72
bytecode, **2347**
byte-code
arquivo, 2259
Bytecode (*classe em dis*), 2267
BYTECODE_SUFFIXES (*no módulo importlib.machinery*), 2177
Bytecode.codeobj (*no módulo dis*), 2267
Bytecode.first_line (*no módulo dis*), 2267
BytecodeTestCase (*classe em test.support.bytecode_helper*), 1944
byteorder (*no módulo sys*), 2023
bytes
formatação, 85
interpolação, 85
métodos, 74
objeto, 71
str (*classe embutida*), 57
bytes (*atributo uuid.UUID*), 1575
bytes (*classe interna*), 71
bytes_le (*atributo uuid.UUID*), 1575
bytes_warning (*atributo sys.flags*), 2028
BytesFeedParser (*classe em email.parser*), 1342
BytesGenerator (*classe em email.generator*), 1345
BytesHeaderParser (*classe em email.parser*), 1344
BytesIO (*classe em io*), 788
BytesParser (*classe em email.parser*), 1343
ByteString (*classe em collections.abc*), 297
ByteString (*classe em typing*), 1805
byteswap() (*método array.array*), 310
BytesWarning, 125
bz2
module, 618
BZ2Compressor (*classe em bz2*), 621
BZ2Decompressor (*classe em bz2*), 621
BZ2File (*classe em bz2*), 619

C

C
linguagem, 40
structures, 197
-C
dis opção de linha de comando, 2266
trace opção de linha de comando, 1989
uuid opção de linha de comando, 1579
-c
calendar opção de linha de comando, 275
idle opção de linha de comando, 1708
pdb opção de linha de comando, 1965
random opção de linha de comando, 417
tarfile opção de linha de comando, 655
trace opção de linha de comando, 1989
unittest opção de linha de comando, 1840
zipapp opção de linha de comando, 2016
zipfile opção de linha de comando, 640
C14NWriterTarget (*classe em xml.etree.ElementTree*), 1456
c_bool (*classe em ctypes*), 899

c_byte (*classe em ctypes*), 896
c_char (*classe em ctypes*), 897
c_char_p (*classe em ctypes*), 897
C_CONTIGUOUS (*atributo inspect.BufferFlags*), 2142
c_contiguous (*atributo memoryview*), 95
c_double (*classe em ctypes*), 897
c_double_complex (*classe em ctypes*), 897
c_float (*classe em ctypes*), 897
c_float_complex (*classe em ctypes*), 897
c_int (*classe em ctypes*), 897
c_int8 (*classe em ctypes*), 897
c_int16 (*classe em ctypes*), 897
c_int32 (*classe em ctypes*), 897
c_int64 (*classe em ctypes*), 897
c_long (*classe em ctypes*), 897
c_longdouble (*classe em ctypes*), 897
c_longdouble_complex (*classe em ctypes*), 897
c_longlong (*classe em ctypes*), 897
C_RAISE (*monitoring event*), 2050
C_RETURN (*monitoring event*), 2050
c_short (*classe em ctypes*), 898
c_size_t (*classe em ctypes*), 898
c_ssize_t (*classe em ctypes*), 898
c_time_t (*classe em ctypes*), 898
c_ubyte (*classe em ctypes*), 898
c_uint (*classe em ctypes*), 898
c_uint8 (*classe em ctypes*), 898
c_uint16 (*classe em ctypes*), 898
c_uint32 (*classe em ctypes*), 898
c_uint64 (*classe em ctypes*), 898
c_ulong (*classe em ctypes*), 898
c_ulonglong (*classe em ctypes*), 898
c_ushort (*classe em ctypes*), 898
c_void_p (*classe em ctypes*), 898
c_wchar (*classe em ctypes*), 898
c_wchar_p (*classe em ctypes*), 898
CACHE (*opcode*), 2272
cache() (*no módulo functools*), 452
cache_clear() (*método functools.lru_cache*), 454
cache_from_source() (*no módulo importlib.util*), 2183
cache_info() (*método functools.lru_cache*), 454
cached (*atributo importlib.machinery.ModuleSpec*), 2182
cached_property() (*no módulo functools*), 453
CacheFTPHandler (*classe em urllib.request*), 1516
calcobjsize() (*no módulo test.support*), 1938
calcsize() (*no módulo struct*), 198
calcvobjsize() (*no módulo test.support*), 1938
calendar
    module, 267
Calendar (*classe em calendar*), 267
calendar opção de linha de comando
    -c, 275
    --css, 275
    -e, 274
    --encoding, 274
    -f, 275
    --first-weekday, 275
    -h, 274
    --help, 274
    -L, 274
    -l, 275
    --lines, 275
    --locale, 274
    -m, 275
    month, 275
    --months, 275
    -s, 275
    --spacing, 275
    -t, 275
    --type, 275
    -w, 275
    --width, 275
    year, 275
calendar() (*no módulo calendar*), 272
Call (*classe em ast*), 2214
CALL (*monitoring event*), 2050
CALL (*opcode*), 2283
call() (*método concurrent.interpreters.Interpreter*), 1102
call() (*no módulo operator*), 466
call() (*no módulo subprocess*), 1118
call() (*no módulo unittest.mock*), 1901
call_annotate_function() (*no módulo annotationlib*), 2146
call_args (*atributo unittest.mock.Mock*), 1877
call_args_list (*atributo unittest.mock.Mock*), 1877
call_at() (*método asyncio.loop*), 1189
call_count (*atributo unittest.mock.Mock*), 1875
call_evaluate_function() (*no módulo annotationlib*), 2146
call_exception_handler() (*método asyncio.loop*), 1202
CALL_FUNCTION_EX (*opcode*), 2283
call_in_thread() (*método concurrent.interpreters.Interpreter*), 1102
CALL_INTRINSIC_1 (*opcode*), 2286
CALL_INTRINSIC_2 (*opcode*), 2286
CALL_KW (*opcode*), 2283
call_later() (*método asyncio.loop*), 1188
call_list() (*método unittest.mock.call*), 1901
call_soon() (*método asyncio.loop*), 1187
call_soon_threadsafe() (*método asyncio.loop*), 1188
call_tracing() (*no módulo sys*), 2023
Callable (*classe em collections.abc*), 297
Callable (*no módulo typing*), 1808
callable()
    built-in function, 10
CallableProxyType (*no módulo weakref*), 315
callback (*atributo optparse.Option*), 966
callback() (*método contextlib.ExitStack*), 2094
callback_args (*atributo optparse.Option*), 966
callback_kwargs (*atributo optparse.Option*), 966
callbacks (*no módulo gc*), 2122

called (*atributo unittest.mock.Mock*), 1875
CalledProcessError, 1106
caminho
    configuração arquivo, 2152
    módulo pesquisa, 524, 2039, 2151
    operações, 471, 499
caminho de importação, **2353**
caminhos UNC
    e os.makedirs(), 741
CAN (*no módulo curses.ascii*), 1012
CAN_BCM (*no módulo socket*), 1247
can_change_color() (*no módulo curses*), 984
can_fetch() (*método urllib.robotparser.RobotFileParser*), 1538
CAN_ISOTP (*no módulo socket*), 1247
CAN_J1939 (*no módulo socket*), 1248
CAN_RAW_FD_FRAMES (*no módulo socket*), 1247
CAN_RAW_JOIN_FILTERS (*no módulo socket*), 1247
can_symlink() (*no módulo test.support.os_helper*), 1946
can_write_eof() (*método asyncio.StreamWriter*), 1165
can_write_eof() (*método asyncio.WriteTransport*), 1216
can_xattr() (*no módulo test.support.os_helper*), 1946
CANCEL (*no módulo tkinter.messagebox*), 1680
cancel() (*método asyncio.Future*), 1212
cancel() (*método asyncio.Handle*), 1204
cancel() (*método asyncio.Task*), 1159
cancel() (*método concurrent.futures.Future*), 1097
cancel() (*método sched.scheduler*), 1124
cancel() (*método threading.Timer*), 1036
cancel() (*método tkinter.dnd.DndHandler*), 1682
cancel_command() (*método tkinter.filedialog.FileDialog*), 1678
cancel_dump_traceback_later() (*no módulo faulthandler*), 1963
cancel_join_thread() (*método multiprocessing.Queue*), 1052
cancelled() (*método asyncio.Future*), 1211
cancelled() (*método asyncio.Handle*), 1204
cancelled() (*método asyncio.Task*), 1160
cancelled() (*método concurrent.futures.Future*), 1097
CancelledError, 1099, 1182
cancelling() (*método asyncio.Task*), 1161
CannotSendHeader, 1545
CannotSendRequest, 1545
canonic() (*método bdb.Bdb*), 1957
canonical() (*método decimal.Context*), 390
canonical() (*método decimal.Decimal*), 380
canonicalize() (*no módulo xml.etree.ElementTree*), 1447
capa() (*método poplib.POP3*), 1558
capitalize() (*método bytearray*), 80
capitalize() (*método bytes*), 80
capitalize() (*método str*), 57
capitals (*atributo decimal.Context*), 388
CapsuleType (*classe em types*), 324
capture_call_graph() (*no módulo asyncio*), 1184
captured_stderr() (*no módulo test.support*), 1937
captured_stdin() (*no módulo test.support*), 1937
captured_stdout() (*no módulo test.support*), 1937
captureWarnings() (*no módulo logging*), 826
capwords() (*no módulo string*), 141
caractere, 187
carregador, **2355**
casefold() (*método str*), 57
cast() (*método memoryview*), 92
cast() (*no módulo ctypes*), 892
cast() (*no módulo typing*), 1794
--catch
    unittest opção de linha de comando, 1840
catch_threading_exception() (*no módulo test.support.threading_helper*), 1945
catch_unraisable_exception() (*no módulo test.support*), 1940
catch_warnings (*classe em warnings*), 2074
category() (*no módulo unicodedata*), 188
cbreak() (*no módulo curses*), 984
cbrt() (*no módulo math*), 365
ccc() (*método ftplib.FTP_TLS*), 1556
cdf() (*método statistics.NormalDist*), 430
CDLL (*classe em ctypes*), 885
ceil() (*no módulo math*), 42, 363
CellType (*no módulo types*), 322
center() (*método bytearray*), 78
center() (*método bytes*), 78
center() (*método str*), 58
CERT_NONE (*no módulo ssl*), 1276
CERT_OPTIONAL (*no módulo ssl*), 1276
CERT_REQUIRED (*no módulo ssl*), 1276
cert_store_stats() (*método ssl.SSLContext*), 1288
cert_time_to_seconds() (*no módulo ssl*), 1274
CertificateError, 1274
CField (*classe em ctypes*), 901
cfmakecbreak() (*no módulo tty*), 2317
cfmakeraw() (*no módulo tty*), 2317
CFUNCTYPE() (*no módulo ctypes*), 889
cget() (*método tkinter.font.Font*), 1675
cgi
    module, 2338
--cgi
    http.server opção de linha de comando, 1596
cgi_directories (*atributo http.server.CGIHTTPRequestHandler*), 1595
CGIHandler (*classe em wsgiref.handlers*), 1505
CGIHTTPRequestHandler (*classe em http.server*), 1594
cgitb
    module, 2338
CGIXMLRPCRequestHandler (*classe em xmlrpc.server*), 1617
chain() (*no módulo itertools*), 438

chain_log (*atributo compression.zstd.CompressionParameter*), 606
ChainMap (*classe em collections*), 276
ChainMap (*classe em typing*), 1804
chamável, **2347**
change_cwd() (*no módulo test.support.os_helper*), 1946
CHANNEL_BINDING_TYPES (*no módulo ssl*), 1281
CHAR_MAX (*no módulo locale*), 1658
CharacterDataHandler() (*método xml.parsers.expat.xmlparser*), 1490
characters() (*método xml.sax.handler.ContentHandler*), 1479
characters_written (*atributo BlockingIOError*), 123
charmap_build() (*no módulo codecs*), 205
Charset (*classe em email.charset*), 1384
charset() (*método gettext.NullTranslations*), 1646
chdir() (*no módulo contextlib*), 2091
chdir() (*no módulo os*), 736
check (*atributo lzma.LZMADecompressor*), 626
check() (*método imaplib.IMAP4*), 1562
check() (*no módulo tabnanny*), 2257
check__all__() (*no módulo test.support*), 1941
check_call() (*no módulo subprocess*), 1118
check_disallow_instantiation() (*no módulo test.support*), 1942
CHECK_EG_MATCH (*opcode*), 2276
CHECK_EXC_MATCH (*opcode*), 2276
check_free_after_iterating() (*no módulo test.support*), 1941
check_hostname (*atributo ssl.SSLContext*), 1293
check_impl_detail() (*no módulo test.support*), 1936
check_no_resource_warning() (*no módulo test.support.warnings_helper*), 1949
check_output() (*método doctest.OutputChecker*), 1833
check_output() (*no módulo subprocess*), 1119
check_returncode() (*método subprocess.CompletedProcess*), 1106
check_syntax_error() (*no módulo test.support*), 1940
check_syntax_warning() (*no módulo test.support.warnings_helper*), 1949
check_unused_args() (*método string.Formatter*), 131
check_warnings() (*no módulo test.support.warnings_helper*), 1949
checkcache() (*no módulo linecache*), 524
CHECKED_HASH (*atributo py_compile.PycInvalidationMode*), 2260
checkfuncname() (*no módulo bdb*), 1960
checksizeof() (*no módulo test.support*), 1938
checksum_flag (*atributo compression.zstd.CompressionParameter*), 608
chflags() (*no módulo os*), 736
chgat() (*método curses.window*), 992
childNodes (*atributo xml.dom.Node*), 1461
ChildProcessError, 123
children (*atributo pyclbr.Class*), 2259
children (*atributo pyclbr.Function*), 2258
children (*atributo tkinter.Tk*), 1663
chksum (*atributo tarfile.TarInfo*), 650
chmod() (*método pathlib.Path*), 495
chmod() (*no módulo os*), 737
--choice
    random opção de linha de comando, 417
choice() (*no módulo random*), 409
choice() (*no módulo secrets*), 707
choices (*atributo optparse.Option*), 966
choices() (*no módulo random*), 409
Chooser (*classe em tkinter.colorchooser*), 1674
chown() (*no módulo os*), 738
chown() (*no módulo shutil*), 529
chr()
    built-in function, 10
chroot() (*no módulo os*), 738
CHRTYPE (*no módulo tarfile*), 644
chunk
    module, 2338
cipher() (*método ssl.SSLSocket*), 1285
circle() (*no módulo turtle*), 1723
CIRCUMFLEX (*no módulo token*), 2250
CIRCUMFLEXEQUAL (*no módulo token*), 2251
clamp (*atributo decimal.Context*), 388
Clamped (*classe em decimal*), 394
CLASS (*atributo symtable.SymbolTableType*), 2243
Class (*classe em pyclbr*), 2259
Class (*classe em symtable*), 2245
ClassDef (*classe em ast*), 2236
classe, **2348**
classe base abstrata, **2345**
classe estilo novo, **2357**
classmethod()
    built-in function, 10
ClassMethodDescriptorType (*no módulo types*), 322
ClassVar (*no módulo typing*), 1771
CLD_CONTINUED (*no módulo os*), 774
CLD_DUMPED (*no módulo os*), 774
CLD_EXITED (*no módulo os*), 774
CLD_KILLED (*no módulo os*), 774
CLD_STOPPED (*no módulo os*), 774
CLD_TRAPPED (*no módulo os*), 774
clean() (*método mailbox.Maildir*), 1405
cleandoc() (*no módulo inspect*), 2129
CleanImport (*classe em test.support.import_helper*), 1948
cleanup() (*método tempfile.TemporaryDirectory*), 516
CLEANUP_THROW (*opcode*), 2274
--clear
    venv opção de linha de comando, 2006
clear (*pdb command*), 1970
Clear Breakpoint, 1705
clear() (*método array.array*), 311
clear() (*método asyncio.Event*), 1171
clear() (*método collections.deque*), 282

clear() (*método curses.window*), 992
clear() (*método dbm.gnu.gdbm*), 563
clear() (*método dbm.ndbm.ndbm*), 565
clear() (*método dict*), 100
clear() (*método email.message.EmailMessage*), 1341
clear() (*método frozenset*), 97
clear() (*método http.cookiejar.CookieJar*), 1603
clear() (*método mailbox.Mailbox*), 1404
clear() (*método sequence*), 52
clear() (*método threading.Event*), 1036
clear() (*método xml.etree.ElementTree.Element*), 1451
clear() (*no módulo turtle*), 1731
clear_all_breaks() (*método bdb.Bdb*), 1959
clear_all_file_breaks() (*método bdb.Bdb*), 1959
clear_bpbynumber() (*método bdb.Bdb*), 1959
clear_break() (*método bdb.Bdb*), 1959
clear_cache() (*método de classe zoneinfo.ZoneInfo*), 265
clear_cache() (*no módulo filecmp*), 512
clear_content() (*método email.message.EmailMessage*), 1341
clear_flags() (*método decimal.Context*), 389
clear_frames() (*no módulo traceback*), 2110
clear_history() (*no módulo readline*), 193
clear_memo() (*método pickle.Pickler*), 542
clear_overloads() (*no módulo typing*), 1798
clear_session_cookies() (*método http.cookiejar.CookieJar*), 1603
clear_tool_id() (*no módulo sys.monitoring*), 2050
clear_traces() (*no módulo tracemalloc*), 1996
clear_traps() (*método decimal.Context*), 389
clearcache() (*no módulo linecache*), 524
clearok() (*método curses.window*), 993
clearscreen() (*no módulo turtle*), 1738
clearstamp() (*no módulo turtle*), 1724
clearstamps() (*no módulo turtle*), 1724
Client() (*no módulo multiprocessing.connection*), 1071
client_address (*atributo http.server.BaseHTTPRequestHandler*), 1591
client_address (*atributo socketserver.BaseRequestHandler*), 1585
CLOCK_BOOTTIME (*no módulo time*), 805
clock_getres() (*no módulo time*), 797
clock_gettime() (*no módulo time*), 797
clock_gettime_ns() (*no módulo time*), 797
CLOCK_HIGHRES (*no módulo time*), 805
CLOCK_MONOTONIC (*no módulo time*), 806
CLOCK_MONOTONIC_RAW (*no módulo time*), 806
CLOCK_MONOTONIC_RAW_APPROX (*no módulo time*), 806
CLOCK_PROCESS_CPUTIME_ID (*no módulo time*), 806
CLOCK_PROF (*no módulo time*), 806
CLOCK_REALTIME (*no módulo time*), 807
clock_seq (*atributo uuid.UUID*), 1576
clock_seq_hi_variant (*atributo uuid.UUID*), 1576
clock_seq_low (*atributo uuid.UUID*), 1576
clock_settime() (*no módulo time*), 797
clock_settime_ns() (*no módulo time*), 797
CLOCK_TAI (*no módulo time*), 806
CLOCK_THREAD_CPUTIME_ID (*no módulo time*), 806
CLOCK_UPTIME (*no módulo time*), 806
CLOCK_UPTIME_RAW (*no módulo time*), 806
CLOCK_UPTIME_RAW_APPROX (*no módulo time*), 807
clone() (*método email.generator.BytesGenerator*), 1346
clone() (*método email.generator.Generator*), 1347
clone() (*método email.policy.Policy*), 1350
clone() (*no módulo turtle*), 1736
CLONE_FILES (*no módulo os*), 721
CLONE_FS (*no módulo os*), 721
CLONE_NEWCGROUP (*no módulo os*), 721
CLONE_NEWIPC (*no módulo os*), 721
CLONE_NEWNET (*no módulo os*), 721
CLONE_NEWNS (*no módulo os*), 721
CLONE_NEWPID (*no módulo os*), 721
CLONE_NEWTIME (*no módulo os*), 721
CLONE_NEWUSER (*no módulo os*), 721
CLONE_NEWUTS (*no módulo os*), 721
CLONE_SIGHAND (*no módulo os*), 721
CLONE_SYSVSEM (*no módulo os*), 721
CLONE_THREAD (*no módulo os*), 721
CLONE_VM (*no módulo os*), 721
cloneNode() (*método xml.dom.Node*), 1462
close() (*método asyncio.BaseTransport*), 1215
close() (*método asyncio.loop*), 1187
close() (*método asyncio.Runner*), 1140
close() (*método asyncio.Server*), 1205
close() (*método asyncio.StreamWriter*), 1165
close() (*método asyncio.SubprocessTransport*), 1218
close() (*método concurrent.interpreters.Interpreter*), 1102
close() (*método contextlib.ExitStack*), 2095
close() (*método dbm.dumb.dumbdbm*), 566
close() (*método dbm.gnu.gdbm*), 563
close() (*método dbm.ndbm.ndbm*), 564
close() (*método email.parser.BytesFeedParser*), 1343
close() (*método ftplib.FTP*), 1555
close() (*método html.parser.HTMLParser*), 1435
close() (*método http.client.HTTPConnection*), 1547
close() (*método imaplib.IMAP4*), 1562
close() (*método io.IOBase*), 785
close() (*método logging.FileHandler*), 839
close() (*método logging.Handler*), 815
close() (*método logging.handlers.MemoryHandler*), 849
close() (*método logging.handlers.NTEventLogHandler*), 847
close() (*método logging.handlers.SocketHandler*), 844
close() (*método logging.handlers.SysLogHandler*), 846
close() (*método mailbox.Mailbox*), 1404
close() (*método mailbox.Maildir*), 1407
close() (*método mailbox.MH*), 1409
close() (*método mmap.mmap*), 1328
close() (*método multiprocessing.connection.Connection*), 1055

close() (*método multiprocessing.connection.Listener*), 1072
close() (*método multiprocessing.pool.Pool*), 1070
close() (*método multiprocessing.Process*), 1049
close() (*método multiprocessing.Queue*), 1051
close() (*método multiprocessing.shared_memory.SharedMemory*), 1085
close() (*método multiprocessing.SimpleQueue*), 1052
close() (*método os.scandir*), 745
close() (*método select.devpoll*), 1308
close() (*método select.epoll*), 1309
close() (*método select.kqueue*), 1310
close() (*método selectors.BaseSelector*), 1315
close() (*método shelve.Shelf*), 556
close() (*método socket.socket*), 1259
close() (*método sqlite3.Blob*), 586
close() (*método sqlite3.Connection*), 573
close() (*método sqlite3.Cursor*), 584
close() (*método tarfile.TarFile*), 648
close() (*método urllib.request.BaseHandler*), 1519
close() (*método wave.Wave_read*), 1640
close() (*método wave.Wave_write*), 1641
Close() (*método winreg.PyHKEY*), 2302
close() (*método xml.etree.ElementTree.TreeBuilder*), 1455
close() (*método xml.etree.ElementTree.XMLParser*), 1456
close() (*método xml.etree.ElementTree.XMLPullParser*), 1457
close() (*método xml.sax.xmlreader.IncrementalParser*), 1484
close() (*método zipfile.ZipFile*), 632
close() (*no módulo fileinput*), 982
close() (*no módulo os*), 722
close() (*no módulo socket*), 1254
close_clients() (*método asyncio.Server*), 1205
close_connection (*atributo http.server.BaseHTTPRequestHandler*), 1591
CloseBoundaryNotFoundDefect, 1355
closed (*atributo http.client.HTTPResponse*), 1549
closed (*atributo io.IOBase*), 785
closed (*atributo mmap.mmap*), 1328
closed (*atributo select.devpoll*), 1308
closed (*atributo select.epoll*), 1309
closed (*atributo select.kqueue*), 1310
CloseKey() (*no módulo winreg*), 2294
closelog() (*no módulo syslog*), 2327
closerange() (*no módulo os*), 722
closing() (*no módulo contextlib*), 2088
clrtobot() (*método curses.window*), 993
clrtoeol() (*método curses.window*), 993
cmath
    module, 370
cmd
    module, 1015
    módulo, 1964
cmd (*atributo subprocess.CalledProcessError*), 1107
cmd (*atributo subprocess.TimeoutExpired*), 1106
Cmd (*classe em cmd*), 1015
cmdloop() (*método cmd.Cmd*), 1016
cmdqueue (*atributo cmd.Cmd*), 1017
cmp() (*no módulo filecmp*), 512
cmp_op (*no módulo dis*), 2288
cmp_to_key() (*no módulo functools*), 454
cmpfiles() (*no módulo filecmp*), 512
CMSG_LEN() (*no módulo socket*), 1257
CMSG_SPACE() (*no módulo socket*), 1257
CO_ASYNC_GENERATOR (*no módulo inspect*), 2141
CO_COROUTINE (*no módulo inspect*), 2141
CO_GENERATOR (*no módulo inspect*), 2141
CO_HAS_DOCSTRING (*no módulo inspect*), 2141
CO_ITERABLE_COROUTINE (*no módulo inspect*), 2141
CO_METHOD (*no módulo inspect*), 2141
CO_NESTED (*no módulo inspect*), 2141
CO_NEWLOCALS (*no módulo inspect*), 2141
CO_OPTIMIZED (*no módulo inspect*), 2141
CO_VARARGS (*no módulo inspect*), 2141
CO_VARKEYWORDS (*no módulo inspect*), 2141
code
    module, 2155
code (*atributo SystemExit*), 122
code (*atributo urllib.error.HTTPError*), 1538
code (*atributo urllib.response.addinfourl*), 1528
code (*atributo xml.etree.ElementTree.ParseError*), 1458
code (*atributo xml.parsers.expat.ExpatError*), 1492
code_context (*atributo inspect.FrameInfo*), 2137
code_context (*atributo inspect.Traceback*), 2137
code_info() (*no módulo dis*), 2268
Codec (*classe em codecs*), 210
CodecInfo (*classe em codecs*), 205
CodecRegistryError, 221
Codecs, 204
    decode, 204
    encode, 204
codecs
    module, 204
coded_value (*atributo http.cookies.Morsel*), 1599
codeop
    module, 2157
codepoint2name (*no módulo html.entities*), 1438
codes (*no módulo xml.parsers.expat.errors*), 1493
CODESET (*no módulo locale*), 1653
CodeType (*classe em types*), 322
codificação
    base64, 1425
    imprimíveis entre aspas, 1431
codificação da localidade, **2355**
codificador de texto, **2361**
col_offset (*atributo ast.AST*), 2207
coleta de lixo, **2352**
collapse_addresses() (*no módulo ipaddress*), 1636
collapse_rfc2231_value() (*no módulo email.utils*), 1389
collect() (*no módulo gc*), 2119

collectedDurations (*atributo unittest.TestResult*), 1862
Collection (*classe em collections.abc*), 297
Collection (*classe em typing*), 1806
collections
    module, 275
collections.abc
    module, 294
colno (*atributo json.JSONDecodeError*), 1398
colno (*atributo re.PatternError*), 161
colno (*atributo tomllib.TOMLDecodeError*), 687
colno (*atributo traceback.FrameSummary*), 2113
colon (*atributo mailbox.Maildir*), 1405
COLON (*no módulo token*), 2249
COLONEQUAL (*no módulo token*), 2252
color() (*no módulo turtle*), 1730
COLOR_BLACK (*no módulo curses*), 1009
COLOR_BLUE (*no módulo curses*), 1009
color_content() (*no módulo curses*), 985
COLOR_CYAN (*no módulo curses*), 1009
COLOR_GREEN (*no módulo curses*), 1009
COLOR_MAGENTA (*no módulo curses*), 1009
color_pair() (*no módulo curses*), 985
COLOR_PAIRS (*no módulo curses*), 998
COLOR_RED (*no módulo curses*), 1009
COLOR_WHITE (*no módulo curses*), 1009
COLOR_YELLOW (*no módulo curses*), 1009
colormode() (*no módulo turtle*), 1742
COLORS (*no módulo curses*), 998
colorsys
    module, 1642
COLS (*no módulo curses*), 998
column() (*método tkinter.ttk.Treeview*), 1693
columnize() (*método cmd.Cmd*), 1016
COLUMNS, 991
columns (*atributo os.terminal_size*), 734
comb() (*no módulo math*), 362
combinations() (*no módulo itertools*), 438
combinations_with_replacement() (*no módulo itertools*), 439
combine() (*método de classe datetime.datetime*), 237
combining() (*no módulo unicodedata*), 188
Combobox (*classe em tkinter.ttk*), 1687
COMError, 903
COMMA (*no módulo token*), 2249
--command
    pdb opção de linha de comando, 1965
command (*atributo http.server.BaseHTTPRequestHandler*), 1591
CommandCompiler (*classe em codeop*), 2158
commands (*pdb command*), 1970
comment (*atributo http.cookiejar.Cookie*), 1607
comment (*atributo http.cookies.Morsel*), 1598
comment (*atributo zipfile.ZipFile*), 635
comment (*atributo zipfile.ZipInfo*), 639
COMMENT (*no módulo token*), 2248
comment() (*método xml.etree.ElementTree.TreeBuilder*), 1455
comment() (*método xml.sax.handler.LexicalHandler*), 1481
Comment() (*no módulo xml.etree.ElementTree*), 1447
comment_url (*atributo http.cookiejar.Cookie*), 1607
commenters (*atributo shlex.shlex*), 2309
CommentHandler() (*método xml.parsers.expat.xmlparser*), 1491
commit() (*método sqlite3.Connection*), 573
common (*atributo filecmp.dircmp*), 513
Common Vulnerabilities and Exposures
    CVE 2020-10735, 112
    CVE 2023-52425, 1440
Common Weakness Enumeration
    CWE 257, 708
common_dirs (*atributo filecmp.dircmp*), 513
common_files (*atributo filecmp.dircmp*), 513
common_funny (*atributo filecmp.dircmp*), 513
common_types (*no módulo mimetypes*), 1422
commonpath() (*no módulo os.path*), 500
commonprefix() (*no módulo os.path*), 500
communicate() (*método asyncio.subprocess.Process*), 1177
communicate() (*método subprocess.Popen*), 1114
--compact
    json opção de linha de comando, 1401
comparação
    operador, 40
comparações
    encadeamento, 40
comparando
    objetos, 40
Compare (*classe em ast*), 2214
compare() (*método decimal.Context*), 390
compare() (*método decimal.Decimal*), 380
compare() (*método difflib.Differ*), 179
compare() (*no módulo ast*), 2242
compare_digest() (*no módulo hmac*), 706
compare_digest() (*no módulo secrets*), 708
compare_networks() (*método ipaddress.IPv4Network*), 1632
compare_networks() (*método ipaddress.IPv6Network*), 1633
COMPARE_OP (*opcode*), 2280
compare_signal() (*método decimal.Context*), 390
compare_signal() (*método decimal.Decimal*), 380
compare_to() (*método tracemalloc.Snapshot*), 1998
compare_total() (*método decimal.Context*), 390
compare_total() (*método decimal.Decimal*), 380
compare_total_mag() (*método decimal.Context*), 390
compare_total_mag() (*método decimal.Decimal*), 381
COMPARISON_FLAGS (*no módulo doctest*), 1822
Compat32 (*classe em email.policy*), 1354
compat32 (*no módulo email.policy*), 1355
compile
    função embutida, 111, 322
Compile (*classe em codeop*), 2158

compile()
    built-in function, 11
compile() (*no módulo py_compile*), 2259
compile() (*no módulo re*), 156
compile_command() (*no módulo code*), 2156
compile_command() (*no módulo codeop*), 2158
compile_dir() (*no módulo compileall*), 2263
compile_file() (*no módulo compileall*), 2264
compile_path() (*no módulo compileall*), 2264
compileall
    module, 2261
compileall opção de linha de comando
    -b, 2262
    -d, 2261
    directory, 2261
    -e, 2262
    -f, 2261
    file, 2261
    --hardlink-dupes, 2262
    -i, 2262
    --invalidation-mode, 2262
    -j, 2262
    -l, 2261
    -o, 2262
    -p, 2262
    -q, 2261
    -r, 2262
    -s, 2262
    -x, 2262
compiler_flag (*atributo __future__._Feature*), 2119
complete() (*método rlcompleter.Completer*), 196
complete_statement() (*no módulo sqlite3*), 569
completedefault() (*método cmd.Cmd*), 1016
CompletedProcess (*classe em subprocess*), 1105
Completer (*classe em rlcompleter*), 196
Complex (*classe em numbers*), 357
complex (*classe interna*), 12
complexo
    função embutida, 41
compreensão de conjunto, **2360**
compreensão de dicionário, **2350**
compreensão de lista, **2355**
comprehension (*classe em ast*), 2217
--compress
    zipapp opção de linha de comando, 2016
compress() (*método bz2.BZ2Compressor*), 621
compress() (*método compression.zstd.ZstdCompressor*), 602
compress() (*método lzma.LZMACompressor*), 625
compress() (*método zlib.Compress*), 613
compress() (*no módulo bz2*), 622
compress() (*no módulo compression.zstd*), 601
compress() (*no módulo gzip*), 617
compress() (*no módulo itertools*), 439
compress() (*no módulo lzma*), 627
compress() (*no módulo zlib*), 611
compress_size (*atributo zipfile.ZipInfo*), 640
compress_type (*atributo zipfile.ZipInfo*), 639
compressed (*atributo ipaddress.IPv4Address*), 1624
compressed (*atributo ipaddress.IPv4Network*), 1630
compressed (*atributo ipaddress.IPv6Address*), 1626
compressed (*atributo ipaddress.IPv6Network*), 1633
compression() (*método ssl.SSLSocket*), 1285
compression_level (*atributo compression.zstd.CompressionParameter*), 606
COMPRESSION_LEVEL_DEFAULT (*no módulo compression.zstd*), 610
CompressionError, 643
CompressionParameter (*classe em compression.zstd*), 605
compression.zstd
    module, 599
compressobj() (*no módulo zlib*), 612
COMSPEC, 771, 1109
concat() (*no módulo operator*), 465
concatenação
    operação, 49
Concatenate (*no módulo typing*), 1770
concurrent.futures
    module, 1091
concurrent.interpreters
    module, 1099
cond (*atributo bdb.Breakpoint*), 1956
Condition (*classe em asyncio*), 1171
Condition (*classe em multiprocessing*), 1057
Condition (*classe em threading*), 1033
condition (*pdb command*), 1970
Condition() (*método multiprocessing.managers.SyncManager*), 1063
config() (*método tkinter.font.Font*), 1675
configparser
    module, 667
ConfigParser (*classe em configparser*), 680
configuração
    arquivo, 667
    arquivo, caminho, 2152
    arquivo, depurador, 1969
configure() (*método tkinter.ttk.Style*), 1696
configure_mock() (*método unittest.mock.Mock*), 1874
CONFORM (*atributo enum.FlagBoundary*), 348
confstr() (*no módulo os*), 777
confstr_names (*no módulo os*), 777
conjugate() (*método de número complexo*), 41
conjugate() (*método decimal.Decimal*), 381
conjugate() (*método numbers.Complex*), 357
connect() (*método ftplib.FTP*), 1552
connect() (*método http.client.HTTPConnection*), 1547
connect() (*método multiprocessing.managers.BaseManager*), 1062
connect() (*método smtplib.SMTP*), 1570
connect() (*método socket.socket*), 1260
connect() (*no módulo sqlite3*), 568
connect_accepted_socket() (*método asyncio.loop*), 1195
connect_ex() (*método socket.socket*), 1260

connect_read_pipe() (*método asyncio.loop*), 1199
connect_write_pipe() (*método asyncio.loop*), 1199
connection (*atributo sqlite3.Cursor*), 585
Connection (*classe em multiprocessing.connection*), 1055
Connection (*classe em sqlite3*), 572
connection_lost() (*método asyncio.BaseProtocol*), 1218
connection_made() (*método asyncio.BaseProtocol*), 1218
ConnectionAbortedError, 123
ConnectionError, 123
ConnectionRefusedError, 123
ConnectionResetError, 123
ConnectRegistry() (*no módulo winreg*), 2294
considerações de segurança, 2341
const (*atributo optparse.Option*), 966
Constant (*classe em ast*), 2209
constructor() (*no módulo copyreg*), 555
consumed (*atributo asyncio.LimitOverrunError*), 1183
contagem de referências, **2360**
Container (*classe em collections.abc*), 297
Container (*classe em typing*), 1806
contains() (*no módulo operator*), 466
CONTAINS_OP (*opcode*), 2280
contêiner
    iteração por, 48
content (*atributo urllib.error.ContentTooShortError*), 1538
content type
    MIME, 1420
content_disposition (*atributo email.headerregistry.ContentDispositionHeader*), 1359
content_manager (*atributo email.policy.EmailPolicy*), 1352
content_size_flag (*atributo compression.zstd.CompressionParameter*), 607
content_type (*atributo email.headerregistry.ContentTypeHeader*), 1359
ContentDispositionHeader (*classe em email.headerregistry*), 1359
ContentHandler (*classe em xml.sax.handler*), 1476
ContentManager (*classe em email.contentmanager*), 1362
contents (*atributo ctypes._Pointer*), 903
contents() (*método importlib.abc.ResourceReader*), 2176
contents() (*método importlib.resources.abc.ResourceReader*), 2192
contents() (*no módulo importlib.resources*), 2191
ContentTooShortError, 1538
ContentTransferEncoding (*classe em email.headerregistry*), 1359
ContentTypeHeader (*classe em email.headerregistry*), 1359
context (*atributo ssl.SSLSocket*), 1287
Context (*classe em contextvars*), 1130
Context (*classe em decimal*), 388
context_aware_warnings (*atributo sys.flags*), 2028
context_diff() (*no módulo difflib*), 172
ContextDecorator (*classe em contextlib*), 2091
contextlib
    module, 2086
ContextManager (*classe em typing*), 1808
contextmanager() (*no módulo contextlib*), 2086
contexto, **2348**
contexto atual, **2349**
ContextVar (*classe em contextvars*), 1129
contextvars
    module, 1128
CONTIG (*atributo inspect.BufferFlags*), 2142
CONTIG_RO (*atributo inspect.BufferFlags*), 2142
contíguo, **2349**
contíguo C, 2349
contíguo Fortran, 2349
contiguous (*atributo memoryview*), 95
CONTINUE (*atributo compression.zstd.ZstdCompressor*), 602
Continue (*classe em ast*), 2223
continue (*pdb command*), 1971
CONTINUOUS (*atributo enum.EnumCheck*), 347
control() (*método select.kqueue*), 1311
controle de arquivo
    UNIX, 2319
controle de E/S
    buffering, 26, 1261
    POSIX, 2315
    tty, 2315
    UNIX, 2319
controlnames (*no módulo curses.ascii*), 1014
CONTTYPE (*no módulo tarfile*), 644
conversion (*atributo string.templatelib.Interpolation*), 145
conversões
    numérico, 42
convert() (*no módulo string.templatelib*), 146
convert_arg_line_to_args() (*método argparse.ArgumentParser*), 935
convert_field() (*método string.Formatter*), 131
CONVERT_VALUE (*opcode*), 2284
Cookie (*classe em http.cookiejar*), 1601
CookieError, 1597
cookiejar (*atributo urllib.request.HTTPCookieProcessor*), 1521
CookieJar (*classe em http.cookiejar*), 1601
CookiePolicy (*classe em http.cookiejar*), 1601
cópia de arquivos, 524
--copies
    venv opção de linha de comando, 2006
Copy, 1705
copy
    module, 326
    módulo, 554
    protocolo, 545

COPY (*opcode*), 2272
copy() (*método collections.deque*), 282
copy() (*método contextvars.Context*), 1131
copy() (*método decimal.Context*), 389
copy() (*método dict*), 100
copy() (*método frozenset*), 96
copy() (*método hashlib.hash*), 696
copy() (*método hmac.HMAC*), 706
copy() (*método http.cookies.Morsel*), 1599
copy() (*método imaplib.IMAP4*), 1562
copy() (*método pathlib.Path*), 494
copy() (*método sequence*), 52
copy() (*método tkinter.font.Font*), 1675
copy() (*método types.MappingProxyType*), 324
copy() (*método zlib.Compress*), 613
copy() (*método zlib.Decompress*), 614
copy() (*no módulo copy*), 326
copy() (*no módulo multiprocessing.sharedctypes*), 1060
copy() (*no módulo shutil*), 526
copy2() (*no módulo shutil*), 527
copy_abs() (*método decimal.Context*), 390
copy_abs() (*método decimal.Decimal*), 381
copy_context() (*no módulo contextvars*), 1130
copy_decimal() (*método decimal.Context*), 389
copy_file_range() (*no módulo os*), 722
COPY_FREE_VARS (*opcode*), 2282
copy_into() (*método pathlib.Path*), 494
copy_location() (*no módulo ast*), 2239
copy_negate() (*método decimal.Context*), 390
copy_negate() (*método decimal.Decimal*), 381
copy_sign() (*método decimal.Context*), 390
copy_sign() (*método decimal.Decimal*), 381
CopyComPointer() (*no módulo ctypes*), 892
copyfile() (*no módulo shutil*), 525
copyfileobj() (*no módulo shutil*), 525
copymode() (*no módulo shutil*), 526
copyreg
    module, 554
copyright (*no módulo sys*), 2023
copyright (*variável interna*), 38
copysign() (*no módulo math*), 364
copystat() (*no módulo shutil*), 526
copytree() (*no módulo shutil*), 527
Coroutine (*classe em collections.abc*), 298
Coroutine (*classe em typing*), 1807
coroutine() (*no módulo types*), 325
CoroutineType (*no módulo types*), 321
correlation() (*no módulo statistics*), 427
corrotina, **2349**
cos() (*no módulo cmath*), 372
cos() (*no módulo math*), 368
cosh() (*no módulo cmath*), 372
cosh() (*no módulo math*), 368
--count
    trace opção de linha de comando, 1989
    uuid opção de linha de comando, 1579
count (*atributo tracemalloc.Statistic*), 1999
count (*atributo tracemalloc.StatisticDiff*), 2000
count() (*método array.array*), 310
count() (*método bytearray*), 74
count() (*método bytes*), 74
count() (*método collections.deque*), 282
count() (*método memoryview*), 93
count() (*método multiprocessing.shared_memory.ShareableList*), 1089
count() (*método sequence*), 51
count() (*método str*), 58
count() (*no módulo itertools*), 440
count_diff (*atributo tracemalloc.StatisticDiff*), 2000
Counter (*classe em collections*), 278
Counter (*classe em typing*), 1805
countOf() (*no módulo operator*), 466
countTestCases() (*método unittest.TestCase*), 1856
countTestCases() (*método unittest.TestSuite*), 1859
covariance() (*no módulo statistics*), 427
CoverageResults (*classe em trace*), 1990
--coverdir
    trace opção de linha de comando, 1989
cProfile
    module, 1978
cProfile opção de linha de comando
    -m, 1977
    -o, 1977
    -s, 1977
cpu_count() (*no módulo multiprocessing*), 1053
cpu_count() (*no módulo os*), 778
CPython, **2349**
cpython_only() (*no módulo test.support*), 1939
CR (*no módulo curses.ascii*), 1011
crawl_delay() (*método urllib.robotparser.RobotFileParser*), 1539
CRC (*atributo zipfile.ZipInfo*), 639
crc32() (*no módulo binascii*), 1430
crc32() (*no módulo zlib*), 612
crc_hqx() (*no módulo binascii*), 1430
--create
    tarfile opção de linha de comando, 655
    zipfile opção de linha de comando, 640
create() (*método imaplib.IMAP4*), 1562
create() (*método venv.EnvBuilder*), 2009
create() (*no módulo concurrent.interpreters*), 1102
create() (*no módulo venv*), 2011
create_aggregate() (*método sqlite3.Connection*), 574
create_archive() (*no módulo zipapp*), 2016
create_autospec() (*no módulo unittest.mock*), 1903
CREATE_BREAKAWAY_FROM_JOB (*no módulo subprocess*), 1118
create_collation() (*método sqlite3.Connection*), 576
create_configuration() (*método venv.EnvBuilder*), 2010
create_connection() (*método asyncio.loop*), 1190
create_connection() (*no módulo socket*), 1252
create_datagram_endpoint() (*método asyncio.loop*), 1192

create_decimal() (*método decimal.Context*), 389
create_decimal_from_float() (*método decimal.Context*), 389
create_default_context() (*no módulo ssl*), 1272
CREATE_DEFAULT_ERROR_MODE (*no módulo subprocess*), 1117
create_eager_task_factory() (*no módulo asyncio*), 1149
create_empty_file() (*no módulo test.support.os_helper*), 1946
create_function() (*método sqlite3.Connection*), 573
create_future() (*método asyncio.loop*), 1189
create_git_ignore_file() (*método venv.EnvBuilder*), 2010
create_module() (*método importlib.abc.Loader*), 2171
create_module() (*método importlib.machinery.ExtensionFileLoader*), 2180
create_module() (*método zipimport.zipimporter*), 2160
CREATE_NEW_CONSOLE (*no módulo subprocess*), 1117
CREATE_NEW_PROCESS_GROUP (*no módulo subprocess*), 1117
CREATE_NO_WINDOW (*no módulo subprocess*), 1117
create_queue() (*no módulo concurrent.interpreters*), 1102
create_server() (*método asyncio.loop*), 1193
create_server() (*no módulo socket*), 1252
create_stats() (*método profile.Profile*), 1979
create_string_buffer() (*no módulo ctypes*), 892
create_subprocess_exec() (*no módulo asyncio*), 1175
create_subprocess_shell() (*no módulo asyncio*), 1175
create_system (*atributo zipfile.ZipInfo*), 639
create_task() (*método asyncio.loop*), 1189
create_task() (*método asyncio.TaskGroup*), 1145
create_task() (*no módulo asyncio*), 1144
create_unicode_buffer() (*no módulo ctypes*), 893
create_unix_connection() (*método asyncio.loop*), 1193
create_unix_server() (*método asyncio.loop*), 1194
create_version (*atributo zipfile.ZipInfo*), 639
create_window_function() (*método sqlite3.Connection*), 575
createAttribute() (*método xml.dom.Document*), 1463
createAttributeNS() (*método xml.dom.Document*), 1464
createComment() (*método xml.dom.Document*), 1463
createDocument() (*método xml.dom.DOMImplementation*), 1460
createDocumentType() (*método xml.dom.DOMImplementation*), 1461
createElement() (*método xml.dom.Document*), 1463
createElementNS() (*método xml.dom.Document*), 1463
createfilehandler() (*método _tkinter.Widget.tk*), 1674
CreateKey() (*no módulo winreg*), 2294
CreateKeyEx() (*no módulo winreg*), 2294
createLock() (*método logging.Handler*), 814
createLock() (*método logging.NullHandler*), 840
createProcessingInstruction() (*método xml.dom.Document*), 1463
createSocket() (*método logging.handlers.SocketHandler*), 844
createSocket() (*método logging.handlers.SysLogHandler*), 846
createTextNode() (*método xml.dom.Document*), 1463
credits (*variável interna*), 38
criptografia, 693
CRITICAL (*no módulo logging*), 814
critical() (*método logging.Logger*), 813
critical() (*no módulo logging*), 822
CRNCYSTR (*no módulo locale*), 1654
CRT_ASSEMBLY_VERSION (*no módulo msvcrt*), 2293
CRT_ASSERT (*no módulo msvcrt*), 2293
CRT_ERROR (*no módulo msvcrt*), 2293
CRT_WARN (*no módulo msvcrt*), 2293
CRTDBG_MODE_DEBUG (*no módulo msvcrt*), 2293
CRTDBG_MODE_FILE (*no módulo msvcrt*), 2293
CRTDBG_MODE_WNDW (*no módulo msvcrt*), 2293
CRTDBG_REPORT_MODE (*no módulo msvcrt*), 2293
CrtSetReportFile() (*no módulo msvcrt*), 2293
CrtSetReportMode() (*no módulo msvcrt*), 2293
crypt
    module, 2338
--css
    calendar opção de linha de comando, 275
cssclass_month (*atributo calendar.HTMLCalendar*), 270
cssclass_month_head (*atributo calendar.HTMLCalendar*), 270
cssclass_noday (*atributo calendar.HTMLCalendar*), 270
cssclass_year (*atributo calendar.HTMLCalendar*), 270
cssclass_year_head (*atributo calendar.HTMLCalendar*), 270
cssclasses (*atributo calendar.HTMLCalendar*), 270
cssclasses_weekday_head (*atributo calendar.HTMLCalendar*), 270
csv, 659
    module, 659
cte (*atributo email.headerregistry.ContentTransferEncoding*), 1359
cte_type (*atributo email.policy.Policy*), 1350
ctermid() (*no módulo os*), 713
ctime() (*método datetime.date*), 233
ctime() (*método datetime.datetime*), 244
ctime() (*no módulo time*), 797
ctrl() (*no módulo curses.ascii*), 1013
CTRL_BREAK_EVENT (*no módulo signal*), 1319

CTRL_C_EVENT (*no módulo signal*), 1319
ctypes
    module, 865
curdir (*no módulo os*), 778
currency() (*no módulo locale*), 1656
current() (*método tkinter.ttk.Combobox*), 1687
current_process() (*no módulo multiprocessing*), 1053
current_task() (*no módulo asyncio*), 1157
current_thread() (*no módulo threading*), 1023
CurrentByteIndex (*atributo xml.parsers.expat.xmlparser*), 1490
CurrentColumnNumber (*atributo xml.parsers.expat.xmlparser*), 1490
currentframe() (*no módulo inspect*), 2138
CurrentLineNumber (*atributo xml.parsers.expat.xmlparser*), 1490
curs_set() (*no módulo curses*), 985
curses
    module, 983
curses.ascii
    module, 1010
curses.panel
    module, 1014
curses.textpad
    module, 1009
Cursor (*classe em sqlite3*), 583
cursor() (*método sqlite3.Connection*), 572
cursyncup() (*método curses.window*), 993
Cut, 1705
cwd() (*método de classe pathlib.Path*), 485
cwd() (*método ftplib.FTP*), 1554
cycle() (*no módulo itertools*), 440
CycleError, 355

D

-d
    compileall opção de linha de comando, 2261
    gzip opção de linha de comando, 618
    http.server opção de linha de comando, 1595
    idle opção de linha de comando, 1708
D_FMT (*no módulo locale*), 1653
D_T_FMT (*no módulo locale*), 1653
dados
    packing binário, 197
    tabular, 659
daemon (*atributo multiprocessing.Process*), 1047
daemon (*atributo threading.Thread*), 1030
daemon_threads (*atributo socketserver.ThreadingMixIn*), 1582
data (*atributo collections.UserDict*), 293
data (*atributo collections.UserList*), 293
data (*atributo collections.UserString*), 293
data (*atributo plistlib.UID*), 691
data (*atributo select.kevent*), 1312
data (*atributo selectors.SelectorKey*), 1314
data (*atributo urllib.request.Request*), 1517
data (*atributo xml.dom.Comment*), 1465
data (*atributo xml.dom.ProcessingInstruction*), 1466
data (*atributo xml.dom.Text*), 1465
data (*atributo xmlrpc.client.Binary*), 1612
data() (*método xml.etree.ElementTree.TreeBuilder*), 1455
data_filter() (*no módulo tarfile*), 652
data_open() (*método urllib.request.DataHandler*), 1523
data_received() (*método asyncio.Protocol*), 1219
database
    Unicode, 187
DatabaseError, 587
dataclass() (*no módulo dataclasses*), 2076
dataclass_transform() (*no módulo typing*), 1796
dataclasses
    module, 2075
DataError, 587
datagram_received() (*método asyncio.DatagramProtocol*), 1220
DatagramHandler (*classe em logging.handlers*), 845
DatagramProtocol (*classe em asyncio*), 1218
DatagramRequestHandler (*classe em socketserver*), 1585
DatagramTransport (*classe em asyncio*), 1214
DataHandler (*classe em urllib.request*), 1516
date (*classe em datetime*), 229
date() (*método datetime.datetime*), 240
date_time (*atributo zipfile.ZipInfo*), 638
date_time_string() (*método http.server.BaseHTTPRequestHandler*), 1593
DateHeader (*classe em email.headerregistry*), 1358
datetime
    module, 223
datetime (*atributo email.headerregistry.DateHeader*), 1358
datetime (*classe em datetime*), 235
DateTime (*classe em xmlrpc.client*), 1612
day (*atributo datetime.date*), 231
day (*atributo datetime.datetime*), 239
Day (*classe em calendar*), 272
DAY_1 (*no módulo locale*), 1653
DAY_2 (*no módulo locale*), 1653
DAY_3 (*no módulo locale*), 1653
DAY_4 (*no módulo locale*), 1653
DAY_5 (*no módulo locale*), 1653
DAY_6 (*no módulo locale*), 1653
DAY_7 (*no módulo locale*), 1653
day_abbr (*no módulo calendar*), 272
day_name (*no módulo calendar*), 272
daylight (*no módulo time*), 807
days (*atributo datetime.timedelta*), 227
DbfilenameShelf (*classe em shelve*), 557
dbm
    module, 560
dbm.dumb

module, 565
dbm.gnu
module, 562
módulo, 556
dbm.ndbm
module, 564
módulo, 556
dbm.sqlite3
module, 562
DC1 (*no módulo curses.ascii*), 1012
DC2 (*no módulo curses.ascii*), 1012
DC3 (*no módulo curses.ascii*), 1012
DC4 (*no módulo curses.ascii*), 1012
dcgettext() (*no módulo locale*), 1659
deactivate_stack_trampoline() (*no módulo sys*), 2045
debug (*atributo imaplib.IMAP4*), 1567
debug (*atributo shlex.shlex*), 2310
debug (*atributo sys.flags*), 2028
debug (*atributo zipfile.ZipFile*), 635
DEBUG (*no módulo logging*), 814
DEBUG (*no módulo re*), 155
debug (*pdb command*), 1974
debug() (*método logging.Logger*), 811
debug() (*método unittest.TestCase*), 1848
debug() (*método unittest.TestSuite*), 1859
debug() (*no módulo doctest*), 1835
debug() (*no módulo logging*), 822
DEBUG_BYTECODE_SUFFIXES (*no módulo importlib.machinery*), 2177
DEBUG_COLLECTABLE (*no módulo gc*), 2123
DEBUG_LEAK (*no módulo gc*), 2123
DEBUG_SAVEALL (*no módulo gc*), 2123
debug_src() (*no módulo doctest*), 1835
DEBUG_STATS (*no módulo gc*), 2123
DEBUG_UNCOLLECTABLE (*no módulo gc*), 2123
debuglevel (*atributo http.client.HTTPResponse*), 1548
DebugRunner (*classe em doctest*), 1836
DECEMBER (*no módulo calendar*), 273
decimal
module, 374
Decimal (*classe em decimal*), 378
decimal() (*no módulo unicodedata*), 188
DecimalException (*classe em decimal*), 394
decode
Codecs, 204
decode (*atributo codecs.CodecInfo*), 205
decode() (*método bytearray*), 75
decode() (*método bytes*), 75
decode() (*método codecs.Codec*), 210
decode() (*método codecs.IncrementalDecoder*), 211
decode() (*método json.JSONDecoder*), 1396
decode() (*método xmlrpc.client.Binary*), 1613
decode() (*método xmlrpc.client.DateTime*), 1612
decode() (*no módulo base64*), 1428
decode() (*no módulo codecs*), 205
decode() (*no módulo quopri*), 1431
decode_header() (*no módulo email.header*), 1383
decode_params() (*no módulo email.utils*), 1389
decode_rfc2231() (*no módulo email.utils*), 1389
decode_source() (*no módulo importlib.util*), 2183
decodebytes() (*no módulo base64*), 1428
DecodedGenerator (*classe em email.generator*), 1347
decodestring() (*no módulo quopri*), 1431
decomposition() (*no módulo unicodedata*), 189
--decompress
gzip opção de linha de comando, 618
decompress() (*método bz2.BZ2Decompressor*), 621
decompress() (*método compression.zstd.ZstdDecompressor*), 603
decompress() (*método lzma.LZMADecompressor*), 626
decompress() (*método zlib.Decompress*), 614
decompress() (*no módulo bz2*), 622
decompress() (*no módulo compression.zstd*), 601
decompress() (*no módulo gzip*), 617
decompress() (*no módulo lzma*), 627
decompress() (*no módulo zlib*), 612
decompressed_size (*atributo compression.zstd.FrameInfo*), 609
DecompressionParameter (*classe em compression.zstd*), 608
decompressobj() (*no módulo zlib*), 613
decorador, **2349**
DEDENT (*no módulo token*), 2248
dedent() (*no módulo textwrap*), 184
deepcopy() (*no módulo copy*), 326
def_prog_mode() (*no módulo curses*), 985
def_shell_mode() (*no módulo curses*), 985
default (*atributo inspect.Parameter*), 2132
default (*atributo optparse.Option*), 966
default (*no módulo email.policy*), 1353
DEFAULT (*no módulo unittest.mock*), 1901
default() (*método cmd.Cmd*), 1016
default() (*método json.JSONEncoder*), 1397
DEFAULT_BUFFER_SIZE (*no módulo io*), 782
default_bufsize (*no módulo xml.dom.pulldom*), 1474
default_exception_handler() (*método asyncio.loop*), 1202
default_factory (*atributo collections.defaultdict*), 285
DEFAULT_FORMAT (*no módulo tarfile*), 645
DEFAULT_IGNORES (*no módulo filecmp*), 514
default_loader() (*no módulo xml.etree.ElementInclude*), 1451
default_max_str_digits (*atributo sys.int_info*), 2036
default_open() (*método urllib.request.BaseHandler*), 1519
DEFAULT_PROTOCOL (*no módulo pickle*), 539
DEFAULT_TIMEOUT (*atributo unittest.mock.ThreadingMock*), 1885
default_timer() (*no módulo timeit*), 1984
DefaultContext (*no módulo decimal*), 387

DefaultCookiePolicy (*classe em http.cookiejar*), 1601
defaultdict (*classe em collections*), 285
DefaultDict (*classe em typing*), 1804
DefaultEventLoopPolicy (*classe em asyncio*), 1227
DefaultHandler() (*método xml.parsers.expat.xmlparser*), 1491
DefaultHandlerExpand() (*método xml.parsers.expat.xmlparser*), 1491
--default-pip
    ensurepip opção de linha de comando, 2004
defaults() (*método configparser.ConfigParser*), 681
DefaultSelector (*classe em selectors*), 1315
defaultTestLoader (*no módulo unittest*), 1864
defaultTestResult() (*método unittest.TestCase*), 1856
defects (*atributo email.headerregistry.BaseHeader*), 1357
defects (*atributo email.message.EmailMessage*), 1342
defects (*atributo email.message.Message*), 1379
defpath (*no módulo os*), 779
DefragResult (*classe em urllib.parse*), 1535
DefragResultBytes (*classe em urllib.parse*), 1535
degrees() (*no módulo math*), 367
degrees() (*no módulo turtle*), 1726
del
    instrução, 51, 98
Del (*classe em ast*), 2212
DEL (*no módulo curses.ascii*), 1012
del_param() (*método email.message.EmailMessage*), 1338
del_param() (*método email.message.Message*), 1377
delattr()
    built-in function, 13
delay() (*no módulo turtle*), 1739
delay_output() (*no módulo curses*), 985
delayload (*atributo http.cookiejar.FileCookieJar*), 1603
delch() (*método curses.window*), 993
dele() (*método poplib.POP3*), 1559
Delete (*classe em ast*), 2220
delete() (*método ftplib.FTP*), 1554
delete() (*método imaplib.IMAP4*), 1562
delete() (*método tkinter.ttk.Treeview*), 1694
DELETE_ATTR (*opcode*), 2277
DELETE_DEREF (*opcode*), 2282
DELETE_FAST (*opcode*), 2282
DELETE_GLOBAL (*opcode*), 2277
DELETE_NAME (*opcode*), 2277
DELETE_SUBSCR (*opcode*), 2273
deleteacl() (*método imaplib.IMAP4*), 1562
deletefilehandler() (*método _tkinter.Widget.tk*), 1674
DeleteKey() (*no módulo winreg*), 2295
DeleteKeyEx() (*no módulo winreg*), 2295
deleteln() (*método curses.window*), 993
deleteMe() (*método bdb.Breakpoint*), 1955
DeleteValue() (*no módulo winreg*), 2295
delimiter (*atributo csv.Dialect*), 664
delitem() (*no módulo operator*), 466
deliver_challenge() (*no módulo multiprocessing.connection*), 1071
delocalize() (*no módulo locale*), 1657
demo_app() (*no módulo wsgiref.simple_server*), 1503
denominator (*atributo fractions.Fraction*), 405
denominator (*atributo numbers.Rational*), 358
deprecated() (*no módulo warnings*), 2073
DeprecationWarning, 124
depuração, 1964
depurador, 1024, 1704, 2033, 2043
    configuração arquivo, 1969
deque (*classe em collections*), 282
Deque (*classe em typing*), 1805
dequeue() (*método logging.handlers.QueueListener*), 852
DER_cert_to_PEM_cert() (*no módulo ssl*), 1275
derive() (*método BaseExceptionGroup*), 126
derwin() (*método curses.window*), 993
description (*atributo inspect.Parameter.kind*), 2133
description (*atributo sqlite3.Cursor*), 585
descritor, **2349**
Desempenho, 1983
deserialize() (*método sqlite3.Connection*), 581
desligamento do interpretador, **2354**
deslocamento
    operações, 42
despacho único, **2360**
dest (*atributo optparse.Option*), 966
detach() (*método io.BufferedIOBase*), 787
detach() (*método io.TextIOBase*), 791
detach() (*método socket.socket*), 1260
detach() (*método tkinter.ttk.Treeview*), 1694
detach() (*método weakref.finalize*), 315
Detach() (*método winreg.PyHKEY*), 2302
DETACHED_PROCESS (*no módulo subprocess*), 1117
--details
    inspect opção de linha de comando, 2142
details (*atributo ctypes.COMError*), 903
detect_api_mismatch() (*no módulo test.support*), 1940
detect_encoding() (*no módulo tokenize*), 2254
deterministic profiling, 1975
dev_mode (*atributo sys.flags*), 2028
device_encoding() (*no módulo os*), 722
devmajor (*atributo tarfile.TarInfo*), 650
devminor (*atributo tarfile.TarInfo*), 650
devnull (*no módulo os*), 779
DEVNULL (*no módulo subprocess*), 1106
devpoll() (*no módulo select*), 1306
DevpollSelector (*classe em selectors*), 1315
dfast (*atributo compression.zstd.Strategy*), 609
dgettext() (*no módulo gettext*), 1644
dgettext() (*no módulo locale*), 1659
dialect (*atributo csv.csvreader*), 665

dialect (*atributo csv.csvwriter*), 665
Dialect (*classe em csv*), 662
Dialog (*classe em tkinter.commondialog*), 1679
Dialog (*classe em tkinter.simpledialog*), 1676
dica de tipo, **2362**
dicionário, **2350**
    objeto, 98
    tipo, operações em, 98
Dict (*classe em ast*), 2211
Dict (*classe em typing*), 1803
dict (*classe interna*), 98
dict() (*método multiprocessing.managers.SyncManager*), 1064
dict_content (*atributo compression.zstd.ZstdDict*), 605
dict_id (*atributo compression.zstd.ZstdDict*), 605
dict_id_flag (*atributo compression.zstd.CompressionParameter*), 608
DICT_MERGE (*opcode*), 2279
DICT_UPDATE (*opcode*), 2279
DictComp (*classe em ast*), 2216
dictConfig() (*no módulo logging.config*), 827
dictionary_id (*atributo compression.zstd.FrameInfo*), 609
DictReader (*classe em csv*), 661
DictWriter (*classe em csv*), 661
diff_bytes() (*no módulo difflib*), 174
diff_files (*atributo filecmp.dircmp*), 513
Differ (*classe em difflib*), 171
difference() (*método frozenset*), 96
difference_update() (*método frozenset*), 97
difflib
    module, 171
dig (*atributo sys.float_info*), 2030
digest() (*método hashlib.hash*), 695
digest() (*método hashlib.shake*), 696
digest() (*método hmac.HMAC*), 705
digest() (*no módulo hmac*), 705
digest_size (*atributo hmac.HMAC*), 706
digit() (*no módulo unicodedata*), 188
digits (*no módulo string*), 129
dir()
    built-in function, 13
dir() (*método ftplib.FTP*), 1554
dircmp (*classe em filecmp*), 513
directory
    compileall opção de linha de comando, 2261
--directory
    http.server opção de linha de comando, 1595
Directory (*classe em tkinter.filedialog*), 1677
DirEntry (*classe em os*), 745
diretório
    alteração, 736
    criação, 741
    exclusão, 528, 743
    percorrer, 754, 755
    site-packages, 2151
    traversal, 754, 755
dirname() (*no módulo os.path*), 500
dirs_double_event() (*método tkinter.filedialog.FileDialog*), 1678
dirs_select_event() (*método tkinter.filedialog.FileDialog*), 1678
DirsOnSysPath (*classe em test.support.import_helper*), 1948
DIRTYPE (*no módulo tarfile*), 644
dis
    module, 2265
dis opção de linha de comando
    -C, 2266
    -h, 2266
    --help, 2266
    -O, 2266
    -P, 2266
    -S, 2266
    --show-caches, 2266
    --show-offsets, 2266
    --show-positions, 2266
    --specialized, 2266
dis() (*método dis.Bytecode*), 2267
dis() (*no módulo dis*), 2268
dis() (*no módulo pickletools*), 2289
DISABLE (*no módulo sys.monitoring*), 2053
disable (*pdb command*), 1970
disable() (*método bdb.Breakpoint*), 1956
disable() (*método profile.Profile*), 1979
disable() (*no módulo faulthandler*), 1962
disable() (*no módulo gc*), 2119
disable() (*no módulo logging*), 822
disable_current_event() (*método bdb.Bdb*), 1960
disable_faulthandler() (*no módulo test.support*), 1937
disable_gc() (*no módulo test.support*), 1937
disable_interspersed_args() (*método optparse.OptionParser*), 970
disabled (*atributo logging.Logger*), 810
DisableReflectionKey() (*no módulo winreg*), 2299
disassemble() (*no módulo dis*), 2269
discard (*atributo http.cookiejar.Cookie*), 1607
discard() (*método frozenset*), 97
discard() (*método mailbox.Mailbox*), 1402
discard() (*método mailbox.MH*), 1409
disco() (*no módulo dis*), 2269
discover() (*método unittest.TestLoader*), 1861
disk_usage() (*no módulo shutil*), 529
dispatch_call() (*método bdb.Bdb*), 1957
dispatch_exception() (*método bdb.Bdb*), 1958
dispatch_line() (*método bdb.Bdb*), 1957
dispatch_return() (*método bdb.Bdb*), 1958
dispatch_table (*atributo pickle.Pickler*), 541
DISPLAY, 1663
display (*pdb command*), 1972
display_name (*atributo email.headerregistry.Address*), 1361

display_name (*atributo email.headerregistry.Group*), 1361
displayhook() (*no módulo sys*), 2024
dist() (*no módulo math*), 366
distance() (*no módulo turtle*), 1726
distb() (*no módulo dis*), 2269
Distribution (*classe em importlib.metadata*), 2198
distribution() (*no módulo importlib.metadata*), 2198
distutils
    module, 2338
Div (*classe em ast*), 2213
divide() (*método decimal.Context*), 390
divide_int() (*método decimal.Context*), 390
divisão pelo piso, **2351**
division (*no módulo __future__*), 2118
DivisionByZero (*classe em decimal*), 394
divmod()
    built-in function, 14
divmod() (*método decimal.Context*), 390
DLE (*no módulo curses.ascii*), 1012
DllCanUnloadNow() (*no módulo ctypes*), 893
DllGetClassObject() (*no módulo ctypes*), 893
dllhandle (*no módulo sys*), 2024
dllist() (*no módulo ctypes.util*), 893
dnd_start() (*no módulo tkinter.dnd*), 1682
DndHandler (*classe em tkinter.dnd*), 1682
dngettext() (*no módulo gettext*), 1644
dnpgettext() (*no módulo gettext*), 1644
Do inglês: Cyclic Redundancy Check, 612
do_clear() (*método bdb.Bdb*), 1958
do_command() (*método curses.textpad.Textbox*), 1010
do_GET() (*método http.server.SimpleHTTPRequestHandler*), 1594
do_handshake() (*método ssl.SSLSocket*), 1284
do_HEAD() (*método http.server.SimpleHTTPRequestHandler*), 1594
do_help() (*método cmd.Cmd*), 1016
do_POST() (*método http.server.CGIHTTPRequestHandler*), 1595
doc (*atributo json.JSONDecodeError*), 1398
doc (*atributo tomllib.TOMLDecodeError*), 687
doc_header (*atributo cmd.Cmd*), 1017
DocCGIXMLRPCRequestHandler (*classe em xmlrpc.server*), 1622
DocFileSuite() (*no módulo doctest*), 1827
doClassCleanups() (*método de classe unittest.TestCase*), 1857
doCleanups() (*método unittest.TestCase*), 1857
docmd() (*método smtplib.SMTP*), 1570
docstring, **2350**
docstring (*atributo doctest.DocTest*), 1830
doctest
    module, 1814
DocTest (*classe em doctest*), 1829
doctest opção de linha de comando
    -f, 1818
    --fail-fast, 1818
    -o, 1817
    --option, 1817
    -v, 1817
    --verbose, 1817
DocTestFailure, 1836
DocTestFinder (*classe em doctest*), 1830
DocTestParser (*classe em doctest*), 1831
DocTestRunner (*classe em doctest*), 1832
DocTestSuite() (*no módulo doctest*), 1827
doctype() (*método xml.etree.ElementTree.TreeBuilder*), 1456
documentação
    geração, 1809
    online, 1809
documentElement (*atributo xml.dom.Document*), 1463
DocXMLRPCRequestHandler (*classe em xmlrpc.server*), 1622
DocXMLRPCServer (*classe em xmlrpc.server*), 1622
domain (*atributo email.headerregistry.Address*), 1361
domain (*atributo http.cookiejar.Cookie*), 1607
domain (*atributo http.cookies.Morsel*), 1598
domain (*atributo tracemalloc.DomainFilter*), 1997
domain (*atributo tracemalloc.Filter*), 1998
domain (*atributo tracemalloc.Trace*), 2000
domain_initial_dot (*atributo http.cookiejar.Cookie*), 1608
domain_return_ok() (*método http.cookiejar.CookiePolicy*), 1604
domain_specified (*atributo http.cookiejar.Cookie*), 1608
DomainFilter (*classe em tracemalloc*), 1997
DomainLiberal (*atributo http.cookiejar.DefaultCookiePolicy*), 1607
DomainRFC2965Match (*atributo http.cookiejar.DefaultCookiePolicy*), 1607
DomainStrict (*atributo http.cookiejar.DefaultCookiePolicy*), 1607
DomainStrictNoDots (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
DomainStrictNonDomain (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
DOMEventStream (*classe em xml.dom.pulldom*), 1474
DOMException, 1466
doModuleCleanups() (*no módulo unittest*), 1868
DomstringSizeErr, 1466
done() (*método asyncio.Future*), 1211
done() (*método asyncio.Task*), 1158
done() (*método concurrent.futures.Future*), 1097
done() (*método graphlib.TopologicalSorter*), 354
done() (*no módulo turtle*), 1741
DONT_ACCEPT_BLANKLINE (*no módulo doctest*), 1821
DONT_ACCEPT_TRUE_FOR_1 (*no módulo doctest*), 1821
dont_write_bytecode (*atributo sys.flags*), 2028
dont_write_bytecode (*no módulo sys*), 2025
doRollover() (*método logging.handlers.RotatingFileHandler*), 842
doRollover() (*método logging.handlers.TimedRotatingFileHandler*),

843
DOT (*no módulo token*), 2250
dot() (*no módulo turtle*), 1723
DOTALL (*no módulo re*), 156
doublequote (*atributo csv.Dialect*), 664
DOUBLESLASH (*no módulo token*), 2251
DOUBLESLASHEQUAL (*no módulo token*), 2251
DOUBLESTAR (*no módulo token*), 2251
DOUBLESTAREQUAL (*no módulo token*), 2251
doupdate() (*no módulo curses*), 985
down (*pdb command*), 1969
down() (*no módulo turtle*), 1727
dpgettext() (*no módulo gettext*), 1644
drain() (*método asyncio.StreamWriter*), 1165
drive (*atributo pathlib.PurePath*), 476
drop_whitespace (*atributo textwrap.TextWrapper*), 186
dropwhile() (*no módulo itertools*), 440
dst() (*método datetime.datetime*), 241
dst() (*método datetime.time*), 250
dst() (*método datetime.timezone*), 258
dst() (*método datetime.tzinfo*), 251
DTDHandler (*classe em xml.sax.handler*), 1476
dump() (*método pickle.Pickler*), 541
dump() (*método tracemalloc.Snapshot*), 1999
dump() (*no módulo ast*), 2240
dump() (*no módulo json*), 1393
dump() (*no módulo marshal*), 559
dump() (*no módulo pickle*), 540
dump() (*no módulo plistlib*), 690
dump() (*no módulo xml.etree.ElementTree*), 1447
dump_c_stack() (*no módulo faulthandler*), 1962
dump_stats() (*método profile.Profile*), 1979
dump_stats() (*método pstats.Stats*), 1980
dump_traceback() (*no módulo faulthandler*), 1961
dump_traceback_later() (*no módulo faulthandler*), 1963
dumps() (*no módulo json*), 1394
dumps() (*no módulo marshal*), 559
dumps() (*no módulo pickle*), 540
dumps() (*no módulo plistlib*), 690
dumps() (*no módulo xmlrpc.client*), 1615
dunder, **2350**
dup() (*método socket.socket*), 1260
dup() (*no módulo os*), 723
dup2() (*no módulo os*), 723
DuplicateOptionError, 685
DuplicateSectionError, 685
--durations
    unittest opção de linha de comando, 1840
dwFlags (*atributo subprocess.STARTUPINFO*), 1115
DynamicClassAttribute() (*no módulo types*), 325

E

-e
    calendar opção de linha de comando, 274
    compileall opção de linha de comando, 2262
    idle opção de linha de comando, 1708
    mimetypes opção de linha de comando, 1424
    tarfile opção de linha de comando, 655
    tokenize opção de linha de comando, 2255
    zipfile opção de linha de comando, 640
e (*no módulo cmath*), 373
e (*no módulo math*), 369
E/S com buffer de linha, 26
E/S sem buffer, 26
E2BIG (*no módulo errno*), 857
EACCES (*no módulo errno*), 858
EADDRINUSE (*no módulo errno*), 862
EADDRNOTAVAIL (*no módulo errno*), 862
EADV (*no módulo errno*), 860
EAFNOSUPPORT (*no módulo errno*), 862
EAFP, **2350**
EAGAIN (*no módulo errno*), 858
eager_task_factory() (*no módulo asyncio*), 1149
EALREADY (*no módulo errno*), 863
east_asian_width() (*no módulo unicodedata*), 188
EAUTH (*no módulo errno*), 864
EBADARCH (*no módulo errno*), 864
EBADE (*no módulo errno*), 860
EBADEXEC (*no módulo errno*), 864
EBADF (*no módulo errno*), 858
EBADFD (*no módulo errno*), 861
EBADMACHO (*no módulo errno*), 864
EBADMSG (*no módulo errno*), 861
EBADR (*no módulo errno*), 860
EBADRPC (*no módulo errno*), 865
EBADRQC (*no módulo errno*), 860
EBADSLT (*no módulo errno*), 860
EBFONT (*no módulo errno*), 860
EBUSY (*no módulo errno*), 858
ECANCELED (*no módulo errno*), 865
ECHILD (*no módulo errno*), 858
echo() (*no módulo curses*), 985
echochar() (*método curses.window*), 993
ECHRNG (*no módulo errno*), 859
ECOMM (*no módulo errno*), 860
ECONNABORTED (*no módulo errno*), 862
ECONNREFUSED (*no módulo errno*), 862
ECONNRESET (*no módulo errno*), 862
EDEADLK (*no módulo errno*), 859
EDEADLOCK (*no módulo errno*), 860
EDESTADDRREQ (*no módulo errno*), 861
EDEVERR (*no módulo errno*), 864
edit() (*método curses.textpad.Textbox*), 1009
EDOM (*no módulo errno*), 859
EDOTDOT (*no módulo errno*), 861
EDQUOT (*no módulo errno*), 863
EEXIST (*no módulo errno*), 858
EFAULT (*no módulo errno*), 858
EFBIG (*no módulo errno*), 858

EFD_CLOEXEC (*no módulo os*), 757
EFD_NONBLOCK (*no módulo os*), 757
EFD_SEMAPHORE (*no módulo os*), 757
effective() (*no módulo bdb*), 1960
EFTYPE (*no módulo errno*), 864
ehlo() (*método smtplib.SMTP*), 1570
ehlo_or_helo_if_needed() (*método smtplib.SMTP*), 1570
EHOSTDOWN (*no módulo errno*), 863
EHOSTUNREACH (*no módulo errno*), 863
EHWPOISON (*no módulo errno*), 863
EIDRM (*no módulo errno*), 859
EILSEQ (*no módulo errno*), 861
EINPROGRESS (*no módulo errno*), 863
EINTR (*no módulo errno*), 857
EINVAL (*no módulo errno*), 858
EIO (*no módulo errno*), 857
EISCONN (*no módulo errno*), 862
EISDIR (*no módulo errno*), 858
EISNAM (*no módulo errno*), 863
EJECT (*atributo enum.FlagBoundary*), 349
EKEYEXPIRED (*no módulo errno*), 863
EKEYREJECTED (*no módulo errno*), 863
EKEYREVOKED (*no módulo errno*), 863
EL2HLT (*no módulo errno*), 860
EL2NSYNC (*no módulo errno*), 859
EL3HLT (*no módulo errno*), 859
EL3RST (*no módulo errno*), 859
Element (*classe em xml.etree.ElementTree*), 1451
element_create() (*método tkinter.ttk.Style*), 1698
element_names() (*método tkinter.ttk.Style*), 1699
element_options() (*método tkinter.ttk.Style*), 1699
ElementDeclHandler() (*método xml.parsers.expat.xmlparser*), 1490
elements() (*método collections.Counter*), 279
ElementTree (*classe em xml.etree.ElementTree*), 1453
ELIBACC (*no módulo errno*), 861
ELIBBAD (*no módulo errno*), 861
ELIBEXEC (*no módulo errno*), 861
ELIBMAX (*no módulo errno*), 861
ELIBSCN (*no módulo errno*), 861
ELLIPSIS (*no módulo doctest*), 1822
ELLIPSIS (*no módulo token*), 2252
Ellipsis (*variável interna*), 38
EllipsisType (*no módulo types*), 323
ELNRNG (*no módulo errno*), 859
ELOCKUNMAPPED (*no módulo errno*), 864
ELOOP (*no módulo errno*), 859
EM (*no módulo curses.ascii*), 1012
email
    module, 1333
email.charset
    module, 1384
email.contentmanager
    module, 1362
email.encoders
    module, 1386
email.errors
    module, 1355
email.generator
    module, 1345
email.header
    module, 1382
email.headerregistry
    module, 1356
email.iterators
    module, 1390
email.message
    module, 1334
EmailMessage (*classe em email.message*), 1335
email.mime
    module, 1379
email.mime.application
    module, 1380
email.mime.audio
    module, 1380
email.mime.base
    module, 1379
email.mime.image
    module, 1380
email.mime.message
    module, 1381
email.mime.multipart
    module, 1379
email.mime.nonmultipart
    module, 1379
email.mime.text
    module, 1381
email.parser
    module, 1342
email.policy
    module, 1348
EmailPolicy (*classe em email.policy*), 1352
email.utils
    module, 1387
Emax (*atributo decimal.Context*), 388
embutido
    tipos, 39
EMEDIUMTYPE (*no módulo errno*), 863
EMFILE (*no módulo errno*), 858
Emin (*atributo decimal.Context*), 388
emit() (*método logging.FileHandler*), 839
emit() (*método logging.Handler*), 815
emit() (*método logging.handlers.BufferingHandler*), 849
emit() (*método logging.handlers.DatagramHandler*), 845
emit() (*método logging.handlers.HTTPHandler*), 850
emit() (*método logging.handlers.NTEventLogHandler*), 848
emit() (*método logging.handlers.QueueHandler*), 850
emit() (*método logging.handlers.RotatingFileHandler*), 842
emit() (*método logging.handlers.SMTPHandler*), 848
emit() (*método logging.handlers.SocketHandler*), 844
emit() (*método logging.handlers.SysLogHandler*), 846

emit() (*método logging.handlers.TimedRotatingFileHandler*), 843
emit() (*método logging.handlers.WatchedFileHandler*), 840
emit() (*método logging.NullHandler*), 840
emit() (*método logging.StreamHandler*), 839
EMLINK (*no módulo errno*), 859
empacotamento (*widgets*), 1669
Empty, 1126
empty (*atributo inspect.Parameter*), 2132
empty (*atributo inspect.Signature*), 2130
empty() (*método asyncio.Queue*), 1179
empty() (*método multiprocessing.Queue*), 1051
empty() (*método multiprocessing.SimpleQueue*), 1052
empty() (*método queue.Queue*), 1126
empty() (*método queue.SimpleQueue*), 1128
empty() (*método sched.scheduler*), 1124
EMPTY_NAMESPACE (*no módulo xml.dom*), 1459
emptyline() (*método cmd.Cmd*), 1016
emscripten_version (*atributo sys._emscripten_info*), 2025
EMSGSIZE (*no módulo errno*), 861
EMULTIHOP (*no módulo errno*), 861
enable (*pdb command*), 1970
enable() (*método bdb.Breakpoint*), 1955
enable() (*método imaplib.IMAP4*), 1562
enable() (*método profile.Profile*), 1979
enable() (*no módulo faulthandler*), 1962
enable() (*no módulo gc*), 2119
enable_callback_tracebacks() (*no módulo sqlite3*), 570
enable_interspersed_args() (*método optparse.OptionParser*), 971
enable_load_extension() (*método sqlite3.Connection*), 577
enable_long_distance_matching (*atributo compression.zstd.CompressionParameter*), 607
enable_traversal() (*método tkinter.ttk.Notebook*), 1689
ENABLE_USER_SITE (*no módulo site*), 2153
enabled (*atributo bdb.Breakpoint*), 1956
EnableReflectionKey() (*no módulo winreg*), 2299
ENAMETOOLONG (*no módulo errno*), 859
ENAVAIL (*no módulo errno*), 863
encadeamento
    comparações, 40
    exceção, 115
enclose() (*método curses.window*), 993
encode
    Codecs, 204
encode (*atributo codecs.CodecInfo*), 205
encode() (*método codecs.Codec*), 210
encode() (*método codecs.IncrementalEncoder*), 211
encode() (*método email.header.Header*), 1383
encode() (*método json.JSONEncoder*), 1398
encode() (*método str*), 58
encode() (*método xmlrpc.client.Binary*), 1613
encode() (*método xmlrpc.client.DateTime*), 1612
encode() (*no módulo base64*), 1428
encode() (*no módulo codecs*), 205
encode() (*no módulo quopri*), 1431
encode_7or8bit() (*no módulo email.encoders*), 1387
encode_base64() (*no módulo email.encoders*), 1387
encode_noop() (*no módulo email.encoders*), 1387
encode_quopri() (*no módulo email.encoders*), 1387
encode_rfc2231() (*no módulo email.utils*), 1389
encodebytes() (*no módulo base64*), 1428
EncodedFile() (*no módulo codecs*), 207
encodePriority() (*método logging.handlers.SysLogHandler*), 846
encodestring() (*no módulo quopri*), 1431
--encoding
    calendar opção de linha de comando, 274
encoding (*atributo curses.window*), 993
encoding (*atributo io.TextIOBase*), 790
encoding (*atributo UnicodeError*), 122
ENCODING (*no módulo tarfile*), 644
ENCODING (*no módulo token*), 2249
encodings
    module, 220
encodings_map (*atributo mimetypes.MimeTypes*), 1423
encodings_map (*no módulo mimetypes*), 1422
encodings.idna
    module, 221
encodings.mbcs
    module, 222
encodings.utf_8_sig
    module, 222
EncodingWarning, 125
end (*atributo UnicodeError*), 122
end() (*método re.Match*), 164
end() (*método xml.etree.ElementTree.TreeBuilder*), 1455
END_ASYNC_FOR (*opcode*), 2274
end_col_offset (*atributo ast.AST*), 2207
end_colno (*atributo traceback.FrameSummary*), 2113
end_fill() (*no módulo turtle*), 1731
END_FOR (*opcode*), 2272
end_headers() (*método http.server.BaseHTTPRequestHandler*), 1593
end_lineno (*atributo ast.AST*), 2207
end_lineno (*atributo SyntaxError*), 121
end_lineno (*atributo traceback.FrameSummary*), 2113
end_lineno (*atributo traceback.TracebackException*), 2111
end_ns() (*método xml.etree.ElementTree.TreeBuilder*), 1456
end_offset (*atributo SyntaxError*), 121
end_offset (*atributo traceback.TracebackException*), 2111
end_poly() (*no módulo turtle*), 1736
END_SEND (*opcode*), 2272

endCDATA() (*método xml.sax.handler.LexicalHandler*), 1481
EndCdataSectionHandler() (*método xml.parsers.expat.xmlparser*), 1491
EndDoctypeDeclHandler() (*método xml.parsers.expat.xmlparser*), 1490
endDocument() (*método xml.sax.handler.ContentHandler*), 1478
endDTD() (*método xml.sax.handler.LexicalHandler*), 1481
endElement() (*método xml.sax.handler.ContentHandler*), 1479
EndElementHandler() (*método xml.parsers.expat.xmlparser*), 1490
endElementNS() (*método xml.sax.handler.ContentHandler*), 1479
endheaders() (*método http.client.HTTPConnection*), 1547
ENDMARKER (*no módulo token*), 2248
EndNamespaceDeclHandler() (*método xml.parsers.expat.xmlparser*), 1491
endpos (*atributo re.Match*), 165
endPrefixMapping() (*método xml.sax.handler.ContentHandler*), 1479
endswith() (*método bytearray*), 75
endswith() (*método bytes*), 75
endswith() (*método str*), 59
endwin() (*no módulo curses*), 985
ENEEDAUTH (*no módulo errno*), 864
ENETDOWN (*no módulo errno*), 862
ENETRESET (*no módulo errno*), 862
ENETUNREACH (*no módulo errno*), 862
ENFILE (*no módulo errno*), 858
ENOANO (*no módulo errno*), 860
ENOATTR (*no módulo errno*), 864
ENOBUFS (*no módulo errno*), 862
ENOCSI (*no módulo errno*), 859
ENODATA (*no módulo errno*), 860
ENODEV (*no módulo errno*), 858
ENOENT (*no módulo errno*), 857
ENOEXEC (*no módulo errno*), 857
ENOKEY (*no módulo errno*), 863
ENOLCK (*no módulo errno*), 859
ENOLINK (*no módulo errno*), 860
ENOMEDIUM (*no módulo errno*), 863
ENOMEM (*no módulo errno*), 858
ENOMSG (*no módulo errno*), 859
ENONET (*no módulo errno*), 860
ENOPKG (*no módulo errno*), 860
ENOPOLICY (*no módulo errno*), 864
ENOPROTOOPT (*no módulo errno*), 861
ENOSPC (*no módulo errno*), 858
ENOSR (*no módulo errno*), 860
ENOSTR (*no módulo errno*), 860
ENOSYS (*no módulo errno*), 859
ENOTACTIVE (*no módulo errno*), 864
ENOTBLK (*no módulo errno*), 858
ENOTCAPABLE (*no módulo errno*), 865
ENOTCONN (*no módulo errno*), 862
ENOTDIR (*no módulo errno*), 858
ENOTEMPTY (*no módulo errno*), 859
ENOTNAM (*no módulo errno*), 863
ENOTRECOVERABLE (*no módulo errno*), 865
ENOTSOCK (*no módulo errno*), 861
ENOTSUP (*no módulo errno*), 862
ENOTTY (*no módulo errno*), 858
ENOTUNIQ (*no módulo errno*), 861
ENQ (*no módulo curses.ascii*), 1011
enqueue() (*método logging.handlers.QueueHandler*), 851
enqueue_sentinel() (*método logging.handlers.QueueListener*), 852
ensure_directories() (*método venv.EnvBuilder*), 2009
ensure_future() (*no módulo asyncio*), 1210
ensurepip
    module, 2003
ensurepip opção de linha de comando
    --altinstall, 2004
    --default-pip, 2004
    --root, 2004
    --user, 2004
enter() (*método sched.scheduler*), 1124
enter_async_context() (*método contextlib.AsyncExitStack*), 2095
enter_context() (*método contextlib.ExitStack*), 2094
enterabs() (*método sched.scheduler*), 1124
enterAsyncContext() (*método unittest.IsolatedAsyncioTestCase*), 1858
enterClassContext() (*método de classe unittest.TestCase*), 1857
enterContext() (*método unittest.TestCase*), 1856
enterModuleContext() (*no módulo unittest*), 1868
entities (*atributo xml.dom.DocumentType*), 1463
EntityDeclHandler() (*método xml.parsers.expat.xmlparser*), 1491
entitydefs (*no módulo html.entities*), 1438
EntityResolver (*classe em xml.sax.handler*), 1476
entrada de caminho, **2358**
entry_points() (*no módulo importlib.metadata*), 2194
EntryPoint (*classe em importlib.metadata*), 2195
EntryPoints (*classe em importlib.metadata*), 2194
enum
    module, 336
Enum (*classe em enum*), 340
enum_certificates() (*no módulo ssl*), 1275
enum_crls() (*no módulo ssl*), 1276
EnumCheck (*classe em enum*), 347
EnumDict (*classe em enum*), 349
enumerate()
    built-in function, 14
enumerate() (*no módulo threading*), 1024
EnumKey() (*no módulo winreg*), 2295
EnumType (*classe em enum*), 338
EnumValue() (*no módulo winreg*), 2296

ENV_DIR
    venv opção de linha de comando, 2006
EnvBuilder (*classe em venv*), 2008
environ (*no módulo os*), 713
environ (*no módulo posix*), 2313
environb (*no módulo os*), 714
EnvironmentError, 123
EnvironmentVarGuard (*classe em test.support.os_helper*), 1946
ENXIO (*no módulo errno*), 857
eof (*atributo bz2.BZ2Decompressor*), 621
eof (*atributo compression.zstd.ZstdDecompressor*), 603
eof (*atributo lzma.LZMADecompressor*), 626
eof (*atributo shlex.shlex*), 2310
eof (*atributo ssl.MemoryBIO*), 1303
eof (*atributo zlib.Decompress*), 614
eof_received() (*método asyncio.BufferedProtocol*), 1220
eof_received() (*método asyncio.Protocol*), 1219
EOFError, 117
EOPNOTSUPP (*no módulo errno*), 862
EOT (*no módulo curses.ascii*), 1011
EOVERFLOW (*no módulo errno*), 861
EOWNERDEAD (*no módulo errno*), 865
EPERM (*no módulo errno*), 857
EPFNOSUPPORT (*no módulo errno*), 862
epilogue (*atributo email.message.EmailMessage*), 1341
epilogue (*atributo email.message.Message*), 1378
EPIPE (*no módulo errno*), 859
epoll() (*no módulo select*), 1306
EpollSelector (*classe em selectors*), 1315
EPROCLIM (*no módulo errno*), 864
EPROCUNAVAIL (*no módulo errno*), 864
EPROGMISMATCH (*no módulo errno*), 864
EPROGUNAVAIL (*no módulo errno*), 864
EPROTO (*no módulo errno*), 860
EPROTONOSUPPORT (*no módulo errno*), 862
EPROTOTYPE (*no módulo errno*), 861
epsilon (*atributo sys.float_info*), 2030
EPWROFF (*no módulo errno*), 865
Eq (*classe em ast*), 2214
eq() (*no módulo operator*), 463
EQEQUAL (*no módulo token*), 2250
EQFULL (*no módulo errno*), 863
EQUAL (*no módulo token*), 2250
era, 795
ERA (*no módulo locale*), 1654
ERA_D_FMT (*no módulo locale*), 1655
ERA_D_T_FMT (*no módulo locale*), 1655
ERA_T_FMT (*no módulo locale*), 1655
ERANGE (*no módulo errno*), 859
erase() (*método curses.window*), 993
erasechar() (*no módulo curses*), 985
EREMCHG (*no módulo errno*), 861
EREMOTE (*no módulo errno*), 860
EREMOTEIO (*no módulo errno*), 863
ERESTART (*no módulo errno*), 861
erf() (*no módulo math*), 368
erfc() (*no módulo math*), 368
ERFKILL (*no módulo errno*), 863
EROFS (*no módulo errno*), 859
ERPCMISMATCH (*no módulo errno*), 865
ERR (*no módulo curses*), 997
errcheck (*atributo ctypes._CFuncPtr*), 889
errcode (*atributo xmlrpc.client.ProtocolError*), 1614
errmsg (*atributo xmlrpc.client.ProtocolError*), 1614
errno
    module, 857
    módulo, 119
errno (*atributo OSError*), 119
Error, 326, 530, 587, 664, 685, 1419, 1431, 1498, 1639, 1651
error, 198, 560, 562, 564, 565, 611, 712, 984, 1133, 1245, 1306, 1486, 2322, 2334
ERROR (*no módulo logging*), 814
ERROR (*no módulo tkinter.messagebox*), 1681
error handler's name
    backslashreplace, 208
    ignore, 208
    namereplace, 208
    replace, 208
    strict, 208
    surrogateescape, 208
    surrogatepass, 208
    xmlcharrefreplace, 208
error() (*método argparse.ArgumentParser*), 936
error() (*método logging.Logger*), 813
error() (*método urllib.request.OpenerDirector*), 1518
error() (*método xml.sax.handler.ErrorHandler*), 1480
error() (*no módulo logging*), 822
error_body (*atributo wsgiref.handlers.BaseHandler*), 1508
error_content_type (*atributo http.server.BaseHTTPRequestHandler*), 1591
error_headers (*atributo wsgiref.handlers.BaseHandler*), 1508
error_leader() (*método shlex.shlex*), 2309
error_message_format (*atributo http.server.BaseHTTPRequestHandler*), 1591
error_output() (*método wsgiref.handlers.BaseHandler*), 1507
error_perm, 1556
error_proto, 1557, 1558
error_received() (*método asyncio.DatagramProtocol*), 1220
error_reply, 1556
error_status (*atributo wsgiref.handlers.BaseHandler*), 1508
error_temp, 1556
ErrorByteIndex (*atributo xml.parsers.expat.xmlparser*), 1489
ErrorCode (*atributo xml.parsers.expat.xmlparser*), 1489
errorcode (*no módulo errno*), 857

ErrorColumnNumber (*atributo xml.parsers.expat.xmlparser*), 1489
ErrorHandler (*classe em xml.sax.handler*), 1476
errorlevel (*atributo tarfile.TarFile*), 647
ErrorLineNumber (*atributo xml.parsers.expat.xmlparser*), 1489
errors (*atributo io.TextIOBase*), 791
errors (*atributo unittest.TestLoader*), 1860
errors (*atributo unittest.TestResult*), 1862
ErrorStream (*classe em wsgiref.types*), 1509
ErrorString() (*no módulo xml.parsers.expat*), 1487
ERRORTOKEN (*no módulo token*), 2249
Erros
    logging, 808
ESC (*no módulo curses.ascii*), 1012
escape (*atributo shlex.shlex*), 2310
escape() (*no módulo glob*), 521
escape() (*no módulo html*), 1433
escape() (*no módulo re*), 160
escape() (*no módulo xml.sax.saxutils*), 1481
escapechar (*atributo csv.Dialect*), 664
escapedquotes (*atributo shlex.shlex*), 2310
escopo aninhado, **2357**
escopo otimizado, **2357**
ESHLIBVERS (*no módulo errno*), 865
ESHUTDOWN (*no módulo errno*), 862
ESOCKTNOSUPPORT (*no módulo errno*), 862
espaço
    em formatação no estilo printf, 69, 86
    na formatação de strings, 133
espaço de nomes, **2357**
especial
    método, 2361
ESPIPE (*no módulo errno*), 858
ESRCH (*no módulo errno*), 857
ESRMNT (*no módulo errno*), 860
estado de thread, **2362**
estado de thread anexado, **2346**
ESTALE (*no módulo errno*), 863
ESTRPIPE (*no módulo errno*), 861
ETB (*no módulo curses.ascii*), 1012
ETH_P_ALL (*no módulo socket*), 1248
ETHERTYPE_ARP (*no módulo socket*), 1251
ETHERTYPE_IP (*no módulo socket*), 1251
ETHERTYPE_IPV6 (*no módulo socket*), 1251
ETHERTYPE_VLAN (*no módulo socket*), 1251
ETIME (*no módulo errno*), 860
ETIMEDOUT (*no módulo errno*), 862
Etiny() (*método decimal.Context*), 389
ETOOMANYREFS (*no módulo errno*), 862
Etop() (*método decimal.Context*), 389
ETX (*no módulo curses.ascii*), 1011
ETXTBSY (*no módulo errno*), 858
EUCLEAN (*no módulo errno*), 863
EUNATCH (*no módulo errno*), 859
EUSERS (*no módulo errno*), 861
eval
    função embutida, 111, 328, 330
eval()
    built-in function, 14
evaluate() (*método annotationlib.ForwardRef*), 2145
evaluate_bound() (*método typing.TypeVar*), 1780
evaluate_constraints() (*método typing.TypeVar*), 1780
evaluate_default() (*método typing.ParamSpec*), 1784
evaluate_default() (*método typing.TypeVar*), 1781
evaluate_default() (*método typing.TypeVarTuple*), 1783
evaluate_forward_ref() (*no módulo typing*), 1802
evaluate_value() (*método typing.TypeAliasType*), 1786
Event (*classe em asyncio*), 1170
Event (*classe em multiprocessing*), 1057
Event (*classe em threading*), 1035
Event() (*método multiprocessing.managers.SyncManager*), 1063
EVENT_READ (*no módulo selectors*), 1313
EVENT_WRITE (*no módulo selectors*), 1313
eventfd() (*no módulo os*), 756
eventfd_read() (*no módulo os*), 757
eventfd_write() (*no módulo os*), 757
EventLoop (*classe em asyncio*), 1207
eventos (*widgets*), 1672
eventos de auditoria, 1951
events (*atributo selectors.SelectorKey*), 1314
EWOULDBLOCK (*no módulo errno*), 859
EX_CANTCREAT (*no módulo os*), 764
EX_CONFIG (*no módulo os*), 765
EX_DATAERR (*no módulo os*), 763
EX_IOERR (*no módulo os*), 764
EX_NOHOST (*no módulo os*), 764
EX_NOINPUT (*no módulo os*), 764
EX_NOPERM (*no módulo os*), 764
EX_NOTFOUND (*no módulo os*), 765
EX_NOUSER (*no módulo os*), 764
EX_OK (*no módulo os*), 763
EX_OSERR (*no módulo os*), 764
EX_OSFILE (*no módulo os*), 764
EX_PROTOCOL (*no módulo os*), 764
EX_SOFTWARE (*no módulo os*), 764
EX_TEMPFAIL (*no módulo os*), 764
EX_UNAVAILABLE (*no módulo os*), 764
EX_USAGE (*no módulo os*), 763
--exact
    tokenize opção de linha de comando, 2255
EXACT_TOKEN_TYPES (*no módulo token*), 2252
example (*atributo doctest.DocTestFailure*), 1836
example (*atributo doctest.UnexpectedException*), 1836
Example (*classe em doctest*), 1830
examples (*atributo doctest.DocTest*), 1829
exc_info (*atributo doctest.UnexpectedException*), 1836
exc_info() (*no módulo sys*), 2026
exc_msg (*atributo doctest.Example*), 1830

exc_type (*atributo traceback.TracebackException*), 2111
exc_type_str (*atributo traceback.TracebackException*), 2111
exceção
    encadeamento, 115
excel (*classe em csv*), 662
excel_tab (*classe em csv*), 662
except
    instrução, 115
ExceptHandler (*classe em ast*), 2225
excepthook() (*no módulo sys*), 2025
excepthook() (*no módulo threading*), 1023
Exception, 117
EXCEPTION (*no módulo _tkinter*), 1674
exception() (*método asyncio.Future*), 1212
exception() (*método asyncio.Task*), 1158
exception() (*método concurrent.futures.Future*), 1097
exception() (*método logging.Logger*), 813
exception() (*no módulo logging*), 822
exception() (*no módulo sys*), 2026
EXCEPTION_HANDLED (*monitoring event*), 2050
ExceptionGroup, 125
exceptions (*atributo BaseExceptionGroup*), 125
exceptions (*atributo traceback.TracebackException*), 2110
exceptions (*pdb command*), 1974
excinfo (*atributo concurrent.interpreters.ExecutionFailed*), 1103
EXCLAMATION (*no módulo token*), 2252
EXDEV (*no módulo errno*), 858
exec
    função embutida, 15, 111
exec()
    built-in function, 15
exec() (*método concurrent.interpreters.Interpreter*), 1102
exec_module() (*método importlib.abc.InspectLoader*), 2173
exec_module() (*método importlib.abc.Loader*), 2172
exec_module() (*método importlib.abc.SourceLoader*), 2175
exec_module() (*método importlib.machinery.ExtensionFileLoader*), 2180
exec_module() (*método zipimport.zipimporter*), 2160
exec_prefix (*no módulo sys*), 2026
execl() (*no módulo os*), 762
execle() (*no módulo os*), 762
execlp() (*no módulo os*), 762
execlpe() (*no módulo os*), 762
executable (*no módulo sys*), 2027
execute() (*método sqlite3.Connection*), 573
execute() (*método sqlite3.Cursor*), 583
executemany() (*método sqlite3.Connection*), 573
executemany() (*método sqlite3.Cursor*), 583
executescript() (*método sqlite3.Connection*), 573
executescript() (*método sqlite3.Cursor*), 584
ExecutionFailed, 1099, 1103
ExecutionLoader (*classe em importlib.abc*), 2173
Executor (*classe em concurrent.futures*), 1091
execv() (*no módulo os*), 762
execve() (*no módulo os*), 762
execvp() (*no módulo os*), 762
execvpe() (*no módulo os*), 762
EXFULL (*no módulo errno*), 860
exists() (*método pathlib.Path*), 486
exists() (*método pathlib.types.PathInfo*), 498
exists() (*método tkinter.ttk.Treeview*), 1694
exists() (*método zipfile.Path*), 636
exists() (*no módulo os.path*), 500
exit (*variável interna*), 38
exit() (*método argparse.ArgumentParser*), 936
exit() (*no módulo _thread*), 1133
exit() (*no módulo sys*), 2027
exitcode (*atributo multiprocessing.Process*), 1047
exitonclick() (*no módulo turtle*), 1744
ExitStack (*classe em contextlib*), 2093
exp() (*método decimal.Context*), 390
exp() (*método decimal.Decimal*), 381
exp() (*no módulo cmath*), 372
exp() (*no módulo math*), 365
exp2() (*no módulo math*), 365
expand() (*método re.Match*), 162
expand_tabs (*atributo textwrap.TextWrapper*), 185
ExpandEnvironmentStrings() (*no módulo winreg*), 2296
expandNode() (*método xml.dom.pulldom.DOMEventStream*), 1474
expandtabs() (*método bytearray*), 81
expandtabs() (*método bytes*), 81
expandtabs() (*método str*), 59
expanduser() (*método pathlib.Path*), 485
expanduser() (*no módulo os.path*), 500
expandvars() (*no módulo os.path*), 501
Expat, 1486
ExpatError, 1486
expected (*atributo asyncio.IncompleteReadError*), 1182
expectedFailure() (*no módulo unittest*), 1846
expectedFailures (*atributo unittest.TestResult*), 1862
expired() (*método asyncio.Timeout*), 1151
expires (*atributo http.cookiejar.Cookie*), 1607
expires (*atributo http.cookies.Morsel*), 1598
exploded (*atributo ipaddress.IPv4Address*), 1624
exploded (*atributo ipaddress.IPv4Network*), 1630
exploded (*atributo ipaddress.IPv6Address*), 1627
exploded (*atributo ipaddress.IPv6Network*), 1633
expm1() (*no módulo math*), 365
expovariate() (*no módulo random*), 411
Expr (*classe em ast*), 2212
expressão, **2350**
expressão geradora, **2352**
expression (*atributo string.templatelib.Interpolation*), 145
Expression (*classe em ast*), 2208
expunge() (*método imaplib.IMAP4*), 1562

extend() (*método array.array*), 310
extend() (*método collections.deque*), 282
extend() (*método sequence*), 52
extend() (*método xml.etree.ElementTree.Element*), 1452
extend_path() (*no módulo pkgutil*), 2161
EXTENDED_ARG (*opcode*), 2284
ExtendedContext (*no módulo decimal*), 387
ExtendedInterpolation (*classe em configparser*), 672
extendleft() (*método collections.deque*), 282
--extension
    mimetypes opção de linha de comando, 1424
EXTENSION_SUFFIXES (*no módulo importlib.machinery*), 2177
ExtensionFileLoader (*classe em importlib.machinery*), 2180
extensions_map (*atributo http.server.SimpleHTTPRequestHandler*), 1594
External Data Representation, 538
external_attr (*atributo zipfile.ZipInfo*), 639
ExternalClashError, 1419
ExternalEntityParserCreate() (*método xml.parsers.expat.xmlparser*), 1488
ExternalEntityRefHandler() (*método xml.parsers.expat.xmlparser*), 1491
extra (*atributo zipfile.ZipInfo*), 639
--extract
    tarfile opção de linha de comando, 655
    zipfile opção de linha de comando, 640
extract() (*método de classe traceback.StackSummary*), 2112
extract() (*método tarfile.TarFile*), 647
extract() (*método zipfile.ZipFile*), 633
extract_cookies() (*método http.cookiejar.CookieJar*), 1602
extract_stack() (*no módulo traceback*), 2109
extract_tb() (*no módulo traceback*), 2109
extract_version (*atributo zipfile.ZipInfo*), 639
extractall() (*método tarfile.TarFile*), 646
extractall() (*método zipfile.ZipFile*), 633
ExtractError, 643
extractfile() (*método tarfile.TarFile*), 647
extraction_filter (*atributo tarfile.TarFile*), 648
extsep (*no módulo os*), 779

F

-f
    calendar opção de linha de comando, 275
    compileall opção de linha de comando, 2261
    doctest opção de linha de comando, 1818
    random opção de linha de comando, 417
    trace opção de linha de comando, 1989
    unittest opção de linha de comando, 1840
f-string, **2351**
f-strings, **2351**
F_CONTIGUOUS (*atributo inspect.BufferFlags*), 2142
f_contiguous (*atributo memoryview*), 95
F_LOCK (*no módulo os*), 725
F_OK (*no módulo os*), 736
F_TEST (*no módulo os*), 725
F_TLOCK (*no módulo os*), 725
F_ULOCK (*no módulo os*), 725
fabs() (*no módulo math*), 363
factorial() (*no módulo math*), 362
factory() (*método de classe importlib.util.LazyLoader*), 2185
fail() (*método unittest.TestCase*), 1855
FAIL_FAST (*no módulo doctest*), 1823
failed (*atributo doctest.TestResults*), 1831
--failfast
    unittest opção de linha de comando, 1840
--fail-fast
    doctest opção de linha de comando, 1818
failfast (*atributo unittest.TestResult*), 1863
failureException (*atributo unittest.TestCase*), 1856
failures (*atributo doctest.DocTestRunner*), 1833
failures (*atributo unittest.TestResult*), 1862
FakePath (*classe em test.support.os_helper*), 1946
False, 39, 48
false, 39
False (*objeto embutido*), 39
False (*variável interna*), 37
families() (*no módulo tkinter.font*), 1676
family (*atributo socket.socket*), 1266
--fast
    gzip opção de linha de comando, 618
fast (*atributo compression.zstd.Strategy*), 609
fast (*atributo pickle.Pickler*), 542
fatalError() (*método xml.sax.handler.ErrorHandler*), 1481
fatia, **2360**
    atribuição, 51
    função embutida, 2284
    operação, 49
Fault (*classe em xmlrpc.client*), 1613
faultCode (*atributo xmlrpc.client.Fault*), 1613
faulthandler
    module, 1961
faultString (*atributo xmlrpc.client.Fault*), 1613
fchdir() (*no módulo os*), 738
fchmod() (*no módulo os*), 723
fchown() (*no módulo os*), 723
fcntl
    module, 2319
fcntl() (*no módulo fcntl*), 2320
fd (*atributo selectors.SelectorKey*), 1313
fd() (*no módulo turtle*), 1720

fd_count() (*no módulo test.support.os_helper*), 1947
fdatasync() (*no módulo os*), 723
fdopen() (*no módulo os*), 721
feature_external_ges (*no módulo xml.sax.handler*), 1477
feature_external_pes (*no módulo xml.sax.handler*), 1477
feature_namespace_prefixes (*no módulo xml.sax.handler*), 1477
feature_namespaces (*no módulo xml.sax.handler*), 1477
feature_string_interning (*no módulo xml.sax.handler*), 1477
feature_validation (*no módulo xml.sax.handler*), 1477
--feature-version
    ast opção de linha de comando, 2242
FEBRUARY (*no módulo calendar*), 273
feed() (*método email.parser.BytesFeedParser*), 1343
feed() (*método html.parser.HTMLParser*), 1435
feed() (*método xml.etree.ElementTree.XMLParser*), 1456
feed() (*método xml.etree.ElementTree.XMLPullParser*), 1457
feed() (*método xml.sax.xmlreader.IncrementalParser*), 1484
feed_eof() (*método asyncio.StreamReader*), 1163
FeedParser (*classe em email.parser*), 1343
fetch() (*método imaplib.IMAP4*), 1563
fetchall() (*método sqlite3.Cursor*), 584
fetchmany() (*método sqlite3.Cursor*), 584
fetchone() (*método sqlite3.Cursor*), 584
FF (*no módulo curses.ascii*), 1011
fflags (*atributo select.kevent*), 1311
Field (*classe em dataclasses*), 2079
field() (*no módulo dataclasses*), 2078
field_size_limit() (*no módulo csv*), 661
fieldnames (*atributo csv.DictReader*), 665
fields (*atributo uuid.UUID*), 1575
fields() (*no módulo dataclasses*), 2079
FIFOTYPE (*no módulo tarfile*), 644
file
    compileall opção de linha de comando, 2261
    gzip opção de linha de comando, 618
--file
    trace opção de linha de comando, 1989
file (*atributo bdb.Breakpoint*), 1956
file (*atributo pyclbr.Class*), 2259
file (*atributo pyclbr.Function*), 2258
file name
    temporary, 514
FILE_ATTRIBUTE_ARCHIVE (*no módulo stat*), 511
FILE_ATTRIBUTE_COMPRESSED (*no módulo stat*), 511
FILE_ATTRIBUTE_DEVICE (*no módulo stat*), 511
FILE_ATTRIBUTE_DIRECTORY (*no módulo stat*), 511
FILE_ATTRIBUTE_ENCRYPTED (*no módulo stat*), 511
FILE_ATTRIBUTE_HIDDEN (*no módulo stat*), 511
FILE_ATTRIBUTE_INTEGRITY_STREAM (*no módulo stat*), 511
FILE_ATTRIBUTE_NO_SCRUB_DATA (*no módulo stat*), 511
FILE_ATTRIBUTE_NORMAL (*no módulo stat*), 511
FILE_ATTRIBUTE_NOT_CONTENT_INDEXED (*no módulo stat*), 511
FILE_ATTRIBUTE_OFFLINE (*no módulo stat*), 511
FILE_ATTRIBUTE_READONLY (*no módulo stat*), 511
FILE_ATTRIBUTE_REPARSE_POINT (*no módulo stat*), 511
FILE_ATTRIBUTE_SPARSE_FILE (*no módulo stat*), 511
FILE_ATTRIBUTE_SYSTEM (*no módulo stat*), 511
FILE_ATTRIBUTE_TEMPORARY (*no módulo stat*), 511
FILE_ATTRIBUTE_VIRTUAL (*no módulo stat*), 511
file_digest() (*no módulo hashlib*), 696
file_open() (*método urllib.request.FileHandler*), 1523
file_size (*atributo zipfile.ZipInfo*), 640
filecmp
    module, 512
fileConfig() (*no módulo logging.config*), 827
FileCookieJar (*classe em http.cookiejar*), 1601
FileDialog (*classe em tkinter.filedialog*), 1678
FileExistsError, 123
FileFinder (*classe em importlib.machinery*), 2179
FileHandler (*classe em logging*), 839
FileHandler (*classe em urllib.request*), 1516
fileinput
    module, 981
FileInput (*classe em fileinput*), 982
FileIO (*classe em io*), 788
filelineno() (*no módulo fileinput*), 982
FileLoader (*classe em importlib.abc*), 2174
filemode() (*no módulo stat*), 507
filename (*atributo doctest.DocTest*), 1829
filename (*atributo http.cookiejar.FileCookieJar*), 1603
filename (*atributo inspect.FrameInfo*), 2136
filename (*atributo inspect.Traceback*), 2137
filename (*atributo netrc.NetrcParseError*), 688
filename (*atributo OSError*), 119
filename (*atributo SyntaxError*), 121
filename (*atributo traceback.FrameSummary*), 2113
filename (*atributo traceback.TracebackException*), 2111
filename (*atributo tracemalloc.Frame*), 1998
filename (*atributo zipfile.ZipFile*), 635
filename (*atributo zipfile.ZipInfo*), 638
filename() (*no módulo fileinput*), 982
filename2 (*atributo OSError*), 119
filename_only (*no módulo tabnanny*), 2257
filename_pattern (*atributo tracemalloc.Filter*), 1998
fileno() (*método bz2.BZ2File*), 620
fileno() (*método http.client.HTTPResponse*), 1548
fileno() (*método io.IOBase*), 785
fileno() (*método multiprocessing.connection.Connection*), 1055
fileno() (*método select.devpoll*), 1308

fileno() (*método select.epoll*), 1309
fileno() (*método select.kqueue*), 1311
fileno() (*método selectors.DevpollSelector*), 1315
fileno() (*método selectors.EpollSelector*), 1315
fileno() (*método selectors.KqueueSelector*), 1315
fileno() (*método socketserver.BaseServer*), 1583
fileno() (*método socket.socket*), 1260
fileno() (*no módulo fileinput*), 982
FileNotFoundError, 123
fileobj (*atributo selectors.SelectorKey*), 1313
files() (*método importlib.abc.TraversableResources*), 2177
files() (*método importlib.resources.abc.TraversableResources*), 2193
files() (*no módulo importlib.metadata*), 2197
files() (*no módulo importlib.resources*), 2188
files_double_event() (*método tkinter.filedialog.FileDialog*), 1678
files_select_event() (*método tkinter.filedialog.FileDialog*), 1678
FileType (*classe em argparse*), 932
FileWrapper (*classe em wsgiref.types*), 1509
FileWrapper (*classe em wsgiref.util*), 1502
fill() (*método textwrap.TextWrapper*), 187
fill() (*no módulo textwrap*), 184
fill() (*no módulo turtle*), 1730
fillcolor() (*no módulo turtle*), 1729
filling() (*no módulo turtle*), 1730
fillvalue (*atributo reprlib.Repr*), 334
--filter
    tarfile opção de linha de comando, 655
filter (*atributo select.kevent*), 1311
Filter (*classe em logging*), 817
Filter (*classe em tracemalloc*), 1997
filter()
    built-in function, 16
filter() (*método logging.Filter*), 818
filter() (*método logging.Handler*), 815
filter() (*método logging.Logger*), 813
filter() (*no módulo curses*), 985
filter() (*no módulo fnmatch*), 523
filter_command() (*método tkinter.filedialog.FileDialog*), 1678
FILTER_DIR (*no módulo unittest.mock*), 1903
filter_traces() (*método tracemalloc.Snapshot*), 1999
FilterError, 643
filterfalse() (*no módulo fnmatch*), 523
filterfalse() (*no módulo itertools*), 441
filterwarnings() (*no módulo warnings*), 2073
Final (*no módulo typing*), 1772
final() (*no módulo typing*), 1798
finalize (*classe em weakref*), 315
finalize_dict() (*no módulo compression.zstd*), 604
find() (*método bytearray*), 76
find() (*método bytes*), 76
find() (*método doctest.DocTestFinder*), 1830
find() (*método mmap.mmap*), 1328
find() (*método str*), 59
find() (*método xml.etree.ElementTree.Element*), 1452
find() (*método xml.etree.ElementTree.ElementTree*), 1454
find() (*no módulo gettext*), 1644
find_class() (*método pickle.Unpickler*), 542
find_class() (*protocolo pickle*), 552
find_library() (*no módulo ctypes.util*), 893
find_longest_match() (*método difflib.SequenceMatcher*), 176
find_msvcrt() (*no módulo ctypes.util*), 893
find_spec() (*método de classe importlib.machinery.PathFinder*), 2178
find_spec() (*método importlib.abc.MetaPathFinder*), 2171
find_spec() (*método importlib.abc.PathEntryFinder*), 2171
find_spec() (*método importlib.machinery.FileFinder*), 2179
find_spec() (*método zipimport.zipimporter*), 2160
find_spec() (*no módulo importlib.util*), 2184
find_unused_port() (*no módulo test.support.socket_helper*), 1942
find_user_password() (*método urllib.request.HTTPPasswordMgr*), 1521
find_user_password() (*método urllib.request.HTTPPasswordMgrWithPriorAuth*), 1521
findall() (*método re.Pattern*), 162
findall() (*método xml.etree.ElementTree.Element*), 1452
findall() (*método xml.etree.ElementTree.ElementTree*), 1454
findall() (*no módulo re*), 158
findCaller() (*método logging.Logger*), 813
findfile() (*no módulo test.support*), 1936
finditer() (*método re.Pattern*), 162
finditer() (*no módulo re*), 158
findlabels() (*no módulo dis*), 2270
findlinestarts() (*no módulo dis*), 2270
findtext() (*método xml.etree.ElementTree.Element*), 1452
findtext() (*método xml.etree.ElementTree.ElementTree*), 1454
finish() (*método socketserver.BaseRequestHandler*), 1585
finish() (*método tkinter.dnd.DndHandler*), 1682
finish_request() (*método socketserver.BaseServer*), 1584
FIRST_COMPLETED (*no módulo asyncio*), 1153
FIRST_COMPLETED (*no módulo concurrent.futures*), 1098
FIRST_EXCEPTION (*no módulo asyncio*), 1153
FIRST_EXCEPTION (*no módulo concurrent.futures*), 1098
firstChild (*atributo xml.dom.Node*), 1461
FirstHeaderLineIsContinuationDefect, 1356

firstkey() (*método dbm.gnu.gdbm*), 563
--first-weekday
    calendar opção de linha de comando, 275
firstweekday (*atributo calendar.Calendar*), 267
firstweekday() (*no módulo calendar*), 271
fix_missing_locations() (*no módulo ast*), 2239
fix_sentence_endings (*atributo textwrap.TextWrapper*), 186
Flag (*classe em enum*), 345
flag_bits (*atributo zipfile.ZipInfo*), 639
FlagBoundary (*classe em enum*), 348
flags (*atributo decimal.Context*), 388
flags (*atributo re.Pattern*), 162
flags (*atributo select.kevent*), 1311
flags (*no módulo sys*), 2027
flash() (*no módulo curses*), 985
flatten() (*método email.generator.BytesGenerator*), 1346
flatten() (*método email.generator.Generator*), 1347
flattening
    objetos, 537
--float
    random opção de linha de comando, 417
float (*classe interna*), 17
float_info (*no módulo sys*), 2029
float_repr_style (*no módulo sys*), 2031
FloatingPointError, 117
FloatOperation (*classe em decimal*), 395
flock() (*no módulo fcntl*), 2321
floor() (*no módulo math*), 42, 363
FloorDiv (*classe em ast*), 2213
floordiv() (*no módulo operator*), 464
flush() (*método bz2.BZ2Compressor*), 621
flush() (*método compression.zstd.ZstdCompressor*), 602
flush() (*método io.BufferedWriter*), 790
flush() (*método io.IOBase*), 785
flush() (*método logging.Handler*), 815
flush() (*método logging.handlers.BufferingHandler*), 849
flush() (*método logging.handlers.MemoryHandler*), 849
flush() (*método logging.StreamHandler*), 839
flush() (*método lzma.LZMACompressor*), 626
flush() (*método mailbox.Mailbox*), 1404
flush() (*método mailbox.Maildir*), 1407
flush() (*método mailbox.MH*), 1409
flush() (*método mmap.mmap*), 1328
flush() (*método xml.etree.ElementTree.XMLParser*), 1456
flush() (*método xml.etree.ElementTree.XMLPullParser*), 1457
flush() (*método zlib.Compress*), 613
flush() (*método zlib.Decompress*), 614
FLUSH_BLOCK (*atributo compression.zstd.ZstdCompressor*), 602
FLUSH_FRAME (*atributo compression.zstd.ZstdCompressor*), 603
flush_headers() (*método http.server.BaseHTTPRequestHandler*), 1593
flush_std_streams() (*no módulo test.support*), 1938
flushinp() (*no módulo curses*), 986
FlushKey() (*no módulo winreg*), 2296
fma() (*método decimal.Context*), 390
fma() (*método decimal.Decimal*), 382
fma() (*no módulo math*), 363
fmean() (*no módulo statistics*), 420
fmod() (*no módulo math*), 363
FMT_BINARY (*no módulo plistlib*), 691
FMT_XML (*no módulo plistlib*), 691
fnmatch
    module, 522
fnmatch() (*no módulo fnmatch*), 523
fnmatchcase() (*no módulo fnmatch*), 523
focus() (*método tkinter.ttk.Treeview*), 1694
fold (*atributo datetime.datetime*), 239
fold (*atributo datetime.time*), 247
fold() (*método email.headerregistry.BaseHeader*), 1357
fold() (*método email.policy.Compat32*), 1354
fold() (*método email.policy.EmailPolicy*), 1353
fold() (*método email.policy.Policy*), 1352
fold_binary() (*método email.policy.Compat32*), 1354
fold_binary() (*método email.policy.EmailPolicy*), 1353
fold_binary() (*método email.policy.Policy*), 1352
Font (*classe em tkinter.font*), 1675
For (*classe em ast*), 2222
FOR_ITER (*opcode*), 2281
forget() (*método tkinter.ttk.Notebook*), 1689
forget() (*no módulo test.support.import_helper*), 1947
fork() (*no módulo os*), 765
fork() (*no módulo pty*), 2318
ForkingMixIn (*classe em socketserver*), 1581
ForkingTCPServer (*classe em socketserver*), 1582
ForkingUDPServer (*classe em socketserver*), 1582
ForkingUnixDatagramServer (*classe em socketserver*), 1582
ForkingUnixStreamServer (*classe em socketserver*), 1582
forkpty() (*no módulo os*), 765
FORMAT (*atributo inspect.BufferFlags*), 2142
format (*atributo memoryview*), 94
format (*atributo multiprocessing.shared_memory.ShareableList*), 1089
format (*atributo struct.Struct*), 204
Format (*classe em annotationlib*), 2144
format()
    built-in function, 17
format() (*método inspect.Signature*), 2131
format() (*método logging.BufferingFormatter*), 817
format() (*método logging.Formatter*), 816
format() (*método logging.Handler*), 815

format() (*método pprint.PrettyPrinter*), 330
format() (*método str*), 59
format() (*método string.Formatter*), 130
format() (*método traceback.StackSummary*), 2112
format() (*método traceback.TracebackException*), 2111
format() (*método tracemalloc.Traceback*), 2001
format_call_graph() (*no módulo asyncio*), 1183
format_datetime() (*no módulo email.utils*), 1389
format_exc() (*no módulo traceback*), 2109
format_exception() (*no módulo traceback*), 2109
format_exception_only() (*método traceback.TracebackException*), 2112
format_exception_only() (*no módulo traceback*), 2109
format_field() (*método string.Formatter*), 131
format_frame_summary() (*método traceback.StackSummary*), 2112
format_help() (*método argparse.ArgumentParser*), 935
format_list() (*no módulo traceback*), 2109
format_map() (*método str*), 60
FORMAT_SIMPLE (*opcode*), 2285
format_spec (*atributo string.templatelib.Interpolation*), 146
format_stack() (*no módulo traceback*), 2110
format_stack_entry() (*método bdb.Bdb*), 1960
format_string() (*no módulo locale*), 1656
format_tb() (*no módulo traceback*), 2110
format_usage() (*método argparse.Action*), 925
format_usage() (*método argparse.ArgumentParser*), 935
FORMAT_WITH_SPEC (*opcode*), 2285
formatação
    bytearray (%), 85
    bytes (%), 85
formatação, string (%), 69
formataddr() (*no módulo email.utils*), 1388
formatargvalues() (*no módulo inspect*), 2135
formatdate() (*no módulo email.utils*), 1389
formatday() (*método calendar.TextCalendar*), 269
FormatError, 1419
FormatError() (*no módulo ctypes*), 893
formatException() (*método logging.Formatter*), 817
formatFooter() (*método logging.BufferingFormatter*), 817
formatHeader() (*método logging.BufferingFormatter*), 817
formatmonth() (*método calendar.HTMLCalendar*), 269
formatmonth() (*método calendar.TextCalendar*), 269
formatmonthname() (*método calendar.HTMLCalendar*), 270
formatmonthname() (*método calendar.TextCalendar*), 269
formatStack() (*método logging.Formatter*), 817
FormattedValue (*classe em ast*), 2209
Formatter (*classe em logging*), 816
Formatter (*classe em string*), 130
formatTime() (*método logging.Formatter*), 816
formatwarning() (*no módulo warnings*), 2073
formatweek() (*método calendar.TextCalendar*), 269
formatweekday() (*método calendar.TextCalendar*), 269
formatweekheader() (*método calendar.TextCalendar*), 269
formatyear() (*método calendar.HTMLCalendar*), 269
formatyear() (*método calendar.TextCalendar*), 269
formatyearpage() (*método calendar.HTMLCalendar*), 270
forward() (*no módulo turtle*), 1720
FORWARDREF (*atributo annotationlib.Format*), 2144
ForwardRef (*classe em annotationlib*), 2144
ForwardRef (*classe em typing*), 1802
fp (*atributo urllib.error.HTTPError*), 1538
fpathconf() (*no módulo os*), 723
Fraction (*classe em fractions*), 404
fractions
    module, 404
frame (*atributo inspect.FrameInfo*), 2136
frame (*atributo tkinter.scrolledtext.ScrolledText*), 1681
Frame (*classe em tracemalloc*), 1998
FrameInfo (*classe em compression.zstd*), 609
FrameInfo (*classe em inspect*), 2136
FrameSummary (*classe em traceback*), 2113
FrameType (*no módulo types*), 323
free_tool_id() (*no módulo sys.monitoring*), 2050
freedesktop_os_release() (*no módulo platform*), 856
freeze() (*no módulo gc*), 2122
freeze_support() (*no módulo multiprocessing*), 1053
frexp() (*no módulo math*), 364
FRIDAY (*no módulo calendar*), 272
from_address() (*método ctypes._CData*), 895
from_buffer() (*método ctypes._CData*), 895
from_buffer_copy() (*método ctypes._CData*), 895
from_bytes() (*método de classe int*), 44
from_callable() (*método de classe inspect.Signature*), 2131
from_decimal() (*método de classe fractions.Fraction*), 406
from_exception() (*método de classe traceback.TracebackException*), 2111
from_file() (*método de classe zipfile.ZipInfo*), 638
from_file() (*método de classe zoneinfo.ZoneInfo*), 264
from_float() (*método de classe decimal.Decimal*), 381
from_float() (*método de classe fractions.Fraction*), 405
from_iterable() (*método de classe itertools.chain*), 438
from_list() (*método de classe traceback.StackSummary*), 2112
from_number() (*método de classe complex*), 46
from_number() (*método de classe decimal.Decimal*), 382

from_number() (*método de classe float*), 45
from_number() (*método de classe fractions.Fraction*), 406
from_param() (*método ctypes._CData*), 896
from_samples() (*método de classe statistics.NormalDist*), 429
from_traceback() (*método de classe dis.Bytecode*), 2267
from_uri() (*método de classe pathlib.Path*), 484
frombuf() (*método de classe tarfile.TarInfo*), 649
frombytes() (*método array.array*), 310
fromfd() (*método select.epoll*), 1309
fromfd() (*método select.kqueue*), 1311
fromfd() (*no módulo socket*), 1253
fromfile() (*método array.array*), 310
fromhex() (*método de classe bytearray*), 73
fromhex() (*método de classe bytes*), 71
fromhex() (*método de classe float*), 46
fromisocalendar() (*método de classe datetime.date*), 230
fromisocalendar() (*método de classe datetime.datetime*), 238
fromisoformat() (*método de classe datetime.date*), 230
fromisoformat() (*método de classe datetime.datetime*), 237
fromisoformat() (*método de classe datetime.time*), 247
fromkeys() (*método collections.Counter*), 280
fromkeys() (*método de classe dict*), 100
fromlist() (*método array.array*), 310
fromordinal() (*método de classe datetime.date*), 230
fromordinal() (*método de classe datetime.datetime*), 237
fromshare() (*no módulo socket*), 1253
fromstring() (*no módulo xml.etree.ElementTree*), 1448
fromstringlist() (*no módulo xml.etree.ElementTree*), 1448
fromtarfile() (*método de classe tarfile.TarInfo*), 649
fromtimestamp() (*método de classe datetime.date*), 230
fromtimestamp() (*método de classe datetime.datetime*), 236
fromunicode() (*método array.array*), 310
fromutc() (*método datetime.timezone*), 258
fromutc() (*método datetime.tzinfo*), 252
FrozenImporter (*classe em importlib.machinery*), 2178
FrozenInstanceError, 2082
FrozenSet (*classe em typing*), 1804
frozenset (*classe interna*), 95
FS (*no módulo curses.ascii*), 1012
fs_is_case_insensitive() (*no módulo test.support.os_helper*), 1947
FS_NONASCII (*no módulo test.support.os_helper*), 1946
fsdecode() (*no módulo os*), 715
fsencode() (*no módulo os*), 714
fspath() (*no módulo os*), 715
fstat() (*no módulo os*), 724
fstatvfs() (*no módulo os*), 724
fstring, 67
FSTRING_END (*no módulo token*), 2248
FSTRING_MIDDLE (*no módulo token*), 2248
FSTRING_START (*no módulo token*), 2248
fsum() (*no módulo math*), 366
fsync() (*no módulo os*), 724
FTP, 1527
    ftplib (*standard module*), 1550
    protocolo, 1527, 1550
FTP (*classe em ftplib*), 1551
ftp_open() (*método urllib.request.FTPHandler*), 1523
FTP_TLS (*classe em ftplib*), 1555
FTPHandler (*classe em urllib.request*), 1516
ftplib
    module, 1550
ftruncate() (*no módulo os*), 724
Full, 1126
FULL (*atributo inspect.BufferFlags*), 2142
full() (*método asyncio.Queue*), 1179
full() (*método multiprocessing.Queue*), 1051
full() (*método queue.Queue*), 1126
full_match() (*método pathlib.PurePath*), 480
FULL_RO (*atributo inspect.BufferFlags*), 2142
full_url (*atributo urllib.request.Request*), 1516
fullmatch() (*método re.Pattern*), 161
fullmatch() (*no módulo re*), 157
fully_trusted_filter() (*no módulo tarfile*), 652
func (*atributo functools.partial*), 463
função, **2351**
função chave, **2354**
função de anotação, **2345**
função de avaliação, **2350**
função de corrotina, **2349**
função de perfilação, 1024, 2033, 2042
função de rastreamento, 1024, 2033, 2043
função de retorno, **2348**
função embutida
    compile, 111, 322
    complexo, 41
    eval, 111, 328, 330
    exec, 15, 111
    fatia, 2284
    hash, 51
    int, 41
    len, 49, 98
    max, 49
    min, 49
    ponto flutuante, 41
    tipo, 111
função genérica, **2352**
funcname (*atributo bdb.Breakpoint*), 1956
function (*atributo inspect.FrameInfo*), 2137
function (*atributo inspect.Traceback*), 2137
FUNCTION (*atributo symtable.SymbolTableType*), 2243
Function (*classe em pyclbr*), 2258

Function (*classe em symtable*), 2244
FunctionDef (*classe em ast*), 2233
FunctionTestCase (*classe em unittest*), 1858
FunctionType (*classe em ast*), 2208
FunctionType (*no módulo types*), 321
functools
    module, 452
funny_files (*atributo filecmp.dircmp*), 514
Future (*classe em asyncio*), 1211
Future (*classe em concurrent.futures*), 1097
future_add_to_awaited_by() (*no módulo asyncio*), 1184
future_discard_from_awaited_by() (*no módulo asyncio*), 1184
FutureWarning, 125
fwalk() (*no módulo os*), 755

G

-g
    trace opção de linha de comando, 1990
gaierror, 1245
gamma() (*no módulo math*), 368
gammavariate() (*no módulo random*), 411
gancho de entrada de caminho, **2358**
garbage (*no módulo gc*), 2122
gather() (*método curses.textpad.Textbox*), 1010
gather() (*no módulo asyncio*), 1148
gauss() (*no módulo random*), 411
gc
    module, 2119
gc_collect() (*no módulo test.support*), 1937
gcd() (*no módulo math*), 362
ge() (*no módulo operator*), 463
generate_tokens() (*no módulo tokenize*), 2253
Generator (*classe em collections.abc*), 297
Generator (*classe em email.generator*), 1346
Generator (*classe em typing*), 1808
generator_stop (*no módulo __future__*), 2118
GeneratorExit, 117
GeneratorExp (*classe em ast*), 2216
generators (*no módulo __future__*), 2118
GeneratorType (*no módulo types*), 321
Generic
    Alias, 104
Generic (*classe em typing*), 1778
generic_visit() (*método ast.NodeVisitor*), 2240
GenericAlias
    objeto, 104
GenericAlias (*classe em types*), 323
genops() (*no módulo pickletools*), 2290
geometric_mean() (*no módulo statistics*), 420
gerador, **2352**
gerador assíncrono, **2346**
gerenciador de contexto, 103, **2349**
gerenciador de contexto assíncrono, **2346**
gerenciador de janela (*widgets*), 1670
get() (*método asyncio.Queue*), 1179
get() (*método configparser.ConfigParser*), 683
get() (*método contextvars.Context*), 1131
get() (*método contextvars.ContextVar*), 1129
get() (*método dict*), 100
get() (*método email.message.EmailMessage*), 1337
get() (*método email.message.Message*), 1375
get() (*método mailbox.Mailbox*), 1403
get() (*método multiprocessing.pool.AsyncResult*), 1070
get() (*método multiprocessing.Queue*), 1051
get() (*método multiprocessing.SimpleQueue*), 1052
get() (*método queue.Queue*), 1126
get() (*método queue.SimpleQueue*), 1128
get() (*método tkinter.ttk.Combobox*), 1687
get() (*método tkinter.ttk.Spinbox*), 1687
get() (*método types.MappingProxyType*), 324
get() (*método xml.etree.ElementTree.Element*), 1451
get() (*no módulo webbrowser*), 1498
GET_AITER (*opcode*), 2274
get_all() (*método email.message.EmailMessage*), 1337
get_all() (*método email.message.Message*), 1375
get_all() (*método wsgiref.headers.Headers*), 1503
get_all_breaks() (*método bdb.Bdb*), 1959
get_all_start_methods() (*no módulo multiprocessing*), 1053
GET_ANEXT (*opcode*), 2274
get_annotate_from_class_namespace() (*no módulo annotationlib*), 2146
get_annotations() (*no módulo annotationlib*), 2146
get_annotations() (*no módulo inspect*), 2136
get_app() (*método wsgiref.simple_server.WSGIServer*), 1504
get_archive_formats() (*no módulo shutil*), 532
get_args() (*no módulo typing*), 1801
get_asyncgen_hooks() (*no módulo sys*), 2034
get_attribute() (*no módulo test.support*), 1940
GET_AWAITABLE (*opcode*), 2274
get_begidx() (*no módulo readline*), 194
get_blocking() (*no módulo os*), 724
get_body() (*método email.message.EmailMessage*), 1340
get_body_encoding() (*método email.charset.Charset*), 1385
get_boundary() (*método email.message.EmailMessage*), 1338
get_boundary() (*método email.message.Message*), 1377
get_bpbynumber() (*método bdb.Bdb*), 1959
get_break() (*método bdb.Bdb*), 1959
get_breaks() (*método bdb.Bdb*), 1959
get_buffer() (*método asyncio.BufferedProtocol*), 1219
get_bytes() (*método mailbox.Mailbox*), 1403
get_bytes() (*método mailbox.mbox*), 1408
get_bytes() (*método mailbox.MMDF*), 1411
get_ca_certs() (*método ssl.SSLContext*), 1289
get_cache_token() (*no módulo abc*), 2105
get_channel_binding() (*método ssl.SSLSocket*), 1286

get_charset() (*método email.message.Message*), 1374
get_charsets() (*método email.message.EmailMessage*), 1338
get_charsets() (*método email.message.Message*), 1377
get_children() (*método symtable.SymbolTable*), 2244
get_children() (*método tkinter.ttk.Treeview*), 1693
get_ciphers() (*método ssl.SSLContext*), 1289
get_clock_info() (*no módulo time*), 797
get_close_matches() (*no módulo difflib*), 173
get_code() (*método importlib.abc.InspectLoader*), 2173
get_code() (*método importlib.abc.SourceLoader*), 2175
get_code() (*método importlib.machinery.ExtensionFileLoader*), 2181
get_code() (*método importlib.machinery.SourcelessFileLoader*), 2180
get_code() (*método zipimport.zipimporter*), 2160
get_completer() (*no módulo readline*), 194
get_completer_delims() (*no módulo readline*), 194
get_completion_type() (*no módulo readline*), 194
get_config_h_filename() (*no módulo sysconfig*), 2060
get_config_var() (*no módulo sysconfig*), 2055
get_config_vars() (*no módulo sysconfig*), 2055
get_content() (*método email.contentmanager.ContentManager*), 1362
get_content() (*método email.message.EmailMessage*), 1340
get_content() (*no módulo email.contentmanager*), 1363
get_content_charset() (*método email.message.EmailMessage*), 1338
get_content_charset() (*método email.message.Message*), 1377
get_content_disposition() (*método email.message.EmailMessage*), 1339
get_content_disposition() (*método email.message.Message*), 1377
get_content_maintype() (*método email.message.EmailMessage*), 1337
get_content_maintype() (*método email.message.Message*), 1375
get_content_subtype() (*método email.message.EmailMessage*), 1337
get_content_subtype() (*método email.message.Message*), 1375
get_content_type() (*método email.message.EmailMessage*), 1337
get_content_type() (*método email.message.Message*), 1375
get_context() (*método asyncio.Handle*), 1204
get_context() (*método asyncio.Task*), 1159
get_context() (*no módulo multiprocessing*), 1054
get_coro() (*método asyncio.Task*), 1159
get_coroutine_origin_tracking_depth() (*no módulo sys*), 2034
get_count() (*no módulo gc*), 2121
get_current() (*no módulo concurrent.interpreters*), 1102
get_current_history_length() (*no módulo readline*), 193
get_data() (*método importlib.abc.FileLoader*), 2174
get_data() (*método importlib.abc.ResourceLoader*), 2172
get_data() (*método zipimport.zipimporter*), 2160
get_data() (*no módulo pkgutil*), 2163
get_date() (*método mailbox.MaildirMessage*), 1412
get_debug() (*método asyncio.loop*), 1202
get_debug() (*no módulo gc*), 2120
get_default() (*método argparse.ArgumentParser*), 934
get_default_backend() (*no módulo pdb*), 1967
get_default_scheme() (*no módulo sysconfig*), 2058
get_default_type() (*método email.message.EmailMessage*), 1338
get_default_type() (*método email.message.Message*), 1376
get_default_verify_paths() (*no módulo ssl*), 1275
get_dialect() (*no módulo csv*), 660
get_disassembly_as_string() (*método test.support.bytecode_helper.BytecodeTestCase*), 1944
get_docstring() (*no módulo ast*), 2239
get_doctest() (*método doctest.DocTestParser*), 1831
get_endidx() (*no módulo readline*), 194
get_environ() (*método wsgiref.simple_server.WSGIRequestHandler*), 1504
get_errno() (*no módulo ctypes*), 894
get_escdelay() (*no módulo curses*), 989
get_event_loop() (*método asyncio.AbstractEventLoopPolicy*), 1227
get_event_loop() (*no módulo asyncio*), 1185
get_event_loop_policy() (*no módulo asyncio*), 1227
get_events() (*no módulo sys.monitoring*), 2053
get_examples() (*método doctest.DocTestParser*), 1831
get_exception_handler() (*método asyncio.loop*), 1202
get_exec_path() (*no módulo os*), 715
get_extra_info() (*método asyncio.BaseTransport*), 1215
get_extra_info() (*método asyncio.StreamWriter*), 1165
get_field() (*método string.Formatter*), 130
get_file() (*método mailbox.Babyl*), 1410
get_file() (*método mailbox.Mailbox*), 1403
get_file() (*método mailbox.Maildir*), 1407
get_file() (*método mailbox.mbox*), 1408

get_file() (*método mailbox.MH*), 1409
get_file() (*método mailbox.MMDF*), 1411
get_file_breaks() (*método bdb.Bdb*), 1959
get_filename() (*método email.message.EmailMessage*), 1338
get_filename() (*método email.message.Message*), 1377
get_filename() (*método importlib.abc.ExecutionLoader*), 2173
get_filename() (*método importlib.abc.FileLoader*), 2174
get_filename() (*método importlib.machinery.ExtensionFileLoader*), 2181
get_filename() (*método zipimport.zipimporter*), 2160
get_filter() (*método tkinter.filedialog.FileDialog*), 1678
get_flags() (*método mailbox.Maildir*), 1405
get_flags() (*método mailbox.MaildirMessage*), 1412
get_flags() (*método mailbox.mboxMessage*), 1414
get_flags() (*método mailbox.MMDFMessage*), 1418
get_folder() (*método mailbox.Maildir*), 1405
get_folder() (*método mailbox.MH*), 1408
get_frame_info() (*no módulo compression.zstd*), 609
get_frees() (*método symtable.Function*), 2245
get_freeze_count() (*no módulo gc*), 2122
get_from() (*método mailbox.mboxMessage*), 1414
get_from() (*método mailbox.MMDFMessage*), 1417
get_full_url() (*método urllib.request.Request*), 1517
get_globals() (*método symtable.Function*), 2245
get_grouped_opcodes() (*método difflib.SequenceMatcher*), 177
get_handle_inheritable() (*no módulo os*), 734
get_header() (*método urllib.request.Request*), 1517
get_history_item() (*no módulo readline*), 193
get_history_length() (*no módulo readline*), 192
get_id() (*método symtable.SymbolTable*), 2244
get_ident() (*no módulo _thread*), 1133
get_ident() (*no módulo threading*), 1023
get_identifiers() (*método string.Template*), 140
get_identifiers() (*método symtable.SymbolTable*), 2244
get_importer() (*no módulo pkgutil*), 2162
get_info() (*método mailbox.Maildir*), 1406
get_info() (*método mailbox.MaildirMessage*), 1413
get_inheritable() (*método socket.socket*), 1260
get_inheritable() (*no módulo os*), 734
get_instructions() (*no módulo dis*), 2269
get_int_max_str_digits() (*no módulo sys*), 2032
get_interpreter() (*no módulo zipapp*), 2017
GET_ITER (*opcode*), 2273
get_key() (*método selectors.BaseSelector*), 1315
get_labels() (*método mailbox.Babyl*), 1410
get_labels() (*método mailbox.BabylMessage*), 1416
get_last_error() (*no módulo ctypes*), 894
GET_LEN (*opcode*), 2276
get_line_buffer() (*no módulo readline*), 192
get_lineno() (*método symtable.SymbolTable*), 2244
get_local_events() (*no módulo sys.monitoring*), 2053
get_locals() (*método symtable.Function*), 2244
get_logger() (*no módulo multiprocessing*), 1074
get_loop() (*método asyncio.Future*), 1212
get_loop() (*método asyncio.Runner*), 1140
get_loop() (*método asyncio.Server*), 1206
get_main() (*no módulo concurrent.interpreters*), 1102
get_makefile_filename() (*no módulo sysconfig*), 2060
get_map() (*método selectors.BaseSelector*), 1315
get_matching_blocks() (*método difflib.SequenceMatcher*), 176
get_message() (*método mailbox.Mailbox*), 1403
get_method() (*método urllib.request.Request*), 1517
get_methods() (*método symtable.Class*), 2245
get_mixed_type_key() (*no módulo ipaddress*), 1636
get_name() (*método asyncio.Task*), 1159
get_name() (*método symtable.Symbol*), 2245
get_name() (*método symtable.SymbolTable*), 2244
get_namespace() (*método symtable.Symbol*), 2247
get_namespaces() (*método symtable.Symbol*), 2247
get_native_id() (*no módulo _thread*), 1134
get_native_id() (*no módulo threading*), 1023
get_nonlocals() (*método symtable.Function*), 2245
get_nonstandard_attr() (*método http.cookiejar.Cookie*), 1608
get_nowait() (*método asyncio.Queue*), 1180
get_nowait() (*método multiprocessing.Queue*), 1051
get_nowait() (*método queue.Queue*), 1126
get_nowait() (*método queue.SimpleQueue*), 1128
get_object_traceback() (*no módulo tracemalloc*), 1996
get_objects() (*no módulo gc*), 2120
get_opcodes() (*método difflib.SequenceMatcher*), 176
get_option() (*método optparse.OptionParser*), 971
get_option_group() (*método optparse.OptionParser*), 961
get_origin() (*no módulo typing*), 1800
get_original_bases() (*no módulo types*), 320
get_original_stdout() (*no módulo test.support*), 1936
get_osfhandle() (*no módulo msvcrt*), 2292
get_output_charset() (*método email.charset.Charset*), 1385
get_overloads() (*no módulo typing*), 1798
get_pagesize() (*no módulo test.support*), 1936
get_param() (*método email.message.Message*), 1376
get_parameters() (*método symtable.Function*), 2244
get_params() (*método email.message.Message*), 1376
get_path() (*no módulo sysconfig*), 2059
get_path_names() (*no módulo sysconfig*), 2059
get_paths() (*no módulo sysconfig*), 2059
get_payload() (*método email.message.Message*), 1373
get_pid() (*método asyncio.SubprocessTransport*), 1217

get_pipe_transport() (*método asyncio.SubprocessTransport*), 1217
get_platform() (*no módulo sysconfig*), 2059
get_poly() (*no módulo turtle*), 1736
get_preferred_scheme() (*no módulo sysconfig*), 2058
get_protocol() (*método asyncio.BaseTransport*), 1215
get_protocol_members() (*no módulo typing*), 1801
get_proxy_response_headers() (*método http.client.HTTPConnection*), 1547
get_python_version() (*no módulo sysconfig*), 2059
get_ready() (*método graphlib.TopologicalSorter*), 354
get_referents() (*no módulo gc*), 2121
get_referrers() (*no módulo gc*), 2121
get_request() (*método socketserver.BaseServer*), 1584
get_returncode() (*método asyncio.SubprocessTransport*), 1217
get_running_loop() (*no módulo asyncio*), 1185
get_scheme() (*método wsgiref.handlers.BaseHandler*), 1507
get_scheme_names() (*no módulo sysconfig*), 2058
get_selection() (*método tkinter.filedialog.FileDialog*), 1678
get_sequences() (*método mailbox.MH*), 1409
get_sequences() (*método mailbox.MHMessage*), 1415
get_server() (*método multiprocessing.managers.BaseManager*), 1062
get_server_certificate() (*no módulo ssl*), 1275
get_shapepoly() (*no módulo turtle*), 1734
get_source() (*método importlib.abc.InspectLoader*), 2173
get_source() (*método importlib.abc.SourceLoader*), 2175
get_source() (*método importlib.machinery.ExtensionFileLoader*), 2181
get_source() (*método importlib.machinery.SourcelessFileLoader*), 2180
get_source() (*método zipimport.zipimporter*), 2160
get_source_segment() (*no módulo ast*), 2239
get_stack() (*método asyncio.Task*), 1158
get_stack() (*método bdb.Bdb*), 1960
get_start_method() (*no módulo multiprocessing*), 1054
get_starttag_text() (*método html.parser.HTMLParser*), 1435
get_stats() (*no módulo gc*), 2120
get_stats_profile() (*método pstats.Stats*), 1981
get_stderr() (*método wsgiref.handlers.BaseHandler*), 1506
get_stderr() (*método wsgiref.simple_server.WSGIRequestHandler*), 1504
get_stdin() (*método wsgiref.handlers.BaseHandler*), 1506
get_string() (*método mailbox.Mailbox*), 1403
get_string() (*método mailbox.mbox*), 1408
get_subdir() (*método mailbox.MaildirMessage*), 1412
get_symbols() (*método symtable.SymbolTable*), 2244
get_tabsize() (*no módulo curses*), 989
get_task_factory() (*método asyncio.loop*), 1190
get_terminal_size() (*no módulo os*), 734
get_terminal_size() (*no módulo shutil*), 535
get_threshold() (*no módulo gc*), 2121
get_token() (*método shlex.shlex*), 2309
get_tool() (*no módulo sys.monitoring*), 2050
get_traceback_limit() (*no módulo tracemalloc*), 1996
get_traced_memory() (*no módulo tracemalloc*), 1996
get_tracemalloc_memory() (*no módulo tracemalloc*), 1996
get_type() (*método symtable.SymbolTable*), 2244
get_type_hints() (*no módulo typing*), 1800
get_unixfrom() (*método email.message.EmailMessage*), 1335
get_unixfrom() (*método email.message.Message*), 1373
get_unpack_formats() (*no módulo shutil*), 534
get_unverified_chain() (*método ssl.SSLSocket*), 1285
get_usage() (*método optparse.OptionParser*), 972
get_value() (*método string.Formatter*), 131
get_verified_chain() (*método ssl.SSLSocket*), 1285
get_version() (*método optparse.OptionParser*), 962
get_visible() (*método mailbox.BabylMessage*), 1416
get_wch() (*método curses.window*), 993
get_write_buffer_limits() (*método asyncio.WriteTransport*), 1216
get_write_buffer_size() (*método asyncio.WriteTransport*), 1216
GET_YIELD_FROM_ITER (*opcode*), 2273
getacl() (*método imaplib.IMAP4*), 1563
getaddresses() (*no módulo email.utils*), 1388
getaddrinfo() (*método asyncio.loop*), 1199
getaddrinfo() (*no módulo socket*), 1254
getallocatedblocks() (*no módulo sys*), 2031
getandroidapilevel() (*no módulo sys*), 2031
getannotation() (*método imaplib.IMAP4*), 1563
getargvalues() (*no módulo inspect*), 2135
getasyncgenlocals() (*no módulo inspect*), 2140
getasyncgenstate() (*no módulo inspect*), 2140
getatime() (*no módulo os.path*), 501
getattr()
    built-in function, 18
getattr_static() (*no módulo inspect*), 2139
getAttribute() (*método xml.dom.Element*), 1464
getAttributeNode() (*método xml.dom.Element*), 1464
getAttributeNodeNS() (*método xml.dom.Element*), 1464
getAttributeNS() (*método xml.dom.Element*), 1464
GetBase() (*método xml.parsers.expat.xmlparser*), 1488

getbegyx() (*método curses.window*), 993
getbkgd() (*método curses.window*), 993
getblocking() (*método socket.socket*), 1261
getboolean() (*método configparser.ConfigParser*), 683
getbuffer() (*método io.BytesIO*), 789
getByteStream() (*método xml.sax.xmlreader.InputSource*), 1485
getcallargs() (*no módulo inspect*), 2135
getcanvas() (*no módulo turtle*), 1743
getch() (*método curses.window*), 993
getch() (*no módulo msvcrt*), 2292
getCharacterStream() (*método xml.sax.xmlreader.InputSource*), 1485
getche() (*no módulo msvcrt*), 2292
getChild() (*método logging.Logger*), 811
getChildren() (*método logging.Logger*), 811
getclasstree() (*no módulo inspect*), 2134
getclosurevars() (*no módulo inspect*), 2136
getcode() (*método http.client.HTTPResponse*), 1549
getcode() (*método urllib.response.addinfourl*), 1528
getColumnNumber() (*método xml.sax.xmlreader.Locator*), 1484
getcomments() (*no módulo inspect*), 2129
getcompname() (*método wave.Wave_read*), 1640
getcomptype() (*método wave.Wave_read*), 1640
getconfig() (*método sqlite3.Connection*), 580
getContentHandler() (*método xml.sax.xmlreader.XMLReader*), 1483
getcontext() (*no módulo decimal*), 386
getcoroutinelocals() (*no módulo inspect*), 2140
getcoroutinestate() (*no módulo inspect*), 2140
getctime() (*no módulo os.path*), 501
getcwd() (*no módulo os*), 738
getcwdb() (*no módulo os*), 738
getdecoder() (*no módulo codecs*), 205
getdefaultencoding() (*no módulo sys*), 2031
getdefaultlocale() (*no módulo locale*), 1655
getdefaulttimeout() (*no módulo socket*), 1258
getdlopenflags() (*no módulo sys*), 2031
getdoc() (*no módulo inspect*), 2129
getDOMImplementation() (*no módulo xml.dom*), 1459
getDTDHandler() (*método xml.sax.xmlreader.XMLReader*), 1483
getEffectiveLevel() (*método logging.Logger*), 811
getegid() (*no módulo os*), 715
getElementsByTagName() (*método xml.dom.Document*), 1464
getElementsByTagName() (*método xml.dom.Element*), 1464
getElementsByTagNameNS() (*método xml.dom.Document*), 1464
getElementsByTagNameNS() (*método xml.dom.Element*), 1464
getencoder() (*no módulo codecs*), 205
getEncoding() (*método xml.sax.xmlreader.InputSource*), 1485
getencoding() (*no módulo locale*), 1656
getEntityResolver() (*método xml.sax.xmlreader.XMLReader*), 1484
getenv() (*no módulo os*), 715
getenvb() (*no módulo os*), 715
getErrorHandler() (*método xml.sax.xmlreader.XMLReader*), 1484
geteuid() (*no módulo os*), 716
getEvent() (*método xml.dom.pulldom.DOMEventStream*), 1474
getEventCategory() (*método logging.handlers.NTEventLogHandler*), 848
getEventType() (*método logging.handlers.NTEventLogHandler*), 848
getException() (*método xml.sax.SAXException*), 1476
getFeature() (*método xml.sax.xmlreader.XMLReader*), 1484
getfile() (*no módulo inspect*), 2129
getFilesToDelete() (*método logging.handlers.TimedRotatingFileHandler*), 843
getfilesystemencodeerrors() (*no módulo sys*), 2032
getfilesystemencoding() (*no módulo sys*), 2031
getfirstweekday() (*método calendar.Calendar*), 267
getfloat() (*método configparser.ConfigParser*), 683
getfqdn() (*no módulo socket*), 1255
getframeinfo() (*no módulo inspect*), 2138
getframerate() (*método wave.Wave_read*), 1640
getfullargspec() (*no módulo inspect*), 2134
getgeneratorlocals() (*no módulo inspect*), 2140
getgeneratorstate() (*no módulo inspect*), 2139
getgid() (*no módulo os*), 716
getgrall() (*no módulo grp*), 2315
getgrgid() (*no módulo grp*), 2315
getgrnam() (*no módulo grp*), 2315
getgrouplist() (*no módulo os*), 716
getgroups() (*no módulo os*), 716
getHandlerByName() (*no módulo logging*), 823
getHandlerNames() (*no módulo logging*), 823
getheader() (*método http.client.HTTPResponse*), 1548
getheaders() (*método http.client.HTTPResponse*), 1548
gethostbyaddr() (*no módulo socket*), 720, 1255
gethostbyname() (*no módulo socket*), 1255
gethostbyname_ex() (*no módulo socket*), 1255
gethostname() (*no módulo socket*), 720, 1255
getincrementaldecoder() (*no módulo codecs*), 206
getincrementalencoder() (*no módulo codecs*), 206
getinfo() (*método zipfile.ZipFile*), 632
getinnerframes() (*no módulo inspect*), 2138
GetInputContext() (*método xml.parsers.expat.xmlparser*), 1488
getint() (*método configparser.ConfigParser*), 683
getitem() (*no módulo operator*), 466
getitimer() (*no módulo signal*), 1322

getkey() (*método curses.window*), 994
GetLastError() (*no módulo ctypes*), 893
getLength() (*método xml.sax.xmlreader.Attributes*), 1486
getLevelName() (*no módulo logging*), 823
getLevelNamesMapping() (*no módulo logging*), 823
getlimit() (*método sqlite3.Connection*), 580
getline() (*no módulo linecache*), 524
getLineNumber() (*método xml.sax.xmlreader.Locator*), 1485
getloadavg() (*no módulo os*), 778
getlocale() (*no módulo locale*), 1655
getLogger() (*no módulo logging*), 821
getLoggerClass() (*no módulo logging*), 821
getlogin() (*no módulo os*), 716
getLogRecordFactory() (*no módulo logging*), 822
getMandatoryRelease() (*método __future__._Feature*), 2119
getmark() (*método wave.Wave_read*), 1640
getmarkers() (*método wave.Wave_read*), 1640
getmaxyx() (*método curses.window*), 994
getmember() (*método tarfile.TarFile*), 646
getmembers() (*método tarfile.TarFile*), 646
getmembers() (*no módulo inspect*), 2125
getmembers_static() (*no módulo inspect*), 2125
getMessage() (*método logging.LogRecord*), 819
getMessage() (*método xml.sax.SAXException*), 1476
getMessageID() (*método logging.handlers.NTEventLogHandler*), 848
getmodule() (*no módulo inspect*), 2129
getmodulename() (*no módulo inspect*), 2126
getmouse() (*no módulo curses*), 986
getmro() (*no módulo inspect*), 2135
getmtime() (*no módulo os.path*), 501
getName() (*método threading.Thread*), 1029
getNameByQName() (*método xml.sax.xmlreader.AttributesNS*), 1486
getnameinfo() (*método asyncio.loop*), 1199
getnameinfo() (*no módulo socket*), 1255
getnames() (*método tarfile.TarFile*), 646
getNames() (*método xml.sax.xmlreader.Attributes*), 1486
getnchannels() (*método wave.Wave_read*), 1640
getnframes() (*método wave.Wave_read*), 1640
getnode() (*no módulo uuid*), 1576
getobjects() (*no módulo sys*), 2033
getopt
    module, 2333
getopt() (*no módulo getopt*), 2333
GetoptError, 2334
getOptionalRelease() (*método __future__._Feature*), 2119
getouterframes() (*no módulo inspect*), 2138
getoutput() (*no módulo subprocess*), 1122
getpagesize() (*no módulo resource*), 2326
getparams() (*método wave.Wave_read*), 1640
getparyx() (*método curses.window*), 994
getpass
    module, 980
getpass() (*no módulo getpass*), 980
GetPassWarning, 980
getpeercert() (*método ssl.SSLSocket*), 1284
getpeername() (*método socket.socket*), 1260
getpen() (*no módulo turtle*), 1736
getpgid() (*no módulo os*), 716
getpgrp() (*no módulo os*), 716
getpid() (*no módulo os*), 716
getpos() (*método html.parser.HTMLParser*), 1435
getppid() (*no módulo os*), 716
getpreferredencoding() (*no módulo locale*), 1655
getpriority() (*no módulo os*), 717
getprofile() (*no módulo sys*), 2033
getprofile() (*no módulo threading*), 1024
getProperty() (*método xml.sax.xmlreader.XMLReader*), 1484
getprotobyname() (*no módulo socket*), 1256
getproxies() (*no módulo urllib.request*), 1513
getPublicId() (*método xml.sax.xmlreader.InputSource*), 1485
getPublicId() (*método xml.sax.xmlreader.Locator*), 1485
getpwall() (*no módulo pwd*), 2314
getpwnam() (*no módulo pwd*), 2314
getpwuid() (*no módulo pwd*), 2314
getQNameByName() (*método xml.sax.xmlreader.AttributesNS*), 1486
getQNames() (*método xml.sax.xmlreader.AttributesNS*), 1486
getquota() (*método imaplib.IMAP4*), 1563
getquotaroot() (*método imaplib.IMAP4*), 1563
getrandbits() (*método random.Random*), 412
getrandbits() (*no módulo random*), 409
getrandom() (*no módulo os*), 779
getreader() (*no módulo codecs*), 206
getrecursionlimit() (*no módulo sys*), 2032
getrefcount() (*no módulo sys*), 2032
GetReparseDeferralEnabled() (*método xml.parsers.expat.xmlparser*), 1489
getresgid() (*no módulo os*), 717
getresponse() (*método http.client.HTTPConnection*), 1546
getresuid() (*no módulo os*), 717
getrlimit() (*no módulo resource*), 2323
getroot() (*método xml.etree.ElementTree.ElementTree*), 1454
getrusage() (*no módulo resource*), 2325
getsampwidth() (*método wave.Wave_read*), 1640
getscreen() (*no módulo turtle*), 1736
getservbyname() (*no módulo socket*), 1256
getservbyport() (*no módulo socket*), 1256
GetSetDescriptorType (*no módulo types*), 323
getshapes() (*no módulo turtle*), 1743
getsid() (*no módulo os*), 720
getsignal() (*no módulo signal*), 1320
getsitepackages() (*no módulo site*), 2154
getsize() (*no módulo os.path*), 501

getsizeof() (*no módulo sys*), 2032
getsockname() (*método socket.socket*), 1260
getsockopt() (*método socket.socket*), 1261
getsource() (*no módulo inspect*), 2129
getsourcefile() (*no módulo inspect*), 2129
getsourcelines() (*no módulo inspect*), 2129
getstate() (*método codecs.IncrementalDecoder*), 211
getstate() (*método codecs.IncrementalEncoder*), 211
getstate() (*método random.Random*), 412
getstate() (*no módulo random*), 408
getstatusoutput() (*no módulo subprocess*), 1121
getstr() (*método curses.window*), 994
getSubject() (*método logging.handlers.SMTPHandler*), 848
getswitchinterval() (*no módulo sys*), 2033
getSystemId() (*método xml.sax.xmlreader.InputSource*), 1485
getSystemId() (*método xml.sax.xmlreader.Locator*), 1485
getsyx() (*no módulo curses*), 986
gettarinfo() (*método tarfile.TarFile*), 648
gettempdir() (*no módulo tempfile*), 517
gettempdirb() (*no módulo tempfile*), 518
gettempprefix() (*no módulo tempfile*), 518
gettempprefixb() (*no módulo tempfile*), 518
getTestCaseNames() (*método unittest.TestLoader*), 1860
gettext
    module, 1643
gettext() (*método gettext.GNUTranslations*), 1647
gettext() (*método gettext.NullTranslations*), 1646
gettext() (*no módulo gettext*), 1644
gettext() (*no módulo locale*), 1659
gettimeout() (*método socket.socket*), 1261
gettrace() (*no módulo sys*), 2033
gettrace() (*no módulo threading*), 1024
getturtle() (*no módulo turtle*), 1736
getType() (*método xml.sax.xmlreader.Attributes*), 1486
getuid() (*no módulo os*), 717
getunicodeinternedsize() (*no módulo sys*), 2031
geturl() (*método http.client.HTTPResponse*), 1549
geturl() (*método urllib.parse.urllib.parse.SplitResult*), 1534
geturl() (*método urllib.response.addinfourl*), 1528
getuser() (*no módulo getpass*), 980
getuserbase() (*no módulo site*), 2154
getusersitepackages() (*no módulo site*), 2154
getvalue() (*método io.BytesIO*), 789
getvalue() (*método io.StringIO*), 793
getValue() (*método xml.sax.xmlreader.Attributes*), 1486
getValueByQName() (*método xml.sax.xmlreader.AttributesNS*), 1486
getwch() (*no módulo msvcrt*), 2292
getwche() (*no módulo msvcrt*), 2292
getweakrefcount() (*no módulo weakref*), 313
getweakrefs() (*no módulo weakref*), 313
getwelcome() (*método ftplib.FTP*), 1552
getwelcome() (*método poplib.POP3*), 1558
getwin() (*no módulo curses*), 986
getwindowsversion() (*no módulo sys*), 2034
getwriter() (*no módulo codecs*), 206
getxattr() (*no módulo os*), 761
getyx() (*método curses.window*), 994
gid (*atributo tarfile.TarInfo*), 650
GIL, **2353**
gil (*atributo sys.flags*), 2028
glob
    module, 520
    módulo, 523
glob() (*método pathlib.Path*), 490
glob() (*no módulo glob*), 520
Global (*classe em ast*), 2235
global_enum() (*no módulo enum*), 351
globals()
    built-in function, 18
globs (*atributo doctest.DocTest*), 1829
gmtime() (*no módulo time*), 798
gname (*atributo tarfile.TarInfo*), 650
GNOME, 1648
GNU_FORMAT (*no módulo tarfile*), 645
gnu_getopt() (*no módulo getopt*), 2334
GNUTranslations (*classe em gettext*), 1647
GNUTYPE_LONGLINK (*no módulo tarfile*), 644
GNUTYPE_LONGNAME (*no módulo tarfile*), 644
GNUTYPE_SPARSE (*no módulo tarfile*), 644
go() (*método tkinter.filedialog.FileDialog*), 1678
got (*atributo doctest.DocTestFailure*), 1836
goto() (*no módulo turtle*), 1721
grantpt() (*no módulo os*), 724
graphlib
    module, 352
GREATER (*no módulo token*), 2250
GREATEREQUAL (*no módulo token*), 2250
greedy (*atributo compression.zstd.Strategy*), 609
GRND_NONBLOCK (*no módulo os*), 780
GRND_RANDOM (*no módulo os*), 780
Group (*classe em email.headerregistry*), 1361
group() (*método pathlib.Path*), 495
group() (*método re.Match*), 163
groupby() (*no módulo itertools*), 441
groupdict() (*método re.Match*), 164
groupindex (*atributo re.Pattern*), 162
groups (*atributo email.headerregistry.AddressHeader*), 1358
groups (*atributo re.Pattern*), 162
groups() (*método re.Match*), 164
grp
    module, 2314
GS (*no módulo curses.ascii*), 1012
Gt (*classe em ast*), 2214
gt() (*no módulo operator*), 463
GtE (*classe em ast*), 2214
guess_all_extensions() (*método mimetypes.MimeTypes*), 1423

guess_all_extensions() (*no módulo mimetypes*), 1421
guess_extension() (*método mimetypes.MimeTypes*), 1423
guess_extension() (*no módulo mimetypes*), 1421
guess_file_type() (*método mimetypes.MimeTypes*), 1423
guess_file_type() (*no módulo mimetypes*), 1421
guess_scheme() (*no módulo wsgiref.util*), 1500
guess_type() (*método mimetypes.MimeTypes*), 1423
guess_type() (*no módulo mimetypes*), 1420
GUI, 1661
gzip
    module, 615
gzip opção de linha de comando
    --best, 618
    -d, 618
    --decompress, 618
    --fast, 618
    file, 618
    -h, 618
    --help, 618
GzipFile (*classe em gzip*), 615

H

-h
    ast opção de linha de comando, 2242
    calendar opção de linha de comando, 274
    dis opção de linha de comando, 2266
    gzip opção de linha de comando, 618
    idle opção de linha de comando, 1708
    json opção de linha de comando, 1401
    mimetypes opção de linha de comando, 1424
    python--m-sqlite3-[-h]-[-v]-[filename]-[sql] opção de linha de comando, 589
    random opção de linha de comando, 416
    timeit opção de linha de comando, 1986
    tokenize opção de linha de comando, 2254
    uuid opção de linha de comando, 1578
    zipapp opção de linha de comando, 2016
halfdelay() (*no módulo curses*), 986
Handle (*classe em asyncio*), 1204
handle() (*método http.server.BaseHTTPRequestHandler*), 1592
handle() (*método logging.Handler*), 815
handle() (*método logging.handlers.QueueListener*), 852
handle() (*método logging.Logger*), 813
handle() (*método logging.NullHandler*), 840
handle() (*método socketserver.BaseRequestHandler*), 1585
handle() (*método wsgiref.simple_server.WSGIRequestHandler*), 1504
handle_charref() (*método html.parser.HTMLParser*), 1435
handle_comment() (*método html.parser.HTMLParser*), 1435
handle_data() (*método html.parser.HTMLParser*), 1435
handle_decl() (*método html.parser.HTMLParser*), 1436
handle_defect() (*método email.policy.Policy*), 1351
handle_endtag() (*método html.parser.HTMLParser*), 1435
handle_entityref() (*método html.parser.HTMLParser*), 1435
handle_error() (*método socketserver.BaseServer*), 1584
handle_expect_100() (*método http.server.BaseHTTPRequestHandler*), 1592
handle_one_request() (*método http.server.BaseHTTPRequestHandler*), 1592
handle_pi() (*método html.parser.HTMLParser*), 1436
handle_request() (*método socketserver.BaseServer*), 1583
handle_request() (*método xmlrpc.server.CGIXMLRPCRequestHandler*), 1621
handle_startendtag() (*método html.parser.HTMLParser*), 1435
handle_starttag() (*método html.parser.HTMLParser*), 1435
handle_timeout() (*método socketserver.BaseServer*), 1584
handleError() (*método logging.Handler*), 815
handleError() (*método logging.handlers.SocketHandler*), 844
Handler (*classe em logging*), 814
handlers (*atributo logging.Logger*), 810
Handlers (*classe em signal*), 1317
hardlink_to() (*método pathlib.Path*), 493
--hardlink-dupes
    compileall opção de linha de comando, 2262
harmonic_mean() (*no módulo statistics*), 420
HAS_ALPN (*no módulo ssl*), 1280
has_children() (*método symtable.SymbolTable*), 2244
has_colors() (*no módulo curses*), 986
has_default() (*método typing.ParamSpec*), 1784
has_default() (*método typing.TypeVar*), 1781
has_default() (*método typing.TypeVarTuple*), 1783
has_dualstack_ipv6() (*no módulo socket*), 1253
HAS_ECDH (*no módulo ssl*), 1280
has_extended_color_support() (*no módulo curses*), 986
has_extn() (*método smtplib.SMTP*), 1571
has_header() (*método csv.Sniffer*), 662
has_header() (*método urllib.request.Request*), 1517

has_ic() (*no módulo curses*), 986
has_il() (*no módulo curses*), 986
has_ipv6 (*no módulo socket*), 1249
has_key() (*no módulo curses*), 986
has_location (*atributo importlib.machinery.ModuleSpec*), 2182
HAS_NEVER_CHECK_COMMON_NAME (*no módulo ssl*), 1280
has_nonstandard_attr() (*método http.cookiejar.Cookie*), 1608
HAS_NPN (*no módulo ssl*), 1280
has_option() (*método configparser.ConfigParser*), 682
has_option() (*método optparse.OptionParser*), 971
HAS_PHA (*no módulo ssl*), 1281
HAS_PSK (*no módulo ssl*), 1281
has_section() (*método configparser.ConfigParser*), 682
HAS_SNI (*no módulo ssl*), 1280
HAS_SSLv2 (*no módulo ssl*), 1281
HAS_SSLv3 (*no módulo ssl*), 1281
has_ticket (*atributo ssl.SSLSession*), 1303
HAS_TLSv1 (*no módulo ssl*), 1281
HAS_TLSv1_1 (*no módulo ssl*), 1281
HAS_TLSv1_2 (*no módulo ssl*), 1281
HAS_TLSv1_3 (*no módulo ssl*), 1281
hasarg (*no módulo dis*), 2288
hasattr()
    built-in function, 18
hasAttribute() (*método xml.dom.Element*), 1464
hasAttributeNS() (*método xml.dom.Element*), 1464
hasAttributes() (*método xml.dom.Node*), 1461
hasChildNodes() (*método xml.dom.Node*), 1462
hascompare (*no módulo dis*), 2288
hasconst (*no módulo dis*), 2288
hasexc (*no módulo dis*), 2288
hasFeature() (*método xml.dom.DOMImplementation*), 1460
hasfree (*no módulo dis*), 2288
hash
    função embutida, 51
hash()
    built-in function, 18
hash_bits (*atributo sys.hash_info*), 2035
hash_info (*no módulo sys*), 2035
hash_log (*atributo compression.zstd.CompressionParameter*), 606
hash_randomization (*atributo sys.flags*), 2028
Hashable (*classe em collections.abc*), 297
Hashable (*classe em typing*), 1808
hasHandlers() (*método logging.Logger*), 813
hash.block_size (*no módulo hashlib*), 695
hash.digest_size (*no módulo hashlib*), 695
hasheável, **2353**
hashlib
    module, 693
hasjabs (*no módulo dis*), 2288
hasjrel (*no módulo dis*), 2288
hasjump (*no módulo dis*), 2288
haslocal (*no módulo dis*), 2288
hasname (*no módulo dis*), 2288
HAVE_ARGUMENT (*opcode*), 2286
HAVE_CONTEXTVAR (*no módulo decimal*), 393
HAVE_DOCSTRINGS (*no módulo test.support*), 1935
HAVE_THREADS (*no módulo decimal*), 393
HCI_CHANNEL_CONTROL (*no módulo socket*), 1250
HCI_CHANNEL_LOGGING (*no módulo socket*), 1250
HCI_CHANNEL_MONITOR (*no módulo socket*), 1250
HCI_CHANNEL_RAW (*no módulo socket*), 1250
HCI_CHANNEL_USER (*no módulo socket*), 1250
HCI_DATA_DIR (*no módulo socket*), 1250
HCI_DEV_NONE (*no módulo socket*), 1250
HCI_FILTER (*no módulo socket*), 1250
HCI_TIME_STAMP (*no módulo socket*), 1250
Header (*classe em email.header*), 1382
header_encode() (*método email.charset.Charset*), 1385
header_encode_lines() (*método email.charset.Charset*), 1385
header_encoding (*atributo email.charset.Charset*), 1385
header_factory (*atributo email.policy.EmailPolicy*), 1352
header_fetch_parse() (*método email.policy.Compat32*), 1354
header_fetch_parse() (*método email.policy.EmailPolicy*), 1353
header_fetch_parse() (*método email.policy.Policy*), 1351
header_items() (*método urllib.request.Request*), 1518
header_max_count() (*método email.policy.EmailPolicy*), 1353
header_max_count() (*método email.policy.Policy*), 1351
header_offset (*atributo zipfile.ZipInfo*), 639
header_source_parse() (*método email.policy.Compat32*), 1354
header_source_parse() (*método email.policy.EmailPolicy*), 1353
header_source_parse() (*método email.policy.Policy*), 1351
header_store_parse() (*método email.policy.Compat32*), 1354
header_store_parse() (*método email.policy.EmailPolicy*), 1353
header_store_parse() (*método email.policy.Policy*), 1351
HeaderDefect, 1355
HeaderError, 643
HeaderParseError, 1355
HeaderParser (*classe em email.parser*), 1344
HeaderRegistry (*classe em email.headerregistry*), 1359
headers
    MIME, 1420, 1421
headers (*atributo http.client.HTTPResponse*), 1548

headers (*atributo http.server.BaseHTTPRequestHandler*), 1591
headers (*atributo urllib.error.HTTPError*), 1538
headers (*atributo urllib.response.addinfourl*), 1528
headers (*atributo xmlrpc.client.ProtocolError*), 1614
Headers (*classe em wsgiref.headers*), 1502
HeaderWriteError, 1355
heading() (*método tkinter.ttk.Treeview*), 1694
heading() (*no módulo turtle*), 1726
heapify() (*no módulo heapq*), 301
heapify_max() (*no módulo heapq*), 301
heapmin() (*no módulo msvcrt*), 2292
heappop() (*no módulo heapq*), 301
heappop_max() (*no módulo heapq*), 301
heappush() (*no módulo heapq*), 300
heappush_max() (*no módulo heapq*), 301
heappushpop() (*no módulo heapq*), 301
heappushpop_max() (*no módulo heapq*), 301
heapq
    module, 300
heapreplace() (*no módulo heapq*), 301
heapreplace_max() (*no módulo heapq*), 301
helo() (*método smtplib.SMTP*), 1570
help
    online, 1809
--help
    ast opção de linha de comando, 2242
    calendar opção de linha de comando, 274
    dis opção de linha de comando, 2266
    gzip opção de linha de comando, 618
    json opção de linha de comando, 1401
    mimetypes opção de linha de comando, 1424
    python--m-sqlite3-[-h]-[-v]-[filename]-[sql] opção de linha de comando, 589
    random opção de linha de comando, 416
    timeit opção de linha de comando, 1986
    tokenize opção de linha de comando, 2254
    trace opção de linha de comando, 1989
    uuid opção de linha de comando, 1578
    zipapp opção de linha de comando, 2016
help (*atributo optparse.Option*), 966
help (*pdb command*), 1969
help()
    built-in function, 18
herror, 1245
hex (*atributo uuid.UUID*), 1576
hex()
    built-in function, 19
hex() (*método bytearray*), 73
hex() (*método bytes*), 72
hex() (*método float*), 46
hex() (*método memoryview*), 90
hexadecimal
    literais, 41
hexdigest() (*método hashlib.hash*), 696
hexdigest() (*método hashlib.shake*), 696
hexdigest() (*método hmac.HMAC*), 705
hexdigits (*no módulo string*), 129
hexlify() (*no módulo binascii*), 1430
hexversion (*no módulo sys*), 2035
hidden() (*método curses.panel.Panel*), 1015
hide() (*método curses.panel.Panel*), 1015
hide() (*método tkinter.ttk.Notebook*), 1689
hide_cookie2 (*atributo http.cookiejar.CookiePolicy*), 1605
hideturtle() (*no módulo turtle*), 1731
HierarchyRequestErr, 1466
HIGH_PRIORITY_CLASS (*no módulo subprocess*), 1117
HIGHEST_PROTOCOL (*no módulo pickle*), 539
hits (*atributo bdb.Breakpoint*), 1956
HKEY_CLASSES_ROOT (*no módulo winreg*), 2300
HKEY_CURRENT_CONFIG (*no módulo winreg*), 2300
HKEY_CURRENT_USER (*no módulo winreg*), 2300
HKEY_DYN_DATA (*no módulo winreg*), 2300
HKEY_LOCAL_MACHINE (*no módulo winreg*), 2300
HKEY_PERFORMANCE_DATA (*no módulo winreg*), 2300
HKEY_USERS (*no módulo winreg*), 2300
hline() (*método curses.window*), 994
hls_to_rgb() (*no módulo colorsys*), 1642
hmac
    module, 705
HMAC (*classe em hmac*), 705
HOME, 501, 1663
home() (*método de classe pathlib.Path*), 485
home() (*no módulo turtle*), 1722
HOMEDRIVE, 501
HOMEPATH, 501
hook_compressed() (*no módulo fileinput*), 983
hook_encoded() (*no módulo fileinput*), 983
horário de verão, 795
host (*atributo urllib.request.Request*), 1516
hostmask (*atributo ipaddress.IPv4Network*), 1630
hostmask (*atributo ipaddress.IPv6Network*), 1633
hostname_checks_common_name (*atributo ssl.SSLContext*), 1294
hosts (*atributo netrc.netrc*), 689
hosts() (*método ipaddress.IPv4Network*), 1630
hosts() (*método ipaddress.IPv6Network*), 1633
hour (*atributo datetime.datetime*), 239
hour (*atributo datetime.time*), 247
hresult (*atributo ctypes.COMError*), 903
HRESULT (*classe em ctypes*), 899
hStdError (*atributo subprocess.STARTUPINFO*), 1116
hStdInput (*atributo subprocess.STARTUPINFO*), 1115
hStdOutput (*atributo subprocess.STARTUPINFO*), 1115
hsv_to_rgb() (*no módulo colorsys*), 1642
HT (*no módulo curses.ascii*), 1011
ht() (*no módulo turtle*), 1731
HTML, 1434, 1527
html
    module, 1433
html5 (*no módulo html.entities*), 1438

HTMLCalendar (*classe em calendar*), 269
HtmlDiff (*classe em difflib*), 171
html.entities
    module, 1438
html.parser
    module, 1434
HTMLParser (*classe em html.parser*), 1434
htonl() (*no módulo socket*), 1256
htons() (*no módulo socket*), 1256
HTTP
    http (*módulo padrão*), 1539
    http.client (*standard module*), 1543
    protocolo, 1527, 1539, 1543, 1589
http
    module, 1539
HTTP (*no módulo email.policy*), 1354
http_error_301() (*método urllib.request.HTTPRedirectHandler*), 1520
http_error_302() (*método urllib.request.HTTPRedirectHandler*), 1520
http_error_303() (*método urllib.request.HTTPRedirectHandler*), 1521
http_error_307() (*método urllib.request.HTTPRedirectHandler*), 1521
http_error_308() (*método urllib.request.HTTPRedirectHandler*), 1521
http_error_401() (*método urllib.request.HTTPBasicAuthHandler*), 1522
http_error_401() (*método urllib.request.HTTPDigestAuthHandler*), 1522
http_error_407() (*método urllib.request.ProxyBasicAuthHandler*), 1522
http_error_407() (*método urllib.request.ProxyDigestAuthHandler*), 1522
http_error_auth_reqed() (*método urllib.request.AbstractBasicAuthHandler*), 1522
http_error_auth_reqed() (*método urllib.request.AbstractDigestAuthHandler*), 1522
http_error_default() (*método urllib.request.BaseHandler*), 1519
http_open() (*método urllib.request.HTTPHandler*), 1522
HTTP_PORT (*no módulo http.client*), 1545
http_response() (*método urllib.request.HTTPErrorProcessor*), 1523
http_version (*atributo wsgiref.handlers.BaseHandler*), 1508
HTTPBasicAuthHandler (*classe em urllib.request*), 1515
http.client
    module, 1543
HTTPConnection (*classe em http.client*), 1543
http.cookiejar
    module, 1600
HTTPCookieProcessor (*classe em urllib.request*), 1514
http.cookies
    module, 1597
httpd, 1589
HTTPDefaultErrorHandler (*classe em urllib.request*), 1514
HTTPDigestAuthHandler (*classe em urllib.request*), 1516
HTTPError, 1538
HTTPErrorProcessor (*classe em urllib.request*), 1516
HTTPException, 1544
HTTPHandler (*classe em logging.handlers*), 849
HTTPHandler (*classe em urllib.request*), 1516
HTTPMessage (*classe em http.client*), 1550
HTTPMethod (*classe em http*), 1542
httponly (*atributo http.cookies.Morsel*), 1598
HTTPPasswordMgr (*classe em urllib.request*), 1515
HTTPPasswordMgrWithDefaultRealm (*classe em urllib.request*), 1515
HTTPPasswordMgrWithPriorAuth (*classe em urllib.request*), 1515
HTTPRedirectHandler (*classe em urllib.request*), 1514
HTTPResponse (*classe em http.client*), 1544
https_open() (*método urllib.request.HTTPSHandler*), 1522
HTTPS_PORT (*no módulo http.client*), 1545
https_response() (*método urllib.request.HTTPErrorProcessor*), 1523
HTTPSConnection (*classe em http.client*), 1543
http.server
    module, 1589
    security, 1596
HTTPServer (*classe em http.server*), 1590
http.server opção de linha de comando
    -b, 1595
    --bind, 1595
    --cgi, 1596
    -d, 1595
    --directory, 1595
    -p, 1596
    port, 1595
    --protocol, 1596
    --tls-cert, 1596
    --tls-key, 1596
    --tls-password-file, 1596
HTTPSHandler (*classe em urllib.request*), 1516
HTTPSServer (*classe em http.server*), 1590
HTTPStatus (*classe em http*), 1539
HV_GUID_BROADCAST (*no módulo socket*), 1251
HV_GUID_CHILDREN (*no módulo socket*), 1251
HV_GUID_LOOPBACK (*no módulo socket*), 1251
HV_GUID_PARENT (*no módulo socket*), 1251
HV_GUID_WILDCARD (*no módulo socket*), 1251
HV_GUID_ZERO (*no módulo socket*), 1251
HV_PROTOCOL_RAW (*no módulo socket*), 1251
HVSOCKET_ADDRESS_FLAG_PASSTHRU (*no módulo socket*), 1251
HVSOCKET_CONNECT_TIMEOUT (*no módulo socket*), 1251

HVSOCKET_CONNECT_TIMEOUT_MAX (*no módulo socket*), 1251
HVSOCKET_CONNECTED_SUSPEND (*no módulo socket*), 1251
hypot() (*no módulo math*), 367

I

-i
    ast opção de linha de comando, 2242
    compileall opção de linha de comando, 2262
    idle opção de linha de comando, 1708
    random opção de linha de comando, 417
I (*no módulo re*), 155
iadd() (*no módulo operator*), 469
iand() (*no módulo operator*), 469
iconcat() (*no módulo operator*), 469
id (*atributo concurrent.interpreters.Interpreter*), 1102
id (*atributo concurrent.interpreters.Queue*), 1103
id (*atributo ssl.SSLSession*), 1303
id()
    built-in function, 19
id() (*método unittest.TestCase*), 1856
idcok() (*método curses.window*), 994
ident (*atributo select.kevent*), 1311
ident (*atributo threading.Thread*), 1029
identchars (*atributo cmd.Cmd*), 1017
identify() (*método tkinter.ttk.Notebook*), 1689
identify() (*método tkinter.ttk.Treeview*), 1694
identify() (*método tkinter.ttk.Widget*), 1686
identify_column() (*método tkinter.ttk.Treeview*), 1694
identify_element() (*método tkinter.ttk.Treeview*), 1695
identify_region() (*método tkinter.ttk.Treeview*), 1694
identify_row() (*método tkinter.ttk.Treeview*), 1694
IDLE, 1701, **2353**
idle opção de linha de comando
    -, 1709
    -c, 1708
    -d, 1708
    -e, 1708
    -h, 1708
    -i, 1708
    -r, 1709
    -s, 1709
    -t, 1709
idle() (*método imaplib.IMAP4*), 1563
IDLE_PRIORITY_CLASS (*no módulo subprocess*), 1117
idlelib
    module, 1712
IDLESTARTUP, 1708, 1709
idlok() (*método curses.window*), 994
IEEE_CONTEXT_MAX_BITS (*no módulo decimal*), 393
IEEEContext() (*no módulo decimal*), 387
if
    instrução, 39
If (*classe em ast*), 2221
if_indextoname() (*no módulo socket*), 1258
if_nameindex() (*no módulo socket*), 1258
if_nametoindex() (*no módulo socket*), 1258
IfExp (*classe em ast*), 2215
ifloordiv() (*no módulo operator*), 469
iglob() (*no módulo glob*), 520
ignorableWhitespace() (*método xml.sax.handler.ContentHandler*), 1480
ignore
    error handler's name, 208
ignore (*atributo bdb.Breakpoint*), 1956
IGNORE (*no módulo tkinter.messagebox*), 1680
ignore (*pdb command*), 1970
ignore_environment (*atributo sys.flags*), 2028
ignore_errors() (*no módulo codecs*), 209
IGNORE_EXCEPTION_DETAIL (*no módulo doctest*), 1822
ignore_patterns() (*no módulo shutil*), 527
ignore_warnings() (*no módulo test.support.warnings_helper*), 1949
IGNORECASE (*no módulo re*), 155
--ignore-dir
    trace opção de linha de comando, 1990
--ignore-module
    trace opção de linha de comando, 1990
IISCGIHandler (*classe em wsgiref.handlers*), 1505
IllegalMonthError, 273
IllegalWeekdayError, 273
ilshift() (*no módulo operator*), 469
imag (*atributo numbers.Complex*), 357
imag (*atributo sys.hash_info*), 2035
imap() (*método multiprocessing.pool.Pool*), 1070
IMAP4
    protocolo, 1560
IMAP4 (*classe em imaplib*), 1560
IMAP4_SSL
    protocolo, 1560
IMAP4_SSL (*classe em imaplib*), 1561
IMAP4_stream
    protocolo, 1560
IMAP4_stream (*classe em imaplib*), 1561
IMAP4.abort, 1560
IMAP4.error, 1560
IMAP4.readonly, 1560
imap_unordered() (*método multiprocessing.pool.Pool*), 1070
imaplib
    module, 1560
imatmul() (*no módulo operator*), 469
imghdr
    module, 2339
immedok() (*método curses.window*), 994
imod() (*no módulo operator*), 469
imortal, **2353**
imp
    module, 2339
impl_detail() (*no módulo test.support*), 1939

implementation (*no módulo sys*), 2035
Import (*classe em ast*), 2221
import_fresh_module() (*no módulo test.support.import_helper*), 1947
IMPORT_FROM (*opcode*), 2280
import_module() (*no módulo importlib*), 2169
import_module() (*no módulo test.support.import_helper*), 1948
IMPORT_NAME (*opcode*), 2280
importação, **2354**
    instrução, 34, 2152
importador, **2354**
ImportError, 117
ImportFrom (*classe em ast*), 2221
importlib
    module, 2168
importlib.abc
    module, 2170
importlib.machinery
    module, 2177
importlib.metadata
    module, 2193
importlib.resources
    module, 2188
importlib.resources.abc
    module, 2191
importlib.util
    module, 2183
ImportWarning, 125
imprimíveis entre aspas
    codificação, 1431
ImproperConnectionState, 1544
imul() (*no módulo operator*), 469
imutável, **2353**
    sequência tipos, 51
in
    operador, 40, 49
In (*classe em ast*), 2214
in_dll() (*método ctypes._CData*), 896
in_table_a1() (*no módulo stringprep*), 190
in_table_b1() (*no módulo stringprep*), 190
in_table_c3() (*no módulo stringprep*), 190
in_table_c4() (*no módulo stringprep*), 190
in_table_c5() (*no módulo stringprep*), 190
in_table_c6() (*no módulo stringprep*), 190
in_table_c7() (*no módulo stringprep*), 191
in_table_c8() (*no módulo stringprep*), 191
in_table_c9() (*no módulo stringprep*), 191
in_table_c11() (*no módulo stringprep*), 190
in_table_c11_c12() (*no módulo stringprep*), 190
in_table_c12() (*no módulo stringprep*), 190
in_table_c21() (*no módulo stringprep*), 190
in_table_c21_c22() (*no módulo stringprep*), 190
in_table_c22() (*no módulo stringprep*), 190
in_table_d1() (*no módulo stringprep*), 191
in_table_d2() (*no módulo stringprep*), 191
in_transaction (*atributo sqlite3.Connection*), 582
inch() (*método curses.window*), 994
include() (*no módulo xml.etree.ElementInclude*), 1451
--include-attributes
    ast opção de linha de comando, 2242
inclusive (*atributo tracemalloc.DomainFilter*), 1997
inclusive (*atributo tracemalloc.Filter*), 1998
Incomplete, 1431
IncompleteRead, 1544
IncompleteReadError, 1182
increment_lineno() (*no módulo ast*), 2239
incrementaldecoder (*atributo codecs.CodecInfo*), 205
IncrementalDecoder (*classe em codecs*), 211
incrementalencoder (*atributo codecs.CodecInfo*), 205
IncrementalEncoder (*classe em codecs*), 210
IncrementalNewlineDecoder (*classe em io*), 793
IncrementalParser (*classe em xml.sax.xmlreader*), 1482
--indent
    ast opção de linha de comando, 2242
    json opção de linha de comando, 1401
indent (*atributo doctest.Example*), 1830
indent (*atributo reprlib.Repr*), 334
INDENT (*no módulo token*), 2248
indent() (*no módulo textwrap*), 184
indent() (*no módulo xml.etree.ElementTree*), 1448
IndentationError, 121
--indentlevel
    pickletools opção de linha de comando, 2289
index (*atributo inspect.FrameInfo*), 2137
index (*atributo inspect.Traceback*), 2137
index() (*método array.array*), 310
index() (*método bytearray*), 76
index() (*método bytes*), 76
index() (*método collections.abc.ByteString*), 297
index() (*método collections.deque*), 282
index() (*método memoryview*), 93
index() (*método multiprocessing.shared_memory.ShareableList*), 1089
index() (*método sequence*), 51
index() (*método str*), 60
index() (*método tkinter.ttk.Notebook*), 1689
index() (*método tkinter.ttk.Treeview*), 1695
index() (*no módulo operator*), 464
IndexError, 118
indexOf() (*no módulo operator*), 466
IndexSizeErr, 1466
INDIRECT (*atributo inspect.BufferFlags*), 2142
inet_aton() (*no módulo socket*), 1256
inet_ntoa() (*no módulo socket*), 1256
inet_ntop() (*no módulo socket*), 1257
inet_pton() (*no módulo socket*), 1257
Inexact (*classe em decimal*), 394
inf (*atributo sys.hash_info*), 2035
inf (*no módulo cmath*), 373
inf (*no módulo math*), 369
infile

    json opção de linha de comando, 1400
infile (atributo shlex.shlex), 2310
Infinity, 17
infj (no módulo cmath), 373
--info
    zipapp opção de linha de comando, 2016
info (atributo pathlib.Path), 488
INFO (no módulo logging), 814
INFO (no módulo tkinter.messagebox), 1681
info() (método dis.Bytecode), 2267
info() (método gettext.NullTranslations), 1646
info() (método http.client.HTTPResponse), 1549
info() (método logging.Logger), 812
info() (método urllib.response.addinfourl), 1528
info() (no módulo logging), 822
infolist() (método zipfile.ZipFile), 632
informações de configuração, 2054
.ini
    arquivo, 667
init() (no módulo mimetypes), 1421
init_color() (no módulo curses), 986
init_pair() (no módulo curses), 987
inited (no módulo mimetypes), 1422
initgroups() (no módulo os), 717
initial_indent (atributo textwrap.TextWrapper), 186
initscr() (no módulo curses), 987
InitVar (classe em dataclasses), 2079
inode() (método os.DirEntry), 746
input()
    built-in function, 19
input() (no módulo fileinput), 981
input_charset (atributo email.charset.Charset), 1385
input_codec (atributo email.charset.Charset), 1385
InputSource (classe em xml.sax.xmlreader), 1483
InputStream (classe em wsgiref.types), 1509
insch() (método curses.window), 994
insdelln() (método curses.window), 994
insert() (método array.array), 310
insert() (método collections.deque), 282
insert() (método sequence), 52
insert() (método tkinter.ttk.Notebook), 1689
insert() (método tkinter.ttk.Treeview), 1695
insert() (método xml.etree.ElementTree.Element), 1452
insert_text() (no módulo readline), 192
insertBefore() (método xml.dom.Node), 1462
insertln() (método curses.window), 994
insnstr() (método curses.window), 995
insort() (no módulo bisect), 306
insort_left() (no módulo bisect), 306
insort_right() (no módulo bisect), 306
inspect
    module, 2123
inspect (atributo sys.flags), 2028
inspect opção de linha de comando
    --details, 2142
InspectLoader (classe em importlib.abc), 2173
insstr() (método curses.window), 995
install() (método gettext.NullTranslations), 1646
install() (no módulo gettext), 1645
install_opener() (no módulo urllib.request), 1512
install_scripts() (método venv.EnvBuilder), 2010
installHandler() (no módulo unittest), 1868
instate() (método tkinter.ttk.Widget), 1686
instr() (método curses.window), 995
instream (atributo shlex.shlex), 2310
instrução, **2361**
    assert, 117
    del, 51, 98
    except, 115
    if, 39
    importação, 34, 2152
    raise, 115
    try, 115
    while, 39
Instruction (classe em dis), 2270
INSTRUCTION (monitoring event), 2050
Instruction.arg (no módulo dis), 2270
Instruction.argrepr (no módulo dis), 2271
Instruction.argval (no módulo dis), 2271
Instruction.baseopcode (no módulo dis), 2270
Instruction.baseopname (no módulo dis), 2270
Instruction.cache_info (no módulo dis), 2271
Instruction.cache_offset (no módulo dis), 2271
Instruction.end_offset (no módulo dis), 2271
Instruction.is_jump_target (no módulo dis), 2271
Instruction.jump_target (no módulo dis), 2271
Instruction.line_number (no módulo dis), 2271
Instruction.offset (no módulo dis), 2271
Instruction.oparg (no módulo dis), 2270
Instruction.opcode (no módulo dis), 2270
Instruction.opname (no módulo dis), 2270
Instruction.positions (no módulo dis), 2271
Instruction.start_offset (no módulo dis), 2271
Instruction.starts_line (no módulo dis), 2271
int
    função embutida, 41
int (atributo uuid.UUID), 1576
int (classe interna), 20
Int2AP() (no módulo imaplib), 1561
int_info (no módulo sys), 2036
int_max_str_digits (atributo sys.flags), 2028
--integer
    random opção de linha de comando, 417
Integral (classe em numbers), 358
Integrated Development Environment, 1701
IntegrityError, 587
inteiro
    literais, 41
    objeto, 40
    tipos, operações em, 42
IntEnum (classe em enum), 343
interact (pdb command), 1973
interact() (método code.InteractiveConsole), 2157
interact() (no módulo code), 2155

interactive (*atributo sys.flags*), 2028
Interactive (*classe em ast*), 2208
InteractiveConsole (*classe em code*), 2155
InteractiveInterpreter (*classe em code*), 2155
interativo, **2354**
Interface Gráfica do Usuário, 1661
InterfaceError, 587
intern() (*no módulo sys*), 2037
internal_attr (*atributo zipfile.ZipInfo*), 639
Internaldate2tuple() (*no módulo imaplib*), 1561
InternalError, 587
internalSubset (*atributo xml.dom.DocumentType*), 1463
Internet, 1497
INTERNET_TIMEOUT (*no módulo test.support*), 1934
interpolação
    bytearray (%), 85
    bytes (%), 85
interpolação, string (%), 69
Interpolation (*classe em ast*), 2210
Interpolation (*classe em string.templatelib*), 145
InterpolationDepthError, 685
InterpolationError, 685
InterpolationMissingOptionError, 685
interpolations (*atributo string.templatelib.Template*), 143
InterpolationSyntaxError, 685
interpretado, **2354**
interpretador, prompts, 2041
Interpreter (*classe em concurrent.interpreters*), 1102
interpreter_requires_environment() (*no módulo test.support.script_helper*), 1943
InterpreterError, 1103
InterpreterNotFoundError, 1103
InterpreterPoolExecutor (*classe em concurrent.futures*), 1094
interrupt() (*método multiprocessing.Process*), 1048
interrupt() (*método sqlite3.Connection*), 576
interrupt_main() (*no módulo _thread*), 1133
InterruptedError, 123
intersection() (*método frozenset*), 96
intersection_update() (*método frozenset*), 97
IntFlag (*classe em enum*), 346
intro (*atributo cmd.Cmd*), 1017
InuseAttributeErr, 1466
inv() (*no módulo operator*), 464
inv_cdf() (*método statistics.NormalDist*), 430
InvalidAccessErr, 1466
invalidate_caches() (*método de classe importlib.machinery.PathFinder*), 2178
invalidate_caches() (*método importlib.abc.MetaPathFinder*), 2171
invalidate_caches() (*método importlib.abc.PathEntryFinder*), 2171
invalidate_caches() (*método importlib.machinery.FileFinder*), 2179
invalidate_caches() (*método zipimport.zipimporter*), 2161
invalidate_caches() (*no módulo importlib*), 2169
invalidate_caches() (*no módulo platform*), 854
--invalidation-mode
    compileall opção de linha de comando, 2262
InvalidBase64CharactersDefect, 1356
InvalidBase64LengthDefect, 1356
InvalidBase64PaddingDefect, 1356
InvalidCharacterErr, 1466
InvalidDateDefect, 1356
InvalidFileException, 691
InvalidModificationErr, 1466
InvalidOperation (*classe em decimal*), 394
InvalidStateErr, 1466
InvalidStateError, 1099, 1182
InvalidTZPathWarning, 267
InvalidURL, 1544
InvalidWriteError, 686
Invert (*classe em ast*), 2213
invert() (*no módulo operator*), 464
io
    module, 780
IO (*classe em typing*), 1794
IO_REPARSE_TAG_APPEXECLINK (*no módulo stat*), 512
IO_REPARSE_TAG_MOUNT_POINT (*no módulo stat*), 512
IO_REPARSE_TAG_SYMLINK (*no módulo stat*), 512
IOBase (*classe em io*), 784
ioctl() (*método socket.socket*), 1261
ioctl() (*no módulo fcntl*), 2320
IOCTL_VM_SOCKETS_GET_LOCAL_CID (*no módulo socket*), 1249
IOError, 123
ior() (*no módulo operator*), 470
ios_ver() (*no módulo platform*), 855
io.StringIO
    objeto, 57
ip (*atributo ipaddress.IPv4Interface*), 1634
ip (*atributo ipaddress.IPv6Interface*), 1635
ip_address() (*no módulo ipaddress*), 1623
ip_interface() (*no módulo ipaddress*), 1623
ip_network() (*no módulo ipaddress*), 1623
ipaddress
    module, 1623
ipow() (*no módulo operator*), 470
ipv4_mapped (*atributo ipaddress.IPv6Address*), 1627
IPv4Address (*classe em ipaddress*), 1623
IPv4Interface (*classe em ipaddress*), 1634
IPv4Network (*classe em ipaddress*), 1629
IPV6_ENABLED (*no módulo test.support.socket_helper*), 1942
ipv6_mapped (*atributo ipaddress.IPv4Address*), 1626
IPv6Address (*classe em ipaddress*), 1626
IPv6Interface (*classe em ipaddress*), 1635
IPv6Network (*classe em ipaddress*), 1632
irshift() (*no módulo operator*), 470
is

operador, 40
Is (*classe em ast*), 2214
is not
operador, 40
is_() (*no módulo operator*), 464
is_absolute() (*método pathlib.PurePath*), 479
is_active() (*método graphlib.TopologicalSorter*), 353
is_alive() (*método multiprocessing.Process*), 1047
is_alive() (*método threading.Thread*), 1029
is_android (*no módulo test.support*), 1933
is_annotated() (*método symtable.Symbol*), 2246
is_anonymous (*atributo ctypes.CField*), 902
is_apple (*no módulo test.support*), 1933
is_apple_mobile (*no módulo test.support*), 1933
is_assigned() (*método symtable.Symbol*), 2246
is_async (*atributo pyclbr.Function*), 2258
is_attachment() (*método email.message.EmailMessage*), 1339
is_authenticated() (*método urllib.request.HTTPPasswordMgrWithPriorAuth*), 1521
is_bitfield (*atributo ctypes.CField*), 902
is_block_device() (*método pathlib.Path*), 488
is_blocked() (*método http.cookiejar.DefaultCookiePolicy*), 1606
is_canonical() (*método decimal.Context*), 390
is_canonical() (*método decimal.Decimal*), 382
is_char_device() (*método pathlib.Path*), 488
IS_CHARACTER_JUNK() (*no módulo difflib*), 175
is_check_supported() (*no módulo lzma*), 627
is_closed() (*método asyncio.loop*), 1186
is_closing() (*método asyncio.BaseTransport*), 1215
is_closing() (*método asyncio.StreamWriter*), 1166
is_comp_cell() (*método symtable.Symbol*), 2246
is_comp_iter() (*método symtable.Symbol*), 2246
is_dataclass() (*no módulo dataclasses*), 2081
is_declared_global() (*método symtable.Symbol*), 2246
is_dir() (*método importlib.abc.Traversable*), 2176
is_dir() (*método importlib.resources.abc.Traversable*), 2192
is_dir() (*método os.DirEntry*), 746
is_dir() (*método pathlib.Path*), 487
is_dir() (*método pathlib.types.PathInfo*), 498
is_dir() (*método zipfile.Path*), 636
is_dir() (*método zipfile.ZipInfo*), 638
is_emscripten (*no módulo test.support*), 1933
is_enabled() (*no módulo faulthandler*), 1962
is_expired() (*método http.cookiejar.Cookie*), 1608
is_fifo() (*método pathlib.Path*), 488
is_file() (*método importlib.abc.Traversable*), 2176
is_file() (*método importlib.resources.abc.Traversable*), 2192
is_file() (*método os.DirEntry*), 746
is_file() (*método pathlib.Path*), 487
is_file() (*método pathlib.types.PathInfo*), 499
is_file() (*método zipfile.Path*), 636
is_finalized() (*no módulo gc*), 2122
is_finalizing() (*no módulo sys*), 2037
is_finite() (*método decimal.Context*), 390
is_finite() (*método decimal.Decimal*), 382
is_free() (*método symtable.Symbol*), 2246
is_free_class() (*método symtable.Symbol*), 2246
is_global (*atributo ipaddress.IPv4Address*), 1625
is_global (*atributo ipaddress.IPv6Address*), 1627
is_global() (*método symtable.Symbol*), 2246
is_hop_by_hop() (*no módulo wsgiref.util*), 1501
is_imported() (*método symtable.Symbol*), 2245
is_infinite() (*método decimal.Context*), 390
is_infinite() (*método decimal.Decimal*), 382
is_integer() (*método float*), 45
is_integer() (*método fractions.Fraction*), 405
is_integer() (*método int*), 45
is_junction() (*método os.DirEntry*), 747
is_junction() (*método pathlib.Path*), 487
is_jython (*no módulo test.support*), 1933
IS_LINE_JUNK() (*no módulo difflib*), 175
is_linetouched() (*método curses.window*), 995
is_link_local (*atributo ipaddress.IPv4Address*), 1626
is_link_local (*atributo ipaddress.IPv4Network*), 1630
is_link_local (*atributo ipaddress.IPv6Address*), 1627
is_link_local (*atributo ipaddress.IPv6Network*), 1633
is_local() (*método symtable.Symbol*), 2246
is_loopback (*atributo ipaddress.IPv4Address*), 1626
is_loopback (*atributo ipaddress.IPv4Network*), 1630
is_loopback (*atributo ipaddress.IPv6Address*), 1627
is_loopback (*atributo ipaddress.IPv6Network*), 1632
is_mount() (*método pathlib.Path*), 487
is_multicast (*atributo ipaddress.IPv4Address*), 1624
is_multicast (*atributo ipaddress.IPv4Network*), 1629
is_multicast (*atributo ipaddress.IPv6Address*), 1627
is_multicast (*atributo ipaddress.IPv6Network*), 1632
is_multipart() (*método email.message.EmailMessage*), 1335
is_multipart() (*método email.message.Message*), 1372
is_namespace() (*método symtable.Symbol*), 2246
is_nan() (*método decimal.Context*), 390
is_nan() (*método decimal.Decimal*), 382
is_nested() (*método symtable.SymbolTable*), 2244
is_none() (*no módulo operator*), 464
is_nonlocal() (*método symtable.Symbol*), 2246
is_normal() (*método decimal.Context*), 390
is_normal() (*método decimal.Decimal*), 382
is_normalized() (*no módulo unicodedata*), 189
is_not() (*no módulo operator*), 464
is_not_allowed() (*método http.cookiejar.DefaultCookiePolicy*), 1606
is_not_none() (*no módulo operator*), 464
IS_OP (*opcode*), 2280
is_optimized() (*método symtable.SymbolTable*), 2244

is_package() (*método importlib.abc.InspectLoader*), 2173
is_package() (*método importlib.abc.SourceLoader*), 2175
is_package() (*método importlib.machinery.ExtensionFileLoader*), 2181
is_package() (*método importlib.machinery.SourceFileLoader*), 2179
is_package() (*método importlib.machinery.SourcelessFileLoader*), 2180
is_package() (*método zipimport.zipimporter*), 2160
is_parameter() (*método symtable.Symbol*), 2245
is_private (*atributo ipaddress.IPv4Address*), 1625
is_private (*atributo ipaddress.IPv4Network*), 1629
is_private (*atributo ipaddress.IPv6Address*), 1627
is_private (*atributo ipaddress.IPv6Network*), 1632
is_protocol() (*no módulo typing*), 1801
is_python_build() (*no módulo sysconfig*), 2060
is_qnan() (*método decimal.Context*), 390
is_qnan() (*método decimal.Decimal*), 382
is_reading() (*método asyncio.ReadTransport*), 1216
is_referenced() (*método symtable.Symbol*), 2245
is_relative_to() (*método pathlib.PurePath*), 479
is_reserved (*atributo ipaddress.IPv4Address*), 1625
is_reserved (*atributo ipaddress.IPv4Network*), 1629
is_reserved (*atributo ipaddress.IPv6Address*), 1627
is_reserved (*atributo ipaddress.IPv6Network*), 1632
is_reserved() (*método pathlib.PurePath*), 480
is_resource() (*método importlib.abc.ResourceReader*), 2176
is_resource() (*método importlib.resources.abc.ResourceReader*), 2192
is_resource() (*no módulo importlib.resources*), 2191
is_resource_enabled() (*no módulo test.support*), 1936
is_running() (*método asyncio.loop*), 1186
is_running() (*método concurrent.interpreters.Interpreter*), 1102
is_safe (*atributo uuid.UUID*), 1576
is_serving() (*método asyncio.Server*), 1206
is_set() (*método asyncio.Event*), 1171
is_set() (*método threading.Event*), 1036
is_signed() (*método decimal.Context*), 390
is_signed() (*método decimal.Decimal*), 382
is_site_local (*atributo ipaddress.IPv6Address*), 1627
is_site_local (*atributo ipaddress.IPv6Network*), 1633
is_skipped_line() (*método bdb.Bdb*), 1958
is_snan() (*método decimal.Context*), 391
is_snan() (*método decimal.Decimal*), 382
is_socket() (*método pathlib.Path*), 487
is_stack_trampoline_active() (*no módulo sys*), 2045
is_subnormal() (*método decimal.Context*), 391
is_subnormal() (*método decimal.Decimal*), 383
is_symlink() (*método os.DirEntry*), 746
is_symlink() (*método pathlib.Path*), 487
is_symlink() (*método pathlib.types.PathInfo*), 499
is_symlink() (*método zipfile.Path*), 636
is_tarfile() (*no módulo tarfile*), 643
is_term_resized() (*no módulo curses*), 987
is_tracing() (*no módulo tracemalloc*), 1996
is_tracked() (*no módulo gc*), 2121
is_type_parameter() (*método symtable.Symbol*), 2245
is_typeddict() (*no módulo typing*), 1801
is_unspecified (*atributo ipaddress.IPv4Address*), 1625
is_unspecified (*atributo ipaddress.IPv4Network*), 1629
is_unspecified (*atributo ipaddress.IPv6Address*), 1627
is_unspecified (*atributo ipaddress.IPv6Network*), 1632
is_valid() (*método string.Template*), 140
is_wasi (*no módulo test.support*), 1933
is_wintouched() (*método curses.window*), 995
is_zero() (*método decimal.Context*), 391
is_zero() (*método decimal.Decimal*), 383
is_zipfile() (*no módulo zipfile*), 630
isabs() (*no módulo os.path*), 501
isabstract() (*no módulo inspect*), 2128
IsADirectoryError, 124
isalnum() (*método bytearray*), 81
isalnum() (*método bytes*), 81
isalnum() (*método str*), 60
isalnum() (*no módulo curses.ascii*), 1013
isalpha() (*método bytearray*), 81
isalpha() (*método bytes*), 81
isalpha() (*método str*), 60
isalpha() (*no módulo curses.ascii*), 1013
isascii() (*método bytearray*), 81
isascii() (*método bytes*), 81
isascii() (*método str*), 60
isascii() (*no módulo curses.ascii*), 1013
isasyncgen() (*no módulo inspect*), 2127
isasyncgenfunction() (*no módulo inspect*), 2127
isatty() (*método io.IOBase*), 785
isatty() (*no módulo os*), 725
isawaitable() (*no módulo inspect*), 2127
isblank() (*no módulo curses.ascii*), 1013
isblk() (*método tarfile.TarInfo*), 651
isbuiltin() (*no módulo inspect*), 2128
ischr() (*método tarfile.TarInfo*), 651
isclass() (*no módulo inspect*), 2126
isclose() (*no módulo cmath*), 373
isclose() (*no módulo math*), 364
iscntrl() (*no módulo curses.ascii*), 1013
iscode() (*no módulo inspect*), 2128
iscoroutine() (*no módulo asyncio*), 1157
iscoroutine() (*no módulo inspect*), 2127
iscoroutinefunction() (*no módulo inspect*), 2126
isctrl() (*no módulo curses.ascii*), 1013
isDaemon() (*método threading.Thread*), 1030
isdatadescriptor() (*no módulo inspect*), 2128

isdecimal() (*método str*), 60
isdev() (*método tarfile.TarInfo*), 651
isdevdrive() (*no módulo os.path*), 502
isdigit() (*método bytearray*), 82
isdigit() (*método bytes*), 82
isdigit() (*método str*), 61
isdigit() (*no módulo curses.ascii*), 1013
isdir() (*método tarfile.TarInfo*), 651
isdir() (*no módulo os.path*), 502
isdisjoint() (*método frozenset*), 96
isdown() (*no módulo turtle*), 1728
iselement() (*no módulo xml.etree.ElementTree*), 1448
isenabled() (*no módulo gc*), 2119
isEnabledFor() (*método logging.Logger*), 811
isendwin() (*no módulo curses*), 987
ISEOF() (*no módulo token*), 2247
isfifo() (*método tarfile.TarInfo*), 651
isfile() (*método tarfile.TarInfo*), 650
isfile() (*no módulo os.path*), 501
isfinite() (*no módulo cmath*), 373
isfinite() (*no módulo math*), 364
isfirstline() (*no módulo fileinput*), 982
isframe() (*no módulo inspect*), 2127
isfunction() (*no módulo inspect*), 2126
isfuture() (*no módulo asyncio*), 1210
isgenerator() (*no módulo inspect*), 2126
isgeneratorfunction() (*no módulo inspect*), 2126
isgetsetdescriptor() (*no módulo inspect*), 2128
isgraph() (*no módulo curses.ascii*), 1013
isidentifier() (*método str*), 61
isinf() (*no módulo cmath*), 373
isinf() (*no módulo math*), 364
isinstance()
    built-in function, 21
isjunction() (*no módulo os.path*), 502
iskeyword() (*no módulo keyword*), 2252
isleap() (*no módulo calendar*), 271
islice() (*no módulo itertools*), 442
islink() (*no módulo os.path*), 502
islnk() (*método tarfile.TarInfo*), 651
islower() (*método bytearray*), 82
islower() (*método bytes*), 82
islower() (*método str*), 61
islower() (*no módulo curses.ascii*), 1013
ismemberdescriptor() (*no módulo inspect*), 2128
ismeta() (*no módulo curses.ascii*), 1013
ismethod() (*no módulo inspect*), 2126
ismethoddescriptor() (*no módulo inspect*), 2128
ismethodwrapper() (*no módulo inspect*), 2128
ismodule() (*no módulo inspect*), 2126
ismount() (*no módulo os.path*), 502
isnan() (*no módulo cmath*), 373
isnan() (*no módulo math*), 364
ISNONTERMINAL() (*no módulo token*), 2247
IsNot (*classe em ast*), 2214
isnumeric() (*método str*), 61
isocalendar() (*método datetime.date*), 232
isocalendar() (*método datetime.datetime*), 242
isoformat() (*método datetime.date*), 233
isoformat() (*método datetime.datetime*), 243
isoformat() (*método datetime.time*), 248
isolado cíclico, **2349**
isolated (*atributo sys.flags*), 2028
IsolatedAsyncioTestCase (*classe em unittest*), 1857
isolation_level (*atributo sqlite3.Connection*), 582
isoweekday() (*método datetime.date*), 232
isoweekday() (*método datetime.datetime*), 242
ispackage() (*no módulo inspect*), 2126
isprint() (*no módulo curses.ascii*), 1013
isprintable() (*método str*), 61
ispunct() (*no módulo curses.ascii*), 1013
isqrt() (*no módulo math*), 362
isreadable() (*método pprint.PrettyPrinter*), 329
isreadable() (*no módulo pprint*), 328
isrecursive() (*método pprint.PrettyPrinter*), 330
isrecursive() (*no módulo pprint*), 328
isreg() (*método tarfile.TarInfo*), 651
isreserved() (*no módulo os.path*), 502
isReservedKey() (*método http.cookies.Morsel*), 1599
isroutine() (*no módulo inspect*), 2128
isSameNode() (*método xml.dom.Node*), 1462
issoftkeyword() (*no módulo keyword*), 2252
isspace() (*método bytearray*), 82
isspace() (*método bytes*), 82
isspace() (*método str*), 61
isspace() (*no módulo curses.ascii*), 1013
isstdin() (*no módulo fileinput*), 982
issubclass()
    built-in function, 21
issubset() (*método frozenset*), 96
issuperset() (*método frozenset*), 96
issym() (*método tarfile.TarInfo*), 651
ISTERMINAL() (*no módulo token*), 2247
istitle() (*método bytearray*), 82
istitle() (*método bytes*), 82
istitle() (*método str*), 61
istraceback() (*no módulo inspect*), 2127
isub() (*no módulo operator*), 470
isupper() (*método bytearray*), 82
isupper() (*método bytes*), 82
isupper() (*método str*), 62
isupper() (*no módulo curses.ascii*), 1013
isvisible() (*no módulo turtle*), 1732
isxdigit() (*no módulo curses.ascii*), 1013
ITALIC (*no módulo tkinter.font*), 1675
item() (*método tkinter.ttk.Treeview*), 1695
item() (*método xml.dom.NamedNodeMap*), 1465
item() (*método xml.dom.NodeList*), 1462
itemgetter() (*no módulo operator*), 467
items() (*método configparser.ConfigParser*), 683
items() (*método contextvars.Context*), 1132
items() (*método dict*), 100
items() (*método email.message.EmailMessage*), 1336
items() (*método email.message.Message*), 1374
items() (*método mailbox.Mailbox*), 1403

items() (*método types.MappingProxyType*), 324
items() (*método xml.etree.ElementTree.Element*), 1452
itemsize (*atributo array.array*), 309
itemsize (*atributo memoryview*), 94
ItemsView (*classe em collections.abc*), 298
ItemsView (*classe em typing*), 1806
iter()
    built-in function, 21
iter() (*método xml.etree.ElementTree.Element*), 1452
iter() (*método xml.etree.ElementTree.ElementTree*), 1454
iter_attachments() (*método email.message.EmailMessage*), 1340
iter_child_nodes() (*no módulo ast*), 2239
iter_fields() (*no módulo ast*), 2239
iter_importers() (*no módulo pkgutil*), 2162
iter_modules() (*no módulo pkgutil*), 2162
iter_parts() (*método email.message.EmailMessage*), 1340
iter_unpack() (*método struct.Struct*), 204
iter_unpack() (*no módulo struct*), 198
Iterable (*classe em collections.abc*), 297
Iterable (*classe em typing*), 1808
iterador, **2354**
iterador assíncrono, **2346**
iterador gerador, **2352**
iterador gerador assíncrono, **2346**
Iterator (*classe em collections.abc*), 297
Iterator (*classe em typing*), 1808
iterável, **2354**
iterável assíncrono, **2346**
iterdecode() (*no módulo codecs*), 207
iterdir() (*método importlib.abc.Traversable*), 2176
iterdir() (*método importlib.resources.abc.Traversable*), 2192
iterdir() (*método pathlib.Path*), 490
iterdir() (*método zipfile.Path*), 636
iterdump() (*método sqlite3.Connection*), 579
iterencode() (*método json.JSONEncoder*), 1398
iterencode() (*no módulo codecs*), 207
iterfind() (*método xml.etree.ElementTree.Element*), 1452
iterfind() (*método xml.etree.ElementTree.ElementTree*), 1454
iteritems() (*método mailbox.Mailbox*), 1403
iterkeys() (*método mailbox.Mailbox*), 1402
itermonthdates() (*método calendar.Calendar*), 268
itermonthdays() (*método calendar.Calendar*), 268
itermonthdays2() (*método calendar.Calendar*), 268
itermonthdays3() (*método calendar.Calendar*), 268
itermonthdays4() (*método calendar.Calendar*), 268
iterparse() (*no módulo xml.etree.ElementTree*), 1448
itertext() (*método xml.etree.ElementTree.Element*), 1452
itertools
    module, 435
itervalues() (*método mailbox.Mailbox*), 1403
iterweekdays() (*método calendar.Calendar*), 268
ITIMER_PROF (*no módulo signal*), 1319
ITIMER_REAL (*no módulo signal*), 1319
ITIMER_VIRTUAL (*no módulo signal*), 1319
ItimerError, 1320
itruediv() (*no módulo operator*), 470
ixor() (*no módulo operator*), 470

J

-j
    compileall opção de linha de comando, 2262
JANUARY (*no módulo calendar*), 273
java_ver() (*no módulo platform*), 855
job_size (*atributo compression.zstd.CompressionParameter*), 608
join() (*método asyncio.Queue*), 1180
join() (*método bytearray*), 76
join() (*método bytes*), 76
join() (*método multiprocessing.JoinableQueue*), 1052
join() (*método multiprocessing.pool.Pool*), 1070
join() (*método multiprocessing.Process*), 1047
join() (*método queue.Queue*), 1127
join() (*método str*), 62
join() (*método threading.Thread*), 1028
join() (*no módulo os.path*), 503
join() (*no módulo shlex*), 2307
join_thread() (*método multiprocessing.Queue*), 1051
join_thread() (*no módulo test.support.threading_helper*), 1945
JoinableQueue (*classe em multiprocessing*), 1052
JoinedStr (*classe em ast*), 2209
joinpath() (*método importlib.abc.Traversable*), 2176
joinpath() (*método importlib.resources.abc.Traversable*), 2192
joinpath() (*método pathlib.PurePath*), 480
joinpath() (*método zipfile.Path*), 637
js_output() (*método http.cookies.BaseCookie*), 1598
js_output() (*método http.cookies.Morsel*), 1599
json
    module, 1391
json opção de linha de comando
    --compact, 1401
    -h, 1401
    --help, 1401
    --indent, 1401
    infile, 1400
    --json-lines, 1401
    --no-ensure-ascii, 1401
    --no-indent, 1401
    outfile, 1401
    --sort-keys, 1401
    --tab, 1401
JSONDecodeError, 1398
JSONDecoder (*classe em json*), 1396
JSONEncoder (*classe em json*), 1397
--json-lines
    json opção de linha de comando, 1401
json.tool

module, 1400
JULY (*no módulo calendar*), 273
JUMP (*monitoring event*), 2050
JUMP (*opcode*), 2287
jump (*pdb command*), 1971
JUMP_BACKWARD (*opcode*), 2280
JUMP_BACKWARD_NO_INTERRUPT (*opcode*), 2280
JUMP_FORWARD (*opcode*), 2280
JUMP_IF_FALSE (*opcode*), 2287
JUMP_IF_TRUE (*opcode*), 2287
JUMP_NO_INTERRUPT (*opcode*), 2287
JUNE (*no módulo calendar*), 273

K

-k
unittest opção de linha de comando, 1840
kbhit() (*no módulo msvcrt*), 2292
kde() (*no módulo statistics*), 421
kde_random() (*no módulo statistics*), 422
KEEP (*atributo enum.FlagBoundary*), 349
kevent() (*no módulo select*), 1307
key (*atributo http.cookies.Morsel*), 1599
key (*atributo zoneinfo.ZoneInfo*), 265
KEY_A1 (*no módulo curses*), 1001
KEY_A3 (*no módulo curses*), 1002
KEY_ALL_ACCESS (*no módulo winreg*), 2300
KEY_B2 (*no módulo curses*), 1002
KEY_BACKSPACE (*no módulo curses*), 1000
KEY_BEG (*no módulo curses*), 1002
KEY_BREAK (*no módulo curses*), 1000
KEY_BTAB (*no módulo curses*), 1002
KEY_C1 (*no módulo curses*), 1002
KEY_C3 (*no módulo curses*), 1002
KEY_CANCEL (*no módulo curses*), 1002
KEY_CATAB (*no módulo curses*), 1001
KEY_CLEAR (*no módulo curses*), 1001
KEY_CLOSE (*no módulo curses*), 1002
KEY_COMMAND (*no módulo curses*), 1002
KEY_COPY (*no módulo curses*), 1002
KEY_CREATE (*no módulo curses*), 1002
KEY_CREATE_LINK (*no módulo winreg*), 2301
KEY_CREATE_SUB_KEY (*no módulo winreg*), 2300
KEY_CTAB (*no módulo curses*), 1001
KEY_DC (*no módulo curses*), 1000
KEY_DL (*no módulo curses*), 1000
KEY_DOWN (*no módulo curses*), 1000
KEY_EIC (*no módulo curses*), 1001
KEY_END (*no módulo curses*), 1002
KEY_ENTER (*no módulo curses*), 1001
KEY_ENUMERATE_SUB_KEYS (*no módulo winreg*), 2300
KEY_EOL (*no módulo curses*), 1001
KEY_EOS (*no módulo curses*), 1001
KEY_EXECUTE (*no módulo winreg*), 2300
KEY_EXIT (*no módulo curses*), 1002
KEY_F0 (*no módulo curses*), 1000
KEY_FIND (*no módulo curses*), 1002
KEY_Fn (*no módulo curses*), 1000
KEY_HELP (*no módulo curses*), 1002
KEY_HOME (*no módulo curses*), 1000
KEY_IC (*no módulo curses*), 1001
KEY_IL (*no módulo curses*), 1000
KEY_LEFT (*no módulo curses*), 1000
KEY_LL (*no módulo curses*), 1001
KEY_MARK (*no módulo curses*), 1002
KEY_MAX (*no módulo curses*), 1005
KEY_MESSAGE (*no módulo curses*), 1002
KEY_MIN (*no módulo curses*), 1000
KEY_MOUSE (*no módulo curses*), 1005
KEY_MOVE (*no módulo curses*), 1002
KEY_NEXT (*no módulo curses*), 1003
KEY_NOTIFY (*no módulo winreg*), 2301
KEY_NPAGE (*no módulo curses*), 1001
KEY_OPEN (*no módulo curses*), 1003
KEY_OPTIONS (*no módulo curses*), 1003
KEY_PPAGE (*no módulo curses*), 1001
KEY_PREVIOUS (*no módulo curses*), 1003
KEY_PRINT (*no módulo curses*), 1001
KEY_QUERY_VALUE (*no módulo winreg*), 2300
KEY_READ (*no módulo winreg*), 2300
KEY_REDO (*no módulo curses*), 1003
KEY_REFERENCE (*no módulo curses*), 1003
KEY_REFRESH (*no módulo curses*), 1003
KEY_REPLACE (*no módulo curses*), 1003
KEY_RESET (*no módulo curses*), 1001
KEY_RESIZE (*no módulo curses*), 1005
KEY_RESTART (*no módulo curses*), 1003
KEY_RESUME (*no módulo curses*), 1003
KEY_RIGHT (*no módulo curses*), 1000
KEY_SAVE (*no módulo curses*), 1003
KEY_SBEG (*no módulo curses*), 1003
KEY_SCANCEL (*no módulo curses*), 1003
KEY_SCOMMAND (*no módulo curses*), 1003
KEY_SCOPY (*no módulo curses*), 1003
KEY_SCREATE (*no módulo curses*), 1003
KEY_SDC (*no módulo curses*), 1003
KEY_SDL (*no módulo curses*), 1003
KEY_SELECT (*no módulo curses*), 1004
KEY_SEND (*no módulo curses*), 1004
KEY_SEOL (*no módulo curses*), 1004
KEY_SET_VALUE (*no módulo winreg*), 2300
KEY_SEXIT (*no módulo curses*), 1004
KEY_SF (*no módulo curses*), 1001
KEY_SFIND (*no módulo curses*), 1004
KEY_SHELP (*no módulo curses*), 1004
KEY_SHOME (*no módulo curses*), 1004
KEY_SIC (*no módulo curses*), 1004
KEY_SLEFT (*no módulo curses*), 1004
KEY_SMESSAGE (*no módulo curses*), 1004
KEY_SMOVE (*no módulo curses*), 1004
KEY_SNEXT (*no módulo curses*), 1004
KEY_SOPTIONS (*no módulo curses*), 1004
KEY_SPREVIOUS (*no módulo curses*), 1004
KEY_SPRINT (*no módulo curses*), 1004
KEY_SR (*no módulo curses*), 1001
KEY_SREDO (*no módulo curses*), 1004

KEY_SREPLACE (*no módulo curses*), 1004
KEY_SRESET (*no módulo curses*), 1001
KEY_SRIGHT (*no módulo curses*), 1004
KEY_SRSUME (*no módulo curses*), 1005
KEY_SSAVE (*no módulo curses*), 1005
KEY_SSUSPEND (*no módulo curses*), 1005
KEY_STAB (*no módulo curses*), 1001
KEY_SUNDO (*no módulo curses*), 1005
KEY_SUSPEND (*no módulo curses*), 1005
KEY_UNDO (*no módulo curses*), 1005
KEY_UP (*no módulo curses*), 1000
KEY_WOW64_32KEY (*no módulo winreg*), 2301
KEY_WOW64_64KEY (*no módulo winreg*), 2301
KEY_WRITE (*no módulo winreg*), 2300
KeyboardInterrupt, 118
KeyError, 118
keylog_filename (*atributo ssl.SSLContext*), 1293
keyname() (*no módulo curses*), 987
keypad() (*método curses.window*), 995
keyrefs() (*método weakref.WeakKeyDictionary*), 314
keys() (*método contextvars.Context*), 1131
keys() (*método dict*), 100
keys() (*método email.message.EmailMessage*), 1336
keys() (*método email.message.Message*), 1374
keys() (*método mailbox.Mailbox*), 1403
keys() (*método sqlite3.Row*), 585
keys() (*método types.MappingProxyType*), 324
keys() (*método xml.etree.ElementTree.Element*), 1452
KeysView (*classe em collections.abc*), 298
KeysView (*classe em typing*), 1806
keyword
    module, 2252
keyword (*classe em ast*), 2215
keywords (*atributo functools.partial*), 463
kill() (*método asyncio.subprocess.Process*), 1177
kill() (*método asyncio.SubprocessTransport*), 1217
kill() (*método multiprocessing.Process*), 1048
kill() (*método subprocess.Popen*), 1114
kill() (*no módulo os*), 766
kill_python() (*no módulo test.support.script_helper*), 1944
kill_workers() (*método concurrent.futures.ProcessPoolExecutor*), 1096
killchar() (*no módulo curses*), 987
killpg() (*no módulo os*), 766
kind (*atributo inspect.Parameter*), 2132
knownfiles (*no módulo mimetypes*), 1422
kqueue() (*no módulo select*), 1307
KqueueSelector (*classe em selectors*), 1315
KW_ONLY (*no módulo dataclasses*), 2081
kwargs (*atributo inspect.BoundArguments*), 2134
kwargs (*atributo typing.ParamSpec*), 1784
kwlist (*no módulo keyword*), 2252

L

-L
    calendar opção de linha de comando, 274
-l
    calendar opção de linha de comando, 275
    compileall opção de linha de comando, 2261
    mimetypes opção de linha de comando, 1424
    pickletools opção de linha de comando, 2289
    tarfile opção de linha de comando, 655
    trace opção de linha de comando, 1989
    zipfile opção de linha de comando, 640
L (*no módulo re*), 155
L2CAP_LM (*no módulo socket*), 1249
laço
    sobre sequência mutável, 49
lambda, **2355**
Lambda (*classe em ast*), 2234
LambdaType (*no módulo types*), 321
LANG, 1643, 1644, 1651, 1655
LANGUAGE, 1643, 1644
LARGEST (*no módulo test.support*), 1935
LargeZipFile, 629
last_accepted (*atributo multiprocessing.connection.Listener*), 1072
last_exc (*no módulo sys*), 2038
last_mode (*atributo compression.zstd.ZstdCompressor*), 603
last_traceback (*no módulo sys*), 2038
last_type (*no módulo sys*), 2038
last_value (*no módulo sys*), 2038
lastChild (*atributo xml.dom.Node*), 1461
lastcmd (*atributo cmd.Cmd*), 1017
lastgroup (*atributo re.Match*), 165
lastindex (*atributo re.Match*), 165
lastResort (*no módulo logging*), 825
lastrowid (*atributo sqlite3.Cursor*), 585
layout() (*método tkinter.ttk.Style*), 1697
lazy (*atributo compression.zstd.Strategy*), 609
lazy2 (*atributo compression.zstd.Strategy*), 609
lazycache() (*no módulo linecache*), 524
LazyLoader (*classe em importlib.util*), 2185
LBRACE (*no módulo token*), 2250
LBYL, **2355**
LC_ALL, 1643, 1644
LC_ALL (*no módulo locale*), 1658
LC_COLLATE (*no módulo locale*), 1657
LC_CTYPE (*no módulo locale*), 1657
LC_MESSAGES, 1643, 1644
LC_MESSAGES (*no módulo locale*), 1657
LC_MONETARY (*no módulo locale*), 1657
LC_NUMERIC (*no módulo locale*), 1657
LC_TIME (*no módulo locale*), 1657
lchflags() (*no módulo os*), 738
lchmod() (*método pathlib.Path*), 496
lchmod() (*no módulo os*), 738
lchown() (*no módulo os*), 739
lcm() (*no módulo math*), 362

ldexp() (*no módulo math*), 364
ldm_bucket_size_log (*atributo compression.zstd.CompressionParameter*), 607
ldm_hash_log (*atributo compression.zstd.CompressionParameter*), 607
ldm_hash_rate_log (*atributo compression.zstd.CompressionParameter*), 607
ldm_min_match (*atributo compression.zstd.CompressionParameter*), 607
le() (*no módulo operator*), 463
leapdays() (*no módulo calendar*), 271
leaveok() (*método curses.window*), 995
left (*atributo filecmp.dircmp*), 513
left() (*no módulo turtle*), 1720
left_list (*atributo filecmp.dircmp*), 513
left_only (*atributo filecmp.dircmp*), 513
LEFTSHIFT (*no módulo token*), 2251
LEFTSHIFTEQUAL (*no módulo token*), 2251
LEGACY_TRANSACTION_CONTROL (*no módulo sqlite3*), 570
len
    função embutida, 49, 98
len()
    built-in function, 21
length (*atributo xml.dom.NamedNodeMap*), 1465
length (*atributo xml.dom.NodeList*), 1462
length_hint() (*no módulo operator*), 466
--lenient
    mimetypes opção de linha de comando, 1424
LESS (*no módulo token*), 2250
LESSEQUAL (*no módulo token*), 2250
level (*atributo logging.Logger*), 809
leves, processos, 1133
LexicalHandler (*classe em xml.sax.handler*), 1476
lexists() (*no módulo os.path*), 500
LF (*no módulo curses.ascii*), 1011
lgamma() (*no módulo math*), 368
libc_ver() (*no módulo platform*), 856
LIBRARIES_ASSEMBLY_NAME_PREFIX (*no módulo msvcrt*), 2293
library (*atributo ssl.SSLError*), 1273
library (*no módulo dbm.ndbm*), 564
LibraryLoader (*classe em ctypes*), 887
license (*variável interna*), 38
LifoQueue (*classe em asyncio*), 1181
LifoQueue (*classe em queue*), 1125
ligação (*widgets*), 1672
limit_denominator() (*método fractions.Fraction*), 406
LimitOverrunError, 1182
line (*atributo bdb.Breakpoint*), 1956
line (*atributo traceback.FrameSummary*), 2113
LINE (*monitoring event*), 2051
line_buffering (*atributo io.TextIOWrapper*), 792
line_num (*atributo csv.csvreader*), 665
linear_regression() (*no módulo statistics*), 428
linecache
    module, 524
lineno (*atributo ast.AST*), 2207
lineno (*atributo doctest.DocTest*), 1829
lineno (*atributo doctest.Example*), 1830
lineno (*atributo inspect.FrameInfo*), 2137
lineno (*atributo inspect.Traceback*), 2137
lineno (*atributo json.JSONDecodeError*), 1398
lineno (*atributo netrc.NetrcParseError*), 689
lineno (*atributo pyclbr.Class*), 2259
lineno (*atributo pyclbr.Function*), 2258
lineno (*atributo re.PatternError*), 160
lineno (*atributo shlex.shlex*), 2310
lineno (*atributo SyntaxError*), 121
lineno (*atributo tomllib.TOMLDecodeError*), 687
lineno (*atributo traceback.FrameSummary*), 2113
lineno (*atributo traceback.TracebackException*), 2111
lineno (*atributo tracemalloc.Filter*), 1998
lineno (*atributo tracemalloc.Frame*), 1998
lineno (*atributo xml.parsers.expat.ExpatError*), 1492
lineno() (*no módulo fileinput*), 982
LINES, 985, 991
--lines
    calendar opção de linha de comando, 275
lines (*atributo os.terminal_size*), 734
LINES (*no módulo curses*), 998
linesep (*atributo email.policy.Policy*), 1350
linesep (*no módulo os*), 779
lineterminator (*atributo csv.Dialect*), 664
LineTooLong, 1545
linguagem
    C, 40
link() (*no módulo os*), 739
linked_to_musl() (*no módulo test.support*), 1940
LinkFallbackError, 644
linkname (*atributo tarfile.TarInfo*), 649
LinkOutsideDestinationError, 644
--list
    tarfile opção de linha de comando, 655
    zipfile opção de linha de comando, 640
List (*classe em ast*), 2211
List (*classe em typing*), 1804
list (*classe interna*), 52
list (*pdb command*), 1971
list() (*método imaplib.IMAP4*), 1564
list() (*método multiprocessing.managers.SyncManager*), 1064
list() (*método poplib.POP3*), 1559
list() (*método tarfile.TarFile*), 646
list_all() (*no módulo concurrent.interpreters*), 1102
LIST_APPEND (*opcode*), 2275
list_dialects() (*no módulo csv*), 661
LIST_EXTEND (*opcode*), 2279
list_folders() (*método mailbox.Maildir*), 1405
list_folders() (*método mailbox.MH*), 1408
lista, **2355**
    objeto, 51, 52
    tipo, operações em, 51

lista de propriedades, 689
ListComp (*classe em ast*), 2216
listdir() (*no módulo os*), 739
listdrives() (*no módulo os*), 739
listen() (*método socket.socket*), 1261
listen() (*no módulo logging.config*), 828
listen() (*no módulo turtle*), 1740
listener (*atributo logging.handlers.QueueHandler*), 851
Listener (*classe em multiprocessing.connection*), 1071
--listfuncs
    trace opção de linha de comando, 1989
listMethods() (*método xmlrpc.client.ServerProxy.system*), 1611
listmounts() (*no módulo os*), 740
listvolumes() (*no módulo os*), 740
listxattr() (*no módulo os*), 761
literais
    binário, 41
    hexadecimal, 41
    inteiro, 41
    numérico, 41
    número complexo, 41
    octal, 41
    ponto flutuante, 41
literais de string formatadas, 67
Literais de strings formatadas, 67
Literal (*no módulo typing*), 1770
literal de string formatada com !, 67
literal de string interpolada, 67
literal_eval() (*no módulo ast*), 2238
LiteralString (*no módulo typing*), 1766
LittleEndianStructure (*classe em ctypes*), 899
LittleEndianUnion (*classe em ctypes*), 899
ljust() (*método bytearray*), 78
ljust() (*método bytes*), 78
ljust() (*método str*), 62
LK_LOCK (*no módulo msvcrt*), 2291
LK_NBLCK (*no módulo msvcrt*), 2291
LK_NBRLCK (*no módulo msvcrt*), 2291
LK_RLCK (*no módulo msvcrt*), 2291
LK_UNLCK (*no módulo msvcrt*), 2291
ll (*pdb command*), 1971
LMTP (*classe em smtplib*), 1569
ln() (*método decimal.Context*), 391
ln() (*método decimal.Decimal*), 383
LNKTYPE (*no módulo tarfile*), 644
Load (*classe em ast*), 2212
load() (*método de classe tracemalloc.Snapshot*), 1999
load() (*método http.cookiejar.FileCookieJar*), 1603
load() (*método http.cookies.BaseCookie*), 1598
load() (*método pickle.Unpickler*), 542
load() (*no módulo json*), 1394
load() (*no módulo marshal*), 559
load() (*no módulo pickle*), 540
load() (*no módulo plistlib*), 689
load() (*no módulo tomllib*), 686
LOAD_ATTR (*opcode*), 2279
LOAD_BUILD_CLASS (*opcode*), 2276
load_cert_chain() (*método ssl.SSLContext*), 1288
LOAD_CLOSURE (*opcode*), 2287
LOAD_COMMON_CONSTANT (*opcode*), 2276
LOAD_CONST (*opcode*), 2277
LOAD_CONST_IMMORTAL (*opcode*), 2287
load_default_certs() (*método ssl.SSLContext*), 1289
LOAD_DEREF (*opcode*), 2282
load_dh_params() (*método ssl.SSLContext*), 1291
load_extension() (*método sqlite3.Connection*), 578
LOAD_FAST (*opcode*), 2281
LOAD_FAST_AND_CLEAR (*opcode*), 2282
LOAD_FAST_BORROW (*opcode*), 2281
LOAD_FAST_BORROW_LOAD_FAST_BORROW (*opcode*), 2281
LOAD_FAST_CHECK (*opcode*), 2282
LOAD_FAST_LOAD_FAST (*opcode*), 2281
LOAD_FROM_DICT_OR_DEREF (*opcode*), 2282
LOAD_FROM_DICT_OR_GLOBALS (*opcode*), 2278
LOAD_GLOBAL (*opcode*), 2281
LOAD_LOCALS (*opcode*), 2278
load_module() (*método importlib.abc.FileLoader*), 2174
load_module() (*método importlib.abc.InspectLoader*), 2173
load_module() (*método importlib.abc.Loader*), 2172
load_module() (*método importlib.abc.SourceLoader*), 2175
load_module() (*método importlib.machinery.SourceFileLoader*), 2179
load_module() (*método importlib.machinery.SourcelessFileLoader*), 2180
load_module() (*método zipimport.zipimporter*), 2160
LOAD_NAME (*opcode*), 2278
load_package_tests() (*no módulo test.support*), 1940
LOAD_SMALL_INT (*opcode*), 2278
LOAD_SPECIAL (*opcode*), 2287
LOAD_SUPER_ATTR (*opcode*), 2280
load_verify_locations() (*método ssl.SSLContext*), 1289
loader (*atributo importlib.machinery.ModuleSpec*), 2181
Loader (*classe em importlib.abc*), 2171
loader_state (*atributo importlib.machinery.ModuleSpec*), 2182
LoadError, 1600
LoadFileDialog (*classe em tkinter.filedialog*), 1678
LoadKey() (*no módulo winreg*), 2296
LoadLibrary() (*método ctypes.LibraryLoader*), 887
loads() (*no módulo json*), 1395
loads() (*no módulo marshal*), 559
loads() (*no módulo pickle*), 540
loads() (*no módulo plistlib*), 690
loads() (*no módulo tomllib*), 687
loads() (*no módulo xmlrpc.client*), 1615

loadTestsFromModule() (*método unittest.TestLoader*), 1860
loadTestsFromName() (*método unittest.TestLoader*), 1860
loadTestsFromNames() (*método unittest.TestLoader*), 1860
loadTestsFromTestCase() (*método unittest.TestLoader*), 1860
local (*classe em threading*), 1027
LOCAL_CREDS (*no módulo socket*), 1250
LOCAL_CREDS_PERSISTENT (*no módulo socket*), 1250
localcontext() (*no módulo decimal*), 387
locale
    module, 1651
--locale
    calendar opção de linha de comando, 274
LOCALE (*no módulo re*), 155
localeconv() (*no módulo locale*), 1651
LocaleHTMLCalendar (*classe em calendar*), 271
LocaleTextCalendar (*classe em calendar*), 271
localizador, **2351**
localizador baseado no caminho, **2358**
localizador de entrada de caminho, **2358**
localizador de metacaminho, **2356**
localize() (*no módulo locale*), 1657
localName (*atributo xml.dom.Attr*), 1465
localName (*atributo xml.dom.Node*), 1461
--locals
    unittest opção de linha de comando, 1840
locals()
    built-in function, 21
localtime() (*no módulo email.utils*), 1387
localtime() (*no módulo time*), 798
Locator (*classe em xml.sax.xmlreader*), 1483
lock (*atributo sys.thread_info*), 2047
Lock (*classe em asyncio*), 1169
Lock (*classe em multiprocessing*), 1057
Lock (*classe em threading*), 1030
lock() (*método mailbox.Babyl*), 1410
lock() (*método mailbox.Mailbox*), 1404
lock() (*método mailbox.Maildir*), 1407
lock() (*método mailbox.mbox*), 1408
lock() (*método mailbox.MH*), 1409
lock() (*método mailbox.MMDF*), 1411
Lock() (*método multiprocessing.managers.SyncManager*), 1063
LOCK_EX (*no módulo fcntl*), 2321
LOCK_NB (*no módulo fcntl*), 2321
LOCK_SH (*no módulo fcntl*), 2321
LOCK_UN (*no módulo fcntl*), 2321
locked() (*método _thread.lock*), 1135
locked() (*método asyncio.Condition*), 1171
locked() (*método asyncio.Lock*), 1170
locked() (*método asyncio.Semaphore*), 1173
locked() (*método multiprocessing.Lock*), 1058
locked() (*método multiprocessing.RLock*), 1058
locked() (*método threading.Condition*), 1033
locked() (*método threading.Lock*), 1031
locked() (*método threading.RLock*), 1032
lockf() (*no módulo fcntl*), 2321
lockf() (*no módulo os*), 725
locking() (*no módulo msvcrt*), 2291
LockType (*no módulo _thread*), 1133
log() (*método logging.Logger*), 813
log() (*no módulo cmath*), 372
log() (*no módulo logging*), 822
log() (*no módulo math*), 366
log1p() (*no módulo math*), 366
log2() (*no módulo math*), 366
log10() (*método decimal.Context*), 391
log10() (*método decimal.Decimal*), 383
log10() (*no módulo cmath*), 372
log10() (*no módulo math*), 366
LOG_ALERT (*no módulo syslog*), 2328
LOG_AUTH (*no módulo syslog*), 2328
LOG_AUTHPRIV (*no módulo syslog*), 2328
LOG_CONS (*no módulo syslog*), 2328
LOG_CRIT (*no módulo syslog*), 2328
LOG_CRON (*no módulo syslog*), 2328
LOG_DAEMON (*no módulo syslog*), 2328
log_date_time_string() (*método http.server.BaseHTTPRequestHandler*), 1593
LOG_DEBUG (*no módulo syslog*), 2328
LOG_EMERG (*no módulo syslog*), 2328
LOG_ERR (*no módulo syslog*), 2328
log_error() (*método http.server.BaseHTTPRequestHandler*), 1593
log_exception() (*método wsgiref.handlers.BaseHandler*), 1507
LOG_FTP (*no módulo syslog*), 2328
LOG_INFO (*no módulo syslog*), 2328
LOG_INSTALL (*no módulo syslog*), 2328
LOG_KERN (*no módulo syslog*), 2328
LOG_LAUNCHD (*no módulo syslog*), 2328
LOG_LOCAL0 (*no módulo syslog*), 2328
LOG_LOCAL1 (*no módulo syslog*), 2328
LOG_LOCAL2 (*no módulo syslog*), 2328
LOG_LOCAL3 (*no módulo syslog*), 2328
LOG_LOCAL4 (*no módulo syslog*), 2328
LOG_LOCAL5 (*no módulo syslog*), 2328
LOG_LOCAL6 (*no módulo syslog*), 2328
LOG_LOCAL7 (*no módulo syslog*), 2328
LOG_LPR (*no módulo syslog*), 2328
LOG_MAIL (*no módulo syslog*), 2328
log_message() (*método http.server.BaseHTTPRequestHandler*), 1593
LOG_NDELAY (*no módulo syslog*), 2328
LOG_NETINFO (*no módulo syslog*), 2328
LOG_NEWS (*no módulo syslog*), 2328
LOG_NOTICE (*no módulo syslog*), 2328
LOG_NOWAIT (*no módulo syslog*), 2328

LOG_ODELAY (*no módulo syslog*), 2328
LOG_PERROR (*no módulo syslog*), 2328
LOG_PID (*no módulo syslog*), 2328
LOG_RAS (*no módulo syslog*), 2328
LOG_REMOTEAUTH (*no módulo syslog*), 2328
log_request() (*método http.server.BaseHTTPRequestHandler*), 1593
LOG_SYSLOG (*no módulo syslog*), 2328
log_to_stderr() (*no módulo multiprocessing*), 1074
LOG_USER (*no módulo syslog*), 2328
LOG_UUCP (*no módulo syslog*), 2328
LOG_WARNING (*no módulo syslog*), 2328
logb() (*método decimal.Context*), 391
logb() (*método decimal.Decimal*), 383
Logger (*classe em logging*), 809
LoggerAdapter (*classe em logging*), 821
logging
    Erros, 808
    module, 808
logging.config
    module, 826
logging.handlers
    module, 838
logical_and() (*método decimal.Context*), 391
logical_and() (*método decimal.Decimal*), 383
logical_invert() (*método decimal.Context*), 391
logical_invert() (*método decimal.Decimal*), 383
logical_or() (*método decimal.Context*), 391
logical_or() (*método decimal.Decimal*), 383
logical_xor() (*método decimal.Context*), 391
logical_xor() (*método decimal.Decimal*), 383
login() (*método ftplib.FTP*), 1552
login() (*método imaplib.IMAP4*), 1564
login() (*método smtplib.SMTP*), 1571
login_cram_md5() (*método imaplib.IMAP4*), 1564
login_tty() (*no módulo os*), 725
LOGNAME, 716, 980
lognormvariate() (*no módulo random*), 411
logout() (*método imaplib.IMAP4*), 1564
LogRecord (*classe em logging*), 818
LONG_TIMEOUT (*no módulo test.support*), 1934
longMessage (*atributo unittest.TestCase*), 1856
longname() (*no módulo curses*), 987
lookup() (*método symtable.SymbolTable*), 2244
lookup() (*método tkinter.ttk.Style*), 1697
lookup() (*no módulo codecs*), 205
lookup() (*no módulo unicodedata*), 187
lookup_error() (*no módulo codecs*), 209
LookupError, 117
loop_factory (*atributo unittest.IsolatedAsyncioTestCase*), 1857
LOOPBACK_TIMEOUT (*no módulo test.support*), 1933
lower() (*método bytearray*), 83
lower() (*método bytes*), 83
lower() (*método str*), 62
LPAR (*no módulo token*), 2249
lpAttributeList (*atributo subprocess.STARTUPINFO*), 1116
lru_cache() (*no módulo functools*), 454
lseek() (*no módulo os*), 725
LShift (*classe em ast*), 2213
lshift() (*no módulo operator*), 465
LSQB (*no módulo token*), 2249
lstat() (*método pathlib.Path*), 486
lstat() (*no módulo os*), 740
lstrip() (*método bytearray*), 78
lstrip() (*método bytes*), 78
lstrip() (*método str*), 62
lsub() (*método imaplib.IMAP4*), 1564
Lt (*classe em ast*), 2214
lt() (*no módulo operator*), 463
lt() (*no módulo turtle*), 1720
LtE (*classe em ast*), 2214
LWPCookieJar (*classe em http.cookiejar*), 1604
lzma
    module, 623
LZMACompressor (*classe em lzma*), 625
LZMADecompressor (*classe em lzma*), 626
LZMAError, 623
LZMAFile (*classe em lzma*), 624

M

-m
    ast opção de linha de comando, 2242
    calendar opção de linha de comando, 275
    cProfile opção de linha de comando, 1977
    pdb opção de linha de comando, 1965
    pickletools opção de linha de comando, 2289
    trace opção de linha de comando, 1989
    zipapp opção de linha de comando, 2016
M (*no módulo re*), 155
mac_ver() (*no módulo platform*), 855
machine() (*no módulo platform*), 853
macros (*atributo netrc.netrc*), 689
MADV_AUTOSYNC (*no módulo mmap*), 1330
MADV_CORE (*no módulo mmap*), 1330
MADV_DODUMP (*no módulo mmap*), 1330
MADV_DOFORK (*no módulo mmap*), 1330
MADV_DONTDUMP (*no módulo mmap*), 1330
MADV_DONTFORK (*no módulo mmap*), 1330
MADV_DONTNEED (*no módulo mmap*), 1330
MADV_FREE (*no módulo mmap*), 1330
MADV_FREE_REUSABLE (*no módulo mmap*), 1330
MADV_FREE_REUSE (*no módulo mmap*), 1330
MADV_HUGEPAGE (*no módulo mmap*), 1330
MADV_HWPOISON (*no módulo mmap*), 1330
MADV_MERGEABLE (*no módulo mmap*), 1330
MADV_NOCORE (*no módulo mmap*), 1330
MADV_NOHUGEPAGE (*no módulo mmap*), 1330
MADV_NORMAL (*no módulo mmap*), 1330
MADV_NOSYNC (*no módulo mmap*), 1330

MADV_PROTECT (*no módulo mmap*), 1330
MADV_RANDOM (*no módulo mmap*), 1330
MADV_REMOVE (*no módulo mmap*), 1330
MADV_SEQUENTIAL (*no módulo mmap*), 1330
MADV_SOFT_OFFLINE (*no módulo mmap*), 1330
MADV_UNMERGEABLE (*no módulo mmap*), 1330
MADV_WILLNEED (*no módulo mmap*), 1330
madvise() (*método mmap.mmap*), 1329
MAGIC_NUMBER (*no módulo importlib.util*), 2183
MagicMock (*classe em unittest.mock*), 1899
mágico
    método, 2355
mailbox
    module, 1401
Mailbox (*classe em mailbox*), 1401
mailcap
    module, 2339
Maildir (*classe em mailbox*), 1405
MaildirMessage (*classe em mailbox*), 1412
--main
    zipapp opção de linha de comando, 2016
main() (*no módulo site*), 2153
main() (*no módulo unittest*), 1865
main_thread() (*no módulo threading*), 1024
mainloop() (*no módulo turtle*), 1741
maintype (*atributo email.headerregistry.ContentTypeHeader*), 1359
major (*atributo email.headerregistry.MIMEVersionHeader*), 1359
major() (*no módulo os*), 742
make_alternative() (*método email.message.EmailMessage*), 1341
make_archive() (*no módulo shutil*), 531
make_bad_fd() (*no módulo test.support.os_helper*), 1947
MAKE_CELL (*opcode*), 2282
make_cookies() (*método http.cookiejar.CookieJar*), 1602
make_dataclass() (*no módulo dataclasses*), 2080
make_file() (*método difflib.HtmlDiff*), 172
MAKE_FUNCTION (*opcode*), 2284
make_header() (*no módulo email.header*), 1384
make_legacy_pyc() (*no módulo test.support.import_helper*), 1948
make_mixed() (*método email.message.EmailMessage*), 1341
make_msgid() (*no módulo email.utils*), 1387
make_parser() (*no módulo xml.sax*), 1475
make_pkg() (*no módulo test.support.script_helper*), 1944
make_related() (*método email.message.EmailMessage*), 1340
make_script() (*no módulo test.support.script_helper*), 1944
make_server() (*no módulo wsgiref.simple_server*), 1503
make_table() (*método difflib.HtmlDiff*), 172
make_zip_pkg() (*no módulo test.support.script_helper*), 1944
make_zip_script() (*no módulo test.support.script_helper*), 1944
makedev() (*no módulo os*), 742
makedirs() (*no módulo os*), 741
makeelement() (*método xml.etree.ElementTree.Element*), 1453
makefile() (*método socket.socket*), 1261
makeLogRecord() (*no módulo logging*), 823
makePickle() (*método logging.handlers.SocketHandler*), 844
makeRecord() (*método logging.Logger*), 813
makeSocket() (*método logging.handlers.DatagramHandler*), 845
makeSocket() (*método logging.handlers.SocketHandler*), 844
maketrans() (*método estático bytearray*), 76
maketrans() (*método estático bytes*), 76
maketrans() (*método estático str*), 62
MalformedHeaderDefect, 1356
manager (*atributo logging.LoggerAdapter*), 821
mangle_from_ (*atributo email.policy.Compat32*), 1354
mangle_from_ (*atributo email.policy.Policy*), 1350
MANPAGER, 1810
mant_dig (*atributo sys.float_info*), 2030
map()
    built-in function, 22
map() (*método concurrent.futures.Executor*), 1091
map() (*método multiprocessing.pool.Pool*), 1069
map() (*método tkinter.ttk.Style*), 1697
MAP_32BIT (*no módulo mmap*), 1330
MAP_ADD (*opcode*), 2275
MAP_ALIGNED_SUPER (*no módulo mmap*), 1330
MAP_ANON (*no módulo mmap*), 1330
MAP_ANONYMOUS (*no módulo mmap*), 1330
map_async() (*método multiprocessing.pool.Pool*), 1069
MAP_CONCEAL (*no módulo mmap*), 1330
MAP_DENYWRITE (*no módulo mmap*), 1330
MAP_EXECUTABLE (*no módulo mmap*), 1330
MAP_HASSEMAPHORE (*no módulo mmap*), 1330
MAP_JIT (*no módulo mmap*), 1330
MAP_NOCACHE (*no módulo mmap*), 1330
MAP_NOEXTEND (*no módulo mmap*), 1330
MAP_NORESERVE (*no módulo mmap*), 1330
MAP_POPULATE (*no módulo mmap*), 1330
MAP_PRIVATE (*no módulo mmap*), 1330
MAP_RESILIENT_CODESIGN (*no módulo mmap*), 1330
MAP_RESILIENT_MEDIA (*no módulo mmap*), 1330
MAP_SHARED (*no módulo mmap*), 1330
MAP_STACK (*no módulo mmap*), 1330
map_table_b2() (*no módulo stringprep*), 190
map_table_b3() (*no módulo stringprep*), 190
map_to_type() (*método email.headerregistry.HeaderRegistry*), 1360
MAP_TPRO (*no módulo mmap*), 1330
MAP_TRANSLATED_ALLOW_EXECUTE (*no módulo mmap*), 1330

MAP_UNIX03 (*no módulo mmap*), 1330
mapeamento, **2356**
    objeto, 98
    tipos, operações em, 98
mapLogRecord() (*método logging.handlers.HTTPHandler*), 849
Mapping (*classe em collections.abc*), 298
Mapping (*classe em typing*), 1806
MappingProxyType (*classe em types*), 324
MappingView (*classe em collections.abc*), 298
MappingView (*classe em typing*), 1806
mapPriority() (*método logging.handlers.SysLogHandler*), 847
maps (*atributo collections.ChainMap*), 276
máquina virtual, **2363**
MARCH (*no módulo calendar*), 273
markcoroutinefunction() (*no módulo inspect*), 2127
marshal
    module, 558
marshalling
    objetos, 537
mascaramento
    operações, 42
master (*atributo tkinter.Tk*), 1663
Match (*classe em ast*), 2226
Match (*classe em re*), 162
Match (*classe em typing*), 1805
match() (*método pathlib.PurePath*), 480
match() (*método re.Pattern*), 161
match() (*no módulo re*), 157
match_case (*classe em ast*), 2226
MATCH_CLASS (*opcode*), 2285
MATCH_KEYS (*opcode*), 2276
MATCH_MAPPING (*opcode*), 2276
MATCH_SEQUENCE (*opcode*), 2276
match_value() (*método test.support.Matcher*), 1942
MatchAs (*classe em ast*), 2230
MatchClass (*classe em ast*), 2229
Matcher (*classe em test.support*), 1942
matches() (*método test.support.Matcher*), 1942
MatchMapping (*classe em ast*), 2228
MatchOr (*classe em ast*), 2231
MatchSequence (*classe em ast*), 2227
MatchSingleton (*classe em ast*), 2227
MatchStar (*classe em ast*), 2228
MatchValue (*classe em ast*), 2226
math
    module, 360
    módulo, 42, 374
matmul() (*no módulo operator*), 465
MatMult (*classe em ast*), 2213
max
    função embutida, 49
max (*atributo datetime.date*), 231
max (*atributo datetime.datetime*), 238
max (*atributo datetime.time*), 247
max (*atributo datetime.timedelta*), 227
max (*atributo sys.float_info*), 2030
MAX (*no módulo uuid*), 1578
max()
    built-in function, 22
max() (*método decimal.Context*), 391
max() (*método decimal.Decimal*), 383
max_10_exp (*atributo sys.float_info*), 2030
max_children (*atributo socketserver.ThreadingMixIn*), 1582
max_count (*atributo email.headerregistry.BaseHeader*), 1357
MAX_EMAX (*no módulo decimal*), 393
max_exp (*atributo sys.float_info*), 2030
MAX_INTERPOLATION_DEPTH (*no módulo configparser*), 684
max_line_length (*atributo email.policy.Policy*), 1350
max_lines (*atributo textwrap.TextWrapper*), 187
max_mag() (*método decimal.Context*), 391
max_mag() (*método decimal.Decimal*), 383
max_memuse (*no módulo test.support*), 1935
MAX_PREC (*no módulo decimal*), 393
max_prefixlen (*atributo ipaddress.IPv4Address*), 1624
max_prefixlen (*atributo ipaddress.IPv4Network*), 1629
max_prefixlen (*atributo ipaddress.IPv6Address*), 1627
max_prefixlen (*atributo ipaddress.IPv6Network*), 1632
MAX_Py_ssize_t (*no módulo test.support*), 1934
maxarray (*atributo reprlib.Repr*), 334
maxdeque (*atributo reprlib.Repr*), 334
maxdict (*atributo reprlib.Repr*), 334
maxDiff (*atributo unittest.TestCase*), 1856
maxfrozenset (*atributo reprlib.Repr*), 334
MAXIMUM_SUPPORTED (*atributo ssl.TLSVersion*), 1282
maximum_version (*atributo ssl.SSLContext*), 1293
maxlen (*atributo collections.deque*), 283
maxlevel (*atributo reprlib.Repr*), 334
maxlist (*atributo reprlib.Repr*), 334
maxlong (*atributo reprlib.Repr*), 334
maxother (*atributo reprlib.Repr*), 334
maxset (*atributo reprlib.Repr*), 334
maxsize (*atributo asyncio.Queue*), 1179
maxsize (*no módulo sys*), 2038
maxstring (*atributo reprlib.Repr*), 334
maxtuple (*atributo reprlib.Repr*), 334
maxunicode (*no módulo sys*), 2038
MAXYEAR (*no módulo datetime*), 224
MAY (*no módulo calendar*), 273
MB_ICONASTERISK (*no módulo winsound*), 2304
MB_ICONERROR (*no módulo winsound*), 2305
MB_ICONEXCLAMATION (*no módulo winsound*), 2304
MB_ICONHAND (*no módulo winsound*), 2304
MB_ICONINFORMATION (*no módulo winsound*), 2305
MB_ICONQUESTION (*no módulo winsound*), 2305
MB_ICONSTOP (*no módulo winsound*), 2305
MB_ICONWARNING (*no módulo winsound*), 2305

MB_OK (*no módulo winsound*), 2305
mbox (*classe em mailbox*), 1407
mboxMessage (*classe em mailbox*), 1413
md5() (*no módulo hashlib*), 694
mean (*atributo statistics.NormalDist*), 429
mean() (*no módulo statistics*), 419
measure() (*método tkinter.font.Font*), 1675
median (*atributo statistics.NormalDist*), 429
median() (*no módulo statistics*), 422
median_grouped() (*no módulo statistics*), 423
median_high() (*no módulo statistics*), 423
median_low() (*no módulo statistics*), 423
member() (*no módulo enum*), 351
member_names (*atributo enum.EnumDict*), 349
MemberDescriptorType (*no módulo types*), 323
memfd_create() (*no módulo os*), 756
memmove() (*no módulo ctypes*), 894
--memo
    pickletools opção de linha de comando, 2289
Memória compartilhada, 1084
MemoryBIO (*classe em ssl*), 1303
MemoryError, 118
MemoryHandler (*classe em logging.handlers*), 849
memoryview
    objeto, 71
memoryview (*classe interna*), 88
memoryview_at() (*no módulo ctypes*), 895
memset() (*no módulo ctypes*), 894
merge() (*no módulo heapq*), 301
message (*atributo BaseExceptionGroup*), 125
Message (*classe em email.message*), 1371
Message (*classe em mailbox*), 1411
Message (*classe em tkinter.messagebox*), 1679
message digest, MD5, 693
message_factory (*atributo email.policy.Policy*), 1350
message_from_binary_file() (*no módulo email*), 1344
message_from_bytes() (*no módulo email*), 1344
message_from_file() (*no módulo email*), 1344
message_from_string() (*no módulo email*), 1344
MessageBeep() (*no módulo winsound*), 2303
MessageClass (*atributo http.server.BaseHTTPRequestHandler*), 1592
MessageDefect, 1355
MessageError, 1355
MessageParseError, 1355
messages (*no módulo xml.parsers.expat.errors*), 1494
meta() (*no módulo curses*), 987
meta_path (*no módulo sys*), 2039
metaclasse, **2356**
metadata() (*no módulo importlib.metadata*), 2196
--metadata-encoding
    zipfile opção de linha de comando, 640
MetaPathFinder (*classe em importlib.abc*), 2171
metavar (*atributo optparse.Option*), 967
MetavarTypeHelpFormatter (*classe em argparse*), 909
method (*atributo urllib.request.Request*), 1517
method_calls (*atributo unittest.mock.Mock*), 1878
methodcaller() (*no módulo operator*), 467
MethodDescriptorType (*no módulo types*), 322
methodHelp() (*método xmlrpc.client.ServerProxy.system*), 1611
methods (*atributo pyclbr.Class*), 2259
methodSignature() (*método xmlrpc.client.ServerProxy.system*), 1611
MethodType (*no módulo types*), 322
MethodWrapperType (*no módulo types*), 322
método, **2356**
    especial, 2361
    mágico, 2355
    objeto, 110
método especial, **2361**
método mágico, **2355**
métodos
    bytearray, 74
    bytes, 74
    string, 57
metrics() (*método tkinter.font.Font*), 1675
MFD_ALLOW_SEALING (*no módulo os*), 756
MFD_CLOEXEC (*no módulo os*), 756
MFD_HUGE_1GB (*no módulo os*), 756
MFD_HUGE_1MB (*no módulo os*), 756
MFD_HUGE_2GB (*no módulo os*), 756
MFD_HUGE_2MB (*no módulo os*), 756
MFD_HUGE_8MB (*no módulo os*), 756
MFD_HUGE_16GB (*no módulo os*), 756
MFD_HUGE_16MB (*no módulo os*), 756
MFD_HUGE_32MB (*no módulo os*), 756
MFD_HUGE_64KB (*no módulo os*), 756
MFD_HUGE_256MB (*no módulo os*), 756
MFD_HUGE_512KB (*no módulo os*), 756
MFD_HUGE_512MB (*no módulo os*), 756
MFD_HUGE_MASK (*no módulo os*), 756
MFD_HUGE_SHIFT (*no módulo os*), 756
MFD_HUGETLB (*no módulo os*), 756
MH (*classe em mailbox*), 1408
MHMessage (*classe em mailbox*), 1415
microsecond (*atributo datetime.datetime*), 239
microsecond (*atributo datetime.time*), 247
microseconds (*atributo datetime.timedelta*), 227
MIME
    base64 encoding, 1425
    codificação de imprimíveis entre aspas, 1431
    content type, 1420
    headers, 1420, 1421
MIMEApplication (*classe em email.mime.application*), 1380
MIMEAudio (*classe em email.mime.audio*), 1380
MIMEBase (*classe em email.mime.base*), 1379
MIMEImage (*classe em email.mime.image*), 1380
MIMEMessage (*classe em email.mime.message*), 1381

MIMEMultipart (*classe em email.mime.multipart*), 1379
MIMENonMultipart (*classe em email.mime.nonmultipart*), 1379
MIMEPart (*classe em email.message*), 1342
MIMEText (*classe em email.mime.text*), 1381
mimetypes
    module, 1420
MimeTypes (*classe em mimetypes*), 1422
mimetypes opção de linha de comando
    -e, 1424
    --extension, 1424
    -h, 1424
    --help, 1424
    -l, 1424
    --lenient, 1424
MIMEVersionHeader (*classe em email.headerregistry*), 1359
min
    função embutida, 49
min (*atributo datetime.date*), 231
min (*atributo datetime.datetime*), 238
min (*atributo datetime.time*), 247
min (*atributo datetime.timedelta*), 227
min (*atributo sys.float_info*), 2030
min()
    built-in function, 23
min() (*método decimal.Context*), 391
min() (*método decimal.Decimal*), 383
min_10_exp (*atributo sys.float_info*), 2030
MIN_EMIN (*no módulo decimal*), 393
MIN_ETINY (*no módulo decimal*), 393
min_exp (*atributo sys.float_info*), 2030
min_mag() (*método decimal.Context*), 391
min_mag() (*método decimal.Decimal*), 383
min_match (*atributo compression.zstd.CompressionParameter*), 606
MINEQUAL (*no módulo token*), 2251
MINIMUM_SUPPORTED (*atributo ssl.TLSVersion*), 1282
minimum_version (*atributo ssl.SSLContext*), 1294
minor (*atributo email.headerregistry.MIMEVersionHeader*), 1359
minor() (*no módulo os*), 742
MINUS (*no módulo token*), 2250
minus() (*método decimal.Context*), 391
minute (*atributo datetime.datetime*), 239
minute (*atributo datetime.time*), 247
MINYEAR (*no módulo datetime*), 224
mirrored() (*no módulo unicodedata*), 188
misc_header (*atributo cmd.Cmd*), 1017
MisplacedEnvelopeHeaderDefect, 1356
--missing
    trace opção de linha de comando, 1989
MISSING (*atributo contextvars.Token*), 1130
MISSING (*no módulo dataclasses*), 2081
MISSING (*no módulo sys.monitoring*), 2054
MISSING_C_DOCSTRINGS (*no módulo test.support*), 1935
missing_compiler_executable() (*no módulo test.support*), 1941
MissingHeaderBodySeparatorDefect, 1356
MissingSectionHeaderError, 686
mkd() (*método ftplib.FTP*), 1554
mkdir() (*método pathlib.Path*), 492
mkdir() (*método zipfile.ZipFile*), 635
mkdir() (*no módulo os*), 741
mkdtemp() (*no módulo tempfile*), 517
mkfifo() (*no módulo os*), 741
mknod() (*no módulo os*), 742
mkstemp() (*no módulo tempfile*), 516
mktemp() (*no módulo tempfile*), 519
mktime() (*no módulo time*), 798
mktime_tz() (*no módulo email.utils*), 1389
mlsd() (*método ftplib.FTP*), 1554
mmap
    module, 1326
mmap (*classe em mmap*), 1326
MMDF (*classe em mailbox*), 1410
MMDFMessage (*classe em mailbox*), 1417
Mock (*classe em unittest.mock*), 1871
mock_add_spec() (*método unittest.mock.Mock*), 1874
mock_calls (*atributo unittest.mock.Mock*), 1878
mock_open() (*no módulo unittest.mock*), 1904
Mod (*classe em ast*), 2213
mod() (*no módulo operator*), 465
--mode
    ast opção de linha de comando, 2242
mode (*atributo bz2.BZ2File*), 620
mode (*atributo compression.zstd.ZstdFile*), 601
mode (*atributo gzip.GzipFile*), 616
mode (*atributo io.FileIO*), 788
mode (*atributo lzma.LZMAFile*), 624
mode (*atributo statistics.NormalDist*), 429
mode (*atributo tarfile.TarInfo*), 649
mode() (*no módulo statistics*), 424
mode() (*no módulo turtle*), 1742
modf() (*no módulo math*), 363
modified() (*método urllib.robotparser.RobotFileParser*), 1539
modify() (*método select.devpoll*), 1308
modify() (*método select.epoll*), 1309
modify() (*método selectors.BaseSelector*), 1314
modify() (*método select.poll*), 1310
modo binário, 26
modos
    arquivo, 24
module
    __future__, 2117
    __main__, 2061
    _thread, 1133
    _tkinter, 1664
    abc, 2101
    aifc, 2337
    annotationlib, 2143
    argparse, 905
    array, 308

ast, 2203
asynchat, 2337
asyncio, 1137
asyncore, 2337
atexit, 2106
audioop, 2338
base64, 1425
bdb, 1955
binascii, 1429
bisect, 305
builtins, 2061
bz2, 618
calendar, 267
cgi, 2338
cgitb, 2338
chunk, 2338
cmath, 370
cmd, 1015
code, 2155
codecs, 204
codeop, 2157
collections, 275
collections.abc, 294
colorsys, 1642
compileall, 2261
compression.zstd, 599
concurrent.futures, 1091
concurrent.interpreters, 1099
configparser, 667
contextlib, 2086
contextvars, 1128
copy, 326
copyreg, 554
cProfile, 1978
crypt, 2338
csv, 659
ctypes, 865
curses, 983
curses.ascii, 1010
curses.panel, 1014
curses.textpad, 1009
dataclasses, 2075
datetime, 223
dbm, 560
dbm.dumb, 565
dbm.gnu, 562
dbm.ndbm, 564
dbm.sqlite3, 562
decimal, 374
difflib, 171
dis, 2265
distutils, 2338
doctest, 1814
email, 1333
email.charset, 1384
email.contentmanager, 1362
email.encoders, 1386
email.errors, 1355
email.generator, 1345
email.header, 1382
email.headerregistry, 1356
email.iterators, 1390
email.message, 1334
email.mime, 1379
email.mime.application, 1380
email.mime.audio, 1380
email.mime.base, 1379
email.mime.image, 1380
email.mime.message, 1381
email.mime.multipart, 1379
email.mime.nonmultipart, 1379
email.mime.text, 1381
email.parser, 1342
email.policy, 1348
email.utils, 1387
encodings, 220
encodings.idna, 221
encodings.mbcs, 222
encodings.utf_8_sig, 222
ensurepip, 2003
enum, 336
errno, 857
faulthandler, 1961
fcntl, 2319
filecmp, 512
fileinput, 981
fnmatch, 522
fractions, 404
ftplib, 1550
functools, 452
gc, 2119
getopt, 2333
getpass, 980
gettext, 1643
glob, 520
graphlib, 352
grp, 2314
gzip, 615
hashlib, 693
heapq, 300
hmac, 705
html, 1433
html.entities, 1438
html.parser, 1434
http, 1539
http.client, 1543
http.cookiejar, 1600
http.cookies, 1597
http.server, 1589
idlelib, 1712
imaplib, 1560
imghdr, 2339
imp, 2339
importlib, 2168
importlib.abc, 2170
importlib.machinery, 2177

importlib.metadata, 2193
importlib.resources, 2188
importlib.resources.abc, 2191
importlib.util, 2183
inspect, 2123
io, 780
ipaddress, 1623
itertools, 435
json, 1391
json.tool, 1400
keyword, 2252
linecache, 524
locale, 1651
logging, 808
logging.config, 826
logging.handlers, 838
lzma, 623
mailbox, 1401
mailcap, 2339
marshal, 558
math, 360
mimetypes, 1420
mmap, 1326
modulefinder, 2164
msilib, 2339
msvcrt, 2291
multiprocessing, 1038
multiprocessing.connection, 1071
multiprocessing.dummy, 1075
multiprocessing.managers, 1062
multiprocessing.pool, 1068
multiprocessing.shared_memory, 1084
multiprocessing.sharedctypes, 1060
netrc, 688
nis, 2339
nntplib, 2339
numbers, 357
operator, 463
optparse, 951
os, 711
os.path, 499
ossaudiodev, 2340
pathlib, 471
pathlib.types, 498
pdb, 1964
pickle, 537
pickletools, 2288
pipes, 2340
pkgutil, 2161
platform, 852
plistlib, 689
poplib, 1557
posix, 2312
pprint, 327
profile, 1978
pstats, 1979
pty, 2317
pwd, 2313
py_compile, 2259
pyclbr, 2257
pydoc, 1809
queue, 1125
quopri, 1431
random, 407
re, 147
readline, 191
reprlib, 333
resource, 2322
rlcompleter, 196
runpy, 2165
sched, 1123
secrets, 706
select, 1306
selectors, 1312
shelve, 555
shlex, 2307
shutil, 524
signal, 1316
site, 2151
sitecustomize, 2153
smtpd, 2340
smtplib, 1568
sndhdr, 2340
socket, 1241
socketserver, 1580
spwd, 2340
sqlite3, 566
ssl, 1270
stat, 506
statistics, 418
string, 129
stringprep, 189
string.templatelib, 142
struct, 197
subprocess, 1104
sunau, 2341
symtable, 2243
sys, 2021
sysconfig, 2054
syslog, 2326
sys.monitoring, 2049
tabnanny, 2257
tarfile, 641
telnetlib, 2341
tempfile, 514
termios, 2315
test, 1930
test.regrtest, 1932
test.support, 1933
test.support.bytecode_helper, 1944
test.support.import_helper, 1947
test.support.os_helper, 1946
test.support.script_helper, 1943
test.support.socket_helper, 1942
test.support.threading_helper, 1945
test.support.warnings_helper, 1949

textwrap, 183
threading, 1021
time, 795
timeit, 1983
tkinter, 1661
tkinter.colorchooser, 1674
tkinter.commondialog, 1679
tkinter.dnd, 1682
tkinter.filedialog, 1676
tkinter.font, 1675
tkinter.messagebox, 1679
tkinter.scrolledtext, 1681
tkinter.simpledialog, 1676
tkinter.ttk, 1682
token, 2247
tokenize, 2253
tomllib, 686
trace, 1988
traceback, 2107
tracemalloc, 1991
tty, 2317
turtle, 1712
turtledemo, 1749
types, 319
typing, 1753
unicodedata, 187
unittest, 1837
unittest.mock, 1869
urllib, 1510
urllib.error, 1537
urllib.parse, 1528
urllib.request, 1511
urllib.response, 1527
urllib.robotparser, 1538
usercustomize, 2153
uu, 2341
uuid, 1574
venv, 2005
warnings, 2066
wave, 1639
weakref, 311
webbrowser, 1497
winreg, 2294
winsound, 2303
wsgiref, 1500
wsgiref.handlers, 1505
wsgiref.headers, 1502
wsgiref.simple_server, 1503
wsgiref.types, 1509
wsgiref.util, 1500
wsgiref.validate, 1504
xdrlib, 2341
xml, 1439
xml.dom, 1458
xml.dom.minidom, 1468
xml.dom.pulldom, 1472
xml.etree.ElementInclude, 1451
xml.etree.ElementTree, 1440
xml.parsers.expat, 1486
xml.parsers.expat.errors, 1493
xml.parsers.expat.model, 1493
xmlrpc, 1609
xmlrpc.client, 1609
xmlrpc.server, 1617
xml.sax, 1474
xml.sax.handler, 1476
xml.sax.saxutils, 1481
xml.sax.xmlreader, 1482
zipapp, 2015
zipfile, 629
zipimport, 2159
zlib, 611
zoneinfo, 262
module (*atributo pyclbr.Class*), 2259
module (*atributo pyclbr.Function*), 2258
MODULE (*atributo symtable.SymbolTableType*), 2243
Module (*classe em ast*), 2208
Module browser, 1701
module_from_spec() (*no módulo importlib.util*), 2184
modulefinder
    module, 2164
ModuleFinder (*classe em modulefinder*), 2164
ModuleInfo (*classe em pkgutil*), 2161
ModuleNotFoundError, 118
modules (*atributo modulefinder.ModuleFinder*), 2164
modules (*no módulo sys*), 2039
modules_cleanup() (*no módulo test.support.import_helper*), 1948
modules_setup() (*no módulo test.support.import_helper*), 1948
ModuleSpec (*classe em importlib.machinery*), 2181
ModuleType (*classe em types*), 322
módulo, **2356**
    __main__, 2166
    _locale, 1651
    array, 71
    base64, 1429
    bdb, 1964
    builtins, 34
    cmd, 1964
    copy, 554
    dbm.gnu, 556
    dbm.ndbm, 556
    errno, 119
    glob, 523
    math, 42, 374
    os, 2312
    pesquisa caminho, 524, 2039, 2151
    pickle, 326, 554, 555, 558
    pty, 727
    pwd, 501
    pyexpat, 1486
    re, 57, 522
    shelve, 558
    signal, 1135

socket, 1497
stat, 747
struct, 1265
sys, 26
tipos, 111
urllib.request, 1543
módulo de extensão, **2350**
modulus (*atributo sys.hash_info*), 2035
MON_1 (*no módulo locale*), 1653
MON_2 (*no módulo locale*), 1653
MON_3 (*no módulo locale*), 1653
MON_4 (*no módulo locale*), 1653
MON_5 (*no módulo locale*), 1653
MON_6 (*no módulo locale*), 1653
MON_7 (*no módulo locale*), 1653
MON_8 (*no módulo locale*), 1653
MON_9 (*no módulo locale*), 1653
MON_10 (*no módulo locale*), 1653
MON_11 (*no módulo locale*), 1653
MON_12 (*no módulo locale*), 1653
MONDAY (*no módulo calendar*), 272
monotonic() (*no módulo time*), 798
monotonic_ns() (*no módulo time*), 798
month
calendar opção de linha de comando, 275
month (*atributo calendar.IllegalMonthError*), 273
month (*atributo datetime.date*), 231
month (*atributo datetime.datetime*), 238
Month (*classe em calendar*), 273
month() (*no módulo calendar*), 272
month_abbr (*no módulo calendar*), 272
month_name (*no módulo calendar*), 272
monthcalendar() (*no módulo calendar*), 271
monthdatescalendar() (*método calendar.Calendar*), 268
monthdays2calendar() (*método calendar.Calendar*), 268
monthdayscalendar() (*método calendar.Calendar*), 268
monthrange() (*no módulo calendar*), 271
--months
calendar opção de linha de comando, 275
Morsel (*classe em http.cookies*), 1598
most_common() (*método collections.Counter*), 279
mouseinterval() (*no módulo curses*), 987
mousemask() (*no módulo curses*), 987
move() (*método curses.panel.Panel*), 1015
move() (*método curses.window*), 995
move() (*método mmap.mmap*), 1329
move() (*método pathlib.Path*), 494
move() (*método tkinter.ttk.Treeview*), 1695
move() (*no módulo shutil*), 529
move_into() (*método pathlib.Path*), 495
move_to_end() (*método collections.OrderedDict*), 291
MozillaCookieJar (*classe em http.cookiejar*), 1604
MRO, **2356**
msg (*atributo http.client.HTTPResponse*), 1548
msg (*atributo json.JSONDecodeError*), 1398
msg (*atributo netrc.NetrcParseError*), 688
msg (*atributo re.PatternError*), 160
msg (*atributo tomllib.TOMLDecodeError*), 687
msg (*atributo traceback.TracebackException*), 2111
msilib
module, 2339
msvcrt
module, 2291
mtime (*atributo gzip.GzipFile*), 616
mtime (*atributo tarfile.TarInfo*), 649
mtime() (*método urllib.robotparser.RobotFileParser*), 1538
mul() (*no módulo operator*), 465
Mult (*classe em ast*), 2213
MultiCall (*classe em xmlrpc.client*), 1614
MULTILINE (*no módulo re*), 155
MultilineContinuationError, 686
multimode() (*no módulo statistics*), 424
MultipartConversionError, 1355
MultipartInvariantViolationDefect, 1356
multiply() (*método decimal.Context*), 391
multiprocessing
module, 1038
multiprocessing.connection
module, 1071
multiprocessing.dummy
module, 1075
multiprocessing.Manager()
built-in function, 1062
multiprocessing.managers
module, 1062
multiprocessing.pool
module, 1068
multiprocessing.shared_memory
module, 1084
multiprocessing.sharedctypes
module, 1060
MutableMapping (*classe em collections.abc*), 298
MutableMapping (*classe em typing*), 1806
MutableSequence (*classe em collections.abc*), 297
MutableSequence (*classe em typing*), 1806
MutableSet (*classe em collections.abc*), 298
MutableSet (*classe em typing*), 1806
mutável, **2356**
sequência tipos, 51
mvderwin() (*método curses.window*), 995
mvwin() (*método curses.window*), 995
myrights() (*método imaplib.IMAP4*), 1564

N

-N
uuid opção de linha de comando, 1579
-n
timeit opção de linha de comando, 1986
uuid opção de linha de comando, 1579

webbrowser opção de linha de comando, 1498
N_TOKENS (*no módulo token*), 2252
n_waiting (*atributo asyncio.Barrier*), 1174
n_waiting (*atributo threading.Barrier*), 1037
NAK (*no módulo curses.ascii*), 1012
--name
uuid opção de linha de comando, 1579
name (*atributo AttributeError*), 117
name (*atributo bz2.BZ2File*), 620
name (*atributo codecs.CodecInfo*), 205
name (*atributo compression.zstd.ZstdFile*), 601
name (*atributo contextvars.ContextVar*), 1129
name (*atributo ctypes.CField*), 901
name (*atributo doctest.DocTest*), 1829
name (*atributo email.headerregistry.BaseHeader*), 1357
name (*atributo enum.Enum*), 340
name (*atributo gzip.GzipFile*), 616
name (*atributo hashlib.hash*), 695
name (*atributo hmac.HMAC*), 706
name (*atributo http.cookiejar.Cookie*), 1607
name (*atributo ImportError*), 117
name (*atributo importlib.abc.FileLoader*), 2174
name (*atributo importlib.abc.Traversable*), 2176
name (*atributo importlib.machinery.AppleFrameworkLoader*), 2182
name (*atributo importlib.machinery.ExtensionFileLoader*), 2180
name (*atributo importlib.machinery.ModuleSpec*), 2181
name (*atributo importlib.machinery.SourceFileLoader*), 2179
name (*atributo importlib.machinery.SourcelessFileLoader*), 2180
name (*atributo importlib.resources.abc.Traversable*), 2192
name (*atributo inspect.Parameter*), 2132
name (*atributo io.FileIO*), 788
name (*atributo logging.Logger*), 809
name (*atributo lzma.LZMAFile*), 625
name (*atributo multiprocessing.Process*), 1047
name (*atributo multiprocessing.shared_memory.SharedMemory*), 1086
name (*atributo NameError*), 118
name (*atributo os.DirEntry*), 746
name (*atributo pathlib.PurePath*), 478
name (*atributo pyclbr.Class*), 2259
name (*atributo pyclbr.Function*), 2258
name (*atributo sys.thread_info*), 2047
name (*atributo tarfile.TarInfo*), 649
name (*atributo tempfile.TemporaryDirectory*), 516
name (*atributo threading.Thread*), 1029
name (*atributo traceback.FrameSummary*), 2113
name (*atributo webbrowser.controller*), 1500
name (*atributo xml.dom.Attr*), 1465
name (*atributo xml.dom.DocumentType*), 1463
name (*atributo zipfile.Path*), 636
Name (*classe em ast*), 2212
name (*no módulo os*), 712
NAME (*no módulo token*), 2247
name() (*no módulo unicodedata*), 187
name2codepoint (*no módulo html.entities*), 1438
NAMED_FLAGS (*atributo enum.EnumCheck*), 348
NamedExpr (*classe em ast*), 2215
NamedTemporaryFile() (*no módulo tempfile*), 515
NamedTuple (*classe em typing*), 1787
namedtuple() (*no módulo collections*), 287
NameError, 118
namelist() (*método zipfile.ZipFile*), 632
nameprep() (*no módulo encodings.idna*), 221
namer (*atributo logging.handlers.BaseRotatingHandler*), 841
namereplace
error handler's name, 208
namereplace_errors() (*no módulo codecs*), 209
names() (*no módulo tkinter.font*), 1676
--namespace
uuid opção de linha de comando, 1579
Namespace (*classe em argparse*), 928
Namespace (*classe em multiprocessing.managers*), 1064
namespace() (*método imaplib.IMAP4*), 1564
Namespace() (*método multiprocessing.managers.SyncManager*), 1064
NAMESPACE_DNS (*no módulo uuid*), 1578
NAMESPACE_OID (*no módulo uuid*), 1578
NAMESPACE_URL (*no módulo uuid*), 1578
NAMESPACE_X500 (*no módulo uuid*), 1578
NamespaceErr, 1466
NamespaceLoader (*classe em importlib.machinery*), 2181
namespaceURI (*atributo xml.dom.Node*), 1461
nametofont() (*no módulo tkinter.font*), 1676
NaN, 17
nan (*atributo sys.hash_info*), 2035
nan (*no módulo cmath*), 373
nan (*no módulo math*), 369
nanj (*no módulo cmath*), 374
NannyNag, 2257
napms() (*no módulo curses*), 987
nargs (*atributo optparse.Option*), 966
native_id (*atributo threading.Thread*), 1029
nb_workers (*atributo compression.zstd.CompressionParameter*), 608
nbytes (*atributo memoryview*), 94
ncurses_version (*no módulo curses*), 998
ND (*atributo inspect.BufferFlags*), 2142
ndiff() (*no módulo difflib*), 173
ndim (*atributo memoryview*), 95
ne() (*no módulo operator*), 463
needs_input (*atributo bz2.BZ2Decompressor*), 622
needs_input (*atributo compression.zstd.ZstdDecompressor*), 603
needs_input (*atributo lzma.LZMADecompressor*), 627
neg() (*no módulo operator*), 465
nested_scopes (*no módulo __future__*), 2118
netmask (*atributo ipaddress.IPv4Network*), 1630

netmask (*atributo ipaddress.IPv6Network*), 1633
NetmaskValueError, 1636
netrc
    module, 688
netrc (*classe em netrc*), 688
NetrcParseError, 688
netscape (*atributo http.cookiejar.CookiePolicy*), 1605
network (*atributo ipaddress.IPv4Interface*), 1634
network (*atributo ipaddress.IPv6Interface*), 1635
network_address (*atributo ipaddress.IPv4Network*), 1630
network_address (*atributo ipaddress.IPv6Network*), 1633
Never (*no módulo typing*), 1766
NEVER_EQ (*no módulo test.support*), 1935
new() (*no módulo hashlib*), 694
new() (*no módulo hmac*), 705
new_child() (*método collections.ChainMap*), 276
new_class() (*no módulo types*), 320
new_event_loop() (*método asyncio.AbstractEventLoopPolicy*), 1227
new_event_loop() (*no módulo asyncio*), 1185
new_panel() (*no módulo curses.panel*), 1014
NEWLINE (*no módulo token*), 2248
newlines (*atributo io.TextIOBase*), 791
newpad() (*no módulo curses*), 987
--new-tab
    webbrowser opção de linha de comando, 1498
NewType (*classe em typing*), 1788
newwin() (*no módulo curses*), 988
--new-window
    webbrowser opção de linha de comando, 1498
next (*pdb command*), 1971
next()
    built-in function, 23
next() (*método tarfile.TarFile*), 646
next() (*método tkinter.ttk.Treeview*), 1695
next_minus() (*método decimal.Context*), 391
next_minus() (*método decimal.Decimal*), 383
next_plus() (*método decimal.Context*), 391
next_plus() (*método decimal.Decimal*), 383
next_toward() (*método decimal.Context*), 391
next_toward() (*método decimal.Decimal*), 383
nextafter() (*no módulo math*), 364
nextfile() (*no módulo fileinput*), 982
nextkey() (*método dbm.gnu.gdbm*), 563
nextSibling (*atributo xml.dom.Node*), 1461
ngettext() (*método gettext.GNUTranslations*), 1647
ngettext() (*método gettext.NullTranslations*), 1646
ngettext() (*no módulo gettext*), 1644
nice() (*no módulo os*), 766
NIL (*no módulo uuid*), 1578
nis
    module, 2339
NL (*no módulo curses.ascii*), 1011
NL (*no módulo token*), 2248
nl() (*no módulo curses*), 988
nl_langinfo() (*no módulo locale*), 1653
nlargest() (*no módulo heapq*), 302
nlst() (*método ftplib.FTP*), 1554
nntplib
    module, 2339
NO (*no módulo tkinter.messagebox*), 1681
no_animation() (*no módulo turtle*), 1739
no_cache() (*método de classe zoneinfo.ZoneInfo*), 264
NO_EVENTS (*monitoring event*), 2051
no_proxy, 1515
no_site (*atributo sys.flags*), 2028
no_tracing() (*no módulo test.support*), 1939
no_type_check() (*no módulo typing*), 1799
no_type_check_decorator() (*no módulo typing*), 1799
no_user_site (*atributo sys.flags*), 2028
NoBoundaryInMultipartDefect, 1355
nocbreak() (*no módulo curses*), 988
NoDataAllowedErr, 1467
node (*atributo uuid.UUID*), 1576
node() (*no módulo platform*), 853
NoDefault (*no módulo typing*), 1802
nodelay() (*método curses.window*), 995
nodeName (*atributo xml.dom.Node*), 1461
NodeTransformer (*classe em ast*), 2240
nodeType (*atributo xml.dom.Node*), 1461
nodeValue (*atributo xml.dom.Node*), 1461
NodeVisitor (*classe em ast*), 2239
noecho() (*no módulo curses*), 988
--no-ensure-ascii
    json opção de linha de comando, 1401
NOEXPR (*no módulo locale*), 1654
NOFLAG (*no módulo re*), 156
--no-indent
    json opção de linha de comando, 1401
nome qualificado, **2359**
Nomeada, memória compartilhada, 1084
nomes de arquivos
    expansão de curingas, 522
    expansão de nome de arquivo, 520
NoModificationAllowedErr, 1467
--nonaliased
    platform opção de linha de comando, 857
NonCallableMagicMock (*classe em unittest.mock*), 1899
NonCallableMock (*classe em unittest.mock*), 1879
None (*Objeto embutido*), 39
None (*variável interna*), 37
NoneType (*no módulo types*), 321
nonl() (*no módulo curses*), 988
Nonlocal (*classe em ast*), 2235
nonmember() (*no módulo enum*), 351
noop() (*método imaplib.IMAP4*), 1564
noop() (*método poplib.POP3*), 1559
NoOptionError, 685
NOP (*opcode*), 2272

noqiflush() (*no módulo curses*), 988
noraw() (*no módulo curses*), 988
--no-report
    trace opção de linha de comando, 1990
NoReturn (*no módulo typing*), 1766
NORMAL (*no módulo tkinter.font*), 1675
NORMAL_PRIORITY_CLASS (*no módulo subprocess*), 1117
NormalDist (*classe em statistics*), 429
normalize() (*método decimal.Context*), 391
normalize() (*método decimal.Decimal*), 384
normalize() (*método xml.dom.Node*), 1462
normalize() (*no módulo locale*), 1656
normalize() (*no módulo unicodedata*), 189
normalize_encoding() (*no módulo encodings*), 220
NORMALIZE_WHITESPACE (*no módulo doctest*), 1822
normalvariate() (*no módulo random*), 412
normcase() (*no módulo os.path*), 503
normpath() (*no módulo os.path*), 503
NoSectionError, 685
NoSuchMailboxError, 1419
not
    operador, 40
Not (*classe em ast*), 2213
not in
    operador, 40, 49
not_() (*no módulo operator*), 464
NOT_TAKEN (*opcode*), 2272
NotADirectoryError, 124
notationDecl() (*método xml.sax.handler.DTDHandler*), 1480
NotationDeclHandler() (*método xml.parsers.expat.xmlparser*), 1491
notations (*atributo xml.dom.DocumentType*), 1463
NotConnected, 1544
Notebook (*classe em tkinter.ttk*), 1689
NotEmptyError, 1419
NotEq (*classe em ast*), 2214
NOTEQUAL (*no módulo token*), 2250
NotFoundErr, 1467
notify() (*método asyncio.Condition*), 1171
notify() (*método threading.Condition*), 1034
notify_all() (*método asyncio.Condition*), 1171
notify_all() (*método threading.Condition*), 1034
notimeout() (*método curses.window*), 995
NotImplemented (*variável interna*), 37
NotImplementedError, 118
NotImplementedType (*no módulo types*), 322
NotIn (*classe em ast*), 2214
NotRequired (*no módulo typing*), 1772
NOTSET (*no módulo logging*), 814
NotShareableError, 1103
NotStandaloneHandler() (*método xml.parsers.expat.xmlparser*), 1491
NotSupportedErr, 1467
NotSupportedError, 587
--no-type-comments
    ast opção de linha de comando, 2242
noutrefresh() (*método curses.window*), 995
novas linhas universais, **2363**
    função csv.reader, 659
    função embutida open(), 25
    io.IncrementalNewlineDecoder class, 793
    io.TextIOWrapper class, 792
    método bytearray.splitlines, 83
    método bytes.splitlines, 83
    Método importlib.abc.InspectLoader.get_source, 2173
    método str.splitlines, 65
    subprocess module, 1107
NOVEMBER (*no módulo calendar*), 273
now() (*método de classe datetime.datetime*), 235
npgettext() (*método gettext.GNUTranslations*), 1647
npgettext() (*método gettext.NullTranslations*), 1646
npgettext() (*no módulo gettext*), 1644
NSIG (*no módulo signal*), 1319
nsmallest() (*no módulo heapq*), 302
NTEventLogHandler (*classe em logging.handlers*), 847
ntohl() (*no módulo socket*), 1256
ntohs() (*no módulo socket*), 1256
ntransfercmd() (*método ftplib.FTP*), 1554
NUL (*no módulo curses.ascii*), 1011
nullcontext() (*no módulo contextlib*), 2089
NullHandler (*classe em logging*), 840
NullTranslations (*classe em gettext*), 1645
num_addresses (*atributo ipaddress.IPv4Network*), 1630
num_addresses (*atributo ipaddress.IPv6Network*), 1633
num_tickets (*atributo ssl.SSLContext*), 1294
--number
    timeit opção de linha de comando, 1986
Number (*classe em numbers*), 357
NUMBER (*no módulo token*), 2247
number_class() (*método decimal.Context*), 391
number_class() (*método decimal.Decimal*), 384
numbers
    module, 357
numerator (*atributo fractions.Fraction*), 405
numerator (*atributo numbers.Rational*), 358
numeric() (*no módulo unicodedata*), 188
numérico
    conversões, 42
    literais, 41
    objeto, 40
    tipos, operações em, 41
número complexo, **2348**
    literais, 41
    objeto, 40
numinput() (*no módulo turtle*), 1742

O

-O
    ast opção de linha de comando, 2242
    dis opção de linha de comando, 2266

-o
    compileall opção de linha de comando, 2262
    cProfile opção de linha de comando, 1977
    doctest opção de linha de comando, 1817
    pickletools opção de linha de comando, 2289
    zipapp opção de linha de comando, 2015
O_APPEND (*no módulo os*), 726
O_ASYNC (*no módulo os*), 727
O_BINARY (*no módulo os*), 727
O_CLOEXEC (*no módulo os*), 727
O_CREAT (*no módulo os*), 726
O_DIRECT (*no módulo os*), 727
O_DIRECTORY (*no módulo os*), 727
O_DSYNC (*no módulo os*), 727
O_EVTONLY (*no módulo os*), 727
O_EXCL (*no módulo os*), 726
O_EXLOCK (*no módulo os*), 727
O_FSYNC (*no módulo os*), 727
O_NDELAY (*no módulo os*), 727
O_NOATIME (*no módulo os*), 727
O_NOCTTY (*no módulo os*), 727
O_NOFOLLOW (*no módulo os*), 727
O_NOFOLLOW_ANY (*no módulo os*), 727
O_NOINHERIT (*no módulo os*), 727
O_NONBLOCK (*no módulo os*), 727
O_PATH (*no módulo os*), 727
O_RANDOM (*no módulo os*), 727
O_RDONLY (*no módulo os*), 726
O_RDWR (*no módulo os*), 726
O_RSYNC (*no módulo os*), 727
O_SEQUENTIAL (*no módulo os*), 727
O_SHLOCK (*no módulo os*), 727
O_SHORT_LIVED (*no módulo os*), 727
O_SYMLINK (*no módulo os*), 727
O_SYNC (*no módulo os*), 727
O_TEMPORARY (*no módulo os*), 727
O_TEXT (*no módulo os*), 727
O_TMPFILE (*no módulo os*), 727
O_TRUNC (*no módulo os*), 726
O_WRONLY (*no módulo os*), 726
obj (*atributo AttributeError*), 117
obj (*atributo memoryview*), 93
object (*atributo UnicodeError*), 122
object (*classe interna*), 23
objeto, **2357**
    Booleano, 40
    bytearray, 51, 71, 72
    bytes, 71
    código, 111, 558
    dicionário, 98
    GenericAlias, 104
    inteiro, 40
    io.StringIO, 57
    lista, 51, 52
    mapeamento, 98
    memoryview, 71
    método, 110
    numérico, 40
    número complexo, 40
    ponto flutuante, 40
    range, 54
    sequência, 49
    set, 95
    socket, 1241
    string, 55
    tipo, 32
    traceback, 2026, 2107
    tupla, 51, 53
    União, 108
objeto arquivo, **2351**
    função embutida open(), 24
    io module, 781
objeto arquivo ou similar, **2351**
objeto bytes ou similar, **2347**
objeto caminho ou similar, **2358**
objeto código, 111, 558
objetos
    comparando, 40
    flattening, 537
    marshalling, 537
    persistente, 537
    pickling, 537
    serialização, 537
oct()
    built-in function, 23
octal
    literais, 41
octdigits (*no módulo string*), 130
OCTOBER (*no módulo calendar*), 273
offset (*atributo ctypes.CField*), 902
offset (*atributo SyntaxError*), 121
offset (*atributo tarfile.TarInfo*), 650
offset (*atributo traceback.TracebackException*), 2111
offset (*atributo xml.parsers.expat.ExpatError*), 1492
offset_data (*atributo tarfile.TarInfo*), 650
OK (*no módulo curses*), 997
OK (*no módulo tkinter.messagebox*), 1680
ok_command() (*método tkinter.filedialog.LoadFileDialog*), 1678
ok_command() (*método tkinter.filedialog.SaveFileDialog*), 1678
ok_event() (*método tkinter.filedialog.FileDialog*), 1678
OKCANCEL (*no módulo tkinter.messagebox*), 1681
old_value (*atributo contextvars.Token*), 1130
OleDLL (*classe em ctypes*), 886
on_motion() (*método tkinter.dnd.DndHandler*), 1682
on_release() (*método tkinter.dnd.DndHandler*), 1682
onclick() (*no módulo turtle*), 1741
ondrag() (*no módulo turtle*), 1735
onecmd() (*método cmd.Cmd*), 1016
onkey() (*no módulo turtle*), 1740

onkeypress() (*no módulo turtle*), 1740
onkeyrelease() (*no módulo turtle*), 1740
onrelease() (*no módulo turtle*), 1735
onscreenclick() (*no módulo turtle*), 1741
ontimer() (*no módulo turtle*), 1741
OP (*no módulo token*), 2248
OP_ALL (*no módulo ssl*), 1278
OP_CIPHER_SERVER_PREFERENCE (*no módulo ssl*), 1279
OP_ENABLE_KTLS (*no módulo ssl*), 1280
OP_ENABLE_MIDDLEBOX_COMPAT (*no módulo ssl*), 1279
OP_IGNORE_UNEXPECTED_EOF (*no módulo ssl*), 1280
OP_LEGACY_SERVER_CONNECT (*no módulo ssl*), 1280
OP_NO_COMPRESSION (*no módulo ssl*), 1279
OP_NO_RENEGOTIATION (*no módulo ssl*), 1279
OP_NO_SSLv2 (*no módulo ssl*), 1278
OP_NO_SSLv3 (*no módulo ssl*), 1278
OP_NO_TICKET (*no módulo ssl*), 1280
OP_NO_TLSv1 (*no módulo ssl*), 1278
OP_NO_TLSv1_1 (*no módulo ssl*), 1279
OP_NO_TLSv1_2 (*no módulo ssl*), 1279
OP_NO_TLSv1_3 (*no módulo ssl*), 1279
OP_SINGLE_DH_USE (*no módulo ssl*), 1279
OP_SINGLE_ECDH_USE (*no módulo ssl*), 1279
Opções de Tipos de Dados do Tk, 1671
Open (*classe em tkinter.filedialog*), 1677
open()
    built-in function, 24
open() (*método de classe tarfile.TarFile*), 646
open() (*método imaplib.IMAP4*), 1564
open() (*método importlib.abc.Traversable*), 2176
open() (*método importlib.resources.abc.Traversable*), 2193
open() (*método pathlib.Path*), 489
open() (*método urllib.request.OpenerDirector*), 1518
open() (*método webbrowser.controller*), 1500
open() (*método zipfile.Path*), 636
open() (*método zipfile.ZipFile*), 632
open() (*no módulo bz2*), 619
open() (*no módulo codecs*), 206
open() (*no módulo compression.zstd*), 600
open() (*no módulo dbm*), 560
open() (*no módulo dbm.dumb*), 565
open() (*no módulo dbm.gnu*), 562
open() (*no módulo dbm.ndbm*), 564
open() (*no módulo dbm.sqlite3*), 562
open() (*no módulo gzip*), 615
open() (*no módulo io*), 782
open() (*no módulo lzma*), 623
open() (*no módulo os*), 726
open() (*no módulo shelve*), 555
open() (*no módulo tarfile*), 641
open() (*no módulo tokenize*), 2254
open() (*no módulo wave*), 1639
open() (*no módulo webbrowser*), 1498
open_binary() (*no módulo importlib.resources*), 2189
open_code() (*no módulo io*), 782
open_connection() (*no módulo asyncio*), 1162
open_flags (*no módulo dbm.gnu*), 563
open_new() (*método webbrowser.controller*), 1500
open_new() (*no módulo webbrowser*), 1498
open_new_tab() (*método webbrowser.controller*), 1500
open_new_tab() (*no módulo webbrowser*), 1498
open_osfhandle() (*no módulo msvcrt*), 2292
open_resource() (*método importlib.abc.ResourceReader*), 2175
open_resource() (*método importlib.resources.abc.ResourceReader*), 2192
open_text() (*no módulo importlib.resources*), 2190
open_unix_connection() (*no módulo asyncio*), 1163
open_urlresource() (*no módulo test.support*), 1940
OpenerDirector (*classe em urllib.request*), 1514
OpenKey() (*no módulo winreg*), 2297
OpenKeyEx() (*no módulo winreg*), 2297
openlog() (*no módulo syslog*), 2327
openpty() (*no módulo os*), 727
openpty() (*no módulo pty*), 2318
OpenSSL
    (use in module hashlib), 693
    (use in module ssl), 1270
OPENSSL_VERSION (*no módulo ssl*), 1281
OPENSSL_VERSION_INFO (*no módulo ssl*), 1281
OPENSSL_VERSION_NUMBER (*no módulo ssl*), 1282
operação
    concatenação, 49
    fatia, 49
    repetição, 49
    subscrição, 49
operações
    bit a bit, 42
    Booleano, 39
    deslocamento, 42
    mascaramento, 42
operações em
    dicionário tipo, 98
    inteiro tipos, 42
    lista tipo, 51
    mapeamento tipos, 98
    numérico tipos, 41
    sequência tipos, 49, 51
operador
    - (*menos*), 41
    % (*porcentagem*), 41
    & (*e comercial*), 42
    * (*asterisco*), 41
    **, 41
    + (*mais*), 41
    / (*barra*), 41
    //, 41
    < (*menor que*), 40
    <<, 42
    <=, 40
    !=, 40

==, 40
> (*maior que*), 40
>=, 40
>>, 42
^ (*circunflexo*), 42
| (*barra vertical*), 42
~ (*til*), 42
and, 39, 40
comparação, 40
in, 40, 49
is, 40
is not, 40
not, 40
not in, 40, 49
or, 39, 40
operador morsa, **2363**
OperationalError, 587
operator
module, 463
opmap (*no módulo dis*), 2288
opname (*no módulo dis*), 2288
optim_args_from_interpreter_flags() (*no módulo test.support*), 1937
--optimize
ast opção de linha de comando, 2242
optimize (*atributo sys.flags*), 2028
optimize() (*no módulo pickletools*), 2290
OPTIMIZED_BYTECODE_SUFFIXES (*no módulo importlib.machinery*), 2177
--option
doctest opção de linha de comando, 1817
Option (*classe em optparse*), 966
Optional (*no módulo typing*), 1769
OptionConflictError, 979
OptionError, 979
OptionGroup (*classe em optparse*), 960
OptionParser (*classe em optparse*), 963
options (*atributo doctest.Example*), 1830
options (*atributo ssl.SSLContext*), 1294
Options (*classe em ssl*), 1280
options() (*método configparser.ConfigParser*), 682
OptionValueError, 980
optionxform() (*método configparser.ConfigParser*), 684
optparse
module, 951
or
operador, 39, 40
Or (*classe em ast*), 2214
or_() (*no módulo operator*), 465
ord()
built-in function, 26
ordem de resolução de métodos, **2356**
ordered_attributes (*atributo xml.parsers.expat.xmlparser*), 1489
OrderedDict (*classe em collections*), 291
OrderedDict (*classe em typing*), 1804
orig_argv (*no módulo sys*), 2039
origin (*atributo importlib.machinery.ModuleSpec*), 2181
origin_req_host (*atributo urllib.request.Request*), 1516
origin_server (*atributo wsgiref.handlers.BaseHandler*), 1508
os
module, 711
módulo, 2312
os_environ (*atributo wsgiref.handlers.BaseHandler*), 1507
OSError, 119
os.path
module, 499
ossaudiodev
module, 2340
OUT_TO_DEFAULT (*no módulo msvcrt*), 2293
OUT_TO_MSGBOX (*no módulo msvcrt*), 2293
OUT_TO_STDERR (*no módulo msvcrt*), 2293
outfile
json opção de linha de comando, 1401
--output
pickletools opção de linha de comando, 2289
zipapp opção de linha de comando, 2015
output (*atributo subprocess.CalledProcessError*), 1107
output (*atributo subprocess.TimeoutExpired*), 1106
output (*atributo unittest.TestCase*), 1852
output() (*método http.cookies.BaseCookie*), 1597
output() (*método http.cookies.Morsel*), 1599
output_charset (*atributo email.charset.Charset*), 1385
output_codec (*atributo email.charset.Charset*), 1385
output_difference() (*método doctest.OutputChecker*), 1833
OutputChecker (*classe em doctest*), 1833
OutputString() (*método http.cookies.Morsel*), 1599
OutsideDestinationError, 644
Overflow (*classe em decimal*), 395
OverflowError, 119
overlap() (*método statistics.NormalDist*), 430
overlap_log (*atributo compression.zstd.CompressionParameter*), 608
overlaps() (*método ipaddress.IPv4Network*), 1630
overlaps() (*método ipaddress.IPv6Network*), 1633
overlay() (*método curses.window*), 996
overload() (*no módulo typing*), 1797
override() (*no módulo typing*), 1799
overwrite() (*método curses.window*), 996
owner() (*método pathlib.Path*), 495

P

-p
dis opção de linha de comando, 2266
-p
compileall opção de linha de comando, 2262

http.server opção de linha de comando, 1596
pdb opção de linha de comando, 1965
pickletools opção de linha de comando, 2289
timeit opção de linha de comando, 1986
unittest-discover opção de linha de comando, 1841
zipapp opção de linha de comando, 2015
p (*pdb command*), 1972
P_ALL (*no módulo os*), 773
P_DETACH (*no módulo os*), 770
P_NOWAIT (*no módulo os*), 770
P_NOWAITO (*no módulo os*), 770
P_OVERLAY (*no módulo os*), 770
P_PGID (*no módulo os*), 773
P_PID (*no módulo os*), 773
P_PIDFD (*no módulo os*), 773
P_WAIT (*no módulo os*), 770
pack() (*método mailbox.MH*), 1409
pack() (*método struct.Struct*), 204
pack() (*no módulo struct*), 198
pack_into() (*método struct.Struct*), 204
pack_into() (*no módulo struct*), 198
PackageMetadata (*classe em importlib.metadata*), 2196
PackageNotFoundError, 2194
PackagePath (*classe em importlib.metadata*), 2197
packages_distributions() (*no módulo importlib.metadata*), 2198
packed (*atributo ipaddress.IPv4Address*), 1624
packed (*atributo ipaddress.IPv6Address*), 1627
packing
binário dados, 197
pacote, 2152, **2357**
pacote de espaço de nomes, **2357**
pacote provisório, **2359**
pacote regular, **2360**
PAGER, 1810
pair_content() (*no módulo curses*), 988
pair_number() (*no módulo curses*), 988
pairwise() (*no módulo itertools*), 442
Parameter (*classe em inspect*), 2132
ParameterizedMIMEHeader (*classe em email.headerregistry*), 1359
parameters (*atributo inspect.Signature*), 2131
parâmetro, **2357**
params (*atributo email.headerregistry.ParameterizedMIMEHeader*), 1359
ParamSpec (*classe em ast*), 2232
ParamSpec (*classe em typing*), 1783
ParamSpecArgs (*no módulo typing*), 1785
ParamSpecKwargs (*no módulo typing*), 1785
paramstyle (*no módulo sqlite3*), 571
pardir (*no módulo os*), 778
parent (*atributo importlib.machinery.ModuleSpec*), 2182
parent (*atributo logging.Logger*), 809
parent (*atributo pathlib.PurePath*), 477
parent (*atributo pyclbr.Class*), 2259
parent (*atributo pyclbr.Function*), 2258
parent (*atributo urllib.request.BaseHandler*), 1519
parent() (*método tkinter.ttk.Treeview*), 1695
parent_process() (*no módulo multiprocessing*), 1053
parentNode (*atributo xml.dom.Node*), 1461
parents (*atributo collections.ChainMap*), 276
parents (*atributo pathlib.PurePath*), 477
paretovariate() (*no módulo random*), 412
parse() (*método doctest.DocTestParser*), 1831
parse() (*método email.parser.BytesParser*), 1343
parse() (*método email.parser.Parser*), 1344
parse() (*método string.Formatter*), 130
parse() (*método urllib.robotparser.RobotFileParser*), 1538
parse() (*método xml.etree.ElementTree.ElementTree*), 1454
Parse() (*método xml.parsers.expat.xmlparser*), 1488
parse() (*método xml.sax.xmlreader.XMLReader*), 1483
parse() (*no módulo ast*), 2237
parse() (*no módulo xml.dom.minidom*), 1469
parse() (*no módulo xml.dom.pulldom*), 1474
parse() (*no módulo xml.etree.ElementTree*), 1448
parse() (*no módulo xml.sax*), 1475
parse_and_bind() (*no módulo readline*), 192
parse_args() (*método argparse.ArgumentParser*), 925
parse_args() (*método optparse.OptionParser*), 970
PARSE_COLNAMES (*no módulo sqlite3*), 570
parse_config_h() (*no módulo sysconfig*), 2060
PARSE_DECLTYPES (*no módulo sqlite3*), 570
parse_headers() (*no módulo http.client*), 1544
parse_intermixed_args() (*método argparse.ArgumentParser*), 936
parse_known_args() (*método argparse.ArgumentParser*), 935
parse_known_intermixed_args() (*método argparse.ArgumentParser*), 936
parse_qs() (*no módulo urllib.parse*), 1530
parse_qsl() (*no módulo urllib.parse*), 1531
parseaddr() (*no módulo email.utils*), 1388
parsebytes() (*método email.parser.BytesParser*), 1343
parsedate() (*no módulo email.utils*), 1388
parsedate_to_datetime() (*no módulo email.utils*), 1389
parsedate_tz() (*no módulo email.utils*), 1388
ParseError (*classe em xml.etree.ElementTree*), 1458
ParseFile() (*método xml.parsers.expat.xmlparser*), 1488
ParseFlags() (*no módulo imaplib*), 1561
parser (*atributo pathlib.PurePath*), 476
Parser (*classe em email.parser*), 1344
ParserCreate() (*no módulo xml.parsers.expat*), 1487
ParseResult (*classe em urllib.parse*), 1535
ParseResultBytes (*classe em urllib.parse*), 1535
parsestr() (*método email.parser.Parser*), 1344

parseString() (*no módulo xml.dom.minidom*), 1469
parseString() (*no módulo xml.dom.pulldom*), 1474
parseString() (*no módulo xml.sax*), 1475
parsing
    URL, 1528
ParsingError, 686
partial (*atributo asyncio.IncompleteReadError*), 1182
partial() (*método imaplib.IMAP4*), 1565
partial() (*no módulo functools*), 456
partialmethod (*classe em functools*), 457
parties (*atributo asyncio.Barrier*), 1174
parties (*atributo threading.Barrier*), 1037
partition() (*método bytearray*), 76
partition() (*método bytes*), 76
partition() (*método str*), 63
partitioned (*atributo http.cookies.Morsel*), 1598
parts (*atributo pathlib.PurePath*), 476
Pass (*classe em ast*), 2220
pass_() (*método poplib.POP3*), 1558
Paste, 1705
patch() (*no módulo test.support*), 1941
patch() (*no módulo unittest.mock*), 1889
patch.dict() (*no módulo unittest.mock*), 1892
patch.multiple() (*no módulo unittest.mock*), 1894
patch.object() (*no módulo unittest.mock*), 1892
patch.stopall() (*no módulo unittest.mock*), 1896
PATH, 497, 530, 762, 763, 768, 769, 779, 1108, 1497, 2007, 2008, 2152
path (*atributo http.cookiejar.Cookie*), 1607
path (*atributo http.cookies.Morsel*), 1598
path (*atributo http.server.BaseHTTPRequestHandler*), 1591
path (*atributo ImportError*), 118
path (*atributo importlib.abc.FileLoader*), 2174
path (*atributo importlib.machinery.AppleFrameworkLoader*), 2182
path (*atributo importlib.machinery.ExtensionFileLoader*), 2180
path (*atributo importlib.machinery.FileFinder*), 2179
path (*atributo importlib.machinery.SourceFileLoader*), 2179
path (*atributo importlib.machinery.SourcelessFileLoader*), 2180
path (*atributo os.DirEntry*), 746
Path (*classe em pathlib*), 483
Path (*classe em zipfile*), 635
path (*no módulo sys*), 2039
Path browser, 1701
path() (*no módulo importlib.resources*), 2190
path_hook() (*método de classe importlib.machinery.FileFinder*), 2179
path_hooks (*no módulo sys*), 2040
path_importer_cache (*no módulo sys*), 2040
path_mtime() (*método importlib.abc.SourceLoader*), 2174
path_return_ok() (*método http.cookiejar.CookiePolicy*), 1605
path_stats() (*método importlib.abc.SourceLoader*), 2174
path_stats() (*método importlib.machinery.SourceFileLoader*), 2179
pathconf() (*no módulo os*), 742
pathconf_names (*no módulo os*), 742
PathEntryFinder (*classe em importlib.abc*), 2171
PATHEXT, 530
PathFinder (*classe em importlib.machinery*), 2178
PathInfo (*classe em pathlib.types*), 498
pathlib
    module, 471
pathlib.types
    module, 498
PathLike (*classe em os*), 715
pathname2url() (*no módulo urllib.request*), 1512
pathsep (*no módulo os*), 779
Path.stem (*no módulo zipfile*), 636
Path.suffix (*no módulo zipfile*), 636
Path.suffixes (*no módulo zipfile*), 636
--pattern
    unittest-discover opção de linha de comando, 1841
pattern (*atributo re.Pattern*), 162
pattern (*atributo re.PatternError*), 160
Pattern (*classe em re*), 161
Pattern (*classe em typing*), 1805
PatternError, 160
pause() (*no módulo signal*), 1320
pause_reading() (*método asyncio.ReadTransport*), 1216
pause_writing() (*método asyncio.BaseProtocol*), 1219
PAX_FORMAT (*no módulo tarfile*), 645
pax_headers (*atributo tarfile.TarFile*), 648
pax_headers (*atributo tarfile.TarInfo*), 650
pbkdf2_hmac() (*no módulo hashlib*), 697
pd() (*no módulo turtle*), 1727
pdb
    module, 1964
Pdb (*classe em pdb*), 1967
Pdb (*classe no pdb*), 1964
pdb opção de linha de comando
    -c, 1965
    --command, 1965
    -m, 1965
    -p, 1965
    --pid, 1965
.pdbrc
    arquivo, 1969
pdf() (*método statistics.NormalDist*), 430
peek() (*método bz2.BZ2File*), 619
peek() (*método compression.zstd.ZstdFile*), 601
peek() (*método gzip.GzipFile*), 616
peek() (*método io.BufferedReader*), 789
peek() (*método lzma.LZMAFile*), 624
peek() (*método weakref.finalize*), 315
PEM_cert_to_DER_cert() (*no módulo ssl*), 1275

pen() (*no módulo turtle*), 1727
pencolor() (*no módulo turtle*), 1728
pending (*atributo ssl.MemoryBIO*), 1303
pending() (*método ssl.SSLSocket*), 1286
PendingDeprecationWarning, 124
pendown() (*no módulo turtle*), 1727
pensize() (*no módulo turtle*), 1727
penup() (*no módulo turtle*), 1727
PEP, **2358**
PERCENT (*no módulo token*), 2250
PERCENTEQUAL (*no módulo token*), 2251
perf_counter() (*no módulo time*), 798
perf_counter_ns() (*no módulo time*), 799
perfilador, 2033, 2042
perm() (*no módulo math*), 362
PermissionError, 124
permutations() (*no módulo itertools*), 443
persistência, 537
persistent_id (*protocolo pickle*), 546
persistent_id() (*método pickle.Pickler*), 541
persistent_load (*protocolo pickle*), 546
persistent_load() (*método pickle.Unpickler*), 542
persistente
    objetos, 537
pesquisa
    caminho, módulo, 524, 2039, 2151
PF_CAN (*no módulo socket*), 1247
PF_DIVERT (*no módulo socket*), 1248
PF_PACKET (*no módulo socket*), 1248
PF_RDS (*no módulo socket*), 1248
pformat() (*método pprint.PrettyPrinter*), 329
pformat() (*no módulo pprint*), 328
pgettext() (*método gettext.GNUTranslations*), 1647
pgettext() (*método gettext.NullTranslations*), 1646
pgettext() (*no módulo gettext*), 1644
PGO (*no módulo test.support*), 1934
phase() (*no módulo cmath*), 371
pi (*no módulo cmath*), 373
pi (*no módulo math*), 369
pi() (*método xml.etree.ElementTree.TreeBuilder*), 1455
pickle
    module, 537
    módulo, 326, 554, 555, 558
pickle opção de linha de comando
    pickle_file, 554
pickle() (*no módulo copyreg*), 555
pickle_file
    pickle opção de linha de comando, 554
    pickletools opção de linha de comando, 2289
PickleBuffer (*classe em pickle*), 543
PickleError, 540
Pickler (*classe em pickle*), 540
pickletools
    module, 2288
pickletools opção de linha de comando
    -a, 2289
    --annotate, 2289
    --indentlevel, 2289
    -l, 2289
    -m, 2289
    --memo, 2289
    -o, 2289
    --output, 2289
    -p, 2289
    pickle_file, 2289
    --preamble, 2289
pickling
    objetos, 537
PicklingError, 540
--pid
    pdb opção de linha de comando, 1965
pid (*atributo asyncio.subprocess.Process*), 1178
pid (*atributo multiprocessing.Process*), 1047
pid (*atributo subprocess.Popen*), 1115
PIDFD_NONBLOCK (*no módulo os*), 766
pidfd_open() (*no módulo os*), 766
pidfd_send_signal() (*no módulo signal*), 1321
PIPE (*no módulo subprocess*), 1106
Pipe() (*no módulo multiprocessing*), 1050
pipe() (*no módulo os*), 727
pipe2() (*no módulo os*), 728
PIPE_BUF (*no módulo select*), 1307
pipe_connection_lost() (*método asyncio.SubprocessProtocol*), 1221
pipe_data_received() (*método asyncio.SubprocessProtocol*), 1220
PIPE_MAX_SIZE (*no módulo test.support*), 1934
pipes
    module, 2340
pkgutil
    module, 2161
placeholder (*atributo textwrap.TextWrapper*), 187
Placeholder (*no módulo functools*), 456
platform
    module, 852
platform (*no módulo sys*), 2040
platform opção de linha de comando
    --nonaliased, 857
    --terse, 857
platform() (*no módulo platform*), 853
platlibdir (*no módulo sys*), 2041
PlaySound() (*no módulo winsound*), 2303
plist
    arquivo, 689
plistlib
    module, 689
plock() (*no módulo os*), 766
PLUS (*no módulo token*), 2249
plus() (*método decimal.Context*), 392
PLUSEQUAL (*no módulo token*), 2251
pm() (*no módulo pdb*), 1967
POINTER() (*no módulo ctypes*), 894
pointer() (*no módulo ctypes*), 894
polar() (*no módulo cmath*), 371
Policy (*classe em email.policy*), 1349

poll() (*método multiprocessing.connection.Connection*), 1055
poll() (*método select.devpoll*), 1308
poll() (*método select.epoll*), 1309
poll() (*método select.poll*), 1310
poll() (*método subprocess.Popen*), 1113
poll() (*no módulo select*), 1307
PollSelector (*classe em selectors*), 1315
poly() (*no módulo turtle*), 1736
ponto flutuante
    função embutida, 41
    literais, 41
    objeto, 40
Pool (*classe em multiprocessing.pool*), 1068
pop() (*método array.array*), 310
pop() (*método collections.deque*), 282
pop() (*método dict*), 100
pop() (*método frozenset*), 97
pop() (*método mailbox.Mailbox*), 1404
pop() (*método sequence*), 52
POP3
    protocolo, 1557
POP3 (*classe em poplib*), 1557
POP3_SSL (*classe em poplib*), 1557
pop_all() (*método contextlib.ExitStack*), 2094
POP_BLOCK (*opcode*), 2287
POP_EXCEPT (*opcode*), 2275
POP_ITER (*opcode*), 2272
POP_JUMP_IF_FALSE (*opcode*), 2281
POP_JUMP_IF_NONE (*opcode*), 2281
POP_JUMP_IF_NOT_NONE (*opcode*), 2281
POP_JUMP_IF_TRUE (*opcode*), 2281
pop_source() (*método shlex.shlex*), 2309
POP_TOP (*opcode*), 2272
Popen (*classe em subprocess*), 1108
popen() (*no módulo os*), 766, 1307
popitem() (*método collections.OrderedDict*), 291
popitem() (*método dict*), 100
popitem() (*método mailbox.Mailbox*), 1404
popleft() (*método collections.deque*), 283
poplib
    module, 1557
porção, **2359**
port
    http.server opção de linha de comando, 1595
port (*atributo http.cookiejar.Cookie*), 1607
port_specified (*atributo http.cookiejar.Cookie*), 1608
pos (*atributo json.JSONDecodeError*), 1398
pos (*atributo re.Match*), 165
pos (*atributo re.PatternError*), 160
pos (*atributo tomllib.TOMLDecodeError*), 687
pos() (*no módulo operator*), 465
pos() (*no módulo turtle*), 1725
position (*atributo xml.etree.ElementTree.ParseError*), 1458
position() (*no módulo turtle*), 1725
positions (*atributo inspect.FrameInfo*), 2137
positions (*atributo inspect.Traceback*), 2137
Positions (*classe em dis*), 2271
Positions.col_offset (*no módulo dis*), 2271
Positions.end_col_offset (*no módulo dis*), 2271
Positions.end_lineno (*no módulo dis*), 2271
Positions.lineno (*no módulo dis*), 2271
POSIX
    controle de E/S, 2315
    threads, 1133
posix
    module, 2312
POSIX, memória compartilhada, 1084
POSIX_FADV_DONTNEED (*no módulo os*), 728
POSIX_FADV_NOREUSE (*no módulo os*), 728
POSIX_FADV_NORMAL (*no módulo os*), 728
POSIX_FADV_RANDOM (*no módulo os*), 728
POSIX_FADV_SEQUENTIAL (*no módulo os*), 728
POSIX_FADV_WILLNEED (*no módulo os*), 728
posix_fadvise() (*no módulo os*), 728
posix_fallocate() (*no módulo os*), 728
posix_openpt() (*no módulo os*), 728
posix_spawn() (*no módulo os*), 767
POSIX_SPAWN_CLOSE (*no módulo os*), 767
POSIX_SPAWN_CLOSEFROM (*no módulo os*), 767
POSIX_SPAWN_DUP2 (*no módulo os*), 767
POSIX_SPAWN_OPEN (*no módulo os*), 767
posix_spawnp() (*no módulo os*), 768
PosixPath (*classe em pathlib*), 483
post_handshake_auth (*atributo ssl.SSLContext*), 1294
post_mortem() (*no módulo pdb*), 1967
post_setup() (*método venv.EnvBuilder*), 2010
postcmd() (*método cmd.Cmd*), 1017
postloop() (*método cmd.Cmd*), 1017
Pow (*classe em ast*), 2213
pow()
    built-in function, 26
pow() (*no módulo math*), 366
pow() (*no módulo operator*), 465
power() (*método decimal.Context*), 392
pp (*pdb command*), 1972
pp() (*no módulo pprint*), 327
pprint
    module, 327
pprint() (*método pprint.PrettyPrinter*), 329
pprint() (*no módulo pprint*), 328
prcal() (*no módulo calendar*), 272
pread() (*no módulo os*), 728
preadv() (*no módulo os*), 729
--preamble
    pickletools opção de linha de comando, 2289
preamble (*atributo email.message.EmailMessage*), 1341
preamble (*atributo email.message.Message*), 1378
prec (*atributo decimal.Context*), 388
precmd() (*método cmd.Cmd*), 1017
prefix (*atributo xml.dom.Attr*), 1465

prefix (*atributo xml.dom.Node*), 1461
prefix (*atributo zipimport.zipimporter*), 2161
prefix (*no módulo sys*), 2041
PREFIXES (*no módulo site*), 2153
prefixlen (*atributo ipaddress.IPv4Network*), 1630
prefixlen (*atributo ipaddress.IPv6Network*), 1633
preloop() (*método cmd.Cmd*), 1017
prepare() (*método graphlib.TopologicalSorter*), 353
prepare() (*método logging.handlers.QueueHandler*), 850
prepare() (*método logging.handlers.QueueListener*), 852
prepare_class() (*no módulo types*), 320
prepare_input_source() (*no módulo xml.sax.saxutils*), 1482
prepare_main() (*método concurrent.interpreters.Interpreter*), 1102
PrepareProtocol (*classe em sqlite3*), 587
PrettyPrinter (*classe em pprint*), 329
prev() (*método tkinter.ttk.Treeview*), 1695
previousSibling (*atributo xml.dom.Node*), 1461
print()
    built-in function, 27
print() (*método traceback.TracebackException*), 2111
print_call_graph() (*no módulo asyncio*), 1183
print_callees() (*método pstats.Stats*), 1981
print_callers() (*método pstats.Stats*), 1981
print_exc() (*método timeit.Timer*), 1986
print_exc() (*no módulo traceback*), 2108
print_exception() (*no módulo traceback*), 2108
print_function (*no módulo __future__*), 2118
print_help() (*método argparse.ArgumentParser*), 935
print_last() (*no módulo traceback*), 2108
print_list() (*no módulo traceback*), 2109
print_stack() (*método asyncio.Task*), 1159
print_stack() (*no módulo traceback*), 2109
print_stats() (*método profile.Profile*), 1979
print_stats() (*método pstats.Stats*), 1981
print_tb() (*no módulo traceback*), 2108
print_usage() (*método argparse.ArgumentParser*), 935
print_usage() (*método optparse.OptionParser*), 972
print_version() (*método optparse.OptionParser*), 962
print_warning() (*no módulo test.support*), 1938
printable (*no módulo string*), 130
printdir() (*método zipfile.ZipFile*), 633
printf, estilo de formatação, 69, 85
PRIO_DARWIN_BG (*no módulo os*), 717
PRIO_DARWIN_NONUI (*no módulo os*), 717
PRIO_DARWIN_PROCESS (*no módulo os*), 717
PRIO_DARWIN_THREAD (*no módulo os*), 717
PRIO_PGRP (*no módulo os*), 717
PRIO_PROCESS (*no módulo os*), 717
PRIO_USER (*no módulo os*), 717
PriorityQueue (*classe em asyncio*), 1180
PriorityQueue (*classe em queue*), 1125
prlimit() (*no módulo resource*), 2323
prmonth() (*método calendar.TextCalendar*), 269
prmonth() (*no módulo calendar*), 272
ProactorEventLoop (*classe em asyncio*), 1207
--process
    timeit opção de linha de comando, 1986
Process (*classe em multiprocessing*), 1046
process() (*método logging.LoggerAdapter*), 821
process_cpu_count() (*no módulo os*), 778
process_exited() (*método asyncio.SubprocessProtocol*), 1221
process_request() (*método socketserver.BaseServer*), 1584
process_time() (*no módulo time*), 799
process_time_ns() (*no módulo time*), 799
process_tokens() (*no módulo tabnanny*), 2257
ProcessError, 1049
processingInstruction() (*método xml.sax.handler.ContentHandler*), 1480
ProcessingInstruction() (*no módulo xml.etree.ElementTree*), 1448
ProcessingInstructionHandler() (*método xml.parsers.expat.xmlparser*), 1490
ProcessLookupError, 124
processo
    grupo, 716
    id, 716
    id do pai, 717
    interrupção, 766
    prioridade de agendamento, 717, 719
    sinalização, 766
processor() (*no módulo platform*), 853
processos leves, 1133
ProcessPoolExecutor (*classe em concurrent.futures*), 1095
prod() (*no módulo math*), 367
product() (*no módulo itertools*), 443
profile
    module, 1978
Profile (*classe em profile*), 1978
profiling, deterministic, 1975
ProgrammingError, 587
Progressbar (*classe em tkinter.ttk*), 1690
--prompt
    venv opção de linha de comando, 2006
prompt (*atributo cmd.Cmd*), 1017
prompts do interpretador, 2041
propagate (*atributo logging.Logger*), 810
property (*classe interna*), 27
property() (*no módulo enum*), 351
property_declaration_handler (*no módulo xml.sax.handler*), 1477
property_dom_node (*no módulo xml.sax.handler*), 1478
property_lexical_handler (*no módulo xml.sax.handler*), 1477
property_xml_string (*no módulo xml.sax.handler*), 1478
property.deleter()

built-in function, 28
property.getter()
built-in function, 28
PropertyMock (*classe em unittest.mock*), 1880
property.setter()
built-in function, 28
Propostas de Melhorias do Python
PEP 1, 2359
PEP 8, 30
PEP 205, 316
PEP 227, 2118
PEP 235, 2168
PEP 236, 2119
PEP 237, 70, 87
PEP 238, 2118, 2351
PEP 246, 587
PEP 249, 566, 569, 582, 587, 590, 596
PEP 0249#threadsafety, 571
PEP 255, 2118
PEP 263, 2168, 2253, 2254
PEP 273, 2159
PEP 278, 2363
PEP 282, 532, 826
PEP 292, 139
PEP 302, 35, 524, 2040, 2159, 2162, 2163, 2166, 2168, 21712173, 2355
PEP 305, 659
PEP 307, 539
PEP 324, 1104
PEP 328, 35, 2118, 2168
PEP 338, 2167
PEP 342, 297
PEP 343, 2099, 2118, 2348, 2349
PEP 362, 2134, 2346, 2358
PEP 366, 2167, 2168
PEP 370, 2154
PEP 378, 134
PEP 380#use-of-stopiteration-to-return-values, 2052
PEP 383, 208, 1242
PEP 387, 124, 125
PEP 393, 214, 2039
PEP 405, 2005
PEP 411, 2034, 2035, 2044, 2359
PEP 412, 453
PEP 420, 2168, 2357, 2359
PEP 421, 2036
PEP 428, 472
PEP 434, 1712
PEP 442, 2122
PEP 443, 2352
PEP 451, 2039, 2166, 2167, 2169
PEP 453, 2003
PEP 461, 87
PEP 468, 291
PEP 475, 26, 124, 726, 731, 733, 773, 787, 800, 1259, 1260, 12621265, 13071311, 1315, 1323, 1324
PEP 479, 120, 2118
PEP 483, 2352
PEP 484, 108, 1755, 1765, 1779, 1798, 2209, 2237, 2242, 2345, 2352, 2362, 2363
PEP 485, 364, 373
PEP 488, 1948, 2169, 2183, 2259
PEP 489, 2169, 2178, 2180, 2181, 2185
PEP 492, 299, 2141, 2346, 2347, 2349
PEP 495, 263
PEP 498, 2351
PEP 506, 707
PEP 515, 134, 405
PEP 519, 2358
PEP 524, 780
PEP 525, 299, 2034, 2044, 2141, 2346
PEP 526, 1771, 1788, 2075, 2083, 2143, 2144, 2237, 2242, 2345, 2363
PEP 529, 738, 2032, 2046
PEP 538, 1657
PEP 540, 712, 1657
PEP 544, 1765, 1789
PEP 552, 2169, 2260
PEP 554, 1100
PEP 557, 2075
PEP 560, 320, 321
PEP 563, 2118, 2143, 2144
PEP 565, 124
PEP 566, 2196
PEP 567, 1128, 1188, 1189, 1212
PEP 574, 539, 552
PEP 578, 1951, 2022
PEP 584, 277, 286, 291, 314, 324, 714
PEP 585, 108, 295, 323, 18021809, 2352
PEP 586, 1771
PEP 591, 1772, 1799
PEP 593, 1775, 1800
PEP 594, 2333, 23372341
PEP 597, 782
PEP 604, 109
PEP 610, 2198
PEP 612, 1757, 1763, 1770, 1785, 1808
PEP 613, 1768
PEP 615, 262
PEP 626, 2270
PEP 632, 2338
PEP 644, 1271
PEP 646, 1782
PEP 647, 1776
PEP 649, 1803, 2118, 2143, 2144, 2345
PEP 649#the-stringizer-and-the-fake-globals-environ, 2146
PEP 655, 1772, 1793
PEP 667, 22, 1966
PEP 673, 1768
PEP 675, 1766
PEP 681, 1797
PEP 682, 134
PEP 683, 2353

PEP 684, 1100, 2036
PEP 686, 713, 782
PEP 688, 299
PEP 688#current-options, 298, 1806
PEP 692, 1777
PEP 695, 1766, 1779, 1781, 1783, 1784, 1809
PEP 698, 1799
PEP 702, 2074
PEP 703, 1022, 2351, 2353
PEP 705, 1773
PEP 706, 651
PEP 709, 22
PEP 734, 1094, 1100, 2036
PEP 742, 1776
PEP 749, 2118, 2143
PEP 750, 142
PEP 3101, 130, 131
PEP 3105, 2118
PEP 3107, 2143, 2144
PEP 3112, 2118
PEP 3115, 320, 2236
PEP 3116, 2363
PEP 3118, 89, 539
PEP 3119, 300, 2101
PEP 3120, 2169
PEP 3134, 116
PEP 3141, 357, 2101
PEP 3147, 1948, 2166, 2169, 2183, 2259, 2260, 22622264
PEP 3148, 1098
PEP 3149, 2021
PEP 3151, 124, 1245, 1306, 2322
PEP 3154, 539
PEP 3155, 2359
PEP 3333, 15001505, 1508, 1509
PEP 3333#input-and-error-streams, 1509
PEP 3333#optional-platform-specific-file-handling, 1509
PEP 3333#the-start-response-callable, 1509
prot_c() (*método ftplib.FTP_TLS*), 1556
prot_p() (*método ftplib.FTP_TLS*), 1556
proto (*atributo socket.socket*), 1266
--protocol
http.server opção de linha de comando, 1596
protocol (*atributo ssl.SSLContext*), 1294
Protocol (*classe em asyncio*), 1218
Protocol (*classe em typing*), 1788
PROTOCOL_SSLv3 (*no módulo ssl*), 1278
PROTOCOL_SSLv23 (*no módulo ssl*), 1278
PROTOCOL_TLS (*no módulo ssl*), 1277
PROTOCOL_TLS_CLIENT (*no módulo ssl*), 1277
PROTOCOL_TLS_SERVER (*no módulo ssl*), 1277
PROTOCOL_TLSv1 (*no módulo ssl*), 1278
PROTOCOL_TLSv1_1 (*no módulo ssl*), 1278
PROTOCOL_TLSv1_2 (*no módulo ssl*), 1278
protocol_version (*atributo http.server.BaseHTTPRequestHandler*), 1591
PROTOCOL_VERSION (*atributo imaplib.IMAP4*), 1567
ProtocolError (*classe em xmlrpc.client*), 1614
protocolo
copy, 545
FTP, 1527, 1550
gerenciamento de contexto, 103
HTTP, 1527, 1539, 1543, 1589
IMAP4, 1560
IMAP4_SSL, 1560
IMAP4_stream, 1560
iterador, 48
POP3, 1557
SMTP, 1568
protocolo de buffer
str (*classe embutida*), 57
tipos de sequência binária, 71
protocolo de gerenciamento de contexto, 103, **2348**
protocolo iterador, 48
proxy() (*no módulo weakref*), 313
proxyauth() (*método imaplib.IMAP4*), 1565
ProxyBasicAuthHandler (*classe em urllib.request*), 1515
ProxyDigestAuthHandler (*classe em urllib.request*), 1516
ProxyHandler (*classe em urllib.request*), 1514
ProxyType (*no módulo weakref*), 315
ProxyTypes (*no módulo weakref*), 315
pryear() (*método calendar.TextCalendar*), 269
ps1 (*no módulo sys*), 2041
ps2 (*no módulo sys*), 2041
pstats
module, 1979
pstdev() (*no módulo statistics*), 424
pthread_getcpuclockid() (*no módulo time*), 796
pthread_kill() (*no módulo signal*), 1321
pthread_sigmask() (*no módulo signal*), 1321
pthreads, 1133
pthreads (*atributo sys._emscripten_info*), 2025
ptsname() (*no módulo os*), 729
pty
module, 2317
módulo, 727
pu() (*no módulo turtle*), 1727
publicId (*atributo xml.dom.DocumentType*), 1463
PullDom (*classe em xml.dom.pulldom*), 1473
punctuation (*no módulo string*), 130
punctuation_chars (*atributo shlex.shlex*), 2311
PurePath (*classe em pathlib*), 473
PurePosixPath (*classe em pathlib*), 474
PureWindowsPath (*classe em pathlib*), 474
purge() (*no módulo re*), 160
Purpose.CLIENT_AUTH (*no módulo ssl*), 1282
Purpose.SERVER_AUTH (*no módulo ssl*), 1282
push() (*método code.InteractiveConsole*), 2157
push() (*método contextlib.ExitStack*), 2094

push_async_callback() (*método contextlib.AsyncExitStack*), 2095
push_async_exit() (*método contextlib.AsyncExitStack*), 2095
PUSH_EXC_INFO (*opcode*), 2276
PUSH_NULL (*opcode*), 2283
push_source() (*método shlex.shlex*), 2309
push_token() (*método shlex.shlex*), 2309
put() (*método asyncio.Queue*), 1180
put() (*método multiprocessing.Queue*), 1051
put() (*método multiprocessing.SimpleQueue*), 1052
put() (*método queue.Queue*), 1126
put() (*método queue.SimpleQueue*), 1128
put_nowait() (*método asyncio.Queue*), 1180
put_nowait() (*método multiprocessing.Queue*), 1051
put_nowait() (*método queue.Queue*), 1126
put_nowait() (*método queue.SimpleQueue*), 1128
putch() (*no módulo msvcrt*), 2292
putenv() (*no módulo os*), 717
putheader() (*método http.client.HTTPConnection*), 1547
putp() (*no módulo curses*), 988
putrequest() (*método http.client.HTTPConnection*), 1547
putwch() (*no módulo msvcrt*), 2292
putwin() (*método curses.window*), 996
pvariance() (*no módulo statistics*), 425
pwd
    module, 2313
    módulo, 501
pwd() (*método ftplib.FTP*), 1555
pwrite() (*no módulo os*), 729
pwritev() (*no módulo os*), 730
py_compile
    module, 2259
Py_DEBUG (*no módulo test.support*), 1934
py_object (*classe em ctypes*), 899
PY_RESUME (*monitoring event*), 2051
PY_RETURN (*monitoring event*), 2051
PY_START (*monitoring event*), 2051
PY_THROW (*monitoring event*), 2051
PY_UNWIND (*monitoring event*), 2051
PY_YIELD (*monitoring event*), 2051
pyc baseado em hash, **2353**
pycache_prefix (*no módulo sys*), 2025
PyCF_ALLOW_TOP_LEVEL_AWAIT (*no módulo ast*), 2241
PyCF_ONLY_AST (*no módulo ast*), 2241
PyCF_OPTIMIZED_AST (*no módulo ast*), 2241
PyCF_TYPE_COMMENTS (*no módulo ast*), 2241
PycInvalidationMode (*classe em py_compile*), 2260
pyclbr
    module, 2257
PyCompileError, 2259
PyDLL (*classe em ctypes*), 886
pydoc
    module, 1809
pyexpat
    módulo, 1486
PYFUNCTYPE() (*no módulo ctypes*), 889
--python
    zipapp opção de linha de comando, 2015
Python 3000, **2359**
Python Editor, 1701
python_branch() (*no módulo platform*), 853
python_build() (*no módulo platform*), 853
python_compiler() (*no módulo platform*), 853
PYTHON_CONTEXT_AWARE_WARNINGS, 2028, 2075
PYTHON_CPU_COUNT, 778, 1053
PYTHON_DOM, 1459
PYTHON_GIL, 2028, 2353
python_implementation() (*no módulo platform*), 854
python_is_optimized() (*no módulo test.support*), 1936
PYTHON_JIT, 2037
python_revision() (*no módulo platform*), 854
PYTHON_THREAD_INHERIT_CONTEXT, 2028
python_version() (*no módulo platform*), 854
python_version_tuple() (*no módulo platform*), 854
PYTHONASYNCIODEBUG, 1202, 1238, 1811
PYTHONBREAKPOINT, 10, 2024
PYTHONCASEOK, 35
PYTHONCOERCECLOCALE, 713
PYTHONDEVMODE, 1811
PYTHONDONTWRITEBYTECODE, 2025
PYTHONFAULTHANDLER, 1811, 1961
PythonFinalizationError, 120
PYTHONHOME, 1943, 2201, 2202
Pythônico, **2359**
PYTHONINTMAXSTRDIGITS, 114, 2036
PYTHONIOENCODING, 713, 2046
PYTHONLEGACYWINDOWSFSENCODING, 2045, 2046
PYTHONLEGACYWINDOWSSTDIO, 2046
PYTHONMALLOC, 1811
python--m-py_compile opção de linha de comando
    -, 2261
    <file>, 2261
    -q, 2261
    --quiet, 2261
python--m-sqlite3-[-h]-[-v]-[filename]-[sql] opção de linha de comando
    -h, 589
    --help, 589
    -v, 589
    --version, 589
PYTHONNOUSERSITE, 2153
PYTHONPATH, 1943, 2039, 2201
PYTHONPLATLIBDIR, 2201
PYTHONPYCACHEPREFIX, 2025
PYTHONSAFEPATH, 2040, 2343
PYTHONSTARTUP, 194, 1138, 1708, 1709, 2037, 2153
PYTHONTRACEMALLOC, 1991, 1997
PYTHONTZPATH, 267

PYTHONUNBUFFERED, 2046
PYTHONUSERBASE, 2153, 2154
PYTHONUSERSITE, 1943
PYTHONUTF8, 713, 2046
PYTHONWARNDEFAULTENCODING, 782
PYTHONWARNINGS, 1811, 2068, 2069
PyZipFile (*classe em zipfile*), 637

Q

-q
    compileall opção de linha de comando, 2261
    python--m-py_compile opção de linha de comando, 2261
qiflush() (*no módulo curses*), 988
QName (*classe em xml.etree.ElementTree*), 1455
qsize() (*método asyncio.Queue*), 1180
qsize() (*método multiprocessing.Queue*), 1051
qsize() (*método queue.Queue*), 1126
qsize() (*método queue.SimpleQueue*), 1128
quantiles() (*método statistics.NormalDist*), 430
quantiles() (*no módulo statistics*), 426
quantize() (*método decimal.Context*), 392
quantize() (*método decimal.Decimal*), 384
QueryInfoKey() (*no módulo winreg*), 2297
QueryReflectionKey() (*no módulo winreg*), 2299
QueryValue() (*no módulo winreg*), 2297
QueryValueEx() (*no módulo winreg*), 2298
QUESTION (*no módulo tkinter.messagebox*), 1681
queue
    module, 1125
queue (*atributo sched.scheduler*), 1125
Queue (*classe em asyncio*), 1179
Queue (*classe em concurrent.interpreters*), 1103
Queue (*classe em multiprocessing*), 1050
Queue (*classe em queue*), 1125
Queue() (*método multiprocessing.managers.SyncManager*), 1064
QueueEmpty, 1181
QueueEmptyError, 1103
QueueFull, 1181
QueueFullError, 1103
QueueHandler (*classe em logging.handlers*), 850
QueueListener (*classe em logging.handlers*), 851
QueueShutDown, 1181
quick_ratio() (*método difflib.SequenceMatcher*), 177
--quiet
    python--m-py_compile opção de linha de comando, 2261
quiet (*atributo sys.flags*), 2028
quit (*pdb command*), 1974
quit (*variável interna*), 38
quit() (*método ftplib.FTP*), 1555
quit() (*método poplib.POP3*), 1559
quit() (*método smtplib.SMTP*), 1573
quit() (*método tkinter.filedialog.FileDialog*), 1678
quitting (*bdb.Bdb attribute*), 1959
quopri
    module, 1431
quote() (*no módulo email.utils*), 1388
quote() (*no módulo shlex*), 2307
quote() (*no módulo urllib.parse*), 1535
QUOTE_ALL (*no módulo csv*), 663
quote_from_bytes() (*no módulo urllib.parse*), 1536
QUOTE_MINIMAL (*no módulo csv*), 663
QUOTE_NONE (*no módulo csv*), 663
QUOTE_NONNUMERIC (*no módulo csv*), 663
QUOTE_NOTNULL (*no módulo csv*), 663
quote_plus() (*no módulo urllib.parse*), 1536
QUOTE_STRINGS (*no módulo csv*), 663
quoteattr() (*no módulo xml.sax.saxutils*), 1482
quotechar (*atributo csv.Dialect*), 664
quotes (*atributo shlex.shlex*), 2310
quoting (*atributo csv.Dialect*), 664

R

-R
    trace opção de linha de comando, 1990
-r
    compileall opção de linha de comando, 2262
    idle opção de linha de comando, 1709
    timeit opção de linha de comando, 1986
    trace opção de linha de comando, 1989
R_OK (*no módulo os*), 736
radians() (*no módulo math*), 367
radians() (*no módulo turtle*), 1727
radix (*atributo sys.float_info*), 2030
radix() (*método decimal.Context*), 392
radix() (*método decimal.Decimal*), 384
RADIXCHAR (*no módulo locale*), 1654
raise
    instrução, 115
Raise (*classe em ast*), 2220
RAISE (*monitoring event*), 2051
raise_on_defect (*atributo email.policy.Policy*), 1350
raise_signal() (*no módulo signal*), 1321
RAISE_VARARGS (*opcode*), 2283
raiseExceptions (*no módulo logging*), 826
RAND_add() (*no módulo ssl*), 1274
RAND_bytes() (*no módulo ssl*), 1274
RAND_status() (*no módulo ssl*), 1274
randbelow() (*no módulo secrets*), 707
randbits() (*no módulo secrets*), 707
randbytes() (*método random.Random*), 412
randbytes() (*no módulo random*), 409
randint() (*no módulo random*), 409
random
    module, 407
Random (*classe em random*), 412
random opção de linha de comando
    -c, 417
    --choice, 417
    -f, 417
    --float, 417
    -h, 416

--help, 416
-i, 417
--integer, 417
random() (*método random.Random*), 412
random() (*no módulo random*), 411
randrange() (*no módulo random*), 409
range
objeto, 54
range (*classe interna*), 54
RARROW (*no módulo token*), 2252
ratio() (*método difflib.SequenceMatcher*), 177
Rational (*classe em numbers*), 358
raw (*atributo io.BufferedIOBase*), 787
raw() (*método pickle.PickleBuffer*), 543
raw() (*no módulo curses*), 988
raw_data_manager (*no módulo email.contentmanager*), 1363
raw_decode() (*método json.JSONDecoder*), 1396
raw_input() (*método code.InteractiveConsole*), 2157
RawArray() (*no módulo multiprocessing.sharedctypes*), 1060
RawConfigParser (*classe em configparser*), 684
RawDescriptionHelpFormatter (*classe em argparse*), 909
RawIOBase (*classe em io*), 786
RawPen (*classe em turtle*), 1745
RawTextHelpFormatter (*classe em argparse*), 909
RawTurtle (*classe em turtle*), 1745
RawValue() (*no módulo multiprocessing.sharedctypes*), 1060
RBRACE (*no módulo token*), 2250
re
module, 147
módulo, 57, 522
re (*atributo re.Match*), 165
READ (*atributo inspect.BufferFlags*), 2142
read() (*método asyncio.StreamReader*), 1164
read() (*método codecs.StreamReader*), 212
read() (*método configparser.ConfigParser*), 682
read() (*método http.client.HTTPResponse*), 1548
read() (*método imaplib.IMAP4*), 1565
read() (*método io.BufferedIOBase*), 787
read() (*método io.BufferedReader*), 789
read() (*método io.RawIOBase*), 786
read() (*método io.Reader*), 794
read() (*método io.TextIOBase*), 791
read() (*método mimetypes.MimeTypes*), 1423
read() (*método mmap.mmap*), 1329
read() (*método sqlite3.Blob*), 586
read() (*método ssl.MemoryBIO*), 1303
read() (*método ssl.SSLSocket*), 1283
read() (*método urllib.robotparser.RobotFileParser*), 1538
read() (*método zipfile.ZipFile*), 634
read() (*no módulo os*), 730
read1() (*método bz2.BZ2File*), 620
read1() (*método io.BufferedIOBase*), 787
read1() (*método io.BufferedReader*), 789
read1() (*método io.BytesIO*), 789
read_binary() (*no módulo importlib.resources*), 2190
read_byte() (*método mmap.mmap*), 1329
read_bytes() (*método importlib.abc.Traversable*), 2176
read_bytes() (*método importlib.resources.abc.Traversable*), 2193
read_bytes() (*método pathlib.Path*), 489
read_bytes() (*método zipfile.Path*), 637
read_dict() (*método configparser.ConfigParser*), 682
read_environ() (*no módulo wsgiref.handlers*), 1508
read_events() (*método xml.etree.ElementTree.XMLPullParser*), 1458
read_file() (*método configparser.ConfigParser*), 682
read_history_file() (*no módulo readline*), 192
read_init_file() (*no módulo readline*), 192
read_mime_types() (*no módulo mimetypes*), 1421
read_string() (*método configparser.ConfigParser*), 682
read_text() (*método importlib.abc.Traversable*), 2177
read_text() (*método importlib.resources.abc.Traversable*), 2193
read_text() (*método pathlib.Path*), 489
read_text() (*método zipfile.Path*), 636
read_text() (*no módulo importlib.resources*), 2190
read_token() (*método shlex.shlex*), 2309
read_windows_registry() (*método mimetypes.MimeTypes*), 1423
READABLE (*no módulo _tkinter*), 1674
readable() (*método bz2.BZ2File*), 620
readable() (*método io.IOBase*), 785
readall() (*método io.RawIOBase*), 786
readbuffer_encode() (*no módulo codecs*), 207
Reader (*classe em io*), 794
reader() (*no módulo csv*), 659
ReadError, 643
readexactly() (*método asyncio.StreamReader*), 1164
readfp() (*método mimetypes.MimeTypes*), 1423
readframes() (*método wave.Wave_read*), 1640
readinto() (*método bz2.BZ2File*), 620
readinto() (*método http.client.HTTPResponse*), 1548
readinto() (*método io.BufferedIOBase*), 787
readinto() (*método io.RawIOBase*), 786
readinto() (*no módulo os*), 731
readinto1() (*método io.BufferedIOBase*), 787
readinto1() (*método io.BytesIO*), 789
readline
module, 191
readline() (*método asyncio.StreamReader*), 1164
readline() (*método codecs.StreamReader*), 213
readline() (*método imaplib.IMAP4*), 1565
readline() (*método io.IOBase*), 785
readline() (*método io.TextIOBase*), 791
readline() (*método mmap.mmap*), 1329
readlines() (*método codecs.StreamReader*), 213
readlines() (*método io.IOBase*), 785
readlink() (*método pathlib.Path*), 486

readlink() (*no módulo os*), 743
readmodule() (*no módulo pyclbr*), 2258
readmodule_ex() (*no módulo pyclbr*), 2258
readonly (*atributo memoryview*), 94
ReadOnly (*no módulo typing*), 1772
ReadTransport (*classe em asyncio*), 1214
readuntil() (*método asyncio.StreamReader*), 1164
readv() (*no módulo os*), 733
ready() (*método multiprocessing.pool.AsyncResult*), 1070
real (*atributo numbers.Complex*), 357
Real (*classe em numbers*), 357
real_max_memuse (*no módulo test.support*), 1935
real_quick_ratio() (*método difflib.SequenceMatcher*), 177
realpath() (*no módulo os.path*), 503
REALTIME_PRIORITY_CLASS (*no módulo subprocess*), 1117
reap_children() (*no módulo test.support*), 1940
reap_threads() (*no módulo test.support.threading_helper*), 1945
reason (*atributo http.client.HTTPResponse*), 1548
reason (*atributo ssl.SSLError*), 1273
reason (*atributo UnicodeError*), 122
reason (*atributo urllib.error.HTTPError*), 1538
reason (*atributo urllib.error.URLError*), 1537
reattach() (*método tkinter.ttk.Treeview*), 1695
recent() (*método imaplib.IMAP4*), 1565
reconfigure() (*método io.TextIOWrapper*), 792
record_original_stdout() (*no módulo test.support*), 1936
RECORDS (*atributo inspect.BufferFlags*), 2142
records (*atributo unittest.TestCase*), 1852
RECORDS_RO (*atributo inspect.BufferFlags*), 2142
rect() (*no módulo cmath*), 371
rectangle() (*no módulo curses.textpad*), 1009
RecursionError, 120
recursive_repr() (*no módulo reprlib*), 333
recv() (*método multiprocessing.connection.Connection*), 1055
recv() (*método socket.socket*), 1261
recv_bytes() (*método multiprocessing.connection.Connection*), 1055
recv_bytes_into() (*método multiprocessing.connection.Connection*), 1055
recv_fds() (*no módulo socket*), 1259
recv_into() (*método socket.socket*), 1263
recvfrom() (*método socket.socket*), 1262
recvfrom_into() (*método socket.socket*), 1263
recvmsg() (*método socket.socket*), 1262
recvmsg_into() (*método socket.socket*), 1263
redirect_request() (*método urllib.request.HTTPRedirectHandler*), 1520
redirect_stderr() (*no módulo contextlib*), 2091
redirect_stdout() (*no módulo contextlib*), 2091
redisplay() (*no módulo readline*), 192
redrawln() (*método curses.window*), 996
redrawwin() (*método curses.window*), 996
reduce() (*no módulo functools*), 458
reducer_override() (*método pickle.Pickler*), 541
ref (*classe em weakref*), 312
refcount_test() (*no módulo test.support*), 1939
ReferenceError, 120
ReferenceType (*no módulo weakref*), 315
referência emprestada, **2347**
referência forte, **2361**
refold_source (*atributo email.policy.EmailPolicy*), 1352
refresh() (*método curses.window*), 996
REG_BINARY (*no módulo winreg*), 2301
REG_DWORD (*no módulo winreg*), 2301
REG_DWORD_BIG_ENDIAN (*no módulo winreg*), 2301
REG_DWORD_LITTLE_ENDIAN (*no módulo winreg*), 2301
REG_EXPAND_SZ (*no módulo winreg*), 2301
REG_FULL_RESOURCE_DESCRIPTOR (*no módulo winreg*), 2301
REG_LINK (*no módulo winreg*), 2301
REG_MULTI_SZ (*no módulo winreg*), 2301
REG_NONE (*no módulo winreg*), 2301
REG_QWORD (*no módulo winreg*), 2301
REG_QWORD_LITTLE_ENDIAN (*no módulo winreg*), 2301
REG_RESOURCE_LIST (*no módulo winreg*), 2301
REG_RESOURCE_REQUIREMENTS_LIST (*no módulo winreg*), 2302
REG_SZ (*no módulo winreg*), 2302
RegexFlag (*classe em re*), 154
register() (*método abc.ABCMeta*), 2101
register() (*método argparse.ArgumentParser*), 937
register() (*método functools.singledispatch*), 458
register() (*método multiprocessing.managers.BaseManager*), 1063
register() (*método select.devpoll*), 1308
register() (*método select.epoll*), 1309
register() (*método selectors.BaseSelector*), 1314
register() (*método select.poll*), 1310
register() (*no módulo atexit*), 2106
register() (*no módulo codecs*), 206
register() (*no módulo faulthandler*), 1963
register() (*no módulo webbrowser*), 1498
register_adapter() (*no módulo sqlite3*), 570
register_archive_format() (*no módulo shutil*), 532
register_at_fork() (*no módulo os*), 768
register_callback() (*no módulo sys.monitoring*), 2053
register_converter() (*no módulo sqlite3*), 570
register_defect() (*método email.policy.Policy*), 1351
register_dialect() (*no módulo csv*), 660
register_error() (*no módulo codecs*), 209
register_function() (*método xmlrpc.server.CGIXMLRPCRequestHandler*), 1621

register_function() (*método xmlrpc.server.SimpleXMLRPCServer*), 1617
register_instance() (*método xmlrpc.server.CGIXMLRPCRequestHandler*), 1621
register_instance() (*método xmlrpc.server.SimpleXMLRPCServer*), 1618
register_introspection_functions() (*método xmlrpc.server.CGIXMLRPCRequestHandler*), 1621
register_introspection_functions() (*método xmlrpc.server.SimpleXMLRPCServer*), 1618
register_multicall_functions() (*método xmlrpc.server.CGIXMLRPCRequestHandler*), 1621
register_multicall_functions() (*método xmlrpc.server.SimpleXMLRPCServer*), 1618
register_namespace() (*no módulo xml.etree.ElementTree*), 1449
register_optionflag() (*no módulo doctest*), 1823
register_shape() (*no módulo turtle*), 1743
register_unpack_format() (*no módulo shutil*), 533
registerDOMImplementation() (*no módulo xml.dom*), 1459
registerResult() (*no módulo unittest*), 1868
REGTYPE (*no módulo tarfile*), 644
relativa
    URL, 1528
relative_to() (*método pathlib.PurePath*), 481
release() (*método _thread.lock*), 1134
release() (*método asyncio.Condition*), 1172
release() (*método asyncio.Lock*), 1170
release() (*método asyncio.Semaphore*), 1173
release() (*método logging.Handler*), 814
release() (*método memoryview*), 91
release() (*método multiprocessing.Lock*), 1057
release() (*método multiprocessing.RLock*), 1058
release() (*método pickle.PickleBuffer*), 543
release() (*método threading.Condition*), 1033
release() (*método threading.Lock*), 1031
release() (*método threading.RLock*), 1032
release() (*método threading.Semaphore*), 1035
release() (*no módulo platform*), 854
reload() (*no módulo importlib*), 2169
reload_environ() (*no módulo os*), 714
relpath() (*no módulo os.path*), 504
remainder() (*método decimal.Context*), 392
remainder() (*no módulo math*), 363
remainder_near() (*método decimal.Context*), 392
remainder_near() (*método decimal.Decimal*), 385
remote_exec() (*no módulo sys*), 2045
RemoteDisconnected, 1545
remove() (*método array.array*), 311
remove() (*método collections.deque*), 283
remove() (*método frozenset*), 97
remove() (*método mailbox.Mailbox*), 1402
remove() (*método mailbox.MH*), 1409
remove() (*método sequence*), 52
remove() (*método xml.etree.ElementTree.Element*), 1453
remove() (*no módulo os*), 743
remove_done_callback() (*método asyncio.Future*), 1212
remove_done_callback() (*método asyncio.Task*), 1158
remove_flag() (*método mailbox.Maildir*), 1406
remove_flag() (*método mailbox.MaildirMessage*), 1412
remove_flag() (*método mailbox.mboxMessage*), 1414
remove_flag() (*método mailbox.MMDFMessage*), 1418
remove_folder() (*método mailbox.Maildir*), 1405
remove_folder() (*método mailbox.MH*), 1409
remove_header() (*método urllib.request.Request*), 1517
remove_history_item() (*no módulo readline*), 193
remove_label() (*método mailbox.BabylMessage*), 1416
remove_option() (*método configparser.ConfigParser*), 684
remove_option() (*método optparse.OptionParser*), 971
remove_reader() (*método asyncio.loop*), 1196
remove_section() (*método configparser.ConfigParser*), 684
remove_sequence() (*método mailbox.MHMessage*), 1415
remove_signal_handler() (*método asyncio.loop*), 1200
remove_writer() (*método asyncio.loop*), 1197
removeAttribute() (*método xml.dom.Element*), 1464
removeAttributeNode() (*método xml.dom.Element*), 1464
removeAttributeNS() (*método xml.dom.Element*), 1464
removeChild() (*método xml.dom.Node*), 1462
removedirs() (*no módulo os*), 743
removeFilter() (*método logging.Handler*), 815
removeFilter() (*método logging.Logger*), 813
removeHandler() (*método logging.Logger*), 813
removeHandler() (*no módulo unittest*), 1868
removeprefix() (*método bytearray*), 74
removeprefix() (*método bytes*), 74
removeprefix() (*método str*), 63
removeResult() (*no módulo unittest*), 1868
removesuffix() (*método bytearray*), 75
removesuffix() (*método bytes*), 75
removesuffix() (*método str*), 63
removexattr() (*no módulo os*), 761
rename() (*método ftplib.FTP*), 1554
rename() (*método imaplib.IMAP4*), 1565
rename() (*método pathlib.Path*), 494
rename() (*no módulo os*), 743
renames() (*no módulo os*), 744

reopenIfNeeded() (*método logging.handlers.WatchedFileHandler*), 840
reorganize() (*método dbm.gnu.gdbm*), 563
--repeat
    timeit opção de linha de comando, 1986
repeat() (*método timeit.Timer*), 1985
repeat() (*no módulo itertools*), 444
repeat() (*no módulo timeit*), 1984
repetição
    operação, 49
REPL, **2360**
replace
    error handler's name, 208
replace() (*método bytearray*), 76
replace() (*método bytes*), 76
replace() (*método curses.panel.Panel*), 1015
replace() (*método datetime.date*), 232
replace() (*método datetime.datetime*), 240
replace() (*método datetime.time*), 248
replace() (*método inspect.Parameter*), 2133
replace() (*método inspect.Signature*), 2131
replace() (*método pathlib.Path*), 494
replace() (*método str*), 63
replace() (*método tarfile.TarInfo*), 650
replace() (*no módulo copy*), 326
replace() (*no módulo dataclasses*), 2081
replace() (*no módulo os*), 744
replace_errors() (*no módulo codecs*), 209
replace_header() (*método email.message.EmailMessage*), 1337
replace_header() (*método email.message.Message*), 1375
replace_history_item() (*no módulo readline*), 193
replace_whitespace (*atributo textwrap.TextWrapper*), 185
replaceChild() (*método xml.dom.Node*), 1462
ReplacePackage() (*no módulo modulefinder*), 2164
--report
    trace opção de linha de comando, 1989
report() (*método filecmp.dircmp*), 513
report() (*método modulefinder.ModuleFinder*), 2164
REPORT_CDIFF (*no módulo doctest*), 1822
REPORT_ERRMODE (*no módulo msvcrt*), 2293
report_failure() (*método doctest.DocTestRunner*), 1832
report_full_closure() (*método filecmp.dircmp*), 513
REPORT_NDIFF (*no módulo doctest*), 1823
REPORT_ONLY_FIRST_FAILURE (*no módulo doctest*), 1823
report_partial_closure() (*método filecmp.dircmp*), 513
report_start() (*método doctest.DocTestRunner*), 1832
report_success() (*método doctest.DocTestRunner*), 1832
REPORT_UDIFF (*no módulo doctest*), 1822
report_unexpected_exception() (*método doctest.DocTestRunner*), 1832
REPORTING_FLAGS (*no módulo doctest*), 1823
Repr (*classe em reprlib*), 333
repr()
    built-in function, 29
repr() (*método reprlib.Repr*), 335
repr() (*no módulo reprlib*), 333
repr1() (*método reprlib.Repr*), 335
ReprEnum (*classe em enum*), 347
reprlib
    module, 333
request (*atributo socketserver.BaseRequestHandler*), 1585
Request (*classe em urllib.request*), 1513
request() (*método http.client.HTTPConnection*), 1545
request_queue_size (*atributo socketserver.BaseServer*), 1584
request_rate() (*método urllib.robotparser.RobotFileParser*), 1539
request_uri() (*no módulo wsgiref.util*), 1500
request_version (*atributo http.server.BaseHTTPRequestHandler*), 1591
RequestHandlerClass (*atributo socketserver.BaseServer*), 1583
requestline (*atributo http.server.BaseHTTPRequestHandler*), 1591
Required (*no módulo typing*), 1772
requires() (*no módulo importlib.metadata*), 2197
requires() (*no módulo test.support*), 1936
requires_bz2() (*no módulo test.support*), 1939
requires_docstrings() (*no módulo test.support*), 1939
requires_freebsd_version() (*no módulo test.support*), 1938
requires_gil_enabled() (*no módulo test.support*), 1939
requires_gzip() (*no módulo test.support*), 1939
requires_IEEE_754() (*no módulo test.support*), 1939
requires_limited_api() (*no módulo test.support*), 1939
requires_linux_version() (*no módulo test.support*), 1938
requires_lzma() (*no módulo test.support*), 1939
requires_mac_version() (*no módulo test.support*), 1939
requires_resource() (*no módulo test.support*), 1939
requires_zlib() (*no módulo test.support*), 1939
RERAISE (*monitoring event*), 2051
RERAISE (*opcode*), 2275
reschedule() (*método asyncio.Timeout*), 1151
reserved (*atributo zipfile.ZipInfo*), 639
RESERVED_FUTURE (*no módulo uuid*), 1578
RESERVED_MICROSOFT (*no módulo uuid*), 1578
RESERVED_NCS (*no módulo uuid*), 1578
reset() (*método asyncio.Barrier*), 1174

reset() (*método bdb.Bdb*), 1957
reset() (*método codecs.IncrementalDecoder*), 211
reset() (*método codecs.IncrementalEncoder*), 211
reset() (*método codecs.StreamReader*), 213
reset() (*método codecs.StreamWriter*), 212
reset() (*método contextvars.ContextVar*), 1129
reset() (*método html.parser.HTMLParser*), 1435
reset() (*método threading.Barrier*), 1037
reset() (*método xml.dom.pulldom.DOMEventStream*), 1474
reset() (*método xml.sax.xmlreader.IncrementalParser*), 1484
reset() (*no módulo turtle*), 1731
reset_mock() (*método unittest.mock.AsyncMock*), 1884
reset_mock() (*método unittest.mock.Mock*), 1874
reset_peak() (*no módulo tracemalloc*), 1996
reset_prog_mode() (*no módulo curses*), 988
reset_shell_mode() (*no módulo curses*), 988
reset_tzpath() (*no módulo zoneinfo*), 266
resetbuffer() (*método code.InteractiveConsole*), 2157
resetscreen() (*no módulo turtle*), 1738
resetty() (*no módulo curses*), 988
resetwarnings() (*no módulo warnings*), 2073
resize() (*método bytearray*), 73
resize() (*método curses.window*), 996
resize() (*método mmap.mmap*), 1329
resize() (*no módulo ctypes*), 894
resize_term() (*no módulo curses*), 989
resizemode() (*no módulo turtle*), 1732
resizeterm() (*no módulo curses*), 989
resolution (*atributo datetime.date*), 231
resolution (*atributo datetime.datetime*), 238
resolution (*atributo datetime.time*), 247
resolution (*atributo datetime.timedelta*), 227
resolve() (*método pathlib.Path*), 485
resolve_bases() (*no módulo types*), 320
resolve_name() (*no módulo importlib.util*), 2183
resolve_name() (*no módulo pkgutil*), 2163
resolveEntity() (*método xml.sax.handler.EntityResolver*), 1480
resource
    module, 2322
resource_path() (*método importlib.abc.ResourceReader*), 2176
resource_path() (*método importlib.resources.abc.ResourceReader*), 2192
ResourceDenied, 1933
ResourceLoader (*classe em importlib.abc*), 2172
ResourceReader (*classe em importlib.abc*), 2175
ResourceReader (*classe em importlib.resources.abc*), 2191
ResourceWarning, 125
response() (*método imaplib.IMAP4*), 1565
ResponseNotReady, 1545
responses (*atributo http.server.BaseHTTPRequestHandler*), 1592
responses (*no módulo http.client*), 1545
restart (*pdb command*), 1974
restart_events() (*método bdb.Bdb*), 1960
restart_events() (*no módulo sys.monitoring*), 2053
restore() (*método test.support.SaveSignals*), 1942
restore() (*no módulo difflib*), 174
restype (*atributo ctypes._CFuncPtr*), 888
result() (*método asyncio.Future*), 1211
result() (*método asyncio.Task*), 1158
result() (*método concurrent.futures.Future*), 1097
results() (*método trace.Trace*), 1990
RESUME (*opcode*), 2285
resume_reading() (*método asyncio.ReadTransport*), 1216
resume_writing() (*método asyncio.BaseProtocol*), 1219
retr() (*método poplib.POP3*), 1559
retrbinary() (*método ftplib.FTP*), 1553
retrlines() (*método ftplib.FTP*), 1553
RETRY (*no módulo tkinter.messagebox*), 1680
RETRYCANCEL (*no módulo tkinter.messagebox*), 1681
Return (*classe em ast*), 2235
return (*pdb command*), 1971
return_annotation (*atributo inspect.Signature*), 2131
RETURN_GENERATOR (*opcode*), 2285
return_ok() (*método http.cookiejar.CookiePolicy*), 1604
return_value (*atributo unittest.mock.Mock*), 1875
RETURN_VALUE (*opcode*), 2275
returncode (*atributo asyncio.subprocess.Process*), 1178
returncode (*atributo subprocess.CalledProcessError*), 1107
returncode (*atributo subprocess.CompletedProcess*), 1105
returncode (*atributo subprocess.Popen*), 1115
retval (*pdb command*), 1974
reveal_type() (*no módulo typing*), 1795
reverse() (*método array.array*), 311
reverse() (*método collections.deque*), 283
reverse() (*método sequence*), 52
reverse_order() (*método pstats.Stats*), 1981
reverse_pointer (*atributo ipaddress.IPv4Address*), 1624
reverse_pointer (*atributo ipaddress.IPv6Address*), 1627
reversed()
    built-in function, 29
Reversible (*classe em collections.abc*), 297
Reversible (*classe em typing*), 1808
revert() (*método http.cookiejar.FileCookieJar*), 1603
rewind() (*método wave.Wave_read*), 1640
RFC
    RFC 821, 1568, 1570
    RFC 822, 802, 1364, 1382, 1547, 1571, 1572, 1574, 1646
    RFC 959, 1550
    RFC 1123, 802

RFC 1321, 693
RFC 1422, 1296, 1305
RFC 1521, 1429, 1431
RFC 1522, 1430, 1431
RFC 1730, 1560
RFC 1738, 1537
RFC 1750, 1274
RFC 1808, 1528, 1529, 1537
RFC 1869, 1568, 1570
RFC 1939, 1557
RFC 2045, 1333, 1337, 1359, 1375, 1376, 1382, 1425, 1428
RFC 2045 Section 6.8, 1613
RFC 2046, 1333, 1364, 1382
RFC 2047, 1333, 1352, 1357, 1358, 1382, 1383, 1388
RFC 2060, 1560, 1566
RFC 2068, 1597
RFC 2104, 705
RFC 2109, 15971602, 16061608
RFC 2177, 1563
RFC 2183, 1333, 1339, 1377
RFC 2231, 1333, 1337, 1338, 1375, 1376, 1382, 1389, 1390
RFC 2295, 1541
RFC 2324, 1541
RFC 2342, 1564
RFC 2368, 1537
RFC 2373, 1625, 1626
RFC 2396, 1532, 1535, 1537
RFC 2397, 1523
RFC 2449, 1558
RFC 2518, 1540
RFC 2595, 1557, 1559
RFC 2616, 1502, 1505, 1520, 1538
RFC 2616 Section 5.1.2, 1545
RFC 2616 Section 14.23, 1545
RFC 2640, 1550, 1552, 1556
RFC 2732, 1537
RFC 2774, 1541
RFC 2821, 1333
RFC 2822, 802, 1374, 1382, 1383, 13871389, 1411, 1544, 1591
RFC 2964, 1602
RFC 2965, 1514, 1517, 16001602, 16041608
RFC 3056, 1627
RFC 3171, 1625
RFC 3229, 1540
RFC 3280, 1284
RFC 3330, 1626
RFC 3454, 190
RFC 3490, 219, 221, 222
RFC 3490 Section 3.1, 221
RFC 3492, 219, 221
RFC 3493, 1270
RFC 3501, 1566
RFC 3542, 1257
RFC 3548, 1429
RFC 3659, 1554
RFC 3879, 1626, 1627
RFC 3927, 1626
RFC 3986, 1528, 1530, 15331535, 1537, 1591
RFC 4007, 1626, 1627
RFC 4086, 1305
RFC 4122, 1574, 1577, 1578
RFC 4180, 659
RFC 4193, 1627
RFC 4217, 1555
RFC 4291, 1626
RFC 4380, 1627
RFC 4627, 1391, 1400
RFC 4648, 1425, 1426, 1428, 2343
RFC 4918, 1540, 1541
RFC 4954, 1571
RFC 5161, 1562
RFC 5246, 1282, 1305
RFC 5280, 1272, 1273, 1275, 1305
RFC 5321, 1361
RFC 5322, 1333, 1334, 1343, 1346, 1347, 1350, 1352, 13551358, 1361, 1362, 1371, 1573
RFC 5424, 846
RFC 5735, 1625
RFC 5789, 1542
RFC 5842, 1540, 1541
RFC 5891, 221
RFC 5895, 221
RFC 5929, 1286
RFC 6066, 1280, 1291, 1305
RFC 6531, 1335, 1352, 1568
RFC 6532, 1333, 1334, 1343, 1352
RFC 6585, 1541
RFC 6855, 1562
RFC 6856, 1559
RFC 7159, 1391, 1398, 1400
RFC 7230, 1513, 1547
RFC 7301, 1280, 1290
RFC 7525, 1305
RFC 7693, 698
RFC 7725, 1541
RFC 7914, 697
RFC 8089, 484
RFC 8089 Section 3, 1513
RFC 8297, 1540
RFC 8305, 1191
RFC 8470, 1541
RFC 9110, 15401542
RFC 9562, 15741576, 1578
RFC 9562 Section 5.1, 1577
RFC 9562 Section 5.3, 1577
RFC 9562 Section 5.4, 1577
RFC 9562 Section 5.5, 1577
RFC 9562 Section 5.6, 1577
RFC 9562 Section 5.7, 1577
RFC 9562 Section 5.8, 1577
RFC 9562 Section 5.9, 1578
RFC 9562 Section 5.10, 1578

rfc2109 (*atributo http.cookiejar.Cookie*), 1607
rfc2109_as_netscape (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
rfc2965 (*atributo http.cookiejar.CookiePolicy*), 1605
RFC_4122 (*no módulo uuid*), 1578
rfile (*atributo http.server.BaseHTTPRequestHandler*), 1591
rfile (*atributo socketserver.DatagramRequestHandler*), 1585
rfind() (*método bytearray*), 77
rfind() (*método bytes*), 77
rfind() (*método mmap.mmap*), 1329
rfind() (*método str*), 63
rgb_to_hls() (*no módulo colorsys*), 1642
rgb_to_hsv() (*no módulo colorsys*), 1642
rgb_to_yiq() (*no módulo colorsys*), 1642
rglob() (*método pathlib.Path*), 491
right (*atributo filecmp.dircmp*), 513
right() (*no módulo turtle*), 1720
right_list (*atributo filecmp.dircmp*), 513
right_only (*atributo filecmp.dircmp*), 513
RIGHTSHIFT (*no módulo token*), 2251
RIGHTSHIFTEQUAL (*no módulo token*), 2251
rindex() (*método bytearray*), 77
rindex() (*método bytes*), 77
rindex() (*método str*), 63
rjust() (*método bytearray*), 78
rjust() (*método bytes*), 78
rjust() (*método str*), 63
rlcompleter
    module, 196
RLIM_INFINITY (*no módulo resource*), 2323
RLIMIT_AS (*no módulo resource*), 2324
RLIMIT_CORE (*no módulo resource*), 2323
RLIMIT_CPU (*no módulo resource*), 2323
RLIMIT_DATA (*no módulo resource*), 2324
RLIMIT_FSIZE (*no módulo resource*), 2324
RLIMIT_KQUEUES (*no módulo resource*), 2325
RLIMIT_MEMLOCK (*no módulo resource*), 2324
RLIMIT_MSGQUEUE (*no módulo resource*), 2324
RLIMIT_NICE (*no módulo resource*), 2324
RLIMIT_NOFILE (*no módulo resource*), 2324
RLIMIT_NPROC (*no módulo resource*), 2324
RLIMIT_NPTS (*no módulo resource*), 2325
RLIMIT_OFILE (*no módulo resource*), 2324
RLIMIT_RSS (*no módulo resource*), 2324
RLIMIT_RTPRIO (*no módulo resource*), 2324
RLIMIT_RTTIME (*no módulo resource*), 2324
RLIMIT_SBSIZE (*no módulo resource*), 2325
RLIMIT_SIGPENDING (*no módulo resource*), 2324
RLIMIT_STACK (*no módulo resource*), 2324
RLIMIT_SWAP (*no módulo resource*), 2325
RLIMIT_VMEM (*no módulo resource*), 2324
RLock (*classe em multiprocessing*), 1058
RLock (*classe em threading*), 1031
RLock() (*método multiprocessing.managers.SyncManager*), 1064
rmd() (*método ftplib.FTP*), 1555
rmdir() (*método pathlib.Path*), 495
rmdir() (*no módulo os*), 744
rmdir() (*no módulo test.support.os_helper*), 1947
rmtree() (*no módulo shutil*), 528
rmtree() (*no módulo test.support.os_helper*), 1947
RobotFileParser (*classe em urllib.robotparser*), 1538
robots.txt, 1538
rollback() (*método sqlite3.Connection*), 573
rollover() (*método tempfile.SpooledTemporaryFile*), 516
ROMAN (*no módulo tkinter.font*), 1675
--root
    ensurepip opção de linha de comando, 2004
root (*atributo pathlib.PurePath*), 476
rotate() (*método collections.deque*), 283
rotate() (*método decimal.Context*), 392
rotate() (*método decimal.Decimal*), 385
rotate() (*método logging.handlers.BaseRotatingHandler*), 841
RotatingFileHandler (*classe em logging.handlers*), 842
rotation_filename() (*método logging.handlers.BaseRotatingHandler*), 841
rotator (*atributo logging.handlers.BaseRotatingHandler*), 841
round()
    built-in function, 29
ROUND_05UP (*no módulo decimal*), 394
ROUND_CEILING (*no módulo decimal*), 393
ROUND_DOWN (*no módulo decimal*), 393
ROUND_FLOOR (*no módulo decimal*), 393
ROUND_HALF_DOWN (*no módulo decimal*), 394
ROUND_HALF_EVEN (*no módulo decimal*), 394
ROUND_HALF_UP (*no módulo decimal*), 394
ROUND_UP (*no módulo decimal*), 394
Rounded (*classe em decimal*), 395
rounding (*atributo decimal.Context*), 388
rounds (*atributo sys.float_info*), 2030
Row (*classe em sqlite3*), 585
row_factory (*atributo sqlite3.Connection*), 582
row_factory (*atributo sqlite3.Cursor*), 585
rowcount (*atributo sqlite3.Cursor*), 585
RPAR (*no módulo token*), 2249
rpartition() (*método bytearray*), 77
rpartition() (*método bytes*), 77
rpartition() (*método str*), 63
rpc_paths (*atributo xmlrpc.server.SimpleXMLRPCRequestHandler*), 1618
rpop() (*método poplib.POP3*), 1559
RS (*no módulo curses.ascii*), 1012
rset() (*método poplib.POP3*), 1559
RShift (*classe em ast*), 2213
rshift() (*no módulo operator*), 465
rsplit() (*método bytearray*), 79
rsplit() (*método bytes*), 79
rsplit() (*método str*), 63
RSQB (*no módulo token*), 2249

rstrip() (*método bytearray*), 79
rstrip() (*método bytes*), 79
rstrip() (*método str*), 63
rt() (*no módulo turtle*), 1720
RTLD_DEEPBIND (*no módulo os*), 779
RTLD_GLOBAL (*no módulo os*), 779
RTLD_LAZY (*no módulo os*), 779
RTLD_LOCAL (*no módulo os*), 779
RTLD_NODELETE (*no módulo os*), 779
RTLD_NOLOAD (*no módulo os*), 779
RTLD_NOW (*no módulo os*), 779
ruler (*atributo cmd.Cmd*), 1017
run (*pdb command*), 1974
Run script, 1703
run() (*método asyncio.Runner*), 1140
run() (*método bdb.Bdb*), 1960
run() (*método contextvars.Context*), 1130
run() (*método doctest.DocTestRunner*), 1832
run() (*método multiprocessing.Process*), 1046
run() (*método pdb.Pdb*), 1968
run() (*método profile.Profile*), 1979
run() (*método sched.scheduler*), 1124
run() (*método threading.Thread*), 1028
run() (*método trace.Trace*), 1990
run() (*método unittest.IsolatedAsyncioTestCase*), 1858
run() (*método unittest.TestCase*), 1848
run() (*método unittest.TestSuite*), 1859
run() (*método unittest.TextTestRunner*), 1865
run() (*método wsgiref.handlers.BaseHandler*), 1506
run() (*no módulo asyncio*), 1138
run() (*no módulo pdb*), 1966
run() (*no módulo profile*), 1978
run() (*no módulo subprocess*), 1104
run_concurrently() (*no módulo test.support.threading_helper*), 1945
run_coroutine_threadsafe() (*no módulo asyncio*), 1155
run_docstring_examples() (*no módulo doctest*), 1826
run_forever() (*método asyncio.loop*), 1186
run_in_executor() (*método asyncio.loop*), 1200
run_in_subinterp() (*no módulo test.support*), 1941
run_module() (*no módulo runpy*), 2166
run_path() (*no módulo runpy*), 2166
run_python_until_end() (*no módulo test.support.script_helper*), 1943
run_script() (*método modulefinder.ModuleFinder*), 2164
run_until_complete() (*método asyncio.loop*), 1186
run_with_locale() (*no módulo test.support*), 1938
run_with_tz() (*no módulo test.support*), 1938
runcall() (*método bdb.Bdb*), 1960
runcall() (*método pdb.Pdb*), 1968
runcall() (*método profile.Profile*), 1979
runcall() (*no módulo pdb*), 1966
runcode() (*método code.InteractiveInterpreter*), 2156
runctx() (*método bdb.Bdb*), 1960
runctx() (*método profile.Profile*), 1979
runctx() (*método trace.Trace*), 1990
runctx() (*no módulo profile*), 1978
runeval() (*método bdb.Bdb*), 1960
runeval() (*método pdb.Pdb*), 1968
runeval() (*no módulo pdb*), 1966
runfunc() (*método trace.Trace*), 1990
Runner (*classe em asyncio*), 1139
running() (*método concurrent.futures.Future*), 1097
runpy
    module, 2165
runsource() (*método code.InteractiveInterpreter*), 2156
runtime (*atributo sys._emscripten_info*), 2025
runtime_checkable() (*no módulo typing*), 1789
RuntimeError, 120
RuntimeWarning, 125
RUSAGE_BOTH (*no módulo resource*), 2326
RUSAGE_CHILDREN (*no módulo resource*), 2326
RUSAGE_SELF (*no módulo resource*), 2326
RUSAGE_THREAD (*no módulo resource*), 2326
RWF_APPEND (*no módulo os*), 730
RWF_DSYNC (*no módulo os*), 730
RWF_HIPRI (*no módulo os*), 729
RWF_NOWAIT (*no módulo os*), 729
RWF_SYNC (*no módulo os*), 730

S

-S
    dis opção de linha de comando, 2266
-s
    calendar opção de linha de comando, 275
    compileall opção de linha de comando, 2262
    cProfile opção de linha de comando, 1977
    idle opção de linha de comando, 1709
    timeit opção de linha de comando, 1986
    trace opção de linha de comando, 1990
    unittest-discover opção de linha de comando, 1841
S (*no módulo re*), 156
S_ENFMT (*no módulo stat*), 510
S_IEXEC (*no módulo stat*), 510
S_IFBLK (*no módulo stat*), 508
S_IFCHR (*no módulo stat*), 508
S_IFDIR (*no módulo stat*), 508
S_IFDOOR (*no módulo stat*), 508
S_IFIFO (*no módulo stat*), 508
S_IFLNK (*no módulo stat*), 508
S_IFMT() (*no módulo stat*), 507
S_IFPORT (*no módulo stat*), 509
S_IFREG (*no módulo stat*), 508
S_IFSOCK (*no módulo stat*), 508
S_IFWHT (*no módulo stat*), 509
S_IMODE() (*no módulo stat*), 507
S_IREAD (*no módulo stat*), 510
S_IRGRP (*no módulo stat*), 509

S_IROTH (*no módulo stat*), 510
S_IRUSR (*no módulo stat*), 509
S_IRWXG (*no módulo stat*), 509
S_IRWXO (*no módulo stat*), 509
S_IRWXU (*no módulo stat*), 509
S_ISBLK() (*no módulo stat*), 506
S_ISCHR() (*no módulo stat*), 506
S_ISDIR() (*no módulo stat*), 506
S_ISDOOR() (*no módulo stat*), 506
S_ISFIFO() (*no módulo stat*), 506
S_ISGID (*no módulo stat*), 509
S_ISLNK() (*no módulo stat*), 506
S_ISPORT() (*no módulo stat*), 506
S_ISREG() (*no módulo stat*), 506
S_ISSOCK() (*no módulo stat*), 506
S_ISUID (*no módulo stat*), 509
S_ISVTX (*no módulo stat*), 509
S_ISWHT() (*no módulo stat*), 506
S_IWGRP (*no módulo stat*), 509
S_IWOTH (*no módulo stat*), 510
S_IWRITE (*no módulo stat*), 510
S_IWUSR (*no módulo stat*), 509
S_IXGRP (*no módulo stat*), 509
S_IXOTH (*no módulo stat*), 510
S_IXUSR (*no módulo stat*), 509
safe (*atributo uuid.SafeUUID*), 1575
safe_path (*atributo sys.flags*), 2028
safe_substitute() (*método string.Template*), 140
saferepr() (*no módulo pprint*), 328
SafeUUID (*classe em uuid*), 1574
same_files (*atributo filecmp.dircmp*), 513
same_quantum() (*método decimal.Context*), 392
same_quantum() (*método decimal.Decimal*), 385
samefile() (*método pathlib.Path*), 488
samefile() (*no módulo os.path*), 504
SameFileError, 525
sameopenfile() (*no módulo os.path*), 504
samesite (*atributo http.cookies.Morsel*), 1598
samestat() (*no módulo os.path*), 504
sample() (*no módulo random*), 410
samples() (*método statistics.NormalDist*), 429
SATURDAY (*no módulo calendar*), 272
save() (*método http.cookiejar.FileCookieJar*), 1603
save() (*método test.support.SaveSignals*), 1942
save() (*no módulo turtle*), 1744
SaveAs (*classe em tkinter.filedialog*), 1677
SAVEDCWD (*no módulo test.support.os_helper*), 1946
SaveFileDialog (*classe em tkinter.filedialog*), 1678
SaveKey() (*no módulo winreg*), 2298
SaveSignals (*classe em test.support*), 1942
savetty() (*no módulo curses*), 989
SAX2DOM (*classe em xml.dom.pulldom*), 1474
SAXException, 1475
SAXNotRecognizedException, 1476
SAXNotSupportedException, 1476
SAXParseException, 1475
scaleb() (*método decimal.Context*), 392
scaleb() (*método decimal.Decimal*), 385
scandir() (*no módulo os*), 744
scanf (*função C*), 166
sched
    module, 1123
SCHED_BATCH (*no módulo os*), 776
SCHED_DEADLINE (*no módulo os*), 776
SCHED_FIFO (*no módulo os*), 776
sched_get_priority_max() (*no módulo os*), 777
sched_get_priority_min() (*no módulo os*), 777
sched_getaffinity() (*no módulo os*), 777
sched_getparam() (*no módulo os*), 777
sched_getscheduler() (*no módulo os*), 777
SCHED_IDLE (*no módulo os*), 776
SCHED_NORMAL (*no módulo os*), 776
SCHED_OTHER (*no módulo os*), 776
sched_param (*classe em os*), 776
sched_priority (*atributo os.sched_param*), 776
SCHED_RESET_ON_FORK (*no módulo os*), 776
SCHED_RR (*no módulo os*), 776
sched_rr_get_interval() (*no módulo os*), 777
sched_setaffinity() (*no módulo os*), 777
sched_setparam() (*no módulo os*), 777
sched_setscheduler() (*no módulo os*), 777
SCHED_SPORADIC (*no módulo os*), 776
sched_yield() (*no módulo os*), 777
scheduler (*classe em sched*), 1123
SCM_CREDS2 (*no módulo socket*), 1250
scope_id (*atributo ipaddress.IPv6Address*), 1627
Screen (*classe em turtle*), 1745
screensize() (*no módulo turtle*), 1738
script_from_examples() (*no módulo doctest*), 1834
scroll() (*método curses.window*), 996
ScrolledCanvas (*classe em turtle*), 1745
ScrolledText (*classe em tkinter.scrolledtext*), 1681
scrollok() (*método curses.window*), 996
scrypt() (*no módulo hashlib*), 697
seal() (*no módulo unittest.mock*), 1908
search() (*método imaplib.IMAP4*), 1565
search() (*método re.Pattern*), 161
search() (*no módulo re*), 157
search_function() (*no módulo encodings*), 221
search_log (*atributo compression.zstd.CompressionParameter*), 606
second (*atributo datetime.datetime*), 239
second (*atributo datetime.time*), 247
seconds (*atributo datetime.timedelta*), 227
secrets
    module, 706
SECTCRE (*atributo configparser.ConfigParser*), 678
sections() (*método configparser.ConfigParser*), 681
secure (*atributo http.cookiejar.Cookie*), 1607
secure (*atributo http.cookies.Morsel*), 1598
secure hash algorithm, SHA1, SHA2, SHA224, SHA256, SHA384, SHA512, SHA3, Shake, Blake2, 693
Secure Sockets Layer, 1270
security
    http.server, 1596

security_level (*atributo ssl.SSLContext*), 1294
see() (*método tkinter.ttk.Treeview*), 1695
seed() (*método random.Random*), 412
seed() (*no módulo random*), 408
seed_bits (*atributo sys.hash_info*), 2035
seek() (*método io.IOBase*), 785
seek() (*método io.TextIOBase*), 791
seek() (*método io.TextIOWrapper*), 793
seek() (*método mmap.mmap*), 1329
seek() (*método sqlite3.Blob*), 586
SEEK_CUR (*no módulo os*), 725
SEEK_DATA (*no módulo os*), 726
SEEK_END (*no módulo os*), 725
SEEK_HOLE (*no módulo os*), 726
SEEK_SET (*no módulo os*), 725
seekable() (*método bz2.BZ2File*), 620
seekable() (*método io.IOBase*), 785
seekable() (*método mmap.mmap*), 1329
segundos desde a era, 795
select
    module, 1306
select() (*método imaplib.IMAP4*), 1565
select() (*método selectors.BaseSelector*), 1314
select() (*método tkinter.ttk.Notebook*), 1689
select() (*no módulo select*), 1307
selected_alpn_protocol() (*método ssl.SSLSocket*), 1286
selected_npn_protocol() (*método ssl.SSLSocket*), 1286
selection() (*método tkinter.ttk.Treeview*), 1695
selection_add() (*método tkinter.ttk.Treeview*), 1696
selection_remove() (*método tkinter.ttk.Treeview*), 1696
selection_set() (*método tkinter.ttk.Treeview*), 1695
selection_toggle() (*método tkinter.ttk.Treeview*), 1696
selector (*atributo urllib.request.Request*), 1517
SelectorEventLoop (*classe em asyncio*), 1206
SelectorKey (*classe em selectors*), 1313
selectors
    module, 1312
SelectSelector (*classe em selectors*), 1315
Self (*no módulo typing*), 1767
semáforos binários, 1133
Semaphore (*classe em asyncio*), 1172
Semaphore (*classe em multiprocessing*), 1058
Semaphore (*classe em threading*), 1034
Semaphore() (*método multiprocessing.managers.SyncManager*), 1064
SEMI (*no módulo token*), 2249
SEND (*opcode*), 2285
send() (*método http.client.HTTPConnection*), 1548
send() (*método imaplib.IMAP4*), 1565
send() (*método logging.handlers.DatagramHandler*), 845
send() (*método logging.handlers.SocketHandler*), 844
send() (*método multiprocessing.connection.Connection*), 1055
send() (*método socket.socket*), 1263
send_bytes() (*método multiprocessing.connection.Connection*), 1055
send_error() (*método http.server.BaseHTTPRequestHandler*), 1592
send_fds() (*no módulo socket*), 1259
send_header() (*método http.server.BaseHTTPRequestHandler*), 1592
send_message() (*método smtplib.SMTP*), 1573
send_response() (*método http.server.BaseHTTPRequestHandler*), 1592
send_response_only() (*método http.server.BaseHTTPRequestHandler*), 1593
send_signal() (*método asyncio.subprocess.Process*), 1177
send_signal() (*método asyncio.SubprocessTransport*), 1217
send_signal() (*método subprocess.Popen*), 1114
sendall() (*método socket.socket*), 1264
sendcmd() (*método ftplib.FTP*), 1552
sendfile() (*método asyncio.loop*), 1195
sendfile() (*método socket.socket*), 1265
sendfile() (*método wsgiref.handlers.BaseHandler*), 1508
sendfile() (*no módulo os*), 731
SendfileNotAvailableError, 1182
sendmail() (*método smtplib.SMTP*), 1572
sendmsg() (*método socket.socket*), 1264
sendmsg_afalg() (*método socket.socket*), 1265
sendto() (*método asyncio.DatagramTransport*), 1217
sendto() (*método socket.socket*), 1264
sentinel (*atributo multiprocessing.Process*), 1048
sentinel (*no módulo unittest.mock*), 1901
sep (*no módulo os*), 778
SEPTEMBER (*no módulo calendar*), 273
Sequence (*classe em collections.abc*), 297
Sequence (*classe em typing*), 1807
SequenceMatcher (*classe em difflib*), 175
sequência, **2360**
    iteração, 48
    objeto, 49
    tipos, imutável, 51
    tipos, mutável, 51
    tipos, operações em, 49, 51
sequência mutável
    laço sobre, 49
serialização
    objetos, 537
serialize() (*método sqlite3.Connection*), 581
serve_forever() (*método asyncio.Server*), 1206
serve_forever() (*método socketserver.BaseServer*), 1583
server
    WWW, 1589

server (*atributo http.server.BaseHTTPRequestHandler*), 1591
server (*atributo socketserver.BaseRequestHandler*), 1585
Server (*classe em asyncio*), 1205
server_activate() (*método socketserver.BaseServer*), 1585
server_address (*atributo socketserver.BaseServer*), 1584
server_bind() (*método socketserver.BaseServer*), 1585
server_close() (*método socketserver.BaseServer*), 1583
server_hostname (*atributo ssl.SSLSocket*), 1287
server_side (*atributo ssl.SSLSocket*), 1287
server_software (*atributo wsgiref.handlers.BaseHandler*), 1507
server_version (*atributo http.server.BaseHTTPRequestHandler*), 1591
server_version (*atributo http.server.SimpleHTTPRequestHandler*), 1594
ServerProxy (*classe em xmlrpc.client*), 1609
service_actions() (*método socketserver.BaseServer*), 1583
session (*atributo ssl.SSLSocket*), 1287
session_reused (*atributo ssl.SSLSocket*), 1287
session_stats() (*método ssl.SSLContext*), 1293
set
    objeto, 95
Set (*classe em ast*), 2211
Set (*classe em collections.abc*), 298
Set (*classe em typing*), 1804
set (*classe interna*), 95
Set Breakpoint, 1705
set() (*método asyncio.Event*), 1170
set() (*método configparser.ConfigParser*), 683
set() (*método configparser.RawConfigParser*), 685
set() (*método contextvars.ContextVar*), 1129
set() (*método http.cookies.Morsel*), 1599
set() (*método multiprocessing.managers.SyncManager*), 1064
set() (*método test.support.os_helper.EnvironmentVarGuard*), 1946
set() (*método threading.Event*), 1036
set() (*método tkinter.ttk.Combobox*), 1687
set() (*método tkinter.ttk.Spinbox*), 1688
set() (*método tkinter.ttk.Treeview*), 1696
set() (*método xml.etree.ElementTree.Element*), 1452
SET_ADD (*opcode*), 2275
set_allowed_domains() (*método http.cookiejar.DefaultCookiePolicy*), 1606
set_alpn_protocols() (*método ssl.SSLContext*), 1290
set_app() (*método wsgiref.simple_server.WSGIServer*), 1504
set_asyncgen_hooks() (*no módulo sys*), 2044
set_authorizer() (*método sqlite3.Connection*), 577
set_auto_history() (*no módulo readline*), 193
set_blocked_domains() (*método http.cookiejar.DefaultCookiePolicy*), 1606
set_blocking() (*no módulo os*), 732
set_boundary() (*método email.message.EmailMessage*), 1338
set_boundary() (*método email.message.Message*), 1377
set_break() (*método bdb.Bdb*), 1959
set_charset() (*método email.message.Message*), 1373
set_children() (*método tkinter.ttk.Treeview*), 1693
set_ciphers() (*método ssl.SSLContext*), 1290
set_completer() (*no módulo readline*), 194
set_completer_delims() (*no módulo readline*), 194
set_completion_display_matches_hook() (*no módulo readline*), 194
set_content() (*método email.contentmanager.ContentManager*), 1362
set_content() (*método email.message.EmailMessage*), 1340
set_content() (*no módulo email.contentmanager*), 1363
set_continue() (*método bdb.Bdb*), 1959
set_cookie() (*método http.cookiejar.CookieJar*), 1603
set_cookie_if_ok() (*método http.cookiejar.CookieJar*), 1603
set_coroutine_origin_tracking_depth() (*no módulo sys*), 2044
set_data() (*método importlib.abc.SourceLoader*), 2175
set_data() (*método importlib.machinery.SourceFileLoader*), 2179
set_date() (*método mailbox.MaildirMessage*), 1412
set_debug() (*método asyncio.loop*), 1202
set_debug() (*no módulo gc*), 2120
set_debuglevel() (*método ftplib.FTP*), 1552
set_debuglevel() (*método http.client.HTTPConnection*), 1546
set_debuglevel() (*método poplib.POP3*), 1558
set_debuglevel() (*método smtplib.SMTP*), 1570
set_default_backend() (*no módulo pdb*), 1967
set_default_executor() (*método asyncio.loop*), 1201
set_default_type() (*método email.message.EmailMessage*), 1338
set_default_type() (*método email.message.Message*), 1376
set_default_verify_paths() (*método ssl.SSLContext*), 1290
set_defaults() (*método argparse.ArgumentParser*), 934
set_defaults() (*método optparse.OptionParser*), 972
set_ecdh_curve() (*método ssl.SSLContext*), 1291
set_errno() (*no módulo ctypes*), 894
set_error_mode() (*no módulo msvcrt*), 2292

set_escdelay() (*no módulo curses*), 989
set_event_loop() (*método asyncio.AbstractEventLoopPolicy*), 1227
set_event_loop() (*no módulo asyncio*), 1185
set_event_loop_policy() (*no módulo asyncio*), 1227
set_events() (*no módulo sys.monitoring*), 2053
set_exception() (*método asyncio.Future*), 1211
set_exception() (*método concurrent.futures.Future*), 1098
set_exception_handler() (*método asyncio.loop*), 1201
set_executable() (*no módulo multiprocessing*), 1054
set_filter() (*método tkinter.filedialog.FileDialog*), 1678
set_flags() (*método mailbox.Maildir*), 1406
set_flags() (*método mailbox.MaildirMessage*), 1412
set_flags() (*método mailbox.mboxMessage*), 1414
set_flags() (*método mailbox.MMDFMessage*), 1418
set_forkserver_preload() (*no módulo multiprocessing*), 1054
set_from() (*método mailbox.mboxMessage*), 1414
set_from() (*método mailbox.MMDFMessage*), 1418
SET_FUNCTION_ATTRIBUTE (*opcode*), 2284
set_handle_inheritable() (*no módulo os*), 734
set_history_length() (*no módulo readline*), 192
set_info() (*método mailbox.Maildir*), 1406
set_info() (*método mailbox.MaildirMessage*), 1413
set_inheritable() (*método socket.socket*), 1265
set_inheritable() (*no módulo os*), 734
set_int_max_str_digits() (*no módulo sys*), 2041
set_labels() (*método mailbox.BabylMessage*), 1416
set_last_error() (*no módulo ctypes*), 894
set_local_events() (*no módulo sys.monitoring*), 2053
set_memlimit() (*no módulo test.support*), 1936
set_name() (*método asyncio.Task*), 1159
set_next() (*método bdb.Bdb*), 1958
set_nonstandard_attr() (*método http.cookiejar.Cookie*), 1608
set_npn_protocols() (*método ssl.SSLContext*), 1290
set_ok() (*método http.cookiejar.CookiePolicy*), 1604
set_param() (*método email.message.EmailMessage*), 1338
set_param() (*método email.message.Message*), 1376
set_pasv() (*método ftplib.FTP*), 1553
set_payload() (*método email.message.Message*), 1373
set_pledged_input_size() (*método compression.zstd.ZstdCompressor*), 602
set_policy() (*método http.cookiejar.CookieJar*), 1602
set_pre_input_hook() (*no módulo readline*), 193
set_progress_handler() (*método sqlite3.Connection*), 577
set_protocol() (*método asyncio.BaseTransport*), 1215
set_proxy() (*método urllib.request.Request*), 1517
set_psk_client_callback() (*método ssl.SSLContext*), 1295
set_psk_server_callback() (*método ssl.SSLContext*), 1295
set_quit() (*método bdb.Bdb*), 1959
set_result() (*método asyncio.Future*), 1211
set_result() (*método concurrent.futures.Future*), 1097
set_return() (*método bdb.Bdb*), 1959
set_running_or_notify_cancel() (*método concurrent.futures.Future*), 1097
set_selection() (*método tkinter.filedialog.FileDialog*), 1678
set_seq1() (*método difflib.SequenceMatcher*), 175
set_seq2() (*método difflib.SequenceMatcher*), 176
set_seqs() (*método difflib.SequenceMatcher*), 175
set_sequences() (*método mailbox.MH*), 1409
set_sequences() (*método mailbox.MHMessage*), 1415
set_server_documentation() (*método xmlrpc.server.DocCGIXMLRPCRequestHandler*), 1622
set_server_documentation() (*método xmlrpc.server.DocXMLRPCServer*), 1622
set_server_name() (*método xmlrpc.server.DocCGIXMLRPCRequestHandler*), 1622
set_server_name() (*método xmlrpc.server.DocXMLRPCServer*), 1622
set_server_title() (*método xmlrpc.server.DocCGIXMLRPCRequestHandler*), 1622
set_server_title() (*método xmlrpc.server.DocXMLRPCServer*), 1622
set_servername_callback() (*método ssl.SSLContext*), 1291
set_start_method() (*no módulo multiprocessing*), 1054
set_startup_hook() (*no módulo readline*), 193
set_step() (*método bdb.Bdb*), 1958
set_subdir() (*método mailbox.MaildirMessage*), 1412
set_tabsize() (*no módulo curses*), 989
set_task_factory() (*método asyncio.loop*), 1190
set_threshold() (*no módulo gc*), 2120
set_trace() (*método bdb.Bdb*), 1959
set_trace() (*método pdb.Pdb*), 1968
set_trace() (*no módulo bdb*), 1961
set_trace() (*no módulo pdb*), 1967
set_trace_async() (*no módulo pdb*), 1967
set_trace_callback() (*método sqlite3.Connection*), 577
set_tunnel() (*método http.client.HTTPConnection*), 1546
set_type() (*método email.message.Message*), 1377
set_unittest_reportflags() (*no módulo doctest*), 1828
set_unixfrom() (*método*

email.message.EmailMessage), 1335
set_unixfrom() (*método email.message.Message*), 1372
set_until() (*método bdb.Bdb*), 1959
SET_UPDATE (*opcode*), 2279
set_url() (*método urllib.robotparser.RobotFileParser*), 1538
set_usage() (*método optparse.OptionParser*), 972
set_userptr() (*método curses.panel.Panel*), 1015
set_visible() (*método mailbox.BabylMessage*), 1416
set_wakeup_fd() (*no módulo signal*), 1322
set_write_buffer_limits() (*método asyncio.WriteTransport*), 1216
setacl() (*método imaplib.IMAP4*), 1565
setannotation() (*método imaplib.IMAP4*), 1565
setattr()
    built-in function, 29
setAttribute() (*método xml.dom.Element*), 1464
setAttributeNode() (*método xml.dom.Element*), 1464
setAttributeNodeNS() (*método xml.dom.Element*), 1465
setAttributeNS() (*método xml.dom.Element*), 1465
SetBase() (*método xml.parsers.expat.xmlparser*), 1488
setblocking() (*método socket.socket*), 1265
setByteStream() (*método xml.sax.xmlreader.InputSource*), 1485
setcbreak() (*no módulo tty*), 2317
setCharacterStream() (*método xml.sax.xmlreader.InputSource*), 1485
SetComp (*classe em ast*), 2216
setcomptype() (*método wave.Wave_write*), 1641
setconfig() (*método sqlite3.Connection*), 581
setContentHandler() (*método xml.sax.xmlreader.XMLReader*), 1483
setcontext() (*no módulo decimal*), 387
setDaemon() (*método threading.Thread*), 1030
setdefault() (*método dict*), 100
setdefault() (*método http.cookies.Morsel*), 1599
setdefaulttimeout() (*no módulo socket*), 1258
setdlopenflags() (*no módulo sys*), 2041
setDocumentLocator() (*método xml.sax.handler.ContentHandler*), 1478
setDTDHandler() (*método xml.sax.xmlreader.XMLReader*), 1483
setegid() (*no módulo os*), 718
setEncoding() (*método xml.sax.xmlreader.InputSource*), 1485
setEntityResolver() (*método xml.sax.xmlreader.XMLReader*), 1484
setErrorHandler() (*método xml.sax.xmlreader.XMLReader*), 1484
seteuid() (*no módulo os*), 718
setFeature() (*método xml.sax.xmlreader.XMLReader*), 1484
setfirstweekday() (*método calendar.Calendar*), 267
setfirstweekday() (*no módulo calendar*), 271
setFormatter() (*método logging.Handler*), 815
setframerate() (*método wave.Wave_write*), 1641
setgid() (*no módulo os*), 718
setgroups() (*no módulo os*), 718
seth() (*no módulo turtle*), 1722
setheading() (*no módulo turtle*), 1722
sethostname() (*no módulo socket*), 1258
setinputsizes() (*método sqlite3.Cursor*), 584
setitem() (*no módulo operator*), 466
setitimer() (*no módulo signal*), 1322
setLevel() (*método logging.Handler*), 815
setLevel() (*método logging.Logger*), 810
setlimit() (*método sqlite3.Connection*), 580
setLocale() (*método xml.sax.xmlreader.XMLReader*), 1484
setlocale() (*no módulo locale*), 1651
setLoggerClass() (*no módulo logging*), 825
setlogmask() (*no módulo syslog*), 2327
setLogRecordFactory() (*no módulo logging*), 825
setMaxConns() (*método urllib.request.CacheFTPHandler*), 1523
setmode() (*no módulo msvcrt*), 2292
setName() (*método threading.Thread*), 1029
setnchannels() (*método wave.Wave_write*), 1641
setnframes() (*método wave.Wave_write*), 1641
setns() (*no módulo os*), 718
setoutputsize() (*método sqlite3.Cursor*), 584
SetParamEntityParsing() (*método xml.parsers.expat.xmlparser*), 1488
setparams() (*método wave.Wave_write*), 1641
setpassword() (*método zipfile.ZipFile*), 634
setpgid() (*no módulo os*), 719
setpgrp() (*no módulo os*), 719
setpos() (*método wave.Wave_read*), 1640
setpos() (*no módulo turtle*), 1721
setposition() (*no módulo turtle*), 1721
setpriority() (*no módulo os*), 719
setprofile() (*no módulo sys*), 2042
setprofile() (*no módulo threading*), 1024
setprofile_all_threads() (*no módulo threading*), 1024
setProperty() (*método xml.sax.xmlreader.XMLReader*), 1484
setPublicId() (*método xml.sax.xmlreader.InputSource*), 1485
setquota() (*método imaplib.IMAP4*), 1565
setraw() (*no módulo tty*), 2317
setrecursionlimit() (*no módulo sys*), 2042
setregid() (*no módulo os*), 719
SetReparseDeferralEnabled() (*método xml.parsers.expat.xmlparser*), 1488
setresgid() (*no módulo os*), 719
setresuid() (*no módulo os*), 719
setreuid() (*no módulo os*), 719
setrlimit() (*no módulo resource*), 2323
setsampwidth() (*método wave.Wave_write*), 1641
setscrreg() (*método curses.window*), 996
setsid() (*no módulo os*), 720

setsockopt() (*método socket.socket*), 1265
setstate() (*método codecs.IncrementalDecoder*), 211
setstate() (*método codecs.IncrementalEncoder*), 211
setstate() (*método random.Random*), 412
setstate() (*no módulo random*), 408
setStream() (*método logging.StreamHandler*), 839
setswitchinterval() (*no módulo sys*), 2042
setswitchinterval() (*no módulo test.support*), 1936
setSystemId() (*método xml.sax.xmlreader.InputSource*), 1485
setsyx() (*no módulo curses*), 989
setTarget() (*método logging.handlers.MemoryHandler*), 849
settimeout() (*método socket.socket*), 1265
setTimeout() (*método urllib.request.CacheFTPHandler*), 1523
settrace() (*no módulo sys*), 2043
settrace() (*no módulo threading*), 1024
settrace_all_threads() (*no módulo threading*), 1024
setuid() (*no módulo os*), 720
setundobuffer() (*no módulo turtle*), 1736
--setup
    timeit opção de linha de comando, 1986
setup() (*método socketserver.BaseRequestHandler*), 1585
setUp() (*método unittest.TestCase*), 1847
setup() (*no módulo turtle*), 1744
SETUP_ANNOTATIONS (*opcode*), 2275
SETUP_CLEANUP (*opcode*), 2287
setup_environ() (*método wsgiref.handlers.BaseHandler*), 1507
SETUP_FINALLY (*opcode*), 2287
setup_python() (*método venv.EnvBuilder*), 2010
setup_scripts() (*método venv.EnvBuilder*), 2010
setup_testing_defaults() (*no módulo wsgiref.util*), 1501
SETUP_WITH (*opcode*), 2287
setUpClass() (*método unittest.TestCase*), 1847
setUpModule() (*no módulo unittest*), 1867
setupterm() (*no módulo curses*), 989
SetValue() (*no módulo winreg*), 2298
SetValueEx() (*no módulo winreg*), 2299
setworldcoordinates() (*no módulo turtle*), 1739
setx() (*no módulo turtle*), 1722
setxattr() (*no módulo os*), 761
sety() (*no módulo turtle*), 1722
SF_APPEND (*no módulo stat*), 511
SF_ARCHIVED (*no módulo stat*), 511
SF_DATALESS (*no módulo stat*), 511
SF_FIRMLINK (*no módulo stat*), 511
SF_IMMUTABLE (*no módulo stat*), 511
SF_MNOWAIT (*no módulo os*), 731
SF_NOCACHE (*no módulo os*), 731
SF_NODISKIO (*no módulo os*), 731
SF_NOUNLINK (*no módulo stat*), 511
SF_RESTRICTED (*no módulo stat*), 511
SF_SETTABLE (*no módulo stat*), 510
SF_SNAPSHOT (*no módulo stat*), 511
SF_SUPPORTED (*no módulo stat*), 511
SF_SYNC (*no módulo os*), 731
SF_SYNTHETIC (*no módulo stat*), 511
sha1() (*no módulo hashlib*), 694
sha3_224() (*no módulo hashlib*), 695
sha3_256() (*no módulo hashlib*), 695
sha3_384() (*no módulo hashlib*), 695
sha3_512() (*no módulo hashlib*), 695
sha224() (*no módulo hashlib*), 694
sha256() (*no módulo hashlib*), 695
sha384() (*no módulo hashlib*), 695
sha512() (*no módulo hashlib*), 695
shake_128() (*no módulo hashlib*), 696
shake_256() (*no módulo hashlib*), 696
shape (*atributo memoryview*), 95
Shape (*classe em turtle*), 1745
shape() (*no módulo turtle*), 1732
shapesize() (*no módulo turtle*), 1733
shapetransform() (*no módulo turtle*), 1734
share() (*método socket.socket*), 1266
ShareableList (*classe em multiprocessing.shared_memory*), 1088
ShareableList() (*método multiprocessing.managers.SharedMemoryManager*), 1087
shared_ciphers() (*método ssl.SSLSocket*), 1285
shared_memory (*atributo sys._emscripten_info*), 2025
SharedMemory (*classe em multiprocessing.shared_memory*), 1085
SharedMemory() (*método multiprocessing.managers.SharedMemoryManager*), 1087
SharedMemoryManager (*classe em multiprocessing.managers*), 1087
shearfactor() (*no módulo turtle*), 1733
Shelf (*classe em shelve*), 556
shelve
    module, 555
    módulo, 558
shield() (*no módulo asyncio*), 1150
shift() (*método decimal.Context*), 392
shift() (*método decimal.Decimal*), 385
shift_path_info() (*no módulo wsgiref.util*), 1501
shlex
    module, 2307
shlex (*classe em shlex*), 2308
shm (*atributo multiprocessing.shared_memory.ShareableList*), 1089
SHORT_TIMEOUT (*no módulo test.support*), 1934
shortDescription() (*método unittest.TestCase*), 1856
shorten() (*no módulo textwrap*), 184
shouldFlush() (*método logging.handlers.BufferingHandler*), 849
shouldFlush() (*método logging.handlers.MemoryHandler*), 849

shouldRollover() (*método logging.handlers.RotatingFileHandler*), 842
shouldRollover() (*método logging.handlers.TimedRotatingFileHandler*), 843
shouldStop (*atributo unittest.TestResult*), 1862
show() (*método curses.panel.Panel*), 1015
show() (*método tkinter.commondialog.Dialog*), 1679
show() (*método tkinter.messagebox.Message*), 1680
show_code() (*no módulo dis*), 2268
show_flag_values() (*no módulo enum*), 351
--show-caches
    dis opção de linha de comando, 2266
--show-empty
    ast opção de linha de comando, 2242
showerror() (*no módulo tkinter.messagebox*), 1680
showinfo() (*no módulo tkinter.messagebox*), 1680
--show-offsets
    dis opção de linha de comando, 2266
--show-positions
    dis opção de linha de comando, 2266
showsyntaxerror() (*método code.InteractiveInterpreter*), 2156
showtraceback() (*método code.InteractiveInterpreter*), 2156
showturtle() (*no módulo turtle*), 1732
showwarning() (*no módulo tkinter.messagebox*), 1680
showwarning() (*no módulo warnings*), 2073
shuffle() (*no módulo random*), 410
SHUT_RD (*no módulo socket*), 1251
SHUT_RDWR (*no módulo socket*), 1251
SHUT_WR (*no módulo socket*), 1251
ShutDown, 1126
shutdown() (*método asyncio.Queue*), 1180
shutdown() (*método concurrent.futures.Executor*), 1092
shutdown() (*método imaplib.IMAP4*), 1566
shutdown() (*método multiprocessing.managers.BaseManager*), 1062
shutdown() (*método queue.Queue*), 1127
shutdown() (*método socketserver.BaseServer*), 1583
shutdown() (*método socket.socket*), 1266
shutdown() (*no módulo logging*), 824
shutdown_asyncgens() (*método asyncio.loop*), 1187
shutdown_default_executor() (*método asyncio.loop*), 1187
shutil
    module, 524
SI (*no módulo curses.ascii*), 1011
side_effect (*atributo unittest.mock.Mock*), 1876
SIG_BLOCK (*no módulo signal*), 1319
SIG_DFL (*no módulo signal*), 1317
SIG_IGN (*no módulo signal*), 1317
SIG_SETMASK (*no módulo signal*), 1320
SIG_UNBLOCK (*no módulo signal*), 1320
SIGABRT (*no módulo signal*), 1317
SIGALRM (*no módulo signal*), 1317
SIGBREAK (*no módulo signal*), 1317
SIGBUS (*no módulo signal*), 1318
SIGCHLD (*no módulo signal*), 1318
SIGCLD (*no módulo signal*), 1318
SIGCONT (*no módulo signal*), 1318
SIGFPE (*no módulo signal*), 1318
SIGHUP (*no módulo signal*), 1318
SIGILL (*no módulo signal*), 1318
SIGINT (*no módulo signal*), 1318
siginterrupt() (*no módulo signal*), 1322
SIGKILL (*no módulo signal*), 1318
Sigmasks (*classe em signal*), 1317
signal
    module, 1316
    módulo, 1135
signal() (*no módulo signal*), 1323
Signals (*classe em signal*), 1317
signature (*atributo inspect.BoundArguments*), 2134
Signature (*classe em inspect*), 2130
signature() (*no módulo inspect*), 2129
sigpending() (*no módulo signal*), 1323
SIGPIPE (*no módulo signal*), 1318
SIGSEGV (*no módulo signal*), 1318
SIGSTKFLT (*no módulo signal*), 1318
SIGTERM (*no módulo signal*), 1319
sigtimedwait() (*no módulo signal*), 1323
SIGUSR1 (*no módulo signal*), 1319
SIGUSR2 (*no módulo signal*), 1319
sigwait() (*no módulo signal*), 1323
sigwaitinfo() (*no módulo signal*), 1323
SIGWINCH (*no módulo signal*), 1319
SIMPLE (*atributo inspect.BufferFlags*), 2142
Simple Mail Transfer Protocol, 1568
SimpleCookie (*classe em http.cookies*), 1597
simplefilter() (*no módulo warnings*), 2073
SimpleHandler (*classe em wsgiref.handlers*), 1506
SimpleHTTPRequestHandler (*classe em http.server*), 1593
SimpleNamespace (*classe em types*), 325
SimpleQueue (*classe em multiprocessing*), 1052
SimpleQueue (*classe em queue*), 1126
SimpleXMLRPCRequestHandler (*classe em xmlrpc.server*), 1617
SimpleXMLRPCServer (*classe em xmlrpc.server*), 1617
sin() (*no módulo cmath*), 372
sin() (*no módulo math*), 368
SingleAddressHeader (*classe em email.headerregistry*), 1358
singledispatch() (*no módulo functools*), 458
singledispatchmethod (*classe em functools*), 461
sinh() (*no módulo cmath*), 372
sinh() (*no módulo math*), 368
SIO_KEEPALIVE_VALS (*no módulo socket*), 1248
SIO_LOOPBACK_FAST_PATH (*no módulo socket*), 1248
SIO_RCVALL (*no módulo socket*), 1248
site
    module, 2151
site opção de linha de comando
    --user-base, 2154

--user-site, 2154
site_maps() (*método urllib.robotparser.RobotFileParser*), 1539
sitecustomize
module, 2153
site-packages
diretório, 2151
sixtofour (*atributo ipaddress.IPv6Address*), 1627
size (*atributo ctypes.CField*), 902
size (*atributo multiprocessing.shared_memory.SharedMemory*), 1086
size (*atributo struct.Struct*), 204
size (*atributo tarfile.TarInfo*), 649
size (*atributo tracemalloc.Statistic*), 2000
size (*atributo tracemalloc.StatisticDiff*), 2000
size (*atributo tracemalloc.Trace*), 2000
size() (*método ftplib.FTP*), 1555
size() (*método mmap.mmap*), 1329
size_diff (*atributo tracemalloc.StatisticDiff*), 2000
Sized (*classe em collections.abc*), 297
Sized (*classe em typing*), 1808
sizeof() (*no módulo ctypes*), 894
sizeof_digit (*atributo sys.int_info*), 2036
SKIP (*no módulo doctest*), 1822
skip() (*no módulo unittest*), 1845
skip_if_broken_multiprocessing_synchronize() (*no módulo test.support*), 1941
skip_unless_bind_unix_socket() (*no módulo test.support.socket_helper*), 1943
skip_unless_symlink() (*no módulo test.support.os_helper*), 1947
skip_unless_xattr() (*no módulo test.support.os_helper*), 1947
skipIf() (*no módulo unittest*), 1845
skipinitialspace (*atributo csv.Dialect*), 664
skipped (*atributo doctest.TestResults*), 1831
skipped (*atributo unittest.TestResult*), 1862
skippedEntity() (*método xml.sax.handler.ContentHandler*), 1480
skips (*atributo doctest.DocTestRunner*), 1833
SkipTest, 1846
skipTest() (*método unittest.TestCase*), 1848
skipUnless() (*no módulo unittest*), 1846
SLASH (*no módulo token*), 2250
SLASHEQUAL (*no módulo token*), 2251
sleep() (*no módulo asyncio*), 1147
sleep() (*no módulo time*), 799
sleeping_retry() (*no módulo test.support*), 1935
Slice (*classe em ast*), 2216
slice (*classe interna*), 30
slow_callback_duration (*atributo asyncio.loop*), 1202
SMALLEST (*no módulo test.support*), 1935
SMTP
protocolo, 1568
SMTP (*classe em smtplib*), 1568
SMTP (*no módulo email.policy*), 1353
SMTP_SSL (*classe em smtplib*), 1568
SMTPAuthenticationError, 1569
SMTPConnectError, 1569
smtpd
module, 2340
SMTPDataError, 1569
SMTPException, 1569
SMTPHandler (*classe em logging.handlers*), 848
SMTPHeloError, 1569
smtplib
module, 1568
SMTPNotSupportedError, 1569
SMTPRecipientsRefused, 1569
SMTPResponseException, 1569
SMTPSenderRefused, 1569
SMTPServerDisconnected, 1569
SMTPUTF8 (*no módulo email.policy*), 1353
Snapshot (*classe em tracemalloc*), 1998
SND_ALIAS (*no módulo winsound*), 2303
SND_APPLICATION (*no módulo winsound*), 2304
SND_ASYNC (*no módulo winsound*), 2304
SND_FILENAME (*no módulo winsound*), 2303
SND_LOOP (*no módulo winsound*), 2303
SND_MEMORY (*no módulo winsound*), 2303
SND_NODEFAULT (*no módulo winsound*), 2304
SND_NOSTOP (*no módulo winsound*), 2304
SND_NOWAIT (*no módulo winsound*), 2304
SND_PURGE (*no módulo winsound*), 2304
SND_SENTRY (*no módulo winsound*), 2304
SND_SYNC (*no módulo winsound*), 2304
SND_SYSTEM (*no módulo winsound*), 2304
sndhdr
module, 2340
sni_callback (*atributo ssl.SSLContext*), 1290
sniff() (*método csv.Sniffer*), 662
Sniffer (*classe em csv*), 662
SO (*no módulo curses.ascii*), 1011
SO_HCI_EVT_FILTER (*no módulo socket*), 1250
SO_HCI_PKT_FILTER (*no módulo socket*), 1250
SO_INCOMING_CPU (*no módulo socket*), 1250
SO_REUSEPORT_LB (*no módulo socket*), 1251
sock_accept() (*método asyncio.loop*), 1198
SOCK_CLOEXEC (*no módulo socket*), 1246
sock_connect() (*método asyncio.loop*), 1198
SOCK_DGRAM (*no módulo socket*), 1245
SOCK_MAX_SIZE (*no módulo test.support*), 1934
SOCK_NONBLOCK (*no módulo socket*), 1246
SOCK_RAW (*no módulo socket*), 1245
SOCK_RDM (*no módulo socket*), 1245
sock_recv() (*método asyncio.loop*), 1197
sock_recv_into() (*método asyncio.loop*), 1197
sock_recvfrom() (*método asyncio.loop*), 1197
sock_recvfrom_into() (*método asyncio.loop*), 1197
sock_sendall() (*método asyncio.loop*), 1197
sock_sendfile() (*método asyncio.loop*), 1198
sock_sendto() (*método asyncio.loop*), 1198
SOCK_SEQPACKET (*no módulo socket*), 1245
SOCK_STREAM (*no módulo socket*), 1245
socket

module, 1241
módulo, 1497
objeto, 1241
socket (*atributo socketserver.BaseServer*), 1584
socket (*classe em socket*), 1251
socket() (*in module socket*), 1307
socket() (*método imaplib.IMAP4*), 1566
socket_type (*atributo socketserver.BaseServer*), 1584
SocketHandler (*classe em logging.handlers*), 844
socketpair() (*no módulo socket*), 1252
sockets (*atributo asyncio.Server*), 1206
socketserver
module, 1580
SocketType (*no módulo socket*), 1253
SOFT_KEYWORD (*no módulo token*), 2249
softkwlist (*no módulo keyword*), 2252
SOH (*no módulo curses.ascii*), 1011
SOL_ALG (*no módulo socket*), 1248
SOL_BLUETOOTH (*no módulo socket*), 1249
SOL_HCI (*no módulo socket*), 1249
SOL_L2CAP (*no módulo socket*), 1249
SOL_RDS (*no módulo socket*), 1248
SOL_RFCOMM (*no módulo socket*), 1249
SOL_SCO (*no módulo socket*), 1249
soma de verificação
Do inglês: Cyclic Redundancy Check, 612
SOMAXCONN (*no módulo socket*), 1246
sort() (*método imaplib.IMAP4*), 1566
sort() (*método list*), 53
sort_stats() (*método pstats.Stats*), 1980
sortdict() (*no módulo test.support*), 1936
sorted()
built-in function, 30
--sort-keys
json opção de linha de comando, 1401
sortTestMethodsUsing (*atributo unittest.TestLoader*), 1861
source (*atributo doctest.Example*), 1830
source (*atributo shlex.shlex*), 2310
source (*pdb command*), 1972
SOURCE_DATE_EPOCH, 2260, 2262
source_from_cache() (*no módulo importlib.util*), 2183
source_hash() (*no módulo importlib.util*), 2184
SOURCE_SUFFIXES (*no módulo importlib.machinery*), 2177
source_to_code() (*método estático importlib.abc.InspectLoader*), 2173
SourceFileLoader (*classe em importlib.machinery*), 2179
sourcehook() (*método shlex.shlex*), 2309
SourcelessFileLoader (*classe em importlib.machinery*), 2180
SourceLoader (*classe em importlib.abc*), 2174
SP (*no módulo curses.ascii*), 1012
--spacing
calendar opção de linha de comando, 275
span() (*método re.Match*), 165
sparse (*atributo tarfile.TarInfo*), 650
spawn() (*no módulo pty*), 2318
spawn_python() (*no módulo test.support.script_helper*), 1944
spawnl() (*no módulo os*), 769
spawnle() (*no módulo os*), 769
spawnlp() (*no módulo os*), 769
spawnlpe() (*no módulo os*), 769
spawnv() (*no módulo os*), 769
spawnve() (*no módulo os*), 769
spawnvp() (*no módulo os*), 769
spawnvpe() (*no módulo os*), 769
spec de módulo, **2356**
spec_from_file_location() (*no módulo importlib.util*), 2184
spec_from_loader() (*no módulo importlib.util*), 2184
SpecialFileError, 525, 644
--specialized
dis opção de linha de comando, 2266
specified_attributes (*atributo xml.parsers.expat.xmlparser*), 1489
speed() (*no módulo turtle*), 1724
Spinbox (*classe em tkinter.ttk*), 1687
splice() (*no módulo os*), 732
SPLICE_F_MORE (*no módulo os*), 732
SPLICE_F_MOVE (*no módulo os*), 732
SPLICE_F_NONBLOCK (*no módulo os*), 732
split() (*método BaseExceptionGroup*), 126
split() (*método bytearray*), 79
split() (*método bytes*), 79
split() (*método re.Pattern*), 162
split() (*método str*), 64
split() (*no módulo os.path*), 504
split() (*no módulo re*), 157
split() (*no módulo shlex*), 2307
splitdrive() (*no módulo os.path*), 504
splitext() (*no módulo os.path*), 505
splitlines() (*método bytearray*), 83
splitlines() (*método bytes*), 83
splitlines() (*método str*), 65
SplitResult (*classe em urllib.parse*), 1535
SplitResultBytes (*classe em urllib.parse*), 1535
splitroot() (*no módulo os.path*), 505
SpooledTemporaryFile (*classe em tempfile*), 516
sprintf, estilo de formatação, 69, 85
spwd
module, 2340
sqlite3
module, 566
SQLITE_DBCONFIG_DEFENSIVE (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_DQS_DDL (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_DQS_DML (*no módulo sqlite3*), 572

SQLITE_DBCONFIG_ENABLE_FKEY (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_ENABLE_FTS3_TOKENIZER (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_ENABLE_LOAD_EXTENSION (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_ENABLE_QPSG (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_ENABLE_TRIGGER (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_ENABLE_VIEW (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_LEGACY_ALTER_TABLE (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_LEGACY_FILE_FORMAT (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_NO_CKPT_ON_CLOSE (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_RESET_DATABASE (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_TRIGGER_EQP (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_TRUSTED_SCHEMA (*no módulo sqlite3*), 572
SQLITE_DBCONFIG_WRITABLE_SCHEMA (*no módulo sqlite3*), 572
SQLITE_DENY (*no módulo sqlite3*), 571
sqlite_errorcode (*atributo sqlite3.Error*), 587
sqlite_errorname (*atributo sqlite3.Error*), 587
SQLITE_IGNORE (*no módulo sqlite3*), 571
SQLITE_OK (*no módulo sqlite3*), 571
sqlite_version (*no módulo sqlite3*), 571
sqlite_version_info (*no módulo sqlite3*), 571
sqrt() (*método decimal.Context*), 392
sqrt() (*método decimal.Decimal*), 385
sqrt() (*no módulo cmath*), 372
sqrt() (*no módulo math*), 366
SSL, 1270
ssl
    module, 1270
ssl_version (*atributo ftplib.FTP_TLS*), 1556
SSLCertVerificationError, 1274
SSLContext (*classe em ssl*), 1287
SSLEOFError, 1274
SSLError, 1273
SSLErrorNumber (*classe em ssl*), 1282
SSLKEYLOGFILE, 1272, 1273
SSLObject (*classe em ssl*), 1302
sslobject_class (*atributo ssl.SSLContext*), 1293
SSLSession (*classe em ssl*), 1303
SSLSocket (*classe em ssl*), 1283
sslsocket_class (*atributo ssl.SSLContext*), 1292
SSLSyscallError, 1273
SSLv3 (*atributo ssl.TLSVersion*), 1282
SSLWantReadError, 1273
SSLWantWriteError, 1273
SSLZeroReturnError, 1273
st() (*no módulo turtle*), 1732
st_atime (*atributo os.stat_result*), 748
ST_ATIME (*no módulo stat*), 508
st_atime_ns (*atributo os.stat_result*), 749
st_birthtime (*atributo os.stat_result*), 749
st_birthtime_ns (*atributo os.stat_result*), 749
st_blksize (*atributo os.stat_result*), 749
st_blocks (*atributo os.stat_result*), 749
st_creator (*atributo os.stat_result*), 750
st_ctime (*atributo os.stat_result*), 749
ST_CTIME (*no módulo stat*), 508
st_ctime_ns (*atributo os.stat_result*), 749
st_dev (*atributo os.stat_result*), 748
ST_DEV (*no módulo stat*), 508
st_file_attributes (*atributo os.stat_result*), 750
st_flags (*atributo os.stat_result*), 750
st_fstype (*atributo os.stat_result*), 750
st_gen (*atributo os.stat_result*), 750
st_gid (*atributo os.stat_result*), 748
ST_GID (*no módulo stat*), 508
st_ino (*atributo os.stat_result*), 748
ST_INO (*no módulo stat*), 507
st_mode (*atributo os.stat_result*), 748
ST_MODE (*no módulo stat*), 507
st_mtime (*atributo os.stat_result*), 748
ST_MTIME (*no módulo stat*), 508
st_mtime_ns (*atributo os.stat_result*), 749
st_nlink (*atributo os.stat_result*), 748
ST_NLINK (*no módulo stat*), 508
st_rdev (*atributo os.stat_result*), 750
st_reparse_tag (*atributo os.stat_result*), 750
st_rsize (*atributo os.stat_result*), 750
st_size (*atributo os.stat_result*), 748
ST_SIZE (*no módulo stat*), 508
st_type (*atributo os.stat_result*), 750
st_uid (*atributo os.stat_result*), 748
ST_UID (*no módulo stat*), 508
stack (*atributo traceback.TracebackException*), 2111
stack viewer, 1704
stack() (*no módulo inspect*), 2138
stack_effect() (*no módulo dis*), 2270
stack_size() (*no módulo _thread*), 1134
stack_size() (*no módulo threading*), 1024
stackable
    streams, 204
StackSummary (*classe em traceback*), 2112
stamp() (*no módulo turtle*), 1724
standard_b64decode() (*no módulo base64*), 1426
standard_b64encode() (*no módulo base64*), 1426
standend() (*método curses.window*), 996
standout() (*método curses.window*), 997
STAR (*no módulo token*), 2250
STAREQUAL (*no módulo token*), 2251
starmap() (*método multiprocessing.pool.Pool*), 1070
starmap() (*no módulo itertools*), 444
starmap_async() (*método multiprocessing.pool.Pool*), 1070
Starred (*classe em ast*), 2212
start (*atributo range*), 54

start (*atributo slice*), 30
start (*atributo UnicodeError*), 122
start() (*método logging.handlers.QueueListener*), 852
start() (*método multiprocessing.managers.BaseManager*), 1062
start() (*método multiprocessing.Process*), 1047
start() (*método re.Match*), 164
start() (*método threading.Thread*), 1028
start() (*método tkinter.ttk.Progressbar*), 1690
start() (*método xml.etree.ElementTree.TreeBuilder*), 1455
start() (*no módulo tracemalloc*), 1997
start_color() (*no módulo curses*), 989
start_new_thread() (*no módulo _thread*), 1133
start_ns() (*método xml.etree.ElementTree.TreeBuilder*), 1456
start_server() (*no módulo asyncio*), 1162
start_serving() (*método asyncio.Server*), 1206
start_threads() (*no módulo test.support.threading_helper*), 1945
start_tls() (*método asyncio.loop*), 1196
start_tls() (*método asyncio.StreamWriter*), 1165
start_trace() (*método bdb.Bdb*), 1957
start_unix_server() (*no módulo asyncio*), 1163
StartBoundaryNotFoundDefect, 1355
startCDATA() (*método xml.sax.handler.LexicalHandler*), 1481
StartCdataSectionHandler() (*método xml.parsers.expat.xmlparser*), 1491
--start-directory
    unittest-discover opção de linha de comando, 1841
StartDoctypeDeclHandler() (*método xml.parsers.expat.xmlparser*), 1490
startDocument() (*método xml.sax.handler.ContentHandler*), 1478
startDTD() (*método xml.sax.handler.LexicalHandler*), 1481
startElement() (*método xml.sax.handler.ContentHandler*), 1479
StartElementHandler() (*método xml.parsers.expat.xmlparser*), 1490
startElementNS() (*método xml.sax.handler.ContentHandler*), 1479
STARTF_FORCEOFFFEEDBACK (*no módulo subprocess*), 1117
STARTF_FORCEONFEEDBACK (*no módulo subprocess*), 1116
STARTF_USESHOWWINDOW (*no módulo subprocess*), 1116
STARTF_USESTDHANDLES (*no módulo subprocess*), 1116
startfile() (*no módulo os*), 770
StartNamespaceDeclHandler() (*método xml.parsers.expat.xmlparser*), 1491
startPrefixMapping() (*método xml.sax.handler.ContentHandler*), 1478
StartResponse (*classe em wsgiref.types*), 1509
startswith() (*método bytearray*), 77
startswith() (*método bytes*), 77
startswith() (*método str*), 65
startTest() (*método unittest.TestResult*), 1863
startTestRun() (*método unittest.TestResult*), 1863
starttls() (*método imaplib.IMAP4*), 1566
starttls() (*método smtplib.SMTP*), 1572
STARTUPINFO (*classe em subprocess*), 1115
stat
    module, 506
    módulo, 747
stat() (*método os.DirEntry*), 747
stat() (*método pathlib.Path*), 486
stat() (*método poplib.POP3*), 1559
stat() (*no módulo os*), 747
stat_result (*classe em os*), 748
state() (*método tkinter.ttk.Widget*), 1686
static_order() (*método graphlib.TopologicalSorter*), 354
staticmethod()
    built-in function, 30
Statistic (*classe em tracemalloc*), 1999
StatisticDiff (*classe em tracemalloc*), 2000
statistics
    module, 418
statistics() (*método tracemalloc.Snapshot*), 1999
StatisticsError, 429
Stats (*classe em pstats*), 1979
status (*atributo http.client.HTTPResponse*), 1548
status (*atributo urllib.response.addinfourl*), 1528
status() (*método imaplib.IMAP4*), 1566
statvfs() (*no módulo os*), 751
STD_ERROR_HANDLE (*no módulo subprocess*), 1116
STD_INPUT_HANDLE (*no módulo subprocess*), 1116
STD_OUTPUT_HANDLE (*no módulo subprocess*), 1116
stderr (*atributo asyncio.subprocess.Process*), 1178
stderr (*atributo subprocess.CalledProcessError*), 1107
stderr (*atributo subprocess.CompletedProcess*), 1106
stderr (*atributo subprocess.Popen*), 1115
stderr (*atributo subprocess.TimeoutExpired*), 1106
stderr (*no módulo sys*), 2046
stdev (*atributo statistics.NormalDist*), 429
stdev() (*no módulo statistics*), 425
stdin (*atributo asyncio.subprocess.Process*), 1178
stdin (*atributo subprocess.Popen*), 1114
stdin (*no módulo sys*), 2046
stdlib, **2361**
stdlib_module_names (*no módulo sys*), 2047
stdout (*atributo asyncio.subprocess.Process*), 1178
stdout (*atributo subprocess.CalledProcessError*), 1107
stdout (*atributo subprocess.CompletedProcess*), 1106
stdout (*atributo subprocess.Popen*), 1115
stdout (*atributo subprocess.TimeoutExpired*), 1106
STDOUT (*no módulo subprocess*), 1106
stdout (*no módulo sys*), 2046
stem (*atributo pathlib.PurePath*), 479
step (*atributo range*), 54
step (*atributo slice*), 30

step (*pdb command*), 1971
step() (*método tkinter.ttk.Progressbar*), 1690
stls() (*método poplib.POP3*), 1559
stop (*atributo range*), 54
stop (*atributo slice*), 30
stop() (*método asyncio.loop*), 1186
stop() (*método logging.handlers.QueueListener*), 852
stop() (*método tkinter.ttk.Progressbar*), 1690
stop() (*método unittest.TestResult*), 1863
stop() (*no módulo tracemalloc*), 1997
stop_here() (*método bdb.Bdb*), 1958
STOP_ITERATION (*monitoring event*), 2051
stop_trace() (*método bdb.Bdb*), 1957
StopAsyncIteration, 120
StopIteration, 120
stopListening() (*no módulo logging.config*), 828
stopTest() (*método unittest.TestResult*), 1863
stopTestRun() (*método unittest.TestResult*), 1863
storbinary() (*método ftplib.FTP*), 1553
Store (*classe em ast*), 2212
store() (*método imaplib.IMAP4*), 1566
STORE_ACTIONS (*atributo optparse.Option*), 978
STORE_ATTR (*opcode*), 2277
STORE_DEREF (*opcode*), 2282
STORE_FAST (*opcode*), 2282
STORE_FAST_LOAD_FAST (*opcode*), 2282
STORE_FAST_STORE_FAST (*opcode*), 2282
STORE_GLOBAL (*opcode*), 2277
STORE_NAME (*opcode*), 2277
STORE_SLICE (*opcode*), 2274
STORE_SUBSCR (*opcode*), 2273
storlines() (*método ftplib.FTP*), 1553
str (*classe embutida*)
    (veja também string), 55
str (*classe interna*), 57
str() (*no módulo locale*), 1657
str_digits_check_threshold (*atributo sys.int_info*), 2036
strategy (*atributo compression.zstd.CompressionParameter*), 607
Strategy (*classe em compression.zstd*), 609
strcoll() (*no módulo locale*), 1656
StreamError, 643
StreamHandler (*classe em logging*), 839
streamreader (*atributo codecs.CodecInfo*), 205
StreamReader (*classe em asyncio*), 1163
StreamReader (*classe em codecs*), 212
StreamReaderWriter (*classe em codecs*), 213
StreamRecoder (*classe em codecs*), 213
StreamRequestHandler (*classe em socketserver*), 1585
streams, 204
    stackable, 204
streamwriter (*atributo codecs.CodecInfo*), 205
StreamWriter (*classe em asyncio*), 1164
StreamWriter (*classe em codecs*), 212
StrEnum (*classe em enum*), 344
strerror (*atributo OSError*), 119
strerror() (*no módulo os*), 720
strftime() (*método datetime.date*), 233
strftime() (*método datetime.datetime*), 244
strftime() (*método datetime.time*), 249
strftime() (*no módulo time*), 800
strict
    error handler's name, 208
strict (*atributo csv.Dialect*), 665
STRICT (*atributo enum.FlagBoundary*), 348
strict (*no módulo email.policy*), 1354
strict_domain (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
strict_errors() (*no módulo codecs*), 209
strict_ns_domain (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
strict_ns_set_initial_dollar (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
strict_ns_set_path (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
strict_ns_unverifiable (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
strict_rfc2965_unverifiable (*atributo http.cookiejar.DefaultCookiePolicy*), 1606
STRIDED (*atributo inspect.BufferFlags*), 2142
STRIDED_RO (*atributo inspect.BufferFlags*), 2142
STRIDES (*atributo inspect.BufferFlags*), 2142
strides (*atributo memoryview*), 95
string
    format() (*função embutida*), 17
    formatação, printf, 69
    interpolação, printf, 69
    literal formatado, 67
    literal interpolado, 67
    métodos, 57
    module, 129
    objeto, 55
    str (*classe embutida*), 57
    str() (*função embutida*), 31
    tipo sequência de texto, 55
STRING (*atributo annotationlib.Format*), 2144
string (*atributo re.Match*), 165
STRING (*no módulo token*), 2247
string_at() (*no módulo ctypes*), 894
StringIO (*classe em io*), 793
stringprep
    module, 189
strings (*atributo string.templatelib.Template*), 143
string.templatelib
    module, 142
strip() (*método bytearray*), 80
strip() (*método bytes*), 80
strip() (*método str*), 65
strip_dirs() (*método pstats.Stats*), 1979
stripspaces (*atributo curses.textpad.Textbox*), 1010
strptime() (*método de classe datetime.date*), 230
strptime() (*método de classe datetime.datetime*), 238
strptime() (*método de classe datetime.time*), 248
strptime() (*no módulo time*), 802

strsignal() (*no módulo signal*), 1320
struct
    module, 197
    módulo, 1265
Struct (*classe em struct*), 204
struct_time (*classe em time*), 802
Structure (*classe em ctypes*), 899
structures
    C, 197
strxfrm() (*no módulo locale*), 1656
STX (*no módulo curses.ascii*), 1011
Style (*classe em tkinter.ttk*), 1696
suavemente descontinuado, **2361**
Sub (*classe em ast*), 2213
SUB (*no módulo curses.ascii*), 1012
sub() (*método re.Pattern*), 162
sub() (*no módulo operator*), 465
sub() (*no módulo re*), 159
subdirs (*atributo filecmp.dircmp*), 514
SubElement() (*no módulo xml.etree.ElementTree*), 1449
subgroup() (*método BaseExceptionGroup*), 126
submit() (*método concurrent.futures.Executor*), 1091
submodule_search_locations (*atributo importlib.machinery.ModuleSpec*), 2181
subn() (*método re.Pattern*), 162
subn() (*no módulo re*), 160
subnet_of() (*método ipaddress.IPv4Network*), 1631
subnet_of() (*método ipaddress.IPv6Network*), 1633
subnets() (*método ipaddress.IPv4Network*), 1631
subnets() (*método ipaddress.IPv6Network*), 1633
Subnormal (*classe em decimal*), 395
suboffsets (*atributo memoryview*), 95
subpad() (*método curses.window*), 997
subprocess
    module, 1104
subprocess_exec() (*método asyncio.loop*), 1203
subprocess_shell() (*método asyncio.loop*), 1204
SubprocessError, 1106
SubprocessProtocol (*classe em asyncio*), 1218
SubprocessTransport (*classe em asyncio*), 1214
subscribe() (*método imaplib.IMAP4*), 1566
subscrição
    atribuição, 51
    operação, 49
Subscript (*classe em ast*), 2215
subsequent_indent (*atributo textwrap.TextWrapper*), 186
substitute() (*método string.Template*), 140
subTest() (*método unittest.TestCase*), 1848
subtract() (*método collections.Counter*), 279
subtract() (*método decimal.Context*), 392
subtype (*atributo email.headerregistry.ContentTypeHeader*), 1359
subwin() (*método curses.window*), 997
successful() (*método multiprocessing.pool.AsyncResult*), 1070
suffix (*atributo pathlib.PurePath*), 478
suffix_map (*atributo mimetypes.MimeTypes*), 1423
suffix_map (*no módulo mimetypes*), 1422
suffixes (*atributo pathlib.PurePath*), 478
suiteClass (*atributo unittest.TestLoader*), 1861
sum()
    built-in function, 31
summarize() (*método doctest.DocTestRunner*), 1833
summarize_address_range() (*no módulo ipaddress*), 1636
--summary
    trace opção de linha de comando, 1990
sumprod() (*no módulo math*), 367
sunau
    module, 2341
SUNDAY (*no módulo calendar*), 272
super (*atributo pyclbr.Class*), 2259
super (*classe interna*), 31
supernet() (*método ipaddress.IPv4Network*), 1631
supernet() (*método ipaddress.IPv6Network*), 1633
supernet_of() (*método ipaddress.IPv4Network*), 1631
supernet_of() (*método ipaddress.IPv6Network*), 1633
supports_bytes_environ (*no módulo os*), 720
supports_dir_fd (*no módulo os*), 751
supports_effective_ids (*no módulo os*), 751
supports_fd (*no módulo os*), 752
supports_follow_symlinks (*no módulo os*), 752
supports_unicode_filenames (*no módulo os.path*), 506
SupportsAbs (*classe em typing*), 1793
SupportsBytes (*classe em typing*), 1794
SupportsComplex (*classe em typing*), 1794
SupportsFloat (*classe em typing*), 1794
SupportsIndex (*classe em typing*), 1794
SupportsInt (*classe em typing*), 1794
SupportsRound (*classe em typing*), 1794
suppress() (*no módulo contextlib*), 2090
SuppressCrashReport (*classe em test.support*), 1942
surrogateescape
    error handler's name, 208
surrogatepass
    error handler's name, 208
SW_HIDE (*no módulo subprocess*), 1116
SWAP (*opcode*), 2272
swap_attr() (*no módulo test.support*), 1937
swap_item() (*no módulo test.support*), 1937
swapcase() (*método bytearray*), 83
swapcase() (*método bytes*), 83
swapcase() (*método str*), 66
Symbol (*classe em symtable*), 2245
SymbolTable (*classe em symtable*), 2244
SymbolTableType (*classe em symtable*), 2243
symlink() (*no módulo os*), 752
symlink_to() (*método pathlib.Path*), 493
--symlinks
    venv opção de linha de comando, 2006
symmetric_difference() (*método frozenset*), 96

symmetric_difference_update() (*método frozenset*), 97
symtable
    module, 2243
symtable() (*no módulo symtable*), 2243
SYMTYPE (*no módulo tarfile*), 644
SYN (*no módulo curses.ascii*), 1012
sync() (*método dbm.dumb.dumbdbm*), 566
sync() (*método dbm.gnu.gdbm*), 563
sync() (*método shelve.Shelf*), 556
sync() (*no módulo os*), 753
syncdown() (*método curses.window*), 997
synchronized() (*no módulo multiprocessing.sharedctypes*), 1060
SyncManager (*classe em multiprocessing.managers*), 1063
syncok() (*método curses.window*), 997
syncup() (*método curses.window*), 997
SyntaxErr, 1467
SyntaxError, 120
SyntaxWarning, 125
sys
    module, 2021
    módulo, 26
sys_version (*atributo http.server.BaseHTTPRequestHandler*), 1591
sysconf() (*no módulo os*), 778
sysconf_names (*no módulo os*), 778
sysconfig
    module, 2054
syslog
    module, 2326
syslog() (*no módulo syslog*), 2327
SysLogHandler (*classe em logging.handlers*), 845
sys.monitoring
    module, 2049
system() (*no módulo os*), 771
system() (*no módulo platform*), 854
system_alias() (*no módulo platform*), 854
system_must_validate_cert() (*no módulo test.support*), 1938
SystemError, 121
SystemExit, 121
systemId (*atributo xml.dom.DocumentType*), 1463
SystemRandom (*classe em random*), 412
SystemRandom (*classe em secrets*), 707
SystemRoot, 1112
--system-site-packages
    venv opção de linha de comando, 2006

T

-T
    trace opção de linha de comando, 1989
-t
    calendar opção de linha de comando, 275
    idle opção de linha de comando, 1709
    tarfile opção de linha de comando, 655
    trace opção de linha de comando, 1989
    unittest-discover opção de linha de comando, 1841
    webbrowser opção de linha de comando, 1498
    zipfile opção de linha de comando, 640
t-string, **2361**
t-strings, **2361**
T_FMT (*no módulo locale*), 1653
T_FMT_AMPM (*no módulo locale*), 1653
--tab
    json opção de linha de comando, 1401
TAB (*no módulo curses.ascii*), 1011
tab() (*método tkinter.ttk.Notebook*), 1689
TabError, 121
tabnanny
    module, 2257
tabs() (*método tkinter.ttk.Notebook*), 1689
tabsize (*atributo textwrap.TextWrapper*), 185
tabular
    dados, 659
tag (*atributo xml.etree.ElementTree.Element*), 1451
tag_bind() (*método tkinter.ttk.Treeview*), 1696
tag_configure() (*método tkinter.ttk.Treeview*), 1696
tag_has() (*método tkinter.ttk.Treeview*), 1696
tagName (*atributo xml.dom.Element*), 1464
tail (*atributo xml.etree.ElementTree.Element*), 1451
take_snapshot() (*no módulo tracemalloc*), 1997
takewhile() (*no módulo itertools*), 444
tan() (*no módulo cmath*), 372
tan() (*no módulo math*), 368
tanh() (*no módulo cmath*), 372
tanh() (*no módulo math*), 368
tar_filter() (*no módulo tarfile*), 652
TarError, 643
tarfile
    module, 641
TarFile (*classe em tarfile*), 645
tarfile opção de linha de comando
    -c, 655
    --create, 655
    -e, 655
    --extract, 655
    --filter, 655
    -l, 655
    --list, 655
    -t, 655
    --test, 655
    -v, 655
    --verbose, 655
target (*atributo xml.dom.ProcessingInstruction*), 1466
target_length (*atributo compression.zstd.CompressionParameter*), 606
tarinfo (*atributo tarfile.FilterError*), 643
TarInfo (*classe em tarfile*), 649
Task (*classe em asyncio*), 1157
task_done() (*método asyncio.Queue*), 1180

task_done() (*método multiprocessing.JoinableQueue*), 1052
task_done() (*método queue.Queue*), 1127
TaskGroup (*classe em asyncio*), 1145
tau (*no módulo cmath*), 373
tau (*no módulo math*), 369
tb_locals (*atributo unittest.TestResult*), 1863
tbreak (*pdb command*), 1970
tcdrain() (*no módulo termios*), 2316
tcflow() (*no módulo termios*), 2316
tcflush() (*no módulo termios*), 2316
tcgetattr() (*no módulo termios*), 2315
tcgetpgrp() (*no módulo os*), 733
tcgetwinsize() (*no módulo termios*), 2316
Tcl() (*no módulo tkinter*), 1664
TCPServer (*classe em socketserver*), 1581
TCSADRAIN (*no módulo termios*), 2315
TCSAFLUSH (*no módulo termios*), 2316
TCSANOW (*no módulo termios*), 2315
tcsendbreak() (*no módulo termios*), 2316
tcsetattr() (*no módulo termios*), 2315
tcsetpgrp() (*no módulo os*), 733
tcsetwinsize() (*no módulo termios*), 2316
tearDown() (*método unittest.TestCase*), 1847
tearDownClass() (*método unittest.TestCase*), 1848
tearDownModule() (*no módulo unittest*), 1867
tee() (*no módulo itertools*), 445
teleport() (*no módulo turtle*), 1721
tell() (*método io.IOBase*), 785
tell() (*método io.TextIOBase*), 791
tell() (*método io.TextIOWrapper*), 793
tell() (*método mmap.mmap*), 1329
tell() (*método sqlite3.Blob*), 586
tell() (*método wave.Wave_read*), 1640
tell() (*método wave.Wave_write*), 1641
telnetlib
    module, 2341
TEMP, 517
temp_cwd() (*no módulo test.support.os_helper*), 1947
temp_dir() (*no módulo test.support.os_helper*), 1947
temp_umask() (*no módulo test.support.os_helper*), 1947
tempdir (*no módulo tempfile*), 518
tempfile
    module, 514
template (*atributo string.Template*), 140
Template (*classe em string*), 140
Template (*classe em string.templatelib*), 142
TemplateStr (*classe em ast*), 2210
tempo da CPU, 799, 804
tempo do processador, 799, 804
Tempo Médio de Greenwich, 795
Tempo Universal Coordenado, 795
temporary
    arquivo, 514
    file name, 514
temporary (*atributo bdb.Breakpoint*), 1956
TemporaryDirectory (*classe em tempfile*), 516
TemporaryFile() (*no módulo tempfile*), 514
teredo (*atributo ipaddress.IPv6Address*), 1627
TERM, 989
termattrs() (*no módulo curses*), 989
terminal_size (*classe em os*), 734
terminate() (*método asyncio.subprocess.Process*), 1177
terminate() (*método asyncio.SubprocessTransport*), 1217
terminate() (*método multiprocessing.pool.Pool*), 1070
terminate() (*método multiprocessing.Process*), 1048
terminate() (*método subprocess.Popen*), 1114
terminate_workers() (*método concurrent.futures.ProcessPoolExecutor*), 1096
terminator (*atributo logging.StreamHandler*), 839
termios
    module, 2315
termname() (*no módulo curses*), 989
--terse
    platform opção de linha de comando, 857
test
    module, 1930
--test
    tarfile opção de linha de comando, 655
    zipfile opção de linha de comando, 640
test (*atributo doctest.DocTestFailure*), 1836
test (*atributo doctest.UnexpectedException*), 1836
TEST_DATA_DIR (*no módulo test.support*), 1934
TEST_HOME_DIR (*no módulo test.support*), 1934
TEST_HTTP_URL (*no módulo test.support*), 1935
TEST_SUPPORT_DIR (*no módulo test.support*), 1934
TestCase (*classe em unittest*), 1847
TestFailed, 1933
testfile() (*no módulo doctest*), 1825
TESTFN (*no módulo test.support.os_helper*), 1946
TESTFN_NONASCII (*no módulo test.support.os_helper*), 1946
TESTFN_UNDECODABLE (*no módulo test.support.os_helper*), 1946
TESTFN_UNENCODABLE (*no módulo test.support.os_helper*), 1946
TESTFN_UNICODE (*no módulo test.support.os_helper*), 1946
TestLoader (*classe em unittest*), 1859
testMethodPrefix (*atributo unittest.TestLoader*), 1861
testmod() (*no módulo doctest*), 1826
testNamePatterns (*atributo unittest.TestLoader*), 1862
test.regrtest
    module, 1932
TestResult (*classe em unittest*), 1862
TestResults (*classe em doctest*), 1831
testsource() (*no módulo doctest*), 1835
testsRun (*atributo unittest.TestResult*), 1862
TestSuite (*classe em unittest*), 1859
test.support

module, 1933
test.support.bytecode_helper
module, 1944
test.support.import_helper
module, 1947
test.support.os_helper
module, 1946
test.support.script_helper
module, 1943
test.support.socket_helper
module, 1942
test.support.threading_helper
module, 1945
test.support.warnings_helper
module, 1949
testzip() (*método zipfile.ZipFile*), 634
text (*atributo ctypes.COMError*), 903
text (*atributo SyntaxError*), 121
text (*atributo traceback.TracebackException*), 2111
text (*atributo xml.etree.ElementTree.Element*), 1451
Text (*classe em typing*), 1805
text_encoding() (*no módulo io*), 783
text_factory (*atributo sqlite3.Connection*), 582
Textbox (*classe em curses.textpad*), 1009
TextCalendar (*classe em calendar*), 269
textdomain() (*no módulo gettext*), 1644
textdomain() (*no módulo locale*), 1659
textinput() (*no módulo turtle*), 1742
TextIO (*classe em typing*), 1794
TextIOBase (*classe em io*), 790
TextIOWrapper (*classe em io*), 791
texto, modo, 26
TextTestResult (*classe em unittest*), 1864
TextTestRunner (*classe em unittest*), 1864
textwrap
module, 183
TextWrapper (*classe em textwrap*), 185
TFD_CLOEXEC (*no módulo os*), 760
TFD_NONBLOCK (*no módulo os*), 760
TFD_TIMER_ABSTIME (*no módulo os*), 760
TFD_TIMER_CANCEL_ON_SET (*no módulo os*), 760
theme_create() (*método tkinter.ttk.Style*), 1700
theme_names() (*método tkinter.ttk.Style*), 1700
theme_settings() (*método tkinter.ttk.Style*), 1700
theme_use() (*método tkinter.ttk.Style*), 1700
THOUSEP (*no módulo locale*), 1654
Thread (*classe em threading*), 1027
thread() (*método imaplib.IMAP4*), 1566
thread_info (*no módulo sys*), 2047
thread_inherit_context (*atributo sys.flags*), 2028
thread_time() (*no módulo time*), 804
thread_time_ns() (*no módulo time*), 804
thread_unsafe() (*no módulo test.support*), 1939
threading
module, 1021
threading_cleanup() (*no módulo test.support.threading_helper*), 1945
threading_setup() (*no módulo test.support.threading_helper*), 1945
ThreadingHTTPServer (*classe em http.server*), 1590
ThreadingHTTPSServer (*classe em http.server*), 1590
ThreadingMixIn (*classe em socketserver*), 1581
ThreadingMock (*classe em unittest.mock*), 1884
ThreadingTCPServer (*classe em socketserver*), 1582
ThreadingUDPServer (*classe em socketserver*), 1582
ThreadingUnixDatagramServer (*classe em socketserver*), 1582
ThreadingUnixStreamServer (*classe em socketserver*), 1582
ThreadPool (*classe em multiprocessing.pool*), 1075
ThreadPoolExecutor (*classe em concurrent.futures*), 1093
threads
POSIX, 1133
threads livres, **2351**
threadsafety (*no módulo sqlite3*), 571
THURSDAY (*no módulo calendar*), 272
ticket_lifetime_hint (*atributo ssl.SSLSession*), 1303
tigetflag() (*no módulo curses*), 989
tigetnum() (*no módulo curses*), 990
tigetstr() (*no módulo curses*), 990
TILDE (*no módulo token*), 2250
tilt() (*no módulo turtle*), 1733
tiltangle() (*no módulo turtle*), 1733
time
module, 795
time (*atributo ssl.SSLSession*), 1303
time (*atributo uuid.UUID*), 1576
time (*classe em datetime*), 246
time() (*método asyncio.loop*), 1189
time() (*método datetime.datetime*), 240
time() (*no módulo time*), 803
Time2Internaldate() (*no módulo imaplib*), 1561
time_hi_version (*atributo uuid.UUID*), 1576
time_low (*atributo uuid.UUID*), 1576
time_mid (*atributo uuid.UUID*), 1576
time_ns() (*no módulo time*), 804
timedelta (*classe em datetime*), 225
TimedRotatingFileHandler (*classe em logging.handlers*), 842
timegm() (*no módulo calendar*), 272
timeit
module, 1983
timeit opção de linha de comando
-h, 1986
--help, 1986
-n, 1986
--number, 1986
-p, 1986
--process, 1986
-r, 1986
--repeat, 1986
-s, 1986
--setup, 1986

-u, 1986
--unit, 1986
-v, 1986
--verbose, 1986
timeit() (*método timeit.Timer*), 1985
timeit() (*no módulo timeit*), 1984
timeout, 1245
timeout (*atributo socketserver.BaseServer*), 1584
timeout (*atributo ssl.SSLSession*), 1303
timeout (*atributo subprocess.TimeoutExpired*), 1106
Timeout (*classe em asyncio*), 1151
timeout() (*método curses.window*), 997
timeout() (*no módulo asyncio*), 1151
timeout_at() (*no módulo asyncio*), 1152
TIMEOUT_MAX (*no módulo _thread*), 1134
TIMEOUT_MAX (*no módulo threading*), 1025
TimeoutError, 124, 1049, 1099, 1182
TimeoutExpired, 1106
Timer (*classe em threading*), 1036
Timer (*classe em timeit*), 1984
timerfd_create() (*no módulo os*), 758
timerfd_gettime() (*no módulo os*), 760
timerfd_gettime_ns() (*no módulo os*), 760
timerfd_settime() (*no módulo os*), 759
timerfd_settime_ns() (*no módulo os*), 759
TimerHandle (*classe em asyncio*), 1205
times() (*no módulo os*), 771
TIMESTAMP (*atributo py_compile.PycInvalidationMode*), 2260
timestamp() (*método datetime.datetime*), 242
timetuple() (*método datetime.date*), 232
timetuple() (*método datetime.datetime*), 241
timetz() (*método datetime.datetime*), 240
timezone (*classe em datetime*), 257
timezone (*no módulo time*), 807
--timing
trace opção de linha de comando, 1990
tipagem pato, **2350**
tipo, **2362**
Booleano, 9
função embutida, 111
objeto, 32
operações em dicionário, 98
operações em lista, 51
união, 108
tipo genérico, **2352**
tipos
embutido, 39
imutável sequência, 51
módulo, 111
mutável sequência, 51
operações em inteiro, 42
operações em mapeamento, 98
operações em numérico, 41
operações em sequência, 49, 51
title() (*método bytearray*), 84
title() (*método bytes*), 84
title() (*método str*), 66
title() (*no módulo turtle*), 1745
Tk, 1661
tk (*atributo tkinter.Tk*), 1663
Tk (*classe em tkinter*), 1663
Tkinter, 1661
tkinter
module, 1661
tkinter.colorchooser
module, 1674
tkinter.commondialog
module, 1679
tkinter.dnd
module, 1682
tkinter.filedialog
module, 1676
tkinter.font
module, 1675
tkinter.messagebox
module, 1679
tkinter.scrolledtext
module, 1681
tkinter.simpledialog
module, 1676
tkinter.ttk
module, 1682
TLS, 1270
--tls-cert
http.server opção de linha de comando, 1596
--tls-key
http.server opção de linha de comando, 1596
--tls-password-file
http.server opção de linha de comando, 1596
TLSv1 (*atributo ssl.TLSVersion*), 1282
TLSv1_1 (*atributo ssl.TLSVersion*), 1282
TLSv1_2 (*atributo ssl.TLSVersion*), 1282
TLSv1_3 (*atributo ssl.TLSVersion*), 1283
TLSVersion (*classe em ssl*), 1282
tm_gmtoff (*atributo time.struct_time*), 803
tm_hour (*atributo time.struct_time*), 803
tm_isdst (*atributo time.struct_time*), 803
tm_mday (*atributo time.struct_time*), 803
tm_min (*atributo time.struct_time*), 803
tm_mon (*atributo time.struct_time*), 803
tm_sec (*atributo time.struct_time*), 803
tm_wday (*atributo time.struct_time*), 803
tm_yday (*atributo time.struct_time*), 803
tm_year (*atributo time.struct_time*), 803
tm_zone (*atributo time.struct_time*), 803
TMP, 517
TMPDIR, 517
TO_BOOL (*opcode*), 2273
to_bytes() (*método int*), 43
to_eng_string() (*método decimal.Context*), 393
to_eng_string() (*método decimal.Decimal*), 385
to_integral() (*método decimal.Decimal*), 385

to_integral_exact() (*método decimal.Context*), 393
to_integral_exact() (*método decimal.Decimal*), 385
to_integral_value() (*método decimal.Decimal*), 386
to_sci_string() (*método decimal.Context*), 393
to_thread() (*no módulo asyncio*), 1154
ToASCII() (*no módulo encodings.idna*), 221
tobuf() (*método tarfile.TarInfo*), 649
tobytes() (*método array.array*), 311
tobytes() (*método memoryview*), 90
today() (*método de classe datetime.date*), 230
today() (*método de classe datetime.datetime*), 235
tofile() (*método array.array*), 311
tok_name (*no módulo token*), 2247
token, **2362**
    module, 2247
token (*atributo shlex.shlex*), 2310
Token (*classe em contextvars*), 1129
token_bytes() (*no módulo secrets*), 707
token_hex() (*no módulo secrets*), 707
token_urlsafe() (*no módulo secrets*), 707
TokenError, 2254
tokenize
    module, 2253
tokenize opção de linha de comando
    -e, 2255
    --exact, 2255
    -h, 2254
    --help, 2254
tokenize() (*no módulo tokenize*), 2253
tolist() (*método array.array*), 311
tolist() (*método memoryview*), 91
TOMLDecodeError, 687
tomllib
    module, 686
toordinal() (*método datetime.date*), 232
toordinal() (*método datetime.datetime*), 242
top() (*método curses.panel.Panel*), 1015
top() (*método poplib.POP3*), 1559
top_panel() (*no módulo curses.panel*), 1014
--top-level-directory
    unittest-discover opção de linha de comando, 1841
TopologicalSorter (*classe em graphlib*), 352
toprettyxml() (*método xml.dom.minidom.Node*), 1470
toreadonly() (*método memoryview*), 91
tostring() (*no módulo xml.etree.ElementTree*), 1449
tostringlist() (*no módulo xml.etree.ElementTree*), 1449
total() (*método collections.Counter*), 280
total_changes (*atributo sqlite3.Connection*), 582
total_nframe (*atributo tracemalloc.Traceback*), 2001
total_ordering() (*no módulo functools*), 455
total_seconds() (*método datetime.timedelta*), 228
touch() (*método pathlib.Path*), 492
touchline() (*método curses.window*), 997
touchwin() (*método curses.window*), 997
tounicode() (*método array.array*), 311
ToUnicode() (*no módulo encodings.idna*), 222
towards() (*no módulo turtle*), 1725
toxml() (*método xml.dom.minidom.Node*), 1470
tparm() (*no módulo curses*), 990
trace
    module, 1988
--trace
    trace opção de linha de comando, 1989
Trace (*classe em trace*), 1990
Trace (*classe em tracemalloc*), 2000
trace opção de linha de comando
    -C, 1989
    -c, 1989
    --count, 1989
    --coverdir, 1989
    -f, 1989
    --file, 1989
    -g, 1990
    --help, 1989
    --ignore-dir, 1990
    --ignore-module, 1990
    -l, 1989
    --listfuncs, 1989
    -m, 1989
    --missing, 1989
    --no-report, 1990
    -R, 1990
    -r, 1989
    --report, 1989
    -s, 1990
    --summary, 1990
    -T, 1989
    -t, 1989
    --timing, 1990
    --trace, 1989
    --trackcalls, 1989
    --version, 1989
trace() (*no módulo inspect*), 2138
trace_dispatch() (*método bdb.Bdb*), 1957
traceback
    module, 2107
    objeto, 2026, 2107
traceback (*atributo tracemalloc.Statistic*), 2000
traceback (*atributo tracemalloc.StatisticDiff*), 2000
traceback (*atributo tracemalloc.Trace*), 2000
Traceback (*classe em inspect*), 2137
Traceback (*classe em tracemalloc*), 2000
traceback_limit (*atributo tracemalloc.Snapshot*), 1999
traceback_limit (*atributo wsgiref.handlers.BaseHandler*), 1507
TracebackException (*classe em traceback*), 2110
tracebacklimit (*no módulo sys*), 2047
TracebackType (*classe em types*), 323
tracemalloc

module, 1991
tracer() (*no módulo turtle*), 1739
traces (*atributo tracemalloc.Snapshot*), 1999
--trackcalls
trace opção de linha de comando, 1989
train_dict() (*no módulo compression.zstd*), 604
transfercmd() (*método ftplib.FTP*), 1553
transient_internet() (*no módulo test.support.socket_helper*), 1943
translate() (*método bytearray*), 77
translate() (*método bytes*), 77
translate() (*método str*), 66
translate() (*no módulo fnmatch*), 523
translate() (*no módulo glob*), 521
translation() (*no módulo gettext*), 1645
transport (*atributo asyncio.StreamWriter*), 1165
Transport (*classe em asyncio*), 1214
Transport Layer Security, 1270
traps (*atributo decimal.Context*), 388
tratador de erros e codificação do sistema de arquivos, **2351**
trava global do interpretador, **2353**
Traversable (*classe em importlib.abc*), 2176
Traversable (*classe em importlib.resources.abc*), 2192
TraversableResources (*classe em importlib.abc*), 2177
TraversableResources (*classe em importlib.resources.abc*), 2193
TreeBuilder (*classe em xml.etree.ElementTree*), 1455
Treeview (*classe em tkinter.ttk*), 1693
triangular() (*no módulo random*), 411
tries (*atributo doctest.DocTestRunner*), 1833
True, 39, 48
true, 39
True (*variável interna*), 37
truediv() (*no módulo operator*), 465
trunc() (*no módulo math*), 42, 363
truncate() (*método io.IOBase*), 785
truncate() (*no módulo os*), 753
truth() (*no módulo operator*), 464
try
instrução, 115
Try (*classe em ast*), 2223
TryStar (*classe em ast*), 2224
TSTRING_END (*no módulo token*), 2248
TSTRING_MIDDLE (*no módulo token*), 2248
TSTRING_START (*no módulo token*), 2248
ttk, 1682
tty
controle de E/S, 2315
module, 2317
ttyname() (*no módulo os*), 733
TUESDAY (*no módulo calendar*), 272
tupla
objeto, 51, 53
tupla nomeada, **2356**
Tuple (*classe em ast*), 2211
tuple (*classe interna*), 53
Tuple (*no módulo typing*), 1804
turtle
module, 1712
Turtle (*classe em turtle*), 1745
turtledemo
module, 1749
turtles() (*no módulo turtle*), 1744
TurtleScreen (*classe em turtle*), 1745
turtlesize() (*no módulo turtle*), 1733
--type
calendar opção de linha de comando, 275
type (*atributo ctypes.CField*), 902
type (*atributo optparse.Option*), 966
type (*atributo socket.socket*), 1266
type (*atributo tarfile.TarInfo*), 649
type (*atributo urllib.request.Request*), 1516
Type (*classe em typing*), 1804
type (*classe interna*), 32
TYPE_ALIAS (*atributo symtable.SymbolTableType*), 2243
type_check_only() (*no módulo typing*), 1800
TYPE_CHECKER (*atributo optparse.Option*), 977
TYPE_CHECKING (*no módulo typing*), 1803
type_comment (*atributo ast.arg*), 2234
type_comment (*atributo ast.Assign*), 2218
type_comment (*atributo ast.For*), 2222
type_comment (*atributo ast.FunctionDef*), 2234
type_comment (*atributo ast.With*), 2225
TYPE_COMMENT (*no módulo token*), 2249
TYPE_IGNORE (*no módulo token*), 2249
TYPE_PARAMETERS (*atributo symtable.SymbolTableType*), 2243
type_repr() (*no módulo annotationlib*), 2148
TYPE_VARIABLE (*atributo symtable.SymbolTableType*), 2243
typeahead() (*no módulo curses*), 990
TypeAlias (*classe em ast*), 2220
TypeAlias (*no módulo typing*), 1768
TypeAliasType (*classe em typing*), 1785
typecode (*atributo array.array*), 309
typecodes (*no módulo array*), 309
TYPED_ACTIONS (*atributo optparse.Option*), 978
typed_subpart_iterator() (*no módulo email.iterators*), 1390
TypedDict (*classe em typing*), 1790
TypeError, 122
TypeGuard (*no módulo typing*), 1776
TypeIgnore (*classe em ast*), 2231
TypeIs (*no módulo typing*), 1775
types
module, 319
TYPES (*atributo optparse.Option*), 977
types_map (*atributo mimetypes.MimeTypes*), 1423
types_map (*no módulo mimetypes*), 1422
types_map_inv (*atributo mimetypes.MimeTypes*), 1423
TypeVar (*classe em ast*), 2232

TypeVar (*classe em typing*), 1778
TypeVarTuple (*classe em ast*), 2233
TypeVarTuple (*classe em typing*), 1781
typing
    module, 1753
TZ, 804, 805
tzinfo (*atributo datetime.datetime*), 239
tzinfo (*atributo datetime.time*), 247
tzinfo (*classe em datetime*), 250
tzname (*no módulo time*), 807
tzname() (*método datetime.datetime*), 241
tzname() (*método datetime.time*), 250
tzname() (*método datetime.timezone*), 258
tzname() (*método datetime.tzinfo*), 252
TZPATH (*no módulo zoneinfo*), 267
tzset() (*no módulo time*), 804

U

-u
    timeit opção de linha de comando, 1986
    uuid opção de linha de comando, 1579
U (*no módulo re*), 156
UAdd (*classe em ast*), 2213
ucd_3_2_0 (*no módulo unicodedata*), 189
udata (*atributo select.kevent*), 1312
UDPServer (*classe em socketserver*), 1581
UF_APPEND (*no módulo stat*), 510
UF_COMPRESSED (*no módulo stat*), 510
UF_DATAVAULT (*no módulo stat*), 510
UF_HIDDEN (*no módulo stat*), 510
UF_IMMUTABLE (*no módulo stat*), 510
UF_NODUMP (*no módulo stat*), 510
UF_NOUNLINK (*no módulo stat*), 510
UF_OPAQUE (*no módulo stat*), 510
UF_SETTABLE (*no módulo stat*), 510
UF_TRACKED (*no módulo stat*), 510
uid (*atributo tarfile.TarInfo*), 650
UID (*classe em plistlib*), 690
uid() (*método imaplib.IMAP4*), 1567
uidl() (*método poplib.POP3*), 1559
ulp() (*no módulo math*), 365
umask() (*no módulo os*), 720
unalias (*pdb command*), 1973
uname (*atributo tarfile.TarInfo*), 650
uname() (*no módulo os*), 720
uname() (*no módulo platform*), 854
UNARY_INVERT (*opcode*), 2273
UNARY_NEGATIVE (*opcode*), 2273
UNARY_NOT (*opcode*), 2273
UnaryOp (*classe em ast*), 2213
UnboundLocalError, 122
uncancel() (*método asyncio.Task*), 1160
UNCHECKED_HASH (*atributo py_compile.PycInvalidationMode*), 2260
unconsumed_tail (*atributo zlib.Decompress*), 613
unctrl() (*no módulo curses*), 990
unctrl() (*no módulo curses.ascii*), 1014
Underflow (*classe em decimal*), 395
undisplay (*pdb command*), 1973
undo() (*no módulo turtle*), 1724
undobufferentries() (*no módulo turtle*), 1737
undoc_header (*atributo cmd.Cmd*), 1017
unescape() (*no módulo html*), 1433
unescape() (*no módulo xml.sax.saxutils*), 1481
UnexpectedException, 1836
unexpectedSuccesses (*atributo unittest.TestResult*), 1862
unfreeze() (*no módulo gc*), 2122
unget_wch() (*no módulo curses*), 990
ungetch() (*no módulo curses*), 990
ungetch() (*no módulo msvcrt*), 2292
ungetmouse() (*no módulo curses*), 990
ungetwch() (*no módulo msvcrt*), 2292
unhexlify() (*no módulo binascii*), 1430
União
    objeto, 108
união
    tipo, 108
Unicode, 187, 204
    database, 187
UNICODE (*no módulo re*), 156
unicode_literals (*no módulo __future__*), 2118
unicodedata
    module, 187
UnicodeDecodeError, 122
UnicodeEncodeError, 122
UnicodeError, 122
UnicodeTranslateError, 122
UnicodeWarning, 125
unidata_version (*no módulo unicodedata*), 189
unified_diff() (*no módulo difflib*), 174
uniform() (*no módulo random*), 411
UnimplementedFileMode, 1544
Union (*classe em ctypes*), 899
Union (*classe em typing*), 1769
union() (*método frozenset*), 96
UnionType (*classe em types*), 323
UNIQUE (*atributo enum.EnumCheck*), 347
unique() (*no módulo enum*), 351
--unit
    timeit opção de linha de comando, 1986
unittest
    module, 1837
unittest opção de linha de comando
    -b, 1840
    --buffer, 1840
    -c, 1840
    --catch, 1840
    --durations, 1840
    -f, 1840
    --failfast, 1840
    -k, 1840
    --locals, 1840
unittest-discover opção de linha de comando
    -p, 1841

--pattern, 1841
-s, 1841
--start-directory, 1841
-t, 1841
--top-level-directory, 1841
-v, 1841
--verbose, 1841
unittest.mock
module, 1869
UNIX
controle de arquivo, 2319
controle de E/S, 2319
unix_dialect (*classe em csv*), 662
unix_shell (*no módulo test.support*), 1933
UnixDatagramServer (*classe em socketserver*), 1581
UnixStreamServer (*classe em socketserver*), 1581
unknown (*atributo uuid.SafeUUID*), 1575
unknown_decl() (*método html.parser.HTMLParser*), 1436
unknown_open() (*método urllib.request.BaseHandler*), 1519
unknown_open() (*método urllib.request.UnknownHandler*), 1523
UnknownHandler (*classe em urllib.request*), 1516
UnknownProtocol, 1544
UnknownTransferEncoding, 1544
unlink() (*método multiprocessing.shared_memory.SharedMemory*), 1086
unlink() (*método pathlib.Path*), 495
unlink() (*método xml.dom.minidom.Node*), 1470
unlink() (*no módulo os*), 753
unlink() (*no módulo test.support.os_helper*), 1947
unload() (*no módulo test.support.import_helper*), 1948
unlock() (*método mailbox.Babyl*), 1410
unlock() (*método mailbox.Mailbox*), 1404
unlock() (*método mailbox.Maildir*), 1407
unlock() (*método mailbox.mbox*), 1408
unlock() (*método mailbox.MH*), 1409
unlock() (*método mailbox.MMDF*), 1411
unlockpt() (*no módulo os*), 733
UNNAMED_SECTION (*no módulo configparser*), 684
UnnamedSectionDisabledError, 686
Unpack (*no módulo typing*), 1777
unpack() (*método struct.Struct*), 204
unpack() (*no módulo struct*), 198
unpack_archive() (*no módulo shutil*), 533
UNPACK_EX (*opcode*), 2277
unpack_from() (*método struct.Struct*), 204
unpack_from() (*no módulo struct*), 198
UNPACK_SEQUENCE (*opcode*), 2277
unparse() (*no módulo ast*), 2238
unparsedEntityDecl() (*método xml.sax.handler.DTDHandler*), 1480
UnparsedEntityDeclHandler() (*método xml.parsers.expat.xmlparser*), 1491
Unpickler (*classe em pickle*), 542
UnpicklingError, 540
unquote() (*no módulo email.utils*), 1388
unquote() (*no módulo urllib.parse*), 1536
unquote_plus() (*no módulo urllib.parse*), 1536
unquote_to_bytes() (*no módulo urllib.parse*), 1536
unraisablehook() (*no módulo sys*), 2047
unregister() (*método select.devpoll*), 1308
unregister() (*método select.epoll*), 1309
unregister() (*método selectors.BaseSelector*), 1314
unregister() (*método select.poll*), 1310
unregister() (*no módulo atexit*), 2106
unregister() (*no módulo codecs*), 206
unregister() (*no módulo faulthandler*), 1963
unregister_archive_format() (*no módulo shutil*), 533
unregister_dialect() (*no módulo csv*), 660
unregister_unpack_format() (*no módulo shutil*), 534
unsafe (*atributo uuid.SafeUUID*), 1575
unselect() (*método imaplib.IMAP4*), 1567
unset() (*método test.support.os_helper.EnvironmentVarGuard*), 1946
unsetenv() (*no módulo os*), 720
unshare() (*no módulo os*), 721
UnstructuredHeader (*classe em email.headerregistry*), 1357
unsubscribe() (*método imaplib.IMAP4*), 1567
UnsupportedOperation, 473, 783
until (*pdb command*), 1971
untokenize() (*no módulo tokenize*), 2254
untouchwin() (*método curses.window*), 997
unused_data (*atributo bz2.BZ2Decompressor*), 621
unused_data (*atributo compression.zstd.ZstdDecompressor*), 603
unused_data (*atributo lzma.LZMADecompressor*), 626
unused_data (*atributo zlib.Decompress*), 613
unverifiable (*atributo urllib.request.Request*), 1517
unwrap() (*método ssl.SSLSocket*), 1286
unwrap() (*no módulo inspect*), 2136
unwrap() (*no módulo urllib.parse*), 1533
up (*pdb command*), 1969
up() (*no módulo turtle*), 1727
update() (*método collections.Counter*), 280
update() (*método dict*), 100
update() (*método frozenset*), 97
update() (*método hashlib.hash*), 695
update() (*método hmac.HMAC*), 705
update() (*método http.cookies.Morsel*), 1599
update() (*método mailbox.Mailbox*), 1404
update() (*método mailbox.Maildir*), 1407
update() (*método trace.CoverageResults*), 1990
update() (*no módulo turtle*), 1740
update_abstractmethods() (*no módulo abc*), 2105
update_authenticated() (*método urllib.request.HTTPPasswordMgrWithPriorAuth*), 1521
update_lines_cols() (*no módulo curses*), 990
update_panels() (*no módulo curses.panel*), 1014
update_visible() (*método mailbox.BabylMessage*), 1417

update_wrapper() (*no módulo functools*), 462
--upgrade
    venv opção de linha de comando, 2006
upgrade_dependencies() (*método venv.EnvBuilder*), 2010
--upgrade-deps
    venv opção de linha de comando, 2006
upper() (*método bytearray*), 84
upper() (*método bytes*), 84
upper() (*método str*), 67
urandom() (*no módulo os*), 780
URL, 1528, 1538, 1589
    parsing, 1528
    relativa, 1528
url (*atributo http.client.HTTPResponse*), 1548
url (*atributo urllib.error.HTTPError*), 1538
url (*atributo urllib.response.addinfourl*), 1528
url (*atributo xmlrpc.client.ProtocolError*), 1614
url2pathname() (*no módulo urllib.request*), 1512
urlcleanup() (*no módulo urllib.request*), 1527
urldefrag() (*no módulo urllib.parse*), 1533
urlencode() (*no módulo urllib.parse*), 1536
URLError, 1537
urljoin() (*no módulo urllib.parse*), 1532
urllib
    module, 1510
urllib.error
    module, 1537
urllib.parse
    module, 1528
urllib.request
    module, 1511
    módulo, 1543
urllib.response
    module, 1527
urllib.robotparser
    module, 1538
urlopen() (*no módulo urllib.request*), 1511
urlparse() (*no módulo urllib.parse*), 1528
urlretrieve() (*no módulo urllib.request*), 1526
urlsafe_b64decode() (*no módulo base64*), 1426
urlsafe_b64encode() (*no módulo base64*), 1426
urlsplit() (*no módulo urllib.parse*), 1531
urlunparse() (*no módulo urllib.parse*), 1531
urlunsplit() (*no módulo urllib.parse*), 1532
urn (*atributo uuid.UUID*), 1576
US (*no módulo curses.ascii*), 1012
use_default_colors() (*no módulo curses*), 991
use_env() (*no módulo curses*), 990
use_rawinput (*atributo cmd.Cmd*), 1017
use_tool_id() (*no módulo sys.monitoring*), 2050
UseForeignDTD() (*método xml.parsers.expat.xmlparser*), 1488
USER, 980
--user
    ensurepip opção de linha de comando, 2004
user() (*método poplib.POP3*), 1558
USER_BASE (*no módulo site*), 2153
user_call() (*método bdb.Bdb*), 1958
user_exception() (*método bdb.Bdb*), 1958
user_line() (*método bdb.Bdb*), 1958
user_return() (*método bdb.Bdb*), 1958
USER_SITE (*no módulo site*), 2153
--user-base
    site opção de linha de comando, 2154
usercustomize
    module, 2153
UserDict (*classe em collections*), 293
UserList (*classe em collections*), 293
USERNAME, 501, 716, 980
username (*atributo email.headerregistry.Address*), 1361
USERPROFILE, 501
userptr() (*método curses.panel.Panel*), 1015
--user-site
    site opção de linha de comando, 2154
UserString (*classe em collections*), 293
UserWarning, 124
USTAR_FORMAT (*no módulo tarfile*), 644
usuário
    definição de id, 720
    id, 717
    id efetivo, 716
USub (*classe em ast*), 2213
UTC, 795
utc (*atributo datetime.timezone*), 258
UTC (*no módulo datetime*), 224
utcfromtimestamp() (*método de classe datetime.datetime*), 236
utcnow() (*método de classe datetime.datetime*), 236
utcoffset() (*método datetime.datetime*), 241
utcoffset() (*método datetime.time*), 249
utcoffset() (*método datetime.timezone*), 257
utcoffset() (*método datetime.tzinfo*), 251
utctimetuple() (*método datetime.datetime*), 242
utf8 (*atributo email.policy.EmailPolicy*), 1352
utf8() (*método poplib.POP3*), 1559
utf8_enabled (*atributo imaplib.IMAP4*), 1567
utf8_mode (*atributo sys.flags*), 2028
utime() (*no módulo os*), 753
uu
    module, 2341
uuid
    module, 1574
--uuid
    uuid opção de linha de comando, 1579
UUID (*classe em uuid*), 1575
uuid opção de linha de comando
    -C, 1579
    --count, 1579
    -h, 1578
    --help, 1578
    -N, 1579
    -n, 1579
    --name, 1579
    --namespace, 1579

-u, 1579
--uuid, 1579
uuid1() (*no módulo uuid*), 1577
uuid3() (*no módulo uuid*), 1577
uuid4() (*no módulo uuid*), 1577
uuid5() (*no módulo uuid*), 1577
uuid6() (*no módulo uuid*), 1577
uuid7() (*no módulo uuid*), 1577
uuid8() (*no módulo uuid*), 1577

V

-v
    doctest opção de linha de comando, 1817
    python--m-sqlite3-[-h]-[-v]-[filename]-[sql] opção de linha de comando, 589
    tarfile opção de linha de comando, 655
    timeit opção de linha de comando, 1986
    unittest-discover opção de linha de comando, 1841
v4_int_to_packed() (*no módulo ipaddress*), 1636
v6_int_to_packed() (*no módulo ipaddress*), 1636
valid_signals() (*no módulo signal*), 1320
validator() (*no módulo wsgiref.validate*), 1504
valores
    Booleano, 48
value
    verdade, 39
VALUE (*atributo annotationlib.Format*), 2144
value (*atributo ctypes._SimpleCData*), 896
value (*atributo enum.Enum*), 340
value (*atributo http.cookiejar.Cookie*), 1607
value (*atributo http.cookies.Morsel*), 1598
value (*atributo StopIteration*), 120
value (*atributo string.templatelib.Interpolation*), 145
value (*atributo xml.dom.Attr*), 1465
Value() (*método multiprocessing.managers.SyncManager*), 1064
Value() (*no módulo multiprocessing*), 1059
Value() (*no módulo multiprocessing.sharedctypes*), 1060
value_decode() (*método http.cookies.BaseCookie*), 1597
value_encode() (*método http.cookies.BaseCookie*), 1597
VALUE_WITH_FAKE_GLOBALS (*atributo annotationlib.Format*), 2144
ValueError, 122
valuerefs() (*método weakref.WeakValueDictionary*), 314
values (*atributo string.templatelib.Template*), 143
Values (*classe em optparse*), 965
values() (*método contextvars.Context*), 1131
values() (*método dict*), 101
values() (*método email.message.EmailMessage*), 1336
values() (*método email.message.Message*), 1374
values() (*método mailbox.Mailbox*), 1403
values() (*método types.MappingProxyType*), 324
ValuesView (*classe em collections.abc*), 298
ValuesView (*classe em typing*), 1807
var (*atributo contextvars.Token*), 1130
variance (*atributo statistics.NormalDist*), 429
variance() (*no módulo statistics*), 425
variant (*atributo uuid.UUID*), 1576
variáveis de ambiente
    definição, 718
    exclusão, 721
variável de ambiente
    BROWSER, 1497, 1498
    COLUMNS, 991
    COMSPEC, 771, 1109
    DISPLAY, 1663
    HOME, 501, 1663
    HOMEDRIVE, 501
    HOMEPATH, 501
    IDLESTARTUP, 1708, 1709
    LANG, 1643, 1644, 1651, 1655
    LANGUAGE, 1643, 1644
    LC_ALL, 1643, 1644
    LC_MESSAGES, 1643, 1644
    LINES, 985, 991
    LOGNAME, 716, 980
    MANPAGER, 1810
    no_proxy, 1515
    PAGER, 1810
    PATH, 497, 530, 762, 763, 768, 769, 779, 1108, 1497, 2007, 2008, 2152
    PATHEXT, 530
    PYTHON_CONTEXT_AWARE_WARNINGS, 2028, 2075
    PYTHON_CPU_COUNT, 778, 1053
    PYTHON_DOM, 1459
    PYTHON_GIL, 2028, 2353
    PYTHON_JIT, 2037
    PYTHON_THREAD_INHERIT_CONTEXT, 2028
    PYTHONASYNCIODEBUG, 1202, 1238, 1811
    PYTHONBREAKPOINT, 10, 2024
    PYTHONCASEOK, 35
    PYTHONCOERCECLOCALE, 713
    PYTHONDEVMODE, 1811
    PYTHONDONTWRITEBYTECODE, 2025
    PYTHONFAULTHANDLER, 1811, 1961
    PYTHONHOME, 1943, 2201, 2202
    PYTHONINTMAXSTRDIGITS, 114, 2036
    PYTHONIOENCODING, 713, 2046
    PYTHONLEGACYWINDOWSFSENCODING, 2045, 2046
    PYTHONLEGACYWINDOWSSTDIO, 2046
    PYTHONMALLOC, 1811
    PYTHONNOUSERSITE, 2153
    PYTHONPATH, 1943, 2039, 2201
    PYTHONPLATLIBDIR, 2201
    PYTHONPYCACHEPREFIX, 2025
    PYTHONSAFEPATH, 2040, 2343
    PYTHONSTARTUP, 194, 1138, 1708, 1709, 2037, 2153

PYTHONTRACEMALLOC, 1991, 1997
PYTHONTZPATH, 264, 267
PYTHONUNBUFFERED, 2046
PYTHONUSERBASE, 2153, 2154
PYTHONUSERSITE, 1943
PYTHONUTF8, 713, 2046
PYTHONWARNDEFAULTENCODING, 782
PYTHONWARNINGS, 1811, 2068, 2069
SOURCE_DATE_EPOCH, 2260, 2262
SSLKEYLOGFILE, 1272, 1273
SystemRoot, 1112
TEMP, 517
TERM, 989
TMP, 517
TMPDIR, 517
TZ, 804, 805
USER, 980
USERNAME, 501, 716, 980
USERPROFILE, 501
variável de classe, **2348**
variável de clausura, **2348**
variável de contexto, **2349**
variável livre, **2351**
vars()
built-in function, 33
vbar (*atributo tkinter.scrolledtext.ScrolledText*), 1681
VBAR (*no módulo token*), 2250
VBAREQUAL (*no módulo token*), 2251
VC_ASSEMBLY_PUBLICKEYTOKEN (*no módulo msvcrt*), 2293
Vec2D (*classe em turtle*), 1746
venv
module, 2005
venv opção de linha de comando
--clear, 2006
--copies, 2006
ENV_DIR, 2006
--prompt, 2006
--symlinks, 2006
--system-site-packages, 2006
--upgrade, 2006
--upgrade-deps, 2006
--without-pip, 2006
--without-scm-ignore-files, 2006
--verbose
doctest opção de linha de comando, 1817
tarfile opção de linha de comando, 655
timeit opção de linha de comando, 1986
unittest-discover opção de linha de comando, 1841
verbose (*atributo sys.flags*), 2028
VERBOSE (*no módulo re*), 156
verbose (*no módulo tabnanny*), 2257
verbose (*no módulo test.support*), 1933
verdade
value, 39
verificador de tipo estático, **2361**
verify() (*método smtplib.SMTP*), 1571
verify() (*no módulo enum*), 351
VERIFY_ALLOW_PROXY_CERTS (*no módulo ssl*), 1277
verify_client_post_handshake() (*método ssl.SSLSocket*), 1286
verify_code (*atributo ssl.SSLCertVerificationError*), 1274
VERIFY_CRL_CHECK_CHAIN (*no módulo ssl*), 1277
VERIFY_CRL_CHECK_LEAF (*no módulo ssl*), 1277
VERIFY_DEFAULT (*no módulo ssl*), 1276
verify_flags (*atributo ssl.SSLContext*), 1294
verify_generated_headers (*atributo email.policy.Policy*), 1350
verify_message (*atributo ssl.SSLCertVerificationError*), 1274
verify_mode (*atributo ssl.SSLContext*), 1294
verify_request() (*método socketserver.BaseServer*), 1585
VERIFY_X509_PARTIAL_CHAIN (*no módulo ssl*), 1277
VERIFY_X509_STRICT (*no módulo ssl*), 1277
VERIFY_X509_TRUSTED_FIRST (*no módulo ssl*), 1277
VerifyFlags (*classe em ssl*), 1277
VerifyMode (*classe em ssl*), 1276
--version
python--m-sqlite3-[-h]-[-v]-[filename]-[sql] opção de linha de comando, 589
trace opção de linha de comando, 1989
version (*atributo email.headerregistry.MIMEVersionHeader*), 1359
version (*atributo http.client.HTTPResponse*), 1548
version (*atributo http.cookiejar.Cookie*), 1607
version (*atributo http.cookies.Morsel*), 1598
version (*atributo ipaddress.IPv4Address*), 1624
version (*atributo ipaddress.IPv4Network*), 1629
version (*atributo ipaddress.IPv6Address*), 1627
version (*atributo ipaddress.IPv6Network*), 1632
version (*atributo sys.thread_info*), 2047
version (*atributo uuid.UUID*), 1576
version (*no módulo curses*), 997
version (*no módulo marshal*), 559
version (*no módulo sys*), 2048
version() (*método ssl.SSLSocket*), 1286
version() (*no módulo ensurepip*), 2004
version() (*no módulo importlib.metadata*), 2196
version() (*no módulo platform*), 854
version_info (*no módulo sys*), 2048
version_string() (*método http.server.BaseHTTPRequestHandler*), 1593
vetores, 308
vformat() (*método string.Formatter*), 130
virtual
Ambientes, 2005
visão de dicionário, **2350**
visit() (*método ast.NodeVisitor*), 2240
visit_Constant() (*método ast.NodeVisitor*), 2240
vline() (*método curses.window*), 997
voidcmd() (*método ftplib.FTP*), 1553

volume (*atributo zipfile.ZipInfo*), 639
vonmisesvariate() (*no módulo random*), 412
VT (*no módulo curses.ascii*), 1011

W

-w
    calendar opção de linha de comando, 275
W_OK (*no módulo os*), 736
wait() (*método asyncio.Barrier*), 1174
wait() (*método asyncio.Condition*), 1172
wait() (*método asyncio.Event*), 1170
wait() (*método asyncio.subprocess.Process*), 1177
wait() (*método multiprocessing.pool.AsyncResult*), 1070
wait() (*método subprocess.Popen*), 1113
wait() (*método threading.Barrier*), 1037
wait() (*método threading.Condition*), 1033
wait() (*método threading.Event*), 1036
wait() (*no módulo asyncio*), 1153
wait() (*no módulo concurrent.futures*), 1098
wait() (*no módulo multiprocessing.connection*), 1072
wait() (*no módulo os*), 771
wait3() (*no módulo os*), 773
wait4() (*no módulo os*), 773
wait_closed() (*método asyncio.Server*), 1206
wait_closed() (*método asyncio.StreamWriter*), 1166
wait_for() (*método asyncio.Condition*), 1172
wait_for() (*método threading.Condition*), 1033
wait_for() (*no módulo asyncio*), 1152
wait_process() (*no módulo test.support*), 1938
wait_threads_exit() (*no módulo test.support.threading_helper*), 1945
wait_until_any_call_with() (*método unittest.mock.ThreadingMock*), 1885
wait_until_called() (*método unittest.mock.ThreadingMock*), 1885
waitid() (*no módulo os*), 772
waitpid() (*no módulo os*), 772
waitstatus_to_exitcode() (*no módulo os*), 774
walk() (*método email.message.EmailMessage*), 1339
walk() (*método email.message.Message*), 1378
walk() (*método pathlib.Path*), 491
walk() (*no módulo ast*), 2239
walk() (*no módulo os*), 754
walk_packages() (*no módulo pkgutil*), 2162
walk_stack() (*no módulo traceback*), 2110
walk_tb() (*no módulo traceback*), 2110
want (*atributo doctest.Example*), 1830
warn() (*no módulo warnings*), 2072
warn_default_encoding (*atributo sys.flags*), 2028
warn_explicit() (*no módulo warnings*), 2072
Warning, 124, 587
WARNING (*no módulo logging*), 814
WARNING (*no módulo tkinter.messagebox*), 1681
warning() (*método logging.Logger*), 812
warning() (*método xml.sax.handler.ErrorHandler*), 1481
warning() (*no módulo logging*), 822
warnings, 2066
    module, 2066
WarningsRecorder (*classe em test.support.warnings_helper*), 1950
warnoptions (*no módulo sys*), 2048
wasSuccessful() (*método unittest.TestResult*), 1863
WatchedFileHandler (*classe em logging.handlers*), 840
wave
    module, 1639
Wave_read (*classe em wave*), 1640
Wave_write (*classe em wave*), 1641
WCONTINUED (*no módulo os*), 773
WCOREDUMP() (*no módulo os*), 775
WeakKeyDictionary (*classe em weakref*), 313
WeakMethod (*classe em weakref*), 314
weakref
    module, 311
WeakSet (*classe em weakref*), 314
WeakValueDictionary (*classe em weakref*), 314
webbrowser
    module, 1497
webbrowser opção de linha de comando
    -n, 1498
    --new-tab, 1498
    --new-window, 1498
    -t, 1498
WEDNESDAY (*no módulo calendar*), 272
weekday (*atributo calendar.IllegalWeekdayError*), 273
weekday() (*método datetime.date*), 232
weekday() (*método datetime.datetime*), 242
weekday() (*no módulo calendar*), 271
weekheader() (*no módulo calendar*), 271
weibullvariate() (*no módulo random*), 412
WEXITED (*no módulo os*), 774
WEXITSTATUS() (*no módulo os*), 775
wfile (*atributo http.server.BaseHTTPRequestHandler*), 1591
wfile (*atributo socketserver.DatagramRequestHandler*), 1585
whatis (*pdb command*), 1972
when() (*método asyncio.Timeout*), 1151
when() (*método asyncio.TimerHandle*), 1205
whence (*atributo concurrent.interpreters.Interpreter*), 1102
where (*pdb command*), 1969
which() (*no módulo shutil*), 530
whichdb() (*no módulo dbm*), 560
while
    instrução, 39
While (*classe em ast*), 2222
whitespace (*atributo shlex.shlex*), 2310
whitespace (*no módulo string*), 130
whitespace_split (*atributo shlex.shlex*), 2310
Widget (*classe em tkinter.ttk*), 1686
--width

calendar opção de linha de comando, 275
width (*atributo sys.hash_info*), 2035
width (*atributo textwrap.TextWrapper*), 185
width() (*no módulo turtle*), 1727
WIFCONTINUED() (*no módulo os*), 775
WIFEXITED() (*no módulo os*), 775
WIFSIGNALED() (*no módulo os*), 775
WIFSTOPPED() (*no módulo os*), 775
win32_code_page_search_function() (*no módulo encodings*), 221
win32_edition() (*no módulo platform*), 855
win32_is_iot() (*no módulo platform*), 855
win32_ver() (*no módulo platform*), 855
WinDLL (*classe em ctypes*), 886
window (*classe em curses*), 991
window() (*método curses.panel.Panel*), 1015
window_height() (*no módulo turtle*), 1744
window_log (*atributo compression.zstd.CompressionParameter*), 606
window_log_max (*atributo compression.zstd.DecompressionParameter*), 609
window_width() (*no módulo turtle*), 1744
WindowsError, 123
WindowsPath (*classe em pathlib*), 483
WindowsProactorEventLoopPolicy (*classe em asyncio*), 1228
WindowsRegistryFinder (*classe em importlib.machinery*), 2178
WindowsSelectorEventLoopPolicy (*classe em asyncio*), 1228
winerror (*atributo OSError*), 119
WinError() (*no módulo ctypes*), 895
WINFUNCTYPE() (*no módulo ctypes*), 889
winreg
    module, 2294
WinSock, 1307
winsound
    module, 2303
winver (*no módulo sys*), 2048
With (*classe em ast*), 2225
WITH_EXCEPT_START (*opcode*), 2276
with_hostmask (*atributo ipaddress.IPv4Interface*), 1635
with_hostmask (*atributo ipaddress.IPv4Network*), 1630
with_hostmask (*atributo ipaddress.IPv6Interface*), 1635
with_hostmask (*atributo ipaddress.IPv6Network*), 1633
with_name() (*método pathlib.PurePath*), 482
with_netmask (*atributo ipaddress.IPv4Interface*), 1635
with_netmask (*atributo ipaddress.IPv4Network*), 1630
with_netmask (*atributo ipaddress.IPv6Interface*), 1635
with_netmask (*atributo ipaddress.IPv6Network*), 1633
with_prefixlen (*atributo ipaddress.IPv4Interface*), 1635
with_prefixlen (*atributo ipaddress.IPv4Network*), 1630
with_prefixlen (*atributo ipaddress.IPv6Interface*), 1635
with_prefixlen (*atributo ipaddress.IPv6Network*), 1633
with_pymalloc() (*no módulo test.support*), 1936
with_segments() (*método pathlib.PurePath*), 482
with_statement (*no módulo __future__*), 2118
with_stem() (*método pathlib.PurePath*), 482
with_suffix() (*método pathlib.PurePath*), 482
with_traceback() (*método BaseException*), 116
withitem (*classe em ast*), 2225
--without-pip
    venv opção de linha de comando, 2006
--without-scm-ignore-files
    venv opção de linha de comando, 2006
WNOHANG (*no módulo os*), 774
WNOWAIT (*no módulo os*), 774
wordchars (*atributo shlex.shlex*), 2310
World Wide Web, 1497, 1528, 1538
wrap() (*método textwrap.TextWrapper*), 187
wrap() (*no módulo textwrap*), 183
wrap_bio() (*método ssl.SSLContext*), 1292
wrap_future() (*no módulo asyncio*), 1210
wrap_socket() (*método ssl.SSLContext*), 1292
wrapper() (*no módulo curses*), 991
WrapperDescriptorType (*no módulo types*), 322
wraps() (*no módulo functools*), 462
WRITABLE (*atributo inspect.BufferFlags*), 2142
WRITABLE (*no módulo _tkinter*), 1674
writable() (*método bz2.BZ2File*), 620
writable() (*método io.IOBase*), 786
WRITE (*atributo inspect.BufferFlags*), 2142
write() (*método asyncio.StreamWriter*), 1164
write() (*método asyncio.WriteTransport*), 1216
write() (*método codecs.StreamWriter*), 212
write() (*método code.InteractiveInterpreter*), 2157
write() (*método configparser.ConfigParser*), 683
write() (*método email.generator.BytesGenerator*), 1346
write() (*método email.generator.Generator*), 1347
write() (*método io.BufferedIOBase*), 788
write() (*método io.BufferedWriter*), 790
write() (*método io.RawIOBase*), 786
write() (*método io.TextIOBase*), 791
write() (*método io.Writer*), 794
write() (*método mmap.mmap*), 1330
write() (*método sqlite3.Blob*), 586
write() (*método ssl.MemoryBIO*), 1303
write() (*método ssl.SSLSocket*), 1284
write() (*método xml.etree.ElementTree.ElementTree*), 1454
write() (*método zipfile.ZipFile*), 634
write() (*no módulo os*), 733
write() (*no módulo turtle*), 1731
write_byte() (*método mmap.mmap*), 1330
write_bytes() (*método pathlib.Path*), 489

write_docstringdict() (*no módulo turtle*), 1748
write_eof() (*método asyncio.StreamWriter*), 1165
write_eof() (*método asyncio.WriteTransport*), 1217
write_eof() (*método ssl.MemoryBIO*), 1303
write_history_file() (*no módulo readline*), 192
write_results() (*método trace.CoverageResults*), 1991
write_text() (*método pathlib.Path*), 489
write_through (*atributo io.TextIOWrapper*), 792
writeframes() (*método wave.Wave_write*), 1641
writeframesraw() (*método wave.Wave_write*), 1641
writeheader() (*método csv.DictWriter*), 665
writelines() (*método asyncio.StreamWriter*), 1165
writelines() (*método asyncio.WriteTransport*), 1216
writelines() (*método codecs.StreamWriter*), 212
writelines() (*método io.IOBase*), 786
writepy() (*método zipfile.PyZipFile*), 637
Writer (*classe em io*), 794
writer() (*no módulo csv*), 660
writerow() (*método csv.csvwriter*), 665
writerows() (*método csv.csvwriter*), 665
writestr() (*método zipfile.ZipFile*), 635
WriteTransport (*classe em asyncio*), 1214
writev() (*no módulo os*), 733
writexml() (*método xml.dom.minidom.Node*), 1470
WrongDocumentErr, 1467
wsgi_file_wrapper (*atributo wsgiref.handlers.BaseHandler*), 1508
wsgi_multiprocess (*atributo wsgiref.handlers.BaseHandler*), 1507
wsgi_multithread (*atributo wsgiref.handlers.BaseHandler*), 1507
wsgi_run_once (*atributo wsgiref.handlers.BaseHandler*), 1507
WSGIApplication (*no módulo wsgiref.types*), 1509
WSGIEnvironment (*no módulo wsgiref.types*), 1509
wsgiref
    module, 1500
wsgiref.handlers
    module, 1505
wsgiref.headers
    module, 1502
wsgiref.simple_server
    module, 1503
wsgiref.types
    module, 1509
wsgiref.util
    module, 1500
wsgiref.validate
    module, 1504
WSGIRequestHandler (*classe em wsgiref.simple_server*), 1504
WSGIServer (*classe em wsgiref.simple_server*), 1504
wShowWindow (*atributo subprocess.STARTUPINFO*), 1116
WSTOPPED (*no módulo os*), 774
WSTOPSIG() (*no módulo os*), 775
wstring_at() (*no módulo ctypes*), 895
WTERMSIG() (*no módulo os*), 776
WUNTRACED (*no módulo os*), 774
WWW, 1497, 1528, 1538
    server, 1589

X

-x
    compileall opção de linha de comando, 2262
X (*no módulo re*), 156
X509 certificate, 1296
X_OK (*no módulo os*), 736
xatom() (*método imaplib.IMAP4*), 1567
XATTR_CREATE (*no módulo os*), 761
XATTR_REPLACE (*no módulo os*), 761
XATTR_SIZE_MAX (*no módulo os*), 761
xcor() (*no módulo turtle*), 1725
xdrlib
    module, 2341
XHTML, 1434
XHTML_NAMESPACE (*no módulo xml.dom*), 1460
xml
    module, 1439
XML() (*no módulo xml.etree.ElementTree*), 1449
XML_ERROR_ABORTED (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_AMPLIFICATION_LIMIT_BREACH (*no módulo xml.parsers.expat.errors*), 1496
XML_ERROR_ASYNC_ENTITY (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_ATTRIBUTE_EXTERNAL_ENTITY_REF (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_BAD_CHAR_REF (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_BINARY_ENTITY_REF (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_CANT_CHANGE_FEATURE_ONCE_PARSING (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_DUPLICATE_ATTRIBUTE (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_ENTITY_DECLARED_IN_PE (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_EXTERNAL_ENTITY_HANDLING (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_FEATURE_REQUIRES_XML_DTD (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_FINISHED (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_INCOMPLETE_PE (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_INCORRECT_ENCODING (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_INVALID_ARGUMENT (*no módulo xml.parsers.expat.errors*), 1496
XML_ERROR_INVALID_TOKEN (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_JUNK_AFTER_DOC_ELEMENT (*no módulo xml.parsers.expat.errors*), 1494

XML_ERROR_MISPLACED_XML_PI (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_NO_BUFFER (*no módulo xml.parsers.expat.errors*), 1496
XML_ERROR_NO_ELEMENTS (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_NO_MEMORY (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_NOT_STANDALONE (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_NOT_STARTED (*no módulo xml.parsers.expat.errors*), 1496
XML_ERROR_NOT_SUSPENDED (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_PARAM_ENTITY_REF (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_PARTIAL_CHAR (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_PUBLICID (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_RECURSIVE_ENTITY_REF (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_RESERVED_NAMESPACE_URI (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_RESERVED_PREFIX_XML (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_RESERVED_PREFIX_XMLNS (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_SUSPEND_PE (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_SUSPENDED (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_SYNTAX (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_TAG_MISMATCH (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_TEXT_DECL (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_UNBOUND_PREFIX (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_UNCLOSED_CDATA_SECTION (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_UNCLOSED_TOKEN (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_UNDECLARING_PREFIX (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_UNDEFINED_ENTITY (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_UNEXPECTED_STATE (*no módulo xml.parsers.expat.errors*), 1495
XML_ERROR_UNKNOWN_ENCODING (*no módulo xml.parsers.expat.errors*), 1494
XML_ERROR_XML_DECL (*no módulo xml.parsers.expat.errors*), 1495
XML_NAMESPACE (*no módulo xml.dom*), 1460
xmlcharrefreplace
    error handler's name, 208
xmlcharrefreplace_errors() (*no módulo codecs*), 209
XmlDeclHandler() (*método xml.parsers.expat.xmlparser*), 1490
xml.dom
    module, 1458
xml.dom.minidom
    module, 1468
xml.dom.pulldom
    module, 1472
xml.etree.ElementInclude
    module, 1451
xml.etree.ElementTree
    module, 1440
XMLFilterBase (*classe em xml.sax.saxutils*), 1482
XMLGenerator (*classe em xml.sax.saxutils*), 1482
XMLID() (*no módulo xml.etree.ElementTree*), 1449
XMLNS_NAMESPACE (*no módulo xml.dom*), 1460
XMLParser (*classe em xml.etree.ElementTree*), 1456
xml.parsers.expat
    module, 1486
xml.parsers.expat.errors
    module, 1493
xml.parsers.expat.model
    module, 1493
XMLParserType (*no módulo xml.parsers.expat*), 1487
XMLPullParser (*classe em xml.etree.ElementTree*), 1457
XMLReader (*classe em xml.sax.xmlreader*), 1482
xmlrpc
    module, 1609
xmlrpc.client
    module, 1609
xmlrpc.server
    module, 1617
xml.sax
    module, 1474
xml.sax.handler
    module, 1476
xml.sax.saxutils
    module, 1481
xml.sax.xmlreader
    module, 1482
xor() (*no módulo operator*), 465
xview() (*método tkinter.ttk.Treeview*), 1696

Y

ycor() (*no módulo turtle*), 1726
year
    calendar opção de linha de comando, 275
year (*atributo datetime.date*), 231
year (*atributo datetime.datetime*), 238
yeardatescalendar() (*método calendar.Calendar*), 268
yeardays2calendar() (*método calendar.Calendar*), 268
yeardayscalendar() (*método calendar.Calendar*), 268

YES (*no módulo tkinter.messagebox*), 1680
YESEXPR (*no módulo locale*), 1654
YESNO (*no módulo tkinter.messagebox*), 1681
YESNOCANCEL (*no módulo tkinter.messagebox*), 1681
Yield (*classe em ast*), 2235
YIELD_VALUE (*opcode*), 2275
YieldFrom (*classe em ast*), 2235
yiq_to_rgb() (*no módulo colorsys*), 1642
yview() (*método tkinter.ttk.Treeview*), 1696

Z

z
    na formatação de strings, 133
z85decode() (*no módulo base64*), 1428
z85encode() (*no módulo base64*), 1428
Zen do Python, **2363**
ZeroDivisionError, 123
zfill() (*método bytearray*), 85
zfill() (*método bytes*), 85
zfill() (*método str*), 67
zip()
    built-in function, 33
ZIP_BZIP2 (*no módulo zipfile*), 630
ZIP_DEFLATED (*no módulo zipfile*), 630
zip_longest() (*no módulo itertools*), 446
ZIP_LZMA (*no módulo zipfile*), 630
ZIP_STORED (*no módulo zipfile*), 630
ZIP_ZSTANDARD (*no módulo zipfile*), 630
zipapp
    module, 2015
zipapp opção de linha de comando
    -c, 2016
    --compress, 2016
    -h, 2016
    --help, 2016
    --info, 2016
    -m, 2016
    --main, 2016
    -o, 2015
    --output, 2015
    -p, 2015
    --python, 2015
zipfile
    module, 629
ZipFile (*classe em zipfile*), 631
zipfile opção de linha de comando
    -c, 640
    --create, 640
    -e, 640
    --extract, 640
    -l, 640
    --list, 640
    --metadata-encoding, 640
    -t, 640
    --test, 640
zipimport
    module, 2159
zipimporter (*classe em zipimport*), 2160
ZipImportError, 2160
ZipInfo (*classe em zipfile*), 629
zlib
    module, 611
ZLIB_RUNTIME_VERSION (*no módulo zlib*), 614
ZLIB_VERSION (*no módulo zlib*), 614
ZLIBNG_VERSION (*no módulo zlib*), 614
zoneinfo
    module, 262
ZoneInfo (*classe em zoneinfo*), 264
ZoneInfoNotFoundError, 267
zscore() (*método statistics.NormalDist*), 430
zstd_version_info (*no módulo compression.zstd*), 610
ZstdCompressor (*classe em compression.zstd*), 601
ZstdDecompressor (*classe em compression.zstd*), 603
ZstdDict (*classe em compression.zstd*), 604
ZstdError, 600
ZstdFile (*classe em compression.zstd*), 600